Reverse Engineering Windows x64

Исключения, стек, PE, .pdata/.xdata и unwind-модель Windows x64

Финальная no-loss редакция v89

Эта сборка исправляет ошибку v88: первые шесть глав больше не заменяются сокращенной версией. В v89 сохранен полный материал v87 и добавлен новый связный лекционный слой перед ним.

Главный принцип редакции: ничего из старых глав 1-6 не удалено.

Как устроена эта версия

Версия v88 была слишком сильным сжатием: вместо глубокой переработки она заменила первые шесть глав компактной лекцией. В этой версии такой потери нет.

Сначала идет новый связный маршрут по главам 1-6. Его задача - дать читателю цельную картину: исключение в Windows x64, PE image, .pdata, .xdata, UNWIND_INFO, RtlVirtualUnwind и переход к анализатору.

После этого идет полный подробный материал глав 1-6 из v87. Он сохранен как глубокий разбор терминов, структур, полей, типичных ошибок и практических связей.

Дальше без сокращения сохранены главы 7-12 и приложения из v87: ручная модель анализа, архитектура EH/Unwind Analyzer, reports, dynamic validation, 27 static findings, глоссарий и вопросы для защиты.

Слой	Что находится внутри	Зачем он нужен
I-A	Связная лекционная версия глав 1-6	Собирает механизм в голове до погружения в подробности
I-B	Полный фундамент v87 без сокращений	Сохраняет глубину, термины, детали и разборы
II	Проектные главы 7-12	Связывает теорию с EH/Unwind Analyzer и 27 findings

Оглавление no-loss редакции

Часть I-A. Связная лекционная версия глав 1-6: один маршрут от исключения до unwind-анализатора.

Часть I-B. Полный подробный фундамент глав 1-6 из v87: исключения, стек, PE, PE32+, RVA/VA/file offset, Exception Directory, .pdata, RUNTIME_FUNCTION, .xdata, UNWIND_INFO, UWOP, RtlLookupFunctionEntry, RtlVirtualUnwind, handler-ы, search/unwind phase и переход к статическому анализу.

Часть II. Главы 7-12: ручная модель анализа, архитектура анализатора, reports, Ghidra/JSON/HTML output, dynamic validation, validation/corpus/fuzzing, 27 findings и evidence families.

Приложения. Глоссарий, схемы, вопросы для защиты и терминологическая карта финальной редакции.

В PDF также добавлены bookmarks. Для такой большой книги удобнее пользоваться боковой панелью навигации в PDF-читалке, а не только печатным оглавлением.

Почему сохранены оба слоя

Связная лекционная версия и полный подробный разбор решают разные задачи. Лекционный слой отвечает за маршрут понимания: он помогает не потеряться, когда читатель впервые входит в тему. Полный разбор отвечает за глубину: он оставляет все подробности, которые нужны для реальной защиты, ручного анализа и разработки инструмента.

Если оставить только полный разбор, книга местами кажется энциклопедией терминов. Если оставить только лекционный слой, теряется глубина. Поэтому no-loss редакция использует два уровня: сначала читатель получает общую механику, затем погружается в полный материал без потери информации.

Главы 1-6 теперь не нужно читать как набор отдельных ответов. Их следует читать как подробную расшифровку маршрута, заданного в части I-A. Каждый термин из старых глав получает место в общей цепочке: событие исключения - машинный контекст - PE image - function table - unwind recipe - восстановление caller context - evidence для анализа.

Часть I-A

Связная лекционная версия глав 1-6

Этот блок добавлен не вместо полного материала, а перед ним. Его задача - снять ощущение FAQ и дать читателю цельную дорогу: от момента исключения до структур .pdata/.xdata и работы static analyzer.

После этой части сразу идет полный подробный фундамент из v87 без сокращений.

Часть I. Переписанный фундамент: Windows x64 exception/unwind как единая история

Эта часть заменяет прежние первые шесть глав. Цель новой редакции - не перечислить термины, а провести читателя через один связный маршрут: от машинной инструкции, вызвавшей исключение, к PE-файлу, .pdata, .xdata, RUNTIME_FUNCTION, UNWIND_INFO, RtlVirtualUnwind, handler-ам и границам static/dynamic анализа.

Главный стиль этой части - лекция, а не FAQ. Термины по-прежнему разжевываются, но появляются не как отдельные карточки, а как инструменты, которые нужны Windows в конкретный момент маршрута. Мы будем постоянно задавать один вопрос: что система знает сейчас, что ей нужно узнать дальше и где лежат данные, позволяющие сделать следующий шаг?

Глава 1. Исключение как живая сцена выполнения, а не слово "ошибка"

Начнем с самой живой точки: поток выполняет инструкцию. Пока мы не почувствуем исключение как событие в конкретном thread-е, все дальнейшие структуры будут казаться мертвыми таблицами. Поэтому первая глава не начинает с PE и не начинает с .pdata. Она начинает с момента, когда процессор больше не может продолжить обычную дорогу выполнения.

1.1. Исключение как момент, когда обычная дорога выполнения обрывается

Представим не абстрактную программу, а живой поток выполнения. В процессоре сейчас находится инструкция. Регистр RIP указывает на ее адрес, RSP указывает на вершину стека, регистры содержат рабочие значения, а память процесса разбита на страницы с правами чтения, записи и выполнения. Пока каждая инструкция завершается штатно, выполнение движется обычной дорогой: следующая инструкция, условный переход, call, ret, снова следующая инструкция. Это normal control flow - обычный поток управления.

Теперь в этой дороге появляется разрыв. Инструкция пытается прочитать память по адресу, который не сопоставлен, или пытается выполнить страницу без execute-права, или делит на ноль, или попадает на breakpoint. Процессор не может просто придумать результат и идти дальше. Он не имеет права silently продолжить выполнение, потому что тогда состояние программы станет ложным. Поэтому возникает exception - исключение, то есть формализованное событие, которое говорит: обычный путь инструкций остановился, дальше решение принимает механизм обработки.

Важно не путать исключение с ошибкой. Ошибка - это смысловая оценка: программист сделал не то, указатель испорчен, память не выделена, индекс вышел за массив. Исключение - это системная форма события: код, адрес, параметры, context record, доставка, handler-ы, возможная раскрутка стека. Одно и то же исключение может быть багом, отладочным breakpoint-ом, частью runtime-а языка, частью защиты или намеренным RaiseException. Поэтому для reverse engineering исключение - не финал истории, а вход в механизм.

С этой точки зрения любой термин первой главы получает место в одной сцене. RIP нужен не потому, что это красивый регистр, а потому что без него нельзя связать событие с машинной инструкцией. RSP нужен не потому, что стек часто упоминают в учебниках, а потому что без него нельзя понять текущий frame. ExceptionRecord нужен не как структура ради структуры, а как способ сделать событие передаваемым между слоями Windows. ContextRecord нужен не как dump регистров, а как материал, из которого unwind построит следующий context.

Когда реверсер смотрит на падение, он должен перестать думать "программа умерла". Он должен думать "Windows получила событие и теперь проходит маршрут". Хороший анализ начинается с вопроса не "какой это баг", а "какой слой маршрута я сейчас наблюдаю". Если ты видишь только код исключения, у тебя один слой. Если добавляешь RIP, RSP и registers, появляется машинная сцена. Если добавляешь .pdata/.xdata, появляется возможность проверить, как Windows будет двигаться по кадрам.

Представь, что ты смотришь на crash dump. В плохой подаче тебе говорят: "смотри код исключения и стек". В хорошей подаче сначала заставляют восстановить сцену: какой thread selected, какой RIP является exception address, что лежит в RSP, какие значения в RCX/RDX/R8/R9, не указывает ли проблемный адрес в freed memory, guard page, null page или non-executable page. Только после этого можно обсуждать, почему handler был найден или не найден.

В этой сцене термин control flow тоже перестает быть абстрактным. Обычный control flow виден в инструкциях: call, jmp, ret, conditional branch. Exception flow часто не виден прямой стрелкой в дизассемблере, потому что переход к обработчику описан metadata и runtime rules. Поэтому начинающий реверсер, который смотрит только CFG, может не увидеть весь возможный маршрут

управления.

Типичная ошибка первой главы - искать "где catch" слишком рано. До catch нужно понять, как Windows вообще дошла до кадра, в котором есть language-specific handler. Если текущий frame нельзя раскрутить, более внешний catch может существовать в исходнике, но система не сможет надежно прийти к нему. Поэтому обработчик зависит не только от кода handler-а, но и от корректности пути к нему.

Еще одна ошибка - думать, что exception handling интересен только при падениях. На практике этот механизм важен и для debugger-а, и для C++ runtime, и для stack walking, и для crash dump analysis, и для антиотладочных/обфускационных техник. То есть exception/unwind metadata - это не редкая аварийная дорожка, а постоянная инфраструктура, которую используют разные участники Windows runtime.

Для проекта EH/Unwind Analyzer первая глава задает границу: анализатор не ищет "ошибку программы" в бытовом смысле. Он проверяет инфраструктуру, через которую Windows должна объяснить себе стек и обработчики. Если инфраструктура повреждена или необычна, это meaningful evidence даже до запуска файла.

1.2. Почему Windows нужен не "текст программы", а машинный контекст

Когда программист смотрит на C++-код, ему кажется, что система должна сразу увидеть try/catch. Но Windows в момент исключения не читает исходник. У нее есть ContextRecord - запись машинного контекста. В этой записи важны RIP, RSP, регистры общего назначения, флаги и другое состояние процессора. Именно этот снимок отвечает на вопрос: где поток был в машинном мире, когда возникло событие.

Рядом находится ExceptionRecord - запись о самом исключении. Она отвечает не на вопрос "где стек", а на вопрос "что произошло": код исключения, адрес события, флаги, дополнительные параметры. Для access violation параметры могут сказать, было ли это чтение, запись или выполнение, и к какому адресу пытались обратиться. Без ExceptionRecord мы не знаем тип события. Без ContextRecord мы не знаем, как это событие связано с реальным кодом и стеком.

Тут появляется первая взрослая связка: исключение - это не одно число вроде 0xC0000005. Это пара "событие плюс состояние". Событие говорит, что случилось. Состояние говорит, где именно программа находилась и с какими значениями. Вся дальнейшая Windows x64 exception/unwind model будет строиться вокруг этой пары.

С этой точки зрения любой термин первой главы получает место в одной сцене. RIP нужен не потому, что это красивый регистр, а потому что без него нельзя связать событие с машинной инструкцией. RSP нужен не потому, что стек часто упоминают в учебниках, а потому что без него нельзя понять текущий frame. ExceptionRecord нужен не как структура ради структуры, а как способ сделать событие передаваемым между слоями Windows. ContextRecord нужен не как dump регистров, а как материал, из которого unwind построит следующий context.

Когда реверсер смотрит на падение, он должен перестать думать "программа умерла". Он должен думать "Windows получила событие и теперь проходит маршрут". Хороший анализ начинается с вопроса не "какой это баг", а "какой слой маршрута я сейчас наблюдаю". Если ты видишь только код исключения, у тебя один слой. Если добавляешь RIP, RSP и registers, появляется машинная сцена. Если добавляешь .pdata/.xdata, появляется возможность проверить, как Windows будет двигаться по кадрам.

Представь, что ты смотришь на crash dump. В плохой подаче тебе говорят: "смотри код исключения и стек". В хорошей подаче сначала заставляют восстановить сцену: какой thread selected, какой RIP является exception address, что лежит в RSP, какие значения в RCX/RDX/R8/R9, не указывает ли проблемный адрес в freed memory, guard page, null page или non-executable page. Только после этого можно обсуждать, почему handler был найден или не найден.

В этой сцене термин control flow тоже перестает быть абстрактным. Обычный control flow виден в инструкциях: call, jmp, ret, conditional branch. Exception flow часто не виден прямой стрелкой в дизассемблере, потому что переход к обработчику описан metadata и runtime rules. Поэтому начинающий реверсер, который смотрит только CFG, может не увидеть весь возможный маршрут управления.

Типичная ошибка первой главы - искать "где catch" слишком рано. До catch нужно понять, как Windows вообще дошла до кадра, в котором есть language-specific handler. Если текущий frame нельзя раскрутить, более внешний catch может существовать в исходнике, но система не сможет надежно прийти к нему. Поэтому обработчик зависит не только от кода handler-а, но и от корректности пути к нему.

Еще одна ошибка - думать, что exception handling интересен только при падениях. На практике этот механизм важен и для debugger-а, и для C++ runtime, и для stack walking, и для crash dump analysis, и для антиотладочных/обфускационных техник. То есть exception/unwind metadata - это не редкая аварийная дорожка, а постоянная инфраструктура, которую используют разные участники Windows runtime.

Для проекта EH/Unwind Analyzer первая глава задает границу: анализатор не ищет "ошибку программы" в бытовом смысле. Он проверяет инфраструктуру, через которую Windows должна объяснить себе стек и обработчики. Если инфраструктура повреждена или необычна, это meaningful evidence даже до запуска файла.

1.3. Поток, процесс и стек: почему исключение происходит не "в программе вообще"

Процесс - это контейнер: виртуальное адресное пространство, загруженные модули, heap, handles, ресурсы. Но процесс сам по себе не идет по инструкциям. По инструкциям идет thread - поток. У каждого потока есть собственный стек и собственный текущий context. Если в процессе десять потоков, то у них десять разных RIP, десять разных RSP и десять разных call stacks.

Исключение всегда привязано к конкретному потоку. Если worker thread упал на access violation, Windows будет искать frame-based обработчик в стеке именно этого worker thread. Она не будет внезапно смотреть в GUI thread, потому что у другого потока другая история вызовов. Это кажется простым, но именно здесь часто ломается анализ дампа: человек видит много стеков и забывает, что exception context принадлежит одному конкретному потоку.

Stack - это не просто место для локальных переменных. В этой теме стек - это память текущей истории вызовов потока. Там лежат return addresses, сохраненные nonvolatile-регистры, shadow space, локальные области, временные значения. Чтобы подняться от текущей функции к caller-у, Windows должна правильно интерпретировать эту память. На x64 она делает это не гаданием, а через unwind metadata.

С этой точки зрения любой термин первой главы получает место в одной сцене. RIP нужен не потому, что это красивый регистр, а потому что без него нельзя связать событие с машинной инструкцией. RSP нужен не потому, что стек часто упоминают в учебниках, а потому что без него нельзя понять текущий frame. ExceptionRecord нужен не как структура ради структуры, а как способ сделать событие передаваемым между слоями Windows. ContextRecord нужен не как dump регистров, а как материал, из которого unwind построит следующий context.

Когда реверсер смотрит на падение, он должен перестать думать "программа умерла". Он должен думать "Windows получила событие и теперь проходит маршрут". Хороший анализ начинается с вопроса не "какой это баг", а "какой слой маршрута я сейчас наблюдаю". Если ты видишь только код исключения, у тебя один слой. Если добавляешь RIP, RSP и registers, появляется машинная сцена. Если добавляешь .pdata/.xdata, появляется возможность проверить, как Windows будет двигаться по кадрам.

Представь, что ты смотришь на crash dump. В плохой подаче тебе говорят: "смотри код исключения и стек". В хорошей подаче сначала заставляют восстановить сцену: какой thread selected, какой RIP является exception address, что лежит в RSP, какие значения в RCX/RDX/R8/R9, не указывает ли проблемный адрес в freed memory, guard page, null page или non-executable page. Только после этого можно обсуждать, почему handler был найден или не найден.

В этой сцене термин control flow тоже перестает быть абстрактным. Обычный control flow виден в инструкциях: call, jmp, ret, conditional branch. Exception flow часто не виден прямой стрелкой в дизассемблере, потому что переход к обработчику описан metadata и runtime rules. Поэтому начинающий реверсер, который смотрит только CFG, может не увидеть весь возможный маршрут управления.

Типичная ошибка первой главы - искать "где catch" слишком рано. До catch нужно понять, как Windows вообще дошла до кадра, в котором есть language-specific handler. Если текущий frame нельзя раскрутить, более внешний catch может существовать в исходнике, но система не сможет надежно прийти к нему. Поэтому обработчик зависит не только от кода handler-а, но и от корректности пути к нему.

Еще одна ошибка - думать, что exception handling интересен только при падениях. На практике этот механизм важен и для debugger-а, и для C++ runtime, и для stack walking, и для crash dump analysis, и для антиотладочных/обфускационных техник. То есть exception/unwind metadata - это не редкая аварийная дорожка, а постоянная инфраструктура, которую используют разные участники Windows runtime.

Для проекта EH/Unwind Analyzer первая глава задает границу: анализатор не ищет "ошибку программы" в бытовом смысле. Он проверяет инфраструктуру, через которую Windows должна объяснить себе стек и обработчики. Если инфраструктура повреждена или необычна, это meaningful evidence даже до запуска файла.

1.4. Почему call stack нельзя надежно угадать простым просмотром памяти

Новичку кажется: раз return addresses лежат на стеке, можно просто пройти по стеку и найти числа, похожие на адреса кода. Иногда такие эвристики действительно помогают, когда metadata нет или стек сильно поврежден. Но как основная модель ОС это слишком ненадежно. В стеке лежат не только адреса возврата. Там могут быть указатели на данные, старые регистры, размеры, флаги, элементы структур, shadow space и случайные значения, которые похожи на адреса в .text.

Windows x64 выбирает более формальный путь. Если RIP находится внутри функции, система должна найти запись этой функции в function table. Запись говорит, где начинается диапазон функции, где заканчивается и где лежит unwind-рецепт. Такой рецепт описывает, что функция сделала со стеком и nonvolatile-регистрами. Поэтому call stack в нормальной x64-модели восстанавливается не как "поиск похожих адресов", а как последовательное применение правил unwind.

Это ключ к всей книге. Мы изучаем PE, .pdata, RUNTIME_FUNCTION, .xdata и UNWIND_INFO не ради формата. Мы изучаем их потому, что без них Windows не может уверенно двигаться по кадрам. А если Windows не может уверенно двигаться по кадрам, то exception handling, stack walking, crash dump analysis и часть reverse engineering превращаются в догадки.

С этой точки зрения любой термин первой главы получает место в одной сцене. RIP нужен не потому, что это красивый регистр, а потому что без него нельзя связать событие с машинной инструкцией. RSP нужен не потому, что стек часто упоминают в учебниках, а потому что без него нельзя понять текущий frame. ExceptionRecord нужен не как структура ради структуры, а как способ сделать событие передаваемым между слоями Windows. ContextRecord нужен не как dump регистров, а как материал, из которого unwind построит следующий context.

Когда реверсер смотрит на падение, он должен перестать думать "программа умерла". Он должен думать "Windows получила событие и теперь проходит маршрут". Хороший анализ начинается с вопроса не "какой это баг", а "какой слой маршрута я сейчас наблюдаю". Если ты видишь только код исключения, у тебя один слой. Если добавляешь RIP, RSP и registers, появляется машинная сцена. Если добавляешь .pdata/.xdata, появляется возможность проверить, как Windows будет двигаться по кадрам.

Представь, что ты смотришь на crash dump. В плохой подаче тебе говорят: "смотри код исключения и стек". В хорошей подаче сначала заставляют восстановить сцену: какой thread selected, какой RIP является exception address, что лежит в RSP, какие значения в RCX/RDX/R8/R9, не указывает ли проблемный адрес в freed memory, guard page, null page или non-executable page. Только после этого можно обсуждать, почему handler был найден или не найден.

В этой сцене термин control flow тоже перестает быть абстрактным. Обычный control flow виден в инструкциях: call, jmp, ret, conditional branch. Exception flow часто не виден прямой стрелкой в дизассемблере, потому что переход к обработчику описан metadata и runtime rules. Поэтому начинающий реверсер, который смотрит только CFG, может не увидеть весь возможный маршрут управления.

Типичная ошибка первой главы - искать "где catch" слишком рано. До catch нужно понять, как Windows вообще дошла до кадра, в котором есть language-specific handler. Если текущий frame нельзя раскрутить, более внешний catch может существовать в исходнике, но система не сможет надежно прийти к нему. Поэтому обработчик зависит не только от кода handler-а, но и от корректности пути к нему.

Еще одна ошибка - думать, что exception handling интересен только при падениях. На практике этот механизм важен и для debugger-а, и для C++ runtime, и для stack walking, и для crash dump analysis, и для антиотладочных/обфускационных техник. То есть exception/unwind metadata - это не редкая аварийная дорожка, а постоянная инфраструктура, которую используют разные участники Windows runtime.

Для проекта EH/Unwind Analyzer первая глава задает границу: анализатор не ищет "ошибку программы" в бытовом смысле. Он проверяет инфраструктуру, через которую Windows должна объяснить себе стек и обработчики. Если инфраструктура повреждена или необычна, это meaningful evidence даже до запуска файла.

1.5. Handler - не обязательно catch и не обязательно один уровень

Handler переводится как обработчик. Но опасно сразу представлять C++ catch. В Windows есть несколько уровней обработки исключений. Есть vectored exception handlers, которые регистрируются на уровне процесса и получают шанс увидеть исключение по отдельным правилам. Есть frame-based handlers, связанные с кадрами стека. Есть language-specific handlers, например __C_specific_handler, которые знают, как интерпретировать данные конкретного компилятора и языка. Есть top-level handling, когда нормальная цепочка не нашла обработчика.

Поэтому handler в этой книге - это не "кусок исходника после catch". Это код или runtime-процедура, которой Windows может передать ExceptionRecord, ContextRecord и данные текущего кадра, чтобы она решила: обработать, продолжить поиск, выполнить cleanup или разрешить продолжение выполнения. Разные handler-ы находятся на разных уровнях маршрута.

Эта многослойность важна для реверса. Если в UNWIND_INFO установлен флаг exception handler, это не значит, что мы прямо нашли пользовательский catch. Это значит, что у данного кадра есть объявленный handler route. Чтобы понять его смысл, нужно смотреть адрес handler-а, handler data, scope tables, compiler/runtime conventions и связь с конкретным кадром.

С этой точки зрения любой термин первой главы получает место в одной сцене. RIP нужен не потому, что это красивый регистр, а потому что без него нельзя связать событие с машинной инструкцией. RSP нужен не потому, что стек часто упоминают в учебниках, а потому что без него

нельзя понять текущий frame. ExceptionRecord нужен не как структура ради структуры, а как способ сделать событие передаваемым между слоями Windows. ContextRecord нужен не как dump регистров, а как материал, из которого unwind построит следующий context.

Когда реверсер смотрит на падение, он должен перестать думать "программа умерла". Он должен думать "Windows получила событие и теперь проходит маршрут". Хороший анализ начинается с вопроса не "какой это баг", а "какой слой маршрута я сейчас наблюдаю". Если ты видишь только код исключения, у тебя один слой. Если добавляешь RIP, RSP и registers, появляется машинная сцена. Если добавляешь .pdata/.xdata, появляется возможность проверить, как Windows будет двигаться по кадрам.

Представь, что ты смотришь на crash dump. В плохой подаче тебе говорят: "смотри код исключения и стек". В хорошей подаче сначала заставляют восстановить сцену: какой thread selected, какой RIP является exception address, что лежит в RSP, какие значения в RCX/RDX/R8/R9, не указывает ли проблемный адрес в freed memory, guard page, null page или non-executable page. Только после этого можно обсуждать, почему handler был найден или не найден.

В этой сцене термин control flow тоже перестает быть абстрактным. Обычный control flow виден в инструкциях: call, jmp, ret, conditional branch. Exception flow часто не виден прямой стрелкой в дизассемблере, потому что переход к обработчику описан metadata и runtime rules. Поэтому начинающий реверсер, который смотрит только CFG, может не увидеть весь возможный маршрут управления.

Типичная ошибка первой главы - искать "где catch" слишком рано. До catch нужно понять, как Windows вообще дошла до кадра, в котором есть language-specific handler. Если текущий frame нельзя раскрутить, более внешний catch может существовать в исходнике, но система не сможет надежно прийти к нему. Поэтому обработчик зависит не только от кода handler-а, но и от корректности пути к нему.

Еще одна ошибка - думать, что exception handling интересен только при падениях. На практике этот механизм важен и для debugger-а, и для C++ runtime, и для stack walking, и для crash dump analysis, и для антиотладочных/обфускационных техник. То есть exception/unwind metadata - это не редкая аварийная дорожка, а постоянная инфраструктура, которую используют разные участники Windows runtime.

Для проекта EH/Unwind Analyzer первая глава задает границу: анализатор не ищет "ошибку программы" в бытовом смысле. Он проверяет инфраструктуру, через которую Windows должна объяснить себе стек и обработчики. Если инфраструктура повреждена или необычна, это meaningful evidence даже до запуска файла.

1.6. Search phase и unwind phase как две разные части одной истории

Когда исключение возникло, Windows не должна сразу разрушать стек. Сначала ей нужно найти, кто вообще готов принять событие. Это search phase - фаза поиска. Система идет по кадрам сверху вниз: текущая функция, ее caller, caller caller-а и так далее. На каждом подходящем кадре она может вызвать language-specific handler и спросить: ты обрабатываешь это исключение или поиск продолжать?

Если обработчик найден, начинается другая история: unwind phase - фаза раскрутки. Теперь система не просто спрашивает. Она должна довести выполнение до найденного обработчика корректно: пройти промежуточные кадры, выполнить termination/cleanup logic, восстановить нужное состояние. Нельзя просто прыгнуть в handler, оставив стек и регистры в состоянии глубокой функции.

Разделение на search и unwind помогает не путать два действия: найти цель и добраться до нее. Для reverse engineer-а это критично: один и тот же handler mechanism может участвовать в разных фазах, но смысл вызова будет разным. В search phase он решает, подходит ли кадр. В unwind phase

он может выполнять завершение или cleanup.

С этой точки зрения любой термин первой главы получает место в одной сцене. RIP нужен не потому, что это красивый регистр, а потому что без него нельзя связать событие с машинной инструкцией. RSP нужен не потому, что стек часто упоминают в учебниках, а потому что без него нельзя понять текущий frame. ExceptionRecord нужен не как структура ради структуры, а как способ сделать событие передаваемым между слоями Windows. ContextRecord нужен не как dump регистров, а как материал, из которого unwind построит следующий context.

Когда реверсер смотрит на падение, он должен перестать думать "программа умерла". Он должен думать "Windows получила событие и теперь проходит маршрут". Хороший анализ начинается с вопроса не "какой это баг", а "какой слой маршрута я сейчас наблюдаю". Если ты видишь только код исключения, у тебя один слой. Если добавляешь RIP, RSP и registers, появляется машинная сцена. Если добавляешь .pdata/.xdata, появляется возможность проверить, как Windows будет двигаться по кадрам.

Представь, что ты смотришь на crash dump. В плохой подаче тебе говорят: "смотри код исключения и стек". В хорошей подаче сначала заставляют восстановить сцену: какой thread selected, какой RIP является exception address, что лежит в RSP, какие значения в RCX/RDX/R8/R9, не указывает ли проблемный адрес в freed memory, guard page, null page или non-executable page. Только после этого можно обсуждать, почему handler был найден или не найден.

В этой сцене термин control flow тоже перестает быть абстрактным. Обычный control flow виден в инструкциях: call, jmp, ret, conditional branch. Exception flow часто не виден прямой стрелкой в дизассемблере, потому что переход к обработчику описан metadata и runtime rules. Поэтому начинающий реверсер, который смотрит только CFG, может не увидеть весь возможный маршрут управления.

Типичная ошибка первой главы - искать "где catch" слишком рано. До catch нужно понять, как Windows вообще дошла до кадра, в котором есть language-specific handler. Если текущий frame нельзя раскрутить, более внешний catch может существовать в исходнике, но система не сможет надежно прийти к нему. Поэтому обработчик зависит не только от кода handler-а, но и от корректности пути к нему.

Еще одна ошибка - думать, что exception handling интересен только при падениях. На практике этот механизм важен и для debugger-а, и для C++ runtime, и для stack walking, и для crash dump analysis, и для антиотладочных/обфускационных техник. То есть exception/unwind metadata - это не редкая аварийная дорожка, а постоянная инфраструктура, которую используют разные участники Windows runtime.

Для проекта EH/Unwind Analyzer первая глава задает границу: анализатор не ищет "ошибку программы" в бытовом смысле. Он проверяет инфраструктуру, через которую Windows должна объяснить себе стек и обработчики. Если инфраструктура повреждена или необычна, это meaningful evidence даже до запуска файла.

1.7. Почему x64 приводит нас к PE, .pdata и .xdata

На x64 Windows использует табличную модель. Табличная модель означает: для диапазонов кода заранее есть данные, по которым система понимает, как раскрутить кадр. Эти данные обычно живут в PE image. Exception Directory ведет к массиву RUNTIME_FUNCTION, обычно в .pdata. Каждая RUNTIME_FUNCTION-запись указывает на UNWIND_INFO, обычно в .xdata.

Цепочка выглядит так:

```
Instruction causes exception
 -> ExceptionRecord + ContextRecord
 -> RIP / RSP / registers
 -> module + ImageBase
 -> RVA inside PE image
 -> Exception Directory
 -> .pdata / RUNTIME_FUNCTION
 -> .xdata / UNWIND_INFO
```

```
-> RtlVirtualUnwind
-> caller context
-> handler search continues
```

С этой точки зрения любой термин первой главы получает место в одной сцене. RIP нужен не потому, что это красивый регистр, а потому что без него нельзя связать событие с машинной инструкцией. RSP нужен не потому, что стек часто упоминают в учебниках, а потому что без него нельзя понять текущий frame. ExceptionRecord нужен не как структура ради структуры, а как способ сделать событие передаваемым между слоями Windows. ContextRecord нужен не как dump регистров, а как материал, из которого unwind построит следующий context.

Когда реверсер смотрит на падение, он должен перестать думать "программа умерла". Он должен думать "Windows получила событие и теперь проходит маршрут". Хороший анализ начинается с вопроса не "какой это баг", а "какой слой маршрута я сейчас наблюдаю". Если ты видишь только код исключения, у тебя один слой. Если добавляешь RIP, RSP и registers, появляется машинная сцена. Если добавляешь .pdata/.xdata, появляется возможность проверить, как Windows будет двигаться по кадрам.

Представь, что ты смотришь на crash dump. В плохой подаче тебе говорят: "смотри код исключения и стек". В хорошей подаче сначала заставляют восстановить сцену: какой thread selected, какой RIP является exception address, что лежит в RSP, какие значения в RCX/RDX/R8/R9, не указывает ли проблемный адрес в freed memory, guard page, null page или non-executable page. Только после этого можно обсуждать, почему handler был найден или не найден.

В этой сцене термин control flow тоже перестает быть абстрактным. Обычный control flow виден в инструкциях: call, jmp, ret, conditional branch. Exception flow часто не виден прямой стрелкой в дизассемблере, потому что переход к обработчику описан metadata и runtime rules. Поэтому начинающий реверсер, который смотрит только CFG, может не увидеть весь возможный маршрут управления.

Типичная ошибка первой главы - искать "где catch" слишком рано. До catch нужно понять, как Windows вообще дошла до кадра, в котором есть language-specific handler. Если текущий frame нельзя раскрутить, более внешний catch может существовать в исходнике, но система не сможет надежно прийти к нему. Поэтому обработчик зависит не только от кода handler-а, но и от корректности пути к нему.

Еще одна ошибка - думать, что exception handling интересен только при падениях. На практике этот механизм важен и для debugger-а, и для C++ runtime, и для stack walking, и для crash dump analysis, и для антиотладочных/обфускационных техник. То есть exception/unwind metadata - это не редкая аварийная дорожка, а постоянная инфраструктура, которую используют разные участники Windows runtime.

Для проекта EH/Unwind Analyzer первая глава задает границу: анализатор не ищет "ошибку программы" в бытовом смысле. Он проверяет инфраструктуру, через которую Windows должна объяснить себе стек и обработчики. Если инфраструктура повреждена или необычна, это meaningful evidence даже до запуска файла.

1.8. Мини-сцена: access violation внутри функции C

Пусть main вызвал A, A вызвала B, B вызвала C. Внутри C инструкция mov eax, dword ptr [rcx] попыталась прочитать память по адресу из RCX. Адрес оказался недоступен. Возник access violation. В этот момент Windows не знает "внутри C" как имени из исходника. Она знает RIP, RSP, registers и ExceptionRecord. По RIP ей нужно понять, какой PE-модуль содержит адрес, какой RVA соответствует этому адресу, какая RUNTIME_FUNCTION покрывает этот RVA и какой UNWIND_INFO описывает кадр.

Если C имеет корректный UNWIND_INFO, Windows может восстановить caller context, то есть контекст B. Если B не обработал исключение, она восстановит A. Если A не обработал, поднимется к main. Эта цепочка не является красивым списком из отладчика. Отладчик показывает результат. Механизм внутри строит этот результат через metadata и стек.

Вот почему первая глава должна закончиться не словом "ошибка", а маршрутом. Исключение - это событие. Событие привязано к потоку. Поток имеет context и stack. Stack состоит из кадров. На x64 кадры надежно раскручиваются через PE function tables и unwind metadata. Дальше мы обязаны изучить PE, потому что именно там лежит карта, без которой Windows не сможет пройти этот маршрут.

С этой точки зрения любой термин первой главы получает место в одной сцене. RIP нужен не потому, что это красивый регистр, а потому что без него нельзя связать событие с машинной инструкцией. RSP нужен не потому, что стек часто упоминают в учебниках, а потому что без него нельзя понять текущий frame. ExceptionRecord нужен не как структура ради структуры, а как способ сделать событие передаваемым между слоями Windows. ContextRecord нужен не как dump регистров, а как материал, из которого unwind построит следующий context.

Когда реверсер смотрит на падение, он должен перестать думать "программа умерла". Он должен думать "Windows получила событие и теперь проходит маршрут". Хороший анализ начинается с вопроса не "какой это баг", а "какой слой маршрута я сейчас наблюдаю". Если ты видишь только код исключения, у тебя один слой. Если добавляешь RIP, RSP и registers, появляется машинная сцена. Если добавляешь .pdata/.xdata, появляется возможность проверить, как Windows будет двигаться по кадрам.

Представь, что ты смотришь на crash dump. В плохой подаче тебе говорят: "смотри код исключения и стек". В хорошей подаче сначала заставляют восстановить сцену: какой thread selected, какой RIP является exception address, что лежит в RSP, какие значения в RCX/RDX/R8/R9, не указывает ли проблемный адрес в freed memory, guard page, null page или non-executable page. Только после этого можно обсуждать, почему handler был найден или не найден.

В этой сцене термин control flow тоже перестает быть абстрактным. Обычный control flow виден в инструкциях: call, jmp, ret, conditional branch. Exception flow часто не виден прямой стрелкой в дизассемблере, потому что переход к обработчику описан metadata и runtime rules. Поэтому начинающий реверсер, который смотрит только CFG, может не увидеть весь возможный маршрут управления.

Типичная ошибка первой главы - искать "где catch" слишком рано. До catch нужно понять, как Windows вообще дошла до кадра, в котором есть language-specific handler. Если текущий frame нельзя раскрутить, более внешний catch может существовать в исходнике, но система не сможет надежно прийти к нему. Поэтому обработчик зависит не только от кода handler-а, но и от корректности пути к нему.

Еще одна ошибка - думать, что exception handling интересен только при падениях. На практике этот механизм важен и для debugger-а, и для C++ runtime, и для stack walking, и для crash dump analysis, и для антиотладочных/обфускационных техник. То есть exception/unwind metadata - это не редкая аварийная дорожка, а постоянная инфраструктура, которую используют разные участники Windows runtime.

Для проекта EH/Unwind Analyzer первая глава задает границу: анализатор не ищет "ошибку программы" в бытовом смысле. Он проверяет инфраструктуру, через которую Windows должна объяснить себе стек и обработчики. Если инфраструктура повреждена или необычна, это meaningful evidence даже до запуска файла.

Сквозной пример главы 1: как это выглядит в живом анализе

Возьмем маленькую ситуацию. В функции process_item есть указатель item. Код пытается прочитать поле item->size. В машинном коде это может превратиться в чтение по адресу из RCX плюс некоторое смещение. Если RCX равен нулю или указывает в освобожденную область, инструкция не может завершиться нормально. Возникает access violation. Для пользователя это может выглядеть как падение приложения, но для Windows это structured event: есть exception code, exception address, parameters and context.

Первое, что делает грамотный анализ, - не ищет сразу виноватую строку C-кода. Он фиксирует сцену: instruction address, module, registers, stack pointer, attempted memory access. Если exception parameters говорят, что это чтение адреса 0x20, а RCX равен 0, то likely это null pointer plus field offset. Если attempted address выглядит как 0xFFFFFFFFFFFFFFFF, картина другая. Если это execute violation на странице данных, картина третья.

Теперь добавим стек. Текущий frame принадлежит process_item, но caller мог быть parse_table, а тот был вызван load_config. Чтобы Windows нашла обработчик выше, ей нужно подняться по этим кадрам. На x64 этот подъем зависит от unwind metadata. Если process_item имеет корректный UNWIND_INFO, caller context будет восстановлен формально. Если metadata повреждены, stack trace может оборваться именно здесь.

Поэтому уже на уровне простого null pointer crash мы видим всю будущую книгу: exception record говорит, что случилось; context record говорит, где случилось; stack показывает историю; PE/unwind metadata позволяют эту историю восстановить; handler route решает, будет ли исключение обработано. Один бытовой crash превращается в системный маршрут.

```
process_item(item)
  RIP -> mov eax, dword ptr [rcx+20h]
  RCX -> 0000000000000000
  Exception -> access violation, read address 0000000000000020
  Next question -> can Windows unwind from process_item to its caller?
```

Итог главы 1: что должно остаться в голове

Исключение - это формализованное событие выполнения в конкретном потоке. Оно связывает ExceptionRecord, ContextRecord, RIP, RSP, stack и handler route. На x64 надежный подъем по кадрам требует unwind metadata, поэтому тема исключений неизбежно приводит к PE, .pdata и .xdata.

Глава 2. PE как карта загруженного образа: почему unwind начинается с файла

Теперь, когда мы поняли, что Windows должна искать маршрут обработки по текущему адресу выполнения, появляется следующий вопрос: где система берет карту функций и unwind-рецепты? Ответ ведет нас в PE. Но PE нужно понять не как "exe-файл", а как договор между файлом на диске, загруженным image и runtime-механизмами Windows.

2.1. EXE на диске и image в памяти: две формы одной программы

Теперь мы берем файл. Не абстрактную "программу", а конкретный EXE или DLL на диске. Внутри лежат байты, но Windows не может просто начать выполнять первый байт файла. Ей нужно понять, где заголовки, где код, где данные, какие секции нужно отображать в память, какие страницы должны быть executable, какие импортируемые функции нужно связать и где находится таблица исключений.

PE file - Portable Executable - это формат такой инструкции для Windows. Слово portable не значит "переносной на флешке". В контексте PE/COFF оно означает, что формат описывает executable image files и object files для Windows и не сводится к одной конкретной архитектуре. Для нашей книги важен PE32+ image, потому что Windows x64 exception/unwind route связан именно с 64-bit image model.

Loaded image - это уже не файл как последовательность байтов на диске. Это отображенное в virtual address space представление модуля. Loader прочитал headers, разложил sections по virtual layout, применил relocations, заполнил imports, выставил page protections. CPU выполняет инструкции из loaded image, а не из "offset файла".

PE удобнее всего понимать как набор координатных обещаний. Заголовки обещают, где читать секции. Section headers обещают, как переводить file layout в virtual layout. Data directories обещают, где находятся специальные таблицы. Exception Directory обещает, где искать function table. Но обещание еще не доказательство. Static analyzer должен проверять, не выходит ли указанный диапазон за файл, не перекрываются ли секции, не противоречат ли размеры, можно ли реально добраться до байтов.

Именно здесь рождается стиль безопасного PE-парсинга. Мы не читаем файл как доверенный объект. Мы читаем его как hostile input - недоверенные bytes. Даже если файл выглядит обычным, analyzer должен быть готов к тому, что любое поле может быть ложью. В контексте EH/Unwind Analyzer это особенно важно, потому что одна неверная RVA-связь способна испортить весь путь к UNWIND_INFO.

Когда читаешь PE, полезно постоянно держать две картинки. Первая картинка - файл как лежащая на диске лента байтов: offset 0, offset 0x100, offset 0x400. Вторая картинка - loaded image: ImageBase, страницы памяти, section permissions, RVA ranges. Многие ошибки начинаются с того, что человек переносит координату из первой картинки во вторую без перевода.

Например, Exception Directory хранит VirtualAddress и Size. Название VirtualAddress провоцирует думать о полном VA, но в Data Directory это RVA. Значит, статический анализатор не может просто сделать seek(VirtualAddress). Он должен найти секцию, проверить, что RVA входит в virtual range, затем вычислить raw offset. Если секции перекрываются или размеры неадекватны, даже этот базовый перевод становится сомнительным.

PE также важен тем, что loader не обязан слепо копировать файл. В памяти появляется image, где sections выровнены по SectionAlignment, недостающие области могут быть заполнены нулями, imports могут быть заполнены runtime-адресами, relocations могут изменить абсолютные значения. Поэтому "байты в файле" и "байты в процессе" совпадают не всегда и не полностью.

Для unwind особенно важны section characteristics. Если metadata лежат в writable секции, это не автоматический приговор, но это меняет доверие: данные маршрута потенциально могут быть изменены во время выполнения. Если handler target лежит в non-executable секции, это еще сильнее влияет на интерпретацию, потому что обработчик должен быть исполняемым кодом. Так PE-права становятся частью evidence model.

В проекте это переходит в table_integrity и metadata_placement. Сначала analyzer проверяет, можно ли доверять PE mapping. Потом он проверяет, где лежит metadata и куда указывают handler-адреса. Если перепутать эти уровни, findings станут шумными: placement без надежного mapping не имеет полной силы.

2.2. Loader и linker как две стороны одной карты

Linker собирает PE до запуска. Он берет object files, code, data, import references, runtime metadata и формирует итоговый image layout. Именно linker обычно раскладывает .text, .rdata, .data, .pdata, .xdata, .reloc и другие секции. Он записывает headers, section table, data directories и многие служебные структуры.

Loader работает уже во время запуска. Он не придумывает структуру файла заново. Он читает то, что записали compiler/linker, и строит image в памяти. Если linker записал корректную Exception Directory, loader и Windows runtime смогут использовать ее для x64 exception handling. Если PE был вручную испорчен или намеренно запутан, разные инструменты могут по-разному пережить эту нестандартность.

Для реверсера это означает: PE - это договор. Compiler генерирует код и unwind metadata. Linker собирает их в image. Loader отображает image. Windows unwinder использует .pdata/.xdata. Static analyzer проверяет, насколько этот договор выглядит корректным.

PE удобнее всего понимать как набор координатных обещаний. Заголовки обещают, где читать секции. Section headers обещают, как переводить file layout в virtual layout. Data directories обещают, где находятся специальные таблицы. Exception Directory обещает, где искать function table. Но обещание еще не доказательство. Static analyzer должен проверять, не выходит ли указанный диапазон за файл, не перекрываются ли секции, не противоречат ли размеры, можно ли реально добраться до байтов.

Именно здесь рождается стиль безопасного PE-парсинга. Мы не читаем файл как доверенный объект. Мы читаем его как hostile input - недоверенные bytes. Даже если файл выглядит обычным, analyzer должен быть готов к тому, что любое поле может быть ложью. В контексте EH/Unwind Analyzer это особенно важно, потому что одна неверная RVA-связь способна испортить весь путь к UNWIND_INFO.

Когда читаешь PE, полезно постоянно держать две картинки. Первая картинка - файл как лежащая на диске лента байтов: offset 0, offset 0x100, offset 0x400. Вторая картинка - loaded image: ImageBase, страницы памяти, section permissions, RVA ranges. Многие ошибки начинаются с того, что человек переносит координату из первой картинки во вторую без перевода.

Например, Exception Directory хранит VirtualAddress и Size. Название VirtualAddress провоцирует думать о полном VA, но в Data Directory это RVA. Значит, статический анализатор не может просто сделать seek(VirtualAddress). Он должен найти секцию, проверить, что RVA входит в virtual range, затем вычислить raw offset. Если секции перекрываются или размеры неадекватны, даже этот базовый перевод становится сомнительным.

PE также важен тем, что loader не обязан слепо копировать файл. В памяти появляется image, где sections выровнены по SectionAlignment, недостающие области могут быть заполнены нулями, imports могут быть заполнены runtime-адресами, relocations могут изменить абсолютные значения. Поэтому "байты в файле" и "байты в процессе" совпадают не всегда и не полностью.

Для unwind особенно важны section characteristics. Если metadata лежат в writable секции, это не автоматический приговор, но это меняет доверие: данные маршрута потенциально могут быть

изменены во время выполнения. Если handler target лежит в non-executable секции, это еще сильнее влияет на интерпретацию, потому что обработчик должен быть исполняемым кодом. Так PE-права становятся частью evidence model.

В проекте это переходит в table_integrity и metadata_placement. Сначала analyzer проверяет, можно ли доверять PE mapping. Потом он проверяет, где лежит metadata и куда указывают handler-адреса. Если перепутать эти уровни, findings станут шумными: placement без надежного mapping не имеет полной силы.

2.3. Заголовки не являются формальностью: они включают карту чтения

Header - это служебное описание, без которого байты остаются сырой массой. PE начинается с DOS header и MZ signature, затем через e_lfanew ведет к PE signature, COFF File Header, Optional Header и Section Table. Это не просто исторический мусор. Это путь, по которому parser понимает, где начинается современная PE-структура и как дальше читать файл.

Optional Header называется optional исторически, но для executable image он практически обязателен: там ImageBase, AddressOfEntryPoint, SectionAlignment, FileAlignment, SizeOfImage и Data Directory. Data Directory особенно важен, потому что через него Windows находит Import Directory, Export Directory, Base Relocation Directory, Exception Directory и другие служебные подсистемы.

Типичная ошибка - искать .pdata только по имени секции. Формально вход к exception/unwind table идет через Exception Directory в Data Directory. Секция может называться необычно, данные могут быть объединены с другой секцией, packer может изменить имена. Имя помогает человеку, но контракт для parser-а - это headers, directories, RVA и sizes.

PE удобнее всего понимать как набор координатных обещаний. Заголовки обещают, где читать секции. Section headers обещают, как переводить file layout в virtual layout. Data directories обещают, где находятся специальные таблицы. Exception Directory обещает, где искать function table. Но обещание еще не доказательство. Static analyzer должен проверять, не выходит ли указанный диапазон за файл, не перекрываются ли секции, не противоречат ли размеры, можно ли реально добраться до байтов.

Именно здесь рождается стиль безопасного PE-парсинга. Мы не читаем файл как доверенный объект. Мы читаем его как hostile input - недоверенные bytes. Даже если файл выглядит обычным, analyzer должен быть готов к тому, что любое поле может быть ложью. В контексте EH/Unwind Analyzer это особенно важно, потому что одна неверная RVA-связь способна испортить весь путь к UNWIND_INFO.

Когда читаешь PE, полезно постоянно держать две картинки. Первая картинка - файл как лежащая на диске лента байтов: offset 0, offset 0x100, offset 0x400. Вторая картинка - loaded image: ImageBase, страницы памяти, section permissions, RVA ranges. Многие ошибки начинаются с того, что человек переносит координату из первой картинки во вторую без перевода.

Например, Exception Directory хранит VirtualAddress и Size. Название VirtualAddress провоцирует думать о полном VA, но в Data Directory это RVA. Значит, статический анализатор не может просто сделать seek(VirtualAddress). Он должен найти секцию, проверить, что RVA входит в virtual range, затем вычислить raw offset. Если секции перекрываются или размеры неадекватны, даже этот базовый перевод становится сомнительным.

PE также важен тем, что loader не обязан слепо копировать файл. В памяти появляется image, где sections выровнены по SectionAlignment, недостающие области могут быть заполнены нулями, imports могут быть заполнены runtime-адресами, relocations могут изменить абсолютные значения. Поэтому "байты в файле" и "байты в процессе" совпадают не всегда и не полностью.

Для unwind особенно важны section characteristics. Если metadata лежат в writable секции, это не автоматический приговор, но это меняет доверие: данные маршрута потенциально могут быть изменены во время выполнения. Если handler target лежит в non-executable секции, это еще сильнее

влияет на интерпретацию, потому что обработчик должен быть исполняемым кодом. Так PE-права становятся частью evidence model.

В проекте это переходит в table_integrity и metadata_placement. Сначала analyzer проверяет, можно ли доверять PE mapping. Потом он проверяет, где лежит metadata и куда указывают handler-адреса. Если перепутать эти уровни, findings станут шумными: placement без надежного mapping не имеет полной силы.

2.4. Секции как контейнеры с двумя жизнями

Section - секция - это крупный контейнер PE. У секции есть raw life на диске и virtual life в памяти. В файле она лежит по PointerToRawData и имеет SizeOfRawData. В памяти она начинается по VirtualAddress, то есть RVA начала секции, и имеет VirtualSize. Эти размеры не обязаны совпадать.

Это различие нужно прочувствовать. В файле секции выровнены по FileAlignment. В памяти - по SectionAlignment. В файле некоторые данные могут отсутствовать физически и быть заполнены нулями при загрузке. В памяти секции получают page protections: read, write, execute. Поэтому адрес, который видит CPU, почти никогда нельзя напрямую использовать как file offset.

Для unwind это не мелочь. RUNTIME_FUNCTION хранит BeginAddress, EndAddress и UnwindInfoAddress как image-relative addresses. Чтобы статически прочитать UNWIND_INFO из файла, нужно перевести RVA через section table в file offset. Если анализатор перепутает эти координаты, он будет читать не тот участок файла и построит ложные findings.

PE удобнее всего понимать как набор координатных обещаний. Заголовки обещают, где читать секции. Section headers обещают, как переводить file layout в virtual layout. Data directories обещают, где находятся специальные таблицы. Exception Directory обещает, где искать function table. Но обещание еще не доказательство. Static analyzer должен проверять, не выходит ли указанный диапазон за файл, не перекрываются ли секции, не противоречат ли размеры, можно ли реально добраться до байтов.

Именно здесь рождается стиль безопасного PE-парсинга. Мы не читаем файл как доверенный объект. Мы читаем его как hostile input - недоверенные bytes. Даже если файл выглядит обычным, analyzer должен быть готов к тому, что любое поле может быть ложью. В контексте EH/Unwind Analyzer это особенно важно, потому что одна неверная RVA-связь способна испортить весь путь к UNWIND_INFO.

Когда читаешь PE, полезно постоянно держать две картинки. Первая картинка - файл как лежащая на диске лента байтов: offset 0, offset 0x100, offset 0x400. Вторая картинка - loaded image: ImageBase, страницы памяти, section permissions, RVA ranges. Многие ошибки начинаются с того, что человек переносит координату из первой картинки во вторую без перевода.

Например, Exception Directory хранит VirtualAddress и Size. Название VirtualAddress провоцирует думать о полном VA, но в Data Directory это RVA. Значит, статический анализатор не может просто сделать seek(VirtualAddress). Он должен найти секцию, проверить, что RVA входит в virtual range, затем вычислить raw offset. Если секции перекрываются или размеры неадекватны, даже этот базовый перевод становится сомнительным.

PE также важен тем, что loader не обязан слепо копировать файл. В памяти появляется image, где sections выровнены по SectionAlignment, недостающие области могут быть заполнены нулями, imports могут быть заполнены runtime-адресами, relocations могут изменить абсолютные значения. Поэтому "байты в файле" и "байты в процессе" совпадают не всегда и не полностью.

Для unwind особенно важны section characteristics. Если metadata лежат в writable секции, это не автоматический приговор, но это меняет доверие: данные маршрута потенциально могут быть изменены во время выполнения. Если handler target лежит в non-executable секции, это еще сильнее

влияет на интерпретацию, потому что обработчик должен быть исполняемым кодом. Так PE-права становятся частью evidence model.

В проекте это переходит в table_integrity и metadata_placement. Сначала analyzer проверяет, можно ли доверять PE mapping. Потом он проверяет, где лежит metadata и куда указывают handler-адреса. Если перепутать эти уровни, findings станут шумными: placement без надежного mapping не имеет полной силы.

2.5. RVA, VA и file offset как три системы координат

File offset - координата на диске. Hex editor показывает именно ее: сколько байт от начала файла до нужного места. RVA - Relative Virtual Address - смещение от ImageBase внутри loaded image. VA - Virtual Address - реальный виртуальный адрес в адресном пространстве процесса: VA = ImageBase + RVA.

Эти три координаты отвечают на разные вопросы. File offset говорит: где байты лежат в файле? RVA говорит: где объект находится внутри образа независимо от конкретной базы загрузки? VA говорит: по какому адресу живой процесс видит память сейчас? Нельзя заменять одно другим только потому, что числа иногда похожи.

Пример:

```
VA = ImageBase + RVA

static file reading:
  RVA -> find section by VirtualAddress/VirtualSize
      -> delta = RVA - Section.VirtualAddress
      -> file_offset = Section.PointerToRawData + delta

runtime memory reading:
  RVA -> VA = ImageBase + RVA
```

PE удобнее всего понимать как набор координатных обещаний. Заголовки обещают, где читать секции. Section headers обещают, как переводить file layout в virtual layout. Data directories обещают, где находятся специальные таблицы. Exception Directory обещает, где искать function table. Но обещание еще не доказательство. Static analyzer должен проверять, не выходит ли указанный диапазон за файл, не перекрываются ли секции, не противоречат ли размеры, можно ли реально добраться до байтов.

Именно здесь рождается стиль безопасного PE-парсинга. Мы не читаем файл как доверенный объект. Мы читаем его как hostile input - недоверенные bytes. Даже если файл выглядит обычным, analyzer должен быть готов к тому, что любое поле может быть ложью. В контексте EH/Unwind Analyzer это особенно важно, потому что одна неверная RVA-связь способна испортить весь путь к UNWIND_INFO.

Когда читаешь PE, полезно постоянно держать две картинки. Первая картинка - файл как лежащая на диске лента байтов: offset 0, offset 0x100, offset 0x400. Вторая картинка - loaded image: ImageBase, страницы памяти, section permissions, RVA ranges. Многие ошибки начинаются с того, что человек переносит координату из первой картинки во вторую без перевода.

Например, Exception Directory хранит VirtualAddress и Size. Название VirtualAddress провоцирует думать о полном VA, но в Data Directory это RVA. Значит, статический анализатор не может просто сделать seek(VirtualAddress). Он должен найти секцию, проверить, что RVA входит в virtual range, затем вычислить raw offset. Если секции перекрываются или размеры неадекватны, даже этот базовый перевод становится сомнительным.

PE также важен тем, что loader не обязан слепо копировать файл. В памяти появляется image, где sections выровнены по SectionAlignment, недостающие области могут быть заполнены нулями, imports могут быть заполнены runtime-адресами, relocations могут изменить абсолютные значения. Поэтому "байты в файле" и "байты в процессе" совпадают не всегда и не полностью.

Для unwind особенно важны section characteristics. Если metadata лежат в writable секции, это не автоматический приговор, но это меняет доверие: данные маршрута потенциально могут быть изменены во время выполнения. Если handler target лежит в non-executable секции, это еще сильнее влияет на интерпретацию, потому что обработчик должен быть исполняемым кодом. Так PE-права

становятся частью evidence model.

В проекте это переходит в table_integrity и metadata_placement. Сначала analyzer проверяет, можно ли доверять PE mapping. Потом он проверяет, где лежит metadata и куда указывают handler-адреса. Если перепутать эти уровни, findings станут шумными: placement без надежного mapping не имеет полной силы.

2.6. Exception Directory как дверь к unwind-модели

Exception Directory - это запись Data Directory, через которую PE image сообщает Windows, где лежат данные exception/unwind. Для x64 это обычно массив RUNTIME_FUNCTION entries. Этот массив обычно находится в .pdata. Каждая запись связывает диапазон кода с UNWIND_INFO.

Важно: Exception Directory не хранит список всех try/catch. Она не является "таблицей catch-блоков". Это вход в function table. Уже дальше, через UNWIND_INFO и handler data, можно дойти до language-specific handler-а, scope tables или других runtime structures.

Если человек не понял Exception Directory, он будет думать, что .pdata - просто вспомогательная секция. На самом деле это часть runtime-контракта Windows x64. Когда исключение возникло, RIP нужно связать с функцией. Именно эта дверь ведет к таблице, которая позволяет это сделать.

PE удобнее всего понимать как набор координатных обещаний. Заголовки обещают, где читать секции. Section headers обещают, как переводить file layout в virtual layout. Data directories обещают, где находятся специальные таблицы. Exception Directory обещает, где искать function table. Но обещание еще не доказательство. Static analyzer должен проверять, не выходит ли указанный диапазон за файл, не перекрываются ли секции, не противоречат ли размеры, можно ли реально добраться до байтов.

Именно здесь рождается стиль безопасного PE-парсинга. Мы не читаем файл как доверенный объект. Мы читаем его как hostile input - недоверенные bytes. Даже если файл выглядит обычным, analyzer должен быть готов к тому, что любое поле может быть ложью. В контексте EH/Unwind Analyzer это особенно важно, потому что одна неверная RVA-связь способна испортить весь путь к UNWIND_INFO.

Когда читаешь PE, полезно постоянно держать две картинки. Первая картинка - файл как лежащая на диске лента байтов: offset 0, offset 0x100, offset 0x400. Вторая картинка - loaded image: ImageBase, страницы памяти, section permissions, RVA ranges. Многие ошибки начинаются с того, что человек переносит координату из первой картинки во вторую без перевода.

Например, Exception Directory хранит VirtualAddress и Size. Название VirtualAddress провоцирует думать о полном VA, но в Data Directory это RVA. Значит, статический анализатор не может просто сделать seek(VirtualAddress). Он должен найти секцию, проверить, что RVA входит в virtual range, затем вычислить raw offset. Если секции перекрываются или размеры неадекватны, даже этот базовый перевод становится сомнительным.

PE также важен тем, что loader не обязан слепо копировать файл. В памяти появляется image, где sections выровнены по SectionAlignment, недостающие области могут быть заполнены нулями, imports могут быть заполнены runtime-адресами, relocations могут изменить абсолютные значения. Поэтому "байты в файле" и "байты в процессе" совпадают не всегда и не полностью.

Для unwind особенно важны section characteristics. Если metadata лежат в writable секции, это не автоматический приговор, но это меняет доверие: данные маршрута потенциально могут быть изменены во время выполнения. Если handler target лежит в non-executable секции, это еще сильнее влияет на интерпретацию, потому что обработчик должен быть исполняемым кодом. Так PE-права становятся частью evidence model.

В проекте это переходит в table_integrity и metadata_placement. Сначала analyzer проверяет, можно ли доверять PE mapping. Потом он проверяет, где лежит metadata и куда указывают handler-адреса. Если перепутать эти уровни, findings станут шумными: placement без надежного mapping не имеет полной силы.

2.7. PE32+ и почему 64-bit PE всё равно хранит много 32-bit RVA

PE32+ - это вариант PE для 64-bit images. Название сбивает, потому что внутри есть "32+", но речь идет о 64-битном image format. Отличие определяется, в частности, Magic в Optional Header: PE32 обычно 0x10B, PE32+ обычно 0x20B. Для Windows x64 нас интересует именно PE32+.

Почему в 64-битном образе RUNTIME_FUNCTION хранит 32-bit offsets относительно ImageBase? Потому что это image-relative модель. Большинство служебных ссылок внутри image удобно хранить как RVA: образ может загрузиться по разной базе из-за ASLR, но offset внутри образа остается тем же. Actual VA получается прибавлением ImageBase.

Это хороший пример того, почему "64-bit" не означает "каждое поле стало 64-bit". 64-bit означает архитектуру, address space, registers, calling convention и PE32+ header model. Но многие tables по-прежнему используют RVA как компактную и переносимую координату внутри image.

PE удобнее всего понимать как набор координатных обещаний. Заголовки обещают, где читать секции. Section headers обещают, как переводить file layout в virtual layout. Data directories обещают, где находятся специальные таблицы. Exception Directory обещает, где искать function table. Но обещание еще не доказательство. Static analyzer должен проверять, не выходит ли указанный диапазон за файл, не перекрываются ли секции, не противоречат ли размеры, можно ли реально добраться до байтов.

Именно здесь рождается стиль безопасного PE-парсинга. Мы не читаем файл как доверенный объект. Мы читаем его как hostile input - недоверенные bytes. Даже если файл выглядит обычным, analyzer должен быть готов к тому, что любое поле может быть ложью. В контексте EH/Unwind Analyzer это особенно важно, потому что одна неверная RVA-связь способна испортить весь путь к UNWIND_INFO.

Когда читаешь PE, полезно постоянно держать две картинки. Первая картинка - файл как лежащая на диске лента байтов: offset 0, offset 0x100, offset 0x400. Вторая картинка - loaded image: ImageBase, страницы памяти, section permissions, RVA ranges. Многие ошибки начинаются с того, что человек переносит координату из первой картинки во вторую без перевода.

Например, Exception Directory хранит VirtualAddress и Size. Название VirtualAddress провоцирует думать о полном VA, но в Data Directory это RVA. Значит, статический анализатор не может просто сделать seek(VirtualAddress). Он должен найти секцию, проверить, что RVA входит в virtual range, затем вычислить raw offset. Если секции перекрываются или размеры неадекватны, даже этот базовый перевод становится сомнительным.

PE также важен тем, что loader не обязан слепо копировать файл. В памяти появляется image, где sections выровнены по SectionAlignment, недостающие области могут быть заполнены нулями, imports могут быть заполнены runtime-адресами, relocations могут изменить абсолютные значения. Поэтому "байты в файле" и "байты в процессе" совпадают не всегда и не полностью.

Для unwind особенно важны section characteristics. Если metadata лежат в writable секции, это не автоматический приговор, но это меняет доверие: данные маршрута потенциально могут быть изменены во время выполнения. Если handler target лежит в non-executable секции, это еще сильнее влияет на интерпретацию, потому что обработчик должен быть исполняемым кодом. Так PE-права становятся частью evidence model.

В проекте это переходит в table_integrity и metadata_placement. Сначала analyzer проверяет, можно ли доверять PE mapping. Потом он проверяет, где лежит metadata и куда указывают handler-адреса. Если перепутать эти уровни, findings станут шумными: placement без надежного mapping не имеет полной силы.

2.8. Мини-проход: от RIP к байтам .xdata в файле

Пусть в живом процессе RIP равен 0x00007FF700123456. Сначала мы определяем, какому модулю принадлежит этот VA. Допустим, ImageBase модуля 0x00007FF700000000. Тогда RVA = 0x123456. По этому RVA Windows ищет RUNTIME_FUNCTION в function table. Запись содержит UnwindInfoAddress, например 0x18000. Это снова RVA, а не file offset.

Чтобы статически прочитать UNWIND_INFO, analyzer берет RVA 0x18000, находит секцию, чей virtual range покрывает этот RVA, вычисляет delta относительно Section.VirtualAddress и прибавляет delta к PointerToRawData. Только после этого он получает file offset. Именно так PE связывает runtime address space с bytes on disk.

Эта сцена должна стать автоматической. Любой раз, когда ты видишь RVA в PE, спрашивай: я сейчас в мире файла или в мире процесса? Если в мире процесса, прибавляю ImageBase и получаю VA. Если в мире файла, ищу section и перевожу в file offset. Без этой дисциплины дальше невозможно надежно читать .pdata и .xdata.

PE удобнее всего понимать как набор координатных обещаний. Заголовки обещают, где читать секции. Section headers обещают, как переводить file layout в virtual layout. Data directories обещают, где находятся специальные таблицы. Exception Directory обещает, где искать function table. Но обещание еще не доказательство. Static analyzer должен проверять, не выходит ли указанный диапазон за файл, не перекрываются ли секции, не противоречат ли размеры, можно ли реально добраться до байтов.

Именно здесь рождается стиль безопасного PE-парсинга. Мы не читаем файл как доверенный объект. Мы читаем его как hostile input - недоверенные bytes. Даже если файл выглядит обычным, analyzer должен быть готов к тому, что любое поле может быть ложью. В контексте EH/Unwind Analyzer это особенно важно, потому что одна неверная RVA-связь способна испортить весь путь к UNWIND_INFO.

Когда читаешь PE, полезно постоянно держать две картинки. Первая картинка - файл как лежащая на диске лента байтов: offset 0, offset 0x100, offset 0x400. Вторая картинка - loaded image: ImageBase, страницы памяти, section permissions, RVA ranges. Многие ошибки начинаются с того, что человек переносит координату из первой картинки во вторую без перевода.

Например, Exception Directory хранит VirtualAddress и Size. Название VirtualAddress провоцирует думать о полном VA, но в Data Directory это RVA. Значит, статический анализатор не может просто сделать seek(VirtualAddress). Он должен найти секцию, проверить, что RVA входит в virtual range, затем вычислить raw offset. Если секции перекрываются или размеры неадекватны, даже этот базовый перевод становится сомнительным.

PE также важен тем, что loader не обязан слепо копировать файл. В памяти появляется image, где sections выровнены по SectionAlignment, недостающие области могут быть заполнены нулями, imports могут быть заполнены runtime-адресами, relocations могут изменить абсолютные значения. Поэтому "байты в файле" и "байты в процессе" совпадают не всегда и не полностью.

Для unwind особенно важны section characteristics. Если metadata лежат в writable секции, это не автоматический приговор, но это меняет доверие: данные маршрута потенциально могут быть изменены во время выполнения. Если handler target лежит в non-executable секции, это еще сильнее влияет на интерпретацию, потому что обработчик должен быть исполняемым кодом. Так PE-права становятся частью evidence model.

В проекте это переходит в table_integrity и metadata_placement. Сначала analyzer проверяет, можно ли доверять PE mapping. Потом он проверяет, где лежит metadata и куда указывают handler-адреса. Если перепутать эти уровни, findings станут шумными: placement без надежного mapping не имеет полной силы.

Сквозной пример главы 2: как это выглядит в живом анализе

Возьмем тот же crash, но теперь посмотрим на него через PE. RIP в дампе равен 0x00007FF612341234. Это VA. Чтобы понять, какая функция выполнялась, нужно сначала определить module. Допустим, адрес принадлежит app.exe, загруженному по ImageBase 0x00007FF612300000. Тогда RVA равен 0x41234. Именно RVA будет использоваться для поиска RUNTIME_FUNCTION.

Если открыть файл app.exe в hex editor и сразу перейти на offset 0x41234, можно ошибиться. Это RVA, а не file offset. Нужно найти секцию, например .text, у которой VirtualAddress = 0x1000, PointerToRawData = 0x400. Тогда delta = 0x41234 - 0x1000, a file offset = 0x400 + delta. Только так мы попадем к тем байтам, которые в памяти соответствуют этому RVA.

Тот же принцип работает для UnwindInfoAddress. Если RUNTIME_FUNCTION говорит, что UNWIND_INFO лежит по RVA 0x88000, analyzer не должен делать seek(0x88000) напрямую. Он должен найти секцию, в которую попадает 0x88000. Если такой секции нет, появляется finding. Если секция есть, но она writable/executable unexpectedly, это уже placement evidence.

Так простой вопрос "где лежит инструкция" приводит к дисциплине координат. PE не про красивые названия секций. PE про то, как связать file bytes, loaded image and runtime addresses.

```
RIP in process:       00007FF612341234
ImageBase:            00007FF612300000
RVA:                  0000000000041234

If section .text:
  VirtualAddress      00001000
  PointerToRawData    00000400
  file_offset = 00000400 + (00041234 - 00001000)
```

Итог главы 2: что должно остаться в голове

PE - это не просто файл с кодом, а карта loaded image. File offset, RVA, VA и ImageBase - разные системы координат. Exception Directory в Data Directory является формальным входом к x64 function table. Без правильного PE mapping невозможно надежно читать .pdata и .xdata.

Глава 3. .pdata и RUNTIME_FUNCTION: как Windows превращает RIP в функцию

После PE-карты мы впервые подходим к данным, которые прямо нужны x64 unwinder-у. .pdata и RUNTIME_FUNCTION отвечают на вопрос: если ControlPc находится по такому-то RVA, какой диапазон функции его покрывает и где лежит рецепт восстановления кадра?

3.1. Почему Windows начинает не с handler-a, а с функции

Исключение случилось по адресу RIP. Человеку хочется спросить: где обработчик? Но Windows сначала задает другой вопрос: какая функция содержит этот ControlPc? Без ответа на этот вопрос нельзя знать, какой unwind-рецепт применять и какие handler data относятся к текущему кадру.

RUNTIME_FUNCTION - это запись, которая превращает адрес внутри image в диапазон функции и ссылку на unwind metadata. Она маленькая: BeginAddress, EndAddress, UnwindInfoAddress. Но смысл огромный. Это не просто "границы функции для дизассемблера". Это часть runtime route: если ControlPc находится в этом диапазоне, функцию нужно раскручивать по указанному UNWIND_INFO.

Поэтому .pdata - не декоративная таблица. Она позволяет Windows построить мост между произвольным RIP и формальным описанием кадра. Без такого моста system unwinder был бы вынужден гадать по стеку.

На уровне .pdata хорошо видно различие между "найти функцию для человека" и "найти функцию для Windows". Декомпилятор может строить функцию эвристически, по переходам и прологам. Windows x64 exception machinery использует function table. Эти источники могут помогать друг другу, но не являются одним и тем же. Если Ghidra видит одну границу, а RUNTIME_FUNCTION объявляет другую, это не повод сразу выбирать одну сторону. Это повод проверить, почему слои расходятся.

Для проекта это превращается в evidence. table_integrity family отвечает именно за доверие к карте. Пока карта функций не выглядит корректной, любые более высокие выводы надо ослаблять. Нельзя уверенно рассуждать о dominance handler-a, если базовые ranges невозможно надежно прочитать. Поэтому аналитический порядок всегда начинается с фундаментальных таблиц.

Проверка .pdata похожа на проверку железнодорожного расписания. Если поезд находится на конкретной станции - ControlPc внутри диапазона - система должна найти единственную строку расписания, которая объясняет дальнейший путь. Если строк несколько, они перекрываются или ведут в никуда, расписание перестает быть надежной картой.

Важно не переоценивать имя .pdata. Формально Exception Directory говорит, где искать таблицу. В обычном MSVC-generated PE она действительно лежит в .pdata, но analyzer должен быть готов к необычным секциям. Поэтому правильный вопрос не "есть ли секция с именем .pdata", а "куда указывает Exception Directory и можно ли эту область прочитать как массив RUNTIME_FUNCTION".

Ручная проверка начинается с числа записей. Размер Exception Directory должен быть кратен размеру RUNTIME_FUNCTION. Затем каждая запись проверяется отдельно. BeginAddress и EndAddress должны образовывать осмысленный диапазон. UnwindInfoAddress должен вести в readable image area. После этого записи проверяются как набор: сортировка, overlaps, duplicate begins, suspicious density.

Иногда странность .pdata не означает злой умысел. Это может быть artifact компоновки, hand-written assembly, hot/cold split, chained unwind info или нестандартный toolchain. Но даже benign explanation не отменяет finding. Finding фиксирует факт: declared function table имеет такое-то свойство. Assessment уже учитывает контекст и альтернативы.

В проекте .pdata является зоной table_integrity, потому что от нее зависит все остальное. Если function ranges недостоверны, то handler dominance или reuse statistics могут считаться на неправильных связях. Поэтому хороший отчет должен ясно показывать: что именно сломано в table layer и как это влияет на confidence остальных выводов.

3.2. .pdata как индекс функций, а не как код

.pdata обычно содержит procedure data - данные процедур. В x64 PE это в первую очередь массив RUNTIME_FUNCTION. Код функции лежит обычно в .text. Подробный unwind-рецепт лежит обычно в .xdata. .pdata стоит между ними как индекс: от диапазона кода к метаданным восстановления.

Это удобно сравнить с библиотечным каталогом. Книга находится на полке, но каталог говорит, где ее искать и какая карточка к ней относится. .text - полка с кодом. .pdata - каталог диапазонов. .xdata - инструкция, как из функции корректно выйти назад по стеку.

Анализатор должен читать .pdata не как набор троек чисел, а как карту обязательств. BeginAddress должен быть меньше EndAddress. Диапазон должен попадать в executable code или допустимую область. Таблица обычно должна быть отсортирована. Диапазоны не должны хаотично пересекаться. UnwindInfoAddress должен вести в читаемую область image. Каждая такая проверка превращается в evidence.

На уровне .pdata хорошо видно различие между "найти функцию для человека" и "найти функцию для Windows". Декомпилятор может строить функцию эвристически, по переходам и прологам. Windows x64 exception machinery использует function table. Эти источники могут помогать друг другу, но не являются одним и тем же. Если Ghidra видит одну границу, а RUNTIME_FUNCTION объявляет другую, это не повод сразу выбирать одну сторону. Это повод проверить, почему слои расходятся.

Для проекта это превращается в evidence. table_integrity family отвечает именно за доверие к карте. Пока карта функций не выглядит корректной, любые более высокие выводы надо ослаблять. Нельзя уверенно рассуждать о dominance handler-а, если базовые ranges невозможно надежно прочитать. Поэтому аналитический порядок всегда начинается с фундаментальных таблиц.

Проверка .pdata похожа на проверку железнодорожного расписания. Если поезд находится на конкретной станции - ControlPc внутри диапазона - система должна найти единственную строку расписания, которая объясняет дальнейший путь. Если строк несколько, они перекрываются или ведут в никуда, расписание перестает быть надежной картой.

Важно не переоценивать имя .pdata. Формально Exception Directory говорит, где искать таблицу. В обычном MSVC-generated PE она действительно лежит в .pdata, но analyzer должен быть готов к необычным секциям. Поэтому правильный вопрос не "есть ли секция с именем .pdata", а "куда указывает Exception Directory и можно ли эту область прочитать как массив RUNTIME_FUNCTION".

Ручная проверка начинается с числа записей. Размер Exception Directory должен быть кратен размеру RUNTIME_FUNCTION. Затем каждая запись проверяется отдельно. BeginAddress и EndAddress должны образовывать осмысленный диапазон. UnwindInfoAddress должен вести в readable image area. После этого записи проверяются как набор: сортировка, overlaps, duplicate begins, suspicious density.

Иногда странность .pdata не означает злой умысел. Это может быть artifact компоновки, hand-written assembly, hot/cold split, chained unwind info или нестандартный toolchain. Но даже benign explanation не отменяет finding. Finding фиксирует факт: declared function table имеет такое-то свойство. Assessment уже учитывает контекст и альтернативы.

В проекте .pdata является зоной table_integrity, потому что от нее зависит все остальное. Если function ranges недостоверны, то handler dominance или reuse statistics могут считаться на неправильных связях. Поэтому хороший отчет должен ясно показывать: что именно сломано в table layer и как это влияет на confidence остальных выводов.

3.3. BeginAddress и EndAddress: полуоткрытый диапазон функции

BeginAddress - RVA начала диапазона. EndAddress - RVA конца диапазона. Обычно диапазон понимается как полуоткрытый: BeginAddress <= RVA < EndAddress. Такая форма удобна: конец одной функции может совпасть с началом другой без пересечения.

Если BeginAddress равен EndAddress или больше EndAddress, диапазон не описывает нормальную функцию. Если диапазон выходит за SizeOfImage или попадает в странную область, это нарушает доверие к function table. Если несколько записей перекрывают один и тот же адрес, Windows и инструменты могут получить неоднозначность.

Важно: границы RUNTIME_FUNCTION не всегда идеально совпадают с тем, как декомпилятор красиво рисует функцию. Компилятор, linker, hot/cold splitting, chained unwind info и оптимизации могут делать картину сложнее. Но это не отменяет базового требования: table должна быть достаточно корректной, чтобы system unwinder мог найти подходящую запись для ControlPc.

На уровне .pdata хорошо видно различие между "найти функцию для человека" и "найти функцию для Windows". Декомпилятор может строить функцию эвристически, по переходам и прологам. Windows x64 exception machinery использует function table. Эти источники могут помогать друг другу, но не являются одним и тем же. Если Ghidra видит одну границу, а RUNTIME_FUNCTION объявляет другую, это не повод сразу выбирать одну сторону. Это повод проверить, почему слои расходятся.

Для проекта это превращается в evidence. table_integrity family отвечает именно за доверие к карте. Пока карта функций не выглядит корректной, любые более высокие выводы надо ослаблять. Нельзя уверенно рассуждать о dominance handler-а, если базовые ranges невозможно надежно прочитать. Поэтому аналитический порядок всегда начинается с фундаментальных таблиц.

Проверка .pdata похожа на проверку железнодорожного расписания. Если поезд находится на конкретной станции - ControlPc внутри диапазона - система должна найти единственную строку расписания, которая объясняет дальнейший путь. Если строк несколько, они перекрываются или ведут в никуда, расписание перестает быть надежной картой.

Важно не переоценивать имя .pdata. Формально Exception Directory говорит, где искать таблицу. В обычном MSVC-generated PE она действительно лежит в .pdata, но analyzer должен быть готов к необычным секциям. Поэтому правильный вопрос не "есть ли секция с именем .pdata", а "куда указывает Exception Directory и можно ли эту область прочитать как массив RUNTIME_FUNCTION".

Ручная проверка начинается с числа записей. Размер Exception Directory должен быть кратен размеру RUNTIME_FUNCTION. Затем каждая запись проверяется отдельно. BeginAddress и EndAddress должны образовывать осмысленный диапазон. UnwindInfoAddress должен вести в readable image area. После этого записи проверяются как набор: сортировка, overlaps, duplicate begins, suspicious density.

Иногда странность .pdata не означает злой умысел. Это может быть artifact компоновки, hand-written assembly, hot/cold split, chained unwind info или нестандартный toolchain. Но даже benign explanation не отменяет finding. Finding фиксирует факт: declared function table имеет такое-то свойство. Assessment уже учитывает контекст и альтернативы.

В проекте .pdata является зоной table_integrity, потому что от нее зависит все остальное. Если function ranges недостоверны, то handler dominance или reuse statistics могут считаться на неправильных связях. Поэтому хороший отчет должен ясно показывать: что именно сломано в table layer и как это влияет на confidence остальных выводов.

3.4. UnwindInfoAddress: ссылка на рецепт, а не на код

UnwindInfoAddress указывает на UNWIND_INFO. Это RVA служебных данных, а не адрес handler-а и не начало функции. Новичок легко путает: раз поле находится в записи функции, значит оно должно вести к коду. Нет. Оно ведет к metadata, которая описывает действия пролога, frame register, flags, unwind codes и, при определенных flags, handler data.

Если UnwindInfoAddress указывает за пределы image, в неотображаемую область или в область, которую нельзя корректно прочитать, текущая функция теряет надежный unwind-рецепт. Тогда Windows не сможет формально восстановить caller context для этого кадра. Для static analyzer это не "мелкая ошибка адреса", а нарушение центральной связи .pdata -> .xdata.

При ручной проверке всегда нужно проходить цепочку полностью: Exception Directory дает массив RUNTIME_FUNCTION; выбранная RUNTIME_FUNCTION покрывает ControlPc; UnwindInfoAddress переводится как RVA; RVA попадает в секцию; секция имеет подходящие права; по полученному offset можно разобрать UNWIND_INFO.

На уровне .pdata хорошо видно различие между "найти функцию для человека" и "найти функцию для Windows". Декомпилятор может строить функцию эвристически, по переходам и прологам. Windows x64 exception machinery использует function table. Эти источники могут помогать друг другу, но не являются одним и тем же. Если Ghidra видит одну границу, а RUNTIME_FUNCTION объявляет другую, это не повод сразу выбирать одну сторону. Это повод проверить, почему слои расходятся.

Для проекта это превращается в evidence. table_integrity family отвечает именно за доверие к карте. Пока карта функций не выглядит корректной, любые более высокие выводы надо ослаблять. Нельзя уверенно рассуждать о dominance handler-а, если базовые ranges невозможно надежно прочитать. Поэтому аналитический порядок всегда начинается с фундаментальных таблиц.

Проверка .pdata похожа на проверку железнодорожного расписания. Если поезд находится на конкретной станции - ControlPc внутри диапазона - система должна найти единственную строку расписания, которая объясняет дальнейший путь. Если строк несколько, они перекрываются или ведут в никуда, расписание перестает быть надежной картой.

Важно не переоценивать имя .pdata. Формально Exception Directory говорит, где искать таблицу. В обычном MSVC-generated PE она действительно лежит в .pdata, но analyzer должен быть готов к необычным секциям. Поэтому правильный вопрос не "есть ли секция с именем .pdata", а "куда указывает Exception Directory и можно ли эту область прочитать как массив RUNTIME_FUNCTION".

Ручная проверка начинается с числа записей. Размер Exception Directory должен быть кратен размеру RUNTIME_FUNCTION. Затем каждая запись проверяется отдельно. BeginAddress и EndAddress должны образовывать осмысленный диапазон. UnwindInfoAddress должен вести в readable image area. После этого записи проверяются как набор: сортировка, overlaps, duplicate begins, suspicious density.

Иногда странность .pdata не означает злой умысел. Это может быть artifact компоновки, hand-written assembly, hot/cold split, chained unwind info или нестандартный toolchain. Но даже benign explanation не отменяет finding. Finding фиксирует факт: declared function table имеет такое-то свойство. Assessment уже учитывает контекст и альтернативы.

В проекте .pdata является зоной table_integrity, потому что от нее зависит все остальное. Если function ranges недостоверны, то handler dominance или reuse statistics могут считаться на неправильных связях. Поэтому хороший отчет должен ясно показывать: что именно сломано в table layer и как это влияет на confidence остальных выводов.

3.5. Сортировка, перекрытия и дубликаты как симптомы сломанной карты

Function table обычно должна быть отсортирована по BeginAddress, потому что поиск записи по ControlPc должен быть эффективным и однозначным. Неразобранная, несортированная или перекрывающаяся таблица похожа на карту города, где улицы подписаны хаотично и два квартала занимают одно место.

rf_unsorted, rf_overlaps, duplicate_begin и invalid_ranges - это не просто format warnings. Они говорят, что сама карта функций вызывает сомнение. Если карта сомнительна, то любая дальнейшая интерпретация handler-а или unwind code тоже становится менее надежной.

При этом нужно сохранять аккуратность. Некоторые бинарники могут быть нестандартными, некоторые области могут описывать fragments, некоторые runtime scenarios могут использовать dynamic function tables. Но static PE analyzer имеет право сказать: в статически объявленной table есть свойство, которое нарушает ожидания обычной Windows x64 unwind model.

На уровне .pdata хорошо видно различие между "найти функцию для человека" и "найти функцию для Windows". Декомпилятор может строить функцию эвристически, по переходам и прологам. Windows x64 exception machinery использует function table. Эти источники могут помогать друг другу, но не являются одним и тем же. Если Ghidra видит одну границу, а RUNTIME_FUNCTION объявляет другую, это не повод сразу выбирать одну сторону. Это повод проверить, почему слои расходятся.

Для проекта это превращается в evidence. table_integrity family отвечает именно за доверие к карте. Пока карта функций не выглядит корректной, любые более высокие выводы надо ослаблять. Нельзя уверенно рассуждать о dominance handler-а, если базовые ranges невозможно надежно прочитать. Поэтому аналитический порядок всегда начинается с фундаментальных таблиц.

Проверка .pdata похожа на проверку железнодорожного расписания. Если поезд находится на конкретной станции - ControlPc внутри диапазона - система должна найти единственную строку расписания, которая объясняет дальнейший путь. Если строк несколько, они перекрываются или ведут в никуда, расписание перестает быть надежной картой.

Важно не переоценивать имя .pdata. Формально Exception Directory говорит, где искать таблицу. В обычном MSVC-generated PE она действительно лежит в .pdata, но analyzer должен быть готов к необычным секциям. Поэтому правильный вопрос не "есть ли секция с именем .pdata", а "куда указывает Exception Directory и можно ли эту область прочитать как массив RUNTIME_FUNCTION".

Ручная проверка начинается с числа записей. Размер Exception Directory должен быть кратен размеру RUNTIME_FUNCTION. Затем каждая запись проверяется отдельно. BeginAddress и EndAddress должны образовывать осмысленный диапазон. UnwindInfoAddress должен вести в readable image area. После этого записи проверяются как набор: сортировка, overlaps, duplicate begins, suspicious density.

Иногда странность .pdata не означает злой умысел. Это может быть artifact компоновки, hand-written assembly, hot/cold split, chained unwind info или нестандартный toolchain. Но даже benign explanation не отменяет finding. Finding фиксирует факт: declared function table имеет такое-то свойство. Assessment уже учитывает контекст и альтернативы.

В проекте .pdata является зоной table_integrity, потому что от нее зависит все остальное. Если function ranges недостоверны, то handler dominance или reuse statistics могут считаться на неправильных связях. Поэтому хороший отчет должен ясно показывать: что именно сломано в table layer и как это влияет на confidence остальных выводов.

3.6. Leaf function: когда отсутствие записи не является ошибкой

Если RtlLookupFunctionEntry не нашел запись, x64 unwind procedure может трактовать код как leaf function. Leaf function в этом контексте - не просто маленькая функция. Это функция, которая не требует полноценного unwind metadata: она не сохранила nonvolatile-регистры и не изменила стек так, чтобы нужен был рецепт восстановления.

Для такой функции caller context можно получить просто: взять return address с вершины стека и увеличить RSP на 8. Это похоже на виртуальное выполнение ret. Но reverse engineer должен быть осторожен. Отсутствие записи может быть нормальным leaf case, а может быть признаком нестандартного кода, packer-а, JIT без регистрации function table или поврежденных metadata.

Поэтому хороший анализ не должен автоматически ругаться на любой код без .pdata-записи. Он должен смотреть контекст: действительно ли это leaf-like участок? Есть ли sub rsp, сохранение RBX/R12, handler flags, calls? Если код ведет себя как non-leaf, но записи нет, это уже другой уровень подозрения.

На уровне .pdata хорошо видно различие между "найти функцию для человека" и "найти функцию для Windows". Декомпилятор может строить функцию эвристически, по переходам и прологам. Windows x64 exception machinery использует function table. Эти источники могут помогать друг другу, но не являются одним и тем же. Если Ghidra видит одну границу, а RUNTIME_FUNCTION объявляет другую, это не повод сразу выбирать одну сторону. Это повод проверить, почему слои расходятся.

Для проекта это превращается в evidence. table_integrity family отвечает именно за доверие к карте. Пока карта функций не выглядит корректной, любые более высокие выводы надо ослаблять. Нельзя уверенно рассуждать о dominance handler-a, если базовые ranges невозможно надежно прочитать. Поэтому аналитический порядок всегда начинается с фундаментальных таблиц.

Проверка .pdata похожа на проверку железнодорожного расписания. Если поезд находится на конкретной станции - ControlPc внутри диапазона - система должна найти единственную строку расписания, которая объясняет дальнейший путь. Если строк несколько, они перекрываются или ведут в никуда, расписание перестает быть надежной картой.

Важно не переоценивать имя .pdata. Формально Exception Directory говорит, где искать таблицу. В обычном MSVC-generated PE она действительно лежит в .pdata, но analyzer должен быть готов к необычным секциям. Поэтому правильный вопрос не "есть ли секция с именем .pdata", а "куда указывает Exception Directory и можно ли эту область прочитать как массив RUNTIME_FUNCTION".

Ручная проверка начинается с числа записей. Размер Exception Directory должен быть кратен размеру RUNTIME_FUNCTION. Затем каждая запись проверяется отдельно. BeginAddress и EndAddress должны образовывать осмысленный диапазон. UnwindInfoAddress должен вести в readable image area. После этого записи проверяются как набор: сортировка, overlaps, duplicate begins, suspicious density.

Иногда странность .pdata не означает злой умысел. Это может быть artifact компоновки, hand-written assembly, hot/cold split, chained unwind info или нестандартный toolchain. Но даже benign explanation не отменяет finding. Finding фиксирует факт: declared function table имеет такое-то свойство. Assessment уже учитывает контекст и альтернативы.

В проекте .pdata является зоной table_integrity, потому что от нее зависит все остальное. Если function ranges недостоверны, то handler dominance или reuse statistics могут считаться на неправильных связях. Поэтому хороший отчет должен ясно показывать: что именно сломано в table layer и как это влияет на confidence остальных выводов.

3.7. Dynamic function tables: когда карта появляется во время выполнения

Не весь x64-код обязан иметь карту в статической .pdata конкретного PE-файла. JIT compiler может генерировать код во время выполнения. Runtime может зарегистрировать function table через RtlAddFunctionTable или предоставить callback через RtlInstallFunctionTableCallback. Для Windows принцип тот же: для unwind нужен function table, но она может быть не статической.

Это задает границу static analysis. Если analyzer читает файл на диске, он видит только то, что объявлено в PE. Он не видит будущие runtime registrations, если они создаются только во время выполнения. Поэтому static finding должен формулироваться честно: "в статическом PE нет такой-то записи" или "Exception Directory содержит такую-то аномалию", а не "Windows никогда не сможет раскрутить этот код".

Dynamic validation нужен именно для таких границ. Static analyzer строит declared model. Dynamic collector может увидеть runtime events и dynamic tables. Вместе они дают более сильную картину, но смешивать их в один слой нельзя.

На уровне .pdata хорошо видно различие между "найти функцию для человека" и "найти функцию для Windows". Декомпилятор может строить функцию эвристически, по переходам и прологам. Windows x64 exception machinery использует function table. Эти источники могут помогать друг другу, но не являются одним и тем же. Если Ghidra видит одну границу, а RUNTIME_FUNCTION объявляет другую, это не повод сразу выбирать одну сторону. Это повод проверить, почему слои расходятся.

Для проекта это превращается в evidence. table_integrity family отвечает именно за доверие к карте. Пока карта функций не выглядит корректной, любые более высокие выводы надо ослаблять. Нельзя уверенно рассуждать о dominance handler-a, если базовые ranges невозможно надежно прочитать. Поэтому аналитический порядок всегда начинается с фундаментальных таблиц.

Проверка .pdata похожа на проверку железнодорожного расписания. Если поезд находится на конкретной станции - ControlPc внутри диапазона - система должна найти единственную строку расписания, которая объясняет дальнейший путь. Если строк несколько, они перекрываются или ведут в никуда, расписание перестает быть надежной картой.

Важно не переоценивать имя .pdata. Формально Exception Directory говорит, где искать таблицу. В обычном MSVC-generated PE она действительно лежит в .pdata, но analyzer должен быть готов к необычным секциям. Поэтому правильный вопрос не "есть ли секция с именем .pdata", а "куда указывает Exception Directory и можно ли эту область прочитать как массив RUNTIME_FUNCTION".

Ручная проверка начинается с числа записей. Размер Exception Directory должен быть кратен размеру RUNTIME_FUNCTION. Затем каждая запись проверяется отдельно. BeginAddress и EndAddress должны образовывать осмысленный диапазон. UnwindInfoAddress должен вести в readable image area. После этого записи проверяются как набор: сортировка, overlaps, duplicate begins, suspicious density.

Иногда странность .pdata не означает злой умысел. Это может быть artifact компоновки, hand-written assembly, hot/cold split, chained unwind info или нестандартный toolchain. Но даже benign explanation не отменяет finding. Finding фиксирует факт: declared function table имеет такое-то свойство. Assessment уже учитывает контекст и альтернативы.

В проекте .pdata является зоной table_integrity, потому что от нее зависит все остальное. Если function ranges недостоверны, то handler dominance или reuse statistics могут считаться на неправильных связях. Поэтому хороший отчет должен ясно показывать: что именно сломано в table layer и как это влияет на confidence остальных выводов.

3.8. Ручной маршрут чтения .pdata

Практически .pdata читается как дисциплинированная процедура. Сначала берем Exception Directory из Data Directory. Затем проверяем, что RVA и Size каталога лежат в пределах image и корректно переводятся в file offset. Затем делим размер на размер RUNTIME_FUNCTION entry и читаем записи. Для каждой записи проверяем диапазон, сортировку, пересечения, UnwindInfoAddress и возможность дойти до UNWIND_INFO.

Такой маршрут важен потому, что он формирует мышление static analyzer-a. Анализатор не доверяет файлу. Он читает PE как недоверенные bytes. Любое поле может быть повреждено, намеренно искажено или просто нестандартно сгенерировано. Поэтому каждая координата проверяется через section table, boundaries and permissions.

После этой главы .pdata должна восприниматься не как очередная секция, а как первый настоящий мост от PE к runtime exception machinery. В следующей главе мы пройдем по этому мосту дальше: в .xdata и UNWIND_INFO, где лежит не просто адрес функции, а рецепт раскрутки кадра.

На уровне .pdata хорошо видно различие между "найти функцию для человека" и "найти функцию для Windows". Декомпилятор может строить функцию эвристически, по переходам и прологам. Windows x64 exception machinery использует function table. Эти источники могут помогать друг другу, но не являются одним и тем же. Если Ghidra видит одну границу, а RUNTIME_FUNCTION объявляет другую, это не повод сразу выбирать одну сторону. Это повод проверить, почему слои расходятся.

Для проекта это превращается в evidence. table_integrity family отвечает именно за доверие к карте. Пока карта функций не выглядит корректной, любые более высокие выводы надо ослаблять. Нельзя уверенно рассуждать о dominance handler-a, если базовые ranges невозможно надежно прочитать. Поэтому аналитический порядок всегда начинается с фундаментальных таблиц.

Проверка .pdata похожа на проверку железнодорожного расписания. Если поезд находится на конкретной станции - ControlPc внутри диапазона - система должна найти единственную строку расписания, которая объясняет дальнейший путь. Если строк несколько, они перекрываются или ведут в никуда, расписание перестает быть надежной картой.

Важно не переоценивать имя .pdata. Формально Exception Directory говорит, где искать таблицу. В обычном MSVC-generated PE она действительно лежит в .pdata, но analyzer должен быть готов к необычным секциям. Поэтому правильный вопрос не "есть ли секция с именем .pdata", а "куда указывает Exception Directory и можно ли эту область прочитать как массив RUNTIME_FUNCTION".

Ручная проверка начинается с числа записей. Размер Exception Directory должен быть кратен размеру RUNTIME_FUNCTION. Затем каждая запись проверяется отдельно. BeginAddress и EndAddress должны образовывать осмысленный диапазон. UnwindInfoAddress должен вести в readable image area. После этого записи проверяются как набор: сортировка, overlaps, duplicate begins, suspicious density.

Иногда странность .pdata не означает злой умысел. Это может быть artifact компоновки, hand-written assembly, hot/cold split, chained unwind info или нестандартный toolchain. Но даже benign explanation не отменяет finding. Finding фиксирует факт: declared function table имеет такое-то свойство. Assessment уже учитывает контекст и альтернативы.

В проекте .pdata является зоной table_integrity, потому что от нее зависит все остальное. Если function ranges недостоверны, то handler dominance или reuse statistics могут считаться на неправильных связях. Поэтому хороший отчет должен ясно показывать: что именно сломано в table layer и как это влияет на confidence остальных выводов.

Сквозной пример главы 3: как это выглядит в живом анализе

Продолжим пример. Теперь у нас есть RVA 0x41234. Windows ищет RUNTIME_FUNCTION, где BeginAddress <= 0x41234 < EndAddress. Допустим, найдена запись BeginAddress = 0x41000, EndAddress = 0x41580, UnwindInfoAddress = 0x88000. Это значит, что текущая инструкция находится внутри диапазона функции, а рецепт раскрутки лежит по RVA 0x88000.

Если рядом есть другая запись BeginAddress = 0x41400, EndAddress = 0x41800, возникает overlap. Какой function entry должен описывать ControlPc 0x41450? Если таблица создает неоднозначность, доверие к stack unwinding падает. Если BeginAddress = EndAddress, запись не описывает реального диапазона. Если UnwindInfoAddress = 0xFFFFFFF0, она ведет за image и рецепт недоступен.

Реверсер может проверить это руками. В PE viewer открыть Exception Directory, посмотреть размер таблицы, пройти записи, убедиться в сортировке и диапазонах. В Ghidra сравнить function boundaries с .pdata. В hex/parser вывести подозрительные entries. Именно такие проверки потом становятся table_integrity findings.

Этот пример показывает, почему .pdata не должна восприниматься как справочная секция. Она участвует в runtime path. Если здесь ошибка, Windows может не дойти до правильного UNWIND_INFO, а значит не сможет корректно восстановить caller context.

```
Control RVA: 00041234

RUNTIME_FUNCTION:
  BeginAddress       00041000
  EndAddress         00041580
  UnwindInfoAddress  00088000

Meaning:
  00041000 <= 00041234 < 00041580
  -> use UNWIND_INFO at RVA 00088000
```

Итог главы 3: что должно остаться в голове

.pdata - это индекс function ranges. RUNTIME_FUNCTION связывает диапазон кода с UNWIND_INFO. Проверка table_integrity начинается с границ, сортировки, пересечений, дубликатов, Exception Directory и UnwindInfoAddress. Без доверия к этой карте нельзя уверенно анализировать higher-level evidence.

Глава 4. .xdata и UNWIND_INFO: рецепт восстановления кадра

.pdata дает адрес рецепта. Теперь нужно понять сам рецепт. .xdata и UNWIND_INFO описывают не весь смысл функции, а минимальную информацию, необходимую, чтобы восстановить caller context. Здесь код функции впервые превращается в обратимую модель изменения стека и регистров.

4.1. Почему unwind data описывает именно пролог

Чтобы раскрутить кадр, нужно понимать, как он был построен. Функция строит кадр в prolog-е: сохраняет nonvolatile-регистры, выделяет место на стеке, иногда устанавливает frame pointer, иногда вызывает helper для большой stack allocation. Если unwind - это движение назад, то он должен уметь отменить важные действия пролога.

UNWIND_INFO не пытается описать всю функцию. Ему не интересно, как функция считает бизнес-логику, какие if-ы есть в теле и какие локальные переменные назывались count. Его интересует только то, что нужно для восстановления caller context: как изменился RSP, где сохранены nonvolatile-регистры, используется ли frame register, есть ли handler flags и дополнительная handler data.

Это принципиальный момент. UNWIND_INFO - не мини-дизассемблер. Это машинно-читаемый рецепт восстановления. Он должен быть достаточно точным, чтобы RtlVirtualUnwind мог из текущего context получить context вызывающей функции.

UNWIND_INFO особенно полезен тем, что заставляет думать о функции не как о черном ящике, а как о обратимом изменении state. Пролог изменил state: сдвинул RSP, сохранил registers, возможно установил frame register. Unwind должен восстановить state caller-a. Если представить это как математическую обратимость, становится ясно, почему каждое действие пролога должно быть описуемым.

Но описание не обязано быть приятным для новичка. Оно компактное, битовое, зависит от alignment, flags и количества slots. Поэтому хороший parser обязан быть скучно-аккуратным: проверять границы каждого чтения, учитывать padding, не читать handler data до того, как понятно, что это действительно handler data. Здесь внимательность важнее смелости.

Хороший способ понять UNWIND_INFO - представить, что обычный epilog не выполнился. Функция не дошла до add rsp, ..., pop rbx, ret. Но Windows все равно должна получить состояние caller-a. Значит, ей нужен внешний рецепт, который говорит, какие действия надо мысленно обратить. Этот рецепт и есть unwind metadata.

UNWIND_CODE не обязан совпадать с машинными инструкциями один-в-один, но обязан описывать их unwind-relevant effects. Если пролог сделал несколько инструкций, но только одна изменила состояние, нужное для caller context, metadata описывает именно этот эффект. Поэтому попытка читать UNWIND_INFO как "копию пролога" приводит к ошибке. Надо читать его как обратимую модель frame construction.

Особенно важно выравнивание. Массив unwind codes имеет slot-based layout, и после него handler/chained data начинается не произвольно, а с учетом alignment. Неправильный parser, который сдвинулся на два байта, дальше будет видеть фантазии: handler где его нет, chained info как адрес, случайные bytes как flags. Поэтому устойчивость parser-a важнее красивых эвристик.

Сравнение code vs metadata - отдельная мощная техника. Если prolog дизассемблера показывает push rbx; sub rsp, 40h, а UNWIND_INFO описывает сохранение другого регистра и другой stack allocation, это не сразу malware, но это серьезный повод проверить function start, chained info, compiler peculiarities или повреждение metadata. Такой mismatch может объяснить сломанный unwind.

В проекте эта глава ведет к unwind_integrity. Finding unwind_parse_fail означает, что рецепт не читается как структура. Finding unwind_anomalies означает, что структура читается, но содержит необычные свойства. Эти два уровня нужно разделять, потому что первый вопрос - можно ли вообще доверять bytes как UNWIND_INFO; второй - как интерпретировать уже прочитанный рецепт.

4.2. Prolog, body и epilog как разные состояния функции

Функция не одинаково "активна" во всех своих адресах. В prolog-е она еще строит кадр. В body она уже работает в нормальном состоянии. В epilog-е она разбирает кадр и уходит к caller-у. Для x64 unwind эти состояния важны, потому что одно и то же UNWIND_INFO нельзя наивно применять одинаково к любой точке.

Если исключение случилось в середине prolog-а, часть действий уже выполнена, часть еще нет. Нужно отменить только то, что реально успело произойти. Если исключение или stack walk попадает в epilog, функция уже выходит: RSP мог быть частично восстановлен, регистры уже могли быть pop-нуты. Поэтому epilog обрабатывается особыми правилами.

В body ситуация самая понятная: кадр построен, и если у UNWIND_INFO есть handler flags, текущий frame может участвовать в search phase. Но даже здесь handler data нужно читать только после корректного разбора flags и unwind code alignment.

UNWIND_INFO особенно полезен тем, что заставляет думать о функции не как о черном ящике, а как о обратимом изменении state. Пролог изменил state: сдвинул RSP, сохранил registers, возможно установил frame register. Unwind должен восстановить state caller-а. Если представить это как математическую обратимость, становится ясно, почему каждое действие пролога должно быть описуемым.

Но описание не обязано быть приятным для новичка. Оно компактное, битовое, зависит от alignment, flags и количества slots. Поэтому хороший parser обязан быть скучно-аккуратным: проверять границы каждого чтения, учитывать padding, не читать handler data до того, как понятно, что это действительно handler data. Здесь внимательность важнее смелости.

Хороший способ понять UNWIND_INFO - представить, что обычный epilog не выполнился. Функция не дошла до add rsp, ..., pop rbx, ret. Но Windows все равно должна получить состояние caller-а. Значит, ей нужен внешний рецепт, который говорит, какие действия надо мысленно обратить. Этот рецепт и есть unwind metadata.

UNWIND_CODE не обязан совпадать с машинными инструкциями один-в-один, но обязан описывать их unwind-relevant effects. Если пролог сделал несколько инструкций, но только одна изменила состояние, нужное для caller context, metadata описывает именно этот эффект. Поэтому попытка читать UNWIND_INFO как "копию пролога" приводит к ошибке. Надо читать его как обратимую модель frame construction.

Особенно важно выравнивание. Массив unwind codes имеет slot-based layout, и после него handler/chained data начинается не произвольно, а с учетом alignment. Неправильный parser, который сдвинулся на два байта, дальше будет видеть фантазии: handler где его нет, chained info как адрес, случайные bytes как flags. Поэтому устойчивость parser-а важнее красивых эвристик.

Сравнение code vs metadata - отдельная мощная техника. Если prolog дизассемблера показывает push rbx; sub rsp, 40h, а UNWIND_INFO описывает сохранение другого регистра и другой stack allocation, это не сразу malware, но это серьезный повод проверить function start, chained info, compiler peculiarities или повреждение metadata. Такой mismatch может объяснить сломанный unwind.

В проекте эта глава ведет к unwind_integrity. Finding unwind_parse_fail означает, что рецепт не читается как структура. Finding unwind_anomalies означает, что структура читается, но содержит необычные свойства. Эти два уровня нужно разделять, потому что первый вопрос - можно ли вообще доверять bytes как UNWIND_INFO; второй - как интерпретировать уже прочитанный рецепт.

4.3. Структура UNWIND_INFO как компактный договор

UNWIND_INFO содержит version, flags, size of prolog, count of unwind codes, frame register/frame offset и массив UNWIND_CODE. После массива, с учетом выравнивания, могут идти handler address, handler data или chained RUNTIME_FUNCTION-like information. Маленькие поля резко меняют интерпретацию следующих байтов.

Version говорит, какой формат используется. Flags говорят, есть ли exception handler, termination handler или chained info. SizeOfProlog помогает понять, попадает ли ControlPc в prolog. CountOfCodes говорит, сколько unwind code slots нужно прочитать. Frame register показывает, используется ли стабильная база кадра.

Если parser ошибся в одном из этих полей, он может сместиться и принять handler data за unwind code или chained info за handler address. Поэтому unwind_integrity findings важны: они проверяют не "красоту" данных, а способность вообще прочитать рецепт без потери синхронизации.

UNWIND_INFO особенно полезен тем, что заставляет думать о функции не как о черном ящике, а как о обратимом изменении state. Пролог изменил state: сдвинул RSP, сохранил registers, возможно установил frame register. Unwind должен восстановить state caller-a. Если представить это как математическую обратимость, становится ясно, почему каждое действие пролога должно быть описуемым.

Но описание не обязано быть приятным для новичка. Оно компактное, битовое, зависит от alignment, flags и количества slots. Поэтому хороший parser обязан быть скучно-аккуратным: проверять границы каждого чтения, учитывать padding, не читать handler data до того, как понятно, что это действительно handler data. Здесь внимательность важнее смелости.

Хороший способ понять UNWIND_INFO - представить, что обычный epilog не выполнился. Функция не дошла до add rsp, ..., pop rbx, ret. Но Windows все равно должна получить состояние caller-a. Значит, ей нужен внешний рецепт, который говорит, какие действия надо мысленно обратить. Этот рецепт и есть unwind metadata.

UNWIND_CODE не обязан совпадать с машинными инструкциями один-в-один, но обязан описывать их unwind-relevant effects. Если пролог сделал несколько инструкций, но только одна изменила состояние, нужное для caller context, metadata описывает именно этот эффект. Поэтому попытка читать UNWIND_INFO как "копию пролога" приводит к ошибке. Надо читать его как обратимую модель frame construction.

Особенно важно выравнивание. Массив unwind codes имеет slot-based layout, и после него handler/chained data начинается не произвольно, а с учетом alignment. Неправильный parser, который сдвинулся на два байта, дальше будет видеть фантазии: handler где его нет, chained info как адрес, случайные bytes как flags. Поэтому устойчивость parser-a важнее красивых эвристик.

Сравнение code vs metadata - отдельная мощная техника. Если prolog дизассемблера показывает push rbx; sub rsp, 40h, а UNWIND_INFO описывает сохранение другого регистра и другой stack allocation, это не сразу malware, но это серьезный повод проверить function start, chained info, compiler peculiarities или повреждение metadata. Такой mismatch может объяснить сломанный unwind.

В проекте эта глава ведет к unwind_integrity. Finding unwind_parse_fail означает, что рецепт не читается как структура. Finding unwind_anomalies означает, что структура читается, но содержит необычные свойства. Эти два уровня нужно разделять, потому что первый вопрос - можно ли вообще доверять bytes как UNWIND_INFO; второй - как интерпретировать уже прочитанный рецепт.

4.4. UNWIND_CODE: не инструкция, а описание эффекта инструкции

UNWIND_CODE часто хочется читать как список инструкций пролога. Это почти правильно по духу, но технически неверно. UNWIND_CODE описывает effect - эффект действия, важный для unwind. Например, UWOP_PUSH_NONVOL говорит, что nonvolatile register был сохранен через push. UWOP_ALLOC_SMALL или UWOP_ALLOC_LARGE описывает выделение stack space. UWOP_SAVE_NONVOL описывает сохранение регистра по смещению.

Почему не хранить сами инструкции? Потому что unwinder-у не нужен полный текст. Ему нужно знать, как восстановить state. Если пролог сохранил RBX, нужно знать где лежит старый RBX. Если выделил 0x80 байт, нужно знать, как вернуть RSP. Если установил frame pointer, нужно знать отношение frame register к стеку.

Это помогает реверсеру сравнивать два мира: дизассемблированный prolog и UNWIND_CODE. Если они рассказывают одну историю, metadata заслуживает доверия. Если расходятся, нужно разбираться: ошибка parser-а, chained info, оптимизация, нестандартный compiler или намеренное искажение.

UNWIND_INFO особенно полезен тем, что заставляет думать о функции не как о черном ящике, а как о обратимом изменении state. Пролог изменил state: сдвинул RSP, сохранил registers, возможно установил frame register. Unwind должен восстановить state caller-а. Если представить это как математическую обратимость, становится ясно, почему каждое действие пролога должно быть описуемым.

Но описание не обязано быть приятным для новичка. Оно компактное, битовое, зависит от alignment, flags и количества slots. Поэтому хороший parser обязан быть скучно-аккуратным: проверять границы каждого чтения, учитывать padding, не читать handler data до того, как понятно, что это действительно handler data. Здесь внимательность важнее смелости.

Хороший способ понять UNWIND_INFO - представить, что обычный epilog не выполнился. Функция не дошла до add rsp, ..., pop rbx, ret. Но Windows все равно должна получить состояние caller-а. Значит, ей нужен внешний рецепт, который говорит, какие действия надо мысленно обратить. Этот рецепт и есть unwind metadata.

UNWIND_CODE не обязан совпадать с машинными инструкциями один-в-один, но обязан описывать их unwind-relevant effects. Если пролог сделал несколько инструкций, но только одна изменила состояние, нужное для caller context, metadata описывает именно этот эффект. Поэтому попытка читать UNWIND_INFO как "копию пролога" приводит к ошибке. Надо читать его как обратимую модель frame construction.

Особенно важно выравнивание. Массив unwind codes имеет slot-based layout, и после него handler/chained data начинается не произвольно, а с учетом alignment. Неправильный parser, который сдвинулся на два байта, дальше будет видеть фантазии: handler где его нет, chained info как адрес, случайные bytes как flags. Поэтому устойчивость parser-а важнее красивых эвристик.

Сравнение code vs metadata - отдельная мощная техника. Если prolog дизассемблера показывает push rbx; sub rsp, 40h, а UNWIND_INFO описывает сохранение другого регистра и другой stack allocation, это не сразу malware, но это серьезный повод проверить function start, chained info, compiler peculiarities или повреждение metadata. Такой mismatch может объяснить сломанный unwind.

В проекте эта глава ведет к unwind_integrity. Finding unwind_parse_fail означает, что рецепт не читается как структура. Finding unwind_anomalies означает, что структура читается, но содержит необычные свойства. Эти два уровня нужно разделять, потому что первый вопрос - можно ли вообще доверять bytes как UNWIND_INFO; второй - как интерпретировать уже прочитанный рецепт.

4.5. Volatile и nonvolatile registers как ответственность между caller и callee

Calling convention делит регистры на volatile и nonvolatile. Volatile-регистр callee может испортить, и caller не должен ждать, что значение переживет вызов. Nonvolatile-регистр callee обязан восстановить, если использовал. В Windows x64 к nonvolatile относятся, например, RBX, RBP, RSI, RDI, R12-R15, а также XMM6-XMM15 для соответствующих случаев.

Почему unwind metadata интересуется nonvolatile-регистрами? Потому что caller имеет право рассчитывать на их сохранность. Если исключение прервало функцию до обычного epilog-а, система должна восстановить эти значения сама, чтобы caller context был честным. Если RBX был сохранен на стеке, UNWIND_INFO должен подсказать, где он лежит.

Так nonvolatile превращается из сухого ABI-термина в runtime obligation - обязательство времени выполнения. Unwind поддерживает это обязательство даже при необычном выходе через исключение.

UNWIND_INFO особенно полезен тем, что заставляет думать о функции не как о черном ящике, а как о обратимом изменении state. Пролог изменил state: сдвинул RSP, сохранил registers, возможно установил frame register. Unwind должен восстановить state caller-a. Если представить это как математическую обратимость, становится ясно, почему каждое действие пролога должно быть описуемым.

Но описание не обязано быть приятным для новичка. Оно компактное, битовое, зависит от alignment, flags и количества slots. Поэтому хороший parser обязан быть скучно-аккуратным: проверять границы каждого чтения, учитывать padding, не читать handler data до того, как понятно, что это действительно handler data. Здесь внимательность важнее смелости.

Хороший способ понять UNWIND_INFO - представить, что обычный epilog не выполнился. Функция не дошла до add rsp, ..., pop rbx, ret. Но Windows все равно должна получить состояние caller-a. Значит, ей нужен внешний рецепт, который говорит, какие действия надо мысленно обратить. Этот рецепт и есть unwind metadata.

UNWIND_CODE не обязан совпадать с машинными инструкциями один-в-один, но обязан описывать их unwind-relevant effects. Если пролог сделал несколько инструкций, но только одна изменила состояние, нужное для caller context, metadata описывает именно этот эффект. Поэтому попытка читать UNWIND_INFO как "копию пролога" приводит к ошибке. Надо читать его как обратимую модель frame construction.

Особенно важно выравнивание. Массив unwind codes имеет slot-based layout, и после него handler/chained data начинается не произвольно, а с учетом alignment. Неправильный parser, который сдвинулся на два байта, дальше будет видеть фантазии: handler где его нет, chained info как адрес, случайные bytes как flags. Поэтому устойчивость parser-а важнее красивых эвристик.

Сравнение code vs metadata - отдельная мощная техника. Если prolog дизассемблера показывает push rbx; sub rsp, 40h, а UNWIND_INFO описывает сохранение другого регистра и другой stack allocation, это не сразу malware, но это серьезный повод проверить function start, chained info, compiler peculiarities или повреждение metadata. Такой mismatch может объяснить сломанный unwind.

В проекте эта глава ведет к unwind_integrity. Finding unwind_parse_fail означает, что рецепт не читается как структура. Finding unwind_anomalies означает, что структура читается, но содержит необычные свойства. Эти два уровня нужно разделять, потому что первый вопрос - можно ли вообще доверять bytes как UNWIND_INFO; второй - как интерпретировать уже прочитанный рецепт.

4.6. Frame register и frame pointer без школьной картинки с RBP

В старых объяснениях стек часто рисуют через RBP: push rbp, mov rbp, rsp, дальше локальные переменные по [rbp-N]. На Windows x64 такая схема возможна, но не обязательна. Оптимизирующий compiler может не использовать RBP как frame pointer, потому что регистр можно потратить на полезную работу.

Frame register в UNWIND_INFO - это способ сказать unwinder-у: в этой функции есть регистр, который используется как стабильная база кадра. Это не обязано выглядеть как классическая x86-картинка. Смысл не в ностальгии по EBP, а в том, что при динамических изменениях RSP иногда нужна стабильная точка для адресации frame data.

Реверсер не должен делать вывод "нет push rbp - нет нормального кадра". На x64 нормальность кадра определяется не привычной формой пролога, а тем, согласованы ли machine code, ABI и unwind metadata.

UNWIND_INFO особенно полезен тем, что заставляет думать о функции не как о черном ящике, а как о обратимом изменении state. Пролог изменил state: сдвинул RSP, сохранил registers, возможно установил frame register. Unwind должен восстановить state caller-a. Если представить это как математическую обратимость, становится ясно, почему каждое действие пролога должно быть описуемым.

Но описание не обязано быть приятным для новичка. Оно компактное, битовое, зависит от alignment, flags и количества slots. Поэтому хороший parser обязан быть скучно-аккуратным: проверять границы каждого чтения, учитывать padding, не читать handler data до того, как понятно, что это действительно handler data. Здесь внимательность важнее смелости.

Хороший способ понять UNWIND_INFO - представить, что обычный epilog не выполнился. Функция не дошла до add rsp, ..., pop rbx, ret. Но Windows все равно должна получить состояние caller-a. Значит, ей нужен внешний рецепт, который говорит, какие действия надо мысленно обратить. Этот рецепт и есть unwind metadata.

UNWIND_CODE не обязан совпадать с машинными инструкциями один-в-один, но обязан описывать их unwind-relevant effects. Если пролог сделал несколько инструкций, но только одна изменила состояние, нужное для caller context, metadata описывает именно этот эффект. Поэтому попытка читать UNWIND_INFO как "копию пролога" приводит к ошибке. Надо читать его как обратимую модель frame construction.

Особенно важно выравнивание. Массив unwind codes имеет slot-based layout, и после него handler/chained data начинается не произвольно, а с учетом alignment. Неправильный parser, который сдвинулся на два байта, дальше будет видеть фантазии: handler где его нет, chained info как адрес, случайные bytes как flags. Поэтому устойчивость parser-a важнее красивых эвристик.

Сравнение code vs metadata - отдельная мощная техника. Если prolog дизассемблера показывает push rbx; sub rsp, 40h, а UNWIND_INFO описывает сохранение другого регистра и другой stack allocation, это не сразу malware, но это серьезный повод проверить function start, chained info, compiler peculiarities или повреждение metadata. Такой mismatch может объяснить сломанный unwind.

В проекте эта глава ведет к unwind_integrity. Finding unwind_parse_fail означает, что рецепт не читается как структура. Finding unwind_anomalies означает, что структура читается, но содержит необычные свойства. Эти два уровня нужно разделять, потому что первый вопрос - можно ли вообще доверять bytes как UNWIND_INFO; второй - как интерпретировать уже прочитанный рецепт.

4.7. Handler flags и chained info: хвост структуры меняет смысл

UNW_FLAG_EHANDLER говорит, что есть exception handler. UNW_FLAG_UHANDLER говорит, что есть termination handler. UNW_FLAG_CHAININFO говорит, что хвост структуры содержит chained unwind information, а не обычный handler address. Это маленькие bits, но они полностью меняют смысл последующих данных.

Если analyzer не учитывает CHAININFO, он может принять embedded RUNTIME_FUNCTION за handler address и сделать ложный вывод. Если он не учитывает alignment массива UNWIND_CODE, он может начать читать handler data с неправильного offset. Если он не различает exception handler и termination handler, он смешает search phase и unwind phase.

В бинарных форматах смысл байтов всегда контекстный. Одно и то же место может быть handler RVA, chained entry или language-specific data в зависимости от flags. Поэтому parser должен читать не только числа, но и грамматику структуры.

UNWIND_INFO особенно полезен тем, что заставляет думать о функции не как о черном ящике, а как о обратимом изменении state. Пролог изменил state: сдвинул RSP, сохранил registers, возможно установил frame register. Unwind должен восстановить state caller-a. Если представить это как математическую обратимость, становится ясно, почему каждое действие пролога должно быть описуемым.

Но описание не обязано быть приятным для новичка. Оно компактное, битовое, зависит от alignment, flags и количества slots. Поэтому хороший parser обязан быть скучно-аккуратным: проверять границы каждого чтения, учитывать padding, не читать handler data до того, как понятно, что это действительно handler data. Здесь внимательность важнее смелости.

Хороший способ понять UNWIND_INFO - представить, что обычный epilog не выполнился. Функция не дошла до add rsp, ..., pop rbx, ret. Но Windows все равно должна получить состояние caller-a. Значит, ей нужен внешний рецепт, который говорит, какие действия надо мысленно обратить. Этот рецепт и есть unwind metadata.

UNWIND_CODE не обязан совпадать с машинными инструкциями один-в-один, но обязан описывать их unwind-relevant effects. Если пролог сделал несколько инструкций, но только одна изменила состояние, нужное для caller context, metadata описывает именно этот эффект. Поэтому попытка читать UNWIND_INFO как "копию пролога" приводит к ошибке. Надо читать его как обратимую модель frame construction.

Особенно важно выравнивание. Массив unwind codes имеет slot-based layout, и после него handler/chained data начинается не произвольно, а с учетом alignment. Неправильный parser, который сдвинулся на два байта, дальше будет видеть фантазии: handler где его нет, chained info как адрес, случайные bytes как flags. Поэтому устойчивость parser-a важнее красивых эвристик.

Сравнение code vs metadata - отдельная мощная техника. Если prolog дизассемблера показывает push rbx; sub rsp, 40h, а UNWIND_INFO описывает сохранение другого регистра и другой stack allocation, это не сразу malware, но это серьезный повод проверить function start, chained info, compiler peculiarities или повреждение metadata. Такой mismatch может объяснить сломанный unwind.

В проекте эта глава ведет к unwind_integrity. Finding unwind_parse_fail означает, что рецепт не читается как структура. Finding unwind_anomalies означает, что структура читается, но содержит необычные свойства. Эти два уровня нужно разделять, потому что первый вопрос - можно ли вообще доверять bytes как UNWIND_INFO; второй - как интерпретировать уже прочитанный рецепт.

4.8. Почему UNWIND_INFO является зоной доверия

UNWIND_INFO - это зона доверия потому, что Windows использует ее для восстановления caller context. Если данные корректны, stack walking и exception dispatch получают формальную опору. Если данные повреждены, система может восстановить неправильный RIP, неправильный RSP или неправильные nonvolatile-регистры.

Для static analyzer это значит: unwind_parse_fail и unwind_anomalies должны быть сформулированы очень осторожно. Parse fail - это невозможность корректно прочитать структуру. Anomaly - это необычное свойство при формально успешном чтении. Эти два случая нельзя смешивать, потому что первый ломает доверие к структуре сильнее, а второй требует интерпретации.

После этой главы .xdata должна быть понятна не как "расширенные данные", а как место, где PE image хранит рецепт выхода из функции назад. В следующей главе мы посмотрим, как Windows применяет этот рецепт через RtlLookupFunctionEntry и RtlVirtualUnwind.

UNWIND_INFO особенно полезен тем, что заставляет думать о функции не как о черном ящике, а как о обратимом изменении state. Пролог изменил state: сдвинул RSP, сохранил registers, возможно установил frame register. Unwind должен восстановить state caller-a. Если представить это как математическую обратимость, становится ясно, почему каждое действие пролога должно быть описуемым.

Но описание не обязано быть приятным для новичка. Оно компактное, битовое, зависит от alignment, flags и количества slots. Поэтому хороший parser обязан быть скучно-аккуратным: проверять границы каждого чтения, учитывать padding, не читать handler data до того, как понятно, что это действительно handler data. Здесь внимательность важнее смелости.

Хороший способ понять UNWIND_INFO - представить, что обычный epilog не выполнился. Функция не дошла до add rsp, ..., pop rbx, ret. Но Windows все равно должна получить состояние caller-a. Значит, ей нужен внешний рецепт, который говорит, какие действия надо мысленно обратить. Этот рецепт и есть unwind metadata.

UNWIND_CODE не обязан совпадать с машинными инструкциями один-в-один, но обязан описывать их unwind-relevant effects. Если пролог сделал несколько инструкций, но только одна изменила состояние, нужное для caller context, metadata описывает именно этот эффект. Поэтому попытка читать UNWIND_INFO как "копию пролога" приводит к ошибке. Надо читать его как обратимую модель frame construction.

Особенно важно выравнивание. Массив unwind codes имеет slot-based layout, и после него handler/chained data начинается не произвольно, а с учетом alignment. Неправильный parser, который сдвинулся на два байта, дальше будет видеть фантазии: handler где его нет, chained info как адрес, случайные bytes как flags. Поэтому устойчивость parser-a важнее красивых эвристик.

Сравнение code vs metadata - отдельная мощная техника. Если prolog дизассемблера показывает push rbx; sub rsp, 40h, а UNWIND_INFO описывает сохранение другого регистра и другой stack allocation, это не сразу malware, но это серьезный повод проверить function start, chained info, compiler peculiarities или повреждение metadata. Такой mismatch может объяснить сломанный unwind.

В проекте эта глава ведет к unwind_integrity. Finding unwind_parse_fail означает, что рецепт не читается как структура. Finding unwind_anomalies означает, что структура читается, но содержит необычные свойства. Эти два уровня нужно разделять, потому что первый вопрос - можно ли вообще доверять bytes как UNWIND_INFO; второй - как интерпретировать уже прочитанный рецепт.

Сквозной пример главы 4: как это выглядит в живом анализе

Теперь мы пришли к RVA 0x88000 и читаем UNWIND_INFO. Допустим, структура говорит: prolog size = 8, count of unwind codes = 2, operations: UWOP_PUSH_NONVOL RBX и UWOP_ALLOC_SMALL 0x40. Это означает, что функция в начале сохранила RBX и выделила 0x40 байт stack space. Для обычного человека это просто детали пролога, а для unwinder-а это рецепт восстановления.

Если исключение произошло в body, значит prolog уже выполнен. Чтобы получить caller context, RtlVirtualUnwind должен учесть оба действия: восстановить RBX из стека и поднять RSP так, чтобы добраться до return address. Если исключение произошло внутри первых байтов prolog-а, нужно учитывать только уже выполненные операции. Поэтому размер prolog-а и offset ControlPc имеют значение.

Сравним с машинным кодом. Если дизассемблер показывает push rbx; sub rsp, 40h, metadata выглядит правдоподобно. Если код показывает sub rsp, 80h, а metadata говорит 0x40, возникает вопрос. Возможно, начало функции найдено неверно. Возможно, есть chained unwind info. Возможно, parser ошибся. Возможно, metadata intentionally misleading.

Так UNWIND_INFO становится не абстрактной структурой, а проверяемым утверждением о том, что функция сделала со state. Чем точнее это утверждение, тем надежнее stack walking.

```
Possible prolog:
  push rbx
  sub  rsp, 40h

UNWIND_INFO says:
  UWOP_PUSH_NONVOL RBX
  UWOP_ALLOC_SMALL 40h

Virtual unwind must undo:
  restore RBX
  RSP = RSP + 40h + saved register area
  RIP = return address
```

Итог главы 4: что должно остаться в голове

UNWIND_INFO - это рецепт восстановления caller context. Он описывает эффекты prolog-а, handler flags, frame register и unwind codes. Ошибка в чтении этой структуры ломает не только красивую картинку, но и способность понять, как Windows будет раскручивать стек.

Глава 5. RtlLookupFunctionEntry и RtlVirtualUnwind: как Windows поднимается к caller-у

Теперь статические структуры начинают работать. Windows берет ControlPc, ищет FunctionEntry, читает UNWIND_INFO и виртуально поднимается к caller-у. Пятая глава показывает этот runtime-маршрут как последовательность действий, а не набор API-имен.

5.1. От ControlPc к FunctionEntry: первый runtime-шаг

В runtime у Windows есть ControlPc - адрес управления, обычно связанный с RIP текущего context. Первый шаг - найти function entry. Для этого используется RtlLookupFunctionEntry: она ищет запись в active function tables и возвращает RUNTIME_FUNCTION плюс ImageBase. Active function tables включают не только статическую .pdata модулей, но и динамически зарегистрированные таблицы.

Этот шаг соединяет runtime-мир с PE-миром. ControlPc - VA внутри процесса. RUNTIME_FUNCTION хранит image-relative values. ImageBase нужен, чтобы правильно интерпретировать RVA. Без ImageBase поле BeginAddress не превратится в реальный диапазон адресов.

Если FunctionEntry не найдена, включается leaf-путь. Если найдена, Windows должна понять, находится ли ControlPc в prolog, epilog или body, и применить соответствующие правила unwind.

Пятая глава переводит статические структуры в runtime-машину. До этого мы видели поля на диске. Теперь эти поля становятся действиями: lookup, classify prolog/body/epilog, call handler, virtual unwind. Именно здесь становится понятно, почему static findings могут влиять на практический stack walking. Если UnwindInfoAddress сломан, это не просто плохой байт в файле. Это потенциально неверный переход от текущего context к caller context.

При этом важно не приписывать static analyzer-у больше, чем он знает. Он может сказать, что declared metadata выглядит нарушенной или необычной. Но он не всегда может знать, будет ли этот конкретный путь достигнут при runtime execution. Поэтому дальше появляется dynamic validation: не как замена static analysis, а как наблюдение за тем, какие маршруты реально проявились.

Runtime-модель полезна тем, что превращает статические findings в причинные сбои. Например, unwind_rva_oob в отчете - это не просто "адрес за границей". В search loop это означает: когда Windows дошла бы до этого FunctionEntry, она не смогла бы получить корректный UNWIND_INFO по заявленному адресу. Значит, переход к caller context становится невозможным или недостоверным.

RtlLookupFunctionEntry и RtlVirtualUnwind часто воспринимают как API-имена, которые надо выучить. Но для книги важнее их роли. Первый API отвечает на вопрос "какая запись описывает текущий ControlPc". Второй отвечает на вопрос "как из текущего context получить context предыдущей функции". Эти два вопроса повторяются на каждом кадре.

Когда handler вызывается, он не живет отдельно от unwind. Он получает сведения, подготовленные dispatch/unwind machinery. DispatcherContext связывает PE function entry, image base, handler data and control PC. EstablisherFrame связывает handler с конкретным frame. Поэтому handler decision зависит от того, насколько правильно Windows дошла до этого кадра.

Для ручного анализа полезно проигрывать loop вслух: current RIP, module, RVA, function entry, unwind info, prolog/body/epilog, handler flags, decision, virtual unwind, next RIP. Если где-то в этой цепочке появляется "не знаю", это не стыдно. Это место для finding, hypothesis or dynamic validation.

В проекте runtime-модель объясняет, почему static findings имеют разный вес. Поврежденный PE mapping ломает самые ранние шаги. UNWIND_INFO anomaly ломает рецепт. Handler placement влияет на target transfer. Reuse/dominance patterns возникают уже после построения множества связей. Score должен учитывать эту причинную глубину.

5.2. Leaf path: виртуальный ret без полноценного рецепта

Когда FunctionEntry отсутствует, x64 unwind procedure может считать код leaf function. Тогда caller context получается простым способом: return address берется с [RSP], а RSP увеличивается на размер адреса, то есть на 8 байт. Это похоже на виртуальное выполнение ret.

Но для reverse engineering это не конец анализа. Нужно спросить: почему записи нет? Это действительно leaf function? Или код изменил RSP, сохранил nonvolatile registers, вызывает другие функции, но не имеет metadata? В первом случае все нормально. Во втором отсутствие function entry может объяснять сломанный stack trace или подозрительное поведение.

Таким образом, leaf path - это не "Windows не знает, что делать". Это формальное ABI-ожидание. Но static analyzer и реверсер должны понимать границу этого ожидания.

Пятая глава переводит статические структуры в runtime-машину. До этого мы видели поля на диске. Теперь эти поля становятся действиями: lookup, classify prolog/body/epilog, call handler, virtual unwind. Именно здесь становится понятно, почему static findings могут влиять на практический stack walking. Если UnwindInfoAddress сломан, это не просто плохой байт в файле. Это потенциально неверный переход от текущего context к caller context.

При этом важно не приписывать static analyzer-у больше, чем он знает. Он может сказать, что declared metadata выглядит нарушенной или необычной. Но он не всегда может знать, будет ли этот конкретный путь достигнут при runtime execution. Поэтому дальше появляется dynamic validation: не как замена static analysis, а как наблюдение за тем, какие маршруты реально проявились.

Runtime-модель полезна тем, что превращает статические findings в причинные сбои. Например, unwind_rva_oob в отчете - это не просто "адрес за границей". В search loop это означает: когда Windows дошла бы до этого FunctionEntry, она не смогла бы получить корректный UNWIND_INFO по заявленному адресу. Значит, переход к caller context становится невозможным или недостоверным.

RtlLookupFunctionEntry и RtlVirtualUnwind часто воспринимают как API-имена, которые надо выучить. Но для книги важнее их роли. Первый API отвечает на вопрос "какая запись описывает текущий ControlPc". Второй отвечает на вопрос "как из текущего context получить context предыдущей функции". Эти два вопроса повторяются на каждом кадре.

Когда handler вызывается, он не живет отдельно от unwind. Он получает сведения, подготовленные dispatch/unwind machinery. DispatcherContext связывает PE function entry, image base, handler data and control PC. EstablisherFrame связывает handler с конкретным frame. Поэтому handler decision зависит от того, насколько правильно Windows дошла до этого кадра.

Для ручного анализа полезно проигрывать loop вслух: current RIP, module, RVA, function entry, unwind info, prolog/body/epilog, handler flags, decision, virtual unwind, next RIP. Если где-то в этой цепочке появляется "не знаю", это не стыдно. Это место для finding, hypothesis or dynamic validation.

В проекте runtime-модель объясняет, почему static findings имеют разный вес. Поврежденный PE mapping ломает самые ранние шаги. UNWIND_INFO anomaly ломает рецепт. Handler placement влияет на target transfer. Reuse/dominance patterns возникают уже после построения множества связей. Score должен учитывать эту причинную глубину.

5.3. RtlVirtualUnwind как машина перехода к caller context

RtlVirtualUnwind не выполняет код назад. Она не вызывает ret, не откатывает инструкции и не запускает функцию в обратную сторону. Она берет current context, ImageBase, ControlPc, FunctionEntry и UNWIND_INFO, затем вычисляет context вызывающей функции.

Если текущая функция выделила 0x80 байт стека, виртуальная раскрутка учитывает это. Если она сохранила RBX, раскрутка восстанавливает RBX из сохраненного места. Если используется frame register, учитывается relation между frame register и RSP. В конце новый RIP обычно становится return address caller-a, а новый RSP соответствует стеку caller-a.

Слово virtual здесь ключевое. Это моделирование state transition. Для search phase оно позволяет пройти по кадрам, не разрушая стек реальным выполнением cleanup. Для stack walking оно позволяет построить call stack из context и metadata.

Пятая глава переводит статические структуры в runtime-машину. До этого мы видели поля на диске. Теперь эти поля становятся действиями: lookup, classify prolog/body/epilog, call handler, virtual unwind. Именно здесь становится понятно, почему static findings могут влиять на практический stack walking. Если UnwindInfoAddress сломан, это не просто плохой байт в файле. Это потенциально неверный переход от текущего context к caller context.

При этом важно не приписывать static analyzer-у больше, чем он знает. Он может сказать, что declared metadata выглядит нарушенной или необычной. Но он не всегда может знать, будет ли этот конкретный путь достигнут при runtime execution. Поэтому дальше появляется dynamic validation: не как замена static analysis, а как наблюдение за тем, какие маршруты реально проявились.

Runtime-модель полезна тем, что превращает статические findings в причинные сбои. Например, unwind_rva_oob в отчете - это не просто "адрес за границей". В search loop это означает: когда Windows дошла бы до этого FunctionEntry, она не смогла бы получить корректный UNWIND_INFO по заявленному адресу. Значит, переход к caller context становится невозможным или недостоверным.

RtlLookupFunctionEntry и RtlVirtualUnwind часто воспринимают как API-имена, которые надо выучить. Но для книги важнее их роли. Первый API отвечает на вопрос "какая запись описывает текущий ControlPc". Второй отвечает на вопрос "как из текущего context получить context предыдущей функции". Эти два вопроса повторяются на каждом кадре.

Когда handler вызывается, он не живет отдельно от unwind. Он получает сведения, подготовленные dispatch/unwind machinery. DispatcherContext связывает PE function entry, image base, handler data and control PC. EstablisherFrame связывает handler с конкретным frame. Поэтому handler decision зависит от того, насколько правильно Windows дошла до этого кадра.

Для ручного анализа полезно проигрывать loop вслух: current RIP, module, RVA, function entry, unwind info, prolog/body/epilog, handler flags, decision, virtual unwind, next RIP. Если где-то в этой цепочке появляется "не знаю", это не стыдно. Это место для finding, hypothesis or dynamic validation.

В проекте runtime-модель объясняет, почему static findings имеют разный вес. Поврежденный PE mapping ломает самые ранние шаги. UNWIND_INFO anomaly ломает рецепт. Handler placement влияет на target transfer. Reuse/dominance patterns возникают уже после построения множества связей. Score должен учитывать эту причинную глубину.

5.4. EstablisherFrame и DispatcherContext: служебные слова с практическим смыслом

EstablisherFrame можно понимать как базу фиксированного стекового выделения текущей функции - опорную точку кадра, которую handler использует для связи language-specific data с конкретным frame. Это не абстрактная "рамка". Это адресное значение, важное для обработчика.

DispatcherContext - это досье текущего шага dispatch. В него попадают FunctionEntry, ImageBase, ControlPc, HandlerData и другие сведения, которые language-specific handler использует, чтобы принять решение. Windows не просто зовет handler "в пустоту". Она передает ему контекст текущего кадра и данные, которые уже были найдены через PE/unwind route.

Если не понимать эти структуры, кажется, что handler вызывается магически. На самом деле handler получает именно тот набор сведений, который связывает ExceptionRecord, ContextRecord, PE metadata и текущий frame.

Пятая глава переводит статические структуры в runtime-машину. До этого мы видели поля на диске. Теперь эти поля становятся действиями: lookup, classify prolog/body/epilog, call handler, virtual unwind. Именно здесь становится понятно, почему static findings могут влиять на практический stack walking. Если UnwindInfoAddress сломан, это не просто плохой байт в файле. Это потенциально неверный переход от текущего context к caller context.

При этом важно не приписывать static analyzer-у больше, чем он знает. Он может сказать, что declared metadata выглядит нарушенной или необычной. Но он не всегда может знать, будет ли этот конкретный путь достигнут при runtime execution. Поэтому дальше появляется dynamic validation: не как замена static analysis, а как наблюдение за тем, какие маршруты реально проявились.

Runtime-модель полезна тем, что превращает статические findings в причинные сбои. Например, unwind_rva_oob в отчете - это не просто "адрес за границей". В search loop это означает: когда Windows дошла бы до этого FunctionEntry, она не смогла бы получить корректный UNWIND_INFO по заявленному адресу. Значит, переход к caller context становится невозможным или недостоверным.

RtlLookupFunctionEntry и RtlVirtualUnwind часто воспринимают как API-имена, которые надо выучить. Но для книги важнее их роли. Первый API отвечает на вопрос "какая запись описывает текущий ControlPc". Второй отвечает на вопрос "как из текущего context получить context предыдущей функции". Эти два вопроса повторяются на каждом кадре.

Когда handler вызывается, он не живет отдельно от unwind. Он получает сведения, подготовленные dispatch/unwind machinery. DispatcherContext связывает PE function entry, image base, handler data and control PC. EstablisherFrame связывает handler с конкретным frame. Поэтому handler decision зависит от того, насколько правильно Windows дошла до этого кадра.

Для ручного анализа полезно проигрывать loop вслух: current RIP, module, RVA, function entry, unwind info, prolog/body/epilog, handler flags, decision, virtual unwind, next RIP. Если где-то в этой цепочке появляется "не знаю", это не стыдно. Это место для finding, hypothesis or dynamic validation.

В проекте runtime-модель объясняет, почему static findings имеют разный вес. Поврежденный PE mapping ломает самые ранние шаги. UNWIND_INFO anomaly ломает рецепт. Handler placement влияет на target transfer. Reuse/dominance patterns возникают уже после построения множества связей. Score должен учитывать эту причинную глубину.

5.5. Search loop как повторяемый алгоритм

Поиск обработчика можно представить как цикл. Начинаем с context места исключения. По RIP ищем FunctionEntry. Если нет записи - применяем leaf path и получаем caller context. Если запись есть - читаем UNWIND_INFO. Если ControlPc в prolog или epilog - применяем особые правила. Если в body и есть handler flags - вызываем language-specific handler. Если handler отказался, вызываем RtlVirtualUnwind и повторяем цикл для caller-а.

Это не исходный код Windows, а mental model. Она нужна, чтобы не потерять порядок: lookup -> interpret function state -> maybe call handler -> virtual unwind -> next frame. Если пропустить один шаг, вся x64 exception model превращается в туман.

Именно этот цикл должен видеть реверсер в отчете. Если finding говорит, что RUNTIME_FUNCTION invalid, значит цикл ломается на lookup/entry stage. Если unwind_parse_fail - ломается на reading recipe stage. Если handler_nonexec_section - проблема уже в handler target stage.

Пятая глава переводит статические структуры в runtime-машину. До этого мы видели поля на диске. Теперь эти поля становятся действиями: lookup, classify prolog/body/epilog, call handler, virtual unwind. Именно здесь становится понятно, почему static findings могут влиять на практический stack walking. Если UnwindInfoAddress сломан, это не просто плохой байт в файле. Это потенциально неверный переход от текущего context к caller context.

При этом важно не приписывать static analyzer-у больше, чем он знает. Он может сказать, что declared metadata выглядит нарушенной или необычной. Но он не всегда может знать, будет ли этот конкретный путь достигнут при runtime execution. Поэтому дальше появляется dynamic validation: не как замена static analysis, а как наблюдение за тем, какие маршруты реально проявились.

Runtime-модель полезна тем, что превращает статические findings в причинные сбои. Например, unwind_rva_oob в отчете - это не просто "адрес за границей". В search loop это означает: когда Windows дошла бы до этого FunctionEntry, она не смогла бы получить корректный UNWIND_INFO по заявленному адресу. Значит, переход к caller context становится невозможным или недостоверным.

RtlLookupFunctionEntry и RtlVirtualUnwind часто воспринимают как API-имена, которые надо выучить. Но для книги важнее их роли. Первый API отвечает на вопрос "какая запись описывает текущий ControlPc". Второй отвечает на вопрос "как из текущего context получить context предыдущей функции". Эти два вопроса повторяются на каждом кадре.

Когда handler вызывается, он не живет отдельно от unwind. Он получает сведения, подготовленные dispatch/unwind machinery. DispatcherContext связывает PE function entry, image base, handler data and control PC. EstablisherFrame связывает handler с конкретным frame. Поэтому handler decision зависит от того, насколько правильно Windows дошла до этого кадра.

Для ручного анализа полезно проигрывать loop вслух: current RIP, module, RVA, function entry, unwind info, prolog/body/epilog, handler flags, decision, virtual unwind, next RIP. Если где-то в этой цепочке появляется "не знаю", это не стыдно. Это место для finding, hypothesis or dynamic validation.

В проекте runtime-модель объясняет, почему static findings имеют разный вес. Поврежденный PE mapping ломает самые ранние шаги. UNWIND_INFO anomaly ломает рецепт. Handler placement влияет на target transfer. Reuse/dominance patterns возникают уже после построения множества связей. Score должен учитывать эту причинную глубину.

5.6. Почему prolog и epilog являются ловушкой для наивного анализа

Наивный анализ говорит: RIP внутри функции, значит можно применять полное UNWIND_INFO. Но если RIP находится в prolog-е, функция еще не успела выполнить все действия пролога. Если применить полный рецепт, можно восстановить слишком много. Если RIP находится в epilog-е, функция уже начала разбирать кадр. Если считать ее обычным body, тоже получится ошибка.

Поэтому x64 unwind различает zones внутри функции. SizeOfProlog и epilog recognition нужны не для красоты, а для корректной интерпретации частично построенного или частично разобранного frame.

Это важно и для ручного reverse engineering. Если stack trace странно обрывается на адресе около начала или конца функции, сначала проверь, не попал ли ControlPc в prolog/epilog zone. Иногда это нормальное объяснение, а не повреждение.

Пятая глава переводит статические структуры в runtime-машину. До этого мы видели поля на диске. Теперь эти поля становятся действиями: lookup, classify prolog/body/epilog, call handler, virtual unwind. Именно здесь становится понятно, почему static findings могут влиять на практический stack walking. Если UnwindInfoAddress сломан, это не просто плохой байт в файле. Это потенциально неверный переход от текущего context к caller context.

При этом важно не приписывать static analyzer-у больше, чем он знает. Он может сказать, что declared metadata выглядит нарушенной или необычной. Но он не всегда может знать, будет ли этот конкретный путь достигнут при runtime execution. Поэтому дальше появляется dynamic validation: не как замена static analysis, а как наблюдение за тем, какие маршруты реально проявились.

Runtime-модель полезна тем, что превращает статические findings в причинные сбои. Например, unwind_rva_oob в отчете - это не просто "адрес за границей". В search loop это означает: когда Windows дошла бы до этого FunctionEntry, она не смогла бы получить корректный UNWIND_INFO по заявленному адресу. Значит, переход к caller context становится невозможным или недостоверным.

RtlLookupFunctionEntry и RtlVirtualUnwind часто воспринимают как API-имена, которые надо выучить. Но для книги важнее их роли. Первый API отвечает на вопрос "какая запись описывает текущий ControlPc". Второй отвечает на вопрос "как из текущего context получить context предыдущей функции". Эти два вопроса повторяются на каждом кадре.

Когда handler вызывается, он не живет отдельно от unwind. Он получает сведения, подготовленные dispatch/unwind machinery. DispatcherContext связывает PE function entry, image base, handler data and control PC. EstablisherFrame связывает handler с конкретным frame. Поэтому handler decision зависит от того, насколько правильно Windows дошла до этого кадра.

Для ручного анализа полезно проигрывать loop вслух: current RIP, module, RVA, function entry, unwind info, prolog/body/epilog, handler flags, decision, virtual unwind, next RIP. Если где-то в этой цепочке появляется "не знаю", это не стыдно. Это место для finding, hypothesis or dynamic validation.

В проекте runtime-модель объясняет, почему static findings имеют разный вес. Поврежденный PE mapping ломает самые ранние шаги. UNWIND_INFO anomaly ломает рецепт. Handler placement влияет на target transfer. Reuse/dominance patterns возникают уже после построения множества связей. Score должен учитывать эту причинную глубину.

5.7. Failure modes: как ломается подъем по стеку

Подъем по стеку может сломаться на нескольких уровнях. PE mapping может быть некорректным, и тогда RVA нельзя надежно перевести. RUNTIME_FUNCTION может иметь invalid range. UnwindInfoAddress может вести за image. UNWIND_INFO может не парситься. Unwind codes могут противоречить прологу. Stack memory может быть повреждена, и return address окажется мусором.

Каждый failure mode дает разную диагностическую ценность. Если таблица функций несортирована, это говорит о качестве карты. Если handler лежит в writable section, это говорит о placement. Если один handler доминирует над сотнями функций, это говорит о statistical pattern. Нельзя сваливать все в один вывод "unwind broken".

Именно поэтому проект вводит findings как отдельные evidence items. Они помогают сказать не "все странно", а "странность находится на таком-то слое маршрута".

Пятая глава переводит статические структуры в runtime-машину. До этого мы видели поля на диске. Теперь эти поля становятся действиями: lookup, classify prolog/body/epilog, call handler, virtual unwind. Именно здесь становится понятно, почему static findings могут влиять на практический stack walking. Если UnwindInfoAddress сломан, это не просто плохой байт в файле. Это потенциально неверный переход от текущего context к caller context.

При этом важно не приписывать static analyzer-у больше, чем он знает. Он может сказать, что declared metadata выглядит нарушенной или необычной. Но он не всегда может знать, будет ли этот конкретный путь достигнут при runtime execution. Поэтому дальше появляется dynamic validation: не как замена static analysis, а как наблюдение за тем, какие маршруты реально проявились.

Runtime-модель полезна тем, что превращает статические findings в причинные сбои. Например, unwind_rva_oob в отчете - это не просто "адрес за границей". В search loop это означает: когда Windows дошла бы до этого FunctionEntry, она не смогла бы получить корректный UNWIND_INFO по заявленному адресу. Значит, переход к caller context становится невозможным или недостоверным.

RtlLookupFunctionEntry и RtlVirtualUnwind часто воспринимают как API-имена, которые надо выучить. Но для книги важнее их роли. Первый API отвечает на вопрос "какая запись описывает текущий ControlPc". Второй отвечает на вопрос "как из текущего context получить context предыдущей функции". Эти два вопроса повторяются на каждом кадре.

Когда handler вызывается, он не живет отдельно от unwind. Он получает сведения, подготовленные dispatch/unwind machinery. DispatcherContext связывает PE function entry, image base, handler data and control PC. EstablisherFrame связывает handler с конкретным frame. Поэтому handler decision зависит от того, насколько правильно Windows дошла до этого кадра.

Для ручного анализа полезно проигрывать loop вслух: current RIP, module, RVA, function entry, unwind info, prolog/body/epilog, handler flags, decision, virtual unwind, next RIP. Если где-то в этой цепочке появляется "не знаю", это не стыдно. Это место для finding, hypothesis or dynamic validation.

В проекте runtime-модель объясняет, почему static findings имеют разный вес. Поврежденный PE mapping ломает самые ранние шаги. UNWIND_INFO anomaly ломает рецепт. Handler placement влияет на target transfer. Reuse/dominance patterns возникают уже после построения множества связей. Score должен учитывать эту причинную глубину.

5.8. Мини-проход от C к B к A

Пусть исключение случилось в функции C. RtlLookupFunctionEntry нашла запись C. UNWIND_INFO C говорит: сохранен RBX и выделено 0x60 байт стека. RtlVirtualUnwind восстанавливает RBX, вычисляет старый RSP, берет return address и получает context B. Теперь ControlPc находится уже в B. Цикл повторяется.

Если B имеет handler flag, Windows вызывает language-specific handler для кадра B. Handler смотрит ExceptionRecord, EstablisherFrame, DispatcherContext и решает, принимать ли исключение. Если не принимает, RtlVirtualUnwind строит context A. Если A принимает, search phase заканчивается, и начинается unwind phase к выбранному кадру.

Эта сцена связывает все предыдущие термины: ExceptionRecord, ContextRecord, RIP, RSP, RUNTIME_FUNCTION, UNWIND_INFO, handler flags, EstablisherFrame, DispatcherContext, RtlVirtualUnwind. Теперь это не словарь, а один живой маршрут.

Пятая глава переводит статические структуры в runtime-машину. До этого мы видели поля на диске. Теперь эти поля становятся действиями: lookup, classify prolog/body/epilog, call handler, virtual unwind. Именно здесь становится понятно, почему static findings могут влиять на практический stack walking. Если UnwindInfoAddress сломан, это не просто плохой байт в файле. Это потенциально неверный переход от текущего context к caller context.

При этом важно не приписывать static analyzer-у больше, чем он знает. Он может сказать, что declared metadata выглядит нарушенной или необычной. Но он не всегда может знать, будет ли этот конкретный путь достигнут при runtime execution. Поэтому дальше появляется dynamic validation: не как замена static analysis, а как наблюдение за тем, какие маршруты реально проявились.

Runtime-модель полезна тем, что превращает статические findings в причинные сбои. Например, unwind_rva_oob в отчете - это не просто "адрес за границей". В search loop это означает: когда Windows дошла бы до этого FunctionEntry, она не смогла бы получить корректный UNWIND_INFO по заявленному адресу. Значит, переход к caller context становится невозможным или недостоверным.

RtlLookupFunctionEntry и RtlVirtualUnwind часто воспринимают как API-имена, которые надо выучить. Но для книги важнее их роли. Первый API отвечает на вопрос "какая запись описывает текущий ControlPc". Второй отвечает на вопрос "как из текущего context получить context предыдущей функции". Эти два вопроса повторяются на каждом кадре.

Когда handler вызывается, он не живет отдельно от unwind. Он получает сведения, подготовленные dispatch/unwind machinery. DispatcherContext связывает PE function entry, image base, handler data and control PC. EstablisherFrame связывает handler с конкретным frame. Поэтому handler decision зависит от того, насколько правильно Windows дошла до этого кадра.

Для ручного анализа полезно проигрывать loop вслух: current RIP, module, RVA, function entry, unwind info, prolog/body/epilog, handler flags, decision, virtual unwind, next RIP. Если где-то в этой цепочке появляется "не знаю", это не стыдно. Это место для finding, hypothesis or dynamic validation.

В проекте runtime-модель объясняет, почему static findings имеют разный вес. Поврежденный PE mapping ломает самые ранние шаги. UNWIND_INFO anomaly ломает рецепт. Handler placement влияет на target transfer. Reuse/dominance patterns возникают уже после построения множества связей. Score должен учитывать эту причинную глубину.

Сквозной пример главы 5: как это выглядит в живом анализе

Теперь проиграем runtime loop. Current context находится в функции C. RtlLookupFunctionEntry нашел запись C. UNWIND_INFO говорит, как C построила frame. RtlVirtualUnwind применяет этот рецепт и получает context B. Теперь RIP уже указывает не на C, а на адрес возврата внутри B. Система снова вызывает lookup, но уже для B.

Если у B есть UNW_FLAG_EHANDLER, Windows может вызвать language-specific handler. Handler получает не только exception code, но и DispatcherContext, EstablisherFrame and ContextRecord. Он

решает, подходит ли этот frame. Если нет, снова virtual unwind - теперь к A. Если да, search phase завершает поиск цели.

Если на шаге C UnwindInfoAddress был сломан, до B можно не дойти. Если на шаге B handler target в non-executable section, это уже другой слой проблемы. Если у сотен функций один и тот же handler, появляется handler_dominance. Один runtime loop объясняет разные classes findings.

Сильный реверсер умеет связать отчет с этим loop. Он смотрит на finding и спрашивает: какой шаг маршрута он затрагивает? lookup? RVA mapping? recipe parsing? handler dispatch? statistics after aggregation? Так отчет перестает быть списком красных слов и становится картой расследования.

```
C context
 -> lookup C
 -> read C.UNWIND_INFO
 -> virtual unwind
 -> B context
 -> B has handler flag?
 -> call language-specific handler
 -> continue search or handled
 -> if continue: virtual unwind to A
```

Итог главы 5: что должно остаться в голове

RtlLookupFunctionEntry и RtlVirtualUnwind превращают статические таблицы в runtime-переходы между кадрами. Search loop идет от текущего context к caller context до тех пор, пока handler не примет исключение или маршрут не закончится.

Глава 6. Фазы обработки исключения и границы static/dynamic анализа

Последняя фундаментальная глава соединяет поиск обработчика, фазы обработки, debugger notifications, VEH, frame-based SEH, language-specific handlers и dynamic validation. Здесь важно не смешать разные уровни: static metadata, runtime dispatch и наблюдения динамического коллектора отвечают на разные вопросы.

6.1. First chance и second chance: исключение сначала получает шанс быть обработанным

Когда исключение возникает, оно не обязано сразу завершать процесс. Windows может дать first-chance notification debugger-у, затем запустить обычный dispatch route. Если handler найден и обработал событие, программа может продолжить по правилам обработки. Если никто не обработал, exception becomes unhandled, и debugger может получить second-chance notification перед завершением процесса.

First chance не означает "ошибка уже страшная". Это первый шанс обработать событие. Debugger видит такие события постоянно: breakpoints, single-step, некоторые runtime exceptions. Second chance обычно серьезнее: нормальные handler-ы не приняли исключение, и процесс идет к аварийному завершению или последним механизмам.

Для reverse engineering это помогает не путать отладочный шум и реальную необработанную аварию. Видеть exception в debugger-е - еще не значит, что программа падает. Нужно понять, обработано ли событие дальше.

Шестая глава закрывает фундамент тем, что разводит уровни обработки. Debugger notifications, VEH, frame-based SEH, language-specific handlers, C++ runtime, unhandled exception и DynamicReport - это не один мешок. Они находятся в разных местах маршрута. Когда эти уровни смешивают, появляются ложные выводы: например, static absence of .pdata путают с runtime impossibility, a first-chance exception путают с crash.

Для защиты проекта это особенно важно. Нужно говорить не "мы находим все исключения", а "мы строим и проверяем статически объявленную Windows x64 exception/unwind model PE-файла, а runtime validation дает отдельный слой наблюдений". Такая формулировка честная и сильная: она не обещает невозможного и точно показывает вклад проекта.

Финальная фундаментальная глава нужна, чтобы не сделать ложный вывод: "раз мы изучили .pdata, значит поняли всю обработку исключений". Нет. .pdata/.xdata дают frame-based unwind route. Но вокруг него существуют debugger notifications, VEH, language-specific runtime, C++ EH semantics, top-level filters and dynamic behavior. Они связаны, но не совпадают.

First-chance exception в debugger-е может быть нормальной частью работы. VEH может перехватить событие до frame-based search. __C_specific_handler может быть не пользовательским handler-ом, a runtime helper-ом. C++ catch может требовать type matching and destructor unwinding. Unhandled exception означает, что маршрут не нашел принятого решения. Каждый уровень добавляет свою семантику.

Для проекта важно сохранять честные claims. Static analyzer не запускает программу, не видит все runtime registrations and generated code, не доказывает реальную вредоносность. Он проверяет declared metadata. Dynamic validation наблюдает runtime. Findings дают evidence. Assessment связывает evidence с осторожными выводами. Это зрелая архитектура, а не слабость.

Когда пишешь защитный ответ, лучше говорить так: "Мы анализируем слой Windows x64 PE exception/unwind metadata и выделяем статически проверяемые нарушения и аномалии. Для runtime-слоя предусмотрена dynamic validation и отдельный DynamicReport". Такая формулировка одновременно технически точна и понятна комиссии.

После шестой главы читатель должен быть готов не зубрить поля, а расследовать маршрут. Вопросы становятся практическими: где карта функций, читается ли рецепт, соответствует ли он коду, куда ведет handler, что подтверждает dynamic collector, какие findings формируют assessment. Это и есть мост к следующим главам.

6.2. VEH, frame-based SEH и top-level handling как разные кольца

Vectored Exception Handling - это механизм обработчиков, зарегистрированных на уровне процесса. Они не являются обычными кадрами стека. Frame-based SEH связан с кадрами функций и идет по call stack через unwind route. Top-level handling - поздний уровень, когда normal dispatch не нашел подходящего обработчика.

Эти слои нельзя смешивать. VEH может увидеть исключение раньше frame-based поиска. Frame-based handler связан с конкретным frame и language-specific data. Top-level filter не заменяет нормальную обработку внутри функции. Каждый слой существует для своей роли.

Для анализа это значит: если static analyzer изучает PE .pdata/.xdata, он описывает frame-based x64 unwind metadata. Он не видит автоматически все VEH registrations, потому что они создаются runtime API. Это еще одна граница static analysis.

Шестая глава закрывает фундамент тем, что разводит уровни обработки. Debugger notifications, VEH, frame-based SEH, language-specific handlers, C++ runtime, unhandled exception и DynamicReport - это не один мешок. Они находятся в разных местах маршрута. Когда эти уровни смешивают, появляются ложные выводы: например, static absence of .pdata путают с runtime impossibility, а first-chance exception путают с crash.

Для защиты проекта это особенно важно. Нужно говорить не "мы находим все исключения", а "мы строим и проверяем статически объявленную Windows x64 exception/unwind model PE-файла, а runtime validation дает отдельный слой наблюдений". Такая формулировка честная и сильная: она не обещает невозможного и точно показывает вклад проекта.

Финальная фундаментальная глава нужна, чтобы не сделать ложный вывод: "раз мы изучили .pdata, значит поняли всю обработку исключений". Нет. .pdata/.xdata дают frame-based unwind route. Но вокруг него существуют debugger notifications, VEH, language-specific runtime, C++ EH semantics, top-level filters and dynamic behavior. Они связаны, но не совпадают.

First-chance exception в debugger-е может быть нормальной частью работы. VEH может перехватить событие до frame-based search. __C_specific_handler может быть не пользовательским handler-ом, а runtime helper-ом. C++ catch может требовать type matching and destructor unwinding. Unhandled exception означает, что маршрут не нашел принятого решения. Каждый уровень добавляет свою семантику.

Для проекта важно сохранять честные claims. Static analyzer не запускает программу, не видит все runtime registrations and generated code, не доказывает реальную вредоносность. Он проверяет declared metadata. Dynamic validation наблюдает runtime. Findings дают evidence. Assessment связывает evidence с осторожными выводами. Это зрелая архитектура, а не слабость.

Когда пишешь защитный ответ, лучше говорить так: "Мы анализируем слой Windows x64 PE exception/unwind metadata и выделяем статически проверяемые нарушения и аномалии. Для runtime-слоя предусмотрена dynamic validation и отдельный DynamicReport". Такая формулировка одновременно технически точна и понятна комиссии.

После шестой главы читатель должен быть готов не зубрить поля, а расследовать маршрут. Вопросы становятся практическими: где карта функций, читается ли рецепт, соответствует ли он коду, куда ведет handler, что подтверждает dynamic collector, какие findings формируют assessment. Это и есть мост к следующим главам.

6.3. __C_specific_handler как мост от Windows к MSVC SEH

Windows дает общий dispatch/unwind mechanism, но не обязана понимать исходный код __try/__except как человек. MSVC использует language-specific handler, например __C_specific_handler, который умеет интерпретировать данные, сгенерированные компилятором для C/SEH constructs.

Когда UNWIND_INFO указывает на такой handler, Windows передает ему ExceptionRecord, EstablisherFrame, ContextRecord и DispatcherContext. Handler уже внутри своей логики смотрит scope tables, filters и handlers, чтобы решить, подходит ли текущий кадр.

Поэтому адрес handler-а в UNWIND_INFO не всегда означает пользовательскую функцию обработки. Часто это runtime helper, который дальше читает language-specific metadata. Для реверсера это критично: увидеть __C_specific_handler - это начало анализа scope data, а не конец.

Шестая глава закрывает фундамент тем, что разводит уровни обработки. Debugger notifications, VEH, frame-based SEH, language-specific handlers, C++ runtime, unhandled exception и DynamicReport - это не один мешок. Они находятся в разных местах маршрута. Когда эти уровни смешивают, появляются ложные выводы: например, static absence of .pdata путают с runtime impossibility, а first-chance exception путают с crash.

Для защиты проекта это особенно важно. Нужно говорить не "мы находим все исключения", а "мы строим и проверяем статически объявленную Windows x64 exception/unwind model PE-файла, а runtime validation дает отдельный слой наблюдений". Такая формулировка честная и сильная: она не обещает невозможного и точно показывает вклад проекта.

Финальная фундаментальная глава нужна, чтобы не сделать ложный вывод: "раз мы изучили .pdata, значит поняли всю обработку исключений". Нет. .pdata/.xdata дают frame-based unwind route. Но вокруг него существуют debugger notifications, VEH, language-specific runtime, C++ EH semantics, top-level filters and dynamic behavior. Они связаны, но не совпадают.

First-chance exception в debugger-е может быть нормальной частью работы. VEH может перехватить событие до frame-based search. __C_specific_handler может быть не пользовательским handler-ом, а runtime helper-ом. C++ catch может требовать type matching and destructor unwinding. Unhandled exception означает, что маршрут не нашел принятого решения. Каждый уровень добавляет свою семантику.

Для проекта важно сохранять честные claims. Static analyzer не запускает программу, не видит все runtime registrations and generated code, не доказывает реальную вредоносность. Он проверяет declared metadata. Dynamic validation наблюдает runtime. Findings дают evidence. Assessment связывает evidence с осторожными выводами. Это зрелая архитектура, а не слабость.

Когда пишешь защитный ответ, лучше говорить так: "Мы анализируем слой Windows x64 PE exception/unwind metadata и выделяем статически проверяемые нарушения и аномалии. Для runtime-слоя предусмотрена dynamic validation и отдельный DynamicReport". Такая формулировка одновременно технически точна и понятна комиссии.

После шестой главы читатель должен быть готов не зубрить поля, а расследовать маршрут. Вопросы становятся практическими: где карта функций, читается ли рецепт, соответствует ли он коду, куда ведет handler, что подтверждает dynamic collector, какие findings формируют assessment. Это и есть мост к следующим главам.

6.4. C++ exceptions: почему throw/catch сверху не отменяет Windows-механику снизу

C++ throw/catch выглядит как языковая конструкция. Но на Windows x64 она строится поверх системной инфраструктуры исключений и unwind. Runtime языка должен найти подходящий catch, выполнить деструкторы для объектов, корректно раскрутить стек и передать управление обработчику.

Это не значит, что C++ EH полностью равен SEH. У языка C++ есть своя semantics: типы исключений, matching catch, object lifetime, destructors. Но физический путь по кадрам, необходимость unwind metadata и взаимодействие с Windows exception machinery остаются частью нижнего слоя.

Поэтому в книге мы сначала изучаем Windows x64 unwind, а уже потом можно углубляться в C++ EH internals. Без базовой модели PE/.pdata/.xdata C++ EH будет казаться магией runtime-а.

Шестая глава закрывает фундамент тем, что разводит уровни обработки. Debugger notifications, VEH, frame-based SEH, language-specific handlers, C++ runtime, unhandled exception и DynamicReport - это не один мешок. Они находятся в разных местах маршрута. Когда эти уровни смешивают, появляются ложные выводы: например, static absence of .pdata путают с runtime impossibility, а first-chance exception путают с crash.

Для защиты проекта это особенно важно. Нужно говорить не "мы находим все исключения", а "мы строим и проверяем статически объявленную Windows x64 exception/unwind model PE-файла, а runtime validation дает отдельный слой наблюдений". Такая формулировка честная и сильная: она не обещает невозможного и точно показывает вклад проекта.

Финальная фундаментальная глава нужна, чтобы не сделать ложный вывод: "раз мы изучили .pdata, значит поняли всю обработку исключений". Нет. .pdata/.xdata дают frame-based unwind route. Но вокруг него существуют debugger notifications, VEH, language-specific runtime, C++ EH semantics, top-level filters and dynamic behavior. Они связаны, но не совпадают.

First-chance exception в debugger-е может быть нормальной частью работы. VEH может перехватить событие до frame-based search. __C_specific_handler может быть не пользовательским handler-ом, а runtime helper-ом. C++ catch может требовать type matching and destructor unwinding. Unhandled exception означает, что маршрут не нашел принятого решения. Каждый уровень добавляет свою семантику.

Для проекта важно сохранять честные claims. Static analyzer не запускает программу, не видит все runtime registrations and generated code, не доказывает реальную вредоносность. Он проверяет declared metadata. Dynamic validation наблюдает runtime. Findings дают evidence. Assessment связывает evidence с осторожными выводами. Это зрелая архитектура, а не слабость.

Когда пишешь защитный ответ, лучше говорить так: "Мы анализируем слой Windows x64 PE exception/unwind metadata и выделяем статически проверяемые нарушения и аномалии. Для runtime-слоя предусмотрена dynamic validation и отдельный DynamicReport". Такая формулировка одновременно технически точна и понятна комиссии.

После шестой главы читатель должен быть готов не зубрить поля, а расследовать маршрут. Вопросы становятся практическими: где карта функций, читается ли рецепт, соответствует ли он коду, куда ведет handler, что подтверждает dynamic collector, какие findings формируют assessment. Это и есть мост к следующим главам.

6.5. Unhandled exception: когда маршрут не нашел решения

Unhandled exception - это не просто "произошло исключение". Это ситуация, когда событие прошло через доступные стадии обработки и не было принято подходящим handler-ом. Тогда Windows переходит к поздним механизмам: top-level filter, debugger second chance, Windows Error Reporting, dump generation, process termination.

В crash dump analysis важно понимать, на какой стадии мы находимся. Дамп после unhandled exception показывает последствия не только исходной ошибки, но и failed route обработки. Stack trace, exception record и context record помогают восстановить путь.

Если unwind metadata повреждены, дамп может показывать неполный или ложный call stack. Поэтому качество .pdata/.xdata влияет не только на теоретическую обработку исключений, но и на практическую диагностику.

Шестая глава закрывает фундамент тем, что разводит уровни обработки. Debugger notifications, VEH, frame-based SEH, language-specific handlers, C++ runtime, unhandled exception и DynamicReport - это не один мешок. Они находятся в разных местах маршрута. Когда эти уровни смешивают, появляются ложные выводы: например, static absence of .pdata путают с runtime impossibility, a first-chance exception путают с crash.

Для защиты проекта это особенно важно. Нужно говорить не "мы находим все исключения", а "мы строим и проверяем статически объявленную Windows x64 exception/unwind model PE-файла, a runtime validation дает отдельный слой наблюдений". Такая формулировка честная и сильная: она не обещает невозможного и точно показывает вклад проекта.

Финальная фундаментальная глава нужна, чтобы не сделать ложный вывод: "раз мы изучили .pdata, значит поняли всю обработку исключений". Нет. .pdata/.xdata дают frame-based unwind route. Но вокруг него существуют debugger notifications, VEH, language-specific runtime, C++ EH semantics, top-level filters and dynamic behavior. Они связаны, но не совпадают.

First-chance exception в debugger-е может быть нормальной частью работы. VEH может перехватить событие до frame-based search. __C_specific_handler может быть не пользовательским handler-ом, a runtime helper-ом. C++ catch может требовать type matching and destructor unwinding. Unhandled exception означает, что маршрут не нашел принятого решения. Каждый уровень добавляет свою семантику.

Для проекта важно сохранять честные claims. Static analyzer не запускает программу, не видит все runtime registrations and generated code, не доказывает реальную вредоносность. Он проверяет declared metadata. Dynamic validation наблюдает runtime. Findings дают evidence. Assessment связывает evidence с осторожными выводами. Это зрелая архитектура, а не слабость.

Когда пишешь защитный ответ, лучше говорить так: "Мы анализируем слой Windows x64 PE exception/unwind metadata и выделяем статически проверяемые нарушения и аномалии. Для runtime-слоя предусмотрена dynamic validation и отдельный DynamicReport". Такая формулировка одновременно технически точна и понятна комиссии.

После шестой главы читатель должен быть готов не зубрить поля, а расследовать маршрут. Вопросы становятся практическими: где карта функций, читается ли рецепт, соответствует ли он коду, куда ведет handler, что подтверждает dynamic collector, какие findings формируют assessment. Это и есть мост к следующим главам.

6.6. Dynamic validation: что статическая книга не может узнать без запуска

Static analysis отвечает на вопрос: что PE-файл объявляет? Dynamic validation отвечает на вопрос: что реально проявилось при выполнении? Это разные слои. Static analyzer видит headers, sections, Exception Directory, RUNTIME_FUNCTION, UNWIND_INFO, placement and static patterns. Dynamic collector может видеть runtime events, registered handlers, dynamic function tables, actual exceptions, observed handler route.

Смешивать эти слои опасно. Если static analyzer не видит dynamic function table в файле, нельзя автоматически сказать, что runtime никогда ее не зарегистрирует. Если dynamic run не вызвал конкретное исключение, нельзя сказать, что static anomaly безопасна. Они отвечают на разные вопросы и усиливают друг друга через correlation.

DynamicReport должен дополнять static report, а не подменять его. В хорошей системе static findings дают declared evidence, dynamic validation дает observed evidence, a final assessment честно разделяет источники.

Шестая глава закрывает фундамент тем, что разводит уровни обработки. Debugger notifications, VEH, frame-based SEH, language-specific handlers, C++ runtime, unhandled exception и DynamicReport - это не один мешок. Они находятся в разных местах маршрута. Когда эти уровни смешивают, появляются ложные выводы: например, static absence of .pdata путают с runtime impossibility, a

first-chance exception путают с crash.

Для защиты проекта это особенно важно. Нужно говорить не "мы находим все исключения", а "мы строим и проверяем статически объявленную Windows x64 exception/unwind model PE-файла, а runtime validation дает отдельный слой наблюдений". Такая формулировка честная и сильная: она не обещает невозможного и точно показывает вклад проекта.

Финальная фундаментальная глава нужна, чтобы не сделать ложный вывод: "раз мы изучили .pdata, значит поняли всю обработку исключений". Нет. .pdata/.xdata дают frame-based unwind route. Но вокруг него существуют debugger notifications, VEH, language-specific runtime, C++ EH semantics, top-level filters and dynamic behavior. Они связаны, но не совпадают.

First-chance exception в debugger-е может быть нормальной частью работы. VEH может перехватить событие до frame-based search. __C_specific_handler может быть не пользовательским handler-ом, а runtime helper-ом. C++ catch может требовать type matching and destructor unwinding. Unhandled exception означает, что маршрут не нашел принятого решения. Каждый уровень добавляет свою семантику.

Для проекта важно сохранять честные claims. Static analyzer не запускает программу, не видит все runtime registrations and generated code, не доказывает реальную вредоносность. Он проверяет declared metadata. Dynamic validation наблюдает runtime. Findings дают evidence. Assessment связывает evidence с осторожными выводами. Это зрелая архитектура, а не слабость.

Когда пишешь защитный ответ, лучше говорить так: "Мы анализируем слой Windows x64 PE exception/unwind metadata и выделяем статически проверяемые нарушения и аномалии. Для runtime-слоя предусмотрена dynamic validation и отдельный DynamicReport". Такая формулировка одновременно технически точна и понятна комиссии.

После шестой главы читатель должен быть готов не зубрить поля, а расследовать маршрут. Вопросы становятся практическими: где карта функций, читается ли рецепт, соответствует ли он коду, куда ведет handler, что подтверждает dynamic collector, какие findings формируют assessment. Это и есть мост к следующим главам.

6.7. Как первые шесть глав собираются в одну модель

Теперь можно собрать маршрут без словаря. Программа выполняет instruction. Исключение прерывает ordinary control flow. Windows получает ExceptionRecord и ContextRecord. По RIP она определяет module и RVA. Через PE Data Directory она знает, где Exception Directory. Через Exception Directory она приходит к .pdata. В .pdata ищет RUNTIME_FUNCTION. RUNTIME_FUNCTION ведет к UNWIND_INFO в .xdata. UNWIND_INFO объясняет, как восстановить caller context. RtlVirtualUnwind применяет этот рецепт. Search phase идет по кадрам, handler-ы принимают или отклоняют событие. Если цель найдена, unwind phase приводит стек к обработчику.

Это не набор терминов. Это единая дорога:

```
exception event
 -> context
 -> module/RVA
 -> Exception Directory
 -> RUNTIME_FUNCTION table
 -> UNWIND_INFO recipe
 -> RtlVirtualUnwind
 -> caller context
 -> language-specific handler decision
 -> search phase or unwind phase
```

Шестая глава закрывает фундамент тем, что разводит уровни обработки. Debugger notifications, VEH, frame-based SEH, language-specific handlers, C++ runtime, unhandled exception и DynamicReport - это не один мешок. Они находятся в разных местах маршрута. Когда эти уровни смешивают, появляются ложные выводы: например, static absence of .pdata путают с runtime impossibility, а first-chance exception путают с crash.

Для защиты проекта это особенно важно. Нужно говорить не "мы находим все исключения", а "мы строим и проверяем статически объявленную Windows x64 exception/unwind model PE-файла, а runtime validation дает отдельный слой наблюдений". Такая формулировка честная и сильная: она не обещает невозможного и точно показывает вклад проекта.

Финальная фундаментальная глава нужна, чтобы не сделать ложный вывод: "раз мы изучили .pdata, значит поняли всю обработку исключений". Нет. .pdata/.xdata дают frame-based unwind route. Но вокруг него существуют debugger notifications, VEH, language-specific runtime, C++ EH semantics, top-level filters and dynamic behavior. Они связаны, но не совпадают.

First-chance exception в debugger-е может быть нормальной частью работы. VEH может перехватить событие до frame-based search. __C_specific_handler может быть не пользовательским handler-ом, а runtime helper-ом. C++ catch может требовать type matching and destructor unwinding. Unhandled exception означает, что маршрут не нашел принятого решения. Каждый уровень добавляет свою семантику.

Для проекта важно сохранять честные claims. Static analyzer не запускает программу, не видит все runtime registrations and generated code, не доказывает реальную вредоносность. Он проверяет declared metadata. Dynamic validation наблюдает runtime. Findings дают evidence. Assessment связывает evidence с осторожными выводами. Это зрелая архитектура, а не слабость.

Когда пишешь защитный ответ, лучше говорить так: "Мы анализируем слой Windows x64 PE exception/unwind metadata и выделяем статически проверяемые нарушения и аномалии. Для runtime-слоя предусмотрена dynamic validation и отдельный DynamicReport". Такая формулировка одновременно технически точна и понятна комиссии.

После шестой главы читатель должен быть готов не зубрить поля, а расследовать маршрут. Вопросы становятся практическими: где карта функций, читается ли рецепт, соответствует ли он коду, куда ведет handler, что подтверждает dynamic collector, какие findings формируют assessment. Это и есть мост к следующим главам.

6.8. Почему дальше можно переходить к ручной модели анализа

Первые шесть глав нужны не как энциклопедия. Они строят внутренний симулятор. Теперь, увидев PE-файл, ты должен мысленно спросить: где его Exception Directory? Какие RUNTIME_FUNCTION entries объявлены? Куда ведут UnwindInfoAddress? Можно ли разобрать UNWIND_INFO? Соответствует ли metadata коду? Что будет делать RtlVirtualUnwind? Где handler? На каком слое может появиться finding?

Именно после этого имеет смысл читать практические главы. Без такой модели инструменты будут казаться магическими. С ней Ghidra, PE viewer, WinDbg, JSON report и DynamicReport становятся разными окнами в одну систему.

Дальше книга переходит к ручной модели анализа: как человек читает эти структуры, как проверяет подозрительные места, как отличает format error от anomaly, а anomaly от реального high-value evidence.

Шестая глава закрывает фундамент тем, что разводит уровни обработки. Debugger notifications, VEH, frame-based SEH, language-specific handlers, C++ runtime, unhandled exception и DynamicReport - это не один мешок. Они находятся в разных местах маршрута. Когда эти уровни смешивают, появляются ложные выводы: например, static absence of .pdata путают с runtime impossibility, а first-chance exception путают с crash.

Для защиты проекта это особенно важно. Нужно говорить не "мы находим все исключения", а "мы строим и проверяем статически объявленную Windows x64 exception/unwind model PE-файла, а runtime validation дает отдельный слой наблюдений". Такая формулировка честная и сильная: она не обещает невозможного и точно показывает вклад проекта.

Финальная фундаментальная глава нужна, чтобы не сделать ложный вывод: "раз мы изучили .pdata, значит поняли всю обработку исключений". Нет. .pdata/.xdata дают frame-based unwind route.

Но вокруг него существуют debugger notifications, VEH, language-specific runtime, C++ EH semantics, top-level filters and dynamic behavior. Они связаны, но не совпадают.

First-chance exception в debugger-е может быть нормальной частью работы. VEH может перехватить событие до frame-based search. __C_specific_handler может быть не пользовательским handler-ом, а runtime helper-ом. C++ catch может требовать type matching and destructor unwinding. Unhandled exception означает, что маршрут не нашел принятого решения. Каждый уровень добавляет свою семантику.

Для проекта важно сохранять честные claims. Static analyzer не запускает программу, не видит все runtime registrations and generated code, не доказывает реальную вредоносность. Он проверяет declared metadata. Dynamic validation наблюдает runtime. Findings дают evidence. Assessment связывает evidence с осторожными выводами. Это зрелая архитектура, а не слабость.

Когда пишешь защитный ответ, лучше говорить так: "Мы анализируем слой Windows x64 PE exception/unwind metadata и выделяем статически проверяемые нарушения и аномалии. Для runtime-слоя предусмотрена dynamic validation и отдельный DynamicReport". Такая формулировка одновременно технически точна и понятна комиссии.

После шестой главы читатель должен быть готов не зубрить поля, а расследовать маршрут. Вопросы становятся практическими: где карта функций, читается ли рецепт, соответствует ли он коду, куда ведет handler, что подтверждает dynamic collector, какие findings формируют assessment. Это и есть мост к следующим главам.

Сквозной пример главы 6: как это выглядит в живом анализе

Финальный пример соединяет static and dynamic. Static analyzer читает PE и говорит: в файле Exception Directory есть 120 RUNTIME_FUNCTION entries; одна запись имеет invalid range; 30 функций используют один и тот же UNWIND_INFO; один handler target доминирует над большой частью functions; часть metadata лежит в writable section. Это declared model.

Dynamic collector запускает controlled scenario и наблюдает runtime: какие исключения реально возникли, какие handlers были вызваны, появились ли dynamic function tables, совпадает ли observed behavior с declared model. Если dynamic report подтверждает, что unusual handler действительно участвует в маршруте, confidence растет. Если static finding не проявился в данном запуске, он не исчезает, но assessment должен честно указать границу наблюдения.

Так проект избегает двух крайностей. Первая крайность - только static: сделать слишком сильный вывод без runtime evidence. Вторая - только dynamic: не увидеть опасную metadata-аномалию, потому что тестовый запуск не прошел по нужному path. Правильная архитектура разделяет источники и потом коррелирует их.

Для книги это означает: первые шесть глав не заканчиваются теорией ради теории. Они дают язык, на котором можно объяснить, почему 27 static findings вообще имеют смысл и почему dynamic validation является отдельным, а не конкурирующим слоем.

```
Static report:
  declared PE/unwind metadata
  findings and assessment

DynamicReport:
  observed runtime behavior
  handler events / dynamic tables / exceptions

Correlation:
  static evidence + dynamic evidence
  -> stronger and more honest assessment
```

Итог главы 6: что должно остаться в голове

Exception handling состоит из нескольких уровней: debugger notifications, VEH, frame-based search, language-specific handlers, unwind phase, unhandled/top-level processing. Static analysis описывает declared PE/unwind model, dynamic validation наблюдает runtime behavior. Эти слои нужно связывать, но не смешивать.

Техническая база переписанных глав 1-6

Переписанные главы 1-6 опираются на официальную документацию Microsoft по PE Format, x64 exception handling, x64 calling convention, x64 prolog/epilog and unwind data, а также на терминологию проекта EH/Unwind Analyzer. Текст написан заново в учебной подаче: вместо набора определений используется один сквозной маршрут от exception event к static/dynamic analysis.

Часть I-B

Полный подробный фундамент глав 1-6 без сокращений

Ниже сохранен полный старый материал v87 по первым шести главам. Он оставлен не как устаревший слой, а как глубокий подробный разбор каждого термина и механизма.

Читать его теперь проще: связная версия выше уже дала общую карту, а этот блок раскрывает детали.

Глава 1. База: что вообще происходит при исключении

Первая глава строит фундамент для всей книги. Нельзя по-настоящему понимать `.pdata`, `RUNTIME_FUNCTION`, `.xdata`, `UNWIND_INFO`, `RtlVirtualUnwind` и SEH на x64, если исключение в голове остается чем-то вроде “программа упала”. В Windows исключение - это событие управления, привязанное к конкретному потоку, конкретному машинному контексту, конкретному адресу инструкции и конкретной цепочке вызовов. Поэтому начинать нужно не с таблиц, а с того, что вообще происходит с выполнением программы.

1.1. Что такое исключение в Windows вообще

Когда человек впервые слышит слово “исключение”, он почти всегда понимает его слишком узко. В голове возникает картина: программа сломалась, появилось окно ошибки, процесс закрылся. В бытовом смысле это понятно. Пользователь видит только финал: приложение перестало работать. Но для reverse engineering такой взгляд почти бесполезен, потому что он пропускает самое важное - путь, по которому система дошла до этого финала.

В Windows исключение - это не просто “ошибка”. Исключение - это формализованное событие выполнения, при котором обычный ход инструкций прерывается, а управление передается специальному механизму обработки. Это событие имеет причину, место возникновения, машинный контекст, код, дополнительные параметры и маршрут доставки. Оно может закончиться аварийным завершением процесса, а может быть перехвачено обработчиком, обработано runtime-библиотекой, передано отладчику, преобразовано в исключение языка C++ или вообще использовано как нормальная часть внутренней логики.

То есть исключение - это не обязательно смерть программы. Исключение - это момент, когда Windows говорит: обычное выполнение больше нельзя или не нужно продолжать напрямую; теперь решение принимает механизм обработки исключений.

Чтобы почувствовать разницу, представь простую инструкцию: программа хочет прочитать значение по адресу. В исходном коде это может выглядеть невинно:

```
int value = *ptr;
```

Пока ты смотришь на C-код, все кажется человеческим: есть переменная `ptr`, есть разыменование, есть переменная `value`. Но процессор не знает слов “переменная”, “указатель” и “разыменование” в том виде, в каком их видит программист. На машинном уровне есть инструкция, которая берет число, трактует его как адрес и пытается прочитать память по этому адресу. Если память существует, страница доступна, права позволяют чтение - инструкция завершится, значение будет загружено. Если адрес мусорный, страница не сопоставлена или права запрещают доступ - инструкция не может завершиться обычным образом.

Вот это “не может завершиться обычным образом” и является ключом. Процессор не может сделать вид, что все хорошо. Он не может сам придумать значение. Он не может просто пропустить инструкцию, потому что тогда программа продолжит работу с ложным состоянием. Он сообщает операционной системе о проблеме, а Windows оформляет это как исключение пользовательского режима. Дальше начинается уже не обычное выполнение твоей функции, а системный маршрут обработки исключения.

Для reverse engineering это принципиально. Если ты видишь `0xC0000005`, то нельзя ограничиться мыслью “ну, access violation”. Нужно раскрывать событие. Какая инструкция вызвала нарушение? По какому адресу она пыталась обратиться? Это было чтение, запись или выполнение? Какое значение было в регистре, из которого взялся адрес? Этот регистр пришел из аргумента функции, из поля структуры, из кучи, из глобальной переменной, из результата предыдущего вызова? Где был RIP? Где был RSP? Есть ли для этой функции unwind-метаданные? Можно ли восстановить caller? Где обработчик? Почему он сработал или не сработал?

Исключение - это не точка. Это дверь в механизм.

Почему “исключение” и “ошибка” - не одно и то же

Слово “ошибка” описывает смысл. Слово “исключение” описывает механизм.

Ошибка - это когда что-то не так с точки зрения программы, программиста или пользователя. Указатель не должен был быть нулевым. Буфер не должен был быть переполнен. Делитель не должен был быть равен нулю. Файл должен был существовать. Индекс должен был попадать в границы массива. Это смысловой уровень.

Исключение - это способ системы перевести необычную ситуацию в управляемое событие. У него есть код, запись исключения, контекст потока, правила доставки, обработчики и фазы обработки. Это машинно-системный уровень.

Эти уровни часто совпадают, но не обязаны совпадать. `Access violation` почти всегда означает, что программа сделала что-то неправильное с памятью. Это ошибка в смысловом плане. Но с точки зрения Windows это еще и исключение: событие с кодом, параметрами и адресом возникновения.

`Breakpoint` может вообще не быть ошибкой. Отладчик специально ставит точку останова. Процессор встречает специальную инструкцию, возникает исключение, Windows передает его отладчику. Для пользователя отладчика это нормальное поведение.

`RaiseException` может быть вызван программой намеренно. Программа сама просит Windows создать исключение с заданным кодом и параметрами. Это может быть частью диагностики, runtime-механизма, защиты, теста, слоя совместимости или внутреннего протокола.

C++ `throw` на Windows в реализации MSVC тоже не является просто “прыгнуть в catch”. Под ним находится сложная система: объект исключения, runtime языка, таблицы, обработчики, раскрутка стека, вызов деструкторов. То, что программист видит как `throw`, на машинном уровне становится процедурой доставки и обработки исключения.

Поэтому правильная формула такая: ошибка - это причина или оценка, исключение - это системная форма доставки необычной ситуации.

Что значит “обычное выполнение прервалось”

Чтобы понять исключение, нужно сначала понять обычное выполнение.

Обычное выполнение программы - это движение процессора по машинным инструкциям. В x64 каждая инструкция находится по какому-то виртуальному адресу. Регистр RIP хранит адрес инструкции, связанной с текущим выполнением. Если инструкция обычная и не меняет

поток управления радикально, после ее выполнения RIP перейдет к следующей инструкции. Если инструкция - `call`, RIP станет адресом вызываемой функции. Если инструкция - `ret`, RIP возьмет адрес возврата со стека. Если инструкция - условный переход, RIP зависит от флагов. Если инструкция - безусловный `jmp`, RIP сразу переходит на цель.

Это называется control flow - поток управления. Не поток в смысле thread, а именно маршрут выполнения по коду. Дизассемблер показывает инструкции, декомпилятор пытается восстановить `if`, `while`, `switch`, но под всем этим лежит поток управления между адресами.

Исключение ломает эту обычную дорогу. Оно не похоже на обычный `jmp`, который явно написан в коде. Оно не похоже на обычный `call`, где вызываемая функция заранее указана инструкцией. Исключение возникает как событие, и дальнейший маршрут выбирается не одной инструкцией, а механизмом Windows.

Это тонкий момент. Если смотреть только на дизассемблированный код функции, можно не увидеть стрелку “сюда управление уйдет при исключении”. В обычном control-flow graph будут видны явные переходы, вызовы и возвраты. Но исключение может передать управление обработчику, который связан с функцией через метаданные, а не через обычный `call` из текущей инструкции. Поэтому exception handling - это не просто часть кода. Это часть общей модели выполнения, где код, стек, регистры и таблицы метаданных работают вместе.

Когда программа доходит до инструкции, которая вызывает исключение, происходит смена режима рассуждения. До исключения нас интересовал обычный поток инструкций. После исключения нас интересует система доставки: кто первым получает событие, где искать обработчик, можно ли продолжить выполнение, нужно ли раскручивать стек, какие кадры нужно пройти, какие регистры восстановить.

Где существует исключение: не “в программе”, а в конкретном потоке

Еще одна частая ошибка - говорить “в программе произошло исключение”. В бытовом смысле нормально, но технически недостаточно.

Исключение происходит в конкретном потоке.

Процесс - это контейнер. У него есть виртуальное адресное пространство, загруженные модули, куча, дескрипторы, таблицы, ресурсы. Но процесс сам по себе не “идет по инструкциям”. По инструкциям идет поток.

Поток - это конкретная линия выполнения внутри процесса. У каждого потока есть свой стек и свой текущий машинный контекст. Если в процессе десять потоков, то у них десять разных RIP, десять разных RSP, десять разных стеков и десять разных текущих историй вызовов. Один поток может спокойно ждать в `WaitForSingleObject`, другой может парсить файл, третий может падать на access violation. Исключение одного потока не означает, что все потоки одновременно оказались в том же месте.

Это важно для unwind. Раскрутка стека происходит по стеку конкретного потока. Если исключение случилось в worker-thread, Windows не будет искать обработчик в стеке GUI-thread. Обработчик ищется в цепочке вызовов того потока, где произошло событие. Это звучит очевидно, но на практике помогает не путаться, когда ты анализируешь crash dump многопоточного процесса. В дампе может быть много стеков, но “падающий” поток - тот, чей контекст содержит исключение.

Контекст потока - это снимок регистров и некоторого состояния процессора. В Windows для этого есть структура CONTEXT. Она может содержать значения регистров общего назначения, RIP, RSP, флаги, SIMD-регистры и другие поля, зависящие от архитектуры. Для нашей темы в начале особенно важны RIP, RSP и nonvolatile-регистры.

RIP отвечает за вопрос: где поток находился в коде? RSP отвечает за вопрос: где вершина стека этого потока? Остальные регистры отвечают за вопрос: с какими значениями поток выполнял инструкцию?

Когда возникает исключение, сама ошибка без контекста почти бесполезна. Код 0xC0000005 говорит “нарушение доступа”, но не объясняет, почему. Контекст объясняет: вот инструкция, вот адрес, вот регистры, вот стек, вот функция, вот модуль, вот цепочка вызовов. Именно поэтому полноценный анализ исключения - это анализ контекста, а не только кода исключения.

Exception record: исключение как запись о событии

Чтобы исключение можно было передавать между слоями системы, его нужно описать как данные. Для этого Windows использует запись исключения - exception record. На концептуальном уровне это структура, в которой содержится информация о самом событии.

Нас здесь интересует не запоминание всех полей, а смысл: Windows должна где-то хранить ответ на вопросы “что произошло?”, “где произошло?”, “с какими дополнительными параметрами?”. Исключение не может быть просто эмоцией системы. Оно должно быть объектом обработки.

У исключения есть код. Код исключения - это числовой идентификатор типа события. Например, access violation имеет один код, breakpoint - другой. Но код - это только категория.

У исключения есть адрес. Это адрес инструкции или места, связанного с возникновением события. В анализе это обычно связывается с RIP или control PC. Адрес нужен, потому что именно он позволяет понять, в какой функции находился поток.

У исключения могут быть параметры. Например, для access violation параметры помогают понять, был ли это доступ на чтение, запись или выполнение, и к какому адресу пыталась обратиться инструкция. Это резко повышает ценность анализа. Без параметров ты знаешь только “память нарушена”. С параметрами ты уже видишь “инструкция пыталась читать адрес 0x...” или “пыталась выполнить память по адресу 0x...”.

Рядом с exception record существует context record - запись контекста. Если exception record говорит “что случилось”, то context record говорит “в каком машинном состоянии это случилось”. Эта пара - событие плюс контекст - и есть основа дальнейшей доставки.

Почему Windows не может сразу вызвать “нужный catch”

Если человек смотрит на C++-код, ему кажется, что все просто:

```
try {
    dangerous();
} catch (...) {
    handle();
}
```

Кажется, что если внутри `dangerous()` что-то случилось, Windows должна сразу прыгнуть в `catch`. Но на машинном уровне нет магической стрелки “из любого места dangerous в этот catch”. Между местом возникновения исключения и обработчиком может быть много функций, много кадров стека, разные языковые runtime-структуры, фильтры, finally-блоки, деструкторы, frame-based handler-ы и vectored handler-ы.

Система должна сначала понять, где она находится. Потом понять, какие обработчики вообще относятся к текущему месту. Потом спросить обработчик или runtime, хочет ли он обработать это исключение. Если не хочет - подняться к вызывающему кадру. Если где-то найден обработчик, нужно не просто прыгнуть туда. Нужно корректно раскрутить стек: выполнить необходимые действия очистки, восстановить состояние, вызвать termination handlers, привести выполнение к точке, где обработчик может работать.

На x64 этот процесс завязан на unwind-метаданные. Windows не может просто “угадать” предыдущую функцию. Она должна восстановить caller context. Для этого используется информация о том, как текущая функция изменила стек и сохранила регистры.

Вот почему тема исключений быстро приводит к `.pdata` и `.xdata`. Они не являются “форматом ради формата”. Это инфраструктура, которая позволяет Windows двигаться по стеку вызовов формально, а не гадательно.

Что такое handler в самом общем смысле

Handler - обработчик. Это код, которому система может передать информацию об исключении и спросить: ты обрабатываешь это или нет?

Но слово “обработчик” опасно тем, что под ним скрывается несколько разных уровней.

Есть обработчик как идея: кусок кода, реагирующий на исключение. Есть frame-based handler: обработчик, связанный с конкретным кадром функции. Он участвует в поиске по стеку. Если исключение случилось глубоко в цепочке вызовов, Windows может подниматься от кадра к кадру и проверять, нет ли у кадра обработчика.

Есть language-specific handler: обработчик, который понимает правила конкретного языка или компилятора. Например, `__C_specific_handler` для MSVC SEH знает, как интерпретировать дополнительные данные, связанные с `__try/__except` и `__try/__finally`. Windows дает общий механизм, но не обязана понимать все языковые конструкции как исходный код. Поэтому компилятор и runtime добавляют свой слой.

Есть vectored exception handler, VEH. Он не привязан к конкретному кадру стека так же, как frame-based обработка. Это обработчик, зарегистрированный на уровне процесса, который получает шанс увидеть исключение по отдельным правилам.

Есть unhandled/top-level handling - финальный слой, когда нормальная обработка не нашла способа продолжить. На этом этапе не нужно запоминать все дерево. Главное - не думать, что “handler” всегда означает один конкретный `catch`. В Windows обработка исключений - это система уровней. Разные обработчики подключаются в разных местах маршрута.

Почему x64-модель так сильно завязана на таблицы

Исторически у x86 и x64 разные модели обработки. На x86 Structured Exception Handling часто ассоциируют с цепочкой записей, связанной со стеком потока. На x64 Windows сделала сильный акцент на табличную модель: информация о функциях и unwind лежит не просто в случайных местах стека, а в данных образа и function tables.

Почему это важно? Стек - это рабочая память. Он меняется во время выполнения. Он может быть поврежден. В нем могут лежать данные, похожие на адреса. Его трудно использовать как единственный надежный источник истины.

Табличная модель говорит: для диапазона кода функции есть запись. Эта запись описывает, где функция начинается, где заканчивается, и где лежит информация для раскрутки. Если поток находится по адресу внутри этого диапазона, система может формально взять соответствующую unwind-информацию и применить ее к контексту.

Это похоже на разницу между “искать дорогу по следам на земле” и “иметь карту здания”. Следы могут быть случайными, карта составлена заранее. Конечно, карта тоже может быть повреждена, отсутствовать или намеренно искажена, но нормальный механизм ОС рассчитывает именно на нее.

Для reverse engineering это дает мощную точку анализа. `.pdata` и `.xdata` показывают, как компилятор описал функции для Windows. Если код есть, а unwind-метаданных нет, это может быть нормально для leaf function, а может быть подозрительно. Если диапазоны функций пересекаются, если `UnwindInfoAddress` указывает в странное место, если unwind-коды не соответствуют прологу - это уже сигнал. Не “просто структура странная”, а нарушение контракта между кодом и механизмом unwinding.

Почему исключение связано с PE-файлом

На первый взгляд исключение - это runtime-событие. Оно происходит во время выполнения. Причем тут PE-файл?

Связь в том, что на x64 часть информации, нужной для обработки исключения, хранится в загруженном образе. А загруженный образ построен из PE-файла.

PE-файл - это формат Windows executable image. Он описывает не только код и данные, но и служебные каталоги: импорты, экспорты, ресурсы, релокации, TLS, exception directory и другие. Когда Windows загружает EXE или DLL, она не просто копирует файл в память. Она создает image: отображает секции, применяет релокации, разрешает импорты, устанавливает права страниц, учитывает заголовки.

Exception Directory - один из data directories PE. Для x64 он ведет к данным, по которым Windows может найти `RUNTIME_FUNCTION`-записи. Эти записи обычно находятся в секции `.pdata`. Каждая запись связывает диапазон кода функции с unwind-информацией, обычно лежащей в `.xdata`.

Здесь появляется очень важная цепочка: исключение происходит при выполнении инструкции; инструкция находится по virtual address; virtual address принадлежит загруженному PE-образу; по адресу нужно определить функцию; данные о функции лежат в Exception Directory и `.pdata`; unwind-описание лежит в `.xdata`; unwind-описание позволяет восстановить предыдущий кадр; предыдущий кадр нужен для поиска обработчика.

Вот почему книга по Windows x64 SEH/Unwind не может быть только про “исключения” или только про “PE”. Это одна система. PE дает статическую карту, runtime использует ее во время исключения, unwind превращает текущий контекст в предыдущий, обработчики принимают решение, а reverse engineer проверяет, согласованы ли все эти слои.

Search phase и unwind phase: два разных движения

Возникновение исключения - это только начало. Дальше нужно доставить его до того, кто может принять решение.

Search phase - фаза поиска. В этой фазе система еще не обязана разрушать кадры и передавать управление обработчику. Она идет по цепочке вызовов и спрашивает: есть ли здесь обработчик, который хочет принять это исключение?

Unwind phase - фаза раскрутки. Если обработчик найден, система уже не просто спрашивает. Она должна привести стек и выполнение к состоянию, в котором управление можно передать обработчику. Для промежуточных кадров могут вызываться cleanup- и termination-действия. Поэтому нельзя просто прыгнуть в handler, оставив все как есть.

На x64 обе фазы завязаны на способность переходить от текущего кадра к предыдущему. А эта способность требует unwind-информации. Поэтому `RUNTIME_FUNCTION`, `UNWIND_INFO` и `UNWIND_CODE` - не детали “для потом”, а центральные элементы самой идеи обработки исключения.

Access violation, RaiseException и breakpoint как три разных лица одного механизма

Access violation показывает аппаратно-системную сторону. Инструкция пыталась обратиться к памяти, но страница недоступна или права не позволяют действие. Windows получает событие, оформляет его, связывает с контекстом и запускает доставку.

`RaiseException` показывает программную сторону. Здесь нет обязательного сбоя памяти. Программа сама вызывает Windows API и просит создать исключение с заданным кодом. Источник другой, но после оформления события маршрут обработки использует тот же общий каркас.

Breakpoint показывает отладочную сторону. Отладчик может поставить специальную инструкцию останова. Процессор генерирует breakpoint exception. Windows передает событие отладчику. Для пользователя отладчика это не авария, а управляемая остановка.

Эти примеры важны, потому что они отучают думать об исключении как о синониме падения. Исключение - это транспорт необычного события через систему обработки. Смысл события может быть разным: баг, диагностика, отладка, языковой runtime, защита, антиотладка.

Главная модель раздела

Программа выполняется не как текст C, а как поток машинных инструкций. Конкретный поток имеет `RIP`, `RSP`, регистры и стек. Инструкция может завершиться нормально, изменить `RIP`, вызвать функцию, вернуться или спровоцировать исключение. Исключение означает, что обычный маршрут выполнения прерван и Windows должна передать событие механизму обработки.

Windows оформляет событие как exception record и связывает его с context record. Дальше начинается доставка. Система может дать шанс отладчику и vectored handlers. Потом она идет по frame-based цепочке вызовов. Чтобы перейти от текущего кадра к предыдущему, она использует unwind-метаданные. На x64 эти данные связаны с PE-образом: Exception Directory, `.pdata`, `RUNTIME_FUNCTION`, `.xdata`, `UNWIND_INFO`. Если обработчик найден, система выполняет нужную раскрутку и передает управление. Если обработчик не найден, исключение становится необработанным, и процесс идет к аварийному завершению или последним механизмам обработки.

Именно это надо понимать под фразой “произошло исключение в Windows”. Не “ошибка”. Не “catch”. Не “краш”. А систему, где событие, контекст, стек, PE-метаданные и обработчики связаны в один маршрут.

1.2. Что такое вызов функции, стек и кадр стека

Теперь нужно разобрать второй фундаментальный слой: что такое вызов функции. Без этого весь дальнейший разговор про unwind будет висеть в воздухе. Слово “unwind” буквально означает размотку или раскрутку назад. Но что именно раскручивается? Не файл, не исходный код и не список функций из PE. Раскручивается текущая цепочка вызовов конкретного потока. А цепочка вызовов существует благодаря тому, как функции вызывают друг друга, как они используют стек и как сохраняют состояние, необходимое для возврата.

Когда ты читаешь исходный код, вызов функции выглядит почти бытово: одна функция попросила другую что-то сделать. Но на машинном уровне это договор между вызывающим кодом, вызываемой функцией, процессором, стеком и ABI. Если этот договор соблюден, функции могут вызывать друг друга, передавать аргументы, возвращать значения, сохранять важные регистры и корректно возвращаться. Если договор нарушен, программа может падать, стек может разрушиться, отладчик может показать ложный call stack, а `RtlVirtualUnwind` может не восстановить caller context.

Поэтому вызов функции - это не маленькая тема “для начинающих”. Это механизм, на котором стоит вся x64-модель исключений.

Функция в исходнике и функция в бинарнике - не одно и то же

В исходном коде функция - это именованный блок логики. Например, `int sum(int a, int b)`. У нее есть имя, типы аргументов, тело, локальные переменные, `return`. Это удобная форма для человека и компилятора.

В бинарнике функция - это область машинного кода. У нее может не быть исходного имени. Она может начинаться по адресу, который дизассемблер считает entry point функции. Она может иметь пролог, тело, эпилоги, вызовы других функций, переходы, обращения к данным. В PE x64 для многих функций существует еще и табличная запись в `.pdata`, которая описывает диапазон кода и указывает на unwind-информацию.

В runtime, то есть во время выполнения, функция проявляется еще иначе: как активный вызов. Один и тот же код функции может быть вызван много раз. Если функция рекурсивная, один участок кода может иметь несколько активных вызовов одновременно. Каждый такой вызов имеет свое состояние: свой адрес возврата, свой участок стека, свои локальные значения, свои сохраненные регистры. Поэтому нужно различать три уровня: функция как исходный текст, функция как машинный код и вызов функции как живое состояние в стеке

потока.

Это различие напрямую связано с обработкой исключений. Windows не ищет обработчик “в тексте функции”. Она работает с текущим машинным состоянием. Если исключение произошло внутри активного вызова функции, система должна понять, где находится код этой функции, как устроен ее текущий кадр и как перейти к кадру вызывающей функции.

Caller и callee: кто кого вызывает

В описании calling convention постоянно встречаются два английских термина: caller и callee.

Caller - вызывающий код. Это функция или участок кода, который делает вызов. Если `main` вызывает `foo`, то `main` в этом месте является caller.

Callee - вызываемая функция. В том же примере `foo` является callee.

Эти слова важны не как словарь, а как разделение ответственности. Вызов функции - это договор между caller и callee. Кто кладет аргументы в регистры? Кто резервирует shadow space? Кто обязан сохранить nonvolatile-регистры? Кто очищает стек? Кто возвращает значение? Все эти вопросы решаются calling convention.

Если caller и callee скомпилированы по одному ABI, они понимают друг друга. Caller кладет первые аргументы туда, где callee ожидает их найти. Callee сохраняет те регистры, которые по договору нельзя портить. Return value возвращается туда, где caller его ждет. Если же код нарушает договор, например вызывает функцию с неправильной сигнатурой или портит RSP, проблемы могут проявиться не сразу, но стек и контекст станут недостоверными.

Для reverse engineering термины caller/callee полезны еще и потому, что они помогают читать дизассемблер. Когда ты видишь `call`, нужно спрашивать: что caller подготовил перед вызовом? Какие регистры заполнены? Есть ли место на стеке? Что callee делает в прологе? Что она возвращает? Какие регистры после вызова больше нельзя считать прежними?

Инструкция call: как машина входит в функцию

На x64 вход в функцию обычно происходит через инструкцию `call`. На уровне идеи `call` делает две вещи: сохраняет адрес возврата и передает управление вызываемой функции.

Адрес возврата - return address - это адрес инструкции, которая должна выполниться после того, как вызываемая функция завершится. Если caller вызывает callee, то после callee выполнение должно продолжиться не с начала caller-а, а ровно с места после инструкции `call`.

Процессор должен где-то сохранить этот адрес. На x86/x64 для этого используется стек. Инструкция `call` помещает return address на стек и меняет RIP на адрес цели вызова. Это можно представить так: перед прыжком в функцию процессор оставляет на стеке записку “когда закончишь, возвращайся сюда”.

Важно: return address - это не декоративная информация для отладчика. Это реальный адрес, который позже использует `ret`. Если он поврежден, функция может вернуться не туда. Поэтому повреждение стека часто приводит к странным переходам, падениям и невозможности построить нормальный stack trace.

Пример условного машинного маршрута:

```
caller:
    ; подготовка аргументов
    call callee      ; return address кладется на стек, RIP переходит в callee
    ; выполнение продолжится здесь после ret
```

После `call` управление уже внутри `callee`. Но стек теперь содержит адрес возврата. Это первый кирпич кадра вызова: чтобы вернуться, нужно знать куда.

На практике `call` может быть прямым или косвенным. Direct call содержит цель относительно текущего адреса или как непосредственную цель, которую дизассемблер обычно показывает явно. Indirect call берет цель из регистра или памяти, например `call rax` или `call qword ptr [rip+...]`. Для реверса indirect call сложнее: нужно понять, откуда взялся указатель на функцию. Но в обоих случаях принцип возврата тот же: return address сохраняется на стеке.

Инструкция ret: как машина выходит из функции

Если `call` - вход, то `ret` - обычный выход. Инструкция `ret` берет адрес возврата со стека и помещает его в `RIP`. После этого процессор продолжает выполнение в caller-е.

Здесь становится видно, почему RSP так важен. `ret` ожидает, что вершина стека указывает на правильный return address. Если функция в процессе работы уменьшала RSP, выделяла стековое место, сохраняла регистры, она должна перед `ret` вернуть RSP в такое состояние, при котором на вершине снова окажется адрес возврата.

Упрощенная схема:

```
callee:
    sub rsp, 40h     ; выделили стековое место
    ; тело функции
    add rsp, 40h     ; вернули RSP назад
    ret             ; взяли return address со стека
```

Если забыть `add rsp, 40h`, то `ret` возьмет не return address, а какие-то данные из локальной области. Управление уйдет по мусорному адресу. В реальном коде компилятор, конечно, не “забывает” просто так, но повреждение стека, неверная сигнатура вызова, ручной ассемблер, обфускация или exploit-сценарии могут привести к похожему эффекту.

Для unwind смысл такой: чтобы виртуально выполнить выход из функции, нужно знать, как вернуть RSP к состоянию caller-а и где взять return address. Обычный `ret` делает это в реальном выполнении. `RtlVirtualUnwind` делает похожую работу аналитически: не исполняет функцию заново, а по metadata и context record вычисляет контекст предыдущего кадра.

Стек потока: почему это не просто “место под переменные”

Stack - стек. В учебниках его часто объясняют как структуру LIFO: last in, first out, последним положили - первым достали. Для общей информатики это полезно. Но для Windows x64 exceptions нужно более живое понимание: стек потока - это память текущей истории вызовов.

Каждый поток имеет свой стек. Это значит, что стек принадлежит не процессу вообще, а конкретной линии выполнения. Процесс может иметь много потоков, но у каждого будет своя цепочка вызовов. Исключение в одном потоке раскручивает стек именно этого потока.

Стек нужен потому, что выполнение функций вложенное. `main` вызывает A, A вызывает B, B вызывает C. Пока выполняется C, программа не забыла про B, A и `main`. Где-то нужно хранить возможность вернуться из C в B, из B в A, из A в `main`. Кроме адресов возврата, нужно хранить сохраненные регистры, временные области, иногда локальные переменные, иногда аргументы, которые не поместились в регистры.

Стек удобен именно для такой вложенности. Последняя вызванная функция завершается первой. Это совпадает с природой LIFO. Поэтому кадр верхней функции можно снять, не трогая нижние кадры. Когда C завершилась, ее состояние больше не нужно; управление возвращается в B. Когда завершилась B, возвращаемся в A. Это естественная форма жизни вызовов.

Но стек не является “безопасным списком функций”. Он просто память. Там лежат байты. Их смысл определяется соглашениями. Если соглашения соблюдены, стек можно интерпретировать как цепочку кадров. Если стек поврежден, эти байты могут перестать быть надежной картой выполнения.

RSP: вершина стека как текущая граница кадра

RSP - stack pointer, указатель стека. Это один из главных регистров x64. Он указывает на вершину стека текущего потока. Слово “вершина” здесь условное: на Windows x64 стек обычно растет к меньшим адресам, поэтому выделение места на стеке уменьшает значение RSP.

Когда функция хочет выделить место на стеке, она делает что-то вроде:

```
sub rsp, 40h
```

Эта инструкция означает: сдвинуть вершину стека на `0x40` байт вниз, к меньшим адресам. Процессор не рисует прямоугольник и не пишет “локальные переменные”. Он просто меняет число в RSP. Но по соглашению между кодом и ABI область около нового RSP теперь принадлежит текущей функции.

Когда функция завершает работу, она должна вернуть RSP назад:

```
add rsp, 40h
```

После этого RSP снова указывает туда, где caller ожидает увидеть свой стек. Если функция не вернет RSP, то `ret` возьмет не тот адрес. Если функция вернет RSP слишком далеко, она перескочит через правильный return address. Если функция портит RSP непредсказуемо, unwind становится сложным или невозможным.

В нормальном скомпилированном коде изменение RSP не произвольное. Пролог функции выделяет стек и сохраняет регистры по правилам, которые компилятор затем описывает в unwind metadata. Именно поэтому x64 exception handling требует, чтобы действия пролога были описуемыми: Windows должна уметь восстановить, что произошло с RSP.

Кадр стека: состояние не функции, а вызова функции

Stack frame - кадр стека. Это один из тех терминов, которые нельзя оставлять на уровне “кусок памяти”. Кадр стека - это область стека и связанное с ней машинное состояние, принадлежащие одному активному вызову функции.

Ключевое слово - активному вызову. Функция как код может быть одна. Вызовов этой функции может быть много. У каждого вызова свой кадр.

Представь рекурсию:

```
int fact(int n) {
    if (n <= 1) return 1;
    return n * fact(n - 1);
}
```

Код `fact` один. Но если `fact(5)` вызывает `fact(4)`, потом `fact(3)`, потом `fact(2)`, потом `fact(1)`, в стеке одновременно существуют несколько активных вызовов одной функции. У каждого свой n, свой return address, свой момент возврата. Поэтому нельзя говорить “у функции есть кадр” слишком грубо. Кадр есть у конкретного вызова функции.

Для обработки исключений это критично. Если исключение произошло в глубокой рекурсии, Windows идет не по списку уникальных функций, а по живой цепочке кадров: текущий вызов `fact`, предыдущий вызов `fact`, еще предыдущий, и так далее. В каждом кадре может быть свой обработчик, свое состояние, своя связь с caller-ом.

Кадр может включать return address, сохраненные nonvolatile-регистры, локальные переменные, временные данные, shadow space, область выравнивания, spill slots, данные для security cookie, иногда указатель на предыдущую базу кадра. Но это не значит, что каждый кадр всегда содержит все перечисленное. Оптимизирующий компилятор может держать переменную только в регистре. Leaf function может почти не трогать стек. Debug-сборка может делать кадр тяжелее и понятнее для отладки. Release-сборка может уничтожить красивую картинку исходника.

Поэтому кадр стека нужно понимать не как фиксированную схему, а как реальное состояние вызова, которое компилятор построил конкретными инструкциями и которое unwind-механизм должен уметь разобрать назад.

Windows x64 calling convention: договор о вызовах

Calling convention - соглашение о вызовах. Это набор правил, по которым функции передают аргументы, возвращают значения, используют регистры и стек. На Windows x64 это часть ABI - application binary interface, двоичного интерфейса. ABI нужен не исходному коду, а уже скомпилированному коду: чтобы функция из одного модуля могла вызвать функцию из другого, чтобы EXE мог вызывать DLL, чтобы компилятор, линкер, отладчик и ОС одинаково понимали машинный договор.

В Windows x64 первые четыре integer/pointer аргумента обычно передаются через регистры RCX, RDX, R8, R9. Floating-point аргументы используют XMM-регистры по соответствующим правилам. Возвращаемое integer/pointer значение обычно приходит через RAX. Это не “стиль компилятора”, а соглашение, которого ожидает код.

Пример C-вызова:

```
foo(a, b, c, d, e);
```

На уровне Windows x64 первые четыре аргумента будут подготовлены через регистры. Пятый уже должен быть передан через стек. Это сразу показывает, почему стек и регистры нельзя изучать отдельно. Вызов функции живет одновременно в регистрах и на стеке.

Calling convention также говорит, какие регистры volatile, а какие nonvolatile. Volatile-регистр вызываемая функция может портить. Caller не должен ожидать, что после вызова значение сохранится. Nonvolatile-регистр вызываемая функция обязана восстановить, если

использовала. Это распределение ответственности экономит работу: не нужно сохранять все регистры всегда, но при этом caller может рассчитывать на сохранность определенной части состояния.

Для unwind это не абстракция. Если callee сохранила nonvolatile-регистр на стеке, Windows должна знать где он сохранен, чтобы при раскрутке восстановить контекст caller-а. Поэтому unwind metadata содержит сведения о сохранении nonvolatile-регистров.

Shadow space: место, которое caller резервирует заранее

Shadow space, его еще называют shadow store или home space, - характерная часть Windows x64 calling convention. Caller обязан зарезервировать на стеке место под четыре регистровых аргумента. Даже если callee не использует это место, оно должно существовать.

Зачем нужна эта странность? Потому что первые аргументы приходят в регистрах, но callee может захотеть сохранить их в память. Например, если нужно взять адрес аргумента, передать его дальше, использовать varargs или просто spill-нуть регистр. Чтобы у callee было стандартное место, куда можно положить эти register parameters, caller заранее резервирует shadow space.

Это не значит, что каждый вызов обязательно записывает туда аргументы. Это значит, что место по соглашению доступно. Поэтому в дизассемблере можно видеть обращения к stack slots, которые соответствуют home space, даже когда аргументы вроде бы были переданы через регистры.

Для новичка shadow space часто выглядит как “лишние 32 байта на стеке”. Для реверсера это маркер Windows x64 ABI. Если ты видишь перед вызовом резервирование места, не нужно сразу думать “локальный массив”. Возможно, это просто обязательная область для вызываемой функции.

Связь с unwind такая: shadow space обычно относится к стороне caller-а, но влияет на форму стека вокруг вызова. Когда ты восстанавливаешь цепочку кадров или читаешь аргументы, важно понимать, что часть стека около return address может быть не локальными переменными текущей функции, а областью, зарезервированной по соглашению для следующего вызова.

Stack alignment: почему стек должен быть выровнен

Stack alignment - выравнивание стека. Это правило о том, что RSP должен иметь определенную кратность, обычно связанную с 16 байтами, в важных точках выполнения. Выравнивание нужно для производительности, корректной работы некоторых инструкций и соблюдения ABI.

Интуитивно выравнивание означает: адрес должен быть кратен определенному числу. Если адрес кратен 16, его младшие 4 бита равны нулю. Для SIMD-инструкций и для соглашений компилятора это может быть важно.

На Windows x64 есть тонкость: инструкция `call` кладет return address на стек, то есть уменьшает RSP на 8 байт. Поэтому на входе в callee значение RSP имеет смещение относительно 16-байтного выравнивания. Пролог функции обычно корректирует стек так, чтобы тело функции работало с нужным выравниванием. Microsoft описывает, что стек должен оставаться 16-byte aligned в областях кода вне пролога и эпилога, за исключением

особых случаев leaf functions.

Для reverse engineering это полезно по двум причинам. Во-первых, по выравниванию можно лучше понимать, почему функция выделяет именно `0x28`, `0x38` или `0x48` байт, а не “ровно сколько нужно переменным”. Часть размера может уходить на alignment и shadow space. Во-вторых, если стек не выровнен там, где должен, это может быть признаком нестандартного кода, ручного asm, перехода не в начало функции, повреждения RSP или неправильного прототипа вызова.

Prolog: место, где функция строит кадр

Prolog - пролог функции. Это начальный участок машинного кода функции, где она готовит свое рабочее состояние. В обычном C-коде ты не пишешь пролог руками. Его генерирует компилятор. Но для Windows x64 unwind пролог - один из важнейших участков функции.

Что может делать пролог? Он может сохранить nonvolatile-регистры, которые функция собирается использовать. Он может выделить место на стеке через `sub rsp, N`. Он может установить frame pointer, если функция использует регистр как стабильную базу кадра. Он может вызвать helper для большой стековой аллокации. Он может подготовить security cookie. Конкретная форма зависит от функции, компилятора, оптимизаций и настроек безопасности.

Почему именно пролог описывается в unwind metadata? Потому что unwind - это обратная задача. Если пролог построил кадр, unwind должен уметь его разобрать. Если пролог сделал `push rbx`, значит старый RBX лежит на стеке. Если пролог сделал `sub rsp, 80h`, значит текущий RSP ниже caller-ового на определенную величину. Если пролог установил frame register, значит часть обращений к кадру может идти через этот регистр. Все это нужно знать, чтобы восстановить caller context.

Microsoft в документации по x64 exception handling прямо связывает unwind data с эффектами функции на stack pointer и с местами сохранения nonvolatile-регистров. Это и есть смысл: metadata не описывает функцию “красиво”, она описывает минимально важные изменения состояния, необходимые для раскрутки.

```
push rbx          ; сохранить nonvolatile RBX
push rdi          ; сохранить nonvolatile RDI
sub  rsp, 60h     ; выделить стековое место
; тело функции
```

Такой пролог для unwind означает не просто три инструкции. Он означает три обратимых действия. При раскрутке нужно знать, где находятся сохраненные RBX и `RDI`, и как вернуть RSP к состоянию caller-а. `UNWIND_CODE` как раз кодирует подобные действия.

Epilog: место, где функция разбирает кадр

Epilog - эпилог функции. Это участок выхода из функции, где она отменяет то, что сделала в прологе: освобождает стековое место, восстанавливает сохраненные регистры и выполняет `ret` или эквивалентный tail transfer.

Если пролог - сборка кадра, то эпилог - разборка кадра. У них зеркальная логика, но не обязательно один к одному в виде красивой симметрии. В функции может быть несколько эпилогов: разные ветки `return`, разные выходы из условий, оптимизации компилятора.

Для x64 unwind эпилог важен отдельно. Если исключение или stack walking происходит, когда RIP находится внутри эпилога, состояние функции уже частично разобрано. Например,

RSP уже мог быть увеличен, один регистр уже восстановлен, другой еще нет. Unwind-механизм должен распознавать эпилоговые участки, чтобы корректно интерпретировать состояние.

Это одна из причин, почему правила прологов и эпилогов на x64 довольно строгие. Код должен быть таким, чтобы ОС могла понять, где функция строит кадр, где разбирает, и как восстановить контекст по любой допустимой точке.

```
; тело функции
add rsp, 60h      ; освободить стековое место
pop rdi           ; восстановить RDI
pop rbx           ; восстановить RBX
ret
```

В реальном коде порядок может отличаться в зависимости от того, как именно пролог сохранял данные. Важно не механически запомнить один шаблон, а понять принцип: эпилог возвращает machine state к договоренному состоянию caller-а.

Frame pointer и frame register: стабильная база внутри кадра

Frame pointer - указатель кадра. В старых учебных схемах часто показывают RBP как постоянную базу кадра: локальные переменные лежат по отрицательным смещениям от RBP, аргументы - по положительным. На x64 такая схема возможна, но не обязательна.

Современный оптимизирующий компилятор часто не использует RBP как классический frame pointer. Почему? Потому что регистр дорогой. Если можно обращаться к стековым данным через RSP и unwind metadata, то RBP можно использовать для других целей. Поэтому в release-коде отсутствие классического frame pointer - нормальная вещь.

Frame register в контексте `UNWIND_INFO` - это регистр, который функция использует как базу для обращения к своему кадру. В unwind-информации есть поле frame register и scaled offset. Это не просто “есть RBP или нет”. Это способ сообщить unwinder-у, что после определенного момента кадр можно описывать через стабильную базу.

Зачем нужна стабильная база? Если функция делает динамические изменения RSP, например выделяет переменный размер на стеке, то обращаться к локальным объектам только через RSP может быть неудобно: RSP двигается. Frame pointer дает точку, которая остается стабильной внутри функции.

Для реверсера frame pointer полезен, потому что упрощает чтение кадра. Если видишь обращения вроде `[rbp-20h]`, можно предположить локальную область. Но нельзя слепо переносить x86-картинку на x64. На x64 RBP может быть обычным регистром, а frame pointer может отсутствовать. Истина не в привычной схеме, а в коде и unwind metadata.

Volatile и nonvolatile registers: кто обязан сохранять состояние

Volatile и nonvolatile - термины calling convention. Они отвечают на вопрос: какие регистры переживают вызов функции, а какие нет.

Volatile register - регистр, который callee может испортить без восстановления. Caller должен считать, что после вызова старое значение такого регистра потеряно. В Windows x64 к volatile относятся, например, RAX, RCX, RDX, R8, R9, R10, R11 и часть XMM-регистров по правилам ABI.

Nonvolatile register - регистр, который callee обязана сохранить, если хочет его использовать. В Windows x64 к nonvolatile относятся, например, RBX, RBP, RDI, RSI, R12, R13,

R14, R15 и некоторые XMM-регистры. Точный набор нужно сверять с ABI, но идея важнее списка: nonvolatile - это состояние caller-a, которое callee не имеет права уничтожить молча.

Представь, что caller держит важный указатель в RBX и вызывает функцию. По ABI caller может ожидать, что после возврата RBX будет прежним. Если callee хочет использовать RBX, она должна сохранить старое значение, например на стеке, а перед выходом восстановить.

```
push rbx          ; сохранить значение caller-a
; использовать RBX внутри функции
pop  rbx          ; восстановить перед выходом
ret
```

Для unwind эта обязанность становится данными. Если исключение произошло до обычного `pop rbx`, но после `push rbx`, Windows должна восстановить RBX при переходе к caller context. Иначе caller context будет ложным. Поэтому unwind metadata описывает сохранение nonvolatile-регистров.

Вот почему фраза “nonvolatile register” не должна оставаться английским ярлыком. Это регистр, сохранность которого является частью договора между функциями. А unwind - механизм, который обязан поддержать этот договор даже при необычном выходе через исключение.

Локальные переменные: почему они не всегда лежат на стеке

Новичок часто думает: локальная переменная - значит стек. Это полезное первое приближение, но в реверсе оно быстро ломается.

Локальная переменная в C - это объект с областью видимости внутри функции и временем жизни, связанным с выполнением блока. Но компилятор не обязан хранить ее именно в памяти. Если переменная маленькая и оптимизация позволяет, она может жить только в регистре. Если значение можно вычислить как константу, переменная может исчезнуть совсем. Если код в debug-сборке, компилятор может специально держать переменные на стеке, чтобы отладчик мог их показывать.

Поэтому когда ты видишь `[rsp+20h]`, это может быть локальная переменная. Но может быть сохраненный регистр, temporary spill, shadow space, stack argument, выравнивание или служебное место. А когда ты не видишь переменную на стеке, это не значит, что ее не было в исходнике.

Для unwind это важно потому, что unwind metadata не обязана описывать все локальные переменные. Ей не интересно, что программист назвал `int count`. Ей интересно только то, что влияет на восстановление caller context: изменение RSP, сохранение nonvolatile-регистров, frame register, handler flags и связанные данные. Локальные переменные важны для логики программы, но не все они важны для unwind.

Leaf function и non-leaf function: почему не каждой функции нужна полноценная unwind-запись

Leaf function - листовая функция. Это функция, которая не вызывает другие функции и не требует сложного кадра, влияющего на unwind. Название “leaf” идет от дерева вызовов: лист - узел, из которого нет дальнейших вызовов вниз.

Non-leaf function - функция, которая вызывает другие функции или делает действия, требующие полноценного описания для unwind: выделяет стек, сохраняет nonvolatile-регистры, имеет обработчики исключений и так далее.

Почему это важно? Потому что на x64 не каждая функция обязана иметь полноценную unwind-метаинформацию. Если leaf function не меняет nonvolatile-регистры и не выделяет стек так, что нужен unwind, ее можно “раскрутить” проще: взять return address со стека и считать, что состояние caller-а почти уже доступно.

Но эта простота опасна для анализа. Если функция выглядит как leaf, но на самом деле делает нестандартные вещи со стеком, отсутствие metadata может быть проблемой. Если упаковщик или ручной код намеренно нарушает ожидания, stack walking может ломаться. Поэтому анализатор должен понимать, когда отсутствие записи нормально, а когда подозрительно.

Leaf/non-leaf - это не просто классификация для красоты. Это ответ на вопрос: нужен ли Windows формальный рецепт раскрутки этой функции? Если функция не построила сложный кадр, рецепт может быть не нужен. Если построила - без рецепта система не сможет надежно восстановить caller context.

Как вызов функции превращается в цепочку кадров

Теперь соберем живую цепочку. Пусть `main` вызывает A, A вызывает B, B вызывает C.

```
main -> A -> B -> C
```

Пока выполняется C, верхний активный кадр принадлежит C. Ниже лежит кадр B, ниже - A, ниже - `main`. Это не список в отдельной таблице Windows. Это состояние стека текущего потока, созданное последовательными `call`, прологами функций и сохранением состояний.

Если C нормально завершается, она разбирает свой кадр и делает `ret`. Управление возвращается в B. Потом B может продолжить выполнение, вызвать что-то еще или завершиться. Обычный возврат идет сверху вниз: последний вызов завершается первым.

Если внутри C происходит исключение, обычный `ret` не выполняется. Но системе все равно нужно понять, что было ниже C. Она должна виртуально получить состояние B, затем A, затем `main`, пока ищет обработчик. То есть unwind делает аналитически то, что обычные `ret` сделали бы постепенно при нормальном завершении, но с учетом обработчиков и cleanup-логики.

Отсюда главная связь раздела: стек нужен для нормального возврата, а unwind нужен для необычного возврата. Оба механизма работают с одной и той же реальностью - цепочкой кадров.

Почему нельзя просто сканировать стек в поисках адресов

Иногда кажется: раз return addresses лежат на стеке, можно просто пройти по памяти стека и найти все числа, похожие на адреса кода. Так действительно делают некоторые эвристики, когда нет metadata или стек поврежден. Но это не надежный способ для нормальной ОС.

Проблема в том, что стек содержит много разных чисел. Там могут быть указатели на данные, старые значения, локальные переменные, сохраненные регистры, размеры, флаги, случайные байты, shadow space. Некоторые из них случайно похожи на адреса в `.text`. Если считать любой похожий адрес return address, можно построить ложный call stack.

x64-модель предпочитает формальный путь: взять текущий RIP, найти RUNTIME_FUNCTION, прочитать UNWIND_INFO, применить правила и получить caller context. Это не исключает эвристики в аварийных случаях, но нормальная модель основана на metadata.

Для static /dynamic это будущий критерий качества: если stack trace получается только гаданием по стеку, доверие ниже. Если есть согласованная цепочка RIP -> RUNTIME_FUNCTION -> UNWIND_INFO -> caller context, доверие выше. Если metadata противоречит коду, это finding. Если metadata отсутствует там, где она должна быть, это finding. Если stack scan показывает одно, а unwind metadata - другое, это повод анализировать глубже.

Мини-сцена: исключение внутри функции с кадром

Представим функцию C. В прологе она сохранила RBX и выделила 0x60 байт на стеке:

```
C:
    push rbx
    sub  rsp, 60h
    ; ...
    mov  eax, dword ptr [rcx]  ; здесь случился access violation
    ; ...
    add  rsp, 60h
    pop  rbx
    ret
```

Исключение произошло на инструкции чтения [rcx]. В этот момент RIP указывает на проблемную инструкцию. RSP уже уменьшен: он не такой, каким был при входе в C. Старое значение RBX лежит где-то на стеке, потому что пролог сделал push rbx. Если Windows хочет получить контекст B, который вызвал C, она не может просто сказать “предыдущий кадр где-то выше”. Ей нужно учесть конкретные действия C.

Unwind metadata для C должна сказать: эта функция сохранила RBX через push; эта функция выделила 0x60 байт; чтобы восстановить caller, нужно вернуть RSP, восстановить RBX и получить return address. Именно так текущий контекст C превращается в контекст B.

Если metadata говорит одно, а код делает другое, система может восстановить неправильный контекст. Поэтому для реверсера сравнение пролога с UNWIND_CODE - не академическая игра, а проверка договора между бинарником и Windows.

Связь раздела с будущими структурами .pdata и .xdata

Пока мы еще не разбираем байтовый формат .pdata и .xdata, но уже можно понять, зачем они нужны.

.pdata отвечает на вопрос: для данного адреса кода какая функция активна и где запись, описывающая ее unwind? Внутри .pdata лежат RUNTIME_FUNCTION entries. Каждая такая запись в обычном случае содержит начало диапазона функции, конец диапазона функции и адрес unwind-информации.

.xdata отвечает на вопрос: как именно раскрутить эту функцию? Там лежит UNWIND_INFO, а внутри - сведения о версии, флагах, размере пролога, количестве unwind-кодов, frame register и массиве UNWIND_CODE.

`UNWIND_CODE` отвечает на вопрос: какие конкретные действия пролога важны для восстановления? Сохранен ли nonvolatile-регистр? Сколько байт выделено на стеке? Используется ли frame pointer? Где лежат сохраненные значения?

Теперь видно, почему в начале книги нужно так подробно разбирать `call`, `ret`, RSP, кадр, prolog, epilog, volatile/nonvolatile и shadow space. Все эти вещи потом превращаются в поля структур. Если не понимать живой механизм, таблицы будут казаться сухим форматом. Если понимать механизм, `.pdata/.xdata` превращаются в карту того, как функция живет и как из нее выйти назад.

Итог раздела

Вызов функции - это не просто строка исходного кода. Это машинный договор. Caller подготавливает аргументы, резервирует нужную область стека и выполняет `call`. `call` сохраняет return address и переводит RIP в callee. Callee строит свой кадр: сохраняет nonvolatile-регистры, выделяет стек, может установить frame pointer. Тело функции работает с регистрами, стеком и памятью. Эпилог разбирает кадр. `ret` берет return address и возвращает управление caller-у.

Стек - это память текущей цепочки вызовов потока. Кадр стека - состояние одного активного вызова. RSP показывает вершину текущего стека. Calling convention задает правила, по которым функции понимают друг друга. Shadow space, alignment, volatile/nonvolatile-регистры и prolog/epilog - это не отдельные мелочи, а части одного договора.

Unwind нужен тогда, когда обычный возврат через `ret` не является тем путем, по которому система сейчас идет. При исключении Windows должна виртуально подняться по цепочке кадров. Чтобы сделать это надежно, она использует табличные данные x64: `.pdata`, `RUNTIME_FUNCTION`, `.xdata`, `UNWIND_INFO` и `UNWIND_CODE`. Поэтому понимание вызова функции и кадра стека - прямой фундамент всей дальнейшей книги.

Источники и техническая база текущей рабочей версии

- План книги: пользовательский документ “план теории по вкр.docx”.
- Microsoft Learn: x64 exception handling - структура `UNWIND_INFO`, флаги, unwind data и связь с обработчиками.
- Microsoft Learn: x64 calling convention - передача первых аргументов через регистры, shadow store, правила ABI.
- Microsoft Learn: x64 stack usage - роль стека, stack allocation, register parameters и stack parameters.
- Microsoft Learn: x64 prolog and epilog - ограничения и смысл пролога/эпилога для unwind.
- Microsoft Learn: RtlLookupFunctionEntry - поиск записи function table по PC/control address.
- Microsoft Learn: RtlVirtualUnwind - получение контекста функции, предшествующей текущему function context.

1.3. Как Windows ищет обработчик по кадрам

В предыдущем разделе мы договорились о главной вещи: исключение в Windows - это не просто слово “ошибка”, а событие выполнения, которое получает системную форму. У события есть запись исключения, машинный контекст, адрес возникновения, стек текущего потока и дальнейший маршрут доставки. Теперь нужно разобрать следующий вопрос: если исключение уже возникло, как Windows вообще понимает, где искать код, который может его обработать?

На человеческом уровне кажется, что всё просто: где-то в программе есть `try`, `catch`, `__try`, `__except`, `__finally` или какой-то обработчик. Но процессор не выполняет исходный код. Windows не листает C-файл и не ищет глазами блок `catch`. В момент исключения у системы нет красивой структуры “вот тут программист написал try”. У неё есть машинное состояние: `RIP`, `RSP`, регистры, стек, загруженные PE-образы, таблицы функций и unwind-метаданные. Поэтому поиск обработчика - это не поиск по тексту программы. Это движение по живой цепочке вызовов текущего потока.

Эта глава важна потому, что именно здесь исключения перестают быть абстракцией языка и становятся конкретной механикой Windows x64. Здесь впервые появляется настоящая цепочка: `RIP` указывает на инструкцию, инструкция находится внутри функции, функция описана записью `RUNTIME_FUNCTION`, запись ведёт к `UNWIND_INFO`, unwind-информация позволяет восстановить caller context, а handler вызывается только если текущий кадр действительно имеет право участвовать в обработке исключения.

> Слово “кадр” дальше будет означать не картинку и не страницу документа. Кадр стека, или stack frame, - это состояние одного активного вызова функции в стеке конкретного потока. Windows ищет обработчик не “в файле”, а среди активных кадров: текущая функция, её caller, caller caller-а и так далее.

1.3.1. Искать обработчик - значит идти по живой цепочке вызовов

Начнём с простой сцены. Есть функция `main`. Она вызвала функцию `A`. `A` вызвала `B`. `B` вызвала `C`. Исключение произошло внутри `C`. На уровне исходного кода можно представить цепочку так:

```
main() -> A() -> B() -> C()   // исключение произошло внутри C
```

Но для Windows это не просто четыре имени. Имена могут быть удалены, изменены, потеряны после оптимизации или вообще никогда не существовать в релизном бинарнике. Для системы это цепочка машинных состояний. В стеке текущего потока есть следы активных вызовов: где текущая функция хранит своё временное состояние, куда она должна вернуться, какие регистры сохранила, каким был стек у вызывающей функции.

Такую цепочку часто называют `call stack`, стек вызовов. Термин `call` означает вызов функции на машинном уровне: выполнение передаётся в другую функцию, а адрес возврата сохраняется, чтобы потом можно было вернуться. Термин `stack` означает стековую память потока, где эта вложенность вызовов получает физическое выражение. Поэтому `call stack` - это не список функций из отладчика как красивый отчёт, а логическая реконструкция текущей вложенности вызовов по машинному состоянию.

Самый верхний кадр - это функция, внутри которой поток находится сейчас. Если исключение произошло в `C`, текущий кадр принадлежит активному вызову `C`. Ниже находится кадр `B`,

потому что B вызвала C и ждала возврата. Ниже - A, ниже - `main`. Поиск обработчика начинается сверху, потому что ближайшая функция к месту возникновения исключения получает первый frame-based шанс. Если текущий кадр не имеет подходящего обработчика, система должна подняться к следующему кадру.

Слово `caller` означает вызывающая функция: та, которая сделала вызов. В цепочке `B -> C` функция B является caller-ом для C. Слово `callee` означает вызываемая функция: та, в которую управление перешло после `call`. В той же паре C является callee. Эти термины важны, потому что unwind почти всегда формулируется как переход от текущего callee-контекста к caller-контексту. `RtlVirtualUnwind` как раз вычисляет состояние функции, которая предшествует текущей, то есть состояние caller-а.

Если сказать живым языком: Windows стоит внутри C на месте исключения и пытается понять, можно ли решить проблему прямо здесь. Если нельзя, ей нужно подняться в B. Но она не может просто сказать “ну давай в B”, потому что в машинной реальности нет кнопки “следующий кадр”. Нужно вычислить, где находится кадр B, какой был RSP, какой был `RIP`, какие регистры должны иметь старые значения. Для этого и нужен unwind.

1.3.2. Почему обработчик привязан не просто к функции, а к кадру

Когда мы говорим “у функции есть обработчик”, это удобное сокращение, но технически оно может запутать. Обработчик связан не с абстрактной функцией как текстом, а с конкретным активным кадром выполнения этой функции.

Представь рекурсию: функция `parse_node` вызывает сама себя много раз. Код функции один и тот же. В PE-образе это один диапазон инструкций. Но в стеке одновременно может быть десять активных кадров `parse_node`. Если исключение произошло в самом глубоком вызове, Windows сначала рассматривает именно самый глубокий кадр. Потом предыдущий рекурсивный вызов. Потом ещё предыдущий. Все они принадлежат одному и тому же коду, но это разные живые состояния.

Это различие кажется философским, пока не начнёшь анализировать реальные исключения. Один и тот же адрес функции может встречаться в stack trace несколько раз. Это не ошибка отладчика. Это означает, что один и тот же код был активен несколько раз из-за рекурсии или повторной вложенности. Обработчик, если он есть у этой функции, может проверяться для каждого кадра как для отдельного участника цепочки.

Термин `frame-based exception handling` как раз говорит об этом: обработка, основанная на кадрах. Не “глобально по всему процессу”, не “один catch на всю программу”, а проверка обработчиков, связанных с конкретными кадрами текущего стека. Это отличается от `vectored exception handling`, где обработчики регистрируются на уровне процесса и вызываются по своим правилам, не будучи обычным кадром текущей цепочки вызовов.

Frame-based модель нужна для нормальной семантики языков. Если внутри глубокой функции возникает исключение, ближайший `__try/__except` или языковой обработчик должен иметь шанс раньше, чем более внешний уровень. Это соответствует вложенности программы: чем ближе обработчик к месту события, тем больше контекста он обычно имеет. Но чтобы соблюдать эту вложенность, системе нужно уметь ходить по кадрам сверху вниз.

1.3.3. Что у Windows есть в руках в момент начала поиска

В момент, когда exception dispatch logic начинает работу, у Windows есть несколько ключевых объектов. Их надо не просто перечислить, а понять роль каждого.

ExceptionRecord - запись исключения. Это данные о событии: код исключения, адрес, флаги, параметры. Если это access violation, параметры помогают понять характер доступа и проблемный адрес. Если это программное исключение, параметры могли быть переданы через RaiseException. Запись исключения отвечает на вопрос: что произошло и какие дополнительные данные описывают событие?

ContextRecord - запись контекста. Это снимок состояния процессора для потока. В ней лежат значения регистров, включая RIP и RSP. Она отвечает на вопрос: в каком машинном состоянии поток находился, когда возникло исключение? Для unwind это центральный объект, потому что раскрутка работает не с исходным кодом, а с контекстом. Когда система “поднимается” к caller-у, она фактически обновляет ContextRecord, превращая контекст текущего кадра в контекст предыдущего кадра.

RIP - instruction pointer. Это адрес инструкции, связанной с текущим выполнением. Для поиска обработчика он нужен как начальная точка: по RIP надо определить, внутри какой функции находится поток. Если Windows не знает функцию, она не знает, какие unwind-данные применять.

RSP - stack pointer. Это вершина стека текущего потока. Он нужен, потому что адрес возврата, сохранённые регистры и стековый кадр находятся относительно стека. Если RIP говорит “где мы в коде”, то RSP говорит “где мы в цепочке вызовов”.

ControlPc - управляющий адрес, или адрес управления. В документации Microsoft этот термин используется для адреса, по которому ищется запись функции. Практически это адрес внутри функции, где находится или откуда ушло управление. Важно: ControlPc - это не обязательно “начало функции”. Это точка внутри диапазона функции, по которой нужно найти соответствующую RUNTIME_FUNCTION.

ImageBase - база образа. PE-образ, EXE или DLL, загружается в виртуальное адресное пространство по некоторому базовому адресу. Многие структуры PE хранят не абсолютные виртуальные адреса, а RVA, то есть смещения относительно ImageBase. Поэтому когда RtlLookupFunctionEntry находит запись, она также возвращает базу модуля, к которому относится эта запись. Без ImageBase нельзя правильно превратить RVA из RUNTIME_FUNCTION или UNWIND_INFO в реальный VA.

FunctionEntry - запись таблицы функций. На x64 для обычной non-leaf функции это обычно указатель на RUNTIME_FUNCTION. Эта запись описывает диапазон кода функции и указывает, где лежит unwind-информация. FunctionEntry отвечает на вопрос: какая табличная запись описывает функцию, внутри которой сейчас находится ControlPc?

DispatcherContext - контекст диспетчера исключений для текущей функции. Это служебная структура, которую получает language-specific handler. Она содержит данные, помогающие обработчику понять текущий кадр, function entry, handler data и другую информацию, нужную для принятия решения. Если ExceptionRecord - это “что произошло”, а ContextRecord - “в каком состоянии”, то DispatcherContext - это “досье текущего кадра для обработчика”.

EstablisherFrame - адрес базы фиксированного стекового выделения для функции. Этот термин звучит мутно, но его смысл практический: handler-у нужно понимать, какой стековый кадр сейчас рассматривается и где находится его устойчивая база. Для

__C_specific_handler это один из ключевых параметров: он помогает соотнести языковые данные обработчика с конкретным кадром выполнения.

> На этом этапе важно не пытаться выучить поля структур как таблицу. Важнее понять роли: ExceptionRecord описывает событие, ContextRecord описывает машину, RIP/ControlPc привязывает событие к функции, FunctionEntry даёт карту функции, UNWIND_INFO объясняет кадр, DispatcherContext приносит всё это language-specific handler-у.

1.3.4. Первый шаг: по RIP найти функцию

Поиск frame-based обработчика начинается не с обработчика. Он начинается с вопроса: в какой функции мы вообще находимся?

Почему нельзя сразу спросить “есть ли handler”? Потому что handler привязан к функции или к данным функции. А чтобы узнать данные функции, нужно сначала найти саму функцию. На уровне x64 Windows делает это через function tables - таблицы функций.

Function table - таблица записей, каждая из которых описывает диапазон кода функции или части функции. Для PE-образа такие записи обычно находятся в .pdata. Каждая запись RUNTIME_FUNCTION содержит три важных поля: начало диапазона функции, конец диапазона функции и адрес unwind-информации. В x64-модели это не косметика, а официальная карта, по которой система ходит во время исключений и stack walking.

RtlLookupFunctionEntry - функция Windows, которая ищет в активных function tables запись, соответствующую заданному PC-адресу. PC здесь означает program counter, то есть адрес текущего управления. В x64-контексте роль program counter выполняет RIP. Функция получает ControlPc, возвращает RUNTIME_FUNCTION и сообщает ImageBase модуля. Microsoft описывает её именно как поиск записи активной таблицы функций по указанному PC value.

Слово active в active function tables тоже важно. Это не только статическая .pdata из EXE на диске. В процессе могут существовать статические function tables загруженных PE-модулей и динамические таблицы, зарегистрированные во время выполнения, например для JIT-кода или особых рантаймов. Поэтому Windows ищет не “в одном файле”, а в активном наборе таблиц, которые относятся к текущему процессу и его загруженным/зарегистрированным областям кода.

В живой картине это выглядит так. Исключение произошло при RIP = 00007FF7A0123456h. Windows должна понять, какому модулю принадлежит этот адрес. Если адрес попадает внутрь загруженного EXE, база может быть, например, 00007FF7A0000000h. Тогда RVA текущей точки будет примерно 00123456h. По этому смещению система ищет запись RUNTIME_FUNCTION, чей диапазон покрывает этот RVA.

```
VA текущей инструкции = 00007FF7A0123456h
ImageBase модуля      = 00007FF7A0000000h
RVA ControlPc         = 0000000000123456h

Теперь нужно найти RUNTIME_FUNCTION, для которой:
BeginAddress <= 00123456h < EndAddress
```

Именно поэтому различие VA, RVA и ImageBase нельзя оставлять туманным. VA - реальный виртуальный адрес в процессе. ImageBase - база загруженного образа. RVA - смещение внутри образа. RUNTIME_FUNCTION хранит диапазоны как RVA, а не как готовые абсолютные VA. Если анализатор перепутает эти системы координат, он будет искать функцию не там и дальше

развалит весь unwind-анализ.

1.3.5. RUNTIME_FUNCTION: запись, которая превращает адрес в функцию

RUNTIME_FUNCTION - одна из центральных структур Windows x64 unwind. Её легко недооценить, потому что на вид она маленькая: три поля. Но смысл у неё огромный. Это не “ещё одна структура в PE”. Это договор между скомпилированным кодом и Windows: если управление находится в таком-то диапазоне адресов, то раскручивать этот диапазон нужно по такой-то unwind-информации.

Первое поле - BeginAddress. Это RVA начала диапазона функции. Оно говорит: с какого смещения внутри образа начинается код, описываемый этой записью.

Второе поле - EndAddress. Это RVA конца диапазона. Обычно диапазон понимается как полуоткрытый: начало включается, конец не включается. То есть адрес принадлежит функции, если BeginAddress <= RVA < EndAddress. Такой формат удобен, потому что конец одной функции может совпадать с началом следующей без пересечения.

Третье поле - UnwindInfoAddress. Это RVA, указывающий на UNWIND_INFO в .xdata или в другом допустимом месте образа. Это поле не указывает на код функции. Оно указывает на метаданные, которые описывают, как функция строит свой стековый кадр и какие обработчики связаны с ней.

Вот почему RUNTIME_FUNCTION можно представить как табличку на двери функции: “этот коридор кода начинается здесь, заканчивается здесь, а схема эвакуации лежит вот там”. Когда случается исключение, Windows по RIP находит эту табличку и получает путь к unwind-карте.

Для reverse engineering важно проверять инварианты. BeginAddress должен быть меньше EndAddress. Диапазон должен попадать в разумную область образа. UnwindInfoAddress должен указывать на читаемые данные внутри образа или корректно зарегистрированной таблицы. Диапазоны не должны хаотично пересекаться. Если запись указывает за пределы образа, если unwind-информация лежит в секции с неожиданными правами, если диапазон функции перекрывает соседний диапазон, это не просто “нестандартно”. Это потенциальное нарушение контракта, по которому Windows должна ходить по стеку.

В твоём будущем static /dynamic такая запись является не только объектом парсинга, но и объектом оценки. Анализатор должен не просто прочитать три числа, а понять: эти числа образуют правдоподобную функцию? Они согласованы с PE-секциями? Согласованы с кодом? Согласованы с UNWIND_INFO? Согласованы с тем, что показывает дизассемблер?

1.3.6. Если запись не найдена: leaf function и опасность упрощения

Что делает Windows, если RtlLookupFunctionEntry не нашла запись для текущего ControlPc? В x64 unwind-процедуре такой код считается leaf function, то есть листовой функцией. Leaf function - функция, которая не вызывает другие функции и не требует полноценной unwind-информации. В минимальной модели она не выделяет сложный стековый кадр и не сохраняет nonvolatile-регистры так, чтобы системе понадобилась таблица для восстановления.

Почему leaf? Представь функцию, которая просто сложила пару регистров и вернулась. Она не делала `sub rsp, ...`, не сохраняла RBX, не вызывала никого. Для такой функции caller context можно восстановить просто: адрес возврата лежит на вершине стека по `[RSP]`. Значит, чтобы виртуально выйти из функции, достаточно взять этот адрес как новый `RIP` и увеличить RSP на 8 байт, будто был выполнен `ret`.

```
Упрощённая leaf-модель:
new RIP = [RSP]       // адрес возврата
new RSP = RSP + 8     // как после ret
```

Но тут важно не превратить правило в грубую догму. “Нет записи - значит leaf” - это правило unwinder-а для нормального x64-кода, но для реверсера оно ещё и повод задуматься. Запись может отсутствовать потому, что функция действительно leaf. А может отсутствовать потому, что код нестандартный, вручную написанный, JIT-сгенерированный без регистрации таблицы, упакованный, обфусцированный или повреждённый. Поэтому анализатор должен отличать “допустимое отсутствие” от “подозрительного отсутствия”.

Leaf function - термин не про “маленькая функция” в бытовом смысле. Это функция, которая в контексте unwind не требует отдельной карты для восстановления caller-а. Она может быть короткой, но главный критерий не длина, а влияние на стек и nonvolatile-регистры. Если функция изменила RSP, сохранила nonvolatile-регистр или имеет handler, она уже не может быть простой leaf в смысле безопасного табличного отсутствия.

Для обучения это место важно: Windows не сканирует стек в панике. Если нет function entry, она применяет конкретное правило leaf-return. Это правило основано на ABI-ожидании. Но reverse engineer всегда держит в голове: ожидание ABI может быть нарушено.

1.3.7. Если запись найдена: почему Windows делит функцию на prolog, body и epilog

Когда function entry найден, кажется, что можно сразу проверить handler. Но x64-модель аккуратнее. Windows должна понять, где внутри функции находится `RIP`: в прологе, в эпилоге или в теле функции. Эти зоны имеют разный смысл для исключений.

`Prolog`, или пролог функции, - начальный участок, где функция только строит свой кадр. Она может сохранять nonvolatile-регистры, уменьшать RSP, выделять стековое место, устанавливать frame pointer. В этот момент функция ещё не полностью вошла в своё нормальное тело. Если исключение произошло в середине пролога, часть действий уже выполнена, часть ещё нет. Поэтому unwinder не может просто применять полное описание кадра. Он должен отменить только те действия пролога, которые реально успели произойти до текущего `RIP`.

`Body`, тело функции, - основная область выполнения после завершения пролога и до начала эпилога. Здесь функция считается нормально активной. Именно в теле обычно имеет смысл проверять, есть ли у функции exception handler. Если код находится в теле и у `UNWIND_INFO` установлен флаг обработчика, language-specific handler может быть вызван для текущего кадра.

`Epilog`, или эпилог функции, - участок выхода. Он делает обратные действия: восстанавливает регистры, освобождает стек, выполняет `ret` или допустимый jump-выход. Если исключение произошло внутри эпилога, функция уже уходит. Microsoft x64 unwind procedure трактует это особо: для текущей функции не может быть handler-а, связанного с этим исключением, а

оставшаяся часть эпилога симулируется, чтобы получить контекст caller-а.

Почему это так важно? Потому что handler логически относится к активному телу функции. Если функция ещё не вошла в тело, её состояние не полностью установлено. Если функция уже выходит, она уже проходит обратный путь. В обоих случаях система не должна наивно вызывать обработчик так, будто функция спокойно исполнялась в середине тела.

Для определения пролога используется размер пролога, закодированный в `UNWIND_INFO`. Если расстояние от начала функции до `RIP` попадает в этот размер, unwinder понимает, что находится в prolog-зоне. Для эпилога механизм другой: unwinder анализирует поток инструкций от текущего `RIP` и проверяет, соответствует ли он допустимому паттерну эпилога. Это тонкий момент: эпилоги не обязаны иметь отдельные записи на каждый адрес, потому что их можно распознавать по ограниченным допустимым формам кода.

> Prolog и epilog - это не просто “начало” и “конец” в литературном смысле. Для Windows x64 это зоны с особыми правилами unwind. Пролог нужно частично отменять, эпилог нужно симулировать до выхода, а тело функции можно рассматривать как место, где handler имеет смысл.

1.3.8. UNWIND_INFO: почему поиск обработчика зависит от описания пролога

`UNWIND_INFO` - структура, которая описывает, как функция меняет стек и сохраняет важные регистры. На первый взгляд странно: мы ищем обработчик исключения, а говорим о прологе функции. Почему? Потому что чтобы найти обработчик выше по стеку, нужно уметь перейти к предыдущему кадру. А перейти к предыдущему кадру можно только если понимаешь, что текущая функция сделала со стеком.

Если функция в прологе сделала `push rbx`, значит старое значение RBX лежит на стеке. Если сделала `sub rsp, 80h`, значит выделила 128 байт стекового пространства. Если сохранила R12 по адресу `[rsp+30h]`, это тоже нужно знать. Если установила frame register, нужно понимать, как он соотносится с RSP. Все эти действия влияют на восстановление caller context.

`UNWIND_CODE` - элементы внутри `UNWIND_INFO`, которые кодируют отдельные действия пролога. Например, выделение небольшого блока стека, выделение большого блока, сохранение nonvolatile-регистра, push nonvolatile-регистра. Эти коды - не дизассемблированные инструкции один к одному, а компактная машинно-читаемая запись действий, которые важны для unwind.

Во время поиска обработчика система использует `UNWIND_INFO` двояко. Во-первых, она смотрит флаги: есть ли у функции exception handler или termination handler. Во-вторых, если нужно перейти к caller-у, она применяет unwind-коды, чтобы обновить `ContextRecord`. Поэтому одна и та же структура отвечает и за “есть ли у кадра обработчик”, и за “как выйти из кадра”.

Это объясняет, почему повреждение `.xdata` может ломать не только обработку исключений, но и stack trace. Если unwind-коды неправильные, Windows может неверно восстановить RSP или nonvolatile-регистры. Если флаги handler-а неправильные, система может попытаться вызвать несуществующий handler или пропустить существующий. Если `UnwindInfoAddress` указывает не туда, весь кадр становится непонятным.

1.3.9. Флаги UNWIND_INFO: почему handler не всегда один и тот же

В UNWIND_INFO есть флаги. Они говорят, что находится после массива unwind-кодов и какую роль играет эта дополнительная информация. Самые важные для текущей темы: UNW_FLAG_NHANDLER, UNW_FLAG_EHANDLER, UNW_FLAG_UHANDLER, UNW_FLAG_CHAININFO.

UNW_FLAG_NHANDLER означает, что у функции нет handler-а, который нужно вызывать. Это не значит, что функция бесполезна для unwind. Она всё равно может иметь unwind-коды, потому что нужно уметь восстановить caller-а. Просто в этой функции нет обработчика исключений или termination handler-а, участвующего в dispatch/unwind через эту запись.

UNW_FLAG_EHANDLER означает, что у функции есть exception handler. Такой handler может быть вызван в search phase, когда система ищет, кто готов обработать исключение. Например, для MSVC SEH это может приводить к вызову `__C_specific_handler`, который уже разбирает таблицы `__try/__except`.

UNW_FLAG_UHANDLER означает, что у функции есть termination handler, который важен во время unwind phase. Termination handler - это код очистки, который должен быть выполнен при раскрутке, даже если он не “ловит” исключение как `except`. В высокоуровневых терминах это ближе к finally/cleanup-логике: кадр покидается не обычным возвратом, и нужно выполнить завершение.

UNW_FLAG_CHAININFO означает chained unwind info, цепочную unwind-информацию. В этом случае после unwind-кодов лежит не обычный handler, а связанная RUNTIME_FUNCTION-подобная запись, которая ведёт к другой unwind-информации. Это используется для более сложного описания функций или частей функций, когда один кусок кода разделяет основную unwind-информацию с другим. Для начального понимания главное: CHAININFO меняет интерпретацию хвоста структуры. Если анализатор этого не учитывает, он может принять chained entry за handler address и построить полную чушь.

Флаги - маленькие биты, но они меняют смысл последующих байтов. Это общий урок бинарных форматов: одно и то же место в файле может означать разные структуры в зависимости от флагов. Поэтому анализатор должен читать не только поля, но и контекст их интерпретации.

1.3.10. Language-specific handler: почему Windows зовёт помощника языка

Если у функции установлен UNW_FLAG_EHANDLER, Windows может вызвать language-specific handler. Этот термин переводится как обработчик, специфичный для языка, но смысл глубже: Windows предоставляет общий механизм доставки исключения, а конкретный компилятор или runtime объясняет, как интерпретировать языковые конструкции этой функции.

Почему Windows не делает всё сама? Потому что Windows не обязана знать семантику каждого языка. Конструкция `__try/__except` в MSVC, C++ `try/catch`, деструкторы, фильтры, cleanup-блоки - всё это имеет правила, заданные компилятором и runtime-ом. ОС даёт каркас: вот исключение, вот текущий кадр, вот контекст, вот handler data. A language-specific handler решает, что это значит для данного языка и данной функции.

`__C_specific_handler` - важный пример такого handler-а для Microsoft C/SEH. Его адрес может быть указан через UNWIND_INFO, когда выставлены соответствующие флаги. Он получает `ExceptionRecord`, `EstablisherFrame`, `ContextRecord` и `DispatcherContext`. То есть ему дают не просто код ошибки, а полный набор данных о событии, текущем кадре и

текущей стадии обработки.

`ExceptionRecord` сообщает, что произошло. `ContextRecord` сообщает, в каком машинном состоянии это произошло или какой unwind-контекст сейчас рассматривается. `EstablisherFrame` даёт базу фиксированного стекового выделения текущей функции, чтобы handler мог соотнести свои данные с кадром. `DispatcherContext` передаёт служебную информацию диспетчера: function entry, image base, handler data и другие детали, нужные для интерпретации.

Живой смысл language-specific handler-а такой: Windows дошла до кадра и говорит: “У этого кадра заявлен handler. Я не буду сама гадать, какие тут `try`-области и фильтры. Вот тебе событие и контекст. Скажи, должен ли этот кадр обработать исключение или поиск нужно продолжать”.

Для reverse engineering это место очень важно. Если ты видишь `UNW_FLAG_EHANDLER`, это ещё не значит “вот прямо тут пользовательский catch”. Это значит, что у функции есть handler-механизм, а конкретный смысл надо выяснять через language-specific данные. Для MSVC SEH нужно смотреть, куда указывает handler, где лежат scope tables, какие диапазоны кода покрываются, какие filters/handlers связаны с ними. Это уже будущие главы, но направление должно быть ясно сейчас.

1.3.11. DispatcherContext: досье текущего кадра

`DispatcherContext` часто звучит как страшное служебное слово, которое хочется пропустить. Но если его объяснить человечески, он становится понятным: это набор данных, который диспетчер исключений передаёт handler-у, чтобы тот понимал, какой кадр сейчас рассматривается и где лежит его дополнительная информация.

Диспетчер исключений - это не один маленький объект, а логика Windows, которая занимается доставкой исключения. Он идёт по кадрам, вызывает нужные функции, обновляет контекст, проверяет результаты. На каждом кадре ему нужно передать language-specific handler-у “папку дела”: какая функция, какая база образа, какая запись функции, где handler data, какой control PC, какие границы, какой establisher frame.

Вот эту “папку дела” удобно воспринимать как `DispatcherContext`. Handler не должен заново искать всё с нуля. Windows уже нашла function entry и подготовила контекст текущего шага. Handler получает эти данные и применяет правила языка.

Почему это важно для твоего будущего анализатора? Потому что многие поля, которые выглядят “просто служебными”, на самом деле являются связующими мостами между слоями: PE-таблицами, unwind-информацией, кодом обработчика и стеком. Если ты анализируешь только `.pdata` и не понимаешь, как эти данные потом попадают в `DispatcherContext`, ты видишь только половину механизма.

1.3.12. Что handler может ответить

Когда language-specific handler вызывается в search phase, он должен принять решение. В упрощённой модели возможны три смысла ответа.

Первый смысл: “я не обрабатываю это исключение, продолжайте поиск”. Это соответствует идее `continue search`. Тогда Windows должна подняться к следующему кадру и повторить

процесс. Такой ответ нормален: не каждый handler обязан принимать каждое исключение. Фильтр может проверить код исключения и отказаться.

Второй смысл: “это исключение обработано этим кадром”. Тогда search phase нашла цель. Но передача управления обработчику не означает мгновенный хаотичный прыжок. Нужно перейти к unwind phase: аккуратно раскрутить промежуточные кадры и привести состояние к месту обработчика.

Третий смысл в некоторых моделях: “можно продолжить выполнение”. Это тонкий случай, связанный с продолжением из места исключения, например через `EXCEPTION_CONTINUE_EXECUTION`. Он опасен и возможен не всегда: продолжить можно только если причина исключения устранена или событие было таким, что продолжение допустимо. Например, breakpoint после обработки отладчиком может быть продолжен. А вот продолжать после настоящего access violation без исправления контекста и памяти бессмысленно: та же инструкция снова вызовет то же исключение.

Важно не запутаться: обработчик не просто “выполняет код”. Он возвращает системе решение о маршруте. Windows остаётся управляющим механизмом доставки. Handler участвует в принятии решения, но переходы по кадрам, восстановление контекста и фазы обработки держатся на системной логике.

1.3.13. Если текущий кадр не обработал исключение: виртуальный подъём к caller-у

Если у текущего кадра нет handler-а или handler сказал “continue search”, Windows должна перейти к следующему кадру. Здесь появляется ключевое слово `virtual` в `RtlVirtualUnwind`.

Virtual unwind - это не обычное выполнение инструкции `ret`. Поток не возвращается реально из функции обычным способом. Windows не выполняет пользовательский код функции, не идёт по её инструкциям назад. Она берёт `ContextRecord` и вычисляет, каким был бы контекст вызывающей функции, если бы текущая функция завершилась и управление вернулось caller-у.

Почему “виртуально”? Потому что раскрутка на этапе поиска - это моделирование. Система обновляет представление о контексте: новый `RIP` становится адресом возврата в caller-е, новый `RSP` соответствует стеку caller-а, nonvolatile-регистры восстанавливаются из мест, куда текущая функция их сохранила. Но это вычисление, а не нормальное выполнение инструкций функции назад.

`RtlVirtualUnwind` - функция, которая именно это и делает: по текущему контексту функции, `ImageBase`, `ControlPc` и `FunctionEntry` вычисляет invocation context предыдущей функции. В документации Microsoft она описана как получение контекста вызова функции, предшествующей заданному контексту функции.

С живой стороны это выглядит так. Windows стоит в кадре `C`. Handler не найден. Нужно перейти к `B`. Для этого `RtlVirtualUnwind` смотрит на `UNWIND_INFO` функции `C`: сколько стекового пространства выделено, какие регистры сохранены, есть ли frame pointer, где адрес возврата. После применения этих правил `ContextRecord` уже описывает не `C`, а `B`. Теперь search loop повторяется: новый `RIP` находится внутри `B`, значит надо искать `RUNTIME_FUNCTION` для `B`.

```
Кадр C не обработал исключение
    -> RtlVirtualUnwind(C)
    -> ContextRecord теперь описывает caller, то есть B
    -> RtlLookupFunctionEntry(new RIP)
    -> проверка кадра B
    -> если не обработал, подъём к A
```

Именно поэтому повреждение unwind-метаданных может приводить к неправильному stack trace. Если `UNWIND_INFO` говорит, что стек выделен на `0x40`, а реальный пролог выделил `0x80`, виртуальный подъём может взять адрес возврата не там. Дальше новый `RIP` окажется мусором, lookup не найдёт функцию или найдёт неправильную. В результате вся цепочка выше будет ложной.

1.3.14. Почему поиск обработчика - это цикл

Поиск обработчика по кадрам можно представить как цикл. Это не финальный код Windows, а концептуальная схема, которая помогает держать механизм в голове.

```
context = контекст места исключения

while True:
    controlPc = context.RIP
    functionEntry = RtlLookupFunctionEntry(controlPc, &imageBase, historyTable)

    if functionEntry == NULL:
        // считаем leaf function
        context.RIP = *(uint64_t*)context.RSP
        context.RSP += 8
        continue

    unwindInfo = imageBase + functionEntry->UnwindInfoAddress

    if controlPc находится в epilog:
        // симулируем остаток выхода из функции
        context = simulate_epilog(context)
        continue

    if controlPc находится в prolog:
        // отменяем только уже выполненные действия пролога
        context = undo_partial_prolog(context, unwindInfo)
        continue

    if unwindInfo имеет exception handler:
        decision = call_language_specific_handler(...)
        if decision == handled:
            break

    context = RtlVirtualUnwind(...)
    // теперь context описывает caller, цикл повторяется
```

Эта схема полезна, но нужно понимать каждую строчку. `RtlLookupFunctionEntry` - не магия, а поиск записи по текущему адресу. `functionEntry == NULL` - не “ничего не понятно”, а применение leaf-правила. Prolog/epilog - особые зоны, где handler текущей функции не рассматривается обычным образом. Body - зона, где handler может иметь смысл. `RtlVirtualUnwind` - не реальный возврат, а вычисление caller context.

Цикл заканчивается одним из нескольких путей. Либо handler найден и принял исключение. Либо обработчик разрешил продолжить выполнение. Либо поиск дошёл до верха стека и

никто не обработал исключение. Тогда событие становится необработанным, и Windows переходит к последним механизмам: top-level filter, Windows Error Reporting, завершение процесса, дамп и так далее. Эти поздние стадии будут отдельно, потому что сейчас нас интересует frame-based поиск.

1.3.15. Почему prolog и epilog ломают наивную картину

Наивная модель говорит: если RIP внутри функции, значит функция активна и можно проверять её handler. x64 unwind-процедура показывает, что это слишком грубо.

Если RIP в прологе, функция ещё строит кадр. Например, она уже сделала `push rbx`, но ещё не сделала `sub rsp, 40h`. Или наоборот, уже выделила стек, но ещё не сохранила все регистры. Если применить полное unwind-описание, как будто весь пролог выполнен, система восстановит неправильный контекст. Поэтому unwinder смотрит на offset текущего RIP от начала функции и отменяет только те действия, которые реально произошли.

Если RIP в эпилоге, функция уже выходит. Она может уже восстановить один регистр, но ещё не восстановить другой. Может уже сделать `add rsp, ...`, но ещё не выполнить `ret`. В такой зоне нельзя рассматривать обработчик текущей функции как обычный активный handler. Нужно симулировать оставшиеся действия эпилога, чтобы попасть к caller-у.

Для reverse engineering prolog/epilog важны ещё и потому, что они связывают машинные инструкции с `UNWIND_CODE`. Если в прологе видно `push rbx; sub rsp, 40h`, но unwind-коды описывают другое, это аномалия. Иногда она объяснима оптимизацией, chained unwind info или нестандартной генерацией. Иногда это признак повреждения или защиты. Но в любом случае анализ должен смотреть на соответствие: код функции и её метаданные должны рассказывать одну и ту же историю.

1.3.16. Чем поиск обработчика отличается от раскрутки стека

Сейчас мы говорим о том, как Windows ищет обработчик по кадрам. Это близко к `search phase`, фазе поиска. Но рядом есть `unwind phase`, фаза раскрутки, и их нельзя смешивать.

В search phase система в основном выясняет: какой кадр готов обработать исключение? Она виртуально поднимается по стеку, вызывает language-specific handlers в режиме поиска, получает ответы, продолжать или остановиться.

В unwind phase система уже знает, куда надо прийти, и начинает реально выполнять логику выхода из промежуточных кадров. Здесь важны termination handlers, cleanup, finally-логика, деструкторы. Это уже не просто “опрос”. Это контролируемое разрушение части цепочки вызовов, чтобы передать управление обработчику в согласованном состоянии.

Почему важно разделять? Потому что один и тот же language-specific handler может вызываться в разных контекстах: во время поиска и во время раскрутки. Его параметры похожи, но смысл стадии другой. В search phase он решает, подходит ли обработчик. В unwind phase он может выполнять завершение/cleanup. `__C_specific_handler` документация Microsoft прямо связывает с вызовом как частью поиска обработчика или как частью unwind.

Если смешать эти фазы, получится каша: будто при первом обнаружении `try` система сразу уничтожает все кадры. На самом деле сначала нужно найти цель. Только после этого можно раскручивать стек к этой цели. Эта двухфазность делает механизм управляемым.

1.3.17. Как это видит reverse engineer

Реверсер смотрит на тот же механизм не как пользователь программы, а как следователь. Пользователь видит: приложение упало. Разработчик видит: исключение в функции. Реверсер видит слои, которые должны совпасть.

Первый слой - машинная точка: RIP, инструкция, регистры, тип доступа. Здесь анализируется непосредственная причина.

Второй слой - PE-образ: какому модулю принадлежит RIP, какая секция содержит код, какой ImageBase, какой RVA. Здесь проверяется координатная система.

Третий слой - function table: есть ли запись RUNTIME_FUNCTION, покрывает ли она текущий RVA, корректны ли BeginAddress, EndAddress, UnwindInfoAddress. Здесь проверяется карта функции.

Четвёртый слой - unwind info: какие unwind-коды описаны, есть ли handler flags, совпадают ли действия с реальным прологом. Здесь проверяется возможность корректно выйти из кадра.

Пятый слой - handler layer: есть ли UNW_FLAG_EHANDLER или UNW_FLAG_UHANDLER, куда указывает handler, какие language-specific данные лежат после unwind-кодов, можно ли их разобрать. Здесь проверяется логика обработки.

Шестой слой - stack walking result: какую цепочку кадров даёт virtual unwind, где она обрывается, есть ли странные переходы, повторяющиеся кадры, адреса вне модулей, переходы в неисполняемые области. Здесь проверяется итоговая история выполнения.

Если все слои согласованы, картина надёжна. Если один слой врёт, нужно понять почему. Иногда это обычная оптимизация или leaf function. Иногда JIT. Иногда packer. Иногда антиотладка. Иногда повреждение стека. Иногда кривой parser. Поэтому хороший static не должен кричать “malware” при каждой странности. Он должен формулировать finding: что именно нарушено, на каком слое, какие доказательства, какие альтернативные объяснения.

1.3.18. Мини-модель для памяти

Чтобы не потеряться в деталях, держи в голове одну компактную модель. Исключение произошло в конкретном потоке. У потока есть ContextRecord. В нём есть RIP. По RIP Windows ищет RUNTIME_FUNCTION. Если запись найдена, она ведёт к UNWIND_INFO. Если RIP в теле функции и есть handler flag, вызывается language-specific handler. Если handler не принимает исключение, RtlVirtualUnwind превращает текущий контекст в контекст caller-а. Цикл повторяется.

```
ExceptionRecord + ContextRecord
        |
        v
      RIP / ControlPc
        |
        v
RtlLookupFunctionEntry
        |
        v
RUNTIME_FUNCTION + ImageBase
        |
        v
UNWIND_INFO / flags / codes
```

```
        |
        +--> language-specific handler, если он есть и место подходящее
        |
        v
RtlVirtualUnwind
        |
        v
caller ContextRecord
        |
        v
следующий кадр
```

Это не просто схема из учебника. Это центральный маршрут Windows x64 exception handling. В следующих главах мы будем увеличивать каждое звено: PE-файл объяснит, где физически лежат `.pdata` и `.xdata`; `RUNTIME_FUNCTION` покажет, как диапазоны функций кодируются в Exception Directory; `UNWIND_INFO` объяснит структуру unwind-кодов; `UWOP_*` покажут, как именно описываются действия пролога; `RtlVirtualUnwind` станет понятным не как загадочный API, а как формальная машина перехода между кадрами.

Главное сейчас: Windows ищет обработчик не по названиям функций и не по исходному коду. Она идёт по кадрам текущего потока, используя `RIP`, function tables и unwind-метаданные. Каждый технический термин в этой цепочке нужен не сам по себе, а потому что без него система не сможет уверенно ответить на вопрос: где мы сейчас, кто нас вызвал и есть ли на этом пути код, который имеет право обработать исключение?

Техническая база раздела 1.3

Этот раздел опирается на официальные материалы Microsoft по Windows x64 exception handling, unwind procedure, `RtlLookupFunctionEntry`, `RtlVirtualUnwind` и `__C_specific_handler`. Важные утверждения: x64 exception dispatch logic использует текущий `RIP` из context record для поиска `RUNTIME_FUNCTION`; если запись не найдена, код считается leaf function и возвратный адрес берётся с вершины стека; если запись найдена, `RIP` различается по положению в prolog, epilog или теле функции; language-specific handler вызывается при соответствующих флагах; `RtlVirtualUnwind` получает invocation context предыдущего кадра; `__C_specific_handler` вызывается компилятором для реализации SEH extensions и получает `ExceptionRecord`, `EstablisherFrame`, `ContextRecord`, `DispatcherContext`.

1.4. Чем x64-модель отличается от x86-модели

Теперь нужно разобрать различие, которое нельзя сводить к фразе “x64 - это просто 64-битный x86”. Для обычного программиста такое сокращение иногда терпимо: регистры шире, адресное пространство больше, компилятор генерирует другой код. Но для темы Windows exception handling и unwind это опасное упрощение. Между x86 и x64 в Windows меняется сама опора механизма обработки исключений: на x86 исторически важна стековая цепочка SEH-записей, а на x64 центральной становится табличная модель: PE Exception Directory, `.pdata`, `RUNTIME_FUNCTION`, `.xdata`, `UNWIND_INFO` и формальная раскрутка кадра через unwind-метаданные.

Эта разница важна не как историческая справка. Она объясняет, почему вся книга дальше уходит в PE-структуры. Если бы Windows x64 искала обработчики так же, как классический x86 SEH, нам пришлось бы в первую очередь читать цепочку записей на стеке. Но в x64 нормальный путь другой: по текущему адресу выполнения система ищет запись функции, по записи функции находит unwind-описание, а уже через него понимает, как двигаться по стеку и где искать handler.

> Главная идея раздела: x86-модель можно грубо представить как “handler-записи живут в стеке и образуют цепочку”, а x64-модель - как “описания функций и unwind живут в таблицах образа или dynamic function tables, а стек интерпретируется через эти описания”. Это не единственное различие, но именно оно главное для твоей темы.

1.4.1. Сначала уточним термины: x86, x64, Win32, Win64

Термины `x86`, `x64`, `Win32` и `Win64` часто смешивают, поэтому перед сравнением нужно развести их по слоям.

`x86` обычно означает 32-битную архитектуру семейства Intel 80386 и совместимых процессоров в контексте Windows-программ. В такой программе основные регистры общего назначения имеют 32-битный размер: `EAX`, `EBX`, `ECX`, `EDX`, `ESI`, `EDI`, `EBP`, `ESP`. Указатели обычно занимают 4 байта. Адрес возврата, сохранённый на стеке инструкцией `call`, тоже занимает 4 байта. Когда говорят “x86 SEH”, обычно имеют в виду 32-битный механизм Structured Exception Handling в Windows, где большое значение имеет цепочка exception registration records, расположенных на стеке потока.

`x64` в документации Microsoft обычно означает 64-битную Windows-архитектуру AMD64 / Intel 64. Это не Itanium и не ARM64. Это 64-битное расширение x86-линии, но в Windows оно имеет свой ABI, свои правила вызова функций, свой формат unwind-данных и свою модель обработки исключений. Регистры имеют 64-битные формы: `RAX`, `RBX`, `RCX`, `RDX`, `RSP`, `RBP`, `R8`-`R15`. Указатели в обычном 64-битном процессе занимают 8 байт. Адрес возврата на стеке тоже занимает 8 байт.

`Win32` и `Win64` - это не только “32-битная” и “64-битная” версии API. Исторически `Win32 API` может использоваться и в 64-битных программах как название семейства Windows API. Поэтому в техническом тексте лучше не полагаться на слово Win32 как на точный признак архитектуры. Если речь о механике исключений и стека, точнее говорить `x86 process` или `x64 process`.

`WOW64` - ещё один термин, который часто всплывает рядом. Это подсистема, позволяющая 32-битным Windows-приложениям работать на 64-битной Windows. В WOW64 32-битный

процесс выполняет x86-код и имеет свою 32-битную модель, хотя ОС под ним 64-битная. Для твоей книги базовая линия - именно native x64 PE32+ код, а не 32-битный процесс под WOW64. Иначе можно начать путать x86 SEH внутри 32-битного процесса с x64 unwind моделью 64-битного кода.

1.4.2. Почему это различие вообще появилось

Можно спросить: зачем Windows понадобилось менять модель? Почему нельзя было просто оставить старую x86-схему, только расширить адреса до 64 бит? Ответ лежит в сочетании нескольких причин: надёжность stack walking, оптимизации компилятора, безопасность, единообразие ABI и необходимость поддерживать исключения без обязательного frame pointer-а.

На x86 историческая модель SEH очень тесно связана со стеком. Когда функция входит в защищённый блок, компилятор может создать на стеке специальную запись обработчика. Такие записи связываются в список. Голова списка находится в thread environment data, доступной через сегментный регистр FS и адрес `FS:[0]`. Когда происходит исключение, система идёт по этому списку и вызывает handler-ы. Это рабочая модель, но она сильно зависит от корректности данных на стеке.

Стек - рабочая область программы. Он меняется постоянно. Его можно повредить переполнением буфера, ошибкой указателя, неправильной записью, эксплойтом. Если обработчики тоже описаны прямо в стеке, повреждение стека может стать не просто причиной падения, а способом повлиять на то, какой handler будет вызван. Именно вокруг этого исторически возникали техники hijacking-а SEH, когда злоумышленник пытался подменить запись обработчика и перенаправить управление.

x64-модель уходит от этой центральной зависимости. Вместо того чтобы хранить frame-based SEH-chain как набор изменяемых handler-записей в стеке, она использует таблицы функций и unwind-информацию. Эти данные обычно генерируются компилятором и линкером и лежат в PE-образе. Стек всё ещё критически важен: там адреса возврата, сохранённые регистры, локальные данные. Но сам маршрут “как понять кадр и где handler” задаётся не произвольной цепочкой записей в стеке, а таблицами.

Это даёт компилятору больше свободы. На x64 frame pointer не обязан быть постоянной опорой каждого кадра. Компилятор может использовать RBP как обычный регистр, оптимизировать размещение данных, строить разные варианты пролога и эпилога. Но взамен он должен честно описать действия, важные для unwind. Табличная модель говорит: “можешь оптимизировать код, но оставь системе формальное описание, как восстановить caller context”.

1.4.3. Классическая x86 SEH: цепочка записей на стеке

Теперь аккуратно разберём x86-сторону. SEH расшифровывается как Structured Exception Handling - структурированная обработка исключений. Слово “structured” здесь означает не “красиво оформленная”, а встроенная в системную модель Windows обработка, где исключения проходят через определённые записи, обработчики и правила.

В классической x86-модели при входе в защищённую область компилятор может разместить на стеке запись, которую часто называют exception registration record или SEH frame. Это небольшая структура, которая содержит ссылку на предыдущую запись и адрес handler-а. Если

записей несколько, они образуют linked list - связанный список. Связанный список означает, что каждый элемент знает, где лежит следующий элемент. Голова списка указывает на самый внутренний, ближайший к текущему месту выполнения обработчик.

На x86 Windows использует `FS:[0]` как указатель на текущую голову SEH-цепочки. FS - сегментный регистр. В обычном C-коде ты с ним почти не сталкиваешься, но на уровне Windows internals через FS в 32-битном процессе доступна структура, связанная с текущим потоком. Запись по смещению 0 указывает на первый exception registration record. Поэтому фраза `FS:[0]` в статьях про x86 SEH означает: “возьми из thread-specific области указатель на первый SEH-frame текущего потока”.

```
x86, очень упрощённо:

FS:[0] -> SEH record #1 -> SEH record #2 -> SEH record #3 -> ... -> конец

SEH record:
    Next     -> предыдущая / следующая запись в цепочке
    Handler  -> адрес функции-обработчика
```

Когда возникает исключение, система начинает с головы списка. Она вызывает handler текущей записи и передаёт ему данные об исключении. Handler может сказать: “я не обрабатываю, продолжайте поиск”. Тогда система переходит к следующей записи списка. Если handler говорит “обработано”, маршрут меняется: исключение считается принятым этим уровнем. Если список закончился, а никто не обработал исключение, оно становится необработанным.

Эта модель очень наглядна. Обработчики буквально сидят в стеке как цепочка. Но именно из-за этой наглядности она уязвима и ограничивающа. Если стек повреждён, цепочка может быть повреждена. Если злоумышленник может перезаписать handler pointer, он потенциально может повлиять на управление. Windows и компиляторы добавляли защиты вроде SafeSEH и SEHOP, но сама архитектурная идея x86 SEH всё равно остаётся стековой.

> SafeSEH - это защитная технология для 32-битных x86 образов, где в load configuration PE может храниться список допустимых SEH handler-ов. Смысл: если handler-адрес подменён на произвольный адрес внутри образа, система может проверить, входит ли он в список разрешённых. Для x64 это не центральный механизм, потому что x64 использует другую табличную модель unwind/exception handling.

1.4.4. Почему x86-цепочка плохо подходит как универсальная модель для x64

Стековая SEH-цепочка кажется простой, но у неё есть несколько проблем. Первая - зависимость от изменяемой памяти. Стек находится в процессе и активно используется программой. Если ошибка как раз связана с повреждением стека, система может потерять доверие к цепочке handler-ов. Но исключения часто возникают именно в аварийных состояниях. Механизм обработки должен быть максимально устойчивым в условиях частично повреждённого процесса.

Вторая проблема - оптимизация. Современный компилятор хочет свободно распоряжаться регистрами и стеком. Он может не использовать классический frame pointer, может раскладывать переменные по регистрам, может выкидывать ненужные записи, может объединять эпилоги, может делать tail call, inline и другие оптимизации. Если механизм исключений слишком сильно привязан к конкретной форме стека, компилятору приходится

либо ограничиваться, либо усложнять runtime-схему.

Третья проблема - stack walking. Отладчики, профилировщики и crash dump analysis должны уметь подниматься по стеку не только во время обработки `__try/__except`, но и просто для построения call stack. Если опираться на эвристики и записи в стеке, можно ошибиться: число похоже на адрес возврата, но на самом деле это локальная переменная; сохранённый указатель похож на code pointer, но это поле структуры; часть кадра повреждена, и цепочка ломается. Табличная модель уменьшает количество гадания.

Четвёртая проблема - безопасность. Если handler-цепочка хранит адреса обработчиков прямо в стеке, то переписывание стека может стать переписыванием маршрута обработки. Для старых exploit-техник это было важно. x64-модель не делает стек полностью безопасным, но переносит описание handler-ов и unwinding-а в более формальные структуры, генерируемые вместе с образом.

Поэтому в x64 Windows делает другой выбор: стек остаётся местом данных выполнения, но карта для интерпретации кадра хранится в таблицах функций. Эта карта говорит, где функция начинается и заканчивается, как она изменила стек, какие nonvolatile-регистры сохранила, где находится handler, если он есть.

1.4.5. x64: табличная модель вместо стековой SEH-цепочки

В x64-модели центральная точка - не `FS:[0]` и не linked list SEH-записей на стеке. Центральная точка - адрес выполнения, то есть `RIP`, и таблицы, которые позволяют по этому адресу найти описание функции.

Когда происходит исключение в x64-коде, у Windows есть `ControlPc` - адрес управления. В простом объяснении это адрес инструкции или близкая к нему точка, по которой система определяет текущую функцию. По этому адресу вызывается поиск function entry. Function entry - это запись, описывающая диапазон функции и ссылку на unwind-информацию. В PE-образе такие записи обычно лежат в секции `.pdata` и имеют форму `RUNTIME_FUNCTION`.

```
x64, очень упрощённо:

RIP / ControlPc
    -> поиск RUNTIME_FUNCTION в function table
        -> BeginAddress / EndAddress определяют диапазон функции
        -> UnwindInfoAddress указывает на UNWIND_INFO
            -> UNWIND_INFO описывает стек и сохранённые регистры
                -> RtlVirtualUnwind получает caller context
```

`RUNTIME_FUNCTION` - это не “запись для красивого отображения функции”. Это контракт между скомпилированным кодом и Windows. Поле `BeginAddress` задаёт RVA начала функции. Поле `EndAddress` задаёт RVA конца диапазона функции. Поле `UnwindInfoAddress` задаёт RVA структуры `UNWIND_INFO`. Все эти адреса являются RVA, то есть смещениями относительно `ImageBase`, а не file offset и не готовыми абсолютными адресами.

`UNWIND_INFO` - это компактное описание того, что функция сделала в прологе с точки зрения unwinding-а: сколько места выделила на стеке, какие nonvolatile-регистры сохранила, использовала ли frame register, есть ли exception handler или termination handler, где лежат дополнительные данные. Это не полный дизассемблированный текст функции. Это именно описание действий, которые нужно уметь обратить или учесть при восстановлении caller context.

`RtlVirtualUnwind` - системная функция, которая использует эти данные для перехода от текущего контекста к контексту вызывающей функции. Слово `Virtual` означает, что функция не выполняет реальный `ret` и не запускает код назад. Она моделирует, каким был бы контекст caller-а: новый RIP, новый RSP, восстановленные nonvolatile-регистры.

Так x64 превращает исключение из “пройти список handler-ов в стеке” в “по адресу найти формальное описание функции и по нему подниматься по кадрам”.

1.4.6. Где в x64 лежат данные: PE, Exception Directory, .pdata, .xdata

Теперь свяжем x64-модель с PE-файлом. PE - Portable Executable, основной формат исполняемых файлов и DLL в Windows. PE-файл описывает, какие секции есть у образа, где код, где данные, где импорты, где релокации, где ресурсы и где служебные каталоги. Когда Windows загружает EXE или DLL, она строит из файла загруженный image в виртуальной памяти процесса.

`Exception Directory` - один из data directories PE. `Data Directory` - это таблица указателей на важные служебные области образа. Для x64 Exception Directory указывает на массив записей, которые описывают функции для exception handling и unwinding-а. Обычно этот массив физически размещается в секции `.pdata`.

`.pdata` - секция, где находятся procedure data. В контексте Windows x64 это прежде всего массив `RUNTIME_FUNCTION`. Каждая запись говорит: “вот диапазон кода функции, а вот где лежит её unwind-информация”.

`.xdata` - секция, где обычно лежит unwind data: структуры `UNWIND_INFO`, массивы `UNWIND_CODE`, handler data, scope tables и связанные данные. Название можно запомнить так: `.pdata` - индекс функций, `.xdata` - подробности раскрутки и обработки.

RVA - relative virtual address, относительный виртуальный адрес. Это смещение от базы загруженного образа `ImageBase`. Почему не хранить сразу абсолютный адрес? Потому что из-за ASLR и релокаций образ может загрузиться по разной базе. Если таблица хранит RVA, то один и тот же файл остаётся корректным при разных базах: реальный VA вычисляется как `ImageBase + RVA`.

```
VA = ImageBase + RVA

Пример:
ImageBase = 00007FF700000000h
RVA       = 0000000000001234h
VA        = 00007FF700001234h
```

Эта адресная дисциплина - одно из мест, где x64-анализ часто ломается у новичков. `BeginAddress` в `RUNTIME_FUNCTION` - это не file offset. `UnwindInfoAddress` - тоже не file offset. Если анализатор возьмёт эти числа и пойдёт читать файл напрямую по такому смещению, он почти наверняка прочитает мусор. Нужно сначала перевести RVA в file offset через таблицу секций или, если анализ идёт в живом процессе, прибавить `ImageBase` и работать с VA.

1.4.7. Почему x64 не требует классического frame pointer-а

На x86 многие учебные объяснения стека строятся вокруг EBP. В типичном не оптимизированном коде функция делает что-то вроде `push ebp; mov ebp, esp`, и после этого EBP становится стабильной базой кадра. Локальные переменные лежат по `[ebp-...]`,

аргументы - по [ebp+...]. Такая схема очень удобна для человека и отладчика.

Но она не обязательна даже на x86, а на x64 тем более не является центральной. RBP на x64 может использоваться как frame pointer, но компилятор часто может использовать его как обычный nonvolatile-регистр. Если привязать unwinding только к цепочке RBP, оптимизации резко ограничатся.

x64-решение другое: frame pointer не обязан быть главным источником истины, потому что есть unwind metadata. Если функция выделила стек через sub rsp, 80h, сохранила rbx, использовала rbp как frame register или не использовала - это должно быть описано в UNWIND_INFO. Тогда Windows может восстановить caller context даже без красивой цепочки RBP -> previous RBP.

Frame register в UNWIND_INFO - это регистр, выбранный функцией как база для обращения к кадру, если такая база нужна. Это может быть RBP или другой допустимый регистр в рамках правил. Frame pointer - более общее привычное название для регистра, который стабильно указывает на область кадра. В x64 важно не путать: наличие RBP в коде ещё не гарантирует классический frame pointer, а отсутствие классического frame pointer-а не означает, что unwind невозможен.

> Для реверса это означает: на x64 нельзя ждать, что каждая функция будет начинаться с “push rbp; mov rbp, rsp”. Хороший анализ должен смотреть не только на паттерн пролога, но и на .pdata/.xdata, потому что именно там Windows получает формальное описание кадра.

1.4.8. Calling convention: на x64 меньше зоопарка, но больше строгих правил

Calling convention - соглашение о вызовах функций. Это набор правил, по которым caller и callee понимают друг друга: куда класть аргументы, где возвращать результат, кто очищает стек, какие регистры нужно сохранять, как выравнивать стек. Без calling convention объектные файлы, библиотеки и системные DLL не смогли бы вызывать друг друга надёжно.

На x86 в Windows исторически есть несколько популярных calling conventions: __cdecl, __stdcall, __fastcall, thiscall и другие. Они отличаются тем, кто чистит стек, как передаются аргументы, как выглядит имя символа, какие регистры используются. Для reverse engineering x86 это отдельный источник сложности: по прологу, эпилогу и инструкции после вызова приходится понимать, какая convention использована.

На Windows x64 ситуация унифицированнее. Есть основной x64 calling convention: первые четыре integer/pointer аргумента передаются через RCX, RDX, R8, R9; результат часто возвращается через RAX; caller резервирует shadow space; стек должен поддерживать нужное выравнивание; часть регистров volatile, часть nonvolatile. Это не значит, что x64 проще во всём. Просто сложность смещается: меньше зоопарка соглашений, больше строгой связи между calling convention, stack layout и unwind metadata.

Shadow space - заранее зарезервированная область на стеке под четыре регистровых аргумента. Даже если функция получила аргументы в RCX, RDX, R8, R9, caller всё равно выделяет место, куда callee при необходимости может сохранить эти аргументы. Shadow space также называют home space. Это не локальные переменные в обычном смысле и не “мусорная пустота”, а часть ABI-контракта.

`Stack alignment` - выравнивание стека. В x64-коде Windows важны правила 16-байтного выравнивания в определённых точках. Выравнивание нужно для корректной и эффективной работы инструкций, особенно SIMD/XMM, а также для ABI-совместимости. Если код вручную ломает выравнивание, последствия могут быть неочевидными: от падений до странных ошибок в вызванных функциях.

Эти правила важны для unwind потому, что unwind восстанавливает не абстрактную “стопку вызовов”, а состояние, которое подчиняется ABI. Если функция выделила shadow space, сохранила регистры, выровняла стек, вызвала другую функцию - всё это влияет на то, как читать стек и как возвращаться к caller-у.

1.4.9. Volatile и nonvolatile: почему x64 unwind интересуется не всеми регистрами

Ещё одно различие, которое нужно понимать глубоко, - деление регистров на volatile и nonvolatile. Эти слова часто переводят как “не сохраняемые” и “сохраняемые”, но лучше держать в голове ответственность.

`Volatile register` - регистр, значение которого callee может свободно испортить. Если caller хочет сохранить такое значение через вызов, caller сам должен его сохранить. В Windows x64 к volatile относятся, например, RAX, RCX, RDX, R8, R9, R10, R11 и часть XMM-регистров.

`Nonvolatile register` - регистр, значение которого callee обязан вернуть в исходное состояние, если он его использовал. В Windows x64 среди GPR к nonvolatile относятся RBX, RBP, RDI, RSI, RSP, R12-R15; также nonvolatile являются XMM6-XMM15. Это значит: функция может использовать RBX, но если она меняет его, она должна сохранить старый RBX и восстановить его перед возвратом.

Почему unwind интересуется именно nonvolatile-регистрами? Потому что они должны переживать вызов. Если исключение произошло внутри функции, и система хочет восстановить caller context, значения nonvolatile-регистров caller-а должны быть правильными. Если текущая функция сохранила старый RBX на стеке, unwind должен знать, где он лежит. Если она сохранила R12 по смещению от RSP, это тоже должно быть описано.

Volatile-регистры в такой же степени могут быть важны для анализа конкретной инструкции, но caller не ожидает, что callee сохранит их через вызов. Поэтому unwind metadata в основном описывает восстановление стека и nonvolatile state. Это объясняет, почему `UNWIND_CODE` содержит операции вроде сохранения nonvolatile-регистра, выделения стековой области, установки frame register, но не описывает каждую арифметическую инструкцию функции.

```
Пример идеи:

push rbx        ; функция сохраняет nonvolatile RBX
sub  rsp, 40h   ; функция выделяет место на стеке
...
add  rsp, 40h   ; функция освобождает стек
pop  rbx        ; функция восстанавливает RBX
ret             ; возврат caller-у

UNWIND_INFO должен описать push rbx и sub rsp, 40h,
потому что это нужно для восстановления caller context.
```

1.4.10. Пролог и эпилог: в x64 это не просто стиль компилятора

`Prolog` - начальный участок функции, где функция строит свой кадр. Обычно там сохраняются nonvolatile-регистры, выделяется стек, иногда устанавливается frame register. `Epilog` - завершающий участок, где функция разбирает кадр: восстанавливает регистры, возвращает RSP, выполняет `ret` или tail jump.

На x86 пролог часто узнаваем по классическому шаблону. На x64 шаблонов больше, но важнее другое: пролог и эпилог должны быть описуемы для unwind. Microsoft накладывает на x64 prolog/epilog строгие правила, потому что unwinder должен иметь возможность корректно обработать исключение или interrupt даже если поток остановился внутри функции или рядом с выходом из неё.

Почему эпилог важен? Представь, что исключение или interrupt произошло в момент, когда функция уже начала выходить: она уже сделала `add rsp, 40h`, но ещё не выполнила `ret`, или уже восстановила часть регистров. Если unwinder будет думать, что поток находится в обычном теле функции, он может применить неправильные правила. Поэтому x64 unwinder должен уметь распознавать эпилоги и симулировать оставшуюся часть выхода.

Это ещё одно отличие от простого “идти по EBP-цепочке”. x64-механизм обязан понимать состояние функции относительно пролога, тела и эпилога. `UNWIND_INFO` хранит размер пролога и коды, описывающие операции пролога. Эпилоги не кодируются тем же способом как список всех эпилогов в обычной версии unwind; они распознаются по строго ограниченным формам инструкций. Поэтому компилятор не может генерировать какой угодно хаос в эпилоге, если хочет, чтобы системный unwinder работал.

1.4.11. Leaf function и non-leaf function: почему не каждая функция имеет .pdata

`Leaf function` - листовая функция. Термин “leaf” взят из дерева вызовов: лист - узел, который никого дальше не вызывает. В контексте Windows x64 leaf function обычно означает функцию, которая не изменяет nonvolatile-регистры и не требует полноценного unwind-описания. Она может быть раскручена просто через имитацию `ret`: взять адрес возврата со стека и поднять RSP на 8 байт.

`Non-leaf function` - функция, которая вызывает другие функции или меняет состояние так, что для восстановления caller-а нужна unwind-информация: выделяет стек, сохраняет nonvolatile-регистры, использует exception handling, строит нетривиальный кадр. Для таких функций нужны записи в function table, то есть `.pdata`/`RUNTIME_FUNCTION`, и соответствующая `.xdata`/`UNWIND_INFO`.

Это важно для reverse engineering: отсутствие `RUNTIME_FUNCTION` для какого-то маленького кусочка кода не всегда означает ошибку или обфускацию. Это может быть leaf function или код, который допустимо раскрутить простым способом. Но если функция явно делает `sub rsp, 100h`, сохраняет `rbx`, вызывает другие функции, имеет обработчики, а записи нет, это уже подозрительно. Тогда stack walking и exception handling могут работать неправильно.

Для анализатора нужно различать “нет записи, потому что она не нужна” и “нет записи, хотя по коду она должна быть”. Это уже будущая тема про invariants и anomalies, но фундамент закладывается здесь.

1.4.12. Что происходит при исключении: x86 и x64 как две разные карты

Теперь сравним общий маршрут обработки исключения в двух моделях. Это не полный код Windows, а концептуальное различие.

```
x86, классическая идея:

исключение
  -> ExceptionRecord + ContextRecord
  -> голова SEH-chain из FS:[0]
  -> handler записи #1
  -> если continue search: следующая запись
  -> если найдено: обработка / unwind
  -> если конец: unhandled
```

В этой модели главным источником frame-based handler-цепочки являются stack-based exception nodes. Handler-адреса и ссылки на следующие записи находятся в стеке. Система идёт по linked list.

```
x64, базовая идея:

исключение
  -> ExceptionRecord + ContextRecord
  -> ControlPc / RIP
  -> RtlLookupFunctionEntry(ControlPc)
  -> RUNTIME_FUNCTION из function table
  -> UNWIND_INFO из .xdata
  -> language-specific handler, если он есть
  -> если continue search: RtlVirtualUnwind
  -> новый caller context
  -> повторить для следующего кадра
```

В x64 главным источником движения по кадрам являются function tables и unwind metadata. Стек всё ещё нужен, потому что там адрес возврата и сохранённые значения. Но система не идёт по списку handler-записей в стеке. Она интерпретирует стек через таблицы функций.

Эта разница меняет стиль реверса. На x86 ты часто смотришь на stack chain, registration records, SafeSEH, SEH overwrite. На x64 ты смотришь на `.pdata`, диапазоны `RUNTIME_FUNCTION`, корректность RVA, структуру `UNWIND_INFO`, операции `UWOP_*`, соответствие прологу, chained info, dynamic function tables.

1.4.13. Dynamic function tables: что делать, если кода нет в обычном PE .pdata

Табличная x64-модель могла бы показаться привязанной только к PE-файлам: есть модуль, есть `.pdata`, есть `.xdata`. Но в реальной жизни код может быть создан динамически. Например, JIT-компилятор генерирует машинный код в памяти во время выполнения. У такого кода нет заранее подготовленной секции `.pdata` внутри обычного PE-образа.

Для этого Windows x64 поддерживает dynamic function tables. `Dynamic function table` - динамическая таблица функций, зарегистрированная программой во время выполнения. API `RtlAddFunctionTable` позволяет добавить массив `RUNTIME_FUNCTION` для динамически созданного кода. `RtlInstallFunctionTableCallback` позволяет установить callback, который будет предоставлять function entries по запросу. Смысл тот же: если код существует вне обычного PE-образа, он всё равно должен дать Windows карту для unwind и stack walking.

Это очень важная деталь. Она показывает, что x64-модель не просто “данные лежат в .pdata”. Более общий принцип такой: на 64-битной Windows для unwind/walk stack нужны function tables. Обычно они приходят из PE-образа, но для dynamically generated code их нужно зарегистрировать отдельно.

Для reverse engineering и твоего будущего static /dynamic это значит: отсутствие нужной записи в статической `.pdata` не всегда означает, что unwind невозможен. Код может пользоваться dynamic function tables. Но для статического анализа PE это уже граница видимости: файл сам по себе может не содержать всей runtime-карты, если часть кода создаётся динамически.

1.4.14. Почему x64-модель лучше для отладчиков и crash dump analysis

Отладчик и crash dump analyzer хотят ответить на вопрос: “как поток дошёл до этого места?” Для этого нужен stack trace - цепочка вызовов. Stack trace не должен быть гаданием по словам из исходника. Он должен быть реконструкцией из машинного состояния.

На x86 без frame pointer-а или при повреждённом стеке stack walking часто становится эвристическим. Можно искать значения, похожие на return addresses, можно использовать символы, можно пытаться интерпретировать кадры по шаблонам, но уверенность может падать. x64 unwind metadata даёт более формальную основу: для non-leaf функций есть записи, которые описывают, как подняться к caller-у.

Конечно, x64-модель не делает дампы волшебно идеальными. Стек всё ещё может быть повреждён. Метаданные могут отсутствовать, быть нестандартными или намеренно искажёнными. Оптимизации могут inline-ить функции, и тогда “вызов” исчезает как отдельный кадр. Tail call может заменить обычный вызов/возврат. Но когда код скомпилирован нормально, `.pdata`/`.xdata` резко повышают качество stack walking.

Для crash dump analysis это особенно важно. В дампе процесс уже не выполняется. Нельзя “спросить программу”, что она хотела. Есть только память, регистры, модули и метаданные. Если unwind-данные корректны, анализатор может восстановить цепочку. Если нет - расследование начинает тонуть.

1.4.15. Почему x64-модель важна для защиты и обфускации

Для обычного приложения unwind-метаданные - служебная необходимость. Для reverse engineer-а - источник истины и источник подозрений. Для защитника или обфускатора - ещё один слой, с которым можно играть.

Если `.pdata` содержит диапазоны функций, она помогает инструментам видеть границы кода. Это полезно отладчику и дизассемблеру. Но защитный код может пытаться усложнить картину: нестандартные диапазоны, смешение кода, ручная генерация function tables, минимизация metadata, динамический код, chained info, странные прологи, расхождение между дизассемблированным прологом и `UNWIND_CODE`.

Важно не уходить в паранойю. Не всякая странность - защита. Компилятор и оптимизатор тоже генерируют сложный код. Link-time optimization, hot/cold splitting, incremental linking, C++ exceptions, security instrumentation, CFG, EH continuation metadata, JIT - всё это может делать картину сложнее. Поэтому анализатор должен не просто кричать “аномалия”, а понимать

классы допустимых отклонений и нарушений контракта.

Но базовая мысль остаётся: x64 unwind metadata - это не мусор рядом с кодом. Это активная часть выполнения и диагностики. Если метаданные противоречат коду, это влияет не только на красоту анализа, но и на способность Windows корректно обрабатывать исключения, строить stack trace и раскручивать стек.

1.4.16. Табличная модель как контракт

Лучше всего понимать отличие x64 от x86 через слово “контракт”. Контракт - это набор обязательств между сторонами. В нашей теме сторонами являются компилятор, линкер, PE loader, Windows exception dispatcher, unwinder, отладчик и сам скомпилированный код.

Компилятор обещает: если функция требует unwind-описания, я сгенерирую корректный `RUNTIME_FUNCTION` и `UNWIND_INFO`. Пролог будет описуемым. Эпилог будет соответствовать правилам. Сохранённые nonvolatile-регистры будут отражены в unwind-кодах. Если есть handler, флаги и handler data будут размещены правильно.

Линкер обещает: я соберу эти данные в PE-образ, сформирую Exception Directory, размещу `.pdata/.xdata`, обеспечу корректные RVA и выравнивания.

Loader обещает: я загружу образ в память, применю relocations, создам правильный image layout, чтобы RVA можно было интерпретировать относительно `ImageBase`.

Windows exception dispatcher и unwinder обещают: если данные корректны, я смогу по `RIP` найти функцию, по функции найти unwind info, восстановить caller context и доставить исключение по кадрам.

Отладчик и dump analyzer используют тот же контракт, чтобы строить stack trace. Reverse engineer проверяет этот контракт и пытается понять, где код ему следует, где нарушает, а где намеренно маскируется.

> Контрактная модель - самый полезный способ мыслить для твоего проекта. Твой инструмент должен видеть не просто “байты в секции”, а цепочку обязательств: поле указывает туда-то, адрес должен быть RVA, диапазон должен попадать в image, unwind-коды должны соответствовать прологу, handler data должны интерпретироваться по флагам.

1.4.17. Что x64 меняет в мышлении реверсера

Если ты пришёл из общего C/ASM уровня, x64 unwind может выглядеть как лишняя бюрократия. Но на самом деле он даёт сильный аналитический рычаг.

На x86 ты мог часто думать: “найду пролог, найду EBP, посмотрю стек, пойму функцию”. На x64 нужно думать шире: “где `RIP`, есть ли `RUNTIME_FUNCTION`, какой диапазон функции, куда ведёт `UnwindInfoAddress`, какие `UWOP_*` описаны, соответствует ли это машинному прологу, есть ли handler flags, что будет делать `RtlVirtualUnwind`”.

Это не значит, что дизассемблер больше не нужен. Наоборот, нужно связывать два мира: код и метаданные. Код показывает, что реально выполняется. Метаданные показывают, что компилятор/линкер заявили Windows. Если они совпадают, картина крепкая. Если нет, начинается исследование.

Например, если `UNWIND_INFO` говорит, что функция сохранила `RBX` через `UWOP_PUSH_NONVOL`, а в прологе действительно есть `push rbx`, это нормальная связка. Если `UNWIND_INFO` говорит

о выделении 0x80 байт стека, а код делает sub rsp, 40h, надо разбираться: ты правильно нашёл начало функции? Это chained info? Это shrink wrapping? Это другой prolog? Или metadata повреждены?

Такой стиль мышления и есть взрослый reverse engineering: не верить одному источнику, а сопоставлять слои.

1.4.18. Итог главы 1: что мы теперь понимаем

Первая глава начиналась с простого вопроса: что вообще происходит при исключении? Теперь у нас есть полный первый слой ответа.

Исключение - это событие, которое прерывает обычный control flow и запускает механизм доставки. Оно возникает в конкретном потоке и связано с context record: RIP, RSP, регистрами и состоянием памяти. Чтобы найти обработчик, Windows должна двигаться по кадрам текущего потока. На x64 она делает это не через классическую x86-цепочку SEH-записей на стеке, а через табличную модель function tables и unwind metadata.

x86 важен как исторический контраст: там SEH часто объясняется через stack-based exception nodes и FS:[0]. x64 важен как основная тема книги: здесь ключом становятся PE Exception Directory, .pdata, RUNTIME_FUNCTION, .xdata, UNWIND_INFO, prolog/epilog restrictions, volatile/nonvolatile registers, calling convention и RtlVirtualUnwind.

Если сформулировать максимально коротко, но уже технически правильно:

> В x86 SEH обработчик исторически ищется через стековую цепочку registration records. В Windows x64 обработка и stack unwinding опираются на таблицы функций и unwind-метаданные, которые связывают диапазоны кода с правилами восстановления caller context.

Эта мысль переводит нас к следующей главе. Если x64-модель держится на таблицах, нужно понять, где эти таблицы физически живут. Ответ: в PE-образе. Поэтому дальше мы переходим к базе PE: что такое PE-файл, что такое PE32+, как устроены секции, чем отличаются RVA, VA и file offset, и как через Data Directory находится Exception Directory.

Глава 2. База PE: в каком файле всё это живёт

2.1. Что такое PE-файл и как он устроен в общем виде

В первой главе мы пришли к важной мысли: Windows x64 не ищет обработчики исключений “на глаз” и не раскручивает стек просто потому, что в памяти стека нашлось число, похожее на адрес возврата. Для нормального x64-кода система опирается на табличную модель. По текущему адресу выполнения она должна найти описание функции, по описанию функции - данные для unwind, а по ним - восстановить состояние вызывающего кадра. Значит, дальше неизбежно появляется вопрос: где эти таблицы физически живут? Ответ начинается с PE-файла.

Эта глава не будет “справочником полей”. Справочник можно открыть отдельно, но он плохо учит думать. Наша задача другая: построить в голове живую модель. Нужно понять, почему Windows вообще нужен специальный формат исполняемого файла, почему EXE на диске и программа в памяти - не одно и то же, зачем нужны заголовки, секции, адреса, таблицы, каталоги данных и почему ошибка в понимании PE сразу ломает понимание `.pdata`, `.xdata` и unwind.

> Правило этой главы: каждый новый технический термин вводится медленно. Сначала перевод и человеческий смысл, потом назначение, место в системе, участники, связь с другими понятиями и типичная ошибка.

2.1.1. PE file (Portable Executable file, переносимый исполняемый файл): не “просто exe”, а инструкция для Windows

PE `file` (Portable Executable file, переносимый исполняемый файл) - это основной формат исполняемых образов Windows. В него упакованы EXE-файлы, DLL-файлы и некоторые другие исполняемые компоненты. Слово “portable” здесь не означает “флешка” или “можно носить с собой”. Исторически оно означает, что формат не привязан к одной конкретной процессорной архитектуре: внутри PE могут быть разные варианты машинного кода и разные настройки для разных архитектур. Для нашей книги главная линия - Windows x64, то есть 64-битный PE32+ образ.

По-человечески PE-файл можно понимать как файл-инструкцию для Windows. Это не просто мешок байтов, где “где-то лежит код”. Windows должна знать, какие байты являются кодом, какие данными, какие таблицами, какие строками, какие адресами импортируемых функций, какие unwind-метаданными, какие релокациями. Если бы EXE был просто сплошной последовательностью инструкций, загрузчик не знал бы, где начинать выполнение, какие DLL подключать, какие участки памяти делать исполняемыми, какие - только читаемыми, какие таблицы нужны для исключений.

Поэтому PE-файл содержит не только содержимое программы, но и карту этого содержимого. В этой карте описано: где начинается важный заголовок, какие секции есть в файле, где находится точка входа, какая база образа желательна, какие DLL импортируются, есть ли таблица исключений, где лежат данные для TLS, релокации, ресурсы и многое другое.

Главная ошибка новичка - думать, что EXE на диске уже является “программой в готовом виде”. На самом деле EXE на диске - это заготовка. Windows должна прочитать его формат и превратить его в загруженный образ процесса. Именно в этом превращении и скрывается большая часть PE-логики.

2.1.2. Executable file (исполняемый файл) и running process (запущенный процесс): файл ещё не выполняется

`Executable file` (исполняемый файл) - это файл на диске, который содержит достаточно информации, чтобы операционная система могла создать выполнение. В Windows это часто `.exe`, но расширение само по себе не магия. Важен формат внутри файла. Если внутри нет корректной PE-структуры, Windows не сможет нормально загрузить файл как программу, даже если назвать его `program.exe`.

`Running process` (запущенный процесс) - это уже не файл, а объект операционной системы: адресное пространство, загруженные модули, потоки, стеки, куча, дескрипторы, таблицы, состояние выполнения. Файл лежит на диске. Процесс живёт в памяти и управляется Windows. Это разные состояния одной программы.

Файл можно закрыть в hex-редакторе, скопировать, отправить другу, посчитать хэш. Процесс нельзя “скопировать” так же просто: у него есть живые потоки, выделенная память, открытые файлы, сетевые соединения, состояние регистров. Когда ты запускаешь EXE, Windows создаёт процесс и отображает в его адресное пространство основной образ EXE и необходимые DLL.

Для reverse engineering различие критичное. Когда ты анализируешь файл статически, ты видишь file layout (файловую раскладку): байты на диске и смещения от начала файла. Когда ты отлаживаешь процесс, ты видишь virtual layout (виртуальную раскладку): адреса в памяти процесса после загрузки. Одни и те же данные могут иметь разные координаты в этих двух мирах.

> Запомнить просто: файл отвечает на вопрос “что лежит на диске?”, процесс отвечает на вопрос “что реально загружено и выполняется сейчас?”. PE - мост между этими мирами.

2.1.3. Loader (загрузчик): кто превращает PE-файл в загруженный образ

`Loader` (загрузчик) - это часть Windows, которая берёт исполняемый файл и подготавливает его к выполнению. Загрузчик читает PE-заголовки, отображает секции в память, подключает нужные DLL, настраивает импорты, применяет релокации, создаёт начальное состояние процесса и передаёт управление в точку входа.

Важно: loader (загрузчик) не является “программой, которая просто копирует exe в RAM”. Его работа гораздо умнее. Он не обязан переносить байты файла один к одному. В PE есть различие между тем, как данные лежат на диске, и тем, как они должны выглядеть в памяти. Секция может занимать в файле один размер, а в памяти - другой. Некоторые области могут быть заполнены нулями уже при загрузке. Права доступа к страницам памяти задаются по характеристикам секций. Импортируемые адреса заполняются уже во время загрузки.

Загрузчик можно представить как строителя, который получил чертёж. PE-файл - это не готовое здание, а набор материалов и инструкция. Headers (заголовки) говорят строителю, где что находится и как это разложить. Sections (секции) - крупные блоки материалов. Data directories (каталоги данных) - указатели на специальные системы здания: импорт, экспорт, исключения, релокации. Loader строит в памяти image (образ), пригодный для выполнения.

Для темы unwind loader важен потому, что `.pdata` и `.xdata` имеют смысл именно в загруженном образе. `RUNTIME_FUNCTION` хранит адреса как RVA, то есть относительные адреса

внутри image. Пока мы не понимаем, что loader делает из файла image, мы не можем правильно интерпретировать эти адреса.

2.1.4. Linker (линкер): кто собирает PE до запуска

`Linker` (линкер, компоновщик) - это инструмент сборки, который берёт объектные файлы и библиотеки и собирает из них итоговый PE-файл. Если compiler (компилятор) превращает исходный код в объектные куски машинного кода и данных, то linker соединяет эти куски в один исполняемый образ или DLL.

Зачем нужен linker (линкер)? Потому что программа редко состоит из одного цельного куска. У тебя есть несколько `.c` или `.cpp` файлов, системные библиотеки, runtime-библиотеки, импортируемые функции, глобальные данные, таблицы исключений. На этапе компиляции каждый объектный файл может ещё не знать окончательные адреса всего. Linker распределяет код и данные по секциям, связывает ссылки между ними, создаёт таблицу импортов, формирует заголовки, записывает точку входа, строит `.pdata` и `.xdata` для x64-функций, если это требуется.

Поэтому PE-файл - результат договора между compiler (компилятором) и linker (линкером). Компилятор генерирует машинный код и unwind-информацию для отдельных функций. Линкер собирает это в единую карту образа. Загрузчик потом читает эту карту. Windows exception machinery использует её во время выполнения. Reverse engineer проверяет, насколько все эти слои согласованы.

Типичная ошибка - думать, что все данные PE “написала Windows”. Нет. Большая часть PE-структур создаётся инструментами сборки: компилятором и линкером. Windows loader их потребляет. Если файл сделан вручную, упакован, модифицирован или испорчен, loader и анализаторы могут столкнуться с нестандартной картиной.

2.1.5. Image (образ): PE в памяти уже не совсем тот же файл

`Image` (образ) - это представление исполняемого файла после загрузки в память. В документации Microsoft PE часто называется image file (файл образа), потому что он предназначен для отображения в память как исполняемый образ. Но в разговоре полезно различать file on disk (файл на диске) и loaded image (загруженный образ).

Loaded image (загруженный образ) - это то, что видит процесс в своём виртуальном адресном пространстве после работы loader-а. У образа есть base address (базовый адрес), секции с виртуальными адресами, права доступа, заполненные импорты, применённые релокации. Когда CPU выполняет инструкцию из `.text`, он выполняет её не “из файла”, а из памяти загруженного образа.

Почему это важно для RVA (Relative Virtual Address, относительный виртуальный адрес)? Потому что RVA считается относительно базы образа в памяти. Если в таблице написано RVA `0x2000`, это означает: “найди base address загруженного image и прибавь 0x2000”. Это не обязательно file offset `0x2000` на диске. На диске эти байты могут лежать совсем в другом месте.

Для unwind это становится критичным. `RUNTIME_FUNCTION` содержит поля `BeginAddress`, `EndAddress`, `UnwindInfoAddress`. Это не абсолютные адреса и не file offsets. Это RVA внутри image. То есть Windows читает их как координаты загруженного образа. Если анализатор перепутает image-координаты с file-координатами, он будет искать функцию и `UNWIND_INFO` не

там.

2.1.6. Module (модуль): один загруженный PE-образ внутри процесса

`Module` (модуль) - это загруженный PE-образ внутри процесса. Обычно модулем называют загруженный EXE или DLL. У процесса есть основной module (главный модуль) - EXE, с которого начался запуск. Также есть modules (модули) DLL: системные библиотеки, runtime, плагины, сторонние зависимости.

Почему термин module (модуль) нужен отдельно от image (образ)? Image подчёркивает представление PE в памяти. Module подчёркивает роль этого образа как участника процесса: у него есть base address, путь к файлу, exports, imports, sections, exception directory. Отладчики часто показывают список модулей процесса: `kernel32.dll`, `ntdll.dll`, `user32.dll`, твой `app.exe` и так далее.

Когда происходит исключение, адрес `RIP` принадлежит какому-то module (модулю). Например, `RIP` может указывать внутрь твоего EXE, внутрь `ucrtbase.dll`, внутрь `ntdll.dll` или внутрь динамически сгенерированного кода. Для обычного PE-модуля Windows может найти function table в его Exception Directory. Поэтому вопрос “какому модулю принадлежит адрес?” - первый шаг к поиску unwind-данных.

Типичная ошибка: думать о процессе как об одном EXE. На деле процесс - это набор загруженных модулей плюс динамическая память. Stack trace почти всегда прыгает между модулями: твоя функция вызвала runtime, runtime вызвал Windows API, Windows API ушёл в `ntdll`. Для каждого модуля действуют свои PE-таблицы.

2.1.7. Header (заголовок): служебное описание, без которого байты не имеют карты

`Header` (заголовок) - это служебная область файла, которая описывает, как читать остальные байты. Заголовок не является “кодом программы” в обычном смысле. Он похож на оглавление, паспорт и технический чертёж одновременно. В нём записано: что это за файл, где находятся следующие структуры, сколько секций есть, для какой архитектуры собран файл, где точка входа, какие выравнивания используются, где каталоги данных.

Почему заголовки нужны? Потому что без них loader (загрузчик) не знает, где заканчивается одна часть и начинается другая. Представь книгу без оглавления, нумерации страниц и названий глав. Ты можешь видеть буквы, но не понимаешь структуру. PE header делает байты осмысленными для системы.

В PE есть несколько уровней заголовков. Самый ранний - DOS header (DOS-заголовок), который начинается с сигнатуры `MZ`. Затем через поле `e_lfanew` он указывает, где находится современный PE-заголовок. Дальше идёт PE signature (PE-сигнатура), COFF File Header (COFF-заголовок файла), Optional Header (дополнительный заголовок, хотя для исполняемого image он фактически обязателен), затем Section Table (таблица секций).

Сейчас не нужно запоминать все поля. Главное - понять роль. Headers (заголовки) - это карта. Sections (секции) - это крупные области содержимого. Data directories (каталоги данных) - указатели на специальные таблицы внутри этих областей. Если заголовки повреждены или намеренно искажены, анализатор может неправильно понять весь файл.

Термин “optional header” часто сбивает. Optional Header (дополнительный заголовок) называется optional исторически в контексте COFF object files, но для PE image он содержит критически важные поля: ImageBase, AddressOfEntryPoint, SectionAlignment, FileAlignment, SizeOfImage, DataDirectory. Для EXE/DLL он не является “необязательным” в практическом смысле.

2.1.8. Signature (сигнатура): короткий маркер “что это за структура”

`Signature` (сигнатура) - это узнаваемый маркер в байтах, по которому программа-парсер понимает, что перед ней определённая структура. В PE есть несколько таких маркеров. Самый известный - `MZ` в начале файла. Другой важный маркер - PE□□, то есть PE signature (PE-сигнатура), которая стоит перед основными PE-заголовками.

Сигнатура сама по себе не доказывает, что файл полностью корректен. Если первые два байта равны `MZ`, это ещё не значит, что весь файл является рабочим PE. Но сигнатура даёт начальную точку: парсер понимает, какой формат пытаться читать дальше.

Почему `MZ` вообще есть в современных Windows PE? Это историческое наследие DOS. В начале PE-файла лежит DOS header и маленькая DOS stub program (DOS-заглушка). Обычно она печатает сообщение вроде “This program cannot be run in DOS mode”. Современной Windows эта заглушка нужна не ради DOS, а потому что PE-формат исторически встроен поверх старой структуры. Главное поле DOS header для нас - `e_lfanew`, которое указывает смещение до PE header.

`e_lfanew` - это file offset (смещение в файле) до PE signature. Здесь полезно сразу заметить: это именно смещение в файле, а не RVA. Парсер читает файл на диске, начинает с нуля, берёт `e_lfanew`, прыгает к этому месту и ожидает увидеть PE□□. Такая маленькая деталь уже показывает, что PE постоянно смешивает разные координатные системы, и их нужно различать.

2.1.9. Section (секция): крупная область файла с назначением

`Section` (секция) - это крупная область PE/COFF-файла, в которой лежит код или данные определённого назначения. Секция - не обязательно “часть программы” в высокоуровневом смысле. Это единица компоновки и загрузки: линкер группирует туда байты, а loader отображает их в память с нужными правами.

По-человечески секцию можно понимать как контейнер с ролью. `.text` обычно содержит машинный код. `.rdata` часто содержит read-only data (данные только для чтения): строковые литералы, константы, таблицы. `.data` содержит изменяемые глобальные данные. `.pdata` на Windows x64 обычно содержит procedure data (данные процедур), то есть массив `RUNTIME_FUNCTION`. `.xdata` содержит расширенные данные, в том числе `UNWIND_INFO`. `.reloc` содержит relocation data (данные релокаций).

Но имена секций - это соглашение, а не абсолютная истина. PE loader в первую очередь смотрит не на красивое имя `.text`, а на поля section header (заголовка секции): virtual address, raw pointer, размеры, characteristics. Пакер или нестандартный линкер может назвать секцию как угодно. Код может лежать не только в `.text`, данные - не только в `.data`. Поэтому reverse engineer не должен слепо верить имени. Имя полезно, но права доступа, адреса, содержимое и ссылки важнее.

Почему секции важны для unwind? Потому что `.pdata` и `.xdata` обычно являются секциями или частями секций, где лежит информация для exception handling x64. Когда Exception Directory указывает на таблицу функций, фактически он ведёт к данным, которые в нормальном PE32+

образе находятся в `.pdata`. Эти записи ссылаются на unwind-информацию, обычно в `.xdata`. Значит, понимание секций - прямой шаг к пониманию `RUNTIME_FUNCTION` и `UNWIND_INFO`.

2.1.10. Section Header (заголовок секции): карточка секции

`Section Header` (заголовок секции) - это запись в таблице секций, которая описывает одну секцию. Если section (секция) - это сам контейнер байтов, то section header (заголовок секции) - карточка этого контейнера. В карточке написано имя секции, где она должна находиться в памяти, где её данные лежат в файле, какой у неё размер и какие права/свойства у неё есть.

Самая важная идея: секция имеет две жизни - file life (жизнь в файле) и memory life (жизнь в памяти). В файле секция лежит по PointerToRawData (указатель на сырые данные в файле) и занимает SizeOfRawData (размер сырых данных). В памяти она отображается по VirtualAddress (виртуальный адрес секции относительно ImageBase, фактически RVA начала секции) и имеет VirtualSize (виртуальный размер).

Эти размеры могут не совпадать. Например, секция `.bss` или часть `.data` может иметь данные, которые в памяти должны быть нулями, но на диске их не нужно хранить полностью. Loader выделит память и заполнит недостающую часть нулями. Поэтому нельзя считать, что “размер секции в файле” всегда равен “размеру секции в памяти”.

Characteristics (характеристики) секции описывают свойства: содержит ли секция код, содержит ли инициализированные данные, доступна ли для чтения, записи, выполнения. Эти флаги помогают loader-у выставить права страниц памяти. Например, кодовая секция обычно executable/readable, но не writable. Данные могут быть readable/writable, но не executable. Для безопасности это важно: если данные не должны выполняться, отсутствие execute-права помогает ловить попытки исполнения данных.

Типичная ошибка при ручном анализе PE - видеть RVA и пытаться сразу читать файл по такому же числу. Чтобы найти байты по RVA в файле, нужно найти секцию, чей виртуальный диапазон содержит этот RVA, а потом перевести RVA через поля section header в file offset. Без section headers такой перевод невозможен.

2.1.11. Raw layout (файловая раскладка) и virtual layout (виртуальная раскладка): два вида одной программы

`Raw layout` (файловая раскладка, сырая раскладка) - это то, как байты лежат в PE-файле на диске. Здесь координата называется file offset (смещение в файле): сколько байт нужно пройти от начала файла, чтобы попасть к нужному месту.

`Virtual layout` (виртуальная раскладка) - это то, как эти же логические части выглядят в памяти после загрузки. Здесь координаты выражаются через RVA или VA. Виртуальная раскладка важна для CPU и Windows во время выполнения. Процессор выполняет код по виртуальным адресам, а не по file offset.

Почему две раскладки не совпадают? Потому что файл оптимизирован для хранения на диске, а image в памяти - для выполнения. В файле секции выравниваются по FileAlignment (файловое выравнивание). В памяти - по SectionAlignment (секционное/виртуальное выравнивание). В памяти нужны права страниц, границы страниц, нулевые области, применённые релокации, заполненные импорты. На диске это всё хранится иначе.

Пример по смыслу. Допустим, `.text` начинается в файле по offset `0x400`, а в памяти должна начинаться по RVA `0x1000`. Инструкция, которую CPU выполняет по адресу `ImageBase + 0x1100`, может физически лежать в файле по offset `0x500`. Если ты перепутаешь эти числа, то будешь смотреть не те байты.

```
Упрощённый перевод RVA в file offset через секцию:

если RVA лежит внутри секции:
  RVA = Section.VirtualAddress + delta
то в файле:
  file_offset = Section.PointerToRawData + delta

Важно: это работает только после того, как ты нашёл правильную секцию.
```

Именно это различие потом будет постоянно возвращаться в `.pdata` и `.xdata`. Поле `UnwindInfoAddress` в `RUNTIME_FUNCTION` - это RVA. Чтобы статически прочитать `UNWIND_INFO` из файла, анализатор должен перевести этот RVA в file offset. Чтобы Windows во время выполнения нашла его в памяти, она прибавит RVA к ImageBase загруженного module.

2.1.12. File offset (смещение в файле): координата на диске

`File offset` (смещение в файле) - это количество байт от начала файла до нужного места. Если file offset равен `0x200`, значит в файле нужно перейти на 512 байт от начала. Hex-редактор обычно показывает именно file offsets. Когда ты открываешь EXE как файл, ты работаешь в этой системе координат.

File offset нужен парсерам, hex-редакторам, статическим анализаторам и любому коду, который читает файл с диска. Например, поле `e_lfanew` в DOS header является file offset до PE signature. `PointerToRawData` в section header тоже является file offset до сырых данных секции.

Но file offset почти никогда не является тем адресом, который увидит CPU во время выполнения. Процессор не говорит “выполняю offset 0x400 файла”. Он выполняет виртуальный адрес в памяти процесса. Поэтому file offset - координата хранения, а не координата выполнения.

> Типичная ошибка: “в Ghidra адрес функции 0x140001000, значит в файле надо идти на offset 0x140001000”. Нет. Это VA. Нужно перейти от VA к RVA, потом через section header от RVA к file offset.

2.1.13. RVA (Relative Virtual Address, относительный виртуальный адрес): координата внутри образа

RVA (Relative Virtual Address, относительный виртуальный адрес) - это смещение от base address загруженного image. Если ImageBase равен `0x140000000`, а RVA равен `0x2000`, то соответствующий VA будет `0x140002000`.

Зачем нужен RVA? Потому что PE-образ может быть загружен по разным адресам. Если бы все таблицы хранили абсолютные адреса, файл был бы жёстко привязан к одной базе. RVA решает проблему переносимости внутри адресного пространства: таблица говорит не “абсолютно 0x140002000”, а “на 0x2000 байт от начала образа”. Loader знает фактический ImageBase и может получить настоящий VA.

RVA особенно удобен для структур внутри PE. Data Directory хранит VirtualAddress как RVA. `RUNTIME_FUNCTION` хранит BeginAddress, EndAddress и UnwindInfoAddress как image-relative значения, то есть по сути RVA. Section VirtualAddress тоже задаёт, где секция начинается в виртуальном образе.

Как работает связь: loader загружает module по base address. Когда Windows или анализатор видит RVA, он прибавляет его к base address, чтобы получить VA. Для статического чтения файла нужно сделать другой путь: найти секцию, в которую попадает RVA, и перевести его в file offset.

Типичная ошибка - думать, что RVA всегда равен file offset. Иногда числа могут случайно быть похожи, особенно в простых маленьких файлах, но полагаться на это нельзя. Правильный анализ всегда идёт через section table.

2.1.14. VA (Virtual Address, виртуальный адрес): адрес, по которому живой процесс видит память

VA (Virtual Address, виртуальный адрес) - это адрес в виртуальном адресном пространстве процесса. Когда CPU выполняет инструкцию, читает память или пишет память, он работает с virtual addresses. В пользовательском режиме Windows каждый процесс имеет своё виртуальное адресное пространство. Один и тот же VA в разных процессах может означать разные физические страницы или вообще быть недоступным.

Для загруженного PE-модуля VA обычно считается так: `VA = ImageBase + RVA`. Если модуль загружен по базе `0x00007FF700000000`, а функция имеет RVA `0x1234`, её VA будет `0x00007FF700001234`. Именно такой адрес ты часто видишь в отладчике.

VA нужен отладчику, CPU, Windows memory manager-у, exception dispatcher-у. Когда происходит исключение, `RIP` содержит VA. `RtlLookupFunctionEntry` получает ControlPc, то есть адрес выполнения, и ищет запись функции относительно module base. Для живого процесса это всё происходит в мире VA.

Но если ты читаешь файл с диска, VA напрямую бесполезен. Сначала нужно понять, какому module он принадлежит, вычесть ImageBase, получить RVA, затем перевести RVA в file offset. Это базовое движение reverse engineer-а между runtime-миром и file-миром.

2.1.15. ImageBase (база образа): точка отсчёта для RVA

`ImageBase` (база образа) - это предпочтительный или фактический базовый адрес, от которого считаются RVA загруженного PE-образа. В PE Optional Header есть поле ImageBase, где линкер записывает желаемую базу загрузки. Loader старается загрузить образ туда, если возможно, но не всегда обязан или может это сделать.

Почему ImageBase нужен? Потому что таблицы PE часто хранят координаты относительно начала образа. Если есть RVA `0x5000`, нужно знать, от чего его прибавлять. ImageBase и есть это “от чего”.

В современном Windows мире важна ASLR. ASLR (Address Space Layout Randomization, рандомизация раскладки адресного пространства) - защитный механизм, который может загружать модули по разным базовым адресам при разных запусках. Это усложняет эксплуатацию уязвимостей, потому что абсолютные адреса становятся менее предсказуемыми. Но RVA внутри образа остаются стабильными относительно базы.

Связь простая: ASLR меняет ImageBase, но не меняет логику RVA. Поэтому PE-таблицы используют относительные координаты. Если сегодня модуль загружен по `0x7ff700000000`, а завтра по `0x7ff680000000`, RVA функции может остаться `0x1200`; изменится только VA.

> Ошибка: копировать VA из отладчика и искать его в другом запуске как абсолютную истину. При ASLR база может измениться. Надёжнее мыслить как module + RVA.

2.1.16. COFF (Common Object File Format, общий формат объектных файлов): исторический сосед PE

COFF (Common Object File Format, общий формат объектных файлов) - это формат, связанный с объектными файлами и частью PE-структур. Microsoft PE официально описывается как PE/COFF, потому что PE использует COFF File Header и многие идеи COFF для описания машинного типа, секций, символов в object files и компоновки.

Для новичка COFF можно понять так: до итогового EXE/DLL существуют object files (объектные файлы), которые создаёт compiler (компилятор). Они содержат куски кода и данных, но ещё не являются полноценной программой. Linker (линкер) берёт эти объектные файлы и собирает из них PE image. COFF - часть этой истории объектных файлов и заголовков.

В самой книге нам не нужно глубоко уходить в COFF symbol table прямо сейчас. Но важно знать, почему документация говорит PE/COFF, а не только PE. PE - это формат исполняемого образа Windows, COFF - связанный формат объектных файлов и часть заголовочной структуры. В PE есть COFF File Header, который указывает machine type, number of sections, timestamp, size of optional header и characteristics.

Типичная путаница: думать, что COFF - это отдельный “конкурирующий exe-формат”. В контексте Windows PE/COFF это семейство связанных структур: object files для сборки и image files для загрузки. Для нашей темы важнее image file, потому что `.pdata/.xdata` живут в загруженном PE32+ image.

2.1.17. PE Headers (PE-заголовки): путь от начала файла до карты образа

PE Headers (PE-заголовки) - это совокупность структур в начале PE-файла, которые позволяют перейти от сырых байтов к осмысленной карте образа. Упрощённо путь такой: DOS Header -> PE Signature -> COFF File Header -> Optional Header -> Section Table.

DOS Header (DOS-заголовок) находится в начале файла и начинается с MZ. Его важнейшее поле для нас - e_lfanew, file offset до PE Signature. PE Signature (PE-сигнатура) - маркер PE□□, подтверждающий начало современных PE-заголовков. COFF File Header (COFF-заголовок файла) даёт общие сведения: архитектура, количество секций, размер Optional Header. Optional Header (дополнительный заголовок) содержит поля загрузки образа: ImageBase, AddressOfEntryPoint, SectionAlignment, FileAlignment, SizeOfImage, DataDirectory. Section Table (таблица секций) перечисляет все секции.

Почему важно видеть путь целиком? Потому что PE parser (парсер PE, программа чтения PE-формата) не может просто “искать строку .pdata”. Он должен читать файл по структуре. Сначала убедиться в MZ, взять e_lfanew, проверить PE□□, прочитать количество секций, понять размер Optional Header, затем корректно найти section headers. Если хотя бы один шаг неверен, всё дальнейшее чтение может сместиться.

Для reverse engineer-а это ещё и защита от обмана. Упаковщики и повреждённые файлы могут иметь странные заголовки, нестандартные выравнивания, секции с необычными именами, перекрытия или намеренно запутанные поля. Если ты знаешь только “в exe есть .text”, тебя

легко сбить. Если ты понимаешь путь парсера, ты видишь, где именно начинается противоречие.

2.1.18. Optional Header (дополнительный заголовок): главный паспорт загрузки образа

`Optional Header` (дополнительный заголовок) в PE image - один из самых важных заголовков. Название “optional” исторически неудачно для EXE/DLL: без него нормальный PE image для Windows не имеет нужной информации загрузки. Он содержит не декоративные данные, а параметры, по которым loader создаёт image в памяти.

В Optional Header находятся поля, которые дальше будут постоянно всплывать. `AddressOfEntryPoint` (адрес точки входа как RVA) говорит, куда передать управление после загрузочной подготовки. `ImageBase` (база образа) задаёт предпочтительную базу загрузки. `SectionAlignment` (выравнивание секций в памяти) говорит, по какой границе секции выравниваются в virtual layout. `FileAlignment` (выравнивание в файле) говорит, по какой границе секции выравниваются в raw layout. `SizeOfImage` (размер образа) описывает общий размер образа в памяти. `DataDirectory` (каталог данных) ведёт к специальным таблицам, включая Exception Directory.

Что важно для unwind? Optional Header содержит Data Directory. Data Directory, в свою очередь, содержит запись Exception Directory. Exception Directory указывает на таблицу `RUNTIME_FUNCTION`. То есть путь к `.pdata` начинается не с имени секции, а с Optional Header -> DataDirectory -> Exception Directory. Имя `.pdata` удобно для человека, но формальная точка входа для loader-а и анализатора - каталог данных.

> Если PE-парсер ищет `.pdata` только по имени секции, он может ошибиться. Правильнее читать Exception Directory из Data Directory. Секция может называться необычно, данные могут лежать в объединённой секции, а имя не является строгим контрактом.

2.1.19. Data Directory (каталог данных): таблица указателей на специальные подсистемы PE

`Data Directory` (каталог данных) - это массив записей в Optional Header. Каждая запись говорит: “в образе есть специальная таблица такого-то назначения; она начинается по такому RVA и имеет такой размер”. Это не сами данные, а указатели на них.

Почему это нужно? PE содержит много разных подсистем: imports (импорты), exports (экспорты), resources (ресурсы), relocations (релокации), TLS, exception table (таблица исключений), debug data (отладочные данные) и другие. Loader и инструменты не должны угадывать, где всё это лежит. Они смотрят в Data Directory и получают координаты.

Каждая запись Data Directory обычно содержит два поля: VirtualAddress (виртуальный адрес как RVA) и Size (размер). VirtualAddress здесь не VA в полном смысле, а RVA. Это типичный источник путаницы: поле называется VirtualAddress, но в PE Data Directory оно хранит относительный виртуальный адрес. Чтобы получить VA в процессе, нужно прибавить ImageBase. Чтобы прочитать из файла, нужно перевести RVA в file offset через секцию.

Для нас самая важная запись - Exception Directory (каталог исключений). В PE32+ x64 она указывает на таблицу function entries, то есть `RUNTIME_FUNCTION`. Эта таблица обычно размещена в `.pdata`. Из неё мы попадём в `.xdata`, где лежит `UNWIND_INFO`.

```
Путь к unwind-таблицам в нормальной x64-картине:

Optional Header
  -> Data Directory
      -> Exception Directory (RVA + Size)
          -> массив RUNTIME_FUNCTION
              -> UnwindInfoAddress (RVA)
                  -> UNWIND_INFO в .xdata или связанной области
```

2.1.20. Import (импорт): как PE говорит “мне нужны функции из других DLL”

`Import` (импорт) - это механизм, через который PE-файл сообщает Windows: “для работы мне нужны функции или данные из других модулей”. Например, программа может использовать `CreateFileW` из `kernel32.dll`, `MessageBoxW` из `user32.dll`, функции C runtime из `ucrtbase.dll`. В файле не лежит полный код всех этих функций; вместо этого PE содержит таблицы импортов.

Зачем здесь import (импорт), если глава про unwind? Потому что imports показывают общий принцип PE: файл содержит не только код, но и таблицы, которые loader должен обработать. Loader читает Import Directory, загружает нужные DLL, находит адреса функций и заполняет Import Address Table.

`Import Address Table` (IAT, таблица адресов импортов) - таблица в памяти образа, где после загрузки лежат реальные VA импортированных функций. Код программы может вызывать не абстрактное имя `CreateFileW`, а адрес, записанный в IAT. До загрузки этот адрес ещё не окончательный. После загрузки loader его заполняет.

Связь с Exception Directory концептуальная: и imports, и exceptions находятся через Data Directory. Import Directory нужен для связывания внешних функций. Exception Directory нужен для связывания адресов кода с unwind-описаниями. Разные назначения, один стиль PE: специальные подсистемы описываются таблицами, на которые указывают каталоги данных.

2.1.21. Export (экспорт): как DLL говорит “вот что я отдаю наружу”

`Export` (экспорт) - это механизм, через который модуль, чаще DLL, сообщает другим модулям: “вот функции или данные, которые можно импортировать у меня”. Если EXE импортирует `CreateFileW`, то соответствующая DLL экспортирует функцию или перенаправляет её через системную цепочку API.

Export Directory (каталог экспортов) содержит таблицы имён, ordinals (порядковых номеров) и адресов экспортируемых функций. Для reverse engineer-а exports полезны тем, что дают имена входных точек DLL. Если функция экспортирована с именем, её проще опознать, чем безымянный внутренний код.

Почему это важно в общей модели PE? Потому что PE - это не только “как запустить EXE”, но и как модули связываются друг с другом. EXE импортирует, DLL экспортирует, loader соединяет адреса. Exception/unwind-данные живут рядом как ещё один слой метаданных, который нужен уже не для связывания функций по имени, а для восстановления выполнения по стеку.

2.1.22. Relocation (релокация): что делать, если модуль загрузили не туда

`Relocation` (релокация, перенос адресов) - это механизм исправления адресных значений, если PE-образ загружен не по предпочтительному ImageBase. Если код или данные содержат абсолютный адрес, рассчитанный под одну базу, а loader загрузил module по другой базе, такие адреса нужно поправить.

Почему образ могут загрузить не по желаемой базе? Например, желаемый диапазон уже занят другим модулем. Или включён ASLR, и система специально выбирает другую базу. Тогда loader применяет relocation table (таблицу релокаций), обычно связанную с `.reloc`, чтобы исправить нужные места.

RVA помогает уменьшить зависимость от абсолютных адресов, но не устраняет все случаи. Некоторые машинные инструкции или данные могут содержать адреса, которые нужно поправить при смене базы. Relocation Directory говорит loader-у, где такие места находятся.

Связь с unwind такая: `RUNTIME_FUNCTION` хранит image-relative addresses, то есть RVA, поэтому сами эти поля не требуют такой же правки, как абсолютные VA. Это одна из причин, почему относительные адреса удобны для PE-таблиц. Но общий механизм загрузки всё равно должен учитывать релокации для других частей образа.

2.1.23. Exception Directory (каталог исключений): дверь к x64 unwind

`Exception Directory` (каталог исключений) - запись в Data Directory, которая для Windows x64 указывает на таблицу данных, нужных для обработки исключений и unwind. Это один из центральных мостов между PE-форматом и первой главой книги.

Для x64 Exception Directory обычно указывает на массив `RUNTIME_FUNCTION` записей. Этот массив часто лежит в секции `.pdata`. Каждая запись описывает диапазон кода функции и указывает, где лежит unwind-информация. Эта unwind-информация обычно лежит в `.xdata` и начинается со структуры `UNWIND_INFO`.

По-человечески Exception Directory говорит: “если во время выполнения адрес `RIP` попал в код этого модуля и системе нужно раскрутить стек, начинать искать карту функций нужно вот здесь”. Это не обработчик сам по себе. Это не `catch`. Это вход в таблицу, которая позволит найти функцию и её правила восстановления кадра.

Очень важная ошибка: думать, что Exception Directory хранит список всех `try/catch`. Нет. В x64 она прежде всего ведёт к function table. А уже внутри unwind-информации и handler data может быть указание на language-specific handler, который понимает конкретные `__try`, `__except`, C++ EH и другие языковые конструкции.

> Вся будущая глава про `.pdata` начинается здесь: Optional Header -> Data Directory -> Exception Directory. Если этот путь не ясен, `RUNTIME_FUNCTION` будет казаться случайной структурой. На самом деле это формальная таблица, на которую указывает PE.

2.1.24. .pdata (procedure data, данные процедур): таблица диапазонов функций

`.pdata` (procedure data, данные процедур) - секция или область PE32+ image, где обычно лежит массив `RUNTIME_FUNCTION` записей. Слово procedure (процедура) здесь близко к function (функция): речь о диапазонах исполняемого кода, которые Windows должна уметь сопоставить с unwind-описанием.

Зачем нужна `.pdata`? Когда происходит исключение, у Windows есть RIP или ControlPc - адрес внутри кода. Нужно понять, какой функции принадлежит этот адрес. `.pdata` даёт таблицу диапазонов: BeginAddress, EndAddress, UnwindInfoAddress. Если ControlPc попадает между BeginAddress и EndAddress, найдена запись для этой функции.

`.pdata` не хранит инструкции функции. Код обычно лежит в `.text`. `.pdata` хранит метаданные о функциях. Это как каталог в библиотеке: книга лежит на полке, но каталог говорит, где её искать и к какой карточке она относится. Код выполняется из `.text`; карта для unwind лежит в `.pdata`; подробные правила восстановления кадра лежат в `.xdata`.

Для reverse engineer-а `.pdata` ценна тем, что помогает увидеть границы функций даже там, где дизассемблер сомневается. Но ей нельзя слепо верить: packed/obfuscated binaries могут искажать данные, а не всякий код обязан иметь запись. Поэтому `.pdata` - источник сильных гипотез и контрактов, но не замена анализу кода.

2.1.25. .xdata (extended data, расширенные данные): где лежит инструкция для unwind

`.xdata` (extended data, расширенные данные) - секция или область, где обычно хранится `UNWIND_INFO` и связанные данные обработчиков. Если `.pdata` говорит “вот функция и вот адрес её unwind-описания”, то `.xdata` содержит само описание.

Почему нельзя было всё хранить прямо в `.pdata`? Потому что `RUNTIME_FUNCTION` должен быть маленькой и однотипной записью для быстрого поиска по диапазонам. Подробности функции могут быть разного размера: разное количество unwind-кодов, разные обработчики, chained info, language-specific handler data. Поэтому `.pdata` хранит ссылку, а `.xdata` хранит переменную по размеру информацию.

`UNWIND_INFO` (информация раскрутки) описывает, что функция сделала в прологе: сколько места выделила на стеке, какие nonvolatile registers (невременные/сохраняемые регистры) сохранила, использовала ли frame register (регистр кадра), есть ли exception handler (обработчик исключения) или termination handler. Эта информация нужна `RtlVirtualUnwind`, чтобы восстановить caller context.

Связь слоёв теперь должна быть очень конкретной: `.text` содержит код функции; `.pdata` содержит запись диапазона этой функции; `.xdata` содержит правила, как выйти из её кадра назад; Windows использует эти правила при исключениях, stack walking и dump analysis.

2.1.26. AddressOfEntryPoint (адрес точки входа): где начинается выполнение после загрузки

`AddressOfEntryPoint` (адрес точки входа) - поле Optional Header, которое задаёт RVA, куда loader передаст управление после подготовки образа. Это не обязательно твой `main`. В C/C++ программа обычно стартует с runtime entry point (точки входа runtime), который инициализирует C runtime, глобальные объекты, аргументы, окружение, а уже потом вызывает `main` или `WinMain`.

Почему это важно? Потому что PE работает на уровне машинного образа, а не на уровне языка C. Windows loader не знает, где у тебя `main` как исходная функция. Он знает AddressOfEntryPoint. Всё остальное - работа runtime-библиотеки, которую добавил компилятор/линкер.

Для reverse engineer-а точка входа - первое место, куда можно пойти при анализе старта программы. Но если файл упакован, AddressOfEntryPoint может вести не в настоящий код программы, а в unpacking stub (заглушку распаковщика). Эта заглушка распакует/расшифрует настоящий код и передаст управление дальше. Поэтому entry point - важная начальная точка, но не всегда “начало логики программы”.

Связь с unwind косвенная: entry point сам является кодом в image и может иметь или не иметь unwind-метаданные. Но главное - AddressOfEntryPoint показывает, что PE задаёт не просто хранение байтов, а маршрут запуска.

2.1.27. Rights / Characteristics (права и характеристики): почему секции читаются, пишутся или выполняются

`Characteristics` (характеристики) секции - набор флагов, который описывает свойства секции: содержит ли она код, данные, можно ли её читать, писать, выполнять. На основе этих свойств loader и memory manager выставляют page protections (защиты страниц памяти).

`Page protection` (защита страницы) - права доступа к странице виртуальной памяти: read (чтение), write (запись), execute (выполнение) и их комбинации. Если код пытается выполнить память без execute-права или записать в read-only область, может возникнуть access violation.

Почему это важно для PE? Секция `.text` обычно должна быть executable, потому что там код. `.rdata` обычно readable, но не writable и не executable. `.data` обычно readable/writable. `.pdata` и `.xdata` обычно readable, потому что Windows читает их как метаданные, но они не должны быть обычным исполняемым кодом.

Для защиты это важно через принцип W^X: память желательно не делать одновременно writable и executable без необходимости. Для reverse engineering это помогает классифицировать области. Если секция writable+executable, это может быть нормально для некоторых JIT/packer сценариев, но в обычном приложении вызывает вопрос. Если unwind-данные лежат в executable области, это не обязательно ошибка, но повод смотреть внимательнее.

2.1.28. Почему PE - это договор между несколькими участниками

Лучший способ мыслить о PE - не как о “формате файла”, а как о договоре между участниками. Compiler (компилятор) генерирует код и часть метаданных. Linker (линкер) собирает итоговый image, секции, заголовки и таблицы. Loader (загрузчик) читает PE и строит image в памяти. Windows runtime mechanisms (механизмы выполнения Windows) используют таблицы импортов, релокаций, исключений. Debugger (отладчик) и dump analyzer (анализатор дампов) используют PE для символов, модулей, секций, stack walking. Reverse engineer (реверсер) читает все эти слои и проверяет их согласованность.

Если PE корректный и обычный, участники понимают друг друга. Linker записал `.pdata`, loader загрузил image, `RtlLookupFunctionEntry` нашла `RUNTIME_FUNCTION`, `RtlVirtualUnwind` прочитала `UNWIND_INFO`, отладчик построил стек. Если PE странный, участники могут расходиться. Loader может принять файл, а статический анализатор запутаться. Или наоборот: анализатор что-то показывает, но runtime поведение отличается из-за dynamic code или runtime registration.

Для твоего будущего static /dynamic эта договорная модель важнее списка структур. Инструмент должен задавать вопросы: поле какого типа перед нами? Это file offset, RVA или VA? Указывает ли RVA внутрь image? Попадает ли он в секцию с подходящими правами? Соответствует ли Exception Directory размеру массива `RUNTIME_FUNCTION`? Согласуется ли `.pdata` с `.text`? Ведёт ли `UnwindInfoAddress` к валидному `UNWIND_INFO`?

Именно так теория PE переходит в практический reverse engineering: не “выучить поля”, а видеть обязательства между байтами файла, образом в памяти и механизмами Windows.

2.1.29. Мини-карта PE без зубрёжки полей

Теперь соберём PE в одну карту. Сначала файл начинается с DOS Header (DOS-заголовок), где есть `MZ` и `e_lfanew`. `e_lfanew` ведёт к PE Signature (PE-сигнатуре) PE□□. После неё идёт COFF File Header (COFF-заголовок файла), который говорит, для какой архитектуры файл и сколько секций нужно читать. Затем идёт Optional Header (дополнительный заголовок), где находится информация загрузки: ImageBase, entry point, размеры, выравнивания, Data Directory. После него идёт Section Table (таблица секций), где каждая запись описывает одну секцию.

Дальше начинаются сами данные секций: `.text`, `.rdata`, `.data`, `.pdata`, `.xdata`, `.reloc` и другие. Их порядок и имена могут различаться, но section headers описывают, где они лежат в файле и где должны оказаться в памяти.

Если Windows нужна таблица импортов, она смотрит Import Directory. Если нужны релокации - Base Relocation Directory. Если нужен unwind для x64-исключений - Exception Directory. Directory даёт RVA и Size, а дальше этот RVA ведёт в соответствующую область образа.

Так PE превращает файл из “набора байтов” в структурированный executable image. Это и есть фундамент для следующей темы: PE32+ и x64. Мы уже знаем, что PE описывает образ. Теперь нужно понять, чем 64-битный вариант PE отличается от 32-битного и почему Windows x64 unwind живёт именно в PE32+ модели.

2.1.30. Контрольное понимание: что теперь должно быть в голове

После этого раздела PE file (Portable Executable file, переносимый исполняемый файл) должен восприниматься не как “exe-шник”, а как техническая карта загрузки. Loader (загрузчик) не копирует файл тупо в память, а читает headers (заголовки), sections (секции), data directories (каталоги данных), imports (импорты), relocations (релокации) и exception data (данные исключений).

Section (секция) - крупная область с ролью, но имя секции не является абсолютной истиной. Section Header (заголовок секции) важнее имени, потому что он связывает raw layout (файловую раскладку) и virtual layout (виртуальную раскладку). File offset (смещение в файле) нужен для чтения диска. RVA (Relative Virtual Address, относительный виртуальный адрес) нужен для координат внутри image. VA (Virtual Address, виртуальный адрес) нужен для живого процесса. ImageBase (база образа) связывает RVA и VA.

Data Directory (каталог данных) - таблица входов в специальные подсистемы PE. Exception Directory (каталог исключений) - вход к x64 function table. `.pdata` (procedure data, данные процедур) обычно хранит `RUNTIME_FUNCTION`. `.xdata` (extended data, расширенные данные) обычно хранит `UNWIND_INFO`. Эта цепочка связывает PE с тем, что мы изучали в первой главе: исключение, `RIP`, поиск функции, unwind и восстановление caller context.

> Главный итог: PE - это не фон. Для Windows x64 exception/unwind PE является местом, где лежит карта функций. Без понимания PE невозможно надёжно понять `.pdata`, `.xdata`, `RUNTIME_FUNCTION` и `UNWIND_INFO`.

Техническая база раздела

Раздел опирается на официальную документацию Microsoft PE Format, где PE/COFF описывается как формат executable image files и object files для Windows, с headers, sections, RVA/VA и data directories; а также на документацию Microsoft по x64 exception handling, где описана связь PE32+ image, `.pdata`, `RUNTIME_FUNCTION` и unwind metadata. Эти источники используются как техническая основа; текст раздела написан заново в учебной подаче.

2.2. Что такое PE32+ и почему для x64 это важно

В предыдущем разделе мы начали строить PE-модель медленно: PE file (Portable Executable file, переносимый исполняемый файл) - это не “просто exe”, а инструкция для Windows loader-а (загрузчика), как превратить файл на диске в image (образ) в памяти. Теперь появляется следующий вопрос: если PE - общий формат, откуда Windows понимает, что перед ней именно 64-битный образ для x64, а не 32-битный образ? Ответ начинается с термина PE32+.

Эта тема кажется маленькой только на бумаге. На деле она связывает сразу несколько важных вещей: разрядность образа, формат Optional Header (дополнительного заголовка), размер адресных полей, `ImageBase` (базу образа), работу ASLR (Address Space Layout Randomization, рандомизация адресного пространства), совместимость парсеров и будущие `.pdata/.xdata` unwind-таблицы. Если здесь оставить туман, дальше постоянно будет путаница: почему файл называется PE32+, если он 64-битный; почему `RUNTIME_FUNCTION` в x64 всё равно хранит 32-битные RVA; почему `ImageBase` стал 64-битным; почему `BaseOfData` исчез; почему нельзя читать PE32+ тем же кодом, что PE32, без проверки Magic.

> Правило раздела: PE32+ вводим не как сухую метку “64-bit PE”, а как вариант формата PE image, который меняет конкретные поля и конкретные ожидания loader-а, parser-а и reverse engineer-а.

2.2.1. PE32+ (Portable Executable 32 Plus, вариант PE для 64-битных образов): почему название сбивает с толку

PE32+ (Portable Executable 32 Plus, “PE32 плюс”) - это вариант PE-формата, используемый для 64-битных исполняемых образов Windows. Самое первое, что здесь сбивает: в названии есть 32, хотя речь идёт о x64. Из-за этого новичок часто думает, что PE32+ - это “почти 32-битный формат”, “гибрид” или “какая-то совместимость”. Это неверно.

Правильнее думать так: PE32+ - это развитие PE32, где формат сохранил общую архитектуру PE, но был изменён для 64-битного адресного пространства. Плюс в названии означает не “чуть-чуть больше”, а “расширенный вариант”. Структура PE в целом осталась узнаваемой: DOS Header (DOS-заголовок), PE Signature (PE-сигнатура), COFF File Header (COFF-заголовок файла), Optional Header (дополнительный заголовок), Section Table (таблица секций), sections (секции), Data Directory (каталог данных). Но некоторые поля Optional Header стали другими.

Почему Microsoft не назвала его просто PE64? Исторически формат развивался с сохранением совместимости и близости к PE32. Многие структуры остались похожими, многие поля остались 32-битными, а некоторые адресные поля расширились до 64 бит. Поэтому официальное имя - PE32+, а не PE64. Для практики важно не спорить с названием, а понимать: если Optional Header Magic равен `0x20B`, перед нами PE32+ image, то есть 64-битный вариант image-формата.

`Magic` (магическое число, сигнатурное значение) - поле в Optional Header, которое говорит, какой вариант optional header используется. Оно “магическое” не потому, что в нём что-то мистическое, а потому что это заранее согласованное число, по которому parser (парсер, программа чтения формата) распознаёт тип структуры. Для PE32 обычно используется `0x10B`, для PE32+ - `0x20B`. Это один из первых контрольных пунктов нормального PE-парсера.

```
Optional Header Magic:
  0x10B  -> PE32  (обычно 32-битный image)
  0x20B  -> PE32+ (64-битный image для x64/других 64-bit архитектур)
  0x107  -> ROM image (для нашей темы не основная линия)
```

Типичная ошибка: определить “64-битность” только по расширению файла или только по тому, что файл запустился на 64-битной Windows. Это слабый критерий. 64-битная Windows может запускать 32-битные процессы через WoW64. Поэтому parser должен смотреть в PE-заголовки: Machine (тип архитектуры) в COFF File Header и Magic в Optional Header. Эти поля вместе дают формальную картину.

2.2.2. x64 (64-bit Windows architecture, 64-битная архитектура Windows): что именно означает “64-битный”

x64 (64-bit Windows architecture, 64-битная архитектура Windows) - это архитектурная линия Windows для процессоров AMD64/Intel 64. В бытовом смысле говорят “64-битная программа”, но для реверса нужно разложить это на несколько уровней.

Первый уровень - instruction set (набор инструкций). 64-битный код использует x64-инструкции, 64-битные регистры общего назначения вроде `RAX`, `RBX`, `RCX`, `RDX`, `RSP`, `RIP`, `R8-R15`, другой calling convention и другую модель exception/unwind.

Второй уровень - address space (адресное пространство). 64-битный процесс живёт в 64-битной модели виртуальных адресов. Это не значит, что любой процесс реально использует все 2^64 адресов. Это значит, что указатели и многие адресные значения в коде и данных рассчитаны на 64-битную модель. `void*` в 64-битном процессе занимает 8 байт, а не 4.

Третий уровень - binary format (двоичный формат). Файл должен сообщить Windows, что это 64-битный image. Для этого в PE используется PE32+ Optional Header и Machine type, например `IMAGE_FILE_MACHINE_AMD64`. Loader не может загружать x64-код как будто это x86-код: регистры, calling convention, таблицы unwind и сами инструкции будут другими.

Четвёртый уровень - runtime contracts (контракты времени выполнения). 64-битный код на Windows использует x64 calling convention (соглашение вызовов x64), x64 unwind model (модель раскрутки стека x64), `RUNTIME_FUNCTION`, `.pdata`, `.xdata`, `UNWIND_INFO`. Это уже напрямую связано с темой книги.

> Слово “64-битный” в книге нельзя понимать только как “адреса длиннее”. Это сразу четыре слоя: инструкции, регистры/указатели, формат PE32+, и runtime-контракты Windows x64.

2.2.3. Optional Header (дополнительный заголовок): почему именно он определяет PE32 или PE32+

`Optional Header` (дополнительный заголовок) - часть PE Headers (PE-заголовков), в которой лежат параметры загрузки image. В object file (объектном файле) он исторически может быть optional (необязательным или почти бесполезным), но для executable image (исполняемого образа) он фактически необходим. Именно в нём находятся `ImageBase`, `AddressOfEntryPoint`, `SectionAlignment`, `FileAlignment`, `SizeOfImage`, `Subsystem`, `DllCharacteristics`, `DataDirectory` и другие поля, без которых loader не поймёт, как строить image в памяти.

Почему тип PE32/PE32+ определяется через Optional Header? Потому что различия между 32-битным и 64-битным image проявляются прежде всего в параметрах загрузки и адресных полях. Кодовые секции могут выглядеть как просто байты, таблица секций имеет похожую форму, но Optional Header должен сказать: какие размеры полей используются, какого

размера ImageBase, есть ли BaseOfData, как читать stack/heap reserve/commit и другие значения.

Magic (магическое число) - первое поле Optional Header. Парсер читает его и выбирает правильную структуру: IMAGE_OPTIONAL_HEADER32 (32-битный дополнительный заголовок) или IMAGE_OPTIONAL_HEADER64 (64-битный дополнительный заголовок). Если парсер ошибся и прочитал PE32+ как PE32, дальнейшие поля “поедут”: часть байтов будет истолкована не как то поле, которым она является. Это одна из самых неприятных ошибок: файл не обязательно сразу “сломается” в глазах парсера, он может дать правдоподобный мусор.

Представь две анкеты, похожие внешне. В одной поле “адрес” занимает одну строку, в другой - две строки. Если ты заполняешь вторую анкету по шаблону первой, все поля после адреса сместятся. В PE32/PE32+ происходит похожая вещь: структура похожая, но размеры некоторых полей отличаются. Поэтому Magic - не декоративная метка, а переключатель схемы чтения.

2.2.4. IMAGE_OPTIONAL_HEADER32 и IMAGE_OPTIONAL_HEADER64: две похожие, но не одинаковые структуры

IMAGE_OPTIONAL_HEADER32 (32-битная структура дополнительного заголовка) и IMAGE_OPTIONAL_HEADER64 (64-битная структура дополнительного заголовка) - это структуры из Windows headers, через которые C/C++ код может читать Optional Header. Они нужны, чтобы парсер понимал, какого размера поля и в каком порядке они лежат.

Они похожи потому, что PE32+ не стал полностью новым форматом. Но они различаются там, где 64-битная модель требует других размеров. Главное отличие: в PE32+ некоторые поля, связанные с адресами и размерами стек/куча reserve/commit, имеют 64-битный размер. Кроме того, в PE32+ отсутствует поле BaseOfData (база данных), которое есть в PE32.

BaseOfData (база секции данных) - поле PE32 Optional Header, которое указывает начало data section относительно image base. В PE32+ это поле удалено. Для практики это означает: нельзя писать универсальный парсер так, будто после BaseOfCode всегда идёт BaseOfData. В PE32+ после BaseOfCode структура продолжается иначе. Если забыть это, парсер будет читать 8-байтный ImageBase с неверного смещения.

ImageBase (база образа) в PE32 - 32-битное значение, а в PE32+ - 64-битное. Это логично: 64-битный процесс может загружать image по адресам, которые не помещаются в 32 бита. Если парсер прочитает PE32+ ImageBase как 4 байта, он потеряет старшую часть адреса и дальше начнёт интерпретировать следующие байты как другие поля.

```
Упрощённо:

PE32 Optional Header:
  Magic
  ...
  BaseOfCode        (DWORD)
  BaseOfData        (DWORD)
  ImageBase         (DWORD)
  ...

PE32+ Optional Header:
  Magic
  ...
  BaseOfCode        (DWORD)
  ImageBase         (ULONGLONG, 8 байт)
  ...
```

```
Главное: PE32+ не имеет BaseOfData, а ImageBase шире.
```

DWORD (double word, двойное слово) - 32-битное значение, 4 байта. ULONGLONG (unsigned long long, беззнаковое 64-битное целое) - 64-битное значение, 8 байт. Эти слова часто встречаются в Windows-структурах. Их надо понимать не как “какие-то типы из C”, а как размер поля в бинарном формате. Когда ты читаешь PE байтами, размер поля определяет, сколько байтов забрать и как сдвинуться дальше.

2.2.5. Machine (тип машины/архитектуры): почему Magic недостаточно смотреть в одиночку

Machine (тип машины, тип архитектуры) - поле в COFF File Header, которое сообщает, для какой процессорной архитектуры предназначен image или object file. Для x64 Windows обычно важен IMAGE_FILE_MACHINE_AMD64. Для 32-битного x86 - IMAGE_FILE_MACHINE_I386.

Зачем нужен Machine, если уже есть Optional Header Magic? Потому что Magic говорит о варианте optional header - PE32 или PE32+. А Machine говорит о целевой архитектуре. В нормальном x64 EXE ты ожидаешь согласованную пару: Machine = AMD64, Magic = PE32+. Если Machine говорит AMD64, а Magic выглядит как PE32, это странная или некорректная ситуация для обычного image. Если Magic PE32+, но Machine не соответствует ожидаемой 64-битной архитектуре, parser должен быть осторожен.

Для reverse engineer-а согласованность Machine и Magic - один из первых sanity checks (проверок здравого смысла). Sanity check - это простая проверка, которая не доказывает, что файл идеален, но быстро ловит грубую нелепость. Например, “количество секций не должно быть абсурдным”, “Optional Header должен помещаться в файл”, “Magic должен соответствовать ожидаемой структуре”, “Exception Directory не должен указывать за пределы image”.

Почему это важно для твоего будущего static /dynamic? Потому что инструмент не должен слепо доверять одному полю. Хороший анализатор собирает картину из нескольких источников. Machine говорит об архитектуре. Magic говорит о варианте Optional Header. Data Directory говорит, где специальные таблицы. Section Table помогает перевести RVA в file offset. .pdata/.xdata дают unwind-картину. Если эти слои противоречат друг другу, это уже finding (обнаружение, зафиксированный результат анализа).

2.2.6. Address space (адресное пространство): почему PE32+ нужен 64-битный ImageBase

Address space (адресное пространство) - это диапазон виртуальных адресов, который процесс может использовать. Виртуальный адрес - это не физический адрес RAM, а адрес внутри модели памяти процесса. Windows и процессор через механизмы страниц связывают виртуальные адреса с реальными физическими страницами или другими backing storage (источниками хранения).

В 32-битной модели указатель обычно занимает 4 байта, значит теоретически можно представить 2^32 разных адресов, то есть 4 ГБ адресного пространства. В 64-битной модели указатель занимает 8 байт, и адресная модель становится намного шире. Реальная Windows не отдаёт пользовательскому процессу весь математический диапазон 2^64, но принципиально адреса могут быть выше 4 ГБ и требуют 64-битных полей в тех местах, где хранится полный VA.

`ImageBase` (база образа) - предпочтительный виртуальный адрес, по которому loader хотел бы разместить первый байт image в памяти. Для 64-битного image это значение должно уметь хранить 64-битный адрес. Поэтому в PE32+ `ImageBase` является 8-байтным полем.

Почему ImageBase важен? Потому что многие адреса внутри PE выражаются как RVA (Relative Virtual Address, относительный виртуальный адрес). Чтобы получить VA (Virtual Address, виртуальный адрес в процессе), нужно прибавить ImageBase:

```
VA = ImageBase + RVA
```

Если ImageBase 64-битный, итоговый VA тоже может быть 64-битным. При этом сам RVA часто остаётся 32-битным. Это нормально: RVA - это смещение внутри image, а не полный адрес в процессе. Image может быть загружен высоко, но размер самого image обычно намного меньше 4 ГБ, поэтому 32-битного RVA достаточно для указания места внутри image.

> Парадокс, который нужно запомнить: PE32+ - 64-битный image, ImageBase - 64-битный, VA - 64-битный, но многие поля формата всё равно хранят RVA как 32-битные смещения. Это не ошибка, а экономная и удобная модель: база широкая, смещения внутри образа компактные.

2.2.7. RVA в PE32+: почему 64-битный образ всё равно использует 32-битные относительные адреса

RVA (Relative Virtual Address, относительный виртуальный адрес) - смещение от ImageBase до объекта внутри загруженного image. Если ImageBase равен `0x00007FF700000000`, а RVA равен `0x00001234`, то VA будет `0x00007FF700001234`.

В PE32+ многие поля, включая адрес точки входа, адреса data directories и поля `RUNTIME_FUNCTION`, используют RVA. Эти RVA обычно 32-битные. Почему не 64-битные? Потому что RVA описывает положение внутри image, а не произвольный адрес во всём процессе. Если image ограничен разумным размером, 32-битного смещения достаточно. Microsoft PE-спецификация прямо связывает PE32+ с 64-битным адресным пространством, но размер image всё равно имеет ограничения, поэтому нет необходимости расширять каждое смещение до 64 бит.

Это важно для unwind. В x64 `RUNTIME_FUNCTION` содержит `BeginAddress`, `EndAddress`, `UnwindInfoAddress`. Эти поля выглядят как “адреса”, но на самом деле это image-relative offsets (адреса/смещения относительно базы image), то есть RVA. Они 32-битные. Чтобы найти реальную функцию в памяти, Windows берёт base address модуля и прибавляет BeginAddress. Чтобы читать из файла, анализатор должен сначала перевести RVA в file offset через секции.

Ошибка здесь очень частая: увидеть `UnwindInfoAddress = 0x3450` и попытаться читать файл по смещению `0x3450`. Иногда случайно получится, если raw layout похож на virtual layout. Но это случайность. Правильный путь: понять, что `0x3450` - RVA; найти секцию, чей virtual range покрывает этот RVA; вычислить file offset через `PointerToRawData` и `VirtualAddress` секции.

```
Правильная мысль:
  RUNTIME_FUNCTION.UnwindInfoAddress = RVA
  RVA не равен file offset
  RVA не равен готовый VA

Для процесса:
  VA = ImageBase + RVA

Для файла:
```

```
file_offset = section.PointerToRawData + (RVA - section.VirtualAddress)
```

2.2.8. VA, RVA и file offset в PE32+: три координатные системы, которые нельзя смешивать

VA (Virtual Address, виртуальный адрес) - адрес в виртуальном адресном пространстве процесса. Это адрес, который видит отладчик во время выполнения: например, `00007FF7ABCD1234`.

RVA (Relative Virtual Address, относительный виртуальный адрес) - смещение от ImageBase внутри image: например, `0x1234`. Это координата “внутри загруженного образа”, но без конкретной базы загрузки.

`File offset` (файловое смещение) - смещение от начала файла на диске: например, `0x400`. Это координата для hex-редактора и статического чтения файла.

Эти три координаты могут указывать на одни и те же байты в разных мирах, но численно они часто различаются. В PE32+ эта разница становится особенно важной, потому что VA может быть большим 64-битным адресом, RVA - компактным 32-битным смещением, а file offset - вообще положением в raw layout файла.

Raw layout (сырая/файловая раскладка) - то, как секции лежат в файле на диске. Virtual layout (виртуальная раскладка) - то, как эти секции расположены в memory image после загрузки. Loader строит virtual layout по заголовкам, а не просто копирует файл как один непрерывный массив.

Чем это опасно для новичка? Он видит адрес в Ghidra или WinDbg и ищет такой же offset в hex-редакторе. Не находит. Или видит RVA в PE-таблице и прибавляет ImageBase, хотя сейчас читает файл, а не процесс. Или берёт file offset из hex-редактора и думает, что это RVA. В итоге `.pdata` читается не оттуда, `UNWIND_INFO` парсится как мусор, а выводы выглядят умными, но неверными.

> Проверочный вопрос к любому “адресу” в PE: это VA, RVA или file offset? Пока ответа нет, число нельзя использовать.

2.2.9. AddressOfEntryPoint (адрес точки входа): почему в PE32+ это всё ещё RVA

`AddressOfEntryPoint` (адрес точки входа) - поле Optional Header, которое указывает, куда loader должен передать управление после того, как image подготовлен к выполнению. Несмотря на слово Address, это не готовый VA, а RVA. То есть это смещение от ImageBase до entry point (точки входа).

`Entry point` (точка входа) - не обязательно твой `main`. Это адрес внутри image, с которого начинается выполнение после загрузочной подготовки. Для C/C++ программ точка входа обычно ведёт сначала в runtime startup code (стартовый код runtime-а), который подготовит C runtime, аргументы, глобальные конструкторы и только потом вызовет `main` или `WinMain`.

Почему entry point хранится как RVA? Потому что image может быть загружен по разной базе. Если бы PE хранил абсолютный VA точки входа, при смене ImageBase его пришлось бы править. RVA говорит проще: “начинай на таком-то смещении от того места, куда ты загрузил image”.

Для PE32+ это особенно естественно: ImageBase может быть 64-битным, а AddressOfEntryPoint остаётся 32-битным RVA. Итоговый адрес выполнения получается только после сложения с реальной базой загрузки.

```
Пример:
  ImageBase           = 00007FF700000000h
  AddressOfEntryPoint = 0000000000001500h (RVA)

  EntryPoint VA       = 00007FF700001500h
```

Связь с исключениями не прямая, но методологически важная: если даже точка входа хранится как RVA, то тем более нельзя читать поля Exception Directory и `RUNTIME_FUNCTION` как file offsets. PE почти постоянно говорит координатами образа, а не координатами файла.

2.2.10. SectionAlignment и FileAlignment: почему файл и память выровнены по-разному

`SectionAlignment` (выравнивание секций в памяти) - поле Optional Header, которое говорит, по какой границе секции должны располагаться в virtual layout, то есть в загруженном image. Часто это значение связано с размером страницы памяти, например 0x1000.

`FileAlignment` (выравнивание секций в файле) - поле Optional Header, которое говорит, по какой границе секции расположены в raw layout, то есть на диске. Часто это может быть 0x200, хотя конкретные значения зависят от файла и правил формата.

Alignment (выравнивание) - это требование размещать начало области по адресу или смещению, кратному определённому числу. Если выравнивание 0x1000, начало секции должно попадать на границу, делящуюся на 0x1000. Это нужно для удобства и требований памяти, страниц, загрузки и производительности.

Почему выравнивание в файле и памяти разное? Потому что файл живёт в файловой системе и оптимизируется под хранение, а image в памяти живёт по правилам virtual memory (виртуальной памяти). Страницы памяти имеют свои размеры и права доступа. Loader должен удобно отображать секции на страницы и назначать им protection flags (права защиты): execute, read, write.

Эта разница объясняет, почему `RVA != file offset`. Секция может начинаться в файле по offset `0x400`, а в памяти по RVA `0x1000`. Внутри секции смещение сохраняется, но база разная. Поэтому для перевода нужна формула через section header, а не простое равенство.

> Если бы PE на диске и PE в памяти были одним и тем же массивом байтов, RVA и file offset почти совпадали бы. Но PE устроен не так: loader строит image по правилам выравнивания и секций.

2.2.11. SizeOfImage (размер образа): почему PE32+ не означает “образ может быть сколь угодно огромным”

`SizeOfImage` (размер образа) - поле Optional Header, которое задаёт общий размер image в памяти, включая все заголовки и секции с учётом SectionAlignment. Это не просто размер файла на диске. Это размер виртуальной области, которую занимает образ после загрузки.

В PE32+ адресное пространство шире, но это не означает, что один PE image должен занимать терабайты. Image - это конкретный модуль: EXE или DLL, его код, данные и таблицы. На практике он обычно намного меньше 4 ГБ. Поэтому большинство внутренних координат

image удобно хранить как 32-битные RVA.

Для парсера SizeOfImage важен как граница здравого смысла. Если Exception Directory указывает RVA за пределами SizeOfImage, это подозрительно. Если `RUNTIME_FUNCTION.EndAddress` выходит за размер образа, это тоже проблема. Если `UnwindInfoAddress` указывает за image, нормальный анализатор должен это заметить, а не слепо читать куда попало.

Для reverse engineering SizeOfImage помогает отделить “адрес внутри модуля” от “адрес где-то в процессе”. Например, VA может указывать в кучу, другой DLL, стек или в основной EXE. Чтобы понять, принадлежит ли адрес данному image, нужно сравнить его с диапазоном `[ImageBase, ImageBase + SizeOfImage)`. Полуоткрытая запись означает: начало включается, конец не включается.

```
Диапазон image в памяти:
  ImageBase <= VA < ImageBase + SizeOfImage

Если VA попадает сюда, можно вычислить:
  RVA = VA - ImageBase
```

2.2.12. PE32+ и ASLR: почему реальная база загрузки может отличаться от ImageBase

ASLR (Address Space Layout Randomization, рандомизация адресного пространства) - защитный механизм, при котором Windows может загружать модули по разным базовым адресам, чтобы усложнить эксплуатацию уязвимостей и предсказание адресов. Если адреса каждый раз разные, атакующему сложнее заранее знать, где находится нужная функция, гаджет или таблица.

`Preferred ImageBase` (предпочтительная база образа) - значение ImageBase в PE-файле, то есть адрес, по которому image хотел бы быть загружен. `Actual load base` (реальная база загрузки) - адрес, по которому loader действительно разместил image в конкретном процессе. Они могут совпадать, а могут отличаться.

В PE32+ ASLR особенно важен из-за широкого адресного пространства. Система получает больше свободы размещать модули по разным адресам. Но из-за этого reverse engineer должен постоянно помнить: ImageBase из файла - это не всегда реальная база в процессе. В статическом анализе инструмент может показывать preferred base. В отладчике ты видишь actual base. В дампе - actual base на момент падения.

RVA как раз помогает пережить смену базы. Если функция имеет RVA `0x2000`, то при одной загрузке она может быть по VA `0x00007FF700002000`, а при другой - по VA `0x00007FFA10002000`. RVA тот же, база разная. Поэтому PE-таблицы любят RVA: они остаются стабильными относительно image.

Relocation (релокация, перенос адресов) нужна для тех мест, где в образе всё-таки есть значения, зависящие от абсолютной базы. Loader применяет relocation table, если image загружен не по preferred base. Но таблицы вроде `RUNTIME_FUNCTION`, где хранятся image-relative offsets, изначально устойчивее к смене базы.

2.2.13. Subsystem и DLL characteristics: почему PE32+ описывает не только “64-битность”

`Subsystem` (подсистема) - поле Optional Header, которое говорит Windows, для какой среды предназначен image: например, console subsystem (консольная подсистема) или Windows GUI subsystem (графическая подсистема Windows). Это влияет на запуск и ожидания системы: консольное приложение и GUI-приложение стартуют по-разному с точки зрения окружения пользователя, хотя оба могут быть PE32+ x64.

`DllCharacteristics` (характеристики DLL/образа) - поле Optional Header с флагами, описывающими важные свойства образа: поддержка ASLR, NX compatibility (совместимость с запретом выполнения данных), Control Flow Guard и другие параметры защиты/загрузки. Название содержит DLL, но поле есть и в EXE image.

Зачем это в разделе про PE32+? Чтобы не свести Optional Header к одному Magic. Magic выбирает схему структуры. Но Optional Header в целом - паспорт загрузки image. Он говорит не только “32 или 64”, а ещё как загружать, где стартовать, сколько памяти резервировать под стек/кучу, какие защиты включены, какие data directories существуют.

Для анализа исключений и unwind эти поля не всегда центральные, но они формируют общий контекст. Например, NX влияет на то, можно ли исполнять данные; нарушение NX может проявляться как exception при попытке execute. ASLR влияет на actual base, а значит на перевод RVA в VA. Control Flow Guard может менять картину косвенных вызовов. Взрослый reverse engineer смотрит на PE не как на набор “полей для галочки”, а как на договор между файлом и Windows.

2.2.14. SizeOfStackReserve, SizeOfStackCommit, SizeOfHeapReserve, SizeOfHeapCommit: почему в PE32+ эти поля шире

`SizeOfStackReserve` (размер резервирования стека) - поле Optional Header, которое задаёт, сколько виртуального адресного пространства зарезервировать для стека основного потока. `Reserve` (резервирование) означает: диапазон адресов выделяется в адресном пространстве, но не обязательно сразу обеспечивается физической памятью.

`SizeOfStackCommit` (размер фиксации/коммита стека) - поле, которое задаёт, сколько памяти стека изначально commit-ится, то есть реально становится доступной с backing store. `Commit` (фиксация/коммит) ближе к “сделать память реально доступной”, а reserve - “забронировать адресный диапазон”.

`SizeOfHeapReserve` (размер резервирования кучи) и `SizeOfHeapCommit` (размер фиксации кучи) - аналогичные параметры для initial heap (начальной кучи) процесса. Heap (куча) - область динамической памяти, из которой runtime и программа выделяют объекты с жизнью, не привязанной строго к одному вызову функции.

Почему эти поля в PE32+ 64-битные? Потому что в 64-битном процессе размеры адресных резервов потенциально могут быть больше 32-битных ограничений. Даже если конкретная программа использует небольшие значения, формат должен уметь выразить 64-битные размеры.

Связь со стеком из первой главы прямая. Когда мы говорим, что поток имеет стек, этот стек не появляется из воздуха. Windows создаёт его при создании потока, используя системные правила и параметры образа для главного потока. Потом во время выполнения RSP движется внутри этой области. При исключении RSP показывает текущую вершину стека, а unwind пытается подняться по кадрам внутри этого же стекового диапазона.

Reserve и commit - важная пара терминов. Reserve бронирует адресный диапазон. Commit делает страницы реально доступными. Ошибка новичка - думать, что весь зарезервированный стек сразу занимает физическую RAM.

2.2.15. Почему PE32+ важен именно для `.pdata` и `.xdata`

Теперь соединяем тему PE32+ с главной линией книги. На Windows x64 unwind-информация для нормального non-leaf кода хранится в табличной модели. В PE32+ image есть Exception Directory, которая указывает на `RUNTIME_FUNCTION` entries. Эти entries обычно находятся в `.pdata`. Каждая запись связывает диапазон кода функции с `UNWIND_INFO`, обычно находящимся в `.xdata`.

`.pdata` (procedure data, данные процедур) - секция или область, где обычно лежит массив `RUNTIME_FUNCTION`. Название секции не является абсолютным контрактом, но в обычных MSVC/Windows x64 образах это типичный вариант. Procedure (процедура) здесь близко к “функция”.

`.xdata` (exception data, данные исключений) - секция или область, где обычно лежит `UNWIND_INFO` и дополнительные данные обработчиков. Это не “код обработки исключений”, а метаданные: компактное описание, как функция построила кадр и есть ли у неё handler.

`RUNTIME_FUNCTION` (запись функции времени выполнения) - структура, которая содержит три ключевых RVA: BeginAddress (начало диапазона функции), EndAddress (конец диапазона функции) и UnwindInfoAddress (адрес unwind-информации). Все эти значения image-relative, то есть относительны к базе image.

Почему PE32+ важен? Потому что именно в PE32+ x64 модель эти таблицы становятся стандартным способом связать код функции с unwind-данными. На x86 историческая SEH-модель была сильно завязана на стековую цепочку. На x64 Windows ожидает табличное описание функций. Значит, анализ `.pdata/.xdata` начинается с правильного понимания PE32+ image.

Самая важная мысль этого подраздела: PE32+ даёт контейнер и адресную модель, `.pdata` даёт функцию как диапазон кода, `.xdata` даёт рецепт раскрутки кадра, `RtlVirtualUnwind` использует этот рецепт для перехода к caller context. Все четыре слоя связаны.

2.2.16. Почему x64 unwind использует 32-битные image-relative offsets

Тут стоит задержаться, потому что это выглядит странно. x64 - 64-битная архитектура. PE32+ - 64-битный image. `ImageBase` - 64-битный. Почему тогда `RUNTIME_FUNCTION` хранит 32-битные поля?

Ответ: потому что эти поля не являются полными адресами. Они являются image-relative offsets (смещениями относительно image base). Чтобы описать положение функции внутри конкретного модуля, не нужно хранить полный 64-битный VA. Достаточно сказать: “функция начинается на 0x1200 байт от начала image”. Реальный VA получится только после прибавления base address.

Это экономит место и делает таблицы независимыми от реальной базы загрузки. Если ASLR загрузит модуль по другой базе, все RVA остаются теми же. Меняется только ImageBase, который прибавляется при вычислении VA.

Для parser-а это даёт строгое правило: поля `BeginAddress`, `EndAddress`, `UnwindInfoAddress` нельзя трактовать как 64-битные VA. Нельзя трактовать их и как file offset. Это 32-битные RVA. Если инструмент ошибается здесь, он не просто “немного смещает адрес”, он разрушает весь смысл unwind-анализа.

```
RUNTIME_FUNCTION в x64 концептуально:
  BeginAddress      -> RVA начала функции
  EndAddress        -> RVA конца функции
  UnwindInfoAddress -> RVA структуры UNWIND_INFO

Для runtime в процессе:
  BeginVA = ModuleBase + BeginAddress

Для статического PE-парсера:
  нужно перевести BeginAddress как RVA через Section Table
```

2.2.17. Parser (парсер) для PE32+: как он должен думать, чтобы не врать

`Parser` (парсер, программа разбора формата) - код, который читает бинарный формат и превращает байты в осмысленные структуры. PE parser читает DOS Header, PE Signature, COFF File Header, Optional Header, Section Table, Data Directories и секции.

Хороший PE32+ parser не должен начинать с предположения “сейчас я прочитаю структуру как PE32+”. Он должен дойти до Optional Header Magic и только после этого выбрать схему чтения. Потом он должен сверить Machine, размеры заголовков, количество секций, границы файла и image, Data Directory count, попадание RVA в секции.

Почему это важно для reverse engineering? Потому что плохой parser создаёт ложную уверенность. Он может напечатать таблицу `.pdata`, но если он перепутал PE32 и PE32+, все адреса после Optional Header могут быть смещены. Он может найти Exception Directory, но если он не проверил NumberOfRvaAndSizes, он может читать несуществующую запись. Он может перевести RVA в file offset, но если не учёл VirtualSize/SizeOfRawData и границы секций, он может попасть в неправильное место.

`NumberOfRvaAndSizes` (количество записей RVA+Size в Data Directory) - поле Optional Header, которое говорит, сколько записей Data Directory реально доступно. Нельзя просто считать, что все возможные каталоги существуют. Parser должен проверить, что нужный индекс помещается в заявленное количество. Это простая, но важная защита от чтения за пределами заголовка.

`Bounds check` (проверка границ) - проверка, что чтение структуры не выходит за пределы файла, секции или image. Для PE-разбора это не бюрократия, а защита от мусора, повреждённых файлов и намеренно злых образов. Если parser читает `UNWIND_INFO` по RVA, он должен убедиться, что RVA переводится в допустимую область, а сама структура помещается в доступные данные.

> Правильный PE32+ parser должен быть недоверчивым. PE-файл - это входные данные. В reverse engineering, malware analysis и security tooling входным данным нельзя верить.

2.2.18. Типичные ошибки при понимании PE32+

Первая ошибка - думать, что PE32+ означает “32-битный плюс что-то”. Нет. Для нашей линии PE32+ - это 64-битный image-формат Windows.

Вторая ошибка - читать PE32+ Optional Header как PE32. Из-за отсутствия `BaseOfData` и расширенного `ImageBase` смещения полей после `BaseOfCode` будут неверными.

Третья ошибка - думать, что все адреса в PE32+ должны быть 64-битными. Нет. Полные VA могут быть 64-битными, но RVA внутри image часто остаются 32-битными. Это особенно важно для `RUNTIME_FUNCTION`.

Четвёртая ошибка - путать RVA, VA и file offset. Это ломает чтение секций, Data Directory, Exception Directory, `.pdata`, `.xdata` и любой ручной разбор.

Пятая ошибка - ориентироваться только на имена секций. `.pdata` и `.xdata` полезны как привычные имена, но формально путь к exception table идёт через Data Directory. Имя секции может быть нестандартным, объединённым или намеренно изменённым.

Шестая ошибка - считать ImageBase из файла реальной базой в процессе. Из-за ASLR actual load base может отличаться. В статике ты работаешь с preferred ImageBase, в динамике - с actual module base.

Седьмая ошибка - не проверять согласованность Machine, Magic, NumberOfSections, SizeOfOptionalHeader, NumberOfRvaAndSizes, SizeOfImage и границ секций. В простых учебных файлах всё красиво, но в реальном reverse engineering могут встретиться упаковщики, обфускаторы, повреждённые дампы и нестандартные линкеры.

2.2.19. PE32+ и WoW64: почему 32-битная программа на 64-битной Windows не становится PE32+

`WoW64` (Windows-on-Windows 64-bit, подсистема запуска 32-битных программ на 64-битной Windows) - слой совместимости, который позволяет 64-битной Windows запускать 32-битные процессы. Это важный термин, потому что он отделяет “операционная система 64-битная” от “конкретный процесс 64-битный”.

Если ты запускаешь 32-битный EXE на 64-битной Windows, этот EXE не превращается в PE32+. Он остаётся PE32 image. У него 32-битный Optional Header, 32-битный код x86, другая модель регистров, другие правила вызова и другая историческая SEH-модель. Windows создаёт для него 32-битное окружение совместимости. Снаружи ОС 64-битная, но внутри процесса действуют 32-битные правила.

Почему это важно для книги? Потому что “у меня Windows x64” ещё не означает “я анализирую x64 PE32+”. Файл может быть 32-битным PE32, 64-битным PE32+, DLL может быть одной разрядности, процесс другой разрядности, отладчик может иметь ограничения при переходе между режимами. Реверсер должен смотреть на сам image, а не на общую систему.

Для твоего анализа unwind это принципиально. Если файл PE32 x86, ты не должен ожидать стандартную x64 `.pdata/.xdata` модель для каждой non-leaf функции. Если файл PE32+ x64, наоборот, отсутствие или странность unwind metadata становится важным сигналом. Сначала определяем формат и архитектуру, потом применяем правила.

> Простая проверка: 64-битная Windows - это свойство ОС. PE32+ - свойство конкретного image. WoW64 - слой, который позволяет PE32 жить внутри 64-битной Windows, но не меняет его формат на PE32+.

2.2.20. 64-bit pointer (64-битный указатель): почему размер указателя меняет мышление реверсера

`Pointer` (указатель) - значение, которое интерпретируется как адрес. В C это может быть `int*`, `char*`, `void*`, указатель на структуру, указатель на функцию. На машинном уровне указатель - просто число, но программа использует его как координату памяти.

`64-bit pointer` (64-битный указатель) в x64-процессе обычно занимает 8 байт. Это влияет не только на размер переменных. Это меняет layout (раскладку) структур, смещения полей, выравнивание, calling convention и вид дизассемблированного кода. Если в 32-битной структуре поле-указатель занимало 4 байта, то в 64-битной оно занимает 8. Следующие поля могут сместиться. Размер всей структуры может измениться.

Почему это связано с PE32+? Потому что PE32+ сообщает: это image для 64-битного выполнения. Значит, код внутри использует 64-битные указатели и x64 ABI. Когда reverse engineer смотрит на структуру в памяти, он не может переносить x86-смещения автоматически. То, что на x86 было `[ecx+8]`, на x64 может стать `[rcx+10h]` из-за расширения указателей и выравнивания.

`Pointer-sized field` (поле размера указателя) - поле, размер которого зависит от разрядности процесса. В 32-битном процессе оно 4 байта, в 64-битном - 8 байт. Многие Windows-типы специально скрывают это за typedef-ами: `ULONG_PTR`, `SIZE_T`, `INT_PTR`. Их смысл: “размер должен соответствовать размеру указателя”.

Ошибка новичка - читать x64-структуры как x86-структуры. В PE-разборе это проявляется при Optional Header. В анализе памяти - при структурах с указателями. В анализе вызовов - при аргументах и возвратах. В unwind - при сохранении 64-битных nonvolatile-регистров и 8-байтных адресов возврата на стеке.

2.2.21. Canonical address (канонический адрес): почему не любое 64-битное число является нормальным адресом

`Canonical address` (канонический адрес) - это адрес, который допустим в правилах x64-адресации для текущей реализации архитектуры. Хотя регистр 64-битный и может хранить любое 64-битное число, не каждое такое число является допустимым виртуальным адресом. В x64 используются правила канонической формы адреса: старшие биты должны соответствовать допустимому расширению знака для реализованной ширины виртуального адреса.

Для начинающего это может звучать слишком низкоуровнево, но практический смысл простой: значение в регистре может выглядеть как “64-битное число”, но это ещё не значит, что оно является валидным указателем. В отладке access violation можно увидеть адрес вроде `0xCCCCCCCCCCCCCCCC`, `0xFFFFFFFFFFFFFFFF` или другое явно подозрительное значение. Нужно понимать, что указатель - это не просто “любые 8 байт”.

Как это связано с PE32+? PE32+ даёт 64-битный image и 64-битные VA, но валидность адреса всё равно определяется правилами процесса и ОС: принадлежит ли адрес user-mode диапазону, сопоставлена ли страница, есть ли права доступа, каноничен ли адрес, указывает ли он в image, стек, кучу или другой модуль.

Для unwind это тоже полезно: когда stack walking восстанавливает return address, полученный адрес должен быть похож на допустимый code address (адрес кода). Если восстановленный RIP указывает в неканоническое значение, несопоставленную память или данные без execute, это может говорить о повреждении стека, неверном unwind-описании или ошибке анализа.

2.2.22. PE32+ глазами дизассемблера и декомпилятора

`Disassembler` (дизассемблер) - инструмент, который превращает машинные байты обратно в инструкции ассемблера. `Decompiler` (декомпилятор) - инструмент, который пытается подняться ещё выше и восстановить псевдо-C или другой высокоуровневый вид. Оба инструмента должны понимать PE32+ не как “обложку”, а как источник координат и контекста.

Когда Ghidra, IDA или другой анализатор открывает PE32+ файл, он читает заголовки, понимает архитектуру, строит memory map (карту памяти), расставляет секции по virtual addresses, находит imports, exports, entry point, exception directory. Если инструмент правильно понял PE32+, он сможет показывать адреса кода так, как они будут выглядеть в image, а не только как смещения в файле.

Для x64 особенно важны function boundaries (границы функций). `RUNTIME_FUNCTION` в `.pdata` часто помогает анализатору понять, где начинаются и заканчиваются функции. Это не единственный способ найти функции, но очень ценный источник. Если таблица `.pdata` корректная, она помогает дизассемблеру не гадать на одних prolog signatures. Если `.pdata` повреждена или намеренно искажена, инструмент может ошибаться в границах функций.

`Function boundary` (граница функции) - место, где функция начинается и где заканчивается. Для человека это кажется простым: функция в исходнике имеет начало и конец. В бинарнике без символов это не всегда очевидно. Таблица `RUNTIME_FUNCTION` задаёт диапазоны кода, и поэтому PE32+ unwind metadata может быть не только механизмом исключений, но и подсказкой для реверса.

Важно: декомпилятор может красиво нарисовать псевдокод, но если исходная PE32+ координатная модель была понята неверно, псевдокод тоже будет стоять на кривом фундаменте. Поэтому сначала формат, потом дизассемблирование, потом декомпиляция, а не наоборот.

2.2.23. PE32+ как контракт: кто что обещает

`Contract` (контракт) в техническом смысле - набор взаимных ожиданий между частями системы. PE32+ является контрактом между linker-ом, loader-ом, Windows runtime, отладчиком, анализатором и самим машинным кодом.

Linker (линкер) обещает создать корректный PE32+ image: правильный Magic, Machine, Section Table, Optional Header, Data Directories, Exception Directory, unwind tables. Loader (загрузчик) обещает читать эти данные по правилам PE, отображать секции, применять релокации, настраивать imports, создать image. Windows runtime обещает использовать function tables и unwind data при исключениях, stack walking и отладочной поддержке. Debugger (отладчик) и reverse engineer ожидают, что эти слои согласованы.

Если контракт нарушен, возможны разные последствия. Файл может вообще не загрузиться. Может загрузиться, но падать при исключении. Может работать нормально до первого stack walk. Может ломать call stack в дампе. Может обманывать дизассемблер. Может быть

намеренно сделан таким упаковщиком или защитой.

Для static /dynamic мысль такая: анализ PE32+ - это анализ контракта. Ты проверяешь не просто “какие поля есть”, а “согласованы ли обещания разных слоёв”. Machine согласован с Magic? Data Directory указывает внутрь image? Exception Directory размер кратен размеру `RUNTIME_FUNCTION`? BeginAddress меньше EndAddress? UnwindInfoAddress указывает в читаемую область? Unwind-коды соответствуют прологу? Каждая такая проверка превращает формат PE из теории в инструмент анализа.

2.2.24. Главная связка раздела

PE32+ - это 64-битный вариант PE image-формата Windows. Он сохраняет общую архитектуру PE, но меняет ключевые части Optional Header: Magic становится `0x20B`, отсутствует `BaseOfData`, `ImageBase` и некоторые размеры становятся 64-битными. При этом множество координат внутри image остаются 32-битными RVA, потому что они измеряются от базы образа, а не являются полными виртуальными адресами процесса.

Для x64 это важно не только потому, что программа использует 64-битные регистры и указатели. Это важно потому, что Windows x64 exception/unwind model живёт внутри PE32+ image. Exception Directory ведёт к `RUNTIME_FUNCTION` entries, `.pdata` хранит диапазоны функций, `.xdata` хранит `UNWIND_INFO`, а `RtlVirtualUnwind` использует эти данные, чтобы перейти от текущего контекста к вызывающему.

Если коротко, но уже с правильным смыслом: PE32+ задаёт контейнер и адресные правила для 64-битного Windows image; x64 unwind использует эти правила, чтобы найти таблицы функций и восстановить стек. Поэтому нельзя изучать `.pdata` и `.xdata` отдельно от PE32+. Они не висят в воздухе. Они являются частью PE32+ образа и читаются по его координатной системе.

> Следующий раздел - RVA, VA, `ImageBase` и sections - нужно будет читать уже не как набор терминов, а как продолжение этой мысли: PE32+ говорит Windows, где лежит образ, как переводить координаты и где искать таблицы, без которых x64 unwind не работает.

2.3. Что такое RVA, VA, ImageBase и секции

В предыдущих разделах мы договорились о главном: PE file (Portable Executable file, переносимый исполняемый файл) - это не просто “exe на диске”, а инструкция для Windows loader-а (загрузчика Windows), как превратить файл в executable image (исполняемый образ) в памяти. Теперь нужно разобрать самую опасную для новичка тему PE: разные системы координат. Если здесь остаться на уровне “RVA - это относительный адрес, VA - виртуальный адрес”, дальше почти всё будет шататься: .pdata будет читаться не оттуда, .xdata будет искаться не там, адреса из отладчика не будут совпадать с адресами в hex-редакторе, а Ghidra будет казаться магией.

Эта глава намеренно идёт медленно. Мы будем вводить мало терминов за раз и каждый термин привязывать к реальной задаче: где лежит байт, кто его ищет, в какой момент файл превращается в память, почему одно число может быть правильным в одной системе координат и бессмысленным в другой. Для темы Windows x64 unwind это не второстепенная подготовка, а база: поля RUNTIME_FUNCTION (запись функции для unwind) хранят RVA, Exception Directory (каталог исключений) задаётся через RVA и размер, .pdata (секция с procedure data) и .xdata (секция с unwind/exception data) надо находить через карту секций, а RtlLookupFunctionEntry (функция поиска записи функции) получает адрес выполнения уже в памяти.

> Главная идея раздела: PE-анализ - это не “посмотреть адрес”. Это постоянный перевод между тремя мирами: file offset (смещение в файле), RVA (relative virtual address, относительный виртуальный адрес) и VA (virtual address, виртуальный адрес). Ошибка в переводе между этими мирами почти всегда даёт неправильный разбор.

2.3.1. Почему вообще существуют разные адреса

Начнём не с аббревиатур, а с простой ситуации. У тебя есть файл app.exe. Он лежит на диске. В hex-редакторе ты можешь открыть этот файл и перейти, например, к позиции 0x400. Hex-редактор не знает, где этот файл будет загружен в память. Он просто показывает байты файла: начало файла, потом байт номер 1, байт номер 2, байт номер 0x400. Это мир файла.

Теперь ты запускаешь app.exe. Windows loader (загрузчик Windows) читает PE headers (PE-заголовки), понимает, какие sections (секции) есть внутри файла, и отображает эти секции в virtual address space (виртуальное адресное пространство) процесса. После загрузки те же байты уже имеют адреса в памяти: например, код может оказаться по адресу 0x00007FF6A1201000. Это мир выполнения.

Между этими двумя мирами находится третий язык - RVA. PE-файл не может заранее знать окончательные VA всех своих данных, потому что из-за ASLR (Address Space Layout Randomization, рандомизация адресного пространства) образ может быть загружен по разным базам. Поэтому PE часто говорит не “абсолютный адрес 0x00007FF6A1201000”, а “смещение 0x1000 от базы образа”. Это и есть RVA.

Получаются три разных вопроса. File offset (смещение в файле) отвечает: “где этот байт лежит в файле на диске?” RVA (относительный виртуальный адрес) отвечает: “где этот байт будет находиться внутри загруженного образа, если считать от ImageBase?” VA (виртуальный адрес) отвечает: “по какому адресу этот байт доступен в памяти

конкретного процесса сейчас?”

Одна и та же сущность может иметь три координаты. Допустим, инструкция функции лежит в файле по смещению 0x600, имеет RVA 0x1200 и во время отладки видна по VA 0x00007FF612341200. Все три числа могут относиться к одному и тому же байту, но каждое число правильно только в своей ситуации. Hex-редактору нужен file offset. PE-заголовкам чаще нужен RVA. Отладчику и CPU во время исполнения нужен VA.

> Термин “coordinate system” (система координат) здесь не математическая роскошь, а практическая защита от ошибок. Когда ты видишь число вроде 0x1200, нельзя сразу говорить “это адрес”. Нужно спросить: это file offset, RVA, VA, смещение внутри структуры, смещение внутри пролога или что-то ещё?

2.3.2. Address (адрес): почему это слово слишком общее

Address (адрес) - это число, которое указывает на место в некотором пространстве. Но слово “некотором” здесь решающее. Адрес в файле, адрес в виртуальной памяти и смещение внутри структуры - разные вещи, хотя все они могут выглядеть как шестнадцатеричные числа.

В исходном C-коде адрес часто всплывает как pointer (указатель). Pointer (указатель) - это значение, которое программа интерпретирует как адрес объекта, функции или области памяти. Например, если переменная типа int* хранит 0x000000000012FF00, программа может попытаться прочитать int по этому адресу. Но указатель существует во время выполнения, то есть в мире VA. Он не обязан совпадать с позицией байта в EXE-файле.

Offset (смещение) - близкий, но другой термин. Offset (смещение) - это расстояние от выбранной базы. Смещение не существует без ответа на вопрос “от чего?”. File offset - от начала файла. RVA - от ImageBase. Смещение поля структуры - от начала структуры. CodeOffset в UNWIND_CODE - от начала пролога функции. Все они “offset”, но базы разные, и путать их нельзя.

Самая вредная привычка - смотреть на число и автоматически называть его адресом. В PE это быстро ломает анализ. Например, значение 0x3000 в Data Directory почти всегда будет RVA, а не file offset. Значение 0x3000 в hex-редакторе как позиция курсора - file offset. Значение 0x00007FF600003000 в отладчике - VA. Одинаковые последние цифры могут обманывать, но система координат решает всё.

2.3.3. File offset (смещение в файле): адрес мира диска

File offset (смещение в файле) - это позиция байта относительно начала файла на диске. Если file offset равен 0x400, это означает: открой файл, начни с первого байта файла как с нуля, отсчитай 0x400 байт и читай оттуда. Это понятие понимает hex editor (шестнадцатеричный редактор), обычный файловый parser (парсер файла) и любой код, который читает файл как поток байтов.

File offset нужен, когда ты анализируешь файл до загрузки. Например, твой инструмент открывает app.exe через ReadFile, mmap или обычный Python open(). В этот момент у тебя ещё нет “адреса в процессе”. У тебя есть файл. Ты читаешь байты по позициям. PE headers (PE-заголовки) тоже физически лежат по file offset: DOS Header (DOS-заголовок) начинается с начала файла, поле e_lfanew (смещение до PE-заголовка) указывает, где

искать PE Signature (PE-сигнатуру), а Section Table (таблица секций) лежит после заголовков.

Но file offset не говорит, где байт окажется в памяти. Это особенно важно из-за sections (секций). PE-файл на диске и PE-image в памяти не обязаны иметь одинаковую раскладку. Секция может начинаться в файле по одному смещению, а в памяти - по другому виртуальному адресу относительно ImageBase. Кроме того, размеры на диске и в памяти могут различаться. Часть данных может быть только в памяти и не иметь байтов в файле, например нулевая область .bss-подобных данных.

Типичная ошибка: увидеть в PE-структуре значение 0x2000 и попытаться перейти в hex-редакторе на file offset 0x2000, хотя это значение было RVA. Иногда случайно что-то там найдётся, и это опаснее, чем явная ошибка: анализатор начнёт читать правдоподобный мусор.

> Практический смысл file offset: это координата для чтения байтов из файла. Если твой инструмент сейчас работает с байтами EXE на диске, ему нужны file offsets. Если он читает поле PE, которое задано как RVA, он обязан сначала перевести RVA в file offset через section table.

2.3.4. VA (Virtual Address, виртуальный адрес): адрес мира выполнения

VA (Virtual Address, виртуальный адрес) - это адрес в virtual address space (виртуальном адресном пространстве) процесса. Когда программа уже запущена, CPU выполняет инструкции по VA, регистры содержат VA, указатели в 64-битном процессе обычно являются VA, отладчик показывает VA, exception context (контекст исключения) содержит RIP как VA.

Virtual address space (виртуальное адресное пространство) - это не физическая оперативная память напрямую. Windows и процессор создают для процесса иллюзию собственного адресного пространства. Процесс работает с виртуальными адресами, а система памяти через page tables (таблицы страниц) переводит их в physical memory (физическую память) или в другие backing stores (источники хранения), например отображённые файлы. Если виртуальный адрес не сопоставлен или права доступа не позволяют операцию, возникает exception (исключение), например access violation.

Почему адрес называется virtual (виртуальный)? Потому что один и тот же VA в разных процессах может указывать на разные физические страницы. Процесс A может иметь по адресу 0x0000000140001000 свой код, а процесс B по такому же числу может иметь совсем другой модуль или вообще ничего. Поэтому VA нужно всегда понимать в контексте конкретного процесса.

Для реверсера VA особенно заметен в debugger (отладчике). Когда WinDbg, x64dbg, Visual Studio Debugger или другой отладчик показывает, что RIP равен 00007FF6A1201430, это VA. Этот адрес уже включает actual ImageBase (фактическую базу образа) конкретного процесса. Если включён ASLR, при следующем запуске тот же код может оказаться по другому VA, но его RVA внутри образа обычно останется тем же.

VA нужен, когда код уже находится в памяти. RtlLookupFunctionEntry (функция поиска записи функции) получает ControlPc (адрес управления, то есть адрес в коде, где сейчас

находится выполнение) как адрес в памяти. Чтобы найти запись функции, система должна связать этот VA с module base (базой модуля), а затем фактически работать с image-relative values (значениями относительно образа), потому что .pdata хранит не абсолютные VA, а RVA.

2.3.5. ImageBase (база образа): ноль координат PE-образа

ImageBase (база образа) - это адрес, от которого отсчитываются RVA внутри загруженного image (образа). Если упростить, ImageBase - это “начальная точка” PE-образа в виртуальной памяти. Если RVA говорит “смещение 0x3000 от начала образа”, то VA получается как ImageBase + 0x3000.

```
VA = ImageBase + RVA
RVA = VA - ImageBase
```

Но у ImageBase есть важная тонкость: бывает preferred ImageBase (предпочтительная база образа) и actual ImageBase (фактическая база образа). Preferred ImageBase записана в Optional Header (дополнительном заголовке) PE-файла. Это адрес, по которому linker (компоновщик) хотел бы видеть образ в памяти. Actual ImageBase - это адрес, по которому Windows loader реально загрузил образ в конкретном процессе.

Linker (компоновщик) - это программа, которая собирает объектные файлы и библиотеки в итоговый EXE или DLL. Он формирует PE-структуру, секции, таблицы, адреса относительно образа, импорты и релокации. Loader (загрузчик Windows) работает позже: уже при запуске он берёт готовый PE-файл и отображает его в память процесса.

Почему actual ImageBase может отличаться от preferred ImageBase? Первая причина - адрес уже занят. Вторая и современно более важная - ASLR (Address Space Layout Randomization, рандомизация адресного пространства). ASLR специально меняет базы загрузки модулей, чтобы усложнить эксплуатацию уязвимостей: атакующему труднее заранее знать, где в памяти окажется код или полезная таблица.

Вот почему PE не может везде хранить готовые VA. Если бы PE-файл был забит абсолютными адресами, каждое изменение базы требовало бы массовой коррекции. Частично это решается relocations (релокациями, таблицей исправлений адресов), но для внутренних указателей на структуры PE гораздо удобнее хранить RVA. RVA не меняется при переносе образа: меняется только ImageBase, а формула остаётся той же.

В 64-битном PE32+ ImageBase является 64-битным полем, потому что 64-битный процесс работает с 64-битной моделью адресов. Но многие внутренние ссылки PE всё равно остаются 32-битными RVA. Это не противоречие. RVA - это смещение внутри образа, а размер обычного PE-образа далеко не требует 64 бит. Поэтому .pdata в x64 хранит BeginAddress, EndAddress и UnwindInfoAddress как 32-битные image-relative offsets (смещения относительно образа), а не как 64-битные указатели.

> Типичная ошибка с ImageBase: использовать preferred ImageBase из файла там, где нужен actual ImageBase процесса. Если ты анализируешь файл статически, preferred база может быть удобной для расчётов. Если ты анализируешь живой процесс или dump, нужно брать фактическую базу загруженного модуля.

2.3.6. RVA (Relative Virtual Address, относительный виртуальный адрес): главный язык PE

RVA (Relative Virtual Address, относительный виртуальный адрес) - это смещение от ImageBase до объекта внутри загруженного image. Если ImageBase равен 0x0000000140000000, а RVA равен 0x0000000000001234, то VA равен 0x0000000140001234. RVA “относительный”, потому что сам по себе не содержит полной базы. Он говорит: “от начала образа пройти столько-то байт”.

Почему RVA называется virtual address, хотя это смещение? Потому что он относится не к сырому файлу на диске, а к virtual layout (виртуальной раскладке) образа после загрузки. Он описывает положение объекта в памяти относительно ImageBase. Это не file offset. Именно эта фраза должна закрепиться: RVA - это координата будущей/загруженной памяти, а не позиция в файле.

RVA нужен PE-формату потому, что образ должен быть relocatable (перемещаемым). Если модуль загружен по preferred ImageBase, RVA легко превращается в VA. Если модуль загружен по другой базе из-за ASLR, RVA всё равно остаётся тем же: меняется только ImageBase. Это даёт стабильную внутреннюю адресацию внутри образа.

Data Directory (каталог данных) использует RVA, чтобы указать на важные таблицы: Import Directory (каталог импортов), Export Directory (каталог экспортов), Resource Directory (каталог ресурсов), Base Relocation Table (таблица релокаций), Exception Directory (каталог исключений) и другие. Section Table тоже описывает секции через VirtualAddress (виртуальный адрес секции относительно ImageBase), то есть фактически через RVA.

Для нашей книги самый важный случай - Exception Directory (каталог исключений). В PE32+ x64 он указывает на массив RUNTIME_FUNCTION (записей функций для unwind). Эти записи обычно находятся в .pdata. Внутри каждой RUNTIME_FUNCTION поля BeginAddress, EndAddress и UnwindInfoAddress также являются RVA. То есть весь путь от каталога исключений до unwind-информации говорит на языке RVA.

```
Пример:
ImageBase = 0x0000000140000000
RVA       = 0x0000000000003000
VA        = 0x0000000140003000

Если ASLR загрузил образ иначе:
Actual ImageBase = 0x00007FF6A1200000
тот же RVA       = 0x0000000000003000
новый VA         = 0x00007FF6A1203000
```

Обрати внимание: RVA не изменился. Именно это делает его удобным для PE. Если твой анализатор хранит результаты по RVA, они часто остаются стабильными между запусками. Если он хранит только VA, ASLR может каждый раз менять числа.

2.3.7. Section (секция): крупная область PE-файла с отдельной ролью

Section (секция) - это крупный блок кода или данных внутри PE/COFF файла. Секция нужна, чтобы PE мог разделить содержимое файла по ролям и правилам загрузки. Код должен быть исполняемым, но обычно не должен быть изменяемым. Строковые

константы должны читаться, но не выполняться. Глобальные изменяемые данные должны читаться и записываться. Таблицы unwind должны быть доступны для чтения системе и отладчикам, но не являются обычным исполняемым кодом.

Секция имеет имя, например .text, .rdata, .data, .pdata, .xdata, .idata, .reloc. Имя полезно для человека и инструментов, но нельзя строить анализ только на имени. PE-формат важнее определяет секцию через Section Header (заголовок секции): там есть VirtualAddress (виртуальное положение секции относительно ImageBase), VirtualSize (виртуальный размер), PointerToRawData (смещение сырых данных секции в файле), SizeOfRawData (размер сырых данных в файле), Characteristics (характеристики/права секции).

Section Header (заголовок секции) - это запись в Section Table (таблице секций), которая объясняет loader-у и parser-у, где секция находится в файле и как она должна выглядеть в памяти. Это мост между raw layout (сырой раскладкой файла) и virtual layout (виртуальной раскладкой образа).

> По-человечески: секция - это контейнер с ролью. Section Header - паспорт этого контейнера. PointerToRawData говорит, где контейнер начинается в файле. VirtualAddress говорит, где он должен оказаться в памяти относительно ImageBase. Characteristics говорят, можно ли его читать, писать, выполнять.

На практике .text часто содержит executable code (исполняемый код). .rdata часто содержит read-only data (данные только для чтения): строки, константы, таблицы, иногда RTTI и служебные структуры. .data содержит initialized writable data (инициализированные изменяемые данные). .idata содержит import data (данные импортов), через которые программа связывается с DLL. .reloc содержит base relocations (релокации базы), необходимые при загрузке не по preferred ImageBase. Для x64 unwind особенно важны .pdata и .xdata: .pdata обычно содержит RUNTIME_FUNCTION entries, а .xdata - UNWIND_INFO и связанные exception data.

Но важно не абсолютизировать имена. Упаковщики, обфускаторы, нестандартные линкеры и ручные сборки могут переименовывать секции или класть данные в необычные места. Нормальный анализатор должен опираться не только на имя .pdata, а на Exception Directory, размеры, диапазоны, права, выравнивание и валидность структур.

2.3.8. Raw layout (сырая раскладка файла) и virtual layout (виртуальная раскладка образа)

Raw layout (сырая раскладка файла) - это то, как байты лежат на диске. Это мир file offset. Hex-редактор видит именно raw layout. Если секция .text имеет PointerToRawData = 0x400, значит её сырые байты начинаются в файле с позиции 0x400.

Virtual layout (виртуальная раскладка образа) - это то, как эти байты должны выглядеть после загрузки в память. Это мир RVA и VA. Если та же секция .text имеет VirtualAddress = 0x1000, значит в памяти она начинается с RVA 0x1000, а её VA будет ImageBase + 0x1000.

Почему эти раскладки не совпадают? Потому что файл и память имеют разные требования. Файл хочет компактности и выравнивания по FileAlignment (выравнивание файла). Память хочет выравнивания по SectionAlignment (выравнивание секций в памяти), часто по размеру страницы или близкому значению. Кроме того, loader может

добавлять нулевые байты в памяти там, где в файле нет сырых данных, например если VirtualSize больше SizeOfRawData.

FileAlignment (выравнивание файла) - это правило, по которому секции выравниваются внутри файла. Например, если FileAlignment = 0x200, начало сырых данных секции обычно будет кратно 0x200. Это удобно для файлового формата и хранения на диске.

SectionAlignment (выравнивание секций в памяти) - это правило, по которому секции располагаются в виртуальном образе. Например, если SectionAlignment = 0x1000, секции в памяти обычно начинаются на RVA, кратных 0x1000. Это связано с моделью страниц памяти и правами доступа.

Отсюда появляется главная практическая проблема: нельзя перевести RVA в file offset простой формулой “offset = RVA”. Нужно найти секцию, в диапазон которой попадает RVA, и использовать поля этой секции. Именно section table делает перевод между мирами.

2.3.9. Поля Section Header: что именно нужно для перевода адресов

Чтобы перевести RVA в file offset, нужно понимать несколько полей Section Header. Разберём их медленно, потому что это один из главных рабочих инструментов PE-анализа.

Name (имя секции) - короткое имя секции, например .text или .pdata. Оно помогает человеку понять предполагаемую роль секции, но не является абсолютной гарантией содержимого. Для reverse engineer-а имя - подсказка, а не доказательство.

VirtualAddress (виртуальный адрес секции, точнее RVA начала секции) - это RVA, с которого секция начинается в загруженном image. Если VirtualAddress = 0x2000, а ImageBase = 0x140000000, то начало секции в памяти будет по VA 0x140002000. Несмотря на слово Address, внутри Section Header это не полный VA, а именно RVA.

VirtualSize (виртуальный размер) - размер секции в памяти. Он говорит, сколько места секция занимает в virtual layout. Если VirtualSize больше SizeOfRawData, хвост секции в памяти может быть заполнен нулями и не иметь соответствующих байтов в файле. Это важно для .bss-подобных областей и некоторых данных.

PointerToRawData (указатель на сырые данные) - file offset, где секция начинается в файле. Это уже не RVA. Это координата диска. Если PointerToRawData = 0x600, значит байты секции в файле начинаются с offset 0x600.

SizeOfRawData (размер сырых данных) - размер секции в файле. Это не всегда равно VirtualSize. Если SizeOfRawData меньше VirtualSize, часть секции существует только в памяти как нули. Если SizeOfRawData больше VirtualSize, parser должен быть осторожен: лишние выровненные байты могут быть padding (добивкой), а не реальными данными секции.

Characteristics (характеристики секции) - набор флагов, описывающих свойства секции: содержит ли код, содержит ли инициализированные данные, можно ли читать, писать, выполнять. Например, секция кода обычно имеет execute/read права, данные - read/write, read-only tables - read. Для анализа защит и аномалий важно проверять, не выглядит ли секция с unwind-таблицами исполняемой или изменяемой без причины.

```
Условная запись секции:
Name             = .pdata
VirtualAddress   = 0x00006000
VirtualSize      = 0x00000420
PointerToRawData = 0x00004000
SizeOfRawData    = 0x00000600
Characteristics  = read-only initialized data
```

Эта запись говорит: в памяти .pdata начинается с RVA 0x6000, а в файле её сырые байты начинаются с offset 0x4000. Если Exception Directory указывает RVA 0x6000, нормальный file parser должен перевести его в file offset 0x4000, а не прыгать в файле на 0x6000.

2.3.10. Как перевести RVA в file offset

RVA to file offset translation (перевод RVA в смещение файла) - это процедура, которая нужна любому статическому PE-анализатору. Статический анализатор читает файл на диске, но многие PE-поля указывают на данные через RVA. Значит, перед чтением байтов нужно найти, где этот RVA физически лежит в файле.

Идея такая: каждая секция задаёт соответствие между диапазоном RVA в памяти и диапазоном file offset в файле. Если RVA попадает внутрь виртуального диапазона секции, то file offset вычисляется как PointerToRawData + (RVA - VirtualAddress).

```
file_offset = PointerToRawData + (RVA - VirtualAddress)
```

Но формула работает только после того, как ты нашёл правильную секцию. Нельзя применять её к первой попавшейся секции. Нужно пройти section table и проверить диапазоны.

```
Условная секция:
VirtualAddress   = 0x2000
PointerToRawData = 0x600

Нужно перевести RVA 0x2340:
смещение внутри секции = 0x2340 - 0x2000 = 0x340
file offset            = 0x600 + 0x340 = 0x940
```

Звучит просто, но есть тонкости. Первая тонкость - какой размер использовать для проверки попадания RVA в секцию: VirtualSize или SizeOfRawData? Для понимания virtual layout важен VirtualSize. Для чтения из файла важен SizeOfRawData. Если RVA попадает в область, которая существует в памяти, но не имеет сырых данных в файле, статический parser не может прочитать байты из файла - там их нет. В памяти loader заполнит эту область нулями.

Вторая тонкость - headers (заголовки) тоже имеют RVA-область в начале image. Некоторые RVA могут указывать внутрь headers, а не внутрь обычной секции. Надёжный parser должен учитывать SizeOfHeaders (размер заголовков) и не считать, что всё обязательно лежит в секциях.

Третья тонкость - повреждённые или злонамеренные PE-файлы могут иметь странные секции: пересекающиеся диапазоны, некорректные размеры, нулевые raw pointers, несогласованное выравнивание. В учебном нормальном EXE всё красиво. В reverse engineering реальных файлов нужно проверять инварианты, а не доверять заголовкам слепо.

2.3.11. Как перевести VA в RVA и обратно

VA to RVA translation (перевод виртуального адреса в RVA) нужен, когда ты работаешь с живым процессом, dump-ом или адресами из отладчика. Отладчик показывает VA, а PE-структуры внутри образа чаще используют RVA. Чтобы связать эти миры, нужно знать module base (базу модуля), то есть actual ImageBase загруженного EXE или DLL.

```
RVA = VA - ModuleBase
VA  = ModuleBase + RVA
```

Module (модуль) - загруженный EXE или DLL внутри процесса. Module base (база модуля) - фактический адрес, с которого этот модуль начинается в виртуальном адресном пространстве процесса. Для основного EXE это база главного image, для DLL - база конкретной библиотеки.

Пример. В отладчике RIP равен 0x00007FF6A1203450. Ты знаешь, что модуль app.exe загружен по базе 0x00007FF6A1200000. Тогда RVA текущей инструкции равен 0x3450. Именно этот RVA можно искать в .pdata: какая RUNTIME_FUNCTION имеет BeginAddress <= 0x3450 < EndAddress?

```
ControlPc VA = 0x00007FF6A1203450
ModuleBase    = 0x00007FF6A1200000
ControlPc RVA = 0x0000000000003450
```

Почему важно использовать фактическую базу модуля, а не preferred ImageBase из файла? Потому что ASLR мог загрузить модуль по другому адресу. Если ты возьмёшь preferred ImageBase 0x140000000, а dump показывает модуль по 0x7FF6A1200000, все RVA будут неправильными. В статическом анализе preferred ImageBase может быть удобной условной базой; в динамическом анализе нужна actual module base.

2.3.12. Почему .pdata и .xdata почти целиком держатся на RVA

Теперь связываем тему с unwind. .pdata (procedure data section, секция процедурных данных) в PE32+ x64 обычно содержит массив RUNTIME_FUNCTION (записей функций). RUNTIME_FUNCTION - это небольшая структура, которая описывает диапазон кода функции и указывает на её unwind metadata (метаданные раскрутки).

Ключевой момент: поля RUNTIME_FUNCTION не являются VA и не являются file offsets. Это RVA относительно ImageBase того image, где лежит запись. BeginAddress (адрес начала) - RVA начала диапазона функции. EndAddress (адрес конца) - RVA конца диапазона функции. UnwindInfoAddress (адрес unwind-информации) - RVA структуры UNWIND_INFO, обычно в .xdata.

```
RUNTIME_FUNCTION (упрощённо):
BeginAddress      -> RVA начала функции
EndAddress        -> RVA конца функции
UnwindInfoAddress -> RVA UNWIND_INFO
```

Почему Microsoft выбрала RVA? Потому что RUNTIME_FUNCTION должен оставаться правильным независимо от того, куда loader загрузил модуль. Если app.exe при одном запуске загружен по 0x140000000, а при другом по 0x7FF6A1200000, диапазоны функций внутри image не меняются. Меняется база. Поэтому BeginAddress = 0x3000 остаётся BeginAddress = 0x3000, а фактический VA функции считается как ModuleBase + 0x3000.

Для RtlLookupFunctionEntry это означает следующую логику. Есть ControlPc (адрес управления) - VA текущей инструкции. Система определяет ImageBase модуля, внутри которого находится ControlPc. Затем получает RVA: ControlPc - ImageBase. После этого ищет в function table запись, где BeginAddress <= ControlPcRva < EndAddress. Найденная запись указывает на UNWIND_INFO через UnwindInfoAddress, тоже RVA.

Если reverse engineer перепутает RVA с file offset, он будет читать не ту функцию и не ту xdata. Если он перепутает RVA с VA, он может решить, что значения BeginAddress “слишком маленькие” и файл повреждён. На самом деле маленькие 32-битные значения в .pdata - это нормально: они относительны к image base.

2.3.13. Semi-open range (полуоткрытый диапазон): почему EndAddress не включается

Semi-open range (полуоткрытый диапазон) - это диапазон вида [begin, end), где начало включается, а конец не включается. Для RUNTIME_FUNCTION диапазон функции обычно понимается именно так: BeginAddress <= RVA < EndAddress. Это означает, что инструкция с RVA, равным BeginAddress, принадлежит функции, а адрес, равный EndAddress, уже не принадлежит этой функции.

Зачем так делать? Полуоткрытые диапазоны удобно ставить подряд без пересечения. Если первая функция занимает [0x1000, 0x1130), а следующая начинается с 0x1130, то адрес 0x1130 однозначно принадлежит второй функции, а не обеим сразу. Это уменьшает неоднозначность при поиске по таблице функций.

Для анализатора это маленькое правило очень важно. Если ошибочно считать EndAddress включительным, можно получить ложное пересечение функций на границе. В обычном файле это даст мелкую ошибку, а в инструменте проверки unwind-инвариантов может привести к неправильному finding (обнаруженному факту/проблеме): анализатор решит, что функции перекрываются, хотя они просто стыкованы по нормальному правилу.

2.3.14. Sections и права памяти: почему .text не то же самое, что .pdata

Memory protection (защита памяти) - это набор прав доступа к страницам памяти: read (чтение), write (запись), execute (выполнение). Loader выставляет права на основе section characteristics (характеристик секций) и системных правил. Это нужно не только для порядка, но и для безопасности: данные не должны исполняться без необходимости, код не должен быть записываемым без необходимости.

.text (секция кода) обычно содержит исполняемые инструкции, поэтому она должна быть executable (исполняемой) и readable (читаемой), но обычно не writable (записываемой). .rdata (секция данных только для чтения) содержит константы и таблицы, поэтому обычно readable, но не writable и не executable. .data (секция изменяемых данных) обычно readable/writable, но не executable.

.pdata и .xdata для x64 unwind - это служебные таблицы. Они должны быть доступны для чтения Windows exception/unwind machinery, отладчикам и анализаторам. Они не должны выполняться как код. Если .pdata помечена как executable без причины, это не

обязательно автоматическая “угроза”, но повод для проверки: либо линкер/пакер странно выставил права, либо секция используется нетипично.

Для твоего будущего static /dynamic это даёт естественные проверки: совпадает ли роль секции с её правами; лежит ли Exception Directory в разумной read-only области; не указывает ли UnwindInfoAddress в исполняемую или записываемую область без объяснения; не выходит ли .pdata за границы image; не пересекаются ли диапазоны функций.

2.3.15. Что видит Ghidra, что видит hex-редактор, что видит отладчик

Ghidra (инструмент статического анализа и декомпиляции) обычно строит модель загруженного image. Она читает PE headers, секции, применяет адресную модель и показывает адреса так, как будто файл загружен по image base. Поэтому адреса в Ghidra часто выглядят как VA на основе preferred ImageBase, например 140001000. Это удобно для анализа кода, но нужно помнить: это модель, а не обязательно actual VA конкретного запуска с ASLR.

Hex editor (шестнадцатеричный редактор) видит file offsets. Если Ghidra показывает адрес 140003450, нельзя просто открыть offset 140003450 в файле. Нужно перевести: VA -> RVA через ImageBase, потом RVA -> file offset через sections. Именно поэтому новичку кажется, что “адреса не совпадают”. Они совпадают, если правильно перевести систему координат.

Debugger (отладчик) видит actual VA в живом процессе. Если ASLR включён, адреса отладчика могут отличаться от адресов Ghidra. Чтобы связать их, нужно взять module base из отладчика, вычислить RVA, а потом этот RVA найти в Ghidra. Поэтому опытные реверсеры часто мыслят не абсолютными VA, а RVA: “интересная инструкция на RVA 0x3450”.

> Практическое правило: для обмена результатами анализа по одному и тому же модулю удобнее указывать RVA, а не VA. VA зависит от конкретной загрузки. RVA стабилен относительно образа.

2.3.16. Типовые ошибки и как их распознавать

Первая ошибка - читать RVA как file offset. Симптом: parser прыгает в файл на позицию из Data Directory и получает мусор или структуру, которая “почти похожа” на правильную. Лечение: всегда переводить RVA через section table.

Вторая ошибка - читать file offset как RVA. Симптом: инструмент показывает адреса, которые не попадают ни в одну секцию virtual layout, или связывает байты файла с неправильным адресом в дизассемблере. Лечение: держать отдельные типы или хотя бы отдельные переменные для file_offset, rva и va.

Третья ошибка - использовать preferred ImageBase вместо actual module base при анализе dump-а или live process. Симптом: вычисленный RVA получается гигантским или отрицательным/непопадающим в SizeOfImage. Лечение: брать базу модуля из PEB, отладчика, списка модулей dump-а или API, а не только из Optional Header файла.

Четвёртая ошибка - доверять имени секции больше, чем Data Directory. Симптом: инструмент ищет .pdata по имени, но пропускает корректный Exception Directory в секции с другим именем. Лечение: сначала читать Data Directory, потом проверять, в какую секцию попал RVA, а имя использовать как дополнительную подсказку.

Пятая ошибка - не учитывать различие VirtualSize и SizeOfRawData. Симптом: parser пытается читать из файла область, которая существует только в памяти как нули, или игнорирует часть raw data из-за неправильного размера. Лечение: разделять вопрос “существует ли RVA в образе?” и вопрос “есть ли соответствующие байты в файле?”.

2.3.17. Мини-карта для будущих глав

В следующих главах мы постоянно будем использовать эту карту. Когда появится Data Directory (каталог данных), он будет давать RVA и size. Когда появится Exception Directory (каталог исключений), его RVA нужно будет перевести в .pdata. Когда появится RUNTIME_FUNCTION, её поля будут RVA. Когда появится UNWIND_INFO, путь к ней тоже будет через RVA. Когда появится RtlVirtualUnwind, она будет работать с runtime context (контекстом выполнения), где RIP - VA, но поиск function entry всё равно сведётся к image-relative диапазонам.

Если выразить всю главу одной формулой, получится так: file offset нужен, чтобы читать файл; RVA нужен, чтобы описывать положение внутри image независимо от загрузки; VA нужен, чтобы выполнять код в конкретном процессе. ImageBase связывает RVA и VA. Section Table связывает RVA и file offset. Exception/unwind machinery использует эту систему, чтобы от текущего RIP прийти к записи функции и unwind-метаданным.

Эта тема кажется технической бухгалтерией, но на самом деле она формирует фундамент реверса. Без неё ты смотришь на числа. С ней ты видишь карту: файл на диске, образ в памяти, текущий процесс, таблицы функций, обработку исключений и стековую раскрутку как части одной системы.

Техническая база раздела

Основная техническая опора этого раздела - Microsoft PE/COFF specification (спецификация PE/COFF), где определены PE files, COFF files, RVA, VA, sections, section headers, image files и data directories. Для связи с unwind используется Microsoft x64 exception handling documentation (документация по обработке исключений x64), где описано, что pdata содержит procedure data, xdata содержит unwind information, а записи функций используют адреса относительно image.

2.4. Что такое Data Directory и где там Exception Directory

В прошлом разделе мы разобрали три главные системы координат PE-анализа: file offset (смещение в файле), RVA (Relative Virtual Address, относительный виртуальный адрес) и VA (Virtual Address, виртуальный адрес). Теперь можно подойти к следующей важной теме: как PE-файл сообщает Windows, где искать не просто “какие-то байты”, а конкретные служебные таблицы. Для этого в PE существует Data Directory (каталог данных).

Эта тема кажется небольшой: в структуре есть массив, в массиве есть элементы, каждый элемент содержит адрес и размер. Но для reverse engineering это один из главных мостов между “форматом файла” и “механикой выполнения”. Именно через Data Directory Windows и инструменты находят imports (импорты), exports (экспорты), resources (ресурсы), relocations (релокации), TLS (thread local storage, локальное хранилище потока), debug information (отладочную информацию) и, главное для нашей книги, Exception Directory (каталог исключений).

Поэтому здесь нельзя просто сказать: “Data Directory - массив RVA и размеров”. Это будет технически похоже на правду, но почти ничего не объяснит. Нам нужно понять, зачем этот массив нужен, почему он находится именно в Optional Header (необязательном заголовке, который для executable image на практике обязателен), чем directory (каталог) отличается от section (секции), почему Exception Directory не равен секции .pdata, почему запись каталога не содержит саму таблицу, а только указывает на неё, и как всё это приводит к RUNTIME_FUNCTION и UNWIND_INFO.

> Главная идея раздела: Data Directory (каталог данных) - это не “папка” и не секция. Это навигационная таблица внутри PE-заголовков. Она говорит: “вот где в загруженном образе находится важная логическая таблица, и вот какой у неё размер”. Exception Directory (каталог исключений) - один из таких указателей. На x64 он ведёт к таблице функций для unwind, обычно расположенной в .pdata.

2.4.1. Почему PE-файлу нужна навигационная таблица

Представь, что PE-файл - это не просто куча байтов, а техническая книга для Windows loader-а (загрузчика Windows) и runtime-механизмов. В этой книге есть разные главы: код, данные, импорты, экспорты, ресурсы, релокации, таблицы исключений. Если бы не было общей навигации, каждому инструменту пришлось бы угадывать: “а где здесь таблица импортов?”, “а где ресурсы?”, “а где unwind-данные?”, “а секция точно называется .pdata?”

Такой подход был бы ненадёжным. Имена секций можно менять. Таблица может лежать не в той секции, где её ожидает новичок. Упаковщик может переименовать секции. Линкер может разместить данные в общей read-only секции. Некоторые таблицы могут отсутствовать. Поэтому PE нужен способ сказать не через имя секции, а через формальную запись: важная таблица такого-то типа находится по такому-то RVA и имеет такой-то size (размер).

Data Directory (каталог данных) решает именно эту задачу. Он находится в Optional Header (дополнительном заголовке PE, который описывает загружаемый образ) и представляет собой массив маленьких записей. Каждая запись соответствует

определённой логической таблице PE. Например, одна запись отвечает за Export Directory (каталог экспортов), другая - за Import Directory (каталог импортов), третья - за Resource Directory (каталог ресурсов), четвёртая - за Exception Directory (каталог исключений).

Слово logical (логический) здесь важно. Data Directory указывает не на “секцию по имени”, а на смысловую область данных. То есть он говорит: “там лежит таблица импортов” или “там лежит таблица исключений”. А в какой секции физически оказались эти байты - это уже отдельный вопрос, который решается через Section Table (таблицу секций) и перевод RVA в file offset.

> Data Directory (каталог данных) - это массив указателей на важные PE-таблицы. Он нужен, чтобы Windows, отладчик, загрузчик, статический анализатор и reverse engineer не гадали по именам секций, а находили нужные структуры через формальные RVA + Size.

2.4.2. Directory (каталог) - плохое слово, если понимать его как папку

Термин Directory (каталог) может сбивать с толку. В обычной файловой системе directory - это папка. В PE это не папка и не список файлов. Здесь directory означает “служебный указатель на таблицу определённого типа”.

Например, Import Directory (каталог импортов) не является папкой “imports”. Это структура данных, которая помогает loader-у понять, какие DLL нужны программе и какие функции из них нужно подключить. Resource Directory (каталог ресурсов) не является папкой картинок и иконок в буквальном смысле, хотя внутри может быть древовидная структура ресурсов. Exception Directory (каталог исключений) не является папкой обработчиков. На x64 это вход в таблицу function entries (записей функций), необходимых для exception handling (обработки исключений) и stack unwinding (раскрутки стека).

Поэтому в этой книге слово directory будем понимать так: это точка входа к логической таблице внутри PE-образа. Она говорит, где искать таблицу и сколько байт она занимает. Сама запись directory обычно очень маленькая. Она не содержит всей таблицы. Она только показывает путь к ней.

Это похоже на оглавление в книге. В оглавлении не написана вся глава. В оглавлении написано: глава начинается на такой-то странице. Data Directory делает похожую вещь, только вместо номера страницы использует RVA, а вместо количества страниц - size.

2.4.3. IMAGE_DATA_DIRECTORY (структура записи каталога данных)

Каждый элемент Data Directory обычно описывается структурой IMAGE_DATA_DIRECTORY (структура каталога данных образа). Название длинное, но его можно разобрать. IMAGE (образ) означает, что речь идёт о загружаемом PE-image (образе в памяти). DATA_DIRECTORY (каталог данных) означает, что запись указывает на какую-то служебную область данных. Эта структура состоит из двух главных полей: VirtualAddress и Size.

VirtualAddress (виртуальный адрес) в IMAGE_DATA_DIRECTORY - название немного обманчивое. В контексте PE Data Directory это обычно не полный VA процесса, а RVA. То есть поле называется VirtualAddress исторически и по формату, но его нужно читать как

“RVA начала соответствующей таблицы внутри image”. Это один из тех случаев, где слово Address (адрес) нельзя понимать автоматически как полный адрес в памяти.

Size (размер) - количество байт, которое занимает соответствующая таблица или область данных. Это поле нужно, чтобы parser (парсер, программа чтения формата) понимал границы области: где таблица начинается и где должна заканчиваться. Для Exception Directory на x64 размер особенно удобен: таблица состоит из записей RUNTIME_FUNCTION фиксированного размера, поэтому количество записей часто вычисляют как Size / sizeof(RUNTIME_FUNCTION).

```
IMAGE_DATA_DIRECTORY (упрощённо):

VirtualAddress  -> RVA начала логической таблицы
Size            -> размер этой таблицы или области в байтах
```

Если VirtualAddress равен нулю или Size равен нулю, это обычно означает, что соответствующая таблица отсутствует. Но хороший анализатор не должен механически делать выводы только по одному полю. Для надёжности он проверяет оба поля, границы image, попадание RVA в секции, возможность перевода RVA в file offset и соответствие размера ожидаемой структуре.

> Типичная ошибка: увидеть поле VirtualAddress в IMAGE_DATA_DIRECTORY и принять его за VA из отладчика. Для PE-анализа это почти всегда ошибка. В Data Directory это RVA. Чтобы получить VA в живом процессе, нужно добавить actual ImageBase. Чтобы прочитать байты из файла, нужно перевести RVA в file offset через Section Table.

2.4.4. Где Data Directory находится внутри PE

Data Directory (каталог данных) находится внутри Optional Header (дополнительного заголовка). Optional Header - ещё один термин с ловушкой. Слово optional (необязательный) пришло из COFF-истории, где некоторые объектные файлы могли не иметь этого заголовка. Но для PE image (загружаемого образа EXE/DLL) этот заголовок фактически обязателен: без него Windows loader не сможет нормально понять, как загружать файл.

Optional Header содержит много полей, которые описывают образ: ImageBase (база образа), SectionAlignment (выравнивание секций в памяти), FileAlignment (выравнивание в файле), SizeOfImage (размер образа), AddressOfEntryPoint (RVA точки входа), Subsystem (подсистема), DllCharacteristics (характеристики DLL/образа) и Data Directory. То есть Data Directory не висит сам по себе. Он находится в той части PE-заголовков, которая описывает, как файл станет образом в памяти.

Перед массивом Data Directory есть поле NumberOfRvaAndSizes (количество RVA-и-размеров). Этот термин звучит странно, но он буквально говорит, сколько элементов каталога данных считается доступными в данном Optional Header. Это защитное поле: parser не должен слепо читать больше directory entries, чем заявлено форматом конкретного файла.

NumberOfRvaAndSizes (количество RVA-и-размеров) важно для устойчивого парсинга. Старый, необычный или повреждённый файл может иметь меньше записей, чем ожидает современный инструмент. Если инструмент без проверки сразу полезет к индексу Exception Directory, он может прочитать мусор за пределами корректного

массива. Хороший PE-parser сначала проверяет, что нужный индекс вообще существует.

```
Упрощённая логика чтения:

1. Найти Optional Header.
2. Прочитать NumberOfRvaAndSizes.
3. Проверить, есть ли нужный индекс Data Directory.
4. Взять VirtualAddress и Size нужной записи.
5. Перевести VirtualAddress(RVA) в file offset через Section Table.
6. Читать саму таблицу уже по полученному file offset.
```

2.4.5. Section (секция) и Data Directory (каталог данных): в чём разница

Это один из самых важных моментов главы. Section (секция) и Data Directory (каталог данных) - разные уровни описания PE-файла.

Section (секция) - это физико-логический контейнер байтов. У секции есть имя, положение в файле, положение в памяти, размер в файле, размер в памяти и характеристики доступа. Например, секция .text обычно содержит код, .rdata - данные только для чтения, .data - изменяемые данные, .pdata - procedure data (процедурные данные), .xdata - exception/unwind data (данные исключений и раскрутки).

Data Directory (каталог данных) - это указатель на конкретную логическую таблицу. Он говорит не “вот секция”, а “вот Import Directory”, “вот Resource Directory”, “вот Exception Directory”. Таблица, на которую он указывает, физически лежит внутри какой-то секции. Чтобы понять, в какой именно, нужно взять RVA из directory entry и найти секцию, чей виртуальный диапазон покрывает этот RVA.

Почему нельзя просто сказать “Exception Directory - это .pdata”? Потому что это неточно. Exception Directory (каталог исключений) - это запись в Data Directory. Эта запись указывает на область данных, которая на x64 обычно лежит в секции .pdata. Но секция может называться иначе, может содержать дополнительные данные, может быть объединена линкером с другой read-only секцией, а сама directory entry остаётся формальной точкой входа.

> По-человечески: section (секция) - это комната в здании. Data Directory (каталог данных) - это табличка в холле: “отдел исключений находится там-то”. Обычно отдел исключений находится в комнате .pdata, но искать его правильно не по названию комнаты, а по табличке Exception Directory.

2.4.6. Почему анализатор должен доверять Data Directory больше, чем имени секции

Имя секции удобно человеку. Когда ты видишь .pdata, сразу ожидаешь увидеть там RUNTIME_FUNCTION entries (записи функций). Когда видишь .xdata, ожидаешь UNWIND_INFO (информацию раскрутки). Но имя секции - слабое доказательство. Его можно изменить. Упаковщик может назвать секцию .abc. Обфускатор может положить данные в необычную секцию. Линкер может объединить разные read-only данные. Ручная сборка может вообще не следовать привычным именам.

Data Directory сильнее, потому что это часть формального PE-контракта. Windows и инструменты не обязаны угадывать по имени .pdata. Они смотрят в PE headers, находят

Exception Directory, берут его RVA и Size, а затем работают с областью, на которую он указывает.

Конечно, Data Directory тоже может быть повреждён или намеренно подделан. Но тогда это уже предмет анализа: если directory entry указывает за пределы image, в writable/executable область без причины, на невыравненные данные, на размер не кратный RUNTIME_FUNCTION, или на область, не совпадающую с реальным содержимым, инструмент может зафиксировать anomaly (аномалию) или finding (обнаруженную проблему).

Для твоего будущего static /dynamic это принципиально. Правильная стратегия: сначала читать Exception Directory как источник истины о расположении таблицы, потом проверять, в какую секцию он попал, как называется секция, какие у неё права, не выходит ли таблица за границы, кратен ли размер структуре, отсортированы ли записи и указывают ли они на корректные UNWIND_INFO.

2.4.7. Exception Directory (каталог исключений): что это такое

Exception Directory (каталог исключений) - это элемент Data Directory, который указывает на таблицу данных, связанных с обработкой исключений и unwinding. На Windows x64 эта таблица содержит RUNTIME_FUNCTION entries (записи функций для runtime/unwind). Именно через неё Windows может по текущему адресу инструкции понять, какая функция сейчас выполняется и где лежит описание раскрутки её кадра.

Важно: Exception Directory не содержит текст обработчика, не содержит C++ catch-блоки напрямую и не является “списком всех исключений, которые может бросить программа”. Это типичная ловушка по названию. Каталог исключений в PE - это не журнал ошибок и не перечень try/catch. На x64 он прежде всего является входом в function table (таблицу функций), нужную для stack unwinding (раскрутки стека).

Зачем это нужно Windows? Когда происходит exception (исключение), у системы есть RIP или ControlPc (адрес управления), то есть адрес инструкции, где поток остановился. Чтобы найти handler (обработчик) или подняться к caller (вызывающей функции), Windows должна понять, какой функции принадлежит этот адрес. Для этого она ищет запись RUNTIME_FUNCTION: диапазон [BeginAddress, EndAddress), в который попадает текущий RVA. Найденная запись указывает на UNWIND_INFO. А UNWIND_INFO уже объясняет, как функция изменила RSP, какие nonvolatile-регистры сохранила и есть ли у неё handler data.

Получается цепочка: Exception Directory не обрабатывает исключение сам. Он даёт вход в таблицу функций. Таблица функций даёт вход в unwind metadata. Unwind metadata даёт возможность восстановить caller context. Caller context нужен для поиска обработчика выше по стеку.

```
Exception Directory -> массив RUNTIME_FUNCTION
RUNTIME_FUNCTION -> диапазон кода функции + RVA UNWIND_INFO
UNWIND_INFO -> как раскрутить кадр и есть ли handler/chain data
```

2.4.8. Где именно находится Exception Directory в Data Directory

В PE Data Directory элементы имеют фиксированные смысловые индексы. Exception Directory соответствует элементу IMAGE_DIRECTORY_ENTRY_EXCEPTION. В обычных

описаниях PE это индекс 3, если считать с нуля: Export Directory - 0, Import Directory - 1, Resource Directory - 2, Exception Directory - 3.

Зачем знать индекс? Потому что в файле Data Directory - это не словарь с названиями, а массив структур IMAGE_DATA_DIRECTORY. Названия вроде IMAGE_DIRECTORY_ENTRY_EXCEPTION - это удобные константы из заголовков Windows, которые говорят parser-у, какой элемент массива читать. Сам PE-файл не хранит рядом строку “Exception Directory”. Он хранит массив пар RVA/Size, а смысл элемента определяется его позицией.

Для человека это неудобно, но для бинарного формата нормально. Бинарный формат экономит место и рассчитывает, что parser знает спецификацию. Поэтому правильный анализатор не ищет текст “exception”, а читает Optional Header, проверяет NumberOfRvaAndSizes и берёт элемент с индексом IMAGE_DIRECTORY_ENTRY_EXCEPTION, если он доступен.

> IMAGE_DIRECTORY_ENTRY_EXCEPTION (элемент каталога исключений) - это не структура таблицы функций, а индекс в Data Directory. По этому индексу лежит IMAGE_DATA_DIRECTORY: RVA начала Exception Directory и Size её области.

2.4.9. Что именно лежит по RVA из Exception Directory на x64

На x64 по RVA из Exception Directory обычно лежит массив RUNTIME_FUNCTION (записей функций). RUNTIME_FUNCTION - маленькая структура, которая описывает одну функцию или один диапазон кода с точки зрения unwind-механизма. Она содержит три 32-битных поля: BeginAddress, EndAddress и UnwindInfoAddress.

BeginAddress (адрес начала) - это RVA начала диапазона кода функции. EndAddress (адрес конца) - это RVA конца диапазона. Обычно диапазон понимается как semi-open range (полуоткрытый диапазон): BeginAddress <= RVA < EndAddress. UnwindInfoAddress (адрес unwind-информации) - это RVA структуры UNWIND_INFO, обычно лежащей в .xdata или в области, куда указывает линкер.

Все три поля являются image-relative offsets (смещениями относительно образа), то есть RVA. Это очень важно. Они не являются file offset и не являются VA. Если BeginAddress равен 0x1230, это не значит, что функция лежит в файле по offset 0x1230. Это значит: в загруженном image функция начинается на RVA 0x1230, а её фактический VA в процессе равен ActualImageBase + 0x1230.

```
RUNTIME_FUNCTION (упрощённо):

BeginAddress      DWORD  -> RVA начала диапазона функции
EndAddress        DWORD  -> RVA конца диапазона функции
UnwindInfoAddress DWORD  -> RVA UNWIND_INFO
```

Почему поля 32-битные, если мы говорим о x64? Потому что это не 64-битные указатели, а RVA внутри PE image. PE32+ даёт 64-битное адресное пространство, но сам размер загруженного image ограничен намного меньшими величинами, поэтому 32-битного RVA достаточно для адресации объектов внутри образа. Это экономит место в таблицах и делает их независимыми от фактической базы загрузки.

2.4.10. Size Exception Directory: как из размера получить количество записей

Поле Size (размер) в IMAGE_DATA_DIRECTORY для Exception Directory задаёт размер области таблицы. На x64, если эта область является массивом RUNTIME_FUNCTION, количество записей можно вычислить как Size / sizeof(RUNTIME_FUNCTION). Обычный размер RUNTIME_FUNCTION - 12 байт: три DWORD-поля по 4 байта.

Если Size не кратен 12, это тревожный признак. Возможны разные причины: повреждённый файл, нестандартное размещение, ошибка parser-а, неправильная интерпретация architecture (архитектуры), чтение не того directory entry или намеренная обфускация. Нормальный анализатор должен не просто падать, а фиксировать это как finding: размер Exception Directory не кратен размеру RUNTIME_FUNCTION.

Но здесь есть нюанс. Size говорит размер области, на которую указывает directory entry. Он не гарантирует, что каждое содержимое внутри области валидно. Нужно отдельно проверять каждую RUNTIME_FUNCTION: выравнивание, сортировку, диапазоны, попадание BeginAddress/EndAddress в executable code, корректность UnwindInfoAddress, доступность UNWIND_INFO, флаги, цепочки и handler data.

```
Пример:
ExceptionDirectory.VirtualAddress = 0x00004000
ExceptionDirectory.Size           = 0x00000180
sizeof(RUNTIME_FUNCTION)          = 12 байт

0x180 / 12 = 32 записи RUNTIME_FUNCTION
```

Для reverse engineer-а это первая практическая точка входа: directory entry даёт не “примерно где-то есть .pdata”, а конкретный диапазон таблицы. Из него можно построить список функций, затем связать его с дизассемблированным кодом и unwind-информацией.

2.4.11. Почему Exception Directory обычно ведёт в .pdata, но не обязан называться .pdata

.pdata (procedure data section, секция процедурных данных) - привычное имя секции, где на x64 обычно лежит массив RUNTIME_FUNCTION. Слово procedure (процедура) здесь означает функцию/подпрограмму на уровне машинного кода. Procedure data - это данные о процедурах: где они начинаются, где заканчиваются и где лежит их unwind information.

Но важно разделять три вещи. Первая вещь - Exception Directory entry (запись каталога исключений) в Data Directory. Вторая вещь - область таблицы RUNTIME_FUNCTION, на которую эта запись указывает. Третья вещь - секция, внутри которой физически лежит эта область, обычно .pdata. Эти три слоя часто совпадают в голове новичка, но технически различаются.

Если секция называется .pdata, но Data Directory не указывает на неё, это подозрительно: возможно, секция есть, но не используется как официальный Exception Directory. Если Data Directory указывает в секцию с другим именем, это не обязательно ошибка: имя могло быть изменено, секции могли быть объединены, файл мог быть собран нестандартно. Поэтому правильная логика такая: Data Directory указывает область;

Section Table объясняет, в какой секции эта область лежит; имя секции помогает интерпретировать, но не заменяет проверку.

> Для static /dynamic: проверка “есть ли секция .pdata” слабее, чем проверка “есть ли Exception Directory, куда он указывает, валиден ли его размер, в какой секции он лежит, какие права у этой секции, и являются ли записи корректными RUNTIME_FUNCTION”.

2.4.12. Как parser должен идти от Data Directory к байтам файла

Parser (парсер) - программа или часть программы, которая читает бинарный формат и превращает байты в осмысленные структуры. PE-parser не должен читать “на глаз”. Он обязан идти по цепочке структур, проверяя границы на каждом шаге.

Для Exception Directory безопасный путь выглядит так. Сначала parser читает DOS Header (DOS-заголовок) и поле e_lfanew (смещение до PE-заголовка). Затем переходит к NT Headers (NT-заголовкам), проверяет PE Signature (PE-сигнатуру), читает File Header (файловый заголовок) и Optional Header. В Optional Header parser проверяет Magic (магическое значение формата, например PE32+), NumberOfRvaAndSizes, затем берёт DataDirectory[IMAGE_DIRECTORY_ENTRY_EXCEPTION].

Дальше parser проверяет VirtualAddress и Size. Если они нулевые, статической таблицы исключений может не быть. Если они ненулевые, parser должен убедиться, что RVA попадает в допустимый диапазон image, что Size не выходит за границы SizeOfImage, что область можно сопоставить с секцией, и только потом перевести RVA в file offset.

После перевода RVA в file offset parser читает массив RUNTIME_FUNCTION. Но и здесь нельзя слепо доверять. Нужно проверить, что каждая запись помещается в заявленную область, что адреса BeginAddress и EndAddress образуют корректный диапазон, что EndAddress больше BeginAddress, что таблица отсортирована, что UnwindInfoAddress указывает на доступную область, и что данные UNWIND_INFO можно разобрать без выхода за границы image.

```
Безопасная цепочка чтения Exception Directory:

DOS Header -> e_lfanew
NT Headers -> Optional Header
NumberOfRvaAndSizes -> DataDirectory[Exception]
VirtualAddress(RVA), Size
RVA -> Section -> file offset
массив RUNTIME_FUNCTION
каждый UnwindInfoAddress(RVA) -> file offset -> UNWIND_INFO
```

Такой путь кажется длинным, но именно он отличает нормальный анализатор от скрипта, который работает только на “красивых” файлах. Reverse engineering часто сталкивается с повреждёнными, упакованными, нестандартными и намеренно враждебными образами. Проверки границ - не бюрократия, а способ не дать файлу обмануть инструмент.

2.4.13. Что делает loader, а что делает exception/unwind runtime

Loader (загрузчик) и exception/unwind runtime (механизм исключений и раскрутки) используют PE-данные по-разному. Loader отвечает за превращение файла в image: он отображает секции, применяет relocations (релокации), разрешает imports (импорты),

выставляет protection (права памяти), подготавливает процесс к запуску. Он читает PE-заголовки, потому что без них не знает, как разложить образ.

Exception/unwind runtime использует уже загруженный image во время выполнения. Когда происходит исключение или нужно построить stack trace, система берёт ControlPc (адрес управления), определяет module base (базу модуля), получает RVA и ищет RUNTIME_FUNCTION в function table. Function table для статического PE-образа берётся из Exception Directory/.pdata. Если код динамический, например JIT (Just-In-Time compilation, компиляция во время выполнения), таблицы могут регистрироваться через RtlAddFunctionTable или RtlInstallFunctionTableCallback.

То есть loader подготавливает карту здания, а runtime ходит по этой карте во время событий выполнения. Если карта неверная, во время обычного запуска программа может даже начать работать, но исключения, stack walking или dump analysis будут ломаться. Поэтому unwind-метаданные важны не только при загрузке, а на протяжении всей жизни процесса.

Debugger (отладчик) и crash dump analyzer (анализатор аварийного дампа) используют ту же идею. Им нужно восстановить call stack (стек вызовов). Они тоже опираются на function entries и unwind information. Поэтому Exception Directory важен не только для “поймать исключение”, но и для понимания исполнения после факта.

2.4.14. Что значит “Exception Directory отсутствует”

Если Exception Directory отсутствует или его Size равен нулю, это не всегда означает, что файл “плохой”. Нужно смотреть на контекст. Для x86 модель исключений исторически другая, и отсутствие x64-style .pdata не является тем же сигналом. Для x64 всё зависит от типа кода, компилятора, линкера, наличия non-leaf functions, exception handling и других факторов.

Leaf function (листовая функция) - функция, которая не вызывает другие функции и не требует полноценного stack frame с сохраняемыми nonvolatile-регистрами и сложной раскруткой. Для некоторых leaf-функций отдельная unwind-запись может быть не нужна: caller context можно восстановить проще. Но большой x64-модуль без нормальной Exception Directory при наличии сложного кода, вызовов и обработчиков - повод для серьёзной проверки.

Также нужно помнить про dynamic function tables (динамические таблицы функций). Если код генерируется во время выполнения, например JIT-компилятором, его unwind-информация может не лежать в статическом PE Exception Directory. Runtime должен зарегистрировать такие таблицы через специальные API. Поэтому статический анализ PE может не увидеть всех function entries, которые появятся во время выполнения.

Вывод: отсутствие Exception Directory нужно интерпретировать осторожно. Это не автоматический verdict (вердикт), а observation (наблюдение), которое нужно сопоставить с архитектурой, типом файла, кодом, секциями, импортами, признаками JIT/packer и ожиданиями x64 ABI.

2.4.15. Особый случай: не все Data Directories используют одинаковые правила

Большинство Data Directory entries указывают на области через RVA. Но в PE есть важные исключения. Например, Certificate Table (таблица сертификатов) имеет особую модель: её адрес в Data Directory интерпретируется как file pointer (указатель в файле), а не RVA. Это связано с тем, что подпись относится к файлу на диске и не загружается как обычная часть image.

Зачем упоминать это сейчас? Чтобы не сформировать ложное правило “все Data Directory VirtualAddress всегда RVA без исключений”. Для Exception Directory на x64 нам действительно нужен RVA. Но хороший reverse engineer держит в голове: спецификация PE содержит особые случаи, и для каждой directory entry нужно знать её правила.

Для нашей темы это скорее предупреждение о стиле мышления. Мы не учим PE как набор заклинаний. Мы учим его как систему контрактов. Если конкретный каталог говорит “это RVA” - работаем как с RVA. Если спецификация говорит “для сертификатов это file pointer” - не применяем к нему автоматом правила Exception Directory. Такой подход защищает от ошибок при расширении анализатора.

2.4.16. Типичные ошибки при работе с Data Directory

Первая ошибка - считать Data Directory секцией. Симптом: человек говорит “Exception Directory лежит в заголовке”. На самом деле в заголовке лежит только запись с RVA и Size. Сама таблица лежит в секции, обычно .pdata.

Вторая ошибка - искать .pdata по имени и игнорировать Exception Directory. Симптом: анализатор не видит unwind-данные, если секция переименована. Правильный подход: сначала DataDirectory[Exception], потом поиск секции по RVA, затем чтение таблицы.

Третья ошибка - принимать VirtualAddress в IMAGE_DATA_DIRECTORY за VA. Симптом: parser пытается обратиться к адресу 0x4000 как к абсолютному адресу процесса или как к file offset. Правильно: это RVA, который нужно перевести либо в VA через ImageBase, либо в file offset через Section Table.

Четвёртая ошибка - не проверять NumberOfRvaAndSizes. Симптом: parser читает за пределами массива data directories на необычном или повреждённом файле. Правильно: сначала убедиться, что индекс Exception Directory доступен.

Пятая ошибка - верить Size без проверки содержимого. Симптом: Size кратен 12, но записи RUNTIME_FUNCTION ведут в мусор. Правильно: проверять не только размер массива, но и каждую запись, диапазоны, сортировку, пересечения, секции, права и корректность UNWIND_INFO.

Шестая ошибка - забывать о dynamic function tables. Симптом: статический анализатор делает вывод “для этого кода нет unwind-информации”, хотя код может регистрировать её во время выполнения. Правильно: разделять статическую PE-таблицу и runtime-registered tables.

2.4.17. Как это переносится на static /dynamic

Для static /dynamic Data Directory должен стать не просто частью парсинга, а первым уровнем evidence (доказательной базы). Когда инструмент говорит “у модуля есть Exception Directory”, он должен уметь показать: из какого поля Optional Header это взято, какой RVA указан, какой Size указан, в какую секцию попадает RVA, какой file offset соответствует началу таблицы, сколько RUNTIME_FUNCTION entries получилось, какие проверки прошли, какие предупреждения возникли.

Evidence (доказательство) в инструменте анализа - это не философское слово. Это конкретная цепочка наблюдений, которая позволяет пользователю понять, почему инструмент сделал вывод. Например, finding “Exception Directory size is not multiple of RUNTIME_FUNCTION size” должен ссылаться на DataDirectory[Exception].Size, ожидаемый размер записи и остаток от деления. Finding “UnwindInfoAddress points outside image” должен ссылаться на конкретную RUNTIME_FUNCTION entry и значение UnwindInfoAddress.

dynamic -часть может использовать те же знания в обратную сторону: если ты защищаешь или нормализуешь бинарь, нельзя ломать контракт PE/unwind. Любая модификация .pdata, .xdata, секций, выравниваний, размеров, прав памяти и relocations должна сохранять способность Windows найти и применить unwind-информацию. Иначе программа может запускаться, но падать при исключении, плохо анализироваться в дампе или давать неверный stack trace.

2.4.18. Итог раздела

Data Directory (каталог данных) - это навигационный массив в Optional Header PE-файла. Каждый его элемент, IMAGE_DATA_DIRECTORY, содержит VirtualAddress и Size. Для большинства каталогов VirtualAddress означает RVA, то есть положение логической таблицы внутри image относительно ImageBase. Size задаёт размер соответствующей области.

Exception Directory (каталог исключений) - один из элементов Data Directory. На Windows x64 он ведёт к таблице RUNTIME_FUNCTION, обычно лежащей в .pdata. Эта таблица нужна Windows, чтобы по текущему RIP/ControlPc найти диапазон функции, получить UnwindInfoAddress и перейти к UNWIND_INFO. UNWIND_INFO затем используется для восстановления caller context, поиска обработчиков и построения stack trace.

Главное различие: Data Directory - это указатель на логическую таблицу, section - контейнер байтов, .pdata - обычное место хранения таблицы функций, RUNTIME_FUNCTION - запись о диапазоне функции, .xdata/UNWIND_INFO - описание раскрутки кадра. Если эти уровни смешать в голове, PE/unwind превратится в кашу. Если разделить их, появляется ясная карта: заголовок указывает, секция хранит, запись описывает функцию, unwind-информация объясняет кадр, Windows использует всё это во время исключения.

2.4.19. Маленький числовой пример: как Data Directory ведёт к .pdata

Разберём простой пример, потому что без чисел Data Directory легко остаётся абстракцией. Пусть у PE32+ image (исполняемого 64-битного образа) ImageBase равен 0x0000000140000000. В Optional Header есть Data Directory. В элементе

IMAGE_DIRECTORY_ENTRY_EXCEPTION лежит VirtualAddress = 0x00005000 и Size = 0x00000078.

Сначала важно правильно назвать эти числа. VirtualAddress = 0x5000 здесь не VA и не file offset. Это RVA. Значит, Exception Directory начинается внутри загруженного image на смещении 0x5000 от ImageBase. Если образ загружен по preferred ImageBase, то VA начала таблицы будет 0x0000000140005000. Но если ASLR загрузит модуль по 0x00007FF612340000, VA будет уже 0x00007FF612345000. RVA останется 0x5000.

Теперь нужно прочитать таблицу из файла. Файл на диске не понимает VA. Поэтому parser берёт RVA 0x5000 и ищет section (секцию), чей virtual range (виртуальный диапазон) покрывает этот RVA. Допустим, секция .pdata имеет VirtualAddress = 0x5000, VirtualSize = 0x1000, PointerToRawData = 0x2400. Тогда RVA 0x5000 соответствует началу этой секции, а значит file offset будет 0x2400.

Size = 0x78. В десятичном виде это 120 байт. RUNTIME_FUNCTION имеет размер 12 байт. Значит, 120 / 12 = 10 записей. Parser читает 10 структур RUNTIME_FUNCTION начиная с file offset 0x2400. Каждая структура описывает диапазон функции и RVA её UNWIND_INFO.

```
Примерная цепочка:

Exception Directory:
  VirtualAddress = 0x5000  (RVA)
  Size           = 0x78    (120 байт)

Section .pdata:
  VirtualAddress    = 0x5000
  PointerToRawData  = 0x2400

RVA 0x5000 -> file offset 0x2400
0x78 / 12 = 10 RUNTIME_FUNCTION entries
```

Если человек перепутает RVA и file offset, он пойдёт в файл на offset 0x5000 и прочитает не ту область. Если он перепутает RVA и VA, он будет искать адрес 0x5000 в памяти процесса и тоже ошибётся. Правильная модель всегда держит три шага отдельно: directory entry даёт RVA, section table переводит RVA в file offset для чтения файла, ImageBase превращает RVA в VA для runtime-анализа.

2.4.20. Почему Exception Directory не говорит напрямую, где обработчик

По названию Exception Directory можно ожидать, что там лежат exception handlers (обработчики исключений). Это естественная, но неверная догадка. На x64 Exception Directory не является прямым списком адресов обработчиков. Он ведёт к RUNTIME_FUNCTION table (таблице записей функций), а уже каждая запись ведёт к UNWIND_INFO. Только внутри или после UNWIND_INFO, в зависимости от флагов, могут находиться данные, связанные с handler-ом или chained unwind info.

Почему так сделано? Потому что обработка исключений на x64 начинается не с вопроса “где handler?”, а с вопроса “в какой функции мы сейчас находимся и как выйти из её кадра?”. Handler имеет смысл только относительно кадра функции. Если система не умеет восстановить caller context, она не сможет двигаться по стеку и корректно выполнять search phase (фазу поиска обработчика) и unwind phase (фазу раскрутки).

Поэтому RUNTIME_FUNCTION - промежуточный, но обязательный слой. Он связывает диапазон кода функции с unwind-описанием. UNWIND_INFO объясняет устройство кадра и может указывать, что у этой функции есть exception handler (обработчик исключения), termination handler (обработчик завершения/finally) или chained info (цепочка к другой unwind-записи).

Для реверса это значит: нельзя открыть Exception Directory и сразу ожидать увидеть “адрес catch”. Нужно пройти цепочку: Exception Directory -> RUNTIME_FUNCTION -> UNWIND_INFO -> flags -> handler data. И только тогда можно говорить о том, есть ли у функции обработчик и как он представлен.

2.4.21. Data Directory как контракт, а не просто удобная таблица

Слово contract (контракт) здесь очень полезно. PE-файл как будто заключает договор с Windows: “если тебе понадобится таблица такого-то типа, смотри в такой-то entry Data Directory; там будет RVA и Size; затем через section table ты сможешь найти байты; структура будет соответствовать правилам спецификации”.

Windows, отладчик и reverse engineering tool (инструмент реверс-инжиниринга) полагаются на этот контракт. Если контракт соблюдён, всё работает предсказуемо: loader загружает image, unwinder находит функции, debugger строит stack trace, analyzer проверяет метаданные. Если контракт нарушен, начинается хаос: таблица может указывать за пределы image, размер может быть неправильным, записи могут быть не отсортированы, unwind-info может вести в мусор.

Важно: нарушение контракта не всегда означает злой умысел. Это может быть ошибка ручной сборки, баг инструмента, повреждение файла, нестандартный packer (упаковщик), JIT-сценарий, экспериментальный компилятор или специально созданный тестовый бинарь. Поэтому книга не должна учить “любой странный файл - вирус”. Она должна учить точнее: “вот инвариант, вот наблюдаемое нарушение, вот возможные объяснения, вот что нужно проверить дальше”.

> Invariant (инвариант) - это правило, которое должно сохраняться для корректной структуры. Например, для обычного x64 Exception Directory разумный инвариант: размер таблицы RUNTIME_FUNCTION должен быть кратен 12 байтам, а BeginAddress должен быть меньше EndAddress. Нарушение инварианта - не готовый приговор, а сигнал для анализа.

2.4.22. Validation (валидация): какие проверки нужны сразу на уровне каталога

Validation (валидация) - это проверка корректности данных по набору правил. Для PE/unwind валидация начинается ещё до чтения UNWIND_INFO. Уже на уровне Data Directory можно поймать много проблем.

Первая проверка - доступность entry. Нужно убедиться, что NumberOfRvaAndSizes больше индекса Exception Directory. Если индекс недоступен, parser не имеет права читать эту запись. Это защита от чтения мусора за пределами Optional Header.

Вторая проверка - пара VirtualAddress/Size. Если VirtualAddress = 0 и Size = 0, каталог отсутствует. Если одно поле нулевое, а другое нет, это повод для предупреждения: такая комбинация может быть повреждением или нестандартностью. Если оба ненулевые,

нужно проверять диапазон [RVA, RVA + Size).

Третья проверка - попадание в SizeOfImage. SizeOfImage (размер образа) описывает общий размер image в памяти. Если Exception Directory указывает за пределы SizeOfImage, такой RVA не должен считаться нормальной областью загруженного образа.

Четвёртая проверка - перевод RVA в section. Directory RVA должен попадать в виртуальный диапазон какой-то секции, если мы читаем таблицу из файла. Если не попадает, нужно понять почему: таблица отсутствует, файл нестандартный, секции повреждены, или parser ошибся в PE32/PE32+ разборе.

Пятая проверка - размер таблицы. Для x64 RUNTIME_FUNCTION entries размер обычно должен быть кратен 12. Если нет, это finding. Но даже если кратен, это только начало: дальше нужно проверять каждую запись.

Шестая проверка - права секции. Если Exception Directory лежит в секции с write (записью) или execute (исполнением), это не всегда ошибка, но требует внимания. Нормальные unwind-таблицы чаще являются read-only metadata (метаданными только для чтения), а не исполняемым кодом и не изменяемой областью.

2.4.23. Difference between absence and invalidity (отсутствие и некорректность)

Для анализатора очень важно различать absence (отсутствие) и invalidity (некорректность). Отсутствие означает, что структура не заявлена: например, Exception Directory имеет RVA = 0 и Size = 0. Некорректность означает, что структура заявлена, но нарушает правила: RVA указывает за image, size не кратен записи, таблица выходит за границы секции, записи пересекаются или ведут в неправильные UNWIND_INFO.

Почему это важно? Потому что выводы разные. Если Exception Directory отсутствует, анализатор должен сказать: “статическая таблица исключений не заявлена в PE Data Directory”. Это observation (наблюдение). Дальше нужно смотреть архитектуру, тип файла, leaf/non-leaf функции, dynamic function tables и поведение runtime.

Если Exception Directory заявлен, но некорректен, это сильнее. Тогда PE как будто обещал: “у меня есть таблица”, но при попытке её прочитать контракт ломается. Это уже более серьёзный finding. Он может влиять на исключения, stack walking, дампы и отладку.

Хороший отчёт не должен смешивать эти вещи. “Не найдено” и “найдено, но сломано” - разные диагностические состояния. В учебной книге это кажется мелочью, но в реальном static /dynamic именно такие различия делают инструмент профессиональным.

2.4.24. Почему Data Directory важен для ручного reverse engineering

Даже если ты не пишешь parser, Data Directory нужен для ручного анализа. Допустим, ты открыл файл в PE-bear, CFF Explorer, Ghidra или другой программе. Инструмент показывает тебе Data Directories. Если ты понимаешь смысл, ты не просто видишь строку “Exception Table RVA 0x5000 Size 0x780”. Ты сразу можешь мысленно построить карту: таблица функций начинается на RVA 0x5000, лежит, вероятно, в .pdata, содержит 0x780 /

12 записей, каждая запись ведёт в .xdata.

Дальше ты можешь проверить дизассемблер. Если функция в .text имеет RVA 0x1230, в .pdata должна быть запись, чей диапазон покрывает 0x1230, если функция non-leaf или требует unwind metadata. Если запись есть, ты смотришь UnwindInfoAddress. Если она ведёт в .xdata, ты разбираешь UNWIND_INFO и сравниваешь его с прологом функции.

Именно так сухая PE-таблица превращается в понимание кода. Ты больше не смотришь на .pdata как на мусорные DWORD-ы. Ты видишь: вот функция, вот её диапазон, вот её обещание Windows, вот как она должна раскручиваться, вот где потенциальная аномалия.

2.4.25. Связь Data Directory с будущей главой о .pdata

Следующая глава будет глубже разбирать .pdata и RUNTIME_FUNCTION. Но теперь уже понятно, как мы туда попадём. Мы не начнём с “открой секцию .pdata”. Мы начнём с DataDirectory[Exception]. Это принципиально правильнее.

DataDirectory[Exception] даёт официальный RVA и Size таблицы. Затем Section Table помогает найти физическое место в файле. После этого мы читаем RUNTIME_FUNCTION entries. Потом каждая запись ведёт к UNWIND_INFO. Таким образом, .pdata появляется не как случайная секция с именем, а как реализация формального каталога исключений.

Эта последовательность будет повторяться в книге много раз, потому что она соответствует реальному мышлению реверсера: сначала найти официальный источник указателя, затем перевести адрес, затем проверить границы, затем разобрать структуру, затем связать структуру с кодом, затем проверить инварианты.

```
Правильный маршрут к .pdata:

Optional Header
  -> Data Directory
    -> Exception Directory entry
      -> RVA + Size
        -> Section Table translation
          -> RUNTIME_FUNCTION array
            -> UNWIND_INFO
              -> prolog/epilog/caller context
```

Техническая база раздела

Основная опора раздела - Microsoft PE Format specification (спецификация PE-формата Microsoft), где описаны Optional Header, Data Directories, IMAGE_DATA_DIRECTORY, RVA/VA, sections и PE32+ image files. Для связи с Windows x64 exception/unwind используется Microsoft x64 exception handling documentation (документация по обработке исключений x64), где описано, что RUNTIME_FUNCTION entries являются image-relative, хранятся в .pdata PE32+ image и ссылаются на unwind data в .xdata.

Глава 3

.pdata, RUNTIME_FUNCTION, .xdata и карта функций Windows x64

Здесь начинается переход от общей PE-модели к конкретной таблице функций.
.pdata дает карту диапазонов кода, RUNTIME_FUNCTION связывает диапазон с UNWIND_INFO, .xdata хранит подробный рецепт unwind.
Главная цель главы - научиться читать exception/unwind metadata как договор между PE-файлом, Windows и реверсером.

3.1. Что такое .pdata и зачем она нужна

Теперь мы переходим от PE-базы к центральной части Windows x64 unwind. В предыдущей главе мы разобрали PE file (Portable Executable file, переносимый исполняемый файл), PE32+ (64-битный вариант PE-формата для x64), RVA (Relative Virtual Address, относительный виртуальный адрес), VA (Virtual Address, виртуальный адрес), ImageBase (база образа), sections (секции) и Data Directory (каталог данных). Это была карта файла и загруженного образа. Сейчас на этой карте появляется место, ради которого вся подготовка была нужна: секция .pdata.

Сразу важное предупреждение: .pdata нельзя понимать как “ещё одну непонятную секцию PE”. Для темы исключений это не второстепенный кусок файла. .pdata - это один из главных мостов между машинным кодом функции и возможностью Windows понять стек при исключении, отладке, построении stack trace (трассы стека) и анализе crash dump (аварийного дампа).

Если сказать очень коротко, .pdata хранит procedure data (данные процедур), то есть таблицу записей о функциях. Но такая фраза сама по себе почти бесполезна. Нужно понять, почему функции вообще надо описывать таблицей, почему машинного кода недостаточно, почему записи лежат отдельно от кода, почему они используют RVA, почему Windows ищет их по RIP/ControlPc, и почему одна неправильная запись в .pdata может испортить весь stack walking (ходьбу по стеку).

> Главная идея раздела: .pdata (procedure data section, секция процедурных данных) - это область PE32+ image, где обычно лежит массив RUNTIME_FUNCTION entries (записей таблицы функций). Каждая такая запись связывает диапазон машинного кода функции с её unwind information (информацией для раскрутки стека), обычно расположенной в .xdata. Windows использует эту связь, чтобы по текущему RIP/ControlPc найти функцию и понять, как перейти к вызывающему кадру.

3.1.1. Почему вообще понадобилась .pdata

Начнём не с структуры, а с проблемы. Представь, что поток остановился внутри функции. Причина может быть разной: access violation (нарушение доступа к памяти), breakpoint (точка останова), RaiseException (явный вызов исключения), C++ throw (выбрасывание исключения C++), запрос отладчика построить call stack (стек вызовов), анализатор дампа пытается понять цепочку вызовов после падения. Во всех этих случаях возникает один и тот же фундаментальный вопрос: “где находится вызывающая функция?”

На уровне исходного кода ответ кажется простым. Если функция C была вызвана из B, а B из A, то можно сказать: C -> B -> A. Но Windows во время выполнения не имеет перед глазами исходный код. У неё есть ContextRecord (запись контекста), то есть значения регистров, среди которых важны RIP (Instruction Pointer, указатель инструкции) и RSP (Stack Pointer, указатель вершины стека). У неё есть память процесса. У неё есть загруженные PE images (образы PE-файлов). И у неё есть метаданные, которые компилятор и линкер заранее положили в образ.

Если бы этих метаданных не было, системе пришлось бы гадать по стеку. Она могла бы смотреть на значения в памяти и пытаться найти числа, похожие на адреса возврата. Такой подход называется stack scanning (сканирование стека), но он ненадёжен. Стек

содержит не только return addresses (адреса возврата), но и локальные переменные, временные значения, сохранённые регистры, указатели, числа, shadow space (теневое место под регистровые аргументы) и старые данные. Некоторые из них случайно могут выглядеть как адреса внутри .text. Поэтому “просто читать стек вверх” нельзя использовать как строгий системный механизм.

Windows x64 решает эту проблему иначе: она требует, чтобы для функций, которым нужен нормальный unwind, существовала табличная информация. Эта табличная информация говорит: “вот диапазон адресов функции, а вот описание того, как функция изменила стек и сохранила регистры”. Именно первая часть этой информации обычно лежит в .pdata.

> Metadata (метаданные) - это данные о данных или данные о коде. В нашем случае .pdata - это не пользовательские данные программы и не машинные инструкции. Это служебное описание функций, которое нужно Windows, отладчику и анализатору, чтобы понимать код во время исключений и раскрутки стека.

Вот почему .pdata существует. Машинный код сам по себе не всегда даёт Windows быстрый и безопасный способ понять границы функции и правила выхода из её кадра. .pdata даёт таблицу, по которой можно формально сопоставить текущий адрес выполнения с записью функции.

3.1.2. Почему машинного кода недостаточно

Может возникнуть естественный вопрос: если функция уже есть в .text, зачем ещё отдельная .pdata? Разве нельзя посмотреть на инструкции и понять всё самому? Теоретически человек-реверсер иногда может это сделать. Практически для операционной системы это слишком медленно, сложно и ненадёжно.

Во-первых, границы функции в машинном коде не всегда очевидны. В исходном коде функция имеет имя и фигурные скобки. В бинарнике это просто последовательность инструкций. Дизассемблер может попытаться восстановить функции по call targets (целям вызовов), prolog patterns (шаблонам прологов), символам, relocation data (данным релокаций) и эвристикам. Но Windows runtime не должен заниматься тяжёлым дизассемблированием во время исключения. Исключение - это уже нестандартная ситуация, и механизм обработки должен быть быстрым и предсказуемым.

Во-вторых, даже если границы функции найдены, нужно понять не всю функцию, а конкретные действия, влияющие на unwind. Функция могла сделать push rbx, sub rsp, 80h, сохранить rsi по адресу [rsp+30h], использовать frame pointer (указатель кадра), иметь несколько epilog (эпилогов), содержать chained unwind info (цепочечную unwind-информацию) или language-specific handler (языково-специфический обработчик). Дизассемблировать весь код каждый раз - плохая идея. Компилятор уже знает, что он сгенерировал, поэтому он может записать короткое формальное описание.

В-третьих, unwind должен работать не только для красивого кода из учебника. Оптимизированный x64-код может не иметь классического RBP-frame. Локальные переменные могут жить в регистрах. Пролог может быть коротким, но строго ограниченным правилами. Эпилог может встречаться в нескольких местах. У функции может быть shrink-wrapping (отложенное сохранение части регистров только на

некоторых путях). Табличная модель позволяет описывать такие случаи через правила, а не через догадки.

> Prolog (пролог функции) - начальная часть функции, где она строит свой стековый кадр: выделяет место через RSP, сохраняет nonvolatile registers (невытесняемые/сохраняемые регистры), иногда устанавливает frame pointer. Unwind (раскрутка стека) должен уметь мысленно отменить действия пролога. Поэтому описание пролога хранится в unwind metadata (метаданных раскрутки).

В итоге .pdata нужна не потому, что Windows “не умеет дизассемблировать”. Она нужна потому, что ОС не должна угадывать. Она должна работать по контракту. Компилятор и линкер создают формальную таблицу, а Windows использует эту таблицу как официальный источник информации о функциях.

3.1.3. Procedure data: почему называется .pdata

Название .pdata расшифровывают как procedure data section (секция процедурных данных). Procedure (процедура) - старый и общий термин для подпрограммы или функции. В разных документациях могут говорить function (функция), procedure (процедура), routine (рутина/подпрограмма). В контексте Windows x64 unwind всё это близко: речь идёт о диапазоне машинного кода, который можно рассматривать как отдельную вызываемую единицу.

Data (данные) в названии .pdata подчёркивает, что это не исполняемый код. Там не должны лежать инструкции процессора. Там лежат записи, которые описывают процедуры. Поэтому .pdata обычно должна быть read-only metadata (метаданные только для чтения), а не execute section (исполняемая секция). Если секция с function table помечена как исполняемая или изменяемая, это не всегда автоматически “вирус”, но для анализатора это повод обратить внимание.

Section (секция) - это крупная область PE-образа с именем, виртуальным адресом, размером, смещением в файле и характеристиками. Секция .pdata обычно содержит именно таблицу RUNTIME_FUNCTION. Но важно не зацикливаться на имени секции. Официальный путь к таблице для x64 начинается через Exception Directory (каталог исключений) в Data Directory, а не через слепой поиск секции с именем .pdata.

Это принципиально: имя .pdata - полезная подсказка, но DataDirectory[Exception] - формальный указатель. Если они расходятся, анализатор должен верить не привычке, а структурам PE и затем проверять инварианты.

3.1.4. .pdata как массив записей, а не как “куча байтов”

В нормальном x64 PE32+ image секция .pdata обычно содержит массив RUNTIME_FUNCTION entries (записей RUNTIME_FUNCTION). Array (массив) означает, что записи идут подряд, одна за другой, одинакового размера. Это важно: .pdata не является произвольной структурой с вложенными списками. Базовая форма - последовательность записей фиксированного размера.

RUNTIME_FUNCTION (запись таблицы функций времени выполнения) - это структура, которая описывает один диапазон кода функции и указывает на unwind information. Название может сбивать с толку: это не “функция, которая выполняется во время

runtime” и не указатель на исполняемый код обработчика. Это запись в function table (таблице функций), которую runtime-механизмы Windows используют во время выполнения.

У RUNTIME_FUNCTION три основных поля: BeginAddress (адрес начала), EndAddress (адрес конца) и UnwindInfoAddress (адрес unwind-информации). Все эти поля на x64 являются image-relative offsets (смещениями относительно образа), то есть RVA, а не абсолютными VA и не file offset.

```
RUNTIME_FUNCTION (упрощённая схема):

BeginAddress       -> RVA начала диапазона функции
EndAddress         -> RVA конца диапазона функции
UnwindInfoAddress  -> RVA структуры UNWIND_INFO в .xdata или рядом
```

BeginAddress (адрес начала) задаёт, где начинается диапазон машинного кода функции относительно ImageBase. EndAddress (адрес конца) задаёт, где диапазон заканчивается. Обычно диапазон понимается как полуоткрытый: BeginAddress <= ControlPcRva < EndAddress. UnwindInfoAddress (адрес unwind-информации) ведёт к структуре UNWIND_INFO, которая объясняет, как раскручивать кадр этой функции.

> RVA (Relative Virtual Address, относительный виртуальный адрес) - это смещение от ImageBase (базы образа). Если ImageBase равна 0x140000000, а BeginAddress равен 0x1230, то VA начала функции в процессе будет 0x140001230. Но в файле это не обязательно offset 0x1230. Для чтения из файла RVA нужно перевести через таблицу секций.

Почему поля RUNTIME_FUNCTION 32-битные, хотя x64 использует 64-битные адреса? Потому что они не хранят полный VA. Они хранят RVA внутри одного image. Это экономит место, делает таблицу независимой от ASLR и позволяет использовать одну и ту же таблицу при загрузке образа по разным базам.

3.1.5. Почему записи .pdata используют RVA, а не VA

ASLR (Address Space Layout Randomization, рандомизация расположения адресного пространства) означает, что PE image может быть загружен не по preferred ImageBase (предпочтительной базе образа), а по другому actual ImageBase (фактической базе загрузки). Если бы .pdata хранила абсолютные VA, эти адреса приходилось бы массово исправлять при каждой загрузке, и таблица была бы более хрупкой.

RVA решает эту проблему. RUNTIME_FUNCTION говорит не “функция находится по абсолютному адресу 0x7FF6...”, а “функция начинается на смещении 0x1230 от базы моего image”. Когда Windows знает actual ImageBase модуля, она может легко получить VA: actual ImageBase + RVA.

Это та же логика, что и во многих PE-структурах. Внутренние ссылки внутри образа часто хранятся как RVA, потому что RVA остаётся стабильным независимо от того, куда loader (загрузчик) поместил image. Меняется база, но внутреннее расстояние от базы до объекта остаётся тем же.

```
Пример:

ActualImageBase = 00007FF700000000h
BeginAddress    = 0000000000001230h  (RVA)
```

```
EndAddress      = 00000000000012A0h  (RVA)

VA начала функции = 00007FF700001230h
VA конца функции  = 00007FF7000012A0h
```

Для reverse engineer-а это означает: если ты смотришь .pdata в файле, не ищи BeginAddress напрямую как file offset. Сначала пойми, что это RVA. Если нужно прочитать код функции из файла - переведи RVA через section table (таблицу секций). Если нужно сопоставить с адресом в отладчике - прибавь actual ImageBase.

3.1.6. Связь Exception Directory и .pdata

В предыдущей главе мы уже ввели Data Directory (каталог данных). Теперь нужно закрепить связь. Exception Directory (каталог исключений) - это запись в Data Directory, которая указывает на область exception-related data (данных, связанных с исключениями). Для Windows x64 в обычном PE32+ эта область является таблицей RUNTIME_FUNCTION entries, то есть тем, что мы привычно называем .pdata.

Но здесь нельзя смешивать уровни. Exception Directory entry (запись каталога исключений) находится в Optional Header (дополнительном заголовке PE). Она содержит VirtualAddress и Size. VirtualAddress здесь нужно понимать как RVA начала таблицы. Size - размер таблицы в байтах. Сама таблица физически обычно лежит в секции .pdata. То есть Exception Directory - это указатель, .pdata - место, RUNTIME_FUNCTION - записи внутри этого места.

```
Уровни:

Optional Header
  -> Data Directory
    -> Exception Directory entry (RVA + Size)
      -> область в image, обычно секция .pdata
        -> массив RUNTIME_FUNCTION
          -> каждая запись указывает на UNWIND_INFO
```

Если секция называется .pdata, но Exception Directory указывает в другое место, надо разбираться. Если Exception Directory указывает в секцию с другим именем, это не обязательно ошибка. Имя секции - не закон. Формальный маршрут начинается с Data Directory. Поэтому static /dynamic должен проверять не только “есть ли секция .pdata”, а “куда указывает Exception Directory и валидна ли таблица по этому указателю”.

3.1.7. Что Windows делает с .pdata во время исключения

Теперь посмотрим на .pdata глазами Windows runtime. Runtime (среда выполнения/механизм выполнения) - это не один файл и не одна функция. Здесь мы говорим о системной логике Windows, которая во время исключений, stack walking и unwind использует таблицы функций.

Когда возникает исключение, у системы есть ControlPc (control program counter, адрес управления). В простом понимании это адрес, связанный с местом, где выполнение покинуло текущую функцию или где оно остановилось. На x64 он используется для поиска function table entry (записи таблицы функций), соответствующей текущей функции.

Функция RtlLookupFunctionEntry (поиск записи таблицы функций) ищет в active function tables (активных таблицах функций) запись, которая покрывает заданный PC value (значение счётчика команд/адрес управления). Для статического PE-модуля эта таблица берётся из Exception Directory/.pdata. Для динамического кода могут существовать dynamic function tables (динамические таблицы функций), зарегистрированные через RtlAddFunctionTable или callback-механизмы.

Если запись найдена, Windows получает RUNTIME_FUNCTION. Из неё берётся UnwindInfoAddress. По этому адресу находится UNWIND_INFO (информация раскрутки), которая говорит, как восстановить caller context (контекст вызывающей функции). Если запись не найдена, ситуация зависит от типа функции. Для leaf function (листовой функции) возможен trivial unwind (тривиальная раскрутка), но для сложной non-leaf function отсутствие записи может сделать корректный unwind невозможным.

> Active function table (активная таблица функций) - это набор function entries, которые Windows считает действительными для поиска по ControlPc. В него входят статические таблицы из загруженных PE images и динамические таблицы, зарегистрированные runtime-ом, например JIT-компилятором.

3.1.8. Почему таблица должна быть отсортирована

RUNTIME_FUNCTION entries в .pdata обычно должны быть sorted (отсортированы) по BeginAddress. Sorting (сортировка) здесь нужна не ради красоты. Она позволяет быстро искать запись по адресу. Если таблица отсортирована, систему можно реализовать через binary search (двоичный поиск), а не через медленный просмотр всех записей подряд.

Binary search (двоичный поиск) - алгоритм поиска в отсортированном массиве. Он каждый раз смотрит на середину диапазона и решает, в какой половине продолжать. Для больших таблиц это намного быстрее, чем linear scan (линейный просмотр), где проверяется каждая запись по очереди.

Почему это важно при исключении? Потому что исключение может произойти в любой момент, и runtime должен быстро найти запись функции. Если модуль содержит десятки тысяч функций, линейный поиск по .pdata был бы плохим решением. Отсортированная таблица делает поиск предсказуемым.

Для reverse engineering сортировка становится инвариантом. Invariant (инвариант) - это правило, которое должно сохраняться для корректной структуры. Если .pdata заявлена как таблица RUNTIME_FUNCTION, но записи не отсортированы, пересекаются или имеют обратные диапазоны, это не просто “странность”. Это нарушение ожиданий runtime-механизма.

> Invariant (инвариант) - устойчивое правило корректности. Для .pdata важны такие инварианты: запись помещается в таблицу, EndAddress больше BeginAddress, диапазоны не должны бессмысленно пересекаться, UnwindInfoAddress должен указывать на доступную unwind-информацию, таблица должна быть пригодна для поиска по адресу.

3.1.9. Диапазон функции: что именно покрывает BeginAddress - EndAddress

BeginAddress и EndAddress описывают range (диапазон) кода функции. Range (диапазон) - это интервал адресов. В .pdata он задаётся в RVA. Если ControlPcRva попадает внутрь этого диапазона, Windows считает, что текущий адрес относится к этой функции и использует соответствующую unwind-информацию.

Здесь важно слово “диапазон”, а не “точка входа”. BeginAddress часто совпадает с началом функции, но для unwind важна не только первая инструкция. Если исключение произошло в середине функции, Windows всё равно должна найти ту же запись. Поэтому запись описывает область кода от начала до конца, а не только адрес входа.

Полуоткрытый диапазон [BeginAddress, EndAddress) означает: начало включается, конец не включается. Если BeginAddress = 0x1000, EndAddress = 0x1100, то адрес 0x1000 принадлежит функции, 0x10FF принадлежит функции, а 0x1100 уже не принадлежит этой записи. Такая модель удобна, потому что соседние функции могут идти подряд: первая заканчивается на 0x1100, следующая начинается на 0x1100, и между ними нет пересечения.

```
Пример диапазонов:

Function A: Begin=1000h, End=1100h  -> покрывает 1000h..10FFh
Function B: Begin=1100h, End=1180h  -> покрывает 1100h..117Fh

Адрес 1100h принадлежит B, а не A.
```

Если EndAddress меньше BeginAddress или равен ему, запись не описывает нормальный диапазон. Если диапазоны двух записей пересекаются без причины, поиск может стать неоднозначным. Если есть большие gaps (дыры) между диапазонами, это может быть нормально для данных, выравнивания или leaf-функций, но может быть и признаком неполного покрытия кода.

3.1.10. .pdata не описывает все байты .text

Важно не ожидать, что .pdata обязана покрывать абсолютно каждый байт секции .text. .text (code section, секция кода) может содержать машинные инструкции, выравнивающие байты, thunk (маленькие переходные функции/заглушки), jump tables (таблицы переходов), padding (выравнивающие пустые области), helper-код, leaf functions и нестандартные участки.

Leaf function (листовая функция) - функция, которая не вызывает другие функции и не требует отдельного unwind-описания, потому что она не меняет стек сложным образом и не сохраняет nonvolatile-регистры. Для такой функции Windows может иногда выполнить простую модель: адрес возврата лежит на вершине стека, и раскрутка похожа на имитацию ret. Но это упрощение допустимо только для функций, которые действительно соответствуют условиям leaf/trivial unwind.

Non-leaf function (нелистовая функция) - функция, которая вызывает другие функции или строит нетривиальный стековый кадр. Если она выделяет стек, сохраняет nonvolatile-регистры, использует exception handling или имеет сложный пролог, для неё нужна корректная unwind-информация. Именно такие функции особенно важны для .pdata.

> Не надо делать грубый вывод: “нет записи в .pdata - значит функции нет” или “все функции обязаны быть в .pdata”. Правильнее спрашивать: какой это код, leaf он или non-leaf, меняет ли RSP, сохраняет ли nonvolatile registers, есть ли handler, ожидается ли unwind metadata по ABI?

3.1.11. Как .pdata связана с .xdata

.pdata и .xdata работают парой. .pdata (procedure data section, секция процедурных данных) обычно содержит короткие записи RUNTIME_FUNCTION. .xdata (exception data section, секция данных исключений/раскрутки) обычно содержит более подробную unwind information. Если .pdata отвечает на вопрос “какая функция покрывает этот адрес?”, то .xdata отвечает на вопрос “как раскрутить кадр этой функции?”

Представь .pdata как указатель на карточку функции: начало, конец, адрес описания. А .xdata - как саму инструкцию по разборке кадра: сколько байт выделено на стеке, какие регистры сохранены, где находится frame register, есть ли exception handler, есть ли chained info.

UnwindInfoAddress в RUNTIME_FUNCTION указывает на UNWIND_INFO. UNWIND_INFO - это структура, которая хранит version (версию), flags (флаги), prolog size (размер пролога), count of unwind codes (количество unwind-кодов), frame register (кадровый регистр) и массив UNWIND_CODE (кодов раскрутки). Подробно мы разберём это в следующей главе, но уже сейчас важно видеть связь: .pdata без .xdata обычно не даёт полного ответа, она только приводит нас к нужной unwind-инструкции.

```
Логика связи:

ControlPc -> найти RUNTIME_FUNCTION в .pdata
RUNTIME_FUNCTION.UnwindInfoAddress -> перейти к UNWIND_INFO в .xdata
UNWIND_INFO -> понять, как восстановить caller context
```

Если RUNTIME_FUNCTION указывает на несуществующий UNWIND_INFO, таблица выглядит как карта, где улица есть, но мост разрушен. Windows может найти функцию, но не сможет корректно понять, как её раскрутить. Для static /dynamic это важный класс findings: function entry найден, но unwind target invalid (цель unwind недействительна).

3.1.12. .pdata как контракт между compiler, linker и Windows

Compiler (компилятор) превращает исходный код в машинный код. Он знает, какие функции он создаёт, какие регистры сохраняет, сколько стекового пространства выделяет, какие обработчики нужны. Linker (линкер/компоновщик) собирает объектные файлы и библиотеки в итоговый PE image. Он формирует итоговые секции, адреса, таблицы, каталоги и связи. Windows loader (загрузчик Windows) отображает image в память. Exception/unwind runtime использует таблицы во время выполнения.

.pdata - это часть договора между этими участниками. Компилятор и линкер говорят Windows: “вот таблица функций, по ней можно искать unwind information”. Windows в ответ не пытается заново анализировать исходный код. Она доверяет этому договору, но ожидает, что таблица соответствует правилам ABI.

ABI (Application Binary Interface, двоичный интерфейс приложения) - набор правил, по которым скомпилированный код взаимодействует на машинном уровне: как передаются

аргументы, какие регистры сохраняет caller или callee, как выравнивается стек, как оформляется пролог и эпилог, как описывается unwind. .pdata является частью этой ABI-картины, потому что без неё x64 stack unwinding был бы ненадёжным.

> Contract (контракт) - не юридический документ, а техническое обещание. Код обещает строить кадры по правилам x64 ABI. Метаданные обещают описывать эти кадры. Windows обещает использовать метаданные для unwind. Если код и метаданные расходятся, ломается не только теория, но и реальные исключения, stack traces и crash dump analysis.

3.1.13. Что происходит, если .pdata неправильная

Ошибки в .pdata бывают разного уровня. Самая простая - таблица заявлена, но её Size не кратен размеру RUNTIME_FUNCTION. Тогда уже непонятно, сколько целых записей содержит таблица. Это структурная ошибка каталога или таблицы.

Следующая ошибка - запись имеет неправильный диапазон: EndAddress меньше BeginAddress или равен ему. Такая запись не описывает нормальную функцию. Если Windows или анализатор попытается использовать её, результат будет некорректным.

Ещё одна ошибка - записи не отсортированы или пересекаются. Тогда поиск по ControlPc может стать неоднозначным. Если две записи утверждают, что один и тот же адрес принадлежит разным функциям с разной unwind information, кому верить? Нормальный бинарник не должен создавать такую ситуацию без специальных причин.

Также возможен invalid UnwindInfoAddress (недействительный адрес unwind-информации). Например, он указывает за пределы image, в область без данных, в исполняемый код вместо metadata, в середину другой структуры, в секцию с неожиданными правами или на UNWIND_INFO, который невозможно разобрать.

Для reverse engineering такие ошибки важны как сигналы. Они могут означать повреждение файла, ошибку генератора, нестандартный компилятор, упаковщик, антиотладочную технику, обфускацию или намеренную попытку сломать анализатор. Но хороший инструмент не должен сразу кричать “malware”. Он должен описывать конкретное нарушение: какой инвариант сломан, где, какими байтами и как это влияет на unwind.

3.1.14. Почему .pdata важна для отладчика и дампа

Debugger (отладчик) часто показывает call stack (стек вызовов): список функций, через которые поток пришёл к текущему месту. Пользователь видит красивую цепочку, но под ней находится сложная работа. Отладчик должен от текущего контекста перейти к caller context, затем к caller caller-а и так далее. На Windows x64 для этого активно используется unwind information.

Crash dump (аварийный дамп) - снимок состояния процесса после падения или в момент диагностики. Dump analyzer (анализатор дампа) не может выполнить программу заново. Он видит память, регистры, загруженные модули и метаданные. Если .pdata/.xdata корректны, он может построить стек и показать, где произошла проблема. Если метаданные сломаны, стек может оборваться, стать ложным или потерять важные кадры.

Поэтому .pdata важна даже тогда, когда исключение не было обработано. Она помогает понять, что случилось после факта. Для разработчика это разница между “где-то упало” и

“упало в функции X, вызванной из Y, с таким путём выполнения”. Для реверсера это разница между слепым анализом байтов и восстановлением реального маршрута выполнения.

3.1.15. Почему .pdata важна для reverse engineer

Reverse engineer (реверс-инженер) смотрит на бинарник без исходников или с неполной информацией. .pdata даёт ему сразу несколько преимуществ. Во-первых, она помогает находить границы функций. Если RUNTIME_FUNCTION говорит, что диапазон 0x1400-0x1490 является функцией, это сильный сигнал для дизассемблера и человека.

Во-вторых, .pdata связывает функцию с UNWIND_INFO. Это позволяет понять, какие действия пролога важны: сколько стекового места выделено, какие nonvolatile registers сохранены, есть ли frame pointer, есть ли exception handler. Даже если декомпилятор показывает псевдо-C, unwind metadata помогает проверить машинную реальность.

В-третьих, .pdata помогает искать аномалии. Если функция явно делает sub rsp, сохраняет RBX и вызывает другие функции, но в .pdata нет записи, это вопрос. Если запись есть, но UNWIND_INFO не соответствует прологу, это ещё более интересный вопрос. Если таблица выглядит искусственно сломанной, это может быть частью защиты.

В-четвёртых, .pdata полезна для восстановления stack traces в собственных инструментах. Если ты пишешь static /dynamic , ты не просто выводишь список записей. Ты проверяешь качество связи: .pdata -> RUNTIME_FUNCTION -> .xdata -> UNWIND_INFO -> реальный пролог в .text.

3.1.16. Как думать о .pdata без путаницы

Самая полезная ментальная модель такая: .pdata - это не код и не обработчики. Это индекс функций. Index (индекс) - структура, которая помогает быстро найти нужную запись по ключу. Ключ здесь - адрес выполнения, ControlPc/RIP. Значение - RUNTIME_FUNCTION entry, которая ведёт к unwind information.

Если .text - это территория кода, то .pdata - это кадастровая карта функций. Она говорит: “этот участок кода относится к такой-то функции; для выхода из неё смотри вот эту инструкцию в .xdata”. Карта не является территорией, но без карты трудно быстро и формально ориентироваться.

Тогда исчезает путаница. .text исполняется. .pdata описывает диапазоны .text. .xdata описывает раскрутку кадров этих диапазонов. Exception Directory показывает, где начинается таблица .pdata. RtlLookupFunctionEntry ищет запись по адресу. RtlVirtualUnwind использует найденную запись и UNWIND_INFO, чтобы получить предыдущий контекст.

```
Модель:

.text   -> машинный код функций
.pdata  -> индекс диапазонов функций
.xdata  -> инструкция по раскрутке кадров

RIP/ControlPc -> .pdata -> RUNTIME_FUNCTION -> .xdata -> UNWIND_INFO -> caller context
```

3.1.17. Минимальная проверка .pdata для будущего
static /dynamic

Здесь мы ещё не пишем практику, но уже можно сформулировать теоретический минимум проверки. Когда инструмент видит Exception Directory, он должен проверить, что directory entry доступен по NumberOfRvaAndSizes, что его VirtualAddress и Size образуют корректный диапазон, что RVA переводится в секцию, что размер таблицы кратен sizeof(RUNTIME_FUNCTION), что каждая запись помещается в таблицу.

Дальше инструмент проверяет сами записи: BeginAddress меньше EndAddress, диапазон находится внутри image, желательно внутри executable section (исполняемой секции), таблица отсортирована, диапазоны не пересекаются бессмысленно, UnwindInfoAddress указывает на допустимую область, UNWIND_INFO можно разобрать, а его содержимое не противоречит базовым правилам x64 unwind.

После этого начинается более сильная проверка: сопоставление metadata (метаданных) с code (кодом). Если UNWIND_INFO говорит, что функция сохранила RBX, нужно увидеть соответствующее действие в prolog. Если говорит, что выделено 0x40 байт стека, пролог должен это подтверждать. Если пролог делает больше, чем описано, или описано то, чего нет, это finding.

> Finding (диагностическая находка) - конкретное наблюдение анализатора, которое может быть ошибкой, подозрением или особенностью. Хороший finding должен отвечать: где найдено, какое правило нарушено, какие байты/поля это показывают, почему это важно и как влияет на unwind или анализ.

3.1.18. Что надо запомнить перед RUNTIME_FUNCTION

В этом разделе мы ещё не разобрали RUNTIME_FUNCTION по всем полям глубоко - это будет следующий раздел. Сейчас важно закрепить роль .pdata в общей системе. .pdata не существует сама по себе. Она появляется потому, что Windows x64 нужна табличная, быстрая и формальная модель перехода от адреса выполнения к unwind-информации функции.

Если ты видишь .pdata в PE32+ image, думай не “секция с какими-то числами”, а “официальная таблица функций для exception/unwind machinery”. Каждая запись там - это обещание: данный диапазон кода можно раскручивать по указанной UNWIND_INFO. Если обещание корректно, Windows, отладчик и анализатор могут подниматься по стеку. Если обещание нарушено, stack walking и обработка исключений могут стать ненадёжными.

Главная связка главы теперь такая: Data Directory находит Exception Directory; Exception Directory обычно указывает в .pdata; .pdata содержит RUNTIME_FUNCTION entries; RUNTIME_FUNCTION ведёт в .xdata; .xdata содержит UNWIND_INFO; UNWIND_INFO позволяет RtlVirtualUnwind восстановить caller context. Это не набор терминов, а одна цепь причин и действий.

3.1.19. .pdata - это не symbol table и не список имён функций

Очень частая путаница: человек открывает бинарник, видит таблицу, где есть диапазоны функций, и думает, что это почти symbol table (таблица символов). Нет. Symbol table (таблица символов) - это структура, которая связывает имена с адресами: например, имя

функции с её адресом, имя глобальной переменной с её расположением. В release-сборках символы часто удалены или вынесены в PDB file (Program Database file, файл базы отладочных символов).

.pdata не обязана знать имена. RUNTIME_FUNCTION не хранит строку “main”, “parse_header” или “check_password”. Она хранит только диапазон кода и ссылку на unwind-информацию. Это значит, что .pdata помогает сказать “здесь есть функция или функциональный диапазон”, но не говорит, как она называлась в исходнике.

PDB (Program Database, база данных программы) - отдельный файл с отладочной информацией Microsoft: именами функций, типами, строками исходного кода, локальными переменными, соответствием адресов и исходников. PDB нужен человеку и отладчику для удобства. .pdata нужна runtime-механизму Windows для раскрутки стека. Это разные уровни.

> Различие: symbol table/PDB отвечает на вопрос “как это называлось у разработчика?”. .pdata отвечает на вопрос “какой диапазон кода покрывает этот адрес и где инструкция для unwind?”. Поэтому .pdata полезна даже в полностью stripped binary (бинарнике без символов).

Для reverse engineer-а это хорошо. Даже если имена функций удалены, .pdata всё равно часто остаётся, потому что она нужна для корректной работы x64 exception handling и stack walking. Поэтому .pdata может дать карту функций там, где символов уже нет.

3.1.20. .pdata - это не таблица обработчиков исключений

Второе важное заблуждение: раз путь к .pdata идёт через Exception Directory, значит .pdata содержит exception handlers (обработчики исключений). В общем случае это неверно. .pdata содержит function table entries (записи таблицы функций). Эти записи могут привести к UNWIND_INFO, а уже UNWIND_INFO через flags (флаги) может сообщить, что у функции есть handler или chained information.

То есть .pdata сама по себе не говорит: “вот адрес catch-блока” или “вот код __except”. Она говорит: “вот диапазон функции; вот адрес unwind-info”. Handler (обработчик) находится на следующем уровне. Для некоторых функций UNWIND_INFO может содержать флаг UNW_FLAG_EHANDLER (флаг exception handler) или UNW_FLAG_UHANDLER (флаг unwind/termination handler), после чего за основной структурой идут дополнительные данные, которые runtime интерпретирует по правилам языка или компилятора.

Такое разделение важно. Если ты ищешь обработчики, нельзя остановиться на .pdata. Нужно пройти цепочку дальше: RUNTIME_FUNCTION -> UNWIND_INFO -> flags -> handler RVA -> handler data. Только там начинается разговор о том, какой обработчик будет вызван и как он поймёт таблицы __try/__except или C++ EH.

```
Упрощённо:

.pdata
  -> RUNTIME_FUNCTION
    -> UNWIND_INFO
      -> flags
        -> handler RVA / chained RUNTIME_FUNCTION / language-specific data
```

Почему Microsoft так разделила? Потому что не каждой функции нужен обработчик исключений, но многим функциям нужна возможность unwind. Поэтому базовая таблица функций и информация для раскрутки отделены от языковой логики обработки. Это экономит место и делает механизм универсальнее.

3.1.21. Как linker собирает .pdata из объектных файлов

Чтобы понять происхождение .pdata, полезно представить путь от исходника к EXE. Source file (исходный файл) компилируется compiler-ом (компилятором) в object file (объектный файл). Object file уже содержит машинный код, но ещё не является готовой программой. В нём могут быть свои секции, релокации, символы и служебные данные. Потом linker (линкер/компоновщик) берёт много object files и libraries (библиотек) и собирает итоговый PE image.

Если compiler генерирует x64-функции, которым нужна unwind-информация, он должен создать соответствующие описания. В MSVC/MASM-мире для x64 assembler есть директивы, позволяющие описывать frame function (функцию с кадром) и завершать пролог директивой .ENDPROLOG. Из этих описаний собираются pdata и xdata. В C/C++ обычный компилятор делает это автоматически.

Linker получает много маленьких кусочков: код одной функции, код другой функции, unwind-описание первой, unwind-описание второй. Его задача - разложить всё по итоговым секциям, назначить RVA, поправить ссылки и сформировать итоговую таблицу. Поэтому .pdata в финальном EXE - это не “написанный вручную список”, а результат компиляции и линковки.

Это объясняет, почему .pdata тесно связана с реальным layout (расположением) кода. Если linker меняет порядок функций, применяет оптимизации, объединяет секции или выкидывает неиспользуемый код, таблицы должны соответствовать уже итоговому образу, а не исходному порядку файлов.

3.1.22. Почему .pdata должна соответствовать реальному прологу

Самая важная техническая связь: .pdata ведёт к UNWIND_INFO, а UNWIND_INFO описывает действия prolog (пролога). Prolog - это не просто первые инструкции “для красоты”. Это место, где функция изменяет машинное состояние так, что потом его нужно уметь восстановить.

Например, функция может сохранить RBX. RBX на Windows x64 относится к nonvolatile registers (сохраняемым регистрам): если функция хочет его использовать, она обязана вернуть caller-у прежнее значение. Обычно она делает push rbx или mov [rsp+...], rbx. Если исключение происходит до нормального ret, Windows всё равно должна восстановить старый RBX при виртуальной раскрутке кадра. Значит, UNWIND_INFO должен знать, где лежит сохранённый RBX.

Функция может выделить стек: sub rsp, 80h. Это меняет RSP. Если unwind-механизм хочет перейти к caller context, он должен мысленно отменить это выделение: добавить обратно нужный размер, учесть сохранённые регистры и найти return address. Если UNWIND_INFO укажет неправильный размер, следующий кадр будет восстановлен неверно.

Поэтому несоответствие между прологом и UNWIND_INFO - очень серьёзная вещь. Оно означает, что код и метаданные рассказывают разные истории. Код говорит “я сделал одно”, а таблица говорит Windows “откатывай другое”. В обычном корректном бинарнике такого быть не должно.

```
Пример идеи:

Код пролога:
  push rbx
  sub  rsp, 40h

Unwind metadata должна описать:
  сохранён RBX
  выделено 40h байт стека

Если metadata этого не описывает, virtual unwind восстановит caller context неправильно.
```

3.1.23. Почему .pdata помогает находить функции, но не заменяет дизассемблер

.pdata даёт сильный сигнал о границах функций, но она не заменяет disassembler (дизассемблер). Disassembler (дизассемблер) переводит машинные байты в инструкции ассемблера. Он показывает, что реально делает код: mov, call, jmp, cmp, ret, lea, push, sub rsp и так далее. .pdata не показывает инструкции. Она показывает диапазоны и ссылки на unwind-информацию.

Если RUNTIME_FUNCTION говорит, что диапазон 0x2000-0x2140 является функцией, это помогает дизассемблеру начать анализ. Но чтобы понять смысл функции, всё равно нужно читать .text. Чтобы понять, соответствует ли unwind-info прологу, тоже нужно смотреть инструкции. Чтобы понять, где условные ветки, какие вызовы, какие данные читаются, какие структуры используются, .pdata недостаточно.

Правильная работа reverse engineer-а - соединять источники. .pdata даёт структурную карту. .text даёт реальные инструкции. .xdata даёт правила unwind. Imports (импорты) показывают внешние API. Strings (строки) дают контекст. Debug symbols (отладочные символы), если есть, дают имена. Ни один источник не является всей правдой по отдельности.

> Практическая мысль: .pdata - отличный вход в анализ функций, но не финальный ответ. Она говорит “вот граница и unwind-связь”. Дизассемблер говорит “вот что код делает”. Сильный анализ начинается, когда эти два слоя проверяются друг об друга.

3.1.24. Почему .pdata может быть полезна против обфускации, но может быть и мишенью обфускации

Obfuscation (обфускация, запутывание кода) пытается сделать анализ сложнее. Обычно люди думают об обфускации как о странных переходах, мусорных инструкциях, зашифрованных строках или виртуализации кода. Но метаданные тоже могут быть частью игры.

С одной стороны, .pdata помогает реверсеру. Даже если код сложный, таблица функций может подсказать границы. Если обфускатор не трогает .pdata, он оставляет карту, по которой можно лучше разобрать функции. Это часто помогает инструментам и человеку.

С другой стороны, защитный инструмент может попытаться сломать доверие к .pdata: удалить записи, сделать их частично неверными, переименовать секции, спрятать xdata, сгенерировать динамические function tables только во время выполнения, использовать необычные prolog/epilog patterns или мешать stack walking. Тогда анализатор, который слепо доверяет .pdata, может ошибиться.

Поэтому static /dynamic должен мыслить не “.pdata есть - всё хорошо”, а “.pdata заявляет контракт; теперь проверим, согласован ли он с PE-структурами, секциями, xdata и реальным кодом”. Это более взрослый подход.

3.1.25. Static function table и dynamic function table

Static function table (статическая таблица функций) - это таблица, которая находится в загруженном PE image и обычно приходит из Exception Directory/.pdata. Она существует уже в файле или в образе после загрузки. Для обычного EXE/DLL, собранного компилятором, это основной источник RUNTIME_FUNCTION entries.

Dynamic function table (динамическая таблица функций) - таблица, которая регистрируется во время выполнения. Она нужна, когда код появляется не из статического PE-файла, а генерируется программой. Самый понятный пример - JIT (Just-In-Time compiler, компилятор “на лету”). JIT генерирует машинный код в памяти уже во время работы процесса. У такого кода нет заранее записанной .pdata в PE-файле. Но если этот код должен нормально участвовать в x64 unwind, runtime обязан зарегистрировать function table через API вроде RtlAddFunctionTable (добавить таблицу функций) или RtlInstallFunctionTableCallback (установить callback для таблицы функций).

Это объясняет важное ограничение статического анализа. Если ты анализируешь только файл на диске, ты видишь статическую .pdata. Но во время выполнения процесс может иметь дополнительные активные таблицы функций. Поэтому отсутствие статической записи для JIT-кода не всегда ошибка. Ошибкой будет отсутствие корректной регистрации динамической таблицы, если код реально должен раскручиваться.

Для обычного курса по PE это можно было бы отложить, но для фундаментальной книги важно сказать заранее: “function table” шире, чем “секция .pdata в файле”. .pdata - основной статический случай. Active function tables в живом процессе могут включать больше источников.

3.1.26. Section characteristics: какими правами обычно должна обладать .pdata

Section characteristics (характеристики секции) - флаги в section header (заголовке секции), которые описывают свойства секции: содержит ли она код, данные, доступна ли для чтения, записи, выполнения. Для .pdata естественная роль - read-only data (данные только для чтения). Runtime читает эту таблицу, но не должен исполнять её как код и обычно не должен изменять её как обычные данные.

Если .pdata находится в секции с execute permission (правом выполнения), это странно, потому что RUNTIME_FUNCTION entries не являются машинными инструкциями. Если секция writable (доступна для записи), это тоже требует вопроса: почему unwind-таблица должна изменяться? Есть легитимные нестандартные случаи, но для обычного PE32+ это

не типичная картина.

Однако нельзя делать грубую сигнатуру “writable .pdata = зло”. Профессиональный анализатор должен формулировать мягче: “Exception Directory points to a writable section” (каталог исключений указывает на секцию с правом записи). Дальше это связывается с другими признаками: упаковка, самораспаковка, динамическая генерация кода, повреждённые размеры, несовпадение .pdata/.xdata, нестандартные permissions.

> Точная формулировка важна: инструмент не должен бездумно объявлять файл вредоносным. Он должен фиксировать технический факт и объяснять риск: unwind metadata обычно ожидается как читаемые служебные данные; writable/execute расположение может быть нестандартным и требует проверки.

3.1.27. Чем .pdata отличается от .rdata и почему иногда данные могут быть рядом

.rdata (read-only data section, секция данных только для чтения) часто содержит строковые литералы, константы, таблицы указателей, RTTI, vtables и другие неизменяемые данные. .pdata тоже обычно read-only, но её смысл другой: она содержит procedure data для function table. То есть обе секции могут быть “данными для чтения”, но их логическая роль разная.

Иногда линкер или нестандартный инструмент может разместить служебные таблицы не в ожидаемой секции или объединить секции. Поэтому reverse engineer не должен мыслить только именами. Если Exception Directory указывает в область внутри .rdata, это может быть нестандартно, но анализ должен идти по содержимому и инвариантам: там действительно массив RUNTIME_FUNCTION? Размер кратен 12? Записи отсортированы? Диапазоны разумные? UnwindInfoAddress ведёт к валидной UNWIND_INFO?

Имя секции - подсказка. Data Directory - официальный маршрут. Содержимое и инварианты - доказательство. Это хороший общий принцип PE-анализа.

3.1.28. Что .pdata может сказать, а чего не может

.pdata может сказать: здесь есть диапазон функции; диапазон начинается на таком RVA; заканчивается на таком RVA; unwind-информация находится по такому RVA. Она может помочь построить карту функций, найти покрытие кода, проверить сортировку и связи с .xdata.

.pdata не может сказать: как функция называлась в исходнике; что она делает по смыслу; какие у неё локальные переменные; какие типы у структур; какой алгоритм реализован; где “главная бизнес-логика”; какой комментарий написал разработчик. Для этого нужны другие источники: disassembly, decompilation, strings, imports, PDB, dynamic analysis, cross-references.

Это ограничение важно, чтобы не переоценивать метаданные. .pdata - фундамент для понимания стека и границ функций, но не замена всему reverse engineering. Она отвечает на вопрос механики выполнения, а не на вопрос бизнес-смысла кода.

3.1.29. Медленная связка на одном воображаемом примере

Представим, что в живом процессе модуль загружен по actual ImageBase 0x00007FF700000000. Произошло исключение, и ControlPc равен 0x00007FF700012345. Сначала система должна понять RVA этого адреса. Она вычитает базу модуля: 0x12345. Теперь нужно найти RUNTIME_FUNCTION, чей диапазон покрывает RVA 0x12345.

Exception Directory говорит, что таблица начинается на RVA 0x5000 и имеет размер 0x300 байт. Если одна RUNTIME_FUNCTION entry занимает 12 байт, значит таблица содержит 0x300 / 12 = 64 записи. Таблица отсортирована по BeginAddress. RtlLookupFunctionEntry может быстро найти запись, где BeginAddress <= 0x12345 < EndAddress.

Допустим, найдена запись: BeginAddress = 0x12000, EndAddress = 0x12400, UnwindInfoAddress = 0x6200. Это означает: текущий ControlPc находится внутри функции, покрытой этой записью; чтобы понять, как подняться к caller-у, нужно прочитать UNWIND_INFO на RVA 0x6200.

Дальше RtlVirtualUnwind берёт текущий ContextRecord. В нём есть RIP, RSP и регистры. По UNWIND_INFO она понимает, что функция выделила 0x80 байт стека и сохранила RBX. Она восстанавливает состояние так, как будто функция завершилась и управление вернулось вызывающему коду. Получается новый ContextRecord - уже для caller frame. Затем поиск повторяется для нового RIP.

```
Воображаемая цепочка:

ControlPc(VA) = 00007FF700012345h
ImageBase     = 00007FF700000000h
ControlPcRVA  = 0000000000012345h

.pdata entry:
  BeginAddress      = 00012000h
  EndAddress        = 00012400h
  UnwindInfoAddress = 00006200h

Вывод:
  ControlPc находится внутри этой функции.
  Для unwind нужно читать UNWIND_INFO на RVA 6200h.
```

Такой пример важен не как лабораторная работа, а как живая картина. Он показывает, что .pdata не “лежит где-то в PE”. Она реально участвует в движении системы от текущего адреса к предыдущему кадру.

3.1.30. Итог раздела

.pdata (procedure data section, секция процедурных данных) нужна потому, что Windows x64 не должна угадывать функции и стековые кадры по сырым байтам. Ей нужна формальная таблица, связывающая диапазоны кода с unwind-информацией. Эта таблица обычно находится через Exception Directory и состоит из RUNTIME_FUNCTION entries.

Каждая запись RUNTIME_FUNCTION говорит: вот диапазон функции, вот где лежит UNWIND_INFO. Сама .pdata не является кодом, не является таблицей имён, не является списком catch-блоков и не является полной инструкцией раскрутки. Она является индексом, который ведёт к .xdata. Поэтому .pdata нужно понимать как первый табличный слой x64 unwind-модели.

Для reverse engineer-а .pdata важна сразу по нескольким причинам: она помогает находить функции, строить stack traces, анализировать crash dumps, проверять соответствие кода и метаданных, искать аномалии, понимать работу Windows exception handling и проектировать собственный static /dynamic . Но пользоваться ей нужно аккуратно: через Data Directory, с переводом RVA, проверкой секций, размеров, сортировки, диапазонов, xdata и соответствия реальному прологу.

Техническая база раздела

Основная опора раздела - Microsoft x64 exception handling documentation (документация Microsoft по обработке исключений x64), где описано, что RUNTIME_FUNCTION entries являются image-relative, сортируются и помещаются в .pdata section PE32+ image, а xdata содержит unwind information и может указывать на handler или chained pdata. Также используется Microsoft PE Format specification (спецификация PE-формата), где описаны PE/COFF, sections, RVA/VA, Data Directory и Exception Directory.

3.2. Что такое RUNTIME_FUNCTION и как читать её поля

В предыдущем разделе мы разобрали .pdata (procedure data section, секция процедурных данных) как таблицу, которая помогает Windows x64 найти сведения о функции по текущему адресу выполнения. Теперь нужно медленно разобрать самую важную единицу этой таблицы - RUNTIME_FUNCTION. Это не вспомогательная мелочь и не “ещё одна структура из winnt.h”. Для всей x64-модели unwind это центральная карточка функции: по ней Windows понимает, какой диапазон кода сейчас выполняется и где искать инструкцию по раскрутке стекового кадра.

На этом месте особенно легко ошибиться. Название RUNTIME_FUNCTION звучит так, будто перед нами “функция времени выполнения”. Но сама структура не является функцией. В ней нет машинных инструкций, она не вызывается через call, она ничего не выполняет. Это запись о функции, точнее запись о диапазоне машинного кода, которому соответствует unwind metadata (метаданные раскрутки стека). Поэтому в этой книге мы будем постоянно проговаривать: RUNTIME_FUNCTION - это не код, а запись таблицы функций.

> Главная идея раздела: RUNTIME_FUNCTION (запись функции времени выполнения / запись диапазона функции) - это 12-байтовая запись, обычно лежащая в .pdata, которая содержит три image-relative address (адреса относительно образа): BeginAddress (начало диапазона функции), EndAddress (конец диапазона функции) и UnwindInfoAddress / UnwindData (адрес unwind-информации). Windows использует эту запись как мост: текущий RIP/ControlPc -> диапазон функции -> UNWIND_INFO -> восстановление caller context.

3.2.1. Почему мы не начинаем с полей

Техническую структуру можно записать за пять секунд. Но если начать с неё, получится конспект, а не понимание. Поля BeginAddress, EndAddress и UnwindInfoAddress кажутся простыми только до тех пор, пока не начинаешь использовать их в реальном анализаторе. Что именно начинается в BeginAddress? Почему EndAddress не является адресом последней инструкции? Почему поля 32-битные, хотя мы говорим о x64? Почему это RVA, а не VA и не file offset? Почему UnwindInfoAddress может вести не только к обычному UNWIND_INFO, но и участвовать в chained unwind info (цепочечной unwind-информации)? Почему функция может существовать в коде, но не иметь записи RUNTIME_FUNCTION?

Если на эти вопросы не ответить, структура останется набором трёх чисел. А задача этой книги - не дать тебе набор чисел, а научить видеть механизм. Поэтому сначала строим модель: какую проблему решает RUNTIME_FUNCTION, где она живёт, кто её создаёт, кто её читает, как она участвует в обработке исключений, и только потом читаем поля.

3.2.2. RUNTIME_FUNCTION: перевод и человеческий смысл

RUNTIME_FUNCTION (запись функции времени выполнения / запись диапазона функции) - это служебная запись Windows x64, описывающая один диапазон кода, который система может рассматривать как функцию или часть функции для целей unwind. Слово runtime (время выполнения) здесь означает не “внутри какой-то библиотеки runtime”, а

“используется во время выполнения программы”. Слово function (функция) здесь не означает исходную функцию C с именем и аргументами, а участок машинного кода, который имеет смысл для механизма раскрутки стека.

Проще всего воспринимать RUNTIME_FUNCTION как карточку на двери кабинета. В самой карточке не сидит человек и не происходит работа. Карточка говорит: “кабинет начинается здесь, заканчивается здесь, а инструкция по эвакуации лежит там”. Для Windows такой “кабинет” - диапазон машинного кода функции, а “инструкция по эвакуации” - UNWIND_INFO (информация для раскрутки), обычно находящаяся в .xdata.

> Не путать: function in source code (функция в исходном коде) - это конструкция языка C/C++ с именем, параметрами и телом. RUNTIME_FUNCTION (запись функции времени выполнения) - это служебная запись в таблице функций. Она может соответствовать исходной функции, но это не одно и то же понятие.

Эта разница особенно важна в reverse engineering. В исходнике функция может называться calculate_hash. В бинарнике имя может исчезнуть. В .pdata не будет строки calculate_hash. Там будет запись: BeginAddress = такой-то RVA, EndAddress = такой-то RVA, UnwindInfoAddress = такой-то RVA. То есть RUNTIME_FUNCTION помогает восстановить механическую границу функции, но не её человеческое имя и не смысл алгоритма.

3.2.3. Где физически лежит RUNTIME_FUNCTION

Обычный статический случай такой: RUNTIME_FUNCTION entries (записи RUNTIME_FUNCTION) лежат в .pdata (procedure data section, секции процедурных данных) PE32+ image (64-битного PE-образа). PE32+ здесь важно, потому что мы говорим о 64-битной Windows x64-модели. Сама .pdata находится в загруженном image (образе), а её расположение описано через Exception Directory (каталог исключений) в Data Directory (каталоге данных) PE Optional Header (дополнительного заголовка PE).

Это длинная цепочка, и её нужно держать без паники. PE-файл содержит Optional Header. В Optional Header есть Data Directory. В Data Directory есть запись Exception Directory. Она указывает RVA и Size таблицы исключений. Для x64 эта таблица обычно представляет собой массив RUNTIME_FUNCTION. То есть Windows не ищет .pdata по имени секции как человек в PE viewer. Официальный маршрут идёт через каталог данных.

```
Маршрут в статическом PE32+ image:

PE Optional Header
  -> Data Directory
      -> Exception Directory
          -> RVA таблицы
          -> Size таблицы
              -> массив RUNTIME_FUNCTION entries
                  -> каждая запись ведёт к UNWIND_INFO
```

Static function table (статическая таблица функций) - таблица функций, которая уже находится в PE-образе и обычно приходит из .pdata. Она создаётся компилятором и линкером заранее. Это основной случай для обычных EXE и DLL.

Dynamic function table (динамическая таблица функций) - таблица, которую программа регистрирует во время выполнения через API вроде RtlAddFunctionTable (добавить таблицу функций) или RtlInstallFunctionTableCallback (установить callback таблицы

функций). Она нужна, например, для JIT (Just-In-Time compiler, компилятора “на лету”), когда машинный код появляется уже в памяти, а не заранее в PE-файле. Поэтому понятие “active function tables” (активные таблицы функций) шире, чем просто .pdata в файле.

3.2.4. Кто создаёт RUNTIME_FUNCTION

Compiler (компилятор) превращает исходный код в машинный код. Он знает, какие функции он создаёт, какие регистры сохраняет, сколько байт выделяет на стеке, где у функции prolog (пролог) и epilog (эпилог), нужны ли handler data (данные обработчика). Поэтому именно компилятор имеет исходное знание, из которого можно построить unwind metadata.

Assembler (ассемблер), если код написан вручную на ассемблере, тоже должен участвовать в создании корректных unwind-описаний. В ручном x64 assembly нельзя просто написать красивый код, который двигает RSP, и забыть о metadata. Если функция должна нормально участвовать в exceptions, stack walking и debugging, для неё должны быть корректные unwind-сведения.

Linker (линкер / компоновщик) собирает объектные файлы в итоговый PE image. Он формирует секции, размещает записи, рассчитывает RVA, объединяет вкладки .pdata/.xdata из разных object files (объектных файлов) и делает так, чтобы Exception Directory указывал на итоговую таблицу. Поэтому в финальном EXE/DLL RUNTIME_FUNCTION уже не просто “локальная запись компилятора”, а часть общей таблицы образа.

Windows loader (загрузчик Windows) отображает PE image в память, но обычно не “изобретает” RUNTIME_FUNCTION для обычного нативного кода заново. Он использует то, что уже записано в образе, и делает образ доступным runtime-механизмам. Для динамического кода ситуация другая: runtime может зарегистрировать функцию через специальные API уже после загрузки.

> Запомнить: RUNTIME_FUNCTION - это результат договора между генератором кода и Windows. Код строит стековый кадр по правилам x64 ABI. Metadata описывает этот кадр. Windows использует metadata, чтобы не угадывать.

3.2.5. Кто читает RUNTIME_FUNCTION

Первый важный читатель - Windows exception/unwind runtime (механизм исключений и раскрутки Windows). Когда происходит исключение или когда нужно пройти по стеку, системе нужно по текущему адресу управления найти запись функции. Для этого используется поиск в active function tables.

Второй читатель - debugger (отладчик). Отладчик показывает call stack (стек вызовов), помогает перейти по кадрам, показывает значения регистров в вызывающих функциях. Для всего этого ему нужна возможность восстанавливать caller context (контекст вызывающего кадра). На x64 это сильно связано с RUNTIME_FUNCTION и UNWIND_INFO.

Третий читатель - dump analyzer (анализатор дампа). Crash dump (аварийный дамп) - это снимок состояния процесса. Код уже не выполняется, но анализатор может читать память, регистры и загруженные модули. Если таблицы функций корректны, дамп можно понять намного лучше. Если таблицы сломаны, call stack может оборваться или стать

ложным.

Четвёртый читатель - reverse engineer (реверс-инженер) и твой будущий static /dynamic . Для нас RUNTIME_FUNCTION - не только системная запись. Это источник фактов: где Windows считает границы функций, куда ведёт unwind, какие диапазоны покрыты, где есть аномалии, какие связи между .pdata и .xdata нарушены.

3.2.6. Как выглядит структура

Теперь можно посмотреть на форму записи. В документации Microsoft x64 exception handling RUNTIME_FUNCTION entries описываются как три ULONG-поля: function start address (адрес начала функции), function end address (адрес конца функции) и unwind info address (адрес информации раскрутки). В памяти структура должна быть DWORD-aligned (выровнена по DWORD), а адреса являются image-relative, то есть 32-битными смещениями от начала image.

```
Концептуальный вид RUNTIME_FUNCTION:

struct RUNTIME_FUNCTION {
    ULONG BeginAddress;       // RVA начала диапазона функции
    ULONG EndAddress;         // RVA конца диапазона функции
    ULONG UnwindInfoAddress;  // RVA структуры UNWIND_INFO
};

Размер обычной записи: 12 байт.
Каждое поле: 4 байта.
Адреса: image-relative, то есть RVA, а не VA и не file offset.
```

ULONG (unsigned long, беззнаковое 32-битное целое) - тип поля размером 4 байта в этом контексте. Важно: даже в x64-образе поля RUNTIME_FUNCTION имеют 32-битный размер. Это не ошибка. Они не хранят абсолютные 64-битные адреса. Они хранят RVA (Relative Virtual Address, относительный виртуальный адрес), то есть смещение от ImageBase.

DWORD alignment (выравнивание по двойному слову) означает, что структура должна начинаться по адресу, кратному 4 байтам. Alignment (выравнивание) нужно процессору и ABI для корректного и эффективного чтения структур. Если таблица сдвинута на странный байт и записи не попадают на ожидаемые границы, parser должен отнестись к этому как к проблеме структуры.

3.2.7. Почему поля 32-битные в x64

Это один из моментов, где новичок часто спотыкается. “Раз x64, значит все адреса должны быть 64-битными”. Нет. В RUNTIME_FUNCTION поля 32-битные, потому что они являются не абсолютными адресами, а RVA.

RVA (Relative Virtual Address, относительный виртуальный адрес) - это смещение относительно ImageBase (базы образа). Если модуль загружен по адресу 0x00007FF700000000, а BeginAddress равен 0x12000, то реальный VA начала диапазона будет 0x00007FF700012000. То есть 32-битного RVA достаточно, чтобы описать положение внутри образа, а абсолютный адрес получается добавлением ImageBase.

Такое решение удобно из-за ASLR (Address Space Layout Randomization, рандомизация размещения адресного пространства). Модуль может быть загружен не по preferred

ImageBase (предпочтительной базе), а по другой actual ImageBase (фактической базе). Если бы .pdata хранила абсолютные VA, их пришлось бы массово исправлять при каждом изменении базы. А RVA остаётся тем же: меняется только ImageBase, к которому он прибавляется.

```
Пример:

Actual ImageBase = 00007FF700000000h
BeginAddress RVA = 00012000h
EndAddress   RVA = 00012450h
UnwindInfo   RVA = 00006200h

Begin VA      = 00007FF700012000h
End VA        = 00007FF700012450h
UnwindInfo VA = 00007FF700006200h
```

Типичная ошибка: прочитать BeginAddress как file offset (смещение в файле) или как готовый VA. Правильно: сначала понять, что это RVA. Для живого процесса VA = ImageBase + RVA. Для чтения из файла RVA ещё нужно перевести в file offset через section table (таблицу секций).

3.2.8. BeginAddress: что именно начинается

BeginAddress (адрес начала) - это RVA начала диапазона кода, который описывает данная запись RUNTIME_FUNCTION. Обычно он соответствует началу функции или началу части функции, если используется chained unwind info (цепочечная unwind-информация).

Важно: BeginAddress не хранит имя функции. Он не говорит “это main” или “это parse_packet”. Он говорит только: диапазон, важный для unwind, начинается по такому RVA. Если у тебя есть символы PDB (Program Database, база отладочных символов), ты можешь сопоставить этот RVA с именем. Но сама RUNTIME_FUNCTION имени не содержит.

BeginAddress нужен Windows для поиска. Когда есть ControlPc (адрес текущего управления), система переводит его в image-relative form (образно-относительную форму), то есть получает ControlPcRva. Затем она ищет запись, где BeginAddress меньше или равен ControlPcRva, а EndAddress больше ControlPcRva. Если такая запись найдена, текущий адрес считается принадлежащим этому диапазону.

Для reverse engineer-а BeginAddress часто полезен как один из источников границ функций. Если дизассемблер ошибся с началом функции, .pdata может подсказать другой вариант. Но доверять слепо нельзя: таблица может быть повреждена, нестандартна или намеренно обманчива. Поэтому BeginAddress - сильная подсказка, но не абсолютная истина.

3.2.9. EndAddress: почему это не “последняя инструкция”

EndAddress (адрес конца) - это RVA конца диапазона функции. Очень важно: в нормальной интерпретации это не адрес последней инструкции, а адрес сразу после диапазона. Такой диапазон называют half-open range (полуоткрытый диапазон): начало включено, конец исключён.

Полуоткрытый диапазон записывают как [BeginAddress, EndAddress). Квадратная скобка у начала означает “включительно”, круглая у конца означает “не включительно”. Если BeginAddress = 0x1000, EndAddress = 0x1100, то адрес 0x1000 принадлежит функции, 0x10FF принадлежит функции, а 0x1100 уже не принадлежит этой записи.

```
Полуоткрытый диапазон:

BeginAddress = 1000h
EndAddress   = 1100h

Адрес 1000h -> внутри
Адрес 10FFh -> внутри
Адрес 1100h -> уже снаружи
```

Зачем так сделано? Потому что соседние функции могут идти подряд. Одна заканчивается на 0x1100, другая начинается на 0x1100. Если считать конец включительным, возникала бы путаница: адрес 0x1100 принадлежит сразу двум диапазонам или нужно отнимать один байт? Полуоткрытая модель убирает эту проблему.

Для parser-а EndAddress даёт несколько важных проверок. EndAddress должен быть больше BeginAddress. Разница между ними должна быть разумной для диапазона кода. EndAddress не должен уходить за пределы исполняемой области image без объяснения. Диапазоны записей не должны бессмысленно пересекаться. Если эти правила нарушены, это finding (обнаруженное несоответствие), а не просто “красный флаг”.

3.2.10. UnwindInfoAddress / UnwindData: куда ведёт третье поле

UnwindInfoAddress (адрес информации раскрутки) - третье поле RUNTIME_FUNCTION. Оно хранит RVA структуры UNWIND_INFO (информации для раскрутки стека), обычно лежащей в .xdata. В некоторых заголовках и контекстах поле может называться UnwindData (данные раскрутки). Для базового понимания важно одно: это не адрес кода функции, а адрес описания того, как раскрутить кадр этой функции.

UNWIND_INFO (информация раскрутки) - структура, которая описывает действия пролога функции: сколько байт стека выделено, какие nonvolatile registers (сохраняемые регистры) сохранены, используется ли frame register (кадровый регистр), есть ли exception handler (обработчик исключения), termination handler (обработчик завершения) или chained info (цепочечная информация).

То есть RUNTIME_FUNCTION отвечает на вопрос “какая запись соответствует текущему адресу?”, а UNWIND_INFO отвечает на вопрос “что делать с кадром этой функции?”. Без третьего поля RUNTIME_FUNCTION была бы только таблицей диапазонов, полезной для границ функций, но бесполезной для настоящего unwind.

```
Смысл третьего поля:

RUNTIME_FUNCTION:
  BeginAddress      -> где начинается диапазон кода
  EndAddress        -> где заканчивается диапазон кода
  UnwindInfoAddress -> где лежит инструкция по раскрутке кадра

UnwindInfoAddress не указывает на .text.
Он обычно указывает на .xdata.
```

Если UnwindInfoAddress указывает за пределы image, в середину другой структуры, в исполняемый код, в несопоставленную область или на байты, которые невозможно разобрать как UNWIND_INFO, это серьёзная проблема. Windows может не суметь корректно раскрутить стек. Отладчик может показать неправильный call stack. Дамп может потерять важные кадры. Анализатор должен фиксировать такую ситуацию как invalid unwind info target (недействительная цель unwind-информации).

3.2.11. Почему RUNTIME_FUNCTION не содержит размер функции явно

Можно спросить: почему структура хранит BeginAddress и EndAddress, а не BeginAddress и Size? Технически размер легко вычислить: Size = EndAddress - BeginAddress. Но пара начало-конец удобнее для поиска по адресу. Система получает ControlPcRva и спрашивает: находится ли он между BeginAddress и EndAddress?

Для сортированной таблицы это особенно удобно. Записи можно упорядочить по BeginAddress. Поиск сводится к нахождению ближайшего диапазона, который начинается не позже текущего адреса, а затем проверке, что текущий адрес меньше EndAddress. Это быстрее и проще, чем каждый раз мыслить размерами.

Для reverse engineering пара Begin/End также полезна: она сразу задаёт карту диапазонов. Можно строить покрытие .text: какие области кода покрыты .pdata, какие не покрыты, где есть gaps (дыры), где диапазоны пересекаются, где функции слишком маленькие или подозрительно огромные.

3.2.12. ControlPc: адрес, по которому ищут запись

ControlPc (control program counter, адрес текущего управления) - адрес в коде, для которого нужно найти запись функции. В живом контексте это близко к RIP (Instruction Pointer, указатель инструкции), но в документации API часто используется более общий термин ControlPc.

Почему не просто всегда RIP? Потому что разные процедуры unwind/search могут работать с текущим контекстом, с виртуально восстановленным контекстом, с адресом возврата, с адресом внутри эпилога. Термин ControlPc подчёркивает: это адрес управления, по которому система сейчас ищет function entry (запись таблицы функций).

Чтобы найти RUNTIME_FUNCTION, система должна понять, в каком module/image находится ControlPc. Module (модуль) - загруженный EXE или DLL в адресном пространстве процесса. Image (образ) - отображённый в память PE-файл. У каждого образа есть ImageBase. Если ControlPc попадает внутрь диапазона образа, можно получить ControlPcRva = ControlPc - ImageBase. Потом этот RVA сравнивается с BeginAddress и EndAddress записей.

```
Идея поиска:

ControlPc(VA) = адрес управления в живом процессе
ImageBase     = база модуля
ControlPcRva  = ControlPc - ImageBase

Найти запись, где:
  BeginAddress <= ControlPcRva < EndAddress
```

3.2.13. RtlLookupFunctionEntry: функция поиска записи

RtlLookupFunctionEntry (найти запись таблицы функций) - Windows API/NT runtime routine, которая ищет в active function tables запись RUNTIME_FUNCTION, соответствующую заданному ControlPc. Она принимает ControlPc, возвращает ImageBase и указатель на найденную RUNTIME_FUNCTION, либо NULL, если запись не найдена.

Здесь нужно аккуратно понять слово “lookup” (поиск по ключу). RtlLookupFunctionEntry не раскручивает стек полностью. Она не вызывает обработчик. Она не читает весь UNWIND_INFO и не строит call stack. Её задача уже и важнее: по адресу управления найти function table entry (запись таблицы функций), которая описывает текущий диапазон кода.

Если запись найдена, дальнейший механизм получает основу: вот FunctionEntry, вот ImageBase, значит можно перейти к UNWIND_INFO. Если запись не найдена, это не всегда ошибка. Для leaf function (листовой функции) Windows может использовать упрощённую модель, считая, что адрес возврата лежит на вершине стека. Но если функция не листовая и запись отсутствует, stack unwind может быть сломан.

> Важно: NULL от RtlLookupFunctionEntry означает “для этого PC не найдена запись в активных таблицах функций”, а не “по этому адресу точно нет кода”. Код может быть leaf, JIT без регистрации, нестандартный, повреждённый, упакованный или просто не покрытый статической таблицей.

3.2.14. Как RtlLookupFunctionEntry связан с RtlVirtualUnwind

RtlVirtualUnwind (виртуально раскрутить стек) - функция, которая по текущему ContextRecord (записи контекста), FunctionEntry (записи RUNTIME_FUNCTION) и ImageBase вычисляет контекст вызывающего кадра. Слово virtual (виртуально) здесь означает, что она не обязана реально выполнять инструкции функции назад. Она моделирует эффект выхода из текущей функции: восстанавливает нужные регистры, вычисляет новый RSP, получает адрес возврата и обновляет RIP для caller-а.

RtlLookupFunctionEntry и RtlVirtualUnwind работают как две стадии. Первая отвечает: “какая запись функции соответствует текущему адресу?” Вторая отвечает: “как по этой записи и unwind metadata подняться к предыдущему кадру?”

```
Две стадии:

1. RtlLookupFunctionEntry(ControlPc)
   -> FunctionEntry + ImageBase

2. RtlVirtualUnwind(..., FunctionEntry, ContextRecord, ...)
   -> обновлённый ContextRecord для caller frame
```

Эта связь важна для static /dynamic . Если твой инструмент проверяет только, что .pdata существует, он видит лишь поверхность. Настоящая проверка должна думать как Windows: можно ли по ControlPc найти правильную запись? Можно ли из этой записи перейти к корректному UNWIND_INFO? Можно ли применить unwind-коды к реальному прологу? Получится ли разумный caller context?

3.2.15. Поля RUNTIME_FUNCTION как контракт, а не как “числа”

Теперь можно сформулировать ключевую мысль: три поля RUNTIME_FUNCTION - это не просто числа. Это контракт.

BeginAddress обещает: “здесь начинается диапазон кода, к которому относится эта запись”. EndAddress обещает: “здесь этот диапазон заканчивается”. UnwindInfoAddress обещает: “по этому RVA находится unwind-описание, применимое к этому диапазону”. Windows принимает этот контракт и использует его для исключений, stack walking, debugging и dump analysis.

Если код и metadata расходятся, возникают проблемы. Например, RUNTIME_FUNCTION говорит, что функция начинается на 0x1000 и выделяет 0x80 байт стека, но реальный пролог выделяет 0x20. Тогда unwind восстановит неправильный RSP. Если запись указывает на UNWIND_INFO другой функции, будут восстановлены неправильные регистры. Если диапазон функции слишком широкий и захватывает соседний код, ControlPc может попасть в неправильную запись. Это уже не косметика, а нарушение механики выполнения.

> Для реверса: RUNTIME_FUNCTION нужно читать не как “таблицу полей”, а как утверждение бинарника о самом себе. Бинарник утверждает: такой-то код раскручивается так-то. Задача анализатора - проверить, насколько это утверждение согласовано с PE, секциями, .xdata и реальными инструкциями.

3.2.16. Почему записи должны быть отсортированы

Sorted entries (отсортированные записи) - записи RUNTIME_FUNCTION, упорядоченные по BeginAddress. Документация Microsoft указывает, что эти entries sorted and placed in .pdata section (отсортированы и помещены в секцию .pdata) PE32+ image. Сортировка нужна для быстрого поиска по адресу.

Представь таблицу из тысяч функций. При каждом исключении или stack walk линейно просматривать все записи было бы медленно. Если записи упорядочены по BeginAddress, можно использовать быстрый поиск: найти ближайшее начало функции, не превышающее ControlPcRva, а затем проверить EndAddress.

Для parser-а сортировка является сильным инвариантом. Если таблица не отсортирована, возможно, обычная логика поиска даст неправильный результат. Иногда нестандартный loader или runtime может иметь свои механизмы, но для статической .pdata нормального PE32+ несортированность - серьёзный finding.

Invariant (инвариант) - правило корректности, которое должно сохраняться. Для RUNTIME_FUNCTION важны инварианты: записи имеют правильный размер, выровнены, отсортированы, BeginAddress меньше EndAddress, диапазоны не конфликтуют, UnwindInfoAddress указывает на валидную UNWIND_INFO.

3.2.17. Почему диапазоны не должны пересекаться

Overlapping ranges (пересекающиеся диапазоны) - ситуация, когда две записи RUNTIME_FUNCTION заявляют, что один и тот же ControlPcRva принадлежит сразу двум функциям. Например, запись A покрывает 0x1000-0x1100, а запись B покрывает 0x1080-0x1180. Адрес 0x1090 попадает в обе.

Для поиска это плохо. Если один адрес принадлежит двум разным записям, какая UNWIND_INFO должна применяться? Если они различаются, результат unwind станет неоднозначным. В нормальной статической таблице такого быть не должно, кроме специальных случаев, которые нужно отдельно понимать, например chained info или нестандартные генераторы кода.

Для reverse engineering пересечение диапазонов - один из самых интересных сигналов. Это может быть ошибка, повреждение, упаковщик, обфускация, ручная модификация таблицы или попытка сломать анализаторы. Но хороший отчёт не должен писать “вирус”. Он должен писать: “RUNTIME_FUNCTION range overlap: entry N and entry M cover the same RVA range; this may make function lookup ambiguous”.

3.2.18. Gaps: почему дыры между записями не всегда ошибка

Gap (разрыв / дыра) - область кода между EndAddress одной записи и BeginAddress следующей, которую не покрывает ни одна RUNTIME_FUNCTION. Например, запись A заканчивается на 0x1100, запись B начинается на 0x1200, значит диапазон 0x1100-0x11FF не покрыт.

Новичок может подумать: “в .pdata не покрыт кусок .text, значит ошибка”. Но это не всегда так. В .text могут быть padding bytes (байты выравнивания), jump tables (таблицы переходов), thunks (маленькие переходные заглушки), leaf functions (листовые функции), холодные участки кода, данные, встроенные рядом с кодом, или код, который не требует unwind metadata.

Правильный подход: gap - это не автоматическая ошибка, а повод задать вопросы. Этот диапазон исполняемый? На него есть cross-references (перекрёстные ссылки)? Там действительно disassembly (дизассемблированный код), а не выравнивание? Функция там меняет RSP? Сохраняет nonvolatile registers? Вызывает другие функции? Если да, отсутствие RUNTIME_FUNCTION может быть проблемой. Если нет, gap может быть нормальным.

> Правило анализа: отсутствие записи само по себе не доказывает некорректность. Важно сопоставить .pdata coverage (покрытие .pdata) с реальным кодом и требованиями x64 ABI.

3.2.19. Leaf function: почему запись может отсутствовать

Leaf function (листовая функция) - функция, которая не вызывает другие функции и не требует сложной раскрутки, потому что не изменяет стек таким образом, который надо описывать таблицей, и не сохраняет nonvolatile registers. В такой функции возврат к caller-у может быть восстановлен простым способом: адрес возврата лежит на вершине стека, RSP можно увеличить на размер адреса возврата.

Но слово “leaf” нельзя понимать слишком грубо как “маленькая функция”. Функция может быть маленькой, но если она выделяет стек или сохраняет nonvolatile-регистр, она уже может требовать unwind-информацию. И наоборот, функция может выполнять какую-то простую арифметику, не трогать стек и не вызывать никого - тогда она действительно может не иметь RUNTIME_FUNCTION entry.

В x64 exception handling отсутствие записи при поиске может трактоваться как leaf-function case: если RtlLookupFunctionEntry не находит запись, unwind может имитировать возврат через [RSP]. Но это безопасно только когда код соответствует этой модели. Если запись отсутствует у функции, которая реально изменила RSP, восстановление caller-а будет неправильным.

3.2.20. Non-leaf function: почему запись обычно нужна

Non-leaf function (нелистовая функция) - функция, которая вызывает другие функции или строит нетривиальный стековый кадр. Например, она делает sub rsp, 40h, сохраняет RBX через push rbx, вызывает API, использует локальный буфер, имеет __try/__except или C++ exception handling. Для такой функции Windows должна знать, как восстановить caller context.

Если non-leaf function не имеет корректной RUNTIME_FUNCTION, Windows может не понять, как подняться выше по стеку. Это влияет не только на исключения, но и на stack trace, отладку, дампы, профилирование. Поэтому table-based exception handling требует entries для функций, которые выделяют стек или вызывают другую функцию, то есть для типичных nonleaf functions.

Для анализа это означает: если дизассемблер показывает функцию, которая явно строит кадр и вызывает другие функции, но .pdata её не покрывает, нужно проверять. Возможно, это код JIT и таблица зарегистрирована динамически. Возможно, это ручной assembly без metadata. Возможно, это обфускация. Возможно, это просто область, которую дизассемблер ошибочно счёл функцией. Но вопрос обязателен.

3.2.21. RUNTIME_FUNCTION и PDB: почему это разные миры

PDB (Program Database, база отладочных символов) - отдельный файл или набор данных с отладочной информацией: имена функций, исходные файлы, номера строк, типы, локальные переменные и многое другое. PDB очень полезен для отладки, но не является обязательной частью PE-файла для выполнения программы.

RUNTIME_FUNCTION не заменяет PDB. Она не хранит имена, типы, сигнатуры функций, аргументы, локальные переменные или строки исходника. Она хранит только минимум, нужный для unwind: диапазон кода и адрес unwind-информации.

И наоборот, PDB не заменяет RUNTIME_FUNCTION. Даже если у отладчика есть имена функций, Windows всё равно нужна формальная unwind-информация для корректной раскрутки кадров. Имена помогают человеку, но не говорят системе, как восстановить RSP и nonvolatile registers.

> Разделение ролей: PDB отвечает за человеческую отладочную семантику. RUNTIME_FUNCTION отвечает за машинную механику unwind. В reverse engineering хорошо иметь оба источника, но нельзя путать их назначение.

3.2.22. RUNTIME_FUNCTION и section names: почему имя .pdata вторично

Section name (имя секции) - текстовое имя вроде .text, .rdata, .pdata, .xdata в section header. Оно помогает человеку и инструментам ориентироваться, но официальный путь

к таблице исключений идёт через Exception Directory, а не через поиск секции с именем .pdata.

Это важно для защиты от ошибок. Некоторые бинарники могут иметь необычные имена секций. Линкер может объединять секции. Упаковщик может переименовывать секции. Таблица RUNTIME_FUNCTION может физически оказаться не в секции с именем .pdata, хотя логически выполнять роль procedure data. Поэтому parser должен идти от Data Directory: взять RVA и Size Exception Directory, перевести RVA в file offset или VA, проверить, что содержимое похоже на массив RUNTIME_FUNCTION.

Имя .pdata - хорошая подсказка, но не доказательство. Data Directory - официальный указатель. Содержимое и инварианты - техническое доказательство.

3.2.23. Как читать одну запись вручную

Представим, что Exception Directory указывает на таблицу, и мы читаем первые 12 байт. Так как каждое поле ULONG занимает 4 байта, запись делится на три DWORD-значения. На x86/x64 Windows использует little-endian (младший байт первым) представление чисел. Поэтому байты в файле нужно читать с учётом порядка.

Little-endian (младший байт первым) - порядок хранения числа, при котором младший байт лежит по меньшему адресу. Например, число 0x00012000 в файле может выглядеть как байты 00 20 01 00. Новичок часто читает такие байты “слева направо” и получает неправильное число. Parser должен собирать DWORD правильно.

```
Байты одной записи:

00 20 01 00   50 24 01 00   00 62 00 00

Читаем как little-endian DWORD:

BeginAddress      = 00012000h
EndAddress        = 00012450h
UnwindInfoAddress = 00006200h
```

Теперь эти числа нельзя сразу использовать как file offset. Это RVA. Если нужно читать UNWIND_INFO из файла на диске, нужно перевести 0x6200 из RVA в file offset через section table. Если нужно получить адрес в живом процессе, нужно прибавить ImageBase. Это одна из самых важных дисциплин PE-анализа.

3.2.24. Перевод RUNTIME_FUNCTION в живые адреса

Когда процесс запущен, модуль имеет actual ImageBase (фактическую базу образа). Допустим, она равна 0x00007FF700000000. Тогда BeginAddress, EndAddress и UnwindInfoAddress превращаются в VA простым добавлением ImageBase. Но это работает именно в живом адресном пространстве, где image уже загружен.

```
RUNTIME_FUNCTION fields:

BeginAddress      = 00012000h
EndAddress        = 00012450h
UnwindInfoAddress = 00006200h

Actual ImageBase  = 00007FF700000000h
```

```
Begin VA          = 00007FF700012000h
End VA            = 00007FF700012450h
UnwindInfo VA     = 00007FF700006200h
```

Если ты работаешь с файлом на диске, такой перевод недостаточен. Файл не лежит по ImageBase. Нужно использовать section table: найти секцию, где VirtualAddress <= RVA < VirtualAddress + VirtualSize/Size, и перевести RVA в PointerToRawData + (RVA - Section.VirtualAddress). Именно поэтому путаница VA/RVA/file offset разрушает анализ RUNTIME_FUNCTION.

3.2.25. Проверка Size Exception Directory

Size (размер) в IMAGE_DATA_DIRECTORY для Exception Directory показывает размер области таблицы в байтах. Если таблица содержит RUNTIME_FUNCTION entries, размер должен позволять разбить область на целое число записей по 12 байт. Если Size не кратен 12, это проблема: часть таблицы не образует полноценную запись.

Но и здесь нужен аккуратный язык. Size не кратен 12 - это structural finding (структурное несоответствие), а не автоматическое доказательство вредоносности. Анализатор должен указать: “Exception Directory size is not a multiple of RUNTIME_FUNCTION size”. Потом уже смотреть контекст: файл повреждён? Упакован? Таблица обрезана? PE-заголовок намеренно искажён?

```
Проверка:

entry_size = 12
count = ExceptionDirectory.Size / 12

Если Size % 12 != 0:
  таблица содержит неполную запись -> finding
```

3.2.26. Проверка BeginAddress и EndAddress

Первая проверка записи: BeginAddress должен быть меньше EndAddress. Если BeginAddress == EndAddress, диапазон пустой. Если BeginAddress > EndAddress, диапазон обратный. Ни то ни другое не описывает нормальную функцию.

Вторая проверка: диапазон должен попадать в разумную область образа. Обычно Begin/End относятся к коду, чаще всего к .text или другой executable section (исполняемой секции). Если диапазон указывает в ресурсную секцию, в данные, за пределы образа или в заголовки, это требует объяснения.

Третья проверка: диапазон не должен бессмысленно пересекаться с соседними диапазонами. Если таблица отсортирована, это легко проверить при последовательном проходе: текущий BeginAddress должен быть не меньше предыдущего BeginAddress, а обычно не меньше предыдущего EndAddress.

> Не упрощать: “Begin/End не в .text” не всегда автоматически ошибка. Иногда код может жить в другой исполняемой секции. Правильная проверка: попадает ли диапазон в допустимую исполняемую область image и согласован ли он с section characteristics.

3.2.27. Проверка UnwindInfoAddress

UnwindInfoAddress должен указывать на место, где можно прочитать UNWIND_INFO. Обычно это область .xdata (exception data section, секция данных исключений/раскрутки) или read-only data section (секция данных только для чтения), если секции объединены. Важен не только name (имя секции), но и содержимое.

Базовые проверки такие: RVA должен лежать внутри image; RVA должен переводиться в file offset при статическом анализе; адрес должен быть достаточно выровнен; по нему должны читаться поля UNWIND_INFO; Version должна быть ожидаемой; CountOfCodes должен помещаться в доступную область; если flags указывают на handler или chained info, дополнительные данные должны быть доступны.

Если UnwindInfoAddress указывает в .text, это подозрительно, потому что UNWIND_INFO не является кодом. Если указывает в writable data, это тоже вопрос. Но again (снова): не делаем грубых выводов. Фиксируем технический факт и объясняем риск.

3.2.28. Что такое chained RUNTIME_FUNCTION

Chained unwind info (цепочечная unwind-информация) - ситуация, когда UNWIND_INFO не просто описывает текущий пролог, а ссылается на другую RUNTIME_FUNCTION-like запись как на связанную основную информацию. В Microsoft-модели это связано с флагом UNW_FLAG_CHAININFO (флаг цепочечной информации).

Зачем это нужно? Иногда функция или часть функции может иметь несколько связанных описаний unwind, например из-за оптимизаций, разделения горячего и холодного кода, shrink-wrapping (оптимизация, при которой сохранение регистров/выделение стека выполняется только на некоторых путях), или особого расположения кода. Тогда одна запись может вести к chained info, а та - к основной записи.

Для нас пока важно не влезать слишком глубоко в chained format, а понять: не всякий UnwindInfoAddress заканчивается простым “прочитал UNWIND_INFO и всё”. Flags в UNWIND_INFO могут изменить интерпретацию последующих данных. Поэтому parser должен читать не только RUNTIME_FUNCTION, но и флаги UNWIND_INFO.

> Осторожность: если анализатор не поддерживает chained unwind info, он может ошибочно объявить корректный бинарник повреждённым. Поэтому findings должны различать “неподдерживаемый сложный случай” и “структурно некорректный случай”.

3.2.29. RUNTIME_FUNCTION в динамических таблицах

Хотя мы чаще говорим о .pdata, структура RUNTIME_FUNCTION используется и для dynamic function tables (динамических таблиц функций). Например, JIT runtime генерирует код в памяти. У этого кода нет статической .pdata в PE-файле. Но Windows всё равно должна уметь unwinding для такого кода, если он вызывает функции, обрабатывает исключения или участвует в stack walking.

Для этого runtime создаёт массив RUNTIME_FUNCTION entries в памяти и регистрирует его через RtlAddFunctionTable или устанавливает callback через RtlInstallFunctionTableCallback. В таком случае RtlLookupFunctionEntry ищет не только в статических таблицах модулей, но и в активных динамически зарегистрированных таблицах.

Для статического static -а это ограничение: анализ файла на диске не увидит динамические function tables, которые появятся только при выполнении. Для динамического dynamic /static -а это наоборот источник интересной информации: можно проверять, какие таблицы процесс зарегистрировал, какие диапазоны памяти они покрывают, соответствуют ли они реально сгенерированному коду.

3.2.30. RUNTIME_FUNCTION и security: почему это может быть атакуемой поверхностью

RUNTIME_FUNCTION участвует в обработке исключений и stack walking. Всё, что влияет на восстановление контекста, потенциально интересно не только отладчику, но и защитам, упаковщикам, exploit mitigation (смягчениям эксплуатации), антиотладочным техникам и malware. Некорректные или специально подобранные unwind-метаданные могут сбить анализатор, сделать stack trace ложным, осложнить crash analysis или спрятать настоящую цепочку вызовов.

При этом нужно сохранять академическую строгость. Нестандартная .pdata не равна автоматически вредоносности. Есть JIT, горячие патчи, нестандартные компиляторы, ручной assembly, оптимизации, packed binaries (упакованные бинарники), dynamic s (защитники/обфускаторы). Задача фундаментального анализа - не гадать о намерении, а фиксировать технические несоответствия и их последствия.

Хороший finding должен быть доказательным: “entry 42 has BeginAddress >= EndAddress”, “entry 57 UnwindInfoAddress points outside image”, “entries 10 and 11 overlap”, “Exception Directory size is not divisible by 12”, “RUNTIME_FUNCTION range points to non-executable section”. Такие формулировки полезны и для разработчика, и для реверсера.

3.2.31. Как связать запись с дизассемблером

Disassembler (дизассемблер) превращает машинные байты в ассемблерные инструкции. Когда ты видишь RUNTIME_FUNCTION BeginAddress, можно открыть этот RVA в дизассемблере и посмотреть, какие инструкции там находятся. Обычно в начале будет prolog: push rbx, sub rsp, mov rbp, rsp и подобные действия, хотя в оптимизированном коде всё может быть сложнее.

Связь должна идти в обе стороны. Сначала .pdata говорит: “я считаю, что функция начинается здесь”. Потом дизассемблер показывает реальные байты. Потом UNWIND_INFO говорит: “пролог сделал такие действия”. И наконец ты сравниваешь: реально ли пролог соответствует unwind-кодам? Если .xdata говорит, что сохранён RBX, а в коде нет сохранения RBX, это несогласованность. Если код делает sub rsp, 80h, а unwind говорит 20h, это серьёзная проблема.

Это будущий мост к главам про UNWIND_CODE. RUNTIME_FUNCTION сам по себе не описывает push/sub/mov. Он только приводит к UNWIND_INFO. Но уже сейчас важно понимать: полноценный анализ никогда не останавливается на .pdata. Он идёт от таблицы к коду и обратно.

3.2.32. Мини-карта чтения записи для static /dynamic

Если проектировать static /dynamic , чтение RUNTIME_FUNCTION можно представить не как “распарсить структуру”, а как цепочку проверок и смысловых выводов.

```
Алгоритм высокого уровня:

1. Найти Exception Directory через Data Directory.
2. Проверить Size и RVA каталога.
3. Перевести RVA таблицы в file offset / VA.
4. Убедиться, что Size кратен 12.
5. Пройти по RUNTIME_FUNCTION entries.
6. Для каждой записи прочитать BeginAddress, EndAddress, UnwindInfoAddress.
7. Проверить BeginAddress < EndAddress.
8. Проверить сортировку и пересечения.
9. Проверить, куда попадает диапазон кода.
10. Проверить, куда указывает UnwindInfoAddress.
11. Прочитать UNWIND_INFO и проверить его инварианты.
12. Сопоставить unwind metadata с реальным прологом.
13. Сформировать findings с доказательствами.
```

Обрати внимание: “прочитать структуру” - это только шаг 6. Настоящий инструмент начинается там, где структура превращается в проверяемое утверждение о бинарнике. Именно поэтому мы так медленно разбираем термины. Без этой медленности анализатор будет просто печатать поля, но не понимать их.

3.2.33. Типичные ошибки понимания

Первая ошибка: считать BeginAddress и EndAddress абсолютными адресами. Они image-relative, то есть RVA. В живом процессе к ним прибавляется ImageBase. В файле они переводятся через секции.

Вторая ошибка: считать EndAddress адресом последней инструкции. Правильно думать о полуоткрытом диапазоне [BeginAddress, EndAddress).

Третья ошибка: считать RUNTIME_FUNCTION самой функцией. Это запись о диапазоне, а не код.

Четвёртая ошибка: считать .pdata таблицей имён функций. Имена приходят из symbols/PDB, а не из RUNTIME_FUNCTION.

Пятая ошибка: думать, что отсутствие записи всегда ошибка. Leaf functions могут не иметь записей. Но отсутствие записи у non-leaf function требует расследования.

Шестая ошибка: доверять имени секции больше, чем Data Directory и содержимому. Официально таблицу нужно находить через Exception Directory, а не просто через строку .pdata.

Седьмая ошибка: читать UnwindInfoAddress как адрес обработчика. Это адрес UNWIND_INFO. Обработчик, если он есть, находится через данные внутри UNWIND_INFO и flags.

Восьмая ошибка: не проверять согласованность с реальным кодом. Даже валидно выглядящая запись может быть ложной, если unwind-коды не соответствуют прологу.

3.2.34. Связь с предыдущей главой и следующей темой

Теперь цепочка x64 unwind стала конкретнее. В главе про .pdata мы сказали: таблица ведёт от ControlPc к записи функции. Теперь мы знаем, что этой записью является RUNTIME_FUNCTION, и что она содержит три ключевых RVA: начало диапазона, конец диапазона и адрес unwind-информации.

Следующая тема - .xdata и UNWIND_INFO. Там мы наконец откроем то, куда указывает третье поле. Если RUNTIME_FUNCTION - это карточка диапазона, то UNWIND_INFO - это инструкция по разборке стекового кадра. Она объясняет, какие действия пролога нужно обратить, чтобы RtlVirtualUnwind смогла восстановить caller context.

То есть RUNTIME_FUNCTION не завершает историю. Она только доводит нас до двери. За дверью находится .xdata.

3.2.35. Итог раздела

RUNTIME_FUNCTION (запись функции времени выполнения / запись диапазона функции) - центральная единица .pdata. Она связывает диапазон машинного кода с unwind-информацией. В обычном статическом PE32+ image эта запись состоит из трёх 32-битных image-relative полей: BeginAddress, EndAddress и UnwindInfoAddress. BeginAddress и EndAddress задают полуоткрытый диапазон кода, а UnwindInfoAddress ведёт к UNWIND_INFO в .xdata или к связанным данным раскрутки.

Эта структура нужна не для красоты и не для справочника. Она участвует в реальной работе Windows: RtlLookupFunctionEntry ищет такую запись по ControlPc; RtlVirtualUnwind использует найденную запись и UNWIND_INFO, чтобы восстановить caller context; exception handling, debugging, stack traces и crash dump analysis зависят от корректности этой цепочки.

Для reverse engineering RUNTIME_FUNCTION ценна как карта и как объект проверки. Она помогает увидеть, как бинарник заявляет свои функции, но требует валидации: размер таблицы, сортировка, диапазоны, RVA, секции, xdata, chained info, соответствие прологу и разумность покрытия кода. Взрослый анализ начинается не с печати полей, а с вопроса: “согласовано ли утверждение RUNTIME_FUNCTION с реальным образом и реальным кодом?”

3.2.36. FunctionEntry: почему указатель на запись тоже имеет значение

FunctionEntry (запись функции / указатель на RUNTIME_FUNCTION) - имя параметра, которое ты часто увидишь рядом с RtlVirtualUnwind. Это не новая структура, а указатель на уже найденную RUNTIME_FUNCTION. Когда RtlLookupFunctionEntry нашла подходящую запись, дальнейшему коду нужно передать её дальше. Поэтому появляется FunctionEntry: “вот та самая запись, по которой надо раскручивать текущий кадр”.

Важно понимать жизненный контекст этого указателя. Если запись пришла из static function table (статической таблицы функций), она указывает в память загруженного PE image или в системное представление этой таблицы. Если запись пришла из dynamic function table (динамической таблицы функций), она может указывать на память, зарегистрированную runtime-ом. То есть FunctionEntry - это не “адрес функции”, а адрес записи о функции. Новичок может перепутать и попытаться дизассемблировать

FunctionEntry как код. Это неверно: дизассемблировать нужно BeginAddress + ImageBase, а FunctionEntry нужно читать как структуру из трёх полей.

Для static /dynamic это различие полезно. В статическом анализе ты читаешь RUNTIME_FUNCTION из файла или образа. В динамическом анализе ты можешь видеть FunctionEntry, возвращённый системной функцией. В обоих случаях смысл один: запись утверждает, как диапазон кода связан с unwind metadata. Но источник этой записи может быть разный.

3.2.37. HistoryTable: зачем в поиске нужна история

UNWIND_HISTORY_TABLE (таблица истории unwind-поиска) - необязательная структура-кэш, которую можно передавать в RtlLookupFunctionEntry. Cache (кэш) - это память для ускорения повторных запросов. Если система много раз ищет function entries рядом или в тех же модулях, история может помочь не выполнять полный поиск каждый раз.

Для фундаментального понимания важно не само устройство HistoryTable, а её роль. Она не меняет смысл RUNTIME_FUNCTION. Она не хранит новые правила раскрутки. Она только помогает быстрее находить записи. То есть логика остаётся прежней: ControlPc -> active function table -> RUNTIME_FUNCTION. HistoryTable лишь может ускорить этот путь.

Типичная ошибка - думать, что если в API есть HistoryTable, значит часть unwind-данных хранится там. Нет. Истина всё равно в function tables и UNWIND_INFO. HistoryTable - вспомогательный механизм поиска, не источник семантики функции.

3.2.38. Почему UnwindInfoAddress не обязан указывать строго в секцию с именем .xdata

В учебной модели мы говорим: .pdata ведёт к .xdata. Это правильно как базовая картина. Но профессиональный анализ должен быть чуть аккуратнее. PE section name (имя секции PE) - это не абсолютный закон. Линкер может объединять read-only sections (секции только для чтения), нестандартный toolchain (цепочка сборочных инструментов) может назвать секции иначе, упаковщик может поменять имена, а данные могут быть размещены в области, которая логически выполняет роль .xdata, но называется иначе.

Поэтому корректная проверка звучит не “UnwindInfoAddress обязан попадать в секцию с именем .xdata”, а “UnwindInfoAddress должен указывать на доступную область image, где можно корректно прочитать UNWIND_INFO, и эта область должна иметь разумные характеристики для read-only metadata (метаданных только для чтения)”.

Это важная зрелость анализа. Если инструмент слишком жёстко требует имя .xdata, он будет ошибаться на нестандартных, но легитимных бинарниках. Если инструмент вообще не проверяет цель UnwindInfoAddress, он пропустит повреждения и обман. Правильный путь между этими крайностями: имя секции - подсказка, инварианты структуры - доказательство.

3.2.39. Почему BeginAddress не всегда совпадает с тем, что декомпилятор назвал функцией

Decompiler (декомпилятор) пытается восстановить высокоуровневый псевдокод из машинных инструкций. Он строит свои функции по эвристикам, графу переходов, символам, call targets и анализу потока управления. RUNTIME_FUNCTION строится другим путём: это unwind-запись, созданная генератором кода и линкером. Поэтому границы, которые показывает декомпилятор, и диапазоны .pdata могут не всегда идеально совпадать.

Например, декомпилятор может объединить несколько маленьких участков в одну функцию или, наоборот, разрезать сложную функцию. Может быть hot/cold split (разделение горячего и холодного кода), когда редко выполняемые участки вынесены отдельно. Может быть thunk (переходная заглушка), которую декомпилятор считает функцией, но для unwind она тривиальна. Может быть tail call (хвостовой вызов), который выглядит как переход вместо обычного call/ret.

Поэтому .pdata - мощный источник границ, но не единственный. Взрослый reverse engineer сравнивает несколько карт: таблицу функций, дизассемблер, декомпилятор, символы, cross-references, реальные prolog/epilog и поведение в отладчике. Если карты расходятся, это не повод сразу выбирать одну. Это повод понять, почему они расходятся.

3.2.40. Tail call: почему конец функции может выглядеть не как ret

Tail call (хвостовой вызов) - оптимизация, при которой функция в конце не вызывает другую функцию обычным call с последующим ret, а передаёт управление напрямую, часто через jmp. С точки зрения исходного кода это может выглядеть как return other_function(x). Компилятор понимает: после вызова other_function текущей функции уже нечего делать, значит можно не создавать новый возврат, а прыгнуть сразу в другую функцию.

Почему это важно для RUNTIME_FUNCTION? Потому что границы функции и её epilog (эпилог) могут выглядеть не так, как ожидает новичок. Конец диапазона EndAddress не обязан стоять после красивого ret в последней строке. В функции могут быть несколько epilog-участков, tail jumps, оптимизированные выходы. Поэтому нельзя проверять корректность функции примитивно: “в конце диапазона должен быть ret”.

Unwind-модель x64 учитывает, что epilog имеет строгие распознаваемые формы. RtlVirtualUnwind может обрабатывать ситуацию, когда ControlPc находится в epilog. Это одна из причин, почему пролог, эпилог и unwind metadata связаны правилами ABI. RUNTIME_FUNCTION только даёт диапазон; точное понимание выхода из функции требует следующего слоя - UNWIND_INFO и правил epilog.

3.2.41. Padding и alignment: почему между функциями могут быть байты

Padding (заполнение / выравнивающие байты) - байты, которые вставляются между участками кода или данных для выравнивания. Alignment (выравнивание) - размещение кода или данных на адресах, кратных определённому числу, например 16 байтам. Компилятор и линкер делают это ради производительности, требований формата или удобства размещения.

Из-за padding между EndAddress одной функции и BeginAddress следующей могут быть байты, которые дизассемблер покажет как nop, int3 или даже случайные инструкции, если читать их неправильно. Эти байты не обязательно принадлежат настоящей функции. Поэтому gaps между RUNTIME_FUNCTION entries не всегда проблема.

Для parser-а полезно различать “дырку с выравнивающими байтами” и “дырку с реальным исполняемым кодом, который строит кадр”. Первая обычно нормальна. Вторая требует вопроса: почему код не покрыт .pdata?

3.2.42. Почему запись должна вести к unwind-информации именно для этого диапазона

Самое тонкое в RUNTIME_FUNCTION - не прочитать три поля, а доказать их согласованность. UnwindInfoAddress может указывать на синтаксически валидный UNWIND_INFO. Begin/End могут указывать на реальный код. Но этого мало. Нужно, чтобы этот UNWIND_INFO подходил именно этому коду.

Представь две функции. Первая делает sub rsp, 20h и не сохраняет регистры. Вторая делает push rbx, push rsi, sub rsp, 80h. Если записи перепутали UnwindInfoAddress местами, обе цели могут быть “валидными” как структуры, но применены к неправильным функциям. В результате unwind восстановит неверный RSP и неверные регистры.

Поэтому полный анализ строится в три слоя: структурная валидность (можно ли прочитать поля), адресная валидность (попадают ли RVA в образ и секции), семантическая согласованность (соответствует ли unwind metadata реальному прологу). Последний слой самый сложный и самый ценный.

3.2.43. Почему нельзя “починить” RUNTIME_FUNCTION без понимания кода

Иногда возникает желание: если таблица повреждена, давай просто пересчитаем Begin/End по дизассемблеру и поставим новый UnwindInfoAddress. Но RUNTIME_FUNCTION нельзя качественно восстановить без понимания кода и пролога. Граница функции - только половина задачи. Главное - правильная unwind-информация.

Если ты угадал BeginAddress и EndAddress, но не знаешь, какие nonvolatile registers сохранены и сколько стекового пространства выделено, ты не сможешь создать корректный UNWIND_INFO. Неправильная “починка” может сделать файл внешне аккуратнее, но сломать исключения ещё сильнее.

Это особенно важно для dynamic . Если инструмент хочет модифицировать unwind metadata, он должен понимать ABI и реальные инструкции. Иначе он будет не защищать, а повреждать бинарник.

3.2.44. Диагностический язык: как писать findings по RUNTIME_FUNCTION

Finding (обнаруженное несоответствие / диагностический вывод) - конкретный результат проверки, который инструмент сообщает пользователю. Хороший finding должен быть

точным, проверяемым и не истеричным. Он должен говорить, что именно нарушено, где это находится и почему это важно.

Плохая формулировка: “bad .pdata” или “malware detected”. Это ничего не объясняет. Хорошая формулировка: “RUNTIME_FUNCTION entry #37 has EndAddress <= BeginAddress; the function range is empty or reversed, so lookup/unwind for this range cannot be interpreted normally”. Такая запись объясняет факт и последствие.

Для каждого finding полезно хранить evidence (доказательство): номер записи, file offset записи, RVA полей, имя секции, raw bytes, ожидаемое правило, фактическое значение. Тогда отчёт будет не набором мнений, а техническим документом.

3.2.45. Медленный итог на одном примере

Представим запись: BeginAddress = 0x1400, EndAddress = 0x1510, UnwindInfoAddress = 0x3000. Сначала мы говорим: это три RVA. Не VA. Не file offset. Не имена. RVA.

Потом мы спрашиваем: где лежит образ? Если actual ImageBase = 0x00007FF700000000, то кодовый диапазон в памяти - от 0x00007FF700001400 до 0x00007FF700001510, конец не включается. UNWIND_INFO в памяти - 0x00007FF700003000.

Потом мы спрашиваем: в какие секции попали эти RVA? Begin/End должны попадать в исполняемую область, например .text. UnwindInfoAddress должен попадать в область метаданных, например .xdata или read-only data. Если это не так - фиксируем вопрос.

Потом открываем код по BeginAddress. Смотрим пролог. Видим, например: push rbx; sub rsp, 40h. Потом открываем UNWIND_INFO по 0x3000. Он должен описывать сохранение RBX и выделение 0x40 байт. Если описывает другое - это уже семантическая несогласованность.

И только после этого можно сказать, что запись понята. Не “мы распарсили три DWORD”. А “мы проверили, что запись как утверждение о коде согласована с образом и с прологом”.

Техническая база раздела

Основная опора раздела - Microsoft x64 exception handling documentation (документация Microsoft по обработке исключений x64), где описаны RUNTIME_FUNCTION entries, image-relative 32-bit offsets, сортировка и размещение в .pdata PE32+ image, а также связь pdata/xdata. Также используется документация RtlLookupFunctionEntry (поиск записи таблицы функций по PC) и RtlVirtualUnwind (виртуальная раскрутка текущего кадра к вызывающему контексту).

3.3. Что такое .xdata и что там лежит

К разделу .xdata нельзя подходить как к очередной строке в списке секций PE. Если .pdata уже дала нам таблицу диапазонов функций, то .xdata объясняет, что делать с найденной функцией дальше. Это другой слой смысла. .pdata отвечает на вопрос: “какая функция содержит текущий адрес выполнения?”. .xdata отвечает на следующий, более трудный вопрос: “как разобрать стековый кадр этой функции так, чтобы восстановить состояние вызывающей функции?”.

Здесь полезно замедлиться. Когда Windows обрабатывает исключение или строит стек вызовов, ей недостаточно знать имя функции, начало функции или конец функции. Имени может вообще не быть. Символы могут быть удалены. PDB-файл может отсутствовать. Декомпилятор может ошибаться. Но системе всё равно нужно двигаться по стеку. Она должна восстановить caller context (контекст вызывающей функции), то есть состояние, которое было бы у потока после возврата из текущей функции к её caller-у. Для этого нужно знать, что текущая функция успела сделать со стеком и nonvolatile-регистрами.

Именно поэтому рядом с .pdata существует .xdata. Если представить .pdata как оглавление функций, то .xdata - это не текст функции и не исходный код, а компактная инструкция по разборке стекового кадра. Она говорит не “что функция делает логически”, а “какие действия пролога важны для обратного восстановления состояния”. Это очень узкая, но невероятно важная задача.

Откуда появляется потребность в .xdata

Вернёмся к моменту исключения. Поток остановился внутри некоторой инструкции. В контексте потока есть RIP (instruction pointer, указатель текущей инструкции). Через RIP система понимает, внутри какого диапазона кода находится выполнение. Для этого она идёт к таблице .pdata (procedure data section, секция процедурных данных) и ищет RUNTIME_FUNCTION (запись функции времени выполнения / запись диапазона функции).

Пусть поиск успешен. Windows нашла запись, в которой BeginAddress (адрес начала функции) и EndAddress (адрес конца функции) охватывают текущий ControlPc (control program counter, адрес управления, по которому ищется функция). На первый взгляд кажется, что задача решена: функция найдена. Но на самом деле это только первый этаж.

Представь, что мы нашли комнату в здании, но нам нужно выйти из неё назад тем же маршрутом, которым мы вошли. Знать номер комнаты мало. Нужно понять, где дверь, какая мебель перекрывает путь, какие вещи мы оставили на полу, что было передвинуто при входе. В машинном коде роль “мебели” играют изменения RSP (stack pointer, указатель вершины стека), сохранённые nonvolatile registers (невытесняемые / сохраняемые регистры), frame pointer (указатель кадра), shadow space (теневое место для регистровых аргументов) и локальные стековые области.

Функция при входе может выполнить push rbx, sub rsp, 80h, mov rbp, rsp, сохранить r12, r13, r14, выделить большой стековый блок через __chkstk, выровнять стек, подготовить место под вызовы других функций. Пока функция выполняется, текущий RSP уже не равен тому RSP, который был у caller-а. Некоторые регистры уже могут содержать новые значения, а старые лежат где-то в текущем кадре. Если просто взять текущий RSP и прочитать “следующий адрес возврата”, можно легко ошибиться.

Значит, системе нужен формальный рецепт: как из текущего состояния получить предыдущее. Этот рецепт и хранится в .xdata (exception data section, секция данных исключений). Название “exception data” может звучать так, будто эта секция нужна только для

исключений. На практике она важна шире: для exception handling (обработки исключений), debugger support (поддержки отладчиков), stack walking (прохода по стеку) и crash dump analysis (анализа аварийных дампов).

Термин .xdata: что означает имя и почему нельзя понимать его слишком буквально

.xdata (exception data section, секция данных исключений) - это обычно секция PE32+ image, где хранятся unwind data (данные раскрутки стека) и связанные с ними сведения для обработки исключений. Слово “обычно” здесь важно. В нормальных MSVC-сборках ты часто увидишь секцию с именем .xdata, и большинство учебных объяснений говорят именно так: .pdata указывает на .xdata. Для понимания механизма это удобно.

Но reverse engineer не должен превращать удобную картинку в жёсткое правило. PE loader (загрузчик PE-образа) и runtime-механизмы работают не с “красивым названием секции” как главным источником истины, а с RVA (relative virtual address, относительный виртуальный адрес) и таблицами. Если RUNTIME_FUNCTION указывает своим UnwindInfoAddress (адрес unwind-информации) на некоторое место в загруженном image (образе), parser (разборщик) должен проверить, куда этот RVA попадает в виртуальном layout-е образа, а не слепо искать секцию с названием .xdata.

Это различие очень важно для твоего будущего static/dynamic. На обычных бинарниках метаданные действительно будут лежать в .xdata. Но упаковщик, нестандартный linker (компоновщик), ручная правка PE или обфускатор могут расположить данные в секции с другим именем, хотя формально RVA всё ещё будет указывать на допустимую область образа. Поэтому корректная логика такая: название .xdata помогает человеку ориентироваться, но машинная проверка должна идти через адреса, размеры секций и data directories.

> Ключевая мысль: .xdata - это не магическое имя. Это роль данных. Обычно эта роль выражена секцией с именем .xdata, но parser обязан смотреть на RVA, границы image и права/размеры секций, а не только на строковое имя секции.

Что именно хранится в .xdata

Внутри .xdata лежит не один большой список одинаковых записей, как в .pdata. Там находятся записи переменной длины. Главный объект, с которого начинается понимание этой секции, называется UNWIND_INFO (структура unwind-информации). Это компактное описание того, как конкретная функция строит свой стековый кадр и какие дополнительные данные нужны для обработки исключений.

UNWIND_INFO (структура unwind-информации) - это не дизассемблированный пролог. Она не хранит байты инструкций push, sub, mov. Она хранит кодированное описание действий, которые важны для восстановления предыдущего контекста. Например: функция сохранила RBX; функция выделила N байт на стеке; функция использует RBP как frame register; у функции есть exception handler; после массива unwind-кодов лежат дополнительные language-specific data.

Здесь важно разделить два уровня. Первый уровень - real code (реальный код), который находится в .text (code section, секция кода) и исполняется процессором. Второй уровень - metadata (метаданные), которые находятся в .xdata и не исполняются как код. Метаданные описывают свойства кода, но сами не являются кодом.

Если функция в .text начинается с нескольких инструкций пролога, то UNWIND_INFO в .xdata является сжатой картой этих инструкций. Не всех инструкций, а только тех, которые

имеют значение для unwind: изменение RSP, сохранение nonvolatile-регистров, установка frame pointer, наличие обработчика или chained info. Если инструкция просто вычисляет значение, сравнивает аргументы или вызывает внутреннюю логику, unwind-формату она неинтересна.

Почему .xdata нельзя читать как обычный массив

.pdata обычно можно воспринимать как массив RUNTIME_FUNCTION-записей фиксированного размера. У каждой записи три 32-битных поля: BeginAddress, EndAddress и UnwindInfoAddress. Это удобно: размер записи понятен, количество записей можно вывести из размера Exception Directory, можно идти по таблице шагом 12 байт.

.xdata устроена иначе. UNWIND_INFO имеет заголовок, затем массив UNWIND_CODE (коды unwind-действий), затем возможное выравнивание, затем optional area (дополнительная область), смысл которой зависит от флагов. Одна функция может иметь короткую запись, другая - длинную. Некоторые unwind operations занимают один слот, другие используют дополнительные слоты. Если есть обработчик, после unwind-кодов будет handler RVA (RVA обработчика) и language-specific data (данные, специфичные для языка/компилятора). Если включён chained unwind info (цепочечная unwind-информация), там будет не handler, а связанная RUNTIME_FUNCTION-подобная запись.

Поэтому .xdata нельзя корректно читать как “возьми структуру одинакового размера и двигайся дальше”. Parser должен начинать с конкретного UnwindInfoAddress из RUNTIME_FUNCTION, читать заголовок, понять CountOfCodes, учесть выравнивание, посмотреть Flags, и только потом вычислить, где заканчивается запись и что лежит после неё. Ошибка на одном шаге смещает всё последующее чтение.

Это одна из причин, почему .xdata так интересна для reverse engineering. В .pdata ошибки часто выглядят как неправильный диапазон или неправильный RVA. В .xdata ошибки тоньше: неверный CountOfCodes, недостаточно байт, странная комбинация флагов, operation code требует дополнительный слот, но его нет, handler RVA выходит за image, chained info указывает на диапазон, не согласованный с основной записью.

Общая форма UNWIND_INFO

Теперь введём UNWIND_INFO (структура unwind-информации) медленно. Это центральная структура .xdata. Она начинается с небольшого заголовка. В нём есть поля Version (версия формата), Flags (флаги), SizeOfProlog (размер пролога), CountOfCodes (количество unwind-code слотов), FrameRegister (регистр кадра) и FrameOffset (смещение кадра). После заголовка идёт массив UNWIND_CODE.

На бумаге это можно записать компактно, но важно помнить: компактная запись не должна превращаться в компактное понимание. Каждое поле нужно связывать с живым процессом выполнения функции.

```
UNWIND_INFO:
  Version + Flags
  SizeOfProlog
  CountOfCodes
  FrameRegister + FrameOffset
  UnwindCode[CountOfCodes]
  padding/alignment if needed
  optional area depending on Flags
```

Эта схема не означает, что каждый UNWIND_INFO одинаков по длине. Наоборот, длина зависит от CountOfCodes и от Flags. Поэтому разбор начинается не с “я знаю размер”, а с “я

читаю заголовок, из него вычисляю, сколько дальше нужно прочитать”.

Version (версия формата): маленькое поле, которое задаёт правила чтения

Version (версия формата) - это поле, которое говорит, по какой версии схемы нужно интерпретировать запись. В классическом x64 UNWIND_INFO обычно встречается версия 1. В более новых расширениях Microsoft также документирует x64 Unwind Information V3 (информация unwind версии 3), которая решает отдельные задачи и имеет свои детали.

Для начинающего reverse engineer версия может казаться формальностью: “ну версия и версия”. Но для parser-а это вопрос безопасности чтения. Если инструмент ожидает одну схему, а встречает другую, он не должен притворяться, что всё нормально. Он должен либо поддерживать эту версию, либо честно выдать finding (обнаруженное замечание/признак): неизвестная или неподдержанная версия unwind-формата.

Зачем это нужно? Форматы живут долго. Windows, компиляторы и runtime-библиотеки развиваются. Поле Version позволяет отличать старые и новые варианты представления unwind-информации. Если такого поля не было бы, любой новый формат ломал бы старые parser-ы молча. С версией инструмент хотя бы может сказать: “я понял, что это не тот вариант записи, который умею читать”.

> Типичная ошибка: считать Version декоративным полем. Для ручного чтения одного обычного файла это может пройти. Для инструмента анализа это ошибка архитектуры: версия определяет правила интерпретации байтов.

Flags (флаги): маленькие биты, которые меняют смысл хвоста структуры

Flags (флаги) - это битовое поле. Битовое поле означает, что внутри одного числа отдельные биты используются как переключатели. Не всё число целиком является одним значением; каждый бит или группа битов может означать отдельное свойство.

В UNWIND_INFO флаги отвечают за то, есть ли у функции дополнительные данные после массива unwind-кодов. Самые важные для нас флаги: UNW_FLAG_EHANDLER (флаг exception handler, обработчика исключений), UNW_FLAG_UHANDLER (флаг termination/unwind handler, обработчика при раскрутке) и UNW_FLAG_CHAININFO (флаг chained information, цепочечной информации).

UNW_FLAG_EHANDLER (флаг обработчика исключений) говорит, что после unwind-кодов находится RVA обработчика исключений и дополнительные данные для него. Это связывает низкоуровневую unwind-информацию с более высоким уровнем обработки: например, с __C_specific_handler и таблицами, которые описывают __try/__except.

UNW_FLAG_UHANDLER (флаг обработчика раскрутки / termination handler) похож тем, что тоже приводит к handler RVA, но его смысл связан с обработчиком, который важен во время unwind phase (фазы раскрутки). В языковых конструкциях это может соответствовать finally-подобной логике или cleanup-действиям.

UNW_FLAG_CHAININFO (флаг цепочечной информации) меняет интерпретацию хвоста ещё сильнее: там ожидается не обычный handler RVA с language-specific data, а chained RUNTIME_FUNCTION-подобная запись. То есть optional area начинает означать “смотри связанную unwind-запись”, а не “вызывай обработчик”.

Вот почему Flags нельзя читать поверхностно. Эти несколько битов решают, как parser должен интерпретировать байты после массива UNWIND_CODE. Один и тот же набор байтов после кодов может быть либо адресом handler-a, либо частью chained info, в зависимости от флагов. Если parser проигнорирует Flags, он может построить красивую, но полностью ложную картину.

Флаг	Человеческий смысл	Что меняет в чтении .xdata
UNW_FLAG_EHANDLER	У функции есть exception handler (обработчик исключения).	После unwind-кодов ожидается RVA handler-a и данные обработчика.
UNW_FLAG_UHANDLER	У функции есть unwind/termination handler (обработчик при раскрутке).	Хвост записи тоже содержит handler RVA и дополнительные данные.
UNW_FLAG_CHAININFO	Запись связана с другой записью функции.	Хвост читается как chained RUNTIME_FUNCTION, а не как обычный handler.

Таблица 3.3.1. Флаги UNWIND_INFO меняют смысл дополнительной области после массива unwind-кодов.

SizeOfProlog (размер пролога): связь .xdata с реальными инструкциями

SizeOfProlog (размер пролога) - это размер пролога функции в байтах. Prolog (пролог функции) - начальный участок машинного кода функции, где функция подготавливает свой стековый кадр: сохраняет нужные регистры, выделяет место на стеке, иногда устанавливает frame pointer, выполняет служебные действия calling convention.

Зачем хранить размер пролога? Потому что unwind зависит от того, где именно находится ControlPc. Если исключение произошло уже в теле функции, можно считать, что пролог полностью выполнен. Тогда все действия пролога надо учитывать при восстановлении caller context. Если исключение произошло внутри самого пролога, ситуация тоньше: часть действий уже выполнена, часть ещё нет. Нельзя “отменять” действие, которое функция ещё не успела сделать.

Простой пример. Функция начинается так:

```
push rbx        ; сохранили RBX на стеке
sub  rsp, 40h   ; выделили 0x40 байт стекового пространства
...             ; дальше тело функции
```

Если исключение произошло после обеих инструкций, unwind должен учитывать и push rbx, и sub rsp, 40h. Если исключение произошло между ними, RBX уже сохранён, но RSP ещё не уменьшен на 0x40. Если исключение произошло на первой инструкции до её завершения, состояние ещё другое. Поэтому UNWIND_CODE хранит offsets (смещения внутри пролога), а SizeOfProlog задаёт границы участка, где такие частичные состояния особенно важны.

Эта деталь показывает, почему .xdata - не просто “таблица для отладчика”. Она нужна для точного восстановления состояния даже тогда, когда поток остановился в середине входа в функцию. Хороший parser должен не только прочитать SizeOfProlog, но и проверять, что offsets в UNWIND_CODE не выходят за разумные границы пролога.

CountOfCodes (количество unwind-code слотов): почему это не всегда количество операций

CountOfCodes (количество unwind-code слотов) - поле, которое говорит, сколько элементов UNWIND_CODE нужно прочитать после заголовка. Но здесь есть ловушка: это количество слотов, а не обязательно количество логических действий.

UNWIND_CODE (код unwind-действия) - маленькая запись, которая описывает одно действие пролога или часть его кодирования. Некоторые unwind operations (операции unwind) помещаются в один слот. Другие требуют дополнительных слотов, потому что им нужно хранить большой размер stack allocation (выделения стека) или offset сохранённого регистра. Поэтому CountOfCodes говорит parser-у: “прочитай столько слотов”, но интерпретатор должен понимать, что одна операция может потребить больше одного слота.

Это похоже на чтение машинных инструкций переменной длины. Если ты просто будешь считать “один слот - одна операция”, ты ошибёшься на UWOP_ALLOC_LARGE (операция большого выделения стека), UWOP_SAVE_NONVOL (сохранение nonvolatile-регистра в стековый слот) и некоторых других случаях. Правильный parser должен идти по массиву, читать operation code, знать, сколько дополнительных слотов ей нужно, и проверять, что эти слоты действительно существуют внутри CountOfCodes.

> Типичная ошибка: CountOfCodes воспринимают как “количество строк операций”. На самом деле это количество 2-байтовых слотов UNWIND_CODE. Некоторые операции занимают несколько слотов.

FrameRegister и FrameOffset: когда функция использует устойчивую базу кадра

FrameRegister (регистр кадра) и FrameOffset (смещение кадра) описывают ситуацию, когда функция использует frame pointer (указатель кадра) или frame register (регистр кадра). На x64 это не обязательно. Многие функции работают только через RSP и не держат отдельный классический RBP-frame. Но иногда компилятору или ABI-логике нужно иметь устойчивую базу внутри кадра.

Почему RSP не всегда удобен как база? Потому что RSP может меняться. При вызовах, временных push/pop или некоторых формах выделения стека он может быть не самым удобным стабильным ориентиром. Frame pointer позволяет сказать: “вот регистр, который указывает на фиксированное место внутри кадра; относительно него можно адресовать локальные данные или восстановить структуру кадра”.

FrameRegister указывает, какой регистр используется как база кадра, например RBP. FrameOffset задаёт смещение в специальных единицах, зависящих от формата. Важно не просто прочитать эти поля, а понять их смысл: они помогают unwind-механизму и иногда отладчику ориентироваться в кадре, когда один RSP как точка отсчёта недостаточен.

Для reverse engineer наличие frame register часто облегчает чтение дизассемблированного кода. Когда локальные переменные адресуются через RBP, структура кадра выглядит устойчивее. Но на оптимизированном x64-коде RBP может использоваться как обычный nonvolatile-регистр, а frame pointer может отсутствовать. Поэтому нельзя автоматически считать, что RBP всегда “указывает на кадр”, как в старых учебных схемах x86.

UNWIND_CODE: язык, на котором .xdata описывает пролог

Теперь можно подойти к UNWIND_CODE (код unwind-действия) как к языку описания пролога. Это не язык высокого уровня и не машинный код процессора. Это маленький специализированный язык, созданный для одной задачи: описать действия пролога, которые нужно уметь обратить при unwind.

Каждый UNWIND_CODE содержит offset внутри пролога и operation code (код операции). Offset говорит, к какому месту пролога относится действие. Operation code говорит, что именно произошло: push nonvolatile-регистра, выделение стека, установка frame pointer,

сохранение регистра по offset и так далее.

Важно понимать, почему формат не хранит сами инструкции. У процессора много способов выразить похожее действие. Например, сохранение регистра можно сделать разными инструкциями, выделение стека может быть коротким или большим, компилятор может выбирать разные формы кодирования. Windows unwind не нужен полный смысл всех инструкций. Ему нужен итог, важный для восстановления: “этот регистр был сохранён здесь”, “стек был уменьшен на столько-то”, “кадр использует такой-то регистр”.

Поэтому UNWIND_CODE - это не disassembly (дизассемблирование), а semantic summary (смысловая сводка) пролога. Она компактнее, чем код, и ориентирована на обратное восстановление состояния.

Эта разница важна для static /dynamic. Инструмент может сравнивать два слоя: реальные инструкции пролога в .text и заявленные unwind-коды в .xdata. Если функция делает push rbx, но unwind-информация этого не описывает, это подозрительно. Если unwind-информация говорит о сохранении r12, но в прологе такого действия нет, это тоже finding. Но сравнение должно быть осторожным: оптимизации, epilog scopes, chained info и нестандартная генерация кода могут усложнять картину.

Порядок UNWIND_CODE и почему offsets важнее красивой записи

UNWIND_CODE entries (элементы массива кодов unwind) не являются просто списком “как человек написал бы инструкции сверху вниз”. В документации для обычного x64 unwind-формата элементы связаны с offsets внутри пролога и упорядочиваются так, чтобы unwind-алгоритм мог корректно применять их при восстановлении. Практически для parser-а важно не предполагать произвольный порядок и не игнорировать offset.

Offset внутри пролога нужен потому, что ControlPc может находиться не только в теле функции, но и в самом прологе. Если функция остановилась внутри пролога, unwind должен применить только те действия, которые уже произошли. Поэтому каждое действие должно быть привязано к месту в прологе. Без offsets система могла бы только грубо сказать “пролог делает вот это”, но не смогла бы корректно обработать частично выполненный пролог.

Представь, что пролог состоит из трёх действий: сохранить RBX, сохранить RDI, выделить стек. Если исключение случилось после первого действия, нельзя восстанавливать RDI как будто он уже был сохранён. Если случилось после третьего, нужно учитывать всё. Отсюда становится понятно, что SizeOfProlog и offsets в UNWIND_CODE работают вместе.

Optional area: хвост записи, который нельзя читать без Flags

После массива UNWIND_CODE может находиться optional area (дополнительная область). Слово optional (необязательная) не означает “неважная”. Оно означает: эта область есть не всегда, а её смысл зависит от Flags.

Если установлен UNW_FLAG_EHANDLER или UNW_FLAG_UHANDLER, после выравнивания обычно следует handler RVA (RVA обработчика). Handler RVA - это relative virtual address обработчика, то есть не file offset и не готовый VA, а смещение относительно ImageBase. Этот обработчик может быть, например, __C_specific_handler (специфический C/SEH-обработчик MSVC), который умеет читать дополнительные language-specific data.

Language-specific data (данные, специфичные для языка/компилятора) - это данные, смысл которых не обязан быть универсальным для всей Windows. Сама Windows unwind-машина знает общий формат UNWIND_INFO, но конкретный handler может интерпретировать хвост по

правилам компилятора. Для __try/__except там будут одни сведения, для C++ EH - более сложные структуры runtime-а.

Если установлен UNW_FLAG_CHAININFO, optional area интерпретируется иначе: там находится chained RUNTIME_FUNCTION (цепочечная запись функции). Это не “обработчик”. Это способ связать текущую unwind-информацию с другой записью диапазона. Если parser не различает эти случаи, он может принять chained info за handler RVA и выдать полный мусор.

```
После UNWIND_CODE:

если Flags содержит EHANDLER / UHANDLER:
    DWORD HandlerRVA
    ... language-specific data ...

если Flags содержит CHAININFO:
    RUNTIME_FUNCTION chained entry

если Flags == 0:
    дополнительной области может не быть
```

Выравнивание: скучная деталь, без которой парсер съедет на байты

Alignment (выравнивание) - это требование располагать данные по адресам, кратным определённому числу байт. В unwind-формате выравнивание важно при переходе от массива UNWIND_CODE к optional area. После CountOfCodes слотов может потребоваться padding (заполнитель), чтобы следующая часть начиналась на нужной границе.

Почему это не мелочь? Потому что parser читает бинарный формат. Если он ошибся на 2 байта, он начнёт читать handler RVA не с того места. Получится неверный адрес обработчика. Затем неверный адрес может попасть вне image, в другую секцию или выглядеть как случайное значение. Дальше все выводы будут ошибочными.

Вручную эта ошибка особенно коварна. Человек видит CountOfCodes, умножает на размер слота, сразу читает следующий DWORD и думает, что это handler. Но если количество слотов нечётное, нужно учесть выравнивание. Поэтому хорошая книга и хороший инструмент должны постоянно повторять: бинарный формат читается не по ощущениям, а по правилам размеров и выравнивания.

Как RtlVirtualUnwind использует .xdata

Теперь свяжем .xdata с реальным runtime-механизмом. RtlVirtualUnwind (виртуальная раскрутка стека) - функция Windows, которая принимает текущий контекст и информацию о функции, а затем вычисляет контекст caller-а. Слово virtual (виртуальная) здесь означает, что система не обязательно реально выполняет ret и не обязательно физически разрушает стек. Она вычисляет, каким был бы контекст после выхода из текущей функции.

RtlVirtualUnwind получает FunctionEntry (запись функции), связанную с RUNTIME_FUNCTION. В этой записи есть UnwindInfoAddress, который ведёт к UNWIND_INFO. Дальше RtlVirtualUnwind читает unwind-коды и применяет их к текущему контексту: восстанавливает nonvolatile-регистры из стека, корректирует RSP, определяет адрес возврата, обновляет RIP для caller-а.

Если у UNWIND_INFO есть handler flags, RtlVirtualUnwind также помогает сформировать данные, нужные для dispatcher-а исключений. Здесь появляется DispatcherContext (контекст диспетчера), EstablisherFrame (кадр установления обработчика) и другие структуры, которые

будут подробно разбираться позже. Сейчас важно понять: .xdata не лежит мёртвым грузом. Она читается runtime-логикой Windows при каждом формальном подъёме по стеку.

Без .xdata Windows могла бы видеть диапазон функции, но не знала бы, как из него выйти. Это как знать, что человек находится в комнате, но не знать, где дверь и какие предметы он переставил перед входом.

Почему .xdata описывает пролог, а не всю функцию

На этом месте часто возникает естественный вопрос: если .xdata нужна для восстановления состояния, почему она не описывает всю функцию? Почему только пролог и связанные сведения?

Ответ в том, что для unwind не нужно понимать бизнес-логику функции. Системе неважно, сортирует ли функция массив, проверяет пароль, парсит PE или считает хэш. Ей важно только то, какие действия изменили стековый кадр и сохраняемые регистры. Обычно эти действия сосредоточены в prolog (прологе функции), а обратные действия - в epilog (эпилоге функции).

Пролог строит кадр. Эпилог разбирает кадр. Тело функции делает полезную работу. Unwind интересуется строительством и разборкой кадра, а не полезной работой. Поэтому UNWIND_INFO компактно описывает пролог, а runtime знает специальные правила распознавания эпилога.

Эта архитектура делает формат небольшим. Если бы Windows требовала полное описание всех инструкций каждой функции, бинарники были бы тяжелее, компиляторы сложнее, а runtime медленнее. Вместо этого существует минимальный контракт: функция должна иметь пролог/эпилог в ограниченной форме, а метаданные должны описывать действия, важные для unwind.

Что .xdata не хранит

Чтобы не переоценивать .xdata, нужно также понимать, чего там нет.

.xdata не хранит имена функций. Имена, если они есть, живут в symbol information (символьной информации), например в PDB (Program Database, файл отладочных символов), экспортах или других источниках. RUNTIME_FUNCTION и UNWIND_INFO работают без имён. Windows не нужно знать, называется функция parse_header или sub_140012340. Ей нужно знать диапазон и способ unwind.

.xdata не хранит исходный код. Она не знает про переменные C, классы C++, циклы или условия. Она знает только машинно-релевантные действия кадра.

.xdata не является списком всех инструкций. Она не заменяет дизассемблер. Если тебе нужно понять логику функции, ты идёшь в .text и анализируешь код. Если тебе нужно понять, как функция раскручивается при исключении или stack walk, ты идёшь в .pdata/.xdata.

.xdata не обязательно доказывает, что код “безопасен”. Корректная unwind-информация говорит, что системный контракт, возможно, соблюдён. Но функция всё равно может содержать баги, уязвимости или вредоносную логику.

> Главное разграничение: .text отвечает за выполняемый код; .pdata отвечает за индекс диапазонов функций; .xdata отвечает за рецепт восстановления кадра и дополнительные exception/unwind данные.

Как .pdata и .xdata работают вместе

Теперь можно собрать связку без спешки. .pdata и .xdata - это не две независимые секции. Они образуют пару.

.pdata (procedure data section, секция процедурных данных) содержит RUNTIME_FUNCTION records (записи диапазонов функций). Каждая такая запись говорит: функция начинается по BeginAddress, заканчивается по EndAddress, а её unwind-информация лежит по UnwindInfoAddress.

.xdata (exception data section, секция данных исключений) содержит то, на что указывает UnwindInfoAddress: UNWIND_INFO и связанные дополнительные данные.

Когда происходит исключение, Windows идёт не “сразу в .xdata”. Сначала есть ControlPc. По нему через function table находится RUNTIME_FUNCTION. Только потом из RUNTIME_FUNCTION система получает UnwindInfoAddress и попадает в .xdata.

```
ControlPc / RIP
    -> найти RUNTIME_FUNCTION в .pdata
        -> взять UnwindInfoAddress
            -> перейти к UNWIND_INFO в .xdata
                -> прочитать unwind-коды
                    -> восстановить caller context
```

Эта цепочка важна для проверки. Если .pdata корректна, но UnwindInfoAddress указывает в мусор, stack walking ломается. Если .xdata корректна сама по себе, но .pdata не указывает на неё, Windows её не найдёт обычным путём. Если обе секции есть, но адреса перепутаны как file offset вместо RVA, parser ошибётся.

Что значит “прочитать .xdata правильно”

Правильное чтение .xdata начинается не с красивого вывода в таблицу, а с дисциплины. Parser должен постоянно задавать вопросы: могу ли я безопасно прочитать следующий байт? знаю ли я, в какой системе координат находится адрес? хватает ли Size секции или image-диапазона? согласованы ли CountOfCodes и фактический размер? что говорят Flags? где заканчивается текущая запись?

Минимальный путь чтения выглядит так. Сначала parser получает RVA из RUNTIME_FUNCTION.UnwindInfoAddress. Затем переводит этот RVA в file offset или в адрес внутри mapped image, в зависимости от режима анализа. Потом проверяет, что по этому месту можно прочитать минимум заголовок UNWIND_INFO. Затем читает Version, Flags, SizeOfProlog, CountOfCodes, FrameRegister и FrameOffset. После этого вычисляет размер массива UNWIND_CODE. Затем учитывает выравнивание. Потом, если Flags требуют дополнительной области, проверяет наличие handler RVA или chained info.

Каждый шаг может дать finding (обнаружение/замечание). Finding - это не обязательно “фатальная ошибка”. Это наблюдение анализатора, которое имеет severity (серьёзность), evidence (доказательную базу) и explanation (объяснение). Например: “UnwindInfoAddress points outside image” - критично; “UNWIND_INFO is located outside a section named .xdata but inside a readable section” - возможно нестандартно, но не обязательно ошибка; “CountOfCodes consumes bytes beyond section boundary” - серьёзная структурная проблема.

Такой подход превращает инструмент из простого парсера в анализатор. Parser читает байты. Analyzer объясняет, насколько эти байты соблюдают контракт Windows x64 unwind.

Типичные ошибки понимания .xdata

Первая ошибка - думать, что .xdata является таблицей функций. Таблица диапазонов функций - это .pdata. .xdata не отвечает на вопрос “где начинается функция?”. Она отвечает на

вопрос “как раскрутить найденную функцию?”.

Вторая ошибка - думать, что UnwindInfoAddress является file offset. Это RVA. Его нужно интерпретировать относительно ImageBase в памяти или переводить через секции при чтении файла. Если сразу использовать его как смещение от начала файла, анализ почти наверняка уйдёт не туда.

Третья ошибка - считать .xdata всегда секцией с именем .xdata. В нормальном случае это хороший ориентир, но не абсолютное правило. Анализатор должен работать с адресами и границами, а не только с именами секций.

Четвёртая ошибка - читать CountOfCodes как количество операций. Это количество слотов, и некоторые операции используют дополнительные слоты.

Пятая ошибка - игнорировать Flags. Без Flags невозможно понять, что лежит после unwind-кодов: handler RVA, language-specific data, chained RUNTIME_FUNCTION или ничего.

Шестая ошибка - воспринимать .xdata как исчерпывающее описание функции. Она описывает только то, что нужно для unwind/exception machinery. Логика функции находится в .text.

Почему .xdata особенно важна для reverse engineering

Для обычного программиста .xdata часто невидима. Он пишет C/C++, компилятор генерирует код, linker собирает PE, Windows всё использует автоматически. Но reverse engineer работает там, где автоматическое доверие опасно.

.xdata даёт дополнительный источник истины о функции. Дизассемблер показывает инструкции. .pdata показывает диапазон. .xdata показывает, как компилятор заявил Windows устройство кадра. Если эти три источника согласованы, анализ становится увереннее. Если они расходятся, появляется вопрос: это оптимизация, нестандартная генерация, упаковщик, JIT, повреждение, антианализ или ошибка parser-а?

Например, .pdata говорит, что функция начинается по RVA 0x1200. Дизассемблер показывает в начале push rbx и sub rsp, 60h. .xdata содержит UNWIND_INFO, где есть PUSH_NONVOL RBX и ALLOC_SMALL на соответствующий размер. Это согласованная картина.

Другой пример: .pdata указывает на функцию, но UnwindInfoAddress выходит за пределы image. Для Windows это проблема надёжного unwind. Для static /dynamic это finding высокого уровня: запись функции не может быть использована для корректного восстановления кадра.

Третий пример: UNWIND_INFO указывает handler RVA, но handler RVA попадает в секцию без execute-права или вообще вне допустимого диапазона. Это уже повод отдельно анализировать, ошибка ли это, обфускация, нестандартный loader или повреждённый файл.

Разбор на живом примере: от пролога к .xdata

Чтобы .xdata перестала быть абстрактной секцией, возьмём воображаемую, но типичную функцию. Пусть компилятор сгенерировал функцию, которая вызывает другие функции, использует RBX как рабочий регистр, сохраняет RBP как frame pointer и выделяет место на стеке. В дизассемблере начало может выглядеть примерно так:

```
sub_140012000:
    push rbp
    push rbx
    sub  rsp, 80h
    lea  rbp, [rsp+20h]
```

```
... тело функции ...
```

Если смотреть только на этот пролог глазами человека, мы понимаем: функция сохранила RBP, сохранила RBX, выделила 0x80 байт стекового пространства и сделала RBP устойчивой базой для обращения к части кадра. Но Windows не будет каждый раз дизассемблировать эти инструкции при исключении. Ей нужен заранее подготовленный, компактный и формальный рецепт. Этот рецепт и будет лежать в UNWIND_INFO.

Важно заметить слово “формальный”. Человек может сказать: “ну здесь примерно понятно, что произошло”. Windows не может работать на “примерно”. Если исключение произошло внутри этой функции, система должна вычислить caller context точно: где был RSP у вызывающей функции, где лежит адрес возврата, как восстановить старые значения RBP и RBX. Ошибка на 8 байт уже может означать неправильный stack trace, неверный handler или невозможность корректно продолжить обработку.

В .pdata для этой функции будет RUNTIME_FUNCTION. Его BeginAddress укажет на начало диапазона, EndAddress - на конец диапазона, а UnwindInfoAddress - на место в .xdata, где лежит UNWIND_INFO. Сама .pdata не будет перечислять push rbp, push rbx и sub rsp, 80h. Она только покажет дверь в .xdata.

В .xdata UNWIND_INFO скажет примерно следующее: пролог имеет такой-то размер; в прологе есть действия сохранения nonvolatile-регистров; есть выделение стекового пространства; используется frame register. В зависимости от точных инструкций и кодировки компилятора эти действия будут представлены конкретными UWOP_* операциями. Мы разберём их позже отдельно, но уже сейчас нужно понять архитектурную роль: .xdata переводит реальные инструкции пролога в системно-понятный язык unwind-действий.

Теперь представь, что access violation произошло уже в теле функции. На входе у Windows есть RIP внутри sub_140012000 и текущий RSP после всех действий пролога. RtlLookupFunctionEntry находит RUNTIME_FUNCTION. RtlVirtualUnwind идёт в .xdata, читает UNWIND_INFO и понимает: чтобы подняться к caller-у, нужно учесть выделенные 0x80 байт, восстановить RBX и RBP из сохранённых мест, затем получить адрес возврата. Так из текущего кадра получается предыдущий.

Если же исключение произошло прямо в середине пролога, например после push rbp, но до sub rsp, 80h, ситуация другая. Нельзя применять весь рецепт целиком. Поэтому в UNWIND_CODE важны offsets внутри пролога: они позволяют понять, какие действия уже произошли к моменту ControlPc, а какие ещё нет. Это именно тот уровень точности, ради которого unwind-данные существуют.

Почему .xdata связана не только с “ошибками”, но и с нормальной структурой программы

Название exception data иногда сбивает с толку: кажется, будто .xdata нужна только при авариях. Но unwind-информация является частью нормального двоичного контракта программы. Компилятор генерирует её не потому, что ожидает ошибку в каждой функции, а потому что 64-битная Windows должна иметь единый способ понимать стек.

Stack trace (трасса стека) в отладчике, crash dump (аварийный дамп), профилирование, обработка SEH, C++ exception handling, вызов termination handlers - все эти задачи в разной степени требуют знания того, как перейти от текущего кадра к предыдущему. Поэтому .xdata лучше понимать как инфраструктуру навигации по выполнению, а не как “кладбище ошибок”.

Это особенно важно для reverse engineering. Когда ты видишь .xdata, ты видишь не просто технический хвост компилятора. Ты видишь способ, которым бинарник объясняет

операционной системе свою стековую дисциплину. Если этот способ нарушен, скрыт, подменён или странно организован, это может быть очень сильным признаком нестандартности бинарника.

Где .xdata находится физически: файл, image и память

Нужно снова развести три слоя: file on disk (файл на диске), loaded image (загруженный образ) и process memory (память процесса). .xdata как секция существует в PE-файле на диске, но Windows использует её уже как часть загруженного image. Это значит, что адреса в RUNTIME_FUNCTION являются image-relative, то есть RVA, а не file offset.

Когда parser читает файл статически, у него есть байты файла и section table (таблица секций). Чтобы добраться до .xdata по RVA, он должен перевести RVA в file offset через соответствующую секцию. Когда debugger или runtime работает с живым процессом, он может использовать VA: actual ImageBase плюс RVA. Это два разных режима, но данные описывают один и тот же логический объект.

Ошибка возникает, когда человек смешивает эти режимы. Например, он берёт UnwindInfoAddress = 0x5000 и идёт на offset 0x5000 в файле. Иногда случайно попадёт в похожее место, но метод неправильный. Правильный вопрос всегда такой: это RVA? Если да, какая секция покрывает этот RVA? Какой у секции VirtualAddress, PointerToRawData, VirtualSize и SizeOfRawData? Только после этого можно читать байты файла.

В живом процессе вопрос другой: какой actual ImageBase у модуля? Если ASLR (Address Space Layout Randomization, рандомизация базового адреса) загрузила модуль не по preferred ImageBase, то VA будет другим. Но RVA внутри образа останется той же координатой относительно базы. Поэтому формат хранит RVA: это делает метаданные независимыми от конкретного адреса загрузки.

Почему .xdata не обязана быть понятной без .text

Можно открыть .xdata и формально распарсить UNWIND_INFO. Но настоящее понимание возникает только при сравнении с .text. .text (code section, секция кода) содержит реальные инструкции функции. .xdata содержит заявленное описание действий пролога. Между ними должен быть смысловой мост.

Если .xdata говорит, что функция выделяет 0x80 байт стека, в прологе обычно должна быть инструкция, которая приводит к такому изменению RSP: например sub rsp, 80h или эквивалентная последовательность. Если .xdata говорит, что сохранён RBX, в прологе должен быть способ сохранения RBX. Если .xdata указывает frame register, код должен устанавливать соответствующую базу кадра.

Но это сравнение нельзя делать грубо. Компиляторы умеют оптимизировать, prolog может иметь ограниченные формы, большой стек может проходить через __chkstk, epilog может встречаться в нескольких местах, а некоторые функции могут быть leaf functions и не иметь полной записи. Поэтому хороший анализатор не должен сразу кричать “ошибка” при первом несовпадении. Он должен собирать evidence: какие байты кода видны, какие unwind-коды заявлены, какие адреса и размеры, какой компилятор вероятен, есть ли признаки упаковки или ручной правки.

Для учебной книги это важный метод мышления: .xdata не изучается отдельно от кода. Она изучается как договор между кодом функции и системой unwind. Договор можно проверить только сравнив обе стороны: что функция реально делает и что она заявила в метаданных.

Мягкая и жёсткая проверка .xdata

Когда ты будешь проектировать static /dynamic, полезно разделять hard validation (жёсткую проверку) и soft validation (мягкую проверку). Жёсткая проверка отвечает на вопрос: можно ли вообще безопасно прочитать структуру и не нарушает ли она очевидные правила формата. Мягкая проверка отвечает: выглядит ли структура согласованной с кодом и нормальными ожиданиями компилятора.

Жёсткие проверки обычно ближе к бинарной корректности. Например: UnwindInfoAddress должен указывать внутрь image; должна быть возможность прочитать заголовок UNWIND_INFO; CountOfCodes не должен уводить чтение за границы секции/image; optional area должна помещаться в доступный диапазон; chained info должна иметь достаточно байт; handler RVA должен быть корректно интерпретируемым RVA.

Мягкие проверки требуют осторожности. Например: unwind-коды должны быть похожи на реальный пролог; .xdata обычно лежит в секции .xdata; handler обычно находится в исполняемом коде соответствующего runtime; диапазоны .pdata обычно отсортированы; несколько функций могут ссылаться на похожие или общие данные в отдельных сценариях. Нарушение мягкого ожидания не всегда является ошибкой, но часто является поводом для finding.

Эта разница защищает анализатор от двух крайностей. Первая крайность - слишком доверять всему и пропускать очевидные повреждения. Вторая - считать всё нестандартное вредоносным. Фундаментальный анализ должен быть спокойным: отделять форматную невозможность от необычности, необычность от подозрительности, подозрительность от доказанного нарушения.

Как описывать findings по .xdata так, чтобы это было полезно

Finding (обнаружение/замечание анализатора) должен быть не просто сообщением “bad xdata”. Такое сообщение бесполезно. Оно не объясняет, где ошибка, почему она важна и что с ней делать.

Хороший finding по .xdata должен содержать три слоя. Первый слой - технический факт: например, UnwindInfoAddress = 0x00005230, section = .rdata, available bytes = 2, required bytes for header = 4. Второй слой - правило: UNWIND_INFO header requires at least 4 bytes and must be readable from the image. Третий слой - смысл: запись RUNTIME_FUNCTION не может быть использована для корректного virtual unwind, потому что указанные unwind data обрываются до полного заголовка.

Такой формат особенно важен для BKP и для реального инструмента. Он показывает не только “нашёл проблему”, но и зрелость анализа. Ты не просто ругаешь файл. Ты объясняешь контракт, предъявляешь evidence и связываешь нарушение с последствиями для Windows unwind, debugger support и crash dump analysis.

В теоретической книге это тоже нужно держать в голове: каждая структура интересна не сама по себе, а тем, какие выводы она позволяет делать и какие ошибки она помогает обнаружить.

Почему маленькая структура создаёт большой объём теории

UNWIND_INFO кажется маленькой структурой. Несколько байтов заголовка, массив коротких записей, optional area. Из-за этого возникает соблазн объяснить её на одной странице. Но такая краткость обманчива. Маленький размер структуры не означает маленький смысл.

В этих байтах сходятся несколько больших тем: PE layout, RVA/VA, секции, calling convention, стек, пролог, эпилог, nonvolatile-регистры, обработчики исключений, language-specific runtime, stack walking, crash dumps, JIT/dynamic tables и reverse-engineering validation. Если объяснить только поля, читатель выучит названия. Если объяснить связи, он начнёт понимать механизм.

Поэтому мы и замедляемся. Наша цель - не пробежать документацию, а построить внутреннюю модель. Когда позже появятся UWOP_PUSH_NONVOL, UWOP_ALLOC_SMALL, UWOP_SET_FPREG, __C_specific_handler или RtlVirtualUnwind, они уже не будут отдельными страшными словами. Они лягут на подготовленную карту: .pdata нашла функцию, .xdata дала рецепт кадра, unwind-коды описали пролог, handler flags подключили обработчик, runtime восстановил caller context.

Как первые байты UNWIND_INFO превращаются в смысл

Когда parser доходит до RVA из UnwindInfoAddress, он не должен сразу думать “я сейчас прочитаю красивую структуру”. Он находится в бинарном файле или в mapped image, где есть только байты. Первый байт UNWIND_INFO уже несёт два смысла одновременно: младшие биты кодируют Version (версию формата), старшие биты кодируют Flags (флаги). Это важный момент: структура не всегда состоит из полей, аккуратно лежащих по одному байту. Иногда несколько смыслов упакованы в один байт.

Bit field (битовое поле) - это способ хранить несколько маленьких значений внутри одного машинного числа. Такой способ экономит место и удобен для низкоуровневых форматов, но требует дисциплины чтения. Нельзя взять первый байт и сказать: “это версия”. Нельзя взять его и сказать: “это флаги”. Нужно масками выделить нужные биты. Маска - это числовой фильтр, который оставляет одни биты и отбрасывает другие.

Почему это важно для reverse engineering? Потому что многие ошибки ручного разбора начинаются именно с неверного чтения упакованных полей. Человек видит байт 0x19 и не понимает, что в нём одновременно сидят Version и Flags. В результате он получает невозможную версию или не видит флаг handler-a. Для книги фундаментального уровня важно сразу привыкнуть: бинарный формат - это не таблица Excel. Это плотная упаковка смыслов в байты.

Следующий байт - SizeOfProlog. Он выглядит простым, но тоже опасен для поверхностного чтения. Это не “количество инструкций в прологе”. Это размер в байтах. Одна инструкция x64 может занимать разное количество байт. push rbx короткая, sub rsp, imm32 длиннее, lea rbp, [rsp+20h] ещё имеет свою кодировку. Поэтому SizeOfProlog связывает .xdata не с количеством строк дизассемблера, а с реальным адресным расстоянием внутри .text.

Третий байт - CountOfCodes. Его тоже нельзя читать как “количество действий”. Это число 2-байтовых слотов UNWIND_CODE. Некоторые действия съедают дополнительные слоты. Поэтому parser должен не просто пройти CountOfCodes раз и вывести операции, а понимать внутренний формат каждой операции. Это уже маленький интерпретатор, а не просто принтер структуры.

Четвёртый байт содержит FrameRegister и FrameOffset. Это ещё один пример упаковки двух смыслов в один байт. Если frame register отсутствует, эти поля могут быть нулевыми. Если он есть, они связывают unwind-информацию с выбранным регистром кадра и его положением. Здесь снова видно: четыре начальных байта UNWIND_INFO выглядят маленькими, но в них уже заключено несколько уровней правил чтения.

Little-endian, DWORD alignment и почему “байты рядом” не значит “поля рядом как в тексте”

Windows на x64 работает в little-endian (младший байт первым) представлении чисел. Little-endian означает, что многобайтовое число в памяти хранится от младшего байта к старшему. Например, DWORD 0x00001234 в байтах будет выглядеть как 34 12 00 00. Для человека это сначала странно: в тексте мы пишем старшие разряды слева, а в памяти видим младшие байты первыми.

Для .xdata это важно при чтении handler RVA, chained RUNTIME_FUNCTION и любых дополнительных 16/32-битных значений в UNWIND_CODE. Если parser или человек забывает про little-endian, он переворачивает число. RVA 0x00005230 может превратиться в 0x30520000, и дальше анализ улетит в совершенно другой диапазон. В бинарном анализе это не мелочь, а базовая грамотность.

DWORD alignment (выравнивание по 4 байтам) означает, что некоторые части структуры должны начинаться на адресе, кратном четырём байтам. Это нужно процессору, ABI и формату для предсказуемого чтения. В UNWIND_INFO особенно важно место после массива UNWIND_CODE. Если количество 2-байтовых слотов нечётное, перед optional area появляется padding (заполнитель), чтобы следующий DWORD начался правильно.

Padding (заполнитель) - это байты, которые существуют не ради отдельного смысла, а ради выравнивания. Новичок часто пытается интерпретировать каждый байт как “поле”. Но в бинарных форматах часть байтов может быть служебной прокладкой. Если принять padding за начало handler RVA, то весь хвост записи будет прочитан неверно.

Живой пример: CountOfCodes равен 3. Каждый слот UNWIND_CODE занимает 2 байта, значит массив занимает 6 байт. После 4-байтового заголовка получается 10 байт от начала записи. Чтобы следующий DWORD начался на 4-байтовой границе, нужно выровнять позицию. Parser должен это учесть. Если он сразу прочитает handler RVA на позиции 10, он ошибётся; правильное чтение может начаться после padding.

Почему optional area является самым опасным местом для ручного разбора

Optional area (дополнительная область) опасна не потому, что она сложнее всех математически, а потому что у неё нет одного постоянного смысла. До неё parser идёт по достаточно прямой схеме: заголовок, CountOfCodes, массив слотов, выравнивание. А дальше смысл байтов зависит от Flags. Это как дверь, за которой могут быть разные комнаты.

Если Flags говорят EHANDLER, мы ожидаем handler RVA и данные обработчика. Если Flags говорят UHANDLER, логика похожа, но семантика относится к обработчику раскрутки. Если Flags говорят CHAININFO, те же байты читаются уже как chained RUNTIME_FUNCTION. Если Flags нулевые, optional area может отсутствовать. Поэтому читать хвост без анализа Flags - всё равно что читать карту города, не зная масштаба.

Handler RVA (RVA обработчика) - это адрес не обработчика на диске, а относительный виртуальный адрес кода обработчика в image. Он обычно ведёт к функции runtime-а или к специальному обработчику, который понимает language-specific data. Для MSVC SEH часто встречается __C_specific_handler. Но сам handler RVA ещё не объясняет всю логику try/except. Он только говорит, какой код будет интерпретировать дополнительные данные.

Language-specific data (данные, специфичные для языка) - это слой, где общий механизм Windows передаёт управление знаниям компилятора и runtime-а. Windows не обязана

понимать исходный синтаксис __try, catch или destructors как человек. Она предоставляет механизм: вот исключение, вот кадр, вот handler, вот дополнительные данные. A handler уже знает, как эти данные читать.

Для reverse engineer это означает: после handler RVA может начинаться другой формат, уже не универсальный UNWIND_INFO. Ошибка здесь состоит в том, что человек продолжает читать всё как будто это часть .xdata в узком смысле. На самом деле это уже данные для конкретного обработчика, и их структура зависит от компилятора, языка и типа обработки.

Chained unwind info: когда одна запись ссылается на другую

Chained unwind info (цепочечная unwind-информация) появляется тогда, когда текущая запись не является полностью самостоятельной, а ссылается на другую RUNTIME_FUNCTION-подобную запись. Это не самый первый механизм, который надо понимать новичку, но его нельзя игнорировать в фундаментальной книге, потому что он меняет смысл optional area.

Слово chained (цепочечный) здесь означает “связанный в цепочку”. Вместо того чтобы после UNWIND_CODE лежал handler RVA, в optional area находится структура, похожая на RUNTIME_FUNCTION: начало, конец и адрес unwind-информации связанной функции/записи. Это позволяет переиспользовать или связывать unwind-описания в некоторых ситуациях генерации кода.

Главная опасность chained info для parser-a - принять её за handler. Если инструмент видит ненулевой хвост и автоматически говорит “это обработчик”, он уже ошибся. Сначала нужно посмотреть Flags. Если стоит CHAININFO, смысл хвоста не обработчик, а chained entry. Следовательно, дальнейшая логика валидации другая: нужно проверять поля связанной записи, диапазоны, RVA и согласованность с текущей функцией.

Для ручного разбора chained info неудобна тем, что ломает простую картинку “одна функция - один UNWIND_INFO”. Реальный бинарник может иметь более сложную структуру. Поэтому в книге мы не должны обещать слишком простую модель. Простая модель нужна для старта, но фундаментальная модель должна выдерживать реальные исключения из правил.

Связь .xdata с __C_specific_handler: где заканчивается общий Windows-слой и начинается слой компилятора

__C_specific_handler (специфический C/SEH-обработчик) - это важный пример language-specific handler на Windows x64. Его имя может сбивать с толку: кажется, что это “какой-то обработчик языка C”. На практике он является частью runtime-механизма MSVC для структурированной обработки исключений и связанных конструкций.

Общий Windows-слой умеет найти функцию, прочитать UNWIND_INFO, восстановить caller context и вызвать handler, если метаданные говорят, что handler есть. Но Windows не обязана сама понимать каждую таблицу try/except конкретного компилятора. Поэтому handler получает DispatcherContext и другие данные, а затем интерпретирует language-specific data, лежащие после handler RVA.

Это похоже на разделение труда. Windows говорит: “я доставила исключение к кадру и дала тебе контекст”. __C_specific_handler говорит: “я знаю, какие try-блоки, фильтры и обработчики сгенерировал компилятор для этой функции; сейчас проверю, подходит ли исключение”. Поэтому .xdata является мостом между универсальным механизмом ОС и конкретной языковой семантикой.

Для reverse engineering это очень важно. Наличие EHANDLER и handler RVA ещё не говорит, какой именно __try/__except был в исходнике. Оно говорит, что у функции есть обработчик, а детали нужно искать в данных, которые понимает этот handler. Поэтому анализ не должен останавливаться на фразе “есть handler”. Нужно спрашивать: какой handler, где его данные, какой формат этих данных, можно ли их безопасно разобрать, к каким диапазонам кода они относятся.

Почему .xdata может быть переиспользована или выглядеть не один-к-одному с функциями

В учебной картинке хочется сказать: одна запись RUNTIME_FUNCTION указывает на один UNWIND_INFO, один UNWIND_INFO принадлежит одной функции. Для первого понимания это удобно. Но реальность может быть менее прямой. Оптимизации компилятора, COMDAT folding, funclets, chained info, общие шаблоны прологов и особенности линковки могут приводить к тому, что связи выглядят не как простая таблица “один к одному”.

Поэтому parser не должен строить логику на психологическом ожидании “у каждой функции должен быть уникальный кусок .xdata”. Он должен строить логику на адресах и правилах формата. Если несколько RUNTIME_FUNCTION указывают на один и тот же UNWIND_INFO, это не автоматически ошибка. Это повод понять контекст. Если одна запись указывает внутрь данных другой записи, это уже может быть подозрительно, но тоже требует доказательства.

В фундаментальной модели важно различать invariant (жёсткое обязательное свойство) и heuristic (эвристика, полезное ожидание). “UnwindInfoAddress должен быть корректно читаемым RVA” - ближе к invariant. “Обычно данные лежат в секции с именем .xdata” - эвристика. “Обычно одна функция имеет свой логически отдельный UNWIND_INFO” - тоже эвристика, а не закон природы.

Такое мышление отличает серьёзный reverse engineering от чеклистов. Чеклист быстро говорит “не похоже - плохо”. Теоретически зрелый анализатор говорит: “какое правило нарушено? это правило формата или ожидание компилятора? какие evidence у меня есть? как это влияет на unwind?”.

Практическая ментальная модель чтения .xdata без превращения главы в лабораторную

Хотя эта книга не должна превращаться в практикум, полезно иметь мысленный маршрут чтения .xdata. Не как упражнение “открой hex-editor”, а как внутренний алгоритм, который должен жить в голове reverse engineer-а.

Сначала ты не ищешь .xdata глазами. Ты начинаешь с вопроса: какая RUNTIME_FUNCTION относится к текущему адресу? Это даёт тебе UnwindInfoAddress. Потом ты уточняешь: это RVA. Значит, его нужно перевести в координату чтения. В статическом файле - через секции к file offset. В живом процессе - через ImageBase к VA.

Затем ты читаешь первые четыре байта UNWIND_INFO, но не как “четыре байта подряд”, а как набор упакованных смыслов: Version, Flags, SizeOfProlog, CountOfCodes, FrameRegister, FrameOffset. После этого ты не прыгаешь к handler-у. Ты сначала вычисляешь массив UNWIND_CODE. Потом учитываешь операции, которые занимают дополнительные слоты. Потом проверяешь выравнивание. Только потом смотришь на optional area.

После этого ты возвращаешься к коду функции. Совпадает ли заявленный SizeOfProlog с реальным началом функции? Похожи ли unwind operations на инструкции пролога? Есть ли

handler flags, и если есть, куда указывает handler RVA? Если chained info, согласована ли связанная запись? Этот маршрут не является лабораторией, но он формирует правильное мышление.

Связь .xdata с будущими разделами книги

Этот раздел намеренно не раскрывает все UWOP_* операции подробно. Это будет отдельная тема. Сейчас важно было построить крупную модель: .xdata хранит UNWIND_INFO; UNWIND_INFO описывает, как функция строит кадр; unwind-коды кодируют действия пролога; flags определяют наличие обработчика, termination handler-а или chained info; optional area читается только после понимания flags и выравнивания.

Дальше мы будем увеличивать масштаб. Сначала разберём UNWIND_INFO как самостоятельную структуру: не просто “что там лежит”, а какая идея за ней стоит. Потом отдельно пойдём в пролог, эпилог, UNWIND_CODE и UWOP_* операции. И только после этого RtlVirtualUnwind станет не набором слов из документации, а понятным алгоритмом: взять текущий контекст, прочитать рецепт из .xdata, восстановить caller context.

Если удерживать одну мысль, то такую: .xdata - это место, где Windows x64 хранит не код функции и не имена функций, а формальный рецепт обратного выхода из стекового кадра. Без этого рецепта .pdata знает только “где функция”, но не знает “как из неё выйти”.

Источники технической основы раздела

Техническая база этого раздела опирается на документацию Microsoft по x64 exception handling, где описаны pdata/xdata, RUNTIME_FUNCTION, UNWIND_INFO и роль unwind data для обработки исключений и поддержки отладки; на Microsoft PE Format, где описаны PE image, sections, RVA/VA и data directories; а также на документацию RtlVirtualUnwind и RtlLookupFunctionEntry, которые связывают эти структуры с реальной работой Windows runtime.

3.4. Что такое UNWIND_INFO и его общая идея

К этому месту книги мы уже прошли путь от исключения как события выполнения до PE-образа, от PE-образа до Exception Directory, от Exception Directory до .pdata, от .pdata до RUNTIME_FUNCTION и от RUNTIME_FUNCTION до .xdata. Теперь нужно остановиться на центральной структуре, ради которой все эти переходы вообще имели смысл. Эта структура называется UNWIND_INFO (unwind information, информация раскрутки стека).

UNWIND_INFO (структура информации раскрутки стека) - это компактное описание того, как функция Windows x64 построила свой stack frame (stack frame, кадр стека) и что нужно знать системе, чтобы этот кадр правильно разобрать в обратную сторону. Слово compact (компактное) здесь важно: структура не хранит исходный код функции, не хранит полный дизассемблированный текст, не объясняет бизнес-логику и не знает имён переменных. Она хранит только ту часть правды о функции, которая нужна для unwind (раскрутки стека), exception handling (обработки исключений), stack walking (прохода по стеку) и debugger support (поддержки отладчика).

Если сказать совсем по-человечески, UNWIND_INFO - это инструкция для Windows: “если поток сейчас находится внутри этой функции, то чтобы понять, кто её вызвал, сделай обратные действия к её прологу: верни RSP туда, где он был у вызывающей функции, найди сохранённые nonvolatile registers, восстанови нужные значения и получи адрес возврата”. Но сразу важно не упростить слишком сильно: это не инструкция в смысле машинной команды CPU. Это data structure (структура данных), которую создаёт compiler (компилятор) и linker (компоновщик), а читает Windows runtime (исполнительная среда Windows) и функции вроде RtlVirtualUnwind (виртуальная раскрутка стека).

Почему UNWIND_INFO появляется именно после RUNTIME_FUNCTION

Предыдущая ступень, RUNTIME_FUNCTION (запись функции времени выполнения / запись диапазона функции), решала задачу поиска. У Windows есть ControlPc (control program counter, адрес текущего управления), чаще всего связанный с RIP (instruction pointer, указатель текущей инструкции). По этому адресу система должна понять, в каком диапазоне кода находится поток. RUNTIME_FUNCTION даёт ответ: функция начинается здесь, заканчивается здесь, а её unwind-описание находится там.

Но найти функцию - не значит уметь из неё выйти. Это очень важная мысль. Представь, что ты нашёл в здании кабинет, где находится человек. У тебя есть табличка: кабинет начинается от этой двери и заканчивается этой стеной. Но если тебе нужно восстановить путь, которым человек вошёл, таблички с номером кабинета мало. Нужно знать, что внутри кабинета было переставлено: где лежат ключи, что было снято с полки, какую дверь закрыли, какой проход перекрыли. В машинном коде такой “перестановкой” является изменение stack pointer (указателя стека), сохранение registers (регистров), выделение локального stack space (стекового пространства), установка frame pointer (указателя кадра) и подготовка к вызовам других функций.

RUNTIME_FUNCTION указывает на UNWIND_INFO именно потому, что после ответа “где функция” немедленно возникает ответ “как эта функция меняет состояние выполнения”. Поля BeginAddress (адрес начала), EndAddress (адрес конца) и UnwindInfoAddress (адрес unwind-информации) образуют мост: первые два поля ведут к коду, третье поле ведёт к рецепту обратного восстановления. Без третьего поля запись RUNTIME_FUNCTION была бы почти бесполезна для exception machinery (механизма обработки исключений). Она сказала

бы, где находится функция, но не сказала бы, как пройти выше по стеку.

> Главная связка: .pdata хранит RUNTIME_FUNCTION, RUNTIME_FUNCTION находит функцию по ControlPc, а UNWIND_INFO объясняет, как восстановить caller context этой функции.

Какую проблему решает UNWIND_INFO

Чтобы понять UNWIND_INFO, нужно на минуту забыть про структуры и поля и посмотреть на саму проблему. Поток остановился внутри функции. Остановка могла случиться из-за access violation (нарушения доступа к памяти), breakpoint exception (исключения точки останова), RaiseException (явно поднятого исключения) или из-за того, что отладчик строит call stack (стек вызовов). В любом случае у системы есть текущий context (контекст), то есть значения RIP, RSP, регистров и другой машинной информации.

Система хочет получить caller context (контекст вызывающей функции). Caller (вызывающая функция) - это та функция, которая вызвала текущую. Если текущая функция называется условно B, а её вызвала A, то caller context - это состояние потока, как если бы мы находились в A сразу после возврата из B. Но функция B могла изменить стек и регистры. Значит, текущее состояние не равно состоянию A. Нужно вычислить его.

Здесь и появляется unwind (раскрутка стека). Unwind - это процесс обратного движения по цепочке вызовов. Не обязательно реальный возврат через ret, не обязательно физическое уничтожение кадров прямо сейчас. Часто это virtual unwind (виртуальная раскрутка): система математически вычисляет, каким должен быть предыдущий контекст. Это похоже на то, как следователь восстанавливает события: он не переносится в прошлое, но по следам понимает, что было до текущего момента.

UNWIND_INFO решает именно эту задачу следователя. Она отвечает на вопросы: насколько функция уменьшила RSP; какие nonvolatile registers (nonvolatile registers, сохраняемые регистры) она положила на стек; где именно они лежат; использует ли функция frame register (регистр кадра); есть ли у функции exception handler (обработчик исключения); есть ли chained unwind info (цепочечная unwind-информация); как связаны действия пролога с частично выполненным прологом.

Без UNWIND_INFO Windows пришлось бы угадывать. Можно было бы сканировать стек и искать значения, похожие на return address (адрес возврата), но это ненадёжно. В стеке могут лежать числа, указатели, старые значения, случайные данные, которые выглядят как адреса кода. Табличная модель x64 нужна именно для того, чтобы не гадать по мусору, а использовать контракт, заранее созданный компилятором.

UNWIND_INFO как контракт между компилятором и Windows

Слово contract (контракт) здесь очень точное. Compiler (компилятор) генерирует машинный код функции. Он сам решает, какие регистры использовать, сколько места выделить на стеке, нужен ли frame pointer, как будет выглядеть prolog (пролог функции) и epilog (эпилог функции). Но если функция не является простой leaf function (leaf function, листовая функция) и затрагивает стек/регистры способом, важным для unwind, компилятор обязан оставить Windows описание этих действий. Это описание и становится UNWIND_INFO.

Windows, в свою очередь, не разбирает произвольный пролог как человек-дизассемблер. Она не должна понимать все возможные творческие варианты машинного кода. x64 ABI (application binary interface, двоичный интерфейс приложения) ограничивает форму прологов и эпилогов, а UNWIND_INFO описывает их действия в стандартизированном виде. Благодаря этому runtime не нужно полноценно дизассемблировать всю функцию. Достаточно прочитать

маленькую структуру данных.

Контракт работает так: компилятор имеет свободу генерировать эффективный код, но если этот код должен поддерживать исключения, отладку и stack walking, он должен оставить корректные metadata (метаданные). Metadata - это данные о данных или данные о коде. В нашем случае метаданные описывают не “что функция делает”, а “какие свойства функции нужны системе для раскрутки стека”.

Для reverse engineer это особенно важно. Если код и UNWIND_INFO расходятся, мы видим нарушение контракта. Иногда это просто нестандартная оптимизация или особый компилятор. Иногда это упаковщик или обфускатор. Иногда это повреждение. Иногда это намеренная попытка усложнить stack walking и анализ crash dump. Поэтому UNWIND_INFO - не скучная служебная таблица, а место, где можно сравнивать заявленную структуру функции с реальным прологом в .text.

Почему UNWIND_INFO описывает не всю функцию, а только важные для unwind действия

Новичку легко ожидать, что если система хочет понимать функцию, структура должна описывать всю функцию. Но Windows не пытается понимать функцию как программист. Ей не нужно знать, что функция парсит заголовок, вычисляет сумму, проверяет пароль или строит строку. Для unwind это всё неважно.

Unwind интересуется только теми действиями, которые меняют machine state (машинное состояние) так, что без их обратного учёта нельзя подняться к caller-у. В первую очередь это изменение RSP (stack pointer, указателя стека), сохранение nonvolatile registers (сохраняемых регистров), установка frame register (регистра кадра) и наличие handler data (данных обработчика).

Тело функции может быть огромным. Там могут быть циклы, условия, вызовы, арифметика, работа с памятью. Но если всё это не меняет правила восстановления кадра, UNWIND_INFO это не описывает. Поэтому UNWIND_INFO выглядит маленькой по сравнению с функцией. Это не потому, что она “обрезанная”, а потому что она отвечает на узкий системный вопрос: как отменить эффекты пролога и найти состояние вызывающей функции.

Prolog (пролог) - это начало функции, где она строит кадр. Epilog (эпилог) - конец функции, где она разбирает кадр и возвращается. UNWIND_INFO в основном описывает действия пролога, потому что unwind идёт назад: чтобы разобрать кадр, надо знать, как он был собран. Эпилог тоже важен, но он обрабатывается специальными правилами распознавания допустимых форм эпилога. Позже мы разберём это отдельно; сейчас достаточно понять: пролог создаёт состояние, UNWIND_INFO кодирует это создание, unwind использует кодировку для обратного восстановления.

Что находится внутри UNWIND_INFO в крупном плане

Теперь можно перейти к внутреннему устройству, но не как к сухой таблице полей. Лучше представить UNWIND_INFO как запись, состоящую из трёх смысловых слоёв. Первый слой - header (заголовок), второй слой - unwind code array (массив кодов раскрутки), третий слой - optional tail (необязательный хвост), смысл которого зависит от flags (флагов).

Header (заголовок) - первая часть структуры. Заголовок сообщает общие сведения: version (версия формата), flags (флаги дополнительных возможностей), size of prolog (размер пролога), count of unwind codes (количество слотов unwind-кодов), frame register (регистр кадра) и frame offset (смещение кадра). Это как паспорт записи: сначала parser узнаёт, с каким видом записи

работает, сколько далее нужно читать и есть ли у функции дополнительные особенности.

Unwind code array (массив кодов раскрутки) - это последовательность маленьких записей UNWIND_CODE (код unwind-действия). Каждая такая запись описывает действие пролога, важное для восстановления состояния: например, push nonvolatile register (сохранение сохраняемого регистра через push), stack allocation (выделение стека), save nonvolatile register (сохранение регистра в определённое место кадра), set frame pointer (установка указателя кадра).

Optional tail (необязательный хвост) - часть после массива кодов. Она есть не всегда. Если flags говорят, что у функции есть exception handler или termination handler, здесь будет handler RVA (RVA обработчика) и дальше language-specific data (данные, специфичные для языка/компилятора). Если flags говорят про chained unwind info, хвост будет интерпретироваться как связанная запись RUNTIME_FUNCTION. Если flags не требуют хвоста, его может не быть.

```
UNWIND_INFO, общий смысловой вид:

header:
    Version + Flags
    SizeOfProlog
    CountOfCodes
    FrameRegister + FrameOffset

unwind code array:
    UNWIND_CODE[CountOfCodes]
    возможно дополнительные слоты для некоторых операций

optional tail:
    handler RVA + language-specific data
    или chained RUNTIME_FUNCTION
    или ничего
```

Эта схема не заменяет точный формат, но помогает правильно думать. Parser не может сразу прыгнуть к handler RVA. Он обязан сначала прочитать header, узнать CountOfCodes, пройти массив кодов, учесть выравнивание и только потом интерпретировать хвост по flags. Если нарушить порядок чтения, структура начнёт “съезжать” на байты, и все дальнейшие выводы будут неправильными.

Version: почему поле версии маленькое, но важное

Version (версия) - поле, которое сообщает версию формата UNWIND_INFO. В классическом x64 unwind-формате ожидается версия 1. На первый взгляд это скучная служебная деталь. Но для серьёзного parser-а это не пустяк.

Любой бинарный формат живёт во времени. Сегодня формат один, завтра появляются расширения, новые варианты кодирования, новые ограничения или дополнительные версии. Если parser видит версию, которую он не поддерживает, он не должен делать вид, что всё понял. Он должен аккуратно зафиксировать finding (обнаружение/замечание): “версия не поддерживается” или “версия необычна”.

Здесь важно не путать строгость и полезность. Если инструмент слишком строгий, он может отказываться анализировать всё, что чуть отличается от привычного MSVC. Если слишком мягкий, он будет неправильно интерпретировать данные и выдавать ложную уверенность. Правильная позиция: версия - это часть evidence (доказательной базы). Если версия стандартная, можно продолжать обычный разбор. Если нет, нужно снижать уверенность, ограничивать интерпретацию и явно объяснять пользователю, что именно не

совпадает с ожидаемой моделью.

Flags: маленькое поле, которое меняет смысл хвоста структуры

Flags (флаги) - это набор битов, которые включают дополнительные свойства UNWIND_INFO. Bit (бит) - минимальная единица двоичного значения, 0 или 1. Когда говорят “флаг установлен”, имеют в виду, что конкретный бит равен 1 и тем самым включает определённый режим.

В контексте UNWIND_INFO flags особенно важны потому, что они меняют смысл данных после массива UNWIND_CODE. Без чтения flags нельзя знать, есть ли у функции exception handler (обработчик исключения), termination handler (обработчик завершения / finally-логика) или chained unwind info (цепочечная unwind-информация).

UNW_FLAG_EHANDLER (exception handler flag, флаг обработчика исключений) говорит, что после unwind-кодов находится адрес exception handler-а и дополнительные данные. Exception handler (обработчик исключения) - это код, который может участвовать в принятии решения, обрабатывать ли исключение для данного кадра. На практике handler может быть не “твоим catch-блоком напрямую”, а runtime-функцией вроде __C_specific_handler, которая уже читает дополнительные language-specific data.

UNW_FLAG_UHANDLER (termination handler flag, флаг обработчика завершения) связан с обработчиками, которые должны выполняться при unwind, например с finally-семантикой. Termination handler (обработчик завершения) не обязательно “ловит” исключение в смысле исправления ошибки. Его задача может быть в cleanup (очистке): закрыть ресурс, выполнить finally-блок, привести состояние к порядку при выходе из кадра.

UNW_FLAG_CHAININFO (chained info flag, флаг цепочечной информации) означает, что хвост структуры содержит не handler RVA, а связанную RUNTIME_FUNCTION-подобную запись. Это полностью меняет интерпретацию. Parser, который увидел ненулевой хвост и автоматически решил “это handler”, ошибётся. Сначала flags, потом смысл хвоста. Никогда наоборот.

> Правило чтения: Flags определяют грамматику продолжения структуры. До анализа Flags нельзя уверенно говорить, что находится после массива UNWIND_CODE.

SizeOfProlog: почему важно знать длину входа в функцию

SizeOfProlog (размер пролога) - это размер prolog (пролога функции) в байтах. Prolog - начальный участок функции, в котором она подготавливает свой кадр: сохраняет регистры, выделяет стек, иногда устанавливает frame pointer.

Если исключение происходит в теле функции, обычно можно считать, что пролог уже полностью выполнен. Все действия пролога уже изменили состояние. Но если исключение происходит внутри самого пролога, картина становится частичной. Функция могла уже выполнить push rbx, но ещё не выполнить sub rsp, 80h. Или она уже выделила стек, но ещё не сохранила r12. При unwind нельзя “отменять” действия, которые ещё не произошли.

SizeOfProlog нужен именно для такого точного мышления. Он задаёт область, где ControlPc может находиться в процессе построения кадра. UNWIND_CODE хранит offsets (смещения) действий внутри пролога. Вместе они позволяют определить, какие действия уже были выполнены к моменту остановки.

Живой пример. Пусть пролог выглядит так:

```
push rbx        ; 1. сохранили RBX
push rdi        ; 2. сохранили RDI
sub  rsp, 60h   ; 3. выделили 0x60 байт стека
mov  rbx, rcx   ; дальше начинается обычная работа функции
```

Если исключение возникло после третьей инструкции, кадр уже построен: RBX и RDI лежат на стеке, RSP уменьшен. Если исключение возникло после первой инструкции, сохранён только RBX. Если parser или runtime будет применять все unwind-операции независимо от позиции ControlPc, он восстановит неверный caller context. Поэтому SizeOfProlog и offsets в UNWIND_CODE нужны не для красоты, а для точного восстановления частично построенного кадра.

CountOfCodes: почему это количество слотов, а не просто количество действий

CountOfCodes (количество кодов раскрутки) часто кажется очевидным полем: сколько UNWIND_CODE записей нужно прочитать. Но даже здесь есть тонкость. В формате UNWIND_INFO речь идёт о количестве slots (слотов), то есть 2-байтовых элементов массива, а не обязательно о количестве логических операций.

Некоторые unwind operations (операции раскрутки) помещаются в один слот. Другие требуют дополнительных слотов, потому что им нужно хранить больше информации: большой размер stack allocation (выделения стека), offset (смещение) сохранённого регистра, расширенное значение. Поэтому интерпретатор должен читать operation code (код операции) и знать, сколько слотов занимает конкретная операция.

Это похоже на чтение инструкций переменной длины. Нельзя сказать: “одна строка - одна инструкция одинакового размера”. Нужно знать грамматику. Если parser просто пройдёт CountOfCodes как “один слот = одна операция”, он неверно прочитает последующие элементы и, возможно, сдвинется к optional tail не с той позиции. После такой ошибки handler RVA, chained info и вся валидация станут мусором.

Поэтому фундаментальное правило такое: CountOfCodes говорит, сколько 2-байтовых slots доступно в массиве. А смысл операций и количество потребляемых слотов определяется кодом каждой операции.

FrameRegister и FrameOffset: зачем нужен регистр кадра, если есть RSP

FrameRegister (регистр кадра) - поле, которое указывает, использует ли функция специальный register (регистр) как устойчивую базу кадра. FrameOffset (смещение кадра) уточняет положение этой базы относительно фиксированной части stack allocation.

На старом учебном x86 часто говорят: RBP или EBP - это указатель кадра. На Windows x64 так думать опасно. RBP может использоваться как обычный nonvolatile register, а frame pointer может вообще отсутствовать. Оптимизированный x64-код часто адресует данные относительно RSP или использует другие схемы. Поэтому наличие frame register нужно читать из метаданных и подтверждать кодом, а не предполагать автоматически.

Зачем frame register вообще нужен, если есть RSP? Потому что RSP - рабочий указатель вершины стека, и в некоторых функциях он может быть неудобной базой для адресации. Если функция делает динамические выделения, сложные вызовы или хочет иметь устойчивую точку отсчёта, компилятор может установить frame pointer. Тогда обращения к локальным данным могут быть стабильнее, а unwind получает дополнительную информацию о структуре

кадра.

Для reverse engineer frame register ценен как ориентир, но не как гарантия простоты. Если FrameRegister равен нулю, это не значит, что функция простая. Это значит, что специальный регистр кадра не заявлен в UNWIND_INFO. Если FrameRegister ненулевой, нужно понять, где в прологе он устанавливается, как связан с RSP и как используется в адресации.

UNWIND_CODE как не машинный код, а сжатый язык восстановления

UNWIND_CODE (код действия раскрутки) - это маленькая запись, которая описывает одно действие пролога или часть кодирования такого действия. Очень важно не путать UNWIND_CODE с instruction (машинной инструкцией).

Instruction (машинная инструкция) исполняется процессором. Например, push rbx реально меняет RSP и записывает RBX на стек. UNWIND_CODE не исполняется процессором. Его читает unwind-механизм Windows. Это описание, созданное компилятором: “в прологе было такое действие, и для восстановления состояния нужно учитывать его так-то”.

Можно сказать, что UNWIND_CODE - это semantic compression (смысловое сжатие) пролога. Вместо хранения полного набора инструкций он хранит смысл только тех операций, которые важны для unwind. Если пролог содержит push rbx, UNWIND_CODE говорит не “байт 53 был выполнен”, а “nonvolatile register RBX был сохранён на стеке через push”. Если есть sub rsp, 80h, UNWIND_CODE говорит “стек был выделен на такой размер”.

Именно поэтому сравнение UNWIND_CODE с реальным дизассемблированным прологом - важная задача анализатора. Они не обязаны быть текстуально одинаковыми, потому что это разные уровни описания. Но они должны быть семантически согласованы. Реальный код строит кадр, а UNWIND_INFO должен описывать построение этого кадра так, чтобы Windows могла его разобрать.

UNWIND_INFO и частично выполненный пролог

Одна из причин, по которой UNWIND_INFO выглядит сложнее, чем хотелось бы, - необходимость правильно работать с частично выполненным прологом. Это ситуация, когда исключение или отладочная остановка случились не после построения кадра, а прямо во время его построения.

Представь функцию, где первые инструкции сохраняют регистры и выделяют стек. Процессор может остановиться после первой инструкции, после второй, внутри большой инструкции или на границе следующей. Runtime должен быть аккуратен: если действие уже произошло, его нужно учитывать; если ещё нет - нельзя учитывать.

Именно поэтому UNWIND_CODE содержит offset in prolog (смещение в прологе). Offset связывает действие не только с типом операции, но и с местом в прологе. Это даёт RtlVirtualUnwind возможность понять, какие операции применимы для текущего ControlPc.

Для обычного читателя это может казаться избыточным. Для crash dump analysis это критично. Краш может произойти где угодно: в теле функции, в прологе, в эпилоге, на инструкции доступа к памяти, на сохранении регистра, на probe стека. Хорошая система unwind должна выдерживать такие случаи.

UNWIND_INFO и эпилог: почему выход из функции тоже особый

Epilog (эпилог функции) - это участок в конце функции или перед выходом из ветки, где функция освобождает стек, восстанавливает сохранённые регистры и выполняет ret или допустимый jmp. Эпилог важен потому, что если ControlPc находится внутри эпилога, часть действий обратного разбора кадра уже выполняется самим кодом функции.

На x64 эпилоги имеют ограниченную форму, чтобы runtime мог распознавать их. Это ещё одна часть контракта ABI. Функция не может произвольно написать любой код и ожидать, что Windows правильно поймёт его как эпилог. Если эпилог соответствует допустимым шаблонам, RtlVirtualUnwind может обработать ситуацию корректно.

Здесь UNWIND_INFO и правила эпилога работают вместе. UNWIND_INFO описывает, как кадр был построен. Эпилог - место, где кадр разбирается прямо исполняемым кодом. Runtime должен различать: поток находится в обычном теле функции, в прологе или в эпилоге. От этого зависит, какие действия считать уже выполненными и как получить caller context.

В будущей главе про prolog и epilog мы разберём это глубже. Сейчас важно увидеть общую идею: UNWIND_INFO не живёт отдельно от формы машинного кода. Она предполагает, что код функции соблюдает правила Windows x64 ABI. Если код нарушает форму пролога/эпилога, unwind становится ненадёжным.

Handler flags: когда UNWIND_INFO становится входом в обработку исключений

До сих пор мы говорили о UNWIND_INFO как о рецепте восстановления кадра. Но эта структура также может указывать на обработчики. Именно поэтому она важна не только для stack walking, но и для exception handling.

Если в Flags установлен UNW_FLAG_EHANDLER (флаг exception handler-а) или UNW_FLAG_UHANDLER (флаг termination handler-а), после массива UNWIND_CODE и выравнивания находится handler RVA (relative virtual address обработчика, относительный виртуальный адрес обработчика). Handler RVA - это не file offset и не готовый VA; это RVA внутри image, который нужно интерпретировать относительно ImageBase.

Handler (обработчик) в этой точке не обязательно является твоим пользовательским блоком catch. Часто это runtime-функция, например __C_specific_handler (специфический C/SEH-обработчик), которая получает управление от Windows и читает дополнительные language-specific data. Language-specific data (данные, специфичные для языка/компилятора) - это хвостовые данные, смысл которых определяется не только Windows, но и компилятором/runtime-ом.

Здесь видно разделение труда. Windows знает общий путь: найти функцию, прочитать UNWIND_INFO, восстановить кадр, вызвать handler. Но детали того, какие try-регионы существуют, какие фильтры проверить и какой cleanup выполнить, могут быть описаны в формате компилятора. UNWIND_INFO даёт вход в этот слой, но не заменяет его.

Chained unwind info: когда структура не самостоятельна

Chained unwind info (цепочечная информация раскрутки) - режим, при котором UNWIND_INFO не заканчивается handler RVA, а содержит связанную RUNTIME_FUNCTION-подобную запись. Слово chained означает, что информация связана с другой записью, а не полностью описывает ситуацию самостоятельно.

Для первого понимания можно держать простую модель: RUNTIME_FUNCTION указывает на UNWIND_INFO, UNWIND_INFO описывает кадр. Но реальный формат допускает цепочки.

Это нужно для некоторых случаев генерации кода, разделения диапазонов, funclets или переиспользования информации.

Главное для parser-а: если установлен UNW_FLAG_CHAININFO, хвост структуры нельзя читать как handler RVA. Там другой смысл. Неверная интерпретация даст ложный адрес обработчика, ложные findings и неправильное понимание функции. Поэтому флаги - это не комментарий, а переключатель грамматики структуры.

UNWIND_INFO с точки зрения parser-а

Parser (разборщик) - часть инструмента, которая читает бинарные данные по правилам формата и превращает байты в структурированное представление. Для UNWIND_INFO parser должен быть особенно осторожен, потому что запись имеет переменную длину. Нельзя просто прочитать фиксированные 16 байт и считать задачу решённой.

Надёжный parser движется постепенно. Сначала он получает UnwindInfoAddress из RUNTIME_FUNCTION и помнит, что это RVA. Затем проверяет, что этот RVA можно перевести в допустимое место чтения. Затем читает минимальный header. Затем проверяет Version. Затем читает Flags и SizeOfProlog. Затем по CountOfCodes вычисляет массив UNWIND_CODE. Затем проходит операции и дополнительные слоты. Затем учитывает alignment (выравнивание). Затем по Flags решает, есть ли optional tail и как его читать.

Каждый шаг должен иметь проверки границ. Boundary check (проверка границ) - это проверка, что чтение не выходит за пределы файла, секции, image или заявленного размера. Без boundary checks parser становится уязвимым к повреждённым или злонамеренным PE-файлам. Для static /dynamic это принципиально: инструмент, который анализирует подозрительные бинарники, не должен сам падать от кривой .xdata.

Но parser - это только первый слой. Analyzer (анализатор) идёт дальше. Он не просто говорит “прочитал Version = 1”. Он оценивает согласованность: попадает ли UNWIND_INFO в разумную область, соответствует ли CountOfCodes доступным байтам, не противоречат ли Flags друг другу, указывает ли handler RVA в допустимый executable или readable image range, согласуются ли unwind operations с дизассемблированным прологом.

UNWIND_INFO с точки зрения reverse engineer-а

Reverse engineer (исследователь бинарного кода) смотрит на UNWIND_INFO не так, как Windows. Windows использует структуру, чтобы выполнить работу. Reverse engineer использует её как источник фактов о функции.

Если UNWIND_INFO говорит, что функция выделяет 0x120 байт стека, это подсказка о размере кадра. Если она сохраняет RBX, RDI и R12, это говорит о том, какие nonvolatile-регистры функция использует. Если есть handler flags, функция участвует в exception handling или cleanup-семантике. Если есть frame register, можно ожидать определённую форму адресации кадра. Если unwind-коды не соответствуют прологу, появляется аномалия.

Но важно не превращать подсказки в абсолютные выводы без проверки. UNWIND_INFO описывает только часть поведения функции. Большой stack allocation не обязательно означает “сложная логика”; он может быть буфером, выравниванием, security cookie, временной областью или следствием оптимизации. Наличие handler-а не означает, что исходник обязательно имел красивый try/catch в том виде, как ты его представляешь. Отсутствие frame pointer не означает, что функцию нельзя анализировать.

Правильное мышление такое: UNWIND_INFO даёт формальные, проверяемые факты о машинной структуре функции. Эти факты нужно соединять с disassembly (дизассемблированием), decompilation (декомпиляцией), imports (импортами), strings (строками), control flow graph (графом потока управления) и поведением программы.

Типичные ошибки при понимании UNWIND_INFO

Первая ошибка - думать, что UNWIND_INFO является описанием функции целиком. Нет. Это описание unwind-значимых действий, прежде всего связанных с прологом, стеком, регистрами и обработчиками.

Вторая ошибка - думать, что UNWIND_INFO нужна только для исключений. Нет. Она также важна для отладчиков, stack traces (трасс стека), crash dumps (аварийных дампов) и других механизмов, которым нужно пройти вверх по стеку.

Третья ошибка - читать UnwindInfoAddress как file offset. Это RVA. Ошибка координат ломает весь разбор. Если ты перепутал RVA, VA и file offset, ты можешь читать вообще не UNWIND_INFO, а случайные байты.

Четвёртая ошибка - игнорировать CountOfCodes как количество slots. Нельзя считать, что один слот всегда равен одной операции. Некоторые операции требуют дополнительных слотов.

Пятая ошибка - читать хвост структуры без Flags. Один и тот же участок после unwind-кодов может означать handler RVA, language-specific data, chained RUNTIME_FUNCTION или ничего - в зависимости от флагов и выравнивания.

Шестая ошибка - считать любое отклонение от привычной .xdata автоматически вредоносным. Для анализатора важны уровни уверенности. Есть нарушения формата, есть нестандартные, но допустимые ситуации, есть эвристические подозрения. Их нельзя смешивать в одну корзину.

Связь с будущими разделами: почему дальше нужны пролог, эпилог и UWOP_*

Этот раздел даёт общую идею UNWIND_INFO, но ещё не раскрывает каждую unwind operation (операцию раскрутки) детально. Это будет позже. И это правильно: если сразу кинуть список UWOP_PUSH_NONVOL, UWOP_ALLOC_SMALL, UWOP_ALLOC_LARGE, UWOP_SET_FPREG и других операций, без большой модели они превратятся в справочник.

Сначала нужно понять, зачем вообще существует язык unwind-кодов. Теперь эта основа есть: пролог строит кадр; UNWIND_INFO описывает unwind-значимые действия пролога; RtlVirtualUnwind читает эти данные, чтобы восстановить caller context; flags связывают структуру с обработчиками и chained info; parser должен читать запись осторожно, учитывая RVA, границы, выравнивание и переменную длину.

Дальше можно будет идти глубже. Мы отдельно разберём prolog (пролог) и epilog (эпилог), потому что без них невозможно понять, почему unwind-информация выглядит именно так. Затем разберём UNWIND_CODE и каждую UWOP_* operation не как сухой список, а как отражение конкретного действия машинного кода. Только после этого RtlVirtualUnwind станет полностью прозрачным: мы увидим, как из записей в .pdata/.xdata получается движение по стеку.

Главный итог раздела

UNWIND_INFO (структура информации раскрутки стека) - это не вспомогательная мелочь и не “ещё одна таблица”. Это центральный рецепт восстановления кадра функции в Windows x64. RUNTIME_FUNCTION говорит системе, какая функция содержит текущий ControlPc и где лежит unwind-рецепт. UNWIND_INFO хранит сам рецепт: что функция сделала со стеком и сохраняемыми регистрами, есть ли frame register, есть ли обработчики, есть ли chained info и как читать дополнительные данные.

Для Windows эта структура нужна, чтобы исключения, отладчики и stack walking могли работать надёжно. Для reverse engineer-а она важна как источник проверяемых фактов о функции. Для static /dynamic она становится основой валидации: можно проверять адреса, границы, версии, флаги, размеры, операции, соответствие прологу, корректность handler RVA и общую согласованность PE-образа с контрактом Windows x64 ABI.

Если удерживать одну фразу, то такую: UNWIND_INFO - это не описание того, что функция делает, а описание того, как из функции безопасно выйти назад по стеку.

Почему UNWIND_INFO нельзя понимать как справочную таблицу полей

Самая плохая форма объяснения UNWIND_INFO - это просто показать таблицу полей и сказать: Version здесь, Flags здесь, SizeOfProlog здесь. Формально такая таблица полезна, но она не учит механизму. Человек запоминает названия, но не понимает, зачем они вместе существуют. В хорошей технической книге поле структуры не должно висеть отдельно. Каждое поле должно быть связано с проблемой, которую оно решает.

Например, SizeOfProlog (размер пролога) существует не потому, что Microsoft захотела сохранить длину начала функции. Он существует потому, что исключение может произойти внутри пролога, а значит unwind-механизм должен отличать уже выполненные действия от ещё не выполненных. CountOfCodes (количество слотов unwind-кодов) существует не потому, что массиву нужен размер вообще, а потому что структура переменной длины, и без явного счётчика невозможно понять, где заканчиваются unwind-коды и где начинается дополнительная область. Flags (флаги) существуют не потому, что надо куда-то положить признаки, а потому что хвост структуры имеет разную грамматику: handler, termination handler, chained info или ничего.

То есть UNWIND_INFO - это не набор полей. Это маленький формат данных, который решает сразу несколько задач: описать построение кадра, позволить частично раскручивать пролог, указать на обработчики, поддержать цепочечную информацию, дать runtime-у возможность восстановить caller context и дать отладчикам возможность построить стек. Если объяснять структуру как таблицу, все эти связи пропадают. Если объяснять структуру как механизм, поля становятся естественными.

UNWIND_INFO как карта обратного движения

Обычный машинный код движется вперёд. Function prolog (пролог функции) выполняется сверху вниз: сначала одна инструкция, потом следующая. Например, функция сохраняет регистр, выделяет стек, устанавливает frame pointer и начинает работу. Unwind движется в обратном смысле. Он спрашивает: если эти действия уже были выполнены, как получить состояние до них?

Это похоже на разбор конструкции. Когда строитель собирает леса, он ставит опоры, закрепляет балки, поднимает настил. Чтобы безопасно разобрать леса, нужно понимать порядок и смысл этих действий. Нельзя вытащить нижнюю опору, если на ней держится верх.

В функции роль опор играют RSP, сохранённые nonvolatile-регистры и адрес возврата. UNWIND_INFO говорит, какие элементы конструкции появились в прологе и где они находятся.

Важно: unwind не обязательно буквально выполняет инструкции в обратном порядке. Он использует описание результата. Если пролог сделал push rbx, unwind знает, что старое значение RBX лежит на стеке, а RSP надо сдвинуть так, чтобы перейти к состоянию caller-а. Если пролог сделал sub rsp, 80h, unwind знает, что фиксированная стековая область занимает 0x80 байт. Если был установлен frame register, unwind учитывает, как RSP связан с этим регистром. Это не обратное исполнение кода, а вычисление предыдущего контекста по метаданным.

Callee, caller и почему UNWIND_INFO описывает именно callee

Callee (вызываемая функция) - это функция, в которой мы сейчас находимся или которую вызвали. Caller (вызывающая функция) - это функция, которая сделала call и передала управление callee. Когда Windows выполняет unwind, она обычно начинает с callee context (контекста вызываемой функции) и пытается получить caller context (контекст вызывающей функции).

UNWIND_INFO принадлежит callee. Это принципиально. Запись описывает, что именно текущая функция сделала со своим кадром. Она не описывает напрямую caller. Но, зная изменения callee, можно восстановить состояние caller-а. Если функция B была вызвана из функции A, то UNWIND_INFO функции B говорит, как из состояния внутри B вычислить состояние A.

Здесь легко ошибиться и думать, что unwind-информация хранит стек вызовов. Нет. Она не хранит готовую цепочку A -> B -> C. Она хранит правила для каждого диапазона функции. Цепочка строится динамически: текущий RIP указывает на функцию C; её UNWIND_INFO даёт caller context B; новый RIP попадает в функцию B; её UNWIND_INFO даёт caller context A. Так шаг за шагом появляется stack trace (трасса стека).

```
Текущий контекст внутри C
    прочитать UNWIND_INFO для C
        получить контекст B
            прочитать UNWIND_INFO для B
                получить контекст A
                    прочитать UNWIND_INFO для A
                        ...
```

Эта модель важна для диагностики. Если на одном шаге UNWIND_INFO повреждена, отсутствует или неправильно интерпретирована, вся дальнейшая цепочка может сломаться. Ошибка в одном кадре портит всё, что выше. Поэтому твой static /dynamic должен не просто читать отдельные записи, а понимать, где именно потенциально рвётся цепочка восстановления.

Return address: адрес возврата как центральная точка перехода к caller

Return address (адрес возврата) - это адрес инструкции в caller-е, куда управление должно вернуться после завершения callee. Когда инструкция call вызывает функцию, процессор сохраняет адрес возврата на стеке. Когда функция выполняет ret, процессор берёт этот адрес и помещает его в RIP. Так обычное выполнение возвращается назад.

Во время virtual unwind (виртуальной раскрутки) Windows не обязательно исполняет ret. Но ей нужно получить тот же результат на уровне контекста: понять, каким должен стать RIP caller-a. Обычно это делается через адрес возврата, который находится в стеке после учёта всех изменений текущего кадра.

Почему это не всегда просто? Потому что текущий RSP указывает не обязательно прямо на return address. Функция могла выделить стек, сохранить регистры, использовать frame pointer, иметь промежуточные stack slots. Поэтому надо сначала отменить влияние кадра, а уже затем прочитать или вычислить адрес возврата. UNWIND_INFO объясняет, как добраться до правильного места.

Если return address повреждён, stack walking может пойти по ложному пути. Если UNWIND_INFO неправильна, Windows может искать return address не там. Если parser перепутал размер выделения стека, он тоже окажется не там. Поэтому return address - это не просто значение на стеке; это узловая точка, к которой ведёт весь алгоритм восстановления caller context.

Fixed stack allocation: что значит фиксированное выделение стека

Fixed stack allocation (фиксированное выделение стека) - это заранее известный размер стековой области, который функция резервирует в прологе. Например, инструкция sub rsp, 80h выделяет 0x80 байт. Эти байты могут использоваться под local variables (локальные переменные), temporary values (временные значения), spill slots (места сброса регистров), alignment padding (заполнитель для выравнивания) и подготовку к вызовам других функций.

Почему слово fixed (фиксированное) важно? Потому что такой размер известен компилятору и может быть записан в UNWIND_INFO через операции выделения стека. Если функция выделила фиксированные 0x80 байт, unwind-механизм может прибавить это значение обратно к RSP при восстановлении caller context. Это простая арифметика, но она должна быть точной.

Если функция использует dynamic stack allocation (динамическое выделение стека), например alloca (динамическое выделение памяти на стеке), ситуация сложнее. Тогда один только фиксированный размер не всегда описывает всё поведение. Для таких случаев особенно важны frame pointer и ограничения ABI. Фундаментальная мысль такая: UNWIND_INFO хорошо описывает те эффекты пролога, которые компилятор может представить в разрешённом формате unwind-операций. Код функции должен подчиняться этим правилам, иначе надёжный unwind невозможен.

Nonvolatile registers: почему Windows должна знать, где они сохранены

Nonvolatile registers (сохраняемые регистры) - это регистры, значение которых callee обязана сохранить для caller-a, если хочет ими пользоваться. На Windows x64 к таким регистрам относятся, например, RBX, RBP, RDI, RSI, R12-R15 и некоторые другие по правилам calling convention. Volatile registers (временные / несохраняемые регистры), наоборот, caller не должен ожидать сохранёнными после вызова.

Связь с UNWIND_INFO прямая. Если callee использует nonvolatile register и меняет его значение, она должна где-то сохранить старое значение, обычно в своём стековом кадре. При обычном возврате функция сама восстановит регистр перед ret. Но если произошло исключение и стек раскручивается необычным путём, Windows тоже должна восстановить эти

значения, чтобы caller context был честным.

Представь, что функция A держала важный указатель в RBX. Затем она вызвала B. По ABI функция B обязана вернуть RBX таким, каким получила. Если внутри B произошло исключение и Windows поднимается обратно к A, RBX в восстановленном контексте A должен быть старым RBX, а не временным значением функции B. Поэтому UNWIND_INFO функции B должна сказать, где B сохранила RBX. Иначе восстановленный stack trace или обработчик исключения получит неверный контекст.

Это объясняет, почему unwind-информация так много внимания уделяет nonvolatile-регистрам. Дело не в том, что volatile-регистры не существуют. Дело в ответственности. Volatile-регистр может быть испорчен callee по правилам ABI, поэтому Windows не обязана восстанавливать его как часть caller-сохранности. Nonvolatile-регистр - часть контракта, поэтому его сохранение и восстановление должны быть описаны.

Как UNWIND_INFO связывает байты структуры с реальными инструкциями

В хорошем анализе нельзя рассматривать .xdata отдельно от .text. .text (code section, секция кода) содержит реальные машинные инструкции. .xdata содержит описание unwind-значимых эффектов этих инструкций. Если читать только .xdata, можно понять форму кадра, но не увидеть реальную функцию. Если читать только .text, можно увидеть инструкции, но не понять, как Windows официально будет раскручивать кадр. Сила анализа появляется при сравнении двух слоёв.

Допустим, дизассемблер показывает в начале функции push rbx, push rdi, sub rsp, 40h. UNWIND_INFO должен содержать операции, семантически соответствующие сохранению RBX, сохранению RDI и выделению 0x40 байт стека. Он не обязан хранить текст push rbx. Он хранит смысл для unwind. Но если смысл расходится с кодом, появляется вопрос.

Возможны разные ситуации. Иногда parser ошибся при переводе RVA в file offset и читает не ту UNWIND_INFO. Иногда дизассемблер неверно определил начало функции. Иногда код обфусцирован. Иногда функция имеет chained info. Иногда компилятор сгенерировал необычный, но допустимый пролог. Иногда метаданные действительно повреждены. Поэтому finding должен быть не криком “всё плохо”, а аккуратным утверждением с evidence: что именно прочитано, где, какое правило ожидалось, что не совпало и насколько это критично.

Две правды о функции: правда кода и правда метаданных

У функции на Windows x64 есть как минимум две правды. Первая - code truth (правда кода): реальные байты в .text и реальные инструкции, которые исполнит процессор. Вторая - metadata truth (правда метаданных): то, что .pdata и .xdata заявляют Windows о диапазоне функции и способе unwind.

В нормальном бинарнике эти две правды согласованы. Код строит кадр определённым образом, метаданные описывают этот способ. Но в reverse engineering часто интересны как раз места, где согласованность нарушена или выглядит необычно. Если код и метаданные расходятся, нужно понять, кто врёт, кто неполон, а кто просто описывает другой уровень.

Важно: метаданные не обязаны помогать декомпилятору идеально восстановить исходник. Их задача не в этом. Они обязаны помогать Windows корректно выполнять unwind. Поэтому если декомпилятор на основе своих эвристик определил функцию иначе, чем .pdata, это ещё не значит, что .pdata неправильна. Нужно разбирать конкретный случай. Иногда .pdata даёт более надёжные границы для non-leaf functions, чем эвристика дизассемблера.

Иногда наоборот, нестандартный код может запутать таблицы или использовать dynamic function tables.

Для твоего проекта это ключевая идея: static должен не просто доверять одному источнику. Он должен сопоставлять PE-структуры, секции, .pdata, .xdata, дизассемблированный пролог, границы функций и runtime-ожидания Windows. Только так появляется глубокий анализ, а не распечатка структур.

Почему UNWIND_INFO важна для crash dump analysis

Crash dump analysis (анализ аварийного дампа) - это разбор сохранённого состояния процесса после падения или диагностической остановки. Dump (дамп) может содержать память, регистры, список модулей, стеки потоков и другую информацию. Когда анализатор дампа показывает call stack, он должен пройти по кадрам от места падения вверх. На x64 для этого критически важна unwind-информация.

Если UNWIND_INFO корректна, анализатор может восстановить цепочку вызовов даже без frame pointer в каждом кадре. Если UNWIND_INFO отсутствует или повреждена, stack trace может оборваться или стать ложным. Человек увидит неправильного caller-а, начнёт искать баг не там и потеряет часы. Поэтому для промышленной диагностики корректность unwind-метаданных не академическая тема, а практический фундамент.

Особенно это заметно в оптимизированных release-сборках. Там локальные переменные могут быть оптимизированы, frame pointer часто отсутствует, код может быть переупорядочен, inline-функции могут исчезать как отдельные кадры. В таких условиях unwind metadata остаётся одним из немногих формальных источников для восстановления стека. Поэтому .pdata/.xdata важны не только для компилятора и Windows, но и для человека, который расследует падение.

Почему UNWIND_INFO важна для защит и модификации бинарника

dynamic (защитник бинарника / механизм защиты) может добавлять обёртки, менять прологи, вставлять проверки, переносить код, шифровать участки, создавать trampolines (переходные участки кода) и делать другие преобразования. На x86 некоторые такие операции могли казаться проще, потому что многие инструменты полагались на более эвристическое понимание стека. На Windows x64 любое серьёзное изменение функции должно учитывать unwind-контракт.

Если dynamic добавил push rbx в начало функции, но не обновил UNWIND_INFO, Windows может не знать, что RBX был сохранён и что RSP изменился. Если dynamic вставил дополнительное sub rsp, 20h, но метаданные остались прежними, virtual unwind может искать return address не там. Если dynamic разбил функцию на части, но не создал корректные RUNTIME_FUNCTION диапазоны, stack walking может оборваться.

Это не означает, что защищать x64-бинарники невозможно. Это означает, что защита должна быть unwind-aware (осознающей unwind). Она должна понимать .pdata, .xdata, границы функций, прологи, эпилоги и dynamic function tables. Поэтому книга по твоему проекту не может поверхностно проходить UNWIND_INFO. Это не справочная структура, а одна из центральных точек безопасности преобразований.

Режимы доверия: когда UNWIND_INFO можно считать источником истины

В обычном анализе хочется спросить: можно ли доверять UNWIND_INFO? Ответ не бинарный. Нужно говорить о trust model (модели доверия). В нормальном signed (подписанном) системном модуле Microsoft, собранном стандартным toolchain (набором инструментов компиляции), доверия больше. В упакованном malware sample (образце вредоносного кода), вручную патченом драйвере или неизвестном protect-е доверия меньше. Но даже там формат может быть корректным.

Поэтому инструмент должен различать уровни. Structural validity (структурная корректность) - можно ли прочитать запись по формату? Semantic consistency (семантическая согласованность) - соответствует ли запись реальному прологу? Contextual normality (контекстная нормальность) - похоже ли это на обычный вывод известного компилятора? Runtime usability (используемость runtime-ом) - сможет ли Windows реально использовать это для unwind? Эти уровни связаны, но не равны.

Например, UNWIND_INFO может быть структурно читаемой, но семантически подозрительной. Или лежать не в секции .xdata, но всё ещё быть доступной по корректному RVA. Или указывать на handler в необычной секции. Хороший анализатор не должен сваливать всё в одну ошибку. Он должен объяснять: это нарушение формата, необычная компоновка, эвристическое подозрение или потенциальная runtime-проблема.

UNWIND_INFO и динамические таблицы функций

Dynamic function tables (динамические таблицы функций) - это таблицы RUNTIME_FUNCTION, которые регистрируются во время выполнения, например для JIT-code (just-in-time code, код, сгенерированный во время выполнения) или других динамически созданных исполняемых областей. Они показывают, что модель RUNTIME_FUNCTION/UNWIND_INFO не ограничивается только статической .pdata внутри PE-файла.

Почему это важно уже сейчас? Потому что UNWIND_INFO как идея шире, чем секция .xdata. В обычном PE32+ image статические записи лежат в .pdata и указывают на .xdata. Но если код создаётся во время выполнения, ему тоже нужна unwind-информация, иначе Windows не сможет корректно проходить по стеку через этот код. Для этого существуют механизмы регистрации function tables, которые мы разберём позже.

Для reverse engineer-а это означает: отсутствие статической .pdata для некоторого участка не всегда значит, что unwind невозможен в runtime. Возможно, таблица регистрируется динамически. Но в статическом анализе EXE-файла ты можешь этого не видеть сразу. Поэтому выводы должны учитывать режим анализа: static analysis (статический анализ файла) видит одно, runtime analysis (динамический анализ процесса) может увидеть больше.

Мини-модель: как одна функция превращается в одну запись unwind

Соберём маленькую мысленную модель. Пусть компилятор генерирует функцию, которая вызывает другие функции и использует RBX. Так как RBX - nonvolatile register, функция должна сохранить его старое значение. Кроме того, ей нужно 0x40 байт локального стекового пространства. Пролог может выглядеть примерно так:

```
func:
    push rbx
    sub  rsp, 40h
    ... тело функции ...
    add  rsp, 40h
    pop  rbx
```

```
ret
```

На уровне кода всё понятно: push rbx сохраняет RBX и уменьшает RSP, sub rsp, 40h выделяет стек, эпилог делает обратное. Но Windows не хочет каждый раз анализировать эти инструкции как произвольный текст. Поэтому компилятор создаёт RUNTIME_FUNCTION для диапазона func и UNWIND_INFO, где будет сказано: в прологе был сохранён RBX и было выделено 0x40 байт стека.

Когда исключение произойдёт внутри тела func, RtlVirtualUnwind прочитает UNWIND_INFO и поймёт: чтобы получить caller context, нужно учесть сохранённый RBX и вернуть RSP через выделенную область. Затем будет найден return address, и RIP caller-a станет адресом после call func. Вот так из маленькой структуры данных получается движение по стеку.

Эта модель простая, но она даёт фундамент. Все сложные случаи - большой stack allocation, сохранение нескольких регистров, frame pointer, handler, chained info - являются расширением той же идеи. Функция строит кадр; UNWIND_INFO описывает построение; unwind использует описание для обратного восстановления.

Чем UNWIND_INFO отличается от PDB и почему это важно

PDB (Program Database, база отладочных символов) - файл с отладочной информацией, который может хранить имена функций, строки исходного кода, типы, локальные переменные, inline-информацию и многое другое. UNWIND_INFO иногда ошибочно воспринимают как ещё один вид отладочной информации. Это неточно.

PDB нужен человеку и отладчику для богатого понимания программы. UNWIND_INFO нужен runtime-у Windows для минимально необходимого восстановления стека. PDB может отсутствовать, а UNWIND_INFO всё равно должна быть в бинарнике для non-leaf x64 functions, которым она требуется. Символы могут быть stripped (удалены), имена могут исчезнуть, но unwind-метаданные остаются частью исполняемого образа, потому что без них сама система не сможет надёжно обрабатывать исключения и строить стек.

Это различие важно психологически. Когда ты смотришь на stripped binary (бинарник без символов), не надо думать, что у тебя нет вообще никакой структуры. У тебя может не быть имён, но есть .pdata/.xdata. Они не скажут, что функция называется parse_packet, но скажут, где её диапазон и как она строит кадр. Для reverse engineering это ценный скелет, на который потом навешивается смысл.

Источники технической основы раздела

Техническая база раздела опирается на документацию Microsoft по x64 exception handling, где описаны pdata/xdata, RUNTIME_FUNCTION, структура UNWIND_INFO, unwind codes и флаги; на документацию RtlVirtualUnwind и RtlLookupFunctionEntry, где видно, как эти данные используются runtime-логикой; а также на PE Format, где описаны PE image, sections, RVA/VA и data directories.

Глава 4

Prolog, epilog, UNWIND_CODE и Windows x64 ABI

Теперь metadata связывается с реальными инструкциями функции.
Пролог строит кадр, эпилог разбирает его, UNWIND_CODE описывает действия, важные для восстановления caller context.
Эта глава объясняет, почему unwind metadata нельзя понимать без calling convention, nonvolatile registers and stack alignment.

4.1. Что такое пролог функции и почему он описывается отдельно

Мы подошли к месту, где тема unwind перестаёт быть просто разговором о таблицах и начинает касаться самой формы машинного кода. До этого момента мы уже разобрали, что .pdata (procedure data section, секция процедурных данных) помогает Windows найти RUNTIME_FUNCTION (запись диапазона функции), а .xdata (exception data section, секция данных исключений) хранит UNWIND_INFO (информацию раскрутки стека). Но сама UNWIND_INFO не появилась из воздуха. Она описывает не абстрактную функцию вообще, а конкретные действия, которые функция выполняет при входе. Этот входной участок называется prolog (пролог функции).

Prolog (пролог функции) - это начальная часть машинного кода функции, где функция подготавливает собственный stack frame (кадр стека), сохраняет нужные nonvolatile registers (сохраняемые регистры), выделяет stack space (место на стеке), иногда устанавливает frame pointer (указатель кадра) и выполняет другие действия, без которых тело функции не сможет безопасно работать. Важно сразу убрать школьное упрощение: пролог - это не просто “первые несколько инструкций”. Пролог - это область, где функция создаёт условия для своего дальнейшего выполнения. Он имеет специальный смысл для компилятора, для ABI, для Windows exception handling и для reverse engineer.

ABI (Application Binary Interface, двоичный интерфейс приложения) - это набор правил, по которым уже скомпилированный машинный код взаимодействует с другим машинным кодом. В исходном коде можно написать вызов функции красиво и просто. Но после компиляции остаётся только машинный код, и этому коду нужны строгие договорённости: какие аргументы передаются через RCX, RDX, R8 и R9, какие регистры можно портить, какие нужно сохранить, как выровнять стек, где лежит return address (адрес возврата), кто резервирует shadow space (теневое место для аргументов), как функция должна выходить. Пролог существует внутри этих правил. Он не произвольный “старт функции”, а часть общего договора между компилятором, вызывающим кодом, вызываемой функцией и Windows runtime.

Пролог как момент перехода от чужого состояния к своему

Чтобы понять пролог живо, представь не структуру данных, а сам момент входа в функцию. До инструкции call (вызов функции) поток находился в вызывающей функции, то есть в caller (вызывающем коде). Caller подготовил аргументы, зарезервировал нужную область на стеке по правилам calling convention (соглашения о вызовах), выполнил call, и процессор перенёс управление в начало callee (вызываемой функции). Callee - это функция, которую вызвали. Теперь выполнение уже находится внутри новой функции, но состояние потока всё ещё несёт следы предыдущего уровня: на стеке лежит return address (адрес возврата), регистры содержат переданные аргументы или временные значения, RSP указывает на вершину стека после call, а новый кадр ещё не создан.

В этот момент новая функция находится как бы на пороге. Она уже получила управление, но ещё не построила собственное рабочее пространство. Ей нужно решить, будет ли она вызывать другие функции, нужны ли ей локальные переменные в памяти, будет ли она использовать RBX, RBP, RDI, RSI или R12-R15, нужно ли ей выравнивать стек, понадобится ли большой локальный буфер, будет ли у неё обработчик исключений, нужно ли установить frame pointer. Всё это нельзя делать “когда-нибудь потом” без порядка, потому что Windows

должна уметь понять состояние функции даже если исключение случится почти сразу после входа.

Поэтому пролог - это переходная зона. До него состояние принадлежало caller. После него функция уже живёт в собственном кадре. Внутри пролога этот переход может быть частичным: один регистр уже сохранён, стек ещё не выделен; стек уже выделен, frame pointer ещё не установлен; часть действий выполнена, часть нет. Именно эта промежуточность делает пролог отдельной темой для unwind. Если бы функция мгновенно телепортировалась из “кадра нет” в “кадр полностью готов”, формат был бы проще. Но процессор выполняет инструкции последовательно. А исключение может случиться между ними.

Для обычного программиста на C это почти невидимо. Он думает: “я вошёл в функцию”. Для Windows x64 это не одно мгновение, а последовательность машинных действий. И если система хочет корректно обработать исключение в любой точке, она должна знать, какие действия пролога уже успели изменить контекст, а какие ещё нет.

> Ключевая мысль: пролог - это не декоративное начало функции. Это участок, где callee превращает состояние caller-а в собственный рабочий кадр. Для unwind именно этот переход нужно уметь описывать и частично отменять.

Почему нельзя описывать всю функцию целиком

На первый взгляд можно спросить: почему Windows вообще выделяет пролог отдельно? Почему бы не хранить полное описание функции, от первой инструкции до последней? Ответ упирается в практичность. Полное описание каждой инструкции было бы слишком большим, слишком сложным и ненужным для задачи unwind. Windows не нужно понимать всю бизнес-логику функции. Ей не важно, сравнивает ли функция пароль, парсит ли PE-заголовок, считает ли хэш или рисует окно. Для unwind важна только та часть функции, которая влияет на восстановление caller context (контекста вызывающей функции).

Caller context (контекст вызывающей функции) - это состояние, которое должно получиться, если мы мысленно выйдем из текущей функции на уровень выше. Чтобы его получить, нужно знать не всё. Нужно знать, как изменился RSP (stack pointer, указатель стека), какие nonvolatile registers были сохранены, где они лежат, был ли установлен frame register (регистр кадра), есть ли handler flags (флаги обработчиков), и как найти optional data (дополнительные данные) после массива unwind-кодов. Внутренние вычисления функции, арифметика, проверки условий и обращения к своим структурам для самого unwind обычно не важны.

Пролог удобен потому, что именно в нём сосредоточены действия построения кадра. Компилятор старается оформить начало функции так, чтобы эти действия можно было компактно описать через UNWIND_CODE (коды действий раскрутки). UNWIND_CODE - это не машинные инструкции, а маленькие записи, которые говорят: здесь был сохранён такой-то регистр, здесь был выделен такой-то объём стека, здесь был установлен frame pointer. Формат не пытается пересказать все байты пролога. Он пересказывает только смысл, который нужен для обратного восстановления.

Если бы Windows хранила “полное описание функции”, это превратило бы exception handling в гигантский встроенный дизассемблер и анализатор всех возможных инструкций. Это было бы медленнее, тяжелее, сложнее и менее надёжно. Вместо этого x64-модель делает более строгий ход: компилятор должен строить пролог по допустимым правилам, а затем описывать важные действия этого пролога в .xdata. Runtime не догадывается по всем инструкциям подряд. Он читает заранее подготовленную таблицу.

Это похоже на инструкцию по сборке мебели. Не нужно записывать каждое движение руки мастера. Нужно записать конструктивные операции: поставили боковую стенку, закрепили полку, прикрутили ножки. Для unwind конструктивные операции - это сохранение регистров, выделение стека и установка базы кадра.

Что именно функция обычно делает в прологе

Теперь медленно разберём, какие действия вообще встречаются в прологе. Не как список для зубрёжки, а как набор причин. Каждое действие существует не потому, что компилятор любит сложность, а потому, что функция должна согласовать себя с правилами Windows x64 ABI.

Первое действие - сохранение nonvolatile registers (nonvolatile registers, сохраняемые регистры). На Windows x64 часть регистров считается volatile (изменяемые / временные регистры), а часть nonvolatile (сохраняемые регистры). Volatile register - это регистр, который callee имеет право испортить. Caller знает: если ему нужно сохранить такое значение, он должен сделать это сам. Nonvolatile register - наоборот: если callee хочет использовать такой регистр, она обязана вернуть его caller-у в прежнем виде. Поэтому если функция собирается использовать, например, RBX или R12, она должна сначала сохранить старое значение, обычно на стеке, а перед выходом восстановить.

Почему это связано с прологом? Потому что сохранение nonvolatile-регистров логично делать в начале, до того как функция начнёт использовать их для своей работы. Если функция сначала перезапишет RBX, а потом вспомнит, что должна была сохранить старое значение, будет поздно. Старое значение уже потеряно. Поэтому пролог часто содержит push rbx или mov [rsp+...], rbx. Для unwind это действие важно: при восстановлении caller context нужно вернуть RBX к значению caller-a. Если Windows не знает, где лежит сохранённый RBX, она не сможет корректно восстановить контекст.

Второе действие - выделение fixed stack allocation (фиксированного выделения стека). Функции может понадобиться место под локальные переменные, временные буферы, spill slots (слоты сброса регистров в память), выравнивание, сохранённые значения и подготовку к дальнейшим вызовам. На x64 стек растёт к меньшим адресам, поэтому выделение места обычно выглядит как sub rsp, N, где N - количество байт. Это не “магическое создание переменных”, а простое изменение RSP. Но по соглашению область между новым и старым RSP становится рабочим пространством текущей функции.

Третье действие - установка frame pointer (указателя кадра), если он нужен. На Windows x64 не каждая функция использует RBP как классический frame pointer. Оптимизатор часто обходится без него. Но в некоторых случаях функции удобна стабильная база, от которой можно адресовать локальные данные, особенно при динамических изменениях RSP. Тогда один из nonvolatile-регистров может стать frame register (регистр кадра). В UNWIND_INFO для этого есть поля FrameRegister и FrameOffset. Это не украшение: runtime должен знать, что адресация кадра построена не только вокруг текущего RSP.

Четвёртое действие - stack probing (проверка / пробивание стека) при больших выделениях. Если функция выделяет на стеке большой объём, компилятор может вставить вызов helper-a вроде __chkstk. Смысл stack probing в том, чтобы аккуратно затронуть страницы стека и дать Windows возможность расширить committed stack (зафиксированную область стека) по guard page mechanism (механизму сторожевой страницы), а не перепрыгнуть через защитную страницу одним большим sub rsp. Это отдельная глубокая тема, но для пролога важно понять: большой локальный буфер может сделать пролог сложнее, потому что перед окончательным выделением стека появляется вспомогательная логика.

Пятое действие - подготовка к security mechanisms (механизмам безопасности), например к stack cookie (стековой cookie / защитному значению) в сборках с защитой от переполнения буфера. Stack cookie - это специальное значение, которое помещается в кадр и проверяется перед выходом, чтобы обнаружить повреждение стека. Не каждая такая деталь напрямую кодируется unwind-операциями, но для reverse engineer она важна: в прологе могут появляться инструкции, которые относятся не к “обычной логике функции”, а к инфраструктуре безопасности.

Эти действия могут сочетаться. Один пролог может сохранить несколько регистров, вызвать __chkstk, выделить большой стек, установить frame pointer, записать cookie и только потом перейти к телу функции. Поэтому читать пролог как “пару технических инструкций” нельзя. Это участок, где видны обязанности функции перед ABI, системой исключений, безопасностью и будущими вызовами.

```
; примерный, учебный вид пролога Windows x64 функции
push rbx              ; сохранить nonvolatile RBX
push rsi              ; сохранить nonvolatile RSI
sub  rsp, 80h         ; выделить фиксированное место на стеке
lea  rbp, [rsp+20h]   ; установить базу кадра, если компилятор так решил
mov  rbx, rcx         ; дальше функция уже использует регистры для своей логики
```

Этот пример не надо воспринимать как единственный шаблон. Он нужен, чтобы увидеть смысл: первые инструкции не “делают бизнес-логику”, а создают рабочее состояние функции. Именно поэтому они интересуют UNWIND_INFO.

Почему пролог должен быть описываемым

На Windows x64 мало того, что функция построила кадр. Она должна построить его так, чтобы этот процесс можно было описать. Это очень важная разница. Компилятор не просто генерирует любые удобные инструкции в любом порядке. Для функций, которым нужна unwind-информация, действия пролога должны быть совместимы с форматом UNWIND_INFO.

Описываемость означает, что runtime сможет взять набор UNWIND_CODE и восстановить смысл пролога. Если функция сохранила регистр, это должно соответствовать одной из поддерживаемых unwind operations (операций раскрутки). Если функция выделила стек, размер должен быть закодирован через соответствующую операцию. Если установлен frame register, это должно быть отражено в полях структуры. Формат не является универсальным языком для описания любого произвольного машинного кода. Он описывает строго определённые действия, важные для unwind.

Здесь появляется важное понятие restriction (ограничение). Prolog restrictions (ограничения пролога) - это правила, из-за которых пролог функции на x64 не может быть полностью произвольным, если функция должна корректно участвовать в исключениях и stack walking. Эти ограничения не мешают нормальному коду, потому что компилятор знает их заранее. Но для reverse engineer они дают критерий: если в начале функции происходит что-то, что невозможно согласовать с unwind metadata, это повод смотреть внимательнее.

Например, если .pdata говорит, что функция имеет UNWIND_INFO, а реальный начальный код делает странные переходы, сам модифицирует стек нестандартным способом или сохраняет регистры так, что это не отражено в unwind-кодах, возникает вопрос: это обфускация, hand-written assembly (ручной ассемблер), packer stub (заглушка упаковщика), JIT-код, повреждённый файл, ошибка парсера или просто мы неверно определили начало функции?

Для твоего static /dynamic такая ситуация превращается в finding (находку / диагностическое наблюдение). Но finding должен быть не “мне не нравится пролог”. Он должен опираться на конкретный evidence (доказательный материал): диапазон RUNTIME_FUNCTION, адрес UNWIND_INFO, список unwind-кодов, дизассемблированные первые байты, нарушение инварианта, объяснение риска для stack walking или exception handling.

Пролог и SizeOfProlog: почему размер входа хранится в UNWIND_INFO

В UNWIND_INFO есть поле SizeOfProlog (размер пролога). Это не случайное справочное поле. Оно говорит, сколько байт от начала функции относится к prolog region (области пролога). То есть Windows получает границу: вот здесь функция ещё строит кадр, а после этой области начинается обычное body (тело функции).

Почему это важно? Потому что исключение может возникнуть внутри пролога. Допустим, функция только начала работу. Первая инструкция уже сохранила RBX. Вторая должна была выделить стек, но ещё не выполнилась. Если сейчас нужно выполнить virtual unwind (виртуальную раскрутку), нельзя применять полное описание кадра, будто весь пролог уже выполнен. Иначе система “отменит” действия, которых ещё не было.

SizeOfProlog работает вместе с offsets in UNWIND_CODE (смещениями в кодах раскрутки). Каждый unwind-код связан с местом в прологе, где произошло соответствующее действие. RtlVirtualUnwind может сравнить ControlPc (адрес текущего управления) с этими смещениями и понять, какие операции уже произошли. Это особенно важно для надёжной обработки исключений в самой ранней части функции.

Представь, что пролог имеет длину 12 байт. На байте 1 был push rbx, на байте 2 push rsi, на байте 6 sub rsp, 80h. Если ControlPc указывает на байт 4, значит первые push уже могли повлиять на стек, а sub rsp ещё нет. Если ControlPc указывает на байт 10, вероятно, кадр уже почти готов. Это упрощённое объяснение, но оно показывает смысл: размер пролога и смещения действий превращают пролог из “чёрного ящика” в область с частично отслеживаемым состоянием.

Для reverse engineer поле SizeOfProlog полезно не только как число. Оно задаёт ожидание. Если SizeOfProlog равен 5, а дизассемблер показывает, что сохранение регистров и выделение стека продолжаются первые 40 байт, нужно выяснить, почему. Может быть, граница функции определена неверно. Может быть, анализатор неправильно преобразовал RVA. Может быть, перед нами нестандартный код. Может быть, часть инструкций не участвует в unwind и не обязана попадать в пролог по смыслу формата. Но вопрос обязан возникнуть.

> SizeOfProlog нельзя читать как “примерная длина красивого начала”. Это техническая граница области, в которой unwind-механизм учитывает частично выполненные действия построения кадра.

Пролог и partial execution: почему половина пролога - это отдельная проблема

Partial execution (частичное выполнение) - это ситуация, когда пролог начался, но ещё не завершился. На уровне исходного кода такого почти не видно. Программист думает: функция либо вызвана, либо нет. Но на уровне процессора между “вошли в функцию” и “кадр готов” может быть несколько инструкций, и каждая меняет состояние.

Пусть функция должна сохранить RBX, RDI и выделить 0x60 байт стека. Если исключение происходит в теле функции, всё просто: все три действия уже произошли. Но если исключение происходит после сохранения RBX, но до сохранения RDI, Windows должна восстановить только то, что реально изменилось. Если она начнёт восстанавливать RDI из места, куда он ещё не был сохранён, она прочитает мусор. Если она добавит к RSP выделение стека, которого ещё не было, она сдвинет стек не туда.

Вот почему unwind-информация хранит не только “какие действия есть”, но и “где в прологе они происходят”. Это делает формат сложнее, но без этого он был бы ненадёжным. Исключения не выбирают удобный момент. Отладчик тоже может остановить поток где угодно. Аппаратное исключение может возникнуть на инструкции, которая сама находится в прологе. Механизм должен выдерживать такие точки.

На практике это означает, что анализатор не должен читать UNWIND_INFO как статический список действий без учёта ControlPc. Для полного stack walking в конкретном контексте важно, где находится RIP относительно пролога, тела и эпилога. Одно и то же UNWIND_INFO может применяться по-разному в зависимости от позиции выполнения.

Это одна из причин, почему поверхностный парсер .xdata даёт мало пользы. Просто распечатать “UWOP_PUSH_NONVOL RBX, UWOP_ALLOC_SMALL 0x60” - это только начало. Нужно понимать, как эти операции применяются, когда функция частично вошла, полностью вошла или уже выходит.

Пролог, тело функции и эпилог: три разные зоны смысла

Функцию удобно делить не только на “код”, а на зоны смысла. Первая зона - prolog (пролог), где строится кадр. Вторая зона - body (тело функции), где выполняется основная логика. Третья зона - epilog (эпилог), где кадр разбирается и управление возвращается caller-у. Такое деление важно не для красоты, а для unwind.

Body (тело функции) - это обычная рабочая часть. Здесь функция уже построила кадр и использует его. Если исключение происходит в теле, unwind обычно применяет полное описание построенного кадра: стек выделен, регистры сохранены, frame pointer установлен, если он был нужен. В этой зоне Windows может также проверять, есть ли у функции exception handler, если соответствующие flags установлены.

Epilog (эпилог функции) - это зона выхода. В эпилоге функция сама выполняет действия, обратные прологу: восстанавливает регистры, освобождает стек, делает ret или допустимый jmp. Эпилог на x64 имеет строгие ограничения, потому что runtime должен уметь распознать, что ControlPc находится внутри выхода из функции. В отличие от пролога, отдельное описание каждого эпилога обычно не хранится в том же виде: runtime может сканировать поток инструкций и распознавать допустимые эпилоговые последовательности.

Prolog (пролог) отличается от эпилога тем, что его действия явно кодируются в UNWIND_INFO через UNWIND_CODE. Почему? Потому что пролог строит кадр, и именно его действия нужно знать для обратного восстановления. Эпилог уже сам разбирает кадр, поэтому если поток находится внутри эпилога, механизм должен учитывать, что функция находится на пути выхода, а не в обычном теле.

Для reverse engineering это деление помогает не путаться. Если ты видишь sub rsp, 80h в начале - это почти наверняка часть построения кадра. Если видишь add rsp, 80h перед ret - это эпилоговая логика. Если видишь обращение mov eax, [rcx+10h] где-то в середине - это уже бизнес-логика или работа с данными. Смешивать эти уровни нельзя: одни инструкции описывают инфраструктуру вызова, другие - смысл программы.

Почему пролог не равен “начало функции по дизассемблеру”

Дизассемблер показывает байты как инструкции. Если он нашёл начало функции, он покажет первые инструкции. Очень хочется назвать первые инструкции прологом. Но технически это не всегда одно и то же. Prolog в смысле unwind - это не “всё, что стоит в начале и выглядит служебно”, а именно область, описанная SizeOfProlog и unwind-кодами.

Например, в начале функции могут быть инструкции, которые не влияют на unwind: загрузка константы, проверка аргумента, прыжок на общий блок, security check или часть оптимизированной схемы. Некоторые из них могут находиться рядом с прологом, но не обязательно являются unwind-действиями. И наоборот, действие пролога может быть закодировано не так, как ожидает новичок: сохранение регистра может быть не push, а store в [rsp+offset] после выделения стека.

Поэтому reverse engineer должен читать начало функции в двух слоях. Первый слой - machine code (машинный код): какие инструкции реально стоят в .text. Второй слой - unwind semantics (семантика раскрутки): какие из этих действий отражены в UNWIND_INFO и как Windows будет их использовать. Если эти слои совпадают по смыслу, всё хорошо. Если расходятся, начинается анализ.

Иногда расхождение нормально. Например, инструкция в начале может быть частью проверки безопасности и не требовать unwind-описания. Иногда расхождение тревожное: метаданные говорят, что RBX сохранён через push, а в коде нет такого сохранения; или код выделяет большой стек, а UNWIND_INFO не отражает выделение; или SizeOfProlog заканчивается до фактического завершения построения кадра. В таких случаях нужно проверять адреса, базу образа, правильность дизассемблирования, архитектуру, режим, возможную обфускацию и корректность парсера.

Роль компилятора: почему исходный C не говорит всей правды

В исходном коде ты можешь написать очень простую функцию. Например:

```
int add(int a, int b) {
    return a + b;
}
```

На уровне C кажется, что функции почти нечего делать. И в оптимизированной сборке она действительно может стать leaf function (листовой функцией), которая не вызывает другие функции, не выделяет стек и не сохраняет nonvolatile-регистры. В таком случае полноценная unwind-метаинформация может быть не нужна: такую функцию можно “раскрутить” простым моделированием ret.

Но добавь локальный массив, вызов другой функции, использование nonvolatile-регистра или обработку исключений - и машинная картина изменится. Компилятор должен построить кадр, сохранить то, что обязан сохранить, подготовить стек к вызовам, соблюсти выравнивание, возможно, вставить stack probing и security cookie. Исходник всё ещё может выглядеть невинно, но машинный код уже содержит инфраструктуру.

Compiler (компилятор) - это участник, который переводит исходный код в машинный код и одновременно создаёт сведения, необходимые для взаимодействия с системой. Он знает, какие регистры собирается использовать, сколько места на стеке нужно, будет ли функция вызывать другие функции, нужны ли обработчики. Поэтому именно компилятор формирует пролог и соответствующие unwind-записи. Linker (компоновщик) затем собирает объектные

файлы в PE-образ и размещает .pdata/.xdata в итоговой структуре файла.

Отсюда важное следствие: по исходному коду нельзя надёжно угадать unwind-картину без учёта компилятора, настроек оптимизации, архитектуры и ABI. Для reverse engineer исходника обычно нет, поэтому он читает результат: код, секции, таблицы, метаданные. Пролог - один из мест, где след компилятора особенно заметен.

Пример: маленькая leaf function и отсутствие полноценного пролога

Leaf function (листовая функция) - это функция, которая не вызывает другие функции. В контексте Windows x64 этот термин часто важен ещё и потому, что простая leaf function может не изменять nonvolatile-регистры и не выделять стек. Тогда для unwind её можно обработать проще: достаточно представить, что выполнен обычный ret, то есть взять return address со стека и перейти к caller context.

Рассмотрим учебный вид:

```
; условная маленькая функция
add:
    lea eax, [ecx+edx]
    ret
```

Здесь нет push rbx, нет sub rsp, нет сохранения R12, нет frame pointer. Функция берёт аргументы из регистров, вычисляет результат и возвращается. В такой функции полноценный пролог в нашем смысле почти отсутствует. Это не ошибка и не подозрительность. Это нормальный случай: если функция не строит кадр, нечего подробно описывать.

Но важно не сделать обратную ошибку. Leaf function - не значит “неважная функция”, и не значит “всегда нет .pdata”. В реальных бинарниках могут быть нюансы: настройки компилятора, выравнивание, hotpatching, security instrumentation, исключения, hand-written assembly. Поэтому правило такое: отсутствие сложного пролога нормально только если оно согласуется с поведением функции и метаданными.

Пример: non-leaf function и настоящий пролог

Non-leaf function (нелистовая функция) - это функция, которая вызывает другие функции. Как только функция вызывает кого-то ещё, ей обычно нужно строже соблюдать calling convention: подготовить стек, сохранить нужные значения, обеспечить alignment (выравнивание), иногда держать аргументы или временные данные между вызовами.

Условный пролог такой функции может выглядеть так:

```
worker:
    push rbx
    push rdi
    sub  rsp, 0A0h
    mov  rbx, rcx
    mov  rdi, rdx
    ; дальше идут проверки, вызовы других функций, работа с данными
```

Здесь уже есть что описывать. push rbx и push rdi сохраняют nonvolatile-регистры. sub rsp, 0A0h выделяет фиксированное место на стеке. После этого функция может использовать RBX и RDI для своих целей, зная, что перед выходом восстановит старые значения. Для Windows unwind это означает: чтобы восстановить caller context, нужно знать, что RBX и RDI лежат на стеке, и что RSP был уменьшен на 0xA0 плюс влияние push-инструкций.

Если исключение произошло после завершения пролога, RtlVirtualUnwind должен отменить весь эффект построения кадра. Если исключение произошло после первого push, но до sub rsp, 0A0h, применять полный набор действий нельзя. Поэтому UNWIND_INFO хранит не просто “эта функция сохраняет RBX/RDI и выделяет стек”, а привязывает действия к offsets внутри пролога.

Для static /dynamic это означает, что одна и та же функция анализируется в нескольких плоскостях. По .pdata ты знаешь диапазон функции. По .xdata ты находишь UNWIND_INFO. По UNWIND_CODE ты видишь заявленные действия пролога. По дизассемблеру ты проверяешь, есть ли реальные инструкции, соответствующие этим действиям. По инвариантам ты решаешь, согласована ли картина.

Stack alignment: почему выравнивание стека относится к прологу

Stack alignment (выравнивание стека) - это требование, чтобы RSP имел определённую кратность, обычно 16-byte alignment (выравнивание по 16 байтам) в большинстве областей кода вне пролога и эпилога. Это нужно не для красоты. Некоторые инструкции и соглашения ожидают, что данные и стек будут выровнены. Вызовы функций, использование XMM-регистров и ABI-правила завязаны на предсказуемое выравнивание.

Call сам влияет на стек, потому что кладёт return address. Return address на x64 занимает 8 байт. Поэтому caller и callee вместе должны соблюдать баланс: caller перед вызовом подготавливает стек, call сдвигает его на 8 байт, callee в прологе может дополнительно выровнять или выделить место так, чтобы в теле функции стек соответствовал требованиям.

Новичок часто видит sub rsp, 28h или sub rsp, 38h и удивляется: почему не ровно 20h или 30h? Ответ часто связан с комбинацией shadow space, return address и alignment. Размер выделения стека выбирается не только по локальным переменным. Он должен удовлетворять ABI. Поэтому анализ пролога без понимания выравнивания превращается в угадывание.

Unwind metadata не является учебником по alignment, но она отражает итоговое действие: сколько байт функция вычла из RSP. Reverse engineer должен понимать, что это число включает инфраструктуру кадра, а не только “размер переменных из C”.

Shadow space: почему caller уже подготовил часть стека

Shadow space (теневое пространство / home space) - это область на стеке, которую caller резервирует для первых четырёх регистровых аргументов Windows x64 calling convention. Обычно речь идёт о 32 байтах: четыре слота по 8 байт. Даже если аргументы передаются через RCX, RDX, R8 и R9, caller всё равно резервирует эту область перед вызовом.

Зачем она нужна? Она даёт callee стандартное место, куда можно при необходимости сохранить регистровые аргументы. Это помогает отладке, профилированию, varargs-сценариям и единообразию соглашения о вызовах. Shadow space находится уже в кадре с точки зрения вызова, но важно понимать, кто его резервирует: не callee в своём прологе, а caller перед call.

Это часто ломает понимание новичка. Он смотрит на стек внутри функции и видит область около return address. Кажется, что всё это выделила текущая функция. Но часть пространства могла быть подготовлена caller-ом. Поэтому при чтении пролога нужно различать: что уже было на стеке после call, а что добавила сама функция через push/sub rsp.

Для unwind это различие важно концептуально. UNWIND_INFO описывает действия callee по построению собственного кадра. Оно не обязано заново объяснять всю подготовку caller-a. Если перепутать эти уровни, можно неверно понимать layout (раскладку) стека и ошибиться при ручном восстановлении цепочки вызовов.

Пролог и сохранение nonvolatile-регистров: почему это не просто push

Когда говорят “функция сохраняет регистр”, часто представляют только push rbx. Но сохранение nonvolatile-регистра может выглядеть по-разному. Компилятор может использовать push, а может сначала выделить стек и затем записать регистр в [rsp+offset]. Формат unwind поддерживает разные способы описания таких действий.

Смысл при этом один: caller ожидает, что после возврата nonvolatile-регистр будет иметь прежнее значение. Если callee хочет использовать RBX как рабочий регистр, она должна сохранить старый RBX. Где именно старое значение лежит - это уже вопрос конкретного пролога и UNWIND_CODE.

Для reverse engineer важно читать не только инструкцию, но и ответственность. Если видишь push rbx в прологе, спрашивай: почему функция сохраняет RBX? Вероятно, она будет использовать RBX в теле. Где восстановление? Обычно в эпилоге должен быть pop rbx или эквивалентная загрузка. Отражено ли это в UNWIND_INFO? Если да, картина согласована. Если нет, возможно, ты неправильно определил функцию, попал в данные, дизассемблер ошибся, файл защищён или unwind-метаданные нестандартны.

Нельзя относиться к push rbx как к “мусорной служебной инструкции”. В x64 unwind это маркер обязательства. Он говорит: эта функция временно забирает себе nonvolatile-регистр и обещает вернуть его. UNWIND_INFO фиксирует это обещание для Windows.

Пролог и frame pointer: почему RBP больше не всегда главный

В старых учебных объяснениях стекового кадра часто фигурирует RBP или EBP как главный указатель кадра. На Windows x64 это может ввести в заблуждение. RBP (base pointer register, регистр базового указателя) действительно может быть frame pointer, но не обязан им быть. Он относится к nonvolatile-регистрам и может использоваться компилятором как обычный сохраняемый регистр.

Frame pointer (указатель кадра) - это регистр, который функция устанавливает как устойчивую базу для обращений к своему кадру. Он полезен, когда RSP меняется или когда компилятору нужна стабильная точка отсчёта. Но оптимизированный код часто обходится без frame pointer, адресуя данные через RSP с фиксированными смещениями или используя другие регистры.

В UNWIND_INFO наличие frame register отражается полями FrameRegister и FrameOffset. Если FrameRegister равен нулю, это значит, что специальный frame register не заявлен. Если он ненулевой, parser должен понимать, какой регистр выбран и как FrameOffset влияет на положение базы. Но даже при наличии frame pointer нужно проверять реальный пролог: где он устанавливается, из какого значения, после какого выделения стека.

Для начинающего reverse engineer здесь важна дисциплина: не навязывать бинарнику старую схему “push rbp; mov rbp, rsp” как обязательную. На x64 такой шаблон возможен, но не универсален. Хорошая книга по реверсу должна отучить от ожидания, что каждый кадр имеет красивую рамку через RBP. Реальный x64-код часто экономит регистры и оптимизирует

кадры.

Почему пролог отдельно важен для обработки исключений

Теперь вернёмся к главной теме книги - exception handling (обработка исключений). Когда исключение происходит в функции, Windows должна решить, как подняться к caller-у и где искать обработчик. Если ControlPc находится в теле функции, кадр уже построен. Если ControlPc находится в прологе, кадр ещё строится. Если ControlPc находится в эпилоге, кадр уже разбирается. Эти три состояния нельзя обрабатывать одинаково.

Пролог описывается отдельно именно потому, что он создаёт состояние, которое unwind должен потом уметь понимать. Если во время пролога произошла ошибка, Windows не может просто сказать: “применим полный unwind”. Она должна учитывать частично выполненные действия. Поэтому UNWIND_INFO хранит SizeOfProlog и коды с offsets. Это делает обработку исключений внутри пролога возможной и формальной.

Представь access violation внутри пролога. Например, функция пытается сохранить регистр в область стека, но из-за повреждённого RSP запись приводит к исключению. Это редкий, но принципиально возможный случай. Система должна иметь способ попытаться построить корректный caller context. Без описания пролога она была бы слепа.

Та же логика важна для debugging (отладки). Отладчик может поставить breakpoint в начале функции. Профилировщик может остановить поток прямо в прологе. Crash dump может быть снят в момент, когда кадр ещё не готов. Все эти инструменты опираются на ту же способность: понять, где в жизненном цикле функции находится поток.

Почему пролог важен для ручного реверса

Когда ты открываешь функцию в Ghidra, IDA, Binary Ninja или отладчике, первое желание - сразу смотреть декомпилированную логику. Но для Windows x64 unwind полезно сначала остановиться на прологе. Пролог говорит не о бизнес-задаче функции, а о её физическом устройстве. Он показывает, сложная ли функция, сколько стека ей нужно, какие nonvolatile-регистры она использует, есть ли признаки больших локальных объектов, есть ли frame pointer, похожа ли функция на compiler-generated code (код, сгенерированный компилятором) или на hand-written assembly (ручной ассемблер).

Если функция сохраняет много регистров и выделяет большой стек, это может означать, что она выполняет сложную работу, держит много состояния или вызывает много других функций. Если пролог почти отсутствует, функция может быть маленькой leaf function. Если пролог выглядит странно и не согласуется с UNWIND_INFO, это может быть признаком нестандартного кода. Конечно, такие выводы не абсолютны, но они дают направление анализа.

Особенно важно сравнивать пролог с .xdata. Реальный пролог отвечает на вопрос “что сделал код?”. UNWIND_INFO отвечает на вопрос “что Windows считает сделанным для целей unwind?”. Когда эти две картины совпадают, stack walking будет надёжнее. Когда расходятся, у reverse engineer появляется тема для расследования.

Это расследование должно быть аккуратным. Нельзя сразу кричать “обфускация”. Сначала проверяется база: правильно ли найден ImageBase? правильно ли переведён RVA? верно ли определена граница функции? не перепутана ли архитектура? не попали ли мы в thunk (переходник) или tail call? не находится ли функция в динамической таблице, а не в статической .pdata? Только после этого можно формулировать finding.

Пролог и инварианты для static /dynamic

Invariant (инвариант) - это условие, которое должно сохраняться, если структура и код согласованы. Для анализа прологов инварианты особенно полезны, потому что они превращают субъективное “выглядит странно” в проверяемые правила.

Первый инвариант: действия, заявленные в UNWIND_INFO, должны иметь смысл относительно реального пролога. Если UNWIND_INFO говорит о сохранении RBX, в прологе должна быть соответствующая семантика сохранения RBX. Она не обязана быть буквально одной ожидаемой инструкцией, но должна быть объяснима.

Второй инвариант: SizeOfProlog должен покрывать область, где находятся действия построения кадра, значимые для unwind. Если unwind-действия по смыслу происходят за пределами заявленного пролога, это требует объяснения.

Третий инвариант: изменения RSP, заявленные unwind-метаданными, должны соответствовать фактическому выделению стека. Если код делает sub rsp, 200h, а UNWIND_INFO сообщает только маленькое выделение, caller context может быть восстановлен неверно.

Четвёртый инвариант: сохранение nonvolatile-регистров должно быть симметрично восстановлению в эпилоге. Если функция сохраняет RDI, она должна вернуть его перед выходом. Unwind-метаданные помогают восстановить значение при исключении, а эпилог - при обычном возврате.

Пятый инвариант: если функция имеет handler flags, её UNWIND_INFO должно корректно вести к handler data. Пролог здесь не единственная тема, но он связан с общей корректностью кадра: обработчик исключения имеет смысл только если система может дойти до кадра и правильно его интерпретировать.

Для dynamic эти инварианты могут использоваться как сигналы риска. Для static - как объяснение пользователю, почему конкретный адрес или структура заслуживает внимания. Главное - не выдавать подозрение без evidence. Хороший анализатор должен показывать: адрес функции, байты пролога, запись RUNTIME_FUNCTION, decoded UNWIND_INFO, ожидаемое действие, фактическое действие и вывод.

Как читать пролог в связке с UNWIND_INFO

Практическое чтение пролога начинается не с механического распознавания push/sub, а с построения цепочки. Сначала берётся ControlPc или начало функции. Затем через RUNTIME_FUNCTION определяется диапазон функции и адрес UNWIND_INFO. После этого читается SizeOfProlog, CountOfCodes и массив UNWIND_CODE. Только потом дизассемблируется пролог и сравнивается с заявленными unwind-действиями.

Такой порядок важен. Если начать только с дизассемблера, можно увидеть “похожее” начало и навязать ему неверный смысл. Если начать только с .xdata, можно поверить метаданным без проверки реального кода. Надёжный анализ появляется на пересечении: метаданные говорят одно, код подтверждает или опровергает.

Когда встречается первый UNWIND_CODE, не нужно читать его как сухое поле. Нужно представить действие в коде: регистр был сохранён, стек был выделен, frame pointer был установлен. Потом нужно найти соответствующий участок пролога. Если совпадение есть, модель усиливается. Если нет, модель требует пересмотра.

В хорошем ручном разборе для каждой функции можно вести мысленную таблицу: что функция делает со стеком; какие nonvolatile-регистры сохраняет; где находится return address относительно текущего RSP; какой размер фиксированного кадра; есть ли frame register; есть ли handler; как это заявлено в UNWIND_INFO. Со временем это перестаёт быть таблицей и становится привычкой видеть физику функции.

Граница между технической глубиной и ложной точностью

Есть опасность: чем глубже мы говорим о прологе, тем сильнее хочется сформулировать слишком жёсткие правила. Например: “каждая функция начинается с push rbp” или “sub rsp всегда означает локальные переменные” или “если нет .pdata, функция листовая” или “если есть push rbx, в UNWIND_INFO обязательно будет такой-то код”. В реальных бинарниках такие правила быстро ломаются.

Правильнее мыслить не шаблонами, а контрактами. Контракт говорит: если функция участвует в Windows x64 unwind как non-leaf или как функция с обработчиком/сохранением nonvolatile/выделением стека, её важные действия должны быть описаны в pdata/xdata. Как именно компилятор расположит инструкции, какие оптимизации применит, будет ли использовать push или mov, будет ли frame pointer - это зависит от контекста.

Техническая глубина не в том, чтобы заучить один шаблон пролога. Глубина в том, чтобы понять, какую проблему решает пролог, какие обязательства он создаёт, какие данные его описывают и как проверить согласованность между кодом и метаданными. Тогда новые варианты кода не будут ломать понимание.

Временная шкала пролога: что происходит с потоком по шагам

Чтобы пролог перестал быть набором инструкций, полезно представить его как временную шкалу. Время здесь измеряется не секундами, а шагами исполнения. До вызова функции поток находится в caller-е. Caller подготовил аргументы по Windows x64 calling convention (соглашению о вызовах Windows x64), зарезервировал shadow space (теневое пространство), обеспечил нужное выравнивание стека и выполнил call. Инструкция call положила return address на стек и изменила RIP так, что выполнение началось в callee.

Сразу после call callee уже выполняется, но её собственный кадр ещё не построен. На вершине стека лежит return address. RSP указывает на этот адрес возврата. Регистры RCX, RDX, R8 и R9 могут содержать первые integer/pointer arguments (целочисленные аргументы или аргументы-указатели). XMM0-XMM3 могут содержать первые floating-point arguments (аргументы с плавающей точкой). Но если функция хочет использовать nonvolatile-регистры, она ещё не имеет права просто перезаписать их без сохранения.

Затем начинается пролог. Первая инструкция может сохранить RBX. После неё стек уже изменился: RSP стал меньше, а старое значение RBX лежит на вершине стека. Это уже новое состояние. Вторая инструкция может сохранить RDI. Состояние снова изменилось. Третья инструкция может вычесть из RSP 0x80 байт. Теперь у функции появилось рабочее пространство. Четвёртая инструкция может установить frame pointer. После неё часть локальных данных будет адресоваться не напрямую от RSP, а от выбранного регистра кадра.

Каждый такой шаг меняет не только “картинку стека”, но и правила восстановления. Если поток остановлен после первого шага, старый RBX уже можно восстановить со стека, но RDI ещё не сохранён. Если поток остановлен после выделения стека, нужно учитывать уже и выделенный размер. Если поток остановлен до установки frame pointer, нельзя считать frame

register готовым. Именно поэтому в UNWIND_INFO действия пролога имеют смещения. Смещение говорит не просто “эта операция есть”, а “она стала фактом после такого места в прологе”.

Для чтения crash dump (дампа падения) эта временная шкала не абстракция. Дамп фиксирует один конкретный момент. В этом моменте функция может быть полностью вошедшей, частично вошедшей или уже выходящей. Хороший unwind обязан различать эти состояния. Плохой анализатор, который считает, что вся функция всегда находится в полностью построенном кадре, будет ошибаться на самых неприятных случаях: падениях в начале функции, отладочных остановках и повреждённых стековых сценариях.

```
Состояние потока вокруг вызова:

caller перед call:
    RSP -> подготовленная область caller-а

после call, в самом начале callee:
    RSP -> return address

после push rbx:
    RSP -> saved RBX
           return address ниже

после sub rsp, 80h:
    RSP -> рабочая область callee
           saved RBX
           return address ниже
```

Return address: почему адрес возврата не является частью пролога, но пролог живёт вокруг него

Return address (адрес возврата) - это адрес инструкции в caller-е, куда управление должно вернуться после завершения callee. На x64 обычная инструкция call помещает return address на стек. Это происходит до того, как первая инструкция callee начнёт выполнять пролог. Поэтому адрес возврата не создаётся прологом, но пролог строит кадр вокруг уже существующего адреса возврата.

Это различие важно. Если функция выполняет push rbx, она кладёт сохранённый RBX выше return address, потому что стек растёт к меньшим адресам. Если потом функция делает sub rsp, 80h, рабочая область появляется ещё выше, то есть по меньшим адресам. В итоге return address оказывается не на текущей вершине стека, а где-то глубже относительно RSP. Чтобы выполнить virtual unwind, Windows должна вычислить, где этот return address находится после всех изменений RSP.

Когда функция завершает работу обычным путём, эпилог возвращает RSP так, чтобы ret снова увидел return address на вершине. Но при исключении обычный ret не выполняется. Runtime должен сам вычислить, где return address находится, прочитать его и сделать его новым RIP caller-а в контексте виртуальной раскрутки. Вот почему информация о выделении стека и сохранениях регистров напрямую связана с адресом возврата.

Новичок часто думает: “адрес возврата всегда лежит по [rsp]”. Это верно только в некоторые моменты. Сразу после call - да. После построения кадра - обычно нет. После sub rsp, 80h [rsp] указывает уже на локальную рабочую область функции, а return address находится выше по адресам, за выделенной областью и сохранёнными регистрами. Поэтому без понимания пролога невозможно вручную понять, где лежит настоящий адрес возврата.

Как RtlVirtualUnwind использует смысл пролога

RtlVirtualUnwind (виртуальная раскрутка стека) - это функция Windows, которая по текущему контексту и найденной записи функции вычисляет контекст вызывающей функции. Слово virtual (виртуальная) означает, что она не обязана реально выполнять ret или физически продолжать программу. Она меняет модель контекста: из состояния текущего кадра получает состояние caller-а.

Когда RtlVirtualUnwind получает RUNTIME_FUNCTION и UNWIND_INFO, она не интересуется всей логикой функции. Ей важно, где находится ControlPc относительно функции: в prolog (прологе), в epilog (эпилоге) или в body (теле функции). Если ControlPc в теле, обычно считается, что пролог полностью завершён, и можно применять полное описание кадра. Если ControlPc в прологе, нужно учитывать только уже выполненные действия. Если ControlPc в эпилоге, ситуация особая: функция уже сама начала разбирать кадр, и runtime должен распознать этот процесс.

Это объясняет, почему пролог описывается в xdata. RtlVirtualUnwind должна знать не просто “у функции есть кадр”, а “как именно кадр возникал во времени”. UNWIND_CODE даёт ей язык для этого. Например, UWOP_PUSH_NONVOL (операция сохранения nonvolatile-регистра через push) сообщает, что такой-то регистр был сохранён через стек. UWOP_ALLOC_SMALL или UWOP_ALLOC_LARGE сообщают, сколько стекового пространства выделено. Пока мы ещё не разбираем все UWOP_* подробно, но уже важно видеть их роль: это маленькие записи, которые превращают пролог в обратимо понимаемую процедуру.

Если UNWIND_INFO неверна, RtlVirtualUnwind может восстановить неправильный caller context. Тогда stack trace будет лживым, обработчик исключений может быть найден неверно, crash dump analysis потеряет цепочку вызовов, а отладчик покажет бессмысленные кадры. Поэтому корректность пролога и корректность его описания - это не “занудство формата”, а основа доверия ко всей картине выполнения.

Почему пролог не должен скрывать важные действия от unwind metadata

С точки зрения обычной работы программы функция может выполнить много странных, но допустимых действий. Но если эти действия влияют на восстановление caller context, они не могут быть невидимыми для unwind metadata (метаданных раскрутки). Если функция изменила RSP, сохранила nonvolatile-регистр или установила frame pointer, Windows должна иметь способ узнать об этом. Иначе при исключении система будет работать с неполной картиной.

Например, если hand-written assembly (ручной ассемблерный код) сохраняет R12 в нестандартное место, но не создаёт корректную .xdata, обычный возврат из функции может работать, потому что эпилог сам восстановит R12. Но исключение внутри этой функции может сломать stack walking, потому что runtime не знает, где искать старый R12. Так появляется разница между “код работает при обычном выполнении” и “код корректно участвует в exception handling”.

Это особенно важно для JIT (Just-In-Time compilation, компиляция во время выполнения) и dynamic function tables (динамических таблиц функций). Если код создаётся во время работы процесса, он не лежит заранее в статической .pdata PE-файла. Но если такой код должен корректно участвовать в unwind, его диапазоны и unwind-данные должны быть зарегистрированы через соответствующие механизмы. И снова проблема та же: runtime должен иметь карту кадра.

Для dynamic это даёт отдельный класс проверок. Статический PE может иметь корректные секции, но внутри них могут быть функции с сомнительным соответствием прологов и unwind-описаний. Динамический код может вообще не быть виден в файле, но появляться в процессе. В рамках этой книги мы идём от статического PE, но фундаментальная идея уже одна: любой код, который меняет кадр и должен быть раскручиваемым, обязан дать runtime понятное описание.

MASM-директивы как подсказка: метаданные не обязательно пишутся вручную байтами

Когда программист пишет assembly (ассемблер) под Windows x64 вручную, он тоже должен думать об unwind metadata. В MASM (Microsoft Macro Assembler, ассемблер Microsoft) существуют специальные directives (директивы), которые помогают описать пролог и сгенерировать нужные .pdata/.xdata. Директива - это команда ассемблеру, а не инструкция процессора. Она не обязательно создаёт машинный код сама по себе, но сообщает инструменту, какую служебную информацию нужно сформировать.

Например, ассемблерный код может использовать PROC FRAME, .PUSHREG, .ALLOCSTACK, .SETFRAME, .ENDPROLOG и похожие директивы, чтобы сказать: вот функция с unwind-рамкой, вот сохранён регистр, вот выделен стек, вот установлен frame pointer, вот закончился пролог. Эти директивы помогают инструментам создать корректные записи .pdata и .xdata.

Зачем это знать reverse engineer-у? Чтобы понимать: UNWIND_INFO не всегда возникает только из C/C++ компилятора. Она может быть создана ассемблером по директивам, линкером из объектных файлов, генератором кода, JIT-системой или другим инструментом. Главное не происхождение, а контракт: метаданные должны описывать то, что реально делает код.

Это также объясняет, почему нельзя думать о .xdata как о “побочном мусоре компилятора”. Для hand-written assembly корректные unwind-директивы - сознательная часть написания системно корректного кода. Без них обычное выполнение может выглядеть нормальным, но исключения, отладка и stack walking станут ненадёжными.

Как инструменты могут ошибаться при определении пролога

Disassembler (дизассемблер) - инструмент, который переводит байты машинного кода в текстовые инструкции ассемблера. Decompiler (декомпилятор) пытается подняться выше и восстановить псевдо-C. Оба инструмента полезны, но оба могут ошибаться, особенно когда функция имеет нестандартный вход, несколько путей входа, padding (выравнивающие байты), thunks (переходники), tail calls (хвостовые вызовы), hot/cold splitting (разделение горячего и холодного кода) или обфускацию.

Hot/cold splitting (разделение горячего и холодного кода) - оптимизация, при которой часто исполняемая часть функции и редко исполняемая часть могут быть расположены отдельно, чтобы улучшить кеширование и производительность. Tail call (хвостовой вызов) - ситуация, когда функция вместо обычного call+ret передаёт управление другой функции как финальное действие, часто через jmp. Thunk (переходник) - маленький участок кода, который перенаправляет выполнение или адаптирует вызов. Эти явления могут усложнять простую картину “функция начинается здесь и красиво заканчивается ret”.

Поэтому при анализе пролога нельзя слепо доверять только визуальному началу, которое показал инструмент. Нужно сверять несколько источников: RUNTIME_FUNCTION range (диапазон RUNTIME_FUNCTION), disassembly (дизассемблирование), UNWIND_INFO, section

boundaries (границы секции), символы, если есть PDB, и поведение в отладчике. Чем больше источников согласуются, тем выше доверие.

Если Ghidra показывает функцию с одного адреса, а .pdata говорит о другом диапазоне, это не обязательно значит, что кто-то “неправ”. Возможно, инструмент и unwind-таблица смотрят на разные уровни: один пытается восстановить логическую функцию, другой хранит диапазон, нужный для runtime. Но такое расхождение обязательно нужно понимать, а не игнорировать.

Учебный разбор одного пролога по байтам, адресам и смыслу

Теперь полезно пройти один искусственный пример медленно, как на доске. Пусть функция начинается по RVA 0x1200. RVA (Relative Virtual Address, относительный виртуальный адрес) - это адрес внутри загруженного образа относительно ImageBase. Для unwind важно, что RUNTIME_FUNCTION хранит BeginAddress и EndAddress как RVA, а не как file offset и не как готовые VA. Это означает: если BeginAddress равен 0x1200, функция начинается не “на 0x1200 байте файла”, а на виртуальном смещении 0x1200 внутри image (образа).

Пусть RUNTIME_FUNCTION говорит: BeginAddress = 0x1200, EndAddress = 0x1300, UnwindInfoAddress = 0x3000. Значит, текущая функция занимает полуоткрытый диапазон [0x1200, 0x1300), а её UNWIND_INFO лежит по RVA 0x3000. Полуоткрытый диапазон означает, что адрес 0x1200 входит в функцию, адрес 0x12FF входит, а 0x1300 уже нет. Это удобно для поиска: соседние функции могут соприкасаться концом и началом без пересечения.

Теперь допустим, дизассемблер показывает такой пролог:

```
RVA 1200:  push rbx          ; 1 байт условно
RVA 1201:  push rsi          ; 1 байт условно
RVA 1202:  sub  rsp, 60h     ; несколько байт
RVA 1206:  lea  rbp, [rsp+20h]
RVA 120B:  mov  rbx, rcx
RVA 120E:  ... тело функции ...
```

Не нужно сейчас цепляться к точным длинам инструкций: они зависят от кодировки. Важно увидеть принцип. До RVA 120B функция строит кадр. Она сохраняет RBX, сохраняет RSI, выделяет 0x60 байт стека и устанавливает RBP как frame pointer. После этого начинается обычная логика: например, mov rbx, rcx уже может быть переносом аргумента в сохранённый регистр для долгого использования.

UNWIND_INFO для такой функции должен передать Windows тот же смысл. SizeOfProlog может быть равен условно 0x0B, то есть 11 байт. CountOfCodes укажет, сколько слотов UNWIND_CODE нужно прочитать. Среди unwind-операций будут операции сохранения RBX и RSI, операция выделения стека на 0x60 байт и операция установки frame register, если frame pointer действительно заявлен. Эти операции не обязаны выглядеть в метаданных в том же порядке, в каком ты читаешь строки дизассемблера глазами; формат имеет свои правила сортировки и кодирования. Но семантика должна сходиться.

Теперь представь, что exception (исключение) случилось при ControlPc = ImageBase + 0x1203. Это адрес внутри пролога, примерно после push-инструкций, но вокруг sub rsp. Runtime должен быть осторожен: какие действия уже произошли? Если push rbx и push rsi уже выполнены, их нужно учитывать. Если sub rsp ещё не завершилась, нельзя считать выделение 0x60 байт состоявшимся. Именно такие ситуации объясняют, почему “пролог” в unwind - не красивое слово, а временно чувствительная зона.

Если же exception случилось при ControlPc = ImageBase + 0x1220, то поток уже в теле функции. Пролог завершён, кадр построен, и RtlVirtualUnwind может применять полную

модель кадра. Если exception случилось около конца функции, внутри эпилога, то возникает третья ситуация: часть обратных действий уже делает сам код эпилога. Поэтому unwind-процедура отдельно различает prolog, body и epilog.

Этот пример показывает, как связываются все предыдущие главы. PE даёт ImageBase и RVA. .pdata даёт RUNTIME_FUNCTION. RUNTIME_FUNCTION даёт границы функции и адрес .xdata. .xdata даёт UNWIND_INFO. UNWIND_INFO описывает пролог. Пролог объясняет, как изменилась физика стека. А физика стека нужна, чтобы из текущего контекста получить caller context.

Почему “пролог полностью выполнен” - это предположение, которое нужно проверять

В обычном ручном разборе мы часто упрощаем: если RIP находится где-то глубоко внутри функции, считаем, что пролог выполнен. Обычно это разумно. Но фундаментальная теория должна показывать не только обычный случай, а границы применимости. “Пролог выполнен” - это не свойство функции вообще, а вывод из положения ControlPc относительно SizeOfProlog и формы эпилога.

ControlPc (control program counter, адрес текущего управления) - это адрес, по которому runtime оценивает, где находится поток внутри функции. Если ControlPc находится меньше BeginAddress или больше либо равен EndAddress, он вообще не принадлежит этой RUNTIME_FUNCTION-записи. Если он находится между BeginAddress и BeginAddress+SizeOfProlog, он попадает в область пролога. Если он находится внутри допустимой эпилоговой последовательности, ситуация считается выходом из функции. В остальных случаях он находится в теле.

Это важно для диагностики. Если твой анализатор просто берёт любую запись RUNTIME_FUNCTION и применяет все unwind-коды без учёта ControlPc, он делает не полноценный unwind, а грубую статическую расшифровку. Такая расшифровка полезна для отчёта о структуре, но недостаточна для имитации поведения RtlVirtualUnwind в конкретной точке выполнения.

Поэтому в книге мы различаем два режима. Первый - static decoding (статическое декодирование), когда мы читаем .pdata/.xdata из файла и объясняем, что там заявлено. Второй - context-sensitive unwinding (контекстно-зависимая раскрутка), когда у нас есть конкретный RIP/RSP/регистры и нужно восстановить caller context. Пролог связывает эти режимы: статически он описан таблицей, но динамически применяется в зависимости от положения ControlPc.

Почему пролог важен для отчёта, а не только для внутреннего алгоритма

Если ты делаешь static /dynamic, важно не просто найти проблему, а объяснить её человеку. Отчёт должен быть не “bad unwind info”, а понятная цепочка. Например: функция по RVA 0x1200 имеет RUNTIME_FUNCTION; её UNWIND_INFO заявляет сохранение RBX и выделение 0x60 байт стека; дизассемблированный пролог не содержит соответствующего сохранения RBX в пределах SizeOfProlog; из-за этого RtlVirtualUnwind может восстановить RBX неверно при исключении в теле функции. Вот это уже инженерный отчёт.

Report (отчёт) - это не украшение поверх анализа. В security и reverse engineering отчёт является частью результата. Если он не объясняет механизм, пользователь не поймёт, почему finding важен. Поэтому в книге мы постоянно связываем терминологию с future reporting

(будущей отчётностью): каждый инвариант должен иметь evidence, каждый evidence должен иметь адреса и поля, каждое поле должно быть объяснено.

Например, finding “prolog size mismatch” должен объяснять, что SizeOfProlog (размер пролога) задаёт область, где unwind-коды привязаны к действиям построения кадра. Если фактическое сохранение nonvolatile-регистра происходит за пределами этой области, система может не учитывать его как часть пролога при частичном unwind. Но такой вывод нужно делать осторожно: сначала показать байты, инструкции, метаданные и проверку границ.

Это превращает книгу из справочника в фундамент. Ты не просто знаешь, что есть SizeOfProlog. Ты понимаешь, почему оно существует, как влияет на runtime, как его проверить, как объяснить проблему и как не сделать ложное обвинение.

Что не нужно требовать от пролога

Теория будет неполной, если не сказать, чего от пролога требовать нельзя. Во-первых, пролог не обязан быть одинаковым у всех компиляторов и всех настроек оптимизации. Debug build (отладочная сборка) и release build (оптимизированная сборка) могут иметь разные прологи. Debug build часто сохраняет больше явного состояния, проще для отладчика, может использовать frame pointer чаще. Release build может быть компактнее, агрессивнее использовать регистры и убирать лишнее.

Во-вторых, пролог не обязан красиво соответствовать твоим ожиданиям из учебника по C. Локальная переменная из исходника может вообще не лежать на стеке. Она может быть оптимизирована в регистр, удалена, объединена с другой переменной или существовать только как временное значение. Поэтому нельзя читать размер stack allocation как точную сумму “вот столько байт под переменные”. Это размер рабочей области кадра с учётом ABI, выравнивания, spill slots, временных данных и решений компилятора.

В-третьих, пролог не обязан начинаться с push rbp. На Windows x64 RBP не священный указатель кадра, а один из nonvolatile-регистров. Если frame pointer не нужен, компилятор может его не использовать. Если нужен, он может быть установлен иначе, чем ожидает новичок. Поэтому старое правило “смотри push rbp; mov rbp, rsp” для x64 слишком бедное.

В-четвёртых, пролог не обязан объяснять логику функции. Он объясняет физику входа. Если ты хочешь понять алгоритм, нужно читать тело. Если хочешь понять способность функции участвовать в unwind, начинай с пролога и метаданных.

Мысленная формула пролога

После всех подробностей можно сформулировать одну рабочую формулу, но не как замену объяснению, а как сжатое напоминание. Пролог - это участок, где функция принимает управление от caller-а и создаёт обратимо описанный кадр callee. Слово “обратимо” здесь главное. Кадр должен быть не просто создан, а создан так, чтобы Windows могла восстановить предыдущий контекст.

Если видишь начало функции, задавай себе не один вопрос “что делает код?”, а несколько. Что было со стеком сразу после call? Какие регистры функция обязана сохранить? Какие она реально сохраняет? Сколько стека выделяет? Есть ли frame pointer? Где заканчивается SizeOfProlog? Что говорит UNWIND_INFO? Что подтверждает дизассемблер? Что произойдёт, если исключение случится в середине этого участка?

Такой набор вопросов кажется медленным только сначала. Потом он превращается в автоматическое зрение. Ты начинаешь видеть не просто инструкции, а договор: caller

передал управление, callee построила кадр, .xdata описала построение, runtime сможет при необходимости пройти назад. Это и есть фундаментальное понимание Windows x64 unwind.

Итог раздела

Пролог функции - это начальная область, где функция Windows x64 строит свой рабочий кадр. В этом участке callee принимает управление от caller-а и создаёт состояние, в котором сможет выполнять тело функции: сохраняет nonvolatile-регистры, выделяет стек, при необходимости устанавливает frame pointer, соблюдает alignment, подготавливается к вызовам и иногда выполняет дополнительные служебные действия вроде stack probing или установки security cookie.

Пролог описывается отдельно, потому что именно его действия нужно уметь отменять или учитывать при unwind. UNWIND_INFO не хранит всю функцию, а компактно описывает те действия пролога, которые влияют на восстановление caller context. SizeOfProlog и offsets в UNWIND_CODE позволяют Windows работать даже с частично выполненным прологом. Это критично для exception handling, debugging, crash dump analysis и stack walking.

Для reverse engineer пролог - это вход в физику функции. Он показывает, как функция относится к стеку, какие регистры берёт во временное пользование, насколько сложен её кадр и согласуются ли машинный код и unwind-метаданные. Для static /dynamic пролог становится источником инвариантов и evidence: можно проверять, соответствует ли то, что заявлено в .xdata, тому, что реально происходит в .text.

Следующий естественный шаг - epilog (эпилог функции). Если пролог строит кадр, то эпилог его разбирает. И на Windows x64 эпилог тоже нельзя понимать как “просто конец функции”: он имеет строгие формы, потому что runtime должен уметь распознать выход из функции и корректно продолжить unwind.

4.2. Что такое эпилог функции и почему он тоже важен

После разговора о prolog (пролог функции, начальный служебный участок функции) естественно хочется сказать: если пролог строит stack frame (кадр стека), значит epilog (эпилог функции, завершающий служебный участок функции) просто разбирает этот кадр. В самом общем смысле это верно. Но для Windows x64 этого объяснения мало. Эпилог важен не только как “конец функции”. Он важен как строго распознаваемый участок машинного кода, в котором функция уже начала возвращать управление caller (вызывающей функции), а значит обычные правила обработки исключения внутри тела функции перестают работать так же, как раньше.

Epilog (эпилог функции, завершающий служебный участок функции) - это участок кода, где callee (вызываемая функция) восстанавливает состояние, которое она изменила при входе, освобождает выделенное место на стеке, возвращает сохранённые nonvolatile registers (сохраняемые регистры) и выполняет ret (return instruction, инструкция возврата) или допустимый переход, который завершает текущий кадр. Если пролог - это момент, где функция “въезжает” в свой кадр, то эпилог - момент, где она “выезжает” из него обратно к вызывающему уровню.

Но эту симметрию нельзя понимать механически. Пролог описывается в UNWIND_INFO (структура unwind-информации) через массив UNWIND_CODE (коды действий раскрутки). Эпилог обычно не описывается отдельным списком “эпилоговых кодов”. Вместо этого Windows x64 умеет распознавать эпилог по допустимому шаблону машинных инструкций. Это сделано не случайно. У функции может быть один пролог, но много выходов: ранний возврат при ошибке, обычный возврат в конце, возврат из нескольких веток switch, tail call (хвостовой вызов) в другую функцию. Хранить отдельное описание каждого выхода было бы дорого и неудобно. Поэтому правила x64 требуют, чтобы эпилоги имели ограниченную форму, которую можно проверить сканированием кода вперёд от текущего RIP.

> Ключевая идея этой части: эпилог - это не просто последние строки функции. Для Windows x64 это узнаваемый машинный шаблон выхода из кадра. Если RIP находится внутри эпилога, RtlVirtualUnwind (функция виртуальной раскрутки) должна понимать, что функция уже покидает свой кадр, и продолжать эффект эпилога, а не обрабатывать текущую функцию как обычное тело с возможным handler.

Эпилог как момент, когда функция перестаёт быть “текущей”

Когда поток находится в середине тела функции, всё ещё считается, что текущий stack frame (кадр стека) принадлежит этой функции. У неё есть своё выделенное stack space (место на стеке), её nonvolatile registers могут быть сохранены где-то в кадре, её local variables (локальные переменные) или temporary values (временные значения) могут лежать относительно RSP (stack pointer, указатель стека) или frame pointer (указатель кадра). Если в этот момент происходит exception (исключение), Windows может рассматривать текущую функцию как активную: проверить, есть ли у неё exception handler (обработчик исключений), и, если нужно, применить UNWIND_INFO для перехода к caller context (контексту вызывающей функции).

Но когда выполнение дошло до эпилога, ситуация меняется. Функция уже не собирается продолжать свою обычную работу. Она начала процедуру выхода: возвращает RSP к

прежнему значению, достаёт сохранённые регистры, готовится взять return address (адрес возврата) и передать управление caller. В этот момент кадр частично или полностью разбирается. Если исключение или прерывание происходит здесь, нельзя бездумно считать, что мы всё ещё находимся в обычном теле функции. Часть состояния уже восстановлена, часть стекового пространства уже освобождена, некоторые сохранённые значения могли быть извлечены обратно в регистры.

Именно поэтому x64 exception handling (обработка исключений x64) отдельно говорит о случае, когда RIP (instruction pointer, указатель текущей инструкции) находится в эпилоге. RIP - это адрес инструкции, с которой связано текущее выполнение. Если этот адрес попал в область эпилогового шаблона, Windows делает вывод: control is leaving the function (управление покидает функцию). В такой ситуации для текущей функции не должен вызываться её frame-based exception handler (кадровый обработчик исключений), потому что логически функция уже выходит. Вместо этого unwind-механизм должен продолжить эффект эпилога и вычислить контекст caller-а.

Это тонкий, но очень важный момент. Функция не перестаёт быть функцией только на инструкции ret. Она начинает выходить раньше - с первого корректного действия эпилога. Если RSP уже восстановлен, если регистры уже попнуты, если осталось только выполнить ret, то попытка обработать исключение как будто функция всё ещё в теле дала бы неверную картину. Поэтому эпилог - не “красивый конец”, а отдельный режим интерпретации машинного состояния.

Почему Windows не хранит отдельную таблицу всех эпилогов

Когда мы изучали пролог, было видно, что его действия кодируются через UNWIND_CODE. Если функция сделала push rbx, sub rsp, 80h или сохранила r12 по смещению, эти действия можно описать компактными записями. Возникает естественный вопрос: почему бы так же не описать каждый эпилог? Почему не сделать список: “вот здесь начинается первый эпилог, вот здесь второй, вот третий”?

Ответ связан с количеством выходов из функции и задачей экономии unwind metadata (метаданных раскрутки). Metadata (метаданные) - это данные о данных или данные о коде: они не являются самой бизнес-логикой, но помогают системе понимать код. В нашем случае unwind metadata нужны Windows, отладчику и анализатору, чтобы восстановить стек. Чем больше метаданных требуется на каждую функцию, тем больше размер бинарника и сложнее работа loader-а и runtime-а.

У функции может быть много эпилогов. Оптимизатор может создать отдельный выход для успешного случая, отдельный выход для ошибки, отдельный выход для раннего return, отдельный tail call. Если каждый такой выход описывать как отдельную структуру, формат раздуется. Поэтому Windows x64 выбрала другой путь: эпилоги не перечисляются как отдельные записи, а должны иметь форму, которую можно распознать по коду. RtlVirtualUnwind (функция виртуальной раскрутки) или логика unwinder-а сканирует инструкции вперёд от текущего RIP и проверяет, похож ли текущий участок на допустимый эпилог.

Scanning forward (сканирование вперёд) - это не “дизассемблировать всю функцию и понимать её смысл”. Это ограниченная проверка: начиная от текущей инструкции, можно ли увидеть последовательность восстановления стека и регистров, которая заканчивается допустимым ret или jmp. Именно поэтому правила эпилога строгие. Чем свободнее форма эпилога, тем труднее распознавать его без отдельной таблицы. Windows не хочет решать

задачу полноценного анализа произвольного кода во время unwind. Ей нужен надёжный маленький распознаватель.

> Эпилог на x64 существует как компромисс: пролог подробно описывается в xdata, а эпилог не требует отдельной записи, но обязан выглядеть так, чтобы runtime мог его распознать. Поэтому ограничения на эпилог - это не прихоть компилятора, а часть договора между компилятором и Windows.

Что именно делает эпилог на уровне состояния

Чтобы эпилог стал понятным, нужно смотреть не на строки ассемблера, а на состояние, которое они меняют. Функция во время работы держит собственный кадр. Этот кадр мог включать выделенное место на стеке, сохранённые nonvolatile registers, home area (домашняя область аргументов), временные значения, alignment padding (байты для выравнивания) и иногда frame pointer. Когда функция готовится выйти, она должна вернуть состояние caller-а в тот вид, который caller ожидает после завершения вызова.

Первое действие эпилога часто связано с RSP (stack pointer, указатель стека). Если в прологе было sub rsp, 80h, то где-то на выходе должно появиться обратное действие: add rsp, 80h или lea rsp, [frame_register + offset]. Это не “очистка памяти” в смысле стирания байтов. Стековое пространство освобождается логически: RSP возвращается к прежнему месту, и область, которая только что принадлежала текущему кадру, больше не считается активной частью этого кадра.

Второе действие - восстановление nonvolatile registers (сохраняемых регистров). Если функция использовала RBX, RBP, RDI, RSI или R12-R15 и сохранила их в прологе, она обязана вернуть их значения перед выходом. Иначе caller получит испорченный регистр. Это нарушит ABI (Application Binary Interface, двоичный интерфейс приложения), то есть базовое соглашение о том, какие значения должны пережить вызов функции.

Третье действие - передача управления назад. Обычно это ret (return instruction, инструкция возврата). Ret берёт return address (адрес возврата) со стека и помещает его в RIP. После этого процессор продолжает выполнение в caller-е, сразу после того call, который когда-то вызвал текущую функцию. Иногда вместо ret используется допустимый jmp (jump instruction, инструкция перехода), например tail call. В этом случае функция не возвращается обычным ret, а передаёт управление другой функции так, что текущий кадр уже разобран.

Получается, эпилог можно понимать как обратный мост. Пролог построил мост от caller-а к callee. Эпилог разбирает этот мост и возвращает поток управления на уровень выше. Но мост разбирается по инструкциям, не мгновенно. Поэтому в середине эпилога состояние может быть переходным. Windows должна уметь распознавать это переходное состояние и не применять к нему неправильные правила.

Пример обычного эпилога: add, pop, ret

Самый понятный эпилог выглядит как зеркальное отражение простого пролога. Пусть функция в начале сохранила два регистра и выделила место на стеке:

```
worker:
    push rbx
    push rdi
    sub  rsp, 80h
    ; тело функции
```

На выходе ей нужно сделать обратное. Один из возможных эпилогов:

```
add  rsp, 80h
pop  rdi
pop  rbx
ret
```

Здесь add rsp, 80h возвращает stack pointer (указатель стека) к состоянию до выделения локального пространства. Pop rdi берёт значение со стека, записывает его в RDI и увеличивает RSP на 8 байт. Pop rbx делает то же для RBX. Ret берёт со стека return address и передаёт туда управление. Если всё согласовано, caller получает ровно то состояние, которое он ожидал: его nonvolatile-регистры не испорчены, RSP вернулся к правильному месту, управление пришло на инструкцию после call.

Теперь представим, что exception или interrupt (прерывание) возникли на инструкции pop rdi. Это уже не обычное тело функции. Стековое место под локальные данные освобождено, но регистры ещё не все восстановлены. Если unwinder решит, что находится в теле, он может применить обычные UNWIND_CODE как будто кадр всё ещё полностью собран. Но это неверно: часть эпилога уже выполнена. Поэтому runtime должен понимать, что RIP внутри эпилога, и продолжить эффект эпилога, вычисляя caller context.

Именно здесь становится ясно, почему эпилог обязан иметь распознаваемую форму. Unwinder не может гадать: “может быть, это выход, а может быть, просто случайный add rsp”. Он должен видеть допустимую последовательность, в которой восстановление стека и регистров идёт к ret или допустимому jmp. Если между add rsp и ret вставить произвольную бизнес-логику, распознавание станет ненадёжным. Поэтому такие вставки в эпилоге запрещены правилами x64 unwind.

Почему эпилог - не место для произвольных инструкций

В обычном теле функции компилятор свободен намного больше. Он может переставлять вычисления, загружать данные, сравнивать значения, делать условные переходы, обращаться к памяти, вызывать функции, выполнять оптимизации. Но эпилог - особая область. Его должен распознавать unwinder, поэтому он не может быть произвольным куском логики.

Если в эпилоге после восстановления RSP вставить, например, обращение к локальной переменной, это уже опасно: локальное пространство формально освобождено. Если вставить условный переход, который не является частью допустимого шаблона, сканирование эпилога может перестать работать. Если вставить запись в память между pop и ret, runtime не обязан понимать, что это всё ещё эпилог. Системе нужен простой ответ: текущий RIP внутри эпилога или нет. Для этого эпилог должен быть похож на эпилог, а не на произвольный фрагмент тела.

Allowed epilog form (допустимая форма эпилога) - это не эстетическое правило. Это условие корректной работы exception handling (обработки исключений), stack walking (ходьбы по стеку), debugging (отладки) и crash dump analysis (анализа аварийных дампов). Если компилятор или hand-written assembly (рукописный ассемблер) нарушает это правило, Windows может не суметь правильно восстановить caller context в редких, но критичных моментах.

Для reverse engineer это даёт практический критерий. Если ты видишь код, который выглядит как выход из функции, но содержит странные инструкции внутри “эпилога”, нужно спросить: это действительно эпилог? Или дизассемблер неправильно определил границы?

Или это обфускация? Или это hand-written code, который рассчитывает на то, что исключений здесь не будет? Или перед нами нестандартная схема, которую static /dynamic должен отметить как risk finding (рискованную находку)?

Epilog markers: как unwind понимает, что выход действительно заканчивается

Epilog marker (маркер эпилога) - это инструкция или форма инструкции, которая показывает, что эпилог действительно завершает кадр. В обычном случае таким маркером является ret. Ret (return instruction, инструкция возврата) читает адрес возврата со стека и передаёт туда управление. Для unwinder-а это естественная точка: если последовательность восстановления приводит к ret, она похожа на эпилог.

Но x64 допускает не только простой ret. Возможны формы ret n, rep ret и некоторые jmp-варианты, если они соответствуют правилам. Jmp (jump instruction, инструкция перехода) в эпилоге обычно связан с tail call (хвостовым вызовом), когда функция вместо того, чтобы вернуться caller-у и затем вызвать следующую функцию, сразу передаёт управление другой функции после разборки своего кадра. Это оптимизация: один кадр не сохраняется лишний раз, цепочка вызовов становится короче.

Ret n (возврат с дополнительной очисткой стека) чаще ассоциируется с соглашениями, где callee очищает аргументы, но в контексте x64 важно не механически переносить старые x86 ожидания. Для нас главное: документация перечисляет допустимые эпилоговые маркеры, и unwinder ориентируется на ограниченный набор форм. Если конец выглядит иначе, это может не распознаваться как эпилог.

Marker (маркер) здесь не значит “служебный байт в таблице”. Это не поле в .pdata и не отдельная запись в .xdata. Это свойство машинного кода: инструкция в потоке кода, которую runtime умеет распознать как завершение эпилогового шаблона. Поэтому эпилог находится в .text (секции кода), а не в .xdata. .xdata описывает прологовые действия и общие свойства функции, а эпилог распознаётся в самом коде.

> Важно не путать: UNWIND_CODE описывает действия пролога; epilog marker распознаётся в машинном коде. У эпилога нет отдельного массива “EPILOG_CODE” в обычной pre-V3 модели. Поэтому анализ эпилога всегда соединяет две вещи: metadata из .xdata и фактические инструкции в .text.

Ret: почему одна инструкция возврата несёт столько смысла

Ret кажется простой инструкцией, но в теме stack frame она центральная. Return address (адрес возврата) был помещён на стек инструкцией call. Когда callee заканчивает работу, ret берёт этот адрес и загружает его в RIP. После этого поток продолжает выполнение в caller-е.

Если стек в порядке, ret почти невидим. Программа просто возвращается. Но если стек повреждён, ret превращается в опаснейшую инструкцию: она возьмёт не тот адрес и передаст управление неизвестно куда. Именно поэтому corruption of return address (повреждение адреса возврата) исторически связано с exploit development (разработкой эксплойтов). Но даже без темы эксплуатации для reverse engineering важно понимать: ret доверяет стеку. Он не проверяет “а это точно адрес возврата?” Он берёт то, что лежит на вершине стека.

В эпилоге перед ret функция должна привести RSP к такому состоянию, чтобы на вершине стека оказался именно return address. Если RSP будет на 8 байт выше или ниже, ret прочитает неправильное значение. Поэтому add rsp, pops и другие действия эпилога должны точно соответствовать тому, что было сделано в прологе. Здесь нет места приблизительности. Один неправильный размер выделения - и возврат ломается.

Для unwind ret важен ещё и как конечная точка распознавания. Если RtlVirtualUnwind видит, что текущий код находится в допустимой последовательности, ведущей к ret, он может рассматривать выполнение как выход из функции. Если ret нет, а последовательность заканчивается непонятно чем, это не нормальный эпилог в смысле правил.

Tail call: почему эпилог может закончиться jmp

Tail call (хвостовой вызов) - это ситуация, когда функция заканчивает свою работу вызовом другой функции и не нуждается в возврате обратно в своё тело. В исходном коде это может выглядеть как return other(args). Если после вызова other текущей функции больше нечего делать, компилятор может оптимизировать схему: вместо call other; ret сделать разборку текущего кадра и jmp other.

Почему это эпилог? Потому что текущая функция уже убрала свой кадр. Она не собирается возвращаться в себя после other. Она передаёт управление дальше так, будто caller текущей функции теперь будет caller-ом следующей. Это меняет вид stack trace (трассировки стека): хвостовой вызов может убрать промежуточный кадр из обычной цепочки, и это иногда удивляет при отладке.

Jmp в эпилоге должен соответствовать допустимым формам. Не любой jmp можно считать эпилоговым. Если внутри функции есть обычный jmp на локальную метку, это просто control flow (поток управления) внутри тела. Эпилоговый jmp завершает функцию. Он появляется после восстановления стека и регистров, когда кадр уже разобран. Поэтому для анализа важно смотреть не только на сам jmp, но и на контекст перед ним: восстановлен ли RSP, возвращены ли nonvolatile-регистры, находится ли цель вне текущей функции в допустимом смысле.

Для static /dynamic tail call важен как источник ложных выводов. Если анализатор считает, что каждая функция заканчивается только ret, он пропустит часть корректных эпилогов. Если он считает любой jmp в конце эпилогом, он получит ложные срабатывания. Нужна аккуратная модель: epilog marker может быть jmp, но только в рамках правил Windows x64.

Множественные эпилоги: почему у функции может быть много выходов

В учебных примерах функция часто имеет один вход и один выход. В реальном оптимизированном коде это редкость. Multiple epilogs (множественные эпилоги) - это обычная ситуация, когда у функции несколько участков выхода. Например, функция проверяет аргумент и сразу возвращает ошибку, если он нулевой. Потом проверяет второе условие и тоже может выйти. Потом идёт основной путь, который выходит в конце. Каждый такой return может иметь собственный эпилог или прыгать в общий epilog block (общий блок эпилога).

Компилятор выбирает форму по оптимизации размера и скорости. Иногда выгодно сделать один общий блок выхода: разные ветки прыгают туда, и там восстанавливаются

регистры и выполняется ret. Иногда выгодно продублировать короткий эпилог в нескольких местах, чтобы избежать лишнего jmp. Для UNWIND_INFO это удобно: не надо перечислять каждый эпилог. Главное, чтобы каждый эпилоговый участок соответствовал допустимому шаблону.

Для reverse engineer множественные эпилоги важны по двум причинам. Во-первых, они помогают понять структуру функции: где ранние выходы, где ошибки, где нормальный путь. Во-вторых, они показывают, что конец функции не всегда один. Если дизассемблер нарисовал несколько returns, это не обязательно странность. Это нормальный результат оптимизации. Но если один из returns не восстанавливает сохранённый регистр, а другой восстанавливает, нужно внимательнее смотреть: может быть, часть кода недостижима, может быть, функция имеет несколько вариантов кадра, может быть, дизассемблер ошибся, а может быть, metadata и код действительно конфликтуют.

Здесь снова видно отличие фундаментального анализа от поверхностного. Поверхностно можно сказать: “эпилог - конец функции”. Фундаментально надо сказать: “функция может иметь несколько формально распознаваемых выходных участков, каждый из которых должен привести состояние к caller context, и каждый должен быть совместим с правилами x64 unwinding”.

Partial execution: что если исключение произошло внутри эпилога

Partial execution (частичное выполнение) - это ситуация, когда часть инструкций участка уже выполнена, а часть ещё нет. Для пролога мы уже видели проблему: если исключение произошло после push rbx, но до sub rsp, полная раскрутка тела функции была бы неправильной. Для эпилога проблема зеркальная: если выполнение уже прошло add rsp, но ещё не дошло до ret, кадр уже частично разобран.

Представим эпилог:

```
add  rsp, 80h
pop  rdi
pop  rbx
ret
```

Если RIP указывает на add rsp, 80h, действие ещё не выполнено. Если RIP указывает на pop rdi, add уже выполнен. Если RIP указывает на pop rbx, add и pop rdi уже выполнены. Если RIP указывает на ret, почти всё уже восстановлено, осталось взять return address. В каждом случае текущее состояние разное. Но все эти случаи находятся внутри единого процесса выхода.

RtlVirtualUnwind должен учитывать это. В случае эпилога логика не такая же, как в теле: не нужно вызывать handler текущей функции, потому что функция уже выходит. Нужно продолжить вычисление эффекта эпилога. Фраза “continue the effect of the epilog” означает: довести логическую разборку кадра до того состояния, которое было бы после завершения эпилога, чтобы получить caller context.

Это важнее, чем кажется. Исключение внутри эпилога может быть редким, но unwinder нужен не только для обычных исключений. Прерывания, отладочные события, профилирование, сэмплирование, дампы - все эти механизмы могут встретить поток в любой инструкции, включая эпилог. Если система не умеет правильно понимать эпилоги, stack walking становится ненадёжным именно в тех местах, где точность особенно нужна.

Почему у текущей функции нет обработчика, если RIP в эпилоге

Когда RIP находится в обычном теле функции, Windows может проверить, есть ли у этой функции exception handler (обработчик исключений). Если UNW_FLAG_EHANDLER (флаг наличия exception handler) установлен в UNWIND_INFO, вызывается language-specific handler (языково-специфический обработчик), например __C_specific_handler для MSVC SEH. Этот handler смотрит свои данные и решает, относится ли исключение к защищённой области.

Но если RIP находится в эпилоге, логика другая. Функция уже уходит. Она уже завершила свою логическую работу и разбирает кадр. Поэтому документация Windows x64 рассматривает это как ситуацию, где control is leaving the function. В таком случае для этой функции не связан exception handler для данного исключения. Unwinder должен перейти к caller-у.

Это можно понять человечески. Если ты уже вышел из комнаты и закрываешь дверь, событие, произошедшее на пороге выхода, не должно обрабатываться так, будто ты всё ещё сидишь за столом внутри комнаты. Эпилог - это порог. Функция ещё физически выполняет свои инструкции, но логически уже возвращает управление. Поэтому обработчики её тела больше не являются правильной точкой принятия решения.

Для reverse engineering это особенно важно при анализе __try/__except. Можно увидеть, что функция имеет handler, но если crash RIP попал в эпилог, этот handler не обязан быть рассмотрен как обработчик текущей ситуации. Чтобы понять поведение, нужно учитывать не только “у функции есть handler”, но и “где именно внутри функции находится RIP: prolog, body или epilog”.

Body, prolog, epilog: три разных режима для одного адресного диапазона функции

У функции в .pdata есть BeginAddress (RVA начала диапазона функции) и EndAddress (RVA конца диапазона функции). Новичок может подумать: любой RIP между ними одинаково “внутри функции”. Формально внутри диапазона - да. Но для unwind этого мало. Внутри одного диапазона есть три разные зоны смысла: prolog (пролог), body (тело), epilog (эпилог).

Prolog - зона построения кадра. Там действия могут быть выполнены частично, и unwind должен учитывать offset внутри пролога. Body - обычная зона выполнения функции. Там кадр считается построенным, и UNWIND_INFO применяется как описание полного кадра. Epilog - зона разборки кадра. Там функция уже выходит, и unwinder должен продолжить эффект эпилога, а не искать handler текущей функции.

Body (тело функции) - это не поле в PE и не отдельная секция. Это смысловая область машинного кода между служебным входом и служебным выходом. Именно в теле обычно находится бизнес-логика: проверки, вычисления, обращения к структурам, вызовы API, работа с объектами. Для exception handling тело важно потому, что именно там frame-based handler текущей функции может быть релевантен.

Получается, адрес внутри функции нельзя анализировать только по диапазону BeginAddress-EndAddress. Нужно ответить на следующий вопрос: этот адрес в прологе, в эпилоге или в теле? Windows x64 unwind procedure строит поведение именно на этом различии. И твой static /dynamic должен мыслить так же, иначе отчёты будут слишком грубыми: “адрес внутри функции” - это недостаточно.

Как дизассемблер может ввести в заблуждение

Disassembler (дизассемблер) - инструмент, который переводит байты машинного кода в ассемблерные инструкции. Он помогает человеку видеть код, но не является источником абсолютной истины. Дизассемблер может угадать границы функции неправильно, особенно в обфусцированном коде, коде без символов, смешанных данных и кода, hand-written assembly или при необычных tail calls.

Эпилог часто выглядит просто: add, pop, ret. Поэтому дизассемблер может считать ret концом функции. Но если перед ret есть общий блок, в который прыгают несколько веток, инструмент может не всегда красиво показать, какие пути реально приходят в эпилог. Если есть tail call через jmp, некоторые инструменты могут показать это как переход, а не как возврат. Если есть несколько эпилогов, граф может выглядеть раздробленным.

Unwind-анализ не должен слепо доверять одному дизассемблеру. Нужно сопоставлять: что говорит .pdata о диапазоне функции; что говорит .xdata о кадре; какие инструкции реально стоят в начале и на выходах; какие участки соответствуют допустимым эпилоговым шаблонам; какие адреса достижимы; какие переходы ведут в общий блок выхода. Только сочетание этих слоёв даёт устойчивую картину.

Для человека это значит: когда видишь epilog, не надо сразу писать “конец функции” и идти дальше. Нужно спросить: это обычный ret-эпилог? Tail-call epilog? Общий эпилог? Один из нескольких ранних выходов? Соответствует ли он сохранённым регистрам? Согласован ли с размером stack allocation? Может ли RtlVirtualUnwind распознать его как эпилог?

Связь эпилога с UNWIND_INFO: почему описание косвенное

UNWIND_INFO (структура unwind-информации) не хранит текст “эпилог начинается по адресу таком-то”. В обычной модели она хранит сведения, по которым можно восстановить кадр: размер пролога, массив unwind-кодов, флаги, frame register и дополнительные данные. Эпилог же распознаётся по коду. Но связь между UNWIND_INFO и эпилогом всё равно сильная.

Во-первых, эпилог должен отменять те действия, которые описаны в UNWIND_INFO. Если UNWIND_INFO говорит, что функция сохранила RBX и выделила 0x80 байт, эпилог должен вернуть RBX и освободить эти 0x80 байт. Иначе caller context будет повреждён.

Во-вторых, RtlVirtualUnwind использует UNWIND_INFO для понимания общей формы кадра, но при RIP внутри эпилога учитывает фактический код эпилога. То есть metadata и code должны быть согласованы. Metadata говорит, что за кадр был построен; эпилоговый код показывает, как он разбирается в этой точке.

В-третьих, наличие handler flags в UNWIND_INFO не означает, что handler будет вызван при любом RIP внутри диапазона функции. Если RIP в эпилоге, текущая функция считается покидаемой. Это ещё раз показывает: поля структуры нельзя читать изолированно. Флаг обработчика имеет смысл вместе с положением RIP относительно пролога, тела и эпилога.

Для parser (парсера) это превращается в задачу валидации согласованности. Parser - это компонент, который читает бинарные структуры и превращает байты в осмысленные объекты анализа. Он должен не просто распарсить UNWIND_INFO, но и помочь проверить, соответствует ли код тем обязательствам, которые metadata заявляет. Для static /dynamic это основа сильных findings (находок): не “поле прочитано”, а “поле прочитано и сопоставлено с реальным кодом”.

Что значит “согласованный эпилог” для static /dynamic

Consistent epilog (согласованный эпилог) - это эпилог, который логически и технически соответствует построенному кадру. Если функция сохранила RBX, эпилог должен восстановить RBX. Если функция вычла из RSP 0x120 байт, эпилог должен вернуть RSP на соответствующую величину. Если функция использовала frame pointer, выход должен корректно привести RSP к месту, где лежат сохранённые значения и return address. Если выход заканчивается jmp, форма должна соответствовать допустимому эпилоговому переходу.

static /dynamic может проверять несколько классов согласованности. Первый класс - structural consistency (структурная согласованность): поля .pdata и .xdata корректны, адреса попадают в образ, размеры не выходят за границы секций. Второй класс - semantic consistency (смысловая согласованность): заявленные unwind-действия соответствуют реальным инструкциям пролога. Третий класс - exit consistency (согласованность выходов): обнаруженные эпилоги действительно отменяют построение кадра и заканчиваются допустимым ret/jmp.

Ошибка в эпилоге может быть обычным багом hand-written assembly. Может быть следствием неверного дизассемблирования. Может быть результатом упаковщика или обфускатора. Может быть сознательной антианализной техникой: код работает при обычном выполнении, но ломает stack walking или вводит в заблуждение анализатор. Поэтому отчёт должен быть осторожным: не “вирус”, не “точно ошибка”, а evidence-based finding (находка на основе доказательств).

Evidence (доказательство) - это конкретная проверяемая опора: адрес инструкции, байты, поле структуры, рассчитанный RVA, ожидаемый размер восстановления, фактическая инструкция, границы функции. Хороший инструмент не просто пишет “эпилог подозрительный”, а показывает: функция такая-то, RUNTIME_FUNCTION такой-то, UNWIND_INFO заявляет сохранение RBX и allocation 0x80, но один из выходов по адресу X выполняет ret без восстановления RBX. Это уже материал для специалиста.

Типичные ошибки понимания эпилога

Первая ошибка - считать эпилог просто “последними инструкциями перед ret”. В маленьком учебном примере это часто совпадает. В реальной функции может быть несколько ret, общий блок выхода, tail call, условные переходы к эпилогу. Поэтому эпилог - не позиция “в конце файла”, а смысловой шаблон выхода из кадра.

Вторая ошибка - думать, что если функция имеет handler, то он применим к любому адресу внутри диапазона функции. На x64 это неверно. Если RIP находится в эпилоге, функция уже покидается, и unwind procedure обрабатывает этот случай отдельно. Handler текущей функции не является обычным кандидатом для такого RIP.

Третья ошибка - думать, что UNWIND_INFO полностью описывает и пролог, и эпилог симметрично. На самом деле прологовые действия описаны данными, а эпилог распознаётся по коду. Это значит, что анализатор должен читать и metadata, и инструкции. Только структура .xdata не даст полного ответа, где все выходы функции.

Четвёртая ошибка - игнорировать tail call. Если функция заканчивается jmp после восстановления кадра, это может быть корректный эпилоговый выход, а не обязательно странная обфускация. Но корректность зависит от формы jmp и состояния кадра перед ним.

Пятая ошибка - не различать “похож на эпилог” и “является допустимым эпилогом для Windows x64 unwind”. Последовательность add rsp; ret визуально похожа на выход, но в сложной функции нужно проверять согласование с сохранёнными регистрами, allocation size, frame register и достижимостью.

Как читать эпилог глазами reverse engineer

Когда reverse engineer подходит к эпилогу, он должен держать в голове не одну, а несколько карт. Первая карта - control flow (поток управления): какие ветки ведут к этому выходу. Вторая карта - stack state (состояние стека): каким должен быть RSP перед ret. Третья карта - register preservation (сохранность регистров): какие nonvolatile-регистры были сохранены и где восстанавливаются. Четвёртая карта - metadata contract (контракт метаданных): что заявлено в UNWIND_INFO. Пятая карта - runtime interpretation (интерпретация runtime-а): распознает ли Windows этот участок как эпилог.

Чтение эпилога начинается с вопроса: какой кадр эта функция построила? Без этого нельзя оценить, корректно ли она выходит. Затем нужно найти выходные участки: ret, допустимые jmp, общие блоки возврата. Потом для каждого выхода смотреть, как восстанавливается RSP и регистры. Потом сопоставлять с UNWIND_INFO. И только после этого делать вывод.

Например, если UNWIND_INFO заявляет UWOP_ALLOC_LARGE (большое выделение стека), то эпилог должен вернуть именно этот объём, пусть и через форму add rsp, imm или lea rsp, [fp+offset] в зависимости от схемы. Если UNWIND_INFO заявляет сохранение R12, где в эпилоге возвращается R12? Если функция использовала frame pointer, где происходит восстановление RSP относительно него? Если эпилог заканчивается jmp, является ли это tail call?

Такой подход медленнее, чем “нашёл ret - закончил функцию”, но это и есть взрослый reverse engineering. Особенно в теме unwind, где один неправильный вывод о кадре ломает всю цепочку stack trace.

Почему эпилог важен для crash dump analysis

Crash dump analysis (анализ аварийного дампа) - это исследование сохранённого состояния процесса или системы после сбоя. В дампе у тебя может не быть живого выполнения. У тебя есть память, регистры, стек, модули и метаданные. Если поток упал или был остановлен внутри эпилога, stack walker должен правильно восстановить caller. Иначе стек вызовов в отчёте будет неверным.

Представь, что поток остановлен на ret или pop внутри эпилога. Если анализатор ошибётся и применит правила тела функции, он может показать лишний кадр или потерять реальный caller. Отладчик может вывести странный stack trace. Исследователь может решить, что crash произошёл в неправильном месте. В больших проектах такая ошибка расследования стоит времени и денег.

Поэтому эпилог важен не только для “теории компиляторов”. Он напрямую влияет на практику диагностики. Когда Windows Error Reporting, WinDbg, профилировщик или твой static пытаются построить цепочку вызовов, эпилоговые случаи должны быть корректно обработаны. Редкие состояния не становятся неважными только потому, что они редкие. Они часто проявляются именно тогда, когда система уже в нестабильном состоянии.

Эпилог и безопасность анализа

В security-oriented reverse engineering (реверсе с уклоном в безопасность) эпилоги интересны ещё и потому, что они связаны с управлением возвратом. Return address, восстановление RSP, сохранённые регистры и tail calls - всё это зоны, где ошибка анализа может скрыть контроль потока. Обфускатор может делать необычные выходы, вставлять переходы, использовать shared epilog blocks, играть с дизассемблером. Не каждый необычный эпилог вредоносен, но каждый необычный эпилог требует аккуратного описания.

dynamic (защитник/модуль защиты) в названии твоего проекта может рассматриваться как компонент, который не просто читает структуры, а помогает понять, соблюдает ли бинарник ожидаемые правила. Если эпилоги нестандартны, это может означать: код сгенерирован необычным компилятором; использован assembly; есть упаковщик; есть JIT; есть оптимизация; есть попытка ломать stack walking. Правильный отчёт должен разделять эти варианты.

Хороший вывод звучит не так: “эпилог плохой”. Хороший вывод звучит так: “у функции по RVA X обнаружен выход по RVA Y. Перед ret не восстановлен регистр, который UNWIND_INFO заявляет как сохранённый, либо восстановление происходит нестандартным образом. Требуется ручная проверка дизассемблированного пролога/эпилога и корректности metadata”. Такой язык сохраняет техническую честность.

Итог: эпилог как проверка зрелости понимания unwind

Эпилог кажется простой темой только до тех пор, пока мы думаем о функции как о красивом блоке исходного кода. На машинном уровне эпилог - это строгий участок перехода от callee обратно к caller. Он разбирает кадр, восстанавливает регистры, возвращает RSP, передаёт управление через ret или допустимый jmp. При этом Windows x64 не хранит отдельную таблицу всех эпилогов, а распознаёт их по ограниченному шаблону машинного кода.

Главная сложность в том, что исключение, прерывание или сэмплирование могут попасть в середину эпилога. Тогда функция уже не в обычном теле, но ещё не полностью вернулась. RtlVirtualUnwind должен понять это промежуточное состояние и продолжить эффект эпилога, чтобы получить caller context. Поэтому эпилог влияет на exception handling, stack walking, debugging, dump analysis и reverse engineering.

Для нас эпилог замыкает пару с прологом. Пролог строит кадр и описывается через UNWIND_CODE. Эпилог разбирает кадр и должен быть распознаваемым в коде. UNWIND_INFO связывает эти две стороны не прямым перечислением всех выходов, а контрактом: вот как функция строит кадр, а вот кодовые шаблоны, по которым runtime понимает, что кадр разбирается. Если код и metadata согласованы, Windows может надёжно ходить по стеку. Если нет, анализ становится подозрительным.

Следующий шаг - перейти от общей идеи пролога и эпилога к UNWIND_CODE (кодам действий раскрутки). Теперь уже понятно, зачем они нужны: это не “байтики из .xdata”, а компактный язык, которым компилятор описывает действия пролога, чтобы Windows могла восстановить состояние caller-а в теле функции и корректно понять переходные состояния при обработке исключений.

Разбор формы эпилога как маленького автомата

Чтобы не воспринимать правила эпилога как набор случайных запретов, полезно представить unwinder (механизм раскрутки) как маленький автомат распознавания. Automaton (автомат распознавания) - это не робот, а логическая модель: есть входной поток инструкций, есть допустимые состояния, и по инструкциям система решает, находится ли она в эпилоге. Windows не хочет строить полную модель программы. Ей нужен короткий ответ: текущий RIP стоит в последовательности, которая законно разбирает кадр и заканчивается допустимым выходом, или нет.

В первом состоянии автомат видит восстановление stack pointer (указателя стека). Это может быть add rsp, constant или lea rsp, [frame_register + constant], если функция использует frame register. Смысл у обоих вариантов один: вернуть RSP к точке, от которой можно достать сохранённые регистры и return address. Разница в том, откуда считается новое значение. Add восстанавливает RSP относительно текущего RSP. Lea через frame register восстанавливает его относительно устойчивой базы кадра.

Frame register (регистр кадра) - это регистр, который функция может использовать как стабильную точку отсчёта внутри кадра. Чаще всего в учебных объяснениях вспоминают RBP, но на x64 нужно быть аккуратнее: компилятор может использовать frame register не всегда, а если использует, это отражается в UNWIND_INFO через поля FrameRegister и FrameOffset. Если эпилог восстанавливает RSP через frame register, анализатор должен понимать, что это не произвольное lea, а часть схемы выхода из кадра.

После восстановления RSP автомат может увидеть серию pops (инструкций pop), которые возвращают nonvolatile-регистры. Pop rbx означает: взять 8 байт по адресу RSP, записать их в RBX и увеличить RSP на 8. Это одновременно чтение памяти, восстановление регистра и продвижение вершины стека. Если в прологе регистр был сохранён через push, в эпилоге часто появляется соответствующий pop. Если регистр был сохранён через mov [rsp+offset], восстановление может выглядеть иначе, например mov rbx, [rsp+offset] до освобождения стека, а затем add rsp. Форма зависит от того, как компилятор построил кадр.

В конце автомат ждёт marker (маркер выхода): ret или допустимый jmp. Если вместо этого появляется обычная инструкция вычисления, запись в память, вызов функции или сложный условный переход, последовательность перестаёт быть простым эпилогом. Это не обязательно значит, что код неработоспособен, но для стандартного x64 unwind такой фрагмент уже нельзя автоматически считать эпилогом. Именно поэтому в hand-written assembly так легко ошибиться: код может работать при обычном запуске, но быть плохо совместимым с exception unwinding.

Эта модель автомата полезна для твоего static /dynamic. Инструмент может не просто искать ret. Он может идти назад и вперёд вокруг ret, смотреть, есть ли допустимая последовательность восстановления, сопоставлять её с UNWIND_INFO и помечать случаи, где форма выхода не распознаётся. Но такой анализ должен быть осторожным: компиляторы оптимизируют код, а Microsoft добавляет новые возможности вроде Unwind Information V3. Поэтому finding должен показывать доказательства, а не делать категоричный приговор без контекста.

Эпилог через frame pointer: когда RSP восстанавливается не простым add

В простых примерах эпилог часто начинается с add rsp, N. Это удобно для обучения, но не покрывает все реальные случаи. Если функция использует frame pointer (указатель кадра), она

может восстанавливать RSP через регистр кадра. Например, RBP может хранить устойчивую точку внутри стекового кадра, от которой удобно обращаться к локальным данным или восстанавливать стек.

```
; условная схема с frame pointer
push rbp
sub  rsp, 100h
lea  rbp, [rsp+80h]
; тело функции
lea  rsp, [rbp+80h]
pop  rbp
ret
```

Здесь важно не заучивать конкретные числа, а понять идею. В прологе функция создала кадр и установила RBP как базу. В эпилоге она может восстановить RSP через RBP, потому что RBP сохранял устойчивую позицию даже если RSP временно менялся. Lea (load effective address, загрузка эффективного адреса) в таком эпилоге не читает память. Она вычисляет адресное выражение и записывает результат в RSP. Это именно арифметика адреса, а не загрузка значения по адресу.

Load effective address (загрузка эффективного адреса) - хороший пример термина, который часто путают. Инструкция lea rax, [rbx+10h] не означает “прочитать память по rbx+10h”. Она означает “посчитать rbx+10h и положить число в rax”. Поэтому lea rsp, [rbp+80h] может использоваться для восстановления указателя стека. Это не обращение к локальной переменной, а настройка RSP.

Для unwind такая схема требует связи с FrameRegister и FrameOffset. FrameRegister (регистр кадра) в UNWIND_INFO говорит, какой регистр используется как frame pointer. FrameOffset (смещение кадра) помогает понять, где этот регистр расположен относительно выделенного кадра. Если анализатор игнорирует эти поля, он будет плохо понимать функции с frame pointer и может неправильно оценить эпилог.

В хорошей технической книге здесь важно удержать баланс: не все функции используют frame pointer, и на оптимизированном x64-коде RBP часто свободен для обычного использования, если компилятор не нуждается в стабильной базе кадра. Но когда frame pointer есть, его роль нужно понимать глубоко. Это не “магический старый rbp из x86”, а конкретный элемент схемы кадра, отражённый в unwind metadata.

Почему эпилог может быть общим блоком для нескольких веток

Shared epilog block (общий блок эпилога) - это участок выхода, куда прыгают разные ветки функции. Например, функция может в разных местах присвоить код результата, а потом перейти в один общий блок, который восстанавливает стек и выполняет ret. Это снижает дублирование кода. Если восстановление кадра длинное, компилятору выгоднее иметь один общий выход, чем копировать одинаковые add/pop/ret несколько раз.

```
    test rcx, rcx
    jz   fail
    ; основной путь
    mov  eax, 1
    jmp  done
fail:
    xor  eax, eax
done:
    add  rsp, 40h
```

```
pop  rbx
ret
```

В этом условном примере done - общий блок выхода. Две ветки приходят туда с разными значениями EAX. EAX (младшая 32-битная часть RAX) обычно используется для integer return value (целочисленного возвращаемого значения). Одна ветка кладёт 1, другая 0, но обе используют один эпилог. Это нормальная структура, а не подозрительность сама по себе.

Однако общий эпилог требует, чтобы все входящие пути приходили в него при одинаковом состоянии кадра. Если одна ветка уже восстановила RSP, а другая ещё нет, общий блок может быть некорректным. Компилятор обычно такого не допускает. Но для reverse engineering и анализа обфусцированного кода это важная проверка: control-flow paths (пути потока управления), входящие в эпилог, должны согласовываться по состоянию стека и сохранённых регистров.

Path-sensitive analysis (анализ с учётом пути) - это подход, при котором состояние оценивается не только по адресу, но и по тому, каким путём выполнение могло туда прийти. Для простой проверки эпилогов можно начать без сложного path-sensitive анализа, но в серьёзном static /dynamic он становится важным: один и тот же адрес done может быть корректен для одного пути и некорректен для другого, если код нестандартный или намеренно запутанный.

Эпилог и возвращаемое значение: почему RAX обычно не восстанавливают

В эпилоге мы много говорим о восстановлении nonvolatile registers. Возникает вопрос: а почему не восстанавливается RAX? Почему функция может перед выходом записать в EAX результат, и это не считается порчей регистра? Ответ в calling convention (соглашении о вызовах).

RAX (accumulator register, аккумуляторный регистр / регистр возвращаемого значения) в Windows x64 обычно используется для integer/pointer return value (возврата целого значения или указателя). Это volatile register (несохраняемый регистр): callee может менять его, и caller не должен ожидать, что старое значение RAX переживёт вызов. Более того, caller как раз ожидает, что после вызова в RAX может лежать результат.

Поэтому в эпилоге запись mov eax, 1 перед add/pop/ret не является нарушением. Это установка возвращаемого значения. Но важно, где именно она находится. Если инструкция установки результата стоит до формального эпилогового шаблона, она часть тела или предэпилоговой логики. Если она вставлена между add rsp и ret, это может нарушить допустимую форму эпилога. Компилятор обычно размещает вычисление результата до начала строгого эпилога, а сам эпилог оставляет чистым для unwinder-а.

Это помогает понять разницу между “код перед возвратом” и “эпилог”. Не каждая инструкция перед ret является эпилогом. Если перед ret стоит mov eax, 0, это может быть логика установки результата, а настоящий эпилог начнётся после неё или будет включать только допустимые восстановительные инструкции. Для точного анализа нужно определить границу эпилога, а не просто брать несколько последних строк перед ret.

Эпилог и сохранённые регистры: что значит “восстановить правильно”

Restore (восстановить) - значит вернуть регистру то значение, которое caller имел право ожидать после вызова. Если функция использовала RBX, она могла временно положить туда что угодно. Но перед выходом RBX должен снова содержать старое значение caller-а. Это не вопрос вежливости, а часть ABI.

Восстановление может быть выполнено через pop, если сохранение было push. Но оно может быть и через mov из стекового слота. Например:

```
; один возможный вариант
mov  rbx, [rsp+30h]
add  rsp, 80h
ret
```

Здесь RBX восстанавливается до освобождения стека, потому что сохранённое значение лежит внутри текущего кадра по смещению от RSP. Если сначала выполнить add rsp, 80h, адрес [rsp+30h] уже будет указывать на другое место. Поэтому порядок восстановления зависит от схемы размещения. Это ещё один пример, почему нельзя читать эпилог как набор независимых инструкций. Порядок - часть смысла.

Для parser-а это значит, что простая проверка “где-то перед ret есть mov rbx” недостаточна. Нужно понимать, от какого RSP считается адрес, было ли уже освобождение стека, соответствует ли offset сохранённому слоту из UNWIND_CODE, не перезаписывается ли регистр позже. Сложность здесь не искусственная: именно такие детали отличают полезный анализатор от поверхностного сканера.

Когда эпилог выглядит несимметрично прологу, но остаётся правильным

Новичок ожидает идеальную зеркальность: был push rbx - будет pop rbx; был sub rsp, 80h - будет add rsp, 80h; был push rdi - будет pop rdi. В учебном коде так часто бывает. Но оптимизированный код может выглядеть менее симметрично, оставаясь корректным.

Например, сохранение регистра могло быть выполнено не push, а mov [rsp+offset], reg после выделения кадра. Тогда восстановление будет mov reg, [rsp+offset], а не pop. Или функция могла использовать frame pointer и восстанавливать RSP через lea. Или несколько выходов могут вести в общий блок, где часть восстановления уже вынесена. Или tail call может заменить ret.

Correctness (корректность) здесь определяется не внешней красотой, а состоянием на границе выхода. Перед передачей управления caller-у должны быть выполнены обязанности ABI: RSP должен указывать туда, куда нужно, nonvolatile-регистры должны быть восстановлены, return address должен быть доступен для ret или кадр должен быть разобран перед tail call. Если эти условия выполнены допустимой формой, эпилог корректен даже без школьной зеркальности.

Это важное предупреждение для static/dynamic: не надо делать слишком примитивные правила, которые принимают только один шаблон. Такой инструмент будет ругаться на нормальный оптимизированный код. Нужна модель, которая понимает несколько корректных форм и умеет объяснять, почему конкретный выход признан нормальным или подозрительным.

Эпилог, interrupts и asynchronous events

Asynchronous event (асинхронное событие) - событие, которое может произойти не потому, что текущая инструкция явно вызвала исключение, а потому что внешний механизм остановил поток между инструкциями или на определённой инструкции. Примеры зависят от уровня: аппаратные прерывания, отладочные события, сэмплирование профилировщика, остановка потока для дампа. Для user-mode exception handling конкретные детали различаются, но общая проблема одна: поток может быть застигнут в любой точке.

Именно из-за асинхронности эпилоговые правила важны. Если бы исключения возникали только в “удобных” местах, можно было бы упростить модель. Но runtime и отладчик должны быть готовы к тому, что RIP указывает на середину пролога, середину тела или середину эпилога. Поэтому unwind-модель строится так, чтобы быть применимой к произвольному RIP внутри функции.

Interrupt-safe unwinding (безопасная раскрутка при прерываниях) требует, чтобы пролог и эпилог были описуемыми или распознаваемыми. Пролог описывается через коды. Эпилог ограничивается формой. Если функция нарушает форму эпилога, редкое асинхронное событие может привести к неверной трассировке стека. Это может не проявляться в обычном запуске, но проявиться в отладке, дампе или обработке исключения.

Такой уровень объяснения нужен, чтобы не думать о unwind как о декоративной таблице для красивых stack traces. Это часть надёжности всей системы исполнения. Windows должна уметь остановить поток в неприятный момент и всё равно понять, где caller.

Как эпилог связан с оптимизациями компилятора

Optimizer (оптимизатор) - часть компилятора, которая преобразует код ради скорости, размера или других целей, сохраняя наблюдаемое поведение программы. Оптимизатор может менять структуру выходов функции: объединять эпилоги, дублировать короткие эпилоги, превращать call+ret в tail jmp, удалять ненужные сохранения, переиспользовать регистры, убирать frame pointer.

Из-за оптимизаций машинный код часто не похож на исходный код. Один return в C может стать несколькими выходами. Несколько returns в C могут стать одним общим done-блоком. Функция, которая в Debug выглядела с классическим push rbp; mov rbp, rsp, в Release может вообще не использовать RBP как frame pointer. Это нормально.

Но оптимизатор связан правилами ABI и unwind. Он может менять форму, но не может произвольно ломать восстановимость кадра для функций, которым требуется unwind metadata. Поэтому эпилоги оптимизированного кода могут быть разнообразными, но всё равно должны оставаться в допустимых границах. Это одна из причин, почему Windows задаёт строгие правила: они дают компилятору свободу внутри понятного коридора.

Для reverse engineer это значит: не надо ожидать “учебный пролог” и “учебный эпилог”. Нужно ожидать контракт. Контракт важнее внешней формы. Если контракт соблюдён, код может быть непривычным, но корректным. Если контракт нарушен, даже красивый на вид код может быть опасен для unwinding.

Как документировать эпилог в отчёте анализатора

Report (отчёт) - это не просто список найденных адресов. В хорошем reverse engineering отчёт должен помогать человеку принять решение. Для эпилогов это особенно важно, потому что многие ситуации неоднозначны. Один и тот же необычный выход может быть

нормальной оптимизацией, hand-written assembly или обфускацией.

Полезная запись в отчёте должна включать контекст: module (модуль), function RVA (RVA функции), epilog candidate RVA (RVA кандидата в эпилог), bytes (байты), disassembly (дизассемблированные инструкции), связанный RUNTIME_FUNCTION, ссылка на UNWIND_INFO, заявленные сохранения регистров, рассчитанный stack allocation и результат проверки. Тогда человек видит не только вывод, но и путь к выводу.

Например: “Candidate epilog at RVA 0x1530 restores RSP with add rsp, 0x80 and returns with ret. UNWIND_INFO declares allocation 0x80 and saves RBX/RDI through pushes. The candidate restores RDI/RBX before ret. Verdict: consistent.” Это сильный отчёт. Он показывает факты и вывод.

Другой пример: “Candidate epilog at RVA 0x2410 ends with ret but does not restore RBX, while UNWIND_INFO declares RBX saved in prolog. Incoming control-flow paths do not show earlier RBX restoration. Verdict: suspicious, requires manual review.” Здесь нет истерики и нет ложной уверенности. Есть evidence, expected behavior и observed behavior.

Такой стиль отчёта соответствует подходу всей книги: не бросать термины, не давать магические ответы, а строить объяснимую модель.

Граница следующей темы

Теперь пролог и эпилог можно поставить на свои места. Пролог - это участок, где функция строит кадр и где действия описываются UNWIND_CODE. Эпилог - участок, где функция разбирает кадр и должен быть распознан по строгому машинному шаблону. Оба нужны, потому что unwind должен работать при любом RIP внутри функции: в начале, в середине, на выходе.

Следующая тема - UNWIND_CODE (код действия раскрутки). Теперь она должна восприниматься не как сухая таблица операций, а как язык описания пролога. Каждый код отвечает на практический вопрос: какое действие функция выполнила при входе, где это действие находится в прологе, и как это действие нужно отменить, чтобы восстановить caller context. Без пролога UNWIND_CODE непонятен. Без эпилога непонятно, почему не всё описывается кодами. Вместе они дают цельную модель.

4.3. Что такое UNWIND_CODE и как он кодирует действия пролога

К этому месту мы подошли не через случайный список структур, а через цепочку необходимости. Сначала стало понятно, что Windows x64 не может надежно “угадывать” стек, потому что оптимизированная функция не обязана сохранять красивый классический frame pointer. Затем мы увидели .pdata (procedure data, секцию процедурных данных), которая дает диапазон функции. Потом появилась .xdata (exception data, секция данных исключений), где лежит подробное описание unwind. Наконец, внутри .xdata мы дошли до UNWIND_INFO (структуры информации раскрутки), которая в общем виде говорит: вот размер пролога, вот флаги, вот возможный frame register, вот массив действий. Теперь нужно раскрыть самый важный внутренний слой этого массива - UNWIND_CODE.

UNWIND_CODE (код действия раскрутки, запись действия пролога для unwinder-а) - это маленькая запись внутри UNWIND_INFO, которая описывает одно смысловое действие пролога функции: сохранить nonvolatile register (сохраняемый регистр), выделить место на стеке, установить frame pointer (указатель кадра), сохранить XMM-регистр или зафиксировать специальный машинный кадр. Важно сразу убрать неправильную мысль: UNWIND_CODE - это не машинная инструкция процессора. Это не opcode из .text. Это не байты push rbx или sub rsp, 40h. Это системная, компактная, табличная запись о том, какой эффект эта инструкция оказала на состояние стека и регистров.

Именно это различие делает тему непростой. Новичок часто смотрит на UNWIND_CODE и думает: “значит, это дизассемблированный пролог в другом виде”. Нет. Дизассемблирование говорит, какие инструкции реально лежат в коде. UNWIND_CODE говорит, какие из этих инструкций важны для восстановления caller context (контекста вызывающей функции). Между этими двумя слоями есть связь, но нет полного равенства. Некоторые инструкции пролога вообще не попадают в unwind-коды, потому что они не нужны для раскрутки. Например, сохранение входного аргумента RCX в shadow space может быть важным для логики функции, но если оно не меняет RSP и не сохраняет nonvolatile state, оно может не быть частью unwind-описания.

> Главная мысль раздела: UNWIND_CODE - это не “байтовая копия пролога”. Это маленький формальный язык, которым компилятор объясняет Windows, какие действия пролога нужно отменить, чтобы из текущего кадра получить кадр caller-а.

Почему вообще понадобился маленький язык действий

Если бы Windows при исключении могла просто выполнить функцию назад, UNWIND_CODE был бы не нужен. Но процессор не умеет “отматывать” произвольные инструкции в обратную сторону. Машинный код обычно необратим. Если инструкция записала значение в регистр, старое значение могло быть потеряно. Если функция вычла число из RSP, сама по себе инструкция не оставила рядом комментарий, сколько и зачем она выделила. Если функция сохранила RBX где-то в стековом кадре, нужно знать точное место, откуда его потом брать.

Reverse execution (обратное выполнение) в общем случае слишком сложная задача. Поэтому Windows x64 решает более узкую задачу: ей не нужно отменять весь смысл функции. Ей нужно восстановить только тот минимум состояния, который нужен для перехода к вызывающему кадру: RSP, return address, nonvolatile registers и, если нужно, frame pointer. Для этой задачи пролог функции можно описать не всеми инструкциями, а набором семантических действий.

Semantic action (смысловое действие) - это описание не конкретной формы инструкции, а ее эффекта для unwinder-а. Например, push rbx как машинная инструкция означает: уменьшить RSP на 8 и записать RBX по новому адресу RSP. Для unwind важнее другое: “RBX был сохранен на стеке, а RSP был сдвинут на 8”. Поэтому появляется UWOP_PUSH_NONVOL (unwind operation push nonvolatile, операция раскрутки для сохранения nonvolatile-регистра через push). Это уже не ассемблерная инструкция, а категория действия, которую RtlVirtualUnwind умеет отменять.

Такой язык действий нужен по трем причинам. Первая причина - надежность. Unwinder (механизм раскрутки) не должен гадать по дизассемблированному коду, особенно в аварийной ситуации. Вторая причина - скорость. Табличные записи читать проще, чем полноценно анализировать x86-64 инструкции. Третья причина - компактность. Вместо длинного описания всего пролога система хранит несколько двухбайтовых слотов и иногда дополнительные слова.

Слово “язык” здесь уместно. Как у обычного языка есть словарь и грамматика, так у UNWIND_CODE есть набор операций UWOP_* и правила их размещения. Нельзя писать что угодно. Нельзя произвольно смешивать действия. Нельзя использовать неверные регистры или неверные смещения. Если код функции и unwind-язык расходятся, Windows может неправильно построить stack trace, неправильно пройти обработчики исключений или вообще потерять цепочку вызовов.

Где физически лежит UNWIND_CODE

UNWIND_CODE физически находится не в .pdata. .pdata уже выполнила свою роль: по текущему RIP она дала RUNTIME_FUNCTION (запись диапазона функции). Внутри RUNTIME_FUNCTION есть поле UnwindInfoAddress, которое указывает на UNWIND_INFO. Вот уже внутри UNWIND_INFO, после первых служебных байтов, располагается массив UNWIND_CODE. Обычно эти байты лежат в области, которую называют .xdata. Но, как и раньше, важно не имя секции, а реальная связь: RUNTIME_FUNCTION -> UnwindInfoAddress -> UNWIND_INFO -> массив UNWIND_CODE.

```
RIP / ControlPc
    |
    v
RtlLookupFunctionEntry
    |
    v
RUNTIME_FUNCTION in .pdata
    BeginAddress
    EndAddress
    UnwindInfoAddress  ----+
                           |
                           v
                  UNWIND_INFO in .xdata
                      Version / Flags
                      SizeOfProlog
                      CountOfCodes
                      FrameRegister / FrameOffset
                      UNWIND_CODE[]  <--- здесь начинается язык действий пролога
```

Эта схема нужна, чтобы не смешивать уровни. RUNTIME_FUNCTION не говорит, какие регистры сохранены. Он только ведет к UNWIND_INFO. UNWIND_INFO не всегда сам по себе дает полный ответ одним заголовком. Его заголовок говорит, сколько дальше читать и как интерпретировать данные. А уже UNWIND_CODE[] (массив кодов раскрутки) описывает действия пролога. Поэтому парсер, который остановился на RUNTIME_FUNCTION, еще не

понял функцию. Он только нашел дверь. Парсер, который прочитал заголовок UNWIND_INFO, подошел к комнате. Но смысл кадра начинается тогда, когда он корректно разобрал UNWIND_CODE.

Важно также помнить про адресные модели. Поле UnwindInfoAddress в RUNTIME_FUNCTION - это RVA (relative virtual address, относительный виртуальный адрес), а не file offset. Но сами UNWIND_CODE находятся уже по вычисленному месту в файле или памяти. Когда анализатор добрался до UNWIND_INFO, он читает байты структуры подряд. Если перепутать RVA и file offset, весь массив будет читаться из неправильного места, а ошибки дальше будут выглядеть как “странные UWOP”, хотя реальная причина раньше: неверное преобразование адреса.

Слот UNWIND_CODE: почему запись маленькая, но не примитивная

В документации UNWIND_CODE выглядит очень коротко: один байт CodeOffset, затем второй байт, внутри которого четыре бита UnwindOp и четыре бита OpInfo. Из-за этого может возникнуть обманчивое ощущение, что тема простая. На самом деле маленький размер записи не означает маленький смысл. Один такой слот может означать сохранение регистра, выделение стека, установку frame pointer или начало операции, у которой есть дополнительные слоты.

```
Один основной слот UNWIND_CODE занимает 2 байта:

+0: CodeOffset   - смещение в прологе
+1: low  4 bits  - UnwindOp, код операции UWOP_*
    high 4 bits  - OpInfo, дополнительная информация операции

При ручном разборе второго байта удобно думать так:
    UnwindOp = byte1 & 0x0F
    OpInfo   = byte1 >> 4
```

Slot (слот, ячейка массива) - это минимальная двухбайтовая единица массива UNWIND_CODE. Документация иногда говорит “node” или “code item”, но для практического парсинга удобнее думать именно о слотах. Некоторые операции занимают один слот. Некоторые занимают основной слот плюс один дополнительный слот. Некоторые занимают основной слот плюс два дополнительных слота. Поэтому CountOfCodes (количество кодов раскрутки) надо понимать очень точно: это не всегда количество смысловых операций. Это количество слотов в массиве.

Например, UWOP_PUSH_NONVOL занимает один слот. UWOP_ALLOC_SMALL тоже занимает один слот. Но UWOP_SAVE_NONVOL занимает два слота: первый говорит, какой регистр сохранен, а второй хранит смещение в стековом кадре. UWOP_SAVE_NONVOL_FAR занимает уже три слота: основной слот плюс два слота для длинного смещения. Если анализатор будет считать “один слот = одна операция” без учета дополнительных данных, он собьется на второй же сложной функции.

> Точная мысль: UNWIND_CODE - это массив слотов. Смысловая операция UWOP_* может занимать один, два или три слота. Поэтому parser должен идти по массиву не просто i += 1, а i += размер_операции_в_слотах.

CodeOffset: маленький байт, который отвечает на вопрос “успела ли инструкция выполниться”

CodeOffset (смещение кода, смещение в прологе) - это первый байт слота UNWIND_CODE. Он не является RVA. Он не является file offset. Он не является смещением внутри .xdata. Он отсчитывается от начала функции, точнее от начала ее пролога. Это смещение указывает на конец инструкции, которая выполнила описанное действие, плюс один, то есть фактически на начало следующей инструкции.

Почему это так важно? Потому что исключение может произойти не только после полного пролога, но и внутри пролога. Если RIP находится в прологе, Windows должна понять, какие прологовые действия уже произошли, а какие еще нет. Если push rbx уже выполнился, то RBX лежит на стеке и RSP уже уменьшен. Если sub rsp, 40h еще не выполнился, то выделенного локального кадра еще нет. Нельзя отменять то, что не успело случиться.

Именно здесь CodeOffset становится границей факта. Он говорит: действие этой записи считается выполненным после того, как управление дошло до следующей инструкции. Если текущий offset внутри функции меньше CodeOffset, то соответствующее действие, возможно, еще не надо учитывать. Если текущий offset уже прошел эту границу, действие считается частью состояния кадра. Такая логика позволяет RtlVirtualUnwind не гадать на уровне исходного кода, а работать с формальными границами инструкций.

```
Условный пролог:

offset 00: push rbx          ; длина 1 байт
offset 01: sub  rsp, 20h     ; длина 4 байта
offset 05: ... тело функции

Для push rbx:
    CodeOffset = 01h, потому что следующая инструкция начинается с offset 01

Для sub rsp, 20h:
    CodeOffset = 05h, потому что следующая инструкция начинается с offset 05
```

Обрати внимание: CodeOffset не обязан совпадать с адресом самой инструкции. Для push rbx он равен не 0, а 1. Для sub rsp, 20h он равен не 1, а 5. Это частая ошибка ручного разбора. Человек видит инструкцию по смещению 01 и ожидает CodeOffset=01. Но документационный смысл другой: CodeOffset показывает точку после инструкции. Так удобнее решать вопрос “эта операция уже завершилась или еще нет?”.

Есть еще один практический вывод. Поскольку CodeOffset занимает один байт, обычный формат UNWIND_INFO ограничивает размер кодируемого пролога величиной, которая должна помещаться в 255 байт. Это не значит, что вся функция маленькая. Это значит, что формально описываемая часть пролога должна быть ограниченной. В MASM для frame-процедур директива .ENDPROLOG тоже связана с окончанием пролога. Пролог в Windows x64 - это не произвольная длинная подготовительная область, а строго ограниченный участок, который должен быть описуемым.

UnwindOp: не инструкция процессора, а операция отмены эффекта

UnwindOp (код операции раскрутки) - это младшие 4 бита второго байта UNWIND_CODE. Он выбирает тип смыслового действия: push nonvolatile register, allocate stack, set frame register, save nonvolatile register by memory store, save XMM register, push machine frame. В именах Microsoft эти операции начинаются с UWOP_*. UWOP означает unwind operation (операция раскрутки).

Слово operation (операция) здесь может сбивать. Это не операция процессора в смысле opcode. Например, машинных способов выделить стек несколько: sub rsp, imm, иногда с предварительным __chkstk для больших выделений. Но в unwind-языке это сводится к UWOP_ALLOC_SMALL или UWOP_ALLOC_LARGE. Машинная форма может отличаться, а смысл для unwinder-а один: RSP был уменьшен на такой-то размер, значит при виртуальной раскрутке этот размер надо вернуть.

Чтобы понять UnwindOp, нужно постоянно держать в голове направление времени. Пролог выполняется вперед: сохранить регистр, уменьшить RSP, установить frame pointer. Unwinder мыслит назад: если мы хотим перейти к caller context, надо восстановить сохраненный регистр, увеличить RSP, убрать текущий кадр. Поэтому UWOP - это описание действия пролога, но используется оно для обратного восстановления.

```
Прямое выполнение пролога:
    push rbx       -> RSP уменьшается, RBX сохраняется в памяти
    sub rsp, 20h   -> RSP уменьшается еще на 20h

Виртуальная раскрутка из тела функции:
    undo sub       -> RSP увеличивается на 20h
    undo push      -> RBX читается из [RSP], RSP увеличивается на 8
    затем берется return address caller-a
```

Это не означает, что Windows буквально выполняет обратные инструкции процессора. Она не исполняет pop rbx из кода функции. Она моделирует эффект: обновляет CONTEXT (контекст регистров), читает нужные значения из памяти стека и вычисляет новый RIP/RSP. Поэтому слово virtual в RtlVirtualUnwind (виртуальная раскрутка) очень важное: система не “возвращается” настоящим ret, а вычисляет, как выглядел бы контекст вызывающей функции.

OpInfo: четыре бита, которые меняют смысл операции

OpInfo (operation information, дополнительная информация операции) - это старшие 4 бита второго байта UNWIND_CODE. Его смысл зависит от UnwindOp. Универсального значения у OpInfo нет. В одной операции он хранит номер регистра. В другой - часть размера выделения стека. В третьей - вариант формы кодирования. Поэтому нельзя написать один parser, который просто печатает OpInfo как “регистровый номер” для всех UWOP.

Например, для UWOP_PUSH_NONVOL OpInfo - это номер nonvolatile-регистра, который был сохранен через push. Если OpInfo равен 3, это RBX. Если 5 - RBP. Если 12 - R12. Но для UWOP_ALLOC_SMALL OpInfo уже не регистр. Там он кодирует размер выделения стека по формуле size = OpInfo * 8 + 8. Если OpInfo равен 3, это 32 байта выделения, а не RBX. Один и тот же битовый рисунок имеет разный смысл в разных операциях.

```
Один и тот же OpInfo = 3:

UWOP_PUSH_NONVOL, OpInfo=3
    -> сохранить RBX

UWOP_ALLOC_SMALL, OpInfo=3
    -> выделить 3*8 + 8 = 32 байта стека

Вывод: OpInfo нельзя интерпретировать отдельно от UnwindOp.
```

Register encoding (кодирование регистров) в UNWIND_CODE использует числовые значения от 0 до 15: RAX, RCX, RDX, RBX, RSP, RBP, RSI, RDI, затем R8-R15. Но наличие числового кода еще не означает, что любой регистр допустим в любой операции. Например, UWOP_PUSH_NONVOL по смыслу должен относиться к nonvolatile integer registers (сохраняемым целочисленным регистрам), а не к произвольному volatile-регистру. Хороший парсер может сначала

декодировать номер, но хороший валидатор должен проверять допустимость номера для конкретного UWOP.

```
Кодирование целочисленных регистров в OpInfo:

0  RAX
1  RCX
2  RDX
3  RBX
4  RSP
5  RBP
6  RSI
7  RDI
8  R8
9  R9
10 R10
11 R11
12 R12
13 R13
14 R14
15 R15

Для операций сохранения nonvolatile-регистров практически значимы RBX, RBP,
RSI, RDI, R12-R15. Volatile-регистры вроде RAX, RCX, RDX, R8-R11 требуют
особой осторожности: само число декодируется, но смысл может быть незаконным
для данной операции.
```

Почему массив отсортирован в обратном порядке

Массив UNWIND_CODE отсортирован по убыванию CodeOffset. Это выглядит неожиданно, если мы привыкли читать код сверху вниз. В машинном прологе инструкции идут от меньшего offset к большему. Но unwind-массив обычно хранит их от более поздних действий к более ранним. Причина простая: unwind отменяет пролог в обратном порядке.

Представим пролог: сначала push rbx, потом sub rsp, 20h. Прямое выполнение сначала сохраняет RBX и сдвигает RSP на 8, потом выделяет еще 32 байта. Если мы находимся в теле функции и хотим вернуться к caller context, нужно сначала отменить выделение 32 байта, а потом восстановить RBX и вернуть RSP еще на 8. То есть порядок отмены обратен порядку исполнения.

```
Пролог в коде, порядок исполнения:

offset 00: push rbx       ; действие A
offset 01: sub rsp, 20h   ; действие B

UNWIND_CODE по убыванию CodeOffset:

CodeOffset=05, UWOP_ALLOC_SMALL, OpInfo=3   ; действие B
CodeOffset=01, UWOP_PUSH_NONVOL, OpInfo=3   ; действие A

Виртуальная раскрутка из тела:
    сначала отменить B
    потом отменить A
```

Эта обратная сортировка особенно важна, когда RIP находится внутри пролога. Допустим, исключение произошло после push rbx, но до sub rsp, 20h. Тогда нельзя отменять выделение 32 байта, потому что оно еще не произошло. Но нужно учитывать, что RBX уже сохранен через push. Документационная логика говорит: unwinder смотрит на offset текущего RIP внутри функции, находит в массиве первую запись, чей CodeOffset меньше или равен текущему offset, и отменяет оставшиеся записи. За этой сухой фразой стоит точная идея: отменять нужно только те прологовые действия, которые уже стали частью состояния.

Если парсер не понимает обратного порядка, он будет строить неправильную картину стека. Он может решить, что сначала надо восстановить RBX, когда RSP еще указывает на область локальных переменных, а не на сохраненный RBX. Или наоборот, увеличит RSP раньше времени. Для человека это выглядит как “немного перепутал порядок”. Для unwinder-а это означает потерянный caller, неверный return address и сломанный stack trace.

Первый ручной пример: push rbx + sub rsp, 20h

Теперь полезно разобрать маленький пример так, как его должен видеть reverse engineer. Возьмем условную функцию, у которой пролог состоит только из сохранения RBX и выделения 32 байт локального стека. RBX выбран не случайно: это nonvolatile register, то есть вызываемая функция обязана вернуть его вызывающей стороне в прежнем виде, если сама его использовала.

```
; условный пролог функции
00: push rbx
01: sub  rsp, 20h
05: ... тело функции ...
```

Machine instruction push rbx (машинная инструкция сохранения регистра через push) меняет RSP и записывает RBX на стек. Для unwind это кодируется как UWOP_PUSH_NONVOL с OpInfo=3, потому что регистровый номер RBX равен 3. CodeOffset равен 1, потому что push rbx занимает один байт и следующая инструкция начинается по offset 1.

Machine instruction sub rsp, 20h (вычитание 32 из указателя стека) выделяет 32 байта. Для unwind это кодируется как UWOP_ALLOC_SMALL, потому что размер 32 попадает в диапазон малых выделений. Формула для OpInfo такая: size = OpInfo * 8 + 8. Чтобы получить 32, нужно OpInfo=3, потому что 3*8+8=32. CodeOffset равен 5, если инструкция sub занимает 4 байта и следующая инструкция начинается с offset 5.

```
UNWIND_CODE[] в смысловом виде:

1) CodeOffset=05, UnwindOp=UWOP_ALLOC_SMALL, OpInfo=3
   смысл: в прологе выделено 32 байта стека

2) CodeOffset=01, UnwindOp=UWOP_PUSH_NONVOL, OpInfo=3
   смысл: RBX сохранен через push
```

Теперь посмотрим на байты. Один слот - это два байта. Во втором байте младшие 4 бита содержат UnwindOp, старшие 4 бита - OpInfo. Для UWOP_ALLOC_SMALL номер операции равен 2. OpInfo=3. Значит второй байт будет 0x32: старшая тетрада 3, младшая 2. Для UWOP_PUSH_NONVOL номер операции 0, OpInfo=3, значит второй байт будет 0x30.

```
Минимальный UNWIND_INFO для примера может выглядеть так:

01 05 02 00   05 32   01 30

Расшифровка:
01          Version=1, Flags=0
05          SizeOfProlog=5
02          CountOfCodes=2 слота
00          FrameRegister=0, FrameOffset=0
05 32       CodeOffset=5,  UnwindOp=2 (ALLOC_SMALL), OpInfo=3 -> 32 bytes
01 30       CodeOffset=1,  UnwindOp=0 (PUSH_NONVOL), OpInfo=3 -> RBX
```

Этот пример маленький, но в нем уже видны почти все ловушки. CodeOffset - не адрес инструкции, а точка после нее. OpInfo меняет смысл в зависимости от UnwindOp. Массив идет не в порядке машинных инструкций, а в порядке обратной раскрутки. CountOfCodes считает слоты. Байты второго поля надо маскировать по тетрадам. Если эти вещи не зафиксировать,

следующий более сложный пример сразу начнет “сыпаться”.

Что именно отменяет RtlVirtualUnwind

RtlVirtualUnwind (функция виртуальной раскрутки) не исполняет код функции. Она работает с CONTEXT (структурой контекста процессора), RUNTIME_FUNCTION и UNWIND_INFO. Ее задача - получить состояние caller-а так, как будто текущая функция уже вернулась, но без фактического выполнения ret. Для этого она применяет UNWIND_CODE как программу восстановления.

Если RIP находится в теле функции, считается, что пролог уже полностью выполнился. Тогда RtlVirtualUnwind может отменять все действия из массива UNWIND_CODE. Если функция выделила стек, RSP виртуально увеличивается. Если регистр был сохранен, значение читается из памяти стека и записывается в соответствующее поле CONTEXT. Если был установлен frame pointer, он используется для вычислений. В конце из вершины восстановленного стека извлекается return address, который становится новым RIP caller-а.

Если RIP находится в прологе, логика тоньше. Нужно отменить только те действия, которые уже успели произойти. Именно для этого нужны CodeOffset и SizeOfProlog. SizeOfProlog (размер пролога) говорит, находится ли текущий RIP в прологовой области. CodeOffset отдельных записей говорит, какие действия уже пересечены текущим RIP. Поэтому UNWIND_CODE - это не только описание “как выглядит кадр в теле функции”, но и способ пережить исключение в переходном состоянии, пока кадр еще строится.

Если RIP находится в эпилоге, RtlVirtualUnwind идет другим путем: она распознает оставшуюся часть эпилога по коду и симулирует ее вперед. Это мы уже разбирали. Здесь важно увидеть связку: пролог описан UNWIND_CODE, эпилог распознается по строгому шаблону, а тело функции использует итоговое описание кадра. Все три случая связаны одной целью - получить caller context без догадок.

> UNWIND_CODE нужен не только для аварии “после пролога”. Он нужен и для середины пролога. Поэтому CodeOffset - не декоративное поле, а ключ к частично построенному стековому кадру.

Какие действия пролога попадают в UNWIND_CODE, а какие нет

Очень важно не ждать от UNWIND_CODE полного пересказа пролога. Пролог может содержать инструкции, которые важны для функции, но не важны для unwind. Например, функция может сохранить входной аргумент в home area (домашнюю область параметров), подготовить значение для __chkstk, выполнить call __chkstk для проброса больших страниц стека, занулить регистр, загрузить адрес. Не каждое такое действие попадает в unwind metadata.

Критерий примерно такой: попадает то, что влияет на восстановление nonvolatile state и RSP. Если действие нужно, чтобы понять, где return address, где сохраненные nonvolatile-регистры и как вернуть RSP, оно должно быть описано. Если действие относится только к логике функции или подготовке вычислений, оно может отсутствовать в UNWIND_CODE.

```
Пролог может содержать:

mov [rsp+8], rcx      ; сохранить параметр в home slot
push rbx              ; сохранить nonvolatile RBX
sub rsp, 40h          ; выделить стек
lea rbp, [rsp+20h]    ; установить frame pointer
```

```
mov rax, 10000h       ; подготовить размер для __chkstk
call __chkstk         ; проверить/расширить стек для большой аллокации

В UNWIND_CODE попадут не все строки, а только те действия, которые нужны
для unwind-модели: сохранение nonvolatile-регистров, выделение стека,
установка frame pointer, специальные формы сохранения.
```

Это объясняет, почему сравнение пролога с UNWIND_CODE нельзя делать как простое “одна инструкция - одна запись”. Правильнее строить semantic comparison (смысловое сравнение). Нужно спрашивать: какие инструкции изменили unwind-состояние? Как они должны быть описаны? Совпадает ли описание с фактическим эффектом? Не отсутствует ли важная запись? Нет ли записи, которая заявляет действие, которого в коде не видно?

Для static /dynamic это особенно важно. Если инструмент будет требовать полного совпадения каждого байта пролога с unwind-кодами, он даст много ложных тревог. Если он вообще не будет сопоставлять код с metadata, он пропустит реальные несоответствия. Нужен средний инженерный путь: извлекать из дизассемблированного пролога только unwind-relevant actions (действия, значимые для раскрутки) и сравнивать их с UNWIND_CODE.

CountOfCodes: почему “количество кодов” не равно количеству действий

CountOfCodes (количество unwind-кодов) - поле заголовка UNWIND_INFO, которое говорит, сколько слотов UNWIND_CODE нужно прочитать. Слово “codes” здесь легко неправильно понять. В практическом парсинге это количество двухбайтовых слотов, а не обязательно количество логических операций. Это различие становится критичным на SAVE_NONVOL, SAVE_XMM128 и ALLOC_LARGE.

Дополнительный слот - это такой же двухбайтовый элемент массива, но он не имеет собственного CodeOffset и UnwindOp в обычном смысле. Его байты используются как число: например, как смещение сохраненного регистра. Поэтому если parser просто выводит каждый слот как отдельную операцию, он начнет видеть мусорные UWOP в дополнительных данных. Правильный parser, прочитав основной слот, должен знать, сколько слотов занимает эта операция, и пропустить дополнительные слоты как payload (полезную нагрузку операции).

```
Пример операции с дополнительным слотом:

UWOP_SAVE_NONVOL, OpInfo=RBX
    дополнительный слот: offset/8

Массив слотов:
    slot 0: основной код операции
    slot 1: число, а не самостоятельный UNWIND_CODE

CountOfCodes увеличивается на 2, хотя смысловая операция одна.
```

Payload (полезная нагрузка, дополнительные данные операции) - это данные, которые принадлежат основной операции. В контексте UNWIND_CODE это обычно смещение в стековом кадре или размер выделения. Payload не надо интерпретировать как новую операцию. Его надо читать по правилам конкретного UWOP. Это обычная ошибка самописных парсеров: человек видит CountOfCodes=5 и пытается распечатать пять операций, хотя реально там может быть три операции и два дополнительных слота.

Есть еще правило выравнивания. Массив UNWIND_CODE должен занимать четное количество слотов с точки зрения расположения последующих данных. Если CountOfCodes нечетный, после фактических слотов может быть один padding slot (слот выравнивания), чтобы optional data (дополнительные данные), например handler RVA или chained

RUNTIME_FUNCTION, начинались с правильного выравнивания. Важно: этот padding slot не является действием пролога. Его нельзя декодировать как UWOP. Для вычисления начала дополнительных данных часто используют формулу (CountOfCodes + 1) & ~1.

```
Начало optional area после массива UNWIND_CODE:

aligned_count = (CountOfCodes + 1) & ~1
optional_area = &UnwindCode[aligned_count]

Если CountOfCodes = 3, реально под массив/выравнивание занято 4 слота.
Четвертый слот - padding, а не операция.
```

Смещения в дополнительных слотах: offset от чего?

Когда UWOP_SAVE_NONVOL или UWOP_SAVE_XMM128 сохраняет регистр через mov в стековый кадр, одной информации “сохранен RBX” недостаточно. Нужно знать, где именно он лежит. Поэтому у таких операций есть дополнительный слот со смещением. Но слово offset (смещение) снова опасно: нужно обязательно спросить - смещение от какой базы?

В unwind-операциях смещение для сохраненного значения относится к fixed stack allocation (фиксированной области стекового выделения), а не к началу файла, не к image base и не к .xdata. Если frame register не используется, база обычно связана с RSP после выделения фиксированного кадра. Если frame register используется, база определяется относительно того места, где RSP находился при установке frame pointer: фактически через FrameRegister и FrameOffset в UNWIND_INFO. Поэтому операции со смещениями нельзя правильно понять отдельно от заголовка UNWIND_INFO.

Это одна из причин, почему разбор UNWIND_CODE должен быть контекстным. Недостаточно прочитать один слот и сказать “offset = 0x20”. Надо сказать полнее: “RBX сохранен по смещению 0x20 от базы кадра, определяемой текущим UNWIND_INFO; если FrameRegister=0, база одна; если FrameRegister не ноль, база вычисляется через frame pointer”. Именно такой текст нужен в хорошей диагностике.

Scaled offset (масштабированное смещение) - еще один важный термин. Для обычных сохранений целочисленных регистров короткая форма хранит offset/8, потому что интересующие значения лежат на 8-байтовых границах. Для XMM128 короткая форма хранит offset/16, потому что 128-битные XMM значения требуют 16-байтового выравнивания. Это экономит место: вместо хранения полного числа можно хранить число, деленное на размер ячейки.

```
Если UWOP_SAVE_NONVOL хранит значение offset/8:

дополнительный слот = 0004h
реальное смещение   = 0004h * 8 = 20h

Если UWOP_SAVE_XMM128 хранит значение offset/16:

дополнительный слот = 0002h
реальное смещение   = 0002h * 16 = 20h
```

Short offset (короткое смещение) и far offset (дальнее/длинное смещение) отличаются размером кодирования. Если смещение маленькое и кратно нужному масштабу, можно использовать короткую форму. Если смещение большое, используется FAR-вариант, где дополнительные два слота вместе хранят более длинное число. Для парсера это означает разные размеры операций. Для валидатора - проверку, что выбранная форма разумна и соответствует ограничениям.

Почему UNWIND_CODE не описывает epilog

После предыдущего раздела об эпилоге может возникнуть вопрос: если UNWIND_CODE так хорошо описывает действия пролога, почему бы им не описывать и действия эпилога? Ответ уже частично был дан, но теперь его нужно связать с массивом кодов. Эпилогов у функции может быть много, а пролог обычно один. Если бы каждый эпилог требовал отдельного массива кодов, .xdata стала бы намного больше.

Windows x64 использует другой контракт. Пролог описывается таблично через UNWIND_CODE. Эпилог должен быть машинно распознаваемым по строгому шаблону. Поэтому UNWIND_CODE - это язык действий пролога, а не полный язык всей функции. Его задача - дать unwinder-у модель построенного кадра. Если управление в теле функции, эта модель применяется целиком. Если управление в прологе, применяется частично. Если управление в эпилоге, система распознает эпилог по коду и симулирует остаток выхода.

Это разделение делает формат компактным. Оно же накладывает ограничения на компилятор и hand-written assembly. Compiler (компилятор) должен строить такие прологи, которые можно описать UNWIND_CODE, и такие эпилоги, которые можно распознать без отдельной таблицы. Assembly programmer (программист на ассемблере) должен использовать директивы вроде .pushreg, .allocstack, .setframe, .savereg и .endprolog, чтобы MASM сгенерировал правильные .pdata и .xdata. Если написать работающий на вид ассемблер, но забыть unwind-директивы, обычный вызов может работать, а исключения и отладка будут ненадежными.

> UNWIND_CODE описывает “как функция построила кадр”. Эпилог отвечает на вопрос “как функция выходит из кадра”, но в Windows x64 он обычно не хранится отдельным массивом UNWIND_CODE, а распознается по допустимой форме машинного кода.

UNWIND_CODE и MASM-директивы: почему ассемблер должен говорить правду

MASM directives (директивы MASM) - это служебные команды ассемблера, которые не всегда сами становятся машинными инструкциями, но заставляют ассемблер и linker сгенерировать нужные метаданные. Для x64 unwind это особенно важно. Если ты пишешь функцию руками на ассемблере, Windows не может сама догадаться о всех твоих намерениях. Нужно явно сообщить: вот здесь я сохранил регистр, вот здесь выделил стек, вот здесь установил frame pointer, вот здесь закончился пролог.

Например, .pushreg rbp говорит не процессору, а инструментам сборки: соответствующий push rbp является сохранением nonvolatile-регистра и должен быть отражен как UWOP_PUSH_NONVOL. .allocstack 40h говорит: выделение стека должно быть отражено как UWOP_ALLOC_SMALL или UWOP_ALLOC_LARGE. .setframe rbp, 20h заполняет FrameRegister/FrameOffset и добавляет UWOP_SET_FPREG. .endprolog отмечает конец области, которая считается прологом.

```
Условная схема в MASM:

myfunc PROC FRAME
    push rbp
    .pushreg rbp
    sub rsp, 40h
    .allocstack 40h
    lea rbp, [rsp+20h]
    .setframe rbp, 20h
    .endprolog
```

```
    ; тело функции

    lea rsp, [rbp+20h]
    pop rbp
    ret
myfunc ENDP
```

Здесь машинные инструкции и unwind-директивы идут рядом, но выполняют разные роли. push rbp реально исполнится процессором. .pushreg rbp не исполнится процессором как команда программы. Она нужна, чтобы в .xdata появился соответствующий UNWIND_CODE. Это ключевое различие: код функции и описание функции живут в разных слоях, но должны говорить об одном и том же.

Если директива не соответствует реальной инструкции, возникает опасная ложь metadata. Например, в коде нет сохранения RBX, но .savereg rbx сообщает, что RBX сохранен по смещению. Тогда unwinder при раскрутке может прочитать из стека случайное значение и записать его в RBX контекста caller-a. Или наоборот: код сохранил RBX, но metadata об этом молчит. Тогда caller context будет восстановлен неполно. В обычном “happy path” функция может возвращаться правильно, но exception handling и debugging будут повреждены.

UNWIND_CODE как договор между compiler, linker и operating system

UNWIND_CODE можно рассматривать как contract (контракт) между несколькими участниками. Compiler (компилятор) генерирует пролог и знает, какие регистры он сохранил и сколько стека выделил. Assembler (ассемблер) и linker (компоновщик) оформляют это в .pdata/.xdata. Operating system runtime (системная среда выполнения) читает эти данные во время исключений, stack walking и отладки. Debugger (отладчик) использует ту же информацию, чтобы показать человеку цепочку вызовов.

Контракт означает: каждая сторона должна соблюдать свою часть. Компилятор не должен генерировать пролог, который нельзя описать допустимыми UWOP. Linker не должен испортить адреса и выравнивание. Loader (загрузчик) должен загрузить образ так, чтобы image-relative addresses можно было привести к реальным VA. Runtime должен корректно интерпретировать структуру. Reverse engineer должен не путать формат хранения с машинным кодом.

Когда мы говорим “UNWIND_CODE кодирует действия пролога”, слово “кодирует” не означает шифрует и не означает сжимает произвольный код без потерь. Encoding (кодирование) здесь - это перевод из мира машинных инструкций в мир формальной модели. Из множества деталей пролога выбираются только те, которые нужны для unwind. Они записываются в маленький набор UWOP. Получается компактное описание, достаточное для конкретной системной задачи.

В этом смысле UNWIND_CODE похож на технический чертеж. Чертеж двери не содержит запах краски, историю дерева и звук закрывания. Он содержит размеры, петли, точки крепления - то, что нужно для сборки и проверки. UNWIND_CODE не содержит всю функцию. Он содержит то, что нужно Windows, чтобы разобрать стековый кадр.

Как reverse engineer должен читать UNWIND_CODE глазами, а не только парсером

Механическое чтение байтов - только первый уровень. Reverse engineer должен уметь читать UNWIND_CODE как рассказ о прологе. Если в массиве есть UWOP_ALLOC_SMALL на 40h,

значит функция имеет фиксированное стековое выделение 64 байта. Если есть UWOP_PUSH_NONVOL RBX и RBP, значит функция использует и обязана восстановить эти регистры. Если есть UWOP_SET_FPREG, значит у функции есть frame pointer, и обращения к локальным данным могут идти относительно выбранного регистра. Если есть SAVE_XMM128, значит функция сохраняет 128-битный nonvolatile XMM-регистр, что часто связано с SIMD-кодом или ABI-требованиями.

Такой взгляд помогает даже без символов. PDB может отсутствовать. Имена функций могут быть сняты. Код может быть stripped (очищен от отладочной информации). Но unwind metadata все равно часто остается, потому что без нее Windows плохо обрабатывает исключения и строит стек. Поэтому UNWIND_CODE дает структурные признаки функции: большой ли кадр, есть ли frame pointer, какие регистры важны, есть ли необычные сохранения, похожа ли функция на обычный компиляторный output или на ручной assembly.

Но нужно соблюдать осторожность. UNWIND_CODE не говорит, что функция “делает по смыслу”. Он не говорит, что функция шифрует данные или парсит сеть. Он говорит о форме кадра. Это низкоуровневый структурный слой. Его сила в том, что он надежнее имен и комментариев, но его нельзя превращать в фантазии. Хороший вывод должен звучать так: “функция имеет большой fixed stack allocation, сохраняет RBX/RDI/RSI, использует frame pointer; это может указывать на сложную функцию или функцию с большим количеством локальных данных, но семантика требует анализа тела”.

В Ghidra, IDA, WinDbg и собственном static полезно показывать UNWIND_CODE рядом с дизассемблированным прологом. Тогда человек видит два слоя одновременно: реальные инструкции и табличную модель. Если они совпадают по смыслу, доверие к stack walking повышается. Если они расходятся, появляется finding (инженерное наблюдение), которое нужно объяснять и проверять.

Типичные ошибки понимания UNWIND_CODE

Первая ошибка - считать UNWIND_CODE машинным кодом. Из-за слова “code” кажется, что это инструкции процессора. На самом деле это кодировка действий для unwinder-а. Машинные инструкции лежат в .text, а UNWIND_CODE лежит внутри UNWIND_INFO в .xdata или аналогичной области.

Вторая ошибка - читать CodeOffset как RVA. CodeOffset - это однобайтовое смещение внутри пролога функции. Чтобы связать его с реальным VA, нужно взять начало функции из RUNTIME_FUNCTION BeginAddress, добавить ImageBase и добавить соответствующий offset. Сам CodeOffset не указывает в файл напрямую.

Третья ошибка - забыть, что массив отсортирован по убыванию CodeOffset. Человек читает массив и думает, что так пролог исполнялся. Обычно наоборот: массив удобен для обратной раскрутки. Если нужна реконструкция прямого пролога, записи часто придется мысленно перевернуть и еще учитывать дополнительные слоты.

Четвертая ошибка - считать CountOfCodes количеством операций. Это количество слотов. SAVE_NONVOL, SAVE_XMM128, ALLOC_LARGE и FAR-варианты используют дополнительные слоты. Padding для выравнивания тоже не является операцией.

Пятая ошибка - интерпретировать OpInfo без UnwindOp. OpInfo=3 может означать RBX, а может означать размер выделения 32 байта. Смысл определяется только парой UnwindOp + OpInfo.

Шестая ошибка - думать, что отсутствующая запись означает отсутствующую инструкция. Не каждая инструкция пролога важна для unwind. И наоборот, одна unwind-операция может соответствовать машинной форме, которую нужно распознать семантически, а не строковым сравнением.

Седьмая ошибка - забыть про FrameRegister. Смещения в операциях сохранения регистров зависят от базы кадра. Если frame pointer используется, простое “offset от RSP” может быть неверным. Нужно читать FrameRegister и FrameOffset из UNWIND_INFO.

Восьмая ошибка - игнорировать валидность регистров. Числовое кодирование позволяет записать много значений, но конкретная UWOP-операция может требовать nonvolatile-регистр. Если metadata заявляет странный регистр, это повод для проверки, а не повод молча строить красивый отчет.

Минимальная логика parser-a для UNWIND_CODE

Теперь можно сформулировать, как должен мыслить parser (синтаксический разборщик) для UNWIND_CODE. Parser - это часть программы, которая берет байты и превращает их в структурные объекты: поля, операции, размеры, смещения. Но для unwind одного синтаксиса мало. Нужен еще semantic layer (смысловой слой), который понимает ограничения и связь с прологом.

```
Упрощенная логика чтения UNWIND_CODE[]:

read UNWIND_INFO header
count = CountOfCodes
index = 0
while index < count:
    code_offset = bytes[index_slot].byte0
    b = bytes[index_slot].byte1
    unwind_op = b & 0x0F
    op_info   = b >> 4

    size_in_slots = size_of_operation(unwind_op, op_info)
    read payload slots if needed
    create semantic operation
    index += size_in_slots

aligned_count = (count + 1) & ~1
optional_area begins after aligned_count slots
```

Функция size_of_operation должна знать правила UWOP. PUSH_NONVOL, ALLOC_SMALL, SET_FPREG и PUSH_MACHFRAME занимают один слот. SAVE_NONVOL и SAVE_XMM128 занимают два. SAVE_NONVOL_FAR и SAVE_XMM128_FAR занимают три. ALLOC_LARGE занимает два или три в зависимости от OpInfo. Если parser не знает этих размеров, он не сможет корректно перейти к следующей операции.

После синтаксического разбора начинается валидация. Validation (проверка корректности) - это не просто “байты читаются без выхода за файл”. Нужно проверить, что CountOfCodes не выходит за границы UNWIND_INFO, что дополнительные слоты помещаются в массив, что порядок CodeOffset не нарушен, что CodeOffset не превышает SizeOfProlog, что операции используют допустимые регистры, что размеры выделения кратны правилам, что optional area начинается по правильному выравниванию.

Дальше начинается сопоставление с кодом. Для этого parser должен иметь доступ к дизассемблированному прологу. Он может проверить: есть ли примерно по CodeOffset инструкция, которая соответствует UWOP? Например, если есть UWOP_ALLOC_SMALL на 40h,

виден ли sub rsp, 40h или эквивалентная допустимая форма? Если есть UWOP_PUSH_NONVOL RBX, виден ли push rbx в прологе? Если есть UWOP_SET_FPREG, виден ли lea/mov, устанавливающий frame pointer?

> Хороший static не должен останавливаться на “байты распарсились”. Он должен идти дальше: байты -> операции -> смысл кадра -> сопоставление с прологом -> аккуратный finding с доказательствами.

Диагностические findings: как говорить о несоответствиях честно

Finding (инженерное наблюдение, диагностический признак) - это не приговор. В контексте dynamic/static finding должен фиксировать конкретный факт и объяснять, почему он важен. Например: “UNWIND_CODE заявляет UWOP_PUSH_NONVOL RBX с CodeOffset=1, но в дизассемблированном прологе по соответствующему месту не обнаружен push rbx или эквивалентное сохранение”. Это лучше, чем “плохой unwind”.

Есть разные уровни серьезности. Ошибка чтения за границы .xdata - сильный структурный риск: metadata может быть повреждена или умышленно сломана. Нарушение сортировки CodeOffset - тоже серьезный признак, потому что unwinder ожидает порядок. Несоответствие register save может быть результатом обфускации, ручного assembly, ошибки дизассемблера или нестандартного компилятора. Необычная секция для .xdata не обязательно ошибка, если RVA корректен и память читаема.

Хороший отчет должен отделять hard error (жесткую ошибку формата) от suspicious pattern (подозрительного паттерна) и от informational note (информационного замечания). Например, “CountOfCodes требует 6 слотов, но доступно только 4” - это hard error. “UNWIND_INFO лежит не в секции с именем .xdata” - чаще informational или suspicious, потому что имя секции не является главным источником истины. “UWOP_SAVE_NONVOL использует volatile-регистр” - suspicious/hard в зависимости от политики валидатора и контекста.

В книге мы постоянно возвращаемся к этой дисциплине, потому что reverse engineering легко превращается в догадки. UNWIND_CODE дает редкую возможность делать доказательные выводы. Но доказательность исчезает, если инструмент путает формат, адреса, смещения и семантику. Поэтому чем ниже уровень данных, тем аккуратнее должен быть язык отчета.

UNWIND_CODE и частично выполненный пролог: подробный мысленный эксперимент

Рассмотрим ситуацию медленно. Поток входит в функцию. Первая инструкция push rbx уже выполнилась. RSP уменьшился на 8, старое значение RBX лежит на вершине стека. Следующая инструкция sub rsp, 20h еще не выполнилась. В этот момент происходит исключение или аппаратное событие, и Windows получает CONTEXT с RIP, который указывает на место внутри пролога после push, но до полного выделения локального кадра.

Если unwinder применит полную модель тела функции, он решит, что 32 байта уже выделены. Тогда он увеличит RSP на 32, хотя RSP никогда не уменьшался на эти 32. После этого он попытается восстановить RBX из неправильного адреса. Return address тоже будет найден не там. Один неверный вывод о том, какая инструкция успела выполниться, ломает всю цепочку.

Если unwinder вообще не учтет push rbx, он оставит RSP слишком низко на 8 байт и не восстановит RBX. Это тоже неверно. Значит, нужна промежуточная логика: отменить push, но не отменять sub. Именно для этого массив отсортирован и снабжен CodeOffset. Он позволяет не мыслить пролог как “выполнен/не выполнен” целиком, а видеть его как последовательность уже завершенных действий.

```
Состояния условного пролога:

До входа:
    RSP -> return address

После push rbx:
    RSP -> saved RBX
           return address

После sub rsp, 20h:
    RSP -> local stack area
           ... 32 bytes ...
           saved RBX
           return address

Если исключение между push и sub:
    надо отменить push
    нельзя отменять sub
```

Это один из лучших способов понять, почему UNWIND_CODE хранит не только тип действия, но и offset. Без offset у Windows была бы только финальная модель кадра. Но финальная модель подходит для тела функции, а не для середины пролога. Windows x64 требует надежной обработки исключений at an arbitrary instruction (на произвольной инструкции), а значит метаданные должны позволять восстановление не только из красивого конечного состояния.

UNWIND_CODE и большие stack allocations

Stack allocation (выделение места на стеке) может быть маленьким и большим. Маленькое выделение кодируется UWOP_ALLOC_SMALL. Большое - UWOP_ALLOC_LARGE. Разница нужна ради компактности. Большинство функций выделяет сравнительно небольшой фиксированный кадр: десятки или сотни байт. Для них достаточно одного слота. Но функции с большими локальными массивами, крупными структурами или специальной логикой могут выделять больше.

UWOP_ALLOC_SMALL использует OpInfo как часть размера. Формула size = OpInfo * 8 + 8 дает диапазон от 8 до 128 байт. Почему шаг 8? Потому что стековые выделения в x64 ABI должны сохранять выравнивание, и интересующие размеры кратны 8. Вместо хранения полного числа формат хранит маленький индекс. Это пример инженерной экономии: одна тетрада дает 16 возможных размеров.

UWOP_ALLOC_LARGE использует дополнительные слоты. У него есть две формы. В одной форме следующий слот хранит size/8, что позволяет компактно описывать большие, но еще умеренные выделения. В другой форме два следующих слота вместе хранят большой размер без такого масштабирования. Для будущего подробного раздела про UWOP_* мы разберем границы и байты отдельно. Сейчас важно понять принцип: один и тот же смысл “уменьшили RSP на N” имеет разные кодировки в зависимости от размера N.

```
Выбор кодировки выделения стека в общем виде:

8..128 bytes:
    UWOP_ALLOC_SMALL, 1 slot
```

```
больше 128, но в пределах короткой large-формы:
    UWOP_ALLOC_LARGE, OpInfo=0, 2 slots

очень большое выделение:
    UWOP_ALLOC_LARGE, OpInfo=1, 3 slots
```

При анализе больших выделений нужно помнить про __chkstk. Если функция выделяет на стеке область размером около страницы памяти или больше, пролог может вызвать helper __chkstk (помощник проверки/проброса стека), прежде чем реально изменить RSP. Но unwind-код описывает итоговое выделение стека, а не каждую подготовительную инструкцию. Поэтому в дизассемблере ты можешь видеть mov rax, size, call __chkstk, sub rsp, rax, а в UNWIND_CODE - одну операцию выделения большого стека. Это нормально, если смысл согласован.

UNWIND_CODE и сохранение регистров через push и mov

Nonvolatile register (сохраняемый регистр) можно сохранить разными способами. Классический способ - push rbx. Он одновременно уменьшает RSP и записывает регистр на вершину стека. Для unwind это UWOP_PUSH_NONVOL. Другой способ - сначала выделить стек, а затем записать регистр в заранее выделенную ячейку через mov [rsp+offset], rbx. Для этого используются UWOP_SAVE_NONVOL или UWOP_SAVE_NONVOL_FAR.

Почему нужны оба способа? Потому что компилятор оптимизирует расположение данных и порядок действий. Push удобен и компактен в начале пролога. Но иногда регистр сохраняют не сразу на вершину, а в конкретное место кадра. Это может быть связано с shrink-wrapping (отложенным сохранением регистра только там, где он действительно нужен), с frame pointer, с удобством адресации или с особенностями сгенерированного кода.

Shrink-wrapping (сжатое/отложенное оборачивание сохранения, оптимизация отложенного сохранения регистров) - это оптимизация, при которой компилятор не сохраняет nonvolatile-регистр прямо в начале функции, если он нужен только в отдельной ветке. В Windows x64 такая ситуация может требовать chained unwind info или специальных записей сохранения. Для нас сейчас важно не углубляться в shrink-wrapping полностью, а понять: сохранение регистра не всегда выглядит как push в первой строке. Metadata должна описывать фактическое место сохранения.

```
Два способа сохранить RBX:

Вариант 1: через push
    push rbx
    ; UNWIND_CODE: UWOP_PUSH_NONVOL, OpInfo=RBX

Вариант 2: через mov в выделенный кадр
    sub rsp, 40h
    mov [rsp+20h], rbx
    ; UNWIND_CODE: UWOP_ALLOC_SMALL/LARGE
    ;              UWOP_SAVE_NONVOL, OpInfo=RBX, payload=20h/8
```

В варианте с mov важно понимать, что payload хранит не байты инструкции mov, а смещение места сохранения. RtlVirtualUnwind потом использует это смещение, чтобы прочитать сохраненный RBX из памяти стека и записать его в CONTEXT. Если смещение неверно, восстановится не RBX, а случайные 8 байт из кадра. Именно поэтому несоответствие SAVE_NONVOL опасно.

UNWIND_CODE и XMM-регистры

XMM register (XMM-регистр, 128-битный SIMD-регистр) - это регистр, который используется для операций с floating point (числами с плавающей точкой), SIMD (одна инструкция - много данных) и различными векторными значениями. В x64 ABI часть XMM-регистров считается volatile, а часть может требовать сохранения при использовании. Когда функция должна сохранить nonvolatile XMM-регистр, обычного 8-байтового SAVE_NONVOL уже недостаточно: XMM-регистр имеет 128 бит, то есть 16 байт.

Для этого существуют UWOP_SAVE_XMM128 и UWOP_SAVE_XMM128_FAR. Они похожи по идее на SAVE_NONVOL, но работают с 16-байтовыми значениями и используют масштаб 16 для короткой формы смещения. Это связано с выравниванием и размером XMM-значений. Если integer nonvolatile save работает с 8-байтовыми ячейками, то XMM128 save работает с 16-байтовыми ячейками.

В практическом реверсе наличие SAVE_XMM128 может быть полезным признаком. Функция использует векторные регистры так, что должна сохранять их состояние для caller-а. Это может встречаться в коде с математикой, графикой, криптографией, обработкой памяти, компиляторными оптимизациями. Но снова нельзя делать слишком сильный вывод: сам факт сохранения XMM не говорит, что функция “криптографическая”. Он говорит о форме использования регистров.

Для парсера отличие XMM-операций важно из-за размера offset scale. Если принять offset/16 за offset/8, адрес сохраненного XMM будет вычислен неправильно. Если забыть, что XMM save занимает дополнительный слот, следующий слот будет ложно декодирован как отдельная операция. Такие ошибки быстро портят весь разбор UNWIND_INFO.

Как связать CodeOffset с реальным адресом в дизассемблере

Когда ты смотришь на UNWIND_CODE в байтах, CodeOffset выглядит как маленькое число: 01h, 05h, 0Ah. Когда ты смотришь на функцию в Ghidra или IDA, ты видишь реальные адреса: например, 00007FF6`12341000, 00007FF6`12341001, 00007FF6`12341005. Чтобы связать эти два мира, нужно помнить три уровня адресации: BeginAddress в RUNTIME_FUNCTION является RVA начала функции, ImageBase дает базу загруженного модуля, а CodeOffset является смещением внутри пролога от начала функции.

```
Переход от CodeOffset к адресу инструкции рядом с прологом:

FunctionStartVA = ImageBase + RUNTIME_FUNCTION.BeginAddress
BoundaryVA      = FunctionStartVA + CodeOffset

Но BoundaryVA - это обычно адрес следующей инструкции после действия,
а не адрес начала инструкции, которая это действие выполнила.
```

Это различие между address of instruction (адресом инструкции) и boundary after instruction (границей после инструкции) особенно важно при визуальном сопоставлении. Если UNWIND_CODE показывает CodeOffset=05 для sub rsp, 20h, а sub начинается по offset 01, это не ошибка. Это означает, что действие считается завершенным на границе offset 05. Чтобы найти саму инструкцию, нужно смотреть назад на инструкцию, которая заканчивается перед этой границей.

В дизассемблере удобнее думать не “CodeOffset должен указывать на строку”, а “CodeOffset должен совпадать с концом строки”. Поэтому хороший static может показывать оба значения: boundary offset и candidate instruction range. Например: CodeOffset=0x05, candidate instruction [0x01..0x05): sub rsp, 20h. Такая подача сразу снимает путаницу и делает отчет понятнее.

Instruction range (диапазон инструкции) удобно записывать полуоткрытым интервалом [start, end). Start включается, end не включается. Если инструкция начинается по offset 1 и занимает 4 байта, ее диапазон [1,5). CodeOffset будет 5. В обычном машинном коде RIP указывает на текущую инструкцию, но unwind metadata часто удобнее работать с границами завершенных действий. Это еще один пример того, что формат отражает нужды unwinder-а, а не стиль чтения человеком.

Второй ручной пример: сохранение регистра через mov и payload-слот

Теперь разберем пример, где одна смысловая операция занимает два слота. Это нужно, чтобы закрепить различие между slot count (количеством слотов) и operation count (количеством операций). Пусть функция выделяет 64 байта стека и сохраняет RBX не через push, а в ячейку внутри выделенного кадра.

```
; условный пролог
00: sub  rsp, 40h
04: mov  [rsp+20h], rbx
09: ... тело функции ...
```

Здесь sub rsp, 40h выделяет 64 байта. 64 попадает в UWOP_ALLOC_SMALL: size = OpInfo*8+8, значит OpInfo=(64-8)/8=7. Если условно считать, что инструкция sub закончилась на offset 04, то CodeOffset для выделения будет 04.

Дальше mov [rsp+20h], rbx сохраняет RBX в фиксированную ячейку кадра. Это уже не UWOP_PUSH_NONVOL, потому что регистр сохранен не через push. Нужна операция UWOP_SAVE_NONVOL. Ее OpInfo хранит номер регистра RBX, то есть 3. Но одной этой информации мало: надо записать offset 20h. Короткая форма SAVE_NONVOL хранит offset/8, значит 20h/8 = 4. Это число 4 кладется в дополнительный слот.

```
Смысловые операции:

1) CodeOffset=09, UWOP_SAVE_NONVOL, OpInfo=3 (RBX)
   payload: 0004h, потому что real_offset = 0004h * 8 = 20h

2) CodeOffset=04, UWOP_ALLOC_SMALL, OpInfo=7
   size = 7*8 + 8 = 64 bytes
```

Теперь посчитаем слоты. UWOP_SAVE_NONVOL занимает два слота: основной и payload. UWOP_ALLOC_SMALL занимает один слот. Значит CountOfCodes будет 3, хотя смысловых операций только 2. И поскольку CountOfCodes нечетный, после этих трех слотов для выравнивания может быть занят еще один пустой слот перед optional area. Это не действие пролога.

```
Возможная раскладка байтов UNWIND_INFO:

01 09 03 00   09 34   04 00   04 72   00 00

01          Version=1, Flags=0
09          SizeOfProlog=9
03          CountOfCodes=3 слота
00          FrameRegister=0, FrameOffset=0
09 34       CodeOffset=9, Op=4 (SAVE_NONVOL), Info=3 (RBX)
04 00       payload: 0004h -> offset 20h
04 72       CodeOffset=4, Op=2 (ALLOC_SMALL), Info=7 -> 64 bytes
00 00       padding slot для выравнивания, не операция
```

Этот пример показывает, почему поверхностный hex-dump легко вводит в заблуждение. Если распечатать каждый двухбайтовый слот как отдельный UNWIND_CODE, payload 04 00

может выглядеть как CodeOffset=4, UnwindOp=0, OpInfo=0, то есть как будто там есть UWOP_PUSH_NONVOL RAX. Это ложный вывод. Эти байты принадлежат предыдущей операции SAVE_NONVOL как число. Правильный parser не имеет права декодировать payload как отдельную операцию.

Для отчета полезно выводить такие операции не сырым списком слотов, а сгруппированно: основной слот плюс payload. Тогда человек сразу видит: “SAVE_NONVOL RBX at +20h” - это одна операция, даже если в массиве она заняла два слота. Сырой список байтов тоже полезен, но только как evidence, а не как основной язык объяснения.

Третий пример: frame pointer и UWOP_SET_FPREG

Теперь добавим frame pointer. Frame pointer (указатель кадра) нужен не каждой функции. Но если функция его использует, UNWIND_INFO должен отразить это двумя способами: в заголовке через FrameRegister/FrameOffset и в массиве UNWIND_CODE через UWOP_SET_FPREG. Иначе unwinder не поймет, от какой базы считать некоторые смещения и как восстановить RSP в сложных случаях.

```
; условный пролог с frame pointer
00: push rbp
01: sub  rsp, 80h
08: lea  rbp, [rsp+40h]
0D: ... тело функции ...
```

В этом примере RBP сохраняется через push, потом функция выделяет 128 байт стека, потом устанавливает RBP внутрь выделенной области: rbp = rsp + 40h. Такая установка frame pointer не означает, что RBP равен старому RSP. Он может указывать в середину кадра. Это делается ради более коротких инструкций доступа: если база находится ближе к локальным данным, некоторые обращения помещаются в короткие displacement-формы.

В UNWIND_INFO заголовок должен сказать, что frame register - это RBP, а frame offset равен 40h/16 = 4, потому что поле FrameOffset масштабировано на 16. В массиве UNWIND_CODE должна быть операция UWOP_SET_FPREG с CodeOffset, соответствующим завершению инструкции lea. У этой операции OpInfo зарезервирован и не должен использоваться как номер регистра. Сам регистр кадра берется из заголовка UNWIND_INFO, а не из OpInfo этой операции.

```
Смысловые элементы:

Header:
    FrameRegister = RBP (номер 5)
    FrameOffset   = 4  -> реальный offset 4*16 = 40h

UNWIND_CODE[]:
    UWOP_SET_FPREG     -> frame pointer установлен
    UWOP_ALLOC_LARGE/SMALL for 80h allocation
    UWOP_PUSH_NONVOL RBP
```

Здесь возникает важная методическая мысль: некоторые смыслы распределены между заголовком UNWIND_INFO и отдельными UNWIND_CODE. UWOP_SET_FPREG говорит “в этой точке пролога frame pointer установлен”. Но какой именно регистр и с каким смещением - это читается из FrameRegister и FrameOffset. Поэтому нельзя анализировать операцию в изоляции от всего UNWIND_INFO.

Если parser увидел UWOP_SET_FPREG, но FrameRegister=0, это подозрительная ситуация. Если FrameOffset не кратен логике инструкции, тоже вопрос. Если в прологе не видно lea/mov, который устанавливает соответствующий регистр, возникает finding. Но опять же, вывод

должен быть аккуратным: возможно, дизассемблер неправильно определил границы, возможно, есть необычная форма, возможно, metadata повреждена. Доказательство должно включать адреса, bytes и интерпретацию.

Как UNWIND_CODE помогает восстановить не только RSP, но и доверие к stack trace

Stack trace (цепочка вызовов) кажется обычной удобной функцией отладчика: нажал кнопку - увидел, кто кого вызвал. Но на x64 хороший stack trace часто стоит на unwind metadata. Если UNWIND_CODE корректен, отладчик может идти от кадра к кадру. Если UNWIND_CODE поврежден или неправильно интерпретирован, стек превращается в набор случайных адресов.

Представь crash dump без символов. В нем есть только память, регистры и загруженные модули. Отладчик берет текущий RIP, ищет RUNTIME_FUNCTION, переходит к UNWIND_INFO, читает UNWIND_CODE, вычисляет caller context, повторяет процесс. На каждом шаге UNWIND_CODE отвечает на вопрос: где теперь должен быть RSP, какие nonvolatile-регистры восстановить, откуда взять return address. Если один шаг ошибочен, все следующие кадры могут быть неверными.

Поэтому UNWIND_CODE важен не только для “обработки исключений” в узком смысле. Он важен для debugging (отладки), crash dump analysis (анализа аварийных дампов), profiling (профилирования), telemetry (телеметрии), sampling (сэмплирования), security monitoring (мониторинга безопасности). Все эти задачи могут нуждаться в ответе: где мы находимся в цепочке вызовов? А этот ответ зависит от правильной раскрутки.

Для твоего проекта это означает, что проверка UNWIND_CODE - не декоративная фича. Она может стать центральным источником доверия или недоверия к стековому анализу. Если metadata хорошая, static может строить более уверенный отчет. Если metadata странная, dynamic может подсветить область, где stack walking потенциально ненадежен, а значит дальнейшие выводы по control flow надо делать осторожнее.

Граница между ошибкой компилятора, ручным assembly и обфускацией

Когда UNWIND_CODE выглядит странно, нельзя автоматически кричать “обфускация”. У странности может быть несколько причин. Первая - hand-written assembly (код, написанный вручную на ассемблере). Автор мог правильно написать машинные инструкции, но неполно или неверно оформить unwind directives. Вторая - необычный compiler backend (задняя часть компилятора), который генерирует корректный, но непривычный код. Третья - JIT или dynamic function table, где metadata регистрируется во время выполнения. Четвертая - packer/dynamic/obfuscator, который сознательно меняет форму кода и данных.

Ошибка хорошего инструмента - смешать все эти случаи в один verdict. Лучше строить layered explanation (послойное объяснение). Сначала сказать, что именно наблюдается: например, “UWOP_SAVE_NONVOL указывает на offset, выходящий за фиксированное выделение” или “порядок CodeOffset нарушает ожидаемую сортировку”. Затем сказать, почему это важно для unwind. Затем дать возможные интерпретации: повреждение metadata, нестандартный генератор кода, ошибка парсера, обфускация. Затем предложить ручную проверку.

Такой стиль делает отчет профессиональным. Он не обещает больше, чем данные позволяют. В reverse engineering это важно: уверенность должна расти из evidence. UNWIND_CODE дает много evidence, но только если ты честно отделяешь факт от гипотезы.

Практический чек-лист чтения одного UNWIND_CODE-массива

Когда ты руками читаешь один UNWIND_INFO, удобно идти не хаотично, а по постоянному чек-листу. Сначала найти RUNTIME_FUNCTION и получить FunctionStart. Затем перейти по UnwindInfoAddress к UNWIND_INFO. Затем прочитать Version/Flags, SizeOfProlog, CountOfCodes, FrameRegister/FrameOffset. Только после этого читать UNWIND_CODE. Не наоборот. Иначе отдельные слоты будут висеть в воздухе без контекста.

```
Чек-лист ручного чтения:

1. FunctionStart = ImageBase + BeginAddress
2. UnwindInfoVA  = ImageBase + UnwindInfoAddress
3. Прочитать заголовок UNWIND_INFO
4. Проверить Version и Flags
5. Прочитать SizeOfProlog
6. Прочитать CountOfCodes как количество слотов
7. Прочитать FrameRegister / FrameOffset
8. Идти по UNWIND_CODE[] с учетом размера каждой операции
9. Для каждого CodeOffset связать boundary с инструкцией пролога
10. Для каждого payload вычислить реальный размер или offset
11. Проверить сортировку по убыванию CodeOffset
12. Проверить соответствие с дизассемблированным прологом
13. Вычислить optional area через выравнивание
14. Если есть handler/chained info - разбирать следующий слой
```

Такой чек-лист полезен не только человеку, но и архитектуре static . Внутри программы можно разделить parsing stage (этап разбора), validation stage (этап проверки), semantic reconstruction stage (этап восстановления смысла) и reporting stage (этап отчета). Тогда ошибки не смешиваются. Если байты не читаются, это проблема парсинга. Если читаются, но нарушают правила, это проблема валидации. Если правила соблюдены, но код не похож на metadata, это проблема согласованности. Если все согласовано, можно строить доверие к unwind-модели.

Именно так фундаментальная теория превращается в инженерный инструмент. Мы не просто заучиваем, что “UNWIND_CODE имеет три поля”. Мы строим процедуру: как из PE-файла перейти к функции, как прочитать unwind metadata, как понять стековый кадр, как доказать вывод и как не соврать в отчете.

UNWIND_CODE как мост к следующему разделу про UWOP_*

В этом разделе мы специально не разбираем каждую UWOP_* операцию до конца. Это будет отдельная большая часть. Сейчас задача другая: понять общий механизм кодирования. UNWIND_CODE - это слот или группа слотов внутри UNWIND_INFO. Он имеет CodeOffset, UnwindOp и OpInfo. Он может иметь payload. Он отсортирован по убыванию offset. Он описывает действия пролога, которые нужны для восстановления caller context.

Когда мы перейдем к UWOP_PUSH_NONVOL, UWOP_ALLOC_SMALL, UWOP_ALLOC_LARGE, UWOP_SET_FPREG, UWOP_SAVE_NONVOL, UWOP_SAVE_XMM128 и UWOP_PUSH_MACHFRAME по отдельности, важно уже не теряться в общем формате. Каждая операция будет просто частным случаем общей грамматики. Мы будем спрашивать: какое действие пролога она описывает? Сколько слотов занимает? Что означает OpInfo? Есть ли payload? Как вычисляется размер или смещение? Какие инструкции пролога обычно ей соответствуют? Какие ошибки

parser-а и reverse engineer-а здесь типичны?

Такой порядок обучения принципиален. Если сразу прыгнуть в таблицу UWOP, она будет выглядеть как набор магических сокращений. Если сначала понять, что UNWIND_CODE - это язык описания эффекта пролога, таблица UWOP становится словарем этого языка. Тогда каждый код перестает быть “цифрой из документации” и превращается в действие над стеком, регистрами и контекстом.

Почему UNWIND_CODE нельзя проверять только по таблице UWOP

Таблица UWOP_* сама по себе полезна, но она не делает анализ завершенным. Допустим, parser увидел UWOP_ALLOC_SMALL и правильно посчитал размер. Это только первый шаг. Дальше нужно понять, соответствует ли этот размер реальному прологу, не конфликтует ли он с frame pointer, не выходит ли сохраненный регистр за пределы разумной области кадра, не нарушен ли порядок CodeOffset, не начинается ли optional area посреди payload.

Иначе получится “табличный парсер”, который красиво распечатывает строки, но не понимает функцию. Такой инструмент может выглядеть убедительно в отчете, но не даст надежности. Фундаментальный подход другой: каждая UWOP-операция должна быть помещена в контекст функции. Контекст - это диапазон RUNTIME_FUNCTION, адрес начала функции, SizeOfProlog, дизассемблированный пролог, FrameRegister, CountOfCodes, выравнивание, optional flags и, если возможно, реальные инструкции вокруг CodeOffset.

Например, UWOP_SAVE_NONVOL RBX + offset 20h сам по себе выглядит нормально. Но если функция выделила только 10h байт фиксированного стека, а сохранение заявлено по 20h, возникает вопрос. Если используется frame pointer, вопрос может сняться или измениться, потому что база смещения другая. Если есть chained unwind info, часть логики может находиться в связанной записи. Поэтому проверка должна быть многослойной.

Это важный стиль мышления для reverse engineering: не оценивать структуру в изоляции. В Windows x64 unwind почти каждое поле получает окончательный смысл только рядом с другими полями. RUNTIME_FUNCTION дает диапазон. UNWIND_INFO дает общий заголовок. UNWIND_CODE дает действия. Дизассемблер дает фактические инструкции. Только вместе они дают инженерное заключение.

Синтаксис, семантика и доверие: три уровня разбора

Для зрелого анализатора полезно разделять три уровня: syntax (синтаксис), semantics (семантика) и trust (доверие). Syntax означает, что байты можно прочитать по формату: есть заголовок, есть нужное количество слотов, payload не выходит за границы, optional area вычисляется. Это самый нижний уровень. Он отвечает на вопрос: “похоже ли это на структуру UNWIND_INFO/UNWIND_CODE?”.

Semantics означает, что прочитанные поля имеют смысл: UWOP допустим, OpInfo корректен для операции, регистры допустимы, размеры кратны нужным значениям, CodeOffset укладывается в SizeOfProlog, операции не противоречат базовым правилам. Это уже не просто чтение байтов. Это понимание правил Windows x64 unwind.

Trust означает, что metadata согласована с кодом и может использоваться как надежная основа stack walking. Даже синтаксически корректная и семантически допустимая структура может быть подозрительной, если дизассемблированный пролог говорит другое. Например, metadata заявляет сохранение RBP, но в реальном прологе RBP не сохраняется. Или metadata

говорит о frame pointer, но код не устанавливает соответствующий регистр. Тогда формат “валиден”, но доверие к нему ниже.

```
Три уровня результата анализа:

Syntax OK:
    байты читаются, границы не нарушены

Semantics OK:
    UWOP, OpInfo, размеры, регистры и порядок допустимы

Trust OK:
    metadata согласована с реальным прологом и может использоваться
    для уверенного stack walking
```

Такое разделение особенно полезно для отчета. Пользователь инструмента должен понимать, где именно проблема. “Не удалось прочитать структуру” - одно. “Структура читается, но содержит недопустимый UWOP” - другое. “Структура формально допустима, но не совпадает с кодом” - третье. Хорошая книга должна учить именно этому различению, потому что оно превращает набор терминов в инженерную методологию.

Почему байтовая компактность формата усложняет ручной разбор

UNWIND_CODE очень компактен. На бумаге это красиво: два байта на основной слот, несколько дополнительных слотов для больших значений. Но компактность почти всегда переносит сложность на parser и на человека. Чем меньше явных полей, тем больше смысл зависит от контекста. OpInfo не подписан. Payload не подписан. Padding не подписан. Один и тот же двухбайтовый кусок может быть операцией, числом или выравниванием в зависимости от того, как к нему пришли.

Поэтому ручной разбор UNWIND_CODE похож не на чтение таблицы Excel, а на разбор маленького бинарного протокола. Binary protocol (бинарный протокол) - это формат, где смысл данных задается порядком байтов, битовыми полями, размерами и состоянием парсера. Если parser на одном шаге ошибся в размере операции, следующий байт будет прочитан не в той роли. Ошибка сдвигается дальше и превращает оставшуюся структуру в мусор.

Эта особенность важна при проектировании static . Нужно хранить не только финальный список операций, но и source span (исходный диапазон байтов) каждой операции: откуда она прочитана, какие слоты заняла, какие байты были payload. Тогда при ошибке можно показать человеку доказательство: “операция началась по offset +4 внутри UNWIND_INFO, заняла 3 слота, ожидала еще 2 байта payload, но структура закончилась”. Такой отчет намного полезнее, чем просто “parse failed”.

Source span (диапазон исходных байтов) - это адресный интервал внутри файла или памяти, из которого была получена структура. Для reverse engineering это почти всегда полезно. Если finding не связан с конкретными байтами и адресами, его трудно проверить. А если finding показывает RVA, file offset, raw bytes и интерпретацию, его можно открыть в hex viewer и подтвердить руками.

Как показывать UNWIND_CODE в отчете static /dynamic

Пользователь отчета не должен видеть только сырые байты. Но и полностью скрывать байты нельзя. Хорошая форма вывода должна иметь несколько уровней. Первый уровень - краткая смысловая строка: “alloc stack 0x40”, “save RBX at frame+0x20”, “set frame pointer

RBP+0x40”. Второй уровень - технические поля: CodeOffset, UWOP, OpInfo, payload, количество слотов. Третий уровень - evidence: RVA/VA, file offset и raw bytes.

```
Пример хорошего вывода одной операции:

[UNWIND] RVA 0x1234, slot 0
  semantic: save nonvolatile RBX at frame+0x20
  raw:      CodeOffset=0x09, UnwindOp=UWOP_SAVE_NONVOL(4), OpInfo=3
  payload: 0x0004 -> offset 0x20 (scaled by 8)
  bytes:    09 34 04 00
  prolog:   candidate instruction [Function+0x04..Function+0x09): mov [rsp+20h], rbx
```

Такой стиль вывода одновременно понятен и проверяем. Новичок читает semantic. Продвинутый человек смотрит raw. Реверсер открывает prolog и сверяет адреса. Если есть ошибка, можно понять, где именно она возникла: в чтении байтов, в вычислении scaled offset, в сопоставлении с инструкцией или в исходной metadata.

Для dynamic важно не только найти проблему, но и объяснить ее риск. Например, если UNWIND_CODE не согласован с прологом, риск не в том, что “функция плохая”, а в том, что stack unwinding через эту функцию может быть недостоверным. Это влияет на exception handling, crash dump, stack trace и любые защитные механизмы, которые используют цепочку вызовов. Формулировка риска должна быть конкретной.

Место UNWIND_CODE в общей лестнице книги

Если посмотреть на всю линию курса, UNWIND_CODE находится ровно в середине между “байтами файла” и “поведением Windows при исключении”. Ниже него - PE, RVA, секции, .pdata, .xdata, RUNTIME_FUNCTION, UNWIND_INFO. Выше него - RtlLookupFunctionEntry, RtlVirtualUnwind, DispatcherContext, search phase, unwind phase, language-specific handlers. Поэтому слабое понимание UNWIND_CODE ломает обе стороны.

Если не понимать нижний уровень, ты не сможешь правильно прочитать байты. Будешь путать RVA и file offset, CountOfCodes и operation count, payload и отдельную операцию. Если не понимать верхний уровень, ты не поймешь, зачем эти байты нужны Windows. Будет казаться, что это просто таблица ради таблицы. На самом деле UNWIND_CODE - это точка, где бинарный формат превращается в действие системного runtime.

Именно поэтому мы так долго задерживаемся на этой теме. Для обычного справочника хватило бы одной таблицы. Для фундаментального труда нужно показать внутреннюю причинность: почему формат именно такой, какую проблему он решает, где экономит место, где требует строгих правил, почему ошибка одного nibble может разрушить стековый анализ и как это переносится в практический инструмент.

Итог: что нужно унести из раздела

UNWIND_CODE - это компактная запись действия пролога внутри UNWIND_INFO. Она не является машинной инструкцией и не заменяет дизассемблер. Ее задача - описать только те эффекты пролога, которые нужны Windows x64 для virtual unwind: изменение RSP, сохранение nonvolatile-регистров, установка frame pointer, сохранение XMM-регистров и специальные случаи вроде machine frame.

Каждый основной слот UNWIND_CODE занимает два байта: CodeOffset и байт с UnwindOp/OpInfo. CodeOffset отвечает за связь действия с местом в прологе и особенно важен для частично выполненного пролога. UnwindOp выбирает тип UWOP-операции. OpInfo дает дополнительный смысл, который зависит от конкретного UnwindOp. Некоторые операции

занимают дополнительные слоты, поэтому CountOfCodes нужно понимать как количество слотов, а не обязательно количество смысловых действий.

Массив UNWIND_CODE отсортирован по убыванию CodeOffset, потому что unwind отменяет пролог в обратном порядке. Это позволяет RtlVirtualUnwind восстанавливать caller context как из тела функции, так и из середины пролога. Эпилог при этом обычно не описывается таким массивом: он распознается по строгому машинному шаблону.

Для reverse engineering UNWIND_CODE ценен тем, что дает формальную модель стекового кадра даже без PDB и имен функций. Но эту модель нельзя читать грубо. Нужно различать RVA, file offset и CodeOffset; понимать payload-слоты; учитывать FrameRegister; проверять допустимость регистров; сопоставлять metadata с реальным прологом не по строкам, а по смысловым действиям. Только тогда static /dynamic сможет выдавать не шум, а инженерно полезные findings.

Следующий раздел будет естественным продолжением: мы возьмем каждую основную операцию UWOP_* отдельно и разберем ее не как строку из таблицы, а как маленький механизм: что она кодирует, какую инструкцию пролога обычно отражает, сколько слотов занимает, как вычисляет размер или offset, как ее должен читать parser и какие ошибки при ней чаще всего возникают.

4.4. Все основные UWOP_* операции по одной и очень подробно

К этому месту мы уже подошли не с пустыми руками. Сначала мы разобрали, что у функции на x64 есть пролог: начальный служебный участок, где функция подготавливает свой стековый кадр, сохраняет нужные регистры и приводит RSP к состоянию, удобному для дальнейшей работы. Потом мы разобрали эпилог: завершающий служебный участок, где функция возвращает стек и регистры к виду, ожидаемому вызывающей стороной. После этого мы перешли к UNWIND_CODE и увидели, что Windows хранит не весь машинный код пролога, а компактную запись о тех действиях пролога, которые важны для обратного восстановления состояния.

Теперь нужно сделать следующий шаг. Мало знать, что внутри UNWIND_INFO есть массив UNWIND_CODE. Нужно понимать, какие именно операции этот массив может выражать. Эти операции и называются UWOP_*.

UWOP_* (unwind operation, операция раскрутки/операция unwind-описания) - это не машинная инструкция процессора. Это элемент маленького формального языка, которым компилятор, ассемблер или линкер рассказывает Windows: "в прологе функции произошло вот такое действие, и при unwind это действие нужно отменить вот таким способом". Если машинная инструкция относится к реальному исполнению программы процессором, то UWOP относится к описанию последствий этого исполнения для механизма восстановления кадра.

Именно здесь часто ломается понимание. Новичок смотрит на UWOP_PUSH_NONVOL и думает: "это push". Почти правильно, но не до конца. В PE-файле внутри .xdata не лежит инструкция push rbx. Там лежит запись, что в прологе когда-то был выполнен push некоторого nonvolatile-регистра, и что при unwind старое значение этого регистра нужно взять из стека, а RSP сдвинуть назад. То есть UWOP описывает не сам байт машинного кода, а смысловое действие пролога с точки зрения восстановления caller context.

Caller context (контекст вызывающей функции) - это состояние, которое должно получиться после того, как система мысленно выйдет из текущей функции к её caller-у. В этот контекст входят RIP вызывающей стороны, RSP вызывающей стороны и значения nonvolatile-регистров, которые по ABI должны быть сохранены callee-функцией. Вся логика UWOP_* существует ради восстановления именно этого состояния, а не ради красивого отображения ассемблера.

Поэтому эту главу нужно читать не как список enum-значений. Если воспринимать её как справочник "opcode 0 значит одно, opcode 1 значит другое", картина останется плоской. Правильнее читать её как описание маленькой модели исполнения: функция вошла, сдвинула стек, сохранила регистры, иногда поставила frame pointer, иногда сохранила XMM-регистры, иногда описывает специальный машинный кадр. А RtlVirtualUnwind потом идёт по этой модели в обратную сторону и восстанавливает состояние, которое было до входа в функцию.

UWOP_* (unwind operation code, код операции раскрутки) - это код абстрактного действия пролога. Он не обязан совпадать один к одному с байтами машинной инструкции. Его задача - достаточно точно описать, как пролог изменил RSP и где сохранил nonvolatile-состояние, чтобы Windows могла восстановить caller context при исключении, stack trace или crash dump

analysis.

В этой главе мы будем идти медленно. Сначала закрепим общие правила, которые одинаковы для всех UWOP-операций. Затем разберём каждую основную операцию: что она означает, какая реальная инструкция обычно за ней стоит, какие поля UNWIND_CODE она использует, сколько slots занимает, как RtlVirtualUnwind отменяет её действие, какие ошибки делает ручной parser и какие аномалии должен видеть static /dynamic.

Важно заранее принять ограничение: UWOP-язык описывает не всё, что может быть в прологе. Он описывает только то, что имеет значение для unwind. Если пролог скопировал аргумент из RCX в другой регистр, это может быть важно для логики функции, но обычно не важно для восстановления caller context. Если пролог выполнил push rbx, это уже важно: RBX является nonvolatile-регистром, и caller вправе ожидать, что после возврата из callee его значение останется прежним. Если пролог выполнил sub rsp, 40h, это тоже важно: без знания этого размера нельзя найти return address вызывающей функции.

Таким образом, UWOP_* - это фильтр смысла. Из всего машинного кода пролога он оставляет только те действия, которые меняют стековый кадр или сохраняют состояние, принадлежащее вызывающей стороне.

Общее правило: UWOP описывает обратимое действие пролога

Чтобы правильно понимать каждую конкретную операцию, нужно держать в голове одну общую идею: unwind не повторяет пролог, а отменяет его эффект.

Когда функция реально исполняется, процессор идёт вперёд: push уменьшает RSP и записывает значение в стек; sub rsp, N уменьшает RSP и выделяет место; mov [rsp+offset], rbx сохраняет RBX в уже выделенном участке; lea rbp, [rsp+20h] создаёт frame pointer. Всё это обычное прямое исполнение.

Когда Windows выполняет unwind, она мысленно идёт назад. Ей нужно восстановить состояние до входа в текущую функцию. Поэтому эффект каждой операции обращается: если пролог уменьшил RSP, unwind увеличит RSP; если пролог сохранил регистр в стек, unwind прочитает регистр из этого места; если пролог сделал push, unwind возьмёт сохранённое значение из [RSP] и затем сдвинет RSP на 8.

Здесь полезно отделять два уровня. Первый уровень - CPU-level action (действие уровня процессора): реальная инструкция, которую выполнил процессор. Второй уровень - unwind-level action (действие уровня раскрутки): абстрактное описание, достаточное для восстановления. Эти уровни связаны, но они не одно и то же. Один и тот же UWOP может соответствовать немного разным машинным инструкциям, если итоговый эффект для unwind одинаков. И наоборот, одна машинная инструкция может быть не описана в unwind-данных, если она не влияет на RSP и nonvolatile-состояние.

Главная проверка понимания: если ты видишь UWOP, спрашивай не "какая это инструкция?", а "какое изменение состояния пролога эта запись позволяет отменить?".

Slot, payload и почему некоторые операции занимают больше двух байт

UNWIND_CODE slot (слот UNWIND_CODE, минимальная ячейка unwind-массива) обычно занимает два байта. Первый байт - CodeOffset. Второй байт делится на две половины: младшие 4 бита содержат UnwindOp, старшие 4 бита содержат OpInfo. Такой формат кажется маленьким, но он специально сделан плотным: в большинстве функций unwind-информация должна быть компактной, потому что таких функций в модуле могут быть тысячи.

Но не каждую операцию можно описать одним slot-ом. Например, UWOP_ALLOC_SMALL может уместить размер выделения стека в OpInfo. Размер там кодируется формулой OpInfo * 8 + 8, поэтому для выделения от 8 до 128 байт хватает одного slot. А вот если функция выделяет 0x300 байт, четырёх бит OpInfo уже недостаточно. Тогда используется UWOP_ALLOC_LARGE, и дополнительные данные размера кладутся в следующие slot-ы массива.

Payload slot (слот полезной нагрузки, дополнительный слот операнда) - это slot, который выглядит как UNWIND_CODE, но в конкретной операции не является самостоятельной операцией. Он хранит число: смещение, размер или часть большого значения. Это очень важный момент для parser-a. Если decoder идёт по массиву и считает каждый slot отдельной операцией, он ошибётся на первом же UWOP_SAVE_NONVOL или UWOP_ALLOC_LARGE. Правильный decoder сначала читает UnwindOp, затем по таблице операции понимает, сколько дополнительных slots нужно пропустить и как их интерпретировать.

CountOfCodes (количество unwind slots) считает именно slots, а не смысловые операции. Если в массиве есть одна операция UWOP_SAVE_NONVOL, она занимает два slots: первый сообщает тип операции и регистр, второй хранит offset. CountOfCodes увеличивается на оба. Поэтому выражение "в UNWIND_INFO пять кодов" на практике значит "в массиве пять slots", а не "пять независимых действий".

```
Один обычный slot UNWIND_CODE:

byte 0: CodeOffset
byte 1: high nibble = OpInfo, low nibble = UnwindOp

Пример второго байта 0x42:
0x2 = low nibble -> UnwindOp = UWOP_ALLOC_SMALL
0x4 = high nibble -> OpInfo = 4
size = 4 * 8 + 8 = 40 bytes

Если операция требует payload, следующий slot читается не как новая операция,
а как часть операнда текущей операции.
```

CodeOffset: почему операция привязана к концу инструкции

CodeOffset (смещение кода, смещение внутри пролога) - это смещение от начала функции до конца инструкции, которая выполнила данное действие, плюс один в терминологии документации Microsoft: фактически это позиция следующей инструкции после описываемого действия. Смысл такого поля особенно важен в ситуации partial prolog execution (частичного выполнения пролога).

Partial prolog execution (частичное выполнение пролога) - это ситуация, когда исключение или прерывание произошло не после всего пролога, а внутри него. Например, функция уже выполнила push rbp, но ещё не успела выполнить sub rsp, 40h. Или уже выделила

стек, но ещё не сохранила RBX. В таком случае нельзя применять весь UNWIND_INFO целиком. Нужно отменить только те действия пролога, которые к моменту исключения действительно успели произойти.

Именно поэтому каждая UWOP-операция имеет CodeOffset. RtlVirtualUnwind сравнивает текущий ControlPc/RIP с размером пролога и со смещениями операций. Если RIP находится внутри пролога, unwinder выбирает только применимые операции. Без CodeOffset система знала бы список всех действий пролога, но не знала бы, какие из них уже исполнены на конкретной точке.

Для reverse engineer-а это означает простую вещь: CodeOffset надо связывать с дизассемблером не абстрактно, а по границам инструкций. Если instruction boundary (граница инструкции) определена неверно, то связь между UNWIND_CODE и реальным кодом тоже станет неверной. Поэтому хороший static /dynamic не просто печатает CodeOffset, а показывает: BeginAddress + CodeOffset попадает после такой-то инструкции пролога.

OpInfo: маленькое поле с разными смыслами

OpInfo (operation info, уточняющая информация операции) - это старшие 4 бита второго байта UNWIND_CODE. Его нельзя переводить одним универсальным правилом. У разных UWOP-операций OpInfo имеет разный смысл.

В UWOP_PUSH_NONVOL OpInfo хранит номер регистра. В UWOP_ALLOC_SMALL OpInfo хранит размер выделения в закодированной форме. В UWOP_ALLOC_LARGE OpInfo выбирает один из двух режимов кодирования большого размера. В UWOP_SET_FPREG OpInfo зарезервирован и не должен использоваться. В UWOP_PUSH_MACHFRAME OpInfo сообщает, есть ли error code в машинном кадре.

Это типичный пример packed binary format (упакованного бинарного формата). Поле маленькое, но контекстно-зависимое. Нельзя написать parser, который просто выводит OpInfo как число и считает работу законченной. Parser обязан сначала определить UnwindOp, затем применить правила именно этой операции.

```
Один и тот же OpInfo значит разное:

UWOP_PUSH_NONVOL: OpInfo = номер GPR-регистра
UWOP_ALLOC_SMALL: OpInfo = size / 8 - 1
UWOP_ALLOC_LARGE: OpInfo = режим 0 или режим 1
UWOP_SET_FPREG: OpInfo = reserved, должен быть 0
UWOP_PUSH_MACHFRAME: OpInfo = 0 без error code, 1 с error code
```

Номера регистров в OpInfo

Когда OpInfo кодирует general-purpose register (регистр общего назначения), используется стандартная нумерация регистров, которую Microsoft показывает для unwind data. Здесь важно не просто запомнить таблицу, а понять её роль. UNWIND_INFO не хранит строку "RBX" или "R12". Он хранит маленькое число от 0 до 15. Уже decoder должен превратить это число в имя регистра.

General-purpose register, GPR (регистр общего назначения) - это обычный целочисленный регистр x64: RAX, RCX, RDX, RBX, RSP, RBP, RSI, RDI, R8-R15. В контексте unwind нас интересуют прежде всего nonvolatile GPR: RBX, RBP, RSI, RDI, R12, R13, R14, R15. Именно их

callee обязан сохранить, если хочет использовать.

Nonvolatile register (невытесняемый/сохраняемый регистр) - это регистр, значение которого должно сохраниться через вызов функции согласно calling convention. Caller может положить туда важное значение, вызвать функцию и ожидать, что после возврата значение останется тем же. Поэтому если callee временно использует такой регистр, она должна сохранить старое значение в стеке и восстановить его в эпилоге. UWOP_* операции фиксируют именно такие сохранения.

Volatile register (вытесняемый/несохраняемый регистр) - это регистр, который callee может свободно испортить. Caller не имеет права рассчитывать, что после вызова RCX, RDX, R8, R9, RAX, R10, R11 сохранят прежние значения. Поэтому обычная unwind-информация не обязана восстанавливать volatile-регистры. Если ты видишь попытку описать volatile-регистр как nonvolatile-save, это требует осторожной проверки: это может быть нетипичная генерация, специальный код, ошибка декодера или намеренная аномалия.

Код	Регистр	Комментарий для unwind
0	RAX	volatile; обычно не должен требовать сохранения ради ABI
1	RCX	volatile; также первый integer-аргумент в Windows x64
2	RDX	volatile; второй integer-аргумент
3	RBX	nonvolatile; частый кандидат для UWOP_PUSH_NONVOL/SAVE_NONVOL
4	RSP	stack pointer; как сохраняемый регистр в этих операциях обычно не имеет смысла
5	RBP	nonvolatile; часто используется как frame pointer
6	RSI	nonvolatile
7	RDI	nonvolatile
8-15	R8-R15	R8-R11 volatile, R12-R15 nonvolatile; для unwind особенно важны R12-R15

Важная тонкость: таблица кодов регистров шире, чем набор регистров, которые разумно сохранять как nonvolatile. Формат позволяет закодировать номер, но ABI говорит, какие регистры действительно должны сохраняться callee. Поэтому validator должен разделять parse-level validity и ABI-level suspiciousness.

Stack offset: откуда считается смещение сохранённого значения

Многие UWOP-операции говорят не только "какой регистр сохранён", но и "где в стековом кадре лежит его сохранённое значение". Для этого используется stack offset (стековое смещение). Однако это смещение нельзя читать как "адрес от текущего RSP на момент исключения". Это распространённая и опасная ошибка.

В документации Microsoft смещение для операций сохранения считается от base of fixed stack allocation (базы фиксированного выделения стека). Если FrameRegister в UNWIND_INFO равен нулю, базой фактически является RSP после fixed allocation. Если FrameRegister ненулевой, ситуация сложнее: base вычисляется относительно того

положения RSP, которое было в момент установки frame pointer, с учётом FrameOffset из заголовка UNWIND_INFO.

Fixed stack allocation (фиксированное выделение стека) - это участок, который функция выделяет в прологе обычным статическим способом, например sub rsp, 40h. Он называется fixed не потому, что память физически неподвижна, а потому что размер известен из unwind-метаданных и не зависит от runtime-значения. В отличие от него dynamic allocation (динамическое выделение стека), например через alloca, может менять RSP уже после пролога и обычно требует frame pointer, чтобы unwind оставался возможным.

Для операций SAVE_NONVOL и SAVE_XMM128 смещение дополнительно масштабируется. У обычных целочисленных сохранений stack values лежат на 8-байтовых границах, поэтому short form хранит offset / 8. У XMM128 сохраняются 128 бит, то есть 16 байт, и требуется 16-байтовое выравнивание, поэтому short form хранит offset / 16. Если забыть про масштабирование, parser покажет неправильный адрес сохранённого регистра.

```
Смысл масштабирования:

UWOP_SAVE_NONVOL:
payload = offset / 8
real offset = payload * 8

UWOP_SAVE_XMM128:
payload = offset / 16
real offset = payload * 16

FAR-версии:
payload хранит полный unscaled offset
умножать на 8 или 16 уже нельзя
```

UWOP_PUSH_NONVOL: сохранение nonvolatile-регистра через push

UWOP_PUSH_NONVOL (push nonvolatile register, помещение сохраняемого регистра в стек) - операция, которая описывает push nonvolatile-регистра в прологе. Она означает: пролог уменьшил RSP на 8 и записал в стек старое значение регистра, который callee обязан потом восстановить.

Начнём с самой интуитивной операции. В классическом прологе часто встречается инструкция вида push rbp, push rbx, push rsi или push rdi. Машинная инструкция push делает сразу два действия: уменьшает RSP на 8 байт и записывает значение регистра по новому адресу [RSP]. Для обычного исполнения это просто способ сохранить значение. Для unwind это обещание: старое значение регистра лежит на вершине того стекового участка, который появился из-за push.

UWOP_PUSH_NONVOL говорит системе: "в прологе был сохранён nonvolatile GPR через push". OpInfo хранит номер регистра. Дополнительных payload slots нет. Одна операция занимает один slot. При полном unwind, когда нужно выйти из тела функции к caller-у, RtlVirtualUnwind концептуально читает значение регистра из [RSP], записывает его обратно в соответствующее поле CONTEXT и затем увеличивает RSP на 8.

CONTEXT (контекст процессора, структура состояния регистров) - это структура Windows, где хранятся значения регистров на момент исключения или на текущем шаге unwind. Когда unwinder восстанавливает RBX, RBP или R12, он не "выполняет pop" процессором.

Он изменяет поля структуры CONTEXT так, как будто pop был выполнен при обратном проходе.

Почему операция называется PUSH_NONVOL, а не просто PUSH? Потому что для unwind важен не любой push, а push регистра, который обязан быть сохранён по ABI. Пролог может временно использовать стек для разных вещей, но unwind-метаданные фиксируют именно то, что нужно восстановить для caller context. Если ассемблерный код делает push rax просто как временное действие, это не должно превращаться в UWOP_PUSH_NONVOL как сохранение ABI-состояния, потому что RAX volatile и caller не ждёт его сохранности.

Порядок UWOP_PUSH_NONVOL имеет особое значение. Microsoft указывает, что из-за ограничений эпилога такие push-операции должны появляться первыми в прологе и, соответственно, последними в unwind-code array, потому что массив отсортирован по убыванию CodeOffset. Это выглядит странно, пока не вспомнить, что массив хранится не в порядке "как новичок читает сверху вниз", а в порядке, удобном для частичного unwind. Если push был самым ранним действием, его CodeOffset маленький, а операции с большими offset идут раньше в массиве.

```
Пример реального пролога:

00: push rbp ; RSP -= 8, [RSP] = old RBP
01: push rbx ; RSP -= 8, [RSP] = old RBX
02: sub rsp, 40h ; fixed stack allocation

Возможные unwind-операции по смыслу:

CodeOffset=01, UWOP_PUSH_NONVOL, OpInfo=RBP
CodeOffset=02, UWOP_PUSH_NONVOL, OpInfo=RBX
CodeOffset=06, UWOP_ALLOC_SMALL, OpInfo=7 ; 7*8+8 = 64 bytes

В массиве UNWIND_CODE они будут отсортированы по убыванию CodeOffset:
ALLOC_SMALL, PUSH_RBX, PUSH_RBP.
```

Для reverse engineering здесь есть две важные проверки. Первая: действительно ли регистр из OpInfo относится к разумному набору nonvolatile-регистров? Если OpInfo кодирует RCX или RAX, это не обязательно мгновенно "вредоносность", но это точно повод вывести diagnostic: metadata describes volatile register as pushed nonvolatile. Вторая: соответствует ли CodeOffset реальной инструкции push в прологе. Если .xdata говорит, что был push rbx, а дизассемблер по этому месту показывает совсем другую инструкцию, возникает несоответствие между code layer и metadata layer.

Metadata layer (слой метаданных) - это слой описания программы, который существует рядом с машинным кодом: .pdata, .xdata, RUNTIME_FUNCTION, UNWIND_INFO, UNWIND_CODE. Code layer (слой кода) - это реальные инструкции в .text. В нормальном PE эти два слоя должны объяснять друг друга. В реверсе особенно ценны места, где они расходятся.

Для static /dynamic UWOP_PUSH_NONVOL даёт простой, но сильный сигнал: какой nonvolatile-регистр функция сохраняет через push, на каком offset пролога это происходит и соответствует ли это реальной инструкции. Но этот сигнал нельзя оценивать в отрыве от ABI и от дизассемблированного пролога.

UWOP_ALLOC_SMALL: маленькое фиксированное выделение стека

UWOP_ALLOC_SMALL (small stack allocation, малое выделение места на стеке) - операция, которая описывает уменьшение RSP на небольшой фиксированный размер от 8 до 128 байт. Размер кодируется прямо в OpInfo по формуле size = OpInfo * 8 + 8.

После сохранения регистров функция часто выделяет место под локальные переменные, временные значения, spill slots, alignment padding и иногда место для outgoing arguments. В машинном коде это обычно выглядит как sub rsp, N. Для процессора это обычная арифметика: RSP уменьшается на N. Для calling convention это создание стекового кадра. Для unwind это критическое изменение: return address caller-а теперь находится не по [RSP], а выше, на расстоянии выделенного участка и сохранённых регистров.

UWOP_ALLOC_SMALL используется тогда, когда размер выделения помещается в диапазон 8-128 байт. OpInfo хранит не сам размер, а compact encoding: size / 8 - 1. Поэтому OpInfo=0 означает 8 байт, OpInfo=1 означает 16 байт, OpInfo=7 означает 64 байта, OpInfo=15 означает 128 байт. Такой способ экономит место: один slot полностью описывает и тип операции, и размер выделения.

Важно понимать, что ALLOC_SMALL описывает fixed allocation, а не каждый байт, который функция когда-либо положит в стек. Если после sub rsp, 40h функция записывает локальные переменные в [rsp+20h] или [rsp+28h], эти записи не становятся отдельными UWOP. Они не нужны для unwind. Системе достаточно знать общий размер выделенного кадра, чтобы правильно восстановить RSP.

При unwind действие UWOP_ALLOC_SMALL отменяется просто: RSP увеличивается на тот же размер. Если пролог сделал sub rsp, 40h, unwinder концептуально делает add rsp, 40h. Но опять же, это не реальное выполнение инструкции add процессором; это изменение поля Rsp в структуре CONTEXT.

```
UWOP_ALLOC_SMALL:

UnwindOp = 2
OpInfo = size / 8 - 1
slots = 1
range = 8..128 bytes
unwind = RSP += size

Пример:
OpInfo = 4
size = 4 * 8 + 8 = 40 bytes

Реальный пролог мог содержать:
sub rsp, 28h
```

Здесь нужно отдельно не путать stack allocation (выделение стека callee-функцией) и shadow space (теневое пространство вызова). Shadow space в Windows x64 - это 32 байта, которые caller резервирует перед вызовом для первых четырёх register arguments. Оно связано с calling convention, но оно не обязательно равно тому, что callee описывает через UWOP_ALLOC_SMALL. Когда ты видишь sub rsp, 28h в маленькой функции, там может быть смесь выравнивания, локальных нужд и подготовки к call. Нельзя автоматически говорить: "это shadow space". UWOP_ALLOC_SMALL говорит только о факте: текущая функция уменьшила RSP на такой-то fixed size.

Для parser-а проверка ALLOC_SMALL выглядит простой, но и здесь есть ловушки. Размер должен быть кратен 8 по самой формуле. Если дизассемблер показывает sub rsp, 30h, а

.xdata кодирует 40h, это может быть ошибка твоего сопоставления, неправильная граница функции, chained info, необычная оптимизация или испорченная metadata. Правильный вывод должен быть осторожным: "unwind allocation size does not match observed prolog allocation", а не "файл вредоносный".

Ещё одна тонкость: ALLOC_SMALL не обязан быть единственной операцией в функции. До него могли быть push nonvolatile-регистров, после него мог быть set frame pointer или save nonvolatile через mov. При восстановлении порядка важно помнить: unwind array отсортирован по CodeOffset, а не обязательно группирует операции так, как удобно человеку.

UWOP_ALLOC_LARGE: большое фиксированное выделение стека

UWOP_ALLOC_LARGE (large stack allocation, большое выделение места на стеке) - операция, которая описывает fixed stack allocation больше диапазона UWOP_ALLOC_SMALL. Она имеет два режима кодирования и поэтому занимает либо 2, либо 3 slots.

Когда функции нужно больше 128 байт фиксированного стекового кадра, одного OpInfo уже недостаточно. Тогда используется UWOP_ALLOC_LARGE. Это та же смысловая операция, что и ALLOC_SMALL: пролог уменьшил RSP на фиксированный размер, а unwind должен увеличить RSP обратно. Разница только в масштабе и способе кодирования размера.

У UWOP_ALLOC_LARGE есть два режима. Если OpInfo равен 0, следующий slot хранит размер, делённый на 8. Такой режим покрывает выделения до 512K - 8 байт. Если OpInfo равен 1, следующие два slots хранят полный unscaled size в little-endian форме, что позволяет описывать размеры до 4GB - 8. Другие значения OpInfo для этой операции в классической схеме являются недопустимыми и должны вызывать diagnostic.

Little-endian (младший байт первым, порядок байтов little-endian) - это способ хранения многобайтовых чисел, при котором младшая часть числа лежит раньше в памяти. Для FAR/large-форм, где значение занимает два USHORT slots, decoder должен собрать их в правильном порядке. Если перепутать порядок, размер стекового кадра станет фантастическим или, наоборот, слишком маленьким.

Microsoft отдельно подчёркивает правило shortest possible encoding (самое короткое возможное кодирование): если размер можно описать через UWOP_ALLOC_SMALL, надо использовать его; если размер больше 128, но меньше 512K, используется UWOP_ALLOC_LARGE с OpInfo=0; если размер ещё больше, используется OpInfo=1. Для валидатора это не всегда hard error, но хорошая strict-ABI диагностика. Неканоническое кодирование может быть результатом нестандартного генератора, обфускации, ручной сборки или просто другого toolchain-а, но оно точно заслуживает отметки.

```
UWOP_ALLOC_LARGE, mode 0:

UnwindOp = 1
OpInfo = 0
slots = 2
payload = size / 8, хранится в следующем USHORT slot
range = 136 bytes .. 512K-8
unwind = RSP += size

Пример:
payload = 0x0026
```

```
size = 0x26 * 8 = 0x130 bytes

UWOP_ALLOC_LARGE, mode 1:

UnwindOp = 1
OpInfo = 1
slots = 3
payload = full unscaled size, два следующих USHORT slots, little-endian
unwind = RSP += size
```

Почему large allocation вообще появляется в реальных программах? Большой стековый кадр может быть нужен для крупных локальных массивов, временных буферов, spill-зон в сложной оптимизированной функции, сохранения большого состояния, обработки SIMD-данных или сгенерированного кода. В обычном прикладном C/C++ очень большие fixed allocations встречаются реже, но в системном коде, автоматической генерации, криптографических библиотеках и обфусцированном коде они возможны.

При очень больших выделениях компилятор может использовать __chkstk или похожую stack probing логику. Stack probing (пробивание стека, проверка страниц стека) нужно для корректного роста стека по страницам: нельзя одним движением RSP перепрыгнуть через guard page и ожидать, что ОС всё правильно расширит. Но UWOP_ALLOC_LARGE всё равно описывает итоговый fixed allocation, а не каждую внутреннюю инструкцию probing-последовательности. Это ещё один пример различия между code layer и unwind-level model.

Для parser-а UWOP_ALLOC_LARGE - один из главных источников ошибок. Нужно проверять, хватает ли slots. Если CountOfCodes заканчивается сразу после первого slot ALLOC_LARGE, нельзя читать следующий USHORT из памяти вслепую. Это не "необычный размер", а truncated unwind data. Нужно проверять OpInfo: 0 или 1. Нужно правильно масштабировать size в mode 0 и не масштабировать в mode 1. Нужно помнить, что payload slots не являются отдельными операциями и не должны попадать в список UWOP при человекочитаемом выводе как самостоятельные действия.

Для `static`/`dynamic` UWOP_ALLOC_LARGE должен выводиться особенно аккуратно: режим, число slots, исходное payload-значение, вычисленный размер, способ масштабирования и проверка bounds. В отчёте важно показывать не только "large allocation", но и как именно decoder пришёл к размеру.

UWOP_SET_FPREG: установка frame pointer

UWOP_SET_FPREG (set frame register, установка frame pointer-регистра) - операция, которая описывает момент, когда пролог установил frame register равным RSP плюс смещение из заголовка UNWIND_INFO.

UWOP_SET_FPREG часто кажется простой операцией, но на самом деле она связывает сразу несколько тем: стековый кадр, fixed allocation, dynamic stack changes, frame register field и FrameOffset в UNWIND_INFO header. Здесь нельзя ограничиться фразой "устанавливает RBP".

Frame pointer (указатель кадра, frame pointer-регистр) - это регистр, который внутри функции служит стабильной точкой отсчёта для доступа к стековому кадру. В x64 Windows frame pointer не обязателен для каждой функции. Многие функции работают только через

RSP, потому что fixed allocation известен и RSP после пролога остаётся стабильным. Но если функция делает dynamic allocation, использует alloca или хочет более удобные короткие смещения до локальных данных, frame pointer становится полезным.

Frame register (кадровый регистр, регистр кадра) - это поле UNWIND_INFO, где записано, какой именно регистр используется как frame pointer. Чаще всего это RBP, но формат допускает и другие nonvolatile-регистры в рамках правил. UWOP_SET_FPREG сам по себе не хранит номер регистра. Это принципиально: номер frame register лежит в заголовке UNWIND_INFO, а не в OpInfo данного slot-а. OpInfo у UWOP_SET_FPREG зарезервирован.

FrameOffset (смещение frame register, масштабированное смещение кадрового регистра) - это второе поле заголовка UNWIND_INFO. Оно хранит scaled offset: реальное смещение равно FrameOffset * 16. Смысл такой: frame pointer может указывать не на начало выделенного кадра, а внутрь него. Это помогает генерировать более короткие инструкции доступа к локальным переменным, потому что часть объектов будет доступна с маленькими положительными или отрицательными displacement.

```
Пример:

sub rsp, 60h
lea rbp, [rsp+20h]

B UNWIND_INFO:
FrameRegister = RBP
FrameOffset = 2 ; 2 * 16 = 0x20
UWOP_SET_FPREG ; OpInfo reserved

Смысл:
frame pointer установлен как RBP = RSP + 0x20
```

При unwind UWOP_SET_FPREG не означает "восстановить RBP из стека". Это другая операция. Если старый RBP был сохранён через push rbp или через mov [rsp+offset], rbp, то это будет описано отдельным UWOP_PUSH_NONVOL или UWOP_SAVE_NONVOL. UWOP_SET_FPREG говорит только о том, что в прологе был создан frame pointer как производная от RSP. Поэтому путать SET_FPREG с сохранением RBP нельзя.

Зачем unwinder-у знать момент установки frame pointer? Во-первых, если RIP находится внутри пролога до выполнения инструкции lea rbp, [rsp+offset], то frame pointer ещё не установлен, и нельзя использовать его для вычисления адресов сохранённых регистров. Во-вторых, операции сохранения с offset могут быть интерпретированы относительно frame base только после того, как frame pointer появился. Microsoft прямо отмечает: если FP register используется, операции, берущие offset, должны применяться только после того, как FP register установлен в прологе.

Для ручного анализа это даёт хороший критерий. Если UNWIND_CODE показывает SAVE_NONVOL с frame-pointer-relative базой до SET_FPREG, стоит проверить порядок CodeOffset. Возможно, ты неправильно восстановил порядок slots. Возможно, дизассемблер ошибся с границей инструкции. Возможно, metadata действительно странная. В любом случае это уже не просто "таблица байтов", а логическая проверка истории пролога.

> UWOP_SET_FPREG - это не сохранение frame pointer, а объявление, что с этого места пролога появился стабильный frame base. Сохранение старого значения RBP описывается отдельно.

UWOP_SAVE_NONVOL: сохранение nonvolatile-регистра через запись в стек

UWOP_SAVE_NONVOL (save nonvolatile register, сохранение сохраняемого регистра в стек через memory store) - операция, которая описывает сохранение nonvolatile GPR в заранее выделенном стековом кадре, обычно инструкцией mov [base+offset], reg, а не push.

На первый взгляд UWOP_SAVE_NONVOL похож на UWOP_PUSH_NONVOL: оба говорят, что nonvolatile-регистр сохранён в стеке. Но инженерно это разные формы сохранения.

UWOP_PUSH_NONVOL соответствует модели "сначала уменьшаем RSP, потом кладём регистр на вершину стека". UWOP_SAVE_NONVOL соответствует другой модели: функция уже выделила фиксированный стековый кадр, и теперь записывает регистр в один из slots внутри этого кадра. Машинный код может выглядеть как mov [rsp+20h], rbx или mov [rbp-10h], rdi. RSP при такой инструкции не меняется. Меняется только память внутри уже выделенного кадра.

Почему компилятору может понадобиться SAVE_NONVOL вместо PUSH_NONVOL? Один важный случай - shrink-wrapping (сужение области сохранения регистра). Shrink-wrapping - это оптимизация, при которой компилятор откладывает сохранение nonvolatile-регистра до того места, где регистр действительно понадобится, вместо того чтобы сохранять его сразу в самом начале функции. Это позволяет не выполнять лишнее сохранение на путях, где регистр не используется. В контексте unwind такая оптимизация усложняет картину: сохранение может быть не первой инструкцией пролога, а сгруппированным действием, иногда описываемым через chained unwind info.

UWOP_SAVE_NONVOL занимает два slots. Первый slot содержит CodeOffset, UnwindOp=4 и OpInfo=номер регистра. Следующий slot хранит scaled offset: реальное смещение равно payload * 8. Это смещение указывает, где в fixed stack allocation лежит сохранённое значение регистра.

```
Пример:

sub rsp, 40h
mov [rsp+20h], rbx

Unwind model:
UWOP_ALLOC_SMALL size=0x40
UWOP_SAVE_NONVOL reg=RBX, offset=0x20

Encoding idea:
first slot: CodeOffset, UWOP_SAVE_NONVOL, OpInfo=RBX
second slot: payload = 0x20 / 8 = 4

Unwind effect:
RBX = *(frame_base + 0x20)
```

Здесь особенно важно понятие frame base. Если FrameRegister равен нулю, offset обычно считается от RSP после fixed allocation. Если FrameRegister установлен, offset интерпретируется относительно базы, определённой правилами frame pointer. Поэтому decoder, который просто пишет "[rsp+offset]" для всех SAVE_NONVOL, вводит пользователя в заблуждение. В человекочитаемом отчёте лучше писать нейтральнее: saved at frame offset 0x20, and frame base is computed from RSP or frame register according to UNWIND_INFO.

При unwind операция обратна обычной записи: RtlVirtualUnwind читает сохранённое значение из рассчитанного адреса и записывает его в соответствующий регистр в CONTEXT. RSP при отмене именно этой операции не изменяется. Это главное отличие от PUSH_NONVOL. Если parser считает, что SAVE_NONVOL увеличивает RSP на 8, он сломает весь расчёт caller context.

SAVE_NONVOL также даёт богатый материал для диагностики. Offset должен быть кратен 8 в short form. Payload должен существовать. Регистр должен быть разумным для nonvolatile save. CodeOffset должен указывать на инструкцию пролога, которая действительно сохраняет этот регистр в стековый кадр. Если сохранение находится после SizeOfProlog, надо проверять chained unwind info или shrink-wrapping. Если offset указывает за пределы fixed allocation, это либо ошибка анализа frame base, либо странная/повреждённая metadata.

UWOP_SAVE_NONVOL_FAR: сохранение GPR с большим offset

UWOP_SAVE_NONVOL_FAR (far save nonvolatile register, сохранение сохраняемого регистра с большим смещением) - расширенная форма UWOP_SAVE_NONVOL, где stack offset хранится полностью, без деления на 8, в двух дополнительных slots.

UWOP_SAVE_NONVOL_FAR решает ту же смысловую задачу, что и UWOP_SAVE_NONVOL: найти в стековом кадре сохранённое старое значение nonvolatile-регистра. Отличие только в размере offset. Если смещение слишком большое для short scaled encoding, используется FAR-форма.

Слово far здесь не означает "дальний переход" в старом x86-смысле сегментов. В контексте unwind data far означает "большое смещение, требующее более длинного кодирования". Это важная терминологическая ловушка. Нельзя переносить смысл far jump/far pointer из 16-битной или segmented-модели на UWOP_SAVE_NONVOL_FAR.

Операция занимает три slots: первый slot содержит CodeOffset, UnwindOp=5 и OpInfo=номер регистра; следующие два slots хранят unscaled offset в little-endian виде. Unscaled означает, что decoder не умножает значение на 8. В short SAVE_NONVOL payload был offset/8; в FAR-версии payload уже является полным offset.

```
UWOP_SAVE_NONVOL_FAR:

UnwindOp = 5
OpInfo = номер GPR-регистра
slots = 3
payload = full unscaled offset, два USHORT slots, little-endian
unwind = REG = *(frame_base + offset)

Главная ошибка:
нельзя делать offset = payload * 8
FAR payload уже хранит полный offset
```

В обычных небольших пользовательских функциях эта операция встречается реже. Но редкость не делает её неважной. Хороший decoder обязан корректно поддерживать редкие легальные случаи. Иначе он будет казаться рабочим на простых примерах, а потом развалится на большой функции, автогенерированном коде, драйвере, JIT-подобном runtime или намеренно сложном PE.

Для static /dynamic FAR-формы полезны как отдельный признак. Сам факт FAR-сохранения не доказывает ничего плохого. Но если в модуле массово встречаются огромные offsets, неканоничные allocation sizes и странные function ranges, это уже может стать частью общей картины: либо необычный toolchain, либо obfuscation/protection, либо повреждение unwind data. Правильный отчёт не должен кричать "malware"; он должен показать адрес, операцию, offset, границы fixed allocation и причину подозрительности.

UWOP_SAVE_XMM128: сохранение 128-битного nonvolatile XMM-регистра

UWOP_SAVE_XMM128 (save 128-bit XMM register, сохранение 128-битного XMM-регистра) - операция, которая описывает сохранение полного 128-битного состояния nonvolatile XMM-регистра в стековом кадре. Offset в short form масштабируется на 16.

До сих пор мы говорили в основном о целочисленных регистрах: RBX, RBP, RSI, RDI, R12-R15. Но Windows x64 calling convention также определяет правила для части XMM-регистров. XMM register (XMM-регистр, SIMD-регистр шириной 128 бит) - это регистр, используемый инструкциями SSE/SIMD для floating-point, vector и некоторых низкоуровневых операций. В Windows x64 регистры XMM0-XMM5 являются volatile, а XMM6-XMM15 считаются nonvolatile в той части, которую callee обязан сохранить при использовании.

UWOP_SAVE_XMM128 описывает сохранение всех 128 бит nonvolatile XMM-регистра. Это не "сохранить double" и не "сохранить float". Формат не знает, какие данные логически лежали в XMM-регистре. Он знает только размер состояния: 128 бит, 16 байт. Поэтому операция требует 16-байтового выравнивания offset и использует масштабирование offset/16.

В машинном коде такое сохранение может выглядеть как movaps [rsp+offset], xmm6, movdqa [rbp+offset], xmm7 или другой эквивалентный store, в зависимости от типа данных, выравнивания и выбора компилятора. UWOP_SAVE_XMM128 не фиксирует точное имя инструкции. Он фиксирует смысл: nonvolatile XMM register был сохранён в stack slot, и при unwind его надо восстановить из этого места.

```
Пример:

sub rsp, 80h
movdqa [rsp+40h], xmm7

Unwind model:
UWOP_ALLOC_SMALL/LARGE size=0x80
UWOP_SAVE_XMM128 reg=XMM7, offset=0x40

Encoding idea:
first slot: CodeOffset, UWOP_SAVE_XMM128, OpInfo=7
second slot: payload = 0x40 / 16 = 4

Unwind effect:
XMM7 = *(frame_base + 0x40), 16 bytes
```

Почему здесь масштабирование на 16, а не на 8? Потому что сохраняемый объект имеет размер 16 байт, и XMM-сохранения должны быть выровнены на 16-байтовую границу. Если payload равен 4, реальный offset равен 64 байта, а не 32. Это одна из частых ошибок при ручном разборе: человек привыкает к SAVE_NONVOL и автоматически умножает все

offsets на 8. Для XMM это неправильно.

Для validator-а есть несколько уровней проверки. Первый уровень - structural: хватает ли payload slot, корректно ли прочитан offset, не выходит ли вычисленный адрес за разумные границы описанного кадра. Второй уровень - ABI: является ли XMM-регистр nonvolatile. Если OpInfo указывает XMM2, это выглядит подозрительно, потому что XMM2 volatile и callee не обязана его сохранять для caller-а. Третий уровень - code matching: есть ли в прологе реальная инструкция store соответствующего XMM-регистра примерно в ожидаемое место.

Нужно быть осторожным с формулировками. Современные оптимизаторы, vector calling, hand-written assembly и специальные ABI-слои могут давать необычные картины. Поэтому лучше писать: "XMM register encoded by UWOP_SAVE_XMM128 is not normally nonvolatile under Windows x64 ABI". Это сильнее и точнее, чем категоричное "ошибка".

UWOP_SAVE_XMM128_FAR: XMM-сохранение с большим offset

UWOP_SAVE_XMM128_FAR (far save 128-bit XMM register, сохранение XMM-регистра с большим смещением) - расширенная форма UWOP_SAVE_XMM128, где offset хранится полностью в двух дополнительных slots и не масштабируется.

UWOP_SAVE_XMM128_FAR - это XMM-аналог UWOP_SAVE_NONVOL_FAR. Смысл тот же: сохранён nonvolatile XMM-регистр, но смещение в стековом кадре слишком большое для short scaled form. Поэтому операция занимает три slots и хранит полный unscaled offset.

Здесь снова важно не попасть в механическую ошибку. У обычного UWOP_SAVE_XMM128 payload умножается на 16. У FAR-версии payload уже является полным offset. Если decoder умножит FAR-offset на 16, он укажет далеко за пределы реального кадра и создаст ложную аномалию. Если decoder, наоборот, забудет умножение для short-версии, он занизит адрес сохранения.

Такие ошибки коварны: на простых функциях без XMM-сохранений decoder будет выглядеть правильным. На реальном бинаре с SIMD-кодом он начнёт выдавать странные offsets, и исследователь может ошибочно решить, что файл защищён или повреждён. Поэтому поддержка XMM UWOP-операций - это не украшение, а часть зрелости анализатора.

```
UWOP_SAVE_XMM128_FAR:

UnwindOp = 9
OpInfo = номер XMM-регистра
slots = 3
payload = full unscaled offset, два USHORT slots
unwind = XMMn = 16 bytes from (frame_base + offset)

Ключевое отличие:
SAVE_XMM128 -> offset = payload * 16
SAVE_XMM128_FAR -> offset = payload
```

С точки зрения отчёта static /dynamic для XMM FAR-форм нужно показывать не только регистр и offset, но и ширину сохранения: 16 bytes. Это помогает не путать такие записи с обычными 8-байтовыми GPR saves. Если твой инструмент позже будет строить карту стекового кадра, XMM-save должен занимать диапазон [offset, offset+16), а не один qword.

Range notation (нотация диапазона, запись [start, end)) - это полуоткрытая запись: начало включено, конец не включён. Для 16-байтового XMM-save по offset 0x40 занимаемый диапазон будет [0x40, 0x50). Такая запись удобна для поиска overlap-ов между сохранёнными регистрами, локальными зонами и подозрительными metadata offsets.

UWOP_PUSH_MACHFRAME: машинный кадр, который отличается от обычного пролога

> UWOP_PUSH_MACHFRAME (push machine frame, помещение машинного кадра в стек) - специальная unwind-операция, описывающая эффект аппаратного interrupt/exception frame. Она не является обычным push регистра из пользовательского пролога.

UWOP_PUSH_MACHFRAME нужно разбирать отдельно, потому что оно принципиально отличается от остальных операций. PUSH_NONVOL, ALLOC_SMALL, SAVE_NONVOL и SAVE_XMM128 описывают обычные действия функции: сохранить регистр, выделить стек, поставить frame pointer. PUSH_MACHFRAME описывает ситуацию, где стек изменил не обычный C/C++ пролог, а аппаратный механизм входа в обработчик прерывания или исключения.

Machine frame (машинный кадр, аппаратный кадр) - это набор значений, который процессор концептуально помещает в стек при переходе в обработчик определённого события: RIP, CS, EFLAGS, старый RSP, SS, иногда error code. Это не локальные переменные функции и не сохранение RBX. Это снимок части машинного состояния, созданный аппаратной логикой перехода.

Документация Microsoft указывает, что UWOP_PUSH_MACHFRAME появляется в dummy prolog (фиктивный пролог). Dummy prolog - это область, которая существует не как реально исполняемый обычный пролог функции, а как место для описания симуляции аппаратного push frame. Такой пролог помещается перед реальной точкой входа interrupt routine. То есть это не та операция, которую ты ожидаешь увидеть в типичной пользовательской функции из Visual Studio.

OpInfo у PUSH_MACHFRAME имеет специальный смысл. Если OpInfo=0, машинный кадр не содержит error code и уменьшает RSP на 40. Если OpInfo=1, кадр содержит error code и уменьшает RSP на 48. Другие значения должны считаться недопустимыми для классической схемы.

```
UWOP_PUSH_MACHFRAME, OpInfo = 0:

RSP+32 SS
RSP+24 old RSP
RSP+16 EFLAGS
RSP+8 CS
RSP RIP

Simulated effect:
RSP -= 40

UWOP_PUSH_MACHFRAME, OpInfo = 1:

RSP+40 SS
RSP+32 old RSP
RSP+24 EFLAGS
RSP+16 CS
RSP+8 RIP
```

```
RSP error code

Simulated effect:
RSP -= 48
```

Для обычного user-mode reverse engineering PUSH_MACHFRAME может встретиться редко, но знать его нужно. Во-первых, формат UNWIND_CODE включает эту операцию, и корректный parser не должен падать на легальном opcode 10. Во-вторых, наличие PUSH_MACHFRAME в неожиданном месте - это сильный повод посмотреть контекст: какой это модуль, какой subsystem, нет ли kernel-like code, thunk-a, runtime-generated table или искусственно созданной unwind metadata.

При этом нельзя автоматически считать PUSH_MACHFRAME вредоносностью. В системном программировании бывают специальные случаи. Но в обычном PE-приложении, где все функции похожи на стандартный MSVC-код, внезапные machine-frame unwind records выглядят необычно. Хороший инструмент должен уметь сформулировать это именно как context-sensitive anomaly: "операция легальна в формате, но нетипична для данного класса кода".

С точки зрения unwind-логики PUSH_MACHFRAME сообщает не "восстанови регистр", а "учти аппаратно созданный кадр и его размер". Это ещё раз показывает, что UWOP-язык описывает не машинные инструкции как текст, а изменения состояния, которые нужно обратить или смоделировать при переходе к предыдущему контексту.

Reserved opcodes: почему нельзя молча принимать всё

В классической x64 unwind-схеме основные UWOP-значения идут не плотным рядом без дыр. После UWOP_SAVE_NONVOL_FAR со значением 5 значения 6 и 7 не используются в базовом enum, затем идут UWOP_SAVE_XMM128 со значением 8, UWOP_SAVE_XMM128_FAR со значением 9 и UWOP_PUSH_MACHFRAME со значением 10. Это означает, что decoder должен иметь явную ветку для reserved/unknown opcode.

Reserved opcode (зарезервированный код операции) - это значение, которое формат не назначает текущей известной операции. Зарезервированность не всегда означает "битый файл". Иногда форматы расширяются, появляются новые версии, toolchain может поддерживать preview-режимы. Но если анализатор заявляет, что разбирает классический UNWIND_INFO version 1, он не должен молча трактовать opcode 6 как что-то своё.

Unknown opcode (неизвестный код операции) должен порождать безопасное поведение. Parser не знает, сколько slots занимает такая операция, значит он не может надёжно продолжать разбор массива. В некоторых инструментах можно попытаться продолжить с предположением one-slot unknown, но это уже recovery mode, а не строгий parse. В отчёте нужно явно писать, что дальнейшая интерпретация может быть недостоверна.

Здесь проявляется различие между permissive parser (терпимый parser) и strict validator (строгий валидатор). Permissive parser старается извлечь максимум информации даже из странных данных. Strict validator фиксирует нарушения спецификации. Для static /dynamic полезны оба режима: первый помогает исследователю не потерять картину, второй помогает формально объяснить, где metadata перестаёт соответствовать ожидаемой модели.

Неизвестный UWOP нельзя "просто пропустить", если неизвестно, сколько slots он занимает. Это вопрос безопасности парсинга, а не эстетики вывода.

Сводная таблица основных UWOP-операций

UWOP	slots	OpInfo	payload	Смысл
PUSH_NONVOL (0)	1	GPR register	-	push nonvolatile GPR; unwind restores REG and RSP += 8
ALLOC_LARGE (1)	2/3	mode 0/1	size/8 or full size	large fixed stack allocation; unwind RSP += size
ALLOC_SMALL (2)	1	size/8 - 1	-	small fixed stack allocation 8..128 bytes
SET_FPREG (3)	1	reserved	-	establish frame register from header fields
SAVE_NONVOL (4)	2	GPR register	offset/8	save nonvolatile GPR in allocated frame
SAVE_NONVOL_FAR (5)	3	GPR register	full offset	same as SAVE_NONVOL, but large unscaled offset
SAVE_XMM128 (8)	2	XMM register	offset/16	save 128-bit nonvolatile XMM register
SAVE_XMM128_FAR (9)	3	XMM register	full offset	same as SAVE_XMM128, but large unscaled offset
PUSH_MACHFRAME (10)	1	0/1 error code	-	simulate hardware machine frame, RSP -= 40 or 48

Как читать полный пример без потери смысла

Теперь соберём всё в одну связную картину. Представим функцию, в которой компилятор сохраняет RBP и RBX, выделяет 0x60 байт fixed stack allocation, устанавливает RBP как frame pointer внутри кадра и сохраняет RSI через mov в уже выделенный stack slot. Это не единственный возможный пролог, но он хорошо показывает взаимодействие операций.

```
Условный пролог:

00: push rbp ; save old RBP
01: push rbx ; save old RBX
02: sub rsp, 60h ; allocate fixed frame
06: lea rbp, [rsp+20h] ; establish frame pointer
0B: mov [rbp+18h], rsi ; save old RSI inside frame
0F: ... ; body begins

UNWIND_INFO, по смыслу:

Version = 1
Flags = 0
SizeOfProlog = 0Fh
CountOfCodes = ...
FrameRegister = RBP
FrameOffset = 2 ; 2 * 16 = 20h
```

```
UNWIND_CODE array in descending CodeOffset:

CodeOffset=0F, UWOP_SAVE_NONVOL, OpInfo=RSI, payload=38h/8
CodeOffset=0B, UWOP_SET_FPREG, OpInfo=0
CodeOffset=06, UWOP_ALLOC_SMALL, OpInfo=11 ; 11*8+8 = 96 = 60h
CodeOffset=02, UWOP_PUSH_NONVOL, OpInfo=RBX
CodeOffset=01, UWOP_PUSH_NONVOL, OpInfo=RBP
```

Если исключение произошло в теле функции, после пролога, unwind применяет всю модель. Он знает, что RSI сохранён в stack slot, RBX и RBP сохранены push-ами, а RSP был уменьшен на 0x60. Он восстанавливает значения в обратной логике и получает caller context.

Если исключение произошло на offset 04, то функция уже выполнила push rbp, push rbx и, возможно, находится внутри или сразу после sub rsp, 60h в зависимости от точной границы инструкции. Но она ещё не установила frame pointer и ещё не сохранила RSI. Поэтому нельзя восстанавливать RSI из [rbp+18h]: RBP как frame pointer ещё не создан, а store RSI ещё не произошёл. Вот зачем CodeOffset существует не как украшение, а как механизм частичной правды о прологе.

Если исключение произошло на offset 0D, после SET_FPREG, но до SAVE_NONVOL RSI, ситуация третья: frame pointer уже установлен, fixed allocation уже существует, push-и уже произошли, но RSI ещё не сохранён. Правильный unwinder должен отменить только исполненные действия. Поэтому массив UNWIND_CODE и сортировка по descending offset работают вместе с проверкой текущего offset внутри пролога.

Такой пример показывает, почему нельзя читать .xdata как статический список без времени. Пролог - это процесс, растянутый по нескольким инструкциям. UNWIND_CODE - это не просто список фактов, а список фактов, привязанных к точкам внутри этого процесса.

Как RtlVirtualUnwind мысленно применяет эти операции

RtlVirtualUnwind (виртуальная раскрутка, функция восстановления предыдущего контекста) не исполняет машинные инструкции текущей функции. Она получает CONTEXT, RUNTIME_FUNCTION и UNWIND_INFO, затем решает, где находится ControlPc: в эпилоге, в прологе или в теле функции. Если ControlPc находится в теле функции, обычно нужно отменить весь эффект пролога. Если ControlPc находится в прологе, нужно отменить только ту часть, которая уже успела выполниться. Если ControlPc находится в эпилоге, используется отдельная логика сканирования эпилога, о которой мы уже говорили.

При полном unwind из тела функции операции можно представить как обратный проход по смыслу пролога. ALLOC_SMALL/ALLOC_LARGE увеличивают RSP. PUSH_NONVOL восстанавливает регистр из [RSP] и увеличивает RSP. SAVE_NONVOL/SAVE_XMM128 восстанавливают регистр из frame offset, но сами по себе не меняют RSP. SET_FPREG помогает понять frame base, но не является восстановлением сохранённого регистра.

Эта логика объясняет, почему структура UWOP-операций такая разная. Некоторые операции меняют RSP, некоторые не меняют. Некоторые читают память и восстанавливают регистр. Некоторые устанавливают геометрию frame base. Некоторые требуют дополнительных payload slots. Если свести все UWOP к "действиям пролога", станет понятно, почему один универсальный обработчик невозможен: каждая операция

имеет собственную семантику.

```
Упрощённая модель действий при полном unwind:

UWOP_ALLOC_SMALL/LARGE:
Context.Rsp += allocation_size

UWOP_PUSH_NONVOL(reg):
Context.reg = *(Context.Rsp)
Context.Rsp += 8

UWOP_SAVE_NONVOL(reg, offset):
Context.reg = *(frame_base + offset)

UWOP_SAVE_XMM128(xmm, offset):
Context.xmm = 16 bytes from (frame_base + offset)

UWOP_SET_FPREG:
участвует в вычислении frame_base; не является pop-ом RBP

UWOP_PUSH_MACHFRAME:
учитывает simulated machine frame, размер 40 или 48 байт
```

Ошибки ручного разбора UWOP_*

Теперь перечислим не как чеклист ради галочки, а как реальные ошибки мышления, которые ломают анализ.

Первая ошибка - считать каждый slot отдельной операцией. Это приводит к тому, что payload от SAVE_NONVOL или ALLOC_LARGE ошибочно декодируется как новый UWOP. После этого весь массив "съезжает", и человек получает фантастический список операций. Правильный подход: opcode определяет длину записи.

Вторая ошибка - забыть про масштабирование. SAVE_NONVOL short form хранит offset/8, SAVE_XMM128 short form хранит offset/16, ALLOC_LARGE mode 0 хранит size/8, ALLOC_LARGE mode 1 хранит полный size. Эти четыре случая нельзя обрабатывать одной формулой.

Третья ошибка - путать SET_FPREG с сохранением RBP. SET_FPREG говорит, что frame pointer установлен. Старое значение RBP, если оно должно быть восстановлено, сохраняется отдельной операцией PUSH_NONVOL или SAVE_NONVOL. Если в отчёте написано "SET_FPREG restores RBP", это концептуально неверно.

Четвёртая ошибка - считать offset адресом от текущего RSP в момент исключения. В реальности offset относится к frame base, который вычисляется по правилам UNWIND_INFO. При dynamic allocation текущий RSP может быть совсем не той базой, от которой считались save offsets.

Пятая ошибка - не различать structural parsing и semantic validation. Structural parsing отвечает: можно ли прочитать bytes без выхода за границы и получить формальную операцию. Semantic validation отвечает: похожа ли эта операция на правильную для Windows x64 ABI и данного пролога. Эти уровни нужно разделять, иначе инструмент либо будет слишком доверчивым, либо будет ругаться на всё подряд.

Шестая ошибка - по одному странному UWOP делать общий вывод о защите или вредоносности. Reverse engineering работает по совокупности evidence. Один необычный SAVE_XMM128 может быть особенностью оптимизации. Но если вместе с ним есть overlapping function ranges, invalid chain, writable xdata placement, handler anomalies и

dynamic function table activity, тогда картина становится серьёзнее.

Какие findings должен уметь формулировать static /dynamic

Для твоего static /dynamic эта глава превращается в практическую модель диагностики. Инструмент должен не просто распечатать enum-имя операции. Он должен показать, как эта операция была прочитана, какие поля она использовала, сколько slots заняла, какой смысл получился и насколько этот смысл согласуется с кодом функции.

Finding (находка анализа, диагностическое наблюдение) - это не приговор. Это структурированная запись: где найдено явление, какое правило применялось, что именно нарушено или подозрительно, каковы последствия для анализа и какие ограничения вывода. Для UWOP_* это особенно важно, потому что многие аномалии находятся между "строго неверно" и "необычно, но возможно".

Хорошая finding по UWOP_ALLOC_LARGE должна указать RVA UNWIND_INFO, индекс slot-а, OpInfo mode, сколько payload slots ожидалось, сколько реально доступно, вычисленный size или причину невозможности вычислить size. Хорошая finding по UWOP_SAVE_NONVOL должна указать регистр, offset, масштабирование, frame base model и связь с инструкцией пролога. Хорошая finding по reserved opcode должна указать, что дальнейший разбор массива может быть ненадёжным, потому что длина неизвестной операции неизвестна.

В идеале отчёт должен позволять эксперту повторить вывод руками. Если инструмент пишет "bad unwind", это почти бесполезно. Если он пишет "UNWIND_INFO at RVA 0x12340, slot 3: UWOP_ALLOC_LARGE has OpInfo=2, but classic x64 unwind allows only modes 0 and 1; decoder cannot determine allocation size reliably", это уже инженерный вывод.

```
Пример хорошего диагностического сообщения:

[unwind.decode.truncated_payload]
UnwindInfoRVA: 0x00012340
SlotIndex: 5
Operation: UWOP_SAVE_NONVOL
Register: RBX
ExpectedSlots: 2
Available: 1
Consequence: saved-register offset cannot be read safely
Severity: parse_error

Пример плохого сообщения:

bad xdata
```

Каноничность и подозрительность: почему это не одно и то же

Когда мы говорим, что UWOP encoding должен быть shortest possible, что SAVE offsets должны быть кратны 8 или 16, что SET_FPREG имеет reserved OpInfo, легко начать мыслить слишком жёстко: любое отклонение равно плохой файл. Но reverse engineering требует аккуратности.

Canonical encoding (каноническое кодирование) - это ожидаемая, наиболее нормальная и спецификационно предпочтительная форма записи. Например, 40-байтовый allocation должен кодироваться через UWOP_ALLOC_SMALL, а не через UWOP_ALLOC_LARGE. Noncanonical encoding (неканоническое кодирование) может быть признаком нестандартного toolchain-а, ручной сборки, обфускации, бага генератора или

повреждения. Но само по себе оно ещё не доказывает намерение.

Suspiciousness (подозрительность) - это не бинарное свойство одного байта, а оценка контекста. Один reserved OpInfo в SET_FPREG - сильное нарушение формата. Один неканонический ALLOC_LARGE для маленького размера - слабее. Массовые неканонические encodings, совпадающие с защищёнными секциями, динамическими таблицами и handler anomalies, дают более сильную гипотезу. Поэтому static /dynamic должен накапливать evidence, а не заменять исследователя автоматическим ярлыком.

Такой подход делает инструмент полезным для BKP и для реальной практики. Он не притворяется антивирусом. Он показывает структуру, связи и нарушения, чтобы человек мог принять обоснованное решение.

Ручной разбор UWOP по байтам: почему нужно идти не глазами, а правилами формата

Теперь полезно отдельно проговорить ручной разбор. Когда reverse engineer открывает .xdata в hex-viewer или пишет собственный parser, он видит не красивые имена UWOP_PUSH_NONVOL и UWOP_ALLOC_SMALL. Он видит байты. Задача анализа - превратить эти байты в смысл, не потеряв контекст. Здесь нельзя идти "на глаз". Нужно идти по формату.

Первый шаг - прочитать заголовок UNWIND_INFO: Version, Flags, SizeOfProlog, CountOfCodes, FrameRegister и FrameOffset. Только после этого можно читать массив UNWIND_CODE. CountOfCodes говорит, сколько slots нужно обработать. Но обработать - не значит напечатать каждый slot как отдельную операцию. Нужно идти индексом i и после каждой операции увеличивать i на количество slots, которое эта операция занимает.

Второй шаг - для каждого первого slot операции разделить второй байт на low nibble и high nibble. Low nibble становится UnwindOp, high nibble становится OpInfo. Это место специально стоит разжевать: если байт равен 0x34, то opcode не 3 и OpInfo не 4 "потому что слева направо". В обычной записи hex 0x34 старшая половина - 3, младшая - 4. Но поле UnwindOp живёт в младших 4 битах, а OpInfo - в старших. Поэтому UnwindOp=4, OpInfo=3. Невнимательность здесь полностью меняет смысл операции.

Третий шаг - по UnwindOp определить длину записи. Если это PUSH_NONVOL, SET_FPREG, ALLOC_SMALL или PUSH_MACHFRAME, операция занимает один slot. Если это SAVE_NONVOL или SAVE_XMM128, нужен ещё один payload slot. Если это SAVE_NONVOL_FAR, SAVE_XMM128_FAR или ALLOC_LARGE mode 1, нужны два payload slots. ALLOC_LARGE mode 0 занимает один payload slot. Именно поэтому parser сначала должен понять opcode и mode, а уже потом читать дополнительные данные.

Четвёртый шаг - вычислить смысловые значения. Для ALLOC_SMALL вычисляется size. Для SAVE_NONVOL вычисляется offset. Для SET_FPREG берётся FrameRegister/FrameOffset из заголовка. Для XMM операций применяется масштаб 16. Для FAR операций масштаб не применяется. На этом этапе байты становятся инженерной моделью: "функция выделила столько-то байт", "сохранила такой-то регистр туда-то", "установила frame pointer таким-то образом".

```
Мини-алгоритм чтения UNWIND_CODE array:

i = 0
```

```
while i < CountOfCodes:
slot = UnwindCode[i]
code_offset = slot.CodeOffset
unwind_op = slot.UnwindOp ; low nibble
op_info = slot.OpInfo ; high nibble

length = length_by_opcode(unwind_op, op_info)
if i + length > CountOfCodes:
report truncated payload
stop or enter recovery mode

decode payload slots according to operation
i += length
```

Такой алгоритм кажется очевидным, пока не начинаешь писать настоящий decoder. На практике хочется быстро вывести "список кодов", а потом оказывается, что payload slots попали в список как отдельные операции. Поэтому в хорошем внутреннем представлении стоит разделять raw slots (сырые slots, как они лежат в файле) и decoded operations (декодированные операции, уже с учётом payload). Тогда можно одновременно показать эксперту байты и не потерять смысл.

Raw slot (сырой slot) - это минимальная двухбайтовая ячейка из массива UNWIND_CODE. Decoded operation (декодированная операция) - это уже смысловой объект: opcode, CodeOffset, OpInfo, payload, вычисленный размер/offset, регистр и длина в slots. Для анализа и отчёта нужен второй уровень, но для доказательности полезно сохранять первый.

Пример декодирования: ALLOC_SMALL из одного slot

Возьмём простой slot: первый байт 0x04, второй байт 0x72. Первый байт означает CodeOffset=4. Второй байт нужно разделить. Младшая половина 0x2 - это UWOP_ALLOC_SMALL. Старшая половина 0x7 - это OpInfo. Для ALLOC_SMALL OpInfo кодирует размер по формуле size = OpInfo * 8 + 8. Значит размер равен 7 * 8 + 8 = 64 байта.

Смысловая запись: к offset 4 пролога функция выполнила маленькое выделение стека на 64 байта. В машинном коде рядом с этой точкой мы ожидаем увидеть что-то вроде sub rsp, 40h. При полном unwind это действие будет отменено как RSP += 40h.

Здесь мы видим, как два байта превращаются в существенный факт о функции. Без этого факта невозможно найти return address caller-a, потому что RSP внутри тела функции смещён вниз на 64 байта.

```
Raw bytes:
04 72

Decode:
CodeOffset = 0x04
second byte = 0x72
UnwindOp = 0x2 -> UWOP_ALLOC_SMALL
OpInfo = 0x7
size = 7 * 8 + 8 = 64 bytes = 0x40

Meaning:
prolog allocated 0x40 bytes from stack
unwind must add 0x40 back to RSP
```

Пример декодирования: SAVE_NONVOL с payload slot

Теперь возьмём операцию, где одного slot недостаточно. Пусть первый slot выглядит как 0x0B 0x34. CodeOffset=0x0B. Второй байт 0x34 делится на low nibble 4 и high nibble 3. UnwindOp=4 означает UWOP_SAVE_NONVOL. OpInfo=3 означает регистр RBX по таблице GPR-регистров. Но это ещё не вся операция. SAVE_NONVOL требует следующий slot, где хранится offset/8.

Пусть следующий slot как USHORT хранит значение 0x0006. Тогда реальный offset равен 0x0006 * 8 = 0x30. Смысловая запись: в прологе на offset 0x0B старое значение RBX было сохранено в stack frame по offset 0x30. При unwind RBX надо восстановить из frame_base + 0x30.

Обрати внимание: второй slot не имеет собственного CodeOffset в смысловом плане. Да, физически это тоже два байта, и если прочитать его как UNWIND_CODE, там тоже можно увидеть какие-то nibbles. Но это будет ложная интерпретация. В контексте текущей операции этот slot является числом offset/8.

```
Raw slots:
slot 0: 0B 34
slot 1: 06 00

Decode slot 0:
CodeOffset = 0x0B
UnwindOp = 0x4 -> UWOP_SAVE_NONVOL
OpInfo = 0x3 -> RBX

Decode payload slot 1:
payload = 0x0006
offset = 0x0006 * 8 = 0x30

Decoded operation:
UWOP_SAVE_NONVOL(RBX, frame_offset=0x30), CodeOffset=0x0B
```

Пример декодирования: ALLOC_LARGE mode 0 и mode 1

ALLOC_LARGE полезно разобрать отдельно, потому что здесь OpInfo меняет не регистр и не размер напрямую, а режим чтения payload. Это маленькое поле управляет длиной всей записи. Поэтому ALLOC_LARGE нельзя декодировать до конца, пока не проверен OpInfo.

Если UnwindOp=1 и OpInfo=0, следующий slot хранит size/8. Например payload=0x0026 означает size=0x130. Если UnwindOp=1 и OpInfo=1, следующие два slots хранят полный размер. Например slots 00 30 00 00 в little-endian USHORT-порядке дают size=0x3000. Это уже unscaled значение: умножать на 8 нельзя.

Если OpInfo=2 или любое другое значение, классическая спецификация не даёт такого режима. Parser не имеет права придумывать, что это значит. Он может вывести unknown/invalid ALLOC_LARGE mode и остановить строгий разбор этой операции.

```
ALLOC_LARGE mode 0:

slot 0: 06 01 ; CodeOffset=6, UnwindOp=1, OpInfo=0
slot 1: 26 00 ; payload=0x0026
size = 0x26 * 8 = 0x130

ALLOC_LARGE mode 1:
```

```
slot 0: 08 11 ; CodeOffset=8, UnwindOp=1, OpInfo=1
slot 1: 00 30
slot 2: 00 00
size = 0x00003000 ; full unscaled size
```

Сопоставление UWOP с дизассемблером: где заканчивается формат и начинается исследование

Декодировать UWOP по байтам - это только половина работы. В reverse engineering важнее следующий вопрос: совпадает ли metadata с реальным кодом? .xdata говорит, что пролог сохранил RBX через push. Дизассемблер должен показать push rbx около соответствующего CodeOffset. .xdata говорит, что было sub rsp, 40h. Дизассемблер должен показать эквивалентное уменьшение RSP. .xdata говорит, что установлен frame pointer. Дизассемблер должен показать lea или mov, создающий этот frame pointer.

Но это сопоставление не всегда идеально механическое. Машинная инструкция может иметь разную длину. Пролог может содержать hotpatch prefix. Сохранение регистра может быть через инструкцию, которая выглядит немного иначе, но имеет тот же эффект. Frame pointer может быть установлен через lea, а не через mov. Поэтому code matching должен быть семантическим, а не только строковым.

Semantic matching (семантическое сопоставление) - это попытка сравнить не текст инструкции, а её эффект. Например, sub rsp, 40h и lea rsp, [rsp-40h] не одинаковы как текст, но похожи по эффекту на RSP. mov [rsp+20h], rbx и mov qword ptr [rsp+20h], rbx - один и тот же смысл для SAVE_NONVOL. Для первого варианта static /dynamic может начинать с простых паттернов, но архитектурно лучше держать в голове именно эффект.

При этом нельзя делать matching слишком свободным. Если metadata говорит SAVE_NONVOL RBX offset 0x20, а код сохраняет RDI в [rsp+20h], это не "почти то же самое". Для unwind это другой регистр, и восстановление RBX станет неверным. Если metadata говорит ALLOC_SMALL 0x40, а код делает sub rsp, 0x80, это тоже существенное расхождение. Чем точнее tool показывает такие расхождения, тем полезнее он для реверса.

Пошаговое восстановление RSP на простом стеке

Чтобы UWOP не оставались абстрактными, представим стек до входа в функцию. Caller выполнил call, процессор положил return address на стек, и управление перешло в callee. На входе в callee RSP указывает на return address. Затем callee делает push rbx и sub rsp, 40h. Теперь текущий RSP указывает уже на нижнюю часть локального кадра, а return address находится выше.

UNWIND_INFO должен позволить пройти обратно. Если функция в теле упала с access violation, Windows не может просто взять [RSP] как return address. Там лежит локальная область, а не адрес возврата. Сначала нужно отменить fixed allocation: RSP += 0x40. Теперь RSP указывает на сохранённый RBX. Затем нужно отменить push rbx: RBX = [RSP], RSP += 8. Теперь RSP снова указывает на return address. После этого unwinder может считать RIP caller-а из [RSP] и сдвинуть RSP ещё на 8, когда перейдёт к следующему кадру.

Вот почему точность UWOP_ALLOC_SMALL и UWOP_PUSH_NONVOL критична. Ошибка в размере allocation на 8 байт сдвинет всю цепочку. Ошибка в push-регистре восстановит не

тот регистр. Ошибка в порядке операций может привести к чтению return address из места, где лежит сохранённый RBX или локальная переменная. Stack trace станет ложным, exception handler может быть найден неверно, crash dump analysis потеряет доверие.

```
До call:
caller stack ...

После call, на входе в callee:
[RSP] = return address

После push rbx:
[RSP] = old RBX
[RSP+8] = return address

После sub rsp, 40h:
[RSP ... RSP+3F] = local/fixed frame area
[RSP+40] = old RBX
[RSP+48] = return address

Unwind from function body:
RSP += 0x40 ; undo allocation
RBX = [RSP] ; restore old RBX
RSP += 8 ; undo push
caller RIP = [RSP]
```

UWOP как контракт между compiler, linker, Windows и debugger

Полезно увидеть UWOP_* не только как формат байтов, но и как контракт между несколькими участниками системы. Compiler (компилятор) генерирует пролог и понимает, какие действия он совершил. Assembler или linker (ассемблер или компоновщик) формирует .pdata и .xdata. Windows runtime читает эти данные при exception dispatch и unwind. Debugger использует их для stack trace. Crash dump analyzer использует их уже после смерти процесса, когда выполнить код невозможно.

Contract (контракт) здесь означает не юридический документ, а техническое соглашение: если код функции делает определённые вещи со стеком и nonvolatile-регистрами, metadata должна описывать эти вещи в установленной форме. Если metadata врёт, Windows может восстановить неверный caller context. Если metadata отсутствует там, где она нужна non-leaf функции, exception handling и debugging становятся ненадёжными. Если metadata формально есть, но parser её читает неправильно, твой инструмент будет видеть ложную картину.

Эта контрактная природа объясняет, почему UWOP-операции такие ограниченные. Нельзя описать произвольный пролог как угодно. Формат поддерживает только набор действий, которые Windows умеет надёжно обратить. Поэтому x64 prolog/epilog coding conventions ограничивают форму пролога и эпилога. Свобода машинного кода уступает место необходимости гарантированного unwind.

Для reverse engineer-а это хорошая новость. Ограничения создают ожидания. Ожидания позволяют искать нарушения. Нарушения дают вопросы. А хорошие вопросы двигают анализ: почему функция описана так, хотя код выглядит иначе? Это оптимизация? Chained unwind? Hand-written assembly? JIT? Обфускация? Повреждение? Ошибка дизассемблера? Без понимания UWOP_* эти вопросы даже не появятся.

Мини-матрица силы аномалий по UWOP_*

В конце полезно разделить аномалии по силе. Это не абсолютная шкала, но хорошая инженерная привычка. Есть ошибки, которые делают дальнейший parse невозможным. Есть нарушения ABI, которые выглядят подозрительно, но могут иметь объяснение. Есть неканоничные формы, которые сами по себе слабые, но усиливают другие признаки.

Сильные parse-level проблемы: не хватает payload slots; ALLOC_LARGE имеет недопустимый mode; unknown opcode в classic UNWIND_INFO; CountOfCodes выходит за пределы доступной .xdata; optional handler/chained area пересекается с массивом кодов из-за неправильного выравнивания. Такие вещи прямо ломают надёжность чтения.

Средние semantic-level проблемы: SAVE_NONVOL указывает offset за пределами ожидаемого fixed allocation; CodeOffset не соответствует ни одной разумной инструкции пролога; SET_FPREG имеет reserved OpInfo; save operation появляется до установки frame pointer, хотя использует frame-pointer модель; PUSH_NONVOL описывает volatile-регистр. Это требует анализа контекста.

Слабые, но полезные heuristic-level признаки: не самое короткое кодирование allocation; редкие FAR-формы в большом количестве; необычная плотность XMM saves; массовое reuse одного UNWIND_INFO; большое количество tiny functions с одинаковой metadata; нетипичное соотношение .pdata/.xdata размеров. Сами по себе они не доказывают защиту, но хорошо работают как часть общей картины.

Переход к следующей теме

Мы разобрали основной алфавит UWOP_* операций. Теперь у нас есть возможность читать UNWIND_CODE не как набор загадочных чисел, а как последовательность действий: сохранить регистр, выделить стек, поставить frame pointer, сохранить XMM-состояние, описать машинный кадр. Но остался ещё один слой, без которого эта картина будет неполной.

Многие UWOP-операции постоянно ссылаются на два понятия: frame register и frame pointer. Мы уже касались их в UWOP_SET_FPREG, но пока только настолько, насколько это было нужно для понимания конкретной операции. Теперь нужно остановиться и разобрать эту тему отдельно. Frame pointer на x64 не является обязательным, но когда он появляется, он меняет способ вычисления offsets, делает возможными некоторые динамические изменения RSP и влияет на то, как unwinder восстанавливает frame base.

Поэтому следующий раздел естественно продолжает эту главу: что такое frame register и frame pointer на x64, почему они не одно и то же с "просто RBP", как они кодируются в UNWIND_INFO, как используются при SAVE_NONVOL/SAVE_XMM128 и какие ошибки возникают у reverse engineer-а, если он автоматически считает любой RBP классическим frame pointer старого x86-стиля.

4.5. Что такое frame register и frame pointer на x64

После разбора UWOP_* операций может показаться, что главная часть unwind-модели уже понятна. Мы увидели, как пролог сохраняет nonvolatile-регистры, как выделяет место на стеке, как описывает большие allocations, как хранит XMM-регистры и как RtlVirtualUnwind потом отменяет эти действия. Но в заголовке UNWIND_INFO остаётся пара полей, которые нельзя воспринимать как маленькую справочную деталь: FrameRegister и FrameOffset. Эти поля отвечают за один из самых старых и одновременно самых неправильно понимаемых вопросов низкоуровневого анализа: что такое frame pointer и почему на x64 он то есть, то исчезает.

Новичок часто приносит в x64 старую привычку из x86: "кадр функции - это то, что висит на EBP/RBP". В старом учебном ассемблере действительно почти всегда показывают пролог вида push ebp; mov ebp, esp; sub esp, N. После такого примера легко решить, что frame pointer является обязательной частью любой нормальной функции. Но Windows x64 устроен иначе. Здесь наличие отдельного frame pointer - не обязательная норма, а один из допустимых вариантов организации кадра. Большая часть оптимизированных функций спокойно живёт без RBP как указателя кадра. При этом unwind всё равно работает, потому что Windows x64 опирается не на обязательную цепочку RBP, а на табличные метаданные .pdata/.xdata.

Именно поэтому этот раздел нельзя свести к фразе "frame pointer - это RBP". Это было бы грубое упрощение. Frame pointer - это роль, а не имя регистра. RBP часто может выполнять эту роль, но на Windows x64 роль frame pointer может быть назначена и другому nonvolatile-регистру, например R13. A frame register в UNWIND_INFO - это не сам физический указатель в стеке, а закодированное заявление метаданных: "в этой функции такой-то nonvolatile-регистр используется как frame pointer".

Здесь особенно важно идти медленно. Если перепутать frame pointer, frame register, stack pointer, frame base и saved RBP, дальше сломается всё: ручной разбор .xdata, проверка UWOP_SET_FPREG, интерпретация SAVE_NONVOL offsets, анализ alloca, проверка эпилогов и выводы reverse engineer-а о том, нормальная перед ним функция или намеренно странная metadata.

frame pointer (указатель кадра, стабильная база стекового кадра) - это регистр, который функция использует как относительно стабильную точку отсчёта внутри своего stack frame. Он нужен не всегда. Его смысл в том, чтобы иметь базу, от которой удобно и безопасно адресовать локальные данные, сохранённые регистры и иногда области, связанные с динамическим изменением RSP.
frame register (регистр кадра в UNWIND_INFO) - это поле unwind-метаданных, которое хранит номер nonvolatile-регистра, выбранного функцией в качестве frame pointer. Если поле равно нулю, функция не заявляет отдельный frame pointer в UNWIND_INFO. Если поле ненулевое, оно говорит не "в функции есть RBP", а "в этой функции такой-то регистр играет роль frame pointer".
frame offset (смещение frame register, масштабированное смещение указателя кадра) - это второе маленькое поле в UNWIND_INFO. Оно показывает, на сколько байт frame pointer смещён относительно RSP в момент его установки. Значение хранится не в байтах, а в единицах по 16 байт: реальное смещение равно FrameOffset * 16.

Откуда вообще появляется потребность в frame pointer

Чтобы понять frame pointer, надо снова вернуться к стеку. Stack (стек) в x64 - это область памяти, которая используется для return address, аргументов сверх регистров, home/shadow space, локальных переменных, сохранённых nonvolatile-регистров и временных данных. В Windows x64 стек растёт вниз: когда функция выделяет место, RSP уменьшается; когда функция освобождает место, RSP увеличивается.

RSP (stack pointer, указатель вершины стека) - это регистр, который указывает на текущую вершину стека. Он абсолютно необходим для работы call/ret, сохранений, локальных областей и обращений к данным на стеке. Но именно потому, что RSP является рабочим указателем вершины, он может быть неудобен как постоянная база адресации. В прологе он меняется из-за push и sub rsp, N. В функции с alloca или другим динамическим выделением он может меняться ещё и внутри тела функции. Если обращаться ко всем локальным данным только через текущий RSP, компилятору и анализатору нужно каждый раз точно знать, в каком состоянии находится RSP именно в этой точке.

Frame pointer решает эту проблему иначе. Функция выбирает nonvolatile-регистр, устанавливает его один раз после формирования фиксированной части кадра и потом использует как устойчивую базу. Если RSP позже временно сдвинется вниз из-за dynamic allocation, frame pointer продолжит указывать на известное место внутри фиксированного кадра. Поэтому доступ к некоторым данным остаётся стабильным.

Слово "stable" здесь не означает, что регистр физически невозможно изменить. Процессор не запрещает менять RBP или R13. Речь о соглашении, которое соблюдает сгенерированный код: после установки frame pointer функция использует его как базу и не портит произвольно, пока кадр существует. Для unwinder-а это не философия, а практический факт: если FrameRegister заявлен в UNWIND_INFO, то offset-операции могут интерпретироваться через базу, связанную с этим регистром.

> Frame pointer нужен не потому, что без него стек "невозможен". Он нужен тогда, когда текущий RSP перестаёт быть удобной или устойчивой базой для адресации fixed stack frame. На x64 отсутствие frame pointer - нормальная оптимизация, а не признак поломанной функции.

Frame pointer - это роль, а не обязательно RBP

Исторически роль frame pointer часто выполнял EBP на x86 и RBP на x64. Из-за этого в учебниках, отладчиках и разговорной речи можно услышать: "RBP - это frame pointer". Но строго технически это не всегда верно. RBP - это конкретный регистр архитектуры x86-64. Frame pointer - это роль в организации стекового кадра. Один и тот же регистр может быть обычным nonvolatile-регистром, может хранить пользовательское значение, может быть сохранён на стеке, а может использоваться как frame pointer. Роль определяется кодом функции и метаданными, а не именем регистра сама по себе.

На Windows x64 UNWIND_INFO хранит FrameRegister как номер регистра в том же стиле кодирования, что и OpInfo у UNWIND_CODE. Документация Microsoft говорит: если это поле ненулевое, функция использует frame pointer, а поле содержит номер nonvolatile-регистра, используемого как FP. Поэтому корректнее говорить не "frame pointer равен RBP", а "в этой функции FrameRegister указывает на RBP" или "в этой функции FrameRegister указывает на R13".

Почему компилятор может выбрать не RBP? Потому что RBP - это просто один из nonvolatile-регистров, и у оптимизатора есть свои причины распределять регистры. Кроме того, в документации Microsoft по prolog/epilog встречается пример, где в качестве frame pointer используется R13: после большого выделения стека выполняется lea R13, 128[RSP]. Это хороший пример для разрушения мифа: Windows x64 unwind-модель не привязана намертво к RBP.

Для reverse engineer-а это значит: нельзя строить анализ вокруг фразы "ищи mov rbp, rsp". Нужно смотреть UNWIND_INFO. Если FrameRegister равен 13, значит frame pointer по metadata - R13. Если FrameRegister равен 5, это RBP. Если FrameRegister равен 0, функция не заявляет frame pointer. А дальше уже нужно проверять, согласуется ли это с реальным прологом.

```
Неправильная мысль:
"Если есть frame pointer, это всегда RBP".

Правильная мысль:
"Frame pointer - это роль. UNWIND_INFO сообщает, какой nonvolatile-регистр
выполняет эту роль в данной функции. Часто это RBP, но не обязательно".

Пример из допустимой x64-логики:
push r13
sub rsp, large_size
lea r13, [rsp+80h]

Здесь frame pointer может быть R13, а не RBP.
```

Stack pointer, frame pointer и saved frame pointer - три разных вещи

Одна из главных причин путаницы - похожие слова. Stack pointer, frame pointer и saved frame pointer звучат рядом, но обозначают разные уровни.

Stack pointer (указатель стека) - это RSP. Он показывает текущую вершину стека. Call, ret, push, pop, sub rsp, add rsp напрямую меняют или используют его. RSP является центральным регистром для исполнения, и его правильное состояние критично для возврата из функции.

Frame pointer (указатель кадра) - это выбранный функцией nonvolatile-регистр, который указывает на стабильную позицию внутри fixed stack allocation. Это может быть RBP, R13 или другой разрешённый nonvolatile-регистр. Он служит базой адресации, особенно когда текущий RSP может меняться.

Saved frame pointer (сохранённое старое значение frame pointer) - это значение регистра frame pointer, принадлежавшее вызывающей функции или предыдущему состоянию, которое callee обязана сохранить, если собирается использовать этот регистр. Например, если функция хочет использовать RBP как frame pointer, она часто сначала делает push rbp или сохраняет RBP в stack slot. Это сохранение описывается UWOP_PUSH_NONVOL или UWOP_SAVE_NONVOL. Но установка нового frame pointer описывается UWOP_SET_FPREG.

Если эти три вещи смешать, можно сделать неправильный вывод. UWOP_SET_FPREG не восстанавливает старый RBP. Он не говорит, где лежит сохранённый RBP. Он говорит, что в прологе был установлен frame pointer. Старый RBP, если он был важен как nonvolatile-регистр, должен быть описан отдельной unwind-операцией. И наоборот: push rbp сам по себе ещё не означает, что RBP стал frame pointer. Это может быть просто

сохранение nonvolatile-регистра ради дальнейшего использования.

Термин	Что означает	Что часто путают
RSP / stack pointer	Текущая вершина стека; меняется push/pop/sub/add/call/ret	Его ошибочно считают стабильной базой для всего кадра
frame pointer	Регистр-роль: стабильная база внутри fixed frame	Его ошибочно отождествляют только с RBP
FrameRegister	Поле UNWIND_INFO с номером регистра frame pointer	Его ошибочно читают как адрес или как сохранённое значение
saved RBP/R13/...	Старое значение nonvolatile-регистра, сохранённое callee	Его ошибочно принимают за UWOP_SET_FPREG

Как FrameRegister и FrameOffset лежат в UNWIND_INFO

В структуре UNWIND_INFO есть байт, который логически делится на два 4-битных поля: FrameRegister и FrameOffset. Это compact representation (компактное представление): один байт несёт сразу два смысла. Младшая половина хранит номер регистра frame pointer, старшая половина хранит масштабированное смещение. Из-за такого плотного формата parser обязан аккуратно работать с битами, а не читать поле как обычное 16- или 32-битное число.

FrameRegister равен нулю, если функция не использует frame pointer в смысле unwind metadata. Это не значит, что в машинном коде вообще нигде не встречается RBP. Это значит только, что UNWIND_INFO не объявляет отдельный FP register. Если FrameRegister ненулевой, его значение интерпретируется как номер nonvolatile-регистра в той же системе кодирования, которая используется в OpInfo у операций вроде UWOP_PUSH_NONVOL.

FrameOffset хранит смещение не в байтах. Реальное смещение вычисляется как FrameOffset * 16. Диапазон получается от 0 до 240 байт, потому что поле 4-битное и может хранить значения 0-15. Microsoft описывает смысл так: selected FP register is set to RSP + 16 * this number. То есть если FrameOffset=2, frame pointer установлен как RSP + 32. Если FrameOffset=8, frame pointer установлен как RSP + 128.

Это место постоянно ломает ручной анализ. Значение FrameOffset=2 не означает "2 байта". Оно означает 2 * 16 = 32 байта. Если забыть масштабирование, все frame-pointer-relative адреса будут съезжать, а потом decoder начнёт обвинять программу в несоответствиях, которых на самом деле нет.

```
UNWIND_INFO byte with FrameRegister/FrameOffset:

low nibble: FrameRegister ; номер регистра FP, 0 = FP не заявлен
high nibble: FrameOffset ; scaled offset, real_offset = value * 16

Пример:
FrameRegister = 13 -> R13
FrameOffset = 8 -> 8 * 16 = 128 bytes

Смысл:
```

```
R13 = RSP + 128 в момент UWOP_SET_FPREG
```

Для отчёта `static` /`dynamic` полезно выводить оба значения: сырое FrameOffset=8 и расшифрованное frame_offset_bytes=0x80. Это сразу убирает типичную ошибку "8 байт вместо 128".

Почему frame pointer часто указывает не на начало кадра, а внутрь него

Интуитивно можно ожидать, что frame pointer должен указывать на начало стекового кадра. Но Microsoft x64 unwind-модель прямо допускает другое: frame pointer может быть установлен как RSP + offset, где offset лежит в диапазоне 0-240 байт. То есть он может указывать внутрь fixed stack allocation, а не на самый нижний адрес выделенного участка.

Зачем так делать? Основная причина - code density (плотность кода, компактность машинных инструкций). В x86-64 многие инструкции обращения к памяти могут использовать короткий 8-битный signed displacement. Signed displacement (знаковое смещение) позволяет обратиться к адресам примерно в диапазоне -128..+127 относительно базового регистра. Если базовый регистр поставить в середину области локальных данных, компилятор сможет обращаться и к данным ниже базы, и к данным выше базы короткими инструкциями. Если же база стоит у края области, часть смещений может не помещаться в короткую форму, и инструкции станут длиннее.

Это звучит как мелочь, но в машинном коде такие мелочи важны. Функций много, обращений к локальным переменным много, разница в длине инструкций накапливается. Поэтому frame pointer в середине allocation - это не странность, а инженерный компромисс между удобной адресацией и компактным кодом.

Для unwind это означает, что FrameOffset не является украшением. Он нужен, чтобы из frame pointer можно было восстановить ту базу, относительно которой считаются offsets в unwind-операциях. Если FP = RSP_at_setup + 0x80, то RSP_at_setup = FP - 0x80. Именно этот обратный расчёт позволяет правильно интерпретировать SAVE_NONVOL и SAVE_XMM128 offsets, если функция использует frame pointer.

```
Схематично, стек растёт вниз:

higher addresses
return address
saved nonvolatile registers
fixed stack allocation start <- RSP после sub rsp, N
local area
...
frame pointer position <- FP = RSP + 0x80
...
more local area
lower addresses

Если FrameOffset = 8:
FP = RSP_at_setup + 8*16 = RSP_at_setup + 0x80
RSP_at_setup = FP - 0x80
```

UWOP_SET_FPREG: момент, когда frame pointer появляется

FrameRegister и FrameOffset в заголовке UNWIND_INFO говорят, какой frame pointer должен существовать и с каким смещением. Но одного заголовка мало. Unwind-массив

должен ещё сказать, где именно в прологе этот frame pointer был установлен. Для этого существует операция UWOP_SET_FPREG.

UWOP_SET_FPREG (set frame pointer register, установка регистра указателя кадра) - это unwind-операция, которая описывает инструкцию пролога вида lea FPReg, [rsp + FrameOffset*16] или эквивалентное действие. Она не имеет собственного payload. Номер регистра и offset берутся не из OpInfo, а из заголовка UNWIND_INFO. OpInfo у UWOP_SET_FPREG зарезервирован и не должен использоваться.

Почему важно знать именно момент установки? Потому что исключение может произойти внутри пролога. Допустим, функция уже сохранила R13 и уже выделила stack space, но ещё не выполнила lea r13, [rsp+80h]. Если в этот момент произойдёт exception, metadata уже содержит FrameRegister=R13, но реальный R13 ещё не является frame pointer этой функции. Unwinder не имеет права использовать его как базу. Поэтому CodeOffset у UWOP_SET_FPREG сообщает: начиная с этой точки пролога FP можно считать установленным.

Это особенно важно для операций, которые берут offset в local stack frame. Microsoft указывает: если FP register используется, unwind-коды, которым нужен offset, должны использоваться только после того, как FP register установлен в прологе. Смысл простой: нельзя вычислять адрес сохранённого регистра через базу, которая ещё не существует.

```
Пример:

00: push r13
02: sub rsp, 100h
09: lea r13, [rsp+80h]
11: mov [r13-20h], rbx

UNWIND_INFO:
FrameRegister = R13
FrameOffset = 8 ; 0x80

UNWIND_CODE:
CodeOffset=..., UWOP_SET_FPREG
CodeOffset=..., UWOP_SAVE_NONVOL RBX, offset=...

Смысл:
до lea r13,[rsp+80h] R13 ещё не frame pointer этой функции.
```

UWOP_SET_FPREG не сохраняет старый frame pointer. Он только фиксирует установку нового FP. Старое значение FP-регистра должно быть описано отдельной операцией сохранения, если callee использует nonvolatile-регистр.

Функция без frame pointer: нормальный современный случай

Теперь полезно посмотреть на противоположный случай. Функция может вообще не использовать frame pointer. Тогда FrameRegister=0, FrameOffset обычно тоже должен быть 0, а в массиве unwind-кодов нет UWOP_SET_FPREG. Это не упрощённая игрушечная функция, а очень распространённый случай в оптимизированном x64-коде.

Почему компилятор отказывается от frame pointer? Потому что отдельный FP-регистр занимает один nonvolatile-регистр, который иначе можно было бы использовать для данных программы. В 64-битном режиме регистров больше, чем на x86, но они всё равно ценны. Если функция имеет фиксированный stack allocation и не меняет RSP после

пролога, то обращаться к локальным переменным можно через RSP с постоянными смещениями. В такой ситуации отдельный frame pointer не даёт большой пользы, зато забирает регистр и добавляет инструкции сохранения/восстановления.

В функции без frame pointer unwind всё равно полностью возможен. .pdata сообщает диапазон функции, .xdata указывает на UNWIND_INFO, UNWIND_CODE описывает push/sub/save, а offsets считаются относительно RSP в фиксированном кадре. RtlVirtualUnwind не нуждается в цепочке RBP. Это принципиальное отличие Windows x64 table-driven unwinding от старого мышления "стек ходится по linked list из frame pointers".

Для reverse engineer-а отсутствие frame pointer не является finding уровня "ошибка". Это нормальная компиляторная оптимизация. Finding может появиться только если есть противоречие: например, FrameRegister=0, но в unwind codes есть UWOP_SET_FPREG; или FrameRegister ненулевой, но UWOP_SET_FPREG отсутствует; или дизассемблер показывает установку RBP как базы, но metadata её не отражает, и при этом функция использует frame-pointer-relative сохранения, которые невозможно корректно объяснить.

```
Функция без frame pointer:

push rbx
sub rsp, 40h
mov [rsp+20h], rsi
...
add rsp, 40h
pop rbx
ret

UNWIND_INFO:
FrameRegister = 0
FrameOffset = 0
UWOP_PUSH_NONVOL RBX
UWOP_ALLOC_SMALL 0x40
UWOP_SAVE_NONVOL RSI, offset=0x20

Здесь нет RBP-chain, но unwind полностью возможен.
```

Функция с frame pointer: когда он действительно нужен

Теперь рассмотрим случаи, где frame pointer становится не просто удобством, а важным механизмом. Самый показательный случай - dynamic stack allocation, то есть выделение стека во время выполнения на размер, который не является фиксированной константой пролога. В C/C++ это может быть связано с _alloca или похожими механизмами.

alloca (automatic stack allocation, динамическое выделение памяти на стеке) - это выделение области на стеке, размер которой известен только во время исполнения. Например, функция получает n и выделяет n байт на стеке. Это отличается от sub rsp, 40h, где размер фиксирован и известен unwind-метаданным. При alloca RSP может измениться внутри тела функции, после пролога. Если после этого пытаться адресовать fixed local data через RSP, смещения станут зависеть от runtime-размера alloca. Это неудобно и опасно для unwind.

Microsoft x64 stack usage прямо говорит: если в функции динамически выделяется пространство через alloca, nonvolatile-регистр должен использоваться как frame pointer, чтобы отметить базу fixed part of the stack; этот регистр должен быть сохранён и инициализирован в прологе. Это очень важная фраза. Она связывает три вещи: dynamic

allocation, nonvolatile frame pointer и prolog initialization.

Fixed part of the stack (фиксированная часть стека) - это область кадра, размер и расположение которой известны из пролога и unwind metadata. Dynamic part (динамическая часть) может появляться позже и иметь runtime-размер. Frame pointer служит якорем между этими частями: даже если RSP ушёл ниже из-за alloca, FP всё ещё позволяет найти фиксированную область.

```
Упрощённая идея с dynamic allocation:

push r13
sub rsp, 80h
lea r13, [rsp+40h] ; R13 = stable frame pointer
...
mov rax, runtime_size
sub rsp, rax ; dynamic allocation, RSP меняется в теле
...
; доступ к fixed locals может идти через r13, а не через текущий rsp

Без frame pointer текущий RSP уже не даёт простой стабильной базы.
```

Как frame pointer влияет на offset-операции

В предыдущих разделах мы разбирали UWOP_SAVE_NONVOL, UWOP_SAVE_NONVOL_FAR, UWOP_SAVE_XMM128 и UWOP_SAVE_XMM128_FAR. Все эти операции используют offset в local stack frame. Теперь нужно аккуратно объяснить, откуда именно считается этот offset.

Если FrameRegister=0, offset считается от RSP. Но здесь важно уточнение: не от произвольного текущего RSP в любой точке функции, а от RSP как базы fixed stack allocation в модели unwind. Для обычной функции с fixed allocation это достаточно понятно: пролог уменьшил RSP, и дальше RSP в теле остаётся стабильным относительно fixed frame.

Если FrameRegister ненулевой, offset считается от места, где RSP находился в момент установки FP. Документация формулирует это так: база равна FP register минус FP register offset, где offset = 16 * scaled FrameOffset. То есть если R13 = RSP_at_setup + 0x80, то база для offsets вычисляется как R13 - 0x80. Это не обязательно текущий RSP на момент исключения. При alloca текущий RSP уже может быть ниже.

Это принципиально важно. Если decoder использует текущий RSP вместо reconstructed frame base, он ошибётся в функции с dynamic allocation. Если decoder использует FP напрямую, не вычитая FrameOffset, он тоже ошибётся. Правильная формула учитывает оба поля: FrameRegister и FrameOffset.

```
Если FrameRegister == 0:
frame_base = RSP_after_fixed_allocation

Если FrameRegister != 0:
frame_base = CONTEXT[FrameRegister] - FrameOffset * 16

Пример:
FrameRegister = R13
FrameOffset = 8
R13 = 0x000000000010F080

Тогда:
frame_base = 0x000000000010F080 - 0x80
```

```
= 0x000000000010F000

SAVE_NONVOL offset=0x30 означает:
saved_value_address = frame_base + 0x30
```

Самая опасная ошибка: считать SAVE_NONVOL offset от текущего RSP всегда. В функциях с frame pointer и особенно с alloca текущий RSP может быть уже не той базой, которую использует unwind metadata.

Frame pointer и эпилог

Frame pointer влияет не только на доступ к локальным данным и offset-операции. Он также влияет на форму выхода из функции. В разделе про эпилог мы уже видели, что x64 epilog должен быть строго распознаваемым, чтобы unwinder мог понять: RIP находится внутри эпилога, значит нужно симулировать остаток выхода из функции.

Если frame pointer не используется, эпилог обычно освобождает fixed allocation через add rsp, constant, затем восстанавливает nonvolatile-регистры через pop и выполняет ret. Если frame pointer используется, эпилог может сначала восстановить RSP относительно FP-регистра: например lea rsp, constant[FPReg]. Microsoft описывает допустимую форму: epilog может состоять из add RSP, constant или lea RSP, constant[FPReg], затем серии pop и ret/jmp в допустимой форме.

Почему это связано с unwind? Потому что unwinder не хранит отдельное описание каждого эпилога. Он распознаёт эпилог, сканируя поток инструкций. Если функция использует frame pointer, законный эпилог должен иметь такие инструкции, которые unwinder умеет распознать и симулировать. Поэтому frame pointer - это не только внутренняя удобная база для компилятора. Это часть контракта между кодогенератором и системным unwind-механизмом.

Для reverse engineer-а это даёт ещё одну проверку. Если UNWIND_INFO заявляет FrameRegister=R13 и FrameOffset=8, то эпилоги, которые восстанавливают RSP через R13, должны быть согласованы с этой формулой. Если эпилог использует lea rsp, [r12+...] при FrameRegister=R13, это повод проверить: может быть, перед нами не эпилог, может быть, дизассемблер неверно определил границы, а может быть, metadata и код расходятся.

```
Без frame pointer:
add rsp, 80h
pop r13
pop r14
ret

С frame pointer:
lea rsp, [r13 + fixed_size_minus_frame_offset]
pop r13
pop r14
ret

Смысл:
RSP может быть восстановлен через FPReg, если текущий RSP уже не является
простой базой fixed allocation.
```

Frame pointer chain и Windows x64 unwind - не одно и то же

Теперь нужно отдельно разобрать распространённое заблуждение: "stack trace строится по frame pointer chain". Иногда это верно для старого кода, для отладочных сборок или для архитектур/режимов, где frame pointer действительно используется как связный список кадров. Но Windows x64 в нормальной модели не требует обязательной frame pointer chain.

Frame pointer chain (цепочка указателей кадров) - это техника, при которой каждый кадр хранит старый frame pointer, а текущий frame pointer указывает на предыдущий кадр. В простом x86-прологе push ebp; mov ebp, esp действительно создаётся такая структура: [EBP] хранит старый EBP, [EBP+4] хранит return address. Отладчик может идти по этой цепочке.

Windows x64 unwind устроен иначе. Основной механизм - table-driven unwinding (табличная раскрутка). По RIP система находит RUNTIME_FUNCTION в .pdata, по нему находит UNWIND_INFO в .xdata, по unwind codes восстанавливает RSP и nonvolatile registers, затем читает return address и переходит к caller frame. Frame pointer может помочь, но не является обязательной опорой.

Поэтому отсутствие RBP-chain не означает, что stack trace невозможен. Если .pdata/.xdata корректны, stack walking работает без frame pointer. И наоборот, наличие RBP в коде не гарантирует корректный stack trace, если unwind metadata повреждена или намеренно противоречит коду. Для Windows x64 официальная истина для системного unwind - это не "похоже на RBP-chain", а согласованная пара code + unwind metadata.

Отладчик может показывать красивые имена функций, локальные переменные и исходные строки с помощью PDB. Но для системного восстановления кадра на x64 ключевыми остаются .pdata/.xdata и правила unwind, а не обязательная цепочка RBP.

Почему FrameRegister должен быть nonvolatile

Если frame pointer должен сохранять стабильную базу кадра, он не может быть произвольным volatile-регистром. Volatile-регистр по calling convention может быть испорчен callee-функцией при вызовах. Если функция использовала бы volatile-регистр как frame pointer и затем вызвала другую функцию, этот регистр мог бы измениться, и база кадра была бы потеряна. Поэтому FrameRegister в UNWIND_INFO описывает nonvolatile-регистр.

Nonvolatile register (сохраняемый регистр) - это регистр, который callee обязана вернуть caller-у в прежнем состоянии, если она его использует. В Windows x64 к таким GPR относятся RBX, RBP, RDI, RSI, R12-R15; RSP имеет особый статус как stack pointer, его нельзя рассматривать как обычный frame pointer через это поле. Если frame pointer выбран из nonvolatile-регистров, функция может сохранить старое значение этого регистра в прологе, использовать регистр как базу кадра, а в эпилоге восстановить старое значение.

Для validator-а это даёт ABI-level проверку. Если FrameRegister кодирует RCX, RDX, R8, R9, RAX, R10 или R11, это не нормальная запись frame pointer для Windows x64. Parser может технически прочитать число, но semantic validator должен отметить suspicious or invalid frame register. Здесь опять важно разделять два уровня: binary parser может извлечь nibble, но analyzer должен понимать ABI.

Регистр	Можно ли обычно использовать как FPReg в Windows x64 metadata	Почему
RBP	Да	Классический nonvolatile frame pointer
RBX, RSI, RDI, R12-R15	Да, если код и metadata согласованы	Nonvolatile GPR; могут быть сохранены и использоваться как стабильная база
RAX, RCX, RDX, R8-R11	Нет как нормальный FrameRegister	Volatile по ABI; caller не ждёт их сохранности
RSP	Нет в обычном смысле FrameRegister	Это сам stack pointer, не отдельный frame pointer register

Как это выглядит при ручном чтении байтов

Теперь переведём идею в ручной бинарный разбор. Представим, что decoder уже дошёл до UNWIND_INFO. В начале структуры лежит первый байт Version/Flags, затем SizeOfProlog, затем CountOfCodes, затем байт FrameRegister/FrameOffset. Именно этот четвёртый байт нас сейчас интересует.

Допустим, этот байт равен 0x85. В little-endian здесь нечего переставлять: это один байт. Младшие 4 бита равны 5, старшие 4 бита равны 8. Значит FrameRegister=5, FrameOffset=8. В стандартной нумерации GPR код 5 соответствует RBP. Реальное смещение равно 8*16 = 128 байт. Смысл: функция заявляет, что в прологе RBP установлен как RSP + 0x80.

Если рядом в UNWIND_CODE есть UWOP_SET_FPREG с CodeOffset, который указывает на инструкцию lea rbp, [rsp+80h], картина согласована. Если UWOP_SET_FPREG отсутствует, возникает подозрение: заголовок говорит о frame pointer, но unwind-коды не показывают момент установки. Если в дизассемблере вместо lea rbp, [rsp+80h] видно mov rbp, rsp, а FrameOffset=8, нужно проверять точнее: возможно, реальный offset другой; возможно, decoder неверно прочитал byte order или поле; возможно, это другой участок кода.

Другой пример: байт равен 0x0D. Младшие 4 бита - 13, значит FrameRegister=R13. Старшие 4 бита - 0, значит FrameOffset=0. Смысл: R13 установлен равным RSP в момент UWOP_SET_FPREG. Это может быть легально, если пролог действительно делает lea r13, [rsp] или mov r13, rsp в допустимой форме. Сам факт, что это не RBP, не является ошибкой.

```
Пример 1:
raw byte = 0x85
FrameRegister = 0x85 & 0x0F = 5 -> RBP
FrameOffset = 0x85 >> 4 = 8 -> 8*16 = 0x80
meaning: RBP = RSP + 0x80

Пример 2:
raw byte = 0x0D
FrameRegister = 13 -> R13
FrameOffset = 0 -> 0 bytes
meaning: R13 = RSP

Пример 3:
raw byte = 0x20
```

```
FrameRegister = 0 -> no FP register
FrameOffset = 2 -> suspicious, потому что FP не заявлен
```

Если FrameRegister=0, ненулевой FrameOffset обычно не имеет смысла и должен попасть в diagnostics. Такой случай может быть результатом повреждения, нестандартного генератора, ошибки parser-а или намеренной путаницы.

Как связать FrameRegister с дизассемблированным прологом

Теоретический разбор полей полезен, но reverse engineering требует ещё одного шага: сверки метаданных с реальным кодом. Если UNWIND_INFO заявляет frame pointer, в прологе должна быть инструкция, которая этот frame pointer устанавливает. Обычно это lea FPReg, [rsp + offset]. Иногда возможны близкие эквиваленты, но смысл должен совпадать: выбранный nonvolatile-регистр получает значение RSP + FrameOffset*16.

Здесь нельзя работать грубо. Нужно учитывать CodeOffset у UWOP_SET_FPREG. Он указывает на конец инструкции, которая установила FP. Поэтому analyzer должен вычислить адрес: FunctionBeginVA + CodeOffset, затем посмотреть инструкцию, которая заканчивается перед этой точкой. Если дизассемблер показывает lea r13, [rsp+80h], а FrameRegister=R13 и FrameOffset=8, всё хорошо. Если показывает lea r13, [rsp+70h], есть расхождение. Если показывает mov r13, rax, это уже серьёзное расхождение.

Но и здесь нельзя сразу кричать "обман". Возможны ошибки дизассемблирования, неправильный image base, неверный перевод RVA->file offset, обрезанная секция, нестандартный ассемблерный код или chained unwind info. Хороший инструмент формулирует finding аккуратно: metadata declares FPReg=R13 and FrameOffset=0x80, but instruction at prolog offset appears not to establish this value. Дальше он показывает адреса, байты, расшифровку полей и дизассемблерную строку.

Такая подача важна для твоего static /dynamic. Ценность инструмента не в том, чтобы выдавать громкое "BAD". Ценность в том, чтобы показать доказательство: что заявлено в metadata, что видно в code layer, какой invariant нарушен и почему это влияет на unwind correctness.

Инварианты для validator-а

Теперь соберём правила, которые analyzer должен проверять. Инвариант - это условие, которое должно выполняться в нормальной согласованной картине. Нарушение инварианта не всегда доказывает злой умысел, но всегда требует объяснения.

Первый инвариант: если FrameRegister равен нулю, функция не должна иметь содержательного FrameOffset. Ненулевой FrameOffset при нулевом FrameRegister выглядит бессмысленно, потому что offset относится к несуществующему FP register.

Второй инвариант: если FrameRegister ненулевой, он должен кодировать допустимый nonvolatile-регистр. Volatile-регистр как FPReg подозрителен, потому что он не обеспечивает стабильность через вызовы.

Третий инвариант: если FrameRegister ненулевой, в unwind codes должна быть операция UWOP_SET_FPREG, показывающая момент установки frame pointer. Без неё заголовок говорит о результате, но не даёт временной точки внутри пролога.

Четвёртый инвариант: UWOP_SET_FPREG должен иметь CodeOffset внутри SizeOfProlog и должен соответствовать реальной инструкции пролога, устанавливающей FPReg = RSP + FrameOffset*16. Это связывает метаданные с кодом.

Пятый инвариант: если FrameRegister используется, unwind-коды, которые берут stack offset, должны применяться только после установки FPReg в прологе. Иначе offset может быть рассчитан от базы, которая на этой точке ещё не существует.

Шестой инвариант: если используется chained unwind info, frame register и fixed-stack allocation fields должны быть согласованы с primary unwind info. Microsoft прямо указывает, что при UNW_FLAG_CHAININFO frame register and fixed-stack allocation fields должны иметь те же значения, что и в primary unwind info. Это важно для функций со сложной оптимизацией, где unwind-информация дробится на несколько связанных структур.

```
Проверки static /dynamic для FrameRegister/FrameOffset:

1. FrameRegister == 0 and FrameOffset != 0
-> suspicious_frame_offset_without_frame_register

2. FrameRegister != 0 and register is volatile
-> invalid_or_suspicious_frame_register

3. FrameRegister != 0 and no UWOP_SET_FPREG
-> missing_set_fpreg

4. UWOP_SET_FPREG CodeOffset > SizeOfProlog
-> set_fpreg_outside_prolog

5. Disassembly instruction does not match FP = RSP + FrameOffset*16
-> frame_register_code_metadata_mismatch

6. Chained unwind info has different frame fields than primary
-> chained_frame_fields_mismatch
```

Типичные ошибки ручного анализа

Первая ошибка - считать, что RBP всегда является frame pointer. В оптимизированном x64-коде RBP может вообще не использоваться как frame pointer. Он может быть обычным nonvolatile-регистром, может быть свободен, может не встречаться. A frame pointer, если он есть, может быть R13 или другой nonvolatile-регистр.

Вторая ошибка - считать отсутствие frame pointer аномалией. На Windows x64 это нормальное состояние многих функций. Если функция имеет корректную .pdata/.xdata и fixed stack allocation, unwinder спокойно восстановит caller context без RBP-chain.

Третья ошибка - читать FrameOffset как байты. Значение 5 означает 80 байт, а не 5 байт. Это настолько частая ошибка, что хороший отчёт должен всегда показывать scaled и byte forms одновременно.

Четвёртая ошибка - путать UWOP_SET_FPREG с сохранением RBP. SET_FPREG устанавливает новый frame pointer. Сохранение старого RBP/R13 должно описываться PUSH_NONVOL или SAVE_NONVOL. Если этого сохранения нет, но регистр nonvolatile используется и потом должен быть возвращён, возникает отдельный вопрос.

Пятая ошибка - считать offsets от текущего RSP даже при наличии FrameRegister. В функции с alloca текущий RSP может быть ниже fixed frame. Правильная база вычисляется через FP - FrameOffset*16.

Шестая ошибка - смотреть только metadata и не сверять с кодом. Unwind metadata сама по себе говорит, что должно быть. Дизассемблированный пролог показывает, что реально лежит в .text. Для reverse engineering особенно важны места, где эти два слоя не совпадают.

Седьмая ошибка - смотреть только код и игнорировать metadata. Можно увидеть lea rbp, [rsp+20h] и решить, что это frame pointer, но если UNWIND_INFO не заявляет FrameRegister, системный unwinder не будет использовать эту информацию как официальную часть unwind-модели. Для анализа Windows x64 важно держать оба слоя одновременно.

Что должен показывать static /dynamic

Для проекта static /dynamic frame pointer - хорошая тема, потому что она находится на границе формального парсинга и смысловой проверки. Просто распечатать FrameRegister=5 мало. Нужно перевести число в регистр, объяснить его роль, показать scaled offset, вычислить byte offset, найти UWOP_SET_FPREG, сверить CodeOffset с прологом и показать, как будет вычисляться frame base.

Хороший отчёт по функции может выглядеть так: функция имеет RUNTIME_FUNCTION range Begin=..., End=..., UnwindInfo=...; UNWIND_INFO заявляет FrameRegister=R13, FrameOffset=8, то есть frame pointer должен быть R13 = RSP + 0x80; в unwind codes найден UWOP_SET_FPREG at CodeOffset=0x0B; дизассемблер показывает lea r13, [rsp+80h]; offset-операции SAVE_NONVOL считаются от frame_base = R13 - 0x80; несоответствий нет.

Если есть проблема, отчёт должен быть таким же доказательным: FrameRegister=R13, FrameOffset=8, но UWOP_SET_FPREG отсутствует; при этом есть SAVE_NONVOL offset-операции, требующие корректной frame base; следовательно, unwind metadata неполна или нестандартна. Severity можно делать средним или высоким в зависимости от того, ломает ли это реально восстановление context и есть ли другие признаки повреждения.

Важно, что static /dynamic не должен превращать frame pointer в бинарную метку "хорошо/плохо". Наличие frame pointer не делает функцию безопасной. Отсутствие frame pointer не делает функцию подозрительной. Значение имеет согласованность: заявленная роль регистра, код пролога, порядок UNWIND_CODE, offset-формулы, эпилоги и chained info.

```
Пример человекочитаемого вывода:

Frame model:
frame_register_raw: 13
frame_register: R13
frame_offset_raw: 8
frame_offset_bytes: 0x80
frame_base_formula: R13 - 0x80

SET_FPREG:
code_offset: 0x0B
expected: R13 = RSP + 0x80
```

```
disasm: lea r13, [rsp+80h]
status: ok

Offset users:
SAVE_NONVOL RBX at frame offset 0x30
effective address formula: (R13 - 0x80) + 0x30

Findings:
none
```

Полный жизненный цикл frame pointer внутри одной функции

Чтобы frame pointer перестал быть абстрактным термином, полезно пройти его жизненный цикл как историю одной функции. Сначала caller вызывает callee. Инструкция call кладёт return address в стек и передаёт управление на начало callee. В этот момент RSP указывает на return address. Никакого нового кадра callee ещё не построила. Она только получила управление.

Дальше начинается пролог. Если callee собирается использовать R13 как frame pointer, она не имеет права просто затереть R13. R13 является nonvolatile-регистром: его старое значение принадлежит caller-у. Поэтому callee сначала сохраняет R13, например push r13. Это действие меняет RSP и помещает старый R13 в стек. В UNWIND_CODE это будет UWOP_PUSH_NONVOL с OpInfo=R13. Пока frame pointer ещё не установлен. Регистр R13 всё ещё содержит старое значение caller-а или какое-то значение, которое не является базой нового кадра.

Затем функция выделяет fixed stack allocation: sub rsp, 100h. Теперь появилась фиксированная область кадра. В UNWIND_CODE это будет UWOP_ALLOC_LARGE или UWOP_ALLOC_SMALL, в зависимости от размера. Но даже сейчас frame pointer ещё может не существовать. RSP уже указывает на нижнюю границу fixed allocation, однако функция может захотеть поставить FP не туда, а внутрь этой области.

Следующая инструкция, например lea r13, [rsp+80h], создаёт новый frame pointer. Только с этой точки R13 становится frame pointer текущей функции. Именно эту точку отмечает UWOP_SET_FPREG. В заголовке UNWIND_INFO уже заранее написано FrameRegister=R13 и FrameOffset=8, но момент, с которого это становится истинным во время исполнения пролога, задаёт CodeOffset у SET_FPREG.

После этого функция может обращаться к локальным переменным через R13: mov [r13-20h], rcx, mov rax, [r13+18h] и так далее. Для обычного исполнения это просто удобная адресация. Для unwind это означает, что сохранённые nonvolatile-регистры, описанные offset-операциями, могут находиться относительно базы, которую нужно вычислять как R13 - 0x80.

Если позже функция делает alloca и уменьшает RSP на runtime-size, R13 остаётся прежним. Текущий RSP ушёл вниз, но fixed frame всё ещё можно найти через R13. Именно ради этого frame pointer и был нужен. В эпилоге функция должна вернуть RSP к fixed allocation, восстановить сохранённые регистры и выполнить ret. Если frame pointer был R13, старый R13 восстанавливается из стека, а не "вычисляется" из FrameRegister. Metadata описывает обе стороны: SET_FPREG сообщает, как R13 стал frame pointer, PUSH/SAVE_NONVOL сообщает, где лежит старый R13.

```
Жизненный цикл на одном примере:

call callee
; RSP -> return address

callee:
00 push r13 ; сохранить старый R13 caller-a
02 sub rsp, 100h ; fixed stack allocation
09 lea r13, [rsp+80h] ; новый frame pointer текущей функции
11 mov [r13-20h], rbx ; доступ через FP
...
30 sub rsp, rax ; dynamic allocation, RSP меняется
...
80 lea rsp, [r13+80h] ; вернуть RSP к зоне сохранений/выходу
87 add rsp, 100h
8E pop r13 ; восстановить старый R13 caller-a
90 ret

Ключевая мысль:
FrameRegister=R13 не говорит, что старый R13 сохранён.
Сохранение старого R13 и установка нового R13 как FP - разные события.
```

Почему frame pointer omission не ломает reverse engineering

Frame pointer omission (опущение frame pointer, отказ от выделенного указателя кадра) - это оптимизация, при которой компилятор не держит отдельный регистр как frame pointer. В старом x86-контексте это часто воспринималось как проблема для отладки: если нет EBP-chain, стек сложнее пройти. На Windows x64 ситуация другая, потому что основная модель stack walking изначально табличная.

Оптимизатор отказывается от frame pointer не из вредности. Он экономит регистр и инструкции. Если функция имеет простой fixed-size frame, RSP после пролога стабилен, а все локальные переменные доступны через RSP+constant. Тогда отдельный RBP/R13 как база часто не нужен. Более того, если сохранить RBP только ради привычки, функция получит лишний push/pop, лишнюю зависимость и потеряет один nonvolatile-регистр для полезной работы.

Для reverse engineer-а это меняет психологию анализа. Отсутствие mov rbp, rsp не означает, что функция "без кадра". Кадр может быть, просто он описан через RSP и unwind metadata. Функция может выделить 0x90 байт, сохранить RBX и RDI, вызвать другие функции и иметь полноценную запись RUNTIME_FUNCTION, но при этом FrameRegister=0. Это абсолютно нормальная картина.

Decompilers (декомпиляторы) тоже умеют восстанавливать локальные переменные без frame pointer. Они строят собственную модель stack variables, анализируя изменения RSP, обращения к памяти, unwind-информацию и иногда PDB. Поэтому когда Ghidra, IDA или Binary Ninja показывают локальную переменную как local_38, это не означает, что в реальном коде был RBP. Это означает, что инструмент построил абстрактную модель кадра.

Проблема начинается не тогда, когда frame pointer отсутствует, а когда отсутствует согласованная информация для восстановления. Например, если функция меняет RSP динамически, не использует frame pointer, не имеет корректных unwind данных и при этом может участвовать в exception unwinding, вот это уже серьёзная проблема. Но обычная optimized no-frame-pointer function - это норма.

Frame pointer omission - нормальная оптимизация. Для Windows x64 более важный вопрос звучит так: можно ли по .pdata/.xdata и коду восстановить caller context? Если да, отсутствие RBP-chain не является проблемой.

Отладочная сборка, релизная сборка и разный вид frame pointer

В отладочной сборке компилятор часто делает код более удобным для debugger-а и человека. Он может сохранять frame pointer чаще, меньше переупорядочивать инструкции, оставлять больше прямых соответствий между исходным кодом и машинным кодом. Поэтому новичок, который смотрит только debug build, быстро привыкает к красивой картине: RBP стоит как база, локальные переменные удобно видны, стек кажется линейным и понятным.

Release build (релизная сборка, оптимизированная сборка) живёт по другим приоритетам. Компилятор старается уменьшить код, ускорить выполнение, использовать регистры эффективнее, убирать лишние сохранения, объединять эпилоги, откладывать сохранение регистра до нужного пути, иногда применять shrink-wrapping. В такой сборке frame pointer может исчезнуть, а стековая картина станет менее похожей на учебник.

Это не значит, что release build менее "правильная". Наоборот, это обычная форма промышленного кода. Правильная книга по reverse engineering должна учить не только красивому debug-прологу, но и тому, как система понимает оптимизированный код. Windows x64 unwind metadata как раз существует для того, чтобы оптимизированный код всё равно оставался unwindable.

PDB (Program Database, база отладочной информации) может дать отладчику имена локальных переменных, типы, строки исходного кода и дополнительные сведения. Но PDB не заменяет unwind metadata для системного механизма исключений. Если PDB отсутствует, unwind всё равно должен работать по .pdata/.xdata. Если PDB есть, он помогает человеку, но не отменяет необходимость корректного UNWIND_INFO.

Как frame pointer видит декомпилятор

Декомпилятор не обязан буквально показывать тот же регистр, который использовался в машинном коде. Он строит high-level representation (высокоуровневое представление): псевдокод, локальные переменные, временные значения, аргументы. Поэтому decompiler может скрыть RSP-relative адресацию и показать обычные local variables. Это удобно, но для анализа unwind иногда опасно: псевдокод сглаживает детали, которые важны для .xdata.

Например, машинный код может использовать R13 как frame pointer, а декомпилятор покажет просто local_40. Если ты смотришь только на псевдокод, ты не увидишь, что FrameRegister=R13 и что эпилог восстанавливает RSP через R13. Поэтому при анализе unwind metadata нельзя ограничиваться decompiler view. Нужны bytes, disassembly и raw unwind fields.

С другой стороны, декомпилятор полезен для проверки здравого смысла. Если metadata говорит о fixed allocation 0x200, frame pointer R13+0x80, а декомпилятор показывает много локальных объектов вокруг этого диапазона, картина выглядит естественно. Если metadata говорит о маленьком кадре, а код активно пишет далеко за его пределы, нужно проверять: может быть, это dynamic allocation, может быть, ошибка восстановления stack

frame, а может быть, нестандартная защита.

Для static /dynamic хорошая стратегия - не пытаться заменить Ghidra/IDA, а давать слой строгой unwind-диагностики: raw fields, decoded fields, formulas, invariants, links to disassembly. Тогда reverse engineer может сопоставить это с decompiler view и понять, где инструмент высокого уровня сгладил опасную деталь.

Frame pointer и chained unwind info

Chained unwind info (связанная unwind-информация) появляется, когда одна UNWIND_INFO структура является не основной для процедуры, а вторичной и ссылается на предыдущую RUNTIME_FUNCTION запись. Полностью этот механизм будет разобран позже, но здесь важно понять одну связь: frame register fields не должны противоречить основной информации.

Microsoft указывает, что если установлен UNW_FLAG_CHAININFO, то frame register and fixed-stack allocation fields должны иметь те же значения, что и в primary unwind info. Это требование не декоративное. Если secondary unwind info внезапно заявит другой FrameRegister или другой fixed allocation, то разные части одной функции будут давать разные формулы frame base. Unwinder мог бы восстановить разные адреса сохранённых регистров в зависимости от того, в какой RUNTIME_FUNCTION entry попал ControlPc. Это разрушает целостность unwind-модели.

Для validator-а это означает отдельную проверку. Когда analyzer видит UNW_FLAG_CHAININFO, он не должен анализировать вторичную структуру как полностью независимую функцию. Нужно добраться до primary unwind info, сравнить frame fields и allocation model. Если frame register отличается, это сильный finding: chained unwind info disagrees with primary frame model.

В реальном reverse engineering chained info может встречаться при оптимизациях вроде shrink-wrapping, где разные диапазоны функции имеют разное состояние сохранённых регистров. Но даже там frame model должна оставаться согласованной в рамках требований формата. Поэтому frame pointer здесь становится не только локальным полем, а частью общей consistency check для всей функции.

Frame pointer и безопасность анализа: что может делать протектор

dynamic или obfuscator может использовать unwind metadata как область для запутывания инструментов. Это не обязательно значит, что он ломает работу Windows. Иногда metadata остаётся формально валидной для системного unwinder-а, но становится неприятной для статического анализа. В других случаях metadata может быть повреждена намеренно, если автор рассчитывает, что код не будет проходить через обычный exception unwinding или что повреждение затронет только сторонние инструменты.

FrameRegister/FrameOffset дают несколько способов создать путаницу. Можно заявить frame pointer, но сделать пролог трудным для сопоставления. Можно использовать нестандартный nonvolatile-регистр, который многие простые анализаторы не ожидают. Можно сделать много chained entries. Можно подобрать FrameOffset так, что наивный decoder, забывший умножение на 16, будет строить неправильные адреса. Можно положить metadata в форму, которая формально парсится, но не совпадает с дизассемблированным прологом.

Хороший static /dynamic не должен пугаться самого факта "R13 as frame pointer". Это легально. Он должен искать доказуемые противоречия: FrameRegister volatile; FrameOffset nonzero when no FrameRegister; missing SET_FPREG; SET_FPREG points to non-matching instruction; offset users appear before frame pointer setup; chained info conflicts with primary; computed frame base leads outside plausible stack allocation.

Именно поэтому в этой книге мы постоянно разделяем normal uncommon и suspicious invalid. Normal uncommon (нормальное, но редкое) - например R13 as frame pointer в большой функции. Suspicious invalid (подозрительное или неверное) - например FrameRegister=RCX или UWOP_SET_FPREG с OpInfo nonzero и без соответствующей инструкции. Для качественного инструмента это разные уровни вывода.

Алгоритм разбора FrameRegister для собственного parser-а

Теперь можно записать алгоритм так, как его должен выполнить parser. Сначала он читает UNWIND_INFO header. Из четвёртого байта он выделяет младшие 4 бита как FrameRegister и старшие 4 бита как FrameOffset. Затем он переводит FrameRegister в имя регистра по таблице GPR encoding. Если значение нулевое, parser помечает модель как no frame pointer. Если значение ненулевое, parser проверяет, относится ли регистр к допустимым nonvolatile-регистрам.

Дальше parser вычисляет frame_offset_bytes = FrameOffset * 16. Он не должен терять raw value, потому что raw useful для forensic/debug отчёта. Но все формулы адресации должны использовать byte value. Затем parser просматривает unwind codes и ищет UWOP_SET_FPREG. Если FrameRegister ненулевой, отсутствие SET_FPREG становится finding. Если SET_FPREG найден при FrameRegister=0, это тоже finding, потому что операция установки FP без заявленного FP register бессмысленна.

Затем analyzer сопоставляет CodeOffset у SET_FPREG с прологом. Для этого ему нужны BeginAddress функции, RVA/VA conversion и дизассемблирование инструкций в диапазоне [BeginAddress, BeginAddress+SizeOfProlog). Analyzer должен найти инструкцию, заканчивающуюся на BeginAddress+CodeOffset. После этого он проверяет, устанавливает ли эта инструкция нужный register как RSP + frame_offset_bytes.

Наконец, все offset-операции получают правильную формулу frame base. Если FrameRegister=0, формула использует RSP model. Если FrameRegister ненулевой, формула использует Context[FPReg] - frame_offset_bytes. В отчёте это нужно показывать явно, иначе пользователь не поймёт, откуда берутся адреса сохранённых регистров.

```
Псевдоалгоритм:

raw = unwind_info.frame_byte
frame_register_raw = raw & 0x0F
frame_offset_raw = raw >> 4
frame_offset_bytes = frame_offset_raw * 16

if frame_register_raw == 0:
frame_model = "no frame pointer"
if frame_offset_raw != 0:
finding("frame_offset_without_frame_register")
else:
fpreg = decode_gpr(frame_register_raw)
if not is_nonvolatile_gpr(fpreg):
finding("suspicious_frame_register")
```

```
set_fpreg = find_uwop_set_fpreg(unwind_codes)
if not set_fpreg:
finding("missing_set_fpreg")
else:
expected = f"{fpreg} = RSP + {frame_offset_bytes:#x}"
check_disassembly_at_code_offset(set_fpreg.CodeOffset, expected)

for op in offset_using_unwind_ops:
if frame_register_raw == 0:
base = "RSP_after_fixed_allocation"
else:
base = f"{fpreg} - {frame_offset_bytes:#x}"
effective = f"{base} + {op.offset:#x}"
```

Мини-глоссарий этой темы

Frame (кадр) - область состояния, связанная с одним вызовом функции: return address, сохранённые регистры, локальные переменные, временные slots, часть параметров и служебные области. Кадр не обязан быть оформлен через RBP. На Windows x64 он может быть восстановлен через unwind metadata.

Stack pointer / RSP (указатель стека) - текущий указатель вершины стека. Он меняется при push/pop/call/ret и при выделении/освобождении stack space. Он всегда важен для unwind, потому что восстановление caller context в итоге должно привести RSP к состоянию caller-a.

Frame pointer / FP (указатель кадра) - выбранный функцией стабильный регистр-база внутри кадра. Он нужен не всегда. Он особенно полезен при dynamic allocation и для компактной адресации локальных данных.

FrameRegister (поле регистра кадра) - 4-битное поле UNWIND_INFO, которое кодирует, какой nonvolatile-регистр выполняет роль frame pointer. Ноль означает отсутствие заявленного FP register.

FrameOffset (поле смещения регистра кадра) - 4-битное поле UNWIND_INFO, которое хранит scaled offset. Реальное смещение равно значению поля, умноженному на 16.

UWOP_SET_FPREG (операция установки frame pointer) - unwind code, который отмечает момент в прологе, когда FPReg был установлен относительно RSP. Он не сохраняет старое значение регистра и не содержит номер регистра внутри OpInfo.

Frame base (база кадра для offset-операций) - адресная база, от которой считаются offsets сохранённых регистров. Если FPReg отсутствует, база связана с RSP. Если FPReg есть, база вычисляется как FPReg - FrameOffset*16.

Связь с предыдущими и следующими разделами

Теперь можно увидеть, почему frame register оказался именно в этой части книги. Сначала мы разобрали .pdata: таблицу диапазонов функций. Потом .xdata: место, где лежит инструкция для unwind. Потом UNWIND_INFO: заголовок и массив описания пролога. Потом UNWIND_CODE и UWOP_*: маленький язык действий пролога. FrameRegister и FrameOffset находятся в центре этой модели, потому что они задают базу для части этих действий.

Без понимания frame pointer нельзя уверенно читать UWOP_SAVE_NONVOL и UWOP_SAVE_XMM128 в функциях с FPReg. Нельзя корректно анализировать alloca. Нельзя нормально проверять эпилоги с lea rsp, constant[FPReg]. Нельзя отличать нормальную оптимизацию от metadata inconsistency. И нельзя строить качественный static/dynamic, потому что инструмент будет либо слишком часто ругаться на нормальный x64-код, либо пропускать реальные противоречия.

Следующий раздел логически продолжит эту тему через volatile и nonvolatile registers. Мы уже несколько раз говорили, что frame pointer должен быть nonvolatile, что PUSH_NONVOL сохраняет nonvolatile-регистр, что caller context требует восстановления именно nonvolatile-состояния. Теперь нужно отдельно и фундаментально разобрать, что вообще означает volatile/nonvolatile на уровне ABI, почему это не "хорошие" и "плохие" регистры, и почему без этого нельзя понять unwind metadata.

4.6. Что такое volatile и nonvolatile регистры

После frame register и frame pointer мы подходим к теме, которая выглядит простой только на первый взгляд. Кажется, что volatile и nonvolatile registers - это просто два списка имён регистров: одни можно портить, другие надо сохранять. Но для Windows x64 unwind это не справочная таблица, а один из главных смысловых мостов между calling convention, прологом функции, .xdata, UNWIND_CODE и восстановлением caller context. Если этот мост не понять, дальше вся тема unwind превратится в набор механических кодов без причины.

Давай начнём не с таблицы, а с живой ситуации. Одна функция вызывает другую. В момент вызова у caller-а уже есть какие-то важные значения: указатели на объекты, счётчики, промежуточные результаты, адреса буферов, значения локальных переменных, которые компилятор решил держать не в памяти, а в регистрах. Затем управление уходит в callee. Callee тоже нужны регистры. Она тоже хочет быстро считать, сравнивать, хранить адреса, готовить аргументы для других вызовов. Но регистров конечное число. Если каждая функция будет использовать их как захочет, без общего правила, то после возврата caller может обнаружить, что важное значение исчезло.

Именно здесь появляется соглашение. Не инструкция процессора, не проверка Windows в каждом вызове, не магия компилятора, а contract (контракт, соглашение сторон): какие регистры caller должен считать временными, а какие callee обязана вернуть в прежнем состоянии. Это соглашение является частью ABI. А ABI, как мы уже постепенно видим, не просто описывает "как красиво писать ассемблер". Оно определяет, какие состояния программы считаются корректными на границе между функциями.

Для unwind этот контракт особенно важен. При обычном возврате callee сама доходит до эпилога, восстанавливает сохранённые nonvolatile-регистры, чинит RSP и выполняет ret. Но при исключении нормального возврата может не быть. Функция может быть прервана в середине тела: до эпилога дело не дошло, сохранённые регистры ещё лежат где-то в stack frame, RSP ещё не восстановлен. Тем не менее Windows должна подняться к caller-у так, будто функция была аккуратно покинута. Значит, системе нужно знать, какие обязательные для сохранения регистры были сохранены и где именно они лежат. Поэтому nonvolatile registers напрямую появляются в unwind metadata.

register (регистр процессора) - это очень маленькое, очень быстрое хранилище внутри CPU, с которым машинные инструкции работают напрямую. Регистр не является переменной языка C/C++ и не живёт в PE-файле. Это часть текущего состояния выполняющегося потока: значения RAX, RBX, RCX, RSP, RIP, XMM0 и других регистров в конкретный момент исполнения.
CPU context / processor context (контекст процессора, состояние регистров потока) - это снимок значений регистров, который описывает, где сейчас находится поток и с какими значениями он работает. Для unwind важен именно контекст: надо восстановить не исходный текст программы, а набор регистров caller-а, включая RIP, RSP и сохранённые nonvolatile-регистры.
volatile register (летучий регистр, caller-saved register, регистр, который вызывающая сторона не должна считать сохранённым) - это регистр, значение которого может быть испорчено callee во время вызова. Если caller хочет сохранить такое значение после call, он должен сам позаботиться о сохранении до вызова.

nonvolatile register (нелетучий регистр, callee-saved register, регистр, который вызываемая функция обязана сохранить) - это регистр, который callee должна вернуть в прежнем состоянии, если она его использовала или изменила. Если функция трогает такой регистр, она должна сохранить старое значение в своём кадре и восстановить его перед возвратом; при исключении местоположение этого старого значения должно быть описано в unwind metadata.

calling convention (соглашение о вызовах) - это набор правил ABI, который говорит, как функции передают аргументы, где возвращают результат, кто выделяет место на стеке, кто обязан сохранять какие регистры и каким должно быть состояние стека в точках вызова и возврата. Volatile/nonvolatile деление является частью calling convention, а не произвольной привычкой компилятора.

Почему нельзя сразу начать с таблицы регистров

Таблица нужна, и мы её обязательно разберём. Но если начать с неё, возникает ложное ощущение, будто тема сводится к запоминанию: RAX плохой, RBX хороший, RCX аргумент, R12 надо сохранять. Это неправильная рамка. Volatile-регистр не плохой. Nonvolatile-регистр не хороший. Это разные роли в распределении ответственности между caller и callee.

CPU не знает, что RAX "volatile", а RBX "nonvolatile". Процессор спокойно выполнит mov rbx, 1234h, даже если функция не сохранила RBX. Железо не остановит программу и не скажет: "ты нарушил calling convention". Правило существует на уровне ABI, компилятора, ассемблерного кода, unwind-метаданных, отладчика и среды выполнения. То есть это социально-техническое правило машинного мира: все части toolchain договорились понимать границы функций одинаково.

Toolchain (инструментальная цепочка) - это не один инструмент, а вся связка, которая превращает исходный код в исполняемый образ и потом позволяет его запускать и анализировать: compiler (компилятор), assembler (ассемблер), linker (компоновщик), loader (загрузчик), runtime (среда выполнения), debugger (отладчик), crash dump analyzer (анализатор дампов), disassembler (дизассемблер). Если компилятор генерирует сохранение RBX, assembler кодирует unwind-операцию, linker кладёт записи в .pdata/.xdata, а RtlVirtualUnwind потом читает эти записи, все они должны иметь одно и то же понимание: RBX является nonvolatile.

Поэтому деление регистров - это не изолированная таблица. Это общий язык, на котором функция обещает окружающему миру: "вот эти значения после меня могут измениться, а вот эти я верну как было". Unwind metadata - это машинно-читаемое доказательство того, как конкретная функция выполняет свою часть обещания.

Главная мысль раздела: volatile/nonvolatile - это не свойства железа, а правила ABI о владении регистрами через границу function call. Именно nonvolatile-часть этого правила интересует unwind, потому что её надо восстановить даже тогда, когда обычный epilog не выполнился.

Caller и callee: две стороны одного вызова

Перед тем как углубляться в регистры, надо ещё раз закрепить пару терминов. Caller (вызывающая функция) - это функция, которая выполняет call. Callee (вызываемая функция) - это функция, в которую управление переходит после call. В исходном коде это

обычно выглядит просто: одна функция вызывает другую. Но на машинном уровне это граница ответственности.

Caller готовит аргументы, резервирует shadow space, может сохранить нужные volatile-регистры, затем выполняет call. Инструкция call кладёт return address на stack и меняет RIP на адрес callee. Callee начинает выполнение со своего пролога: может сохранить nonvolatile-регистры, выделить stack space, установить frame pointer, выполнить тело функции, восстановить состояние в эпилоге и выполнить ret. Инструкция ret берёт return address со стека и возвращает управление caller-у.

Если бы все регистры были обязанностью caller-а, каждый call требовал бы сохранять слишком много значений заранее, даже если callee их не тронет. Если бы все регистры были обязанностью callee, каждая функция вынуждена была бы сохранять и восстанавливать кучу регистров, даже если caller-у их значения не нужны. Поэтому ABI делит регистры на два класса. Часть регистров удобна для быстрых временных операций и передачи аргументов - они volatile. Другая часть удобна для значений, которые должны переживать вызовы других функций - они nonvolatile.

```
Caller/Callee на машинном уровне:

caller:
; здесь у caller есть своё состояние: регистры, стек, локальные данные
mov rcx, arg1
mov rdx, arg2
call callee
; после call caller снова продолжает выполнение

callee:
; здесь callee получает управление
; она может пользоваться volatile-регистрами свободно
; но nonvolatile-регистры обязана вернуть как было
ret
```

Volatile-регистры: рабочая зона, которую caller не имеет права считать вечной

Volatile register лучше понимать не как "опасный" регистр, а как scratch register. Scratch register (черновой регистр, временный рабочий регистр) - это регистр, который функция может использовать для промежуточных вычислений без обязанности сохранять старое значение для caller-а. В Windows x64 такие регистры нужны постоянно: через них передаются первые аргументы, в них возвращаются результаты, ими удобно пользоваться для адресной арифметики, сравнений, временных указателей и подготовки следующих вызовов.

Когда ABI говорит, что RAX volatile, это означает: caller не должен рассчитывать, что после call в RAX останется прежнее значение. Более того, RAX обычно используется для return value (возвращаемого значения), поэтому после вызова он как раз должен измениться, если функция возвращает integer или pointer. Когда RCX, RDX, R8 и R9 volatile, это логично: они используются как первые регистры аргументов. Callee получает в них входные значения и может дальше распоряжаться ими как рабочими регистрами.

Ошибка новичка здесь часто такая: он смотрит в отладчике, видит, что до call в RCX был какой-то указатель, после call он другой, и думает, что функция "сломала" caller-а. Но ABI заранее говорит: RCX volatile. Если caller хотел сохранить значение RCX после вызова, он

должен был сам переложить его в память или в nonvolatile-регистр до call. Callee не нарушила правило, она использовала регистр, который по соглашению свободен для порчи.

Слово "порча" в русских объяснениях звучит грубо, но оно полезно. Volatile register may be clobbered - это значит, что его значение может быть затёрто. Не обязательно злонамеренно, не обязательно мусором, не обязательно случайно. Просто caller не имеет права строить дальнейшую логику на предположении, что старое значение сохранилось.

scratch register (черновой регистр, временный рабочий регистр) - это регистр, который функция использует как временное место для вычислений. В контексте Windows x64 большинство scratch-регистров являются volatile: callee может менять их без сохранения старого значения.

clobber (затереть, испортить значение регистра) - это распространённый англоязычный термин из ABI/assembly-разговоров. Он не означает повреждение процессора или памяти. Он означает, что старое значение регистра больше не гарантируется после некоторого кода, например после function call.

Nonvolatile-регистры: обещание callee вернуть значение

Nonvolatile register работает по другой логике. Если caller положил важный указатель в RBX, затем вызвал функцию, то после возврата caller имеет право ожидать, что RBX всё ещё содержит тот же указатель. Это не потому, что RBX физически защищён. Это потому, что ABI возлагает на callee обязанность: если ты хочешь использовать RBX для своих целей, сначала сохрани старое значение, потом перед возвратом восстанови.

Callee может вообще не трогать RBX. Тогда сохранять нечего. Может использовать RBX только для чтения, не меняя значение. Тогда тоже нет необходимости сохранять. Но если callee записывает в RBX новое значение, она должна сохранить старое состояние. На x64 это обычно делается в прологе через push rbx или mov [rsp+offset], rbx. В unwind metadata это выражается через UWOP_PUSH_NONVOL или UWOP_SAVE_NONVOL. Позже в эпилоге функция выполнит pop rbx или mov rbx, [rsp+offset], возвращая старое значение.

И вот здесь появляется центральная связь с исключениями. Если функция нормально доходит до эпилога, регистр восстанавливается обычной машинной инструкцией. Но если в середине функции произошло исключение, обычного эпилога может не быть. Windows начинает unwind. Она должна построить caller context так, как если бы callee уже ушла. Значит, старый RBX надо восстановить прямо во время unwind. Но откуда Windows узнает, где лежит старый RBX? Из UNWIND_INFO. Поэтому nonvolatile-регистры - это именно те регистры, сохранение которых имеет смысл описывать в .xdata.

```
Обычная логика nonvolatile-регистра:

caller:
mov rbx, important_pointer ; caller хочет сохранить это через вызов
call callee
; здесь RBX должен быть тем же important_pointer

callee:
push rbx ; сохранить старый RBX
sub rsp, 20h
mov rbx, some_temporary ; callee использует RBX для себя
...
```

```
add rsp, 20h
pop rbx ; вернуть старый RBX
ret

UNWIND_INFO обязан описать push rbx, потому что исключение может произойти
между push rbx и pop rbx.
```

Таблица Windows x64: какие general-purpose registers volatile, а какие nonvolatile

Теперь таблица будет понятнее. General-purpose register (регистр общего назначения) - это integer/pointer-регистр архитектуры x86-64: RAX, RBX, RCX, RDX, RSI, RDI, RSP, RBP, R8-R15. Они называются "общего назначения" не потому, что у всех одинаковая роль в ABI, а потому, что архитектурно ими можно выполнять разные целочисленные, адресные и логические операции.

В Microsoft x64 calling convention volatile general-purpose registers обычно такие: RAX, RCX, RDX, R8, R9, R10, R11. Nonvolatile general-purpose registers: RBX, RBP, RDI, RSI, R12, R13, R14, R15. RSP стоит отдельно: он не просто "ещё один nonvolatile" в бытовом смысле, а stack pointer, который обязан быть восстановлен к корректному состоянию на границе возврата и описывается unwind-операциями изменения стека.

Важно не зубрить таблицу механически, а видеть роли. RAX удобен для return value и временных вычислений, поэтому volatile. RCX/RDX/R8/R9 используются для первых аргументов, поэтому volatile. R10/R11 часто используются как scratch registers, например в переходных механизмах, thunks, системных вызовах и сгенерированном коде, поэтому тоже volatile. А RBX, RBP, RSI, RDI, R12-R15 дают компилятору набор устойчивых регистров, в которых удобно держать значения, живущие через один или несколько nested calls.

Класс	Регистры	Главный смысл для caller/callee	Связь с unwind
Volatile GPR	RAX, RCX, RDX, R8, R9, R10, R11	Caller не ждёт сохранения после call. Callee может использовать как scratch.	Обычно не описываются как сохраняемые nonvolatile-регистры. Если caller хочет сохранить их, это его забота.
Nonvolatile GPR	RBX, RBP, RDI, RSI, R12, R13, R14, R15	Callee обязана вернуть прежнее значение, если изменила.	Если функция сохраняет их в прологе, это описывается UWOP_PUSH_NONVOL или UWOP_SAVE_NONVOL.
RSP	Stack pointer	Должен указывать на корректный стек в точках вызова/возврата.	Восстанавливается через отмену stack allocation, pop/push effects, epilog recognition и virtual unwind.

Не надо формулировать так: "RAX можно ломать, RBX нельзя трогать". Правильно: callee может менять RBX только при условии, что вернёт его прежнее значение. Поэтому RBX использовать можно; просто цена использования выше, потому что нужно сохранение, восстановление и unwind-описание.

Почему RSP стоит особняком

RSP (stack pointer, указатель стека) иногда попадает в списки nonvolatile-регистров, но его нужно понимать отдельно. Обычный nonvolatile-регистр вроде RBX можно сохранить на стеке, использовать как угодно, а потом восстановить. RSP - это не просто хранитель значения. Это текущая граница stack frame, механизм доступа к return address, основа работы push/pop/call/ret. Если RSP неверен, функция может вернуться не туда, прочитать не те данные, уничтожить чужую область стека или упасть при обращении к памяти.

Во время выполнения функции RSP вполне может меняться. Пролог делает sub rsp, N, выделяя fixed allocation. Push-инструкции уменьшают RSP. Epilog делает обратные действия. Поэтому в середине функции RSP не равен тому значению, которое было у caller-a. Но к моменту возврата из функции стек должен быть приведён к ожидаемому состоянию: ret должен снять правильный return address, а caller должен продолжить работу со своим кадром.

Unwind восстанавливает RSP не потому, что где-то сохранён "старый RSP" как обычный nonvolatile-регистр. Чаще всего старый RSP вычисляется. RtlVirtualUnwind смотрит на UNWIND_INFO и как бы отменяет эффекты пролога: если был push rbx, надо поднять RSP на 8 после восстановления RBX; если был sub rsp, 80h, надо прибавить 80h; если произошёл выход из функции, надо прочитать return address и перейти к caller context. То есть RSP восстанавливается как результат модели кадра, а не как простая загрузка из одного stack slot.

```
RSP - особый случай:

Внутри callee:
push rbx ; RSP -= 8
sub rsp, 40h ; RSP -= 40h
...

При virtual unwind:
восстановить RBX из [RSP]
RSP += 8
RSP += 40h
RIP = [RSP]
RSP += 8

Смысл: RSP caller-a получается через обратное применение модели кадра.
```

RAX, RCX, RDX, R8, R9: volatile-регистры, которые постоянно видны в вызовах

Если открыть дизассемблер и смотреть обычный Windows x64 код, чаще всего на глаз бросаются RCX, RDX, R8 и R9. Это первые четыре integer/pointer argument registers. Argument register (регистр аргумента) - это регистр, через который calling convention передаёт параметр функции. Например, если функция принимает указатель this, длину буфера и флаг, то первые значения часто окажутся в RCX, RDX и R8.

Эти регистры volatile именно потому, что они являются входными каналами в callee. Callee получает аргумент в RCX, но не обязана хранить RCX как священное значение до конца. Она может скопировать аргумент в RBX, если он нужен долго. Может положить его на stack. Может использовать RCX для следующего call. Может полностью перезаписать. Caller не должен после возврата ожидать, что в RCX всё ещё тот же аргумент.

RAX тоже volatile, но его особая видимая роль - return value. Return value (возвращаемое значение) для целочисленных и pointer-результатов обычно приходит в RAX. Поэтому RAX почти по определению не может быть callee-saved: если функция возвращает результат, она должна иметь право записать его в RAX. Это хорошая демонстрация: volatile не значит "случайно портится". Иногда volatile-регистр меняется именно потому, что так устроен нормальный результат вызова.

R10 и R11 менее заметны новичку, потому что они не входят в первые четыре обычных регистровых аргумента Microsoft x64. Но они тоже volatile и часто применяются как scratch registers в сгенерированном коде, helper-последовательностях, thunk-обвязках и low-level переходах. Для reverse engineer-а это означает: если R10/R11 меняются вокруг call или в служебном коде, это само по себе не нарушение ABI.

```
Пример передачи аргументов:

; foo(a, b, c, d)
mov rcx, a ; 1-й integer/pointer argument
mov rdx, b ; 2-й
mov r8, c ; 3-й
mov r9, d ; 4-й
call foo

; После call значения RCX/RDX/R8/R9 не гарантированы.
; Результат, если это integer/pointer, обычно в RAX.
```

RBX, RBP, RSI, RDI, R12-R15: зачем нужны nonvolatile-регистры

Nonvolatile-регистры дают компилятору место для long-lived values. Long-lived value (долго живущее значение) - это значение, которое нужно не на две инструкции, а через большой участок кода, возможно через несколько вызовов других функций. Если держать такое значение только в volatile-регистре, перед каждым call его пришлось бы сохранять и после call восстанавливать. Иногда выгоднее один раз сохранить nonvolatile-регистр в прологе, положить в него важное значение и пользоваться им весь body функции.

Например, функция обрабатывает массив и в цикле вызывает helper. У неё есть указатель на начало массива, текущий индекс, конечный адрес или объект состояния. Если эти значения нужны после каждого helper call, компилятор может положить одно из них в RBX или R12. Тогда каждый отдельный call не уничтожит значение, потому что callee обязаны сохранять nonvolatile-регистры. Цена оплачивается один раз: текущая функция сохраняет старые значения этих регистров в своём прологе и восстанавливает в эпилоге.

Это очень важно для чтения дизассемблера. Когда ты видишь в функции push rbx; push rsi; push rdi; push r14, это не просто "много мусора в начале". Это сигнал: функция собирается использовать эти nonvolatile-регистры как устойчивые рабочие хранилища. Для decompiler-а это помогает реконструировать долгоживущие переменные. Для unwind parser-а это означает наличие нескольких saved nonvolatile locations. Для static /dynamic это источник масок saved_nonvol_mask, проверок offsets и сопоставления metadata с реальным прологом.

```
Типичный смысл сохранения nonvolatile-регистров:

push rbx ; функция хочет использовать RBX
push rsi ; функция хочет использовать RSI
sub rsp, 38h ; local stack + alignment
mov rbx, rcx ; сохранить важный аргумент в RBX на долго
```

```
mov rsi, rdx ; сохранить другой аргумент в RSI
call helper ; helper может испортить volatile-регистры
; RBX/RSI должны сохраниться через helper call
```

Почему unwind metadata в первую очередь заботится о nonvolatile-регистрах

Теперь можно сформулировать центральную причину. Volatile-регистр после вызова не обещан caller-у. Если исключение произошло внутри callee, и система поднимается к caller-у, она не обязана восстанавливать старый RAX или RCX caller-а так, будто callee их не трогала. Caller не имел права ожидать их сохранения через call. Если caller хотел сохранить volatile-значение, он должен был сделать это сам до вызова, например в своём stack frame или nonvolatile-регистре. Тогда это уже часть caller-а, а не обязанность callee.

Nonvolatile-регистр - другое дело. Caller имеет право ожидать сохранения RBX, RBP, RDI, RSI, R12-R15. Если callee изменила RBX, она обязана вернуть старое значение. При исключении callee не успевает выполнить обычный epilog. Но контракт ABI всё равно должен быть выполнен при построении caller context. Поэтому RtlVirtualUnwind должен знать, где старое значение RBX лежит в callee frame.

Отсюда становится ясно, почему UWOP_PUSH_NONVOL называется именно NONVOL. Он не описывает любой push. Он описывает push nonvolatile-регистра как действие пролога, имеющее смысл для восстановления caller context. Если функция по какой-то причине делает push rax, это не может быть UWOP_PUSH_NONVOL, потому что RAX volatile. Такой push может быть частью stack allocation, временного сохранения, выравнивания или ручного кода, но для ABI он не является сохранением callee-saved состояния caller-а.

caller context (контекст вызывающей функции) - это набор регистров и состояния стека, который должен быть восстановлен, когда unwind переходит с текущего кадра на кадр caller-а. В нём важны не все старые значения подряд, а те значения, которые ABI обещает сохранять через вызов, плюс RIP/RSP и связанные элементы управления.
callee-saved (сохраняемый вызываемой функцией) - другое название nonvolatile-регистра. Оно подчёркивает не физическое свойство регистра, а сторону ответственности: если callee его меняет, callee сохраняет и восстанавливает.
caller-saved (сохраняемый вызывающей функцией) - другое название volatile-регистра. Оно подчёркивает, что caller сам должен сохранить значение, если оно понадобится после вызова.

Связь с UNWIND_CODE: какие операции относятся к nonvolatile-регистрам

Предыдущие разделы уже разобрали UNWIND_CODE и UWOP_* операции. Теперь эти операции можно увидеть через призму volatile/nonvolatile. UWOP_PUSH_NONVOL описывает сохранение nonvolatile integer register через push. UWOP_SAVE_NONVOL и UWOP_SAVE_NONVOL_FAR описывают сохранение nonvolatile integer register через store в stack slot. UWOP_SAVE_XMM128 и UWOP_SAVE_XMM128_FAR описывают сохранение nonvolatile XMM register. Эти операции не случайно ограничены определёнными регистрами: unwind metadata не должна объявлять callee-saved восстановление для регистра, который ABI не требует сохранять.

Если parser видит UWOP_PUSH_NONVOL с OpInfo, соответствующим RAX или RCX, это не "странный, но допустимый стиль". Это нарушение формата или по крайней мере серьёзная ABI/unwind anomaly, потому что операция по смыслу предназначена для nonvolatile integer registers. Microsoft для MASM-директивы .PUSHREG прямо указывает использовать её с nonvolatile integer registers; для push volatile-регистра нужно описывать изменение стека иначе, например как allocation, если это вообще часть пролога, требующая описания.

Для reverse engineer-а это важная граница. Машинный код может содержать push rax. Но unwind metadata не обязана и не должна говорить "PUSH_NONVOL RAX". Если видишь такое в metadata, не надо радостно расширять список nonvolatile-регистров. Надо подозревать неправильную генерацию, повреждение, обфускацию, ручной ассемблер или ошибку parser-а.

UWOP	Что сохраняет	Какие регистры имеют смысл	Что проверять
UWOP_PUSH_NONVOL	Nonvolatile integer register через push	RBX, RBP, RSI, RDI, R12-R15	OpInfo должен указывать nonvolatile GPR, не RAX/RCX/RDX/R8/R9/R10/R11.
UWOP_SAVE_NONVOL	Nonvolatile integer register в stack slot	RBX, RBP, RSI, RDI, R12-R15	Offset должен попадать в допустимую область кадра, операция должна согласоваться с прологом.
UWOP_SAVE_NONVOL_FAR	То же, но с большим offset	RBX, RBP, RSI, RDI, R12-R15	Не путать far payload с двумя отдельными unwind-операциями.
UWOP_SAVE_XMM128	Nonvolatile XMM state в 16-байтный slot	XMM6-XMM15 в обычной x64 модели	Offset масштабируется на 16, stack slot должен быть разумно выровнен.
UWOP_SAVE_XMM128_FAR	Nonvolatile XMM с большим offset	XMM6-XMM15	Far offset читается как 32-битное значение payload-слотов.

Почему push volatile-регистра не превращает его в nonvolatile

Есть тонкая ловушка. Допустим, в прологе встретился push rax. Можно ли сказать, что RAX стал nonvolatile? Нет. Volatile/nonvolatile - это правило ABI, а не результат инструкции push. Инструкция push rax всего лишь уменьшает RSP и записывает текущее значение RAX на стек. Это может быть временное сохранение, выравнивание, резервирование 8 байт, ручной трюк или часть нестандартного кода. Но она не создаёт у caller-а право ожидать, что RAX сохранится через вызов.

Если callee сама решила временно сохранить RAX, она может это сделать. Но это её внутренняя логика. Внешний контракт с caller-ом от этого не меняется. Caller всё равно не имеет права ожидать старый RAX после call, потому что RAX volatile. Поэтому unwind metadata, предназначенная для восстановления caller context по ABI, не должна относиться к RAX как к nonvolatile-регистру.

Иногда это особенно важно при анализе обфусцированного кода. dynamic может использовать push/pop volatile-регистров как шум: создавать видимость сложного пролога, двигать RSP, мешать линейному сопоставлению unwind-кодов с инструкциями. Но parser должен отделять "изменение стека" от "сохранения callee-saved register". Это разные смыслы даже тогда, когда обе вещи физически выглядят как push.

```
Пример ловушки:

push rax ; физически значение RAX записано на стек
sub rsp, 20h
...
add rsp, 20h
pop rax
ret

Это НЕ делает RAX nonvolatile по ABI.
Корректная metadata не должна описывать это как UWOP_PUSH_NONVOL RAX.
Если stack effect надо учитывать, он учитывается как часть изменения RSP,
а не как восстановление callee-saved состояния caller-a.
```

XMM-регистры: почему volatile/nonvolatile касается не только RAX и RBX

До сих пор мы говорили в основном про general-purpose registers, потому что они чаще видны в обычном коде и прямо участвуют в адресах, указателях, аргументах и RSP/RIP-логике. Но Windows x64 ABI делит также SIMD/floating-point registers. XMM register (XMM-регистр, 128-битный векторный регистр) используется для floating-point значений, SIMD-операций, векторных данных и некоторых оптимизаций компилятора.

В обычной Microsoft x64 calling convention XMM0-XMM5 считаются volatile, а XMM6-XMM15 - nonvolatile. Это значит, что если функция хочет использовать XMM6, она должна сохранить и восстановить его 128-битное состояние. Для этого в unwind model существуют UWOP_SAVE_XMM128 и UWOP_SAVE_XMM128_FAR. Эти операции отличаются от SAVE_NONVOL для GPR тем, что сохраняемый кусок имеет размер 16 байт, а offset масштабируется по 16, не по 8.

Здесь важно не смешивать уровни. XMM6-XMM15 nonvolatile в смысле сохранения 128-битного XMM-состояния в обычной Windows x64 модели. Более широкие YMM/ZMM-регистры, AVX-состояние, верхние половины и новые расширения вроде APX/Unwind V3 - это отдельные детали ABI и OS context management. В этой главе мы держим основной фокус на классической unwind-информации Windows x64, где .xdata описывает сохранение nonvolatile XMM через UWOP_SAVE_XMM128. Когда будем говорить о новых версиях unwind или расширенных состояниях, это надо будет делать отдельно, чтобы не смешать базовую модель с расширениями.

XMM register (XMM-регистр, 128-битный SIMD/floating-point регистр) - это регистр, используемый для операций с floating-point и векторными данными. В Windows x64 часть XMM-регистров volatile, а XMM6-XMM15 являются nonvolatile в обычной модели и при сохранении описываются через UWOP_SAVE_XMM128.

SIMD (single instruction, multiple data; одна инструкция, много данных) - это стиль вычислений, где одна машинная инструкция обрабатывает сразу несколько элементов данных внутри векторного регистра. Для unwind важна не математика SIMD сама по себе, а факт, что некоторые XMM-регистры тоже входят в ABI preservation contract.

```
Сохранение nonvolatile XMM-регистра:

sub rsp, 60h
movaps [rsp+40h], xmm6
...
movaps xmm6, [rsp+40h]
add rsp, 60h
ret

UNWIND_INFO может содержать:
UWOP_SAVE_XMM128 XMM6, offset=4

Потому что реальный offset = 4 * 16 = 40h.
```

MXCSR и FPCSR: почему иногда volatile/nonvolatile делят даже части регистра

В официальной x64 calling convention есть ещё более тонкая идея: некоторые register state components могут быть частично volatile и частично nonvolatile. Например, MXCSR (SSE control/status register, регистр управления и состояния SSE) содержит флаги состояния и управляющие биты. В документации Microsoft volatile и nonvolatile части MXCSR разделены: статусные флаги считаются volatile, а управляющая часть должна иметь устойчивые значения. Похожая логика применяется к floating-point control state.

Для нашей основной unwind-линии это не центральная тема, потому что UNWIND_INFO в .xdata не хранит привычные UWOP-коды для каждого бита MXCSR. Но этот пример полезен концептуально. Он показывает, что volatile/nonvolatile - это не обязательно "весь регистр целиком простой". Это правило сохранения состояния, и оно может применяться на разной гранулярности: к GPR, к XMM-регистрам, к частям control/status state.

Для static /dynamic на базовом этапе не нужно превращать этот раздел в анализ MXCSR. Но важно не делать грубых утверждений вроде "volatile/nonvolatile относится только к RAX/RBX". Нет. Это общая ABI-категория сохранения состояния через вызов. Просто x64 unwind metadata наиболее явно и практически связано с nonvolatile GPR и XMM-регистрами, потому что именно их сохранение закодировано в UNWIND_CODE.

MXCSR (SSE control/status register, регистр управления и состояния SSE) - это специальный регистр, связанный с SSE floating-point/SIMD операциями. Он показывает, что ABI может задавать правила сохранения не только для обычных GPR, но и для частей вычислительного состояния процессора.

Что происходит при обычном возврате и что происходит при unwind

Теперь полезно сравнить два пути: normal return и exception unwind. Normal return (обычный возврат) - это ситуация, когда функция сама дошла до epilog, выполнила восстановление регистров, освободила stack allocation и сделала ret. В этом случае машинный код callee сам исполняет contract. Если RBX был сохранён через push rbx, то epilog выполнит pop rbx. Если XMM6 был сохранён в [rsp+40h], epilog загрузит его обратно. Caller получает управление уже с восстановленными nonvolatile-регистрами.

Exception unwind (раскрутка стека при исключении) - другой путь. Функция может быть прервана где угодно: после сохранения RBX, после выделения стека, после установки frame pointer, в середине тела. Она не исполнила epilog. Но Windows должна как бы

виртуально совершить выход из функции. Поэтому RtlVirtualUnwind читает UNWIND_INFO и восстанавливает caller context без выполнения инструкций epilog-a. Он не запускает pop rbx как машинную инструкцию из программы. Он сам вычисляет, где лежит сохранённый RBX, читает значение из памяти и записывает его в контекст.

Вот почему unwind metadata должна быть точной. Если metadata говорит, что RBX сохранён в одном месте, а реально он лежит в другом, обычный возврат может всё равно работать, потому что epilog использует настоящий код. Но exception unwind, stack trace или crash dump analysis могут получить ложный caller context. Это один из ключевых случаев, где программа "вроде работает", но её metadata опасно некорректна.

```
Normal return:
CPU исполняет инструкции epilog-a:
mov rbx, [rsp+30h]
add rsp, 50h
ret

Virtual unwind:
RtlVirtualUnwind НЕ исполняет epilog как код программы.
Она читает UNWIND_INFO и сама строит caller context:
RBX = memory[calculated_address]
RSP = RSP + calculated_frame_size
RIP = return_address
```

Почему nonvolatile-регистр может быть важнее локальной переменной

В исходном коде программист думает о переменных: int count, char *buffer, Object *this. Но на машинном уровне компилятор решает, где эти значения жить будут: в памяти, в volatile-регистре, в nonvolatile-регистре, временно на стеке, иногда вообще исчезнут из-за оптимизации. Когда значение живёт в nonvolatile-регистре, оно становится частью ABI-visible state - состояния, которое должно пережить вызовы.

Представь, что caller держит указатель на объект в RBX. Затем вызывается функция. Если эта функция неправильно восстановит RBX, caller после возврата начнёт работать с неправильным объектом. Это не просто "потеря локальной переменной". Это нарушение межфункционального контракта, которое может привести к записи не туда, use-after-free, чтению чужой памяти или падению далеко от реальной причины.

При exception unwind это ещё тоньше. Если обработчик исключения выше по стеку получает управление с неправильным RBX, он может считать, что видит корректное состояние caller-a, хотя контекст уже повреждён. Отладчик покажет странные локальные переменные. Crash dump analyzer построит неверный след. Reverse engineer может обвинить не ту функцию. Поэтому сохранение nonvolatile-регистров - это не декоративная аккуратность компилятора, а часть достоверности анализа всего стека.

Как компилятор выбирает, куда положить значение

Compiler register allocator (распределитель регистров компилятора) решает, какие значения держать в каких регистрах. Это сложная оптимизационная задача, но для нашего понимания достаточно главного принципа. Если значение короткоживущее и не нужно после call, его удобно держать в volatile-регистре. Если значение должно пережить call, его можно держать в nonvolatile-регистре, либо сохранять вокруг каждого call

вручную, либо держать в памяти.

Например, значение нужно только для вычисления адреса перед ближайшей инструкцией. Тогда RAX/R10/R11 отлично подходят: быстро, свободно, без обязательств. А если значение нужно на протяжении всей функции и функция много раз вызывает helpers, nonvolatile-регистр может быть выгоднее. Один раз сохранить RBX в прологе, потом держать там base pointer на структуру, затем восстановить в эпилоге - часто дешевле, чем перед каждым call сохранять и восстанавливать volatile-регистр.

Эта оптимизация влияет на вид пролога. Чем больше nonvolatile-регистров компилятор решил использовать, тем больше сохранений появится в начале функции и тем больше unwind-кодов будет в UNWIND_INFO. Поэтому по набору saved nonvolatile registers можно иногда судить о характере функции: leaf helper может не сохранять ничего, а большая функция с несколькими long-lived pointers может сохранять RBX, RSI, RDI, R14, R15 и XMM6.

register allocator (распределитель регистров) - часть компилятора, которая решает, какие значения программы хранить в каких регистрах, когда переносить их в память и какие регистры использовать для долгоживущих или временных значений. Его решения напрямую влияют на пролог и unwind metadata.

Leaf function и volatile-регистры

Leaf function (листовая функция, функция без вызовов других функций) часто может обходиться только volatile-регистрами. Если функция не вызывает никого дальше, ей не нужно сохранять свои временные значения через чужие call. Она может использовать RAX, RCX, RDX, R8-R11 как рабочие регистры, посчитать результат и вернуть его. Если она не трогает nonvolatile-регистры и не выделяет stack frame, полноценная unwind metadata может быть не нужна: unwinder может симулировать обычный return.

Но "leaf" не означает автоматически "нет стека" и "нет сложности" во всех возможных смыслах. Функция может быть leaf по вызовам, но всё равно использовать stack для локальных данных или выравнивания, если компилятор так решил. А может быть non-leaf, но сохранять мало nonvolatile-регистров. Эти детали мы подробнее будем разбирать позже. Сейчас важно другое: volatile-регистры позволяют писать маленькие функции без обязательного сохранения состояния callee. Это одна из причин, почему ABI не делает все регистры nonvolatile.

Для reverse engineer-а leaf function часто выглядит "голой": нет push rbx, нет sub rsp, нет .pdata entry или нет содержательного UNWIND_INFO. Но это не ошибка. Если функция использует только volatile state и не создаёт кадр, ей действительно нечего описывать для восстановления callee-saved состояния. Если же такая функция внезапно имеет сложную .xdata с сохранением volatile-регистров как nonvolatile, это уже повод внимательно смотреть на metadata.

Non-leaf function и цена сохранения состояния

Non-leaf function (нелистовая функция, функция, которая вызывает другие функции) чаще сталкивается с проблемой сохранения значений через calls. Когда функция вызывает helper, helper может испортить volatile-регистры. Если текущей функции после helper всё ещё нужны какие-то значения, она должна заранее решить, где они будут переживать вызов.

Один путь - spill to stack. Spill (сброс регистра в память) - это когда значение, которое хотелось бы держать в регистре, временно записывается в stack slot. Другой путь - держать значение в nonvolatile-регистре. Но если текущая функция использует nonvolatile-регистр, она сама становится callee по отношению к своему caller-у и должна сохранить старое значение этого регистра. Поэтому non-leaf функции часто имеют пролог с сохранениями.

Эта вложенность хорошо показывает, что caller/callee - относительные роли. Одна и та же функция одновременно callee для того, кто её вызвал, и caller для тех функций, которые вызывает она сама. Как callee она обязана сохранить nonvolatile-регистры своего caller-а. Как caller она должна не рассчитывать на volatile-регистры после собственных calls. Поэтому компилятор постоянно балансирует: что положить в RBX, что оставить в RAX, что spill-нуть, что пересчитать позже.

spill (сброс регистра в память, временное сохранение значения из регистра в stack slot) - это действие компилятора или ручного ассемблера, когда значение переносится из регистра в память, потому что регистр нужен для другого значения или значение должно пережить участок кода. Spill volatile-регистра не делает его nonvolatile; это внутренняя стратегия хранения.

Как это видно в дизассемблере

В дизассемблере volatile/nonvolatile проявляется не цветом регистра, а поведением вокруг call, prolog и epilog. Volatile-регистры часто активно меняются перед вызовами: в RCX/RDX/R8/R9 кладутся аргументы, RAX используется для результата, R10/R11 - для временных адресов. Если значение в volatile-регистре нужно после call, ты увидишь его сохранение до call или повторное вычисление после call.

Nonvolatile-регистры обычно видны в прологе и эпилоге. Если функция использует RBX, RSI, RDI, R12-R15, часто в начале будет push или store, а в конце - pop или load. В теле функции эти регистры могут выглядеть как "локальные переменные в регистрах": один держит this, другой base pointer массива, третий счётчик, четвёртый указатель на конец.

Но не надо ожидать идеальной симметрии в одном линейном участке. Оптимизированный код может иметь несколько epilog paths, общий cleanup block, shrink-wrapping, сохранение регистра не в самом первом байте функции, chained unwind info и другие усложнения. Поэтому ручной анализ должен быть осторожным: сначала понять ABI-класс регистра, потом сопоставить реальные инструкции с UNWIND_CODE, потом учесть, где находится RIP - в prolog, body или epilog.

```
Сигналы в дизассемблере:

Volatile use:
mov rcx, rbx ; подготовка 1-го аргумента
mov rdx, r14 ; подготовка 2-го аргумента
call helper
test rax, rax ; RAX = return value

Nonvolatile use:
push rbx
push r14
sub rsp, 28h
mov rbx, rcx ; долгоживущий this/base pointer
mov r14, rdx ; долгоживущий указатель/размер
```

```
...
add rsp, 28h
pop r14
pop rbx
ret
```

Как это видно в UNWIND_INFO

В UNWIND_INFO volatile/nonvolatile видно через выбор UWOP-кодов и OpInfo. Если CountOfCodes содержит UWOP_PUSH_NONVOL RBX, значит metadata заявляет: в прологе был push RBX, и это сохранение callee-saved регистра. Если есть UWOP_SAVE_XMM128 XMM6, значит metadata заявляет сохранение nonvolatile XMM-регистра. Если есть UWOP_ALLOC_SMALL или UWOP_ALLOC_LARGE, это относится к stack allocation, а не напрямую к сохранению конкретного регистра.

Важно различать два слоя: metadata describes obligations, not every instruction. UNWIND_INFO не является полной трассой всех инструкций пролога. Он не обязан описывать каждую mov, xor, lea, подготовку аргументов или временный push volatile-регистра в исходном смысле. Он описывает те действия, которые нужны для восстановления caller context: изменение RSP, сохранение nonvolatile-регистров, установку frame pointer и особые случаи вроде machine frame.

Поэтому хороший parser не должен требовать, чтобы каждый push в прологе был UWOP_PUSH_NONVOL. Он должен спросить точнее: этот push относится к сохранению nonvolatile-регистра? Если да, metadata должна это описать. Если push относится к allocation или временной внутренней логике, смысл другой. Но если metadata сама заявляет PUSH_NONVOL для volatile-регистра, это уже нарушение semantic contract формата.

Ключ для static /dynamic: UNWIND_INFO описывает не "всё, что происходило", а всё, что нужно для корректной виртуальной раскрутки кадра. Поэтому volatile-регистры могут встречаться в машинном коде, но не становятся объектами UWOP_PUSH_NONVOL/SAVE_NONVOL.

Context record: куда в итоге восстанавливаются регистры

Когда RtlVirtualUnwind работает, она оперирует не абстрактными переменными C, а CONTEXT-like state. CONTEXT record (запись контекста, структура состояния потока) - это структура Windows, в которой хранятся значения регистров потока: RIP, RSP, RAX, RBX, RCX, XMM-регистры и другие части состояния. Во время unwind эта структура постепенно превращается из контекста текущей функции в контекст caller-a.

Если текущий кадр сохранил RBX, RtlVirtualUnwind находит соответствующий stack slot, читает старое значение и записывает его в поле RBX контекста. Если функция выделила stack allocation, RSP в контексте корректируется. Если найден return address, RIP в контексте становится адресом продолжения caller-a. Так шаг за шагом debugger, exception dispatcher или crash dump analyzer поднимается вверх по стеку.

Именно поэтому ошибка в nonvolatile unwind metadata опасна: она не просто неверно подписывает байты. Она приводит к неверному CONTEXT. А неверный CONTEXT - это неверный stack trace, неверная локализация exception handler, неверная картина crash dump и потенциально неправильное поведение exception dispatch. В теоретической книге

это надо проговорить явно: metadata влияет не на красоту анализа, а на восстановление машинного состояния.

CONTEXT record (запись контекста, структура состояния регистров потока) - это структура Windows, которая хранит значения регистров для потока. Во время unwind RtlVirtualUnwind изменяет такой контекст, превращая состояние текущего кадра в состояние caller-а.

Почему PDB не заменяет ABI и unwind metadata

PDB (Program Database, база отладочной информации) может дать имена функций, исходные строки, имена локальных переменных и дополнительные сведения для отладчика. Но PDB не является основанием для работы Windows x64 unwind. Исключения, stack walk в рантайме и системная раскрутка не должны зависеть от наличия PDB. На машине пользователя PDB может отсутствовать. В релизной сборке его может не быть рядом. В malware или packed binary его почти наверняка нет.

Volatile/nonvolatile правила живут в ABI, а конкретные сохранения живут в UNWIND_INFO. PDB может помочь человеку понять, что RBX соответствует переменной pObject, а R14 - vector_end. Но когда RtlVirtualUnwind восстанавливает caller context, она не спрашивает PDB: "как называлась эта переменная?" Она читает .pdata, .xdata и machine context.

Это важно для твоего проекта. static /dynamic не должен строить core correctness только на symbol information. Symbols полезны для отчёта, удобства и объяснений, но фундаментальная проверка unwind должна идти от PE Exception Directory, RUNTIME_FUNCTION, UNWIND_INFO, UWOP-кодов, ABI-классов регистров и реального машинного кода. Тогда инструмент будет работать и на stripped binaries.

PDB (Program Database, база отладочной информации Microsoft) - файл с символами и отладочными данными. Он помогает человеку и debugger-у, но не является обязательной основой x64 unwind. Unwind должен работать по PE/.pdata/.xdata даже без PDB.

stripped binary (бинарь без символов, исполняемый файл без полезной отладочной информации) - файл, в котором нет PDB или символы недоступны. Для reverse engineering это обычная ситуация, поэтому анализ ABI/unwind нельзя делать зависимым от имён функций и переменных.

Типичная ошибка 1: считать volatile-регистры "мусорными"

Первая ошибка - говорить: "RAX, RCX, RDX - мусорные регистры". Это плохая формулировка. Volatile-регистр может содержать абсолютно важное значение в текущий момент. RCX может держать this pointer, RDX - указатель на буфер, RAX - результат функции, R10 - адрес таблицы. Просто это значение не гарантируется через call, если caller сам не сохранил его.

Для анализа это различие критично. Если ты находишь значение в RCX перед call, его надо анализировать как реальный аргумент или временный указатель. Но если после call в RCX другое значение, не нужно автоматически считать это ошибкой. Volatile означает "не сохраняется через границу вызова", а не "неважен".

В хорошей книге эту тонкость стоит держать постоянно. Volatile-регистры важны для текущих вычислений. Nonvolatile-регистры важны для межфункционального сохранения состояния. Это разные масштабы времени жизни значения.

Типичная ошибка 2: считать любое сохранение регистра обязанностью ABI

Вторая ошибка - видеть push или mov [rsp+offset], register и автоматически считать, что это ABI-сохранение. Но функция может сохранять volatile-регистр для своих внутренних причин. Например, caller может сохранять RCX перед call, потому что значение понадобится после. Это caller-saved сохранение volatile-регистра. Оно может находиться в body caller-a, а не быть частью callee prolog. Оно не превращает RCX в nonvolatile и не описывается как UWOP_PUSH_NONVOL RCX.

Другой пример: ручной ассемблер может использовать push rax/pop rax вокруг маленького участка, чтобы временно освободить RAX. Это внутренний stack effect. Для unwind важен вопрос: может ли exception произойти в этом участке, находится ли он в prolog/body, нужно ли описывать изменение RSP для корректного unwinding, соответствует ли это допустимой форме пролога. Но нельзя из самого факта push rax вывести, что RAX стал частью callee-saved contract.

Поэтому analyzer должен разделять mechanical effect и ABI meaning. Mechanical effect (механический эффект) - что инструкция физически сделала с RSP или памятью. ABI meaning (ABI-смысл) - какую обязанность по сохранению состояния она реализует. UNWIND_CODE в первую очередь кодирует именно те mechanical effects, которые нужны для восстановления ABI-relevant caller context.

mechanical effect (механический эффект инструкции) - прямое действие инструкции на регистры и память: push уменьшил RSP и записал значение, sub rsp выделил область, mov записал данные.

ABI meaning (смысл инструкции в рамках ABI) - роль этого действия в соглашении о вызовах: сохранение callee-saved регистра, подготовка аргумента, восстановление stack pointer, возврат значения. Одна и та же машинная форма может иметь разный ABI-смысл в разном контексте.

Типичная ошибка 3: думать, что absence of save значит absence of use

Третья ошибка тоньше. Если в UNWIND_INFO нет сохранения RBX, можно ли сказать, что функция точно не использует RBX? Не всегда так просто. В корректно сгенерированном обычном коде, если функция изменяет RBX, она должна сохранить и восстановить его. Но она может читать RBX без изменения. Она может использовать RBX только в недостижимом коде. Она может иметь нестандартный hand-written assembly. Она может быть частью обфусцированного или повреждённого бинаря. A parser metadata сам по себе не дизассемблирует весь body.

Поэтому из metadata безопасно выводить: "metadata не заявляет сохранение RBX". A утверждение "функция не трогает RBX" требует анализа машинного кода. Даже если машинный код показывает запись в RBX, дальше нужно понять, восстановлен ли он, где, соответствует ли это UNWIND_INFO и может ли exception попасть в участок между изменением и восстановлением.

Для отчёта важно не переобещать. Хорошая формулировка finding-a: "UNWIND_INFO не содержит сохранения RBX, однако дизассемблированный пролог/тело показывает запись

в RBX до потенциального unwind-safe восстановления". Плохая формулировка: "RBX не используется", если ты видел только metadata. Разница между ними - разница между строгим анализом и гаданием.

Типичная ошибка 4: не учитывать RBP как одновременно nonvolatile и frame pointer

RBP может быть просто nonvolatile-регистром, а может быть frame pointer. Эти роли могут пересекаться, но не идентичны. Если функция использует RBP как frame pointer, она обычно сначала сохраняет старый RBP, потому что RBP nonvolatile, потом устанавливает новое значение RBP/FP. В metadata это может выглядеть как UWOP_PUSH_NONVOL RBP плюс UWOP_SET_FPREG. Но UWOP_PUSH_NONVOL RBP и UWOP_SET_FPREG - разные сообщения.

UWOP_PUSH_NONVOL RBP говорит: старое значение RBP сохранено на стеке и должно быть восстановлено для caller-a. UWOP_SET_FPREG говорит: в этой функции установлен frame pointer, указанный в FrameRegister. Если FrameRegister=RBP, то да, RBP стал базой кадра. Но сам push rbp ещё не доказывает использование RBP как frame pointer. В оптимизированном коде RBP может быть просто обычным nonvolatile-регистром для хранения значения.

Эта ошибка часто ломает ручной разбор. Человек видит push rbp и автоматически рисует классический кадр: saved rbp, old rbp chain, locals below rbp. На Windows x64 это может быть неверно. Надо читать UNWIND_INFO: есть ли FrameRegister? есть ли UWOP_SET_FPREG? какой FrameOffset? совпадает ли это с реальным lea/mov в прологе?

Volatile/nonvolatile и эпилог

Эпилог функции восстанавливает то, что пролог сохранил, и возвращает стек к состоянию, в котором ret сможет взять правильный return address. Если функция сохранила RBX, эпилог должен вернуть RBX. Если сохранила R14, вернуть R14. Если выделила stack allocation, освободить его. Если установила frame pointer, выйти из этой модели корректно. Поэтому volatile/nonvolatile деление видно и в эпилоге.

Но, как мы уже разбирали, epilog на Windows x64 не описывается отдельным списком unwind-кодов для каждого выхода. Вместо этого epilog должен иметь ограниченную форму, чтобы RtlVirtualUnwind мог распознать его сканированием инструкций. Это сделано именно потому, что nonvolatile-регистры и RSP должны быть правильно восстановлены даже если RIP находится внутри выхода из функции.

Если RIP находится в эпилоге после pop rbx, то RBX уже восстановлен настоящей инструкцией. Если RIP находится до pop rbx, RBX ещё лежит на стеке. RtlVirtualUnwind должна понимать, в какой фазе выхода мы находимся. Поэтому строгая форма эпилога, ABI-класс регистров и UNWIND_INFO связаны между собой. Это не три отдельные темы, а одна система.

Volatile/nonvolatile и exception handler

Когда exception dispatcher ищет обработчик, он проходит кадры. На каждом кадре ему нужен корректный контекст. Если функция имеет language-specific handler, например __C_specific_handler, система должна передать ему данные в состоянии, которое

соответствует правилам Windows x64. Handler может смотреть exception record, establisher frame, dispatcher context и другую информацию. Но фундамент всё равно тот же: stack walk должен понимать границы функций и сохраняемое состояние.

Nonvolatile-регистры важны потому, что обработчик выше по стеку может ожидать состояние своей функции. Если при подъёме через промежуточный кадр неверно восстановить RBX или RSP, дальше search phase или unwind phase может пойти по ложной картине. Это может проявиться как неправильный стек, неправильный handler, падение при обработке исключения или странный crash dump.

Поэтому корректность volatile/nonvolatile metadata - это не только "чтобы отладчик красиво показывал регистры". Это часть механизма обработки исключений. Особенно в x64, где frame-based SEH уже не строится на linked list в стеке, как в старой x86-модели, а опирается на табличные unwind-данные.

Как это переносится на reverse engineering

Для reverse engineer-а volatile/nonvolatile деление даёт несколько практических преимуществ. Первое - понимание, какие значения могут переживать call. Если в RBX положили this pointer и потом внутри функции много вызовов, скорее всего RBX остаётся важной базой. Если значение лежало только в RCX перед call и не было сохранено, после call оно не гарантировано.

Второе - чтение пролога. Сохранённые nonvolatile-регистры показывают, какие устойчивые регистровые переменные функция собирается использовать. Это помогает при ручной реконструкции логики: RBX может быть объектом, RSI - указателем на массив, RDI - текущим элементом, R14 - end pointer, R15 - контекстом. Не всегда так, но pattern полезный.

Третье - проверка metadata. Если .xdata заявляет сохранение R14, а в прологе нет соответствующего push/store, это подозрительно. Если пролог сохраняет RBX, а UNWIND_INFO молчит, это тоже подозрительно. Если UWOP говорит сохранить volatile-регистр как nonvolatile, это уже сильный сигнал ошибки или искусственной аномалии. Такие вещи особенно важны при анализе packed/protected binaries, где metadata может быть намеренно искажена, чтобы ломать stack walking и инструменты.

packed/protected binary (упакованный или защищённый бинарный файл) - исполняемый файл, прошедший через packer/dynamic, который может менять структуру кода, добавлять обфускацию, скрывать настоящие функции, создавать trampolines и иногда искажать metadata, чтобы усложнить анализ.

Что именно должен проверять static /dynamic

Для проекта static /dynamic volatile/nonvolatile - это не просто текст в справке. Это набор проверок и интерпретаций. Минимальный слой - правильно классифицировать register numbers в UWOP-кодах. Если операция предназначена для nonvolatile GPR, OpInfo должен соответствовать RBX/RBP/RSI/RDI/R12-R15. Если операция сохраняет XMM128, номер должен быть в диапазоне nonvolatile XMM-регистров. Если FrameRegister ненулевой, он должен указывать допустимый nonvolatile-регистр, который может быть frame pointer.

Второй слой - строить masks. saved_nonvol_mask показывает, какие nonvolatile GPR metadata заявляет сохранёнными. saved_xmm_mask показывает сохранённые nonvolatile XMM-регистры. Эти маски полезны не только для вывода. Они помогают искать дубли, конфликтующие сохранения, странные сочетания, несоответствие chained unwind info, необычно тяжёлые прологи и функции, где metadata выглядит синтетически.

Третий слой - сопоставление с машинным кодом. Parser .xdata сам по себе может сказать, что metadata формально содержит UWOP_PUSH_NONVOL RBX. Но static как reverse engineering tool может дополнительно сверить: действительно ли в prolog по CodeOffset есть push rbx? Действительно ли offset SAVE_NONVOL соответствует store? Нет ли volatile register в операции nonvol? Нет ли сохранения, которое указывает за пределы разумного stack frame? Нет ли эпилога, который восстанавливает другой набор регистров?

Четвёртый слой - аккуратность формулировок. Если есть только metadata, finding должен говорить о metadata. Если есть metadata плюс disassembly evidence, finding может говорить о mismatch. Если анализ не доказывает достижимость кода, нельзя заявлять runtime corruption как факт. В фундаментальной BKP это важно: инструмент должен быть строгим, а не драматичным.

Проверка	Что значит	Пример finding-а
UWOP register class	UWOP_PUSH_NONVOL/SAVE_NONVOL должен ссылаться на nonvolatile GPR	UWOP_PUSH_NONVOL uses volatile register RAX - invalid register class for nonvolatile save.
XMM register class	SAVE_XMM128 должен ссылаться на nonvolatile XMM state	UWOP_SAVE_XMM128 uses XMM2, which is volatile in Microsoft x64 ABI.
FrameRegister class	FrameRegister должен быть допустимым nonvolatile FP register	FrameRegister points to a volatile register; frame pointer declaration is ABI-inconsistent.
Metadata/prolog match	Unwind save должен соответствовать реальной инструкции пролога	UNWIND_CODE declares RBX save, but prolog bytes do not contain matching push/store at CodeOffset.
Epilog restore match	Эпилог должен восстанавливать сохранённые nonvolatile-регистры	Recognized epilog restores R14/R15 but metadata declares RBX/R15.

saved_nonvol_mask и saved_xmm_mask как смысловые поля, а не украшение

Когда в проекте появляется поле saved_nonvol_mask, его нельзя воспринимать как декоративную битовую маску для красивого JSON. Это компактное выражение того, какие callee-saved obligations заявлены конкретной функцией. Если установлен бит RBX, metadata говорит: старое значение RBX нужно восстановить при unwind. Если установлен бит R14, то R14 тоже часть caller context, которую функция сохраняла.

saved_xmm_mask работает аналогично для XMM6-XMM15. Он важен реже в простых примерах, но в реальном оптимизированном коде с floating-point, SIMD, криптографией,

обработкой массивов, графикой или ML-инференсом такие сохранения встречаются. Если инструмент игнорирует XMM nonvolatile registers, он даёт неполную картину unwind.

Такие маски полезны для статистики. Например, можно увидеть функции с необычно большим числом сохранённых nonvolatile-регистров. Это может быть обычная большая функция, а может быть dispatcher, thunk, compiler-generated state machine или dynamic stub. Можно искать функции, где saved_nonvol_mask не согласуется с размером пролога. Можно строить отчёт: "функция сохраняет RBX, RSI, RDI, R12, R13, R14, R15 и XMM6-XMM9" - это сразу говорит reverse engineer-у, что кадр тяжёлый и состояние значительное.

Граница между строгой ошибкой и подозрительной странностью

В анализаторе важно различать malformed, suspicious и informational. Malformed (некорректное, нарушающее формат) - это когда metadata противоречит формальным правилам: например, UWOP_PUSH_NONVOL указывает volatile register class, count slots выходит за границы массива, payload не помещается. Suspicious (подозрительное) - это когда формат читается, но сочетание выглядит необычно: много сохранений, странное смещение, нет ожидаемого совпадения с типичным prolog, overlapping ranges, chained info в неожиданном месте. Informational (информационное наблюдение) - это просто полезный факт: функция сохраняет такие-то nonvolatile-регистры.

Volatile/nonvolatile classification чаще всего даёт строгие проверки register class. Но сопоставление с disassembly может давать разные уровни уверенности. Например, если байты пролога недоступны из-за повреждённой секции, нельзя уверенно сказать mismatch. Если код self-modifying или packed, static disassembly может быть неполным. Если функция попала в dynamic function table, PE .pdata может не дать всей картины. Поэтому выводы должны быть точными по источнику evidence.

Это качество отличает серьёзный инструмент от набора эвристик. Серьёзный инструмент говорит: "по данным UNWIND_INFO операция X ссылается на register class, запрещённый для этого UWOP". Или: "при линейном сопоставлении с bytes .text не найдено ожидаемое сохранение; возможны hand-written prolog, obfuscation, parse error или unsupported pattern". Такая формулировка не врёт и помогает человеку.

malformed (некорректный по формату) - состояние, при котором данные нарушают структурные правила формата и не могут быть корректно интерпретированы обычным parser-ом.
suspicious (подозрительный) - состояние, которое формально может читаться, но выглядит необычно или рискованно и требует внимания reverse engineer-а.
informational finding (информационное наблюдение) - сообщение анализа, которое не утверждает ошибку, а фиксирует полезный факт о функции, например набор сохранённых nonvolatile-регистров.

Почему это важно для crash dump analysis

Crash dump analysis (анализ аварийного дампа) часто происходит без живого процесса. Есть snapshot памяти, регистров и модулей на момент падения. Отладчик пытается построить stack trace. Для этого он использует unwind metadata. Если nonvolatile-регистры восстановлены неверно, стек может выглядеть правдоподобно, но быть ложным.

Допустим, crash произошёл внутри функции, которая сохранила RBX как указатель на объект. Обработчик выше по стеку или отладчик пытается показать локальные переменные caller-а. Если RBX не восстановлен из правильного места, локальная переменная может отображаться как мусорный pointer. Человек начинает искать проблему в объекте, которого на самом деле не было. А корень проблемы - повреждённая или неверно интерпретированная unwind metadata.

Поэтому для ВКР можно формулировать сильно: качество анализа .pdata/.xdata влияет на достоверность post-mortem debugging. Post-mortem debugging (посмертная отладка) - это разбор падения после факта, по dump-файлу. В таком режиме нельзя "просто выполнить программу ещё раз" и посмотреть. Надо доверять metadata, либо иметь инструмент, который покажет, где ей доверять опасно.

crash dump (аварийный дамп) - файл со снимком состояния процесса или системы в момент сбоя. Для восстановления call stack и регистров caller-ов анализатор опирается на unwind-информацию модулей.

post-mortem debugging (посмертная отладка, анализ после падения) - отладка не живой программы, а сохранённого состояния после аварии. Ошибки unwind metadata в таком режиме особенно опасны, потому что повторно выполнить путь часто нельзя.

Почему это важно для security и anti-analysis

В обычной программе некорректная unwind metadata чаще всего является ошибкой toolchain, ручного ассемблера или редкого edge case. В защищённом или вредоносном коде искажение metadata может быть частью anti-analysis. Anti-analysis (противодействие анализу) - это техники, которые мешают отладчику, дизассемблеру, трассировке, stack walking или статическому анализу.

Если dynamic создаёт функцию, где .pdata указывает на диапазон, а .xdata содержит странные сохранения nonvolatile-регистров, отладчик может строить неправильный стек. Если metadata формально валидна, но не совпадает с реальным прологом, обычные инструменты могут довериться ей и уйти не туда. Если используются динамические function tables, JIT-like regions или callbacks, статический PE-анализ может видеть только часть картины.

Volatile/nonvolatile checks здесь дают простой, но мощный фильтр. ABI-классы регистров трудно "объяснить" произвольным стилем: UWOP_PUSH_NONVOL RAX противоречит смыслу операции. FrameRegister на volatile-регистр противоречит модели frame pointer. SAVE_XMM128 для volatile XMM выглядит как минимум недопустимо для обычной модели. Такие сигналы не доказывают malware сами по себе, но дают технически строгие findings.

anti-analysis (противодействие анализу) - набор приёмов, которые усложняют работу reverse engineer-а, debugger-а или автоматического инструмента: обфускация, ложные переходы, странная metadata, dynamic code, traps, anti-debug и другие техники.

Маленький ручной пример: почему RBX надо описать, а RAX нет

Разберём короткий пример медленно. Пусть функция в начале делает push rbx и sub rsp, 20h. Потом использует RBX как долгоживущий указатель. В середине происходит access violation. Обычного epilog-а нет. Windows должна подняться к caller-у. Caller ожидает, что RBX такой же, каким был до call. Поэтому RtlVirtualUnwind должна прочитать старый RBX из

stack slot, созданного push rbx.

Теперь другой случай: функция использует RAX как временный счётчик. В середине происходит exception. Нужно ли восстановить старый RAX caller-a? Нет, потому что caller не имел права ожидать сохранения RAX через call. Если caller-у был нужен старый RAX, caller должен был сохранить его до вызова. Следовательно, callee UNWIND_INFO не обязана описывать RAX как сохранённый nonvolatile-регистр.

Вот так одна и та же физическая идея "значение в регистре" разделяется ABI-контрактом. RBX может быть частью caller context, которую callee обязана сохранить. RAX - рабочая зона, за которую caller сам отвечает. Unwind metadata кодирует это различие не словами, а набором UWOP-кодов.

```
Case A: RBX

callee:
push rbx ; old RBX saved for caller
sub rsp, 20h
mov rbx, rcx
mov byte ptr [0], 1 ; exception

Virtual unwind должен:
RBX = [current_rsp] ; восстановить old RBX
RSP = RSP + 8 + 20h
RIP = return_address

Case B: RAX

callee:
mov rax, rcx
add rax, 10h
mov byte ptr [0], 1 ; exception

Virtual unwind не обязан восстанавливать old RAX caller-a,
потому что RAX volatile.
```

Ручной пример с caller-saved volatile значением

Теперь пример, где volatile-регистр всё-таки сохраняется, но caller-ом. Пусть caller держит в RCX важный указатель и собирается вызвать helper, который может затереть RCX. Если caller хочет использовать этот указатель после helper, он может сохранить RCX в своём stack frame или переложить в nonvolatile-регистр перед call.

Если caller сохраняет RCX на своём стеке, это часть кадра caller-a. Когда later unwind поднимается через helper к caller-у, сохранённый RCX не восстанавливается как nonvolatile-регистр helper-a. Он остаётся обычной локальной сохранённой копией в caller frame. Отладчик или decompiler может использовать её для реконструкции переменной, но ABI не делает RCX callee-saved.

Это ещё раз показывает, почему нельзя смотреть только на физическое сохранение. Вопрос всегда: какая сторона сохраняет, для какой цели, в каком кадре и какое правило ABI это реализует?

```
caller:
sub rsp, 40h
mov [rsp+20h], rcx ; caller сам сохраняет volatile RCX
call helper ; helper может clobber RCX
mov rcx, [rsp+20h] ; caller сам восстанавливает своё значение
```

```
add rsp, 40h
ret

helper не обязан сохранять RCX.
Это caller-saved logic, а не callee-saved UNWIND_INFO helper-a.
```

Как не потерять живой смысл при чтении таблиц

Таблицы ABI легко превращаются в мёртвое запоминание. Чтобы этого не произошло, полезно каждый регистр читать через вопрос: кто имеет право ожидать его сохранения после call? Если ответ "caller не имеет права" - это volatile. Если ответ "caller имеет право, а callee должна сохранить при изменении" - это nonvolatile.

RAX: caller не ждёт старого значения, потому что RAX используется для return value. RCX/RDX/R8/R9: caller не ждёт старого значения, потому что это argument registers и рабочие регистры callee. RBX/R12-R15: caller может держать в них долгоживущие значения и ожидать, что callee вернёт их как было. RSP: не просто значение, а механизм стека; он должен быть приведён к корректному состоянию. XMM6-XMM15: SIMD/floating-point часть callee-saved состояния.

Если держать такую живую логику, тогда UNWIND_CODE становится понятнее. UWOP_PUSH_NONVOL - не "код 0". Это запись о выполнении callee-saved promise. UWOP_SAVE_XMM128 - не "странный XMM payload". Это сохранение части vector state, которую caller имеет право получить обратно. UWOP_ALLOC_SMALL - не "просто число". Это часть восстановления RSP, без которого return address и stack frame caller-a будут неверны.

Итоговая связка: ABI -> prolog -> UNWIND_INFO -> RtlVirtualUnwind

Теперь можно собрать весь раздел в одну цепочку. ABI говорит: такие-то регистры volatile, такие-то nonvolatile. Compiler, генерируя функцию, решает использовать некоторые nonvolatile-регистры. Чтобы не нарушить ABI, он добавляет в prolog сохранения старых значений. Assembler/linker создают UNWIND_INFO, где эти сохранения записаны как UNWIND_CODE. Когда происходит exception или stack walk, RtlVirtualUnwind читает UNWIND_INFO и восстанавливает nonvolatile-регистры в caller context.

Если эта цепочка честная, Windows может подняться по стеку. Debugger может показать разумный call stack. Crash dump analyzer может восстановить контекст. Reverse engineer может доверять базовым границам функций. Если цепочка нарушена, появляются ложные стеки, неверные регистры, странные handlers и хорошие места для findings.

Именно поэтому volatile/nonvolatile registers - это не маленькая вставка перед calling convention. Это ключевой слой смысла, который объясняет, почему unwind вообще хранит сохранённые регистры и почему UWOP-операции ограничены конкретными register classes. Следующий раздел про x64 calling convention станет естественным продолжением: мы расширим эту картину от сохранения регистров к передаче аргументов, shadow space, stack alignment, return value и полной связи ABI с unwind.

Короткая формула для запоминания: volatile - caller отвечает сам; nonvolatile - callee обязана вернуть; unwind metadata объясняет, как callee выполнила эту обязанность, если обычный epilog не успел выполниться.

Практический чек-лист для ручного анализа

Когда ты руками смотришь функцию в PE32+ AMD64 и хочешь понять, как volatile/nonvolatile связано с unwind, полезно идти не по памяти, а по порядку. Сначала посмотри на .pdata entry и найди UNWIND_INFO. Затем прочитай CountOfCodes и все UWOP. Отметь, какие nonvolatile GPR и XMM metadata заявляет сохранёнными. Потом открой пролог в дизассемблере и проверь, есть ли соответствующие push/store. После этого посмотри на эпилоги: восстанавливают ли они тот же набор. И только потом делай вывод о нормальности функции.

Если видишь volatile register в UWOP_NONVOL operation, это сильная ошибка register class. Если видишь сохранение nonvolatile в машинном коде, но metadata молчит, это потенциально опасное несоответствие. Если metadata сохраняет регистр, но реальный prolog не похож, это mismatch или unsupported pattern. Если всё совпадает, можно зафиксировать informational result: функция сохраняет такой-то callee-saved state.

Такой порядок дисциплинирует анализ. Он не даёт превращать каждую странность в "обфускацию" и не даёт пропускать реальные нарушения. Для ВКР это особенно важно: показывать не только знания терминов, но и метод проверки.

Шаг	Вопрос	Что фиксировать
1	Какие UWOP_* операции есть в UNWIND_INFO?	ALLOC, PUSH_NONVOL, SAVE_NONVOL, SAVE_XMM128, SET_FPREG.
2	Какие nonvolatile GPR заявлены сохранёнными?	saved_nonvol_mask: RBX/RBP/RSI/RDI/R12-R15.
3	Какие XMM заявлены сохранёнными?	saved_xmm_mask: XMM6-XMM15 в обычной модели.
4	Есть ли volatile-регистры в nonvolatile UWOP?	Если да - register-class error.
5	Совпадает ли metadata с prolog bytes?	Push/store/offset/CodeOffset match или mismatch.
6	Совпадает ли epilog restore с сохранениями?	Проверка выходных путей и восстановлений.
7	Как сформулировать finding?	Malformed, suspicious или informational, с указанием evidence.

Live range: почему одни значения кладут в volatile, а другие в nonvolatile

Чтобы ещё глубже понять выбор регистров, полезно ввести термин live range. Live range (диапазон жизни значения) - это участок машинного кода, на котором значение ещё нужно программе. Значение "живое", если позже оно будет прочитано. Значение "мёртвое", если после текущей точки оно больше не используется. Компилятор постоянно анализирует такие диапазоны жизни, даже если программист в исходном коде об этом не думает.

Если live range очень короткий, volatile-регистр идеален. Например, надо сложить два числа, сравнить результат и сразу перейти по условию. Значение живёт несколько инструкций и не должно переживать call. Сохранять ради него nonvolatile-регистр было бы лишней ценой: push в прологе, pop в эпилоге, UWOP-запись, больше давления на стек и больше кода.

Если live range длинный и проходит через вызовы других функций, ситуация меняется. Значение должно пережить call. Если держать его в RCX, RDX или RAX, caller перед каждым call должен сохранять его сам или пересчитывать. Если держать его в RBX/R12/R13, оно переживёт чужие calls по ABI, но текущая функция должна один раз сохранить старое значение этого nonvolatile-регистра для своего caller-a. Поэтому выбор volatile/nonvolatile связан не только с "какой регистр свободен", а с длиной жизни значения.

Reverse engineer видит результат этих решений, а не сам процесс выбора. Но по коду можно угадывать: если RBX загружается в начале и читается после нескольких call, вероятно, это long-lived value. Если RAX используется между двумя соседними инструкциями и сразу теряется, это temporary. Такие наблюдения помогают понимать декомпилятор и не удивляться, почему одна переменная живёт в RBX, а другая прыгает через RAX/RCX.

live range (диапазон жизни значения) - участок кода, на котором конкретное значение ещё понадобится позже. Если значение должно пережить function call, компилятор часто выбирает nonvolatile-регистр или stack slot; если значение короткое, volatile-регистр обычно дешевле.

liveness analysis (анализ живости значений) - анализ компилятора или reverse engineering инструмента, который определяет, какие значения ещё будут использоваться после данной точки кода. Он объясняет, почему некоторые значения сохраняются через вызовы, а другие спокойно затираются.

```
Короткий live range - удобно volatile:

mov rax, [rcx+10h]
add rax, 5
cmp rax, rdx
jg short done

RAX нужен только на несколько инструкций.
Сохранять его как nonvolatile нет смысла.

Длинный live range - часто удобно nonvolatile:

push rbx
sub rsp, 20h
mov rbx, rcx ; object pointer нужен долго
call validate
mov rcx, rbx ; object pointer всё ещё нужен
call process
mov rax, [rbx+18h] ; object pointer снова нужен
add rsp, 20h
pop rbx
ret
```

Subregisters: почему EAX, AX, AL всё равно относятся к RAX

В дизассемблере ты часто видишь не только RAX, RBX и RCX, но и EAX, AX, AL, ECX, CL и так далее. Это subregisters (подрегистры, частичные имена одного физического регистра). RAX - полный 64-битный регистр. EAX - младшие 32 бита RAX. AX - младшие 16 бит. AL - младшие 8 бит. Для ABI-классификации это всё равно один логический регистр RAX. Если ABI говорит, что RAX volatile, то EAX/AX/AL тоже не дают caller-у гарантии сохранения старого значения RAX через call.

Это важно для чтения кода. Инструкция mov eax, 1 на x86-64 не просто записывает младшие 32 бита: она обнуляет верхние 32 бита RAX. Инструкция mov al, 1 меняет только младший байт, оставляя остальное как было. Такие детали влияют на data flow, но не меняют ABI-класс регистра. Нельзя сказать: "AL volatile, но RAX nonvolatile". В Microsoft x64 RAX как регистр результата является volatile, и все его подимена живут внутри этого правила.

Для nonvolatile-регистров аналогично. RBX nonvolatile как 64-битный регистр. Если callee записывает в EBX, она фактически меняет RBX, потому что запись в 32-битный подрегистр обнуляет верхнюю половину. Значит, если функция меняет EBX, она обязана восстановить RBX целиком перед возвратом. При ручном анализе нельзя пропустить нарушение только потому, что в коде написано ebx, а не rbx.

subregister (подрегистр, частичное имя регистра) - имя части большего регистра: EAX/AX/AL являются частями RAX, EBX/BX/BL - частями RBX. ABI-класс применяется к логическому регистру целиком, поэтому запись в EBX считается изменением RBX.

```
Подрегистры и ABI:

mov eax, 1234h ; меняет RAX, RAX volatile - сохранять для caller-а не надо
mov ebx, 1234h ; меняет RBX, RBX nonvolatile - old RBX должен быть сохранён

Если функция делает mov ebx, ... без сохранения RBX,
это такое же ABI-нарушение, как mov rbx, ... без сохранения RBX.
```

Register home и shadow space: не путать с сохранением nonvolatile

В Windows x64 есть shadow space, который caller резервирует на стеке перед вызовом. Shadow space (теневое пространство, home space для первых четырёх регистровых аргументов) - это 32 байта, связанные с RCX, RDX, R8 и R9. Callee может использовать это место как home addresses для аргументов, если ей нужно сбросить register arguments в память. Подробно shadow space будет разобран в следующем разделе про calling convention, но уже сейчас важно не спутать его с сохранением nonvolatile-регистров.

Когда callee записывает RCX в [rsp+...] как home argument, это не callee-saved сохранение RCX. RCX volatile. Запись в home slot нужна для удобства доступа к аргументу через память, для varargs, для отладочной информации или для внутренних нужд компилятора. Она не создаёт обязанности восстановить RCX перед возвратом. Caller не ждёт RCX обратно.

А когда callee сохраняет RBX в [rsp+offset], это другой смысл. RBX nonvolatile, и старое значение должно быть восстановлено. Такое сохранение имеет отношение к UNWIND_INFO. Поэтому одинаковая инструкция mov [rsp+offset], reg может иметь два разных смысла: home/spill volatile argument или save nonvolatile register. Отличить их можно только по контексту ABI, регистру, месту в прологе и metadata.

shadow space (теневое пространство, 32-байтная область home space для первых четырёх аргументов) - область на стеке, которую caller резервирует по Microsoft x64 calling convention. Она связана с аргументами RCX/RDX/R8/R9, но не является автоматическим сохранением этих volatile-регистров как nonvolatile state.

```
Две похожие записи, разный смысл:

; home/spill volatile argument
mov [rsp+20h], rcx ; RCX volatile, это не callee-saved preservation

; save nonvolatile register
mov [rsp+30h], rbx ; RBX nonvolatile, это может требовать UWOP_SAVE_NONVOL
```

Hand-written assembly: почему ручной код особенно опасен

Hand-written assembly (рукописный ассемблерный код) может быть корректным, быстрым и необходимым. Но именно ручной код часто становится источником ошибок ABI и unwind metadata. Компилятор обычно автоматически знает, какие регистры nonvolatile, какие нужно сохранить, как описать пролог и где поставить .ENDPROLOG. Человек может ошибиться: использовать RBX как временный регистр без сохранения, сохранить RAX через .PUSHREG, забыть описать SAVE_XMM128, неверно указать offset или создать эпилог, который не распознаётся unwinder-ом.

В MSVC x64 inline assembly в привычном старом стиле недоступен для обычного C/C++ x64, поэтому ручной низкоуровневый код часто пишут через отдельные .asm файлы, MASM, intrinsics или специальные compiler-generated stubs. В MASM unwind-метаданные описываются директивами вроде PROC FRAME, .PUSHREG, .ALLOCSTACK, .SETFRAME, .SAVEREG, .SAVEXMM128, .ENDPROLOG. Эти директивы должны соответствовать реальным инструкциям. Если реальная инструкция есть, а директивы нет, unwind будет неполным. Если директива есть, а инструкции нет, unwind будет лгать.

Для reverse engineering это означает: странная metadata не всегда означает злой dynamic. Иногда это просто hand-written assembly. Но с точки зрения проверки механизма разницы мало: Windows будет доверять metadata при unwind. Поэтому static/dynamic должен уметь показать несоответствие без эмоционального вывода: "возможен hand-written assembly mismatch" лучше, чем "malware detected".

hand-written assembly (рукописный ассемблерный код) - код, написанный человеком на assembly, а не автоматически сгенерированный compiler-ом из C/C++. Он может нарушить ABI/unwind правила, если автор неправильно сохранил nonvolatile-регистры или неверно описал пролог директивами.

Register pressure: почему иногда сохраняется много nonvolatile-регистров

Register pressure (давление на регистры, нехватка свободных регистров для всех живых значений) возникает, когда в функции одновременно нужно держать много значений. Чем больше live ranges пересекаются, тем сложнее compiler-у разложить их по доступным volatile и nonvolatile регистрам. Если volatile-регистров не хватает, а значения должны переживать calls, компилятор начинает активнее использовать RBX, RSI, RDI, R12-R15 и иногда XMM6-XMM15.

Большой saved_nonvol_mask не обязательно подозрителен. Он может означать, что функция сложная: много указателей, циклов, вызовов, объектов состояния, обработки ошибок. Но в защитных stubs тоже часто встречаются тяжёлые сохранения: dynamic может сохранить почти всё, чтобы свободно менять состояние, вызывать внутренние helpers, шифровать/дешифровать код, прыгать через dispatchers и потом восстановить окружение.

Поэтому показатель "много nonvolatile saves" нужно интерпретировать осторожно. Сам по себе он informational или mildly suspicious в зависимости от контекста, но не malformed. Сильным finding-ом он становится, если сохранения не соответствуют коду, регистры имеют неправильный class, offsets выходят за разумные границы или эпилоги восстанавливают не тот набор.

register pressure (давление на регистры) - ситуация, когда одновременно живых значений больше, чем удобно держать в доступных регистрах. Тогда компилятор вынужден использовать больше nonvolatile-регистров или сбрасывать значения в память.

Как объяснять это на защите ВКР

Если на защите тебя спросят, почему твой проект вообще смотрит на volatile/nonvolatile registers, не надо отвечать: "потому что так в таблице Microsoft". Сильный ответ должен быть причинным. Windows x64 unwind восстанавливает caller context. Caller context должен сохранять те регистры, которые ABI обещает сохранять через вызов. Эти регистры называются nonvolatile. Поэтому parser обязан понимать register class каждого UWOP, а validator обязан ловить случаи, когда metadata заявляет невозможное или подозрительное сохранение.

Можно сказать так: "Мой инструмент анализирует не просто байты .xdata. Он проверяет, соответствует ли описание функции соглашению Windows x64 ABI. Например, UWOP_PUSH_NONVOL по смыслу может относиться только к nonvolatile integer registers. Если metadata указывает volatile register, это не косметическая странность, а нарушение semantic contract между функцией и unwinder-ом".

Такая формулировка показывает глубину. Ты не просто знаешь список RBX/RBP/R12. Ты понимаешь, почему этот список существует, как он связан с caller/callee responsibility, как код пролога превращается в UNWIND_CODE и почему неправильная metadata ломает stack walking, exception handling и crash dump analysis.

Финальная формулировка для теоретической главы

Volatile и nonvolatile registers в Windows x64 - это способ разделить ответственность за сохранение машинного состояния между caller и callee. Volatile-регистры дают callee свободу для аргументов, return value и временных вычислений; caller не должен ожидать их сохранения через call. Nonvolatile-регистры дают caller-у устойчивое состояние через вызовы; если callee меняет такой регистр, она обязана сохранить старое значение и восстановить его.

В обычном исполнении это обещание выполняет пролог и эпилог функции. В аварийном исполнении, когда происходит exception и обычный epilog не выполняется, обещание выполняется через unwind metadata. UNWIND_INFO хранит сведения о том, где были сохранены nonvolatile-регистры и как восстановить RSP. RtlVirtualUnwind применяет эти

сведения к текущему CONTEXT и строит caller context. Поэтому таблица volatile/nonvolatile регистров является не приложением к calling convention, а фундаментальной причиной существования значительной части UWOP-кодов.

Для reverse engineering это даёт два результата. Во-первых, человек начинает читать функцию не как набор случайных push/mov/pop, а как реализацию ABI-контракта. Во-вторых, инструмент получает строгие критерии проверки: какие register classes допустимы в metadata, какие сохранения должны согласовываться с прологом, какие несоответствия являются ошибкой, а какие требуют осторожной формулировки. Именно на этом уровне static /dynamic перестаёт быть просто parser-ом байтов и становится анализатором смысла unwind-механизма.

4.7. Что такое x64 calling convention и как она связана с unwind

К этому месту мы уже разобрали несколько деталей, которые по отдельности выглядят как отдельные темы: пролог, эпилог, UNWIND_CODE, UWOP-операции, frame pointer, volatile и nonvolatile registers. Но теперь нужно собрать их в одну систему. Эта система называется x64 calling convention. На русском это обычно переводят как "соглашение о вызовах x64". Слово "соглашение" здесь очень точное: процессор сам по себе не знает, что первый аргумент должен быть в RCX, что caller обязан выделить shadow space, что callee должна сохранить RBX, что стек должен быть выровнен. Всё это не врождённые физические законы CPU, а договор между компилятором, линкером, рантаймом, операционной системой, отладчиком, библиотеками и человеком, который пишет или анализирует машинный код.

Если сказать совсем просто, calling convention отвечает на вопрос: как одна функция вызывает другую так, чтобы обе стороны понимали друг друга? Но для нашей темы этого простого ответа мало. Нас интересует не только обычный вызов, когда функция спокойно вернулась через ret. Нас интересует и аварийный выход: исключение, прерывание нормального потока, stack walk, crash dump, debugger backtrace. Поэтому для Windows x64 calling convention - это не просто "куда кладутся аргументы". Это фундаментальная рамка, без которой unwind не может знать, где искать return address, какие регистры надо восстановить, как интерпретировать пролог и почему одни изменения стека должны быть описаны в .xdata, а другие нет.

В хорошей технической книге эта тема не должна подаваться как сухая таблица. Таблица нужна, но она является следствием. Сначала надо увидеть проблему. Представь, что программа состоит из множества функций, написанных разными компиляторами, собранных из разных библиотек, иногда даже написанных вручную на assembler. Одна функция вызывает другую. В момент перехода нужно передать аргументы, сохранить адрес возврата, не потерять важные регистры, не сломать стек, обеспечить возможность вызвать ещё третью функцию, а при исключении - безопасно подняться назад. Если каждый участник будет придумывать свои правила, программа развалится не сразу, а самым неприятным способом: то один указатель окажется затёрт, то ret вернёт управление в мусор, то отладчик не сможет построить call stack, то исключение разрушит контекст caller-a.

Поэтому calling convention является нижним слоем дисциплины. Он говорит: на границе вызова вот такие регистры считаются аргументными, вот такая область на стеке должна существовать, вот такие регистры можно портить, вот такие нужно сохранять, вот так возвращается результат, вот так должен выглядеть стек, а пролог и эпилог должны быть устроены так, чтобы операционная система могла их понять. Именно последняя часть делает тему calling convention напрямую связанной с unwind.

> x64 calling convention (соглашение о вызовах x64) - это набор правил, по которым 64-битный код Windows вызывает функции: какие аргументы идут в регистры, какие уходят на стек, кто выделяет stack space, кто сохраняет регистры, где возвращается результат, как должен быть выровнен стек и какие предположения могут делать compiler, debugger, unwinder и runtime.

ABI (application binary interface, двоичный интерфейс приложения) - это более широкое понятие, чем calling convention. ABI описывает правила совместимости уже скомпилированного машинного кода: calling convention, размеры и выравнивание типов, формат объектных файлов, правила линковки, обработку исключений и другие соглашения, по которым разные бинарные части программы могут работать вместе.

unwind (раскрутка стека, восстановление предыдущего контекста вызова) - это процесс, при котором система по текущему состоянию функции вычисляет состояние её caller-а: новый RIP, новый RSP и значения тех nonvolatile-регистров, которые должны пережить вызов. Calling convention задаёт ожидания, а .pdata/.xdata дают конкретные данные для выполнения этих ожиданий.

Calling convention как договор на границе между функциями

Граница функции - это не строка в исходном коде. В машинном коде нет красивой таблички "здесь началась функция" и "здесь закончилась функция". Есть адреса инструкций, есть stack pointer, есть register state, есть инструкция call, которая кладёт return address на стек и передаёт управление по другому адресу. Всё остальное - смысл, который добавляют компилятор, линкер, отладчик и операционная система.

Именно поэтому слово convention важно переводить не как "механизм", а как "соглашение". Механизмом является, например, инструкция call: CPU действительно выполняет push return address и переходит к цели. A calling convention - это набор правил поверх этой инструкции. CPU не заставляет тебя положить первый аргумент в RCX. CPU не знает, что [rsp+20h] может быть частью shadow space. CPU не проверяет, сохранил ли ты RBX. Но если ты пишешь код для Windows x64 и хочешь, чтобы он вызывался из нормального C/C++ кода, чтобы отладчик видел стек, чтобы исключения проходили через функцию, чтобы библиотеки могли вызывать твой код, ты обязан играть по этим правилам.

Calling convention можно представить как контракт между caller и callee. Caller (вызывающая функция) готовит вызов: размещает аргументы, выделяет обязательную область на стеке, выполняет call. Callee (вызываемая функция) принимает управление: читает аргументы из ожидаемых мест, при необходимости сохраняет nonvolatile-регистры, выделяет локальный стек, выполняет работу, возвращает результат и восстанавливает то состояние, которое по контракту должна восстановить.

В нормальном потоке этот контракт проверяется косвенно: если callee не восстановит RBX, caller позже получит неправильное значение и программа сломается. Но при unwind контракт становится ещё строже. Система должна восстановить состояние caller-а без выполнения нормального эпилога. Поэтому ей нужны не только правила ABI в общем виде, но и конкретная запись того, что сделала данная функция. Так появляется связка: calling convention задаёт обязанность сохранять certain registers, а UNWIND_INFO описывает, как конкретная функция эту обязанность реализовала.

caller (вызывающая функция, вызывающая сторона) - это функция, которая выполняет call. Она отвечает за подготовку аргументов, выделение обязательной shadow space и сохранение volatile-регистров, если ей нужно их значение после вызова.

callee (вызываемая функция, вызываемая сторона) - это функция, на которую передаётся управление. Она получает аргументы согласно calling convention, может использовать

volatile-регистры свободно, но обязана восстановить nonvolatile-регистры, если меняла их значения.

call boundary (граница вызова) - это логическая граница между caller и callee: момент, в котором управление переходит из одной функции в другую и начинают действовать правила calling convention. Для reverse engineer это важная граница смысла, потому что по ней отделяются обязанности двух функций.

Минимальная картина вызова: что происходит вокруг инструкции call

Чтобы calling convention не оставался абстрактным словом, полезно медленно пройти обычный вызов. Допустим, функция A вызывает функцию B. До вызова A живёт в своём stack frame, держит какие-то значения в регистрах, имеет свой return address, свои локальные переменные и может готовить аргументы для B. Затем A должна привести состояние машины к виду, который B ожидает на входе.

В Windows x64 первые четыре целочисленных или pointer-аргумента передаются через RCX, RDX, R8 и R9. Если аргумент является floating-point в обычном прототипированном вызове, для него используются XMM0-XMM3 в соответствующих позициях. Это значит, что аргументная позиция важнее, чем просто тип: первый аргумент связан с RCX или XMM0, второй - с RDX или XMM1, третий - с R8 или XMM2, четвёртый - с R9 или XMM3. Остальные аргументы передаются через стек.

Но перед call caller делает не только раскладку аргументов. Он также обязан обеспечить shadow space. Shadow space (теневое место, home space, домашняя область для регистровых аргументов) - это 32 байта на стеке, которые caller резервирует для callee. Даже если у функции меньше четырёх аргументов, даже если callee никогда не будет туда записывать, сама область по правилу должна быть доступна. Это один из самых характерных признаков Microsoft x64 calling convention.

Затем выполняется instruction call. Call (инструкция вызова) делает две вещи: кладёт return address на стек и меняет RIP на адрес callee. Return address (адрес возврата) - это адрес инструкции, с которой caller должен продолжить выполнение после возврата callee. В момент входа в callee верхушка стека указывает на return address, а сразу за ним располагается shadow space, выделенная caller-ом.

argument register (регистр аргумента) - это регистр, через который calling convention передаёт входной параметр функции. В Microsoft x64 первые четыре integer/pointer позиции используют RCX, RDX, R8, R9; floating-point позиции используют XMM0-XMM3.

shadow space / home space (теневая область / домашняя область регистровых аргументов) - это обязательные 32 байта на стеке, которые caller резервирует перед вызовом. Они дают callee место, куда при необходимости можно сохранить регистровые аргументы RCX, RDX, R8, R9 как обычные stack slots.

return address (адрес возврата) - это адрес в caller-е, куда управление должно вернуться после ret. Инструкция call кладёт этот адрес на стек, а ret потом снимает его со стека и загружает в RIP. Для unwind return address является главным мостом от текущей функции к caller context.

```
Упрощённая картина на входе в callee после call:

RSP -> [ return address ] ; адрес продолжения caller-а
```

```
[ shadow/home slot for RCX ] ; 8 байт
[ shadow/home slot for RDX ] ; 8 байт
[ shadow/home slot for R8 ] ; 8 байт
[ shadow/home slot for R9 ] ; 8 байт
[ stack argument 5 ] ; если есть
[ stack argument 6 ] ; если есть
...

Регистры на входе:
RCX / XMM0 - первая аргументная позиция
RDX / XMM1 - вторая аргументная позиция
R8 / XMM2 - третья аргументная позиция
R9 / XMM3 - четвёртая аргументная позиция
```

Типичная ошибка: считать shadow space локальными переменными caller-а или случайным запасом стека. Это не "мусорные 32 байта" и не оптимизация компилятора. Это часть calling convention. Caller обязан её предоставить, а callee может использовать её как home area для регистровых аргументов.

Почему shadow space вообще существует

Shadow space часто вызывает раздражение у начинающих reverse engineer-ов: кажется, что компилятор зачем-то постоянно делает sub rsp, 20h или sub rsp, 28h, хотя аргументов мало. Чтобы понять смысл, надо помнить, что регистровые аргументы удобны для скорости, но не всегда удобны для адресации. Иногда функции нужен адрес аргумента. Иногда отладчику удобно показывать аргументы как будто они имеют stack locations. Иногда varargs-механизмам и ABI-совместимости нужна предсказуемая домашняя область. Иногда callee хочет освободить RCX/RDX/R8/R9 для других вычислений и сохранить входные значения в память.

Home space решает эту задачу: у первых четырёх аргументов есть потенциальное место в стеке, даже если фактически они пришли через регистры. Это не значит, что каждая функция обязана записывать туда RCX, RDX, R8, R9. Оптимизирующий компилятор часто не будет этого делать, если в этом нет нужды. Но место должно существовать, чтобы callee могла сделать это без дополнительного договора с caller-ом.

Для unwind shadow space важна косвенно. Она сама по себе обычно не описывается как отдельная UWOP-операция, потому что её выделяет caller перед call, а не callee в своём прологе. Но она влияет на форму stack frame, на offsets, на то, где находятся stack arguments, и на то, как reverse engineer понимает обращения вида [rsp+20h], [rsp+28h], [rsp+30h]. Если перепутать shadow space с locals, можно неверно подписать стек, неверно понять аргументы и неверно решить, что функция "использует неинициализированную память".

Особенно важно помнить: shadow space принадлежит вызову. Она существует для callee в контексте конкретного call. Когда callee вызывает ещё одну функцию, она как caller уже должна подготовить shadow space для следующей callee. Поэтому в non-leaf function часто видно, что функция выделяет stack space не только под свои locals и сохранённые регистры, но и под outbound call area - область, которая нужна для её собственных вызовов других функций.

home address / home slot (домашний адрес / домашняя ячейка аргумента) - это stack slot, связанный с регистровым аргументом. Если аргумент пришёл в RCX, его home slot находится в

shadow space; callee может сохранить туда значение, чтобы дальше обращаться к нему как к памяти.

outbound call area (область исходящих вызовов) - это часть stack frame, которую функция использует как caller, когда сама вызывает другие функции. В Windows x64 для каждого такого вызова нужно место как минимум под shadow space, а при большем числе аргументов - ещё и под stack arguments.

Пример: как вызов выглядит в машинном коде

Посмотрим на маленькую схему. Это не конкретный вывод MSVC, который обязан совпасть байт в байт, а учебная форма, показывающая роли. Пусть caller хочет вызвать функцию с шестью integer/pointer аргументами. Первые четыре аргумента уходят в регистры, пятый и шестой - в стек. Но кроме них caller должен иметь shadow space.

В реальном оптимизированном коде порядок инструкций может быть переупорядочен, часть значений может уже находиться в нужных регистрах, stack arguments могут записываться заранее, а sub rsp может быть сделан один раз в прологе функции и использоваться для нескольких вызовов. Но смысл остаётся: перед call машинное состояние должно соответствовать ABI.

```
; caller вызывает:
; target(a1, a2, a3, a4, a5, a6)

sub rsp, 38h ; 20h shadow space + 10h два stack args + выравнивание
mov rcx, a1 ; 1-я integer/pointer позиция
mov rdx, a2 ; 2-я позиция
mov r8, a3 ; 3-я позиция
mov r9, a4 ; 4-я позиция
mov [rsp+20h], a5 ; 5-й аргумент после shadow space
mov [rsp+28h], a6 ; 6-й аргумент
call target
add rsp, 38h ; caller убирает область вызова, если выделял её здесь
```

В оптимизированной функции sub/add вокруг каждого call может не быть. Компилятор часто выделяет нужную outbound area один раз в прологе и использует её для нескольких call-ов. Поэтому reverse engineer должен смотреть не только на одну инструкцию call, а на весь stack frame функции.

RCX, RDX, R8, R9: почему именно эти регистры так часто мелькают перед call

Когда ты смотришь Windows x64 дизассемблирование, перед call постоянно встречаются mov rcx, ..., lea rcx, ..., mov rdx, ..., xor r8d, r8d, mov r9, ... Это не случайный стиль компилятора. Это проявление calling convention. RCX, RDX, R8 и R9 являются первыми четырьмя integer/pointer argument registers. Поэтому подготовка вызова часто выглядит как набор записей в эти регистры.

RCX особенно часто оказывается указателем this в C++ member function. This pointer (указатель this) - это скрытый первый аргумент нестатического метода C++, указывающий на объект, для которого вызывается метод. В Microsoft x64 он обычно передаётся через RCX, потому что занимает первую аргументную позицию. Поэтому, когда reverse engineer видит перед call инструкцию lea rcx, [rbp-...] или mov rcx, rbx, это может быть подготовка this для метода, но не всегда. Это просто первая аргументная позиция; смысл

определяется контекстом.

RDX часто становится вторым аргументом, R8 третьим, R9 четвёртым. Если функция принимает больше аргументов, остальные окажутся на стеке после shadow space. Но тут есть тонкость: "пятый аргумент" не всегда лежит по одинаковому offset от текущего RSP во всей функции. Offset зависит от того, где именно мы смотрим: перед call в caller-е, внутри callee после пролога, после sub rsp, после push сохранённых регистров, при наличии frame pointer. Поэтому calling convention задаёт логический порядок, а конкретный адрес всегда нужно считать относительно текущего stack layout.

this pointer (указатель this) - это скрытый аргумент нестатического метода C++, который указывает на объект. На Windows x64 для обычного метода он обычно передаётся в RCX как первая integer/pointer аргументная позиция.

argument position (позиция аргумента) - это порядковое место параметра в вызове: первый, второй, третий, четвёртый и так далее. В Microsoft x64 именно позиция определяет, какой регистр или stack slot используется, а не просто имя параметра из исходного кода.

XMM0-XMM3 и floating-point аргументы

Calling convention касается не только целочисленных регистров. Для floating-point аргументов используются XMM-регистры. Floating-point argument (аргумент с плавающей точкой) - это значение вроде float или double. В обычном прототипированном вызове первые четыре floating-point позиции передаются в XMM0-XMM3. Если, например, первый аргумент double, он пойдёт в XMM0; если второй аргумент double - в XMM1. При этом integer/pointer аргументы соответствующих позиций используют RCX/RDX/R8/R9.

Для reverse engineering это важно, потому что вызов функции может готовиться не только mov rcx, ..., но и movsd xmm0, ..., movss xmm1, ..., xorps xmm2, xmm2. Если анализатор или человек видит только GPR-регистры, он может решить, что аргументов мало, хотя часть аргументов передаётся через XMM. Особенно часто это проявляется в математическом коде, графике, обработке аудио/видео, играх, SIMD-вычислениях и библиотеках, где double/float встречаются постоянно.

С unwind floating-point аргументы связаны косвенно. Передача аргументов сама по себе не описывается в UNWIND_INFO. Но XMM-регистры имеют volatile/nonvolatile-правила, а часть nonvolatile XMM-регистров при сохранении может описываться через UWOP_SAVE_XMM128. Поэтому calling convention объясняет не только путь аргументов, но и то, почему сохранение некоторых XMM-регистров может появиться в unwind metadata.

XMM register (XMM-регистр) - это 128-битный SIMD/векторный регистр архитектуры x86-64, который используется для floating-point значений, SIMD-операций и некоторых ABI-задач. В Windows x64 XMM0-XMM3 участвуют в передаче первых floating-point аргументов.

floating-point argument (аргумент с плавающей точкой) - это параметр типа float, double или похожего числового вида, который передаётся не через обычный integer-регистр, а через XMM-регистр в соответствующей аргументной позиции.

Varargs и дублирование floating-point значений

Есть неприятная, но важная тонкость: varargs. Varargs (переменное число аргументов) - это механизм функций вроде printf, где точное количество и типы некоторых аргументов

определяются не обычным строгим прототипом, а форматом или соглашением внутри функции. Для таких вызовов Microsoft x64 ABI требует особой совместимости: floating-point значения в первых четырёх позициях могут дублироваться и в XMM-регистрах, и в соответствующих general-purpose регистрах.

Зачем это нужно? Varargs-функция не всегда заранее знает тип каждого аргумента так же строго, как обычная функция с полным прототипом. Поэтому ABI делает передачу более универсальной: значение доступно так, чтобы механизм разбора аргументов мог корректно его достать. Для reverse engineer это означает, что перед вызовом printf-подобной функции можно увидеть, например, подготовку XMM0 и одновременно запись в RCX/RDX/R8/R9 для совместимости. Если этого не знать, легко принять дублирование за странную лишнюю работу или за два разных аргумента.

К unwind varargs опять же относятся косвенно. Сам механизм переменного числа аргументов не говорит Windows, как раскручивать стек. Но он влияет на то, как выглядит caller-side preparation: какие регистры устанавливаются, какие stack slots используются, почему некоторые значения продублированы. Для static /dynamic это важно на уровне интерпретации call site: не надо считать любое дублирование аргументов аномалией, если вызов может быть varargs или unprototyped.

varargs (variable arguments, переменное число аргументов) - это форма вызова, где функция принимает фиксированную часть параметров и затем неопределённое заранее количество дополнительных аргументов. Классический пример - printf. В Windows x64 такие вызовы имеют специальные ABI-правила для floating-point аргументов.
unprototyped function (функция без полного прототипа) - это функция, для которой вызывающая сторона не имеет полного объявления типов параметров на этапе компиляции. Для совместимости такие вызовы могут использовать правила, похожие на varargs, включая дублирование floating-point значений.

Return value: где функция возвращает результат

Calling convention описывает не только вход в функцию, но и выход из неё. Return value (возвращаемое значение) обычно возвращается через RAX для integer/pointer результатов и через XMM0 для floating-point результатов. Если возвращаемое значение простое: int, pointer, bool, enum, 64-битное целое - ожидай RAX. Если double или float - часто XMM0. Более сложные структуры могут возвращаться иначе: через скрытый pointer на буфер, который caller передаёт callee, или по специальным правилам для небольших aggregate types.

Для обычного исполнения это понятно: caller после call читает RAX или XMM0. Для reverse engineering это помогает понять, почему после call сразу идёт test rax, rax, mov [rbx], rax, cmp eax, ..., movsd [mem], xmm0. Это не случайный выбор регистров: caller забирает результат из места, определённого ABI.

Для unwind return value обычно не является главным объектом восстановления. Если произошло исключение, функция не возвращает обычный результат. Но знание return value важно для понимания эпилогов, tail calls и восстановления контекста. Кроме того, RAX является volatile, поэтому unwind не обязан сохранять его как значение caller-a. Если caller хотел сохранить RAX через следующий вызов, это ответственность caller-a. Так calling convention снова объясняет, почему одни регистры важны для unwind, а другие обычно

нет.

return value (возвращаемое значение) - это результат функции, который caller получает после завершения callee. В Microsoft x64 простые integer/pointer результаты обычно возвращаются в RAX, floating-point результаты - в XMM0, а сложные aggregate-результаты могут использовать скрытый адрес буфера.

aggregate type (агрегатный тип) - это составной тип вроде структуры или массива. В ABI такие типы могут передаваться и возвращаться по особым правилам, потому что их размер и выравнивание не всегда удобно помещаются в один регистр.

Stack alignment: почему выравнивание стека не декоративная мелочь

Теперь надо поговорить о stack alignment. На первый взгляд выравнивание кажется низкоуровневой бухгалтерией: ну стоит адрес кратно 16 или не стоит, какая разница? Разница большая. Многие инструкции, ABI-правила, оптимизации и соглашения ожидают, что стек в определённых участках кода выровнен. В Windows x64 стек должен быть 16-byte aligned в стабильных участках выполнения функции, кроме специальных зон вроде пролога и эпилога. В момент самого входа после call ситуация имеет естественный сдвиг из-за return address: call положил 8 байт, поэтому callee своим прологом обычно приводит RSP к нужному выравниванию для тела функции.

Alignment (выравнивание) означает, что адрес кратен некоторому числу. 16-byte alignment значит, что адрес делится на 16 без остатка. Для стека это важно потому, что компилятор может генерировать инструкции и обращения к данным, предполагая корректное выравнивание. Если hand-written assembly нарушает это правило, программа может не падать сразу. Она может работать в одном месте, а потом странно ломаться при вызове библиотечной функции, использовании SIMD, обработке исключения или при оптимизированном коде, который полагается на ABI.

Для unwind выравнивание важно не потому, что UNWIND_INFO хранит отдельный флаг "стек выровнен". Важен общий контракт: RtlVirtualUnwind восстанавливает RSP, исходя из описанных push/sub/save операций и распознавания эпилога. Если функция нарушает ABI и держит стек в неожидаемом состоянии в местах, где это не разрешено, stack walking может стать ненадёжным. А если пролог/эпилог сделаны нестандартно, их нельзя корректно описать обычными UWOP-кодами.

stack alignment (выравнивание стека) - это требование, чтобы значение RSP в определённых точках выполнения имело адрес, кратный заданной границе, обычно 16 байтам. Это часть ABI, влияющая на корректность вызовов, SIMD-операций, оптимизаций и unwind.

16-byte aligned (выровнено по 16 байтам) - это значит, что адрес делится на 16 без остатка. Для Windows x64 это важное состояние стека в теле функции, вне пролога/эпилога и специальных исключений.

```
Упрощённая интуиция выравнивания:

перед call caller подготавливает RSP так, чтобы вызов был ABI-correct
call кладёт 8-байтный return address
callee входит с return address на [rsp]
callee в прологе выделяет stack frame так, чтобы тело функции имело корректное alignment

Поэтому часто встречается sub rsp, 28h:
```

```
20h - shadow space для будущих вызовов
08h - компенсация из-за return address / alignment
```

Не надо автоматически думать, что sub rsp, 28h всегда означает "32 байта локальных переменных плюс 8 байт мусора". Очень часто это ABI-форма: shadow space для исходящих вызовов плюс выравнивание. Смысл зависит от функции, наличия call-ов, локальных данных и сохранённых регистров.

Stack arguments: что происходит после первых четырёх аргументов

Первые четыре аргументные позиции получают привилегию регистров. Но функции могут принимать больше параметров. Тогда дополнительные аргументы размещаются на стеке. Важно: они идут после shadow space. Поэтому пятый аргумент на стороне caller-а перед call часто оказывается по адресу [rsp+20h], шестой - [rsp+28h], седьмой - [rsp+30h], если считать от RSP, который уже указывает на начало области вызова до выполнения call. После входа в callee из-за return address смещения будут выглядеть иначе.

Эта разница смещений - один из источников ошибок. Reverse engineer смотрит в caller-е и видит запись [rsp+20h]. Потом смотрит внутри callee и видит обращение [rsp+28h] или [rsp+30h] и пытается механически сопоставить их без учёта return address и prolog allocation. Так делать нельзя. Нужно каждый раз задавать вопрос: относительно какого RSP сейчас считается offset? Это RSP до call? RSP на входе в callee? RSP после push rbp? RSP после sub rsp, N? Или используется RBP/R13 как frame pointer?

Calling convention даёт логический порядок stack arguments, но физические offsets зависят от текущей точки кода. Поэтому в хорошей ручной разметке нужно отделять abstract parameter number от concrete memory address. Пятый аргумент - это логическое понятие. Его конкретное местоположение может быть выражено разными offset-ами в разных фазах функции.

stack argument (стековый аргумент) - это аргумент, который передаётся через память на стеке, а не через регистр. В Microsoft x64 аргументы после первых четырёх позиций обычно размещаются на стеке после обязательной shadow space.

offset (смещение) - это расстояние от базового адреса до нужной ячейки памяти. В выражении [rsp+28h] смещение равно 28h от текущего значения RSP. Важно: если RSP изменился, то тот же offset указывает уже на другое место.

Caller-saved и callee-saved как часть calling convention

Предыдущий раздел был посвящён volatile и nonvolatile registers. Теперь нужно прямо встроить его в calling convention. Volatile register и caller-saved register - это две стороны одной идеи: если caller хочет сохранить значение через вызов, он должен сделать это сам. Callee не обязана беречь такие регистры. Nonvolatile register и callee-saved register - обратная сторона: callee может пользоваться таким регистром, но обязана вернуть прежнее значение.

Зачем ABI делит ответственность? Потому что регистры ограничены. Если бы callee была обязана сохранять все регистры, каждая маленькая функция имела бы дорогой пролог и эпилог. Если бы caller был обязан сохранять все регистры, каждый вызов стал бы дорогим даже тогда, когда callee почти ничего не делает. Деление позволяет компилятору выбирать: быстрые временные значения держать в volatile-регистрах, долгоживущие

значения - в nonvolatile, а сохранить только то, что действительно нужно.

Связь с unwind здесь прямая. Unwind не обязан восстанавливать весь процессорный мир в том виде, в каком он был до каждого call. Он должен восстановить caller context согласно ABI. То есть особенно важны RIP, RSP и nonvolatile-регистры, которые callee должна была сохранить. Именно поэтому UWOP_PUSH_NONVOL, UWOP_SAVE_NONVOL и UWOP_SAVE_XMM128 существуют как коды описания пролога. Они не описывают "все записи в регистры". Они описывают те действия, которые нужны для выполнения ABI-обещания при аварийном выходе.

caller-saved register (регистр, сохраняемый caller-ом) - это volatile-регистр: если caller хочет, чтобы значение пережило call, caller сам сохраняет его до вызова. Callee может его затереть.

callee-saved register (регистр, сохраняемый callee) - это nonvolatile-регистр: если callee меняет его, callee должна сохранить старое значение и восстановить его перед возвратом; при unwind это восстановление выполняется по unwind metadata.

Почему calling convention определяет форму пролога

Теперь становится понятно, почему Microsoft x64 prolog не является произвольным набором инструкций. Пролог - это место, где callee выполняет свои ABI-обязанности: сохраняет nonvolatile registers, выделяет stack frame, иногда устанавливает frame pointer. Но Windows x64 хочет уметь восстановить caller context из любой точки функции. Значит, действия пролога должны быть не только корректными, но и описуемыми.

Если callee сохраняет RBX через push rbx, это можно описать UWOP_PUSH_NONVOL. Если выделяет стек через sub rsp, 80h, это можно описать UWOP_ALLOC_SMALL или UWOP_ALLOC_LARGE. Если сохраняет R12 по адресу [rsp+40h], это можно описать UWOP_SAVE_NONVOL. Если устанавливает frame pointer через lea rbp, [rsp+...] или mov rbp, rsp в допустимой форме, это связывается с UWOP_SET_FPREG и полями FrameRegister/FrameOffset. То есть calling convention говорит, что нужно сохранить, а unwind encoding говорит, как это сохранение записать в .xdata.

Если hand-written assembly делает что-то экзотическое: меняет RSP несколько раз в непредсказуемом порядке, сохраняет nonvolatile-регистр по адресу, который потом сложно выразить, использует нестандартный эпилог, прыгает в середину восстановления стека, - такой код может работать при обычном исполнении, но стать плохим гражданином Windows x64 ABI. Его сложно или невозможно корректно описать стандартной unwind metadata. Тогда исключение, debugger stack walk или crash dump могут показать мусор.

describable prolog (описуемый пролог) - это пролог, действия которого можно выразить через стандартные UNWIND_CODE операции: сохранение nonvolatile-регистров, выделение стека, установка frame pointer и некоторые специальные случаи. Windows x64 требует такой дисциплины, чтобы unwind был надёжным.

hand-written assembly (ассемблер, написанный вручную) - это код, который написал человек, а не сгенерировал компилятор. Такой код обязан самостоятельно соблюдать calling convention и правильно порождать или объявлять unwind information, иначе исключения и stack walk могут ломаться.

Почему calling convention определяет форму эпилога

Эпилог - это обратная сторона пролога. Если в прологе функция взяла на себя долг, то в эпилоге она этот долг возвращает: восстанавливает RSP, возвращает nonvolatile registers, выполняет ret или допустимый tail jmp. Для обычного исполнения это понятно. Но для unwind важна ещё одна деталь: RtlVirtualUnwind может обнаружить, что ControlPc находится внутри эпилога. Тогда функция уже начала разбирать свой stack frame. Если применить к ней обычную модель "мы в теле функции, надо отменять весь пролог", можно восстановить неверный контекст.

Поэтому эпилоги на Windows x64 должны быть распознаваемыми. Unwinder не хранит отдельный список всех эпилогов в UNWIND_INFO в классической модели. Он сканирует код по допустимым шаблонам эпилога. Это возможно только потому, что ABI и правила prolog/epilog ограничивают, как эпилог может выглядеть. Разрешаются определённые формы восстановления RSP, pop nonvolatile-регистров и завершающий ret или jmp в допустимой ситуации.

Calling convention здесь снова работает как дисциплина формы. Функция не просто "как-нибудь возвращается". Она возвращается так, чтобы система могла понять, на какой стадии выхода она находится. Это особенно важно для асинхронных событий: исключение может быть поймано не в удобной точке, а прямо между add rsp, 80h и pop rbx, или перед ret. Если эпилог распознаваем, Windows может правильно симулировать завершение функции.

recognizable epilog (распознаваемый эпилог) - это завершающая последовательность инструкций, которую Windows x64 unwinder может определить как эпилог по машинному коду. Это нужно, чтобы правильно обрабатывать ситуации, когда ControlPc находится в процессе выхода из функции.
ControlPc (control program counter, адрес текущего управления) - это адрес инструкции, относительно которого unwinder определяет, где сейчас находится выполнение: в прологе, в теле функции, в эпилоге или вне функции. На x64 он связан с текущим RIP, но используется как термин в механизмах unwind.

Leaf function и non-leaf function в свете calling convention

Calling convention также объясняет, почему leaf function часто проще для unwind. Leaf function (листовая функция) - это функция, которая не вызывает другие функции. В практическом Windows x64 контексте особенно важен ещё более строгий случай: функция не изменяет nonvolatile-регистры и не требует полноценного stack frame. Такая функция может не иметь записи в .pdata, потому что её возврат можно смоделировать простым снятием return address со стека. Но это не значит, что "leaf function всегда без .pdata". Если leaf function выделяет стек, сохраняет nonvolatile-регистр или делает то, что требует unwind description, metadata может понадобиться.

Non-leaf function (нелистовая функция) вызывает другие функции. Значит, она часто должна подготовить outbound call area, соблюдать shadow space для своих вызовов, выравнивать стек, сохранять значения через nested calls и иметь полноценный пролог/эпилог. Поэтому non-leaf функции чаще имеют .pdata/.xdata. Но опять же правило не механическое: наличие или отсутствие metadata определяется тем, нужно ли системе специальное описание для восстановления контекста.

Для reverse engineer это важно потому, что нельзя считать каждую функцию без .pdata "мусором" или "обфускацией". Но и нельзя игнорировать отсутствие metadata у сложной функции, которая явно меняет RSP и сохраняет nonvolatile-регистры. static /dynamic должен смотреть на соответствие: что код реально делает и можно ли это согласовать с ABI/unwind expectations.

leaf function (листовая функция) - это функция, которая не вызывает другие функции. В контексте unwind особенно важны leaf-функции, которые не меняют nonvolatile-регистры и не требуют описанного stack frame; такие функции могут обходиться без полноценной unwind metadata.

non-leaf function (нелистовая функция) - это функция, которая вызывает другие функции. Ей часто нужны shadow space для исходящих вызовов, stack allocation, сохранение nonvolatile-регистров и, как следствие, unwind metadata.

Как calling convention видит стек: не "куча байтов", а структура ответственности

В отладчике стек часто выглядит как длинный массив адресов и значений. Но calling convention заставляет видеть в нём структуру. На вершине кадра есть return address. За ним есть home space. Дальше могут быть stack arguments. Внутри callee после пролога могут появиться saved nonvolatile registers, local variables, spill slots, alignment padding, outbound call area. Эти области могут физически соседствовать, но смысл у них разный.

Saved register slot (ячейка сохранённого регистра) существует потому, что callee обязана вернуть nonvolatile-регистр. Local variable slot (ячейка локальной переменной) существует потому, что компилятору нужно место для локального значения. Spill slot (ячейка выгрузки регистра) появляется, когда компилятору не хватает регистров или нужно временно переложить значение в память. Outbound argument slot (ячейка исходящего аргумента) нужна для вызова следующей функции. Shadow space нужна по ABI для callee следующего вызова.

Для UNWIND_INFO не все эти области одинаково важны. Unwind не обязан описывать каждую локальную переменную. Он не обязан объяснять высокоуровневый смысл spill slots. Его интересуют те изменения, которые нужны для восстановления caller context: как изменился RSP, где сохранены nonvolatile-регистры, установлен ли frame pointer, есть ли machframe, есть ли handler/chained info. Поэтому calling convention помогает понять границу: что важно для выполнения программы вообще, что важно для decompiler-а, а что важно именно для unwind.

spill slot (ячейка выгрузки) - это место в stack frame, куда компилятор временно сохраняет значение из регистра, например из-за нехватки регистров или необходимости пережить участок кода. Spill slot не обязательно является локальной переменной исходного кода и обычно не описывается в unwind metadata.

saved register slot (ячейка сохранённого регистра) - это место в стеке, куда callee сохраняет старое значение nonvolatile-регистра. Такие ячейки важны для unwind, потому что RtlVirtualUnwind должен восстановить эти регистры при аварийном выходе.

Связь с .pdata и .xdata: где calling convention превращается в данные

Calling convention существует как правило. Но rule сам по себе не лежит в PE-файле рядом с каждой функцией. В PE лежит конкретизация: .pdata говорит, какие диапазоны кода являются функциями с unwind-описанием; .xdata хранит UNWIND_INFO; UNWIND_INFO хранит коды действий пролога. То есть ABI даёт общий закон, а .pdata/.xdata дают факты о конкретной функции.

Например, ABI говорит: RBX nonvolatile, если callee его меняет, старое значение должно быть восстановлено. Но ABI не знает, где именно функция сохранила RBX. Одна функция сделала push rbx. Другая сохранила mov [rsp+38h], rbx. Третья вообще не трогала RBX. Четвёртая использовала RBP как frame pointer. Эти конкретные решения компилятора описываются в UNWIND_INFO. Поэтому RtlVirtualUnwind не просто "знает calling convention". Она берёт FunctionEntry из .pdata, получает UnwindInfoAddress, читает .xdata и применяет конкретную последовательность unwind actions.

Это важнейшая мысль всей главы: Windows x64 unwind не угадывает кадр по красивому виду кода. Она опирается на table-driven model. Но эта таблица имеет смысл только потому, что код сгенерирован по calling convention. Если код полностью нарушает ABI, таблица либо не сможет его описать, либо будет лгать. Если таблица лжёт, unwind восстановит неправильный caller context.

table-driven unwind (табличная раскрутка стека) - это модель, где система восстанавливает предыдущий кадр не угадыванием по стеку, а по таблицам .pdata/.xdata. .pdata находит функцию, .xdata описывает, как отменить действия её пролога.
unwind action (действие раскрутки) - это логическая операция, которую RtlVirtualUnwind выполняет при восстановлении caller context: восстановить nonvolatile-регистр, прибавить к RSP выделенный размер стека, обработать frame pointer, прочитать return address и так далее.

RtlVirtualUnwind как исполнитель договора

RtlVirtualUnwind можно понимать как исполнителя договора между ABI и конкретной функцией. Она получает текущий ControlPc, контекст регистров и FunctionEntry. По ControlPc определяется, где мы находимся: в прологе, теле, эпилоге. По FunctionEntry находится UNWIND_INFO. По UNWIND_INFO система понимает, какие действия пролога уже должны быть отменены, чтобы получить caller context.

Если мы в теле функции, обычно нужно отменить весь пролог: восстановить сохранённые nonvolatile-регистры, вернуть RSP к состоянию перед входом в callee, снять return address. Если мы в прологе, нужно быть осторожнее: не все действия пролога могли уже выполниться. Поэтому CodeOffset в UNWIND_CODE помогает понять, какие операции уже произошли к текущему ControlPc. Если мы в эпилоге, unwinder распознаёт эпилог и симулирует его завершение, потому что функция уже начала возвращаться.

Calling convention здесь задаёт то, что считается правильным результатом. Caller context - это не произвольный предыдущий набор байтов. Это состояние, которое caller имел бы после корректного возврата callee: RIP указывает на continuation после call, RSP указывает на стек caller-a, nonvolatile-регистры имеют значения caller-a, volatile-регистры не гарантируются. Поэтому RtlVirtualUnwind не обязана восстанавливать всё подряд. Она восстанавливает именно ABI-значимый минимум для продолжения stack walk и обработки исключения.

caller context (контекст вызывающей функции) - это восстановленное состояние caller-a: адрес продолжения выполнения, stack pointer и ABI-значимые регистры, которые должны пережить вызов. Это цель одного шага virtual unwind.
virtual unwind (виртуальная раскрутка) - это восстановление предыдущего кадра без фактического выполнения машинных инструкций эпилога. Система как бы моделирует, что функция вернулась, используя UNWIND_INFO и правила calling convention.

Почему volatile-регистры обычно не спасаются unwind-ом

Один из важных моментов: начинающий может ожидать, что unwind должен восстановить все регистры ровно такими, какими они были у caller-a до call. Но это не так. ABI не обещает caller-у сохранность volatile-регистров. Значит, у unwind нет обязанности восстанавливать их как старые значения caller-a. Если исключение произошло внутри callee, значения RAX, RCX, RDX, R8, R9, R10, R11 могут отражать внутреннюю работу callee. Они могут быть полезны для отладки, но не являются гарантированной частью caller state.

Зато nonvolatile-регистры должны быть восстановлены. Если caller держал важный указатель в RBX, вызвал callee, а callee сохранила RBX на стеке, то при unwind старый RBX надо достать из saved register slot. Именно это делает UWOP_PUSH_NONVOL или UWOP_SAVE_NONVOL значимым. Они не просто описывают "какая инструкция была в прологе". Они объясняют, как вернуть ABI-обещанное состояние caller-a.

Это также помогает читать crash dumps. В дампе после аварии часть volatile-регистров может быть значениями глубокой внутренней функции, а nonvolatile-регистры после шагов unwind должны постепенно возвращаться к значениям outer callers. Если инструмент неправильно применил unwind, nonvolatile-регистры могут стать неверными, а дальнейший stack trace потеряет смысл.

Почему не все обращения к стеку относятся к unwind

В машинном коде функций много обращений к стеку: [rsp+20h], [rsp+30h], [rbp-10h], [rsp+88h]. Не каждое такое обращение важно для unwind. Calling convention помогает разделить их по смыслу. Если запись сохраняет nonvolatile-регистр в прологе, это unwind-significant действие. Если запись кладёт локальную переменную, это важно для логики функции, но не обязательно для раскрутки. Если запись готовит пятый аргумент для следующего call, это часть caller-side call preparation, но не действие восстановления текущего кадра.

Поэтому static не должен превращать every stack store в finding о unwind. Нужно сначала понять роль: это save nonvolatile? stack allocation? frame pointer setup? home parameter? spill? local? outbound argument? Только часть этих ролей должна совпадать с UNWIND_CODE. Слишком агрессивный анализ даст ложные срабатывания. Слишком слабый анализ пропустит реальные несоответствия, например функцию, которая сохраняет RBX, но не описывает это в UNWIND_INFO.

Хорошая модель такая: calling convention даёт карту обязанностей, disassembly показывает фактические действия, .xdata показывает официальное описание для Windows. Reverse engineer сравнивает три слоя. Если код делает ABI-значимое действие, но metadata молчит, это подозрительно. Если metadata говорит о сохранении R12, а в прологе нет соответствующего действия, это тоже подозрительно. Если metadata не

описывает обычный local store, это нормально.

unwind-significant action (действие, значимое для unwind) - это действие функции, которое влияет на восстановление caller context: изменение RSP, сохранение nonvolatile-регистра, установка frame pointer, machframe и другие операции, выражаемые через UNWIND_CODE.

false positive (ложное срабатывание) - это ситуация, когда инструмент считает нормальное поведение ошибкой или аномалией. В анализе unwind false positive часто возникает, если путать locals/spills/outbound arguments с сохранёнными nonvolatile-регистрами.

Как calling convention помогает декомпилятору

Декомпилятор не восстанавливает исходный код магически. Он строит модель функции из машинных инструкций, control flow, data flow, calling convention и metadata. Если он знает, что RCX - первая аргументная позиция, он может предположить первый параметр. Если видит, что перед call записывается RDX, он может подписать второй аргумент. Если знает, что RAX после call содержит return value, он может построить выражение вида result = foo(...). Если знает, что RBX nonvolatile и сохраняется в прологе, он не будет ошибочно считать push rbx частью пользовательской логики.

Без calling convention дизассемблер видит только инструкции. С calling convention он начинает видеть роли: это аргумент, это return value, это сохранение callee-saved регистра, это shadow space, это stack argument, это frame pointer. Поэтому неправильная calling convention в настройках инструмента может разрушить декомпиляцию. Код всё ещё будет дизассемблироваться, но смысл будет странным: параметры не там, стековые переменные перепутаны, call signatures выглядят неверно, saved registers превращаются в непонятные locals.

Для Windows x64 особенно важно не переносить механически знания из x86. В x86 было много calling conventions: cdecl, stdcall, fastcall, thiscall, разные правила очистки стека. В Windows x64 модель унифицированнее: стандартная convention использует RCX/RDX/R8/R9, shadow space, callee-saved набор регистров и table-driven unwind. Поэтому x64-код надо читать как x64-код, а не как "x86, только регистры длиннее".

decompiler (декомпилятор) - это инструмент, который пытается поднять машинный код до псевдо-C/псевдо-C++ представления. Он использует calling convention, чтобы понять параметры, возвращаемые значения, сохранённые регистры и структуру stack frame.

data flow (поток данных) - это анализ того, откуда значение пришло и куда оно дальше используется. Calling convention задаёт важные источники и приёмники data flow: argument registers на входе, return register после call, stack arguments и saved registers.

Windows API и практическое чтение вызовов

Когда программа вызывает Windows API, calling convention проявляется особенно наглядно. Возьмём условный вызов CreateFileW или MessageBoxW. Перед call ты увидишь подготовку RCX, RDX, R8, R9 и запись дополнительных аргументов на стек. Если есть PDB или импортная таблица с именем функции, инструмент может показать красивые имена параметров. Но даже без них базовая механика остаётся той же: первые четыре позиции в регистрах, остальные на стеке после shadow space.

Это важно для реверса malware, packed binaries, game executables, драйверных утилит и обычных приложений: по подготовке регистров перед call можно восстановить, что передаётся функции. Но надо помнить о переупорядочивании. Оптимизирующий компилятор может подготовить RCX сильно раньше call, затем использовать несколько инструкций, потом подготовить RDX. Может держать this в RBX и перед call сделать mov rcx, rbx. Может переиспользовать outbound area между несколькими вызовами. Поэтому правильный анализ call site требует смотреть backward slice, а не только две строки перед call.

Backward slice (обратный срез) - это мысленный или автоматический анализ: от места использования значения назад к месту, где оно было определено. Для аргумента RCX это означает: где последнее значение RCX перед call было установлено? Не было ли оно сохранено через другой регистр? Не изменилось ли оно условной веткой? Не является ли оно this pointer? Не загружено ли оно из stack slot? Calling convention говорит, что RCX важен, но data flow объясняет, какое именно значение там оказалось.

Windows API (прикладной интерфейс Windows) - это набор функций операционной системы, которые вызываются обычным кодом программы. На x64 такие вызовы подчиняются Microsoft x64 calling convention, поэтому аргументы API тоже раскладываются по RCX/RDX/R8/R9 и стеку.

backward slice (обратный срез данных) - это анализ, который идёт от использования значения назад к его источнику. При разборе call это помогает понять, откуда пришли аргументы в RCX, RDX, R8, R9 и stack slots.

Tail call: когда вызов становится выходом из функции

Tail call (хвостовой вызов) - это ситуация, когда функция вместо обычного call + ret передаёт управление другой функции как своему финальному действию. В машинном коде это часто выглядит как jmp target после восстановления собственного stack frame. С точки зрения обычного потока это оптимизация: не нужно создавать ещё один return address и затем сразу возвращаться. Но для unwind tail call связан с эпилогом.

В Windows x64 допустимый tail call должен быть совместим с эпилогом: функция должна уже восстановить RSP и nonvolatile-регистры так, как будто она завершилась, и только затем выполнить jmp в другую функцию. Поэтому Microsoft rules допускают некоторые формы jmp как завершение эпилога. Это не произвольный прыжок из середины функции. Это контролируемая форма выхода.

Для reverse engineer tail call может сбивать call graph. В коде нет call, есть jmp, но логически это вызов следующей функции без возврата в текущую. Для stack trace это тоже важно: tail-called функция может выглядеть так, будто промежуточный кадр отсутствует. Это не обязательно обфускация. Это нормальная оптимизация, если выполнены правила ABI и эпилога.

tail call (хвостовой вызов) - это передача управления другой функции в самом конце текущей функции, часто через jmp вместо call/ret. В Windows x64 он должен быть совместим с правилами эпилога, чтобы unwind мог корректно понимать выход из функции.

call graph (граф вызовов) - это представление отношений "какая функция вызывает какую". Tail calls усложняют call graph, потому что логический вызов может быть реализован через jmp, а не через инструкцию call.

Thunks, wrappers и маленькие переходные функции

В реальных бинарях много маленьких функций, которые почти ничего не делают: переставляют аргументы, добавляют this, прыгают на импорт, вызывают другую реализацию, проверяют флаг и переходят дальше. Такие функции часто называют thunks или wrappers. Thunk (переходная функция-прокладка) - это маленький фрагмент кода, который адаптирует вызов или перенаправляет управление. Wrapper (обёртка) - функция, которая вызывает другую функцию, добавляя небольшой слой логики.

Calling convention особенно важна для таких функций, потому что их смысл часто полностью состоит в перестановке ABI-состояния. Например, thunk может взять значение из RDX, переложить в RCX и прыгнуть в метод. Или wrapper может подготовить shadow space, добавить скрытый параметр, вызвать API и вернуть RAX. Если читать такой код без calling convention, он выглядит как бессмысленные mov/jmp. Если читать с calling convention, видно: код адаптирует аргументы между двумя интерфейсами.

Для unwind маленькие thunks могут быть leaf-like и не иметь полноценной .xdata, если они не трогают stack frame и nonvolatile registers. Но wrapper, который выделяет стек или сохраняет регистры, должен быть описан. Поэтому при анализе нельзя судить только по размеру функции. Маленькая функция может быть ABI-важной, а большая может иметь простой unwind. Нужно смотреть действия.

thunk (переходная функция-прокладка) - это маленький фрагмент кода, который перенаправляет вызов, адаптирует аргументы, переходит к импортной функции или связывает разные уровни ABI.
wrapper (обёртка) - это функция, которая вызывает другую функцию, добавляя небольшой слой логики: проверку, подготовку аргументов, преобразование типов, логирование или обработку ошибки.

Почему x64 calling convention не равна старым x86 соглашениям

Если у человека есть опыт x86, он может принести вредные ожидания в x64. В 32-битном Windows-коде встречались cdecl, stdcall, fastcall, thiscall; аргументы часто шли через стек; важным вопросом было, кто очищает стек: caller или callee; SEH на x86 имела linked-list модель через FS:[0]. На x64 всё иначе. Стандартная Microsoft x64 convention более единая: первые аргументы через регистры, обязательная shadow space, stack alignment, table-based exception handling через .pdata/.xdata.

Это не значит, что x64 проще во всём. Он проще в разнообразии calling conventions, но глубже связан с unwind metadata. На x86 часто можно было пройти стек эвристически или опираться на frame pointer chains, хотя это тоже имело ограничения. На x64 Windows официально использует таблицы. Поэтому calling convention и unwind metadata становятся частью нормального существования функции, а не редким дополнением.

Отсюда важное правило: не анализировать Windows x64 как "x86 с большими регистрами". RCX перед call - это не просто случайный регистр, а первая аргументная позиция. 20h на стеке - это часто shadow space, а не четыре локальные переменные. Отсутствие push ebp/mov ebp,esp не означает отсутствие кадра. Наличие .pdata не означает обработчик исключения. UNWIND_INFO не является отладочной информацией, но критично для stack walk.

cdecl / stdcall / fastcall / thiscall (старые 32-битные соглашения о вызовах) - это семейство x86 calling conventions с разными правилами передачи аргументов и очистки стека. Их нельзя напрямую переносить на Microsoft x64, где действует единая базовая x64 convention.

frame pointer chain (цепочка frame pointer-ов) - это старый способ прохода по стеку через сохранённые EBP/RBP, когда каждый кадр ссылается на предыдущий. На x64 Windows unwind не обязан полагаться на такую цепочку, потому что использует .pdata/.xdata.

Как hand-written assembly должен дружить с calling convention

Если код генерирует MSVC, clang-cl или другой нормальный компилятор под Windows x64, большую часть ABI-дисциплины он берёт на себя. Но если человек пишет assembly вручную, ответственность переходит к нему. Он должен правильно принять аргументы, сохранить nonvolatile-регистры, обеспечить stack alignment, выделить shadow space перед собственными call-ами, корректно вернуть результат и создать правильную unwind information, если функция не является простой leaf-функцией.

В MASM для x64 существуют специальные средства вроде PROC FRAME и unwind directives. Они нужны не для красоты листинга. Они помогают assembler/linker сгенерировать .pdata и .xdata для функции. Если написать пролог вручную, но не описать его директивами, код может выполняться при обычном ret, но ломаться при исключении или stack walk. Это классическая ловушка: "у меня функция работает" не означает "моя функция корректна для Windows x64 runtime".

Особенно опасны функции, которые вызывают Windows API или могут находиться на стеке во время исключения. Почти любой реальный код может оказаться в stack trace: исключение может прилететь глубже, profiler может остановить поток, debugger может запросить call stack, crash dump analysis может идти через эту функцию. Если unwind metadata неверна, ошибка проявится не там, где она была создана, а позже, при анализе аварии.

MASM (Microsoft Macro Assembler, макроассемблер Microsoft) - это ассемблер Microsoft. В x64-режиме он поддерживает FRAME/unwind-директивы, которые позволяют генерировать .pdata/.xdata для hand-written assembly функций.

unwind directive (директива unwind-описания) - это указание assembler-у о том, какие действия пролога нужно записать в unwind metadata. Она связывает ручной ассемблерный код с table-driven unwind моделью Windows x64.

Проверка функции глазами static : три слоя соответствия

Для твоего static /dynamic эту тему удобно превратить в модель трёх слоёв. Первый слой - ABI expectation: что должно быть верно для корректной Windows x64 функции. Второй слой - code reality: что реально делает машинный код. Третий слой - metadata claim: что заявлено в .pdata/.xdata. Хороший анализ не должен смотреть только на один слой.

Например, ABI говорит: если callee меняет RBX, она должна восстановить его. Code reality показывает: в прологе есть push rbx, в теле RBX используется, в эпилоге есть pop rbx. Metadata claim говорит: UNWIND_INFO содержит UWOP_PUSH_NONVOL RBX. Всё согласовано. Это normal finding или отсутствие finding-а.

Другой пример: код делает push rbx, но UNWIND_INFO не содержит сохранения RBX. Обычный ret может работать, но при исключении из тела функции RtlVirtualUnwind не восстановит RBX caller-a. Это уже серьёзное несоответствие. Третий пример: UNWIND_INFO говорит, что сохранён R12, но в прологе нет соответствующего save. Тогда unwinder может читать чужой stack slot и записывать мусор в R12 при восстановлении контекста. Это тоже finding.

Четвёртый пример: функция имеет sub rsp, 20h и no calls, не трогает nonvolatile-регистры, но имеет .pdata. Это не обязательно ошибка. Компилятор мог сгенерировать metadata по своим причинам, функция может быть не такой leaf-like, как кажется, или внутри есть особенности. `static` должен оценивать severity аккуратно. Не всякая необычность - вредоносная аномалия. Но всякое противоречие между ABI-significant code actions и unwind metadata требует внимания.

ABI expectation (ожидание ABI) - это правило, которое должно выполняться для корректной функции: выравнивание стека, сохранение nonvolatile-регистров, наличие shadow space при вызовах, корректный возврат результата и так далее.
metadata claim (утверждение метаданных) - это то, что .pdata/.xdata официально сообщает Windows о функции: диапазон кода, адрес UNWIND_INFO, сохранённые регистры, stack allocation, frame pointer и flags.
severity (серьёзность finding-a) - это оценка важности обнаруженного несоответствия. В `static` /`dynamic` разные нарушения должны иметь разную тяжесть: от информационной странности до критической ошибки unwind.

Типовые ошибки понимания calling convention

Первая ошибка - думать, что calling convention нужна только компилятору. На самом деле она нужна всему бинарному миру: linker-у, debugger-у, exception runtime, decompiler-у, hand-written assembly, Windows API и твоему анализатору. Если нарушить convention, код может быть синтаксически корректным машинным кодом, но бинарно несовместимым с окружающим миром.

Вторая ошибка - считать shadow space локальными переменными. Иногда компилятор действительно может использовать home space для хранения аргументов или временных значений, но исходная причина её существования - ABI. Перед каждым call caller должен обеспечить 32 байта. Если этого не знать, stack layout превращается в хаос.

Третья ошибка - думать, что unwind восстанавливает все регистры. Нет. Он восстанавливает caller context согласно ABI. Volatile-регистры не гарантируются. Nonvolatile-регистры важны, потому что callee обещала их сохранить. Это объясняет, почему unwind metadata фокусируется на RBX/RBP/RSI/RDI/R12-R15 и части XMM-регистров, а не на каждом mov в функции.

Четвёртая ошибка - переносить x86 SEH и x86 stack habits на x64. На Windows x64 исключения и stack walking завязаны на .pdata/.xdata. Поэтому отсутствие frame pointer не означает невозможность stack trace, а наличие таблиц не означает C++ catch. Это разные слои.

Пятая ошибка - анализировать call site по двум инструкциям перед call. Оптимизированный код может готовить аргументы раньше, использовать регистры

повторно, строить значения через ветвления, использовать stack arguments, XMM-регистры и home slots. Нужен data-flow анализ, иначе аргументы будут восстановлены неправильно.

Почему эта тема завершает главу о функции перед переходом к реальному подъёму по стеку

Теперь можно увидеть, почему calling convention стоит именно здесь, после пролога, эпилога, UWOP-кодов, frame pointer и volatile/nonvolatile registers. Все эти темы являются частями одного механизма. Пролог реализует обязанности callee. Эпилог возвращает состояние caller-у. UWOP-коды описывают ABI-значимые действия пролога. Frame pointer нужен, когда стек нельзя удобно описывать только через RSP. Volatile/nonvolatile registers задают, какие значения должны пережить вызов. Calling convention связывает это в единый контракт.

Без calling convention RUNTIME_FUNCTION, UNWIND_INFO и UNWIND_CODE превращаются в набор структур. С calling convention становится понятно, почему они именно такие. BeginAddress/EndAddress нужны, чтобы найти функцию по ControlPc. UnwindInfoAddress нужен, чтобы найти описание кадра. UWOP_PUSH_NONVOL нужен, потому что callee-saved register должен быть восстановлен. UWOP_ALLOC_SMALL нужен, потому что RSP был изменён. UWOP_SET_FPREG нужен, потому что frame pointer стал базой кадра. Epilog rules нужны, потому что функция может быть прервана в момент выхода.

Следующая большая глава будет уже про то, как Windows реально поднимается по стеку. Там появятся RtlLookupFunctionEntry и RtlVirtualUnwind как действующие лица, а не просто названия API. Но теперь у нас есть основа: мы понимаем, что именно они пытаются восстановить. Они не восстанавливают "красивую картинку исходного кода". Они восстанавливают ABI-корректный caller context, используя table-driven metadata, построенную на правилах Microsoft x64 calling convention.

Главная мысль раздела: x64 calling convention - это не справочная таблица регистров. Это договор, который делает возможными совместимые вызовы, корректные прологи/эпилоги, сохранение nonvolatile-регистров, shadow space, stack alignment и table-driven unwind. Без этого договора .pdata и .xdata не имели бы устойчивого смысла.

Краткая карта связей для ручного разбора

Когда ты вручную смотришь функцию в Ghidra, IDA, x64dbg или WinDbg, полезно идти не хаотично, а по устойчивой карте. Сначала найди границы функции: по символам, .pdata, дизассемблеру или control flow. Затем посмотри вход: какие регистры используются до первого присваивания? RCX/RDX/R8/R9 могут быть параметрами; XMM0-XMM3 могут быть floating-point параметрами. Затем посмотри пролог: какие nonvolatile-регистры сохраняются, сколько стека выделяется, есть ли frame pointer. Потом сравни это с UNWIND_INFO. Затем смотри call sites внутри функции: выделена ли outbound area, как готовятся аргументы, где stack arguments, соблюдается ли alignment. Потом смотри эпилоги: восстанавливают ли они то, что пролог сохранил, и похожи ли на распознаваемые epilog patterns.

Эта карта не заменяет глубокий анализ, но защищает от грубых ошибок. Она заставляет видеть функцию как ABI-объект, а не как случайный набор инструкций. Для reverse

engineering это огромная разница. Машинный код становится не просто потоком opcodes, а поведением участника договора: где он принимает аргументы, где берёт ответственность за регистры, где выделяет стек, где вызывает других, где возвращает управление и где оставляет следы для Windows unwinder.

Что смотреть	Зачем смотреть	Связь с calling convention/unwind
RCX/RDX/R8/R9 перед call или на входе функции	Понять integer/pointer аргументы	Первые четыре аргументные позиции Microsoft x64
XMM0-XMM3	Понять floating-point аргументы и результаты	Floating-point часть calling convention
sub rsp, N / add rsp, N	Понять stack allocation и alignment	Описывается UWOP_ALLOC_* если относится к прологу
push/pop RBX/RBP/RSI/RDI/R12-R15	Понять сохранение callee-saved регистров	Должно согласоваться с UWOP_PUSH_NONVOL или SAVE_NONVOL
shadow space 20h	Не спутать home area с locals	Обязательная область caller-а перед вызовом
ret или tail jmp	Понять выход из функции	Эпилог должен быть распознаваемым для unwind
.pdata/.xdata	Проверить официальное описание для Windows	Table-driven unwind зависит от корректной metadata

Учебная проходка 1: простая функция без вызовов

Теперь полезно пройти несколько учебных ситуаций, потому что calling convention лучше всего закрепляется не списком правил, а повторным узнаванием одних и тех же ролей в разном коде. Начнём с простой функции, которая получает два integer-аргумента и возвращает сумму. Такая функция может вообще не трогать стек. Первый аргумент приходит в ECX/RCX, второй в EDX/RDX, результат возвращается в EAX/RAX. Если функция не вызывает другие функции, не сохраняет nonvolatile-регистры и не выделяет stack frame, она может быть leaf-like.

В такой функции почти нечего описывать для unwind. Если во время неё нужно подняться к caller-у, достаточно знать return address на вершине стека. Nonvolatile-регистры не менялись, RSP не был сдвинут прологом, frame pointer не устанавливался. Поэтому отсутствие .pdata для такой функции не является проблемой само по себе. Это нормальная экономия: если для восстановления caller context не нужно специальное описание, таблица может быть не нужна.

Но важный вывод не в том, что "маленькая функция без .pdata всегда нормальна". Важный вывод в критерии: unwind metadata нужна не по размеру функции, а по её действиям. Если маленькая функция вдруг делает sub rsp, 40h, сохраняет RBX или вызывает другую функцию, ситуация меняется. Размер кода не является достаточным доказательством. Смотри на ABI-significant actions.

```
; int add(int a, int b)
; a -> ECX, b -> EDX, result -> EAX
```

```
add_func:
lea eax, [ecx+edx]
ret

Что видно:
- аргументы пришли через register arguments;
- stack frame не создавался;
- nonvolatile-регистры не тронуты;
- обычный unwind может быть тривиальным: снять return address.
```

Учебная проходка 2: функция вызывает другую функцию

Теперь усложним картину. Функция не просто считает сама, а вызывает helper. Как только появляется call внутри функции, текущая функция сама становится caller-ом по отношению к helper. Значит, она должна обеспечить shadow space для helper-а, подготовить аргументы, сохранить свои значения, если они должны пережить вызов, и соблюсти stack alignment.

Именно здесь в дизассемблировании часто появляется sub rsp, 28h. Для новичка это выглядит как локальные переменные, хотя иногда локальных переменных нет вообще. Почему 28h? Потому что 20h - обязательная shadow space для вызываемой функции, а ещё 8 байт часто нужны для выравнивания стека после того, как call/return-address логика создаёт сдвиг. Точное значение зависит от контекста, но сама форма очень типична.

Если функция выделяет стек в прологе, это уже действие, значимое для unwind. RtlVirtualUnwind должен знать, что при восстановлении caller context нужно прибавить этот размер к RSP. Поэтому в UNWIND_INFO появится UWOP_ALLOC_SMALL или UWOP_ALLOC_LARGE. Обрати внимание: shadow space выделяется текущей функцией как caller для её будущих вызовов, но технически она является частью stack allocation текущего frame. Поэтому если выделение сделано в прологе через sub rsp, оно описывается как stack allocation текущей функции.

```
caller_func:
sub rsp, 28h ; место для outbound shadow space + alignment
mov ecx, 10 ; 1-й аргумент helper
mov edx, 20 ; 2-й аргумент helper
call helper
add eax, 1 ; result helper-a в EAX
add rsp, 28h
ret

UNWIND_INFO для такой функции должен описать sub rsp, 28h,
потому что исключение может произойти после выделения стека,
но до add rsp, 28h.
```

Учебная проходка 3: функция сохраняет RBX, потому что значение должно пережить call

Теперь представим, что функция получает важный указатель и хочет держать его через вызов helper. Можно держать указатель в RCX? До первого call - можно. Но RCX volatile. После call helper может затереть RCX. Значит, если значение должно пережить call, компилятор должен либо сохранить его в stack slot, либо переложить в nonvolatile-регистр, например RBX. Но если callee использует RBX, она обязана сохранить старый RBX caller-а.

Так появляется типичный пролог: push rbx, потом sub rsp, N, потом mov rbx, rcx или похожая инструкция. Push rbx - не логика бизнес-алгоритма. Это выполнение ABI-обязанности. Функция говорит: я хочу пользоваться RBX для своих целей, но caller имеет право ожидать прежний RBX после моего возврата, поэтому сначала сохраню его. В эпилоге будет pop rbx.

Для unwind push rbx чрезвычайно важен. Если исключение произойдёт в середине функции, обычный pop rbx не выполнится. Но caller context всё равно должен получить правильный RBX. Поэтому UNWIND_INFO обязан сообщить, что старый RBX лежит на стеке. Именно для этого существует UWOP_PUSH_NONVOL.

```
use_pointer:
push rbx ; сохранить caller-ов RBX
sub rsp, 20h ; shadow space для будущего call
mov rbx, rcx ; держим важный указатель в nonvolatile RBX
call helper ; helper может затереть volatile RCX/RDX/R8/R9/RAX
mov [rbx], eax ; RBX всё ещё должен быть нашим указателем
add rsp, 20h
pop rbx ; вернуть caller-ов RBX
ret

UNWIND_INFO должен описать:
- UWOP_PUSH_NONVOL RBX;
- UWOP_ALLOC_SMALL/ALLOC_LARGE для sub rsp, 20h.
```

Учебная проходка 4: C++ method call и скрытый this

В C++ нестатический метод выглядит в исходном коде так, будто он вызывается у объекта: obj.method(x). Но на машинном уровне метод всё равно является функцией, которой нужно передать указатель на объект. Этот скрытый указатель называется this pointer. В Microsoft x64 this обычно занимает первую integer/pointer аргументную позицию, то есть RCX. Поэтому огромное количество C++ вызовов перед call содержит подготовку RCX как адреса объекта.

Если метод принимает ещё один обычный параметр, он пойдёт во вторую позицию: RDX или XMM1 в зависимости от типа. Если параметров больше, подключатся R8, R9 и stack arguments. Для reverse engineer это значит, что RCX перед call не надо автоматически называть "первым видимым параметром". В C++ это часто hidden this. Тогда видимые параметры исходного кода сдвинуты: первый явный параметр метода будет второй ABI-позицией.

С unwind this напрямую не связан, потому что this - это аргумент, а не сохранение кадра. Но для понимания функции он важен. Если ты перепутаешь this с обычным числом, декомпиляция станет странной: обращения [rcx+offset] окажутся не полями объекта, а будто бы доступом по случайному указателю. Calling convention даёт ключ: RCX на входе метода - кандидат на объект.

```
; obj->method(value)

mov rcx, rbx ; RCX = this pointer, объект лежал в RBX
mov edx, 5 ; EDX = первый явный int-параметр
call SomeClass_method

Внутри метода:
; RCX часто используется как указатель на объект
mov eax, [rcx+10h] ; чтение поля объекта
```

Учебная проходка 5: вызов Windows API с большим числом параметров

Многие функции Windows API имеют больше четырёх параметров. Это хороший материал для тренировки calling convention. Первые четыре параметра уходят в RCX, RDX, R8, R9, а остальные записываются в стековую область после shadow space. Если функция принимает указатели, флаги, размеры, handle-ы, все они с точки зрения ABI являются integer/pointer values и используют general-purpose registers или stack slots.

Handle (дескриптор) - это значение, которым Windows обозначает объект ядра или системный ресурс: файл, процесс, поток, событие и так далее. В машинном коде handle обычно выглядит как pointer-sized integer. Поэтому он передаётся по тем же правилам, что указатель или 64-битное целое: через RCX/RDX/R8/R9 или стек.

Когда анализируешь API call, полезно сопоставить известную сигнатуру функции с регистрами и stack slots. Если импорт известен, это относительно просто. Если импорт скрыт или вызов идёт через function pointer, calling convention всё равно помогает: количество подготовленных аргументов, типы операций перед call, строки/буферы/флаги могут выдать назначение вызова.

handle (дескриптор) - это значение, через которое Windows-программа обращается к системному объекту: файлу, процессу, потоку, событию, ключу реестра и т.п. На x64 handle обычно передаётся как pointer-sized integer по обычным правилам calling convention.

function pointer (указатель на функцию) - это значение, содержащее адрес функции. Вызов через function pointer может выглядеть как call rax или call [mem], но подготовка аргументов всё равно следует calling convention.

```
; Условный API call с 6 аргументами:

sub rsp, 38h
mov rcx, arg1 ; 1-я позиция
mov rdx, arg2 ; 2-я позиция
mov r8, arg3 ; 3-я позиция
mov r9, arg4 ; 4-я позиция
mov [rsp+20h], arg5 ; 5-я позиция после shadow space
mov [rsp+28h], arg6 ; 6-я позиция
call qword ptr [__imp_SomeApi]
add rsp, 38h
```

Структуры и указатели: почему большой объект часто превращается в адрес

Не все параметры удобно передавать непосредственно в регистре. Pointer-sized значения легко помещаются в RCX/RDX/R8/R9. Но большие structures, массивы и сложные объекты часто передаются через адрес. В исходном C++ ты можешь видеть параметр как const STRUCT& или как указатель. В машинном коде это будет адрес структуры. Поэтому RCX может быть не самим объектом, а адресом, по которому лежит объект.

Это важно для понимания memory access. Если функция получила RCX и дальше делает mov eax, [rcx+14h], это часто означает чтение поля структуры/объекта. Если получила RDX и записывает [rdx], возможно, RDX - output buffer. Calling convention не говорит тебе имя структуры, но говорит, где искать первый уровень смысла: входные указатели находятся в аргументных регистрах.

Для unwind это снова косвенная связь. Передача большой структуры через pointer не требует отдельной unwind metadata. Но понимание параметров помогает не спутать stack frame функции с данными, на которые указывают аргументы. Новички иногда видят [rcx+20h] и думают, что это stack offset. Нет: если RCX - pointer argument, это offset внутри объекта или буфера, а не часть текущего стека.

structure / struct (структура) - это составной тип данных, состоящий из нескольких полей. На машинном уровне работа со структурой часто выглядит как обращения по адресу base+offset, например [rcx+10h].

output buffer (выходной буфер) - это область памяти, адрес которой передаётся функции, чтобы функция записала туда результат. В calling convention сам адрес буфера обычно передаётся как pointer argument.

Скрытый return buffer для сложных возвращаемых значений

Простые результаты удобно вернуть в RAX или XMM0. Но что делать, если функция возвращает большую структуру? Иногда ABI использует hidden return buffer pointer. Это скрытый указатель на память, которую подготовил caller, чтобы callee записала туда результат. Такой указатель занимает одну из аргументных позиций, часто первую. В исходном коде ты видишь return BigStruct, а в машинном коде callee получает адрес, куда надо записать результат.

Для reverse engineer это важная ловушка. Функция может выглядеть так, будто принимает дополнительный pointer-параметр в RCX, хотя в исходном коде такого параметра не было. На самом деле это может быть sret pointer - structure return pointer. Потом видимые аргументы сдвигаются на следующие позиции. Если не учитывать такие ABI-детали, сигнатура функции будет восстановлена неправильно.

С unwind hidden return buffer напрямую не связан, но связан с calling convention как ABI-реальностью. А наша задача - видеть машинный код именно как ABI-реальность, не как исходный код. В исходнике "возвращается структура". В бинаре "передан адрес буфера, callee пишет туда поля". Это разные уровни описания одного поведения.

hidden return buffer pointer / sret pointer (скрытый указатель буфера возврата) - это неявный аргумент, через который caller передаёт callee адрес памяти для записи большого возвращаемого значения. Он может занимать аргументный регистр и сдвигать видимые параметры.

Почему stack cleanup на x64 обычно не похож на x86 stdcall

В старом x86 stdcall callee могла очищать стек от аргументов через ret N. В cdecl это делал caller. На Windows x64 стандартная модель другая: caller управляет областью вызова, включая shadow space и stack arguments. Callee обычно возвращается обычным ret без immediate. Поэтому после call можно увидеть add rsp, N, если caller выделял область прямо перед вызовом, или не увидеть, если область была выделена заранее в прологе текущей функции.

Это важно для чтения дизассемблера. Если ты привык искать ret 10h или add esp, N как признак количества аргументов, на x64 такой подход ломается. Количество аргументов определяется не очисткой стека, а подготовкой RCX/RDX/R8/R9, stack stores и знанием сигнатуры. Shadow space присутствует даже при малом числе параметров, поэтому размер

выделения стека не равен напрямую количеству аргументов.

Для unwind это тоже меняет картину. На x64 восстановление RSP не основывается на предположении "callee сняла аргументы". Оно основывается на структуре кадра текущей функции и return address. Поэтому x64 unwind metadata описывает prolog allocation текущей функции, а не старую x86-style cleanup semantics.

stack cleanup (очистка стека после вызова) - это восстановление RSP после размещения аргументов/области вызова. В Microsoft x64 caller обычно отвечает за свою область вызова; callee не возвращается через ret N как типичный x86 stdcall.

ret N (return with immediate, возврат с добавочным снятием байтов) - это форма x86-инструкции возврата, которая после снятия return address дополнительно увеличивает stack pointer на N. Для стандартной Windows x64 convention это не обычная модель очистки аргументов.

Как calling convention влияет на сигнатуры, которые восстанавливает reverse engineer

Signature (сигнатура функции) - это описание того, какие параметры принимает функция и что возвращает. В исходном коде сигнатура очевидна: int f(int a, char* p). В бинаре сигнатуру надо восстановить. Calling convention даёт основу: какие регистры на входе читаются до определения, какие stack slots используются как входные, где возвращается результат, какие аргументы передаются дальше в другие функции.

Если функция на входе читает RCX и RDX, но никогда не читает R8/R9 и входные stack arguments, вероятно, у неё два основных параметра. Если она читает XMM0, возможно, есть floating-point параметр. Если она пишет в [rcx], RCX может быть pointer/output buffer. Если она читает [rcx+offset] много раз, RCX может быть pointer to struct или this pointer. Если после call используется RAX, результат integer/pointer. Если используется XMM0, результат floating-point.

Но нужно быть осторожным. Оптимизации могут удалить неиспользуемые параметры, inline может смешать роли, thunk может перекидывать аргументы, hidden return buffer может добавлять скрытый параметр, thiscall-like метод может иметь скрытый this. Поэтому calling convention - не магический оракул. Это грамматика. Она говорит, как читать предложение, но смысл предложения надо извлекать анализом data flow и контекста.

function signature (сигнатура функции) - это описание параметров и возвращаемого значения функции. В reverse engineering сигнатура восстанавливается по calling convention, data flow, типам обращений к памяти, импортам, символам и контексту вызовов.

inline / inlining (встраивание функции) - это оптимизация, при которой компилятор вставляет тело вызываемой функции прямо в caller, убирая отдельный call. Это усложняет восстановление границ функций и сигнатур.

Calling convention и exception handlers: почему обработчик тоже вызывается по правилам

Когда Windows находит language-specific handler, этот handler тоже является функцией. А значит, он тоже вызывается по calling convention. __C_specific_handler, C++ exception handler, personality-like runtime functions - всё это машинный код, которому нужно

передать параметры и получить результат. DispatcherContext, ExceptionRecord, ContextRecord и другие структуры не попадают в handler магически. Их адреса передаются согласно ABI.

Это важный мост к будущим главам. Пока мы говорили о вызовах обычных функций. Но exception dispatch тоже состоит из вызовов функций runtime-а и обработчиков. Разница в том, что эти вызовы инициирует не пользовательский call из твоего кода, а механизм обработки исключений. Но раз код выполняется на той же архитектуре и в той же ABI-среде, calling convention продолжает действовать.

Поэтому, когда мы дойдём до DispatcherContext и __C_specific_handler, нужно будет помнить: структуры exception handling имеют сложный смысл, но передаются они через обычные x64 calling convention mechanisms. Опять один и тот же фундамент соединяет уровни: ABI, runtime, SEH, unwind, compiler-generated metadata.

__C_specific_handler (специфический C-обработчик Microsoft для SEH на x64) - это runtime-функция, которую Windows/C runtime используют для обработки SEH-конструкций языка C на x64. Она получает данные через обычные ABI-механизмы и использует DispatcherContext/language-specific data.

DispatcherContext (контекст диспетчеризации исключения) - это структура, которую runtime передаёт handler-у, чтобы тот понимал текущую функцию, её scope data, establisher frame и другие данные, необходимые для обработки исключения.

Нормальная функция, ABI-нарушение и намеренная обфускация

В обычной программе нарушения calling convention чаще всего являются багами hand-written assembly, JIT-кода, hook-ов, trampolines или повреждённой metadata. Но в reverse engineering встречается и намеренная обфускация. Обфускатор может генерировать код, который выглядит как функция, но плохо совпадает с нормальными ABI-паттернами. Может использовать нестандартные переходы, manual stack switching, fake prologs, overlapping functions, повреждённые .pdata entries, ложные unwind records.

Однако важно не объявлять аномалией всё, что отличается от учебного примера. Оптимизированный MSVC тоже может генерировать непривычный код: shrink-wrapping, shared epilog blocks, tail calls, отсутствие frame pointer, сохранение регистров не тем способом, который ты ожидал. Поэтому static должен различать "редко, но легально" и "невозможно согласовать с ABI/unwind".

Критерий такой: можно ли объяснить поведение через Microsoft x64 calling convention и documented unwind rules? Если да, это может быть нормальный оптимизированный код. Если код требует от unwinder-а невозможного, если metadata указывает на несуществующие сохранения, если RSP меняется вне описуемых зон, если epilog не распознаваем, если nonvolatile-регистры портятся без восстановления, тогда это уже серьёзная проблема или намеренная техника.

obfuscation (обфускация, запутывание кода) - это намеренное усложнение анализа программы. В контексте unwind обфускация может пытаться ломать ожидания calling convention, .pdata/.xdata, prolog/epilog recognition и stack walking.

trampoline (трамплин, переходная вставка) - это небольшой фрагмент кода, который перехватывает или перенаправляет выполнение, часто используется в hooks, instrumentation,

hotpatching и dynamics. Trampoline должен соблюдать ABI, если через него проходят обычные вызовы.

Мини-чеклист для твоего будущего parser-a

В конце полезно сформулировать practical checklist. Parser не должен просто читать структуры. Он должен понимать, какие ABI-ожидания стоят за структурами. Если найден RUNTIME_FUNCTION, нужно проверить, что диапазон похож на кодовую область. Если найден UNWIND_INFO, нужно сопоставить операции с прологом. Если есть UWOP_SAVE_NONVOL, нужно проверить, что регистр действительно nonvolatile. Если код сохраняет volatile-регистр, это не обязательно unwind action. Если код сохраняет nonvolatile-регистр, но metadata молчит, это повод для finding-a.

Для call sites parser может отдельно помечать preparation pattern: записи в RCX/RDX/R8/R9, XMM0-XMM3, stack stores после shadow space. Это не unwind validation в строгом смысле, но полезно для reverse-engineering слоя static . Такой слой может показывать: "вероятные аргументы вызова", "outbound shadow space", "возможный this pointer", "stack argument candidate". Это делает инструмент не просто валидатором, а помощником чтения кода.

Для dynamic обратная задача: если ты генерируешь или трансформируешь код, нельзя ломать ABI. Любая вставка, wrapper, instrumentation stub, trampoline или encrypted-block dispatcher должна сохранять nonvolatile-регистры, поддерживать stack alignment, выделять shadow space перед call, корректно описывать unwind, если может оказаться в stack walk. Иначе защита будет ломать не только reverse engineer-a, но и саму программу.

Проверка	Нормальная причина	Опасный признак
Есть sub rsp в прологе	Stack frame / outbound area / alignment	Нет соответствующего UWOP_ALLOC_* при наличии .xdata
Сохраняется RBX/R12/etc.	Callee использует nonvolatile-регистр	Сохранение не описано или описан другой регистр
Используется RCX на входе	Первый аргумент или this pointer	Нельзя автоматически считать local variable
Запись [rsp+20h] перед call	5-й аргумент или home/outbound область	Неверная интерпретация как локальной переменной без контекста
jmp в конце функции	Tail call / epilog jmp	Прыжок до восстановления RSP/nonvolatile-регистров
Нет .pdata	Возможная простая leaf function	Функция явно меняет RSP/nonvolatile-регистры и всё равно неописана

Отладчик и crash dump: почему calling convention видна даже после аварии

Когда программа падает и ты открываешь crash dump, исходного "нормального" исполнения уже нет. Есть снимок памяти, регистров, потоков и модулей. Чтобы построить call stack, отладчик должен сделать несколько шагов unwind. На каждом шаге он пытается

понять, какая функция сейчас активна, где её caller, какой return address должен стать новым RIP, каким должен быть RSP, какие nonvolatile-регистры нужно восстановить.

Calling convention здесь присутствует незримо. Отладчик не просто идёт по случайным значениям, похожим на адреса. В Windows x64 он использует .pdata/.xdata и ожидает, что функции соблюдали ABI. Если функция сохранила RBX, metadata должна сказать, где лежит старый RBX. Если функция выделила стек, metadata должна сказать, сколько нужно прибавить к RSP. Если функция находится в эпилоге, код эпилога должен быть распознаваемым. И всё это имеет смысл только потому, что caller/callee договорились о правилах.

Когда unwind ломается, call stack часто выглядит характерно: после одной функции идут странные адреса, модули не совпадают, return address указывает в данные, WinDbg пишет предупреждения о невозможности unwind, декомпилятор теряет границы функций. Это не всегда вина отладчика. Часто это следствие повреждённого стека, неверной metadata, hand-written assembly без unwind info, JIT-кода без function table или намеренного dynamic -а.

crash dump (дамп аварии) - это сохранённый снимок состояния процесса или системы после сбоя: память, регистры, список модулей, потоки и другая диагностическая информация. Для анализа call stack в x64 dump критически важны calling convention и unwind metadata.
stack trace / call stack (трасса стека / стек вызовов) - это цепочка активных вызовов функций. На Windows x64 она строится через последовательные шаги unwind, а не простым поиском похожих адресов на стеке.

JIT-код и dynamic function tables: тот же ABI, но таблицы создаются во время работы

Не весь исполняемый код заранее лежит в PE-файле как обычная .text-секция с .pdata рядом. JIT (just-in-time compilation, компиляция во время выполнения) может создавать машинный код прямо в памяти процесса. Но если этот код хочет быть нормальным участником Windows x64 мира, он всё равно должен соблюдать calling convention. Он должен принимать аргументы в RCX/RDX/R8/R9, сохранять nonvolatile-регистры, держать stack alignment, выделять shadow space перед call-ами и обеспечивать unwind information.

Разница в том, что для такого кода статической .pdata в PE может не быть. Вместо этого runtime регистрирует dynamic function tables через механизмы вроде RtlAddFunctionTable или RtlInstallFunctionTableCallback. Эти таблицы сообщают Windows: вот диапазоны динамически созданного кода, вот соответствующие unwind records. То есть модель остаётся той же - table-driven unwind, - но источник таблицы меняется: не PE-файл на диске, а runtime-регистрация в памяти.

Для reverse engineer это важно: отсутствие записи в PE .pdata не всегда означает отсутствие unwind information вообще. Для JIT, runtime-generated code, некоторых dynamic s и виртуализаторов function table может быть добавлена динамически. Поэтому static уровня PE видит только статическую часть картины. Если задача - анализировать живой процесс, нужно учитывать dynamic function tables. Но ABI-логика всё равно та же: динамический код либо соблюдает calling convention, либо становится опасным для exception handling и stack walking.

JIT / just-in-time compilation (компиляция во время выполнения) - это генерация машинного кода прямо во время работы программы. Такой код не обязан иметь статическую .pdata в PE, но для корректного Windows x64 unwind должен зарегистрировать function tables и соблюдать ABI.

dynamic function table (динамическая таблица функций) - это таблица диапазонов кода и unwind-описаний, зарегистрированная во время выполнения, а не взятая из PE .pdata. Она нужна для JIT-кода, runtime-generated code и некоторых защитных механизмов.

Не путать ABI-уровень и исходный язык

Ещё одна важная привычка: не думать, что calling convention принадлежит C++ как языку. Она принадлежит бинарному уровню. Один и тот же ABI могут использовать C, C++, Rust, Zig, hand-written assembly, JIT runtime, системные библиотеки. Исходные языки разные, но когда они хотят вызывать друг друга в одном Windows x64 процессе, им нужен общий бинарный язык вызовов.

Например, C++ метод, C-функция WinAPI, Rust extern "C" функция и ассемблерный stub могут все встретиться на одной цепочке вызовов. Если они используют один ABI, то caller и callee понимают друг друга. Если один компонент ожидает другую convention, аргументы окажутся не там, стек будет восстановлен неправильно, nonvolatile-регистры могут быть испорчены. Поэтому FFI и interop почти всегда явно говорят о calling convention.

FFI (foreign function interface, интерфейс вызова чужого кода) - это слой, через который один язык вызывает код, написанный на другом языке или предоставленный внешней библиотекой. На уровне FFI calling convention становится юридическим документом: если объявить её неправильно, программа может скомпилироваться, но упасть во время выполнения или дать странные повреждения данных.

FFI / foreign function interface (интерфейс вызова внешнего кода) - это механизм, позволяющий одному языку вызывать функции, написанные на другом языке или лежащие во внешней бинарной библиотеке. Корректная calling convention здесь обязательна, иначе аргументы и стек будут интерпретированы неправильно.

interop / interoperability (межоперабельность, совместная работа разных компонентов) - это способность кода из разных языков, библиотек или рантаймов корректно вызывать друг друга на бинарном уровне. ABI и calling convention являются основой такой совместимости.

Почему dynamic не должен ломать calling convention ради "защиты"

В защитных системах иногда возникает соблазн: раз reverse engineer опирается на calling convention, можно специально её сломать, чтобы ему было труднее. Это опасная мысль. Да, dynamic может запутывать control flow, шифровать блоки, вставлять trampolines, виртуализировать участки кода. Но если защищённый код должен нормально работать в Windows-процессе, вызывать API, переживать исключения, давать диагностируемые crash dumps и не ломать чужие библиотеки, он не может безнаказанно разрушать ABI.

Правильная защита должна различать внешний ABI-контур и внутреннюю запутанную реализацию. На границах, где код взаимодействует с обычным миром - экспортируемые функции, Windows API calls, callbacks, exception handlers, thread entry points, hooks, trampolines - calling convention должна быть сохранена. Внутри закрытого

виртуализированного блока можно иметь свой внутренний протокол, но при входе и выходе нужно восстановить нормальное ABI-состояние.

Для unwind это особенно важно. Если dynamic вставляет dispatcher-функцию, которая может оказаться в call stack, она должна иметь корректную unwind metadata. Если trampoline сохраняет nonvolatile-регистры, это должно быть либо прозрачно для внешнего frame, либо описано. Если dynamic меняет RSP нестандартно и не даёт Windows способа восстановить frame, он ломает не только отладчик reverse engineer-а, но и обработку исключений реальной программы.

ABI boundary (граница ABI) - это место, где код взаимодействует с внешним бинарным миром: вызывается другой функцией, вызывает API, отдаёт callback, входит в runtime или возвращает управление. На такой границе calling convention должна соблюдаться особенно строго.

dispatcher (диспетчер выполнения) - это код, который перенаправляет или управляет выполнением других блоков, например в dynamic-е, virtual machine obfuscation или JIT runtime. Если dispatcher является реальной x64 функцией, для него действуют ABI и unwind rules.

Финальная связка: что именно должен помнить reverse engineer

После всех деталей можно сформулировать практическое ядро. Когда ты видишь call в Windows x64, перед тобой не просто переход. Перед тобой событие ABI. В нём есть caller, callee, register arguments, shadow space, stack arguments, return address, volatile-регистры, nonvolatile-регистры, возможный return value и последствия для stack frame. Когда ты видишь пролог, перед тобой не просто подготовка функции, а запись долгов callee перед caller-ом. Когда ты видишь .xdata, перед тобой не просто бинарная структура, а инструкция Windows, как восстановить эти долги при аварийном выходе.

Эта оптика сильно меняет чтение кода. Вместо "почему тут опять sub rsp, 28h" появляется понимание: функция готовит ABI-correct область для вызовов и alignment. Вместо "зачем push rbx" появляется понимание: callee хочет использовать nonvolatile-регистр и сохраняет promise перед caller-ом. Вместо "почему .pdata указывает в .xdata" появляется понимание: Windows должна не гадать, а точно восстановить caller context. Вместо "почему RCX так важен" появляется понимание: это первая позиция бинарного договора.

Именно поэтому calling convention стоит в фундаменте reverse engineering. Она не решает всю задачу анализа, но без неё почти все наблюдения распадаются. Дизассемблер показывает инструкции. Calling convention превращает инструкции в роли. Unwind metadata превращает часть этих ролей в официальные данные для Windows. A reverse engineer сравнивает то, что должно быть, с тем, что реально есть.

Переход к следующей главе

На этом глава о том, как функция описывается для unwind, получает логическое завершение. Мы начали с пролога и эпилога, потому что именно они меняют и восстанавливают форму кадра. Затем разобрали UNWIND_CODE и UWOP-операции, потому что через них действия пролога превращаются в компактную запись. Потом отдельно разобрали frame register и frame pointer, потому что не всякий кадр удобно понимать только через RSP. Потом volatile/nonvolatile registers, потому что unwind должен знать, какие значения callee обязана вернуть caller-у. И теперь calling convention

связывает всё это вместе.

Дальше начинается другой уровень. Мы уже знаем, какие данные существуют и почему они устроены именно так. Теперь нужно проследить, как Windows реально ими пользуется. Что происходит, когда есть текущий RIP? Как по нему находится RUNTIME_FUNCTION? Что делает RtlLookupFunctionEntry? Что делает RtlVirtualUnwind? Что такое EstablisherFrame? Почему DispatcherContext нужен language-specific handler-у? Это уже не описание формата, а исполнение механизма.

Глава 5. Как Windows реально поднимается по стеку

5.1. Как система по RIP находит запись функции

Мы закончили главу 4 на calling convention не случайно. До этого момента книга долго собирала детали: где в PE лежит Exception Directory, что такое .pdata, что такое .xdata, как устроен RUNTIME_FUNCTION, зачем нужен UNWIND_INFO, почему пролог и эпилог описываются отдельно, какие регистры callee обязана сохранять, почему shadow space не является случайной дырой на стеке. Всё это могло выглядеть как набор отдельных технических тем. Но теперь начинается место, где они превращаются в работающий механизм.

Предыдущие главы отвечали на вопрос: какие данные вообще существуют в файле и что они значат. Глава 5 отвечает на другой вопрос: как Windows использует эти данные во время выполнения программы. Это очень важный переход. Static view (статический взгляд) - это когда reverse engineer смотрит на PE-файл, секции, таблицы, RVA и дизассемблирование. Runtime view (взгляд времени выполнения) - это когда процесс уже загружен в память, у потока есть регистры, в RSP лежит текущий стек, в RIP находится конкретный адрес инструкции, и системе нужно прямо сейчас понять, к какой функции относится этот адрес.

Если сказать по-человечески, Windows во время unwind сначала должна ответить на простой, но фундаментальный вопрос: "где мы вообще находимся?" Не в смысле имени функции из исходного кода, потому что имени может не быть. Не в смысле символа из PDB, потому что PDB может отсутствовать. Не в смысле красивой функции, которую показала Ghidra, потому что дизассемблер тоже строит модель. Системе нужен строгий бинарный ответ: текущий RIP попадает в какой диапазон кода, и какая RUNTIME_FUNCTION-запись описывает этот диапазон.

Пока этот ответ не найден, нельзя правильно читать UNWIND_INFO. Нельзя уверенно восстановить RSP caller-a. Нельзя понять, какие nonvolatile-регистры надо вернуть из стека. Нельзя решить, есть ли у функции language-specific handler. Нельзя корректно построить stack trace. Поэтому поиск записи функции по RIP - это не маленькая техническая деталь перед настоящим unwind. Это входная дверь во весь механизм x64 unwind.

RIP (instruction pointer, указатель инструкции) - это регистр x86-64, который содержит адрес инструкции, с которой сейчас связано выполнение потока. В обычном разговоре говорят "текущий адрес выполнения". Для unwind RIP важен потому, что именно по нему система понимает, внутри какого диапазона кода находится поток.
PC / program counter (счётчик программы, адрес текущего управления) - более общее архитектурное название для регистра или значения, указывающего текущую точку выполнения. В документации Windows рядом с x64 часто встречается имя ControlPc: это не отдельный физический регистр, а значение адреса, по которому ищут соответствующую запись функции.
Function table entry (запись таблицы функций) - запись, описывающая один диапазон исполняемого кода. На Windows x64 такой записью обычно является структура RUNTIME_FUNCTION. Она не хранит имя функции и не содержит байты машинного кода; она связывает диапазон адресов с unwind-данными.

От красивой картинки функции к строгому диапазону адресов

Когда человек смотрит на дизассемблер, он привык видеть функции как отдельные блоки: имя, начало, конец, список basic blocks, псевдокод. Это удобная модель для анализа. Но процессор не исполняет "функции" в таком человеческом виде. Процессор исполняет инструкции по адресам. В какой-то момент RIP равен одному виртуальному адресу, потом другому, потом третьему. Call изменяет поток, ret возвращает, jmp прыгает. Сам по себе CPU не спрашивает: "а это тело функции или эпилог?" и "есть ли у этой функции обработчик исключения?".

Для Windows x64 функция как объект unwind-механизма появляется только тогда, когда существует диапазон адресов, описанный в function table. Это очень важная мысль. Function для компилятора, function для дизассемблера, function для PDB и function для unwinder-а - близкие, но не абсолютно одинаковые сущности. Компилятор знает исходную функцию. Дизассемблер пытается восстановить функцию по коду и метаданным. PDB может хранить символическое имя и границы. А unwinder-у нужен диапазон, по которому можно найти RUNTIME_FUNCTION и потом UNWIND_INFO.

Именно поэтому в этой главе мы будем говорить не "Windows ищет функцию по имени", а "Windows ищет запись функции по адресу". Имя вообще может не участвовать. Для обычной обработки исключения системе не нужно знать, что функция называлась MyClass::DoWork. Ей нужно знать, что RIP попал в диапазон [BeginAddress, EndAddress), а у этого диапазона есть UnwindInfoAddress. Название функции может быть полезно человеку в отладчике, но механизм unwind работает не на названиях.

Function range (диапазон функции) - полуформальное имя для интервала адресов машинного кода, который описывает одна запись RUNTIME_FUNCTION. В таблице это выражено двумя полями: BeginAddress и EndAddress.
Symbol name (символическое имя) - имя функции или объекта, которое может храниться в PDB или символической таблице. Для unwind оно не является обязательным. Unwind работает по адресам и метаданным, а не по красивым именам.
Ошибка новичка: думать, что если отладчик показывает имя функции, значит Windows нашла её по имени. В реальности имя обычно приходит из symbols/PDB, а unwind-механизм опирается на адреса, function table entries и unwind metadata.

Почему первым ключом становится именно RIP

Представь момент исключения. Поток выполнял инструкцию, и случилось событие: access violation, деление на ноль, RaiseException, breakpoint, guard page, что угодно. У системы есть context record. Context record (запись контекста) - это снимок важных регистров потока: RIP, RSP, RAX, RBX, RCX, XMM-регистры и другие поля, в зависимости от того, что именно сохраняется. Для нашей темы центральны RIP и RSP. RIP говорит, где остановилось выполнение. RSP говорит, где сейчас вершина стека.

Но одного RSP мало. Стек - это просто память. В нём лежат return address, сохранённые регистры, локальные переменные, shadow space, stack arguments, временные spill-значения, иногда security cookie, иногда выравнивающие пустоты. Без знания того, какая функция сейчас активна и как она построила свой stack frame, нельзя надёжно сказать: вот здесь сохранён RBX, вот здесь старый RSP, вот здесь return address caller-а.

Поэтому Windows начинает не с чтения "всего стека подряд", а с вопроса: какой код сейчас исполнялся?

Ответ на этот вопрос даёт RIP. Если RIP находится внутри диапазона, описанного RUNTIME_FUNCTION, система получает ключ к .xdata. Если RIP не найден в таблицах, система может предположить leaf function и применить упрощённую логику. Но в любом случае входной точкой является адрес текущего управления. В документации рядом с RtlLookupFunctionEntry используется параметр ControlPc. Для текущего кадра ControlPc обычно соответствует адресу выполнения из контекста. Для последующих кадров stack walker может работать уже с return address, и там появляются дополнительные тонкости, о которых мы поговорим отдельно.

Context record (запись контекста, структура CONTEXT) - структура Windows, в которой сохраняется состояние потока: значения регистров, включая RIP и RSP. Unwind изменяет эту структуру виртуально: не выполняет машинный ret, а вычисляет, каким был бы контекст caller-a.

ControlPc (control program counter, адрес управления для поиска функции) - адрес, по которому RtlLookupFunctionEntry ищет запись function table. На x64 это virtual address внутри функции или рядом с местом, где управление покинуло функцию. Важно: ControlPc - это значение для поиска, а не отдельный регистр CPU.

Когда мы говорим "по RIP", мы немного упрощаем язык. В API часто используется термин ControlPc. Для текущего кадра он связан с RIP из CONTEXT. Для уже разматываемых кадров источником ControlPc может быть return address. В ручном stack walking иногда применяют поправку на границу инструкции, но её нельзя слепо применять ко всем ситуациям.

Loaded image: почему PE-файл в памяти уже не просто файл

Чтобы искать запись по RIP, нужно понимать, что RIP является virtual address. Это адрес в виртуальном адресном пространстве процесса. Он не равен file offset. Он не равен просто RVA из PE. Он не равен номеру байта в файле на диске. Когда Windows loader загружает PE-файл, он создаёт loaded image: отображение образа в памяти процесса. Секции получают свои virtual addresses, базовый адрес модуля становится ImageBase, а RVA начинают превращаться в VA через сложение с этим ImageBase.

В статическом анализе ты часто смотришь на RVA: например, BeginAddress = 00012340. Это удобно, потому что RVA не зависит от того, куда модуль реально загрузился. Но во время выполнения RIP будет выглядеть как полный virtual address, например 00007FF6_40123456. Чтобы сравнить этот адрес с BeginAddress и EndAddress в RUNTIME_FUNCTION, нужно привести адреса к одному пространству. Обычно это делается так: из ControlPc вычитается ImageBase, и получается относительный адрес внутри образа, то есть RVA-like offset. После этого этот offset можно сравнивать с BeginAddress и EndAddress.

Именно поэтому функция поиска возвращает не только pointer на RUNTIME_FUNCTION, но и ImageBase. ImageBase нужен следующему шагу: поля RUNTIME_FUNCTION являются image-relative. Чтобы получить настоящий virtual address .xdata или диапазона функции, нужно снова прибавить ImageBase. Без ImageBase запись RUNTIME_FUNCTION остаётся неполной: она говорит "смещение такое-то от начала образа", но не говорит, где этот образ реально лежит в памяти процесса.

Loaded image (загруженный образ) - PE-модуль после отображения загрузчиком Windows в виртуальное адресное пространство процесса. Это уже не просто файл на диске: секции размещены по виртуальным адресам, релокации могли быть применены, импорт может быть разрешён, а ImageBase имеет конкретное runtime-значение.

Virtual address / VA (виртуальный адрес) - адрес в виртуальном адресном пространстве процесса. RIP содержит именно такой адрес.

RVA / relative virtual address (относительный виртуальный адрес) - смещение относительно ImageBase загруженного образа. В .pdata поля RUNTIME_FUNCTION хранятся как image-relative offsets, поэтому их надо интерпретировать относительно ImageBase.

ImageBase (базовый адрес образа) - virtual address, с которого загруженный PE-образ начинается в памяти. Чтобы получить VA из RVA, нужно сделать VA = ImageBase + RVA. Чтобы получить RVA из VA внутри данного модуля, нужно сделать RVA = VA - ImageBase.

```
Пример приведения адреса к RVA:

ImageBase = 00007FF6_40000000
ControlPc = 00007FF6_40123456

ControlRva = ControlPc - ImageBase
= 00000000_00123456

Теперь ControlRva можно сравнивать с BeginAddress/EndAddress
из записей RUNTIME_FUNCTION в .pdata.
```

Почему поиск начинается с модуля, а не сразу с .pdata

В простом учебном примере можно сказать: "берём .pdata и ищем там запись". Но во время выполнения всё немного богаче. В процессе может быть много модулей: main executable, DLL, system DLL, runtime libraries, plug-in modules. Один RIP должен относиться к какому-то диапазону адресов. Поэтому прежде чем сравнивать ControlPc с BeginAddress конкретной .pdata, нужно понять, к какому image/module этот ControlPc относится.

Module (модуль) - это загруженный исполняемый образ: EXE или DLL, у которого есть base address, размер, секции и metadata. Если ControlPc попал внутрь диапазона адресов конкретного модуля, тогда можно использовать его Exception Directory и .pdata. Если ControlPc не принадлежит обычному PE-модулю, это ещё не обязательно ошибка. Код может быть сгенерирован во время выполнения: JIT, runtime code generation, trampolines, защитные механизмы, shellcode-like regions в легитимных рантаймах. Для такого кода function table может быть зарегистрирована динамически.

Поэтому точнее говорить не "Windows ищет только в .pdata файла", а "Windows ищет в active function tables". Static .pdata обычного модуля - самый важный и привычный случай. Но active function tables могут включать dynamic function tables, зарегистрированные через специальные API. Это особенно важно для reverse engineering: статический анализ PE может увидеть только то, что лежит в файле, а runtime может иметь дополнительные таблицы, которых в файле не было.

Module (модуль) - загруженный EXE или DLL в адресном пространстве процесса. У модуля есть ImageBase, размер образа, секции и PE-метаданные.

Active function tables (активные таблицы функций) - совокупность таблиц, которые Windows учитывает при поиске function table entry для данного ControlPc. В обычном случае это

таблицы из .pdata загруженных PE32+ модулей; в особых случаях это также dynamic function tables, зарегистрированные во время выполнения.

Dynamic function table (динамическая таблица функций) - function table, добавленная во время выполнения, например для JIT-сгенерированного кода. Она не обязана физически существовать в .pdata исходного PE-файла, но может участвовать в runtime lookup.

Ошибка статического анализатора: считать, что вся runtime-картина unwind полностью исчерпывается секцией .pdata на диске. Для обычного PE это часто достаточно, но для JIT, dynamics, runtime code generation и некоторых системных механизмов могут существовать dynamic function tables.

RUNTIME_FUNCTION как ответ на вопрос "какая функция покрывает этот адрес"

Когда подходящий модуль или function table найден, начинается собственно поиск записи. Мы уже подробно разбирали RUNTIME_FUNCTION, но теперь посмотрим на неё глазами runtime-поиска. Для системы это запись вида: диапазон начинается здесь, заканчивается здесь, unwind-информация лежит там. Внутри RUNTIME_FUNCTION нет байтов пролога, нет инструкций, нет списка basic blocks, нет имени функции. Но для первого шага этого достаточно.

Смысл поиска такой: у нас есть ControlRva. Нужно найти такую запись, для которой BeginAddress <= ControlRva < EndAddress. Это называется попаданием в полуоткрытый интервал. Почему именно полуоткрытый? Потому что BeginAddress включён, а EndAddress уже не включён. Если функция занимает байты от 1000h до 1100h, то адрес 1000h принадлежит функции, адрес 10FFh принадлежит функции, а 1100h - уже первый адрес после неё. Это удобно для соседних диапазонов: одна функция может заканчиваться на 1100h, следующая начинаться на 1100h, и адрес 1100h не будет одновременно принадлежать обеим.

Для обычной корректной таблицы RUNTIME_FUNCTION записи отсортированы по BeginAddress. Это не эстетика, а условие быстрого поиска. Если таблица отсортирована, lookup может использовать binary search: не идти линейно по тысячам записей, а быстро отбрасывать половины диапазона. В больших модулях это важно, потому что функций может быть много, а stack walking может вызываться часто: исключения, отладчик, профилировщик, ETW, crash dump collection.

Half-open interval [BeginAddress, EndAddress) (полуоткрытый интервал) - диапазон, где начало включено, а конец исключён. Адрес принадлежит функции, если BeginAddress <= address < EndAddress. Такая форма устраняет неоднозначность на границах соседних функций.

Binary search (двоичный поиск) - алгоритм поиска по отсортированному массиву, который на каждом шаге отбрасывает половину оставшихся записей. Для .pdata это удобно, потому что RUNTIME_FUNCTION-записи отсортированы по BeginAddress.

```
Условие попадания ControlRva в запись RUNTIME_FUNCTION:

entry.BeginAddress <= ControlRva < entry.EndAddress

Если условие истинно:
FunctionEntry = &entry
ImageBase = base of module/table
следующий шаг = RtlVirtualUnwind(...)
```

```
Если условие ложно для всех записей:
FunctionEntry = NULL
возможна leaf-логика или ошибка/ограничение анализа
```

Небольшой числовой пример: от RIP до найденной записи

Сейчас полезно пройти весь путь на числах. Пусть есть загруженный модуль app.exe. Его ImageBase равен 00007FF6_40000000. Исключение произошло при RIP = 00007FF6_40123456. Это полный virtual address. Он красив для отладчика, но RUNTIME_FUNCTION хранит не такие полные адреса, а image-relative offsets. Значит сначала вычисляем ControlRva = 00123456.

Дальше смотрим на function table. В ней есть несколько записей. Поиск не обязан понимать, что первая функция называлась parse_header, вторая process_packet, третья cleanup. Таблица может вообще не иметь имён. Она просто содержит диапазоны. Если ControlRva попадает между BeginAddress и EndAddress одной из записей, мы нашли функцию в смысле unwind.

```
ImageBase = 00007FF6_40000000
ControlPc = 00007FF6_40123456
ControlRva = 00123456

Фрагмент .pdata / function table:

BeginAddress EndAddress UnwindInfoAddress
00122000 001226A0 00204000
001226A0 00123110 00204120
00123110 00123880 00204270 <- ControlRva 00123456 здесь
00123880 00124000 00204310

Результат lookup:
FunctionEntry = запись с Begin=00123110, End=00123880
ImageBase = 00007FF6_40000000
UnwindInfo VA = ImageBase + 00204270
= 00007FF6_40204270
```

Такой пример специально показывает, что RUNTIME_FUNCTION не обязан начинаться с красиво округлённого адреса вроде 1000h. Реальные функции имеют разные размеры, оптимизация меняет форму кода, а границы задаются компилятором и линкером через unwind metadata.

Почему нельзя просто искать ближайший предыдущий prolog

Иногда кажется, что запись функции можно было бы найти проще: взять RIP, пойти назад по коду до push rbp или sub rsp, ... и сказать: вот начало функции. Такой подход выглядит соблазнительно, если ты привык к учебным примерам. Но для Windows x64 он ненадёжен.

Во-первых, не каждая функция начинается с push rbp. На x64 frame pointer часто не используется. Пролог может начинаться с push rbx, sub rsp, mov [rsp+...], rsi, save XMM, stack probing, security cookie, или вообще быть очень коротким. Во-вторых, некоторые функции являются leaf functions и могут не иметь обычного unwind-описания. В-третьих, оптимизация может создавать tail calls, thunks, hot/cold splitting, общий epilog block, jump tables. В-четвёртых, байты машинного кода неоднозначны: если начать дизассемблировать не с той границы, можно получить правдоподобную, но ложную

последовательность инструкций.

Табличная модель решает эту проблему иначе. Она не пытается угадывать функцию из байтов. Компилятор и линкер заранее записали диапазоны функций в .pdata. Windows не должна анализировать машинный код как reverse engineer. Она должна быстро и детерминированно найти запись по адресу. Это одна из причин, почему x64-модель отличается от старой x86 SEH-модели: на x64 stack unwinding становится metadata-driven, то есть управляемым метаданными.

Metadata-driven unwind (раскрутка, управляемая метаданными) - подход, при котором система восстанавливает предыдущий кадр не через угадывание машинных инструкций, а через заранее созданные таблицы .pdata/.xdata.

Heuristic scan (эвристическое сканирование) - попытка угадать структуру функции по шаблонам байтов или инструкций. Для надёжного Windows x64 unwind это не основной механизм, потому что исключения и отладка требуют детерминированности.

Если твой анализатор строит unwind только по дизассемблированию и игнорирует .pdata/.xdata, он будет выглядеть умным на простых примерах и ломаться на оптимизированном, защищённом, JIT-сгенерированном и системном коде.

Static .pdata и runtime lookup: похожие, но не одинаковые задачи

В static /dynamic тебе придётся различать два режима мышления. Static parser читает PE-файл на диске. Он видит Data Directory, Exception Directory, raw sections, RVA, file offsets. Он может построить массив RUNTIME_FUNCTION, проверить сортировку, пересечения, выходы за границы секций, валидность UnwindInfoAddress. Но static parser не находится внутри живого процесса. У него нет настоящего RIP потока. У него есть только модель.

Runtime lookup работает в адресном пространстве процесса. Там уже есть ControlPc как VA. Там есть module base. Там могут быть dynamic function tables. Там могут быть изменения, появившиеся после загрузки. Поэтому статический аналог RtlLookupFunctionEntry в твоём проекте должен ясно писать: "я моделирую поиск по статически найденной таблице", а не притворяться, что видит всю runtime-картину Windows.

Это не делает static analysis бесполезным. Наоборот, он очень ценен. Большинство обычных PE32+ модулей имеют .pdata в файле, и её можно проверить заранее. Но честный инструмент должен понимать границы своего знания. Если он видит отсутствие записи для диапазона кода, это может быть finding. Но severity такого finding зависит от контекста: это leaf code, junk code, thunk, packed region, динамический код, ошибка линковки или попытка обфускации? Без runtime-контекста часть ответов будет вероятностной.

Static parser (статический парсер) - инструмент, который читает файл без выполнения программы. Он видит PE-структуры на диске и может строить модель .pdata/.xdata, но не видит dynamic function tables, добавленные во время работы процесса.

Runtime lookup (поиск во время выполнения) - поиск function table entry по реальному ControlPc в адресном пространстве процесса. Он учитывает loaded image и активные таблицы функций.

Active function tables как карта, которую Windows использует для unwind

Фраза active function tables звучит сухо, но её надо понять как карту. У Windows есть набор карт, которые говорят: для такого диапазона адресов существуют такие function entries. Обычный PE-модуль приносит свою карту через Exception Directory. JIT или другой runtime может зарегистрировать дополнительную карту. Когда появляется ControlPc, система ищет, в какую карту он попадает, а потом ищет конкретную запись внутри этой карты.

В простом PE-кейсе карта устроена понятно: Exception Directory указывает на массив RUNTIME_FUNCTION. Этот массив лежит обычно в .pdata. Каждая запись содержит три DWORD-поля. BeginAddress и EndAddress задают диапазон кода. UnwindInfoAddress ведёт в .xdata. Таблица отсортирована. Всё красиво.

В dynamic-кейсе карта может быть создана не линкером, а runtime-компонентом. Например, JIT сгенерировал машинный код в памяти. Если он хочет, чтобы stack walking, exception handling и profiling работали нормально, он должен сообщить Windows, как раскручивать этот код. Для этого существуют API семейства dynamic function tables. В этой главе мы только фиксируем факт: lookup не обязан ограничиваться .pdata на диске. Подробно dynamic function tables будут разобраны позже.

JIT (just-in-time compilation, компиляция во время выполнения) - механизм, при котором машинный код создаётся уже во время работы программы. Такой код может не иметь .pdata в исходном PE, поэтому unwind-метаданные для него регистрируются динамически.

RtlAddFunctionTable (добавить таблицу функций) - Windows API, позволяющий зарегистрировать массив RUNTIME_FUNCTION для dynamically generated code. Подробно мы разберём его позже, но уже здесь важно понимать: runtime lookup может учитывать не только статическую .pdata.

Как выглядит концептуальный алгоритм поиска

Теперь соберём картину в алгоритм. Это не попытка описать все внутренние структуры Windows, потому что внутренние реализации могут меняться. Нам нужен уровень, достаточный для reverse engineer-а и автора анализатора. На этом уровне процесс выглядит так.

Сначала есть ControlPc. Это полный virtual address. Система должна понять, принадлежит ли он какому-то зарегистрированному диапазону кода. Если принадлежит обычному loaded module, определяется ImageBase этого модуля. Затем ControlPc переводится в relative offset: ControlRva = ControlPc - ImageBase. После этого по отсортированной function table ищется запись, чей диапазон покрывает ControlRva. Если запись найдена, возвращается указатель на неё и ImageBase. Если запись не найдена, следующий уровень должен решить, можно ли трактовать текущий код как leaf function.

Главное: lookup сам по себе ещё не раскручивает стек. Он не восстанавливает RBX, не меняет RSP, не читает UNWIND_CODE. Он только отвечает на вопрос: "какая RUNTIME_FUNCTION-запись соответствует этому ControlPc?" Настоящая раскрутка начнётся в RtlVirtualUnwind. Но без lookup у RtlVirtualUnwind нет FunctionEntry, а значит нет входа к .xdata.

```
Концептуальный алгоритм:

input: ControlPc as virtual address

1. Найти active function table, диапазон которой может содержать ControlPc.
2. Определить ImageBase / base address для этой таблицы.
3. Вычислить ControlRva = ControlPc - ImageBase.
4. В отсортированном массиве RUNTIME_FUNCTION найти entry:
entry.BeginAddress <= ControlRva < entry.EndAddress
5. Если entry найден:
return FunctionEntry, ImageBase
6. Если entry не найден:
return NULL; caller должен применить leaf-логику или сообщить ограничение.
```

RtlLookupFunctionEntry: имя функции, которую мы скоро разберём отдельно

Следующий раздел по плану будет посвящён RtlLookupFunctionEntry отдельно. Сейчас нам нужно только разместить её в общей картине. RtlLookupFunctionEntry - это API Windows, который получает ControlPc и ищет соответствующую function table entry. То есть он выполняет тот самый переход от адреса выполнения к RUNTIME_FUNCTION.

Почему мы не разбираем её полностью прямо сейчас? Потому что этот раздел про общую идею поиска по RIP. Нам важно сначала закрепить мышление: ControlPc -> active function tables -> ImageBase -> RVA -> RUNTIME_FUNCTION. В следующем разделе мы уже отдельно разберём сигнатуру API, параметры ControlPc, ImageBase, HistoryTable, возвращаемый PRUNTIME_FUNCTION, NULL-случай, ошибки ручного вызова и то, как этот API связан с RtlVirtualUnwind.

Но уже сейчас надо зафиксировать одну вещь: RtlLookupFunctionEntry не возвращает имя функции. Она возвращает запись таблицы. Если человеку потом хочется увидеть имя, в дело вступают symbols, PDB, dbghelp, debugger UI. Но механический unwind не нуждается в имени. Это опять возвращает нас к главной идее: адреса и метаданные первичны, имена вторичны.

RtlLookupFunctionEntry (поиск записи функции в runtime-таблицах) - Windows API, которая по ControlPc ищет RUNTIME_FUNCTION/function table entry и возвращает её вместе с ImageBase. Это не unwind целиком, а подготовительный поиск записи, без которой RtlVirtualUnwind не сможет прочитать .xdata.

PRUNTIME_FUNCTION (pointer to RUNTIME_FUNCTION, указатель на запись RUNTIME_FUNCTION) - возвращаемый pointer на найденную запись function table. Сам pointer указывает на запись, а поля внутри записи всё равно являются image-relative значениями.

HistoryTable: зачем lookup может иметь память о прошлых поисках

В сигнатуре RtlLookupFunctionEntry есть параметр HistoryTable. На первый взгляд он кажется второстепенным, и многие учебные объяснения его просто пропускают. Но для фундаментального понимания полезно сказать, зачем он вообще существует. Stack walking часто делает не один поиск, а цепочку поисков: текущий frame, caller frame, caller caller frame и так далее. При каждом шаге появляется новый ControlPc. Часто эти адреса лежат в одних и тех же модулях или рядом по таблицам. Значит можно ускорять повторные lookup-операции.

HistoryTable (таблица истории) - это структура-кэш. Cache (кэш) - это маленькая память о недавних результатах, которая позволяет не повторять дорогой поиск полностью, если похожий запрос уже был недавно. В случае unwind это особенно полезно для stack walking, профилировщиков и отладчиков, потому что они могут многократно ходить по стеку и искать function entries в одних и тех же модулях.

Для статического static -а HistoryTable не является центральной структурой. Твой parser может искать по массиву сам, использовать бинарный поиск, строить свои индексы. Но термин нужно знать, потому что он встречается в API и в документации. Нельзя пугаться его и думать, что это часть .pdata или .xdata. Это не данные PE-файла. Это runtime-вспомогательная структура для ускорения lookup.

HistoryTable / UNWIND_HISTORY_TABLE (таблица истории unwind-поиска) - optional runtime cache, который помогает ускорять повторные поиски function table entries. Это не секция PE и не часть .pdata/.xdata, а вспомогательная структура, которую можно передавать в RtlLookupFunctionEntry.

Cache (кэш) - механизм запоминания недавних результатов, чтобы не выполнять один и тот же поиск заново. В контексте function lookup кэш полезен при многократном stack walking.

Что происходит, если запись найдена

Найденная запись RUNTIME_FUNCTION ещё не означает, что стек уже раскручен. Она означает, что система получила карту входа к unwind-информации. Дальше из FunctionEntry можно взять BeginAddress, EndAddress и UnwindInfoAddress. Begin/End помогают понимать диапазон функции и положение ControlPc. UnwindInfoAddress указывает на UNWIND_INFO в .xdata или аналогичной runtime-области. Теперь RtlVirtualUnwind сможет прочитать заголовок UNWIND_INFO, массив UNWIND_CODE, флаги обработчиков, chained info и применить правила восстановления context.

Именно поэтому lookup и unwind полезно разделять в голове. Lookup отвечает на вопрос "какая запись?". Virtual unwind отвечает на вопрос "как из текущего контекста получить контекст caller-а?". Если смешать эти этапы, легко начать думать, будто .pdata сама восстанавливает регистры. Нет. .pdata даёт RUNTIME_FUNCTION. RUNTIME_FUNCTION указывает на .xdata. .xdata содержит UNWIND_INFO. UNWIND_INFO содержит кодированное описание пролога и дополнительных данных. А RtlVirtualUnwind уже использует всё это для изменения CONTEXT.

Для reverse engineer-а найденная запись тоже ценна. Она подтверждает границы функции в смысле unwind. Она показывает, где лежит unwind-info. Она позволяет сравнить дизассемблированный пролог с UNWIND_CODE. Она помогает понять, есть ли handler flags. Она может дать evidence, что функция non-leaf, сохраняет nonvolatile-регистры, имеет frame pointer или chained metadata. Но всё это начинается с простого попадания ControlRva в диапазон Begin/End.

```
Цепочка после успешного lookup:

ControlPc
-> RtlLookupFunctionEntry(ControlPc)
-> FunctionEntry = RUNTIME_FUNCTION
-> ImageBase
-> UnwindInfoVA = ImageBase + FunctionEntry.UnwindInfoAddress
-> UNWIND_INFO
```

```
-> UNWIND_CODE[]
-> optional handler/chained data
-> RtlVirtualUnwind обновляет CONTEXT caller-a
```

Что происходит, если запись не найдена

NULL-результат lookup нужно понимать осторожно. Он не обязательно означает: "функции не существует". Он означает только: для данного ControlPc не найдена function table entry в активных таблицах. Это разные утверждения. Машинный код может существовать, исполняться и быть корректным, но не иметь отдельной RUNTIME_FUNCTION-записи. Самый классический случай - leaf function.

Leaf function (листовая функция) - функция, которая не вызывает другие функции и не требует сложной unwind-метаинформации. В упрощённой модели она не меняет nonvolatile-регистры и не строит сложный stack frame. Тогда caller можно восстановить, как будто функция просто собирается выполнить ret: взять return address с вершины стека, увеличить RSP на 8 и поставить RIP равным этому return address. Это называется simulated return, то есть виртуальная имитация возврата.

Но тут есть опасность. Если запись не найдена потому, что функция действительно leaf и не трогала важные вещи, всё нормально. Если запись не найдена потому, что метаданные повреждены, удалены, спрятаны защитой или не зарегистрированы для динамического кода, leaf-предположение может быть ложным. Тогда stack walk уйдёт не туда. Он может прочитать не return address, а локальную переменную или сохранённый регистр. Поэтому для анализатора NULL-случай не должен автоматически считаться "всё хорошо". Его нужно сопровождать контекстной проверкой.

Leaf function (листовая функция) - функция, которая в классическом смысле не вызывает другие функции и не требует полноценной unwind-записи, потому что не создаёт сложный кадр и не сохраняет nonvolatile state, требующий восстановления через .xdata.

Simulated return (виртуальная имитация возврата) - способ unwind для простого случая без function entry: система ведёт себя так, как будто выполняется ret, берёт return address со стека и увеличивает RSP на размер адреса.

```
Упрощённая leaf-логика, если FunctionEntry == NULL:

caller_rip = *(uint64_t*)RSP
caller_rsp = RSP + 8

Новый виртуальный контекст:
RIP = caller_rip
RSP = caller_rsp

Это безопасно только если текущая функция действительно не изменила стек
так, что return address больше не лежит на текущем RSP.
```

Опасная ошибка: "нет RUNTIME_FUNCTION - значит это точно leaf". В нормальном x64 unwind это допустимое предположение для настоящих leaf functions, но в reverse engineering отсутствие записи может быть следствием обфускации, повреждения таблиц, ручного кода, неверного парсинга Exception Directory или незарегистрированного dynamic code.

Граница функции и проблема return address

Теперь нужно аккуратно разобрать тонкость, которая часто ломает ручной stack walking: ControlPc не всегда ведёт себя одинаково для текущего кадра и для кадров выше. Когда исключение произошло внутри текущей функции, RIP обычно указывает на инструкцию, которая выполнялась или должна была выполняться. Это адрес внутри функции, и lookup должен найти запись по этому адресу.

Но когда мы поднялись на один кадр выше, новый RIP часто получается из return address. Return address - это адрес инструкции после call в caller-е. Он указывает не на сам call, а на следующую инструкцию. Обычно это всё ещё внутри caller-функции, и поиск работает. Но если call был последней инструкцией перед границей функции, или если return address равен EndAddress какой-то записи, может возникнуть граничная ситуация. Полуоткрытый интервал не включает EndAddress. Поэтому адрес ровно на границе уже не принадлежит предыдущему диапазону.

Из-за этого многие stack walking реализации для non-current frames используют идею "control PC is return address minus one". Смысл не в магическом байте, а в том, чтобы привязать return address к месту вызова внутри caller-а, а не к следующему диапазону. Но применять это правило нужно аккуратно. Для текущего exception context нельзя бездумно отнимать 1 от faulting RIP. Для уже разматываемых кадров такая поправка часто полезна. В этой книге мы фиксируем принцип: всегда понимать, является ли адрес текущим execution RIP или return address, полученным из стека.

Return address (адрес возврата) - адрес, который инструкция call кладёт на стек. Он указывает на место, куда управление должно вернуться после ret, обычно на инструкцию сразу после call.

Boundary case (граничный случай) - ситуация, когда адрес попадает ровно на начало или конец диапазона. Для .pdata это важно, потому что диапазоны полуоткрытые: EndAddress не включается.

В практическом stack walking часто важно различать "RIP текущего кадра" и "return address предыдущего шага". Их нельзя обрабатывать совершенно одинаково. Эта тонкость особенно важна, когда адрес лежит на границе функции или рядом с эпилогом.

RIP в прологе, теле и эпилоге: lookup один, интерпретация разная

Когда ControlPc попадает в диапазон функции, это ещё не говорит, где именно внутри функции находится выполнение. RIP может быть в прологе, в теле, в эпилоге, иногда в области chained layout или в необычной оптимизированной форме. Поиск RUNTIME_FUNCTION обычно даёт одну и ту же запись для всего диапазона. Но следующий этап, RtlVirtualUnwind, будет учитывать положение ControlPc внутри функции.

Если RIP находится в прологе, часть действий пролога могла ещё не выполниться. Например, UNWIND_INFO говорит, что функция сохраняет RBX и выделяет 80h байт стека, но исключение случилось после push rbx и до sub rsp, 80h. Тогда нельзя применять полный unwind тела функции, потому что stack allocation ещё не произошёл. Именно поэтому UNWIND_CODE содержит CodeOffset: чтобы понять, какие действия пролога уже были выполнены к данному моменту.

Если RIP находится в эпилоге, ситуация другая: функция уже начала разбирать свой кадр. Возможно, RSP уже восстановлен, часть nonvolatile-регистров уже загружена обратно, и ближайший ret почти произойдёт. Для эпилога Windows x64 использует отдельное

распознавание допустимых шаблонов. Но опять же, первым шагом всё равно было найти RUNTIME_FUNCTION по ControlPc. Lookup не решает всю задачу, но он открывает доступ к данным, без которых следующий этап невозможен.

Function body (тело функции) - основная часть функции между служебным входом и выходом. В контексте unwind это участок, где пролог уже считается выполненным, а эпилог ещё не начался.

Prolog state (состояние выполнения пролога) - понимание того, какие операции пролога уже успели выполниться к текущему CodeOffset. Это нужно для корректного unwind, если исключение произошло в середине пролога.

Epilog recognition (распознавание эпилога) - логика, с помощью которой Windows понимает, что RIP находится в допустимой последовательности выхода из функции, и применяет особую unwind-семантику.

Почему таблица должна быть отсортирована и что ломается при нарушении

Ранее мы уже говорили, что RUNTIME_FUNCTION-записи в обычной .pdata отсортированы. Теперь понятно, почему это не мелочь. Если lookup использует бинарный поиск, он предполагает порядок. Если порядок нарушен, поиск может не найти существующую запись или найти не ту. Даже если конкретная реализация в каком-то случае сможет пройти таблицу иначе, нарушение сортировки остаётся серьёзным признаком проблемной metadata.

Для нормального linker-generated PE записи должны идти по возрастанию BeginAddress. Кроме того, диапазоны не должны перекрываться обычным образом: если одна запись говорит [1000, 1100), а другая [1080, 1200), возникает неоднозначность. Какую запись выбрать для ControlRva=10A0? В реальной системе могут существовать сложные случаи вроде chained unwind info или hot/cold regions, но базовое правило для static validation остаётся: сортировка, непустые диапазоны, корректные границы, валидный UnwindInfoAddress.

Для static /dynamic это означает, что поиск по RIP не должен быть просто "нашёл что-то похожее". Он должен уметь сообщать evidence: таблица отсортирована или нет; найденная запись покрывает адрес строго по правилу [Begin, End); нет ли пересечений; не выходит ли EndAddress за executable section; указывает ли UnwindInfoAddress в читаемую область; можно ли интерпретировать найденный UNWIND_INFO. Чем лучше инструмент объясняет путь поиска, тем полезнее он на защите и в реальном анализе.

Overlap (перекрытие диапазонов) - ситуация, когда два function ranges покрывают один и тот же адрес. Для обычной function table это проблема, потому что lookup должен иметь однозначный ответ.

Invariant (инвариант) - условие, которое должно сохраняться для корректной структуры. Например, BeginAddress < EndAddress, таблица отсортирована, UnwindInfoAddress указывает в допустимую область.

Проверка	Что означает	Почему важно для lookup
BeginAddress < EndAddress	Диапазон функции не пустой	Иначе невозможно определить покрытие ControlPc
Сортировка по BeginAddress	Записи идут в порядке адресов	Позволяет корректный и быстрый binary search
Нет обычных перекрытий	Один адрес не описан двумя конфликтующими записями	Lookup должен вернуть однозначную FunctionEntry
UnwindInfoAddress валиден	Найденная запись ведёт к читаемой unwind metadata	Иначе lookup успешен, но следующий этап сломан
Begin/End в code region	Диапазон похож на исполняемый код	Иначе возможна повреждённая или обфусцированная таблица

Как lookup связан с отладчиком и stack trace

Обработка исключений - не единственное место, где нужен поиск записи по RIP. Отладчик, когда показывает call stack, тоже должен подниматься по кадрам. Crash dump analyzer, когда анализирует дамп после падения, тоже должен восстанавливать цепочку вызовов. Профилировщик, когда собирает sampled stack, тоже должен понимать, какие кадры находятся выше текущего. Во всех этих задачах повторяется один и тот же фундаментальный мотив: есть адрес управления, нужно найти function entry, потом выполнить unwind.

Если lookup работает, stack trace выглядит осмысленно. Ты видишь цепочку функций, даже если frame pointer omitted. Если lookup не работает, stack trace может оборваться. Отладчик может показать странные адреса, "??", неверные frames или вообще не подняться выше текущего участка. Именно поэтому unwind metadata важна не только для языка C++ или SEH. Она является инфраструктурой наблюдаемости программы.

В reverse engineering это даёт практический критерий. Если бинарь имеет странную .pdata, поломанную сортировку, missing function entries, некорректные UnwindInfoAddress, это влияет не только на исключения. Это влияет на то, как отладчик и инструменты видят программу. Иногда это просто ошибка генерации. Иногда следствие упаковки. Иногда сознательная антианализная техника. Иногда признак ручной сборки или JIT. Поэтому поиск по RIP является ещё и диагностическим инструментом: он показывает, насколько metadata согласована с кодом.

Stack trace / call stack (трасса стека / цепочка вызовов) - список кадров, показывающий, какие функции вызвали текущую. На x64 без frame pointer такой список часто строится через unwind metadata.
Crash dump analysis (анализ аварийного дампа) - разбор сохранённого состояния процесса после сбоя. Для восстановления цепочки вызовов анализатору нужны те же данные: RIP/ControlPc, function table entries и unwind metadata.

Почему PDB помогает человеку, но не заменяет .pdata

Отдельно нужно развести PDB и unwind. PDB может содержать имена функций, исходные строки, типы, локальные переменные, inline-информацию, иногда очень богатую

отладочную картину. Но PDB не является основой системного x64 unwind. Операционная система не должна зависеть от наличия PDB для обработки исключения. В production PDB может вообще отсутствовать. Тем не менее исключения, stack walking и unwinding должны работать.

Поэтому .pdata/.xdata - это runtime-critical metadata, а PDB - debug metadata. Debug metadata полезна человеку и отладчику для красивой интерпретации. Runtime-critical metadata нужна системе для механической корректности. Если есть PDB, но повреждена .pdata, имена могут быть красивыми, а unwind всё равно может ломаться. Если PDB нет, но .pdata/.xdata корректны, stack walking может быть механически правильным, хотя имена будут бедными или отсутствовать.

Для static /dynamic это принципиально. Нельзя строить проверку корректности unwind по PDB. PDB можно использовать как дополнительный слой: подписать функции, сравнить границы, улучшить отчёт. Но базовый анализ должен уметь работать без символов, потому что реальные reverse engineering задачи часто начинаются именно с stripped binary, где нет исходников и нормальных symbols.

PDB (program database, база отладочной информации) - файл Microsoft с debug symbols: именами, типами, строками исходного кода и другой отладочной информацией. PDB помогает человеку, но не является обязательной основой для x64 unwind.

Runtime-critical metadata (метаданные, критичные для выполнения) - данные, которые нужны системе во время работы программы, например .pdata/.xdata для unwind. В отличие от PDB, они должны присутствовать или быть зарегистрированы так, чтобы runtime мог ими пользоваться.

Как это видит reverse engineer в Ghidra, IDA и WinDbg

Инструменты reverse engineering используют разные источники правды. Ghidra и IDA дизассемблируют код, строят functions, анализируют cross-references, смотрят на символы, если они есть, и также могут использовать exception/unwind metadata. Для Windows x64 .pdata часто помогает найти функции, даже когда нет символов. Это особенно полезно в оптимизированных бинарях, где прологи не всегда выглядят учебно.

Но нужно помнить: функция в Ghidra и запись RUNTIME_FUNCTION не всегда совпадают один к одному идеально. Дизассемблер может объединить или разделить диапазоны по своей логике. Chained unwind info, hot/cold split, thunks и tail calls могут усложнить картину. Иногда .pdata показывает диапазон, который инструмент интерпретировал иначе. Это не значит автоматически, что один из них "глупый". Это значит, что у них разные задачи и разные модели.

WinDbg ближе к runtime-сценарию: он работает с адресами, символами, loaded modules, stack walking и unwind. Когда WinDbg строит стек, ему нужны function entries и unwind data. Если symbols есть, он покажет имена. Если symbols нет, он всё равно может подниматься по кадрам, если unwind metadata корректна. Поэтому, когда reverse engineer видит странный stack trace, надо проверять не только PDB, но и .pdata/.xdata.

Ghidra / IDA (дизассемблеры и платформы reverse engineering) - инструменты, которые строят модель функций из машинного кода, ссылок, символов и метаданных. Их модель полезна человеку, но не тождественна внутреннему runtime lookup Windows.

WinDbg (Windows Debugger) - отладчик Microsoft, который при stack walking использует loaded modules, symbols и unwind metadata. Symbols дают имена, unwind metadata даёт механическую возможность подняться по стеку.

static /dynamic : как превратить поиск по RIP в проверяемую логику

Для твоего проекта эту тему надо сразу переводить в архитектуру анализатора. Статический static может реализовать функцию, похожую по смыслу на RtlLookupFunctionEntry, но работающую не с живым процессом, а с моделью PE-файла. На вход она получает RVA или VA плюс сведения о модуле. На выходе она должна вернуть найденный RuntimeFunctionRecord или объяснить, почему запись не найдена.

Хороший инструмент не должен просто молча возвращать null. Он должен оставлять trace of reasoning - след рассуждения. Например: ControlPc такой-то; ImageBase такой-то; ControlRva такой-то; Exception Directory найден по RVA/Size; количество RUNTIME_FUNCTION-записей такое-то; таблица отсортирована; бинарный поиск проверил такие диапазоны; найден entry номер N; UnwindInfoAddress указывает туда-то; UNWIND_INFO прочитан или не прочитан. Такой отчёт ценен, потому что он превращает "магический анализатор" в объяснимый инструмент.

dynamic , если он будет модифицировать или проверять бинарь, должен понимать обратную сторону: любое изменение code ranges, перемещение функций, обфускация пролога, изменение секций или генерация новых кодовых областей может требовать согласования с unwind metadata. Нельзя "просто переписать код" и забыть про .pdata/.xdata, если функция non-leaf и должна корректно участвовать в exceptions/stack walking. В Windows x64 это не декоративные таблицы, а часть договора с операционной системой.

Trace of reasoning (след рассуждения анализатора) - объяснимый лог того, как инструмент пришёл к выводу: какие адреса вычислил, какую таблицу прочитал, какую запись нашёл, какие проверки прошли или провалились. Для ВКР это важно, потому что демонстрирует не только результат, но и метод.

Finding (обнаруженный факт/предупреждение анализатора) - формализованный результат проверки: например, "RUNTIME_FUNCTION table is unsorted", "ControlPc is not covered", "UnwindInfoAddress points outside readable section". Finding должен иметь evidence, а не быть голословным.

Псевдолог статического static -а:

```
Input VA: 00007FF6_40123456
ImageBase: 00007FF6_40000000
Computed RVA: 00123456
ExceptionDir: RVA=00300000, Size=00001200
Entries count: 0x1200 / 12 = 384
Table sorted: yes
Lookup result: entry[157]
Begin/End: 00123110..00123880
UnwindInfoRVA: 00204270
UnwindInfo read: ok
Result: ControlPc is covered by a valid RUNTIME_FUNCTION entry
```

Какие findings естественно рождаются из поиска по RIP

Когда механизм поиска понятен, становятся понятны и проверки. Если ControlPc не покрыт никакой записью, это может быть нормальный leaf case, а может быть проблема. Если таблица не отсортирована, binary search становится недостоверным. Если найденная запись имеет EndAddress меньше BeginAddress, диапазон невозможен. Если UnwindInfoAddress указывает за пределы readable section, следующий этап не сможет прочитать UNWIND_INFO. Если BeginAddress находится в non-executable section, стоит проверить, не ошибся ли parser в RVA/file offset mapping.

Важно, чтобы findings были не просто красными лампочками. Каждый finding должен объяснять, почему он связан с unwind. Например, "UnwindInfoAddress invalid" - это не абстрактная ошибка PE. Это значит: lookup может найти function range, но RtlVirtualUnwind не сможет получить инструкции восстановления кадра. "Unsorted .pdata" значит: поиск по ControlPc может быть недетерминированным или несовместимым с ожиданиями Windows. "Overlapping ranges" значит: один RIP может иметь две конкурирующие записи, а unwind требует однозначного FunctionEntry.

Такой язык делает отчёт зрелым. Он показывает, что анализатор понимает не только формат структур, но и runtime-смысл. Это как раз то, что отличает фундаментальный проект от простого "парсера PE-заголовков".

Finding	Evidence	Runtime-смысл
ControlPc not covered	Ни один диапазон [Begin, End) не содержит RVA	Unwinder не получает FunctionEntry; возможна leaf-логика или обрыв stack walk
Unsorted function table	BeginAddress нарушает порядок	Быстрый lookup может стать недостоверным
Overlapping ranges	Две записи покрывают один RVA	Нет однозначного ответа, какая FunctionEntry описывает RIP
Invalid unwind RVA	UnwindInfoAddress не мапится в допустимую область	Lookup успешен, но RtlVirtualUnwind не сможет прочитать metadata
Code range outside executable area	Begin/End не попадают в секцию кода	Возможна ошибка парсинга, упаковка, обфускация или нестандартная компоновка

Тонкость: запись функции не всегда равна исходной функции

В фундаментальном труде важно заранее убрать ещё одно неправильное ожидание. RUNTIME_FUNCTION entry не является идеальным отражением исходной функции из C/C++. Компилятор может трансформировать код. Он может inline-ить функции, то есть вставить тело одной функции внутрь другой. Он может вынести холодные ветки отдельно. Он может превратить маленькую функцию в thunk. Он может использовать tail call, при котором классического возврата из текущей функции уже не будет. Он может объединить эпилоги. Всё это влияет на то, как человек видит функцию.

Unwind metadata описывает не исходный текст, а машинный код, которому нужен корректный unwind. Поэтому запись RUNTIME_FUNCTION нужно читать как: "этот диапазон кода имеет такой способ восстановления предыдущего контекста". Не как: "это ровно одна функция из исходника". Обычно совпадение хорошее, но в сложных случаях оно может быть не таким прямым.

Для reverse engineer-а это полезно. Если .pdata показывает границы, которые не совпадают с твоей интуицией по дизассемблеру, это повод не ругаться на таблицу сразу, а проверить оптимизации. Может быть, это thunk. Может быть, tail call. Может быть, hot/cold split. Может быть, chained unwind info. Может быть, дизассемблер неверно угадал начало функции. А может быть, действительно metadata повреждена. Отличить эти варианты можно только тщательным анализом.

Inline / inlining (встраивание функции) - оптимизация, при которой компилятор вставляет тело вызываемой функции прямо в caller, убирая отдельный call. В результате исходная функция может не иметь отдельного runtime-диапазона в ожидаемом месте.
Thunk (переходная/прокладочная функция) - маленький участок кода, который часто просто перенаправляет управление, адаптирует calling convention, подставляет this pointer или выполняет служебный переход. Thunk может иметь необычную форму для обычного анализа функций.
Hot/cold splitting (разделение горячего и холодного кода) - оптимизация, при которой часто выполняемые участки функции и редко выполняемые ветки могут размещаться отдельно для лучшей производительности. Это усложняет простое представление "одна функция - один непрерывный кусок кода".

Мини-сценарий: access violation внутри non-leaf function

Теперь пройдём живой сценарий, но пока только до момента найденной записи. Пусть программа упала на access violation внутри функции process_message. Эта функция non-leaf: она сохраняла RBX, выделяла stack frame, вызывала другие функции. Исключение случилось в середине тела. Windows получила context record. В нём RIP указывает на инструкцию mov eax, dword ptr [rcx], где RCX оказался плохим указателем. RSP указывает на текущую вершину кадра process_message.

Система не начинает гадать по стеку. Она берёт ControlPc из контекста и ищет function table entry. Сначала определяется loaded module, которому принадлежит адрес. Затем вычисляется RVA. Затем по function table находится RUNTIME_FUNCTION. Теперь система знает: текущий RIP принадлежит диапазону process_message в смысле unwind metadata. По UnwindInfoAddress можно перейти в .xdata и прочитать UNWIND_INFO. Только после этого можно корректно понять, где сохранён RBX, сколько байт было вычтено из RSP и как вернуться к caller context.

Если бы lookup не нашёл запись, система могла бы попробовать leaf-логику. Но для non-leaf function это было бы опасно. RSP уже мог быть уменьшен на 80h, return address уже не лежал бы на текущем RSP, RBX был бы сохранён где-то глубже в кадре. Простая имитация ret прочитала бы не тот адрес. Вот почему наличие корректной RUNTIME_FUNCTION-записи для non-leaf code так важно.

```
Сценарий:

Exception: Access violation
Context.Rip = 00007FF6_40123456
Context.Rsp = 000000A1_BF7FEA80

Lookup:
ControlRva = Rip - ImageBase
Find entry where Begin <= ControlRva < End
```

```
Found:
BeginAddress = 00123110
EndAddress = 00123880
UnwindInfoAddress = 00204270

Следующий шаг:
RtlVirtualUnwind читает UNWIND_INFO и восстанавливает caller context.
```

Мини-сценарий: RIP попал в leaf function

Теперь другой сценарий. Есть маленькая leaf function, которая просто складывает пару значений или проверяет флаг. Она не вызывает другие функции, не сохраняет nonvolatile-регистры, не выделяет сложный stack frame. Для неё может не быть отдельной RUNTIME_FUNCTION-записи. Исключение или stack walk попадает в этот адрес. Lookup возвращает NULL.

В таком случае unwinder может трактовать кадр как leaf: текущая функция не сдвинула RSP так, чтобы return address ушёл с вершины стека. Значит caller context можно получить через simulated return. Это быстрый и простой путь. Он существует именно потому, что требовать .pdata-запись для каждой микроскопической функции было бы избыточно.

Но повторим: это не универсальное оправдание любых дыр в .pdata. Для хорошего анализа нужно смотреть на код. Если функция действительно не трогает RSP и nonvolatile state, NULL-случай выглядит разумно. Если код делает sub rsp, сохраняет RBX, вызывает другие функции, но записи нет, это уже подозрительно. Такой участок не должен спокойно списываться на leaf без evidence.

Как не перепутать VA, RVA и file offset в своём поиске

Одна из самых частых практических ошибок при реализации собственного lookup - смешение VA, RVA и file offset. В PE-анализе эти три числа постоянно превращаются друг в друга, и если сделать одно неверное сравнение, результат будет выглядеть почти правдоподобно, но окажется неправильным.

ControlPc в живом процессе - VA. BeginAddress и EndAddress в RUNTIME_FUNCTION - image-relative, то есть RVA-like значения. Raw file offset - позиция байта в файле на диске, которая нужна, когда ты читаешь содержимое секции из файла. Нельзя сравнивать ControlPc напрямую с BeginAddress, если ControlPc полный VA. Нельзя прибавлять ImageBase к file offset. Нельзя считать, что RVA всегда равен raw offset. Такое иногда случайно работает на простых файлах с совпадающими выравниваниями, но это ловушка.

Правильная дисциплина простая: сначала явно называй пространство каждого адреса. Если это VA - пиши VA. Если RVA - пиши RVA. Если file offset - пиши file offset. В отчётах static -а лучше выводить все три формы, когда это полезно. Тогда ошибки становятся видны глазами.

File offset (файловое смещение) - позиция байта внутри файла на диске. Это не VA и не RVA. Чтобы прочитать данные из PE-файла, часто нужно преобразовать RVA в file offset через таблицу секций.
Address space discipline (дисциплина адресных пространств) - привычка явно различать VA, RVA и file offset и не сравнивать их напрямую без преобразования. Для PE/unwind-парсинга это критически важно.

```
Неправильно:
if ControlPc >= entry.BeginAddress and ControlPc < entry.EndAddress:
found = entry

Потому что ControlPc может быть VA, а Begin/End - RVA.

Правильно:
ControlRva = ControlPc - ImageBase
if entry.BeginAddress <= ControlRva < entry.EndAddress:
found = entry
```

Почему этот раздел стоит перед RtlLookupFunctionEntry и RtlVirtualUnwind

Можно было бы сразу открыть документацию и разобрать сигнатуру RtlLookupFunctionEntry. Но тогда есть риск выучить API как набор параметров и не понять, зачем они нужны. Теперь сигнатура станет естественной. ControlPc - это адрес, по которому ищем. ImageBase нужен, потому что записи image-relative. HistoryTable нужен для ускорения повторных поисков. Возвращаемый PRUNTIME_FUNCTION нужен как вход в RtlVirtualUnwind.

После этого RtlVirtualUnwind тоже будет проще. Она не магически "знает функцию". Ей дают FunctionEntry, ImageBase, ControlPc и CONTEXT. FunctionEntry найден через lookup. ImageBase нужен для перевода RVA в VA. ControlPc нужен, чтобы понять положение внутри функции. CONTEXT нужен, потому что unwind изменяет виртуальное состояние регистров. То есть вся следующая глава механики строится на этом первом поиске.

Поэтому 5.1 - это не промежуточная страница. Это поворотный момент книги: мы перестаём просто читать структуры и начинаем смотреть, как Windows превращает один runtime-адрес в конкретную запись метаданных. Дальше каждый шаг будет продолжением этой цепочки.

Короткая формула раздела: RIP/ControlPc даёт адрес; ImageBase переводит его в пространство RVA; active function tables дают массив диапазонов; RUNTIME_FUNCTION даёт найденную запись; UnwindInfoAddress открывает дверь к .xdata; RtlVirtualUnwind делает настоящий подъём к caller context.

Контрольное резюме без потери глубины

Итак, когда Windows нужно подняться по стеку, она не начинает с угадывания по байтам машинного кода. На x64 нормальный путь metadata-driven. Система берёт ControlPc, то есть адрес текущего управления, определяет соответствующую активную таблицу функций, переводит адрес в относительную форму через ImageBase, ищет RUNTIME_FUNCTION-запись, чей диапазон покрывает этот адрес, и только потом переходит к .xdata.

Если запись найдена, это означает: для данного участка кода есть табличное описание, позволяющее следующему этапу понять пролог, сохранённые регистры, stack allocation, frame pointer, handler flags и chained info. Если запись не найдена, это не равносильно отсутствию кода. Это означает отсутствие найденной function table entry, после чего возможна leaf-логика или диагностический warning в анализаторе.

Для reverse engineering ключевой навык - уметь пройти этот путь руками: взять RIP или VA, найти ImageBase, получить RVA, проверить .pdata, найти диапазон, перейти к

UNWIND_INFO, сопоставить с дизассемблированным прологом. Для static /dynamic ключевая задача - автоматизировать этот путь так, чтобы инструмент не просто выдавал ответ, а объяснял каждое преобразование адреса и каждое предположение.

Что значит "активная" таблица: таблица должна быть не просто где-то в файле

Слово active в active function tables нужно воспринимать буквально. Для runtime-механизма важно не то, что где-то на диске лежит PE-файл с секцией .pdata, а то, что в текущем процессе существует таблица, которую Windows считает пригодной для поиска. Обычный загруженный модуль попадает в эту картину через loader: образ отображён в память, его PE-заголовки прочитаны, Exception Directory известна, и function table может участвовать в lookup. Динамически сгенерированный код должен быть явно зарегистрирован, иначе для Windows он может выглядеть как исполняемая память без unwind-карты.

Это различие особенно важно для защиты и упаковки. Packed binary (упакованный бинарь) может иметь маленький loader stub, который распаковывает настоящий код в память. На диске .pdata может описывать только stub или быть намеренно странной. Настоящий код может появиться позже. Если dynamic корректный и хочет поддерживать exceptions/stack walking, он должен позаботиться о runtime unwind metadata. Если dynamic антианализный, он может наоборот ломать или скрывать эту картину. Поэтому static static видит одну реальность, а runtime debugger может увидеть другую.

Active table - это не философское слово, а практическая граница ответственности. Если таблица не активна, RtlLookupFunctionEntry её не использует. Если таблица активна, она может влиять на stack walking даже тогда, когда static PE-parser её не видел. Поэтому в отчёте анализатора полезно разделять findings по типу: "static PE metadata finding" и "runtime function table hypothesis". Первое доказуемо из файла. Второе требует процесса, дампа или инструментирования.

Packed binary (упакованный бинарь) - исполняемый файл, в котором настоящий код может быть сжат, зашифрован или спрятан и распаковываться уже во время выполнения. Для unwind это опасно тем, что static .pdata может не описывать runtime-код полностью.
Loader stub (загрузочная заглушка упаковщика) - маленький участок кода, который стартует первым, подготавливает окружение и распаковывает/расшифровывает основной код. У stub-а может быть своя metadata, а у распакованного кода - отдельная runtime-картина.

Как поиск по RIP повторяется при stack walking

Один lookup - это только один шаг. Stack walking почти всегда является циклом. Сначала есть текущий context: RIP0, RSP0. Lookup по RIP0 находит FunctionEntry0. RtlVirtualUnwind использует её и вычисляет caller context: RIP1, RSP1. Теперь нужно снова выполнить lookup, но уже по RIP1 или ControlPc, полученному из нового контекста. Потом появится RIP2, RSP2, и процесс повторится. Так строится цепочка кадров.

Именно здесь становится понятно, почему lookup должен быть быстрым и почему HistoryTable вообще имеет смысл. Один crash dump может требовать раскрутки десятков потоков. Один profiler может собирать тысячи stack samples. Один отладчик при каждом шаге может пересчитывать стек. Если каждый раз линейно просматривать тысячи

функций во всех модулях, это было бы дорого. Отсортированные function tables, binary search и cache делают механизм пригодным для реальной системы.

Для reverse engineer-а цикл stack walking полезен как мысленная модель. Если stack trace оборвался на определённом кадре, нужно спросить: на каком именно шаге? Lookup не нашёл FunctionEntry? FunctionEntry найден, но UNWIND_INFO не читается? RtlVirtualUnwind неверно восстановила RSP? Return address оказался невалидным? Handler flags привели к особой логике? Без разделения этих этапов человек видит только "стек сломался", а с разделением можно поставить точный диагноз.

```
Цикл stack walking в упрощённом виде:

context = current thread CONTEXT

while context.Rip is valid:
functionEntry, imageBase = LookupFunctionEntry(context.Rip)

if functionEntry != NULL:
context = VirtualUnwind(imageBase, context.Rip, functionEntry, context)
else:
context.Rip = *(uint64_t*)context.Rsp
context.Rsp = context.Rsp + 8

print frame(context.Rip, context.Rsp)
```

Stack walker (механизм прохода по стеку) - код, который многократно применяет lookup и unwind, чтобы перейти от текущего кадра к caller-у, потом к его caller-у и так далее.

Frame (кадр стека) - состояние одного активного вызова функции: его участок стека, адрес возврата, сохранённые регистры и служебная информация. В x64 unwind frame восстанавливается через metadata, а не обязательно через цепочку RBP.

Почему "ближайший модуль" не всегда достаточно хорошее объяснение

В простом отладочном выводе адрес часто записывают как module+offset: например, app.exe+123456. Это удобно: сразу видно, что адрес лежит внутри app.exe, а offset равен 123456h. Но для lookup этого мало. Нужно не просто знать ближайший модуль, а убедиться, что адрес попадает в диапазон загруженного образа, что у этого образа есть Exception Directory, что размер directory кратен размеру RUNTIME_FUNCTION, что RVA записи корректно сравнивается с ControlRva.

Более того, адрес может лежать в memory region, которая не является обычным PE module. Например, JIT code heap, manually mapped image, private executable memory, trampoline page. В таких случаях формула module+offset может отсутствовать или быть искусственной. Если dynamic function table зарегистрирована, lookup всё равно может работать. Если не зарегистрирована - stack walking может применить leaf-логику или оборваться.

Manually mapped image (вручную отображённый образ) особенно интересен для reverse engineering. Если код загрузили не стандартным loader-ом, а своим загрузчиком, обычные списки модулей могут не знать о нём. PE-структуры могут быть частично стёрты. Exception Directory может существовать в памяти, но не быть зарегистрированной так, как ожидает Windows. Поэтому правильный анализ должен различать "адрес похож на лежащий

внутри PE" и "адрес участвует в active function tables".

module+offset (модуль плюс смещение) - удобная запись адреса как имени модуля и смещения от его ImageBase, например kernel32.dll+1A2B0. Это помогает человеку, но само по себе ещё не доказывает корректность unwind lookup.

Manually mapped image (вручную отображённый образ) - PE-образ, загруженный не стандартным Windows loader-ом, а пользовательским кодом. Такой образ может не участвовать в обычных списках модулей и не иметь корректно зарегистрированной unwind-картины.

Executable memory region (исполняемая область памяти) - участок памяти с правами выполнения. Наличие execute permissions не означает, что для него есть function table entry.

Как manual mapper и dynamic могут сломать поиск по RIP

Теперь посмотрим на антипример. Пусть dynamic переместил код функции в новую секцию или распаковал его в private memory. Машинный код выполняется. Call и ret физически работают. Но .pdata в исходном PE всё ещё описывает старые адреса или вообще не описывает новый регион. Когда произойдёт исключение внутри такого кода, ControlPc будет указывать на новый runtime-адрес. Lookup попытается найти active function table для этого адреса. Если dynamic не зарегистрировал соответствующую таблицу, FunctionEntry не найдётся.

Если код простой и leaf-like, это может случайно сойти с рук. Но если код non-leaf, сохраняет RBX/RBP/R12, выделяет stack frame, использует exception handlers, то отсутствие metadata становится реальной проблемой. Stack walking может оборваться, exception dispatch может не найти language-specific handler, crash dump станет менее полезным, profiler потеряет frames. Иногда dynamic -ы именно этого и хотят: ухудшить анализ. Но тогда это не бесплатная обфускация, а вмешательство в системный механизм.

Поэтому dynamic , который хочет быть корректным, должен относиться к unwind metadata как к части кода. Переместил диапазон - обнови или зарегистрируй function table. Изменил пролог - обнови UNWIND_CODE. Добавил сохранение nonvolatile-регистра - отрази это в .xdata. Иначе бинарь может работать на "обычном пути" и ломаться при исключениях, отладке или сборе дампов.

Антианализная ловушка: бинарь может запускаться и проходить простые тесты, но иметь поломанную unwind-картину. Ошибка проявится только при исключении, stack walking, crash dump analysis, профилировании или отладке.

Как проверять lookup руками на одном адресе

Для практики полезно иметь ручной ритуал проверки одного адреса. Он дисциплинирует мышление и помогает находить ошибки в своём парсере. Первый шаг: записать адрес, который проверяем, и явно указать, это VA или RVA. Второй шаг: определить ImageBase модуля. Третий шаг: если адрес был VA, получить RVA. Четвёртый шаг: найти Exception Directory и убедиться, что она корректно мапится в file offset. Пятый шаг: прочитать массив RUNTIME_FUNCTION. Шестой шаг: проверить сортировку. Седьмой шаг: найти диапазон, покрывающий RVA. Восьмой шаг: проверить UnwindInfoAddress.

Этот ритуал кажется длинным, но он спасает от хаоса. Большинство ошибок в PE/unwind-анализе возникает не потому, что человек не знает сложную теорию, а потому,

что он один раз сравнил VA с RVA или прочитал file offset как RVA. Если каждый шаг проговаривается, ошибка почти сразу становится видна.

В отчёте по ВКР такой пошаговый разбор будет выглядеть сильнее, чем просто "программа нашла функцию". Он показывает научную аккуратность: исходные данные, преобразования, критерий поиска, результат, проверка границ, вывод.

```
Ручной чек одного ControlPc:

1. ControlPc = ? ; VA или RVA?
2. ImageBase = ? ; base загруженного образа или preferred base для модели?
3. ControlRva = ControlPc - ImageBase, если ControlPc является VA
4. Exception Directory RVA/Size = ?
5. RuntimeFunctionCount = Size / sizeof(RUNTIME_FUNCTION)
6. Таблица отсортирована по BeginAddress?
7. Найдена запись Begin <= ControlRva < End?
8. UnwindInfoAddress мапится в readable section?
9. UNWIND_INFO имеет допустимую Version/Flags/CountOfCodes?
10. Пролог в дизассемблере согласуется с UNWIND_CODE?
```

Почему lookup не должен доверять только секционным именам

Ещё одна практическая ловушка - поиск .pdata по имени секции. В нормальном MSVC-бинаре Exception Directory часто указывает в секцию с именем .pdata. Но PE-формат в целом не обязывает анализатора искать данные только по имени секции. Автор бинаря может изменить имя секции. Линкер или packer может разместить данные в секции с другим именем. Важнее не строка ".pdata" в section header, а Data Directory entry для exception data.

Для lookup это означает: authoritative source для static table - Exception Directory, а не просто секция с красивым именем. Секция помогает проверить права, границы и file mapping. Но если parser говорит "я не нашёл .pdata, значит exception metadata нет", он может ошибиться. Правильнее: "я проверил IMAGE_DIRECTORY_ENTRY_EXCEPTION; он пуст/непуст; его RVA/Size мапятся туда-то".

Это особенно важно для dynamic s. Они могут переименовать секции, слить данные, изменить layout. Если твой static держится только за имя .pdata, его легко обмануть. Если он идёт от Data Directory, он ближе к тому, как Windows воспринимает PE metadata.

IMAGE_DIRECTORY_ENTRY_EXCEPTION (запись каталога данных исключений) - элемент Data Directory в PE Optional Header, который указывает на exception/function table data для архитектур, где такая таблица используется. Для Windows x64 именно он является важной точкой входа к .pdata-like данным.

Section name (имя секции) - строковое имя вроде .text, .rdata, .pdata. Оно удобно человеку, но не должно быть единственным источником истины для парсера.

Ошибка parser-a: искать unwind table только по section name ".pdata". Надёжнее начинать с Exception Directory и уже потом проверять, в какую секцию попал указанный RVA.

Lookup в дампе: когда файла на диске может быть мало

Crash dump добавляет ещё один слой. В дампе есть память процесса на момент сбоя, список модулей, контексты потоков, иногда части heaps и stacks. Анализатор дампа пытается сделать то же самое: взять RIP потока, найти модуль, найти function entry, выполнить unwind. Но файл на диске и память в дампе могут различаться. Модуль мог

быть загружен с релокацией. Код мог быть пропатчен. Dynamic function tables могли существовать в памяти. Не все страницы могли попасть в mini dump.

Поэтому crash dump analysis часто зависит от качества сохранённых данных. Если в дампе нет нужной памяти .pdata/.xdata или нет сведений о dynamic function tables, stack trace может быть неполным. Если symbols отсутствуют, имена будут бедными, но unwind всё ещё может работать, если metadata доступна. Если metadata повреждена, symbols не спасут механический подъём по стеку.

Для книги это важно потому, что показывает: lookup по RIP - не только теория PE-файла. Это реальный рабочий процесс в диагностике. Когда разработчик видит падение в production и открывает dump, его способность понять call stack часто опирается на те самые таблицы, которые мы сейчас разбираем.

Mini dump (минимальный дамп) - компактный crash dump, который содержит не всю память процесса, а выбранные данные: контексты потоков, список модулей, части стека и другую информацию. Если нужная unwind metadata не попала в дамп, stack walking может пострадать.

Почему важно объяснять lookup в отчёте, а не скрывать его за одной функцией

В программировании легко спрятать всё за функцией findRuntimeFunction(controlPc). Но в теоретическом труде и в ВКР этого мало. Нужно показать, что внутри этой функции происходит. Иначе комиссия или читатель видит только название и не понимает, где именно твоя работа: ты просто вызвал библиотеку, или сам реализовал корректную модель PE/unwind?

Хороший текст должен объяснять, что findRuntimeFunction не является магией. Она получает адрес, нормализует его к RVA, выбирает function table, проверяет инварианты, ищет диапазон, возвращает запись, прикладывает evidence. Тогда даже если реализация небольшая, за ней видна глубокая теория. Это особенно важно в reverse engineering: ценность инструмента не только в том, что он что-то нашёл, а в том, что он может объяснить, почему найденное соответствует формату и runtime-логике Windows.

Поэтому в дальнейшем, когда мы будем писать про RtlLookupFunctionEntry, RtlVirtualUnwind, EstablisherFrame и DispatcherContext, надо держать стиль: каждый API - это не просто функция, а закреплённый в коде этап большого процесса. Мы не запоминаем названия ради названий. Мы понимаем, какую проблему решает каждый этап.

Ещё один числовой пример: несколько модулей в процессе

Пусть процесс имеет три модуля: app.exe, engine.dll и ntdll.dll. У каждого свой ImageBase и свой диапазон виртуальных адресов. RIP равен 00007FFD_81234567. Нельзя вычитать ImageBase app.exe просто потому, что это главный EXE. Сначала надо определить, какому модулю принадлежит этот VA. Если адрес лежит внутри engine.dll, то ControlRva считается относительно engine.dll, а не относительно app.exe.

Это звучит очевидно, пока не начинаешь писать анализатор. В статическом режиме часто открыт один файл, и кажется, что ImageBase один. Но в runtime/дампе модулей много.

Если stack trace идёт через несколько DLL, каждый кадр может требовать lookup в своей таблице и со своим ImageBase. Поэтому ImageBase возвращается вместе с FunctionEntry: следующий этап должен знать, относительно какой базы интерпретировать UnwindInfoAddress.

```
Loaded modules:

app.exe ImageBase=00007FF6_40000000 Range=00007FF6_40000000..00007FF6_405A0000
engine.dll ImageBase=00007FFD_81000000 Range=00007FFD_81000000..00007FFD_81300000
ntdll.dll ImageBase=00007FFF_AA000000 Range=00007FFF_AA000000..00007FFF_AA250000

ControlPc = 00007FFD_81234567

Адрес принадлежит engine.dll.
ControlRva = 00007FFD_81234567 - 00007FFD_81000000
= 00000000_00234567

Искать нужно в function table engine.dll, а не app.exe.
```

Нормальная ошибка человека: перепутать найденную запись с уже восстановленным кадром

Ещё раз закрепим важное разделение, потому что оно часто теряется. Найти RUNTIME_FUNCTION - значит найти описание диапазона. Это не значит восстановить caller. Это как найти страницу инструкции, но ещё не выполнить инструкцию. RtlLookupFunctionEntry возвращает карту. RtlVirtualUnwind идёт по карте и вычисляет новый context.

В коде анализатора это должно отражаться архитектурно. Не стоит делать одну функцию, которая одновременно ищет entry, читает UNWIND_INFO, применяет все UWOP, проверяет handlers и печатает stack trace. Для учебного проекта лучше разделить pipeline. Тогда каждая стадия тестируется отдельно. Можно отдельно проверить RVA mapping. Отдельно сортировку .pdata. Отдельно lookup. Отдельно decoding UNWIND_INFO. Отдельно virtual unwind simulation. Это делает проект устойчивее и проще для объяснения.

Такой дизайн также совпадает с логикой Windows API. RtlLookupFunctionEntry и RtlVirtualUnwind - разные функции не случайно. Первая отвечает за поиск записи. Вторая отвечает за вычисление предыдущего контекста. Если в твоём проекте это разделение сохранено, архитектура будет выглядеть профессиональнее.

Pipeline (конвейер обработки) - разбиение анализа на последовательные этапы: чтение PE, маппинг RVA, поиск RUNTIME_FUNCTION, чтение UNWIND_INFO, моделирование unwind, формирование findings. Такой подход проще проверять и объяснять.

Предварительная схема главы 5

Этот раздел открыл главу 5, но дальше мы будем двигаться ещё глубже. Сейчас мы поняли, как из RIP получается FunctionEntry. Следующий раздел сфокусируется на RtlLookupFunctionEntry как на конкретной Windows-функции. После этого RtlVirtualUnwind покажет, как найденная запись превращается в новый CONTEXT. Затем EstablisherFrame объяснит, какой frame считается установившим обработчик. DispatcherContext покажет, какие служебные данные передаются language-specific handler-у.

Иначе говоря, глава 5 будет идти не как словарь API, а как движение одного исключения по системе. Сначала адрес. Потом поиск записи. Потом виртуальная раскрутка. Потом кадр обработчика. Потом контекст диспетчера. Такой порядок важен: каждый следующий термин появляется только тогда, когда без него уже нельзя объяснить следующий шаг.

Переход к следующему разделу

Теперь, когда общий механизм понятен, можно разбирать конкретный API. Следующий раздел будет посвящён RtlLookupFunctionEntry. Мы посмотрим на её сигнатуру не как на сухую строчку из winnt.h, а как на сжатое выражение всего только что разобранного процесса: ControlPc как входной адрес, ImageBase как базу найденного модуля, HistoryTable как ускоритель повторных поисков, возвращаемый PRUNTIME_FUNCTION как мост к RtlVirtualUnwind и NULL как отдельный смысловой случай, который нельзя трактовать слишком наивно.

5.2. Что делает RtlLookupFunctionEntry

В предыдущем разделе мы разобрали общую идею: у Windows есть текущий адрес выполнения, обычно связанный с RIP или ControlPc, и по этому адресу нужно найти запись функции. Теперь мы спускаемся на один уровень конкретнее. В Windows для этого существует функция RtlLookupFunctionEntry. Это не декоративное имя из документации и не удобная вспомогательная обёртка для учебника. Это один из центральных входов в табличную модель unwind на 64-битной Windows.

Если говорить совсем простым языком, RtlLookupFunctionEntry отвечает на вопрос: "какая запись function table описывает код по этому адресу?" Она берёт адрес ControlPc, ищет его в активных таблицах функций и, если находит подходящий диапазон, возвращает указатель на RUNTIME_FUNCTION. Одновременно она сообщает ImageBase того модуля или таблицы, к которому относится найденная запись. Это маленький результат по размеру, но огромный по смыслу: после него система уже может перейти к RtlVirtualUnwind и начать восстанавливать предыдущий кадр стека.

Важно сразу убрать ложное ожидание. RtlLookupFunctionEntry не делает сам unwind. Она не восстанавливает RSP. Она не читает все UWOP_* операции. Она не вызывает обработчик исключения. Она не строит красивый stack trace для человека. Она делает более раннюю и более узкую работу: превращает адрес выполнения в ссылку на запись таблицы функций. Но без этой узкой работы всё остальное не сможет начаться корректно.

RtlLookupFunctionEntry (lookup function entry, функция поиска записи функции) - WinAPI/NT runtime-функция, которая ищет в active function tables запись, соответствующую заданному PC value. На x64 такой записью обычно является RUNTIME_FUNCTION. Название можно читать буквально: lookup - найти, function entry - запись функции, а не саму функцию.
Entry (запись, элемент таблицы) - один элемент структурированной таблицы. В нашем контексте entry - это не вход в функцию и не entry point программы, а строка function table, где указаны BeginAddress, EndAddress и UnwindInfoAddress.
Lookup (поиск по ключу) - операция, где есть ключ и таблица, а результатом является найденная запись или отсутствие записи. Для RtlLookupFunctionEntry ключом является ControlPc, а таблицей - active function tables.

Почему эта функция заслуживает отдельного раздела

На первый взгляд RtlLookupFunctionEntry можно было бы объяснить одной строкой: она ищет RUNTIME_FUNCTION по адресу. Но для фундаментального понимания этого недостаточно. В этой одной строке спрятано несколько слоёв: что считать адресом, где искать, что значит "активная таблица", почему возвращается ImageBase, почему результат может быть NULL, почему NULL не всегда ошибка, почему нельзя слепо заменить эту функцию простым парсером .pdata, и почему для reverse engineering это место часто становится границей между статическим анализом и реальной картиной выполнения.

Мы уже видели .pdata как секцию или область, где лежат RUNTIME_FUNCTION-записи. Но RtlLookupFunctionEntry работает не с твоим файлом на диске, а с процессом во время

выполнения. Это разные миры. В файле ты можешь руками открыть Exception Directory, взять RVA и Size, пройти по массиву записей и сделать бинарный поиск. Это полезно и необходимо для твоего static . Но Windows во время выполнения смотрит на loaded images, на registered function tables и на то, какие таблицы сейчас действительно доступны для данного ControlPc.

Поэтому RtlLookupFunctionEntry является хорошей точкой, где нужно одновременно держать в голове два взгляда. Первый взгляд - static reverse engineering: мы читаем PE, .pdata, .xdata и делаем выводы по байтам. Второй взгляд - runtime mechanism: система находится внутри процесса, у неё есть virtual address, активные модули, динамические таблицы и текущий контекст потока. Если спутать эти взгляды, можно написать анализатор, который хорошо разбирает учебные EXE, но начинает ошибаться на JIT, packed binary, manual mapping или на коде, у которого runtime-картина не совпадает с картиной файла.

В этой главе мы не переписываем уже изученную .pdata. Мы смотрим на неё глазами Windows: не как на красивую секцию в PE-файле, а как на один из источников active function tables, которые участвуют в поиске RUNTIME_FUNCTION по ControlPc.

Прототип как карта смысла

Официальный прототип функции выглядит компактно. Но в нём почти каждое слово важно. В таких API нельзя читать только типы и думать, что смысл понятен автоматически. Тип DWORD64 говорит, что передаётся 64-битное число, но не говорит, является ли оно RVA, VA, file offset или просто произвольным числом. Тип PRUNTIME_FUNCTION говорит, что возвращается указатель на запись, но не говорит, откуда эта запись взялась. Поэтому прототип нужно читать медленно.

```
NTSYSAPI PRUNTIME_FUNCTION RtlLookupFunctionEntry(
DWORD64 ControlPc,
PDWORD64 ImageBase,
PUNWIND_HISTORY_TABLE HistoryTable
);
```

NTSYSAPI (NT system API declaration, объявление системной NT-функции) - макрос из Windows headers, который участвует в объявлении функций низкоуровневого Windows runtime. Для понимания unwind он не важен как механизм, но показывает, что мы имеем дело не с функцией языка C, а с системным API Windows.

PRUNTIME_FUNCTION (pointer to RUNTIME_FUNCTION, указатель на RUNTIME_FUNCTION) - тип результата. Если поиск успешен, возвращается адрес записи таблицы функций. Если поиск неуспешен, возвращается NULL. Это не копия записи и не адрес машинного кода функции, а указатель на метаданные.

DWORD64 (64-битное беззнаковое значение Windows-типа) - тип, которым здесь передаётся virtual address. Важно не путать размер типа со смыслом значения: ControlPc имеет 64-битный тип не потому, что это "любой большой номер", а потому что в x64 virtual address может быть 64-битным значением.

PDWORD64 (pointer to DWORD64, указатель на 64-битное значение) - выходной параметр. RtlLookupFunctionEntry записывает туда ImageBase найденного модуля или таблицы. Это нужно, потому что поля RUNTIME_FUNCTION image-relative и без ImageBase их нельзя правильно превратить в virtual addresses.

PUNWIND_HISTORY_TABLE (pointer to unwind history table, указатель на таблицу истории unwind-поиска) - необязательный параметр, связанный с ускорением повторных поисков. Для базового ручного понимания можно передавать NULL, но в настоящем stack walking повторные lookup-операции идут одна за другой, поэтому кэш/история может быть полезна.

ControlPc: адрес, по которому задаётся вопрос

Первый параметр - ControlPc. Его легко прочитать слишком быстро: "ну это RIP". В учебном упрощении это допустимо, но в серьёзной книге нужно точнее. ControlPc - это virtual address, по которому RtlLookupFunctionEntry должна найти функцию. Когда исключение произошло в текущем кадре, ControlPc действительно обычно связан с RIP из CONTEXT. Но когда стек уже разматывается и мы переходим к caller-кадрам, значение ControlPc может быть получено из return address предыдущего шага. Поэтому ControlPc лучше понимать не как физический регистр, а как адрес управления, для которого нужно найти function table entry.

Официальное описание говорит, что ControlPc - virtual address of an instruction bundle within the function. Для x64 в обычном чтении это означает адрес инструкции внутри функции. Важное слово здесь - within. Если адрес лежит внутри диапазона функции, lookup должен найти соответствующую запись. Если адрес попал ровно на EndAddress, запись уже не подходит, потому что диапазон полуоткрытый. Если адрес вообще не принадлежит ни одному зарегистрированному диапазону, функция вернёт NULL.

Для reverse engineer-а ControlPc опасен тем, что он выглядит как обычный адрес из отладчика. Хочется взять его, вычесть ImageBase и сразу сравнить с .pdata. Но ImageBase ещё нужно узнать. В простом случае можно определить модуль по адресу. В реальной системе это делает runtime. Именно поэтому ImageBase является выходным параметром, а не входным: RtlLookupFunctionEntry сама находит, к какому модулю или активной таблице относится найденная запись.

ControlPc (control program counter, адрес управления) - virtual address, для которого ищется запись function table. Это значение отвечает на вопрос "в какой функции находится или находилось управление?". Его нельзя путать с RVA и нельзя считать file offset.

Instruction bundle (пакет инструкции / участок инструкции в терминах документации) - формулировка из API-документации. Для практического x64-разбора достаточно понимать это как адрес внутри исполняемого участка функции, а не как адрес секции, файла или имени символа.

Within the function (внутри функции) - не человеческое "где-то рядом с функцией", а попадание в формальный диапазон RUNTIME_FUNCTION: BeginAddress <= ControlRva < EndAddress.

```
Логика параметра ControlPc в нормальном x64 случае:

1. Есть CONTEXT текущего кадра.
2. В CONTEXT.Rip лежит virtual address текущей точки выполнения.
3. Этот адрес передают как ControlPc.
4. RtlLookupFunctionEntry ищет RUNTIME_FUNCTION, чей диапазон покрывает ControlPc.
5. Если запись найдена, её передают в RtlVirtualUnwind.
```

Типичная ошибка: передать в RtlLookupFunctionEntry RVA вместо VA. Функция ждёт virtual address в адресном пространстве процесса. Если передать 00012345 вместо 00007FF6_4012345, lookup будет искать адрес около нуля, а не внутри загруженного модуля.

ImageBase: почему результат поиска состоит не только из FunctionEntry

Второй параметр - ImageBase. На первый взгляд странно: зачем функция поиска записи возвращает базовый адрес образа? Разве не достаточно самой RUNTIME_FUNCTION? Нет, недостаточно. RUNTIME_FUNCTION хранит BeginAddress, EndAddress и UnwindInfoAddress как image-relative values. То есть это не полные virtual addresses. Это смещения относительно начала образа. Если у тебя есть только сама запись, но нет ImageBase, ты не можешь правильно перейти ни к коду функции, ни к UNWIND_INFO.

Представь, что RUNTIME_FUNCTION говорит: BeginAddress = 00122000, EndAddress = 001226A0, UnwindInfoAddress = 00204000. Эти числа сами по себе не являются адресами процесса. Они начинают означать реальные virtual addresses только после прибавления ImageBase. Если ImageBase = 00007FF6_40000000, то начало функции будет 00007FF6_40122000. Если другой запуск загрузил модуль по другому адресу, полные VA изменятся, а RVA в .pdata останутся прежними.

Поэтому ImageBase - не лишняя справочная информация. Это половина результата. FunctionEntry отвечает: "вот запись, описывающая функцию". ImageBase отвечает: "вот база, относительно которой надо читать поля этой записи". Только вместе они дают пригодную для следующего шага картину.

ImageBase output (выходной ImageBase) - значение, которое RtlLookupFunctionEntry записывает в указанный пользователем DWORD64. Это base address того модуля или таблицы, к которой относится найденная RUNTIME_FUNCTION-запись.

Image-relative value (значение относительно образа) - число, которое нужно интерпретировать как смещение от ImageBase. В .pdata это принципиально: поля RUNTIME_FUNCTION не являются file offsets и не являются готовыми VA.

```
Если lookup вернул:

FunctionEntry->BeginAddress = 00122000
FunctionEntry->EndAddress = 001226A0
FunctionEntry->UnwindInfoAddress = 00204000
ImageBase = 00007FF6_40000000

Тогда реальные runtime-адреса:

FunctionBeginVA = ImageBase + BeginAddress = 00007FF6_40122000
FunctionEndVA = ImageBase + EndAddress = 00007FF6_401226A0
UnwindInfoVA = ImageBase + UnwindInfoAddress = 00007FF6_40204000
```

Для твоего `static` в offline-режиме ImageBase может быть выбран условно: например preferred ImageBase из PE или ноль для чистого RVA-анализа. Но при сравнении с runtime API нужно помнить: RtlLookupFunctionEntry работает с VA текущего процесса и возвращает фактический ImageBase загруженного образа.

HistoryTable: параметр, который легко переоценить и легко недооценить

Третий параметр - HistoryTable. Он редко нужен новичку для первого эксперимента. Во многих примерах его просто передают как NULL. Это нормально: смысл функции от этого не исчезает. Lookup всё равно может найти запись. Но игнорировать этот параметр полностью тоже не стоит, потому что он показывает важную практическую вещь: stack

walking - это не один поиск, а серия поисков.

Когда система строит цепочку кадров, она делает lookup для текущего ControlPc, затем RtlVirtualUnwind вычисляет контекст caller-а, потом берётся новый ControlPc, потом снова lookup, потом снова virtual unwind. В глубоком стеке это повторяется много раз. Если каждый раз заново искать модуль и таблицу без каких-либо подсказок, это может быть дороже. HistoryTable можно понимать как вспомогательную структуру истории/кэша, которая помогает runtime быстрее проходить повторяющиеся lookup-паттерны.

Но важно не сделать из HistoryTable магическую часть семантики unwind. Она не содержит "правильный ответ" о функции. Она не заменяет .pdata. Она не хранит UNWIND_INFO. Она не должна быть основой твоей модели данных в static . Для ручного разбора и базового использования API её можно передавать NULL. Для производительного stack walker-а можно учитывать, что такой параметр существует, но не надо строить выводы о корректности бинаря только по тому, использовалась история или нет.

UNWIND_HISTORY_TABLE (таблица истории unwind-поиска) - вспомогательная структура Windows, которую можно передать в RtlLookupFunctionEntry для повторных lookup-операций. Её практическая роль ближе к кэшу/подсказке, чем к источнику новых unwind-данных.

Cache (кэш) - временное хранилище уже найденных или часто используемых результатов, нужное для ускорения. Кэш не меняет смысл данных: если кэш пустой, правильный lookup всё равно должен опираться на active function tables.

Ошибка понимания: считать HistoryTable частью PE-метаданных. В PE-файле нет такой секции. Это runtime-параметр API, а не объект, который ты обязан найти рядом с .pdata или .xdata.

Что именно функция ищет: active function tables

Самое важное слово в описании RtlLookupFunctionEntry - active. Функция ищет не просто "в .pdata" и не просто "в таблице текущего EXE". Она ищет в active function tables. Это означает, что Windows учитывает те function tables, которые сейчас зарегистрированы и применимы в адресном пространстве процесса. У обычного PE32+ модуля такая таблица приходит из Exception Directory и секции .pdata. Но это только основной случай.

Active function tables включают статические таблицы загруженных модулей и могут включать динамические таблицы. Динамические таблицы нужны, когда код появился не как обычная секция .text загруженного PE, а был сгенерирован или зарегистрирован во время выполнения. Классический пример - JIT compiler. JIT создаёт машинный код в памяти. Если этот код может участвовать в исключениях, stack walking или отладке, операционная система должна знать, как его разматывать. Для этого runtime обязан предоставить function table через RtlAddFunctionTable или RtlInstallFunctionTableCallback.

Здесь появляется важный мост к reverse engineering. Если ты анализируешь только файл на диске, ты увидишь только статическую .pdata. Если программа во время выполнения добавляет dynamic function table, offline-парсер PE её не увидит. Это не значит, что твой анализатор бесполезен. Это значит, что в отчёте нужно разделять static evidence и runtime evidence. Static evidence говорит: "в файле такая-то Exception Directory и такие-то RUNTIME_FUNCTION entries". Runtime evidence мог бы дополнить: "во время выполнения зарегистрированы ещё такие-то tables".

Active function table (активная таблица функций) - таблица функций, которую Windows runtime реально учитывает при lookup. Она может происходить из .pdata загруженного PE32+ образа или быть зарегистрирована динамически.
Static function table (статическая таблица функций) - таблица RUNTIME_FUNCTION-записей, которая лежит в PE-образе и обычно попадает в .pdata через Exception Directory. Она создаётся компилятором/линкером до запуска программы.
Dynamic function table (динамическая таблица функций) - таблица, предоставленная системе во время выполнения. Она нужна для JIT, runtime-generated code и других случаев, когда исполняемый код не описан обычной .pdata файла.
JIT compiler (just-in-time compiler, компилятор во время выполнения) - runtime, который генерирует машинный код уже в процессе работы программы. Такой код тоже может нуждаться в unwind-метаданных, иначе stack walking и исключения становятся ненадёжными.

Почему RtlLookupFunctionEntry не является простым парсером PE

Полезно чётко разделить два действия. Первое действие: ты открываешь PE-файл, читаешь Data Directory, находишь Exception Directory, переводишь RVA в file offset, читаешь массив RUNTIME_FUNCTION и делаешь поиск. Это offline parsing. Второе действие: Windows внутри процесса получает ControlPc и вызывает RtlLookupFunctionEntry. Это runtime lookup. Похожие шаги есть в обоих случаях, но это не одно и то же.

Offline parser работает с байтами файла. Он должен сам решать, как переводить RVA в file offset, как проверять секции, как обрабатывать повреждённые таблицы, как не выйти за размер файла. Runtime lookup работает с уже загруженными образами и зарегистрированными таблицами. Он получает virtual address и возвращает pointer на запись, существующую в памяти процесса. В нормальном простом случае результаты совпадут. Но в сложных случаях различия станут важными: релокации, manual mapping, динамические таблицы, выгруженные модули, сгенерированный код, защитные механизмы.

Поэтому в книге мы не заменяем RtlLookupFunctionEntry своим парсером и не заменяем парсер вызовом RtlLookupFunctionEntry. Это разные инструменты. Парсер нужен для твоего static , потому что ты анализируешь PE как объект. Runtime API нужен для понимания того, как Windows действительно связывает ControlPc с function table entry во время выполнения.

Offline parsing (разбор без выполнения) - анализ файла или дампа как данных, без участия живого процесса и его runtime-состояния. В таком режиме ты сам читаешь PE-структуры и проверяешь их корректность.
Runtime lookup (поиск во время выполнения) - поиск, который делает Windows внутри адресного пространства процесса по реальному virtual address. Он может учитывать больше, чем просто байты PE-файла на диске.
Pointer (указатель) - значение адреса в памяти процесса. Возвращаемый PRUNTIME_FUNCTION является указателем на запись в памяти, а не смещением в файле. Это принципиально для различия runtime lookup и offline parsing.

Ошибка reverse engineer-а: увидеть pointer, вернуть его как "RVA записи" и дальше считать его file offset. Pointer из RtlLookupFunctionEntry существует в адресном пространстве процесса. Для offline-отчёта его нужно переводить и нормализовать отдельно.

Успешный результат: что у нас появляется после lookup

Если RtlLookupFunctionEntry нашла запись, у нас появляется три важные вещи. Первая - сам FunctionEntry, то есть pointer на RUNTIME_FUNCTION. Вторая - ImageBase. Третья - уверенность, что для данного ControlPc существует формальное описание функции в active function table. Эти вещи вместе меняют состояние анализа: до lookup у нас был просто адрес, после lookup у нас есть структурированная запись о функции.

Теперь можно сделать следующий шаг. RUNTIME_FUNCTION указывает диапазон кода и адрес UNWIND_INFO. ImageBase позволяет превратить image-relative поля в реальные адреса. ControlPc говорит, где именно внутри функции находится выполнение. С этими данными RtlVirtualUnwind сможет решить, находится ли ControlPc в прологе, эпилоге или теле функции, какие unwind codes применять, как восстановить RSP и как вернуть сохранённые nonvolatile-регистры.

Но даже успешный lookup не означает, что всё остальное автоматически корректно. Запись может указывать на повреждённый UNWIND_INFO. Диапазоны могут быть странными. UnwindInfoAddress может выходить за читаемую область. Flags могут конфликтовать с layout. Поэтому lookup - это не конец анализа, а переход от вопроса "есть ли запись?" к вопросу "можно ли этой записи доверять и как её интерпретировать?".

FunctionEntry (запись функции, найденная запись таблицы функций) - результат успешного lookup. Обычно это RUNTIME_FUNCTION, в которой есть BeginAddress, EndAddress и UnwindInfoAddress.

Successful lookup (успешный поиск) - ситуация, когда ControlPc попал в один из зарегистрированных диапазонов функции, и RtlLookupFunctionEntry вернула ненулевой PRUNTIME_FUNCTION.

```
Типичный successful path:

DWORD64 imageBase = 0;
PRUNTIME_FUNCTION fn = RtlLookupFunctionEntry(controlPc, &imageBase, history);

if (fn != NULL) {
// Есть запись функции.
// fn->BeginAddress / EndAddress / UnwindInfoAddress - image-relative.
// imageBase нужен для перехода к VA.
// Следующий смысловой шаг - RtlVirtualUnwind.
}
```

NULL: отсутствие записи не равно автоматической катастрофе

Если RtlLookupFunctionEntry не находит подходящую запись, она возвращает NULL. Это место нужно понимать спокойно, без паники и без наивности. NULL означает: в active function tables нет записи, которая соответствует указанному PC value. Но причина этого может быть разной.

Первая нормальная причина - leaf function. Leaf function - функция-лист, которая не вызывает другие функции и не делает действий, требующих полноценной unwind-информации. В x64-модели такие функции могут не иметь

RUNTIME_FUNCTION-записи. Если функция не изменила RSP сложным образом и не сохранила nonvolatile-регистры, unwinder может восстановить caller context простым способом: взять return address со стека, как будто выполняется обычный ret. Это не полноценное чтение UNWIND_INFO, потому что UNWIND_INFO нет.

Вторая причина - адрес действительно не относится к обычному коду с зарегистрированной таблицей. Это может быть ошибка адреса, повреждённый контекст, прыжок в данные, уже выгруженный модуль, испорченный стек, shellcode-like region или код, который runtime сгенерировал, но не зарегистрировал function table. В таком случае NULL уже может означать проблему для надёжного stack walking.

Третья причина - ограничение твоего взгляда. Если ты сравниваешь результат offline-парсера и runtime API, можно увидеть расхождение не потому, что Windows ошиблась, а потому что ты учёл не все таблицы или неправильно определил ImageBase. Поэтому NULL надо анализировать в контексте: где находится ControlPc, принадлежит ли он исполняемой памяти, есть ли модуль, есть ли .pdata, является ли функция leaf, не используется ли dynamic function table.

NULL (нулевой указатель, отсутствие результата) - специальное значение указателя, означающее, что RtlLookupFunctionEntry не нашла function table entry для заданного ControlPc. NULL не доказывает вредоносность и не доказывает ошибку сам по себе.

Leaf function (функция-лист, функция без собственного сложного кадра) - функция, которая в контексте unwind может быть восстановлена без полноценной RUNTIME_FUNCTION-записи. Обычно она не вызывает другие функции и не требует сохранения сложного состояния кадра.

Trivial unwind (тривиальная раскрутка) - простейшее восстановление caller-а, когда система берёт return address из стека и имитирует обычный выход из leaf function. Это работает только при условиях, когда функция не изменила стек так, что требуется UNWIND_INFO.

```
Типичный NULL path в stack walker-е:

fn = RtlLookupFunctionEntry(controlPc, &imageBase, history);

if (fn == NULL) {
// Нет записи в function table.
// Возможный нормальный случай: leaf function.
// Возможный проблемный случай: код без unwind metadata.
// Следующий шаг зависит от контекста и политики stack walker-а.
}
```

Ошибка анализа: писать finding "missing unwind metadata" для каждого NULL. Сначала нужно проверить, может ли это быть leaf function, попадает ли адрес в исполняемый диапазон, есть ли признаки изменения RSP, есть ли dynamic table и не произошла ли ошибка адресного пространства.

Граница между lookup и virtual unwind

RtlLookupFunctionEntry и RtlVirtualUnwind часто идут рядом, поэтому их легко спутать. Но их роли разные. RtlLookupFunctionEntry отвечает на вопрос "какая запись описывает функцию по этому адресу?". RtlVirtualUnwind отвечает на вопрос "как из текущего контекста получить контекст вызывающей функции?". Первая функция ищет метаданные. Вторая применяет метаданные.

Можно представить это как работу с картой. RtlLookupFunctionEntry находит нужную страницу карты: вот район, вот диапазон, вот ссылка на правила прохода. RtlVirtualUnwind уже идёт по этой карте: читает UNWIND_INFO, смотрит на пролог/эпилог, восстанавливает RSP, возвращает RIP caller-а и при необходимости сообщает handler information. Если первой страницы нет, вторая либо не может работать обычным способом, либо переходит в leaf-логику.

Именно поэтому в правильном stack walking loop сначала делается lookup, потом virtual unwind. Нельзя начинать с RtlVirtualUnwind без FunctionEntry, если функция не leaf и требует unwind-данных. Нельзя считать, что найденная RUNTIME_FUNCTION сама уже восстановила стек. Она только дала адрес инструкции для следующего механизма.

RtlVirtualUnwind (virtual unwind, виртуальная раскрутка кадра) - функция, которая по ImageBase, ControlPc, FunctionEntry и ContextRecord вычисляет контекст caller-а. Она не ищет запись функции сама как основную работу; FunctionEntry обычно получают через RtlLookupFunctionEntry.

Apply metadata (применить метаданные) - использовать уже найденные описания, чтобы восстановить состояние. Lookup находит описание, virtual unwind применяет его к ContextRecord.

```
Упрощённая цепочка Windows x64 stack walking:

while (Context.Rip != 0) {
FunctionEntry = RtlLookupFunctionEntry(Context.Rip, &ImageBase, HistoryTable);

if (FunctionEntry != NULL) {
Handler = RtlVirtualUnwind(
HandlerType,
ImageBase,
Context.Rip,
FunctionEntry,
&Context,
&HandlerData,
&EstablisherFrame,
ContextPointers);
} else {
// Leaf-like path: имитация return через [RSP].
Context.Rip = *(DWORD64*)Context.Rsp;
Context.Rsp += 8;
}
}
```

Этот псевдокод нужен для понимания идеи, а не как готовый production stack walker. В настоящем коде есть проверки доступности памяти, границ стека, каноничности адресов, обработки epilog/prolog cases, ограничение числа кадров и защита от циклов.

Как lookup связан с Exception Directory и .pdata

Возникает естественный вопрос: если RtlLookupFunctionEntry ищет в active function tables, где физически лежит нужная информация для обычного PE? Ответ: для статического x64 PE32+ она приходит из Exception Directory. В PE Optional Header есть Data Directory. Один из элементов этой директории указывает на Exception Directory. Для x64 этот каталог обычно указывает на массив RUNTIME_FUNCTION-записей, который размещают в .pdata.

Loader загружает модуль, и эта информация становится частью runtime-картины. Для lookup важны уже не file offsets, а адреса загруженного образа.

RUNTIME_FUNCTION-записи остаются image-relative, но их можно интерпретировать через ImageBase. Если ControlPc принадлежит этому модулю, lookup может найти соответствующую запись в таблице.

Важно: имя секции .pdata удобно для человека, но для строгого анализа главным является Exception Directory. Некоторые бинарные инструменты могут переименовывать секции или размещать данные нестандартно. Windows интересует не красивое имя секции, а корректные directory/table данные. Поэтому static не должен искать unwind только по строке имени .pdata. Он должен идти от PE Data Directory и затем проверять, куда указывает RVA и какой диапазон покрывает Size.

Exception Directory (каталог исключений PE) - элемент Data Directory, который для x64 указывает на таблицу RUNTIME_FUNCTION-записей. Это официальный путь к function table в PE, а не просто поиск секции по имени .pdata.
.pdata (procedure data section, секция процедурных данных) - типичное место размещения RUNTIME_FUNCTION-записей в PE32+ x64. Но имя секции не должно заменять анализ Data Directory.
Data Directory (каталог данных PE) - массив указателей в Optional Header PE, где разные элементы указывают на важные таблицы образа: imports, exports, resources, exceptions и другие.

Как можно эмулировать RtlLookupFunctionEntry в offline-анализаторе

Для твоего static полезно уметь делать собственный lookup по PE-файлу. Это не будет полным аналогом RtlLookupFunctionEntry, потому что offline-анализатор не знает всех runtime-таблиц живого процесса. Но для статического PE это основной и нужный механизм: взять ControlRva или интересующий RVA, пройти по RUNTIME_FUNCTION-массиву и найти запись, чей диапазон покрывает адрес.

Алгоритм начинается не с .pdata-name, а с Exception Directory. Нужно прочитать RVA и Size. Затем проверить, что Size кратен размеру RUNTIME_FUNCTION, то есть 12 байтам. Дальше нужно перевести RVA директории в file offset через section table, убедиться, что весь диапазон таблицы доступен в файле или образе, прочитать записи и проверить базовые инварианты: BeginAddress < EndAddress, записи отсортированы, диапазоны не пересекаются в нормальном случае, UnwindInfoAddress указывает на доступную область unwind metadata.

После этого можно делать lookup. Если таблица отсортирована, используется binary search. Если таблица не отсортирована, это уже finding. Можно всё равно сделать линейный поиск для диагностики, но нельзя делать вид, что таблица нормальна. Несортированная .pdata может ломать ожидания runtime-поиска и является серьёзным сигналом для анализатора.

```
Offline lookup по статической .pdata, концептуально:

input: image, controlRva

exceptionDir = OptionalHeader.DataDirectory[IMAGE_DIRECTORY_ENTRY_EXCEPTION]
if exceptionDir.Size == 0:
return NOT_FOUND_NO_EXCEPTION_DIRECTORY
```

```
entries = parse_runtime_function_array(exceptionDir.RVA, exceptionDir.Size)
validate(entries)

lo = 0
hi = entries.count - 1
while lo <= hi:
mid = (lo + hi) / 2
e = entries[mid]

if controlRva < e.BeginAddress:
hi = mid - 1
else if controlRva >= e.EndAddress:
lo = mid + 1
else:
return e

return NOT_FOUND
```

Emulation (эмуляция поведения) - воспроизведение части логики другой системы. Offline lookup эмулирует статическую часть RtlLookupFunctionEntry, но не становится полной заменой runtime API.

Invariant (инвариант, условие корректности) - правило, которое должно сохраняться для нормальной структуры. Например, BeginAddress должен быть меньше EndAddress, a table size должен быть кратен 12 байтам.

Finding (диагностический вывод анализатора) - структурированное сообщение о подозрительном или некорректном свойстве: код, severity, адрес, доказательство, объяснение и возможные mitigations.

Что lookup не обязан знать: имена функций и PDB

Начинающие часто ждут, что RtlLookupFunctionEntry каким-то образом найдёт "функцию" в человеческом смысле: имя, сигнатуру, типы аргументов, исходную строку. Но function table не про это. RUNTIME_FUNCTION-запись не хранит имя. В .pdata нет списка C++ методов. В UNWIND_INFO нет типа возвращаемого значения. Для lookup всё это лишнее.

Имена могут прийти из PDB, export table, symbols, debug info, map-файла или эвристик дизассемблера. Но unwind-механизм не зависит от наличия PDB. Это хорошая новость: даже stripped binary без символов может иметь корректную .pdata/.xdata, и Windows сможет делать stack walking. Это также плохая новость для человека: найденная RUNTIME_FUNCTION может дать границы и unwind, но не объяснит бизнес-логику функции.

Для static это означает: не надо смешивать уровни. Symbol enrichment - это дополнительный слой отчёта. Core unwind validation должен работать без символов. Если PDB есть, можно красиво показать имя функции рядом с диапазоном. Если PDB нет, finding всё равно должен быть доказуем через RVA, bytes, section, Data Directory и поля RUNTIME_FUNCTION.

PDB (Program Database, база отладочных символов Microsoft) - файл с отладочной информацией: имена функций, типы, строки исходного кода, локальные переменные и другие сведения. PDB помогает человеку, но RtlLookupFunctionEntry не требует PDB для поиска RUNTIME_FUNCTION.

Symbol (символ, именованная сущность отладки/линковки) - имя функции, переменной или другого объекта. Символ полезен для читаемости, но не является основой x64 unwind lookup.

Stripped binary (бинарь без символов) - исполняемый файл, из которого удалены или недоступны отладочные имена. Он всё равно может иметь корректные unwind-метаданные.

Пограничные адреса: почему EndAddress не принадлежит функции

Lookup особенно чувствителен к границам диапазона. Если ControlPc лежит в середине функции, всё понятно. Но что делать, если адрес равен EndAddress? Ответ: такая запись не подходит. Диапазон RUNTIME_FUNCTION считается полуоткрытым. BeginAddress включён, EndAddress исключён. Это нужно, чтобы соседние записи могли стыковаться без пересечения.

В stack walking есть ещё один тонкий момент: return address обычно указывает на инструкцию после call. Если caller вызвал callee, то после возврата выполнение продолжится по адресу сразу после call. Когда мы строим stack trace, этот return address является ControlPc для caller-а. Иногда в ручных анализаторах встречается практика вычитать 1 из return address, чтобы попасть "внутрь" call-инструкции или предыдущего диапазона. Но это не универсальное правило, и слепое применение может исказить результат. В Windows API есть свои правила интерпретации ControlPc, и их нельзя заменять бытовой эвристикой без понимания контекста.

Для твоего анализатора правильная позиция такая: явно фиксировать, что именно ты ищешь. Если ты проверяешь принадлежность текущего RIP диапазону, используй условие Begin <= RVA < End. Если ты анализируешь return address из стека для отображения call site, можешь отдельно вычислять call-site approximation, но не смешивай это с формальной принадлежностью ControlPc без пометки.

Return address (адрес возврата) - адрес, который call кладёт на стек; после ret процессор перейдёт именно туда. Обычно это адрес инструкции после call, а не адрес самой call-инструкции.

Call site (место вызова) - участок кода caller-а, где была выполнена инструкция call. Return address находится после call site, поэтому для отображения "откуда вызвали" иногда нужны дополнительные эвристики дизассемблера.

Boundary condition (граничное условие) - случай на краю диапазона: ровно BeginAddress, ровно EndAddress, соседние функции, пустой диапазон. Такие случаи часто ломают наивные парсеры.

Ошибка parser-а: использовать условие BeginAddress <= RVA <= EndAddress. Правильно: BeginAddress <= RVA < EndAddress. Включение EndAddress создаёт ложное пересечение соседних функций.

Пример полного пути: от исключения до найденной записи

Соберём теперь всю цепочку в один спокойный пример. Пусть в потоке произошло access violation. Windows получила EXCEPTION_RECORD и CONTEXT. В CONTEXT лежит RIP - адрес инструкции, на которой произошло исключение. Этот RIP становится ControlPc для первого lookup. RtlLookupFunctionEntry ищет в active function tables запись, диапазон которой покрывает этот ControlPc.

Если запись найдена, система получает FunctionEntry и ImageBase. FunctionEntry даёт BeginAddress, EndAddress и UnwindInfoAddress. Через ImageBase можно получить VA начала функции и VA UNWIND_INFO. После этого RtlVirtualUnwind сможет прочитать UNWIND_INFO и вычислить caller context. Новый ContextRecord будет содержать RIP caller-а и обновлённый RSP. Затем цикл повторится: новый RIP снова отправляется в RtlLookupFunctionEntry, и так постепенно строится цепочка кадров.

Здесь видно, почему lookup является не разовым событием, а регулярным шагом stack walking. Для каждого кадра нужен свой поиск. Если на одном кадре lookup неуспешен, stack walker должен решить, можно ли применить leaf logic или нужно остановиться. Если применить leaf logic там, где функция на самом деле имела сложный stack frame, цепочка дальше может стать мусором.

```
Идеальная линия одного кадра:

Exception -> CONTEXT.Rip = ControlPc
ControlPc -> RtlLookupFunctionEntry
RtlLookupFunctionEntry -> FunctionEntry + ImageBase
FunctionEntry + ImageBase -> адрес UNWIND_INFO
UNWIND_INFO + CONTEXT -> RtlVirtualUnwind
RtlVirtualUnwind -> CONTEXT caller-a
CONTEXT caller-a -> следующий ControlPc
```

Как это выглядит глазами WinDbg, Ghidra и твоего static

Разные инструменты смотрят на тот же механизм под разными углами. WinDbg работает близко к runtime-картине: у него есть адреса, модули, символы, dump memory, unwind information. Когда он строит stack trace, ему важно не просто красиво разобрать инструкции, а пройти от кадра к кадру по правилам платформы. RtlLookupFunctionEntry-подобная логика является частью этой культуры: найти function entry, затем размотать кадр.

Ghidra и IDA чаще используются как статические дизассемблеры. Они могут использовать .pdata как источник function boundaries. Это особенно полезно, когда обычное function discovery затруднено. Но дизассемблер может строить собственную модель функций, объединять или разделять блоки, называть функции по эвристикам. Поэтому результаты дизассемблера нужно сравнивать с .pdata, но не смешивать одно с другим.

static /dynamic в твоём проекте должен занять третью позицию. Он не просто показывает дизассемблирование и не просто повторяет WinDbg. Его задача - объяснить, насколько unwind-метаданные согласованы с PE, кодом и ожидаемой моделью Windows x64. Для этого lookup-понимание необходимо: каждый finding вокруг .pdata должен отвечать на вопрос, как это повлияет на поиск записи по ControlPc и последующий unwind.

WinDbg (Windows Debugger, отладчик Windows) - инструмент Microsoft для отладки процессов и дампов. В контексте этой книги важен как пример инструмента, который строит stack trace и опирается на unwind-данные платформы.

Function discovery (обнаружение функций) - задача дизассемблера: определить, где начинаются и заканчиваются функции. .pdata помогает, но не всегда полностью совпадает с логическими функциями исходного кода.

Stack trace (трасса стека, цепочка вызовов) - список кадров, показывающий, через какие функции поток пришёл к текущей точке. На x64 корректный stack trace зависит от unwind metadata и lookup/virtual unwind логики.

Findings: какие проблемы вокруг lookup должен видеть анализатор

Теперь переведём теорию в язык будущего отчёта static . Если RtlLookupFunctionEntry ищет запись по ControlPc, значит любые повреждения, которые мешают этому поиску, важны. Но важны они по-разному. Нельзя кричать "critical" на каждую странность. Надо объяснять, как именно странность влияет на lookup.

Например, если .pdata не отсортирована, это серьёзнее, чем просто "плохой стиль". Табличная модель x64 предполагает отсортированные RUNTIME_FUNCTION entries. Несортированная таблица может мешать бинарному поиску и приводить к неправильным lookup-результатам. Если две записи пересекаются, один ControlPc может формально подходить к двум функциям - это ломает однозначность. Если BeginAddress >= EndAddress, запись не описывает нормальный диапазон. Если UnwindInfoAddress выходит за image, lookup может быть успешен, но следующий RtlVirtualUnwind столкнётся с мусором.

Особенно аккуратно нужно писать findings для отсутствующей записи. Если адрес внутри исполняемой секции не покрыт .pdata, это может быть нормальная leaf function. Но если дизассемблирование показывает push nonvolatile, sub rsp с большим allocation, call внутри функции, а записи нет, это уже сильнее похоже на проблему: функция выглядит nonleaf или frame-using, но не имеет unwind metadata. Такой finding должен включать доказательства: адрес диапазона, инструкции, отсутствие RUNTIME_FUNCTION, возможные mitigations.

Finding idea	Что проверяется	Почему связано с RtlLookupFunctionEntry
PDATA_UNSORTED	BeginAddress записей не возрастает	Lookup обычно предполагает отсортированную таблицу; бинарный поиск становится ненадёжным.
PDATA_OVERLAP	Диапазоны двух записей пересекаются	Один ControlPc может подходить сразу к нескольким entries, что ломает однозначность.
RUNTIME_EMPTY_RANGE	BeginAddress >= EndAddress	Запись не задаёт нормальный полуоткрытый диапазон функции.
UNWIND_RVA_INVALID	UnwindInfoAddress вне допустимой области	Lookup может найти entry, но следующий шаг не сможет корректно прочитать UNWIND_INFO.
COVERAGE_GAP_SUSPICIOUS	Исполняемый диапазон не покрыт function table	Может быть leaf, но для nonleaf-похожего кода это проблема для stack walking.
STATIC_RUNTIME_MISMATCH	Static .pdata не совпадает с runtime lookup	Возможны dynamic tables, manual mapping, повреждение или ошибка модели анализа.

Severity (уровень серьёзности) - оценка влияния finding-а. Для unwind-анализа severity должна зависеть не от красоты структуры, а от того, ломает ли проблема lookup, virtual unwind,

handler dispatch или только ухудшает читаемость.

Mitigation (смягчающее объяснение) - нормальная причина, которая уменьшает подозрительность finding-а. Например, отсутствие RUNTIME_FUNCTION для маленькой leaf function может быть допустимым.

Manual mapping и packed binaries: почему static и runtime могут расходиться

В обычной программе loader Windows загружает PE, регистрирует нужные таблицы, и runtime lookup хорошо соответствует тому, что ты видишь в файле. Но reverse engineering редко живёт только в идеальном мире. Manual mapper может загрузить PE не обычным загрузчиком. `dynamic` может распаковать код в память. Packer может держать на диске один вид, а во время выполнения создавать другой. JIT может генерировать код динамически. Всё это влияет на lookup.

Manual mapping (ручное отображение модуля) особенно важно для понимания. Если кто-то вручную скопировал секции PE в память, применил релокации и разрешил импорты, это ещё не значит, что Windows знает о модуле как о нормальном loaded image со всеми metadata. Если exception directory/function tables не зарегистрированы корректно, RtlLookupFunctionEntry может не найти записи для адресов, которые в файле вроде бы имели .pdata. В результате исключения и stack walking становятся нестабильными.

Packed binary (упакованный бинарь) добавляет другую сложность. На диске .pdata может описывать только unpacking stub, а настоящий код появляется позже. Или таблицы могут быть повреждены специально, чтобы мешать отладчику и crash analysis. В отчёте нельзя автоматически писать "malware". Но можно писать строго: runtime unwind reliability is reduced; static .pdata does not describe observed executable regions; lookup may fail for generated ranges unless dynamic function tables are registered.

Manual mapping (ручное отображение PE) - загрузка PE-образа в память без обычного полного пути Windows loader-а. Такой код может выполняться, но системные структуры для lookup/unwind могут быть не зарегистрированы как у нормального модуля.

Packer (упаковщик) - механизм, который хранит программу на диске в изменённом/сжатом/зашифрованном виде и восстанавливает настоящий код во время выполнения. Это может менять картину .pdata/.xdata между static и runtime состоянием.

`dynamic` (защитный инструмент) - система, которая усложняет анализ программы: обфускация, упаковка, anti-debug, runtime code generation. `dynamic` не обязательно вредоносен, но может создавать аномалии unwind-метаданных.

Мини-лаборатория мысли: почему один адрес может не найтись

Представь, что у нас есть ControlPc = 00007FF6_50101580. Ты открываешь модуль и видишь, что он находится по ImageBase 00007FF6_50000000. ControlRva равен 00101580. В .pdata есть записи до 00101000, потом следующая начинается с 00102000. Формально адрес 00101580 находится в gap - промежутке между описанными функциями. Что это значит?

Первый вариант: это действительно маленький leaf code region, которому не нужна запись. Тогда NULL lookup может быть нормальным. Второй вариант: это data-in-code или jump table, и ControlPc туда вообще не должен попадать как в инструкцию. Третий вариант: функция была неправильно выделена или .pdata повреждена. Четвёртый вариант: код появился после распаковки, а статический файл не отражает runtime-картину. Пятый вариант: ты ошибся с ImageBase или взял wrong module.

Такой пример показывает, почему хороший анализатор не должен выдавать одну простую фразу. Он должен собирать evidence: права памяти, принадлежность модулю, секцию, дизассемблирование рядом, наличие call/sub rsp/push nonvolatile, coverage соседних записей, dynamic table hints, соответствие Exception Directory. Только после этого finding становится инженерным выводом, а не гаданием.

Gap (промежуток покрытия) - участок исполняемых адресов, который не попадает ни в одну RUNTIME_FUNCTION-запись. Gap может быть нормальным для leaf-кода, но может быть и признаком проблемы.

Evidence (доказательство, фактическая опора finding-а) - конкретные байты, адреса, поля таблиц, права секций, результаты lookup или дизассемблирования, на которых основан вывод.

```
Пример gap:

RUNTIME_FUNCTION #1: [00100000, 00101000)
RUNTIME_FUNCTION #2: [00102000, 00103000)

ControlRva = 00101580

00101000 <= 00101580 < 00102000
=> адрес находится между описанными диапазонами
=> RtlLookupFunctionEntry для такого ControlPc может вернуть NULL
=> нужна диагностика: leaf? data? obfuscation? wrong image? dynamic code?
```

Практическое правило: lookup даёт право перейти к unwind, но не заменяет проверку

После всех деталей можно сформулировать спокойное правило. RtlLookupFunctionEntry - это дверь к unwind. Если дверь открылась, мы получили FunctionEntry и ImageBase. Теперь можно читать UNWIND_INFO и делать RtlVirtualUnwind. Если дверь не открылась, нужно решить, допустима ли leaf-логика или анализ должен остановиться. Но ни успешный lookup, ни NULL не являются полным диагнозом сами по себе.

Успешный lookup не гарантирует, что UNWIND_INFO корректен. NULL не гарантирует, что бинарь сломан. ImageBase не гарантирует, что ты правильно нормализовал адреса для отчёта. HistoryTable не является источником истины. RUNTIME_FUNCTION не является именем функции. Все эти вещи становятся понятными, если помнить главную архитектуру: lookup связывает адрес с записью таблицы, а не "понимает программу" целиком.

Для reverse engineer-а ценность RtlLookupFunctionEntry в том, что она показывает официальный способ Windows задавать вопрос к unwind-метаданным. Если твой static строит собственный static lookup, он должен объяснять, насколько его результат совпадает с этой моделью. Если твой dynamic меняет .pdata/.xdata, он должен понимать, что меняет не просто декоративные таблицы, а входные данные для системного lookup и

последующей раскрутки стека.

Короткая формула раздела: RtlLookupFunctionEntry не разматывает стек. Она находит RUNTIME_FUNCTION по ControlPc и возвращает ImageBase. Но именно этот маленький шаг решает, есть ли у Windows формальный путь к UNWIND_INFO для текущего кадра.

Почему на x64 такой lookup вообще стал нормой

Чтобы оценить RtlLookupFunctionEntry по-настоящему, полезно вспомнить отличие x64-модели от старой x86-интуиции. В 32-битном Windows SEH часто вспоминают цепочку обработчиков в стеке. Там у многих начинающих возникает образ: обработчики как будто лежат прямо в стековых кадрах, и система идёт по цепочке. В x64 картина другая: unwind и обработка исключений в большой степени опираются на таблицы. Поэтому перед системой постоянно стоит вопрос не "какая запись SEH лежит в этом кадре", а "какая табличная запись описывает функцию, где сейчас находится управление".

Это не косметическая замена одной структуры другой. Табличный подход делает stack frames менее зависимыми от размещения handler-записей прямо в стеке, даёт компилятору более строгий способ описывать пролог, эпилог и сохранённые nonvolatile-регистры, а отладчикам даёт единый источник для stack trace. Но за это приходится платить требованием к метаданным: если функция изменила стек или сохраняла регистры, система должна иметь способ узнать, как это было сделано. RtlLookupFunctionEntry и есть первый шаг к этому знанию.

Поэтому в x64 мире нельзя относиться к .pdata/.xdata как к второстепенным украшениям линкера. Для RtlLookupFunctionEntry это не украшение, а рабочая карта. Если карта отсутствует там, где функция не является leaf, восстановление caller context становится догадкой. Иногда догадка сработает. Иногда даст красивый, но ложный stack trace. А ложный stack trace в crash dump analysis хуже, чем честная остановка: он уводит инженера в неправильную функцию.

Table-based exception handling (табличная обработка исключений) - модель, где сведения для unwind и обработки исключений хранятся в таблицах, связанных с диапазонами кода. На x64 это базовая идея: сначала lookup по адресу, затем чтение unwind-данных.

Stack-based SEH intuition (стековая интуиция SEH) - привычка думать об обработчиках как о цепочке записей прямо в стеке. Для x64 такая интуиция опасна: ключевые сведения приходят из таблиц .pdata/.xdata, а не из простой linked list в стеке.

Пошаговая история одного lookup без магии

Пройдём ещё один раз, но теперь как историю системы. Поток выполняет код. В какой-то момент возникает исключение или отладчик хочет построить стек. У системы есть CONTEXT. Внутри CONTEXT есть RIP. Этот RIP - просто число, virtual address. Сам по себе он ничего не рассказывает о том, сколько байт функция выделила на стеке, какие регистры сохранила и где лежит её UNWIND_INFO.

Система передаёт этот RIP как ControlPc. RtlLookupFunctionEntry сначала должна понять, относится ли этот адрес к какому-то известному диапазону кода. Если адрес попадает внутрь загруженного PE-модуля, система может обратиться к function table этого модуля. Если адрес относится к runtime-generated code, может понадобиться dynamic function table. Если адрес вообще не похож на зарегистрированный исполняемый диапазон,

lookup заканчивается NULL.

Когда подходящая таблица найдена, начинается поиск внутри неё. У таблицы есть записи, у каждой записи есть BeginAddress и EndAddress. Эти поля не сравниваются с ControlPc напрямую как VA; их смысл image-relative. Поэтому логически система приводит адрес к базе соответствующего образа/таблицы. Затем проверяет попадание в полуоткрытый интервал. Найденная запись и ImageBase возвращаются вызывающему коду. Всё. Никакого чтения UWOP_* ещё не было. Никакого восстановления регистров ещё не было. Но путь к этому уже открыт.

```
Один lookup в словах:

ControlPc is a VA
-> определить relevant active function table
-> получить или знать base address этой table/image
-> привести ControlPc к относительному смещению
-> найти RUNTIME_FUNCTION, где Begin <= offset < End
-> вернуть pointer на запись
-> записать ImageBase в out-параметр
```

Как нормализовать результат lookup для отчёта static

Если твой инструмент когда-нибудь будет сравнивать runtime lookup с offline-разбором, тебе придётся нормализовать адреса. Нормализация здесь означает перевод разных видов адресов к одному понятному виду. Runtime API возвращает pointer на запись и ImageBase. Offline parser обычно работает с RVA и file offsets. Отчёт человеку лучше показывать в нескольких формах: module name, RVA, VA при наличии runtime ImageBase, file offset при наличии соответствующей секции.

Например, RtlLookupFunctionEntry вернула FunctionEntry pointer 00007FF6_40208030 и ImageBase 00007FF6_40000000. Сам pointer на entry интересен как runtime-адрес, но в отчёте по PE важнее вычислить, что эта запись находится в Exception Directory на RVA 00208030 или file offset 001F7430, если перевод возможен. Поля самой записи тоже надо показывать как RVA: BeginAddress, EndAddress, UnwindInfoAddress. Рядом можно дать VA, полученные прибавлением ImageBase.

Такой отчёт будет понятен и тебе, и проверяющему. Он не говорит "где-то там указатель". Он показывает цепочку доказательства: ControlPc VA -> ImageBase -> ControlRva -> RUNTIME_FUNCTION entry RVA/file offset -> UnwindInfo RVA -> section/bytes. Это ровно тот стиль, который нужен в фундаментальном инженерном проекте: каждый вывод можно проверить руками.

Normalization (нормализация адресов) - приведение VA, RVA, file offset и runtime pointers к согласованной форме, чтобы их можно было сравнивать и показывать в отчёте без путаницы.
Module name (имя модуля) - имя загруженного EXE/DLL, к которому относится адрес. В отчёте полезно писать не только адрес, но и module+offset, например app.exe+0x123456.
Module+offset (модуль плюс смещение) - форма записи адреса, где показывается имя модуля и смещение от ImageBase. Это удобный мост между runtime VA и статическим RVA-анализом.

```
Пример нормализации для отчёта:

Runtime:
ControlPc = 00007FF6_40123456
ImageBase = 00007FF6_40000000
```

```
Normalized:
Module = app.exe
ControlRva = 00123456
Display = app.exe+0x123456

Found entry:
BeginAddress = 00122000 -> app.exe+0x122000
EndAddress = 001226A0 -> app.exe+0x1226A0
UnwindInfoAddress = 00204000 -> app.exe+0x204000
```

Отдельно про current process: почему API нельзя бездумно применять к чужому файлу

RtlLookupFunctionEntry выполняется в контексте процесса, где она вызвана. Это звучит очевидно, но на практике часто рождает ошибки. Если у тебя на диске лежит sample.exe, ты не можешь просто взять его RVA, вызвать RtlLookupFunctionEntry в своём analyzer.exe и ждать, что Windows найдёт запись внутри sample.exe. Если sample.exe не загружен в адресное пространство analyzer.exe как модуль с соответствующими таблицами, такого runtime lookup не будет.

Offline-анализ чужого файла должен читать PE самостоятельно. Runtime lookup через RtlLookupFunctionEntry имеет смысл для адресов, которые существуют в текущем процессе: собственный код, загруженные DLL, специально mapped image, JIT code с registered function tables. Для crash dump analysis ситуация ещё тоньше: ты анализируешь снимок чужого процесса, а не выполняешься внутри него. Там обычно используют отладочные библиотеки, DbgHelp/DbgEng или собственный парсер dump memory, а не просто вызывают RtlLookupFunctionEntry на адресах из дампа.

Это очень важно для архитектуры static . Если он является static PE analyzer, его core не должен зависеть от того, что sample загружен в текущий процесс. Если он имеет runtime mode, тогда можно отдельно реализовать сбор runtime evidence. Но эти режимы надо разделять. Иначе появится класс ошибок, где analyzer случайно анализирует свои собственные модули вместо исследуемого файла.

Current process (текущий процесс) - процесс, внутри которого выполняется код, вызвавший API. RtlLookupFunctionEntry ищет адреса, существующие в этом адресном пространстве, а не произвольные байты любого файла на диске.
Crash dump (дамп падения, снимок состояния процесса) - сохранённое состояние памяти, потоков и модулей после сбоя. Анализ dump требует восстановления картины чужого процесса, поэтому обычный runtime API текущего процесса не всегда применим напрямую.
DbgHelp / DbgEng (отладочные библиотеки Windows) - инструменты и API для работы с символами, стеками и дампами. Они ближе к задачам offline/debug analysis, чем прямой вызов RtlLookupFunctionEntry в текущем процессе.
Ошибка архитектуры инструмента: смешать static PE analysis и live process lookup. Если пользователь дал файл, анализируй его структуры как файл. Если пользователь дал PID/live process, тогда можно говорить о runtime lookup. Если пользователь дал dump, нужен dump-aware путь.

Что происходит при неправильном ImageBase

ImageBase кажется простой арифметикой, пока не появляется первая ошибка. Пусть ты взял ControlPc, вычел не тот ImageBase и получил ControlRva. После этого binary search по .pdata может найти совсем другую запись или не найти ничего. В отчёте это будет выглядеть как "metadata missing" или "range mismatch", хотя реальная причина - неправильная нормализация адреса.

Откуда берётся неправильный ImageBase? В offline-анализе можно случайно использовать preferred ImageBase, хотя в runtime модуль был relocated. В dump-анализе можно взять базу другого модуля с похожим адресом. В manual mapping сценарии preferred ImageBase из заголовка может не соответствовать фактическому размещению. В 64-битном процессе с множеством DLL адреса длинные, и человек легко ошибается глазами.

Поэтому любой серьёзный анализатор должен явно хранить пару: module identity + image base. Нельзя хранить просто число base без доказательства, к какому модулю оно относится. Нельзя сравнивать RVA из одного модуля с ImageBase другого. Нельзя показывать function range без module context. Это звучит бюрократично, но именно такие мелочи отличают инструмент от набора скриптов.

Preferred ImageBase (предпочтительный базовый адрес) - адрес, по которому PE хотел бы быть загружен. Фактический runtime ImageBase может отличаться из-за ASLR, конфликтов адресов и релокаций.
ASLR (Address Space Layout Randomization, рандомизация размещения адресного пространства) - механизм защиты, при котором модули могут загружаться по разным базовым адресам в разных запусках. Это делает различие RVA/VA особенно важным.
Relocation (релокация, исправление адресов при загрузке) - процесс корректировки адресных ссылок, если образ загружен не по preferred ImageBase. Для lookup важно использовать фактический ImageBase.

Почему lookup важен не только для исключений

Название главы связано с unwind и исключениями, но RtlLookupFunctionEntry важна шире. Любой механизм, которому нужно пройти по стеку на x64, сталкивается с той же проблемой: как из текущего RIP получить сведения о текущем кадре. Это нужно отладчику, профилировщику, crash reporter, системам телеметрии, трассировщикам производительности, иногда антивирусным и EDR-инструментам. Все они в той или иной форме нуждаются в function table lookup.

Profiler (профилировщик) может периодически останавливать поток, смотреть RIP и строить sample stack. Если unwind metadata плохие, profiler будет показывать неправильные горячие функции. Crash reporter может отправить stack trace разработчику. Если lookup сломался на середине, trace обрывается или показывает мусор. Debugger может пытаться показать call stack в момент исключения; без корректных таблиц он теряет доверие к кадрам.

Поэтому повреждение .pdata/.xdata - это не локальная проблема одного __try/__except. Это удар по наблюдаемости программы. Observability - способность понять, что произошло в программе после факта. В современном инженерном мире это критично: если программа падает у пользователя, разработчик часто видит только dump и stack trace. Если

lookup/unwind сломаны, dump становится намного менее полезным.

Profiler (профилировщик) - инструмент, который измеряет, где программа тратит время. Для call stack sampling на x64 ему нужны корректные unwind-данные.

Crash reporter (система отчётов о падениях) - компонент, собирающий сведения о сбое: exception code, context, modules, stack trace, dump. Качество stack trace зависит от lookup и unwind.

Observability (наблюдаемость) - способность системы давать понятные данные о своём состоянии и сбоях. Unwind metadata напрямую влияет на наблюдаемость native x64 программ.

Практический C-фрагмент и почему он всё ещё не весь анализ

Для закрепления полезно увидеть минимальный C-подобный фрагмент. Он показывает, как API вызывается технически. Но важно помнить: такой фрагмент не заменяет всю теорию. Он не проверяет, корректна ли запись. Он не строит полный stack walker. Он не защищён от плохих адресов. Он просто демонстрирует договор функции.

```
#include <windows.h>
#include <winnt.h>

void inspect_current_rip(CONTEXT* ctx) {
DWORD64 imageBase = 0;
UNWIND_HISTORY_TABLE history = {0};

PRUNTIME_FUNCTION fn = RtlLookupFunctionEntry(
ctx->Rip,
&imageBase,
&history
);

if (fn) {
DWORD64 beginVA = imageBase + fn->BeginAddress;
DWORD64 endVA = imageBase + fn->EndAddress;
DWORD64 unwindVA = imageBase + fn->UnwindInfoAddress;

// Теперь можно диагностировать диапазон
// и передавать fn/imageBase в RtlVirtualUnwind.
} else {
// Нет function table entry для этого RIP.
// Возможна leaf-логика, но нужна осторожность.
}
}
```

C-like snippet (C-подобный фрагмент) - учебный пример кода, показывающий идею вызова API. Он полезен для ориентира, но не является production-ready реализацией stack walker-а.

Production-ready (готовый для реального использования) - код, который учитывает ошибки, границы памяти, разные платформы, плохие данные, конкуренцию потоков и диагностику. Учебный фрагмент обычно таким не является.

Лестница доверия для static

Закончим раздел практической моделью доверия. Когда static анализирует function table lookup, он может мыслить лестницей. Первый уровень: таблица существует и доступна. Второй уровень: таблица формально корректна. Третий уровень: lookup по интересующим адресам даёт однозначный результат. Четвёртый уровень: найденная UNWIND_INFO корректно разбирается. Пятый уровень: unwind metadata согласуются с

реальным прологом/эпилогом. Шестой уровень: stack walking, построенный на этих данных, выглядит устойчивым.

Эта лестница лучше, чем один бинарный verdict "good/bad". Реальные бинарники часто находятся между крайностями. Таблица может существовать, но иметь пару странных gaps. Или lookup работает, но UNWIND_INFO содержит нестандартные chained records. Или static .pdata корректна, но runtime добавляет dynamic tables. Хороший анализатор должен уметь объяснять степень уверенности.

В отчёте это можно выражать как confidence. Confidence - не то же самое, что severity. Severity говорит, насколько проблема опасна. Confidence говорит, насколько мы уверены в выводе. Например, пересечение двух RUNTIME_FUNCTION-записей с точными адресами даёт высокий confidence. Отсутствие записи для маленького участка кода без дизассемблирования даёт низкий confidence. Такой подход делает инструмент честнее.

Confidence (уверенность вывода) - степень, с которой анализатор уверен в finding-е на основе evidence. Высокая severity без высокого confidence опасна: можно напугать пользователя ложным выводом.
Verdict (итоговая оценка) - общий вывод по объекту анализа. В хорошей системе verdict должен вытекать из findings, severity, confidence и mitigations, а не из одной эвристики.

Связь со следующим разделом

Теперь у нас есть всё для следующего шага. Мы знаем, как Windows берёт адрес выполнения и находит запись функции. Мы знаем, что RtlLookupFunctionEntry возвращает FunctionEntry и ImageBase. Мы знаем, почему NULL требует осторожной интерпретации. Но пока мы ещё не разобрали главный вычислительный этап: как из найденной записи и текущего CONTEXT получить предыдущий CONTEXT.

Этим занимается RtlVirtualUnwind. Если RtlLookupFunctionEntry - это поиск нужной карточки в каталоге, то RtlVirtualUnwind - это чтение инструкции на этой карточке и восстановление состояния caller-а. Следующий раздел будет значительно глубже: мы увидим, как система использует UNWIND_INFO, как различает пролог, тело и эпилог, как восстанавливает RSP, как возвращает nonvolatile-регистры, зачем нужен EstablisherFrame и почему virtual unwind является ядром всей x64 SEH-модели.

Разбор API как инженерного договора

В системном программировании API нужно читать как договор между вызывающим кодом и операционной системой. Договор RtlLookupFunctionEntry выглядит так: ты передаёшь мне адрес управления в виде virtual address; я попробую найти в активных таблицах функций запись, которая покрывает этот адрес; если найду, верну pointer на запись и base address соответствующего образа; если не найду, верну NULL. Всё остальное не входит в этот договор.

Почему такая формулировка важна? Потому что она защищает от лишних ожиданий. Нельзя требовать от RtlLookupFunctionEntry, чтобы она доказала, что код безопасен. Нельзя требовать от неё, чтобы она назвала функцию. Нельзя требовать, чтобы она сказала, почему записи нет. Она выполняет одну конкретную роль. Чем точнее мы понимаем эту роль, тем лучше проектируем анализатор вокруг неё.

В static этот договор можно отразить прямо в архитектуре. Один компонент отвечает за static function table extraction. Второй - за lookup simulation. Третий - за validation of unwind metadata. Четвёртый - за correlation with disassembly. Пятый - за report findings. Если всё смешать в одну функцию "analyze_pdata", код быстро станет непонятным: одно место будет и парсить, и искать, и дизассемблировать, и решать severity. Фундаментальное понимание Windows API помогает сделать не только правильную теорию, но и нормальную архитектуру проекта.

API contract (договор API) - точное соглашение о том, что функция принимает, что возвращает и чего не обещает. Для системного анализа важно не приписывать API лишних обязанностей.

Correlation (сопоставление) - связывание данных из разных источников: .pdata, .xdata, дизассемблирование, section permissions, symbols, runtime observations. Хороший finding обычно рождается именно из correlation, а не из одного поля.

Почему "найти функцию" звучит проще, чем есть на самом деле

Фраза "найти функцию по адресу" звучит почти бытово. Но в бинарном мире она неоднозначна. Какую функцию? Функцию из исходного кода? Диапазон, который выделила Ghidra? Символ из PDB? Exported function? RUNTIME_FUNCTION entry? Inline-фрагмент? Thunk? Tail-call block? Все эти варианты могут по-человечески называться функцией, но для RtlLookupFunctionEntry существует только function table entry.

Например, компилятор мог inline-ить маленькую функцию внутрь другой. В исходном коде была отдельная функция, а в machine code отдельного диапазона уже нет. Lookup её не найдёт, потому что для unwind нет отдельной записи. Или наоборот: один исходный метод мог породить несколько code ranges, cold blocks, separated regions или chained unwind structures. Тогда человеческое "одна функция" не совпадает с одной записью таблицы. Это нормально для оптимизирующего компилятора.

Поэтому когда ты пишешь отчёт, лучше использовать точные формулировки. Не "функция отсутствует", а "для RVA X отсутствует покрывающая RUNTIME_FUNCTION-запись в статической Exception Directory". Не "функция странная", а "диапазон RUNTIME_FUNCTION пересекается с соседним диапазоном". Такой язык звучит чуть тяжелее, зато он инженерно честный и защищает от ложных выводов.

Inline (встраивание функции) - оптимизация, когда код вызываемой функции вставляется прямо в caller, и отдельного машинного вызова/диапазона может не быть. Для unwind lookup это означает, что исходная функция не обязательно имеет отдельный RUNTIME_FUNCTION.

Thunk (переходник, маленькая служебная функция) - небольшой участок кода, который перенаправляет управление, адаптирует вызов или служит прослойкой. Thunk может иметь или не иметь отдельную unwind-запись в зависимости от того, что он делает со стеком.

Cold block (холодный блок) - редко выполняемый участок кода, который компилятор может вынести отдельно ради оптимизации размещения. Это может усложнять соответствие между исходной функцией и диапазонами в binary.

Ещё раз о доверии: системная функция не делает бинарь честным

Есть ещё одна тонкость. Если RtlLookupFunctionEntry вернула запись, это не значит, что бинарь "честный". Система нашла запись в активной таблице. Но сама таблица могла

быть необычной, специально сконструированной, неполной или вредной для анализа. Windows использует предоставленные метаданные в рамках своих правил; reverse engineer должен проверять, соответствуют ли они коду и PE-структуре.

Например, запись может покрывать диапазон, который в дизассемблере выглядит как часть другой функции. UnwindInfoAddress может указывать в область, которая формально readable, но семантически странная. Chained info может вести к предыдущей записи с несовместимым диапазоном. Dynamic function table может описывать код в памяти, которого нет в исходном файле. Всё это не отменяет факта lookup, но требует анализа.

Хороший вывод должен различать "Windows сможет найти запись" и "запись выглядит согласованной с ожидаемой моделью". Это разные уровни. Первый уровень - availability. Второй - validity. Третий - consistency with code. Четвёртый - security/reverse-engineering interpretation. Если смешать их, отчёт станет либо слишком мягким, либо слишком истеричным.

Availability (наличие/доступность) - факт, что запись или данные существуют и могут быть найдены.
Validity (формальная корректность) - соответствие структуры базовым правилам формата: размеры, выравнивание, диапазоны, допустимые флаги.
Consistency (согласованность) - соответствие разных источников друг другу: таблицы, кода, секций, дизассемблирования и runtime-поведения.

Контрольный итог раздела

RtlLookupFunctionEntry - это точка, где адрес выполнения превращается в табличную запись функции. Она принимает ControlPc как virtual address, ищет active function tables, возвращает PRUNTIME_FUNCTION и записывает ImageBase. Найденная RUNTIME_FUNCTION затем используется как вход для RtlVirtualUnwind. Если запись не найдена, возможна leaf-логика или диагностическая проблема, но сам по себе NULL требует контекста.

Для ручного анализа и проекта static особенно важны четыре идеи. Первая: lookup работает по формальным диапазонам, а не по именам функций. Вторая: поля RUNTIME_FUNCTION image-relative, поэтому ImageBase обязателен для правильной интерпретации. Третья: active function tables шире, чем просто .pdata на диске, потому что существуют dynamic function tables. Четвёртая: успешный lookup не отменяет validation - он только даёт запись, которую дальше нужно проверять и применять.

Если эти идеи удерживаются в голове, следующая тема станет естественной. RtlVirtualUnwind больше не будет казаться магией. Она получает то, что подготовил lookup: ImageBase, ControlPc, FunctionEntry и ContextRecord. А затем начинает главную работу - вычисляет, каким был контекст вызывающей функции до текущего вызова.

5.3. Что делает RtlVirtualUnwind

Мы дошли до места, где табличная модель Windows x64 перестаёт быть просто набором структур в PE-файле и превращается в действие. До этого мы долго собирали карту. Мы выясняли, что такое .pdata, как устроена запись RUNTIME_FUNCTION, почему поля этой записи являются RVA, как .pdata ведёт к .xdata, как внутри .xdata лежит UNWIND_INFO, как UNWIND_CODE описывает действия пролога, почему важны prolog, epilog, frame pointer, volatile и nonvolatile registers, и как calling convention задаёт правила игры для caller и callee. Всё это могло казаться большим количеством отдельных деталей. Но теперь эти детали сходятся в одной функции: RtlVirtualUnwind.

RtlLookupFunctionEntry, разобранная в предыдущем разделе, дала Windows ответ на первый вопрос: "какая RUNTIME_FUNCTION-запись описывает код, где сейчас находится RIP?" Но этот ответ ещё не поднимает нас по стеку. Он только говорит: "вот функция, вот её диапазон, вот адрес её unwind-метаданных". После этого появляется второй, более сложный вопрос: "как из текущего состояния этой функции получить состояние её caller-а?" Именно на этот вопрос отвечает RtlVirtualUnwind.

Если говорить живо, RtlVirtualUnwind - это не просто ещё одна системная функция из winnt.h. Это маленький исполнительный механизм, который берёт описание кадра и аккуратно разворачивает его назад. Она смотрит на текущий CONTEXT record, на RUNTIME_FUNCTION, на UNWIND_INFO, на расположение RIP внутри функции и обновляет контекст так, будто текущая функция уже вернулась в вызывающую функцию. При этом она не запускает код программы, не делает настоящий ret, не прыгает по стеку как процессор, не вызывает C++ catch и не печатает stack trace. Она делает более тонкую вещь: виртуально вычисляет, каким был бы контекст caller-а, если бы мы безопасно сняли текущий frame.

RtlVirtualUnwind (runtime virtual unwind, функция виртуальной раскрутки кадра) - системная функция Windows, которая по текущему ContextRecord, ImageBase, ControlPc и FunctionEntry вычисляет контекст предыдущего кадра. Слово virtual здесь означает: функция не исполняет машинный код целевой программы, а моделирует эффект выхода из текущей функции по unwind-метаданным.

Virtual unwind (виртуальная раскрутка) - восстановление состояния caller-а без реального выполнения инструкции ret и без продолжения работы программы. Это вычисление: где должен быть RIP caller-а, каким станет RSP, какие nonvolatile-регистры нужно восстановить из stack slots.

Unwind (раскрутка стека / размотка кадра) - процесс перехода от текущего stack frame к предыдущему. В обычном выполнении похожий эффект даёт возврат из функции, но при исключениях, crash dump analysis и stack walking его нужно получить искусственно и безопасно.

Очень важная интуиция: RtlLookupFunctionEntry отвечает "где описание функции?", а RtlVirtualUnwind отвечает "как снять один кадр этой функции?". Первая функция находит запись. Вторая функция применяет смысл этой записи к CONTEXT record.

Почему без RtlVirtualUnwind вся предыдущая теория была бы неполной

Можно выучить формат RUNTIME_FUNCTION, можно распознать .pdata, можно красиво вывести BeginAddress, EndAddress и UnwindInfoAddress, можно даже разобрать все UWOP_* операции по байтам. Но пока мы не понимаем, как эти операции применяются к реальному CONTEXT record, мы видим только половину механизма. Unwind-метаданные существуют не ради таблицы как таблицы. Они существуют ради восстановления состояния выполнения.

На x64 Windows не может надёжно подниматься по стеку простым сканированием памяти. В стеке лежат return addresses, сохранённые регистры, локальные переменные, home space, выравнивание, временные значения, иногда данные обработчиков, иногда следы оптимизаций компилятора. Если просто идти по памяти и искать числа, похожие на адреса кода, получится гадание. Иногда оно сработает, иногда даст красивую ложь. Windows нужен не гадательный алгоритм, а формальное правило: для данной функции известно, что она сделала с RSP и nonvolatile-регистрами, значит эти действия можно обратить.

RtlVirtualUnwind как раз и является местом, где формальное описание превращается в новый контекст. Она не угадывает. Она не пытается "понять" исходный C-код. Она не обязана знать имя функции, типы аргументов или локальные переменные. Ей нужно другое: текущее значение RIP, текущее значение RSP, описание диапазона функции, указатель на UNWIND_INFO и доступ к памяти стека. Этого достаточно, чтобы сделать один шаг вверх по call stack.

CONTEXT record (запись контекста процессора) - структура Windows, где сохранены регистры потока: RIP, RSP, RBP, RBX, RSI, RDI, R12-R15, XMM-регистры и другие части состояния CPU. Для RtlVirtualUnwind это главный изменяемый объект: на входе он описывает текущий кадр, на выходе - уже caller frame.
Caller context (контекст вызывающей функции) - состояние регистров, которое должно соответствовать функции, вызвавшей текущую. После успешного virtual unwind ContextRecord должен выглядеть так, будто управление уже вернулось из callee в caller.
Callee context (контекст вызываемой функции) - состояние регистров внутри текущей функции. До virtual unwind ContextRecord обычно описывает именно callee: RIP находится внутри неё, а RSP указывает на её текущий stack frame.

Что значит "виртуально": функция не исполняет программу

Слово virtual здесь легко недооценить. В обычной жизни возврат из функции происходит физически: процессор выполняет ret, читает return address из памяти по RSP, записывает его в RIP, увеличивает RSP, и выполнение продолжается уже в caller-е. Но во время обработки исключения или анализа дампа мы не хотим реально продолжать выполнение неизвестного кода. Нам нужен расчёт. Нужно понять, какой адрес был бы следующим кадром, но не запускать его.

Именно поэтому virtual unwind похож не на выполнение программы, а на работу судебного эксперта. Программа оставила следы: значение RSP, сохранённые регистры в stack slots, return address, unwind-метаданные. RtlVirtualUnwind читает эти следы и

реконструирует состояние предыдущего кадра. Она как бы говорит: "если текущая функция была построена по этим правилам, то caller находится вот здесь, его RSP должен быть вот таким, а его RIP нужно взять из этого return address".

Это особенно важно для crash dump analysis. В crash dump программа уже не выполняется. У нас есть снимок памяти и регистров в момент падения. Никакой ret уже физически не будет исполнен. Но отладчик всё равно показывает call stack. Он делает это не потому, что стек сам рассказывает красивую историю, а потому что stack walker шаг за шагом применяет логику, похожую на RtlVirtualUnwind: для каждого кадра вычисляет предыдущий контекст.

Return address (адрес возврата) - адрес в caller-е, куда управление должно вернуться после завершения callee. На x64 инструкция call кладёт return address на стек. При virtual unwind этот адрес обычно становится новым RIP в caller context.

Stack walking (проход по стеку) - последовательное восстановление кадров: текущий frame -> caller frame -> caller of caller -> и так далее. RtlVirtualUnwind делает один шаг такого прохода, а полный stack trace получается повторением этого шага.

Crash dump analysis (анализ дампа падения) - разбор сохранённого снимка процесса или системы после сбоя. В дампе код уже не исполняется, поэтому стек приходится восстанавливать по регистрам, памяти и unwind-метаданным.

Прототип RtlVirtualUnwind как схема механизма

Прототип RtlVirtualUnwind выглядит устрашающе только на первый взгляд. В нём много параметров, но почти все они уже нам знакомы по предыдущим главам. Важно читать этот прототип не как набор типов, а как схему потока данных: что мы уже знаем, что передаём в функцию, что она изменяет и что возвращает наружу.

В типичном x64 stack walking сценарии порядок такой. Сначала у нас есть ContextRecord с текущим RIP и RSP. По RIP вызывается RtlLookupFunctionEntry. Lookup возвращает FunctionEntry и ImageBase. Затем эти значения вместе с тем же ContextRecord передаются в RtlVirtualUnwind. После вызова ContextRecord уже изменён: его RIP, RSP и часть регистров соответствуют caller-у. Дальше цикл повторяется.

```
PEXCEPTION_ROUTINE RtlVirtualUnwind(
DWORD HandlerType,
DWORD64 ImageBase,
DWORD64 ControlPc,
PRUNTIME_FUNCTION FunctionEntry,
PCONTEXT ContextRecord,
PVOID *HandlerData,
PDWORD64 EstablisherFrame,
PKNONVOLATILE_CONTEXT_POINTERS ContextPointers
);
```

PEXCEPTION_ROUTINE (pointer to exception routine, указатель на функцию-обработчик исключения) - тип возвращаемого значения. RtlVirtualUnwind может вернуть адрес language-specific handler-а, если текущая функция имеет обработчик соответствующего типа. Если обработчик не найден или не применим, результат может быть нулевым.

HandlerType (тип интересующего обработчика) - входной флаговый параметр, который связан с тем, какие handler flags рассматриваются при virtual unwind. В учебной модели его удобно понимать как указание: ищем ли мы exception handler, termination/unwind handler или

обрабатываем chained info.

FunctionEntry (запись функции) - указатель на RUNTIME_FUNCTION, найденную заранее через RtlLookupFunctionEntry. Эта запись указывает диапазон функции и RVA её UNWIND_INFO. Без неё RtlVirtualUnwind не знает, какие правила применить к текущему кадру.

HandlerData (данные обработчика) - выходной указатель на language-specific data, если у функции есть обработчик. Для C/C++ SEH это может вести к данным, которые затем читает __C_specific_handler. Это не "текст catch", а машинно-ориентированные данные языка/компилятора.

EstablisherFrame (устанавливающий кадр / база кадра обработчика) - значение, связывающее найденный handler с конкретным stack frame. Это один из ключевых объектов для language-specific handler-а; подробно он будет разобран в следующем разделе.

ContextPointers (указатели на сохранённые части контекста) - необязательная структура, куда unwinder может записать адреса мест, откуда были восстановлены nonvolatile-регистры. Это полезно для отладчиков и более сложных механизмов восстановления состояния.

Главный параметр здесь - ContextRecord. Он передаётся как in/out: на входе содержит текущий контекст, на выходе уже содержит контекст предыдущего кадра. Если это забыть, RtlVirtualUnwind начнёт казаться функцией, которая "возвращает данные". На самом деле её главный результат - изменённый ContextRecord.

Как выглядит один шаг подъёма по стеку

Чтобы не потеряться в API, держим в голове одну простую картину. Есть текущая функция, внутри которой находится RIP. Эта функция когда-то была вызвана через call. Поэтому где-то в стеке лежит return address, ведущий назад в caller. Но между текущим RSP и этим return address может находиться много служебного слоя: сохранённые nonvolatile-регистры, локальный стековый блок, выравнивание, область под dynamic allocation, иногда frame pointer. RtlVirtualUnwind должна понять, как этот слой снять.

Если текущая функция выделила 0x80 байт под локальные переменные, сохранила RBX и RBP, а затем ушла в тело функции, простой доступ к [RSP] не даст return address. По [RSP] может лежать локальная переменная или временный слот. Return address находится выше, после всей области кадра. Чтобы добраться до него, нужно знать, сколько байт выделено и где лежат сохранённые регистры. Эти сведения и берутся из UNWIND_INFO.

После virtual unwind новый ContextRecord должен соответствовать caller-у. RIP становится return address. RSP становится значением, которое caller имел после возврата из callee. Nonvolatile-регистры восстанавливаются так, как они должны выглядеть для caller-а. В этом и состоит смысл "раскрутки": мы не просто нашли предыдущий адрес, мы восстановили согласованное состояние регистров и стека.

```
Упрощённый один шаг stack walking:

current ContextRecord:
RIP = адрес внутри callee
RSP = текущий стековый указатель callee

lookup:
FunctionEntry = RtlLookupFunctionEntry(RIP, &ImageBase, ...)

virtual unwind:
RtlVirtualUnwind(..., ImageBase, RIP, FunctionEntry, &ContextRecord, ...)
```

```
updated ContextRecord:
RIP = return address в caller
RSP = RSP caller-a после возврата
nonvolatile registers = значения, восстановленные для caller-a
```

Три зоны функции: пролог, тело и эпилог

Одна из причин, почему RtlVirtualUnwind нельзя заменить одной формулой, заключается в том, что RIP может находиться в разных частях функции. Мы уже отдельно разбирали prolog и epilog, но теперь нужно увидеть, зачем это разделение существует внутри самого алгоритма unwind.

Функция в смысле unwind не является одинаковой на всём диапазоне BeginAddress..EndAddress. В прологе кадр ещё строится. Некоторые действия уже могли выполниться, а некоторые ещё нет. В теле функции кадр считается полностью построенным: все сохранения и stack allocation, описанные в прологе, уже применены. В эпилоге кадр уже начинает разбираться: часть стека могла быть освобождена, часть регистров уже могла быть восстановлена, а return ещё может не произойти. Поэтому один и тот же UNWIND_INFO нужно применять по-разному в зависимости от позиции RIP.

Документация Microsoft описывает три основных случая: RIP находится в эпилоге; RIP находится в прологе; RIP находится в остальном коде, где может быть связан handler. RtlVirtualUnwind должна распознать эти случаи и выбрать соответствующую логику. Если сделать это неправильно, можно восстановить слишком много или слишком мало. Например, если RIP стоит в середине пролога, нельзя отменять действия, которые ещё не были выполнены. Если RIP стоит в эпилоге, нельзя обращаться с кадром как с обычным телом функции: функция уже уходит, и нужно симулировать остаток эпилога.

Prolog region (область пролога) - начальный участок функции, где она строит stack frame: сохраняет nonvolatile-регистры, выделяет стек, устанавливает frame pointer. В UNWIND_INFO длина этой области задаётся SizeOfProlog.
Function body (тело функции) - основной участок функции после пролога и вне эпилогов. Для unwind это зона, где кадр считается полностью построенным, а значит можно применять полную обратную схему пролога.
Epilog region (область эпилога) - участок выхода из функции, где она восстанавливает стек и nonvolatile-регистры перед ret или допустимым jmp. На x64 эпилог распознаётся по строгим шаблонам кода, а не хранится отдельным списком в обычной x64 unwind info версии 1.

Если RIP находится в теле функции

Начнём с самого спокойного случая: RIP находится не в прологе и не в эпилоге. Это значит, что функция уже полностью построила свой кадр и ещё не начала его разбирать. Для virtual unwind такой случай наиболее прямой: нужно обратить все действия пролога, которые влияют на выход к caller-у.

Если пролог сделал push rbx, то в теле функции RBX может быть изменён как угодно, но оригинальное значение RBX caller-a лежит в сохранённом stack slot. Значит unwinder должен восстановить RBX из этого slot. Если пролог сделал sub rsp, 80h, значит в теле RSP уменьшен на 0x80 относительно позиции после push-ов; при unwind этот fixed allocation

нужно отменить. Если пролог установил frame pointer, то для вычисления некоторых offsets может потребоваться учитывать FrameRegister и FrameOffset. В конце нужно получить return address и сделать его новым RIP.

Здесь важно не думать, что RtlVirtualUnwind "исполняет пролог назад" буквально инструкцию за инструкцией. UNWIND_CODE - это не машинный код. Это компактная семантическая запись действий, важных для раскрутки. Если в прологе было mov [rsp+20h], rsi и эта операция описана как UWOP_SAVE_NONVOL, unwinder не обязан декодировать исходную машинную инструкцию mov. Он смотрит на unwind operation и понимает: nonvolatile-регистр RSI сохранён по такому-то offset, значит при восстановлении caller context нужно прочитать значение оттуда.

Fixed stack allocation (фиксированное выделение стека) - заранее известный объём стека, который функция выделяет в прологе инструкцией вроде sub rsp, N. Он отличается от dynamic allocation: размер фиксирован и может быть описан в UNWIND_INFO.

Stack slot (ячейка стека) - конкретное место в памяти стека, где сохранено значение: например старый RBX, старый RBP или локальная переменная. Для unwind важны именно slots, где лежат сохранённые nonvolatile-регистры и return address.

Restore (восстановить) - в контексте unwind значит не "откатить программу", а записать в ContextRecord значение регистра, прочитанное из места, где callee его сохранила для caller-а.

```
Упрощённый пример тела функции:

callee prolog:
push rbx
sub rsp, 40h
... тело функции ...

UNWIND_INFO сообщает:
UWOP_PUSH_NONVOL RBX
UWOP_ALLOC_SMALL 40h

virtual unwind из тела:
1. Учитывает, что fixed allocation = 40h.
2. Находит stack slot, где лежит старый RBX.
3. Записывает старый RBX в ContextRecord.Rbx.
4. Вычисляет адрес return address.
5. Записывает return address в ContextRecord.Rip.
6. Обновляет ContextRecord.Rsp так, как будто callee вернулась.
```

Если RIP находится в прологе

Пролог - более опасный случай. Когда исключение или прерывание происходит в прологе, функция ещё не обязательно успела построить полный кадр. Например, первая инструкция push rbx уже могла выполниться, а sub rsp, 80h ещё нет. Или наоборот, stack allocation уже произошёл, а сохранение какого-то регистра ещё нет. Если unwinder слепо применит все unwind operations, он восстановит контекст так, будто весь пролог выполнен, хотя это может быть неправдой.

Поэтому в UNWIND_INFO есть SizeOfProlog, а в каждом UNWIND_CODE есть CodeOffset. Эти значения позволяют определить, какие действия пролога уже должны считаться выполненными на момент текущего RIP. Если RIP находится внутри пролога, RtlVirtualUnwind должна отменить только те эффекты, которые действительно могли уже произойти. Это одна из самых тонких причин, почему unwind metadata хранит не просто

"функция сохраняет RBX и выделяет 0x80", а ещё связывает эти действия с offsets внутри пролога.

Здесь особенно важна формулировка из документации: для пролога unwinder использует offset текущего RIP от начала функции и сканирует unwind code array, чтобы найти границу действий. Это не просто произвольная эвристика. Это следствие того, что CodeOffset кодирует конец инструкции, выполняющей действие, плюс один, то есть позицию начала следующей инструкции. Поэтому если текущий RIP ещё не дошёл до этой позиции, соответствующее действие нельзя считать полностью выполненным.

SizeOfProlog (размер пролога) - поле UNWIND_INFO, задающее длину пролога в байтах. Если offset текущего RIP от начала функции меньше или равен этой длине, unwinder рассматривает ситуацию как нахождение в прологе.

CodeOffset (смещение кода в прологе) - поле UNWIND_CODE, указывающее offset конца инструкции, выполняющей unwind-значимое действие, плюс 1. Оно нужно, чтобы понять, какие действия пролога уже произошли, если RIP находится внутри пролога.

Partial execution (частичное выполнение) - ситуация, когда функция успела выполнить только часть пролога или часть эпилога. Для unwind это критично: нельзя отменять действия, которые ещё не были выполнены, и нельзя второй раз отменять то, что эпилог уже отменил.

Типичная ошибка парсера: видеть UWOP_ALLOC_SMALL и всегда прибавлять allocation size при unwind, не проверяя, находится ли RIP в середине пролога. В теле функции это нормально, а в прологе может быть неверно: инструкция sub rsp, N могла ещё не выполниться.

Если RIP находится в эпилоге

Эпилог - другой особый случай. В теле функции кадр полностью построен. В прологе кадр ещё строится. А в эпилоге кадр уже разбирается. Если исключение или прерывание происходит в эпилоге, часть работы выхода уже могла быть сделана: RSP мог быть увеличен, nonvolatile-регистры могли быть восстановлены через pop, и до ret осталось несколько инструкций. Здесь нельзя просто применить полную обратную схему пролога, потому что функция уже сама начала эту схему выполнять.

На x64 обычная unwind info не перечисляет каждый эпилог отдельно как отдельную таблицу действий. Вместо этого эпилоги должны иметь строгую форму, чтобы unwinder мог распознать их сканированием кода вперёд от текущего RIP. Это важный компромисс дизайна: в функции может быть много выходов, но хранить подробное описание каждого выхода дорого. Поэтому Microsoft ограничивает допустимые формы эпилога. Если код соответствует шаблону, RtlVirtualUnwind может симулировать оставшийся хвост эпилога и получить caller context.

Симуляция эпилога означает, что unwinder смотрит на инструкции выхода и виртуально применяет их эффект к ContextRecord. Если следующая инструкция add rsp, 40h, ContextRecord.Rsp увеличивается. Если дальше pop rbx, значение по текущему RSP записывается в ContextRecord.Rbx, а RSP увеличивается на 8. Если дальше ret, значение по RSP становится новым RIP, а RSP снова увеличивается. Всё это происходит без реального исполнения кода целевой программы.

Epilogue marker (маркер эпилога) - допустимая инструкция или форма выхода, по которой unwinder распознаёт, что текущий RIP находится в эпилоге. Для x64 документация

перечисляет ret, ret n, rep ret и ограниченные формы jmp.

Simulated execution (симулированное выполнение) - вычисление эффекта инструкций на ContextRecord без реального выполнения этих инструкций процессором. В эпилоге RtlVirtualUnwind симулирует оставшиеся pop/add/ret/jmp-подобные действия.

ret (return, возврат из функции) - машинная инструкция, которая берёт return address из стека и передаёт управление caller-у. В virtual unwind эффект ret моделируется: RIP берётся из памяти по RSP, а RSP увеличивается.

```
Пример эпилога:

add rsp, 40h
pop rbx
ret

Если ControlPc указывает на add rsp, 40h, virtual unwind симулирует:

RSP = RSP + 40h
RBX = [RSP]
RSP = RSP + 8
RIP = [RSP]
RSP = RSP + 8

Программа при этом не продолжает выполнение. Меняется только ContextRecord.
```

Именно поэтому прологи и эпилоги в Microsoft x64 ABI так строго ограничены. Ограничение формы кода нужно не для красоты, а для возможности надёжно распознать и смоделировать кадр в любой точке выполнения.

Leaf function и тривиальная раскрутка

Теперь нужно вспомнить leaf function. Если RtlLookupFunctionEntry не находит запись функции, алгоритм x64 unwind может предположить, что код относится к leaf function. В такой модели функция не изменила nonvolatile-регистры и не построила сложный stack frame. Значит caller можно получить проще: RSP указывает прямо на return address. Тогда virtual unwind сводится к имитации одиночного ret: RIP берётся из [RSP], а RSP увеличивается на 8.

Это полезное и необходимое правило, но его нельзя превращать в магию. Отсутствие RUNTIME_FUNCTION не доказывает, что перед нами хорошая leaf function. Оно только заставляет unwinder использовать leaf-логику, если другой информации нет. В корректно скомпилированном обычном коде это работает для настоящих leaf functions. В повреждённом, обфусцированном, вручную написанном или неправильно зарегистрированном коде отсутствие записи может означать проблему. Stack trace тогда может сломаться, потому что функция на самом деле меняла RSP, но метаданных для восстановления нет.

Для reverse engineering это различие очень важно. В отчёте нельзя писать: "записи нет, значит функции нет". И нельзя автоматически писать: "записи нет, значит всё нормально, это leaf". Корректнее: "для данного ControlPc не найдена function table entry; системная leaf-логика предполагает trivial unwind, но это безопасно только если код действительно не создаёт нетривиальный кадр".

Leaf function (листовая функция) - функция, которая в контексте Microsoft x64 calling convention не требует pdata/xdata для раскрутки: она не меняет nonvolatile-регистры и может быть

unwound простой имитацией return.
Trivial unwind (тривиальная раскрутка) - упрощённый unwind без UNWIND_INFO: взять return address из [RSP], записать его в RIP и увеличить RSP на 8. Это соответствует одиночному ret.
NULL FunctionEntry (нулевая запись функции) - отсутствие найденной RUNTIME_FUNCTION для данного ControlPc. В контексте unwind это может привести к leaf/trivial logic, но для анализа защищённого кода такое отсутствие требует осторожной интерпретации.

RtlVirtualUnwind не ищет запись функции заново

Новичок часто смешивает RtlLookupFunctionEntry и RtlVirtualUnwind в одну туманную операцию "система раскручивает стек". На уровне общей картины это терпимо, но для проекта static /dynamic такое смешение вредно. RtlVirtualUnwind получает FunctionEntry уже готовой. Она не должна заново искать .pdata по ControlPc как главный шаг. Её задача начинается после lookup.

Это разделение важно и архитектурно, и практически. Lookup отвечает за поиск диапазона в active function tables. Virtual unwind отвечает за применение найденной записи к ContextRecord. Благодаря этому можно отдельно тестировать две части анализатора. Первая часть проверяет: корректно ли найден RUNTIME_FUNCTION для RIP_RVA? Вторая часть проверяет: можно ли по найденному UNWIND_INFO восстановить caller context? Ошибка в первой части даст неправильную запись. Ошибка во второй части даст неправильную интерпретацию правильной записи.

В реальной Windows эти функции часто используются подряд, поэтому в голове они слипаются. Но книга должна удерживать границу: RtlLookupFunctionEntry - это поиск описания функции. RtlVirtualUnwind - это расчёт предыдущего контекста по этому описанию.

Как FunctionEntry ведёт к UNWIND_INFO

Когда RtlVirtualUnwind получает FunctionEntry, она не получает сразу готовый список действий в удобном виде. RUNTIME_FUNCTION содержит UnwindInfoAddress - RVA, который нужно интерпретировать относительно ImageBase. То есть снова всплывает связка, которую мы уже много раз видели: RUNTIME_FUNCTION даёт адрес не сам по себе, а как image-relative pointer. Чтобы получить реальный адрес UNWIND_INFO в памяти процесса, нужно сделать ImageBase + UnwindInfoAddress.

Дальше начинается чтение UNWIND_INFO. Из него берётся Version, Flags, SizeOfProlog, CountOfCodes, FrameRegister и FrameOffset. Затем читается массив UNWIND_CODE. Если CountOfCodes нечётный, массив всё равно выравнивается до чётного количества slots, потому что после него могут лежать handler RVA, language-specific data или chained RUNTIME_FUNCTION. Если Flags указывают на handler, RtlVirtualUnwind может вычислить HandlerData и вернуть pointer на обработчик. Если Flags указывают на chained info, нужно перейти к связанной записи и учитывать её metadata.

Эта цепочка показывает, почему маленькая ошибка в RVA conversion ломает всё. Если UnwindInfoAddress принять за file offset, RtlVirtualUnwind-подобный анализатор начнёт читать не UNWIND_INFO, а случайные байты файла. Если забыть про alignment после UNWIND_CODE array, handler RVA будет прочитан не с той позиции. Если перепутать

chained info с handler data, анализатор может объявить обработчик там, где его нет.

UnwindInfoAddress (адрес unwind-информации) - поле RUNTIME_FUNCTION, содержащее RVA структуры UNWIND_INFO или связанной unwind-информации. Для получения VA нужно прибавить ImageBase; для offline file parsing нужно отдельно перевести RVA в file offset.

Alignment (выравнивание) - требование располагать данные по границам определённого размера. В UNWIND_INFO выравнивание slots важно, потому что после массива кодов могут идти дополнительные поля. Неверное выравнивание приводит к чтению неправильных данных.

Chained info (цепочная unwind-информация) - режим, при котором UNWIND_INFO ссылается на другую RUNTIME_FUNCTION-like запись. Это нужно для фрагментов функции или переиспользования unwind-описания. Для RtlVirtualUnwind это означает: одного блока UNWIND_INFO может быть недостаточно.

HandlerType и возвращаемый handler

RtlVirtualUnwind не только вычисляет caller context. Она ещё участвует в механике exception dispatch, где нужно понять, есть ли у текущей функции обработчик. Поэтому у функции есть HandlerType и возвращаемое значение типа PEXCEPTION_ROUTINE. Это место часто воспринимается туманно: "она возвращает handler". Но что это значит?

В UNWIND_INFO есть Flags. Среди них могут быть флаги, указывающие на exception handler или termination/unwind handler. Если функция имеет такой handler, после массива UNWIND_CODE лежит RVA обработчика и затем language-specific data. RtlVirtualUnwind может вернуть адрес routine, а через HandlerData дать указатель на данные, которые эта routine будет использовать. Но сама логика языка не находится в RtlVirtualUnwind. Например, для Microsoft C/C++ handler может быть __C_specific_handler, а его данные будут описывать области __try/__except/__finally, filters и scope table. Это уже следующий слой.

Поэтому корректнее говорить так: RtlVirtualUnwind подготавливает информацию для возможного language-specific handler-а и возвращает pointer на него, если он есть и соответствует рассматриваемому типу. Она не решает сама, какой catch в C++ выбрать, не вычисляет filter expression на уровне исходника и не "лечит" исключение. Она даёт dispatcher-у следующий объект для вызова.

Language-specific handler (языковой обработчик) - функция, которая знает правила конкретного языка/компилятора для обработки исключений или unwind-действий. Для C/SEH на Windows важным примером является __C_specific_handler.

Exception handler (обработчик исключения) - обработчик, который может участвовать в search phase и решить, подходит ли он для данного исключения. В UNWIND_INFO его наличие связано с флагом UNW_FLAG_EHANDLER.

Termination handler / unwind handler (завершающий обработчик / обработчик размотки) - обработчик, который выполняется при размотке, например для finally-like логики. В UNWIND_INFO его наличие связано с флагом UNW_FLAG_UHANDLER.

Handler RVA (RVA обработчика) - image-relative pointer на функцию-обработчик, лежащий после выровненного массива UNWIND_CODE, если соответствующий flag установлен.

Ошибка понимания: считать, что RtlVirtualUnwind "вызывает catch". Нет. Она может вернуть адрес handler-а и данные для него. Решение language-specific semantics происходит в

обработчике и диспетчере исключений, а не в самом формате RUNTIME_FUNCTION.

HandlerData: указатель не на магию, а на данные языка

HandlerData - выходной параметр, который легко назвать "данные обработчика" и быстро забыть. Но для понимания SEH на x64 это важная связка. Адрес handler-а сам по себе говорит только: "вызови такую-то функцию-обработчик". Но обработчику нужно знать, какие области исходной функции покрыты try-блоками, где находятся filters, какие cleanup-действия связаны с каким диапазоном. Эти сведения не универсальны для всех языков. Они зависят от компилятора и модели исключений.

Именно поэтому в документации это называется language-specific handler data. Формат этих данных не является общей универсальной частью UNWIND_INFO. Windows задаёт место, где эти данные лежат, и механизм передачи, но смысл данных определяет конкретный handler. Для __C_specific_handler это один мир, для другой runtime-системы может быть другой. На уровне твоего фундаментального труда важно не пытаться заранее разобрать все языковые форматы, а понять границу ответственности: UNWIND_INFO приводит к handler RVA и HandlerData, а дальше language-specific handler читает эти данные по своим правилам.

Для reverse engineering HandlerData часто интереснее, чем просто сам handler address. Адрес handler-а может быть одним и тем же для сотен функций, например __C_specific_handler. А вот HandlerData у каждой функции будет указывать на её конкретные scope records. Поэтому две функции могут иметь один и тот же handler routine, но разную таблицу областей. Если анализатор показывает только handler RVA и не показывает, где лежит HandlerData, он теряет половину смысла.

Language-specific data (языко-зависимые данные) - данные, формат которых определяется конкретным обработчиком и компилятором. Windows не обязана понимать их как C-код; она передаёт pointer handler-у, а handler уже интерпретирует содержимое.

Scope table (таблица областей) - компиляторная структура, которая может описывать диапазоны кода и связанные с ними обработчики, filters или finally-действия. Для SEH/C++ это важный слой поверх базового unwind.

__C_specific_handler (C-specific handler, C-специфический обработчик) - стандартный обработчик Microsoft для SEH/C-совместимой модели на x64. Он использует DispatcherContext и language-specific data, чтобы решить, какие __try/__except/__finally области относятся к текущему исключению.

EstablisherFrame: почему обработчику нужен якорь в стеке

EstablisherFrame в этом разделе мы вводим только настолько, насколько нужно для понимания RtlVirtualUnwind, потому что следующий раздел будет посвящён ему отдельно. Но полностью пройти мимо нельзя. Если у функции есть handler, handler должен быть связан не только с кодом функции, но и с конкретным stack frame этой функции. Один и тот же код функции может быть вызван много раз рекурсивно. У каждой активации будет свой stack frame. Handler должен понимать, с каким именно кадром он работает.

EstablisherFrame можно представить как якорь кадра. Это значение, которое помогает handler-у и dispatcher-у связать абстрактную информацию из .xdata с конкретной областью стека текущего вызова. Если frame pointer есть, вычисление может опираться на

него. Если frame pointer нет, база кадра выводится из RSP и фиксированной allocation-модели. В любом случае смысл один: обработчик не работает "вообще с функцией". Он работает с конкретным вызовом функции, а этот вызов живёт в конкретном месте стека.

Именно поэтому повреждённая unwind metadata может ломать не только stack trace. Она может привести к неправильному EstablisherFrame. Тогда handler получит неверный якорь и начнёт интерпретировать scope data или локальные привязки относительно неправильного кадра. Для обычного корректного кода это редкая проблема. Для обфусцированного или защищённого образца это уже предмет анализа.

Frame base (база кадра) - опорное значение, относительно которого можно рассуждать о данных конкретного stack frame. В разных контекстах таким якорем может быть RSP, frame pointer или вычисленный EstablisherFrame.
Stack frame instance (экземпляр кадра стека) - конкретная активация функции во время выполнения. Одна и та же функция может иметь много экземпляров кадра при рекурсии или повторных вызовах.

ContextPointers: зачем иногда нужны адреса сохранённых регистров

Параметр ContextPointers выглядит второстепенным, но он показывает ещё одну важную мысль: иногда мало знать восстановленное значение регистра. Иногда важно знать, откуда именно оно было восстановлено. Например, если RBX был сохранён в stack slot, RtlVirtualUnwind может не только записать старое значение в ContextRecord.Rbx, но и сообщить адрес stack slot, откуда это значение взято. Для обычного ручного понимания это не обязательно, но для отладчиков и низкоуровневых механизмов это полезно.

Представь отладчик, который хочет показать пользователю не просто "RBX в caller-е был таким", а связать это значение с памятью дампа. Или механизм, который должен потом обновить сохранённый nonvolatile-регистр в кадре. Тогда знание location становится важным. Значение отвечает на вопрос "что восстановлено?", а context pointer отвечает на вопрос "из какого места это восстановлено?".

Для твоего static на первом этапе не обязательно полностью эмулировать ContextPointers. Но в теоретической части важно объяснить, что такой слой существует. Он подчёркивает, что virtual unwind работает не только с числами, но и с происхождением этих чисел: часть значений берётся из регистров текущего ContextRecord, часть - из stack memory, часть вычисляется из metadata.

Location (местоположение значения) - адрес или источник, откуда восстановлено значение регистра. В debugging это может быть важно не меньше, чем само значение.
Nonvolatile context pointer (указатель на сохранённый nonvolatile-контекст) - элемент структуры ContextPointers, который может указывать на память, где лежит сохранённое значение nonvolatile-регистра.

Почему RtlVirtualUnwind изменяет ContextRecord, а не создаёт новый объект

С точки зрения программирования было бы удобно думать, что RtlVirtualUnwind принимает current context и возвращает caller context как отдельный новый объект. Но API устроен иначе: ContextRecord передаётся как in/out. Это значит, что он изменяется на месте. После вызова в нём уже не прежний RIP и RSP, а значения предыдущего кадра.

Такое устройство хорошо соответствует циклу stack walking. Есть один ContextRecord, который постоянно продвигается вверх по стеку. Сначала он описывает место падения или текущую точку исключения. После первого unwind он описывает caller. После второго - caller of caller. И так далее. Каждый шаг перезаписывает контекст новым уровнем. Поэтому, если анализатор хочет сохранить всю цепочку кадров, он должен копировать нужные значения до следующего вызова, иначе предыдущий кадр будет потерян.

Это кажется мелкой API-деталью, но она формирует правильное мышление. RtlVirtualUnwind - это не функция "прочитать метаданные". Это функция "продвинуть контекст на один кадр". Если представить ContextRecord как курсор в call stack, то RtlVirtualUnwind двигает этот курсор вверх.

In/out parameter (параметр ввода-вывода) - параметр, который функция и читает, и изменяет. ContextRecord в RtlVirtualUnwind именно такой: на входе current frame, на выходе caller frame.

Cursor (курсор обхода) - удобная метафора: объект, показывающий текущую позицию в последовательности. ContextRecord при stack walking можно понимать как курсор, который перемещается от кадра к кадру.

Псевдокод: не реализация Windows, а учебная модель

Теперь соберём всё в псевдокод. Это не точная внутренняя реализация Windows. Нельзя выдавать его за исходники ntdll или kernel. Но как учебная модель он полезен: показывает порядок вопросов, которые должен задать unwinder. Где находится RIP? Есть ли FunctionEntry? Где UNWIND_INFO? Это пролог, эпилог или тело? Есть ли handler? Как обновить ContextRecord?

Такая модель особенно полезна для static /dynamic, потому что проекту не обязательно полностью повторять Windows dispatcher. Но проект должен понимать, какие проверки имеют смысл. Если metadata не позволяет даже построить такую логическую цепочку, это серьёзный finding. Если цепочка строится, но противоречит дизассемблированному прологу, это другой finding. Если FunctionEntry указывает за пределы доступных секций, цепочка обрывается ещё раньше.

```
Учебная модель одного вызова RtlVirtualUnwind:

input:
HandlerType
ImageBase
ControlPc
FunctionEntry
ContextRecord

if FunctionEntry == NULL:
// leaf / trivial model
ContextRecord.Rip = *(uint64_t*)ContextRecord.Rsp
ContextRecord.Rsp += 8
return NULL

UnwindInfo = ImageBase + FunctionEntry->UnwindInfoAddress
Read UNWIND_INFO header
```

```
Read UNWIND_CODE array

if ControlPc is inside recognized epilog:
simulate remaining epilog instructions
update ContextRecord to caller
return NULL

else if ControlPc is inside prolog:
undo only prolog effects that already happened
update ContextRecord to caller
return NULL

else:
if UNWIND_INFO flags indicate requested handler type:
set HandlerData
set EstablisherFrame
return LanguageHandler

undo full prolog effects
update ContextRecord to caller
return NULL
```

Этот псевдокод не должен подменять официальную документацию и реальные edge cases. Его задача - удержать смысл. Реальный Windows unwinder учитывает дополнительные детали архитектуры, chained info, epilog recognition, context pointers и режимы dispatch/unwind.

Пример: как одно и то же UNWIND_INFO применяется по-разному

Возьмём маленькую функцию. Она сохраняет RBX, выделяет 0x40 байт и затем работает в теле. Её пролог можно представить так: push rbx; sub rsp, 40h. В UNWIND_INFO это будет описано примерно как UWOP_PUSH_NONVOL для RBX и UWOP_ALLOC_SMALL для 0x40. Теперь представим три ситуации.

Первая ситуация: RIP находится в теле функции после пролога. Тогда оба действия уже выполнены. Virtual unwind должен учитывать и сохранение RBX, и allocation. Он найдёт старый RBX в стеке, восстановит его в ContextRecord.Rbx, снимет выделение стека и дойдёт до return address.

Вторая ситуация: RIP находится сразу после push rbx, но до sub rsp, 40h. Тогда RBX уже сохранён, но allocation ещё не произошло. Если unwinder применит оба действия, он сдвинет RSP слишком далеко и прочитает return address не там. Правильная логика должна отменить только push-сохранение, но не allocation.

Третья ситуация: RIP находится в эпилоге на pop rbx после add rsp, 40h. Тогда fixed allocation уже снят самой функцией. Если применить обычную логику тела и ещё раз снять allocation, получится двойной сдвиг. Правильная логика должна симулировать остаток эпилога от текущей точки, а не заново применять полный набор unwind operations.

```
Один и тот же пролог:

push rbx ; CodeOffset = 1
sub rsp, 40h ; CodeOffset = 5

Но три разных ControlPc дают разные действия:

1. ControlPc в теле:
undo alloc 40h + restore RBX + read return address

2. ControlPc между push и sub:
```

```
restore/pop-effect for RBX only, allocation 40h не трогать

3. ControlPc в эпилоге:
simulate remaining epilog, не применять body-unwind вслепую
```

Почему это критично для исключений, а не только для красивого stack trace

Легко подумать, что virtual unwind нужен в основном отладчику, чтобы показать человеку список вызовов. Это правда, но не вся правда. Механизм обработки исключений тоже должен проходить по кадрам. Когда исключение возникло, Windows должна понять, есть ли в текущей функции обработчик. Если нет, нужно перейти к caller-у. Потом к caller caller-а. И так далее, пока не найдётся подходящий handler или пока исключение не станет unhandled.

Чтобы перейти к следующему кадру, dispatcher должен получить caller context. Именно здесь virtual unwind становится частью exception handling. Если кадр восстановлен неверно, dispatcher может пропустить правильный handler, вызвать неправильный handler, неправильно вычислить EstablisherFrame или вообще потерять стек. Поэтому битая UNWIND_INFO - это не косметическая проблема, а потенциальный разрыв всей модели исключений.

На x64 это особенно важно, потому что SEH больше не строится на цепочке EXCEPTION_REGISTRATION_RECORD в стеке, как классический x86 frame-based SEH. Здесь система сильно опирается на табличные данные. Поэтому корректность RtlVirtualUnwind-подобного понимания является центральной для твоей темы: PE metadata перестаёт быть пассивным описанием и становится частью runtime control flow.

Exception dispatcher (диспетчер исключений) - системная логика, которая проходит по кадрам, ищет подходящие handlers и управляет фазами обработки исключения. Он использует unwind-механизм, чтобы переходить от текущего кадра к предыдущему.
Unhandled exception (необработанное исключение) - ситуация, когда проход по кадрам не нашёл подходящий обработчик или обработка не была выполнена. Тогда включаются top-level/unhandled mechanisms, crash reporting или завершение процесса.
Runtime control flow (поток управления во время выполнения) - фактическое движение выполнения программы и системной обработки, включая вызовы, возвраты, исключения и переходы к handlers. Unwind metadata влияет на этот поток не как комментарий, а как рабочая информация.

Чем RtlVirtualUnwind отличается от обычного disassembly analysis

Дизассемблер показывает инструкции. Это необходимо, но недостаточно. Он может показать push rbx, sub rsp, 40h, mov [rsp+20h], rsi. Но RtlVirtualUnwind не обязана каждый раз делать полноценный disassembly пролога, чтобы понять смысл кадра. Она опирается на UNWIND_INFO, где этот смысл уже записан компилятором и линкером.

Reverse engineer работает сразу с двумя источниками правды. Первый источник - код: реальные инструкции в .text. Второй источник - metadata: .pdata/.xdata. В нормальном бинаре они согласованы. Если пролог сохраняет RBX, metadata должна описывать сохранение RBX. Если metadata говорит о сохранении R12, а в коде такого нет, это

подозрительно. Если metadata описывает allocation 0x80, а пролог делает sub rsp, 20h, это тоже проблема. RtlVirtualUnwind доверяет metadata как части ABI-контракта; reverse engineer должен уметь проверить это доверие.

Именно поэтому твой static /dynamic не должен ограничиваться красивым выводом полей. Он должен строить мост между байтами кода и байтами unwind metadata. RtlVirtualUnwind даёт теоретическую цель этого моста: если metadata применить к ContextRecord, должен получиться корректный caller context. Любая несогласованность может привести к неверной раскрутке.

Disassembly analysis (анализ дизассемблированного кода) - изучение машинных инструкций после перевода байтов в ассемблерное представление. Для unwind этого мало: нужно сопоставлять инструкции с UNWIND_CODE.
Metadata trust (доверие к метаданным) - предположение runtime о том, что .pdata/.xdata корректно описывают пролог/кадр функции. В нормальном ABI это предположение нужно для скорости и надёжности; в reverse engineering его полезно проверять.

Где RtlVirtualUnwind может "сломаться" с точки зрения анализа

Строго говоря, сама Windows-функция не обязана "ломаться" так, как ломается учебный парсер. Она работает с тем, что ей дали. Но для анализатора существует несколько мест, где модель virtual unwind может стать недостоверной. Первое место - неверный FunctionEntry. Если lookup нашёл не ту запись или запись повреждена, дальнейший unwind идёт по чужим правилам.

Второе место - неверный UnwindInfoAddress. Если он указывает за пределы образа, в нечитабельную область, в середину другой структуры или в данные, которые не являются UNWIND_INFO, весь разбор становится бессмысленным. Третье место - несогласованность UNWIND_CODE с реальным прологом. Например, metadata говорит о сохранении nonvolatile-регистра в stack slot, но код не делает такого сохранения. Четвёртое место - некорректный epilog pattern. Если код выхода не соответствует разрешённым формам, unwinder может не распознать эпилог правильно.

Пятое место - dynamic/runtime-картина. Offline PE parser может видеть один набор function entries, но во время выполнения программа может зарегистрировать dynamic function table или сгенерировать код. Поэтому для статического отчёта важно честно обозначать границу: "проверена статическая Exception Directory исходного образа; runtime-registered tables не наблюдались". Это не слабость, а корректность формулировки.

Inconsistency (несогласованность) - ситуация, когда разные источники информации противоречат друг другу: например пролог говорит одно, а UNWIND_CODE - другое. Для static это один из главных типов findings.
Finding (обнаружение / диагностический вывод) - зафиксированное анализатором наблюдение: ошибка, подозрительная аномалия, несоответствие или важная особенность. Хороший finding должен иметь evidence: адреса, поля, байты и объяснение риска.
Evidence (доказательная база finding-а) - конкретные данные, на которых основан вывод: RVA, VA, bytes, имя секции, значение поля, диапазон функции, offset в UNWIND_INFO, сопоставление с инструкцией.

Какие findings логично строить вокруг RtlVirtualUnwind

Теперь переведём теорию в язык твоего проекта. Если static /dynamic хочет быть не просто PE viewer, а анализатором unwind-надёжности, он должен задавать вопросы, близкие к RtlVirtualUnwind. Не "есть ли .xdata вообще", а "можно ли по этой .xdata восстановить caller context?" Это более сильный критерий.

Первый класс findings - structural validity. Сюда попадает всё, что мешает даже прочитать metadata: FunctionEntry выходит за границы Exception Directory, UnwindInfoAddress не переводится в допустимый offset, CountOfCodes не помещается в доступный диапазон, handler RVA лежит не там, chained info указывает на невозможную запись. Такие findings говорят: virtual unwind может не иметь корректного входа.

Второй класс - semantic consistency. Metadata читается, но её смысл не похож на код: UWOP_PUSH_NONVOL нет в прологе, CodeOffset не соответствует границам инструкций, allocation size не совпадает с sub rsp, SET_FPREG описан, но frame pointer фактически не устанавливается, epilog не соответствует разрешённому шаблону. Такие findings говорят: вход есть, но доверять ему опасно.

Третий класс - runtime limitation. Static parsing показывает корректный PE, но есть признаки manual mapping, JIT-like code, packed sections или runtime-generated code paths, где статическая .pdata не покрывает всю картину. Это не обязательно ошибка, но это ограничение уверенности. В отчёте его нужно формулировать честно.

```
Примеры findings вокруг RtlVirtualUnwind-модели:

[STRUCTURAL]
- RUNTIME_FUNCTION.UnwindInfoAddress points outside image
- UNWIND_INFO.CountOfCodes exceeds available bytes
- handler RVA is not inside readable/executable image range
- chained RUNTIME_FUNCTION has invalid Begin/End range

[SEMANTIC]
- UWOP_ALLOC_SMALL size does not match prolog sub rsp, imm
- UWOP_PUSH_NONVOL register not found in prolog prefix
- CodeOffset points into middle of unrelated instruction
- epilog sequence is not a legal x64 epilog pattern

[RUNTIME LIMITATION]
- static .pdata covers only image code, but sample uses dynamic code generation
- packed code region has executable memory without static function entries
```

Почему нельзя "добавить RtlVirtualUnwind" простым чтением .xdata

Иногда возникает соблазн сказать: "мы разобрали .xdata, значит мы реализовали unwind". Это неверно. Разбор .xdata - это только чтение описания. RtlVirtualUnwind - это применение описания к конкретному контексту. Ему нужен не абстрактный файл, а конкретный RIP, конкретный RSP, конкретная память стека, конкретные значения сохранённых регистров и понимание, в какой зоне функции находится ControlPc.

Offline-анализатор может хорошо проверить metadata и даже построить symbolic model: если RSP такой-то, то return address будет по такому-то offset; если RBX сохранён, то он должен лежать там-то. Но без реального stack memory он не восстановит фактический caller RIP из конкретного запуска. Это нормальное ограничение. В статическом проекте

задача другая: доказать, что metadata структурно и семантически пригодна для unwind, а не обязательно выполнить полный runtime unwind конкретного потока.

Поэтому в ВКР важно аккуратно формулировать уровень реализации. Если проект читает PE и проверяет .pdata/.xdata, он делает static unwind metadata analysis. Если он принимает CONTEXT и память процесса/дампа и реально вычисляет caller frames, он приближается к stack walker. Это разные уровни сложности. Оба полезны, но смешивать их нельзя.

Static unwind metadata analysis (статический анализ unwind-метаданных) - проверка .pdata/.xdata без выполнения программы и без реального ContextRecord. Такой анализ отвечает на вопрос: корректно ли описан потенциальный unwind?
Runtime stack walker (обходчик стека во время выполнения) - механизм, который имеет доступ к ContextRecord и памяти стека и реально вычисляет последовательность caller frames. Это уже ближе к работе отладчика или системного runtime.
Symbolic model (символическая модель) - описание зависимости без конкретных runtime-значений: например "return address находится по адресу RSP + 0x58" вместо чтения реального адреса из памяти.

Минимальная ручная трассировка одного virtual unwind

Чтобы закрепить смысл, полезно пройти один ручной сценарий. Пусть функция имеет диапазон BeginAddress..EndAddress, а ControlPc находится в её теле. RtlLookupFunctionEntry уже вернула FunctionEntry и ImageBase. В RUNTIME_FUNCTION лежит UnwindInfoAddress. Мы прибавляем ImageBase и читаем UNWIND_INFO. Видим: пролог сохраняет RBX через push и выделяет 0x40 байт стека.

Теперь смотрим на текущий ContextRecord. В нём есть RSP, который указывает на нижнюю часть текущего кадра. Из metadata мы знаем, что выше fixed allocation лежит сохранённый RBX, а ещё выше - return address. Поэтому virtual unwind может вычислить адрес stack slot для RBX, прочитать оттуда значение, записать его в ContextRecord.Rbx, затем вычислить положение return address, прочитать его и записать в ContextRecord.Rip. После этого ContextRecord.Rsp должен указывать уже за return address, как после настоящего ret.

В реальности детали offset-ов зависят от конкретного порядка push/save/alloc и правил UWOP_* операций. Но общий смысл не меняется: metadata говорит, где искать сохранённые части caller context, а RtlVirtualUnwind переносит эти части обратно в ContextRecord.

```
Абстрактная ручная трассировка:

Дано:
Context.Rip = FunctionBegin + 0x120 ; тело функции
Context.Rsp = 00000000_0012FDC0
UNWIND_INFO:
UWOP_ALLOC_SMALL 40h
UWOP_PUSH_NONVOL RBX

Идея восстановления:
saved_rbx_addr = вычисляется по RSP и unwind layout
return_addr = вычисляется после снятия frame layout

После virtual unwind:
Context.Rbx = *(uint64_t*)saved_rbx_addr
```

```
Context.Rip = *(uint64_t*)return_addr
Context.Rsp = return_addr + 8
```

Не запоминай конкретные offsets из этого абстрактного примера как универсальные. Универсальна не цифра, а метод: читать UNWIND_INFO, учитывать порядок операций, позицию ControlPc и layout кадра.

Связь с search phase и unwind phase

В следующей главе мы будем подробно разбирать фазы обработки исключения: search phase и unwind phase. Но RtlVirtualUnwind уже заранее подводит нас к этим понятиям. В search phase система ищет подходящий обработчик. Для этого она идёт по кадрам и спрашивает: есть ли handler у текущей функции? Если нет или handler отказался, нужно перейти к caller frame. Переход к caller frame снова требует virtual unwind.

В unwind phase, когда обработчик уже выбран и нужно реально размотать стек до нужного кадра, система снова должна проходить по кадрам. Но теперь важны termination handlers, cleanup-действия и корректное вычисление EstablisherFrame/DispatcherContext. То есть RtlVirtualUnwind не принадлежит только одной фазе как отдельная игрушка. Она является низкоуровневым механизмом движения между кадрами, а разные фазы используют это движение с разной целью.

Эта мысль помогает не путать слои. RtlVirtualUnwind - это механизм одного шага. Exception dispatcher - это управляющая логика, которая решает, сколько шагов делать, кого вызывать и когда остановиться. Language-specific handler - это слой, который интерпретирует данные конкретного языка. Все три слоя связаны, но они не одно и то же.

Search phase (фаза поиска) - первая фаза обработки исключения, где система проходит по кадрам и ищет подходящий handler, ещё не выполняя полную размотку до выбранного обработчика.
Unwind phase (фаза размотки) - фаза, где система уже разматывает стек до выбранного обработчика и выполняет нужные cleanup/termination actions.
Dispatcher layer (слой диспетчера) - управляющая логика exception handling, которая использует lookup, virtual unwind и language-specific handlers для принятия решения по исключению.

RtlVirtualUnwind глазами reverse engineer-a

Для reverse engineer-a RtlVirtualUnwind - это не просто API, которое можно вызвать из C-кода. Это модель того, как Windows будет воспринимать кадр. Если ты видишь функцию в дизассемблере, вопрос не только "что она делает?". Вопрос ещё и "как Windows сможет выйти из её кадра при исключении или stack walk?". Ответ находится в согласовании кода и metadata.

Если бинарь защищён, обфусцирован или собран необычным тулчейном, unwind metadata может использоваться как поверхность для трюков. Иногда её повреждают. Иногда делают минимальной. Иногда создают dynamic tables. Иногда реальный control flow не похож на красивую линейную функцию. Иногда epilog-like последовательности появляются в неожиданных местах. Задача reverse engineer-a - не паниковать, а разложить ситуацию на слои: что говорит .pdata, что говорит .xdata, что говорит

disassembly, что было бы нужно RtlVirtualUnwind, чтобы пройти этот кадр.

В хорошем отчёте по такой функции должна быть не только строка "unwind info invalid". Нужно объяснить, на каком уровне invalid: lookup не находит entry; entry есть, но UnwindInfoAddress плохой; UNWIND_INFO читается, но codes невозможны; codes возможны, но не соответствуют прологу; handler есть, но HandlerData указывает в странную область; epilog не распознаётся как legal pattern. Тогда вывод становится инженерным, а не эмоциональным.

Obfuscation (обфускация) - намеренное усложнение кода или метаданных, чтобы затруднить анализ. В контексте unwind это может проявляться через нестандартные прологи, странные секции, динамические таблицы или несогласованность metadata.
Toolchain (инструментальная цепочка сборки) - компилятор, ассемблер, линкер и связанные инструменты, которые создают машинный код и metadata. Разные toolchains могут по-разному оформлять unwind-информацию, оставаясь в рамках ABI.
Legal pattern (разрешённый шаблон) - форма кода, которую ABI и unwinder признают допустимой. Для x64 epilog это особенно важно: не всякая последовательность, похожая на выход, является legal epilog для unwind.

Что важно вынести в static/dynamic

Если переносить этот раздел в архитектуру static/dynamic, то RtlVirtualUnwind задаёт направление развития. Даже если первая версия проекта не делает настоящий runtime unwind с CONTEXT и памятью стека, она может проверять пригодность метаданных для такого unwind. Это уже ценно. Проект может сказать: "если Windows попробует применить virtual unwind к этому диапазону, она будет опираться на такие-то данные; эти данные выглядят корректно/сомнительно/повреждённо по таким-то причинам".

Хороший модуль анализа должен разделять несколько уровней уверенности. Первый: function lookup possible - можно найти RUNTIME_FUNCTION по RVA. Второй: unwind info readable - можно прочитать UNWIND_INFO и массив codes. Третий: unwind operations valid - коды имеют допустимые opcodes, slot counts и payloads. Четвёртый: code/metadata consistent - пролог и metadata согласованы. Пятый: handler linkage plausible - handler RVA и HandlerData лежат в допустимых областях и соответствуют flags. Шестой: runtime limitations - есть ли признаки, что статической картины недостаточно.

Такой подход делает проект зрелым. Он не обещает невозможного, не притворяется Windows kernel debugger, но и не сводится к выводу таблицы. Он объясняет, насколько metadata пригодна для той самой операции, ради которой существует: для virtual unwind.

```
Уровни проверки static/dynamic вокруг virtual unwind:

1. Lookup layer:
RIP_RVA попадает в ровно одну RUNTIME_FUNCTION-запись.

2. Address layer:
UnwindInfoAddress корректно переводится в доступную область образа.

3. Structure layer:
UNWIND_INFO header, CountOfCodes, alignment и optional fields читаются без выхода за границы.

4. Operation layer:
Каждый UWOP_* имеет допустимый OpInfo, slot count и payload.
```

```
5. Semantic layer:
UNWIND_CODE согласуется с реальным прологом и допустимыми эпилогами.

6. Handler layer:
Handler RVA, HandlerData и chained info интерпретируются в соответствии с Flags.

7. Confidence layer:
Отчёт честно различает static evidence и runtime-only behavior.
```

Разбор параметров как цепочки ответственности

Теперь полезно ещё раз пройти параметры RtlVirtualUnwind не списком, а как цепочку ответственности. HandlerType говорит, какой тип handler-а сейчас интересует диспетчерской логике. ImageBase говорит, от какой базы интерпретировать image-relative values. ControlPc говорит, в какой точке функции находится управление. FunctionEntry говорит, какая RUNTIME_FUNCTION-запись описывает эту точку. ContextRecord несёт текущее состояние регистров и принимает новое состояние. HandlerData и EstablisherFrame получают сведения, нужные handler-у. ContextPointers позволяют сохранить информацию о местах, откуда были восстановлены nonvolatile-регистры.

Если убрать любой из этих элементов, картина станет неполной. Без HandlerType непонятно, какого обработчика мы ищем. Без ImageBase нельзя превратить RVA из RUNTIME_FUNCTION в VA. Без ControlPc нельзя определить зону функции: пролог, тело или эпилог. Без FunctionEntry нельзя найти UNWIND_INFO. Без ContextRecord нечего раскручивать. Без HandlerData и EstablisherFrame handler может быть найден как адрес, но не получит полноценный контекст для работы. ContextPointers можно опустить, но тогда теряется дополнительная информация о происхождении восстановленных регистров.

Эта цепочка хороша тем, что она дисциплинирует анализатор. Если проект строит статическую модель RtlVirtualUnwind, он должен явно понимать, какие элементы у него есть, а какие отсутствуют. Например, offline PE analysis имеет ImageBase из заголовков, RUNTIME_FUNCTION и UNWIND_INFO, но не имеет настоящего runtime ContextRecord и stack memory. Значит он может проверить пригодность metadata, но не всегда может восстановить фактический caller RIP конкретного потока. Это не ошибка проекта, если она честно названа.

Responsibility chain (цепочка ответственности) - способ смотреть на параметры API как на последовательность ролей. Один параметр задаёт базу адресов, другой задаёт текущую точку выполнения, третий задаёт описание функции, четвёртый хранит изменяемый контекст.
Offline analysis (анализ без запуска) - разбор файла или дампа без живого исполнения процесса. В таком режиме можно проверить структуру и согласованность unwind metadata, но нельзя всегда выполнить полный runtime unwind без ContextRecord и памяти стека.
Runtime analysis (анализ во время выполнения) - анализ, где есть живой процесс или достаточно полный дамп: можно иметь реальные VA, ContextRecord, stack memory и registered dynamic function tables.

Почему порядок обратных действий не всегда совпадает с порядком чтения

UNWIND_CODE array хранится в порядке убывания CodeOffset. Это уже было разобрано, но в контексте RtlVirtualUnwind нужно почувствовать, зачем это нужно практически. Пролог исполняется от начала функции вперёд: push, sub, save, set frame и так далее. А unwind часто должен двигаться как бы назад: от состояния, полученного после пролога, к состоянию до вызова. Поэтому metadata упорядочена так, чтобы unwinder мог удобно определить, какие действия применимы, особенно когда RIP находится внутри пролога.

Но нельзя механически сказать: "массив в обратном порядке, значит всегда просто делаем всё сверху вниз". В теле функции обычно нужно учитывать полный набор действий. В прологе нужно учитывать только действия, которые уже успели произойти относительно ControlPc. При chained info нужно после одного массива перейти к другому. При operations с payload slots нужно отличать собственно операцию от дополнительных slots. То есть порядок чтения байтов, порядок логических операций и порядок восстановления ContextRecord - связанные, но не тождественные вещи.

Для ручного анализа это особенно важно. Когда ты видишь CountOfCodes = 5, это не значит, что есть пять независимых операций. Некоторые операции занимают два или три slots. Если принять payload за отдельный UWOP, можно придумать несуществующее действие и испортить всю модель RSP. RtlVirtualUnwind-подобный анализатор должен сначала построить нормализованный список операций, где payload уже прикреплён к своей операции, и только потом применять семантику.

Normalized unwind operation (нормализованная unwind-операция) - удобное внутреннее представление parser-a, где один UWOP_* уже объединён со своими payload slots и превращён в одну логическую операцию: например "save RBX at offset 0x30".
Payload slot (слот полезной нагрузки) - дополнительный 2-байтовый slot после основной UNWIND_CODE-операции. Он не является самостоятельной операцией, а хранит extra data: размер allocation или offset сохранённого регистра.
Application order (порядок применения) - порядок, в котором unwinder применяет смысл unwind-операций к ContextRecord. Его нельзя путать с физическим порядком байтов в .xdata без учёта prolog/epilog/body case.

```
Плохая модель:
for each slot in UnwindCodeArray:
treat slot as independent operation

Более правильная модель:
1. Прочитать CountOfCodes slots.
2. Объединить операции с payload slots.
3. Проверить допустимость opcode/OpInfo/payload.
4. Определить положение ControlPc: prolog/body/epilog.
5. Применить только те логические операции, которые нужны для этого случая.
```

Подробный случай с frame pointer

С frame pointer virtual unwind становится менее очевидным, потому что часть адресов может вычисляться не напрямую от текущего RSP. В простейшей функции без frame pointer достаточно знать, насколько RSP был уменьшен и где лежат сохранённые регистры. Но если функция использует frame register, например RBP или R13, то UNWIND_INFO сообщает не только факт установки frame pointer, но и scaled FrameOffset. Это нужно для восстановления логической базы fixed allocation.

Когда в прологе выполняется lea rbp, [rsp+20h], frame pointer начинает указывать не обязательно на начало кадра, а часто внутрь выделенной области. Это делается ради удобства адресации: короткие смещения в инструкциях могут доставать больше локальных slots. Но для unwinder-а это означает: offsets в UWOP_SAVE_NONVOL и похожих операциях могут интерпретироваться относительно базы, связанной с frame pointer. Нельзя просто взять текущий RSP и прибавить offset без проверки FrameRegister.

RtlVirtualUnwind должна учитывать, был ли frame pointer уже установлен. В теле функции - да, если пролог полностью выполнен. В середине пролога - возможно, ещё нет. Поэтому снова всплывает CodeOffset: если RIP находится до UWOP_SET_FPREG, нельзя делать вид, что frame pointer уже является рабочей базой unwind. Если RIP находится после установки frame pointer, offsets могут использовать его как якорь. Это ещё один пример, почему partial prolog logic не является второстепенной деталью.

Frame pointer based addressing (адресация через frame pointer) - способ обращаться к stack slots относительно устойчивого регистра кадра, а не только относительно RSP. На x64 это не всегда используется, но важно для dynamic allocation и некоторых layout-решений компилятора.

FrameOffset scaling (масштабирование FrameOffset) - правило, по которому поле FrameOffset в UNWIND_INFO умножается на 16. Это позволяет компактно описать положение frame pointer внутри fixed stack allocation.

Effective frame base (эффективная база кадра) - практическая база, от которой parser/unwinder считает offsets для сохранённых регистров. Она зависит от того, есть ли FrameRegister и был ли он уже установлен на момент ControlPc.

Ошибка ручного анализа: увидеть FrameRegister = RBP и автоматически считать, что все stack offsets надо считать от RBP в любой точке функции. Внутри пролога RBP мог ещё не быть установлен. Внутри эпилога RSP мог уже быть перестроен. Context matters.

Разница между "восстановить регистр" и "доверять текущему регистру"

Nonvolatile-регистры создают важное различие. Если callee по calling convention обязана сохранить RBX, это не значит, что текущий ContextRecord.Rbx внутри callee уже равен значению caller-a. Callee могла использовать RBX для своих нужд после сохранения старого значения в стеке. Поэтому при unwind из тела функции старое значение нужно восстановить из сохранённого места. Текущее значение RBX в ContextRecord может быть уже новым рабочим значением callee, а не тем, что ожидает caller.

А вот volatile-регистры обычно не восстанавливаются как часть caller context. Calling convention говорит caller-у: если тебе важен RAX, RCX, RDX, R8, R9, R10, R11 или volatile XMM-регистры, сохраняй сам. Callee не обязана возвращать их прежними. Поэтому UNWIND_INFO в основном заботится о nonvolatile-регистрах и RSP. RtlVirtualUnwind не пытается сделать магически полный "исторический снимок" всех регистров caller-a. Она восстанавливает то, что ABI гарантирует и metadata описывает.

Это важная граница. Если в crash dump ты видишь старый кадр и хочешь узнать, каким был RAX у caller-a до вызова, unwind metadata обычно не даст надёжного ответа. А вот для RBX/RBP/RSI/RDI/R12-R15, если они были сохранены и корректно описаны, шансы есть.

Поэтому stack walking и register reconstruction - не одно и то же.

Register reconstruction (реконструкция регистров) - восстановление значений регистров для предыдущего кадра. Unwind metadata хорошо подходит для nonvolatile-регистров, но не обещает восстановить всю историю volatile-регистров.
ABI guarantee (гарантия ABI) - правило calling convention, на которое может опираться код и runtime. Например, callee обязан сохранить nonvolatile-регистры, если изменяет их.
Volatile loss (потеря volatile-состояния) - нормальная ситуация, когда значения volatile-регистров caller-а уже нельзя восстановить через unwind, потому что callee имела право их перезаписать.

Память стека как второй источник данных

UNWIND_INFO говорит, где должны лежать сохранённые значения, но сами значения находятся в памяти стека. Поэтому virtual unwind нельзя свести к чтению одной .xdata. Если нужно восстановить RBX, unwinder должен прочитать 8 байт из stack memory. Если нужно получить return address, он должен прочитать 8 байт по вычисленному RSP. Если память недоступна, повреждена или не входит в дамп, восстановление становится частичным или невозможным.

В живом процессе Windows может читать память потока, потому что она находится внутри того же адресного пространства и работает как системный runtime. В offline crash dump всё зависит от того, какие страницы памяти попали в дамп. В minidump может не быть всех страниц стека или всех регионов кода. Тогда stack walker может остановиться не потому, что metadata плохая, а потому что нет данных для чтения. Это отдельный класс ограничений.

Для твоего проекта это означает: если анализатор статический, он не должен обещать восстановление конкретных register values. Он может сказать, где они должны быть. Если анализатор работает с дампом, он должен различать "metadata говорит, что значение лежит по такому адресу" и "память по такому адресу реально доступна и прочитана". Это разные уровни evidence.

Stack memory (память стека) - реальные байты в памяти, на которые указывает RSP и где лежат return addresses, сохранённые регистры, locals и служебные slots. Без stack memory можно проверить layout, но нельзя прочитать фактические значения.
Minidump (минидамп) - компактный дамп процесса, который может содержать не всю память. При анализе unwind это важно: metadata может быть доступна, а нужный stack slot может отсутствовать.
Partial unwind (частичная раскрутка) - ситуация, когда можно вычислить часть модели кадра, но нельзя восстановить полный caller context из-за отсутствия metadata, stack memory или допустимого FunctionEntry.

RtlVirtualUnwind и безопасность чтения памяти

Любой unwinder в какой-то момент читает память по адресам, вычисленным из RSP, frame pointer и metadata. Это значит, что теоретически он должен быть осторожен. Если metadata повреждена и говорит, что сохранённый RBX лежит по огромному offset, наивный анализатор может попытаться читать за пределами доступной области. Если RSP

повреждён, return address может указывать в невалидную память. Если ImageBase неверный, UNWIND_INFO будет читаться не оттуда.

Windows runtime работает в своих условиях и имеет свои проверки, но для твоего анализатора практическое правило простое: каждое чтение должно быть проверяемым. Перед чтением UNWIND_INFO нужно убедиться, что RVA переводится в область файла или образа. Перед чтением stack slot в дампе нужно убедиться, что address попадает в доступный memory range. Перед чтением handler data нужно проверить alignment и границы. Анализатор не должен падать от плохого бинаря. Плохой бинарь должен становиться finding-ом.

Это особенно важно для dynamic -части. Если проект будет анализировать защищённые или подозрительные файлы, нельзя доверять значениям из файла как добросовестным. Любой CountOfCodes, любой UnwindInfoAddress, любой chain target может быть намеренно выбран так, чтобы сломать простой parser. Поэтому robust parsing - часть безопасности самого инструмента.

Bounds checking (проверка границ) - проверка, что чтение структуры не выходит за пределы доступного буфера, секции, image range или memory range. Для unwind parser-а это базовая защита от повреждённых и злонамеренных данных.
Robust parsing (устойчивый парсинг) - стиль разбора, при котором некорректные данные не приводят к падению анализатора, а превращаются в диагностируемую ошибку с понятным evidence.
Malformed binary (повреждённый или некорректно сформированный бинарь) - файл, чьи структуры нарушают ожидаемые правила формата. В reverse engineering это может быть случайная порча, нестандартный тулчейн или намеренная защита.

RtlVirtualUnwind2: почему важно знать о новой форме, но не смешивать темы

В документации Microsoft существует и RtlVirtualUnwind2. Для нашей текущей линии важно не уходить в отдельную новую главу, но упомянуть это полезно. Наличие второй версии показывает, что тема virtual unwind не является исторической мелочью. Microsoft расширяет и уточняет API для более сложных сценариев, дополнительных параметров и архитектурных деталей.

Однако в фундаментальной базе по Windows x64 SEH/unwind сначала нужно хорошо понять RtlVirtualUnwind как классическую точку входа. Если ученик плохо понимает, что такое ContextRecord, FunctionEntry, HandlerData и EstablisherFrame, переход к RtlVirtualUnwind2 только добавит тумана. Поэтому здесь мы фиксируем: RtlVirtualUnwind2 существует, может иметь дополнительные возможности и детали, но conceptual core остаётся тем же - виртуальное восстановление caller context по metadata и текущему состоянию.

Для проекта это тоже практично. Если первая версия static ориентируется на PE32+ x64 metadata и классическую модель RUNTIME_FUNCTION/UNWIND_INFO, не нужно сразу притворяться полным совместимым unwind runtime со всеми современными расширениями. Лучше качественно реализовать базовую модель, правильно описать ограничения и заложить архитектуру, куда позже можно добавить поддержку

расширений.

RtlVirtualUnwind2 (вторая версия API виртуальной раскрутки) - более новая форма Windows API для virtual unwind. В этой книге она важна как указание на развитие механизма, но базовое понимание строится вокруг классической RtlVirtualUnwind и x64 unwind procedure.

Conceptual core (концептуальное ядро) - главный смысл механизма, который сохраняется несмотря на API-детали: получить caller context из current context, function entry и unwind metadata.

Контрольные вопросы для самого себя при чтении любого кадра

Когда ты смотришь на конкретную функцию в Ghidra, IDA, WinDbg или своём static -е, полезно задавать один и тот же набор вопросов. Они переводят RtlVirtualUnwind из абстрактной API-функции в практический метод мышления.

Первый вопрос: какой ControlPc мы рассматриваем? Это VA внутри живого процесса, RVA внутри образа или offset файла? Второй вопрос: какая RUNTIME_FUNCTION покрывает этот адрес? Третий вопрос: куда ведёт UnwindInfoAddress? Четвёртый вопрос: где находится ControlPc относительно функции - в прологе, теле или эпилоге? Пятый вопрос: какие действия пролога уже применимы? Шестой вопрос: где лежит return address? Седьмой вопрос: какие nonvolatile-регистры нужно восстановить? Восьмой вопрос: есть ли handler flags, HandlerData и EstablisherFrame? Девятый вопрос: есть ли признаки, что статическая картина не совпадает с runtime-картиной?

Если ответов на эти вопросы нет, значит разговор об unwind ещё не закончен. Если ответы есть и они согласованы, ты уже не просто "прочитал структуру". Ты понял, как Windows могла бы пройти через этот кадр.

```
Мини-чеклист RtlVirtualUnwind-мышления:

[1] ControlPc: VA, RVA или file offset?
[2] FunctionEntry: какая RUNTIME_FUNCTION покрывает адрес?
[3] ImageBase: от какой базы считаются image-relative fields?
[4] UNWIND_INFO: читается ли структура полностью?
[5] Region: prolog, body или epilog?
[6] Operations: какие UWOP_* реально применимы в этой точке?
[7] Stack: где return address и saved nonvolatile registers?
[8] Handler: есть ли EHANDLER/UHANDLER/CHAININFO?
[9] Confidence: static evidence достаточно или нужны runtime данные?
```

Короткое, но точное резюме раздела

RtlVirtualUnwind - это функция одного шага вверх по стеку. Она берёт текущий ContextRecord и по FunctionEntry/UNWIND_INFO вычисляет, как должен выглядеть ContextRecord предыдущего кадра. Она не ищет функцию заново, не исполняет код программы, не строит весь stack trace за один вызов и не решает сама всю language-specific логику исключений.

Её главная работа - виртуально отменить или досимулировать состояние текущей функции. Если RIP в теле, применяется полная модель кадра. Если RIP в прологе, отменяются только уже выполненные действия пролога. Если RIP в эпилоге, симулируется оставшаяся часть эпилога. Если записи функции нет и применима leaf-логика, выполняется тривиальная имитация return. Если есть handler flags, функция может вернуть handler pointer, HandlerData и EstablisherFrame для дальнейшей работы

dispatcher-а.

Для reverse engineering эта функция важна как эталонный вопрос: "смогла бы Windows корректно выйти из этого кадра?" Всё, что мы читали в .pdata, .xdata, RUNTIME_FUNCTION, UNWIND_INFO и UNWIND_CODE, в итоге должно служить этому вопросу. Следующий раздел естественно продолжает тему: если RtlVirtualUnwind может вычислять EstablisherFrame, нужно понять, что это за значение, почему оно нужно handler-у и почему его нельзя сводить к простому RBP.

Переход к следующему разделу: EstablisherFrame - это не просто "ещё один указатель". Это связь между обработчиком, конкретным stack frame и language-specific data. Без него handler видел бы код функции, но не имел бы надёжного якоря для текущего вызова.

5.4. Что такое EstablisherFrame

После разбора RtlVirtualUnwind может возникнуть ощущение, что основная работа уже понятна: функция взяла текущий ContextRecord, прочитала UNWIND_INFO, восстановила caller context - и всё. Но это только половина правды. Для обычного stack trace, где нам нужно просто получить цепочку адресов вызова, такого понимания почти хватает. А для настоящего механизма исключений Windows этого мало, потому что исключение - это не просто подъём по стеку. Исключение ещё должно найти конкретный обработчик и передать ему информацию так, чтобы обработчик понял: "это мой кадр, мои данные, моя область try/finally/except, мои локальные переменные и мой участок функции".

Именно здесь появляется EstablisherFrame. Это слово неприятное даже для человека, который уже не боится .pdata, UNWIND_INFO и RtlVirtualUnwind. Оно выглядит как очередной системный термин, который в документации описан одной сухой строкой. Но на деле это один из самых важных мостов между двумя мирами: с одной стороны, есть табличная unwind-модель, где всё описано через RUNTIME_FUNCTION, .xdata и машинные диапазоны; с другой стороны, есть языковой обработчик исключения, которому нужно понять конкретный вызов функции в конкретном стеке.

Если сказать совсем просто, EstablisherFrame - это значение, которое связывает найденный обработчик не просто с функцией как участком кода, а с конкретным экземпляром этой функции на стеке. Одна и та же функция может быть вызвана тысячу раз, может быть рекурсивной, может существовать в разных потоках, может иметь один и тот же RUNTIME_FUNCTION и один и тот же HandlerData. Но у каждого живого вызова будет свой stack frame. Когда система вызывает language-specific handler, ей нужно передать не абстрактное "у этой функции есть обработчик", а "вот конкретный кадр, который установил этот обработчик". Это и есть роль EstablisherFrame.

EstablisherFrame (establisher frame, устанавливающий кадр / кадр-установщик обработчика) - значение, которое идентифицирует конкретный stack frame функции, установившей exception handler или termination handler. В документации Microsoft для __C_specific_handler оно описано как адрес базы fixed stack allocation этой функции. Это не имя функции, не адрес кода, не адрес UNWIND_INFO и не просто "RBP". Это привязка обработчика к конкретному кадру выполнения.
Establisher (установивший / установивший обработчик) - слово от глагола establish: установить, учредить, сделать действующим. В контексте исключений frame established a handler означает: этот конкретный вызов функции является тем кадром, с которым связан обработчик.
Frame (кадр стека) - область состояния конкретного вызова функции: return address, сохранённые nonvolatile-регистры, локальный стековый блок, иногда frame pointer и дополнительные служебные данные. Важно: frame - это не просто одна инструкция и не вся функция в PE-файле, а конкретный экземпляр выполнения на стеке.
Главная мысль раздела: EstablisherFrame нужен не для того, чтобы найти функцию. Функцию уже нашли через RUNTIME_FUNCTION. EstablisherFrame нужен, чтобы найденный handler был связан с правильным экземпляром функции на стеке.

Почему после RtlVirtualUnwind вообще нужен ещё какой-то frame

В предыдущем разделе мы рассматривали RtlVirtualUnwind как механизм, который делает один шаг вверх по стеку. Она обновляет ContextRecord так, чтобы он стал похож на контекст caller-a. Но RtlVirtualUnwind делает ещё одну важную вещь: если текущая функция имеет обработчик, она помогает подготовить данные для вызова этого обработчика. Среди этих данных есть HandlerData и EstablisherFrame.

HandlerData отвечает на вопрос: "где лежат дополнительные данные обработчика?" Для C SEH это может вести к compiler-specific таблице областей, фильтров и обработчиков. Для C++ это могут быть другие данные языка. Формат этой области не универсален: он определяется конкретным language-specific handler. Но HandlerData всё ещё остаётся статическим или почти статическим указателем: он связан с кодом функции и её metadata.

EstablisherFrame отвечает на другой вопрос: "к какому конкретному stack frame эти данные надо применить?" Без этого handler увидел бы только схему, но не имел бы точки привязки к живому вызову. Представь чертёж квартиры и реальную квартиру. HandlerData похож на чертёж: где какие области try, какие фильтры, какие offsets, какие правила. EstablisherFrame похож на адрес конкретной квартиры: именно с этим экземпляром чертёж сейчас связан.

Это особенно важно при рекурсии. Рекурсивная функция имеет одну запись в .pdata, одну UNWIND_INFO и один набор language-specific данных. Но каждый рекурсивный вызов имеет свой stack frame. Если на глубине 15 возникает access violation, обработчик должен быть связан именно с кадром глубины 15, а не с кадром глубины 14 и не с абстрактной функцией вообще. EstablisherFrame и даёт эту привязку.

```
Одна функция в PE:
.pdata -> одна RUNTIME_FUNCTION
.xdata -> одна UNWIND_INFO
HandlerData -> одни language-specific данные

Несколько вызовов во время выполнения:
call #1 -> свой stack frame -> свой EstablisherFrame
call #2 -> свой stack frame -> свой EstablisherFrame
call #3 -> свой stack frame -> свой EstablisherFrame

Вывод:
static metadata может быть общей,
но EstablisherFrame всегда относится к конкретному кадру выполнения.
```

EstablisherFrame нельзя путать с RBP

Самая частая ошибка - увидеть слово frame и сразу решить, что речь идёт о RBP. Это понятно: в учебных материалах по ассемблеру часто говорят, что RBP - это frame pointer, указатель кадра. Но в Windows x64 такая привычка быстро ломает понимание. На x64 многие функции вообще не используют RBP как frame pointer. Компилятор может использовать RBP как обычный nonvolatile-регистр, может выбрать другой frame register, может вообще обходиться без frame pointer, если фиксированный RSP-доступ достаточен.

EstablisherFrame - это не синоним RBP. Иногда вычисляемое значение может быть связано с frame pointer, если функция его использует. Иногда оно ближе к RSP после фиксированного выделения стека. В документации для __C_specific_handler смысл формулируется как address of the base of the fixed stack allocation for this function - адрес

базы фиксированного стекового выделения этой функции. Поэтому безопасное правило такое: EstablisherFrame надо понимать не как конкретный регистр, а как runtime-значение базы кадра, передаваемое обработчику.

Почему это важно? Потому что reverse engineer часто смотрит на дизассемблер и хочет быстро сказать: "Ага, RBP есть, значит EstablisherFrame = RBP". Иногда это может быть близко к правде, но как общее правило это неверно. Для корректного анализа нужно смотреть на UNWIND_INFO, FrameRegister, FrameOffset, UWOP_SET_FPREG, stack allocation и фактический контекст. EstablisherFrame появляется не из имени регистра, а из unwind-семантики кадра.

RBP (base pointer register, регистр базового указателя) - регистр общего назначения x64, исторически часто применявшийся как frame pointer. На Windows x64 он является nonvolatile-регистром, но не обязан быть указателем кадра в каждой функции.

Frame pointer (указатель кадра) - выбранный регистр, который внутри функции служит стабильной базой для доступа к данным кадра. На x64 его наличие описывается через FrameRegister и UWOP_SET_FPREG, но EstablisherFrame не сводится механически к имени этого регистра.

Fixed stack allocation (фиксированное выделение стека) - заранее известный участок стека, выделенный функцией в прологе. Он называется fixed, потому что его размер известен из unwind-метаданных, в отличие от динамического alloca. EstablisherFrame для __C_specific_handler связан именно с базой этого фиксированного участка.

Нельзя писать в анализаторе правило "EstablisherFrame = RBP". Это будет работать только на части функций и сломается на оптимизированном x64-коде, на функциях без frame pointer, на функциях с другим frame register и на вариантах с dynamic allocation.

Чем EstablisherFrame отличается от stack frame

Есть ещё одна тонкая ловушка. Stack frame - это широкое понятие: весь кадр функции в стеке. В него входят return address, сохранённые регистры, локальные переменные, shadow space, выравнивание, иногда область для outgoing arguments, иногда dynamic allocation. EstablisherFrame же не означает "весь кадр". Это конкретное значение, которое handler получает как базу для своей логики.

Можно сказать так: stack frame - это территория, а EstablisherFrame - это опорная точка на этой территории. Территория может быть сложной: сверху return address, ниже сохранённые регистры, ниже локальные данные, потом дополнительные временные области. Но handler-у нужен не полный географический атлас, а стабильная точка отсчёта, относительно которой compiler-generated код сможет понимать свои данные.

С точки зрения языка это похоже на ситуацию с объектом и указателем на объект. Объект может занимать много памяти, иметь поля и внутренние структуры. Но функция получает один pointer, через который она к этому объекту обращается. EstablisherFrame играет похожую роль: он не является всеми байтами кадра, но даёт точку, от которой конкретный handler может интерпретировать кадр.

Stack frame (кадр стека) - вся область состояния одного вызова функции в stack memory. Это понятие шире, чем EstablisherFrame.

Base of fixed stack allocation (база фиксированного стекового выделения) - опорный адрес фиксированной области кадра. Именно так Microsoft описывает EstablisherFrame в контексте __C_specific_handler. Эта база нужна handler-у как координата конкретного кадра.

Stack memory (память стека) - область виртуальной памяти потока, где хранятся return addresses, stack frames, локальные стековые данные и служебное состояние вызовов. У каждого потока свой стек.

Понятие	Что означает	Почему нельзя смешивать
Stack frame	Вся область состояния конкретного вызова функции в стеке.	Слишком широкое понятие. EstablisherFrame - не весь кадр, а опорное значение.
Frame pointer	Регистр, выбранный функцией как стабильная база доступа к кадру.	Может отсутствовать. Может быть не RBP. Не равен автоматически EstablisherFrame.
FrameRegister в UNWIND_INFO	Номер регистра, который unwind metadata считает frame pointer.	Это поле метаданных, а не runtime-значение кадра само по себе.
EstablisherFrame	Runtime-значение базы кадра, передаваемое обработчику.	Это значение конкретного вызова, а не статическое поле PE-файла.

Почему EstablisherFrame появляется именно рядом с handler-ами

Если функция не имеет exception handler и termination handler, EstablisherFrame обычно не интересует обычного наблюдателя. Для stack trace достаточно восстановить caller context: новый RIP, новый RSP и nonvolatile-регистры. Но когда функция имеет handler, простого перехода к caller-у недостаточно. Система должна дать handler-у возможность принять решение или выполнить cleanup.

В Windows x64 handler не является магической записью в цепочке стека, как в старой x86-модели SEH. Он описан в UNWIND_INFO через flags и address of exception handler. Когда установлен UNW_FLAG_EHANDLER или UNW_FLAG_UHANDLER, после массива unwind codes в .xdata появляется image-relative address language-specific handler-а и дополнительные language-specific data. Но handler должен работать не только со статическими данными. Он должен понимать, к какому кадру они относятся.

Поэтому в вызове language-specific handler-а EstablisherFrame стоит рядом с ExceptionRecord, ContextRecord и DispatcherContext. ExceptionRecord говорит, какое исключение произошло. ContextRecord говорит, в каком процессорном состоянии оно произошло или в каком состоянии идёт unwind. DispatcherContext даёт расширенный контекст диспетчера. EstablisherFrame говорит: "вот кадр функции, которая установила этот обработчик".

Exception handler (обработчик исключения) - код, который может рассмотреть исключение и решить, обрабатывать его или пропустить дальше. В C SEH это связано с __try/__except.

Termination handler (обработчик завершения / unwind handler) - код, который вызывается при размотке стека для cleanup: например __finally или деструкторы объектов в C++. Он не обязательно "лечит" исключение, но должен выполнить освобождение ресурсов.

Language-specific handler (языковой обработчик) - обработчик, формат данных и логика которого определяются языком и компилятором. Для C SEH в MSVC важную роль играет __C_specific_handler. Для C++ используются другие механизмы и данные.

```
Упрощённая сигнатура __C_specific_handler:

__C_specific_handler(
EXCEPTION_RECORD *ExceptionRecord,
void *EstablisherFrame,
CONTEXT *ContextRecord,
DISPATCHER_CONTEXT *DispatcherContext
);

Смысл набора параметров:
ExceptionRecord -> что произошло
EstablisherFrame -> с каким кадром связан этот handler
ContextRecord -> контекст исключения или текущий unwind-контекст
DispatcherContext -> данные диспетчера исключений по этому кадру
```

HandlerData и EstablisherFrame: статическая схема против живого кадра

Теперь важно развести два соседних понятия, которые легко перепутать: HandlerData и EstablisherFrame. Оба появляются рядом с RtlVirtualUnwind и обработчиками. Оба нужны language-specific handler-у. Но они отвечают на разные вопросы.

HandlerData - это указатель на дополнительные данные обработчика. Эти данные лежат рядом с UNWIND_INFO или логически связаны с ним. Их формат не определяется общей x64 unwind-моделью полностью: Microsoft прямо говорит, что language-specific handler data определяется конкретным handler-ом. То есть обычный unwinder не обязан понимать внутренности этих данных. Он должен найти их и передать handler-у.

EstablisherFrame - это не данные из .xdata. Это runtime-значение, вычисленное для текущего кадра. Оно не лежит как постоянная в PE-файле, потому что заранее невозможно знать, где в памяти окажется конкретный стековый кадр. В одном запуске адрес будет один, в другом другой. В одном потоке один, в другом другой. В одном рекурсивном вызове один, в соседнем другой.

Поэтому HandlerData и EstablisherFrame образуют пару: HandlerData говорит, как интерпретировать обработчик, а EstablisherFrame говорит, к какой живой рамке выполнения применить эту интерпретацию. Без HandlerData handler не знает своей схемы. Без EstablisherFrame он не знает своего экземпляра кадра.

HandlerData (данные обработчика) - указатель на language-specific data, передаваемые обработчику. Это не сам EstablisherFrame и не stack frame. Обычно эти данные относятся к metadata функции.

Runtime value (значение времени выполнения) - значение, которое существует только в конкретном запуске программы: адрес stack frame, текущий RSP, текущий RIP, указатель на конкретный объект. EstablisherFrame является именно runtime value.

Static metadata (статические метаданные) - данные, заранее сгенерированные компилятором/линкером и лежащие в PE: .pdata, .xdata, RUNTIME_FUNCTION, UNWIND_INFO. HandlerData обычно ближе к этому миру, а EstablisherFrame - к миру живого выполнения.

Параметр	Где возникает	Что даёт handler-у
HandlerData	Из UNWIND_INFO/.xdata, если есть handler flags.	Схему: какие языковые данные связаны с обработчиком.
EstablisherFrame	Вычисляется при unwind для конкретного stack frame.	Экземпляр: к какому кадру применять эту схему.
ContextRecord	Приходит из состояния потока/исключения и меняется при unwind.	Процессорное состояние: RIP, RSP, регистры.
DispatcherContext	Формируется диспетчером исключений для текущего шага.	Служебный контекст: function entry, image base, control pc, scope/handler context и т.п.

Почему рекурсия идеально показывает смысл EstablisherFrame

Лучший способ почувствовать EstablisherFrame - посмотреть на рекурсию. Пусть есть функция parse_node, внутри которой есть __try/__except. Эта функция вызывает сама себя для дочерних узлов дерева. В PE-файле функция одна. В .pdata запись одна. В .xdata описание unwind одно. Address of exception handler один. HandlerData тоже может быть одно и то же. Но во время выполнения в стеке лежит несколько кадров parse_node.

Если исключение происходит в самой глубокой рекурсивной ветке, Windows начинает подниматься по стеку. На каждом уровне, где текущий ControlPc попадает в диапазон parse_node, RtlLookupFunctionEntry может возвращать одну и ту же RUNTIME_FUNCTION. Но EstablisherFrame будет разным, потому что каждый уровень рекурсии - это отдельный вызов и отдельная область fixed stack allocation.

Это разрушает опасную иллюзию: "если я нашёл функцию, значит я нашёл обработчик". На самом деле нужно найти не просто функцию, а конкретный кадр этой функции. Handler принадлежит функции как коду, но действует в рамках кадра как экземпляра выполнения. EstablisherFrame - это способ не потерять этот экземпляр.

```
Рекурсивный пример как модель:

parse_node(depth=0) -> EstablisherFrame A
parse_node(depth=1) -> EstablisherFrame B
parse_node(depth=2) -> EstablisherFrame C
parse_node(depth=3) -> EstablisherFrame D <-- здесь исключение

Статически:
RUNTIME_FUNCTION(parse_node) одна.
UNWIND_INFO(parse_node) одна.
HandlerData(parse_node) часто одна.

Динамически:
EstablisherFrame A, B, C, D разные.
```

Как EstablisherFrame связан с fixed stack allocation

Документация __C_specific_handler формулирует EstablisherFrame как адрес базы fixed stack allocation этой функции. Эту фразу нужно разобрать аккуратно, потому что она звучит просто, но в ней скрыто много смысла.

Fixed stack allocation - это та часть стекового кадра, размер которой известен из пролога и unwind-метаданных. Например, если функция делает sub rsp, 80h, то она выделяет 0x80 байт фиксированной локальной области. Если дополнительно она сохраняет nonvolatile-регистры через push или mov, эти сохранения тоже описаны в unwind operations. В совокупности unwind-модель может понять, где в кадре находится стабильная база.

EstablisherFrame не хранится в UNWIND_INFO как готовое число. В UNWIND_INFO нет поля "EstablisherFrame = ...". Есть описание действий пролога: выделение стека, сохранение регистров, установка frame pointer. RtlVirtualUnwind, имея текущий ContextRecord и это описание, может вычислить значение, которое будет передано как establisher frame pointer. Поэтому offline PE parser может увидеть, что функция имеет handler и какие unwind-операции участвуют в вычислении, но не может знать конкретное значение EstablisherFrame без runtime-контекста и stack memory.

Именно здесь важно разделять статический анализ файла и динамический анализ выполнения. В файле лежит рецепт. В процессе лежит приготовленное блюдо. EstablisherFrame относится к приготовленному блюду: он появляется только когда есть конкретный поток, конкретный RSP, конкретный вызов и конкретная позиция ControlPc.

Offline PE parser (офлайновый парсер PE-файла) - инструмент, который читает файл на диске или image в памяти без живого выполнения конкретного кадра. Он может разобрать .pdata/.xdata, но не знает адреса конкретного stack frame без ContextRecord и памяти стека.
Runtime unwinder (runtime-раскрутчик) - механизм, который работает с живым или сохранённым контекстом выполнения: регистрами, стеком, загруженными модулями и function tables. Он может вычислять значения вроде EstablisherFrame.
Stack bounds (границы стека) - допустимый диапазон памяти стека потока. Для надёжного анализа важно проверять, что EstablisherFrame и stack reads попадают в разумные границы, иначе повреждённый стек может породить ложный анализ.
В static /dynamic нельзя обещать пользователю "мы вычислили EstablisherFrame" только по байтам PE-файла. По PE можно сказать: функция имеет handler metadata и unwind-рецепт. Конкретный EstablisherFrame требует контекста выполнения: RSP/RIP, stack memory и результата unwind.

FrameRegister, FrameOffset и EstablisherFrame

В предыдущем разделе мы подробно разбирали FrameRegister и frame pointer. Теперь нужно соединить эту тему с EstablisherFrame. Если UNWIND_INFO указывает FrameRegister, это значит, что функция использует выбранный nonvolatile-регистр как frame pointer. Этот регистр устанавливается через UWOP_SET_FPREG: selected FP register = RSP + FrameOffset * 16. Это помогает обращаться к локальным данным, особенно когда есть dynamic stack allocation.

Но наличие FrameRegister не означает, что EstablisherFrame надо просто заменить значением этого регистра. FrameRegister - это часть способа адресации кадра внутри функции. EstablisherFrame - это значение, которое unwinder передаёт handler-у как базу fixed stack allocation. Эти понятия связаны, потому что оба говорят о геометрии кадра, но находятся на разных уровнях.

Можно представить кадр как систему координат. FrameRegister - это один из удобных указателей внутри этой системы координат. FrameOffset говорит, куда относительно RSP он поставлен. EstablisherFrame - это та опорная точка, которую Windows и language-specific handler используют для привязки обработчика к кадру. В простых случаях они могут оказаться близко связанными. В сложных случаях попытка механически приравнять одно к другому приводит к ошибкам.

FrameRegister (регистр кадра) - поле UNWIND_INFO, указывающее, какой nonvolatile-регистр используется как frame pointer. Если поле нулевое, функция не использует frame pointer в этой unwind-модели.
FrameOffset (смещение регистра кадра) - масштабированное поле UNWIND_INFO; выбранный frame pointer устанавливается как RSP + FrameOffset * 16. Это позволяет поместить указатель в середину fixed stack allocation для более коротких инструкций доступа.
UWOP_SET_FPREG (unwind operation set frame pointer register, операция установки регистра кадра) - unwind-операция, описывающая момент, когда пролог установил frame pointer. Она важна для восстановления геометрии кадра, но сама по себе не равна EstablisherFrame.

EstablisherFrame в search phase

Когда возникает исключение, Windows сначала выполняет search phase - фазу поиска обработчика. На этой фазе система поднимается по кадрам и спрашивает у потенциальных language-specific handler-ов: "ты хочешь обработать это исключение?" Для C SEH это означает, что могут вычисляться __except-фильтры. Фильтр может вернуть решение: обработать исключение, продолжить поиск или продолжить выполнение, если это допустимо для данного случая.

В этой фазе EstablisherFrame нужен, чтобы handler понимал, к какому кадру относится проверяемая область. Одна и та же функция может иметь несколько __try-блоков, вложенные области, фильтры, finally-блоки и разные состояния выполнения. HandlerData может описывать таблицу этих областей, но привязка к конкретному кадру идёт через EstablisherFrame и DispatcherContext.

Важно: search phase не означает, что стек уже уничтожается. На фазе поиска система ещё ищет, кто согласится обработать исключение. Поэтому ContextRecord в handler-е относится к exception context - состоянию на момент исключения. Но при этом текущий шаг обхода стека всё равно должен знать, какой кадр сейчас рассматривается. EstablisherFrame и даёт handler-у эту опору.

Search phase (фаза поиска) - первая фаза обработки исключения, где система проходит по кадрам и ищет обработчик, готовый принять исключение. На этой фазе кадры ещё не обязательно разматываются окончательно.
__except filter (фильтр __except) - выражение/код в C SEH, который решает, будет ли данный __except-блок обрабатывать исключение. Фильтр вызывается через compiler-generated механизм, а не как обычная произвольная функция из ниоткуда.
Exception disposition (решение обработчика) - результат, который сообщает системе, что делать дальше: обработать исключение, продолжить поиск, продолжить выполнение или выполнить другие действия, зависящие от модели handler-а.

EstablisherFrame в unwind phase

Если на search phase обработчик найден, начинается unwind phase - фаза размотки. Теперь Windows должна пройти от места исключения до кадра выбранного обработчика и вызвать termination handlers для кадров, которые покидаются. Сюда относятся __finally-блоки, C++ destructors и другие cleanup-механизмы. На этой фазе EstablisherFrame снова нужен, но смысл чуть смещается: handler теперь может не решать, обрабатывать ли исключение, а выполнять очистку ресурсов для конкретного кадра.

Представь функцию, которая открыла файл, взяла lock, выделила буфер и вошла в __try/__finally. Если исключение уводит выполнение наружу, __finally должен выполниться именно для того вызова функции, который держит этот lock и этот буфер. Статическая информация в .xdata говорит: у функции есть termination handler. Но какой именно lock освобождать и где лежат данные этого вызова - вопрос конкретного кадра. EstablisherFrame помогает связать cleanup с правильным frame instance.

Поэтому EstablisherFrame важен не только для "поймать исключение". Он важен для безопасного выхода из промежуточных функций. Если перепутать кадр, cleanup станет опасным: можно освободить не тот ресурс, обратиться не к той области стека, принять решение по чужому состоянию или разрушить инварианты программы.

Unwind phase (фаза размотки) - вторая фаза обработки исключения, где система реально выходит из кадров до найденного обработчика и вызывает termination handlers/cleanup для покидаемых кадров.
Cleanup (очистка ресурсов) - действия, которые должны быть выполнены при выходе из области: закрыть handle, освободить память, снять lock, вызвать destructor. В exception-safe коде cleanup особенно важен.
Frame instance (экземпляр кадра) - конкретный вызов функции в конкретном потоке и конкретном месте стека. Это динамический объект выполнения, а не статическая функция в PE.

Связь с __C_specific_handler

Для Windows reverse engineering особенно важно понимать __C_specific_handler, потому что он часто встречается в MSVC-коде, использующем Structured Exception Handling. Когда UNWIND_INFO содержит флаг UNW_FLAG_EHANDLER или UNW_FLAG_UHANDLER, после unwind codes может лежать RVA language-specific handler-а. Для C SEH этим handler-ом часто является __C_specific_handler.

__C_specific_handler получает четыре основных параметра: ExceptionRecord, EstablisherFrame, ContextRecord и DispatcherContext. Это очень показательный набор. ExceptionRecord объясняет, что случилось. ContextRecord показывает состояние процессора. DispatcherContext даёт системный контекст текущего шага. EstablisherFrame говорит, к какому кадру применяется handler. Если убрать EstablisherFrame из этой картины, handler остался бы с событием и метаданными, но без живой базы кадра.

Reverse engineer часто видит в дизассемблере не исходный __try/__except, а только признаки: UNW_FLAG_EHANDLER в UNWIND_INFO, адрес __C_specific_handler, scope table или похожие compiler-specific данные. EstablisherFrame при этом не лежит в файле. Он появится только во время dispatch/unwind. Поэтому для правильного анализа нужно

разделять: "в файле есть C-specific handler metadata" и "в runtime handler будет вызван с EstablisherFrame конкретного кадра".

__C_specific_handler (си-специфический обработчик, C-specific handler) - language-specific handler, который MSVC использует для реализации C Structured Exception Handling extensions. Он получает ExceptionRecord, EstablisherFrame, ContextRecord и DispatcherContext.

Structured Exception Handling / SEH (структурная обработка исключений) - модель Windows для обработки исключений через структурированные области, например __try/__except и __try/__finally в C/C++. На x64 она опирается на table-based unwind metadata, а не на простую цепочку записей в стеке как в x86.

Scope table (таблица областей) - compiler-specific структура, которая может описывать защищённые диапазоны, фильтры и обработчики. Общая x64 unwind-модель не задаёт её универсальный формат; её читает конкретный language-specific handler.

Microsoft прямо описывает параметр EstablisherFrame у __C_specific_handler как адрес базы fixed stack allocation функции. Это хороший якорь для понимания: EstablisherFrame - runtime-база кадра, а не поле .pdata и не имя регистра.

Почему x86-интуиция мешает понять x64

Если человек приходит к x64 SEH после старой x86-модели, у него может быть вредная интуиция: обработчики исключений лежат в цепочке записей в стеке, значит нужно искать linked list, FS:[0], next pointer и handler pointer. Для x86 SEH такая картина действительно была фундаментальной. Но Windows x64 устроена иначе. На x64 обработка исключений стала table-based: функция описана в .pdata/.xdata, а unwind и handler discovery идут через function tables и metadata.

Из-за этого EstablisherFrame особенно важен. В x86-модели сам stack registration record уже явно жил в стеке и был связан с конкретным кадром. В x64-модели handler metadata в основном вынесены в таблицы. Но системе всё равно нужно связать статический handler с конкретным runtime frame. EstablisherFrame помогает восстановить эту связь в table-based мире.

Можно сказать так: x64 убрала из стека явную SEH-chain запись в привычном x86-смысле, но не убрала необходимость знать, какой кадр установил обработчик. Просто теперь эта привязка выражается через результат unwind-анализа и передаётся handler-у как EstablisherFrame.

Table-based exception handling (табличная обработка исключений) - модель, где сведения о функциях, unwind и handler-ах хранятся в таблицах .pdata/.xdata, а не в простой runtime-цепочке записей внутри стека каждой функции.

Registration record (регистрационная запись обработчика) - характерный для x86 SEH объект в стеке, связывающий кадр с обработчиком. На x64 основная модель другая: связь восстанавливается через таблицы и unwind.

SEH chain (цепочка SEH) - в x86-контексте цепочка регистрационных записей обработчиков. В x64 reverse engineering нельзя механически переносить эту модель на table-based unwind.

Миниатюрный сценарий: access violation внутри __try

Теперь соберём картину в один сценарий. Пусть внутри функции worker есть __try-блок. Внутри этого блока происходит access violation. Процессор передаёт управление в системный механизм исключений. Windows получает ExceptionRecord и ContextRecord: известно, какой код упал и в каком состоянии были регистры.

Дальше система берёт ControlPc из контекста и вызывает RtlLookupFunctionEntry. Lookup находит RUNTIME_FUNCTION для worker. Из неё известно, где лежит UNWIND_INFO. Затем RtlVirtualUnwind смотрит на UNWIND_INFO, понимает, где находится ControlPc: в прологе, теле или эпилоге, и может подготовить данные для handler-а. Если у worker в UNWIND_INFO стоит UNW_FLAG_EHANDLER, будет найден language-specific handler, например __C_specific_handler. Вместе с ним появятся HandlerData и EstablisherFrame.

После этого __C_specific_handler получает ExceptionRecord, EstablisherFrame, ContextRecord и DispatcherContext. Он уже не просто знает "произошёл access violation". Он знает это в связке с конкретным кадром worker. Он может проверить, относится ли адрес ControlPc к protected range, какой filter нужно выполнить, какие данные scope table соответствуют этому месту, и какое решение вернуть диспетчеру.

```
Сценарий в сокращённом виде:

1. Исключение:
access violation внутри worker

2. Начальный контекст:
ExceptionRecord -> код и адрес исключения
ContextRecord -> RIP/RSP/registers на момент исключения

3. Поиск функции:
RtlLookupFunctionEntry(ControlPc)
-> RUNTIME_FUNCTION(worker), ImageBase

4. Виртуальный unwind/подготовка handler-а:
RtlVirtualUnwind(...)
-> HandlerData
-> EstablisherFrame
-> possible EXCEPTION_ROUTINE

5. Языковой обработчик:
__C_specific_handler(ExceptionRecord,
EstablisherFrame,
ContextRecord,
DispatcherContext)

Главное:
HandlerData говорит "какая схема обработчика".
EstablisherFrame говорит "какой конкретный кадр worker".
```

Почему EstablisherFrame важен для декомпилятора

Декомпилятор обычно пытается восстановить высокоуровневую структуру функции: где вход, где выходы, какие переменные, какие блоки, какие условные переходы. Но когда в функции есть SEH, обычного control flow graph недостаточно. Нужно ещё понять protected regions, filters, finally blocks и связь этих областей с x64 unwind metadata.

EstablisherFrame здесь напрямую не всегда отображается пользователю. В интерфейсе декомпилятора ты можешь увидеть __try, __except или комментарий о handler-е. Но внутренне инструмент должен понимать, что обработчик связан не просто с адресом handler function, а с frame instance. Особенно это важно для корректного отображения

локальных переменных внутри filters/finally. Если инструмент неверно понимает базу кадра, он может показать неправильные stack variables или вообще потерять связь между handler data и локальным состоянием.

Для reverse engineer это означает: когда видишь __C_specific_handler, scope table и странные ссылки на stack offsets, не нужно думать, что всё решается только чтением .xdata. Нужно понимать, что во время реального exception dispatch эти данные получают смысл через EstablisherFrame. Без него ты видишь схему, но не видишь, к какому live frame она применяется.

Decompiler (декомпилятор) - инструмент, который пытается поднять машинный код до псевдо-C или другой высокоуровневой формы. Для SEH ему нужно восстанавливать не только инструкции, но и структуру exception regions.
Control flow graph / CFG (граф потока управления) - представление функции как набора basic blocks и переходов между ними. Для обычной логики if/loop этого часто достаточно, но SEH добавляет скрытые пути через exception dispatch и unwind.
Protected region (защищённая область) - участок кода, находящийся под защитой обработчика исключения или cleanup-области. В исходном C это может соответствовать телу __try.

Почему EstablisherFrame важен для crash dump analysis

В crash dump analysis программа уже не выполняется. Есть снимок: память, регистры, загруженные модули, возможно часть стеков. Если стек повреждён, оптимизирован или содержит SEH, анализатору нужно очень аккуратно восстанавливать кадры. EstablisherFrame помогает понять, какой кадр был связан с handler-ом на конкретном шаге обхода.

Для обычного stack trace отладчик может показать цепочку адресов. Но когда нужно понять, почему обработчик был выбран или почему cleanup не выполнился, одной цепочки адресов мало. Нужно смотреть handler metadata, scope data, фазу обработки исключения, ContextRecord и frame association. EstablisherFrame является частью этой association.

Практически это важно при расследовании сложных падений: повреждение стека, исключение внутри finally, вложенное исключение в filter, падение при unwinding, corrupted scope data, нестандартный JIT-код, вручную зарегистрированные function tables. В таких случаях вопрос "какой кадр установил этот handler?" становится не академическим, а диагностическим. Неправильный ответ ведёт к неверному выводу о причине сбоя.

Nested exception (вложенное исключение) - исключение, возникшее во время обработки другого исключения, например внутри filter или cleanup-кода. Такие случаи особенно чувствительны к корректному контексту кадра.
Corrupted stack (повреждённый стек) - состояние, когда stack memory не соответствует ожидаемой структуре: испорчен return address, нарушены сохранённые регистры, записаны мусорные значения или повреждены границы кадра.
Frame association (сопоставление обработчика с кадром) - понимание, какой handler относится к какому конкретному stack frame. EstablisherFrame является ключевым

runtime-значением такого сопоставления.

Что может и не может сделать static /dynamic

Для твоего static /dynamic важно не превращать EstablisherFrame в мистический термин. Его можно разложить на две задачи: статическую и динамическую. Статически инструмент может найти функции с handler flags, показать адрес language-specific handler-a, выделить HandlerData, проверить согласованность UNWIND_INFO, обнаружить странные offsets, chained info, dynamic table hints и подозрительные несоответствия между .pdata и .xdata. Это всё можно делать по PE/image.

Но конкретное значение EstablisherFrame статически не существует. Его нельзя достать из файла как число. Его можно вычислить только в контексте stack walking: есть текущий RIP, текущий RSP, есть память стека, есть найденный FunctionEntry и результат применения unwind-семантики. Поэтому static должен честно различать: "potential establisher frame semantics" и "computed establisher frame value".

dynamic в свою очередь может использовать понимание EstablisherFrame для защиты и диагностики. Например, можно проверять, что функции с exception handlers имеют корректную unwind metadata; что handler RVA указывает в разумную секцию кода; что HandlerData не выходит за допустимые границы; что функции, которые выглядят как SEH-функции, не имеют сломанного chained info. Но если речь идёт о runtime verification, нужно подключать контекст выполнения или дамп: без него невозможно проверить конкретное значение EstablisherFrame.

Potential establisher frame semantics (потенциальная семантика establisher frame) - статическое понимание того, что у функции есть metadata, при которой во время unwind будет вычисляться EstablisherFrame. Это не само runtime-значение.
Computed establisher frame value (вычисленное значение establisher frame) - конкретный адрес, полученный при анализе живого контекста или дампа. Он зависит от RSP, RIP, stack memory и конкретного вызова.
Runtime verification (проверка во время выполнения) - анализ с доступом к реальному процессу, потоку, контексту или дампу, где можно проверять не только структуру PE, но и фактические значения регистров/стека.

Findings, которые имеет смысл выдавать

В отчёте static /dynamic можно формулировать выводы так, чтобы не обещать лишнего. Например: "Function has language-specific handler and may produce EstablisherFrame during exception dispatch" - функция имеет языковой обработчик и во время dispatch может дать EstablisherFrame. Это честный статический вывод.

Если есть дамп или runtime context, можно писать сильнее: "Computed EstablisherFrame = ...; value is inside thread stack bounds; aligned; consistent with fixed stack allocation". Здесь уже есть конкретное значение и можно проверять его качество. Но если значение вне стека, не выровнено, указывает в guard page или противоречит ожидаемой геометрии кадра, это может быть признаком corrupted stack, неправильной unwind metadata, manual mapping issues или намеренной обфускации.

```
Пример аккуратных findings:
```

```
[static]
Function 0000000140012000..00000001400124A0
UNW_FLAG_EHANDLER set
language handler RVA = 0002F430
handler data present after UNWIND_INFO
EstablisherFrame: not computable from PE alone

[runtime/dump]
Frame #04 ControlPc = 00000001400122D1
FunctionEntry found
EstablisherFrame = 000000A0F3BFE2C0
stack bounds check: OK
alignment check: OK
handler data RVA check: OK

[warning]
EstablisherFrame outside stack bounds
possible corrupted stack / invalid unwind metadata / nonstandard runtime mapping
```

Типичные ошибки понимания

Первая ошибка - считать EstablisherFrame адресом обработчика. Адрес обработчика берётся из UNWIND_INFO как address of exception handler, если выставлены соответствующие flags. EstablisherFrame - не код, а frame value.

Вторая ошибка - считать EstablisherFrame адресом UNWIND_INFO или HandlerData. UNWIND_INFO и HandlerData находятся в metadata области модуля. EstablisherFrame находится в runtime-мире и обычно указывает в стек конкретного потока.

Третья ошибка - считать его глобальным свойством функции. У функции один RUNTIME_FUNCTION, но много возможных runtime вызовов. Каждый вызов может дать своё значение.

Четвёртая ошибка - приравнивать его к RBP. RBP может участвовать, но не является универсальным ответом.

Пятая ошибка - игнорировать фазу обработки. В search phase и unwind phase ContextRecord и смысл вызова handler-а могут отличаться. EstablisherFrame остаётся привязкой к кадру, но окружающий контекст меняется.

Шестая ошибка - пытаться вычислить EstablisherFrame офлайново по PE и выдавать конкретный адрес. По PE можно восстановить рецепт, но не адрес конкретного кадра. Адрес появляется только с runtime context.

Ошибка	Почему неверно	Правильная мысль
EstablisherFrame = адрес handler-а	Handler code и frame value - разные сущности.	Адрес handler-а берётся из UNWIND_INFO; EstablisherFrame описывает кадр.
EstablisherFrame = RBP	RBP может не быть frame pointer, а frame pointer может отсутствовать.	Нужно смотреть unwind-семантику и runtime context.
EstablisherFrame лежит в PE	В PE лежит metadata, а конкретный frame зависит от стека.	PE даёт рецепт; runtime даёт значение.

Ошибка	Почему неверно	Правильная мысль
Один EstablisherFrame на функцию	Одна функция может иметь много вызовов и потоков.	Значение относится к конкретному frame instance.
HandlerData = EstablisherFrame	HandlerData - схема, EstablisherFrame - привязка к кадру.	Handler-у нужны оба вида информации.

Связь с безопасностью и обфускацией

В нормальном компиляторном коде EstablisherFrame обычно воспринимается как служебная деталь. Но для reverse engineering и защиты это место интересно, потому что оно находится на стыке трёх хрупких зон: stack integrity, unwind metadata integrity и exception handler control flow. Если обфускатор, packer или ручной loader ломает одну из этих зон, exception handling может начать вести себя нестабильно.

Например, если .pdata/.xdata не соответствуют реальному прологу, RtlVirtualUnwind может вычислить неправильную геометрию кадра. Тогда EstablisherFrame, HandlerData или caller context могут оказаться ошибочными. Если handler data указывает в странное место, language-specific handler может прочитать мусор. Если stack memory повреждена, даже корректная metadata не спасёт: unwinder будет читать неверные сохранённые значения.

Для dynamic это означает: проверка unwind metadata - не косметика. Это часть доверия к exception handling и debugging support. Microsoft прямо указывает, что отсутствие корректных function tables для динамически сгенерированного кода приводит к ненадёжной обработке исключений и отладке. EstablisherFrame как runtime-результат этой системы наследует все риски: если таблицы или стек плохие, handler получает плохую привязку к кадру.

Stack integrity (целостность стека) - соответствие памяти стека ожидаемой структуре вызовов: корректные return addresses, сохранённые регистры, frame layout и границы.
Metadata integrity (целостность метаданных) - соответствие .pdata/.xdata реальному машинному коду. Если metadata лжёт, unwinder будет применять неправильный рецепт.
Packer (упаковщик) - инструмент, который изменяет структуру исполняемого файла, часто сжимает/шифрует код и разворачивает его в runtime. Packer может ломать или имитировать unwind metadata.

Ручная модель: как думать об EstablisherFrame без магии

Чтобы EstablisherFrame перестал быть туманным словом, удобно держать простую ручную модель. Сначала есть статическая функция в модуле. У неё есть RUNTIME_FUNCTION и UNWIND_INFO. Затем есть конкретное выполнение этой функции: поток зашёл в неё, call положил return address, пролог построил кадр, функция начала работать. Если в этой функции есть handler, то metadata описывает, где handler и какие данные ему нужны. Но handler должен быть вызван не "для функции вообще", а для конкретного кадра.

Когда exception dispatcher доходит до этого кадра, RtlVirtualUnwind умеет получить не только следующий caller context, но и frame value, который можно передать handler-у. Это значение и называется EstablisherFrame. Оно говорит: "обработчик, сейчас речь вот об

этом кадре".

Если держать эту модель, термин становится намного спокойнее. EstablisherFrame - не новый вид таблицы, не скрытая секция PE, не отдельная инструкция и не особая магия компилятора. Это runtime-координата кадра, необходимая для того, чтобы language-specific handler правильно применил свои данные к конкретному месту стека.

Короткая формула для памяти: HandlerData = что делать по правилам языка; EstablisherFrame = к какому кадру это применить; ContextRecord = в каком состоянии процессор/поток; DispatcherContext = системная упаковка сведений для текущего шага.

Как этот раздел готовит DispatcherContext

Следующий раздел будет посвящён DispatcherContext. Теперь его будет проще понять. EstablisherFrame - это один важный runtime-адрес кадра. Но одному handler-у обычно нужно больше: FunctionEntry, ImageBase, ControlPc, HandlerData, история unwind, данные текущей фазы, иногда scope-related сведения. Всё это собирается в более широкую структуру контекста диспетчера.

Если EstablisherFrame отвечает на вопрос "какой кадр установил этот handler?", то DispatcherContext отвечает на вопрос "в каком системном контексте этот handler сейчас вызывается?". Поэтому EstablisherFrame не нужно воспринимать изолированно. Это часть набора данных, который связывает низкоуровневую табличную unwind-модель с высокоуровневой логикой обработки исключений.

Мы уже прошли путь: RIP -> RtlLookupFunctionEntry -> RUNTIME_FUNCTION -> UNWIND_INFO -> RtlVirtualUnwind -> caller context и handler outputs. Теперь EstablisherFrame добавил в эту цепочку очень важное звено: привязку handler-а к конкретному кадру. DispatcherContext в следующем разделе покажет, как Windows упаковывает вокруг этой привязки весь остальной служебный контекст.

DispatcherContext (контекст диспетчера) - структура/набор данных, который передаёт language-specific handler-у сведения о текущем шаге exception dispatch или unwind. В следующем разделе она будет разобрана отдельно и подробно.

Exception dispatcher (диспетчер исключений) - системный механизм Windows, который проходит по кадрам, вызывает language-specific handlers, принимает их решения и управляет search/unwind phases.

Handler outputs (выходные данные для обработчика) - значения вроде HandlerData, EstablisherFrame и возвращаемого EXCEPTION_ROUTINE, которые появляются при virtual unwind и нужны для вызова handler-а.

EstablisherFrame и локальные переменные: почему тут легко обмануться

Когда говорят, что EstablisherFrame связан с базой fixed stack allocation, у новичка быстро появляется следующая мысль: "значит через него можно сразу восстановить локальные переменные". Это опасное упрощение. Локальные переменные в оптимизированном x64-коде - не такие аккуратные сущности, как в учебнике по C. Часть локальных значений может вообще не жить в стеке, а находиться в регистрах. Часть может быть переиспользована: один и тот же stack slot в разные моменты функции служит разным

переменным. Часть может быть удалена оптимизатором полностью. Часть может быть описана только в PDB, и без PDB декомпилятор будет угадывать.

EstablisherFrame даёт handler-у базу конкретного кадра, но он не превращает машинный код обратно в исходный C. Он не говорит: "по offset +0x30 лежит переменная count", "по offset -0x48 лежит указатель buffer". Эти имена и типы относятся к debug information или к результату анализа. На уровне unwind/SEH есть более грубая, но очень важная задача: правильно привязать обработчик к кадру, в котором эти данные потенциально находятся.

Поэтому правильная мысль такая: EstablisherFrame помогает compiler-generated handler-коду работать с кадром, потому что компилятор сам знает свои offsets и свою схему. Reverse engineer же не должен автоматически считать, что EstablisherFrame плюс некоторое смещение даёт достоверную исходную переменную. Для этого нужны дополнительные источники: дизассемблирование, PDB, анализ обращений к stack slots, lifetime analysis и понимание оптимизаций.

Local variable (локальная переменная) - переменная исходной функции, область видимости которой обычно ограничена этой функцией или блоком. В машинном коде она может жить в регистре, в stack slot, быть переиспользована или полностью исчезнуть после оптимизации.

Debug information / PDB (отладочная информация / Program Database) - дополнительные данные, которые могут хранить имена функций, типы, локальные переменные, соответствие исходных строк и машинного кода. Unwind metadata не заменяет PDB.

Lifetime analysis (анализ времени жизни значения) - попытка понять, в какие моменты выполнения конкретный register или stack slot содержит определённое логическое значение. Это сложнее, чем просто прочитать offset от RSP или EstablisherFrame.

Не надо писать в книге или инструменте: "EstablisherFrame позволяет узнать локальные переменные". Точнее: он даёт runtime-базу кадра, которую compiler-generated handler может использовать вместе со своими language-specific данными. Восстановление локальных переменных - отдельная задача анализа.

EstablisherFrame и filters: почему фильтр должен знать свой кадр

В C SEH __except-фильтр может быть не просто константой вроде EXCEPTION_EXECUTE_HANDLER. Он может быть выражением, которое проверяет код исключения, состояние программы, значения переменных или результат вспомогательной функции. В исходном коде это выглядит почти естественно: если произошло такое-то исключение и локальный флаг равен такому-то значению, обработать; иначе пропустить дальше.

Но на машинном уровне filter - это уже не магическая строка из C. Это compiler-generated логика, которую нужно вызвать в правильном контексте. Она должна относиться к конкретному вызову функции, потому что локальные значения и состояние отличаются от вызова к вызову. Если функция рекурсивная, один и тот же filter-код может проверять разные значения на разных глубинах. Если функция вызвана двумя потоками, у каждого потока свой стек и свой кадр.

EstablisherFrame в этой картине нужен как якорь. Он сообщает C-specific handler-у, с каким кадром связан рассматриваемый filter. Без такой привязки фильтр был бы оторван от конкретного выполнения. Он мог бы знать код исключения, но не имел бы корректной

базы для доступа к данным кадра, которые компилятор считает относящимися к данному __try/__except.

```
Упрощённый исходный образ:

void worker(int mode) {
int local_state = mode + 1;
__try {
dangerous_call();
} __except (local_state == 3
? EXCEPTION_EXECUTE_HANDLER
: EXCEPTION_CONTINUE_SEARCH) {
recover();
}
}

Машинная реальность:
- local_state может быть в stack slot или регистре;
- filter связан с compiler-specific handler data;
- handler должен применить filter к конкретному frame instance;
- EstablisherFrame помогает привязать эту логику к нужному кадру.
```

EXCEPTION_EXECUTE_HANDLER (выполнить обработчик исключения) - одно из типичных решений SEH-фильтра: данный __except-блок принимает исключение и будет выполнен.

EXCEPTION_CONTINUE_SEARCH (продолжить поиск) - решение SEH-фильтра: этот обработчик не принимает исключение, система должна искать дальше по стеку.

Filter expression (выражение фильтра) - выражение в __except(...), которое вычисляется во время search phase и решает судьбу исключения для данного кадра.

EstablisherFrame и __finally: почему cleanup тоже кадр-зависим

С __finally ситуация похожая, но психологически иногда понятнее. __finally - это код, который должен выполниться при выходе из защищённой области, в том числе при исключении. Если функция взяла ресурс, то finally должен вернуть систему в нормальное состояние. Но ресурс принадлежит конкретному вызову функции. У одного вызова один handle, у другого - другой. У одного вызова lock уже взят, у другого ещё нет. У одного буфер выделен, у другого allocation не успел произойти.

Когда идёт unwind phase, система вызывает termination handlers для покидаемых кадров. Каждый такой вызов должен относиться к правильному frame instance. Если cleanup привязать не к тому кадру, он может закрыть чужой handle, освободить чужую память или снять lock, который не принадлежит этому вызову. В корректной программе такие ошибки должны быть невозможны именно потому, что language-specific handler получает правильную frame-базу.

Поэтому EstablisherFrame важен не только для "умного" фильтра __except. Он важен и для обычного дисциплинированного cleanup. В exception-safe программировании cleanup должен быть не абстрактным, а привязанным к конкретному кадру, где ресурс был установлен.

__finally (блок финализации) - конструкция SEH, задающая код, который должен быть выполнен при выходе из защищённой области, включая выход из-за исключения. На x64 реализуется через language-specific termination handling и unwind metadata.

Termination handling (обработка завершения области) - вызов cleanup-кода при размотке стека. Это не обязательно обработка исключения в смысле "поймать и продолжить", а гарантированное выполнение служебного выхода.

Exception-safe code (код, безопасный при исключениях) - код, который сохраняет инварианты и освобождает ресурсы даже если выполнение прерывается исключением.

Почему EstablisherFrame нельзя надёжно вывести из одного ControlPc

ControlPc говорит, где находилось управление внутри функции. По нему можно найти RUNTIME_FUNCTION. По RUNTIME_FUNCTION можно найти UNWIND_INFO. Но одного ControlPc недостаточно, чтобы получить EstablisherFrame. Нужен ContextRecord, потому что именно в нём текущее значение RSP, frame register и другие регистры. Нужна память стека, потому что часть восстановления может требовать чтения сохранённых значений. Нужен ImageBase, потому что metadata описана image-relative адресами. Нужен корректный FunctionEntry, потому что без него непонятна геометрия функции.

Это важное различие для проектирования анализатора. Если пользователь загружает PE-файл и кликает на адрес инструкции, инструмент может сказать: "эта инструкция находится в функции, которая имеет EHANDLER". Но он не может честно сказать: "EstablisherFrame равен 000000A0...". Такого значения ещё нет в данных. Оно рождается только при конкретном проходе unwinder-а.

В crash dump ситуация другая. Там уже есть ContextRecord и стековая память на момент сбоя. Тогда можно попытаться воспроизвести virtual unwind и вычислить EstablisherFrame. Но даже в дампе нужно быть осторожным: стек может быть частично повреждён, страницы могут отсутствовать, symbols могут быть неполными, а function tables - динамическими. Поэтому хороший инструмент обязан показывать не только значение, но и confidence: насколько мы уверены, что оно корректно.

Confidence (уверенность анализа) - оценка того, насколько результат можно считать надёжным. Для EstablisherFrame это зависит от корректности function table, доступности stack memory, попадания адреса в stack bounds и согласованности unwind metadata.

Image-relative metadata (метаданные с image-relative адресами) - данные, где адреса заданы как RVA относительно ImageBase. Для получения VA нужно прибавить ImageBase. .pdata/.xdata в x64 PE активно используют такую модель.

Current thread (текущий поток) - поток выполнения, стек и регистры которого анализируются. EstablisherFrame относится именно к кадру конкретного потока, а не ко всему процессу сразу.

Условная числовая модель: не формула Microsoft, а способ думать

Теперь полезно дать числовую модель, но с оговоркой: это не универсальная формула для всех функций. Это учебный способ увидеть, почему EstablisherFrame зависит от stack layout. Пусть функция после call получила return address на вершине стека. Затем в прологе она сохранила RBX и RBP, выделила 0x80 байт локального стека и, возможно, установила frame pointer. В теле функции текущий RSP указывает уже на нижнюю часть построенного кадра.

Если случилось исключение в теле, unwinder должен понимать, где находятся сохранённые регистры, где fixed allocation, где return address. EstablisherFrame как база fixed stack allocation будет связан с этой геометрией. Но точное значение зависит от того, как пролог построил кадр и как UNWIND_INFO это описал. Именно поэтому нельзя угадать EstablisherFrame по одному виду инструкции ret или по одному наличию push rbp.

В реальном RtlVirtualUnwind эта логика не является человеческим рассуждением по картинке. Она следует формализованным unwind operations. Но для reverse engineer полезно держать в голове карту: return address выше, fixed allocation и сохранённые значения образуют кадр, а EstablisherFrame - это одна из системных баз, позволяющая handler-у говорить об этом кадре.

```
Условный стековый образ, не универсальная формула:

higher addresses
[return address to caller]
[saved RBX]
[saved RBP]
[fixed stack allocation: local area, spills, temp slots]
^
|
conceptual base used as EstablisherFrame for handler logic
lower addresses

Важно:
- реальная геометрия зависит от пролога;
- frame pointer может указывать не туда же;
- UNWIND_INFO задаёт рецепт;
- runtime context задаёт конкретные адреса.
```

EstablisherFrame и thunks/wrappers/runtime helpers

В реальном коде между местом исключения и функцией с логикой приложения могут встречаться wrappers, thunks, маленькие adapter-функции, runtime helpers, compiler-generated frames. Некоторые из них имеют полноценные unwind entries, некоторые leaf, некоторые не имеют handler-ов, а некоторые только передают управление дальше. Когда Windows идёт по стеку, она рассматривает каждый кадр по function tables.

EstablisherFrame появляется как важное значение именно для кадра, где есть handler semantics. Если промежуточный thunk не имеет handler-а, он всё равно может быть unwound, но handler context для него не формируется в том же смысле. Если wrapper имеет __finally или C++ cleanup, его кадр уже важен для unwind phase. Поэтому в сложной цепочке вызовов EstablisherFrame помогает отличать просто кадр для stack walking от кадра, который устанавливает обработчик или cleanup.

Для reverse engineer это полезно при чтении больших трасс. Не каждый кадр в stack trace одинаково важен для исключений. Одни кадры просто проходят как ступеньки. Другие имеют language-specific metadata. Третьи участвуют в cleanup. EstablisherFrame относится к тем кадрам, которые нужно передать handler-логике как установившие соответствующую область обработки.

Thunk (переходник / тонкая служебная функция) - небольшая функция, которая часто только перенаправляет вызов, адаптирует соглашение вызова, делает переход через импорт/виртуальную таблицу или связывает разные уровни runtime.

Wrapper (обёртка) - функция, которая окружает другой вызов дополнительной логикой: проверкой, логированием, обработкой ошибок, cleanup или адаптацией интерфейса.

Runtime helper (служебная функция runtime-а) - вспомогательная функция библиотеки выполнения или компилятора, которую программист может не видеть в исходной логике, но которая появляется в машинном коде.

Практическая проверка: какие вопросы задавать при анализе кадра

Когда ты смотришь на функцию с handler metadata, полезно идти не хаотично, а по списку вопросов. Первый вопрос: есть ли у функции запись RUNTIME_FUNCTION и корректный диапазон BeginAddress..EndAddress? Второй: куда ведёт UnwindInfoAddress? Третий: какие flags стоят в UNWIND_INFO? Если есть UNW_FLAG_EHANDLER или UNW_FLAG_UHANDLER, нужно смотреть address of exception handler и HandlerData.

Дальше вопрос меняется: мы анализируем файл или runtime/dump? Если только файл, EstablisherFrame остаётся потенциальной runtime-сущностью. Если есть ContextRecord и stack memory, можно попытаться сделать virtual unwind и получить значение. После этого нужно проверить: находится ли значение в пределах стека текущего потока? выглядит ли оно выровненным? согласуется ли с ожидаемым stack frame? не указывает ли оно в image memory, heap или guard page? не возникло ли оно из повреждённого RSP?

Такая дисциплина важна, потому что EstablisherFrame легко сделать "красивым числом" в отчёте, но трудно сделать надёжным фактом. Хороший reverse engineering инструмент должен не только выводить адрес, но и объяснять, почему он ему доверяет.

```
Checklist для static /dynamic :

Static pass:
[ ] RUNTIME_FUNCTION range valid
[ ] UnwindInfoAddress valid RVA
[ ] UNWIND_INFO aligned and parseable
[ ] handler flags consistent
[ ] handler RVA points to executable/code-like region
[ ] HandlerData is within readable metadata region
[ ] chained info rules respected

Runtime/dump pass:
[ ] ContextRecord available
[ ] stack memory readable
[ ] RtlLookupFunctionEntry-compatible result found
[ ] virtual unwind simulated
[ ] EstablisherFrame computed
[ ] value inside thread stack bounds
[ ] alignment plausible
[ ] value consistent with fixed stack allocation geometry
[ ] confidence reported
```

EstablisherFrame как граница между книгой и конспектом

Если писать эту тему как конспект, получилось бы: "EstablisherFrame - output parameter of RtlVirtualUnwind; passed to language-specific handler". Формально это не ложь, но понимания почти не даёт. Читатель узнает, что параметр существует, но не поймёт, зачем он родился, почему без него handler нельзя правильно связать с кадром, почему он отличается от RBP, почему он runtime-only, почему он важен при рекурсии, filters, finally,

crash dumps и reverse engineering.

В фундаментальной книге этот термин нужно рассматривать как узел всей x64 exception model. Он показывает, что unwind - это не только восстановление адресов возврата. Unwind ещё связывает код, metadata, stack memory и языковую семантику. EstablisherFrame - маленькое имя для большой идеи: обработчик исключения должен быть привязан к конкретному живому кадру, иначе он превращается в абстрактный адрес без контекста.

Поэтому дальше, когда мы перейдём к DispatcherContext, нужно сохранить эту мысль. DispatcherContext не будет просто "ещё одной структурой". Это продолжение той же линии: Windows должна передать handler-у достаточно контекста, чтобы статические таблицы и живое выполнение соединились в один корректный механизм.

Что происходит, если ControlPc находится не в теле функции

До сих пор мы в основном говорили о спокойном случае: исключение произошло в теле функции, кадр уже построен, fixed stack allocation существует, handler можно связать с кадром обычным способом. Но RtlVirtualUnwind обязан работать не только в этом красивом случае. ControlPc может находиться в прологе, в эпилоге или в так называемой null frame region. Это резко меняет смысл некоторых значений.

Если ControlPc находится в прологе, функция ещё не обязательно установила свой полный кадр. Возможно, она уже сделала push rbx, но ещё не выполнила sub rsp, 80h. Возможно, frame pointer ещё не установлен. Значит, рассуждать о complete fixed stack allocation как о полностью готовой области опасно. Unwinder должен учитывать только те действия пролога, которые действительно уже произошли до данного offset. Поэтому EstablisherFrame в таких краевых ситуациях нельзя мыслить как простую статическую базу, которая всегда есть одинаково.

Если ControlPc находится в эпилоге, функция уже начала разбирать кадр. Она могла восстановить часть nonvolatile-регистров, вернуть RSP ближе к caller-у и ещё не выполнить ret. В таком состоянии unwinder часто должен симулировать остаток эпилога, а не применять обычную логику тела. Это важно и для EstablisherFrame: если кадр уже выходит, вопрос "какой frame established handler" становится связанным с фазой и положением в коде. Именно поэтому Microsoft отдельно описывает эпилоговые маркеры для RtlVirtualUnwind.

Null frame region (нулевая область кадра / область без полноценного кадра) - ситуация, где текущая позиция не соответствует обычному телу функции с полностью установленным кадром. В документации RtlVirtualUnwind упоминается как одна из зон, отличных от тела функции.
Complete frame (полностью построенный кадр) - состояние после выполнения всех релевантных действий пролога: сохранений nonvolatile-регистров, fixed stack allocation и установки frame pointer, если он есть.
Partial prolog execution (частичное выполнение пролога) - состояние, когда ControlPc находится внутри пролога, и только часть описанных UNWIND_CODE действий уже фактически произошла.

Нельзя считать, что EstablisherFrame всегда одинаково осмыслен в любой точке функции. Положение ControlPc внутри prolog/body/epilog влияет на то, как unwinder интерпретирует кадр и какие значения можно считать определёнными.

EstablisherFrame и hand-written assembly

Отдельная зона риска - hand-written assembly, то есть функции, написанные вручную на ассемблере. Компилятор обычно генерирует согласованный комплект: машинный пролог, unwind operations, handler flags и данные. Человек же может написать красивый ассемблерный код, но забыть правильно описать его через MASM FRAME directives или другой механизм генерации unwind info. Тогда код может работать в обычном пути выполнения, но ломаться при исключении, отладке или stack walking.

Для EstablisherFrame это особенно опасно. Если функция вручную сохраняет регистры, выделяет стек, использует RBP или R13 как frame pointer, но metadata не соответствует реальному прологу, unwinder вычислит неправильную геометрию кадра. В результате handler может получить неправильный EstablisherFrame или вообще не быть найден корректно. Это классическая ситуация, где программа "нормально работает", пока не возникает исключение или не нужен корректный dump.

Поэтому при reverse engineering handwritten assembly важно смотреть не только на инструкции, но и на то, как они представлены в .pdata/.xdata. Если пролог делает одно, а UNWIND_INFO описывает другое, это не мелкая неточность. Это разрыв между реальным выполнением и системной моделью unwind. EstablisherFrame, как runtime-результат этой модели, тоже становится подозрительным.

Hand-written assembly (ассемблер, написанный вручную) - машинный код, созданный программистом напрямую, а не обычным компилятором из C/C++. Такой код требует особенно аккуратной unwind-разметки на x64.

MASM FRAME directives (FRAME-директивы MASM) - директивы ассемблера Microsoft, которые помогают описать prolog/unwind поведение функции и сгенерировать .pdata/.xdata. Без них handwritten x64-функция может быть невидимой или неверной для unwinder-а.

Unwind annotation (unwind-аннотация) - описание того, как функция меняет RSP и сохраняет nonvolatile-регистры. В компиляторном коде она генерируется автоматически, а в ручном ассемблере требует дисциплины.

EstablisherFrame в JIT и dynamic function tables

JIT-компиляторы и runtime-системы создают код не заранее в PE-файле, а во время выполнения. Но Windows x64 всё равно требует, чтобы nonleaf-функции, для которых важны исключения и отладка, были описаны через function tables. Поэтому для динамического кода существуют RtlAddFunctionTable и RtlInstallFunctionTableCallback. Они регистрируют сведения, которые позволяют RtlLookupFunctionEntry и RtlVirtualUnwind работать не только с обычной .pdata, но и с runtime-generated code.

EstablisherFrame в JIT-мире сохраняет тот же смысл: это runtime-база конкретного кадра, установившего handler. Но источник metadata меняется. Вместо статической .pdata в PE может быть dynamic function table, созданная runtime-ом. Если JIT неправильно описал пролог или handler data, unwinder будет ошибаться так же, как при повреждённой .xdata в обычном модуле.

Для static /dynamic это означает, что анализ PE-файла не покрывает весь мир function tables. Если процесс использует JIT, в памяти могут существовать функции, которых нет в исходном PE как обычных .text/.pdata записей. При runtime-анализе нужно учитывать active function tables целиком. И если для такого динамического кадра есть handler, EstablisherFrame будет вычисляться на основе динамически предоставленной unwind-информации.

JIT compiler (just-in-time compiler, компилятор во время выполнения) - система, которая генерирует машинный код во время работы программы, например для managed runtime, скриптового движка или виртуальной машины.
Dynamic function table (динамическая таблица функций) - таблица RUNTIME_FUNCTION-подобных записей, зарегистрированная во время выполнения через Windows API, чтобы unwinder мог работать с кодом, созданным не статическим линкером.
Active function tables (активные таблицы функций) - совокупность таблиц, в которых Windows ищет function entry для ControlPc: статические таблицы загруженных модулей и динамические таблицы, зарегистрированные runtime-ом.

Если EstablisherFrame выглядит подозрительно

В нормальном случае вычисленный EstablisherFrame должен выглядеть скучно: адрес внутри стека текущего потока, с разумным выравниванием, согласованный с текущим RSP и геометрией кадра. Но в reverse engineering часто интересны именно нескучные случаи. Если EstablisherFrame указывает в .text, heap, freed memory, guard page или вообще за пределы доступной памяти, это сигнал, что что-то не так.

Причины могут быть разные. Возможно, стек повреждён переполнением буфера. Возможно, exception возникло после stack pivot, когда RSP специально перенаправлен атакующим или обфускатором. Возможно, function table зарегистрирована неверно. Возможно, packer переместил код и не обновил unwind metadata. Возможно, dump неполный и нужной страницы стека нет. Возможно, analyzer неправильно прибавил ImageBase к RVA и дальше всё посыпалось.

Хорошая диагностика должна не сразу кричать "malware", а аккуратно раскладывать гипотезы. Подозрительный EstablisherFrame - это не приговор, а evidence. Его нужно сопоставлять с другими признаками: валидностью RUNTIME_FUNCTION, чтением UNWIND_INFO, stack bounds, наличием handler flags, доступностью HandlerData, корректностью epilog/prolog, поведением RtlLookupFunctionEntry и состоянием ContextRecord.

Stack pivot (перенаправление стека) - техника, при которой RSP переводится на другую область памяти, часто используемая в exploit development или сложной обфускации. Для unwinder-а это может сделать stack-based выводы подозрительными.
Evidence (свидетельство / признак) - отдельный наблюдаемый факт анализа. В хорошей системе evidence не равно окончательному verdict; несколько evidence собираются в обоснованный вывод.
Verdict (итоговый вывод) - заключение инструмента после сопоставления нескольких признаков. Например: normal, suspicious, corrupted unwind metadata, possible stack corruption.

Итог раздела

EstablisherFrame - это не второстепенная странность в прототипе RtlVirtualUnwind. Это ключ к пониманию того, как x64 Windows связывает статические handler metadata с конкретным кадром выполнения. Без него у системы была бы карта функций, таблица unwind-действий и адрес handler-а, но не было бы ясной привязки: какой именно вызов функции сейчас рассматривается.

Для reverse engineer главный вывод такой: EstablisherFrame нельзя искать как поле в PE и нельзя автоматически приравнивать к RBP. Его нужно понимать как runtime-значение, вычисляемое в процессе unwind для конкретного frame instance. В статическом анализе мы можем описать условия его появления. В динамическом анализе или дампе можем проверять конкретное значение, его границы, выравнивание и согласованность с unwind metadata.

Для static /dynamic это понятие особенно полезно как граница честности анализа. Статический отчёт должен говорить о metadata и потенциальной семантике. Runtime/dump отчёт может говорить о вычисленном EstablisherFrame. Если инструмент смешивает эти уровни, он начинает врать. Если разделяет - он становится серьёзным reverse engineering инструментом, а не просто красивым парсером структур.

5.5. Что такое DispatcherContext

К этому месту мы уже прошли почти весь путь, который нужен для понимания одного шага Windows x64 unwind. Сначала система получает RIP или ControlPc. Потом через RtlLookupFunctionEntry находит RUNTIME_FUNCTION. Затем RtlVirtualUnwind читает UNWIND_INFO, понимает, где находится управление - в прологе, теле или эпилоге, - и вычисляет caller context. После этого появляется EstablisherFrame, то есть runtime-привязка найденного обработчика к конкретному кадру стека.

Но механизм исключений не заканчивается на том, что найден обработчик и найден кадр. Если Windows собирается вызвать language-specific handler, ей нужно передать ему не разрозненный набор значений, а связанный служебный контекст текущего шага dispatch/unwind. Handler должен знать, в какой функции он сейчас рассматривается, каков ImageBase модуля, какая RUNTIME_FUNCTION-запись описывает функцию, где лежит HandlerData, какой ControlPc проверяется, какой EstablisherFrame соответствует кадру и какой ContextRecord сейчас используется. Для этого и существует DispatcherContext.

DispatcherContext на первый взгляд выглядит как простая структура с несколькими полями. Но это обманчивое впечатление. В фундаментальном смысле DispatcherContext - это не просто "ещё один указатель" и не просто "параметр функции". Это конверт, который диспетчер исключений собирает для language-specific handler-а. В этот конверт он кладёт всё, что обработчику нужно знать о текущем кадре, не заставляя обработчик заново искать .pdata, пересчитывать ImageBase, вычислять EstablisherFrame и вытаскивать HandlerData.

Именно поэтому DispatcherContext надо понимать не как отдельную деталь, а как место встречи всех уже изученных сущностей. В нём рядом оказываются ControlPc, ImageBase, FunctionEntry, EstablisherFrame, TargetIp, ContextRecord, LanguageHandler и HandlerData. Каждое поле само по себе уже знакомо или почти знакомо, но в DispatcherContext они впервые собираются в одну рабочую картину: "вот текущий кадр, вот его функция, вот его метаданные, вот его обработчик, вот его языковые данные, вот состояние процессора, вот цель unwind".

DispatcherContext (dispatcher context, контекст диспетчера исключений) - служебная структура, которую Windows передаёт language-specific handler-у при обработке исключения или при размотке стека. Она описывает текущий кадр с точки зрения exception dispatcher-а: где управление, из какого модуля функция, какая RUNTIME_FUNCTION-запись найдена, какой EstablisherFrame вычислен, где HandlerData и какой ContextRecord используется.
Dispatcher (диспетчер) - компонент логики, который не выполняет саму "языковую" обработку исключения, а организует процесс: идёт по кадрам, вызывает нужные helper-функции, передаёт параметры обработчикам и принимает их решения. В слове dispatcher важна идея маршрутизации: он распределяет управление между участниками механизма.
Context (контекст) - набор информации, без которого нельзя правильно понять текущую ситуацию. В этой теме context может означать CPU context, stack context, exception dispatch context или unwind context. Поэтому ContextRecord и DispatcherContext нельзя смешивать: первый хранит процессорное состояние, второй хранит служебную картину текущего шага обработки исключения.

Главная мысль раздела: DispatcherContext - это не замена CONTEXT record и не копия UNWIND_INFO. Это runtime-сводка по текущему кадру, подготовленная для language-specific handler-а. Она связывает статические метаданные PE с живым состоянием потока.

Почему обычного ContextRecord недостаточно

Если смотреть на проблему грубо, можно спросить: зачем вообще нужен DispatcherContext, если уже есть ContextRecord? В ContextRecord ведь лежит RIP, RSP, регистры, состояние процессора. Разве handler-у этого мало? Для простого понимания аварии иногда достаточно. Но для настоящей обработки исключения этого не хватает.

ContextRecord говорит: "процессор в таком состоянии". Но он не говорит, какая RUNTIME_FUNCTION-запись была найдена для текущего ControlPc. Он не говорит, какая ImageBase нужна для перевода image-relative адресов. Он не говорит, где лежат language-specific handler data. Он не говорит, какой LanguageHandler был выбран. Он не говорит, какой EstablisherFrame соответствует именно этому кадру. Все эти данные можно было бы вычислить заново, но тогда каждый обработчик должен был бы повторять работу системного диспетчера, а это опасно, дублирует логику и ломает разделение обязанностей.

Поэтому Windows делает иначе. Системный unwind/dispatch-код уже прошёл часть пути: нашёл модуль, нашёл FunctionEntry, разобрал flags в UNWIND_INFO, получил HandlerData, вычислил EstablisherFrame и знает ControlPc. После этого он складывает получившуюся картину в DispatcherContext и передаёт её language-specific handler-у. Handler не обязан заново быть мини-диспетчером исключений. Он получает готовую рабочую рамку и занимается своим делом: проверяет языковые таблицы, ищет подходящий __try/__except или __finally, принимает решение, продолжать поиск или запускать unwind.

Это похоже на судебное дело. ContextRecord - это фотография места происшествия: где стоял процессор, какие значения были в регистрах, какой адрес выполнялся. DispatcherContext - это папка дела, которую следователь передал эксперту: сюда добавлены сведения о здании, комнате, участниках, документах, полномочиях и цели процедуры. Фотография полезна, но эксперт не должен каждый раз самостоятельно устанавливать адрес здания и список допустимых документов. Ему передают собранный контекст.

ContextRecord (запись контекста процессора) - структура CONTEXT, в которой хранится состояние регистров процессора: RIP, RSP, регистры общего назначения, XMM-регистры и другие архитектурные поля. При exception handling это либо контекст момента исключения, либо текущий unwind-контекст.

CPU context (процессорный контекст) - состояние процессора в конкретный момент: instruction pointer, stack pointer, general-purpose registers, SIMD-регистры и flags. Это состояние отвечает на вопрос "где и с какими регистрами сейчас было выполнение?".

Dispatch context (контекст диспетчеризации) - служебная информация о том, как текущий кадр участвует в механизме обработки исключения. Он отвечает не только на вопрос "где процессор?", но и "какая функция, какая metadata, какой handler, какой frame и какие handler data относятся к этому месту?".

Типичная ошибка: считать DispatcherContext "ещё одним CONTEXT record". Это неверно. CONTEXT описывает процессор. DISPATCHER_CONTEXT описывает шаг работы exception

dispatcher-а по конкретному кадру.

Как выглядит DISPATCHER_CONTEXT в документации

В официальном описании x64 exception handling структура DISPATCHER_CONTEXT показана довольно компактно. Но компактность записи не означает простоту смысла. Каждое поле является ссылкой на уже изученную часть механизма. Поэтому сначала посмотрим на структуру целиком, а затем разберём поля медленно и по отдельности.

```
typedef struct _DISPATCHER_CONTEXT {
ULONG64 ControlPc;
ULONG64 ImageBase;
PRUNTIME_FUNCTION FunctionEntry;
ULONG64 EstablisherFrame;
ULONG64 TargetIp;
PCONTEXT ContextRecord;
PEXCEPTION_ROUTINE LanguageHandler;
PVOID HandlerData;
} DISPATCHER_CONTEXT, *PDISPATCHER_CONTEXT;
```

Здесь важно не застрять на синтаксисе C. DISPATCHER_CONTEXT - это имя структуры. PDISPATCHER_CONTEXT - это pointer to DISPATCHER_CONTEXT, то есть указатель на такую структуру. Handler получает не копию всех полей по отдельности, а адрес структуры, где эти поля собраны. Это удобно: если dispatcher или handler должен читать или уточнять несколько связанных значений, он работает с одним объектом.

Нельзя также думать, что эта структура прямо лежит в PE-файле. В PE лежат .pdata, .xdata, RUNTIME_FUNCTION, UNWIND_INFO и language-specific data. DispatcherContext создаётся на стороне runtime-логики во время обработки исключения или unwind. Он использует значения из PE-метаданных, но сам является runtime-объектом.

DISPATCHER_CONTEXT (структура контекста диспетчера) - C-структура, в которую Windows собирает параметры текущего кадра для передачи language-specific handler-у. Это runtime-структура, а не постоянная таблица внутри PE-файла.

PDISPATCHER_CONTEXT (pointer to DISPATCHER_CONTEXT, указатель на DISPATCHER_CONTEXT) - адрес структуры DISPATCHER_CONTEXT в памяти. Буква P в Windows-типах часто означает pointer, то есть указатель.

ULONG64 (unsigned long 64-bit, беззнаковое 64-битное целое) - тип для хранения 64-битных значений без знака. В этой структуре через него передаются адреса или адресоподобные значения, например ControlPc, ImageBase, EstablisherFrame и TargetIp.

PVOID (pointer to void, универсальный указатель) - тип указателя без уточнения конкретного типа данных. Он говорит: "здесь есть адрес, но общая часть системы не знает или не фиксирует формат данных по этому адресу". HandlerData имеет тип PVOID именно потому, что формат language-specific data определяется конкретным handler-ом.

DispatcherContext как точка сборки всего механизма

Самый правильный способ понять DispatcherContext - не заучивать поля, а увидеть, как он собирается из уже знакомых этапов. Пусть произошло исключение. Windows сохранила CPU context. По RIP она нашла FunctionEntry через RtlLookupFunctionEntry. Из FunctionEntry вышла к UNWIND_INFO. В UNWIND_INFO увидела флаг обработчика и адрес LanguageHandler. После массива UNWIND_CODE нашла HandlerData. Через

RtlVirtualUnwind вычислила EstablisherFrame. Теперь все эти значения существуют рядом, но пока не связаны в один объект. DispatcherContext как раз и делает из них одну связную запись.

Можно представить, что на каждом кадре Windows заполняет карточку. В карточке написано: текущий ControlPc такой-то, модуль начинается по ImageBase такой-то, функция описана такой-то RUNTIME_FUNCTION-записью, её кадр установлен таким EstablisherFrame, продолжать unwind надо к такому TargetIp, процессорное состояние находится в таком ContextRecord, вызывать надо такой LanguageHandler, а данные для него лежат по такому HandlerData. Эта карточка и есть DispatcherContext.

Поэтому DispatcherContext особенно важен для понимания frame-based exception handling на x64. В старой x86-модели можно было видеть цепочку SEH-записей в стеке. На x64 прямой цепочки обработчиков в стеке уже нет в том же виде. Система строит смысл через таблицы и runtime-контекст. DispatcherContext - один из объектов, который показывает, как табличная модель превращается в вызов реального обработчика.

Frame-based exception handling (кадровая обработка исключений) - модель, в которой обработка исключения рассматривается по кадрам стека: система поднимается от текущего кадра к вызывающим кадрам и проверяет, есть ли у каждого кадра подходящий обработчик.
Runtime object (объект времени выполнения) - объект или структура, существующая во время работы программы. DispatcherContext относится к runtime-объектам: он формируется во время обработки исключения, а не хранится заранее как статическая PE-таблица.
Exception dispatcher (диспетчер исключений) - логика Windows, которая организует поиск обработчика и размотку стека. Она использует unwind metadata, вызывает language-specific handlers и управляет переходом между search phase и unwind phase.

Поле ControlPc: адрес, относительно которого задаётся вопрос

Первое поле DispatcherContext - ControlPc. Мы уже встречали его раньше, но здесь его роль становится ещё яснее. ControlPc - это не просто любой адрес. Это адрес управления внутри текущей функции, по которому dispatcher рассматривает этот кадр. В ситуации исключения он может быть связан с адресом, где возникло исключение. В ситуации unwind он может быть адресом, по которому управление покинуло establishing function.

Почему нельзя просто сказать RIP? Потому что RIP - это имя регистра процессора. ControlPc - это смысловая роль адреса в механизме unwind/dispatch. Часто ControlPc берётся из RIP, но термин ControlPc подчёркивает не архитектурный регистр, а то, что по этому значению система принимает решения: в какой функции мы находимся, попадает ли адрес в guarded construct, нужно ли вызывать handler, находится ли управление в прологе, теле или эпилоге.

Language-specific handler, получив DispatcherContext, может использовать ControlPc как координату внутри функции. Для C SEH важно понять, попадает ли ControlPc внутрь защищённого блока __try. Если исключение произошло вне guarded region, handler этой функции не должен заявлять, что он его обрабатывает, даже если у функции вообще есть handler metadata. Наличие handler-а у функции ещё не означает, что любое место функции находится под этим handler-ом.

ControlPc (control program counter, адрес текущего управления) - виртуальный адрес инструкции или точки управления, относительно которой dispatcher рассматривает текущую функцию. Это значение используется для проверки положения внутри функции и guarded regions.

RIP (instruction pointer register, регистр указателя инструкции) - архитектурный регистр x64, содержащий адрес следующей или текущей выполняемой инструкции. ControlPc часто связан с RIP, но является не именем регистра, а параметром логики exception dispatch/unwind.

Guarded construct / guarded region (защищённая языковая область) - участок кода, который находится под действием языковой конструкции обработки исключений, например __try. Handler должен понимать, относится ли текущий ControlPc к такой области.

Ошибка анализа: считать, что если функция имеет UNW_FLAG_EHANDLER, то любое исключение внутри диапазона функции будет обработано этим handler-ом. На самом деле handler ещё должен проверить свои language-specific data и понять, лежит ли ControlPc в нужной guarded region.

Поле ImageBase: почему без базы модуля нельзя читать RVA

Второе поле - ImageBase. На первый взгляд оно кажется технической мелочью: база модуля, ну и что. Но для Windows x64 unwind это одно из ключевых значений, потому что большая часть адресов в RUNTIME_FUNCTION и UNWIND_INFO хранится как image-relative offsets. То есть в метаданных лежит не полный виртуальный адрес, а смещение относительно базы образа.

FunctionEntry содержит BeginAddress, EndAddress и UnwindInfoAddress как RVA. Address of exception handler в UNWIND_INFO также является image-relative pointer. Если handler или dispatcher хочет получить настоящий VA, ему нужно сложить ImageBase и RVA. Без ImageBase статические offsets не превращаются в реальные адреса текущего процесса.

Это особенно важно из-за ASLR. Один и тот же EXE или DLL может быть загружен по разным адресам в разных запусках. В файле лежит preferred ImageBase, но в реальном процессе loader может выбрать другую базу. Поэтому runtime-система не может работать только с "адресами из файла". Она должна знать фактическую ImageBase загруженного образа. DispatcherContext передаёт это значение handler-у рядом с FunctionEntry и HandlerData.

ImageBase (база образа) - виртуальный адрес, по которому PE image фактически загружен в адресное пространство процесса. В DispatcherContext это не обязательно preferred image base из файла, а runtime-база загруженного модуля.

RVA (relative virtual address, относительный виртуальный адрес) - смещение от ImageBase до нужной сущности внутри загруженного образа. Чтобы получить VA, обычно используют формулу VA = ImageBase + RVA.

VA (virtual address, виртуальный адрес) - адрес в виртуальном адресном пространстве процесса. Именно VA используется процессором и указателями во время выполнения.

ASLR (address space layout randomization, рандомизация размещения адресного пространства) - механизм защиты, из-за которого модули могут загружаться по разным базовым адресам. Из-за ASLR ImageBase в конкретном процессе особенно важна.

```
Смысл ImageBase в DispatcherContext:

реальный_адрес_function_start = ImageBase + FunctionEntry->BeginAddress
реальный_адрес_unwind_info = ImageBase + FunctionEntry->UnwindInfoAddress
реальный_адрес_handler = ImageBase + HandlerRva_из_UNWIND_INFO

Без ImageBase эти RVA остаются только смещениями, а не рабочими адресами.
```

Поле FunctionEntry: какая именно запись .pdata описывает кадр

Третье поле - FunctionEntry. Это pointer to RUNTIME_FUNCTION, то есть указатель на запись таблицы функций, найденную для текущего ControlPc. Здесь снова важно не упростить мысль до "там лежит функция". FunctionEntry не указывает на машинный код функции. Он указывает на запись, описывающую диапазон функции и адрес её unwind-информации.

Почему handler-у может понадобиться FunctionEntry? Потому что RUNTIME_FUNCTION - это центральная связка между кодовым диапазоном и UNWIND_INFO. Через него можно понять границы функции, найти UnwindInfoAddress, а в некоторых ситуациях - увидеть chained unwind info или function fragment. Для системной логики это не просто справочная информация: это доказательство, на основании чего dispatcher решил, что текущий ControlPc относится именно к этой функции.

Для reverse engineer-а FunctionEntry в DispatcherContext особенно ценен в отладчике. Если ты остановился внутри language-specific handler-а, по DispatcherContext->FunctionEntry можно вернуться к .pdata-записи и проверить: какой диапазон функции сейчас рассматривается, куда ведёт UnwindInfoAddress, совпадает ли это с дизассемблером и не подменены ли данные. Это уже не абстрактный анализ файла, а наблюдение живой работы механизма.

FunctionEntry (запись функции / указатель на RUNTIME_FUNCTION) - поле DispatcherContext, указывающее на RUNTIME_FUNCTION entry, найденную для текущего ControlPc. Оно связывает runtime-ситуацию с записью .pdata.
PRUNTIME_FUNCTION (pointer to RUNTIME_FUNCTION, указатель на запись RUNTIME_FUNCTION) - тип указателя на структуру, содержащую BeginAddress, EndAddress и UnwindInfoAddress.
Function fragment (фрагмент функции) - часть функции, которая может иметь отдельную RUNTIME_FUNCTION-запись, особенно при chained unwind info, shrink wrapping или оптимизациях. В unwind-анализе функция не всегда выглядит как один простой непрерывный диапазон.
FunctionEntry в DispatcherContext говорит handler-у и исследователю: "вот конкретная запись таблицы функций, на основании которой этот кадр сейчас рассматривается". Это не имя функции и не PDB-символ, а структурное основание unwind-решения.

Поле EstablisherFrame: база конкретного кадра внутри общей структуры

Четвёртое поле - EstablisherFrame. Отдельный раздел уже был посвящён этому понятию, но в DispatcherContext оно получает более ясное место. Раньше мы говорили: EstablisherFrame связывает обработчик с конкретным stack frame. Теперь видно, где это значение передаётся дальше: оно лежит не только отдельным параметром в сигнатуре language-specific handler-а, но и внутри DispatcherContext.

Это дублирование на первый взгляд может смутить: если EstablisherFrame уже передан отдельным параметром, зачем он ещё в DispatcherContext? Ответ в том, что DispatcherContext является полной карточкой текущего кадра. Handler может получить EstablisherFrame напрямую, но ему также полезно иметь его рядом с ControlPc, ImageBase, FunctionEntry и HandlerData. Это делает контекст самодостаточным: структура описывает не только адреса metadata, но и runtime-базу кадра.

Для __C_specific_handler EstablisherFrame описан как адрес базы fixed stack allocation. Это важно, потому что language-specific data часто содержит offsets, которые должны быть применены к конкретному кадру. Handler должен не просто знать, что в функции есть __try. Он должен иметь точку отсчёта в стеке, относительно которой compiler-generated код сможет найти нужные runtime-данные, выполнить фильтр, запустить finally или подготовить переход к handler-блоку.

EstablisherFrame (устанавливающий кадр / база кадра обработчика) - runtime-значение, идентифицирующее конкретный stack frame, с которым связан текущий handler. В DispatcherContext оно хранится рядом с другими данными текущего шага обработки.

Fixed stack allocation (фиксированное стековое выделение) - область стека, размер и положение которой описаны unwind-метаданными функции. Для __C_specific_handler EstablisherFrame связан с базой этой области.

Compiler-generated code (код, сгенерированный компилятором) - служебный код и данные, которые компилятор создаёт для реализации высокоуровневых конструкций языка: __try, __except, __finally, C++ EH, security cookie checks и т.п.

Поле TargetIp: куда unwind должен привести выполнение

Пятое поле - TargetIp. Это одно из тех полей, которые часто пролистывают, потому что на раннем этапе оно кажется не таким важным, как ControlPc или FunctionEntry. Но TargetIp становится принципиальным, когда мы переходим от простого поиска обработчика к реальной размотке стека и продолжению выполнения.

TargetIp означает target instruction pointer, то есть целевой адрес инструкции, к которому должна прийти процедура unwind. В обычной search phase система только ищет, кто готов обработать исключение. Но когда обработчик найден и начинается unwind phase, система должна размотать промежуточные кадры, вызвать нужные termination handlers и в конце передать управление туда, где будет продолжено выполнение после обработки. TargetIp связан именно с этой целевой точкой.

Важно не путать TargetIp с ControlPc. ControlPc - это адрес, который сейчас анализируется в текущем кадре. TargetIp - это адрес, куда unwind должен в итоге привести управление или где находится продолжение. В документации RtlUnwindEx есть похожая пара TargetFrame и TargetIp: TargetFrame задаёт кадр, до которого нужно разматывать стек, а TargetIp задаёт continuation address. DispatcherContext хранит TargetIp, чтобы language-specific handler и dispatcher могли согласованно понимать цель unwind-процедуры.

TargetIp (target instruction pointer, целевой указатель инструкции) - адрес продолжения, связанный с unwind. Он отвечает не на вопрос "где мы сейчас?", а на вопрос "куда должна привести размотка".

Continuation address (адрес продолжения) - адрес, с которого выполнение должно продолжиться после завершения unwind/exception handling. Это не обязательно адрес, где возникло исключение.

TargetFrame (целевой кадр) - кадр стека, до которого должна дойти процедура размотки. В RtlUnwindEx TargetFrame и TargetIp задают цель unwind-а; если TargetFrame отсутствует, речь может идти об exit unwind.

RtlUnwindEx (системная функция запуска размотки стека) - функция Windows, которая инициирует unwind procedure для procedure call frames. Она используется, когда нужно не просто посмотреть предыдущий кадр, а реально выполнить размотку до целевого кадра или выходную размотку.

Нельзя смешивать ControlPc и TargetIp. ControlPc - анализируемая точка текущего кадра. TargetIp - целевая точка продолжения unwind. В простых картинках они могут казаться похожими адресами, но их роли разные.

Поле ContextRecord: живое состояние, которое меняется при размотке

Шестое поле - ContextRecord. Оно указывает на структуру CONTEXT, с которой работает системный exception dispatch/unwind code. Важный момент: ContextRecord не всегда означает "оригинальный контекст исключения в неизменном виде". В search phase он может отражать состояние на момент исключения. В unwind phase он может быть текущим unwind-контекстом, который постепенно изменяется по мере подъёма по кадрам.

Это делает ContextRecord опасным для поверхностного понимания. Новичок думает: исключение случилось, значит ContextRecord всегда показывает точку аварии. Но в termination handler case документация указывает, что ContextRecord может указывать на текущий unwind context. То есть по мере размотки система может работать уже не с тем контекстом, где всё началось, а с контекстом текущего шага. Именно поэтому нужно различать exception context и unwind context.

DispatcherContext хранит указатель на ContextRecord, чтобы handler видел состояние, с которым сейчас работает dispatcher. Для __except filter это может быть контекст исключения. Для __finally во время unwind это уже контекст размотки. На уровне reverse engineering это объясняет, почему в разных местах exception machinery один и тот же тип CONTEXT не всегда отвечает на один и тот же смысловой вопрос.

PCONTEXT (pointer to CONTEXT, указатель на структуру CONTEXT) - адрес структуры процессорного контекста. В DispatcherContext это поле связывает служебный dispatch context с текущим CPU/unwind context.

Exception context (контекст исключения) - состояние процессора в момент, когда исключение было поднято. Оно отвечает на вопрос: "что выполнялось, когда произошла ошибка или RaiseException?".

Unwind context (контекст размотки) - состояние, которое система поддерживает во время прохода по кадрам при unwind. Оно может уже представлять не исходную точку аварии, а текущий кадр размотки.

Termination handler case (случай обработчика завершения) - ситуация, когда вызывается cleanup/termination handler во время размотки. В этом случае ContextRecord может отражать текущий unwind context, а не исходный exception context.

Поле LanguageHandler: кто именно будет вызван

Седьмое поле - LanguageHandler. Оно указывает на language-specific handler routine, то есть на функцию обработчика, которую dispatcher вызывает для текущего кадра. Сам адрес этого обработчика берётся из UNWIND_INFO, если установлены соответствующие flags: UNW_FLAG_EHANDLER или UNW_FLAG_UHANDLER. Но в DispatcherContext он уже представлен как готовый function pointer.

Почему этот handler называется language-specific? Потому что общая x64 unwind-модель не знает всех правил C, C++, Rust, JIT-языков или ручных схем. Общая модель знает, как найти функцию, как восстановить регистры, где взять HandlerData и как вызвать обработчик. Но смысл HandlerData и решение "обработать или продолжить поиск" зависят от языка и компилятора. Поэтому handler является языковым слоем поверх общей табличной unwind-модели.

Для MSVC C SEH важен __C_specific_handler. Его задача - реализовать structured exception handling extensions: __try/__except и __try/__finally. Он получает ExceptionRecord, EstablisherFrame, ContextRecord и DispatcherContext. Используя эти данные, он может просмотреть свои language-specific tables, проверить ControlPc относительно guarded regions, вызвать filter expression или подготовить unwind.

LanguageHandler (языковой обработчик) - указатель на функцию, которая понимает language-specific handler data и принимает решение для текущего кадра. Это не универсальный парсер Windows, а обработчик конкретной схемы компилятора/языка.

PEXCEPTION_ROUTINE (pointer to exception routine, указатель на функцию обработчика исключения) - тип указателя на callback, который соответствует прототипу language-specific handler-a: ExceptionRecord, EstablisherFrame, ContextRecord, DispatcherContext.

__C_specific_handler (C-specific handler, C-специфический обработчик) - обработчик, который компилятор использует для реализации SEH-конструкций языка C/C++ в стиле Microsoft: __try, __except, __finally.

Callback (обратный вызов) - функция, адрес которой передаётся системе, чтобы система вызвала её в нужный момент. LanguageHandler в этом смысле является callback-ом exception machinery.

Важная граница ответственности: RtlVirtualUnwind и exception dispatcher знают, как дойти до LanguageHandler и как передать ему DispatcherContext. Но внутренний смысл HandlerData должен понимать сам LanguageHandler.

Поле HandlerData: данные, которые знает только handler

Восьмое поле - HandlerData. Оно указывает на language-specific handler data для текущей функции. В документации прямо сказано, что формат этих данных unspecified and completely determined by the specific exception handler in use. Для reverse engineer-а это фраза огромной важности. Она означает: общая документация x64 unwind объясняет, где находится HandlerData, но не обещает, что все handler data имеют один универсальный

формат.

Для C SEH там будет одна схема. Для C++ exception handling - другая. Для нестандартного компилятора или JIT - третья. Для ручного языка или обфускатора - вообще своя. Поэтому parser, который умеет читать UNWIND_INFO, ещё не обязательно умеет понимать HandlerData. Это разные уровни. Сначала надо безопасно найти границу UNWIND_INFO, выровнять массив UNWIND_CODE, прочитать handler RVA, получить HandlerData pointer. А уже потом, если известен LanguageHandler, можно пытаться интерпретировать HandlerData.

HandlerData в DispatcherContext говорит handler-у: "вот твоя дополнительная таблица". Но только сам handler знает, что в этой таблице означает каждый байт. Для static /dynamic это рождает важную архитектурную идею: универсальный слой должен валидировать факт наличия handler data, диапазон, выравнивание, принадлежность секции, базовую связность с UNWIND_INFO. А специализированный слой должен подключаться по типу LanguageHandler и разбирать формат глубже.

HandlerData (данные обработчика) - указатель на дополнительные language-specific данные, которые передаются LanguageHandler-у. Общая x64 unwind-модель не задаёт универсальный формат этих данных.
Unspecified format (неуточнённый формат) - ситуация, когда документация общего механизма намеренно не фиксирует внутреннее устройство данных. Это не значит, что формата нет; это значит, что формат принадлежит конкретному handler-у/компилятору, а не общей ABI-части.
Specialized parser (специализированный парсер) - модуль анализатора, который разбирает не общий UNWIND_INFO, а конкретный формат language-specific data, например данные __C_specific_handler или C++ EH metadata.

Поле DISPATCHER_CONTEXT	Главный смысл	Типичная ошибка
ControlPc	Адрес управления, по которому рассматривается текущий кадр.	Путать с TargetIp или считать просто любым RIP.
ImageBase	Фактическая база загруженного модуля для перевода RVA в VA.	Использовать preferred base из файла вместо runtime ImageBase.
FunctionEntry	Указатель на RUNTIME_FUNCTION entry текущей функции/фрагмента.	Думать, что это адрес машинного кода функции.
EstablisherFrame	Runtime-база кадра, связанного с handler-ом.	Автоматически приравнивать к RBP.
TargetIp	Адрес продолжения/цель unwind.	Путать с ControlPc.
ContextRecord	Текущий процессорный или unwind-контекст.	Считать всегда неизменным контекстом точки аварии.
LanguageHandler	Функция, которая понимает конкретную handler-схему.	Считать её универсальным Windows-парсером всех языков.
HandlerData	Данные для LanguageHandler-а.	Пытаться разбирать одним универсальным форматом.

Почему DispatcherContext не лежит в .xdata

На этом этапе полезно специально остановиться и сказать, где DispatcherContext не находится. Он не лежит в .pdata. Он не лежит как готовая структура в .xdata. Он не является частью RUNTIME_FUNCTION. Он не записан в PE-файле на диске. Это важно, потому что внешне часть его полей похожа на данные из PE: ImageBase, FunctionEntry, HandlerData. Но сама структура возникает только во время работы механизма.

.pdata даёт карту диапазонов функций. .xdata даёт инструкцию unwind и, при наличии handler flags, адрес LanguageHandler и дополнительные HandlerData. Но DispatcherContext объединяет статическую информацию с runtime-состоянием: ControlPc берётся из текущего контекста, ImageBase - из загруженного модуля, FunctionEntry - из активной таблицы функций, EstablisherFrame - вычисляется unwinder-ом, ContextRecord - указывает на живое состояние потока. Нельзя заранее положить такой объект в файл, потому что половина его полей зависит от конкретного запуска.

Это различие особенно важно для offline PE analyzer-a. Если `static` читает файл на диске, он может найти потенциальный handler RVA и потенциальные language-specific data. Но он не может честно знать реальный DispatcherContext без процесса, потока, стека, ControlPc и runtime ImageBase. Он может построить модель или симуляцию, но не должен выдавать её за наблюдённый runtime DispatcherContext.

Offline PE analyzer (офлайн-анализатор PE) - инструмент, который читает PE-файл без запуска процесса. Он может разобрать статические метаданные, но не видит реальный стек, runtime ImageBase конкретного процесса и живой DispatcherContext.
Runtime observation (наблюдение во время выполнения) - анализ процесса в момент работы: через debugger, instrumentation, dump или live tracing. Только такой режим может увидеть реальные ContextRecord, EstablisherFrame и DispatcherContext.
Model / simulation (модель / симуляция) - приближённое восстановление поведения на основе статических данных. Модель полезна, но её нужно отличать от факта, полученного из живого процесса.
Для `static` /`dynamic` нельзя писать в отчёте: "DispatcherContext в файле содержит...". В файле его нет. Корректнее: "по статическим данным можно предположить, какие значения будут участвовать в DispatcherContext при runtime-вызове handler-a".

DispatcherContext и две фазы: search phase и unwind phase

Следующая большая глава будет посвящена фазам обработки исключения, но DispatcherContext уже заставляет нас подготовиться к этой теме. В Windows exception handling есть минимум две важные логические фазы: search phase и unwind phase. В первой система ищет обработчик, который согласится обработать исключение. Во второй система реально разматывает стек до найденного обработчика, вызывая termination handlers по пути.

DispatcherContext участвует в обеих картинах, но смысл некоторых полей может восприниматься по-разному. В search phase ContextRecord чаще связан с exception context: нужно понять, где произошло исключение и какой __try region покрывает этот ControlPc. В unwind phase ContextRecord может отражать текущий unwind context: система уже поднимается по кадрам и вызывает cleanup-код. EstablisherFrame при этом

по-прежнему указывает на кадр, который устанавливает handler, но роль handler-а может быть не "поймать исключение", а выполнить завершение.

Именно поэтому Microsoft в описаниях различает exception handler case и termination handler case. Для beginner-а это выглядит как формальность. Для reverse engineer-а это ключ: один и тот же callback prototype обслуживает разные сценарии. Handler вызывается не только для вопроса "будешь ли ты ловить это исключение?", но и для вопроса "должен ли ты выполнить cleanup при размотке?". DispatcherContext несёт данные, необходимые для обоих сценариев.

Search phase (фаза поиска обработчика) - этап, на котором система поднимается по кадрам и спрашивает language-specific handlers, может ли кто-то обработать исключение. На этом этапе стек ещё не обязательно реально размотан до обработчика.
Unwind phase (фаза размотки) - этап, на котором система реально проходит по кадрам к найденной цели, вызывает termination handlers и восстанавливает/обновляет контексты.
Exception handler case (случай обработчика исключения) - ситуация, когда handler рассматривает исходное исключение и может решить, обрабатывать его или продолжать поиск.
Termination handler case (случай обработчика завершения) - ситуация, когда handler вызывается во время unwind для cleanup, например для __finally.

EXCEPTION_DISPOSITION: как handler отвечает диспетчеру

Раз уж DispatcherContext передаётся language-specific handler-у, нужно понять и обратную сторону вызова: что handler возвращает. Прототип PEXCEPTION_ROUTINE возвращает EXCEPTION_DISPOSITION. Это значение можно воспринимать как ответ handler-а диспетчеру. Handler не просто выполняется и исчезает. Он сообщает системе, что делать дальше.

В разных сценариях handler может сказать: продолжать поиск, выполнить вложенную обработку, продолжить выполнение или что произошла collided unwind-ситуация. Важно не превращать это в магический enum без смысла. EXCEPTION_DISPOSITION - это часть протокола общения между dispatcher-ом и language-specific handler-ом. Dispatcher даёт handler-у ExceptionRecord, EstablisherFrame, ContextRecord и DispatcherContext. Handler смотрит свои данные, принимает решение и возвращает disposition.

Для reverse engineering это важно потому, что control flow при исключениях не всегда виден как обычные call/jmp в дизассемблере. Часть управления существует как протокол между Windows exception dispatcher-ом и language-specific handler-ами. Если смотреть только на инструкции программы, можно не увидеть, почему выполнение внезапно попадает в __except block или __finally. Нужно понимать этот скрытый диспетчерский протокол.

EXCEPTION_DISPOSITION (решение/распоряжение обработчика исключения) - возвращаемое значение language-specific handler-а, через которое handler сообщает dispatcher-у, что делать дальше: продолжать поиск, считать исключение обработанным, учесть collided unwind и т.п.
Protocol (протокол взаимодействия) - заранее определённый порядок обмена данными между участниками системы. Здесь протокол такой: dispatcher вызывает handler с набором параметров, handler возвращает disposition, dispatcher продолжает работу согласно ответу.

Control flow (поток управления) - последовательность переходов выполнения программы. Exception handling создаёт неявные ветки control flow, которые не всегда выглядят как прямые инструкции jmp/call в исходной функции.

DispatcherContext как граница между системным и языковым слоями

Теперь можно сформулировать более глубокую мысль. DispatcherContext находится на границе двух уровней: системного и языкового. Системный уровень знает про .pdata, .xdata, UNWIND_INFO, RUNTIME_FUNCTION, ContextRecord, ControlPc, ImageBase и unwind. Языковой уровень знает про __try, __except, __finally, C++ catch, destructors, filters, cleanup и таблицы компилятора. DispatcherContext является интерфейсом между ними.

Системный уровень не хочет знать, как именно MSVC кодирует все области __try. Он только должен передать HandlerData туда, где это понимают. Языковой уровень не хочет заново искать RUNTIME_FUNCTION по RIP и разбирать PE sections. Он получает FunctionEntry, ImageBase и ControlPc уже подготовленными. Такое разделение делает систему расширяемой: разные компиляторы и языки могут иметь свои handler data, пока они соблюдают общий протокол x64 exception handling.

Для книги по reverse engineering это одна из центральных идей. Windows x64 exception handling - это не один монолитный алгоритм, где всё жёстко закодировано в одном месте. Это слоёная система. На нижнем уровне архитектура и стек. Затем ABI и calling convention. Затем .pdata и .xdata. Затем RtlLookupFunctionEntry и RtlVirtualUnwind. Затем DispatcherContext как упаковка текущего кадра. Затем language-specific handler, который уже понимает конкретный язык. Если перепутать уровни, анализ становится грязным.

System layer (системный слой) - часть механизма, которую обеспечивает Windows: поиск function table entries, unwind, передача контекстов, вызов handlers по общему протоколу.

Language layer (языковой слой) - часть механизма, которую определяет компилятор и язык: формат handler data, semantics __try/__except/__finally, C++ destructors/catch и т.п.

Interface (интерфейс) - граница взаимодействия между слоями. DispatcherContext является интерфейсной структурой: системный слой заполняет её, языковой handler читает её.

Если держать эту границу в голове, становится легче читать и документацию, и дизассемблер: UNWIND_INFO не обязан объяснять C++ catch; __C_specific_handler не обязан заново строить .pdata; DispatcherContext переносит нужные значения между слоями.

Как это выглядит пошагово при вызове language-specific handler-а

Соберём всё в один пошаговый рассказ. Пусть исключение произошло внутри функции, у которой есть UNW_FLAG_EHANDLER. Система начинает search phase. В ContextRecord сохранено состояние момента исключения. По RIP/ControlPc система находит RUNTIME_FUNCTION через active function tables. По UnwindInfoAddress она читает UNWIND_INFO. Видит, что у функции есть exception handler. Проверяет, не находится ли ControlPc в прологе или эпилоге, потому что в этих областях handler для тела функции не должен применяться как обычно.

Если ControlPc находится в теле функции, dispatcher получает address of exception handler и HandlerData. Затем RtlVirtualUnwind или связанная unwind-логика вычисляет

EstablisherFrame для текущего кадра. После этого система формирует DISPATCHER_CONTEXT: записывает ControlPc, ImageBase, FunctionEntry, EstablisherFrame, TargetIp, ContextRecord, LanguageHandler и HandlerData. И только теперь вызывается LanguageHandler.

LanguageHandler получает два уровня информации одновременно. Через отдельные параметры он видит ExceptionRecord, EstablisherFrame и ContextRecord. Через DispatcherContext он видит расширенную карточку текущего кадра. Например, __C_specific_handler может использовать ControlPc и HandlerData, чтобы проверить, входит ли исключение в нужный __try block. Если filter expression говорит EXCEPTION_EXECUTE_HANDLER, начинается подготовка к unwind phase. Если нет - handler возвращает решение продолжать поиск, и dispatcher поднимается к caller-у.

```
Упрощённая логика одного кадра:

1. ContextRecord->Rip даёт исходный ControlPc.
2. RtlLookupFunctionEntry(ControlPc, &ImageBase, HistoryTable) даёт FunctionEntry.
3. FunctionEntry->UnwindInfoAddress ведёт к UNWIND_INFO.
4. UNWIND_INFO flags говорят, есть ли LanguageHandler.
5. HandlerData находится после unwind codes и handler RVA.
6. RtlVirtualUnwind вычисляет caller context и EstablisherFrame.
7. Dispatcher создаёт DISPATCHER_CONTEXT для текущего кадра.
8. LanguageHandler(ExceptionRecord, EstablisherFrame, ContextRecord, &DispatcherContext)
принимает решение через EXCEPTION_DISPOSITION.
```

Почему DispatcherContext важен для отладчика

В отладчике DispatcherContext становится способом увидеть exception machinery не как абстракцию, а как живой объект. Если поставить breakpoint на __C_specific_handler или другой language-specific handler, можно увидеть параметры вызова. Среди них будет DispatcherContext. Через него можно проверить, какой ControlPc рассматривается, какой ImageBase используется, какая FunctionEntry найдена, куда указывает HandlerData и какой LanguageHandler был вызван.

Это полезно, когда обычный stack trace непонятен. Например, программа упала, а ты видишь, что перед переходом в handler вызываются системные функции. Без понимания DispatcherContext кажется, будто Windows "магически" прыгает в обработчик. Но если посмотреть поля структуры, станет ясно: никакой магии нет. Есть текущий кадр, его metadata, handler data и решение диспетчера вызвать конкретный language-specific handler.

Для анализа защит и обфускации это ещё важнее. Обфускатор может играть не только машинными переходами, но и exception-driven control flow: строить управление через исключения, handler-и и unwind. Тогда DispatcherContext помогает понять, какой кадр действительно устанавливает обработчик и какие данные ему передаются. Это уже уровень ниже, чем просто "IDA показала блок catch".

Breakpoint (точка останова) - отладочная точка, при достижении которой выполнение программы останавливается. Breakpoint на language-specific handler позволяет увидеть реальные параметры exception machinery.

Stack trace (трасса стека) - список кадров вызовов, восстановленный по стеку и unwind metadata. DispatcherContext не заменяет stack trace, но объясняет, как конкретный кадр участвует в обработке исключения.

Exception-driven control flow (поток управления через исключения) - техника, при которой переходы выполнения строятся не обычными jmp/call, а через RaiseException, faults, handlers и unwind. Это может быть легитимной логикой или приёмом обфускации.

Что должен делать static /dynamic с DispatcherContext

Для твоего static /dynamic DispatcherContext важен не как структура, которую всегда можно прочитать из файла, а как концептуальная модель runtime-связки. Static analyzer может заранее подготовить данные: какие функции имеют handler flags, где лежит HandlerData, какой LanguageHandler RVA указан, какие RUNTIME_FUNCTION entries ведут к этим UNWIND_INFO, какие ranges покрывают подозрительный код. Но сам DispatcherContext появляется только когда есть конкретный ControlPc и конкретный stack frame.

Поэтому правильная архитектура отчёта должна разделять два вида утверждений. Первое: static evidence, то есть факты из файла. Например: "UNWIND_INFO функции имеет UNW_FLAG_EHANDLER; handler RVA указывает в .text; HandlerData находится после массива unwind codes; FunctionEntry покрывает диапазон X-Y". Второе: runtime implication, то есть что из этого будет следовать во время выполнения. Например: "если ControlPc попадёт в этот диапазон и handler будет вызван, DispatcherContext будет связывать этот FunctionEntry, ImageBase, HandlerData и EstablisherFrame текущего кадра".

Такой подход защищает анализатор от чрезмерных заявлений. Он не говорит, что видел реальный DispatcherContext, если он его не видел. Но он показывает, какие поля будут формироваться и какие связи подозрительны. Например, если HandlerData указывает за пределы ожидаемой секции, LanguageHandler находится в необычной области, FunctionEntry пересекается с другим диапазоном или UNWIND_INFO имеет странные flags, static может выдать finding с объяснением, как это повлияет на future DispatcherContext.

Static evidence (статическое доказательство) - факт, который можно проверить в PE-файле без запуска: значения таблиц, секции, RVA, размеры, flags, диапазоны.

Runtime implication (следствие во время выполнения) - вывод о том, как статические данные будут участвовать в runtime-механизме при конкретном ControlPc, ImageBase и stack frame.

Finding (находка анализатора) - сформулированный результат проверки: что обнаружено, где обнаружено, почему это важно, насколько уверенно и как это может влиять на выполнение или анализ.

Confidence (уровень уверенности) - оценка того, насколько твёрдо finding подтверждён данными. Для DispatcherContext важно различать "наблюдали в runtime" и "предсказали по статическим метаданным".

Проверка static	Что она даёт	Как связана с DispatcherContext
Handler RVA валиден и находится в исполняемой области	Показывает, что LanguageHandler потенциально вызываем.	Поле LanguageHandler будет указывать на этот адрес.
HandlerData находится в ожидаемой readable области	Снижает риск повреждённой или обфусцированной metadata.	Поле HandlerData будет передано handler-у.

Проверка static	Что она даёт	Как связана с DispatcherContext
FunctionEntry диапазон не пересекается аномально с соседями	Повышает доверие к поиску по ControlPc.	Поле FunctionEntry в DispatcherContext будет корректно описывать кадр.
UNWIND_INFO flags согласованы	Показывает, есть ли handler/chained info и нет ли конфликтов.	Определяет, будет ли вообще LanguageHandler/HandlerData.
ImageBase/RVA пересчёты корректны	Защищает от ошибок VA/RVA parsing.	DispatcherContext использует runtime ImageBase для реальных адресов.

Типичные ошибки понимания DispatcherContext

Первая ошибка - считать DispatcherContext статическим объектом из PE. Это неверно. В PE есть исходные кирпичи, но не готовая структура DispatcherContext. Она формируется в runtime.

Вторая ошибка - смешивать DispatcherContext и ContextRecord. ContextRecord хранит состояние процессора. DispatcherContext хранит картину текущего кадра для диспетчера и handler-а. Они связаны, но не взаимозаменяемы.

Третья ошибка - думать, что HandlerData всегда имеет один формат. Общая структура DISPATCHER_CONTEXT даёт PVOID HandlerData именно потому, что формат зависит от LanguageHandler. Без знания handler-а глубокий разбор HandlerData может быть гаданием.

Четвёртая ошибка - игнорировать TargetIp. Пока мы смотрим только search phase, TargetIp может казаться второстепенным. Но при реальной unwind phase и RtlUnwindEx он становится частью цели размотки.

Пятая ошибка - думать, что LanguageHandler сам всё ищет с нуля. На самом деле dispatcher уже передаёт ему FunctionEntry, ImageBase, EstablisherFrame, ControlPc и HandlerData. Это не случайный набор, а результат предыдущих этапов unwind-механизма.

Если кратко: DispatcherContext нельзя понимать как "кучу параметров для красоты". Это собранная runtime-карточка кадра, без которой language-specific handler не получил бы нормальной связи между исключением, функцией, модулем, метаданными и стековым кадром.

Минимальная mental model для ручного анализа

Для ручного анализа удобно держать в голове такую модель. ControlPc говорит: "какое место функции сейчас проверяем". ImageBase говорит: "от какой базы переводим все RVA". FunctionEntry говорит: "какая .pdata-запись описывает этот код". EstablisherFrame говорит: "какой конкретный кадр стека установил этот handler". TargetIp говорит: "куда должна привести размотка". ContextRecord говорит: "какое процессорное/unwind-состояние сейчас актуально". LanguageHandler говорит: "какую функцию обработчика вызываем". HandlerData говорит: "какие данные этот handler должен читать".

Если хотя бы одно из этих значений понять неверно, вся картина exception handling может поехать. Неправильный ImageBase превращает RVA в мусор. Неправильный FunctionEntry ведёт к не той UNWIND_INFO. Неправильный EstablisherFrame ломает привязку к кадру. Неправильный ContextRecord заставляет handler смотреть на не тот RIP/RSP. Неправильно разобранный HandlerData превращает __try/__except в набор бессмысленных байтов.

Поэтому DispatcherContext - хороший проверочный список для reverse engineer-а. Если ты можешь объяснить каждое его поле применительно к конкретному исключению, значит ты понимаешь не только таблицы PE, но и живой механизм Windows x64 exception handling.

Проверка понимания: если ты можешь ответить, откуда взялось каждое поле DISPATCHER_CONTEXT и кто его будет использовать, значит ты уже видишь exception handling не как "магический SEH", а как связку PE metadata, runtime context и language-specific logic.

Конкретный пример: функция с __try/__except

Чтобы DispatcherContext перестал быть абстрактным набором полей, рассмотрим обычную C-функцию с конструкцией __try/__except. В исходном коде программист видит довольно простую картину: есть защищённый блок, внутри которого может произойти access violation, и есть фильтр, который решает, обрабатывать исключение или нет. Но на машинном уровне никакой магической записи "try" в стеке x64 нет. Компилятор должен превратить эту языковую конструкцию в metadata, а Windows должна суметь вызвать нужный handler в нужный момент.

При компиляции функция получает обычный машинный код в .text. Для unwind она получает RUNTIME_FUNCTION в .pdata и UNWIND_INFO в .xdata. Если функция содержит SEH-обработку, UNWIND_INFO получает handler flag и адрес language-specific handler-а. Для MSVC таким обработчиком часто выступает __C_specific_handler. После адреса handler-а лежат данные, которые описывают scope table: какие диапазоны кода защищены, какие filter expressions связаны с ними, куда переходить при обработке.

Теперь происходит исключение внутри защищённого блока. Windows находит FunctionEntry и UNWIND_INFO. Видит, что функция имеет exception handler. Но сам факт наличия handler-а ещё не отвечает, относится ли конкретный ControlPc к нужному __try. Поэтому dispatcher вызывает __C_specific_handler и передаёт ему DispatcherContext. Внутри него есть ControlPc, HandlerData и EstablisherFrame. Именно эта тройка позволяет C-specific handler-у сказать: "да, исключение произошло внутри такого-то protected range, для него есть такой-то filter, а если filter согласится, unwind должен привести выполнение к такому-то handler block".

__try (защищённый блок Microsoft SEH) - языковая конструкция, обозначающая участок кода, для которого задана обработка исключений или завершения. На x64 она реализуется не через стековую SEH-цепочку как на x86, а через metadata и language-specific handler.

__except (блок обработки исключения / фильтрованный обработчик) - часть SEH-конструкции, где filter expression решает, обрабатывать исключение или продолжать поиск. На машинном уровне логика __except связана с таблицами, которые понимает __C_specific_handler.

Scope table (таблица областей действия обработчиков) - compiler-specific таблица, описывающая диапазоны защищённого кода и связанные с ними фильтры/обработчики. Это пример HandlerData для C-specific handler-а, но общий unwind-механизм не обязан понимать

её формат.

```
Упрощённый исходный код:

void f() {
__try {
*(int*)0 = 123; // access violation внутри protected region
}
__except(filter()) {
recover();
}
}

Что видит Windows x64 на нижнем уровне:
.pdata -> RUNTIME_FUNCTION для f
.xdata -> UNWIND_INFO для f
flags -> UNW_FLAG_EHANDLER
handler -> __C_specific_handler
HandlerData -> таблица ranges/filter/handler для этой функции

DispatcherContext связывает:
ControlPc -> где именно внутри f произошло исключение
FunctionEntry -> какая запись .pdata описывает f
EstablisherFrame -> какой конкретный кадр f сейчас активен
HandlerData -> таблица __try/__except для f
```

Почему HandlerData без ControlPc почти бесполезна

Очень легко переоценить HandlerData. Кажется: вот есть таблица обработчиков, значит по ней уже можно понять всё. Но handler data без ControlPc похожа на карту города без отметки "вы находитесь здесь". В таблице могут быть несколько protected ranges, несколько фильтров и несколько handlers. Нужно понять, какой именно диапазон относится к текущему исключению.

ControlPc превращает HandlerData из общей схемы функции в конкретное решение. Handler берёт ControlPc, сравнивает его с диапазонами внутри своих language-specific данных и определяет, покрывается ли текущая точка protected region. Если покрывается, он смотрит, какой filter или finally связан с этим диапазоном. Если не покрывается, handler должен вернуть решение продолжать поиск.

В reverse engineering это объясняет важную вещь: по одному факту наличия __C_specific_handler нельзя автоматически восстановить точный high-level control flow. Нужно ещё разобрать HandlerData и понять соответствие range -> filter -> handler. А для runtime-решения нужен ControlPc. Поэтому DispatcherContext хранит оба элемента одновременно: и координату события, и указатель на таблицу.

Protected range (защищённый диапазон) - участок машинного кода, который соответствует языковому защищённому блоку. Handler должен проверить, находится ли ControlPc внутри такого диапазона.

Filter expression (фильтрующее выражение) - код в __except(...), который возвращает решение: обработать исключение, продолжить поиск или продолжить выполнение. Это не просто комментарий компилятора, а реальный код, вызываемый через SEH-механику.

Range check (проверка попадания в диапазон) - сравнение адреса ControlPc с границами protected range. Без такой проверки handler не знает, относится ли исключение к конкретному __try.

Почему EstablisherFrame без HandlerData тоже недостаточен

Обратная сторона та же самая. EstablisherFrame сообщает, какой stack frame сейчас устанавливает обработчик. Но сам по себе он не объясняет, какие protected ranges есть в функции, какие filters существуют и куда нужно перейти при обработке. EstablisherFrame - это живой экземпляр кадра, а HandlerData - схема языковой обработки для функции.

Если у нас есть только EstablisherFrame, мы знаем "чей кадр". Если есть только HandlerData, мы знаем "какая схема". Полезное решение появляется только при их соединении. DispatcherContext как раз делает это соединение явным: в одной структуре лежит и EstablisherFrame, и HandlerData, и ControlPc, и FunctionEntry.

Для анализа вредоносных или защищённых программ это особенно важно. Некоторые техники могут пытаться играть с обработчиками, искусственно поднимать исключения или строить управление через SEH. Если анализатор смотрит только на handler RVA, он видит лишь функцию обработчика. Если смотрит только на stack frame, он видит лишь runtime-состояние. Но DispatcherContext-подход заставляет связывать все элементы сразу: где произошло, в каком кадре, по какой таблице, каким обработчиком и при каком контексте.

SEH-based obfuscation (обфускация через SEH) - техника усложнения control flow, при которой исключения и обработчики используются как часть логики переходов. Для x64 такая техника часто опирается на unwind metadata и language-specific handlers.

Handler RVA (RVA обработчика) - относительный адрес language-specific handler-а, записанный в UNWIND_INFO при наличии handler flags. Это только вход в обработчик, а не полная картина обработки исключения.

DispatcherContext в search phase: вопрос "подходит ли этот кадр?"

В search phase DispatcherContext помогает handler-у ответить на вопрос: "этот кадр должен обработать исключение или нет?" Система ещё не обязана разрушать кадры стека и выполнять cleanup. Она идёт вверх по стеку и спрашивает каждого подходящего language-specific handler-а, есть ли у него право и желание обработать текущее исключение.

В этой фазе особенно важны ExceptionRecord, ControlPc, HandlerData и EstablisherFrame. ExceptionRecord сообщает природу исключения: например access violation, illegal instruction или пользовательское RaiseException. ControlPc показывает, где внутри функции произошло событие. HandlerData даёт таблицу protected ranges и filters. EstablisherFrame привязывает всё к конкретному кадру.

Если filter возвращает "continue search", dispatcher не должен воспринимать это как ошибку. Это нормальный ответ: текущий кадр говорит, что он не берёт исключение на себя. Тогда система использует unwind-механику, чтобы перейти к caller context, и повторяет проверку для следующего кадра. Так search phase может пройти через много функций, пока не найдёт подходящий handler или не дойдёт до top-level/unhandled logic.

Continue search (продолжить поиск) - решение handler-а, означающее, что текущий кадр не обрабатывает исключение и dispatcher должен подняться к следующему кадру.

Unhandled exception (необработанное исключение) - ситуация, когда после прохода по доступным кадрам не найден handler, который обработает исключение. Тогда включается top-level/unhandled exception handling.

RaiseException (поднять исключение программно) - Windows API, позволяющая программе явно создать exception event. В дальнейшем оно проходит через похожий механизм dispatch/search/unwind.

DispatcherContext в unwind phase: вопрос "что выполнить при размотке?"

В unwind phase DispatcherContext используется в другой логике. Теперь система уже знает цель размотки. Её задача - пройти промежуточные кадры и выполнить то, что должно выполниться при выходе: __finally, cleanup, destructors, release resources. Это не просто чтение stack trace. Это реальное изменение хода выполнения и вызов termination handlers.

В этой фазе ContextRecord может уже быть current unwind context, а не исходным exception context. TargetIp становится важнее, потому что unwind идёт к целевой точке продолжения. EstablisherFrame показывает, какой кадр сейчас нужно обработать. HandlerData говорит, какие termination handlers связаны с этим кадром. LanguageHandler вызывается уже не обязательно для поиска "кто поймает", а для выполнения действий завершения.

Поэтому один и тот же DispatcherContext-подход обслуживает две разные задачи. В search phase он помогает принять решение о пригодности handler-а. В unwind phase он помогает корректно выполнить cleanup для кадра. Если эти фазы смешать, можно неправильно понять, почему handler вызывается даже тогда, когда он не является конечным __except block. Он может быть вызван как termination handler по пути к целевому обработчику.

Cleanup (очистка ресурсов) - действия, которые нужно выполнить при выходе из области: закрыть handle, освободить память, вызвать destructor, снять lock. В SEH для этого часто используется __finally.

__finally (блок завершения) - SEH-конструкция, код которой должен выполниться при выходе из защищённой области, в том числе при размотке из-за исключения.

Destructor (деструктор) - функция C++, вызываемая при уничтожении объекта. В exception handling деструкторы локальных объектов часто выполняются во время unwind phase.

Коллизия смыслов: DispatcherContext и "контекст" в обычной речи

Слово context в этой теме встречается слишком часто, и это может раздражать. ContextRecord, DispatcherContext, unwind context, exception context, CPU context. Но это не случайная путаница, а отражение того, что механизм исключений работает на нескольких уровнях одновременно.

CPU context - это низкий уровень: регистры процессора. Exception context - это CPU context в момент исключения. Unwind context - это CPU context, изменяемый по мере размотки. DispatcherContext - это не регистры, а служебная карточка текущего кадра. HandlerData context - это уже языковая таблица, которую понимает конкретный handler. Если всё это назвать просто "контекст", анализ быстро превращается в кашу.

Поэтому в книге дальше нужно держать дисциплину терминов. Когда речь о регистрах - говорим ContextRecord или CPU context. Когда речь о полях для handler-a - говорим DispatcherContext. Когда речь о конкретном stack frame - говорим EstablisherFrame или stack frame. Когда речь о compiler tables - говорим HandlerData или language-specific data. Такая точность не занудство, а способ не потеряться в Windows x64 SEH.

Terminology discipline (дисциплина терминов) - привычка использовать разные слова для разных уровней механизма. В reverse engineering это снижает риск ложных выводов, особенно когда несколько структур имеют похожие названия.

Динамические таблицы и JIT: откуда берётся FunctionEntry в необычном коде

В обычном PE-модуле FunctionEntry приходит из статической .pdata. Но Windows x64 должна поддерживать не только классические EXE/DLL, скомпилированные линкером. Есть JIT, dynamic code generation, runtime-генерация функций, иногда manual mapping. Для такого кода FunctionEntry может прийти из dynamic function table, добавленной через RtlAddFunctionTable или предоставленной callback-механизмом.

DispatcherContext от этого не перестаёт быть полезным. Наоборот, он показывает, что system layer не обязан знать, была ли function table статической или динамической. Если active function tables смогли дать FunctionEntry для ControlPc, дальше handler получает DispatcherContext с тем же набором смысловых полей. Источник FunctionEntry может быть другим, но роль поля остаётся прежней: оно описывает текущую функцию или фрагмент для unwind.

Для static /dynamic это значит, что статический анализ PE не покрывает весь runtime-мир. Если программа использует JIT или регистрирует function tables динамически, реальный DispatcherContext может ссылаться на FunctionEntry, которого не было в исходном .pdata файла. Поэтому в отчётах нужно различать static .pdata coverage и runtime active function table coverage.

JIT (just-in-time compilation, компиляция во время выполнения) - техника, при которой машинный код создаётся во время работы программы. Такой код тоже должен иметь unwind metadata, если он участвует в исключениях и stack walking на x64.

Dynamic function table (динамическая таблица функций) - таблица RUNTIME_FUNCTION entries, зарегистрированная во время выполнения, например для JIT-кода. Она входит в active function tables и может использоваться RtlLookupFunctionEntry.

Active function tables (активные таблицы функций) - совокупность function tables, доступных runtime-поиску: статические таблицы загруженных образов и зарегистрированные динамические таблицы.

Manual mapping (ручная загрузка модуля) - загрузка PE не стандартным Windows loader-ом, а пользовательским кодом. Если при этом не зарегистрировать или неправильно обработать unwind metadata, exception handling и stack walking могут работать некорректно.

Безопасность и повреждённые метаданные

DispatcherContext собирается из разных источников: часть приходит из текущего процессорного состояния, часть из loader/module state, часть из function tables, часть из

.xdata. Если один из источников повреждён или намеренно подменён, структура может стать логически опасной. Например, HandlerData может указывать в неожиданную область, FunctionEntry может иметь некорректные границы, LanguageHandler может оказаться в нетипичной секции, а ImageBase/RVA-пересчёт может вести за пределы образа.

Windows runtime имеет свои проверки и правила, но reverse engineer не должен слепо доверять каждой таблице в подозрительном бинаре. Задача анализатора - не только распарсить значения, но и оценить их правдоподобие. DispatcherContext как модель помогает сформулировать вопросы: если эта запись будет использована при исключении, какой LanguageHandler будет вызван? Какие HandlerData ему передадут? К какому диапазону FunctionEntry это относится? Не ведёт ли цепочка к области, которая выглядит как данные, а не код?

Особенно аккуратно нужно относиться к packers и dynamic s. Они могут намеренно создавать странные unwind metadata, чтобы ломать отладчики, crash dump analysis, stack walking или статические инструменты. Не всякая странность означает вредоносность, но всякая странность должна быть описана через evidence: какое поле, какой диапазон, какое expected rule нарушено и как это повлияет на DispatcherContext при runtime.

Packer (упаковщик исполняемого файла) - инструмент, который изменяет структуру программы, часто сжимая/шифруя код и распаковывая его во время выполнения. Packers могут влиять на PE sections, metadata и анализ unwind.
dynamic (защитник/обфускатор программы) - инструмент защиты кода, который может усложнять reverse engineering через обфускацию control flow, metadata tricks, anti-debugging и другие приёмы.
Plausibility check (проверка правдоподобия) - проверка того, выглядит ли значение согласованным с ожидаемой структурой: адрес в допустимой секции, RVA внутри image, корректное выравнивание, непротиворечивые flags.
Опасная ошибка: доверять HandlerData только потому, что до него удалось дойти через UNWIND_INFO. Дойти - не значит доказать корректность. Нужны проверки диапазона, секции, выравнивания, связи с LanguageHandler и общей согласованности.

Как читать DispatcherContext в WinDbg концептуально

Даже если сейчас мы не строим практическую лабораторную работу, полезно понимать, как эта структура проявляется в отладчике. Когда выполнение остановлено внутри language-specific handler-a, параметры функции и регистры вызова по Windows x64 calling convention могут указывать на ExceptionRecord, EstablisherFrame, ContextRecord и DispatcherContext. Напомню: первые четыре integer/pointer arguments на Windows x64 передаются через RCX, RDX, R8 и R9.

Это значит, что при входе в __C_specific_handler логически RCX связан с ExceptionRecord, RDX - с EstablisherFrame, R8 - с ContextRecord, R9 - с DispatcherContext. Детали могут зависеть от оптимизаций, обёрток и конкретной точки остановки, но сама calling convention объясняет, почему четвертый аргумент оказывается именно в R9. А уже через этот указатель можно смотреть поля DispatcherContext.

Так соединяются две главы книги: calling convention и exception dispatch. Без знания calling convention ты видишь в отладчике набор регистров и не понимаешь, почему R9 внезапно важен. Без знания DispatcherContext ты видишь указатель в R9, но не понимаешь, что это карточка текущего кадра exception machinery. Фундаментальная теория нужна именно для таких пересечений.

WinDbg (Windows Debugger, отладчик Windows) - отладчик Microsoft для анализа процессов, драйверов и crash dumps. Он позволяет смотреть регистры, память, stack trace и структуры Windows.

Calling convention argument registers (регистры передачи аргументов) - по Windows x64 первые четыре целочисленных/указательных аргумента обычно передаются через RCX, RDX, R8 и R9. Это помогает понимать параметры вызова __C_specific_handler в отладчике.

```
Логика параметров __C_specific_handler по Windows x64 calling convention:

RCX -> ExceptionRecord
RDX -> EstablisherFrame
R8 -> ContextRecord
R9 -> DispatcherContext

Это не "магия WinDbg", а следствие ABI: первые четыре pointer/integer
аргумента передаются через RCX, RDX, R8, R9.
```

DispatcherContext и PDB: почему имена не решают механизм

PDB может дать имена функций, исходные строки, типы и иногда сильно облегчить отладку. Но DispatcherContext не зависит от того, есть ли у тебя PDB. Windows exception machinery работает по .pdata, .xdata, active function tables, CONTEXT и runtime state. Символы помогают человеку, но не являются фундаментом unwind.

Если PDB есть, ты можешь красивее назвать LanguageHandler, исходную функцию, protected ranges и переменные. Если PDB нет, механизм всё равно работает: FunctionEntry всё равно найден, HandlerData всё равно передан, EstablisherFrame всё равно вычислен. Поэтому reverse engineer должен уметь читать оба слоя. PDB - это удобная карта с подписями. DispatcherContext - это рабочий runtime-документ, по которому система принимает решения.

Это важно для защищённых бинарей. Они почти никогда не дают тебе нормальный PDB. Если теория построена только на символах, анализ ломается. Если теория построена на .pdata/.xdata/DispatcherContext, отсутствие имён неприятно, но не уничтожает понимание. Ты можешь не знать красивого имени функции, но видеть её FunctionEntry, ControlPc, handler RVA и HandlerData.

PDB (Program Database, база отладочной информации) - файл Microsoft с символами, типами, исходными строками и другой debug information. Он помогает человеку и инструментам, но не является обязательной основой Windows x64 unwind.

Symbol (символ отладки / имя) - имя функции, переменной или другого объекта, известное из debug information. Symbol удобен, но механизм exception handling опирается не на имена, а на метаданные и адреса.

Переход к следующей главе

Теперь мы закончили главу о том, как Windows реально поднимается по стеку на уровне отдельных системных сущностей. Мы разобрали путь от RIP к FunctionEntry, затем RtlLookupFunctionEntry как механизм поиска записи, затем RtlVirtualUnwind как механизм вычисления caller context, затем EstablisherFrame как runtime-привязку handler-а к кадру и DispatcherContext как полную карточку текущего кадра для language-specific handler-а.

Следующая логическая тема - фазы обработки исключения. До сих пор мы много раз упоминали search phase и unwind phase, но пока использовали их как предварительные понятия. Теперь можно разобрать их полноценно. Это важный переход: раньше мы смотрели на отдельные кирпичи, теперь будем смотреть на сценарий целиком. Как именно Windows сначала ищет обработчик? Почему в search phase стек ещё не обязан быть реально размотан? Почему потом начинается unwind phase? Когда вызываются __finally и termination handlers? Почему обработчик, который "согласился" принять исключение, не просто вызывается обычным call из текущего места?

Без DispatcherContext этот переход был бы преждевременным. Теперь же у нас есть все нужные детали: FunctionEntry показывает функцию, UNWIND_INFO показывает правила, RtlVirtualUnwind вычисляет кадр, EstablisherFrame связывает handler с экземпляром кадра, DispatcherContext передаёт handler-у полную картину. Значит, можно переходить от устройства деталей к динамике всего процесса.

Глава 6. Фазы обработки исключения

6.1. Что такое search phase и unwind phase

Мы подошли к месту, где отдельные механизмы Windows x64 начинают собираться в один большой процесс. До этого мы долго разбирали кирпичи: PE-файл, Exception Directory, .pdata, RUNTIME_FUNCTION, .xdata, UNWIND_INFO, UNWIND_CODE, пролог, эпилог, volatile и nonvolatile регистры, calling convention, RtlLookupFunctionEntry, RtlVirtualUnwind, EstablisherFrame и DispatcherContext. По отдельности эти темы могли выглядеть как набор таблиц и структур. Но в реальной системе они существуют не ради самих себя. Они нужны для одного центрального действия: когда выполнение программы сорвалось из нормального хода, Windows должна понять, кто имеет право обработать это событие, и как безопасно добраться до этого обработчика.

Именно здесь появляются две фазы, без которых нельзя нормально понять SEH на x64: search phase и unwind phase. Эти названия часто переводят слишком бедно: search phase - просто "фаза поиска", unwind phase - просто "фаза раскрутки". Формально это верно, но такого перевода мало. Он не объясняет, что именно ищется, почему нельзя сразу прыгнуть в обработчик, зачем проходить стек два раза, почему в одной фазе можно только спрашивать обработчики, а в другой уже разрушать промежуточные кадры. Поэтому этот раздел нужно читать не как еще два термина, а как объяснение внутреннего ритма всего механизма исключений.

> search phase (фаза поиска обработчика)
> Это первая логическая фаза frame-based обработки исключения. Система идет от текущего кадра стека к более старым кадрам и выясняет, есть ли где-то обработчик, который согласен принять текущее исключение. Важный смысл: на этой фазе система еще не обязана разрушать стековые кадры. Она в основном спрашивает: "кто будет отвечать за это исключение?"

> unwind phase (фаза раскрутки стека / фаза размотки стека)
> Это вторая логическая фаза. Она начинается только после того, как найден целевой обработчик. Теперь система должна привести поток выполнения к этому обработчику: пройти назад по кадрам, вызвать нужные termination handlers, выполнить cleanup-логику, восстановить контексты и убрать те кадры, которые больше не продолжат обычное выполнение.

Почему Windows не может сразу перейти к обработчику

Наивное представление обычно такое: произошла ошибка, Windows нашла ближайший __except, прыгнула туда, готово. Но реальная картина сложнее. Между точкой падения и обработчиком может лежать несколько функций. У каждой функции мог быть собственный стековый кадр, сохраненные nonvolatile-регистры, локальные объекты, __finally-блоки, C++ destructors, промежуточные обработчики, динамически выделенный стек, frame pointer или особая unwind-метаинформация. Если просто перепрыгнуть в обработчик, не разобравшись с этими кадрами, программа окажется в состоянии, которое не соответствует ни машинному коду, ни правилам языка, ни ожиданиям runtime-библиотеки.

Представь цепочку вызовов. Функция A вызвала B, B вызвала C, C вызвала D. Исключение произошло внутри D. Обработчик, который реально хочет обработать исключение, находится не в D и не в C, а в B. Чтобы попасть из D в обработчик B, нельзя просто поставить RIP на адрес handler-a. Надо понять, что делать с D и C. Если у них были __finally-блоки, их нужно вызвать. Если были сохраненные регистры, нужно правильно восстановить состояние. Если C++ объекты должны быть разрушены, language-specific runtime должен получить шанс это сделать. Поэтому Windows сначала ищет цель, а потом отдельно выполняет раскрутку пути к этой цели.

Главная идея
Search phase отвечает на вопрос: "кто обработает исключение?" Unwind phase отвечает на вопрос: "как безопасно дойти до этого обработчика, не оставив стек и runtime в сломанном состоянии?" Эти два вопроса нельзя смешивать.

Нормальное выполнение и исключительное выполнение

Чтобы почувствовать смысл двух фаз, нужно вспомнить обычный вызов функции. При нормальном выполнении функция возвращается сама: она доходит до эпилога, восстанавливает стек, выполняет ret, и управление возвращается вызывающей функции. Это обычная дорога назад. Она проложена машинными инструкциями самой функции.

Исключение ломает эту привычную дорогу. Если внутри D произошел access violation, функция D может не дойти до своего эпилога. Она может находиться посреди тела, в середине пролога, в середине работы с локальными данными. Обычного ret может не быть. Тем не менее системе все равно нужно понять, как выглядел бы переход к вызывающему кадру, если бы мы могли безопасно подняться на уровень выше. Для этого и используется RtlVirtualUnwind. Но RtlVirtualUnwind сама по себе еще не решает вопрос, кто обработает исключение. Она только позволяет перейти от одного кадра к предыдущему, виртуально восстановив caller context.

normal execution (нормальное выполнение)
Обычный ход программы, когда функции вызываются через call, возвращаются через ret, а стек разбирается самими инструкциями эпилога.

exceptional execution (исключительное выполнение)
Ход программы после исключения, когда обычный путь через return может быть прерван, а переход к обработчику выполняется системным механизмом исключений и unwind-инфраструктурой.

Поэтому search phase и unwind phase можно понимать как специальный системный маршрут назад по стеку. При нормальном return каждая функция сама убирает свой кадр. При исключении этот процесс берет на себя Windows вместе с language-specific handlers.

Frame-based обработка и почему фазы относятся именно к стеку

Когда мы говорим о search phase и unwind phase, мы говорим прежде всего о frame-based exception handling - обработке исключений, привязанной к кадрам стека. Это не единственный уровень обработки в Windows. Существуют debugger first chance notification, vectored exception handling, unhandled exception filter и другие элементы общего маршрута. Но две фазы поиска и раскрутки особенно важны именно внутри frame-based модели, потому что они идут по цепочке стековых кадров и используют для этого данные .pdata/.xdata.

> frame-based exception handling (обработка исключений по кадрам стека)
> Модель, в которой обработчики связываются не просто с глобальным списком функций, а с конкретными функциями и их кадрами стека. Система идет от текущего кадра к вызывающим кадрам и спрашивает обработчики, связанные с этими кадрами. На x64 эта модель опирается на таблицы функций и unwind-метаданные, а не на linked list в стеке, как классический x86 SEH.

На x64 это особенно важно: в стеке нет простой цепочки SEH-записей, которую можно было бы пройти как список. Windows должна идти по стеку иначе: по текущему RIP найти RUNTIME_FUNCTION, через него добраться до UNWIND_INFO, вызвать RtlVirtualUnwind, получить контекст предыдущего кадра, повторить. Поэтому фазы обработки исключения на x64 неотделимы от всей таблицной модели, которую мы уже разобрали.

Общая картина маршрута исключения

В реальном Windows-маршруте есть много деталей: переход из kernel-mode в user-mode dispatch, уведомление отладчика, vectored handlers, frame-based handlers, second-chance notification и top-level filter. Но внутри этой книги нас сейчас интересует центральная frame-based часть. Ее можно представить как последовательность, где каждый шаг опирается на уже знакомые структуры.

```
exception occurs
-> Windows получает EXCEPTION_RECORD и CONTEXT
-> определяется ControlPc / RIP
-> начинается frame-based search phase
-> RtlLookupFunctionEntry(ControlPc)
-> RtlVirtualUnwind(...)
-> language-specific handler получает DispatcherContext
-> handler отвечает: искать дальше или принять исключение
-> если handler найден
-> начинается unwind phase
-> промежуточные кадры раскручиваются
-> cleanup / termination handlers получают управление
-> выполнение передается к выбранному обработчику
-> если handler не найден
-> exception становится unhandled / second chance / process termination
```

Эта схема грубая, но она полезна. Она показывает, что поиск обработчика и раскрутка стека - это не один магический прыжок. Это дисциплинированный обход стековых кадров, где на каждом уровне Windows должна держать в руках две вещи: описание исключения и описание текущего машинного контекста.

EXCEPTION_RECORD (запись исключения)
Структура, которая описывает само исключительное событие: код исключения, флаги, адрес исключения и дополнительные параметры. Например, для access violation в дополнительных параметрах можно увидеть тип доступа и адрес, к которому пытались обратиться. EXCEPTION_RECORD отвечает на вопрос: "что произошло?"

CONTEXT record (запись контекста процессора)
Структура, которая описывает состояние регистров процессора в момент исключения или в момент виртуальной раскрутки. Она отвечает на вопрос: "где поток был и какие значения регистров у него были?" Для unwind особенно важны RIP, RSP, nonvolatile-регистры и иногда XMM-регистры.

Search phase как неразрушающий проход по кадрам

Search phase начинается с текущего места выполнения. Это может быть инструкция, которая вызвала access violation, или место после RaiseException. Система берет текущий CONTEXT record и смотрит на ControlPc. По этому адресу она пытается понять, внутри какой функции находится поток. Для этого используется RtlLookupFunctionEntry. Если запись найдена, она указывает на RUNTIME_FUNCTION, а та ведет к unwind-метаданным. Потом RtlVirtualUnwind позволяет вычислить, каким был бы caller context, то есть контекст вызывающей функции.

Но на search phase важен не только технический подъем по стеку. Главная цель этой фазы - спросить у обработчиков, связанных с кадрами, готовы ли они обработать исключение. В C/SEH таким обработчиком обычно будет __C_specific_handler, который смотрит на scope table, фильтры __except и блоки __finally. В C++ это может быть другой language-specific handler, который понимает таблицы C++ EH. Windows не обязана знать правила всех языков программирования. Она дает языковому обработчику контекст и спрашивает: "это твой случай?"

language-specific handler (языковой обработчик)
Функция runtime-уровня, которая понимает правила конкретного языка или конкретного типа exception handling. Для C SEH на Windows таким важным обработчиком является __C_specific_handler. Windows знает, что handler есть, но не обязана сама понимать все таблицы __try/__except или C++ EH. Она передает управление обработчику языка, а тот уже интерпретирует свои данные.

Слово "неразрушающий" здесь нужно понимать аккуратно. Search phase обычно не разбирает стек физически: кадры не исчезают, RSP потока не продвигается окончательно, выполнение еще не переезжает в обработчик. Но это не значит, что на search phase вообще нет пользовательского кода. Фильтр __except может быть выполнен именно во время поиска. Если фильтр написан плохо и имеет side effects, эти side effects действительно произойдут. Поэтому правильнее сказать так: search phase не является фазой удаления кадров, но она может вызывать код, который решает, принимать исключение или нет.

__except filter (фильтр __except)
Выражение или код, который решает, будет ли конкретный __except-блок обрабатывать текущее исключение. Фильтр может вернуть значение, которое означает "обработать", "продолжить поиск" или "продолжить выполнение". Это не cleanup-код. Это решение о применимости обработчика.

Ответы обработчика во время поиска

Когда language-specific handler получает управление на search phase, он не просто молча смотрит на структуру. Он должен вернуть системе определенное решение. В терминах Windows это связано с EXCEPTION_DISPOSITION - результатом, который говорит диспетчеру исключений, что делать дальше. Для понимания книги сейчас важнее смысл этих решений, чем все редкие угловые случаи.

EXCEPTION_DISPOSITION (решение обработчика исключения)
Значение, которое handler возвращает системе, чтобы сказать, как продолжать обработку. Смысл: "можно продолжить выполнение", "ищи дальше", "есть вложенное исключение", "произошел collided unwind" и так далее. Для базового понимания самые важные идеи: handler либо не берет исключение и поиск идет дальше, либо он участвует в обработке.

Если обработчик говорит ExceptionContinueSearch, система поднимается к следующему кадру. Если обработчик говорит, что исключение обработано или что нужно продолжать выполнение, маршрут меняется. В обычной ситуации __except-фильтр, который решил обработать исключение, делает текущий кадр target frame - целевым кадром. После этого search phase выполнила свою главную задачу: найден кадр, в котором есть обработчик. Но стек еще не приведен к состоянию этого кадра. Поэтому нужен второй проход.

target frame (целевой кадр)
Кадр стека, в котором найден обработчик, выбранный для текущего исключения. До него нужно раскрутить стек во второй фазе. Все более новые кадры, которые находятся ниже по стеку относительно target frame, должны быть корректно покинуты.

Здесь часто возникает ошибка понимания: если обработчик найден в кадре B, это не значит, что система уже находится в B. В момент окончания search phase поток все еще фактически находится в исходном месте исключения. Search phase нашла цель. Unwind phase должна довести выполнение до этой цели.

Почему нельзя выполнять cleanup во время поиска

Может показаться: раз система все равно идет по кадрам, почему бы сразу не выполнять __finally и destructors по пути? Ответ простой: во время поиска еще неизвестно, будет ли вообще найден обработчик и какой именно кадр станет целевым. Если во время search phase начать разрушать кадры, а потом окажется, что обработчик не найден, программа окажется в еще более странном состоянии: часть кадров уже уничтожена, но некому принять исключение.

Search phase должна быть фазой выбора маршрута. Она как разведка: пройти по стеку, посмотреть, кто готов обработать исключение, но не сжигать мосты. Unwind phase - это

уже движение по выбранному маршруту, где промежуточные кадры действительно покидаются. Такое разделение делает обработку исключений предсказуемой: сначала система узнает цель, затем выполняет обязательные действия на пути к этой цели.

cleanup (очистка / завершающие действия)
Общий смысловой термин для кода, который должен выполниться при выходе из области: освобождение ресурса, закрытие handle, разблокировка mutex, вызов destructors, выполнение __finally. В Windows SEH для __finally часто используется термин termination handling.

termination handler (завершающий обработчик)
Обработчик, который должен быть вызван не для выбора, обрабатывать исключение или нет, а для выполнения завершающей логики при покидании защищенной области. В C SEH это связано с __finally. В unwind-метаданных x64 наличие такого обработчика связано с флагом UNW_FLAG_UHANDLER.

Unwind phase как контролируемое покидание кадров

Когда target frame найден, начинается unwind phase. Теперь задача меняется. Система уже не спрашивает: "кто обработает?" Она знает ответ. Теперь нужно дойти от текущего кадра до target frame. На этом пути могут находиться кадры, которые уже никогда не продолжат нормальное выполнение. Если исключение произошло в D, а обработчик выбран в B, то D и C будут покинуты. Их нормальные ret-инструкции уже не будут выполнены как обычный return. Но логика выхода из этих областей все равно должна быть соблюдена.

Unwind phase использует те же фундаментальные инструменты: RtlLookupFunctionEntry, RtlVirtualUnwind, UNWIND_INFO, DispatcherContext, EstablisherFrame. Но теперь обработчики получают другой смысловой контекст. Для termination handlers важно не "берешь ли ты исключение?", а "тебя нужно вызвать, потому что твой кадр покидается". Именно поэтому EXCEPTION_RECORD содержит специальные признаки unwind-состояния, а обработчики могут понимать, что их вызвали не во время обычного поиска, а во время раскрутки.

EXCEPTION_UNWINDING (флаг раскрутки исключения)
Флаг в EXCEPTION_RECORD, который показывает, что обработчик вызывается во время unwind phase, а не во время обычного поиска обработчика. Это важно для language-specific handler-а: тот же handler может получать управление в разных фазах и должен понимать, зачем его вызвали.

RtlUnwindEx (системная функция раскрутки)
API уровня Windows, связанный с выполнением unwind phase. В широком смысле это механизм, который помогает раскрутить стек до целевого кадра и целевого адреса, используя CONTEXT record, EXCEPTION_RECORD и историю поиска функций. В этой книге он важен как системный исполнитель второй фазы, тогда как RtlVirtualUnwind - низкоуровневый шаг виртуального подъема на один кадр.

Два прохода по одним и тем же кадрам

Важная деталь: search phase и unwind phase могут проходить по похожей цепочке кадров. На search phase система прошла D, C, B и выяснила, что B принимает исключение. На unwind phase она снова идет от D к B, но теперь не для вопроса, а для действия. Это может казаться лишним, но именно такой двойной проход дает системе возможность сначала узнать цель, а потом безопасно выполнить обязательную логику выхода.

```
Call chain before exception:

A()
B() <- target frame: здесь выбран __except
C() <- будет покинут во время unwind phase
D() <- исключение произошло здесь

Search phase:
D: handler? no
C: handler? no
B: handler? yes

Unwind phase:
D: покинуть кадр, выполнить нужные termination handlers
C: покинуть кадр, выполнить нужные termination handlers
B: передать управление выбранному обработчику
```

Эта схема полезна еще и потому, что показывает отличие обработчика исключения от завершающего обработчика. __except в B выбирает, обработать ли исключение. __finally в C не выбирает, принимать исключение или нет; он должен выполниться, потому что C покидается. Если перепутать эти роли, весь механизм SEH станет казаться хаотичным.

Search phase в деталях: что происходит на каждом кадре

Теперь посмотрим на один кадр внимательнее. Допустим, ControlPc находится внутри некоторой функции F. Windows должна понять, есть ли у F метаданные. Сначала вызывается RtlLookupFunctionEntry. Если функция имеет запись в active function table, будет найден RUNTIME_FUNCTION. Из этой записи система получает UnwindInfoAddress и идет к UNWIND_INFO. Там видны флаги: есть ли exception handler, termination handler или chained info.

Если у функции нет обработчика, это не значит, что кадр бесполезен. RtlVirtualUnwind все равно может использовать unwind-коды, чтобы перейти к вызывающему кадру. Если функция является leaf function и не имеет записи RUNTIME_FUNCTION, применяется упрощенная логика: return address берется со стека, потому что такая функция не меняла состояние, требующее сложного восстановления. Но если запись есть и флаги показывают наличие handler-а, система получает адрес language-specific handler-а и вызывает его с набором аргументов, среди которых ExceptionRecord, EstablisherFrame, ContextRecord и DispatcherContext.

handler flag (флаг обработчика)
Бит в UNWIND_INFO, который сообщает, есть ли у функции дополнительный обработчик. UNW_FLAG_EHANDLER означает exception handler, UNW_FLAG_UHANDLER - termination handler, UNW_FLAG_CHAININFO - chained unwind info. Эти флаги определяют, что находится после массива UNWIND_CODE и как система должна интерпретировать хвост UNWIND_INFO.

UNW_FLAG_EHANDLER (флаг exception handler)
Флаг, который показывает, что у функции есть обработчик исключений. После unwind-кодов в UNWIND_INFO будет лежать адрес language-specific handler-а и связанные с ним данные. Для C SEH это обычно ведет к __C_specific_handler и scope table.

UNW_FLAG_UHANDLER (флаг termination handler)
Флаг, который показывает, что у функции есть termination handler, важный при раскрутке. Такой handler особенно существенен во второй фазе, когда кадр действительно покидается.

Почему search phase использует виртуальную раскрутку

Название RtlVirtualUnwind может сбивать с толку. Кажется, что unwind - это уже вторая фаза. Но RtlVirtualUnwind используется и во время поиска. Дело в том, что слово unwind в имени этой функции означает не обязательно физическую фазу уничтожения кадров, а операцию восстановления caller context. Чтобы перейти от кадра D к кадру C во время search phase, системе нужно виртуально понять, каким был контекст C. Она не обязана еще менять реальный RSP потока, но ей нужно вычислить следующий уровень.

virtual unwind (виртуальная раскрутка)
Вычисление контекста вызывающего кадра без обязательного физического выполнения эпилога и без немедленного продолжения программы с этого кадра. Это математическая реконструкция: по текущему CONTEXT и unwind-метаданным понять, какой RIP/RSP/регистры были у caller-а.

Это один из важнейших моментов для reverse engineering. Если анализатор хочет имитировать Windows-логику поиска обработчика, он должен не просто прочитать .pdata и сказать "вот функции". Он должен понимать, что система поднимается по стеку через последовательность виртуальных реконструкций. Ошибка в одном кадре может испортить все последующие кадры: неверный RSP даст неверный return address, неверный return address даст неверный ControlPc, неверный ControlPc приведет к неверной RUNTIME_FUNCTION.

Unwind phase в деталях: что значит покинуть кадр

Покинуть кадр - это не просто прибавить число к RSP. Кадр функции может содержать сохраненные nonvolatile-регистры, локальные переменные, временные области, shadow space, выравнивание, иногда frame pointer. С точки зрения unwind metadata, восстановить caller context означает отменить служебные действия пролога: если пролог сделал push rbx, нужно восстановить RBX; если сделал sub rsp, 40h, нужно вернуть RSP; если сохранил XMM6, нужно знать, где он лежит. Но с точки зрения языка и runtime этого мало: если внутри кадра есть __finally или destructors, их нужно вызвать.

Именно поэтому unwind phase имеет два слоя. Нижний слой - машинный: восстановление регистров и RSP по UNWIND_INFO. Верхний слой - языковой: вызов termination handlers, C++ destructors и другой cleanup-логики. Windows предоставляет инфраструктуру прохода по кадрам и передачи контекста, а конкретный language-specific handler решает, какие таблицы языка нужно применить.

machine-level unwind (машинная раскрутка)
Уровень, на котором восстанавливаются регистры и стековый указатель. Здесь важны RSP, RIP, сохраненные nonvolatile-регистры, UNWIND_CODE и правила x64 ABI.

language-level unwind (языковая раскрутка)
Уровень, на котором выполняются правила конкретного языка: __finally, C++ destructors, обработка scope table, состояние try/catch. Этот уровень опирается на машинную раскрутку, но не сводится к ней.

Почему __finally относится к unwind phase

Блок __finally часто путают с __except. __except выбирает, обработать ли исключение. __finally не выбирает. Его смысл: "когда поток покидает эту защищенную область, выполни завершающий код". Причина выхода может быть разной: нормальный выход, return, break, goto, exception unwind. В контексте исключений __finally особенно важен именно во время unwind phase, потому что кадры покидаются не обычным ret, а системной раскруткой.

Если функция C содержит __try/__finally и исключение произошло ниже, а обработчик найден выше, то C может быть уничтожена как промежуточный кадр. Но ее __finally должен получить шанс закрыть ресурс. Например, если C взяла lock, открыла file handle или временно изменила состояние, пропуск __finally приведет к утечке ресурса или deadlock. Поэтому unwind phase не является "механическим прыжком". Это дисциплинированное покидание областей с соблюдением правил cleanup.

__finally (завершающий блок SEH)
Блок C/SEH, который выполняется при выходе из защищенной области. Он не решает, обрабатывать исключение или нет. Его задача - выполнить завершающие действия. В контексте x64 unwind он важен как пример termination handling, вызываемого при раскрутке.

First chance и second chance - это не то же самое, что search и unwind

Еще одна типичная путаница: first chance exception и second chance exception. Эти термины относятся к взаимодействию с отладчиком и общему маршруту exception dispatching, а не напрямую к двум фазам frame-based unwind. Когда исключение возникает в отлаживаемом процессе, отладчик может получить first chance notification - первое уведомление. Если отладчик не обработал исключение, система продолжает обычный поиск обработчиков. Если обработчик не найден, отладчик может получить second chance notification. Если и тогда исключение не обработано, процесс обычно завершается.

first chance exception (исключение первого шанса)
Ситуация, когда отладчик получает первое уведомление об исключении до того, как система окончательно решила, что приложение не может его обработать. Это не означает, что программа уже падает. Многие исключения первого шанса нормально перехватываются программой.

second chance exception (исключение второго шанса)
Ситуация, когда обычные механизмы обработки не нашли обработчик, и отладчик получает последнюю возможность вмешаться. Если и после этого исключение не обработано, оно становится unhandled для процесса.

Search phase и unwind phase живут внутри frame-based механики. First/second chance - это более широкий слой dispatching вокруг нее. Для reverse engineer-а это принципиально: сообщение отладчика о first chance access violation еще не значит, что не будет найден __except. A second chance обычно означает, что frame-based поиск не дал подходящего обработчика или обработка сорвалась.

VEH и почему его нельзя смешивать с frame-based фазами

Vectored exception handling тоже часто смешивают с search phase. VEH-обработчики регистрируются глобально для процесса и вызываются не как кадры стека, а как список callback-функций. Они могут увидеть исключение раньше frame-based обработчиков. Но VEH не является обычным проходом по стековым кадрам, не опирается на RUNTIME_FUNCTION каждого кадра как frame-based SEH и не имеет того же смысла target frame.

VEH / vectored exception handling (векторная обработка исключений)
Механизм Windows, при котором обработчики регистрируются в глобальном списке процесса и вызываются при исключениях независимо от конкретного стекового кадра. VEH полезен для отладчиков, защит, instrumentation и низкоуровневых техник, но его нельзя считать тем же самым, что frame-based search/unwind по кадрам.

В этой главе мы не игнорируем VEH, но держим границу: search phase и unwind phase в текущем смысле - это стековая, frame-based логика. VEH может повлиять на общий маршрут исключения, но он не заменяет понимание .pdata/.xdata и RtlVirtualUnwind.

Связь с __C_specific_handler

__C_specific_handler - хороший пример того, почему Windows разделяет общую инфраструктуру и языковую логику. Windows умеет идти по кадрам, находить RUNTIME_FUNCTION, получать UNWIND_INFO, вычислять EstablisherFrame и собирать DispatcherContext. Но Windows не обязана сама понимать каждую C scope table как часть ядра механизма. Вместо этого она вызывает __C_specific_handler и передает ему достаточно данных, чтобы он мог принять решение.

__C_specific_handler (специальный C-обработчик SEH)
Language-specific handler, который используется для C Structured Exception Handling на x64. Он интерпретирует данные, связанные с __try, __except и __finally, и помогает системе понять, есть ли подходящий обработчик или какие завершающие действия нужно выполнить при unwind.

Во время search phase __C_specific_handler может смотреть на фильтры __except и решать, подходит ли текущий scope. Во время unwind phase он может участвовать в вызове termination handlers, связанных с __finally. Один и тот же механизм получает разные смысловые роли в зависимости от флагов исключения и контекста вызова. Именно поэтому в предыдущем разделе был важен DispatcherContext: он несет handler-у не просто адреса, а весь пакет сведений о текущем кадре и фазе обработки.

TargetIp и TargetFrame как ориентиры второй фазы

Когда начинается unwind phase, системе нужен ориентир: куда именно раскручивать стек и где продолжить выполнение. В разных API и внутренних структурах появляются понятия TargetFrame и TargetIp. Их нельзя понимать как обычные поля из PE-файла. Это runtime-ориентиры конкретного процесса и конкретного исключения.

TargetIp (целевой instruction pointer)
Адрес, к которому должна прийти раскрутка или куда должно быть передано управление после завершения нужных unwind-действий. В DispatcherContext поле TargetIp помогает handler-ам понимать цель unwind-маршрута.

TargetFrame (целевой стековый кадр)
Кадр, до которого выполняется раскрутка. Все более новые кадры по пути должны быть покинуты. В пользовательской документации чаще встречается EstablisherFrame и параметры RtlUnwindEx, но смысл target frame важен для понимания второй фазы.

Здесь важна разница между статическими данными и runtime-состоянием. .pdata и .xdata говорят, как устроены функции вообще. TargetFrame и TargetIp говорят, куда именно нужно прийти в этой конкретной аварийной ситуации. Reverse engineer, который видит только файл на диске, может реконструировать возможные маршруты, но реальный target frame возникает только во время выполнения, когда есть конкретный стек и конкретное исключение.

Collided unwind и почему реальность сложнее идеальной схемы

Идеальная картина выглядит линейно: поиск, нашли обработчик, раскрутили стек, пришли в обработчик. Но обработчики - это код. А код может сам вызвать исключение, может инициировать новый unwind, может вмешаться в процесс. Поэтому в Windows существуют более сложные состояния, например collided unwind. Для базового понимания не нужно сразу разбирать все внутренности collided unwind, но важно знать, зачем такой термин существует.

collided unwind (столкнувшаяся раскрутка)
Сложная ситуация, когда во время уже идущей раскрутки возникает другой unwind-процесс или обработчик меняет ожидаемый ход раскрутки. Это редкий, но важный механизм для корректной работы вложенных исключений и сложных языковых runtimes.

Для нашей книги collided unwind важен как предупреждение: unwind - это не просто арифметика RSP. Это управляемый протокол между системой, runtime-библиотеками и обработчиками. Если анализатор делает вид, что существует только линейный happy path, он будет неправильно объяснять сложные аварии и нестандартные бинарники.

Пример: RaiseException

RaiseException полезен как учебный пример, потому что он создает программное исключение. В отличие от access violation, здесь исключение не обязательно связано с ошибкой обращения к памяти. Код сам говорит Windows: "сгенерируй исключение с таким кодом и такими параметрами". Но дальше frame-based логика похожа: формируется EXCEPTION_RECORD, появляется CONTEXT, начинается dispatching, затем search phase ищет подходящий обработчик.

RaiseException (вызов программного исключения)
WinAPI-функция, которая позволяет коду явно сгенерировать исключение. Это не аппаратная ошибка процессора, а программный способ войти в механизм исключений Windows. Для unwind-модели важно, что после генерации события все равно используются системные фазы поиска и, при необходимости, раскрутки.

Если ближайший __except-фильтр принимает это исключение, search phase завершится выбором target frame. Затем unwind phase покинет промежуточные кадры. Если фильтры не принимают исключение, поиск пойдет выше. Если никто не примет, исключение станет unhandled. Поэтому RaiseException показывает, что search/unwind - это не только про "краши". Это общий механизм управления исключительными переходами.

Пример: access violation

Access violation кажется более драматичным случаем, потому что его часто связывают с падением программы. Но для Windows это тоже исключение, описанное EXCEPTION_RECORD. В record будет код нарушения доступа и дополнительные параметры: например, была ли это попытка чтения, записи или исполнения, и какой адрес участвовал. Дальше система так же должна решить: есть ли обработчик?

access violation (нарушение доступа к памяти)
Исключение, возникающее при недопустимом обращении к памяти: чтении, записи или выполнении по адресу, к которому у процесса нет подходящих прав или который не отображен. В Windows оно входит в общий механизм exception dispatching и может быть перехвачено SEH-обработчиком.

На search phase фильтр __except может посмотреть код исключения через GetExceptionCode или данные через GetExceptionInformation и решить, что делать. Например, некоторые системы используют access violation для guard pages, lazy allocation,

instrumentation или sandbox-механик. Если обработчик принимает исключение, unwind phase должна корректно довести выполнение до него. Если нет, процесс получит second chance / unhandled exception.

Почему для x64 все это завязано на .pdata/.xdata

На x86 классический SEH часто объясняют через цепочку записей в стеке. На x64 такой модели уже недостаточно и в нормальной Windows x64 она не является центральной. Здесь система не может просто пройти linked list обработчиков, лежащих в стеке. Она должна понимать функции как диапазоны кода и знать, как из каждого кадра подняться к предыдущему. Именно поэтому .pdata и .xdata являются не вспомогательной информацией, а несущей конструкцией всей модели.

Во время search phase таблицы функций позволяют перейти от ControlPc к FunctionEntry. Во время unwind phase те же метаданные позволяют восстанавливать контекст и вызывать handlers в правильных кадрах. Если .pdata повреждена, отсортирована неправильно, указывает за пределы image или ведет к неверной .xdata, система может не найти обработчик, неправильно пройти стек или упасть при анализе исключения. Для обычного пользователя это просто crash. Для reverse engineer-а это сигнал: unwind metadata может быть либо ошибочной, либо намеренно искаженной.

> Вывод для x64
> На Windows x64 search/unwind phases нельзя понять без таблиц. Исключения, stack trace, crash dump analysis и работа отладчика опираются на одну и ту же основу: активные function tables и unwind metadata.

Что видит debugger во время этих фаз

Отладчик может показывать исключение раньше, чем пользовательский обработчик его примет. Поэтому начинающий reverse engineer иногда пугается: WinDbg или Visual Studio остановились на access violation, значит программа сломалась. Но это может быть first chance exception, который затем будет обработан. Нужно смотреть, продолжится ли выполнение, есть ли handler, какая цепочка вызовов, и что произойдет после команды continue.

> debugger (отладчик)
> Инструмент, который подключается к процессу и получает уведомления о событиях выполнения: исключениях, загрузке модулей, создании потоков, breakpoint-ах. В контексте исключений debugger может видеть first chance и second chance события, но это не то же самое, что внутреннее решение frame-based handler-а.

Во время анализа crash dump отладчик уже не управляет живым процессом, а смотрит на снимок. Там особенно важна способность восстановить стек. Если unwind metadata повреждена, stack trace может оборваться или показать мусор. Поэтому главы про RtlVirtualUnwind, EstablisherFrame, DispatcherContext и две фазы обработки исключения напрямую связаны с dump analysis. Это не академическая теория, а объяснение того, почему отладчик иногда показывает нормальный стек, а иногда - кашу.

Типичная ошибка: считать unwind phase просто stack walking

Stack walking и unwind phase родственны, но это не одно и то же. Stack walking - более общий термин: пройти по стеку и построить цепочку вызовов. Это может делать отладчик, профилировщик, crash dump analyzer или runtime. Он может не вызывать cleanup-код и не менять выполнение программы. Unwind phase при исключении - это активное покидание кадров, связанное с обработкой исключения. Она может вызывать handlers и менять дальнейший control flow.

> stack walking (обход стека)
> Процесс построения цепочки вызовов: текущая функция, ее caller, caller caller-а и так далее. Может выполняться для отладки, профилирования, логирования или crash dump analysis. Stack walking может использовать unwind metadata, но не обязан быть фазой обработки исключения.

> control flow (поток управления)
> Последовательность адресов и переходов, по которым реально движется выполнение программы. Исключение меняет control flow: вместо обычного return может произойти переход к handler-у после системной раскрутки.

В этой разнице много практического смысла. static может уметь читать .pdata/.xdata и строить stack trace, но этого еще мало, чтобы корректно моделировать exception dispatch. Для exception dispatch нужно понимать, какие handlers существуют, какие флаги выставлены, как работает search phase, где начинается unwind phase и какие кадры являются target/protected/cleanup.

Что должен проверять static /dynamic

Для твоего static /dynamic этот раздел задает не только теорию, но и будущие проверки. Если инструмент анализирует PE и unwind metadata, он должен видеть не отдельные структуры, а их роль в маршруте исключения. Каждая странность должна оцениваться через вопрос: "что произойдет во время search phase или unwind phase?"

Например, запись RUNTIME_FUNCTION может указывать на UNWIND_INFO с EHANDLER, но handler RVA может вести в странную область. Для обычного parser-а это просто поле. Для static это finding: во время search phase система может попытаться вызвать language-specific handler из подозрительного места. Другой пример: у функции есть UHANDLER, но unwind-коды плохо согласуются с прологом. Это влияет на unwind phase: при покидании кадра может быть неверно восстановлен RSP или вызван cleanup с неправильным EstablisherFrame.

> finding (находка анализа / диагностический вывод)
> Формализованное наблюдение инструмента: не просто "поле равно X", а смысловая запись о том, почему это поле важно, какие правила нарушены и к чему это может привести при загрузке, unwind, exception dispatch или reverse engineering.

verdict (вердикт анализа)
Итоговая оценка находки или группы находок: нормально, подозрительно, повреждено, нестандартно, похоже на obfuscation, возможно manual mapping, возможно compiler-specific pattern. В хорошей системе verdict должен опираться на evidence, а не на догадку.

Минимальная модель для ручного анализа

Когда ты вручную смотришь бинарник в Ghidra, IDA или WinDbg, полезно держать в голове короткую модель. Исключение произошло не "в функции вообще", а в конкретном ControlPc. По ControlPc система должна найти FunctionEntry. По FunctionEntry - UNWIND_INFO. По UNWIND_INFO - понять, есть ли handler и как подняться к caller-у. Search phase повторяет это, пока не найдет target frame. Unwind phase повторяет похожий проход, но уже выполняет выход из кадров.

```
Manual mental model:

1. Есть ExceptionRecord: что случилось.
2. Есть ContextRecord: где поток был.
3. ControlPc -> RtlLookupFunctionEntry -> RUNTIME_FUNCTION.
4. RUNTIME_FUNCTION -> UNWIND_INFO.
5. UNWIND_INFO говорит:
- как восстановить caller context;
- есть ли EHANDLER / UHANDLER;
- есть ли chained info.
6. Search phase спрашивает handlers.
7. После выбора target frame unwind phase покидает промежуточные frames.
8. Handler получает управление уже не как случайный jump, а как результат протокола.
```

Эта модель не заменяет документацию и не покрывает все угловые случаи. Но она дает правильную ось. Если держать ее в голове, DispatcherContext перестает быть просто структурой, EstablisherFrame перестает быть загадочным указателем, а UNWIND_INFO перестает быть набором байтов. Все они становятся участниками одного маршрута.

Как это связано с будущими разделами

В следующих разделах мы будем разбирать language-specific handler, __C_specific_handler, реальный смысл SEH на x64, VEH и top-level handling. Без текущей темы эти разделы будут казаться списком API и структур. С текущей темой они складываются в архитектуру: Windows обеспечивает общий проход по кадрам, а языковые обработчики наполняют этот проход смыслом конкретного языка.

Search phase и unwind phase - это граница между вопросом и действием. Пока идет search phase, система выясняет, кто берет ответственность. Когда начинается unwind phase, ответственность уже выбрана, и система начинает менять судьбу потока: покидать кадры, выполнять завершающий код и переносить управление к обработчику. Это один из самых важных концептов во всей теме Windows x64 SEH/unwind.

Dispatch, search и unwind: три слова, которые нельзя смешивать

В документации и отладчиках рядом часто встречаются слова dispatch, search и unwind. Они звучат похоже, потому что все относятся к обработке исключений, но обозначают разные уровни. Если их смешать, то весь механизм станет мутным: будет казаться, что Windows то ищет, то раскручивает, то вызывает обработчики без ясного порядка. Поэтому закрепим различие прямо здесь.

> exception dispatching (диспетчеризация исключения)
> Общий процесс доставки исключения от места возникновения к возможным обработчикам. Dispatching включает более широкий маршрут: уведомление debugger-а, vectored handlers, frame-based поиск, возможную раскрутку, second chance и unhandled handling. Search phase и unwind phase находятся внутри этого большого маршрута, но не равны всему dispatching.

> dispatch (диспетчеризация / доставка события)
> Общее слово для передачи события тому коду, который должен на него отреагировать. В контексте исключений dispatch означает: система получила исключение и начинает маршрутизировать его через установленные механизмы обработки.

Search phase - это не весь dispatch. Это тот участок dispatching, где система проходит по frame-based обработчикам и ищет, кто согласится принять исключение. Unwind phase - тоже не весь dispatch. Это участок после выбора цели, где система физически и логически покидает промежуточные кадры. Поэтому правильная лестница понятий такая: exception dispatching шире, внутри него может быть frame-based search, а после успешного поиска - unwind.

Что именно передается обработчику во время фаз

Обработчик не вызывается в вакууме. Ему передается набор данных, который позволяет понять не только "что произошло", но и "в каком кадре мы сейчас рассуждаем", "какой код связан с этим кадром", "какие runtime-данные лежат после UNWIND_INFO" и "в какой фазе обработки мы находимся". Именно поэтому предыдущие разделы про EstablisherFrame и DispatcherContext были не второстепенными. Без них search/unwind phases выглядели бы как абстрактные слова.

Во время search phase handler получает ExceptionRecord, EstablisherFrame, ContextRecord и DispatcherContext. ExceptionRecord говорит, что за исключение возникло. ContextRecord показывает состояние потока, причем это может быть уже виртуально поднятый контекст определенного кадра. EstablisherFrame связывает handler с конкретным стековым кадром. DispatcherContext дает технический пакет: ControlPc, ImageBase, FunctionEntry, LanguageHandler, HandlerData, TargetIp и другие поля.

Во время unwind phase часть этих данных остается похожей, но смысл меняется. Теперь handler смотрит не только на "подходит ли исключение", а на то, нужно ли выполнить завершение области, потому что кадр покидается. Поэтому один и тот же набор структур может обслуживать разные смысловые режимы. Это похоже на один и тот же паспорт, который предъявляют на разных этапах: при поиске - чтобы понять, кто имеет право принять событие; при раскрутке - чтобы понять, кого надо уведомить о выходе из области.

Exception flags: как обработчик понимает, что сейчас идет unwind

EXCEPTION_RECORD содержит не только код исключения. В нем есть поле флагов. Для обычного прикладного кода эти флаги часто незаметны, но для обработчиков и runtime-библиотек они важны: по ним можно понять, является ли текущий вызов handler-а обычным поиском, раскруткой, выходной раскруткой или другой особой ситуацией.

ExceptionFlags (флаги исключения)
Поле в EXCEPTION_RECORD, которое уточняет состояние исключения. Флаги помогают отличать обычную обработку от unwind-состояний, noncontinuable exception и других специальных ситуаций. Это не код исключения, а дополнительные признаки режима обработки.

EXCEPTION_NONCONTINUABLE (нельзя продолжить выполнение)
Флаг, который означает, что после такого исключения нельзя просто вернуть управление на место возникновения и продолжить выполнение. Если handler попытается продолжить выполнение после noncontinuable exception, это само по себе является ошибкой.

EXCEPTION_EXIT_UNWIND (выходная раскрутка)
Флаг, связанный с особым видом раскрутки, когда поток покидает область не в обычной логике поиска обработчика, а в контексте выхода. Для фундаментального понимания важно не заучивать флаг отдельно, а видеть: обработчики должны различать разные режимы unwind, а не считать каждый вызов одинаковым.

Эти флаги нужны потому, что handler может быть вызван несколько раз в разных ролях. Один вызов может спрашивать: "ты обработаешь?" Другой вызов может означать: "твой кадр покидается, выполни завершение". Без флагов handler не смог бы надежно отличить эти ситуации.

Возвраты __except filter: три решения на уровне C

В C SEH фильтр __except обычно возвращает одно из трех смысловых значений. Они выглядят простыми, но за каждым стоит изменение маршрута всей системы. Эти значения не являются тем же самым, что EXCEPTION_DISPOSITION, но для пользователя C они выражают похожую идею: что делать с текущим исключением.

EXCEPTION_EXECUTE_HANDLER (выполнить обработчик)
Результат фильтра __except, который означает: этот __except-блок принимает исключение, search phase может считать текущий кадр целевым, дальше нужно выполнить unwind phase до этого кадра и передать управление телу обработчика.

EXCEPTION_CONTINUE_SEARCH (продолжить поиск)
Результат фильтра __except, который означает: этот блок не берет исключение. Система должна продолжить search phase и подняться к следующему frame-based обработчику.

EXCEPTION_CONTINUE_EXECUTION (продолжить выполнение)
Результат фильтра, который означает попытку продолжить выполнение с места исключения. Это допустимо не для всех исключений и не во всех состояниях. Если контекст не исправлен, повторится та же ошибка. Если исключение noncontinuable, продолжать нельзя.

EXCEPTION_CONTINUE_EXECUTION особенно важен для понимания низкоуровневых техник. Некоторые системы специально ловят исключение, исправляют контекст или память, а затем продолжают выполнение. Например, guard page может вызвать исключение, handler подготовит страницу, после чего инструкция повторится. Но если просто вернуть CONTINUE_EXECUTION при обычном access violation без исправления причины, программа снова упадет на той же инструкции.

Почему фильтр может быть опаснее, чем кажется

__except filter выглядит как условие: если код исключения такой-то, обработать. Но фильтр - это исполняемый код. Он может читать память, вызывать функции, менять глобальные переменные, логировать, даже сам спровоцировать новое исключение. Поэтому search phase нельзя считать полностью пассивным чтением таблиц. Таблицы приводят систему к handler-у, но handler может выполнить реальный код фильтра.

Это создает важный практический вывод. В reverse engineering нельзя смотреть только на наличие __except и думать, что это статическое решение. Нужно смотреть на filter expression или скомпилированный код фильтра: что он проверяет, какие exception codes принимает, вызывает ли GetExceptionInformation, меняет ли ContextRecord, зависит ли от глобального состояния. Иногда именно фильтр является центральной частью защиты или обфускации.

filter expression (выражение фильтра)
Код или выражение после __except(...), которое вычисляется во время search phase и решает, будет ли обработчик выбран. Оно может быть простым сравнением кода исключения, а может быть сложной функцией с побочными эффектами.

side effect (побочный эффект)
Изменение состояния программы при вычислении выражения или вызове функции: запись в память, изменение глобальной переменной, вывод, блокировка, изменение ContextRecord. Для фильтров SEH это важно, потому что фильтр выполняется до окончательной раскрутки стека.

Protected range и scope: где именно действует обработчик

Обработчик не обязательно относится ко всей функции целиком. В исходном коде __try/__except или __try/__finally покрывает определенную область. Компилятор должен как-то передать runtime-у, какие диапазоны кода защищены каким обработчиком. Для этого используются language-specific данные, например scope table для __C_specific_handler.

protected range (защищенный диапазон)
Диапазон инструкций, для которого действует конкретная защищенная область __try или аналогичная языковая конструкция. Если исключение произошло вне этого диапазона, связанный обработчик не должен считаться подходящим только потому, что функция вообще имеет EHANDLER.

scope (область действия / защищенная область)
Логическая область программы, к которой применяются определенные правила обработки: __try, __except, __finally, catch, destructor cleanup. В бинарнике scope обычно превращается в таблицу диапазонов и обработчиков, которую понимает language-specific handler.

scope table (таблица областей)
Языковая таблица, которую использует handler вроде __C_specific_handler. Она связывает диапазоны кода с фильтрами, обработчиками и finally-блоками. Это не универсальная .pdata-таблица Windows, а данные конкретного runtime/компилятора.

Это объясняет, почему наличие UNW_FLAG_EHANDLER у функции еще не значит: любое исключение внутри любого байта функции будет обработано одним и тем же __except. EHANDLER говорит, что у функции есть language-specific handler и дополнительные данные. А уже handler по своим таблицам решает, попадает ли ControlPc в protected range и какой scope применим.

Search phase как серия вопросов разного уровня

Можно представить search phase как серию вложенных вопросов. Сначала системный вопрос: есть ли вообще запись функции для ControlPc? Потом unwind-вопрос: можем ли мы корректно подняться к caller-у? Потом handler-вопрос: есть ли у этой функции language-specific handler? Потом языковой вопрос: попадает ли ControlPc в защищенный диапазон? Потом пользовательский вопрос: что вернул filter expression? Только если вся цепочка сошлась, кадр становится кандидатом или target frame.

```
Frame search logic, упрощенно:

1. ControlPc принадлежит функции?
2. Есть RUNTIME_FUNCTION?
3. Есть UNWIND_INFO?
4. Есть EHANDLER / UHANDLER?
5. HandlerData указывает на корректные language-specific данные?
6. ControlPc попадает в нужный protected range?
7. Filter говорит EXECUTE_HANDLER или CONTINUE_SEARCH?
8. Если EXECUTE_HANDLER - найден target frame.
9. Если CONTINUE_SEARCH - перейти к caller context.
```

Эта многоступенчатость важна для построения анализатора. Плохой анализатор говорит: "в функции есть EHANDLER, значит функция обрабатывает исключения". Хороший анализатор говорит: "в функции есть EHANDLER, нужно проверить language-specific handler, HandlerData, scope ranges, фильтры, target ranges и согласованность с текущим ControlPc".

Что происходит, если обработчик не найден

Не каждое исключение получает обработчик. Если search phase дошла до верхних кадров и никто не принял исключение, оно становится unhandled с точки зрения приложения. Но даже здесь общий Windows-маршрут еще может включать debugger second chance и top-level handling. Для пользователя это часто выглядит как "программа упала", но внутри это результат неудачного поиска обработчика.

> unhandled exception (необработанное исключение)
> Исключение, для которого приложение не нашло подходящего обработчика в своем маршруте обработки. Обычно это приводит к уведомлению отладчика, вызову top-level/unhandled filter или завершению процесса.

> top-level exception filter (верхнеуровневый фильтр необработанных исключений)
> Механизм, который может быть вызван, когда исключение не обработано обычными frame-based обработчиками. Это уже не то же самое, что локальный __except внутри функции. Подробно он будет разбираться позже.

Для crash dump analysis это означает: если ты видишь unhandled access violation, полезно спросить не только "где упало", но и "почему search phase не нашла обработчик". Возможно, обработчиков действительно не было. Возможно, фильтр отказался. Возможно, .pdata/.xdata повреждены. Возможно, stack corruption помешал пройти к нужному кадру.

Stack corruption: когда фазы ломаются не из-за handler-а

Иногда механизм исключений выглядит сломанным не потому, что в PE нет нужных метаданных, а потому что сам стек поврежден. Например, перезаписан return address, испорчен RSP, повреждены сохраненные nonvolatile-регистры, разрушена область, где лежал saved register. RtlVirtualUnwind может честно применить UNWIND_INFO, но если память стека уже содержит мусор, восстановленный caller context тоже будет мусором.

> stack corruption (повреждение стека)
> Состояние, при котором данные в стеке больше не соответствуют ожидаемому layout-у вызовов функций: испорчен return address, saved registers, локальные области или выравнивание. Для unwind это опасно, потому что система восстанавливает caller context, читая значения из стека.

Поэтому static /dynamic должен отличать два класса проблем. Первый класс - статические аномалии: странные .pdata, .xdata, handler RVA, нарушенные диапазоны. Второй класс - runtime-аномалии: стек в конкретном процессе или dump-е поврежден. Статический PE-анализ не всегда может доказать второе, но он может предупредить, что метаданные не дадут надежной раскрутки при ошибке.

Почему packer и obfuscator интересуются этими фазами

Packer, dynamic или obfuscator может использовать исключения как часть управления программой. Исключение может быть не аварией, а специально созданным переходом. Handler может расшифровывать код, менять контекст, перескакивать через блоки, скрывать настоящую цепочку control flow. В такой программе search phase и unwind phase становятся не только системной безопасностью, но и частью логики защиты.

> packer (упаковщик)
> Инструмент или механизм, который преобразует исполняемый файл: сжимает, шифрует, скрывает код, добавляет loader-стадию. Packer может использовать исключения и нестандартные unwind-данные, чтобы усложнить статический анализ.

> obfuscator (обфускатор)
> Инструмент или техника, которая намеренно делает код труднее для понимания, сохраняя его работоспособность. В контексте SEH/unwind обфускация может использовать исключения как скрытые переходы управления или создавать странные metadata patterns.

Если защитник подменяет обычные jumps на RaiseException + handler, дизассемблер может показывать странный разорванный control flow. Если handler меняет ContextRecord и возвращает CONTINUE_EXECUTION, выполнение может продолжиться не так, как ожидал статический анализатор. Поэтому глубокое понимание двух фаз полезно не только для crash analysis, но и для reverse engineering защищенных бинарников.

Как писать findings по двум фазам

Хороший finding не должен звучать как "UNW_FLAG_EHANDLER найден". Это слишком бедно. Нужно объяснять, что это значит для search phase. Например: "Функция содержит EHANDLER; во время frame-based search phase Windows может вызвать language-specific handler для этого кадра. Handler RVA указывает в секцию, которая не выглядит исполняемой; это снижает доверие к metadata и может указывать на повреждение или ручную сборку".

То же самое для UHANDLER. Плохой finding: "есть UHANDLER". Хороший finding: "Кадр содержит termination handler; во время unwind phase при покидании этого кадра должен быть вызван cleanup-код. Если unwind-коды не соответствуют прологу, EstablisherFrame может быть вычислен неверно, и cleanup получит неправильную базу кадра".

```
Пример формулировки finding:

Finding: Suspicious EHANDLER target
Evidence:
- RUNTIME_FUNCTION range: .text+0x1240..0x12A8
- UNWIND_INFO flags: UNW_FLAG_EHANDLER
- Handler RVA: 0x0005F000
- RVA points outside executable sections
Impact:
- During search phase the OS/runtime may attempt to dispatch
to a handler address that does not match normal compiler output.
Verdict:
- suspicious metadata / possible obfuscation or malformed image
```

Такие формулировки связывают байты с поведением. Именно это отличает фундаментальный reverse engineering от простого парсинга структур.

Практическая интуиция: две фазы как суд и эвакуация

Иногда полезна бытовая аналогия, но ее нужно держать под контролем. Search phase похожа на суд или диспетчерский опрос: кто имеет право взять дело? У каждого кадра спрашивают, относится ли исключение к его области ответственности. Пока решение не принято, нельзя уничтожать документы, закрывать кабинет и отправлять людей домой.

Unwind phase похожа на эвакуацию по утвержденному маршруту. Цель уже известна. Теперь нужно безопасно пройти через помещения, закрыть двери, выключить оборудование, забрать важные данные и не оставить систему в опасном состоянии. В машинном мире вместо дверей - стековые кадры, вместо оборудования - ресурсы и сохраненные регистры, вместо маршрута - цепочка contexts и handlers.

Аналогия не заменяет техническую модель, но помогает запомнить главное: search phase принимает решение, unwind phase исполняет последствия этого решения.

Continuable и noncontinuable: можно ли вернуться туда, где упали

Не каждое исключение означает, что выполнение обязано уйти в handler и покинуть кадры. Иногда обработчик может исправить причину и продолжить выполнение почти с того же места. Но иногда продолжение запрещено. Отсюда появляется различие continuable и noncontinuable. Оно важно именно на границе search phase: handler может захотеть сказать "продолжай выполнение", но система должна понимать, допустимо ли это.

continuable exception (продолжаемое исключение)
Исключение, после которого теоретически можно продолжить выполнение, если обработчик исправил причину и вернул подходящее решение. Например, handler может подготовить память, изменить CONTEXT или обработать специальный сигнал.

noncontinuable exception (непродолжаемое исключение)
Исключение, после которого нельзя корректно вернуться к прежнему месту выполнения. Попытка продолжить выполнение после такого исключения считается ошибкой и может вызвать новое исключение.

Для reverse engineering это важно потому, что некоторые защитные схемы строят управление не через EXECUTE_HANDLER, а через CONTINUE_EXECUTION. Handler меняет ContextRecord: например, сдвигает RIP на другой адрес, меняет регистр, подготавливает страницу памяти, а затем говорит системе продолжить. Снаружи это может выглядеть как странный crash-first control flow: код как будто падает, но на деле exception является частью нормального маршрута.

Context modification (изменение контекста)
Изменение полей CONTEXT record обработчиком. Если handler меняет RIP, RSP или регистры и затем просит продолжить выполнение, он фактически меняет будущий control flow потока. Это мощный низкоуровневый прием, но он требует точного понимания состояния процессора.

Почему CONTEXT record во время поиска не всегда означает исходный момент падения

Еще одна тонкость: ContextRecord, который видит handler при обходе кадров, не всегда нужно воспринимать как ровно тот самый context из точки падения. На первом кадре он близок к исходному состоянию исключения. Но когда система виртуально поднимается выше по стеку, ContextRecord может описывать уже восстановленный контекст caller-а. Поэтому важно спрашивать: этот ContextRecord относится к какому кадру? К месту фактического исключения или к кадру, который сейчас рассматривается во время виртуального обхода?

Это особенно важно при чтении отладочных трасс и при проектировании анализатора. Если ты видишь ContextRecord в параметрах handler-а, нельзя автоматически думать: "это просто регистры в момент crash". В процессе поиска и раскрутки контекст является рабочей моделью текущего шага. Он может быть исходным, виртуально восстановленным или подготовленным для передачи управления.

> working context (рабочий контекст раскрутки)
> Состояние CONTEXT record, которое система использует как текущую модель кадра при обходе стека. Оно может изменяться от кадра к кадру, потому что RtlVirtualUnwind постепенно превращает callee context в caller context.

C++ destructors и RAII: языковой смысл второй фазы

Хотя в этой книге мы часто говорим про C SEH, на практике unwind phase важна и для C++. В C++ локальные объекты с destructors должны быть уничтожены, когда область покидается из-за исключения. Это часть языковой модели. Windows предоставляет низкоуровневую инфраструктуру раскрутки, но C++ runtime добавляет свои таблицы и правила: какие объекты живы, какие destructors нужно вызвать, в каком порядке.

> destructor (деструктор)
> Метод C++-объекта, который вызывается при уничтожении объекта. При исключении destructors локальных объектов должны быть вызваны во время stack unwinding, если объект был уже создан и его область покидается.

> RAII / Resource Acquisition Is Initialization (захват ресурса при инициализации)
> C++-идея: ресурс привязывается к объекту, а освобождение ресурса выполняется в destructor-е. Поэтому корректная unwind phase критична: если объект покидает область из-за исключения, destructor должен освободить ресурс.

Для reverse engineer-а это означает: за одним и тем же словом unwind могут скрываться разные уровни. В SEH ты видишь __finally и __except. В C++ ты видишь destructors, catch-блоки и type matching. Но низкоуровневый фундамент похож: нужно пройти по кадрам, понять их состояние, вызвать нужный language-specific runtime и безопасно перенести управление.

Почему порядок cleanup важен

Во время unwind phase промежуточные кадры покидаются в строгом порядке: от самого младшего, ближе к месту исключения, к более старым, ближе к target frame. Это не случайность. Ресурсы обычно захватываются в порядке входа в функции и области, а освобождаться должны в обратном порядке. Если D вызвала C, C захватила ресурс, потом D захватила свой ресурс, то при выходе из-за исключения сначала должен завершиться D-уровень, потом C-уровень.

> LIFO / Last In, First Out (последним вошел - первым вышел)
> Принцип стека: последний добавленный элемент снимается первым. Stack frames, сохраненные return addresses и cleanup-логика естественно следуют этому принципу. Unwind phase тоже идет от более новых кадров к более старым.

Если порядок нарушить, можно освободить ресурс, от которого еще зависит более младший cleanup. Или наоборот - оставить ресурс висеть. Поэтому unwind phase должна быть не просто "найти handler и прыгнуть", а именно пройти все промежуточные уровни в правильном порядке.

Что происходит с return address во время фаз

Return address играет двойную роль. При обычном выполнении ret берет return address со стека и возвращает управление caller-у. При виртуальной раскрутке return address помогает системе вычислить caller RIP без выполнения ret. Если функция leaf и не имеет сложной unwind-информации, return address может быть главным источником следующего ControlPc.

> return address (адрес возврата)
> Адрес инструкции в caller-е, куда callee должна вернуться после завершения. На x64 call кладет return address на стек. Во время unwind он помогает восстановить RIP вызывающего кадра.

Но важно помнить: во время exception unwind return address не обязательно будет реально использован инструкцией ret. Система может прочитать его как данные для восстановления контекста, а затем передать управление совсем не обычным путем, например к handler-у. Поэтому return address - это часть модели восстановления, а не гарантия, что будет выполнен обычный return.

Почему листовые функции являются особым случаем

Leaf function кажется мелкой темой, но она важна для фаз. Если функция не меняла RSP сложным образом, не сохраняла nonvolatile-регистры и не вызывала другие функции, для нее может не быть полноценной записи RUNTIME_FUNCTION. В таком случае система может подняться к caller-у упрощенно: взять return address со стека и считать его новым RIP.

leaf function (листовая функция)
Функция, которая не вызывает другие функции и не требует полноценной unwind-метаинформации для восстановления caller context. В реальных компиляторах критерии зависят от того, меняет ли функция состояние, которое нужно описывать для unwind.

Но leaf-логика не означает, что можно игнорировать такие функции при анализе. Во время search phase leaf function обычно не содержит frame-based handler-а, но она все равно может быть текущим местом исключения. Система должна корректно перепрыгнуть от нее к caller-у. Если стек поврежден, даже простая leaf-логика может дать неправильный адрес.

Unwind history table: зачем системе память о поиске

В нескольких API встречается HistoryTable или UNWIND_HISTORY_TABLE. На первый взгляд это несущественная оптимизация, но она показывает, что поиск FunctionEntry может повторяться много раз. При обходе стека система снова и снова ищет записи функций по ControlPc. Кэширование помогает не выполнять одинаковый поиск каждый раз с нуля.

UNWIND_HISTORY_TABLE (таблица истории раскрутки)
Вспомогательная структура, которую можно использовать для ускорения повторных поисков function entries во время stack walking или unwind. Она не является главным источником истины, как .pdata/.xdata, а помогает быстрее находить уже встречавшиеся данные.

Для static /dynamic этот термин важен как граница: HistoryTable не нужно искать в PE-файле как секцию. Это runtime-вспомогательная структура. В PE лежат статические function tables и unwind data. HistoryTable живет во время выполнения и помогает системным API работать быстрее.

Сравнение фаз в одной таблице

Признак	Search phase	Unwind phase
Главный вопрос	Кто обработает исключение?	Как безопасно дойти до выбранного обработчика?
Отношение к стеку	Обычно не разрушает кадры, а виртуально проходит по ним.	Кадры действительно покидаются, cleanup-код получает управление.
Основные участники	RtlLookupFunctionEntry, RtlVirtualUnwind, language-specific handler, filter.	RtlUnwindEx, RtlVirtualUnwind, termination handlers, cleanup runtime.
Роль handler-а	Решить, подходит ли исключение этому кадру/scope.	Выполнить завершающую логику для покидаемых областей.
Типичная ошибка понимания	Думать, что поиск уже означает переход в обработчик.	Думать, что раскрутка - это просто stack trace или прибавление к RSP.

Таблица не заменяет повествование, но помогает зафиксировать различие. Search phase - выбор. Unwind phase - исполнение выбранного маршрута. Обе используют похожие низкоуровневые механизмы, но смысл действий разный.

Псевдокод C/SEH: как две фазы выглядят на исходнике

Чтобы связать системную механику с исходным кодом, полезно посмотреть на простую форму C/SEH. Здесь код намеренно учебный: он не показывает все детали компилятора, но помогает понять, где находится protected range, где filter, где handler body и где finally.

```
__try {
open_resource();

__try {
risky_operation(); // здесь может возникнуть exception
}
__finally {
close_inner_resource(); // cleanup при покидании внутренней области
}
}
__except (filter(GetExceptionCode(), GetExceptionInformation())) {
handle_exception(); // сюда попадем, если filter принял exception
}
```

Если исключение происходит внутри risky_operation, search phase сначала выясняет, есть ли обработчик в текущем кадре и текущей защищенной области. Внутренний __finally не отвечает "обрабатывать или нет". Он не является __except-фильтром. Поэтому поиск может идти дальше к внешнему __except. Если filter принимает исключение, target frame выбран. Потом unwind phase должна покинуть внутреннюю protected area и вызвать close_inner_resource. Только после этого управление может оказаться в handle_exception.

Этот пример важен тем, что показывает: исходный код имеет логические области, а бинарник хранит таблицы, по которым runtime восстанавливает эти области. __try не обязан выглядеть как отдельная машинная инструкция. __except filter и __finally превращаются в code ranges, handler addresses, scope records и language-specific data.

Почему handler body не равен filter

В __try/__except есть две разные части: filter и handler body. Filter выполняется во время search phase, чтобы решить, подходит ли обработчик. Handler body выполняется позже, после того как принято решение и выполнена необходимая раскрутка. Если их смешать, можно неправильно понять control flow.

> handler body (тело обработчика)
> Код внутри __except-блока, который выполняется после того, как фильтр принял исключение и система довела выполнение до нужного кадра. Это не то же самое, что filter expression.

Filter может быть коротким сравнением: GetExceptionCode() == EXCEPTION_ACCESS_VIOLATION. Handler body может выполнять полноценное восстановление, логирование, возврат ошибки, аварийное завершение части операции. Между filter и body стоит unwind phase. Поэтому в бинарном анализе нельзя просто найти адрес filter-а и считать, что это адрес обработки.

Почему система должна знать, находится ли RIP в prolog, body или epilog

Мы уже разбирали RtlVirtualUnwind и пролог/эпилог, но теперь это нужно привязать к двум фазам. Во время search phase система поднимается по кадрам. Чтобы подняться корректно, она должна понимать, в какой части функции находился ControlPc: в прологе, в теле или в эпилоге. Это влияет на то, какие действия пролога уже успели выполниться и как восстанавливать caller context.

> function body (тело функции)
> Основная часть функции после пролога и до эпилога. В теле обычно считается, что пролог уже полностью выполнил сохранение регистров и выделение стека, а эпилог еще не начал обратные действия.

Если исключение произошло в середине пролога, нельзя применять полную обратную схему так, будто весь пролог уже выполнен. Если RIP находится в эпилоге, функция уже начала разбирать кадр, и обычная логика тела тоже может быть неверной. Поэтому prolog/body/epilog - это не косметическое деление для дизассемблера, а условие правильной работы search phase и unwind phase.

Отсюда видно, почему Microsoft так строго ограничивает форму prolog и epilog на x64. Чем свободнее компилятор мог бы писать эти участки, тем сложнее было бы надежно распознать, какие части кадра уже созданы или уже уничтожены. Строгость ABI - цена за надежный unwind.

Почему два одинаковых crash-а могут вести себя по-разному

Два access violation с одинаковым кодом исключения могут пройти по разным маршрутам. Один может быть обработан локальным __except. Другой может пройти мимо всех фильтров и стать unhandled. Третий может быть first chance событием, которое защита использует как нормальный механизм перехода. Причина в том, что решение зависит не только от ExceptionCode, но и от ControlPc, текущего кадра, protected ranges, scope tables, фильтров, состояния стека и debugger-а.

Поэтому плохой анализ звучит так: "access violation - значит падение". Хороший анализ звучит иначе: "возник access violation; нужно проверить route dispatching: first chance, VEH, frame-based search, подходящие scopes, filter results, unwind path, second chance". Такой подход медленнее, но он соответствует реальной Windows-модели.

Алгоритм для static : как моделировать две фазы на статическом уровне

Статический анализатор не имеет живого стека, поэтому он не может полностью выполнить search phase как Windows в конкретном процессе. Но он может подготовить карту возможностей: какие функции имеют EHANDLER, какие UHANDLER, куда ведет HandlerData, какие ranges выглядят защищенными, где metadata согласована, а где нет. Это не runtime-исполнение, но это полезная подготовка к объяснению поведения.

`Static model for static :`

```
1. Parse PE headers and Exception Directory.
2. Build function range map from .pdata / RUNTIME_FUNCTION.
3. For each function:
- parse UNWIND_INFO;
- validate unwind codes;
- detect EHANDLER / UHANDLER / CHAININFO;
- resolve handler RVA;
- classify handler location and section permissions;
- if known handler (__C_specific_handler), try parse scope data;
- record protected ranges and cleanup ranges when possible.
4. Emit findings in terms of search/unwind impact.
5. Do not claim runtime handler selection without concrete ControlPc/stack/context.
```

Последняя строка особенно важна. Инструмент не должен переобещать. Если у него нет конкретного runtime ContextRecord, он не знает, какой именно frame станет target frame. Но он может сказать: "эта функция потенциально участвует в search phase" или "этот UHANDLER потенциально будет вызван во время unwind phase". Это честная и полезная формулировка.

Какие вопросы задавать при ручном разборе исключения

Когда ты смотришь конкретное исключение в отладчике или dump-е, полезно идти не хаотично, а по вопросам. Это дисциплинирует анализ и не дает прыгать сразу к красивым, но неверным выводам.

```
Questions for manual analysis:

1. What is ExceptionCode?
2. Is it first chance or second chance?
3. What is ControlPc / RIP?
4. Which module contains ControlPc?
5. Is there a RUNTIME_FUNCTION for this ControlPc?
6. Does UNWIND_INFO have EHANDLER or UHANDLER?
7. What handler address is referenced?
8. Is HandlerData understandable? Is it a known scope table?
9. Could a filter accept this exception?
10. If accepted, which frames must be unwound before target frame?
11. Are there __finally / cleanup / destructors on the path?
12. Does the stack look trustworthy enough for unwind?
```

Эти вопросы не являются отдельной практической главой, но они показывают, как теория превращается в метод. В хорошей книге теория должна давать не только определения, но и способ думать. Search phase и unwind phase дают именно такой способ: сначала маршрут поиска, потом маршрут выхода.

Итог раздела

Search phase - это фаза поиска подходящего обработчика по цепочке кадров. Она использует ControlPc, RtlLookupFunctionEntry, RtlVirtualUnwind, RUNTIME_FUNCTION, UNWIND_INFO, DispatcherContext и language-specific handlers. Ее задача - не разрушить стек, а найти target frame.

Unwind phase - это фаза контролируемого перехода к найденному обработчику. Она раскручивает промежуточные кадры, вызывает termination handlers и cleanup-логику, восстанавливает машинный контекст и приводит поток к выбранной цели. Она не

является простым stack walking и не сводится к изменению RSP.

Вместе эти две фазы объясняют, почему Windows x64 нужна такая строгая таблицная модель. .pdata сообщает, где функции. .xdata и UNWIND_INFO сообщают, как из них выходить. DispatcherContext связывает системный проход с языковым обработчиком. EstablisherFrame привязывает handler к конкретному кадру. A search/unwind phases задают порядок: сначала понять, кто отвечает, потом безопасно дойти до этого ответа.

6.2. Что такое language-specific handler

В предыдущем разделе мы разделили обработку исключения на две большие фазы: search phase и unwind phase. Это был важный шаг, потому что без него легко спутать два разных вопроса. Первый вопрос: кто вообще готов обработать исключение? Второй вопрос: как безопасно дойти до этого места, не оставив за собой разрушенный стек и невыполненную очистку? Теперь нужно добавить следующий слой. Windows умеет идти по стеку, читать .pdata, находить RUNTIME_FUNCTION, разбирать UNWIND_INFO и вычислять следующий кадр. Но Windows сама по себе не обязана понимать внутренние правила каждого языка программирования, каждого компилятора и каждой runtime-библиотеки.

Именно здесь появляется language-specific handler. На русском этот термин можно перевести как языково-специфический обработчик, но перевод звучит немного тяжело. Смысл проще: это специальная функция-обработчик, которая знает правила не всей Windows, а конкретной языковой модели. Windows приносит ей уже собранные материалы по текущему кадру: какое исключение произошло, где находится текущая функция, какой у нее establisher frame, где лежат дополнительные данные обработчика, какой сейчас ContextRecord. А дальше эта функция говорит: да, этот кадр подходит; нет, ищите выше; во время раскрутки нужно выполнить cleanup; этот блок __finally относится к текущему диапазону кода; этот C++ catch подходит или не подходит по типу.

language-specific handler (языково-специфический обработчик)
Это функция, которую Windows exception dispatcher вызывает для конкретного кадра стека, если unwind-метаданные функции указывают, что у нее есть обработчик. Она называется language-specific не потому, что внутри написан "английский" или "русский" язык, а потому что она понимает правила конкретного языка программирования, компилятора или runtime-модели: C SEH, C++ exceptions, managed runtime, JIT-код или другой механизм, построенный поверх системного x64 unwind.

Очень важное различие
Windows x64 unwind знает, как подняться по кадрам. Language-specific handler знает, что означают дополнительные данные конкретного языка: scope table, фильтры, finally-блоки, C++ catch-области, карты деструкторов и другие runtime-структуры. Без первого невозможно надежно идти по стеку. Без второго невозможно понять смысл языковых конструкций.

Почему Windows разделяет системную механику и языковой смысл

Представь, что Windows пришлось бы самой понимать все языки. Для C ей пришлось бы знать __try, __except, __finally и фильтры. Для C++ - типы исключений, таблицы catch-блоков, правила разрушения объектов, состояние try-блоков и внутренние структуры MSVC runtime. Для JIT-среды - свои таблицы, свои managed frames, свои мосты между машинным и управляемым кодом. Для каждого компилятора - свои детали генерации. Такая система стала бы огромной, хрупкой и почти нерасширяемой.

Поэтому архитектура устроена иначе. Windows берет на себя общий низкоуровневый фундамент: исключение как событие, машинный контекст, поиск функции по RIP,

восстановление caller context, переход по кадрам, вызов handler-функции по стандартизованному интерфейсу. А компилятор и runtime добавляют свой слой: они кладут рядом с UNWIND_INFO дополнительные данные и указывают адрес функции, которая умеет эти данные читать.

runtime (среда выполнения / рантайм)
Набор библиотек и служебного кода, который поддерживает выполнение программы на уровне выше чистых машинных инструкций. Например, C runtime помогает с запуском программы, библиотечными функциями и некоторыми механизмами языка. C++ runtime дополнительно участвует в исключениях, destructors, type matching и других вещах. Runtime - это не "сама Windows", а слой, который поставляется компилятором, языковой платформой или библиотекой.

Такое разделение удобно и для reverse engineering. Если ты видишь в UNWIND_INFO флаг handler-а, это еще не значит, что ты сразу знаешь весь смысл обработки исключения. Ты знаешь системный факт: у функции есть handler RVA. Но чтобы понять, является ли это C SEH, C++ exception handling, cleanup-only обработчиком или чем-то нестандартным, нужно смотреть, куда указывает handler RVA, какие данные лежат после него и как этот обработчик обычно интерпретирует HandlerData.

Где language-specific handler записан в PE и почему он не лежит в .pdata

Чтобы не потеряться, нужно снова собрать путь от файла к обработчику. В PE-файле есть Exception Directory. Для x64 он обычно указывает на массив RUNTIME_FUNCTION-записей, физически часто лежащий в секции .pdata. Каждая запись RUNTIME_FUNCTION описывает диапазон кода функции и указывает на unwind data. Эта unwind data обычно находится в .xdata и начинается со структуры UNWIND_INFO.

Пока UNWIND_INFO описывает только пролог, сохранение регистров, выделение стека и frame pointer, мы имеем чистую информацию для раскрутки кадра. Но у UNWIND_INFO есть Flags. Если в этих flags выставлены специальные признаки, после массива UNWIND_CODE появляется дополнительный хвост. В этом хвосте может лежать RVA обработчика - тот самый адрес language-specific handler-а. После этого RVA могут идти дополнительные данные, смысл которых уже зависит от конкретного handler-а.

Flags (флаги)
Небольшое битовое поле, где каждый бит сообщает о наличии или отсутствии некоторого свойства. В UNWIND_INFO флаги говорят, есть ли exception handler, unwind handler или chained unwind info. Флаг - это не текстовое описание, а компактный машинный признак, который parser должен читать побитово.

UNW_FLAG_EHANDLER (флаг exception handler-а)
Флаг в UNWIND_INFO, который означает, что после массива unwind codes находится RVA exception handler-а. Такой handler может участвовать в поиске обработчика исключения, то есть в search phase. Но сам флаг не говорит, какой именно язык или runtime используется. Он только открывает дверь к handler RVA и HandlerData.

UNW_FLAG_UHANDLER (флаг unwind handler-а)
Флаг в UNWIND_INFO, который означает, что у функции есть handler, важный во время unwind phase. На человеческом уровне это часто связано с cleanup-логикой: __finally, destructors, освобождение ресурсов или другой код, который должен выполниться при выходе из кадра из-за исключения.

Handler RVA (RVA обработчика)
Image-relative адрес функции-обработчика. Это не file offset и не обязательно абсолютный VA. Чтобы получить runtime-адрес, нужно прибавить ImageBase. Важно: Handler RVA обычно указывает не на твой пользовательский catch-блок напрямую, а на служебную runtime-функцию, которая читает language-specific data и уже по ним решает, какой пользовательский код относится к ситуации.

```
Упрощенная схема размещения:

RUNTIME_FUNCTION
BeginAddress
EndAddress
UnwindInfoAddress ---> UNWIND_INFO
header
UNWIND_CODE[]
padding до выравнивания
Handler RVA ; если EHANDLER/UHANDLER
HandlerData... ; смысл зависит от handler-а
```

Это место часто вызывает ошибку у новичков. Кажется, что если мы нашли Handler RVA, значит мы нашли конкретный обработчик из исходного кода. Но в реальном MSVC-коде множество функций могут указывать на один и тот же __C_specific_handler или __CxxFrameHandler. Сам runtime handler общий. Разными являются данные после Handler RVA: именно они описывают protected ranges, filters, finally blocks, catch maps или другие структуры.

Почему handler общий, а смысл кадра локальный

Хорошая аналогия: language-specific handler - это не личный адвокат каждой функции, а общий юрист, который умеет читать документы определенного формата. У разных функций документы разные, но юрист один. Когда происходит исключение, Windows не просит каждую функцию иметь собственный уникальный машинный код обработчика. Она вызывает общий обработчик и передает ему папку документов по текущему кадру. В этой папке находятся ExceptionRecord, ContextRecord, EstablisherFrame, DispatcherContext и HandlerData.

Для reverse engineer-а это критично. Если ты открыл таблицу и увидел, что тысячи функций указывают на один handler, не надо сразу думать, что это странность. В нормальной программе это может быть обычный runtime handler. Странность начинается не от самого факта повторного использования, а от контекста: handler лежит в необычной секции, HandlerData выходит за image, флаги конфликтуют, metadata повреждена, один нестандартный handler внезапно навешан на огромную часть функций, а секции имеют подозрительные permissions.

common handler (общий обработчик)
Одна функция-обработчик, используемая многими кадрами. Общий handler не означает, что все кадры обрабатывают исключения одинаково. Он означает, что обработчик читает разные language-specific data для разных функций.

local meaning (локальный смысл кадра)
Конкретный смысл обработки в данной функции: какие диапазоны кода защищены, какие фильтры есть, какие finally-блоки нужно вызвать, какие catch-блоки доступны. Этот смысл обычно живет не в адресе handler-а, а в данных, связанных с конкретной функцией.

Что получает language-specific handler

Чтобы понять роль language-specific handler-а, надо посмотреть не только на то, где он лежит, но и на то, какие данные ему передают. В Windows x64 handler вызывается не вслепую. Ему дают набор параметров, который связывает исключение, кадр, машинный контекст и служебные данные диспетчера. Для начинающего это может выглядеть как набор страшных указателей, но каждый из них отвечает на понятный вопрос.

Параметр	Зачем нужен handler-у
ExceptionRecord	Что произошло: код исключения, flags, адрес исключения, дополнительные параметры вроде адреса нарушения памяти при access violation.
EstablisherFrame	К какому конкретному стековому кадру относится этот handler. Это якорь кадра, через который handler может связать runtime-данные с текущей функцией.
ContextRecord	Как выглядел машинный контекст в момент исключения или текущий unwind-контекст. Здесь RIP, RSP, регистры и другое состояние потока.
DispatcherContext	Папка с системными сведениями по текущему кадру: FunctionEntry, ImageBase, HandlerData, ControlPc, LanguageHandler и другие поля.

ExceptionRecord (запись исключения)
Структура, описывающая само исключение как событие. Она отвечает на вопрос: что произошло? Например, это может быть access violation, divide by zero, breakpoint, illegal instruction или исключение, созданное RaiseException. Для handler-а это входной материал: он может решить, относится ли такой ExceptionCode к его области ответственности.

ContextRecord (запись машинного контекста)
Структура с состоянием регистров потока. В ней лежат RIP, RSP, RBP/RBX/RDI/RSI/R12-R15 и другие регистры. Handler может смотреть на этот контекст, а в некоторых ситуациях он может быть изменяемым. Но любое изменение контекста - сильное действие: оно влияет на то, куда и как продолжит выполняться поток.

DispatcherContext (контекст диспетчера)
Служебная структура, которую Windows готовит для текущего кадра перед вызовом language-specific handler-а. Она не описывает само исключение целиком и не заменяет ContextRecord. Ее роль - связать текущий кадр с найденной RUNTIME_FUNCTION, ImageBase, HandlerData и другими данными, нужными handler-у для решения.

Если сказать совсем по-человечески, Windows говорит handler-у: “Вот событие, вот кадр, вот машинное состояние, вот мои расчеты по этой функции, вот твои дополнительные данные. Решай по своим правилам”. Это и есть смысл стандартизованного интерфейса.

Что возвращает language-specific handler

Handler не просто выполняется и исчезает. Он должен вернуть системе ответ. Этот ответ сообщает dispatcher-у, что делать дальше. В Windows для этого используется семейство значений EXCEPTION_DISPOSITION. Слово disposition здесь можно понимать как решение, направление дальнейшего хода, результат рассмотрения.

EXCEPTION_DISPOSITION (решение обработчика по исключению)
Возвращаемое значение language-specific handler-а. Оно сообщает системному dispatcher-у, надо ли продолжать поиск, выполнить обработчик, продолжить выполнение или считать ситуацию вложенным исключением. Для reverse engineering это важно: handler - это не просто адрес кода, а участник протокола, который возвращает формальное решение.

Значение	Общий смысл
ExceptionContinueExecution	Handler говорит, что выполнение можно продолжить. Это сильное решение: поток должен вернуться к выполнению с некоторым ContextRecord. Не всякое исключение допускает продолжение.
ExceptionContinueSearch	Этот кадр не берет исключение на себя. Dispatcher должен идти выше по стеку и спрашивать следующие кадры.
ExceptionExecuteHandler	Найден обработчик, который должен принять исключение. После этого search phase фактически нашла цель, и система может перейти к unwind phase.
ExceptionNestedException / CollidedUnwind	Более сложные случаи, связанные с исключениями во время обработки или столкновением unwind-процессов. Для первой модели важно понимать, что протокол handler-а учитывает не только простой happy path.

На этом месте появляется еще одна типичная ошибка: думать, что handler всегда “ловит” исключение. Нет. Handler может быть вызван и вернуть ContinueSearch. Это нормально. Вызов handler-а означает только то, что текущий кадр имеет механизм, который нужно спросить. Ответ может быть отрицательным.

Как handler участвует в search phase

Во время search phase система идет по кадрам от места исключения вверх по стеку. Для каждого кадра она может найти RUNTIME_FUNCTION, прочитать UNWIND_INFO и увидеть: есть ли у этой функции handler. Если handler-а нет, система просто поднимается к следующему кадру. Если handler есть, Windows вызывает language-specific handler и передает ему контекст.

Задача handler-а в этой фазе - не обязательно выполнять пользовательский обработчик. Его первая задача - решить, относится ли текущее исключение к protected range этой функции и есть ли внутри данных кадра подходящее правило. Например, для C SEH это может быть проверка __try/__except ranges и вызов filter expression. Для C++ это может быть проверка, соответствует ли тип брошенного исключения одному из catch-блоков, видимых в текущем состоянии функции.

> protected range (защищенный диапазон кода)
> Диапазон инструкций, для которого язык или компилятор объявил особые правила обработки исключений. Например, тело __try в C SEH или область try в C++. Если исключение произошло вне защищенного диапазона, конкретный handler может решить, что этот кадр не относится к ситуации.

> filter (фильтр __except)
> Выражение или функция, которая во время C SEH search phase решает, что делать с исключением: обработать его, продолжить поиск или иногда продолжить выполнение. Фильтр - это не то же самое, что language-specific handler. Handler является runtime-механизмом, а filter - пользовательская или сгенерированная логика, которую handler может вызвать.

Вот почему один и тот же language-specific handler может быть вызван много раз по пути вверх. Он каждый раз смотрит не только на ExceptionCode, но и на текущий кадр. В одном кадре данные говорят: этот try-блок не активен. В другом: активен, но фильтр вернул ContinueSearch. В третьем: фильтр согласился обработать. Только тогда search phase получает target frame.

```
Упрощенная логика search phase:

for each frame from current to older:
entry = RtlLookupFunctionEntry(ControlPc)
unwind_info = entry.UnwindInfo

if unwind_info has no EHANDLER:
ControlPc = virtual_unwind_to_caller()
continue

handler = ImageBase + HandlerRva
decision = handler(ExceptionRecord, EstablisherFrame, ContextRecord, DispatcherContext)

if decision == ExceptionContinueSearch:
ControlPc = virtual_unwind_to_caller()
continue

if decision == ExceptionExecuteHandler:
target frame found
```

```
start unwind phase
```

Этот псевдокод не является точной реализацией Windows, но он полезен как умственная модель. Главное в нем - роль handler-а как “решающего слоя”. Windows умеет добраться до кадра и вызвать handler. Но применимость кадра определяет не generic dispatcher, а языковой runtime-код.

Как handler участвует в unwind phase

Когда search phase нашла target frame, начинается unwind phase. Здесь handler-ы получают другую роль. Теперь вопрос уже не “кто хочет обработать исключение?”, а “какие промежуточные кадры нужно корректно покинуть?”. На этом пути могут быть кадры, которые сами не ловят исключение, но обязаны выполнить cleanup.

> cleanup (очистка ресурсов)
> Код, который должен выполниться при выходе из области: освобождение памяти, закрытие файла, разблокировка mutex-а, вызов destructor-а, выполнение __finally. Cleanup важен именно потому, что исключение обрывает обычный путь return, и без специального unwind-процесса этот код мог бы не выполниться.

> termination handler (завершающий обработчик)
> Обработчик, вызываемый при выходе из защищенной области, особенно во время unwind. В C SEH близкая конструкция - __finally. Он не обязательно решает, перехватывать ли исключение; часто его задача - выполнить завершающее действие при покидании области.

На уровне UNWIND_INFO эта разница отражается через флаги. EHANDLER больше связан с поиском обработчика исключения, UHANDLER - с unwind-логикой. Но в реальном runtime одна и та же handler-функция может участвовать и в поиске, и в раскрутке, различая режим через параметры и контекст. Поэтому нельзя читать флаги как бытовые ярлыки “этот ловит, этот чистит” без понимания конкретного handler-а.

Во время unwind phase language-specific handler может пройти по своим данным и выполнить нужные действия для диапазонов, которые покидаются. Для C SEH это может быть __finally. Для C++ - вызов destructors для объектов, которые уже были сконструированы и теперь должны быть разрушены. Для другого runtime - своя логика выхода из кадра.

Почему language-specific data нельзя понимать универсально

После Handler RVA в UNWIND_INFO могут идти дополнительные данные. Microsoft на уровне x64 unwind не навязывает им единый универсальный формат на все языки. Это намеренно. Система знает, где начинается этот хвост, и передает указатель handler-у. А вот что именно лежит в хвосте, зависит от handler-а.

HandlerData (данные обработчика)
Указатель на дополнительные данные после Handler RVA в хвосте UNWIND_INFO. Эти данные не являются частью универсального набора UWOP_* операций. Их формат зависит от language-specific handler-а. Для __C_specific_handler это один формат scope information. Для C++ handler-а - другой набор структур. Для нестандартного runtime - третий.

scope table (таблица областей / таблица scope-ов)
Структура, которая описывает диапазоны защищенного кода и связанные с ними filter/finally/handler-адреса. Слово scope здесь означает область действия языковой конструкции. Для reverse engineering scope table полезна тем, что связывает машинные адреса с логикой __try/__except/__finally, но ее конкретный формат зависит от runtime handler-а.

Поэтому статический анализатор обязан быть аккуратным. Он может надежно сказать: UNWIND_INFO имеет EHANDLER, Handler RVA = такой-то, HandlerData начинается по такому-то RVA. Если handler известен и поддерживается анализатором, можно попробовать декодировать scope table. Но если handler неизвестен, нельзя притворяться, что все данные имеют формат __C_specific_handler. Это путь к ложным выводам.

Граница доказательства для reverse engineering
Факт “есть Handler RVA” доказывается из UNWIND_INFO. Факт “handler похож на __C_specific_handler” доказывается через символы, импорт, адрес, сигнатуру или fingerprint. Факт “этот __except реально обработает именно это исключение” требует анализа scope table, filter-а, ControlPc и runtime-контекста. Это разные уровни уверенности.

Language-specific handler - это не то же самое, что пользовательский обработчик

Здесь нужно остановиться, потому что это одна из самых неприятных ловушек. В исходном коде ты видишь что-то вроде __except { ... } или catch(...) { ... }. Естественно хочется найти в PE “адрес этого обработчика”. Но x64-модель часто устроена не так прямолинейно. В UNWIND_INFO может лежать адрес общей runtime-функции. Эта функция читает таблицу областей, вызывает фильтр, определяет целевой блок и только потом приводит управление к пользовательскому коду.

user handler (пользовательский обработчик)
Код, который программист написал или который виден в исходнике как тело __except, catch, finally или cleanup-блока. Он может не совпадать с Handler RVA в UNWIND_INFO. Handler RVA чаще указывает на runtime-функцию, которая знает, как найти и выполнить пользовательский обработчик.

runtime handler (runtime-обработчик)
Служебная функция, предоставленная компилятором или runtime-библиотекой. Она читает language-specific data и управляет языковыми правилами обработки. __C_specific_handler и C++ frame handler - примеры runtime handler-ов.

Для реверса это означает: если дизассемблер показывает handler = __C_specific_handler, на этом анализ не закончен, а только начался. Настоящий вопрос: какие данные передаются этому handler-у? Какие диапазоны они описывают? Где filter-функции? Где finally-блоки? Какие адреса относятся к пользовательскому control flow, а какие к служебному runtime-коду?

Пример с C SEH: __try, __except и общий handler

Возьмем простую идею без привязки к конкретным байтам. В функции есть блок __try, внутри него может произойти access violation, а после него есть __except-фильтр. Компилятор не обязан вставлять в стек x86-подобную SEH-запись. В x64 он может описать функцию через UNWIND_INFO, выставить handler-флаг и положить в хвост данные, которые понимает __C_specific_handler.

```
Условный исходный смысл:

void f() {
__try {
risky_memory_access();
}
__except(filter(GetExceptionInformation())) {
handle_error();
}
}

Условный x64 EH-смысл:

UNWIND_INFO flags: EHANDLER
Handler RVA: __C_specific_handler
HandlerData: scope information for __try / __except
```

Когда исключение происходит внутри risky_memory_access, Windows доходит до кадра f и видит handler. Она вызывает __C_specific_handler. Но __C_specific_handler не магически знает исходный код f. Он смотрит в HandlerData. Там описано, какие диапазоны кода относятся к __try, где находится filter, куда вести выполнение при успешном фильтре, какие действия нужны при unwind.

Если filter возвращает решение обработать исключение, search phase находит target frame. Затем unwind phase поднимает стек до этого кадра, выполняет нужную cleanup-логику на пути, и только после этого управление попадает туда, куда должен вести обработчик. Поэтому пользовательское тело __except не равно самому __C_specific_handler. __C_specific_handler - это механизм выбора и маршрутизации.

Пример с C++: почему одного Handler RVA еще меньше достаточно

В C++ ситуация еще богаче. C++ exceptions должны учитывать тип брошенного объекта, доступные catch-блоки, порядок catch-ов, вложенность try-блоков, уже сконструированные объекты, destructors и состояние функции. Для этого компилятор генерирует довольно сложные metadata. Windows не обязана понимать C++ type system. Она передает управление C++ frame handler-у, а тот уже работает со своими структурами.

C++ frame handler (кадровый обработчик C++ исключений)
Runtime handler, который понимает внутреннюю модель C++ exceptions конкретного компилятора. В MSVC-среде часто встречаются обработчики семейства __CxxFrameHandler. Они работают не просто с кодом исключения Windows, а с C++ metadata: картами try-блоков, catch-обработчиками, состояниями объектов и логикой разрушения.

type matching (сопоставление типов)
Процесс проверки, подходит ли брошенное C++ исключение под конкретный catch. Например, catch(Base&) может принимать объект Derived, а catch(int) не принимает исключение другого типа. Для Windows dispatcher-а это слишком высокоуровневое знание; этим занимается C++ runtime handler.

destructor (деструктор)
Специальная функция C++, вызываемая при уничтожении объекта. При исключении destructors для уже созданных локальных объектов должны быть вызваны во время stack unwinding. Это языковая обязанность C++, а не просто механическое восстановление RSP.

Именно из-за C++ хорошо видно, почему термин language-specific handler выбран удачно. Это не декоративное название. Обработчик действительно специфичен для языка и runtime-а: он знает такие вещи, которые Windows на уровне generic x64 unwind знать не должна.

Как dispatcher понимает, какого handler-а вызвать

Windows dispatcher не выбирает handler по имени языка. В PE нет красивой строки “это C++” или “это C SEH” на уровне универсальной unwind-модели. Dispatcher действует проще и ниже уровнем: он читает UNWIND_INFO, видит флаг, находит Handler RVA, вычисляет VA обработчика через ImageBase и вызывает эту функцию по ожидаемому протоколу.

LanguageHandler (языковой обработчик в DispatcherContext)
Поле или смысловое значение в dispatcher context, указывающее на handler-функцию, которую dispatcher будет вызывать для текущего кадра. Это runtime-адрес функции, полученный из Handler RVA и ImageBase.

ImageBase + HandlerRVA (вычисление адреса обработчика)
Типичная формула перехода от image-relative адреса к runtime VA. Handler RVA хранится относительно начала загруженного image. Чтобы вызвать функцию в памяти процесса, система прибавляет ImageBase. Новичок часто ошибается, принимая RVA за file offset или уже готовый абсолютный адрес.

После этого уже сам handler определяет, как читать HandlerData. Поэтому один dispatcher может работать с разными языками, не зная их детально. Он вызывает разные функции, а каждая функция знает свой формат хвоста. Это похоже на интерфейс: Windows задает форму вызова, runtime задает смысл данных.

Зачем handler-у нужен EstablisherFrame

Мы уже отдельно разбирали EstablisherFrame, но здесь важно увидеть его глазами language-specific handler-а. HandlerData может описывать области функции, но handler должен применить эти области к конкретному runtime-кадру. Одна и та же функция может быть вызвана много раз рекурсивно. Ее metadata в PE будет одна, а стековых кадров в процессе может быть несколько. Поэтому handler-у нужен не только статический адрес функции, но и привязка к конкретному экземпляру кадра.

EstablisherFrame дает эту привязку. Через него handler может понимать, где находится фиксированная часть стекового выделения функции и как соотнести language-specific данные с текущим кадром. Если EstablisherFrame вычислен неверно из-за поврежденной unwind metadata или нестандартного кода, handler может начать читать или применять данные к неправильной основе. Это уже не просто ошибка дизассемблера - это потенциальное нарушение всего exception route.

> Почему это важно для защит и упаковщиков
> Если программа сознательно портит или усложняет unwind metadata, она может ломать не только красивый stack trace, но и понимание того, какой handler относится к какому кадру. Для анализатора такие места важны: они показывают, где control flow может зависеть от metadata, а не от очевидных прямых переходов в коде.

EHANDLER и UHANDLER: не читать слишком грубо

В учебной таблице хочется написать: EHANDLER - для exception handler, UHANDLER - для unwind handler. Это правда, но как первое приближение. Если остановиться на этой фразе, возникнет ложная простота. Реальный language-specific handler может вызываться в разных фазах и по-разному интерпретировать контекст. Один и тот же общий handler может поддерживать и поиск, и cleanup.

Правильнее думать так: flags в UNWIND_INFO говорят dispatcher-у, что после unwind codes есть handler tail и что этот кадр требует дополнительной обработки. Они не раскрывают весь языковой смысл. Этот смысл возникает только в связке: flags + Handler RVA + HandlerData + текущая фаза + ExceptionRecord + DispatcherContext.

Элемент	Что он реально доказывает
UNW_FLAG_EHANDLER	В unwind tail есть handler, связанный с обработкой исключений. Это сильный признак участия кадра в search phase, но не полное описание пользовательской логики.
UNW_FLAG_UHANDLER	В unwind tail есть handler, важный для unwind/cleanup. Это признак того, что при раскрутке кадра может понадобиться языковая логика выхода.
Handler RVA	Адрес runtime-функции обработчика относительно ImageBase. Сам по себе не говорит, какие ranges и filters активны.
HandlerData	Дополнительные данные конкретного handler-а. Их можно декодировать только зная формат, который ожидает этот handler.

Чем language-specific handler отличается от VEH

Новичку легко смешать все обработчики в одну кучу: SEH handler, VEH handler, language-specific handler, unhandled exception filter. Но это разные уровни маршрута. Vectored Exception Handler вызывается как более глобальный механизм, не привязанный к конкретному стековому кадру через .pdata/.xdata. Language-specific handler, о котором мы говорим сейчас, находится внутри frame-based x64-модели: его вызывают для конкретной функции, если ее UNWIND_INFO указывает на handler.

> VEH / Vectored Exception Handling (векторная обработка исключений)
> Глобальнее устроенный механизм Windows, при котором зарегистрированные vectored handlers получают уведомления об исключениях независимо от конкретной функции, чей кадр сейчас рассматривается через .pdata/.xdata. VEH не заменяет language-specific handlers: это другой слой маршрута исключения.

Подробно VEH будет отдельным разделом позже. Сейчас важно только не путать: language-specific handler является частью frame-based обработки кадра, а VEH - отдельная система предварительных или дополнительных обработчиков на уровне процесса.

Чем language-specific handler отличается от top-level / unhandled handler

Top-level или unhandled exception handling - это поздний этап, когда обычные способы обработки не нашли подходящего решения. Language-specific handler работает раньше и тоньше: он участвует в проходе по кадрам, может принять исключение в конкретной функции или выполнить cleanup при выходе из этой функции. Unhandled filter - это уже последняя линия поведения процесса, когда исключение осталось необработанным.

> top-level exception filter (верхнеуровневый фильтр необработанных исключений)
> Механизм последнего уровня, который может быть вызван, когда исключение не было обработано обычными путями. Он не является заменой frame-based language-specific handler-ам и не описывается через UNWIND_INFO конкретной функции.

Если в crash dump ты видишь unhandled exception, это не значит, что language-specific handlers не вызывались. Возможно, они вызывались и каждый вернул ContinueSearch. Возможно, фильтры отказались. Возможно, metadata была повреждена. Поэтому вывод “нет обработчиков” нельзя делать только по факту финального падения.

Почему один handler может быть нормальным, а другой подозрительным

Для reverse engineering важна не только теория, но и инстинкт проверки. В нормальном MSVC-бинаре повторяющиеся handlers - обычное дело. __C_specific_handler или C++ handler могут встречаться во многих функциях. Но анализатор должен смотреть не только на имя, а на совокупность признаков.

handler dominance (доминирование обработчика)
Ситуация, когда один handler RVA встречается у очень большого числа функций. Само по себе это не обязательно плохо: runtime handler может быть общим. Но в сочетании с необычной секцией, отсутствием символов, поврежденными HandlerData или странными permissions это может стать признаком обфускации, упаковки или нестандартного exception-based control flow.

section permissions (права секции)
Атрибуты PE-секции: readable, writable, executable и другие. Если handler RVA указывает в исполняемую секцию - это ожидаемо. Если handler или HandlerData находятся в странной writable/executable области, это не автоматическое доказательство вредоносности, но повод для дополнительной проверки.

Хороший static не должен кричать “malware” только потому, что handler нестандартный. Он должен собирать evidence: где находится handler, сколько функций на него указывает, читается ли HandlerData, согласованы ли ranges, есть ли chained info, не выходит ли RVA за SizeOfImage, совпадает ли поведение с известными runtime-шаблонами, есть ли динамические function tables. Только такая совокупность превращается в внятный verdict.

Что именно может проверить статический static

Статический static не исполняет программу и не видит живой стек. Поэтому он не может полностью повторить search phase и unwind phase. Но он может очень много подготовить. Он может пройти по RUNTIME_FUNCTION, найти UNWIND_INFO, увидеть flags, вычислить handler tail, проверить handler RVA, классифицировать handler, попробовать распознать HandlerData и построить карту потенциального участия функций в обработке исключений.

```
Static analysis checklist for language-specific handlers:

1. Parse RUNTIME_FUNCTION and resolve UnwindInfoAddress.
2. Parse UNWIND_INFO header and UNWIND_CODE array.
3. Align after unwind codes to the handler tail.
4. If EHANDLER or UHANDLER is set:
read HandlerRVA;
check HandlerRVA < SizeOfImage;
map HandlerRVA to section;
compute handler VA conceptually as ImageBase + HandlerRVA;
classify known handler patterns if possible;
locate HandlerData start;
decode HandlerData only if handler format is known.
5. Emit evidence, not fantasy:
"handler present";
"handler points to known __C_specific_handler-like routine";
"handler data decodable as scope table";
"handler data malformed";
"handler target outside image".
```

Обрати внимание на фразу “decode only if handler format is known”. Это дисциплина. Многие ошибки в подобных проектах появляются, когда анализатор берет один известный формат и натягивает его на все случаи. В коротких тестах это может выглядеть красиво, но на реальных бинарях быстро рождает ложные срабатывания.

Как описывать результаты без переобещаний

В отчете проекта нужно очень аккуратно выбирать слова. Разница между “функция имеет handler RVA” и “функция обработает access violation” огромная. Первое доказывается байтами UNWIND_INFO. Второе требует знания конкретного исключения, ControlPc, активного кадра, фильтра, результатов filter expression и состояния runtime.

Плохая формулировка	Хорошая формулировка
Эта функция ловит все исключения.	UNWIND_INFO функции содержит EHANDLER; handler RVA указывает на известный/неизвестный runtime handler. Для вывода о конкретном перехвате нужен анализ HandlerData и runtime-контекста.
Это адрес catch-блока.	Это Handler RVA. Он может указывать на общий language-specific handler; пользовательский catch/except может быть описан в language-specific data.
Данные после handler-a - это всегда scope table.	Данные после handler RVA являются language-specific. Их формат зависит от handler-a.
Нестандартный handler = вредоносность.	Нестандартный handler - finding, усиливающийся при сочетании с placement anomalies, parse failures, suspicious permissions или runtime behavior.

Такой стиль отчета может казаться осторожным, но именно он профессионален. Reverse engineering - это не угадывание. Это сбор проверяемых фактов и построение вывода с указанием границ уверенности.

Как это выглядит глазами дизассемблера

Дизассемблер может показать функцию, ее диапазон, references, unwind info, handler RVA и иногда имя handler-a, если есть символы или распознавание. Но дизассемблер не всегда автоматически объясняет всю языковую логику. Особенно если бинарь stripped, packed, написан не MSVC, использует custom runtime или генерирует код динамически.

> stripped binary (бинарь без символов)
> Исполняемый файл, из которого удалены имена функций, PDB-связи и другая отладочная информация. В таком файле handler может быть виден как адрес, но не как красивое имя __C_specific_handler. Аналитику приходится опираться на структуру, сигнатуры, imports, patterns и поведение.

> custom runtime (нестандартная среда выполнения)
> Runtime, который не совпадает с привычными MSVC CRT/C++ patterns. Это может быть JIT, виртуальная машина, защита, игровой движок, язык со своей системой исключений или ручная assembly-инфраструктура. Для custom runtime нельзя автоматически применять правила __C_specific_handler.

Если в Ghidra или IDA ты видишь handler tail, полезно идти по уровням. Сначала доказать, что handler tail вообще корректно прочитан. Потом посмотреть, куда указывает handler RVA. Потом проверить, находится ли он в кодовой секции. Потом попытаться понять, известный ли это handler. И только потом переходить к смыслу HandlerData. Это

медленнее, зато не приводит к фантазиям.

Почему language-specific handler важен для exception-based control flow

Некоторые программы используют исключения не только как аварийный механизм, но и как часть управления потоком. Это может быть легитимный runtime-прием, защитная техника, обфускация или антиотладочный маршрут. В таких случаях прямой граф переходов по jmp/call не показывает весь контрольный поток. Часть логики может проходить через raise, exception dispatch, filters, handlers и resume/continue decisions.

> exception-based control flow (управление потоком через исключения)
> Стиль или техника, при которой исключения используются как часть маршрута выполнения программы. Вместо очевидного if/jmp программа может вызвать исключение, дать handler-у принять решение и продолжить выполнение в другом месте. Для reverse engineering это сложно, потому что часть control flow спрятана в системном dispatcher-е и language-specific logic.

Language-specific handler в такой картине становится точкой, где системный и пользовательский миры встречаются. Если анализатор игнорирует handlers и HandlerData, он может пропустить важные переходы. Если он преувеличивает их смысл, он получит ложный CFG. Поэтому задача не просто “найти handler”, а построить честную модель: где есть потенциальный exception route, какие данные его поддерживают, какие выводы подтверждены, а какие требуют runtime-проверки.

Как handler связан с __C_specific_handler, который будет следующим разделом

Следующий раздел будет посвящен __C_specific_handler. Важно заранее понять, почему он заслуживает отдельного разговора. Language-specific handler - это общий класс механизма. __C_specific_handler - конкретный важный представитель этого класса для C structured exception handling в Windows x64. Он показывает, как абстрактная идея превращается в реальные scope tables, __try/__except, __finally и решения handler-а.

Если сейчас смешать эти уровни, дальше будет каша. Поэтому держим разделение: language-specific handler - архитектурная роль. __C_specific_handler - конкретная реализация этой роли для определенной модели. C++ frame handler - другая реализация. Custom runtime handler - третья. Все они вызываются Windows по похожему системному протоколу, но читают разные данные и принимают решения по разным правилам.

> __C_specific_handler (C-специфический обработчик SEH)
> Конкретный runtime handler Windows/MSVC-среды, который используется для реализации C structured exception handling extensions: __try, __except, __finally. Он будет разобран отдельно, потому что там появляются scope table, фильтры и практический мост между UNWIND_INFO и исходными конструкциями C SEH.

Минимальная модель, которую нужно удержать в голове

Теперь можно собрать все в одну компактную, но не поверхностную модель. Когда исключение дошло до frame-based части x64-маршрута, Windows идет по кадрам. Для каждого кадра она использует RUNTIME_FUNCTION и UNWIND_INFO. Если UNWIND_INFO не указывает handler, система просто может подняться дальше. Если handler есть, Windows не сама интерпретирует языковые конструкции. Она вызывает language-specific handler.

Этот handler получает ExceptionRecord, EstablisherFrame, ContextRecord и DispatcherContext. Через DispatcherContext он видит FunctionEntry, ImageBase, HandlerData и другие сведения. По своим правилам он решает: продолжить поиск, принять исключение, выполнить cleanup, участвовать в unwind. Он может читать scope table, filters, C++ metadata или другие данные.

В PE это выражается скромно: флаг в UNWIND_INFO, Handler RVA и хвост данных. Но смысл у этого хвоста большой. Именно здесь низкоуровневая таблицная модель x64 соединяется с высокоуровневыми конструкциями языка. Поэтому language-specific handler - один из ключевых мостов между байтами PE и реальным поведением программы.

Типичные ошибки понимания

Под конец полезно явно перечислить ошибки, потому что они постоянно встречаются у начинающих и в слабых описаниях темы.

```
Common mistakes:

1. Treat Handler RVA as the address of the user catch/except block.
2. Assume every HandlerData tail has the same format.
3. Confuse language-specific handler with VEH.
4. Confuse language-specific handler with unhandled exception filter.
5. Think EHANDLER means "this frame definitely handles every exception".
6. Ignore EstablisherFrame and lose the link to the concrete stack frame.
7. Decode unknown handler data as __C_specific_handler scope table.
8. Treat repeated common handler usage as suspicious without context.
9. Forget that search phase and unwind phase may call handler for different reasons.
10. Claim runtime behavior from static metadata without ControlPc, stack and filter
results.
```

Если ты избежишь этих десяти ошибок, ты уже будешь читать x64 exception metadata гораздо аккуратнее, чем большинство поверхностных разборов.

Почему handler нельзя анализировать отдельно от фазы

Один и тот же адрес language-specific handler-а может появиться и во время search phase, и во время unwind phase, но смысл вызова будет разным. Это тонкость, которую легко пропустить, если смотреть только на дизассемблер. В дизассемблере адрес один, имя одно, ссылка из UNWIND_INFO одна. Но runtime-ситуация разная: в search phase система спрашивает, принимает ли кадр исключение; в unwind phase система сообщает, что кадр покидается и нужно выполнить действия выхода.

Представь преподавателя, который в одном случае принимает экзамен, а в другом подписывает обходной лист. Человек тот же, кабинет тот же, но процедура другая. Так же

и language-specific handler: сама функция может быть общей точкой входа, но режим работы определяется параметрами, DispatcherContext, HandlerData и фазой обработки. Поэтому анализ вида "handler есть - значит исключение поймано" слишком грубый. Правильный анализ спрашивает: в какой фазе handler вызывается, какие данные он получает и какое EXCEPTION_DISPOSITION может вернуть.

> phase-sensitive behavior (поведение, зависящее от фазы)
> Ситуация, когда один и тот же runtime handler ведет себя по-разному в search phase и unwind phase. В search phase он может проверять filters или catch-типы. В unwind phase он может выполнять cleanup, finally или destructors. Для reverse engineering важно не только наличие handler-а, но и роль handler-а в конкретной фазе.

Отсюда появляется важное практическое правило: когда ты документируешь handler в отчете, не надо писать просто "handler = X" и ставить точку. Лучше писать: "UNWIND_INFO содержит handler tail; флаги указывают на участие в exception/unwind механизме; handler RVA указывает на X; дальнейший смысл зависит от HandlerData и фазы". Это длиннее, но это честнее.

Как compiler, linker и loader участвуют в этой истории

Language-specific handler не появляется в бинаре сам по себе. За ним стоит цепочка инструментов. Compiler (компилятор) видит исходный код и понимает, где programmer написал __try, __except, __finally, try/catch или конструкции, требующие cleanup. Compiler генерирует машинный код функции, unwind-инструкции для пролога, а также language-specific metadata. Linker (компоновщик) собирает итоговый PE, размещает .pdata/.xdata, связывает ссылки на handler-функции и может подтянуть нужные runtime-библиотеки. Loader (загрузчик Windows) загружает image в память, после чего exception dispatcher может работать с активными function tables.

> compiler (компилятор)
> Инструмент, который переводит исходный код в машинный код и служебные metadata. В нашей теме compiler важен потому, что именно он знает исходные языковые конструкции и поэтому может создать HandlerData, scope tables и ссылки на нужный language-specific handler.

> linker (компоновщик)
> Инструмент, который собирает объектные файлы и библиотеки в итоговый PE-файл. Он размещает секции, таблицы и ссылки так, чтобы RUNTIME_FUNCTION, UNWIND_INFO и handler RVA оказались в согласованном виде. Linker не придумывает смысл __try или catch с нуля, но он формирует итоговую картину файла.

> loader (загрузчик)
> Компонент Windows, который отображает PE image в память процесса, применяет relocations при необходимости, подготавливает imports и делает image пригодным для выполнения. Для exception handling важно, что после загрузки адреса становятся runtime-адресами, а image-relative данные интерпретируются относительно ImageBase.

Такое разделение помогает не путать уровни. Если ты видишь HandlerData в .xdata, это не "Windows туда что-то записала во время выполнения". Обычно это результат работы

компилятора и компоновщика, уже лежащий в PE. A Windows во время выполнения это читает и передает handler-у. Для dynamic code и JIT ситуация другая: таблицы могут регистрироваться во время выполнения, но сама идея остается прежней - системный слой получает function table и handler metadata, а language-specific handler понимает дополнительный смысл.

JIT и dynamic code: почему language-specific handler не обязан жить только в обычном PE

До сих пор мы говорили в основном об обычном PE-файле. Но Windows x64 unwind поддерживает и динамически созданный код. JIT-компилятор может сгенерировать машинный код в памяти, создать для него function table и зарегистрировать ее через специальные API. Если такой код хочет нормально участвовать в исключениях и stack walking, ему тоже нужно дать системе данные, похожие по роли на .pdata/.xdata.

> JIT / Just-In-Time compilation (компиляция во время выполнения)
> Модель, при которой код генерируется не заранее в PE-файле, а прямо во время работы программы. JIT-среда может создавать машинные функции в памяти, а затем регистрировать для них unwind-информацию, чтобы Windows могла ходить по стеку и обрабатывать исключения.

> dynamic function table (динамическая таблица функций)
> Таблица RUNTIME_FUNCTION-записей, зарегистрированная во время выполнения, а не взятая из статической .pdata обычного PE. Она нужна для JIT-кода, runtime-generated code и некоторых нестандартных сценариев. В таких случаях language-specific handler и HandlerData могут быть связаны не с привычной секцией .xdata файла, а с данными, подготовленными runtime-ом.

Для reverse engineering это означает: если статический PE-анализатор не видит handler-а в файле, это еще не доказывает, что во время выполнения handler-ов не будет. JIT или защитный runtime может зарегистрировать function table позже. Поэтому статический отчет должен различать "не найдено в статическом image" и "невозможно в принципе". Это разные утверждения.

Ручное чтение handler tail: медленная, но надежная процедура

Когда ты руками читаешь .xdata, у тебя должна быть спокойная процедура. Не надо сразу прыгать к красивому имени handler-а. Сначала нужно убедиться, что ты правильно дошел до хвоста. UNWIND_INFO начинается с header-а. Потом идет массив UNWIND_CODE. Количество элементов задается CountOfCodes. После массива может быть padding для выравнивания. И только после этого при наличии EHANDLER или UHANDLER находится 4-байтовый Handler RVA.

> padding (выравнивающая вставка)
> Служебные байты, которые могут добавляться, чтобы следующая часть структуры начиналась с правильной границы выравнивания. В UNWIND_INFO важно не перепутать padding с настоящим handler RVA или с payload-слотом UNWIND_CODE. Ошибка на несколько байт полностью ломает дальнейший разбор.

```
Manual reading discipline:

1. Read UNWIND_INFO header.
2. Read CountOfCodes.
3. Read exactly CountOfCodes slots of UNWIND_CODE.
4. Remember that some operations consume extra slots as payload.
5. Move to the aligned address after the code array.
6. If flags contain EHANDLER or UHANDLER:
read 4 bytes as HandlerRVA;
the next byte after HandlerRVA starts HandlerData.
7. If flags contain CHAININFO instead:
tail is not HandlerRVA; it is chained RUNTIME_FUNCTION.
```

Последняя строка особенно важна. UNW_FLAG_CHAININFO меняет смысл хвоста. Если parser увидел CHAININFO и все равно пытается читать Handler RVA, он уже ушел в неверную модель. Поэтому флаги нужно читать до интерпретации хвоста, а не после.

Когда metadata корректна, но вывод все равно может быть неверным

Бывает, что все байты формально корректны: Handler RVA внутри image, UNWIND_INFO читается, flags не конфликтуют, HandlerData доступна. Но даже тогда вывод о поведении может быть неверным, если анализатор не учитывает runtime-условия. Например, C SEH filter может зависеть от значения регистра, глобальной переменной, результата функции или содержимого ExceptionRecord. Статический анализатор видит, что filter существует, но не всегда может заранее знать, что он вернет.

> runtime condition (условие времени выполнения)
> Условие, результат которого известен только при конкретном запуске программы: значения переменных, регистров, памяти, состояния потока, внешних данных. Exception filters и некоторые handlers могут зависеть от таких условий, поэтому статическая metadata не всегда дает окончательный ответ.

Поэтому аккуратный вывод звучит так: "функция содержит metadata, позволяющую handler-у рассмотреть исключение в таком диапазоне". А не так: "функция точно обработает исключение". Чтобы утверждать второе, нужно знать, что исключение произошло именно в нужном protected range, что filter выполнится именно так, что stack/context доверенные, и что handler не вернет ContinueSearch.

Как это проверять в отладчике

В debugger-е полезно смотреть не только на место падения, но и на маршрут. Для начинающего это сначала неприятно: хочется увидеть одну строку, где "все понятно". Но exception handling редко дает такую роскошь. Нужно смотреть RIP, stack trace, модуль, function table, handler flags, а при возможности - точки вызова RtlLookupFunctionEntry, RtlVirtualUnwind и language-specific handler-а.

debugger (отладчик)
Инструмент для наблюдения и управления выполнением программы: остановки, breakpoints, просмотр регистров, памяти, стека и исключений. В нашей теме debugger помогает увидеть runtime-часть, которую статический PE-анализ не видит: конкретный ControlPc, конкретный ContextRecord, вызовы handler-ов и результаты фильтров.

breakpoint (точка останова)
Условная или безусловная остановка выполнения на выбранной инструкции или событии. В анализе исключений breakpoint на language-specific handler может показать, какие кадры реально доходят до handler-а и в какой фазе это происходит.

```
Debugger-oriented questions:

1. Where is RIP when exception is raised?
2. Is this first chance or second chance?
3. Which module owns ControlPc?
4. Does the current frame have UNWIND_INFO with handler flags?
5. Which handler VA is called?
6. Is it a known runtime handler or custom code?
7. What is HandlerData for this frame?
8. What does the handler return?
9. Does the system continue search or start unwind?
10. Which cleanup handlers run before target frame?
```

Эти вопросы помогают связать статический и динамический миры. Статический static строит карту. Debugger показывает, какой маршрут из этой карты реально используется в данном запуске.

Почему beginner-у кажется, что тема слишком абстрактная

Если ты только начинаешь, language-specific handler может казаться слишком далеким от реального кода: какие-то RVA, флаги, DispatcherContext, scope tables. Но на самом деле это одна из самых практичных тем. Когда программа падает, когда стек выглядит странно, когда декомпилятор не показывает очевидный control flow, когда C++ исключения проходят через непонятные функции, когда защита использует RaiseException, именно эта тема объясняет, почему обычный взгляд на call/jmp недостаточен.

Нужно просто не пытаться проглотить все сразу. Сначала удерживай базовую цепочку: функция имеет RUNTIME_FUNCTION; RUNTIME_FUNCTION ведет к UNWIND_INFO; UNWIND_INFO может иметь handler flags; handler flags открывают Handler RVA; Handler RVA ведет к runtime handler; HandlerData дает этому handler-у смысл текущей функции; handler возвращает решение dispatcher-у. Эта цепочка уже дает рабочую карту. Все остальное - детализация конкретных handler-ов.

Мини-карта для памяти
System layer: .pdata, RUNTIME_FUNCTION, UNWIND_INFO, RtlVirtualUnwind, DispatcherContext.
Language layer: language-specific handler, HandlerData, scope table, filters, catch, finally, destructors. Хороший анализ всегда держит оба слоя и не подменяет один другим.

Итог раздела

Language-specific handler - это функция, через которую Windows передает управление языковому или runtime-слою при обработке исключений. Windows отвечает за общий механизм: найти кадр, подготовить dispatcher context, вычислить establisher frame, передать exception record и context record. Handler отвечает за смысл: какие protected ranges относятся к исключению, какой filter выполнить, есть ли подходящий catch, какие cleanup-действия нужны при unwind.

В PE связь выглядит так: RUNTIME_FUNCTION ведет к UNWIND_INFO; flags UNWIND_INFO сообщают о наличии handler-а; после массива UNWIND_CODE лежит Handler RVA; после него начинается HandlerData. Но эти данные нельзя понимать универсально. Их формат задает конкретный handler.

Для reverse engineering это означает: handler metadata - не декоративный хвост .xdata, а реальный слой control flow. Он помогает восстановить, где функция может участвовать в search phase, где она может выполнять cleanup во время unwind phase, где скрыты filters, finally-блоки или C++ catch-логика. Для static /dynamic такой слой должен анализироваться аккуратно: с проверкой RVA, классификацией handler-а, осторожной интерпретацией HandlerData и явным разделением фактов, предположений и runtime-зависимых выводов.

6.3. Что такое __C_specific_handler и его роль

Теперь мы можем перейти от общей идеи language-specific handler к самому узнаваемому обработчику в мире Microsoft C SEH на x64 - __C_specific_handler. Это название часто встречается в дизассемблере, WinDbg, crash dump analysis и статьях по Windows internals. Новичок обычно видит его и думает: “ага, вот обработчик исключения”. Но такая мысль слишком грубая. __C_specific_handler действительно участвует в обработке исключений, но он не равен пользовательскому блоку __except, не равен catch из C++, не равен адресу падения и не является магической функцией, которая сама по себе чинит программу. Это служебный мост между системной x64 unwind-моделью Windows и конструкциями Structured Exception Handling языка C/MSVC.

Мы уже прошли несколько слоев. .pdata отвечает за поиск функции по адресу RIP. .xdata и UNWIND_INFO описывают, как функция изменила стек и где сохранила nonvolatile-регистры. DispatcherContext собирает для handler-а все важные сведения о текущем кадре. Language-specific handler объясняет Windows языковой смысл дополнительных данных. Теперь __C_specific_handler становится конкретным примером такого handler-а: он знает не все языки вообще, а прежде всего Microsoft-расширения C для структурированной обработки исключений: __try, __except и __finally.

> Как читать этот раздел
> Не нужно пытаться сразу запомнить все внутренние поля. Сначала держи простую линию: компилятор видит в C-коде __try/__except/__finally; затем он записывает в PE metadata; Windows при исключении доходит до кадра; __C_specific_handler читает эти metadata и решает, что делать. После этого уже можно аккуратно разбирать scope table, фильтры, JumpTarget и участие в search/unwind phase.

Название __C_specific_handler: что в нем спрятано

Начнем не с байтов, а с имени. В reverse engineering имя функции часто обманчиво: оно выглядит знакомо, но внутри может скрывать целую архитектуру. В имени __C_specific_handler важны все части. Два нижних подчеркивания в начале не означают “секретная функция для хакеров”. В мире Microsoft runtime и compiler helper functions такие имена часто используются для служебных функций, которые не предназначены как обычный красивый пользовательский API. Буква C указывает, что handler связан с C-специфичной моделью SEH. Слово specific говорит: это не универсальный обработчик всех исключений Windows. Слово handler означает: Windows вызывает эту функцию как обработчик кадра, когда UNWIND_INFO говорит, что у функции есть language-specific handler.

> __C_specific_handler (C-специфический обработчик Microsoft для SEH)
> Служебная функция, которую компилятор и runtime используют для реализации Structured Exception Handling в стиле Microsoft C на x64. Она получает от Windows сведения об исключении, текущем кадре и dispatcher context, затем читает C-специфические handler data, обычно связанные со scope table, и решает, продолжать поиск, выполнять filter, запускать __except-блок или выполнять __finally/cleanup при unwind.

compiler helper (вспомогательная функция компилятора)
Служебная функция, которую компилятор использует для реализации конструкции языка, которую нельзя или неудобно выразить одной машинной инструкцией. Программист может написать __try/__except в исходнике, но в машинном коде это превращается в набор metadata, переходов и вызовов runtime helper-а. __C_specific_handler как раз относится к такому уровню.

Важно не путать это с обычной функцией из твоего кода. Обычная функция вызывается потому, что программист написал call или компилятор сгенерировал call в обычном control flow. __C_specific_handler вызывается системой обработки исключений в особом контексте. Его вызывает не твоя функция напрямую как обычную библиотечную процедуру, а exception dispatcher, когда доходит до кадра, чьи unwind-данные говорят: “у этой функции есть C-специфическая логика обработки”.

Structured Exception Handling: что именно реализует этот handler

Чтобы понять __C_specific_handler, нужно спокойно разобрать SEH. Structured Exception Handling, или SEH, переводится как структурированная обработка исключений. Слово structured здесь означает, что обработка исключений привязана к структурным областям кода: есть защищенный участок, есть фильтр, есть обработчик, есть завершающий блок. Это не просто произвольный goto в случайное место. Компилятор видит эти области в исходном коде и описывает их в служебных таблицах.

Structured Exception Handling / SEH (структурированная обработка исключений)
Механизм Windows и Microsoft C/C++ для обработки исключительных ситуаций через структурные области кода. В C-расширениях MSVC это выражается через __try, __except и __finally. На x64 SEH не строится как старая x86-цепочка записей в стеке; он опирается на table-based model: .pdata, .xdata, UNWIND_INFO и language-specific handler.

__try (защищенный блок)
Расширение Microsoft C/C++, которое помечает участок кода как область, внутри которой может произойти исключение, требующее особой обработки. Сам по себе __try не ловит исключение. Он обозначает диапазон машинных инструкций, для которого compiler/runtime должны записать правила обработки.

__except (блок обработки с фильтром)
Конструкция C SEH, которая связывает защищенный блок с фильтром и пользовательским кодом обработки. Фильтр решает, подходит ли исключение, а тело __except выполняется только тогда, когда фильтр выбрал обработку. В x64 эта логика обычно проходит через __C_specific_handler и его handler data.

__finally (завершающий блок)
Конструкция C SEH, которая задает код, выполняемый при выходе из protected region, в том числе при исключении. __finally ближе к cleanup/termination handler, чем к catch. Его задача часто не “поймать” исключение, а корректно закрыть ресурс, разблокировать состояние или выполнить завершающее действие.

Теперь видно, зачем нужен специальный handler. Windows не может просто увидеть слово __try: в бинарном файле исходных слов уже нет. Ей нужны машинные сведения.

Компилятор должен превратить исходную структуру в диапазоны адресов, указатели на фильтры, адреса пользовательских блоков и служебные правила. __C_specific_handler - это тот runtime-код, который знает, как эти сведения читать.

Почему __C_specific_handler не является пользовательским обработчиком

Самая частая ошибка новичка: увидеть Handler RVA в UNWIND_INFO, прибавить ImageBase, получить адрес __C_specific_handler и сказать: “вот адрес обработчика”. Формально это адрес language-specific handler-а, но не адрес пользовательского тела __except. Пользовательский блок находится глубже: он задается через данные, которые __C_specific_handler читает.

> Handler RVA (RVA обработчика)
> Image-relative адрес, записанный в хвосте UNWIND_INFO, когда выставлены handler-флаги. Он указывает на language-specific handler, например на __C_specific_handler. Это не обязательно адрес пользовательского блока __except или __finally.

> user handler (пользовательский обработчик)
> Код, который программист воспринимает как “мой обработчик”: тело __except, тело __finally или другая логика обработки. На x64 этот код часто находится не в Handler RVA напрямую, а достигается через scope table и решения runtime handler-а.

Хорошая аналогия: __C_specific_handler - это не пожарный, который тушит конкретный пожар, а диспетчер внутри пожарной станции. Ему принесли карту здания, координаты проблемы и правила дежурства. Он смотрит: это наш участок или нет, какой фильтр надо спросить, какой выходной маршрут выбрать, какие комнаты надо пройти при эвакуации. Пользовательский __except-блок - это уже конкретная команда действий, до которой он может довести управление.

> Главная мысль
> Если в UNWIND_INFO указан __C_specific_handler, это доказывает наличие C SEH-совместимого runtime handler-а. Но чтобы понять конкретную обработку, нужно читать handler data: диапазоны, фильтры, JumpTarget, termination records и связи с текущим ControlPc.

Как компилятор превращает __try/__except в metadata

Представим функцию, где есть обычный код, затем __try, внутри risky operation, после него __except. В исходнике это выглядит почти как обычная структурная конструкция. Но процессор не знает слова __try. В машинном коде есть только инструкции, регистры, стек и переходы. Поэтому компилятор делает две работы одновременно. Во-первых, он генерирует обычный машинный код тела функции. Во-вторых, он генерирует metadata, которые описывают, какие адреса относятся к защищенным областям и какой runtime handler должен быть вызван при исключении.

```
Условный исходный смысл:

void f() {
__try {
```

```
risky_memory_access();
}
__except(filter(GetExceptionInformation())) {
handle_error();
}
}

В бинарном виде это превращается не в одну магическую инструкцию,
а в сочетание:

1) обычного машинного кода функции;
2) RUNTIME_FUNCTION в .pdata;
3) UNWIND_INFO в .xdata;
4) Handler RVA, который указывает на __C_specific_handler;
5) language-specific data, где описаны protected ranges,
filter-адреса и переходы к пользовательским блокам.
```

> metadata (метаданные)
> Данные о коде, а не сам исполняемый код. В нашем случае metadata говорят Windows и runtime: где начинается функция, как она меняла стек, есть ли у нее handler, где лежит таблица областей. Metadata не выполняются как инструкции процессора, но без них система не сможет надежно обработать исключение.

На x64 это особенно принципиально. Старый x86 SEH часто объясняют через linked list записей в стеке. Но x64 Windows сделала ставку на табличную модель. Это означает, что значительная часть правды о функции находится не только в дизассемблированном .text, но и в .pdata/.xdata. Для reverse engineer это подарок и ловушка одновременно. Подарок - потому что metadata дают структуру. Ловушка - потому что если читать только инструкции и игнорировать handler data, можно не увидеть реальную логику обработки.

Где именно появляется __C_specific_handler в цепочке .pdata -> .xdata

Путь начинается привычно. По RIP система находит RUNTIME_FUNCTION. Эта запись указывает на UNWIND_INFO. В начале UNWIND_INFO находятся version, flags, SizeOfProlog, CountOfCodes, FrameRegister и массив UNWIND_CODE. После unwind-кодов, с учетом выравнивания, может находиться дополнительный хвост. Если выставлен UNW_FLAG_EHANDLER или UNW_FLAG_UHANDLER, в этом хвосте лежит image-relative address handler-a. Для C SEH таким handler-ом часто является __C_specific_handler.

> UNW_FLAG_EHANDLER (флаг exception handler)
> Флаг в UNWIND_INFO, который говорит, что у функции есть exception handler, вызываемый системой для проверки и потенциальной обработки исключения. Для C SEH это связано с __try/__except.

> UNW_FLAG_UHANDLER (флаг unwind/termination handler)
> Флаг в UNWIND_INFO, который говорит, что у функции есть handler, вызываемый при unwind для выполнения завершающей логики. Для C SEH это связано с __finally, а для C++ - с cleanup вроде destructors.

После адреса handler-а могут идти language-specific handler data. Для Windows x64 общий формат этого хвоста намеренно не универсален. Система знает: “вот указатель на данные”. А что это за данные, должен понимать конкретный handler. Если handler - __C_specific_handler, то данные нужно читать как C SEH-специфическое описание областей. Если handler другой, например C++ frame handler, формат будет другим.

```
Упрощенная форма хвоста UNWIND_INFO при handler-флагах:

UNWIND_INFO header
UNWIND_CODE array
optional padding for alignment
ULONG HandlerRva ; например RVA на __C_specific_handler
variable HandlerData ; данные, понятные именно этому handler-у

Важно: HandlerData начинается не с фиксированного адреса,
а после массива UNWIND_CODE с учетом выравнивания.
```

Четыре параметра __C_specific_handler

Официальный прототип __C_specific_handler важен не для того, чтобы заучить типы, а чтобы увидеть архитектуру. Handler получает четыре крупных блока информации. Первый говорит, что случилось. Второй привязывает handler к конкретному кадру. Третий содержит машинное состояние. Четвертый приносит служебный dispatcher context, где есть указатели на FunctionEntry, ImageBase, HandlerData и другие сведения.

```
Упрощенный вид прототипа:

__C_specific_handler(
EXCEPTION_RECORD *ExceptionRecord,
void *EstablisherFrame,
CONTEXT *ContextRecord,
DISPATCHER_CONTEXT *DispatcherContext
);

Это не обычный пользовательский вызов.
Такой вызов делает системный exception dispatcher
в процессе search phase или unwind phase.
```

EXCEPTION_RECORD (запись об исключении)
Структура Windows, которая описывает само исключение: код исключения, адрес, флаги, дополнительные параметры. Для access violation там будет один смысл, для breakpoint - другой, для пользовательского RaiseException - третий. __C_specific_handler смотрит на нее не из любопытства, а потому что filter-логика может зависеть от типа исключения.

EstablisherFrame (кадр, установивший handler)
Runtime-адрес базы фиксированного стекового выделения для функции, чей handler сейчас вызван. В контексте __C_specific_handler это не “любой RBP” и не “адрес функции”. Это привязка C SEH-данных к конкретному активному стековому кадру.

CONTEXT record (запись машинного контекста)
Структура с состоянием регистров процессора. В exception-handler case она указывает на контекст в момент возникновения исключения. В termination-handler case она может отражать текущий unwind context. Поэтому нельзя механически считать, что ContextRecord всегда означает одно и то же состояние выполнения.

DISPATCHER_CONTEXT (контекст диспетчера исключений)
Служебная структура, которую Windows передает language-specific handler-у. В ней собраны ControlPc, ImageBase, FunctionEntry, EstablisherFrame, TargetIp, ContextRecord, LanguageHandler, HandlerData и другие сведения, нужные handler-у для принятия решения. Для __C_specific_handler особенно важен HandlerData, потому что через него handler получает C SEH-таблицы областей.

Параметр	Какую роль играет для __C_specific_handler
ExceptionRecord	Дает информацию о причине исключения. Фильтр __except может смотреть на код исключения, адрес и дополнительные параметры.
EstablisherFrame	Связывает обработку с конкретным кадром стека. Нужен для корректного доступа к frame-relative данным и для вызова filter/finally в правильном контексте.
ContextRecord	Показывает машинное состояние. В search phase это контекст исключения, а при unwind может быть текущий раскручиваемый контекст.
DispatcherContext	Передает системные сведения о текущей функции и указатель на HandlerData. Без него __C_specific_handler не знал бы, какую scope table читать.

Scope table: данные, которые делают __C_specific_handler полезным

Если остановиться на адресе __C_specific_handler, мы почти ничего не поймем. Важнейшая часть находится в HandlerData. Для C SEH это обычно таблица областей, которую удобно называть scope table. Именно она связывает диапазоны машинного кода с фильтрами и целевыми блоками обработки.

scope table (таблица областей)
C SEH-специфическая таблица, описывающая защищенные диапазоны кода и связанные с ними действия. Обычно она содержит количество записей и набор scope records. Каждая запись говорит: с какого адреса по какой адрес действует правило, где находится filter или termination handler, куда перейти при обработке.

scope record (запись области)
Одна запись внутри scope table. Она описывает один protected range и связанные с ним адреса. В типичной модели для x64 C SEH в записи встречаются поля уровня BeginAddress, EndAddress, HandlerAddress и JumpTarget. Названия могут быть представлены разными инструментами по-разному, но смысл остается: диапазон, проверка/handler, цель перехода.

BeginAddress / EndAddress (начало и конец защищенного диапазона)
Image-relative границы участка кода, к которому относится конкретное правило обработки. Если ControlPc не попадает в этот диапазон, данная scope record не подходит. EndAddress обычно удобно мыслить как границу после последней инструкции диапазона, то есть как полуоткрытый интервал.

HandlerAddress (адрес фильтра или завершающей логики)
Поле scope record, которое указывает на код, связанный с обработкой. Для __except это может быть filter function/expression. Для __finally - termination handler. Важно: это не всегда “тело обработчика”, поэтому его нужно интерпретировать в контексте конкретного типа записи.

JumpTarget (цель перехода к пользовательскому handler-коду)
Адрес, к которому управление должно прийти, когда принято решение выполнять __except-блок. Если filter сказал EXCEPTION_EXECUTE_HANDLER, runtime должен довести выполнение до соответствующего handler body. JumpTarget описывает эту цель в терминах metadata.

```
Концептуальная форма C SEH scope table:

struct ScopeTable {
ULONG Count;
ScopeRecord Records[Count];
};

struct ScopeRecord {
ULONG BeginAddress; // RVA начала protected range
ULONG EndAddress; // RVA конца protected range
ULONG HandlerAddress; // RVA filter/finally/служебной логики
ULONG JumpTarget; // RVA пользовательского __except-блока или 0
};

Это концептуальная модель для чтения.
Конкретные детали нужно сверять с handler-ом и компилятором.
```

Почему здесь нужна осторожность
UNWIND_INFO официально говорит только то, что есть language-specific handler data, формат которых определяется конкретным handler-ом. Поэтому parser должен сначала распознать handler как __C_specific_handler, а уже потом пытаться читать HandlerData как C SEH scope table. Иначе можно навязать чужому runtime неправильный формат.

Как __C_specific_handler работает в search phase

Теперь посмотрим на search phase живым языком. Исключение произошло внутри некоторой функции. Windows берет текущий ControlPc, находит RUNTIME_FUNCTION, раскручивает кадр, смотрит UNWIND_INFO. Если у функции нет handler-флага, спрашивать некого: система идет выше. Если handler-флаг есть и handler RVA указывает на __C_specific_handler, Windows вызывает его и передает четыре параметра.

__C_specific_handler в этот момент не обязан сразу хватать исключение. Он сначала должен понять: относится ли исключение к одному из protected ranges текущей функции. Для этого он берет ControlPc, смотрит scope table и ищет подходящую запись. Если ни одна запись не покрывает место исключения, этот кадр не является целью обработки. Тогда handler возвращает результат, означающий продолжить поиск.

ControlPc (адрес текущего управления)
Адрес инструкции, относительно которой система сейчас ищет обработку. В search phase это адрес, связанный с местом исключения или текущим кадром при проходе вверх. Для scope table он нужен как ключ: попадает ли этот адрес в BeginAddress..EndAddress конкретной protected region.

ExceptionContinueSearch (продолжить поиск)
Решение handler-а, означающее: этот кадр не берет исключение на себя, нужно продолжать search phase выше по стеку. Важно: вызов __C_specific_handler еще не означает, что исключение обработано.

Если подходящая scope record найдена, начинается следующий слой. Для __except нужно выполнить filter. Filter может смотреть на ExceptionRecord, ContextRecord, GetExceptionCode, GetExceptionInformation и собственную логику программы. Он может сказать: продолжать выполнение, продолжать поиск или выполнить handler. В этом месте впервые появляется решение, которое похоже на “ловим исключение”.

filter expression (фильтрующее выражение __except)
Код, который решает судьбу исключения для конкретного __except. Он может вернуть EXCEPTION_CONTINUE_SEARCH, EXCEPTION_CONTINUE_EXECUTION или EXCEPTION_EXECUTE_HANDLER. Filter - это пользовательская/сгенерированная логика, которую вызывает __C_specific_handler, а не сам системный dispatcher.

EXCEPTION_EXECUTE_HANDLER (выполнить обработчик)
Результат filter-а, означающий, что исключение должно быть принято этим __except-блоком. После такого решения система должна перейти от простого поиска к unwind-процессу, чтобы корректно покинуть промежуточные кадры и попасть в handler body.

```
Упрощенная логика search phase для C SEH:

__C_specific_handler receives DispatcherContext
scope_table = DispatcherContext->HandlerData

for each scope_record in scope_table:
if ControlPc not in [BeginAddress, EndAddress):
continue

if record describes __except:
result = call_filter(ExceptionRecord, ContextRecord, EstablisherFrame)

if result == EXCEPTION_CONTINUE_SEARCH:
continue searching

if result == EXCEPTION_CONTINUE_EXECUTION:
resume execution if exception is continuable

if result == EXCEPTION_EXECUTE_HANDLER:
choose this frame as target
arrange unwind/transfer to JumpTarget

return ExceptionContinueSearch
```

Этот псевдокод полезен именно как карта. Реальная реализация сложнее, но основные роли видны: scope table связывает адрес с правилом, filter принимает решение, JumpTarget указывает, куда попадет управление при обработке.

Почему filter - не то же самое, что handler body

В исходном коде __except часто выглядит так, будто есть одно место обработки. Но там две разные части. Первая часть - выражение в круглых скобках после __except. Это filter. Вторая часть - тело в фигурных скобках. Это handler body. Filter выполняется во время поиска, когда стек еще не обязательно раскручен до целевого кадра. Handler body выполняется после того, как принято решение обработать исключение и выполнены необходимые действия unwind.

handler body (тело обработчика)
Код внутри фигурных скобок __except. Это пользовательская логика, которая выполняется после принятия решения обработать исключение. Handler body не следует путать с __C_specific_handler и не следует путать с filter expression.

GetExceptionCode (получить код исключения)
Макрос/компиляторная конструкция Microsoft, доступная внутри filter expression или exception-handler block. Она возвращает код текущего исключения, например access violation, breakpoint или пользовательский код RaiseException. Для reverse engineering важно понимать, что filter может принимать решение не только по адресу, но и по ExceptionCode.

GetExceptionInformation (получить сведения об исключении)
Конструкция, которая дает filter-у доступ к EXCEPTION_POINTERS, то есть к связке EXCEPTION_RECORD и CONTEXT. Благодаря этому filter может смотреть не только код исключения, но и адрес, параметры, регистры и машинный контекст.

EXCEPTION_POINTERS (указатели на данные исключения)
Структура верхнего уровня, которая обычно содержит два указателя: на EXCEPTION_RECORD и на CONTEXT. Для C SEH это удобный способ дать filter-у доступ и к описанию исключения, и к состоянию процессора.

Это разделение принципиально для чтения дизассемблера. Если ты нашел адрес filter-а, это еще не адрес тела __except. Если ты нашел JumpTarget, это уже ближе к пользовательскому блоку обработки. Если ты нашел __C_specific_handler, ты нашел runtime-механизм, который связывает одно с другим.

Три результата filter-а и что они означают для потока управления

Filter не просто возвращает true или false. В C SEH у него есть три важных результата. Они похожи на ответы на три разных вопроса: “можно ли продолжить с места аварии?”, “это не мой случай?”, “да, это мой обработчик?”.

Результат	Смысл
EXCEPTION_CONTINUE_EXECUTION	Попытаться продолжить выполнение там, где произошло исключение. Это допустимо только для continuable exception. В реальном коде используется осторожно: если причина исключения не устранена, программа может снова упасть на той же инструкции.
EXCEPTION_CONTINUE_SEARCH	Этот __except не берет исключение. Search phase должна продолжиться выше по стеку. Для reverse engineer это означает: наличие scope record не равно успешной обработке.
EXCEPTION_EXECUTE_HANDLER	Исключение принято этим __except. Теперь нужно выполнить unwind до target frame и передать управление в handler body, связанный с JumpTarget.

continuable exception (продолжаемое исключение)
Исключение, после которого теоретически можно продолжить выполнение с места возникновения. Не все исключения продолжаемы. Если попытаться продолжить noncontinuable exception, это само по себе приводит к ошибке.

noncontinuable exception (непродолжаемое исключение)
Исключение, после которого система не разрешает вернуться к месту возникновения как ни в чем не бывало. Для таких исключений EXCEPTION_CONTINUE_EXECUTION не является нормальным выходом.

Для начинающего здесь важен простой вывод: __C_specific_handler не просто “ловит”. Он может сказать “не мое”, может подготовить выполнение handler body, а в некоторых случаях через filter может разрешить продолжить выполнение. Все зависит от C SEH-данных и результата filter-a.

Что происходит при EXCEPTION_EXECUTE_HANDLER

Когда filter возвращает EXCEPTION_EXECUTE_HANDLER, начинается переход от вопроса “кто обработает?” к вопросу “как безопасно туда добраться?”. Это как раз связь с предыдущим разделом про search phase и unwind phase. Нельзя просто прыгнуть в __except body из глубины стека. Между текущим местом и целевым кадром могут быть функции, которые должны быть корректно покинуты. У них могут быть __finally, C++ destructors, cleanup-логика и восстановление состояния.

target frame (целевой кадр)
Кадр стека, в котором найден обработчик, принимающий исключение. Если filter в этом кадре вернул EXCEPTION_EXECUTE_HANDLER, именно к нему нужно раскрутить стек перед передачей управления в handler body.

JumpTarget (цель перехода)
В контексте C SEH это адрес пользовательского __except-блока, к которому нужно передать управление после принятия решения обработать исключение и выполнения необходимого unwind. JumpTarget не должен восприниматься как простая “ветка if”: это цель в общей exception control flow.

На практическом уровне __C_specific_handler помогает связать найденную scope record с дальнейшим transfer of control. Он не существует отдельно от RtlVirtualUnwind, RtlUnwindEx и dispatcher-а. Система должна пройти unwind phase, вызвать нужные termination handlers на пути и восстановить управление так, чтобы execution продолжилось уже в обработчике.

> transfer of control (передача управления)
> Изменение текущего пути исполнения программы. В обычном коде это call, ret, jmp или conditional branch. В exception handling передача управления сложнее: она может включать поиск handler-а, раскрутку стека, cleanup промежуточных кадров и переход к handler body.

Как __C_specific_handler участвует в unwind phase и __finally

__C_specific_handler важен не только для __except. В C SEH есть еще __finally. Он не выбирает, обработать ли исключение по коду ошибки. Его смысл другой: “если мы покидаем эту область, выполни завершающий код”. При обычном выходе это одно поведение, при исключении - другое, но идея одна: не оставить ресурс в подвешенном состоянии.

> termination handler (завершающий обработчик)
> Код, который выполняется при покидании protected region. В C SEH это соответствует __finally. Он называется termination не потому, что завершает процесс, а потому что завершает жизненный участок конкретной области кода.

> AbnormalTermination (ненормальное завершение области)
> Признак, который показывает __finally-логике, была ли область покинута необычным путем, например из-за исключения или forced unwind. В исходном коде Microsoft C это связано с макросом AbnormalTermination(). Для анализа это важно: finally-блок может вести себя по-разному при обычном выходе и при исключении.

Во время unwind phase система проходит кадры и области, которые нужно покинуть. Если в функции есть termination handler, связанный с __finally, __C_specific_handler может быть вызван как часть unwind. В этом режиме ContextRecord уже может описывать не исходную точку падения, а текущий unwind context. Поэтому официальный прототип и говорит, что ContextRecord в exception handler case и termination handler case имеет разный практический смысл.

Для reverse engineering это означает: если ты видишь UHANDLER и __C_specific_handler, не спеши искать “catch”. Возможно, речь о cleanup. Тело __finally может быть важнее для поведения программы, чем сам факт исключения. Например, оно может освобождать critical section, откатывать флаг, закрывать handle, восстанавливать TLS-состояние или логировать ошибку.

Почему EstablisherFrame особенно важен именно для __C_specific_handler

В предыдущем разделе мы уже разобрали EstablisherFrame, но для __C_specific_handler стоит повторить смысл на новом уровне. Filter и finally-код часто должны работать так, будто они находятся внутри той самой функции, где написан __try. Им могут понадобиться локальные переменные, сохраненное состояние, адреса объектов и frame-relative данные. Для этого handler должен знать, с каким конкретным стековым кадром он связан.

Официальное описание __C_specific_handler говорит, что EstablisherFrame - это адрес базы фиксированного стекового выделения этой функции. Это очень точная формулировка. Не “RBP всегда”, не “начало всей stack memory”, не “адрес return address”. Это база fixed stack allocation, то есть точка, относительно которой runtime может корректно связать C SEH logic с локальным кадром функции.

fixed stack allocation (фиксированное стековое выделение)
Часть стекового кадра, размер которой известен из пролога и unwind-метаданных. Функция выделяет ее, например, через sub rsp, N. EstablisherFrame для __C_specific_handler связан именно с базой этого фиксированного выделения, а не с произвольным текущим RSP.

frame-relative access (доступ относительно кадра)
Способ обращаться к данным через смещение от базы кадра или frame pointer. Для handler-а это важно, потому что filter/finally могут использовать локальные переменные функции, а значит должны быть привязаны к правильному экземпляру кадра, а не просто к статическому адресу кода.

Ошибка понимания
Нельзя автоматически писать: EstablisherFrame = RBP. На x64 функция может вообще не использовать RBP как frame pointer. Даже если использует, смысл EstablisherFrame определяется unwind/runtime-моделью, а не одной привычкой из старых x86 объяснений.

Почему ContextRecord может означать разное в разных режимах

ContextRecord выглядит просто: это структура с регистрами. Но в обработке исключений простые слова часто опасны. Когда __C_specific_handler вызывается как exception handler во время поиска, ContextRecord связан с контекстом момента исключения. Filter может смотреть на RIP, RSP, регистры и параметры. Но когда handler вызывается как termination handler во время unwind, ContextRecord относится к текущему unwind-процессу.

exception context (контекст исключения)
Машинное состояние в момент, когда исключение было поднято. Например, RIP может указывать на инструкцию, где произошел access violation, а регистры показывают значения на момент аварии.

unwind context (контекст раскрутки)
Машинное состояние, с которым система работает во время прохождения unwind phase. Это уже не обязательно исходный момент падения; это текущая точка процесса раскрутки стека.

Если analyzer или человек в WinDbg смешивает эти два смысла, появляются странные выводы. Например, можно подумать, что filter видел регистры одного кадра, хотя на самом деле handler выполнялся в другом режиме. Или можно неправильно восстановить, почему finally-блок был вызван. Поэтому в книге мы постоянно держим дисциплину: один термин - один контекст применения, а если контекстов несколько, проговариваем их отдельно.

DISPATCHER_CONTEXT как рабочая папка для __C_specific_handler

DispatcherContext для __C_specific_handler можно представить как рабочую папку, которую Windows кладет на стол handler-у. В ней лежит не только “что случилось”, но и “где мы сейчас”, “какая функция сейчас рассматривается”, “какие metadata к ней относятся”, “где лежат handler data”. Без DispatcherContext __C_specific_handler не смог бы быть универсальной служебной функцией для многих разных функций.

> HandlerData (данные обработчика)
> Поле/указатель, через который handler получает дополнительные language-specific data из хвоста UNWIND_INFO. Для __C_specific_handler это путь к C SEH-описанию protected ranges и связанных действий.

> ImageBase (база образа)
> Адрес, по которому PE image загружен в память. Он нужен потому, что многие адреса в .pdata, .xdata и handler data являются RVA. Чтобы получить VA, runtime прибавляет ImageBase к RVA.

> RVA -> VA conversion (преобразование RVA в VA)
> Операция ImageBase + RVA. Для __C_specific_handler это важно, потому что scope table часто хранит image-relative адреса диапазонов, filter-ов и JumpTarget. Ошибка “прочитал RVA как абсолютный VA” ломает весь анализ.

```
Упрощенная картина для одного scope record:

ImageBase = 0x00007FF700000000
BeginAddress = 0x00001200 -> VA 0x00007FF700001200
EndAddress = 0x00001280 -> VA 0x00007FF700001280
HandlerAddress = 0x00001500 -> VA 0x00007FF700001500
JumpTarget = 0x00001620 -> VA 0x00007FF700001620

Если анализатор забудет прибавить ImageBase,
он будет искать код по адресам 0x1200, 0x1500, 0x1620,
что для загруженного процесса обычно бессмысленно.
```

Как это выглядит глазами reverse engineer

Теперь представим, что ты открыл PE в Ghidra, IDA, Binary Ninja или своем будущем static . Ты находишь функцию, у нее есть RUNTIME_FUNCTION, затем UNWIND_INFO. В flags стоит EHANDLER. После unwind-кодов лежит Handler RVA. Если символы доступны, инструмент может подписать адрес как __C_specific_handler. Что это значит практически?

Во-первых, эта функция, вероятно, содержит C SEH-области или связанную termination-логику. Во-вторых, пользовательский обработчик не равен адресу __C_specific_handler. В-третьих, нужно найти HandlerData и попытаться разобрать scope

table. В-четвертых, для каждой записи нужно проверить диапазоны, фильтр, JumpTarget и попадание ControlPc. Только после этого можно говорить, какой блок кода связан с каким исключением.

Ghidra / IDA / Binary Ninja (инструменты статического анализа)
Программы, которые помогают дизассемблировать и анализировать бинарные файлы. Они могут показать функции, ссылки, структуры, pseudo-code и иногда unwind metadata. Но даже хороший инструмент не освобождает reverse engineer от понимания модели: если ты не понимаешь __C_specific_handler, ты можешь неправильно прочитать показанный тебе результат.

static analysis (статический анализ)
Анализ файла или кода без запуска программы. Для __C_specific_handler это чтение PE, .pdata, .xdata, UNWIND_INFO и handler data. Статический анализ может многое увидеть, но не всегда знает runtime-значения, результаты filter-а и реальные пути исполнения.

dynamic analysis (динамический анализ)
Анализ во время выполнения программы: debugger, breakpoints, трассировка, наблюдение за регистрами и стеком. Для __C_specific_handler dynamic analysis полезен тем, что можно остановиться на handler-е и увидеть реальные ExceptionRecord, EstablisherFrame, ContextRecord и DispatcherContext.

Мини-алгоритм ручного разбора __C_specific_handler

Для практики полезно иметь не просто знание терминов, а аккуратный порядок действий. Но это не “магический чеклист”, а способ не потерять слои. Каждый шаг отвечает на свой вопрос и не должен подменять следующий.

Шаг	Что проверяем
1. Найти RUNTIME_FUNCTION	По адресу функции или ControlPc найти запись в .pdata / active function tables. Без этого мы не привязаны к x64 unwind-модели.
2. Перейти к UNWIND_INFO	Прочитать UnwindInfoAddress как RVA и найти .xdata. Проверить version, flags, CountOfCodes, alignment.
3. Проверить handler flags	Если нет EHANDLER/UHANDLER, __C_specific_handler из этой UNWIND_INFO не вызывается. Если есть CHAININFO, правила другие.
4. Найти Handler RVA	После массива UNWIND_CODE и padding прочитать address of exception handler. Преобразовать RVA в VA.
5. Распознать handler	Сравнить адрес с символами, импортом, known runtime function, pattern или debugger name. Не угадывать C SEH без оснований.
6. Найти HandlerData	Установить, где начинается хвост данных после Handler RVA. Для __C_specific_handler читать его как scope table.
7. Проверить scope records	Диапазоны должны быть осмысленными, адреса должны попадать в image/code regions, JumpTarget должен быть достижимым, filter/finally должны соответствовать ожидаемой логике.

Шаг	Что проверяем
8. Связать с ControlPc	Для конкретного исключения важно не просто наличие записи, а попадание ControlPc в protected range.

Дисциплина анализа
Нельзя перескакивать от шага 4 сразу к выводу “функция ловит access violation”. Handler RVA говорит только о механизме. Реальное решение зависит от scope table, filter-a, ExceptionCode, runtime state и режима search/unwind.

Типичные ошибки при чтении __C_specific_handler

Ошибки вокруг __C_specific_handler обычно рождаются из желания упростить модель. Но эта тема как раз опасна тем, что слишком простая модель дает уверенные, но ложные выводы. Разберем несколько ошибок не списком ради списка, а как реальные ловушки анализа.

Первая ошибка - считать, что __C_specific_handler является телом __except. На самом деле это runtime helper. Он может быть один и тот же для множества функций, а конкретика лежит в данных. Если в двух разных функциях Handler RVA ведет к одному и тому же __C_specific_handler, это не значит, что они обрабатывают исключения одинаково. У них разные scope tables.

Вторая ошибка - считать, что наличие __C_specific_handler гарантирует обработку любого исключения. Нет. Filter может вернуть ContinueSearch. Protected range может не покрывать текущий ControlPc. JumpTarget может относиться к другому диапазону. У функции может быть только termination handler. Поэтому правильнее говорить: функция имеет C SEH machinery, а не “ловит все”.

Третья ошибка - читать HandlerData как универсальную структуру для всех handler-ов. Это особенно плохо в dynamic /static , потому что инструмент начнет выдавать красивые, но фальшивые findings. Если handler не распознан как __C_specific_handler, нельзя насильно интерпретировать хвост как C SEH scope table.

Четвертая ошибка - путать RVA и VA. В PE и unwind metadata многие адреса image-relative. В runtime они превращаются в реальные виртуальные адреса через ImageBase. Если пропустить этот шаг, таблица будет выглядеть мусором или, что хуже, частично правдоподобным мусором.

Пятая ошибка - забывать про режим вызова. __C_specific_handler может участвовать и в search phase, и в unwind phase. В одном случае он помогает решить, кто обработает исключение. В другом - помогает выполнить termination/cleanup при раскрутке. Один и тот же handler name не означает один и тот же смысл вызова.

Что может насторожить в защищенном или обфусцированном бинаре

Для обычной программы __C_specific_handler и C SEH metadata часто выглядят скучно и закономерно. Но для reverse engineering и защитных проектов важны не только нормальные случаи, но и аномалии. Обфускатор, packer, dynamic или ручной assembler-код могут использовать exception machinery как часть control flow, антиотладки или скрытия настоящих переходов.

> packer (упаковщик)
> Инструмент, который преобразует бинарь, часто сжимая, шифруя или пряча исходный код программы за загрузчиком-распаковщиком. Packer может ломать привычную структуру PE, генерировать необычные sections и мешать статическому анализу.

> dynamic (защитный инструмент)
> Система, которая усложняет анализ программы: обфускация, виртуализация, антиотладка, integrity checks, нестандартный control flow. Exception handling может использоваться dynamic-ом как скрытый механизм переходов или как способ реагировать на debug events.

> obfuscation (обфускация)
> Искусственное усложнение программы без изменения ее внешнего поведения. В контексте __C_specific_handler это может быть странная scope table, необычные protected ranges, handler data, которые выглядят почти валидно, или намеренное использование исключений как части обычного control flow.

Подозрительными могут быть scope records, чьи диапазоны выходят за пределы функции; HandlerAddress или JumpTarget указывают в нетипичную секцию; HandlerData находится в writable/executable области; таблица содержит перекрывающиеся диапазоны без понятной вложенности; фильтры ведут в код, который похож на dispatcher/VM-loop; JumpTarget используется как скрытый переход между далекими блоками. Все это не автоматически “вредоносность”, но это хорошее evidence для дальнейшего анализа.

> evidence (доказательный признак)
> Наблюдение, которое само по себе еще не является финальным выводом, но повышает или снижает уверенность. Например, “UNWIND_INFO указывает на __C_specific_handler” - evidence C SEH machinery. “JumpTarget ведет в executable section вне ожидаемой функции” - evidence аномального control flow.

> confidence (уверенность вывода)
> Оценка того, насколько вывод анализатора обоснован. Хороший static не должен писать “malware” только потому, что увидел __C_specific_handler. Он должен разделять факты, вероятные интерпретации и гипотезы.

Как это переносится в static /dynamic

Если переносить эту тему в твой static /dynamic , __C_specific_handler должен стать не просто строкой в отчете, а узлом смысловой модели. Инструмент должен уметь сказать: у функции есть handler flags; Handler RVA указывает на распознанный C-specific handler; HandlerData начинается здесь; scope table выглядит валидно или подозрительно; protected ranges пересекаются с function range; filter/Jumptarget указывают туда-то; уровень уверенности такой-то.

Самое важное - не смешивать уровни. Parser level читает байты и структуры. Semantic level пытается понять SEH-смысл. Risk level оценивает аномальность. Report level объясняет человеку, что найдено. Если все смешать, отчет станет либо слишком сухим, либо слишком уверенным. Хороший анализатор должен писать как хороший преподаватель: “вот факт, вот почему он значит именно это, вот где граница вывода”.

Уровень static -а	Что он должен делать с __C_specific_handler
Parser	Корректно найти handler RVA, HandlerData, scope table, записи и RVA-поля. Проверить выравнивание и границы.
Validator	Проверить, что диапазоны и адреса попадают в ожидаемые области image, не выходят за function range без причины, не указывают в мусор.
Semantic analyzer	Распознать C SEH pattern: protected ranges, filters, JumpTarget, finally/termination patterns. Разделить __except и __finally cases.
Risk analyzer	Отметить аномалии: странные секции, перекрытия, невозможные адреса, executable+writable handler data, hidden control flow.
Report	Объяснить человеку не только “найден __C_specific_handler”, а что это означает для функции и почему вывод имеет такой уровень уверенности.

Формулировка для отчета
Плохой отчет: “Function has __C_specific_handler”. Хороший отчет: “Function has UNW_FLAG_EHANDLER; Handler RVA resolves to __C_specific_handler; HandlerData decodes as C SEH scope table with N records; record #1 protects RVA range A..B and jumps to handler body C if filter D returns EXCEPTION_EXECUTE_HANDLER”.

Связь с C++ exceptions: почему не надо все сваливать в одну кучу

Раз мы говорим о C-specific handler, нужно аккуратно отделить его от C++ exception handling. В C++ есть catch, destructors, type matching, object lifetime, RTTI-like structures и своя runtime-модель. Она тоже живет поверх x64 unwind, тоже использует language-specific handler, тоже может иметь cleanup, но конкретные handler data и правила будут другими.

C++ frame handler (кадровый обработчик C++ исключений)
Runtime handler, который понимает C++ exception model: типы исключений, catch-блоки, разрушение объектов и состояние функции. Он не должен автоматически отождествляться с __C_specific_handler, даже если оба механизма используют x64 unwind infrastructure.

type matching (сопоставление типа исключения)
C++-специфическая логика: catch-блок подходит или не подходит в зависимости от типа брошенного объекта. В C SEH __except filter обычно работает иначе: он возвращает числовое решение на основе exception information.

Для начинающего хорошее правило такое: x64 unwind - общий фундамент; language-specific handler - мост к конкретному языку; __C_specific_handler - мост к C SEH; C++ frame handler - мост к C++ exceptions. Если держать эти уровни отдельно, тема перестает быть кашей.

Почему __C_specific_handler важен для crash dump analysis

В crash dump analysis __C_specific_handler помогает понять, почему программа не просто упала, а пошла определенным путем обработки. Если стек содержит кадры с C SEH, dump может показать не только место исключения, но и функции, которые имели protected regions, filters или finally. Это особенно важно, когда исключение было поймано, преобразовано, залогировано или скрыто.

crash dump (дамп аварийного состояния)
Снимок процесса в момент сбоя или другого важного события. Он может содержать память, стек, регистры, список модулей и иногда дополнительные сведения. Для x64 unwind dump analysis зависит от корректных .pdata/.xdata и handler metadata.

WinDbg (отладчик Windows)
Отладчик Microsoft, часто используемый для анализа crash dumps, kernel dumps и live debugging. В контексте __C_specific_handler он полезен тем, что может показать стек, модули, символы, exception record, context и иногда перейти к runtime handler-ам.

Если dump показывает, что исключение прошло через __C_specific_handler, это повод спросить: какой filter был активен? Что он вернул? Был ли выполнен __finally? Не была ли ошибка проглочена и заменена другой? В production debugging такие вопросы иногда важнее, чем сам факт access violation.

Нормальный мысленный образ

Теперь можно собрать окончательную картину. __C_specific_handler - это не страшная черная магия. Это специальный читатель C SEH-таблиц, которого Windows вызывает через общий x64 exception/unwind протокол. Он стоит между сухой системной механикой и смыслом конструкций __try, __except, __finally.

Снизу у нас процессор, стек, RIP и регистры. Над этим - Windows unwind: .pdata, RUNTIME_FUNCTION, .xdata, UNWIND_INFO, RtlLookupFunctionEntry, RtlVirtualUnwind. Над этим - exception dispatcher, который проходит search phase и unwind phase. Над этим - language-specific handler, которому система передает кадр и данные. А __C_specific_handler - конкретный представитель этого слоя для Microsoft C SEH.

```
Иерархия смысла:

Machine state:
RIP, RSP, registers, stack memory
```

```
Windows x64 unwind infrastructure:
.pdata -> RUNTIME_FUNCTION -> .xdata -> UNWIND_INFO

Dispatcher layer:
search phase, unwind phase, DispatcherContext

Language-specific layer:
__C_specific_handler

C SEH language data:
scope table, protected ranges, filters, JumpTarget, finally

User-visible constructs:
__try, __except, __finally
```

> Формула понимания
> __C_specific_handler отвечает не на вопрос “как восстановить стек?” - это делает unwind-механика. И не на вопрос “что хотел программист?” - это видно только через scope table и пользовательский код. Он отвечает на промежуточный вопрос: как применить C SEH-правила к текущему кадру, исключению и dispatcher context.

Медленный walkthrough: что происходит при исключении внутри __try

Чтобы тема не осталась набором терминов, пройдем один сценарий почти как глазами системы. Представим, что внутри функции f есть __try-блок, а внутри него происходит access violation. На уровне процессора это просто аварийная ситуация при обращении к памяти. Но на уровне Windows начинается дисциплинированный процесс: создается exception record, фиксируется context record, затем dispatcher начинает искать, есть ли в стеке кадр, готовый принять это исключение.

> access violation (нарушение доступа к памяти)
> Исключение Windows, возникающее при недопустимом чтении, записи или исполнении памяти. Для C SEH это обычный пример исключения, которое может попасть в __except filter. Но сам факт access violation еще не означает, что текущий __except обязан его обработать.

Сначала dispatcher работает не с исходником, а с адресом. У него есть ControlPc, то есть адрес, связанный с местом исключения. По этому адресу через RtlLookupFunctionEntry находится RUNTIME_FUNCTION. Через нее система выходит на UNWIND_INFO. Если в flags нет EHANDLER/UHANDLER, кадр не имеет language-specific логики, и система идет выше. Если флаг есть, из хвоста UNWIND_INFO читается Handler RVA. Допустим, он указывает на __C_specific_handler.

Теперь __C_specific_handler получает DispatcherContext. В нем находится HandlerData. Этот указатель ведет к данным C SEH. Handler начинает не с “выполнить обработчик”, а с более скромного действия: он ищет, есть ли scope record, whose protected range covers current ControlPc. Если ControlPc лежит вне всех диапазонов, текущая функция может иметь SEH вообще, но не для данного места исключения. Тогда поиск продолжается.

Если подходящий диапазон найден, handler смотрит связанную запись. Если это __except, нужно выполнить filter. Filter получает доступ к данным исключения и возвращает решение. Только при EXCEPTION_EXECUTE_HANDLER текущий кадр становится target frame. После этого нельзя просто сделать jmp в JumpTarget. Нужно пройти unwind phase: покинуть промежуточные кадры, выполнить termination handlers, восстановить нужное состояние и только потом передать управление в handler body.

```
Один и тот же __try/__except с точки зрения разных слоев:

Исходник:
__try { risky(); }
__except(filter()) { recover(); }

PE/unwind слой:
UNWIND_INFO has UNW_FLAG_EHANDLER
HandlerRva -> __C_specific_handler
HandlerData -> scope table

Scope table слой:
protected range: risky-code RVA range
HandlerAddress: filter RVA
JumpTarget: recover-block RVA

Runtime слой:
call filter
if filter accepts:
unwind to target frame
transfer control to JumpTarget
```

> Зачем такой медленный разбор
> Пока это не разложено по слоям, все выглядит как магия. После разложения видно: __C_specific_handler не заменяет .pdata, не заменяет RtlVirtualUnwind и не заменяет пользовательский __except. Он связывает C SEH data с системной фазой поиска и раскрутки.

Вложенные __try-блоки и почему scope table может быть неочевидной

В реальном коде __try-блоки могут быть вложенными. Это значит, что один protected range находится внутри другого. Для исходника это выглядит естественно: внутри внешнего try есть внутренний try. Для binary analysis это превращается в несколько scope records, диапазоны которых могут пересекаться или быть вложенными. Новичок легко пугается: “диапазоны пересекаются, значит таблица битая”. Нет, не всегда. Пересечение может быть нормальной вложенностью языковых областей.

> nested scope (вложенная область)
> Ситуация, когда одна protected region находится внутри другой. В исходном коде это соответствует вложенным __try или комбинации __try/__finally внутри другого protected block. В metadata это может выглядеть как несколько scope records с перекрывающимися диапазонами.

enclosing scope (охватывающая область)
Внешняя область, которая содержит внутреннюю. Для exception analysis важно понимать, какая область ближе к месту исключения, потому что ближайший подходящий filter может получить шанс первым.

Для __C_specific_handler это значит, что простая проверка “найди первую запись, где ControlPc между BeginAddress и EndAddress” может быть недостаточной как универсальное объяснение. Реальный runtime должен учитывать порядок записей, вложенность и правила конкретной реализации. Для человека на начальном уровне важнее другое: scope table не просто список случайных адресов. Это закодированная карта областей исходного языка. Если области вложены в исходнике, в таблице тоже будет видна вложенная структура.

Инструмент анализа должен показывать такие записи аккуратно. Не “overlap = error”, а “overlap detected; likely nested protected ranges” или “overlap not consistent with recognized nesting pattern”. То есть снова нужна граница между фактом, интерпретацией и подозрением.

MASM и ручной assembler: как __C_specific_handler может появиться без C-исходника

Еще одна важная деталь для reverse engineering: наличие __C_specific_handler не доказывает, что исходная функция была написана именно на C с красивым __try. Такие metadata можно создать и через assembler directives. В x64 MASM есть FRAME-механика и директивы для генерации .pdata/.xdata. Это нужно нормальным разработчикам низкоуровневого кода, но этим же может пользоваться человек, который вручную строит необычную exception-модель.

MASM (Microsoft Macro Assembler)
Ассемблер Microsoft. На x64 он поддерживает специальные директивы, которые помогают описывать пролог, сохранение регистров, stack allocation и exception handler metadata, чтобы Windows unwind могла работать с hand-written assembly.

FRAME directive / PROC FRAME (директива кадра)
MASM-механизм, который говорит assembler-у генерировать function table entry в .pdata и unwind information в .xdata. Если указан ehandler, этот handler может быть записан как language-specific handler.

.seh_handler / .seh_handlerdata (директивы SEH-метаданных)
Ассемблерные директивы, используемые для описания handler-а и данных handler-а в x64 SEH/unwind metadata. Они позволяют связать hand-written assembly function с language-specific handler, например __C_specific_handler, и записать соответствующие handler data.

```
Концептуальный вид hand-written assembly с SEH metadata:

myfunc PROC FRAME
sub rsp, 40h
.allocstack 40h
.endprolog
```

```
; body code

.seh_handler __C_specific_handler, @except
.seh_handlerdata
; scope table-like data goes here
myfunc ENDP

Это не готовый рецепт для копирования.
Смысл в том, что handler metadata может быть создана явно,
а не только автоматически из C-исходника.
```

Для dynamic /static это означает: “compiler generated” и “SEH metadata present” - не одно и то же. В большинстве обычных программ такие структуры действительно сгенерированы компилятором. Но в защищенном коде или hand-written assembly они могут быть собраны вручную. Поэтому отчет должен писать осторожно: “metadata is consistent with MSVC C SEH” вместо слишком уверенного “это точно исходный __try”.

Отличие от VEH и top-level filter: похожие слова, разные этажи

Когда человек только входит в Windows exceptions, все слова handler начинают сливаться. __C_specific_handler, VEH, top-level exception filter, unhandled exception filter, __except handler - кажется, что это одно семейство и все делает одно и то же. На деле это разные этажи. __C_specific_handler является frame-based language-specific handler-ом. Он привязан к конкретному кадру через UNWIND_INFO и DispatcherContext. VEH и top-level filter работают иначе.

> VEH / Vectored Exception Handling (векторная обработка исключений)
> Механизм Windows, где обработчики регистрируются глобально для процесса через vectored handler list и вызываются до или независимо от frame-based поиска. VEH не привязан к конкретному RUNTIME_FUNCTION и scope table так, как __C_specific_handler.

> top-level exception filter (верхнеуровневый фильтр необработанных исключений)
> Обработчик последнего уровня, который может быть вызван, когда исключение не было обработано обычными механизмами. Он не является тем же самым, что C SEH __except filter внутри конкретной функции.

__C_specific_handler живет там, где есть функция, кадр, UNWIND_INFO, handler flags и language-specific data. VEH живет на уровне процесса как список callbacks. Top-level filter находится ближе к финальному сценарию необработанного исключения. Для анализа важно не просто найти слово handler, а понять, какой механизм его вызывает, какие данные он получает и к какому уровню стека он привязан.

Почему оптимизации компилятора усложняют чтение scope table

В учебном примере все красиво: один __try, один filter, один handler body. В optimized binary все может выглядеть менее романтично. Компилятор может объединять блоки выхода, переставлять код, делать tail merge, выносить холодные обработчики в отдельные участки, использовать общие epilog blocks, инлайнить функции и менять форму control flow. Metadata при этом должна оставаться корректной, но ее визуальная связь с исходником становится менее прямой.

> optimization (оптимизация компилятора)
> Преобразование кода ради скорости, размера или других свойств без изменения наблюдаемого поведения программы. Для exception handling оптимизации могут менять расположение блоков, объединять переходы и затруднять прямую реконструкцию исходного __try/__except.

> cold code (холодный код)
> Код, который редко выполняется. Обработчики исключений часто относятся к cold path, поэтому компилятор может располагать их отдельно от горячего основного пути функции. В дизассемблере это может выглядеть как “кусок функции далеко в стороне”.

> tail merge (слияние хвостов)
> Оптимизация, при которой одинаковые завершения разных веток объединяются в один общий блок. Для exception handling это может делать control flow менее похожим на исходный текст, хотя metadata остается связующей картой.

Поэтому reverse engineer не должен требовать от бинаря школьной красоты. JumpTarget может вести в блок, который визуально расположен далеко. Handler body может не следовать сразу после protected range. Несколько записей могут использовать общую завершающую логику. Это не сразу ошибка. Сначала нужно проверить, согласуется ли такая форма с compiler optimization и unwind/SEH metadata.

Что именно показывать новичку в отчете, чтобы не запутать

Если твой static будет выводить __C_specific_handler для начинающего пользователя, важно не перегрузить и не обмануть. Сухой вывод “__C_specific_handler found” почти бесполезен. Слишком смелый вывод “function catches exceptions” может быть ложным. Хороший вывод должен объяснять слой за слоем.

```
Пример понятного отчета:

Function: sub_140012000
Unwind flags: UNW_FLAG_EHANDLER
Language handler: __C_specific_handler
HandlerData: RVA 0x0002A430
Decoded as: C SEH scope table
Scope records: 2

Record 0:
Protected range: 0x140012050 .. 0x1400120A0
Filter: 0x140012300
JumpTarget: 0x140012380
Meaning: possible __except region
```

```
Record 1:
Protected range: 0x1400120B0 .. 0x140012110
Termination: 0x140012420
Meaning: possible __finally/cleanup region

Confidence: high for C SEH metadata,
medium for recovered source-level structure.
```

Такая подача хороша тем, что она не притворяется исходником, но дает человеку карту. Он видит, что доказано из metadata, что распознано как C SEH, а где остается реконструкция. Именно так и должен работать фундаментальный reverse engineering: не угадывать намерения, а аккуратно собирать доказательства.

Связь с безопасностью: почему злоупотребляют exception handling

Exception handling изначально нужен для надежности, но любой мощный механизм control flow можно использовать странно. Если программа намеренно вызывает исключения, а затем через __C_specific_handler и scope table переводит управление в разные блоки, обычный линейный disassembly может выглядеть обманчиво. Некоторые защитные техники используют исключения как скрытый dispatcher: вместо прямого jmp происходит исключение, затем runtime-механика выбирает путь.

> exception-driven control flow (управление потоком через исключения)
> Прием, при котором исключения используются не только для аварийных ситуаций, а как часть обычного или намеренно запутанного control flow. Это может быть легитимная техника восстановления, антиотладочный прием или элемент обфускации.

> anti-debugging (антиотладка)
> Приемы, которые мешают работе debugger-а или меняют поведение программы под debugger-ом. Исключения часто используются в антиотладке, потому что debugger участвует в first-chance/second-chance обработке и может менять наблюдаемый путь выполнения.

Но здесь тоже нельзя впадать в паранойю. Обычный C/C++ код может использовать SEH легитимно. Наличие __C_specific_handler не равно вредоносности. Подозрительным становится сочетание признаков: странные ranges, неожиданные handler data, executable writable sections, намеренные access violations, breakpoints, фильтры с проверками debugger-а, необычные JumpTarget и несовпадение metadata с дизассемблированной структурой.

Итог раздела

__C_specific_handler - один из тех терминов, которые легко запомнить по имени и трудно правильно понять по роли. Его нельзя воспринимать как тело обработчика. Нельзя воспринимать как C++ catch. Нельзя считать, что он автоматически обрабатывает любое исключение. Нельзя читать его HandlerData без проверки формата. Правильнее видеть в нем runtime helper, который реализует C SEH поверх table-based x64 exception handling.

Для reverse engineer ценность __C_specific_handler огромна. Он показывает, что перед нами не просто набор инструкций, а функция с описанной системой protected ranges и

возможных переходов при исключениях. Если анализатор умеет читать этот слой, он видит control flow, который не всегда очевиден из линейного дизассемблера. Если не умеет - он пропускает часть поведения программы.

На следующем шаге мы сможем подняться еще выше и спокойно ответить на более широкий вопрос: что такое SEH на x64 в реальном смысле. Теперь это уже не будет абстрактное “обработка исключений Windows”. Мы знаем, что за этим стоят таблицы функций, unwind metadata, фазы диспетчеризации, language-specific handler-ы и конкретный пример __C_specific_handler.

6.4. Что такое SEH на x64 в реальном смысле

К этому месту книги мы уже успели разобрать много деталей: как PE-файл хранит таблицы функций, как .pdata ведет к .xdata, как RUNTIME_FUNCTION задает диапазон кода, как UNWIND_INFO описывает пролог, как RtlLookupFunctionEntry находит запись по RIP, как RtlVirtualUnwind восстанавливает предыдущий кадр, и как language-specific handler получает DispatcherContext. Теперь важно остановиться и собрать эти детали в одно целое. Иначе слово SEH будет казаться знакомым, но внутренне пустым: вроде бы это “обработка исключений”, но где она реально живет на x64, кто ее исполняет, чем она отличается от x86, чем отличается от VEH и почему в дизассемблере она выглядит не так, как ожидает начинающий.

Раздел специально будет идти медленно. SEH - одна из тех тем, где человек часто запоминает одно внешнее слово, а потом годами путает уровни: синтаксис __try, системную диспетчеризацию, таблицы unwind, language-specific handler, vectored handler, top-level filter и C++ exceptions. В исходном коде все может выглядеть как простая конструкция из нескольких строк. В бинарном файле это превращается в несколько слоев: машинный код, таблицы .pdata, записи .xdata, указатель на runtime-handler, служебные таблицы областей, фильтры, адреса перехода и системные вызовы диспетчера исключений. Именно это и есть “реальный смысл” SEH на Windows x64.

> SEH (Structured Exception Handling, структурированная обработка исключений) - это механизм Windows и Microsoft C/C++ для обработки исключительных ситуаций, в том числе аппаратных ошибок вроде access violation и программных исключений, вызванных RaiseException. Слово “structured” здесь означает не “красиво оформленный код”, а то, что обработка привязана к структуре выполнения программы: к защищенным областям кода, кадрам функций, фазам поиска и раскрутки, а не к хаотическим goto-переходам.

Если совсем грубо, SEH отвечает на вопрос: что делать, когда выполнение кода внезапно стало невозможным обычным путем? Например, программа обратилась по неверному адресу памяти, процессор сгенерировал исключение, Windows получила состояние потока, а дальше нужно решить: можно ли передать управление какому-то обработчику, нужно ли продолжать поиск, нужно ли разматывать стек, нужно ли вызвать finally-блоки, нужно ли дать шанс отладчику или завершить процесс. Но грубое объяснение быстро ломается, потому что на x64 SEH - не одна функция и не одна таблица. Это договор между компилятором, линкером, загрузчиком, системным runtime, обработчиками языка и самим кодом программы.

Почему после __C_specific_handler нужно отдельно объяснять SEH

В предыдущем разделе мы разобрали __C_specific_handler. Там была важная мысль: __C_specific_handler - это не тело пользовательского __except, а служебный обработчик, который компилятор использует для реализации C SEH. Но если остановиться на этом месте, останется ложное ощущение, будто SEH и есть __C_specific_handler. Это не так. __C_specific_handler - только один возможный участник общей системы. Он обслуживает конкретный вариант SEH, созданный Microsoft C/C++ компилятором для конструкций

__try, __except и __finally. Но сама идея structured exception handling шире.

Чтобы понять это без путаницы, удобно смотреть на механизм слоями. Самый нижний слой - процессор и операционная система: возникла исключительная ситуация, нужно сохранить контекст и начать диспетчеризацию. Следующий слой - Windows exception dispatcher: он идет по возможным обработчикам и вызывает их по правилам. Следующий слой - x64 unwind: чтобы идти по кадрам, системе нужны .pdata и .xdata. Еще выше - language-specific handler: он понимает данные конкретного языка или компилятора. И только сверху находится то, что программист видит в исходном коде: __try, __except, __finally, try/catch, деструкторы, cleanup-блоки.

> exception dispatcher (диспетчер исключений) - это системная логика Windows, которая получает исключение и организует его обработку: дает шанс отладчику, вызывает vectored handlers, ищет frame-based обработчик, запускает unwind phase и в конце решает, обработано исключение или нет. Это не один “магический обработчик” в твоей программе, а часть системного пути обработки исключения.

На x64 этот диспетчер не может просто взять стек и найти там цепочку обработчиков, как это исторически делалось в x86-модели. Поэтому x64 SEH почти невозможно правильно понять без всей предыдущей главы про unwind. Таблицы функций, прологи, эпилоги, сохраненные nonvolatile-регистры, EstablisherFrame и DispatcherContext здесь не второстепенные детали. Это каркас, на котором держится возможность найти обработчик и корректно дойти до него.

SEH - это не только синтаксис __try / __except

Начинающий часто впервые встречает SEH через код на C или C++ для Windows. Он видит __try, __except и __finally, а потом естественно думает: “Ага, SEH - это такие специальные блоки в C”. Для учебного старта это нормально, но для reverse engineering такое понимание опасно. В бинарном файле ты не увидишь красивую надпись __try. Ты увидишь обычные инструкции, секции .pdata и .xdata, флаги в UNWIND_INFO, адрес __C_specific_handler, и какие-то данные handler-а рядом с unwind-информацией. Если знать только синтаксис, этот набор будет выглядеть как бессмысленный мусор.

> __try (защищенный блок SEH) - это Microsoft-specific конструкция исходного кода, которая говорит компилятору: участок кода нужно рассматривать как protected range, то есть область, для которой может существовать обработка исключения или завершения. В бинарном файле __try не остается как отдельная инструкция; компилятор превращает его в машинный код плюс служебные записи, которые затем читает runtime-handler.

> __except (блок фильтра и обработки SEH) - это часть C SEH, которая задает условие обработки исключения. В исходном коде он похож на “catch”, но работает иначе: сначала выполняется filter expression, то есть фильтрующее выражение, и оно возвращает решение: обработать исключение, продолжить поиск или продолжить выполнение. В бинарном виде это обычно отражается через scope table и __C_specific_handler.

> __finally (блок завершения, termination handler) - это часть SEH, которая нужна не для выбора “поймать или не поймать”, а для гарантированного выполнения cleanup-кода

при выходе из защищенной области, в том числе при unwind. Его смысл ближе к “прибраться перед уходом”, чем к “обработать ошибку”.

В хорошей ментальной модели __try - это не “ловушка”, поставленная прямо в стек вручную. Это указание компилятору создать описание защищенной области. __except - не просто кусок кода, который Windows вызывает напрямую; между Windows и твоим кодом обычно стоит language-specific handler. __finally - не exception handler в узком смысле, потому что он не выбирает, нужно ли считать исключение обработанным. Он должен быть вызван в правильный момент, когда управление покидает защищенный участок, особенно при раскрутке стека.

Что вообще считается исключением в SEH

Слово exception часто переводят как “исключение”, но само по себе оно слишком широкое. В разговоре программисты могут называть исключением C++ throw, ошибку доступа к памяти, деление на ноль, breakpoint в отладчике, системный RaiseException или даже логическую ошибку. Windows SEH работает не с абстрактной “неприятностью”, а с конкретным объектом исключения и конкретным сохраненным состоянием потока.

EXCEPTION_RECORD (запись исключения) - это структура Windows, которая описывает само событие исключения: код исключения, флаги, адрес, где оно произошло, дополнительные параметры. В ней хранится не “как исправить ошибку”, а факт: что случилось и с какими характеристиками.
CONTEXT record (запись контекста процессора) - это структура, в которой Windows сохраняет состояние регистров потока на момент исключения или на текущем шаге unwind. Для x64 это особенно важно: RIP показывает текущую инструкцию, RSP показывает стек, остальные регистры помогают понять состояние вычислений. Без CONTEXT невозможно корректно продолжить выполнение или перейти к предыдущему кадру.

Аппаратные исключения возникают из-за событий на уровне процессора или памяти. Например, access violation появляется, когда код пытается читать, писать или исполнять память, к которой у процесса нет подходящего доступа. Illegal instruction возникает, когда процессор не может декодировать или выполнить инструкцию. Divide by zero - классический случай арифметического исключения. Breakpoint и single-step особенно важны для отладчиков: они тоже проходят через механизм исключений, хотя по смыслу не являются “авариями” программы.

hardware exception (аппаратное исключение) - это исключение, инициированное процессором или подсистемой памяти: нарушение доступа, деление на ноль, недопустимая инструкция, breakpoint, single-step и похожие события. Такое исключение не вызывается обычной функцией приложения, хотя затем Windows представляет его в виде стандартного EXCEPTION_RECORD.

Программные исключения создаются самим кодом. В Windows для этого есть RaiseException. Программа может вызвать RaiseException с собственным кодом исключения и параметрами. С точки зрения диспетчера это тоже исключение: есть EXCEPTION_RECORD, есть CONTEXT, есть поиск обработчиков. Именно поэтому SEH называют единым механизмом: он позволяет одинаковым путем провести и аппаратное

событие, и программно созданное исключение.

> RaiseException (создать программное исключение) - это WinAPI-функция, с помощью которой код явно инициирует исключение. Она не “ломает процесс” сама по себе; она передает Windows исключительное событие, после чего запускается обычная логика диспетчеризации: отладчик, vectored handlers, frame-based handlers, search phase и возможный unwind.

```
Пример исходного уровня:

__try {
*(int*)0 = 1; // вероятный access violation
}
__except(EXCEPTION_EXECUTE_HANDLER) {
// код обработки
}

В исходнике это выглядит как один защищенный блок.
В бинарном виде это будет обычный машинный код + unwind-метаданные +
handler RVA + language-specific data, которые читает __C_specific_handler.
```

Три разных смысла слова “обработчик”

Одна из причин, почему SEH тяжело входит в голову, - слово handler. Оно встречается в разных местах, но каждый раз немного означает другое. Если не развести эти смыслы, вся архитектура превращается в кашу. В исходном коде handler-ом могут назвать тело __except. В xdata handler-ом называют функцию, чей адрес записан в UNWIND_INFO. В системном API handler-ом может быть vectored exception handler. В C++ runtime handler-ом называют __CxxFrameHandler. Это не одно и то же.

> handler (обработчик) - общее слово для кода, который участвует в реакции на исключение. Но в Windows x64 нужно всегда уточнять, какой именно handler имеется в виду: пользовательский код обработки, language-specific handler, vectored handler, top-level filter или runtime helper вроде __C_specific_handler.

Пользовательский handler - это то, что программист хотел выполнить при исключении. Например, тело __except, где он пишет лог, закрывает ресурс или выбирает запасной путь. Но Windows обычно не прыгает туда напрямую сразу после access violation. Сначала вызывается служебная runtime-функция, которая понимает таблицы компилятора и решает, подходит ли этот участок.

> user handler (пользовательский обработчик) - это код, который написал программист: тело __except, блок catch или cleanup-логика. Он является целью высокого уровня, но системный диспетчер часто добирается до него через промежуточный runtime-handler и служебные таблицы.

Language-specific handler - это обработчик другого уровня. Он не обязан быть “бизнес-логикой” программы. Он читает данные, созданные компилятором. Для C SEH это __C_specific_handler. Для C++ - свои frame handler-ы. Для другого runtime может быть другая функция. Система Windows вызывает такой handler, потому что сама не знает синтаксиса и семантики всех языков.

runtime handler (обработчик среды выполнения) - это служебная функция runtime или компилятора, которая реализует правила конкретного языка или расширения языка. __C_specific_handler - пример runtime handler-а для C SEH. Он находится между Windows dispatcher-ом и пользовательским телом __except / __finally.

Vectored handler - это вообще другой маршрут. Он регистрируется через API и вызывается в порядке отдельного списка, не как handler конкретного стекового кадра. Он может увидеть исключение до frame-based поиска. Поэтому его нельзя смешивать с SEH-блоком внутри функции. Да, Microsoft называет VEH расширением SEH, но архитектурно это не то же самое, что __try/__except, привязанный к кадру.

VEH (Vectored Exception Handling, векторная обработка исключений) - это механизм, где обработчики регистрируются глобально для процесса и вызываются как список наблюдателей/обработчиков исключений. VEH не привязан к конкретному кадру функции через .pdata/.xdata так же, как frame-based SEH. Подробно VEH будет разобран в следующем разделе, но уже здесь важно не путать его с __try/__except.

Почему x64 SEH нельзя понимать через старую x86-картинку

Исторически многие статьи по SEH рассказывали x86-модель: в стеке есть цепочка записей, каждая запись содержит указатель на предыдущую и адрес обработчика, а голова цепочки хранится через FS:[0] в TEB. Для старого x86 это было очень важно. Но если механически перенести эту картинку на x64, понимание сломается. На Windows x64 основная модель обработки исключений - table-based, то есть основанная на таблицах. Это значит, что система ищет информацию о функциях не через связный список записей, лежащих в стеке, а через метаданные образа и активные таблицы функций.

table-based exception handling (табличная обработка исключений) - это модель, где сведения о функциях, кадрах и обработчиках хранятся в таблицах метаданных, а не в виде цепочки handler-записей, прошитых прямо в стек. Для Windows x64 это фундаментально: .pdata и .xdata становятся не вспомогательной информацией, а частью механизма SEH/unwind.

frame-based exception handling (покадровая обработка исключений) - это подход, где обработчик связан с конкретным кадром выполнения функции. На x64 эта связь существует, но выражена не через простую запись “в стеке лежит handler”, а через RUNTIME_FUNCTION, UNWIND_INFO, Handler RVA, HandlerData, EstablisherFrame и DispatcherContext.

Разница здесь не косметическая. В x86-стиле атака на стек могла попытаться перезаписать handler pointer в цепочке SEH. Поэтому исторически существовали механизмы вроде SafeSEH. В x64 классическая картинка “переписал SEH-запись на стеке” уже не является базовой. Handler не берется из случайной записи на стеке; он находится через таблицы образа и unwind-информацию. Это не делает x64 магически неуязвимым, но меняет природу анализа и защиты.

SafeSEH (таблица безопасных SEH-обработчиков) - это механизм линкера для x86-образов, при котором создается список допустимых exception handlers. Для x64 его нельзя считать центральным механизмом, потому что x64 использует табличную модель unwind/exception handling через .pdata/.xdata.

Для reverse engineer это означает простую практическую вещь: если ты анализируешь x64 бинарь и ищешь SEH так же, как в старом x86-туториале, ты будешь смотреть не туда. Нужно начинать не со стека, а с Exception Directory, .pdata, RUNTIME_FUNCTION, .xdata и UNWIND_INFO. Если у функции есть UNW_FLAG_EHANDLER или UNW_FLAG_UHANDLER, в ее unwind-информации после основных unwind-кодов будет адрес language-specific handler-a и данные для него. Именно там начинается реальная x64-картина.

Где SEH реально живет в x64-бинаре

Теперь разложим вопрос буквально: где искать SEH в файле и памяти? В исходном коде SEH живет как __try/__except или __try/__finally. Но после компиляции исходного кода уже нет. В исполняемом файле остаются следы, которые нужно связывать друг с другом.

Первый слой - машинный код в .text. Там находится обычное тело функции, включая инструкции, которые могут вызвать исключение, и пользовательские фрагменты обработчика. Но сам факт, что участок был внутри __try, не обязан быть очевиден по одной инструкции. Компилятор может разнести фильтр, handler body и основной код по разным местам, добавить переходы, оптимизировать ветвления. Поэтому анализ только .text почти всегда недостаточен.

> .text (секция кода) - это секция PE-образа, где обычно находятся машинные инструкции программы. В контексте SEH она содержит исполняемый код protected block, filter-кода и handler-кода, но не обязательно содержит саму “карту” того, какие диапазоны кода защищены.

Второй слой - .pdata. Там лежат RUNTIME_FUNCTION entries, которые говорят: вот диапазон функции и вот адрес ее unwind-информации. Если функция участвует в x64 unwind, система должна уметь найти ее запись по RIP. Это еще не говорит, что у функции есть SEH-обработчик; это только входная дверь к .xdata.

Третий слой - .xdata. Именно там живет UNWIND_INFO. В нем, кроме описания пролога, могут быть flags. Если установлен UNW_FLAG_EHANDLER, функция имеет exception handler. Если установлен UNW_FLAG_UHANDLER, функция имеет termination/unwind handler. После массива UNWIND_CODE появляется дополнительная область: адрес language-specific handler-a и language-specific data. Для C SEH там появляется связь с __C_specific_handler и его данными.

> UNW_FLAG_EHANDLER (флаг exception handler-a) - бит в UNWIND_INFO, который означает: у этой функции есть обработчик исключений, и после unwind-кодов хранится адрес language-specific handler-a. Это не адрес пользовательского тела __except напрямую, а адрес handler-a уровня runtime, который будет вызван системной логикой.

> UNW_FLAG_UHANDLER (флаг unwind/termination handler-a) - бит в UNWIND_INFO, который означает: у этой функции есть обработчик, важный при раскрутке стека, например для __finally или cleanup. Он участвует не столько в выборе “поймать исключение”, сколько в корректном завершении покидаемого кадра.

> HandlerData (данные обработчика) - область после адреса language-specific handler-a, смысл которой зависит от конкретного runtime-handler-a. Для __C_specific_handler это данные, описывающие защищенные диапазоны, фильтры и адреса обработчиков.

> Windows не обязана понимать эти данные до конца; она передает их handler-у.

Четвертый слой - runtime-функции. Например, __C_specific_handler находится в runtime/системной библиотеке и вызывается как language-specific handler. Он читает HandlerData и DispatcherContext, чтобы понять, какие protected ranges относятся к текущему ControlPc, какие фильтры нужно вызвать и куда передать управление, если исключение обработано.

Пятый слой - системный runtime в памяти процесса. Даже если файл на диске содержит .pdata и .xdata, во время выполнения Windows работает с загруженным образом, ImageBase, активными таблицами функций и текущим CONTEXT. Поэтому parser файла и runtime-поиск похожи, но не идентичны. Parser смотрит байты PE offline. Система смотрит активное состояние процесса.

Мини-карта: что из чего вытекает

Чтобы удержать эту систему в голове, полезно видеть не “кучу названий”, а цепочку смысла. Исключение возникает не в метаданных, а в исполняемом коде. Но чтобы обработать его, Windows должна перейти от адреса инструкции к записи функции, потом к unwind-описанию, потом к handler-у, потом к данным языка, и только потом к решению.

Шаг	Что известно системе	Что ищется дальше	Почему это важно для SEH
1	Есть RIP / ControlPc - адрес инструкции, где возникло исключение.	Нужно понять, в какой функции находится этот адрес.	Без функции нельзя найти ее unwind-метаданные и привязанный обработчик.
2	Через active function tables найдена RUNTIME_FUNCTION.	По UnwindInfoAddress нужно перейти в .xdata.	RUNTIME_FUNCTION связывает диапазон кода с UNWIND_INFO.
3	Прочитан UNWIND_INFO.	Нужно увидеть flags и возможный Handler RVA.	Флаги показывают, есть ли у функции language-specific handler.
4	Найден language-specific handler и HandlerData.	Нужно вызвать handler с DispatcherContext.	Runtime-handler понимает данные компилятора и решает, подходит ли обработчик.
5	Handler вернул EXCEPTION_DISPOSITION.	Нужно продолжить поиск, обработать или начать unwind.	Именно здесь SEH превращается из таблиц в реальное управление потоком.

Как __try / __except превращается в x64 SEH

Представим обычный C-код. В нем есть функция, внутри функции есть защищенный блок, а после него __except. В исходном виде это выглядит почти как if: если что-то пошло не так, выполнить обработчик. Но компилятор не может оставить это как простой if, потому что исключение может возникнуть не как обычный return-код функции. Access violation не возвращает значение. Процессор прерывает нормальное выполнение, Windows сохраняет контекст и запускает exception dispatching.

```
int example(int *p)
{
__try {
*p = 123;
return 1;
}
__except(GetExceptionCode() == EXCEPTION_ACCESS_VIOLATION
? EXCEPTION_EXECUTE_HANDLER
: EXCEPTION_CONTINUE_SEARCH) {
return -1;
}
}
```

Для такого кода компилятор должен сделать несколько вещей. Во-первых, он генерирует обычный машинный код для записи *p = 123. Во-вторых, он генерирует код filter expression, потому что условие в __except нужно где-то выполнить. В-третьих, он генерирует код handler body, где возвращается -1. В-четвертых, он создает служебные данные, описывающие, какой диапазон инструкций является protected range, где находится filter, где находится handler body. В-пятых, он делает так, чтобы UNWIND_INFO функции указывал на __C_specific_handler как language-specific handler.

protected range (защищенный диапазон) - это диапазон инструкций, для которого действует определенная SEH-логика: фильтр, обработчик или finally. Это не обязательно весь текст функции. Обычно это конкретный участок внутри функции, и именно поэтому handler-у нужны дополнительные данные, а не просто “функция имеет обработчик”.
filter expression (фильтрующее выражение) - это код, который вычисляет, что делать с исключением: обработать, продолжить поиск или продолжить выполнение. В __except(...) скобки - не просто комментарий. Это исполняемая логика, которую runtime может вызвать во время search phase.
EXCEPTION_EXECUTE_HANDLER (выполнить обработчик) - значение, которое filter expression возвращает, когда исключение нужно считать подходящим для данного __except, а управление должно перейти в handler body после выполнения необходимых действий диспетчера.
EXCEPTION_CONTINUE_SEARCH (продолжить поиск) - значение, которое говорит: этот __except не берет исключение на себя, нужно искать следующий подходящий обработчик выше по стеку.
EXCEPTION_CONTINUE_EXECUTION (продолжить выполнение) - значение, которое означает попытку продолжить выполнение после места исключения. Это опасный и редкий вариант, потому что для него контекст должен быть исправлен так, чтобы повторное выполнение не привело к той же ошибке.

Когда access violation возникает внутри protected range, Windows не знает, что написано в __except. Она знает только текущий ControlPc и текущий ContextRecord. Дальше система через RtlLookupFunctionEntry находит RUNTIME_FUNCTION, через него доходит до UNWIND_INFO, видит UNW_FLAG_EHANDLER, получает адрес __C_specific_handler, формирует DispatcherContext и вызывает этот handler. __C_specific_handler уже читает свои C-specific данные, видит, что ControlPc попадает в защищенный диапазон, вызывает filter expression и получает одно из специальных значений.

Если фильтр возвращает EXCEPTION_CONTINUE_SEARCH, поиск продолжается вверх по стеку. Если возвращает EXCEPTION_EXECUTE_HANDLER, текущий кадр становится target frame - целевым кадром обработки. Но это не значит, что Windows просто прыгает в handler body мгновенно. Если между местом исключения и целевым обработчиком есть кадры, которые нужно покинуть, начинается unwind phase. Во время этой фазы должны быть выполнены cleanup-и, finally-блоки и другая логика завершения. Только после этого управление попадает туда, где пользователь ожидает обработку.

> target frame (целевой кадр) - кадр стека, чей handler согласился обработать исключение. Search phase ищет именно такой кадр. Unwind phase затем приводит поток выполнения к этому кадру, корректно покидая промежуточные кадры.

Что значит “структурированная” обработка

Слово structured в SEH полезно воспринимать буквально: обработка исключений встроена в структуру программы. Есть вложенность функций. Есть вложенность protected ranges. Есть LIFO-порядок стека. Есть search phase, которая идет от текущего кадра к вызывающим кадрам. Есть unwind phase, которая покидает кадры в обратном порядке. Это не просто “поймать любую ошибку где-нибудь”.

> LIFO (Last In, First Out; последним пришел - первым ушел) - принцип стека: последний созданный кадр функции должен быть покинут первым. При unwind это критично, потому что cleanup должен выполняться в порядке, обратном входу в функции и защищенные области.

Представь обычный вызов функций: main вызвал A, A вызвала B, B вызвала C. Исключение произошло в C. Если обработчик находится в A, нельзя просто перепрыгнуть из C в A так, будто B и C никогда не существовали. Их кадры нужно корректно покинуть. Если в них были finally-блоки, cleanup-и, C++ деструкторы или сохраненные ресурсы, их нужно обработать. Именно поэтому SEH тесно связан с unwind. Структурированность - это дисциплина выхода из вложенных областей.

На x64 эта дисциплина выражена не только в логике языка, но и в машинной форме. Чтобы покинуть кадр, нужно знать, как функция изменила RSP, где сохранила nonvolatile-регистры, есть ли frame pointer, находится ли RIP в прологе или эпилоге. То есть структурированная обработка исключений требует структурированного описания кадра. Поэтому x64 SEH и x64 unwind фактически идут рядом.

Почему SEH на x64 - это не “try/catch для бедных”

Иногда SEH упрощенно сравнивают с try/catch. Это полезно только на самом первом уровне: и то, и другое позволяет реагировать на исключительные ситуации. Но в Windows-мире SEH имеет особую роль. Он работает с аппаратными исключениями. Он встроен в Win32-модель исключений. Он используется отладчиками. Он связан с системным EXCEPTION_RECORD и CONTEXT. Он может обслуживать access violation, breakpoint, single-step, illegal instruction и программные RaiseException. C++ exceptions имеют свою семантику типов, catch-объектов и деструкторов; SEH - другая архитектурная линия.

C++ exception (исключение C++) - это исключение, созданное механизмом языка C++ через throw и обработанное через catch. На Windows x64 C++ exceptions тоже используют общую инфраструктуру unwind, но их semantics, то есть правила смысла, задает C++ runtime, а не C SEH-фильтр.

semantics (семантика, смысловые правила) - это не байтовое представление, а правила значения. Например, C++ catch выбирает обработчик по типу исключения, а SEH __except filter может смотреть на код EXCEPTION_RECORD. Оба механизма могут использовать unwind, но смысл выбора обработчика у них разный.

В reverse engineering это различие очень важно. Если ты видишь language-specific handler, нельзя сразу говорить “это SEH” в узком C-смысле. Нужно посмотреть, какой handler: __C_specific_handler, __CxxFrameHandler3, __CxxFrameHandler4 или что-то другое. Один handler может означать C SEH, другой - C++ exception handling. Оба используют x64 exception/unwind infrastructure, но данные handler-а и логика выбора разные.

__CxxFrameHandler (C++ frame handler, обработчик кадра C++) - семейство runtime-handler-ов Microsoft C++, которые реализуют обработку C++ exceptions, вызов деструкторов и логику catch. Он связан с общей x64 unwind-инфраструктурой, но не равен __C_specific_handler и не должен автоматически называться C SEH.

Как SEH связан с first chance и second chance

В реальной отладке человек часто встречает выражения first chance exception и second chance exception. Они звучат как что-то отдельное от SEH, но на самом деле это часть маршрута исключения через систему и отладчик. Когда исключение возникает, Windows может сначала сообщить о нем отладчику. Это first chance - первый шанс увидеть и, возможно, обработать исключение. Если никто не обработал исключение обычным путем, отладчик может получить второй шанс перед тем, как процесс будет завершен.

first chance exception (исключение первого шанса) - момент, когда исключение только возникло и отладчик получает возможность увидеть его до обычного завершения поиска обработчиков. First chance не означает, что программа уже упала; многие исключения первого шанса могут быть потом обработаны приложением.

second chance exception (исключение второго шанса) - момент, когда исключение не было обработано обычными механизмами, и отладчик получает последнюю возможность вмешаться перед тем, как Windows сочтет исключение необработанным.

Для reverse engineer это важно по двум причинам. Во-первых, наличие first chance access violation в отладчике не всегда означает фатальную ошибку. Некоторые программы, runtime-ы, защитные оболочки и JIT-ы специально используют исключения как часть логики. Во-вторых, обработка исключения может быть невидима в обычном linear disassembly: управление уходит через exception dispatcher, handler-таблицы и runtime-функции, а не через обычный call в пользовательском коде.

SEH как скрытый канал управления потоком

Обычное управление потоком видно в инструкциях: call, ret, jmp, conditional jump. SEH добавляет другой канал. Инструкция может вызвать исключение, дальше управление временно уйдет в системный dispatcher, затем в handler, затем через unwind попадет в

другой участок функции или вообще в другой кадр. В дизассемблере это выглядит необычно: нет прямого jmp от места ошибки к __except body, но логически такой переход возможен.

control flow (поток управления) - порядок, в котором программа переходит от одной инструкции к другой. Обычно его анализируют по call/jmp/ret, но exception handling создает дополнительные переходы, которые задаются метаданными и runtime-логикой, а не прямыми инструкциями перехода.
exception-driven control flow (управление потоком через исключения) - ситуация, когда программа намеренно или побочно использует исключения для перехода между участками кода. Это может быть нормальная защитная обработка ошибок, runtime-механизм, антиотладочная техника или обфускация.

Здесь легко ошибиться. Не каждый SEH - обфускация. Нормальная программа может использовать __try/__except для защиты от аппаратных исключений, особенно в низкоуровневом коде, обработчиках плагинов, JIT, sandbox-логике, системных компонентах. Но если бинарь постоянно вызывает исключения, использует нестандартные handler-таблицы, прячет переходы через RaiseException или намеренно делает access violation ради управления, reverse engineer должен отметить это как важный сигнал.

Как вручную узнавать SEH в x64 PE

Теперь перейдем к тому, что увидит твой будущий static /dynamic или ты сам в Ghidra/IDA/WinDbg. Нельзя просто искать строку “__try”. Ее нет. Нужно искать признаки, которые вместе создают картину. Один признак редко достаточен, но несколько признаков дают уверенность.

Первый признак - в UNWIND_INFO установлен UNW_FLAG_EHANDLER или UNW_FLAG_UHANDLER. Это самый прямой вход в тему: функция имеет handler-данные. Если handler RVA указывает на __C_specific_handler, вероятность C SEH очень высокая. Если на __CxxFrameHandler, скорее речь про C++ exceptions. Если на неизвестный адрес - нужно анализировать осторожно: это может быть custom runtime, hand-written assembly, JIT или защитный код.

Второй признак - HandlerData похожа на scope table. Для __C_specific_handler данные описывают области и переходы. Там могут быть begin/end диапазоны, filter addresses, handler addresses или jump targets. Формат деталей зависит от компилятора и окружения, но сама идея остается: runtime handler получает карту защищенных областей.

Третий признак - в коде есть вызовы RaiseException, GetExceptionCode, GetExceptionInformation, AbnormalTermination или другие SEH-related функции. Это не доказывает наличие frame-based SEH само по себе, но хорошо дополняет картину. Программа может использовать RaiseException без __try в текущей функции, а обработчик может быть выше по стеку.

GetExceptionCode (получить код исключения) - helper, который используется внутри SEH-фильтра или handler-а для получения кода текущего исключения. В исходном

коде он часто встречается внутри __except(...), но в бинарном виде его наличие нужно интерпретировать вместе с handler-данными.
GetExceptionInformation (получить информацию об исключении) - helper, возвращающий доступ к структурам EXCEPTION_POINTERS, где находятся EXCEPTION_RECORD и CONTEXT. Он полезен, когда фильтр или handler хочет смотреть не только код исключения, но и адрес, параметры и состояние процессора.
AbnormalTermination (ненормальное завершение protected block) - helper для termination handling, который позволяет внутри __finally понять, был ли защищенный блок покинут обычным путем или из-за исключения/unwind.

Четвертый признак - странная форма control flow. Например, есть код, который логически достижим, но обычные прямые переходы к нему не очевидны. Это может быть handler body. У хорошего дизассемблера часто есть анализ EH-таблиц, но полагаться только на автокомментарии опасно. Нужно уметь проверить руками: какой RUNTIME_FUNCTION, какой UNWIND_INFO, какие flags, какой handler RVA, где HandlerData, куда ведут protected ranges.

Почему отсутствие SEH-признаков тоже нужно понимать аккуратно

Если у функции нет UNW_FLAG_EHANDLER, это не значит, что в программе вообще нет исключений. Исключение может возникнуть в этой функции, но обработчик может находиться выше по стеку. Функция может быть leaf function и не иметь полноценной записи. Программа может использовать VEH. C++ exception logic может находиться в других функциях. JIT-код может иметь dynamic function tables. A packed binary может ломать привычные ожидания.

absence of evidence (отсутствие признака) - в reverse engineering это ситуация, когда мы не нашли конкретный артефакт, но не имеем права автоматически делать сильный вывод. Например, нет EHANDLER flag у одной функции - это не доказательство, что исключения не используются во всем модуле.

Хороший анализатор должен различать “не найдено” и “доказано, что отсутствует”. Если Exception Directory пустой в обычном x64 PE, это одно. Если есть .pdata, но у конкретной функции нет handler flag, это другое. Если есть dynamic function tables, но offline parser их не видит, это третье. Если бинарь упакован и реальный код появится только в памяти, это четвертое. Для static /dynamic нужно не просто печатать “SEH: no”, а объяснять уровень уверенности.

SEH и безопасность: что изменилось на x64

x64-архитектура обработки исключений изменила важную поверхность атак. В x86-мире классическая тема SEH overwrite была связана с тем, что handler-записи лежали в стеке и могли быть перезаписаны при stack overflow. На x64 основной путь другой: handler берется из таблиц unwind, связанных с образом. Поэтому старые x86-техники нельзя механически переносить.

SEH overwrite (перезапись SEH-записи) - историческая техника эксплуатации, особенно известная в x86-контексте, когда переполнение стека перезаписывает запись обработчика исключений. Для Windows x64 классическая форма этой техники не является базовой из-за table-based модели.

Но это не значит, что SEH на x64 не интересен злоумышленникам и защитникам. Метаданные могут быть повреждены, подделаны, намеренно усложнены или созданы динамически. Packer может использовать исключения для передачи управления. JIT должен регистрировать function tables, иначе unwind и обработка исключений будут ломаться. Malware может использовать VEH/SEH для антиотладки. Поэтому “безопасность SEH” на x64 - это уже не только “можно ли перезаписать handler pointer”, а более широкая тема доверия к runtime metadata, корректности unwind, динамических таблиц и поведения при исключениях.

packer (упаковщик) - программа или слой защиты, который изменяет структуру исполняемого файла, часто сжимает/шифрует код и восстанавливает его во время выполнения. В контексте SEH packer может использовать исключения и нестандартные метаданные как часть защиты или запутывания control flow.
malware (вредоносное ПО) - код, написанный для вредоносных действий. В теме SEH malware интересно тем, что может использовать exception handling как антиотладочный механизм, способ скрытого перехода управления или способ усложнить статический анализ.

Как это должно выглядеть в static /dynamic

Если переносить тему в твой static /dynamic , важно не превращать SEH-анализ в один флажок. Хороший инструмент должен строить объяснимую цепочку. Например: у функции найден RUNTIME_FUNCTION; по нему найден UNWIND_INFO; в flags установлен UNW_FLAG_EHANDLER; handler RVA указывает на __C_specific_handler; HandlerData находится в допустимой области; scope entries указывают на диапазоны внутри функции; filter/handler addresses попадают в исполняемый код. Такой отчет будет полезен, потому что он показывает доказательства, а не просто финальный вывод.

finding (обнаруженный факт анализа) - конкретное наблюдение инструмента: например, “UNW_FLAG_EHANDLER установлен”, “handler RVA указывает на __C_specific_handler”, “HandlerData выходит за пределы секции”. Finding не обязан сразу быть ошибкой; это единица evidence для дальнейшего verdict.
evidence (доказательная база, свидетельства) - набор findings, на котором строится вывод анализатора. Для SEH evidence особенно важен, потому что один признак может быть нормальным, а комбинация признаков может указывать на защиту, обфускацию или повреждение.
verdict (итоговый вывод) - интерпретация evidence: нормальная SEH-структура, подозрительная metadata, поврежденные границы, нестандартный handler, возможная обфускация. Verdict должен быть осторожным и объяснимым, иначе инструмент будет пугать ложными срабатываниями.

Примеры полезных findings для SEH-анализа: функция имеет EHANDLER flag; функция имеет UHANDLER flag; handler RVA совпадает с импортом или символом

__C_specific_handler; handler RVA указывает внутрь неизвестного кода; HandlerData RVA выходит за пределы читаемой секции; protected range выходит за EndAddress функции; filter address не попадает в исполняемую секцию; scope table содержит пересекающиеся диапазоны; у функции есть handler flag, но unwind info выглядит поврежденной; в модуле есть RaiseException при отсутствии видимых frame-based handlers.

> Важно: такой анализ должен избегать грубой логики “необычно = вредоносно”. Для системного кода, JIT, языковых runtime, драйверных компонентов и защитных оболочек нестандартное использование exception handling может быть нормальной инженерной необходимостью. Но оно все равно заслуживает подробного отчета, потому что влияет на stack walking, crash dump analysis и reverse engineering.

Типичные ошибки начинающего

Первая ошибка - думать, что SEH на x64 хранится “в стеке как на x86”. На x64 основная модель табличная. Стек все еще важен, потому что там живут кадры, return address, локальные данные и сохраненные регистры, но handler-информация ищется через .pdata/.xdata и active function tables.

Вторая ошибка - считать, что __C_specific_handler и есть пользовательский обработчик. На самом деле это посредник. Пользовательский handler body может находиться совсем в другом месте, а __C_specific_handler читает HandlerData и решает, когда туда перейти.

Третья ошибка - путать VEH и frame-based SEH. VEH регистрируется глобально через API и не связан с конкретным стековым кадром так, как __try/__except. Он может увидеть исключение раньше frame-based поиска, но это другой механизм.

Четвертая ошибка - думать, что C++ exceptions и SEH полностью одно и то же. На Windows x64 они используют общую инфраструктуру unwind, но runtime-handler-ы и смысловые правила различаются. C++ catch работает с типами и объектами исключений; SEH-фильтр работает с EXCEPTION_RECORD и кодами исключений.

Пятая ошибка - воспринимать exception handling как “редкий аварийный путь”, который можно не анализировать. В обычном приложении это иногда правда. Но в reverse engineering, защитах, JIT, отладке, crash dump analysis и malware exception paths могут быть центральной частью поведения.

Связь с тем, что уже было пройдено

Теперь можно собрать всю цепочку в одну фразу, но уже не как заученный лозунг, а как осознанную модель. SEH на x64 начинается с исключения, описанного через EXCEPTION_RECORD и CONTEXT. Система берет ControlPc, через active function tables ищет RUNTIME_FUNCTION, через .xdata читает UNWIND_INFO, по flags понимает наличие handler-а, вызывает language-specific handler с EstablisherFrame и DispatcherContext, а handler по своим данным решает, продолжать поиск, обработать исключение или участвовать в unwind. Если нужно покинуть кадры, RtlVirtualUnwind и unwind metadata помогают корректно восстановить caller contexts и выполнить cleanup.

Эта модель длинная, но именно она честная. Упрощение до “SEH - это __try/__except” полезно только на первом занятии. Для reverse engineering оно слишком слабое. На x64

настоящий SEH - это соединение исходной конструкции, compiler metadata, PE-структуры, unwind-механизма, runtime-handler-а и системной диспетчеризации исключений.

Почему Windows не может просто “прыгнуть в except”

Еще один важный момент нужно проговорить отдельно. Когда исключение произошло внутри __try, хочется представить себе простую картинку: Windows увидела ошибку и сразу прыгнула в __except. Но такая картинка слишком грубая и технически неверная. Чтобы попасть в __except, системе нужно не просто выбрать адрес. Ей нужно понять, принадлежит ли текущий RIP защищенному диапазону, не находится ли поток в прологе или эпилоге, какой кадр является establisher frame, какие промежуточные кадры надо покинуть, какие finally-блоки должны выполниться, и можно ли вообще считать исключение обработанным.

Причина в том, что обработчик не живет в вакууме. Он находится внутри функции, у этой функции есть стековый кадр, локальные переменные, сохраненные регистры, аргументы и состояние компилятора. Если перепрыгнуть в handler body без подготовки, можно получить невозможную ситуацию: RSP указывает не туда, nonvolatile-регистры не восстановлены, локальные переменные читаются из неправильного места, а finally-блоки промежуточных областей пропущены. Поэтому exception handling - это не просто transfer of control, а дисциплинированная процедура восстановления и передачи управления.

> transfer of control (передача управления) - переход выполнения программы из одной точки в другую. В обычном коде это может быть jmp/call/ret. В SEH передача управления к handler-у проходит через системный dispatcher и unwind, поэтому она гораздо сложнее обычного прыжка по адресу.

В этом смысле SEH похож на аварийный выход из здания. Нельзя телепортировать человека с пятого этажа на улицу, игнорируя двери, лестницы и людей на пути. Нужно пройти по маршруту, закрыть опасные участки, дать выйти тем, кто был внутри, и только потом оказаться в безопасной зоне. Search phase находит, где выход. Unwind phase ведет поток к этому выходу. Language-specific handler объясняет, какая дверь относится именно к этому языку и этому protected range.

Роли компилятора, линкера, загрузчика и runtime

SEH на x64 полезно понимать как коллективную работу нескольких участников. Ни один из них в одиночку не владеет всей картиной. Компилятор знает исходный код и может понять, где находится __try, где filter, где handler body, какие локальные переменные нужны и как функция меняет стек. Линкер собирает объектные файлы в PE, размещает секции, таблицы и относительные адреса. Загрузчик Windows отображает образ в память и назначает ImageBase. Runtime-функции вроде __C_specific_handler читают данные компилятора. Системные функции вроде RtlLookupFunctionEntry и RtlVirtualUnwind выполняют поиск и виртуальную раскрутку. Если хотя бы один слой ошибается или данные повреждены, вся цепочка может сломаться.

> compiler (компилятор) - программа, которая переводит исходный код в машинный код и служебные данные. В теме SEH компилятор не просто генерирует инструкции; он

создает protected ranges, handler data, unwind annotations и выбирает language-specific handler.
linker (линкер, компоновщик) - программа, которая объединяет объектные файлы и библиотеки в итоговый PE-образ. В теме x64 SEH он размещает .pdata/.xdata, связывает RVA, импортирует нужные runtime-функции и делает так, чтобы таблицы были согласованы с кодом.
loader (загрузчик) - компонент Windows, который отображает PE-образ в память процесса, применяет relocations, подготавливает imports и делает модуль частью адресного пространства. Для exception handling важно, что runtime работает уже не с “файлом вообще”, а с загруженным image и конкретным ImageBase.
runtime (среда выполнения) - набор служебного кода, который помогает реализовать правила языка и компилятора во время выполнения программы. __C_specific_handler и C++ frame handler-ы - примеры runtime-участников exception handling.

Это объясняет, почему ручной анализ SEH требует аккуратности. Нельзя смотреть только на исходный код, если мы занимаемся reverse engineering. Нельзя смотреть только на машинные инструкции, если обработка задана метаданными. Нельзя смотреть только на .xdata, если реальные таблицы могли быть зарегистрированы динамически. Нельзя доверять одному декомпилятору, если binary защищен или поврежден. Нужно собрать evidence из разных слоев.

Scope table как “карта смысла” для __C_specific_handler

Мы уже несколько раз упоминали scope table, но здесь стоит объяснить ее медленнее. Windows x64 dispatcher знает, что у функции есть language-specific handler. Он может вызвать __C_specific_handler и передать ему DispatcherContext. Но сам dispatcher не знает, какие именно байты функции соответствуют __try, где начинается и заканчивается protected block, куда вести выполнение при обработке, где находится filter. Эти знания принадлежат C compiler/runtime. Поэтому рядом с handler-ом лежат данные, которые можно представить как карту областей.

scope table (таблица областей, таблица защищенных участков) - служебные данные language-specific handler-а, описывающие, какие диапазоны кода защищены, какие фильтры к ним относятся и куда передавать управление при обработке или завершении. Это не универсальная PE-таблица Windows, а данные конкретного runtime/compiler-а.

Название scope хорошо передает смысл. Scope - это область действия. В языке программирования областью действия может быть блок кода, участок видимости переменной или защищенная зона обработки исключений. В SEH scope table помогает runtime понять: текущий ControlPc находится внутри какой области? есть ли у этой области filter? есть ли finally? какой target нужно выбрать, если filter согласился обработать исключение?

scope (область действия) - участок программы, в котором действует определенное правило: переменная видима, cleanup должен быть выполнен, exception filter относится к protected block. В SEH scope обычно связан с защищенным диапазоном инструкций.

На уровне идеи scope record можно представить как строку таблицы: начало защищенного диапазона, конец защищенного диапазона, адрес filter-кода, адрес handler-кода или jump target. Реальный формат может иметь нюансы, но для понимания важно другое: __C_specific_handler не “угадывает” обработчик по дизассемблеру. Он читает компактные данные, созданные компилятором.

> scope record (запись области) - одна запись внутри scope table. Она описывает один protected range и связанные с ним действия: filter, handler target или termination handler. Для reverse engineering это ценный артефакт, потому что он восстанавливает структуру исходного exception-handling кода лучше, чем обычный список инструкций.

Именно из-за scope table декомпилятор может иногда восстановить конструкции, похожие на __try/__except или __try/__finally. Он смотрит не только на flow graph кода, но и на EH metadata. Если метаданные повреждены, декомпилятор может ошибиться: показать странный псевдокод, потерять handler body, неправильно построить границы блока или вообще не распознать SEH. Поэтому человек должен понимать, откуда декомпилятор берет эту информацию.

Чем SEH отличается от обычной проверки ошибок

Чтобы тема не оставалась абстрактной, сравним SEH с обычной проверкой return value. Когда функция CreateFile возвращает INVALID_HANDLE_VALUE, программа может проверить результат обычным if. Здесь нет исключения: выполнение идет по нормальному control flow. CPU не прерывал поток, Windows не создавала EXCEPTION_RECORD, RtlVirtualUnwind не раскручивал кадры. Это просто условная ветка.

> return value (возвращаемое значение) - обычный результат функции, который вызывающий код получает через регистр или память по правилам calling convention. Проверка return value - нормальный путь выполнения, не требующий exception dispatcher-а.

SEH нужен для другого класса ситуаций. Например, код читает память по указателю, который оказался недоступен. Инструкция не может нормально вернуть “ошибку” в регистре, потому что она не является функцией WinAPI. Процессор фиксирует исключение, Windows превращает его в стандартную структуру, и дальше вступает механизм exception handling. Поэтому SEH особенно важен для низкоуровневого кода, который работает с памятью, плагинами, JIT, sandbox, чужими callbacks, драйвероподобными паттернами и рискованными участками.

Отсюда вытекает важная инженерная мысль: SEH не заменяет нормальную обработку ошибок. Нельзя превращать каждую ошибку в access violation и гордиться “устойчивостью”. SEH - мощный механизм, но дорогой и сложный. Он полезен там, где ошибка действительно выходит за рамки обычного return-кода или где нужно интегрироваться с системной exception model.

SEH и продолжение выполнения

Особенно опасная часть SEH - возможность продолжить выполнение после исключения. В теории filter может вернуть EXCEPTION_CONTINUE_EXECUTION. Тогда система попытается продолжить поток с тем ContextRecord, который есть после обработки. Но

если причина исключения не устранена, программа просто упадет снова на той же инструкции. Поэтому продолжение выполнения требует изменения контекста, памяти или условий так, чтобы инструкция стала корректной.

continuable exception (продолжаемое исключение) - исключение, после которого теоретически можно продолжить выполнение, если handler вернет соответствующее решение и контекст будет пригоден для продолжения. Не все исключения безопасно или разрешено продолжать.
noncontinuable exception (непродолжаемое исключение) - исключение, после которого продолжение выполнения запрещено или бессмысленно. Попытка продолжить такое исключение может привести к новому исключению или завершению процесса.

Для reverse engineering продолжение выполнения интересно тем, что оно может скрывать необычный control flow. Handler может изменить RIP в ContextRecord, пропустить проблемную инструкцию, изменить регистры или подложить корректный адрес. В обычном графе переходов такого пути не видно. Это особенно любят защитные и антиотладочные техники, хотя легитимные применения тоже существуют.

ContextRecord modification (изменение записи контекста) - ситуация, когда handler меняет сохраненные регистры в CONTEXT, например RIP или RSP. После этого продолжение выполнения может пойти не туда, куда показывал обычный дизассемблированный поток. Для анализатора это сильный сигнал необычного поведения.

SEH и __finally: почему cleanup - не второстепенная деталь

Когда говорят об исключениях, обычно думают о “поймать ошибку”. Но не менее важна другая задача: корректно выйти из уже начатых операций. Если функция захватила mutex, открыла файл, временно изменила состояние, выделила ресурс, то при исключении нужно привести систему в нормальное состояние. Для этого существует termination handling - обработка завершения, которую в C SEH часто выражают через __finally.

cleanup (очистка, завершающие действия) - код, который освобождает ресурсы или восстанавливает состояние при выходе из области, особенно при исключительном выходе. Cleanup не обязан “обрабатывать” исключение; он может просто гарантировать, что ресурс не останется захваченным.
termination handling (обработка завершения) - механизм, при котором код выполняется при выходе из protected block независимо от того, был выход обычным или исключительным. __finally - классический SEH-инструмент termination handling.

На x64 наличие __finally отражается через unwind handler path. Когда идет unwind phase, промежуточные кадры должны получить шанс выполнить cleanup. Именно поэтому UNW_FLAG_UHANDLER важен отдельно от UNW_FLAG_EHANDLER. EHANDLER больше связан с поиском обработчика исключения, а UHANDLER - с действиями, которые нужно выполнить во время раскрутки.

Для начинающего это может звучать как мелочь, но в реальном анализе это принципиально. Если analyzer игнорирует UHANDLER, он может не увидеть код, который

выполняется при исключительном выходе. Если декомпилятор не восстановил finally, человек может не понять, где освобождается ресурс. Если malware прячет действия в cleanup path, обычный linear analysis может пропустить важное поведение.

Что происходит, если метаданные SEH повреждены

В нормальном компиляторском бинаре SEH-метаданные согласованы: диапазоны попадают в код, handler RVA указывает на реальную функцию, HandlerData читается, unwind codes корректны. Но reverse engineer часто работает не с идеальными учебными программами. Файл может быть поврежден, упакован, намеренно испорчен, собран нестандартным компилятором или загружен нештатно. Тогда возникает вопрос: что ломается?

Если повреждена .pdata, RtlLookupFunctionEntry может не найти запись или найти неправильный диапазон. Если поврежден UnwindInfoAddress, система может перейти к неверным .xdata. Если повреждены unwind codes, RtlVirtualUnwind может неверно восстановить RSP и регистры. Если поврежден handler RVA, dispatcher может вызвать неправильный обработчик. Если повреждена HandlerData, language-specific handler может неверно понять protected ranges. Любая из этих ошибок может привести к падению, неправильной обработке исключения, невозможности построить stack trace или к подозрительному поведению.

> metadata corruption (повреждение метаданных) - ситуация, когда служебные данные не соответствуют коду или выходят за допустимые границы. В x64 exception handling это особенно опасно, потому что метаданные участвуют не только в отладке, но и в реальном unwind/exception dispatch.

Для static /dynamic важно не только находить “есть SEH”, но и проверять согласованность. Нормальная запись должна ссылаться на валидные области. RVA должны преобразовываться в доступные участки образа. Handler RVA должен быть исполняемым или разрешаться как допустимая runtime-функция. HandlerData должна помещаться в читаемую секцию. Protected ranges не должны выходить за границы функции без объяснения. Если tool находит нарушение, он должен показать цепочку: какое поле, какое значение, какой ожидаемый диапазон, почему это влияет на unwind/SEH.

SEH в crash dump analysis

Crash dump analysis (анализ дампа падения) - еще одна область, где SEH на x64 становится практическим, а не академическим знанием. Когда программа падает, дамп содержит состояние процесса: память, стек, регистры, список модулей, иногда exception record. Анализатор дампа должен понять, где возникло исключение, кто мог его обработать, почему не обработал, корректен ли stack trace, не сломан ли unwind.

> crash dump (дамп падения) - сохраненное состояние процесса или системы в момент аварии. В нем можно анализировать EXCEPTION_RECORD, CONTEXT, стек, модули, .pdata/.xdata и попытаться восстановить путь выполнения до падения.

Если unwind metadata повреждена или отсутствует, stack trace может быть неполным или неверным. Если функция использует SEH, но handler не сработал, нужно понять почему: ControlPc не попал в protected range? filter вернул CONTINUE_SEARCH? exception был noncontinuable? handler data повреждена? был second chance? Без понимания SEH аналитик будет видеть только верхушку: “access violation here”. С пониманием SEH он сможет спросить правильные вопросы.

В дампе особенно полезно различать faulting context и unwind context. Faulting context - состояние в момент исключения. Unwind context - состояние, которое система вычисляет при переходе к вызывающим кадрам. RtlVirtualUnwind не просто читает стек; он меняет ContextRecord шаг за шагом. Поэтому dump analyzer должен доверять метаданным, но одновременно проверять их разумность.

faulting context (контекст сбоя) - состояние регистров и потока в момент, когда произошло исключение. Это исходная точка анализа: какой RIP упал, какой RSP был, какие параметры исключения сохранены.
unwind context (контекст раскрутки) - состояние, вычисленное после виртуального выхода из одного или нескольких кадров. Это не новое реальное выполнение кода, а реконструкция того, как выглядел бы caller context.

SEH и ручная ассемблерная вставка

Еще один источник путаницы - hand-written assembly. На x86 человек мог встретить примеры, где SEH-регистрация делалась вручную через стековые записи. На x64 так думать нельзя. Если пишется функция на ассемблере, которая должна корректно участвовать в unwind и SEH, ее нужно описать через специальные директивы ассемблера и метаданные. Иначе Windows не сможет надежно понять, как раскрутить кадр и где искать handler.

hand-written assembly (вручную написанный ассемблер) - код, написанный человеком на assembly, а не сгенерированный C/C++ компилятором. На x64 такой код обязан корректно описывать prolog/unwind-информацию, если он меняет стек, сохраняет nonvolatile-регистры или участвует в exception handling.
MASM directives (директивы MASM) - специальные указания Microsoft Macro Assembler, которые помогают описать функцию, пролог, сохранение регистров, выделение стека и SEH handler. Примеры в этой теме: .seh_proc, .seh_pushreg, .seh_stackalloc, .seh_handler, .seh_handlerdata, .seh_endproc.

Директива .seh_handler сообщает ассемблеру, что у функции есть exception/unwind handler. .seh_handlerdata позволяет разместить данные handler-а. Это важно: handler-информация не возникает сама собой только потому, что в коде есть какой-то label. Ассемблер и линкер должны создать правильные .xdata/.pdata записи, иначе системный dispatcher не увидит handler через нормальный x64-маршрут.

```
Смысловая схема, не полноценный готовый листинг:

my_func PROC FRAME
; prolog описывается .seh_* директивами
.seh_handler __C_specific_handler, @except
.seh_handlerdata
```

```
; здесь размещаются language-specific данные handler-а
.seh_code
; тело функции
my_func ENDP

Идея: на x64 недостаточно написать handler-код.
Нужно, чтобы unwind/exception metadata сообщила Windows, как до него дойти.
```

SEH и динамический код

До сих пор мы в основном говорили о PE-файле на диске. Но x64-модель допускает динамические таблицы функций. Это нужно JIT-компиляторам и runtime-ам, которые создают машинный код уже во время выполнения. Если такой код может участвовать в исключениях или stack walking, он должен зарегистрировать function table через RtlAddFunctionTable или callback-механизм. Иначе Windows не сможет найти RUNTIME_FUNCTION для адреса внутри динамического кода.

dynamic code (динамический код) - машинный код, созданный или измененный во время выполнения процесса, а не заранее лежащий как обычная .text секция PE-файла. Примеры: JIT, runtime stubs, unpacked code, generated trampolines.
JIT (Just-In-Time compilation, компиляция во время выполнения) - подход, при котором runtime генерирует машинный код на лету. Для Windows x64 такой код должен быть описан function tables, если он должен корректно поддерживать unwind, exceptions, profiling и debugging.

Это важно для reverse engineering, потому что offline PE parser не увидит все. Он может идеально разобрать .pdata в файле, но не знать о таблицах, зарегистрированных уже после старта процесса. Поэтому анализ x64 SEH бывает статическим и динамическим. Статический анализ отвечает: что лежит в PE? Динамический отвечает: какие function tables активны сейчас в процессе?

static analysis (статический анализ) - изучение файла или кода без его выполнения. Для SEH это чтение PE, .pdata, .xdata, UNWIND_INFO и handler data на диске.
dynamic analysis (динамический анализ) - изучение поведения программы во время выполнения. Для SEH это наблюдение реальных исключений, active function tables, VEH registrations, runtime-generated code и поведения dispatcher-а.

Как объяснять SEH самому себе одной связной историей

Когда терминов становится много, полезно иметь одну живую историю. Представь, что поток выполняет функцию. В функции есть обычные инструкции, и некоторые из них находятся внутри protected range. Компилятор заранее оставил в .xdata карту: если исключение случится вот здесь, вызови такой runtime handler и передай ему такие данные. Во время выполнения одна инструкция падает. Windows сохраняет EXCEPTION_RECORD и CONTEXT. По RIP она находит функцию. По функции - UNWIND_INFO. По UNWIND_INFO - handler. Handler смотрит свои данные и говорит системе, что делать. Если обработчик найден, система аккуратно разматывает стек и передает управление туда, где код действительно способен продолжить сценарий.

В этой истории нет магии. Есть адреса, диапазоны, таблицы, flags, структуры и соглашения. Но она сложнее обычного call/return, потому что работает поперек нормального потока выполнения. Именно поэтому SEH в reverse engineering одновременно труден и ценен: он показывает скрытую структуру программы, которую нельзя полностью восстановить по одним инструкциям.

Переход к VEH

Следующая тема - VEH, vectored exception handling. Она нужна сразу после SEH, потому что новичок часто смешивает эти механизмы. Оба работают с исключениями Windows, оба могут увидеть EXCEPTION_RECORD и CONTEXT, оба могут влиять на дальнейшую судьбу исключения. Но frame-based SEH идет через кадры, .pdata/.xdata, RUNTIME_FUNCTION и language-specific handlers, а VEH работает как зарегистрированный список обработчиков процесса. Если SEH - это обработка, встроенная в структуру стека и функций, то VEH - это скорее внешний наблюдатель или фильтр, который получает шанс вмешаться в поток исключений.

Для реверсера это различие принципиально. SEH часто виден через unwind metadata. VEH часто виден через вызовы AddVectoredExceptionHandler и RemoveVectoredExceptionHandler, через глобальные callback-и и поведение в отладчике. В следующем разделе мы разберем VEH отдельно, без смешивания с x64 frame-based SEH.

> Короткая опора для памяти: SEH на x64 - это не стековая цепочка x86 и не просто синтаксис __try. В реальном бинаре это путь от исключения к обработчику через EXCEPTION_RECORD, CONTEXT, ControlPc, RUNTIME_FUNCTION, UNWIND_INFO, handler flags, language-specific handler, HandlerData, DispatcherContext и возможный unwind. Именно поэтому x64 SEH нельзя изучать отдельно от .pdata/.xdata.

6.5. VEH: vectored exception handling, обработчики процесса вне кадра

В конце предыдущего раздела мы специально остановились на границе между SEH и VEH. Это правильная граница: если перепутать эти два механизма, дальше в голове снова появится старая каша. Человек видит слово exception, видит EXCEPTION_RECORD, видит CONTEXT, видит handler и быстро решает: “ну это всё одно и то же”. Но для reverse engineering такая мысль опасна. SEH на x64 мы только что разобрали как frame-based механизм, связанный с активными кадрами стека, .pdata, .xdata, RUNTIME_FUNCTION, UNWIND_INFO, language-specific handler-ами и HandlerData. VEH устроен иначе. Он тоже участвует в обработке исключений Windows, но не живет внутри конкретного кадра функции и не описывается через UNWIND_INFO этой функции.

VEH расшифровывается как Vectored Exception Handling. По-русски это можно перевести как векторная обработка исключений или обработка исключений через вектор обработчиков. Слово vectored здесь не про математический вектор в школьном смысле “стрелка с направлением и длиной”. В контексте Windows оно ближе к идее “управление направляется в зарегистрированный список callback-ов”. То есть программа заранее говорит системе: “если в процессе возникнет исключение, вызови вот эту функцию-обработчик как одного из наблюдателей”.

Сразу зафиксируем главную разницу. Frame-based SEH привязан к пути выполнения через кадры стека. VEH привязан к процессу как к целому. SEH отвечает на вопрос: “есть ли у текущего кадра или у какого-то вызывающего кадра обработчик, который относится к этому месту кода?” VEH отвечает на другой вопрос: “есть ли в процессе глобально зарегистрированные обработчики, которые должны получить шанс увидеть исключение до обычного frame-based поиска?”

Microsoft описывает vectored exception handlers как расширение structured exception handling; приложение может зарегистрировать функцию, которая будет наблюдать или обрабатывать все исключения приложения. Важная формулировка: vectored handlers are not frame-based, то есть они не основаны на кадрах. Это и есть ключ к правильному пониманию: VEH не требует, чтобы исключение произошло внутри конкретного __try-блока, и не ищется через protected ranges текущей функции.

Почему VEH нужен сразу после SEH

Порядок в книге здесь не случайный. Мы сначала разобрали SEH, потому что на Windows x64 frame-based обработка тесно связана с PE/unwind-моделью. Теперь VEH нужен как контраст. Он похож на SEH тем, что получает exception information и может влиять на дальнейшую судьбу исключения. Но он отличается тем, где зарегистрирован, как найден, когда вызывается и что показывает реверсеру.

Если SEH в x64-бинаре часто можно увидеть статически через .pdata/.xdata, то VEH чаще виден через код регистрации: вызовы AddVectoredExceptionHandler, AddVectoredContinueHandler, RemoveVectoredExceptionHandler и

RemoveVectoredContinueHandler. Это уже другой стиль анализа. Для SEH мы спрашивали: “есть ли у функции handler flag в UNWIND_INFO?” Для VEH мы спрашиваем: “зарегистрировал ли процесс глобальный callback, который будет получать исключения?”

Из этого следует важное практическое правило. Если в PE-файле нет очевидных __try/__except и нет UNW_FLAG_EHANDLER в конкретной функции, это ещё не значит, что исключения никак не участвуют в поведении программы. Программа может использовать VEH. Она может поставить глобальный обработчик, специально вызвать исключение через RaiseException, выполнить int 3, словить breakpoint exception, изменить ContextRecord и продолжить выполнение. Такой путь может почти не оставлять следа в .pdata как “handler текущей функции”.

Что значит vectored в названии

Слово vectored нужно разжевать отдельно, потому что оно звучит слишком абстрактно. В обычном языке vector - это направление или массив. В Windows API слово vectored в этом механизме означает, что есть зарегистрированный список обработчиков, через который система проходит при исключении. Программа добавляет callback в список. Когда происходит исключение, Windows вызывает эти callback-и в установленном порядке.

Callback - обратный вызов. Это функция, которую ты передаешь другой системе, чтобы она вызвала её позже при нужном событии. В обычном вызове ты сам пишешь call some_function. В callback-модели ты передаешь адрес функции системе, а система вызывает её сама, когда наступает событие. VEH handler как раз является callback-ом: ты регистрируешь его адрес, а Windows вызывает его при исключении.

Это отличается от __try/__except. В __try/__except компилятор видит исходную конструкцию, строит protected range, filter, handler body, language-specific data и связывает это с unwind-информацией функции. В VEH компилятор не обязан видеть никакого специального блока. Ты просто вызываешь API регистрации и передаешь указатель на функцию. Поэтому VEH больше похож на подписку на событие процесса.

Где VEH “живет”: не в кадре, а в процессе

Frame-based SEH живет в логике текущего стека. Если исключение случилось в потоке, Windows идёт по кадрам именно этого потока: текущая функция, caller, caller caller-а и дальше вверх. Обработчик связан с функцией и её metadata. VEH живет иначе: зарегистрированные vectored handlers принадлежат процессу. Это не значит, что они “выполняются всеми потоками одновременно”. Это значит, что любой поток процесса, в котором возникло исключение и который дошёл до соответствующей стадии dispatch, может вызвать этот общий список обработчиков.

Process-wide (на уровне процесса) - термин, который здесь очень важен. Process-wide handler не принадлежит одному стековому кадру и не исчезает при выходе из функции, которая его зарегистрировала. Если функция вызвала AddVectoredExceptionHandler, получила handle и потом вернулась, сам handler может остаться зарегистрированным, пока его явно не удалят или пока процесс не

завершится.

Отсюда первая опасность. Если handler-функция находится в DLL, а DLL выгрузили, регистрация может остаться с адресом в уже выгруженном модуле. Microsoft прямо предупреждает для AddVectoredExceptionHandler и AddVectoredContinueHandler: если параметр handler указывает на функцию в DLL и эта DLL выгружена, handler всё равно остается зарегистрированным, что может привести к ошибкам приложения. Для reverse engineering это важная мысль: lifetime handler-а должен быть согласован с lifetime кода, куда он указывает.

Lifetime (время жизни) - период, в течение которого объект, функция, модуль или регистрация остаются допустимыми. В VEH lifetime особенно важен: регистрация хранит адрес callback-а. Если код по этому адресу больше не существует, следующая попытка вызвать handler может привести к crash или к переходу в случайную память.

API регистрации: AddVectoredExceptionHandler

Главная функция для регистрации VEH называется AddVectoredExceptionHandler. Add - добавить. VectoredExceptionHandler - векторный обработчик исключений. То есть имя почти буквально говорит: “добавить обработчик в список vectored exception handlers”.

```
PVOID AddVectoredExceptionHandler(
    ULONG First,
    PVECTORED_EXCEPTION_HANDLER Handler
);
```

PVOID - pointer to void, указатель на void. В Windows API это универсальный указатель без конкретного типа данных. Здесь возвращаемое PVOID фактически является handle-ом регистрации. Его нужно сохранить, если потом ты хочешь удалить handler через RemoveVectoredExceptionHandler.

ULONG - unsigned long, беззнаковое целое типа long в Windows-типах. Параметр First задаёт порядок добавления handler-а: если First ненулевой, handler ставится как первый; если First равен нулю, handler ставится как последний. Но “первый” означает первый на момент регистрации. Если позже другой код тоже вызовет AddVectoredExceptionHandler с First != 0, новый handler может оказаться перед старым. Поэтому VEH-порядок не стоит воспринимать как вечную гарантию “мой handler всегда первый в процессе”.

PVECTORED_EXCEPTION_HANDLER - pointer to vectored exception handler, указатель на функцию обратного вызова. Это тип функции, которую Windows будет вызывать при исключении. То есть второй аргумент - не результат handler-а и не структура, а адрес функции.

Если регистрация успешна, AddVectoredExceptionHandler возвращает handle. Если не успешна, возвращает NULL. NULL - нулевой указатель, стандартный способ сказать “ничего не получилось” для многих Windows API. Для нормального кода это значит: результат регистрации нужно проверять. Для reverse engineering это значит: после вызова AddVectoredExceptionHandler полезно смотреть, сохраняется ли возвращенный handle где-то в глобальной переменной, поле структуры или

локальном состоянии. Если сохраняется, возможно, позже будет RemoveVectoredExceptionHandler.

Как выглядит VEH callback

VEH callback имеет тип PVECTORED_EXCEPTION_HANDLER. В документации он описан через placeholder-имя VectoredHandler, то есть “имя здесь условное, реальная функция у приложения может называться как угодно”. Смысловая форма такая:

```
LONG CALLBACK VectoredHandler(
    PEXCEPTION_POINTERS ExceptionInfo
);
```

LONG - 32-битное знаковое значение в Windows-типах. Handler возвращает LONG не потому, что это “обычное число результата”, а потому что через это значение он сообщает системе решение: продолжать выполнение или продолжать поиск следующего handler-а.

CALLBACK - макрос соглашения вызова. В Windows-заголовках он обычно обозначает, что функция вызывается системой по определенному calling convention. Для реверсера слово CALLBACK в прототипе говорит: эта функция не обязательно вызывается обычным прямым call из бизнес-логики программы; её может вызвать Windows как обратный вызов.

PEXCEPTION_POINTERS - pointer to EXCEPTION_POINTERS, указатель на структуру EXCEPTION_POINTERS. Эта структура содержит два главных указателя: ExceptionRecord и ContextRecord. ExceptionRecord описывает само исключение: код, адрес, флаги, параметры. ContextRecord описывает машинное состояние потока: RIP, RSP, регистры, флаги и другие архитектурные данные.

Это делает VEH очень сильным механизмом. Handler видит не просто “что-то упало”. Он получает доступ к информации о событии и к контексту выполнения. Он может прочитать ExceptionCode, посмотреть ExceptionAddress, изучить регистры, иногда изменить ContextRecord и вернуть системе решение продолжить выполнение. Именно поэтому VEH часто встречается в отладочных инструментах, защитах, антиотладке и runtime-механизмах.

Что handler может вернуть

У vectored exception handler-а основные возвращаемые значения похожи на то, что мы уже видели в SEH-фильтрах, но их нужно понимать в VEH-контексте. Документация для PVECTORED_EXCEPTION_HANDLER говорит: чтобы вернуть управление в точку, где произошло исключение, handler возвращает EXCEPTION_CONTINUE_EXECUTION; чтобы продолжить поиск handler-а, он возвращает EXCEPTION_CONTINUE_SEARCH.

EXCEPTION_CONTINUE_SEARCH - “продолжить поиск”. Handler говорит: “я увидел исключение, возможно, что-то записал или проверил, но не беру ответственность за обработку”. Тогда Windows идёт к следующему vectored exception handler-у в списке. Если vectored handlers закончились, система переходит к дальнейшей обычной обработке, включая frame-based SEH.

EXCEPTION_CONTINUE_EXECUTION - “продолжить выполнение”. Handler говорит: “я считаю, что можно продолжить выполнение в месте исключения или в изменённом контексте”. Это мощное и опасное решение. Если handler ничего не исправил, та же инструкция снова вызовет то же исключение. Поэтому нормальный handler либо меняет ContextRecord, либо исправляет состояние памяти, либо работает с исключением, для которого продолжение действительно допустимо, например с breakpoint/single-step сценариями.

Здесь появляется важная разница между “увидеть исключение” и “обработать исключение”. VEH handler может быть чистым наблюдателем: залогировать код исключения и вернуть CONTINUE_SEARCH. А может быть активным участником: изменить RIP, изменить флаги, эмулировать инструкцию, снять breakpoint и вернуть CONTINUE_EXECUTION. Для reverse engineering эти два поведения принципиально разные.

```
LONG CALLBACK MyVeh(PEXCEPTION_POINTERS info)
{
    if (info->ExceptionRecord->ExceptionCode == EXCEPTION_BREAKPOINT) {
        // теоретически можно поправить ContextRecord
        // и продолжить выполнение
        return EXCEPTION_CONTINUE_EXECUTION;
    }

    return EXCEPTION_CONTINUE_SEARCH;
}
```

Этот листинг не нужно воспринимать как готовый безопасный код. Он нужен только как форма мысли: VEH получает EXCEPTION_POINTERS, смотрит ExceptionRecord и ContextRecord, затем возвращает решение.

Порядок вызова: debugger first chance, VEH, потом frame-based SEH

Теперь надо аккуратно поставить VEH в общий маршрут исключения. Мы уже обсуждали first chance и second chance, но теперь добавим туда vectored handlers. В нормальной концептуальной модели пользовательского режима порядок можно держать так: сначала debugger получает first chance notification, если процесс отлаживается; если отладчик не обработал исключение и пропустил его в процесс, вызываются vectored exception handlers; затем начинается обычный frame-based поиск SEH; если никто не обработал, отладчик может получить second chance; если и это не спасло, исключение становится unhandled.

First chance notification - первое уведомление отладчика об исключении. Это не значит, что программа уже умерла. Это значит: “произошло исключение; отладчик, хочешь обработать его сам или отдаёшь программе?” Breakpoint exception в отладке почти всегда выглядит как first chance событие, которое отладчик нормально обрабатывает.

Second chance notification - второе уведомление отладчика, когда процесс сам не смог обработать исключение. Это уже гораздо ближе к реальному падению. Если debugger

на second chance не вмешивается, процесс обычно завершится.

```
исключение в потоке
    -> debugger first chance, если есть отладчик
    -> vectored exception handlers процесса
    -> frame-based SEH по кадрам текущего потока
    -> vectored continue handlers / continue handling paths, если применимо
    -> debugger second chance, если никто не обработал
    -> unhandled exception / завершение / WER / dump
```

Эта схема упрощённая, но полезная. Главное: VEH вызывается до обычного frame-based SEH-поиска. Поэтому VEH может увидеть исключение даже тогда, когда позже оно будет обработано обычным __try/__except. Именно поэтому VEH удобен для диагностики и инструментирования: он стоит как ранний наблюдатель перед кадрами.

Для реверсера отсюда следует очень важная вещь: если ты поставил breakpoint и видишь, что исключение кто-то “съел” до __except, возможно, это не frame-based SEH. Это может быть VEH, который получил управление раньше, изменил контекст или вернул CONTINUE_EXECUTION.

Vectored continue handlers: похожее имя, другая стадия

Рядом с AddVectoredExceptionHandler существует AddVectoredContinueHandler. Название похоже, и из-за этого его легко пропустить или смешать с обычным VEH. Но continue handler - это отдельный тип vectored handler-а, который участвует не в том же самом месте маршрута.

```
PVOID AddVectoredContinueHandler(
    ULONG First,
    PVECTORED_EXCEPTION_HANDLER Handler
);
```

Прототип выглядит почти так же: First задаёт порядок, Handler указывает на callback, возвращается handle или NULL. Удаляется такая регистрация через RemoveVectoredContinueHandler. Но смысл continue handler-а связан с фазой продолжения, когда система уже рассматривает возможность продолжить выполнение после обработки исключения.

Для начального reverse engineering достаточно держать в голове различие: vectored exception handler - ранний процессный наблюдатель исключения; vectored continue handler - процессный callback, связанный с продолжением после обработки. Оба регистрируются явно через API, оба не являются frame-based, оба могут быть интересны при антиотладке и инструментации. Но если ты видишь AddVectoredContinueHandler, не называй это автоматически обычным VEH в узком смысле AddVectoredExceptionHandler. Это соседний механизм.

Continue handler (обработчик продолжения) - callback, который вызывается на стадии продолжения обработки исключения. Он не заменяет __except и не является language-specific handler-ом. Его смысл в другом месте маршрута исключения.

Почему VEH не использует .pdata/.xdata как SEH

Теперь связуем это с главной линией книги. SEH на x64 мы разбирали через PE Exception Directory, .pdata, RUNTIME_FUNCTION, .xdata и UNWIND_INFO. VEH не ищется таким путём. Если программа зарегистрировала vectored handler, Windows не должна искать текущий RUNTIME_FUNCTION, читать handler flags и смотреть language-specific handler текущей функции только для того, чтобы вызвать VEH. VEH-список существует как процессная регистрация.

Это не значит, что .pdata/.xdata вообще не участвуют в исключении, где есть VEH. Исключение всё равно произошло в каком-то потоке и по какому-то RIP. Если VEH вернет CONTINUE_SEARCH, дальше frame-based SEH может использовать .pdata/.xdata. Если нужен stack trace внутри VEH, инструмент или сам handler может вызывать функции, которые используют unwind metadata. Если handler меняет ContextRecord, дальнейший unwind может зависеть от корректности нового RIP/RSP. Но сам факт регистрации VEH не кодируется как UNW_FLAG_EHANDLER у функции, где произошло исключение.

Это полезно помнить при статическом анализе. Поиск по .pdata не покажет тебе “все VEH”. Для VEH нужно искать вызовы API регистрации, импорты, динамическое разрешение адресов, обращения к kernel32/ntdll wrappers, сохранение handle-ов, паттерны callback-функций и поведение при исключениях.

Как VEH выглядит в PE и дизассемблере

В статическом PE-анализе VEH часто начинается с импортов. Если в Import Address Table видны AddVectoredExceptionHandler или AddVectoredContinueHandler, это сильный признак, что программа может регистрировать vectored handlers. Но отсутствие явного импорта не доказывает отсутствие VEH. Адрес API может быть получен через GetProcAddress, спрятан за wrapper-ом, импортирован по ordinal, зашифрован, разрешён динамически или вызван через runtime-слой.

Import Address Table, IAT - таблица адресов импортов. После загрузки модуля loader записывает туда реальные адреса импортированных функций. Если программа вызывает AddVectoredExceptionHandler через IAT, дизассемблер может показать call qword ptr [__imp_AddVectoredExceptionHandler]. Это удобный признак для реверсера.

Wrapper - обёртка. Это функция-посредник, которая вызывает другую функцию. Например, программа может иметь свою функцию register_exception_callbacks(), внутри которой вызывается AddVectoredExceptionHandler. Тогда прямые вызовы API есть только в одном месте, а логика регистрации размазана по более высокоуровневым wrapper-ам.

```
; условная форма x64-кода регистрации VEH
mov  ecx, 1                         ; First != 0, поставить ближе к началу списка
lea  rdx, [rip + MyVehHandler]       ; адрес callback-а
call qword ptr [__imp_AddVectoredExceptionHandler]
mov  [rip + g_VehHandle], rax        ; сохранить handle регистрации
```

Здесь видно сразу несколько признаков. Первый аргумент Windows x64 calling convention идёт в RCX, поэтому ecx = 1 задаёт First. Второй аргумент идёт в RDX, поэтому lea rdx, [rip + MyVehHandler] подготавливает адрес callback-а. Возвращаемое значение приходит в RAX. Если оно сохраняется в g_VehHandle, вероятно, программа планирует позже удалить handler.

Но оптимизация и обфускация могут сильно менять картину. Handler address может быть взят из таблицы, расшифрован, выбран условно, передан через несколько функций. Поэтому хороший анализатор не должен искать только один паттерн. Он должен строить data flow: какое значение попало во второй аргумент AddVectoredExceptionHandler? Откуда оно пришло? Указывает ли оно в исполняемую секцию? Есть ли у этой функции прототип, похожий на LONG CALLBACK handler(PEXCEPTION_POINTERS)?

Как вручную отличать VEH callback от обычной функции

VEH callback не имеет волшебной сигнатуры в машинном коде. В релизном бинаре имена могут отсутствовать. Поэтому реверсер использует контекст. Если адрес функции передаётся вторым аргументом в AddVectoredExceptionHandler, это уже сильное доказательство. Дальше можно смотреть, как функция использует свой первый аргумент.

В Windows x64 первый аргумент callback-а приходит в RCX. Для VEH это PEXCEPTION_POINTERS. Значит внутри handler-а часто будут обращения через RCX: сначала к полям EXCEPTION_POINTERS, затем к EXCEPTION_RECORD или CONTEXT. Например, handler может читать ExceptionInfo->ExceptionRecord->ExceptionCode, проверять код EXCEPTION_BREAKPOINT или EXCEPTION_ACCESS_VIOLATION, потом читать или менять ContextRecord->Rip.

```
; концептуальная картина, не точные смещения
MyVehHandler:
    ; RCX = PEXCEPTION_POINTERS ExceptionInfo
    mov rax, [rcx + ExceptionRecord_offset]
    mov edx, [rax + ExceptionCode_offset]
    cmp edx, EXCEPTION_BREAKPOINT
    jne continue_search

    mov rax, [rcx + ContextRecord_offset]
    ; изменить Rip / EFlags / registers при необходимости
    mov eax, EXCEPTION_CONTINUE_EXECUTION
    ret

continue_search:
    xor eax, eax ; EXCEPTION_CONTINUE_SEARCH
    ret
```

Это не готовый шаблон для сигнатурного поиска, а логика чтения. Если функция получает указатель, лезет в структуру, сравнивает exception code, работает с context и возвращает 0 или -1, это похоже на vectored exception handler или SEH-filter-подобный callback. Чтобы не ошибиться, нужно смотреть источник вызова: кто передал адрес этой функции? AddVectoredExceptionHandler? __C_specific_handler через scope table?

SetUnhandledExceptionFilter? Это разные уровни.

VEH и изменение ContextRecord

Одна из причин, почему VEH так интересен для reverse engineering, - handler получает ContextRecord. ContextRecord - это структура машинного состояния потока. На x64 в ней есть Rip, Rsp, регистры общего назначения, флаги, XMM-состояние и другие поля. Если handler меняет ContextRecord и возвращает EXCEPTION_CONTINUE_EXECUTION, Windows может продолжить выполнение уже с изменённым состоянием.

Это не “обычный jump” в коде. В дизассемблере ты можешь не увидеть прямую инструкцию jmp target. Вместо этого handler меняет ContextRecord->Rip на target, возвращает CONTINUE_EXECUTION, и система продолжает выполнение с нового RIP. Для статического control-flow graph такая передача управления может быть невидимой.

Control-flow graph, CFG в смысле анализа - граф потока управления программы: какие basic blocks могут переходить в какие. VEH может создавать exception-driven edges, то есть переходы, возникающие не из обычной инструкции jmp/call, а из обработки исключения и изменения контекста. Это особенно важно для обфускации: код может специально вызывать исключение, а настоящий переход управления прятать в handler-е.

Пример идеи: программа ставит breakpoint-инструкцию int 3 в середине кода. При исполнении возникает EXCEPTION_BREAKPOINT. VEH handler видит это, увеличивает Rip или переносит Rip на скрытую ветку, возвращает CONTINUE_EXECUTION. Обычный линейный анализатор видит “здесь breakpoint/падение”, а реальное выполнение идёт дальше по специальному маршруту.

VEH и антиотладка

Antidebugging (антиотладка) - набор техник, которые пытаются обнаружить отладчик, помешать отладке или изменить поведение программы под отладчиком. VEH часто встречается в антиотладке, потому что исключения тесно связаны с debugger first chance/second chance и breakpoint/single-step событиями.

Breakpoint exception - исключение, возникающее при breakpoint-инструкции, например int 3. Отладчик обычно использует такие исключения для остановки программы. Но программа тоже может намеренно генерировать breakpoint exception и смотреть, кто его обработает: отладчик, VEH, SEH или никто.

Single-step exception - исключение, связанное с пошаговым выполнением, например через trap flag. Отладчики используют single-step для выполнения одной инструкции. Защитный код может играть с этим состоянием и VEH, чтобы усложнить трассировку.

Важно: VEH сам по себе не означает malware и не означает антиотладку. Нормальные инструменты диагностики, профилировщики, crash reporters, sandbox-слои, JIT runtime-ы и защитные механизмы тоже могут использовать VEH. Но сочетание признаков может быть подозрительным: регистрация VEH, намеренные int 3 / illegal instruction / access violation, изменение ContextRecord->Rip, проверка BeingDebugged,

NtQueryInformationProcess, guard pages, single-step logic.

Для static /dynamic правильный вывод должен быть осторожным: не “обнаружен VEH, значит вирус”, а “обнаружен process-wide vectored exception handler; handler проверяет EXCEPTION_BREAKPOINT и изменяет ContextRecord->Rip; рядом присутствуют антиотладочные проверки; возможна exception-driven control flow obfuscation”. Это намного полезнее и честнее.

VEH и обычная диагностика

Чтобы не сделать из VEH демона, нужно показать нормальные сценарии. Crash reporter может поставить VEH, чтобы увидеть исключения раньше обычной обработки, собрать минимальный контекст, записать лог или инициировать дамп. Инструмент мониторинга может использовать VEH для отслеживания breakpoint/single-step событий. Runtime может использовать VEH в низкоуровневых механизмах.

Crash reporter - компонент, который собирает данные о падении: код исключения, стек, модули, регистры, minidump. VEH удобен ему тем, что handler получает EXCEPTION_POINTERS. Но писать сложную логику прямо внутри handler-а опасно. Документация для PVECTORED_EXCEPTION_HANDLER предупреждает, что handler не должен вызывать функции, которые захватывают synchronization objects или выделяют память, потому что это может вызвать проблемы; обычно handler просто обращается к exception record и возвращает управление.

Synchronization object - объект синхронизации, например mutex, critical section, event, semaphore. Если exception handler пытается захватить lock в аварийном состоянии, он может попасть в deadlock. Deadlock - взаимная блокировка, когда выполнение зависает, потому что участники ждут друг друга. Поэтому “умный” handler не должен превращаться в огромную подсистему внутри исключения.

Это важный инженерный урок: VEH даёт ранний доступ к исключениям, но чем раньше и ниже уровень вмешательства, тем осторожнее должен быть код. В момент исключения состояние процесса может быть уже повреждено. Память может быть неконсистентной, lock-и могут быть удержаны, стек может быть плохим. Поэтому handler должен быть минимальным и предсказуемым.

Удаление handler-а: RemoveVectoredExceptionHandler

Если программа зарегистрировала VEH, она должна уметь удалить его, когда он больше не нужен. Для этого используется RemoveVectoredExceptionHandler. Remove - удалить. Функция получает handle, который ранее вернул AddVectoredExceptionHandler.

```
ULONG RemoveVectoredExceptionHandler(
    PVOID Handle
);
```

Если удаление успешно, функция возвращает ненулевое значение. Если не успешно - ноль. Для reverse engineering вызов RemoveVectoredExceptionHandler важен потому,

что он показывает lifetime регистрации. Если handler ставится на короткое время вокруг чувствительного участка, программа может делать так: зарегистрировать VEH, выполнить опасный/защищённый код, удалить VEH. Если handler ставится один раз в начале и не удаляется, это process-wide наблюдатель на всю жизнь процесса.

```
PVOID h = AddVectoredExceptionHandler(1, MyVeh);
if (h != NULL) {
    // чувствительный участок
    RemoveVectoredExceptionHandler(h);
}
```

В дизассемблере такой сценарий можно узнать по сохранению handle и последующему вызову RemoveVectoredExceptionHandler с тем же значением. Если handle теряется, удалить handler позже невозможно обычным способом через этот API.

VEH, SEH и SetUnhandledExceptionFilter: три разных уровня

Рядом с VEH и SEH часто появляется SetUnhandledExceptionFilter. Его тоже нельзя смешивать с ними. SetUnhandledExceptionFilter задаёт top-level unhandled exception filter, то есть фильтр последнего уровня для необработанных исключений. Он не является тем же самым, что VEH, и не является frame-based __try/__except.

Top-level filter - фильтр верхнего уровня. Он получает шанс, когда обычные механизмы не обработали исключение. Crash reporters часто используют этот путь, но он стоит позже в маршруте, чем VEH и frame-based обработка.

Для начинающего все три механизма выглядят как “какая-то функция получает EXCEPTION_POINTERS”. Но различие такое: VEH - process-wide callback, вызываемый рано; frame-based SEH - обработка по кадрам и metadata; unhandled exception filter - поздний фильтр, когда исключение дошло до необработанного состояния. Если смешать эти уровни, ты неправильно восстановишь маршрут исключения.

```
VEH:
    AddVectoredExceptionHandler -> process-wide ранний callback

Frame-based SEH:
    __try/__except -> metadata функции, кадры стека, language-specific handler

Top-level unhandled filter:
    SetUnhandledExceptionFilter -> поздний фильтр необработанного исключения
```

Почему VEH не равен “глобальный catch”

Очень соблазнительно назвать VEH “глобальным catch на весь процесс”. Это полезная бытовая метафора, но технически она неверна. Catch в C++ имеет языковую семантику: типы исключений, объекты, stack unwinding, destructors. SEH __except имеет filter expression и frame-based protected ranges. VEH не знает ни типов C++ exceptions, ни protected ranges, ни локального смысла функции. Он получает низкоуровневое событие Windows и машинный контекст.

VEH может увидеть исключение до того, как оно дошло до catch или __except. Но “увидеть” не значит “быть языковым обработчиком”. Если VEH вернул CONTINUE_SEARCH, дальше обычные механизмы продолжают работать. Если VEH вернул CONTINUE_EXECUTION, он фактически перехватил маршрут и решил продолжить выполнение. Но это не превращает его в C++ catch.

Поэтому в декомпиляторе не нужно восстанавливать VEH как try/catch. VEH лучше описывать как зарегистрированный exception callback. Он может влиять на control flow, но его структура не совпадает с блоковой структурой исходного кода.

VEH и потоки процесса

Поскольку VEH регистрация process-wide, возникает вопрос: если handler общий для процесса, как он связан с потоками? Ответ: исключение всё равно происходит в конкретном потоке, и callback выполняется в контексте этого исключения. Но список обработчиков общий для процесса. То есть разные потоки могут вызвать один и тот же handler при своих исключениях.

Thread context - контекст потока. У каждого потока свой RIP, RSP, регистры и стек. Если VEH handler вызван из-за исключения в worker thread, его EXCEPTION_POINTERS относятся именно к этому worker thread. Handler не получает автоматически “контекст всех потоков”. Он видит событие текущего исключения.

Это важно для анализа многопоточных программ. Глобальный VEH handler может быть зарегистрирован в main thread, а сработать позже в другом потоке. Если реверсер ищет “где этот handler был установлен” и “где он сработал”, это могут быть разные стеки и разные моменты времени.

VEH и page guard / guard pages

Один из практических сценариев, где VEH может быть частью хитрой логики, - guard pages. Guard page - страница памяти с флагом PAGE_GUARD. При первом доступе к такой странице Windows генерирует исключение guard page violation. Программа может использовать это для ленивой инициализации, мониторинга доступа, защиты памяти или антиотладки.

Если VEH handler видит guard page exception, он может определить, кто обратился к странице, изменить права, восстановить guard-флаг, изменить ContextRecord или просто залогировать событие. Для обычного линейного дизассемблера это поведение плохо видно, потому что переход управления возникает из memory access, а не из явного call.

Здесь снова важно не параноить. Guard pages используются и нормальными механизмами, например для stack growth и memory management. Но если в пользовательском коде явно ставятся PAGE_GUARD и рядом регистрируется VEH, это strong signal: исключения участвуют в runtime-логике программы.

VEH и RaiseException

RaiseException - Windows API, который позволяет программе намеренно сгенерировать исключение. В контексте VEH это очень удобная связка: программа регистрирует vectored handler, затем вызывает RaiseException с собственным кодом, handler видит этот код и выполняет скрытую логику.

Software exception - программное исключение. Оно не обязательно вызвано аппаратной ошибкой вроде access violation. Программа сама попросила Windows создать исключение. Для анализа это важно: исключение может быть не багом, а намеренным механизмом управления.

```
AddVectoredExceptionHandler(1, MyVeh);
RaiseException(0xE1234567, 0, 0, NULL);
```

Если MyVeh проверяет код 0xE1234567 и выполняет специальную ветку, то exception handling становится частью protocol-а программы. Protocol здесь означает внутреннее соглашение: “если пришёл такой exception code, делаем такую логику”. В malware и защитах это может использоваться для запутывания, но и в нормальном коде программные исключения иногда применяются для диагностики или runtime-сигналов.

Что должен делать static /dynamic с VEH

Для твоего будущего static /dynamic VEH должен быть отдельной категорией анализа, а не подпунктом “нашли SEH”. У него другой источник данных и другие findings.

Первый уровень - import/API level. Инструмент должен искать явные импорты AddVectoredExceptionHandler, AddVectoredContinueHandler, RemoveVectoredExceptionHandler, RemoveVectoredContinueHandler, RaiseException, SetUnhandledExceptionFilter. Но он должен понимать: отсутствие импорта не равно отсутствие использования.

Второй уровень - call-site level. Нужно анализировать места вызова AddVectoredExceptionHandler: какое значение передано в First, какой адрес передан как Handler, сохраняется ли возвращённый handle, есть ли проверка NULL, где handler удаляется.

Третий уровень - callback level. Нужно анализировать саму функцию handler-a: читает ли она EXCEPTION_POINTERS, проверяет ли ExceptionCode, работает ли с ContextRecord, меняет ли Rip/Rsp/EFlags, возвращает ли CONTINUE_EXECUTION или CONTINUE_SEARCH, вызывает ли подозрительные API внутри handler-a.

Четвёртый уровень - behavior level. Нужно смотреть, есть ли намеренные исключения: int 3, ud2/illegal instruction, RaiseException, guard pages, memory access в protected scenarios. Если есть и они связаны с VEH, это может быть exception-driven control flow.

Пятый уровень - risk/confidence level. Обнаруженный VEH сам по себе - neutral finding. VEH + изменение ContextRecord->Rip + искусственные исключения + антиотладочные API - уже stronger evidence для защиты/обфускации. VEH + crash logging + minidump API

- может быть нормальный crash reporter.

```
Пример отчёта:

Finding: AddVectoredExceptionHandler is called.
First: nonzero, handler requested at the front of VEH list.
Handler: RVA 0x123450, inside .text, function-like code.
Handler behavior: reads ExceptionRecord->ExceptionCode.
Handler behavior: writes ContextRecord->Rip.
Return values: can return EXCEPTION_CONTINUE_EXECUTION.
Related signals: nearby int 3 instructions and RaiseException call.
Verdict: possible exception-driven control flow / anti-debug logic.
Confidence: medium, requires dynamic validation.
```

Такой отчёт полезен потому, что он показывает evidence. Он не пугает словом malware без доказательств, но и не игнорирует важный механизм.

Типичные ошибки начинающего при VEH

Первая ошибка - думать, что VEH это просто другое имя SEH. Нет. VEH является расширением общей обработки исключений Windows, но он не frame-based. Он не живёт в конкретном __try, не ищется по RUNTIME_FUNCTION текущей функции и не описывается через UNW_FLAG_EHANDLER.

Вторая ошибка - искать VEH только в .pdata/.xdata. Для VEH основной статический след - вызовы API регистрации и callback-функции. .pdata/.xdata важны для stack walking и frame-based SEH, но не являются списком VEH registrations.

Третья ошибка - считать AddVectoredExceptionHandler вредоносным признаком. Это слишком грубо. Он может быть в crash reporter-е, диагностике, защите, runtime-е, sandbox-инструменте. Вредоносность или обфускация оценивается по поведению handler-а и окружению.

Четвёртая ошибка - забывать lifetime. Если handler указывает в DLL, которую потом выгрузили, регистрация может остаться с недействительным адресом. Это может быть обычный баг, а может быть интересный артефакт в анализе нестандартного loader-а.

Пятая ошибка - не смотреть возвращаемые значения. Handler, который всегда возвращает CONTINUE_SEARCH, может быть наблюдателем. Handler, который иногда возвращает CONTINUE_EXECUTION и меняет ContextRecord, уже влияет на control flow. Это разные уровни важности.

Шестая ошибка - не учитывать debugger first chance. Если процесс отлаживается, отладчик видит исключения до VEH. Поэтому поведение под отладчиком может отличаться от поведения без отладчика. Это особенно важно при анализе антиотладки.

Мини-модель VEH для памяти

Чтобы VEH не расползался в голове, держи одну компактную модель. Программа регистрирует process-wide callback через AddVectoredExceptionHandler. Когда в любом потоке процесса возникает исключение и отладчик first chance не забирает его себе, Windows проходит по списку vectored exception handlers. Каждый handler получает EXCEPTION_POINTERS: ExceptionRecord плюс ContextRecord. Handler может вернуть CONTINUE_SEARCH и пропустить событие дальше, либо CONTINUE_EXECUTION и попытаться продолжить выполнение. Если vectored handlers не обработали исключение, дальше включается frame-based SEH по кадрам, .pdata/.xdata и language-specific handlers.

```
Регистрация:
    AddVectoredExceptionHandler(First, Handler)
        -> Handler добавлен в process-wide list

Исключение:
    debugger first chance
        -> VEH list
            -> Handler(PEXCEPTION_POINTERS)
                -> CONTINUE_SEARCH: следующий handler / потом SEH
                -> CONTINUE_EXECUTION: продолжить выполнение
        -> frame-based SEH, если VEH не обработал
```

Главная короткая формула: VEH - это не __try и не .xdata-handler текущей функции. VEH - это явно зарегистрированный process-wide callback, который получает ранний шанс увидеть исключение и может либо пропустить его дальше, либо изменить дальнейшее выполнение.

Связь с уже пройденным SEH

Теперь можно аккуратно поставить SEH и VEH рядом. SEH на x64 показывает, как Windows связывает исключение с текущей цепочкой кадров через RUNTIME_FUNCTION и UNWIND_INFO. VEH показывает другой слой: до frame-based поиска процесс может иметь глобальных наблюдателей, которые не зависят от того, какой кадр сейчас активен.

Если SEH отвечает на вопрос “какой защищённый участок текущей функции или её callers может принять это исключение?”, VEH отвечает на вопрос “какие зарегистрированные callback-и процесса хотят увидеть это исключение до обычного обхода кадров?”.

Для reverse engineering это означает, что полная карта exception behavior должна включать оба слоя. Недостаточно разобрать .pdata и сказать “мы поняли все обработчики”. Недостаточно найти AddVectoredExceptionHandler и сказать “SEH не важен”. Реальный маршрут исключения может пройти через debugger notification, VEH, frame-based SEH, unwind phase, continue handlers, top-level filter и crash reporting. Каждый слой имеет свои следы и свои ошибки анализа.

Переход к следующей теме

После VEH естественно разобрать ещё один поздний слой: unhandled exception handling и top-level filters. Если VEH - ранний process-wide наблюдатель, а frame-based SEH - обработка по кадрам, то unhandled exception filter - почти последний шанс процесса перед тем, как исключение станет настоящим crash. Там появятся SetUnhandledExceptionFilter, UnhandledExceptionFilter, Windows Error Reporting, minidump-сценарии и вопрос, почему crash reporter может видеть исключение, которое не обработал ни VEH, ни frame-based SEH.

Но перед этим важно закрепить текущий раздел одной фразой: VEH расширяет Windows exception handling не через новые .pdata/.xdata записи, а через явную регистрацию callback-ов процесса. Поэтому реверсер должен искать не только handler flags в UNWIND_INFO, но и API регистрации, callback-и, работу с EXCEPTION_POINTERS и возможное изменение ContextRecord.

Техническая база раздела 6.5

Раздел опирается на официальные материалы Microsoft по Vectored Exception Handling, AddVectoredExceptionHandler, AddVectoredContinueHandler, RemoveVectoredExceptionHandler, RemoveVectoredContinueHandler и PVECTORED_EXCEPTION_HANDLER. Важные технические утверждения: vectored exception handlers являются расширением SEH; они не frame-based; приложение регистрирует их через AddVectoredExceptionHandler; параметр First управляет помещением handler-а в начало или конец списка; callback получает EXCEPTION_POINTERS; callback может вернуть EXCEPTION_CONTINUE_SEARCH или EXCEPTION_CONTINUE_EXECUTION; handler, расположенный в выгруженной DLL, может остаться зарегистрированным и привести к ошибкам. Текст раздела написан заново в учебной подаче и связан с уже построенной моделью Windows x64 SEH/unwind.

6.6. Top-level / unhandled exception handling: последний шанс перед crash

В предыдущих разделах мы уже построили несколько этажей механизма обработки исключений Windows. Сначала был общий маршрут: исключение возникает в конкретном потоке, у него есть ExceptionRecord, ContextRecord, RIP, RSP и дальнейшая доставка. Потом мы разобрали frame-based SEH на x64: Windows идёт по активным кадрам стека, ищет RUNTIME_FUNCTION через .pdata, читает UNWIND_INFO из .xdata, при необходимости вызывает language-specific handler, например __C_specific_handler, и постепенно поднимается к caller-ам через RtlVirtualUnwind. Затем мы добавили VEH: process-wide callback-и, которые не привязаны к кадру и вызываются до обычной раскрутки стека.

Теперь нужно разобрать следующий слой - top-level / unhandled exception handling. Это уже не ранний наблюдатель вроде VEH и не нормальная frame-based обработка через __try/__except. Это почти последний рубеж. Если исключение прошло через обычные механизмы и никто не сказал “я беру это на себя”, Windows должна решить, что делать с необработанным исключением: передать его отладчику как second chance, вызвать top-level filter, показать системный диалог, дать crash reporter-у собрать dump или завершить процесс.

Слова top-level и unhandled звучат бытово, но за ними стоит конкретная техническая роль. Top-level exception filter - обработчик верхнего уровня. Unhandled exception - необработанное исключение. Это не значит “исключение вообще невозможно обработать”. Это значит: к моменту, когда система дошла до этого слоя, обычные более близкие обработчики не приняли исключение. VEH не продолжил выполнение, frame-based SEH не нашёл подходящий __except или language-specific handler отказался, C++ runtime не поймал своё исключение на нужном уровне, и теперь Windows смотрит на последнюю установленную process/thread-wide логику верхнего уровня.

Главная мысль раздела: top-level filter - это не замена SEH и не универсальный catch вокруг всей программы. Это поздний механизм, который получает управление тогда, когда исключение уже стало unhandled для обычного маршрута. Он полезен для crash reporting и диагностики, но плох как основная логика восстановления программы.

Что значит unhandled exception

Unhandled exception переводится как необработанное исключение. Слово handled значит “обработанное”, “принятое обработчиком”. Приставка un- делает отрицание: не обработанное. Но важно понять, на каком именно уровне оно “не обработано”.

Исключение может быть “не обработано” не потому, что в программе вообще нет кода обработки. Оно может быть не обработано конкретным handler-ом. Например, VEH handler увидел исключение и вернул EXCEPTION_CONTINUE_SEARCH. Для него это исключение не обработано, но поиск продолжается. SEH filter может вернуть EXCEPTION_CONTINUE_SEARCH. Для этого __try-блока исключение не обработано, но

внешний блок ещё может его принять. C++ catch может не подходить по типу, тогда runtime ищет следующий подходящий catch.

Поэтому unhandled exception в контексте top-level filter означает более позднее состояние: исключение прошло обычную цепочку поиска и не нашло того, кто готов корректно принять его на себя. Это как если человек постучался во все кабинеты по маршруту, и каждый ответил: “не ко мне”. В конце он доходит до дежурного верхнего уровня. Дежурный может собрать отчёт, вызвать стандартное поведение, дать отладчику шанс или завершить процесс, но это уже не нормальное локальное восстановление внутри конкретной функции.

Для reverse engineering это различие критично. Когда ты видишь SetUnhandledExceptionFilter, не надо сразу думать: “ага, тут главный обработчик всех ошибок, значит вся логика исключений здесь”. Нет. Это скорее “аварийная сетка” процесса. Реальная логика исключений может быть раньше: VEH, SEH, C++ EH, __finally, cleanup, runtime. Top-level filter видит только то, что дошло до конца.

Top-level exception filter: что означает “верхний уровень”

Top-level exception filter можно перевести как фильтр исключений верхнего уровня. Слово filter здесь такое же важное, как в __except-фильтрах: функция не просто “выполняет обработку”, она возвращает решение. Она получает информацию об исключении и должна сказать системе, что делать дальше.

Top-level - верхний уровень. Но “верхний” здесь не значит “самый ранний” или “самый главный в смысле приоритета”. Скорее наоборот: это уровень, к которому приходят, когда нормальные ближайшие обработчики не справились. Он стоит высоко в смысле обобщённости, а не раньше всех по времени.

Обычный frame-based handler привязан к конкретному месту программы. Например, функция открыла файл, обернула опасный участок в __try/__except и знает, как закрыть ресурс или вернуть код ошибки. Такой handler близок к смыслу операции. Top-level filter далёк от смысла. Он часто не знает, что именно пыталась сделать функция глубоко в стеке. Он видит только: в таком-то потоке, на такой-то инструкции, с таким-то кодом исключения, программа дошла до необработанного состояния.

Именно поэтому top-level filter хорошо подходит для диагностики, но плохо подходит для “исправить всё и продолжить как ни в чём не бывало”. На этом этапе стек может быть повреждён, heap может быть повреждена, lock-и могут быть удержаны, поток может упасть на invalid stack, а состояние программы может быть уже недостоверным.

SetUnhandledExceptionFilter: установка верхнего фильтра

Главная API-функция этого слоя называется SetUnhandledExceptionFilter.

Set - установить. UnhandledExceptionFilter - фильтр необработанных исключений. То есть название буквально означает: “установить фильтр, который будет использоваться для необработанных исключений”.

Microsoft описывает SetUnhandledExceptionFilter как функцию, которая позволяет приложению заменить top-level exception handler каждого потока процесса. После вызова этой функции, если в процессе, который не отлаживается, возникает исключение и оно доходит до unhandled exception filter, этот filter вызывает функцию, указанную в lpTopLevelExceptionFilter.

Сигнатура выглядит так:

```
LPTOP_LEVEL_EXCEPTION_FILTER SetUnhandledExceptionFilter(
    LPTOP_LEVEL_EXCEPTION_FILTER lpTopLevelExceptionFilter
);
```

LPTOP_LEVEL_EXCEPTION_FILTER - это тип указателя на функцию top-level exception filter. Расшифруем по частям. LP в старом Windows-стиле часто означает long pointer, то есть указатель. TOP_LEVEL_EXCEPTION_FILTER - фильтр исключений верхнего уровня. Вместе: указатель на функцию, которую Windows сможет вызвать как фильтр необработанного исключения.

Параметр lpTopLevelExceptionFilter - адрес твоей callback-функции. Если передать NULL, это означает вернуть стандартную обработку внутри UnhandledExceptionFilter. NULL - нулевой указатель, отсутствие пользовательской функции.

Возвращаемое значение SetUnhandledExceptionFilter - адрес предыдущего фильтра. Это очень важный момент. Функция не добавляет новый filter в список, как AddVectoredExceptionHandler. Она заменяет текущий top-level filter и возвращает старый. Поэтому если два компонента процесса вызывают SetUnhandledExceptionFilter, второй может перезаписать фильтр первого.

```
LPTOP_LEVEL_EXCEPTION_FILTER oldFilter;

oldFilter = SetUnhandledExceptionFilter(MyTopLevelFilter);
```

Здесь oldFilter - предыдущий filter. MyTopLevelFilter - новый filter. Если oldFilter равен NULL, значит пользовательского top-level handler-а раньше не было. Но если oldFilter не NULL, программа может сохранить его и при необходимости вызвать или восстановить позже.

Для реверсера вызов SetUnhandledExceptionFilter означает не “в процесс добавился ещё один обработчик”, а “текущий верхний фильтр процесса был заменён”. Это другое поведение, чем у VEH.

Почему SetUnhandledExceptionFilter - не AddVectoredExceptionHandler

Эти два API легко спутать, потому что оба принимают указатель на callback и оба связаны с исключениями. Но их место в маршруте разное.

AddVectoredExceptionHandler добавляет callback в список vectored handlers. Этот список вызывается рано: после first chance notification отладчику, но до frame-based unwind/search. VEH не frame-based и может наблюдать исключения независимо от текущего call frame.

SetUnhandledExceptionFilter устанавливает один top-level filter, который имеет смысл поздно: когда исключение уже дошло до unhandled exception filter. Он не является ранним наблюдателем. Он не получает каждое исключение до SEH. Он не заменяет __try/__except. Он скорее последняя пользовательская точка перед стандартным unhandled-поведением.

Упрощённо:

```
VEH:
    ранний process-wide список callback-ов
    AddVectoredExceptionHandler добавляет handler в список
    handler может увидеть много исключений до frame-based SEH

Top-level filter:
    поздний process-wide filter
    SetUnhandledExceptionFilter заменяет текущий filter
    вызывается, когда исключение дошло до unhandled layer
```

Типичная ошибка начинающего: увидеть оба API и сказать “это глобальный обработчик исключений”. Формально оба имеют process-wide оттенок, но один ранний и списочный, другой поздний и заменяемый. Для анализа это огромная разница.

Как выглядит top-level filter callback

Функция-фильтр верхнего уровня имеет сигнатуру, похожую на UnhandledExceptionFilter. Она принимает один аргумент типа LPEXCEPTION_POINTERS и возвращает LONG.

```
LONG WINAPI MyTopLevelFilter(
    struct _EXCEPTION_POINTERS* ExceptionInfo
);
```

LONG - 32-битное целое значение в Windows API. WINAPI - calling convention macro, макрос соглашения вызова Windows API. На x64 разные старые calling convention-макросы во многом сходятся к единому Windows x64 calling convention, но в исходных заголовках они всё равно остаются для совместимости и читаемости.

ExceptionInfo - указатель на EXCEPTION_POINTERS. Мы уже встречали эту структуру в VEH. Она содержит два указателя: ExceptionRecord и ContextRecord.

ExceptionRecord отвечает на вопрос “что произошло”: код исключения, адрес, флаги, параметры. ContextRecord отвечает на вопрос “в каком машинном состоянии это произошло”: RIP, RSP, регистры, флаги и другое состояние процессора.

Пример простого фильтра:

```
LONG WINAPI MyTopLevelFilter(EXCEPTION_POINTERS* info)
{
    // Минимально: записать код исключения и адрес.
    // В реальном crash reporter-е здесь часто инициируют запись minidump.
    return EXCEPTION_EXECUTE_HANDLER;
}
```

На x64 первый аргумент callback-а приходит в RCX. Значит в дизассемблере top-level filter часто будет начинаться как обычная функция, где RCX используется как указатель на EXCEPTION_POINTERS. Если функция читает [rcx + offset] как ExceptionRecord, потом читает ExceptionCode, ExceptionAddress, затем ContextRecord->Rip/Rsp, это похоже на код обработки исключения.

Но сама по себе сигнатура ещё не доказывает, что это именно top-level filter. Такой же EXCEPTION_POINTERS получают VEH callback-и и __except-фильтры через GetExceptionInformation. Поэтому контекст регистрации важен: адрес функции передали в SetUnhandledExceptionFilter или в AddVectoredExceptionHandler? Это разные маршруты.

Возвращаемые значения top-level filter

Top-level filter возвращает LONG, но это не произвольное число. Смысловые значения знакомы нам по SEH/VEH, но их эффект на этом позднем уровне нужно понимать отдельно.

EXCEPTION_EXECUTE_HANDLER имеет значение 1. Для SetUnhandledExceptionFilter-документации это означает: вернуть управление из UnhandledExceptionFilter и выполнить связанный exception handler. На практике это обычно ведёт к завершению процесса. То есть это не “всё спасли и программа живёт дальше”. Это скорее “мы приняли необработанное исключение и идём по пути завершения/обработки верхнего уровня”.

EXCEPTION_CONTINUE_EXECUTION имеет значение -1, то есть 0xffffffff в 32-битном представлении LONG. Это означает попытку продолжить выполнение с точки исключения. Microsoft прямо указывает, что filter function может изменить continuation state через EXCEPTION_POINTERS. То есть теоретически top-level filter может изменить ContextRecord и вернуть CONTINUE_EXECUTION.

EXCEPTION_CONTINUE_SEARCH имеет значение 0. Для SetUnhandledExceptionFilter это означает продолжить нормальное выполнение UnhandledExceptionFilter: учитывать SetErrorMode flags или вызвать Application Error message box.

Схема:

```
EXCEPTION_EXECUTE_HANDLER:
    filter принял исключение на верхнем уровне
    обычно путь ведёт к завершению процесса

EXCEPTION_CONTINUE_EXECUTION:
    попытаться продолжить выполнение
    допустимо только если причина исключения реально устранена

EXCEPTION_CONTINUE_SEARCH:
    не принимать исключение здесь
    продолжить стандартную unhandled-логику Windows
```

Для reverse engineering возвращаемое значение важно не меньше, чем сам факт SetUnhandledExceptionFilter. Фильтр, который всегда возвращает EXCEPTION_EXECUTE_HANDLER, часто похож на crash reporter: собрать данные и дать

процессу завершиться. Фильтр, который проверяет ExceptionCode, меняет ContextRecord->Rip и возвращает CONTINUE_EXECUTION, уже может быть частью защитной или нестандартной control-flow логики.

Почему CONTINUE_EXECUTION здесь особенно опасен

В VEH-разделе мы уже говорили, что EXCEPTION_CONTINUE_EXECUTION опасен, если причина исключения не исправлена. В top-level filter это ещё опаснее, потому что мы находимся очень поздно. Исключение уже не было обработано обычными локальными механизмами. Значит программа, скорее всего, не ожидала нормального восстановления на этом уровне.

Представь access violation из-за записи по нулевому указателю:

```
int* p = NULL;
*p = 123;
```

Если top-level filter просто вернёт CONTINUE_EXECUTION, процесс вернётся к той же инструкции, снова попытается записать по NULL и снова получит access violation. Получится цикл исключений. Чтобы продолжить, filter должен реально изменить состояние: например, изменить RIP, изменить регистр, изменить память или исправить права страницы. Но это уже хирургия по живому процессу, причём в момент, когда состояние может быть повреждено.

Поэтому в нормальной инженерной логике top-level filter используется не для продолжения, а для фиксации аварии: собрать minidump, записать лог, возможно уведомить внешнюю систему, затем дать процессу завершиться. Продолжение из top-level filter - признак очень специфичной логики, защиты, эксперимента или потенциальной ошибки проектирования.

UnhandledExceptionFilter: функция, которую часто путают с твоим filter

Теперь нужно разделить две сущности: SetUnhandledExceptionFilter и UnhandledExceptionFilter.

SetUnhandledExceptionFilter - API для установки пользовательского top-level filter-a.

UnhandledExceptionFilter - системная функция, которая выполняет стандартную логику обработки необработанного исключения. Microsoft описывает её как application-defined function? Точнее, в документации эта функция принимает EXCEPTION_POINTERS, передаёт unhandled exceptions отладчику, если процесс отлаживается, иначе может показать Application Error message box и привести к выполнению exception handler-a. Также указано, что она может вызываться только из filter expression обработчика исключений.

Сигнатура:

```
LONG UnhandledExceptionFilter(
    EXCEPTION_POINTERS* ExceptionInfo
);
```

Здесь легко ошибиться. UnhandledExceptionFilter - это не “та функция, которую ты сам зарегистрировал”. Твою функцию ты передаёшь в SetUnhandledExceptionFilter. А UnhandledExceptionFilter - системная функция верхней unhandled-логики, которая при определённых условиях вызывает установленный тобой top-level filter.

Можно представить так:

```
SetUnhandledExceptionFilter(MyFilter)
    -> сохранить MyFilter как текущий top-level filter

Позже исключение стало unhandled:
    -> системная unhandled-логика вызывает UnhandledExceptionFilter(ExceptionInfo)
        -> если нужно, внутри маршрута вызывается MyFilter(ExceptionInfo)
        -> возвращается решение: continue search / execute handler / etc.
```

Это упрощённая модель, но она убирает путаницу: SetUnhandledExceptionFilter устанавливает, UnhandledExceptionFilter участвует в поздней обработке.

Отладчик: first chance и second chance рядом с top-level filter

Чтобы понять unhandled layer, нужно снова вспомнить debugger notifications. Debugger - отладчик. Notification - уведомление. Когда процесс отлаживается, Windows сообщает отладчику о событиях, включая исключения. Для исключений часто говорят о first chance и second chance.

First chance - первый шанс отладчика. Когда исключение только возникло, отладчик получает возможность увидеть его до того, как обычная обработка процесса решит, что делать. Если отладчик говорит “продолжить и передать программе”, исключение идёт дальше: VEH, SEH, runtime handlers и так далее.

Second chance - второй шанс отладчика. Он появляется, когда исключение не было обработано программой. То есть если VEH/SEH/top-level путь не дал нормального решения, отладчик получает уже почти финальное уведомление: “программа не обработала исключение; хочешь вмешаться перед завершением?”

Microsoft в документации UnhandledExceptionFilter указывает: если процесс отлаживается, необработанные исключения передаются отладчику. В таблице возвращаемых значений EXCEPTION_CONTINUE_SEARCH описан как случай, когда процесс отлаживается и исключение должно быть передано как second chance отладчику.

Для реверсера это означает: поведение одной и той же программы под отладчиком и без отладчика может отличаться. Без отладчика top-level filter может собрать dump и показать/подавить системный диалог. Под отладчиком исключение может уйти в debugger second chance, и пользователь увидит остановку в WinDbg/x64dbg/Visual Studio.

Упрощённый маршрут:

```
исключение
    -> debugger first chance, если есть отладчик
    -> VEH
    -> frame-based SEH / language-specific handlers
```

```
-> если никто не обработал:
    -> unhandled exception logic / top-level filter
    -> debugger second chance, если процесс отлаживается
    -> стандартное crash/error-reporting поведение
```

Точный порядок деталей зависит от конкретного состояния процесса и отладчика, но для понимания книги важно: top-level filter находится поздно, около unhandled-состояния, а отладчик может получить второй шанс именно тогда, когда программа не справилась сама.

Process-wide, но выполняется в faulting thread

SetUnhandledExceptionFilter устанавливает top-level exception handler для процесса, точнее Microsoft говорит: для каждого потока процесса. Это звучит как “глобально”. Но сама callback-функция выполняется в контексте того потока, который вызвал fault.

Fault - сбой, аварийное событие. Faulting thread - поток, в котором произошло исключение. Если access violation случился в worker thread, top-level filter будет выполняться именно в этом worker thread, а не в каком-то безопасном отдельном служебном потоке.

Это очень важная инженерная деталь. Microsoft прямо предупреждает: handler, указанный через SetUnhandledExceptionFilter, выполняется в контексте потока, вызвавшего fault; это может повлиять на способность handler-а восстановиться от некоторых исключений, например invalid stack.

Invalid stack - недействительный или повреждённый стек. Если исключение связано с переполнением стека, разрушением RSP или guard page стека, handler может сам не иметь нормального стека для работы. А если handler пытается вызвать сложные функции, выделять память, брать lock-и, грузить DLL, писать в лог через сложную подсистему, он может упасть повторно или зависнуть.

Поэтому crash reporter-ы часто стараются быть осторожными: минимальная логика в аварийном потоке, сбор самых важных данных, иногда передача работы отдельному процессу. Это не “паранойя”, а следствие того, что unhandled exception layer работает уже внутри повреждённого выполнения.

SetErrorMode и системный Application Error dialog

Когда исключение остаётся необработанным и процесс не отлаживается, Windows может показать Application Error message box - системное окно ошибки приложения. В документации UnhandledExceptionFilter указано, что отображение такого окна зависит от current error mode, а флаг SEM_NOGPFAULTERRORBOX, установленный через SetErrorMode, может отключить этот диалог.

SetErrorMode - API, который задаёт режим обработки некоторых системных ошибок процесса. Error mode - режим ошибок. В нашем контексте важен не весь SetErrorMode, а идея: стандартное unhandled-поведение Windows может быть настроено так, чтобы не показывать обычное окно “программа выполнила недопустимую операцию” / Application Error.

SEM_NOGPFAULTERRORBOX можно разложить как “no general protection fault error box”, то есть не показывать коробку/окно ошибки для таких аварийных ситуаций. Исторические названия тут старые, но смысл практический: процесс может подавить стандартный диалог и полагаться на crash reporting или собственную логику.

Для reverse engineering это означает: если ты видишь SetErrorMode с SEM_NOGPFAULTERRORBOX рядом с SetUnhandledExceptionFilter или MiniDumpWriteDump, это похоже на приложение, которое хочет контролировать crash-reporting поведение. Если это защитный код, он может наоборот подавлять диалоги, чтобы пользователь не видел аварийные внутренние исключения.

Windows Error Reporting: что происходит после необработанного исключения

Windows Error Reporting, сокращённо WER, - система Windows для отчётов об ошибках приложений. Error reporting - отчётность об ошибках. Когда процесс падает из-за необработанного исключения, Windows может собрать информацию, предложить отправить отчёт, создать дамп по настройкам системы или корпоративной политики, вызвать зарегистрированные механизмы local dump и так далее.

В этой книге нам не нужно уходить глубоко в WER API, потому что наша ось - x64 exception/unwind. Но важно понимать место WER в маршруте. WER не заменяет .pdata/.xdata и не является frame-based handler-ом. Это более поздний слой диагностики после того, как выполнение уже пришло к аварийному завершению.

Если SEH/VEH/top-level filter - это механизмы внутри процесса, которые могут получить управление во время обработки исключения, то WER - это системный crash-reporting слой вокруг падения. Для реверсера это означает: dump может быть создан не самой программой, а Windows. А программа может создать свой dump через MiniDumpWriteDump в top-level filter-е.

Minidump: зачем crash reporter-у EXCEPTION_POINTERS

Minidump - минидамп, компактный файл с диагностической информацией о процессе. Он не обязан содержать всю память процесса. Обычно он включает достаточно информации, чтобы потом открыть файл в отладчике: список потоков, модули, регистры, стек, выбранные области памяти, exception record и context record.

Функция MiniDumpWriteDump из DbgHelp позволяет записать dump. Для исключений важна структура MINIDUMP_EXCEPTION_INFORMATION. Microsoft описывает её как структуру, содержащую exception information, записываемую в minidump функцией MiniDumpWriteDump. Внутри есть ThreadId, ExceptionPointers и ClientPointers.

ThreadId - идентификатор потока, который выбросил исключение. ExceptionPointers - указатель на EXCEPTION_POINTERS, то есть на ту самую пару ExceptionRecord + ContextRecord. ClientPointers - флаг, который говорит, где находятся указанные данные: в адресном пространстве вызывающей программы или целевого процесса в debugger-сценарии.

Пример общей идеи crash reporter-а:

```
LONG WINAPI MyTopLevelFilter(EXCEPTION_POINTERS* info)
{
    MINIDUMP_EXCEPTION_INFORMATION mei;
    mei.ThreadId = GetCurrentThreadId();
    mei.ExceptionPointers = info;
    mei.ClientPointers = FALSE;

    // Открыть файл dump-а и вызвать MiniDumpWriteDump.
    // После этого обычно вернуть EXCEPTION_EXECUTE_HANDLER.
    return EXCEPTION_EXECUTE_HANDLER;
}
```

Это не полный production-код. Здесь важна логика: top-level filter получает EXCEPTION_POINTERS, а minidump хочет именно эти данные, потому что они фиксируют точку аварии. Без exception context dump может быть гораздо менее полезным: отладчик покажет не тот контекст или не сможет сразу перейти к инструкции, вызвавшей исключение.

Microsoft в документации MiniDumpWriteDump отдельно предупреждает, что по возможности MiniDumpWriteDump следует вызывать из отдельного процесса, а не изнутри падающего target process, особенно если target уже нестабилен; одним из побочных эффектов может быть loader deadlock. Это хорошо совпадает с нашим инженерным выводом: в unhandled handler-е процесс уже может быть повреждён.

Loader deadlock - взаимная блокировка вокруг загрузчика. Loader lock - внутренний lock Windows loader-а, который защищает загрузку/выгрузку модулей и связанные операции. Если в аварийном состоянии вызвать сложный код, который косвенно попытается загрузить DLL или войти в loader-логику, можно зависнуть. Поэтому серьёзные crash reporter-ы часто используют отдельный helper process.

Почему top-level filter не должен быть тяжёлым

Top-level filter возникает в плохой момент. Исключение уже стало необработанным. Это значит, что программа находится в состоянии, которому нельзя полностью доверять. Поэтому тяжёлая логика внутри top-level filter-а опасна.

Тяжёлая логика - это выделение памяти через heap, форматирование больших строк, загрузка DLL, обращение к сложному логгеру, отправка HTTP-запроса, захват mutex-а, ожидание другого потока, работа с GUI, сложная сериализация. Всё это может работать в нормальном состоянии, но упасть или зависнуть в аварийном.

Heap - куча, область памяти для динамических выделений. Если исключение было вызвано heap corruption, попытка выделить память в handler-е может привести ко второму падению. Mutex - объект синхронизации. Если поток упал, удерживая lock, а handler пытается взять тот же lock или lock, который ждёт другой зависший поток, можно получить deadlock. GUI-подсистема может быть не готова, если падение произошло в момент message processing или loader activity.

Поэтому хороший top-level handler обычно минималистичен: сохранить уже доступные данные, инициировать dump, возможно уведомить отдельный процесс, вернуть решение. Чем больше “лечения” он пытается делать, тем выше шанс

превратить один crash в двойной crash или hang.

Как это выглядит в дизассемблере

Для реверсера основной статический след top-level filter-а - импорт или динамическое разрешение SetUnhandledExceptionFilter. В простом случае будет виден вызов через IAT:

```
lea  rcx, [rip + MyTopLevelFilter] ; первый аргумент: адрес callback-а
call qword ptr [__imp_SetUnhandledExceptionFilter]
mov  [rip + g_OldUnhandledFilter], rax ; сохранить предыдущий filter
```

Windows x64 calling convention говорит: первый аргумент идёт в RCX. Поэтому перед SetUnhandledExceptionFilter адрес filter-функции обычно попадёт в RCX. Возвращаемое значение приходит в RAX. Если RAX сохраняется в глобальную переменную, вероятно, программа хранит previous filter.

Если импорт скрыт, картина будет сложнее:

```
; условно
lea  rcx, [rip + str_SetUnhandledExceptionFilter]
call GetProcAddress
mov  rbx, rax
lea  rcx, [rip + MyTopLevelFilter]
call rbx
```

Здесь API не виден как прямой импорт. Программа находит адрес SetUnhandledExceptionFilter динамически через GetProcAddress и вызывает через регистр. Такой приём может быть обычной late binding-логикой, а может быть частью сокрытия импортов. Нужен контекст.

Late binding - позднее связывание. Адрес функции определяется во время выполнения, а не фиксируется в Import Table на этапе загрузки. Для reverse engineering это усложняет статический анализ: API-имя может лежать строкой, быть зашифровано, собираться по кускам или вообще искаться по hash-у.

Как отличить top-level filter от VEH callback

Оба получают EXCEPTION_POINTERS. Оба возвращают LONG. Оба могут читать ExceptionRecord и ContextRecord. Поэтому по телу функции иногда трудно сразу сказать, кто она: VEH callback, top-level filter или filter expression для __except.

Нужно смотреть путь регистрации.

Если адрес функции передаётся в AddVectoredExceptionHandler - это VEH callback. Если передаётся в SetUnhandledExceptionFilter - это top-level filter. Если адрес появляется в scope table, которую читает __C_specific_handler, это может быть SEH filter или handler-адрес. Если функция вызывается из C++ EH runtime, это уже другая модель.

Сравнение:

```
VEH callback:
    регистрируется через AddVectoredExceptionHandler
    вызывается рано, до frame-based unwind
```

```
      часть process-wide списка

  Top-level filter:
      устанавливается через SetUnhandledExceptionFilter
      вызывается поздно, когда исключение дошло до unhandled layer
      заменяет предыдущий top-level filter

  SEH filter:
      связан с __try/__except
      находится через language-specific data и __C_specific_handler
      относится к protected range функции/кадра
```

Если анализатор не различает эти пути, он будет строить неправильную карту exception behavior. Например, он может решить, что top-level filter ловит “все исключения сразу”, хотя на самом деле большинство исключений могут быть перехвачены раньше и никогда до него не дойти.

Почему разные библиотеки перезаписывают SetUnhandledExceptionFilter

В реальных программах SetUnhandledExceptionFilter часто становится местом конфликтов. Один crash reporter устанавливает свой filter. Потом runtime-библиотека, защитный модуль, SDK аналитики или игра устанавливает другой. Поскольку SetUnhandledExceptionFilter заменяет текущий filter, последний вызов может победить предыдущий.

Это объясняет странные ситуации: в коде есть crash reporter, но он “не срабатывает”. Возможно, его filter позже был заменён другим. Или наоборот: защитный модуль специально вызывает SetUnhandledExceptionFilter, чтобы перехватить unhandled crashes и скрыть стандартное поведение. Некоторые программы сохраняют previous filter и вызывают его вручную после своей обработки, но это уже вопрос дисциплины конкретного кода.

Для реверсера это значит: нужно смотреть не только факт установки, но и порядок. Кто вызывает SetUnhandledExceptionFilter первым? Кто позже? Сохраняется ли returned previous filter? Вызывается ли он? Есть ли попытки восстановить старый filter? Есть ли патчи, запрещающие другим библиотекам менять filter?

Особенно в крупных приложениях может быть несколько слоёв: CRT, crash reporter SDK, game engine, anti-cheat, protection layer, plugin system. Каждый может хотеть “последний шанс” на crash. Это не баг Windows, а следствие того, что top-level filter один и заменяемый, а не список как VEH.

Важная граница: top-level filter и C++ exceptions

C++ exception и Windows structured exception - разные уровни. В MSVC C++ исключения внутри реализации могут использовать structured exception machinery, но семантика языка C++ остаётся отдельной: throw, catch, типы объектов, destructors, std::terminate.

Top-level unhandled exception filter работает на уровне structured exceptions Windows. Он может увидеть некоторые аварийные structured exceptions, например access

violation, breakpoint, illegal instruction, stack overflow. Но не надо думать, что SetUnhandledExceptionFilter является правильным способом ловить все C++ throw по всей программе. Для C++ есть свои механизмы: try/catch, terminate handler, unexpected в старых моделях, runtime logic.

Для reverse engineering важно не смешивать: если видишь SetUnhandledExceptionFilter, это не значит, что C++ EH полностью заменён. Если видишь C++ catch, это не значит, что top-level filter не нужен. Они могут сосуществовать на разных слоях.

Stack overflow и invalid stack: особый ужас top-level обработки

Stack overflow - переполнение стека. Обычно поток получает ограниченный стек. Когда выполнение слишком глубоко рекурсивно или выделяет слишком много на стеке, поток может дойти до guard page и вызвать исключение stack overflow.

Guard page - сторожевая страница памяти. Она используется как сигнал: если поток дошёл до этой страницы, Windows понимает, что стек нужно расширить или что случилось переполнение. Но после настоящего stack overflow запас стека может быть очень маленьким. Handler, который пытается выполнить сложный код, может не иметь места на стеке.

Top-level filter выполняется в faulting thread. Если faulting thread упал из-за stack overflow, то top-level filter может стартовать на почти исчерпанном стеке. Поэтому восстановление после stack overflow требует особой осторожности, иногда специальных API и отдельного заранее зарезервированного пространства. В обычном crash reporter-е безопаснее не пытаться “лечить” поток, а собрать минимум информации и завершить.

Invalid stack - ещё хуже. Если RSP повреждён и указывает в неправильное место, обычная функция-filter может не иметь нормального стекового кадра. Вызовы, сохранения регистров, return address - всё становится ненадёжным. Поэтому Microsoft и предупреждает, что выполнение handler-а в контексте faulting thread влияет на способность восстановиться от ошибок вроде invalid stack.

Что top-level filter может сделать полезного

Несмотря на ограничения, top-level filter очень полезен для диагностики.

Первое - записать minidump. Это главный сценарий. У filter-а есть EXCEPTION_POINTERS, значит можно передать exception context в MiniDumpWriteDump и потом открыть dump в WinDbg. Для reverse engineering и debugging это бесценно: можно увидеть точный RIP, ExceptionCode, call stack, модули и часть памяти.

Второе - записать минимальный лог. Например, код исключения, адрес, thread id, module list, build version. Но лог должен быть максимально простым, без сложных зависимостей.

Третье - уведомить внешний watchdog или helper process. Watchdog - сторожевой процесс или поток, который следит за состоянием основного процесса. Helper process

может быть безопаснее, потому что он не повреждён тем же crash-ем.

Четвёртое - управлять пользовательским опытом: не показывать стандартное системное окно, а показать своё crash-сообщение или молча отправить report, если это допустимо политикой программы.

Пятое - в редких специфичных случаях изменить ContextRecord и продолжить. Но это не обычный crash-reporting сценарий, а отдельная низкоуровневая техника.

Что top-level filter не должен обещать

Top-level filter не должен обещать “спасти программу от любых ошибок”. Это ложное ожидание.

Если память повреждена, продолжение может привести к потере данных. Если поток упал внутри critical section, другие потоки могут зависнуть. Если heap corrupted, любое дальнейшее выделение может добить процесс. Если ошибка была security-sensitive, продолжение может открыть ещё более опасное состояние. Если исключение возникло из-за stack corruption, return addresses могут быть недостоверны.

Поэтому правильная формула: top-level filter - это механизм последнего наблюдения и диагностики, а не универсальная система восстановления.

Эта мысль особенно важна при чтении чужого кода. Если разработчик поставил SetUnhandledExceptionFilter и возвращает CONTINUE_EXECUTION почти для всего, это повод внимательно смотреть, что он исправляет. Возможно, это антиотладочная техника. Возможно, попытка скрыть краши. Возможно, плохая инженерия. Возможно, особый runtime, который действительно знает, что делает. Но доверять автоматически нельзя.

Как это связано с .pdata/.xdata

На первый взгляд top-level filter почти не связан с .pdata/.xdata. Он устанавливается через API, а не через UNWIND_INFO. Но связь всё равно есть.

Чтобы исключение стало unhandled, Windows уже пыталась пройти frame-based маршрут. На x64 этот маршрут использовал .pdata, RUNTIME_FUNCTION, .xdata, UNWIND_INFO и RtlVirtualUnwind. Если unwind metadata повреждена, система может неправильно пройти кадры, не найти handler или сломать stack trace. В результате top-level filter может получить исключение в ситуации, где нормальная обработка не сработала из-за плохих metadata.

Кроме того, top-level filter сам является обычной функцией. У него может быть свой RUNTIME_FUNCTION, свой UNWIND_INFO, свой пролог, свои nonvolatile-регистры. Если он упадёт внутри себя, обработка станет ещё сложнее. То есть top-level filter не живёт в .xdata как handler текущей падающей функции, но он всё равно является частью x64-кода, который подчиняется ABI и unwind-модели.

Для reverse engineering общий вывод такой: SetUnhandledExceptionFilter ищется через API/data-flow, а последствия его работы анализируются через тот же exception context, stack walking и unwind consistency.

Мини-сцена: access violation без локального обработчика

Представим простую программу:

```
LONG WINAPI CrashFilter(EXCEPTION_POINTERS* info)
{
    // условно: записать dump
    return EXCEPTION_EXECUTE_HANDLER;
}

int main()
{
    SetUnhandledExceptionFilter(CrashFilter);

    int* p = 0;
    *p = 123;
    return 0;
}
```

Что произойдёт концептуально?

Сначала main устанавливает CrashFilter как top-level filter. Затем инструкция записи по NULL вызывает access violation. Windows создаёт ExceptionRecord и ContextRecord. Если есть отладчик, он получит first chance. Затем VEH, если зарегистрированы. Затем frame-based SEH. В нашем простом коде нет __try/__except, поэтому локального handler-а нет. Поиск доходит до unhandled layer. Там системная логика вызывает top-level filter CrashFilter с EXCEPTION_POINTERS. Filter может записать dump и вернуть EXCEPTION_EXECUTE_HANDLER. После этого процесс обычно идёт к завершению.

Схема:

```
main registers CrashFilter
    -> access violation
    -> debugger first chance
    -> VEH list, если есть
    -> frame-based SEH по .pdata/.xdata, но подходящего handler-а нет
    -> unhandled exception filter
    -> CrashFilter(EXCEPTION_POINTERS)
    -> dump / log
    -> process termination
```

Обрати внимание: CrashFilter не “окружает” main как __try-блок. Он не получает управление сразу при любой ошибке до VEH/SEH. Он установлен как верхний filter для необработанных исключений.

Мини-сцена: top-level filter пытается продолжить выполнение

Теперь представим более странный вариант:

```
LONG WINAPI StrangeFilter(EXCEPTION_POINTERS* info)
{
    if (info->ExceptionRecord->ExceptionCode == EXCEPTION_BREAKPOINT) {
        info->ContextRecord->Rip += 1;
        return EXCEPTION_CONTINUE_EXECUTION;
    }
    return EXCEPTION_EXECUTE_HANDLER;
```

```
}
```

Здесь filter проверяет breakpoint exception, меняет RIP и возвращает CONTINUE_EXECUTION. Это уже не обычный crash reporter. Это изменение control flow через exception context. Если breakpoint-инструкция была длиной 1 байт, увеличение RIP на 1 может пропустить её и продолжить выполнение после неё. Но такой код зависит от точной инструкции, архитектуры, контекста и от того, кто ещё видел исключение.

Для реверсера такой pattern означает: исключение может быть частью управления, а не просто crash. Нужно искать deliberate exceptions - намеренно созданные исключения: int 3, ud2, RaiseException, guard page tricks, access violation by design. Если рядом есть top-level filter или VEH, который меняет ContextRecord, обычный control-flow graph неполон.

Почему top-level filter может не вызваться

Новичок часто думает: “Я поставил SetUnhandledExceptionFilter, значит моя функция точно вызовется при любом падении”. В реальности есть много причин, почему это не так.

Первая причина - исключение обработали раньше. VEH вернул CONTINUE_EXECUTION, SEH filter принял исключение, C++ catch поймал объект, runtime обработал событие. Тогда до unhandled filter-а ничего не дойдёт.

Вторая причина - процесс отлаживается. Отладчик может получить first chance или second chance и изменить маршрут. Поведение под отладчиком может не совпадать с поведением без него.

Третья причина - другой компонент перезаписал top-level filter позже. SetUnhandledExceptionFilter заменяет текущий filter, а не добавляет в список.

Четвёртая причина - падение настолько тяжёлое, что handler не может нормально выполниться: invalid stack, повреждённый TEB/PEB, авария в критическом системном состоянии, fail-fast сценарии, аварийное завершение через TerminateProcess или fast fail.

Пятая причина - исключение не проходит обычный SEH/unhandled маршрут, потому что процесс завершили явно, например через abort/terminate/runtime policy, или включились специальные механизмы безопасности. Здесь нужно анализировать конкретный случай, а не ждать универсальности.

Для reverse engineering это полезно: наличие SetUnhandledExceptionFilter - это намерение программы установить верхний filter, но не гарантия, что именно он всегда будет видеть все аварии.

Fail-fast и bypass обычной надежды на handler

Fail-fast - “быстро провалиться”, немедленно завершить выполнение при критическом нарушении. Windows и компиляторные runtime-ы имеют механизмы, которые при некоторых security-critical ошибках предпочитают быстро завершить процесс, а не пытаться продолжать через обычные handler-ы. Например, повреждение security cookie, CFG-нарушение, некоторые fast fail instructions и runtime abort-сценарии могут вести себя иначе, чем обычный access violation.

В этой книге нам пока не нужно глубоко разбирать fail-fast-архитектуру, но важно не строить миф: SetUnhandledExceptionFilter не является “богом над всеми падениями”. В современных системах есть аварийные пути, где продолжение нежелательно или запрещено. Иногда система специально не даёт программе мягко проглотить критическое нарушение, потому что это опасно.

Для анализатора это значит: если программа упала без вызова ожидаемого top-level filter-a, это не обязательно ошибка твоего анализа. Нужно смотреть тип исключения, механизм генерации, наличие отладчика, политики безопасности, runtime, WER, fail-fast и то, был ли filter вообще ещё установлен к моменту падения.

Практический поиск в PE: импорты, строки, xrefs

Когда анализируешь бинарь, начать можно с imports. Ищи SetUnhandledExceptionFilter, UnhandledExceptionFilter, SetErrorMode, MiniDumpWriteDump, AddVectoredExceptionHandler, RaiseException. Эти функции часто находятся рядом в crash-reporting или exception-management коде.

Если импортов нет, ищи строки: "SetUnhandledExceptionFilter", "MiniDumpWriteDump", "dbghelp.dll", ".dmp", "crash", "dump", "Unhandled", "Exception". Но строки могут быть удалены или зашифрованы.

Дальше смотри xrefs - cross references, перекрёстные ссылки. Кто вызывает SetUnhandledExceptionFilter? Какой адрес передаётся в RCX? Куда сохраняется RAX? Есть ли позже вызов предыдущего filter-a? Есть ли SetErrorMode? Есть ли загрузка DbgHelp.dll?

Пример finding-a:

```
Finding: top-level unhandled exception filter is installed.
Evidence: call to SetUnhandledExceptionFilter at RVA 0x14520.
Filter callback: RVA 0x132A0, inside .text.
Previous filter: return value saved to global g_PreviousFilter.
Callback behavior: reads ExceptionRecord->ExceptionCode and ContextRecord->Rip.
Related API: MiniDumpWriteDump, SetErrorMode(SEM_NOGPFAULTERRORBOX).
Likely purpose: crash reporting.
Risk/interest: late unhandled-exception handling, not early SEH/VEH.
```

Такой отчёт полезен, потому что он не переоценивает механизм. Он говорит: это top-level filter, он поздний, похож на crash reporter, есть evidence.

Другой пример:

```
Finding: unusual top-level exception filter control-flow behavior.
```

```
Evidence: SetUnhandledExceptionFilter installs callback at RVA 0x401870.
Callback writes to ContextRecord->Rip and returns EXCEPTION_CONTINUE_EXECUTION.
Related signals: intentional int 3 instructions in .text, RaiseException calls.
Likely purpose: exception-driven control flow or anti-debug behavior.
Confidence: medium, requires dynamic validation.
```

Здесь вывод уже другой: не просто crash reporting, а возможное управление через исключения.

Разница между crash dump и normal log

Log - обычный журнал событий программы. Он может писаться постоянно: пользователь нажал кнопку, запрос ушёл на сервер, файл открыт, операция завершилась. Crash dump - снимок аварийного состояния. Он нужен, когда программа уже не может нормально объяснить себя через обычные логи.

Top-level filter ближе к crash dump, чем к normal log. Он получает EXCEPTION_POINTERS, то есть точку аварии. Если в dump записать ExceptionRecord и ContextRecord, отладчик потом может восстановить место падения. Обычный лог может сказать “что-то пошло не так”, но без регистров, стека, модулей и memory snapshot расследование часто будет слабым.

Для reverse engineering dump особенно ценен: он связывает runtime-мир с PE-миром. В dump есть VA, модули и контексты. По VA можно получить module + RVA. По RVA можно найти RUNTIME_FUNCTION. По .pdata/.xdata можно восстановить stack trace. То есть даже top-level crash reporting возвращает нас к тем же структурам книги.

Типичные ошибки начинающего с top-level filters

Первая ошибка - думать, что SetUnhandledExceptionFilter ловит вообще всё и всегда. Нет. Он получает управление только когда исключение дошло до unhandled layer и когда другие условия не изменили маршрут.

Вторая ошибка - путать SetUnhandledExceptionFilter с AddVectoredExceptionHandler. VEH добавляет ранний callback в список. SetUnhandledExceptionFilter заменяет поздний filter.

Третья ошибка - думать, что top-level filter находится в .pdata/.xdata как SEH handler. Нет. Его адрес передаётся через API. Он сам как функция может иметь unwind metadata, но его регистрация не хранится как UNW_FLAG_EHANDLER падающей функции.

Четвёртая ошибка - считать возвращение CONTINUE_EXECUTION безопасным. Если причина исключения не устранена, программа снова упадёт или станет ещё более повреждённой.

Пятая ошибка - игнорировать previous filter. Если SetUnhandledExceptionFilter возвращает старый filter, программа может его сохранить. Это влияет на цепочку crash reporters внутри процесса.

Шестая ошибка - забывать про faulting thread. Top-level filter выполняется в потоке, где произошла авария. Если стек/heap/locks повреждены, сложная логика handler-а может

быть опасна.

Седьмая ошибка - делать вывод “malware” только по SetUnhandledExceptionFilter. Нормальные приложения используют его для crash reporting. Подозрительным становится поведение callback-а и контекст: изменение RIP, deliberate exceptions, подавление диалогов, антиотладочные проверки, скрытые импорты.

Мини-модель top-level handling для памяти

Держи простую модель:

```
SetUnhandledExceptionFilter(MyFilter)
    -> MyFilter становится текущим top-level filter-ом процесса
    -> старый filter возвращается как return value

Позже происходит исключение:
    -> debugger first chance
    -> VEH
    -> frame-based SEH / language-specific handlers / unwind
    -> если никто не обработал:
        -> unhandled exception logic
        -> если процесс не отлаживается и filter установлен:
            -> MyFilter(EXCEPTION_POINTERS)
        -> в зависимости от return value:
            -> execute handler / continue execution / normal unhandled behavior
        -> debugger second chance или Application Error / WER / termination
```

Главная короткая формула: top-level filter - это поздний process-level фильтр для исключений, которые уже не обработали обычные механизмы. Он важен для crash reporting и диагностики, но не заменяет VEH, SEH, C++ EH и x64 unwind.

Переход к следующей теме

Теперь у нас есть три крупных слоя пользовательской обработки исключений вокруг Windows x64 unwind.

Frame-based SEH - обработка по кадрам, связанная с .pdata/.xdata, RUNTIME_FUNCTION, UNWIND_INFO и language-specific handler-ами.

VEH - ранние process-wide callback-и, которые не основаны на кадрах и вызываются до unwind-раскрутки стека.

Top-level / unhandled exception handling - поздний слой, который получает шанс, когда исключение уже дошло до необработанного состояния.

Следующий логичный раздел - собрать порядок доставки исключения целиком. Не отдельными кусками, а как маршрут: debugger first chance, VEH, frame-based SEH search phase, unwind phase, vectored continue handlers, unhandled filter, debugger second chance, WER и завершение процесса. После этого у нас появится полная карта, по которой реверсер сможет смотреть на любой crash или deliberate exception и понимать, на каком именно слое сейчас находится управление.

Техническая база раздела 6.6

Раздел опирается на официальные материалы Microsoft по SetUnhandledExceptionFilter, UnhandledExceptionFilter, EXCEPTION_POINTERS, Structured Exception Handling, Vectored Exception Handling, MINIDUMP_EXCEPTION_INFORMATION и MiniDumpWriteDump. Важные технические утверждения: SetUnhandledExceptionFilter заменяет top-level exception handler для потоков процесса и возвращает предыдущий filter; пользовательский filter получает LPEXCEPTION_POINTERS и возвращает LONG; возможные значения включают EXCEPTION_EXECUTE_HANDLER, EXCEPTION_CONTINUE_EXECUTION и EXCEPTION_CONTINUE_SEARCH; UnhandledExceptionFilter передаёт unhandled exception отладчику, если процесс отлаживается, иначе участвует в стандартном Application Error поведении; handler выполняется в контексте потока, вызвавшего fault; MiniDumpWriteDump может принимать MINIDUMP_EXCEPTION_INFORMATION с EXCEPTION_POINTERS, а Microsoft рекомендует по возможности писать dump из отдельного процесса, если target уже нестабилен. Текст раздела написан заново в учебной подаче и связан с уже построенной моделью Windows x64 SEH/VEH/unwind.

6.7. Порядок доставки исключения в Windows: весь маршрут от fault до crash

В предыдущих разделах мы разобрали отдельные слои: frame-based SEH, __C_specific_handler, VEH и top-level / unhandled exception handling. Каждый слой сам по себе уже стал понятнее, но пока в голове может оставаться опасная разрозненность. Кажется, что есть несколько независимых механизмов: где-то __try/__except, где-то AddVectoredExceptionHandler, где-то SetUnhandledExceptionFilter, где-то debugger first chance, где-то .pdata и .xdata. На практике это не отдельные острова. Это один маршрут доставки исключения, где каждый слой получает управление в определённый момент и с определённой ролью.

Этот раздел нужен как фундаментальная карта. Не как “схема на память”, а как связная модель, которая позволит потом смотреть на crash dump, дизассемблер, лог отладчика или странную защиту и понимать: на каком именно этапе сейчас находится выполнение, кто уже получил шанс, кто ещё получит шанс, где нужен unwind, где .pdata/.xdata уже участвуют, а где они ещё не нужны.

Слово delivery в английском выражении exception delivery можно перевести как доставка. Это не доставка файла и не отправка сообщения по сети. Это путь, по которому Windows доставляет событие исключения от места возникновения к тем участникам, которые могут его увидеть или обработать: отладчику, vectored handlers, frame-based handlers, language-specific handlers, top-level filter-у и, если никто не помог, стандартному механизму аварийного завершения.

В этом разделе мы будем постоянно возвращаться к одной мысли: исключение - это не одномоментный прыжок в catch. Исключение - это событие, которое проходит через несколько ворот. На каждых воротах кто-то может сказать: “я обработал”, “ищите дальше”, “можно продолжать выполнение” или “никто не справился, процесс должен умереть”.

6.7.1. Почему порядок доставки важнее, чем кажется

Новичок обычно спрашивает: “какой обработчик сработает?” Более правильный вопрос: “какой обработчик получит шанс первым, в каком состоянии будет поток, и что будет, если он откажется?” Разница огромная. Один и тот же access violation может выглядеть по-разному в зависимости от того, запущена ли программа под debugger-ом, зарегистрирован ли VEH, есть ли frame-based SEH в текущем стеке, есть ли top-level filter, не вызван ли fail-fast механизм, и можно ли вообще продолжать выполнение после этого исключения.

Порядок особенно важен для reverse engineering. Если ты видишь, что программа регистрирует AddVectoredExceptionHandler, это не просто “ещё один обработчик”. Это ранний process-wide слой, который может увидеть исключение до обычного прохода по кадрам. Если ты видишь __try/__except, это другой слой: обработчик связан с кадром и будет рассматриваться во время frame-based search. Если ты видишь SetUnhandledExceptionFilter, это поздний слой: он важен, когда исключение уже не обработали обычные механизмы. Если ты видишь debugger first chance exception в x64dbg или WinDbg, это вообще ещё более ранняя точка наблюдения.

Process-wide означает “на уровне процесса”. Такой обработчик не принадлежит одному конкретному кадру стека. Он зарегистрирован для процесса и может увидеть исключение независимо от того, в какой функции оно возникло. Frame-based означает “основанный на кадрах”. Такой обработчик связан с цепочкой активных вызовов конкретного потока. Top-level означает “верхнего уровня” или “последнего уровня” в пользовательской логике: это шанс перед стандартным необработанным поведением.

Если перепутать эти уровни, можно сделать ложный вывод. Например, увидеть crash reporter и решить, что программа специально управляет control flow через исключения. Или наоборот: увидеть deliberate breakpoint exception и не заметить, что VEH меняет RIP и превращает исключение в скрытый переход. Поэтому нам нужна не только терминология, а порядок.

6.7.2. Сначала возникает событие: fault, trap или программное исключение

Маршрут начинается не с handler-а. Он начинается с события, которое нарушает обычный ход выполнения. Такое событие может появиться разными путями.

Fault - это вид процессорного исключения, при котором инструкция не смогла завершиться нормально. Типичный пример - access violation при обращении к недоступной памяти. Процессор пытался выполнить инструкцию, но память или права доступа не позволили это сделать. В таком случае важно, что RIP обычно указывает на проблемную инструкцию или связанную с ней точку. Система может теоретически вернуться к той же инструкции, если причина была исправлена.

Trap - это событие, которое часто возникает после выполнения инструкции или как результат специально организованной отладочной ситуации. Breakpoint exception через int 3 - хороший пример. Отладчик ставит или использует инструкцию, которая приводит к исключению останова. Для debugger-а это нормальный инструмент, а не обязательно ошибка программы.

Software-generated exception - программно созданное исключение. Оно возникает не потому, что процессор сам столкнулся с плохой памятью, а потому что код вызвал API вроде RaiseException. RaiseException буквально “поднимает исключение” в вызывающем потоке. Microsoft описывает его как функцию, которая raises an exception in the calling thread, то есть создаёт исключение именно в том потоке, который её вызвал.

Для книги важно не то, какой из источников “главнее”. Важно, что после оформления события Windows получает общую форму: есть ExceptionRecord и ContextRecord. Источник может быть аппаратным, программным, отладочным или защитным, но дальше механизму доставки нужны данные: что произошло и в каком машинном состоянии это произошло.

6.7.3. ExceptionRecord и ContextRecord: две половины одного события

ExceptionRecord - запись исключения. Это данные о самом событии. В ней важны код исключения, флаги, адрес исключения и дополнительные параметры. Если это access violation, параметры помогают понять, было ли это чтение, запись или выполнение и к какому адресу пытались обратиться. Если это RaiseException, параметры могли быть переданы самой программой как application-defined data, то есть данные, определённые приложением.

ContextRecord - запись контекста. Это снимок машинного состояния потока: RIP, RSP, регистры общего назначения, флаги и другие архитектурные поля. Если ExceptionRecord отвечает на вопрос “что случилось?”, то ContextRecord отвечает на вопрос “где и с каким состоянием это случилось?”.

Эти две структуры постоянно проходят через весь маршрут. Debugger смотрит на них, VEH получает доступ к ним через EXCEPTION_POINTERS, frame-based фильтры могут получить информацию через GetExceptionInformation, top-level filter тоже получает LPEXCEPTION_POINTERS. Внутри x64 unwind именно ContextRecord превращается из контекста текущей функции в контекст caller-а.

EXCEPTION_POINTERS - структура-обёртка, которая содержит два указателя: на EXCEPTION_RECORD и на CONTEXT. По-русски её можно понимать как “указатели на описание исключения”. Это не само исключение как философская сущность, а удобный пакет: вот запись события, вот состояние процессора. Через этот пакет callback-и получают возможность не только прочитать, но иногда и изменить continuation state, то есть состояние, с которого система попробует продолжить выполнение.

```
EXCEPTION_POINTERS
    ExceptionRecord -> что произошло
    ContextRecord   -> в каком машинном состоянии произошло
```

Для reverse engineering это значит: когда ты видишь handler, который принимает EXCEPTION_POINTERS, не пропускай это как обычный аргумент. Это

вход к самому ценному месту: к коду исключения, адресу fault-a, RIP, RSP и регистрам. Если handler меняет ContextRecord->Rip, ContextRecord->Rsp или другие поля и затем возвращает CONTINUE_EXECUTION, это уже может быть не crash reporting, а управление выполнением через исключения.

6.7.4. Нулевая развилка: процесс отлаживается или нет

До разговора о SEH и VEH нужно ввести debugger. Debugger - отладчик. Это внешний наблюдатель и управляющий инструмент, который Windows может уведомлять об исключениях в отлаживаемом процессе. x64dbg, WinDbg, Visual Studio debugger - примеры таких инструментов.

Когда процесс отлаживается, Windows сначала даёт отладчику first-chance notification. First chance буквально “первый шанс”. Это означает: исключение только возникло, обычные пользовательские обработчики ещё не исчерпали маршрут, и debugger первым получает возможность посмотреть событие и решить, обработать его самому или пропустить дальше.

First chance не означает “программа уже умерла”. Наоборот, это раннее уведомление. Поэтому в отладчике можно видеть много first-chance exceptions, которые программа потом сама нормально обрабатывает. Например, breakpoint, guard page, C++ runtime exceptions, deliberate RaiseException или internal probing. Если каждый first-chance event воспринимать как crash, анализ превратится в панику.

Second chance или last-chance notification - позднее уведомление отладчику. Оно появляется, когда обычный поиск обработчиков не нашёл того, кто обработал исключение. Смысл такой: “я уже пытался доставить исключение в программу, никто его не обработал; debugger, это твой последний шанс перед стандартным завершением”. Microsoft в материалах по debugging описывает first-chance notification как первое уведомление debugger-у, а если debugger не обработал и подходящий exception handler не найден, система уведомляет debugger снова как last-chance notification.

```
Очень грубо для процесса под debugger-ом:

исключение возникло
 -> debugger first chance
 -> если debugger обработал: программа может продолжить/изменить маршрут
 -> если debugger не обработал: Windows доставляет исключение дальше
 -> если программа не обработала: debugger second/last chance
 -> если debugger снова не обработал: процесс завершается
```

Для реверсера это объясняет частую ситуацию: программа “падает” в отладчике на int 3, но без отладчика идёт дальше. Возможно, debugger остановился на first chance breakpoint exception, а программа рассчитывала, что исключение дойдёт до VEH или SEH. Поэтому при анализе защит и антидебага важно понимать, где именно остановился debugger: до пользовательских handler-ов или после того, как они уже отказались.

6.7.5. VEH: ранний process-wide слой после первого шанса debugger-а

После first-chance debugger notification, если debugger не обработал исключение или процесса вообще не отлаживают, в игру может вступить VEH. VEH - Vectored Exception Handling, векторная обработка исключений. “Vectored” здесь не про SIMD-векторы и не про математику. Это историческое название списка callback-ов, зарегистрированных для процесса. Microsoft описывает vectored exception handlers как extension to structured exception handling: приложение может зарегистрировать функцию, которая будет watch or handle all exceptions for the application. Эти handlers не являются frame-based.

Callback - функция обратного вызова. Это функция, адрес которой программа заранее передала системе, чтобы система вызвала её позже при определённом событии. AddVectoredExceptionHandler регистрирует такой callback. RemoveVectoredExceptionHandler удаляет регистрацию.

Главная особенность VEH - он не привязан к текущему стековому кадру. Если исключение произошло внутри глубокой функции C, VEH не спрашивает: “есть ли у C __try?” Он вызывается потому, что зарегистрирован на уровне процесса. Поэтому VEH может увидеть исключение раньше, чем Windows начнёт искать frame-based handler через .pdata/.xdata и unwind.

Это место напрямую связывает раздел 6.5 с первой главой. В первой главе мы объясняли, что x64 frame-based поиск требует движения по кадрам и unwind. VEH стоит до этого движения. Он видит исходный faulting context, то есть контекст места падения, ещё до раскрутки стека. Именно поэтому VEH так любят диагностические инструменты, трассировка, антидебаг и некоторые защиты: он получает ранний глобальный обзор.

```
Упрощённо:

исключение
 -> debugger first chance
 -> vectored exception handlers (VEH)
 -> только потом frame-based поиск по стеку
```

Но ранний доступ не означает всемогущество. VEH должен вернуть решение. Если он возвращает EXCEPTION_CONTINUE_SEARCH, система идёт к следующему vectored handler-у или дальше к frame-based поиску. Если он возвращает EXCEPTION_CONTINUE_EXECUTION, система попытается продолжить выполнение с текущим или изменённым ContextRecord. Это опасно и имеет смысл только если handler исправил причину. Если это был access violation из-за плохого указателя, а handler ничего не исправил, та же инструкция снова вызовет исключение.

Для reverse engineering ключевой вопрос к VEH такой: он только логирует исключения или меняет маршрут выполнения? Если callback просто смотрит

ExceptionRecord->ExceptionCode и возвращает CONTINUE_SEARCH, это может быть диагностика. Если он проверяет int 3, меняет RIP, снимает guard page, переключает state machine и возвращает CONTINUE_EXECUTION, это уже активная часть control flow.

6.7.6. Почему документация иногда показывает маршрут короче

Официальные страницы Microsoft могут описывать маршрут на разных уровнях детализации. Например, документация RaiseException перечисляет общий поиск handler-а так: сначала система пытается уведомить debugger; если debugger не обработал, система пытается найти frame-based exception handler, searching the stack frames of the thread; если frame-based handler не найден или не обработал, debugger уведомляется второй раз; если и это не помогло, выполняется default handling. Отдельная страница VEH уточняет, что vectored handlers вызываются после first-chance debugger notification, но до того, как система begins unwinding the stack.

Это не противоречие, а разный масштаб карты. RaiseException даёт сжатую магистраль: debugger -> frame-based handlers -> debugger -> default. VEH-документация добавляет ранний process-wide слой между первым шансом debugger-а и frame-based unwind/search. Для нашей книги важно держать именно расширенную карту, потому что реверсер будет видеть AddVectoredExceptionHandler в бинаре и должен понимать, куда он вставляется.

Иначе можно сделать ошибку: увидеть в документации RaiseException “frame-based handlers” и решить, что VEH где-то после SEH. На самом деле VEH как vectored exception handlers стоит раньше frame-based unwinding-а. Это принципиально: VEH получает шанс до того, как стек начнут раскручивать и до того, как __C_specific_handler будет разбирать scope tables конкретного кадра.

6.7.7. Frame-based SEH search: когда начинается движение по кадрам

Если debugger на first chance не обработал исключение, и vectored exception handlers не остановили маршрут, Windows переходит к frame-based search. Frame-based search - поиск обработчика по кадрам стека. Это тот слой, где на x64 начинают работать наши PE/unwind структуры: .pdata, RUNTIME_FUNCTION, .xdata, UNWIND_INFO, RtlLookupFunctionEntry и RtlVirtualUnwind.

Слово frame здесь означает stack frame, кадр стека. Кадр - это состояние одного активного вызова функции. Если main вызвал A, A вызвала B, B вызвала C, и исключение произошло в C, frame-based search начинается с кадра C. Если C не обработала, Windows виртуально поднимается к B. Если B не обработала -

к A. И так далее.

Но на Windows x64 “подняться к B” не означает просто прочитать какое-то next-поле из стека, как в классическом x86 SEH-chain объяснении. На x64 нужно по текущему RIP найти RUNTIME_FUNCTION, по нему найти UNWIND_INFO, понять, не находится ли RIP в prolog или epilog, посмотреть flags, вызвать language-specific handler, если он есть, а если обработчик не подошёл, выполнить virtual unwind к caller context.

```
Frame-based x64 search в живой модели:

ContextRecord.Rip -> текущий ControlPc
RtlLookupFunctionEntry(ControlPc) -> RUNTIME_FUNCTION
RUNTIME_FUNCTION.UnwindInfoAddress -> UNWIND_INFO
UNWIND_INFO flags -> есть ли EHANDLER/UHANDLER/CHAININFO
если есть EHANDLER и место подходящее -> вызвать language-specific handler
если не обработал -> RtlVirtualUnwind -> caller ContextRecord
повторить
```

Вот здесь связывается почти вся книга. PE нужен, потому что Exception Directory ведёт к .pdata. .pdata нужна, потому что RUNTIME_FUNCTION говорит, где функция начинается, заканчивается и где лежит unwind info. .xdata нужна, потому что UNWIND_INFO объясняет кадр. UNWIND_CODE нужен, потому что без него нельзя восстановить RSP и nonvolatile-регистры. __C_specific_handler нужен, потому что Windows сама не знает, как MSVC закодировал __try/__except в language-specific data.

Поэтому frame-based search - это не просто “искать catch”. Это центральная точка, где статические данные PE становятся runtime-механизмом. Байты, лежавшие в файле как .pdata/.xdata, внезапно используются для принятия решения во время живого исключения.

6.7.8. Search phase: опрос, а не разрушение стека

Первую часть frame-based обработки удобно называть search phase, фазой поиска. Search - поиск, phase - этап. В этой фазе Windows пытается выяснить, есть ли кадр, который готов обработать исключение. Важно: search phase сама по себе не означает, что стек уже реально разобран и cleanup уже выполнен. Это прежде всего опрос.

Почему это важно? Представь цепочку main -> A -> B -> C, исключение в C, обработчик в A. Windows не должна сразу уничтожать C и B, пока не знает, что A действительно примет исключение. Сначала надо спросить C, потом B, потом A. Только когда A сказал “да, я обработаю”, появляется цель для реальной раскрутки.

Language-specific handler, например __C_specific_handler, в search phase получает ExceptionRecord, EstablisherFrame, ContextRecord и DispatcherContext. Он смотрит свои данные: попадает ли ControlPc в protected range, есть ли filter,

что filter возвращает. Если filter возвращает EXCEPTION_CONTINUE_SEARCH, этот кадр отказывается. Если filter возвращает EXCEPTION_EXECUTE_HANDLER, этот кадр найден как цель обработки.

Protected range - защищённый диапазон кода. Это не обязательно отдельная функция. Это часть функции, соответствующая __try-блоку или другой языковой protected area. Handler должен понять: исключение произошло внутри области, которую этот handler имеет право рассматривать, или нет. Если RIP вообще не попадает в protected range, handler не должен ловить событие просто потому, что функция где-то рядом имеет __try.

Filter - фильтр. Это выражение или код, который решает: это исключение моё или нет. Filter может смотреть ExceptionCode, параметры access violation, адрес fault-a, состояние программы, флаги. Он не является handler body. Handler body - это блок, куда управление пойдёт, если filter выбрал обработку.

6.7.9. Три решения: continue search, execute handler, continue execution

В маршруте исключений постоянно встречаются три смысловых решения. Они появляются в SEH filters, VEH callbacks, top-level filters, хотя детали применения зависят от слоя.

EXCEPTION_CONTINUE_SEARCH означает “продолжайте поиск”. Это самый безопасный отказ. Handler говорит: я посмотрел на исключение, но не буду его обрабатывать. Для VEH это означает переход к следующему vectored handler-у или дальше к frame-based search. Для frame-based фильтра это означает подъём к следующему кадру. Для top-level filter-а это означает отдать управление обычному unhandled behavior.

EXCEPTION_EXECUTE_HANDLER означает “выполнить обработчик”. В SEH filter-е это значит: этот protected block принимает исключение, и после нужной unwind phase управление должно попасть в handler block. В top-level filter-е это обычно связано с выполнением стандартного связанного обработчика и, как правило, с завершением процесса. Важно не механически переносить смысл между всеми слоями: число то же, но место в маршруте разное.

EXCEPTION_CONTINUE_EXECUTION означает “попробовать продолжить выполнение”. Это самый опасный вариант. Система попытается вернуться к точке исключения или к изменённому continuation state. Continuation state - состояние продолжения: значения регистров, RIP, RSP и другое, что определяет, откуда и как поток продолжит работу. Если handler исправил ContextRecord или состояние памяти, продолжение может быть осмысленным. Если не исправил - будет повторное исключение или ещё более повреждённое выполнение.

```
Три базовых смысла:
```

```
CONTINUE_SEARCH     -> я не беру это исключение, ищите дальше
EXECUTE_HANDLER     -> я беру это исключение, переходите к обработчику
CONTINUE_EXECUTION  -> причина устранена или контекст изменён, продолжайте выполнение
```

Для реверсера особенно важна третья ветка. Код, который меняет ContextRecord->Rip и возвращает CONTINUE_EXECUTION, может скрывать настоящий control flow. В исходном графе дизассемблера перехода нет, но runtime переход есть: исключение привело к handler-у, handler изменил RIP, и поток продолжил с другого адреса. Это не обычный jmp, но для программы эффект похож на неявный переход.

6.7.10. Если handler найден: начинается unwind phase

Когда search phase нашла кадр, который принимает исключение, маршрут не заканчивается мгновенным прыжком. Начинается unwind phase - фаза раскрутки. Unwind здесь означает не просто “прочитать стек”, а контролируемо покинуть промежуточные кадры между местом исключения и кадром обработчика.

Вернёмся к цепочке main -> A -> B -> C. Исключение произошло в C, обработчик найден в A. Между C и A лежит B. Нельзя просто поставить RIP на handler внутри A и оставить стек как будто C и B всё ещё активны. Нужно привести выполнение к состоянию, в котором кадры C и B считаются покинутыми. Если у этих кадров есть termination handlers, __finally, C++ destructors или cleanup-логика, она должна получить шанс выполниться.

Termination handler - завершающий обработчик. Он не обязательно ловит исключение. Его роль - выполнить код завершения при выходе из protected region или кадра: освободить ресурс, закрыть handle, разблокировать mutex, откатить состояние. В C это может быть __finally. В C++ похожую роль для объектов играют деструкторы, хотя C++ exception handling имеет свой runtime-слой.

На x64 unwind phase снова использует .pdata/.xdata. Чтобы покинуть кадр, нужно знать, что функция сделала со стеком и nonvolatile-регистрами. UNWIND_INFO даёт эти правила. Если в UNWIND_INFO установлен UNW_FLAG_UHANDLER, language-specific handler может быть вызван во время unwind для cleanup/termination поведения.

Здесь видно, почему мы так много говорили о prolog, epilog, RSP, nonvolatile registers. Если функция сохранила RBX и выделила 0x80 байт на стеке, unwind phase должна восстановить состояние caller-а не “примерно”, а формально. Иначе handler в A получит ложный контекст, а cleanup может работать с неправильным стеком.

6.7.11. Почему unwind phase не равна обычным ret

Обычный возврат из функции происходит через инструкции эпилога и ret. Функция сама выполняет код: add rsp, ..., pop rbx, ret. Unwind phase отличается: система моделирует и выполняет выход из кадров по правилам exception machinery. Некоторые пользовательские инструкции тела функции уже не будут выполнены. Код после fault-а в C может никогда не дойти до нормального return. Но cleanup, который должен быть выполнен при необычном выходе, может быть вызван специально.

Это важно для понимания безопасности и ресурсов. Если программист полагался только на “в конце функции я вызову CloseHandle”, но исключение перескочило через этот участок, handle может утечь. Если используется __finally или RAII в C++, cleanup может выполниться во время unwind. Поэтому exception handling - это не только “куда перейти при ошибке”, а ещё и “как корректно покинуть уже активные уровни”.

Для reverse engineering это означает: не вся логика cleanup будет видна как обычная последовательность после call dangerous. Часть пути включается только при unwind. Если декомпилятор плохо восстанавливает EH, ты можешь не увидеть, почему при исключении вызывается определённый cleanup block. Нужно смотреть не только на обычный control flow graph, но и на exception metadata.

6.7.12. Vectored continue handlers: поздний vectored-слой, который легко спутать с VEH

Рядом с AddVectoredExceptionHandler существует AddVectoredContinueHandler. Названия похожи, поэтому их легко перепутать. Vectored exception handler - ранний обработчик, который вызывается после first-chance debugger notification и до начала frame-based unwinding. Vectored continue handler - отдельный вид vectored callback-а, связанный с продолжением обработки, а не с самым ранним просмотром исключения.

На уровне практической карты для реверсера важно различать две регистрации. AddVectoredExceptionHandler говорит: “хочу увидеть исключения рано”. AddVectoredContinueHandler говорит: “хочу участвовать в continue-этапе”. Обе регистрации process-wide, обе возвращают handle, который потом можно удалить через соответствующий Remove-функционал, но их место в маршруте не одинаковое.

Точную внутреннюю реализацию Windows не нужно превращать в догму по псевдокоду из интернета. Для нашей фундаментальной модели достаточно закрепить: vectored exception handlers - ранний слой перед frame-based unwind/search; vectored continue handlers - не то же самое, их нужно искать отдельно и не приписывать им роль __try/__except. Если в бинаре есть обе регистрации, это уже сигнал, что автор кода тонко работает с exception pipeline, а не просто поставил crash reporter.

Reverse engineer должен задавать простые вопросы: какой именно API вызывается? AddVectoredExceptionHandler или AddVectoredContinueHandler? Где callback? Что он возвращает? Меняет ли EXCEPTION_POINTERS? Удаляется ли handler через Remove...? Находится ли callback в DLL, которая может быть выгружена? Microsoft прямо предупреждает для vectored handlers: если handler указывает в DLL и DLL выгружена, handler остаётся зарегистрированным, что может привести к ошибкам. Это тоже практический след: регистрация живёт отдельно от жизненного цикла кода, если разработчик не удалил её явно.

6.7.13. Если никто не обработал: second chance debugger

Если frame-based search не нашёл подходящего handler-а или все handlers отказались, исключение становится необработанным на этом уровне. Но если процесс отлаживается, Windows ещё раз уведомляет debugger. Это second chance или last chance.

Second chance означает: пользовательские механизмы уже не помогли. Это намного ближе к настоящему crash-у, чем first chance. В отладчике second-chance access violation обычно означает: если debugger не вмешается, процесс будет завершён. Но даже здесь важно слово “обычно”: debugger может изменить контекст, обработать событие или сделать другие действия. Для обычного пользователя без debugger-а этот слой вообще не виден.

Почему first chance и second chance так важны для антидебага? Потому что программа может рассчитывать на один маршрут без debugger-а и получать другой маршрут под debugger-ом. Если debugger останавливается на first chance и не пропускает событие правильно, программа может выглядеть “сломавшейся”, хотя без debugger-а её VEH или SEH обработал бы исключение. Некоторые защиты специально используют это: вызывают исключения, проверяют, дошли ли они до нужного handler-а, измеряют поведение debugger-а.

```
Под debugger-ом важно различать:

first chance  -> раннее уведомление, программа ещё может обработать
second chance -> обработчики программы не справились, это последний шанс debugger-а
```

6.7.14. Top-level / unhandled layer: последний пользовательский шанс

Если исключение не обработано frame-based механизмами и debugger не забрал его на last chance, маршрут приходит к top-level / unhandled exception handling. Этот слой мы подробно разобрали в разделе 6.6. Здесь повторим его роль в общей карте.

SetUnhandledExceptionFilter устанавливает top-level exception filter для процесса. Top-level filter не является ранним VEH и не является frame-based

handler-ом текущей функции. Он получает шанс, когда исключение дошло до unhandled layer. Поэтому он хорошо подходит для crash reporting: записать dump, сохранить минимальную диагностику, возможно показать пользователю сообщение. Но он плохо подходит как универсальный механизм восстановления программы.

Почему плохо подходит для восстановления? Потому что к этому моменту многое уже пошло не так. Исключение не обработали ближайшие уровни. Поток находится в состоянии аварии. Возможно, стек повреждён. Возможно, heap повреждён. Возможно, locks удерживаются. Microsoft подчёркивает, что filter, установленный через SetUnhandledExceptionFilter, выполняется в контексте потока, вызвавшего fault, и это может влиять на способность восстановиться, например при invalid stack.

В общей карте top-level filter - позднее место. Если ты видишь его в бинаре, сначала думай не “скрытый control flow”, а “crash reporter / final diagnostics”. Подозрение появляется, если callback делает необычные вещи: меняет ContextRecord, продолжает выполнение, подавляет стандартные механизмы, скрывает crash, взаимодействует с debugger-ом, строит сложную state machine.

6.7.15. Default handling, WER и завершение процесса

Если никто не обработал исключение, остаётся standard/default handling. Default - поведение по умолчанию. Для большинства необработанных исключений итогом становится завершение процесса. Документация RaiseException описывает, что если debugger не обработал и связанный handler не найден, система предоставляет default handling based on exception type; for most exceptions default action is to call ExitProcess.

ExitProcess - системное завершение процесса. Это уже не “обработать ошибку и продолжить”. Это финальная стадия, где процесс закрывается. Перед этим в обычной Windows-среде могут участвовать Application Error dialog, Windows Error Reporting, сбор дампа, корпоративные политики, настройки error mode. Но для фундаментальной модели важно: если маршрут дошёл сюда, нормальная пользовательская обработка исключения не спасла выполнение.

WER - Windows Error Reporting, отчётность об ошибках Windows. Это инфраструктура, которая может собирать сведения о падении, показывать диалог, отправлять отчёты, создавать dumps в зависимости от настроек. Для реверсера WER важен не как первый участник обработки, а как поздний слой после того, как приложение уже не обработало исключение обычными средствами.

Если анализируешь crash dump, созданный WER или своим top-level filter-ом, ты опять возвращаешься к PE/unwind модели. В dump есть faulting thread, ExceptionRecord, ContextRecord, список модулей. По RIP находишь module + RVA, по Exception Directory находишь .pdata, по RUNTIME_FUNCTION - .xdata, по

UNWIND_INFO - caller context. Даже финальный crash возвращает нас к тому же фундаменту.

6.7.16. Особый путь: noncontinuable exception

Noncontinuable exception - непродолжаемое исключение. Это исключение, после которого нельзя законно продолжить выполнение. В RaiseException есть флаг EXCEPTION_NONCONTINUABLE. Документация Microsoft указывает: если попытаться продолжить выполнение после noncontinuable exception, возникает EXCEPTION_NONCONTINUABLE_EXCEPTION.

Смысл простой: не все исключения можно “починить” и продолжить. Иногда сама природа события говорит: продолжение запрещено. Если handler в такой ситуации возвращает CONTINUE_EXECUTION, он не становится героем, который спас процесс. Он нарушает контракт, и система создаёт новое исключение о попытке продолжения непродолжаемого исключения.

Для реверсера noncontinuable flag важен как часть ExceptionRecord. Если видишь RaiseException с EXCEPTION_NONCONTINUABLE, а потом handler пытается продолжить, это странный маршрут. Возможно, автор проверяет поведение handler-ов. Возможно, это ошибка. Возможно, это обфускация или anti-analysis. Но вывод надо делать по evidence: код исключения, flags, return value handler-a, изменение контекста.

6.7.17. Особый путь: fail-fast

Есть исключения и API, которые специально обходят обычные обработчики. В заголовке errhandlingapi.h Microsoft описывает RaiseFailFastException как функцию, которая raises an exception that bypasses all exception handlers, frame or vector based. Fail-fast можно перевести как “быстрое аварийное завершение”. Идея: состояние настолько опасное, что нельзя доверять обычной попытке восстановления.

Почему это важно в книге про SEH/VEH? Потому что нельзя думать, что любой fault обязательно пройдёт через все слои: debugger, VEH, SEH, top-level. Некоторые механизмы безопасности и критические ошибки специально не дают frame/vector handlers обычного шанса. Это сделано не для удобства реверсера, а для безопасности и надёжности: когда обнаружено нарушение целостности, продолжение может быть опаснее завершения.

В будущем, когда мы будем говорить об анализаторе, это станет отдельной проверкой: если код вызывает RaiseFailFastException или компиляторная защита приводит к fail-fast поведению, не надо искать обычный __try/__except route. Такой путь намеренно короткий. Он сообщает: “не пытайтесь восстановиться через пользовательские handlers”.

6.7.18. Особый путь: guard pages и deliberate exceptions

Guard page - сторожевая страница памяти. Page guard позволяет сделать страницу такой, что первое обращение к ней вызывает guard page exception, после чего состояние страницы может измениться. Этот механизм может использоваться легитимно, например для роста стека или контроля доступа к памяти, а может использоваться защитами и отладочными инструментами.

Deliberate exception - намеренно вызванное исключение. Это не баг, а приём. Программа может выполнить int 3, вызвать RaiseException, обратиться к guard page, выполнить illegal instruction или создать access violation, чтобы получить управление в handler-е. Для обычного линейного дизассемблера это может выглядеть как авария. Для runtime это может быть часть control flow.

Именно здесь полная карта доставки становится практической. Если deliberate exception должно попасть в VEH, debugger first chance может изменить поведение. Если exception должно пройти до __C_specific_handler, VEH может перехватить его раньше. Если оно должно дойти до top-level filter, frame-based handler может неожиданно забрать его. Разработчик такой схемы обязан понимать порядок, иначе его защита будет нестабильной. Реверсер, в свою очередь, должен восстановить, на какой слой рассчитан трюк.

6.7.19. Полная карта маршрута в одном виде

Теперь соберём маршрут целиком. Это не исходный код Windows и не обещание, что каждый внутренний случай всегда проходит абсолютно все стрелки. Это рабочая модель для reverse engineering обычных пользовательских исключений в Windows x64.

```
Исключение возникло в конкретном потоке
    -> сформированы ExceptionRecord и ContextRecord
    -> если процесс отлаживается: debugger first chance notification
        -> debugger может обработать или пропустить
    -> vectored exception handlers (AddVectoredExceptionHandler)
        -> process-wide callbacks, не frame-based
        -> могут CONTINUE_EXECUTION или CONTINUE_SEARCH
    -> frame-based search по стеку текущего потока
        -> RtlLookupFunctionEntry(ControlPc)
        -> RUNTIME_FUNCTION из .pdata / function table
        -> UNWIND_INFO из .xdata
        -> language-specific handler, например __C_specific_handler
        -> filters / protected ranges / handler data
    -> если handler найден: unwind phase
        -> cleanup / termination handlers / __finally / destructors
        -> переход к handler block
    -> если handler не найден или не обработал:
        -> поздние continuation/unhandled стадии
        -> debugger second/last chance, если процесс отлаживается
        -> UnhandledExceptionFilter / top-level filter
        -> WER / Application Error / ExitProcess для большинства исключений
```

Главное в этой карте - не стрелки сами по себе, а изменение состояния. До VEH стек ещё не раскручен. Во frame-based search Windows виртуально

поднимается по кадрам. После найденного handler-а начинается unwind phase, где промежуточные кадры покидаются. Top-level filter появляется поздно, когда обычные handlers уже не спасли исключение. Debugger может вмешаться рано и поздно, поэтому поведение под отладчиком может отличаться от поведения без него.

6.7.20. Где в этом маршруте живут .pdata и .xdata

Важно ещё раз зафиксировать: .pdata и .xdata не участвуют во всех слоях одинаково. Debugger first chance может получить исключение ещё до того, как Windows начала frame-based unwind. VEH тоже вызывается до обычного unwind stack search. Top-level filter не регистрируется через .pdata/.xdata. Но frame-based search и unwind phase на x64 без function tables не живут.

Когда маршрут дошёл до stack frame search, Windows должна ответить: какая функция содержит текущий ControlPc? Для обычного PE32+ модуля ответ находится через Exception Directory и .pdata. Там лежат RUNTIME_FUNCTION entries. Каждая entry говорит: BeginAddress, EndAddress, UnwindInfoAddress. Это привязывает текущий RIP/RVA к функции и к её unwind info.

Когда нужно понять, что делать с кадром, используется .xdata. Там лежит UNWIND_INFO. Он говорит: размер пролога, unwind codes, frame register, flags. Если flags говорят EHANDLER или UHANDLER, после unwind codes лежит адрес language-specific handler и handler data. Так __C_specific_handler получает возможность понять __try/__except/__finally.

Когда handler не подошёл, RtlVirtualUnwind использует UNWIND_INFO, чтобы превратить текущий ContextRecord в caller ContextRecord. Это и есть движение по кадрам. Поэтому .pdata/.xdata - не “таблица где-то рядом с PE”. Это механизм, который делает frame-based часть маршрута возможной.

6.7.21. Где в этом маршруте живёт __C_specific_handler

__C_specific_handler не является глобальным первым обработчиком всех исключений. Он не стоит перед VEH. Он не является top-level filter-ом. Он появляется тогда, когда frame-based search дошёл до кадра функции, чья UNWIND_INFO говорит: у этой функции есть language-specific handler, и этим handler-ом является __C_specific_handler.

Windows в этот момент не говорит: “я нашла __except”. Она говорит: “у текущего кадра есть handler function; вызову её и передам ExceptionRecord, EstablisherFrame, ContextRecord, DispatcherContext”. Уже __C_specific_handler читает C-specific language data: protected ranges, filters, handler targets, finally/termination information. Он знает формат данных, созданных MSVC для SEH-расширений C/C++.

Поэтому правильная роль __C_specific_handler такая: посредник между общим Windows x64 unwind dispatcher-ом и конкретным форматом SEH-данных MSVC. Если в бинаре есть __C_specific_handler, это не значит, что он сам содержит пользовательскую обработку. Пользовательская логика лежит в filter-ах и handler blocks, на которые указывают language-specific tables.

6.7.22. Где в этом маршруте живут C++ exceptions

Мы пишем книгу прежде всего про Windows x64 exception/unwind фундамент, а не полную реализацию C++ EH. Но важно не создать ложную картину: C++ try/catch тоже использует системную инфраструктуру unwind, но имеет свой runtime и свои language-specific данные. C++ throw - это не просто RaiseException в бытовом смысле и не просто __try/__except. Это отдельная языковая семантика: типы исключений, catch matching, destruction of objects, runtime helper-ы, таблицы состояния.

Связь такая: базовая Windows x64 машина умеет доставлять исключения и раскручивать стек по metadata. C++ runtime поверх этой машины реализует свои правила: какие catch подходят, какие destructors вызвать, как переносить объект исключения. Поэтому изучение .pdata/.xdata и RtlVirtualUnwind не заменяет изучение C++ EH, но даёт фундамент, без которого C++ EH в бинаре будет казаться магией.

Для реверсера практический вывод: не надо смешивать SEH, VEH, top-level filter и C++ catch в один термин “исключения”. Все они рядом, все могут участвовать в аварийном маршруте, но каждый имеет свой слой. Если смешать, невозможно правильно объяснить, почему один handler сработал раньше другого.

6.7.23. Как читать лог debugger-а через эту карту

Представь, что x64dbg или WinDbg показывает first chance exception 0xC0000005. Что делать? Не писать сразу “краш”. Сначала спросить: debugger остановился на first chance или second chance? Если first chance, программа ещё могла бы обработать событие. Нужно понять, если пропустить exception, дойдёт ли оно до VEH/SEH и продолжится ли выполнение.

Если после пропуска first chance срабатывает VEH и программа идёт дальше, значит exception был частью управляемого маршрута. Если после пропуска Windows проходит frame-based handlers и попадает в __except, это обычная SEH-обработка. Если после пропуска debugger получает second chance, значит пользовательские handlers не обработали. Если затем появляется WER или процесс завершается, это unhandled crash.

```
Вопросы при остановке в debugger-е:

1. Это first chance или second chance?
2. Код исключения какой: access violation, breakpoint, single step, custom RaiseException?
```

```
3. Есть ли зарегистрированные VEH?
4. Есть ли frame-based handler у текущего/внешнего кадра?
5. Что показывает .pdata/.xdata для faulting RIP?
6. Менял ли handler ContextRecord?
7. Исключение исчезает при Continue или повторяется в той же инструкции?
```

Такая проверка превращает отладку из гадания в маршрутное мышление. Ты не просто нажимаешь Continue. Ты понимаешь, какой слой получит управление следующим.

6.7.24. Как читать дизассемблер через эту карту

В статическом анализе нет живого debugger event-a. Есть код и metadata. Но карта маршрута всё равно применима. Если видишь вызов AddVectoredExceptionHandler, ищи callback и анализируй его как ранний global exception observer. Если видишь SetUnhandledExceptionFilter, анализируй callback как поздний crash/unhandled layer. Если видишь RaiseException, int 3, ud2, обращения к guard pages или deliberate access violation, ищи, кто должен это обработать.

Для frame-based части смотри .pdata и .xdata. У функции с __try/__except должен быть handler flag и language-specific handler. Если это MSVC C SEH, часто будет __C_specific_handler. Дальше надо искать handler data: protected ranges, filters, handler targets. Не жди, что все связи будут обычными jmp/call в основном control-flow graph.

Если код вызывает RaiseException с custom exception code, а рядом есть __try filter, который проверяет этот code, это может быть нормальный внутренний протокол. Если код вызывает RaiseException и ловит его VEH, это другой стиль. Если код вызывает RaiseFailFastException, обычные handlers могут быть намеренно обойдены. Если код ставит top-level filter и пишет dump, это, скорее всего, диагностический слой.

6.7.25. Как будущий static должен думать о порядке доставки

Для будущего анализатора порядок доставки исключения должен стать не картинкой в документации, а моделью отчёта. Инструмент не должен просто перечислять: “найден AddVectoredExceptionHandler, найден SetUnhandledExceptionFilter, найдена .pdata”. Он должен объяснять, как эти факты связаны.

Например, если найден AddVectoredExceptionHandler, отчёт должен сказать: это ранний process-wide handler, вызываемый до frame-based unwind/search; callback находится по такому-то RVA; callback возвращает такие-то значения; callback меняет или не меняет CONTEXT; рядом есть deliberate exceptions. Это сразу даёт смысл.

Если найден __C_specific_handler через UNWIND_INFO, отчёт должен сказать: это frame-based language-specific handler для функции или диапазона функций; он связан с .pdata/.xdata; пользовательские filters/handlers нужно искать в language-specific data, а не считать __C_specific_handler пользовательской логикой.

Если найден SetUnhandledExceptionFilter, отчёт должен сказать: это поздний unhandled layer; он не заменяет VEH и не связан напрямую с кадром faulting function; вероятное назначение - crash reporting, если callback пишет dump и не продолжает выполнение; подозрительно, если callback меняет RIP и возвращает CONTINUE_EXECUTION.

```
Пример связного finding-а:

Finding: exception-driven control flow through VEH
Evidence:
  - AddVectoredExceptionHandler registered callback at RVA 0x140012340
  - callback checks EXCEPTION_BREAKPOINT
  - callback increments ContextRecord->Rip and returns EXCEPTION_CONTINUE_EXECUTION
  - .text contains intentional int 3 instructions near opaque branches
Meaning:
  - exception is likely part of control flow before frame-based SEH search
  - ordinary CFG may miss this path
```

Такой отчёт лучше, чем “обнаружен VEH”. Он объясняет слой, порядок, evidence и влияние на анализ.

6.7.26. Типичные ошибки при понимании полного маршрута

Первая ошибка - думать, что first chance exception всегда означает crash. Нет. First chance - это раннее уведомление, до того как программа исчерпала свои handlers.

Вторая ошибка - думать, что VEH и SEH одно и то же. VEH process-wide и не frame-based. SEH frame-based связан с кадрами, а на x64 - с .pdata/.xdata и language-specific handler-ами.

Третья ошибка - думать, что SetUnhandledExceptionFilter ловит раньше всех. Наоборот, это поздний слой. Если VEH или frame-based handler обработали исключение, top-level filter может вообще не увидеть событие.

Четвёртая ошибка - думать, что __C_specific_handler - это пользовательский catch. Нет. Это системно-компиляторный language-specific handler, который читает данные MSVC и направляет маршрут к filter/handler code.

Пятая ошибка - думать, что CONTINUE_EXECUTION всегда спасает. Нет. Он осмыслен только если handler исправил память, регистры или причину исключения. Иначе будет повторное исключение.

Шестая ошибка - забывать, что исключение происходит в конкретном thread. Stack frames, RSP, ContextRecord и frame-based search относятся к этому потоку, а не ко всему процессу сразу.

Седьмая ошибка - игнорировать debugger. Под debugger-ом маршрут может останавливаться на first chance или second chance, и программа может вести себя иначе, чем без debugger-а.

Восьмая ошибка - думать, что .pdata/.xdata нужны только для crash-а. Они нужны для нормального stack walking, exception handling, unwind phase, debugger support и dump analysis.

6.7.27. Мини-сцена: deliberate int 3 и VEH

Представим функцию, где есть намеренная инструкция int 3. В обычном дизассемблере она выглядит как breakpoint. Новичок думает: “программа тут специально падает”. Но рядом при старте процесса был вызов AddVectoredExceptionHandler. Callback проверяет ExceptionCode == EXCEPTION_BREAKPOINT, увеличивает ContextRecord->Rip на размер инструкции и возвращает EXCEPTION_CONTINUE_EXECUTION.

Маршрут будет такой: процессор вызывает breakpoint exception; если debugger есть, он получает first chance. Если debugger пропустил, вызывается VEH. VEH меняет RIP так, чтобы поток продолжил после int 3. Frame-based SEH может вообще не начаться, потому что VEH сказал CONTINUE_EXECUTION. Top-level filter тоже не увидит исключение. Для живого выполнения это не crash, а скрытый переход через exception pipeline.

Для реверсера вывод: CFG по обычным jmp/call неполный. Нужно добавить exception edge: int 3 -> VEH callback -> modified RIP -> continuation. Без этой связи программа кажется сломанной или бессмысленной.

6.7.28. Мини-сцена: access violation внутри __try

Теперь другая сцена. Код делает запись по плохому адресу внутри __try-блока. VEH не зарегистрирован или возвращает CONTINUE_SEARCH. Debugger first chance тоже не обработал. Windows начинает frame-based search. По RIP находит RUNTIME_FUNCTION текущей функции. По UnwindInfoAddress читает UNWIND_INFO. Видит EHANDLER и вызывает __C_specific_handler. Тот смотрит scope table: faulting ControlPc попадает в protected range. Filter возвращает EXCEPTION_EXECUTE_HANDLER. Search phase нашла обработчик.

После этого начинается unwind phase. Если между faulting function и target handler есть промежуточные кадры с cleanup, они должны быть обработаны. Потом управление попадёт в handler block. Top-level filter не сработает, потому что исключение обработано раньше. Debugger second chance тоже не будет, если обработка прошла успешно.

Для реверсера вывод: настоящий путь обработки лежит в .pdata/.xdata и language-specific data. Если просто смотреть код вокруг faulting instruction, handler может быть не очевиден.

6.7.29. Мини-сцена: crash reporter через SetUnhandledExceptionFilter

Третья сцена. Программа на старте вызывает SetUnhandledExceptionFilter(CrashFilter). Позже где-то происходит access violation. VEH нет. Frame-based SEH нет или не подходит. Debugger отсутствует. Исключение доходит до unhandled layer. UnhandledExceptionFilter получает управление и вызывает CrashFilter. CrashFilter получает EXCEPTION_POINTERS, пишет minidump и возвращает EXCEPTION_EXECUTE_HANDLER. Затем процесс обычно завершается.

Здесь важно: CrashFilter не сделал программу устойчивой к access violation. Он помог сохранить диагностику. Это огромная практическая польза, но другой слой. Если анализатор увидел такой filter, не нужно сразу говорить “исключения используются как control flow”. Нужно смотреть поведение callback-а.

6.7.30. Главная модель раздела

Полный маршрут доставки исключения на Windows x64 можно понимать как лестницу с разными этажами. Сначала событие возникает в конкретном потоке и получает ExceptionRecord + ContextRecord. Если есть debugger, он получает first chance. Потом ранние vectored exception handlers могут увидеть исходное событие до frame-based unwind. Если они не обработали, Windows идёт по кадрам текущего потока, используя .pdata, RUNTIME_FUNCTION, .xdata, UNWIND_INFO и language-specific handlers. Если handler найден, начинается unwind phase и управление приходит в handler block. Если никто не обработал, debugger может получить second chance. Потом поздний top-level/unhandled layer может записать crash dump или выполнить final filter. Если и это не спасает или обработка по умолчанию требует завершения, процесс завершается.

Эта лестница - фундамент для всех следующих практических глав. Когда мы будем читать .pdata, мы будем понимать, что это не просто таблица. Когда будем читать .xdata, мы будем понимать, что она включается в конкретной фазе маршрута. Когда будем искать VEH, мы будем понимать, что он стоит раньше stack unwind. Когда будем видеть top-level filter, мы будем понимать, что это поздний слой. Когда будем смотреть debugger first chance, мы не будем путать его с crash-ом.

Теперь можно переходить к следующему разделу: как всё это выглядит для реверсера на практике. Там мы уже будем смотреть не только на

теоретический порядок, а на признаки в бинаре: вызовы API, импорты, callback-и, .pdata/.xdata, __C_specific_handler, RaiseException, int 3, guard pages, crash filters и то, как собрать их в единую картину анализа.

Техническая база раздела 6.7

Раздел опирается на официальные материалы Microsoft по RaiseException, Vectored Exception Handling, AddVectoredExceptionHandler, AddVectoredContinueHandler, Exception Handling Functions for Debugging, UnhandledExceptionFilter, SetUnhandledExceptionFilter, errhandlingapi.h и x64 exception/unwind model. Важные технические утверждения: RaiseException создаёт исключение в calling thread; общий dispatcher сначала уведомляет debugger, затем ищет frame-based handlers по стековым кадрам потока, затем при неудаче уведомляет debugger повторно и переходит к default handling; vectored exception handlers вызываются после first-chance debugger notification и до начала stack unwinding; vectored handlers не являются frame-based; SetUnhandledExceptionFilter устанавливает top-level filter для процесса; UnhandledExceptionFilter участвует в обработке unhandled exceptions и взаимодействии с debugger-ом; RaiseFailFastException относится к механизмам, обходящим frame/vector handlers. Текст раздела написан заново в учебной подаче и связан с уже построенной моделью PE, .pdata, .xdata, UNWIND_INFO, SEH, VEH и top-level handling.

6.8. Как это выглядит для реверсера на практике: от признаков в бинаре к единой карте

В предыдущих разделах мы долго собирали механизм по слоям. Сначала исключение было просто событием: поток выполнял инструкцию, что-то пошло не обычным путём, Windows получила ExceptionRecord и ContextRecord. Потом мы добавили стек, кадры, caller/callee, .pdata, .xdata, RUNTIME_FUNCTION, UNWIND_INFO, __C_specific_handler, VEH и top-level фильтр. Теперь наступает момент, ради которого вся эта медленная теоретическая подготовка и нужна: как реверсер должен смотреть на настоящий бинарь, чтобы не видеть отдельные куски, а собрать их в одну карту.

Этот раздел специально будет практическим, но не лабораторным. Мы не будем превращать текст в набор команд “нажми сюда, потом сюда”. Наша задача глубже: сформировать способ мышления. Реверсер не просто ищет API по именам. Он спрашивает: какой слой механизма я сейчас вижу? Это ранний vectored слой? Это frame-based SEH? Это top-level crash filter? Это обычный compiler-generated unwind? Это deliberate exception как control-flow trick? Это антидебаг? Это легитимная диагностика? Или это вообще ложный след, который появился из-за оптимизации компилятора?

Слово “практика” здесь означает не отказ от теории, а наоборот: умение приложить теорию к конкретному PE-файлу. Если теория не связывается с байтами, импортами, callback-ами, RIP, RSP и таблицами, она остаётся красивой картинкой. Если байты читаются без теории, реверс превращается в угадывание. Поэтому этот раздел будет мостом: от уже построенной модели Windows x64 exception handling к тому, как это видит человек в дизассемблере, отладчике и будущем static -e.

6.8.1. Практический анализ начинается не с инструмента, а с вопроса “какой слой передо мной?”

Новичок часто открывает бинарь и сразу начинает искать “интересные функции”: AddVectoredExceptionHandler, RaiseException, SetUnhandledExceptionFilter, __C_specific_handler, IsDebuggerPresent. Это понятно: хочется зацепиться за знакомые имена. Но зрелый подход начинается раньше. Нужно сначала спросить не “какую строку я нашёл?”, а “какой слой механизма эта строка представляет?”

Если ты видишь AddVectoredExceptionHandler, это не просто импорт из Kernel32. Это признак, что программа может зарегистрировать vectored exception handler, то есть обработчик процесса, который получает исключение до frame-based unwind. Если ты видишь __C_specific_handler, это не “пользовательский обработчик”, а language-specific handler для C SEH, который читает compiler-generated данные о protected ranges, filters и handlers. Если ты видишь SetUnhandledExceptionFilter, это не механизм локального восстановления программы, а поздний top-level слой, который обычно нужен для crash reporting. Если ты видишь RaiseException, это не обязательно “падение”, а программное создание исключения в calling thread.

Такой вопрос о слое защищает от главной ошибки: смешать все exception-механизмы в одну кашу. В Windows есть несколько этажей доставки. Один и тот же ExceptionRecord может пройти через debugger first chance, VEH, frame-based SEH, debugger second chance и top-level handling. Реверсер должен понимать, на каком этаже он стоит. Иначе он будет ожидать от

top-level filter поведения SEH, от VEH - поведения stack frame handler-а, а от __C_specific_handler - поведения пользовательского catch-блока.

Поэтому практическая мантра раздела такая: найденный признак нужно сразу привязать к маршруту доставки исключения. Не просто “нашёл API”, а “нашёл API, который ставит ранний процессный обработчик до unwind”. Не просто “нашёл .pdata”, а “нашёл карту функций, которая понадобится frame-based search и stack walking”. Не просто “нашёл int 3”, а “нашёл потенциальный источник EXCEPTION_BREAKPOINT, который может быть breakpoint-ом отладчика, deliberate control-flow узлом или антидебаговой проверкой”.

6.8.2. Static analysis и dynamic analysis: два взгляда на один exception pipeline

Static analysis - статический анализ. Static означает “не выполняя”. Ты читаешь файл на диске или загруженный image как данные: PE headers, imports, exports, sections, .pdata, .xdata, строки, дизассемблированные инструкции. В статическом анализе программа не живёт. Нет настоящего текущего RIP, нет настоящего исключения, нет активного ContextRecord. Зато есть структура файла, которую можно разбирать спокойно.

Dynamic analysis - динамический анализ. Dynamic означает “во время выполнения”. Ты запускаешь программу, ставишь breakpoint, смотришь регистры, ловишь exception events, проверяешь first chance и second chance, видишь, вызывается ли VEH callback, меняется ли ContextRecord->Rip, доходит ли исключение до SEH или top-level filter. Здесь программа живёт, и ты видишь реальный маршрут, но уже зависишь от входных данных, окружения, антидебага, timing-а и состояния ОС.

Обе стороны нужны. Статически можно увидеть, что бинарь импортирует AddVectoredExceptionHandler, но нельзя автоматически доказать, что handler реально зарегистрировался в данном запуске. Динамически можно увидеть, что handler вызвался, но без статического чтения кода трудно понять, как он связан с другими участками программы. Статически можно разобрать .pdata/.xdata и понять, где handler data, но динамически можно проверить, к какому ControlPc это реально относится при исключении.

Для будущего static -а это означает: не надо делать вид, будто один слой отвечает на все вопросы. Статический анализ даёт гипотезы и карту. Динамический анализ подтверждает маршрут. Хороший отчёт должен различать: “в файле найден импорт AddVectoredExceptionHandler”, “в коде есть вызов регистрации”, “callback address вычисляется статически”, “в runtime callback реально был зарегистрирован”, “при исключении callback реально изменил RIP”. Это разные уровни уверенности.

6.8.3. Первый практический слой: imports как указатели на намерения, но не доказательства поведения

Import table - таблица импортов. Мы уже говорили, что PE импортирует функции из DLL через Import Directory и Import Address Table. Для реверсера imports часто являются первым быстрым способом понять, какие подсистемы программа потенциально использует. Если в imports есть AddVectoredExceptionHandler, RaiseException, SetUnhandledExceptionFilter, MiniDumpWriteDump, IsDebuggerPresent, CheckRemoteDebuggerPresent, VirtualProtect, это сразу задаёт направление анализа.

Но слово “потенциально” здесь очень важно. Import не равен выполнению. Если функция импортируется, это значит только, что модуль может её вызвать напрямую через IAT. Она могла остаться от библиотеки, неиспользуемого кода, общего runtime, статически подключённого crash reporter-a. Обратная сторона тоже верна: если импорта нет, это не значит, что функция не используется. Программа может получить адрес через GetProcAddress, использовать hashed imports, manual resolving, syscall wrapper, delay import или динамическую загрузку DLL.

Поэтому imports - это не приговор, а начало расследования. Увидел AddVectoredExceptionHandler - ищи call site. Увидел RaiseException - ищи, с каким ExceptionCode вызывается. Увидел VirtualProtect - проверь, не ставится ли PAGE_GUARD или executable/writeable protection. Увидел MiniDumpWriteDump - проверь, связан ли он с top-level filter или это просто аварийная диагностика. Увидел IsDebuggerPresent - проверь, влияет ли результат на exception route или просто включает дополнительный лог.

```
Пример первого статического вопроса:

Import: AddVectoredExceptionHandler
Вопрос 1: где call site?
Вопрос 2: какой First parameter? 1 или 0?
Вопрос 3: какой callback address?
Вопрос 4: что callback возвращает?
Вопрос 5: меняет ли callback ContextRecord?
Вопрос 6: какие ExceptionCode он проверяет?
```

Такой подход медленнее, чем “нашёл API - сделал вывод”, но он надёжнее. В reverse engineering почти всегда вредно прыгать от признака к смыслу без промежуточных вопросов. Признак говорит: “возможна такая подсистема”. Смысл появляется только после связи признака с кодом и маршрутом выполнения.

6.8.4. AddVectoredExceptionHandler: что именно искать после найденного импорта

AddVectoredExceptionHandler - функция регистрации vectored exception handler-a. Слово add означает “добавить”. Vectored exception handler - обработчик исключений, зарегистрированный на уровне процесса и не привязанный к конкретному stack frame. Он получает шанс увидеть исключение раньше frame-based SEH. Поэтому этот API важен для антидебага, защит, crash instrumentation, runtime tricks и иногда нормальной диагностики.

Практический анализ начинается с call site. Call site - место вызова. Если функция импортирована, нужно найти, где она вызывается. На Windows x64 первые аргументы обычно идут в RCX, RDX, R8, R9. Для AddVectoredExceptionHandler первый аргумент First обычно определяет положение handler-a в цепочке: если ненулевой, handler вызывается первым; если нулевой, последним относительно других vectored handlers. Второй аргумент Handler - адрес callback-функции.

Callback - функция обратного вызова. Это код, который программа не вызывает как обычную бизнес-логику напрямую в каждый момент. Она регистрирует его, а потом Windows вызывает его при событии. В нашем случае событие - исключение. Поэтому, когда ты видишь адрес callback-a, нужно анализировать не только “кто его вызывает” через обычный xref. Его может вызывать exception dispatcher, а не обычный call из твоего кода.

```
; условная x64-картина регистрации VEH
```

```
lea  rdx, [rip+VehCallback] ; второй аргумент: адрес callback
mov  ecx, 1                 ; первый аргумент: First = TRUE
call AddVectoredExceptionHandler
```

После этого главный объект анализа - VehCallback. Внутри callback-а обычно надо искать доступ к EXCEPTION_POINTERS. EXCEPTION_POINTERS - структура, которая содержит два указателя: ExceptionRecord и ContextRecord. ExceptionRecord отвечает на вопрос “что произошло”, ContextRecord отвечает на вопрос “в каком состоянии поток был”. Если handler проверяет ExceptionRecord->ExceptionCode, он выбирает тип исключения. Если handler меняет ContextRecord->Rip, ContextRecord->Rsp или регистры, он меняет будущее выполнение потока.

Для реверсера критична именно модификация ContextRecord. Если callback только пишет лог и возвращает CONTINUE_SEARCH, он не меняет control flow. Если callback при EXCEPTION_BREAKPOINT увеличивает RIP, он превращает int 3 в скрытый переход. Если callback при STATUS_GUARD_PAGE_VIOLATION снова ставит PAGE_GUARD и меняет состояние, он может реализовать memory watchpoint или антианализ. Если callback при access violation подставляет адрес, выделяет память или переписывает RIP, это уже активный exception-based control flow.

6.8.5. Как читать callback VEH не как обычную функцию, а как участника маршрута исключения

Обычная функция вызывается тогда, когда ты видишь call или косвенный call. VEH callback живёт иначе. Его регистрация создаёт связь “исключение -> Windows dispatcher -> callback”. Поэтому в графе обычных xref-ов callback может выглядеть почти висящим. Инструмент может показать, что на него ссылается только lea перед AddVectoredExceptionHandler. Новичок может подумать: “функция почти не используется”. На самом деле она может быть центральным узлом выполнения.

Когда читаешь VEH callback, не начинай с локальных переменных. Начни с вопроса о входе. Handler получает PEXCEPTION_POINTERS. Это указатель на структуру, через которую открываются ExceptionRecord и ContextRecord. Значит первые обращения к памяти часто будут похожи на чтение указателей из этой структуры, затем чтение ExceptionCode, ExceptionAddress, ExceptionInformation или регистров из CONTEXT.

CONTEXT - огромная структура. Для x64 она содержит RIP, RSP, RAX, RBX, RCX, RDX, R8-R15, флаги и другие поля. В дизассемблере это часто выглядит как обращения по большим смещениям от указателя. Если символов нет, имена полей ты не увидишь. Поэтому реверсер узнаёт логику по смыслу: “из аргумента handler-а достали указатель на ContextRecord, потом записали что-то в поле, похожее на RIP”.

```
Упрощённая смысловая форма callback-а:

LONG CALLBACK Veh(EXCEPTION_POINTERS* p)
{
    if (p->ExceptionRecord->ExceptionCode == EXCEPTION_BREAKPOINT) {
        p->ContextRecord->Rip += 1;
        return EXCEPTION_CONTINUE_EXECUTION;
    }
    return EXCEPTION_CONTINUE_SEARCH;
}
```

Эта маленькая функция меняет весь смысл инструкции int 3. Без VEH int 3 выглядит как breakpoint и может остановить программу. С таким VEH int 3 становится управляемым событием: исключение возникло, callback поправил RIP, выполнение продолжилось после инструкции. В обычном control-flow graph между int 3 и продолжением нет привычной стрелки. Стрелка проходит через exception pipeline.

Поэтому реверсер должен дополнять CFG. CFG - control-flow graph, граф потока управления. Обычный CFG строится по jmp, call, ret, conditional branches. Exception-aware CFG должен добавлять edge от faulting instruction к handler/callback и от callback к новому continuation address, если callback модифицирует ContextRecord. Без этого анализ защиты и обфускации будет неполным.

6.8.6. RaiseException: когда исключение создаёт сама программа

RaiseException - Windows API, через который программа сама создаёт software exception, то есть программно сгенерированное исключение. Слово raise здесь значит “поднять”, “возбудить”, “создать событие”. В отличие от access violation, где событие рождается из неудачного обращения к памяти, RaiseException является явным вызовом: программа говорит Windows “создай исключение с таким кодом и параметрами”.

Практически это означает: если ты видишь call RaiseException, не думай автоматически “тут баг”. Это может быть нормальный runtime-механизм, диагностика, тест, реализация внутреннего протокола, C++/language runtime слой или защитный control-flow trick. Главное - посмотреть аргументы.

На Windows x64 первые аргументы идут через RCX, RDX, R8, R9. Для RaiseException важны ExceptionCode, ExceptionFlags, NumberOfArguments и Arguments. ExceptionCode - числовой код события. ExceptionFlags может делать исключение noncontinuable. Arguments - массив дополнительных параметров, который попадёт в ExceptionInformation внутри EXCEPTION_RECORD.

```
; условная картина вызова RaiseException
mov  ecx, 0E0424242h ; ExceptionCode
xor  edx, edx        ; ExceptionFlags = 0, continuable
xor  r8d, r8d        ; NumberOfArguments = 0
xor  r9d, r9d        ; Arguments = NULL
call RaiseException
```

После такого вызова важно искать не только следующий код. Нужно искать, кто перехватывает этот ExceptionCode. Если рядом зарегистрирован VEH, он может проверять именно 0xE0424242. Если вокруг есть __try/__except, filter может проверять GetExceptionCode. Если исключение не обработано, оно дойдёт до top-level. Один call RaiseException сам по себе - половина истории. Вторая половина - маршрут, где этот код признаётся “своим”.

Для будущего static -а полезная проверка звучит так: найден программный exception source; найден ли matching handler? Matching handler - обработчик, который явно проверяет тот же ExceptionCode или работает с тем же типом события. Если source есть, а handler не найден, это может быть диагностический crash, unreachable path или динамически скрытый handler. Если source и handler связаны, это уже exception-based protocol.

6.8.7. GetExceptionCode и GetExceptionInformation: почему фильтр SEH узнаётся по своим источникам данных

В C SEH есть две специальные сущности: GetExceptionCode и GetExceptionInformation. Первая возвращает код текущего исключения. Вторая возвращает указатель на EXCEPTION_POINTERS, то есть связку ExceptionRecord + ContextRecord. Их нельзя понимать как обычные универсальные функции, которые можно вызвать где угодно. В модели Microsoft они имеют смысл внутри filter expression или exception-handler block-a SEH.

Filter expression - выражение-фильтр внутри __except(...). Оно решает, обрабатывать исключение или продолжить поиск. Именно в фильтре часто видны проверки вроде “если код access violation - обработать, иначе искать дальше”. В исходнике это выглядит чисто. В бинаре это превращается в код, связанный с language-specific data и __C_specific_handler.

```
__try {
    dangerous();
}
__except (GetExceptionCode() == EXCEPTION_ACCESS_VIOLATION
          ? EXCEPTION_EXECUTE_HANDLER
          : EXCEPTION_CONTINUE_SEARCH) {
    recover();
}
```

Для реверсера важна связь: GetExceptionCode в исходнике не означает, что где-то обязательно будет обычный импорт с таким именем. Компилятор может обрабатывать это как специальный keyword или intrinsic-like механизм. Поэтому искать только импорт GetExceptionCode недостаточно. Нужно смотреть на структуру SEH: __C_specific_handler, handler data, scope table, filter code, handler code.

Когда ты видишь код, который сравнивает ExceptionCode с 0xC0000005, 0x80000003, 0x80000001 или пользовательским кодом, спрашивай: этот код находится в VEH callback? В SEH filter? В top-level filter? В обычной функции, которую вызывает filter? Один и тот же if по ExceptionCode имеет разный смысл в разных слоях. В VEH он ранний и может остановить stack unwind. В SEH filter он frame-based и зависит от protected range. В top-level filter он поздний и обычно диагностический.

6.8.8. __C_specific_handler: почему его наличие не означает “я нашёл пользовательский handler”

__C_specific_handler - один из самых частых источников путаницы. Его имя видно в unwind metadata или в символах системного runtime, и новичок может подумать: “вот обработчик исключения”. Но это не пользовательский обработчик в смысле тела __except. Это language-specific handler, который Windows вызывает, когда у функции в UNWIND_INFO есть нужный флаг и когда нужно интерпретировать C SEH данные.

Представь посредника. Windows dispatcher умеет идти по кадрам, читать .pdata, находить RUNTIME_FUNCTION, читать UNWIND_INFO, понять “у этой функции есть EHANDLER”. Но Windows не обязана сама знать, как именно MSVC закодировал список protected ranges, filters и handlers для C SEH. Она вызывает __C_specific_handler и передаёт ему ExceptionRecord, EstablisherFrame, ContextRecord, DispatcherContext. Дальше этот обработчик читает language-specific data.

Поэтому практическая задача реверсера - не остановиться на имени __C_specific_handler, а пройти дальше: где handler data? Какие ranges? Какие filters? Какие target handler blocks? Какой ControlPc попадает в какой диапазон? Если у тебя есть только адрес __C_specific_handler, ты знаешь, что функция участвует в C SEH-модели. Но ты ещё не знаешь, что именно будет обработано.

```
Смысловая цепочка для MSVC C SEH:

RIP внутри функции
 -> RtlLookupFunctionEntry
 -> RUNTIME_FUNCTION
 -> UNWIND_INFO with EHANDLER
 -> handler RVA = __C_specific_handler
 -> handler data / scope-like table
 -> protected range
 -> filter
 -> handler block
```

В книге мы уже готовили фундамент для этой цепочки. Теперь важно применить его: __C_specific_handler - не конец анализа, а вход в данные компилятора. Если декомпилятор показывает красивый __try/__except, он уже сделал часть этой работы за тебя. Если не показывает, ты должен понимать, какие звенья искать вручную.

6.8.9. .pdata как практический источник границ функций

.pdata для x64 реверсера - это не только exception-инфраструктура. Это ещё и мощный источник информации о границах функций. В обычном PE32+ образе Exception Directory ведёт к массиву RUNTIME_FUNCTION. Каждая запись говорит: BeginAddress, EndAddress, UnwindInfoAddress. То есть у нас есть диапазон кода, который компилятор/линкер объявил функцией или функциональным регионом для unwind.

Почему это полезно? Дизассемблер может ошибаться в границах функций. Особенно если код оптимизирован, есть tail calls, hot/cold splitting, alignment padding, jump tables, ручные вставки, packed участки. .pdata даёт альтернативную карту. Если BeginAddress и EndAddress покрывают область кода, это сильный аргумент, что Windows тоже будет мыслить этот диапазон как unwind-регион.

Но .pdata нельзя превращать в абсолютную истину. Во-первых, leaf function может не иметь записи. Во-вторых, записи могут описывать не “функции исходника”, а регионы, нужные для unwind. В-третьих, packed или obfuscated бинарь может намеренно исказить таблицы. В-четвёртых, dynamic function tables могут существовать только в runtime. Поэтому правильная формула такая: .pdata - официальный runtime-контракт для stack walking, но реверсер должен сверять его с кодом.

Практическая проверка: если запись .pdata говорит, что функция начинается по RVA A и заканчивается по RVA B, посмотри дизассемблер в этом диапазоне. Есть ли нормальный entry? Есть ли пролог? Есть ли epilog? Не попадает ли диапазон в середину другой функции? Не выходит ли EndAddress за исполняемую секцию? Не указывает ли UnwindInfoAddress в нечитабельную область? Эти вопросы превращают структуру в аналитический инструмент.

6.8.10. .xdata и UNWIND_INFO: где практический анализ переходит от “границы функции” к “как функция устроила кадр”

Если .pdata отвечает на вопрос “какой диапазон кода перед нами?”, то .xdata отвечает на вопрос “как выйти из этого диапазона назад к caller-у?”. В .xdata обычно лежит UNWIND_INFO. Эта структура описывает действия пролога: выделение стека, сохранение nonvolatile-регистров, frame register, флаги handler-а, chained info. Для практического анализа это означает, что мы можем сверять заявленное поведение функции с реальными инструкциями.

Допустим, UNWIND_INFO говорит, что в прологе есть UWOP_PUSH_NONVOL для RBX и UWOP_ALLOC_SMALL на 0x40 байт. Тогда в коде мы ожидаем увидеть смысловую пару: сохранить RBX и уменьшить RSP. Это не обязано быть буквально в учебном порядке, но должно быть совместимо с правилами x64 prolog. Если код делает sub rsp, 80h, а unwind говорит 0x40, это повод остановиться. Может быть ты смотришь не тот prolog. Может быть есть chained info. Может быть функция split. Может быть metadata битая. А может быть это защитный трюк.

Такой анализ очень важен для будущего static -а. Нельзя просто распарсить UNWIND_INFO и вывести поля. Нужно формулировать проверку договора: “metadata claims X; code appears to do Y; X and Y agree/disagree/uncertain”. Именно разница между “прочитал” и “проверил” делает инструмент полезным.

```
Пример практической сверки:

Код:
    push rbx
    sub  rsp, 40h

UNWIND_INFO:
    UWOP_PUSH_NONVOL RBX
    UWOP_ALLOC_SMALL 40h

Вывод:
    Метаданные согласованы с прологом. Virtual unwind, вероятно, сможет
    восстановить caller context для обычного случая.
```

6.8.11. Почему handler flags в UNWIND_INFO важнее, чем кажется

В UNWIND_INFO есть флаги. На первый взгляд это просто маленькое поле. Практически это переключатель интерпретации хвоста структуры. Если флаг говорит NHANDLER, после unwind codes нет обычного exception handler-а. Если EHANDLER - после выравнивания лежит RVA language-specific handler-а и затем handler data. Если UHANDLER - логика связана с unwind/termination. Если CHAININFO - хвост структуры нужно читать как chained RUNTIME_FUNCTION-подобную запись, а не как адрес handler-а.

Ошибка чтения флагов ломает всё. Если принять CHAININFO за EHANDLER, анализатор может “найти” несуществующий handler. Если пропустить EHANDLER, он не увидит реальную SEH/C++ EH логику. Если не учесть выравнивание массива unwind codes, можно прочитать handler RVA с неправильного смещения. В бинарных форматах маленькое поле часто решает, какая структура лежит дальше.

Для реверсера handler flag - это момент, где обычный unwind превращается в обработку исключений. Функция может иметь unwind codes только для stack walking. Но если у неё есть EHANDLER, она может участвовать в search phase. Это значит: по текущему ControlPc Windows может вызвать language-specific handler. Значит, в control-flow модели появляется потенциальный exception edge.

Поэтому практический вывод такой: чтение .xdata должно заканчиваться не только восстановлением стека, но и вопросом “есть ли у этой функции обработчик, который меняет маршрут исключения?” Это мост между технической структурой и анализом поведения.

6.8.12. Deliberate exceptions: когда исключение - не авария, а инструкция управления

Deliberate exception - намеренное исключение. Deliberate означает “сделанное специально”. Это один из самых важных практических паттернов. Программа может использовать исключение не потому, что сломалась, а потому что так построена её логика. Для обычного программиста это может быть странно. Для защит, packer-ов, антидебага и некоторых runtime-механизмов это нормальный инструмент.

Типичные источники deliberate exceptions: int 3, ud2 или другая invalid instruction, RaiseException с пользовательским кодом, guard page violation, single-step exception, access violation на специально подготовленном адресе. Смысл один: обычная инструкция вызывает exception event, а handler решает, куда идти дальше.

Чем это отличается от обычного if? Тем, что переход не виден как обычная ветка в коде. Условие может быть спрятано в handler-е. Состояние может быть передано через ExceptionInformation или регистры в ContextRecord. Адрес продолжения может быть вычислен в callback-е. Декомпилятор, который строит CFG только по обычным переходам, может потерять эту логику.

```
Обычная ветка видна в коде:
    cmp eax, 1
    je  target

Exception-based ветка может выглядеть так:
    int 3
    ; обычный CFG думает: остановка / breakpoint

Но реальный путь:
    int 3 -> EXCEPTION_BREAKPOINT -> VEH/SEH -> ContextRecord->Rip = target
```

Практическое правило: всякий намеренный exception source нужно связывать с handler-ом. Если источник есть, но handler не виден, не спеши делать вывод. Handler может быть VEH, frame-based SEH, debugger-assisted behavior, dynamic registration, hidden import resolving. Но если source и handler связались, ты нашёл скрытый control flow.

6.8.13. int 3 и EXCEPTION_BREAKPOINT: почему breakpoint не всегда принадлежит отладчику

int 3 - короткая инструкция breakpoint на x86/x64. Отладчики используют её, чтобы остановить выполнение: они временно заменяют байт инструкции на 0xCC, процессор вызывает breakpoint exception, debugger получает событие, показывает пользователю остановку и потом восстанавливает/симулирует выполнение. Поэтому новичок привык думать: int 3 равно отладчик.

Но программа сама тоже может содержать int 3. В этом случае EXCEPTION_BREAKPOINT может быть частью логики. Если debugger присутствует, он получает first chance и может остановиться. Если debugger пропустит событие, дальше может сработать VEH или SEH. Если handler изменит RIP, int 3 станет не остановкой, а контролируемым узлом перехода.

Для реверсера важно различать три ситуации. Первая: int 3 вставлен отладчиком в runtime, в файле на диске его не было. Вторая: int 3 реально лежит в .text как часть программы. Третья: int 3 используется как padding/alignment или unreachable trap, но не участвует в логике. Чтобы отличать, нужно смотреть file bytes, runtime bytes, xrefs, handler registration и поведение под debugger-ом.

Если int 3 лежит в файле и рядом есть AddVectoredExceptionHandler, это уже не случайность, но всё ещё не доказательство. Нужно проверить callback. Если callback обрабатывает EXCEPTION_BREAKPOINT и возвращает CONTINUE_EXECUTION, связь сильная. Если callback игнорирует breakpoint, int 3 может быть другим механизмом или просто отладочным следом.

6.8.14. Guard pages: почему PAGE_GUARD создаёт exception не как баг, а как сигнал

Guard page - сторожевая страница памяти. Guard означает “страж”, “охранник”. В Windows PAGE_GUARD делает страницу особой: первая попытка доступа вызывает STATUS_GUARD_PAGE_VIOLATION. После этого система очищает guard-статус, то есть следующий доступ уже не вызовет такое же исключение, если guard не поставить заново. Поэтому guard page часто описывают как one-shot alarm - одноразовая сигнализация.

Для reverse engineering guard pages важны по двум причинам. Во-первых, это легитимный системный механизм. Например, guard pages используются для роста стека и мониторинга доступа. Во-вторых, это удобный инструмент защиты и антианализа. Программа может поставить PAGE_GUARD на область кода или данных, дождаться исключения при доступе, а handler узнает, что кто-то прочитал или выполнил страницу. После этого handler может снова поставить PAGE_GUARD, изменить поведение или обнаружить отладчик/трассировку.

Практические признаки: вызовы VirtualProtect/VirtualProtectEx/VirtualAlloc с PAGE_GUARD; обработка STATUS_GUARD_PAGE_VIOLATION; повторная установка защиты в handler-е; проверка адреса из ExceptionInformation; связь с single-step или trap flag. Если видишь PAGE_GUARD без handler-а, это может быть обычная память. Если видишь handler, который заново ставит guard, это уже механизм наблюдения.

```
Смысловая схема guard-based watch:

VirtualProtect(page, size, PAGE_EXECUTE_READ | PAGE_GUARD, &old)

... доступ к page ...
```

```
-> STATUS_GUARD_PAGE_VIOLATION
-> VEH callback
-> проверить ExceptionAddress / fault address
-> снова поставить PAGE_GUARD
-> CONTINUE_EXECUTION или CONTINUE_SEARCH
```

Здесь особенно видно, почему мы готовили фундамент про ExceptionInformation. Для access violation и некоторых memory-related exceptions дополнительные параметры помогают понять, какой адрес вызвал событие и какой тип доступа был. Без чтения этих данных handler выглядит как странный набор сравнений. С пониманием EXCEPTION_RECORD он становится понятным фильтром доступа.

6.8.15. Access violation как control-flow trick: осторожно, не всякий 0xC0000005 - обычный баг

0xC0000005, STATUS_ACCESS_VIOLATION, access violation - один из самых частых кодов исключений. Обычно он означает реальную ошибку: чтение по неправильному указателю, запись в запрещённую память, выполнение неисполняемой страницы. Но в защитном или обфусцированном коде access violation может быть намеренным.

Например, программа может обратиться к адресу, который заведомо invalid, но handler поймает исключение и по текущему состоянию регистров вычислит следующий шаг. Или программа может попытаться выполнить страницу без execute-права, поймать DEP-like violation, поменять protection и продолжить. Или handler может использовать faulting address как индекс в таблице. В таких случаях crash на первый взгляд является частью программы.

Практическая разница между багом и control-flow trick не всегда очевидна. Нужно смотреть повторяемость, наличие handler-а, обработку конкретного ExceptionCode, изменение ContextRecord, наличие специально подготовленных адресов, странных constants, совпадение с антидебаговыми проверками. Если исключение происходит один раз и затем программа продолжает логически, это не похоже на обычный баг. Если оно происходит в защищённом диапазоне и handler всегда переводит RIP, это уже паттерн.

Но нельзя уходить в обратную крайность и считать каждый access violation защитой. Большинство access violations - всё-таки ошибки или некорректное состояние. Хороший реверсер не делает вывод “обфускация” только по коду исключения. Он связывает источник, handler, решение handler-а и последующий control flow.

6.8.16. Debugger-aware поведение: IsDebuggerPresent, NtQueryInformationProcess и exception route

Debugger-aware behavior - поведение, зависящее от наличия отладчика. Программа может проверять, запущена ли она под user-mode debugger-ом, и менять маршрут. Самый простой API - IsDebuggerPresent. Он возвращает ненулевое значение, если текущий процесс выполняется в контексте debugger-а. Есть CheckRemoteDebuggerPresent для другого процесса. Есть более низкоуровневые варианты через NtQueryInformationProcess, например ProcessDebugPort или получение PEB.

Почему это связано с исключениями? Потому что debugger находится прямо в exception pipeline. First chance debugger notification происходит раньше VEH. Second chance - ближе к концу маршрута. Если программа знает, что debugger есть, она может вызвать int 3,

RaiseException, single-step, guard page trick и ожидать другого поведения. Или наоборот: она может проверить debugger и отказаться регистрировать handler, чтобы под отладчиком программа “ломалась”, а без отладчика работала.

Для реверсера это означает: анализ exception tricks под debugger-ом может отличаться от запуска без debugger-а. Если ты видишь IsDebuggerPresent рядом с AddVectoredExceptionHandler или RaiseException, это сильная связь. Но даже если явной проверки нет, сам факт debugger-а влияет на first chance события. Debugger может перехватить и не передать исключение системе, а может пропустить его. От этого зависит, дойдёт ли событие до VEH/SEH.

```
Практический вопрос при exception-анализе:

1. Есть ли debugger?
2. Что debugger делает с first chance exception?
3. Передаёт ли он событие системе?
4. Если передаёт, какой handler получает его следующим?
5. Если не передаёт, какой runtime path мы потеряли?
```

Поэтому в отчёте важно писать не просто “при запуске упало на int 3”. Нужно уточнять: “под debugger-ом int 3 остановился как first chance EXCEPTION_BREAKPOINT; без debugger-а или при continue событие может быть передано VEH”. Это разные наблюдения.

6.8.17. Top-level filter и crash reporter: как отличить диагностику от control-flow abuse

SetUnhandledExceptionFilter часто встречается в нормальных приложениях. Программа ставит top-level filter, чтобы при необработанном исключении записать crash dump, лог, информацию о версии, thread list, modules list. Это не обязательно защита и не обязательно обфускация. Это может быть обычная инженерная забота: если приложение упало у пользователя, разработчик хочет получить dump.

Практический признак crash reporter-а - вызов MiniDumpWriteDump, создание файла .dmp, сбор версии, путей модулей, имени пользователя, времени, stack trace. Такой filter обычно не пытается восстановить выполнение после access violation. Он получает EXCEPTION_POINTERS, сохраняет диагностику и позволяет процессу завершиться или возвращает EXECUTE_HANDLER.

Control-flow abuse выглядит иначе. Handler может менять ContextRecord, выбирать новый RIP, чинить protection, повторно ставить guard pages, расшифровывать код, зависеть от debugger-а. Top-level filter редко хорош для тонкого control-flow, потому что он поздний. Если исключение дошло до него, frame-based SEH и VEH уже не обработали событие. Поэтому использовать top-level как регулярный диспетчер выполнения неудобно и грубо.

Вывод для реверсера: не демонизируй SetUnhandledExceptionFilter. Смотри callback. Если он пишет dump - это диагностический слой. Если он меняет контекст и возвращает CONTINUE_EXECUTION - это уже другое поведение. Если он специально восстанавливает old filter, защищает свой filter от перезаписи или патчит UnhandledExceptionFilter, это может быть антианализ или защита crash pipeline-а.

6.8.18. MiniDumpWriteDump: почему dump writing - это не “обработка” ошибки в смысле продолжения

MiniDumpWriteDump - функция из DbgHelp, которая записывает minidump. Minidump - компактный дамп процесса: не полный снимок всей памяти, а выбранный набор данных, достаточный для диагностики. При падении crash reporter часто получает EXCEPTION_POINTERS и передаёт информацию об исключении в MINIDUMP_EXCEPTION_INFORMATION.

Важно: запись dump-а не делает программу здоровой. Если поток упал на invalid write, а crash filter записал dump, причина не исчезла. После записи dump-а процесс обычно завершается. Это как фотография места аварии, а не ремонт машины. Поэтому top-level crash reporting нужно отделять от настоящего recovery.

Для reverse engineering MiniDumpWriteDump полезен как признак нормального crash infrastructure. Если рядом видны SetUnhandledExceptionFilter, CreateFile, MiniDumpWriteDump, CloseHandle, строки .dmp, crash, report, logs - скорее всего, это диагностика. Если же dump-writing код смешан с проверками debugger-а, шифрованием, guard pages и deliberate exceptions, тогда нужно смотреть шире: иногда защитный код тоже пишет дампы или имитирует crash reporting.

6.8.19. Как строить exception-aware карту функции

Exception-aware карта функции - это карта, которая учитывает не только обычные переходы, но и возможные exception edges. Начинается она с обычного CFG: basic blocks, conditional jumps, calls, returns. Затем добавляются источники исключений: инструкции, которые могут fault-нуть; явные RaiseException; int 3; ud2; guard page access; memory accesses в protected ranges; calls, которые могут генерировать SEH/C++ exceptions.

Следующий слой - handlers. Для VEH handler может быть глобальным по процессу. Для SEH handler связан с protected ranges в конкретной функции. Для top-level он поздний. Для debugger-а это вообще внешний участник. Поэтому edge от источника к handler-у не всегда локален. Исключение внутри функции C может быть обработано handler-ом функции A выше по стеку. Значит, карта одной функции иногда недостаточна: нужен stack-aware взгляд.

На практике можно начать проще. Для каждого function range из .pdata отметь: есть ли EHANDLER/UHANDLER? Если есть, какой language-specific handler? Для каждого участка кода отметь явные exception sources. Для каждого source спроси: локальный handler? VEH? top-level? unknown? Для каждого handler спроси: возвращает SEARCH, EXECUTE, CONTINUE_EXECUTION? меняет ContextRecord? вызывает RtlUnwind? пишет dump?

```
Мини-формат exception-aware заметки:

Source:
    RVA 0x1234: int 3 -> EXCEPTION_BREAKPOINT
Handlers:
    VEH callback RVA 0x3000 checks 0x80000003
Action:
    ContextRecord->Rip += 1; return CONTINUE_EXECUTION
Effect:
    hidden edge RVA 0x1234 -> RVA 0x1235
Confidence:
    high, if callback registered before source executes
```

Такой формат уже похож на будущий static finding. Он не просто говорит “есть int 3”. Он объясняет маршрут, действие и уверенность.

6.8.20. Как не перепутать compiler-generated сложность с защитой

Когда начинаешь видеть .pdata, .xdata, handlers, filters и странные переходы, легко стать параноиком. Любая сложность начинает казаться обфускацией. Но нормальный оптимизирующий компилятор тоже создаёт сложную картину. C++ exceptions, destructors, try/catch, inlining, tail calls, hot/cold splitting, security cookies, CFG instrumentation, profile-guided optimization - всё это может усложнять код без злого умысла.

Поэтому нужно различать “сложно” и “подозрительно”. Сложно - это когда механизм непростой, но слои согласованы: .pdata покрывает код, UNWIND_INFO соответствует прологу, handler data указывает в разумные диапазоны, imports объяснимы, callback-и не скрываются. Подозрительно - это когда слои противоречат друг другу: .pdata указывает в середину мусора, UNWIND_INFO не соответствует прологу, handler address вне image или в writable memory, код намеренно fault-ит и handler меняет RIP по сложной схеме, imports resolved вручную и скрыты.

Очень полезная категория для отчёта - “uncertain but notable”. Не надо заставлять инструмент выбирать между normal и malware. Можно написать: “Функция содержит EHANDLER и нестандартные handler data; это может быть compiler-generated SEH/C++ EH или custom obfuscation; требуется сопоставление с прологом и runtime behavior”. Такой стиль честнее и полезнее.

6.8.21. Dynamic function tables: почему статическая .pdata не всегда вся правда

Мы уже упоминали dynamic function tables. Практически они важны там, где код создаётся во время выполнения: JIT, runtime code generation, unpacking, manual mapping, custom loaders. Если код не лежит как обычный PE section с готовой .pdata, Windows всё равно может раскручивать его, если программа зарегистрирует function table через RtlAddFunctionTable или callback-механизм.

Для статического анализа файла это проблема видимости. Ты читаешь EXE на диске и видишь его .pdata. Но во время выполнения он может выделить память, записать туда код, зарегистрировать function table и начать исполнять. В файле этой runtime-карты нет как обычной секции. Значит, статический static должен уметь сказать: “в статическом PE таких записей нет; но код вызывает RtlAddFunctionTable, поэтому runtime может добавить unwind карту”.

Практический признак dynamic unwind: импорты или динамическое разрешение RtlAddFunctionTable, RtlDeleteFunctionTable, RtlInstallFunctionTableCallback; выделение executable memory; запись структур, похожих на RUNTIME_FUNCTION; переход в динамическую область. Это сложная тема, но фундамент уже понятен: x64 требует function table не обязательно из .pdata, а из active function tables. .pdata - обычный источник, но не единственный.

6.8.22. Manual mapping и packers: когда PE на диске и image в памяти расходятся

Manual mapping - ручная загрузка PE без обычного loader-а или с частичным повторением его работы. Packer - упаковщик, который хранит настоящий код в сжатом/зашифрованном виде и распаковывает его в runtime. В обоих случаях статический PE на диске может быть не похож на тот код, который реально исполняется.

Для exception/unwind анализа это особенно болезненно. На диске .pdata может быть маленькой, пустой или относиться только к unpacking stub. Настоящий код появится позже. Если этот код x64 non-leaf и должен корректно обрабатываться при исключениях, ему нужны function tables. Некоторые packer-ы могут не заботиться о корректном unwind, если исключения в распакованном коде не предполагаются. Другие могут регистрировать динамические таблицы. Третьи могут намеренно ломать stack walking.

Реверсер должен различать “я анализирую файл до распаковки” и “я анализирую runtime image после распаковки”. Если исключение происходит в динамической области, смотреть только статическую .pdata исходного файла бессмысленно. Нужно смотреть active memory, registered function tables, runtime modules, allocation protections и debugger events.

6.8.23. Как это видит debugger: exception events, context и continuation status

Debugger работает с exception pipeline не как внешний зритель, а как участник. Когда возникает debugging event, система может остановить потоки debuggee и передать событие debugger-у. Debugger смотрит EXCEPTION_DEBUG_EVENT, ExceptionRecord, код исключения, first chance/second chance смысл, затем возвращает continuation status через ContinueDebugEvent.

Continuation status - статус продолжения. Если debugger говорит DBG_CONTINUE, он сообщает, что событие обработано debugger-ом. Если DBG_EXCEPTION_NOT_HANDLED, система продолжает обычный маршрут доставки исключения. Для реверсера это важно: нажатие “continue” в отладчике не всегда означает одно и то же. Ты можешь продолжить так, что VEH/SEH вообще не увидят исключение, или так, что событие будет передано дальше.

Поэтому при dynamic analysis нужно фиксировать, что именно произошло. “Я нажал continue” - недостаточно. Нужно понимать, какой continuation status использовал debugger. В x64dbg, WinDbg или Visual Studio это может быть скрыто за UI, но смысл один: debugger может обработать exception сам или дать Windows продолжить dispatch.

Если ты анализируешь deliberate int 3, особенно важно не “съесть” его debugger-ом. Если debugger обработал breakpoint как обычную точку останова, VEH callback может не получить событие. Тогда ты сделаешь ложный вывод, что программа ломается или handler не работает. Правильный эксперимент - передать first chance breakpoint системе и посмотреть, есть ли handler.

6.8.24. Практическая последовательность анализа exception-heavy бинаря

Теперь соберём рабочую последовательность. Это не жёсткий алгоритм для всех случаев, а спокойный порядок вопросов, который помогает не утонуть.

Сначала определить архитектуру и формат: PE32+ или нет, x64 или x86, обычный module или что-то странное. Это важно, потому что x86 SEH и x64 table-based unwind - разные модели. Потом проверить Exception Directory: есть ли .pdata, сколько RUNTIME_FUNCTION записей, покрывают ли они .text, есть ли крупные дыры, есть ли странные диапазоны.

Затем пройти к .xdata: валидны ли UnwindInfoAddress, какие функции имеют EHANDLER/UHANDLER/CHAININFO, встречается ли __C_specific_handler, есть ли необычные handler addresses. После этого посмотреть imports и dynamic resolving: AddVectoredExceptionHandler, AddVectoredContinueHandler, RemoveVectoredExceptionHandler, RaiseException, SetUnhandledExceptionFilter, UnhandledExceptionFilter, MiniDumpWriteDump, VirtualProtect, IsDebuggerPresent, NtQueryInformationProcess, RtlAddFunctionTable.

Дальше искать call sites. Import сам по себе мало значит. Нужно увидеть, где handler регистрируется, какой callback передаётся, когда это происходит относительно main logic. Затем анализировать callback-и и filters: какие ExceptionCode проверяются, какие fields EXCEPTION_RECORD/CONTEXT читаются, меняется ли RIP/RSP/registers, возвращается ли CONTINUE_EXECUTION или CONTINUE_SEARCH.

Потом связать sources и handlers: где RaiseException, int 3, guard pages, invalid instructions, suspicious memory accesses. Сопоставить коды: source генерирует X, handler проверяет X. Если связь есть, построить exception edge. Если связи нет, оставить как hypothesis.

И только после этого делать поведенческий вывод: normal crash reporting, compiler-generated SEH, deliberate control flow, anti-debug, packer/unpacker, malformed metadata, dynamic code. Такой порядок не гарантирует стопроцентную истину, но он заставляет не пропускать слои.

6.8.25. Как формулировать findings для будущего static -a

Finding - находка анализа. В хорошем инструменте finding - это не крик “опасно”, а маленькое расследование: что найдено, где найдено, почему это важно, какие доказательства, какая уверенность, какие альтернативные объяснения. Для exception/unwind темы это особенно важно, потому что одни и те же признаки могут быть нормальными или подозрительными в зависимости от контекста.

```
Плохой finding:
    Найден AddVectoredExceptionHandler. Возможно malware.

Лучший finding:
    В функции RVA 0x401000 найден вызов AddVectoredExceptionHandler(First=1,
    Handler=RVA 0x4052A0). Callback проверяет EXCEPTION_BREAKPOINT и
    изменяет ContextRecord->Rip перед возвратом EXCEPTION_CONTINUE_EXECUTION.
    Это создаёт exception-based control-flow edge для int 3 instructions.
    Severity: suspicious. Confidence: high. Alternative: custom instrumentation.
```

Такой формат полезен человеку. Он не заставляет верить инструменту на слово. Он показывает маршрут. Реверсер может открыть RVA, проверить callback, увидеть изменение RIP и подтвердить или опровергнуть вывод.

Для unwind anomalies finding должен быть таким же доказательным. Не “битая .xdata”, а “RUNTIME_FUNCTION RVA A-B указывает на UnwindInfoAddress C; C вне readable image range” или “UNWIND_INFO declares UWOP_ALLOC_SMALL 0x40, but detected prolog subtracts RSP by 0x80; possible mismatch or incorrect function start”. Чем точнее формулировка, тем меньше магии.

6.8.26. Мини-кейс: от импорта до скрытого перехода

Представим маленький кейс. В imports есть AddVectoredExceptionHandler и RaiseException. В init-функции найден вызов AddVectoredExceptionHandler с callback VehA. В основной функции найден RaiseException с кодом 0xE1234567. В VehA есть сравнение ExceptionRecord->ExceptionCode с 0xE1234567. Если совпало, callback читает RAX из ContextRecord, прибавляет его к базовому адресу таблицы и записывает результат в ContextRecord->Rip, затем возвращает CONTINUE_EXECUTION.

Без exception-aware анализа основная функция выглядит странно: вызвали RaiseException, а дальше будто программа должна была уйти в системный dispatcher или crash. С exception-aware анализом видно: RaiseException - это не конец, а dispatch instruction. Код исключения выбирает VehA. VehA вычисляет новый RIP. Значит, реальный control flow идёт через handler. Это похоже на switch, но реализованный не через jmp table напрямую, а через exception pipeline.

Какие слои мы использовали? Import table дала начальный признак. Calling convention помогла понять аргументы AddVectoredExceptionHandler и RaiseException. EXCEPTION_RECORD объяснил ExceptionCode. CONTEXT объяснил изменение RIP. Модель VEH объяснила, почему frame-based SEH может не начаться. Модель debugger-а объяснила, почему под отладчиком маршрут может отличаться. Вот она - взаимосвязь, ради которой строился фундамент.

6.8.27. Мини-кейс: обычный crash reporter, который не надо путать с защитой

Теперь другой кейс. В imports есть SetUnhandledExceptionFilter и MiniDumpWriteDump. В начале программы устанавливается CrashFilter. CrashFilter получает EXCEPTION_POINTERS, создаёт файл crash.dmp, заполняет MINIDUMP_EXCEPTION_INFORMATION, вызывает MiniDumpWriteDump, закрывает handles и возвращает EXCEPTION_EXECUTE_HANDLER. Он не меняет ContextRecord, не проверяет int 3, не ставит PAGE_GUARD, не вызывает VirtualProtect.

Здесь тоже есть exception handling, но смысл другой. Это не скрытый control flow, а поздняя диагностика. Если access violation дошёл до CrashFilter, программа уже не нашла ранний handler. CrashFilter сохраняет снимок для разработчика. В отчёте это нужно назвать normal crash reporting или diagnostic top-level filter, а не suspicious exception abuse.

Разница между кейсами не в наличии API. В обоих есть exception-related functions. Разница в роли слоя и действии callback-а. Первый callback меняет RIP и продолжает выполнение. Второй пишет dump и завершает. Поэтому основное правило практического анализа: смотри не только “что вызвано”, но и “что callback делает с маршрутом”.

6.8.28. Мини-кейс: SEH в функции и почему без .xdata ты не увидишь всю картину

Представим функцию, в которой есть __try/__except. В обычном дизассемблере ты видишь risky call и где-то ниже блок recovery. Между ними может не быть явного jmp, который говорит “если exception - сюда”. Связь живёт в metadata: .pdata указывает на UNWIND_INFO, UNWIND_INFO имеет EHANDLER, handler RVA ведёт к __C_specific_handler, handler data описывает protected range и handler block.

Если анализировать только инструкции, recovery block может казаться unreachable или странно достижимым. Если анализировать только .xdata без кода, ты узнаешь про handler, но не поймёшь смысл. Правильный анализ соединяет: faulting ControlPc попадает в protected range; filter возвращает EXECUTE_HANDLER; handler block соответствует recovery code. Тогда структура C SEH становится видна даже без исходника.

Этот кейс показывает, почему книга так долго объясняла .pdata и .xdata до практики. Для Windows x64 SEH логика часто лежит не только в обычном потоке инструкций. Она распределена между кодом и метаданными. Реверсер должен читать оба слоя как один текст.

6.8.29. Что записывать в рабочие заметки реверсера

Когда анализируешь exception-heavy код, рабочие заметки должны быть структурированными. Иначе через час ты забудешь, какой handler к какому source относился. Хорошая заметка должна содержать: address/RVA source-a, тип exception source-a, expected ExceptionCode, зарегистрированные handlers, layer handler-a, действие handler-a, return disposition, изменение ContextRecord, связанные .pdata/.xdata entries, уверенность.

```
Шаблон заметки:

Source:
    Module: sample.exe
    RVA: 0x1400
    Type: int 3 / RaiseException / guard page / access violation
Exception:
    Code: 0x80000003 / 0xC0000005 / custom
Handler layer:
    VEH / frame-based SEH / top-level / debugger
Handler location:
    RVA or symbol
Action:
    continue search / execute handler / continue execution / modify RIP
Metadata:
    RUNTIME_FUNCTION range, UNWIND_INFO flags, handler data
Conclusion:
    diagnostic / normal EH / suspicious control flow / unknown
```

Такой шаблон будущий static может превратить в структуру данных. Но даже вручную он помогает думать. Ты перестаёшь видеть “много странных исключений” и начинаешь видеть конкретные связи.

6.8.30. Главная модель раздела

Практический reverse engineering exception-механизмов - это искусство связывать признаки с маршрутом. Import table даёт нам возможные API. Call sites показывают регистрацию и источники исключений. .pdata показывает диапазоны функций. .xdata показывает unwind и handler flags. __C_specific_handler указывает на C SEH data. VEH callback показывает ранний процессный слой. Top-level filter показывает поздний аварийный слой. Debugger меняет наблюдаемое поведение через first chance и second chance. Guard pages и deliberate exceptions превращают memory/protection events в сигналы управления.

Главное - не читать эти вещи отдельно. AddVectoredExceptionHandler без callback-а - слабый признак. int 3 без handler-а - неясный источник. __C_specific_handler без scope data - только вход в языковой слой. .pdata без сверки с кодом - неполная карта. SetUnhandledExceptionFilter без анализа callback-а - неизвестно, crash reporter это или попытка восстановления. Только когда источник, handler, metadata и действие соединяются, появляется настоящая картина.

Именно поэтому мы строим фундамент медленно. В следующих главах мы сможем перейти к ручной модели анализа: как пройти от RIP к RUNTIME_FUNCTION, как прочитать .pdata, как разобрать .xdata, как сверить UWOP_* с прологом и как мысленно выполнить RtlVirtualUnwind. Но теперь у нас есть практический смысл этих действий. Мы не будем парсить структуры ради структур. Мы будем восстанавливать маршрут: где произошло событие, как Windows нашла кадр, какой handler увидел исключение, почему управление пошло туда, а не сюда, и можно ли доверять этой картине.

6.8.31. Vectored continue handlers: поздний родственник VEH, который легко забыть

Vectored continue handler - это ещё один vectored-механизм, который работает не так, как обычный vectored exception handler. В обычной цепочке VEH получает шанс рано: после first chance debugger notification и до frame-based search. Vectored continue handlers относятся к сценарию продолжения после того, как система уже пришла к решению о продолжении выполнения. На практике они встречаются реже, но для полной карты исключений их нельзя забывать.

Почему это важно для реверсера? Потому что программа может использовать не только AddVectoredExceptionHandler, но и AddVectoredContinueHandler. Если анализатор ищет только первый API, он может пропустить часть exception-aware логики. Особенно это важно в защитном коде, где авторы любят редкие механизмы: чем менее привычен слой, тем выше шанс, что инструмент или человек его не учтёт.

Практически при встрече AddVectoredContinueHandler нужно задать те же вопросы, что и для VEH: где call site, какой callback, что callback читает из EXCEPTION_POINTERS, меняет ли ContextRecord, какие exception codes ожидает. Но вывод формулируется аккуратнее: это не ранний перехват до unwind, а слой, связанный с продолжением. В отчёте нельзя смешивать его с обычным VEH, иначе порядок маршрута будет неверным.

6.8.32. RemoveVectoredExceptionHandler: почему снятие handler-а тоже является поведением

RemoveVectoredExceptionHandler - функция удаления ранее зарегистрированного vectored handler-а. На первый взгляд это неинтересно: ну зарегистрировали, потом удалили. Но в reverse engineering снятие handler-а может быть важным признаком. Программа может регистрировать handler только на короткий участок, выполнить deliberate exception, затем удалить handler, чтобы статический или динамический наблюдатель позже его не увидел в активном состоянии.

Это похоже на временную установку ловушки. Если ты запускаешь программу и проверяешь handlers слишком поздно, ты можешь не увидеть того, что уже сыграло свою роль. Поэтому call sites RemoveVectoredExceptionHandler нужно связывать с lifetime handler-а. Lifetime - время жизни. Когда handler установлен? До какого участка он действует? Что происходит между Add и Remove? Есть ли там RaiseException, int 3, guard page access, invalid instruction?

Для будущего static -а это значит, что finding должен учитывать последовательность. “Handler registered and later removed” - не то же самое, что “handler permanently registered”. В защитном коде временные handlers могут быть особенно важны, потому что они создают короткие exception windows: маленькие участки выполнения, где исключение имеет специальный смысл.

6.8.33. Invalid instruction и UD2: когда процессор сам создаёт понятный trap

Invalid instruction - недопустимая инструкция. На x86/x64 есть инструкция UD2, которую специально используют как guaranteed invalid instruction: она должна вызвать invalid opcode exception. Компиляторы могут вставлять подобные traps для unreachable code, sanitizer-ов, assert-like ситуаций или защиты. Обфускатор может использовать их как deliberate exception source.

В дизассемблере UD2 выглядит более “аварийно”, чем int 3. Breakpoint легко связать с отладчиком, а invalid instruction кажется настоящей невозможностью продолжения. Но если рядом есть handler, который ловит EXCEPTION_ILLEGAL_INSTRUCTION и меняет ContextRecord->Rip, то UD2 тоже становится управляемым узлом control flow.

Практически проверка такая же: не смотреть на instruction alone. Нужно смотреть, есть ли обработчик этого exception code, меняется ли RIP, пропускается ли инструкция, вычисляется ли новый адрес. Если UD2 находится в unreachable block после compiler optimization, это может быть нормальный trap. Если UD2 встречается регулярно между смысловыми блоками и есть VEH/SEH, это уже похоже на exception-based obfuscation.

6.8.34. Single-step exception и trap flag: тонкая связка с отладкой и защитой

Single-step exception возникает, когда процессор после выполнения одной инструкции сообщает о событии трассировки. На x86/x64 это связано с trap flag в EFLAGS/RFLAGS. Отладчики используют single-step для пошагового выполнения. Защитный код может использовать тот же механизм, чтобы обнаружить трассировку или организовать собственный mini-debugger внутри процесса.

Для реверсера single-step важен потому, что это exception, который может происходить очень часто и сильно менять наблюдаемую картину. Если handler включает trap flag, выполняется одна инструкция, затем возникает EXCEPTION_SINGLE_STEP, handler снова смотрит контекст, решает, что делать дальше, и так далее. Такой механизм может использоваться для расшифровки инструкций по одной, контроля трассировки или запутывания обычного debugger-а.

Практические признаки: обработка EXCEPTION_SINGLE_STEP, запись в EFlags/RFlags внутри ContextRecord, проверка TF-бита, связка с PAGE_GUARD, int 3 или hardware breakpoints. Здесь особенно важно помнить, что ContextRecord - не пассивный снимок. Если handler пишет в flags, он меняет будущую работу процессора.

6.8.35. ExceptionInformation: маленький массив, который часто несёт главный смысл

В EXCEPTION_RECORD есть массив ExceptionInformation. Новичок может пропустить его, потому что главными кажутся ExceptionCode и ExceptionAddress. Но для некоторых exception codes именно дополнительные параметры объясняют, что произошло. Например, для access violation первый параметр говорит тип операции: чтение, запись или DEP/execution violation, а второй параметр указывает проблемный virtual address.

Для реверсера это означает: handler, который читает ExceptionInformation[0] и [1], не просто делает странные обращения к массиву. Он узнаёт, что именно произошло с памятью. Если значение 0 - чтение, 1 - запись, 8 - попытка исполнения, то handler может различать разные сценарии. Например, выполнение неисполняемой страницы может использоваться для unpacking-а: handler меняет protection и продолжает. Запись в guard page может использоваться как watchpoint.

В будущих главах, когда мы будем строить анализатор, это станет отдельной проверкой: если handler проверяет ExceptionInformation, нужно распознать семантику этих индексов для конкретного ExceptionCode. Нельзя просто вывести “читает параметр 1”. Нужно объяснить человеку: “проверяет faulting address” или “различает read/write/execute access”.

6.8.36. CONTEXT как место, где handler может переписать будущее

CONTEXT часто называют snapshot-ом регистров, то есть снимком состояния. Но для handler-а это не только фотография. В некоторых ситуациях это редактируемая карта будущего выполнения. Если handler возвращает EXCEPTION_CONTINUE_EXECUTION после изменения ContextRecord, Windows попытается продолжить поток уже с новым состоянием.

Самое заметное поле - RIP. Изменить RIP значит изменить следующую инструкцию. Но не только RIP важен. Изменение RSP может переключить стековый контекст или сломать его. Изменение RAX/RCX/RDX может подставить возвращаемое значение или аргумент. Изменение EFlags/RFlags может изменить результаты условных переходов или включить trap flag. Поэтому handler может быть почти как маленький интерпретатор, который правит состояние CPU.

Практически это значит: в callback-е или filter-е нужно искать writes, а не только reads. Read ExceptionCode - это фильтр. Write ContextRecord->Rip - это control flow. Write

ContextRecord->Rax - возможно, подмена результата. Write ContextRecord->EFlags - возможно, трассировка или условная логика. Именно записи в CONTEXT превращают exception handling из “реакции на ошибку” в механизм управления выполнением.

6.8.37. Почему return value handler-а нужно читать вместе с изменениями контекста

Return value handler-а - это не формальность. EXCEPTION_CONTINUE_SEARCH означает “я не обработал, ищите дальше”. EXCEPTION_CONTINUE_EXECUTION означает “попробуйте продолжить выполнение”. EXCEPTION_EXECUTE_HANDLER в SEH filter-е означает “переходите к handler block”. Эти значения похожи на маленькие integers, но они управляют всем маршрутом.

Важно читать return value вместе с контекстом. Если callback меняет RIP, но возвращает CONTINUE_SEARCH, изменение может не дать ожидаемого продолжения, потому что поиск пойдёт дальше. Если callback не меняет причину access violation, но возвращает CONTINUE_EXECUTION, та же инструкция может снова вызвать исключение. Если filter возвращает EXECUTE_HANDLER, система ещё должна выполнить unwind phase до handler block-а.

Поэтому при анализе нельзя останавливаться на одном признаке. “Меняет RIP” и “возвращает CONTINUE_EXECUTION” вместе дают сильный вывод о hidden edge. “Проверяет ExceptionCode” и “возвращает CONTINUE_SEARCH” говорят, что handler только пропускает неинтересные события. “Пишет dump” и “возвращает EXECUTE_HANDLER” говорят о crash reporting. Смысл рождается из пары: действие + disposition.

6.8.38. Почему порядок регистрации handler-ов может менять результат

У vectored handlers есть порядок. AddVectoredExceptionHandler принимает параметр First, который влияет на то, будет ли handler помещён ближе к началу цепочки. Если несколько библиотек или компонентов регистрируют VEH, первый вызванный handler может изменить контекст или обработать исключение так, что остальные его уже не увидят. Поэтому порядок регистрации - это не мелочь.

Практически это означает: если найдено несколько AddVectoredExceptionHandler, нужно смотреть sequence. Какая функция вызывается раньше? First=1 или 0? Есть ли Remove? Есть ли повторная регистрация? Один handler может быть логгером, другой - защитой, третий - runtime-механизмом. Если защитный handler стоит первым, он может перехватывать события до crash reporter-а или до инструментации.

В статическом анализе порядок иногда трудно доказать, особенно если регистрации происходят в разных init-функциях, TLS callbacks, DllMain, constructors. Поэтому finding может иметь uncertainty: “static order appears to be...” или “runtime confirmation required”. Это честнее, чем уверенно заявлять порядок только по адресам функций.

6.8.39. TLS callbacks: где exception logic может появиться до main

TLS callback - callback, связанный с Thread Local Storage initialization. Для реверсера TLS callbacks важны потому, что они могут выполниться до привычной точки входа runtime/main. Программа может зарегистрировать VEH, поставить top-level filter, настроить guard pages или сделать антидебаговую проверку ещё до того, как ты дошёл до main-like логики.

Если анализировать только main или entry point после runtime setup, можно пропустить раннюю настройку exception pipeline. Поэтому в PE-анализе нужно смотреть TLS Directory. Если там есть callbacks, они являются ранними участниками запуска. В packed/protected бинарях TLS callbacks часто используются для антидебага и ранней инициализации защиты.

Связь с текущей темой простая: handler registration может произойти не там, где новичок её ждёт. AddVectoredExceptionHandler в TLS callback-е означает, что к моменту выполнения основной логики VEH уже активен. Тогда deliberate int 3 в main code не “сам по себе”, а часть заранее подготовленного маршрута.

6.8.40. Delay-load и GetProcAddress: почему отсутствие импорта не снимает подозрение

Мы уже сказали, что import table даёт только потенциальные намерения. Теперь добавим важное практическое исключение: функция может быть использована без прямого статического импорта. Программа может вызвать LoadLibrary/GetProcAddress, получить адрес AddVectoredExceptionHandler или RaiseException по имени, по ordinal-у или через hash. Тогда в Import Directory нужного имени не будет.

Для реверсера это значит: если exception behavior есть в runtime, но imports “чистые”, нужно искать dynamic resolution. Строки с именами API, hashes, walking PEB loader lists, ручной парсинг export table kernel32/ntdll, indirect calls через function pointers. В protected коде это особенно частый приём: скрыть явные imports, чтобы статический сканер не увидел AddVectoredExceptionHandler или IsDebuggerPresent.

Практически полезно помечать indirect call sites, куда передаются адреса из GetProcAddress-like resolver-a. Если resolver возвращает указатель и потом этот указатель вызывается с аргументами, похожими на регистрацию VEH, можно восстановить API даже без имени. Здесь снова помогает calling convention: RCX/RDX/R8/R9 и форма аргументов подсказывают смысл вызова.

6.8.41. Что значит “понять exception-механику” для человека

Понять exception-механику - это не запомнить список API. Это уметь рассказать историю выполнения. Хорошая история отвечает на вопросы: где событие возникло, почему оно возникло, какое состояние было в ContextRecord, кто получил первый шанс, какой handler посмотрел событие, почему он принял или не принял его, что изменилось в контексте, куда поток пошёл дальше, какие метаданные Windows использовала по пути.

Если ты можешь рассказать эту историю простыми словами, значит механизм понят. Если ты можешь только перечислить: “тут VEH, тут SEH, тут .pdata”, значит связи ещё слабые. Поэтому в книге мы не торопимся. Тебе нужен фундамент не для экзамена по терминам, а для способности открыть незнакомый Windows x64 бинарь и не потеряться.

Дальше, когда мы перейдём к ручному чтению .pdata и .xdata, постоянно держи этот раздел в голове. Мы будем читать структуры не ради красоты. Каждое поле будет отвечать на практический вопрос. BeginAddress/EndAddress - где функция для текущего RIP. UnwindInfoAddress - где рецепт выхода. Flags - есть ли handler. UWOP_* - как восстановить RSP и nonvolatile registers. Handler data - какой языковой слой принимает решение. Все эти поля - части одной истории.

Техническая база раздела 6.8

Раздел опирается на официальные материалы Microsoft по Vectored Exception Handling, AddVectoredExceptionHandler, RaiseException, EXCEPTION_RECORD, GetExceptionCode/GetExceptionInformation, Creating Guard Pages, IsDebuggerPresent, NtQueryInformationProcess, Writing the Debugger Main Loop, SetUnhandledExceptionFilter, MiniDumpWriteDump и x64 exception/unwind model. Важные технические опоры: VEH является процессным обработчиком, который получает EXCEPTION_POINTERS и может вернуть CONTINUE_SEARCH или CONTINUE_EXECUTION; RaiseException создаёт software exception в вызывающем потоке и заполняет ExceptionRecord; guard page создаёт одноразовый сигнал доступа STATUS_GUARD_PAGE_VIOLATION и после первого доступа теряет PAGE_GUARD статус; IsDebuggerPresent сообщает, выполняется ли процесс под user-mode debugger-ом; debugger получает exception debug events и управляет продолжением через ContinueDebugEvent; .pdata/.xdata и UNWIND_INFO остаются основой frame-based x64 unwinding. Текст раздела написан заново как учебная глава и связывает эти факты с практическим reverse engineering.

Часть II

Проектная и прикладная часть

Дальше сохранены главы 7-12 и приложения из v87: ручная модель анализа, архитектура EH/Unwind Analyzer, static analysis, dynamic validation, reports и каталог 27 static findings.

Эта часть связывает фундамент Windows x64 unwind с конкретным проектом.

Глава 7. Практическая ручная модель анализа

7.1. Как руками пройти путь от RIP до функции: VA, ImageBase, RVA и RUNTIME_FUNCTION

В предыдущей главе мы собрали большой маршрут доставки исключения: fault, ExceptionRecord, ContextRecord, debugger first chance, VEH, frame-based SEH, unwind phase, top-level handling и crash. Но если оставить эту схему только как красивую общую картину, она быстро забудется. Настоящее понимание появляется тогда, когда ты можешь взять один конкретный адрес, например значение RIP из crash dump или отладчика, и медленно провести его через все координатные системы Windows x64: от виртуального адреса в процессе к модулю, от модуля к ImageBase, от ImageBase к RVA, от RVA к .pdata, от .pdata к RUNTIME_FUNCTION, а затем к .xdata и UNWIND_INFO.

Именно этот ручной путь - фундамент будущего static -а. Любой автоматический анализатор, каким бы умным он ни выглядел, внутри всё равно должен решать эту базовую задачу: “у меня есть адрес выполнения; какому куску кода он принадлежит и какие unwind-данные описывают этот кусок?” Если человек сам не умеет пройти этот путь, он не сможет проверить инструмент. Он будет верить выводу анализатора как магии. А в reverse engineering это опасно: инструменты ошибаются, дизассемблеры угадывают, debug symbols отсутствуют, PE может быть нестандартным, а обфускация специально ломает привычные ожидания.

Поэтому в этой главе мы начинаем не с парсинга всех полей .pdata подряд, а с более важного умения: как мыслить координатами. На Windows x64 один и тот же “адрес” может быть VA, RVA, file offset, displacement внутри инструкции, pointer в памяти, индексом в таблице или смещением от начала функции. Если эти системы координат смешать, весь анализ дальше превращается в мусор. Ты можешь найти правильный RUNTIME_FUNCTION, но читать его как file offset. Можешь правильно увидеть VA в отладчике, но искать его в PE-файле как сырое смещение. Можешь правильно вычислить RVA, но сравнить его с диапазоном из другого модуля. Все эти ошибки выглядят маленькими, но ломают саму основу unwind-анализа.

Раздел будет идти медленно. Мы будем повторять уже знакомые слова, но теперь не как отдельные определения, а как рабочий маршрут. RIP отвечает на вопрос “где поток был в коде”. Module отвечает на вопрос “какому загруженному PE-образу принадлежит этот адрес”. ImageBase даёт точку отсчёта внутри этого образа. RVA переводит живой виртуальный адрес в координату внутри image. Exception Directory указывает, где начинается таблица функций. .pdata хранит массив RUNTIME_FUNCTION. RUNTIME_FUNCTION связывает диапазон кода с UnwindInfoAddress. И только после этого мы вообще имеем право говорить: “вот unwind-описание функции, в которой произошло исключение”.

7.1.1. Почему путь от RIP к функции важнее, чем кажется

Когда программа падает, человек обычно спрашивает: “в какой функции она упала?” Отладчик часто сразу показывает имя функции, например app.exe!parse_header+0x35. И из-за этого создаётся иллюзия, что функция - это простая данность. Но для Windows и для твоего будущего анализатора имя функции не является первой реальностью. Первая реальность - это машинный адрес. В ContextRecord лежит RIP. Этот RIP - число. Система должна понять, что это число означает.

RIP - instruction pointer, указатель инструкции. На x64 это регистр, который связан с текущей точкой выполнения. Если исключение произошло на инструкции чтения памяти, RIP указывает на эту инструкцию или на адрес, связанный с fault-ом. В человеческом смысле RIP отвечает на вопрос: “где именно поток был остановлен?” Но сам по себе RIP ещё не говорит, какая это функция. Он не говорит, где начало функции, где конец, есть ли у функции handler, сколько она выделила стека и где лежит UNWIND_INFO.

Function name - имя функции - в релизном бинаре может вообще отсутствовать. Оно может прийти из PDB symbols, export table, debug information, heuristic analysis дизассемблера или вообще из домысла инструмента. Windows exception dispatcher не нуждается в красивом имени parse_header. Ему нужно другое: function table entry, то есть запись таблицы функций. На x64 обычной такой записью является RUNTIME_FUNCTION.

Вот почему путь “RIP -> функция” на самом деле означает не “найти имя”, а “найти диапазон кода и unwind metadata”. Для reverse engineering это более честный взгляд. Если у тебя есть имя, отлично. Но даже без имени ты можешь знать, что адрес 0x140012345 попадает в диапазон RVA 0x12000..0x126A0, а этот диапазон описан записью RUNTIME_FUNCTION, которая указывает на UNWIND_INFO по RVA 0x31080. Это уже настоящая техническая информация. Имя функции - приятная надпись на карте. RUNTIME_FUNCTION - часть самой карты.

Эта мысль особенно важна при анализе malware, packers, stripped binaries и crash dumps. Там имён может не быть. Но если .pdata и .xdata корректны, Windows всё равно может раскручивать стек. Значит, реверсер тоже может использовать эти таблицы. Нужно только уметь начать с RIP и не заблудиться в координатах.

```
Человеческий вопрос:
    В какой функции упала программа?

Технический вопрос Windows x64:
    Какой RUNTIME_FUNCTION покрывает ControlPc/RIP,
    и где находится связанный UNWIND_INFO?
```

7.1.2. ControlPc: почему в документации не всегда пишут просто RIP

В документации Microsoft рядом с RtlLookupFunctionEntry часто встречается термин ControlPc. Его можно перевести как “адрес управления” или “адрес текущего управления”. PC - program counter, счётчик программы. На x64 роль program counter фактически играет RIP. Поэтому новичок обычно думает: ControlPc - это просто RIP. В большинстве начальных объяснений так можно говорить, но лучше сразу понимать тонкость.

RIP - конкретный регистр архитектуры x64. ControlPc - более общий термин уровня unwind/dispatcher API. Это адрес, по которому система ищет function table entry. Он может быть взят из ContextRecord->Rip, но в некоторых ситуациях может корректироваться правилами unwind или быть адресом, связанным с caller context при подъёме по стеку. Когда RtlVirtualUnwind вычисляет предыдущий кадр, новый ControlPc уже относится к caller-у, а не к месту исходного fault-а.

Почему это важно? Потому что exception analysis - это не один статичный адрес. В начале у нас есть RIP места исключения. Потом система виртуально поднимается к caller-у. На каждом шаге появляется новый ControlPc: адрес возврата в вызывающей функции. Для каждого нового ControlPc снова вызывается lookup в function tables. Поэтому маршрут не “один раз нашли

функцию и всё”, а повторяющийся цикл.

```
Первый шаг:
    ControlPc = ContextRecord.Rip   // место fault-a

После virtual unwind:
    ControlPc = caller ContextRecord.Rip

Следующий шаг:
    снова RtlLookupFunctionEntry(ControlPc, ...)
```

В ручном анализе это означает: если ты идёшь по стеку вверх, не надо держаться только за исходный RIP. Исходный RIP объясняет место падения. Но каждый следующий кадр имеет свой адрес управления. Именно поэтому stack walking на x64 - это серия преобразований контекста, а не поиск одного “главного адреса”.

7.1.3. VA: адрес, который видит живой процесс

VA - Virtual Address, виртуальный адрес. По-русски: адрес в виртуальном адресном пространстве процесса. Когда ты видишь в x64dbg, WinDbg или crash dump адрес вроде 00007FF7A0123456, это обычно VA. Процессор выполняет инструкции по виртуальным адресам. Память читается по виртуальным адресам. RIP хранит виртуальный адрес. ExceptionRecord->ExceptionAddress тоже обычно указывает адрес в виртуальной памяти процесса.

Virtual address не означает “адрес в файле”. Это первое, что нужно отбить в голове. Файл лежит на диске. Процесс живёт в виртуальной памяти. Loader взял PE-файл, отобразил его секции в память, применил релокации, разрешил импорты, выставил page protections. В результате код, который физически лежал где-то по file offset 0x400, может выполняться по VA 0x140001000. Эти числа связаны, но не равны.

На x64 VA обычно выглядит большим. Например, main module может быть загружен по базе 0x00007FF700000000 или 0x0000000140000000. System DLL вроде ntdll.dll часто живут в диапазонах 0x00007FFB... Это не правило для угадывания, а просто привычная картина. Главное: VA имеет смысл только внутри конкретного процесса и конкретного запуска. Из-за ASLR один и тот же модуль может получить другую базу при другом запуске. Поэтому “адрес 0x7FF7A0123456” без module + RVA - плохая долговременная координата.

ASLR - Address Space Layout Randomization, рандомизация раскладки адресного пространства. Это защитный механизм, который может менять базы загрузки модулей. Для нас важен практический итог: абсолютный VA нестабилен между запусками, а RVA внутри модуля обычно стабильнее. Поэтому реверсер часто переводит VA в пару “module + RVA”.

```
Плохая долговременная запись:
    crash at 00007FF7A0123456

Лучше:
    crash in app.exe + RVA 0x123456

Ещё лучше:
    app.exe, RVA 0x123456, inside RUNTIME_FUNCTION 0x123400..0x1234F0
```

7.1.4. Module: сначала нужно понять, какому образу принадлежит VA

Module - модуль. В Windows-процессе модулем обычно называют загруженный PE-образ: основной EXE или DLL. У каждого модуля есть base address, size, sections, imports, exports, exception directory и другие свойства. Когда мы имеем VA из RIP, первый практический вопрос звучит так: какой module содержит этот VA?

Почему это первый вопрос? Потому что RVA считается относительно ImageBase конкретного модуля. Нельзя взять VA и вычесть “какую-нибудь базу”. Нужно вычесть базу именно того образа, которому адрес принадлежит. Если RIP находится в ntdll.dll, а ты вычтешь ImageBase своего app.exe, получишь бессмысленный RVA. Если RIP находится в dynamically allocated memory, а ты будешь искать его в списке обычных PE-модулей, тоже получишь неправильную картину.

В отладчике модуль обычно можно увидеть по командам списка модулей или по символическому выводу. WinDbg может показать что-то вроде app!function+0x35 или ntdll!RtlDispatchException+0x... Но если символов нет, всё равно можно найти диапазон: module base <= VA < module base + SizeOfImage. Это простая проверка принадлежности адреса модулю.

```
Проверка принадлежности VA модулю:

if ModuleBase <= VA < ModuleBase + SizeOfImage:
    VA принадлежит этому module
else:
    ищем другой module или dynamic code region
```

SizeOfImage - размер образа в памяти. Он берётся из PE Optional Header и описывает, какой виртуальный диапазон занимает загруженный image. Здесь снова важно: SizeOfImage - не размер файла на диске. Это размер virtual layout. Если модуль имеет ImageBase 0x140000000 и SizeOfImage 0x250000, его виртуальный диапазон примерно 0x140000000..0x140250000.

Типичная ошибка: видеть VA рядом с известным модулем и автоматически считать, что адрес внутри него. Надо проверять диапазон. В больших процессах много модулей, ASLR меняет базы, JIT-код может быть вне модулей, а manual mapping может создать область, которая не зарегистрирована как обычный loader module. Поэтому правило “сначала module” - не формальность, а защита от неверной системы координат.

7.1.5. ImageBase: точка отсчёта для всего, что живёт внутри модуля

ImageBase - база образа. Это адрес, с которого загруженный PE image начинается в виртуальном адресном пространстве процесса. Если module - это здание, ImageBase - адрес его главного входа. Все RVA внутри этого здания считаются от этой точки.

В PE Optional Header есть preferred ImageBase, то есть желательная база, которую записал linker. Но фактическая база в процессе может отличаться из-за ASLR или конфликта адресов. В runtime-анализе важна фактическая база. Именно её возвращает lookup logic или показывает отладчик. Именно от неё нужно вычитать VA, чтобы получить RVA.

Формула простая, но её нужно прожить, а не просто выучить:

```
RVA = VA - ImageBase
VA  = ImageBase + RVA
```

Допустим, RIP равен 0x00007FF7A0123456. Отладчик показывает, что этот адрес принадлежит app.exe, загруженному по ImageBase 0x00007FF7A0000000. Тогда RVA будет 0x00123456. Это уже координата внутри образа app.exe. Именно с таким числом можно сравнивать BeginAddress и EndAddress в RUNTIME_FUNCTION, потому что эти поля в PE32+ x64 function table являются image-relative.

```
VA        = 00007FF7A0123456h
ImageBase = 00007FF7A0000000h
RVA       = 0000000000123456h
```

Почему Windows хранит многие внутренние ссылки как RVA? Потому что RVA не зависит от того, куда образ реально загрузился. Если при следующем запуске app.exe получит базу 0x00007FF6B0000000, тот же участок кода может иметь RVA 0x123456, но VA станет 0x00007FF6B0123456. Для анализа файла RVA часто важнее, чем абсолютный VA.

7.1.6. RVA: координата внутри image, а не внутри файла

RVA - Relative Virtual Address, относительный виртуальный адрес. Это смещение от ImageBase внутри загруженного image. Слово virtual здесь очень важно: RVA относится к виртуальной раскладке образа, то есть к тому, как PE выглядит в памяти после загрузки. Это не raw file offset.

Почему слово “relative” полезно? Потому что оно говорит: адрес не абсолютный, а относительный. RVA 0x123456 сам по себе не говорит, где в памяти находится код. Он говорит: “на 0x123456 байт от базы образа”. Чтобы получить VA, прибавь ImageBase. Чтобы прочитать байты из файла, найди секцию, куда попадает этот RVA, и переведи его в file offset.

В x64 exception/unwind теме RVA появляется постоянно. Exception Directory в Data Directory даёт RVA таблицы исключений. RUNTIME_FUNCTION хранит BeginAddress, EndAddress и UnwindInfoAddress как RVA. Section headers имеют VirtualAddress, который тоже является RVA начала секции. Поэтому если ты плохо понимаешь RVA, ты не сможешь корректно читать .pdata.

Типичная ошибка: открыть PE в hex-редакторе и перейти к offset 0x123456, потому что в RUNTIME_FUNCTION написано 0x123456. Это почти всегда неправильно. Нужно понять, в какой секции этот RVA находится. Если секция .text имеет VirtualAddress 0x1000 и PointerToRawData 0x400, то RVA 0x1234 может соответствовать file offset 0x634, а не 0x1234.

```
RVA -> file offset через section header:

RVA = Section.VirtualAddress + delta
file_offset = Section.PointerToRawData + delta

Где delta = RVA - Section.VirtualAddress
```

Запомни важное разделение: Windows во время выполнения чаще мыслит VA и RVA. Hex-редактор мыслит file offset. PE parser должен уметь ходить между ними. RUNTIME_FUNCTION на x64 даёт тебе RVA. Значит, если ты анализируешь живой процесс, прибавляй ImageBase. Если анализируешь файл на диске, переводи RVA через section table.

7.1.7. File offset: когда он нужен и почему он не участвует в runtime lookup напрямую

File offset - смещение в файле. Это координата на диске: сколько байт от начала PE-файла нужно пройти, чтобы попасть к нужному месту. Hex-редакторы, файловые парсеры и статические анализаторы работают с file offsets. Но CPU и Windows exception dispatcher не выполняют file offset. Они работают с виртуальной памятью.

Если исключение произошло в живом процессе, в ContextRecord->Rip лежит VA. RtlLookupFunctionEntry получает ControlPc как виртуальный адрес и ищет активную function table. В этом runtime-маршруте file offset не нужен как начальная координата. Он нужен тебе, если ты хочешь открыть файл на диске и прочитать соответствующие байты.

Это различие полезно сформулировать очень грубо: VA отвечает на вопрос “где это в живой памяти?”, RVA - “где это внутри образа?”, file offset - “где это в файле на диске?”. Один и тот же байт кода может иметь все три координаты. Но они не равны и не заменяют друг друга.

```
Один участок кода может иметь три адреса:

VA:
    00007FF7A0123456h  // в живом процессе

RVA:
    00123456h          // внутри image app.exe

file offset:
    000F2856h          // внутри файла app.exe на диске, после пересчёта через section table
```

Для будущего static -а это означает: каждое поле в отчёте должно иметь тип координаты. Нельзя писать просто “address: 0x123456” без уточнения. Это RVA? VA? file offset? offset внутри UNWIND_INFO? offset от начала функции? Хороший инструмент явно подписывает систему координат, потому что иначе человек не сможет проверить результат.

7.1.8. Практический пример: переводим RIP в RVA

Теперь пройдём маленький пример медленно. Пусть crash dump говорит, что исключение произошло при RIP:

```
RIP = 00007FF7A0123456h
```

Первое действие - не искать сразу .pdata. Сначала нужно понять, в каком module находится этот VA. Допустим, список модулей показывает:

```
app.exe:
    ImageBase   = 00007FF7A0000000h
    SizeOfImage = 0000000000250000h
```

Проверяем диапазон:

```
Module range:
    00007FF7A0000000h .. 00007FF7A0250000h

RIP:
    00007FF7A0123456h

RIP попадает в этот диапазон.
```

Теперь можно вычислить RVA:

```
RVA = RIP - ImageBase
RVA = 00007FF7A0123456h - 00007FF7A0000000h
RVA = 0000000000123456h
```

Эта операция кажется простой, но она меняет смысл адреса. До неё у нас был абсолютный виртуальный адрес конкретного запуска. После неё у нас координата внутри app.exe. Теперь мы можем сравнивать её с PE-таблицами app.exe: section table, Exception Directory, RUNTIME_FUNCTION entries.

Если бы RIP не попал в диапазон app.exe, нужно было бы искать другой модуль. Если бы он не попал ни в один обычный модуль, это мог бы быть heap code, JIT, shellcode, manually mapped image, trampoline, freed memory или повреждённый контекст. Это уже другой класс анализа. Но базовый путь всегда начинается с вопроса: чей это адрес?

7.1.9. Exception Directory: где начинается путь к .pdata

Теперь у нас есть RVA внутри модуля. Следующий вопрос: где в этом модуле лежит таблица функций для exception/unwind? Формальная точка входа - не имя секции .pdata, а Exception Directory в Data Directory.

Data Directory - каталог данных в Optional Header PE. Это массив записей, где каждая запись указывает на специальную подсистему образа: imports, exports, resources, relocations, exceptions и так далее. Exception Directory - запись, которая для x64 ведёт к таблице RUNTIME_FUNCTION. В обычном PE32+ эта таблица физически лежит в секции .pdata, но анализатор не должен начинать с поиска имени .pdata. Имя секции может быть нестандартным. Directory - более формальный источник.

Exception Directory содержит VirtualAddress и Size. Поле VirtualAddress здесь фактически является RVA. Size - размер области таблицы. Если одна запись RUNTIME_FUNCTION занимает 12 байт, то количество записей можно грубо получить как Size / 12, если речь о стандартной x64 таблице function entries.

```
Exception Directory:
    VirtualAddress = RVA начала function table
    Size           = размер function table в байтах

Для x64 RUNTIME_FUNCTION:
    sizeof(RUNTIME_FUNCTION) = 12 bytes

entry_count = Size / 12
```

Здесь важно слово “грубо” не потому, что математика неправильная, а потому что хороший анализатор должен проверять валидность: делится ли Size на 12 без остатка, попадает ли RVA таблицы в mapped image, находится ли область в секции с разумными правами, не выходит ли таблица за границы секции, отсортированы ли записи. Но для первого ручного понимания достаточно: Exception Directory даёт начало массива RUNTIME_FUNCTION.

7.1.10. .pdata: таблица, где адрес превращается в диапазон функции

.pdata - procedure data, данные процедур. В контексте Windows x64 это обычно секция или область, где лежит массив RUNTIME_FUNCTION. Её задача - не хранить код и не хранить сам обработчик. Её задача - дать таблицу диапазонов функций. Каждый элемент говорит: такой-то диапазон кода описывается такой-то unwind-информацией.

Если очень упрощённо, .pdata - это индекс. Код функции лежит в .text. Подробное описание unwind лежит в .xdata. А .pdata связывает одно с другим: “код от BeginAddress до EndAddress имеет UNWIND_INFO по UnwindInfoAddress”.

```
RUNTIME_FUNCTION:
    BeginAddress       // RVA начала диапазона функции
    EndAddress         // RVA конца диапазона функции
    UnwindInfoAddress  // RVA структуры UNWIND_INFO
```

Почему это таблица диапазонов, а не таблица имён? Потому что во время исключения у Windows нет имени. У Windows есть ControlPc. Нужно быстро понять, в какой диапазон попадает этот ControlPc. Если найден диапазон BeginAddress <= ControlPcRva < EndAddress, значит найдена function entry для текущего кадра.

На практике .pdata обычно отсортирована по BeginAddress. Это позволяет искать запись быстро, например бинарным поиском, а не просматривать всю таблицу линейно. Но для ручного обучения можно представить линейный просмотр: берём наш RVA 0x123456 и ищем запись, чей диапазон его покрывает.

```
Условие попадания адреса в RUNTIME_FUNCTION:

BeginAddress <= ControlPcRva < EndAddress
```

Обрати внимание на знак меньше у EndAddress. Диапазон обычно полуоткрытый: начало включается, конец не включается. Это удобно, потому что конец одной функции может совпадать с началом следующей. Если адрес ровно равен EndAddress, он уже не принадлежит этой записи.

7.1.11. RUNTIME_FUNCTION: маленькая структура с огромным смыслом

RUNTIME_FUNCTION выглядит почти обидно маленькой структурой: три 32-битных поля. Но она держит на себе большую часть x64 unwind-модели. Это тот самый мост между “адресом выполнения” и “правилами восстановления кадра”.

BeginAddress - RVA начала диапазона. Это не обязательно красивое начало функции с понятным символом. Это начало диапазона кода, который данная запись описывает для unwind. В обычном случае оно совпадает с началом функции, но в сложных сценариях могут быть funclets, hot/cold splitting, chained info, compiler-generated pieces. Поэтому лучше говорить “диапазон функции или части функции”, если мы хотим быть технически осторожными.

EndAddress - RVA конца диапазона. Это граница, до которой запись действует. Если ControlPcRva меньше EndAddress и не меньше BeginAddress, запись подходит. Если ControlPcRva вышел за EndAddress, нужно искать другую запись.

UnwindInfoAddress - RVA unwind-информации. Это поле особенно часто неправильно понимают. Оно не указывает на обработчик. Оно не указывает на начало функции. Оно указывает на UNWIND_INFO. Уже внутри UNWIND_INFO есть флаги, unwind codes и, при наличии EHANDLER/UHANDLER, дополнительный handler RVA и handler data.

```
Не так:
    UnwindInfoAddress -> handler code

Правильно:
    UnwindInfoAddress -> UNWIND_INFO
        UNWIND_INFO -> unwind codes
```

```
UNWIND_INFO -> maybe handler RVA
handler RVA -> language-specific handler, например __C_specific_handler
```

Почему поля 32-битные, если PE32+ и x64? Потому что это RVA, а не полный 64-битный VA. RVA внутри PE image обычно укладывается в 32 бита, а реальный VA получается через ImageBase + RVA. Это ещё раз показывает, что “64-битный” не означает “каждое поле стало 64-битным”. В PE32+ много важных таблиц всё ещё используют относительные 32-битные координаты.

7.1.12. Ищем запись RUNTIME_FUNCTION для нашего примера

Вернёмся к примеру. У нас есть ControlPcRva:

```
ControlPcRva = 00123456h
```

Допустим, в .pdata есть несколько записей:

```
Entry 100:
    BeginAddress      = 00122000h
    EndAddress        = 00122A80h
    UnwindInfoAddress = 00310020h

Entry 101:
    BeginAddress      = 00123000h
    EndAddress        = 001236A0h
    UnwindInfoAddress = 00310140h

Entry 102:
    BeginAddress      = 001236A0h
    EndAddress        = 00124010h
    UnwindInfoAddress = 00310210h
```

Проверяем entry 100: 0x123456 больше 0x122000, но больше и чем 0x122A80, значит не подходит. Проверяем entry 101: 0x123456 больше или равно 0x123000 и меньше 0x1236A0. Значит, запись найдена. Entry 102 начинается ровно с 0x1236A0, наш адрес меньше этого начала, поэтому entry 102 не подходит.

```
00123000h <= 00123456h < 001236A0h

Да. Значит, текущий ControlPc описывает Entry 101.
```

Теперь у нас есть сразу три важные вещи. Во-первых, мы знаем диапазон текущей функции или unwind-диапазона: RVA 0x123000..0x1236A0. Во-вторых, мы можем вычислить offset внутри диапазона: 0x123456 - 0x123000 = 0x456. Это полезно, чтобы сказать “упали на +0x456 от начала функции”. В-третьих, мы знаем, где искать UNWIND_INFO: RVA 0x310140.

```
FunctionRange:
    Begin = 00123000h
    End   = 001236A0h

OffsetInsideFunction:
    00123456h - 00123000h = 456h

UNWIND_INFO RVA:
    00310140h
```

Это уже полноценный технический ответ на вопрос “где мы?” Даже если имени функции нет, мы знаем её диапазон и unwind-описание. Дальше можно переходить к .xdata. Но перед этим нужно понять несколько ловушек.

7.1.13. Что делать, если RUNTIME_FUNCTION не найден

Если RtlLookupFunctionEntry не находит запись для ControlPc, x64 unwind procedure рассматривает такой код как leaf function в смысле unwind. Leaf function - листовая функция. В простейшем случае это функция, которая не вызывает другие функции, не выделяет сложный стековый кадр и не сохраняет nonvolatile-регистры так, чтобы потребовалась таблица unwind. Для неё можно виртуально выполнить return: взять адрес возврата с вершины стека и увеличить RSP на 8.

```
Leaf-предположение для x64 unwind:

new RIP = *(uint64_t*)RSP
new RSP = RSP + 8
```

Но для реверсера это место опасное. “Запись не найдена” не всегда означает “всё нормально, это leaf”. Это может означать leaf function. А может означать, что адрес находится в dynamic code region, JIT-коде, shellcode, manually mapped image, повреждённом модуле, packed stub-е, эпилоге необычного кода или просто в области, для которой static parser не нашёл active function table.

Поэтому правильная формулировка finding-а должна быть осторожной. Не “функция leaf”, а “для данного ControlPc не найдена RUNTIME_FUNCTION в доступных function tables; стандартный unwinder будет применять leaf-like правило, если это обычный сценарий”. Затем нужно смотреть код: меняет ли он RSP? Сохраняет ли RBX/RBP/R12-R15? Делает ли calls? Если да, отсутствие записи подозрительно.

Вот почему будущий static не должен автоматически ругаться на каждую дырку в .pdata. Нужно сопоставлять отсутствие записи с машинным кодом. Маленький stub, который просто jmp rax, может не нуждаться в unwind entry. Большая функция с sub rsp, 200h и сохранением rbx без записи - уже совсем другое дело.

7.1.14. Почему нельзя искать функцию только по дизассемблеру

Дизассемблер пытается восстановить функции по паттернам: entry points, call targets, prologs, symbols, relocations, alignment, flow analysis. Это полезно, но не является тем же самым, что Windows x64 function table. Дизассемблер может показать функцию с одним началом и концом, а .pdata - другой диапазон. Иногда ошибка у дизассемблера. Иногда .pdata описывает не всю “человеческую функцию”, а funclet или cold part. Иногда код обфусцирован. Иногда функция начинается не с привычного push rbp или sub rsp.

RUNTIME_FUNCTION важен именно потому, что это не эвристика дизассемблера, а данные, которые Windows использует для unwind. Это не значит, что .pdata всегда “истиннее” кода. Метаданные тоже могут быть повреждены или специально искажены. Но это другой источник истины. Взрослый анализ сопоставляет оба слоя: что говорит код, что говорит .pdata, что говорит .xdata, что показывает runtime.

Если дизассемблер говорит, что функция начинается на RVA 0x122F90, а RUNTIME_FUNCTION начинается на 0x123000, это не повод сразу паниковать. Нужно спросить: может ли 0x122F90 быть thunk-ом? Может ли это быть padding или jump stub? Может ли function entry описывать только основное тело? Есть ли символы? Что показывает prolog? Где реально находится call target?

```
Зрелый вопрос:
    Почему disassembler function range и RUNTIME_FUNCTION range отличаются?
```

```
Незрелый вывод:
    Один из инструментов точно врёт.
```

7.1.15. Offset внутри функции: почему +0x456 часто полезнее абсолютного адреса

Когда мы нашли BeginAddress, можно вычислить offset внутри функции. Это простая операция: ControlPcRva - BeginAddress. В отчётах это часто пишется как function+0x456. Даже если имени функции нет, можно написать sub_123000+0x456 или range_123000+0x456.

Почему это полезно? Абсолютный VA меняется при ASLR. RVA стабильнее, но всё равно иногда человеку удобнее видеть смещение от начала функции. Если ты сравниваешь два запуска одной версии программы, exception может происходить в том же function offset, даже если модуль загрузился по другой базе. Если ты читаешь дизассемблер, offset помогает быстро найти место внутри функции.

```
ControlPcRva = 00123456h
BeginAddress  = 00123000h

Offset = 456h

Человеческая запись:
    app.exe + 0x123456
    function RVA 0x123000 + 0x456
```

Но есть ловушка. Offset имеет смысл только относительно правильного BeginAddress. Если ты взял начало функции из дизассемблера, а не из RUNTIME_FUNCTION, offset может отличаться. Иногда это нормально, иногда нет. Поэтому в серьёзном отчёте полезно указывать источник диапазона: “offset relative to RUNTIME_FUNCTION BeginAddress” или “offset relative to disassembler symbol”.

7.1.16. Section table: как понять, где лежит код и где лежит UNWIND_INFO

После нахождения RUNTIME_FUNCTION у нас есть несколько RVA: BeginAddress, EndAddress, UnwindInfoAddress. Чтобы статически прочитать байты из файла, нужно перевести их в file offsets. Для этого используется Section Table.

Section Table - таблица секций. Каждая section header говорит: секция начинается в памяти по VirtualAddress, имеет VirtualSize, а в файле её сырые данные лежат по PointerToRawData и SizeOfRawData. Чтобы перевести RVA в file offset, нужно найти секцию, чей виртуальный диапазон покрывает этот RVA.

```
Если:
    Section.VirtualAddress <= RVA < Section.VirtualAddress + Section.VirtualSize

то:
    delta = RVA - Section.VirtualAddress
    file_offset = Section.PointerToRawData + delta
```

Для BeginAddress и EndAddress обычно ожидается попадание в исполняемую секцию, часто .text. Для UnwindInfoAddress ожидается попадание в читаемую data-секцию, часто .xdata или .rdata. Для Exception Directory ожидается попадание в область .pdata. Но имена секций не абсолютны. Важнее права и содержимое: код должен находиться в executable области, unwind info - в readable области с корректной структурой.

Если UnwindInfoAddress указывает в executable writable секцию, это не автоматически malware, но вопрос. Если он указывает за пределы image, это серьёзная проблема. Если он указывает в середину .text, возможно, файл нестандартный или анализатор перепутал RVA/file offset. Поэтому каждый RVA нужно не только “прибавить к базе”, но и проверить на принадлежность валидной области.

7.1.17. UnwindInfoAddress: почему это следующий мост, а не конец анализа

Когда RUNTIME_FUNCTION найден, новичок иногда думает: “Всё, функция найдена”. Для границ функции - да, первый этап выполнен. Но для exception/unwind анализа это только мост. Главное поле теперь - UnwindInfoAddress. Оно ведёт к UNWIND_INFO, а UNWIND_INFO отвечает на вопрос: как функция построила кадр и есть ли у неё handler-related данные.

UNWIND_INFO содержит version, flags, prolog size, count of unwind codes, frame register/offset и массив UNWIND_CODE. Если flags говорят UNW_FLAG_EHANDLER или UNW_FLAG_UHANDLER, после выровненного массива unwind codes находится handler RVA и handler data. Если flags говорят UNW_FLAG_CHAININFO, хвост структуры интерпретируется иначе - как chained RUNTIME_FUNCTION-like запись. Значит, одно поле UnwindInfoAddress открывает целое дерево интерпретации.

Пока в 7.1 мы не будем подробно читать все UWOP_*, потому что это следующий слой. Но важно уже сейчас закрепить: путь от RIP к функции не заканчивается названием функции. Он заканчивается найденным UNWIND_INFO. Именно UNWIND_INFO позволит дальше понять prolog, stack allocation, saved nonvolatile registers, frame register, EHANDLER/UHANDLER и language-specific handler.

```
Маршрут 7.1:

RIP / ControlPc
 -> module
 -> ImageBase
 -> RVA
 -> Exception Directory
 -> .pdata
 -> RUNTIME_FUNCTION
 -> UnwindInfoAddress
 -> UNWIND_INFO
```

7.1.18. Как этот путь выглядит в псевдокоде

Теперь соберём маршрут в псевдокод. Это не копия Windows-кода, а учебная модель для будущего анализатора. Она показывает последовательность вопросов, которые нужно задать.

```
input:
    ControlPcVA
    list of loaded modules or parsed PE image

step 1: find module
    for each module:
        if module.ImageBase <= ControlPcVA < module.ImageBase + module.SizeOfImage:
            currentModule = module
            break

step 2: compute RVA
    ControlPcRva = ControlPcVA - currentModule.ImageBase
```

```
step 3: find Exception Directory
    exceptionDirRva  = OptionalHeader.DataDirectory[Exception].VirtualAddress
    exceptionDirSize = OptionalHeader.DataDirectory[Exception].Size

step 4: read RUNTIME_FUNCTION array
    entries = read entries from exceptionDirRva, count = exceptionDirSize / 12

step 5: find covering entry
    for entry in entries:
        if entry.BeginAddress <= ControlPcRva < entry.EndAddress:
            functionEntry = entry
            break

step 6: if not found
    treat as possible leaf / dynamic / invalid / suspicious

step 7: if found
    unwindInfoRva = functionEntry.UnwindInfoAddress
    read UNWIND_INFO at unwindInfoRva
```

В реальном инструменте сюда добавятся проверки: валидность PE headers, корректность Data Directory, перевод RVA в file offset, проверка размеров, сортировка .pdata, binary search, dynamic function tables, bounds checking, handling CHAININFO, support for images in memory. Но базовый скелет именно такой.

Важно, что каждый шаг меняет уровень абстракции. Сначала у нас runtime VA. Потом module. Потом image-relative RVA. Потом PE directory. Потом array entry. Потом unwind info. Если на любом шаге перепутать уровень, дальше всё будет неправильно.

7.1.19. Почему RtlLookupFunctionEntry делает именно этот смысловой шаг

RtlLookupFunctionEntry - системная функция, которая ищет в active function tables запись, соответствующую заданному PC value. То есть она автоматизирует часть маршрута: по ControlPc находит RUNTIME_FUNCTION и возвращает ImageBase модуля. Но чтобы понимать её вывод, нужно знать весь ручной путь.

Active function tables - активные таблицы функций. Это важный термин. Он означает, что Windows ищет не только в статической .pdata файла на диске. Для загруженных PE-модулей таблицы приходят из Exception Directory. Для dynamically generated code могут быть зарегистрированы dynamic function tables через RtlAddFunctionTable или callback-механизмы. Поэтому RtlLookupFunctionEntry работает с runtime-картой, а не просто с тем, что ты видишь в одном PE-файле.

Если RtlLookupFunctionEntry возвращает FunctionEntry, это значит: для данного ControlPc найдено табличное описание функции. Если возвращает NULL, это не всегда ошибка, но это важное состояние: unwinder может применить leaf assumption или анализ должен искать dynamic/nonstandard explanation. Для реверсера важно не просто вызвать API, а понять, почему результат такой.

```
Смысл RtlLookupFunctionEntry:

ControlPc VA
 -> поиск в active function tables
 -> RUNTIME_FUNCTION* или NULL
 -> ImageBase модуля, если найдено
```

Заметь: RtlLookupFunctionEntry возвращает ImageBase. Это не случайный удобный бонус. Без ImageBase нельзя правильно интерпретировать RVA из RUNTIME_FUNCTION. FunctionEntry говорит BeginAddress/EndAddress/UnwindInfoAddress как image-relative значения. ImageBase нужен, чтобы превратить их в живые VA.

7.1.20. Static parser и runtime API: почему результаты могут отличаться

Если ты пишешь статический parser, ты читаешь PE-файл на диске. Ты находишь Exception Directory, .pdata и RUNTIME_FUNCTION entries. Если ты вызываешь RtlLookupFunctionEntry в живом процессе, Windows смотрит active function tables. Обычно для обычного PE-модуля результаты совпадают по смыслу. Но совпадение не гарантировано во всех случаях.

Во-первых, образ мог быть relocated: VA отличается, но RVA те же. Это нормально. Во-вторых, код может быть dynamically generated и зарегистрирован отдельно. Статический файл об этом не знает. В-третьих, manual mapping может загрузить PE не через обычный loader, и статический путь по списку модулей может не видеть такой образ как normal module. В-четвёртых, packer может распаковать код в память, где static .pdata исходного файла уже не описывает исполняемую область.

Поэтому future static должен различать режимы: file mode и process mode. File mode отвечает: “что заявлено в PE-файле?” Process mode отвечает: “что реально зарегистрировано и загружено в процессе?” Для обычного приложения они близки. Для защит, JIT и packers они могут расходиться.

```
File mode:
    PE on disk -> DataDirectory -> .pdata -> .xdata

Process mode:
    loaded modules + dynamic function tables -> active function tables
```

Это разделение поможет не делать ложных выводов. Если статический анализатор не нашёл function entry для runtime address, возможно, потому что этот address вообще не из анализируемого файла. Если runtime lookup нашёл entry для dynamic code, static PE parser может его не увидеть. В отчёте нужно честно писать, какой режим анализа использован.

7.1.21. Типичная ошибка номер один: перепутать VA и RVA

Самая частая ошибка в начале - взять VA из отладчика и сравнивать его напрямую с BeginAddress из RUNTIME_FUNCTION. Например, RIP = 0x00007FF7A0123456, а BeginAddress = 0x00123000. Новичок говорит: “не попадает”. Конечно не попадает, потому что сравниваются разные системы координат. BeginAddress - RVA. RIP - VA. Нужно сначала вычесть ImageBase.

```
Неправильно:
    00007FF7A0123456h сравнивают с 00123000h..001236A0h

Правильно:
    ControlPcRva = 00007FF7A0123456h - ImageBase
    потом сравнивают ControlPcRva с BeginAddress..EndAddress
```

Эта ошибка особенно вредна потому, что иногда числа могут случайно выглядеть “почти похожими” в маленьких тестовых файлах, и человек не замечает проблемы. Но в реальном x64 процессе VA почти всегда большой, а RVA относительно маленький. Если в сравнении участвуют разные масштабы чисел, остановись и спроси: это VA, RVA или file offset?

7.1.22. Типичная ошибка номер два: перепутать RVA и file offset

Вторая ошибка возникает при статическом чтении файла. Анализатор видит UnwindInfoAddress = 0x310140 и пытается читать файл по offset 0x310140. Иногда это может случайно попасть в похожую область, но чаще это неправильно. UnwindInfoAddress - RVA. Чтобы прочитать UNWIND_INFO из файла, нужно перевести RVA через section table.

Правильная логика: найти секцию, где VirtualAddress <= 0x310140 < VirtualAddress + VirtualSize. Допустим, это .xdata с VirtualAddress 0x300000 и PointerToRawData 0x1F0000. Тогда delta = 0x310140 - 0x300000 = 0x10140. File offset = 0x1F0000 + 0x10140 = 0x200140.

```
UnwindInfoAddress RVA = 00310140h

.xdata:
    VirtualAddress   = 00300000h
    PointerToRawData = 001F0000h

Delta:
    00310140h - 00300000h = 00010140h

File offset:
    001F0000h + 00010140h = 00200140h
```

В отчёте полезно показывать обе координаты: UNWIND_INFO RVA и file offset. Тогда человек сможет проверить и в дизассемблере, и в hex-редакторе.

7.1.23. Типичная ошибка номер три: забыть про модуль

Третья ошибка - вычислить RVA от неправильной базы. Например, исключение произошло в ucrtbase.dll, а человек вычитает ImageBase основного EXE. Получается число, которое может выглядеть как “RVA”, но не имеет смысла. Потом он ищет это число в .pdata EXE и удивляется, что ничего не находится.

Правило простое: сначала найти module, потом ImageBase. Нельзя выбирать базу по привычке. В stack trace часто участвуют ntdll.dll, kernelbase.dll, ucrtbase.dll, vcruntime, user32.dll и другие DLL. Каждая DLL имеет собственную .pdata и .xdata. Если ControlPc внутри DLL, надо читать function table этой DLL, а не основного EXE.

```
Неправильно:
    RVA = RIP - mainExe.ImageBase   // если RIP находится в ntdll.dll

Правильно:
    module = module that contains RIP
    RVA = RIP - module.ImageBase
    .pdata = module.ExceptionDirectory
```

7.1.24. Типичная ошибка номер четыре: доверять имени секции сильнее, чем Data Directory

Многие учебные объяснения говорят: “.pdata содержит RUNTIME_FUNCTION”. Это обычно верно, но формально входом является Exception Directory. Если анализатор ищет секцию строго по имени .pdata, он может промахнуться в нестандартном PE. Линкер или packer может назвать секцию иначе, объединить данные, изменить порядок секций. Loader и Windows используют directory RVA, а не красивое имя.

Правильный подход: получить RVA и Size Exception Directory из Optional Header Data Directory. Потом определить, в какую секцию попадает этот RVA. Если секция называется .pdata - отлично. Если иначе - не страшно, но нужно отметить. Если directory указывает в странную writable/executable область - это finding или как минимум notable observation.

7.1.25. Типичная ошибка номер пять: считать отсутствие entry доказательством обфускации

Если для адреса нет RUNTIME_FUNCTION, это не автоматически защита. На x64 leaf functions могут обходиться без полноценной записи. Маленькие thunks и stubs тоже могут не иметь отдельной записи. Динамический код может иметь function table, но не в статическом файле. Поэтому отсутствие entry - это повод задать вопросы, а не сразу делать громкий вывод.

Правильная проверка такая: посмотреть код вокруг ControlPc. Есть ли пролог, который меняет RSP? Есть ли сохранение nonvolatile-регистров? Есть ли calls? Есть ли handler-related логика? Если код действительно простой и leaf-like, отсутствие записи нормально. Если код явно non-leaf, отсутствие записи подозрительно. Если адрес вне PE-модуля, ищи dynamic function tables или manual mapping.

```
Нет RUNTIME_FUNCTION:
    - может быть leaf function
    - может быть thunk/stub
    - может быть dynamic code
    - может быть manual mapped image
    - может быть corrupted/obfuscated code
    - может быть ошибка анализа координат

Вывод делается только после проверки контекста.
```

7.1.26. Что должен вывести будущий static на этом этапе

Теперь полезно представить, как будущий static должен формулировать результат пути от RIP к функции. Он не должен просто писать “function found”. Нужно показать цепочку доказательств. Пользователь должен видеть, как адрес был переведён и почему выбрана именно эта запись.

```
Example finding / trace:

Input ControlPc:
    VA 00007FF7A0123456h

Containing module:
    app.exe
    ImageBase 00007FF7A0000000h
    SizeOfImage 0000000000250000h

Computed RVA:
    00123456h

Exception Directory:
    RVA 002F0000h
    Size 00012000h
    Entries 6144

Matched RUNTIME_FUNCTION:
    Index 101
    BeginAddress 00123000h
    EndAddress   001236A0h
```

```
    Offset        +456h
    UnwindInfo    00310140h

UNWIND_INFO location:
    RVA 00310140h
    file offset 00200140h

Status:
    matched, ready for UNWIND_INFO parsing
```

Такой вывод хорош тем, что он проверяемый. Если человек не согласен, он может проверить каждый шаг: принадлежность модулю, вычитание ImageBase, Exception Directory, entry index, диапазон, RVA-to-file-offset. Это и есть инженерный стиль: не просто вердикт, а прозрачная цепочка.

7.1.27. Почему эта ручная модель нужна даже если есть WinDbg

Можно спросить: зачем всё это руками, если WinDbg и так показывает stack trace? Ответ: потому что reverse engineering начинается там, где готовому выводу нужно перестать слепо верить. WinDbg использует unwind data, symbols, runtime state и свои механизмы. Обычно он прав. Но если стек повреждён, метаданные нестандартны, код динамический, модуль вручную загружен, packer ломает ожидания или symbols misleading, тебе нужно понимать, что именно сделал отладчик.

Когда WinDbg показывает функцию, спроси: откуда это имя? Из PDB? Export? Symbol server? Heuristic? Когда показывает stack trace, спроси: использован ли unwind? Не было ли warning-ов? Не оборвалась ли цепочка? Когда показывает exception address, спроси: это VA какого module? Какой RVA? Есть ли RUNTIME_FUNCTION? Соответствует ли UNWIND_INFO прологу?

Знание ручного пути превращает тебя из пользователя отладчика в проверяющего механизм. Это особенно важно для твоей книги, потому что она готовит фундамент не только для чтения чужих текстов, а для собственного анализатора. Инструмент, который ты построишь, должен делать то же самое: не просто повторять вывод WinDbg, а проверять договор между PE, кодом и unwind metadata.

7.1.28. Как этот раздел связывается с предыдущей главой

В главе 6 мы говорили о маршруте исключения: exception dispatcher получает событие, debugger может получить first chance, VEH может увидеть событие до unwind, frame-based SEH ищет обработчики по кадрам, top-level filter получает последний шанс. Но в центре frame-based части всегда был вопрос: как перейти от текущего адреса к текущему кадру?

Теперь мы начали отвечать на этот вопрос практически. Чтобы frame-based search работал, система должна по ControlPc найти RUNTIME_FUNCTION. Чтобы найти RUNTIME_FUNCTION, нужно знать module, ImageBase и RVA. Чтобы прочитать RUNTIME_FUNCTION array, нужен Exception Directory. Чтобы получить unwind rules, нужен UnwindInfoAddress. Все прежние термины теперь собираются в рабочий маршрут.

VEH может быть процессным и не frame-based, но даже VEH callback часто смотрит ExceptionRecord->ExceptionAddress или ContextRecord->Rip. Top-level filter тоже получает EXCEPTION_POINTERS с ExceptionRecord и ContextRecord. Debugger exception event тоже сообщает адрес исключения. То есть даже там, где ещё не начался frame-based unwind, путь от адреса к модулю и RVA остаётся полезным. Это базовый навык всего exception-aware reverse

engineering.

7.1.29. Мини-упражнение для закрепления

Закрепим модель маленьким упражнением. Пусть есть данные:

```
RIP = 0000000140054321h

module app.exe:
    ImageBase   = 0000000140000000h
    SizeOfImage = 0000000000100000h

Exception Directory:
    RVA 000C0000h
    Size 00000300h

RUNTIME_FUNCTION entries:
    #0: Begin 00010000h End 00010220h Unwind 000D0000h
    #1: Begin 00054000h End 00054680h Unwind 000D0120h
    #2: Begin 00054680h End 00054A10h Unwind 000D0190h
```

Первое: проверяем, что RIP внутри module. Диапазон app.exe: 0x140000000..0x140100000. RIP 0x140054321 попадает. Второе: вычисляем RVA: 0x140054321 - 0x140000000 = 0x54321. Третье: ищем RUNTIME_FUNCTION. Entry #1 покрывает 0x54321, потому что 0x54000 <= 0x54321 < 0x54680. Значит, функция или диапазон начинается на RVA 0x54000, offset внутри диапазона равен 0x321, а UNWIND_INFO лежит по RVA 0xD0120.

```
Результат:
    ControlPcRva = 00054321h
    Matched entry = #1
    Function range = 00054000h..00054680h
    Offset = +321h
    UNWIND_INFO RVA = 000D0120h
```

Если ты можешь сделать это руками без паники, ты уже понимаешь основу .pdata-анализа. Дальше останется научиться читать саму запись .pdata аккуратно и массово: сортировка, границы, пересечения, странные диапазоны, chained info, связь с .text и .xdata. Именно это будет следующим шагом.

7.1.30. Итог раздела

Путь от RIP к функции - это не мелкая арифметика, а фундаментальная операция Windows x64 reverse engineering. RIP или ControlPc даёт нам живой VA. Сначала нужно определить module, которому принадлежит этот VA. Затем взять фактический ImageBase этого module и вычислить RVA. Потом через Optional Header Data Directory найти Exception Directory. Exception Directory ведёт к массиву RUNTIME_FUNCTION, обычно в .pdata. Среди записей нужно найти диапазон BeginAddress <= ControlPcRva < EndAddress. Найденная запись даёт UnwindInfoAddress, то есть вход в .xdata и UNWIND_INFO.

На каждом шаге нужно помнить систему координат. VA - адрес в живом процессе. RVA - смещение внутри image. File offset - координата в файле. ImageBase связывает VA и RVA. Section Table связывает RVA и file offset. RUNTIME_FUNCTION использует RVA. Отладчик показывает VA. Hex-редактор показывает file offset. Если эти миры смешать, анализ сломается.

Главная практическая мысль: функция для Windows x64 exception/unwind - это не прежде всего имя, а диапазон в function table и связанная unwind-информация. Имя может помочь человеку. Но механизм держится на RUNTIME_FUNCTION и UNWIND_INFO. Поэтому будущий static

должен показывать не только “нашёл функцию”, а всю проверяемую цепочку: ControlPc VA, module, ImageBase, RVA, Exception Directory, matched RUNTIME_FUNCTION, offset внутри диапазона, UnwindInfoAddress и статус готовности к разбору UNWIND_INFO.

Теперь, когда ручной маршрут от адреса к записи функции понятен, можно переходить к следующему разделу: как читать .pdata не на одном примере, а как настоящую таблицу. Там мы разберём размер записи, порядок, сортировку, границы, пересечения, пустые directories, странные секции, leaf gaps и то, как из массива RUNTIME_FUNCTION делать не просто список чисел, а карту функций PE32+ образа.

Техническая база раздела 7.1

Раздел опирается на официальную документацию Microsoft по PE Format, где PE/COFF описывает executable image, sections, RVA и data directories; на документацию по x64 exception handling, где RUNTIME_FUNCTION, .pdata, .xdata и UNWIND_INFO описаны как основа exception handling и debugger support; на документацию RtlLookupFunctionEntry, где функция определяется как поиск записи в active function tables по PC value; и на документацию RtlVirtualUnwind, где показана связь FunctionEntry, ImageBase, HandlerType и восстановления caller context. Текст раздела написан в учебной форме и связывает эти источники с ручной моделью reverse engineering.

7.2. Как читать .pdata руками: от Exception Directory к карте функций

В предыдущем разделе мы научились делать первый ручной переход: взять конкретный адрес выполнения, то есть RIP или ControlPc, определить модуль, вычесть ImageBase, получить RVA и понять, что дальше нужно искать запись RUNTIME_FUNCTION. Это был путь от одной живой точки выполнения к одному кандидату на функцию. Теперь мы расширяем взгляд. Мы больше не смотрим на один адрес как на случайную цифру из отладчика. Мы начинаем читать .pdata как таблицу, как карту, как слой PE-образа, который Windows использует для x64 unwinding и exception dispatching.

Здесь важно не сорваться в сухую механику: “запись 12 байт, три DWORD, идём циклом”. Это правда, но этого мало. Если оставить только такую инструкцию, .pdata останется в голове как скучный массив чисел. А нам нужен фундамент. Нужно понять, почему эта таблица вообще существует, почему она устроена именно так, почему она хранит не имена функций, а диапазоны адресов, почему в ней лежит не сам handler, а адрес unwind-информации, почему записи должны быть отсортированы, почему пустоты между диапазонами не всегда ошибка и почему реверсер не должен воспринимать .pdata как абсолютный список всех функций программы.

.pdata нужно читать как индекс. Индекс - это не сама книга, а способ быстро найти нужное место в книге. Код функции лежит в .text или в другой исполняемой области. Подробное описание раскрутки лежит в .xdata или связанной области. А .pdata стоит между ними и отвечает на очень конкретный вопрос: если у меня есть RVA инструкции, какая запись функции покрывает этот адрес и куда идти за правилами unwind?

Поэтому главный смысл раздела такой: .pdata - это не “секция с обработчиками исключений”. Это function table, таблица функций, которая связывает диапазоны кода с unwind-информацией. На Windows x64 именно эта связь позволяет Windows перейти от конкретного RIP к формальному описанию текущего кадра.

7.2.1. Почему .pdata надо читать не как список функций, а как карту диапазонов

Начнём с самого опасного упрощения: “.pdata хранит функции”. В разговоре так иногда говорят, но технически это неточно. .pdata не хранит машинный код функций. Она не хранит исходные имена функций. Она не хранит параметры функций. Она не хранит локальные переменные. Она не хранит красивую границу “вот функция foo, вот функция bar” в человеческом смысле.

.pdata хранит записи, которые описывают диапазоны адресов кода. Каждая запись говорит: начиная с такого RVA и до такого RVA действует вот это unwind-описание. В нормальном PE32+ образе такая запись соответствует функции или части функции, которую Windows должна уметь раскрутить. Но слово “функция” здесь нужно понимать не как имя из C-кода, а как диапазон машинного кода, для которого существует один рецепт unwinding.

Диапазон - это промежуток адресов. Если запись говорит BeginAddress = 0x1200, EndAddress = 0x1280, это означает, что адреса от 0x1200 до 0x127F включительно относятся к этому диапазону, а 0x1280 уже не относится. Такой диапазон называют полуоткрытым: начало включается, конец не включается.

Полуоткрытый диапазон удобен потому, что соседние функции могут стыковаться без пересечения:

```
Function A: [0x1200, 0x1280)
Function B: [0x1280, 0x1300)
```

Адрес 0x1280 принадлежит B, а не A. Нет двойного владения одним адресом. Для таблиц, поиска и сортировки это очень удобно. Если мыслить не полуоткрытыми диапазонами, а “начало и последний байт”, легко получить путаницу на границах.

Когда Windows получает ControlPc, она ищет запись, для которой выполняется условие:

```
BeginAddress <= ControlRva < EndAddress
```

Здесь ControlRva - это не VA из отладчика, а RVA внутри модуля. То есть перед сравнением нужно перейти из системы координат живого процесса в систему координат PE-образа:

```
ControlRva = ControlPc - ImageBase
```

Эта формула выглядит простой, но она является мостом между runtime и file/image metadata. Пока поток выполняется, процессор видит виртуальный адрес. Пока .pdata хранит записи, она использует image-relative addresses. Поэтому реверсер постоянно переводит адрес из одного мира в другой.

7.2.2. Exception Directory: почему вход в .pdata формально находится не по имени секции

В обычном PE32+ файле .pdata часто видна как отдельная секция. Дизассемблер или PE-viewer может показать тебе секции: .text, .rdata, .data, .pdata, .xdata, .reloc. Новичок естественно думает: “значит, чтобы найти function table, надо найти секцию с именем .pdata”. Иногда это сработает. Но как фундаментальная модель это слабый путь.

Формально Windows ориентируется не на красивое имя секции, а на Exception Directory из Data Directory. Data Directory - это массив входов в специальные подсистемы PE: imports, exports, resources, relocations, TLS, debug data, exception table и другие. У каждой записи есть VirtualAddress и Size. В случае Exception Directory этот VirtualAddress ведёт к таблице записей RUNTIME_FUNCTION для x64.

Почему это важно? Потому что имя секции - это в значительной степени соглашение. Линкер обычно называет область .pdata, но нестандартный инструмент, packer, dynamic , manual builder или специально модифицированный файл может положить данные exception directory в секцию с другим именем. Данные могут лежать в .rdata, в объединённой секции, в секции со странным именем или рядом с другими read-only

структурами. Если анализатор ищет только имя .pdata, он может пропустить настоящую таблицу. Если он идёт через Optional Header -> Data Directory -> Exception Directory, он следует формальному PE-контракту.

Слово directory в Exception Directory не означает “папка” в файловой системе. Это каталог в смысле таблицы входов: запись, которая говорит, где в образе лежит специальная структура. Exception Directory - это входная дверь к function table. A .pdata - типичное место, где эта таблица физически лежит.

Правильный путь выглядит так:

```
PE Optional Header
    -> DataDirectory[Exception]
        -> VirtualAddress = RVA таблицы функций
        -> Size = размер таблицы в байтах
            -> массив RUNTIME_FUNCTION
```

Здесь появляется важная дисциплина: имя секции помогает человеку, но Data Directory задаёт формальный маршрут. Хороший реверсер использует оба слоя. Он смотрит: Exception Directory указывает сюда; это место попадает в такую-то секцию; секция называется .pdata или иначе; права секции такие-то; размер directory такой-то. Чем больше слоёв согласованы, тем выше доверие.

7.2.3. RUNTIME_FUNCTION: маленькая структура с большим смыслом

Запись RUNTIME_FUNCTION на x64 кажется почти смешно маленькой. В классическом PE32+ варианте это три 32-битных поля:

```
typedef struct _RUNTIME_FUNCTION {
    DWORD BeginAddress;
    DWORD EndAddress;
    DWORD UnwindInfoAddress;
} RUNTIME_FUNCTION;
```

DWORD - это 32-битное беззнаковое значение. Слово пришло из Windows-типов: double word, двойное слово. В старых архитектурных обозначениях word часто был 16 бит, double word - 32 бита. В Windows API DWORD обычно означает unsigned 32-bit integer.

На первый взгляд странно: мы говорим про x64, 64-битный код, 64-битные адреса, регистры RAX/RSP/RIP, а поля RUNTIME_FUNCTION всего 32-битные. Но это не ошибка. Эти поля являются image-relative, то есть относительными адресами внутри образа. Они хранят RVA, а не полный VA. RVA в PE обычно 32-битный, потому что он является смещением внутри образа, а не произвольным 64-битным адресом процесса.

Реальный VA получается прибавлением ImageBase:

```
BeginVA      = ImageBase + BeginAddress
EndVA        = ImageBase + EndAddress
UnwindInfoVA = ImageBase + UnwindInfoAddress
```

Эта особенность одновременно экономит место и делает таблицу независимой от конкретной базы загрузки. Если ASLR загрузил модуль по другому адресу, сами RVA в .pdata остаются теми же. Меняется только ImageBase. Именно поэтому RUNTIME_FUNCTION может быть компактной 12-байтной записью даже для x64.

Теперь разберём поля не как список, а как роли.

BeginAddress - RVA начала диапазона кода. Это точка, с которой начинается область, описываемая записью. Обычно это начало функции в машинном смысле: адрес пролога или entry point функции. Но в сложных случаях это может быть не “красивая C-функция”, а диапазон, который нужно понимать через unwind-контракт.

EndAddress - RVA конца диапазона. Это не адрес последней инструкции. Это граница “после диапазона”. Поэтому адрес принадлежит записи, если он меньше EndAddress. Ошибка на один байт на границе функции может привести к тому, что анализатор выберет соседнюю запись или не выберет никакую.

UnwindInfoAddress - RVA unwind-информации. Это поле особенно часто понимают неверно. Оно не указывает на обработчик исключения. Оно не указывает на __except-блок. Оно не указывает на C++ catch. Оно указывает на структуру UNWIND_INFO или на связанные unwind-данные, где уже описано, как функция строит и разбирает кадр, есть ли handler flags, и где лежат дополнительные сведения.

То есть RUNTIME_FUNCTION отвечает не на вопрос “что делать с исключением?”, а на вопрос “где взять описание этой функции для unwinding?”. Решение по исключению появится позже, когда Windows прочитает UNWIND_INFO, увидит flags, возможно вызовет language-specific handler, возможно прочитает handler data. RUNTIME_FUNCTION - это указатель на дверь, а не всё помещение.

7.2.4. Почему запись 12 байт, и почему размер directory должен читаться осторожно

Если каждая RUNTIME_FUNCTION запись состоит из трёх DWORD, то обычный размер записи равен 12 байтам. Значит, если Exception Directory Size равен 0x120, можно ожидать:

```
0x120 / 12 = 0x18 = 24 записи
```

Но хороший анализатор не должен просто молча делить и радоваться. Он должен проверять остаток. Если Size не кратен 12, это важный сигнал. Может быть, файл повреждён. Может быть, directory size округлён или странно задан. Может быть, parser смотрит не туда. Может быть, перед нами нестандартный или специально запутанный образ.

Здесь важно не делать моментальный вывод “вредонос”. Для reverse engineering вообще вредно прыгать от “странно” к “malware”. Правильнее формулировать finding: Exception Directory Size не кратен размеру RUNTIME_FUNCTION, поэтому последние байты не образуют полную запись; доверие к таблице снижено; нужно проверить секцию,

raw/virtual sizes, корректность Data Directory и поведение loader-а.

Сам размер directory также нужно соотносить с секцией, в которую он попадает. Если Exception Directory начинается внутри .pdata, но Size выходит далеко за пределы секции или указывает в область с неподходящими правами, это повод задуматься. Возможно, PE всё ещё загружается, но статический анализ должен показать, что таблица выходит за ожидаемые границы.

Минимальная проверка для static -а может звучать так:

```
1. DataDirectory[Exception].VirtualAddress != 0?
2. DataDirectory[Exception].Size != 0?
3. RVA попадает в существующую секцию?
4. Size помещается в виртуальный диапазон секции?
5. Size кратен sizeof(RUNTIME_FUNCTION) = 12?
6. Каждая запись имеет BeginAddress < EndAddress?
7. UnwindInfoAddress указывает в допустимую область image?
```

Но в книге мы не хотим превращать это в чеклист без смысла. Каждый пункт - это проверка договора. PE говорит: “тут лежит таблица функций”. Реверсер отвечает: “хорошо, но таблица должна иметь форму таблицы, а не случайного набора байтов”.

7.2.5. Сортировка .pdata: зачем Windows нужна упорядоченная таблица

В документации Microsoft для x64 exception handling указывается, что записи RUNTIME_FUNCTION сортируются и помещаются в .pdata PE32+ образа. Это не декоративное требование. Сортировка нужна, чтобы быстро искать запись по адресу. Если таблица отсортирована по BeginAddress, можно применять бинарный поиск, а не линейно просматривать тысячи записей.

Бинарный поиск - это алгоритм поиска в отсортированном массиве. Он не проверяет каждую запись подряд. Он смотрит в середину, решает, идти левее или правее, потом снова делит диапазон. Для больших таблиц это намного быстрее. Во время exception handling скорость важна: исключение - уже необычная ситуация, но механизм должен работать формально и эффективно.

Представь модуль с десятками тысяч функций. Если при каждом исключении или stack walk нужно просматривать всю .pdata от начала до конца, это дорого. Отсортированная таблица превращает поиск в задачу “найти диапазон, покрывающий RVA”.

Условная таблица:

```
Begin   End     Unwind
1000    1080    5000
1080    10F0    5020
1200    1290    5040
1300    1380    5060
```

Если ControlRva = 0x1250, запись [0x1200, 0x1290) подходит. Если ControlRva = 0x1100, подходящей записи нет: между 0x10F0 и 0x1200 есть gap, то есть промежуток.

Gap - это разрыв между диапазонами. Он не всегда ошибка. Там может быть alignment padding, маленькая leaf-функция без записи, данные внутри исполняемой секции, jump table, hot/cold layout, dead space или нестандартный код. Но gap требует интерпретации. Он говорит: по формальной .pdata для этого RVA нет non-leaf unwind-записи.

Если таблица не отсортирована, это серьёзный признак проблемы. Windows может ожидать сортировку для корректного поиска. Некоторые инструменты могут вести себя по-разному: один parser просто покажет записи в порядке файла, другой попытается отсортировать для анализа, третий сообщит об ошибке. Но сам факт несортированности нарушает обычный x64 PE-контракт.

Для static -а сортировка - не мелкая эстетика. Это отдельная проверка: BeginAddress текущей записи должен быть не меньше BeginAddress предыдущей. Если диапазоны ещё и пересекаются, возникает вопрос, какая запись должна считаться владельцем адреса.

7.2.6. Пересечения диапазонов: почему это опаснее, чем просто “странно”

Пересечение диапазонов означает, что два RUNTIME_FUNCTION claim-ят, то есть заявляют, один и тот же адрес кода. Например:

```
Entry A: [0x2000, 0x2100)
Entry B: [0x2080, 0x2180)
```

Адрес 0x2090 попадает и в A, и в B. Для человека это может выглядеть как “ну ладно, две записи”. Для unwind-механизма это проблема: по одному ControlRva должна находиться одна корректная запись. Если возможны две разные unwind-информации, возникает неоднозначность. Какой пролог отменять? Какие nonvolatile-регистры восстанавливать? Где брать return address? Есть ли handler у текущего кадра?

Иногда в сложных бинарях могут встречаться ситуации, которые внешне похожи на пересечения, но объясняются chained unwind info, split functions, cold/hot regions или особенностями линкера. Поэтому опять нельзя кричать “ошибка” без анализа. Но базовое правило такое: обычная .pdata должна задавать не хаотичное облако диапазонов, а непротиворечивую карту.

Нужно различать три случая.

Первый случай - нормальное соседство:

```
A: [0x2000, 0x2100)
B: [0x2100, 0x2200)
```

Здесь всё чисто. EndAddress A равен BeginAddress B. Адрес 0x2100 принадлежит B.

Второй случай - gap:

```
A: [0x2000, 0x2100)
```

```
B: [0x2180, 0x2200)
```

Между 0x2100 и 0x2180 нет записи. Это может быть нормально или подозрительно в зависимости от содержимого.

Третий случай - overlap:

```
A: [0x2000, 0x2150)
B: [0x2100, 0x2200)
```

Адреса 0x2100-0x214F покрыты двумя записями. Это уже сильная аномалия, если нет специального объяснения.

Для реверсера пересечение опасно тем, что оно ломает доверие к функции как к единице анализа. Если один адрес может иметь два рецепта раскрутки, значит либо parser неверно понял таблицу, либо бинарь нестандартный, либо данные повреждены, либо используется трюк.

7.2.7. Почему .pdata не обязана покрывать каждый байт .text

Новичок часто ожидает, что если .text - это кодовая секция, то .pdata должна покрывать всю .text от начала до конца. Это неверно. .pdata покрывает не “все байты кода”, а те диапазоны, для которых нужна function table entry в x64 unwind-модели.

В .text могут быть разные вещи: обычные функции, маленькие leaf-функции, alignment padding, jump tables, import thunks, hotpatch padding, куски runtime-a, unreachable code, данные, используемые как таблицы переходов, compiler-generated helpers, иногда даже намеренно запутанные вставки. Не всё это обязано иметь RUNTIME_FUNCTION.

Leaf function - листовая функция - может не иметь записи, если её можно виртуально раскрутить простым способом. В x64 unwind-процедуре отсутствие function entry для ControlPc обычно трактуется как leaf-сценарий: return address берётся с вершины стека, RSP увеличивается на 8. Это возможно только если функция не построила сложный кадр, который требует unwind-описания.

Но это правило нужно применять осторожно. Если в gap между .pdata диапазонами лежит код, который делает:

```
sub rsp, 80h
push rbx
call something
```

и при этом записи нет, это уже не похоже на безобидную leaf-функцию. Такая функция меняет стек и nonvolatile state, значит без unwind metadata корректная раскрутка может быть невозможна. В таком случае gap становится suspicious gap - подозрительным разрывом.

Если же в gap лежат только нули, int 3 padding, nop padding или короткий jmp thunk, отсутствие записи может быть нормальным. Поэтому .pdata нельзя оценивать отдельно от кода. Нужно связать таблицу с байтами .text.

Вот здесь возникает принцип всей главы 7: ручной анализ - это не чтение одной структуры. Это постоянное сопоставление слоёв:

```
.pdata говорит: здесь функция
.text показывает: здесь реальный код
.xdata говорит: как раскручивать кадр
дизассемблер показывает: какой пролог реально сгенерирован
отладчик показывает: как это повело себя во время исключения
```

Если слои совпадают, доверие растёт. Если расходятся, начинается расследование.

7.2.8. BeginAddress: не просто начало, а привязка unwind-контракта к коду

BeginAddress кажется самым простым полем. Но именно вокруг него часто возникают ошибки. BeginAddress - это RVA начала диапазона, описанного записью. Для обычной функции оно совпадает с адресом первой инструкции функции. Но реверсер должен понимать, что “начало функции” - не всегда очевидное понятие.

В исходном коде функция начинается с имени:

```
void process() {
    ...
}
```

В машинном коде имени может не быть. Функция начинается там, куда передаётся управление при вызове, где компилятор разместил пролог или оптимизированный entry sequence. Дизассемблер может угадать начало по call-target, символу, прологу, таблице функций, relocations или эвристикам. .pdata даёт ещё один источник: если BeginAddress указывает на RVA, это сильная подсказка, что с этой точки начинается диапазон, значимый для unwind.

Но сильная подсказка - не абсолютная истина. В обфусцированном коде, packed binary или hand-written assembly BeginAddress может указывать в место, которое выглядит странно. Иногда дизассемблер может неправильно дизассемблировать байты до этого места или после него. Иногда у функции есть несколько входов. Иногда hot/cold splitting делает код функции физически разнесённым.

Поэтому BeginAddress нужно читать как заявление: “для Windows unwind этот диапазон начинается здесь”. Это заявление надо сверить с кодом. Если по BeginAddress действительно начинается разумный prolog или entry block, всё хорошо. Если BeginAddress попадает в середину другой инструкции, в данные, в неисполняемую секцию или за пределы image, это серьёзная проблема.

В практическом отчёте static -а это можно формулировать так:

```
RUNTIME_FUNCTION[123].BeginAddress = 0x401200
- RVA попадает в секцию .text
- секция имеет execute/read права
- дизассемблер видит допустимую инструкцию на границе
- адрес является целью call/jmp или соответствует функции из symbols
- начальные инструкции согласуются с UNWIND_INFO
```

Такое описание намного лучше, чем просто “BeginAddress OK”. Оно показывает цепочку доказательств.

7.2.9. EndAddress: почему конец диапазона не всегда конец “логической функции”

EndAddress задаёт верхнюю границу диапазона. Но не надо автоматически думать, что EndAddress - это “последний ret функции плюс один”. В обычном простом коде часто примерно так и будет: функция занимает непрерывный кусок кода, где в конце есть ret или jmp. Но оптимизирующий компилятор может делать сложнее.

Функция может иметь несколько return paths, то есть несколько путей выхода. В машинном коде это могут быть несколько эпилогов. Они могут быть внутри одного диапазона. Функция может делать tail call - хвостовой вызов, когда вместо call+ret управление передаётся другой функции через jmp. Функция может иметь cold block - редко исполняемый участок, вынесенный отдельно. Link-time optimization может менять расположение кода. Поэтому EndAddress - это граница диапазона, а не всегда интуитивная граница “тела функции” из C.

Для unwind главное, чтобы любой ControlPc внутри диапазона можно было интерпретировать с помощью связанной UNWIND_INFO. Если адрес находится внутри пролога, unwind применит правила частично. Если внутри тела - обычным образом. Если внутри эпилога - распознает epilog path. Но всё это работает только если диапазон корректно описывает область, к которой относится рецепт unwinding.

Ошибки EndAddress бывают разными.

EndAddress меньше или равен BeginAddress - очевидная структурная ошибка. Диапазон пустой или обратный.

EndAddress выходит за пределы исполняемой области - подозрительно, если нет объяснения. Например, диапазон начинается в .text, а заканчивается уже в .rdata. Может быть ошибка парсинга или намеренная путаница.

EndAddress обрезает функцию раньше реального кода - тогда часть инструкций может оказаться в gap без записи. Если исключение случится там, Windows может применить leaf-правило, хотя функция на самом деле non-leaf. Это может сломать stack walking.

EndAddress слишком широкий - диапазон может захватывать соседнюю функцию. Тогда для адреса соседней функции будет выбрана неправильная unwind-информация.

Поэтому EndAddress - не “просто число конца”. Это граница ответственности записи. Она говорит, за какие адреса данная запись отвечает перед unwinder-ом.

7.2.10. UnwindInfoAddress: почему .pdata не ведёт сразу в .xdata “по имени”

Третье поле, UnwindInfoAddress, является мостом от .pdata к .xdata. Но важно говорить осторожно: оно не обязано “вести в секцию с именем .xdata”. Оно ведёт к RVA unwind-информации. В обычном MSVC/PE32+ образе эта информация часто физически лежит в .xdata. Но формально важен адрес, а не имя секции.

Если UnwindInfoAddress указывает в .xdata, это ожидаемая картина. Если указывает в .rdata, это может быть нормально, если линкер объединил read-only данные. Если указывает в .text, это уже странно, но не обязательно мгновенно невозможно: может быть нестандартная компоновка. Если указывает вне image, это сильная ошибка. Если указывает в writable/executable секцию, это повод проверить, не packer ли перед нами.

UnwindInfoAddress нужно проверять в нескольких координатах.

Сначала это RVA. Нужно убедиться, что RVA попадает в виртуальный диапазон образа. Потом найти секцию, которая содержит этот RVA. Потом проверить, что секция читаемая. Потом перевести RVA в file offset, если мы читаем файл статически. Потом прочитать первый байт UNWIND_INFO и разобрать Version/Flags.

Здесь легко допустить классическую ошибку: взять UnwindInfoAddress = 0x5000 и открыть файл по offset 0x5000. Это будет правильно только случайно. Правильный путь:

```
UnwindInfoRva = 0x5000
найти секцию, где Section.VirtualAddress <= 0x5000 < Section.VirtualAddress + Section.VirtualSize
Delta = 0x5000 - Section.VirtualAddress
FileOffset = Section.PointerToRawData + Delta
```

То есть .pdata хранит image-coordinate, а file parser должен перевести её в raw-coordinate. Пока эта дисциплина не стала автоматической, любая работа с PE будет давать ложные выводы.

7.2.11. Почему .pdata и .xdata разделены: индекс и полезная нагрузка

Теперь стоит остановиться на архитектуре разделения. Почему RUNTIME_FUNCTION не хранит все unwind-коды прямо внутри .pdata? Почему не сделать одну большую запись: начало, конец, размер стека, сохранённые регистры, handler, scope table? Ответ в том, что .pdata должна быть компактным индексом.

Индекс должен быть однотипным. Если каждая запись имеет одинаковый размер, таблицу легко сортировать, легко искать, легко делить на записи, легко применять бинарный поиск. RUNTIME_FUNCTION ровно такая: 12 байт, три поля. Она говорит минимум, необходимый для первого перехода: где диапазон и где подробности.

А подробности переменного размера. Одна функция может вообще не сохранять регистры и иметь крошечный UNWIND_INFO. Другая может сохранять много nonvolatile-регистров, выделять большой стек, иметь frame register, exception handler, chained info, language-specific data. Если пытаться хранить всё в .pdata, записи стали бы разного размера или огромными. Это разрушило бы простую модель поиска.

Поэтому система разделена:

```
.pdata = быстрый индекс диапазонов
.xdata = подробная инструкция для unwinding и handler data
```

Такая архитектура встречается во многих форматах. Сначала маленькая таблица говорит, куда идти. Потом отдельная область хранит данные переменного размера. Это делает поиск быстрым, а формат гибким.

Для реверсера это означает: .pdata нельзя понимать без .xdata, но и .xdata нельзя найти без .pdata. Они образуют пару. .pdata отвечает на вопрос “какая функция?”, .xdata отвечает на вопрос “как её раскрутить и есть ли обработчик?”.

7.2.12. Exception Directory Size как граница доверия

Data Directory даёт не только VirtualAddress, но и Size. Size - это размер области exception table в байтах. Этот размер важен, потому что он задаёт границу массива RUNTIME_FUNCTION. Без Size parser не знает, где таблица заканчивается.

Представим, что Exception Directory начинается по RVA 0x3000. Если Size = 0x60, а запись 12 байт, значит ожидается 8 записей:

```
0x60 / 0x0C = 0x08
```

Parser не должен читать дальше просто потому, что после этих записей в секции ещё есть байты. После таблицы могут лежать другие данные. Если читать до конца секции, можно принять чужие данные за RUNTIME_FUNCTION. Это частая ошибка грубых парсеров.

С другой стороны, если Size слишком маленький, parser может увидеть только часть таблицы. Если Size указывает на ноль, формально Exception Directory отсутствует. Для x64 обычного non-leaf кода это может быть проблемой, но не всегда: маленький модуль, особый код, объект без обычных функций, динамические таблицы - возможны разные случаи.

Здесь появляется важная мысль о доверии к PE headers. Статический анализатор вынужден начать с заголовков. Но заголовки могут быть неправильными, неполными или намеренно обманчивыми. Поэтому каждый вывод должен иметь уровень уверенности. Если Exception Directory Size говорит одно, имя .pdata и содержимое секции другое, нужно не насильно выбирать одно, а показать противоречие.

Пример аккуратной формулировки:

```
Exception Directory указывает на RVA 0x3000 размером 0x60, что даёт 8 полных RUNTIME_FUNCTION записей.
```

Это зрелый подход. Он отделяет “что физически лежит рядом” от “что PE официально объявил таблицей”.

7.2.13. Почему .pdata помогает восстанавливать границы функций, но не заменяет дизассемблер

На Windows x64 .pdata часто очень помогает при разметке функций. Если дизассемблер сомневается, где начинается функция, BeginAddress из RUNTIME_FUNCTION может дать сильный сигнал. Если символов нет, .pdata всё равно даёт диапазоны. Это особенно полезно в stripped binaries, где имена удалены.

Stripped binary - бинарь без отладочных символов или с минимальным количеством имён. Слово stripped буквально означает “раздетый”, “очищенный”. В таком файле может не быть названий внутренних функций, но .pdata остаётся нужна для runtime. Поэтому даже без символов у нас может быть таблица диапазонов.

Но .pdata не является полноценным дизассемблером. Она не показывает control flow внутри функции. Она не показывает basic blocks. Она не показывает, какие функции вызываются. Она не показывает типы аргументов. Она не отличает “логическую функцию process_request” от compiler-generated helper. Она не знает исходных имён.

Кроме того, .pdata может не покрывать leaf-функции. Поэтому если анализатор строит список функций только из .pdata, он может пропустить часть кода. Дизассемблер, наоборот, может найти функции по call targets, но ошибиться в границах. Лучший подход - объединять источники:

```
.pdata ranges
call targets
symbols, если есть
exports
entry point
exception/unwind metadata
relocations
recursive disassembly
linear sweep
```

Каждый источник даёт гипотезы. .pdata - один из самых сильных источников для non-leaf функций x64, потому что он нужен самой Windows. Но даже сильный источник не является единственным.

Для будущего static -а полезно разделять:

```
FunctionRangeFromPdata     - диапазон из RUNTIME_FUNCTION
FunctionStartFromCallTarget - цель call/jmp
FunctionSymbol             - символическое имя, если есть
FunctionAnalyzerBoundary    - граница, восстановленная дизассемблером
```

Если все совпадают, отлично. Если .pdata говорит одно, а дизассемблер другое, это не повод сразу выбирать “кто прав”. Это finding: разные слои не согласованы.

7.2.14. Чтение одной записи руками: медленный пример

Теперь пройдём одну запись без спешки. Допустим, у нас есть модуль, загруженный по базе:

```
ImageBase = 0x00007FF700000000
```

Exception Directory указывает на таблицу по RVA 0x00030000. Внутри таблицы есть запись:

```
BeginAddress      = 0x00001200
EndAddress        = 0x000012A0
UnwindInfoAddress = 0x00028040
```

Первый шаг: понять диапазон функции в VA:

```
BeginVA = 0x00007FF700000000 + 0x1200 = 0x00007FF700001200
EndVA   = 0x00007FF700000000 + 0x12A0 = 0x00007FF7000012A0
```

Значит, запись покрывает адреса:

```
0x00007FF700001200 <= ControlPc < 0x00007FF7000012A0
```

Если исключение произошло при:

```
RIP = 0x00007FF700001256
```

то ControlRva:

```
0x00007FF700001256 - 0x00007FF700000000 = 0x1256
```

Проверяем:

```
0x1200 <= 0x1256 < 0x12A0
```

Да, запись подходит.

Второй шаг: понять, где лежит unwind-информация:

```
UnwindInfoVA = ImageBase + 0x28040
             = 0x00007FF700028040
```

Но если мы читаем файл на диске, нам нельзя идти к file offset 0x28040 автоматически. Нужно найти секцию, в которую попадает RVA 0x28040. Допустим, секция .xdata имеет:

```
VirtualAddress   = 0x00028000
VirtualSize      = 0x00002000
PointerToRawData = 0x0001F000
```

Тогда:

```
Delta      = 0x28040 - 0x28000 = 0x40
FileOffset = 0x1F000 + 0x40 = 0x1F040
```

И только теперь статический parser идёт читать файл по offset 0x1F040.

Третий шаг: не делать преждевременных выводов. Мы ещё не знаем, есть ли у функции handler. Мы только нашли UNWIND_INFO. Нужно открыть структуру, прочитать Version, Flags, SizeOfProlog, CountOfCodes, FrameRegister/FrameOffset, массив UNWIND_CODE и только потом смотреть, что лежит после массива. Если Flags говорят EHANDLER или UHANDLER, после unwind codes будет handler RVA. Если Flags говорят CHAININFO, хвост структуры читается иначе.

Вот почему фраза “нашли RUNTIME_FUNCTION” - это не конец, а начало следующего уровня анализа.

7.2.15. Ручной поиск записи по ControlRva

Если таблица маленькая, человек может просмотреть записи глазами. Если таблица большая, нужно понимать принцип поиска. Допустим, Exception Directory содержит N записей. Каждая запись отсортирована по BeginAddress. Мы ищем запись, где:

```
BeginAddress <= ControlRva < EndAddress
```

Линейный поиск выглядит просто:

```
for each entry in pdata:
    if entry.BeginAddress <= ControlRva < entry.EndAddress:
        found
```

Но Windows и быстрые инструменты могут использовать бинарный поиск по BeginAddress. Для ручного понимания достаточно знать: таблица должна быть отсортирована, поэтому если мы уже прошли записи, у которых BeginAddress больше ControlRva, дальше искать бессмысленно.

Пример:

```
[0] Begin 1000 End 1080
[1] Begin 1080 End 1120
[2] Begin 1200 End 12A0
[3] Begin 1400 End 1500
```

ControlRva = 0x1300. Запись [2] заканчивается на 0x12A0, значит 0x1300 туда не попадает. Следующая запись начинается с 0x1400, а это уже больше 0x1300. Значит, записи нет. Мы попали в gap 0x12A0-0x1400.

И вот здесь ручной анализ задаёт вопрос: что находится по RVA 0x1300? Если там padding, нормально. Если там leaf thunk, возможно нормально. Если там полноценная функция с sub rsp и call, подозрительно. Если там данные, значит ControlPc не должен был туда попадать как в код. Если туда реально попал RIP во время исключения, это отдельное расследование.

7.2.16. Что значит “active function tables”, и почему .pdata в файле - не вся вселенная

Ранее мы говорили о .pdata внутри PE-файла. Но документация RtlLookupFunctionEntry говорит об active function tables. Active означает “активные”, то есть те function tables, которые Windows учитывает в текущем процессе во время выполнения. Обычно для обычного модуля это таблица из Exception Directory. Но могут быть и dynamic function tables.

Dynamic function table - динамическая таблица функций. Она нужна, когда код появляется во время выполнения, а не лежит заранее в PE .text. Например, JIT-компилятор может сгенерировать x64 машинный код в памяти. Чтобы Windows могла раскручивать стек через этот код, runtime должен зарегистрировать function table через механизмы вроде RtlAddFunctionTable или callback-based registration.

Это важно для честной теории. Когда мы читаем статический PE, мы видим только то, что лежит в файле. Но во время выполнения процесс может зарегистрировать дополнительные таблицы. Поэтому статический анализ .pdata отвечает на вопрос: “что PE-образ объявил заранее?” Он не всегда отвечает на вопрос: “какие function tables будут активны в конкретный момент runtime?”

Для обычного EXE/DLL без JIT разница мала. Для .NET, JavaScript engines, браузеров, виртуальных машин, dynamic s, unpackers и manual mapping сценариев разница может быть огромной.

Manual mapping - ручное отображение модуля в память без стандартного loader-а Windows. В таком сценарии код может быть загружен не как обычный модуль, и стандартные PE-runtime сервисы могут не видеть его так же, как обычный LoadLibrary-loaded image. Если manual mapper не зарегистрировал function tables, exception handling и stack walking могут ломаться.

Поэтому static должен ясно разделять:

```
StaticExceptionDirectory - что объявлено в PE-файле
RuntimeActiveFunctionTables - что реально зарегистрировано в процессе
```

В статическом PDF-разделе мы учимся читать первое. Но теория должна помнить второе.

7.2.17. Как .pdata связана с search phase

Теперь вернёмся к главе 6. Там мы разбирали порядок доставки исключения: debugger first chance, VEH, frame-based SEH, unwind/search, second chance, unhandled layer. Где именно в этом маршруте участвует .pdata?

.pdata становится важной, когда Windows должна идти по frame-based цепочке вызовов. У неё есть ContextRecord с RIP. Чтобы понять текущий кадр, она должна найти function entry для ControlPc. Для этого она использует function tables. Для обычного PE32+

модуля function table приходит из Exception Directory, то есть из .pdata.

Если запись найдена, Windows получает UnwindInfoAddress и переходит к UNWIND_INFO. Там она может увидеть, есть ли handler flags. Если handler flags есть, в search phase может быть вызван language-specific handler. Если handler не принимает исключение, RtlVirtualUnwind использует unwind codes, чтобы получить caller context. Потом процесс повторяется для следующего кадра.

Значит, .pdata участвует не только в “красивом stack trace”, а в самом маршруте исключения. Без корректной function table Windows может не найти handler, неправильно восстановить caller или принять код за leaf.

Связь выглядит так:

```
ExceptionRecord + ContextRecord
    -> ControlPc / RIP
        -> RtlLookupFunctionEntry
            -> .pdata / RUNTIME_FUNCTION
                -> .xdata / UNWIND_INFO
                    -> handler flags или unwind codes
                        -> search phase / virtual unwind
```

Когда это понимаешь, .pdata перестаёт быть “технической секцией”. Она становится частью живой обработки исключений. Именно поэтому мы уделяем ей столько внимания.

7.2.18. Как .pdata связана с debugger support и crash dump analysis

Исключения - не единственный пользователь .pdata. Отладчик и crash dump analyzer тоже хотят строить stack trace. Stack trace - это цепочка вызовов: кто кого вызвал, как выполнение дошло до текущего места. На x64 хороший stack walking опирается на unwind metadata.

Когда процесс упал и создан dump, программа уже не выполняется. У анализатора есть снимок памяти, регистры, список модулей и PE metadata. Чтобы восстановить стек, он делает похожую работу: берёт RIP, ищет RUNTIME_FUNCTION, читает UNWIND_INFO, виртуально поднимается к caller-у, повторяет.

Если .pdata повреждена или отсутствует, stack trace может стать неполным или ложным. Иногда отладчик покажет только верхний кадр. Иногда начнёт гадать по стеку. Иногда покажет адреса, которые выглядят как функции, но на самом деле являются случайными значениями.

Поэтому .pdata важна не только для “поймать исключение”. Она важна для диагностики после падения. Если ты видишь в crash dump странный стек, один из вопросов: корректна ли .pdata/.xdata для модулей в цепочке?

Для reverse engineer-а это полезно и в другую сторону. Если stack trace выглядит подозрительно, нужно проверить не только стек, но и unwind metadata. Возможно, код

нормальный, но metadata сломана. Возможно, metadata нормальная, но стек повреждён. Возможно, оба слоя конфликтуют.

7.2.19. Почему .pdata - источник доверия и одновременно источник подозрений

В нормальном скомпилированном x64 PE .pdata создаётся компилятором и линкером, потому что она нужна Windows. Это делает её сильным источником истины. Но в reverse engineering нельзя забывать: бинарь может быть не “нормальным”. Он может быть упакован, защищён, модифицирован, сгенерирован нестандартным toolchain, повреждён или специально сделан так, чтобы сбивать анализаторы.

.pdata может помогать восстановить функции там, где дизассемблер ошибается. Но .pdata также может быть намеренно искажена. Например, dynamic может использовать нестандартные handler-и, deliberate exceptions, dynamic code, manual function tables. Packer может скрывать настоящий код до распаковки. Manual mapping может сделать статическую .pdata менее значимой для runtime.

Поэтому зрелая позиция такая: .pdata - это не “истина последней инстанции”, а формальное заявление PE-образа. Мы проверяем это заявление против других фактов.

Факты могут быть такие:

```
- секции PE и их права
- реальные байты кода
- дизассемблированный пролог
- UNWIND_INFO
- handler flags
- stack trace в отладчике
- поведение при исключении
- зарегистрированные runtime function tables
```

Если .pdata заявляет, что диапазон функции начинается в .rdata без execute-права, это странно. Если .pdata указывает UnwindInfoAddress в пустую область, это странно. Если .pdata покрывает код, но UNWIND_INFO не соответствует прологу, это странно. Если .pdata отсутствует, но код clearly non-leaf, это странно. Но каждую странность нужно формулировать точно.

7.2.20. Типичные ошибки при чтении .pdata

Первая ошибка - читать .pdata по имени секции, игнорируя Exception Directory. Так можно пропустить таблицу, если она лежит не в секции с именем .pdata, или наоборот принять за таблицу данные, которые просто находятся в секции с таким именем, но не объявлены Data Directory.

Вторая ошибка - путать RVA и file offset. BeginAddress, EndAddress и UnwindInfoAddress - это image-relative values. Если анализатор читает файл, он должен переводить RVA в file offset через section table.

Третья ошибка - считать, что .pdata содержит все функции. Leaf-функции могут отсутствовать. Некоторые куски кода могут быть вне таблицы. Динамический код может регистрировать таблицы во время выполнения.

Четвёртая ошибка - думать, что UnwindInfoAddress указывает на handler. Нет. Он указывает на UNWIND_INFO. Handler появляется только если flags внутри UNWIND_INFO говорят EHANDLER или UHANDLER, и даже тогда handler RVA - это language-specific handler, а не обязательно пользовательский __except-блок.

Пятая ошибка - не проверять сортировку и пересечения. Таблица должна быть картой, а не хаосом. Если диапазоны перекрываются или идут в неожиданном порядке, это влияет на поиск.

Шестая ошибка - не сверять .pdata с кодом. Таблица может выглядеть структурно валидной, но не соответствовать реальному прологу функции. Такой конфликт обнаружится только при переходе к .xdata и дизассемблеру.

Седьмая ошибка - делать выводы без уровня уверенности. “Нет записи” может означать leaf. Может означать ошибку. Может означать dynamic table. Может означать obfuscation. Нужно смотреть контекст.

7.2.21. Как будущий static должен представлять .pdata не как сырые числа, а как объясняемую карту

Если будущий инструмент просто выведет:

```
Begin=0x1200 End=0x12A0 Unwind=0x28040
```

это полезно, но недостаточно. Хороший static должен превращать числа в объяснение. Он должен показывать координаты, секции, проверки и связи.

Например:

```
Function table entry #42
Range RVA: 0x1200-0x12A0
Range VA:  0x00007FF700001200-0x00007FF7000012A0
Section:   .text, execute/read
Size:      0xA0 bytes
Unwind RVA: 0x28040
Unwind VA:  0x00007FF700028040
Unwind section: .xdata, read-only
Status: range is sorted, non-overlapping, unwind RVA is inside image
Next: parse UNWIND_INFO
```

Такой вывод обучает. Он не просто сообщает результат, а показывает путь. Пользователь видит, как VA связан с RVA, как RVA связан с секцией, как .pdata ведёт в .xdata, как проверяется валидность.

Для книги это тоже важно. Мы не просто учим читать таблицу. Мы учим строить доказательную цепочку. Reverse engineering - это не угадывание. Это сбор

согласованных фактов.

7.2.22. Мини-алгоритм чтения .pdata руками

Теперь можно собрать медленную ручную процедуру. Не как код, а как мыслительный маршрут.

Сначала открыть PE headers и найти Optional Header. Убедиться, что это PE32+ image, потому что для нашей модели важна x64 exception handling структура. Проверить Machine, чтобы понимать архитектуру.

Потом найти Data Directory. Взять запись Exception Directory. Проверить VirtualAddress и Size. Если они нулевые, официальная статическая exception table отсутствует. Это не конец анализа, но важный факт.

Если RVA ненулевой, найти секцию, в которую он попадает. Посмотреть имя секции, права, raw и virtual sizes. Перевести RVA в file offset, если анализ статический.

Дальше разделить Size на 12 байт. Проверить кратность. Получить число записей. Прочитать каждую запись как BeginAddress, EndAddress, UnwindInfoAddress.

Для каждой записи проверить базовые инварианты: BeginAddress < EndAddress; диапазон попадает в image; начало диапазона обычно попадает в исполняемую секцию; UnwindInfoAddress попадает в читаемую область image; записи отсортированы; диапазоны не пересекаются.

Потом для интересующего ControlRva найти подходящую запись. Если она есть, перейти к UnwindInfoAddress. Если нет, оценить gap и содержимое кода в этом месте.

И только после этого переходить к .xdata. То есть .pdata - это не весь анализ. Это подготовка правильного входа в UNWIND_INFO.

7.2.23. Как этот раздел связан с 7.1 и почему следующий шаг - .xdata

Раздел 7.1 научил нас переходить от конкретного RIP к координатам PE: module, ImageBase, RVA. Без этого .pdata невозможно читать правильно, потому что все её поля image-relative.

Раздел 7.2 научил нас читать саму таблицу .pdata: откуда она находится, как делится на RUNTIME_FUNCTION записи, как искать диапазон, как проверять сортировку, gaps, overlaps, границы и UnwindInfoAddress.

Но пока мы всё ещё не знаем, как именно функция раскручивается. Мы знаем только, где лежит рецепт. Этот рецепт начинается в .xdata со структуры UNWIND_INFO. Именно там появятся Version, Flags, SizeOfProlog, CountOfCodes, FrameRegister, FrameOffset, UNWIND_CODE и хвост структуры, который может означать handler или chained info.

Поэтому следующий раздел естественно продолжает маршрут:

```
7.1: RIP -> module -> ImageBase -> RVA
7.2: RVA -> .pdata -> RUNTIME_FUNCTION -> UnwindInfoAddress
7.3: UnwindInfoAddress -> .xdata -> UNWIND_INFO
```

Так строится фундамент. Мы не прыгаем сразу к SEH handler-ам. Мы медленно идём по тому же пути, по которому должна идти Windows: адрес, модуль, таблица функций, unwind-информация, обработчики.

Техническая база раздела 7.2

Раздел опирается на официальную документацию Microsoft по PE Format, где PE/COFF описывает executable image files, sections, RVA, Data Directory и Exception Directory; на документацию Microsoft по x64 exception handling, где указано, что RUNTIME_FUNCTION записи имеют image-relative адреса, сортируются и помещаются в .pdata PE32+ image, а также ведут к unwind data; на документацию RtlLookupFunctionEntry, где функция определяется как поиск записи в active function tables по PC value; и на документацию RtlVirtualUnwind, где FunctionEntry, ImageBase, ControlPc и ContextRecord связываются с получением контекста вызывающей функции. Текст раздела написан как учебное объяснение и связывает эти технические источники с практической моделью reverse engineering.

7.3. Как читать .xdata руками: UNWIND_INFO как инструкция по разборке кадра

В предыдущем разделе мы разобрали .pdata как карту диапазонов функций. Это был очень важный шаг, но сама по себе .pdata ещё не объясняет, как именно Windows должна раскрутить текущую функцию. .pdata отвечает на вопрос: "какому диапазону кода принадлежит текущий RIP и где лежит описание этого диапазона?" Но после этого сразу появляется следующий вопрос: "а что именно написано в описании?" Вот тут мы переходим к .xdata и UNWIND_INFO.

.xdata надо понимать не как отдельную скучную секцию, а как вторую половину x64 unwind-контракта. .pdata говорит: вот функция, вот её начало, вот её конец, вот адрес unwind-описания. .xdata говорит: вот как эта функция построила свой кадр, вот какие действия пролога нужно учитывать, вот какие регистры сохранены, вот есть ли у функции обработчик, и вот где лежат дополнительные handler-данные. Если .pdata - это указатель на карту, то .xdata - это сама инструкция, по которой Windows может разобрать кадр.

В этой теме нельзя спешить. Если быстро сказать "UNWIND_INFO содержит флаги и массив UNWIND_CODE", у читателя останется только набор названий. А нам нужна не зубрёжка полей, а понимание роли каждого поля в общей системе. Почему вообще нужно хранить размер пролога? Почему количество unwind-кодов не равно количеству инструкций функции? Почему после массива кодов иногда лежит адрес handler-а, а иногда chained info? Почему всё это выровнено по границе? Почему один и тот же хвост структуры меняет смысл в зависимости от флагов? Эти вопросы важнее, чем механическое чтение таблицы.

7.3.1. .xdata: почему подробности вынесены отдельно от .pdata

.xdata обычно расшифровывают как extended data - расширенные данные. Слово extended означает "расширенные", "дополнительные", "выходящие за рамки короткой записи". В контексте Windows x64 это хорошее название: .pdata хранит короткие, однотипные записи RUNTIME_FUNCTION, а .xdata хранит более подробные и переменные по размеру данные. То есть .pdata можно быстро искать как таблицу диапазонов, а .xdata можно читать уже после того, как нужная функция найдена.

Почему нельзя было положить все прямо в .pdata? Потому что RUNTIME_FUNCTION должен быть простой и одинаковой длины. В нормальном PE32+ image каждая запись RUNTIME_FUNCTION занимает три 32-битных поля: BeginAddress, EndAddress и UnwindInfoAddress. Это удобно для сортировки, бинарного поиска, проверки диапазонов и прохода по таблице. Если бы туда сразу вставляли весь UNWIND_INFO, записи стали бы разной длины. Тогда поиск по .pdata стал бы сложнее, а сама таблица перестала бы быть аккуратной картой диапазонов.

Функции бывают очень разные. Одна функция вообще не трогает стек и может не иметь записи. Другая сохраняет один RBX и выделяет 0x20 байт. Третья сохраняет RBX, RBP, RSI, RDI, R12, выделяет большой стек, использует frame register и имеет SEH handler. Четвертая использует chained unwind info. У всех этих функций разный объём описания. Поэтому архитектурно удобно разделить: короткий индекс функций лежит в .pdata, а подробная переменная информация лежит отдельно, обычно в .xdata.

Это разделение важно для reverse engineering. Когда ты смотришь на .pdata, ты не должен ожидать увидеть там весь смысл функции. .pdata - это вход. Она ведёт к .xdata. А когда ты смотришь на .xdata, ты уже читаешь не просто "адрес следующей структуры", а описание поведения функции относительно стека и обработчиков. Поэтому маршрут всегда такой: RIP -> module -> ImageBase -> RVA -> .pdata -> RUNTIME_FUNCTION -> UnwindInfoAddress -> .xdata -> UNWIND_INFO.

```
RIP / ControlPc
  -> module + ImageBase
  -> RVA текущей точки
  -> Exception Directory
  -> .pdata / RUNTIME_FUNCTION
  -> UnwindInfoAddress
  -> .xdata / UNWIND_INFO
```

Эта цепочка должна стать почти рефлексом. Мы не прыгаем сразу к handler-у. Мы не начинаем с догадки "это try/catch". Мы идём так же, как должна идти Windows: от адреса выполнения к записи функции, от записи функции к unwind-описанию, от unwind-описания к способу восстановления кадра и только потом к обработчикам.

7.3.2. UNWIND_INFO: не комментарий компилятора, а машинно-читаемый договор

UNWIND_INFO - это структура unwind-информации. Слово unwind мы уже переводили как "раскрутка" или "размотка" стека. Но важно повторить: unwind - это не физическое движение назад по инструкциям. Windows не исполняет функцию в обратную сторону. Она берёт текущий машинный

контекст и по заранее подготовленному описанию вычисляет, каким был бы контекст вызывающей функции.

Поэтому UNWIND_INFO можно назвать машинно-читаемой инструкцией по разборке кадра. Машинно-читаемой - потому что её читает не человек глазами, а системный unwinder, отладчик, dump analyzer или твой будущий static . Инструкцией - потому что она описывает, какие изменения кадра нужно учитывать. Разборкой кадра - потому что функция в прологе собрала кадр, а unwind должен понять, как этот кадр снять.

Тут полезно вспомнить пролог. Prolog - начальный участок функции, где она готовит рабочее состояние: сохраняет nonvolatile-регистры, выделяет место на стеке, иногда устанавливает frame register. Тело функции работает уже после этой подготовки. Эпилог делает обратное: освобождает стек, восстанавливает регистры и возвращается к caller-у. UNWIND_INFO описывает не весь код функции, а именно те действия, которые важны для восстановления caller context.

Caller context - контекст вызывающей функции. Context здесь означает значения регистров, RIP, RSP и другое машинное состояние. Если исключение произошло внутри функции C, а Windows хочет перейти к кадру функции B, которая вызвала C, ей нужно восстановить контекст B. Она должна знать, каким был RSP у B, где лежит return address, какие nonvolatile-регистры C сохранила и откуда их надо вернуть.

Вот почему UNWIND_INFO - это не справочная информация "для красоты". Если она неправильная, Windows может восстановить неправильный RSP, взять return address не из того места, получить мусорный RIP и построить ложный stack trace. Если у функции есть exception handler, но флаги в UNWIND_INFO не говорят об этом, Windows может не вызвать нужный language-specific handler. Если флаги говорят о handler-е, но адрес handler-а испорчен, механизм обработки исключений может уйти в неверное место. То есть .xdata влияет на реальное выполнение при исключениях и на реальную диагностику.

7.3.3. Где начинается UNWIND_INFO: UnwindInfoAddress как RVA, а не file offset

Путь к UNWIND_INFO начинается с поля UnwindInfoAddress внутри RUNTIME_FUNCTION. Это поле часто хочется прочитать слишком буквально: "адрес unwind info". Но в PE32+ x64 это не готовый абсолютный адрес в памяти и не смещение в файле. Это image-relative address, то есть RVA относительно ImageBase загруженного образа.

RVA - Relative Virtual Address, относительный виртуальный адрес. Он говорит: "отступи от начала загруженного image на такое-то количество байт". Если module загружен по ImageBase 0x00007FF700000000, а UnwindInfoAddress равен 0x000000000000A300, то реальный VA структуры в живом процессе будет 0x00007FF70000A300. Но если ты читаешь файл с диска, тебе нужно не прибавлять ImageBase, а перевести RVA в file offset через section table.

```
В живом процессе:
VA_UNWIND_INFO = ImageBase + UnwindInfoAddress

При статическом чтении файла:
file_offset = rva_to_file_offset(UnwindInfoAddress)
```

Это место настолько важное, что его лучше повторить ещё раз. UnwindInfoAddress не указывает "в .xdata" магически сам по себе. Это RVA. Обычно этот RVA попадает в секцию .xdata, но формально анализатор должен проверить, в какую секцию он попадает. В обычном MSVC PE это часто .xdata. В нестандартном файле данные могут лежать в секции с другим именем. Поэтому хороший анализ не должен говорить "ищем секцию по имени .xdata и читаем там". Он должен говорить: "берём RVA из UnwindInfoAddress, переводим его в место образа, проверяем секцию, права, границы и читаем структуру".

Типичная ошибка новичка - открыть hex-редактор, взять UnwindInfoAddress, перейти на такой же offset в файле и увидеть мусор. Это не значит, что файл сломан. Это значит, что перепутаны координатные системы. В PE всегда нужно держать в голове три мира: VA - адрес в процессе, RVA - адрес относительно ImageBase, file offset - смещение в файле. .xdata живёт в image-мире, а файл на диске нужно читать через перевод RVA -> file offset.

7.3.4. Общая форма UNWIND_INFO: маленькая структура с переменным хвостом

UNWIND_INFO кажется небольшой структурой, но её нельзя понимать как фиксированную запись одного размера. У неё есть начальная часть с несколькими полями, затем массив UNWIND_CODE, затем выравнивание, а после этого - хвост, смысл которого зависит от Flags. Именно этот переменный хвост часто ломает парсеры и понимание.

Упрощённо структура выглядит так:

```
UNWIND_INFO:
  Version + Flags
  SizeOfProlog
  CountOfCodes
  FrameRegister + FrameOffset
  UnwindCode[CountOfCodes]
```

```
padding до четного количества UNWIND_CODE slots
затем, в зависимости от Flags:
  handler RVA + language-specific handler data
  или chained RUNTIME_FUNCTION
  или ничего
```

Здесь важно слово "slots" - слоты. UNWIND_CODE обычно описывают как массив элементов, но некоторые unwind-операции занимают больше одного слота. Поэтому CountOfCodes - это не просто "количество логических действий". Это количество двухбайтовых записей UNWIND_CODE, занятых в массиве. Например, операция большого выделения стека может требовать дополнительный слот для размера. Сохранение регистра по большому смещению тоже может занимать дополнительные слоты.

У структуры есть начало, которое можно прочитать почти всегда одинаково. Но после массива UNWIND_CODE начинается зона, где надо быть аккуратным. Если флагов handler-а нет и CHAININFO нет, хвоста может не быть. Если есть EHANDLER или UHANDLER, после выравнивания лежит RVA language-specific handler-а, а затем handler data. Если есть CHAININFO, после выравнивания лежит не handler RVA, а запись, похожая на RUNTIME_FUNCTION. Одинаковые байты могут означать разные вещи в зависимости от Flags. Это фундаментальное правило бинарных форматов: поле не существует отдельно от контекста интерпретации.

7.3.5. Version: почему версия нужна даже у маленькой структуры

Первое поле внутри UNWIND_INFO упаковано вместе с Flags. Младшие биты обычно содержат Version - версию формата. Version переводится как "версия". На первый взгляд это скучное поле: ну версия и версия. Но в бинарном формате версия нужна для будущей расширяемости. Она говорит читателю структуры, по каким правилам понимать остальные поля.

В обычной классической x64 unwind-информации используется ранняя версия формата, которую чаще всего и разбирают в учебных материалах. Но сама идея version важна: Microsoft может расширять правила unwind-информации, компилятор может генерировать новые варианты для новых возможностей процессора, а парсер должен не слепо читать всё как старую структуру, а хотя бы проверить, что версия ему понятна.

Для будущего static -а это означает: если Version неожиданная, нельзя молча продолжать так, будто всё нормально. Нужно сформулировать finding: "версия UNWIND_INFO не соответствует ожидаемой модели; дальнейшая интерпретация может быть ненадёжной". Это не обязательно malware и не обязательно ошибка файла. Но это важное условие доверия к анализу.

Связь с теорией здесь такая: unwind-контракт работает только если обе стороны понимают формат. Компилятор записал данные в определённой версии. Windows unwinder читает их по правилам этой версии. Анализатор должен делать то же самое. Если анализатор не учитывает Version, он может неправильно разобрать структуру и обвинить бинарь в аномалии там, где виноват сам парсер.

7.3.6. Flags: маленькие биты, которые меняют смысл хвоста

Flags - флаги. Флаг в бинарных структурах - это отдельный бит или набор битов, которые включают или выключают определённое свойство. Если поле Version говорит "по какой версии читать структуру", то Flags говорят "какой дополнительный смысл есть у этой unwind-информации".

Для нашей темы важнее всего четыре значения: NHANDLER, EHANDLER, UHANDLER и CHAININFO. Их нужно понимать не как набор сокращений, а как развилки в маршруте Windows.

UNW_FLAG_NHANDLER означает, что в этой unwind-информации нет language-specific handler-а и нет chained info. Функция всё равно может иметь unwind-коды. Она может сохранять регистры, выделять стек, иметь нормальный пролог и эпилог. Просто после массива UNWIND_CODE нет дополнительного handler-хвоста. Такая функция нужна для stack walking и восстановления caller context, но сама она не заявляет отдельный обработчик исключения через эту запись.

UNW_FLAG_EHANDLER означает exception handler. Exception handler - обработчик исключения. Этот флаг говорит: после массива unwind-кодов и выравнивания находится RVA language-specific handler-а. Во время search phase Windows может вызвать этот handler и спросить, хочет ли текущий кадр обработать исключение. Для MSVC C SEH таким handler-ом часто является __C_specific_handler.

UNW_FLAG_UHANDLER означает unwind handler или termination handler. Его смысл особенно связан с unwind phase. Такой handler может быть нужен не для того, чтобы "поймать" исключение, а чтобы выполнить действия при раскрутке: cleanup, finally-логику, освобождение ресурсов. Поэтому UHANDLER нельзя переводить в голове как "ещё один catch". Это скорее участник контролируемого выхода из кадра.

UNW_FLAG_CHAININFO означает chained unwind info - цепочная unwind-информация. Если этот флаг установлен, хвост структуры не является handler RVA. Там лежит связанная запись, похожая на RUNTIME_FUNCTION, которая ведёт к другой unwind-информации. Это

используется для более сложных случаев, например когда часть функции разделяет основную unwind-информацию с другой частью. Для парсера это критично: принять chained entry за handler address - значит полностью сломать анализ.

```
Если Flags = NHANDLER:
  после UNWIND_CODE ничего специального нет

Если Flags содержит EHANDLER или UHANDLER:
  после UNWIND_CODE лежит Handler RVA
  затем language-specific handler data

Если Flags содержит CHAININFO:
  после UNWIND_CODE лежит chained RUNTIME_FUNCTION-like entry
```

Вот почему флаги нужно читать до хвоста. Нельзя прыгать к байтам после UNWIND_CODE и заранее решать, что это handler. Сначала Version и Flags, потом CountOfCodes, потом массив кодов, потом выравнивание, потом интерпретация хвоста строго по Flags.

7.3.7. SizeOfProlog: зачем Windows знать длину пролога

SizeOfProlog - размер пролога. Prolog, или пролог функции, - это начальная область, где функция строит свой кадр. SizeOfProlog говорит, сколько байт от начала функции занимает этот пролог. Казалось бы, зачем это хранить? Ведь unwind-коды уже описывают действия пролога. Но Windows нужно понимать, находится ли текущий ControlPc внутри пролога, тела или эпилога.

Представим функцию:

```
push rbx
sub  rsp, 40h
mov  rbx, rcx
... тело функции ...
```

Если исключение произошло уже в теле, после полного выполнения пролога, unwind может считать, что все действия пролога уже выполнены: RBX сохранён, RSP уменьшен на 0x40, кадр построен. Но если исключение произошло на самой первой инструкции пролога или между push rbx и sub rsp, 40h, ситуация другая. Нельзя применять полное описание кадра, потому что часть действий ещё не успела произойти.

SizeOfProlog помогает определить эту зону. Если offset текущего ControlPc от BeginAddress функции меньше SizeOfProlog, значит мы внутри пролога. Тогда unwinder должен частично отменять только те операции, которые уже реально произошли к этой точке. Это очень тонкая, но важная часть

корректности.

Почему это связано с reverse engineering? Потому что если ты руками анализируешь crash и видишь, что RIP находится в первых байтах функции, нельзя сразу думать о кадре так, будто он полностью построен. Нужно проверить SizeOfProlog и конкретные offset-ы unwind-кодов. Возможно, функция упала до сохранения регистра или до выделения стека. Тогда ручное восстановление caller context будет отличаться от обычной ситуации в теле функции.

Для будущего static -а SizeOfProlog - это не декоративное поле. Это поле для проверки согласованности: действительно ли пролог такой длины? Попадают ли описанные операции внутрь этой зоны? Не указывает ли SizeOfProlog на странно маленькое или странно большое значение? Не заявляет ли metadata пролог, который не похож на реальные инструкции? Такие вопросы помогают отличать нормальный компиляторный код от повреждённого или намеренно запутанного.

7.3.8. CountOfCodes: количество слотов, а не количество человеческих действий

CountOfCodes - количество unwind-кодов. Но здесь сразу нужна поправка: это количество записей UNWIND_CODE в массиве, а не количество инструкций пролога и не количество логических действий, которые человек видит в дизассемблере. Некоторые действия занимают один слот, некоторые - два, некоторые - три. Поэтому CountOfCodes нужно читать как размер массива в специальных двухбайтовых элементах.

UNWIND_CODE - это элемент массива, который описывает одну unwind-операцию или часть операции. Он содержит offset в прологе и код операции. Offset говорит, в какой точке пролога эта операция была выполнена. Operation code, или unwind operation, говорит, что это было за действие с точки зрения unwind: push nonvolatile-регистра, выделение стека, сохранение регистра по смещению и так далее.

Почему CountOfCodes не равен количеству asm-инструкций? Потому что unwind-метаданные не описывают все инструкции. Если в прологе есть mov rbx, rcx, это может быть логика функции, но для unwind она не важна, если не меняет RSP и не сохраняет nonvolatile state. А если есть sub rsp, 1000h, это одна asm-инструкция, но для кодирования размера может потребоваться дополнительный слот. Поэтому связь не "один к одному", а "важные для unwind эффекты пролога кодируются compact-структурами".

Compact - компактный. UNWIND_INFO не пытается быть вторым дизассемблером. Он хранит только то, что нужно для восстановления. Это

ключевая мысль. Если ты ищешь в UNWIND_CODE каждую инструкцию пролога, ты неправильно понимаешь назначение структуры. Она не для восстановления исходного кода. Она для восстановления caller context.

7.3.9. FrameRegister и FrameOffset: когда RSP уже не удобная база

В UNWIND_INFO есть поля FrameRegister и FrameOffset. Они связаны с идеей frame register - регистра кадра. Frame register - это регистр, который функция использует как более стабильную базу для обращения к своему стековому кадру. В старых учебных схемах часто показывают RBP как frame pointer: функция делает push rbp; mov rbp, rsp, и дальше локальные переменные доступны через [rbp-...] или [rbp+...]. Но на Windows x64 классический frame pointer не обязателен.

Почему frame register вообще может понадобиться? Потому что RSP может двигаться. Если функция делает только один sub rsp, N в прологе и потом не меняет RSP до эпилога, обращаться к локальным данным через RSP удобно. Но если функция делает динамическое выделение на стеке, например alloca, или имеет сложную организацию кадра, RSP перестаёт быть стабильной базой. Тогда компилятор может выбрать регистр, который будет указывать на определённую точку внутри кадра, и использовать его как frame register.

FrameOffset - смещение, связанное с этим frame register. Важная идея: frame register не просто "какой-то указатель". Он должен быть согласован с unwind-моделью. Если unwinder восстанавливает контекст и видит, что функция использовала frame register, он должен понимать, как этот регистр связан со стековым выделением. Поэтому FrameRegister и FrameOffset находятся прямо в UNWIND_INFO, рядом с остальными данными о кадре.

Новичок часто ошибается так: видит RBP в коде и автоматически считает, что это frame pointer. На x64 это неверно. RBP - nonvolatile-регистр, и компилятор может использовать его как обычный регистр. Истина не в имени регистра, а в сочетании кода и metadata. Если UNWIND_INFO указывает FrameRegister, тогда это часть unwind-контракта. Если не указывает, RBP может быть просто сохранённым nonvolatile-регистром или вообще не иметь роли классического указателя кадра.

Для reverse engineering FrameRegister полезен как мост между дизассемблером и unwind-структурами. Если декомпилятор показывает локальные переменные от RBP, а UNWIND_INFO подтверждает frame register, картина становится сильнее. Если код активно использует RBP как базу, но metadata этого не отражает, надо разбираться: это оптимизация? другой регистр? неправильная граница функции? обфускация? ошибка анализа?

7.3.10. UNWIND_CODE: компактный язык описания пролога

Теперь подходим к сердцу UNWIND_INFO - массиву UNWIND_CODE. Его можно понимать как маленький специализированный язык, на котором компилятор рассказывает Windows: "в прологе я сделал такие действия, и вот как их нужно учитывать при раскрутке". Это не машинный код x64. Это метаязык для unwinder-а.

Meta-language - метаязык, язык описания другого уровня. Машинный код исполняется процессором. UNWIND_CODE не исполняется процессором как инструкции программы. Его читает unwinder, чтобы понять, как интерпретировать текущий стек. Поэтому если в .text лежит код функции, то в .xdata лежит не "ещё один код", а описание эффектов этого кода.

Каждый UNWIND_CODE обычно содержит offset in prolog - смещение в прологе, и unwind operation - тип действия. Offset нужен для частичного unwind-а внутри пролога. Operation нужен для понимания, что именно произошло: был ли push nonvolatile-регистра, было ли выделение стека, был ли сохранён XMM-регистр, была ли установка frame pointer.

Почему offset хранится именно относительно пролога? Потому что при исключении внутри пролога нужно знать, какие операции уже выполнены. Если операция записана с offset 5, а текущий ControlPc соответствует offset 3, значит эта операция ещё не произошла. Если ControlPc дальше offset 5, значит произошла. Так unwinder может аккуратно отменить не весь пролог, а только выполненную часть.

Здесь важно не смешивать два времени. Время выполнения функции идёт от начала пролога к телу. Unwind, наоборот, восстанавливает состояние caller-а. Поэтому unwind-коды часто применяются в обратном смысловом направлении: если пролог уменьшил RSP, unwind увеличивает RSP; если пролог сохранил RBX на стеке, unwind читает RBX из этого места; если пролог сделал push rbx, unwind понимает, что старое значение RBX лежит по соответствующему адресу.

7.3.11. Почему UNWIND_CODE не обязан совпадать с дизассемблером один к одному

Очень частая ошибка: открыть функцию, увидеть три инструкции в прологе и ожидать три unwind-кода. Или наоборот: увидеть пять unwind slots и думать, что в прологе должно быть пять инструкций. Так не работает. UNWIND_CODE описывает эффекты, а не текст инструкций.

Например, инструкция sub rsp, 40h может быть описана как allocation operation - операция выделения стекового пространства. Если размер небольшой, она может занимать один вариант кодирования. Если размер большой, нужен другой вариант и дополнительные данные. Одна asm-инструкция может занимать несколько UNWIND_CODE slots. А инструкция mov rbx, rcx может вообще не попасть в unwind-информацию, потому что она не восстанавливает caller context и не является сохранением nonvolatile-регистра.

С другой стороны, push rbx обычно важен, потому что RBX является nonvolatile-регистром. Nonvolatile register - сохраняемый регистр по соглашению вызовов. Если функция его меняет, она обязана сохранить старое значение и восстановить перед выходом. Если исключение прервало функцию до обычного восстановления, unwinder должен сам восстановить RBX при переходе к caller context. Поэтому сохранение RBX попадает в UNWIND_CODE.

Правильная модель такая: дизассемблер показывает, какие инструкции реально выполняет CPU; UNWIND_CODE показывает, какие эффекты этих инструкций важны для системной раскрутки кадра. Эти два источника должны быть согласованы по смыслу, но не обязаны быть одинаковыми по количеству строк.

```
Дизассемблер отвечает:
  какие инструкции лежат в .text?

UNWIND_CODE отвечает:
  какие действия пролога нужны для восстановления caller context?

Reverse engineer отвечает:
  согласованы ли эти два рассказа?
```

7.3.12. Порядок unwind-кодов: почему читать надо внимательно

В UNWIND_INFO unwind-коды хранятся не просто как произвольный список. Их порядок связан с offset-ами пролога и правилами применения. Для ручного анализа важно не только извлечь операции, но и понять их временную связь с прологом.

Когда функция строит кадр, действия идут в одном порядке: сначала может быть push rbx, потом sub rsp, потом save r12, потом установка frame register. Но при раскрутке кадра мы часто думаем в обратном направлении: чтобы получить caller context, нужно отменить последствия этих действий. Поэтому многие описания и реализации работают с unwind-кодами так, чтобы правильно восстановить состояние независимо от того, где именно произошла остановка.

Практический вывод: нельзя просто прочитать массив и механически вывести "вот список". Нужно связывать каждый unwind-code с offset-ом. Если ControlPc в теле функции, можно считать, что пролог выполнен полностью. Если ControlPc внутри пролога, нужно учитывать только операции, offset которых уже достигнут. Если ControlPc в эпилоге, применяется отдельная логика эпилога. Это опять связывает .xdata с маршрутом exception dispatch, а не оставляет её "статической таблицей".

Для будущего static -а это означает, что вывод должен быть не только структурным, но и контекстным. Плохо: "UWOP_ALLOC_SMALL, UWOP_PUSH_NONVOL". Лучше: "функция при полном прологе выделяет 0x40 байт стека и сохраняет RBX; если исключение произошло после offset X, эти действия считаются выполненными; если до offset X, часть действий не применяется". Такой вывод помогает человеку понять механизм, а не просто увидеть названия.

7.3.13. Выравнивание после массива UNWIND_CODE

После массива UNWIND_CODE есть маленькая, но неприятная деталь: выравнивание. UNWIND_CODE элементы занимают по два байта, а хвост с handler RVA или chained info должен быть выровнен так, чтобы 32-битные значения читались правильно. Поэтому если CountOfCodes нечётный, после массива фактически появляется padding - пустой слот для выравнивания.

Padding переводится как "заполнение". Это байты, которые добавляются не ради логического содержимого, а ради выравнивания структуры. В бинарных форматах padding встречается постоянно. Он нужен, чтобы следующие поля начинались по удобной или требуемой границе. В нашем случае важно, что handler RVA или chained RUNTIME_FUNCTION-like entry начинается не сразу после последнего логического unwind-кода в человеческом смысле, а после массива, дополненного до нужной границы.

```
Если CountOfCodes четный:
  хвост начинается сразу после CountOfCodes slots

Если CountOfCodes нечетный:
  добавляется один padding slot
  хвост начинается после него
```

Почему это важно? Потому что ошибка на два байта полностью ломает чтение хвоста. Ты можешь принять padding за младшую часть handler RVA или сдвинуть чтение chained info. В результате адрес handler-а станет мусором, language-specific data будет прочитана не с того места, а анализатор начнёт видеть аномалии, которых нет. Поэтому ручное чтение .xdata всегда должно учитывать правило выравнивания.

Это также хороший пример того, почему теория нужна даже для практики. Человек, который хочет "просто распарсить структуру", но не понимает, зачем там padding, быстро ошибается. А человек, который понимает: UNWIND_CODE - двухбайтовые слоты, хвост содержит 32-битные RVA или RUNTIME_FUNCTION-like поля, поэтому нужен aligned boundary, - уже видит структуру как систему.

7.3.14. Handler RVA: когда UNWIND_INFO ведёт к language-specific handler

Если в Flags установлен EHANDLER или UHANDLER, после массива UNWIND_CODE и выравнивания лежит RVA handler-а. Это не RVA пользовательского __except-блока напрямую. Это RVA language-specific handler-а. Language-specific handler - обработчик, специфичный для языка или runtime-модели. Для Microsoft C SEH таким handler-ом часто является __C_specific_handler.

Вот здесь новички часто делают серьёзную ошибку. Они видят handler RVA и думают: "это и есть мой catch" или "это и есть код обработки ошибки". Но в x64 SEH чаще всего это не так. Handler RVA указывает на функцию-посредник, которая понимает таблицы конкретного языка/компилятора. Для C SEH __C_specific_handler читает дополнительные данные, смотрит protected ranges, filters, handler blocks и решает, относится ли исключение к текущему кадру.

То есть handler RVA в UNWIND_INFO отвечает на вопрос: "кого Windows должна вызвать как языкового помощника для этого кадра?" А не на вопрос: "куда прыгнуть как в пользовательский обработчик?" Это фундаментальная разница.

```
UNWIND_INFO flags = EHANDLER
  -> Handler RVA = __C_specific_handler
  -> после него лежат language-specific data
  -> __C_specific_handler читает эти данные
  -> там уже находятся protected ranges / filters / handlers
```

Почему Windows устроена так? Потому что ОС не хочет знать внутренний формат каждого языка. C SEH, C++ exceptions, cleanup-механизмы и другие runtime-модели могут иметь разные таблицы. Windows даёт общий каркас: найти функцию, прочитать UNWIND_INFO, вызвать language-specific handler. А handler уже понимает свои данные. Это разделение ответственности: Windows отвечает за frame walking и вызов handler-а, компилятор/runtime отвечает за смысл языковых конструкций.

Для reverse engineering это меняет стиль поиска. Если ты хочешь найти реальный пользовательский обработчик, недостаточно увидеть

__C_specific_handler. Нужно читать handler data после handler RVA. Там уже будет таблица областей, фильтров и блоков обработки. Поэтому 7.3 не заканчивает тему SEH, а готовит фундамент для 7.6, где мы будем искать SEH-логику в бинаре.

7.3.15. Language-specific handler data: данные, которые Windows не обязана понимать

После handler RVA могут лежать language-specific handler data - данные обработчика, специфичные для языка. Это один из самых важных терминов для понимания x64 SEH. Language-specific означает "зависящий от языка" или "специфичный для языкового runtime-а". Handler data - данные, которые этот handler будет читать.

Windows знает, что эти данные есть, но не обязана понимать их внутренний формат универсально. Формат определяется конкретным handler-ом. Если handler - __C_specific_handler, данные имеют одну модель. Если handler относится к C++ exception handling, модель другая. Если используется другой компилятор или runtime, возможны свои варианты. Поэтому нельзя написать один универсальный парсер "language-specific data" без знания handler-а.

Это очень красивое архитектурное разделение. UNWIND_INFO - общий формат Windows x64. Он говорит, как раскрутить функцию и где находится handler. А language-specific data - частная территория конкретного языка. Windows до неё доходит, передаёт указатель handler-у через dispatcher context, но смысл этих байтов определяется не PE-форматом вообще, а конкретным runtime-обработчиком.

Для будущего static -а это означает: сначала нужно распознать handler. Если handler RVA указывает на __C_specific_handler, можно пытаться интерпретировать handler data как C SEH scope table. Если handler неизвестен, нужно честно сказать: "есть language-specific data, но handler не распознан; автоматическая интерпретация ограничена". Это лучше, чем строить ложную уверенность.

Здесь снова появляется принцип доказательств. Хороший reverse engineering не говорит "я не понимаю, значит вредоносно". Он говорит: "общая unwind-структура валидна; handler RVA указывает туда-то; handler name/symbol/import соответствует или не соответствует известной модели; handler data имеет такой размер и расположение; интерпретация возможна/невозможна по таким причинам".

7.3.16. CHAININFO: когда хвост - это не handler

UNW_FLAG_CHAININFO - один из самых опасных для новичка флагов, потому что он меняет смысл хвоста UNWIND_INFO. Если CHAININFO установлен, после массива UNWIND_CODE и выравнивания лежит не handler RVA, а chained unwind information - цепочная unwind-информация. Обычно это запись, похожая на RUNTIME_FUNCTION, которая указывает на другую unwind-информацию.

Chained переводится как "связанный цепочкой". Идея такая: текущая запись не содержит полностью самостоятельного описания, а ссылается на другую запись/информацию, которая является основной. Это может понадобиться для сложных функций, разделённых участков кода, оптимизаций, shrink wrapping и других ситуаций, где один кусок функции использует общую unwind-карту.

Shrink wrapping - оптимизация, при которой сохранение регистров или выделение части кадра может быть сдвинуто ближе к тому месту, где оно реально нужно, а не обязательно стоять в самом начале функции. Для нас сейчас важен не полный разбор этой оптимизации, а вывод: современные компиляторы не всегда строят функции как идеальный учебный прямоугольник "один пролог - одно тело - один эпилог". Chained info помогает описывать более сложные формы.

Ошибка здесь простая: парсер видит после UNWIND_CODE четыре байта и говорит "это handler RVA". Но если Flags = CHAININFO, это не handler. Это начало chained записи. Значит, вся дальнейшая интерпретация другая. Такой баг может привести к абсурдному выводу: анализатор "нашёл handler" там, где его нет, или пропустил настоящий основной UNWIND_INFO.

```
Правило:
  сначала читаем Flags
  если CHAININFO установлен:
    хвост читаем как chained RUNTIME_FUNCTION-like entry
  иначе если EHANDLER/UHANDLER установлен:
    хвост читаем как handler RVA + handler data
  иначе:
    хвоста handler/chained нет
```

Для фундаментального понимания важно: CHAININFO показывает, что .xdata - не всегда простая линейная структура. Там могут быть ссылки на другие unwind-описания. Поэтому анализ .xdata требует не только чтения одного адреса, но и построения маленького графа зависимостей: текущий RUNTIME_FUNCTION ведёт к UNWIND_INFO; этот UNWIND_INFO может быть основным или chained; chained может вести дальше. Хороший инструмент должен уметь показывать эту цепочку.

7.3.17. Почему .xdata не равна "таблица обработчиков"

После разговора про EHANDLER, UHANDLER и language-specific data может возникнуть другая крайность: думать, что .xdata - это место, где лежат обработчики исключений. Это тоже неверно. .xdata в первую очередь хранит unwind-информацию. Обработчики - только один из возможных дополнительных слоёв.

Функция может иметь .xdata без exception handler-а. Например, она сохраняет RBX и выделяет стек. Ей нужна UNWIND_INFO, чтобы Windows могла восстановить caller context при stack walking или исключении глубже. Но у неё нет __try/__except, нет __finally, нет language-specific handler-а. В таком случае .xdata нужна, но не для обработки исключения на этом кадре, а для перехода выше по стеку.

Функция может иметь EHANDLER, и тогда .xdata действительно ведёт к языковому handler-у. Но даже тогда .xdata сама по себе не содержит весь пользовательский смысл. Она содержит адрес handler-а и данные, которые handler понимает. Пользовательский блок __except может лежать в .text как обычный код, а связь с ним будет описана в language-specific data.

Функция может иметь UHANDLER, связанный с unwind/cleanup. Это тоже не обязательно "поймать исключение". Это может быть "выполнить завершение при выходе". Поэтому корректнее думать так: .xdata - это место, где функция описывает Windows, как с ней обращаться при нестандартном движении по стеку. Иногда это только раскрутка. Иногда раскрутка плюс обработчики. Иногда chained info. Но базовая роль - unwind-контракт.

7.3.18. Ручное чтение одной .xdata-записи: общий алгоритм

Теперь соберём ручную процедуру чтения UNWIND_INFO. Это ещё не код парсера, а мыслительный порядок, чтобы не перепутать слои.

Сначала у нас есть RUNTIME_FUNCTION из .pdata. Мы уже проверили, что текущий RVA попадает в диапазон BeginAddress..EndAddress. Теперь берём UnwindInfoAddress. Это RVA. Мы переводим его в VA, если работаем с живым процессом, или в file offset, если читаем файл.

Дальше читаем первые байты UNWIND_INFO: Version, Flags, SizeOfProlog, CountOfCodes, FrameRegister, FrameOffset. На этом этапе мы ещё не трогаем handler data. Мы только понимаем общий формат и размеры.

Затем читаем CountOfCodes slots массива UNWIND_CODE. Для каждого элемента мы смотрим offset и operation. Если operation требует

дополнительных slots, учитываем это. Важно не выйти за границы массива и не принять дополнительный slot за отдельную независимую операцию.

После этого вычисляем начало хвоста: если CountOfCodes нечётный, добавляем padding slot. Затем смотрим Flags. Если EHANDLER или UHANDLER - читаем handler RVA и отмечаем начало language-specific data. Если CHAININFO - читаем chained RUNTIME_FUNCTION-like entry. Если ничего - на этом структура заканчивается.

```
Ручной порядок:
1. Взять UnwindInfoAddress из RUNTIME_FUNCTION.
2. Помнить: это RVA.
3. Перевести RVA в VA или file offset.
4. Прочитать Version и Flags.
5. Прочитать SizeOfProlog, CountOfCodes, FrameRegister/FrameOffset.
6. Прочитать CountOfCodes UNWIND_CODE slots.
7. Учесть дополнительные slots внутри операций.
8. Учесть padding до правильной границы.
9. По Flags решить, что означает хвост.
10. Связать результат с реальным прологом функции.
```

Последний пункт особенно важен. Прочитать .xdata - это ещё не значит понять функцию. Нужно вернуться к .text и посмотреть пролог. Если UNWIND_INFO говорит, что функция сохраняет RBX, в прологе должен быть соответствующий эффект. Если говорит, что выделяет 0x80 байт, код должен делать совместимое изменение RSP. Если говорит, что есть frame register, код должен это объяснять. Поэтому ручной анализ .xdata всегда двунаправленный: из .pdata в .xdata и обратно в код.

7.3.19. Как .xdata участвует в search phase

Search phase - фаза поиска обработчика. Мы уже разбирали её в главе 6, но теперь нужно связать её с .xdata. Когда исключение произошло, Windows идёт по кадрам. Для текущего кадра она по RIP находит RUNTIME_FUNCTION, затем UNWIND_INFO. Если текущий ControlPc находится в теле функции и Flags указывают EHANDLER, Windows может вызвать language-specific handler.

То есть .xdata не просто помогает перейти к caller-у. Она ещё говорит, есть ли у текущего кадра обработчик, который имеет смысл спросить во время search phase. Если EHANDLER не установлен, Windows не будет вызывать language-specific handler для обработки исключения в этом кадре. Она просто использует unwind-информацию, чтобы подняться выше, если нужно.

Если EHANDLER установлен, Windows берёт handler RVA из хвоста UNWIND_INFO, превращает его в VA через ImageBase, подготавливает параметры вроде ExceptionRecord, EstablisherFrame, ContextRecord, DispatcherContext и вызывает handler. Handler смотрит language-specific data

и возвращает решение: продолжать поиск, обработать, иногда продолжить выполнение.

Здесь видно, почему .pdata и .xdata неразрывны. .pdata помогает найти текущий кадр как диапазон кода. .xdata помогает понять, что с этим кадром делать: можно ли спросить handler, как раскрутить кадр, где искать дополнительные данные. Без .pdata мы не найдём UNWIND_INFO. Без .xdata .pdata будет только картой без инструкций.

7.3.20. Как .xdata участвует в unwind phase

Unwind phase - фаза раскрутки. Она начинается после того, как найден обработчик или когда система должна выйти из части кадров по особым правилам. В этой фазе .xdata становится ещё важнее, потому что теперь нужно не только спрашивать "кто обработает?", но и реально выполнять действия выхода: восстанавливать контекст, вызывать termination handlers, проходить промежуточные кадры.

Если у кадра есть UHANDLER, он может быть вызван во время unwind phase. Это может соответствовать finally-логике или cleanup-действиям. Например, функция вошла в защищённую область, открыла ресурс, и при исключении нужно выполнить завершающий код. Этот завершающий код не обязательно "ловит" исключение, но он должен выполниться, когда кадр покидается необычным путём.

UNWIND_CODE при этом нужен для восстановления машинного состояния. Если функция сохранила nonvolatile-регистры, их нужно вернуть. Если функция выделила стек, RSP нужно поднять. Если использовался frame register, нужно учесть его связь с кадром. Всё это делает RtlVirtualUnwind или внутренняя логика unwinder-а, опираясь на .xdata.

Таким образом, одна и та же структура участвует в двух разных смыслах. В search phase она говорит, есть ли кого спросить. В unwind phase она говорит, как выйти из кадра и нужно ли выполнить cleanup. Если смешать эти фазы, можно неправильно представить механизм как "сразу прыгнули в обработчик". На самом деле сначала система ищет, потом раскручивает, и .xdata нужна в обоих движениях.

7.3.21. .xdata и отладчик: почему stack trace зависит от этих байтов

Когда мы говорим про .xdata, нельзя ограничиваться исключениями. Microsoft x64 unwind data нужна не только для exception handling, но и для debugger support. Debugger support - поддержка отладчика. Отладчик должен уметь построить call stack, показать цепочку вызовов, восстановить

значения некоторых регистров, понять, где находится caller.

Если stack trace в отладчике выглядит красиво, это не потому, что отладчик "угадывает имена функций". Он использует символы, модули, PE-таблицы, .pdata, .xdata и текущий контекст. Для x64 нормальный stack walking опирается на unwind metadata. Если metadata корректна, отладчик может подняться от кадра к кадру. Если metadata повреждена или отсутствует там, где она нужна, stack trace может оборваться или стать ложным.

Stack trace - трасса стека, список активных вызовов. В crash dump он особенно важен, потому что процесс уже не работает. У анализатора есть только память, регистры и модули. Если .xdata даёт правильную инструкцию по раскрутке, можно восстановить историю. Если .xdata врёт, история искажается. Поэтому для reverse engineering .xdata - это не только тема "исключений", но и тема доверия к любой диагностике.

Для будущего static -а это означает: проверка .xdata - это проверка качества stack walking. Если инструмент говорит "stack trace ненадёжен", он должен объяснить почему: нет RUNTIME_FUNCTION, UnwindInfoAddress вне image, Version неизвестна, CountOfCodes выходит за границы, handler/chained хвост смещён неправильно, UNWIND_CODE не соответствует прологу. Тогда вывод будет не мистическим, а доказательным.

7.3.22. .xdata и безопасность: почему эти данные любят ломать упаковщики и защиты

Упаковщики, dynamic s и обфускаторы часто работают не только с кодом, но и с метаданными. Packer - упаковщик, программа или слой защиты, который упаковывает, шифрует, сжимает или перестраивает исполняемый файл, чтобы затруднить анализ. dynamic - защитник, более общий слой, который может добавлять антидебаг, антидамп, виртуализацию, проверку целостности, скрытые переходы и другие техники.

Почему .xdata интересна для таких инструментов? Потому что она помогает анализатору понимать функции. Если .pdata и .xdata аккуратны, дизассемблеру, отладчику и crash analyzer-у проще строить границы функций и stack trace. Если эти данные отсутствуют, повреждены, минимизированы или странно связаны, анализ становится труднее. Но здесь нужно быть осторожным: странная .xdata не всегда означает злой умысел. Это может быть JIT, ручной asm, нестандартный компилятор, оптимизации, hot/cold splitting, chained info или просто особенности линкера.

Правильный подход - не эмоциональный, а контрактный. Мы спрашиваем: соответствует ли UnwindInfoAddress границам image? Версия поддерживается? CountOfCodes разумен? Хвост читается по Flags? Handler

RVA указывает в исполняемую область? Language-specific data лежит в читаемой области? UNWIND_CODE согласуется с прологом? Если ответы плохие, это finding. Но finding должен быть описан как нарушение конкретного слоя, а не как автоматический вердикт "malware".

Это снова показывает, зачем мы так долго строим фундамент. Без теории человек видит "байты странные". С теорией он видит: "нарушен такой-то участок договора между компилятором, линкером, loader-ом, Windows unwinder-ом и отладчиком". Это гораздо сильнее и профессиональнее.

7.3.23. Типичные ошибки при чтении .xdata

Первая ошибка - читать UnwindInfoAddress как file offset. Мы уже повторяли это несколько раз, но именно эта ошибка настолько частая, что её нужно закрепить. UnwindInfoAddress - RVA. Для живого процесса прибавляем ImageBase. Для файла переводим через section table.

Вторая ошибка - думать, что .xdata всегда находится в секции с именем .xdata. Обычно да, но имя секции не является строгим контрактом. Формально важен RVA и section mapping. Если Exception Directory и RUNTIME_FUNCTION ведут в область с другим именем, это может быть нестандартно, но парсер должен сначала проверить адресную валидность, а не просто падать.

Третья ошибка - считать CountOfCodes количеством инструкций. Это количество UNWIND_CODE slots. Некоторые операции занимают дополнительные slots. Некоторые инструкции пролога вообще не попадают в unwind-информацию. Сравнивать нужно смысл, а не количество строк.

Четвёртая ошибка - забыть padding после нечётного CountOfCodes. Тогда handler RVA или chained info читаются со смещением на два байта, и весь хвост становится мусором.

Пятая ошибка - принять CHAININFO за handler. Если установлен CHAININFO, хвост не является handler RVA. Это chained entry. Сначала Flags, потом интерпретация хвоста.

Шестая ошибка - принять __C_specific_handler за пользовательский обработчик. Это language-specific handler, посредник. Пользовательская логика находится в filter/handler blocks, связь с которыми описана в language-specific data.

Седьмая ошибка - думать, что отсутствие EHANDLER означает отсутствие важности .xdata. Даже без handler-а UNWIND_INFO нужна для stack walking и восстановления caller context.

Восьмая ошибка - не сверять .xdata с реальным прологом. Структура может быть формально читаемой, но смыслово не совпадать с кодом. Для reverse engineering важна именно согласованность слоёв.

7.3.24. Как будущий static должен объяснять .xdata человеку

Если мы думаем о будущем static -е, он не должен просто печатать сырые поля. Сырые поля полезны, но не объясняют человеку механизм. Хороший вывод должен идти в той же последовательности, в которой мы строим понимание.

Сначала: откуда пришли. Например: "RUNTIME_FUNCTION для диапазона RVA 0x1200..0x1350 указывает UnwindInfoAddress = 0x3000". Затем: куда это попадает. "RVA 0x3000 находится в секции .xdata, file offset такой-то, область readable". Затем: что прочитано. "UNWIND_INFO version 1, flags EHANDLER, prolog size 0x0A, CountOfCodes 3". Затем: что это значит. "Функция сохраняет RBX, выделяет 0x40 байт стека, имеет language-specific handler". Затем: кто handler. "Handler RVA указывает на __C_specific_handler". Затем: что дальше. "После handler RVA начинается language-specific data, вероятно scope table".

Такой вывод отличается от простого дампа полей. Он показывает цепочку доказательств. Человек видит не только числа, но и смысл: где данные лежат, почему они считаются валидными, какую роль играют, как связаны с кодом и что нужно проверить следующим шагом.

```
Плохой вывод:
  Flags=1 Count=3 Handler=0x5000

Хороший вывод:
  UNWIND_INFO имеет EHANDLER. Значит после массива UNWIND_CODE
  и padding должен лежать RVA language-specific handler-а.
  RVA 0x5000 указывает в .text на __C_specific_handler.
  Поэтому следующие байты интерпретируются не PE-форматом вообще,
  а правилами MSVC C SEH scope data.
```

Именно такой стиль нам нужен в книге. Мы не просто учим поля. Мы учим говорить о бинаре как о системе взаимосвязанных слоёв. Это потом позволит писать инструмент, отчёты, разборы крашей и объяснения для других людей.

7.3.25. Мини-сцена: функция с обычным стековым кадром

Представим простую функцию, которая сохраняет RBX и выделяет место на стеке:

```
func:
    push rbx
    sub  rsp, 40h
    mov  rbx, rcx
    call other
    add  rsp, 40h
    pop  rbx
    ret
```

Что важно для UNWIND_INFO? Не вся функция. mov rbx, rcx - логика функции, но для восстановления caller context важен факт, что RBX был сохранён старым значением через push. call other - важен для обычного выполнения, но UNWIND_INFO текущей функции не описывает все внутренние call. add/pop/ret - эпилог, который должен быть распознаваемым, но UNWIND_CODE обычно описывает действия пролога.

UNWIND_INFO должен рассказать примерно такую историю: в прологе сохранён nonvolatile RBX; выделено 0x40 байт stack allocation; пролог имеет такой-то размер; frame register не используется; handler-а нет. Тогда если исключение произошло внутри call other или в теле после пролога, Windows сможет восстановить caller context: вернуть RSP, восстановить RBX из стека, взять return address и перейти к вызывающему кадру.

Если исключение произошло на инструкции sub rsp, 40h, ситуация тоньше: push rbx уже мог выполниться, а allocation ещё нет или как раз выполняется. Поэтому offset-ы UNWIND_CODE и SizeOfProlog нужны для частичного анализа. Вот почему структура хранит не только "какие действия", но и "где в прологе они произошли".

7.3.26. Мини-сцена: функция с EHANDLER

Теперь представим функцию с __try/__except. В исходном коде программист видит защищённый блок и обработчик. Но в x64 бинаре связь не обязана выглядеть как обычная стрелка control flow. Компилятор создаёт код защищённой области, код filter-а, код handler-блока и metadata, которая связывает их.

В .pdata будет RUNTIME_FUNCTION для функции. В .xdata будет UNWIND_INFO. В Flags будет EHANDLER. После UNWIND_CODE и padding будет handler RVA, часто указывающий на __C_specific_handler. После него будут language-specific data, которые __C_specific_handler понимает. Эти данные описывают, какие диапазоны кода защищены, какой filter связан с диапазоном, куда передавать управление при принятии исключения.

То есть маршрут такой:

```
исключение внутри guarded range
  -> Windows находит RUNTIME_FUNCTION
```

```
-> читает UNWIND_INFO
-> видит EHANDLER
-> вызывает __C_specific_handler
-> __C_specific_handler читает handler data
-> filter решает: search / execute / continue
-> если execute: начинается нужная unwind/transfer logic
```

Эта сцена показывает, почему .xdata находится в центре между PE и SEH. Без .xdata Windows не узнает, что у функции есть language-specific handler. Без language-specific data __C_specific_handler не узнает, какие __try/__except относятся к текущему RIP. Без .pdata вообще не будет найдено нужное UNWIND_INFO. Всё связано.

7.3.27. Мини-сцена: chained info и осторожность анализа

Теперь представим более сложный случай: RUNTIME_FUNCTION ведёт к UNWIND_INFO, а в Flags стоит CHAININFO. Новичок может ожидать handler, но его там нет. Хвост структуры содержит связанную запись. Эта запись может вести к основной unwind-информации. Значит, анализ должен сделать ещё один переход.

Практически это означает, что простая модель "один RUNTIME_FUNCTION - один UNWIND_INFO - один набор кодов" иногда недостаточна. Нужно быть готовым к цепочке. При этом цепочка тоже должна быть валидной: адреса должны попадать в image, не должно быть бесконечного цикла, конечная запись должна привести к основной unwind-информации без CHAININFO или к допустимой структуре.

Для static -a chained info - это отдельный слой отчёта. Он должен не просто спрятать сложность, а показать: "данная unwind-запись является chained; основной unwind info находится там-то; итоговые правила раскрутки берутся оттуда". Тогда человек не будет думать, что данные пропали или handler отсутствует случайно.

7.3.28. Почему теория .xdata готовит нас к UWOP_*

Следующий раздел будет посвящён сверке UNWIND_CODE с прологом и конкретным UWOP_*. UWOP означает unwind operation - операция раскрутки. Это конкретные коды вроде UWOP_PUSH_NONVOL, UWOP_ALLOC_SMALL, UWOP_ALLOC_LARGE, UWOP_SET_FPREG и другие. Но разбирать их без текущего раздела было бы рано.

Почему? Потому что UWOP_* имеют смысл только внутри общей структуры. UWOP_PUSH_NONVOL не просто "код 0" или "push регистра". Это заявление компилятора: в прологе сохранён nonvolatile-регистр, и при unwind его

нужно восстановить из стека. UWOP_ALLOC_SMALL не просто "выделили стек". Это часть ответа на вопрос, как из текущего RSP вернуться к RSP caller-a. UWOP_SET_FPREG не просто "установили RBP". Это связь frame register с layout-ом кадра.

Если сначала понять .xdata как систему, UWOP_* станут не списком команд, а маленькими кирпичами общей модели. Каждый UWOP отвечает на вопрос: какое действие пролога влияет на восстановление caller context и как его закодировать? Именно поэтому следующий раздел должен будет постоянно возвращаться к тому, что мы сделали здесь: .pdata нашла функцию, UnwindInfoAddress привёл в .xdata, UNWIND_INFO дал структуру, CountOfCodes указал массив, а UWOP_* объясняют конкретные действия пролога.

7.3.29. Итог раздела

.xdata - это не мусорная секция и не просто приложение к .pdata. Это место, где обычно лежит UNWIND_INFO - машинно-читаемая инструкция по раскрутке кадра функции. .pdata даёт короткую карту диапазонов функций, а .xdata даёт подробности: как функция построила стековый кадр, какие nonvolatile-регистры сохранила, использует ли frame register, есть ли language-specific handler, есть ли chained unwind info.

UNWIND_INFO начинается с Version и Flags. Version говорит, по какой версии формата читать структуру. Flags меняют смысл хвоста: нет handler-a, есть exception handler, есть unwind/termination handler, или есть chained info. SizeOfProlog нужен, чтобы отличать пролог от тела и корректно обрабатывать исключения внутри пролога. CountOfCodes задаёт количество UNWIND_CODE slots. FrameRegister и FrameOffset описывают стабильную базу кадра, если функция её использует.

UNWIND_CODE - это компактный язык описания эффектов пролога, важных для unwind. Он не обязан совпадать с дизассемблером один к одному. Он описывает не весь код, а только то, что нужно для восстановления caller context: сохранение nonvolatile-регистров, выделение стека, установку frame register и похожие действия.

После массива UNWIND_CODE нужно учитывать выравнивание. Затем хвост читается строго по Flags. При EHANDLER/UHANDLER там лежит handler RVA и language-specific data. При CHAININFO - chained RUNTIME_FUNCTION-like entry. Ошибка в этой точке ломает всё дальнейшее понимание.

Главная связка раздела такая: RUNTIME_FUNCTION не раскручивает функцию сам. Он только ведёт к UNWIND_INFO. UNWIND_INFO не является исходным кодом функции. Он описывает unwind-значимые эффекты пролога. Handler

RVA не является пользовательским __except-блоком. Он часто указывает на language-specific handler, который уже читает свои данные. Так строится фундамент: .pdata даёт карту, .xdata даёт правила, Windows получает caller context, а реверсер проверяет согласованность всех слоёв.

Следующий раздел естественно продолжит эту линию. Мы уже знаем, где лежит массив UNWIND_CODE и зачем он нужен. Теперь нужно медленно разобрать, как конкретные UWOP_* соответствуют реальным инструкциям пролога: push nonvolatile-регистра, выделение места на стеке, сохранение регистра по смещению, установка frame register, большие allocation-ы и типичные ошибки сопоставления.

Техническая база раздела 7.3

Раздел опирается на официальную документацию Microsoft по x64 exception handling, где описаны UNWIND_INFO, UNWIND_CODE, Flags, exception handler data, chained unwind information и связь .pdata/.xdata с PE32+ image; на документацию Microsoft PE Format, где описаны PE/COFF, sections, Data Directory, RVA и Exception Directory; на документацию RtlLookupFunctionEntry, где функция определяется как поиск записи в active function tables по PC value; и на документацию RtlVirtualUnwind, где описано получение invocation context вызывающей функции. Текст раздела написан как учебное объяснение и связывает эти технические источники с практической моделью reverse engineering.

7.4. Как сверять UNWIND_CODE с прологом функции: когда код и .xdata рассказывают одну историю

В предыдущем разделе мы научились смотреть на .xdata не как на безымянный хвост после .pdata, а как на место, где лежит настоящая инструкция для unwind. Но пока эта инструкция всё ещё может казаться отдельной от кода. Есть функция в .text, есть структура UNWIND_INFO в .xdata, есть массив UNWIND_CODE, есть какие-то UWOP_*. Новичок часто читает это как два несвязанных мира: дизассемблер отдельно, метаданные отдельно. Это опасная ошибка.

В Windows x64 эти два мира должны рассказывать одну и ту же историю. Машинный пролог функции реально меняет стек и сохраняет регистры. UNWIND_CODE описывает эти изменения в табличной форме. Пролог - это действие. UNWIND_CODE - это запись о действии. Если действие и запись расходятся, то RtlVirtualUnwind, отладчик, crash dump analyzer и обработка исключений могут начать восстанавливать не тот caller context.

Поэтому этот раздел нельзя писать как короткий справочник вида “UWOP_PUSH_NONVOL значит push, UWOP_ALLOC_SMALL значит sub rsp”. Так запомнить можно, но понять нельзя. Нам нужно медленно построить связь: что именно делает пролог, почему Windows не хочет дизассемблировать произвольную функцию во время exception dispatch, как компилятор превращает прологовые действия в компактные unwind-коды, почему эти коды отсортированы не так, как код читается глазами, и как реверсер должен сверять одно с другим.

Главная мысль раздела такая: UNWIND_CODE - это не копия инструкций функции. Это минимальное формальное описание тех эффектов пролога, которые нужны, чтобы отменить кадр функции и восстановить состояние вызывающего кода. Поэтому сверять нужно не “инструкция к инструкции”, а “машинный эффект к unwind-операции”.

7.4.1. Почему мы вообще сверяем метаданные с кодом

Начнём с вопроса, который кажется слишком простым: зачем сверять .xdata с прологом, если .xdata уже создана компилятором? Раз компилятор сгенерировал, значит можно доверять? Для обычного нормального приложения - чаще всего да. Но reverse engineering начинается именно там, где мы не имеем права наивно доверять одному источнику.

Есть исполняемый код. Он лежит в .text или в другой executable-области. Этот код реально выполняет процессор. Если функция делает push rbx, то RSP реально уменьшается на 8, а старое значение RBX реально оказывается на стеке. Если функция делает sub rsp, 40h, то вершина стека реально сдвигается вниз на 0x40 байт. Это физическое состояние выполнения.

Есть unwind-метаданные. Они лежат обычно в .xdata. Эти данные не выполняются процессором как инструкции. Они читаются системным unwinder-ом, отладчиком и механизмом обработки исключений. Если в UNWIND_INFO записано, что функция выделила 0x40 байт стека, то RtlVirtualUnwind будет использовать это как правило восстановления caller-а.

Если код и метаданные совпадают, всё хорошо: реальная функция построила кадр, а unwind знает, как этот кадр разобрать. Если код и метаданные не совпадают, появляется конфликт между реальностью выполнения и официальной картой. Такой конфликт может быть случайной ошибкой парсера, результатом нестандартной компиляции, обфускацией, повреждением файла, ручным ассемблером или намеренным трюком.

Для будущего static -а это фундаментальный принцип. Инструмент не должен просто вывести:

```
UWOP_ALLOC_SMALL, size = 0x40
```

Он должен спросить: а в прологе действительно есть действие, которое создаёт фиксированную стековую аллокацию 0x40? Где эта инструкция? До неё или после неё сохранены nonvolatile-регистры? Не находится ли RIP в прологе? Совпадает ли SizeOfProlog с границей, где кадр уже построен? Есть ли FrameRegister, и действительно ли код его устанавливает?

Иными словами, анализ unwind-данных - это не чтение таблицы ради таблицы. Это проверка договора между машинным кодом и Windows x64 exception/unwind model.

7.4.2. Prolog: не “начало функции”, а место построения кадра

Слово prolog, или по-русски пролог функции, мы уже вводили. Но теперь нужно сделать его ещё точнее. В бытовом чтении дизассемблера прологом часто называют первые несколько инструкций функции. Это удобно, но слишком грубо. Для Windows x64 пролог - это область, где функция выполняет действия, которые строят её stack frame, то есть кадр стека.

Stack frame, кадр стека, - это состояние одного активного вызова функции: выделенная область стека, сохранённые nonvolatile-регистры, место под локальные данные, иногда frame pointer, иногда security cookie, иногда home space/temporary spill slots. Пролог - это не украшение и не шаблон из учебника. Это момент, когда функция превращает “я только что была вызвана” в “у меня теперь есть рабочее место”.

На Windows x64 пролог может выполнять несколько типов действий. Он может сохранять nonvolatile-регистры. Nonvolatile register - сохраняемый регистр по соглашению вызовов: если callee хочет использовать такой регистр, она обязана вернуть caller-у прежнее значение. Пример - RBX, RBP, RDI, RSI, R12, R13, R14, R15.

Пролог может выделять фиксированное место на стеке. Это обычно видно как sub rsp, N. Такая инструкция не “создаёт переменные” напрямую. Она просто уменьшает RSP. Но по соглашению область между новым и старым положением RSP теперь принадлежит текущей функции.

Пролог может установить frame register, то есть регистр кадра. Чаще всего это RBP, но в x64 это не священное правило. Frame register нужен, когда функции удобнее иметь стабильную точку внутри кадра, особенно если RSP может дополнительно меняться или если компилятор хочет обращаться к разным частям fixed allocation короткими смещениями.

Пролог может сохранить XMM-регистры. XMM-регистры - это SIMD-регистры, используемые для floating-point и векторных данных. Некоторые XMM-регистры на

Windows x64 относятся к nonvolatile-состоянию и должны быть восстановлены, если функция их использует.

Пролог может вызвать __chkstk. Это helper-функция для большой стековой аллокации. Если функция выделяет большой блок на стеке, Windows должна корректно “пробить” guard pages стека, чтобы стек расширился постепенно. Поэтому компилятор может сначала положить размер в RAX, вызвать __chkstk, и только затем уменьшить RSP.

Всё это важно потому, что UNWIND_CODE описывает не “красивый синтаксис пролога”, а именно эти эффекты. Если ты смотришь на код и видишь в начале функции обычные вычисления, которые не меняют RSP и не сохраняют nonvolatile-состояние, они могут не иметь отражения в unwind-кодах. Если ты видишь push rbx, это уже действие, которое должно быть отражено, потому что без него нельзя правильно восстановить RBX caller-а при unwind.

7.4.3. Почему Windows не хранит полный дизассемблированный пролог

Можно спросить: почему .xdata не хранит просто список инструкций пролога? Например: push rbx, sub rsp, 40h, lea rbp, [rsp+20h]. Тогда всё было бы понятно.

Проблема в том, что Windows не нужен полный текст инструкций. Ей нужен формальный, компактный и быстрый рецепт восстановления. Во время exception dispatch система работает в уже плохой ситуации: возникло исключение, нужно быстро и надёжно найти обработчик или построить caller context. В такой момент нельзя превращать каждую функцию в полноценный дизассемблируемый объект с анализом всех вариантов инструкций. Нужен заранее подготовленный формат.

UNWIND_CODE - это такой формат. Он говорит не “в прологе была именно такая машинная инструкция”, а “к моменту выполнения функции произошло такое изменение unwind-состояния”. Например, UWOP_ALLOC_SMALL сообщает, что была выполнена небольшая фиксированная аллокация стека. Для unwind не важно, как программист называл локальные переменные. Не важно, почему компилятор выбрал именно этот размер. Важно: чтобы подняться к caller-у, нужно вернуть RSP назад на этот размер.

То же самое с сохранением регистра. Если функция сохранила RBX, unwind должен знать, где лежит старое значение RBX. Ему не нужен исходный комментарий “сохраняем указатель на объект”. Ему нужна механика: какой регистр, каким способом сохранён, по какому смещению от базы кадра его искать.

Поэтому unwind-информация является абстракцией над кодом. Она ближе к “эффектам пролога”, чем к “байтам инструкций”. Реверсер должен уметь читать и то, и другое, а потом связывать.

7.4.4. UNWIND_CODE: маленький элемент с большим смыслом

UNWIND_CODE - это элемент массива внутри UNWIND_INFO. В документации Microsoft он описан как компактная запись операции пролога. С точки зрения структуры у него есть три важных части: CodeOffset, UnwindOp, OpInfo.

CodeOffset - смещение внутри пролога. Слово offset переводится как смещение. Оно отвечает на вопрос: где в прологе находится действие, которое описывает эта unwind-запись? Это не RVA всей функции и не file offset. Это маленькое смещение от начала функции, связанное с конкретной прологовой операцией.

UnwindOp - код операции unwind. Это не opcode процессора. Opcode процессора - машинный код инструкции вроде push, sub, lea. UnwindOp - код в формате Windows unwind metadata, например UWOP_PUSH_NONVOL, UWOP_ALLOC_SMALL, UWOP_ALLOC_LARGE, UWOP_SET_FPREG.

OpInfo - дополнительная информация операции. Его смысл зависит от UnwindOp. Для UWOP_PUSH_NONVOL это номер регистра. Для UWOP_ALLOC_SMALL это часть формулы размера. Для некоторых операций он зарезервирован или выбирает вариант формата.

Это очень частая точка путаницы. Новичок видит “op” и думает: это почти инструкция. Нет. UnwindOp - это не инструкция CPU. Это запись в метаданных. Она описывает эффект, который CPU-инструкция уже произвела или должна быть учтена при virtual unwind.

В виде структуры это можно представить так:

```
typedef union _UNWIND_CODE {
    struct {
        unsigned char CodeOffset;
        unsigned char UnwindOp : 4;
        unsigned char OpInfo   : 4;
    };
    unsigned short FrameOffset;
} UNWIND_CODE;
```

Эта C-форма важна не для зубрёжки, а для понимания, почему один UNWIND_CODE slot занимает 2 байта. Но не каждая unwind-операция занимает только один slot. Например, операция может требовать дополнительного slot-а для offset-а или размера. Поэтому CountOfCodes в UNWIND_INFO - это количество двухбайтовых slots, а не количество “понятных человеку действий”.

7.4.5. Почему массив UNWIND_CODE отсортирован “не по-человечески”

Когда ты читаешь пролог глазами, ты идёшь сверху вниз:

```
push rbx
push rsi
sub  rsp, 40h
lea  rbp, [rsp+20h]
```

Кажется естественным ожидать, что UNWIND_CODE будет идти в таком же порядке. Но Microsoft x64 unwind procedure использует массив unwind-кодов, отсортированный в descending order, то есть в порядке убывания. Это значит, что записи идут от более поздних смещений пролога к более ранним. В документации прямо указано, что unwind code array sorted into descending order.

Зачем так? Смысл становится понятен, если вспомнить partial prolog unwinding. Partial значит частичный. Если исключение произошло внутри пролога, то пролог ещё не успел выполниться полностью. Например, функция уже сделала push rbx, но ещё не сделала sub rsp, 40h. Unwinder должен отменить только реально выполненные действия.

Чтобы понять, какие действия уже произошли, используется CodeOffset. Если RIP находится на некотором смещении от начала функции, unwinder выбирает unwind-записи, относящиеся к операциям, которые уже были выполнены к этой точке. Порядок массива помогает эффективно пройти нужную часть.

Для реверсера практический вывод простой: не пытайся читать UNWIND_CODE как прямой листинг пролога сверху вниз. Сначала восстанови смысл каждой записи, потом сопоставь её с конкретной инструкцией по CodeOffset, а затем уже проверь общую историю кадра.

Важно: “порядок в массиве” и “порядок выполнения инструкций” - разные вещи. Они связаны, но не совпадают один к одному. Если это забыть, можно решить, что unwind-коды “перепутаны”, хотя они просто записаны в формате, удобном для unwinder-а.

7.4.6. UWOP_PUSH_NONVOL: сохранение nonvolatile-регистра через push

Начнём с самой наглядной операции: UWOP_PUSH_NONVOL.

UWOP можно читать как UnWind OPeration, то есть операция unwind. Это неофициальная расшифровка для памяти, но она помогает: UWOP_* - это коды действий, значимых для раскрутки стека.

PUSH - машинное действие помещения значения на стек. В x64 инструкция push rbx уменьшает RSP на 8 и записывает старое значение RBX по новому адресу [RSP].

NONVOL - сокращение от nonvolatile, то есть сохраняемый по calling convention регистр. Если функция использует RBX, она обязана вернуть caller-у прежний RBX. Один простой способ - сделать push rbx в прологе и pop rbx в эпилоге.

Код:

```
push rbx
```

Смысл выполнения:

```
RSP = RSP - 8
[RSP] = old RBX
```

Смысл для unwind:

```
Чтобы восстановить caller context,
нужно взять старый RBX со стека
и вернуть RSP назад на 8 байт.
```

Именно это описывает UWOP_PUSH_NONVOL. В OpInfo хранится номер регистра. Для RBX это один номер, для RBP другой, для R12 - другой. Регистровая карта нужна потому, что в двухбайтовом slot-е нельзя хранить строку “RBX”. Там хранится маленькое число.

Если в прологе есть push rbx, а в unwind-кодах нет UWOP_PUSH_NONVOL RBX, это подозрительно. Но нужно быть аккуратным: возможно, ты неправильно определил начало функции, читаешь не тот UNWIND_INFO, находишься в chained info или рассматриваешь не настоящий пролог. Хороший анализатор должен не сразу кричать “сломано”, а показать несоответствие и возможные объяснения.

Типичная нормальная пара:

```
; prolog
push rbx
```

```
sub  rsp, 20h

UNWIND_CODE:
  UWOP_ALLOC_SMALL, size = 0x20
  UWOP_PUSH_NONVOL, reg = RBX
```

Человечески: функция сохранила RBX, потом выделила место на стеке. Метаданные говорят то же самое: восстановить нужно фиксированную аллокацию и сохранённый RBX.

7.4.7. Почему push nonvolatile должен быть именно про nonvolatile

Здесь важно не пропустить слово nonvolatile. Если функция в прологе делает push rax, это не обязательно должно быть UWOP_PUSH_NONVOL, потому что RAX - volatile-регистр. Volatile register - регистр, который callee имеет право испортить. Caller не ждёт, что значение RAX переживёт вызов, если только оно не является return value в конце.

Но иногда компилятор или ручной asm может временно пушить volatile-регистр ради выравнивания, временного хранения или другой цели. Для unwind это не “сохранение caller state”. Однако изменение RSP всё равно может иметь значение. В MASM-директивах Microsoft для push volatile-регистра предлагается использовать allocation, а не .PUSHREG, потому что .PUSHREG именно про nonvolatile integer register.

Это место важно для теории. UNWIND_CODE интересуется не всем, что лежит на стеке. Он интересуется тем, что нужно для восстановления caller context. Caller context - состояние вызывающей функции: её RSP, её return address, её nonvolatile-регистры, которые должны быть такими, как будто callee корректно вернулась.

Если на стеке есть временный scratch, unwinder не обязан знать его смысл. Если на стеке лежит старый RBX, unwinder обязан знать, потому что иначе caller получит неправильный RBX после virtual unwind.

7.4.8. UWOP_ALLOC_SMALL: небольшая фиксированная аллокация стека

Следующая базовая операция - UWOP_ALLOC_SMALL.

ALLOC - allocation, выделение. В контексте стека это не malloc и не heap. Это именно уменьшение RSP, чтобы текущая функция получила место для своего кадра. SMALL - маленькая форма кодирования. В Windows x64 unwind metadata небольшие аллокации от 8 до 128 байт можно закодировать очень компактно.

Типичная инструкция:

```
sub rsp, 40h
```

Смысл выполнения:

```
RSP = RSP - 0x40
```

Смысл для функции:

```
Функция получила 0x40 байт фиксированного стекового пространства.
```

Смысл для unwind:

```
Чтобы выйти к caller context,
нужно вернуть RSP назад на 0x40.
```

UWOP_ALLOC_SMALL кодирует размер по формуле:

```
allocation_size = OpInfo * 8 + 8
```

Если OpInfo = 7, размер будет 7 * 8 + 8 = 64, то есть 0x40.

Почему размер кратен 8? Потому что x64 stack values и сохранённые адреса/регистры обычно работают с 8-байтовыми единицами, а сам стек поддерживается в рамках ABI-выравнивания. Поэтому нет смысла тратить байты формата на произвольный размер 37 байт. Нормальная фиксированная stack allocation будет кратна нужным границам.

Для реверсера важно не только вычислить размер. Важно понять, почему именно такой размер появился. sub rsp, 40h может покрывать локальные переменные, spill slots, alignment, temporary area, security cookie, home space для будущих вызовов, сохранение XMM-регистров через movdqa, и другие нужды. Unwind metadata не объясняет, какая часть этих 0x40 относится к какой переменной. Она говорит только: кадр стал глубже на 0x40, при unwind это надо отменить.

7.4.9. UWOP_ALLOC_LARGE: большая аллокация и почему тут два формата

Если выделение стека больше 128 байт, UWOP_ALLOC_SMALL уже не подходит. Тогда используется UWOP_ALLOC_LARGE.

LARGE - большой. Но “большой” здесь не обязательно гигантский. Это просто размер, который не влезает в компактную small-форму. У UWOP_ALLOC_LARGE есть две формы.

Первая форма используется, когда размер ещё достаточно умеренный. В дополнительном slot-е хранится размер, делённый на 8. Это экономит место, потому что размер всё равно кратен 8. Такая форма покрывает большие, но не огромные fixed allocations.

Вторая форма используется для ещё более крупных аллокаций. Тогда размер хранится в двух дополнительных slots уже как unscaled value, то есть без деления на 8. Это нужно для очень больших стековых кадров.

С точки зрения реверсера важно не перепутать CountOfCodes. Одна человеческая операция UWOP_ALLOC_LARGE может занять два или три двухбайтовых slot-а. Если парсер считает “один UWOP - один slot”, он съедет и начнёт читать хвост UNWIND_INFO не с того места. Тогда handler RVA, chained info или language-specific data будут интерпретированы как мусор.

Пример:

```
sub rsp, 200h
```

Смысл:

```
Функция выделила 0x200 байт фиксированного стекового пространства.
```

В unwind metadata это, скорее всего, будет UWOP_ALLOC_LARGE с дополнительным slot-ом, где хранится масштабированный размер.

Но снова важно: UWOP_ALLOC_LARGE не говорит “у функции был массив 512 байт”. Он говорит только про эффект на RSP. Массив, временные данные, alignment, сохранённые регистры - это уже дальнейший анализ кода и layout-а кадра.

7.4.10. __chkstk: почему большая аллокация может выглядеть не как простой sub

Большая stack allocation на Windows x64 может включать вызов __chkstk. Это имя часто пугает новичка: будто какая-то скрытая функция проверки безопасности. На самом деле смысл проще и глубже одновременно.

Стек в Windows растёт постепенно. Рядом с текущей границей стека есть guard page, то есть страница-сторож. Когда поток обращается к guard page, система понимает, что стек нужно расширить. Если функция сразу резко уменьшит RSP на очень большой размер и потом начнёт писать далеко ниже старого стека, она может перепрыгнуть guard page и попасть в невыделенную область. Поэтому большие аллокации должны “пробиваться” корректно.

__chkstk - helper, который пробует страницы будущего стекового диапазона, чтобы Windows расширила стек по правилам. В прологе это может выглядеть так:

```
mov rax, 3000h
call __chkstk
sub rsp, rax
```

Для unwind главный эффект всё равно такой:

```
RSP уменьшен на 0x3000.
```

Но в коде ты видишь не только sub rsp, immediate, а целую последовательность. Поэтому сверка должна быть смысловой. Нужно понять: размер загружен в RAX, helper вызван, потом RSP уменьшен. UNWIND_CODE будет описывать итоговую fixed allocation, а не каждую внутреннюю деталь __chkstk как самостоятельное unwind-состояние.

Для static -а это важный пример: “не нашёл простого sub rsp, imm” не равно “metadata неверная”. Надо учитывать допустимые формы пролога.

7.4.11. UWOP_SET_FPREG: установка frame register

Теперь перейдём к UWOP_SET_FPREG.

FPREG - frame pointer register, регистр указателя кадра. В x64 это не всегда RBP, но в обычных примерах часто именно RBP. Frame register нужен, когда функция хочет иметь стабильную базу внутри fixed stack allocation.

Пример:

```
push rbp
sub  rsp, 40h
lea  rbp, [rsp+20h]
```

Что здесь происходит? Сначала сохраняется старый RBP, потому что RBP nonvolatile. Затем выделяется 0x40 байт. Потом новый RBP устанавливается не равным RSP, а равным RSP + 0x20. То есть он указывает внутрь выделенной области.

Зачем указывать в середину? Это может улучшить code density, то есть плотность кода: обращения к данным кадра могут использовать короткие смещения и вверх, и вниз от frame register. Если база стоит в середине, больше объектов кадра доступны короткими displacement-ами.

В UNWIND_INFO есть два поля: FrameRegister и FrameOffset. FrameRegister говорит, какой регистр используется как frame register. FrameOffset хранит масштабированное смещение, обычно умножаемое на 16. UWOP_SET_FPREG говорит, что в прологе был момент, где этот frame register был установлен.

Смысл для unwind: если дальнейшие сохранения или обращения описаны относительно frame register, unwinder должен понимать, как этот регистр соотносится с RSP и fixed allocation. Без этого восстановление может быть неверным.

Типичная ошибка: увидеть rbp в коде и сразу решить “это классический frame pointer как на x86”. На x64 RBP может быть обычным nonvolatile-регистром. Истину показывает сочетание кода и UNWIND_INFO: есть ли FrameRegister, есть ли UWOP_SET_FPREG, как именно установлен регистр.

7.4.12. UWOP_SAVE_NONVOL: сохранение регистра через mov, а не push

Не всегда nonvolatile-регистр сохраняется через push. Иногда функция сначала выделяет стековую область, а потом сохраняет регистр инструкцией mov в заранее выделенный slot.

Пример:

```
sub rsp, 60h
mov [rsp+30h], rsi
```

Здесь старый RSI сохраняется не через push rsi, а через запись в память. Для unwind это всё равно важно: RSI nonvolatile, значит caller ожидает его прежнее значение. Unwinder должен знать, где на стеке лежит сохранённый RSI.

Такое действие описывает UWOP_SAVE_NONVOL. SAVE - сохранить. NONVOL - nonvolatile integer register. В OpInfo хранится номер регистра, а в дополнительном slot-е хранится offset. Этот offset показывает, где относительно базы кадра находится сохранённое значение.

Почему это важно для reverse engineering? Потому что если искать только push rbx, можно пропустить сохранение через mov. А оптимизирующий компилятор может выбирать разные формы. Особенно это связано с shrink-wrapping.

Shrink-wrapping - оптимизационный приём, при котором сохранение nonvolatile-регистра откладывается до того места, где он реально нужен, а не выполняется обязательно в самом начале функции. Это усложняет красивую картину “всё сохраняется в первых пяти инструкциях”. Chained unwind info может использоваться для описания таких ситуаций. Поэтому реверсер должен понимать не один шаблон, а смысл: где и как сохранено nonvolatile-состояние.

7.4.13. UWOP_SAVE_NONVOL_FAR: когда offset слишком большой

UWOP_SAVE_NONVOL_FAR похож на UWOP_SAVE_NONVOL, но используется, когда offset до сохранённого регистра слишком велик для короткой формы. FAR здесь не означает far pointer из старой x86-сегментации. Здесь это просто “дальний offset”, то есть большее смещение, которое требует большего кодирования.

Обычный UWOP_SAVE_NONVOL хранит offset в компактной форме, масштабированной на 8. UWOP_SAVE_NONVOL_FAR использует больше slots и хранит более широкий offset. Для парсера это снова важно: одна операция занимает больше места в массиве UNWIND_CODE.

Для реверсера теория такая: чем больше стековый кадр и чем сложнее расположение сохранённых данных, тем чаще нужны формы, которые хранят дополнительные offsets. Но смысл не меняется: unwinder должен знать, где взять старое значение nonvolatile-регистра.

Если анализатор показывает “SAVE_NONVOL_FAR R12 at offset ...”, это не должно восприниматься как отдельная магия. Это то же сохранение caller state, просто место сохранения далеко от базы кадра.

7.4.14. UWOP_SAVE_XMM128 и XMM-регистры

До этого мы говорили в основном о general-purpose registers, то есть регистрах общего назначения: RBX, RBP, RDI, RSI, R12-R15. Но Windows x64 ABI также определяет правила для XMM-регистров.

XMM registers - 128-битные SIMD-регистры. SIMD означает Single Instruction, Multiple Data: одна инструкция может работать сразу с несколькими элементами данных. XMM-регистры используются для floating-point, vector operations, иногда для оптимизированного копирования и других задач.

Некоторые XMM-регистры считаются nonvolatile в нужной части. Если функция использует их так, что обязана сохранить состояние caller-a, она должна записать их на стек и восстановить позже.

Для этого существуют UWOP_SAVE_XMM128 и UWOP_SAVE_XMM128_FAR. Они похожи по идее на UWOP_SAVE_NONVOL, но работают с 128-битным XMM-состоянием. Offset для XMM-сохранения масштабируется иначе, потому что 128 бит - это 16 байт, и такие данные должны быть 16-byte aligned.

Пример по смыслу:

```
sub    rsp, 80h
movdqa [rsp+40h], xmm6
```

movdqa - move aligned double quadword, инструкция перемещения 128-битных выровненных данных. Если сохраняется nonvolatile XMM-регистр, unwind metadata должна знать, где лежит старое значение.

Для реверсера важно не игнорировать XMM. В простых учебных примерах их часто нет, но в реальном оптимизированном коде XMM-состояние может быть частью caller context. Если static анализирует только integer registers, он будет неполным.

7.4.15. UWOP_PUSH_MACHFRAME: особый случай машинного frame

UWOP_PUSH_MACHFRAME - особая операция. Она не похожа на обычный push rbx или sub rsp. Она описывает ситуацию, где процессор/система концептуально поместила на стек machine frame, то есть машинный кадр, связанный с interrupt/exception transfer.

Machine frame - это набор значений вроде return RIP, segment state, flags и иногда error code, которые возникают в низкоуровневых сценариях. Для обычного пользовательского C-кода это не основной случай. Но формат unwind должен уметь описывать и такие ситуации, потому что Windows должна раскручивать не только красивые функции, но и специальные переходы, interrupt/trap-related stubs, системные куски.

Для нашей книги сейчас важно не углубляться в kernel-level детали, а понять место операции: UWOP_PUSH_MACHFRAME показывает, что UWOP_* - это не “набор шаблонов MSVC для обычного C”. Это формат, который описывает разные unwind-релевантные изменения состояния.

Практическое правило: если в обычном пользовательском PE ты видишь UWOP_PUSH_MACHFRAME, это требует осторожного контекста. Это может быть системный/особый код, ручная генерация, необычный stub или ошибка парсинга. В будущем разделе про аномалии мы вернёмся к таким случаям.

7.4.16. CodeOffset: почему важно место операции в прологе

Теперь вернёмся к CodeOffset. Он часто выглядит как маленькое служебное поле, но без него partial prolog unwinding был бы невозможен.

Представим пролог:

```
00: push rbx
01: sub  rsp, 40h
05: mov  [rsp+20h], rsi
0A: ... тело функции ...
```

Если исключение произошло уже в теле функции, всё просто: пролог выполнен полностью. При unwind нужно отменить все прологовые действия: вернуть RSP, восстановить RBX, восстановить RSI, если он был сохранён как nonvolatile.

Но если исключение произошло внутри пролога, картина другая. Допустим, RIP указывает на инструкцию после push rbx, но до sub rsp, 40h. Тогда RBX уже лежит на стеке, а 0x40 байт ещё не выделены. Если unwinder применит полное описание пролога, он ошибётся.

CodeOffset помогает определить, какие действия уже произошли к текущей точке. Поэтому offset в unwind-коде связан с положением прологовой инструкции. Это не просто декоративное “где инструкция была”. Это условие правильного частичного восстановления.

Для reverse engineer-а это даёт важный уровень проверки. Недостаточно увидеть, что UWOP_ALLOC_SMALL есть. Нужно понять: его CodeOffset соответствует месту sub rsp, ...? Не указывает ли он на странную область? Не больше ли он SizeOfProlog? Не создаёт ли он невозможную историю, где unwind говорит, что аллокация произошла раньше инструкции?

7.4.17. SizeOfProlog: граница между “кадр строится” и “кадр построен”

SizeOfProlog - поле UNWIND_INFO, которое говорит размер пролога в байтах. Это не количество инструкций, а размер области от начала функции, где prolog actions считаются частью построения кадра.

Почему это важно? В x64 unwind procedure есть отдельный случай: если RIP находится в прологе, control has not entered the function body in обычном смысле. Тогда для этой функции не рассматривается обычный exception handler, a unwinder должен отменить только уже выполненные эффекты пролога и перейти к caller context.

Это звучит странно: “если исключение произошло в функции, почему её handler не работает?” Но логика такая: функция ещё не вошла в нормальное тело. Её кадр может быть частично построен. Состояние не такое, как в обычном body. Поэтому handler текущей функции не должен восприниматься так, будто функция уже полностью активна.

Для реверсера SizeOfProlog - это мост между .xdata и .text. Он задаёт область, внутри которой CodeOffset имеет смысл. Если SizeOfProlog = 0x0A, то пролог занимает первые 10 байт функции. Если ты видишь прологовые действия после этой границы, нужно разбираться. Возможно, это shrink-wrapping/chained info, возможно, дизассемблер неправильно определил функцию, возможно, метаданные сложнее, чем кажется.

7.4.18. Пример 1: простой prolog без frame pointer

Возьмём простой пример:

```
func:
    push rbx
    sub  rsp, 20h
    ; body
    add  rsp, 20h
    pop  rbx
    ret
```

Что делает функция?

Первое: push rbx. Она сохраняет nonvolatile RBX, потому что собирается использовать его внутри. RSP уменьшается на 8, старый RBX лежит на вершине стека.

Второе: sub rsp, 20h. Она выделяет 0x20 байт fixed stack allocation. Это может быть место под локальные данные, alignment или временные slots.

В теле функция работает. В эпилоге она делает обратное: сначала освобождает 0x20 байт, потом восстанавливает RBX, потом ret берёт return address.

Какая unwind-история должна быть?

```
UNWIND_INFO:
  Version = 1
  Flags = NHANDLER
  SizeOfProlog = размер push + sub
  CountOfCodes = 2
  FrameRegister = none

UNWIND_CODE:
  UWOP_ALLOC_SMALL, size = 0x20
```

```
  UWOP_PUSH_NONVOL, reg = RBX
```

Почему в массиве сначала может стоять allocation, а потом push? Потому что unwind-коды отсортированы в порядке убывания offset-ов. sub rsp, 20h находится позже в прологе, чем push rbx, поэтому его запись может идти раньше в массиве.

Как это использовать при virtual unwind из тела функции? Если RIP в body, пролог выполнен полностью. Чтобы получить caller context, unwinder должен:

```
1. Отменить fixed allocation: RSP = RSP + 0x20.
2. Восстановить RBX из [RSP].
3. Увеличить RSP ещё на 8 после pop-like восстановления.
4. Взять return address из [RSP] как caller RIP.
5. RSP = RSP + 8.
```

Это не обязательно буквально порядок внутренней реализации, но смысл такой: убрать эффекты текущей функции и получить состояние caller-а.

7.4.19. Пример 2: prolog с frame pointer

Теперь пример с frame pointer:

```
func:
    push rbp
    sub  rsp, 40h
    lea  rbp, [rsp+20h]
    ; body
    lea  rsp, [rbp+20h]
    pop  rbp
    ret
```

Здесь функция использует RBP как frame register. Сначала старый RBP сохраняется, потому что RBP nonvolatile. Затем выделяется 0x40 байт. Потом RBP устанавливается в RSP + 0x20, то есть в середину fixed allocation.

Как это отражается в unwind metadata?

```
FrameRegister = RBP
FrameOffset   = 2        ; 2 * 16 = 0x20

UNWIND_CODE:
  UWOP_SET_FPREG
  UWOP_ALLOC_SMALL, size = 0x40
  UWOP_PUSH_NONVOL, reg = RBP
```

FrameOffset масштабируется на 16. Если в поле стоит 2, реальное смещение 0x20. Это одна из типичных ошибок парсера: вывести FrameOffset = 2 и не объяснить, что это scaled field. Человеку нужно показывать и raw value, и interpreted value.

Смысл для unwind такой: если в дальнейшем сохранения или offsets интерпретируются относительно frame register, нужно знать, что RBP = RSP + 0x20 после fixed allocation. Это помогает правильно вычислить адреса сохранённых значений.

Здесь видно, почему нельзя анализировать RBP по привычке x86. В x86 часто mov ebp, esp делал EBP базой кадра с простым отношением. В x64 frame register может указывать не на старую вершину стека, а внутрь fixed allocation. Это не ошибка, а нормальный способ улучшить адресацию.

7.4.20. Пример 3: сохранение register через mov после allocation

Рассмотрим другой стиль:

```
func:
    sub  rsp, 60h
    mov  [rsp+50h], rsi
    mov  [rsp+48h], rdi
    ; body
    mov  rdi, [rsp+48h]
    mov  rsi, [rsp+50h]
    add  rsp, 60h
    ret
```

Здесь нет push rsi и push rdi. Функция выделяет 0x60 байт, а затем сохраняет RSI и RDI внутрь этой области через mov. Это допустимая логика.

Unwind metadata должна описать:

```
UWOP_ALLOC_SMALL or UWOP_ALLOC_LARGE, size = 0x60
UWOP_SAVE_NONVOL, reg = RSI, offset = 0x50
UWOP_SAVE_NONVOL, reg = RDI, offset = 0x48
```

Смысл offset-а: где относительно базы кадра лежит сохранённое значение. Но база может быть RSP после allocation или frame register, если он установлен. Поэтому просто увидеть offset недостаточно; нужно знать frame model функции.

Для реверсера это важный пример того, что “сохранение регистра” не равно “push”. Если парсер ищет только push-паттерны, он не поймёт реальные nonvolatile saves. А если он читает .xdata, он должен показать человеку: RSI сохранён по offset 0x50, проверьте в прологе запись mov [rsp+50h], rsi или эквивалентную форму.

7.4.21. Почему эпилог не имеет отдельного массива UNWIND_CODE

Когда мы сверяем пролог и UNWIND_CODE, возникает вопрос: а где описаны эпилоги? Ведь функция может иметь несколько выходов.

На Windows x64 эпилоги должны следовать строгим правилам, чтобы unwinder мог распознавать их, сканируя код вперёд от текущего RIP. Поэтому для каждого эпилога обычно не нужна отдельная таблица unwind-кодов. Это экономит место и упрощает формат. Пролог описан явно, эпилог должен быть распознаваемым по допустимым формам.

Типичный эпилог без frame pointer:

```
add rsp, 40h
pop rbx
ret
```

Типичный эпилог с frame pointer:

```
lea rsp, [rbp+20h]
pop rbp
ret
```

Смысл: эпилог делает обратное прологу. Но unwind metadata не хранит “эпилог номер 1, эпилог номер 2”. Вместо этого x64 unwinder умеет определить, что текущая инструкция находится в допустимой эпилоговой последовательности, и симулировать оставшийся выход.

Для реверсера это объясняет, почему UNWIND_CODE сверяется в первую очередь с прологом, а не с каждым return. Если эпилог выглядит необычно, это может нарушить unwindability. Но проверка эпилогов - отдельная тема: нужно

смотреть, соответствует ли последовательность допустимым формам. Мы вернёмся к этому в главе про аномалии.

7.4.22. Почему “одна инструкция - один UWOP” не всегда верно

Очень важно разрушить удобную, но неверную модель: будто каждая инструкция пролога соответствует ровно одному UWOP, а каждый UWOP соответствует ровно одной инструкции.

Иногда это примерно так. push rbx соответствует UWOP_PUSH_NONVOL RBX. sub rsp, 40h соответствует UWOP_ALLOC_SMALL size=0x40. Но есть исключения.

Одна unwind-операция может занимать несколько slots. Например, UWOP_ALLOC_LARGE использует дополнительные slots для размера. UWOP_SAVE_NONVOL использует дополнительный slot для offset-a. Значит, CountOfCodes больше количества понятных действий.

Одна смысловая операция в коде может быть частью более широкой прологовой схемы. Большая аллокация через __chkstk включает mov rax, size, call __chkstk, sub rsp, rax. Unwind metadata описывает stack allocation как эффект, а не все инструкции.

Некоторые инструкции пролога могут не иметь отдельного unwind-кода, если они не влияют на unwind-relevant state. Например, сохранение register parameter в home space может быть важно для логики функции, но не обязательно нужно для восстановления caller context.

Поэтому правильная формула такая: UNWIND_CODE описывает только те изменения состояния, которые нужны системному unwind. Он не является полным логом пролога.

7.4.23. Как реверсер должен сверять: не глазами, а по слоям

Теперь соберём практическую методику. Не как сухой алгоритм, а как способ мышления.

Сначала ты находишь функцию через .pdata: RUNTIME_FUNCTION даёт BeginAddress, EndAddress, UnwindInfoAddress. Это даёт диапазон кода и адрес UNWIND_INFO.

Потом ты читаешь UNWIND_INFO: версия, флаги, размер пролога, количество slots, frame register, frame offset, массив UNWIND_CODE, хвост с handler/chained info, если он есть.

Потом ты открываешь код функции от BeginAddress и внимательно смотришь первые SizeOfProlog байт. Не “первые пять строк дизассемблера”, а именно байтовую область пролога.

Затем для каждой unwind-операции задаёшь вопрос: какой эффект она описывает?

```
UWOP_PUSH_NONVOL RBX
 -> в прологе должен быть эффект сохранения RBX через push.

UWOP_ALLOC_SMALL 0x40
 -> в прологе должен быть эффект fixed allocation 0x40.
```

```
UWOP_SET_FPREG RBP+0x20
 -> в прологе должна быть установка RBP относительно RSP.

UWOP_SAVE_NONVOL RSI offset 0x38
 -> в прологе должно быть сохранение RSI в stack slot.
```

После этого ты проверяешь не только наличие инструкций, но и порядок. Nonvolatile-регистр должен быть сохранён до первого использования в прологе. Stack allocation должна соответствовать размеру. Frame register должен быть установлен после того, как есть база для него. Offsets должны быть кратны нужным единицам. CodeOffset должен попадать в SizeOfProlog.

И только потом формулируешь вывод: metadata согласована с прологом, metadata частично расходится, metadata невалидна, metadata относится к chained info, или требуется более сложный анализ.

7.4.24. Типичные ошибки новичка при сверке

Первая ошибка - считать UnwindOp машинным opcode. Нет. UWOP_ALLOC_SMALL - не инструкция CPU. Это запись эффекта stack allocation.

Вторая ошибка - считать CountOfCodes количеством операций. Нет. Это количество двухбайтовых slots. Некоторые операции занимают несколько slots.

Третья ошибка - читать массив UNWIND_CODE в порядке выполнения пролога. Нет. Он отсортирован по offset-ам в формате, удобном unwinder-у, и часто выглядит обратным относительно человеческого чтения.

Четвёртая ошибка - искать только push. Сохранение nonvolatile-регистра может быть через mov [rsp+offset], reg и описываться UWOP_SAVE_NONVOL.

Пятая ошибка - игнорировать frame register. Если функция установила RBP или другой frame register, offsets нужно интерпретировать в этой системе координат.

Шестая ошибка - принимать имя секции или декомпилятор за истину. Реальный контракт строится между .pdata, .xdata, байтами кода и rules Windows x64 ABI.

Седьмая ошибка - сразу считать любое несоответствие вредоносностью. Несоответствие - это finding, но не всегда malware. Возможны ручной asm, необычный компилятор, JIT, chained info, оптимизация, ошибка твоего парсера или неполная модель.

7.4.25. Как это связано с RtlVirtualUnwind

Теперь нужно вернуть всю тему к RtlVirtualUnwind. Если забыть про него, UWOP_* снова превращаются в скучные поля.

RtlVirtualUnwind получает текущий контекст функции и должен вычислить контекст caller-а. Для этого он использует RUNTIME_FUNCTION и UNWIND_INFO. Если UNWIND_INFO говорит: “был UWOP_ALLOC_SMALL 0x40”, то при virtual unwind надо прибавить 0x40 к RSP. Если говорит: “был UWOP_PUSH_NONVOL RBX”, то надо восстановить старый RBX из стека и передвинуть RSP так, будто был отменён push. Если говорит: “был UWOP_SAVE_NONVOL RSI offset 0x38”, надо взять RSI из указанного stack slot.

Именно поэтому сверка с прологом критична. Если UNWIND_CODE врёт, RtlVirtualUnwind честно применит ложную инструкцию. Он не обязан подозревать, что файл обфусцирован или повреждён. Он действует по контракту. Если контракт

нарушен, результат становится неправильным.

Для crash dump это означает ложный stack trace. Для exception dispatch - неправильный поиск handler-a. Для debugger-a - странные кадры. Для reverse engineer-a - риск построить неверную историю выполнения.

7.4.26. Как это связано с SEH и language-specific handler

На первый взгляд UWOP_* описывают только стек и регистры. При чём тут SEH? Связь прямая.

Чтобы Windows дошла до frame-based SEH handler-a, она должна пройти по кадрам. Чтобы пройти по кадрам, ей нужен virtual unwind. Чтобы virtual unwind был правильным, UNWIND_CODE должен соответствовать прологу. Если на каком-то кадре восстановление caller context сломалось, Windows может не дойти до обработчика выше по стеку или дойти с неправильным контекстом.

Кроме того, UNWIND_INFO содержит flags. Если установлен UNW_FLAG_EHANDLER, после unwind-кодов лежит address language-specific handler-a, например __C_specific_handler. Но чтобы корректно вызвать handler для текущего кадра, система всё равно должна понимать establisher frame и состояние функции. Эти вещи опираются на правильное описание кадра.

То есть UWOP_* - это нижний слой под SEH. Без него __C_specific_handler может быть найден, но общая дорога по стеку будет ненадёжной. SEH кажется “логикой обработчиков”, но под ним лежит геометрия кадра: где RSP, где сохранённые регистры, где caller, где return address.

7.4.27. Как будущий static должен показывать сверку

Инструмент не должен просто выводить таблицу:

```
0: UWOP_ALLOC_SMALL 0x40
1: UWOP_PUSH_NONVOL RBX
```

Такой вывод полезен, но недостаточен для обучения и анализа. Хороший static должен показывать связку:

```
Function RVA: 0x1230 - 0x12A0
UnwindInfo RVA: 0x4020
Prolog size: 5 bytes

Prolog bytes / disassembly:
  0x1230: push rbx
  0x1231: sub rsp, 40h

Unwind interpretation:
  CodeOffset 0x05: UWOP_ALLOC_SMALL, size 0x40
      matches: sub rsp, 40h
  CodeOffset 0x01: UWOP_PUSH_NONVOL, reg RBX
      matches: push rbx

Verdict:
  prolog and unwind metadata are consistent.
```

Если есть проблема, finding должен быть конкретным:

```
Finding: unwind metadata describes saving RBX, but no matching prolog action was found inside SizeOfPro
Evidence:
  Function RVA: ...
  SizeOfProlog: ...
```

```
    UWOP_PUSH_NONVOL RBX at CodeOffset ...
    Disassembly in prolog range: ...
  Possible explanations:
    wrong function start, chained unwind info, unusual compiler output, corrupted metadata, obfuscation.
  Severity:
    suspicious / requires manual review.
```

Такой стиль важен потому, что мы пишем не просто книгу, а фундамент для будущего анализатора. Инструмент должен объяснять, а не только ругаться.

7.4.28. Что считать нормой, а что вопросом

Нормально, если простая функция без сложного кадра не имеет .pdata записи, потому что leaf function может раскручиваться простой симуляцией return.

Нормально, если UNWIND_CODE не описывает каждую инструкцию пролога, а только unwind-relevant действия.

Нормально, если порядок UNWIND_CODE не совпадает с визуальным порядком инструкций.

Нормально, если allocation кодируется small или large формой в зависимости от размера.

Нормально, если nonvolatile-регистры сохраняются не только через push, но и через mov в stack slot.

Нормально, если frame register указывает внутрь fixed allocation, а не ровно на старый RSP.

Вопрос возникает, если UNWIND_CODE описывает действие, которого нет в прологе; если CodeOffset выходит за SizeOfProlog; если allocation size не совпадает с реальным изменением RSP; если сохранённый register offset указывает за пределы reasonable frame; если FrameRegister заявлен, но код его не устанавливает; если CountOfCodes приводит к чтению handler data из неверного места; если chained info интерпретируется как handler или наоборот.

Но “вопрос” - не значит “сразу вредоносность”. В reverse engineering зрелость начинается с аккуратной формулировки: что именно видно, почему это необычно, какие есть нормальные объяснения, какие проверки нужны дальше.

7.4.29. Мини-модель раздела

Теперь соберём всё в одну компактную модель.

Функция в .text строит кадр в прологе. Она может сохранять nonvolatile-регистры, выделять место на стеке, устанавливать frame register, сохранять XMM-регистры. Эти действия меняют машинное состояние: RSP, stack memory, nonvolatile registers.

UNWIND_INFO в .xdata содержит массив UNWIND_CODE, который описывает unwind-relevant эффекты этих действий. Он не является полным дизассемблированным прологом. Он является компактным рецептом для восстановления caller context.

RtlVirtualUnwind использует этот рецепт, чтобы отменить кадр: восстановить RSP, nonvolatile-регистры и caller RIP. Exception dispatch использует этот процесс, чтобы идти по кадрам и искать handler. Отладчик использует похожую информацию для stack trace. Crash dump analyzer использует её для

восстановления истории падения.

Реверсер сверяет код и metadata, потому что между ними должен быть контракт. Если контракт выполнен, анализ устойчив. Если контракт нарушен, это finding.

Короткая формула:

```
prolog делает
UNWIND_CODE описывает
RtlVirtualUnwind отменяет
SEH/stack walking используют результат
reverse engineer проверяет согласованность
```

Эта формула переводит нас к следующему разделу. Теперь, когда мы умеем связывать UWOP_* с прологом, можно сделать следующий шаг: мысленно повторить RtlVirtualUnwind целиком. То есть не просто прочитать unwind-коды, а пройти руками путь от текущего CONTEXT к caller context: где станет новый RSP, откуда возьмётся новый RIP, какие nonvolatile-регистры восстановятся и почему именно так.

7.4.30. Обратная проверка: не только от UWOP к коду, но и от кода к UWOP

До этого мы шли в основном от UNWIND_CODE к прологу: увидели UWOP_PUSH_NONVOL, ищем соответствующий push; увидели UWOP_ALLOC_SMALL, ищем изменение RSP. Но хорошая сверка должна идти в обе стороны.

Первая сторона - metadata-driven. Metadata-driven значит “ведомый метаданными”. Мы берём UNWIND_INFO как источник заявлений и проверяем, подтверждает ли код эти заявления. Например: metadata говорит, что RBP установлен как frame register. Мы смотрим, есть ли в прологе lea rbp, [rsp+...] или эквивалентная установка.

Вторая сторона - code-driven. Code-driven значит “ведомый кодом”. Мы читаем пролог как машинные действия и спрашиваем: каждое ли unwind-relevant действие отражено в metadata? Например: код делает push r15, а в UNWIND_CODE нет сохранения R15. Это уже не просто странность в таблице, а потенциально серьёзное расхождение: при virtual unwind старый R15 не будет восстановлен.

Обе стороны нужны, потому что каждая ловит разные ошибки. Если ты идёшь только от metadata к коду, ты можешь не заметить дополнительное действие в прологе, которое metadata не описывает. Если идёшь только от кода к metadata, можешь неправильно интерпретировать chained info, frame register или multi-slot операции. Поэтому зрелая проверка всегда двунаправленная.

Пример code-driven проверки:

```
push rbx
push r15
sub  rsp, 30h
```

Ожидаемая история:

```
RBX сохранён через push -> нужен UWOP_PUSH_NONVOL RBX
R15 сохранён через push -> нужен UWOP_PUSH_NONVOL R15
RSP уменьшен на 0x30 -> нужен UWOP_ALLOC_SMALL 0x30
```

Если в UNWIND_CODE есть только RBX и allocation, но нет R15, анализатор должен сформулировать finding: “код сохраняет nonvolatile R15, но unwind

metadata не описывает восстановление R15”. Это не значит автоматически, что программа упадёт прямо сейчас, но означает, что при исключении/stack walking восстановление caller context может быть неверным.

7.4.31. Почему длина инструкции важна для CodeOffset

CodeOffset кажется простым: смещение внутри пролога. Но на x64 инструкции имеют переменную длину. Variable-length instruction означает инструкцию переменной длины: одна инструкция может занимать 1 байт, другая 3, третья 7, четвёртая 10 или больше. Это наследие x86/x64 encoding.

Поэтому нельзя считать: “первая инструкция - offset 1, вторая - offset 2, третья - offset 3”. Offset считается в байтах от начала функции, а не в номере инструкции.

Например:

```
00: push rbx          ; 1 байт
01: sub rsp, 40h      ; может быть 4 байта
05: lea rbp, [rsp+20h]; может быть 5 байт
```

Если sub rsp, 40h заканчивается на смещении 5, то unwind-запись для allocation может иметь CodeOffset, связанный с этим концом. Если lea rbp, [rsp+20h] заканчивается на смещении 0x0A, то UWOP_SET_FPREG будет связан с более поздним offset-ом.

Для ручного анализа это значит: иногда нужно смотреть не только дизассемблированные строки, но и адреса инструкций. Дизассемблер обычно показывает адрес каждой инструкции. Разница между адресом инструкции и BeginAddress функции даёт offset. Если функция начинается на RVA 0x1000, а sub rsp, 40h стоит на RVA 0x1001 и заканчивается перед 0x1005, то действие связано с маленьким offset-ом около 5, а не просто “вторая строка”.

Для static -а это особенно важно. Инструмент должен уметь выводить не только “matching instruction found”, но и почему она match-ится: какая длина инструкции, какой offset от начала функции, попадает ли этот offset в SizeOfProlog, соответствует ли он CodeOffset.

7.4.32. MASM-директивы как мост между ручным asm и .xdata

Чтобы лучше понять, как код превращается в unwind metadata, полезно посмотреть не только на C/C++ compiler, но и на MASM. MASM - Microsoft Macro Assembler, ассемблер Microsoft. Когда программист пишет x64 assembler вручную, ему нужно не просто написать инструкции. Если функция nonleaf и требует unwind, нужно сгенерировать правильные .pdata и .xdata.

Для этого в MASM существуют специальные директивы. Директива - команда ассемблеру, которая не обязательно создаёт машинную инструкцию напрямую, но влияет на сборку, метаданные или структуру объекта.

Например:

```
my_func PROC FRAME
    push rbx
    .pushreg rbx
    sub  rsp, 40h
    .allocstack 40h
    .endprolog
    ; body
```

```
    add  rsp, 40h
    pop  rbx
    ret
my_func ENDP
```

push rbx - настоящая машинная инструкция. .pushreg rbx - директива, которая говорит ассемблеру: для этой инструкции нужно сгенерировать UWOP_PUSH_NONVOL RBX. sub rsp, 40h - настоящая инструкция. .allocstack 40h - директива, которая говорит: сгенерируй unwind-запись для stack allocation. .endprolog сообщает, где заканчивается пролог.

Эта пара “инструкция + директива” очень хорошо показывает философию Windows x64. Код сам по себе меняет стек. Но для ОС нужно ещё формальное описание этих изменений. В C/C++ компилятор создаёт его автоматически. В ручном asm автор должен помочь ассемблеру.

Для реверсера это объясняет, почему metadata не является “автоматически выведенной Windows из кода”. Она генерируется инструментами сборки. Если автор ручного asm ошибся в директивах, код может быть логически рабочим при обычном вызове/return, но плохо раскручиваться при исключении.

7.4.33. Обычный return может работать, а unwind - ломаться

Это очень важная теоретическая мысль. Функция может нормально вызываться и нормально возвращаться через ret, но при этом иметь неправильные unwind-метаданные. Почему?

Обычный return выполняет реальный эпилог. Если код сделал add rsp, 40h, pop rbx, ret, то процессор реально восстановил RSP, вернул RBX и взял return address. В этом сценарии процессор не читает .xdata. Он просто выполняет инструкции.

Virtual unwind работает иначе. Он не выполняет эпилог функции. Он смотрит на metadata и вычисляет, что должен был бы сделать эпилог. Если metadata неправильная, virtual unwind может ошибиться, даже если реальный эпилог в коде был правильным.

Это значит, что баг или обман в .xdata может быть невидим при обычном тестировании программы. Программа запускается, функции возвращаются, всё вроде работает. Но стоит случиться исключению, breakpoint-у, stack walk-у, профилированию, crash dump analysis - и внезапно call stack становится неправильным.

Для будущего static -а это важнейший мотив: проверка unwind metadata - это не декоративная “валидность PE”. Это проверка поведения программы в исключительных и диагностических сценариях.

7.4.34. Несоответствие бывает разных типов

Когда код и unwind metadata расходятся, полезно классифицировать расхождение. Не всё одинаково опасно.

Первый тип - missing metadata. Код делает unwind-relevant действие, но metadata его не описывает. Например, push r14 есть, а UWOP_PUSH_NONVOL R14 нет. Это опасно для восстановления nonvolatile-состояния.

Второй тип - extra metadata. Metadata описывает действие, которого нет в коде. Например, заявлен UWOP_ALLOC_SMALL 0x40, но в прологе нет выделения 0x40. Тогда unwinder может прибавить к RSP лишний размер и взять return address не там.

Третий тип - wrong size. Действие есть, но размер не совпадает. Код делает sub rsp, 50h, metadata говорит 0x40. Это почти всегда серьёзное расхождение, если нет объяснения через неправильную границу функции или chained info.

Четвёртый тип - wrong register. Код сохраняет RBX, metadata говорит RSI. Тогда один регистр будет восстановлен не оттуда, другой не восстановлен вообще.

Пятый тип - wrong coordinate system. Metadata говорит offset, но анализатор неправильно понял, относительно чего этот offset считается: RSP, frame register, scaled value. Это может быть не ошибка бинаря, а ошибка инструмента.

Шестой тип - wrong prolog boundary. SizeOfProlog не покрывает действия, которые явно выглядят как prolog actions, или наоборот включает слишком много обычного body code. Это влияет на partial prolog unwinding.

Седьмой тип - wrong tail interpretation. Анализатор неправильно учёл padding после UNWIND_CODE и принял handler RVA за дополнительный unwind slot или chained info за handler. Это ошибка парсинга, но она может создать ложные findings.

Такая классификация нужна, чтобы будущий отчёт был человеческим. Не просто “bad unwind”, а “allocation size mismatch between prolog and unwind metadata”. Чем точнее finding, тем легче реверсеру понять, что проверять дальше.

7.4.35. Медленная ручная проверка одной функции

Представим, что мы руками проверяем функцию. Не торопимся.

Сначала записываем координаты:

```
ImageBase = 0x140000000
Function Begin RVA = 0x1230
Function End RVA   = 0x1290
UnwindInfo RVA     = 0x4020
```

Потом смотрим UNWIND_INFO:

```
Version = 1
Flags = 0
SizeOfProlog = 0x08
CountOfCodes = 3
FrameRegister = none
```

Потом читаем unwind slots:

```
CodeOffset 0x08: UWOP_ALLOC_SMALL, size 0x30
CodeOffset 0x04: UWOP_PUSH_NONVOL, reg RSI
CodeOffset 0x01: UWOP_PUSH_NONVOL, reg RBX
```

Теперь открываем первые 8 байт функции в дизассемблере:

```
0x140001230: push rbx
0x140001231: push rsi
0x140001232: sub  rsp, 30h
```

Сверяем медленно.

push rbx есть. Он начинается в начале функции, меняет RSP на -8, сохраняет nonvolatile RBX. Metadata содержит UWOP_PUSH_NONVOL RBX. Совпадает.

push rsi есть. RSI тоже nonvolatile. Metadata содержит UWOP_PUSH_NONVOL RSI. Совпадает.

sub rsp, 30h есть. Metadata содержит UWOP_ALLOC_SMALL size 0x30. Совпадает.

Порядок в metadata обратный человеческому чтению, но это нормально: более поздний offset идёт раньше. SizeOfProlog покрывает все три инструкции. Frame register не заявлен, и в коде нет установки frame pointer. Флаги handler-а не установлены, значит хвоста handler data нет. Вердикт: пролог и unwind metadata согласованы.

Такой медленный стиль проверки полезен даже тогда, когда потом это будет автоматизировано. Инструмент должен делать быстро то, что человек умеет объяснить медленно.

7.4.36. Почему этот раздел - фундамент для следующего шага

Мы всё ещё не “закончили unwind”. Мы только научились проверять соответствие пролога и UNWIND_CODE. Но именно это делает следующий раздел возможным.

В следующем разделе мы будем мысленно повторять RtlVirtualUnwind. Это уже не просто чтение metadata. Это попытка пройти вычисление: текущий RSP такой-то, функция выделила столько-то, сохранила такие-то регистры, значит caller RSP будет таким-то, caller RIP возьмётся отсюда, RBX восстановится отсюда.

Если не понимать текущий раздел, RtlVirtualUnwind будет казаться магической функцией Windows. Если понимать текущий раздел, RtlVirtualUnwind становится почти арифметикой над кадром: отменить allocation, восстановить nonvolatile-регистры, найти return address, перейти к caller context.

Поэтому мы и идём медленно. Книга строит не набор справочных фактов, а внутреннюю модель. .pdata сказала, где функция. .xdata сказала, как её раскручивать. UNWIND_CODE сказал, какие эффекты пролога нужно отменить. Следующий шаг - применить эти эффекты к живому CONTEXT.

Техническая база раздела 7.4

Раздел опирается на официальную документацию Microsoft по x64 exception handling, где описаны UNWIND_INFO, UNWIND_CODE, порядок массива unwind-кодов, UWOP_PUSH_NONVOL, UWOP_ALLOC_SMALL, UWOP_ALLOC_LARGE, UWOP_SET_FPREG, UWOP_SAVE_NONVOL, UWOP_SAVE_XMM128, UWOP_PUSH_MACHFRAME, chained unwind info и общий алгоритм unwind procedure.

Также использованы материалы Microsoft по x64 prolog and epilog, где объясняется, что prolog сохраняет nonvolatile-регистры, выделяет fixed stack allocation и может устанавливать frame pointer, а epilog обязан следовать строгим формам, чтобы unwinder мог распознавать и симулировать выход из функции. Дополнительно раздел связан с документацией Microsoft по x64 calling convention: volatile/nonvolatile registers, shadow space, alignment, leaf/nonleaf functions, pdata/xdata и unwindability.

7.5. Как мысленно повторить RtlVirtualUnwind: из текущего CONTEXT получить caller context

В предыдущем разделе мы сделали важную, но пока еще подготовительную работу. Мы научились смотреть на `UNWIND_CODE` не как на странные байты в `.xdata`, а как на сжатое описание того, что функция сделала в своем прологе. Мы увидели, что `push rbx`, `sub rsp, 30h`, сохранение `r12`, установка frame register и другие действия существуют в двух мирах сразу. В первом мире это обычные машинные инструкции в `.text`. Во втором мире это записи в `UNWIND_INFO`, которые Windows может прочитать без дизассемблирования всей функции.

Теперь нужно сделать следующий шаг. Если `.pdata` сказала нам, где находится функция, а `.xdata` сказала, как функция построила свой кадр, то что именно с этой информацией делает Windows? Ответ связан с функцией `RtlVirtualUnwind`. Но этот раздел не должен превращаться в пересказ сигнатуры API. Для reverse engineering гораздо важнее другое: научиться мысленно выполнять тот же переход, который делает unwinder. То есть брать текущий машинный контекст, понимать, где находится текущая функция, и получать контекст вызывающей функции.

Слово `RtlVirtualUnwind` надо разобрать сразу. `Rtl` - это префикс функций Runtime Library в Windows, часто связанных с низкоуровневыми механизмами `ntdll` и системного runtime. `Virtual` здесь означает не “виртуальная машина” и не “ненастоящая память”, а “моделируемая, вычисленная без реального выполнения обычного возврата”. `Unwind` - раскрутка стека, то есть переход от текущего кадра функции к предыдущему кадру, к caller-у. Поэтому грубый перевод имени такой: “runtime-функция, которая виртуально раскручивает один кадр”.

Очень важно слово “один”. `RtlVirtualUnwind` не строит весь stack trace целиком за один вызов. Она делает один шаг: из контекста текущей функции получает invocation context предыдущей функции. Если нужен весь стек, отладчик, crash analyzer или твой будущий static вызывают похожую логику циклом: текущий кадр -> caller -> caller caller-а -> еще выше. Именно поэтому в голове надо строить не магическую кнопку “покажи стек”, а маленький механический переход между двумя соседними кадрами.

7.5.1. Почему этот раздел является мостом между таблицами и живым выполнением

До этого момента мы много говорили о структурах. `RUNTIME_FUNCTION`, `UNWIND_INFO`, `UNWIND_CODE`, `UNW_FLAG_EHANDLER`, `UNW_FLAG_CHAININFO` - все это выглядит как статические данные. Они лежат в PE-образе, их можно открыть в hex-редакторе, вывести в таблицу, распарсить скриптом. Но исключение происходит не в таблице. Исключение происходит в живом потоке, у которого есть `RIP`, `RSP`, регистры, стековая память и текущая цепочка вызовов.

Главная идея этого раздела: `RtlVirtualUnwind` соединяет статическую карту с живым состоянием. Статическая карта говорит: “если ты находишься внутри этой функции, ее кадр был построен вот так”. Живое состояние говорит: “вот где поток сейчас, вот какой `RSP`, вот какие значения регистров”. Unwind берет оба слоя и делает вывод: “значит, вызывающая функция выглядела вот так”.

Поэтому нельзя изучать `RtlVirtualUnwind` отдельно от всего предыдущего. Если ты не понимаешь `RIP`, ты не знаешь, какую функцию искать. Если не понимаешь `RUNTIME_FUNCTION`, ты не знаешь границы функции и адрес `UNWIND_INFO`. Если не понимаешь `UNWIND_CODE`, ты не знаешь, как вернуть `RSP` и nonvolatile-регистры. Если не понимаешь calling convention, ты не понимаешь, какие регистры вообще нужно восстанавливать. Если не понимаешь search phase, ты не понимаешь, зачем Windows

делает этот переход много раз.

То есть `RtlVirtualUnwind` - это не отдельная тема. Это место, где вся книга начинает собираться в работающий механизм.

7.5.2. Что такое CONTEXT: снимок машины, а не “стек”

Перед тем как раскручивать кадр, нужно понять, что именно раскручивается. Частая ошибка новичка: “unwind раскручивает стек”. Это верно, но неполно. На практике unwind обновляет `CONTEXT`.

`CONTEXT` - структура Windows, которая хранит машинное состояние потока. Слово context переводится как “контекст”, но здесь это не общий жизненный контекст программы, а конкретный снимок регистров и части процессорного состояния. В контексте x64 нас особенно интересуют `RIP`, `RSP`, nonvolatile-регистры и иногда другие регистры, нужные для анализа.

`RIP` - instruction pointer, указатель инструкции. Он отвечает на вопрос: где сейчас управление в коде?

`RSP` - stack pointer, указатель стека. Он отвечает на вопрос: где сейчас вершина стека текущего потока?

Nonvolatile registers - сохраняемые по соглашению вызовов регистры. На Windows x64 это, например, `RBX`, `RBP`, `RSI`, `RDI`, `R12`, `R13`, `R14`, `R15`, а также часть XMM-регистров. Смысл nonvolatile не в том, что регистр физически “не изменяется”, а в том, что callee обязан вернуть его caller-у в прежнем состоянии, если использовал.

Когда исключение происходит, Windows получает или формирует `ContextRecord`. Это не просто “где упали”. Это состояние, из которого можно начать рассуждение. Если `RIP` указывает на инструкцию внутри функции `C`, а `RSP` уже уменьшен прологом `C`, то текущий `CONTEXT` описывает именно состояние внутри `C`, а не состояние `B`, которая вызвала `C`.

Unwind должен превратить этот контекст:

```
CONTEXT текущего кадра C:
    RIP = адрес внутри C
    RSP = стек внутри кадра C
    RBX/RBP/R12... = текущие значения внутри C
```

в такой контекст:

```
CONTEXT вызывающего кадра B:
    RIP = адрес возврата в B
    RSP = стек после возврата из C
    RBX/RBP/R12... = значения, которые B ожидала сохранить
```

Вот почему `RtlVirtualUnwind` работает не только со стековой памятью. Она работает с полной идеей “контекста выполнения”. Стек дает адрес возврата и сохраненные значения, но результатом является обновленный набор регистров.

7.5.3. Caller context: что значит “получить вызывающую функцию”

Термин `caller context` нужно разжевать отдельно. `Caller` - вызывающий код. Если функция `B` вызвала функцию `C`, то `B` является caller-ом, а `C` является callee. `Context` - машинное состояние. Поэтому `caller context` - это не просто “адрес функции B”, а машинное состояние, соответствующее моменту, когда выполнение снова оказалось бы в `B` после выхода из `C`.

При обычном выполнении это сделал бы `ret`. Функция `C` разобрала бы свой кадр, восстановила бы nonvolatile-регистры, взяла бы return address со стека и вернула

управление в `B`. После этого `RIP` был бы адресом инструкции после `call C`, а `RSP` указывал бы на стек `B`.

Но при исключении обычный `ret` не выполняется. Код функции не доходит спокойно до конца. Windows должна получить похожий результат аналитически. Она не выполняет инструкции тела функции назад. Она не исполняет `add rsp, ...`, `pop rbx`, `ret` как настоящий код. Она читает metadata и вычисляет, что эти действия означали бы для контекста.

Вот почему слово `virtual` в `RtlVirtualUnwind` важно. Это виртуальный возврат, виртуальная разборка кадра. Не настоящий control flow, а вычисление следующего контекста для анализа, поиска обработчика или построения стека.

Когда мы говорим “перешли к caller-у”, нужно не представлять, что процесс реально начал выполнять `B`. Во время search phase Windows может просто обновлять рабочий `ContextRecord`, чтобы проверить следующий кадр. Это похоже на следователя, который не перемещает реальную машину времени назад, а по следам восстанавливает предыдущий момент.

7.5.4. Нормальный return и virtual unwind: одна цель, разные пути

Чтобы понять unwind, полезно сравнить его с обычным возвратом из функции. Обычный возврат происходит через машинный код. Функция доходит до эпилога, выполняет инструкции, возвращает стек и регистры к состоянию caller-а, потом выполняет `ret`.

Например:

```
C:
    push rbx
    sub  rsp, 40h
    ; тело функции
    add  rsp, 40h
    pop  rbx
    ret
```

При нормальном возврате процессор реально выполняет:

```
add rsp, 40h      -> освобождает локальную область C
pop rbx           -> восстанавливает RBX caller-a
ret               -> берет return address со стека и кладет его в RIP
```

Virtual unwind хочет получить тот же итог, но без выполнения этих инструкций как пользовательского кода. Почему без выполнения? Потому что исключение могло произойти в середине функции, а до эпилога программа не дошла. Или система просто строит stack trace в дампе, где процесс уже остановлен. Или отладчик хочет понять цепочку вызовов, не продолжая выполнение.

Поэтому виртуальная раскрутка делает смысловую обратную операцию:

```
если функция выделила 40h байт стека, надо учесть это при вычислении caller RSP;
если функция сохранила RBX, надо прочитать сохраненное значение и положить его в CONTEXT.Rbx;
если на вершине восстановленного стека лежит return address, надо сделать его новым RIP;
если return address взят, надо поднять RSP еще на 8 байт.
```

Это не тот же набор инструкций, но цель та же: получить состояние, которое было бы после выхода из `C` в `B`.

7.5.5. Один шаг unwind: почему нельзя сразу думать “весь stack trace”

Отладчик показывает call stack как список:

```
C
B
A
main
```

Из-за этого кажется, что Windows где-то хранит готовый список активных функций. Но нормальная x64-модель так не работает. Стек вызовов восстанавливается шаг за шагом. Каждый шаг отвечает только на вопрос: “если сейчас мы в этой функции, кто нас вызвал?”

`RtlVirtualUnwind` не знает заранее всю историю программы. Она получает текущий `ControlPc`, текущий `ContextRecord`, `ImageBase`, `FunctionEntry` и unwind-данные текущей функции. На основе этого она получает предыдущий context. Потом внешний код снова делает lookup уже по новому `RIP`. Новый `RIP` может попасть в другой модуль, в другую секцию, в другую функцию, с другой `.pdata` и другой `.xdata`.

Так рождается цикл:

```
1. Взять текущий RIP.
2. Найти RUNTIME_FUNCTION для этого RIP.
3. Прочитать UNWIND_INFO.
4. Выполнить virtual unwind одного кадра.
5. Получить RIP caller-а.
6. Повторить для caller-а.
```

Это принципиально для будущего static -а. Он не должен мыслить как “одна большая функция построения стека”. Он должен уметь объяснить каждый переход отдельно:

```
Frame 0 -> Frame 1:
    RIP принадлежал такой-то функции.
    Найдена такая-то RUNTIME_FUNCTION.
    UNWIND_INFO говорит: stack allocation 0x40, saved RBX.
    Return address прочитан отсюда.
    Новый RIP такой-то.
```

Если каждый маленький переход доказан, весь stack trace становится проверяемым. Если один переход мутный, весь верхний стек выше него становится менее надежным.

7.5.6. Входные данные RtlVirtualUnwind: что нужно для одного перехода

Теперь посмотрим на входные данные не как на скучную сигнатуру, а как на роли участников.

`HandlerType` - тип обработчика, который интересует вызывающий код. Здесь могут использоваться значения, связанные с `UNW_FLAG_EHANDLER`, `UNW_FLAG_UHANDLER` и `UNW_FLAG_NHANDLER`. Смысл такой: unwind может идти не только ради построения стека, но и в контексте exception dispatch, где важно понять, есть ли у текущей функции exception handler или termination handler.

`ImageBase` - база загруженного модуля. Без нее нельзя превратить RVA из `RUNTIME_FUNCTION` или `UNWIND_INFO` в реальные адреса памяти. Если `UnwindInfoAddress` равен 0x3000, это не значит “адрес 0x3000 в процессе”. Это значит `ImageBase + 0x3000`.

`ControlPc` - управляющий адрес, по которому определяется текущая позиция в функции. Обычно это адрес, связанный с `RIP`. Именно по нему `RtlLookupFunctionEntry` нашла `RUNTIME_FUNCTION`. Но в деталях exception handling `ControlPc` может иметь тонкости около границ инструкции или return address, поэтому лучше понимать его как “адрес управления, который сейчас классифицируется как принадлежащий текущей функции”.

`FunctionEntry` - указатель на `RUNTIME_FUNCTION`. Он говорит, где функция начинается, где заканчивается и где лежит unwind-информация.

`ContextRecord` - тот самый `CONTEXT`, который будет изменяться. Это важно: функция не просто возвращает новый объект, она обновляет контекст. До вызова он описывает текущий кадр. После вызова он должен описывать caller-а.

`HandlerData` - выходной указатель на дополнительные данные обработчика, если у функции есть handler и текущая стадия требует его учитывать. Для `__C_specific_handler` это особенно важно, потому что после address handler-а в `UNWIND_INFO` лежат language-specific data, например scope-like данные.

`EstablisherFrame` - база фиксированного stack allocation для текущей функции. Это значение нужно language-specific handler-у, чтобы понимать текущий кадр. Термин сложный, но можно мыслить так: это опорная точка кадра, через которую handler связывает таблицы с живым stack frame.

`ContextPointers` - дополнительная структура, через которую можно получить указатели на места, откуда были восстановлены некоторые nonvolatile-регистры. Для обычного объяснения stack walking это не первый предмет, но для продвинутого анализа важно: иногда нужно знать не только “какое значение восстановлено”, но и “из какого места памяти оно было взято”.

Вся эта сигнатура говорит одну мысль: virtual unwind - это не чтение одной таблицы. Это операция на стыке адресов, PE-модуля, функции, `.xdata`, стека и машинного контекста.

7.5.7. Самый простой случай: leaf function без RUNTIME_FUNCTION

Начнем с самого простого случая. `RtlLookupFunctionEntry` не нашла запись для текущего `ControlPc`. В x64 unwind-модели это обычно трактуется как leaf function.

Leaf function - листовая функция. “Лист” здесь взят из дерева вызовов: функция находится на конце ветви и не вызывает другие функции. В контексте unwind еще важнее другое: такая функция не требует формального описания кадра, потому что не сделала действий, которые нужно сложно отменять. Она не сохранила nonvolatile-регистры, не выделила нетривиальный кадр, не имеет handler-а.

Для leaf-функции виртуальный выход очень похож на обычный `ret`:

```
new RIP = [RSP]
new RSP = RSP + 8
```

Почему 8? Потому что в x64 return address занимает 8 байт. Инструкция `call` положила 64-битный адрес возврата на стек. При виртуальном возврате мы читаем этот адрес и считаем, что `ret` его забрал.

Но это правило надо держать аккуратно. Для Windows это нормальная fallback-модель, если function entry нет. Для reverse engineer-а это не доказательство, что функция точно “хорошая leaf”. Это только предположение unwind-механизма. Если код на самом деле изменил `RSP`, сохранил регистры или сделал нестандартный кадр, но metadata нет, stack walking может сломаться.

То есть один и тот же факт имеет два чтения:

```
Для unwinder-а:
    записи нет -> применяем leaf-return правило.

Для реверсера:
    записи нет -> проверяем, действительно ли код похож на leaf.
```

Это разница между выполнением алгоритма и аналитическим доверием к результату.

7.5.8. Non-leaf function: почему без UNWIND_INFO уже нельзя

Теперь возьмем обычную non-leaf функцию. Non-leaf - “не листовая”, то есть функция, которая делает что-то, из-за чего простой `new RIP = [RSP]` уже недостаточен. Она могла вызвать другую функцию, выделить место на стеке, сохранить nonvolatile-регистры, использовать frame pointer или иметь обработчики.

Пример:

```
C:
    push rbx
    sub  rsp, 40h
    call risky
    add  rsp, 40h
    pop  rbx
    ret
```

Если исключение произошло внутри `C` после `sub rsp, 40h`, текущий `RSP` указывает не на return address. Между текущим `RSP` и адресом возврата лежит локальная область, а ниже или выше в зависимости от схемы сохранения лежит сохраненный `RBX`.

Если применить leaf-правило, unwinder прочитает `[RSP]` как return address. Но `[RSP]` может быть локальной переменной, временным spill slot, shadow space или просто неинициализированной областью. Получится ложный `RIP`, дальше lookup уйдет в мусор, и stack trace разрушится.

Поэтому для non-leaf функции нужна `UNWIND_INFO`. Она говорит:

```
Функция сохранила RBX через push.
Функция выделила 0x40 байт стека.
Чтобы выйти к caller-у, надо учесть оба действия.
```

Именно здесь видно, почему `.xdata` не является “лишней таблицей”. Без нее Windows не знает, где настоящий return address.

7.5.9. Тело функции: основной нормальный случай virtual unwind

Основной случай для ручного понимания - `ControlPc` находится в body функции, то есть не в прологе и не в эпилоге. `Body` - тело функции, основная область выполнения, где пролог уже построил кадр, а эпилог еще не начал его разбирать.

В этом случае можно считать, что все действия пролога уже выполнены. Если `UNWIND_INFO` говорит, что был `push rbx`, значит старый `RBX` уже лежит на стеке. Если говорит, что был `sub rsp, 40h`, значит текущий `RSP` уже уменьшен на 0x40. Если говорит, что установлен frame register, значит он уже установлен.

Ручная логика такая:

```
1. Берем текущий CONTEXT внутри функции.
2. Читаем UNWIND_CODE как описание уже выполненных действий пролога.
3. Обращаем эти действия в смысловом порядке.
4. Восстанавливаем nonvolatile-регистры.
5. Находим return address.
6. Делаем return address новым RIP.
7. Поднимаем RSP за return address.
```

Слово “обращаем” важно. Пролог строит кадр. Unwind разбирает кадр. Если пролог сделал `sub rsp, 40h`, то для выхода надо прибавить 0x40. Если пролог сделал `push rbx`, то для выхода надо восстановить `RBX` из стекового места и сдвинуть `RSP` так, будто `pop rbx` уже произошел. Но порядок этих обратных действий зависит от того, как именно рассчитываются offsets в unwind metadata.

На практике поэтому нельзя слепо вручную применять “сначала то, потом это” без понимания layout. Но для модели полезно помнить: unwind отменяет эффект пролога.

7.5.10. Пролог: почему частично построенный кадр нельзя раскручивать как полный

Если `ControlPc` находится внутри prolog, ситуация тоньше. Prolog - участок, где функция только строит кадр. Исключение или остановка могут произойти в середине этого строительства.

Например:

```
C:
    push rbx        ; offset 1
    push rsi        ; offset 2
    sub  rsp, 40h   ; offset 6
    ; тело
```

Если `RIP` указывает сразу после `push rbx`, но до `push rsi`, то на стеке уже лежит старый `RBX`, но старый `RSI` еще не сохранен, а `sub rsp, 40h` еще не выполнен. Если unwinder применит полное описание пролога, он будет думать, что сохранены оба регистра и выделены 0x40 байт. Это будет неправда.

Поэтому unwind в прологе должен учитывать только те операции, которые уже успели выполниться к текущему offset. Для этого в `UNWIND_CODE` есть `CodeOffset`. Он говорит, на каком смещении от начала функции завершилась соответствующая операция пролога.

Теоретический смысл такой:

```
Если операция пролога находится после текущего ControlPc, она еще не произошла.
Если операция пролога находится до текущего ControlPc, ее эффект уже есть в CONTEXT/стеке.
```

Именно поэтому `SizeOfProlog` и `CodeOffset` - не декоративные поля. Они позволяют Windows различать “функция уже построила кадр” и “функция еще строит кадр”. Для reverse engineer-а это важно при анализе crash dump-ов, где падение может произойти в самом начале функции, например в security cookie, stack probing, обращении к аргументу или даже внутри helper-а.

7.5.11. Эпилог: почему sometimes unwinder идет не назад, а вперед

Epilog - участок выхода из функции. Здесь функция уже разбирает кадр: возвращает `RSP`, восстанавливает регистры, делает `ret` или допустимый tail jump. Если `ControlPc` находится в эпилоге, ситуация отличается от тела.

В теле функции unwind может “отменить пролог” по metadata. В эпилоге часть отмены уже происходит прямо сейчас. Например:

```
    add  rsp, 40h
    pop  rbx
    ret
```

Если поток остановлен на `pop rbx`, значит `add rsp, 40h` уже выполнен. Текущий `RSP` уже другой. Если применить полное body-unwind правило, можно дважды учесть освобождение стека.

Поэтому x64 unwinder распознает эпилог и симулирует оставшуюся часть эпилога вперед. Это тонкий момент: в прологе unwind как бы отменяет уже выполненные действия; в эпилоге он смотрит на код вперед и доводит выход до состояния caller-а.

Почему можно распознавать эпилог по коду? Потому что Windows x64 накладывает на epilog строгие правила. Эпилог должен иметь ограниченную форму, чтобы unwinder мог

надежно его определить. Это уменьшает объем unwind-данных: не нужно хранить отдельную таблицу для каждого эпилога обычной функции, если можно распознать допустимую последовательность инструкций.

Для реверсера отсюда два вывода. Первый: эпилог - не обычный кусок body. Второй: если код выглядит как выход из функции, но нарушает epilog rules, unwinder может не распознать его как эпилог. Тогда stack walking или exception handling может пойти по другой ветке анализа.

7.5.12. Return address: главный мост между двумя кадрами

Return address - адрес возврата. Это адрес в caller-е, куда callee должна вернуться после выполнения. На x64 instruction `call` кладет return address на стек, а `ret` потом забирает его в `RIP`.

При virtual unwind return address играет роль моста между кадрами. Пока мы внутри `C`, текущий `RIP` находится в `C`. Чтобы получить caller context, нужно найти адрес возврата в `B` и сделать его новым `RIP`.

Важно: return address не лежит “где-то всегда по [RSP]”. Он лежит там, где он оказался после всех изменений `RSP` и сохранений. В leaf-функции он действительно может быть на вершине стека. В non-leaf функции текущий `RSP` может быть ниже, потому что функция выделила локальную область. Тогда return address нужно искать после виртуального освобождения этой области и восстановления layout-а.

Концептуально итог unwind такой:

```
caller_RIP = return_address
caller_RSP = address_after_return_address
```

Почему `caller_RSP` указывает “после return address”? Потому что обычный `ret` снимает return address со стека. Он читает 8 байт по текущему `RSP`, кладет их в `RIP`, затем увеличивает `RSP` на 8. Virtual unwind должен смоделировать этот эффект.

Если return address поврежден, вся цепочка выше ломается. Unwind может идеально восстановить `RSP`, но если в месте return address лежит мусор из-за stack corruption, новый `RIP` будет мусором. Поэтому хороший анализатор должен различать: metadata могла быть корректной, а stack memory повреждена.

7.5.13. Восстановление nonvolatile-регистров: зачем это нужно stack trace-y

Может показаться, что для построения stack trace нужны только `RIP` и `RSP`. Но x64 unwind восстанавливает также nonvolatile-регистры. Почему? Потому что caller context - это не только адрес возврата. Caller мог держать важные значения в `RBX`, `RBP`, `R12-R15`. По calling convention callee обязана сохранить эти значения. Если исключение прервало callee до обычного восстановления, unwinder должен восстановить их сам.

Например:

```
B:
    mov  rbx, rcx      ; B хранит важный указатель в RBX
    call C
    mov  rax, [rbx]    ; после возврата B ожидает прежний RBX
```

Если `C` использовала `RBX`, она должна была сохранить старое значение:

```
C:
    push rbx
    mov  rbx, rdx
    ; исключение где-то здесь
```

В момент исключения `RBX` в `CONTEXT` содержит значение, которое использовала `C`, а не старый `RBX` функции `B`. Если мы хотим получить context `B`, нужно восстановить `RBX` из места сохранения. Иначе caller context будет ложным.

Это особенно важно не только для красивого stack trace, но и для language-specific handlers. Некоторые обработчики, фильтры, cleanup-логика и отладочные механизмы могут зависеть от корректного контекста. Если nonvolatile-регистр восстановлен неправильно, анализ выше по стеку может ошибиться даже при правильном `RIP`.

7.5.14. UWOP_PUSH_NONVOL: как виртуально отменяется push

Теперь возьмем конкретную операцию, которую мы уже видели: `UWOP_PUSH_NONVOL`. Она соответствует сохранению nonvolatile-регистра через push в прологе.

Код:

```
push rbx
```

Смысл `push rbx` на x64:

```
RSP = RSP - 8
[RSP] = RBX
```

То есть стек растет вниз, к меньшим адресам, и старое значение `RBX` записывается на новую вершину стека. Когда функция потом выходит, она делает обратное:

```
pop rbx
```

Смысл `pop rbx`:

```
RBX = [RSP]
RSP = RSP + 8
```

Virtual unwind должен получить эффект, как будто этот `pop rbx` произошел. Значит, если при текущем layout-е сохраненный `RBX` находится по адресу, соответствующему месту push-а, unwinder читает это значение в `CONTEXT.Rbx` и сдвигает виртуальный `RSP` дальше.

Но здесь нельзя забывать порядок. Если функция сделала несколько push-ей и потом `sub rsp, ...`, текущий `RSP` может быть далеко от сохраненного `RBX`. Поэтому unwind не просто выполняет “pop с текущего RSP” наугад. Он использует схему кадра, полученную из всех unwind-кодов. Смысловой итог тот же: восстановить регистр из его сохраненного места.

Для static -а важно показывать это не как “UWOP 0”. Нужно объяснять:

```
UWOP_PUSH_NONVOL RBX:
    Пролог сохранил RBX на стеке.
    При virtual unwind значение RBX caller-а должно быть восстановлено из stack slot.
    Если slot недоступен или выходит за пределы стека, stack walk становится ненадежным.
```

7.5.15. UWOP_ALLOC_SMALL и UWOP_ALLOC_LARGE: как возвращается RSP

Операции allocation описывают выделение места на стеке. `ALLOC` здесь от allocation - выделение. В прологе это обычно выражается через `sub rsp, N`.

Пример:

```
sub rsp, 40h
```

Смысл:

```
RSP = RSP - 0x40
```

Функция выделила 0x40 байт под локальные нужды: temporary values, spill slots, shadow-related layout, alignment, local buffers и другое. При обычном выходе эпилог делает:

```
add rsp, 40h
```

Virtual unwind должен учесть этот эффект:

```
виртуальный RSP = текущий RSP + 0x40
```

`UWOP_ALLOC_SMALL` используется для небольших выделений, размер которых можно компактно закодировать в `OpInfo`. `UWOP_ALLOC_LARGE` используется для более крупных выделений, где требуется дополнительный slot или slots для размера. Для теории важнее не форма кодирования, а смысл: эти операции говорят unwinder-у, на сколько нужно поднять `RSP`, чтобы выйти из локальной области текущей функции.

Ошибка новичка: считать stack allocation “локальными переменными” и больше ничем. На деле область, выделенная через `sub rsp, N`, может содержать локальные переменные, временные сохранения регистров, выравнивание, home slots, security cookie, spill slots, данные для вызовов. Unwind не интересуется именами переменных. Ему важно только: `RSP` был уменьшен на N, значит для caller context его надо увеличить на N.

7.5.16. UWOP_SAVE_NONVOL: сохранение регистра не через push

Nonvolatile-регистр можно сохранить не только через `push`. Компилятор может выделить стековую область, а затем записать регистр по смещению:

```
sub rsp, 80h
mov [rsp+40h], rbx
```

Такой способ описывается через `UWOP_SAVE_NONVOL` или близкие операции для far offset. Смысл: “старое значение nonvolatile-регистра сохранено в stack slot по определенному offset”.

Почему это нужно? Потому что не все сохранения удобно делать через цепочку push. При оптимизациях, больших кадрах, выравнивании, сохранении многих регистров или XMM-регистров компилятор может выбирать layout, где сохраненные значения лежат в конкретных ячейках внутри выделенной области.

Для virtual unwind это означает:

```
Найти stack slot по offset.
Прочитать оттуда старое значение регистра.
Записать его в CONTEXT соответствующего nonvolatile-регистра.
```

Важно: offset считается не “как попало”, а по правилам unwind-формата относительно определенного состояния stack pointer или frame base. Поэтому ручной анализ требует аккуратности. Нельзя увидеть `UWOP_SAVE_NONVOL RBX offset 0x40` и сразу механически прибавить 0x40 к любому `RSP`, не понимая, на какой базе этот offset интерпретируется.

Практическая мысль для реверса: если metadata говорит, что `RBX` сохранен по stack offset, а в коде нет соответствующей записи в прологе, это сильный сигнал несоответствия. Если запись есть, но offset другой, это тоже finding. Если stack slot попадает в unreadable memory в дампе, восстановление регистра будет неполным.

7.5.17. Frame register: когда RSP уже не удобная база

Frame register - регистр кадра. Он нужен, когда функция хочет иметь стабильную базу для обращения к своему кадру. Часто это `RBP`, но не нужно думать, что `RBP` всегда является frame pointer. На x64 `RBP` может быть обычным nonvolatile-регистром, если функция не использует его как базу кадра.

Почему стабильная база вообще нужна? Потому что `RSP` может меняться. Функция может делать динамическое выделение через `alloca`, вызывать helpers, временно

двигать стек или иметь сложный layout. Если обращаться ко всем локальным данным через `RSP`, смещения могут стать неудобными. Frame register дает точку, которая остается стабильной.

В `UNWIND_INFO` есть поля `FrameRegister` и `FrameOffset`. Они говорят unwinder-у, использует ли функция frame register и как он связан с областью кадра. Для ручного анализа это значит: не вся геометрия кадра может считаться от текущего `RSP`. Иногда нужно учитывать frame base.

Примерная идея:

```
push rbp
sub  rsp, 80h
lea  rbp, [rsp+20h]
```

Здесь `RBP` становится не просто сохраненным регистром, а базой, от которой функция может обращаться к данным. Unwind должен понимать, как восстановить caller context, если текущий `RSP` не является единственной удобной координатой.

Для static -а это важный момент. Если frame register заявлен, отчет должен показать не только “есть RBP”, а объяснить:

```
Функция использует frame register.
Это меняет способ интерпретации части stack offsets.
Для корректного unwind нужно учитывать связь frame register с RSP.
```

7.5.18. HandlerData и EstablisherFrame: почему RtlVirtualUnwind не только про стек

Если бы `RtlVirtualUnwind` использовалась только для stack walking, можно было бы думать только о `RIP`, `RSP` и регистрах. Но она также участвует в exception dispatch. Поэтому у нее есть связь с handler-ами.

Если у функции в `UNWIND_INFO` установлен `UNW_FLAG_EHANDLER`, это значит, что у функции есть exception handler. Если установлен `UNW_FLAG_UHANDLER`, значит есть termination/unwind handler. В хвосте `UNWIND_INFO` после unwind-кодов и выравнивания лежит handler RVA, а после него - handler data, формат которой зависит от language-specific handler.

`HandlerData` - выходной указатель на эти дополнительные данные. Для `__C_specific_handler` это может быть таблица, описывающая protected ranges, filters и handler blocks. Windows не должна сама понимать все детали C SEH. Она передает данные language-specific handler-у.

`EstablisherFrame` - значение, которое помогает handler-у связать language-specific данные с конкретным кадром. Слово establisher можно понимать как “тот, кто установил обработчик”. Frame - кадр. То есть это кадр, в котором данный handler установлен и рассматривается.

Почему это важно в разделе про virtual unwind? Потому что при поиске exception handler-а Windows не просто поднимается по стеку. На каждом кадре она может спросить: “у этой функции есть handler? если да, вот ему данные, пусть решит”. `RtlVirtualUnwind` помогает не только перейти к caller-у, но и обнаружить handler-слой текущего кадра.

7.5.19. Первый ручной пример: push rbx, sub rsp, crash

Возьмем простой пример. Пусть функция выглядит так:

```
B:
    ; вызывает C
    call C
    ; return address указывает сюда
```

```
C:
    push rbx
    sub  rsp, 40h
    mov  rax, [rcx]     ; здесь исключение
    add  rsp, 40h
    pop  rbx
    ret
```

Представим, что исключение произошло на `mov rax, [rcx]`. Мы находимся в body функции `C`: пролог уже выполнен, эпилог еще не начался.

Текущий `CONTEXT` условно:

```
RIP = адрес mov rax, [rcx] внутри C
RSP = значение после push rbx и sub rsp, 40h
RBX = текущее значение, которое использует C
```

На стеке условно:

```
[RSP + 00h ... 3Fh]  локальная область C размером 0x40
[RSP + 40h]          сохраненный RBX функции B
[RSP + 48h]          return address в B
```

Теперь мысленно делаем virtual unwind.

Первое: `UWOP_ALLOC_SMALL 0x40` говорит, что локальную область надо убрать. Значит, виртуально:

```
RSP = RSP + 0x40
```

Теперь `RSP` указывает на сохраненный `RBX`.

Второе: `UWOP_PUSH_NONVOL RBX` говорит, что старый `RBX` был сохранен через push. Значит:

```
RBX = [RSP]
RSP = RSP + 8
```

Теперь `RSP` указывает на return address.

Третье: виртуальный `ret`:

```
RIP = [RSP]
RSP = RSP + 8
```

Итоговый `CONTEXT` теперь описывает `B`, а не `C`.

Эта схема выглядит простой, но она фундаментальна. В более сложных функциях будет больше saved registers, frame register, large allocation, XMM save, handler data. Но цель останется той же: восстановить caller context.

7.5.20. Почему порядок операций нельзя понимать слишком механически

В предыдущем примере мы изложили unwind как последовательность: убрать allocation, восстановить RBX, взять return address. Это удобно для обучения. Но в реальном формате `UNWIND_CODE` порядок хранения записей не совпадает с человеческим порядком чтения пролога. Массив отсортирован в обратном порядке относительно выполнения пролога: ближе к телу функции идут раньше.

Это сделано не для красоты, а потому что unwind по смыслу часто отменяет действия пролога в обратном порядке. Последняя важная операция пролога становится первой для раскрутки.

Однако при ручном анализе нельзя превращать это в простое правило “читай массив сверху вниз и делай pop”. Некоторые операции используют дополнительные slots. Некоторые offsets интерпретируются относительно конкретного состояния. Frame register меняет базу. Epilog и prolog cases требуют особой логики.

Поэтому правильная умственная модель такая:

```
UNWIND_CODE не является программой, которую надо тупо исполнить.
UNWIND_CODE является описанием эффектов пролога.
RtlVirtualUnwind применяет эти эффекты по правилам x64 unwind procedure.
```

Для будущего static -а это значит: отчет должен быть не просто “операция 0, операция 1”. Он должен переводить operations в смысл:

```
Сохранен RBX.
Выделено 0x40 байт.
Frame register не используется.
Return address ожидается после восстановления stack pointer.
```

7.5.21. Второй ручной пример: сохранение RBX через stack slot

Теперь пример чуть сложнее:

```
C:
    sub  rsp, 80h
    mov  [rsp+50h], rbx
    mov  [rsp+58h], rsi
    ; тело
```

Здесь `RBX` и `RSI` сохранены не через push, а через записи в уже выделенную область. Это может быть описано через `UWOP_SAVE_NONVOL`.

Текущий `RSP` в body уже после `sub rsp, 80h`. Сохраненные значения лежат внутри этой области:

```
[RSP + 50h] = старый RBX
[RSP + 58h] = старый RSI
```

Virtual unwind должен:

```
RBX = [RSP + 50h]
RSI = [RSP + 58h]
RSP = RSP + 80h
RIP = [RSP]
RSP = RSP + 8
```

Смысл отличается от push-схемы. При push сохраненный регистр лежит прямо в последовательности стекового роста. При save-to-slot регистр лежит внутри выделенного кадра по offset. Но цель одинаковая: восстановить nonvolatile state caller-a.

Для реверса это место важно: один и тот же high-level факт “функция сохранила RBX” может выглядеть в коде по-разному. Поэтому анализатор должен понимать не только `push rbx`, но и `mov [rsp+offset], rbx`, `mov [rbp-offset], rbx` и другие формы, допустимые для unwind metadata.

7.5.22. Третий пример: frame pointer и стабильная база

Теперь пример с frame pointer:

```
C:
    push rbp
    sub  rsp, 90h
    lea  rbp, [rsp+20h]
    mov  [rbp+60h], rbx
    ; тело
```

Здесь `RBP` используется как frame register. Функция не просто сохранила старый `RBP`, но и назначила новый `RBP` базой для доступа к кадру.

Чем это усложняет ручной анализ? Если ты будешь смотреть только на текущий `RSP`, часть обращений может казаться странной. Код использует `[rbp+60h]`, а не `[rsp+...]`. Чтобы понять layout, нужно знать связь между `RBP` и `RSP`, заданную прологом и описанную в `UNWIND_INFO`.

В normal body case unwinder может использовать frame register information, чтобы правильно вычислять establisher frame и stack locations. Для нас сейчас главное: frame register - это не украшение. Это часть координатной системы кадра.

Типичная ошибка новичка: увидеть `push rbp` и автоматически решить, что это классический x86-style frame pointer `mov rbp, rsp`. На Windows x64 это не всегда так. Надо смотреть, есть ли `UWOP_SET_FPREG`, какое поле `FrameRegister`, какой `FrameOffset`, какие обращения к стеку реально используются.

7.5.23. Как RtlVirtualUnwind участвует в search phase

Теперь вернемся к exception handling. В search phase Windows ищет обработчик. Она начинает с текущего кадра. Если у текущей функции есть `UNW_FLAG_EHANDLER` и `ControlPc` находится в подходящей зоне body, вызывается language-specific handler. Если handler говорит continue search, нужно подняться к caller-у.

Вот здесь вызывается virtual unwind:

```
Кадр C не обработал исключение.
Нужно проверить кадр B.
Для этого надо получить CONTEXT кадра B.
RtlVirtualUnwind превращает context C в context B.
```

После этого цикл повторяется:

```
Новый RIP находится внутри B.
RtlLookupFunctionEntry ищет RUNTIME_FUNCTION для B.
Читается UNWIND_INFO функции B.
Если есть handler, он получает шанс.
Если нет, снова virtual unwind.
```

То есть `RtlVirtualUnwind` не решает, обработано исключение или нет. Она дает диспетчеру возможность перейти к следующему кадру, где это решение будет проверяться. Это как лестница: handler logic решает, открылась ли дверь на текущем этаже, а virtual unwind позволяет подняться на этаж выше.

7.5.24. Как RtlVirtualUnwind участвует в unwind phase

В unwind phase ситуация другая. Обработчик уже найден. Теперь нужно реально раскрутить промежуточные кадры и выполнить cleanup/termination logic. Здесь `UNW_FLAG_UHANDLER` и termination handlers становятся особенно важны.

Search phase спрашивает:

```
Кто хочет обработать исключение?
```

Unwind phase делает:

```
Мы уже знаем, куда идем.
Теперь надо корректно покинуть кадры между местом исключения и обработчиком.
```

В этой фазе virtual unwind снова нужен для движения по кадрам, но смысл движения уже не только “поиск”. Система может вызывать termination handlers, `__finally`, C++ destructors или другие cleanup-механизмы, в зависимости от языка и runtime. Если кадр имеет `UNW_FLAG_UHANDLER`, его handler может быть вызван при раскрутке.

Для reverse engineer-а это объясняет, почему один и тот же `UNWIND_INFO` может участвовать в разных сценариях. Он нужен и чтобы найти handler, и чтобы выполнить корректный выход. Поэтому нельзя думать, что unwind data - это только “для crash stack”. Это часть реального control flow при исключениях.

7.5.25. Почему обычный stack trace в отладчике зависит от той же механики

Когда ты в отладчике видишь stack trace, кажется, что это отдельная функция отладчика. Но на x64 хороший stack trace опирается на ту же unwind-модель. Отладчик берет текущий context потока и шаг за шагом восстанавливает caller contexts.

Если `.pdata` отсутствует там, где нужна, отладчик может применить leaf-правило или эвристику. Если `.xdata` повреждена, он может неправильно поднять `RSP`. Если return address затерт, stack trace оборвется или покажет мусор. Если функция inline-нута, отдельного кадра может вообще не быть, потому что call не существовал в машинном коде как отдельный вызов.

Это важно психологически. Stack trace - не истина от Бога. Это результат алгоритма. Алгоритм опирается на контекст, стековую память и metadata. Если эти слои согласованы, stack trace надежен. Если нет, он становится гипотезой.

Будущий static должен показывать уровень доверия:

```
Frame 3 восстановлен по валидной RUNTIME_FUNCTION и согласованной UNWIND_INFO.
Frame 4 восстановлен leaf-правилом без function entry.
Frame 5 имеет return address вне известных модулей.
```

Такой отчет сразу учит пользователя мыслить слоями, а не слепо верить списку функций.

7.5.26. Контекстные указатели: зачем знать, откуда восстановлен регистр

`ContextPointers` - тема более тонкая, но для фундаментальной книги ее нужно хотя бы объяснить. Иногда важно не только восстановить значение регистра, но и знать, где было сохраненное значение. Например, если `RBX` восстановлен из `[RSP+50h]`, анализатору может быть полезно показать этот адрес.

Почему? Потому что это дает доказательство. Если регистр восстановлен из памяти, можно проверить, доступна ли эта память, лежит ли она в пределах стека, не выглядит ли значение подозрительно, не пересекается ли stack slot с другой структурой.

Пример отчета:

```
RBX restored from stack address 000000AABBCC1230.
Address is inside current thread stack.
Value points into module app.exe .rdata.
```

Это намного полезнее, чем просто:

```
RBX = 00007FF7....
```

Для обычного читателя API это может казаться лишним, но для reverse engineering и crash analysis это ценная линия доказательств. Мы не просто получаем результат. Мы можем показать происхождение результата.

7.5.27. Когда metadata корректная, но stack memory повреждена

Нужно отдельно разделить две поломки: поломка metadata и поломка стека.

Metadata может быть корректной. `.pdata` правильно покрывает функцию. `UNWIND_INFO` правильно описывает пролог. `UWOP_*` совпадают с инструкциями. Но stack memory может быть повреждена. Например, буфер переполнил локальную область и затер return address или сохраненный `RBX`.

В таком случае unwinder делает правильные вычисления, но читает неправильные данные. Он честно идет к месту, где должен лежать return address, а там мусор. Или он читает сохраненный `RBX`, а там уже перезаписанный pattern.

Это дает важную диагностическую мысль:

```
Если stack walk сломался, не всегда виновата .xdata.
Иногда .xdata правильная, а стек поврежден.
```

И наоборот:

```
Если стек выглядит нормальным, но unwind прыгает не туда, возможно, metadata противоречит коду.
```

Хороший анализатор должен уметь разделять эти случаи. Для этого он проверяет не только результат, но и путь: какая function entry, какая unwind info, какие stack addresses читались, какие значения получены.

7.5.28. Когда stack memory нормальная, но metadata лжет

Обратная ситуация: stack memory содержит нормальный return address, но metadata описывает неправильный размер allocation. Например, реальный код сделал:

```
sub rsp, 80h
```

а `UNWIND_INFO` почему-то говорит:

```
ALLOC 40h
```

Тогда virtual unwind поднимет `RSP` только на 0x40 и прочитает return address не там. Он может взять локальную переменную или сохраненный указатель как новый `RIP`. Stack trace сломается, хотя настоящий return address лежал рядом и был цел.

Для reverse engineering это особенно интересно в контексте packers, obfuscation, manual code generation, malformed binaries. Несоответствие кода и metadata может быть случайной ошибкой, следствием нестандартной сборки или намеренным приемом против анализа.

static должен формулировать finding аккуратно:

```
UNWIND_INFO stack allocation does not match prolog instruction.
Expected allocation from code: 0x80.
Encoded allocation in metadata: 0x40.
Virtual unwind may compute wrong caller RSP.
Severity: high for stack walking reliability.
```

Такой finding не говорит сразу “malware”. Он говорит, какой контракт нарушен и к чему это ведет.

7.5.29. Ошибка координат: VA, RVA и file offset внутри virtual unwind

Даже когда человек понимает саму идею unwind, он часто ломается на адресах. Нужно постоянно различать `VA`, `RVA` и `file offset`.

`ControlPc` в живом процессе - это virtual address. `ImageBase` - тоже virtual address базы модуля. `RUNTIME_FUNCTION.BeginAddress`, `EndAddress`, `UnwindInfoAddress` - image-relative values, то есть RVA. Чтобы получить адрес `UNWIND_INFO` в памяти, нужно сделать:

```
UnwindInfoVA = ImageBase + UnwindInfoAddress
```

Чтобы прочитать тот же `UNWIND_INFO` из файла на диске, нельзя просто идти по offset `UnwindInfoAddress`. Надо перевести RVA в file offset через section table.

Почему это важно именно для `RtlVirtualUnwind`? Потому что в live-процессе Windows работает с memory image, а статический анализатор работает с файлом. Если твой

инструмент хочет повторить unwind статически, он должен правильно имитировать адресную дисциплину loader-a.

Ошибка координат может выглядеть как битая `.xdata`, хотя на самом деле parser просто прочитал не те байты. Поэтому перед любым выводом о malformed unwind info нужно убедиться: адрес был переведен правильно.

7.5.30. Мини-модель ручного RtlVirtualUnwind

Теперь соберем все в одну ручную модель. Она не заменяет официальную процедуру, но помогает думать.

```
Дано:
    текущий CONTEXT
    текущий RIP / ControlPc
    текущий RSP

Шаг 1:
    Найти module, которому принадлежит ControlPc.

Шаг 2:
    Получить ImageBase module.

Шаг 3:
    Вычислить RVA ControlPc.

Шаг 4:
    Найти RUNTIME_FUNCTION, чей диапазон покрывает RVA.

Шаг 5:
    Если FunctionEntry нет:
        применить leaf-rule:
            RIP = [RSP]
            RSP = RSP + 8
        остановить один шаг.

Шаг 6:
    Если FunctionEntry есть:
        прочитать UNWIND_INFO по ImageBase + UnwindInfoAddress.

Шаг 7:
    Определить позицию ControlPc:
        prolog
        body
        epilog

Шаг 8:
    Если epilog:
        симулировать оставшуюся часть epilog.

Шаг 9:
    Если prolog:
        отменить только уже выполненные операции prolog.

Шаг 10:
    Если body:
        применить полное unwind-описание кадра.

Шаг 11:
    Восстановить nonvolatile registers из stack slots.

Шаг 12:
    Найти return address и сделать его новым RIP.

Шаг 13:
    Обновить RSP как после ret.
```

```
Итог:
    CONTEXT теперь описывает caller frame.
```

Эта модель должна быть в голове перед следующим разделом. Потому что SEH-логику мы будем искать именно на этой основе: кадр за кадром, функция за функцией, metadata за metadata.

7.5.31. Почему это нужно будущему static -y

Твой будущий static не должен просто парсить PE и печатать поля. Если он хочет быть полезным, он должен объяснять переходы. `RtlVirtualUnwind` - идеальный пример такого перехода.

Плохой отчет:

```
FunctionEntry found. Unwind OK.
```

Хороший отчет:

```
ControlPc belongs to app.exe RVA 0x12345.
Matched RUNTIME_FUNCTION [0x12000, 0x12480).
UnwindInfoAddress = 0x30020.
UNWIND_INFO describes:
    push rbx
    alloc 0x40
Current RSP = 0x000000AABBCC1000.
After allocation undo: RSP = 0x000000AABBCC1040.
Restored RBX from 0x000000AABBCC1040.
Return address read from 0x000000AABBCC1048.
Caller RIP = app.exe+0x11890.
Confidence: high, metadata and code agree.
```

Такой отчет не просто показывает результат. Он учит. Он дает человеку возможность проверить каждый шаг. Он превращает unwind из черного ящика в объяснимую процедуру.

И это совпадает с целью книги. Мы не учим “нажми кнопку в отладчике”. Мы строим фундамент, на котором можно будет написать инструмент, понять чужой инструмент, проверить дизассемблер, анализировать crash dump и видеть, где Windows доверяет таблицам, где стеку, а где вынуждена гадать.

7.5.32. Типичные ошибки при ручном повторении unwind

Первая ошибка - думать, что `RSP` всегда указывает на return address. Это верно только для простых leaf-like ситуаций. В non-leaf функции текущий `RSP` часто указывает на локальную область.

Вторая ошибка - читать `UnwindInfoAddress` как file offset. Это RVA. Для live-процесса нужен `ImageBase + RVA`; для файла нужен перевод через section table.

Третья ошибка - думать, что `UNWIND_CODE` хранит инструкции. Он хранит unwind-значимые эффекты пролога, а не весь prolog и не весь asm.

Четвертая ошибка - забывать prolog/epilog special cases. Если `ControlPc` находится в прологе или эпилоге, body-логика может быть неверной.

Пятая ошибка - игнорировать nonvolatile-регистры. Stack trace может требовать не только `RIP` и `RSP`; корректный caller context включает восстановленные сохраняемые регистры.

Шестая ошибка - считать любой сбой stack walk доказательством плохой metadata. Иногда виноват поврежденный стек. Иногда return address затерт. Иногда функция leaf. Иногда код динамический и function table зарегистрирована runtime-ом.

Седьмая ошибка - не отделять результат от доверия к результату. Unwinder может выдать caller context. Но reverse engineer должен спросить: насколько этот context доказан?

7.5.33. Связь с поиском SEH-логики

Следующий раздел будет про поиск SEH-логики в бинаре. Теперь мы готовы к нему лучше. Почему? Потому что SEH на x64 живет не отдельно от unwind. Чтобы найти frame-based handler, Windows должна идти по кадрам. Чтобы идти по кадрам, она должна virtual unwind-ить текущий context. Чтобы virtual unwind работал, нужны `.pdata` и `.xdata`.

Если функция имеет `UNW_FLAG_EHANDLER`, это еще не значит, что handler будет немедленно выполнен. Сначала нужно понять, где находится `ControlPc`: prolog, body или epilog. Нужно получить handler RVA. Нужно получить `HandlerData`. Нужно вызвать language-specific handler. Если он сказал continue search, нужно сделать virtual unwind к caller-у.

То есть поиск SEH-логики держится на той же лестнице:

```
RIP -> RUNTIME_FUNCTION -> UNWIND_INFO -> handler flag -> language-specific handler -> decision -> RtlVi
```

Именно поэтому мы так долго строили фундамент. Без него `__C_specific_handler` кажется непонятной магией. С ним он становится обычным участником маршрута: Windows дошла до кадра, увидела handler flag, передала данные обработчику языка, получила решение, а если решение “искать дальше”, поднялась к caller-у через virtual unwind.

7.5.34. Итог раздела

`RtlVirtualUnwind` - это не просто API из `winnt.h`. Это формальная операция перехода от текущего машинного контекста к контексту вызывающей функции. Она не исполняет пользовательский код назад. Она использует `RUNTIME_FUNCTION`, `UNWIND_INFO`, `UNWIND_CODE`, stack memory и текущий `CONTEXT`, чтобы вычислить, каким был caller context.

Для leaf-функции переход может быть простым: взять return address с вершины стека и поднять `RSP` на 8. Для non-leaf функции нужна unwind metadata. В body применяются эффекты полного пролога. В prolog учитываются только уже выполненные операции. В epilog симулируется оставшаяся часть выхода. Nonvolatile-регистры восстанавливаются из мест, где функция их сохранила. Return address становится новым `RIP`. `RSP` обновляется как после `ret`.

Для reverse engineering главное не “знать название функции”, а понимать доказательную цепочку. Хороший анализ показывает: какой адрес был исходным, какая function entry найдена, какая unwind info прочитана, какие операции применены, откуда восстановлены регистры, откуда взят return address, какой caller context получился и насколько этому можно доверять.

Следующий раздел естественно продолжит эту линию. Теперь, когда мы умеем мысленно подниматься по кадрам, можно искать SEH-логику в бинаре: `UNW_FLAG_EHANDLER`, `__C_specific_handler`, handler data, protected ranges, filters и handler blocks.

Техническая база раздела 7.5

Раздел опирается на официальную документацию Microsoft по `RtlVirtualUnwind`, где функция описана как получение invocation context функции, предшествующей текущему function context; на документацию по x64 exception handling, где описаны `RUNTIME_FUNCTION`, `UNWIND_INFO`, `UNWIND_CODE`, prolog/body/epilog cases и unwind procedure; на документацию по x64 prolog and epilog, где объяснены ограничения эпилогов для надежного unwinding-а; и на x64 calling convention, где описаны nonvolatile-регистры, stack alignment и unwindability nonleaf-функций. В тексте эти сведения пересобраны в учебную модель для reverse engineering и будущего static -а.

7.6. Как найти SEH-логику в бинаре: UNW_FLAG_EHANDLER, __C_specific_handler, protected range, filter и handler block

В предыдущем разделе мы научились мысленно повторять один шаг `RtlVirtualUnwind`. Это был важный момент: мы перестали воспринимать stack trace как готовый список, который Windows где-то хранит, и увидели более честную модель. Есть текущий `CONTEXT`, в нем есть `RIP` и `RSP`, по `RIP` находится функция, по функции находится `UNWIND_INFO`, а затем unwind-данные помогают получить caller context. Так Windows, отладчик или наш будущий static поднимаются по стеку кадр за кадром.

Теперь можно вернуться к SEH, но уже не на уровне общих слов. Раньше мы говорили, что SEH - это Structured Exception Handling, структурированная обработка исключений Windows. Мы уже разбирали, что на x64 она не похожа на старую x86-картинку с цепочкой записей в стеке. В x64 frame-based SEH опирается на табличную модель: `.pdata` дает диапазон функции, `.xdata` дает `UNWIND_INFO`, `UNWIND_INFO` может указать language-specific handler, а уже этот handler разбирает языковые данные, созданные компилятором.

В этом разделе мы будем делать следующий практический шаг: как увидеть SEH-логику в бинаре. Но слово “практический” здесь не означает “быстро нажали в IDA и побежали дальше”. Наоборот, нам нужно замедлиться. Потому что SEH-логика в x64-бинаре не лежит одним очевидным куском, как строка `__try` в исходнике. Она размазана по нескольким слоям: машинный код в `.text`, таблица функций в `.pdata`, unwind-описание в `.xdata`, флаги `UNWIND_INFO`, адрес `__C_specific_handler`, language-specific data, filter-код, handler block, а также живой `ContextRecord` во время исключения.

Если смотреть только на один слой, легко ошибиться. Если смотреть только на `.pdata`, ты увидишь функцию, но не поймешь, есть ли в ней `__try`. Если смотреть только на `__C_specific_handler`, ты увидишь системно-компиляторный helper, но не увидишь пользовательский `__except`. Если смотреть только на handler block в коде, ты можешь не понять, как туда вообще попадает управление. Поэтому задача раздела - собрать карту.

Короткая формула будет такая:

```
SEH-логика в x64-бинаре ищется не одной стрелкой.
Она восстанавливается как связь:

RIP внутри функции
 -> RUNTIME_FUNCTION в .pdata
 -> UNWIND_INFO в .xdata
 -> UNW_FLAG_EHANDLER
 -> language-specific handler, часто __C_specific_handler
 -> handler data / scope-like table
 -> protected range
 -> filter expression
 -> exception handler block
```

И теперь каждое слово в этой цепочке нужно снова разжевать, потому что дальше именно на ней будет строиться анализ исключений, обфускации и будущего static -а.

7.6.1. Почему SEH-логика не равна одному адресу handler-а

Новичок часто хочет найти “адрес обработчика”. Желание понятное. В исходном коде есть видимый блок:

```
__try {
    risky_code();
}
```

```
__except (EXCEPTION_EXECUTE_HANDLER) {
    recover();
}
```

На уровне исходника кажется, что есть защищенный блок и есть обработчик. Значит, в бинаре, вроде бы, должен быть один адрес: “вот сюда прыгнуть при исключении”. Но x64 SEH устроен более многослойно.

Во-первых, есть сам `__try`-диапазон. Это не функция и не отдельная инструкция. Это участок машинного кода, для которого компилятор говорит: если исключение произошло пока управление находилось внутри этого диапазона, надо рассматривать такой-то фильтр и такой-то обработчик. Такой диапазон удобно называть protected range.

`Protected range` переводится как “защищенный диапазон”. Слово protected здесь не про защиту от вирусов, а про то, что этот участок находится под действием exception handler-а. Range - диапазон адресов. В x64 это обычно диапазон RVA или offsets внутри функции: от начала защищенного участка до конца защищенного участка.

Во-вторых, есть filter expression. Это выражение-фильтр после `__except(...)`. Оно не является handler block. Оно решает, что делать с исключением: обработать, продолжить поиск или попробовать продолжить выполнение.

В-третьих, есть exception handler block. Это тело после `__except`, то есть код, который выполняется, если filter решил `EXCEPTION_EXECUTE_HANDLER`.

В-четвертых, есть language-specific handler. Для MSVC C SEH на x64 это часто `__C_specific_handler`. И вот это особенно важно: `__C_specific_handler` - не пользовательский обработчик. Это не тот код, который ты написал внутри `__except { ... }`. Это помощник компилятора и runtime-а, который знает, как читать специальные данные компилятора.

Если перепутать эти четыре уровня, анализ сломается. Реверсер увидит `__C_specific_handler` и подумает: “Вот он, handler”. На самом деле это только вход в механизм. Пользовательский смысл лежит глубже: в таблице областей, фильтрах и jump target-ах.

7.6.2. Что значит frame-based SEH именно на x64

Термин frame-based exception handling мы уже встречали, но здесь он становится практическим. `Frame` - кадр стека, состояние одного активного вызова функции. `Based` - основанный. Frame-based SEH - это обработка исключений, при которой система рассматривает кадры текущего стека: текущая функция, ее caller, следующий caller и так далее.

Но на x64 кадр не распознается просто по красивой цепочке `RBP`. В обычном оптимизированном коде `RBP` может вообще не быть классическим frame pointer-ом. Поэтому x64 frame-based SEH не может зависеть от “пройдемся по RBP”. Оно зависит от function tables и unwind metadata.

То есть “frame-based” на x64 означает не “в стеке лежит готовая SEH-запись”, а “Windows может восстановить кадр через таблицы, а потом спросить у этого кадра, есть ли handler”.

Это тонкий, но центральный сдвиг мышления:

```
x86-ассоциация новичка:
    SEH record лежит в стеке.

x64-модель этой книги:
    кадр восстанавливается через .pdata/.xdata,
    а handler принадлежит функции через UNWIND_INFO и language-specific data.
```

Поэтому поиск SEH-логики в x64 начинается не с поиска `__try`, которого в бинаре уже нет как текста. Он начинается с вопроса: какая функция содержит текущий `RIP`, и есть ли у этой функции в `UNWIND_INFO` флаг handler-а?

7.6.3. От исходника к бинарю: что компилятор должен сохранить

Когда компилятор видит `__try/__except`, он должен сохранить смысл этой конструкции в машинной форме. Исходный код исчезнет. Комментарии исчезнут. Имена локальных переменных могут исчезнуть. Но runtime-смысл должен остаться, иначе Windows не сможет обработать исключение.

Возьмем исходную форму:

```
void f()
{
    __try {
        risky_code();
    }
    __except (filter()) {
        recover();
    }
}
```

Компилятор должен ответить на несколько вопросов.

Первый вопрос: где начинается и заканчивается функция `f` как диапазон машинного кода? Это попадет в `.pdata` как `RUNTIME_FUNCTION`.

Второй вопрос: как функция `f` строит свой кадр? Сколько она выделяет на стеке? Какие nonvolatile-регистры сохраняет? Это попадет в `.xdata` как `UNWIND_INFO` и `UNWIND_CODE`.

Третий вопрос: есть ли у функции обработчик исключений или unwind-handler? Это попадет во flags `UNWIND_INFO`, например через `UNW_FLAG_EHANDLER`.

Четвертый вопрос: какой language-specific handler должен быть вызван? Для MSVC C SEH это часто `__C_specific_handler`.

Пятый вопрос: какие участки кода защищены? Это уже не общий PE-формат, а language-specific handler data. В учебной модели это можно представить как scope-like table: таблица областей, где каждая запись говорит: вот protected range, вот filter, вот target handler-а.

Шестой вопрос: куда перейти, если filter согласился обработать исключение? Это связь с handler block, то есть с пользовательским кодом после `__except`.

Именно поэтому в бинаре SEH - это не “одна вещь”. Это результат компиляции исходной конструкции в несколько взаимосвязанных объектов.

7.6.4. Почему .pdata сама по себе не говорит “тут есть __try”

`.pdata` хранит `RUNTIME_FUNCTION`. В обычной x64-картине каждая запись говорит: вот диапазон функции, вот адрес unwind-информации. Но сама запись `RUNTIME_FUNCTION` не содержит слово `__try`, не содержит filter, не содержит handler block.

Типичная запись выглядит концептуально так:

```
RUNTIME_FUNCTION:
    BeginAddress       RVA начала функции
    EndAddress         RVA конца функции
    UnwindInfoAddress  RVA структуры UNWIND_INFO
```

Если ты читаешь только `.pdata`, ты можешь сказать: “вот функция и вот ее unwind info”. Но ты еще не можешь сказать: “в этой функции есть C SEH”. Для этого нужно перейти по `UnwindInfoAddress` и прочитать `.xdata`.

Здесь полезна аналогия. `.pdata` - это указатель на папку дела. Она говорит, что дело существует и где лежит подробное описание. Но сама папка еще не раскрыта. SEH-признаки обычно находятся уже внутри `UNWIND_INFO`: во flags и в хвосте структуры.

Это важно для будущего static -a. Он не должен выдавать finding уровня “функция в `.pdata`, значит SEH”. Это неверно. Большинство функций в `.pdata` имеют unwind metadata просто потому, что они non-leaf, выделяют стек или сохраняют регистры. SEH начинается там, где `UNWIND_INFO` сообщает о handler-е.

7.6.5. Первый настоящий признак: UNW_FLAG_EHANDLER

`UNW_FLAG_EHANDLER` - один из ключевых флагов `UNWIND_INFO`. `EHANDLER` можно читать как exception handler, обработчик исключения. Если этот флаг установлен, то функция имеет exception handler, который должен быть вызван во время поиска обработчика исключения.

Но опять нужно быть осторожным. Флаг не означает “вот тут лежит пользовательский `__except`”. Флаг означает: после массива `UNWIND_CODE` и выравнивания в `UNWIND_INFO` есть 4-байтный RVA language-specific handler-а. Этот handler будет вызван exception dispatcher-ом в подходящей стадии.

В памяти это концептуально выглядит так:

```
UNWIND_INFO:
    header fields
    unwind codes
    padding до 4-byte alignment
    ExceptionHandlerRVA        если UNW_FLAG_EHANDLER или UNW_FLAG_UHANDLER
    LanguageSpecificData       формат зависит от handler-а
```

Если `ExceptionHandlerRVA` указывает на `__C_specific_handler`, это сильный признак, что функция использует MSVC C-style SEH или родственный механизм, который этот handler умеет обслуживать.

Но сам `UNW_FLAG_EHANDLER` еще не говорит, какой именно protected range сработает. Он только открывает дверь к следующему слою.

```
UNW_FLAG_EHANDLER говорит:
    у функции есть language-specific handler.

Он не говорит напрямую:
    какой __try-блок;
    какой filter;
    какой handler block;
    обработает ли этот кадр конкретное исключение.
```

Чтобы ответить на эти вопросы, нужно читать handler data.

7.6.6. UNW_FLAG_UHANDLER и почему finally не то же самое, что except

Рядом с `UNW_FLAG_EHANDLER` есть `UNW_FLAG_UHANDLER`. Буква `U` обычно связывается с unwind handler. Этот флаг означает, что у функции есть обработчик, важный во время unwind phase. В человеческих терминах это ближе к cleanup или termination handler-у, например `__finally`, а не обязательно к `__except`.

Почему это важно? Потому что новичок видит слово handler и думает: “значит, функция ловит исключение”. Не всегда. Handler может не ловить исключение как “мое”. Он может быть нужен, чтобы выполнить завершающий код при выходе из protected region.

Например:

```
__try {
    acquire_resource();
    risky_code();
}
__finally {
    release_resource();
}
```

`__finally` не спрашивает: “обрабатывать ли access violation?” Его смысл другой: “когда мы покидаем этот protected block, обычным или необычным путем, надо выполнить cleanup”. Поэтому `UHANDLER` связан с unwind phase. Когда исключение уже нашло обработчик выше и стек начинает раскручиваться, кадры между местом исключения и обработчиком должны выполнить свои termination handlers.

Для реверсера это значит: `EHANDLER` и `UHANDLER` нельзя механически переводить как “есть catch”. Один флаг связан с поиском того, кто готов обработать исключение, другой - с действиями при раскрутке. Иногда они могут сочетаться, иногда нет. Смысл всегда надо читать в контексте handler-а и его данных.

7.6.7. Language-specific handler: почему Windows зовет не пользователя, а посредника

Когда `UNWIND_INFO` говорит, что есть handler, Windows не обязана сама понимать все детали языка. Она вызывает language-specific handler.

`Language-specific` - специфичный для языка. В нашей теме “язык” - это не русский/английский, а язык программирования или runtime-модель: C SEH, C++ EH, другой компиляторный механизм. Windows exception dispatcher предоставляет общий каркас: исключение, контекст, кадр, unwind metadata. Но детали того, как исходные `__try`, `__except`, `catch`, деструкторы или cleanup-блоки превращены в таблицы, знает компилятор и runtime.

Поэтому language-specific handler - это переводчик между Windows и данными компилятора.

Для MSVC C SEH важное имя - `__C_specific_handler`.

Разберем имя еще раз:

```
__C_specific_handler
    C          - относится к C/SEH-модели Microsoft
    specific   - специфичный, не общий для всех языков
    handler    - обработчик, вызываемый exception dispatcher-ом
```

Но его нельзя понимать как “код пользователя”. Он не является `recover()` из примера. Его роль ближе к диспетчеру внутри диспетчера: Windows дошла до кадра и сказала “у тебя есть C-specific обработчик, вот тебе ExceptionRecord, ContextRecord и DispatcherContext, разберись”. `__C_specific_handler` читает handler data и решает, какой filter нужно выполнить и куда перейти, если filter разрешил обработку.

7.6.8. Параметры __C_specific_handler как мост между живым исключением и таблицами

`__C_specific_handler` вызывается с несколькими параметрами. Их нужно понимать не как сухую сигнатуру, а как мосты между слоями.

`ExceptionRecord` - запись исключения. Она отвечает на вопрос: что произошло? Там есть код исключения, адрес, флаги, дополнительные параметры. Например, для access violation важны параметры доступа и проблемный адрес.

`EstablisherFrame` - опорный кадр функции, связанной с handler-ом. Термин сложный, но для реверсера он означает: речь идет не о функции как тексте, а о конкретном активном кадре, который сейчас рассматривается.

`ContextRecord` - машинный контекст. Он отвечает на вопрос: где находился поток и какие регистры были в момент исключения или в текущей стадии unwind/search.

`DispatcherContext` - досье текущего шага exception dispatcher-а. В нем есть информация о function entry, image base, control PC, handler data и других вещах, которые language-specific handler использует, чтобы принять решение.

Здесь снова видно, почему фундамент про регистры, адрес возврата и кадр был нужен. `__C_specific_handler` не работает в пустоте. Он получает живой контекст потока, конкретный кадр и указатель на данные, которые компилятор положил рядом с `UNWIND_INFO`.

```
Windows exception dispatcher:
    нашел кадр через .pdata/.xdata
    увидел handler RVA
    вызвал __C_specific_handler

__C_specific_handler:
    читает ExceptionRecord
    смотрит ControlPc / кадр
    читает HandlerData
    проверяет protected ranges
    вызывает filter при необходимости
    возвращает решение dispatcher-у
```

7.6.9. HandlerData: почему после адреса __C_specific_handler лежит еще один мир

После handler RVA в `UNWIND_INFO` идут language-specific data. В контексте `__C_specific_handler` это те данные, которые объясняют, какие protected ranges, filters и handler blocks существуют в функции.

`HandlerData` - термин удобный: данные обработчика. Но важно не думать, что это один универсальный PE-формат. PE и x64 unwind описывают общий механизм: где лежит handler RVA и что после него могут идти данные, формат которых определяется конкретным handler-ом. Если handler - `__C_specific_handler`, эти данные имеют C-specific смысл. Если handler другой, смысл может быть другим.

Для учебной модели MSVC C SEH удобно представить `HandlerData` как scope table.

`Scope` - область видимости, диапазон действия. В программировании scope часто означает область, где действует имя переменной. В SEH-контексте это ближе к области кода, где действует защищенный блок. Поэтому scope table - таблица областей: какие участки машинного кода связаны с какими filters и handler blocks.

Типичная учебная форма такой записи выглядит так:

```
ScopeTable:
    Count
    ScopeRecord[0]
    ScopeRecord[1]
    ...

ScopeRecord:
    BeginAddress
    EndAddress
    HandlerAddress
    JumpTarget
```

Эта форма полезна для понимания, но ее нельзя путать с универсальным стандартом для всех компиляторов и всех handler-ов. Это C-specific слой. Для реверсера вывод такой:

если ты видишь `__C_specific_handler`, надо искать не “один обработчик”, а таблицу областей, которую он будет интерпретировать.

7.6.10. BeginAddress и EndAddress в scope record: protected range внутри функции

Теперь разберем поля учебной scope record. `BeginAddress` и `EndAddress` задают диапазон, где эта запись действует. Важно: это обычно не абсолютные VA. Это смещения или RVA-координаты, которые нужно интерпретировать относительно image base или функции в зависимости от конкретной реализации и инструмента. В дизассемблере ты часто увидишь, что инструменты уже перевели их в удобные адреса.

Смысл простой: если `ControlPc` попадает в этот диапазон, значит исключение произошло внутри protected range этой записи.

```
if BeginAddress <= ControlPc < EndAddress:
    эта scope record подходит для текущего места исключения
```

Но здесь есть тонкость: `ControlPc` - не обязательно “адрес первой инструкции, которая упала” в наивном смысле. Во время exception dispatch и unwind есть правила около вызовов, return address и границ диапазона. Поэтому в реальном анализе нужно учитывать, как именно Windows классифицирует control PC. Но базовая мысль остается: protected range связывает участок кода с фильтром и обработчиком.

На уровне исходника:

```
__try {
    risky_code();
}
__except (filter()) {
    recover();
}
```

на уровне handler data превращается в идею:

```
код risky_code находится в диапазоне A..B
для A..B есть filter F
если F разрешит, перейти к handler target H
```

Вот почему реверсеру нужно сопоставлять scope range с реальными basic blocks в `.text`. Если диапазон покрывает только часть функции, handler относится не ко всей функции, а только к этой части.

7.6.11. HandlerAddress: filter, константа или промежуточный код

Поле `HandlerAddress` в учебной форме scope record часто вызывает путаницу, потому что имя звучит как “адрес обработчика”. Но в `__try/__except` есть и filter, и handler block. Поэтому нужно смотреть на конкретную модель.

В обычном C SEH смысл часто такой: `HandlerAddress` связан с filter expression, то есть с кодом или значением, которое решает судьбу исключения. Если filter возвращает `EXCEPTION_CONTINUE_SEARCH`, поиск идет дальше. Если возвращает `EXCEPTION_EXECUTE_HANDLER`, будет выполнен handler block. Если возвращает `EXCEPTION_CONTINUE_EXECUTION`, система попробует продолжить выполнение с места исключения.

Здесь важна терминология.

`Filter` - фильтр. Он не чинит программу сам по себе. Он принимает решение.

`Handler block` - блок обработчика. Это код после `__except`, который выполняется после положительного решения filter-а.

`JumpTarget` - цель перехода. В учебной модели это адрес или смещение, куда надо передать управление, если filter решил execute handler.

Почему это важно? Потому что в дизассемблере filter-код и handler-код могут быть разнесены. Filter может быть маленькой функцией или фрагментом, который вызывает `GetExceptionCode`, проверяет значение и возвращает 0/1/-1. Handler block может находиться рядом с обычным кодом функции, в конце функции, в cold block или в отдельном участке.

Если реверсер увидел только `JumpTarget`, но не понял filter, он не знает, при каких исключениях туда попадут. Если увидел filter, но не нашел jump target, он не знает, что будет после принятия исключения. Нужно связывать оба.

7.6.12. JumpTarget: почему обработчик может выглядеть как обычный unreachable block

`JumpTarget` - цель перехода при обработке. В исходнике это тело `__except { ... }`. В машинном коде это просто участок инструкций. Иногда декомпилятор может показать его как часть функции. Иногда он кажется “оторванным” блоком, до которого нет обычной стрелки `jmp` или `call` из основного control flow.

Это нормально. SEH handler block может быть достижим не через обычный прямой переход, а через exception machinery. То есть в нормальном выполнении функция могла никогда туда не попасть. Но если exception dispatcher, `__C_specific_handler` и filter приняли исключение, управление будет перенаправлено туда через механизм обработки исключений.

Для реверсера это важнейшая мысль:

```
Не всякий достижимый код достижим обычной стрелкой CFG.
SEH добавляет скрытые exception edges.
```

`CFG` - control-flow graph, граф потока управления. Обычный CFG строится по `jmp`, `jcc`, `call`, `ret`, switch tables. Но SEH добавляет еще один вид ребер: из protected range к handler block при исключении. Если анализатор не учитывает эти ребра, он может пометить handler block как dead code, хотя во время исключения он достижим.

Это особенно важно для обфускации. Защищенный код может специально использовать исключения как control-flow trick: обычный CFG говорит “сюда не попасть”, но реальное выполнение через exception route туда попадает.

7.6.13. Filter expression: почему это место часто содержит настоящую логику

Filter expression в `__except(...)` может быть простым:

```
__except (EXCEPTION_EXECUTE_HANDLER)
```

В этом случае filter фактически всегда говорит: “обрабатываем”. Но он может быть и сложным:

```
__except (GetExceptionCode() == EXCEPTION_ACCESS_VIOLATION
          ? EXCEPTION_EXECUTE_HANDLER
          : EXCEPTION_CONTINUE_SEARCH)
```

Здесь уже есть логика. Программа ловит не все исключения, а только access violation. Остальные исключения пропускаются выше по стеку.

Filter может использовать `GetExceptionCode`. Это специальный механизм Microsoft, который возвращает код текущего исключения. Filter также может использовать `GetExceptionInformation`, чтобы получить `EXCEPTION_POINTERS`, где есть указатели на `EXCEPTION_RECORD` и `CONTEXT`. Это значит, что filter может смотреть не только код

исключения, но и адрес, параметры, регистры.

Для reverse engineering filter часто интереснее handler block-а. Handler block может просто писать лог или возвращать ошибку. А filter может решать, является ли исключение ожидаемой частью логики, антиотладкой, проверкой guard page или настоящим crash.

Пример смысловой логики filter-а:

```
если exception code == STATUS_GUARD_PAGE_VIOLATION:
    восстановить защиту страницы
    продолжить выполнение

если exception code == EXCEPTION_BREAKPOINT:
    изменить RIP
    продолжить выполнение

если exception code == EXCEPTION_ACCESS_VIOLATION:
    обработать как ошибку

иначе:
    continue search
```

Такой filter уже не просто “аварийный обработчик”. Это часть control flow.

7.6.14. EXCEPTION_CONTINUE_SEARCH: почему отказ тоже является действием

`EXCEPTION_CONTINUE_SEARCH` обычно имеет значение 0. По смыслу это ответ filter-а: “это исключение не мое, ищите дальше”. Новичок может думать, что это ничего не значит. На самом деле это важное решение маршрута.

Если текущий кадр отказался, Windows должна подняться к caller-у. Для x64 это означает снова применить `RtlVirtualUnwind`: текущий `CONTEXT` превращается в caller context, затем по новому `RIP` ищется следующая `RUNTIME_FUNCTION`, следующая `UNWIND_INFO`, следующий возможный handler.

То есть `EXCEPTION_CONTINUE_SEARCH` напрямую связан с предыдущим разделом. Это не просто return value. Это команда exception dispatcher-у продолжать лестницу кадров.

```
filter вернул CONTINUE_SEARCH
    -> текущий кадр не принял исключение
    -> выполнить virtual unwind к caller-у
    -> проверить следующий кадр
```

Для реверсера это значит: если ты анализируешь filter, нужно понимать не только “какой handler выполнится”, но и “какие исключения этот кадр намеренно пропускает”. Иногда это помогает понять архитектуру программы. Например, внутренний слой ловит только guard page, а все остальные ошибки уходят в crash reporter.

7.6.15. EXCEPTION_EXECUTE_HANDLER: когда search phase нашла цель

`EXCEPTION_EXECUTE_HANDLER` обычно имеет значение 1. По смыслу это ответ: “да, этот кадр хочет обработать исключение”. Но даже здесь нельзя представлять грубый мгновенный прыжок.

Сначала search phase находит подходящий обработчик. Затем система должна перейти к unwind phase. Если исключение произошло глубже, а обработчик находится выше, промежуточные кадры должны быть раскручены. При этом могут выполняться termination handlers, `__finally`, cleanup-логика, деструкторы C++ в других моделях.

Только после корректной раскрутки управление попадет в handler block.

Это означает, что `EXCEPTION_EXECUTE_HANDLER` является не просто “прыгнуть в `JumpTarget`”. Это решение, которое меняет стадию обработки:

```
search phase:
    нашли кадр, который принимает исключение

unwind phase:
    раскручиваем промежуточные кадры
    выполняем cleanup/termination actions

transfer:
    управление попадает в handler block
```

Для анализа это важно, потому что между местом исключения и handler block может быть много работы. Если ты видишь handler block как конечную цель, но не учитываешь unwind phase, ты можешь пропустить `__finally` или другие cleanup effects.

7.6.16. EXCEPTION_CONTINUE_EXECUTION: самый опасный вариант для понимания

`EXCEPTION_CONTINUE_EXECUTION` обычно имеет значение -1. По смыслу это ответ: “остановите поиск и продолжите выполнение с места, где произошло исключение”. Это самый необычный и самый опасный вариант для понимания.

Продолжение возможно только если причина исключения устранена. Если была instruction breakpoint и отладчик или handler поправил контекст, можно продолжить. Если была guard page violation и handler восстановил права/флаги, можно продолжить. Если был access violation из-за `NULL`-указателя и никто не изменил память или `RIP`, то продолжение просто снова вызовет то же исключение.

Именно `EXCEPTION_CONTINUE_EXECUTION` часто используется в exception-based control flow. Handler может изменить `ContextRecord->Rip`, `ContextRecord->Rsp` или другие регистры, а затем вернуть continue execution. Тогда программа продолжит не обязательно с той же инструкции. Она продолжит с тем контекстом, который handler оставил.

Для реверсера это критично:

```
Если handler меняет CONTEXT и возвращает CONTINUE_EXECUTION,
то exception handler фактически становится скрытым управляющим переходом.
```

Обычный дизассемблерный CFG может не показать эту стрелку. Нужно смотреть на запись в `ContextRecord`, изменение `Rip`, изменение `EFlags`, изменение памяти, восстановление page protection. Это уже не просто “обработка ошибки”, а активная логика управления.

7.6.17. Как увидеть __C_specific_handler в статическом анализе

В статическом анализе есть несколько признаков, которые могут привести к `__C_specific_handler`.

Первый путь - через символы. В debug-сборке или при наличии PDB/символов имя может быть видно напрямую. В релизном бинаре пользовательские символы часто удалены, но импортированные или системные имена могут быть распознаны инструментом.

Второй путь - через адрес handler RVA в `UNWIND_INFO`. Если `UNW_FLAG_EHANDLER` установлен, после unwind codes лежит RVA handler-a. Анализатор переводит его в VA и смотрит, куда он указывает. Если адрес попадает в `ntdll`, `vcruntime` или другой runtime-модуль и совпадает с известным `__C_specific_handler`, это сильный признак.

Третий путь - через сигнатуру поведения. Даже если имя не восстановлено, handler data может иметь форму scope table, а код вокруг может соответствовать MSVC SEH. Но этот путь требует осторожности: по одним байтам нельзя слишком уверенно заявлять “это точно MSVC”, если нет достаточных доказательств.

Четвертый путь - через декомпилятор. IDA, Ghidra или другие инструменты могут распознать x64 SEH и показать `__try/__except` в псевдокоде. Но это результат анализа инструмента, а не первичный факт. Хороший реверсер должен уметь проверить, откуда декомпилятор это взял: `UNW_FLAG_EHANDLER`, handler RVA, handler data, protected ranges.

Для будущего static -а правильный отчет должен выглядеть не так:

```
SEH found.
```

А так:

```
Function app+0x12000 has UNW_FLAG_EHANDLER.
Handler RVA resolves to __C_specific_handler.
HandlerData begins at app+0x33040.
Scope table count = 2.
Scope[0] covers app+0x12110..app+0x12180.
Filter candidate = app+0x12500.
Jump target candidate = app+0x121F0.
Conclusion: MSVC-style C SEH likely present.
Confidence: high.
```

7.6.18. Почему __try не обязательно покрывает всю функцию

Еще одна частая ошибка: если функция имеет `UNW_FLAG_EHANDLER`, значит вся функция находится внутри `__try`. Это неверно. Handler принадлежит функции как кадру, но protected ranges внутри handler data могут покрывать только часть функции.

Например:

```
void f()
{
    prepare();

    __try {
        risky();
    }
    __except (EXCEPTION_EXECUTE_HANDLER) {
        recover();
    }

    finish();
}
```

Функция `f` целиком имеет один `RUNTIME_FUNCTION`. У нее может быть один `UNWIND_INFO` с `UNW_FLAG_EHANDLER`. Но protected range относится только к коду `risky()`, а не к `prepare()` и не к `finish()`.

На уровне анализа это значит:

```
Function range:
    f_begin ... f_end

Protected range:
    try_begin ... try_end
```

Protected range лежит внутри function range, но не равен ему. Поэтому при конкретном исключении `__C_specific_handler` должен проверить `ControlPc`: попадает ли текущая точка в один из protected ranges? Если нет, этот frame может иметь handler data, но конкретное исключение не относится к этому `__try`.

Это важное различие между “у функции есть SEH metadata” и “исключение произошло внутри защищенного участка”.

7.6.19. Несколько __try внутри одной функции

Одна функция может содержать несколько `__try/__except` блоков. Тогда handler data будет описывать несколько областей.

Исходно это может выглядеть так:

```
void f()
{
    __try {
        risky_a();
    }
    __except (filter_a()) {
        recover_a();
    }

    __try {
        risky_b();
    }
    __except (filter_b()) {
        recover_b();
    }
}
```

В `.pdata` все равно может быть одна `RUNTIME_FUNCTION` для функции `f`. В `.xdata` может быть один `UNWIND_INFO` с handler flag. А в handler data будет несколько scope records.

Это снова показывает разделение уровней:

```
.pdata:
    одна запись функции

.xdata:
    одно unwind-описание функции

handler data:
    несколько protected ranges внутри функции
```

Если анализатор смотрит только на `RUNTIME_FUNCTION`, он не увидит, сколько `__try` блоков внутри. Если смотрит только на `UNW_FLAG_EHANDLER`, он поймет, что есть handler-механизм, но не поймет структуру protected areas. Для этого нужна интерпретация handler data.

7.6.20. Вложенные __try и почему scope table не всегда линейна для человека

`__try` блоки могут быть вложенными. Это значит, что один protected range может лежать внутри другого. В исходнике это выглядит понятно:

```
__try {
    before();

    __try {
        risky_inner();
    }
    __except (filter_inner()) {
        recover_inner();
    }

    after();
}
__except (filter_outer()) {
    recover_outer();
}
```

Но в бинаре это уже не красивое дерево с фигурными скобками. Это набор диапазонов и правил. Некоторые диапазоны могут перекрываться, некоторые handler blocks могут сами находиться внутри внешних protected regions, и language-specific handler должен правильно выбрать, какой record применять.

Для реверсера это означает: не всякое пересечение ranges является ошибкой. В `.pdata` пересечение function ranges обычно подозрительно. Но в scope-like handler data overlap может быть частью логики вложенных protected blocks. Нужно различать уровни.

```
Пересечение RUNTIME_FUNCTION ranges:
    часто подозрительно, потому что функции как диапазоны должны быть упорядочены.

Пересечение protected ranges в handler data:
    может быть нормальным при вложенности или compiler-generated layout.
```

Это пример того, почему анализатор должен понимать семантику структуры, а не просто применять одно правило “overlap bad” ко всему.

7.6.21. Где физически искать filter-код

Filter-код может быть сгенерирован как отдельный внутренний фрагмент функции, как helper, как маленький блок рядом с handler data или как часть compiler-generated layout. Инструмент может показать его по-разному.

Для реверсера важнее не форма, а признаки. Filter должен вернуть одно из решений SEH. Поэтому в его коде часто встречаются:

```
сравнение ExceptionCode с константами;
вызов или использование GetExceptionCode;
использование GetExceptionInformation;
обращение к EXCEPTION_RECORD;
обращение к CONTEXT;
возврат 0, 1 или -1;
условный выбор между CONTINUE_SEARCH и EXECUTE_HANDLER.
```

Но в оптимизированном коде это может быть не очевидно. Если filter простейший `EXCEPTION_EXECUTE_HANDLER`, компилятор может закодировать это как константу или очень короткий фрагмент. Если filter сложный, он может выглядеть как обычная функция с условиями.

Важно помнить: filter выполняется в особом контексте. Он имеет доступ к информации о текущем исключении. Если ты видишь код, который читает exception code и возвращает маленькие константы, это может быть filter, а не обычная бизнес-функция.

7.6.22. GetExceptionCode и GetExceptionInformation: почему это не обычные API в привычном смысле

`GetExceptionCode` и `GetExceptionInformation` выглядят как функции, но в MSVC они имеют специальный статус в контексте SEH. Их можно использовать в определенных местах `__try/__except`, особенно в filter expression. `GetExceptionCode` возвращает код исключения. `GetExceptionInformation` возвращает указатель на `EXCEPTION_POINTERS`, где есть `ExceptionRecord` и `ContextRecord`.

`EXCEPTION_POINTERS` - структура с двумя указателями:

```
EXCEPTION_POINTERS:
    ExceptionRecord -> EXCEPTION_RECORD
    ContextRecord   -> CONTEXT
```

Это еще один мост к фундаменту. `ExceptionRecord` объясняет, что произошло. `ContextRecord` объясняет, в каком машинном состоянии это произошло. Поэтому filter может принимать решение не только по типу исключения, но и по адресу, регистрам, флагам, параметрам access violation.

Например, антидебаг-логика может проверять `EXCEPTION_BREAKPOINT` и менять `ContextRecord->Rip`. Guard-page логика может проверять `STATUS_GUARD_PAGE_VIOLATION`. Защитный runtime может смотреть, какая инструкция упала, и решать, является ли это ожидаемой частью VM/obfuscation.

Для реверсера наличие `GetExceptionInformation` в filter-е - сигнал: filter может не просто классифицировать ошибку, а активно смотреть и менять машинное состояние.

7.6.23. Как handler block выглядит в машинном коде

Handler block - это обычный машинный код. В нем нет мистической метки “я обработчик”. Он может вызывать функции, записывать лог, выставлять return value, освобождать ресурсы, прыгать к общему выходу функции.

Например, исходный код:

```
__except (EXCEPTION_EXECUTE_HANDLER) {
    result = -1;
}
```

может превратиться в обычный блок:

```
mov dword ptr [rsp+...], -1
jmp common_exit
```

Если смотреть только на обычный CFG, этот блок может выглядеть как отдельная ветка без входа. Но вход в него приходит не обычным `jcc`, а через SEH transfer после search/unwind phase.

Именно поэтому декомпиляторы иногда показывают странные конструкции вокруг SEH. Они пытаются восстановить высокоуровневую структуру из нескольких слоев: обычного CFG, exception metadata, handler data и runtime semantics. Если хотя бы один слой не понят, псевдокод может быть некрасивым.

Для ручного анализа полезно задавать вопрос:

```
Если это handler block, что он делает после обработки?
    возвращает значение?
    прыгает к общему epilog?
    вызывает cleanup?
    снова поднимает исключение?
    меняет состояние и продолжает?
```

Ответ помогает отличить аварийное восстановление от скрытого control flow.

7.6.24. Как отличить SEH от обычных if/else в декомпиляторе

Декомпилятор может показать `__try/__except`, но может и не показать. Иногда он выдает обычные blocks, странные gotos, unreachable code. Поэтому нужно понимать признаки на уровне metadata.

Обычный `if/else` живет в control flow инструкций. Ты видишь условный переход, две ветки, merge point.

SEH живет в двух слоях: обычный код protected range выполняется как нормальный код, а переход к handler block существует через exception metadata. В коде может не быть прямого `jcc` из risky block в handler block.

Поэтому отличительный признак SEH - не “похожесть на if”, а наличие handler metadata:

```
UNW_FLAG_EHANDLER установлен.
Handler RVA указывает на language-specific handler.
HandlerData описывает protected range.
Есть filter/handler target.
Handler target не имеет обычного входа из main CFG.
```

Если эти признаки сходятся, перед нами не обычный `if`, а exception edge. В отчете static -а это нужно показывать как отдельный вид control-flow edge:

```
Exception edge:
    from protected range app+0x12110..app+0x12180
    to handler target app+0x121F0
    condition: filter app+0x12500 returns EXECUTE_HANDLER
```

Такой формат сразу объясняет человеку, что переход не обычный, а исключительный.

7.6.25. Почему SEH-логика зависит от текущего RIP, а не просто от функции

Функция может иметь handler, но конкретное исключение должно произойти в правильном месте. Поэтому `ControlPc` принципиален.

Допустим, функция имеет три участка:

```
prepare block
try protected block
finish block
```

Если access violation произошел в `prepare block`, scope record для `try protected block` не подходит. Если произошел в `try protected block`, filter рассматривается. Если произошел в `finish block`, снова может не подходить.

Значит, вопрос “есть ли у функции SEH?” недостаточен. Нужно спрашивать:

```
RIP/ControlPc конкретного исключения попадает в protected range?
```

Это особенно важно в crash dump analysis. В дампе ты знаешь конкретный `RIP`. По нему нужно не просто найти функцию, а проверить, попадает ли он в одну из SEH-областей этой функции. Если не попадает, наличие `__C_specific_handler` у функции не означает, что этот handler должен был обработать конкретное падение.

7.6.26. Почему адрес возврата тоже может влиять на понимание SEH range

В предыдущем разделе мы говорили о return address. Теперь он снова появляется в SEH.

Когда исключение возникает внутри вызываемой функции, а `__try` находится вокруг `call`, важна граница protected range. С точки зрения исходника ты можешь написать:

```
__try {
    risky_call();
}
__except (...) {
    recover();
}
```

Если исключение произошло внутри `risky_call`, поиск обработчика сначала пройдет кадры внутри `risky_call`. Когда эти кадры не обработают исключение, unwind поднимется к caller-у, то есть к функции с `__try`. Но `ControlPc` для этого кадра будет связан с return address после `call`, а не с внутренней инструкцией `risky_call`.

Поэтому compiler layout должен учитывать, какие адреса входят в protected range. На практике вокруг `call` и границ диапазонов есть тонкости. Важно не упрощать до “range покрывает только байты инструкции call”. Protected range должен быть таким, чтобы exception dispatch понимал: исключение из callee произошло во время выполнения protected block caller-а.

Для начинающего реверсера вывод такой: если ты видишь `__try` вокруг вызова, не ищи исключение только на инструкции `call`. При подъеме к caller-кадру система работает

с return address и range-правилами. Поэтому границы protected range иногда могут выглядеть чуть неинтуитивно.

7.6.27. SEH и ordinary error handling: не путать исключения с кодами ошибок

В Windows-программах есть два больших стиля обработки проблем. Первый - обычные коды ошибок. Функция возвращает `FALSE`, `NULL`, `HRESULT`, `GetLastError`, `NTSTATUS`. Это обычный control flow: вызвали функцию, получили значение, проверили `if`.

Второй - исключения. Произошло событие, обычное выполнение прервано, запущен exception dispatcher.

В исходном коде они могут быть рядом:

```
if (!ReadFile(...)) {
    return GetLastError();
}

__try {
    risky_memory_touch();
}
__except (EXCEPTION_EXECUTE_HANDLER) {
    return ERROR_ACCESS_DENIED;
}
```

Для реверсера важно не смешивать эти стили. Обычный `if` виден по ветвлениям. SEH виден по metadata и exception edges. Handler block может в итоге вернуть обычный error code, но путь к нему был необычным.

Это особенно важно при анализе драйверов, низкоуровневого кода, runtime-библиотек и защит. Там SEH может использоваться как способ безопасно пробовать доступ к памяти, а обычные error codes - как способ сообщить результат наверх.

7.6.28. SEH и C++ exceptions: почему __C_specific_handler не равен catch(...)

Нужно еще раз отделить C SEH от C++ exceptions. В C++ есть `try`, `throw`, `catch`, типы исключений, destructors, stack unwinding с вызовом деструкторов. В MSVC C++ exceptions тоже опираются на Windows x64 unwind infrastructure, но language-specific handler и данные могут быть другими.

`__try/__except` - Microsoft SEH extension. Это не стандартный C++ `try/catch`. Он работает с structured exceptions Windows: access violation, breakpoint, RaiseException и подобными событиями.

C++ `catch(...)` и SEH `__except(...)` похожи внешне только тем, что оба “что-то ловят”. Но внутренние данные, handler-ы, типовая информация и семантика отличаются.

Для реверсера ошибка выглядит так:

```
Увидел __C_specific_handler -> решил, что это C++ catch.
```

Это неверно. `__C_specific_handler` связан именно с C-style SEH extensions. Для C++ exceptions есть свои runtime-механизмы и данные. Они могут использовать общую x64 unwind инфраструктуру, но language-specific слой другой.

Правильная мысль:

```
.pdata/.xdata - общая инфраструктура x64 unwind.
SEH, C++ EH и cleanup-логика используют ее по-своему.
Language-specific handler определяет, какой именно слой перед нами.
```

7.6.29. Как анализировать SEH-логику вручную: общий маршрут

Теперь соберем ручной маршрут. Он должен быть медленным, потому что это будущий алгоритм static -а.

```
Шаг 1.
    Найти функцию по RIP или по интересующему адресу кода.

Шаг 2.
    Через ImageBase получить RVA.

Шаг 3.
    Найти RUNTIME_FUNCTION в .pdata.

Шаг 4.
    Перейти по UnwindInfoAddress к UNWIND_INFO.

Шаг 5.
    Проверить Flags.

Шаг 6.
    Если UNW_FLAG_EHANDLER или UNW_FLAG_UHANDLER установлен:
        найти handler RVA после unwind codes и padding.

Шаг 7.
    Перевести handler RVA в VA и определить handler.

Шаг 8.
    Если handler похож на __C_specific_handler:
        интерпретировать последующие bytes как C-specific handler data.

Шаг 9.
    Прочитать scope-like records.

Шаг 10.
    Сопоставить protected ranges с реальным кодом функции.

Шаг 11.
    Найти filter candidates.

Шаг 12.
    Найти handler jump targets.

Шаг 13.
    Построить exception edges:
        protected range -> filter -> handler block.

Шаг 14.
    Проверить, какие exception codes filter принимает.

Шаг 15.
    Оценить, это обычная аварийная обработка или control-flow механизм.
```

Этот маршрут выглядит длинным, но в нем нет лишних шагов. Каждый шаг связывает один слой с другим. Если пропустить адресную дисциплину, можно прочитать не те bytes. Если пропустить flags, можно принять обычную unwind info за SEH. Если пропустить filter, можно не понять условие. Если пропустить handler block, можно не понять эффект.

7.6.30. Что должен показывать будущий static для SEH

Если инструмент просто пишет “SEH detected”, он почти бесполезен. Человек должен видеть доказательства.

Хороший отчет должен строиться как цепочка:

```
Function:
    module: app.exe
```

```
        range: RVA 0x12000..0x12380

    .pdata:
        RUNTIME_FUNCTION found at .pdata+0x1A0
        BeginAddress = 0x12000
        EndAddress = 0x12380
        UnwindInfoAddress = 0x34020

    .xdata:
        UNWIND_INFO flags = UNW_FLAG_EHANDLER
        CountOfCodes = 4
        Handler RVA = 0x56010
        Handler VA resolves to __C_specific_handler

    HandlerData:
        probable C scope table
        Count = 1
        Scope[0]: 0x12120..0x12190
        Filter candidate = 0x122F0
        Handler target = 0x12220

    Interpretation:
        exceptions in app+0x12120..app+0x12190 are filtered by app+0x122F0;
        if filter returns EXECUTE_HANDLER, control can transfer to app+0x12220.

    Confidence:
        high, because handler symbol and table shape match MSVC C SEH.
```

Такой отчет учит пользователя. Он показывает не только итог, но и путь рассуждения. Это важно, потому что reverse engineering почти всегда работает с неполными доказательствами. Хороший инструмент не скрывает неопределенность, а показывает, почему он сделал вывод.

7.6.31. Как понять, что SEH используется как control-flow trick

Обычный аварийный SEH часто выглядит скучно. Protected range вызывает risky operation. Filter принимает access violation. Handler пишет ошибку, освобождает ресурс или возвращает код отказа.

Control-flow trick выглядит иначе. Программа намеренно вызывает исключение, а handler продолжает выполнение в измененном месте. Признаки могут быть такими:

```
инструкция int 3 без обычной причины;
UD2 или illegal instruction;
явная запись по плохому адресу;
guard page setup перед выполнением кода;
RaiseException с application-defined code;
filter меняет ContextRecord->Rip;
handler возвращает CONTINUE_EXECUTION;
обычный CFG имеет unreachable blocks, которые достижимы через exception route;
много маленьких protected ranges;
нестандартная handler data;
несоответствие между декомпилятором и runtime-поведением.
```

Важно не объявлять любое исключение обфускацией. Access violation может быть багом. Guard page может быть нормальной техникой memory management. Breakpoint может быть отладочной вставкой. `RaiseException` может быть нормальным внутренним сигналом. Но если исключения систематически используются для переходов, изменения `RIP`, переключения состояний, антидебага или обхода обычного CFG, это уже finding.

Формулировка finding должна быть аккуратной:

```
Suspicious exception-based control flow:
    protected range intentionally triggers EXCEPTION_BREAKPOINT;
    filter modifies ContextRecord->Rip;
    returns EXCEPTION_CONTINUE_EXECUTION;
```

```
    target block has no ordinary CFG predecessor.
```

Это лучше, чем просто “malware-like SEH”.

7.6.32. Почему наличие __C_specific_handler не означает вредоносность

`__C_specific_handler` - нормальный механизм MSVC. Он может появляться в обычных программах, библиотеках, runtime-коде, драйверах, обработчиках ошибок. Его наличие само по себе не является подозрением.

Это надо прямо зафиксировать, потому что будущий static не должен пугать пользователя ложными тревогами. Сам факт `UNW_FLAG_EHANDLER` или `__C_specific_handler` говорит только: функция имеет frame-based exception handling metadata.

Подозрительными могут быть не сами SEH-структуры, а их использование:

```
handler data указывает за пределы image;
protected ranges странно перекрываются без понятной причины;
filter targets указывают в writable/executable memory;
handler меняет RIP на область без нормальной разметки;
код намеренно вызывает исключения для обычных переходов;
metadata противоречит коду;
function table создана динамически для скрытого кода;
handler data выглядит crafted/manual, а не compiler-generated.
```

То есть SEH - не зло. Это нормальная часть Windows. Но потому что это мощный механизм управления исключительными переходами, его можно использовать и для защиты, и для обфускации.

7.6.33. Связь SEH-логики с предыдущими разделами

Теперь полезно остановиться и увидеть, как вся книга снова сходится в одну точку.

Из главы 1 мы взяли идею: исключение - это событие выполнения конкретного потока. Оно имеет `ExceptionRecord` и `ContextRecord`.

Из главы про стек мы взяли идею: функция существует как код, но активный вызов функции существует как кадр стека. `caller`, `callee`, `return address`, `RSP`, nonvolatile-регистры - это не детали, а основа перехода между кадрами.

Из PE-главы мы взяли идею: x64 Windows хранит карту функций в PE-образе. `Exception Directory` ведет к `.pdata`, `.pdata` ведет к `RUNTIME_FUNCTION`, `RUNTIME_FUNCTION` ведет к `.xdata`.

Из разделов про `.xdata` мы взяли идею: `UNWIND_INFO` описывает, как разобрать кадр и есть ли handler.

Из раздела про `RtlVirtualUnwind` мы взяли идею: Windows может получить caller context без обычного `ret`.

Теперь SEH-логика становится понятной:

```
SEH не висит отдельно.
SEH использует тот же путь:
    исключение -> контекст -> функция -> unwind info -> handler flag -> language-specific handler -> fil
```

Если читатель удерживает эту взаимосвязь, `__C_specific_handler` перестает быть загадочным именем. Это просто один участник маршрута, который получает данные текущего кадра и решает, относится ли exception к protected ranges этой функции.

7.6.34. Мини-сцена: один __try глазами реверсера

Представим функцию:

```
int f()
{
    int result = 0;

    __try {
        risky();
        result = 1;
    }
    __except (GetExceptionCode() == EXCEPTION_ACCESS_VIOLATION
              ? EXCEPTION_EXECUTE_HANDLER
              : EXCEPTION_CONTINUE_SEARCH) {
        result = -1;
    }

    return result;
}
```

Как это будет выглядеть в голове реверсера?

Сначала функция `f` имеет обычный машинный код. Она строит кадр, вызывает `risky`, записывает `result`, возвращается.

Затем в `.pdata` есть `RUNTIME_FUNCTION`, который покрывает диапазон функции `f`.

`UnwindInfoAddress` ведет к `UNWIND_INFO` в `.xdata`.

`UNWIND_INFO` описывает пролог функции и имеет `UNW_FLAG_EHANDLER`.

После unwind codes лежит handler RVA, который ведет к `__C_specific_handler`.

После handler RVA лежит handler data. Там есть scope record, который покрывает участок кода, соответствующий `__try` body.

Scope record указывает на filter-код. Filter смотрит `GetExceptionCode()` и сравнивает с `EXCEPTION_ACCESS_VIOLATION`.

Если filter возвращает `EXCEPTION_EXECUTE_HANDLER`, управление после unwind phase попадет в handler block, где `result = -1`.

Если filter возвращает `EXCEPTION_CONTINUE_SEARCH`, Windows через `RtlVirtualUnwind` поднимется к caller-у и будет искать другой handler.

Вот полная цепочка:

```
исключение внутри risky
 -> кадры risky не обработали
 -> unwind к caller f
 -> ControlPc в f попал в protected range
 -> __C_specific_handler вызвал filter
 -> filter принял только access violation
 -> handler block записал result = -1
 -> функция вернулась обычным способом из общего выхода
```

Такое объяснение длиннее, чем “там try/except”, но оно правильное для reverse engineering. Оно показывает все точки, где можно ошибиться или где защитный код может вмешаться.

7.6.35. Типичные ошибки при поиске SEH в бинаре

Первая ошибка - искать строку `__try` или ожидать, что в бинаре будет явная конструкция как в исходнике. В бинаре есть metadata, ranges и code blocks, а не текст исходника.

Вторая ошибка - считать `__C_specific_handler` пользовательским обработчиком. Это language-specific handler, который читает данные компилятора.

Третья ошибка - думать, что `UNW_FLAG_EHANDLER` означает “исключение будет обработано”. Флаг означает только наличие handler-механизма. Конкретное исключение

еще должно попасть в protected range, а filter должен принять его.

Четвертая ошибка - путать filter и handler block. Filter решает, handler block выполняется после положительного решения.

Пятая ошибка - игнорировать `ControlPc`. Нельзя судить по функции целиком, нужно смотреть, где именно произошло исключение.

Шестая ошибка - считать `HandlerData` универсальным форматом PE. Это language-specific data. Ее смысл зависит от handler-а.

Седьмая ошибка - не учитывать unwind phase. Если handler найден выше по стеку, промежуточные кадры могут выполнять cleanup.

Восьмая ошибка - принимать декомпиляторный псевдокод за первичный источник. Декомпилятор полезен, но нужно уметь проверить его вывод через `.pdata/.xdata`.

Девятая ошибка - считать любой SEH подозрительным. SEH нормален. Подозрительным может быть конкретный pattern использования.

7.6.36. Итог раздела

SEH-логика в Windows x64 бинаре находится не в одном месте. Она собирается из нескольких слоев. `.pdata` дает функцию. `.xdata` дает `UNWIND_INFO`. Flags в `UNWIND_INFO` говорят, есть ли handler-механизм. Handler RVA указывает на language-specific handler, часто `__C_specific_handler` для MSVC C SEH. Handler data описывает protected ranges, filters и handler targets. Filter принимает решение: continue search, execute handler или continue execution. Handler block выполняется только если filter разрешил обработку. Если текущий кадр не принимает исключение, Windows поднимается к caller-у через `RtlVirtualUnwind`.

Главное для реверсера - не искать “один обработчик”, а строить карту взаимосвязей:

```
какой адрес анализируем;
какая функция его содержит;
какая RUNTIME_FUNCTION это подтверждает;
какой UNWIND_INFO описывает функцию;
какие флаги handler-а установлены;
какой language-specific handler указан;
какие handler data лежат после него;
какие protected ranges описаны;
какие filters принимают решения;
куда ведут handler targets;
является ли это обычной обработкой ошибки или exception-based control flow.
```

Теперь у нас есть ручная модель поиска SEH. Следующий естественный шаг - отделить обычный unwind и обычную обработку исключений от случаев, когда исключения специально используются как control-flow механизм. То есть дальше нужно разобрать, как отличать нормальный crash/error-handling route от намеренной exception-based логики, антидебага и обфускации.

Техническая база раздела 7.6

Раздел опирается на официальную документацию Microsoft по x64 exception handling, где описаны `RUNTIME_FUNCTION`, `UNWIND_INFO`, `UNW_FLAG_EHANDLER`, `UNW_FLAG_UHANDLER`, language-specific handler data и общий unwind procedure; на документацию по `__C_specific_handler`, где указано, что language-specific handler вызывается при поиске обработчика или unwind и получает `ExceptionRecord`, `EstablisherFrame`, `ContextRecord`, `DispatcherContext`; на документацию по `__try/__except`, `GetExceptionCode`, `GetExceptionInformation` и exception-handler syntax, где описаны filter expression, exception handler block и возвращаемые решения `EXCEPTION_CONTINUE_SEARCH`, `EXCEPTION_EXECUTE_HANDLER`, `EXCEPTION_CONTINUE_EXECUTION`. Текст пересобирает эти сведения в учебную модель для reverse engineering: от `.pdata/.xdata` к protected ranges, filters, handler targets и findings будущего static -а.

7.7. Как отличать обычный unwind от exception-based control flow: где статическая карта заканчивается и начинается динамическая валидация

Мы уже прошли довольно длинный путь. Сначала мы учились видеть функцию не как имя из исходного кода, а как диапазон машинных адресов. Потом мы связали этот диапазон с `RUNTIME_FUNCTION` в `.pdata`, а `RUNTIME_FUNCTION` связали с `UNWIND_INFO` в `.xdata`. После этого мы разобрали, почему `UNWIND_CODE` описывает не красивые строки дизассемблера, а эффекты пролога: куда сдвинулся `RSP`, какие nonvolatile-регистры были сохранены, где лежат данные, необходимые для восстановления caller context. Затем мы подошли к SEH и увидели, что у функции может быть не только обычный путь исполнения через `call` и `ret`, но и скрытые для новичка exception edges - ребра управления, которые появляются только при исключении.

Теперь нужно сделать следующий фундаментальный шаг. Реверсеру недостаточно найти слово “handler” или увидеть флаг `UNW_FLAG_EHANDLER`. Нужно понять, является ли найденная exception-механика обычной частью корректной работы Windows x64, или программа специально использует исключения как маршрут управления. Именно здесь рождается различие между обычным unwind и exception-based control flow.

Термин `exception-based control flow` можно перевести как “поток управления, построенный на исключениях”. Control flow - это поток управления, то есть ответ на вопрос: какая инструкция будет выполнена следующей, потом следующей, потом следующей. В обычном коде поток управления задают `call`, `ret`, `jmp`, условные переходы, циклы, switch-таблицы, indirect calls и indirect jumps. Но Windows позволяет передавать управление и через механизм исключений: программа может намеренно создать исключение, попасть в обработчик, изменить `RIP` в `ContextRecord`, вернуть `EXCEPTION_CONTINUE_EXECUTION` и продолжить работу уже в другой точке. Для процессора это выглядит как аварийная ситуация или trap. Для программы это может быть заранее задуманный маршрут.

Такой прием особенно важен для нашего проекта. static строит статическую карту: что написано в PE, где `.pdata`, куда указывают `RUNTIME_FUNCTION`, какие `UNWIND_INFO` существуют, какие handler-ы видны по metadata и какие признаки выглядят необычно. dynamic , наоборот, занимается динамической валидацией. Он проверяет живое исполнение: куда реально ушла функция, какое исключение реально произошло, какой handler реально был достигнут, продолжился ли поток после handler-а, изменился ли `RIP`, была ли регистрация VEH или UEF, был ли вызов dynamic function table API.

Поэтому этот раздел - мост между двумя мирами: статической картой static -а и живым маршрутом dynamic -а.

> В этой книге дальше слово static означает статический аналитический слой: чтение PE, `.pdata`, `.xdata`, `UNWIND_INFO`, handler metadata и формирование findings. Слово dynamic означает динамическую валидацию: запуск под контролем, наблюдение исключений, handler hit-ов, переходов, `RIP` trace и фактического маршрута обработки. Это не “защитный продукт” в абстрактном смысле, а проверочный runtime-слой, который показывает, совпадает ли статическая карта с реальным поведением.

7.7.1. Что значит “обычный unwind”

Начнем с нормального случая. Обычный unwind - это не обязательно crash. И не обязательно обработка ошибки. В самом широком смысле unwind - это восстановление состояния вызывающего кода. Когда функция была вызвана, инструкция `call` положила на стек адрес возврата. После этого callee, то есть вызванная функция, могла построить собственный кадр: уменьшить `RSP`, сохранить `RBX`, `RBP`, `RDI`, `RSI`, `R12-R15`, выделить место под локальные переменные, возможно назначить frame pointer. Когда функция завершается нормально, она сама разбирает свой кадр и делает `ret`. Но когда управление надо покинуть не обычным `ret`, а из-за исключения, runtime должен понять, как разобрать кадр без помощи самой функции.

Поэтому обычный unwind - это использование `.pdata` и `.xdata` по их прямому назначению. Эти данные нужны Windows, debugger-у и runtime-библиотекам, чтобы восстановить caller context: где был `RSP` у вызывающей функции, какой `RIP` надо поставить как адрес продолжения, какие nonvolatile-регистры нужно вернуть. Если программа упала из-за access violation, debugger строит stack trace через тот же слой. Если исключение поймано frame-based SEH, dispatcher использует metadata, чтобы пройти по кадрам. Если C++ exception раскручивает стек, runtime тоже опирается на unwind-информацию. Во всех этих случаях unwind не является хитрым control flow сам по себе. Это инфраструктура корректного восстановления стека.

Новичок часто ошибается именно здесь. Он видит `.pdata`, `.xdata`, `UNW_FLAG_EHANDLER`, `__C_specific_handler` и думает: “раз есть исключения, значит программа что-то скрывает”. Это неверно. Для Windows x64 наличие unwind metadata - нормальный контракт ABI. ABI - application binary interface, двоичный интерфейс приложения: набор правил, по которым скомпилированный код, операционная система, debugger, loader и runtime понимают друг друга. В этом контракте `.pdata` и `.xdata` - не украшение и не след компилятора “для удобства”. Это часть договора: если функция non-leaf, если она меняет стек или сохраняет nonvolatile-регистры, система должна иметь способ восстановить предыдущее состояние.

Поэтому обычный unwind имеет несколько признаков. Он связан с прологом функции. Его `UNWIND_CODE` соответствует реальным инструкциям. Его handler, если он есть, находится в ожидаемом месте и связан с нормальным language-specific механизмом, например с `__C_specific_handler`. Исключение, если оно возникает, не обязательно меняет смысл программы: оно может завершить процесс, привести к диагностике, попасть в crash reporter или выйти в обычный handler ошибки. Важнее всего: обычный unwind не пытается заменить привычные `call`, `jmp`, `ret` как основной способ маршрутизации логики.

7.7.2. Что значит “управление через исключение”

Exception-based control flow начинается там, где исключение становится не редкой аварией и не инфраструктурным событием, а обычным способом передать управление. В таком коде исключение похоже не на “что-то пошло не так”, а на скрытый `jmp`. Программа специально создает ситуацию, которую Windows оформляет как exception, после чего handler принимает решение, куда продолжить исполнение. Иногда handler просто возвращает `EXCEPTION_CONTINUE_EXECUTION`. Иногда он вручную правит `ContextRecord->Rip`. Иногда он меняет `RSP`, флаги процессора, регистры аргументов или внутреннее состояние декодера. Снаружи это выглядит странно: обычный граф управления в дизассемблере показывает один маршрут, а живой runtime уходит по другому.

Здесь важно аккуратно понимать слово “скрытый”. Оно не означает автоматически “вредоносный”. Разработчик может использовать exceptions для sandboxing, диагностики, JIT-runtime, lazy commitment памяти, guard-page механики, совместимости, обработки аппаратных traps или редких системных условий. Но для reverse engineering это все равно сложнее обычного `if` или `switch`, потому что часть маршрута лежит не в обычном control-flow graph, а в exception dispatcher-е Windows.

Control-flow graph, или CFG, - это граф потока управления. Узлы такого графа обычно соответствуют basic block-ам, то есть прямолинейным участкам машинного кода без внутренних переходов. Ребра графа показывают, куда может пойти управление после блока. В простом коде ребра видны по `jcc`, `jmp`, `call`, `ret`. Exception edge - это особое ребро: оно не написано как обычная инструкция перехода. Оно появляется потому, что внутри protected range может произойти exception, dispatcher найдет handler, и управление попадет в filter или handler block. Для новичка такой handler block может выглядеть как “мертвый код”, потому что на него не ведет обычный `jmp`. На самом деле на него ведет exception-механизм.

Когда exception-механика используется как основной маршрут, обычные инструменты статического анализа могут ошибиться. Декомпилятор может показать странную функцию с unreachable block-ами. Дизассемблер может не

связать адрес после `int 3` с handler-ом. Graph view может показать неполный путь. Именно поэтому static и dynamic должны работать вместе. static может сказать: “здесь есть handler metadata, здесь есть deliberate exception site, здесь handler может изменить context”. dynamic может проверить: “при запуске действительно возникло исключение, действительно был handler hit, после него RIP ушел не туда, куда ожидался обычный fall-through”.

```
Обычный видимый control flow:

    block_A
        cmp eax, 1
        je  block_B
        jmp block_C

Exception-based route:

    block_A
        int 3              ; видимого jmp на block_H нет

    ...

    block_H:               ; handler reached by exception dispatcher
        изменить ContextRecord->Rip
        return EXCEPTION_CONTINUE_EXECUTION
```

7.7.3. Почему одно наличие handler-а еще ничего не доказывает

В предыдущем разделе мы нашли SEH-логику через `UNW_FLAG_EHANDLER`, `__C_specific_handler`, protected range, filter и handler block. Но теперь нужно сделать осторожную аналитическую паузу. Handler - это не приговор. Handler означает только то, что у функции есть дополнительная exception-семантика. Чтобы сказать больше, нужно понять, зачем он существует.

Один handler может быть обычной обработкой ошибки. Например, функция обращается к памяти, которая может быть недоступна, и при `EXCEPTION_ACCESS_VIOLATION` аккуратно возвращает код ошибки. Другой handler может быть cleanup-механизмом, который освобождает ресурс при исключении. Третий handler может быть частью C++ exception handling, где таблицы и runtime обслуживают destructors и catch-блоки. Четвертый handler может быть crash reporter-ом: записать minidump, лог, состояние потока и завершить процесс. Все эти случаи могут быть нормальными.

Но handler может быть и частью управления. Например, программа вызывает `RaiseException` с собственным кодом, filter проверяет этот код и возвращает `EXCEPTION_EXECUTE_HANDLER`. Или VEH handler ловит breakpoint exception, сдвигает `RIP` за зашифрованный байт, декодирует следующий блок и продолжает выполнение. Или guard page специально ставится на страницу, чтобы первый доступ вызвал `STATUS_GUARD_PAGE_VIOLATION`, а handler использовал это как сигнал переключения состояния. Во всех этих сценариях

exception становится не побочным эффектом, а частью алгоритма.

Значит, аналитический вопрос должен звучать не “есть ли handler?”, а “какова роль handler-а в маршруте управления?”. Для ответа нужны признаки. Статические признаки ищет static . Динамические признаки проверяет dynamic . И только соединение этих слоев дает нормальный confidence, то есть уверенность вывода.

7.7.4. Нормальный cleanup, crash path и diagnostic path

Чтобы не видеть подозрительность везде, полезно выделить нормальные категории. Первая категория - cleanup path, путь очистки. Cleanup - это освобождение ресурсов: закрыть handle, вернуть lock, освободить память, вызвать destructor, восстановить состояние. В C++ и SEH такие пути могут появляться в metadata и в коде, но они не обязательно меняют смысл управления. Они обеспечивают корректное покидание функции при исключении.

Вторая категория - crash path, путь аварийного завершения. Здесь программа не пытается восстановиться. Она может собрать диагностику, записать dump, вывести сообщение, вызвать `TerminateProcess` или позволить Windows завершить процесс. Наличие `SetUnhandledExceptionFilter` или crash-reporting кода часто относится сюда. Такой код может быть сложным и большим, но его цель обычно не скрыть обычный control flow, а сохранить evidence о падении.

Третья категория - diagnostic path, диагностический путь. Программа может регистрировать VEH, чтобы логировать исключения, собирать telemetry, делать отладочную печать, считать статистику. Такой handler может видеть first-chance exceptions, но почти всегда возвращает `EXCEPTION_CONTINUE_SEARCH`, то есть говорит системе: “я посмотрел, но не обрабатываю, ищите дальше”. Это важный признак. Handler, который только наблюдает и не берет управление на себя, не является control-flow dispatcher-ом.

Разумеется, эти категории могут смешиваться. Crash reporter может ставить VEH. Диагностический handler может иногда обработать один собственный код. Cleanup может быть рядом с filter-ом. Поэтому мы не строим вывод по одному слову API. Мы строим цепочку: где exception создается, кто его видит, кто принимает решение, возвращается ли управление, меняется ли context, куда идет `RIP` после handler-а.

7.7.5. Признаки exception-based control flow в статике

Теперь посмотрим глазами static -а. static не запускает программу. Он не знает, какой путь реально будет выбран при конкретном вводе. Но он может найти места, где exception-механизм потенциально используется как управление.

Первый признак - deliberate exception site, намеренное место создания исключения. Это может быть явный вызов `RaiseException`. Это может быть `int 3`, то есть breakpoint exception. Это может быть `ud2`, invalid instruction. Это может быть деление на ноль, намеренный access violation, чтение из `NULL`, запись в guard page. Не каждое такое место подозрительно: `int 3` бывает в debug build, `ud2` может использоваться для unreachable path, guard page может быть нормальной техникой управления памятью. Но если рядом есть handler, который продолжает выполнение, эта связка становится интересной.

Второй признак - handler, который смотрит на `ExceptionCode` и принимает разные решения. Сам по себе `GetExceptionCode` или обращение к `ExceptionRecord->ExceptionCode` нормальны. Но если код сравнивается с `EXCEPTION_BREAKPOINT`, `EXCEPTION_SINGLE_STEP`, `STATUS_GUARD_PAGE_VIOLATION`, пользовательским exception code из `RaiseException`, а дальше handler возвращает `EXCEPTION_CONTINUE_EXECUTION`, это уже похоже на управляемую маршрутизацию.

Третий признак - изменение `ContextRecord`. `ContextRecord` - это снимок состояния потока: `RIP`, `RSP`, регистры общего назначения, flags и другие поля. Если handler просто пишет лог, context ему может быть нужен для диагностики. Но если handler записывает новое значение в `Rip`, `Rsp`, `EFlags`, `Rax`, `Rcx` или другие регистры, он фактически редактирует будущее исполнение. Это одно из самых сильных статических указаний на exception-based control flow.

Четвертый признак - отсутствие нормального видимого перехода к коду, который реально выглядит живым. Handler block может содержать осмысленную логику, но обычные ребра CFG к нему не ведут. Иногда такой блок расположен рядом с функцией, иногда далеко, иногда декомпилятор показывает его как unreachable. Если metadata говорит, что это handler, а код внутри handler-а не похож на простую очистку или crash-reporting, static должен поднять finding с аккуратной формулировкой.

Пятый признак - repeated handler reuse. Если множество функций используют один и тот же handler или один и тот же `UNWIND_INFO`, это может быть нормальной оптимизацией или artifact компилятора, но может быть и признаком централизованного exception dispatcher-а. Здесь нельзя делать резкий вывод. Нужно смотреть, какие функции покрыты, где расположен handler, что он делает, есть ли deliberate exception sites, подтверждает ли dynamic runtime hits.

Шестой признак - связка с dynamic function table API: `RtlAddFunctionTable`, `RtlInstallFunctionTableCallback`, `RtlDeleteFunctionTable`. Для JIT или runtime-generated code это нормальный механизм: если код создается динамически, Windows все равно нужна function table, чтобы делать unwind. Но для packer-а, manual mapping или obfuscation это тоже удобный способ создать runtime-карту функций, которой не было в статическом PE. static

может увидеть импорты или вызовы, но dynamic должен проверить, происходят ли они реально и какие диапазоны регистрируются.

```
Статическая цепочка подозрения может выглядеть так:

    1. В коде найден deliberate exception site: int 3 / RaiseException / guard page.
    2. В PE metadata найден handler или VEH/UEF registration path.
    3. Handler читает ExceptionRecord / ContextRecord.
    4. Handler изменяет Rip или возвращает EXCEPTION_CONTINUE_EXECUTION.
    5. Видимый CFG не объясняет переход к следующему смысловому блоку.

Вывод: это не доказательство вредоносности, но сильный признак exception-based control flow.
```

7.7.6. Почему `EXCEPTION_CONTINUE_EXECUTION` особенно важен

В SEH и VEH есть несколько разных решений handler-а. `EXCEPTION_CONTINUE_SEARCH` означает: “я не обработал, ищите дальше”. `EXCEPTION_EXECUTE_HANDLER` в frame-based SEH означает: “переходим в handler block”. `EXCEPTION_CONTINUE_EXECUTION` означает: “считайте исключение обработанным и продолжайте выполнение”. Именно третий вариант особенно важен для exception-based control flow.

Чтобы понять почему, нужно вернуться к базовой модели. Когда произошло исключение, Windows сформировала `ExceptionRecord` и `ContextRecord`. `ExceptionRecord` описывает само событие: код исключения, адрес, параметры. `ContextRecord` описывает состояние потока в момент события: какой был `RIP`, какой был `RSP`, какие значения были в регистрах. Если handler возвращает continue execution и не меняет context, система попробует продолжить с тем же состоянием. Иногда это бессмысленно: если инструкция снова обращается к плохому адресу, произойдет то же исключение. Поэтому реальные восстановительные handler-ы часто чинят причину: меняют защиту памяти, исправляют указатель, пропускают инструкцию, эмулируют результат или двигают `RIP`.

С точки зрения reverse engineering, изменение `RIP` - это почти прямой невидимый переход. Обычный `jmp target` виден в инструкции. А запись `ContextRecord->Rip = target` видна только если реверсер понимает структуру context-а и смысл возвращаемого значения handler-а. Поэтому эта связка должна быть в книге особенно хорошо запомнена: handler, который правит `RIP` и возвращает continue execution, не просто “обработал ошибку”. Он мог переписать следующий шаг исполнения.

Но и здесь нужна осторожность. Иногда сдвиг `RIP` легитимен: например, handler эмулирует запрещенную инструкцию, пропускает breakpoint, обслуживает guard page, реализует runtime probe. Вопрос не в том, “можно ли так делать”, а в том, соответствует ли это ожидаемому назначению программы и подтверждается ли весь маршрут evidence-цепочкой.

7.7.7. First chance и second chance как динамическая граница

Теперь переходим к dynamic -слою. Когда target запускается под debugger-ом, Windows при исключении сначала уведомляет debugger. Это first-chance notification, первая попытка дать debugger-у возможность обработать событие. Если debugger говорит, что исключение не обработано, Windows продолжает обычный поиск обработчиков: VEH, frame-based SEH, top-level path - в зависимости от конкретного маршрута. Если никто не обработал исключение, debugger может получить second-chance notification. После second chance, если событие не обработано, процесс обычно завершается.

Для dynamic -а это принципиально. First chance exception не означает crash. Это просто момент, когда исключение родилось. Многие нормальные программы могут генерировать first-chance exceptions и продолжать работу. Поэтому динамический отчет должен различать: exception observed, handler hit, continued, second chance, termination. Иначе пользователь увидит “access violation first chance” и решит, что программа упала, хотя handler мог восстановить поток.

Здесь появляется хороший мост между static и dynamic . static может сказать: “в этой функции есть protected range и handler”. dynamic может запустить программу, увидеть first-chance exception по адресу внутри этого range, потом увидеть handler hit, потом записать RIP после продолжения. Если RIP вернулся в ожидаемое место, это один вывод. Если RIP ушел в неожиданный блок, это другой вывод. Если exception дошло до second chance, это третий вывод.

Именно поэтому dynamic validation не должна быть простой строкой “было исключение”. Она должна отвечать на маршрутный вопрос: где исключение возникло, кто его увидел, кто его обработал, как изменился context, куда ушло управление после обработки.

7.7.8. Что значит “куда уходит функция” в динамической валидации

Фраза “куда уходит функция” в контексте dynamic -а не должна пониматься слишком бытово. Функция в машинном коде не “уходит” как объект. Уходит поток исполнения. Был текущий RIP, он указывал на инструкцию внутри функции. Затем произошел переход: обычный `call`, `jmp`, `ret`, exception dispatch, handler transfer, continuation после handler-а или termination path. dynamic наблюдает именно это: как меняется RIP и какие события объясняют изменение.

Если функция завершилась обычным `ret`, поток должен перейти на return address, который лежал на стеке. Если функция вызвала другую функцию, `RIP` уйдет в callee. Если произошел indirect jump, `RIP` уйдет по адресу из регистра или памяти. Если произошло исключение, `RIP` может сначала попасть в системный dispatcher, затем в VEH или frame handler, затем в handler block или обратно в измененный context. Для человека все это выглядит как “функция ушла куда-то”, но для инструмента нужно точное описание события.

Поэтому dynamic должен показывать не только конечный адрес, но и причинную цепочку. Например: “исключение `EXCEPTION_BREAKPOINT` возникло по адресу `module+0x1234`; был вызван VEH handler `module+0x5000`; handler вернул `EXCEPTION_CONTINUE_EXECUTION`; после события `RIP` стал `module+0x2000`”. Это уже не просто trace. Это объяснение маршрута.

Такой подход особенно полезен рядом с static . static заранее знает, что `module+0x5000` выглядит как handler, потому что он найден в metadata или через registration API. dynamic подтверждает, что этот адрес не просто лежит в файле, а реально получил управление. Если после handler hit `RIP` меняется на адрес, который статический CFG не показывал как достижимый, это сильное evidence для exception-based control flow.

7.7.9. Пример нормального unwind: access violation как ошибка, а не управление

Представим функцию, которая читает пользовательский указатель. Внутри есть protected range. Если указатель плохой, происходит `EXCEPTION_ACCESS_VIOLATION`. Filter проверяет код исключения, возвращает `EXCEPTION_EXECUTE_HANDLER`, handler block записывает код ошибки и функция возвращает false. В дизассемблере handler block может выглядеть отдельно от основного тела функции. В `.xdata` будет `UNW_FLAG_EHANDLER`, в handler data будут protected ranges. Но это не обязательно exception-based control flow в подозрительном смысле.

Почему? Потому что исключение здесь соответствует ошибочной ситуации, а handler переводит ее в обычный результат. Он не используется для каждого нормального шага алгоритма. Он не декодирует следующий block. Он не меняет `RIP` на скрытую ветку. Он не превращает exception dispatcher в главный switch. static должен описать это как SEH-protected region. dynamic , если запуск дойдет до плохого указателя, покажет first-chance exception, handler hit, потом возврат в обычный путь ошибки.

Здесь важна мера. Даже если такой код встречается много раз, это может быть стиль runtime-библиотеки или защитная проверка. Finding должен быть нейтральным или informational, если нет дополнительных признаков: context rewrite, deliberate traps, suspicious handler reuse, executable/writable metadata, dynamic function table injection.

7.7.10. Пример exception-based route: breakpoint как скрытый переход

Теперь другой пример. В функции есть `int 3`, но рядом нет обычного debug build поведения. В начале программы регистрируется VEH через `AddVectoredExceptionHandler`. VEH handler проверяет `ExceptionCode == EXCEPTION_BREAKPOINT`, затем читает байт после `RIP`, использует его как индекс, вычисляет новый адрес, записывает его в `ContextRecord->Rip` и возвращает `EXCEPTION_CONTINUE_EXECUTION`. В статическом graph view блок после `int 3` выглядит странно: либо как падение, либо как мусор, либо как недостижимый участок. Но в runtime программа нормально продолжает работу.

Это уже другой класс. Здесь breakpoint exception не является ошибкой. Оно является dispatch signal - сигналом диспетчеризации. Handler играет роль невидимого switch. `ContextRecord->Rip` становится переменной, через которую handler выбирает следующий basic block. Такой маршрут нельзя корректно понять, если смотреть только на обычные `jmp` и `call`.

static в этом случае должен собрать статическую цепочку: наличие `AddVectoredExceptionHandler`, наличие `int 3`, handler code, обращение к `ExceptionRecord`, изменение `ContextRecord->Rip`, возврат `EXCEPTION_CONTINUE_EXECUTION`. dynamic должен подтвердить динамически: first-chance breakpoint, handler hit, continue execution, новый `RIP`, trace последующих шагов. Только вместе они дают уверенный вывод: exception используется как управление.

7.7.11. Guard pages: нормальный механизм памяти или маршрут управления

Guard page - охраняемая страница памяти. На Windows страница с guard-флагом вызывает специальное исключение при первом доступе. Это может быть нормальный механизм: рост стека, lazy commitment, детектор переполнения, runtime-probe. Но guard pages также могут использоваться для exception-based routing: первый доступ к странице становится сигналом, handler переключает защиту, меняет состояние, переносит `RIP` или включает следующий участок кода.

Почему guard pages важны именно для нас? Потому что они связывают memory protection и exception dispatch. Статически static может увидеть вызовы `VirtualProtect` или `VirtualAlloc` с guard protection, но не всегда сможет понять, произойдет ли доступ реально. dynamic может увидеть runtime exception `STATUS_GUARD_PAGE_VIOLATION`, handler hit и дальнейший `RIP` trace. Если после guard exception поток идет в заранее подготовленный decoder или dispatcher, это уже сильнее, чем простой факт `PAGE_GUARD`.

Здесь снова нельзя делать грубый вывод. Guard page сама по себе не подозрительна. Подозрительна цепочка: guard page ставится на область, затем доступ намеренно вызывает exception, handler берет управление, меняет context или protection, а статический CFG не объясняет дальнейший маршрут. В книге это важно повторять: finding - это не слово API, а доказательная связка.

7.7.12. Dynamic function tables: когда карты функций появляются во время выполнения

До сих пор мы в основном говорили о `.pdata` как о таблице, лежащей в PE-файле. Но Windows x64 поддерживает и динамические function tables. Они нужны, например, JIT-компиляторам: код рождается во время выполнения, значит заранее в PE не может быть обычной `.pdata` для этого кода. Runtime регистрирует диапазоны и unwind metadata через специальные API, чтобы debugger и exception dispatcher могли делать unwind даже для динамического кода.

Для обычного JIT это нормальная инженерная необходимость. Но для reverse engineering это всегда особое место внимания. Если код распаковывается, генерируется, manual-map-ится или переносится в память, статический static может не увидеть полной картины в исходном файле. Он может только заметить API, suspicious sections, writable/executable memory, отсутствие ожидаемой `.pdata` или необычные references. dynamic же может поймать сам факт регистрации dynamic function table и показать, какие диапазоны были добавлены во время выполнения.

Это прекрасный пример разделения ролей. static говорит: “в файле есть признаки, что статическая карта неполная”. dynamic говорит: “во время выполнения появилась новая карта”. В отчете это должно быть связано: статическая `.pdata` описывает один мир, runtime function table добавляет другой мир. Если handler или unwind для важного кода существует только динамически, то без dynamic -а реверсер легко пропустит часть exception-механики.

7.7.13. Как формулировать findings без истерики

Одна из целей нашего проекта - не просто найти признаки, а объяснять их профессионально. Плохой отчет пишет: “обнаружена обфускация”. Хороший отчет пишет: “найдена цепочка признаков, совместимая с exception-based control flow; требуется динамическая проверка; confidence такой-то; evidence такое-то”.

Почему это важно? Потому что почти каждый отдельный признак имеет нормальное объяснение. `RaiseException` может быть частью API контракта. VEH может быть диагностикой. `SetUnhandledExceptionFilter` может быть crash

reporter-ом. `RtlAddFunctionTable` может быть JIT. `UNW_FLAG_EHANDLER` может быть обычным `__try/__except`. Даже `int 3` может быть отладочным остатком. Ошибка начинается, когда инструмент превращает один признак в окончательный вердикт.

Поэтому finding должен иметь слои. Первый слой - evidence: что именно найдено. Второй слой - location: где найдено, в каком модуле, RVA, функции, handler-е. Третий слой - interpretation: что это может означать. Четвертый слой - confidence: насколько сильна связка. Пятый слой - validation advice или dynamic result: что показал dynamic или что нужно проверить динамически.

```
Слабая формулировка:

    Suspicious exception handling detected.

Сильная формулировка:

    Function module+0x1230 contains a deliberate breakpoint site.
    A VEH registration path is present at module+0x4010.
    The candidate VEH handler reads ExceptionRecord->ExceptionCode and writes ContextRecord->Rip.
    Static CFG has no normal edge to the post-handler target.
    Dynamic validation observed EXCEPTION_BREAKPOINT first-chance, handler hit at module+0x5000,
    and continuation at module+0x6200.
    Interpretation: exception-based control flow is likely.
```

7.7.14. Как static и dynamic должны сходиться в одной карте

Теперь соберем модель. static начинает с файла. Он видит bytes, PE headers, sections, RVAs, Exception Directory, `.pdata`, `.xdata`, function ranges, unwind metadata, handler RVAs, suspicious placements and reuse. Он отвечает на вопрос: “какую карту обещает файл?”

dynamic начинает с процесса. Он видит debug events, modules loaded into memory, exceptions, handler hits, API breakpoints, dynamic function table registrations, `RIP` trace. Он отвечает на вопрос: “какой маршрут реально произошел?”

Между этими вопросами есть напряжение. Файл может обещать одно, runtime делать другое. Статическая metadata может быть корректной, но путь никогда не достигаться. Handler может существовать, но быть diagnostic-only. Dynamic function table может появиться только после определенного условия. Exception-based route может включаться только при debugger-е или только без debugger-а. Поэтому архитектура проекта должна мыслиться не как “static против dynamic”, а как два слоя одной доказательной системы.

В идеале отчет должен позволять человеку пройти цепочку от адреса к поведению. Например: `RIP` попал в `module+0x1450`; static нашел, что это RVA внутри функции `[0x1400, 0x1510)`; `RUNTIME_FUNCTION` указывает на `UNWIND_INFO`; у `UNWIND_INFO` есть `UNW_FLAG_EHANDLER`; handler RVA равен `0x3000`; dynamic

подтвердил handler hit; после handler-а `RIP` стал `module+0x1800`. Такая цепочка превращает хаос исключений в читаемую историю.

7.7.15. Практическая шкала уверенности

Для будущего анализа полезно держать простую шкалу. Низкая уверенность: есть один слабый статический признак, например импорт `RaiseException` или наличие VEH API. Средняя уверенность: есть несколько связанных статических признаков, например deliberate exception site плюс handler, который читает exception code. Высокая уверенность: handler изменяет context или возвращает continue execution, а рядом есть маршрут, который обычный CFG не объясняет. Очень высокая уверенность: dynamic динамически подтвердил exception, handler hit и продолжение исполнения в неожиданную точку.

Эта шкала не является математической истиной. Это инженерная дисциплина. Она заставляет инструмент не кричать “обфускация” там, где есть только один API, и не молчать там, где static и dynamic evidence сошлись. Для пользователя отчета это критично: он должен понимать, какие выводы надежны, а какие являются гипотезами.

В книге мы будем постоянно возвращаться к этой идее. Reverse engineering - это не гадание по одной инструкции. Это построение объяснения. Хорошее объяснение связывает адреса, metadata, код, стек, context, handler decisions и runtime observations. Именно поэтому мы так медленно разбирали `RIP`, `RSP`, return address, `RUNTIME_FUNCTION`, `UNWIND_INFO`, `UNWIND_CODE`, `__C_specific_handler`: без этих кирпичей невозможно честно объяснить exception-based route.

7.7.16. Типичные ошибки начинающего реверсера

Первая ошибка - считать любое исключение падением. First-chance exception может быть частью нормального маршрута. Если после него handler обработал событие и программа продолжила работу, это не crash. dynamic должен явно отделять observed exception от unhandled termination.

Вторая ошибка - считать любой handler подозрительным. На Windows x64 unwind и exception metadata являются нормальной частью ABI. Наличие `.pdata`, `.xdata`, `UNW_FLAG_EHANDLER` и `__C_specific_handler` само по себе ничего криминального не доказывает.

Третья ошибка - игнорировать `ContextRecord`. Если handler меняет `RIP`, `RSP` или важные регистры, именно там может быть скрыт реальный переход. Реверсер, который смотрит только на graph view, может пропустить главный маршрут.

Четвертая ошибка - анализировать API без контекста. `RaiseException`, `AddVectoredExceptionHandler`, `SetUnhandledExceptionFilter`, `RtlAddFunctionTable`

- это не ярлыки “плохо” или “хорошо”. Их смысл появляется только в связке с code sites, handlers, return values, metadata и runtime behavior.

Пятая ошибка - верить статике без динамики или динамике без статики. Статика может показать потенциальные routes, которые не достигаются. Динамика может показать один route, но пропустить другие из-за неполного покрытия. Поэтому static и dynamic нужны вместе.

Шестая ошибка - не различать cleanup path и control-flow path. Cleanup может выглядеть как отдельная ветка и даже быть недостижимым обычным `jmp`, но его смысл - освободить ресурсы при выходе. Control-flow path использует exception как способ выбрать следующий смысловой блок исполнения. Это разные вещи.

7.7.17. Главная модель раздела

Теперь можно сформулировать короткую, но очень важную модель. Обычный unwind отвечает на вопрос: “как корректно вернуться к вызывающему контексту или пройти стек при исключении?”. Exception-based control flow отвечает на другой вопрос: “как использовать исключение как механизм выбора следующего адреса исполнения?”. Эти два явления используют одни и те же системные детали: `RIP`, `RSP`, `CONTEXT`, `EXCEPTION_RECORD`, `.pdata`, `.xdata`, handlers. Но назначение у них разное.

static видит статический договор: что PE обещает Windows и какие exception routes потенциально существуют. dynamic видит runtime-исполнение: какие exceptions реально возникли, какие handlers реально получили управление, куда ушел `RIP` после обработки. Реверсер должен соединить оба слоя и не путать наличие механизма с его назначением.

В следующем большом блоке мы сможем перейти к аномалиям, обфускации и защите. Теперь у нас есть основание для этого перехода. Мы уже умеем читать нормальную карту `.pdata/.xdata`. Мы умеем понимать, что такое handler. Мы умеем мыслить не только `call/ret`, но и exception edges. И теперь мы можем задавать правильный вопрос: когда metadata выглядит странно, это ошибка компилятора, особенность runtime-а, JIT/manual mapping, packer, обфускация или намеренная exception-based маршрутизация?

Именно этот вопрос станет входом в следующую главу. Там мы будем смотреть не на нормальную механику, а на случаи, где карта функций, unwind metadata и реальное исполнение начинают расходиться.

Техническая база раздела

Этот раздел опирается на официальную модель Windows x64 exception handling: таблицы `RUNTIME_FUNCTION` в `.pdata`, связанные `UNWIND_INFO` в `.xdata`,

поведение unwind procedure, vectored exception handling, debugger first-chance / second-chance notifications, `ContinueDebugEvent`, а также API, связанные с deliberate exceptions и dynamic function tables. В проектной терминологии раздел также связывает static как статическую карту PE и dynamic как динамическую проверку фактического маршрута исполнения.

Глава 8. Аномалии, обфускация и защитные трюки

8.1. Когда `.pdata` выглядит странно: медленный разбор от нормальной карты функций к реальному сигналу

В предыдущей главе мы построили ручную модель: взять текущий RIP, найти модуль, перевести адрес в RVA, попасть в Exception Directory, прочитать массив RUNTIME_FUNCTION, перейти в .xdata, разобрать UNWIND_INFO, сверить UNWIND_CODE с прологом и понять, как Windows сможет восстановить caller context. Это была не просто учебная цепочка. Это именно тот путь, который лежит в основе static -a: статического слоя анализатора, который читает файл как данные и строит карту функций, кадров, handler-ов и возможных проблем.

Теперь мы меняем угол зрения. До этого мы спрашивали: как всё должно быть устроено в нормальной модели Windows x64? В главе 8 вопрос становится другим: что делать, если эта модель выглядит странно? Что значит странно? Это ошибка компилятора? Особенность линкера? Результат упаковщика? Следствие JIT-кода? Намеренная обфускация? Или просто нормальный маленький gap между двумя функциями, который новичок принял за катастрофу? Именно здесь начинается аккуратная инженерная криминалистика, но слово “криминалистика” не должно сбивать: мы не бросаемся обвинять файл. Мы учимся отличать доказательство от подозрения, сигнал от шума, нарушение контракта от допустимой вариативности.

> Главная мысль раздела: .pdata надо читать не как таблицу “хорошо/плохо”, а как карту обещаний. Каждая запись говорит Windows: “если RIP попадёт в этот диапазон, раскручивать кадр надо вот так”. Если карта неполная, противоречивая или указывает в странные места, static фиксирует это как статический сигнал. dynamic затем может проверить, проявляется ли сигнал в живом выполнении: куда реально ушёл RIP, какой handler сработал, возникло ли исключение, совпала ли динамическая траектория со статической картой.

8.1.1. Сначала договоримся о словах: аномалия, ошибка, обфускация, сигнал

Термин “аномалия” звучит сильно, но в анализе бинарников он не означает автоматически “вредоносность”. Аномалия - это отклонение от ожидаемой формы. Если мы ожидаем, что BeginAddress меньше EndAddress, а встречаем обратное, это аномалия. Если мы ожидаем, что UnwindInfoAddress ведёт в читаемую область образа, а он указывает за пределы SizeOfImage, это аномалия. Если сотни разных функций указывают на один и тот же handler, это тоже может быть аномалией, хотя сама по себе

она ещё не доказывает злой умысел.

Слово “ошибка” уже сильнее. Ошибка - это такое состояние, при котором структура нарушает базовый контракт и нормальный потребитель этой структуры не может надёжно её использовать. Например, если запись `RUNTIME_FUNCTION` имеет диапазон `[0x2000, 0x1000)`, она не описывает ни одного нормального участка кода: начало стоит после конца. Такая запись не просто необычна, она логически сломана. Но даже здесь надо быть осторожным: файл мог быть повреждён, криво сгенерирован учебным генератором, частично обрезан, намеренно испорчен упаковщиком или просто не являться тем типом PE, который мы думаем анализировать.

Слово “обфускация” означает намеренное усложнение понимания. Обфускация не обязательно ломает программу. Хорошая обфускация часто делает наоборот: программа работает, но её структура становится неудобной для статического анализа. В контексте `.pdata` обфускация может проявляться как странная неполная карта функций, повторное использование metadata, перенаправление многих диапазонов к одному handler-у, динамическая регистрация function table через `RtlAddFunctionTable`, или разрыв между тем, что видно в файле, и тем, что реально появляется во время выполнения.

Слово “сигнал” в этой книге мы используем особенно аккуратно. Сигнал - это наблюдение, которое повышает интерес аналитика к месту. Сигнал не равен приговору. Если static говорит: “metadata лежит в writable section”, он не говорит: “это malware”. Он говорит: “обычная compiler-generated unwind metadata редко должна быть изменяемой во время выполнения; проверь, почему она writable”. А dynamic , если включена динамическая валидация, может посмотреть, действительно ли эта область участвует в обработке исключений, переписывается ли она, попадает ли туда RIP, вызываются ли dynamic function table API.

Эта разница между сигналом и выводом принципиальна для всей архитектуры проекта. static строит статическую evidence-карту. Evidence - это “свидетельство”, наблюдаемый факт: запись пересекается с другой, handler лежит в секции без execute-права, `UNWIND_INFO` не парсится. dynamic добавляет runtime evidence: был handler hit, был first-chance exception, был вызов `RtlAddFunctionTable`, был RIP trace после попадания в handler. Только соединение этих слоёв даёт более серьёзный вывод.

8.1.2. Нормальная модель `.pdata`: что именно мы считаем “обычным”

Чтобы понять странность, сначала надо снова медленно восстановить норму. `.pdata` - это распространённое имя секции, где в PE32+ AMD64 обычно лежит таблица `RUNTIME_FUNCTION`. Но важный нюанс: Windows не должна верить имени секции. Имя `.pdata` удобно человеку, но для загрузчика и системных функций главным является `Exception Directory` в PE Data

Directory. Data Directory - это таблица указателей в PE Optional Header, где образ говорит: “вот где у меня лежит такая-то служебная структура”. Поэтому правильный путь static -а начинается не с поиска секции по имени `.pdata`, а с чтения `IMAGE_DIRECTORY_ENTRY_EXCEPTION`.

Термин `Exception Directory` можно перевести как “каталог данных исключений”, но в данном контексте это не каталог файлов и не папка. Это запись в PE-заголовке, у которой есть `VirtualAddress` и `Size`. `VirtualAddress` здесь означает RVA, то есть relative virtual address - адрес относительно `ImageBase`. `Size` показывает, сколько байт занимает таблица. На x64 эта таблица обычно состоит из записей `RUNTIME_FUNCTION`, каждая по 12 байт: начало функции, конец функции и адрес unwind-информации.

```
typedef struct _RUNTIME_FUNCTION {
    ULONG BeginAddress;      // RVA начала диапазона функции
    ULONG EndAddress;        // RVA конца диапазона функции
    ULONG UnwindInfoAddress; // RVA структуры UNWIND_INFO в .xdata/.rdata/etc.
} RUNTIME_FUNCTION;
```

Три поля выглядят простыми, и именно поэтому их часто недооценивают. `BeginAddress` - это не “адрес красивого имени функции в дизассемблере”, а начало диапазона машинного кода, для которого действует данная unwind-запись. `EndAddress` - это граница, где этот диапазон заканчивается. `UnwindInfoAddress` - это не адрес обработчика исключения, а адрес структуры `UNWIND_INFO`, которая объясняет, как откатить последствия пролога функции: сколько функция выделила на стеке, какие nonvolatile-регистры сохранила, где они лежат, есть ли handler или chained info.

Здесь появляется термин `range` - диапазон. В `.pdata` функция описывается не именем, а диапазоном адресов. Если `BeginAddress = 0x1000`, а `EndAddress = 0x1080`, запись говорит: “все инструкции с RVA от `0x1000` до `0x107F` относятся к этой функции или части функции с этой unwind-моделью”. Обычно такой диапазон удобно записывать как `[BeginAddress, EndAddress)`. Квадратная скобка слева означает “начало включается”, круглая справа означает “конец не включается”. Поэтому адрес `0x1000` внутри диапазона, а `0x1080` уже нет.

Почему это важно? Потому что системная логика ищет запись по текущему `RIP`. Если во время исключения `RIP` указывает на инструкцию с RVA `0x1042`, Windows должна найти такую запись `RUNTIME_FUNCTION`, где `BeginAddress <= 0x1042 < EndAddress`. Найденная запись становится входом в `.xdata`. Так из одного текущего адреса выполнения получается вся цепочка: `RIP` -> `RUNTIME_FUNCTION` -> `UNWIND_INFO` -> восстановление кадра или вызов language-specific handler.

Нормальная таблица должна быть внутренне понятной. Диапазоны не должны быть перевёрнутыми. Записи должны быть отсортированы по `BeginAddress`, потому что это позволяет быстро искать нужный диапазон.

Диапазоны в обычной модели не должны перекрываться, потому что один и тот же RIP не должен одновременно принадлежать двум разным unwind-контрактам. UnwindInfoAddress должен указывать на корректно читаемую структуру UNWIND_INFO, а не в пустоту, не за пределы образа и не в середину случайных инструкций.

8.1.3. Почему .pdata - это не список всех функций

Одна из самых частых ошибок новичка - ожидать, что .pdata является полным списком всех функций программы. Это удобно психологически: видим таблицу, видим диапазоны, хочется считать её “каталогом функций”. Но это неточно. В Windows x64 .pdata существует не ради красоты дизассемблера, а ради exception handling, unwinding и debugger support. То есть её главная задача - дать системе достаточно данных, чтобы восстановить состояние caller-а из произвольной точки внутри nonleaf-функции.

Термин leaf function - “листовая функция” - означает функцию, которая не вызывает другие функции и не требует сложной раскрутки кадра. В дереве вызовов leaf - это “лист”, крайняя точка: она никого ниже себя не вызывает. Если такая функция не выделяет стек, не сохраняет nonvolatile-регистры и не делает ничего, что требует unwind metadata, для неё может не быть отдельной записи в .pdata. Если во время unwind система не находит запись function table для текущего RIP, она может трактовать код как leaf case: return address лежит прямо по текущему RSP, и caller context можно получить простым чтением [RSP] и увеличением RSP на 8.

Поэтому отсутствие покрытия части .text не всегда означает проблему. В секции кода могут быть короткие leaf helper-ы, thunks, padding, выравнивание, служебные вставки, горячие патчи, import stubs, jump islands, компиляторские куски, которые не требуют самостоятельной unwind-записи. Термин thunk обычно означает маленький переходник: короткий фрагмент кода, который почти ничего сам не делает, а передаёт управление дальше. Например, thunk может быть просто jmp в реальную функцию. Для такого фрагмента отдельная unwind metadata может быть не нужна.

Но отсюда не следует обратная крайность: “если gap возможен, то любые gaps нормальны”. Gap - это промежуток между диапазонами .pdata, который лежит в executable code, но не покрыт RUNTIME_FUNCTION. Маленький gap между функциями может быть padding-ом: набором байтов для выравнивания, например nop или int 3, которые не являются обычной логикой функции. Большой gap внутри плотного кода, где есть call, sub rsp, сохранение rbx и сложный control flow, уже выглядит иначе. static должен не просто считать gap, а пытаться понять контекст: лежит ли он в executable section, похож ли он на код, есть ли там prolog-like instructions, ведут ли туда cross-references, попадает ли туда RIP в динамике.

> Правильное мышление: `.pdata` - это не “все функции”, а “все диапазоны, для которых системе нужна таблица раскрутки”. Поэтому отсутствие записи может быть нормой, а может быть слепым пятном. Отличие между нормой и слепым пятном рождается не из одного факта, а из контекста.

8.1.4. Coverage: покрытие кода unwind-картой

Термин `coverage` переводится как “покрытие”. В нашем контексте это вопрос: какая часть executable code покрыта записями `RUNTIME_FUNCTION`? Если у нас есть секция `.text` от `0x1000` до `0x9000`, а `.pdata` описывает диапазоны только от `0x2000` до `0x4000`, то покрытие частичное. Само по себе частичное покрытие не равно ошибке, но это повод спросить: что находится в непокрытой области?

Coverage полезно мыслить не только процентом. Процент может обмануть. Например, если огромная непокрытая область - это нули, padding или таблицы данных, случайно лежащие в executable section, это один сценарий. Если непокрытая область содержит плотный код, на который есть переходы из покрытых функций, это совсем другой сценарий. Если непокрытая область появляется только после unpacking-а во время выполнения, статический static увидит пустоту, а dynamic может увидеть, что target создаёт динамическую function table через `RtlAddFunctionTable` или `RtlInstallFunctionTableCallback`.

Здесь начинает проявляться реальная связка проекта. static, как статический слой, может сказать: “в файле есть executable region, для которого я не вижу обычной `.pdata`”. Это static evidence. dynamic, как динамическая валидация, может проверить: выполняется ли этот регион, регистрируются ли для него runtime function tables, возникают ли exceptions внутри него, возможно ли построить stack trace, куда уходит RIP после handler hit. Если динамика показывает, что код действительно исполняется и для него в рантайме появляется function table, статическая “дыра” получает объяснение. Если код исполняется, exceptions происходят, но корректной runtime metadata нет, сигнал становится сильнее.

Важно понимать, что coverage не должен превращаться в примитивную метрику “чем меньше, тем хуже”. Маленькие программы, loader stubs, специфичные рантаймы и некоторые compiler/linker configurations могут давать непривычную форму. Поэтому хорошая система findings должна писать не “bad coverage”, а объяснять: какой регион непокрыт, почему он похож на код, какие признаки усиливают подозрение, какие признаки ослабляют, какие шаги ручной или динамической проверки нужны дальше.

8.1.5. Invalid range: когда запись сама себе противоречит

Самая простая и самая фундаментальная проблема `.pdata` - invalid range, то есть недопустимый диапазон. Диапазон в `RUNTIME_FUNCTION` должен иметь смысл: `BeginAddress` меньше `EndAddress`. Если начало равно концу, диапазон пустой. Если начало больше конца, диапазон перевёрнут. В обоих случаях запись не может нормально отвечать на вопрос “попадает ли RIP внутрь этой функции?”.

```
Нормально:
    BeginAddress = 0x1400
    EndAddress   = 0x1480
    Диапазон: [0x1400, 0x1480)

Пусто:
    BeginAddress = 0x1400
    EndAddress   = 0x1400
    Диапазон не содержит ни одного адреса

Перевёрнуто:
    BeginAddress = 0x1480
    EndAddress   = 0x1400
    Логический диапазон невозможен
```

Почему это важно не только для парсера? Потому что вся unwind-модель начинается с поиска диапазона по `ControlPc`. `ControlPc` - это адрес управления, обычно связанный с текущим RIP внутри функции. Если таблица содержит логически невозможные диапазоны, поиск превращается в неопределённую ситуацию для анализатора. Разные потребители могут отфильтровать такую запись, пропустить её, сломаться, дать неверный stack trace или посчитать код leaf-функцией там, где leaf-модель неверна.

static должен считать invalid range сильным static signal-ом, потому что здесь нарушается базовая геометрия карты. Но даже такой сигнал надо объяснять. Если одна запись битая в огромном benign-файле, это может быть редкая ошибка инструмента, повреждение файла или edge case. Если битых диапазонов много, если они совпадают с другими проблемами - например `UnwindInfoAddress` тоже указывает за пределы образа, секции пересекаются, metadata writable, - тогда картина становится гораздо серьёзнее.

dynamic в такой ситуации не исправляет таблицу магически. Его роль другая: проверить, проявляется ли проблема при исполнении. Если исключение происходит внутри диапазона, который статически выглядит битым или непокрытым, динамический слой может показать, нашла ли система function table entry, был ли handler hit, смог ли процесс продолжить выполнение, ушёл ли RIP туда, куда ожидалось. Даже если dynamic run не попал в проблемный адрес, это всё равно полезная информация: статический сигнал остался непроверенным, и в отчёте честно пишется, что runtime correlation отсутствует.

8.1.6. Out-of-bounds: когда адрес указывает за пределы допустимого мира

Термин `out-of-bounds` буквально означает “за границами”. В PE-анализе границы бывают разные. Есть границы файла на диске: нельзя читать байты за пределами `file_size`. Есть границы virtual image: нельзя считать нормальным RVA, который выходит за `SizeOfImage`. Есть границы секции: RVA может попадать или не попадать в какую-то section mapping. Есть границы конкретной структуры: `Exception Directory` имеет размер, кратный размеру записи `RUNTIME_FUNCTION`, иначе последняя запись может быть неполной.

Для `.pdata` особенно важны два типа out-of-bounds. Первый: сама таблица `RUNTIME_FUNCTION` выходит за пределы образа или файла. Тогда static не может безопасно прочитать все записи. Второй: поле внутри записи указывает за пределы образа. Например, `UnwindInfoAddress` указывает на RVA, который не принадлежит никакой читаемой секции. Это означает, что связка `.pdata -> .xdata` разорвана. У нас есть диапазон кода, но нет достоверной инструкции, как этот код раскручивать.

Новичок может спросить: если поле `UnwindInfoAddress` просто число, почему нельзя попытаться интерпретировать его как file offset? Потому что в PE почти все такие адреса внутри runtime structures являются RVA, а не file offset. RVA - это адрес относительно `ImageBase` после загрузки образа в память. `file offset` - смещение байта внутри файла на диске. Между ними есть перевод через section table. Если перепутать эти два пространства, можно случайно “найти” структуру там, где её нет, или объявить нормальный адрес битым.

Out-of-bounds-сигнал особенно ценен для static -а, потому что он относится к trust boundary. Trust boundary - граница доверия. Входной PE-файл недоверенный. Анализатор не имеет права читать по указателю просто потому, что файл так сказал. Он обязан проверять: помещается ли чтение в буфер, помещается ли RVA в образ, можно ли перевести RVA в file offset, не переполняется ли арифметика. Иначе сам анализатор станет жертвой malformed sample-а.

В книге мы готовим не только реверсера-человека, но и архитектуру инструмента. Поэтому out-of-bounds надо объяснять как двойной вопрос: с одной стороны, это признак странного PE; с другой - это проверка надёжности самого static -а. Хороший static не падает на сломанном PE. Он сохраняет parse failure reason и формулирует finding. Хороший dynamic не должен запускать всё подряд без изоляции: динамическая валидация пересекает execution boundary, поэтому untrusted target должен выполняться в отдельной VM или sandbox-среде.

8.1.7. Sorting: почему порядок записей не косметика

Записи RUNTIME_FUNCTION обычно должны быть отсортированы по BeginAddress. На первый взгляд сортировка кажется удобством, но не фундаментом: ведь можно же пройти таблицу линейно и найти нужный диапазон. Для маленькой таблицы да. Но системная модель рассчитана на быстрый поиск по текущему RIP, а отсортированный массив позволяет применять binary search - двоичный поиск. Двоичный поиск работает только тогда, когда порядок записей соответствует правилу сортировки.

Binary search - это способ искать элемент, каждый раз отбрасывая половину массива. Если мы ищем адрес 0x5000, смотрим в середину таблицы и видим диапазон, начинающийся с 0x7000, мы понимаем, что нужный диапазон, если он есть, должен быть левее. Но это рассуждение верно только при сортировке. Если таблица перемешана, двоичный поиск может пропустить правильную запись или прийти к неправильному выводу.

Для static -a unsorted table - это сигнал table integrity family. table integrity можно перевести как “целостность таблицы”. Речь не о криптографической целостности, а о внутренней структурной согласованности. Таблица должна быть такой формы, чтобы её можно было потреблять по ожидаемому алгоритму. Если порядок нарушен, это не обязательно означает, что программа не работает, но это означает, что карта функций уже не похожа на обычный compiler/linker output.

При этом важно не делать вывод слишком быстро. Иногда файл может содержать необычные toolchain-specific артефакты, post-link transformations, секции после упаковки или вручную собранные таблицы. Поэтому finding должен звучать так: “table not sorted by BeginAddress; verify whether this image uses nonstandard generation, packing, or runtime function table registration”. Человеческим языком: “карта функций разложена не в ожидаемом порядке; надо проверить, почему”.

dynamic может добавить к этому runtime context. Например, если статическая таблица unsorted, но в динамике target регистрирует корректную function table для реально исполняемого JIT-региона, статическая проблема может относиться только к файлу на диске, а живой маршрут обслуживается другими данными. Если же исключения происходят внутри статически странных диапазонов и динамика показывает handler hits по тем же адресам, это уже корреляция: странная карта не просто лежит в файле, она реально участвует в выполнении.

8.1.8. Overlap: самое опасное нарушение карты диапазонов

Overlap - перекрытие. В .pdata это ситуация, когда два диапазона RUNTIME_FUNCTION покрывают один и тот же адрес. Например, первая запись описывает [0x1000, 0x1100), а вторая - [0x1080, 0x1180). Адрес 0x1090 попадает сразу в обе. Для человека это уже выглядит странно, но для unwind-модели проблема ещё глубже: если исключение происходит по RIP =

0x1090, какой UNWIND_INFO надо использовать?

```
Запись A:
    BeginAddress = 0x1000
    EndAddress   = 0x1100
    UnwindInfo   = 0x3000

Запись B:
    BeginAddress = 0x1080
    EndAddress   = 0x1180
    UnwindInfo   = 0x3400

Адрес 0x1090 находится и в A, и в B.
Это означает конфликт двух unwind-контрактов.
```

Слово contract - договор - здесь очень полезно. UNWIND_INFO говорит: “если ты находишься внутри этого диапазона, чтобы вернуться к caller-у, надо отменить такие-то действия пролога”. Если два разных контракта претендуют на один и тот же RIP, они могут говорить разные вещи. Один может считать, что RBX сохранён по [RSP+0x20], другой - что он не сохранён. Один может считать, что функция выделила 0x40 байт стека, другой - что 0x100. Один может иметь EHANDLER, другой - нет. Ошибка выбора контракта ломает stack walking, exception dispatch и иногда саму возможность корректно обработать исключение.

Overlap не всегда одинаков по тяжести. Перекрытие на один байт около границы может быть следствием malformed metadata или off-by-one ошибки генератора. Большие перекрытия внутри плотного кода выглядят сильнее. Перекрытие, которое сопровождается duplicate begin, out-of-bounds unwind RVA, parse failures и suspicious placement, превращается в сильный комплексный сигнал. Именно поэтому проектная система findings должна иметь не только одну строку “rf_overlaps”, но и семейства evidence, score, confidence и manual validation steps.

Для dynamic -а overlap интересен тем, что он может проверить реальный маршрут. Статически мы видим две возможные интерпретации. Динамически можно поставить breakpoints, дождаться exception или handler hit и посмотреть, какой адрес реально исполняется, какая запись используется косвенно через поведение системы, куда уходит RIP после обработки. Иногда динамика не сможет прямо сказать “Windows выбрала запись A”, но она может показать, что исключение в перекрытой зоне приводит к конкретному handler-у, конкретному continuation path или crash path. Это уже runtime evidence, которое помогает отделить мёртвую странность в файле от живой логики.

8.1.9. Duplicate BeginAddress: две функции начинаются в одной точке?

Duplicate BeginAddress - ситуация, когда две записи RUNTIME_FUNCTION имеют одинаковый BeginAddress. Наивная реакция: “две функции не могут

начинаться с одного адреса”. Обычно да, но надо снова говорить точнее. В `.pdata` запись не обязательно означает “исходная функция с именем”. Она означает диапазон с unwind-информацией. Поэтому duplicate begin - это не философская невозможность, а структурная неоднозначность: два table entry начинают описывать код с одной точки, но могут иметь разные `EndAddress` или разные `UnwindInfoAddress`.

Если два диапазона имеют один и тот же begin и один и тот же end, это может выглядеть как дубликат записи. Если begin одинаковый, а end разный, возникает overlap. Если unwind адреса разные, возникает ещё более неприятный вопрос: какая `.xdata` является правдой? Если handler data различается, один и тот же код как будто участвует в разных exception-contracts. Для нормального compiler output это нетипично.

В анализаторе duplicate begin удобно выделять отдельно от overlap, потому что это другая форма симптома. Overlap может произойти между двумя разными start address. Duplicate begin говорит: конфликт начинается уже в первой координате диапазона. Это может указывать на повреждение таблицы, ошибку генерации, ручную правку, упаковку или намеренную попытку запутать инструменты, которые строят функцию по первой найденной записи.

В Ghidra/IDA-подобной работе duplicate begin может порождать практическую путаницу. Дизассемблер может уже иметь функцию с этим start address, а импорт unwind CSV пытается добавить комментарии или warning bookmarks для нескольких записей. Если инструмент слепо назовёт всё “function”, человек может решить, что перед ним две нормальные функции. Поэтому книга должна приучать к правильной фразе: “две `RUNTIME_FUNCTION` записи претендуют на один и тот же start RVA”, а не “две функции реально начинаются в одном месте”.

8.1.10. Gaps: почему пустоты между диапазонами надо читать глазами

Gap - промежуток. Если запись A заканчивается на `0x2000`, а следующая запись начинается на `0x2040`, между ними есть gap `[0x2000, 0x2040)`. Новичок может сразу испугаться: “сорок байт кода не покрыто `.pdata`”. Но сначала надо спросить: это код или не код? И даже если это executable section, являются ли эти байты реальными инструкциями, которые достижимы при выполнении?

В PE executable section может содержать не только “логический код функции”. Там могут быть alignment bytes, padding, короткие trap sequences, таблицы переходов, хвосты, hotpatch padding. Иногда дизассемблер по инерции пытается декодировать любые байты как инструкции, но это не доказывает, что они являются достижимым кодом. Поэтому gap надо анализировать на нескольких уровнях.

- Байтовый уровень: какие байты лежат в gap? Нули, `0xCC`, `nop`, случайный кодоподобный мусор, реальные инструкции?
- Секционный уровень: gap лежит в executable section или в данных? Какие у секции права: read, write, execute?
- CFG-уровень: есть ли переходы или calls в этот gap? Есть ли fall-through из предыдущего диапазона?
- ABI-уровень: есть ли prolog-like pattern: `push rbx`, `sub rsp, ...`, сохранение nonvolatile-регистров?
- Динамический уровень: попадает ли туда `RIP` при запуске под dynamic -ом? Возникают ли exceptions внутри gap?

Именно поэтому gap - хороший пример того, почему static не должен выдавать слишком грубый verdict. Статически он может отметить “uncovered executable region” или “possible code gap near runtime-function table”. Но если gap состоит из `int 3` padding между функциями, это нормальная картина. Если gap содержит плотный пролог и вызывается из других функций, это уже другое. Если dynamic показывает, что `RIP` реально входит в этот gap и затем через exception уходит в handler, сигнал становится намного сильнее.

В хорошем отчёте gap должен быть объяснён человеку в координатах, которые он может проверить: RVA начала, RVA конца, секция, размер, ближайшая предыдущая и следующая `RUNTIME_FUNCTION`, байтовый preview, наличие cross-reference, и рекомендация: “проверь в дизассемблере, является ли region code-like”. Это и есть difference между тезисным списком и инженерным анализом.

8.1.11. `UnwindInfoAddress`: почему это поле часто неправильно понимают

В `RUNTIME_FUNCTION` третье поле называется `UnwindInfoAddress`, и его легко прочитать поверхностно: “адрес unwind info”. Но последствия ошибки здесь большие. Это поле не указывает на пользовательский exception handler. Оно указывает на структуру `UNWIND_INFO`. Уже внутри `UNWIND_INFO`, если выставлены флаги `UNW_FLAG_EHANDLER` или `UNW_FLAG_UHANDLER`, после массива `UNWIND_CODE` может лежать address of exception handler - обычно language-specific handler, например `__C_specific_handler`, а не непосредственно тело пользовательского `__except`.

Поэтому finding “unwind RVA is out of bounds” не означает “handler плохой”. Он означает, что цепочка обрывается ещё до handler-а: `RUNTIME_FUNCTION` не может привести нас к валидному `UNWIND_INFO`. A finding “handler target in non-executable section” появляется только после успешного чтения `UNWIND_INFO`, определения flags, вычисления aligned tail и чтения handler RVA. Это разные слои, и смешивать их нельзя.

Ещё одна типичная ошибка - считать, что `UnwindInfoAddress` должен всегда вести именно в секцию `.xdata`. В MSVC-картине часто так и будет, но имя секции не является абсолютным законом. Unwind data может находиться в `.rdata`, `.pdata` или toolchain-specific секциях. Более важны права и смысл: структура должна быть читаемой, корректно выровненной, не выходить за границы образа и соответствовать формату `UNWIND_INFO`. Секция с необычным именем - слабее, чем секция с executable/write правами. Writable/executable placement - более сильный сигнал, потому что metadata становится изменяемой или смешивается с исполняемым кодом.

Здесь полезно вспомнить `UNWIND_INFO` как инструкцию по разборке кадра. Если эта инструкция лежит в обычной read-only data section, картина спокойная: компилятор записал статический контракт. Если инструкция лежит в writable section, появляется вопрос: кто и зачем может менять контракт во время выполнения? Если инструкция лежит в executable section, появляется другой вопрос: это действительно data или анализатор смотрит в код? Если handler RVA ведёт в non-executable section, вопрос ещё острее: как handler может исполняться, если секция не исполняемая? Возможно, права в памяти меняются, возможно, секционная модель нестандартная, возможно, metadata битая. Но это нужно проверять, а не додумывать.

8.1.12. Metadata placement: почему права секции важнее имени секции

Термин `placement` означает размещение. В нашем случае - где именно лежит служебная metadata: в какой секции, с какими правами, в каком диапазоне RVA. Нормальная compiler-generated unwind metadata обычно является данными, а не кодом. Она нужна системе для чтения, но не должна выполняться как инструкции. Она также обычно не должна изменяться во время работы программы. Поэтому read-only data placement выглядит спокойнее, чем writable или executable placement.

Секции PE имеют characteristics - признаки, среди которых важны `IMAGE_SCN_MEM_READ`, `IMAGE_SCN_MEM_WRITE`, `IMAGE_SCN_MEM_EXECUTE`. Если секция executable, процессор может исполнять байты из неё как код. Если секция writable, программа может менять байты в ней после загрузки. Комбинации прав дают разные смысловые вопросы. Read-only metadata - нормальная модель. Writable metadata - возможность самопереписывающегося unwind-контракта. Executable metadata - смешение кода и данных. Writable + executable - особенно интересная комбинация, потому что байты могут и изменяться, и исполняться.

Но опять же нельзя превращать это в примитивное правило. Некоторые dynamic s, packers, JIT-runtimes и instrumentation frameworks могут иметь нестандартную layout-модель. Иногда секции получают широкие права из-за простоты упаковщика, а не из-за конкретного exception-трюка. Иногда handler

target может находиться в секции с необычным именем, но корректными execute-правами. Иногда `.xdata` может оказаться объединённой в `.rdata`. Поэтому правильный finding должен разделять: unusual section name, executable unwind section, writable unwind section, non-executable handler target, writable handler target. Это разные сигналы разной силы.

В проектной архитектуре это соответствует семье `metadata_placement`. Семья - это группа родственных findings. Зачем нужна семья? Чтобы не считать двадцать однотипных симптомов двадцатью независимыми доказательствами. Если все признаки говорят только “metadata лежит в необычной секции”, confidence надо ослаблять. Если placement сочетается с integrity damage - например parse failures, out-of-bounds, overlaps, - сигнал усиливается, потому что разные семьи evidence указывают в одну сторону.

> Хороший static не говорит: “секция называется странно, значит всё плохо”. Хороший static говорит: “unwind metadata расположена в секции с такими-то правами; это отклонение от обычной compiler-generated модели; проверь, связано ли это с упаковкой, JIT, ручной генерацией или повреждением”.

8.1.13. Reuse: когда много функций делят один `UNWIND_INFO`

Reuse - повторное использование. В `.pdata` это ситуация, когда много разных `RUNTIME_FUNCTION` записей указывают на один и тот же `UnwindInfoAddress`. Иногда это нормально. Если множество маленьких функций имеют одинаковый простой пролог - например только `sub rsp, 0x28` и больше ничего особенного, - теоретически toolchain может переиспользовать одинаковую unwind description. Chained info тоже может уменьшать дублирование. Поэтому сам факт reuse не является автоматическим нарушением.

Но heavy reuse - массовое повторное использование - становится интересным, когда оно слишком сильное или сочетается с другими признаками. Представим, что сотни разных диапазонов кода указывают на один `UNWIND_INFO`, а этот `UNWIND_INFO` имеет handler, и почти все handler entries ведут к одному target. Такая картина может быть похожа на centralized dispatcher: множество функций формально описаны разными диапазонами, но исключительная логика сводится к одному центральному обработчику. Это может быть легитимный runtime, а может быть obfuscation/control-flow layer.

Важно разжевать слово `dominant`. Dominant handler - это handler, который “доминирует” статистически: слишком большая доля handler-bearing функций ведёт к одному и тому же handler RVA. Доминирование не означает, что handler злой. Оно означает, что архитектура обработки исключений централизована. В обычной программе тоже может быть общий language-specific handler вроде `__C_specific_handler`, поэтому здесь надо смотреть глубже: речь о handler RVA на уровне metadata? Это стандартный

CRT/language handler? Или это нестандартная функция в writable/executable области? Есть ли разные `HandlerData`? Есть ли исключения, которые реально попадают в этот handler?

Вот здесь особенно легко перепутать уровни. `__C_specific_handler` действительно может встречаться много раз как language-specific handler. Это не обязательно suspicious. Подозрительнее становится не сам факт общего handler-а, а комбинация: необычная секция, writable metadata, parse anomalies, массовая reuse-pattern, handler target вне ожидаемого runtime-кода, dynamic handler hits при deliberate exceptions. Поэтому finding `handler_dominance` должен быть осторожным и контекстным, а не истеричным.

dynamic в reuse/dominance-сценариях помогает ответить на вопрос “это мёртвая статистика или живая логика?”. Если общий handler никогда не hit-ится в динамике, сигнал остаётся статическим и требует ручной проверки. Если target под Debug API показывает first-chance exceptions, handler breakpoints, RIP trace после handler hit, и этот dominant handler реально управляет продолжением, тогда static dominance получает runtime подтверждение. Именно это и есть смысл dynamic validation: не заменить static , а проверить, какие статические пути реально используются.

8.1.14. Handler dominance и ошибка “один handler = сразу плохо”

Надо отдельно остановиться на опасной ошибке: увидеть один общий handler и сразу назвать это obfuscation. Windows x64 exception handling устроен так, что `UNWIND_INFO` хранит address of exception handler, но этот handler часто является language-specific dispatcher-ом. Language-specific handler - это обработчик, который понимает формат данных конкретного языка или runtime-а. Например, для C SEH конструкций MSVC использует `__C_specific_handler`, который получает `ExceptionRecord`, `EstablisherFrame`, `ContextRecord`, `DispatcherContext`, смотрит `HandlerData`, находит scope table и уже через неё решает, какой filter вызвать и какой handler block выполнить.

Если десятки функций указывают на `__C_specific_handler`, это может быть совершенно нормальным. Настоящая пользовательская логика не в адресе `__C_specific_handler` как таковом, а в language-specific data после `UNWIND_INFO`: scope table, protected ranges, filter expressions, handler targets. Поэтому static должен уметь разделять “общий системно-языковой диспетчер” и “нестандартный центральный dispatcher, к которому сведены необычные исключительные маршруты”.

Слово `dispatcher` можно перевести как “диспетчер”: сущность, которая не обязательно сама делает всю работу, а направляет управление дальше. `__C_specific_handler` - dispatcher в этом смысле: он получает управление от OS exception machinery и решает, какие language-level конструкции проверять. Обфускационный dispatcher тоже может направлять управление, но уже с

другой целью: скрыть обычный control flow за исключениями, разорвать статический CFG, заставить реверсера следить за handler-ами вместо прямых переходов.

Поэтому вопрос не “есть ли общий handler?”, а “какой это handler, где он расположен, какие данные он читает, какие exception codes обрабатывает, возвращает ли `CONTINUE_SEARCH` или `CONTINUE_EXECUTION`, меняет ли `ContextRecord`, есть ли deliberate exception sites, подтверждается ли это динамически?”. Только такая цепочка делает анализ зрелым.

8.1.15. `.pdata` как карта, которую можно атаковать

Теперь можно сказать более глубоко: `.pdata` - это не просто справочная таблица. Это часть runtime contract-а между кодом, компилятором, загрузчиком, ОС, отладчиком и crash-reporting инфраструктурой. Если этот контракт честный, Windows может корректно раскрутить стек, вызвать нужные handlers, построить stack trace, выполнить cleanup. Если контракт странный, он может мешать анализу, ломать stack walking, скрывать реальные границы функций, заставлять отладчик показывать неполный call stack или направлять exception dispatch по неочевидному пути.

Обфускатор может использовать `.pdata` как поверхность воздействия. Surface - “поверхность”, место, через которое можно влиять на анализ. Например, можно сделать статическую таблицу неинформативной, а реальную function table зарегистрировать динамически. Можно массово переиспользовать один `UNWIND_INFO`, чтобы скрыть различия между функциями. Можно поместить metadata в необычную секцию. Можно завязать control flow на exceptions так, что прямой CFG кажется оборванным, а реальный переход происходит через handler.

Но книга должна учить не паранойе, а методике. Если мы видим странную `.pdata`, мы не “угадываем упаковщик”. Мы задаём цепочку вопросов. Сначала: корректна ли PE-карта? Потом: корректен ли `Exception Directory`? Потом: читаются ли записи `RUNTIME_FUNCTION`? Потом: валидны ли диапазоны? Потом: ведут ли `UnwindInfoAddress` к структурам? Потом: есть ли handler flags? Потом: где расположены metadata и handler targets? Потом: есть ли reuse/dominance? Потом: есть ли динамическое подтверждение?

```
Методический путь:
    PE headers
      -> sections and RVA mapping
      -> Exception Directory
      -> RUNTIME_FUNCTION ranges
      -> range validity / sorting / overlaps / gaps
      -> UnwindInfoAddress targets
      -> UNWIND_INFO parse
      -> flags / handlers / chained info
      -> placement / reuse / dominance
      -> static       findings
      -> dynamic      runtime validation
```

8.1.16. Dynamic function tables: когда нормальная `.pdata` появляется не в файле

До сих пор мы в основном говорили о `.pdata`, лежащей в PE-файле. Но Windows x64 поддерживает dynamic function tables - динамические таблицы функций. Это механизм для кода, который создаётся или меняется во время выполнения. Классический пример - JIT compiler. JIT, just-in-time compiler, генерирует машинный код прямо в памяти процесса. Такого кода не было в исходном PE-файле, поэтому статическая `.pdata` не могла заранее описать его диапазоны. Но если этот код nonleaf и может участвовать в exceptions/stack walking, ОС всё равно нужны unwind-данные. Поэтому runtime может зарегистрировать function table через `RtlAddFunctionTable` или callback-механизм через `RtlInstallFunctionTableCallback`.

Это важнейшее место для понимания static /dynamic . static читает файл. Если функция появляется только в памяти после запуска, статический файл не обязан содержать её `RUNTIME_FUNCTION`. Поэтому статический “не вижу `.pdata` для этого runtime-кода” не всегда ошибка. dynamic , напротив, работает на живом процессе под Windows Debug API и может увидеть вызовы `RtlAddFunctionTable` / `RtlInstallFunctionTableCallback`, аргументы этих вызовов, диапазоны, которые регистрируются, и дальнейшие exception events.

Но dynamic function tables могут быть и нормой, и трюком. Норма - JIT, runtime code generation, некоторые instrumentation frameworks. Трюк - manual mapping, packers, dynamic s, loaders, которые держат важную карту функций вне файла, регистрируют её поздно, меняют её, скрывают реальную структуру до запуска. Поэтому в проектной логике dynamic table activity - не “плохо”, а “важно”. Она объясняет, что статическая карта неполна, и одновременно показывает, где нужно искать живую карту.

Здесь снова нужен хороший язык findings. Плохо: “RtlAddFunctionTable detected = suspicious”. Лучше: “Target registered runtime function table; static `.pdata` may not fully describe runtime code; correlate registered ranges with executed RIP addresses and handler hits”. По-русски: “процесс зарегистрировал динамическую таблицу функций; статическая карта файла может быть неполной; сопоставь зарегистрированные диапазоны с реальным выполнением и обработчиками”.

8.1.17. Как static должен читать странную `.pdata`

Теперь опишем поведение static -а как спокойный алгоритм. static не исполняет файл. Это принципиально: static mode не пересекает execution boundary. Он работает с bytes, headers, sections, RVA mapping и structures. Поэтому его сила - безопасность, воспроизводимость и полнота статического просмотра. Его слабость - он не знает, какие ветки реально выполнятся, какие dynamic tables будут зарегистрированы, какие handlers hit-нутся.

Первый шаг static -a - убедиться, что PE вообще можно читать. Здесь проверяются заголовки, размеры, секции, raw/virtual ranges, alignment. Это скучно, но без этого дальнейшие выводы бессмысленны. Если section table уже противоречит файлу, чтение `.pdata` может быть не анализом, а случайным блужданием по байтам.

Второй шаг - найти `Exception Directory` и превратить его в массив `RUNTIME_FUNCTION`. Размер директории должен позволять читать целое число записей. Каждая запись читается как три 32-битных image-relative address. Здесь static уже фиксирует базовые проблемы: пустая таблица при наличии executable code, выходы за границы, некратный размер, невозможность перевести RVA в file offset.

Третий шаг - анализ диапазонов: `BeginAddress`, `EndAddress`, sorting, overlaps, duplicate begins, coverage. Это слой геометрии карты. Он ещё не говорит, как устроен стек, но говорит, можно ли вообще по `RIP` выбрать однозначный contract. Если карта диапазонов противоречива, все последующие уровни становятся менее надёжными.

Четвёртый шаг - переход к `.xdata` через `UnwindInfoAddress`. Здесь static разбирает `UNWIND_INFO`: version, flags, prolog size, count of codes, frame register, unwind codes, padding, handler tail, chained info. На этом шаге возникают parse failures, anomalies, ABI warnings. Важно разделять hard parse failure и strict ABI warning: первое может ломать базовое чтение, второе может быть формальным отклонением, которое встречается у совместимых toolchain-ов и не должно автоматически повышать suspicion score.

Пятый шаг - placement/reuse/dominance. Где лежит metadata? Сколько функций делит один `UNWIND_INFO`? Сколько handler-bearing entries ведёт к одному handler RVA? Есть ли suspicious combinations: writable metadata плюс writable handler, executable unwind section плюс non-executable handler target, reuse плюс handler dominance, placement плюс parse failures? Именно комбинации часто важнее одиночных признаков.

Шестой шаг - формулирование findings. Finding должен быть не обрывком “rf_overlaps: 5”, а понятной связкой: код finding-а, family, severity, representative RVA, affected count, почему это важно, что проверить дальше. Это важно и для человека, и для будущей проектной главы, где мы подробно разберём schema/reporting layer.

8.1.18. Как dynamic проверяет статическую странность

dynamic , как ты уточнил, - это динамическая валидация. Его задача не “защитить файл” и не “починить `.pdata`”, а показать живой маршрут: куда реально уходит функция, как реально проходит exception handling, какой handler срабатывает, куда уходит `RIP` после handler-а, какие dynamic function table API вызываются.

Если static говорит: “у этой функции handler metadata выглядит подозрительно”, dynamic может поставить breakpoint на handler target или на API регистрации handler-ов. Если static говорит: “возможна централизованная обработка исключений”, dynamic может собирать first-chance exceptions, handler hits и RIP trace. Если static говорит: “статическая `.pdata` неполная, но есть признаки runtime tables”, dynamic может ловить `RtlAddFunctionTable` и `RtlInstallFunctionTableCallback`, сохранять аргументы, сопоставлять registered ranges с исполняемыми адресами.

Очень важно: dynamic не отменяет необходимость теории. Напротив, без теории он будет просто логгером событий. Чтобы понять, что значит handler hit, нужно знать путь Windows exception dispatch. Чтобы понять, что значит RIP после hit, нужно знать `CONTEXT`. Чтобы понять, почему RSP изменился, нужно знать unwind. Чтобы понять, почему одна функция “ушла” не через обычный `ret`, а через exception path, нужно знать разницу между ordinary control flow и exception edge. Поэтому книга идёт медленно: она строит язык, на котором потом можно осмысленно читать динамический отчёт.

В идеальном случае static и dynamic сходятся. static статически находит handler-bearing function, странную placement-модель и reuse pattern. dynamic показывает, что во время выполнения происходит deliberate exception, handler hit, а после handler-а RIP попадает не туда, куда вёл обычный fall-through, а в continuation address, выбранный через обработку исключения. Это уже не просто “в файле лежит странная таблица”. Это correlated evidence: статическая карта и динамическая траектория указывают на одну и ту же механику.

8.1.19. Пример 1: маленький gap между функциями

Представим простой случай. static видит две записи `.pdata`: `[0x1000, 0x1080)` и `[0x1090, 0x1200)`. Между ними gap 16 байт: `[0x1080, 0x1090)`. На первый взгляд это непокрытая область в `.text`. Но при просмотре байтов там лежит последовательность `0xCC`, то есть `int 3`, часто используемая как padding/debug fill. Переходов в середину gap нет. Fall-through из первой функции отсутствует: первая функция заканчивается `ret`. В динамике RIP туда не попадает.

Какой вывод? Это не сильный сигнал. static может вообще не повышать suspicion score или оставить слабое замечание только в подробном режиме. Человеческое объяснение: “между двумя runtime-function ranges есть padding-like gap; no evidence of executable logic”. dynamic ничего подозрительного не подтверждает. Это пример того, как глубокая модель защищает нас от ложной тревоги.

8.1.20. Пример 2: overlap с разными `UNWIND_INFO`

Теперь другой случай. Две записи перекрываются на значительном участке, и каждая ведёт к разному UNWIND_INFO. В первой metadata описывает UWOP_ALLOC_SMALL, во второй - сохранение RBX и RBP, а также UNW_FLAG_EHANDLER. Один и тот же RIP может получить разные правила раскрутки. Это уже не косметическая странность. Это конфликт contract-ов.

static должен зафиксировать rf_overlaps в family table_integrity, дать representative RVA, показать обе записи и объяснить, что overlap может привести к неоднозначному выбору function table entry. Если рядом есть parse failures или suspicious placement, severity растёт. dynamic может попытаться подтвердить, исполняется ли overlap-region, возникают ли там exceptions, hit-ится ли handler из одной из записей. Если hit подтверждён, finding становится не только статическим, но и динамически значимым.

8.1.21. Пример 3: heavy reuse без динамического подтверждения

В большом файле 500 функций, и 180 из них указывают на один и тот же UnwindInfoAddress. Это больше похоже на reuse-pattern. Но при этом ranges корректны, metadata лежит в read-only .xdata, handler flags почти отсутствуют, parse failures нет, секции нормальные. Это интересный, но не обязательно сильный сигнал. Возможно, toolchain эффективно переиспользует короткие одинаковые unwind descriptions.

Правильный отчёт не должен звучать как обвинение. Он может сказать: “high UNWIND_INFO reuse ratio; no corroborating integrity or placement signals; manual review recommended if sample uses dynamic -like layout”. Если dynamic не видит exceptions, handler hits или dynamic function table activity, dynamic evidence level остаётся none/weak. Это пример, где статистика без контекста не должна превращаться в громкий verdict.

8.1.22. Пример 4: writable metadata и handler hit

Теперь представим более сильную картину. static видит, что значительная часть UNWIND_INFO лежит в writable section. Несколько handler targets тоже находятся в writable/executable области. Есть handler dominance. dynamic запускает target в изолированной среде, видит first-chance exception, breakpoint на handler target срабатывает, а затем RIP trace показывает переход в область, которую обычный статический CFG не связывал с местом исключения. Это уже не одиночная странность. Это соединение static placement, handler evidence и runtime trajectory.

Даже здесь корректный язык остаётся важным. Мы не пишем “доказано malware”. Мы пишем: “static metadata placement and handler dominance are dynamically exercised; exception handling participates in control transfer; manual reverse engineering should inspect handler logic and context modifications”. То есть: “странная metadata реально используется; исключения участвуют в передаче управления; теперь человеку надо смотреть handler и изменения

ContextRecord”.

8.1.23. Почему severity, score и confidence - разные вещи

В инженерном отчёте полезно разделять три понятия: severity, score и confidence. Severity - тяжесть конкретного finding-а. Например, overlap диапазонов может иметь high severity, потому что нарушает базовую карту. Score - суммарный triage-балл, который помогает расставить приоритеты. Confidence - уверенность в том, что verdict опирается на достаточно разнообразные и сильные свидетельства.

Почему это не одно и то же? Потому что один сильный finding может быть важным, но confidence всё равно ограничена, если он одинок и ничем не подтверждён. И наоборот, несколько средних findings из разных семейств могут дать более уверенную картину, потому что они сходятся: table integrity, metadata placement, reuse, dynamic hits. Поэтому проектная модель с evidence families лучше простого списка предупреждений.

Например, unwind_in_exec_section может быть high severity, но если это единственный сигнал в tiny sample, confidence надо ослабить. rf_overlaps плюс unwind_parse_fail плюс handler_nonexec_section плюс dynamic handler hit - уже другая история. Там confidence растёт, потому что мы видим не одну странность, а цепочку взаимно усиливающих наблюдений.

8.1.24. Чего не должен делать static

Чтобы книга была честной, надо описать не только что инструмент должен делать, но и чего он делать не должен. static не должен притворяться, что статический анализ видит runtime-поведение. Он не должен писать “этот handler сработает”, если он только увидел handler RVA в .xdata. Он должен писать “handler referenced by metadata”. Разница огромная. Первое - утверждение о будущем выполнении. Второе - факт о файле.

static не должен смешивать UnwindInfoAddress и handler RVA. Не должен считать любую необычную section name подозрительной на уровне high. Не должен считать каждую leaf-like uncovered область ошибкой. Не должен превращать strict ABI warning в malware-сигнал. Не должен скрывать parse failure reason: если UNWIND_INFO не разобран, человеку важно знать почему - неверная version, нехватка bytes, invalid opcode, chain depth, tail out-of-bounds.

И самое важное: static не должен быть оракулом. Его задача - дать структурированную карту и доказательства. Там, где статике не хватает информации, он должен честно передавать работу dynamic -у или человеку: “requires manual review”, “dynamic validation recommended”, “correlate with handler hits”, “inspect in Ghidra”.

8.1.25. Чего не должен делать dynamic

dynamic тоже не всесилен. Dynamic validation зависит от coverage выполнения. Если target не дошёл до нужной ветки, отсутствие handler hit не доказывает, что handler никогда не используется. Если sample требует особых аргументов, сети, файлов, времени, user interaction или environment checks, один dynamic run может увидеть только малую часть поведения. Поэтому dynamic должен честно хранить timeout state, target exit status, event completeness, dropped events, breakpoint completeness, trace limits.

dynamic не должен исполнять untrusted target без изоляции. Он пересекает execution boundary: файл уже не просто байты, он становится процессом. Поэтому dynamic mode должен быть отдельным режимом, а в книге это надо постоянно держать в голове: static analysis безопаснее и шире, dynamic validation глубже, но опаснее и неполнее.

dynamic также не должен подменять объяснение логом. Список событий “first-chance exception, handler hit, RIP trace” полезен только тогда, когда связан с картой static -а. Какой это handler? Он был найден в static `UNWIND_INFO` или зарегистрирован через VEH? Какой exception code? Изменился ли `ContextRecord`? Продолжилось ли выполнение через `CONTINUE_EXECUTION`? Попал ли `RIP` в область, которую static CFG не связывал? Вот эти связи и превращают динамику в evidence.

8.1.26. Ручной чек-лист для реверсера

Когда реверсер видит странную `.pdata`, ему полезно пройти не по списку “страшных слов”, а по медленной последовательности вопросов. Эта последовательность дисциплинирует мышление и помогает не потерять уровни адресации.

- Где находится `Exception Directory`? Его RVA и Size корректны? Размер кратен 12 байтам?
- Какой `ImageBase` у модуля? Не перепутаны ли VA, RVA и file offset?
- Какие секции покрывают `.pdata` и `.xdata`? Какие у них read/write/execute права?
- Все ли `RUNTIME_FUNCTION` имеют `BeginAddress < EndAddress`?
- Отсортирована ли таблица по `BeginAddress`? Есть ли duplicate begins?
- Есть ли overlaps? Если да, какие записи конфликтуют и разные ли у них `UNWIND_INFO`?
- Есть ли gaps в executable code? Похожи ли они на padding, thunks или настоящий код?
- Куда ведёт каждый `UnwindInfoAddress`? Парсится ли `UNWIND_INFO`?
- Есть ли `EHANDLER`, `UHANDLER`, `CHAININFO`? Где лежит handler target?

- Есть ли массовый reuse или handler dominance? Это стандартный language handler или нестандартный dispatcher?
- Что показывает динамика: exceptions, handler hits, dynamic function tables, RIP trace?

Такой чек-лист важен не ради бюрократии. Он повторяет архитектуру самой Windows-модели. PE даёт карту образа. Exception Directory даёт function table. `RUNTIME_FUNCTION` даёт диапазон. `.xdata` даёт unwind contract. Handler data даёт language-specific смысл. Debug events дают живое подтверждение. Если пропустить один слой, легко сделать красивый, но неверный вывод.

8.1.27. Как это должно выглядеть в хорошем finding-е

Хороший finding - это маленький аналитический абзац, а не просто флаг. Он должен отвечать на пять вопросов: что найдено, где найдено, на каком слое модели найдено, почему это важно, что делать дальше. Например, finding по overlap должен показывать affected count и representative RVA, но человеку полезно также увидеть две конфликтующие записи, их ranges и unwind RVA.

```
Плохой стиль:
    rf_overlaps: true

Лучше:
    Finding: rf_overlaps
    Family: table_integrity
    Severity: high
    Representative RVA: 0x00001080
    Meaning: two RUNTIME_FUNCTION ranges cover the same ControlPc region.
    Why it matters: exception dispatch and stack unwinding expect a unique function entry for a giv
    Next step: inspect both UNWIND_INFO records and run dynamic validation if the region is reachab
```

На русском в книге это можно сформулировать так: “две записи карты функций претендуют на один и тот же участок кода; это создаёт неоднозначность выбора unwind-контракта; надо посмотреть обе `.xdata` и проверить, попадает ли выполнение в этот участок”. Такой текст не просто сообщает ошибку, он обучает человека видеть связь между байтами и runtime-поведением.

8.1.28. Переход к следующему разделу

В этом разделе мы специально долго задержались на `.pdata`, потому что `.pdata` - первая карта, на которой начинают проявляться аномалии. Но сама по себе она ещё не объясняет, как именно функция строит кадр, какие регистры сохраняет и какой handler вызывается. Для этого нужно идти дальше, в `.xdata`. Если `.pdata` отвечает на вопрос “какой диапазон кода мы сейчас рассматриваем?”, то `.xdata` отвечает на вопрос “какой contract раскрутки действует для этого диапазона?”.

Следующий раздел будет естественным продолжением: когда `.xdata` выглядит странно. Там мы будем разбирать уже не геометрию диапазонов, а

внутреннюю инструкцию unwind-а: UNWIND_INFO, version, flags, count of codes, padding, handler tail, chained info, неверные opcodes, странные frame register settings, parse failures и несовпадения с реальным прологом. И там связь static /dynamic станет ещё важнее: static покажет, что metadata говорит; dynamic поможет проверить, как эта metadata участвует в живом исключении и куда реально уходит управление.

8.1.29. Медленный проход по одной записи .pdata: как мыслит реверсер

Чтобы материал не оставался абстрактным, пройдём одну запись так, как её должен проходить реверсер. Допустим, static нашёл в Exception Directory запись RUNTIME_FUNCTION: BeginAddress = 0x00015320, EndAddress = 0x000154A7, UnwindInfoAddress = 0x0002C010. Сначала мы не называем это “функцией” в привычном исходном смысле. Мы называем это “диапазоном кода, для которого образ предоставляет unwind contract”. Это аккуратнее. Возможно, дизассемблер уже назвал этот диапазон sub_140015320, возможно, имя пришло из symbols, возможно, это часть bigger function, возможно, это chained fragment. Но с точки зрения .pdata перед нами именно диапазон.

Первый вопрос: диапазон логически валиден? 0x15320 < 0x154A7, значит он не пустой и не перевёрнутый. Второй вопрос: попадает ли диапазон в executable section? Если да, это похоже на нормальный code range. Если нет, это уже сильный сигнал: RUNTIME_FUNCTION описывает неисполняемую область. Третий вопрос: какова длина диапазона? Здесь длина 0x187 байт. Это не доказывает ничего само по себе, но помогает контексту: перед нами не двухбайтовый stub и не гигантский мегабайтный участок.

Дальше мы переходим к третьему полю. UnwindInfoAddress = 0x2C010 - это RVA. Мы не открываем файл по смещению 0x2C010 напрямую. Мы спрашиваем PE section table: какая секция покрывает RVA 0x2C010? Допустим, это .xdata, read-only. Тогда placement выглядит спокойно. Затем static переводит RVA в file offset и читает первые байты UNWIND_INFO: version, flags, prolog size, count of codes, frame register. Если version допустимая, count помещается в структуру, padding корректен, tail читается, запись получает нормальный статус parse ok.

Теперь появляется связь с предыдущей главой. Если RIP во время исключения окажется внутри [0x15320, 0x154A7), Windows может выбрать эту запись, прочитать UNWIND_INFO и понять, как откатить кадр. Если flags не содержат EHANDLER, эта функция не добавляет language-specific exception handler на этом уровне. Тогда при поиске handler-а диспетчер, не найдя обработчик в этой функции, должен виртуально раскрутить её кадр и перейти к caller-у. Если flags содержат EHANDLER, путь другой: будет вызван language-specific handler, и только он решит, относится ли текущий ControlPc

к protected range и надо ли выполнять filter.

Видишь, даже одна запись `.pdata` уже тянет за собой всю систему: PE section mapping, RVA arithmetic, понятие executable section, структуру `UNWIND_INFO`, flags, search phase, caller context. Поэтому плохая подача “поле такое-то, поле такое-то” недостаточна. Реверсеру нужно видеть, зачем каждое поле участвует в общей машине Windows exception handling.

8.1.30. Медленный проход по подозрительной записи: где именно рождается сигнал

Теперь возьмём другой пример. `BeginAddress = 0x00040100`, `EndAddress = 0x0003F000`, `UnwindInfoAddress = 0x00090000`. Уже первое сравнение ломает картину: `BeginAddress` больше `EndAddress`. Это invalid range. Важно не перепрыгнуть сразу к выводу “обфускация”. Пока у нас есть только один факт: запись не задаёт нормальный half-open range. static должен сохранить этот факт, representative RVA и affected count. Если таких записей одна, это одно. Если их сотни, это другое.

Дальше мы смотрим `UnwindInfoAddress`. Допустим, `0x90000` выходит за `SizeOfImage`. Значит даже если бы диапазон был нормальным, цепочка к `.xdata` всё равно оборвалась бы. Теперь у нас уже два сигнала из одной семьи table/unwind integrity: bad range и out-of-bounds unwind RVA. Если рядом обнаруживается, что `Exception Directory` size не кратен 12, а секции имеют virtual overlaps, это уже не единичная помарка. Это системная проблема структуры PE.

Но хороший анализатор всё равно не должен писать “протектор найден”. Он пишет: “runtime-function table contains malformed entries; unwind metadata cannot be resolved for affected ranges; stack unwinding and exception dispatch may be unreliable; manual review and optional dynamic validation recommended”. Человеку такой текст полезен: он объясняет, где сломался путь и что именно не получится сделать надёжно.

dynamic здесь может быть полезен, но не всегда сможет подтвердить проблему. Если sample не исполняет повреждённые ranges, dynamic run может завершиться без событий. Это не отменяет static finding. Это означает: static evidence есть, runtime evidence для конкретного маршрута нет. Если же при запуске возникает exception в области, которую статическая таблица описывает плохо, и процесс падает на second chance или даёт странный stack trace, тогда статическое повреждение получает динамическое значение.

8.1.31. Почему “куда уходит функция” нельзя понять только по `ret`

Ты отдельно уточнил, что dynamic показывает, куда уходит функция и как проходит обработка исключения. Это важно встроить в теорию. В обычной модели функция “уходит” через `ret`: callee закончил работу, взял return address со стека, записал его в `RIP`, и управление вернулось к caller-у. Но

exception-модель добавляет не такой прямой путь. Если внутри функции возникает исключение, управление может сначала попасть в dispatcher, затем в VEH, затем в frame-based handler search, затем в language-specific handler, затем в filter, затем в handler block, или вообще продолжиться с изменённым `ContextRecord`.

Поэтому фраза “куда уходит функция” в динамической валидации должна пониматься шире, чем “какая инструкция после `call`”. Функция может уйти обычным `ret`. Может быть виртуально раскручена во время stack unwinding. Может быть пропущена как leaf case. Может передать управление handler-у. Может после handler-а продолжить выполнение с другим RIP, если обработчик вернул continuation semantics и изменил context. Может закончиться crash path, если handler не найден. dynamic нужен именно потому, что некоторые такие маршруты плохо видны в статическом CFG.

CFG - control-flow graph, граф потока управления. В обычном CFG узлы - basic blocks, а рёбра - переходы, calls, fall-through. Exception edge - особое ребро: оно не выглядит как обычный `jmp` в коде. Оно рождается из runtime exception machinery и metadata. Если дизассемблер строит CFG только по прямым переходам, он может не показать, что инструкция `mov [rax], rcx` при `rax = 0` ведёт к access violation, затем к handler-у, затем к continuation block. Именно поэтому static должен показывать статические handler references, а dynamic - реальный handler hit и RIP trace.

8.1.32. Как не потерять связь RIP, RSP, return address и `.pdata`

Перед разделами об аномалиях полезно снова склеить базовые термины. RIP - это instruction pointer, адрес инструкции, которую поток выполняет или собирается выполнять. RSP - stack pointer, вершина текущего стека. Return address - адрес, который `call` положил на стек, чтобы `ret` знал, куда вернуться. `.pdata` не хранит return address. `.pdata` помогает системе понять, как от текущего RIP и текущего RSP добраться до caller context, то есть до состояния вызывающей функции.

Если функция простая leaf и для неё нет `RUNTIME_FUNCTION`, система может предположить: return address лежит по `[RSP]`. Но если функция nonleaf выделила стек, сохранила регистры, изменила RSP, то такой простой вывод будет неправильным. Нужно знать, сколько байт добавить к RSP, где лежат сохранённые RBX/RBP/R12-R15, где находится return address после локальной аллокации. Это и описывает `UNWIND_INFO`, а `.pdata` помогает найти правильный `UNWIND_INFO` по текущему RIP.

Когда `.pdata` странная, страдает именно эта связь. Если текущий RIP не попадает ни в один правильный диапазон, Windows может ошибочно пойти по leaf-модели. Если RIP попадает в два диапазона, возникает неоднозначность. Если диапазон найден, но `UnwindInfoAddress` битый, нельзя корректно узнать, как двигать RSP. Если `UNWIND_INFO` найден, но не

соответствует реальному прологу, stack walking может показать красивую, но ложную цепочку вызовов. Поэтому аномалии `.pdata` - это не “табличные мелочи”, а возможная поломка связи между живым состоянием потока и историей вызовов.

8.1.33. Что именно должен увидеть читатель в Ghidra или x64dbg

Книжная теория должна приводить к практическому зрению. В Ghidra реверсер может увидеть функцию, её start address, body ranges, references, decompiler output. Но `.pdata` может дать другую карту: range может начинаться раньше или позже, может покрывать только часть того, что Ghidra назвала функцией, может иметь chained info, может указывать на handler. Поэтому хороший workflow - не верить одному представлению. Надо сравнивать function boundaries из дизассемблера с `RUNTIME_FUNCTION` ranges.

В x64dbg или WinDbg реверсер видит уже живое состояние: текущий `RIP`, RSP, exception code, call stack, first-chance/second-chance события. Если call stack выглядит оборванным, если после exception управление уходит не туда, куда ожидалось, если handler hit происходит в неожиданном месте, надо вернуться к `.pdata`/`.xdata` и спросить: что статическая карта обещала системе? Иногда “странный отладчик” на самом деле честно показывает последствия странной metadata.

dynamic в проекте автоматизирует часть такого наблюдения: он собирает exception events, registrations, handler hits, bounded RIP trace. Но человеческое понимание остаётся тем же. Событие в отчёте надо уметь прочитать: это first-chance exception? handler был VEH или frame-based? handler был найден static -ом в `UNWIND_INFO` или зарегистрирован API? RIP trace после hit показывает возврат, crash path или exception-based transfer?

8.1.34. Как этот раздел готовит следующую тему

Мы расширили 8.1 не ради объёма. Цель была медленно закрепить: `.pdata` - это карта диапазонов, а аномалии `.pdata` - это проблемы или странности именно карты. Но карта диапазонов ещё не описывает внутреннюю механику кадра. Она только говорит: “для этого участка кода смотри вот такую `.xdata`”. Поэтому следующий раздел должен перейти к `.xdata` так же медленно: что считается нормальной `UNWIND_INFO`, почему version/flags/count/padding важны, как возникают invalid tails, почему chained info может быть нормой или трюком, как parse failure отличается от ABI warning, как static должен это объяснять, и что dynamic может подтвердить динамически.

Если 8.1 отвечает на вопрос “можно ли по `RIP` выбрать правильную запись function table?”, то 8.2 будет отвечать на следующий вопрос: “если запись выбрана, можно ли доверять инструкции, которая объясняет, как разобрать кадр?”. Это разные уровни. Их нельзя смешивать. И именно такая поэтапность

делает книгу фундаментом, а не набором тезисов.

Техническая опора раздела

Раздел опирается на официальную модель Microsoft x64 exception handling: `RUNTIME_FUNCTION` записи являются image-relative, обычно находятся в `.pdata`, указывают на `UNWIND_INFO`, а dynamic code может предоставлять function tables через `RtlAddFunctionTable` или `RtlInstallFunctionTableCallback`. PE Format используется как базовая модель Data Directory, sections, RVA и mapping между файлом и образом. В проектной части раздел согласован с архитектурой EH/Unwind Analyzer: static - статический разбор PE/unwind metadata и findings; dynamic - динамическая валидация через runtime events, handler hits и RIP trace.

8.2. Когда .xdata выглядит странно: медленный разбор UNWIND_INFO, хвостов, handler data и цепочек

В предыдущем разделе мы долго разбирали .pdata. Это было нужно не ради самой таблицы, а ради более важной мысли: в Windows x64 функция существует для системы не только как набор инструкций в .text. Для механизма исключений и раскрутки стека функция существует как диапазон адресов, к которому привязан набор машинно-читаемых правил. .pdata отвечает на первый вопрос: какой диапазон кода сейчас выполняется и где лежит описание этого диапазона. Но сама .pdata почти ничего не говорит о том, как именно функция построила свой кадр стека, какие регистры сохранила, сколько байт отняла от RSP, есть ли у нее обработчик исключения, есть ли chained unwind information. Все это начинается уже в .xdata.

Поэтому .xdata в этой главе надо воспринимать не как соседнюю секцию, не как техническое приложение к .pdata и не как скучную бинарную структуру. .xdata - это место, где Windows получает инструкцию по восстановлению контекста. Контекст здесь означает состояние потока: где находится выполнение, какой RIP, какой RSP, какие значения в nonvolatile-регистрах, где можно найти адрес возврата, как перейти от текущего кадра к вызывающему кадру. Если .pdata можно назвать картой районов города, то .xdata - это не просто адресная книга, а инструкция, как выбраться из конкретного здания, не сломав лестницу и не перепутав этажи.

Именно поэтому странности в .xdata надо разбирать медленнее, чем странности в .pdata. Ошибка в .pdata часто означает, что мы не можем надежно найти запись для функции. Ошибка в .xdata означает более тонкую вещь: запись вроде бы найдена, диапазон функции вроде бы понятен, но правила восстановления состояния могут быть неверными, неполными, специально запутанными или помещенными туда, где нормальный компилятор обычно их не оставляет. Для реверсера это критично: дизассемблер может показать функцию, декомпилятор может нарисовать псевдокод, но stack walk, exception dispatch и обработчики зависят от тех байтов, которые лежат в .xdata.

В терминах нашего проекта это раздел про внутреннюю работу `static -a. static` читает файл как статическую карту: он берет RuntimeFunction из .pdata, переходит по UnwindInfoAddress, разбирает UNWIND_INFO и пытается понять, нормальна ли эта структура. Но это также раздел про будущую работу `dynamic -a. dynamic` не просто верит статике. Он запускает цель и смотрит, совпадает ли статическая карта с живым маршрутом выполнения: куда ушел RIP, какой handler реально получил управление, произошло ли исключение, как повел себя стек после обработки. Поэтому странная .xdata - это не окончательный приговор, а повод построить хорошую гипотезу для

динамической проверки.

8.2.1. Почему .xdata важнее, чем кажется после первого знакомства

На первом проходе новичок часто воспринимает .xdata слишком узко: “там лежит UNWIND_INFO”. Формально это правда, но такая формулировка почти ничего не объясняет. Надо спросить иначе: почему Windows вообще нужна отдельная структура UNWIND_INFO? Почему нельзя просто выполнить ret? Почему нельзя посмотреть на инструкции функции и догадаться, что делать со стеком? Почему компилятор обязан оставить системе отдельный машинно-читаемый договор?

Ответ начинается с того, что обычный ret работает только в нормальном пути выполнения. Нормальный путь - это когда функция дошла до своего эпилога, восстановила сохраненные регистры, вернула RSP туда, где он был после call, и затем ret забрал address возврата со стека. Но исключение часто возникает не в конце функции. Оно может возникнуть в середине тела, внутри вызова, после выделения локального стека, после сохранения RBX, но до восстановления RBP, или в момент, когда функция уже использует frame pointer. В этот момент обычного ret еще нет. До эпилога выполнение не дошло. Но системе все равно надо понять, кто вызвал эту функцию, где лежит return address и как перейти к caller context.

Caller context - это не абстрактное красивое выражение. Caller - вызывающая функция, то есть та функция, которая раньше выполнила call. Context - состояние процессора, которое должно соответствовать моменту, как будто текущая функция уже завершилась и управление вернулось назад. Когда RtlVirtualUnwind восстанавливает caller context, он не “продолжает программу” и не исполняет ret. Он вычисляет новое состояние: каким должен быть RIP вызывающего уровня, каким должен быть RSP вызывающего уровня, какие nonvolatile-регистры надо вернуть из стека. Чтобы сделать это надежно, ему нужна не догадка, а точное описание пролога функции.

Пролог, как мы уже разбирали, - это не просто “первые инструкции”. Prolog - это участок, где функция строит свой кадр. Она может выполнить push rbx, push rbp, sub rsp, 0x80, сохранить r12 в [rsp+...], установить rbp как frame register. Для человека это выглядит как несколько инструкций. Для Windows это набор эффектов: какой регистр сохранен, на каком смещении, сколько байт занял локальный стек, какой регистр стал базой кадра. UNWIND_INFO описывает именно эти эффекты. Поэтому странная .xdata опасна не потому, что “байты некрасивые”, а потому что она может исказить модель кадра.

> Главная мысль: .xdata отвечает не за красоту дизассемблера, а за способность Windows и инструментов восстановить состояние вызывающей функции без обычного ret.

Теперь можно аккуратно связать это с static -ом. static смотрит на .xdata и задает не один вопрос, а целую цепочку вопросов. Можно ли безопасно прочитать UNWIND_INFO по этому RVA? Совпадает ли версия структуры с ожидаемой? Не выходит ли массив UNWIND_CODE за границы секции или файла? Согласован ли CountOfCodes с реальным размером? Правильно ли расположен padding? Если установлен EHANDLER, можно ли прочитать handler RVA? Если установлен CHAININFO, выглядит ли хвост как RuntimeFunction? Если UNWIND_CODE утверждает, что сохранен nonvolatile-регистр, похоже ли это на пролог функции?

dynamic задает другой вопрос, но опирается на ту же базу. Он спрашивает: если эта статическая карта говорит, что здесь есть handler или такой-то unwind layout, проявляется ли это в живом выполнении? Попадаем ли мы в handler? Видим ли исключение? Меняется ли RIP после обработки? Сходится ли динамический маршрут с тем, что static прочитал из .xdata? Именно поэтому .xdata - центр связи между статикой и динамикой.

8.2.2. Что именно лежит в UNWIND_INFO: повторяем медленно, но уже глазами проверяющего

Перед тем как говорить о странностях, нужно еще раз пройти по самой структуре. Не как по таблице полей, а как по рассказу о том, что Windows должна узнать о функции. UNWIND_INFO начинается с маленького заголовка. В этом заголовке упакованы version и flags, затем идут size of prolog, count of unwind codes, frame register и frame offset. После этого идет массив UNWIND_CODE. После массива может быть выравнивающий padding. А после padding начинается хвост, смысл которого зависит от flags.

```
UNWIND_INFO, упрощенно:

  header:
    Version + Flags
    SizeOfProlog
    CountOfCodes
    FrameRegister + FrameOffset

  UnwindCode[CountOfCodes]

  optional padding to even slot count

  tail, depending on Flags:
    handler RVA + language-specific data
    or chained RUNTIME_FUNCTION
    or nothing
```

Version - это версия формата. Для обычной Windows x64 unwind-информации ожидается версия 1. Если static видит другую версию, это не просто “цифра не та”. Это означает, что остальные байты нельзя уверенно трактовать по привычным правилам. В нормальном инженерном отчете это должно звучать спокойно: unsupported unwind info version, parsing

confidence low. Не “точно вредонос”, а “мы не можем надежно применить известную схему”.

Flags - это поле, которое меняет смысл конца структуры. Слово flag переводится как флаг, признак, переключатель. В низкоуровневых структурах флаг обычно означает один бит или несколько битов, которые включают дополнительное поведение. В UNWIND_INFO важны три смысловые ситуации: нет handler-а, есть exception handler или termination handler, либо структура является chained unwind information. Если static пропустит flags, он неправильно поймет хвост. Он может принять handler RVA за chained RuntimeFunction, или наоборот. Это типичная ошибка самодельных парсеров.

SizeOfProlog - размер пролога в байтах. Это не размер функции и не количество unwind-инструкций. Это граница области, где функция строит кадр. Для unwinder-а это нужно, потому что поведение внутри пролога отличается от поведения в теле функции. Если исключение случилось в прологе, не все действия пролога могли быть выполнены. Например, UNWIND_INFO в целом знает, что функция должна сохранить RBX и выделить стек, но если ControlPc указывает на байт до push rbx, то считать RBX уже сохраненным нельзя. Поэтому SizeOfProlog участвует в точном определении того, какие действия уже произошли.

CountOfCodes - одно из самых коварных полей. Оно говорит не “сколько инструкций в прологе”, а сколько двухбайтовых slots занимает массив UNWIND_CODE. Slot - маленькая ячейка фиксированного размера. Некоторые unwind operations занимают один slot, некоторые используют дополнительные slots для операнда. Поэтому CountOfCodes нельзя напрямую сравнивать с числом push/sub/mov в прологе. Если программа проверки делает такое сравнение в лоб, она будет давать ложные findings.

FrameRegister и FrameOffset описывают frame pointer модель. Frame register - это регистр, который используется как стабильная база для доступа к локальным данным. На x64 это часто RBP, но не обязательно всегда именно он. Frame offset - дополнительное смещение, которое помогает восстановить связь между frame register и реальным RSP. Важно понимать: frame register не нужен каждой функции. Многие функции живут только через RSP. Поэтому отсутствие FrameRegister само по себе нормально. Подозрительным оно становится только тогда, когда реальный код явно строит frame pointer, а metadata этого не отражает, или наоборот metadata заявляет frame register, которого в прологе не видно.

8.2.3. Странность номер один: структура читается, но ее границы не сходятся

Самый первый класс проблем в .xdata связан не с семантикой, а с границами. Граница - это вопрос “где структура начинается и где заканчивается”. В PE-анализе это всегда важнее, чем кажется. Если

UnwindInfoAddress указывает внутрь файла, это еще не значит, что весь UNWIND_INFO можно безопасно прочитать. Заголовок может помещаться, но массив UNWIND_CODE уже выходить за секцию. Массив может помещаться, но хвост handler data может выходить за raw data. CountOfCodes может быть таким большим, что чтение приведет за пределы файла.

Для обычного человека это выглядит как скучная проверка размера. Для static -а это одна из базовых гарантий безопасности. Анализатор читает недоверенный бинарь. Недоверенный означает: файл может быть поврежден случайно, сгенерирован нестандартным тулчейном, упакован, специально испорчен или создан так, чтобы сломать анализатор. Поэтому любой переход по RVA должен быть проверен. RVA надо перевести в file offset. Потом надо проверить, что нужное количество байтов находится внутри секции и внутри файла. Потом надо проверить, что структура не пересекает область, которую нельзя читать как данные.

Здесь полезно разделить три уровня ошибки. Первый уровень - physical out of bounds: нужные байты физически выходят за файл. Второй уровень - section out of bounds: байты вроде есть в файле, но RVA не соответствует нормальной mapped секции. Третий уровень - semantic out of bounds: байты можно прочитать, но по смыслу структура упирается туда, где ожидается другая сущность, например в начало следующей структуры или в исполняемый код. Для хорошего отчета эти уровни нельзя смешивать.

Почему это важно для реверсера? Потому что разные уровни дают разные гипотезы. Физический выход за файл часто похож на поврежденный PE или на намеренно битую metadata. Section mismatch может быть следствием packer-а, manual mapping модели или криво созданного образа. Semantic mismatch может быть трюком против инструментов: структура формально читается, но ее хвост направляет парсер в бессмысленную область. static должен не кричать, а объяснять: “UNWIND_INFO header readable, but code array extends beyond mapped section; unwind semantics unreliable”.

dynamic в такой ситуации становится особенно полезным. Если статическая .xdata не читается, но программа успешно выполняется, это не отменяет finding. Это означает, что либо этот диапазон не был достигнут, либо Windows использует другую function table, либо код регистрирует dynamic function table, либо ошибка проявится только при исключении/stack walk. Динамическая валидация должна проверять не просто “запустилось или нет”, а был ли затронут конкретный проблемный диапазон и что произошло при исключении рядом с ним.

8.2.4. Padding: маленькое выравнивание, из-за которого ломается большой смысл

Один из терминов, который легко пропустить, - padding. Padding переводится как заполнение или выравнивающая прокладка. В бинарных

структурах padding нужен, когда следующую часть структуры надо расположить на удобной границе. В UNWIND_INFO массив UNWIND_CODE состоит из двухбайтовых slots, но хвост после массива должен начинаться после четного количества slots. Поэтому если CountOfCodes нечетный, после массива есть дополнительный пустой slot.

Новичок часто думает: “пустой slot, значит можно не обращать внимания”. Но для парсера это критично. Если static забудет про padding, он начнет читать handler RVA на два байта раньше. Тогда четыре байта handler RVA будут собраны из половины последнего UNWIND_CODE и половины настоящего handler RVA. Получится ложный адрес. Этот ложный адрес может указывать в .text, в .rdata, за файл, куда угодно. Потом heuristic engine может решить, что handler находится в странной секции, хотя настоящая ошибка была раньше: неправильно посчитан offset хвоста.

Поэтому хороший анализатор должен отделять “аномалию бинаря” от “ошибки собственного парсинга”. Это важная инженерная дисциплина. Когда мы пишем книгу под реальный проект, это надо проговаривать: static не должен строить выводы на плохо рассчитанной структуре. Сначала он доказывает, что правильно нашел начало хвоста. Только потом делает finding о handler RVA, chained info или language-specific data.

```
Правильная мысль для static -а:

  tail_index = (CountOfCodes + 1) & ~1

То есть хвост начинается не просто после CountOfCodes slots,
а после CountOfCodes, округленного вверх до четного числа slots.
```

Это маленькая формула, но за ней стоит большой смысл. Она напоминает, что бинарный формат - это не текстовый JSON, где поля явно названы. В PE и unwind metadata значение байта зависит от позиции. Сдвинулся на два байта - и вся последующая интерпретация стала другой. Поэтому при анализе .xdata нельзя быть приблизительным. Нельзя говорить “примерно здесь handler”. Нужно уметь показать цепочку: вот RVA UNWIND_INFO, вот header, вот CountOfCodes, вот массив slots, вот padding, вот с этого offset начинается хвост.

Для dynamic -а padding напрямую может быть невидим. Динамический слой не читает padding как отдельную сущность, он наблюдает поведение. Но если static из-за padding ошибся с handler RVA, dynamic может поставить breakpoint не туда. Тогда динамическая валидация будет молчать не потому, что handler не сработал, а потому что статический слой дал неправильную точку. Это еще один пример, почему хорошая динамика начинается с аккуратной статики.

8.2.5. Flags как развилка смысла: EHANDLER, UHANDLER и CHAININFO

Теперь перейдем к flags. Это место, где .xdata перестает быть просто описанием пролога и начинает связываться с обработкой исключений. Если флагов handler-а нет, хвост после UNWIND_CODE отсутствует. Такая структура говорит: “я умею раскрутить кадр, но у этой функции нет собственного language-specific обработчика”. Это нормальный случай для огромного числа функций.

Если установлен UNW_FLAG_EHANDLER, структура говорит: у функции есть exception handler. Слово exception handler здесь надо понимать аккуратно. Это не обязательно пользовательский блок __except в прямом виде. В MSVC для SEH на x64 handler RVA часто указывает на runtime-функцию вроде __C_specific_handler. Именно она получает управление от системного dispatcher-а и уже по language-specific data разбирает protected ranges, filters и handler blocks. Поэтому handler RVA - это вход в язык/рантайм обработки, а не всегда “адрес кода, который написал программист”.

Если установлен UNW_FLAG_UHANDLER, речь идет о termination handler. Termination handler - обработчик завершения/очистки, связанный с unwind phase. Его смысл не в том, чтобы решить “обрабатывать исключение или искать дальше”, а в том, чтобы выполнить cleanup при раскрутке. В C++ мире это связано с деструкторами и cleanup-логикой, в SEH - с конструкциями вроде __finally. Для реверсера важно: UHANDLER не означает тот же сценарий, что EHANDLER. Один связан с поиском решения, другой - с обязательной уборкой при уходе из кадра.

Если установлен UNW_FLAG_CHAININFO, смысл хвоста меняется еще сильнее. Там больше не handler RVA. Там лежит chained RuntimeFunction. Chained переводится как связанный, сцепленный, построенный цепочкой. Такая запись говорит: текущий UNWIND_INFO является вторичным описанием, которое ссылается на другое, основное описание. Это может использоваться для shrink-wrapping, разделенных участков кода и оптимизаций, когда одна функция имеет не совсем один простой пролог на весь диапазон.

Критическая ошибка новичка - читать хвост одинаково всегда. Но хвост UNWIND_INFO - это не поле фиксированного смысла. Его смысл задают flags. Если flags говорят EHANDLER, читаем handler RVA. Если flags говорят CHAININFO, читаем три DWORD как RuntimeFunction. Если flags говорят nothing, хвоста нет. Если одновременно выставлены несовместимые комбинации, это уже отдельная аномалия. Хороший static должен первым делом классифицировать хвост, а не просто извлекать “следующие 4 байта”.

> В .xdata одно и то же место в файле может означать разные вещи. Смысл определяется не адресом сам по себе, а flags, CountOfCodes и правилами выравнивания.

Для dynamic -а эта развилка тоже имеет значение. Если static нашел EHANDLER, dynamic может пытаться поставить breakpoint на handler RVA или на runtime handler, а затем наблюдать handler hit. Если найден CHAININFO,

динамическая проверка должна ожидать не handler hit, а корректную unwind-связь с primary unwind info. Если это перепутать, динамический отчет будет задавать неправильные вопросы.

8.2.6. Неподдержанная версия и странные флаги: когда неизвестность честнее ложной уверенности

В технической книге важно не только объяснять правильный формат, но и учить честному незнанию. Когда static видит version, которую он не поддерживает, правильное поведение - не пытаться героически дорисовать смысл. Если формат неизвестен, дальнейшая интерпретация становится предположением. Можно сохранить сырые байты, можно отметить RVA, можно показать, какой RuntimeFunction на это ссылается, но нельзя уверенно говорить, что handler находится там-то, а prolog size означает именно то-то.

То же относится к странным combinations of flags. Combination - сочетание. В UNWIND_INFO не все сочетания имеют нормальный смысл. Например, CHAININFO меняет формат хвоста на chained RuntimeFunction, а EHANDLER/UHANDLER ожидают handler RVA и language-specific data. Если парсер видит такую ситуацию, он должен не “выбрать то, что удобнее”, а зафиксировать conflict. Conflict означает, что структура предъявляет несовместимые требования к одному и тому же месту байтов.

Почему это важно для книги? Потому что реверсер должен отличать “я нашел вредоносную технику” от “мой инструмент потерял уверенность”. В хорошем анализе есть confidence. Confidence - уверенность вывода. Если структура неизвестной версии, confidence падает. Если хвост конфликтует по flags, confidence падает. Если байты читаются, но семантически не согласованы, confidence падает. Это не слабость инструмента, а честная модель доказательств.

В отчете static -а такая ситуация должна быть выражена языком evidence, а не эмоцией. Например: “UNWIND_INFO version is unsupported; decoder stopped before interpreting unwind codes; manual review required”. Это намного лучше, чем ложный список десяти подозрительных handler-ов, которые появились из-за неправильной интерпретации формата.

dynamic здесь может помочь, но не обязан все спасать. Динамическая валидация может показать, что при реальном исключении Windows проходит этот диапазон нормально или, наоборот, падает при stack walk. Но если путь выполнения не затронул проблемную функцию, dynamic честно должен сказать: “static anomaly not exercised”. Это тоже ценный результат. Он не закрывает finding, но отделяет подтвержденное runtime-поведение от непроверенной статической гипотезы.

8.2.7. CountOfCodes и ложная простота массива UNWIND_CODE

UNWIND_CODE выглядит маленьким и поэтому обманчивым. Каждая запись всего два байта, но смысл этих двух байтов зависит от UnwindOp и OpInfo. UnwindOp - операция раскрутки. OpInfo - дополнительная маленькая информация, встроенная в тот же slot. Но некоторые операции требуют дополнительных slots. Например, большой stack allocation не помещается в один маленький OpInfo, поэтому рядом лежит дополнительное значение. Сохранение регистра по большому offset тоже может требовать дополнительного slot.

Отсюда рождается важная проверка: CountOfCodes должен покрывать не только основные операции, но и их дополнительные slots. Если операция заявляет, что ей нужен еще один или два slots, а массив заканчивается, структура malformed. Malformed переводится как неправильно сформированная. Это не обязательно означает злой умысел. Это означает, что структура нарушает правила формата, и unwinder или анализатор не может надежно восстановить смысл.

Надо также помнить, что порядок UNWIND_CODE не обязан совпадать с тем, как человек читает пролог сверху вниз. В документации x64 unwind operations хранятся с учетом offset-а конца инструкции в прологе и используются unwinder-ом для обратного применения эффектов. Если читатель ожидает увидеть “push rbx, потом sub rsp” в том же простом порядке, он может ошибиться. В книге мы уже говорили: UNWIND_CODE описывает эффекты пролога, а не красивую распечатку ассемблера.

Странность в CountOfCodes может быть трех типов. Первый тип - техническая невозможность: массив обрывается. Второй тип - семантическое несоответствие: массив читается, но операции выглядят бессмысленно относительно функции. Третий тип - подозрительная сложность: операции формально допустимы, но построены так, что затрудняют ручное чтение, повторяют сохранения, ведут к странным offsets или не похожи на привычный output компилятора. static должен разделять эти типы, потому что severity у них разная.

Severity - серьезность. Если массив физически обрывается, это высокая техническая проблема: unwind info unreliable. Если операции просто необычны, это может быть compiler-specific pattern, hand-written assembly, obfuscation или packer artifact. Если операции противоречат прологу, это уже сильнее: static metadata/code mismatch. Хороший отчет должен говорить не только “странно”, но и почему именно странно.

dynamic в такой ситуации особенно ценен при mismatch. Если static говорит, что UNWIND_CODE не похож на пролог, dynamic может проверить живой сценарий: вызвать исключение в этом диапазоне, посмотреть, сможет ли система пройти unwind, какой stack trace получается, уходит ли управление к handler-у, не повреждается ли caller context. Это и есть динамическая валидация “куда уходит функция” в широком смысле: не только через call/ret,

но и через exception route.

8.2.8. SizeOfProlog: почему один байт границы может изменить весь unwind

SizeOfProlog кажется простым числом: сколько байт занимает пролог. Но для unwinder-а это не справка, а граница режима. Когда ControlPc находится в теле функции, считается, что пролог уже завершился: все описанные действия пролога применены. Когда ControlPc находится внутри пролога, unwinder должен учитывать только те действия, которые уже успели выполниться до этого адреса. Иначе он восстановит состояние, которого еще не было.

Представим функцию. В начале она делает push rbx, затем sub rsp, 0x40. Если исключение возникло после push rbx, но до sub rsp, то RBX уже сохранен, а стек еще не уменьшен на 0x40. Если unwinder ошибочно решит, что весь пролог выполнен, он прибавит к RSP лишние 0x40 и возьмет return address не оттуда. Один неверный вывод - и caller context станет неправильным. А неправильный caller context означает сломанный stack trace, ошибочный handler search или неверное продолжение.

Поэтому странный SizeOfProlog надо разбирать осторожно. Если он меньше фактического пролога, часть действий будет считаться телом функции. Если он больше, часть обычного тела может восприниматься как прологовая зона. Оба варианта опасны не всегда, но оба требуют проверки. Особенно если CodeOffset у unwind operations выходит за SizeOfProlog или если дизассемблированный пролог явно длиннее, чем заявлено.

Для static -а здесь появляется связка metadata-to-code. Он не должен рассматривать SizeOfProlog в одиночку. Надо смотреть реальный BeginAddress функции, первые инструкции, offsets unwind operations и границу prolog size. Если все согласовано, хорошо. Если нет, finding должен прямо объяснить слой: “metadata/code disagreement in prolog boundary”. Это лучше, чем абстрактное “bad unwind info”.

Для dynamic -а такая проблема может проявиться только в особый момент. Если исключение случается глубоко в теле функции, неправильная prolog boundary может не проявиться. Если исключение возникает прямо в прологе, эффект становится критичным. Поэтому динамическая валидация должна понимать: не всякая статическая странность проявится при обычном запуске. Иногда нужен targeted runtime scenario - целевой сценарий, который доводит выполнение до конкретного диапазона и конкретной фазы функции.

8.2.9. FrameRegister и FrameOffset: где RSP перестает быть единственной опорой

Теперь разберем frame register глубже. В простейшей функции RSP сам является главной опорой кадра. Функция уменьшила RSP, получила место под

локальные данные, обращается к ним через [rsp+offset], потом вернула RSP назад. Но иногда компилятор или рукописный код использует frame pointer - отдельный регистр, который остается стабильной базой даже если RSP дополнительно меняется. На x64 часто таким регистром бывает RBP, но формат допускает кодирование других регистров.

FrameRegister в UNWIND_INFO говорит: у этой функции есть регистр, который играет роль базы кадра. FrameOffset говорит, как эта база соотносится с реальной областью стека. Это нужно не только декомпилятору для красивых локальных переменных. Это нужно unwinder-у, чтобы правильно восстановить frame state в сложных случаях. Если metadata утверждает наличие frame register, а реальный код его не устанавливает, это подозрительно. Если код устанавливает RBP как frame pointer, а metadata об этом молчит, это тоже может быть проблемой.

Но опять важно не спешить. Не каждая функция с RBP обязательно использует его как frame pointer. RBP может быть обычным сохраненным nonvolatile-регистром, если компилятор решил использовать его для данных. И наоборот, некоторые оптимизации могут сделать prolog непривычным. Поэтому static должен искать не символическое “RBP встретился”, а именно семантическую связку: сохранение старого frame register, установка нового значения из RSP, согласование с UWOP_SET_FPREG и FrameOffset.

Здесь хорошо видно, почему книга должна постоянно возвращать читателя к базовым терминам. RSP - текущая вершина стека. RBP или другой frame register - возможная стабильная база кадра. Return address лежит там, куда указывает логика call/ret, но после sub rsp и сохранений до него надо правильно добраться. UNWIND_INFO нужен, чтобы не гадать. Если frame register metadata неверна, весь путь “текущий кадр -> caller frame” может стать неправильным.

dynamic может валидировать это через наблюдение after-handler path или stack trace после исключения. Если при живом исключении stack trace обрывается, уходит в мусор или показывает невозможные адреса, это может быть следствием поврежденной frame/unwind metadata. Но динамический симптом сам по себе не говорит, какое поле виновато. Поэтому dynamic нужен вместе с static -ом: один показывает симптом в живом выполнении, второй раскладывает возможные причины по байтам .xdata.

8.2.10. Handler RVA: почему адрес обработчика не равен “вот сюда прыгнет программа”

Одна из самых важных ловушек .xdata - handler RVA. Когда начинающий реверсер видит handler address, ему хочется представить прямой переход: “если будет исключение, программа прыгнет сюда”. Это слишком грубая модель. На x64 handler RVA в UNWIND_INFO указывает на language-specific handler или termination handler. В случае MSVC SEH это часто

__C_specific_handler. Но __C_specific_handler - не пользовательский __except block. Это runtime-диспетчер, который получает системные структуры и читает language-specific data.

Language-specific data переводится как данные, специфичные для языка или рантайма. Windows не обязана знать внутренний формат C __try/__except, C++ destructors или другого языка. Системный x64 unwinder знает общий контейнер: есть функция, есть UNWIND_INFO, есть handler RVA, есть хвост данных. А уже handler конкретного языка понимает, что лежит в этом хвосте. Для SEH от MSVC там будут scope table и связанные поля. Для C++ исключений логика будет другой.

Поэтому static должен аккуратно формулировать: “UNWIND_INFO has EHANDLER and handler RVA points to __C_specific_handler”, а не “exception goes to user handler at ...”. User handler может находиться глубже, в scope table. Filter expression может быть отдельным кодом. Handler block может выглядеть как unreachable block с точки зрения обычного CFG. Все это надо восстанавливать через связку handler RVA + language-specific data + понимание рантайма.

Странности handler RVA бывают разными. Handler может указывать за пределы executable image. Handler может указывать в non-executable section. Handler может указывать в writable section. Handler может быть один и тот же для огромного числа функций - что для __C_specific_handler нормально, но для неизвестного handler-а может быть сигналом dominance. Handler может быть спрятан через packer так, что на диске RVA выглядит странно, а в памяти после распаковки появляется другая картина.

Именно здесь связка static /dynamic становится очень естественной. static говорит: “у этой функции handler RVA такой-то, он расположен там-то, используется таким-то количеством RuntimeFunction”. dynamic проверяет: “при живом исключении действительно ли управление достигает этого handler-а, и куда RIP идет после handler hit”. Если handler RVA указывает на __C_specific_handler, dynamic может фиксировать не только сам runtime handler hit, но и последующее движение к filter/handler block, если такие точки можно поставить.

> Handler RVA - это не всегда конечная точка обработки. Часто это вход в маленький рантайм-диспетчер, который уже решает, какой пользовательский блок имеет отношение к исключению.

8.2.11. Language-specific data: самая важная область, которую легко назвать “мусором”

После handler RVA может лежать language-specific data. Для неопытного парсера это просто “оставшиеся байты”. Для реверсера это потенциально самая интересная часть: именно там может быть карта protected ranges, filters и handler blocks. Protected range - защищенный диапазон кода, то есть участок, для которого действует __try. Filter - выражение, которое решает, обрабатывать

исключение или искать дальше. Handler block - код, который выполняется, если filter вернул EXCEPTION_EXECUTE_HANDLER.

Windows x64 на уровне общего unwind format не описывает универсальную структуру language-specific data. Это важный момент. Общая система говорит: “если есть handler, вот указатель на handler, а дальше данные для него”. Как эти данные устроены, знает сам handler. Поэтому универсальный static должен быть осторожным: он может уверенно найти границу начала этих данных, но чтобы интерпретировать их как scope table, ему нужна поддержка конкретного ABI/компилятора/рантайма.

ABI - application binary interface, прикладной бинарный интерфейс. Это набор правил о том, как код в бинарном виде взаимодействует: как передаются аргументы, какие регистры сохраняются, как устроены структуры, как вызываются обработчики. В нашей книге ABI - это не модное слово, а причина, почему один и тот же C-код после компиляции должен оставлять предсказуемые следы. Если static знает MSVC SEH ABI, он может разбирать __C_specific_handler data. Если ABI другой или код сгенерирован нестандартно, confidence должен снижаться.

Странности language-specific data могут проявляться как невозможные protected ranges, фильтры вне .text, handler blocks в writable секции, пересечения scope entries, слишком много записей, ссылки назад в неожиданные места. Но каждую такую странность надо привязывать к конкретной модели. Нельзя просто сказать “после handler RVA какие-то подозрительные байты”. Надо сначала доказать, что мы правильно распознали handler, правильно определили формат данных и правильно посчитали offsets.

Для dynamic -a language-specific data становится проверяемой через реальные hits. Если static считает, что protected range A должен вести к filter B и handler block C, dynamic может попытаться увидеть, что при исключении внутри A действительно вызывается соответствующая логика. Это не всегда просто и не всегда автоматически, но архитектурно это правильная связь: статическая scope table превращается в динамический exception route.

8.2.12. CHAININFO: когда одна функция описывается не одной простой записью

CHAININFO часто пугает, потому что ломает привычную картинку: RuntimeFunction указывает на UNWIND_INFO, а UNWIND_INFO самодостаточен. При chained unwind information структура становится вторичной. Ее хвост содержит три поля, похожие на RuntimeFunction: begin, end и unwind info address для связанной записи. Смысл в том, что часть описания может ссылаться на primary unwind info.

Зачем это нужно? Одна причина - оптимизации вроде shrink-wrapping. Shrink-wrapping - это когда компилятор не обязательно делает весь пролог в

самом начале функции. Он может отложить сохранение регистров или выделение части стека до той ветки, где это реально нужно. В результате одна функция может иметь участки с разной unwind-ситуацией. Chained info помогает не дублировать полный набор unwind-кодов и связать вторичный участок с основным описанием.

Для реверсера важно понять: CHAININFO сам по себе не подозрителен. Это нормальная часть формата. Подозрительными становятся циклы цепочек, выходы за пределы image, ссылки на несуществующие диапазоны, chained entry с несовместимыми границами, бесконечные переходы, слишком глубокие цепочки или цепочки, которые не приводят к primary UNWIND_INFO без CHAININFO. То есть снова: не флаг плохой, а нарушение связности.

Связность - ключевое слово. Если одна unwind-структура ссылается на другую, static должен строить граф. Граф здесь означает не обязательно красивую картинку, а модель отношений: запись A ведет к записи B, B ведет к C, C является primary. Если A ведет назад к A, это cycle. Если A ведет в никуда, это dangling reference. Если A ведет к записи, чей диапазон не связан с текущей функцией, это semantic mismatch. Каждое из этих состояний надо формулировать отдельно.

dynamic может проверить chained info косвенно. Если выполнение попадает в chained range и возникает исключение, Windows должна пройти через правильную unwind-схему. Если цепочка битая, это может проявиться как неправильный stack walk, crash во время exception dispatch, отсутствие handler hit или странный RIP trace. Но опять: если этот участок никогда не исполнялся, динамическое молчание не отменяет статический finding.

8.2.13. Metadata placement: где лежит .xdata и почему права секции важны

В прошлом разделе мы уже говорили о placement для .pdata. Для .xdata placement еще тоньше. Metadata placement - размещение метаданных. В нормальной картине unwind metadata лежит в read-only data области, часто в .xdata или .rdata, и не исполняется как код. Ей не нужны права execute, потому что Windows читает ее как данные. Ей не нужны права write в обычном image, потому что компилятор сформировал ее заранее, а loader просто мапит образ.

Если UNWIND_INFO лежит в executable section, это не автоматический malware. Некоторые тулчейны, packer-ы или нестандартные сборки могут размещать данные рядом с кодом. Но это сигнал. Почему? Потому что executable metadata стирает границу между “байты, которые CPU исполняет” и “байты, которые unwinder читает как структуру”. Для анализа это неудобно, а для обфускации полезно. Одни и те же bytes потенциально могут быть показаны инструменту как code, data или metadata в зависимости от точки входа.

Если UNWIND_INFO лежит в writable section, сигнал другой. Writable означает, что во время выполнения данные могут быть изменены. Обычной статической unwind metadata это не нужно. Но packer, JIT, dynamic-like runtime или self-modifying code могут менять metadata. Это не значит, что файл вредоносен, но это означает: статический static видит только начальное состояние на диске, а dynamic должен проверить состояние в памяти во время выполнения.

Особенно интересна комбинация writable + executable. RWX section - секция, которую можно читать, писать и исполнять. В современных защитных моделях это часто считается подозрительным, потому что код может генерироваться или менять себя на лету. Если .xdata или handler-related metadata указывает туда, static должен поднять confidence осторожно, но заметно: это уже не просто необычное имя секции, а опасная комбинация прав.

Для хорошего отчета важно не писать ".xdata not in .xdata section = bad". Имя секции не является источником истины. Источник истины - RVA mapping, section flags, data directory и возможность корректно прочитать структуру. Поэтому finding должен звучать примерно так: "UNWIND_INFO for function range X is located in writable executable section Y; static unwind metadata may be mutable at runtime; dynamic validation recommended".

Здесь dynamic получает прямую задачу: снять memory map процесса, проверить, где находится runtime copy этих данных, не меняются ли они после unpacking, не регистрируются ли dynamic function tables через RtlAddFunctionTable или RtlInstallFunctionTableCallback, и совпадает ли runtime handler route с тем, что static видел на диске.

8.2.14. Reuse UNWIND_INFO: экономия, артефакт компилятора или намеренная маскировка

Reuse - повторное использование. В контексте .xdata это ситуация, когда несколько RuntimeFunction записей указывают на один и тот же UNWIND_INFO. Само по себе это не обязательно плохо. Если несколько маленьких функций имеют одинаковый unwind layout, компилятор или линкер теоретически может переиспользовать описание. Также могут существовать thunks, wrappers, маленькие фрагменты, у которых одинаковая схема кадра. Поэтому reuse нельзя автоматически считать обфускацией.

Но reuse становится сигналом, когда масштаб или контекст необычны. Например, сотни разных функций разного размера, разного поведения и разных handler situations указывают на один UNWIND_INFO. Или один и тот же UNWIND_INFO с handler flags используется для множества несвязанных диапазонов. Или reuse сосредоточен вокруг странной секции, где одновременно лежат packed code и writable metadata. В таком случае речь уже не просто об экономии, а о возможном искажении карты функций.

Для static -a reuse должен быть не бинарным finding, а статистикой с объяснением. Сколько функций ссылаются на один unwind RVA? Каковы их размеры? Находятся ли они рядом? Одинаковы ли prolog patterns? Есть ли handler flags? Где лежит сам UNWIND_INFO? Есть ли совпадение с known compiler pattern? Только после этого можно говорить о suspicion.

Known compiler pattern - известный шаблон компилятора. Это важная идея для уменьшения ложных срабатываний. Если MSVC или MinGW в определенных случаях генерирует похожую metadata, static должен это знать или хотя бы не делать слишком сильный вывод без подтверждения. В нашем проекте это особенно важно, потому что пользовательский инструмент должен быть инженерным, а не истеричным. Он должен помогать реверсеру, а не засыпать его красными флагами.

dynamic может проверить reuse через runtime coverage. Если несколько диапазонов используют один UNWIND_INFO, но в живом исполнении только один из них достигается, мы пока подтвердили только один маршрут. Если разные функции реально исполняются и при исключениях дают разные handler routes, хотя metadata у них одинаковая, это сильный повод изучать глубже. Если reuse статический, но runtime регистрирует новую таблицу и перекрывает поведение, это уже мост к dynamic function tables.

8.2.15. Handler dominance в .xdata: когда один обработчик становится центром карты

Handler dominance - ситуация, когда один handler обслуживает очень большую часть unwind-карты. Слово dominance можно перевести как доминирование. Но в анализе это не эмоциональное слово, а статистическое наблюдение: много RuntimeFunction через .xdata ведут к одному handler RVA. В случае __C_specific_handler это нормально: MSVC SEH использует общий runtime handler, и множество функций могут указывать на него. Поэтому handler dominance надо интерпретировать только после идентификации handler-а.

Если handler - известный runtime helper, например __C_specific_handler, dominance может означать активное использование SEH или compiler/runtime pattern. Но если handler неизвестен, расположен в странной секции, writable/executable, не имеет понятного символа, используется сотнями функций и связан с deliberate exceptions, это уже другая картина. Тогда dominance может быть частью exception-based control flow или obfuscation layer.

Важно не путать handler dominance с protected range dominance. Один общий runtime handler может обслуживать разные protected ranges через разные language-specific data. То есть адрес handler RVA одинаковый, но реальная логика обработки различается в хвосте данных. Если анализатор смотрит только на handler RVA, он видит “везде одно и то же”. Если он читает

scope tables, он может увидеть, что реальные filters и handler blocks разные. Поэтому хороший static должен идти глубже, когда умеет распознавать формат данных.

Для dynamic -a handler dominance означает, что один breakpoint может давать много hits. Это удобно, но опасно для интерпретации. Если breakpoint на __C_specific_handler сработал, это еще не значит, что сработал конкретный пользовательский handler. Нужно смотреть параметры, DispatcherContext, текущий RuntimeFunction, EstablisherFrame, а затем route после handler-a. В более простой версии можно хотя бы записывать: handler hit occurred for function range X, exception code Y, RIP before/after Z.

В книге это надо закрепить так: статическая dominance отвечает на вопрос “куда указывают записи”. Динамическая валидация отвечает на вопрос “какой путь реально прошел поток”. Эти вопросы похожи, но не равны. Если их смешать, можно принять общий runtime helper за злой центральный dispatcher или, наоборот, пропустить настоящую custom exception dispatcher логику.

8.2.16. Несоответствие .xdata и пролога: когда две правды спорят друг с другом

Одна из самых полезных проверок static -a - сверка .xdata с реальным прологом. В нормальном мире пролог функции и UNWIND_CODE рассказывают одну историю разными языками. Ассемблер говорит: push rbx, sub rsp, 0x40, mov rbp, rsp. Metadata говорит: saved RBX, allocated 0x40 bytes, set frame pointer. Если эти два рассказа совпадают, confidence повышается. Если расходятся, реверсер должен остановиться.

Расхождение может быть мягким. Например, оптимизированный компилятор использует непривычный порядок, или функция имеет несколько участков, или chained info объясняет то, что на первый взгляд кажется несоответствием. Расхождение может быть жестким: metadata говорит, что RBX сохранен, но в коде нет сохранения; metadata говорит, что выделено 0x200 байт стека, а код выделяет 0x20; metadata описывает frame pointer, которого нет; CodeOffset указывает за пределы prolog.

Надо помнить, что дизассемблер тоже может ошибаться. Если код смешан с данными, если функция начинается не там, если есть obfuscation, если disassembler неправильно определил границу, то “пролог” в окне инструмента может быть не тем прологом, который использовал компилятор. Поэтому static должен сверять не псевдокод декомпилятора, а реальные байты с BeginAddress RuntimeFunction. И даже тогда finding должен сохранять аккуратность: metadata/code mismatch suspected, because observed instructions do not match declared unwind operations.

Почему это важно для dynamic -a? Потому что mismatch может не проявляться до исключения. Программа может нормально выполнять call/ret,

потому что обычный эпилог написан правильно. Но как только возникает исключение в середине функции, Windows использует metadata, а не обычное выполнение эпилога. Тогда ложная metadata может привести к неправильному caller context. В живом маршруте это может выглядеть как странный stack trace, невозможный return address или уход RIP в неисполняемую память.

Здесь хорошо видно различие между normal control flow и exception control flow. Normal control flow - обычное выполнение инструкций: call, jmp, ret. Exception control flow - путь, который возникает через исключение, dispatcher, handler search и unwind. .xdata почти не влияет на обычный путь, пока не нужен stack walk или exception handling. Поэтому бинарь может “работать”, но быть плохим для unwinding. static замечает это заранее, dynamic проверяет при живом исключении.

8.2.17. Когда .xdata специально запутывает инструменты

Теперь можно осторожно говорить об обфускации. Обфускация - это намеренное усложнение понимания программы. В контексте .xdata это не обязательно шифрование и не обязательно malware. Это может быть защитный слой, packer, anti-analysis техника, ручная сборка, JIT runtime или нестандартный loader. Но общий смысл один: статическая карта становится менее прямой.

Какие приемы возможны вокруг .xdata? UNWIND_INFO может лежать в неожиданной секции. Несколько функций могут ссылаться на один unwind blob. Handler может быть центральным dispatcher-ом. Chained info может усложнять связь диапазонов. Пролог может быть написан так, чтобы его трудно было сопоставить с metadata. Dynamic function tables могут появляться только во время выполнения. На диске .pdata/.xdata может быть неполной, а настоящая карта регистрируется через API.

Здесь важно не перейти грань и не учить “как ломать анализаторы”. Наша цель - научить реверсера и автора static /dynamic -а понимать признаки. Мы не строим инструкции по созданию вредоносной обфускации. Мы строим модель проверки: что считается нормальным, где появляется uncertainty, какие findings нужны, что можно подтвердить динамически.

В такой модели static не должен ставить один большой verdict только на основании одного признака. Writable .xdata - сигнал. Handler dominance - сигнал. Mismatch с прологом - сигнал. CHAININFO cycle - сильный сигнал. Dynamic function table call - важный runtime-сигнал. Но итоговая оценка должна складываться из evidence chain. Evidence chain - цепочка доказательств: каждое звено показывает, что именно было найдено, на каком уровне, почему это важно и чем подтверждено.

dynamic в обфусцированных сценариях особенно важен, потому что статическая карта может быть ложной или неполной. Но dynamic тоже не

всесилен. Он видит только те маршруты, которые были исполнены. Если защитный код активируется по времени, окружению, вводу или anti-debug проверкам, динамическая валидация может не покрыть все ветки. Поэтому отчет должен честно разделять observed runtime behavior и unobserved static suspicion.

8.2.18. Как это должно выглядеть в отчете static -а

Теперь переведем теорию в язык отчета. Плохой отчет пишет: “bad .xdata”. Хороший отчет пишет: какая RuntimeFunction запись ссылается на какой UnwindInfoAddress, где этот RVA расположен, какие байты удалось прочитать, на каком поле возникла проблема, какова серьезность, насколько уверен анализатор и что должен сделать человек дальше.

Например, если CountOfCodes выходит за границы секции, finding должен содержать function range, unwind RVA, section name/permissions, expected size и available size. Если handler RVA указывает в writable executable section, finding должен объяснять, что handler metadata ведет в область с изменяемым исполняемым кодом, поэтому статическая интерпретация требует динамической проверки. Если CHAININFO образует цикл, report должен показать цепочку RVA, а не просто сказать “cycle”.

Для реального проекта это особенно важно, потому что отчет может читать не только автор кода. Его может открыть человек в Ghidra, посмотреть JSON, сравнить CSV, использовать HTML summary. Чем подробнее evidence, тем меньше магии. Хороший static не должен говорить “поверь мне”. Он должен показывать путь: PE -> Exception Directory -> RuntimeFunction -> UnwindInfoAddress -> UNWIND_INFO field -> anomaly -> interpretation.

Это прямо связано с будущей проектной главой, которую мы вставим в конце книги. Архитектура EH/Unwind Analyzer должна быть описана как конвейер evidence. Utility layer безопасно читает байты. PE parser строит карту RVA. Unwind decoder читает .pdata/.xdata. Heuristic engine формулирует findings. Report layer сохраняет доказательства. dynamic проверяет живые маршруты. Все это не отдельные куски, а одна система доверия.

8.2.19. Как dynamic проверяет подозрительную .xdata в живом выполнении

dynamic , в твоем смысле, - динамическая валидация. Он не просто “защищает” и не просто “запускает файл”. Он отвечает на вопрос: совпадает ли статическая карта static -а с тем, что реально происходит при выполнении. В контексте .xdata это означает: если metadata говорит, что у функции есть handler, можно ли увидеть handler hit? Если статически найден странный handler RVA, достигается ли он? Если функция должна раскручиваться через конкретный layout, как выглядит RIP trace после исключения?

RIP trace - последовательность адресов instruction pointer во времени. RIP показывает, какая инструкция выполняется или будет выполняться. Когда мы говорим “куда уходит функция”, это не только “какой ret address”. При исключении функция может уйти в dispatcher, handler, filter, cleanup, resume point или crash path. Поэтому dynamic должен наблюдать маршрут: exception event, current RIP, handler breakpoint, continue behavior, next RIP values. Это живое дополнение к статической карте .xdata.

Однако динамическая проверка должна быть честной. Если dynamic не увидел handler hit, возможны разные причины. Handler действительно не используется. Исключение не произошло. Нужный protected range не был достигнут. Breakpoint поставлен не туда. Handler RVA был неправильно интерпретирован из-за ошибки parsing. Код зарегистрировал dynamic function table после запуска. Target обнаружил debugger и изменил поведение. Поэтому “не увидели hit” не равно “handler отсутствует”.

Хороший dynamic report должен говорить в терминах coverage. Coverage - покрытие. Какие модули были загружены? Какие функции исполнялись? Какие exceptions были first-chance? Какие дошли до second-chance? Какие handler addresses были поставлены на breakpoint? Какие сработали? Был ли записан RIP trace после hit? Это не просто техническая детализация, а способ не делать ложные выводы.

Связка static /dynamic для .xdata выглядит так. static нашел структуру и сформулировал static hypothesis. dynamic проверил runtime evidence. Report связал два слоя: “static EHANDLER at RVA X; runtime handler hit observed for exception code Y; RIP after handler moved to Z” или “static anomaly not exercised during run”. Так книга должна подвести читателя к архитектуре проекта без преждевременного превращения главы в документацию по коду.

8.2.20. Несколько медленных сценариев: как думать, а не просто отмечать галочки

Первый сценарий: UNWIND_INFO находится в обычной read-only секции, version нормальная, flags без handler-a, CountOfCodes корректный, операции соответствуют прологу. Здесь нет драмы. static может записать normal unwind metadata. dynamic ничего особенного не обязан подтверждать, если нет подозрений. Это базовая нормальная картина.

Второй сценарий: UNWIND_INFO читается, но CountOfCodes требует дополнительных slots, которых нет. Это уже malformed. static должен остановить глубокий разбор этой записи и не строить handler выводы из мусора. dynamic может проверить, достигается ли функция и что происходит при исключении, но статический finding остается: metadata unreliable.

Третий сценарий: flags говорят EHANDLER, handler RVA указывает на __C_specific_handler, language-specific data выглядит как scope table, protected ranges находятся внутри функции. Это может быть нормальная SEH-логика.

static должен не пугать, а объяснить структуру. dynamic при targeted exception может подтвердить handler route.

Четвертый сценарий: flags говорят EHANDLER, handler RVA указывает в writable executable section, один и тот же unknown handler обслуживает сотни функций, а рядом в коде есть deliberate RaiseException или int 3. Здесь уже появляется сильная связка сигналов. static формулирует подозрение на exception-based control flow или защитный runtime. dynamic проверяет, реально ли исключения ведут в этот handler и как меняется RIP после обработки.

Пятый сценарий: flags говорят CHAININFO, но chain возвращается к уже посещенной структуре. Это cycle. Для формата unwind это серьезная проблема, потому что поиск primary info может не завершиться. static должен показать chain path. dynamic может выявить runtime failure только если этот путь реально задействован. Если нет, finding остается статическим, но не подтвержденным динамически.

Шестой сценарий: .xdata на диске выглядит нормальной, но при запуске target вызывает RtlAddFunctionTable и регистрирует новую карту для памяти с JIT/payload кодом. Тогда static static видит только часть картины. dynamic становится главным источником runtime evidence: он фиксирует регистрацию dynamic table, диапазоны base/end, handler routes и исключения внутри динамического кода.

8.2.21. Типичные ошибки начинающего реверсера при чтении .xdata

Первая ошибка - считать .xdata “комментарием компилятора”. Это не комментарий. Это часть ABI-контракта с Windows. Если она неверна, ошибаться могут stack walk, exception dispatch, crash analysis и инструменты.

Вторая ошибка - читать UnwindInfoAddress как file offset. Это RVA. Его надо переводить через PE sections. Если сразу прыгнуть в file offset, можно прочитать совершенно другие байты и построить ложные findings.

Третья ошибка - забыть padding после UNWIND_CODE. Из-за этого handler RVA читается с неправильного места, а вся дальнейшая логика становится мусором.

Четвертая ошибка - думать, что CountOfCodes равно числу операций. CountOfCodes считает slots. Дополнительные slots тоже входят в счетчик.

Пятая ошибка - считать handler RVA конечным пользовательским обработчиком. Часто это runtime handler вроде __C_specific_handler, а реальный filter/handler block спрятан в language-specific data.

Шестая ошибка - пугаться CHAININFO. Chained unwind info может быть нормальной оптимизацией. Подозрительность появляется не от самого флага,

а от битой или странной цепочки.

Седьмая ошибка - делать verdict только по статике. Статика показывает карту. Но если вопрос звучит “куда реально ушла функция и как обработалось исключение”, нужна динамическая валидация dynamic -а.

Восьмая ошибка - делать verdict только по динамике. Если dynamic не увидел handler hit, это может означать отсутствие покрытия, а не отсутствие handler-а. Поэтому dynamic evidence надо связывать со static coverage.

8.2.22. Главная модель раздела

Теперь можно собрать все в одну спокойную модель. .pdata говорит: для данного RIP есть такой-то диапазон функции и такой-то UnwindInfoAddress. .xdata говорит: вот как функция построила кадр, какие действия пролога надо обратить, есть ли handler-related хвост, есть ли chained info, где начинается language-specific data. static читает эту модель на диске и проверяет ее внутреннюю согласованность. dynamic проверяет, как эта модель проявляется в живом процессе.

Когда .xdata выглядит странно, мы не должны сразу делать большой вывод. Мы должны спросить: странность физическая, структурная, семантическая или поведенческая? Физическая - не хватает байтов. Структурная - поля противоречат формату. Семантическая - metadata не похожа на код или handler указывает в странное место. Поведенческая - runtime-маршрут не совпадает со статической картой. Это разные уровни, и хороший инструмент обязан их различать.

Для будущего static -а этот раздел дает набор статических вопросов. Можно ли прочитать UNWIND_INFO? Правильна ли версия? Согласованы ли flags? Не ломается ли CountOfCodes? Правильно ли найден хвост? Где лежит handler или chained info? Похожа ли metadata на пролог? Для dynamic -а этот раздел дает набор динамических вопросов. Достигалась ли функция? Было ли исключение? Сработал ли handler? Куда ушел RIP после hit? Появились ли dynamic function tables?

Именно поэтому .xdata - это не “следующая структура после .pdata”. Это сердце практического reverse engineering exception/unwind модели на Windows x64. Если .pdata дает нам адресную карту функций, то .xdata дает правила выхода из каждого кадра. А если правила выхода странные, то реверсер обязан остановиться, не спеша разобрать байты, связать их с кодом, а затем проверить живой маршрут там, где это возможно.

Переход к следующему разделу

Мы разобрали странности .pdata и .xdata отдельно. Но настоящая сила анализа появляется тогда, когда мы сравниваем metadata с кодом. Следующий раздел будет посвящен несоответствию пролога и unwind metadata: ситуации,

когда инструкции функции говорят одно, а UNWIND_CODE утверждает другое. Это уже не просто “структура лежит странно”. Это конфликт двух источников истины: реального машинного кода и машинно-читаемого договора с Windows.

8.3. Несовпадение кода и unwind metadata: когда пролог делает одно, а .xdata рассказывает другое

В предыдущих разделах мы отдельно разобрали две большие области: .pdata как карту диапазонов функций и .xdata как набор инструкций для восстановления кадра. Теперь нужно сделать следующий, более тонкий шаг. Нужно перестать смотреть на metadata изолированно. У реверсера есть два рассказчика: реальный машинный код функции и unwind metadata, которую оставил компилятор или другой генератор кода. В нормальном исполняемом файле эти два рассказчика говорят об одном и том же событии разными языками. Код показывает, что функция реально делает с RSP, регистрами и стеком. Metadata объясняет Windows, как эти действия надо мысленно отменить, если исключение возникло не в конце функции, а где-то внутри.

Если эти два рассказа совпадают, Windows может надежно пройти назад по стеку, отладчик может построить call stack, обработчик исключения может получить правильный context, а анализатор может уверенно сказать: да, эта функция описана согласованно. Если рассказы расходятся, появляется mismatch. Mismatch - несовпадение. В нашем контексте это не любое отличие байтов от ожиданий реверсера, а именно расхождение между тем, что делает пролог функции, и тем, что о нем заявляет UNWIND_INFO.

Это очень важное место книги, потому что именно здесь теория начинает превращаться в инженерный анализ. До этого мы могли читать структуры и говорить: поле такое, флаг такой, RVA такой. Теперь этого мало. static должен не просто распарсить .xdata, а сверить ее с инструкциями функции. dynamic должен не просто запустить target, а понять, проявляется ли это расхождение в живом маршруте: ломается ли stack walk, уходит ли RIP туда, куда ожидалось, попадает ли исключение в нужный handler, совпадает ли фактический route с тем, что обещала статическая карта.

Нужно сразу убрать одну опасную мысль: mismatch не равен malware. Несовпадение может быть следствием ошибки парсера, нестандартного компилятора, hand-written assembly, hotpatch-пролога, оптимизации, packed code, обрезанного файла, намеренной обфускации или действительно поврежденной unwind metadata. Поэтому хороший инструмент не должен кричать. Он должен объяснять. Он должен показать, где именно расходятся код и metadata, почему это важно, какой слой затронут, какие есть нормальные объяснения и что можно проверить динамически.

> Главная мысль раздела: .xdata нельзя считать правильной только потому, что она синтаксически читается. Ее надо сверять с тем, что функция реально делает со стеком, регистрами и адресом возврата.

8.3.1. Сначала снова связываем фундамент: функция, кадр, RSP и адрес возврата

Чтобы mismatch не превратился в сухую проверку полей, надо еще раз восстановить базовую картину. Функция в машинном коде - это не просто “кусок .text”. Когда одна функция вызывает другую, процессор выполняет инструкцию call. Эта инструкция делает две вещи: кладет на стек адрес возврата и передает управление в callee. Callee - вызываемая функция. Caller - вызывающая функция. Адрес возврата, return address, - это адрес инструкции, с которой caller должен продолжить работу после завершения callee.

Стек в x64 растет в сторону меньших адресов. Регистр RSP указывает на текущую вершину стека. Когда call кладет return address, RSP уменьшается, а по адресу [RSP] оказывается адрес продолжения. После этого callee начинает работать уже поверх этого состояния. Если callee нужна память для локальных переменных, она делает sub rsp, N. Если ей нужно сохранить nonvolatile-регистры, она кладет их на стек или записывает в заранее выделенные места.

Nonvolatile-регистры - это регистры, значение которых callee обязана сохранить для caller-а, если она хочет ими пользоваться. В Windows x64 к таким регистрам относятся, например, RBX, RBP, RSI, RDI, R12-R15, а также часть XMM-регистров. Смысл слова nonvolatile можно перевести как “не летучий”, “не испаряющийся”. Caller имеет право ожидать, что после вызова эти регистры останутся прежними. Поэтому если callee временно использует RBX, она должна где-то сохранить старый RBX и перед возвратом восстановить его.

Пролог функции - это участок, где callee строит свой рабочий кадр. Кадр стека, stack frame, - это не отдельный объект Windows и не структура C. Это соглашение о том, какая область стека принадлежит текущему вызову функции и где внутри этой области лежат сохраненные регистры, локальные переменные, временные значения и иногда служебные данные. Пролог создает этот кадр. Эпилог разбирает его обратно. Но если исключение произошло в середине body, до эпилога, Windows должна уметь разобрать кадр виртуально, не выполняя обычный код функции.

Именно здесь появляется unwind metadata. UNWIND_INFO говорит: если ты находишься внутри этой функции и тебе надо получить caller context, вот какие эффекты пролога нужно отменить. Если пролог сделал push rbx, при unwind нужно извлечь сохраненный RBX из стека и сдвинуть виртуальный RSP. Если пролог сделал sub rsp, 40h, при unwind нужно прибавить 40h к RSP. Если функция использовала frame pointer, надо понимать, как RBP связан с исходным стеком.

Поэтому mismatch всегда надо связывать не с абстрактным “код не совпал”, а с вопросом: сможет ли Windows восстановить правильный caller context? Caller context - состояние вызывающей функции. В нем RIP должен стать return

address, RSP должен указывать на стек caller-а, nonvolatile-регистры должны иметь значения, которые caller ожидал увидеть. Если metadata врет, этот caller context может быть собран неверно.

8.3.2. Что именно мы сравниваем: байты кода против договора metadata

Слово “сравниваем” здесь звучит просто, но за ним стоит сложная мысль. Мы не сравниваем дизассемблированный текст с текстом из .xdata. .xdata вообще не содержит инструкций вида push rbx или sub rsp, 40h. Она содержит unwind codes - компактные записи об эффектах пролога. Поэтому static должен перевести оба мира в общий язык.

Первый мир - мир инструкций. В нем мы видим байты машинного кода, которые дизассемблер показывает как команды: push rbx, sub rsp, 20h, mov [rsp+10h], rdi, lea rbp, [rsp+80h]. Эти инструкции реально исполняются процессором. Они меняют регистры, память, flags, RIP. Но Windows во время unwind не хочет заново исполнять эти инструкции. Она хочет знать их итоговые структурные эффекты.

Второй мир - мир UNWIND_CODE. В нем есть операции вроде UWOP_PUSH_NONVOL, UWOP_ALLOC_SMALL, UWOP_ALLOC_LARGE, UWOP_SET_FPREG, UWOP_SAVE_NONVOL, UWOP_SAVE_XMM128. Эти операции не исполняются процессором. Это описание для unwinder-а: такой-то nonvolatile-регистр был сохранен; столько-то байт было выделено на стеке; frame register установлен; такой-то XMM-регистр лежит по такому-то offset-у.

Общий язык между ними - модель кадра. static должен спросить: если я прочитаю первые инструкции функции и вычислю их effect model, получится ли то же самое, что утверждает UNWIND_INFO? Effect model - модель эффектов. Например, push rbx можно выразить как “RSP уменьшается на 8, старое значение RBX сохраняется по новому [RSP]”. A UWOP_PUSH_NONVOL(RBX) выражает почти тот же смысл: “в прологе был сохранен nonvolatile-регистр RBX через push”.

Но даже здесь нужна осторожность. Реальный код может быть оптимизирован. В прологе могут быть инструкции, которые не относятся к unwind. Например, mov eax, ecx может подготавливать локальную переменную или преобразовывать аргумент. Такая инструкция не обязана иметь unwind code, потому что она не меняет RSP и не сохраняет nonvolatile-регистр. Поэтому mismatch - это не “в прологе есть инструкция, которой нет в UNWIND_CODE”. Mismatch - это когда действия, важные для восстановления кадра, не отражены правильно или отражены неправильно.

```
Пример смыслового сравнения:

Код функции:
    push rbx
    sub  rsp, 40h
```

```
Ожидаемая metadata по эффектам:
    UWOP_PUSH_NONVOL RBX
    UWOP_ALLOC_SMALL/ALLOC_LARGE для 40h байт

Важно: metadata не обязана повторять текст инструкций,
она обязана описывать unwind-значимые эффекты.
```

Это различие принципиально. Новичок часто хочет сделать checker по принципу “увидел push - найди UWOP_PUSH”. Но хороший static должен понимать, что функция может иметь сложную форму, что порядок unwind codes не обязательно совпадает с порядком чтения дизассемблера, что prolog size ограничивает зону, а epilog вообще распознается особыми правилами. Поэтому сверка - это не поиск строк, а построение модели.

8.3.3. Почему синтаксически валидная .xdata может быть семантически неправильной

Синтаксическая валидность означает, что структуру можно прочитать без выхода за границы. Version выглядит допустимой, CountOfCodes не ломает размер, padding посчитан, handler RVA лежит там, где его можно интерпретировать как число, chained info имеет правильный минимальный размер. Такой UNWIND_INFO может пройти базовый parser. Но это еще не значит, что он правдиво описывает функцию.

Семантическая валидность означает, что прочитанные поля имеют смысл относительно реального кода. Например, UNWIND_INFO говорит, что функция сохраняет RBX через push, но в прологе нет push rbx и нет другого сохранения RBX. Или metadata говорит, что функция выделяет 0x80 байт стека, а код делает sub rsp, 0x20. Или metadata говорит, что frame register - RBP, но RBP в коде не устанавливается как база кадра. В таких случаях структура может быть красивой внутри себя, но ложной относительно функции.

Это как паспорт и человек. Паспорт может быть напечатан без ошибок: имя, дата, номер, фотография на месте. Но если фотография не совпадает с человеком, это уже проблема другого уровня. .xdata может быть правильно распарсена как структура, но не совпадать с “человеком” - реальной функцией в .text. static должен уметь отделять эти два уровня.

Такое разделение напрямую влияет на качество findings. Если parser не смог прочитать UNWIND_INFO, это parse failure. Если parser прочитал UNWIND_INFO, но сверка с прологом выявила расхождение, это prolog/unwind mismatch. Если mismatch влияет на восстановление RSP, severity выше. Если mismatch касается frame register, который реально не используется, severity может быть ниже или требовать ручной проверки. Если mismatch затрагивает handler route, это уже связано с exception dispatch и может требовать динамической проверки dynamic -ом.

В нашем проекте это особенно важно, потому что static должен быть не просто “парсером структур”, а анализатором evidence. Evidence -

доказательная опора. Он должен уметь сказать не только “поле X странное”, но и “поле X обещает восстановить RBX из offset-a Y, однако в прологе функции нет соответствующего сохранения, поэтому unwinder может восстановить неверный caller context”.

8.3.4. Самый опасный класс mismatch: неправильная модель RSP

RSP - центральный регистр для всего unwind. Если неправильно восстановлен RSP, почти все остальное начинает рушиться. Почему? Потому что return address обычно находится относительно RSP. Сохраненные nonvolatile-регистры часто лежат на стеке относительно RSP или frame pointer. Переход от текущего кадра к caller-у выражается через изменение RSP. Поэтому ошибка в размере stack allocation, пропущенный push или лишний push в metadata может увести unwinder к неправильному месту памяти.

Представим простую функцию. После call на стеке лежит return address. Затем функция делает sub rsp, 40h, выделяя 64 байта под локальные нужды и shadow space/выравнивание. Если исключение происходит в середине функции, текущий RSP указывает уже на нижнюю границу выделенного кадра. Чтобы найти return address, virtual unwind должен прибавить 40h, а затем прочитать адрес возврата с соответствующего места. Если metadata говорит не 40h, а 20h, unwinder будет искать return address не там.

```
Условная картина:

После call:
    [RSP] = return_address

После sub rsp, 40h:
    RSP = RSP - 40h

Правильный virtual unwind:
    RSP = RSP + 40h
    RIP = *(RSP)

Если metadata ошибочно говорит 20h:
    RSP = RSP + 20h
    RIP = *(не то место)
```

Такая ошибка может проявиться по-разному. Отладчик может показать странный call stack. Exception dispatcher может перейти не к тому caller-у. Stack walking в crash dump может оборваться. dynamic при динамической проверке может увидеть, что после handler-a RIP trace не совпадает с ожидаемой картой. Иногда программа может продолжать работу, если неправильный путь не используется, но это не делает metadata правильной.

Для static -a проверка RSP начинается с пролога. Он должен увидеть, какие инструкции меняют RSP: push, pop в прологе обычно только push, sub rsp, imm, иногда stack probing через __chkstk для больших allocations, выравнивание. Потом он должен сопоставить это с UWOP_PUSH_NONVOL,

UWOP_ALLOC_SMALL, UWOP_ALLOC_LARGE и другими unwind operations. Нельзя ограничиться одним sub rsp. В x64 пролог может сохранять регистры push-ами, а каждый push тоже меняет RSP на 8 байт.

Очень частая ошибка начинающего анализатора: он считает только sub rsp, N и забывает, что push rbx тоже часть изменения стека. Но для unwind push важен не только как изменение RSP. Он еще говорит, где лежит сохраненный RBX. Поэтому push одновременно влияет на stack pointer и на восстановление nonvolatile-регистра. Именно поэтому сверка должна быть структурной.

8.3.5. Несовпадение stack allocation: sub rsp и UWOP_ALLOC_*

Stack allocation - выделение места на стеке. В x64 это обычно выглядит как sub rsp, N. Функция уменьшает RSP, потому что стек растет вниз, и получает область памяти для локальных переменных, временных буферов, spill slots и выравнивания. Для unwind metadata этот эффект описывается операциями UWOP_ALLOC_SMALL или UWOP_ALLOC_LARGE. Названия говорят о размере кодирования, а не о логической важности. Small и large - это способы компактно описать размер allocation.

UWOP_ALLOC_SMALL используется для сравнительно небольших выделений, которые можно закодировать компактно. UWOP_ALLOC_LARGE используется для больших выделений и имеет дополнительные slots в массиве UNWIND_CODE. Новичку важно не зазубрить только названия, а понять причину: формат UNWIND_INFO экономит место, потому что таких записей много, поэтому частые маленькие allocations кодируются короче, а редкие большие - длиннее.

Mismatch возникает, когда реальная инструкция sub rsp, N и заявленный allocation в metadata расходятся. Например, код выделяет 0x60 байт, а metadata описывает 0x40. Или metadata говорит о 0x1000, а в коде виден только 0x20. Иногда это ошибка анализа: может быть stack probing. Для больших выделений компилятор может вызвать __chkstk или __chkstk_ms, чтобы по страницам “потрогать” стек и корректно обработать guard page. В таком случае наивный анализатор может увидеть call __chkstk и запутаться, если не понимает шаблон.

Stack probing - проверочное касание стека при большом выделении. На Windows стек растет с guard pages, и нельзя просто прыгнуть RSP далеко вниз, не дав системе шанс расширить committed stack. Поэтому компилятор использует helper, который касается страниц. Для unwind важно не то, сколько внутренних инструкций было в helper-е, а итоговый размер allocation в текущей функции. Поэтому static должен знать этот типичный pattern и не считать его автоматически подозрительным.

```
Типовая идея большого allocation:
```

```
    mov  eax, 5000h       ; размер будущего stack allocation
    call __chkstk         ; helper трогает страницы стека
    sub  rsp, rax         ; фактическое изменение RSP

Metadata должна описывать итоговый размер allocation,
а не внутреннюю работу __chkstk.
```

Опасный случай - когда allocation mismatch не объясняется ни __chkstk, ни нестандартным прологом, ни ошибкой дизассемблирования. Тогда возникает сильный finding: восстановление RSP может быть неверным. Но даже здесь отчет должен быть точным. Не “bad unwind”, а “UNWIND_INFO describes stack allocation 0x40, while prolog analysis observes stack allocation 0x60; virtual unwind may compute incorrect caller RSP”. Такой текст сразу объясняет и факт, и последствия.

dynamic может проверить такой finding через controlled exception. Если в тестируемом участке вызвать исключение или дождаться естественного exception event, а затем посмотреть stack walk и RIP trace, можно увидеть, совпадает ли runtime маршрут с ожиданием. Но важно помнить: если mismatch существует в редко исполняемой ветке, обычный запуск может его не покрыть. Динамическая валидация показывает то, что было исполнено, а не магически доказывает весь файл.

8.3.6. Несовпадение сохранения nonvolatile-регистров

Nonvolatile-регистры в Windows x64 - это часть ABI. ABI, Application Binary Interface, - бинарный договор между компилятором, операционной системой, библиотеками и машинным кодом. Он определяет, как передаются аргументы, кто сохраняет какие регистры, как выравнивается стек, как выполняются вызовы. Если функция меняет nonvolatile-регистр, она должна восстановить его перед возвратом. И если исключение возникло до обычного восстановления, unwinder должен уметь восстановить этот регистр виртуально.

Допустим, функция делает push rbx. Это значит: старое значение RBX сохранено на стеке. Позже функция может использовать RBX для своих нужд. В эпилоге она сделает pop rbx или эквивалентное восстановление, чтобы caller увидел прежний RBX. Если исключение произошло в body, до pop rbx, RtlVirtualUnwind должен восстановить RBX из места, где он был сохранен. Для этого в UNWIND_INFO должна быть соответствующая запись.

Если код сохраняет RBX, а metadata молчит, unwinder может оставить RBX в измененном состоянии. Если metadata говорит, что RBX сохранен, но код его не сохранял, unwinder может прочитать случайные данные со стека и записать их как RBX в caller context. Оба сценария плохи. В первом caller теряет свое значение. Во втором caller получает выдуманное значение. Для обычного ret это может не проявляться, потому что эпилог сам восстановит регистр. Но для исключения, stack walk или crash dump это может стать критичным.

Код:

```
    push rbx
    sub  rsp, 20h
    ... body uses rbx ...

Ожидаемый смысл metadata:
    RBX saved by push
    RSP allocation described

Если UWOP_PUSH_NONVOL RBX отсутствует,
virtual unwind не знает, откуда вернуть старый RBX.
```

Есть и другой способ сохранения nonvolatile-регистра: не push, а mov [rsp+offset], rbx после выделения стека. В metadata это может быть UWOP_SAVE_NONVOL или UWOP_SAVE_NONVOL_FAR. Слово FAR здесь не значит “далекий переход”, как в старом x86-смысле, а указывает на более широкий формат offset-а. Снова важна не терминология сама по себе, а связь: регистр сохранен не на вершине стека через push, а в slot внутри кадра.

static должен различать эти способы. Push-сохранение меняет RSP сразу. Save-to-memory после allocation обычно не меняет RSP, но создает место, откуда later unwind восстановит регистр. Если анализатор все сохранения воспринимает как push, он ошибется в модели RSP. Если все воспринимает как простые stores, он потеряет изменение стека от push. Поэтому сверка nonvolatile-регистров тесно связана со сверкой RSP.

Для dynamic -а этот класс mismatch сложнее наблюдать напрямую, чем неправильный RIP. Программа может не падать сразу, даже если восстановленный RBX неправильный. Но это может проявиться после handler-а, когда caller продолжает работу с испорченным nonvolatile-регистром. Поэтому dynamic validation должна уметь не только записывать “handler hit”, но и по возможности фиксировать context до и после: значения nonvolatile-регистров, RSP, RIP, путь возврата.

8.3.7. Frame register mismatch: когда RBP или другой регистр объявлен базой кадра, но код этого не подтверждает

Frame register - регистр кадра. В старых объяснениях стека часто говорят “RBP указывает на начало кадра”. В x64 это уже не всегда так. Компилятор может вообще не использовать frame pointer и обращаться ко всему относительно RSP. Но если функция использует frame register, UNWIND_INFO должен это описать: какой регистр выбран и с каким смещением. Поля FrameRegister и FrameOffset существуют именно для этого.

Frame pointer полезен, когда RSP меняется или когда компилятору удобно иметь стабильную базу для обращения к локальным переменным. Например, функция может установить rbp = rsp + offset. После этого локальные переменные и saved registers могут адресоваться относительно rbp, а не текущего rsp. Для unwinder-а это важно, потому что восстановление кадра может требовать понимания, где находится establisher frame.

Establisher frame - еще один термин, который легко проскочить слишком быстро. В контексте exception handling это frame, относительно которого установлен конкретный handler. Проще: это та база кадра, которую language-specific handler использует, чтобы понимать локальный контекст защищенного блока. Если frame register описан неправильно, handler может получить неверное представление о кадре функции.

Mismatch здесь выглядит так: metadata говорит, что frame register - RBP, но в прологе нет инструкции, которая устанавливает RBP как frame pointer. Или наоборот: код явно делает lea rbp, [rsp+30h], но metadata не указывает frame register. Иногда это может быть безобидно, если RBP используется как обычный nonvolatile-регистр, а не как frame pointer. Поэтому static должен не просто искать mov rbp, rsp, а понимать surrounding pattern.

```
Условный frame-pointer пролог:

    push rbp
    sub  rsp, 80h
    lea  rbp, [rsp+40h]

Metadata должна объяснять:
    RBP сохранен
    стек выделен
    RBP установлен как frame register с frame offset
```

Новичок часто думает: “Если вижу RBP, значит это frame pointer”. Это не всегда правда. RBP - просто регистр. ABI делает его nonvolatile, но не заставляет каждую функцию использовать его как frame pointer. Компилятор может использовать RBP для обычной переменной, если frame pointer omission включен. Поэтому finding по frame register должен быть осторожным: “metadata declares frame register but prolog pattern does not establish it” или “prolog establishes frame-like base but unwind metadata does not declare frame register”.

dynamic может проверить последствия через handler route. Если SEH handler зависит от establisher frame, неверная frame model может проявиться в неправильной обработке исключения. Но автоматическая проверка здесь сложна. Динамический слой может зафиксировать context, handler hit и параметры, но интерпретация часто требует связи с конкретным language-specific data. Поэтому static должен дать хорошую статическую карту, а dynamic - добавить runtime evidence, а не заменять статический анализ полностью.

8.3.8. Prolog size mismatch: когда граница пролога описана неправильно

SizeOfProlog - размер пролога в байтах. Это поле кажется простым, но оно влияет на то, как unwinder интерпретирует текущий RIP. Если исключение произошло внутри пролога, Windows должна учитывать, что не все действия пролога могли уже выполниться. Например, функция собиралась сделать push

rbx и sub rsp, 40h, но исключение произошло после push rbx, до sub rsp. Тогда нельзя просто отменять весь пролог как будто он завершен.

Документация по x64 unwind procedure различает ситуацию, когда RIP находится в prolog, epilog или body. В prolog-case unwinder должен отменять только те эффекты пролога, которые уже произошли к текущему offset-у. Поэтому CodeOffset внутри UNWIND_CODE и SizeOfProlog важны не как декоративные числа, а как способ понять временную позицию внутри построения кадра.

Mismatch может быть разным. Если SizeOfProlog меньше реального пролога, часть unwind-значимых инструкций окажется как будто вне пролога. Если больше, обычный body может ошибочно считаться продолжением пролога. Если CodeOffset у unwind operation не соответствует положению инструкции, unwind внутри пролога может отменить слишком много или слишком мало действий.

Для обычного exception в body это может не проявиться, потому что весь пролог уже выполнен. Но для исключения внутри пролога, для breakpoint-а, для single-step, для guard page, для anti-debug трюков это становится важным. Если программа намеренно вызывает исключение внутри пролога или около него, неправильная prolog metadata может стать частью control-flow trick.

```
Пример идеи:

offset 0: push rbx
offset 2: sub rsp, 40h
offset 6: mov [rsp+20h], rdi

Если SizeOfProlog = 2, то sub rsp уже как будто не часть пролога.
Если exception случится около offset 3, unwind-логика может быть неверной.
```

static должен сверять SizeOfProlog не только с количеством байт до первой “непрологовой” инструкции, но и с набором unwind-значимых операций. Это сложная задача, потому что определить “конец пролога” в оптимизированном коде не всегда тривиально. Но минимальный полезный подход возможен: все CodeOffset должны попадать в SizeOfProlog, unwind-relevant instructions должны быть внутри declared prolog region, а declared prolog не должен явно захватывать большой кусок обычного body без объяснимой причины.

dynamic в этой теме особенно полезен, если динамическая валидация умеет ловить исключения в ранних offset-ах функции. Но обычный запуск редко дает такое покрытие. Поэтому finding по prolog size часто будет статическим с рекомендацией manual review, а не динамически подтвержденным.

8.3.9. CodeOffset: маленькое поле, которое связывает unwind operation с моментом выполнения

CodeOffset - смещение в прологе, связанное с unwind operation. Это не RVA функции и не offset в файле. Это локальная позиция внутри пролога функции. Через CodeOffset unwinder понимает, к какому моменту построения кадра относится конкретный эффект. Если RIP находится в body, весь пролог уже выполнен. Если RIP внутри prolog, нужно знать, какие эффекты успели произойти.

Наивно можно подумать, что CodeOffset - просто “адрес инструкции”. Но в документации и реальном формате есть нюансы: offset часто указывает на конец инструкции или позицию после выполнения соответствующего действия. Поэтому простая сверка “CodeOffset должен равняться offset начала push” может быть неверной. static должен учитывать правила формата, а не человеческую привычку считать адрес инструкции ее началом.

Почему это важно? Потому что ошибка на один-два байта может не влиять на body unwind, но влиять на prolog unwind. A prolog unwind нужен не только для редких crash-ситуаций. Debugger, instrumentation, breakpoint-и, anti-debug, guard pages - все это может остановить выполнение внутри пролога. Если CodeOffset неправильно описывает порядок эффектов, восстановление caller context в этой точке станет неверным.

В отчете это надо формулировать спокойно. Например: “UWOP_PUSH_NONVOL for RBX has CodeOffset outside declared prolog size” - сильный синтаксико-семантический сигнал. Или: “observed push rbx at offset 0x01, unwind CodeOffset indicates operation at 0x08; body unwind unaffected, prolog unwind may be inaccurate” - более мягкий finding, требующий ручной проверки.

8.3.10. Порядок UNWIND_CODE: почему “не в том порядке” не всегда ошибка

Еще один источник ложных срабатываний - порядок unwind codes. Человек читает дизассемблер сверху вниз. Он видит: push rbx, push rbp, sub rsp, 40h. Ему хочется увидеть UNWIND_CODE в том же порядке. Но формат unwind metadata имеет свои правила хранения. В документации указано, что массив кодов хранит элементы в определенном порядке, и unwinder сканирует их по своим правилам. Поэтому “порядок в массиве не совпадает с человеческим чтением” сам по себе не finding.

Здесь снова помогает мысль об effect model. Нас интересует не эстетический порядок записей, а то, можно ли из них правильно отменить эффекты пролога. Если операции покрывают нужные регистры, allocation, frame register и offsets, порядок может быть корректным для формата даже если визуально кажется обратным. Поэтому static не должен делать вывод только по порядку отображения.

Хороший UI или отчет должен помогать человеку. Можно показывать два представления. Первое - raw order, как записи лежат в UNWIND_INFO. Второе -

reconstructed prolog effects, как анализатор сопоставил их с инструкциями. Тогда реверсер видит и байтовую правду, и смысловую реконструкцию. Это особенно важно для Ghidra integration: комментарий рядом с функцией должен не перегружать, а объяснять.

dynamic в этой теме обычно не проверяет порядок metadata напрямую. Он проверяет результат: если exception route и stack walk работают, порядок скорее всего интерпретирован Windows корректно. Но динамическое подтверждение не отменяет необходимости статически показывать raw evidence. Иначе анализатор становится черным ящиком.

8.3.11. Epilog mismatch: почему эпилог не описывается так же подробно, как пролог

Эпилог - участок, где функция разбирает кадр и возвращается к caller-у. Обычно там восстанавливаются nonvolatile-регистры, RSP возвращается к прежнему значению, затем выполняется ret или jmp в допустимой форме tail call. В x64 Windows эпилоги имеют строгие правила, потому что unwinder распознает их, сканируя поток инструкций, а не читая отдельную запись для каждого эпилога.

Это важная деталь: UNWIND_INFO подробно описывает пролог, но не хранит список всех эпилогов так, как новичок мог бы ожидать. Почему? Потому что у функции может быть несколько return paths, несколько эпилогов, оптимизированные tail calls. Чтобы не раздувать metadata, Windows требует, чтобы эпилоги имели узнаваемую форму. Тогда unwinder может определить: RIP сейчас находится в эпилоге, значит frame уже разбирается обычными инструкциями, и надо обработать ситуацию по особым правилам.

Epilog mismatch возникает, когда код выглядит как эпилог для человека, но не соответствует правилам, которые может распознать unwinder. Например, между add rsp, N и ret вставлена недопустимая инструкция. Или восстановление регистров сделано в форме, которую epilog scanner не признает. Или tail jump оформлен так, что не считается допустимым epilog ending. Такой код может нормально выполняться как обычный код, но быть плохо unwindable, если исключение/interrupt/debug event происходит внутри эпилога.

Это место особенно тонкое для reverse engineering. Декомпилятор может сказать: “да, функция возвращается”. CPU действительно может выполнить этот код. Но Windows unwind model требует не просто исполнимости, а распознаваемости. Unwindable code - код, через который можно надежно раскрутить стек в произвольной точке. Это более сильное требование, чем “код иногда работает”.

static должен поэтому отличать prolog mismatch от epilog pattern anomaly. Для пролога мы сверяем UNWIND_CODE с эффектами. Для эпилога мы проверяем, соответствует ли последовательность инструкций допустимым

epilog rules. Это разные классы findings. Их нельзя смешивать в одну кучу “metadata bad”.

dynamic может увидеть epilog anomaly, если breakpoint или exception попадает внутрь эпилога и stack walk ведет себя странно. Но поймать это динамически трудно без специального покрытия. Поэтому такие findings часто остаются статическими и требуют ручного подтверждения.

8.3.12. Несовпадение handler metadata и реального control flow

До сих пор мы говорили в основном о кадре: RSP, регистры, prolog, epilog. Но .xdata также может связывать функцию с exception handler-ом. Если установлен UNW_FLAG_EHANDLER или UNW_FLAG_UHANDLER, после массива UNWIND_CODE и padding лежит handler RVA и language-specific data. Здесь тоже возможен mismatch, но другого типа: metadata говорит, что у функции есть handler route, а реальный код или dynamic behavior не подтверждает понятного маршрута.

Handler route - маршрут обработки исключения. Это путь от exception event к handler-логике и дальнейшему решению: continue search, execute handler, continue execution, unwind. В статике static видит только указатели и таблицы. Он может сказать: этот RuntimeFunction имеет handler flag, handler RVA указывает на __C_specific_handler, а после него лежат данные, похожие на scope table. Но статическая карта еще не гарантирует, что при реальном исключении выполнение попадет в ожидаемый block.

Mismatch здесь может быть таким: EHANDLER установлен, но handler RVA указывает в неисполняемую или writable-only секцию. Или handler RVA указывает на код, который не похож на handler helper. Или scope table содержит protected range, который не пересекается с диапазоном функции. Или filter/handler target выходит за пределы функции без объяснимой причины. Для C++ exceptions и SEH форматы отличаются, поэтому static должен быть осторожен с language-specific data.

Особенно важно не путать __C_specific_handler с пользовательским обработчиком. __C_specific_handler - runtime helper, который получает управление от Windows и уже по scope table решает, какие filters и handlers применимы. Пользовательский __except block может быть вообще не тем адресом, который лежит в handler RVA. Поэтому finding “handler points to __C_specific_handler” обычно нормален. А вот “scope table target points outside image” или “filter RVA in non-executable writable section” уже требует внимания.

dynamic здесь особенно ценен. Он может поставить breakpoint на handler helper или на конкретные target-адреса, собрать exception code, RIP before exception, handler hits, route after handler. Именно поэтому в архитектуре проекта dynamic - не “защитная оболочка”, а динамическая валидация: он проверяет, куда реально уходит функция и как фактически проходит

обработка исключения.

8.3.13. Когда mismatch является нормальной вариативностью компилятора

Хороший анализатор обязан знать, что компиляторы не обязаны генерировать код так, как нам удобно читать. MSVC, MinGW, Clang, hand-written assembly, link-time optimization, profile-guided optimization, hotpatching - все это может менять форму пролога и эпилога. Если инструмент не понимает вариативность, он будет видеть “подозрительность” в нормальных бинарях.

Например, один компилятор может сохранять RBX через push, другой - через mov [rsp+offset], rbx. Один может использовать frame pointer, другой - нет. Один может делать allocation сразу, другой - после проверки. Один может создавать несколько epilog paths. Один может генерировать chained unwind info. Все это не плохо. Это разные способы выполнить ABI contract.

ABI contract - вот ключ. Не надо требовать от всех функций одинаковой формы. Надо требовать, чтобы форма кода и форма unwind metadata были совместимы с ABI и с правилами Windows unwind. Поэтому static должен ориентироваться не на “как обычно в моих примерах”, а на “может ли это быть корректно размотано”.

Для этого полезно разделять findings по confidence. Confidence - уверенность. Если UNWIND_INFO выходит за границы файла, confidence высокий: структура не читается. Если allocation отличается на 0x20 и есть непонятный __chkstk-like pattern, confidence ниже: нужен ручной анализ. Если handler лежит в RWX секции и при этом есть deliberate exceptions, confidence растет, но все равно надо объяснять связь.

dynamic помогает снижать неопределенность. Если статическая форма необычна, но runtime exception route стабильно работает, это не отменяет необычности, но меняет вывод: возможно, это нестандартный, но рабочий паттерн. Если статическая форма необычна и runtime показывает странный RIP trace, handler confusion или break stack walk, finding усиливается.

8.3.14. Когда mismatch похож на обфускацию

Обфускация - намеренное усложнение анализа. В контексте unwind metadata обфускация может использовать то, что многие инструменты и реверсеры доверяют .pdata/.xdata слишком сильно. Если metadata и код расходятся, разные инструменты могут построить разные функции, разные call stacks, разные CFG edges. Это создает туман: человек видит одну картину, Windows в runtime может использовать другую, а анализатор может третью.

Но снова: не всякая странность является обфускацией. Чтобы говорить “похоже на obfuscation”, нужен набор признаков. Например: multiple RuntimeFunction overlaps, writable/executable metadata placement, handler

dominance на неизвестный handler, deliberate exceptions, guard pages, dynamic function table registration, mismatch между prolog и UNWIND_INFO, runtime route через handler вместо обычного branch. Один признак - вопрос. Несколько согласованных признаков - гипотеза.

Exception-based obfuscation особенно интересна. Она строит управление не через обычные jmp/call/ret, а через исключения. Например, код намеренно вызывает access violation, breakpoint exception или guard page violation, handler меняет ContextRecord->Rip, затем возвращает EXCEPTION_CONTINUE_EXECUTION. Для обычного CFG это выглядит как crash или тупик. Для runtime это может быть переход в следующий блок.

Если .xdata при этом еще и не совпадает с прологом, анализ усложняется. static может видеть статический mismatch, но не знать, используется ли он реально. dynamic может увидеть, что exception route действительно меняет RIP. Вместе они дают сильную evidence chain: “metadata anomaly + deliberate exception + handler hit + RIP redirection”.

Такой вывод надо писать аккуратно. Не “malware”. Не “packer точно”. А “exception-based control flow suspected; static unwind metadata does not match observed prolog; dynamic validation observed handler-mediated RIP transition”. Это длиннее, но это инженерный язык. Он не пугает, а объясняет.

8.3.15. Когда mismatch появляется из-за packer-а или manual mapping

Packer - упаковщик. Он может сжимать, шифровать или перестраивать код, а при запуске распаковывать его в память. Manual mapping - ручная загрузка PE или фрагмента кода без обычного loader path, когда секции, relocation, imports и runtime tables могут подготавливаться нестандартно. Для unwind metadata это означает простую, но неприятную вещь: файл на диске может не соответствовать тому, что реально будет в памяти.

static читает файл. Он видит начальное состояние. Если packed image содержит placeholder .pdata/.xdata или странные таблицы, static честно сообщает об этом. Но runtime может зарегистрировать новую таблицу через RtlAddFunctionTable или RtlInstallFunctionTableCallback. Тогда во время выполнения Windows будет использовать dynamic function table, а не только статическую Exception Directory из файла.

Dynamic function table - таблица функций, добавленная во время выполнения. Она нужна, например, JIT-компиляторам и другим системам, которые создают код в памяти. JIT, just-in-time compilation, генерирует машинный код во время работы программы. Чтобы исключения и stack walking работали в таком сгенерированном коде, runtime должен сообщить Windows unwind metadata. Для этого и существуют API вроде RtlAddFunctionTable.

Но тот же механизм может использоваться packer-ом, dynamic -ом, loader-ом или обфускатором. Поэтому сам факт RtlAddFunctionTable не плох. Он является сильным сигналом контекста: статическая карта файла может быть неполной. Если static видит странную .xdata, а dynamic видит runtime registration новой function table, это объясняет, почему статическая карта не вся правда.

Здесь особенно важно различать наш термин dynamic и generic software dynamic . В этой книге dynamic - динамическая валидация проекта. Он не защищает target. Он наблюдает target. А "dynamic " в индустрии может означать упаковщик/защитную оболочку, которая усложняет анализ. Чтобы не путать, в проектной главе надо будет явно закрепить: наш dynamic - validation layer, а не packer/dynamic of binaries.

8.3.16. Как static должен сверять код и metadata медленно, без магии

Теперь можно сформулировать практическую модель. static начинает не с вывода, а с цепочки. Сначала он получает RuntimeFunction: BeginAddress, EndAddress, UnwindInfoAddress. Потом переводит BeginAddress и UnwindInfoAddress из RVA в file offset через PE mapping. Потом читает UNWIND_INFO. Потом определяет declared prolog size, flags, unwind codes, frame register, handler tail. Только после этого он идет в .text и дизассемблирует начало функции.

Дизассемблирование здесь должно быть ограниченным и осторожным. Не надо пытаться сразу декомпилировать всю функцию. Для сверки unwind достаточно понять prolog region и потенциальные epilog patterns. static должен читать инструкции от BeginAddress до разумной границы: declared SizeOfProlog, плюс небольшой lookahead для проверки, если declared boundary кажется странной. Но любое расширение анализа должно быть отмечено как эвристика.

Эвристика - правило, которое помогает анализу, но не является абсолютной математической гарантией. Например, "если первые инструкции сохраняют RBX, RBP и делают sub rsp, это похоже на пролог" - эвристика. А "UNWIND_INFO CountOfCodes требует N slots, но в файле осталось меньше байт" - почти строгая структурная ошибка. Хороший отчет должен различать hard validation и heuristic finding.

После чтения инструкций static строит observed effect model. Он не просто хранит текст "push rbx". Он хранит смысл: saved RBX by push at prolog offset X; RSP delta +8 for unwind; stack allocation N; frame register established; saved RDI at stack offset Y. Потом он строит declared effect model из UNWIND_CODE. И только потом сравнивает declared vs observed.

Такой подход позволяет избежать множества ложных ошибок. Например, если инструкции отличаются по форме, но дают тот же unwind effect, это не

mismatch. Если metadata хранит операции в другом порядке, но эффект совпадает, это не mismatch. Если инструкция в прологе не влияет на unwind, ее отсутствие в metadata нормально. Если metadata описывает сохранение регистра, а observed model не нашла никакого соответствия, это уже finding.

```
Модель static      -a:

RUNTIME_FUNCTION
    -> function range
    -> UNWIND_INFO
    -> declared effects

Function bytes
    -> prolog disassembly
    -> observed effects

Comparison
    -> exact match / acceptable variation / mismatch / unknown

Report
    -> evidence + consequence + confidence + recommendation
```

Слово unknown здесь важно. Не всякий случай нужно насильно классифицировать. Если дизассемблер не смог надежно разобрать пролог, если код самораспаковывается, если функции перекрываются, если секция странная, static может честно сказать: “unknown, manual review required”. Это не слабость. Это честность инженерного инструмента.

8.3.17. Как dynamic валидирует mismatch в живом выполнении

dynamic в нашем проектном смысле - динамическая валидация. Он не заменяет static и не спорит с ним. Он берет статические гипотезы и проверяет, что реально происходит во время выполнения. Если static говорит: “у этой функции подозрительный handler route”, dynamic может поставить breakpoint на handler. Если static говорит: “metadata может неверно восстанавливать caller context”, dynamic может смотреть stack trace и context во время exception event. Если static говорит: “есть dynamic function table suspicion”, dynamic может наблюдать RtlAddFunctionTable и RtlInstallFunctionTableCallback.

Что значит “куда уходит функция”? В обычной модели функция уходит через ret: RIP становится return address, который лежал на стеке. В exception-based control flow функция может “уйти” иначе: через handler, через изменение ContextRecord->Rip, через longjmp-like mechanism, через unwind в другой frame, через crash path, через top-level filter. dynamic должен фиксировать не только факт исключения, но и маршрут после него.

RIP trace - последовательность адресов выполнения после события. Это не полный фильм всей программы, а следы, которые помогают понять направление движения. Если handler hit произошел, а затем RIP оказался не в

ожидаемом handler block, а в новом dispatcher block, это важно. Если after-exception RIP совпал с адресом, который статически не имеет обычного входящего edge, это может подтверждать exception edge.

Но dynamic тоже не всесилен. Динамика зависит от входных данных, окружения, timing, coverage, наличия debugger-а. Если маршрут не был исполнен, dynamic не увидит его. Если программа меняет поведение под debugger-ом, dynamic может наблюдать debugger-aware route. Поэтому результат dynamic validation надо читать как “в этом запуске наблюдалось”, а не как “во всей программе невозможно иначе”.

Именно связка static + dynamic дает зрелый анализ. static говорит: “карта выглядит так, вот подозрительные места, вот причины”. dynamic говорит: “при выполнении мы наблюдали такие exceptions, такие handler hits, такой RIP trace”. Report соединяет это в evidence chain. Evidence chain - цепочка доказательств от статического байта до runtime события.

8.3.18. Пример первый: metadata занижает stack allocation

Рассмотрим простой сценарий. Функция в коде делает push rbx и sub rsp, 60h. UNWIND_INFO описывает push rbx, но allocation указывает только 40h. Синтаксически metadata валидна. Все fields читаются. Handler flags нет. На первый взгляд файл не выглядит сломанным. Но если исключение произойдет в body этой функции, virtual unwind прибавит к RSP меньше, чем нужно. Return address будет искаться со смещением на 20h ниже правильного места.

```
Observed prolog:
    push rbx
    sub  rsp, 60h

Declared unwind effects:
    save RBX by push
    allocate 40h bytes

Mismatch:
    observed allocation 60h != declared allocation 40h
```

static должен вывести не просто “allocation mismatch”. Хороший вывод должен объяснить последствие: caller RSP may be reconstructed incorrectly; return address lookup may be shifted; stack walk may become unreliable for exceptions in function body. Severity высокая, если функция nonleaf, имеет handler, часто вызывается или находится в suspicious region. Confidence высокая, если дизассемблирование пролога надежно и нет __chkstk-like паттерна.

dynamic может попробовать подтвердить это через runtime exception inside the function. Если handler hit или first-chance exception возникает в этой функции, можно сравнить stack trace, RSP before/after virtual unwind, фактический route. Если stack walk ломается именно на этом frame, finding

усиливается. Если запуск не покрывает функцию, finding остается статическим.

8.3.19. Пример второй: metadata заявляет сохраненный RBX, которого нет в коде

Второй сценарий мягче, но тоже важен. UNWIND_INFO содержит UWOP_PUSH_NONVOL RBX, но в прологе нет push rbx и нет save RBX to stack. Возможно, анализатор ошибся и начал читать функцию не с правильного BeginAddress. Возможно, код нестандартный и сохраняет RBX позже, что уже само по себе проблемно для unwind rules. Возможно, metadata специально ложная. В любом случае это не надо скрывать.

```
Observed prolog:
    sub  rsp, 20h
    mov  eax, ecx

Declared unwind effects:
    save RBX by push
    allocate 20h bytes

Mismatch:
    declared saved RBX has no observed save operation
```

Последствие отличается от allocation mismatch. Здесь RSP может быть восстановлен неправильно, если UWOP_PUSH_NONVOL заставит unwinder считать, что на стеке лежит сохраненный RBX и что RSP надо дополнительно сдвинуть. Кроме того, caller context может получить RBX из случайного stack slot. Если caller потом использует RBX, программа после exception handling может вести себя странно.

static должен показать адрес функции, declared operation, ожидаемый register, observed prolog fragment и уровень уверенности. dynamic , если поймает handler route через эту функцию, может зафиксировать nonvolatile register values до и после exception handling. Это уже более продвинутая динамическая проверка, но именно такие данные превращают анализ из “подозреваем” в “видим эффект”.

8.3.20. Пример третий: handler route есть в metadata, но runtime уходит другим путем

Третий сценарий связывает mismatch с control flow. static видит EHANDLER и handler RVA. Он распознает __C_specific_handler, находит language-specific data, строит предположение о protected range и handler block. Статическая карта говорит: исключение внутри диапазона A должно привести к filter B и затем к handler block C, если filter вернет EXCEPTION_EXECUTE_HANDLER.

dynamic запускает target, ловит first-chance exception внутри A, видит hit на __C_specific_handler, но после обработки RIP оказывается не в C, а в другом блоке D. Что это значит? Не обязательно, что static ошибся. Возможно,

filter вернул CONTINUE_EXECUTION и изменил context. Возможно, сработал VEH раньше frame-based handler-a. Возможно, dynamic function table перекрыл статическую карту. Возможно, handler block C был не тем, что мы решили по статике.

Такой случай учит главному: static map и dynamic route надо сопоставлять, а не подменять друг другом. static говорит, что записано в metadata. dynamic говорит, что произошло в конкретном запуске. Различие между ними само становится evidence. Иногда это evidence нормального раннего VEH. Иногда - anti-debug. Иногда - exception-based control flow. Иногда - ошибка нашей модели.

В отчете такой случай должен быть представлен как discrepancy, расхождение. Например: “Static SEH scope suggests handler target 0x..., but dynamic validation observed post-handler RIP 0x... after first-chance exception 0x...; possible VEH intervention or context modification”. Такой finding оставляет место для объяснения и не выдает гипотезу за факт.

8.3.21. Как писать findings так, чтобы они были полезны человеку

Плохой finding звучит так: “bad unwind”. Он короткий, эмоциональный и почти бесполезный. Человек не понимает, что именно плохо, где смотреть, чем это грозит и можно ли доверять выводу. Хороший finding должен иметь структуру. Он должен назвать слой, факт, доказательство, последствие и рекомендацию.

Слой - это место модели: .pdata, .xdata, prolog, epilog, handler data, dynamic route. Факт - конкретное наблюдение: declared allocation 0x40, observed allocation 0x60. Доказательство - адреса и байты/инструкции: function RVA, UNWIND_INFO RVA, prolog fragment. Последствие - что может сломаться: caller RSP, return address, nonvolatile restore, stack walk, handler dispatch. Рекомендация - что делать: manual review, dynamic validation, breakpoint on handler, compare runtime function table.

Для нашего проекта такая структура важна еще и потому, что report может уходить в JSON, HTML, CSV, Ghidra. В консоли можно показать краткую форму. В HTML - подробную. В Ghidra - комментарий рядом с функцией. Но внутренний смысл должен быть одинаковым: не “красный флаг ради красного флага”, а объяснимая evidence chain.

```
Полезная форма finding:

Title:
    Prolog/unwind stack allocation mismatch

Evidence:
    Function RVA 0x1400
    UNWIND_INFO RVA 0x5200
    Observed: sub rsp, 60h
```

```
    Declared: UWOP_ALLOC_* = 40h

Consequence:
    Virtual unwind may reconstruct caller RSP incorrectly.

Recommendation:
    Validate dynamically if function is reachable; inspect prolog manually.
```

Severity и confidence не должны быть одним и тем же. Severity - насколько серьезны последствия, если finding реален. Confidence - насколько мы уверены, что finding реален. Allocation mismatch может иметь высокую severity, но среднюю confidence, если пролог нестандартный и анализатор не уверен. Parse failure может иметь высокую confidence, но severity зависит от того, используется ли функция и есть ли dynamic replacement. Это разделение делает отчет честнее.

8.3.22. Где новичок чаще всего ошибается

Первая ошибка - доверять .xdata без сверки. Если структура читается, новичок считает, что все хорошо. Но мы уже увидели: синтаксис и семантика разные. Вторая ошибка - считать любое отличие злонамеренным. Оптимизация, compiler variation и hand-written assembly существуют. Третья ошибка - искать точное текстовое совпадение инструкций и unwind codes, хотя формат описывает эффекты, а не строки дизассемблера.

Четвертая ошибка - забывать, что push меняет RSP. Если анализатор считает только sub rsp, он неверно построит stack delta. Пятая ошибка - путать handler RVA с пользовательским __except block. Для MSVC SEH handler RVA часто ведет к __C_specific_handler, а реальная логика лежит в language-specific data. Шестая ошибка - игнорировать epilog rules, потому что “функция же возвращается”. Возвращаться CPU может, но unwindable epilog требует распознаваемой формы.

Седьмая ошибка - думать, что dynamic validation отменяет static analysis. dynamic показывает конкретный execution. Если он ничего не увидел, это может означать не отсутствие проблемы, а отсутствие покрытия. Восьмая ошибка - думать, что static finding без runtime hit бесполезен. Наоборот, static finding часто показывает путь, куда надо направить динамическую проверку.

Девятая ошибка - не различать file image и memory image. Packed или manually mapped code может иметь одну картину на диске и другую в памяти. static честно анализирует файл. dynamic честно наблюдает выполнение. Разница между ними - не баг архитектуры, а один из главных источников информации.

8.3.23. Минимальная модель проверки для будущего static -a

Если сжать весь раздел до инженерной модели, static должен пройти несколько уровней. Первый уровень - структурный: прочитать .pdata и .xdata безопасно. Второй уровень - синтаксический: проверить поля UNWIND_INFO, размеры, slots, padding, flags, хвосты. Третий уровень - семантический: сверить declared unwind effects с observed prolog effects. Четвертый уровень - контекстный: учесть section permissions, compiler patterns, handler dominance, reuse, dynamic function table hints. Пятый уровень - отчетный: сформулировать finding с evidence, consequence, confidence.

Эта модель не требует сразу идеального дизассемблера. Можно развивать ее постепенно. Сначала проверять очевидные prolog patterns: push nonvolatile, sub rsp, mov save, lea frame register. Потом добавлять __chkstk. Потом epilog recognition. Потом chained info. Потом compiler-specific patterns. Важно, чтобы архитектура не была тупиковой. Нужно строить не “набор if-ов”, а систему, которая расширяет effect model.

Effect model - центральная идея. Она позволяет отделить raw instruction от смысла для unwind. Когда появится новый pattern, мы добавим способ распознать его effect. Сравнение declared vs observed останется тем же. Это делает static устойчивее и понятнее.

В конце книги, когда мы будем описывать архитектуру проекта целиком, этот раздел станет основой для блока unwind_integrity и prolog_metadata_consistency. Там можно будет показать, какие структуры данных нужны, какие поля отчета полезны, как связать static function record с dynamic validation record, как формировать HTML/JSON/Ghidra output.

8.3.24. Минимальная модель проверки для dynamic -а

Для dynamic -а этот раздел дает другой набор задач. Если static нашел mismatch, dynamic должен по возможности ответить на вопрос: проявляется ли он в живом выполнении? Для этого нужно знать, достигается ли функция, возникает ли exception внутри нее, какие debug events приходят, какой first-chance/second-chance status, какой handler hit, какой RIP trace после события, менялись ли dynamic function tables.

dynamic не обязан сам понимать весь UNWIND_INFO так глубоко, как static. Но он должен уметь привязать runtime event к статической карте. Если exception RIP попал в диапазон RuntimeFunction X, dynamic должен сказать: это та функция, для которой static нашел finding Y. Если handler hit произошел по адресу H, dynamic должен связать его с handler RVA или dynamic handler registration. Если RIP после handler-а ушел в неожиданный блок, dynamic должен записать это как dynamic evidence.

Так появляется настоящая связка: static строит карту и гипотезы, dynamic проверяет маршруты. Один без другого слабее. Только static может переоценить статическую странность. Только dynamic может не увидеть неисполненный участок. Вместе они дают реверсеру не лозунг, а

объяснение: “в файле заявлено это; в коде видно это; при выполнении произошло это”.

8.3.25. Почему shadow space и stack alignment нельзя путать с локальным стеком

Чтобы сверять пролог правильно, надо понимать еще две вещи из Windows x64 calling convention: shadow space и stack alignment. Shadow space - это область в 32 байта, которую caller резервирует перед вызовом для первых четырех register arguments. Ее иногда называют home space. Смысл в том, что даже если аргументы передаются через RCX, RDX, R8 и R9, callee при необходимости может “положить” их в заранее известные stack slots. Это часть ABI, а не прихоть конкретной функции.

Новичок часто видит sub rsp, 20h или sub rsp, 28h и пытается напрямую связать это с локальными переменными. Но часть stack movement может быть нужна для вызовов других функций, shadow space и выравнивания. В Windows x64 перед call стек должен быть выровнен определенным образом. Alignment - выравнивание. Это требование нужно для эффективной и корректной работы инструкций, ABI и некоторых SIMD-операций. Поэтому allocation в прологе часто включает не только “локальные переменные”, но и служебную подготовку для будущих call.

Для unwind metadata это различие не так важно на уровне “зачем выделили байты”. Unwinder не спрашивает, это локальная переменная или shadow space. Ему важно, сколько RSP отняли и как это отменить. Но для реверсера и static -а важно не делать ложные выводы. Если функция выделила 0x28 байт, это не обязательно значит “у нее локальный буфер 0x28”. Это может быть 0x20 shadow space плюс 0x8 для alignment.

Mismatch по allocation нужно поэтому описывать в терминах фактического stack delta, а не бытового объяснения “локальные переменные не совпали”. Правильная формулировка: observed RSP delta differs from declared unwind allocation. Неправильная: “compiler allocated wrong locals”. Мы не знаем мотив компилятора, но можем проверить договор с unwinder-ом.

dynamic в этой части обычно не различает shadow space и locals. Он видит runtime RSP, call boundaries, exception events и stack walk. Но если RIP trace показывает, что после exception route выполнение продолжилось в caller-е с корректным RSP, это косвенно подтверждает, что stack delta был восстановлен правильно хотя бы в покрытом маршруте.

8.3.26. Почему leaf-функция может выглядеть как mismatch, если забыть про особый случай

Leaf function - листовая функция. Листом она называется по аналогии с деревом вызовов: она не вызывает другие функции и обычно не изменяет

nonvolatile-регистры. В Windows x64 leaf-функции могут не иметь отдельной unwind metadata, потому что их можно “размотать” проще: если функция не строила сложный кадр, не сохраняла nonvolatile-регистры и не меняла RSP нестандартно, caller context можно получить как обычный return.

Проблема в том, что граница между leaf и nonleaf для новичка не всегда очевидна. Функция может быть очень маленькой, но все равно nonleaf, если она вызывает другую функцию. Функция может не делать call, но менять RSP для временного буфера. Функция может не выглядеть сложной, но использовать nonvolatile-регистр. Поэтому “нет .pdata записи” не равняется “все нормально”. Надо смотреть на реальные свойства кода.

Связь с mismatch такая: если static ошибочно считает leaf-функцию обычной nonleaf-функцией, он может начать требовать metadata там, где ее отсутствие нормально. И наоборот, если он считает функцию leaf только потому, что она маленькая, он может пропустить реальную проблему. Поэтому раздел про отсутствие metadata будет отдельным, но уже сейчас важно понимать: сверка кода и metadata начинается с вопроса, должна ли эта функция вообще иметь unwind-запись.

Для нашего текущего раздела это важно как защита от ложных тревог. Если .pdata не покрывает маленький thunk, это может быть нормально. Если .pdata покрывает функцию и .xdata говорит одно, а код другое - это уже другая категория. Нельзя смешивать “нет записи” и “запись есть, но врет”.

8.3.27. Tail call и jmp-эпилог: когда функция уходит не через ret

Tail call - хвостовой вызов. Это ситуация, когда функция в конце не возвращается сама через ret, а передает управление другой функции через jmp. В обычном исходном коде это может выглядеть как return other_function(args). Компилятор может оптимизировать это так: текущая функция уже не нужна, ее кадр можно разобрать, а затем прыгнуть в другую функцию. Для CPU это не ret, но для ABI это допустимый способ завершить текущий кадр, если соблюдены правила.

Почему это важно для epilog mismatch? Потому что реверсер может увидеть jmp и подумать: “это не эпилог”. Но в x64 unwind rules определенные формы jmp могут быть допустимым epilog ending. Снова нельзя судить по одному слову инструкции. Нужно понимать форму, контекст и правила распознавания epilogue. Если функция восстановила RSP и nonvolatile-регистры, а потом делает допустимый tail jmp, это может быть нормальным эпилогом.

Но tail call может быть и частью обфускации. Если metadata говорит о нормальном эпилоге, а код после восстановления кадра прыгает в необычный writable/executable block, это уже интересно. Если при этом handler route или exception-based path ведет к этому jmp, dynamic может показать, куда реально

ушел RIP. Здесь static и dynamic снова соединяются: static видит подозрительный epilog/tail pattern, dynamic подтверждает фактическое направление перехода.

В findings tail call надо называть аккуратно. Не “ret missing” как ошибка сама по себе. Лучше: “function epilog appears to end with tail jump; verify whether target and instruction form are valid for x64 epilog recognition”. Если target необычный, добавить evidence про section permissions, module boundary или runtime RIP trace.

8.3.28. Hotpatch, instrumentation и служебные вставки в начале функции

Еще одна причина, почему сверка пролога не должна быть тупой, - hotpatch и instrumentation. Hotpatch - возможность исправить код функции патчем без полной пересборки или остановки системы. Instrumentation - вставка дополнительного кода для наблюдения, профилирования, защиты, логирования или диагностики. Оба механизма могут добавлять служебные инструкции около начала функции.

Например, в начале функции могут появиться nop-ы, mov edi, edi в старых x86-паттернах, alignment nop sequences, jumps to instrumentation thunks, security checks, stack cookie initialization. Не все это относится к unwind. Некоторые вставки находятся до настоящего пролога, некоторые внутри, некоторые после построения кадра. Поэтому вопрос “где начинается реальный пролог” может стать нетривиальным.

Windows x64 unwind metadata обычно привязана к function begin из RuntimeFunction. Если перед настоящим прологом есть служебная вставка, metadata должна все равно корректно описывать те эффекты, которые важны для unwind. static должен уметь отличать harmless prefix instructions от unwind-relevant instructions. Harmless здесь значит не “бесполезные”, а “не меняющие модель RSP/nonvolatile/frame”.

Если анализатор не учитывает такие вставки, он может ошибочно сказать, что SizeOfProlog слишком большой или что CodeOffset не совпадает. Поэтому confidence для таких findings должен учитывать наличие известных prefix patterns. В будущем проектной архитектуре это можно оформить как pattern database: набор известных compiler/instrumentation шаблонов, которые снижают ложные срабатывания, но не скрывают evidence.

8.3.29. Security cookie и почему не всякий код пролога относится к unwind

Security cookie - защитное значение, которое компилятор может использовать для обнаружения повреждения стека, например stack buffer overflow. В MSVC это часто связано с механизмом /GS. Функция может

записывать cookie в свой кадр, а перед возвратом проверять, не изменился ли он. Это важная защитная логика, но она не всегда напрямую отражается в unwind codes.

Почему? Потому что unwind metadata описывает то, что нужно для восстановления caller context: RSP, saved nonvolatile registers, frame register, handler data. Запись cookie в локальный stack slot не обязательно нужна для virtual unwind. Поэтому код пролога может содержать mov с cookie, xor с RSP, обращение к __security_cookie, но отсутствие соответствующего UNWIND_CODE не является mismatch.

Это хороший пример различия между “пролог как человеческое начало функции” и “unwind-relevant prolog effects”. Для реверсера security cookie важен, потому что он говорит о защитах компилятора и структуре локального кадра. Для unwinder-а он может быть не важен. static должен поэтому не требовать unwind code на каждую инструкцию в начале функции.

dynamic может видеть security cookie только косвенно, если нарушение cookie приводит к fail-fast или другому exception-like пути. Но в обычной сверке .xdata это не главная тема. Главное - не принять нормальную защитную вставку за metadata mismatch.

8.3.30. Почему некоторые mismatch лучше считать “нужна ручная проверка”, а не “ошибка”

В инженерном инструменте очень важно иметь промежуточное состояние. Не все должно быть normal или suspicious. Иногда правильный вывод - manual review. Manual review - ручная проверка. Это не отказ анализатора работать, а честное признание, что данных недостаточно для автоматического уверенного вывода.

Например, hand-written assembly может быть корректным, но выглядеть необычно для compiler pattern. Или packed binary может иметь временную metadata на диске, которая заменяется в памяти. Или prolog содержит indirect jump в shared prolog stub. Или function range из .pdata начинается не там, где дизассемблер автоматически поставил function start. В таких случаях автоматический high-severity finding может быть ложным.

Но manual review тоже должен быть полезным. Нельзя просто написать “manual review”. Нужно объяснить, что проверять: function begin, prolog boundaries, RSP delta, saved registers, handler flags, dynamic function table registration, runtime route. Тогда человек не остается один на один с байтами.

В будущем HTML report это можно сделать особенно хорошо: рядом с finding показать mini checklist. Например: “1) verify function start in disassembler; 2) inspect first 32 bytes; 3) compare RSP delta; 4) run with --dynamic if reachable; 5) check handler hits”. Такой подход превращает анализатор в учебный и рабочий инструмент одновременно.

8.3.31. Детальный ручной разбор мини-функции

Теперь сделаем маленький ручной разбор, чтобы связать все термины. Пусть RuntimeFunction говорит: BeginAddress = 0x1000, EndAddress = 0x1060, UnwindInfoAddress = 0x3000. Это значит, что для любого RIP в диапазоне [0x1000, 0x1060) Windows будет использовать UNWIND_INFO по RVA 0x3000. static переводит RVA 0x1000 в file offset и дизассемблирует начало функции.

```
Код по RVA 0x1000:

1000: push rbx
1002: push rdi
1003: sub  rsp, 30h
1007: mov  rbx, rcx
100A: call qword ptr [rip+...]
100F: test eax, eax
1011: je   1040h
...
```

Что мы видим? push rbx сохраняет RBX и меняет RSP на 8. push rdi сохраняет RDI и еще раз меняет RSP на 8. sub rsp, 30h выделяет 48 байт. После этого mov rbx, rcx уже использует RBX, но не является отдельным unwind effect: старый RBX уже сохранен, а новое значение RBX - рабочее значение callee. Call внутри функции делает ее nonleaf. Значит unwind metadata нужна.

Теперь declared model из UNWIND_INFO должна сказать примерно: save RBX by push, save RDI by push, allocate 0x30 bytes. Если она говорит только allocate 0x30, но молчит про RBX/RDI, восстановление nonvolatile-регистров будет неполным. Если она говорит push RBX/RDI, но allocation 0x20, RSP будет восстановлен неверно. Если она говорит frame register RBP, но RBP в коде не участвует, это отдельный frame register mismatch.

Такой пример полезен тем, что показывает: сверка не требует сначала понимать всю бизнес-логику функции. Нам не нужно знать, что делает call, что означает test eax, eax, зачем branch в 0x1040. Для unwind consistency достаточно понять первые эффекты, которые построили кадр. Это делает задачу static -а ограниченной и реалистичной.

dynamic для этой функции мог бы поставить breakpoint на 0x100A или дождаться exception внутри диапазона. Если exception случится после 0x1003, весь пролог уже выполнен. Тогда virtual unwind должен восстановить caller context, добавив 0x30, отменив два push и взяв return address. Если runtime stack walk показывает неверный caller, это может подтверждать mismatch.

8.3.32. Детальный ручной разбор функции с frame pointer

Теперь другой пример - функция с frame pointer. Она сохраняет RBP, выделяет стек и устанавливает RBP как стабильную базу. Это часто делает код более удобным для обращения к локальным переменным, особенно если RSP может меняться или если нужна стабильная точка отсчета.

Код:

```
2000: push rbp
2001: push rbx
2003: sub  rsp, 80h
200A: lea  rbp, [rsp+40h]
200F: mov  [rbp+20h], rcx
...
```

Здесь RBP не просто сохранен как nonvolatile-регистр. Он становится frame register. Старое значение RBP сохранено через push rbp, а новое RBP устанавливается через lea rbp, [rsp+40h]. Поле FrameRegister в UNWIND_INFO должно отразить, что RBP используется как регистр кадра, а FrameOffset должен объяснить его смещение относительно RSP после allocation.

Почему это важно? Потому что language-specific handler, scope table, local variable recovery и некоторые debugger views могут использовать establisher frame. Если frame register metadata неверна, обработчик может получить неправильную базу кадра. Это уже не просто “stack trace некрасивый”. Это может повлиять на обработку исключения.

static должен поэтому различать два факта: RBP saved и RBP established as frame register. Первое описывается сохранением nonvolatile-регистра. Второе - полями FrameRegister/FrameOffset и UWOP_SET_FPREG. Если есть только одно из двух, надо понять, соответствует ли это коду. Сохранить RBP можно без использования его как frame pointer. Использовать RBP как frame pointer без корректного сохранения старого RBP - уже нарушение ABI expectations.

dynamic может подтвердить проблему, если exception route входит в SEH/C++ handler и параметры handler-a или post-handler route выглядят странно. Но в простом запуске frame mismatch может не проявиться. Поэтому static должен давать ясное объяснение, почему finding важен даже без runtime crash.

8.3.33. Детальный ручной разбор handler-related mismatch

Рассмотрим функцию, у которой UNWIND_INFO содержит EHANDLER. Это значит, что после unwind codes и padding находится handler RVA, а после него - language-specific data. Допустим, handler RVA указывает на __C_specific_handler. Для MSVC SEH это нормально. Но дальше scope table говорит, что protected range начинается до BeginAddress функции или handler target уходит в writable section. Это уже не обычная compiler variation.

Здесь mismatch не между push/sub и UWOP, а между semantic boundaries. Protected range должен иметь смысл относительно функции. Filter и handler block должны указывать на исполняемый код, который соответствует ожидаемой модели. Если scope table указывает за пределы image или в середину данных, это сильный сигнал. Если target вне функции, но внутри

runtime helper, это может быть нормальным. Значит анализ требует контекста.

static должен представить это как отдельный класс: handler data consistency. Не надо смешивать с prolog/unwind mismatch. Пользователь отчета должен понимать: кадр может быть описан правильно, но exception routing data выглядит странно. Или наоборот: handler data нормальная, но frame restoration broken. Это разные проблемы, и у них разные последствия.

dynamic здесь дает наиболее понятное подтверждение: breakpoint на __C_specific_handler, breakpoint на target block, запись exception code и RIP trace. Если статический target никогда не достигается, но другой handler перехватывает исключение раньше, нужно записать VEH intervention possibility. Если target достигается, но затем RIP уходит по измененному ContextRecord, это уже динамический маршрут, который статическая CFG могла не показать.

8.3.34. Как это должно выглядеть в Ghidra и x64dbg для живого реверсера

Книга не должна оставаться только на уровне структур. Реверсер обычно смотрит в Ghidra, IDA, x64dbg или WinDbg. Поэтому надо представить, как mismatch выглядит глазами человека. В Ghidra функция может иметь нормальный pseudocode, но комментарий static -а рядом с function start сообщает: “declared allocation differs from observed prolog”. Человек сразу идет в первые инструкции и сравнивает sub rsp с unwind info.

В x64dbg человек может поставить breakpoint внутри функции, вызвать путь, посмотреть RSP, return address на стеке, увидеть exception event. Если dynamic собирает trace автоматически, он делает то же самое системно: фиксирует RIP, RSP, exception code, handler hits, after-handler RIP. Разница в том, что человек делает это вручную для одного случая, а dynamic может собрать structured evidence для отчета.

В WinDbg stack walking особенно чувствителен к unwind metadata. Если .xdata неверна, команда stack trace может оборваться или показать странный caller. Это не всегда значит, что программа испортила стек. Иногда стек нормален, а metadata лжет. Иногда metadata нормальна, а стек поврежден. Различить эти случаи можно только сопоставляя код, metadata и runtime state.

Поэтому в книге дальше важно удерживать три экрана одновременно. Первый экран - PE/metadata, то есть static . Второй экран - disassembly, то есть реальный код функции. Третий экран - debugger events, то есть dynamic /dynamic validation. Хороший реверсер учится не жить в одном экране, а переключаться между ними.

8.3.35. Почему этот раздел нужен перед следующими аномалиями

Теперь должно быть понятно, почему мы так долго остановились на mismatch. Это не частная проверка “совпадает ли sub rsp”. Это центральный мост между статическим PE-анализом и живой обработкой исключений. Если код и metadata не согласованы, ломается доверие к stack walking, handler routing, crash analysis, debugger output и выводам декомпилятора.

Также этот раздел готовит нас к более сложным темам. Dynamic function tables будут понятнее, потому что мы уже знаем: runtime может принести новую metadata, которая заменит или дополнит файл. Leaf functions будут понятнее, потому что мы уже отличаем отсутствие metadata от ложной metadata. Anti-debug через exceptions будет понятнее, потому что мы уже видим, как handler route может стать реальным control flow. Architecture chapter проекта будет понятнее, потому что static и dynamic теперь имеют четкие роли в одной evidence chain.

8.3.25. Переход к следующему разделу

Теперь мы умеем говорить о mismatch между кодом и unwind metadata не как о грубом “сломано”, а как о связке нескольких слоев. Мы понимаем, что пролог создает кадр, UNWIND_INFO описывает эффекты пролога, RtlVirtualUnwind использует это описание для восстановления caller context, а exception handlers зависят от правильной карты функций и кадров. Мы также понимаем, что static должен строить статическую evidence chain, а dynamic - проверять живой маршрут выполнения.

Следующий шаг естественно вытекает из этого раздела. Если metadata может не совпадать с кодом, то отсутствие metadata тоже надо понимать аккуратно. Не каждая функция обязана иметь отдельную unwind-запись. Leaf-функции, thunks, padding, короткие wrappers и некоторые compiler artifacts могут выглядеть как “дырки” в .pdata, но быть нормальными. А иногда отсутствие записи - сильный сигнал: исполняемый nonleaf-код выпадает из unwind-карты. Поэтому дальше нужно разобрать leaf-функции и ложные тревоги: когда отсутствие unwind-записи нормально, а когда оно говорит о проблеме.

8.4. Leaf-функции и ложные тревоги: почему отсутствие .pdata не всегда поломка

В предыдущих разделах мы много раз смотрели на `.pdata` как на карту функций. Это полезная модель, но у неё есть опасная сторона: когда человек впервые привыкает искать функцию через `RUNTIME_FUNCTION`, он начинает воспринимать отсутствие записи как прямое доказательство поломки. Видит адрес в `.text`, не находит для него диапазон в `.pdata` и сразу делает вывод: “здесь что-то скрывают”, “unwind сломан”, “packer”, “обфускация”. Иногда такой вывод действительно будет правильным. Но очень часто он будет ложной тревогой.

Именно поэтому этот раздел нужен не как очередной список признаков. Нам нужно медленно разобрать саму идею: почему Windows x64 вообще допускает код без записи в `.pdata`, при каких условиях это безопасно, где проходит граница между нормальной минимальной функцией и настоящим повреждением unwind-карты, и как static не должен наказывать бинарь за нормальное поведение компилятора. Для dynamic-а это тоже важно: динамическая проверка должна показывать реальный маршрут выполнения, но она не должна превращать любое отсутствие metadata в “подтверждённую угрозу”.

Главная мысль раздела такая: `.pdata` нужна не для того, чтобы перечислить все байты кода в программе. Она нужна там, где Windows не может восстановить вызывающий контекст простым способом. Если простой способ достаточно надёжен, отдельная запись может быть не нужна. Отсюда и начинается тема leaf-функций.

8.4.1. Сначала разберём слово “leaf” без магии

Английское слово `leaf` переводится как “лист”. В программировании его часто используют по аналогии с деревом вызовов. Если представить программу как дерево, где одна функция вызывает другую, потом третью, потом четвёртую, то функция на самом конце ветки похожа на лист: от неё уже не растут новые вызовы. Поэтому новичок обычно запоминает: leaf function - это функция, которая никого не вызывает.

Такое объяснение полезно как первая картинка, но для Windows x64 оно неполное. В ABI, то есть в application binary interface - двоичном соглашении между компилятором, операционной системой, отладчиком и рантаймом, leaf-функция важна не сама по себе как “функция без вызовов”. Она важна потому, что её можно безопасно “раскрутить” без внешней инструкции.

Здесь надо остановиться. “Раскрутить” - это наш привычный unwind: восстановить состояние вызывающей функции. Когда мы говорим, что Windows должна сделать unwind, мы не говорим “просто красиво показать stack trace”. Мы говорим о более глубокой операции: есть текущий RIP, текущий RSP, набор регистров и память стека. Из этого состояния нужно получить состояние caller-а - той функции, которая вызвала текущую.

Если текущая функция построила свой кадр, сдвинула RSP, сохранила RBX, RBP, R12 или другие nonvolatile-регистры, то Windows должна знать, где теперь лежат старые значения и насколько нужно сдвинуть стек обратно. Для этого и существует `.xdata`: она описывает эффекты пролога. Но если функция ничего такого не делала, ситуация становится намного проще.

Leaf-функция в практическом смысле Windows x64 - это функция, для которой восстановление caller-а можно выполнить как обычное возвращение: взять адрес возврата с вершины стека и представить, что функция сделала `ret`. В этом случае отдельная запись `.pdata` может быть не нужна.

8.4.2. Почему Windows не требует metadata для абсолютно каждого куска кода

Представим самую маленькую функцию:

```
xor eax, eax
ret
```

Она не выделяет локальные переменные на стеке. Она не делает `sub rsp, ...`. Она не сохраняет RBX, RBP, R12-R15. Она не вызывает другую функцию. Она просто кладёт результат в EAX и возвращается. Что находится на вершине стека, когда эта функция выполняется? Там лежит `return address` - адрес возврата, который инструкция `call` положила туда перед передачей управления.

Теперь допустим, что внутри такой функции случилось исключение. Например, не в этой конкретной, а в похожей маленькой функции была инструкция чтения памяти, и она упала на access violation. Windows смотрит на текущий RIP и пытается найти для него `RUNTIME_FUNCTION`. Не находит. Что можно сделать? Если предположить, что функция leaf, то достаточно прочитать `[RSP]` как адрес возврата и увеличить RSP на 8 байт. То есть сделать то, что сделал бы `ret`:

```
caller.RIP = *(uint64_t*)current.RSP
caller.RSP = current.RSP + 8
nonvolatile registers = unchanged
```

Это не настоящая инструкция `ret`, потому что процессор её не выполняет. Это виртуальное восстановление состояния. Но логика та же: если функция не меняла форму стека, то вершина стека всё ещё указывает на адрес возврата. Значит Windows может перейти к анализу caller-а.

Вот почему отсутствие записи в `.pdata` не всегда означает ошибку. Иногда оно означает: “для этого участка кода операционная система может обойтись простой симуляцией возврата”. Это экономит место в PE-файле, уменьшает количество metadata и не заставляет компилятор описывать то, что не требует описания.

Но здесь есть важная оговорка. Эта простая симуляция безопасна только тогда, когда предположение о leaf-функции действительно верно. Если функция на самом деле сдвинула RSP, сохранила регистры, построила кадр, вызывает другие функции или имеет exception-handler, а записи в `.pdata` нет, то Windows будет использовать неправильную модель. Она подумает, что `[RSP]` - это адрес возврата, хотя там может лежать локальная переменная, сохранённый регистр, cookie, временный буфер или вообще случайные данные.

8.4.3. Ложная тревога начинается там, где инструмент путает “нет записи” и “нет функции”

Теперь введём термин `false positive` - ложноположительное срабатывание, или по-человечески ложная тревога. Это ситуация, когда анализатор сообщает о проблеме, хотя реальной проблемы нет. В нашем контексте типичная ложная тревога звучит так: “адрес находится в `.text`, но не покрыт `.pdata`, значит unwind metadata повреждена”.

Почему это неверно? Потому что `.pdata` не обязана покрывать каждый байт `.text`. В `.text` могут лежать полноценные nonleaf-функции, leaf-функции, маленькие переходники, padding между функциями, alignment bytes, импортные thunks, jump stubs, участки hotpatch/instrumentation и другие служебные фрагменты. Некоторые из них являются функциями в человеческом смысле, некоторые - нет. Некоторые участвуют в обычном выполнении, некоторые существуют как техническая прокладка. Одного факта “байт находится в executable section” недостаточно, чтобы требовать от него записи `RUNTIME_FUNCTION`.

Здесь static должен быть осторожным. Его задача не ругаться на every uncovered address. Его задача - понять, похож ли uncovered region на код, для которого unwind metadata действительно необходима. Слово `uncovered` означает “не покрытый диапазоном `.pdata`”. Но uncovered - это ещё не broken. Это только повод спросить: что это за код и нужна ли ему отдельная unwind-запись?

Если инструмент не делает это различие, он становится шумным. А шумный анализатор быстро перестают слушать. В реверсе это особенно опасно: если отчёт выдаёт десятки “подозрительных” мест, которые оказываются обычными leaf-функциями и padding-ом, человек перестаёт доверять даже тем finding-ам, которые действительно важны.

8.4.4. Функция, кадр и return address: повторяем базовую модель

Чтобы не потерять фундамент, ещё раз соберём модель функции. Когда caller вызывает callee, он выполняет инструкцию `call`. Эта инструкция делает две вещи: кладёт адрес следующей инструкции на стек и передаёт управление на начало callee. Этот адрес следующей инструкции и есть `return address` - адрес возврата.

После `call` callee начинает выполняться. Если callee простая, она может вообще не трогать RSP. Тогда стек в течение всей функции выглядит почти так же, как сразу после `call`: на вершине лежит return address. Если callee в конце делает `ret`, процессор снимает этот адрес со стека и продолжает выполнение caller-a.

Но если callee сложная, она строит кадр. “Кадр” - это не обязательно красивая структура с полем имени функции. Это просто участок стека, который функция организовала под свои нужды: локальные переменные, сохранённые nonvolatile-регистры, временные данные, иногда security cookie, иногда выравнивание. Чтобы выделить такой участок, функция обычно меняет RSP, например делает `sub rsp, 0x40`.

После этого return address уже не лежит прямо на вершине текущего RSP. Он где-то выше, по старому адресу. Если в этот момент случится exception и Windows попытается “просто взять `[RSP]` как return address”, она ошибётся. Поэтому для таких функций нужна `.pdata` и `.xdata`: они объясняют, как из текущего RSP вернуться к состоянию caller-a.

Отсюда рождается граница: если функция не изменила форму кадра и не сохранила nonvolatile-состояние, её можно unwind-ить простым return simulation. Если изменила - нужна инструкция для обратной сборки контекста.

8.4.5. Leaf в графе вызовов и leaf в ABI - не одно и то же

Теперь разберём тонкость, из-за которой часто ломается анализ. Есть “leaf” как понятие графа вызовов: функция не вызывает другие функции. А есть “leaf” как понятие unwind-модели Windows x64: функция не делает того, что требует внешней unwind metadata. Эти понятия похожи, но не совпадают один к одному.

Функция может никого не вызывать, но всё равно быть не leaf для unwind. Например:

```
push rbx
sub  rsp, 20h
mov  rbx, rcx
; работа с локальными данными
add  rsp, 20h
pop  rbx
ret
```

Она не делает `call`, но она меняет RSP и сохраняет RBX. RBX - nonvolatile-регистр, то есть по calling convention callee обязан вернуть его caller-у в прежнем состоянии. Если exception случится посередине этой функции, Windows должна знать, где лежит сохранённый RBX и как отменить `sub rsp, 20h`. Значит такая функция нуждается в unwind metadata.

И наоборот, в бинаре может быть маленький jump thunk:

```
jmp qword ptr [rip + target]
```

Он передаёт управление дальше, но не создаёт своего кадра. Он не кладёт новый return address через `call`, не выделяет стек, не сохраняет nonvolatile-регистры. Для реверсера это может выглядеть как отдельный кусок кода, а для unwind-модели это часто просто прозрачная прокладка. Требовать от него полноценной записи `.pdata` было бы странно.

Поэтому static не должен спрашивать только “есть ли у этого куска исходящие вызовы”. Он должен спрашивать глубже: “создаёт ли этот код состояние, которое Windows не сможет восстановить простым return simulation?” Это и есть правильная теоретическая рамка.

8.4.6. Почему вызов другой функции почти всегда меняет ситуацию

В документации Microsoft nonleaf-функции связываются в том числе с вызовом других функций. Это не случайная формальность. На Windows x64 caller перед вызовом должен подготовить стек: обеспечить shadow space, сохранить нужное выравнивание, иногда разместить дополнительные аргументы. На практике это часто означает `sub rsp, ...` в прологе или перед вызовом.

`shadow space` - это 32 байта домашнего места для первых четырёх аргументов. Даже если аргументы передаются в RCX, RDX, R8, R9, ABI требует, чтобы caller зарезервировал на стеке место, куда callee при необходимости сможет их сохранить. Новичку это кажется странным: “зачем место на стеке, если аргументы в регистрах?” Но это часть соглашения. Она упрощает varargs, отладку, прологи и некоторые compiler/runtime сценарии.

Если функция делает обычный `call`, она почти всегда не является простым leaf-случаем. Почему “почти”? Потому что бывают оптимизированные jump wrappers, tail calls и специальные ручные ассемблерные фрагменты. Но в нормальном C/C++ коде вызов другой функции обычно означает: стек был подготовлен, RSP менялся, и для корректного unwind нужна metadata.

Важно не заучить “call = обязательно suspicious без `.pdata`”, а понять причинную цепочку. Если вызов сопровождается изменением стека, сохранением nonvolatile-регистров или exception-handling состоянием, отсутствие `.pdata` опасно. Если перед нами не настоящий call frame, а tail jump или thunk, отсутствие metadata может быть нормой.

8.4.7. Padding, alignment и байты, которые вообще не являются функцией

Одна из самых частых причин ложных тревог - disassembler создаёт функцию там, где человек или компилятор не имели в виду функцию. Между настоящими функциями могут лежать байты выравнивания: `nop`, `int3`, повторяющиеся паттерны, пустое место, служебные вставки. Это называется padding - набивка, технический промежуток.

Зачем он нужен? Процессоры любят выравнивание. Компилятор и linker могут располагать функции по удобным границам: 16 байт, 32 байта и так далее. Иногда это улучшает декодирование инструкций, cache behavior, hotpatch-возможности или просто соответствует настройкам компоновки. Пустое место между функциями заполняется байтами, которые не должны выполняться как нормальная функция.

Если такой padding лежит в `.text`, он имеет executable права вместе со всей секцией. Но из этого не следует, что он является функцией. А если это не функция, то отсутствие `RUNTIME_FUNCTION` записи не является проблемой. `.pdata` описывает функции, для которых нужен unwind, а не каждый executable byte.

Реверсер должен быть осторожен с автоматическими boundaries. Ghidra, IDA, Binary Ninja, objdump и другие инструменты иногда создают функции эвристически. Они видят reachable address, `ret`, `jmp`, cross-reference или похожую последовательность и решают: “возможно, здесь функция”. Это полезно, но не является доказательством. В теме `.pdata` источник истины другой: если Windows x64 должна unwind-ить nonleaf функцию, она ожидает function table entry. Если перед нами padding, ожидать нечего.

Для `static` -а это означает: uncovered gap внутри `.text` надо сначала классифицировать. Он может быть padding-ом, leaf-кодом, thunk-ом, данными внутри code section, результатом оптимизации, а может быть настоящей missing unwind metadata. Finding должен отражать эту неопределённость.

8.4.8. Thunk, stub, wrapper: маленькие переходники без собственного кадра

Теперь введём несколько терминов, которые часто встречаются рядом с ложными тревогами. Thunk - переходник. Это маленький фрагмент кода, который принимает управление и почти сразу передаёт его дальше. Он может адаптировать адрес, перейти через import table, прыгнуть на real implementation, переключить calling layer или служить частью hotpatch/instrumentation механики. `Stub` - заглушка или короткий служебный фрагмент. `Wrapper` - обёртка, код вокруг другого кода. Эти слова пересекаются, и в реальных разговорах их часто используют нестрого.

Для нас важен не словарь сам по себе, а unwind-смысл. Маленький переходник может не создавать собственного кадра. Например:

```
jmp real_target
```

Такой код не вызывает функцию через `call`. Он не кладёт новый return address. Он просто передаёт управление. Если позже `real_target` сделает `ret`, он вернётся не в thunk, а к тому caller-у, который изначально пришёл в thunk. Значит thunk не стал отдельным stack frame. Для unwind-модели он может быть прозрачным.

Другой пример - import thunk, связанный с импортируемой функцией из DLL. В PE вокруг импортов могут быть маленькие прыжки через IAT. Они находятся в executable section, на них есть cross-references, но требовать от каждого такого переходника полноценной `.pdata` записи часто неправильно.

Однако нельзя впадать в обратную крайность. Не всякий маленький кусок кода безопасен. Если stub сохраняет регистр, меняет стек, строит trampoline с собственным return path, выполняет instrumentation callback или участвует в exception-based control flow, он может перестать быть прозрачным. Поэтому static должен смотреть не на размер фрагмента, а на его эффект: меняет ли он unwind-состояние?

8.4.9. Tail call: когда функция “уходит” через jmp, а не возвращается через ret

Теперь важная тема для dynamic -а: куда уходит функция. В обычной модели функция заканчивается `ret`: она берёт return address со стека и возвращает управление caller-у. Но оптимизатор может заменить последний вызов на `tail call` - хвостовой вызов. Вместо `call target` и последующего `ret` он делает `jmp target`.

Почему это возможно? Если текущая функция уже закончила свою работу, восстановила стек и регистры, и дальше ей нужно только вернуть результат другой функции, нет смысла создавать дополнительный кадр. Она может передать управление напрямую. Тогда target вернётся сразу к исходному caller-у. Текущая функция как бы исчезает из цепочки вызовов.

Для реверсера это может выглядеть неожиданно. Он видит функцию, которая не заканчивается `ret`, а уходит в `jmp`. Если он ожидает только классическую форму, он может подумать, что control flow странный. Но tail call - нормальная оптимизация. В x64 unwind-модели эпилог тоже может завершаться допустимой формой `jmp`, если соблюдены правила эпилога.

Теперь связь с `.pdata`. Если функция имела полноценный пролог, выделяла стек и сохраняла nonvolatile-регистры, её epilog/tail jump должен быть понятен unwinder-у. Значит metadata нужна. Но если перед нами чистый jump thunk без кадра, отдельная metadata может быть не нужна. Разница снова не в слове

jmp, а в том, было ли состояние, которое нужно восстанавливать.

dynamic здесь полезен, потому что он показывает живой маршрут. Static static может сказать: “этот uncovered fragment похож на jump thunk или tail-transfer”. dynamic может проверить, куда реально уходит RIP, появляется ли новый stack frame, возникает ли exception route, hit-ается ли handler, и возвращается ли выполнение в ожидаемую точку.

8.4.10. Настоящая проблема: nonleaf-код без .pdata

Теперь перейдём к ситуации, где тревога действительно может быть оправданной. Представим функцию:

```
push rbx
sub  rsp, 40h
mov  rbx, rcx
call some_api
add  rsp, 40h
pop  rbx
ret
```

Если для такой функции нет RUNTIME_FUNCTION, Windows при exception внутри неё может ошибочно применить leaf-модель. Она попытается взять [RSP] как return address. Но после sub rsp, 40h текущий RSP указывает на локальную область, а не на адрес возврата. На вершине может лежать временное значение, сохранённый регистр или вообще неинициализированная память. В результате caller context будет восстановлен неправильно.

Последствия зависят от момента exception. Иногда получится просто плохой stack trace. Иногда SEH search phase пропустит нужный handler, потому что unwinder не сможет правильно подняться по кадрам. Иногда debugger покажет бессмысленную цепочку вызовов. Иногда crash dump будет почти бесполезен. В совсем плохом случае exception handling станет unreliable: то есть система формально делает то, что может, но её входные данные описывают не тот кадр.

Вот здесь static должен повышать серьёзность finding-а. Не потому, что “нет .pdata”. А потому, что есть статические признаки nonleaf-эффектов: изменение RSP, сохранение nonvolatile-регистров, обычные call, использование frame pointer, обращения к локальным stack slots, возможно security cookie, возможно handler-related code. Если такие признаки есть, а записи нет, это уже не нормальный leaf gap.

Обрати внимание: причина finding-а должна быть сформулирована через эффект. Плохой отчёт скажет: “missing .pdata”. Хороший отчёт скажет: “адресной диапазон выглядит как функция, которая меняет RSP и сохраняет RBX, но не имеет RUNTIME_FUNCTION; если exception произойдёт внутри этого диапазона, Windows может применить leaf-unwind simulation и восстановить неверный caller context”. Это длиннее, но это объясняет суть.

8.4.11. Почему отсутствие handler-а тоже важно

Если для функции нет RUNTIME_FUNCTION, у неё не может быть нормально описанного UNW_FLAG_EHANDLER или UNW_FLAG_UHANDLER в стандартной table-based x64 модели. То есть Windows не найдёт для неё associated language-specific handler через .xdata, потому что нет входа в .pdata, от которого можно перейти к UNWIND_INFO.

Это важно для __try/__except, __finally и C++ cleanup-механики. Если функция содержит protected region, filter, handler block или termination handler, она не может быть "просто leaf без metadata" в нормальной модели. Ей нужен путь: RIP внутри function range -> RUNTIME_FUNCTION -> UNWIND_INFO -> flags -> handler RVA -> language-specific data. Если первый шаг отсутствует, вся цепочка не начинается.

Поэтому static должен особенно насторожиться, если uncovered region выглядит как код с exception-логикой. Например, рядом есть обращения к GetExceptionCode, GetExceptionInformation, необычные handler-like blocks, переходы на cleanup path, вызовы runtime helper-ов, patterns scope table, или если другой анализ показывает protected ranges. Само по себе это ещё не окончательный приговор, но это уже не "обычная маленькая функция без .pdata".

dynamic может проверить это динамически: если exception внутри такого региона реально приводит к handler hit, нужно понять, через какой механизм это произошло. Может быть, это не frame-based SEH, а VEH. Может быть, metadata зарегистрирована динамически через RtlAddFunctionTable. Может быть, код в памяти отличается от файла на диске. А может быть, static static действительно нашёл повреждённую или скрытую unwind-картину.

8.4.12. Dynamic function tables: когда статический файл не знает всей правды

Тема dynamic function tables будет подробно разобрана дальше, но в разделе о ложных тревогах её нельзя полностью игнорировать. Статический static смотрит на PE-файл на диске. Он видит его sections, headers, .pdata, .xdata. Но Windows x64 допускает ситуацию, когда function table для кода предоставляется во время выполнения. Это важно для JIT, runtime-generated code и некоторых loader/dynamic сценариев.

Если код создаётся или перемещается в памяти после загрузки, обычной .pdata в файле может быть недостаточно. Тогда runtime должен зарегистрировать function table через соответствующие механизмы Windows. Для статического static -а это выглядит как "в файле нет записи". Для живого процесса это может быть нормально, если запись появилась динамически.

Здесь появляется чёткая граница между static и dynamic. static говорит: “в PE-файле этот диапазон не покрыт статической .pdata”. dynamic проверяет: “во время выполнения была ли зарегистрирована dynamic function table, покрывающая этот код; куда ушёл RIP; как реально прошёл exception route”.

Именно поэтому в твоём проекте dynamic validation - не украшение, а необходимый второй слой. Без dynamic-а статический отчёт может быть честным, но неполным. Он видит файл. Он не всегда видит runtime-картину.

8.4.13. Как static должен думать об uncovered code

Теперь сформулируем не чек-лист, а правильный образ мышления для static-а. Когда он находит адрес в executable section, который не покрыт .pdata, он не должен сразу писать “bad”. Он должен построить гипотезу.

Первая гипотеза: это не функция. Это padding, alignment, данные внутри .text, trap bytes, unreachable filler. Тогда отсутствие .pdata нормально.

Вторая гипотеза: это leaf-код. Он может выполнять простые операции в volatile-регистрах, читать аргументы, вернуть результат через RAX, сделать ret, но не строить кадр и не сохранять nonvolatile-регистры. Тогда отсутствие .pdata тоже нормально.

Третья гипотеза: это прозрачный переходник - thunk, jump stub, import wrapper, tail-transfer fragment. Он может передавать управление дальше, но не создавать собственного кадра. Здесь отсутствие .pdata может быть нормальным, но нужно смотреть осторожно, потому что некоторые wrappers всё же строят кадр.

Четвёртая гипотеза: это nonleaf-код без metadata. Вот это уже сигнал. Особенно если видны sub rsp, push nonvolatile-регистров, mov [rsp+...] для сохранения состояния, обычные call, обращения к локальным переменным, frame pointer, security cookie, stack probing, или handler-related patterns.

Пятая гипотеза: статическая картина неполна из-за runtime registration, unpacking, manual mapping или JIT-like поведения. Тогда static должен не делать окончательный вывод, а явно передать вопрос dynamic-у: нужна динамическая валидация.

Это и есть зрелый анализ. Не “есть запись / нет записи”, а цепочка гипотез с evidence и confidence.

8.4.14. Как dynamic проверяет leaf-гипотезу

dynamic, в твоём смысле, не заменяет static. Он берёт статическую гипотезу и проверяет её в живом процессе. Для leaf-функций это особенно полезно, потому что главный вопрос звучит так: “если управление реально попадёт в этот uncovered region и там произойдёт exception или переход, будет

ли маршрут восстановления выглядеть разумно?”

Представь, что static нашёл uncovered region и осторожно пометил его как leaf candidate. dynamic может наблюдать, попадает ли туда RIP, какие инструкции выполняются, происходит ли call, меняется ли RSP, появляются ли first-chance exceptions, куда уходит управление после выхода, и не возникает ли handler route, которого статическая карта не объясняет.

Если регион действительно ведёт себя как leaf, динамическая картина будет спокойной: нет регистрации handler-ов для него, нет странного восстановления stack trace, нет unexpected second-chance exception, RSP не превращается в мусор, RIP trace показывает обычный переход назад или прозрачный jump. Тогда finding может быть понижен или вообще оформлен как informational.

Если же dynamic видит, что внутри uncovered region происходит exception, после которого стек разваливается, return address не похож на адрес в вызывающем модуле, handler hit не соответствует статической карте, или RIP уходит в неожиданную область, тогда leaf-гипотеза слабеет. Статический “возможно missing unwind” превращается в более серьёзное динамически подтверждённое наблюдение.

Важно: dynamic тоже не всевидящий. Он проверяет выполненные маршруты, а не все возможные. Если тестовый запуск не прошёл через подозрительный region, отсутствие события не доказывает безопасность. Поэтому отчёт должен различать “not observed” и “validated normal”.

8.4.15. Несколько маленьких сценариев, чтобы закрепить модель

Первый сценарий: маленькая arithmetic-функция.

```
lea eax, [rcx + rdx]
ret
```

Она использует аргументы в volatile-регистрах, кладёт результат в EAX и возвращается. Нет sub rsp, нет сохранения nonvolatile-регистров, нет call, нет handler-а. Если .pdata записи нет, это нормально. static может вообще не создавать warning или создать very low confidence informational note, если регион почему-то попал в анализ uncovered gaps.

Второй сценарий: маленькая функция без calls, но с сохранением RBX.

```
push rbx
mov  rbx, rcx
; работа
pop  rbx
ret
```

Здесь уже есть nonvolatile state. Если exception случится после push rbx, текущий RSP указывает на сохранённый RBX, а return address лежит выше.

Простая leaf simulation ошибётся. Такая функция должна быть описана unwind metadata. Если `.pdata` нет, static имеет основание поднять severity.

Третий сценарий: jump thunk.

```
jmp qword ptr [rip + imported_target]
```

Собственного кадра нет. Управление уходит в target. Такой фрагмент может быть uncovered без проблемы. Но dynamic может показать, куда реально ушёл RIP, и подтвердить, что это прозрачный переход, а не скрытая exception-ветка.

Четвёртый сценарий: функция с вызовом API и локальным стеком.

```
sub rsp, 28h
call qword ptr [rip + SomeApi]
add rsp, 28h
ret
```

Даже если функция очень короткая, она меняет RSP и делает call. Если exception произойдёт между `sub` и `add`, leaf simulation неправильна. Тут отсутствие `.pdata` уже гораздо более серьёзный сигнал.

Пятый сценарий: код в памяти после unpacking. На диске static не видит нормальной `.pdata`, но dynamic наблюдает вызов `RtlAddFunctionTable` и потом корректные handler hits. Здесь статическая неполнота объясняется runtime-регистрацией. Это не отменяет интерес finding-а, но меняет его смысл: проблема не “metadata отсутствует вообще”, а “static PE не содержит полной runtime unwind-карты”.

8.4.16. Почему это важно для хорошего отчёта

Теперь свяжем всё с твоим EH/Unwind Analyzer. static должен быть полезным реверсеру, а не просто громким. Поэтому сообщение о leaf/uncovered code должно быть объясняющим.

Плохой стиль отчёта:

```
[HIGH] Missing pdata for code range 0x140012340-0x140012350
```

Такой текст оставляет человека в одиночестве. Он не объясняет, почему это high, какие эффекты найдены, является ли регион функцией, есть ли признаки nonleaf, возможно ли dynamic registration, проверял ли dynamic runtime route.

Хороший стиль отчёта должен говорить примерно так:

```
Finding: executable range is not covered by static .pdata
Evidence: region contains sub rsp, 0x28 and a normal call instruction
Consequence: if an exception occurs inside this range, leaf unwind simulation may treat the cur
Confidence: medium/high, because prolog-like stack allocation is present
dynamic     status: not validated / validated / runtime route observed
Recommendation: inspect function boundary and check dynamic function table registration
```

Такой finding уже обучает. Он не просто пугает, а показывает причинную цепочку: что найдено, почему это важно, что может сломаться, насколько мы уверены и что делать дальше.

Для книги это принципиально. Мы не учим тебя зубрить признаки. Мы строим модель, из которой признаки становятся понятными. Если есть модель кадра, return address, RSP, nonvolatile-регистров и `.pdata/.xdata`, то leaf-функция перестаёт быть исключением из правил. Она становится естественным случаем: когда отдельная unwind-инструкция не нужна, потому что состояние не было усложнено.

8.4.17. Главная граница: “не покрыто” против “невозможно безопасно восстановить”

В конце раздела сформулируем самую важную мысль. Опасно не само отсутствие `.pdata`. Опасно несоответствие между реальным поведением кода и моделью, которую Windows будет вынуждена применить.

Если кода нет, это padding. Если код есть, но он leaf, return simulation достаточно. Если код является прозрачным jump thunk, отдельного кадра может не быть. Если код зарегистрирован через dynamic function table, статическая `.pdata` в файле может быть неполной. Всё это нормальные или объяснимые варианты.

Но если код строит кадр, меняет RSP, сохраняет nonvolatile-регистры, вызывает другие функции, содержит protected regions или handler-логику, а Windows не получает function table entry, тогда unwind-модель становится ложной. Система будет думать, что перед ней простая функция, хотя перед ней функция со внутренним состоянием. Вот это и есть реальный сигнал.

static должен найти этот конфликт статически. dynamic должен проверить, как он проявляется в живом маршруте: куда ушёл RIP, что случилось с RSP, был ли handler hit, восстановился ли caller context, не развалился ли stack trace, появилась ли dynamic function table. И только вместе эти два слоя дают зрелый вывод.

Следующий раздел естественно продолжит эту линию. Если leaf-функции учат нас не ругаться на нормальное отсутствие `.pdata`, то dynamic function tables учат ещё более важной вещи: не вся unwind-карта обязана жить в PE-файле на диске. Иногда часть карты появляется только во время выполнения, и без dynamic -а статическая картина будет принципиально неполной.

8.4.18. Почему границы функции иногда менее очевидны, чем кажется

Когда мы говорим “функция не покрыта `.pdata`”, мы предполагаем, что уже знаем, где функция начинается и где заканчивается. В исходном коде это

просто: есть имя, открывающая фигурная скобка, закрывающая фигурная скобка. В машинном коде всё сложнее. PE-файл не обязан хранить универсальный список “вот все функции программы” в удобном для реверсера виде. Символы могут быть удалены. Отладочная информация может отсутствовать. Компилятор может склеивать, разделять и переупорядочивать куски кода. Linker может выбрасывать неиспользуемые фрагменты и объединять секции. Поэтому границы функции часто являются результатом анализа, а не готовым фактом.

`.pdata` помогает с границами, но только для тех функций, которые участвуют в table-based unwind. Если запись есть, она даёт сильный сигнал: `BeginAddress` и `EndAddress` задают диапазон, который Windows считает одной функцией для целей unwind. Но если записи нет, это не означает, что границы можно автоматически вывести по ближайшему `ret`. Маленький фрагмент без `.pdata` может быть leaf-функцией, а может быть хвостом предыдущей функции, padding-ом, jump island, thunk-ом или ошибочно распознанным disassembler-ом куском данных.

Из-за этого static должен разделять два вопроса. Первый вопрос: “существует ли в этом месте исполняемый код?” Второй вопрос: “является ли этот исполняемый код самостоятельной функцией, для которой Windows должна иметь unwind-описание?” Это разные вопросы. На первый можно ответить через секции, права, disassembly и reachability. На второй - только через анализ эффекта кода на стек, регистры и exception route.

Для человека это тоже важный навык. Когда Ghidra показывает функцию в uncovered gap, нельзя сразу считать, что Microsoft ABI нарушен. Надо спросить: почему Ghidra решила, что здесь функция? Есть ли cross-reference на начало? Есть ли `call` именно на этот адрес, или это target `jmp`? Есть ли нормальный function prolog? Есть ли `ret`? Есть ли stack frame? Есть ли признаки padding? Только после этого можно обсуждать `.pdata`.

8.4.19. Uncovered code, uncovered function и uncovered nonleaf - три разных уровня

Слово `uncovered` удобно, но оно слишком широкое. Если не уточнять, что именно не покрыто, отчёт становится мутным. Давай разделим три уровня.

Первый уровень - uncovered code. Это просто байты в executable area, которые не попали ни в один диапазон `RUNTIME_FUNCTION`. Такой факт сам по себе почти ничего не доказывает. В `.text` могут быть байты, которые никогда не являются отдельной nonleaf-функцией.

Второй уровень - uncovered function candidate. Это уже сильнее: анализатор считает, что uncovered region похож на функцию. Почему похож? Может быть, на него есть `call`, он имеет входной адрес, последовательность инструкций, `ret`, обращения к аргументам. Но даже здесь ещё не доказана проблема,

потому что функция может быть leaf.

Третий уровень - uncovered nonleaf candidate. Вот это уже важный сигнал. Это uncovered region, который не просто похож на функцию, а демонстрирует эффекты, требующие unwind metadata: меняет RSP, сохраняет nonvolatile-регистры, делает обычные вызовы, использует frame pointer, содержит handler-related patterns. Если для такого региона нет `.pdata`, Windows при exception может применить неправильную leaf-модель.

Эта трёхуровневая модель резко уменьшает шум. static не должен бросать high severity на первом уровне. Он должен постепенно повышать уверенность: executable uncovered bytes -> probable code -> probable function -> probable nonleaf -> probable missing unwind metadata. В хорошем отчёте человек должен видеть, на каком именно уровне сделан вывод.

8.4.20. Что именно Windows делает, когда не нашла RUNTIME_FUNCTION

Теперь разберём это почти пошагово, потому что здесь рождается вся логика ложных тревог. Есть текущий `ControlPc` - адрес, где поток находился с точки зрения unwind-процедуры. Windows ищет для него запись function table. В обычном PE это значит: найти `RUNTIME_FUNCTION`, у которого `BeginAddress <= RVA(ControlPc) < EndAddress`. Если запись найдена, можно перейти к `.xdata` и понять, как восстанавливать кадр.

Если запись не найдена, Windows не может прочитать `UNWIND_INFO`, потому что до неё нет пути. Тогда остаётся простая модель: считать текущий код leaf-фрагментом. В этой модели нет пролога, который надо отменять. Нет сохранённых nonvolatile-регистров, которые надо доставать из stack slots. Нет associated handler-a. Есть только return address на вершине стека.

В голове это можно представить так:

```
lookup(ControlPc) -> no RUNTIME_FUNCTION
assume leaf-like frame
NextRip = *(uint64_t*)Rsp
NextRsp = Rsp + 8
continue unwind from NextRip
```

Эта модель прекрасна, когда она соответствует реальности. Она ужасна, когда реальность другая. Если текущий RSP указывает не на return address, а внутрь локального кадра, всё дальнейшее дерево анализа строится на ложном основании. Это как взять неправильную точку на карте: потом можно очень аккуратно идти по компасу, но всё равно прийти не туда.

Поэтому “нет `RUNTIME_FUNCTION`” не является finding-ом само по себе. Finding появляется, когда static видит, что leaf assumption неправдоподобна. То есть отсутствует не просто запись, а отсутствует запись там, где простая модель возврата недостаточна.

8.4.21. Почему это особенно важно при исключениях, а не только при stack trace

Можно подумать: “Ну сломается stack trace, и что? Это же только отладочная красота”. Нет. В Windows x64 unwind-механика нужна не только отладчику. Она участвует в exception handling. Когда происходит exception, система должна пройти по кадрам и найти обработчик. Если кадр восстановлен неправильно, поиск может уйти не туда.

В frame-based SEH на x64 обработчик не висит в linked list на стеке так, как многие привыкли по старому x86 SEH. Система смотрит на текущий `RIP`, находит function table entry, читает `UNWIND_INFO`, смотрит flags, вызывает language-specific handler, если он есть, и передаёт ему `HandlerData`. Если на одном шаге unwinder ошибся и получил неправильный caller `RIP`, следующий поиск пойдёт уже по неправильной функции.

Отсюда понятна цена missing metadata для настоящей nonleaf-функции. Это не просто “неудобно анализировать”. Это может сделать exception route ненадёжным. А если программа специально играет исключениями, использует VEH/SEH/control-flow tricks или packer/runtime code, ошибка в понимании unwind-маршрута может полностью исказить картину реверса.

dynamic как раз нужен, чтобы увидеть живую сторону этой проблемы. static говорит: “если здесь случится exception, leaf assumption может быть неверной”. dynamic может попытаться поймать exception route, handler hit, изменение `RIP`, восстановление RSP и показать, действительно ли runtime идёт по ожидаемой карте или уходит в неожиданную область.

8.4.22. No red zone: почему нельзя автоматически считать стек под RSP рабочей областью

Есть ещё одна тонкость Windows x64, которая помогает не спутать leaf-функцию с чем-то другим. В некоторых ABI существует red zone - область ниже текущего stack pointer, которую leaf-функция может временно использовать без изменения RSP. В Windows x64 такой привычной red zone для обычного user-mode кода нет. Это значит, что код не должен просто писать “под RSP” и считать это безопасной локальной областью.

Почему это важно для анализа? Если мы видим, что функция использует память относительно RSP, нужно понять, какая это память. Обращение к `[rsp+8]` может быть доступом к home space или аргументам. А вот запись в область ниже текущего RSP без выделения стека может быть подозрительной или зависеть от особого контекста. Но главное: нормальная локальная область обычно требует явного изменения RSP, а изменение RSP уже приближает нас к nonleaf/unwind-needed сценарию.

Это не значит, что любое обращение к RSP делает функцию nonleaf. Входной стековый layout на Windows x64 содержит return address, shadow space, stack arguments. Маленькая функция может читать аргументы со стека и оставаться leaf. Но если она создаёт новую локальную область через `sub rsp`, сохраняет туда регистры или размещает temporaries, простая leaf simulation уже становится опасной.

8.4.23. Почему leaf-функция не имеет своего SEH-слоя

Если функция не имеет `RUNTIME_FUNCTION`, у неё нет стандартного места, где лежали бы flags `UNW_FLAG_EHANDLER` или `UNW_FLAG_UHANDLER`. Поэтому leaf-функция без `.pdata` не может быть нормальным носителем frame-based SEH handler-а в x64 table-based модели. Она может выполняться в процессе, где есть VEH. Она может вызвать API, который где-то внутри использует exceptions. Но сама она не имеет своего описанного через `.xdata` protected range.

Это помогает в анализе. Если uncovered region выглядит как место, которое должно иметь собственный handler, например содержит явный filter path, cleanup path, scope-like transitions или runtime-specific handler data, то “это просто leaf” становится слабой гипотезой. Leaf без metadata не несёт своего SEH-описания.

Но опять нужна аккуратность. Обработчик может быть не frame-based SEH, а VEH, зарегистрированный на уровне процесса. VEH не привязан к кадру через `.pdata`. Он живёт в другом слое доставки exception. Поэтому uncovered leaf-код всё равно может быть частью exception-based control flow через VEH. В этом случае static видит статический gap, но не может полностью понять маршрут без dynamic-а. dynamic смотрит first-chance exception, handler hit и последующий `RIP trace`.

8.4.24. Как не перепутать tiny wrapper с опасной функцией

Очень короткие функции обманывают глаз. Человек видит пять инструкций и думает: “ну это наверняка leaf”. Но размер сам по себе не решает. Важен эффект. Сравним две короткие функции.

```
; Вариант А
mov eax, [rcx]
ret

; Вариант Б
sub rsp, 28h
call qword ptr [rip + helper]
add rsp, 28h
ret
```

Обе короткие. Но вариант А может быть нормальной leaf-функцией без .pdata. Вариант Б уже меняет RSP и вызывает helper. Если exception произойдёт после sub rsp, 28h, на вершине стека не лежит return address caller-а. Поэтому для варианта Б отсутствие metadata гораздо серьёзнее.

Теперь другая пара:

```
; Вариант В
jmp qword ptr [rip + imported_function]

; Вариант Г
push rbx
jmp  qword ptr [rip + imported_function]
```

Вариант В может быть прозрачным thunk-ом. Вариант Г уже меняет стек и сохраняет nonvolatile-регистр, но не показывает нормального восстановления перед jmp. Это совсем другой сигнал. Маленький размер не делает код безопасным. Отсутствие call тоже не гарантирует leaf-совместимость. Снова смотрим на эффект.

Это мышление должно лечь в основу static -а: не классифицировать по внешнему виду, а строить model of effects. Что код сделал с RSP? Что он сделал с nonvolatile-регистрами? Создал ли он новый call frame? Есть ли нормальный epilog? Совместима ли эта модель с отсутствием RUNTIME_FUNCTION?

8.4.25. Практический разбор адресного диапазона

Представим, что static нашёл gap между двумя RUNTIME_FUNCTION:

```
.pdata:
  00011000-00011180 -> unwind A
  00011240-00011410 -> unwind B

gap:
  00011180-00011240
```

Плохой анализ сразу скажет: “между функциями есть gap, suspicious”. Хороший анализ начнёт с вопросов. Размер gap 0xC0 байт. Что внутри? Если там CC CC CC, nop, alignment pattern и нет cross-reference, это почти наверняка padding. Если там несколько jmp qword ptr [rip+...], это может быть thunk island. Если там есть входящий call 0x111A0, нормальный код, sub rsp, call, ret, то это уже candidate function.

Дальше static смотрит на effects. Есть ли sub rsp? Есть ли push rbx/rbp/rsi/rdi/r12-r15? Есть ли mov [rsp+...] для сохранения nonvolatile? Есть ли frame pointer? Есть ли ordinary call? Есть ли stack probing через __chkstk? Если да, leaf-гипотеза слабеет.

Потом static смотрит на окружение. Может быть, этот gap находится в секции, где linker положил thunks. Может быть, рядом import table. Может быть, это hotpatch stub. Может быть, это compiler-specific fragment. Может быть, предыдущий RUNTIME_FUNCTION range слишком короткий из-за

obfuscation или повреждения. Может быть, disassembler ошибся и начал функцию с середины data blob.

И только после этого формируется finding. Не “gap = bad”, а “gap contains probable nonleaf function not covered by static `.pdata`”. Это уже не шум, а аргументированное наблюдение.

8.4.26. Как это должно лечь в будущую проектную главу

Когда мы в конце книги будем описывать архитектуру EH/Unwind Analyzer, этот раздел станет основанием для одного из важных правил: static обязан различать raw evidence и interpretation. Raw evidence: “адрес 0x... не покрыт `.pdata`”. Interpretation: “вероятно padding”, “вероятно leaf”, “вероятно thunk”, “вероятно nonleaf without unwind metadata”, “нужна dynamic validation”.

dynamic в этой архитектуре не просто запускает программу ради красивого runtime-лога. Он проверяет те места, где статическая интерпретация не может быть окончательной. Если static говорит “uncovered region may be leaf”, dynamic может показать, исполняется ли он как leaf. Если static говорит “possible nonleaf missing unwind”, dynamic может попытаться поймать exception path и посмотреть, как реально восстанавливается маршрут. Если static говорит “maybe dynamic table”, dynamic смотрит регистрацию function table в runtime.

Так книга постепенно подводит к реальному проекту: не сначала код, потом объяснение, а сначала теория, из которой архитектура проекта становится естественной. static строит карту. dynamic проверяет живой маршрут. Report связывает оба слоя в evidence chain.

8.5. Dynamic function tables / JIT / runtime code: когда unwind-карта появляется не из PE-файла, а во время выполнения

В предыдущих разделах мы постепенно построили спокойную модель: есть PE-файл на диске, в нём есть заголовки, секции, Exception Directory, затем таблица .pdata, затем записи RUNTIME_FUNCTION, затем .xdata и UNWIND_INFO. Эта модель правильная, но она пока описывает только самый привычный случай: код был известен компилятору и компоновщику заранее, таблицы были записаны в файл заранее, загрузчик Windows отобразил этот файл в память, а дальше операционная система может пользоваться уже готовой картой функций.

Но у Windows x64 есть важное усложнение, без которого картина будет неполной. Не весь исполняемый код процесса обязан существовать в PE-файле на диске в момент запуска. Программа может создать новый машинный код уже во время выполнения. Такой код может появиться у JIT-компилятора, у runtime-системы языка, у движка регулярных выражений, у виртуальной машины, у скриптового движка, у упаковщика, у распаковщика, у защиты, у instrumentation-системы, у hot patch механизма. И если этот код будет выполняться как настоящий x64-код, то для Windows он сразу становится участником тех же правил, что и обычные функции из .text.

Вот здесь и появляется тема dynamic function tables. Dynamic значит "динамический", то есть появляющийся или изменяющийся во время работы процесса. Function table значит "таблица функций", но не в смысле списка имён функций из исходника. Как мы уже много раз проговаривали, function table в x64 exception/unwind-модели - это таблица диапазонов машинного кода, по которой Windows может понять: если RIP попал в такой-то диапазон, то для восстановления кадра нужно взять такую-то unwind-информацию. Поэтому dynamic function table - это не украшение и не вспомогательный лог. Это способ сказать Windows: "в памяти появился код; пожалуйста, считай его частью общей unwind-карты процесса".

> Ключевая мысль раздела: .pdata в PE-файле - это не вся вселенная unwind-информации. Это статическая часть карты. Во время выполнения процесс может добавить ещё одну карту - динамическую. `static` видит прежде всего файл. `dynamic` видит живой процесс и может проверить, появляется ли такая карта в runtime.

8.5.1. Почему вообще возникает эта проблема

Чтобы не воспринимать RtlAddFunctionTable и похожие API как очередные странные имена из WinAPI, нужно начать с самой проблемы. Представим обычную nonleaf-функцию, то есть функцию, которая не является простой

leaf-функцией. Она выделяет стек, возможно сохраняет RBX или RBP, вызывает другие функции, может иметь обработчик исключения или попасть в stack trace. Пока мы имеем дело с кодом из PE-файла, компилятор заранее создаёт для такой функции запись RUNTIME_FUNCTION и UNWIND_INFO. Windows потом не угадывает устройство кадра по дизассемблеру, а читает готовую машинно-читаемую инструкцию.

Теперь представим другой случай. Программа во время выполнения выделила память, записала туда байты машинного кода, сделала эту память исполняемой и прыгнула туда. С точки зрения процессора всё нормально: RIP указывает на адрес в памяти, байты декодируются как инструкции, инструкции выполняются. Но с точки зрения Windows exception/unwind-механики возникает вопрос: если в этом коде случится исключение, если debugger попросит stack trace, если profiler или crash reporter попробует пройти по стеку, как Windows поймёт, где начинается функция, где заканчивается функция, сколько байт было вычтено из RSP, где сохранён RBX и где лежит return address?

Процессор этого не знает. Машинный код сам по себе не содержит универсальной таблицы "как меня раскручивать". У x64 Windows, в отличие от некоторых более эвристических подходов, ставка сделана на table-based unwinding - табличную раскрутку. Table-based значит, что система опирается не на догадку и не на обязательный frame pointer в каждой функции, а на специальные таблицы. Для кода из PE такие таблицы лежат в .pdata/.xdata. Для кода, который появился позже, таблицу должен предоставить тот runtime, который этот код породил.

Слово runtime здесь тоже важно. Runtime - это не "время выполнения" в абстрактном смысле, а часто ещё и программный слой, который обслуживает язык или движок во время работы: .NET runtime, JavaScript engine, LuaJIT, JVM-подобная система, встроенный JIT-компилятор, собственная виртуальная машина защиты. Такой runtime может сам генерировать нативный машинный код. Если он хочет, чтобы исключения, отладка и stack walking работали надёжно, он должен не только создать инструкции, но и объяснить Windows, как эти инструкции раскручивать.

8.5.2. On-disk PE, mapped image и runtime memory: три разных слоя, которые нельзя смешивать

Новичок часто смешивает три понятия: файл на диске, образ в памяти и произвольную память процесса. Для обычного исполняемого модуля это смешение иногда не сразу ломает понимание, потому что файл действительно отображается в память, секции получают адреса, RVA можно перевести в VA, а .pdata в памяти похожа на .pdata в файле. Но при динамическом коде это смешение становится смертельным для анализа.

On-disk PE - это PE-файл как набор байтов на диске. Его читает static . Там есть DOS header, NT headers, section table, data directories, Exception Directory. Если static видит .pdata в файле, он может разобрать статическую часть unwind-карты. Но он не видит того кода, который будет сгенерирован через секунду после запуска процесса. Файл на диске ещё не содержит этого кода.

Mapped image - это загруженный модуль в адресном пространстве процесса. Windows loader берёт PE-файл, отображает секции, применяет relocation, настраивает imports, выставляет защиты страниц. Для обычных функций из .text мы часто можем сказать: есть ImageBase, есть RVA, есть VA = ImageBase + RVA. Это уже живая память процесса, но она всё ещё связана с исходным PE-модулем.

Runtime memory - это память, которую программа выделила сама. Например, через VirtualAlloc, heap allocator, memory manager runtime-а или другой механизм. Такая память может быть сначала writable, чтобы туда записали байты кода, а потом executable, чтобы процессор мог эти байты исполнять. У этой памяти нет section table из PE. У неё может не быть имени секции, raw offset, .text, .pdata, .xdata в привычном файловом смысле. Но у неё есть адреса в процессе. А если RIP туда попал, то для Windows это уже реальное выполнение.

```
Файл на диске:
    PE headers
    .text
    .pdata
    .xdata

Образ в памяти:
    module ImageBase
    .text mapped as executable pages
    .pdata mapped as runtime function table
    .xdata mapped as unwind metadata

Динамическая память:
    VirtualAlloc / runtime allocator
    generated code bytes
    generated unwind metadata bytes
    dynamic function table registered through API
```

Именно поэтому статический анализатор не должен говорить: "в файле нет записи для этого runtime-кода, значит всё сломано". У static -а просто нет полной картины runtime memory. Его задача - честно описать то, что есть в файле, и найти признаки того, что во время выполнения может появиться дополнительная карта. А задача dynamic -а - запустить процесс, поймать регистрацию динамической таблицы и проверить, куда реально уходит управление.

8.5.3. Что такое JIT и почему он особенно важен для unwind

JIT - это сокращение от just-in-time compilation, то есть "компиляция прямо перед использованием". Смысл JIT в том, что программа не хранит весь конечный машинный код заранее. Вместо этого она хранит более высокий уровень представления: байткод, промежуточное представление, дерево операций, скрипт, регулярное выражение, внутренний IR. А уже во время работы она превращает этот материал в нативные инструкции под текущую архитектуру, например под Windows x64.

С точки зрения производительности JIT понятен: можно оптимизировать код под реальные данные, под текущую платформу, под горячие участки программы. Но с точки зрения reverse engineering JIT усложняет картину. Файл на диске может выглядеть почти пустым: небольшой engine, runtime, таблицы, dispatcher. А основная интересная логика появляется только после запуска, когда runtime сгенерировал исполняемый блок в памяти.

Теперь связываем это с unwind. Если JIT сгенерировал только leaf-код, который не трогает RSP и не вызывает другие функции, Windows ещё может обойтись leaf-моделью. Но настоящий JIT-код часто сложнее. Он может вызывать runtime helpers, сохранять nonvolatile-регистры, выделять временный стек, иметь safepoints, переходить в exception stubs, участвовать в profiling, переходить в slow paths. Такой код становится nonleaf-кодом. А nonleaf-код в x64 Windows должен быть описан для unwinding. Иначе stack walk, exception dispatch и debugger support становятся ненадёжными.

Здесь важно увидеть принцип: JIT-компилятор отвечает не только за генерацию инструкций, но и за генерацию метаданных о них. Если он создаёт пролог, он должен создать соответствующий UNWIND_INFO. Если он создаёт несколько функций внутри одной memory region, он должен создать RUNTIME_FUNCTION entries для их диапазонов. Если код может удаляться или перемещаться, runtime должен следить, чтобы Windows не осталась с устаревшей картой.

8.5.4. Active function tables: общая карта процесса шире, чем .pdata одного файла

До этого момента мы часто говорили "найти запись в .pdata". Для ручного анализа PE это правильная формулировка. Но если смотреть глазами Windows во время выполнения, формулировка шире: системе нужна active function table view - активная картина таблиц функций для процесса. В неё входят не только статические таблицы, пришедшие из загруженных PE-модулей, но и динамические таблицы, которые процесс добавил через специальные API.

Слово active здесь полезно понимать буквально: это не архивная информация в файле, а та информация, которой Windows может пользоваться сейчас. Если модуль загружен - его .pdata доступна. Если модуль выгружен - её больше нельзя использовать. Если runtime зарегистрировал динамическую таблицу - она участвует в поиске. Если runtime удалил таблицу через

RtlDeleteFunctionTable или аналогичный механизм - она больше не должна участвовать. Поэтому unwind-карта процесса является живой структурой, а не только содержимым PE-файла.

Когда происходит исключение или stack walk, Windows получает ControlPc - адрес, относительно которого нужно понять текущую функцию. Мы уже говорили, что ControlPc близок к RIP, но в деталях может быть адресом, по которому выполняется операция раскрутки. Система должна найти function entry для этого адреса. Для обычного модуля поиск ведёт к его .pdata. Для динамического кода поиск может привести к динамической function table. В обоих случаях идея одна: найти диапазон BeginAddress <= ControlPc < EndAddress и получить UnwindInfoAddress.

> Важный поворот мышления: .pdata - это файловая форма статической таблицы. Function table - это более общее понятие. Dynamic function table - это способ добавить такую таблицу в живой процесс, когда кода ещё не было в PE на диске.

8.5.5. RtlAddFunctionTable: простой случай динамической карты

Первый API, который нужно понять, - RtlAddFunctionTable. Его имя можно разложить почти дословно: Rtl - слой runtime library в Windows; Add - добавить; FunctionTable - таблицу функций. То есть функция сообщает системе: "вот массив RUNTIME_FUNCTION entries; добавь его в список динамических таблиц, чтобы unwind и stack walking могли им пользоваться".

Почему это не просто "передать .pdata"? Потому что .pdata как секция PE здесь может вообще отсутствовать. Runtime может выделить память для кода и отдельно выделить память для массива RUNTIME_FUNCTION. Эти записи выглядят концептуально так же, как записи из .pdata: begin, end, unwind info. Но base address для вычисления полных адресов задаётся параметром BaseAddress. Это ключевой момент: поля BeginAddress, EndAddress и UnwindInfoAddress остаются относительными, а BaseAddress говорит, от какого начала их считать.

```
Упрощённая модель RtlAddFunctionTable:

    generated_code_base = 00000240`10000000

    RUNTIME_FUNCTION[0]:
        BeginAddress      = 0x0000
        EndAddress        = 0x0060
        UnwindInfoAddress = 0x2000

    RtlAddFunctionTable(table, 1, generated_code_base)

Для Windows это означает:
    function begins at  generated_code_base + 0x0000
    function ends at    generated_code_base + 0x0060
    unwind info at      generated_code_base + 0x2000
```

Эта схема похожа на ImageBase + RVA, но не обязана быть связана с PE-модулем. BaseAddress может быть началом выделенной runtime-области. Поэтому реверсеру нужно быть осторожным: если он видит RUNTIME_FUNCTION entries в памяти процесса, он не всегда может перевести их через ImageBase какого-то модуля. Иногда это не RVA внутри PE, а offset относительно runtime base.

RtlAddFunctionTable подходит для кода, который был сгенерирован один раз или относительно стабилен. Например runtime создал блок функций из шаблона, подготовил для них unwind metadata, зарегистрировал таблицу и дальше использует её до конца жизни процесса или до явного удаления. В таком сценарии таблица уже готова в момент регистрации. Windows не должна каждый раз спрашивать callback: она может искать по массиву entries.

Но вместе с удобством появляется строгий контракт. Память с таблицей должна оставаться живой, пока таблица зарегистрирована. Память с UNWIND_INFO тоже должна оставаться живой. Память с кодом должна соответствовать тем диапазонам, которые объявлены в entries. Если runtime освободил код, но забыл удалить function table, Windows может позже попытаться unwind-ить адрес, который уже не соответствует старому коду. Если runtime переписал код, но не обновил metadata, появится mismatch между действительным прологом и объявленной моделью. Для обычной программы это баг. Для обфускации или защиты это может быть намеренным приёмом. Для static -а это не видно полностью по файлу. Для dynamic -а это как раз предмет динамической проверки.

8.5.6. RtlDeleteFunctionTable: почему lifetime важен не меньше, чем регистрация

Если есть добавление динамической таблицы, должен быть и способ удалить её. Для таблиц, добавленных через RtlAddFunctionTable, используется RtlDeleteFunctionTable. На первый взгляд это скучная housekeeping-функция: зарегистрировали - потом удалили. Но для понимания безопасности и корректности это очень важный момент.

Lifetime - это время жизни объекта. В нашем случае есть несколько объектов с разным временем жизни: код в памяти, массив RUNTIME_FUNCTION, структуры UNWIND_INFO, возможно language-specific handler data, возможно сами handler stubs. Все они должны жить согласованно. Нельзя оставить Windows ссылку на таблицу, если таблица уже освобождена. Нельзя оставить table entry, который ведёт к UNWIND_INFO в странице, уже возвращённой allocator-у. Нельзя удалить таблицу раньше, чем поток перестал выполнять код из этой области.

Для реверсера это важная мысль: динамическая function table - это не просто факт "была регистрация". Нужно смотреть жизненный цикл. Когда таблица создана? Когда код стал executable? Когда таблица удалена? Были ли

исключения или stack walk между этими событиями? Не появляется ли подозрительный паттерн: зарегистрировать таблицу, выполнить маленький участок, быстро удалить, стереть memory region? Такой паттерн может быть абсолютно легитимным для runtime-а, но может быть и способом оставить мало следов для статического анализа.

Здесь снова помогает разделение ролей. static может увидеть imports или indirect resolve на RtlAddFunctionTable/RtlDeleteFunctionTable, может заметить код, который выделяет executable memory, может увидеть suspicious RWX pattern. Но static не знает, была ли функция реально вызвана и какая таблица реально зарегистрировалась. dynamic должен поймать сам runtime event: была добавлена таблица, с таким BaseAddress, с таким EntryCount, затем в этот диапазон попал RIP, затем exception или stack walk пошёл через этот диапазон.

8.5.7. RtlInstallFunctionTableCallback: карта не хранится целиком, а выдаётся по запросу

Второй важный API - RtlInstallFunctionTableCallback. Его название длинное, но смысл можно разложить. Install - установить, зарегистрировать. FunctionTableCallback - callback для получения function table. Callback - это функция обратного вызова: не мы вызываем систему напрямую для каждой записи, а система позже вызывает нашу функцию, когда ей нужна информация. Если RtlAddFunctionTable говорит "вот готовая таблица", то RtlInstallFunctionTableCallback говорит "вот диапазон памяти с кодом; когда тебе понадобится unwind-информация для адреса внутри этого диапазона, вызови мой callback, и я верну нужную запись".

Зачем так сложно? Потому что очень динамический runtime может генерировать, перемещать, удалять, перекомпилировать и оптимизировать код постоянно. Создавать и поддерживать одну полностью готовую таблицу для всего диапазона может быть неудобно. Runtime может знать внутреннюю карту кода в своей структуре данных и выдавать RUNTIME_FUNCTION только тогда, когда Windows действительно спросила про конкретный ControlPc. Это ленивый подход: не строить всё заранее, а обслуживать запросы по мере необходимости.

У callback-модели есть несколько терминов, которые нужно закрепить. BaseAddress - начало области памяти, которой управляет callback. Length - размер этой области. Вместе они задают range, диапазон адресов. Если ControlPc попадает внутрь этого диапазона, система понимает, что может обратиться к callback-у. Context - пользовательский указатель, который runtime передаёт сам себе: например указатель на объект JIT-компилятора, таблицу методов, дерево code ranges, lock-free map. TableIdentifier - идентификатор динамической таблицы; в документации указывается, что у него должны быть выставлены два младших бита, например BaseAddress | 0x3. Это не просто

произвольная эстетика, а часть соглашения API.

```
Упрощённая модель callback-таблицы:

    RtlInstallFunctionTableCallback(
        TableIdentifier = BaseAddress | 0x3,
        BaseAddress     = start_of_jit_region,
        Length          = jit_region_size,
        Callback        = get_runtime_function_for_control_pc,
        Context         = jit_runtime_state,
        OutOfProcessCallbackDll = optional
    )

Позже при unwind:
    ControlPc попал в jit_region
    Windows вызывает Callback(Context, ControlPc)
    Callback возвращает подходящий RUNTIME_FUNCTION
```

Для книги это особенно важно, потому что callback-модель показывает предел чисто статического мышления. В PE-файле может не быть не только динамической .pdata, но даже готовой runtime-таблицы в простом виде. Карта может жить как логика внутри runtime-а. Она может быть вычисляемой, ленивой, зависящей от состояния процесса. Поэтому попытка "просто открыть файл и найти все функции" принципиально не покрывает такие сценарии.

Но callback-модель не означает, что всё становится магией. Наоборот, она подчёркивает контракт: если Windows должна unwind-ить runtime-generated code, runtime обязан ответить. Он должен вернуть корректный RUNTIME_FUNCTION для указанного адреса. Если callback зависнет на lock-е, который удерживает код generator-а, можно получить deadlock. Если callback вернёт неправильную запись, unwind будет неправильным. Если callback скрывает часть карты, stack traces и exception handling станут неполными или намеренно искажёнными.

8.5.8. Growable tables: когда карта растёт вместе с кодом

Есть ещё одна полезная форма динамических таблиц - growable function table, то есть "растущая таблица функций". Смысл здесь промежуточный между готовым массивом и callback-моделью. Runtime заранее сообщает системе о memory range, где будет появляться код, и даёт массив RUNTIME_FUNCTION entries с максимальной вместимостью. По мере генерации новых функций runtime заполняет новые entries и сообщает, что таблица выросла.

Почему это удобно? Представим JIT, который выделил большой code heap - область памяти под сгенерированный код. Сначала в ней нет ни одной функции. Потом появилась первая. Потом ещё десять. Потом горячий метод был оптимизирован и добавлен новый вариант. Callback для каждого запроса может быть лишним, а полностью готовой таблицы заранее нет. Growable table позволяет поддерживать отсортированный массив entries, который увеличивается по мере появления новых функций.

Для реверсера эта модель важна тем, что dynamic function table может быть не только "разово добавили" и не только "вычисляем через callback". Она может эволюционировать. И тогда динамическая валидация должна учитывать время. Если dynamic начал наблюдение после того, как часть таблицы уже выросла, он увидит одну картину. Если он стартовал с самого начала процесса, он может увидеть последовательность: memory allocation, code emission, table growth, execution, exception. Именно последовательность событий часто объясняет то, что статически выглядит странно.

8.5.9. Почему отсутствие динамической таблицы ломает не только исключения

Легко подумать: "ну и что, нет function table для JIT-кода; исключения ведь редкие". Это опасное упрощение. Function tables на 64-битной Windows нужны не только для обработки C __try или C++ exceptions. Они нужны для stack walking, debugger support, crash dump analysis, profiling, sampling, diagnostics. Всё, что пытается пройти от текущего кадра к caller-у, зависит от возможности понять, как восстановить RSP, RIP и nonvolatile-регистры.

Вспомним базовую модель кадра. В обычном выполнении функция возвращается через ret: процессор берёт return address по адресу [RSP], кладёт его в RIP и увеличивает RSP. Но virtual unwind не исполняет ret. Он должен вычислить состояние caller-а искусственно. Если текущий RIP находится в функции, которая вычитала 0x80 из RSP и сохранила RBX по [RSP+0x70], unwinder должен знать эти факты. Если фактов нет, он может применить leaf-модель, то есть предположить, что return address лежит прямо на текущем RSP. Для nonleaf-кода это может быть неверно. Тогда stack trace уедет в мусор, handler search пойдёт неправильно, crash dump покажет ложную цепочку вызовов.

Поэтому Microsoft-документация формулирует это жёстко: динамически сгенерированным функциям нужно предоставить информацию операционной системе через RtlAddFunctionTable или RtlInstallFunctionTableCallback, иначе exception handling и debugging будут ненадёжны. Это не "рекомендация для красоты". Это следствие самой архитектуры x64 unwinding.

8.5.10. Статическая картина static -а: что можно увидеть без запуска

Теперь переведём теорию в архитектуру твоего проекта. static - это статический слой. Он читает PE-файл как объект на диске. Поэтому в теме dynamic function tables его возможности ограничены, но не бесполезны. Он не увидит уже созданную runtime-таблицу, потому что её ещё нет. Но он может увидеть подготовку к такой таблице.

Первый признак - imports. Если PE импортирует RtlAddFunctionTable, RtlInstallFunctionTableCallback, RtlAddGrowableFunctionTable, RtlGrowFunctionTable или RtlDeleteFunctionTable, это прямой сигнал: программа потенциально работает с динамическими function tables. Но imports - это только evidence, доказательное наблюдение, а не итоговый вердикт. Наличие импорта может означать нормальный JIT runtime, profiler, language engine, exception support layer. Отсутствие импорта тоже не доказывает отсутствие механизма, потому что функция может быть найдена через GetProcAddress, скрыта behind import wrapper, резолвиться по hash-у или вызываться через ntdll/kernel32 thunk.

Второй признак - работа с executable memory. Если код выделяет память, записывает туда байты, меняет protection на executable и затем прыгает в эту область, static должен спросить: есть ли рядом регистрация function table? Если есть runtime-generated nonleaf code без unwind registration, это может быть причиной ненадёжного stack walking. Если регистрация есть, static должен не ругаться на отсутствие .pdata в файле для этого кода: карта может быть предоставлена динамически.

Третий признак - внутренние структуры runtime-а. Иногда в бинаре можно увидеть шаблоны RUNTIME_FUNCTION, массивы, сортировку entries, поля begin/end/unwind, callback-функцию, которая по ControlPc ищет запись. Это уже не просто импорт, а более глубокая evidence chain: программа не только знает API, но и строит данные, похожие на dynamic unwind map.

static может сформировать не вердикт, а гипотезу:

```
Evidence:
    import/use of RtlAddFunctionTable
    executable memory allocation
    generated code region
    memory writes before execution
    possible RUNTIME_FUNCTION array construction

Hypothesis:
    target may generate code at runtime and register dynamic unwind metadata

Need dynamic :
    confirm actual registration
    capture BaseAddress / EntryCount / range
    observe execution inside registered range
    validate exception/handler route if triggered
```

Именно так должен работать хороший анализатор: не делать вид, что он знает runtime-факты без runtime-а, а честно разделять static evidence и dynamic validation. Это важная дисциплина, которая делает отчёт полезным, а не шумным.

8.5.11. Динамическая картина dynamic -а: что нужно подтвердить в живом процессе

dynamic в твоём проекте - это динамическая валидация. В теме dynamic function tables он становится особенно ценным, потому что именно он может увидеть то, чего нет в PE-файле. Если static говорит: "похоже, программа может регистрировать динамическую unwind-карту", dynamic должен ответить: "да, регистрация реально произошла" или "в этом запуске не произошла".

Что именно нужно наблюдать? Во-первых, сам вызов API. Для RtlAddFunctionTable важно получить адрес массива FunctionTable, EntryCount и BaseAddress. Для RtlInstallFunctionTableCallback - TableIdentifier, BaseAddress, Length, Callback, Context. Для growable table - range и последующие growth events. Эти параметры превращают туманную фразу "динамическая таблица" в конкретную карту адресов.

Во-вторых, нужно наблюдать попадание RIP в соответствующий диапазон. Регистрация таблицы сама по себе ещё не означает, что код реально исполнялся. Runtime может подготовить таблицу заранее. dynamic должен уметь показать: вот registered range, вот RIP действительно оказался внутри этого range, вот после этого произошёл call/exception/handler hit/return path.

В-третьих, нужно наблюдать exception route. Если в сгенерированном коде возникло исключение, Windows должна найти соответствующую function table entry. Если есть handler, можно ставить breakpoint на handler или на language-specific stub. Если handler hit произошёл, dynamic может показать реальный маршрут: exception возникла здесь, dispatch пошёл через такую-то таблицу, handler был достигнут, после handler-а RIP ушёл туда-то.

В-четвёртых, нужно собирать RIP trace после handler hit. Это как раз соответствует твоему уточнению: dynamic показывает, куда уходит функция и обработка исключения. Не в философском смысле, а буквально: какой RIP был до exception, какой handler был достигнут, какие следующие RIP были после обработки, продолжилось ли выполнение, произошёл ли second chance, был ли process termination.

8.5.12. Почему dynamic function tables особенно важны для packers и manual mapping

Теперь нужно аккуратно связать тему с обфускацией и защитными трюками. Dynamic function tables сами по себе не являются плохим признаком. JIT-компилятор имеет законную причину их использовать. Но packer, dynamic или manual loader тоже могут создавать код в памяти. И тогда статическая .pdata из файла перестаёт быть полной картой реального исполнения.

Packer - это упаковщик. В простом варианте он хранит настоящий код в сжатом или зашифрованном виде, а при запуске распаковывает его в память и передаёт туда управление. Manual mapping - это ручная загрузка PE или PE-подобного образа без обычного Windows loader-а. dynamic в смысле

защитного средства может намеренно менять структуру кода и метаданных, чтобы затруднить анализ. В каждом из этих сценариев файл на диске не равен живому коду в памяти.

Если такой runtime-код хочет корректно участвовать в x64 exception handling, ему нужно предоставить Windows unwind-карту. Но он может сделать это по-разному: честно зарегистрировать dynamic function table; частично зарегистрировать только нужные диапазоны; использовать leaf-like stubs, чтобы избежать unwind metadata; намеренно ломать stack walking; создавать таблицы временно; регистрировать callback, который возвращает карту только при определённых условиях. Для реверсера каждый вариант несёт разный смысл.

Здесь важно не скатиться в примитивный вывод: "RtlAddFunctionTable = malware". Это неверно. Правильная мысль другая: RtlAddFunctionTable говорит, что static PE map потенциально неполна. Значит, если мы анализируем безопасность, обфускацию или устойчивость unwind, нам нужна динамическая проверка. static фиксирует признак. dynamic проверяет живое поведение.

8.5.13. Ошибка новичка: искать .pdata там, где её по определению нет

Представим, что в отладчике мы остановились на RIP = 00000240`10001234. Новичок пытается найти модуль, ImageBase, RVA, .pdata. Но адрес не попадает ни в один обычный PE-модуль. Он лежит в private memory region, возможно выделенной как PAGE_EXECUTE_READ. Если продолжать мыслить только моделью PE-файла, анализ заходит в тупик: "адрес не принадлежит модулю, значит это мусор". Но для JIT или unpacked code это может быть нормальным адресом исполняемого кода.

В такой ситуации нужно сменить вопрос. Не "в какой .pdata секции файла лежит запись?", а "есть ли для этого диапазона active function table?". Если да, то она может быть динамической. Нужно искать события регистрации. Нужно смотреть, какой BaseAddress был передан. Нужно проверить, какие RUNTIME_FUNCTION entries были добавлены. Нужно понять, где лежит UNWIND_INFO: в той же runtime region, в отдельной read-only memory, в структуре runtime-а, выдаётся ли оно callback-ом.

Это очень важный переход от файлового мышления к процессному мышлению. Файловое мышление спрашивает: что лежит в PE? Процессное мышление спрашивает: что реально существует в адресном пространстве сейчас? Для обычного reverse engineering нужны оба. Для твоей связки static /dynamic они прямо разделены по ролям.

8.5.14. Где может лежать dynamic unwind metadata

Когда мы говорим "динамическая таблица", не надо представлять, что Windows обязательно создаёт новую секцию .pdata. Секция - это понятие PE-файла. В runtime memory могут быть просто байты. Runtime сам решает, где хранить массив RUNTIME_FUNCTION, где хранить UNWIND_INFO, где хранить handler data. Эти области могут находиться рядом с кодом или отдельно от него.

Самый простой вариант: одна выделенная область содержит и код, и unwind metadata. Например первые страницы - executable code, последние страницы - read-only metadata. Более аккуратный вариант: код лежит в RX memory, metadata лежит в read-only memory. Более подозрительный или ленивый вариант: всё лежит в RWX memory, то есть страница одновременно writable и executable. Это не автоматически злонамеренно, но для современного анализа это важный сигнал, потому что RWX облегчает изменение кода на лету.

Ещё один вариант: callback вообще не хранит готовый UNWIND_INFO в простой форме до запроса. Он может построить запись на лету, вернуть указатель на временную структуру или на cache. Тут появляются вопросы lifetime и thread safety. Если Windows или debugger использует эту информацию вне ожидаемого момента, неправильный lifetime может привести к нестабильности. Поэтому dynamic unwind metadata - это не только вопрос формата, но и вопрос дисциплины владения памятью.

8.5.15. Как dynamic table меняет оценку gaps и uncovered code

В разделе 8.4 мы говорили про leaf-функции и ложные тревоги. Теперь нужно добавить ещё одну причину, почему uncovered code не всегда означает повреждение. Если код находится не в PE-модуле, то он по определению не покрывается .pdata этого PE. Его покрытие может быть предоставлено отдельно, через dynamic function table. Поэтому finding "uncovered executable code" должен учитывать адресный слой.

Если uncovered code лежит внутри .text обычного PE-модуля и выглядит как nonleaf-функция, это один сигнал. Если uncovered code лежит в private executable memory и рядом есть RtlAddFunctionTable registration, это другой сигнал. Если private executable memory активно исполняется, но function table registration не наблюдается, это третий сигнал. Если код deliberately вызывает исключения и handler-ы, но unwind-карта отсутствует или появляется временно, это уже отдельный сценарий для dynamic -а.

То есть один и тот же симптом - "статическая карта неполна" - имеет разные объяснения. Хороший анализатор не должен сваливать их в одну корзину. Он должен показать контекст: где адрес, кому принадлежит память, есть ли PE module, есть ли dynamic registration, был ли runtime hit, был ли exception route.

8.5.16. Мини-сценарий: нормальный JIT

Рассмотрим нормальный сценарий. Процесс запускает runtime языка. Runtime выделяет code heap. Затем компилирует метод в машинный код, создаёт для него UNWIND_INFO, добавляет RUNTIME_FUNCTION entry и вызывает RtlAddFunctionTable или growable-table механизм. Позже этот метод вызывается. Внутри метода происходит исключение или debugger строит stack trace. Windows находит динамическую запись, корректно восстанавливает caller context, stack trace выглядит осмысленно.

Что должен сказать static ? Он может сказать: обнаружены признаки runtime code generation и dynamic function table support. Это не suspicious само по себе, если контекст соответствует JIT/runtime. Что должен сказать dynamic ? Он может подтвердить: dynamic table registered, code range executed, unwind path valid, handler route observed or stack walk stable. Итоговый вывод спокойный: dynamic unwind infrastructure present and functionally consistent.

8.5.17. Мини-сценарий: packer или unpacker

Теперь другой сценарий. PE на диске содержит небольшой loader stub. Основной код зашифрован. При запуске stub выделяет память, расшифровывает туда код, возможно создаёт таблицу функций или использует callback. Затем передаёт управление в unpacked region. В статике static видит мало полезной .pdata для настоящего кода, потому что настоящего кода в файле ещё нет в открытом виде.

Здесь dynamic function table может быть очень важным следом. Если packer хочет, чтобы его unpacked code нормально переживал исключения или debugger stack walk, ему нужно как-то обслужить unwind. Он может зарегистрировать таблицу. dynamic может поймать этот момент и получить BaseAddress unpacked region. Даже если сам код защищён, факт регистрации таблицы уже даёт карту диапазонов: где начинаются и заканчиваются runtime-функции. Это ценная информация для реверсера.

Но возможен и другой вариант: packer вообще не заботится о корректном unwind. Тогда stack traces могут ломаться, exceptions могут вести себя нестабильно, а failure path будет отличаться от нормального compiled code. Это тоже evidence. Не абсолютное доказательство злонамеренности, но сильный сигнал: runtime code exists without reliable unwind support.

8.5.18. Мини-сценарий: exception-based control flow в runtime-коде

Самый интересный для нашей книги сценарий возникает, когда runtime-generated code не просто выполняется, а использует исключения как часть control flow. Например код намеренно вызывает int 3, guard page violation или access violation, а handler переносит RIP в другое место. Если такой код

находится в динамической memory region, то обычный статический PE-анализ видит только подготовку: API, memory allocation, possibly handlers. Реальный маршрут виден только в процессе.

Здесь dynamic function table становится частью маршрута. Если exception возникает внутри runtime range, Windows должна найти function table entry для этого RIP. Если handler находится в статическом модуле, route пересекает два мира: динамический код и статический handler. Если handler тоже сгенерирован динамически, route остаётся внутри runtime memory. dynamic должен показать этот путь как цепочку: exception site -> dynamic function table lookup -> handler hit -> context modification or unwind -> next RIP.

Именно здесь фраза "куда уходит функция" получает точный технический смысл. Она может уйти не только через ret. Она может уйти через exception dispatcher, через handler, через изменение ContextRecord, через continuation после EXCEPTION_CONTINUE_EXECUTION, через second chance termination. Dynamic function tables позволяют Windows хотя бы понять кадр, в котором всё это происходит.

8.5.19. Как это должно лечь в findings

Теперь сформулируем, как static должен говорить об этом в отчёте. Плохой finding звучит так: "Найден RtlAddFunctionTable - подозрительно". Это почти бесполезно. Он пугает, но не объясняет. Хороший finding должен разделять факт, слой, смысл, риск и необходимость динамической проверки.

```
Пример хорошей формулировки:

Finding:
    Dynamic function table API is referenced or resolved.

Layer:
    Runtime unwind metadata / dynamic code support.

Evidence:
    RtlAddFunctionTable resolved through import table or GetProcAddress.
    Executable private memory allocation pattern is present.

Meaning:
    The on-disk PE .pdata may not represent the complete function map used at runtime.

Consequence:
    Static stack/unwind analysis is incomplete without dynamic validation.

Recommended dynamic check:
    Capture function table registration.
    Record BaseAddress, EntryCount, and described ranges.
    Observe whether RIP enters registered ranges.
    If exception occurs, trace handler hit and continuation path.
```

Видишь разницу? Такой finding не делает лишних обвинений. Он объясняет, почему факт важен для анализа. Он говорит, какой слой затронут. Он прямо связывает static и dynamic . И он помогает реверсеру понять

следующий шаг.

8.5.20. Что dynamic должен сохранять как evidence

Для динамической валидации мало написать "RtlAddFunctionTable called". Нужно сохранить параметры и связать их с последующими событиями. Иначе отчёт будет красивым, но непроверяемым. Evidence - это данные, по которым другой человек может повторить твою мысль.

Минимальный набор для RtlAddFunctionTable: адрес массива FunctionTable, EntryCount, BaseAddress, список begin/end/unwind entries после чтения из памяти target-a, memory protection для code range и metadata range, module ownership или private memory classification. Для callback-таблицы: TableIdentifier, BaseAddress, Length, Callback, Context, возможно адрес DLL для out-of-process callback. Для growable table: RangeBase, RangeEnd, current count, maximum count, growth events.

После этого dynamic должен связать регистрацию с execution evidence: RIP entered registered range, exception occurred inside range, handler hit at address, continuation RIP, stack pointer before/after, thread id, first-chance/second-chance state. Тогда динамическая часть перестаёт быть логом API-вызовов и становится настоящей валидацией маршрута.

8.5.21. Главная ошибка анализа dynamic function tables

Главная ошибка - думать, что dynamic function table является отдельной экзотической темой. На самом деле это прямое продолжение всего, что мы уже изучили. У нас была функция, кадр, RSP, return address, nonvolatile-регистры, .pdata, .xdata, RtlVirtualUnwind, handler search. Dynamic table не отменяет эту модель. Она просто меняет источник данных: вместо PE-секции .pdata источник может быть runtime-registered table.

Если держать это в голове, всё становится проще. Windows всё равно хочет ответить на тот же вопрос: где функция для текущего ControlPc и как её раскрутить? Для статического модуля ответ лежит в PE. Для generated code ответ должен дать runtime. Если runtime дал честный ответ, механизм работает. Если не дал или дал ложный ответ, ломается stack walking, exception handling, debugging или становится возможным confusing control flow.

Так мы приходим к зрелой модели: static строит файловую карту и гипотезы о runtime-карте. dynamic наблюдает живую карту и проверяет маршрут. Вместе они дают не просто "подозрительно/не подозрительно", а объяснимую evidence chain: что объявлено в файле, что создано в памяти, какой код реально исполнялся, какая таблица была активна, какой handler сработал и куда ушёл RIP.

8.5.22. Ручная модель поиска: что происходит, когда RIP попал в generated code

Чтобы dynamic function tables не остались абстракцией, разберём ручную модель. Пусть поток остановился на RIP внутри области, которая не принадлежит ни одному обычному PE-модулю. Это может быть адрес вроде 00000240`10001357. Для процессора это обычный адрес инструкции. Для реверсера это вопрос: кто владеет этой памятью и как Windows сможет подняться из этого кадра к caller-у?

Первое действие - не искать сразу .pdata. Нужно определить memory region. В отладчике или в динамическом collector-е мы смотрим, к какой memory region относится адрес: private memory это или mapped image, какие protection flags, есть ли execute permission, где начало и конец региона. Если это mapped image, можно вернуться к модульной модели: ImageBase, RVA, Exception Directory. Если это private executable memory, нужно перейти к модели dynamic function table.

Второе действие - проверить, зарегистрирована ли для этой region динамическая таблица. Реверсер вручную может искать вызовы API, а dynamic должен ловить их во время выполнения. Если регистрация была, у нас появляется runtime base и диапазоны. Тогда адрес RIP можно выразить как offset от BaseAddress dynamic table. Это похоже на RVA, но это не RVA PE-файла. Лучше мысленно называть это runtime-relative offset, чтобы не спутать с RVA секции.

```
Пример ручного рассуждения:

    RIP                       = 00000240`10001357
    dynamic table BaseAddress = 00000240`10000000

    runtime-relative offset:
        0x1357 = RIP - BaseAddress

    Function entry:
        BeginAddress      = 0x1200
        EndAddress        = 0x1480
        UnwindInfoAddress = 0x3000

    Проверка:
        0x1200 <= 0x1357 < 0x1480

    Значит RIP находится внутри runtime-функции,
    а unwind info нужно искать по адресу:
        BaseAddress + 0x3000
```

Третье действие - не забыть, зачем мы всё это делаем. Мы ищем не имя функции, а способ восстановить caller context. То есть хотим понять, как из текущего RIP/RSP получить состояние вызывающего кадра: новый RIP должен стать return address caller-a, новый RSP должен указывать на стек caller-a, nonvolatile-регистры должны быть восстановлены. Dynamic function table

просто даёт Windows вход в тот же самый UNWIND_INFO-механизм, который мы уже разбирали для .xdata.

8.5.23. BaseAddress: маленький параметр, который меняет весь смысл чисел

BaseAddress в RtlAddFunctionTable кажется технической мелочью, но на деле это один из главных источников ошибок. В PE-модели мы привыкли к ImageBase: RVA 0x1234 превращается в VA через ImageBase + 0x1234. В dynamic-модели происходит похожее сложение, но BaseAddress может не быть ImageBase загруженного модуля. Это может быть начало JIT code heap, начало блока, начало manually mapped image или даже искусственно выбранная база, относительно которой runtime организовал свои entries.

Если реверсер по привычке подставит ImageBase основного EXE, он получит бессмысленные адреса. Если static формирует отчёт только в терминах RVA файлового PE, он не сможет выразить dynamic range корректно. Поэтому в отчёте нужно явно различать file_rva, module_va и runtime_relative_address. Это не педантичность. Это защита от неправильной карты.

Представь, что RUNTIME_FUNCTION.BeginAddress равен 0x1200. В статической .pdata это почти всегда читается как RVA относительно ImageBase модуля. В динамической таблице это offset относительно BaseAddress, который передал runtime. Одно и то же число 0x1200 может указывать на совершенно разные места. Поэтому finding должен хранить не только begin/end, но и base kind: module image base или dynamic table base.

```
Одна и та же запись, два разных смысла:

Статическая .pdata:
    ImageBase = 00007FF6`40000000
    BeginAddress = 0x1200
    VA = 00007FF6`40001200

Dynamic function table:
    BaseAddress = 00000240`10000000
    BeginAddress = 0x1200
    VA = 00000240`10001200

Число BeginAddress одинаковое.
Реальный адрес разный.
```

Для dynamic -а это значит, что при захвате RtlAddFunctionTable нельзя просто логировать EntryCount. Нужно сохранить BaseAddress и использовать его при декодировании каждого entry. Иначе динамическая карта в отчёте будет выглядеть красивой, но адреса будут неверными.

8.5.24. Почему callback-модель особенно сложна для отладчика

Callback-модель удобна для very dynamic code, но она создаёт тонкую зависимость между Windows, debugger-ом и runtime-ом. Когда debugger строит stack trace и доходит до адреса внутри callback-managed range, система может вызвать callback, чтобы получить RUNTIME_FUNCTION. Если это происходит внутри самого процесса, callback должен безопасно работать в условиях, когда программа уже находится в странном состоянии: исключение, остановленный поток, lock-и, возможно частично обновлённая карта кода.

Документация отдельно предупреждает о deadlock-риске между callback-функцией и code generator-ом. Deadlock - это взаимная блокировка: один участник ждёт ресурс, который удерживает другой, а другой ждёт первого или событие, которое уже не произойдёт. В контексте JIT это может выглядеть так: поток компилятора удерживает lock карты методов, в другом потоке случается исключение, unwinder вызывает callback, callback пытается взять тот же lock, но lock уже удержан остановленным или ожидающим участником. Stack walking зависает там, где должен был помочь.

Для реверсера это объясняет, почему некоторые runtime-системы очень аккуратно проектируют свои unwind callbacks, а защитные системы могут использовать callback-механику как дополнительный слой сложности. Если карта выдаётся только по запросу, то статическая реконструкция становится труднее. Если callback ведёт себя по-разному под debugger-ом, dynamic должен фиксировать не только факт регистрации, но и фактические ответы callback-а или хотя бы наблюдаемое поведение unwinding.

8.5.25. Динамическая таблица не обязана быть честной

До сих пор мы говорили о dynamic function table как о корректной инфраструктуре. Но в книге про reverse engineering нужно помнить: metadata может быть не только ошибочной, но и намеренно вводящей в заблуждение. В x64 Windows unwind metadata является договором между кодом и системой. Если программа контролирует runtime-generated code и runtime metadata, она может попытаться построить такой договор, который работает только в нужных ей сценариях или ломает анализатор в остальных.

Например, dynamic table может покрывать диапазон шире, чем реальные функции. Это создаёт впечатление, что большая область имеет корректную unwind-карту, хотя внутри могут быть data islands, encrypted chunks или dispatcher blocks. Другой вариант - слишком узкие ranges, где отдельные переходы попадают в uncovered code. Третий вариант - несколько функций с одинаковым UNWIND_INFO, хотя их прологи различаются. Четвёртый вариант - handler RVA или language-specific data ведут к stub-у, который дальше вручную меняет ContextRecord.

Но здесь нельзя становиться параноиком. Один и тот же UNWIND_INFO для нескольких функций может быть нормальной оптимизацией, если функции имеют одинаковый пролог. Широкий range может быть нормальным для

runtime-generated template. Callback может быть нормальной частью JIT. Поэтому правильный подход снова evidence-based: не "динамическая таблица значит обман", а "динамическая таблица расширяет поверхность проверки; нужно сравнить объявленную модель с реальным выполнением".

8.5.26. Как dynamic может проверять честность dynamic unwind map

dynamic не обязан полностью доказывать математическую корректность каждой динамической таблицы. Но он может проверять ключевые связки. Первая связка: registration -> memory region. Таблица должна описывать существующую executable region или region, которая станет executable к моменту выполнения. Если BaseAddress указывает в несуществующую память или unwind info уходит в freed memory, это сильный сигнал.

Вторая связка: function entry -> actual code. Если entry говорит BeginAddress/EndAddress, dynamic может проверить, что RIP действительно входит в этот range при выполнении, и снять несколько байтов кода вокруг начала функции. Если пролог виден, его можно сравнить с UNWIND_INFO примерно так же, как это делает static для статического кода, только источником будет memory snapshot, а не файл.

Третья связка: unwind metadata -> handler route. Если entry имеет EHANDLER/UHANDLER или handler-like target, dynamic может поставить breakpoint на handler или на известный runtime stub. Если exception происходит внутри dynamic range, handler hit подтверждает, что route не просто теоретический. A RIP trace после handler-a показывает, вернулось ли выполнение назад, ушло ли в recovery block, завершилось ли process termination.

Четвёртая связка: lifetime -> use. Если таблица удалена, но код продолжает выполняться, это опасный сигнал. Если код освобождён, но таблица остаётся зарегистрированной, это тоже сигнал. Если таблица регистрируется только на короткое окно вокруг выполнения, это нужно отметить не как автоматическую злонамеренность, а как dynamic behavior requiring careful review.

8.5.27. Почему static-only отчёт здесь честно должен признавать неполноту

Хороший static не должен притворяться dynamic -ом. Если анализ был только статическим, в отчёте по dynamic function tables нужно честно писать: "on-disk .pdata does not include runtime-generated code; dynamic behavior was not observed in this run" или "dynamic validation not performed". Это не слабость инструмента, а честность модели.

В reverse engineering очень вредно, когда инструмент выдаёт уверенный verdict там, где он видел только часть evidence. Dynamic code - как раз такой

случай. Статически можно увидеть подготовку, imports, strings, wrappers, suspicious memory patterns. Но реальная таблица, реальные BaseAddress, реальные handler hits и реальные RIP traces появляются только в процессе. Поэтому отчёт должен различать confidence: static hypothesis имеет один уровень уверенности, dynamic confirmation - другой.

Для книги это важный философский момент: теория нужна не для того, чтобы написать больше проверок, а для того, чтобы знать границы своих выводов. Если ты понимаешь, почему dynamic function table не лежит в PE, ты не будешь требовать от static невозможного. И если ты понимаешь, что dynamic видит живой маршрут, ты не будешь заменять динамическую валидацию догадкой.

8.5.28. Практический словарь раздела

Закрепим термины медленно. Dynamic function table - это таблица диапазонов функций, зарегистрированная в процессе во время выполнения, а не взятая из .pdata PE-файла. Runtime-generated code - машинный код, созданный программой после запуска. JIT - runtime, который компилирует промежуточное представление в машинный код прямо перед использованием или по мере необходимости. BaseAddress - база, от которой считаются относительные адреса dynamic RUNTIME_FUNCTION entries. Callback - функция, которую Windows может вызвать, чтобы получить runtime function entry для адреса в managed range. Managed range - область памяти, за которую отвечает callback или dynamic table.

Unwind reliability - надёжность раскрутки стека. Она означает, что Windows, debugger или crash reporter могут корректно восстановить caller context из текущего контекста. Если dynamic code не имеет корректной function table, эта надёжность падает. Handler route - фактический путь исключения: где оно возникло, какая таблица была использована, какой handler был вызван, как изменился CONTEXT, куда ушёл RIP. static строит карту и гипотезы. dynamic подтверждает или опровергает путь в живом процессе.

8.5.29. Переход к следующему разделу

Dynamic function tables показывают, что unwind-карта может быть живой, изменяемой и зависящей от runtime-а. Следующий шаг - посмотреть на исключения как на сознательный инструмент управления потоком. До сих пор исключение часто выглядело как авария или редкий путь обработки ошибки. Но в защищённом или обфусцированном коде exception может быть частью нормальной логики: breakpoint exception, guard page, deliberate access violation, RaiseException, single-step, handler-driven jump. Чтобы понять такие трюки, нам уже нужен весь фундамент: RIP, RSP, return address, CONTEXT, .pdata/.xdata, dynamic tables, static и dynamic как две половины одной проверки.

Техническая база раздела

Раздел опирается на официальную документацию Microsoft по x64 exception handling, где описано, что для dynamically generated functions runtime должен предоставить сведения операционной системе через RtlInstallFunctionTableCallback или RtlAddFunctionTable; на документацию RtlAddFunctionTable, где объясняется добавление dynamic function table и параметр BaseAddress; на документацию RtlInstallFunctionTableCallback, где описана callback-модель для very dynamic code; на документацию RtlDeleteFunctionTable и growable function table APIs, где раскрывается время жизни и рост динамических таблиц.

8.6. Исключения как anti-debug и control-flow trick: когда авария становится маршрутом управления

До этого места мы долго строили спокойную, почти инженерную картину: у функции есть диапазон, у диапазона есть RUNTIME_FUNCTION, у RUNTIME_FUNCTION есть ссылка на UNWIND_INFO, а UNWIND_INFO объясняет Windows, как восстановить кадр. Это была модель корректного сотрудничества между компилятором, PE-файлом, загрузчиком, операционной системой, отладчиком и crash reporter-ом. Теперь нужно аккуратно сделать следующий шаг: те же самые механизмы можно использовать не только для восстановления после ошибки, но и как часть логики программы.

Эта мысль часто неприятна новичку, потому что слово exception, исключение, кажется почти синонимом аварии. В обычном программировании так и объясняют: исключение - это что-то пошло не так. Деление на ноль, доступ по плохому указателю, illegal instruction, нарушение доступа к памяти, breakpoint, stack overflow. Но в Windows exception-механизм шире. Он действительно обслуживает аварии, но технически он является ещё и механизмом доставки события: процессор или программа создают исключительную ситуацию, Windows строит EXCEPTION_RECORD и CONTEXT, затем запускает маршрут доставки: debugger first chance, VEH, frame-based SEH, continue handlers, second chance, unhandled path. Если у программы есть обработчик, этот обработчик может не только записать лог и умереть. Он может принять решение продолжить выполнение.

Вот из этой возможности и рождается exception-based control flow. Control flow - поток управления, то есть ответ на вопрос: какая инструкция будет выполнена следующей и почему. В обычном коде поток управления виден прямо: jmp, call, ret, conditional jump. В exception-based control flow часть перехода прячется не в обычной инструкции перехода, а в маршруте обработки исключения. Код намеренно создаёт исключение, Windows вызывает обработчик, обработчик смотрит на состояние потока и иногда меняет CONTEXT, например RIP или RSP, а затем возвращает решение продолжить выполнение. Для процессора это выглядит как будто управление каким-то образом “перепрыгнуло” в другое место. Для реверсера это ловушка: если смотреть только на обычный control-flow graph, можно не увидеть настоящую дорогу.

> Главная идея раздела: исключение не всегда означает crash. Иногда исключение - это дверь в скрытый маршрут управления. `static` должен увидеть статические следы такой двери. `dynamic` должен проверить, открывается ли она во время выполнения и куда после неё реально уходит RIP.

8.6.1. Сначала обычная модель: как функция уходит без исключений

Чтобы понять exception-trick, нужно ещё раз медленно вспомнить нормальную модель. Функция вызывается инструкцией call. Call делает две вещи: передаёт управление в callee, то есть в вызываемую функцию, и кладёт на стек return address - адрес, куда нужно вернуться после завершения callee. Return address по-русски можно назвать адресом возврата: это не абстрактная метка, а конкретное число в памяти стека. Потом callee выполняет работу, возможно двигает RSP, сохраняет nonvolatile-регистры, вызывает другие функции, а в конце приходит к ret. Инструкция ret берёт return address со стека и загружает его в RIP. Так управление возвращается caller-у, то есть вызывавшей функции.

В этой нормальной модели реверсер видит почти прямую механику. Есть call - значит есть переход в функцию и адрес возврата. Есть ret - значит функция закончилась и должна вернуться туда, куда положил call. Есть условный переход - значит поток управления делится на две ветки. Есть jmp - значит управление безусловно переходит по адресу. Да, оптимизации могут усложнять картину: tail call может заменить ret на jmp, inline может убрать отдельный вызов, compiler может переставить блоки. Но всё равно переходы обычно представлены инструкциями управления потоком.

Exception-based flow ломает эту привычку. В нём следующая инструкция может определяться не только локальной инструкцией jmp или ret, а решением exception dispatcher-а и обработчика. На поверхности мы видим, например, int 3 или доступ к guard page. Обычный дизассемблер показывает: здесь инструкция вызывает исключение. Но куда управление попадёт после неё, зависит от обработчиков, от наличия debugger-а, от first-chance политики, от return value handler-а, а иногда от того, как handler поменял CONTEXT. Поэтому такой переход нельзя понимать глазами обычного CFG без exception edges.

Exception edge - это ребро графа управления, которое возникает не из обычной инструкции перехода, а из возможности исключения. Edge значит ребро, связь между базовыми блоками. В обычном CFG ребро идёт из блока с conditional jump в два возможных блока. В exception-aware CFG нужно добавить ещё один тип ребра: из инструкции, которая может создать исключение, в обработчик или в продолжение после обработки. Это уже не бытовая “ошибка”, а отдельная дорога исполнения.

8.6.2. Почему Windows вообще позволяет продолжить выполнение после исключения

Может возникнуть вопрос: зачем вообще разрешать продолжение после исключения? Почему система не должна сразу завершать процесс? Ответ в том,

что не каждое исключение означает необратимую катастрофу. Некоторые исключения являются частью нормального взаимодействия программы, отладчика и операционной системы. Breakpoint exception используется debugger-ом, single-step exception используется для пошагового исполнения, guard page exception может служить сигналом, что поток впервые коснулся защищённой страницы. В некоторых случаях обработчик может исправить ситуацию: выделить страницу, изменить защиту памяти, скорректировать состояние и продолжить.

В терминах Windows обработчик может вернуть разные решения. Мы уже обсуждали EXCEPTION_CONTINUE_SEARCH: “я не обработал, ищите дальше”. EXCEPTION_EXECUTE_HANDLER: “я обработаю через handler block, идём по обычной SEH-ветке обработки”. И EXCEPTION_CONTINUE_EXECUTION: “я обработал ситуацию, можно продолжать выполнение из текущего или изменённого контекста”. Последний вариант особенно важен для этой темы. Если обработчик возвращает continue execution, он фактически говорит системе: процесс можно продолжить, crash не нужен, поиск дальше не нужен.

Но продолжить выполнение можно по-разному. Если CONTEXT не меняли, продолжение может попытаться снова выполнить ту же инструкцию. Иногда это нужно: например guard page был снят системой, повторный доступ уже пройдёт. Иногда это приведёт к бесконечному циклу исключений: инструкция снова падает, handler снова возвращает continue execution, и процесс застревает. Если же handler меняет RIP, он может направить выполнение в другое место. Вот здесь exception уже становится маршрутизатором управления.

```
Упрощённая мыслительная схема:

    instruction causes exception
        -> Windows creates EXCEPTION_RECORD + CONTEXT
        -> debugger first chance notification
        -> VEH / SEH handler sees CONTEXT
        -> handler may modify ContextRecord->Rip
        -> handler returns EXCEPTION_CONTINUE_EXECUTION
        -> thread resumes at the new RIP

Для реверсера главный вопрос:
    где был RIP до исключения,
    кто получил управление как handler,
    что стало с RIP после обработки.
```

В этом фрагменте важно слово CONTEXT. CONTEXT - это снимок состояния потока: регистры, RIP, RSP, flags и другие части архитектурного состояния. Если обычный call/ret работает через стек и return address, то exception continuation может работать через изменение CONTEXT. Это не отменяет стек, но добавляет другой слой управления: не “ret взял адрес со стека”, а “dispatcher восстановил поток из изменённого context”. Для dynamic -а это буквально центральная вещь: показать, куда ушёл RIP после handler-а.

8.6.3. Anti-debug: что означает этот термин без мифологии

Теперь аккуратно введём термин anti-debug. Anti-debug - это набор приёмов, которые меняют поведение программы при наличии отладчика или затрудняют анализ под отладчиком. Anti значит “против”, debug значит “отладка”. Но важно не превратить это слово в страшилку. Не каждый вызов IsDebuggerPresent, не каждый int 3, не каждое first-chance exception автоматически означает malware. Легитимные программы тоже могут проверять наличие debugger-а для diagnostics, licensing, crash reporting, telemetry, developer assertions. Debug build может содержать breakpoint-инструкции. Runtime может использовать исключения как часть нормальной инфраструктуры.

Для этой книги anti-debug важен не как ярлык “плохое”, а как аналитическая категория. Если программа специально создаёт исключение, потому что знает, что debugger увидит его раньше SEH/VEH, она может использовать различие между “под debugger-ом” и “без debugger-а” как условие ветвления. Под debugger-ом исключение может остановить процесс, быть обработано debugger-ом или изменить timing. Без debugger-а оно может спокойно пройти в VEH/SEH handler. Тогда одна и та же инструкция становится датчиком среды.

Debug-aware behavior - поведение, зависящее от отладчика. Это более нейтральное слово, чем anti-debug. Например программа может сказать: если debugger есть, включить подробные логи. А может сказать: если debugger есть, уйти в ложную ветку. Для static -а оба случая могут выглядеть похожими: imports, checks, exceptions. Разница появляется в контексте: что проверяется, куда уходит управление, есть ли ложные пути, есть ли handler-driven control flow, есть ли попытка сломать stack walk или запутать exception route. dynamic здесь нужен именно потому, что статический признак сам по себе недостаточен.

8.6.4. First chance и second chance как граница между debugger-ом и обработчиками программы

Чтобы понять anti-debug через исключения, нужно снова вспомнить порядок доставки исключения. Когда возникает исключение в процессе, который отлаживается, Windows сначала уведомляет debugger. Это first-chance notification - первая возможность обработать исключение. First chance не значит “первая ошибка” и не значит “процесс уже падает”. Это значит: debugger получил шанс увидеть событие до того, как обычные обработчики процесса начнут его обрабатывать.

Если debugger говорит системе, что он не обработал исключение, Windows продолжает обычный поиск обработчиков: VEH, frame-based SEH, unwind/search logic. Если никто не обработал исключение, debugger получает

ещё одно уведомление - second chance. Second chance уже означает: программа сама не справилась, и если debugger тоже не вмешается, процесс обычно будет завершён. Поэтому в логах отладчика first-chance exception часто выглядит пугающе, но сам по себе он не равен crash. Это просто событие на раннем этапе маршрута.

Для exception-based control flow это различие критично. Программа может специально создавать исключение, ожидая, что без debugger-а оно дойдёт до её VEH handler-а и будет обработано. Но под debugger-ом поведение меняется: debugger остановится на first chance, пользователь увидит breakpoint или access violation, автоматизация может продолжить не тем флагом, timing изменится, а иногда debugger может “съесть” исключение. Поэтому правильная динамическая валидация должна не просто записать “exception happened”, а понять: first chance или second chance, кто продолжил, дошло ли до handler-а, вернулся ли поток и куда.

```
Одна и та же инструкция может дать разные наблюдения:

    без debugger-а:
        int 3 -> VEH -> handler changes RIP -> continue execution

    под debugger-ом:
        int 3 -> first-chance debugger event -> debugger policy decides

    при неправильной политике продолжения:
        handler may not run, or route may differ
```

В твоём проекте dynamic как раз должен быть осторожен с этим местом. Если он сам запускает target под Debug API, он уже является участником маршрута. Он не просто “смотрит со стороны”. Он получает EXCEPTION_DEBUG_EVENT и должен продолжить процесс так, чтобы не уничтожить тот маршрут, который он пытается измерить. Это тонкая мысль: динамический валидатор может своим присутствием изменить динамику. Поэтому в отчёте важна не только строка “exception seen”, но и политика обработки debug event: событие передано дальше, обработано debugger-ом, дошло до user handler-а или нет.

8.6.5. Breakpoint exception: int 3 как сигнал, ловушка и обычный инструмент

Самый известный пример - breakpoint exception. На x86/x64 короткая инструкция int 3 кодируется байтом CC и традиционно используется debugger-ами для software breakpoints. Debugger хочет остановить программу на конкретном адресе. Он временно заменяет байт инструкции на CC. Когда процессор выполняет int 3, возникает breakpoint exception. Debugger получает событие, восстанавливает исходный байт, корректирует RIP при необходимости и показывает пользователю остановку.

Но если int 3 лежит в программе изначально, смысл зависит от контекста. Это может быть __debugbreak в debug build. Это может быть assert-путь, который разработчик оставил намеренно. Это может быть padding/trap в unreachable code. Это может быть контрольная точка для crash diagnostics. А может быть intentional exception, намеренно созданное исключение, которое должно попасть в VEH handler и превратиться в скрытый переход. Поэтому задача реверсера не сказать “CC значит anti-debug”, а задать правильные вопросы.

Первый вопрос: есть ли рядом зарегистрированный VEH или SEH handler, который ожидает EXCEPTION_BREAKPOINT или STATUS_BREAKPOINT? Второй вопрос: что handler делает с ContextRecord? Он просто логирует и возвращает CONTINUE_SEARCH? Или меняет RIP, RSP, EFlags? Третий вопрос: есть ли normal edge из этого места, или единственный живой маршрут проходит через exception? Четвёртый вопрос: меняется ли поведение под debugger-ом? Пятый вопрос: видит ли dynamic handler hit после этого int 3?

Для static -а int 3 - это статический факт в коде. Он может пометить место как deliberate breakpoint site только с некоторой уверенностью, если видит рядом обработчики, сравнение exception code, регистрацию VEH, вызовы AddVectoredExceptionHandler, SetUnhandledExceptionFilter или странный control-flow вокруг места. Для dynamic -а этот же int 3 становится проверяемой гипотезой: при выполнении target действительно пришёл EXCEPTION_BREAKPOINT, затем был hit конкретный handler, затем RIP ушёл в такой-то диапазон.

8.6.6. Single-step exception и Trap Flag: управление через один шаг

Single-step exception - ещё один механизм, который легко звучит как “тема отладчика”, но может участвовать в control-flow tricks. Single-step означает пошаговое выполнение: процессор выполняет одну инструкцию и затем генерирует исключение, чтобы debugger или handler получил управление. На x64 это связано с Trap Flag в регистре RFLAGS. Flag - это бит состояния. Trap Flag, если включён, заставляет процессор после инструкции создать debug exception.

Зачем это может быть нужно нормальной системе? Debugger использует single-step, чтобы выполнить одну инструкцию и снова остановиться. Это обычная функция “step into/step over” на низком уровне. Но программа тоже может играть с этим состоянием: установить Trap Flag, выполнить инструкцию, получить exception, попасть в handler, принять решение, куда двигаться дальше. Опять же, сама идея не равна malware. Но для reverse engineering это важный сигнал: поток управления может быть разбит на микрошаги, а handler может становиться скрытым диспетчером.

Слово dispatcher мы уже употребляли в главе про исключения. Dispatcher - диспетчер, механизм, который доставляет событие подходящему обработчику. В exception-based trick обработчик сам может стать мини-диспетчером: он смотрит на текущий RIP, exception code, flags, внутреннее состояние программы и выбирает следующий RIP. Тогда обычный CFG превращается в неполную карту: настоящая логика лежит не в явных jmp, а в таблице решений внутри handler-а.

Для static -а single-step trick статически увидеть сложнее, чем int 3, потому что явной инструкции CC может не быть. Нужно смотреть на работу с flags, на pushfq/popfq, на инструкции, меняющие RFLAGS через стек, на handler-ы, которые проверяют STATUS_SINGLE_STEP, на необычные возвраты CONTINUE_EXECUTION. Для dynamic -а это более естественно: он может наблюдать поток debug exceptions, повторяющиеся handler hits, изменение RIP между событиями и trace, который не совпадает с простым дизассемблерным маршрутом.

8.6.7. Guard pages: страница памяти как одноразовая сигнализация

Guard page - охраняемая страница памяти. Guard значит “сторожевой”, “охранный”. В Windows у страницы памяти можно выставить модификатор PAGE_GUARD. При первом доступе к такой странице система создаёт STATUS_GUARD_PAGE_VIOLATION и снимает guard-состояние. Поэтому guard page можно думать как об одноразовой сигнализации: кто-то коснулся страницы - сработал сигнал - дальше страница уже обычная, если guard не поставить снова.

Зачем это нужно легитимно? Например для роста стека: когда поток подходит к guard page внизу стека, система понимает, что стек нужно расширить. Также guard pages могут использоваться для memory debugging, lazy initialization, monitoring доступа, защитных механизмов. Но этот же механизм может использоваться как control-flow trick. Программа ставит guard на страницу с кодом или данными, затем намеренно обращается к ней, получает exception, handler понимает, что произошло, и выбирает дальнейшую ветку.

Здесь особенно полезно не терять связь с RSP/RIP. Если guard page стоит на данных, исключение может возникнуть на инструкции чтения или записи. RIP указывает на инструкцию, которая тронула страницу. EXCEPTION_RECORD содержит код исключения и дополнительные сведения. CONTEXT содержит состояние потока. Handler может снять или восстановить guard, изменить внутреннее состояние, затем вернуть CONTINUE_EXECUTION. При повторном выполнении инструкция может пройти, потому что guard уже снят, или снова сработать, если handler снова установил guard.

```
Guard page как модель:
```

```
    page has PAGE_GUARD
    instruction reads/writes/executes memory in that page
        -> STATUS_GUARD_PAGE_VIOLATION
        -> system clears guard modifier
        -> handler may inspect address and context
        -> execution may continue

Почему это tricky:
    второй доступ к той же странице уже может не дать exception,
    если handler специально не поставит PAGE_GUARD снова.
```

Для static -a guard-based logic может проявляться через VirtualProtect или похожие функции управления защитой памяти, через PAGE_GUARD constants, через handler, который проверяет exception code, через код, который переустанавливает защиту страницы. Но опять же: VirtualProtect сам по себе не злой. Почти любая программа, работающая с JIT, hooks, memory management, sandboxing или instrumentation, может менять protection. Сигнал появляется в связке: PAGE_GUARD плюс exception handler плюс CONTINUE_EXECUTION плюс изменение route.

dynamic здесь особенно ценен. Статика может сказать: возможно, здесь guard-page based dispatch. Динамика может показать: действительно был STATUS_GUARD_PAGE_VIOLATION, handler hit произошёл, после handler-a RIP продолжил выполнение не так, как ожидал обычный CFG, guard page была снята или поставлена снова, а trace подтвердил, что это не мёртвый код.

8.6.8. Deliberate access violation: намеренная ошибка доступа как переход

Access violation - нарушение доступа к памяти. В обычной жизни это самый узнаваемый crash: программа прочитала, записала или попыталась выполнить память, к которой у неё нет подходящего доступа. Например чтение по null pointer или запись в read-only page. Но access violation тоже может быть создан намеренно. Deliberate значит намеренный. Deliberate access violation - это ситуация, где код специально делает доступ, который должен упасть, чтобы попасть в обработчик.

Звучит грубо, но с точки зрения механики это тот же принцип: исключение создаёт событие, событие доставляется обработчику, обработчик решает, что делать. Например handler может увидеть, что faulting address равен заранее ожидаемому значению, понять это как команду, изменить RIP и продолжить. Или handler может использовать разные адреса как разные “опкоды” внутренней виртуальной машины: доступ к одному адресу означает одну ветку, к другому - другую. Это уже похоже на виртуализацию control flow, но нам сейчас важен фундамент: переход спрятан в exception route.

Реверсеру здесь нужно быть осторожным в обе стороны. Нельзя каждую access violation считать намеренной: большинство access violation - просто баги, null pointer, use-after-free, неправильная обработка данных. Но нельзя и

автоматически считать access violation тупиком. Если вокруг есть VEH/SEH, если handler проверяет ExceptionInformation, если после exception поток продолжает работу, если dynamic видит повторяемый маршрут, значит это может быть deliberate mechanism.

В Windows EXCEPTION_RECORD для access violation содержит дополнительные параметры: тип доступа и адрес, к которому пытались обратиться. Это удобно для легитимной диагностики, но также удобно для handler-driven logic. Handler может смотреть не только на ExceptionCode, но и на ExceptionInformation. Поэтому static , разбирая handler, должен искать не просто сравнение exception code, а работу с полями ExceptionRecord и ContextRecord. dynamic должен подтвердить, какие значения реально приходят в runtime.

8.6.9. Illegal instruction и UD2: когда невозможная инструкция становится сигналом

Illegal instruction - недопустимая инструкция. Процессор пытается декодировать или выполнить байты, которые в данном режиме не являются допустимой инструкцией, и возникает исключение. В x86/x64 есть инструкция UD2, специально предназначенная для генерации invalid opcode exception. Компиляторы и runtime-ы могут использовать такие trap-инструкции для unreachable code, assertions, sanitizer paths, abort paths. Поэтому UD2 не надо автоматически читать как обфускацию.

Но как и int 3, invalid instruction может быть deliberate exception site. Если программа ставит рядом handler, который ожидает illegal instruction, то невозможная инструкция превращается в метку маршрута: сюда нельзя прийти обычным путём и продолжить без обработки, но можно прийти через exception dispatcher. Это даёт автору кода способ спрятать переход от простого дизассемблера. Дизассемблер видит “падает”. Реальный процесс после handler-а может идти дальше.

Отличие от breakpoint в том, что debugger-ы особенно ожидают breakpoint и single-step events. Illegal instruction обычно выглядит более аварийно. Поэтому такой приём может создавать шум в анализе: автоматический инструмент считает участок dead/crash path, а на самом деле handler оживляет его. Здесь снова нужна связка: static видит invalid opcode site и handler metadata; dynamic проверяет, был ли exception обработан и куда ушёл RIP.

8.6.10. RaiseException: исключение, созданное не процессором, а самой программой

До сих пор мы говорили о ситуациях, где исключение рождается из выполнения инструкции: int 3, single-step, guard page access, bad memory access, illegal instruction. Но Windows даёт программе и прямой способ создать

исключение программно - RaiseException. Raise значит “поднять”, “сгенерировать”. RaiseException создаёт exception event в текущем потоке с заданным exception code и параметрами. Это не баг процессора, а сознательный вызов API.

RaiseException легитимно используется runtime-ами, языковыми исключениями, внутренними протоколами, диагностикой, тестами crash handling. Но в reverse engineering он важен потому, что делает exception route явным. Если мы видим вызов RaiseException, надо спросить: какой code передаётся? Это стандартный код или пользовательский? Есть ли рядом handler, который ждёт именно этот code? Что происходит после обработки? Возвращается ли управление обычным путём или handler меняет context?

Custom exception code - пользовательский код исключения. Windows позволяет программе передать свой exception code, и handler может использовать его как внутреннее сообщение. Например один код значит “перейти в slow path”, другой - “сработал виртуальный opcode”, третий - “диагностический сигнал”. Это не обязательно плохо. Но если программа использует RaiseException как замену обычным переходам, то перед нами exception-based control flow.

```
Обычный переход:
    cmp state, 3
    je  target_block

Exception-based route:
    RaiseException(CUSTOM_CODE, ...)
        -> handler checks ExceptionCode
        -> handler decides next RIP or handler block
        -> execution continues elsewhere
```

Для static -а вызов RaiseException - сильный статический признак, но не финальный вывод. Нужно связать его с обработчиками: VEH, SEH, UEF, __C_specific_handler, filter expressions. Нужно посмотреть, что происходит с возвращаемым маршрутом. Для dynamic -а задача яснее: поймать exception event, записать code, проверить first/second chance, handler hit, изменение RIP, продолжение или завершение процесса.

8.6.11. Handler как скрытый диспетчер: почему главный код может лежать не там, где мы смотрим

Теперь сведём примеры в одну модель. Int 3, guard page, deliberate access violation, RaiseException - это разные способы создать событие. Но если после них управление попадает в один и тот же handler, который смотрит на exception code, faulting RIP, faulting address, flags, внутреннее состояние и выбирает следующий адрес, то этот handler становится скрытым dispatcher-ом. Он не просто “обработал ошибку”. Он распределяет выполнение по маршрутам.

Для обычного reverse engineering это меняет место внимания. Мы привыкли искать main logic в функциях, которые вызываются напрямую. Но здесь часть логики может быть сосредоточена в обработчике исключений. Причём обработчик может быть зарегистрирован далеко от места исключения: VEH регистрируется на уровне процесса, SEH logic может быть привязана к функции через .xdata и __C_specific_handler, UEF может стоять как last chance. Поэтому “рядом с int 3 ничего интересного” не означает “ничего интересного нет”. Нужно найти того, кто получает исключение.

Здесь особенно важна связь с главой 6. VEH не frame-based: он не привязан к конкретному кадру стека, вызывается раньше frame-based SEH и может видеть исключения из разных мест процесса. SEH на x64 связан с .pdata/.xdata и protected ranges: handler участвует только для определённых диапазонов кода. Top-level filter находится ещё позже, когда обычная обработка не справилась. Если реверсер не различает эти слои, он легко перепутает: процессный router через VEH, локальный __try/__except через __C_specific_handler и crash reporter через SetUnhandledExceptionFilter.

static должен строить карту этих слоёв: где зарегистрирован VEH, какие функции имеют EHANDLER, какие protected ranges покрывают exception sites, где есть RaiseException/int 3/guard access, какие handlers потенциально получают события. dynamic должен затем превратить карту в живое наблюдение: это исключение действительно пошло в этот handler, handler действительно вернул такое решение, RIP действительно ушёл туда.

8.6.12. Почему это не всегда злонамеренно: диагностика, runtime и defensive engineering

Очень важно не воспитать у себя плохую привычку: увидел exception trick - написал malware. Это примитивный анализ. Исключения как управление встречаются и в нормальных системах. Runtime языка может использовать исключения для rare paths. Debug-инфраструктура может использовать breakpoints. Защитный код может использовать guard pages для обнаружения нежелательного доступа. Санитайзеры, профилировщики, instrumented builds, sandboxing и exploit mitigations тоже могут создавать необычные exception маршруты. Даже твой dynamic по сути будет использовать debugger events и breakpoints, чтобы наблюдать выполнение.

Разница между нормой и подозрением появляется из цели, контекста и структуры. Диагностический handler обычно старается объяснить ошибку, записать dump, логировать code/address, завершить или безопасно продолжить. Runtime handler обычно имеет понятную связь с языковой моделью или JIT-кодом. Obfuscation handler часто пытается скрыть route, размазать логику, создать ложные ветки, зависеть от debugger-а, ломать stack trace или менять context так, чтобы статическая CFG-карта была бесполезной. Это не математически абсолютное различие, но это хорошая аналитическая

рамка.

Поэтому в книге и в проекте нужно избегать языка “обнаружено = вредоносно”. Лучше говорить evidence, confidence, consequence. Evidence - доказательство или наблюдаемый факт. Confidence - насколько мы уверены в интерпретации. Consequence - что это означает для анализа: обычный stack walk может быть неполным, control-flow graph требует exception edges, динамический запуск нужен для подтверждения, route зависит от debugger policy.

8.6.13. Как static должен видеть exception-based control flow

static работает статически, то есть читает файл и код без исполнения. Поэтому он не может честно сказать: “этот handler точно сработал”. Но он может построить карту подозрительных возможностей. Сначала он видит exception sites: int 3, ud2, вызовы RaiseException, обращения к VirtualProtect/PAGE_GUARD, потенциальные deliberate memory faults. Потом он ищет receivers - получателей исключений: AddVectoredExceptionHandler, AddVectoredContinueHandler, SetUnhandledExceptionFilter, EHANDLER/UHANDLER в UNWIND_INFO, __C_specific_handler, scope tables, filters.

Затем static должен связать sites и receivers. Это не всегда просто. VEH может получить исключение из любого места процесса, поэтому связь будет процессной и вероятностной. Frame-based SEH можно связать точнее: если RIP exception site попадает в protected range, значит этот __try/__except потенциально участвует. UEF - last chance, поэтому его нельзя считать нормальным маршрутизатором раннего управления без дополнительного evidence. Если handler читает ExceptionRecord->ExceptionCode, ContextRecord->Rip, ExceptionInformation или меняет context, confidence повышается.

Но хороший static не должен шуметь. Один int 3 в debug секции - низкая важность. RaiseException без CONTINUE_EXECUTION и с явным crash path - возможно диагностика. PAGE_GUARD в коде JIT runtime - может быть нормой. А вот связка “много deliberate exception sites + ранний VEH + handler меняет RIP + обычный CFG имеет разрывы + stack/unwind metadata выглядит странно” уже сильнее похожа на exception-driven control flow.

Пример evidence chain для static :

```
static site:
    0x140012340: int 3

possible receiver:
    AddVectoredExceptionHandler(First=1, Handler=0x140030000)

handler behavior:
    reads ExceptionRecord->ExceptionCode
```

```
        reads/writes ContextRecord->Rip
        returns EXCEPTION_CONTINUE_EXECUTION

    interpretation:
        possible exception-based control flow

    required validation:
        run dynamic     and confirm handler hit + post-handler RIP trace
```

Такой вывод намного лучше, чем “anti-debug found”. Он показывает, что именно найдено, на каком слое, почему это имеет смысл, и чего не хватает для уверенности. Это и есть переход от “угадайки” к инженерному анализу.

8.6.14. Как dynamic должен проверять живой маршрут

dynamic в твоём проекте - это динамическая валидация. Для этого раздела его смысл особенно ясен. static может сказать: здесь есть потенциальная скрытая дорога через exception. dynamic должен ответить: эта дорога реально открылась или нет. Для этого ему нужно наблюдать не только факт исключения, но и маршрут.

Маршрут исключения можно представить как последовательность событий. Сначала поток достигает exception site. Затем возникает exception event. Если процесс отлаживается, dynamic как debugger получает first-chance notification. Затем в зависимости от политики продолжения событие либо передаётся дальше, либо считается обработанным debugger-ом. Если передано дальше, могут сработать VEH/SEH. dynamic должен поставить точки наблюдения так, чтобы увидеть handler hit. После handler-а он должен посмотреть, куда ушёл RIP. Это и есть ответ на твою формулировку: куда уходит функция и куда уходит обработка исключения.

Тонкость в том, что dynamic не должен ломать маршрут своим наблюдением. Если он поставил breakpoint на handler, он сам создаёт breakpoint exception. Если он продолжил не тем кодом, программа может пойти не туда. Если он слишком грубо вмешался в first-chance exception, он может не дать user handler-у сработать. Поэтому динамическая валидация должна быть не просто “запустили и посмотрели”, а аккуратной процедурой наблюдения: какие события мы породили сами, какие пришли от target, какие breakpoints являются instrumentation, какие exceptions являются subject of analysis.

Отчёт dynamic -а должен отвечать на практические вопросы: был ли достигнут exception site; какой ExceptionCode был получен; first chance или second chance; был ли hit VEH/SEH/UEF handler; изменился ли RIP после handler-а; остался ли RSP согласованным; дошёл ли поток до expected continuation; повторяется ли маршрут стабильно. Только тогда можно говорить, что статическая гипотеза static -а подтверждена или опровергнута.

8.6.15. Что значит “куда уходит функция” в присутствии исключений

Ты отдельно уточнил, что dynamic должен показывать, куда уходит функция и обработка исключения. В обычном коде “куда уходит функция” часто означает: какой call был выполнен, какой ret вернул управление, какой branch выбран. Но при исключениях вопрос шире. Функция может не уйти через ret. Она может попасть в handler. Handler может выполнить cleanup. Handler может передать поиск дальше. Handler может изменить CONTEXT и продолжить в другом basic block. Handler может привести к second chance и завершению процесса. Поэтому route функции включает не только ordinary edges, но и exception edges.

Если функция содержит deliberate exception site, её настоящий выход может быть спрятан. В дизассемблере блок заканчивается “падающей” инструкцией. Декомпилятор может показать unreachable или abort. Но в runtime handler меняет RIP на адрес продолжения. Тогда функция фактически имеет скрытый выход, который не выглядит как ret/jmp. Для static -а это статическая проблема реконструкции CFG. Для dynamic -а - динамическая задача трассировки: показать post-exception RIP.

Здесь снова возвращаемся к фундаменту. RIP - где поток выполняется сейчас. RSP - где находится верх стека. Return address - куда ret вернул бы управление при обычном выходе. CONTEXT - снимок этих значений при исключении. Если handler меняет RIP, он меняет не return address на стеке, а будущую точку продолжения потока. Если handler меняет RSP, он может ещё сильнее изменить модель кадра. Поэтому dynamic должен смотреть не только на адреса кода, но и на согласованность стека.

8.6.16. Типичные ошибки реверсера

Первая ошибка - считать first-chance exception crash-ем. В отладчике всё красное, процесс остановился, значит “упал”. Нет. First chance часто означает: debugger получил раннее уведомление, а программа ещё может обработать исключение и жить дальше. Особенно это важно для runtime-ов и exception-based control flow.

Вторая ошибка - смотреть только на место исключения и не искать handler. Int 3 сам по себе почти ничего не объясняет. Access violation сам по себе может быть багом. Guard page сам по себе может быть нормой. Смысл появляется только после вопроса: кто получит это исключение и что сделает с CONTEXT?

Третья ошибка - путать anti-debug и diagnostics. Проверка debugger-а, breakpoint или RaiseException могут быть частью нормального debug/crash пути. Чтобы назвать поведение anti-debug, нужны дополнительные признаки: зависимость маршрута от debugger-а, ложные ветки, препятствие анализу, hidden control flow, намеренное искажение stack walking, странная реакция на

instrumentation.

Четвёртая ошибка - думать, что статический CFG полный. В присутствии exception edges он почти наверняка неполный, если инструмент не учитывает SEH/VEH/UEF. Декомпилятор может красиво нарисовать функцию, но пропустить маршрут через handler. Поэтому в сложных бинарях нужно читать не только код, но и exception metadata, registrations и runtime events.

Пятая ошибка - забывать, что динамический валидатор сам является участником debug-маршрута. dynamic наблюдает через Debug API, значит target видит debugger-like environment. Это не причина отказаться от динамики, но причина честно фиксировать метод наблюдения и отличать события target-а от событий instrumentation.

8.6.17. Практическая модель для будущих finding-ов

Теперь можно сформулировать спокойную модель, которую мы будем использовать дальше. Если static видит исключение как возможный элемент управления, он не должен сразу ставить максимальную тревогу. Он должен описать evidence chain: место создания исключения, потенциальный handler, тип handler-а, признаки работы с ExceptionRecord/ContextRecord, возможное продолжение, влияние на CFG, необходимость динамической проверки. Если dynamic подтверждает handler hit и post-handler RIP trace, confidence растёт. Если динамика не подтверждает маршрут, finding остаётся гипотезой или понижается.

Сильный finding в этой теме должен быть не коротким “anti-debug detected”, а объясняющим. Например: “В функции обнаружен deliberate breakpoint site. Ранее зарегистрирован VEH с высоким приоритетом. Handler сравнивает ExceptionCode с breakpoint code, изменяет ContextRecord->Rip и возвращает CONTINUE_EXECUTION. dynamic подтвердил first-chance exception, hit handler-а и продолжение в адрес 0x... вместо обычного fallthrough. Это указывает на exception-based control flow; статический CFG без exception edges неполон”.

Такая формулировка длиннее, но она инженерная. Она показывает пользователю не эмоциональный ярлык, а причинную цепочку. Именно такой стиль нужен твоему проекту: static строит карту возможностей, dynamic показывает живой маршрут, отчёт связывает оба слоя в понятное доказательство.

8.6.18. Мини-сцена: один breakpoint, два разных мира

Представим функцию, внутри которой есть int 3. Если открыть её в дизассемблере без контекста, можно решить, что это debug artifact или точка аварии. Но в начале программы зарегистрирован AddVectoredExceptionHandler с First=1. Handler проверяет, что ExceptionCode соответствует breakpoint, смотрит на ExceptionAddress, находит этот адрес в своей таблице и меняет

ContextRecord->Rip на адрес следующего логического блока. После этого возвращает EXCEPTION_CONTINUE_EXECUTION.

Без debugger-а маршрут может быть таким: поток дошёл до int 3, Windows создала exception, VEH получил управление, handler изменил RIP, поток продолжил выполнение в скрытом блоке. Для обычного пользователя программа работает. Для простого статического анализа блок после int 3 кажется unreachable или непонятным. Для отладчика first-chance event останавливает выполнение, и аналитик может подумать, что программа “падает на breakpoint”.

Правильный анализ не спорит с фактами, а собирает их. static говорит: есть breakpoint site, есть ранний VEH, есть изменение context, есть возможный hidden edge. dynamic запускает и показывает: событие реально произошло, handler реально hit-нулся, RIP после handler-а стал таким-то. После этого мы уже не гадаем. Мы понимаем маршрут.

8.6.19. Как читать такой handler глазами реверсера

Когда реверсер добрался до handler-а, ему нельзя сразу читать его как обычную функцию с обычным назначением. Handler получает не просто пользовательские аргументы, а описание исключения и состояние потока. Поэтому первый вопрос к handler-у всегда такой: какие части exception state он использует? Если он читает только ExceptionCode и пишет лог, это один смысл. Если он читает ExceptionAddress, ExceptionInformation, ContextRecord->Rip, ContextRecord->Rsp, flags, а потом возвращает CONTINUE_EXECUTION, это совсем другой смысл.

ExceptionAddress - адрес, где возникло исключение. Для breakpoint это может быть адрес trap-инструкции или близкий к нему адрес с архитектурными нюансами. Для access violation это адрес инструкции, которая пыталась обратиться к памяти. Для guard page это тоже место, где произошёл доступ. Handler может использовать ExceptionAddress как ключ: если исключение пришло из такого-то site, перейти сюда; если из другого site, перейти туда. То есть адрес исключения становится почти switch-case по скрытым точкам программы.

ExceptionInformation - дополнительные параметры исключения. Для access violation там обычно есть тип доступа и адрес памяти. Для пользовательских исключений через RaiseException там могут быть параметры, которые сама программа передала как payload, полезную нагрузку. Payload здесь не надо понимать как что-то вредоносное. Это просто данные, приложенные к событию. Если handler использует эти параметры как входные данные для выбора следующего блока, перед нами уже не просто обработка ошибки, а внутренний протокол управления.

ContextRecord - самый важный объект для понимания продолжения. Если handler только читает ContextRecord, он может диагностировать, где поток был в момент падения. Если он пишет в ContextRecord, он меняет будущее выполнение. Изменение Rip означает смену следующей инструкции. Изменение Rsp означает смену представления о стеке. Изменение EFlags/RFlags может влиять на условные переходы, single-step, direction flag и другие аспекты исполнения. Поэтому запись в ContextRecord для static -а должна быть более сильным сигналом, чем обычное чтение ExceptionCode.

Но и тут нельзя быть грубым. Некоторые легитимные обработчики тоже корректируют context. Например runtime может пропустить проблемную инструкцию, эмулятор может заменить недоступную инструкцию программной реализацией, guard-page механизм может восстановить защиту и продолжить. Поэтому finding должен объяснять не "handler пишет RIP, ужас", а "handler пишет RIP, поэтому обычный CFG неполон; требуется динамическое подтверждение маршрута".

```
Что искать в handler-е концептуально:

    1. Читает ли он ExceptionRecord->ExceptionCode?
    2. Сравнивает ли ExceptionAddress с известными sites?
    3. Читает ли ExceptionInformation?
    4. Пишет ли ContextRecord->Rip или ContextRecord->Rsp?
    5. Возвращает ли EXCEPTION_CONTINUE_EXECUTION?
    6. Есть ли таблица соответствия: exception site -> next RIP?
    7. Подтверждает ли dynamic     этот маршрут в runtime?
```

8.6.20. Почему декомпилятор может врать без злого умысла

Отдельно нужно проговорить роль декомпилятора. Декомпилятор пытается превратить машинный код в похожий на C псевдокод. Для этого он строит control-flow graph, выделяет basic blocks, восстанавливает условия, циклы, вызовы, возвраты. Но если часть управления идёт через exception dispatcher, декомпилятор может не иметь полной картины. Он может показать int 3 как конец пути. Может решить, что блок после trap unreachable. Может не связать handler с местом исключения. Может показать странный goto, которого нет в исходной логике.

Это не значит, что декомпилятор плохой. Он работает с теми evidence, которые умеет учитывать. Но exception route находится не только в обычных инструкциях. Он распределён между кодом, metadata, Windows dispatcher-ом и runtime behavior. Поэтому в сложных местах реверсер должен временно отойти от красивого псевдокода и вернуться к фундаменту: где RIP, где exception site, какой ExceptionCode, какой handler, что с ContextRecord, куда продолжилось выполнение.

Ghidra, IDA, Binary Ninja, x64dbg, WinDbg - все они могут помогать, но ни один инструмент не отменяет модели в голове. Если инструмент показывает

непонятный dead end, а рядом есть VEH и CONTINUE_EXECUTION, нужно не спорить с инструментом, а расширить модель: возможно, это не dead end, а exception edge. Именно поэтому книга постоянно возвращается к базовым словам: RIP, RSP, return address, CONTEXT, handler. Без них реверсер превращается в зрителя интерфейса.

Для твоего static -а это означает: отчёт должен помогать человеку увидеть то, что не всегда видно в декомпиляторе. Например: "этот trap site имеет потенциальный dynamic edge через VEH handler". Для dynamic -а: "этот edge был подтверждён, post-handler RIP = ...". Тогда Ghidra-интеграция может не просто импортировать функции, а ставить комментарии и bookmarks в тех местах, где обычный граф неполон без exception route.

8.6.21. Три уровня уверенности: site, route, confirmed route

Чтобы не смешивать наблюдения, удобно разделить три уровня уверенности. Первый уровень - exception site. Это место, где исключение может возникнуть или явно создаётся: int 3, UD2, RaiseException, access to guard page, потенциальная access violation. На этом уровне мы ещё почти ничего не знаем о смысле. Мы знаем только: здесь есть дверь в exception dispatcher.

Второй уровень - route. Route - маршрут. Мы нашли потенциального получателя: VEH, SEH protected range, top-level filter, handler в .xdata, callback, код, который проверяет ExceptionCode. Теперь у нас есть гипотеза: если site сработает, управление может пройти через этот обработчик. Но это всё ещё статическая гипотеза. Может быть, site недостижим. Может быть, handler никогда не регистрируется в данной конфигурации. Может быть, debugger policy меняет route. Может быть, code path зависит от входных данных.

Третий уровень - confirmed route. Подтверждённый маршрут. dynamic запустил target, увидел exception event, увидел handler hit, записал post-handler RIP trace и показал, что выполнение реально пошло по этой дороге. Вот здесь confidence сильно выше. Мы уже не просто предполагаем exception-based control flow, а видим его проявление. Даже если не все условия покрыты, один подтверждённый маршрут уже ценнее десятка статических подозрений.

```
Уровни вывода:

    Level 1: site
        "Здесь есть инструкция/вызов, способные создать исключение."

    Level 2: route hypothesis
        "Есть handler, который потенциально может принять это исключение."

    Level 3: confirmed route
        "dynamic     подтвердил: exception -> handler hit -> post-handler RIP."
```

Эта шкала нужна не только для красивого отчёта. Она дисциплинирует мышление. Реверсер перестаёт прыгать от одного признака к финальному обвинению. Инструмент перестаёт шуметь. Книга перестаёт быть набором "опасных паттернов" и становится способом думать.

8.6.22. Ложные тревоги: почему спокойствие важнее агрессии

В теме anti-debug особенно легко начать видеть врага везде. Программа вызвала IsDebuggerPresent - anti-debug. Есть int 3 - anti-debug. Есть RaiseException - anti-debug. Есть guard page - anti-debug. Такой подход делает отчёт бесполезным: если всё подозрительно, то ничего не подозрительно. Хороший анализатор должен быть спокойным.

Например IsDebuggerPresent может использоваться для выбора режима логирования. Debug build может специально останавливаться на assertions. .NET или другой runtime может генерировать исключения как часть своей модели. JIT может использовать guard pages. Профилировщик может ставить breakpoints. Даже security products и defensive instrumentation могут активно работать с исключениями. Поэтому finding должен учитывать роль бинаря, imports, строки, секции, known runtime patterns, наличие PDB/debug artifacts, compiler style, surrounding code.

С другой стороны, чрезмерное спокойствие тоже вредно. Если программа рано регистрирует VEH, затем имеет десятки int 3 sites, handler меняет RIP, обычный CFG разваливается, .pdata/.xdata выглядит необычно, а dynamic подтверждает скрытые переходы, это уже нельзя списывать на debug artifact. Баланс достигается не интуицией, а evidence chain. Мы не пытаемся угадать намерение автора. Мы описываем наблюдаемую механику и её последствия для анализа.

Вот почему в твоём проекте особенно важны confidence и requires_manual_review. Confidence говорит, насколько сильна интерпретация. Manual review говорит: человеку нужно посмотреть, потому что автоматическая классификация может ошибиться. Для сложных exception tricks это честнее, чем выдавать категоричный verdict.

8.6.23. Мини-сцена: guard page как скрытый выключатель ветки

Представим функцию, которая выглядит почти обычной. В середине она читает данные по адресу, который заранее помечен PAGE_GUARD. На первом чтении возникает STATUS_GUARD_PAGE_VIOLATION. Без понимания guard pages это выглядит как странная ошибка памяти. Но VEH handler проверяет faulting address и понимает: это именно ожидаемая страница. Он увеличивает внутренний счётчик состояния, возможно снова ставит PAGE_GUARD, затем

возвращает CONTINUE_EXECUTION.

Что увидит обычный статический анализ? Он увидит чтение памяти и, возможно, вызов VirtualProtect где-то раньше. Если не связать эти события, картина неполна. Что увидит dynamic ? Он увидит exception code, faulting address, handler hit, а затем продолжение. Если trace показывает, что после нескольких таких сигналов функция выбирает другую ветку, значит guard page стала частью state machine - машины состояний.

State machine - машина состояний: логика, где поведение зависит от текущего состояния и входного события. В обычной программе state machine может быть реализована через switch. В exception-based варианте событием становится исключение, а состояние хранится внутри handler-а или runtime-а. Это уже гораздо глубже, чем “обнаружен PAGE_GUARD”.

8.6.24. Мини-сцена: RaiseException как внутреннее сообщение

Теперь другой пример. Функция вызывает RaiseException с пользовательским кодом. В параметрах передаётся маленькое число. Handler получает исключение, читает ExceptionCode и arguments, затем выбирает один из нескольких continuation blocks. В исходном коде это могло быть макросом, внутренним runtime-протоколом или обфускационным слоем. В машинном коде это выглядит как вызов API и странное продолжение.

Если static просто напишет “RaiseException found”, пользы мало. Нужно показать: где вызов, какой code, есть ли nearby/registered handler, читает ли handler arguments, возвращает ли continue execution, есть ли mapping argument -> next block. Если dynamic подтвердит, что при argument=2 RIP ушёл в block B, а при argument=3 в block C, мы уже видим не ошибку, а dispatcher protocol.

Эта сцена полезна тем, что показывает: exception-based control flow не обязан быть низкоуровневым трюком с битами процессора. Иногда это почти обычный протокол сообщений, просто доставляется он через Windows exception machinery. Поэтому теория исключений нужна не ради редких краевых случаев, а ради понимания того, как управление может быть вынесено из обычных call/jmp/ret.

8.6.25. Как это потом ляжет в архитектуру проекта

В финальной проектной главе мы отдельно распишем архитектуру EH/Unwind Analyzer. Но уже здесь можно зафиксировать будущую связь. Модуль static должен иметь представление exception sites и potential handlers. Модуль heuristics должен уметь не только ставить score, но и объяснять, почему site и handler связаны. Модуль dynamic должен собирать runtime evidence: EXCEPTION_DEBUG_EVENT, first/second chance, handler hit,

post-handler RIP, возможно краткий trace после продолжения. Report layer должен соединять эти данные в одну историю.

Тогда пользователь отчёта увидит не набор строк, а понятную картину: “статически найдено вот это; динамически подтверждено вот это; обычный CFG неполон вот здесь; для ручного анализа откройте такой-то handler и такой-то continuation block”. Это и есть цель книги: не просто знать названия Windows API, а понимать, как из них строится инженерный анализ.

8.6.26. Переход к packers и manual mapping

Этот раздел завершает одну важную линию главы 8. Мы начали с аномалий .pdata, перешли к странной .xdata, затем к mismatch между кодом и metadata, потом к leaf-функциям и dynamic function tables. Теперь мы увидели, что исключения могут быть не только служебной инфраструктурой восстановления, но и способом направлять выполнение. Следующий шаг естественный: посмотреть на packers, manual mapping и ситуации, где файл на диске намеренно перестаёт быть честной картой того, что будет жить в памяти.

Packer - упаковщик: программа или слой, который хранит настоящий код в сжатом, зашифрованном или преобразованном виде, а при запуске распаковывает его в память. Manual mapping - ручная загрузка модуля без обычного loader-а Windows или с частичной имитацией его работы. В таких сценариях static видит один PE на диске, а dynamic может увидеть совсем другую runtime-картину: новые секции в памяти, runtime code, dynamic function tables, handlers, exception routes. Поэтому следующий раздел будет не про “пакеры страшные”, а про расхождение между on-disk картой и in-memory реальностью.

> Финальная модель раздела: исключение - это не только аварийный сигнал. В руках runtime-а, debugger-а, защиты или обфускатора оно становится управляемым событием. Реверсер должен видеть оба слоя: статическую возможность и динамически подтверждённый маршрут.

8.7. Packers, manual mapping и сломанная PE-картина: почему файл на диске и программа в памяти могут быть разными мирами

В предыдущих разделах мы уже привыкли думать о Windows x64 не как о наборе случайных таблиц, а как о системе договоров. PE-файл обещает загрузчику определенную структуру. Секции обещают, где лежит код, где данные, где ресурсы. Exception Directory обещает, где искать карту функций. `.pdata` обещает диапазоны функций. `.xdata` обещает инструкцию, как разобрать кадр стека, восстановить RSP, найти `return address`, вернуть nonvolatile-регистры и, если нужно, найти обработчик исключения. Эта модель удобна, потому что она дает реверсеру почти школьную цепочку рассуждения: есть RIP, значит можно найти модуль; есть модуль, значит можно получить `ImageBase`; есть `ImageBase`, значит можно получить RVA; есть RVA, значит можно попасть в `.pdata`; есть `.pdata`, значит можно дойти до `.xdata`; есть `.xdata`, значит можно понять unwind и SEH.

Но в реальном reverse engineering эта аккуратная цепочка иногда ломается не потому, что мы плохо выучили формат PE, а потому, что сама программа специально или технически перестает быть равной своему файлу на диске. На диске лежит один объект. В памяти после загрузки появляется другой объект. После распаковки, ручного маппинга, генерации кода или изменения защит страниц появляется третий объект. И когда мы говорим "сломанная PE-картина", мы не имеем в виду, что PE обязательно битый. Мы имеем в виду, что статическая картина, которую видит static , больше не совпадает с живой картиной, которую наблюдает dynamic .

Именно здесь легко скатиться в конспект: перечислить packer, manual mapping, stub, payload, `VirtualAlloc`, `VirtualProtect`, private memory, dynamic function table - и сделать вид, что читатель понял. Но тогда теряется главное. Все эти слова описывают одну большую проблему: Windows, реверсер, отладчик и анализатор должны понимать, где находится исполняемый код и как из этого кода безопасно выйти назад. Если код живет там, где обычный PE-загрузчик его не описал, то вопрос уже не в том, "подозрительно это или нет". Вопрос в том, существует ли для этого кода надежная карта: кто его загрузил, где его границы, как перевести адрес в смысл, как пройти по стеку, как обработать исключение и как доказать, куда реально ушло управление.

Поэтому этот раздел нужно читать не как каталог трюков. Это глава о расхождении между тремя реальностями: файлом, образом в памяти и живым исполнением.

Три реальности: file bytes, mapped image и running process

Начнем с самого простого, потому что без этого packer и manual mapping превращаются в магические слова. Когда у нас есть `program.exe` на диске, у нас есть последовательность байтов. Это file bytes - байты файла. Они лежат подряд так, как их записал компоновщик, упаковщик или другой генератор. Внутри этих байтов есть DOS-заголовок, NT-заголовки, таблица секций, data directories, raw data секций и прочие части PE. Но это еще не программа в работающем смысле. Это только материал, который загрузчик Windows должен превратить в образ процесса.

Когда Windows загружает PE нормальным способом, она не просто копирует файл "как есть" в память. Она создает image mapping - отображение образа. Секции получают виртуальные адреса, которые обычно отличаются от file offset. `VirtualAddress` секции не обязан совпадать с `PointerToRawData`. RVA - relative virtual address, относительный виртуальный адрес - выражает положение объекта не в файле, а в образе после загрузки. Поэтому мы уже давно повторяем: RVA не равен file offset. Это не занудная формальность. Это граница между дисковым миром и адресным пространством процесса.

После нормальной загрузки появляется mapped image - загруженный образ. У него есть `ImageBase`, секции в памяти, примененные relocation-и, разрешенные imports, возможно выполненные TLS callbacks, вызванный entry point. И только затем мы получаем running process - живой процесс, в котором инструкции реально исполняются, RIP указывает на текущую инструкцию, RSP указывает на текущую вершину стека, а исключения идут через механизмы Windows.

На обычной простой программе эти три реальности почти совпадают по смыслу. Файл говорит: "код здесь". Образ в памяти подтверждает: "код действительно здесь". Исполнение показывает: "RIP ходит по этим адресам". Но при packer-ах и manual mapping эта связь ослабляется. Файл может содержать маленький loader stub, а настоящий payload появится позже. Образ может быть изменен после старта. Реальный RIP может уйти в память, которая не принадлежит ни одной нормальной PE-секции. Тогда статический анализ файла остается полезным, но он больше не является полной правдой.

Для static -а это означает: нельзя обещать больше, чем видно из файла. Для dynamic -а это означает: нужно смотреть живое исполнение и проверять, куда реально попал RIP, какие memory regions исполняются, какие handler-ы реально вызываются, какие dynamic function tables регистрируются и какой маршрут проходит исключение.

Почему loader важнее, чем кажется

Чтобы понять manual mapping, надо сначала уважить обычный Windows loader. Частая ошибка новичка - думать, что loader просто "загружает exe в память". На самом деле loader - это слой, который превращает PE-файл в участника операционной системы. Он не просто раскладывает байты. Он связывает файл с системной инфраструктурой.

Во-первых, loader создает правильную карту секций. Код получает execute-права, данные - write-права, read-only данные остаются read-only. Это важно не только для безопасности, но и для анализа. Если секция заявлена как кодовая, если она mapped как executable, если `RIP` попадает внутрь нее, у нас появляется нормальный путь рассуждения: адрес принадлежит модулю, модуль имеет заголовки, заголовки имеют data directories, data directories ведут к exception directory.

Во-вторых, loader обрабатывает relocations - перемещения адресов. Если образ не загрузился по preferred `ImageBase`, абсолютные адреса внутри кода и данных нужно поправить. Иначе инструкция может обращаться в старое место, которого в этом процессе нет. Для unwind-темы это важно косвенно: `RUNTIME_FUNCTION` в PE хранит image-relative адреса, а не абсолютные. Но если сам код или связанные таблицы были вручную перенесены, неправильная база может сломать всю адресную арифметику.

В-третьих, loader разрешает imports. Импорт - это не просто строка с названием функции. Это механизм, через который код получает адреса функций из DLL. Если программа вызывает `VirtualAlloc`, `VirtualProtect`, `AddVectoredExceptionHandler`, `RtlAddFunctionTable`, то в обычном случае эти адреса попадают в Import Address Table. static может увидеть импорт и сказать: "в файле есть статический след такого поведения". Но если код сам ищет функции через `GetProcAddress`, через hashed imports или вообще получает адреса другим способом, статический импорт может отсутствовать, а runtime-поведение все равно будет.

В-четвертых, loader учитывает TLS - thread local storage. TLS callbacks могут выполниться до обычного entry point. Для реверсера это критично: программа может начать распаковку, регистрацию VEH, установку exception filter-а или настройку памяти еще до того, как новичок поставит breakpoint на `main`. Если мы говорим "файл на диске говорит одно, а память живет другой жизнью", TLS - один из ранних механизмов, который может начать эту другую жизнь.

В-пятых, loader делает модуль видимым для системных механизмов. У процесса появляется список загруженных модулей. Отладчик может сообщить load module events. API и внутренние структуры могут сопоставлять адрес с модулем. Crash dump может показать, что `RIP` находится в `some.dll+0x1234`, а не просто в безымянной области памяти. Это кажется косметикой, пока не начинается unwind. Для unwind-а важна не красота имени модуля, а возможность найти function table для адреса. У нормального PE32+ образа static function table приходит из Exception Directory. Для кода вне нормального

образа ее надо предоставить отдельно.

Вот почему manual mapping нельзя понимать как "сам скопировал DLL в память". Если ты только скопировал байты и прыгнул на entry point, ты сделал лишь часть работы loader-а. CPU может начать исполнять инструкции, но Windows может не знать, как этот код связан с модулями, секциями, exception directory, unwind metadata и обработчиками.

Packer: не ярлык malware, а изменение жизненного цикла кода

Теперь можно аккуратно ввести слово `packer`. Packer - упаковщик. В бытовом reverse engineering под packer-ом часто понимают инструмент, который берет программу, сжимает или шифрует ее основной код, добавляет маленький loader stub и на старте распаковывает настоящий код в память. Но важно сразу убрать плохую привычку: packer не равен malware. Упаковка может использоваться для уменьшения размера, защиты коммерческого кода, лицензирования, анти-тамперов, усложнения анализа, а может использоваться и вредоносным кодом. Сам факт packer-а - это сигнал об измененном жизненном цикле кода, а не окончательный приговор.

Слово `stub` переводят как заглушка, но в packer-контексте это не пустышка. Stub - маленький загрузочный код, который присутствует в файле явно. Именно его видит static при первом статическом анализе. Он может быть корректным PE-кодом, иметь нормальную `.pdata`, нормальную `.xdata`, обычный entry point и вполне приличные секции. Но его задача - не быть всей программой. Его задача - подготовить настоящую программу: выделить память, распаковать payload, поправить imports, возможно изменить protection страниц и передать управление.

`Payload` - полезная нагрузка, то есть настоящий код, ради которого все сделано. В malware-анализе слово payload часто звучит страшно, но технически это просто содержимое, которое stub готовит и запускает. В коммерческом dynamic -е payload может быть обычной защищаемой программой. В учебном примере payload может быть простым `MessageBox`. Для нашей темы важно другое: payload может не существовать в явном виде в PE на диске. Он может быть сжат, зашифрован, разделен на куски, восстановлен в памяти или даже сгенерирован.

`Original Entry Point`, часто сокращают как OEP, - исходная точка входа программы до упаковки. Entry point packed-файла ведет в stub. А OEP - это место, куда stub передаст управление после распаковки. Реверсер часто ищет OEP, потому что именно там начинается нормальная логика исходной программы. Но в Windows x64 exception/unwind-анализе вопрос глубже: когда управление дошло до OEP, есть ли вокруг этого кода корректная unwind-карта? Принадлежит ли OEP нормальному mapped image? Есть ли для функций

payload-а `.pdata` и `.xdata`? Или RIP теперь находится в private executable memory, а Windows видит этот код как безымянный leaf-мир?

Представим простую картину. На диске есть packed.exe. static открывает файл и честно разбирает его PE. Он видит заголовки, секции, exception directory и `.pdata` stub-а. Все это может быть валидно. Если static на этом этапе скажет: "я полностью понял программу", он ошибется. Он понял только статический контейнер. Чтобы понять payload, нужен либо unpacked dump, либо dynamic validation, либо memory snapshot после распаковки.

dynamic в этой сцене делает другую работу. Он запускает target, смотрит, куда уходит RIP, видит переход из stub-а в новую область, фиксирует memory region, protection, принадлежность модулю, exception events, handler hits, dynamic function table registration. Если после распаковки RIP ушел в адрес, который не принадлежит исходному image module, это не просто "интересный адрес". Это доказательство, что runtime-картина шире статического PE.

Почему packer ломает не только дизассемблер, но и нашу unwind-модель

Новичок часто думает о packer-е как о проблеме дизассемблера: "Ghidra не видит нормальный код, потому что он зашифрован". Это верно, но для нашей книги недостаточно. Packer ломает не только удобство просмотра кода. Он ломает предположение, что статический PE содержит полную карту функций, исключений и unwind-а.

В обычной модели у нас есть цепочка:

```
RIP -> module -> ImageBase -> RVA -> Exception Directory -> .pdata -> RUNTIME_FUNCTION -> .xdata
```

У packed payload-а эта цепочка может оборваться в нескольких местах. RIP может попасть в память, которая не принадлежит модулю. Тогда нет нормального `ImageBase` в PE-смысле. Если нет модуля, нельзя получить RVA относительно модуля. Если нельзя получить RVA, нельзя искать запись в static `.pdata` исходного файла. Если static `.pdata` описывает только stub, она ничего не говорит о payload-е. Если payload имеет свою `.pdata`, но она распакована в память без регистрации, Windows может о ней не знать. Если payload вообще не имеет unwind metadata, то exception handling и stack walking будут зависеть от leaf-допущений, а для nonleaf-кода это опасно.

Здесь надо вспомнить раздел 8.4 про leaf-функции. Windows может unwound leaf-функцию через симуляцию `ret`, потому что leaf-функция не меняет RSP так, чтобы это требовало внешней инструкции восстановления. Но если payload-функция делает `sub rsp, 0x80`, сохраняет RBX, вызывает другие функции, использует SEH или имеет сложный пролог, она уже не должна жить без unwind metadata. Если для такого кода нет видимой function table, Windows может ошибиться при раскрутке стека. Debugger может показать кривой call

stack. Crash dump может стать менее полезным. SEH route может не дойти туда, куда должен.

И вот здесь появляется важное различие между "код исполняется" и "код интегрирован в Windows exception/unwind-модель". CPU может исполнять почти любые корректные инструкции из executable memory. Но Windows exception handling требует большего: для nonleaf-кода ей нужно понимать кадры. Если packer распаковал код и просто прыгнул в него, выполнение может идти до первого серьезного исключения. А когда исключение произойдет, внезапно выяснится, что операционная система не имеет карты, которая объясняет, как выйти из текущей функции к caller-у.

Для static -а это означает: статический отчет должен честно говорить "я вижу только disk image". Для dynamic -а: нужно проверять, появляется ли runtime code, регистрируются ли dynamic function tables, где находятся handler-ы и можно ли сопоставить живой RIP с какой-либо function table.

Manual mapping: когда кто-то решил сыграть в loader

Теперь перейдем к `manual mapping`. Manual mapping - ручное отображение модуля в память. Слово manual здесь принципиально: вместо того чтобы попросить Windows loader нормально загрузить DLL через стандартный путь, код сам выделяет память, копирует туда секции, применяет relocations, разрешает imports, вызывает TLS callbacks и entry point. В идеале manual mapper повторяет большую часть работы loader-а. В плохом или намеренно скрытном варианте он делает только минимум, чтобы инструкции начали исполняться.

Почему вообще кто-то так делает? Причины разные. Иногда это legitimate loader, plugin system, sandbox, custom runtime, исследовательский инструмент. Иногда это защитный механизм. Иногда это malware injection. Для нашей книги мотивация вторична. Главное - manual mapping меняет статус кода. Код может быть PE-подобным внутри памяти, но не быть нормальным loaded module с точки зрения Windows loader-а.

Обычный `LoadLibrary` делает DLL видимой как загруженный модуль. Manual mapper может выделить память через `VirtualAlloc`, скопировать туда image и передать управление, но не добавить этот image в обычный loader module list. Тогда отладчик и crash dump могут видеть не `module.dll+RVA`, а просто адрес в `MEM_PRIVATE` executable region. Для реверсера это сразу меняет вопрос. Мы уже не можем спокойно сказать: "адрес принадлежит модулю, значит найдем Exception Directory этого модуля". Нужно сначала доказать, что у этого memory region вообще есть PE-структура, что у него есть разумный image base, что заголовки не стерты, что sections соответствуют памяти, что exception directory доступна, что function table зарегистрирована или хотя бы может быть реконструирована.

Manual mapping особенно важен для unwind-а, потому что `.pdata` и `.xdata` могут физически присутствовать в скопированном образе, но Windows может не использовать их автоматически. Если модуль не загружен как нормальный image, одного наличия байтов `.pdata` в памяти мало. Система должна знать, что для диапазона адресов существует function table. Для dynamically generated code и подобных сценариев на x64 предусмотрены dynamic function tables через `RtlAddFunctionTable` или callback-модель. Если manual mapper не регистрирует unwind-информацию для своего диапазона, то он оставляет Windows без официальной карты.

Вот здесь надо быть очень точным. Не всякий manual mapping автоматически ломает исключения. Хороший manual mapper может зарегистрировать function table, может корректно настроить protections, может вызвать TLS, может обеспечить imports и relocations. Но плохой или скрытный mapper может сделать только "чтобы код побежал". И это создает разницу между работой CPU и работой Windows runtime-инфраструктуры. CPU доволен: есть executable bytes. Windows unwind-модель недовольна: для адреса нет надежной записи.

Разница между mapped image и private executable memory

Слова `mapped image` и `private memory` звучат сухо, но они дают реверсеру очень важную развилку. Mapped image - это память, которая связана с образом файла. Когда Windows загружает EXE или DLL как image, секции этого image отображаются в процесс. У таких областей обычно есть связь с модулем, путем к файлу, image base, section layout. Это мир, где цепочка `RIP -> module -> RVA` обычно работает.

Private memory - это память, выделенная самим процессом, например через `VirtualAlloc`. Она может быть read/write, read/execute, execute/read/write и так далее. Если процесс выделил private region, записал туда байты и сделал страницу executable, CPU может оттуда исполнять код. Но для реверсера это уже другой тип доказательства. Адрес может не иметь имени модуля. RVA относительно PE может не иметь смысла. `.pdata` исходного файла не обязана покрывать этот адрес.

Когда dynamic видит, что `RIP` ушел в private executable memory, он не должен автоматически писать "malware". JIT-компилятор тоже может исполнять private memory. Некоторые legitimate runtimes создают код на лету. Но dynamic должен зафиксировать сам факт смены мира: выполнение вышло из normal image-backed code в runtime-created executable region. Дальше нужно смотреть, есть ли registration через `RtlAddFunctionTable`, какие exceptions происходят, удается ли stack walking, есть ли handler hits, возвращается ли управление в исходный модуль, меняются ли protections страниц.

static с диска такого увидеть не может. Он может увидеть подготовительные признаки: imports `VirtualAlloc`, `VirtualProtect`, `WriteProcessMemory`, `RtlAddFunctionTable`, странные секции, packed entropy, маленький stub, overlay, compressed blob. Но это только hints - намеки. dynamic дает runtime evidence - живые доказательства.

Хорошая формулировка finding-а должна разделять эти уровни. Неправильно писать: "найден `VirtualAlloc`, значит payload исполняется из памяти". Правильнее: "статически найден импорт/вызов `VirtualAlloc` и `VirtualProtect`, что может соответствовать подготовке executable memory; динамически подтвержден переход RIP в MEM_PRIVATE executable region после unpacking stage". Это уже не гадание, а evidence chain.

Что именно происходит с адресами: BaseAddress, ImageBase, RVA и ошибка на уровне системы координат

В manual mapping и runtime code самая разрушительная ошибка часто не в байтах кода, а в системе координат. Мы уже много раз говорили про VA, RVA, `ImageBase`, file offset. Здесь это перестает быть учебным упражнением и становится вопросом выживания анализа.

`ImageBase` - это база образа, относительно которой считаются RVA внутри PE. Если PE нормально загружен, `VA = ImageBase + RVA`. Если модуль был перемещен, relocations должны поправить абсолютные адреса, но RVA-логика внутри image всё равно сохраняет смысл относительно фактической базы. Если же код скопировали вручную в произвольную область, нужно четко понимать: какая база используется для вычислений? Та, которая была в Optional Header? Та, куда реально выделили память? Та, которую передали в `RtlAddFunctionTable` как `BaseAddress`?

Для dynamic function table это особенно важно. `RUNTIME_FUNCTION` записи хранят begin/end/unwind как offsets относительно базы, переданной системе. Если `BaseAddress` указан неверно, то запись может выглядеть синтаксически правильной, но указывать не на тот runtime-код. Windows будет искать function entry для `ControlPc`, но диапазоны будут смещены. Или найдет запись, но `UnwindInfoAddress` приведет к неправильной `.xdata`. В итоге unwind сломается не из-за сложной обфускации, а из-за ошибки координат.

Реверсер должен научиться задавать очень простой вопрос: относительно чего считается этот адрес? Если это file offset - это позиция в файле. Если это RVA - это смещение относительно image base. Если это VA - это адрес в памяти процесса. Если это dynamic table offset - это смещение относительно `BaseAddress`, который runtime передал Windows. Если эти системы координат смешать, можно получить красивый, но ложный отчет.

В контексте твоего проекта это разделяет обязанности. static хорошо работает там, где есть file bytes и PE-координаты. Он может проверить

RVA-to-file mapping, exception directory, `.pdata`, `.xdata`. dynamic должен проверять runtime-координаты: actual memory base, memory region, module ownership, dynamic function table base, actual RIP, post-handler RIP. Если затем отчет объединяет эти данные, он должен явно показывать, из какой системы координат пришел каждый факт.

Как нормальный loader поддерживает exception/unwind, а manual mapper может забыть

Вернемся к главной теме книги: exception/unwind. В обычном PE32+ image unwind metadata лежит в `.pdata` и `.xdata`, а Exception Directory говорит, где находится function table. Для nonleaf-функций Windows x64 ожидает, что сможет найти запись и понять, как восстановить caller context. Это нужно не только для C++ exceptions или `__try/__except`. Это нужно для stack walking, debugging support, crash dump analysis и корректного unwind при исключениях.

Когда DLL загружена обычным loader-ом, ее static function table является частью image. Но если DLL вручную скопировали в память, Windows loader может не знать, что этот диапазон адресов является загруженным image с exception directory. Тогда даже если внутри скопированной памяти есть байты `.pdata`, система может не обращаться к ним как к function table. Manual mapper должен сам решить эту проблему. Он может зарегистрировать dynamic function table через `RtlAddFunctionTable`. Он может использовать callback-модель. Он может не делать ничего.

Если он не делает ничего, возникает опасная полуисправная ситуация. Код работает, пока всё идет по обычному control flow. `call` кладет `return address`, `ret` возвращает управление, RSP двигается по инструкциям. Но как только появляется исключение, отладочная раскрутка стека или необходимость выйти из середины функции без обычного `ret`, Windows нужна metadata. А ее нет. Это как здание без пожарного плана: пока все ходят по лестнице спокойно, кажется, что план не нужен. Но в момент аварии выясняется, что никто не знает, какой выход безопасный.

Это хороший образ для всей главы. `.pdata` и `.xdata` - не украшение. Это аварийный план выхода из функции. Packer или manual mapper может построить в памяти новый этаж, но забыть нарисовать его на плане. CPU будет по нему ходить, но Windows exception dispatcher не сможет надежно вывести поток наружу при аварии.

Почему статический unpacked dump тоже не всегда решает проблему

Может показаться: если проблема в том, что payload появляется в памяти, значит достаточно сделать dump после распаковки и анализировать его как

PE. Иногда это действительно помогает. Но dump - это не магическая отмена всех проблем. Dump фиксирует состояние памяти в некоторый момент. Он может содержать распакованный код, восстановленные секции, таблицы импортов, возможно `.pdata` и `.xdata`. Но он не всегда восстанавливает loader context.

Во-первых, dump может иметь неправильные headers. Packer мог стереть или исказить PE-заголовки. Dumping tool может реконструировать их приблизительно. Секции в dump-е могут не совпадать с исходной виртуальной картой. File offsets могут быть искусственными. Если static воспринимает такой dump как обычный PE, он должен быть осторожен: это уже не оригинальный файл, а реконструированный артефакт.

Во-вторых, runtime function table могла быть зарегистрирована динамически и не храниться в dump-е как обычная `.pdata`. Например, JIT или loader мог создать массив `RUNTIME_FUNCTION` в отдельной области памяти и вызвать `RtlAddFunctionTable`. Dump кода без этой таблицы покажет инструкции, но не покажет официальную unwind-карту. Или наоборот: dump содержит `.pdata`, но в реальном процессе Windows использовала dynamic table из другого места.

В-третьих, handler-и и callbacks могут быть завязаны на runtime-состояние: адреса allocated regions, decoded keys, TLS data, process heap, thread-local pointers. Статический dump может показать байты, но не объяснить, какой путь выполнения был живым.

Поэтому в проектной архитектуре static и dynamic должны дополнять друг друга. static может анализировать original file и unpacked dump как два разных объекта. dynamic может сказать, какой из них соответствует реальному execution route. Финальный отчет должен не смешивать их: original disk image, memory dump, runtime events - это разные источники evidence.

Как выглядит unpacking path как история, а не как список API

Попробуем рассказать типичный unpacking path медленно, как историю выполнения.

Процесс стартует в entry point packed-файла. `RIP` находится внутри stub-а. Пока всё нормально: адрес принадлежит модулю, `ImageBase` известен, `.pdata` stub-а может быть валидна. Stub выделяет память, например через memory allocation API. На этом этапе появляется новая memory region. Сама по себе она может быть read/write, потому что туда нужно записать bytes payload-а.

Затем stub распаковывает или расшифровывает payload. Это может быть цикл, копирование блоков, decompression, XOR, AES, custom transform. Для static -а это может выглядеть как непонятный dense loop. Для dynamic -а это

просто этап, на котором memory region наполняется новыми байтами.

Потом stub меняет protections. Если память была writable, но теперь из нее надо исполнять код, protection может стать executable. Это может быть `PAGE_EXECUTE_READ`, `PAGE_EXECUTE_READWRITE` или другой вариант. Здесь важно не путать факт с выводом. Изменение страницы на executable - это факт подготовки к исполнению, но еще не доказательство, что `RIP` туда ушел. Доказательством будет actual branch/call/jmp и runtime trace.

Затем stub может разрешить imports payload-а. Иногда он строит import table, иногда вызывает `LoadLibrary/GetProcAddress`, иногда использует hashed names. На уровне книги важно понять: payload должен как-то получить адреса внешних функций, иначе он не сможет нормально работать с Windows API. Если эти адреса не видны в original import table, static не должен делать вывод "program never uses this API". Он должен сказать: "static imports do not show this API; dynamic resolution is possible; dynamic should observe runtime resolution if needed".

Затем stub передает управление. Это может быть direct jump, indirect call, `push address; ret`, exception-driven transfer, callback invocation, TLS-like route. С этого момента `RIP` может покинуть static image region. Вот настоящий перелом. До этого у нас был static suspicion. После этого у dynamic -а есть runtime event: execution entered unpacked code.

После входа в payload начинается новая задача: есть ли для него unwind map? Если payload был восстановлен как normal image и зарегистрирован корректно, Windows может иметь function table. Если нет, any exception внутри payload-а может привести к странному stack walk. Поэтому dynamic должен не просто сказать "jumped to payload", а проверить, как этот payload участвует в exception/unwind machinery.

Как manual mapping path отличается от unpacking path

Manual mapping может выглядеть похоже, но логически это другой сценарий. При unpacking payload часто является скрытой частью того же файла. При manual mapping payload может быть отдельным PE-образом, который кто-то прочитал из файла, ресурса, сети, encrypted blob или памяти, а затем вручную загрузил как DLL/EXE-подобный объект.

Manual mapper должен скопировать headers и sections в память. Но просто копирование sections не равно загрузке. Нужно применить relocations, если фактическая база не совпала с preferred image base. Нужно разрешить imports. Нужно обработать TLS. Нужно выставить правильные protections секций: `.text` executable/read, `.rdata` read-only, `.data` read/write и так далее. Нужно вызвать entry point с правильным reason, если это DLL. И, для нашей темы особенно важно, нужно обеспечить exception/unwind visibility.

Если mapper сделал это хорошо, runtime-код может вести себя почти как нормально загруженная DLL. Если плохо - он будет похож на живой остров в адресном пространстве: инструкции есть, но системные карты вокруг них неполные.

Здесь нельзя писать "manual mapping всегда плохо". Это слишком грубо. Нужно писать точнее: manual mapping разрывает стандартный loader contract, поэтому анализатор должен проверить, какие части этого контракта были восстановлены вручную. Для обычной логики программы достаточно одного набора частей. Для exception/unwind reliability нужен другой набор: function tables, correct bases, handler addresses, memory protections, valid code ranges.

static по original file может заметить manual mapping hints: код, который парсит PE headers, строки MZ/PE, обработку relocation directory, import directory, вызовы allocation/protection API, indirect calls. Но static не видит, какой image реально был mapped. dynamic может увидеть allocated region, writes, protection changes, jump to region, dynamic table registration, exceptions inside region. Вместе они дают картину.

Где именно ломается `RIP -> module -> RVA -> .pdata -> .xdata`

Полезно пройти нашу любимую цепочку и прямо показать, где она ломается.

Первое звено - `RIP -> module`. Если RIP находится внутри normal mapped image, отладчик и анализатор могут сопоставить адрес с модулем. Если RIP в private executable memory, модуль может отсутствовать. Тогда у нас нет надежного имени, нет обычного image base и нет section table. Это не значит, что кода нет. Это значит, что PE-модель адреса не доказана.

Второе звено - `module -> RVA`. RVA считается относительно image base. Если module ownership неясен, RVA может быть искусственным. В manual mapped image можно выбрать base, но нужно доказать, что это именно база образа, а не просто начало allocated region. Если headers стерты или sections переложены, вычисление RVA становится гипотезой.

Третье звено - `RVA -> .pdata`. У normal PE exception directory дает диапазон function table. Если в памяти есть распакованный image, но exception directory не восстановлена или не зарегистрирована, static `.pdata` может быть невидима для Windows. Если function table создана динамически, она может вообще не лежать в `.pdata` как секции.

Четвертое звено - `.pdata -> .xdata`. Даже если function entry найден, `UnwindInfoAddress` должен вести к valid `UNWIND_INFO`. В runtime scenarios эта `.xdata` может лежать рядом с generated code, в отдельной memory region, в read-only block, в writable block или быть callback-generated. Если адресная база

неправильная, `UnwindInfoAddress` уедет.

Пятое звено - `.xdata -> handler`. Handler RVA может вести к language-specific handler, например `__C_specific_handler`, или к runtime-generated handler. В manual mapped code handler может лежать в той же private region. Если dynamic ставит breakpoint на static handler из original file, он может не поймать runtime handler payload-a. Нужно ставить breakpoints на реальные runtime addresses, а не только на disk-derived RVA.

Так мы видим, что packer/manual mapping - это не одна проблема, а серия возможных разрывов. Хороший анализатор не должен говорить "все сломано". Он должен показывать, на каком звене цепочка перестала быть доказанной.

Почему `VirtualAlloc` и `VirtualProtect` сами по себе не доказывают packer

Теперь разберем одну частую ложную тревогу. Многие программы вызывают `VirtualAlloc` и `VirtualProtect` по нормальным причинам. Runtime, JIT, memory manager, sandbox, game engine, browser, .NET/JavaScript runtime, custom allocator, decompression library - все они могут работать с памятью. Если static видит эти imports, это еще не packer.

`VirtualAlloc` выделяет или резервирует memory region. `VirtualProtect` меняет protection уже committed pages. Эти API становятся интересны для нашей темы, когда появляются в связке: выделение памяти, запись туда байтов, изменение protection на executable, передача управления в эту область, регистрация или отсутствие dynamic unwind metadata, exception behavior внутри этой области. Один API - слабый сигнал. Последовательность действий - сильный сигнал. Подтвержденный RIP в этой области - еще сильнее. Exception route внутри этой области - уже предмет для dynamic -а.

Это важный стиль мышления для будущего проекта. static не должен быть паникером. Он должен быть следователем. Он собирает признаки, но не называет гипотезу фактом раньше времени. dynamic добавляет живые наблюдения. Если runtime trace показывает, что выделенная область стала executable и туда ушел RIP, гипотеза усиливается. Если дополнительно нет dynamic function table, а внутри области возникает exception, это уже конкретная unwind-проблема. Если же memory allocation была просто буфером для данных, finding должен быть низкой уверенности или вообще не возникать.

TLS callbacks, entry point и ранняя жизнь программы

Один из способов, которым файл на диске начинает жить другой жизнью раньше, чем реверсер ожидает, - TLS callbacks. TLS - thread local storage, хранилище данных, отдельных для потока. Но в PE есть не только TLS data, а и

callbacks - функции, которые loader может вызвать при определенных событиях, включая загрузку модуля. Для новичка это часто ловушка: он ставит breakpoint на entry point и думает, что поймал самое начало. Но часть кода могла выполниться раньше.

В packed/protected binaries TLS callback может быть удобным местом для ранней подготовки: антиотладочные проверки, регистрация VEH, изменение memory protections, распаковка части кода, установка hooks. Это не значит, что TLS callbacks вредоносны. Многие обычные библиотеки используют TLS для легитимной инициализации. Но для reverse engineering это означает: "entry point" не всегда первое исполняемое место, которое важно для анализа.

Для static -а TLS directory - отдельный источник static evidence. Он может сказать: в файле есть TLS callbacks, значит программа имеет pre-entry execution. Для dynamic -а это место, где нужно наблюдать runtime route с самого старта процесса, иначе можно пропустить важную подготовку. В контексте unwind-а TLS callback может зарегистрировать VEH или dynamic table до того, как основной код начнет работу. Если dynamic стартует наблюдение слишком поздно, он увидит последствия, но не увидит причину.

Overlay, resources и скрытые данные: где может лежать payload

Packer-ы и dynamic s могут хранить payload не только в явных секциях кода. Он может лежать в overlay - данных после конца последней секции, которые PE loader обычно не мапит как часть image. Он может лежать в resources. Он может быть в `.rdata`, `.data`, custom section, compressed blob, encrypted blob. Он может быть разделен на куски и собран только во время выполнения.

Для нашей темы это важно по двум причинам. Первая: static code coverage не показывает payload, если payload пока не код. Зашифрованные байты в resource не являются инструкциями до тех пор, пока stub их не расшифрует и не перенесет/не сделает executable. Вторая: metadata payload-а может быть скрыта вместе с ним. `.pdata` и `.xdata` исходной программы могут быть сжаты вместе с кодом. После распаковки они могут появиться в памяти, но original Exception Directory packed-файла будет описывать только stub.

static может искать такие признаки: unusually high entropy, large overlay, executable section with packed data, resources with suspicious size, mismatch between entry point and code volume. Но снова: это hypotheses. dynamic проверяет, читаются ли эти данные, куда они копируются, становятся ли executable, превращаются ли в code regions и появляется ли function table.

Роль dynamic function tables в unpacked/runtime code

Раздел 8.5 уже вводил dynamic function tables, но здесь нужно связать их именно с packer/manual mapping. Когда код появляется во время выполнения, Windows x64 все равно нуждается в способе понять его функции. Для generated code можно зарегистрировать function table. Тогда система получает диапазоны функций и `UNWIND_INFO`, даже если они не пришли из PE Exception Directory.

Для packer-а это развилка. Если после unpacking payload исполняется как normal image, возможно его static `.pdata` можно использовать, если image правильно зарегистрирован/загружен. Если payload исполняется как generated/private code, runtime должен предоставить dynamic table. Если не предоставляет, dynamic может зафиксировать: execution entered runtime code; no corresponding module/static function table; no observed dynamic function table registration; exception/unwind reliability questionable.

Но и здесь нужно быть осторожным. dynamic может не увидеть registration, если она произошла до начала наблюдения, если breakpoint не поставлен, если используется другой механизм, если таблица уже есть в snapshot-е. Поэтому finding должен различать "not observed" и "confirmed absent". В хорошей инженерной книге это важно: честность анализа не менее важна, чем глубина технических знаний.

Как dynamic должен доказывать "куда ушла функция"

Ты отдельно уточнил, что dynamic - это динамическая валидация, которая показывает в том числе, куда уходит функция и обработка исключения. В контексте packers/manual mapping это особенно хорошо видно. Статический static может сказать: "в entry point есть indirect jump". Но куда он пойдет после распаковки, статически часто неясно. dynamic должен превратить это из догадки в наблюдение.

Минимальная динамическая цепочка выглядит так:

```
start at stub RIP
observe allocation/protection/decode stage
observe transfer instruction or exception-driven transfer
record target RIP
classify target memory region
check module ownership and protections
check function table availability
if exception occurs: record handler path and post-handler RIP
```

Слова `post-handler RIP` здесь важны. Если исключение используется как control-flow trick, сам факт exception event еще не объясняет маршрут. Нужно увидеть, куда продолжилось выполнение после handler-a. Handler мог вернуть `EXCEPTION_CONTINUE_SEARCH`, и тогда обработка пошла дальше. Handler мог вернуть `EXCEPTION_EXECUTE_HANDLER`, и тогда управление перешло в handler block. Handler мог вернуть `EXCEPTION_CONTINUE_EXECUTION`, предварительно изменив `ContextRecord->Rip`, и тогда управление продолжилось вообще в

другом месте. dynamic должен фиксировать именно это: не просто "исключение было", а "после обработки поток продолжил исполнение по адресу X".

В packer-сценарии transfer в payload может быть сделан не обычным `jmp`, а через exception. Например, stub намеренно вызывает access violation, VEH меняет RIP на распакованный код, возвращает continue execution, и execution route исчезает из обычного CFG. Для static -а это route hypothesis. Для dynamic -а это подтверждаемый факт: exception site, handler hit, context modification, new RIP.

Как static должен честно говорить о границах видимости

У статического анализатора всегда есть соблазн быть слишком уверенным. Он прочитал PE, разобрал `.pdata`, нашел 200 functions, выдал красивый отчет. Но если файл packed, эти 200 functions могут быть только stub-ом, а настоящая программа появится позже. Поэтому static должен иметь язык ограничений.

Например, хороший отчет может сказать: "Static PE exception directory parsed successfully; unwind map covers visible image code. However, entry point belongs to a small unpacking stub, executable memory creation patterns are present, and dynamic validation is recommended to determine whether runtime code leaves the static image". Это лучше, чем "все нормально" и лучше, чем "упаковщик, всё плохо". Он объясняет, что именно проверено, что остается за границей статического анализа и какую роль должен сыграть dynamic .

Если static обнаруживает overlay или packed section, он должен объяснять, почему это влияет именно на exception/unwind-анализ. Не просто "overlay found". A: "overlay may contain code or metadata that is not represented in current Exception Directory; static unwind coverage applies only to mapped image described by headers". Такой finding связывает факт с последствием.

Если static видит `.pdata` только для stub-а, он не должен автоматически считать это ошибкой. Packed stub может быть корректным маленьким PE. Ошибка начинается, когда отчет делает вид, что stub-карта является картой всей runtime-программы. Поэтому правильная severity часто будет не "malformed", а "limited static visibility" или "dynamic validation required".

Как dynamic должен не переобещать

Динамический анализ тоже не всесилен. dynamic видит один или несколько запусков. Программа может вести себя по-разному в зависимости от аргументов, окружения, времени, сети, anti-debug checks, user input. Если dynamic не увидел unpacking route, это не доказывает, что его нет. Если увидел один route, это не доказывает, что других нет. Поэтому динамическая

валидация должна быть точной в формулировках.

Правильно: "Observed execution entered MEM_PRIVATE RX region at address X after protection change at Y". Неправильно: "Program always unpacks to X". Правильно: "No `RtlAddFunctionTable` call observed during this run". Неправильно: "No dynamic function tables exist" - если инструмент не проверял внутренние списки или мог пропустить ранний вызов. Правильно: "Handler at X was hit and post-handler RIP became Y". Неправильно: "All exceptions are handled by X".

Эта дисциплина особенно важна для твоего проекта. static и dynamic должны не соревноваться, кто громче скажет "подозрительно", а строить combined evidence. Статика дает карту возможного. Динамика дает трассу конкретного запуска. Отчет связывает их честно.

Пример 1: обычная packed-программа с честным stub-ом

Представим файл. Entry point ведет в маленький stub. `.pdata` есть, `.xdata` валидна, функций мало. В секциях есть большой encrypted blob. Imports содержат memory allocation/protection API. static может корректно разобрать Exception Directory и не найти структурных ошибок. Но он замечает: видимый код маленький, данных много, есть признаки runtime code creation. Хороший вывод: static unwind map is valid for visible stub, but may not represent post-unpacking payload.

dynamic запускает программу. Видит, что stub выделяет memory region, записывает туда decoded bytes, меняет protection на RX и делает jump. После jump RIP находится в MEM_PRIVATE executable memory. Dynamic function table registration не наблюдается. Внутри payload-а возникает first-chance exception, stack trace выглядит нестабильно или handler route не сопоставляется со static `.pdata`. Теперь combined finding становится сильнее: static PE was valid but incomplete; runtime execution left the static image; unwind coverage for runtime code is not confirmed.

Такой вывод не кричит "вирус". Он говорит именно то, что важно для EH/Unwind Analyzer: статическая exception/unwind-карта не покрывает весь реально исполняемый код.

Пример 2: manual mapped DLL с корректной dynamic table

Другой сценарий. Процесс вручную маппит DLL-like image в память. static по исходному файлу видит код, который парсит PE headers и применяет relocations. Это suspicious для loader-like behavior, но не доказательство ошибки. dynamic запускает и видит allocation, copy sections, import resolution, затем вызов `RtlAddFunctionTable` для диапазона mapped image. Потом RIP входит в этот диапазон. При exception внутри mapped code Windows находит dynamic function table, handler hit срабатывает, post-handler

RIP ожидаемый, stack walk стабильный.

Здесь finding должен быть тоньше. Да, runtime code исполняется вне normal loader module. Да, static PE на диске не является полной картой. Но dynamic unwind support присутствует. Значит verdict не "unwind broken", а что-то вроде "manual mapping behavior observed; dynamic unwind registration present; runtime exception route validated in observed run". Это совсем другой уровень инженерной точности.

Пример 3: manual mapped payload без unwind-регистрации

Третий сценарий похож, но mapper не вызывает `RtlAddFunctionTable`. Код исполняется, делает `sub rsp`, сохраняет RBX, вызывает API, внутри происходит exception. Windows пытается обработать исключение. Для `RIP` в private region нет static module function table и нет observed dynamic table. Если функция nonleaf, leaf fallback небезопасен. Stack trace может обрываться или показывать мусор. Handler-и могут не находиться.

Здесь finding уже серьезнее, но его всё равно нужно формулировать объясняюще: "Runtime execution entered non-image executable region. The observed code pattern is nonleaf-like, but no corresponding runtime function table registration was observed. If an exception or stack walk occurs inside this region, Windows may be unable to reconstruct caller context reliably". Это не просто "bad". Это объяснение последствия через теорию: нет карты - нет надежного восстановления кадра.

Пример 4: exception-driven unpacking route

Четвертый сценарий связывает 8.6 и 8.7. Stub регистрирует VEH. Потом намеренно вызывает exception: `int 3`, guard page violation или access violation. VEH получает `EXCEPTION_POINTERS`, смотрит `ExceptionRecord`, меняет `ContextRecord->Rip` на адрес распакованного payload-а и возвращает `EXCEPTION_CONTINUE_EXECUTION`. В обычном CFG это может выглядеть как crash site или странная инструкция. Но фактически exception стал скрытым jump.

static может увидеть VEH registration, deliberate exception site, writable/executable memory, handler code that writes to `ContextRecord->Rip`. Это сильная route hypothesis. dynamic подтверждает: exception occurred, VEH hit, context changed, post-handler `RIP` equals payload entry. В этом случае "куда уходит функция" определяется не `ret` и не обычным `jmp`, а handler-ом исключения. Именно поэтому dynamic в твоем проекте так важен: без него часть маршрутов остается гипотезой.

Как это переносится в Ghidra и почему декомпилятор не обязан помочь

Ghidra и другие дизассемблеры любят нормальную PE-картину. Когда есть module, sections, functions, xrefs, `.pdata`, `.xdata`, всё постепенно складывается. Но packed/manual-mapped code может появиться не в исходном файле. Декомпилятор не видит runtime bytes, если они зашифрованы. Он не знает post-handler RIP, если маршрут идет через exception. Он может не понимать dynamic function table, если она создается в памяти.

Поэтому импорт findings в Ghidra должен быть осторожным. Для original PE можно разметить stub, exception directory, suspicious memory API, possible unpacking loop. Для unpacked dump можно разметить payload functions и reconstructed `.pdata`, если они есть. Для dynamic trace можно добавить bookmarks: "runtime branch target", "handler hit", "post-handler RIP", "entered private executable region". Но нельзя заставить один статический вид файла изображать всю runtime-реальность без пометок.

Именно здесь проектная связка отчетов важнее красивой картинки. JSON/HTML/Ghidra CSV должны не просто показывать функции, а объяснять источник каждой метки: static original file, static memory dump, dynamic trace, exception event, handler breakpoint. Тогда реверсер не запутается, почему адреса из отчета не находятся в исходном `.text`.

Findings для packer/manual mapping: как писать не конспект, а инженерный вывод

В конце раздела полезно собрать не список признаков, а шаблон мышления. Finding должен отвечать на несколько вопросов.

Первый вопрос: какой факт наблюдается? Например: RIP вошел в private executable memory. Или: static Exception Directory покрывает только stub. Или: dynamic function table registration observed for runtime code range. Факт должен быть проверяемым.

Второй вопрос: на каком слое факт находится? Disk PE, mapped image, runtime memory, exception route, handler route, unwind metadata. Без слоя finding становится мутным. `VirtualProtect` в файле - это static import evidence. `VirtualProtect` call during run - dynamic API evidence. RIP in RX private memory - execution evidence. Это разные веса.

Третий вопрос: почему это важно для exception/unwind? Не просто "private executable memory". A: "addresses in this region cannot be resolved through the original PE Exception Directory; unwind support depends on dynamic function table or leaf assumptions". Так finding связывается с темой книги.

Четвертый вопрос: насколько мы уверены? Если есть только imports - low confidence. Если есть allocation/protection/jump trace - medium/high. Если есть exception inside region and no function table observed - higher. Если есть handler hit and broken unwind - confirmed in observed run.

Пятый вопрос: что делать реверсеру? Dump memory after unpacking. Capture dynamic function table registration. Compare original file and memory bytes. Import runtime trace into Ghidra. Set breakpoints on `RtlAddFunctionTable`, VEH registration, exception handlers. Проверить, является ли runtime code leaf-like или nonleaf-like.

Такой finding не просто сообщает проблему. Он учит продолжать анализ.

Отдельная медленная сцена: почему `ret` не спасает, когда исключение случилось посреди функции

Чтобы не потерять фундамент, остановимся на самом болезненном месте. Можно подумать: если код в unpacked payload-е обычный, если он делает `call`, потом когда-нибудь `ret`, значит Windows как-нибудь справится. Это смешивает два разных режима. Обычный возврат из функции и unwinding при исключении похожи по конечной цели, но совершенно разные по способу.

При нормальном возврате callee сама выполняет свой эпилог. Она знает, сколько байт вернуть в RSP, какие регистры восстановить, где лежит return address. Инструкции эпилога реально исполняются процессором. Если функция сделала `sub rsp, 0x80`, потом в эпилоге будет `add rsp, 0x80`. Если функция сохранила RBX на стеке, она сама его загрузит назад. После этого `ret` возьмет return address с вершины стека и передаст управление caller-у.

При исключении посреди функции ситуация другая. Мы можем находиться не в эпилоге. Мы можем быть в середине body, после `sub rsp`, после сохранения nonvolatile-регистров, после частичного построения кадра. И если теперь Windows нужно понять caller context, она не может просто выполнить эпилог. Эпилог может быть далеко, может быть недостижим по текущему пути, а выполнение чужих инструкций во время exception dispatch было бы небезопасным. Поэтому Windows читает unwind metadata и мысленно отменяет эффекты пролога. Это и есть причина существования `.xdata`.

Если unpacked/manual-mapped code не имеет доступной unwind metadata, Windows не знает, как мысленно отменить пролог. Она может применить leaf-допущение: будто на вершине стека лежит return address. Но для nonleaf-функции вершина RSP после `sub rsp, 0x80` уже не обязательно указывает на return address. Return address находится выше, по адресу вроде `RSP + 0x80` или дальше, с учетом сохраненных регистров и alignment. Если Windows ошибется и прочитает не тот qword как return address, caller context станет ложным.

Теперь видно, почему packer/manual mapping опасны именно для unwind-а, а не только для дизассемблера. Программа может часами нормально выполнять обычные `call/ret`, потому что сами функции корректно исполняют свои эпилоги. Но как только исключение, stack walk или debugger support требует внешней раскрутки кадра, система спрашивает: "Где твой план восстановления?" Если плана нет, успешное обычное исполнение ничего не доказывает.

dynamic в таком сценарии должен проверять не только факт выполнения кода, но и ситуации, где нужен unwind. Например, exception inside runtime code, breakpoint inside runtime code с попыткой stack walk, handler hit, `RtlVirtualUnwind`-подобное поведение, post-handler route. Если runtime code never throws in observed run, отсутствие failure не равно доказательству корректности. Это только значит, что опасный режим не был проверен.

Что значит "код принадлежит модулю" и почему это не философия

Фраза "адрес принадлежит модулю" звучит как удобная метка в отладчике: `kernel32.dll+0x1234`, `target.exe+0x4567`. Но для нашей темы это не просто косметика. Принадлежность модулю означает, что адрес можно поместить в определенную систему координат.

Если RIP находится внутри загруженного image-модуля, мы можем вычесть module base и получить RVA. Этот RVA можно сравнить с section ranges, Exception Directory, export/import-related structures, `.pdata` ranges. То есть модуль дает нам не имя, а контекст интерпретации адреса. Без этого контекста `0x0000021A5F301000` - просто число в адресном пространстве процесса. С контекстом это может быть `payload.dll+0x1000`, entry point, функция, handler, кусок `.text` или адрес внутри unpacked private region.

Manual mapped code может быть PE-подобным, но не зарегистрированным как module. Тогда у нас возникает странная промежуточная ситуация. По байтам память может начинаться с MZ, иметь PE, section headers и даже `.pdata`. Но системно это может быть просто private allocation. Отладчик может не присвоить ей модульное имя. Loader notifications могли не произойти. Exception directory этого pseudo-image может не быть встроена в обычный поиск function table. Для человека это "как будто DLL". Для Windows - не обязательно DLL.

Именно поэтому dynamic должен классифицировать runtime target не только как "адрес X", а как "адрес X находится внутри image-backed module" или "адрес X находится внутри private executable region", или "адрес X находится внутри manually mapped PE-like region". Эти категории имеют разные последствия для unwind.

Manual mapper как неполный загрузчик: разбор обязанностей по слоям

Разложим manual mapper не как одно слово, а как набор обязанностей. Тогда станет понятно, где именно возникают разные классы ошибок.

Первый слой - memory layout. Mapper должен выделить память под image, скопировать headers и sections по их virtual addresses, а не просто подряд как raw bytes. Если он копирует секции неправильно, адреса внутри image перестают соответствовать PE-структуре. Для обычного кода это может проявиться сразу. Для unwind metadata это может проявиться позднее: `UnwindInfoAddress` вроде бы ведет в `.xdata`, но фактически там другие байты.

Второй слой - relocations. Если image оказался не по preferred base, absolute addresses нужно исправить. Иначе код может прыгать или обращаться туда, куда он прыгал бы при старой базе. В x64 многие обращения RIP-relative, но absolute references и таблицы все равно встречаются. Для exception/unwind важно, что handler addresses, pointers in language-specific data или runtime-internal tables могут зависеть от корректной базы.

Третий слой - imports. Код payload-а должен вызывать Windows API и функции других DLL. Если Import Address Table не заполнена, indirect calls уйдут в ноль, мусор или старые placeholders. Иногда mapper разрешает imports вручную через `LoadLibrary`/`GetProcAddress`. static может увидеть такой resolver как признак loader behavior. dynamic может наблюдать runtime resolution.

Четвертый слой - TLS. Если TLS callbacks не вызваны, программа может работать неправильно, потому что часть инициализации не произошла. Если TLS callbacks вызваны, они могут изменить memory map, зарегистрировать VEH, подготовить exception routes до основного entry point. Поэтому TLS влияет не только на данные, но и на маршрут анализа.

Пятый слой - section protections. После копирования секций mapper должен выставить права памяти согласно смыслу секций. Если всё остается RWX, это удобно, но подозрительно и менее безопасно. Если `.xdata` оказывается writable/executable без причины, static /dynamic должны фиксировать metadata placement risk. Если code page не executable, CPU упадет. Если writable payload потом становится executable, это важный runtime event.

Шестой слой - exception/unwind visibility. Это слой, который часто забывают в поверхностных объяснениях. Даже если первые пять слоев достаточны, чтобы код исполнялся, для Windows x64 nonleaf-код должен быть описан function table. Если mapper не обеспечивает этот слой, он оставляет программу без надежного аварийного плана.

Такой разбор полезен тем, что manual mapping перестает быть страшным ярлыком. Мы видим конкретные обязанности и конкретные failure modes. Инженерный отчет должен указывать, какой слой подтвержден, какой отсутствует, какой неизвестен.

Почему `.pdata` payload-а может существовать физически, но не работать логически

Это одна из тонких идей, которую обязательно надо проговорить. Реверсер может сделать memory dump manual-mapped image, открыть его и увидеть там `.pdata`. Или даже в памяти найти массив `RUNTIME_FUNCTION`. Возникает соблазн сказать: "Ну вот же unwind metadata, значит всё нормально". Но для Windows важно не только физическое наличие байтов, а то, участвуют ли они в механизме поиска function entry.

Когда `RtlLookupFunctionEntry` получает `ControlPc`, система должна найти соответствующую запись в active function tables. Static tables приходят из загруженных image modules. Dynamic tables добавляются через runtime registration. Если массив `RUNTIME_FUNCTION` просто лежит в private memory, но никто не сообщил системе, что он описывает диапазон кода, то для Windows это просто байты. Она не обязана их искать.

Это похоже на дорожный знак, который лежит в багажнике автомобиля. Физически знак существует. Но участники дорожного движения не используют его, потому что он не установлен на дороге. `.pdata` в не зарегистрированной private memory - такой же знак в багажнике: анализатор может его найти, человек может понять намерение, но Windows exception dispatcher может о нем не знать.

dynamic должен по возможности различать эти состояния: metadata present in memory, metadata registered with runtime, metadata used during exception route. Это три разных уровня. Первый дает надежду. Второй дает системную видимость. Третий дает фактическое подтверждение.

Как отличить восстановленный image от сгенерированного кода

Не каждый runtime code является manual mapped PE. Иногда код действительно генерируется: JIT-компилятор, шаблонный code generator, runtime thunk builder, dynamic dispatcher. В этом случае в памяти может вообще не быть PE headers, секций, `.pdata` в привычном виде. Но Windows x64 все равно может поддерживать unwind, если runtime зарегистрирует function table для generated code.

Реверсеру важно не требовать от каждого executable private region наличия `MZ`. Если `RIP` вошел в private RX region, есть несколько гипотез. Это может быть unpacked PE-like payload. Это может быть JIT code. Это может быть shellcode-like

code. Это может быть trampoline/thunk region. Это может быть dynamically generated stubs. Эти варианты различаются по структуре памяти, плотности кода, наличию headers, наличию reloc/import-like logic, dynamic table registration, повторяемости паттернов.

static original file может увидеть только подготовку. dynamic должен классифицировать runtime region осторожно: has PE-like headers, has executable pages, has nearby runtime function table, has calls into known APIs, has dense small thunks, has exception behavior, has module association or not. Только после этого можно говорить о типе региона.

Что значит "сломанная PE-картина" без истерики

Сломанная PE-картина - это не обязательно malformed PE. Это ситуация, когда PE-модель перестает быть достаточной для объяснения runtime behavior. Файл может быть формально валидным. Exception Directory может быть корректной. `.pdata` может быть отсортирована. `.xdata` может разбираться идеально. Но если реальное исполнение уходит в payload, которого нет в этой карте, то PE-картина неполна.

Это очень важная мысль для книги. Мы не должны учить тебя смотреть на каждую аномалию как на поломку. Мы должны учить смотреть на границы применимости модели. Если модель описывает stub, она полезна для stub-а. Если runtime ушел дальше, нужна новая модель для нового кода. Иногда эту модель можно построить из memory dump. Иногда из dynamic function table. Иногда только из trace. Иногда ее нельзя восстановить полностью.

Хороший анализатор должен говорить не "я не вижу - значит нет", а "в текущем источнике evidence это не представлено". Это зрелый язык reverse engineering.

Разбор finding-а в стиле будущего отчета

Представим, что dynamic наблюдает такую картину: entry point packed image, allocation of memory region, writes into it, protection changed to executable, branch into region, no module owner, no observed dynamic function table registration, later first-chance access violation inside region.

Плохой отчет напишет: "Packer detected. Suspicious. No pdata."

Хороший отчет напишет примерно так: "Static PE analysis describes the visible loader stub only. During dynamic validation, execution transferred from the image-backed module to a MEM_PRIVATE executable region. The target address cannot be resolved through the original module Exception Directory. No dynamic function table registration for the observed region was captured in this run. Therefore, unwind support for nonleaf code in this runtime region is not established. If exceptions occur inside this region, stack walking and handler

routing may be unreliable. Manual review or memory dump analysis is recommended."

Видишь разницу? Второй вариант не просто длиннее. Он объясняет причинную цепочку: источник evidence, переход между мирами, потеря PE-координат, отсутствие function table evidence, consequence для unwind, следующий шаг. Именно такой стиль нужен книге и проекту.

Что делать реверсеру руками

Если ты руками анализируешь такой файл, полезно идти не по эмоциям, а по маршруту.

Сначала зафиксируй original PE: headers, sections, entry point, Exception Directory, `.pdata`, `.xdata`, imports, TLS, overlay. Это зона static -а. Здесь ты строишь карту того, что файл сам о себе говорит.

Потом посмотри, не является ли entry point маленьким loader stub-ом. Но не по одному признаку, а по совокупности: маленький видимый код, большие странные данные, декодирующие циклы, allocation/protection APIs, dynamic import resolution, indirect transfer.

Затем в динамике проверь, выходит ли `RIP` за пределы image-backed module. Если выходит, классифицируй memory region: image/private, protection, size, neighboring regions, PE-like headers, entropy, writes before execute, source of bytes.

Потом проверь unwind-видимость: есть ли module static table для этого адреса, есть ли dynamic function table registration, можно ли выполнить stack walk, что происходит при exception, какие handler-и срабатывают.

И только потом формулируй вывод. Не раньше. Потому что ранний вывод почти всегда превращается в миф: "упаковано значит плохо", "нет импорта значит не вызывает", "есть `.pdata` значит всё покрыто", "код исполняется значит unwind нормальный". Все четыре фразы могут быть ложными.

Почему это место готовит переход к архитектуре Analyzer-а

Этот раздел завершает главу 8 не случайно. Мы прошли путь от локальных аномалий `.pdata` и `.xdata` к более сложному случаю, где сама граница файла и памяти становится проблемой. Теперь становится очевидно, почему один статический парсер недостаточен, но и одна динамическая трасса недостаточна.

static нужен, чтобы читать договоры из файла: PE, sections, Exception Directory, static function tables, unwind metadata, handler data, findings. dynamic нужен, чтобы проверять живую реальность: execution transfer, memory regions,

runtime function table registration, exceptions, handler hits, post-handler RIP. Report layer нужен, чтобы не смешать эти миры и показать человеку evidence chain.

Именно поэтому финальная архитектура твоего проекта должна строиться вокруг источников evidence. Не "у нас есть парсер и запускатель", а "у нас есть статическая модель обещаний и динамическая модель фактического поведения". Packer/manual mapping - лучший пример, где это различие становится не теорией, а необходимостью.

Главная модель раздела

Если свести всю главу к одной мысли, она будет такой: PE-файл на диске - это не всегда вся программа. В простом случае он является хорошей картой будущего исполнения. В packed, protected, manually mapped или runtime-generated сценариях он может быть только оболочкой, stub-ом, контейнером или стартовой фазой. Поэтому статическая карта `.pdata/.xdata`, которую строит static , должна иметь границу применимости. Она верна для видимого образа, но не обязана описывать код, который появится позже.

dynamic нужен не как декоративный "запускатель", а как способ проверить живую правду: куда реально ушел RIP, из какой memory region исполняется код, есть ли для него function table, какие exceptions произошли, какой handler был hit, куда пошло управление после handler-a. Только вместе эти два слоя дают честную картину.

Для реверсера это означает практическую дисциплину. Не верить слепо файлу. Не паниковать от одного `VirtualAlloc`. Не считать отсутствие `.pdata` в original PE доказательством поломки payload-a. Не считать валидную `.pdata` stub-a доказательством валидности всей runtime-программы. Всегда спрашивать: какой мир я сейчас анализирую - файл, mapped image, dump или живое исполнение? И какая unwind-карта соответствует именно этому миру?

Следующий раздел после этого должен уже не расширять тему packer-ов, а перевести нас к архитектурной дисциплине: как анализатор формирует pipeline, как он отделяет сырые факты от findings, как связывает статическую и динамическую evidence, и как не превращает сложный Windows-механизм в шумный набор тревог.

Техническая база раздела

Этот раздел опирается на официальную модель PE/COFF, где RVA определяется как адрес объекта после загрузки образа за вычетом base address image, a data directories задают расположение важных таблиц образа.

Для Windows x64 exception/unwind-модели ключевым остается требование: nonleaf-функции должны иметь корректную unwind-информацию, чтобы exception handling, stack walking и debugger support могли восстановить caller context.

Для кода, который создается или размещается во время выполнения, Windows предоставляет dynamic function tables, включая регистрацию через RtlAddFunctionTable и callback-модель. Поэтому runtime-код должен быть не только исполнимым для CPU, но и описанным для Windows unwind-инфраструктуры, если он ведет себя как nonleaf-код.

Глава 9. Архитектура анализатора: как теория превращается в инструмент

9.1. Pipeline static -a: от недоверенного PE-файла к доказательной карте

К этому месту книги мы уже прошли длинный путь. Сначала мы разбирали отдельные механизмы: как устроен стек, что делает call, почему ret берет адрес возврата из памяти, зачем функции сохраняют nonvolatile-регистры, откуда берутся .pdata и .xdata, как RUNTIME_FUNCTION ведет к UNWIND_INFO, как Windows доставляет исключение и почему handler может стать не просто местом аварийного выхода, а частью маршрута управления. Все это было необходимо, но само по себе еще не делает из нас автора анализатора.

Анализатор начинается не там, где мы знаем много терминов. Он начинается там, где мы умеем заставить эти термины работать в правильном порядке. Если человек видит в PE-файле секцию .pdata и сразу прыгает к выводу, он еще не анализирует. Он угадывает. Если программа находит байты, похожие на UNWIND_INFO, и сразу пишет warning, она тоже еще не анализирует. Она шумит. Настоящий static должен двигаться медленнее. Он должен не просто находить структуры, а строить доказательную карту: откуда взялся каждый факт, почему ему можно доверять, на каком слое он был проверен, где заканчивается уверенность и где начинается гипотеза.

Слово pipeline обычно переводят как конвейер или цепочка обработки. Но для нашей темы это слово опасно, потому что может создать слишком механическую картинку: будто у нас есть набор этапов, и файл просто едет по ним, как деталь на заводе. В хорошей архитектуре static -a pipeline - это не просто последовательность функций в коде. Это дисциплина мышления. Каждый этап получает не “истину”, а результат предыдущего этапа вместе с ограничениями этого результата. Каждый следующий шаг не забывает, откуда пришли данные. Именно поэтому pipeline важен не меньше, чем сами парсеры .pdata и .xdata.

От механизма Windows к инженерной дисциплине

Пока мы изучали Windows x64 exception handling, мы смотрели на систему в основном с точки зрения самой Windows. Вот есть RIP - адрес текущей инструкции. Вот есть RSP - вершина текущего стека. Вот есть return address - адрес, куда функция должна вернуться после завершения. Вот есть RUNTIME_FUNCTION - запись, которая говорит, какой диапазон кода относится к функции и где лежит unwind-информация. Вот есть UNWIND_INFO - компактная инструкция, по которой Windows может восстановить кадр вызывающей функции без обычного ret. Эта модель нужна системе для исключений, stack walking, crash dump-ов, отладчиков, профилировщиков и всего, что пытается понять стек в момент, когда программа не идет обычным спокойным путем.

Но static работает в другой позиции. Windows обычно работает с уже загруженным образом, с реальными адресами в памяти, с активными function tables, с текущим CONTEXT потока. static же часто начинает с файла на диске. Перед ним лежит набор байтов. Эти байты могут быть нормальным компиляторским PE. А могут быть файлом после packer-а, результатом ручной сборки, поврежденным образцом, тестовым бинарем, обфусцированным объектом, файлом с нарочно странными секциями или просто криво собранной программой. static не имеет права открыть файл и сразу вести себя так, будто все заголовки честные.

Именно здесь возникает инженерная разница между “я знаю формат PE” и “я умею анализировать PE”. Знать формат - значит помнить, где лежит DOS header, NT headers, Optional Header, section table, data directories. Анализировать - значит понимать, что каждое поле в этих структурах является заявлением файла о себе. Файл как бы говорит: “моя секция начинается здесь”, “мой ImageBase вот такой”, “Exception Directory лежит по такому RVA”, “в этом диапазоне находятся записи RUNTIME_FUNCTION”. static должен выслушать эти заявления, но не обязан им верить без проверки.

> Главная мысль 9.1: static - это не парсер ради парсинга. Это система, которая превращает недоверенные байты в цепочку проверенных наблюдений. Чем аккуратнее эта цепочка, тем меньше ложных тревог и тем ценнее каждый finding.

Файл как источник показаний, а не источник истины

Начнем с самой важной психологической перестройки. PE-файл не является абсолютной истиной о программе. Он является источником показаний. Это почти судебная метафора, но она хорошо подходит. Свидетель может говорить правду, может ошибаться, может умалчивать, может быть неполным, а иногда может намеренно вводить в заблуждение. PE-заголовки тоже могут быть корректными, частично корректными, противоречивыми или враждебными к инструменту.

Когда мы читаем Optional Header и видим ImageBase, это еще не значит, что все адресные вычисления автоматически стали надежными. Когда мы видим section table, это еще не значит, что каждая секция действительно лежит в файле так, как обещано. Когда мы видим Data Directory для Exception Directory, это еще не значит, что там действительно лежит таблица RUNTIME_FUNCTION, что ее размер кратен 12 байтам, что записи отсортированы, что BeginAddress меньше EndAddress, что UnwindInfoAddress указывает в допустимое место, и что .xdata читается без выхода за границы файла.

Такое отношение не является паранойей. Это нормальная позиция анализатора. Компилятор и linker в обычной программе действительно создают аккуратную структуру. Но reverse engineering почти никогда не занимается только идеальными учебными программами. Мы смотрим на чужой файл. Мы не знаем, какой toolchain его сделал. Мы не знаем, не прошел ли он через packer. Мы не знаем, не были ли секции передвинуты, не были ли таблицы частично стерты, не был ли файл специально подготовлен так, чтобы сбить простой парсер. Поэтому первый слой static -а должен быть не “читать красивые структуры”, а “аккуратно установить, какие части файла вообще можно читать безопасно”.

Здесь появляется термин trust boundary - граница доверия. По-русски это можно перевести как граница, после которой данные больше нельзя считать своими и безопасными. В нашем случае такая граница проходит прямо по входному PE-файлу. Все, что пришло из файла, сначала недоверенно. Даже если поле называется SizeOfImage, это еще не настоящий размер безопасного буфера. Даже если поле называется NumberOfSections, это еще не гарантия, что section headers действительно помещаются в файл. Даже если data directory обещает Exception Directory, это еще не гарантия, что по этому RVA можно получить корректный file offset.

Хороший static должен не просто иметь функцию read_u32. Он должен иметь культуру чтения: перед каждым переходом по адресу он спрашивает, в каком пространстве координат находится этот адрес, какой диапазон он обещает, где этот диапазон должен лежать, можно ли его отобразить в файл,

не выходит ли чтение за конец буфера, не противоречит ли это секционной карте. Только после этого он имеет право сказать: “я наблюдаю такую-то структуру”.

Три пространства координат: file offset, RVA и VA

Один из главных источников ошибок в PE-анализе - смешение координат. Новичок часто видит число и пытается сразу понять, “это адрес или смещение?”. Но в PE одно и то же число может иметь смысл только внутри конкретной системы координат. Поэтому pipeline static -а должен очень рано построить для себя карту перевода между file offset, RVA и VA.

File offset - это смещение внутри файла на диске. Если открыть PE как массив байтов, file offset отвечает на простой вопрос: на каком байте файла находится нужная структура. Это координата архива. Она важна для static -а, потому что static читает именно файл. Когда он хочет взять четыре байта из входного буфера, ему нужен file offset.

RVA - Relative Virtual Address, относительный виртуальный адрес. В PE-документации RVA означает адрес объекта после загрузки образа в память, но без прибавленного ImageBase. Иначе говоря, если код в памяти лежит по адресу ImageBase + 0x1234, то RVA этого кода равен 0x1234. RVA - это координата внутри образа, а не внутри файла. Она ближе к тому, как loader и Windows думают о загруженном image.

VA - Virtual Address, виртуальный адрес. Это уже полный адрес в адресном пространстве процесса: ImageBase + RVA. Именно VA увидит CPU в RIP, именно VA увидит отладчик в disassembly, именно VA будет фигурировать в контексте потока. Но файл на диске не обязан быть разложен так же, как образ в памяти. Поэтому VA нельзя просто использовать как смещение в файле. Его нужно сначала превратить в RVA, а RVA потом сопоставить с секциями и перевести в file offset.

```
file offset: где байты лежат в файле на диске
RVA        : где объект будет находиться внутри загруженного image
VA         : полный адрес в памяти процесса, обычно ImageBase + RVA
```

Эта тройка кажется простой, пока мы не начинаем читать .pdata. Запись RUNTIME_FUNCTION в PE32+ хранит BeginAddress, EndAddress и UnwindInfoAddress как image-relative offsets, то есть как RVA относительно image base. Это очень важный момент. Если static прочитал BeginAddress = 0x1400, он не должен думать, что это 0x1400-й байт файла. Он должен понять: это RVA внутри image. Чтобы прочитать код функции из файла, нужно найти секцию, чей virtual range покрывает 0x1400, и через эту секцию вычислить file offset.

Именно поэтому PE parser в static -е не является “скучной подготовкой”. Это основа всего последующего анализа. Ошибка в RVA-to-file-offset превращает весь unwind decoder в генератор мусора. Если мы неверно перевели UnwindInfoAddress, мы прочитаем не тот байт как Version/Flags, не то значение как CountOfCodes, не тот хвост как handler RVA. Потом heuristic engine честно найдет “аномалии”, но эти аномалии будут порождены не файлом, а ошибкой нашего перевода координат.

В хорошей книге это место нельзя пробежать. Для reverse engineering оно фундаментальное. Когда ты видишь RIP в debugger-е, это VA. Чтобы связать его с PE-файлом, ты находишь module base, вычитаешь его и получаешь RVA. Когда static читает Exception Directory из файла, он получает RVA и должен перевести его в file offset. Когда dynamic динамически показывает, куда ушел RIP после handler-а, он показывает живой VA; чтобы сравнить это со статической картой static -а, снова нужен переход VA -> module -> RVA -> запись .pdata. Вся связка static /dynamic держится на этой адресной дисциплине.

Секционная карта как первая настоящая модель файла

После чтения заголовков static должен построить section map - карту секций. Секция в PE - это не просто имя вроде .text, .rdata или .pdata. Секция задает связь между тем, как данные лежат в файле, и тем, как они должны выглядеть в памяти после загрузки. У нее есть raw range - диапазон в файле, и virtual range - диапазон в образе. Эти два диапазона похожи, но не равны. Именно между ними static строит мост.

Если объяснять на человеческом образе, файл на диске похож на сложенный чертеж, а mapped image в памяти - на развернутую конструкцию. В сложенном виде детали лежат в одном порядке и с одними выравниваниями. После загрузки они занимают виртуальные адреса, получают protection flags, попадают в разные страницы памяти. PE-секция описывает, как часть сложенного чертежа должна оказаться в развернутой конструкции. Поэтому без section map нельзя честно говорить ни о .pdata, ни о .xdata, ни о handler-ах.

Здесь же появляется еще одна важная мысль: имя секции полезно, но не является абсолютной истиной. В нормальном MSVC-бинаре exception function table часто лежит в .pdata, а unwind information - в .xdata. Но static не должен искать только строку “.pdata” в таблице секций и на этом успокаиваться. Официальный путь к exception data идет через Data Directory, а не через имя секции. Имя помогает человеку ориентироваться, но analyzer должен опираться на структуру PE: directory entry, RVA, size, section mapping, bounds check.

Это особенно важно для странных файлов. Packer может переименовать секции. Linker может объединять данные. Образ может иметь нестандартные имена секций. Ручной генератор может положить данные туда, где имя не соответствует ожиданию. Если static говорит “я не нашел секцию .pdata, значит exception information нет”, он ошибается. Правильная мысль звучит иначе: “я проверил Exception Directory; он отсутствует, пуст, или указывает на такой-то диапазон; этот диапазон отображается/не отображается через section map”.

Так pipeline начинает превращаться в доказательную карту. Мы не просто “нашли .pdata”. Мы можем объяснить путь: PE headers сообщили Data Directory для Exception Directory; этот directory дал RVA и size; section map показала, что этот RVA попадает в такую-то секцию; raw range секции позволяет безопасно прочитать соответствующие байты; размер directory кратен размеру RUNTIME_FUNCTION; каждая запись может быть разобрана дальше. У каждого шага есть основание. У каждого шага есть место, где он может отказать.

Exception Directory как вход в unwind-модель

Exception Directory в PE для x64 - это дверь в таблицу функций, участвующих в unwind-модели. Важно сказать именно так: не “дверь в список всех функций”, не “дверь в обработчики исключений”, не “дверь в SEH”. Это дверь в function table, где каждая запись описывает диапазон кода и адрес unwind-информации для этого диапазона. Уже из этой фразы видно, почему новички часто ошибаются. Они слышат слово exception и думают, что directory обязательно связан только с try/except. На x64 это шире: эта таблица нужна не только для явных exception handler-ов, а для самой способности Windows раскрутить стек через nonleaf-функции.

Nonleaf-функция - это функция, для которой обычной симуляции ret недостаточно. Обычно она либо вызывает другие функции, либо выделяет место на стеке, либо сохраняет nonvolatile-регистры, либо строит frame pointer, либо делает другие действия, из-за которых состояние caller-а нельзя восстановить простой догадкой. Для такой функции Windows нужен договор: если исключение случится внутри тела функции, как понять, где лежит return address, насколько был сдвинут RSP, какие регистры нужно восстановить и где они сохранены. Этот договор лежит не в комментариях, не в символах и не в исходном коде. Он лежит в .pdata/.xdata.

Поэтому Exception Directory в static -е должен восприниматься как начало declared unwind model - заявленной модели раскрутки. Слово declared здесь важно. Файл заявляет: “для диапазона BeginAddress..EndAddress используй UNWIND_INFO по UnwindInfoAddress”. static пока не знает, правда ли эта модель соответствует машинным инструкциям. Он знает только, что файл так утверждает. Позже он сможет сверить эту модель с

прологом, секциями, handler-ами, dynamic behavior. Но первый честный вывод именно такой: это заявленная модель.

В хорошей архитектуре static -а на этом этапе нельзя перепрыгивать к подозрительности. Если Exception Directory отсутствует, это может быть нормально для маленького leaf-only executable, а может быть странно для большого PE с большим количеством nonleaf-кода. Если directory есть, но маленький, это может быть результатом оптимизации, а может быть неполное покрытие. Если directory указывает в необычную секцию, это может быть нормальная сборочная особенность, а может быть metadata placement signal. Сама по себе находка еще не verdict. Она становится значимой только в контексте последующих проверок.

RUNTIME_FUNCTION как диапазон, а не имя функции

Теперь pipeline входит в .pdata. Здесь особенно важно не обмануться словом function. RUNTIME_FUNCTION не хранит имя функции, не хранит ее сигнатуру, не знает, сколько у нее аргументов, не знает, как она называлась в C или C++. Это запись диапазона. Она говорит: начиная с такого RVA и до такого RVA действует такая unwind-информация. Для отладчика и Windows это не “функция как в исходнике”, а участок машинного кода, для которого можно применить определенное правило восстановления кадра.

Почему это важно для книги и для static -а? Потому что reverse engineering почти всегда страдает от попытки насильно вернуть машинный код к удобным исходным понятиям. Человек хочет видеть “функции”. Но бинарь часто дает ему диапазоны, переходы, хвостовые вызовы, thunk-и, padding, объединенные эпилоги, куски runtime-а, compiler-generated blocks. RUNTIME_FUNCTION - это не обещание красивой функции из учебника. Это обещание unwind-диапазона. Иногда он совпадает с тем, что декомпилятор назовет функцией. Иногда нет.

BeginAddress - это RVA начала диапазона. EndAddress - RVA конца диапазона. UnwindInfoAddress - RVA структуры UNWIND_INFO или chained data, которая объясняет, как раскручивать кадр для этого диапазона. На x64 эти поля 32-битные, потому что они image-relative offsets, а не полные 64-битные VA. Это экономит место и делает таблицу компактной. Но для static -а это означает еще одну дисциплину: любое такое поле нужно интерпретировать относительно image base и section map, а не как file offset.

Хороший static на этом этапе не должен просто сложить все записи в массив. Он должен начать строить range map - карту диапазонов. У range map есть свои свойства: записи должны иметь BeginAddress меньше EndAddress; они обычно отсортированы; они не должны бессмысленно пересекаться; они могут иметь gaps; они могут указывать на один и тот же UNWIND_INFO; они могут покрывать только часть .text. Все эти свойства не являются отдельными

“галочками”. Они вытекают из того, как Windows потом будет искать запись по PC value.

Когда Windows или инструмент вызывает RtlLookupFunctionEntry, смысл операции простой: есть ControlPc, то есть адрес внутри текущей функции; нужно найти function table entry, соответствующий этому PC. Если range map неоднозначна, пересекается или содержит странные интервалы, поиск становится семантически опасным. Даже если байты таблицы синтаксически читаются, модель может быть плохой. Именно поэтому static не ограничивается parsing stage. После parsing stage обязательно идет validation stage: не только “прочитали”, но и “поняли, насколько эта карта похожа на карту, с которой Windows сможет работать предсказуемо”.

UNWIND_INFO как заявленное поведение кадра

После .pdata pipeline переходит к .xdata. Здесь соблазн написать decoder и считать задачу выполненной особенно велик. Version, Flags, SizeOfProlog, CountOfCodes, FrameRegister, FrameOffset, массив UNWIND_CODE, padding, handler RVA или chained info - все это выглядит как набор полей. Но для книги и для хорошего static -а важнее не сами поля, а смысл, который из них собирается.

UNWIND_INFO - это заявленное поведение кадра функции. Оно не исполняется процессором напрямую. CPU не читает .xdata, когда выполняет инструкции. Процессор просто исполняет push, sub rsp, mov, call, ret. UNWIND_INFO нужен Windows и инструментам, когда обычный путь выполнения нарушен или когда нужно посмотреть стек со стороны. Если исключение случилось внутри функции, Windows не может просто выполнить ret, потому что функция могла не дойти до эпилога. Если отладчику нужно построить call stack, он не может надеяться, что каждая функция сейчас аккуратно завершится. Нужна внешняя инструкция: как отменить эффект пролога и получить caller context.

Вот почему UNWIND_INFO нужно объяснять не как “структуру”, а как договор. Компилятор создал машинный код и одновременно оставил Windows договор: если ты окажешься внутри этой функции, вот как понять, сколько места она выделила на стеке, какие nonvolatile-регистры сохранила, где их искать, используется ли frame pointer, есть ли language-specific handler, есть ли chained entry. Windows потом пользуется этим договором в RtlVirtualUnwind и в exception dispatch. Если договор соответствует коду, стек можно восстановить. Если договор не соответствует коду, система может восстановить неправильный caller context, потерять return address, не туда передать управление или построить ложный stack trace.

static на этапе .xdata должен поэтому делать две разные вещи. Первая - синтаксически разобрать структуру: не выходит ли CountOfCodes за границы,

правильно ли выровнен хвост, не конфликтуют ли flags, указывает ли handler RVA в допустимый диапазон. Вторая - построить declared effect model, то есть модель эффекта пролога, которую заявляет metadata. Эта модель отвечает не просто “какие UWOP_* встретились”, а “что, по мнению metadata, функция сделала со стеком и регистрами”.

Например, если UNWIND_CODE говорит UWOP_ALLOC_SMALL на 0x40 байт, static должен понять это как изменение модели RSP: в теле функции RSP сдвинут относительно состояния после call. Если metadata говорит UWOP_PUSH_NONVOL RBX, static должен понять: RBX был сохранен на стеке и при unwinding должен быть восстановлен из соответствующего места. Если установлен UNW_FLAG_EHANDLER, static должен понять: после массива unwind codes и выравнивания есть handler RVA и language-specific data. Эти выводы уже не просто байты. Это семантика.

Declared model и observed model

Здесь полезно ввести две модели, которые должны стать центральными для static -а. Declared model - заявленная модель. Она строится из PE-метаданных: .pdata, .xdata, UNWIND_INFO, flags, handler data. Observed model - наблюдаемая модель. Она строится из реального кода: дизассемблированный пролог, инструкции push/sub/mov, сохранения nonvolatile-регистров, эпилоги, call-и, переходы, фактические handler-like blocks. В идеальном бинаре declared model и observed model рассказывают одну и ту же историю разными языками.

Скажем, код начинается с push rbx; sub rsp, 0x40. Metadata говорит: был сохранен RBX и выделено 0x40 байт стека. Это согласованная история. Или код строит frame pointer через rbp, и UNWIND_INFO содержит UWOP_SET_FPREG. Снова согласованная история. Но если код явно делает sub rsp, 0x80, а metadata заявляет только 0x20, это уже mismatch - несовпадение. Важно не просто назвать его подозрительным, а объяснить последствие: при virtual unwind Windows восстановит RSP так, будто кадр был меньше, чем он есть, и может прочитать return address не оттуда.

Именно так finding должен вырастать из теории. Мы не пишем “bad unwind info”, потому что нам не понравились байты. Мы пишем: “metadata заявляет размер кадра X, наблюдаемый пролог создает размер кадра Y; это влияет на восстановление RSP и поиск return address; consequence - unreliable stack walking / exception handling; confidence зависит от качества дизассемблирования и распознавания пролога”. Такой finding не орет. Он объясняет.

Эта пара моделей также связывает static с dynamic -ом. static может построить declared и observed model статически. Но если файл сложный, packer-ный, JIT-овый или использует runtime code, static model

может быть неполной. Тогда static не должен притворяться всевидящим. Он должен сформулировать вопрос для dynamic -а: “при попадании RIP в этот диапазон есть ли активная function table entry?”, “срабатывает ли этот handler?”, “куда уходит RIP после handler hit?”, “совпадает ли runtime route с declared metadata?”. dynamic не заменяет static . Он проверяет те места, где статическая карта нуждается в живом подтверждении.

```
Declared model  : что файл заявляет через PE/.pdata/.xdata
Observed model  : что видно в реальном машинном коде
Runtime route   : что dynamic наблюдает при исполнении
```

Finding как маленькое расследование

Теперь можно аккуратно перейти к слову finding. В технических отчетах finding часто переводят как находка, выявленный факт или результат проверки. Но в нашем проекте finding не должен быть просто строкой warning. Хороший finding - это маленькое расследование, сжатое до понятной формы. Он должен сохранять связь между наблюдением, слоем механизма, причиной важности и ограничением уверенности.

Представим плохой finding: “Suspicious .pdata overlap”. Он вроде бы сообщает о проблеме, но почти ничему не учит. Человек видит слово suspicious и overlap, но не понимает, где именно overlap, почему это плохо, что может сломаться, насколько это уверенно, и что должен сделать следующий инструмент. Такой вывод раздражает, потому что он похож на сигнализацию без объяснения.

Хороший finding устроен иначе. Он говорит: “Две записи RUNTIME_FUNCTION имеют пересекающиеся диапазоны RVA: запись A покрывает 0x1400..0x1480, запись B покрывает 0x1460..0x1500. Это создает неоднозначность range map для ControlPc внутри 0x1460..0x1480. Поскольку RtlLookupFunctionEntry ищет entry по PC value, такая неоднозначность может привести к неправильному выбору unwind metadata или указывает на нестандартную/поврежденную таблицу. Confidence высокий, потому что оба диапазона получены из Exception Directory и корректно отображены через section map. Рекомендуется сверить с дизассемблированным кодом и, при dynamic run, проверить фактический active function table.”

Да, такой finding длиннее. Но он полезнее. Он обучает. Он не требует от читателя слепой веры инструменту. Он показывает, как static думает. Для нашей книги это принципиально: мы пишем не только для того, чтобы перечислить API и структуры, а чтобы сформировать мышление реверсера и автора инструмента.

Внутри finding-а есть несколько слоев. Evidence - доказательство или наблюдение: конкретные адреса, поля, секции, байты, диапазоны. Layer - слой механизма: PE mapping, .pdata range map, .xdata decoding, handler metadata,

code-vs-metadata match, dynamic validation. Consequence - последствие: что может сломаться или почему это важно. Confidence - уверенность: насколько надежно это установлено. Severity - серьезность: насколько потенциально опасно или существенно обнаруженное поведение. Эти слова нельзя путать.

Confidence и severity особенно часто смешивают. Confidence отвечает на вопрос: насколько мы уверены, что наблюдение реально. Severity отвечает на вопрос: насколько серьезно это наблюдение, если оно реально. Можно иметь высокий confidence и низкий severity: например, мы точно нашли небольшой gap между функциями, но он похож на padding. Можно иметь низкий confidence и высокий potential severity: например, статически есть признаки handler-driven control flow, но без dynamic validation пока нельзя утверждать маршрут. Хороший static должен уметь честно разделять эти вещи.

Unknown как честный результат

В плохих инструментах часто есть скрытый страх перед словом unknown. Кажется, что analyzer должен обязательно выдать verdict: normal, suspicious, malicious, broken. Но в reverse engineering честное unknown иногда ценнее уверенного nonsense. Если static не может безопасно перевести RVA в file offset, он должен сказать об этом. Если .xdata синтаксически не читается из-за выхода за границы, он должен остановить этот путь анализа, а не продолжать интерпретировать мусор. Если static evidence недостаточно, чтобы отличить нормальный packer stub от malicious unpacking route, он должен сформулировать dynamic question для dynamic -а.

Unknown не означает слабость инструмента. Unknown означает, что инструмент не перешел границу доказанности. Это очень важная инженерная культура. В безопасности и reverse engineering слишком легко построить красивый отчет, который выглядит уверенно, но держится на догадках. Наша архитектура должна быть другой. static должен уметь говорить: “я проверил это; это доказано”, “я не могу проверить это статически; требуется dynamic validation”, “это похоже на нормальный вариант, но есть такой-то открытый риск”, “это выглядит подозрительно, но confidence ограничен, потому что дизассемблирование неполное”.

Именно здесь появляется правильная связь с dynamic -ом. dynamic - не просто “второй режим программы”. Это динамическая проверка тех мест, где статическая карта либо неполна, либо нуждается в подтверждении. Если static видит handler RVA, dynamic может поставить breakpoint и увидеть handler hit. Если static видит private executable memory или dynamic function table API, dynamic может показать, куда реально ушел RIP. Если static предполагает exception-based control flow, dynamic может подтвердить exception event, continuation status, post-handler RIP trace. То есть unknown в static -е часто превращается не в тупик, а в конкретное задание

для dynamic -а.

Связь с реальным проектом без превращения главы в документацию

В конце книги мы отдельно разберем архитектуру твоего проекта подробно: PE parser, unwind decoder, heuristic engine, reporting layer, dynamic collector, Ghidra integration, тесты, corpus, fuzzing, JSON schema. Но уже здесь важно аккуратно привязать теорию к реальному коду, чтобы книга не висела в воздухе.

То, что в этой главе называется static pipeline, в реальном проекте соответствует не одной функции, а нескольким слоям. Условно первый слой читает файл и строит безопасные примитивы чтения. Следующий слой разбирает PE и создает section map. Затем unwind decoder проходит по Exception Directory, читает RUNTIME_FUNCTION entries и UNWIND_INFO. Потом heuristics превращают сырые наблюдения в findings. Reporting layer делает эти findings пригодными для человека и других инструментов. Это не случайное разбиение. Оно повторяет саму природу анализа: от байтов к координатам, от координат к структурам, от структур к семантике, от семантики к доказательному выводу.

Если смешать эти слои, получится хрупкий инструмент. Например, если unwind decoder сам “на глаз” переводит RVA в file offset без общей section map, разные части программы начнут по-разному понимать один и тот же адрес. Если heuristics начнут читать байты напрямую, минуя валидированный parser, findings будут зависеть от частных допущений. Если report будет показывать только verdict без evidence chain, пользователь не сможет проверить вывод. Хорошая архитектура не просто удобна программисту. Она защищает смысл анализа от распада.

Для книги это тоже урок. Мы не строим static как набор “фич”: распарсить PE, распарсить .pdata, распарсить .xdata, найти странности. Мы строим инструмент как продолжение теории Windows x64 unwind. Если Windows сама ищет function table entry по ControlPc, значит static должен уметь объяснить, какая entry соответствует какому диапазону. Если Windows восстанавливает caller context через RtlVirtualUnwind, значит static должен уметь объяснить, какую модель RSP и nonvolatile-регистров заявляет UNWIND_INFO. Если Windows может использовать dynamic function tables, значит dynamic должен наблюдать runtime registration, а static должен честно признавать границы анализа файла на диске.

Пример прохождения одного адреса через pipeline

Чтобы модель стала живой, пройдем мысленно один адрес. Допустим, у нас есть RIP из crash dump-а или debugger-а: 0x00007FF6A1234567. Сам по себе этот адрес почти ничего не говорит. Это полный виртуальный адрес в адресном пространстве процесса. Первое, что нужно сделать, - понять, какому модулю он принадлежит. Если module base равен 0x00007FF6A1200000, то RVA будет 0x34567. Теперь этот адрес можно сравнивать с PE-структурами модуля.

static , работая по файлу, заранее построил section map и прочитал Exception Directory. В .pdata он ищет такую запись RUNTIME_FUNCTION, у которой BeginAddress <= 0x34567 < EndAddress. Если запись найдена, например 0x34000..0x346A0, то static понимает: текущий RIP лежит внутри unwind-диапазона этой функции. Дальше он берет UnwindInfoAddress из записи, переводит его через section map в file offset и читает UNWIND_INFO. Из UNWIND_INFO он строит declared model: размер пролога, выделение стека, сохраненные регистры, handler flags, possible handler data.

Но хороший static не останавливается на “нашел”. Он спрашивает: эта запись корректна? Диапазон не пересекается с соседними? UnwindInfoAddress указывает в readable место? CountOfCodes не выводит чтение за границы? Flags не конфликтуют? Handler RVA, если есть, попадает в исполняемый или ожидаемый диапазон? Пролог кода действительно похож на то, что заявляет metadata? Только после этой цепочки можно говорить не просто “RIP принадлежит функции”, а “для этого RIP есть такая-то статическая unwind-модель с такой-то степенью уверенности”.

Если дальше включается dynamic , он берет уже не абстрактный RVA, а живой runtime route. Он может увидеть exception event с этим RIP, проверить, какая function table entry реально активна в процессе, hit-нулся ли handler, какой был post-handler RIP, ушло ли управление обычным unwind-ом, crash-ем, continue execution или скрытым exception edge. Тогда static map и dynamic route встречаются. static сказал, что карта выглядит так. dynamic показал, что в живом процессе управление прошло так. Это и есть цель архитектуры: не спор “статика против динамики”, а согласование двух источников наблюдения.

```
RIP in debugger
  -> module base
  -> RVA
  -> RUNTIME_FUNCTION range
  -> UNWIND_INFO
  -> declared unwind model
  -> static finding / dynamic question
  -> dynamic     validation
```

Отчет как продолжение анализа, а не украшение

В конце pipeline-а появляется report. Его легко недооценить: кажется, что главное - распарсить, а отчет можно как-нибудь вывести. Но для reverse engineering отчет - это часть анализа. Если отчет теряет evidence chain, инструмент теряет ценность. Если отчет пишет только “suspicious”, человек не понимает, что именно проверять. Если отчет не различает static и dynamic evidence, читатель не понимает, что доказано файлом, а что подтверждено исполнением.

Поэтому report должен быть устроен как рассказ о проделанном пути. Он должен показать input file, его hash, архитектуру, image base, секции, Exception Directory, количество function table entries, основные anomalies, findings с evidence, confidence и severity, а также dynamic validation, если она проводилась. Это не бюрократия. Это возможность вернуться к выводу и проверить его руками.

Для Ghidra integration это особенно важно. Когда static экспортирует CSV для Ghidra, он не просто переносит таблицу адресов. Он помогает реверсеру наложить unwind-карту на дизассемблированный код. Комментарии, bookmarks, предупреждения, function ranges - все это должно быть привязано к той же доказательной модели. Иначе интеграция станет красивой, но бессмысленной. Хороший инструмент не просто “показывает много данных”. Он показывает данные в таком виде, чтобы реверсер мог продолжить мысль.

Главная модель раздела

Теперь можно собрать весь раздел в одну картину. static начинается с недоверенного PE-файла. Он не верит заголовкам слепо, а проверяет их как показания. Он строит section map, потому что без нее нельзя честно переводить RVA в file offset. Он входит в Exception Directory не через имя секции, а через PE data directory. Он читает RUNTIME_FUNCTION entries как range map, а не как список функций из исходника. Он читает UNWIND_INFO как declared unwind model, а не как набор полей. Он сравнивает эту модель с кодом, чтобы отличить синтаксически валидные байты от семантически правдивой metadata. Он формулирует findings как маленькие расследования, а не как резкие ярлыки. И когда статическая карта не может ответить на вопрос до конца, он передает этот вопрос dynamic -у.

Эта модель важна не только для главы 9. Она объясняет, зачем мы столько времени тратили на фундамент. Если ты не понимаешь, что такое return address, ты не поймешь, почему ошибка RSP ломает unwind. Если ты не понимаешь разницу RVA и file offset, ты неправильно прочитаешь .xdata. Если ты не понимаешь, что RUNTIME_FUNCTION - это диапазон, ты будешь ждать от

.pdata списка красивых функций. Если ты не понимаешь first chance, handler hit и post-handler RIP, ты не поймешь, что именно dynamic валидирует. Все механизмы, изученные раньше, теперь превращаются в архитектуру инструмента.

Следующий раздел будет естественным продолжением этого разговора. Мы уже поняли, что finding - это не просто warning. Теперь нужно отдельно разобрать evidence model: как именно описывать факт, слой, причину важности, последствия, confidence, severity и manual review так, чтобы отчет static -а учил реверсера, а не засыпал его красными лампочками.

> Редакторский принцип для следующих разделов: сначала объясняется проблема и ментальная модель, затем вводятся термины, затем показывается связь с Windows, затем связь с static /dynamic, и только после этого появляются findings. Не наоборот.

9.2. Evidence model: finding как маленькое расследование, а не строка предупреждения

После того как static научился проходить путь от недоверенного PE-файла к секциям, от секций к Exception Directory, от Exception Directory к .pdata, а от .pdata к .xdata, у него появляется соблазн сделать то, что делают многие слабые анализаторы: увидеть странность и сразу вывести предупреждение. Например: rf_overlaps, unwind_rva_oob, handler_writable_section. На экране это выглядит убедительно. Есть короткий код, есть severity, есть красная строка. Но для реверсера такая строка сама по себе почти ничего не решает. Она говорит, что где-то сработало правило, но не объясняет, что именно было увидено, на каком уровне это было увидено, почему это важно, какие нормальные объяснения ещё возможны и что нужно проверить дальше.

Именно здесь начинается модель доказательств. Слово evidence переводится как «свидетельство», «доказательное основание», «то, на что мы опираемся». Это не математическое доказательство в строгом смысле, где из аксиом неизбежно следует теорема. В reverse engineering почти всегда другая ситуация. Мы смотрим на чужой бинарный файл, не знаем исходников, не знаем намерения автора, не всегда знаем компилятор, настройки линковки, упаковку, защиту, runtime-поведение. Поэтому мы редко можем честно сказать: «это точно означает X». Чаще мы можем сказать: «я наблюдаю такой факт; в нормальной модели Windows он обычно означает одно; при такой-то комбинации с другими фактами он начинает выглядеть подозрительно; для уверенности нужна такая-то динамическая проверка». Хороший finding должен сохранять эту честность.

В слабом отчёте finding похож на сигнализацию без объяснения. Она орёт, но не учит. В сильном отчёте finding похож на короткую запись опытного исследователя в рабочем журнале: где именно он увидел странность, как он перевёл адрес, почему это относится именно к unwind-механике, какие последствия это может иметь для RtlVirtualUnwind, и почему человеку стоит открыть это место в Ghidra или проверить его через dynamic. Разница между этими двумя подходами огромная. Первый подход производит шум. Второй строит понимание.

Доказательство начинается раньше, чем появляется слово suspicious

Нужно привыкнуть к простой мысли: finding не начинается с verdict-а. Он не начинается со слов «подозрительно», «плохо», «сломано» или «malware». Он начинается с наблюдения. Наблюдение - это то, что можно показать пальцем в

данных. Например: в PE-файле есть запись RUNTIME_FUNCTION, у которой BeginAddress больше или равен EndAddress. Или UnwindInfoAddress указывает за пределы SizeOfImage. Или UNWIND_INFO расположен в секции с правом записи. Или несколько сотен функций ссылаются на один и тот же handler RVA.

Такое наблюдение ещё не является выводом. Это только первый камень. Чтобы построить из него доказательство, static должен пройти несколько смысловых ступеней. Сначала он фиксирует сырой факт. Потом переводит его в модель Windows: что этот факт означает для поиска function table, для раскрутки стека, для поиска обработчика, для crash dump-а, для debugger-а. Потом он оценивает нормальные объяснения: мог ли такой факт появиться из-за маленького sample-а, из-за leaf-функций, из-за thunk-ов, из-за runtime-generated code, из-за packer stub-а, из-за нестандартного компилятора. И только после этого он может осторожно сказать: этот факт слабый, средний или сильный; он одинокий или подтверждается другими слоями; он требует ручной проверки или уже даёт устойчивый case.

Эта последовательность важна не ради красивой архитектуры. Она защищает анализатор от главной болезни автоматических инструментов: превращать каждое отклонение от учебного идеала в тревогу. Windows x64 unwind-модель строгая, но реальные бинарники разнообразны. В одном файле отсутствие .pdata на кусочке .text может быть нормальной leaf-функцией. В другом тот же uncovered range может содержать sub rsp, 0x80, сохранение RBX, несколько call и фактически быть nonleaf-кодом без корректной unwind-записи. Одинаковое поверхностное наблюдение приводит к разным выводам, если мы смотрим на контекст.

Поэтому evidence model - это не украшение отчёта. Это способ заставить static думать слоями. Слой файла отвечает на вопрос: где байты. Слой PE отвечает на вопрос: как эти байты заявлены как image. Слой unwind metadata отвечает на вопрос: что компилятор или автор бинаря сказал Windows о функциях. Слой кода отвечает на вопрос: что инструкции реально делают с RSP, регистрами и переходами. Слой динамики, который относится уже к dynamic-у, отвечает на вопрос: что действительно произошло при запуске, какие exception events были замечены, какой handler был hit, куда ушёл RIP после обработки.

Observation, interpretation и conclusion как три разные операции мышления

В книге дальше часто будут встречаться слова observation, interpretation и conclusion. Их важно развести, потому что именно из-за смешения этих уровней отчёты становятся нечестными.

Observation - наблюдение. Это максимально близкий к данным уровень. Если static говорит, что UnwindInfoAddress = 0x00124000, а SizeOfImage =

0x00110000, то это observation. Его можно перепроверить арифметически. Если он говорит, что секция, содержащая UNWIND_INFO, имеет флаг writable, это тоже observation. Мы можем открыть section header и увидеть Characteristics. На этом уровне ещё не надо спорить о мотивах автора. Здесь важны точность чтения, правильный перевод адресов и отсутствие фантазии.

Interpretation - интерпретация. Она появляется, когда observation помещается в модель Windows. Если UnwindInfoAddress указывает за пределы image, то RtlLookupFunctionEntry может найти запись RUNTIME_FUNCTION, но переход к unwind info уже не даст корректной инструкции для восстановления кадра. Это не просто «число большое». Это нарушение связи .pdata -> .xdata. Если metadata лежит в writable section, это не автоматически вредоносность, но это означает, что данные, от которых зависит unwind, потенциально могут быть изменены после загрузки. Интерпретация отвечает не на вопрос «какое число прочитано», а на вопрос «что это число ломает или ослабляет в модели».

Conclusion - вывод. Он появляется ещё позже. Вывод может быть осторожным: «требуется ручная проверка», «статическое evidence слабое», «вероятна нестандартная упаковка», «динамическая проверка подтвердила handler route», «сильного case-а нет». Хороший conclusion не перепрыгивает через ступени. Если есть только один writable unwind record в маленьком sample-е, честный вывод не должен звучать как обвинение. Если же writable metadata сочетается с handler dominance, runtime handler hits и переходом RIP в private executable memory, вывод становится гораздо сильнее, потому что разные слои подтверждают одну историю.

Здесь появляется важная дисциплина: finding не должен прятать ход мысли. Пользователь отчёта должен понимать, что является фактом, что является объяснением факта, а что является итоговой оценкой. Когда эти уровни смешаны, инструмент начинает казаться умным, но на самом деле становится опасным. Он перестаёт помогать реверсеру думать и начинает подсовывать готовые ярлыки.

Finding как запись о маршруте от байтов к смыслу

В проекте finding имеет поля вроде code, family, severity, message, affected_count, representative_rva, representative_section. Сами по себе эти поля выглядят формально, почти бюрократически. Но за ними стоит правильная идея: каждый finding должен сохранять маршрут от байтов к смыслу.

code - это стабильное имя правила. Оно нужно не человеку в первую очередь, а системе: тестам, JSON schema, regression-сравнениям, фильтрации, документации. Например, rf_overlaps или unwind_writable_section. Такое имя должно быть стабильным, потому что вокруг него строятся проверки и отчёты. Но code не должен быть единственным объяснением. Если человек видит

только rf_overlaps, он знает название симптома, но ещё не понимает болезнь.

family группирует finding по слою evidence. Это очень важно. Если повреждена сама таблица RUNTIME_FUNCTION, мы находимся в семействе table_integrity. Если не читается UNWIND_INFO, это уже unwind_integrity. Если metadata лежит в writable или executable секции, это metadata_placement. Если слишком много функций используют один UnwindInfoRVA, это unwind_reuse. Если слишком много функций сходятся к одному handler target, это handler_dominance. Семейство помогает не смешивать разные природы сигналов. Десять однотипных признаков из одного семейства не равны трём независимым признакам из разных семейств.

severity говорит о тяжести возможного последствия. Это не то же самое, что уверенность. Если handler target лежит в non-executable section, последствие потенциально серьёзное: Windows может попытаться передать управление туда, где выполнение невозможно или где адрес вообще не соответствует нормальному коду. Severity высокая, потому что эффект для механизма опасный. Но confidence может быть ниже, если sample маленький, если запись не достигается динамически, если есть packer stub, если нет подтверждения от dynamic-а. Наоборот, finding с умеренной severity может иметь высокую confidence: например, таблица действительно unsorted, это легко доказано, но само по себе может не означать сильную угрозу.

affected_count показывает масштаб. Один испорченный record и тысяча испорченных records - разные истории. Но масштаб тоже не надо читать механически. Один испорченный record может быть важнее сотни слабых, если он относится к функции, через которую реально проходит exception route. Поэтому количество помогает, но не заменяет анализ. Хороший static показывает count, но не превращает count в автоматическую истину.

representative_rva нужен, чтобы finding был проверяемым. Если отчёт говорит «есть проблема», но не даёт адреса, это не расследование, а слух. Представительный RVA позволяет открыть место в PE viewer, Ghidra, x64dbg или собственном static UI. Он не обязан быть единственным affected location. Он нужен как входная точка для человека: начни отсюда, здесь видно симптом.

representative_section связывает finding с картой секций. Это кажется мелочью, но для unwind-анализа это часто решает смысл. Один и тот же RVA в .rdata, .xdata, .text, .data или unknown range даёт разные интерпретации. Если UNWIND_INFO находится в read-only data, это ближе к обычной модели. Если он находится в executable section, появляется вопрос: это смешение кода и metadata, packer layout, generated section или попытка спрятать данные там, где реверсер ищет инструкции? Если он writable, появляется ещё другой вопрос: может ли runtime менять unwind-карту после загрузки?

Адреса как часть доказательства, а не техническая мелочь

Одна из причин, почему finding может обманывать, - путаница адресных пространств. В PE-анализе есть file offset, RVA и VA. Эти слова уже появлялись раньше, но в evidence model они становятся не просто терминами, а частью доказательной дисциплины.

File offset - положение байта в файле на диске. Это координата для чтения fread, для hex editor-а, для проверки границ raw data. RVA - relative virtual address, адрес относительно image base после загрузки образа. Это язык PE directory, section virtual ranges, .pdata, .xdata. VA - virtual address в процессе, то есть ImageBase + RVA для загруженного image, если мы говорим о модульной памяти. Когда dynamic видит RIP, он видит VA. Когда static читает RUNTIME_FUNCTION, он видит RVA. Когда парсер читает байты из файла, он должен перевести RVA в file offset через section map.

Если анализатор смешивает эти пространства, evidence разваливается. Он может сказать, что UnwindInfoAddress указывает в .xdata, хотя на самом деле сравнил RVA с raw offset. Он может пометить handler как outside image, хотя применил ImageBase дважды. Он может поставить breakpoint не туда, потому что взял RVA как VA. Поэтому хороший finding должен неявно или явно сохранять координатную цепочку. Не обязательно каждый раз печатать всю арифметику на экран, но внутри модели она должна быть железной.

В повествовательной форме это выглядит так: static нашёл RUNTIME_FUNCTION с BeginAddress = 0x12340, EndAddress = 0x12410, UnwindInfoAddress = 0x21000. Эти три значения являются image-relative. Чтобы прочитать UNWIND_INFO, он не прыгает в файл по offset 0x21000. Он сначала ищет секцию, чей virtual range покрывает RVA 0x21000, затем переводит этот RVA в raw offset. Если секция корректна и raw range находится внутри файла, только тогда он читает байты. Если на любом этапе цепочка рвётся, finding должен говорить не просто «bad unwind», а объяснять, где именно оборвалась связь: в PE layout, в Exception Directory, в section mapping или в самой структуре UNWIND_INFO.

Эта точность нужна не для красоты. Она нужна потому, что static и dynamic живут в разных координатах. static начинает с файла. dynamic начинает с процесса. Чтобы они сошлись, они должны говорить на общем языке: module base, RVA, VA, memory region, handler address. Когда статический finding говорит «handler RVA находится в writable section», dynamic должен уметь превратить этот RVA в runtime VA и проверить, был ли этот handler hit. Если адресная цепочка нечёткая, динамическая валидация превращается в гадание.

Severity не равна confidence

Очень частая ошибка новичка - думать, что severity и confidence почти одно и то же. В бытовом языке это понятно: если проблема «серьёзная», значит мы как будто «уверены», что всё плохо. Но в анализе бинарей эти оси независимы.

Severity отвечает на вопрос о потенциальном ущербе или важности последствия. Если finding правдив, насколько он опасен для модели? Например, rf_overlaps может быть тяжёлым, потому что две записи RUNTIME_FUNCTION претендуют на пересекающиеся диапазоны кода. Для поиска function entry по ControlPc это создаёт неоднозначность: какая запись описывает функцию? Какая .xdata должна использоваться? Какой handler участвует в search phase? Даже если Windows выбирает одну запись по конкретному алгоритму, для анализатора такая карта уже не является чистой и простой. Severity высокая не потому, что мы доказали злой умысел, а потому, что нарушение бьёт в фундаментальную связь RIP -> RUNTIME_FUNCTION -> UNWIND_INFO.

Confidence отвечает на другой вопрос: насколько мы уверены в нашей интерпретации. Если overlap найден арифметически между двумя валидными записями внутри нормального Exception Directory, confidence по факту overlap может быть высокой. Но confidence по выводу «это обфускация» может быть ниже, если нет других признаков. Если overlap сочетается с executable metadata, handler dominance и runtime route через подозрительный handler, confidence итогового case-а растёт.

Иногда бывает высокая severity и низкая confidence. Например, handler RVA указывает в странную область, но сам файл упакован, статическая картина явно неполная, а dynamic не смог завершить сессию из-за timeout-а. Последствие потенциально серьёзное, но вывод надо держать осторожным. Иногда бывает низкая severity и высокая confidence. Например, .pdata слегка unsorted в одном месте, это точно видно, но не приводит к сильному подозрению без других признаков. Такой finding полезен как качество карты, но не как громкий verdict.

Поэтому в книге и в проекте нельзя писать findings так, будто severity автоматически поднимает confidence. Хороший отчёт должен уметь сказать: «это потенциально важно, но пока не доказано» или «это точно есть, но само по себе не страшно». Такая фраза может казаться менее эффектной, зато она инженерно честная.

Score как инженерная сумма, а не процент вредоносности

В проекте есть score и suspicion. Это удобные агрегаты, но их легко неправильно понять. Если отчёт показывает score = 70, это не означает «70% malware». Если confidence = 0.8, это не означает, что статистически восемь из десяти таких файлов вредоносны. В текущей инженерной модели score - это

сумма весов, заданных правилами. Suspicion - нормализация этой суммы. Confidence - отдельная оценка полноты и устойчивости evidence.

Почему это важно? Потому что reverse engineering не должен притворяться машинным обучением там, где его нет. Веса вроде +25 за сильный table-integrity signal или +10 за умеренный reuse - это не закон природы и не требование Microsoft. Это инженерная договорённость проекта: такие признаки считаются достаточно важными, чтобы влиять на итоговую оценку. Они нужны, чтобы отчёты были стабильными, сравнимыми и reviewable. Но они не дают права забыть про контекст.

Представим два файла с одинаковым score. Первый получил score из-за нескольких независимых признаков: повреждённые ranges, writable unwind metadata, handler dominance и runtime handler hit. Второй получил похожий score из-за большого количества однотипных parse failures в маленьком synthetic sample-е. Число может быть похожим, но история разная. Поэтому report должен показывать не только итоговый score, но и families, counts, weakening rules, assessment, manual validation steps. Человек должен видеть, из чего score собрался.

В этом смысле score похож на температуру у пациента. Высокая температура говорит, что стоит обратить внимание, но не говорит сама по себе, чем человек болен. Нужны симптомы, история, анализы, контекст. Finding families и evidence layers играют роль этих дополнительных анализов. Они не отменяют score, а не дают score стать единственным источником смысла.

Evidence families как способ не считать один и тот же симптом десять раз

Семейство evidence - это не просто категория для красивой группировки. Это способ не обмануть себя множеством похожих сигналов. Если один и тот же underlying defect порождает десять сообщений, анализатор не должен думать, что у него десять независимых доказательств.

Например, если Exception Directory указывает в испорченную область, downstream-парсер может увидеть массу проблем: invalid ranges, unwind RVA out of bounds, parse failures. Формально findings много. Но их причина может быть одна: сломанная или намеренно искажённая таблица. Если модель просто сложит всё как независимые факты, она переоценит case. Поэтому families помогают понять, насколько evidence разнообразно.

table_integrity говорит о целостности карты функций и секционного layout-а. Это уровень, где мы спрашиваем: можно ли вообще доверять таблице диапазонов? unwind_integrity говорит о качестве самих unwind records: читаются ли они, согласованы ли поля, не ломается ли структура UNWIND_INFO. metadata_placement смотрит на то, где лежат эти данные: в обычных read-only областях или в местах, где они могут исполняться или

изменяться. unwind_reuse замечает концентрацию одного UnwindInfoRVA. handler_dominance замечает концентрацию одного handler target.

Эти families соответствуют разным способам сломать или спрятать unwind-модель. Можно повредить таблицу диапазонов. Можно оставить таблицу красивой, но дать ей битые .xdata. Можно положить metadata туда, где она выглядит как код или может быть изменена. Можно сделать множество функций, которые внешне разные, но ведут к одному unwind или одному handler. Каждое семейство рассказывает свою историю. Когда несколько историй сходятся, case становится сильнее.

Для static -a family count - это способ оценить ширину статического evidence. Для dynamic -a это список гипотез, которые стоит проверять динамически. Если подозрение в metadata_placement, dynamic смотрит, появляется ли runtime activity вокруг dynamic function tables или handler hits. Если подозрение в handler_dominance, dynamic особенно внимательно относится к тому, срабатывает ли этот handler и куда уходит RIP после него.

Weakening rules как признак честного анализатора

На первый взгляд weakening rules могут показаться странными. Зачем анализатор сам ослабляет свои выводы? Разве его задача не находить подозрительное? На самом деле именно наличие weakening rules показывает, что инструмент пытается быть честным.

Weakening rule - это не оправдание файла и не отмена observation. Если unwind_writable_section найден, этот факт остаётся. Если rf_unsorted найден, он не исчезает. Weakening rule говорит о другом: текущий набор evidence не даёт достаточно оснований для сильной интерпретации. Например, есть ровно один слабый signal. Или sample слишком маленький, чтобы делать широкие выводы. Или вся картина построена только на placement anomaly без повреждений ranges, parse failures или runtime confirmation. Тогда confidence итогового вывода снижается.

Это особенно важно в reverse engineering, потому что нормальные бинарники часто не похожи на учебные примеры. Компиляторы, линкеры, runtime-библиотеки, dynamic s, packers, instrumentation tools, hotpatching, JIT - всё это создаёт реальные варианты layout-а. Если анализатор не умеет ослаблять выводы, он будет постоянно пугать человека. А человек быстро перестанет ему доверять.

Здесь полезно запомнить принцип: weakening rule не спорит с фактом, она спорит с поспешностью вывода. Она говорит: «да, факт есть; нет, одного этого факта пока недостаточно для сильного case-а». Это очень зрелая логика. В будущем, когда проект будет развиваться, weakening rules можно делать более тонкими: учитывать compiler family, known runtime patterns, section naming conventions, dynamic confirmation, corpus statistics. Но даже простая модель уже

лучше, чем плоское суммирование тревог.

Static evidence и runtime evidence как две половины одной истории

static и dynamic должны дополнять друг друга, а не заменять. static видит структуру, заявленную файлом. dynamic видит события в процессе. Статика говорит: «вот карта, которую я смог построить из PE». Динамика говорит: «вот маршрут, который реально прошёл поток при запуске». Evidence model нужна именно для того, чтобы эти две истории можно было свести.

Представим, что static нашёл handler dominance: множество функций ведут к одному handler target. Статически это может означать разные вещи. Это может быть обычная runtime-структура, centralized exception handling, compiler artifact, обфускация, packer dispatcher, anti-debug path. Сам факт концентрации ещё не раскрывает намерение. dynamic может поставить breakpoint на этот handler target, запустить target под Debug API, наблюдать exception events, поймать handler hit и записать post-handler RIP. Если handler реально срабатывает и после него управление уходит не туда, куда ожидал обычный crash path, static finding получает runtime reinforcement.

Но обратная ситуация тоже важна. Если dynamic не увидел handler hit, это не обязательно опровержение. Возможно, execution path не дошёл до нужной ветки. Возможно, exception route требует input. Возможно, target завершился раньше. Возможно, сессия incomplete. Поэтому runtime evidence не должно использоваться грубо. Отсутствие события - не всегда доказательство отсутствия механизма. Хороший report должен различать «не подтвердилось» и «было проверено достаточно, чтобы считать маловероятным». Это разные фразы.

Когда статическое и динамическое evidence сходятся, появляется correlation. Correlation - это не магическое слово «доказано всё». Это означает, что разные источники наблюдений начали рассказывать одну согласованную историю. Статика показала подозрительный handler. Динамика показала handler hit. Статика показала writable metadata. Динамика показала dynamic function-table activity. Статика показала code outside normal module mapping. Динамика показала RIP в private executable memory. Чем лучше совпадает история, тем выше confidence.

Нормальный finding учит читать бинарь

Хороший finding должен быть полезен даже тогда, когда итоговый verdict окажется мягким. Он должен учить реверсера смотреть на бинарь. Это особенно важно для твоей книги и проекта: цель не просто сделать инструмент, который печатает JSON, а построить систему, которая помогает понять Windows x64 exception/unwind-механику.

Плохой finding выглядит так:

```
WARNING: rf_overlaps
```

Он короткий, но почти бесполезный. Пользователь не знает, какие записи пересекаются, где они находятся, почему overlap важен, может ли это быть нормой, что делать дальше.

Лучший finding выглядит иначе:

```
code: rf_overlaps
family: table_integrity
severity: high
affected_count: 2
representative_rva: 0x00013240
message: Two RUNTIME_FUNCTION ranges overlap. This makes the PC-to-function mapping ambiguous for the
```

Даже эта запись ещё машинная, но она уже учит. Она показывает location. Называет слой. Объясняет consequence. Даёт следующий шаг. Не кричит «malware». Не скрывает, что речь идёт о mapping ambiguity. Если такой finding выводится в HTML/JSON/Ghidra comments, реверсер получает не просто предупреждение, а направление мысли.

В идеале рядом с machine-readable finding должен существовать human explanation. JSON нужен для интеграции. HTML нужен для чтения. Ghidra CSV нужен для переноса адресов в рабочую среду. Console output нужен для быстрого просмотра. Но смысл должен сохраняться везде: finding - это не лампочка, а сжатое расследование.

Пример: пересечение диапазонов в `.pdata`

Рассмотрим один пример медленно, потому что именно на примерах видно отличие evidence model от чек-листа. Пусть static читает Exception Directory и получает массив RUNTIME_FUNCTION. Каждая запись задаёт полуинтервал [BeginAddress, EndAddress). Это означает: если ControlPc, то есть адрес выполнения внутри функции, попадает в этот диапазон, данная запись может описывать unwind-модель для этого участка кода.

Теперь представим, что две записи пересекаются. Первая покрывает [0x1200, 0x1300), вторая покрывает [0x1280, 0x1400). Адрес 0x12A0 попадает в обе. На уровне observation всё просто: ranges overlap. На уровне interpretation появляется проблема: карта PC-to-function перестаёт быть однозначной. Если мы хотим по RIP найти RUNTIME_FUNCTION, какая запись должна победить? Какая .xdata описывает кадр? Какой handler data относится к exception route? Даже если конкретная реализация поиска выберет одну запись, для реверсера такая таблица уже требует внимания, потому что она нарушает ожидаемую картину непересекающихся диапазонов.

На уровне conclusion static не обязан сразу говорить «обфускация». Он должен сказать: это сильный table-integrity signal, потому что он касается

базовой карты функций. Далее он смотрит контекст. Если overlap один, sample маленький, dynamic route не затрагивает эти адреса, итог может быть осторожным. Если overlaps много, они находятся в executable-packed area, связаны с reused handler-ami, a dynamic видит переходы через exception route, case усиливается.

Такой разбор учит важной вещи: finding - это не сам факт overlap. Finding - это связка overlap с моделью Windows и с последствиями для анализа. Без этой связки finding превращается в галочку.

Пример: writable unwind metadata

Другой пример - unwind_writable_section. На поверхности всё просто: UNWIND_INFO лежит в секции, у которой есть право записи. Но почему это вообще интересно? Ведь CPU не запрещает читать данные из writable memory. Windows тоже может прочитать UNWIND_INFO из такой области, если адрес валиден. Значит, само по себе writable не делает запись невозможной.

Смысл в другом. Unwind metadata - это не обычные данные программы, которые она свободно меняет в ходе вычислений. Это часть договора между кодом и системой. Когда возникает исключение, когда отладчик строит stack trace, когда RtlVirtualUnwind восстанавливает caller context, Windows опирается на эти байты. Если эти байты можно менять во время выполнения, карта функций перестаёт быть стабильной. То, что static увидел на диске или в начале анализа, может не совпасть с тем, что система прочитает позже.

Это не означает, что writable metadata всегда злонамеренна. Возможны нестандартные loader-ы, generated tables, упаковщики, странные toolchain decisions. Но это означает, что finding должен поднять вопрос о стабильности metadata. dynamic здесь особенно важен. Он может проверить, были ли вызовы dynamic function table API, менялись ли protections, попадал ли RIP в области, где static PE не даёт полной unwind-карты, срабатывали ли handler-ы, которые static считал статически подозрительными.

Хороший conclusion здесь может звучать так: «обнаружена unwind metadata в writable section; это ослабляет доверие к статической модели, потому что runtime может изменить bytes, влияющие на unwind; требуется динамическая проверка или memory snapshot». Это совсем другая подача, чем «writable section = bad».

Пример: parse failure как разрыв договора

unwind_parse_fail часто хочется воспринимать как простую ошибку парсера. Но в хорошей модели это сигнал о разрыве договора. RUNTIME_FUNCTION обещает: по этому UnwindInfoAddress лежит структура, которую Windows и инструменты могут прочитать как UNWIND_INFO. Если парсер не может корректно разобрать эту структуру, надо понять, почему.

Причина может быть банальной: RVA указывает за пределы файла, section mapping сломан, файл обрезан. Может быть структурной: CountOfCodes требует больше slots, чем доступно до конца image. Может быть семантической: flags говорят, что после unwind codes должен быть handler RVA, но хвост не помещается. Может быть защитной: metadata намеренно искажена для статического инструмента, а runtime восстанавливает или подменяет её позже. Может быть результатом несовместимости с более новой схемой unwind information, если анализатор не поддерживает нужный формат.

Поэтому finding не должен говорить только «parse failed». Он должен сохранить место разрыва: на каком RVA, в какой секции, после какого поля, при каком ожидании. Если не хватает байтов после CountOfCodes, это один смысл. Если неизвестный opcode в UNWIND_CODE, это другой смысл. Если handler data выходит за image, это третий смысл. Для человека эти различия принципиальны, потому что они ведут к разным действиям.

В связке static /dynamic такой finding становится особенно интересным. static говорит: declared unwind model не читается статически. dynamic проверяет, что происходит, когда поток реально попадает в соответствующий code range и возникает exception или stack walk. Если процесс нормально unwound через dynamic function table, значит статическая PE-картина была неполной. Если exception route ломается или приводит к second-chance crash, это уже другая история. Если code range вообще не достигается, finding остаётся статическим и требует ручного анализа.

Manual review как нормальная часть процесса

Некоторые люди воспринимают requires_manual_review как слабость автоматического анализатора. На самом деле это честная граница. Reverse engineering слишком контекстен, чтобы хороший инструмент всегда выдавал окончательный verdict. Если finding затрагивает unwind metadata, handler placement, dynamic code или packer-like behavior, человек часто должен посмотреть на место сам.

Manual review не означает «инструмент не справился». Оно означает: инструмент сделал свою часть работы и подготовил маршрут. Он нашёл адреса, перевёл координаты, сгруппировал evidence, отделил observation от interpretation, показал severity/confidence, предложил dynamic checks. Теперь реверсер может открыть Ghidra, перейти к representative RVA, посмотреть section context, сравнить prolog и UNWIND_CODE, проверить handler target, запустить dynamic , собрать RIP trace.

В хорошем workflow static экономит человеку часы. Он не заменяет мышление, но убирает хаос. Вместо «где-то в PE что-то странное» человек получает: «вот диапазон, вот metadata, вот нарушение, вот почему оно важно, вот что проверить дальше». Именно так автоматический анализ становится

преподавателем, а не просто шумным сканером.

Finding как мост к Ghidra, JSON, HTML и dynamic

Один и тот же finding живёт в нескольких формах. В JSON он должен быть строгим и schema-compatible. В HTML он должен быть читаемым. В Ghidra он должен стать comment-ом, bookmark-ом или таблицей адресов. В dynamic-е он должен превратиться в dynamic question: поставить ли breakpoint, наблюдать ли handler, искать ли dynamic function table activity, записывать ли post-handler RIP.

Из-за этого finding должен быть одновременно машинным и человеческим. Машинная часть требует стабильных кодов, enum-значений, счётчиков, адресов. Человеческая часть требует объяснения. Если оставить только человеческий текст, интеграция развалится. Если оставить только enum-ы, человек перестанет понимать отчёт. Сильный проект держит обе стороны.

Например, handler_dominance в JSON нужен как code и family, чтобы assessment layer мог учитывать его в scoring и correlation. В HTML рядом должна быть фраза, объясняющая, что множество функций сходятся к одному handler target, из-за чего exception route может быть централизованным. В Ghidra этот handler target стоит отметить, потому что он становится узлом управления. В dynamic-е на него можно поставить breakpoint и проверить, является ли он реально достигнутым маршрутом или только статическим артефактом.

Так finding становится мостом между слоями проекта. Он начинается как байты в PE, проходит через модель Windows, появляется как запись в отчёте, переносится в инструмент реверсера и заканчивается динамической проверкой маршрута выполнения.

Evidence model как защита от самообмана

Самая большая польза evidence model даже не в отчётах, а в дисциплине мышления. Она постоянно напоминает: не путай наличие признака с доказательством намерения; не путай тяжесть последствия с уверенностью; не путай отсутствие runtime event с отсутствием механизма; не путай file view с memory view; не путай RVA с file offset; не путай static map с runtime route.

В reverse engineering очень легко захотеть быстрый ярлык. Видишь int 3 - anti-debug. Видишь writable metadata - malware. Видишь RaiseException - control-flow trick. Видишь missing .pdata - сломано. Но вся предыдущая книга была нужна именно для того, чтобы не мыслить такими ярлыками. int 3 может быть отладочным остатком, deliberate exception, compiler artifact или частью протокола. Missing .pdata может быть нормальной leaf-функцией. Writable metadata может быть нестандартным runtime layout-ом. Handler dominance может быть централизованной обработкой ошибок. А может быть и защитным

маршрутизатором. Разница появляется только в контексте.

Evidence model заставляет этот контекст собирать. Она говорит: сначала покажи observation. Потом объясни его в модели Windows. Потом проверь альтернативные объяснения. Потом посмотри другие evidence families. Потом, если возможно, попроси dynamic проверить живой маршрут. И только потом формулируй conclusion.

Это и есть зрелая позиция анализатора. Он не пытается выглядеть всезнающим. Он показывает, на чём стоит его вывод.

Роль dynamic -а в усилении или ограничении вывода

dynamic в нашем разговоре - это динамическая валидация. Он показывает, куда реально уходит функция, как проходит exception handling, какой handler срабатывает, какой RIP наблюдается после обработки. Поэтому его роль в evidence model не декоративная. Он не просто добавляет красивый runtime block в JSON. Он проверяет самый важный вопрос: совпадает ли статическая карта с живым маршрутом.

Если static нашёл suspicious handler target, dynamic может подтвердить, что target действительно был hit. Если static подозревает exception-based control flow, dynamic может увидеть first-chance exception, решение debugger-а через ContinueDebugEvent, вход в handler и продолжение исполнения по изменённому context. Если static говорит, что static PE не описывает runtime code, dynamic может увидеть RIP в memory region, не соответствующем обычному module image, или регистрацию dynamic function table. Если static видит writable metadata, dynamic может помочь понять, меняется ли unwind route во время выполнения.

Но dynamic тоже не всесилен. Один запуск не покрывает все ветки. Input может не активировать нужный маршрут. Anti-debug logic может изменить поведение под debugger-ом. Timeout может обрезать наблюдение. Некоторые handler-ы срабатывают только при ошибках, которые трудно воспроизвести. Поэтому runtime evidence тоже имеет confidence и limitations. Хороший assessment должен уметь сказать: runtime signal strong, medium, weak или none; session complete или incomplete; correlation present или absent; absence of hit is not proof of absence.

Эта осторожность не делает dynamic слабым. Она делает его инженерным. Динамика не отменяет статику, а статика не заменяет динамику. Вместе они дают более честную картину.

Итоговая модель case strength

Когда findings, families, score, confidence, weakening rules и runtime signals собираются вместе, проект формирует assessment. Здесь появляется

case_strength: no_case, weak_case, medium_case, strong_unconfirmed_case, medium_correlated_case, strong_correlated_case. Эти названия важны, потому что они отделяют силу evidence от окончательного обвинения.

strong_unconfirmed_case означает, что статически или динамически есть сильные признаки, но между слоями ещё нет подтверждённой связи. Это честный статус: инструмент говорит, что основание серьёзное, но cross-layer proof пока не построен. strong_correlated_case сильнее, потому что static и runtime начали подтверждать друг друга. weak_case означает, что признаки есть, но они слабые или изолированные. no_case означает не «файл гарантированно нормальный», а «в рамках доступного анализа meaningful evidence не найдено».

Это различие тонкое, но принципиальное. Анализатор не должен обещать больше, чем видел. Если он не нашёл подозрений, он не доказал безопасность файла. Если он нашёл strong static evidence, он не обязательно доказал злой умысел. Если dynamic подтвердил route, он подтвердил route, а не всю семантику программы. Хороший report говорит именно то, что может доказать.

Такой подход делает проект устойчивым. Он не пытается быть антивирусом, не выдаёт фантазий о намерениях автора, не заменяет реверсера. Он делает то, что должен делать EH/Unwind Analyzer: строит доказательную карту Windows x64 exception/unwind-механики и показывает, где эта карта нормальна, где неполна, где повреждена, где подозрительна и где требует динамической проверки.

Нормализация фактов как скрытая основа честного вывода

Прежде чем finding станет finding-ом, данные должны пройти нормализацию. Это слово звучит сухо, но смысл у него очень человеческий: привести разрозненные наблюдения к такому виду, в котором их можно сравнивать и обсуждать без путаницы. Пока мы читаем PE-файл как поток байтов, у нас нет ещё «функций», «handler-ов» и «аномалий». Есть только байты по file offsets. Потом parser строит PeImage, где появляются секции, размеры, флаги, границы image. Потом unwind decoder строит массив RuntimeFunction и summaries для UNWIND_INFO. Только после этого heuristics layer имеет право говорить о признаках.

Если пропустить этот этап в объяснении, у читателя возникает ложное ощущение, будто finding рождается мгновенно: увидели .pdata - нашли подозрение. Но это не так. Между файлом и finding-ом стоит цепочка преобразований. Каждое преобразование может ошибиться, и каждая ошибка ниже по цепочке отравит вывод выше. Если section map построена неверно, то UnwindInfoAddress может быть переведён в неправильный file offset. Если

UNWIND_INFO прочитан с неверным padding, handler RVA окажется сдвинутым. Если handler RVA неверен, handler dominance станет фантомом. Поэтому сильный анализатор сначала заботится не о verdict-е, а о качестве нормализации.

Эта мысль важна для static -а. static не должен быть набором эвристик, прикрученных прямо к байтам. Он должен быть конвейером смыслов. На первом этапе он осторожно читает файл. На втором строит карту image. На третьем строит function range map. На четвёртом строит declared unwind model. На пятом сравнивает модели и формирует evidence. Чем дальше мы идём, тем больше смысла появляется, но тем сильнее зависимость от предыдущих этапов. Поэтому finding должен помнить, откуда он пришёл.

Представь, что в отчёте появился unwind_parse_fail. Неправильный анализатор сразу скажет: «плохая .xdata». Более зрелый static сначала спросит себя: действительно ли проблема в .xdata? Может быть, Exception Directory указывает не туда? Может быть, section raw range выходит за файл? Может быть, RVA-to-offset mapping уже был сомнительным? Может быть, CountOfCodes указывает на хвост, который попадает в конец секции? Пока эти вопросы не пройдены, finding не должен притворяться более точным, чем он есть.

Отсюда появляется ещё один принцип: чем ниже слой, на котором найдено повреждение, тем осторожнее нужно читать все последующие выводы. Если PE section layout сломан, многие unwind findings могут быть следствием этой поломки, а не независимыми признаками. Это не значит, что их надо скрывать. Наоборот, их надо показывать, но связывать с первопричиной. Хороший отчёт должен говорить: «у нас есть проблема section mapping; из-за неё unwind parsing в таких-то местах ненадёжен». Это лучше, чем вывести десять ярких warnings и создать иллюзию десяти независимых подозрений.

HeuristicReport как память о пройденном пути

В проекте HeuristicReport - это не просто структура для красивого JSON. Это место, где анализатор сохраняет итог своего рассуждения. Он хранит score, suspicion, confidence, evidence families, findings, weakening rules, counts и details. Если смотреть на это как на поля, получится опять сухой конспект. Но если смотреть как на память о пути, структура становится понятной.

total говорит, сколько function records вообще анализировалось. Это важно, потому что один finding на трёх функциях и один finding на десяти тысячах функций читаются по-разному. В маленьком sample-е даже один странный record может сильно искажать ratio. В большом sample-е один странный record может быть локальным повреждением или хвостом нестандартного кода. Поэтому counts без denominator-а опасны. affected_count всегда надо читать рядом с общим масштабом.

Counters вроде rf_invalid_ranges, rf_oob, rf_unwind_oob, rf_unsorted, rf_overlaps сохраняют не один итоговый verdict, а распределение симптомов. Это как медицинский анализ крови: врач не смотрит только на одну итоговую строку «плохо/хорошо», он смотрит, какие показатели вышли за норму и как они связаны. В unwind-анализе так же. Если все симптомы находятся в table integrity, история одна. Если есть table integrity, unwind integrity и metadata placement, история другая. Если статические симптомы слабые, но runtime evidence сильное, история третья.

details нужен для того, чтобы не потерять объяснительный слой. Иногда finding слишком краток, потому что должен быть структурным. Details позволяет сохранить более длинное описание, ratios, дополнительные notes, mitigation explanations. В хорошей книге и хорошем инструменте details - не мусорное поле. Это место, где анализатор показывает, как он думал.

Но есть и опасность: report не должен превращаться в склад всего подряд. Если в него свалить тысячи сырых чисел, человек перестанет видеть смысл. Поэтому report должен быть не только полным, но и организованным. Сначала summary, потом findings, потом details, потом manual validation steps. Это похоже на хорошую техническую главу: сначала дать читателю карту, потом пройти путь, потом показать детали.

Message как человеческое объяснение, а не декоративный текст

Поле message часто недооценивают. Кажется, что главное - code, а message можно сделать коротким. Но для учебного инструмента message - это важнейшая часть. Именно message превращает finding из машинного события в человеческое знание.

Плохой message описывает только симптом: «runtime functions overlap». Он не врёт, но и почти не помогает. Человек должен сам вспомнить, почему overlap важен, что такое полуинтервал, как Windows ищет function entry, как это связано с ControlPc, .xdata, handler data. Если человек уже эксперт, он догадается. Если он учится, он получит очередную непонятную красную строку.

Хороший message объясняет consequence. Он говорит не только «ranges overlap», но и «PC-to-function mapping becomes ambiguous for the affected address interval». Ещё лучше, если он добавляет следующий шаг: «inspect both RUNTIME_FUNCTION records and their UNWIND_INFO, then validate runtime route if the code is reachable». Тогда finding начинает вести человека. Он не просто сообщает о проблеме, он показывает дорогу.

В книге надо держать тот же принцип. Когда мы пишем про UNW_FLAG_EHANDLER, недостаточно сказать: «флаг означает наличие exception handler». Нужно объяснить: этот флаг меняет смысл хвоста

UNWIND_INFO; после массива UNWIND_CODE и padding-а появляются дополнительные данные; handler RVA часто указывает на language-specific handler, например __C_specific_handler; реальный пользовательский __except может быть найден через scope table; значит, реверсер не должен принимать первый handler RVA за финальный пользовательский блок. Такая подача длиннее, но она учит.

Для проекта это означает, что финальные отчёты должны стремиться к объяснительному message. Не обязательно превращать каждую строку JSON в абзац учебника, но HTML-отчёт и документация findings должны быть именно такими. В идеале пользователь видит короткую машинную строку и рядом человеческое пояснение: почему это важно, как проверить, какие альтернативы возможны.

Representative location как точка входа в расследование

Адрес в finding-е - это не просто украшение. Это дверь, через которую человек входит в расследование. Если finding говорит об overlap, representative RVA должен привести к одному из пересекающихся диапазонов. Если finding говорит о writable metadata, representative RVA должен вести к конкретному UNWIND_INFO. Если речь о handler dominance, representative RVA должен вести к handler target или к одной из функций, которая на него ссылается. Без адреса finding остаётся абстрактным.

Но representative location надо понимать правильно. Он не обязан представлять всю проблему полностью. Если affected_count равно 500, один RVA не покажет все 500 случаев. Он нужен как первая зацепка. Поэтому сильный report может иметь несколько уровней: summary finding с representative RVA, а в details - список top offenders, ratios, examples, maybe CSV для полного списка. В Ghidra можно импортировать полный набор, но в console output показать только representative location.

Здесь снова важна связь с static/dynamic. static использует representative RVA как статическую точку. dynamic может превратить её в runtime VA, если соответствующий модуль загружен, и поставить breakpoint или наблюдать memory region. Если finding не хранит адресную цепочку, dynamic validation становится сложнее. Поэтому representative RVA - это не просто удобство для человека, а интерфейс между статическим и динамическим слоем.

В учебном смысле representative location помогает читателю перестать думать о finding-ах как о магических ярлыках. Он открывает адрес и видит реальные байты, реальные инструкции, реальную секцию. Так абстрактное «metadata placement» превращается в конкретное место: вот section header, вот Characteristics, вот RVA, вот bytes UNWIND_INFO, вот handler RVA. Эта конкретность и делает обучение настоящим.

Evidence level как язык осторожности

В отчёте есть levels: none, weak, medium, strong. Их нельзя читать как моральные оценки файла. Это язык силы evidence. none значит, что в рамках данной модели meaningful evidence не найдено. Это не доказательство безопасности. weak значит, что есть слабый или изолированный сигнал. medium значит, что сигнал уже нельзя игнорировать, но он может требовать подтверждения. strong значит, что evidence серьёзное по правилам модели: например, несколько strong findings или высокий score.

Этот язык нужен, чтобы избежать двух крайностей. Первая крайность - всё называть suspicious. Тогда инструмент бесполезен, потому что пользователь тонет в тревогах. Вторая крайность - выводить только «точно доказанные» случаи. Тогда инструмент пропустит важные ранние сигналы, которые ещё требуют ручной проверки. Evidence level занимает середину: он показывает силу основания, не превращая её в окончательный приговор.

Runtime evidence level работает похожим образом, но с другой природой данных. First-chance exception сам по себе может быть weak runtime activity. Handler hit уже сильнее. Handler hit плюс дополнительный runtime signal - ещё сильнее. Но incomplete session ограничивает вывод. Если target завис, упал, был остановлен по timeout-у или не дошёл до natural exit, runtime evidence надо читать осторожно. Это не потому, что события бесполезны, а потому что контекст неполный.

В хорошей архитектуре static evidence level и runtime evidence level показываются рядом. Тогда человек видит, где сила. Бывает strong static / none runtime: карта выглядит тревожно, но динамика пока ничего не подтвердила. Бывает weak static / medium runtime: PE на диске выглядит почти нормально, но во время выполнения появляются события. Бывает medium static / medium runtime с correlation: это уже гораздо интереснее. Такая матрица гораздо честнее, чем один verdict.

Correlation как совпадение историй, а не механическое сложение

Correlation в проекте сейчас намеренно консервативная. Это правильно. Correlation должна появляться не от того, что у нас есть «что-то статическое» и «что-то динамическое», а от того, что эти два слоя говорят об одном и том же механизме.

Если статически найден handler dominance, а динамически есть handler hits или VEH/VCH/UEF hits, между ними может появиться смысловая связь. Если статически есть placement anomalies, а динамически видны dynamic function table calls, связь тоже возможна. Но если статически есть небольшой rf_unsorted, а динамически просто был unrelated first-chance exception в другом модуле, это не correlation. Это две отдельные новости.

Почему это важно? Потому что механическое сложение создаёт ложную уверенность. В реальном процессе всегда много событий: DLL loads, first-chance exceptions, breakpoints, thread starts. Если любой runtime event усиливает любой static finding, confidence станет искусственно высокой. Хороший dynamic должен не просто собирать события, а связывать их с адресами, модулями, handler targets, ranges и hypotheses, которые подготовил static .

Идеальный future direction - per-function proof graph. То есть граф, где конкретный static finding на конкретной функции связан с конкретным runtime hit, конкретным exception code, конкретным post-handler RIP. Текущая coarse correlation уже полезна, но книга должна честно объяснять её границы. Она говорит: «слои похожи по смыслу», но не всегда доказывает каждую функцию отдельно. Это нормальный инженерный этап развития проекта.

Unknown как честный результат

В инженерной культуре часто недооценивают слово unknown. Кажется, что инструмент должен всегда сказать: good или bad. Но в reverse engineering unknown - один из самых честных и полезных результатов. Он означает: доступных данных недостаточно для сильного вывода; модель видит границу своей компетенции.

Например, static анализирует packed binary. Stub валиден, .pdata stub-а читается, но payload распаковывается в private executable memory. Статический PE на диске не содержит всей карты. Если dynamic не запускался, честный результат: статическая видимость ограничена. Нельзя притворяться, что весь runtime code проанализирован. Можно показать признаки упаковки, показать границы, предложить dynamic validation. Но нельзя говорить, что unwind-модель всего процесса нормальна.

Другой пример: dynamic запущен, но target требует input и не доходит до интересной ветки. Handler, найденный static -ом, не hit-нулся. Честный вывод: handler not observed in this run. Это не то же самое, что handler not reachable. Чтобы сказать unreachable, нужны другие основания. Такое различие кажется мелким, но именно оно отделяет профессиональный анализ от поверхностного.

Unknown не должен быть пустым. Хороший unknown объясняет, что именно неизвестно и что нужно сделать дальше. «Не удалось подтвердить dynamic route, потому что execution path не достиг affected range; повторить с другим input; проверить breakpoint completeness; снять memory snapshot после unpacking». Такой unknown полезнее, чем фальшивый verdict.

Ошибка поспешного обвинения

В контексте security и reverse engineering очень легко перегнуть. Термины вроде packer, writable metadata, handler dominance, breakpoint exception звучат тревожно. Но книга должна воспитывать спокойное мышление. Не каждое странное место - вредоносность. Не каждый packer - malware. Не каждый RaiseException - anti-debug. Не каждый missing unwind record - ошибка. Не каждый handler dominance - dispatcher защиты.

Поспешное обвинение вредно по двум причинам. Во-первых, оно технически неверно. Во-вторых, оно мешает учиться. Когда человек привыкает думать ярлыками, он перестаёт видеть механизм. Он видит PAGE_GUARD и сразу думает «защита». Он не спрашивает, как guard page работает, почему exception first-chance, кто обработал, был ли ContextRecord изменён, куда ушёл RIP, снялся ли guard после первого access. А именно эти вопросы и являются реверсом.

Evidence model должна постоянно возвращать нас к механике. Finding не говорит: «это зло». Finding говорит: «здесь есть наблюдение, которое в модели Windows имеет такие последствия; вот насколько оно сильное; вот что может быть нормальным объяснением; вот что проверить динамически». Это более длинный путь, но он формирует специалиста.

Ошибка чрезмерного оправдания

Есть и обратная ошибка: всё объяснять нормой. Встречается writable metadata - «ну, наверное, компилятор». Handler dominance - «наверное, runtime». Dynamic code - «наверное, JIT». Такие объяснения возможны, но они не должны автоматически гасить evidence. Анализатор должен быть осторожным, но не слепым.

Правильная позиция между паникой и наивностью. Если есть нормальное объяснение, оно снижает confidence, но не стирает факт. Если есть несколько независимых признаков, нормальное объяснение должно становиться конкретнее. Нельзя бесконечно говорить «наверное, это нормально», если static placement, handler dominance и runtime route указывают в одну сторону. Тогда evidence model должна усилить case.

Это особенно важно для dynamic-а. Динамическая валидация помогает отличить «похоже на странную карту» от «эта странная карта реально участвует в исполнении». Если handler dominance только лежит в файле, одно дело. Если handler hit происходит после deliberate exception, а post-handler RIP уходит в новый region, это уже другая история. dynamic добавляет реальность к статической гипотезе.

Как писать findings в стиле книги

Для этой книги важно не только что анализатор делает, но и как мы об этом пишем. Если текст главы сам похож на JSON, читатель устанет. Поэтому дальше

каждую архитектурную тему надо подавать как историю механизма.

Сначала показываем проблему. Например: автоматический анализатор видит тысячи чисел и должен не потерять смысл. Потом строим модель: байты превращаются в секции, секции в ranges, ranges в unwind contracts, contracts в evidence. Потом вводим термин: finding, severity, confidence, family. Потом связываем с Windows: RtlLookupFunctionEntry, RtlVirtualUnwind, SEH dispatch, debugger events. Потом связываем с проектом: static строит статическую карту, dynamic проверяет runtime route. И только потом можно показывать поля отчёта.

Если начать с полей отчёта, получится конспект. Если начать с проблемы и вести читателя, получится книга. Это правило надо сохранить для всех следующих разделов: dynamic pipeline, связка static /dynamic , reports, Ghidra integration, финальная архитектура проекта.

Один finding в полном человеческом разборе

Возьмём finding handler_writable_section и разберём его так, как должен думать реверсер. На уровне данных static видит handler RVA. Этот RVA появляется не из воздуха: он может быть извлечён из хвоста UNWIND_INFO, если flags говорят о handler-е, или из decoded language-specific layer. Затем static переводит RVA в section context. Section header говорит, что область writable. На этом observation заканчивается.

Теперь начинается интерпретация. Handler - это адрес, куда система или language-specific runtime может передать управление при обработке исключения или unwind. Если target лежит в writable section, возникает вопрос о стабильности и намеренности. Код обычно должен быть executable. Данные обычно writable. Если handler target оказывается writable, нужно проверить, является ли эта секция также executable, не является ли это generated code, unpacked region, custom loader artifact, corrupted metadata или ложный decode.

Дальше conclusion. Если handler RVA единичный, section executable, binary явно packed, a dynamic validation не проводилась, conclusion должен быть осторожным: placement unusual, manual review required. Если handler target writable and executable, множество functions ссылается на него, dynamic фиксирует handler hit, a post-handler RIP уходит в нестандартный dispatch loop, confidence растёт. Теперь finding становится частью correlated case.

Такой разбор длиннее одной строки, но он формирует мышление. Он показывает, что finding - это не кнопка «подозрительно», а цепочка: extraction, address translation, section interpretation, Windows consequence, static context, runtime validation.

Второй finding как пример слабого сигнала

Теперь возьмём rf_unsorted. Сортировка .pdata важна, потому что function table entries ожидаются в порядке, удобном для поиска по PC value. Если записи не отсортированы, это нарушает ожидаемую форму таблицы. Но сама по себе локальная inversion не всегда означает сильную проблему. Может быть, файл создан нестандартным инструментом, может быть, sample маленький, может быть, записи всё равно не пересекаются и читаются корректно, может быть, runtime вообще не достигает этих областей.

Поэтому rf_unsorted часто должен быть medium или weak context signal, а не automatic high alarm. Он усиливается, если рядом есть overlaps, invalid ranges, OOB pointers, parse failures. Он ослабляется, если isolated. Это хороший пример того, почему score и confidence должны учитывать не только факт, но и компанию фактов.

В учебном тексте это надо подавать именно так: сортировка важна не потому, что «так красиво», а потому что function table - это структура поиска. Нарушение сортировки может влиять на lookup assumptions и на инструменты анализа. Но без дополнительных признаков оно не доказывает обфускацию. Такой баланс и есть зрелый reverse engineering.

Третий finding как пример сильного сигнала

Теперь рассмотрим pe_exception_directory_oob. Если Exception Directory выходит за image или указывает в невозможную область, это уже не тонкая эвристика про стиль компилятора. Это удар по входной двери unwind-модели. .pdata для x64 PE находится через Data Directory. Если directory сам недействителен, static не может честно построить обычную карту functions из файла.

Severity здесь высокая, потому что последствие фундаментальное. Но даже здесь conclusion должен быть точным. Мы не говорим: «это malware». Мы говорим: «PE image declares an exception directory that cannot be mapped correctly; static unwind analysis from the file is unreliable or impossible for that directory». Если при этом dynamic видит dynamic function table registration, история может быть: файл на диске не содержит нормальную static table, но runtime создаёт свою. Если dynamic ничего не видит, возможно, файл malformed или нецелевой для анализа. Контекст всё равно нужен.

Такой finding показывает, что high severity не означает истеричный язык. Чем сильнее сигнал, тем яснее надо объяснять, что именно он ломает.

Роль тестов и стабильной матрицы evidence

Evidence model должна быть стабильной, иначе инструмент невозможно развивать. Если сегодня rf_overlaps даёт один score, завтра другой, послезавтра меняется severity без причины, regression tests теряют смысл. Поэтому проект фиксирует evidence matrix. Это не значит, что матрица вечная.

Это значит, что изменения должны быть осознанными, документированными и проверяемыми.

Стабильная матрица нужна и для защиты проекта. Когда тебя спрашивают, почему анализатор дал такой verdict, ты можешь показать не субъективное «мне показалось», а rules: какие findings существуют, какие weights, какие weakening rules, какие evidence levels, какие final case rules. Это делает проект reviewable. Даже если кто-то спорит с весом, спор идёт о конкретном правиле, а не о чёрном ящике.

Но стабильность не должна превращаться в догму. Если тестовый корпус покажет, что правило даёт много false positives, его надо пересматривать. Если появятся новые toolchain patterns, weakening rules надо расширять. Если dynamic начнёт давать более точную per-function correlation, final case logic можно усилить. Evidence model - живой инженерный слой, но изменения в нём должны быть дисциплинированными.

Связь с будущей финальной архитектурной главой

Когда в конце книги мы будем описывать весь проект, 9.2 станет мостом к архитектуре. PE parser и unwind decoder дают facts. Heuristic engine превращает facts в findings. Report layer сохраняет findings в JSON/HTML/CSV/Ghidra. Dynamic collector, то есть dynamic, добавляет runtime events и correlation. Assessment block собирает static and runtime evidence в final verdict.

Если читатель поймёт 9.2, он сможет читать архитектуру проекта не как набор модулей, а как систему мышления. util.c нужен не просто для helper-функций, а для безопасного чтения недоверенных байтов. pe.c нужен не просто для headers, а для построения адресной карты. unwind.c нужен не просто для структур, а для declared unwind model. heuristics.c нужен не просто для score, а для evidence reasoning. dynamic_win.c нужен не просто для Debug API, а для проверки runtime route. report.c нужен не просто для вывода, а для передачи доказательной карты человеку.

Так теория связывается с кодом проекта. И именно так книга должна быть написана дальше: не «модуль делает X», а «в системе анализа возникает такая проблема; этот модуль решает её таким способом; вот какие термины нужны; вот какие ошибки понимания возможны; вот как это связано с Windows и с предыдущими главами».

Переход к следующему разделу

Теперь мы можем перейти к dynamic-у осмысленно. Если 9.1 объяснил, как static строит статическую карту, а 9.2 объяснил, как из этой карты рождается evidence, то следующий шаг должен показать, как динамическая валидация проверяет живое поведение. dynamic не должен просто запускать

процесс и собирать события ради событий. Он должен брать вопросы, подготовленные static -ом, и проверять их в реальном execution context.

Главная мысль для следующего раздела такая: статика говорит нам, какой маршрут возможен или заявлен; динамика показывает, какой маршрут действительно был пройден. Между этими двумя слоями и живёт настоящая уверенность.

Техническая база раздела: PE Format и x64 exception handling в документации Microsoft; описание RtlLookupFunctionEntry, RtlVirtualUnwind, debugger exception events и ContinueDebugEvent; фактическая evidence matrix и JSON schema проекта EH/Unwind Analyzer.

9.3. Dynamic validation pipeline: Windows dynamic collector как слой живого свидетельства

После статического анализа легко попасть в очень соблазнительную ловушку. Мы уже умеем открыть PE-файл, прочитать его заголовки, построить карту секций, найти Exception Directory, пройти по таблице RUNTIME_FUNCTION, добраться до UNWIND_INFO, увидеть handler RVA, заметить странное размещение metadata и оформить finding. Кажется, что теперь программа у нас почти в руках: файл прочитан, структура разобрана, подозрительные места названы. Но именно здесь начинается граница, которую хороший reverse engineer обязан чувствовать особенно остро: файл на диске не равен живому процессу.

Файл на диске молчит о том, какой путь реально выберет процесс при запуске. Он не рассказывает, дойдёт ли выполнение до конкретного участка кода. Он не показывает, случится ли исключение, сработает ли обработчик, будет ли зарегистрирован vectored handler, появится ли dynamic function table, изменится ли память после распаковки. Статический анализатор видит заявленную, сохранённую, записанную в байтах сторону программы. Это уже очень много, но это не то же самое, что наблюдать выполнение. Поэтому в проекте появляется отдельный слой dynamic validation.

Dynamic validation можно перевести как динамическая валидация или динамическая проверка. Слово dynamic здесь означает, что анализ происходит не только над файлом, а во время выполнения программы. Слово validation означает не “магическое доказательство всего”, а проверку конкретной гипотезы наблюдением. Это важный оттенок. Dynamic validation не превращает инструмент во всевидящего судью. Она говорит скромнее: “в этом запуске я видел такие-то события, такие-то остановки, такие-то исключения, такие-то попадания в обработчики; вот насколько это усиливает или ограничивает статический вывод”.

В терминах проекта этот слой реализован как Windows dynamic collector. Collector - это сборщик. Он не обязан сам выносить финальный приговор, как это делает assessment layer; его задача - собрать runtime evidence, то есть свидетельства времени выполнения. Runtime переводится как время выполнения: это не то, что лежит в PE-файле, а то, что произошло с процессом, когда Windows начала его загружать, выделять память, создавать потоки, передавать исключения и останавливать выполнение для debugger-a. Evidence - свидетельство или доказательный материал. В проекте это слово надо понимать осторожно: evidence не всегда доказывает окончательный вывод, но оно добавляет наблюдаемую опору под finding.

От записанной модели к наблюдаемому событию

Чтобы понять место dynamic validation, надо ещё раз спокойно пройти путь, который уже проделала статическая часть. Static analysis начинает с недоверенного массива байтов. Он не верит PE-заголовкам на слово, а проверяет: где секции, где raw data, где virtual ranges, как RVA переводится в file offset, не выходят ли таблицы за границы image, не пересекаются ли диапазоны странным образом. Затем он добирается до unwind-инфраструктуры: Exception Directory указывает на таблицу RUNTIME_FUNCTION, каждая запись RUNTIME_FUNCTION описывает диапазон кода, а поле UnwindInfoAddress ведёт к UNWIND_INFO. Эта цепочка даёт declared model - заявленную модель.

Declared model можно перевести как объявленная или заявленная модель. Это то, что файл сообщает Windows и анализатору: “для такого диапазона кода unwind-инструкция лежит здесь; у этой функции такой пролог; если понадобится раскрутить стек, используй такие unwind codes; если есть handler, его адрес находится в хвосте UNWIND_INFO”. В нормальном бинаре эта модель создаётся компилятором и linker-ом, а Windows loader делает её доступной механизму exception handling и stack walking. Для static analyzer-а declared model - центральный материал: её можно прочитать, проверить, сравнить с кодом и превратить в findings.

Но declared model остаётся моделью. Она может быть корректной, неполной, намеренно искажённой или просто не иметь отношения к коду, который будет выполнен после распаковки. Dynamic validation начинается там, где static analysis уже сформулировал вопрос, но не может сам на него ответить. Например, static analysis видит handler RVA и подозревает, что обработчик используется как скрытый маршрут управления. Это ещё не факт поведения. Поведение появится только тогда, когда поток реально получит исключение, Windows передаст это исключение по своему маршруту, а управление действительно дойдёт до обработчика.

Поэтому dynamic validation не заменяет static analysis. Она стоит рядом с ним как второй тип наблюдения. Static analysis говорит: “в файле есть такая структура, и она выглядит так”. Dynamic validation говорит: “во время запуска произошло такое событие, и управление оказалось здесь”. Между этими фразами находится вся инженерная честность проекта.

> В дальнейшем важно держать в голове простую формулу: static analysis работает с записанной моделью, dynamic validation - с наблюдаемым событием. Findings становятся сильнее, когда эти два мира не противоречат друг другу, но даже тогда вывод остаётся инженерным, а не мистическим.

Execution boundary: момент, когда анализ перестаёт быть безопасным чтением файла

До dynamic validation проект находится по одну сторону границы доверия. Он читает файл как данные. Даже если PE-файл злонамеренный, сломанный или специально созданный для атаки на парсер, static analysis не должен выполнять его код. Здесь главная опасность - ошибиться при чтении, выйти за границу буфера, неправильно перевести RVA, неверно освободить память. Поэтому такие модули, как util.c, pe.c и unwind.c, заняты скучной, но жизненно важной работой: bounds-safe reads, ownership, structural validation. Bounds-safe reads - это чтение с проверкой границ: прежде чем взять байт или поле структуры, код убеждается, что эти байты действительно лежат внутри входного буфера.

Dynamic validation пересекает другую границу. Теперь файл больше не просто набор байтов. Он запускается как child process - дочерний процесс. Слово child здесь означает, что процесс создаётся инструментом и находится под его управлением как цель анализа. На Windows такой процесс можно запустить под debugger-ом, то есть отладчиком. Debugger - это программа, которая получает от операционной системы специальные уведомления о важных событиях в другом процессе: запуск, загрузка DLL, исключение, завершение потока, завершение процесса. Процесс, который отлаживают, называется debuggee.

Debuggee - отлаживаемый процесс. Но это не просто “процесс, на который смотрят со стороны”. В момент debug event Windows реально приостанавливает выполнение debuggee и передаёт управление debugger-у. Это принципиально. Dynamic collector не просто читает лог. Он участвует в ритме выполнения: дождался события, получил структуру DEBUG_EVENT, разобрал её, решил, что делать, и только потом позволил debuggee продолжить работу. Поэтому dynamic validation требует timeout-ов, ограничений, аккуратного завершения и честной фиксации неполных сессий. Если запуск завис, упал раньше времени или не дошёл до интересного пути, это тоже результат, но не сильное доказательство отсутствия механизма.

В проектной архитектуре эта граница прямо отделена: static mode treats the input as untrusted bytes and performs no code execution, a dynamic mode starts the input as a child process under a debugger. Это не декоративная фраза. Она объясняет, почему static analysis можно применять к большим корпусам файлов относительно спокойно, а dynamic validation всегда является более тяжёлой, рискованной и сценарно зависимой процедурой.

Debug loop как дыхание dynamic collector-а

Когда цель запущена под отладчиком, dynamic collector входит в debug loop. Loop - это цикл, но перевод “цикл” слишком сухой. Здесь лучше представить

ритм: Windows останавливает debuggee на событии; collector просыпается; collector смотрит, что случилось; collector обновляет DynamicReport; collector решает, как продолжить; Windows отпускает debuggee до следующего события. Этот ритм повторяется, пока процесс не завершится, не сработает timeout или collector не решит остановить сессию.

В Windows Debug API центральная функция ожидания называется WaitForDebugEvent. Wait - ждать, debug event - событие отладки. Смысл функции почти буквально соответствует названию: debugger ждёт, пока Windows сообщит ему о событии в debuggee. В документации Microsoft подчёркивается, что при debug event система приостанавливает все потоки отлаживаемого процесса и уведомляет debugger. Это значит, что collector получает снимок момента, когда процесс остановлен не произвольно, а в точке, признанной Windows важной для отладки.

Но ожидание события - только половина механизма. После обработки события debugger должен вызвать ContinueDebugEvent. Continue - продолжить. Эта функция сообщает Windows, что debugger закончил рассматривать событие и debuggee можно продолжать. Для exception events это особенно важно, потому что значение продолжения влияет на дальнейший exception processing. Если debugger говорит, что исключение обработано, Windows не обязана дальше искать обработчики тем же путём. Если debugger не обработал исключение, Windows продолжит обычную доставку исключения. Поэтому dynamic collector должен быть очень аккуратен: наблюдение не должно бездумно ломать естественный маршрут исключения, иначе runtime evidence перестанет отражать нормальное поведение цели.

```
while debug session is active:
    wait for DEBUG_EVENT
    translate event into runtime evidence
    update DynamicReport
    decide continue status
    call ContinueDebugEvent
```

Этот псевдокод выглядит маленьким, но за ним скрывается важная философия проекта. Dynamic collector не пытается “понять всю программу”. Он собирает ограниченные, проверяемые события. Он видит не абстрактный смысл программы, а конкретные остановки: здесь создан процесс, здесь загружен модуль, здесь произошло исключение, здесь сработал breakpoint, здесь был вызван API регистрации handler-а, здесь управление дошло до адреса, который static analysis считал handler-ом. Такой подход ограничен, но именно поэтому он надёжен: report сообщает не фантазию анализатора, а наблюдения с понятными границами.

Debug event: событие, в котором Windows временно передаёт процесс отладчику

Термин debug event надо разжевать отдельно, потому что без него dynamic validation превращается в набор названий API. Event - событие. Debug event - событие отладки, то есть специальное уведомление от Windows к debugger-у о том, что в debuggee произошло нечто, требующее внимания. Это может быть запуск процесса, создание потока, загрузка DLL, выгрузка DLL, исключение, завершение потока или процесса. Для нашей книги особенно важны exception events, потому что проект изучает x64 exception/unwind-механику.

Exception event - событие исключения. Когда в debuggee возникает исключение, Windows сначала сообщает об этом debugger-у. Это называется first-chance notification. First chance переводится как первый шанс. Смысл не в том, что программа “уже упала”, а в том, что debugger получает первый шанс увидеть исключение до того, как оно будет передано обычным обработчикам процесса. Новичок часто пугается first-chance exception и думает, что это crash. Но first-chance exception может быть нормальной частью работы: приложение могло намеренно вызвать RaiseException, runtime мог использовать исключение как внутренний механизм, код мог ожидать guard page violation и обработать его.

Если debugger не обработал first-chance exception, Windows продолжает обычный маршрут доставки исключения: VEH, frame-based handlers, language-specific handlers, unwind, top-level handling - в зависимости от ситуации, которую мы разбирали в предыдущих главах. Если никто не обработал исключение, debugger получает second-chance notification. Second chance - второй шанс. Вот он уже намного ближе к аварии: Windows как бы говорит debugger-у, что обычные обработчики не справились, и это последний момент перед default handling, которое часто ведёт к завершению процесса.

Для dynamic validation эта пара first chance / second chance важна не как терминология WinDbg, а как способ понять маршрут. Если collector видит first-chance exception и затем handler hit, это одно наблюдение: исключение было частью обработанного пути. Если он видит first-chance, затем second-chance, затем завершение процесса, это другой сценарий: исключение дошло до аварийной стадии. Если он видит множество first-chance исключений без second-chance, это может быть нормальный exception-based protocol, anti-debug trick, runtime behavior или управляемая диагностика. Нельзя делать вывод без контекста.

Runtime evidence: свидетельство живого выполнения, а не замена статического анализа

Runtime evidence - свидетельство времени выполнения. Это один из тех английских терминов, который нельзя просто оставить без объяснения. Runtime - время, когда программа уже загружена и выполняется. Evidence -

наблюдаемый материал, который можно положить в основу вывода. В dynamic validation таким материалом может быть сам факт exception event, код исключения, адрес RIP в момент остановки, имя загруженного модуля, адрес API, на котором сработал breakpoint, адрес handler-a, куда реально пришло управление, или короткий RIP trace после handler-a.

RIP trace тоже надо понимать не как красивое слово. RIP - instruction pointer в x64, регистр, который указывает на следующую выполняемую инструкцию или на текущую точку остановки, в зависимости от контекста события. Trace - след. RIP trace - короткий след адресов исполнения: collector делает несколько контролируемых шагов или фиксирует последовательность адресов после важного события, чтобы понять, куда управление начинает уходить. Это не полный control-flow graph и не доказательство всего поведения функции. Это маленькое окно после handler hit: было управление в обработчике, а затем оно двинулось в такой-то диапазон.

Почему это полезно? Потому что в exception-based control flow реальный переход часто не похож на обычный call или jmp в дизассемблере. Static analysis может увидеть handler address, но не всегда может уверенно сказать, станет ли этот handler маршрутизатором управления. Runtime evidence добавляет наблюдение: да, handler был hit, после него RIP пошёл не обратно в очевидный epilog, а в другой участок кода; или наоборот, handler был hit и затем процесс пошёл к нормальному завершению. Оба результата важны, но они значат разные вещи.

Такой подход защищает проект от двух крайностей. Первая крайность - верить только статике и объявлять подозрительным всё, что выглядит необычно в PE. Вторая крайность - верить только динамике и считать, что если в одном запуске событие не произошло, значит механизма нет. Правильная инженерная позиция середина: static analysis даёт структурную картину и hypotheses; dynamic validation добавляет runtime evidence по конкретному сценарию запуска; assessment соединяет эти уровни осторожно.

Адреса в двух мирах: почему RVA из файла нельзя просто поставить как breakpoint

Один из самых важных мостов между static analysis и dynamic validation - адреса. В статике мы часто говорим RVA. Relative Virtual Address - относительный виртуальный адрес. Relative означает относительный, потому что адрес считается не от нуля всей памяти процесса, а от ImageBase модуля. Если ImageBase равен 0x140000000, а функция имеет RVA 0x1234, то её ожидаемый VA при такой загрузке будет 0x140001234. VA - virtual address, виртуальный адрес в адресном пространстве процесса.

Но во время выполнения появляется ASLR. Address Space Layout Randomization - рандомизация раскладки адресного пространства. Это защитный механизм,

из-за которого модуль не обязан загрузиться по preferred ImageBase из PE. В результате static RVA остаётся полезным, но runtime VA надо вычислять заново по реальному base address загруженного модуля. Dynamic collector поэтому должен иметь module map - карту модулей. Module map не просто “список DLL”. Это таблица, которая связывает имя или путь модуля, его runtime base, размер, а иногда и дополнительные свойства. Без такой карты нельзя честно перенести static finding в runtime breakpoint.

Представим, что static analysis нашёл handler RVA 0x5000 в основном image. Это не значит, что breakpoint надо ставить на 0x5000. Это даже не значит, что его надо ставить на preferred ImageBase + 0x5000. Правильная мысль такая: “к какому модулю относится этот RVA, какой runtime base у этого модуля в debuggee, и какой VA получится после сложения?” Только после этого адрес становится пригодным для динамического наблюдения.

```
static side:
    handler_rva = 0x00005000

runtime side:
    module_runtime_base = 0x00007ff6a1200000
    handler_runtime_va  = module_runtime_base + handler_rva
```

Эта разница особенно важна потому, что analyzer process и debuggee process живут в разных адресных пространствах. Адрес функции в процессе анализатора не равен адресу той же функции в target-е. Даже системные DLL могут быть загружены по разным адресам. Поэтому breakpoint на AddVectoredExceptionHandler, SetUnhandledExceptionFilter или RtlAddFunctionTable нельзя ставить по адресу, найденному в собственном процессе инструмента, без проверки адреса в debuggee. Нужно смотреть module events, module bases и exports в контексте target process. Иначе dynamic validation будет наблюдать не то место или вообще ничего не поймает.

Breakpoint как искусственная остановка, которую collector обязан поставить и убрать аккуратно

Breakpoint обычно переводят как точка останова. Но для понимания dynamic collector-а этого перевода мало. Software breakpoint на x86/x64 часто реализуется байтом 0xCC, инструкцией INT 3. Collector временно заменяет первый байт инструкции в debuggee на 0xCC. Когда CPU доходит до этого места, возникает breakpoint exception, Windows превращает его в exception debug event, debuggee останавливается, а collector получает шанс посмотреть, куда пришло управление.

Эта техника кажется простой только на поверхности. На самом деле collector вмешивается в память процесса. Он должен сохранить оригинальный байт инструкции, записать 0xCC, дождаться срабатывания, понять, что это именно

его breakpoint, восстановить оригинальный байт, поправить RIP, выполнить один шаг через оригинальную инструкцию и при необходимости поставить breakpoint снова. Поправить RIP нужно потому, что после INT 3 процессор обычно оказывается уже за байтом breakpoint-а; если оставить RIP как есть, выполнение продолжится не с начала оригинальной инструкции, а со следующего байта, что сломает поток инструкций.

Single-step - одиночный шаг. Для него используется Trap Flag в EFlags/RFlags. Trap Flag можно перевести как флаг ловушки: если он установлен, процессор генерирует исключение после выполнения одной инструкции. Collector использует это, чтобы аккуратно пройти оригинальную инструкцию после временного снятия breakpoint-а, а потом снова вооружить breakpoint. Re-arm breakpoint - снова поставить точку останова после того, как она уже сработала и была временно убрана.

Зачем такая сложность? Потому что dynamic validation должна быть наблюдением, а не разрушением программы. Если breakpoint поставлен грубо, он изменит поведение target-а. Если не восстановить байт, программа будет снова и снова попадать в INT 3. Если не поправить RIP, она выполнит мусор. Если не re-arm breakpoint, collector увидит только первое попадание и потеряет следующие. Поэтому software breakpoint - не просто “поставили ловушку”, а маленький договор с машинным кодом: остановить, посмотреть, восстановить, пропустить, вернуть наблюдение на место.

Регистрации обработчиков как момент, когда программа объявляет будущий маршрут

Dynamic collector интересуется не только тем, куда управление уже пришло, но и тем, какие маршруты программа регистрирует заранее. Registration - регистрация. В контексте exception handling это момент, когда код сообщает Windows: “вот функция, которую надо вызвать при определённых исключениях или при top-level handling”. Для vectored exception handling это AddVectoredExceptionHandler. Для vectored continue handling - AddVectoredContinueHandler. Для unhandled exception filter - SetUnhandledExceptionFilter. Для dynamically generated code - RtlAddFunctionTable или RtlInstallFunctionTableCallback.

Здесь важно не свалить всё в один список API. У каждой регистрации своя роль. AddVectoredExceptionHandler добавляет process-wide обработчик, который не привязан к конкретному стековому кадру. Он получает шанс увидеть исключение до frame-based search. AddVectoredContinueHandler относится к продолжению после обработки: это другой слой, который может появиться, когда система пытается продолжить выполнение. SetUnhandledExceptionFilter задаёт top-level filter - последний пользовательский фильтр перед default crash handling, когда обычные

обработчики не справились. RtlAddFunctionTable и RtlInstallFunctionTableCallback вообще не про “обработчик исключения” напрямую, а про регистрацию function table для кода, который не описан обычной .pdata в PE-файле.

Dynamic collector ставит breakpoints на такие API не потому, что каждый вызов подозрителен. Нормальная программа может ставить crash reporter через SetUnhandledExceptionFilter, runtime может регистрировать function tables для generated code, framework может использовать VEH для диагностики. Интересен не сам факт вызова, а его место в общей доказательной картине. Если static analysis видит странную metadata placement, а dynamic validation наблюдает RtlAddFunctionTable в ранней фазе выполнения, это может объяснить, почему файл на диске не содержит полной unwind-карты. Если static analysis видит handler dominance, а dynamic validation наблюдает частые handler hits, static finding получает runtime support.

Registration event - это ещё не handler hit. Регистрация говорит: “такой маршрут возможен”. Handler hit говорит: “управление действительно дошло до этого адреса”. Между возможностью и произошедшим событием лежит вся разница между static hypothesis и runtime evidence.

Handler hit: не “адрес существует”, а “управление реально пришло”

Handler hit - попадание в обработчик. Hit буквально означает удар или попадание, но в отладочной речи это означает, что breakpoint сработал на нужном адресе. Если static analysis нашёл handler RVA в UNWIND_INFO и dynamic collector поставил breakpoint на соответствующий runtime VA, то срабатывание этого breakpoint-а даёт сильное наблюдение: поток выполнения действительно пришёл туда, куда static model указывала как на handler-related code.

Но даже handler hit надо трактовать осторожно. Во-первых, адрес мог быть не только entry point handler-a, но и shared runtime routine, например language-specific handler вроде __C_specific_handler. Во-вторых, один handler может обслуживать много функций, и сам факт попадания не говорит, какой protected range был причиной. В-третьих, если breakpoint стоит на API регистрации, а не на теле handler-a, это вообще другой тип события. В-четвёртых, если код packed или manually mapped, статическая связь RVA -> module -> runtime VA может быть неполной.

Поэтому хороший report не должен писать “handler confirmed” без контекста. Он должен сказать примерно так: статическая модель указывала на handler RVA; этот RVA был перенесён в runtime VA через module map; на этом адресе был установлен breakpoint; во время сессии breakpoint сработал; после срабатывания был записан RIP trace; событие усиливает handler-related static

finding, но не является полным proof graph protected-range-to-handler. Именно такая осторожность отличает инженерный анализ от красивой, но ложной уверенности.

Dynamic function tables: когда unwind-карта появляется во время выполнения

Одна из важнейших причин существования dynamic validation в этом проекте - dynamic function tables. Function table - таблица функций, то есть таблица RUNTIME_FUNCTION-like записей, по которым Windows может понять, как unwound код. Dynamic function table - такая таблица, добавленная во время выполнения, а не взятая из .pdata PE-файла. Это нужно JIT-движкам, runtime-generated code, некоторым упаковщикам, loaders и инструментирующим системам.

Если программа создаёт исполняемый код в памяти, CPU может начать его выполнять. Но Windows exception/unwind subsystem от этого автоматически не узнает, как восстанавливать стек в этом коде. Для обычного PE компилятор и linker заранее кладут .pdata/.xdata в image. Для generated code такой роскоши нет. Поэтому приложение должно предоставить function table через RtlAddFunctionTable или callback-механизм вроде RtlInstallFunctionTableCallback. Microsoft прямо описывает, что function tables на 64-bit Windows используются для unwind или stack walking, а dynamically generated code должен предоставить такие сведения системе.

Static analysis файла на диске может увидеть признаки: мало .pdata, странные sections, imports или вызовы VirtualAlloc/VirtualProtect, но он не увидит таблицу, которая будет создана только после запуска. Dynamic collector здесь закрывает очень конкретную дыру наблюдения: он может поймать вызов RtlAddFunctionTable, зафиксировать base address, количество записей, диапазон, а затем связать это с runtime memory. Это не отменяет статический анализ, а объясняет, почему статическая карта на диске была неполной.

Для assessment это особенно важно. Если static analysis ругается на uncovered executable memory, а dynamic validation показывает корректную регистрацию dynamic function table, confidence подозрения может измениться: часть странности получила нормальное runtime explanation. Если же executable private memory появилась, exceptions происходят внутри неё, а dynamic function table не зарегистрирована, это другой сигнал: Windows может не иметь нормальной unwind-инструкции для кода, который реально исполняется.

DynamicReport как дневник наблюдений, а не приговор

DynamicReport - динамический отчёт. Report переводится как отчёт, но в проектном смысле это не литературное описание, а структурированная

модель событий, которую затем можно сериализовать в JSON/HTML и объединить с assessment. В DynamicReport должны попадать не только “успешные” подтверждения, но и ограничения: session ok или not ok, timeout, incomplete instrumentation, dropped events, отсутствие natural target exit. Эти поля нужны не для красоты, а чтобы downstream assessment не делал слишком сильный вывод из слабой или неполной сессии.

Например, если target завершился естественно, collector успел поставить breakpoints, увидел module load events, поймал handler hit и записал RIP trace, runtime evidence будет сильнее. Если target завис, был остановлен по timeout, breakpoints не все поставились, а события обрезаны, dynamic_validation всё равно может сохранить raw observations, но evidence level должен быть ограничен. Неполная dynamic session не должна превращаться в “ничего не найдено”. Она означает: “в этом запуске при таких ограничениях наблюдение неполное”.

Это один из самых важных уроков для будущего reader-а и для самого инструмента. Хороший анализатор ценен не тем, что всегда говорит уверенно. Он ценен тем, что знает, когда уверенность невозможна. Если dynamic validation не дошла до нужного пути, честный report должен оставить место для manual review. Manual review - ручная проверка аналитиком: открыть Ghidra, посмотреть imported CSV/comments, запустить target под WinDbg/x64dbg, проверить адреса, воспроизвести сценарий, который ведёт к интересному exception path.

Assessment как место встречи статического и динамического слоёв

Assessment - оценка, итоговое аналитическое заключение. В проекте assessment не должен быть плоским “bad/good”. Он соединяет разные классы evidence. Static analysis может дать findings из table_integrity, unwind_integrity, metadata_placement, unwind_reuse, handler_dominance. Heuristic/evidence engine превращает их в score, suspicion, confidence, evidence level, weakening rules. Dynamic validation добавляет runtime evidence: exceptions, registrations, handler hits, dynamic function table activity, RIP trace. Assessment должен объяснить, как эти слои связаны и насколько вывод надёжен.

Здесь легко ошибиться. Если static analysis нашёл handler dominance, а dynamic validation увидела handler hit, это не значит, что “всё доказано”. Это значит, что handler-related static evidence получило runtime support. Если static analysis нашёл writable unwind metadata, а dynamic validation не увидела никаких исключений, это не опровергает static finding. Возможно, сценарий не дошёл до нужного кода. Если dynamic validation увидела RtlAddFunctionTable, а static analysis видел маленькую или неполную .pdata, это может быть benign explanation или часть более сложной runtime code story. В каждом случае

assessment обязан говорить слоями.

Именно поэтому в проекте полезна идея case strength. Case - случай или дело, strength - сила. Это не юридический термин, а удобная инженерная шкала: no_case, weak_case, medium_case, strong_unconfirmed_case, correlated case. Correlation - соотнесение, связь между статическим и динамическим evidence. Но проектная документация подчёркивает, что correlation слой консервативный и грубый: это не per-function proof graph. Он не доказывает каждую функцию и каждый protected range, а фиксирует, что некоторые классы static evidence получили совместимое runtime evidence.

Такой стиль assessment должен отражаться и в книге. Мы не учим читателя нажимать кнопку “scan” и верить цифре. Мы учим его читать evidence chain. Evidence chain - цепочка свидетельств: байты PE, структурная проверка, decoded unwind model, finding, runtime event, DynamicReport, final assessment. Если одно звено слабое, весь вывод должен стать осторожнее.

Мини-сцена: обработчик найден в metadata и реально hit-нулся

Представим цель, где static analysis обнаруживает UNW_FLAG_EHANDLER в UNWIND_INFO нескольких функций. Этот флаг говорит, что после массива unwind codes в .xdata есть хвост, связанный с exception handler. Static analyzer переводит UnwindInfoAddress, декодирует структуру, находит handler RVA и замечает, что один и тот же handler target встречается необычно часто. Heuristic/evidence engine оформляет finding из семейства handler_dominance. Пока это только static evidence: файл заявляет, что многие функции связаны с одним обработчиком.

Затем включается dynamic validation. Windows dynamic collector запускает target как debuggee, получает module base, переносит handler RVA в runtime VA и ставит breakpoint. Во время выполнения происходит first-chance exception. Debug loop получает exception event. Collector продолжает обработку так, чтобы Windows могла доставить исключение обычным маршрутом. Через несколько событий срабатывает breakpoint на handler runtime VA. Это handler hit. После этого collector записывает короткий RIP trace: куда выполнение двинулось после входа в handler.

Теперь finding стал богаче. Он больше не говорит только “в metadata много ссылок на один handler”. Он говорит: “metadata указывает на dominant handler; во время dynamic session этот handler был реально достигнут; post-handler RIP trace показывает начало дальнейшего маршрута”. Но даже здесь report должен оставить честные границы: это один сценарий запуска; он не доказывает, что все функции с этим handler-ом проходят через него; он не объясняет весь protected range; он не заменяет ручной разбор handler body. Он усиливает case, но не отменяет анализ.

Мини-сцена: static finding остался без runtime подтверждения

Теперь другой сценарий. Static analysis обнаруживает writable section, в которой лежит unwind metadata. Это необычно, потому что unwind-информация - инструкция для Windows о восстановлении кадра, и её изменение во время выполнения может менять stack walking и exception handling. Heuristic/evidence engine создаёт finding из metadata_placement и, возможно, requires_manual_review. Затем запускается dynamic validation, но target в данном запуске не генерирует исключений в интересных диапазонах, handler hits не случается, RtlAddFunctionTable не вызывается, процесс быстро завершается.

Плохой анализатор сказал бы: “не подтвердилось, значит неважно”. Хороший анализатор так не говорит. Отсутствие runtime event в одном запуске не является доказательством отсутствия возможности. Может быть, нужен другой input, другой command line, другая дата, другой network state, другой пользовательский путь. Поэтому DynamicReport должен сохранить, что events not observed, а assessment должен оставить static finding в силе, возможно с пометкой unconfirmed by this dynamic session. Это и есть дисциплина evidence model.

Мини-сцена: runtime activity объясняет статическую неполноту

Третий сценарий показывает, почему dynamic validation может не только усиливать подозрение, но и снижать чрезмерную тревогу. Static analysis видит, что .pdata в файле покрывает только маленький loader stub. Основной код, судя по imports и memory behavior, может появиться позже. Само по себе это требует внимания. Но dynamic collector во время запуска видит вызов RtlAddFunctionTable: target регистрирует function table для диапазона памяти, который позже становится executable. В таком случае runtime evidence объясняет часть статической неполноты: unwind map действительно появляется не из PE на диске, а через Windows API во время выполнения.

Это не автоматически делает программу безопасной. Packer, JIT, instrumentation runtime и manual loader могут использовать похожие механизмы. Но вывод становится точнее: вместо “у файла нет нормальной .pdata, всё сломано” report может сказать: “статическая PE-карта неполна; во время выполнения наблюдалась регистрация dynamic function table; требуется сопоставить runtime table range с исполняемой памятью и проверить handler/unwind behavior”. Такой вывод учит аналитика думать, а не просто бояться.

Язык отчёта как часть качества анализа

Один и тот же факт можно написать плохо и хорошо. Плохо: “dynamic handler confirmed”. Лучше: “runtime breakpoint на handler-derived address был hit во время debug session; событие поддерживает handler-related static evidence, но не является полной per-function reconstruction”. Плохо: “no dynamic evidence”. Лучше: “в данной dynamic session не наблюдалось handler hits или dynamic function table activity; session была complete/incomplete; отсутствие события не опровергает static finding без сценарного покрытия”.

Этот язык кажется многословным, но он защищает от ложных выводов. В reverse engineering опасны не только ошибки парсера. Опасны слишком уверенные фразы. Если report звучит как приговор там, где есть только один слабый сигнал, аналитик начинает видеть malware в нормальном runtime. Если report звучит слишком мягко там, где static и dynamic evidence согласованы, он теряет важный сигнал. Поэтому project terminology - findings, evidence families, weakening rules, DynamicReport, assessment - нужна не для красоты, а чтобы вывод был честным и воспроизводимым.

Module map и перенос API breakpoints: почему адрес из своего процесса нельзя использовать в target-е

В dynamic collector-е есть ещё один слой, который легко недооценить, если писать слишком быстро: перенос адресов системных API. Нам нужно ставить breakpoints на функции вроде AddVectoredExceptionHandler или RtlAddFunctionTable. Наивная мысль звучит так: “получим адрес функции через GetProcAddress у себя и поставим breakpoint по этому адресу в target”. Эта мысль неправильная, потому что analyzer process и debuggee process - разные процессы. У каждого своё виртуальное адресное пространство. Даже если оба процесса используют одну и ту же DLL, её base address может отличаться из-за ASLR.

Поэтому проект использует module map. Мы уже говорили, что module map - карта модулей, но теперь надо понять её роль глубже. Это не просто справочник для красивого вывода “kernel32.dll+offset”. Это мост между локальным знанием analyzer-a и удалённым адресным пространством debuggee. Collector может у себя найти, что символ AddVectoredExceptionHandler фактически находится не в том модуле, который кажется очевидным, а в forwarded target. Forwarded export - перенаправленный экспорт. В Windows DLL может экспортировать имя функции, но реальный код этой функции находится в другой DLL. Например, имя может быть запрошено через kernel32.dll, а фактическая реализация жить в kernelbase.dll. Поэтому collector должен определить не только address symbol-a, но и real local module, затем вычислить offset внутри этого real module, а уже потом найти такой же module в target и сложить target module base с offset.

Offset здесь означает смещение внутри модуля. Если локально функция лежит по адресу LocalModuleBase + 0x12340, то 0x12340 - это offset. При условии, что в target загружена совместимая версия той же DLL, функция ожидается по TargetModuleBase + 0x12340. Это инженерное предположение. Оно разумно для обычного запуска на одной Windows installation, но это всё ещё предположение. Если target подменил DLL, если архитектура отличается, если используется unusual loader или если export forwarding ведёт иначе, перенос может быть неверным. Хороший DynamicReport должен помогать аудиту: показывать module+offset и self-check установки breakpoint-а, а не просто молча утверждать, что breakpoint поставлен.

Термин self-check переводится как самопроверка. В проекте это важная маленькая деталь: после установки breakpoint-а collector заново читает byte из remote process и проверяет, что там действительно стоит 0xCC. Это не философия, а практическая защита от ложной уверенности. Если WriteProcessMemory сказал success, но по факту memory не изменилась или изменилась не там, report должен дать warning. Иначе dynamic validation будет строить вывод на предположении “мы поставили breakpoint”, хотя фактического breakpoint-а не было.

Remote memory write: почему постановка breakpoint-а сама является вмешательством

Remote memory write - запись в память другого процесса. Remote здесь означает удалённый не в сетевом смысле, а находящийся в другом process address space. Когда collector ставит software breakpoint, он меняет byte в debuggee. Это уже вмешательство. В нормальной ситуации WriteProcessMemory может записать 0xCC прямо в нужный адрес. Но executable pages часто имеют protection вроде PAGE_EXECUTE_READ, то есть исполнять можно, а писать нельзя. Тогда collector временно меняет protection страницы через VirtualProtectEx, записывает byte, возвращает прежнюю protection и вызывает FlushInstructionCache.

Protection страницы - это права доступа к странице памяти: можно ли читать, писать, исполнять. PAGE_EXECUTE_READWRITE, часто сокращаемое как RWX, означает, что страницу можно читать, писать и исполнять. Для security-мышления это опасная комбинация, потому что writable executable memory облегчает self-modifying code и injection tricks. Но debugger иногда вынужден временно расширять права, чтобы поставить breakpoint. Поэтому факт временного PAGE_EXECUTE_READWRITE внутри collector-а не надо путать с тем, что target сам держит RWX memory как часть своего поведения. Это разные evidence layers: одно - вмешательство инструмента, другое - свойство debuggee.

FlushInstructionCache - сброс instruction cache. Процессор может заранее держать instruction bytes в кэше. Если debugger изменил байт инструкции в памяти, нужно обеспечить, чтобы CPU при следующем fetch увидел новый байт. Иначе возникнет странный эффект: память уже изменилась, а процессор продолжает выполнять старые bytes. В обычном прикладном коде об этом редко думают, но в dynamic collector-е это часть корректности наблюдения. Мы не просто пишем 0xCC; мы должны сделать так, чтобы hardware и Windows согласованно увидели изменение.

Эта тема важна для книги, потому что показывает: dynamic validation не является “чистым взглядом со стороны”. Она контролируемо вмешивается. Поэтому отчёт должен быть честным о completeness и limitations. Если sample anti-debug-sensitive, даже сам факт debugger-a, breakpoints, single-step и изменённой page protection может изменить поведение. Dynamic evidence полезна, но она всегда evidence of this instrumented run, а не абсолютная истина о всех возможных запусках.

Аргументы API в Windows x64: почему collector смотрит в RCX, RDX, R8 и R9

Когда breakpoint срабатывает на entry функции вроде AddVectoredExceptionHandler, collector хочет понять, какой handler регистрирует программа. Но как получить аргументы вызова? Здесь возвращается фундамент calling convention. Calling convention - соглашение о вызовах: набор правил, по которым caller передаёт аргументы callee, кто сохраняет какие регистры, где лежит return address, как выравнивается стек. На Windows x64 первые четыре целочисленных или pointer-аргумента передаются в регистрах RCX, RDX, R8 и R9. Поэтому на входе AddVectoredExceptionHandler collector читает RCX как First, а RDX как Handler.

First в AddVectoredExceptionHandler - это параметр порядка. Он говорит, куда поместить обработчик в цепочке vectored handlers: ближе к началу или в обычное место. Handler - указатель на функцию-обработчик. Указатель, pointer, это адрес. Когда collector видит nonzero Handler, он может поставить breakpoint уже на сам handler address. Но здесь снова нужна осторожность: collector наблюдает entry API и arguments, но не обязательно наблюдает return value. Return value - значение, которое функция возвращает caller-у, обычно через RAX. Для AddVectoredExceptionHandler return value является handle регистрации. Если collector не ставит return breakpoint, он не знает наверняка, успешна ли регистрация. Поэтому registration event означает “был вызов с такими аргументами”, а handler hit позже означает “управление реально дошло до handler-а”.

Та же логика работает для AddVectoredContinueHandler. Для SetUnhandledExceptionFilter первый аргумент находится в RCX и является

pointer-ом на top-level filter. Для RtlAddFunctionTable Windows x64 ABI даёт RCX = FunctionTable, RDX = EntryCount, R8 = BaseAddress. Эти три значения очень важны: FunctionTable - адрес массива runtime function entries в памяти debuggee; EntryCount - количество записей; BaseAddress - база, относительно которой interpreting entries получает смысл. Но текущий collector, согласно проектной документации, фиксирует вызов и arguments, не читая полностью remote function table entries. Значит runtime event подтверждает activity, но не декодирует содержимое dynamic table. Это опять граница evidence.

Для RtlInstallFunctionTableCallback аргументы ещё интереснее: TableIdentifier, BaseAddress, Length, Callback. Callback - функция обратного вызова, которую система может использовать, чтобы получить function table entries по запросу. Callback переводится как обратный вызов: не программа прямо вызывает Windows, а Windows позже вызывает функцию, которую программа ей передала. Такой механизм более динамический: таблица может вычисляться по требованию. Для полного анализа потребовалось бы ставить breakpoint на callback и разбирать его поведение. Текущий collector фиксирует регистрацию callback table как runtime evidence, но не строит полную реконструкцию.

Continuation status: как одно неправильное решение debugger-а меняет судьбу исключения

После debug event collector должен выбрать continuation status - статус продолжения. Это маленькое поле управляет большим смыслом. Для собственных служебных breakpoint-ов collector обычно возвращает DBG_CONTINUE, потому что это его breakpoint, его INT3, его искусственная остановка. Он восстановил byte, поправил RIP, включил single-step и говорит Windows: “это событие я обработал”. Если он передаст такой INT3 приложению как не обработанный, target может увидеть чужой breakpoint exception и пойти по совершенно неестественному маршруту.

Для обычных исключений target-а collector должен поступать иначе. Если это не его breakpoint и не его single-step, он не должен самовольно поглощать исключение. Поэтому используется DBG_EXCEPTION_NOT_HANDLED. Эта фраза переводится как “исключение не обработано debugger-ом”. Но смысл не в том, что исключение вообще не обработано; смысл в том, что debugger отдаёт его дальше обычному механизму Windows. Тогда VEH, SEH, language-specific handlers и top-level handling получают шанс сработать естественно. Если здесь ошибиться и вернуть DBG_CONTINUE, collector может случайно скрыть исключение от программы. Тогда dynamic validation будет уже не наблюдением маршрута, а созданием другого маршрута.

Это место особенно важно для нашего учебного фундамента. Windows exception flow - это не просто “произошла ошибка”. Это диспетчеризация события между debugger-ом, vectored handlers, frame-based handlers,

unwind-механикой и default handling. Debugger стоит в этом маршруте очень рано. Поэтому dynamic collector, если он хочет быть исследовательским инструментом, должен знать, когда вмешиваться, а когда отступить и дать Windows продолжить нормальную доставку.

Event bounds, timeout и dropped events: честность важнее красивой полноты

Проект ограничивает количество dynamic events и trace entries. Это инженерное решение. Любой target может породить слишком много событий: тысячи exceptions, бесконечный loop, множество DLL load events, повторяющиеся breakpoints. Если report будет бесконечно расти, сам анализатор станет ненадёжным. Поэтому DynamicReport использует bounded events - ограниченный массив событий. Когда место заканчивается, новые events не перезаписывают старые; увеличивается dropped_events. Dropped events - отброшенные события. Это не ошибка морали, а часть контракта: отчёт обязан сказать, что видел не всё.

Timeout - ограничение времени сессии. Dynamic validation не должна зависнуть навсегда из-за target-а, который ждёт input, спит, ушёл в бесконечный loop или намеренно тормозит debugger. Timeout означает, что сессия была остановлена по времени. Такой report может содержать ценные observations, но его runtime evidence level должен быть ограничен. Нельзя считать отсутствие handler hit сильным отрицательным доказательством, если target просто не дошёл до нужного пути до timeout-а.

Trace_steps - количество шагов RIP trace, которое collector пытается записать после первого handler hit. Это тоже bounded. Если trace_steps = 32, report сохраняет entry handler RIP и до 31 последующего адреса. Почему не трассировать бесконечно? Потому что цель dynamic validation не в полной эмуляции программы, а в локальном свидетельстве: после handler-а управление начало двигаться туда-то. Чем больше шагов, тем выше overhead и сильнее вмешательство single-step. Чем меньше шагов, тем короче окно наблюдения. Это инженерный компромисс, который report должен сохранять как часть metadata.

Сценарность dynamic validation: один запуск не равен всей программе

Ещё одна причина, почему dynamic validation надо объяснять осторожно, - сценарность. Scenario - сценарий запуска: command line arguments, current directory, environment variables, input files, network availability, time, user interaction, privileges, installed DLLs. Одна и та же программа может иметь десятки путей выполнения. В одном запуске она не дойдёт до exception-based control flow, в другом - дойдёт. В одном запуске packer распакует payload, в

другом - завершится раньше. В одном запуске crash reporter зарегистрируется, но не сработает, потому что crash не произошёл.

Проект учитывает это через command line handling и report metadata. Windows CreateProcessA принимает mutable command line string, а не массив argv как в привычной Unix-модели. Поэтому collector должен корректно процитировать arguments: пробелы, tabs, кавычки, backslashes. Если ошибиться в quoting, target получит другие arguments, а dynamic evidence будет относиться к другому сценарию. Это прекрасный пример того, как “мелкая инженерная деталь” становится частью достоверности анализа.

Report metadata может скрывать child arguments по умолчанию, потому что они могут содержать secrets. Secrets - секретные данные: токены, пароли, пути, ключи, приватные параметры. Значит инструмент должен одновременно сохранять воспроизводимость и не утекать лишнее. Это не связано напрямую с .pdata, но связано с качеством настоящего анализатора: техническая точность без аккуратного обращения с данными пользователя тоже недостаточна.

Куда этот раздел ведёт дальше

Теперь у нас есть более цельная картина архитектурной середины проекта. Static analysis строит не просто карту, а проверяемую модель PE и unwind-инфраструктуры. Heuristic/evidence engine превращает структурные факты в осторожные findings, score, confidence, weakening rules и assessment inputs. Dynamic validation через Windows dynamic collector добавляет runtime evidence: debug events, exceptions, registrations, handler hits, dynamic function table activity и RIP trace. Report layer затем должен показать это человеку так, чтобы тот видел не только итог, но и путь от байтов до вывода.

Следующий естественный шаг - разобрать, как эти наблюдения попадают в reports: console, CSV, JSON, HTML и Ghidra CSV. Но делать это надо не как перечисление форматов. Важно понять, что report - это не упаковка результата в красивый файл, а способ сохранить evidence chain так, чтобы другой человек или инструмент мог её проверить. JSON нужен для машины и schema validation. HTML нужен для чтения человеком. CSV и Ghidra CSV нужны для рабочего reverse engineering процесса. И все они должны не терять осторожность assessment, которую мы только что построили.

Короткое закрепление

Dynamic validation в проекте означает наблюдение выполнения target-а под Windows Debug API. Она не отменяет static analysis и не доказывает всё поведение программы. Её задача - собрать runtime evidence: exception events, registrations, handler hits, dynamic function table calls, RIP trace и ограничения самой сессии. Windows dynamic collector работает через debug loop: ждёт

debug event, разбирает его, обновляет DynamicReport и продолжает debuggee через ContinueDebugEvent. Самые важные термины здесь - debuggee, debug event, first-chance exception, second-chance exception, breakpoint, handler hit, runtime VA, module map, DynamicReport и assessment. Каждый из них описывает не абстрактную кнопку инструмента, а конкретное место, где живое выполнение связывается с ранее построенной статической моделью.

Главный вывод раздела простой: static evidence говорит, что файл заявляет и как он устроен; runtime evidence говорит, что удалось увидеть в конкретном запуске; assessment должен соединять эти слои осторожно. Сильный инструмент не тот, который всегда громко обвиняет, а тот, который умеет объяснить, где факт, где интерпретация, где подтверждение, а где честное неизвестно.

Техническая база раздела

Этот раздел опирается на документацию Microsoft по Windows Debug API: debug loop, WaitForDebugEvent, ContinueDebugEvent, first-chance и second-chance exception notifications. Для unwind-механики используется документация по x64 exception handling, RUNTIME_FUNCTION, UNWIND_INFO и dynamic function tables. Для проектной терминологии использованы фактические документы EH/Unwind Analyzer: ARCHITECTURE.md, evidence_matrix.md, описание heuristic/evidence engine и Windows dynamic collector.

9.4. Static + dynamic correlation: как статические свидетельства встречаются с живым выполнением

К этому месту книги мы уже достаточно долго шли по двум дорогам, которые сначала казались разными. Первая дорога начиналась в файле: в байтах PE-образа, в заголовках, в секциях, в таблице исключений, в записях `RUNTIME_FUNCTION`, в структурах `UNWIND_INFO` и в тех маленьких несоответствиях, которые static analyzer может заметить, не запуская программу. Вторая дорога начиналась уже не в файле, а в живом процессе: Windows загрузила образ, создала потоки, передала отладчику события, остановила debuggee на исключении или breakpoint-е, позволила dynamic validation увидеть handler hit, регистрацию vectored handler-а или вызов dynamic function table API. Пока эти две дороги рассматриваются отдельно, у нас есть либо аккуратная карта, либо отдельные наблюдения за движением. Но настоящий инженерный смысл появляется в момент, когда мы начинаем осторожно сопоставлять одно с другим.

Это сопоставление в проекте удобно называть correlation. Английское слово `correlation` обычно переводят как "корреляция", но в этой главе лучше думать о нём проще: это не магическая статистика и не автоматическое доказательство, а аккуратная проверка связи между двумя свидетельствами. Одно свидетельство говорит: "в файле есть такой-то признак". Другое говорит: "во время выполнения наблюдалось такое-то событие". Correlation начинается только тогда, когда эти два высказывания не просто стоят рядом в отчёте, а действительно относятся к одной и той же смысловой области: к handler-ам, к unwind metadata, к dynamic function tables, к placement-anomalies или к маршруту исключения.

Сразу важно удержать границу. Correlation не превращает анализатор в пророка. Если static analysis нашёл странное размещение `UNWIND_INFO`, а dynamic validation увидела вызов `RtlAddFunctionTable`, это усиливает версию о runtime-коде или нестандартной unwind-карте, но не доказывает автоматически вредоносность. Если static analysis нашёл handler-related evidence, а Windows dynamic collector увидел handler hit, это говорит, что какая-то часть статической гипотезы действительно проявилась в запуске, но не доказывает, что все функции, все обработчики и все маршруты уже полностью поняты. Такой осторожный язык не слабость. Это признак честного reverse engineering: мы разделяем то, что увидели, то, что поняли, и то, что пока только предполагаем.

Два вида свидетельств и один объект исследования

Обычно новичок хочет очень быстро получить итоговый вердикт. Он видит PE-файл, запускает анализатор и ждёт короткой фразы: нормальный,

подозрительный, сломанный, упакованный. Но программа на Windows x64 слишком многослойна, чтобы честный вывод рождался мгновенно. Один и тот же объект - executable image - можно наблюдать в разных состояниях. На диске он существует как последовательность file bytes, то есть байтов файла. После загрузки он становится mapped image, образом в адресном пространстве процесса. Во время исполнения он превращается в running process, где появляются потоки, стек, runtime-allocated memory, debug events, исключения и динамически зарегистрированные таблицы функций. Каждое состояние открывает одни факты и скрывает другие.

Static analysis работает с первым состоянием. Он читает файл как недоверенные bytes. Слово `static` здесь означает "статический", то есть выполняемый без запуска target-a. Static analyzer не даёт коду файла исполняться, не вызывает его entry point, не позволяет ему распаковать payload, не даёт ему зарегистрировать dynamic function table. Он смотрит на то, что файл заявляет о себе прямо сейчас: какие у него секции, какие RVA считаются valid, где находится Exception Directory, какие диапазоны описаны в `RUNTIME_FUNCTION`, какие `UNWIND_INFO` можно декодировать, какие handler RVA указаны в metadata. Это безопасный и воспроизводимый взгляд, но он ограничен: он видит только то, что доступно в файле и в статической модели образа.

Dynamic validation работает с другим состоянием. Слово `dynamic` означает "динамический", связанный с выполнением. Здесь target уже запускается как debuggee. Debuggee - это отлаживаемый процесс, то есть процесс, жизнь которого временно проходит под наблюдением debugger-a. Debugger в этой роли не просто "смотрит со стороны". Windows сообщает ему о debug events, приостанавливает потоки debuggee и ждёт решения, как продолжить. Поэтому runtime evidence - свидетельство времени выполнения - имеет другую природу. Это уже не байты файла, а наблюдения за тем, что произошло: возникло исключение, сработал handler, зарегистрировался VEH, был вызван `RtlAddFunctionTable`, после handler-a `RIP` ушёл в определённый диапазон.

Оба вида свидетельств говорят об одной программе, но говорят на разных языках. Static evidence говорит языком структур: RVA, sections, ranges, flags, handler RVA, parse failures, placement. Runtime evidence говорит языком событий: debug event, exception code, first chance, second chance, breakpoint hit, handler hit, module load, post-handler instruction pointer. Correlation нужна именно потому, что эти языки сами по себе не совпадают. Нельзя просто сложить две таблицы рядом и назвать это пониманием. Нужно перевести статическую находку в runtime-вопрос, а runtime-событие вернуть обратно в статическую карту.

> В этой главе слово `evidence` удобно переводить как "свидетельство". Это не абсолютная истина, а наблюдаемый факт, который может поддерживать или ослаблять некоторую интерпретацию. Static evidence рождается из файла и metadata. Runtime evidence рождается из

выполнения target-а под наблюдением Windows dynamic collector.

Статическая находка как начало вопроса, а не конец расследования

Когда heuristic/evidence engine формирует finding, он не должен мыслить так: "я нашёл странность, значит дело закрыто". Finding - это начало аккуратного расследования. Внутри него есть observation, interpretation и conclusion. Observation - это то, что реально увидено: например, handler RVA указывает в writable section, или несколько функций используют один и тот же handler target, или часть `UNWIND_INFO` лежит в executable section. Interpretation - это попытка понять, что это может означать в контексте Windows x64 unwind-модели. Conclusion - это осторожный вывод: signal, anomaly, requires manual review, possible runtime-generated code, possible packed image, possible metadata corruption.

На этапе static analysis всё это ещё остаётся моделью файла. Даже если finding сильный, он не видел исполнения. Представим, что static analyzer обнаружил handler dominance. Термин `handler dominance` означает ситуацию, когда один handler target оказывается связан с необычно большим числом функций или unwind-records. Само по себе это не приговор. В нормальной программе runtime-библиотека, compiler support code или общий exception-handling механизм тоже могут создавать концентрацию handler-ов. Но если такой признак сочетается с writable handler placement, parse failures или executable unwind metadata, он начинает выглядеть как более интересный сигнал. На этом месте static analysis не должен кричать "вредоносность". Он должен сформулировать вопрос для следующего слоя: проявляется ли этот handler как живой маршрут выполнения?

Вот здесь начинается связь с dynamic validation. Статическая находка превращается в runtime-вопрос не сама по себе, а через смысл. Если finding связан с handler-ом, dynamic validation может попробовать увидеть handler hit. Handler hit - это не "handler существует в файле". Это событие, при котором выполнение реально дошло до адреса обработчика или до связанного callback-а в данном запуске. Если finding связан с placement-anomaly, dynamic validation может смотреть на dynamic function table activity: вызывает ли процесс `RtlAddFunctionTable` или `RtlInstallFunctionTableCallback`, появляются ли признаки runtime-кода, которого static analyzer не мог полностью восстановить из PE-файла. Если finding связан с deliberate exception route, dynamic validation может смотреть exception events и дальнейший `RIP trace`.

Важно, что один static finding не обязан иметь прямое dynamic-подтверждение в каждом запуске. Программа могла не пройти по нужной ветке. Нужный input мог отсутствовать. Защитный код мог активироваться только при определённом окружении. Dynamic collector мог остановиться по timeout. Поэтому отсутствие runtime event не отменяет static

evidence автоматически. Оно только говорит: "в этом сценарии мы не наблюдали проявления". Это одна из самых важных дисциплин анализа: не превращать ненаблюдение в доказательство отсутствия.

Runtime-событие как живой факт, но не вся истина

С другой стороны, runtime evidence тоже нельзя романтизировать. Когда Windows dynamic collector видит exception event, это действительно живое событие: поток остановился, Windows передала debugger-у структуру `DEBUG_EVENT`, внутри неё может быть `EXCEPTION_DEBUG_INFO`, где лежит `EXCEPTION_RECORD` с кодом исключения и адресом. Но даже такой факт не говорит автоматически, почему программа устроена именно так. Исключение могло быть обычной ошибкой, проверкой guard page, частью нормального crash-reporting, антиотладочным приёмом, переходом через handler или следствием того, что debugger сам изменил условия выполнения.

Поэтому runtime evidence нуждается в возвращении к статической модели. Если collector увидел first-chance exception, надо спросить: где находится `RIP` этого события? В каком module? Можно ли перевести runtime VA обратно в RVA? Есть ли для этого диапазона `RUNTIME_FUNCTION`? Какой `UNWIND_INFO` связан с этой функцией? Есть ли handler metadata? Если после handler-а `RIP` ушёл в другой диапазон, этот диапазон лежит в известной секции, в private executable memory или в newly mapped region? Без таких вопросов dynamic event остаётся просто строкой лога.

Термин `RIP trace` нужно понимать именно в этом духе. `RIP` - это instruction pointer, указатель на текущую инструкцию в x64. Trace - след, короткая последовательность наблюдений. `RIP trace` после handler hit не является полным графом исполнения программы. Это маленький след: куда управление пошло сразу после важного события. Его ценность в том, что он помогает отличить handler, который просто записал лог и позволил процессу упасть дальше, от handler-а, который действительно перенаправил выполнение в другой участок кода. Но этот след ограничен длиной, сценарием, timing-ом и instrumentation completeness. Он не заменяет статический анализ control-flow graph, а дополняет его живым наблюдением.

Вот почему проект разделяет static analysis, heuristic assessment и dynamic validation. Static analysis создаёт модель из файла. Heuristic assessment превращает структурные наблюдения в осторожные findings и confidence. Dynamic validation добавляет runtime evidence. А итоговый assessment должен собрать это в форму, которую человек может проверить, не путая уровни доказательства.

Смысл correlation в проекте

В обычной речи слово "корреляция" иногда звучит слишком математически, будто за ним обязательно стоит статистическая формула. В

нашем проекте correlation проще и инженернее. Это правило сопоставления: есть ли между static evidence и runtime evidence такая связь, которая делает case сильнее. Case здесь можно перевести как "случай анализа" или "собранное дело". Не уголовное дело, конечно, а инженерная папка наблюдений: какие признаки есть в файле, какие события произошли при запуске, насколько они согласуются, что остаётся неизвестным.

Проектная evidence matrix задаёт correlation консервативно. Это слово - `conservative`, консервативный - означает осторожный, не склонный делать сильный вывод из слабой связи. В текущей модели correlation считается meaningful не всегда, а только в ограниченных ситуациях. Первая ситуация: есть suspicious static handler-related evidence и одновременно runtime видит handler, VEH, VCH или UEF hits. Вторая ситуация: есть static placement anomalies и одновременно runtime видит dynamic function-table activity. Это не попытка построить полный per-function proof graph. Это coarse correlation layer, то есть грубый, верхнеуровневый слой сопоставления.

Здесь нужно разжевать два выражения. `Per-function proof graph` можно перевести как "доказательный граф по каждой функции". Такой граф связывал бы конкретную функцию, конкретную запись `RUNTIME_FUNCTION`, конкретный `UNWIND_INFO`, конкретное исключение, конкретный handler hit и конкретный маршрут после handler-а. Это было бы очень мощно, но дорого, сложно и далеко не всегда возможно в одном запуске. Проект прямо не делает вид, что такой полный граф уже построен. Он использует `coarse correlation layer` - грубый слой корреляции: не доказывает каждую дугу маршрута, а говорит, что два класса evidence поддерживают друг друга на уровне семейства признаков.

Такой подход хорош именно потому, что он честный. Например, static analysis нашёл handler dominance и writable handler target. Dynamic validation увидела, что в запуске действительно были handler hits или VEH hits. Это не доказывает, что каждый доминирующий handler из metadata сработал именно так, как мы думаем. Но это повышает case strength: статическая странность не осталась чисто бумажной, во время выполнения действительно наблюдалась активность вокруг обработчиков. В отчёте это должно выглядеть как усиление evidence, а не как окончательный приговор.

Другой пример: static analysis заметил, что часть unwind metadata лежит в необычной или writable/executable области. Dynamic validation увидела `RtlAddFunctionTable` или `RtlInstallFunctionTableCallback`. Это может означать, что процесс работает с runtime function tables, то есть добавляет unwind-карту во время выполнения. Такая связь важна, потому что объясняет, почему PE-файл на диске может быть неполной картой. Но даже тут нельзя делать скачок к "packer" или "malware" без дальнейшей проверки. JIT, runtime code generators, instrumentation frameworks и некоторые легитимные runtime-и тоже могут регистрировать function tables.

Assessment как место встречи, а не просто итоговая строка

Когда все эти наблюдения попадают в report, они не должны превращаться в кашу. Поэтому проекту нужен `assessment`. Слово `assessment` можно перевести как "оценка", но в нашем контексте это не школьная оценка и не один вердикт. Assessment - это слой, где разные evidence levels складываются в понятный человеку вывод: насколько есть статические признаки, насколько есть runtime-признаки, есть ли meaningful correlation, требует ли случай ручной проверки, какая case strength получается.

Здесь легко ошибиться и сделать красивый, но вредный интерфейс: вывести один большой score и заставить пользователя думать, что это процент опасности. Но evidence model проекта специально устроена иначе. Score - это инженерная сумма весов по статическим findings. Suspicion - нормализованное представление score. Confidence - отдельная оценка уверенности в интерпретации. Evidence level - грубая категория силы evidence: none, weak, medium, strong. Case strength - итоговая формулировка уровня случая: correlated или unconfirmed, weak или strong. Эти слова нужны не для красоты. Они защищают analyst-а от ложной уверенности.

Представь два случая. В первом static analysis нашёл сильные повреждения `UNWIND_INFO`, но dynamic validation не запускалась. Case может быть strong_unconfirmed_case: статически evidence сильное, но runtime-подтверждения нет. Во втором static analysis нашёл средний handler-related signal, а dynamic validation увидела handler hit. Case может стать medium_correlated_case: статическая и динамическая стороны встретились, но сила каждой стороны не обязательно максимальная. Эти два результата нельзя ранжировать простым "больше/меньше опасно". Они разные по природе: один сильный, но без runtime-корреляции; другой может быть слабее по статике, но проявился в живом запуске.

Assessment должен помогать реверсеру понять именно природу вывода. Он должен отвечать не только "насколько тревожно", но и "почему мы так считаем", "какие evidence families участвуют", "что было ослаблено weakening rules", "какие manual validation steps нужны дальше". И здесь снова нельзя писать сухо: если report показывает `unknown` или `weak`, это не провал инструмента. Иногда честное "мы не можем подтвердить" полезнее, чем громкий ложный verdict.

Неполная dynamic session и дисциплина молчания

Особенно важно объяснить incomplete session. Session - это сеанс динамической проверки: target запущен, collector получает события, ставит breakpoints, ждёт завершения или timeout. Incomplete session - неполный сеанс, в котором что-то помешало получить полноценную картину: timeout, аварийное завершение, отсутствие natural target exit, dropped events,

неполная instrumentation, невозможность поставить breakpoint на нужный адрес. В таком случае raw events не выбрасываются. Они всё ещё полезны. Но их нельзя использовать так, будто сеанс был полноценным.

Проектная evidence matrix поэтому ограничивает runtime evidence level для incomplete sessions. Если session неполная, runtime evidence не должна резко повышать final confidence. Это похоже на ситуацию в расследовании: свидетель видел часть события через окно, но не весь эпизод. Его слова важны, но нельзя строить на них полный вывод. Так же и dynamic validation: если она увидела first-chance exception до timeout, это факт. Но если target не дошёл до ветки, где должен был сработать handler, отсутствие handler hit не доказывает, что handler не используется.

Эта дисциплина молчания очень важна для качественного инструмента. Хороший static analyzer и dynamic collector должны уметь говорить не только "нашёл", но и "не видел", "не проверено", "сессия неполная", "нужен другой сценарий". Для пользователя это сначала может звучать менее эффектно, чем уверенный verdict, зато именно так строится инструмент, которому можно доверять. Он не придумывает evidence там, где его нет.

Сопоставление адресов как техническая основа correlation

До сих пор мы говорили о correlation концептуально, но за ней стоит очень земная адресная работа. Static analysis оперирует RVA и file offsets. Runtime events приходят с VA - виртуальными адресами внутри процесса. Чтобы связать их, нужно пройти мост: определить module, узнать его runtime base, вычесть base из VA и получить RVA. Только после этого runtime address можно сопоставить с `RUNTIME_FUNCTION`, секцией, handler RVA или finding location. Если этот мост сделать неаккуратно, correlation станет ложной.

Здесь нужно заново проговорить три адресных слова. `File offset` - смещение в файле на диске: сколько байтов от начала файла. `RVA` - relative virtual address, относительный виртуальный адрес внутри образа: смещение от image base после загрузки. `VA` - virtual address, настоящий адрес в виртуальном адресном пространстве процесса. В static report часто естественно хранить RVA, потому что он стабилен относительно image base. В dynamic report события приходят как VA, потому что процесс уже загружен в память и ASLR мог выбрать другой base. Correlation требует перевода между этими представлениями.

ASLR - Address Space Layout Randomization, рандомизация размещения адресного пространства - специально мешает предполагать, что модуль всегда загружен по одному и тому же адресу. Поэтому нельзя взять handler RVA из static analysis и напрямую поставить breakpoint по такому же числу в target-е. Нужно знать runtime base модуля, сложить base + RVA и только тогда получить runtime VA. Точно так же, когда debug event сообщает адрес

исключения, нужно найти, какому module этот VA принадлежит, и перевести его обратно в RVA. Без этой дисциплины можно легко "подтвердить" не тот handler, не ту функцию и не тот module.

В этом смысле module map - карта загруженных модулей - становится техническим сердцем dynamic correlation. Module map хранит, какие образы загружены в process address space, по каким base addresses, с какими ranges. Она нужна не как справочник имён DLL, а как переводчик между runtime-событием и статической моделью. Если `RIP` лежит внутри известного module range, мы можем говорить на языке RVA. Если он лежит в private executable memory, цепочка меняется: возможно, это generated code, unpacked payload, manually mapped image или runtime thunk. И тогда static analysis исходного PE уже не может быть единственным источником карты.

Четыре характерные сцены correlation

Чтобы эта модель не осталась абстрактной, полезно пройти несколько характерных сцен. Они не должны восприниматься как шаблон verdict-а. Это именно учебные ситуации, которые показывают, как static и runtime evidence начинают разговаривать друг с другом.

Сцена первая: handler-related static evidence получает runtime-эхо

Static analysis находит, что большой процент функций ведёт к одному handler target. Heuristic/evidence engine оформляет это как handler dominance. Одновременно есть дополнительный контекст: handler target лежит в необычной секции или связан с несколькими suspicious ranges. Пока это только статическая история. Она говорит: "в metadata есть концентрация вокруг обработчика". Затем dynamic validation запускает target и видит handler hit или VEH hit. Теперь runtime evidence не доказывает каждую деталь static finding-а, но показывает: handler-механика действительно активна в живом выполнении. Assessment может повысить case strength, потому что две разные природы evidence встретились в одной смысловой зоне.

Сцена вторая: placement anomaly и dynamic function-table activity

Static analysis видит metadata placement anomaly. Например, unwind metadata находится там, где мы не ожидали бы спокойные read-only структуры, или section properties выглядят необычно для `UNWIND_INFO`. Сама по себе placement anomaly может иметь benign explanation: нестандартный linker script, packed stub, runtime, compiler-specific layout. Затем Windows dynamic collector видит вызов `RtlAddFunctionTable`. Это уже runtime evidence того, что процесс работает с function tables во время выполнения. Correlation здесь не говорит: "это точно packer". Она говорит тоньше: "статическая странность размещения metadata согласуется с runtime-активностью вокруг dynamic function tables". Это сильнее, чем отдельная placement anomaly, но всё ещё требует manual review.

Сцена третья: сильная статика без динамического проявления

Static analysis находит invalid ranges, unwind parse failures и overlaps в `RUNTIME_FUNCTION`. Это может быть strong static evidence. Dynamic validation запускается, но нужные участки кода не исполняются, handler hits не наблюдаются, target быстро завершает работу или session неполная. Такой случай не должен искусственно ослаблять static findings до нуля. Он должен стать strong_unconfirmed_case или похожим выводом: статические признаки значимы, но runtime-корреляции нет. Аналитик понимает, что следующий шаг - не забыть finding, а подобрать другой input, другой сценарий запуска или вручную посмотреть problematic ranges в Ghidra.

Сцена четвертая: runtime activity без сильной статической опоры

Бывает и обратная ситуация. Dynamic validation видит first-chance exceptions, VEH registrations или даже dynamic function table calls, но static analysis исходного PE не показывает сильных нарушений. Это не значит, что static analyzer ошибся. Возможно, программа использует легитимный runtime, JIT-компонент, crash handling или библиотеку, которая регистрирует обработчики нормально. Assessment должен отразить runtime activity, но не раздувать static suspicion. В хорошей модели такая ситуация становится поводом для дальнейшего исследования, а не для автоматического high verdict.

Отчет как место, где нельзя смешивать слои

В финальном report-е очень легко испортить всю аккуратность предыдущих этапов. Если JSON или HTML просто покажет одну строку "correlated suspicious behavior", пользователь не поймёт, что именно с чем связано. Поэтому reports должны сохранять слои. Static findings должны оставаться static findings. Dynamic events должны оставаться dynamic events. Assessment должен объяснять, какая correlation была установлена и почему она ограничена.

JSON report особенно важен, потому что он не только читается человеком, но и может быть входом для других инструментов, тестов, validators, dashboards. Поэтому schema должна фиксировать структуру: input, report metadata, sections, exception directory, functions, heuristic analysis, dynamic validation, assessment. HTML report должен быть понятным человеку: не просто таблица кодов, а история evidence. Ghidra CSV должен помочь перенести статические findings и comments в reverse-engineering workflow. У каждого формата своя роль, но все они должны сохранять главную дисциплину: не смешивать observation и conclusion.

Это особенно важно для manual validation steps. Ручная проверка не является признанием слабости анализатора. Наоборот, хороший инструмент должен подсказывать analyst-у следующий разумный шаг. Если correlation есть, report должен показать, где её проверить: какой RVA, какой handler, какой

module, какой runtime address, какой event. Если correlation нет, report должен объяснить, что именно не было наблюдено и почему это не отменяет static evidence.

Внутренняя честность модели

Самое ценное в такой архитектуре - не наличие красивых терминов, а внутренняя честность. Static analyzer не притворяется, что видел runtime. Windows dynamic collector не притворяется, что восстановил весь CFG. Heuristic/evidence engine не притворяется статистическим malware classifier-ом. Assessment не притворяется приговором. Каждый слой говорит своим голосом, а correlation связывает их только там, где связь действительно имеет смысл.

Для reverse engineering это почти философия работы. Хороший реверсер не прыгает от одного факта к финальному выводу. Он строит цепочку: где факт наблюдался, на каком слое, какой механизм Windows объясняет этот факт, какие нормальные объяснения возможны, какие признаки усиливают подозрение, какие ограничения остаются. Проектный assessment должен помогать мыслить именно так. Тогда report становится не красной лампочкой, а учебной картой расследования.

Именно поэтому раздел о correlation важен после dynamic validation pipeline. Мы уже поняли, что static analysis даёт модель файла, а dynamic validation даёт runtime evidence. Теперь нужно научиться не смешивать эти вещи и не противопоставлять их. Они работают вместе, но не одинаково. Static analysis говорит: "вот что файл заявляет и какие структурные признаки в нём есть". Dynamic validation говорит: "вот что удалось наблюдать в конкретном запуске". Assessment говорит: "вот насколько эти свидетельства согласуются, насколько силён случай и что должен сделать человек дальше".

Маршрут одного finding через всю систему

Чтобы correlation не осталась красивым словом, полезно прожить путь одного finding почти как сцену из отладочного дневника. Пусть static analysis читает PE-файл и доходит до записи `RUNTIME_FUNCTION`. Сама запись говорит: для диапазона кода от такого-то RVA до такого-то RVA unwind-инструкция находится по такому-то `UnwindInfoAddress`. Анализатор переводит этот RVA через section map, безопасно читает байты, декодирует `UNWIND_INFO` и видит флаг обработчика. Затем он получает handler RVA. На этом месте ещё ничего не исполнялось. Это важная мысль: handler пока существует только как значение в metadata. Он ещё не стал событием.

Дальше heuristic/evidence engine смотрит не только на само число handler RVA, но и на окружение. В какой section находится этот адрес? Исполняемая ли она? Writable ли она? Повторяется ли этот handler у многих функций? Есть ли рядом parse failures, overlaps, strange placement или другие семьи evidence?

Если handler target лежит в non-executable section, это может стать finding семейства `metadata_placement`. Если один handler dominates слишком много записей, это может стать finding семейства `handler_dominance`. Но хороший finding не должен звучать как "плохо, потому что плохо". Он должен сохранить путь: какой RVA, какое семейство, какой affected_count, почему это имеет consequence для unwind/exception handling и почему confidence не является абсолютной уверенностью.

Теперь этот finding превращается в вопрос для dynamic validation. Не в приказ "докажи вредоносность", а именно в вопрос: будет ли в живом выполнении наблюдаться handler-related activity? Может ли Windows dynamic collector поставить breakpoint на соответствующий runtime VA? Попадёт ли туда выполнение? Будут ли видны VEH/VCH/UEF registrations или hits? Возникнет ли exception event, после которого `RIP` окажется в ожидаемом обработчике? Если да, static evidence получает runtime-эхо. Если нет, finding не исчезает, потому что программа могла не пройти нужный путь. Но assessment должен честно записать: static signal present, runtime manifestation not observed in this session.

Такая форма мышления резко отличается от конспектного "handler найден, handler hit есть, correlation true". В реальном анализе важны промежуточные мосты. Нужно перевести static RVA в runtime VA. Нужно учесть ASLR. Нужно проверить, что адрес принадлежит тому же module, а не другому модулю с похожим именем. Нужно понимать, что breakpoint hit мог быть вызван обычным входом в функцию, а не обязательно exception dispatch route, если адрес выбран слишком грубо. Нужно смотреть context: какой exception code, какой thread, какой call stack доступен, что произошло после продолжения. Только тогда finding начинает превращаться в полезное расследование.

В отчёте этот путь не должен исчезать. Пользователь не должен видеть одну строку "correlated" без основания. Ему нужно показать, что static side нашла конкретное место, dynamic side увидела конкретное событие, а assessment связал их именно по смыслу. Если связь слабая, это тоже надо показать. Если static finding относится к handler placement, а runtime event относится только к unrelated first-chance exception в другом module, это не correlation. Это два события рядом, но не одно доказательство.

Три уровня совпадения

Внутри correlation есть несколько уровней строгости. Если их не различать, анализатор начнёт либо слишком много обещать, либо слишком сильно молчать. Первый уровень можно назвать family-level correlation, сопоставление на уровне семейства признаков. Например, static analysis видит handler-related findings, а dynamic validation видит handler/VEH/VCH/UEF hits. Это не связывает конкретную функцию с конкретным hit-ом, но показывает, что обе стороны говорят об одном классе механики: обработчиках исключений.

Второй уровень - address-level correlation, сопоставление по адресу. Здесь runtime VA события переводится в RVA, затем проверяется, попадает ли он в диапазон или target, который static analysis уже описал. Такой уровень сильнее, потому что мы связываем не просто "обработчики вообще", а конкретное место. Но он требует аккуратной module map, правильного base address, учёта ASLR и понимания, что packed/manual-mapped code может жить вне обычного image range.

Третий уровень - route-level correlation, сопоставление маршрута. Это уже самая интересная и самая сложная форма: static analysis предполагает, что исключение может перевести управление через handler в некоторый участок, а runtime evidence показывает exception event, handler hit и post-handler `RIP trace`, согласующийся с этой гипотезой. Такой уровень ближе к настоящему пониманию control-flow, но и он редко бывает полным. Несколько шагов `RIP trace` не равны полной реконструкции CFG. Они лишь показывают ближайший живой след.

Текущая проектная модель, насколько её нужно описывать в книге на этом этапе, остаётся консервативной. Она не обещает полный route-level proof graph для каждой функции. Она больше похожа на аккуратный family/address-aware assessment: есть static evidence в определённых семьях, есть runtime evidence определённых типов, есть правила, когда их можно считать связанными. Это честный инженерный компромисс. Он не закрывает путь к более детальной корреляции в будущем, но не заставляет текущий инструмент обещать то, чего он не доказывает.

First-chance exception в контексте correlation

Отдельно нужно медленно разобрать first-chance exception, потому что этот термин легко превращается в шум. First chance - "первый шанс" - означает, что исключение произошло в процессе, который находится под debugger-ом, и Windows сначала сообщает об этом debugger-у до того, как обычный механизм поиска обработчика окончательно решит судьбу исключения. В этот момент debugger получает возможность посмотреть событие, но само наличие first-chance exception ещё не означает crash. Многие программы нормально переживают first-chance exceptions. Некоторые используют их как часть runtime-логики. Некоторые получают их из библиотек, из guard pages, из breakpoint-ов или из deliberately raised exceptions.

Для dynamic validation first-chance event ценен как сигнал: вот точка, где живой процесс вошёл в exception machinery. Но correlation требует большего вопроса: связано ли это исключение со статическими findings? Если static analysis говорил о handler dominance в одном module, а first-chance exception возник в совершенно другом module и не привёл к handler hit-у, связь слабая. Если exception address лежит в диапазоне функции, где static analysis нашёл `EHANDLER`, а после продолжения виден handler hit, связь становится сильнее. Если после handler-а `RIP` меняет маршрут необычным образом, появляется

ещё более интересная runtime-гипотеза.

Second-chance exception - "второй шанс" - имеет другой смысл. Это ситуация, когда обычная обработка не справилась, и debugger получает событие уже перед тем, как процесс будет считаться не обработавшим исключение. Second chance чаще ближе к crash-path, но и здесь нельзя механически делать вывод без контекста. Для assessment важно, first chance это или second chance, потому что первый чаще говорит "исключение возникло, но ещё может быть обработано", а второй чаще говорит "обработчики не помогли". Это разные runtime evidence levels и разные manual validation questions.

Handler hit как событие, требующее контекста

Слово handler hit звучит просто: breakpoint на обработчике сработал. Но для хорошей книги этого мало. Handler - обработчик - в этой теме может быть разного рода. Это может быть language-specific handler, например `__C_specific_handler`, который является runtime-механизмом компилятора, а не пользовательским блоком `__except`. Это может быть vectored exception handler, зарегистрированный через `AddVectoredExceptionHandler`. Это может быть vectored continue handler, top-level unhandled filter или callback, связанный с crash-reporting. Поэтому handler hit без роли handler-а ещё не объясняет поведение.

Когда dynamic validation фиксирует hit, report должен помогать понять, какой именно слой затронут. Если hit пришёлся на language-specific handler, это может означать, что Windows дошла до frame-based exception logic для функции. Если hit пришёлся на VEH callback, это другой слой маршрута исключения, более ранний относительно frame-based handler search. Если hit пришёлся на UEF, это уже ближе к unhandled/top-level фазе. Смешивать эти слои нельзя. Они могут выглядеть похожими как "адрес обработчика", но в маршруте Windows они находятся в разных местах и имеют разные последствия.

Для correlation это означает, что static handler-related evidence и runtime handler hit должны быть сопоставлены не только по слову handler, но и по роли. Если static analysis нашёл `UNW_FLAG_EHANDLER` и handler RVA указывает на `__C_specific_handler`, а dynamic validation видит VEH hit, это может быть важным runtime evidence, но это не прямое подтверждение именно frame-based handler metadata. Оно говорит об активности exception handling вообще, но связь слабее. Если же runtime hit соответствует тому же handler path, confidence может расти сильнее.

Dynamic function tables как мост между файлом и памятью

Второе важное семейство correlation связано с dynamic function tables. На Windows x64 unwind-модель опирается на function tables: для nonleaf-кода системе нужны сведения, как восстановить caller context. В обычном PE эти сведения приходят через Exception Directory, `.pdata` и `.xdata`. Но runtime-generated code, JIT, unpacked payload или manually mapped code могут появиться уже после загрузки файла. Тогда PE на диске не содержит полной карты. Чтобы Windows всё равно могла делать stack walking и exception handling для такого кода, runtime может зарегистрировать function table через специальные API.

Когда static analysis видит странное placement metadata или неполную карту, это может иметь разные объяснения. Одно из них - код и unwind-информация частично создаются во время выполнения. Но static analysis не может сам увидеть будущий вызов `RtlAddFunctionTable`. Он может только сформулировать гипотезу: static map may be incomplete, runtime function-table activity would be relevant. Когда Windows dynamic collector действительно наблюдает такой вызов, появляется runtime evidence, которое согласуется с этой гипотезой.

И снова важно не перепрыгивать. Вызов `RtlAddFunctionTable` сам по себе не означает злонамеренность. Он может быть нормальным для JIT/runtime. Но если он появляется вместе со static placement anomalies, private executable memory, handler-related activity или packed-image signs, assessment становится богаче. Мы уже не говорим "файл странный" отдельно и "в runtime что-то было" отдельно. Мы говорим: static map выглядит неполной или необычной, а runtime действительно регистрирует дополнительные unwind structures. Это хорошая инженерная correlation.

Ошибки, которые ломают correlation

Самая частая ошибка - смешать адресные пространства. Аналитик видит RVA в static report и VA в runtime event и решает, что одинаковые последние цифры уже означают совпадение. Это опасно. ASLR, разные modules, forwarded exports, relocated images и private memory могут легко сделать такое совпадение случайным. Правильный путь всегда один: runtime VA -> module range -> module base -> RVA -> static model. Если module не найден, это тоже важный факт, а не повод насильно подогнать адрес.

Вторая ошибка - считать любое runtime-событие подтверждением любого static finding. Например, static analysis нашёл overlaps в `.pdata`, а dynamic validation увидела обычный first-chance breakpoint exception от debugger-а. Это не meaningful correlation. Есть static evidence и есть runtime event, но между ними нет смыслового моста. Assessment должен быть устойчив к такому соблазну.

Третья ошибка - забывать про сценарий запуска. Dynamic validation видит только тот путь, по которому target прошёл в данном запуске. Если protected

code активируется после ввода license key, после сетевого ответа или после определённого input-а, простой smoke run может ничего не показать. Поэтому отсутствие handler hit-а или dynamic table call-а не должно автоматически снижать static evidence до нуля. Оно должно попадать в report как ограничение: not observed in this run.

Четвёртая ошибка - превращать confidence в статистическую вероятность. Confidence в проекте rule-based. Это инженерная оценка полноты и взаимного подтверждения evidence, а не calibrated probability. Если confidence выросла после correlation, это значит, что evidence стало согласованнее по правилам модели. Это не значит, что вероятность malware стала, например, 80%. Такой язык надо запрещать в книге и в отчёте.

Как хороший report должен учить

Если цель проекта - не просто выдать JSON, а помочь reverse engineer-у, report должен быть написан как продолжение этой главы. Он должен показывать слои. Сначала input и metadata: какой файл, какой hash, какая архитектура, какая версия schema. Затем PE facts: sections, image base, exception directory. Затем functions and unwind summaries. Потом heuristic analysis: findings, families, score, confidence, weakening rules. Затем dynamic validation: session status, events, hits, registrations, dynamic table activity, RIP traces. И только после этого assessment, который объясняет, как static and runtime evidence согласованы.

Такой report не только информирует, но и обучает. Пользователь видит не магическую оценку, а путь мышления. Он может открыть Ghidra, перейти к representative RVA, посмотреть section permissions, сверить handler target, запустить target с другим input-ом, проверить whether handler hit repeats. Это и есть здоровая роль инструмента: не заменить реверсера, а дать ему карту свидетельств и следующий разумный шаг.

В финальной архитектурной главе книги это нужно будет связать с реальными модулями проекта. `pe.c` даёт section/RVA foundation. `unwind.c` даёт decoded unwind model. `heuristics.c` создаёт findings и static assessment. `dynamic_win.c` собирает runtime evidence. `report.c` сохраняет всё так, чтобы человек и инструменты могли продолжить работу. Но сейчас важнее не список файлов, а принцип: каждый слой добавляет свой вид свидетельства и не должен выдавать его за другой вид.

Короткое закрепление

В этой главе мы не добавили новый низкоуровневый механизм Windows вроде ещё одного флага `UNWIND_INFO`. Вместо этого мы собрали дисциплину мышления. Static evidence и runtime evidence нельзя смешивать, но их нельзя и держать в разных мирах. Correlation - это осторожная встреча этих миров. Она не доказывает всё, не отменяет manual review и не превращает score в

вероятность. Она помогает понять, когда статическая находка получила runtime-эхо, а когда осталась сильной, но неподтверждённой гипотезой.

Для проекта это означает простую, но строгую архитектурную линию. Static analysis и heuristic/evidence engine строят findings и static assessment. Windows dynamic collector добавляет DynamicReport и runtime evidence. Assessment соединяет их через evidence level, case strength, weakening rules и manual validation steps. Хороший report должен показывать не только итог, но и путь, по которому этот итог получился.

9.5. Reports и эвристики: как читать отчёт EH/Unwind Analyzer без магии и самообмана

Предыдущие разделы подвели нас к важной мысли: анализатор не должен просто сказать человеку: "подозрительно". Такая фраза почти ничего не стоит, если рядом нет дороги, по которой можно вернуться от вывода к фактам. Хороший отчёт должен быть устроен как карта расследования. Он должен показывать, какие байты были прочитаны, какие структуры PE были восстановлены, какие unwind-записи удалось декодировать, какие эвристики сработали, какие runtime-события были замечены во время dynamic validation и каким образом всё это превратилось в итоговую assessment. Если человек не может по отчёту повторить рассуждение анализатора, значит отчёт красивый, но бесполезный.

В проекте EH/Unwind Analyzer отчёт не является последним декоративным шагом. Это не "напечатали результат" и не "сохранили лог". Отчёт является boundary, границей между внутренним миром C-кода и внешним миром реверсера, backend-а, Ghidra, тестов и будущих сравнений. Внутри программы существуют структуры `PeImage`, массив `RuntimeFunction[]`, массив `UnwindSummary[]`, флаги успешности разбора `us_ok[]`, `HeuristicReport`, `DynamicReport` и `ReportMetadata`. Пока они живут в памяти процесса анализатора, ими удобно пользоваться коду. Но человек их не видит. Другие инструменты тоже не видят. Поэтому reporting layer превращает внутренние модели в устойчивые внешние представления: console summary, CSV, Ghidra CSV, JSON и HTML.

Слово `report` здесь лучше переводить не просто как "отчёт", а как "зафиксированное объяснение результата". Отчёт отвечает не только на вопрос "что нашлось", но и на вопросы "где это нашлось", "какой слой анализа это породил", "насколько этому выводу можно доверять", "какие нормальные объяснения ещё возможны" и "что нужно проверить вручную". Именно поэтому в проекте рядом существуют `findings`, `weakening_rules`, `assessment`, `manual_validation_steps` и отдельный блок `dynamic_validation`. Каждый из этих блоков нужен не для красоты, а для того, чтобы не смешивать факт, интерпретацию и итоговый вывод.

> Главное правило чтения отчёта: не начинать с финального verdict. Сначала надо понять input и metadata, затем PE layout, затем function/unwind records, затем heuristic findings, затем dynamic validation, и только после этого смотреть final assessment. Итоговый verdict без evidence path превращается в суеверие.

Отчёт как serialization boundary

Внутренние структуры анализатора живут в оперативной памяти. Там адреса являются числами, указатели указывают на выделенные буферы, строки могут быть временными, массивы имеют capacity, а ошибки памяти должны останавливаться до записи результата. Когда всё это превращается в JSON, HTML или CSV, происходит serialization. Слово `serialization` переводится как "сериализация": превращение внутренней структуры данных в последовательность байтов, которую можно записать в файл, передать другому процессу или сохранить в базе. Это важная граница доверия. Если на этой границе ошибиться, можно не просто некрасиво вывести текст, а исказить анализ.

Например, если `representative_rva` будет напечатан как десятичное число без указания, что это RVA, человек может принять его за file offset. Если 64-битный runtime address записать как обычный JSON number, JavaScript consumer может потерять точность. Если строка из входного path попадёт в HTML без escaping, отчёт станет потенциальной HTML-инъекцией. Если генерация упадёт посередине, но файл уже будет лежать на диске как будто полный, backend может сохранить повреждённый отчёт как достоверный artifact. Поэтому `report.c` в проекте делает не только форматирование. Он проверяет metadata, проверяет bounds модели, экранирует строки, строит JSON/HTML/CSV в буфере и пишет файл атомарно.

Атомарная запись - atomic write - означает, что отчёт сначала формируется как временный файл, а затем переименовывается в итоговый путь. Для пользователя это выглядит как мелочь, но для инструмента анализа это принципиально. Если analyzer упал, закончилась память или прервалась запись, consumer не должен получить половину JSON и принять её за полноценный результат. Хороший отчёт должен быть либо старым полным, либо новым полным. Полуотчёт - это не отчёт.

Пять форм вывода и почему они не заменяют друг друга

Проект выдаёт несколько видов отчётов не потому, что автору захотелось разнообразия. Разные consumer-ы читают разные формы. Console summary нужен человеку в терминале. Обычный CSV удобен для таблиц и быстрой фильтрации. Ghidra CSV нужен для переноса найденных диапазонов и комментариев в рабочую среду reverse engineering. JSON является machine-readable contract, то есть машинно-читаемым договором между analyzer-ом и внешним кодом. HTML нужен как self-contained dashboard, файл, который можно открыть глазами и пройти по результатам без дополнительного frontend-а.

Console output намеренно краток. Он показывает input path, machine, `ImageBase`, `SizeOfImage`, размер headers, количество sections, Exception Directory и количество `RUNTIME_FUNCTION` entries. Такой вывод хорош для быстрой проверки: файл распознан, это AMD64, Exception Directory найден, entries столько-то. Но console text не является стабильным API. Его нельзя

парсить backend-ом как контракт. Он для человека, а не для автоматической интеграции.

Обычный CSV делает одну строку на одну runtime-function entry. Здесь появляются `begin_rva`, `end_rva`, `size`, `unwind_rva`, версия unwind, flags, размер пролога, fixed stack, frame register, frame offset, handler RVA, признак chain, anomalies и parse error. CSV плоский: в нём нет вложенных объектов, нет null-семантики как в JSON, нет удобного разделения dynamic events и assessment. Но именно эта плоскость делает его удобным для spreadsheet-а, сортировки и быстрых фильтров.

Ghidra CSV устроен иначе. В нём всего четыре столбца: `begin_rva`, `end_rva`, `name`, `comment`. Имя строится в стиле `sub_%08X`, а comment превращает unwind-сводку в читаемую подсказку: версия, flags, prolog, stack bytes, frame register, saved GP/XMM registers, handler, chain, anomalies. Такой файл не предназначен для полной аналитики. Его задача - перенести карту функций и смысловые комментарии в Ghidra, чтобы реверсер увидел рядом с дизассемблером не голые адреса, а результат decoding-а.

JSON - главный машинный формат. Он сохраняет структуру анализа: metadata, PE layout, sections, function records, heuristic analysis, dynamic validation и final assessment. JSON важен тем, что у него есть schema: `docs/report.schema.json`. Schema - это формальное описание того, какие поля обязаны быть в отчёте, какие типы у этих полей и какие значения допустимы. Если backend, тесты или будущий UI читают отчёт, они должны ориентироваться именно на schema, а не на внешний вид HTML.

HTML report - это уже не контракт для программы, а интерфейс понимания для человека. Он показывает dashboard, PE image, structural validation, sections, final assessment, manual validation plan, static findings, weakening rules, dynamic validation, runtime log и таблицу runtime functions с фильтром. Важная деталь: HTML self-contained и имеет Content Security Policy. Это значит, что отчёт не тянет картинки, скрипты или стили из сети. Строки экранируются. Если имя файла содержало символы вроде `<script>`, они должны попасть в отчёт как текст, а не как исполняемый HTML.

ReportMetadata: provenance, или откуда взялся этот результат

Первый слой отчёта - provenance. Английское `provenance` переводится как "происхождение". В анализе бинарников происхождение результата так же важно, как сам результат. Представь два отчёта по файлу с одинаковым именем. Один сделан старой версией analyzer-а, другой новой. Один с `--strict-abi`, другой без. Один с chain following, другой без. Один по исходному файлу, другой по слегка изменённому. Если отчёт не фиксирует tool version, schema version, input hash, command line и build information, потом невозможно честно сравнить результаты.

В report_metadata находятся schema_version и tool_version. Schema version говорит consumer-у, какой формы JSON он читает. Tool version говорит, какая версия EH/Unwind Analyzer создала этот файл. Они похожи, но не одно и то же. Можно исправить внутренний алгоритм, не меняя JSON schema. И наоборот, можно изменить контракт отчёта, даже если многие алгоритмы остались прежними. Поэтому эти версии нельзя смешивать.

Поле generated_at_utc показывает время генерации отчёта в UTC. UTC нужен, чтобы не спорить о локальных часовых поясах. input_sha256 и input_size идентифицируют входные bytes. SHA-256 - криптографический hash, отпечаток файла. Он нужен, чтобы понимать: этот отчёт действительно относится к этим байтам, а не просто к файлу с похожим именем. command_line фиксирует режим запуска analyzer-a. build_compiler, build_platform и build_mode помогают понять, каким build-ом инструмента был создан отчёт.

Отдельно важны опции strict_abi, follow_chain и max_chain_depth. Strict ABI включает более формальные диагностики Microsoft x64 unwind ABI. Но проект специально не превращает эти diagnostics в suspicion score сами по себе, потому что совместимые production toolchains могут давать записи, которые формально выглядят спорно, но практически допустимы. follow_chain и max_chain_depth описывают, как analyzer обращался с chained unwind info. Если chain following выключен или depth ограничен, отчёт должен это сохранять, иначе consumer может переоценить полноту анализа.

PE image и sections: отчёт сначала объясняет карту адресов

После metadata отчёт показывает базовую модель PE image: image_base, size_of_image, size_of_headers, section_alignment, file_alignment и exception_directory. Эти поля кажутся скучными только до первого серьёзного разбора. Вся книга держится на различении file offset, RVA и VA. Если этот фундамент потерять, дальше невозможно понять ни functions, ни findings, ни runtime correlation.

Поле image_base - это предпочитаемая база загрузки образа, то есть адрес, относительно которого проектируется VA-мир PE-файла. Но в реальном запуске ASLR может дать другой runtime base. Поэтому JSON хранит image_base как статическое свойство файла, а dynamic validation отдельно работает с module map. size_of_image задаёт границу образа в памяти. Если RUNTIME_FUNCTION или UnwindInfoRVA выходят за эту границу, это уже сильный structural signal. exception_directory хранит RVA и size корневой directory, из которой static analysis получает массив RUNTIME_FUNCTION.

Блок sections показывает каждую PE section: имя, virtual address, virtual size, raw offset, raw size, original characteristics и удобные booleans readable, writable, executable. Имя section - это подсказка, но не закон. Права section

важнее имени, потому что права ближе к тому, как bytes будут восприниматься после загрузки. Если unwind metadata лежит в section с execute permission, это один смысл. Если handler target лежит в writable section, это другой смысл. Поэтому report сохраняет и сырые flags, и расшифрованные permissions.

pe_validation: структурная проверка до эвристик

Блок `pe_validation` важен потому, что он появляется раньше heuristic score. Это parser-level facts, факты уровня PE parser-а. Они ещё не являются финальным подозрением. Они отвечают на более базовый вопрос: можно ли вообще доверять карте файла и образа настолько, чтобы читать из неё Exception Directory, sections и unwind metadata.

`section_raw_oob` считает sections, у которых raw range выходит за фактический размер файла. Raw range - это диапазон bytes на диске. Если section говорит: "мои bytes начинаются здесь и имеют такой размер", а файла физически не хватает, analyzer не может честно читать полный диапазон. Для EH/unwind это важно потому, что .pdata или .xdata могут формально находиться в такой section, но нужных bytes реально нет.

`section_virtual_oob` похож, но относится к virtual layout: section выходит за `SizeOfImage`. Здесь ломается не доступность bytes на диске, а образ в памяти. Поскольку `BeginAddress`, `EndAddress`, `UnwindInfoRVA` и handler RVA выражены в RVA, ошибка virtual layout разрушает основу всех последующих проверок.

`section_raw_overlaps` и `section_virtual_overlaps` считают перекрытия. Raw overlap означает, что одни и те же file bytes объявлены частью двух sections. Virtual overlap означает, что один RVA может принадлежать нескольким sections. Для reverse engineering это опасно не только само по себе, а из-за неоднозначности: один tool может считать bytes данными, другой кодом; один parser может дать одни permissions, другой другие. Virtual overlap особенно критичен для EH/unwind, потому что вся unwind-модель работает через RVA.

`section_misaligned` показывает нарушение alignment policy: raw offsets и virtual addresses не согласованы с `FileAlignment` и `SectionAlignment`. Это не всегда катастрофа, но это сигнал, что PE layout нестандартен. `exception_directory_oob` говорит уже совсем прямо: корневая Exception Directory указывает за доступные границы. Тогда static analyzer не может безопасно получить полный массив `RUNTIME_FUNCTION`.

> Важно: поля `pe_validation` не надо читать как "malware". Они говорят о целостности модели PE. Иногда это повреждение файла, иногда memory dump, иногда custom linker, иногда packer, иногда просто объект вне области допустимого input contract. Но без этого слоя любые выводы по .pdata/.xdata становятся шаткими.

functions: где отчёт хранит карту RUNTIME_FUNCTION и UNWIND_INFO

Массив `functions` - это место, где отчёт показывает результат основного статического разбора unwind-механики. Каждая запись соответствует одной записи `RUNTIME_FUNCTION`, то есть одному диапазону кода, для которого Windows x64 может искать unwind-информацию. Здесь нельзя думать "одна запись = исходная C-функция". Правильнее: одна запись - это function table entry, диапазон машинного кода, который участвует в unwind-контракте.

Поля `begin_rva` и `end_rva` задают half-open interval: начало включается, конец не включается. Именно так удобно проверять попадание `ControlPc` или `RIP` в диапазон. `size` - это разница между end и begin. `begin_section` показывает, в какой section находится начало диапазона. Это помогает понять, действительно ли диапазон похож на кодовую область, например .text, или указывает куда-то странно.

`unwind_rva` - адрес unwind metadata относительно image base. Он ведёт не к handler-у и не к коду обработчика, а к структуре `UNWIND_INFO`. `unwind_section` показывает, где эта metadata размещена. Если всё обычно, она находится в .xdata, .rdata или похожей read-only data section. Если metadata в executable или writable section, это не автоматический приговор, но это уже материал для эвристики placement.

`unwind_ok` говорит, удалось ли разобрать `UNWIND_INFO`. Если false, отчёт всё равно сохраняет запись функции, потому что сама `RUNTIME_FUNCTION` entry существует, но decoded model неполная. `unwind_error` объясняет, где парсер остановился: truncation, bad opcode, bad operand, loop, invalid chain или другое нарушение. Это важный момент: отчёт не должен скрывать поломанную запись только потому, что её нельзя красиво декодировать. Неполный объект тоже является evidence.

Внутренний объект `unwind` показывает decoded summary. `version` - версия структуры `UNWIND_INFO`. `flags` - битовое поле, где могут быть признаки handler-а или chain. `prolog_size` говорит, сколько байтов начала функции считаются prolog-ом. `count_codes` - количество slots в массиве unwind codes, а не количество инструкций. `fixed_stack_bytes` - оценка фиксированного stack allocation, который decoder смог вывести из unwind operations. `frame_reg` и `frame_off` описывают frame pointer model, если она есть.

Поля `has_handler`, `handler_rva` и `handler_section` надо читать особенно аккуратно. Handler RVA - это не обязательно пользовательский `__except` block. На x64 он часто ведёт к language-specific handler, например к runtime-функции, которая уже читает language-specific data и решает, где настоящий filter/handler. Поэтому report показывает target, но человек должен понимать слой: это handler entry point в unwind metadata, а не обязательно весь смысл обработки исключения.

has_chain и chain относятся к chained unwind info. Chain означает, что текущая запись может ссылаться на другой RUNTIME_FUNCTION-подобный triplet. Это используется для особых случаев описания функции, например fragment/chained layouts. Отчёт показывает chain, потому что без него невозможно понять, где заканчивается одна unwind-модель и начинается переиспользование другой. anomalies хранит numeric bitmask, а anomalies_text превращает bits в compact labels вроде trunc, op, arg, chain_loop.

Поле comment - это не источник истины, а human-readable summary. Оно собирает версию, flags, prolog size, stack bytes, frame register, saved GP/XMM registers, handler, chain и anomalies в одну строку. Такой comment особенно полезен для Ghidra CSV, потому что реверсер получает рядом с функцией маленькую памятку: какой stack frame заявлен, какие nonvolatile registers сохраняются, есть ли handler, есть ли anomalies.

heuristic_analysis: где сырые факты становятся инженерными признаками

Теперь мы доходим до блока, который чаще всего читают неправильно. heuristic_analysis не является malware classifier-ом. Он не говорит: "вероятность вредоносности 73%". Он делает другое: собирает структурные и контекстные признаки, связанные с PE/unwind-моделью, группирует их по смысловым семействам, назначает score, confidence, evidence level, finding-и и weakening rules. Это heuristic/evidence engine - эвристический движок свидетельств. Эвристический значит rule-based, основанный на инженерных правилах и порогах, а не на математически доказанной вероятностной модели.

Поле score лежит в диапазоне от 0 до 100 и показывает suspiciousness score: насколько много и насколько сильных статических признаков накопилось. suspicion - это просто нормализованный score, то есть score / 100. Это не вероятность. Если suspicion равен 0.40, это не означает "40% malware". Это означает, что по текущей policy сработали признаки примерно на 40 баллов.

confidence - другое поле. Его нельзя смешивать со score. Confidence показывает уверенность в интерпретации статического verdict-а. Возможна ситуация, где score высокий, но confidence ограничена из-за малого sample size или одиночного семейства evidence. Возможна и обратная ситуация: score умеренный, но несколько независимых families дают более уверенную картину. Поэтому отчёт хранит и score, и confidence. Один отвечает на вопрос "насколько сильны признаки", другой - "насколько уверенно мы можем читать этот набор признаков".

verdict в static block принимает значения вроде no_strong_static_anomalies, low_suspicion, moderate_suspicion,

high_suspicion. Это static verdict, то есть verdict только по статическому heuristic analysis. Он ещё не учитывает dynamic correlation так, как итоговый assessment. Поэтому нельзя вырывать static verdict из контекста.

requires_manual_review означает, что отчёт сам просит человека посмотреть руками. Это поле становится true не только при высоком score. Например, writable unwind metadata, non-executable handler, parse failures или применённые weakening rules тоже требуют ручной проверки. И это правильно: иногда низкий score с необычной структурой важнее для реверсера, чем средний score без ясной точки проверки.

evidence_level обобщает силу статических свидетельств как none, weak, medium или strong. Он считается из количества strong/medium findings и score. Это не замена score, а более человеческая градация. Поля weak_finding_count, medium_finding_count, strong_finding_count показывают, сколько findings попало в каждую severity-группу.

Evidence families: почему признаки сгруппированы по слоям

Finding-и в проекте не лежат в одной куче. Они разбиты на evidence families - семейства свидетельств. Family отвечает на вопрос: "на каком смысловом слое возник сигнал?" Это важно, потому что два finding-а из разных families обычно сильнее, чем два finding-а из одной и той же причины. Если все признаки говорят только о placement, confidence может быть снижена. Если одновременно есть table integrity, unwind integrity и metadata placement, картина становится богаче.

table_integrity - семейство целостности таблиц и layout-а. Сюда входят повреждения PE sections, Exception Directory, RUNTIME_FUNCTION ranges, overlaps, out-of-bounds и сортировка. Это слой вопроса: можно ли доверять самой карте функций и image ranges?

unwind_integrity - семейство целостности UNWIND_INFO. Здесь проблема уже не столько в диапазоне функции, сколько в том, можно ли декодировать unwind metadata и не нарушает ли она базовые правила структуры. Если здесь ошибка, Windows или analyzer может неверно восстановить caller context.

metadata_placement - семейство размещения metadata и handler targets. Оно смотрит, где лежит UNWIND_INFO и куда указывает handler RVA: в обычные data sections, executable sections, writable sections, non-executable sections. Это не только вопрос "адрес валиден", но и вопрос доверия к окружению адреса.

unwind_reuse - семейство переиспользования unwind metadata. Оно отвечает на вопрос: не слишком ли много функций ссылаются на один и тот же UnwindInfoRVA. Сам reuse бывает нормальным, но слишком централизованный pattern может быть признаком dispatcher-style metadata или защитного runtime.

`handler_dominance` - семейство доминирования handler target. Оно смотрит, не обслуживает ли один handler слишком большую долю функций с handler metadata. Это особенно тонкое семейство, потому что runtime handlers вроде `__C_specific_handler` законно могут быть общими. Поэтому finding по handler dominance требует контекста и часто manual review.

Все 27 finding codes: что именно означает каждый сигнал

В текущей версии heuristic/evidence engine публично выдаёт 27 finding codes. Это не 27 атак и не 27 диагнозов. Это 27 именованных способов сказать: "вот здесь нарушен инвариант, замечен unusual pattern или несколько признаков усиливают друг друга". Один finding может агрегировать много однотипных мест. Поэтому `affected_count` важен: он показывает масштаб явления, а `representative_rva` даёт одну точку, с которой удобно начать ручную проверку.

pe_section_raw_oob

`pe_section_raw_oob` относится к family `table_integrity` и имеет severity `high`. Section raw range выходит за пределы файла. Analyzer видит, что section header обещает bytes, которых физически нет во входном buffer-е. Для unwind-разбора это опасно потому, что .pdata или .xdata могут формально находиться в такой section, но реальные bytes отсутствуют. Обычно ручная проверка начинается с сравнения PointerToRawData + SizeOfRawData с фактическим размером файла.

pe_section_virtual_oob

`pe_section_virtual_oob` относится к family `table_integrity` и имеет severity `high`. Virtual range section выходит за SizeOfImage. Здесь ломается модель image в памяти: RVA в хвосте такой section нельзя уверенно считать принадлежащим загружаемому образу. Для EH/unwind это важно, потому что все ключевые адреса - BeginAddress, EndAddress, UnwindInfoRVA, HandlerRVA - являются image-relative.

pe_section_raw_overlap

`pe_section_raw_overlap` относится к family `table_integrity` и имеет severity `medium`. Две sections перекрываются по file bytes. Это создаёт неоднозначность: одни и те же bytes файла можно интерпретировать как часть разных sections. Не каждое raw-перекрытие связано с unwind metadata, но оно предупреждает о parser differential: разные tools могут построить разную картину файла.

pe_section_virtual_overlap

`pe_section_virtual_overlap` относится к family `table_integrity` и имеет severity `high`. Две sections перекрываются по RVA-пространству. Для unwind это

сильнее raw overlap, потому что runtime-function entries и handler targets живут именно в RVA-модели. Если один RVA имеет две section identities, permissions и section name становятся неоднозначными.

pe_section_misaligned

`pe_section_misaligned` относится к family `table_integrity` и имеет severity `medium`. Section layout не согласован с FileAlignment или SectionAlignment. Это общий PE-layout signal: файл может быть handcrafted, custom-linked, damaged или intentionally weird. Сам по себе этот finding не доказывает поломку .pdata, но снижает доверие к аккуратности PE-контракта.

pe_exception_directory_oob

`pe_exception_directory_oob` относится к family `table_integrity` и имеет severity `high`. Exception Directory указывает за доступные границы image/file mapping. Это корневая проблема для всего EH/unwind pipeline: если directory недоступна, analyzer не может безопасно прочитать полный массив RUNTIME_FUNCTION.

no_runtime_functions

`no_runtime_functions` относится к family `table_integrity` и имеет severity `medium`. В image есть executable content, но RUNTIME_FUNCTION entries отсутствуют. Это не всегда ошибка: leaf-only image теоретически может обходиться без .pdata. Но для обычного крупного x64 executable полное отсутствие function table требует объяснения: надо смотреть call, sub rsp, сохранение nonvolatile registers и другие признаки nonleaf-кода.

invalid_ranges

`invalid_ranges` относится к family `table_integrity` и имеет severity `medium/high`. У части RUNTIME_FUNCTION entries EndAddress не больше BeginAddress. Диапазон функции получается пустым или направленным назад. Такой record не может нормально описывать участок кода. Severity становится high, если affected ratio выходит за порог, потому что массовость превращает единичную битость в системную проблему table integrity.

rf_oob

`rf_oob` относится к family `table_integrity` и имеет severity `medium/high`. BeginAddress или EndAddress выходит за SizeOfImage. Function table заявляет код вне образа. Это может быть повреждение, ошибочный RVA/VA mixup, packer/manual mapping side effect или мусор в .pdata. Ручная проверка должна начать с SizeOfImage, section boundaries и исходных 12 bytes record-a.

unwind_rva_oob

`unwind_rva_oob` относится к family `table_integrity` и имеет severity `medium/high`. UnwindInfoAddress равен нулю или выходит за SizeOfImage.

Function range может выглядеть нормальным, но указатель на инструкцию раскрутки кадра недоступен. Это опасно: Windows при исключении или stack walk ожидает найти UNWIND_INFO, а запись ведёт в никуда.

rf_unsorted

`rf_unsorted` относится к family `table_integrity` и имеет severity `medium`. RUNTIME_FUNCTION entries идут не по возрастанию BeginAddress. На x64 function table обычно ожидается отсортированной для эффективного поиска. Нарушение сортировки может ломать бинарный поиск или показывать, что таблица сгенерирована/искажена нестандартным способом.

rf_overlaps

`rf_overlaps` относится к family `table_integrity` и имеет severity `high`. Function ranges пересекаются. Один и тот же RIP может попасть в несколько entries. Это уже не просто косметика: unwind lookup должен выбрать одну запись, а overlap создаёт неоднозначность caller context, handler association и stack walking.

duplicate_begin

`duplicate_begin` относится к family `table_integrity` и имеет severity `medium`. Несколько RUNTIME_FUNCTION entries имеют одинаковый BeginAddress. Даже если EndAddress отличается, это говорит о неоднозначном описании начала функции. Может быть артефактом повреждения, toolchain corner case или попыткой запутать lookup.

unwind_parse_fail

`unwind_parse_fail` относится к family `unwind_integrity` и имеет severity `medium/high`. Analyzer не смог разобрать UNWIND_INFO. Причины могут быть разные: truncation, плохой opcode, плохой operand, invalid chain, handler OOB. Finding агрегирует parse failures на уровне public code, а низкоуровневые причины сохраняются в unwind_error/anomalies_text.

unwind_anomalies

`unwind_anomalies` относится к family `unwind_integrity` и имеет severity `medium/high`. UNWIND_INFO разобран, но внутри есть anomaly bits: неизвестная версия, bad opcode, code offset outside prolog, chain loop, bad alignment и т.д. Это значит, что metadata не полностью соответствует ожидаемому контракту Windows x64 unwind-модели. Severity зависит от доли affected records.

unwind_reuse_moderate

`unwind_reuse_moderate` относится к family `unwind_reuse` и имеет severity `medium`. Один UnwindInfoRVA используется заметно многими функциями, но сигнал умеренный и обычно создаётся только при дополнительном контексте.

Сам reuse не преступление: одинаковые маленькие prolog-и могут иметь одинаковые unwind descriptions. Важен масштаб и соседние findings.

unwind_reuse_strong

`unwind_reuse_strong` относится к family `unwind_reuse` и имеет severity `high`. Один UnwindInfoRVA доминирует в большой таблице: ratio выше сильного порога при достаточно большом total. Это уже не маленькая случайность. Возможна централизованная metadata model, dispatcher-style layout, packer/runtime helper или необычный toolchain. Нужна сверка с prolog-и и compiler/runtime context.

handler_dominance

`handler_dominance` относится к family `handler_dominance` и имеет severity `medium`. Один handler target обслуживает слишком большую долю functions with handler. Это тонкий сигнал: __C_specific_handler и C++ runtime handlers могут законно быть общими. Finding значимее, если рядом есть writable/non-executable placement, integrity errors или dynamic handler hits.

unwind_in_exec_section

`unwind_in_exec_section` относится к family `metadata_placement` и имеет severity `high`. UNWIND_INFO находится в executable section. Metadata является данными, а не кодом; обычно она живёт в read-only data. Executable placement может означать объединение code/data, custom linker, packer или intentionally confusing layout. Это требует ручной проверки, но не является автоматическим malware verdict.

unwind_unusual_section

`unwind_unusual_section` относится к family `metadata_placement` и имеет severity `medium/high`. UNWIND_INFO находится в section с необычным именем, не похожим на .xdata/.rdata/.pdata-like. Имя section слабее permissions, потому что оно произвольно. Но необычное имя помогает найти нестандартные compiler/dynamic /runtime regions. Если это единственный сигнал, confidence может быть снижена.

unwind_writable_section

`unwind_writable_section` относится к family `metadata_placement` и имеет severity `medium/high`. UNWIND_INFO находится в writable section. Это сильнее необычного имени, потому что writable - функциональное свойство: metadata потенциально может изменяться после загрузки. Benign explanation возможна для JIT/custom runtime, но для обычного PE это требует проверки disk-vs-memory bytes и page protections.

handler_nonexec_section

`handler_nonexec_section` относится к family `metadata_placement` и имеет severity `high`. HandlerRVA указывает в non-executable или unknown section. Handler должен быть исполняемым кодом, поэтому static section context противоречит обычной модели. Возможны повреждение metadata, custom loader, runtime VirtualProtect или ошибочная section map. Проверять надо runtime protections и реальные bytes target-a.

handler_writable_section

`handler_writable_section` относится к family `metadata_placement` и имеет severity `medium`. Handler target находится в writable section. Writable code или writable handler area усложняют доверие к disk bytes: то, что лежало в файле, могло измениться до исполнения. Finding medium, потому что runtime-generated code иногда проходит через writable stage, но без объяснения это подозрительно.

placement_exec_nonexec_handler_combo

`placement_exec_nonexec_handler_combo` относится к family `metadata_placement` и имеет severity `high`. Комбинация: unwind metadata лежит в executable section, а handler target - в non-executable/unknown section. Два разных направления placement-аномалии усиливают друг друга: данные выглядят как кодовая область, а кодовый target выглядит как не-кодовая область.

placement_integrity_combo

`placement_integrity_combo` относится к family `unwind_integrity` и имеет severity `medium`. Placement anomaly подкреплена integrity problem: parse failure, unwind anomaly или unwind RVA OOB. Combo не является независимой новой сырой находкой, а говорит: два слоя проблемы встретились, поэтому вывод сильнее, чем у одиночного placement-only signal.

reuse_dominance_combo

`reuse_dominance_combo` относится к family `unwind_reuse` и имеет severity `medium`. Одновременно есть заметный reuse unwind metadata и handler dominance. Это похоже на централизацию: много functions используют общий описатель и общий обработчик. Benign explanation возможна через runtime helper, но pattern требует сравнения нескольких functions, handler identity и dynamic events.

writable_metadata_handler_combo

`writable_metadata_handler_combo` относится к family `metadata_placement` и имеет severity `high`. И UNWIND_INFO, и handler target находятся в writable areas. Это сильный комбинированный сигнал, потому что изменяемыми становятся обе стороны unwind/handler-связки: описание кадра и место обработки. Это не доказывает факт изменения, но показывает возможность

согласованной runtime-подмены.

Counters и ratios внутри heuristic_analysis

После понимания 27 finding codes легче читать counters. Многие поля `heuristic_analysis` напрямую считают количество affected records или structural facts. `total` - количество `RUNTIME_FUNCTION` entries. Без total нельзя правильно читать ratios: одна проблема из трёх записей и сто проблем из десяти тысяч - разные ситуации.

Поля `rf_invalid_ranges`, `rf_oob`, `rf_unwind_oob`, `rf_unsorted`, `rf_overlaps`, `rf_duplicate_begins` показывают состояние function table. Это слой .pdata. Они отвечают на вопрос: сама таблица диапазонов вообще похожа на корректную карту nonleaf functions?

Поля `unwind_parse_fail`, `unwind_anomalies` и `unwind_abi_warnings` относятся к .xdata/UNWIND_INFO. Parse fail означает, что запись нельзя нормально декодировать. Anomalies означают, что декодирование удалось, но внутри обнаружены структурные странности. ABI warnings - отдельная категория strict diagnostics: они видны per function, но не увеличивают suspicion score сами по себе, потому что могут встречаться у совместимых production toolchains.

Поля placement - `unwind_unusual_section`, `unwind_exec_section`, `unwind_writable_section`, `handler_nonexec_section`, `handler_writable_section` - помогают понять, где лежат metadata и обработчики. Здесь важно помнить: section name слабее permissions. Необычное имя может быть просто custom linker. Writable или executable permission уже сильнее, потому что говорит о свойствах memory mapping.

`unwind_reuse_ratio` показывает максимальную долю functions, которые ссылаются на один и тот же `UnwindInfoRVA`. `unwind_unique_ratio` показывает, насколько разнообразны unwind descriptors относительно total. `handler_count` считает functions с handler metadata, а `handler_reuse_ratio` показывает концентрацию одного handler target среди них. Эти ratio нужны, потому что некоторые patterns видны только статистически: отдельная запись нормальна, но массовая централизация уже требует объяснения.

Weakening rules: почему отчёт умеет ослаблять собственный вывод

Очень важная часть проекта - `weakening_rules_applied`. Это правила ослабления уверенности. Они нужны, чтобы analyzer не вел себя как истеричная сирена. Если найден один слабый сигнал, если sample слишком маленький, если всё evidence пришло из одного семейства или если есть только unusual placement без других повреждений, confidence снижается. Это не скрывает finding. Finding остаётся в отчёте. Но интерпретация становится осторожнее.

`single_signal_only` появляется, когда есть ровно один non-strong finding. Смысл простой: один умеренный сигнал редко должен превращаться в уверенный вывод. `small_sample_size` появляется, когда функций меньше 32, findings есть, но strong finding нет. На маленькой выборке ratios легко переоценить. `placement_only_signal` говорит, что необычное размещение без integrity/range damage может иметь benign explanations. `single_family_only` означает, что evidence пришло только из одного смыслового семейства, а значит нужна cross-layer corroboration.

В отчёте weakening rule не является "отмазкой". Это честная запись: analyzer видит сигнал, но сам же сообщает, почему confidence нельзя разгонять. Для учебной книги это принципиально: хороший reverse engineering - это не только умение находить странности, но и умение не преувеличивать их значение.

dynamic_validation: что показывает Windows dynamic collector

Блок `dynamic_validation` появляется, когда запускалась dynamic validation. Его нельзя читать как "всё, что делает программа". Он показывает только то, что Windows dynamic collector смог наблюдать в конкретной сессии через Debug API и поставленные breakpoints. Это runtime evidence, но не всевидение. Если event отсутствует, это не значит, что механизм невозможен. Возможно, сценарий запуска до него не дошёл, timeout остановил сессию, instrumentation была неполной или нужная ветка требует других аргументов.

`enabled` показывает, был ли dynamic mode запрошен. `ran` говорит, стартовала ли dynamic session. `ok` показывает, завершился ли collector без внутренней ошибки. `timed_out` отличает нормальное завершение от остановки по времени. `target_exited` и `target_exit_code` говорят, дошёл ли debuggee до естественного выхода и с каким кодом. Эти поля нужно читать вместе: session может ran=true и иметь events, но ok=false или timed_out=true. Тогда events полезны, но assessment обязан помнить, что сессия неполная.

`timeout_ms` и `trace_steps` фиксируют параметры наблюдения. Timeout ограничивает время debug loop. Trace steps задаёт, сколько single-step адресов collector пытается записать после handler hit. Если trace_steps маленький, `RIP trace` будет коротким. Если timeout маленький, часть runtime activity может не проявиться. Report обязан хранить эти параметры, иначе повторить или сравнить dynamic run невозможно.

`instrumentation_complete` - поле качества наблюдения. Оно становится особенно важным рядом с `module_refresh_failures`, `module_list_truncations`, `breakpoint_set_failures`, `breakpoint_capacity_drops`, `pending_rearm_failures`, `breakpoint_rearm_failures` и `dropped_events`. Если collector не смог поставить часть breakpoints или потерял events из-за capacity, отсутствие

handler hit уже нельзя читать как сильный negative evidence. Это просто неполная сессия.

`modules_observed_peak` показывает максимальное количество modules, которое collector видел во время module refresh. Module map нужен, чтобы runtime VA превратить обратно в module + offset/RVA. Без module map адрес `0x7ff6...` мало что говорит. С ним можно понять: это адрес внутри main image, ntdll, kernel32 или manual/private memory. `module_refresh_failures` и `module_list_truncations` показывают, насколько collector был уверен в этой карте.

`unwind_handlers_discovered` показывает, сколько handler targets static side смогла передать dynamic collector-у как кандидаты для breakpoints. `breakpoint_set_attempts` и `breakpoint_set_failures` показывают, сколько попыток поставить breakpoints было сделано и сколько не удалось. `unwind_handler_bp_set` - сколько breakpoints на unwind handlers реально было armed. `unwind_handler_hits` - сколько раз такие handler breakpoints сработали. Это один из главных мостов между static analysis и runtime evidence: metadata говорила о handler, dynamic session увидела hit.

`exceptions_seen`, `first_chance_seen` и `second_chance_seen` описывают exception flow. First chance - первый шанс debugger-а увидеть исключение до обычного окончательного решения. Second chance - ситуация, когда обработка не справилась и debugger получает событие перед unhandled/crash path. В отчёте эти counters говорят не просто "были ошибки", а "процесс входил в exception machinery". Для correlation нужно смотреть, связаны ли эти events с конкретными static findings.

`veh_registrations`, `veh_hits`, `vch_registrations`, `vch_hits`, `uef_set_calls`, `uef_hits` относятся к runtime exception-handling APIs. VEH - vectored exception handler, VCH - vectored continue handler, UEF - unhandled exception filter. Registration показывает, что программа зарегистрировала обработчик. Hit показывает, что обработчик реально был достигнут. Registration без hit - слабее, чем hit. Hit без контекста тоже требует проверки: надо смотреть address, module, exception code и post-handler route.

`rtl_add_function_table_calls` и `rtl_install_function_table_cb_calls` показывают dynamic function table activity. Это особенно важно для JIT, runtime-generated code, manual mapping и packer scenarios. Static PE может не содержать полной .pdata для runtime-кода, но процесс во время выполнения может зарегистрировать таблицы через Windows API. Поэтому эти counters помогают объяснить ситуацию, где static map неполна, но runtime пытается предоставить Windows unwind metadata другим способом.

`events` - это timeline, ограниченный capacity `DYNAMIC_MAX_EVENTS`. Каждый event имеет `type`, `module`, `thread_id`, `exception_code`, `first_chance`, `address` и аргументы `arg0..arg3`. Meaning аргументов зависит от event type.

Поэтому event нельзя читать в отрыве от type. `trace` - массив RIP addresses после первого handler hit. Он не является полным CFG, но показывает ближайший живой след исполнения после интересного runtime-события.

assessment: итоговая сборка без притворства

Блок `assessment` появляется, когда есть heuristic report, dynamic report или оба. Это место, где report пытается сказать итоговую фразу, но делает это осторожно. Здесь есть `final_verdict`, `final_confidence`, `static_evidence_level`, `runtime_evidence_level`, `case_strength`, `static_dynamic_correlation`, `dynamic_signal_count`, `weakening_rule_count` и `manual_validation_steps`.

`final_verdict` отличается от static verdict. Static verdict строится по score из heuristic_analysis. Final verdict учитывает, была ли correlation и были ли runtime signals. Возможны значения `high_suspicion_static`, `high_suspicion_correlated`, `moderate_suspicion_static`, `moderate_suspicion_correlated`, `low_suspicion_static`, `low_suspicion_correlated`, `runtime_signals_only` и `no_strong_correlated_evidence`. Суффикс correlated не означает "доказано всё". Он означает, что по conservative rules static evidence и runtime evidence встретились на совместимом смысловом слое.

`final_confidence` начинается от static confidence, если static report есть, или от маленькой базы, если есть только dynamic side. Если dynamic session complete и есть dynamic signals, добавляется небольшой бонус. Если есть static/dynamic correlation, добавляется ещё один бонус. Максимум ограничен. Это rule-based confidence, а не Bayesian probability. Нельзя читать 0.82 как математическую вероятность. Надо читать так: по правилам проекта evidence set достаточно устойчивый, но всё ещё требует инженерного контекста.

`static_evidence_level` и `runtime_evidence_level` показывают силу двух сторон отдельно. Это важно: strong static + none runtime и weak static + medium runtime - разные истории. `case_strength` собирает их в классы: `no_case`, `weak_case`, `medium_case`, `strong_unconfirmed_case`, `medium_correlated_case`, `strong_correlated_case`. Слово case здесь означает аналитический набор evidence, а не юридический приговор.

`dynamic_signal_count` считает категории runtime-сигналов, а не все повторные events. Например, VEH registration и VEH hit относятся к одной категории VEH. Это сделано, чтобы repeated events не раздували значимость. `static_dynamic_correlation` в текущей модели conservative: она true только при handler-related static evidence плюс runtime handler/VEH/VCH/UEF hits, либо при placement anomalies плюс dynamic function table activity. Это coarse correlation layer, а не per-function proof graph.

`manual_validation_steps` - один из самых полезных блоков отчёта. Он говорит не "дальше думай сам", а предлагает проверочный маршрут. Если есть

.pdata integrity problems, надо проверить ranges в PE viewer. Если есть UNWIND_INFO issues, открыть blob в hex view и сравнить с prolog в Ghidra. Если metadata placement странный, проверить section permissions и benign compiler layout. Если dynamic table calls есть, capture arguments `RtlAddFunctionTable` / `RtlInstallFunctionTableCallback`. Если есть registered handlers, single-step от handler hit. Это превращает отчёт в рабочий план реверсера.

Как читать HTML report глазами

HTML report устроен как dashboard. Сначала он показывает большие cards: количество functions, parse failures, anomalous entries, suspicion score, static verdict, confidence и dynamic validation status. Эти cards нужны для ориентации, но по ним нельзя заканчивать анализ. Они как приборная панель автомобиля: показывают, где стоит обратить внимание, но не заменяют диагностику двигателя.

Дальше HTML показывает provenance: generated time, tool/schema, SHA-256, build, analysis options, input size и command line. Затем идёт PE image: image base, image size, headers size, exception directory, alignments. После этого structural validation и sections. Только потом отчёт переходит к final assessment, manual validation plan, static findings, weakening rules, analysis details, dynamic validation и runtime functions.

Таблица Static findings показывает severity, code, family, affected, RVA, section и message. Здесь важно читать не только message. Code является стабильным ключом. Family говорит о слое. Affected показывает масштаб. Representative RVA даёт точку входа для ручной проверки. Section связывает адрес с PE layout. Message - display text, удобное объяснение, но не основной машинный идентификатор.

Таблица Dynamic validation показывает counters и events. Для каждого event есть type, module, thread, exception, chance, address и args. Человек должен читать её как хронологию наблюдений, а не как список обвинений. Один first-chance event может быть нормальным. Один handler hit может быть нормальным runtime helper. Смысл появляется при связке с static findings, module map, post-handler RIP trace и manual validation.

Таблица Runtime functions - самая длинная. Она показывает begin/end/size, code section, unwind RVA, unwind section, status, version/flags, stack, handler, anomalies/error. Filter в HTML помогает искать по RVA, section, flag или error. Для реверсера это мост к Ghidra: можно найти функцию в report, перейти к соответствующему RVA в дизассемблере, посмотреть prolog и сравнить его с decoded unwind summary.

Как читать JSON report машиной и не сломать смысл

JSON report надо читать через schema. Top-level обязательные блоки фиксируют input, metadata, image fields, exception directory, pe_validation,

sections и functions. Optional blocks `heuristic_analysis`, `dynamic_validation` и `assessment` появляются только когда соответствующие stages выполнялись. Отсутствие `heuristic_analysis` не равно score zero. Score zero означает: heuristics запускались и не нашли сильных static anomalies. Отсутствие блока означает: этого stage в отчёте нет.

Адреса в JSON записываются строками с `0x`, а counts и ratios - числами. Это сделано специально. Адрес - не обычное количество. Его часто нужно сохранять как hex, не теряя leading zeros и не рискуя 64-bit precision в JavaScript. Consumer должен уважать типовой контракт: `"0x00001000"` - это address-like string, а 42 - count.

Backend не должен парсить HTML. Он должен читать JSON, валидировать его через `report.schema.json`, сохранять raw JSON как source of truth и только потом нормализовать selected fields в таблицы. Хорошая database-модель не выбрасывает original report, потому что schema может развиваться, а future migration может потребовать полей, которые раньше не нормализовали.

Если backend сравнивает два запуска, absolute runtime VA нельзя сравнивать напрямую из-за ASLR. Нужно нормализовать к module + offset/RVA. Для static functions стабильнее использовать begin_rva. Для findings - code + representative_rva + family. Для dynamic events - type + module + normalized offset, а не голый address.

Итог: отчёт должен учить, а не только сигналить

Если собрать всё вместе, отчёт EH/Unwind Analyzer показывает не один ответ, а слоистую модель. Внизу находятся input bytes и PE layout. Выше - sections и Exception Directory. Затем - function table и decoded unwind summaries. Затем - heuristic findings, grouped by evidence families. Затем - dynamic validation events, если запуск был выполнен. Затем - assessment, которая осторожно связывает static и runtime evidence. И наконец - manual validation steps, которые говорят реверсеру, что делать дальше.

Именно так отчёт должен читаться в этой книге. Не как табло "зелёный/красный", а как учебная карта анализа. Сначала мы спрашиваем: корректно ли прочитан PE? Потом: что заявляет .pdata? Потом: что рассказывает .xdata? Потом: какие инварианты нарушены? Потом: какие эвристики сработали и почему? Потом: что удалось увидеть в runtime? Потом: усиливает ли runtime статическую гипотезу или только добавляет отдельные observations? И только в самом конце мы смотрим final verdict.

Это и есть здоровая инженерная дисциплина. Она защищает нас от двух крайностей: от наивного доверия к PE-файлу и от такой же наивной паники при любом unusual signal. Отчёт не должен думать вместо реверсера. Он должен показывать реверсеру достаточно связей, чтобы тот мог думать точнее.

Поле за полем: как не потеряться в heuristic_analysis

Чтобы отчёт был не просто красивым, а учебным, полезно пройти блок `heuristic_analysis` почти так, как это делает внимательный analyst. Сначала он смотрит не на verdict, а на масштаб данных. Поле `total` говорит, сколько записей `RUNTIME_FUNCTION` было в таблице. Это число задаёт фон для всех процентов и ratios. Если total равен 8, одна странная запись уже даёт большую долю, но это ещё маленькая выборка. Если total равен 8000, даже маленький процент может означать десятки функций, и его нельзя отмахнуть как случайность.

Затем analyst смотрит counters integrity-слоя. `rf_invalid_ranges` показывает записи, где диапазон функции не имеет смысла. `rf_oob` показывает записи, где begin/end уходят за image. `rf_unwind_oob` показывает записи, где pointer на unwind metadata недоступен. `rf_unsorted` говорит о нарушении ожидаемого порядка table entries. `rf_overlaps` и `rf_duplicate_begins` говорят об неоднозначности: один и тот же участок кода может описываться несколькими способами. На этом этапе ещё не нужно думать о malware. Нужно думать о более базовой вещи: может ли Windows и analyzer вообще построить стабильную карту функций?

После этого analyst переходит к unwind integrity. `unwind_parse_fail` - это не "немного странно", а ситуация, где analyzer не смог получить полноценную `UNWIND_INFO` model. Поле `unwind_anomalies` мягче: model получена, но внутри есть anomaly bits. `unwind_abi_warnings` ещё мягче и отдельно: это strict ABI diagnostics, которые остаются видимыми, но не увеличивают suspicion score сами по себе. Такое разделение важно: parse failure, structural anomaly и формальное ABI warning не должны смешиваться в один бак "плохо".

Потом идут placement counters. Они отвечают на другой вопрос: не "можно ли прочитать metadata", а "в каком окружении она лежит". `unwind_unusual_section` говорит об имени/контексте section. `unwind_exec_section` говорит, что metadata лежит там, где допускается исполнение кода. `unwind_writable_section` говорит, что metadata лежит там, где bytes могут быть изменены. `handler_nonexec_section` говорит, что handler target не выглядит исполняемым по section flags. `handler_writable_section` говорит, что handler target находится в writable окружении. Эти поля надо читать в связке: writable metadata без handler anomaly - один уровень; writable metadata плюс writable handler target - уже другой.

Reuse ratios читаются после integrity и placement, а не до них. `unwind_reuse_ratio` и `unwind_unique_ratio` могут быть абсолютно нормальными для маленьких функций, thunks или compiler-generated code. Поэтому project policy не делает вывод только из одного красивого ratio. Она смотрит на total, thresholds и соседние findings. `handler_reuse_ratio` ещё

тоньше, потому что runtime/language handlers часто легитимно общие. Если dominant handler похож на известный runtime helper и больше нет повреждений, finding может быть подавлен или ослаблен. Если dominant handler совпадает с writable/non-executable placement или runtime hits, история становится серьёзнее.

Как score набирается и почему его нельзя читать как вероятность

Внутри heuristic/evidence engine у разных findings есть веса. Например, грубые layout-ошибки вроде `pe_section_raw_oob` или `pe_exception_directory_oob` имеют высокий вес, потому что они ломают физическую доступность или корневой вход в exception table. Overlap, duplicate begin, unusual section name или moderate reuse имеют меньший вес, потому что у них больше benign alternatives. Combined findings добавляют ограниченный бонус, потому что они не являются полностью независимыми новыми фактами: они выражают усиление смысла при совместном появлении нескольких сигналов.

Здесь легко ошибиться. Score не является вероятностью. Он не говорит: "вероятность обфускации 25%" или "вредоносность 70%". Score - это инженерная шкала suspiciousness по текущей policy. Policy - это набор правил, выбранных разработчиком и закреплённых тестами. Она нужна для стабильности: один и тот же input при той же версии tool/schema должен давать понятный результат. Но policy не претендует на универсальную статистическую истину для всех компиляторов, упаковщиков и рантаймов мира.

Вердикты score thresholds тоже надо читать аккуратно. `score >= 70` даёт high suspicion, `>= 40` moderate, `>= 15` low. Ни один из этих порогов не является юридическим заключением. Они помогают triage - первичной сортировке. Triage нужен, чтобы понять, куда смотреть сначала. Если high suspicion возникает из-за нескольких independent families, это одно. Если moderate suspicion возникает из-за одного placement-only signal и weakening rule, это другое. Поэтому рядом со score всегда живут confidence, evidence families, findings и weakening rules.

Confidence строится иначе. Базово он получает вклад от количества evidence families, от числа strong и medium findings, от hard table-integrity и hard unwind-integrity issues. Но затем confidence может снижаться. Это важнее, чем кажется. Без weakening rules анализатор становился бы слишком самоуверенным. Он бы видел один unusual section и радостно повышал подозрение. С weakening rules он говорит: "да, сигнал есть, но это одиночный сигнал, sample маленький или family одна, поэтому вывод должен быть осторожнее".

Что именно report показывает по каждому finding

Каждая запись в `findings` имеет несколько полей, и каждое поле нужно читать. `code` - стабильный идентификатор. Его должен использовать backend, тесты, фильтры и документация. `family` показывает слой evidence. `severity` показывает локальную серьёзность finding-а, но не равен final verdict. `message` объясняет finding человеческим языком, но не должен быть machine key. `affected_count` показывает масштаб. `representative_rva` даёт точку для начала ручной проверки. `representative_section` помогает сразу увидеть section context.

Почему representative RVA один, если affected_count может быть большим? Потому что report не должен превращаться в бесконечный dump всех однотипных ошибок. Finding агрегирует класс сигнала. Representative RVA даёт analyst-у первый конкретный адрес, с которого можно начать. Если нужно увидеть все affected locations, future version могла бы добавить подробный location list, но текущий report design сознательно балансирует полноту и читаемость.

Важно не путать severity и affected_count. Один high finding с affected_count=1 может быть критически важным, если это корневая Exception Directory OOB. Medium finding с affected_count=500 тоже нельзя игнорировать, если он показывает массовый pattern. В хорошем чтении отчёта analyst держит сразу три оси: насколько сильный сигнал, насколько он массовый, на каком слое он возник.

Anomalies внутри UNWIND_INFO: компактные labels и реальный смысл

В functions/unwind и CSV встречается `anomalies_text`. Это компактная строка из labels. Она нужна, чтобы таблица оставалась читаемой. Label `ver` говорит о неизвестной версии UNWIND_INFO. `trunc` - о недостатке bytes для чтения структуры. `op` - о плохом unwind opcode. `arg` - о плохом operand. `off` - о code offset outside expected prolog. `chain_loop` и `chain_depth` связаны с chained unwind info, который либо зациклился, либо ушёл глубже разрешённого лимита.

Labels `handler_oob`, `order`, `operand_sanity`, `alignment`, `chain_invalid` показывают более частные нарушения: handler target вне ожидаемых границ, странный порядок offsets, подозрительные operands, нарушение alignment или недопустимую chained запись. Отдельно существуют ABI labels вроде `abi_register_class`, `abi_reserved_opinfo`, `abi_push_order`, `abi_chain_restriction`, `abi_save_offset`. Они важны для строгой диагностики, но не должны механически увеличивать suspicion score без контекста.

Почему report хранит и numeric mask, и text? Numeric mask стабилен для программной обработки. Text удобен человеку. Если оставить только text,

backend начнёт парсить строку. Если оставить только number, человек потеряет смысл. Хороший report даёт оба слоя: machine layer и human layer.

Dynamic events: как читать type, module, address и arg0..arg3

Dynamic event - это одна запись о runtime-наблюдении. Но слово "event" не должно обманывать: разные event types имеют разную смысловую нагрузку. Exception event говорит о входе в exception machinery. Handler hit говорит о достижении handler target или breakpoint-a. Registration event говорит, что программа зарегистрировала runtime handler. Dynamic function table event говорит, что процесс сообщил Windows о runtime unwind map. Поэтому поле `type` обязательно должно читаться первым.

`module` связывает runtime address с загруженным module. Без module field адрес почти бесполезен, потому что ASLR каждый запуск меняет base. `thread_id` показывает, в каком потоке произошло событие. Это важно для исключений: handler в одном потоке не доказывает, что все потоки шли тем же маршрутом. `exception_code` показывает тип исключения: breakpoint, guard page, access violation, illegal instruction и так далее. `first_chance` говорит, в какой фазе debugger увидел exception.

`address` обычно указывает на основной runtime address события: instruction pointer exception-a, handler target, API hit или related location. Но точный смысл зависит от event type. Поля `arg0`..`arg3` дают дополнительные значения. Для API-hook-like events это могут быть аргументы вызова; для handler-related events - адреса handler/registration/context; для exception events - дополнительные exception information. Поэтому отчёт должен документировать adapter semantics, а analyst должен не читать arg0 как универсальное поле.

Если `dropped_events` больше нуля, timeline неполный. Это не значит, что все выводы бесполезны, но значит, что отсутствие какого-то события нельзя использовать как сильное отрицательное доказательство. Если `instrumentation_complete` false, dynamic side нужно читать как partial evidence. Хороший assessment должен это учитывать, и проект действительно ограничивает runtime evidence level для incomplete sessions.

Manual validation plan как мост от отчёта к работе руками

Manual validation steps в assessment - это не декоративные советы. Они кодируют следующий разумный путь. Если report говорит о .pdata range integrity, analyst должен отсортировать RUNTIME_FUNCTION entries, проверить begin/end/unwind RVAs и убедиться, что они внутри SizeOfImage. Если report говорит о UNWIND_INFO issues, analyst открывает blob в hex view, проверяет Version/Flags/CountOfCodes/tail layout и сравнивает decoded prolog actions с

Ghidra disassembly. Если report говорит о metadata placement, analyst смотрит section permissions, raw offset, RVA и возможные benign linker/runtime explanations.

Если handler target non-executable, ручная проверка не заканчивается фразой "странно". Нужно дизассемблировать target, проверить page protections после загрузки, посмотреть, менялись ли permissions через VirtualProtect, не является ли это thunk-ом, pointer table или ошибкой section map. Если есть writable metadata или writable handler, нужно сравнить disk bytes и memory bytes, а также подумать о JIT/runtime scenario. Если есть handler dominance, нужно выяснить identity handler-a: известный runtime helper или custom dispatcher.

Dynamic-related steps тоже конкретны. При вызовах `RtlAddFunctionTable` / `RtlInstallFunctionTableCallback` нужно capture arguments и посмотреть runtime table contents. При VEH/VCH/UEF registration нужно записать target address и single-step от handler-a. При unwind handler hits нужно понять, это known language/runtime handler или dispatcher/interceptor path, referenced by many protected functions. Если static findings есть, а dynamic signals нет, report предлагает запуск с `--dynamic` на representative exception path. Это аккуратный язык: не "динамика ничего не нашла", а "нужен сценарий, который действительно проходит интересную ветку".

Почему отчёты показывают не только плохое

Хороший отчёт обязан показывать и нормальность. Если parse failures ноль, anomalies ноль, score ноль и functions выглядят согласованно, это тоже результат. Он не доказывает, что программа безопасна или незащищена, но говорит: на уровне PE/unwind metadata strong anomalies не найдено. Такая формулировка точнее, чем "чисто". Анализатор специализируется на EH/unwind-слое, а не на всех аспектах поведения программы.

Поэтому значение `no_strong_static_anomalies` надо читать буквально: "текущие эвристики не нашли сильных статических аномалий". Это не значит, что файл benign. Это не значит, что нет packer-a. Это не значит, что нет runtime behavior. Это значит, что в проверенных структурах не набралось существенного статического evidence по текущей policy. Такая честность особенно важна для дипломного/инженерного проекта: инструмент должен ясно говорить о границах своей компетенции.

Точно так же `runtime_signals_only` не означает, что static analysis провалился. Это означает, что статический score ниже threshold, но dynamic validation увидела meaningful runtime activity. Например, были registrations, handler hits или dynamic function table calls. Такой результат ценен: он говорит, что runtime слой активен, даже если PE metadata не выглядит сильно аномальной.

Мини-сценарий чтения отчёта: от dashboard к конкретному RVA

Представим, что HTML dashboard показывает moderate suspicion, confidence около среднего, dynamic validation completed. Новичок может сразу смотреть verdict и паниковать. Правильнее идти медленно. Сначала проверить input SHA-256 и command line: тот ли файл анализировали и с какими опциями. Затем посмотреть PE image и sections: есть ли странные permissions, где Exception Directory. Затем открыть structural validation: нет ли raw/virtual OOB или overlaps. После этого перейти к static findings и увидеть, какие именно codes сработали.

Допустим, сработали `unwind_writable_section` и `handler_writable_section`, а рядом combo `writable_metadata_handler_combo`. Теперь мы понимаем: проблема не в том, что "что-то подозрительное". Проблема в том, что и описание unwind, и handler target находятся в writable context. Следующий шаг - взять representative RVA, посмотреть section, открыть соответствующую запись functions, найти handler_rva, перейти в Ghidra, проверить bytes и permissions. Затем посмотреть dynamic_validation: были ли handler hits, были ли dynamic function table calls, менялся ли RIP trace после handler-а.

Если dynamic_validation показывает handler hit, case становится сильнее. Но всё ещё нужно понять identity handler-а. Если это известный runtime handler, вывод один. Если это custom dispatcher в writable region, вывод другой. Если dynamic session timed out, handler hit отсутствует, но instrumentation incomplete, отсутствие hit-а нельзя считать оправданием. Именно так report учит думать: не от verdict к эмоции, а от evidence к проверяемой модели.

Что нужно будет улучшить в будущем report contract

Текущий report уже полезен, но его можно развивать. Например, manual validation steps сейчас являются display strings, а не stable IDs. Для backend localization, UI workflow и автоматизации было бы полезно дать каждому step code. Dynamic event args сейчас имеют meaning by event type, но это лучше явно документировать и, возможно, разделить разные event payloads на typed objects. Anomalies text компактный, но для долгосрочной schema хорошо иметь массив полных anomaly codes alongside numeric mask.

Для более сильной static/dynamic correlation будущая версия могла бы хранить per-function proof links: static finding code, representative RVA, candidate runtime VA, event index, normalized module offset, confidence note. Тогда report мог бы отличать family-level correlation от address-level и route-level correlation не только в тексте, но и в JSON. Сейчас модель conservative и coarse, что честно для версии 1.0.0, но путь развития очевиден.

Важно, что эти улучшения не должны ломать главное: отчёт должен оставаться объяснимым. Чем больше полей добавляется, тем выше риск снова

превратить report в свалку терминов. Поэтому каждый новый field должен отвечать на простой вопрос analyst-а: что я теперь могу понять или проверить лучше, чем раньше?

9.6. Границы вывода: где анализатор обязан сказать unknown

После раздела об отчетах легко попасть в приятную, но опасную иллюзию. У нас есть parser, decoder, heuristic/evidence engine, dynamic validation, JSON, HTML, Ghidra CSV, assessment, manual validation steps. Вроде бы инструмент уже умеет посмотреть на PE-файл с разных сторон и выдать аккуратный вывод. Но именно в этот момент хороший инженер должен сделать шаг назад и спросить не “что еще можно автоматически найти”, а “где мы не имеем права делать вид, что знаем больше, чем реально знаем”.

Это не второстепенная тема и не раздел с оправданиями. Границы вывода - часть самой архитектуры анализатора. Если инструмент не умеет говорить unknown, он будет неизбежно врать. Он может врать мягко, через слишком уверенное слово. Может врать красиво, через красный verdict в HTML. Может врать машинно, через score, который кто-то прочитает как вероятность. Но результат один: пользователь начнет доверять выводу сильнее, чем позволяют данные. Для reverse engineering это особенно вредно, потому что реверсер работает не с прозрачной системой, а с чужим бинарем, который может быть поврежден, упакован, вручную загружен, специально запутан или просто собран необычным toolchain-ом.

Слово unknown здесь лучше не переводить как “неизвестно” в бытовом ленивом смысле. В хорошей аналитике unknown - это честное состояние знания. Оно означает: на этом слое наблюдений у нас недостаточно evidence, чтобы сделать определенный вывод. Не “мы ничего не поняли”, а “мы понимаем, где заканчивается сила текущего метода”. Это важная мысль. Инструмент, который различает clean, suspicious and unknown, гораздо взрослее инструмента, который любой неясный случай превращает в suspicious или silently ignores it.

В EH/Unwind Analyzer эта честность особенно важна, потому что проект соединяет разные источники свидетельств. Static analysis читает файл как bytes и извлекает заявленную структуру: headers, sections, Exception Directory, RUNTIME_FUNCTION, UNWIND_INFO, handlers and decoded properties. Heuristic/evidence engine оценивает набор статических признаков. Dynamic validation, если она запущена на Windows, наблюдает часть живого поведения через Windows Debug API. Reports связывают эти слои в artifact. Но ни один слой не является всевидящим. Static analysis не видит будущие branches, которые не представлены в файле. Dynamic validation не проходит все возможные пути выполнения. Heuristic model не является математическим классификатором. Report не превращает неполное наблюдение в полное знание.

Ограничение как часть модели, а не слабость характера

Термин limitation обычно переводят как “ограничение”. В технической книге его важно понимать не как признание провала, а как описание области применимости. У молотка есть ограничение: им неудобно измерять напряжение. Это не делает молоток плохим. Плохим он становится тогда, когда его продают как мультиметр. Точно так же static analyzer плох не тогда, когда он не видит runtime-only table, а тогда, когда делает вид, что отсутствие этой table в PE означает ее отсутствие в процессе.

Область применимости - это граница, внутри которой метод дает осмысленный результат. Static analysis применим к тому, что можно вывести из файла и его metadata. Он может много: проверить PE structure, найти Exception Directory, построить RVA mapping, прочитать RUNTIME_FUNCTION, декодировать UNWIND_INFO, обнаружить поврежденные ranges, unusual metadata placement, reuse, handler dominance, parse failures. Но он не может гарантировать, что target при запуске не выделит private executable memory, не зарегистрирует dynamic function table, не распакует другой payload and не изменит route обработки исключения.

Dynamic validation применима к тому, что произошло в конкретном запуске. Она может увидеть first-chance exception, handler hit, VEH/VCH/UEF registration, RtlAddFunctionTable, post-handler RIP trace. Но она не может сказать, что другой input, другой аргумент командной строки, другая дата, другой environment variable, другая машина или другая ветка GUI никогда не приведут к другой картине. Поэтому dynamic validation усиливает evidence, но не превращает один запуск в доказательство полного поведения программы.

Так появляется зрелое инженерное правило: каждый слой должен говорить только о том, что он реально наблюдал. Файл не наблюдает runtime. Runtime session не наблюдает все paths. Heuristic score не наблюдает intent автора программы. Report не наблюдает внешний threat model. Если мы удерживаем эти границы, анализатор остается честным. Если смешиваем их, текст отчета становится красивым, но ненадежным.

Static analysis видит файл, а не всю будущую жизнь процесса

Начнем со static analysis. Слово static означает “статический”, то есть неподвижный, рассматриваемый без выполнения. В проекте static analysis не запускает target. Он читает input bytes, строит PeImage, проверяет headers and sections, переводит RVA в file offset, находит Exception Directory, читает массив RUNTIME_FUNCTION, затем идет в .xdata and decodes UNWIND_INFO. Это огромный слой работы, но его объектом остается файл и структуры, которые удалось извлечь из файла.

Здесь важно не потерять уже построенную модель PE. PE-файл на диске содержит headers and sections. Windows loader при запуске отображает image in memory, применяет relocations, загружает imports, выполняет TLS callbacks, передает управление entry point. Static analysis смотрит на file representation and declared metadata. Он может сказать: “в этом файле Exception Directory имеет такой RVA and size; эти RUNTIME_FUNCTION entries лежат здесь; этот UnwindInfoRVA указывает туда; этот handler target попадает в такую section”. Но он не может заранее увидеть все memory pages, которые программа создаст после старта через VirtualAlloc, не может узнать, какие handlers будут зарегистрированы later, and не может выполнить branch, который зависит от runtime input.

Поэтому отсутствие finding не означает безопасность, чистоту or benign nature. Это очень важная дисциплина языка. Если static analysis возвращает verdict no_strong_static_anomalies, его нужно читать буквально: реализованный набор статических правил не обнаружил сильных статических аномалий. Не “файл безопасен”. Не “файл нормальный”. Не “unwind-модель идеальна во всех возможных runtime-состояниях”. Только ограниченное утверждение о текущем слое анализа.

```
Правильное чтение:
  no_strong_static_anomalies = в реализованных static rules сильных признаков не найдено

Неправильное чтение:
  no_strong_static_anomalies = файл безопасен / все чисто / анализ завершен
```

Это различие кажется словесным, но оно меняет поведение реверсера. Если человек видит “safe”, он закрывает файл. Если видит “no strong static anomalies”, он понимает: текущий инструмент не нашел сильных признаков в своей зоне видимости, но при подозрительном context можно продолжать manual review, dynamic validation, сравнение memory image, проверку imports, strings, behavior and environment-sensitive paths.

Static analysis также зависит от того, насколько корректно удалось построить mapping. Mapping - это соответствие между разными системами координат файла: file offset, RVA and VA. File offset - положение байта в файле на диске. RVA, relative virtual address, - смещение относительно ImageBase после загрузки image. VA, virtual address, - фактический адрес в виртуальной памяти процесса. Если PE sections повреждены, пересекаются or имеют странные raw/virtual ranges, analyzer может аккуратно зафиксировать structural findings, но часть последующих выводов станет слабее. Нельзя надежно декодировать .xdata, если путь от RVA к bytes неоднозначен или выходит за файл.

Именно поэтому project reports показывают pe_validation, sections, exception_directory and functions как разные слои. Это не декоративное дробление. Если верхний слой поврежден, нижние выводы нужно читать осторожнее. Хороший report должен позволить пользователю проследить: проблема возникла в PE layout, в runtime-function table, в unwind decoding, в heuristic interpretation or in dynamic session. Без этой вертикали ограничение превращается в мутное “что-то не так”.

Heuristic/evidence engine не является классификатором вредоносности

Следующая граница проходит внутри heuristic/evidence engine. Heuristic переводится как “эвристика”, но это слово часто звучит слишком умно и поэтому начинает казаться сильнее, чем оно есть. В этом проекте эвристика - это инженерное правило, которое связывает наблюдаемый признак с осторожной оценкой. Например, overlapping RUNTIME_FUNCTION ranges нарушают однозначность function table. Writable unwind metadata снижает доверие к данным, от которых зависит unwinding. Dominant handler target может быть нормальным runtime pattern, а может быть признаком централизованного exception control flow. Эвристика говорит: этот признак стоит внимания. Она не говорит: это точно вредоносный файл.

Поэтому score - не вероятность. Score в проекте является ограниченной суммой weights, то есть весов, заданных разработчиком в code and evidence matrix. Suspicion - нормализация score к диапазону 0..1. Confidence - отдельная оценка уверенности в интерпретации, построенная по rule-based model: сколько evidence families задействовано, есть ли strong findings, medium findings, hard table-integrity issues, parse failures, weakening rules. Ни score, ни confidence не были получены как статистическая вероятность на огромном независимом наборе с calibration. Их нельзя читать как “70% malware”.

Это место нужно повторить жестко, потому что ошибка соблазнительная. Человек видит число и мозг хочет прочитать его как процент. Но score 70 в EH/Unwind Analyzer означает не “70 процентов вредоносности”, а “суммарная масса реализованных suspicious static rules достигла порога high_suspicion”. Confidence 0.80 означает не “в 80 процентах случаев такой verdict верен”, а “по текущим правилам evidence достаточно разнообразен and reinforced, чтобы интерпретация была сильной в рамках модели”.

```
score        = weighted anomaly mass внутри rule-based model
suspicion    = score / 100
confidence   = rule-based confidence in interpretation, not probability
verdict      = label по score thresholds
case strength= итоговая сила случая после static/dynamic evidence
```

Weakening rules, которые в C model исторически названы mitigations, тоже нужно читать осторожно. Mitigation в бытовом смысле звучит как “смягчение” или “устранение”. В проектном смысле weakening rule не отменяет факт. Если найден unusual section, правило placement_only_signal не говорит “это нормально”. Оно говорит: этот признак изолирован, у него есть benign explanation, поэтому уверенность в сильной подозрительной интерпретации нужно снизить. Факт остается в report. Меняется не наблюдение, а сила вывода.

У heuristic/evidence engine есть и технические ограничения реализации. Finding storage имеет limit HEURISTICS_MAX_FINDINGS. Если структурированных findings больше лимита, score and details могут продолжить учитывать события, но structured finding array не растет бесконечно. Это значит, что severity counts and evidence level могут недоучесть то, что не попало в structured storage. В обычном наборе правил текущего проекта это не должно ломать typical run, но как инженерное ограничение оно должно быть названо. Для production-grade evolution логичным улучшением был бы dynamic array или явный truncation flag.

Еще один пример - frequency maps для reuse and handler dominance. Они используют hash map and may grow. Если allocation fails during grow, текущая реализация может получить неполные frequency results без отдельного failure propagation. Практически это редкий memory-pressure case, но он показывает важный принцип: даже evidence engine, который звучит как “умная логика”, остается C-кодом with memory ownership, allocation and limits. Книга должна учить не только механике Windows, но и инженерной скромности: автоматический вывод хорош ровно настолько, насколько честны его inputs, implementation and reporting.

Strict ABI diagnostics отделены от suspicion

Отдельно стоит разобрать strict ABI diagnostics. ABI - application binary interface, прикладной бинарный интерфейс. Это набор правил, по которым compiled code взаимодействует на уровне регистров, стека, вызовов, сохранения nonvolatile-регистров, prolog/epilog and unwind metadata. В x64 Windows ABI связан с тем, как функция строит frame, как сохраняет registers, как описывает unwind operations and how OS can restore caller context.

Флаг --strict-abi в проекте включает более формальную диагностику отклонений от Microsoft x64 unwind ABI. Но эти diagnostics не должны автоматически увеличивать suspicion score. Почему? Потому что ABI strictness and suspiciousness are not the same axis. Реальный production compiler, custom toolchain or hand-written assembly может выдавать unwind records, которые

выглядят необычно относительно строгой формальной модели, но не являются malicious. Если каждый strict ABI warning превращать в suspicious finding, analyzer начнет шуметь и терять полезность.

Поэтому strict ABI diagnostics остаются видимыми per function, но не являются самостоятельным основанием для verdict. Это хороший пример зрелого разделения слоев. Structural corruption может влиять на score. Suspicious placement может влиять на score. Handler dominance may contribute. Formal ABI warnings помогают manual review, but do not automatically claim malicious or suspicious. Пользователь отчета должен видеть их, но не путать с heuristic anomaly assessment.

Dynamic validation наблюдает сценарий, а не все поведение программы

Теперь перейдем к dynamic validation. Dynamic означает “динамический”, то есть связанный с выполнением. В проекте dynamic validation реализуется Windows dynamic collector: он запускает target as debuggee, получает debug events, ставит breakpoints, наблюдает exceptions, registrations, handler hits and trace. Это мощный слой, потому что он добавляет runtime evidence - свидетельства живого выполнения. Но именно из-за своей силы он особенно опасен, если его переоценить.

Dynamic validation не является исчерпывающим исследованием программы. Она видит только то, что произошло в конкретной session. Session - это один запуск с конкретными аргументами, окружением, user privileges, system DLLs, временем, input files and execution path. Если программа имеет menu, network check, delayed branch, date trigger, command-line mode or hidden code path, collector может просто не попасть туда. Отсутствие handler hit в одной session не доказывает, что handler никогда не сработает. Оно доказывает только: в этом запуске, при этих условиях, collector не observed this hit.

Coverage - полезный термин, но его тоже нужно разжевать. Coverage можно перевести как покрытие. В нашем контексте это доля поведения, которую удалось фактически наблюдать. Dynamic collector не содержит полноценного coverage engine для всех basic blocks and branches. Он собирает targeted runtime evidence: exceptions, handler-related events, VEH/VCH/UEF registrations, function table registrations, selected handler breakpoints and RIP trace. Поэтому dynamic coverage здесь тематическое, а не полное. Он покрывает интересующие exception/unwind events, not the entire behavior space of target.

Есть и platform boundary. Static analysis в проекте кроссплатформенный: Linux, macOS and Windows могут читать Windows PE as bytes. Dynamic validation доступна только в Windows build, потому что использует Windows Debug API and needs a Windows x64 target execution environment. Non-Windows stub честно сообщает unsupported platform. Это не слабость parser-а; это правильное разделение backend-ов. Эмулировать Windows Debug API на Linux “для галочки” было бы хуже, чем честно сказать, что runtime collector здесь недоступен.

```
Linux/macOS host:
  static analysis of Windows PE = possible
  Windows dynamic validation = unsupported

Windows x64 host:
  static analysis = possible
  dynamic validation through Windows Debug API = possible
```

Наконец, dynamic validation пересекает trust boundary гораздо сильнее, чем static analysis. Trust boundary - граница доверия. Пока analyzer читает PE as bytes, target не исполняется. Когда Windows dynamic collector запускает target, чужой executable получает возможность выполнять код: читать файлы, писать файлы, создавать процессы, обращаться к сети, зависать, потреблять ресурсы, пытаться обнаружить debugger. DEBUG_ONLY_THIS_PROCESS ограничивает дерево отладки, но не является sandbox. Timeout ограничивает длительность session, но не является sandbox. Поэтому production usage должен запускать dynamic worker в disposable Windows VM or hardened sandbox with resource limits and restricted network. Это не “паранойя”; это нормальная цена живого выполнения недоверенного файла.

Timeout, incomplete session and dropped events

Теперь нужно понять, почему поля session state in DynamicReport так важны. DynamicReport хранит enabled, ran, ok, timed_out, target_exited, target_exit_code, counters, events, trace and log. Эти поля не технический мусор. Они говорят, насколько мы вообще имеем право использовать runtime evidence.

enabled означает, что dynamic mode был запрошен. ran означает, что CreateProcess succeeded and target was actually created. ok означает, что session получила CREATE_PROCESS event, не встретила internal failure and did not time out. target_exited показывает, что target дошел до EXIT_PROCESS_DEBUG_EVENT. Эти состояния нельзя сводить к одной галочке “dynamic succeeded”. Например, session может ran=true, собрать несколько events, затем timed_out=true. Эти events полезны, но session incomplete. Она не должна усиливать final confidence так же, как clean completed run.

Timeout - это ограничение времени. Collector измеряет elapsed time through GetTickCount64 and calls WaitForDebugEvent with примерно 100 ms granularity. Если elapsed превысил timeout_ms, session завершается как timed out. В report это не просто “программа долго работала”. Это граница наблюдения. Все, что случилось до timeout, можно учитывать as observed. Все, что могло случиться после timeout, остается unknown. Поэтому incomplete session cannot establish strong static/dynamic correlation and should cap runtime evidence level. Evidence matrix прямо следует этой логике: raw events сохраняются, но их сила ограничивается.

dropped_events - еще одно поле, которое легко недооценить. Dynamic events bounded: maximum 256. После заполнения array новые события не перезаписывают старые, а counter dropped_events увеличивается. Это правильнее, чем молча терять состояние. Если report показывает dropped_events > 0, timeline truncated. Пользователь должен понимать: collector saw more activity than it could store. Значит отсутствие позднего event в array уже нельзя читать как отсутствие такого event in execution. Это снова не failure всего анализа, а честная отметка границы отчета.

Trace тоже bounded. trace_steps clamped до 256, и RIP trace является коротким следом instruction pointers, а не полным execution trace. RIP trace отвечает на локальный вопрос: куда управление пошло после выбранного handler hit or breakpoint-related situation. Он не заменяет full tracing, dynamic instrumentation or symbolic execution. Поэтому запись “post-handler RIP trace ends at address X” нельзя читать как “функция полностью проанализирована”. Это только маленькое окно живого маршрута.

Адреса в dynamic validation зависят от module map

Одна из самых практических границ dynamic validation связана с адресами. Static analysis обычно говорит на языке RVA: относительных адресов внутри image. Runtime работает с VA: фактическими virtual addresses в процессе. Между ними стоит module base, который может измениться из-за ASLR. ASLR - address space layout randomization, рандомизация расположения адресного пространства. Из-за нее module может быть загружен не туда, куда “предпочитал” PE header. Поэтому нельзя взять local address API in analyzer process and directly use it in target process.

Проект решает это через module map. Module map - карта загруженных модулей target: basename, base, size. Collector refreshes modules through Toolhelp snapshot, учитывает forwarded APIs, вычисляет offset локального export-а относительно real local module base, затем переносит этот offset на соответствующий module target-a. Это разумная engineering model for one

Windows installation, but it has assumptions. Она предполагает совместимые module binaries between analyzer process and target process. Если target использует другой module version, unusual loader behavior or module substitution, offset transfer can be wrong unless remote export parsing is implemented.

Сама module list bounded: максимум 128 modules. Если target загрузил больше, остаток не сохраняется, и explicit truncation flag отсутствует. Это значит, что API resolution for late module can fail silently from the perspective of complete observation. В обычной session это может быть несущественно, но для аккуратного analysis такой limit должен быть назван. Когда dynamic_validation показывает отсутствие какого-то API breakpoint, нужно помнить: это может быть не только отсутствие API call, но и неполное module observation, unsupported forwarded resolution case, timing issue or breakpoint placement failure.

Breakpoints являются инструментом наблюдения, а не невидимой магией

Software breakpoint в Windows/x86/x64 обычно реализуется через байт 0xCC, инструкцию INT3. Collector temporarily writes INT3 into target memory at address of interest. Когда debuggee reaches this address, CPU raises breakpoint exception, debugger получает EXCEPTION_DEBUG_EVENT, затем должен восстановить original byte, отмотать RIP назад, single-step original instruction and re-arm breakpoint. Это не абстрактная “метка”. Это реальное изменение code byte in target process.

Отсюда появляются ограничения. Если page not writable through WriteProcessMemory, breakpoint placement can fail. Если code is self-modifying, original byte may change. Если module unloads, breakpoint address may become unmapped. Project notes that UNLOAD_DLL_DEBUG_EVENT refreshes module list, but breakpoints inside unloaded module are not removed immediately from BpList; final cleanup clears best-effort. If another module later maps at same address, stale breakpoint model may become a problem. Это редкий, но честный limit dynamic collector.

Thread-local breakpoint state тоже не бесконечен. Один thread может иметь только один pending breakpoint in the current model. Это соответствует обычной последовательности: поток остановился на breakpoint, debugger восстанавливает instruction, ставит Trap Flag, выполняет single-step, затем возвращает breakpoint. Но если control flow or debugging events become unusual, simplified pending model can be stressed. Опять же, это не делает collector бесполезным. Это означает, что его trace and hits нужно читать как

bounded observation produced by a debugger-like tool, not as omniscient instrumentation.

Unknown breakpoints требуют особой осторожности. Windows обычно генерирует initial system breakpoint после создания debuggee. Collector допускает один unknown breakpoint as initial and continues it. Следующие unknown breakpoints are passed as not handled. Это pragmatic rule. Но target может сам выполнить early INT3, order may be unusual, and anti-debug logic may intentionally exploit breakpoint behavior. Поэтому breakpoint events in dynamic report need context: was it our breakpoint role, initial breakpoint, ordinary exception or unknown INT3 passed through?

Dynamic events не сохраняют все детали исключения

DynamicEvent в проекте compact. Он содержит type and generic arg0..arg3. Такой формат удобен: разные события можно сериализовать одной простой структурой. Например, VEH registration can store First and Handler. RtlAddFunctionTable can store Table, EntryCount and BaseAddress. Handler hit can store address-related facts. Но generic args имеют цену: schema itself does not express event-specific types richly. Consumer должен знать mapping for each event type. Для долгосрочной публичной API более сильной моделью были бы typed subobjects or event-specific fields with backward-compatible base.

Есть и конкретное ограничение exception events. Collector counts ordinary exceptions, first_chance and second_chance, stores exception address, exception code, thread id and module. Но он не сериализует NumberParameters and ExceptionInformation[]. Для access violation это означает, что report currently does not preserve attempted address and operation type. А это важные детали: чтение или запись, какой address tried, whether it was null page, guard page, freed memory or executable page. Поэтому access violation in current DynamicReport является useful signal, but not full forensic record.

Это хороший пример того, как должен думать читатель. Не “collector увидел access violation, значит все понятно”. А: collector зафиксировал exception code and address in this session; for deeper analysis manual debugger run or extended event schema would be needed to capture ExceptionInformation. Такой стиль чтения отчета превращает limitation в actionable next step, а не в слепую зону.

Correlation остается coarse, а не per-function proof graph

После static analysis and dynamic validation возникает соблазн сказать: “если оба слоя что-то показали, значит доказано”. Но current correlation layer in evidence matrix intentionally conservative and coarse. Coarse означает грубый, крупнозернистый. Он связывает не каждую отдельную функцию с отдельным runtime route, а классы evidence. Например, suspicious static handler-related evidence plus runtime handler/VEH/VCH/UEF hits can establish static/dynamic correlation. Static placement anomalies plus runtime dynamic function-table activity can also correlate. Но это не per-function proof graph.

Per-function proof graph означал бы, что для каждой RUNTIME_FUNCTION entry, handler RVA, dynamic event and RIP trace построена точная причинная цепочка: вот статическая запись, вот runtime address, вот event, вот handler hit exactly corresponding to this record, вот post-handler flow, вот final conclusion. Текущая система делает более осторожную вещь: она говорит, что static and runtime evidence are related at a broader case level. Это полезно, но не должно быть переупаковано как математическое доказательство каждой функции.

Поэтому assessment должен сохранять нюанс. strong_correlated_case не означает “все подозрительные функции подтверждены runtime”. Он означает: есть static/dynamic correlation по правилам проекта, and either static or runtime evidence is strong. medium_correlated_case означает более умеренную связку. strong_unconfirmed_case означает, что сильное evidence есть, но cross-layer correlation was not established. Эти labels помогают читать дело целиком, но они не заменяют manual validation of specific address ranges.

Reports обязаны различать отсутствующий блок, нулевой результат и неполную сессию

В предыдущем разделе мы уже говорили о report contract, но в теме limitations это нужно закрепить отдельно. Отсутствие блока - это одно состояние. Нулевые counters в присутствующем блоке - другое. Failed or incomplete session - третье. Если consumer отчета смешивает эти состояния, он начинает обманывать пользователя.

```
dynamic_validation absent:
  dynamic mode не запускался или не включен в отчет

dynamic_validation present, ran=false:
  dynamic mode requested, но target не был создан

dynamic_validation present, ran=true, ok=false:
  session началась, но закончилась failure/timeout/incomplete

dynamic_validation present, ok=true, events=[]:
  session завершилась без сохраненных relevant events
```

Такая разница кажется мелкой только до первого автоматического dashboard-a. Если UI пишет “0 handler hits” там, где dynamic_validation absent, пользователь может подумать, что runtime проверка была проведена and handler did not fire. Правильная фраза: dynamic validation not run. Если session timed out, правильная фраза: dynamic validation incomplete; observed events are partial. Если dropped_events > 0, правильная фраза: event log truncated. Честный report не должен делать неполные наблюдения визуально похожими на отрицательный результат.

То же относится к heuristic_analysis. Если heuristic stage absent, это не score zero. Если score zero, это значит, что heuristic stage ran and no score-increasing findings were emitted. Если weakening_rules_applied non-empty, это не “проблемы исправлены”, а “confidence lowered due to isolation/sample/benign explanation”. Report должен учить читателя этим различиям. Иначе даже правильно собранные данные будут прочитаны неправильно.

Packed, manual-mapped and runtime-generated code разрывают статическую видимость

Глава 8 уже подробно говорила о packers, manual mapping and runtime code. В разделе об ограничениях нужно соединить это с architecture analyzer-a. Static analysis starts from file bytes. Но packed program может хранить real payload compressed or encrypted, a on startup unpack it into memory. Manual mapper может загружать image-like code without registering all loader-visible structures. JIT or runtime code generator может создать executable pages after process start and register function tables through RtlAddFunctionTable or related API. Все это разрывает простую цепочку file bytes -> mapped image -> all code -> all unwind metadata.

Static analyzer в таких случаях может обнаружить признаки: small stub, unusual sections, writable/executable memory indicators in metadata, missing expected coverage, dynamic function table APIs in imports, suspicious handler dominance. But it cannot reconstruct all runtime code from nothing unless payload is statically recoverable. Windows dynamic collector can observe runtime activity if scenario reaches it: dynamic function table registration, exceptions in generated code, handler hits, RIP in private executable memory. Но если route не выполнен in current session, dynamic validation also cannot claim full absence.

Здесь особенно важно language discipline. Нельзя писать “payload has no unwind metadata” только потому, что file .pdata did not cover it. Правильнее: static analysis did not observe file-backed unwind metadata for this runtime region; dynamic validation did or did not observe dynamic function table registration in this session. Такая фраза длиннее, но она честнее. Она говорит именно то, что

мы знаем, and does not silently assume loader model that may not hold.

Manual validation steps как продолжение автоматического вывода

Когда автоматический слой доходит до границы, его задача не просто поставить точку. Он должен передать работу человеку в понятной форме. Поэтому manual_validation_steps в assessment - важная часть проекта. Manual validation - ручная проверка. Это не наказание пользователя за слабость tool-a, а нормальный следующий этап там, где автоматический вывод не имеет права закрывать вопрос.

Хороший manual step должен быть привязан к evidence. Если finding говорит unwind_rva_oob, manual step может предложить проверить Exception Directory size, section mapping, raw bytes around table, compare with PE viewer. Если finding говорит handler_nonexec_section, manual step должен вести к section permissions, handler RVA mapping, disassembly at target address and possible parse mismatch. Если dynamic validation timed out, manual step может предложить run with longer timeout, narrower input, VM snapshot, targeted breakpoint or reproduce under full debugger. Если dropped_events > 0, manual step should mention event truncation and need higher event budget in future version.

Такой подход сохраняет педагогическую ценность отчета. Analyzer не говорит “я не знаю, разбирайся сам”. Он говорит: “вот где заканчивается моя автоматическая уверенность; вот какие observations у меня есть; вот какие states remain unknown; вот как ты можешь продолжить”. Это именно тот стиль, который нужен и книге, и проекту: не магический verdict, а обучающая evidence chain.

Главная дисциплина языка

Самая большая практическая польза этого раздела - не в перечне limits. Польза в дисциплине формулировок. В reverse engineering маленькая неаккуратность языка быстро превращается в большую ошибку вывода. “Не найдено” начинает звучать как “нет”. “Не наблюдалось в этой session” превращается в “не бывает”. “Score high” превращается в “malware”. “Confidence” превращается в “вероятность”. “Dynamic validation incomplete” превращается в “runtime clean”. Именно от таких превращений хороший анализатор должен защищать пользователя.

Правильный язык выглядит осторожнее, но он не слабее. Он сильнее, потому что его можно проверить. Static analysis observed. Heuristic/evidence engine interpreted. Dynamic validation observed during this session. Report

serialized with provenance. Assessment correlated where rules allow. Unknown remains unknown. Manual validation steps explain what to do next. Такая цепочка не обещает невозможного, зато она не ломается при первом сложном бинаре.

В конечном счете EH/Unwind Analyzer ценен не тем, что “находит все странное”. Такой инструмент невозможен. Он ценен тем, что проходит по сложному слою Windows x64 exception/unwind mechanics, извлекает факты, группирует evidence, отделяет score от confidence, отличает static from runtime, сохраняет session limits, показывает reports and говорит пользователю, где автоматический вывод заканчивается. Это и есть зрелая инженерная позиция.

Переход к финальной сборке

Теперь архитектурная глава получила необходимую честность. Мы разобрали pipeline static analysis, evidence model, dynamic validation, correlation, reports and limitations. Следующий крупный шаг - собрать всю книгу обратно в единую mental model Windows x64 exception/unwind. Нужно пройти путь от RIP and RSP к функции, от функции к RUNTIME_FUNCTION, от RUNTIME_FUNCTION к UNWIND_INFO, от UNWIND_INFO к RtlVirtualUnwind, от исключения к handler dispatch, and then to report evidence. Только после такой сборки проектная архитектура EH/Unwind Analyzer будет восприниматься не как набор C-файлов, а как естественное продолжение изученной системы.

Поэтому следующая глава должна быть не “еще одним списком компонентов”, а итоговой картой. Мы уже знаем детали. Теперь надо показать, как они соединяются: как Windows использует metadata, как реверсер читает эти следы, как static analyzer extracts declared model, how heuristic/evidence engine assigns meaning, how dynamic validation observes runtime route, and how reports preserve the evidence chain. Это будет мост от учебника к финальной проектной главе.

Техническая база раздела: docs/ARCHITECTURE.md, docs/evidence_matrix.md, docs/report.schema.json, include/report.h, src/report.c, include/dynamic.h, src/dynamic_win.c, src/heuristics.c. Внешняя техническая база: Microsoft PE Format, x64 exception handling, debugger main loop, debugger exception handling, ContinueDebugEvent and RtlAddFunctionTable documentation.

Глава 10. Сборка модели Windows x64 exception/unwind

10.1. Полный маршрут от RIP до unwind-решения: как один адрес превращается в понимание кадра

В предыдущих главах мы долго разбирали отдельные части Windows x64 exception/unwind-модели: PE, секции, RVA, .pdata, .xdata, RUNTIME_FUNCTION, UNWIND_INFO, пролог функции, сохранённые регистры, frame-based SEH, VEH, top-level handling, static analysis, heuristic/evidence engine и dynamic validation. Каждая часть сама по себе уже достаточно сложная. Но настоящий смысл этой системы появляется только тогда, когда мы перестаём смотреть на термины как на набор независимых карточек и собираем их в одну линию движения.

Эта линия начинается с очень маленькой вещи - с одного адреса исполнения. В отладчике это будет значение регистра RIP. В crash dump это будет адрес инструкции, на которой поток остановился. В отчёте анализатора это может быть VA, RVA или адрес внутри диапазона функции. На первый взгляд кажется, что адрес - это просто число. Но в reverse engineering число почти никогда не является ответом само по себе. Адрес становится знанием только тогда, когда мы понимаем, в каком модуле он находится, к какому диапазону кода относится, какая unwind-запись описывает этот диапазон, как был построен стековый кадр и каким образом Windows сможет выйти из текущего кадра к кадру вызывающей функции.

Поэтому в этом разделе мы не будем бежать по списку структур. Мы пройдём маршрут медленно, как будто перед нами остановленный поток Windows x64 и нам нужно не угадать, а восстановить картину. Мы будем двигаться от живого состояния процессора к PE-метаданным, от PE-метаданных к модели кадра, от модели кадра к восстановлению caller context и только потом к обработчикам исключений. Именно так эта тема начинает становиться не набором слов, а рабочей mental model - внутренней моделью, которой пользуется реверсер и на которой строится статический и динамический анализ в твоём проекте.

Остановленный поток как исходная сцена анализа

Представим не абстрактную программу, а конкретный момент. Поток выполнялся, дошёл до некоторой машинной инструкции и остановился: из-за access violation, breakpoint exception, deliberate RaiseException, single-step события или просто потому, что debugger поставил breakpoint. В этот момент у нас появляется снимок состояния потока. Слово “снимок” здесь важнее, чем кажется. Процессор в каждый момент имеет набор регистров, флагов и указателей. Если мы хотим понять, где поток находится и куда он может вернуться, нам недостаточно смотреть только на дизассемблер. Нужно посмотреть на состояние машины.

Центральный регистр в этой сцене - RIP. Английское имя RIP исторически раскрывается как instruction pointer, указатель инструкции. В 64-битном режиме x86-64 этот регистр хранит адрес инструкции, которая выполняется сейчас или будет выполнена следующей в зависимости от момента наблюдения. По-русски его удобно понимать как “текущая точка исполнения”. Но если сказать только “RIP - адрес текущей инструкции”, мы почти ничего не объяснили. Нужно добавить: RIP - это та точка, от которой начинается расследование текущего кадра. Через неё Windows понимает, в какой функции находится поток, какие unwind metadata надо применить, есть ли у этой функции language-specific handler, и как из этого кадра выйти к вызывающему кадру.

Рядом с RIP почти всегда должен стоять RSP. RSP - stack pointer, указатель стека. Он показывает на текущую вершину стека потока. Стек в обычном вызове функций хранит временные данные, spill-регистры, локальные области, выравнивание, а главное для нашей темы - адрес возврата, return address. Если RIP говорит “где мы сейчас”, то RSP помогает понять “как сюда пришли и куда можно вернуться”. В нормальном линейном выполнении функция завершилась бы инструкцией ret, которая взяла бы return address со стека и перенесла бы управление туда. Но exception/unwind-модель существует именно потому, что Windows часто нужно восстановить caller context без выполнения обычного ret.

Эта фраза - ключевая. Обычный ret - это действие самой программы. Он реально меняет RIP и RSP в процессе исполнения. Unwind - это реконструкция. Windows или debugger не обязаны исполнять тело функции до конца. Они могут находиться в середине функции, в момент исключения, и всё равно должны понять, как выглядел бы контекст вызывающей функции, если бы текущий кадр был корректно снят. Значит, им нужна внешняя инструкция: как отменить эффекты пролога, где лежат сохранённые регистры, сколько байт выделено на стек, откуда взять return address. Этой внешней инструкцией и является unwind metadata.

Мини-сцена

Представь, что в crash dump мы видим RIP = 0x00007FF6A1234567 и RSP = 0x0000008F3A7FF120. Эти два числа ещё не рассказывают историю. Они только дают две точки: где поток остановился и где сейчас находится вершина его стека. Настоящая работа начинается дальше: нужно понять, как этот RIP соотносится с загруженным модулем, какая функция покрывает этот адрес, как эта функция изменила RSP в прологе, какие регистры она сохранила и где теперь лежит return address. Только после этого можно говорить, кто вызвал текущую функцию и как будет выглядеть caller context.

Адрес без модуля остаётся сиротой

Новичок часто делает первую ошибку уже здесь: он видит адрес и начинает искать его в файле как file offset. Это почти всегда неправильный ход. Адрес, который мы видим в RIP во время выполнения, - это virtual address, виртуальный адрес в адресном пространстве процесса. Он относится не к файлу на диске, а к уже загруженной в память картине. В Windows каждый процесс имеет своё виртуальное адресное пространство, и один и тот же PE-файл может быть загружен по разным базовым адресам из-за ASLR - Address Space Layout Randomization, рандомизации расположения адресного пространства.

Поэтому адрес без модуля ничего не значит в инженерном смысле. Число 0x00007FF6A1234567 может быть адресом внутри main executable, внутри DLL, внутри JIT-generated code, внутри manual-mapped области или вообще внутри памяти, которая не соответствует обычному PE-модулю. Пока мы не нашли module - модуль, то есть загруженный образ executable или DLL, - мы не можем честно перевести этот адрес в RVA и не можем перейти к PE-структурам этого модуля.

Слово module в этом контексте не стоит переводить слишком бытово. Это не просто “кусок программы”. Для Windows loader-а модуль - это загруженный image: исполняемый файл или DLL, отображённый в виртуальную память процесса. У модуля есть base address, границы image, секции, заголовки, data directories. Когда debugger или dynamic validation строит module map, он не просто собирает красивые имена DLL. Он создаёт карту мира, в котором каждый runtime VA можно попытаться привязать к конкретному образу.

ImageBase - базовый адрес образа - это точка, от которой считаются RVA внутри загруженного модуля. В PE-файле есть preferred image base, предпочитаемый базовый адрес, но во время запуска loader может загрузить образ по другому адресу. Поэтому для runtime-анализа важен не только тот ImageBase, который записан в Optional Header, а фактический runtime base, который получил модуль в данном процессе. Static analysis работает с тем, что заявлено в файле. Dynamic validation видит реальную загрузку в процессе и поэтому может сказать, где этот модуль живёт сейчас.

```
Runtime VA      = адрес в живом процессе, например значение RIP
Module base     = фактический базовый адрес загруженного модуля
RVA             = Runtime VA - Module base
File offset     = смещение в файле на диске, вычисляется через section table
```

Эти четыре строки похожи на простую арифметику, но за ними стоит вся дисциплина PE-анализа. VA, или virtual address, живёт в памяти процесса. RVA, relative virtual address, - относительный виртуальный адрес внутри образа: смещение от base address. File offset - смещение в файле на диске. Эти три системы координат нельзя смешивать. Если static analyzer перепутает RVA с file offset, он начнёт читать не те байты. Если dynamic validation перепутает runtime VA с RVA, он поставит breakpoint не туда. Если реверсер не понимает, какой адрес перед ним, он будет думать, что “код исчез” или “таблица сломана”, хотя на самом деле он просто перешёл в другую систему координат.

Поэтому первый глубокий вывод такой: маршрут от RIP к unwind-решению начинается не с .pdata. Он начинается с географии адресного пространства. Сначала адрес должен получить свой мир: модуль, base, RVA, секцию. Только после этого можно говорить о PE-структурах и unwind metadata.

RVA как способ отвязать анализ от конкретного запуска

RVA - relative virtual address - обычно переводят как “относительный виртуальный адрес”. Но сухой перевод мало помогает. Гораздо важнее понять роль RVA. Это язык PE-файла, который позволяет описывать внутренние адреса образа без привязки к тому, где именно loader разместил образ в памяти. Если сегодня модуль загружен по 0x00007FF6A0000000, а завтра по 0x00007FF700000000, RVA функции останется тем же самым. Меняется только base, к которому этот RVA прибавляется.

Именно поэтому в PE так много полей записано в RVA. Exception Directory указывает не на абсолютный адрес в памяти конкретного запуска, а на область внутри image. RUNTIME_FUNCTION хранит begin/end/unwind адреса как image-relative значения. UNWIND_INFO ссылается на handler RVA. Это не случайная экономия байтов. Это способ сделать metadata переносимой внутри образа: файл описывает собственную внутреннюю карту, а loader уже решает, где эта карта окажется в конкретном процессе.

Но отсюда появляется и практическая опасность. Когда мы работаем с файлом на диске, RVA ещё надо превратить в file offset. Для этого используется section table - таблица секций. Секция говорит: такой-то виртуальный диапазон образа соответствует такому-то raw range в файле. Если RVA попал в виртуальный диапазон секции, можно вычислить, где соответствующие байты лежат на диске. Если RVA указывает за пределы всех секций или внутрь странного диапазона, static analyzer не имеет права молча читать произвольные байты. Он должен сформировать finding или validation error, потому что файл заявил ссылку, которую нельзя честно разрешить.

Вот здесь уже видна связь с heuristic/evidence engine. Ошибка адресной арифметики - не мелочь реализации. Это граница между честным анализом и самообманом. Если анализатор неверно разрешил RVA, все последующие выводы о .pdata, .xdata, handler data и findings будут построены на песке. Поэтому хороший static analysis сначала строит надёжную section/RVA mapping модель, и только потом идёт к Exception Directory.

Exception Directory как официальный вход в unwind-карту

Когда адрес привязан к модулю и превращён в RVA, появляется следующий вопрос: где у этого модуля находится карта функций, нужная для unwind? Наивный ответ - “в секции .pdata”. Часто это действительно так, но для анализатора такой ответ недостаточен. В PE не имя секции является официальным входом, а data directory. Data Directory можно понимать как оглавление важных системных областей образа. В нём есть запись Exception Directory. Именно она говорит Windows и инструментам: вот где в этом image находится массив записей, описывающих exception/unwind информацию для функций.

Exception Directory - это не обработчик исключений и не список crash-ов. Его название может сбивать. В PE32+ для AMD64 эта directory указывает на function table, обычно представленную массивом RUNTIME_FUNCTION. Слово function table тоже надо понимать аккуратно. Это не таблица имён функций, не symbol table и не список всего, что дизассемблер назвал функцией. Это таблица диапазонов кода, для которых Windows может найти unwind metadata.

Почему важно идти через Exception Directory, а не просто искать секцию с именем .pdata? Потому что имя секции - это соглашение и удобство, а не абсолютный контракт. Packers, custom linkers, unusual toolchains, manual transformations могут переименовать секции или разместить данные необычно. Static analyzer должен анализировать то, что PE официально заявляет в directory, а имя секции использовать как дополнительный контекст. Если Exception Directory указывает в диапазон секции с необычными правами или странным именем, это уже отдельный факт анализа, но сам факт входа берётся из directory.

Когда Windows получает ControlPc - адрес управления, близкий к RIP, - ей нужно найти соответствующую function table entry. В user-mode API для этого существует RtlLookupFunctionEntry. Эта функция не “угадывает имя функции”. Она ищет запись active function tables, которая покрывает данный PC value. В обычном PE это будет запись из .pdata. Для runtime-generated code это может быть dynamic function table, зарегистрированная через RtlAddFunctionTable или связанную callback-модель. Поэтому к моменту поиска RUNTIME_FUNCTION мы уже должны понимать: таблица может быть статической в PE или динамической в памяти процесса.

RUNTIME_FUNCTION как граница ответственности, а не имя функции

RUNTIME_FUNCTION часто воспринимают слишком буквально: раз структура называется “runtime function”, значит это функция. Но в рамках unwind-модели это прежде всего диапазон ответственности. Запись говорит: начиная с BeginAddress и до EndAddress существует область кода, для которой надо применять вот этот UnwindInfoAddress. BeginAddress и EndAddress задают полуоткрытый диапазон: начало включено, конец обычно понимается как граница после последнего байта функции. UnwindInfoAddress ведёт к UNWIND_INFO, то есть к инструкции восстановления кадра.

Почему это не просто “имя функции”? Потому что PE-файл может быть stripped, без символов. У функции может не быть имени. Дизассемблер может по-своему разметить границы. Компилятор может создать thunks, hot/cold split, chained unwind info, tail-call участки. Но для Windows важна не красивая подпись, а ответ на практический вопрос: если ControlPc находится в этом диапазоне, как восстановить caller context? RUNTIME_FUNCTION даёт именно такую привязку: адрес исполнения внутри диапазона -> конкретная unwind-модель.

BeginAddress и EndAddress в этой структуре являются RVA, а не VA и не file offset. Это значит, что запись не содержит готового runtime адреса. Чтобы сравнить её с RIP, нужно привести всё к одной системе координат. Обычно runtime VA превращается в RVA вычитанием module base, а затем этот RVA сравнивается с begin/end. Если диапазон найден, UnwindInfoAddress тоже трактуется как RVA внутри image и переводится к байтам UNWIND_INFO через section mapping.

Здесь важно увидеть внутреннюю логику Windows. Система не смотрит на исходный C-код и не знает, как программист назвал функцию. Она видит адрес исполнения и таблицу диапазонов. Это очень инженерная модель: пока ControlPc находится внутри такого-то range, используется такая-то инструкция unwind. Если диапазон не найден, Windows может применить leaf-function assumption - предположение, что функция является leaf-функцией и её можно unwound как простой return. Но если это предположение применено к настоящей nonleaf-функции, стековая картина может стать ложной.

Маленький переход от числа к смыслу

```
RIP runtime VA:          0x00007FF6A1234567
Module runtime base:     0x00007FF6A1200000
RVA:                     0x0000000000034567

RUNTIME_FUNCTION:
    BeginAddress         0x00034000
    EndAddress           0x00034980
    UnwindInfoAddress    0x0008A120
```

В этой маленькой сцене RIP сам по себе ещё был только числом. После вычитания module base он стал RVA внутри модуля. После сравнения с BeginAddress и EndAddress он попал в конкретную RUNTIME_FUNCTION-запись. После чтения UnwindInfoAddress он получил связь с UNWIND_INFO. Именно так “адрес падения” превращается в вход к модели кадра.

UNWIND_INFO как описание прошлого, нужное в будущем

UNWIND_INFO - один из тех терминов, который легко выучить формально и не понять по-настоящему. Его часто описывают как структуру с полями Version, Flags, SizeOfProlog, CountOfCodes, FrameRegister, FrameOffset и массивом UNWIND_CODE. Всё это верно, но если остановиться на полях, мы снова получим конспект. Смысл UNWIND_INFO глубже: это компактное описание того, что функция сделала со стеком и сохранёнными регистрами в своём прологе, чтобы Windows могла потом мысленно отменить эти действия.

Пролог функции - это начальный участок, где callee, то есть вызываемая функция, строит свой кадр. Она может уменьшить RSP, чтобы выделить место под локальные данные. Она может сохранить nonvolatile-регистры на стек, потому что обязана вернуть их caller-у в прежнем виде. Она может установить frame pointer через RBP или другой регистр. В обычном выполнении всё это будет отменено эпилогом: RSP вернётся назад, сохранённые регистры будут восстановлены, ret заберёт return address. Но при исключении поток может остановиться в середине body, далеко до эпилога. Тогда Windows должна знать, как выглядели эффекты пролога, чтобы отменить их без исполнения кода функции.

Вот почему UNWIND_INFO - это не комментарий компилятора и не подсказка для отладчика. Это часть runtime-контракта между скомпилированным кодом и Windows. Компилятор породил машинный код, который изменил стек определённым образом. Одновременно он обязан породить metadata, которая описывает эти изменения в формате, понятном операционной системе. Если код и metadata расходятся, обычный ret в конце функции может всё ещё работать, но exception handling, stack walking, crash dump analysis и dynamic validation начнут видеть искажённую картину.

Массив UNWIND_CODE внутри UNWIND_INFO не является дизассемблером. Он не хранит все инструкции пролога. Он хранит эффекты, важные для обратного восстановления кадра: push nonvolatile register, allocate stack space, save register at stack offset, establish frame register и другие операции. Поэтому одно из важных учебных различий такое: дизассемблер показывает, какие инструкции реально лежат в .text; UNWIND_INFO показывает, какие последствия этих инструкций Windows должна уметь отменить.

CONTEXT как снимок живой машины

Теперь вернёмся к остановленному потоку. Чтобы сделать unwind, Windows нужна не только таблица metadata, но и текущее состояние процессора. Это состояние передаётся через структуру CONTEXT. Английское слово context переводится как “контекст”, но в техническом смысле здесь это не абстрактная обстановка, а конкретный снимок регистров и части процессорного состояния потока. В нём есть RIP, RSP, nonvolatile-регистры, флаги и другие поля в зависимости от выбранных context flags.

CONTEXT можно представить как фотографию машины в момент остановки. На фотографии видно, где сейчас исполняется поток, где его стек, какие значения держат регистры. Но фотография сама по себе не говорит, как откатиться на один кадр назад. Для отката нужна инструкция. Эту инструкцию даёт UNWIND_INFO, найденный через RUNTIME_FUNCTION. Поэтому RtlVirtualUnwind работает не с одной структурой и не с одним адресом. Он соединяет текущий CONTEXT, найденную function entry, image base и unwind metadata, чтобы вычислить новый CONTEXT - контекст вызывающей функции.

Caller context - контекст вызывающей функции - тоже надо понимать не как “куда реально прыгнула программа”. Это вычисленная модель состояния, которое соответствовало бы caller-кадру. В обычном выполнении callee дошла бы до ret, ret взял бы return address со стека и продолжил выполнение caller-а. При virtual unwind код не исполняется. Система читает metadata и память стека, восстанавливает сохранённые nonvolatile-регистры, вычисляет новый RSP и берёт return address как новый RIP. Поэтому результат похож на ret по смыслу, но достигается совсем другим путём.

RSP, return address и стек как цепочка возвращения

Чтобы понять unwind, нужно хорошо почувствовать стек. На Windows x64 стек растёт вниз: когда функция выделяет место, RSP обычно уменьшается. Инструкция call передаёт управление callee и одновременно помещает на стек return address - адрес инструкции, куда нужно вернуться после завершения callee. Этот return address является мостом между кадрами. Пока функция выполняется, он лежит где-то в её стековой области, но из-за пролога текущий RSP уже может быть далеко ниже него.

Возьмём простую ситуацию. Caller вызывает callee. До call у caller есть свой RSP. Call кладёт return address на стек и уменьшает RSP. Callee начинает пролог: сохраняет RBX, RBP или R12, выделяет место через sub rsp, 0x80, возможно выравнивает стек. Через несколько инструкций RSP уже указывает не на return address, а на нижнюю часть локальной области callee. Если прямо сейчас случится exception, простое чтение [RSP] как return address даст мусор или локальные данные. Windows поэтому не может “просто сделать ret в голове”. Она должна знать, сколько именно callee вычла из RSP и какие

значения положила выше.

UNWIND_INFO позволяет восстановить движение назад. Если metadata говорит, что функция выделила 0x80 байт стека и сохранила RBX по определённому offset, virtual unwind может прибавить к RSP нужный размер, восстановить RBX из памяти и дойти до места, где лежит return address. Затем этот return address становится новым RIP в caller context. Новый RSP становится таким, каким он был бы после снятия return address или в состоянии, соответствующем вызывающему кадру, в зависимости от точного шага unwind-механики.

Здесь легко сделать ошибку понимания. Return address - это не “адрес функции caller”. Это адрес продолжения после call. Caller мог вызвать callee из середины своей функции. Поэтому после unwind новый RIP указывает не на начало caller-функции, а на инструкцию после call-site, то есть после места вызова. Чтобы найти RUNTIME_FUNCTION для caller-а, Windows затем повторяет тот же процесс: новый RIP снова привязывается к module/RVA, ищется covering function entry, читается его UNWIND_INFO, и так строится stack walk вверх по цепочке вызовов.

Nonvolatile-регистры как обещание callee перед caller-ом

Nonvolatile-регистры - ещё один термин, который нельзя просто бросать. В calling convention, соглашении о вызовах Windows x64, регистры делятся на volatile и nonvolatile. Volatile можно перевести как “летучие” или “не сохраняемые”: caller не должен ожидать, что после вызова они останутся прежними. Nonvolatile - “нелетучие”, сохраняемые: если callee хочет использовать такой регистр, она обязана вернуть его caller-у в прежнем состоянии.

Это соглашение нужно для того, чтобы функции могли вызывать друг друга без полного сохранения всего процессорного состояния каждый раз. Caller знает: RAX, RCX, RDX и ряд других регистров могут измениться. Но RBX, RBP, RSI, RDI, R12-R15 должны быть сохранены, если callee их трогает. Поэтому пролог nonleaf-функции часто содержит push rbx, push rbp, mov [rsp+offset], r12 или похожие инструкции. Это не просто технические детали компилятора. Это договор между функциями.

При обычном возврате callee сама восстанавливает эти регистры в эпилоге. При unwind Windows должна восстановить их вместо callee, потому что callee может не дойти до эпилога. Если UNWIND_INFO неверно описывает, где сохранён RBX или R12, caller context получится повреждённым. Stack trace может пойти дальше, но значения регистров будут ложными. Для crash analysis это может означать неправильные локальные выводы. Для exception handling это может означать неверное восстановление состояния при переносе управления.

Вот почему mismatch между кодом и unwind metadata так важен. Это не эстетическая проблема “таблица не совпала с дизассемблером”. Это проблема доверия к реконструкции прошлого. Если metadata говорит, что RBX сохранён по одному offset, а код сохранил его по другому, virtual unwind восстановит не тот RBX. Если metadata говорит, что stack allocation равен 0x40, а код вычел 0x90, return address будет искаться не там. Так из маленького расхождения рождается большой разрыв между реальным выполнением и моделью Windows.

RtlVirtualUnwind как реконструкция, а не исполнение

Теперь можно осмысленно подойти к RtlVirtualUnwind. Название содержит слово virtual, и оно здесь очень уместно. Это не реальный unwind в смысле выполнения инструкций функции назад. Процессор вообще не умеет “исполнить функцию назад”. RtlVirtualUnwind берёт текущее представление CPU context внутри функции и вычисляет представление context родительского, caller-кадра. То есть он строит следующий кадр stack walk-а математически и структурно, а не выполняя машинный код.

Чтобы это сделать, ему нужны несколько вещей. Нужен ControlPc - адрес управления, по которому определяется текущая функция. Нужен ImageBase, чтобы правильно интерпретировать RVA в function entry. Нужен RUNTIME_FUNCTION, найденный для ControlPc. Нужен текущий CONTEXT, где лежат RIP, RSP и регистры. Нужна memory stack, потому что сохранённые регистры и return address физически лежат в памяти. И нужен UNWIND_INFO, который объясняет, как эта функция строила кадр.

Сначала система определяет, где ControlPc находится относительно функции. Если адрес внутри пролога, ситуация одна: часть пролога могла уже выполниться, часть ещё нет, и unwind должен отменять только те действия, которые реально могли произойти. Если адрес внутри эпилога, Windows может распознать эпилог по инструкции потока и продолжить эффект выхода из функции. Если адрес в body, то применяется обычная модель: функция уже построила кадр, и нужно откатить его по описанию UNWIND_CODE. Эти различия важны, потому что одна и та же функция в разных точках исполнения требует разной интерпретации metadata.

Результат RtlVirtualUnwind - не “прыжок” программы. Это обновлённый CONTEXT. В этом обновлённом контексте RIP обычно становится return address, RSP сдвигается к caller-кадру, nonvolatile-регистры получают значения, восстановленные из стека. После этого можно повторить процедуру для следующего кадра. Так строится stack walk. Так debugger показывает call stack. Так crash dump tools поднимаются от места падения к цепочке вызовов. Так exception dispatcher может двигаться по кадрам, когда ищет frame-based handler.

Handler data как место, где кадр становится участником исключения

До этого момента мы говорили главным образом о восстановлении caller context. Но x64 exception/unwind-модель решает не только задачу stack walking. Она также участвует в обработке исключений. Если UNWIND_INFO имеет флаг UNW_FLAG_EHANDLER или UNW_FLAG_UHANDLER, после массива unwind codes появляется хвост, который указывает на language-specific handler и associated handler data. Здесь кадр перестаёт быть просто механическим участком стека и становится потенциальным участником маршрута исключения.

Language-specific handler - термин, который лучше раскрывать буквально: “обработчик, специфичный для языка или runtime-модели”. Windows не обязана знать семантику каждого языка. Она предоставляет общий механизм доставки exception и unwind, а компилятор/рантайм языка добавляет свой handler, который понимает свои таблицы. Для MSVC SEH таким обработчиком может быть __C_specific_handler. Для C++ exceptions - другой runtime handler. Поэтому handler RVA в UNWIND_INFO не всегда ведёт к пользовательскому блоку __except. Часто он ведёт к runtime-функции, которая уже читает language-specific data и решает, какой filter или handler block относится к текущему protected range.

Protected range - защищённый диапазон - это участок кода, для которого действует определённая логика обработки исключений. Filter - выражение или функция, решающая, подходит ли handler для данного exception. Handler block - код, который выполняется, если filter решил обработать исключение. Эти сущности могут не выглядеть как обычные стрелки CFG в декомпиляторе. Поэтому exception route - маршрут исключения - часто существует как скрытый путь управления: CPU не выполняет обычный jmp из faulting instruction в handler block, но Windows dispatcher через metadata и runtime handler может передать управление туда.

В этой точке маршрут от RIP до unwind-решения соединяется с маршрутом exception handling. Сначала ControlPc помогает найти текущую function entry и UNWIND_INFO. Затем metadata отвечает на два вопроса. Первый механический: как восстановить caller context? Второй семантический: есть ли у этой функции handler-логика, которую надо вызвать в search phase или unwind phase? Если handler отсутствует, dispatcher поднимается выше. Если handler есть, language-specific handler получает ExceptionRecord, ContextRecord, DispatcherContext и свои handler data, чтобы решить, относится ли исключение к этому protected range.

Один и тот же маршрут глазами Windows, реверсера и анализатора

Теперь полезно посмотреть на один и тот же маршрут с трёх сторон. Windows смотрит на него как на рабочий механизм исполнения: есть ControlPc, active function tables, RtlLookupFunctionEntry, RtlVirtualUnwind, exception dispatcher, language-specific handlers. Для Windows это не теория, а инфраструктура, без которой не получится корректно обработать исключение, построить стек или выполнить unwind through frames.

Реверсер смотрит на ту же систему иначе. Он начинает с адреса в отладчике, crash dump или дизассемблере. Ему нужно понять, что это за модуль, какой RVA, какая функция покрывает этот адрес, где её запись в .pdata, что лежит в .xdata, соответствует ли UNWIND_INFO реальному прологу, есть ли handler flags, куда ведёт handler RVA, и почему декомпилятор показывает или не показывает exception edges. Для реверсера unwind metadata - это не только системная служебная информация, а источник структуры программы.

Static analysis в EH/Unwind Analyzer смотрит на эту же цепочку как на проверяемую модель файла. Он читает PE bytes, строит section/RVA mapping, находит Exception Directory, декодирует RUNTIME_FUNCTION и UNWIND_INFO, проверяет bounds, overlaps, placement, parse failures, reuse, handler dominance, ABI-related warnings. Его задача - не “угадать поведение программы”, а честно описать, что файл заявляет о своей unwind-модели и где эта заявленная модель выглядит нормальной, странной или повреждённой.

Dynamic validation смотрит уже на живой запуск. Windows dynamic collector не заменяет static analysis. Он наблюдает runtime evidence: какие модули реально загружены, какие exceptions произошли, какие handler hits были замечены, регистрировались ли VEH/VCH/UEF, вызывались ли dynamic function table API, куда ушёл RIP после handler-a. Это не магическое доказательство всей программы. Это живое свидетельство конкретной сессии, которое может подтвердить часть статической гипотезы, ослабить её или показать, что без дополнительного сценария наблюдений недостаточно.

Ручная сцена анализа падения

Соберём это в одну спокойную сцену. Пусть у нас есть crash: поток остановился на access violation. В ExceptionRecord указан ExceptionAddress, а в ContextRecord лежит RIP. Мы не начинаем с вывода “сломана функция”. Мы начинаем с географии: этот RIP находится внутри какого модуля? Если модуль найден, вычитаем runtime base и получаем RVA. Если модуль не найден, это уже отдельный факт: возможно, execution ушёл в private executable memory, generated code, unpacked payload или manual-mapped область.

Допустим, модуль найден. Теперь RVA сравнивается с Exception Directory function table. Если покрывающая RUNTIME_FUNCTION-запись найдена, мы читаем BeginAddress, EndAddress и UnwindInfoAddress. Begin/End говорят, какой диапазон кода Windows считает одной unwind-единицей. UnwindInfoAddress ведёт к UNWIND_INFO. На этом этапе static analysis может проверить, что RVA не выходит за image, что запись не пересекается с соседней, что .xdata доступна для чтения, что структура UNWIND_INFO синтаксически валидна.

Дальше мы смотрим на UNWIND_INFO как на рассказ о прологе. SizeOfProlog говорит, где заканчивается область пролога. CountOfCodes говорит, сколько slots занимает массив unwind operations. UNWIND_CODE показывает эффекты: сохранён такой-то nonvolatile-регистр, выделено столько-то байт стека, установлен frame register. Мы не просто читаем поля. Мы мысленно спрашиваем: если поток остановился в body функции, как эти эффекты надо отменить, чтобы вернуться к caller context? Где теперь должен лежать return address? Какие регистры нужно восстановить?

Если у UNWIND_INFO есть handler flags, сцена становится богаче. Теперь эта функция не только имеет способ восстановить кадр, но и может участвовать в exception route. Handler RVA указывает на language-specific handler, а handler data объясняет этому handler-у, какие protected ranges и filters существуют. Реверсер в этот момент должен быть особенно осторожен: пользовательский handler block может не быть прямой целью call/jmp. Он может быть достижим только через exception dispatcher. Поэтому обычный CFG без exception edges может быть неполным.

После этого можно представить шаг RtlVirtualUnwind. Текущий CONTEXT содержит RIP/RSP/регистры. Function entry и UNWIND_INFO говорят, как отменить кадр. Stack memory даёт сохранённые значения и return address. Результатом становится caller context. Если нужно построить stack trace, процесс повторяется для caller-а. Если идёт поиск SEH-handler-а, dispatcher использует эту же способность двигаться по кадрам, но дополнительно вызывает language-specific handlers там, где metadata говорит о наличии EHANDLER.

И вот теперь, только теперь, адрес падения стал осмысленным. Мы знаем не просто число RIP. Мы знаем модуль, RVA, функцию в смысле unwind range, модель кадра, способ восстановления caller context, потенциальное участие handler data и границы уверенности. Именно так один адрес превращается в понимание кадра.

Технический итог как компактная карта, а не замена объяснения

После длинного маршрута можно позволить себе короткую карту. Она не заменяет объяснение, а только фиксирует его в голове. Когда перед нами один runtime RIP, полный путь анализа выглядит так:

```
1. Runtime RIP belongs to some address space of a process.
2. Module map tells whether this VA belongs to a loaded image.
3. Runtime VA - module base gives RVA.
4. Exception Directory gives the function table entry range.
5. RUNTIME_FUNCTION maps code range to UnwindInfoAddress.
6. UNWIND_INFO describes how the current frame was built.
7. CONTEXT supplies current RIP/RSP/register state.
8. Stack memory supplies saved registers and return address.
9. RtlVirtualUnwind computes caller context.
10. Handler flags and handler data may add exception route semantics.
```

Главное здесь не в количестве шагов, а в их зависимости. Нельзя читать .xdata, не понимая RVA. Нельзя доверять return address, не понимая, как функция изменила RSP. Нельзя говорить о handler route, не понимая, что handler RVA часто ведёт к runtime handler-у, а не к пользовательскому блоку. Нельзя делать finding по одному странному байту, не понимая нормальной модели Windows. И нельзя считать runtime observation абсолютной истиной, если dynamic validation видела только один сценарий запуска.

Эта глава нужна для того, чтобы связать все предыдущие главы в одну рабочую мысль. Windows x64 exception/unwind - это не отдельная таблица, не отдельный handler и не отдельный API. Это цепочка доверия между машинным кодом, стеком, PE-метаданными и runtime-диспетчеризацией. Если реверсер видит эту цепочку, он перестаёт воспринимать .pdata и .xdata как служебный шум и начинает читать их как часть поведения программы.

Почему Windows не может обойтись простым disassembly

На этом месте полезно сделать ещё один шаг назад и спросить не как разработчик анализатора, а как архитектор системы: почему Windows вообще не может просто посмотреть на машинные инструкции и понять, как выйти из функции? Ведь инструкции лежат в .text. Теоретически можно было бы разобрать пролог, найти push, sub rsp, mov [rsp+...] и восстановить кадр по коду. Но такая идея красива только на бумаге. В реальной системе она слишком хрупкая.

Во-первых, инструкциям можно попасть в середину. Exception может произойти не только после завершения пролога, но и внутри него. Часть действий уже выполнена, часть ещё нет. Если просто прочитать весь пролог как завершённый, мы восстановим слишком много. Если проигнорировать пролог, восстановим слишком мало. Именно поэтому SizeOfProlog и

CodeOffset в unwind metadata имеют значение: они позволяют соотнеститекущий offset от начала функции с тем, какие эффекты пролога уже надосчитать произошедшими.

Во-вторых, один и тот же эффект может быть выражен разными инструкциями. Компилятор, оптимизация, hotpatch, security cookie, stack probing через __chkstk, frame pointer, сохранение XMM-регистров - всё это усложняет прямой анализ кода. Windows не хочет превращать exception dispatcher в полноценный дизассемблер с эвристиками. Ей нужен короткий, проверяемый, формальный контракт. Поэтому компилятор обязан оставить не просто код, а описание того, как этот код влияет на кадр.

В-третьих, при обработке исключения важна надёжность. Если поток уже упал, память может быть частично повреждена, стек может содержать странные значения, а код может находиться в необычном состоянии. Чем меньше диспетчер зависит от сложного анализа инструкций, тем лучше. UNWIND_INFO не делает систему абсолютно безопасной, но он задаёт формальную дорожку: для данного диапазона кода надо применить такие-то операции восстановления.

Отсюда рождается важное уважение к metadata. Новичок часто думает, что .pdata и .xdata - это “служебные таблицы для отладчика”. На самом деле это часть исполняемой модели Windows x64. Они не исполняются как инструкции, но влияют на то, как система понимает стек и исключения. Это особый слой программы: не code path, но path of understanding - путь понимания, по которому Windows реконструирует кадры.

Процесс поиска function entry как переход от точки к области

RIP указывает на точку. RUNTIME_FUNCTION описывает область. Между ними происходит очень важный переход мышления: мы перестаём смотреть на один байт инструкции и начинаем спрашивать, какая область кода несёт ответственность за этот байт. Эта область - не обязательно то, что IDA или Ghidra назвали функцией. Это область, для которой Windows имеет одну unwind-модель.

Если адрес попал в диапазон BeginAddress..EndAddress, это значит: для целей unwinding Windows должна применять именно эту запись. Внутри диапазона может быть много инструкций, basic blocks, условных переходов, calls, локальных branches. Но unwind-модель обычно одна. Она описывает кадр функции, а не каждую ветку control flow. Поэтому RUNTIME_FUNCTION - это мост от instruction-level view к frame-level view. Дизассемблер показывает инструкции. RUNTIME_FUNCTION показывает границы кадра.

Для реверсера это меняет способ чтения. Когда ты видишь crash на конкретной инструкции, не надо сразу закапываться только в локальную

логику этой инструкции. Нужно параллельно поднять голову и спросить: какаяfunction table entry покрывает эту инструкцию? Какой кадр был построен? Естьли handler? Если адрес находится в gap между записями, это leaf-functionassumption, thunk, padding, сгенерированный код или проблема покрытия?Один и тот же RIP может вести к разным выводам в зависимости от того, какон ложится на карту диапазонов.

Static analysis должен делать этот переход строго. Он не должен говорить “функция такая-то”, если у него есть только диапазон RUNTIME_FUNCTION. Он должен различать symbol/function name, disassembler function boundary и unwind range. В отчёте это особенно важно: если finding относится к RUNTIME_FUNCTION range, надо так и писать. Иначе человек прочитает report как утверждение о высокоуровневой функции, хотя анализатор говорил только о metadata range.

Prolog, body и epilog как разные состояния одной функции

Когда мы говорим “адрес внутри функции”, легко представить функцию как однородную область: начало, середина, конец. Но для unwinding это не так. Windows x64 различает по крайней мере три состояния: prolog, body и epilog. Они принадлежат одной функции, но означают разное состояние кадра.

Prolog - это участок построения кадра. Его можно сравнить с моментом, когда человек только раскладывает инструменты на рабочем столе. Стол ещё не в окончательном состоянии: часть инструментов уже лежит, часть ещё в руках. Если exception случился в прологе, нельзя считать, что весь кадр уже построен. Поэтому unwind procedure должна учитывать offset текущего RIP от начала функции и отменять только те операции пролога, которые уже произошли.

Body - тело функции - это основная область, где кадр уже построен. Здесь модель проще: предполагается, что пролог завершён, стековая область выделена, сохранённые nonvolatile-регистры лежат там, где metadata обещает. Если exception случился в body, unwind может применять полную модель кадра.

Epilog - участок выхода из функции. Это обратная сторона пролога: функция уже начала разрушать свой кадр. Она восстанавливает регистры, возвращает RSP, готовится к ret или jump-эпилогу. Если exception или stack walk попадает в эпилог, Windows не может трактовать состояние так же, как body. Документация описывает, что поток инструкций от текущего RIP может анализироваться для распознавания эпилога. Это один из немногих случаев, где Windows действительно смотрит на код, но делает это в очень ограниченном, формальном смысле: эпилоги на x64 обязаны иметь строгую форму, чтобы их можно было распознать.

Эта тройка prolog/body/epilog важна для ручного анализа. Когда ты смотришь на RIP, нельзя просто сказать “он внутри функции”. Надо спросить, внутри какой фазы кадра находится поток. Если RIP в prolog, часть сохранений могла ещё не случиться. Если в body, кадр обычно полностью построен. Если в epilog, часть кадра уже разобрана. Один и тот же UNWIND_INFO используется с учётом этой позиции.

Leaf-модель как запасной путь и источник ошибок

Теперь надо вернуться к ситуации, когда RtlLookupFunctionEntry не находит RUNTIME_FUNCTION. В Windows x64 это не обязательно немедленная катастрофа. Для leaf-функции отсутствие unwind-записи может быть нормой. Leaf function в unwind-смысле - это такая функция, для выхода из которой системе достаточно симулировать обычный return. Она не требует сложного восстановления nonvolatile-регистров и не имеет собственного frame-based exception handler-а, описанного через .xdata.

Но здесь очень легко ошибиться. В бытовом программировании leaf-функцией часто называют функцию, которая никого не вызывает. В unwind-модели важнее другое: может ли кадр быть восстановлен без metadata. Функция могла быть маленьким thunk-ом, короткой обёрткой, tail jump-ом, padding-переходом. В таких случаях отсутствие .pdata не равно поломке. Но если код уменьшает RSP, сохраняет RBX/R12, вызывает другие функции и при этом не имеет корректной RUNTIME_FUNCTION-записи, leaf-модель станет ложью.

Почему это опасно? Потому что fallback-модель будет искать return address там, где он был бы у простой функции. Но настоящая nonleaf-функция могла уже сдвинуть RSP далеко вниз. Тогда чтение return address пойдёт не из того места. Stack trace сломается, caller context будет ошибочным, exception dispatcher может не найти нужный handler выше по стеку. Ошибка проявится не обязательно как немедленный crash анализатора; она проявится как убедительная, но неверная картина.

Хороший static analysis не должен автоматически ругаться на любой uncovered code. Но он должен различать признаки leaf-like кода и признаки настоящего кадра без metadata. Именно здесь появляется инженерная осторожность: uncovered byte, uncovered function-like region и uncovered nonleaf candidate - разные уровни вывода. Первый может быть нормой. Второй требует контекста. Третий уже может быть серьёзным finding-ом.

Handler data не отменяет unwind, а добавляет смысл маршрута

Есть соблазн думать, что как только мы увидели handler flags, тема восстановления кадра закончилась и началась отдельная тема “обработчиков”. На самом деле это не две независимые системы. Handler data лежит поверх unwind-модели. Чтобы вызвать language-specific handler, Windows всё равно должна знать текущий кадр, establisher frame, context, dispatcher context. Механика восстановления и семантика обработки исключения соединены.

Establisher frame - термин, который стоит понимать как “кадр, установивший обработчик”. Это не обязательно красивый frame pointer в RBP. Это значение, через которое language-specific handler понимает, к какому экземпляру функции и к какой стековой области относятся handler data. Когда __C_specific_handler получает управление, он не просто спрашивает “какой exception code?”. Он должен связать exception address с protected ranges, найти filter, понять, есть ли подходящий handler block, и вернуть диспетчеру решение.

Search phase и unwind phase тоже нельзя смешивать. Search phase - фаза поиска: система поднимается по кадрам и спрашивает, есть ли обработчик, который хочет принять exception. Unwind phase - фаза раскрутки: когда обработчик выбран, надо снять промежуточные кадры, вызвать termination handlers/finally-подобную логику, восстановить состояние до нужного места. Один и тот же UNWIND_INFO может участвовать в обеих фазах, но смысл действий разный. В search phase важен вопрос “есть ли подходящий handler?”. В unwind phase важен вопрос “как корректно разрушить кадры на пути?”.

Exception route поэтому не является обычной стрелкой control flow. В обычном CFG переход возникает из инструкции jmp/call/ret. В exception route переход возникает из события исключения, metadata и решения dispatcher-а. Поэтому reverse engineering без учёта exception metadata может недостроить граф: handler block будет выглядеть как странный unreachable code или как отдельный кусок, хотя в реальном выполнении он достижим через exception dispatcher.

Связь с отчётом проекта без подмены теории документацией к коду

Когда эта модель попадает в EH/Unwind Analyzer, она не должна превращаться в набор голых полей. Static analysis берёт PE-файл и пытается пройти тот же логический маршрут, только без запуска: найти модульную географию внутри файла, построить RVA mapping, прочитать Exception Directory, разобрать RUNTIME_FUNCTION, декодировать UNWIND_INFO, проверить handler data и сформировать findings. Отчёт должен показывать не просто “вот значения”, а цепочку: где это найдено, в какой системе координат, к какому диапазону относится, какая норма ожидалась и где обнаружено отклонение.

Dynamic validation добавляет другой тип свидетельства. Она не читает .xdata глубже, чем static analysis; её задача не в этом. Она запускает target под Windows Debug API и наблюдает: какой модуль реально загружен, какое исключение пришло, first-chance оно или second-chance, был ли hit на handler-е, был ли вызван RtlAddFunctionTable, куда пошёл RIP после обработчика. Это runtime evidence - свидетельство живого выполнения. Но оно всегда привязано к сценарию запуска. Если handler не был hit, это не значит, что handler невозможен. Это значит, что в данном наблюдаемом маршруте он не проявился.

Assessment должен соединять эти два мира без преувеличений. Если static analysis нашёл EHANDLER и dynamic validation увидела hit на соответствующем runtime handler-e, case strength усиливается. Если static analysis нашёл подозрительный handler placement, но dynamic session была incomplete или timed out, нельзя делать сильный отрицательный вывод. Если dynamic validation увидела execution в private executable memory без соответствующего module/RVA mapping, это может объяснить, почему file-based .pdata не покрывает текущий RIP. Такой отчёт учит реверсера мыслить слоями, а не нажимать на красную кнопку “suspicious”.

Разбор одной функции на уровне пролога и unwind-смысла

Чтобы модель не оставалась общей, разберём вымышленную, но типичную функцию. Нам не важно, как она называлась в исходнике. Для Windows важнее другое: какой диапазон кода её покрывает и что она делает со стеком. Пусть в дизассемблере мы видим начало функции примерно такого вида:

```
push rbx
push rbp
sub  rsp, 0x80
lea  rbp, [rsp+0x20]
...
call qword ptr [rip+...]
```

```
...
add  rsp, 0x80
pop  rbp
pop  rbx
ret
```

Если смотреть только глазами человека, смысл понятен: функция сохраняет RBX и RBP, выделяет 0x80 байт стековой области, устанавливает RBP как frame register для удобного доступа к локальным данным, потом делает работу, вызывает другую функцию, а в конце возвращает стек и регистры назад. Но Windows не хочет каждый раз заново разбирать эти инструкции как текст. Компилятор должен записать те же эффекты в UNWIND_INFO.

Для push rbx появится unwind operation, смысл которой: nonvolatile-регистр RBX был сохранён на стек через push. Для push rbp - аналогично. Для sub rsp, 0x80 появится операция stack allocation: стек был уменьшен на 0x80 байт. Для lea rbp, [rsp+0x20] появится указание frame register и frame offset, если компилятор действительно использует RBP как frame register. Важно: metadata описывает не синтаксис инструкции, а её unwind-смысл. Инструкция lea сама по себе может использоваться где угодно, но в прологе она может означать установление frame register.

Теперь представим, что исключение случилось после call в body функции. Текущий RSP уже указывает внутрь выделенной области, а return address caller-а лежит выше: над локальной областью и сохранёнными регистрами. Если RtlVirtualUnwind просто прочитает [RSP], он получит не return address. Поэтому он применяет UNWIND_INFO: отменяет allocation, восстанавливает RBP и RBX из тех мест, где они были сохранены, и только потом добирается до return address. Это и есть смысл metadata: она возвращает стековую геометрию в состояние caller-а.

Если исключение случилось раньше, например сразу после push rbx, но до push rbp и sub rsp, ситуация другая. Нельзя отменять весь пролог, потому что большая часть его ещё не случилась. Поэтому CodeOffset в unwind codes важен не как декоративное поле, а как способ понять частично выполненный пролог. Windows должна отличать “RBX уже на стеке” от “RBP ещё не сохранён” и “0x80 байт ещё не выделены”. Без этого unwind в середине пролога был бы систематически неправильным.

Если исключение случилось в эпилоге, например после add rsp, 0x80, но до pop rbp, состояние тоже уже не похоже на body. Функция начала разбирать кадр. Поэтому x64 ABI накладывает ограничения на форму эпилога: система должна уметь распознать, что поток находится в последовательности выхода, и продолжить вычисление caller context с учётом уже начавшегося разрушения кадра. Это ещё одна причина, почему пролог/эпилог на Windows x64 нельзя писать произвольно, если функция должна быть unwindable.

Этот пример показывает, что RtlVirtualUnwind не просто “смотрит таблицу”. Он интерпретирует metadata относительно конкретной позиции RIP внутри

функции. Одна и та же функция, один и тот же UNWIND_INFO, но разные точкиостановки дают разные действия восстановления. Поэтому адрес внутрифункции важен не только для поиска RUNTIME_FUNCTION, но и для пониманияфазы кадра.

Ошибочная metadata как ложная память о функции

Полезно представить UNWIND_INFO как память Windows о том, что функция сделала в начале. Если эта память верна, система может восстановить путь назад. Если память ложна, система будет уверенно идти не туда. Это особенно неприятный класс ошибок: механизм не обязательно падает сразу. Он может построить стек, который выглядит правдоподобно, но относится не к реальному выполнению.

Допустим, функция реально сделала sub rsp, 0x80, а metadata говорит allocation 0x40. При virtual unwind RSP будет поднят недостаточно. Return address будет искаться слишком низко. На месте ожидаемого return address может лежать локальная переменная, сохранённый pointer, временный буфер. Если это значение похоже на canonical address, stack walker может принять его за адрес кода и продолжить строить ложную цепочку. Результат будет опаснее явной ошибки: человек увидит стек и поверит ему.

Другой пример - неверный offset сохранённого nonvolatile-регистра. Если RBX используется caller-ом как base pointer на важную структуру, а unwind восстановил RBX из неправильного места, последующий анализ caller context будет искажён. Даже если RIP/RSP выглядят нормально, регистровая часть контекста уже повреждена. Поэтому отчёт анализатора должен уметь объяснять consequence: не просто “UWOP mismatch”, а “восстановление caller context может получить неверное значение nonvolatile-регистра”.

Третий пример связан с handler flags. Если metadata говорит EHANDLER, но хвост UNWIND_INFO повреждён или handler RVA указывает в странную область, проблема уже затрагивает не только stack walking. Она затрагивает exception route. Search phase может вызвать не тот language-specific handler, не суметь прочитать handler data, ошибочно продолжить поиск или привести к crash during dispatch. Поэтому structural validity .xdata и semantic plausibility handler data - разные, но связанные уровни анализа.

Один адрес в обычном PE, packed PE и runtime-generated code

Один и тот же маршрут от RIP к unwind-решению ведёт себя по-разному в обычном PE, packed PE и runtime-generated code. В обычном сценарии RIP попадает в загруженный модуль, RVA ложится в section, Exception Directory указывает на .pdata, .pdata ведёт к .xdata, и весь путь остаётся внутри image. Это самый спокойный случай. Static analysis может заранее пройти почти тот же путь по файлу, а dynamic validation при запуске только подтверждает runtime base и события.

В packed PE ситуация меняется. Файл на диске может содержать stub - маленький загрузчик payload-а. У stub-а может быть корректная .pdata и .xdata. Static analysis честно опишет этот stub. Но настоящий payload может быть распакован в память позже, возможно в private executable region. Тогда RIP во время реального выполнения уйдёт в область, которую нельзя связать с исходной Exception Directory файла. Это не значит, что static analysis ошибся. Это значит, что его область видимости ограничена тем image, который был на диске.

Runtime-generated code, например JIT, ещё интереснее. Код может быть создан уже после запуска. У него нет .pdata в PE-файле, потому что самого кода не было в файле. Чтобы Windows могла корректно unwound такой код, runtime должен зарегистрировать dynamic function table через соответствующие механизмы. Тогда active function tables процесса будут шире, чем статическая Exception Directory PE. Dynamic validation может увидеть вызовы RtlAddFunctionTable или RtlInstallFunctionTableCallback и объяснить, почему адрес в runtime memory всё же имеет unwind-поддержку.

Для реверсера и отчёта вывод должен быть аккуратным. Если RIP не принадлежит file-backed module, это важный runtime факт. Но он не превращается автоматически в “вредоносность”. Нужно смотреть memory protection, origin, dynamic function table registrations, handler hits, module map, сценарий запуска. Хорошая книга должна научить именно такой осторожности: сначала восстановить цепочку, потом оценивать её качество.

Как хороший отчёт должен сопровождать этот маршрут

Если EH/Unwind Analyzer показывает этот маршрут в report, он не должен заставлять читателя самому догадываться, в какой системе координат находится каждое значение. Для каждого адресного факта полезно показывать слой: file offset, RVA или runtime VA. Для function entry полезно показывать begin/end как RVA и дополнительно секцию, в которую попадает диапазон. Для UNWIND_INFO полезно показывать placement: где лежит metadata, какие права у секции, валидна ли структура, есть ли handler flags.

Finding по этому маршруту должен быть маленьким рассказом. Например не “invalid_unwind_rva”, а: RUNTIME_FUNCTION for RVA range 0x34000-0x34980 references UnwindInfoAddress 0x8A120, but this RVA cannot be resolved to a readable file range through section mapping. Consequence: static analysis cannot decode declared unwind model for this function range; stack walking and exception handling assumptions for this range require manual review. Confidence: high for the address-resolution failure, lower for behavioral consequence unless runtime evidence confirms execution in this range.

Такое описание длиннее, но оно честнее. Оно разделяет observation, interpretation и consequence. Observation: RVA не разрешился. Interpretation: declared unwind model недоступна. Consequence: анализ кадра ограничен. Confidence: высокая для факта, осторожная для поведения. Если dynamic validation потом увидит RIP в этом диапазоне и exception event, case strength усилится. Если execution туда не заходил в данном запуске, finding остаётся static, но не исчезает.

Именно так теория превращается в инженерный инструмент. Не через магические verdict-ы, а через цепочку объяснений, где каждый адрес, каждая metadata-структура и каждое runtime-событие имеют своё место.

Типичные провалы мышления на этом маршруте

Первый провал - считать адрес самодостаточным. Адрес без module map - это координата без карты. Он может выглядеть точным, но не имеет контекста. Поэтому любой анализ RIP должен начинаться с вопроса о модуле и base address.

Второй провал - смешивать VA, RVA и file offset. Эта ошибка особенно коварна, потому что числа иногда выглядят правдоподобно. Анализатор может читать байты, которые существуют, но относятся совсем не к той структуре. В результате отчёт будет формально заполнен, но фактически построен на неверной адресной системе.

Третий провал - думать, что RUNTIME_FUNCTION означает исходную функцию. На практике это unwind range. Он может совпадать с функцией из исходника, а может не совпадать с тем, как дизассемблер или символы показывают код. Для выводов нужно аккуратно говорить: диапазон unwind-модели, function table entry, covering range.

Четвёртый провал - воспринимать UNWIND_INFO как второстепенную служебную структуру. Для Windows x64 это инструкция восстановления кадра. Если она неверна, страдает не красота отчёта, а способность системы восстановить caller context и пройти exception route.

Пятый провал - думать, что dynamic validation всё доказывает. Она наблюдает живой маршрут, но не все возможные маршруты. Хороший отчёт должен различать confirmed, observed, not observed, not tested, timed out,

incomplete. Это не слабость инструмента, а честность.

Техническая база раздела

Раздел опирается на официальную документацию Microsoft по x64 exception handling, где описаны RUNTIME_FUNCTION, UNWIND_INFO, UNWIND_CODE, handler flags, chained unwind info и поведение unwind procedure; на документацию RtlVirtualUnwind, где функция описана как получение invocation context вызывающей функции; и на PE Format, где описаны image files, sections, RVA и data directories.

Глава 10. Сборка модели Windows x64 exception/unwind

10.2. Полный маршрут исключения: от fault до unhandled path

Исключение как событие, а не как слово "ошибка"

В предыдущем разделе мы шли от одного RIP к unwind-решению. Мы брали остановленный поток, привязывали адрес к модулю, переводили VA в RVA, находили RUNTIME_FUNCTION, читали UNWIND_INFO и мысленно восстанавливали caller context. Это была вертикальная линия: из текущего кадра к вызывающему кадру. Теперь нужно собрать вторую линию - горизонтальную, событийную. Она отвечает не только на вопрос, как выйти из кадра, а на вопрос, что вообще происходит с исключением в Windows: кто первым получает шанс его увидеть, когда начинается поиск обработчика, где подключается debugger, где находится VEH, где начинается frame-based SEH, когда вызывается top-level filter и почему в конце иногда получается crash.

Слово exception часто автоматически переводят как "исключение", но в голове новичка оно быстро превращается в синоним аварии. Это опасное упрощение. В Windows exception - это не обязательно смерть процесса. Это форма сообщения о необычной ситуации в потоке. Иногда ситуация действительно аварийная: процесс попытался прочитать память по адресу, к которому у него нет доступа. Иногда это намеренный сигнал: программа сама вызвала RaiseException. Иногда это часть отладочной механики: breakpoint exception от инструкции int 3. Иногда это защитный или обфускационный трюк: программа специально создаёт исключение, чтобы управление попало в handler и вернулось уже в другой участок кода. Поэтому в этой главе мы будем считать исключение не "ошибкой", а событием управления, которое Windows должна доставить по строго определённому маршруту.

Маршрут исключения удобно понимать как цепочку остановок. Сначала появляется причина: процессорный fault, trap или программно созданное исключение. Затем система оформляет эту причину в структуры, которые можно передавать обработчикам: exception record и context record. Потом, если процесс находится под отладчиком, debugger получает first-chance notification - первый шанс увидеть событие до пользовательских обработчиков. Если debugger не забрал исключение себе, Windows передаёт его дальше: vectored exception handlers, затем frame-based обработчики, найденные через unwind-модель и metadata конкретных кадров. Если никто не обработал исключение, debugger может получить second-chance notification. После этого остаётся top-level/unhandled path, где процесс либо завершится, либо выполнит последний пользовательский фильтр, либо

передаст управление стандартному механизму ошибки.

Эта цепочка важна потому, что каждый слой отвечает за разный тип знания. Debugger видит событие снаружи и может остановить процесс до того, как код процесса начнёт реагировать. VEH - process-wide слой: он не привязан к конкретному стековому кадру и может смотреть почти на все исключения процесса. Frame-based SEH - слой кадров: обработчик принадлежит функции или области кода, а его доступность зависит от того, где находится RIP и какие кадры есть в стеке. Top-level handler - последний шанс, когда нормальный поиск обработчиков уже не дал результата. Если перепутать порядок этих слоёв, reverse engineering превращается в угадайку: ты видишь handler, но не понимаешь, почему он не вызвался; видишь first-chance exception и думаешь, что программа упала; видишь SetUnhandledExceptionFilter и думаешь, что он поймает всё, хотя до него exception может вообще не дойти.

Рождение события: processor fault, trap и software-generated exception

Исключение должно откуда-то взяться. Первая группа источников - сам процессор. Когда инструкция не может быть выполнена обычным образом, процессор создаёт аппаратное исключительное событие. Например, access violation возникает, когда поток обращается к памяти, которую нельзя читать, писать или исполнять с текущими правами. Divide-by-zero появляется при делении на ноль. Illegal instruction возникает, когда байты в потоке команд не являются допустимой инструкцией для текущего режима. Это не C++ exception и не __try/__except само по себе. Это низкоуровневый сигнал: машина не может продолжить как обычный поток инструкций.

Здесь полезно различать слова fault и trap. Fault можно понимать как " сбой до завершения инструкции": инструкция попыталась что-то сделать, но не смогла корректно завершиться. После обработки fault теоретически можно вернуться к той же инструкции, если handler исправил причину. Например, guard page может сгенерировать исключение, handler снимет или переустановит защиту, и выполнение может продолжиться. Trap ближе к событию после инструкции или вокруг контрольного события: breakpoint, single-step. В практическом reverse engineering это различие помогает не воспринимать все исключения одинаково. Одни exception sites указывают на настоящую ошибку доступа, другие - на отладочную или control-flow механику.

Вторая группа источников - software-generated exception, программно созданное исключение. Самый прямой пример - RaiseException. Эта функция не ждёт, пока процессор столкнётся с невозможной инструкцией. Она сама создаёт исключение в вызывающем потоке. Важно словосочетание "в вызывающем потоке". Исключение не разлетается магически по процессу. Оно принадлежит конкретному thread, то есть потоку выполнения, у которого

есть свой стек, свой RIP, свой RSP и свой CONTEXT. Поэтому exception routeвсегда надо мыслить thread-local: маршрут доставки идёт по состояниюконкретного потока, даже если некоторые обработчики зарегистрированы науровне процесса.

Программно созданное исключение может быть нормальным. Runtime языка может использовать его для своих целей. Приложение может использовать его как внутренний protocol. Защитный код может использовать его как anti-debug или control-flow trick. Поэтому сам факт RaiseException в импортах или дизассемблере ничего не доказывает. Он задаёт вопрос: зачем программа решила не вызвать функцию обычным call, а создать exception event и пустить управление через системный dispatcher? Ответ надо искать в exception code, handler logic, returned disposition и dynamic validation.

Exception record и context record как паспорт события и снимок потока

Когда исключение родилось, Windows не может передавать дальше просто фразу "что-то пошло не так". Системе нужен формализованный пакет данных. В этом пакете есть две принципиально разные части. Первая описывает само событие: какой exception code, где оно произошло, какие параметры с ним связаны. Вторая описывает состояние потока в момент события: регистры, RIP, RSP и другую машинную обстановку. В Windows эти роли обычно связывают с EXCEPTION_RECORD и CONTEXT, а вместе handler часто получает их через EXCEPTION_POINTERS.

EXCEPTION_RECORD можно понимать как паспорт исключения. В паспорте есть ExceptionCode - код, который говорит тип события: access violation, breakpoint, single-step, software exception и так далее. Есть ExceptionAddress - адрес, с которым событие связано. Есть ExceptionInformation - дополнительный массив параметров, смысл которого зависит от кода. Например, для нарушения доступа параметры могут объяснять тип доступа и адрес памяти. Важно: этот record не равен стеку и не равен маршруту. Он описывает событие, но не решает, кто его обработает.

CONTEXT - это снимок состояния потока. В нём находится машинная часть истории: RIP показывает точку исполнения, RSP показывает текущую вершину стека, регистры содержат значения на момент события. Если handler вернёт EXCEPTION_CONTINUE_EXECUTION и изменит CONTEXT, он потенциально может изменить продолжение выполнения. Это очень сильная возможность. Поэтому exception handling в Windows - не только механизм "поймать ошибку". Это ещё и механизм, через который можно менять продолжение потока, если система признаёт исключение обработанным и состояние потока стало пригодным для продолжения.

EXCEPTION_POINTERS - удобный контейнер, который даёт handler-у обе части сразу: pointer to EXCEPTION_RECORD и pointer to CONTEXT. Английское pointers здесь означает указатели: handler получает не копии красивых отчётных строк, а доступ к структурам, которые описывают живое событие. Для реверсера это объясняет, почему обработчик исключения так часто читает поля вроде ExceptionCode, ExceptionAddress, ContextRecord->Rip или ContextRecord->Rsp. Такой handler не просто логирует ошибку. Он принимает решение о маршруте: продолжить поиск, обработать и перейти в handler block, изменить context и продолжить выполнение, записать minidump или дать процессу упасть.

Debugger first chance как внешний первый свидетель

Если процесс не отлаживается, маршрут сразу пойдёт по внутренним слоям процесса. Но если процесс запущен под debugger-ом, появляется внешний наблюдатель. Debugger - отладчик - это процесс или инструмент, которому Windows разрешила получать debug events от debuggee. Debuggee - отлаживаемый процесс, то есть target, чьи потоки могут быть остановлены, пока debugger принимает решение. Эти слова надо понимать не как UI Visual Studio, а как системную связь: Windows удерживает выполнение debuggee и передаёт debugger-у событие.

First-chance notification - первый шанс debugger-а. Слово chance здесь буквально значит шанс, возможность. Windows ещё не дала пользовательским frame-based обработчикам решить судьбу exception. Сначала debugger получает событие и может сказать: я считаю это обработанным, продолжай как будто всё нормально, или я это не обрабатываю, передай дальше обычной exception dispatch logic. В Debug API это решение выражается через ContinueDebugEvent и статус продолжения. Если debugger возвращает DBG_CONTINUE для exception event, он фактически говорит, что exception handled. Если возвращает DBG_EXCEPTION_NOT_HANDLED на first chance, Windows продолжает обычный поиск обработчиков внутри процесса.

Эта деталь объясняет множество странных наблюдений в reverse engineering. Под отладчиком программа может вести себя иначе не только из-за IsDebuggerPresent. Сам факт first-chance notification меняет временную последовательность. Debugger видит breakpoint exception до VEH, до SEH и до top-level handler. Если debugger автоматически обрабатывает breakpoint, внутренний handler может его не увидеть. Если debugger пропускает exception дальше, тогда уже вступают внутренние механизмы. Поэтому dynamic validation должна фиксировать не просто "было исключение", а на каком chance оно наблюдалось, как debugger продолжил событие и дошёл ли exception до handler-а.

Для реверсера first chance часто выглядит пугающе: отладчик остановился, вывел exception message, и кажется, что программа упала. Но first-chance exception не равен crash. Это всего лишь первый момент, когда debugger получил уведомление. Программа может иметь нормальный handler ниже по маршруту. Она может намеренно использовать исключение как control-flow. Она может ожидать, что debugger вернёт DBG_EXCEPTION_NOT_HANDLED и позволит Windows найти обработчик. Поэтому читать лог отладчика надо не эмоционально, а маршрутом: first chance - это начало доставки, а не финальный диагноз.

Vectored exception handling как слой вне кадра

Если debugger не обработал exception на first chance, Windows передаёт событие дальше. Один из ранних внутренних слоёв - vectored exception handling, обычно сокращённо VEH. Слово vectored здесь исторически связано с идеей векторного, то есть направляемого через список callback-ов, обработчика. VEH регистрируется через AddVectoredExceptionHandler и существует на уровне процесса, а не на уровне конкретного стекового кадра. Это центральное отличие от frame-based SEH.

Frame-based handler живёт в связи с кадром функции. Его доступность зависит от того, есть ли соответствующий frame в текущем стеке и попадает ли RIP в protected range. VEH работает иначе. Он не спрашивает: находится ли exception внутри __try этой функции? Он получает шанс независимо от конкретного call frame. Поэтому VEH удобен для глобального наблюдения, диагностики, crash logging, anti-debug, guard page tricks и exception-based dispatch. Но именно потому, что он не frame-based, его нельзя понимать как обычный __except. Он стоит раньше stack unwinding и может вообще не дать Windows дойти до поиска frame-based handler-ов.

Обработчик VEH получает EXCEPTION_POINTERS. Это значит, что он видит и паспорт события, и CONTEXT потока. Он может проверить ExceptionCode, посмотреть RIP, понять, из какого module пришло событие, изменить ContextRecord и вернуть значение, определяющее дальнейшую судьбу exception. Если он возвращает EXCEPTION_CONTINUE_SEARCH, он говорит: я посмотрел, но не беру событие на себя, пусть поиск идёт дальше. Если он возвращает EXCEPTION_CONTINUE_EXECUTION, он говорит: событие обработано, CONTEXT теперь пригоден, можно продолжать выполнение. Именно второй вариант делает VEH мощным инструментом control-flow. Handler может передвинуть RIP, поправить регистры или снять guard condition, и программа продолжит не там, где обычный CFG ожидал.

Порядок VEH тоже не случайный. AddVectoredExceptionHandler принимает параметр First, который влияет на порядок вызова. Это не абсолютная гарантия навсегда, потому что последующие регистрации могут изменить, кто будет первым, но для анализа важно: VEH образуют ordered process-wide layer.

Поэтому static analysis ищет регистрации и callbacks, a dynamic validation можетнаблюдать registration events and handler hits. Но отсутствие hit-a в однойсессии не доказывает, что VEH не используется. Возможно, нужный exceptionpath просто не был достигнут.

Frame-based SEH как поиск по живому стеку

Если VEH не обработал exception, Windows переходит к тому, что обычно называют frame-based structured exception handling. Structured exception handling, SEH, переводят как структурированная обработка исключений. Слово structured здесь важно: обработка не является хаотическим goto в случайный callback. Она встроена в структуру функций, protected ranges, scope tables, language-specific handlers и unwind metadata. На x64 эта структура не лежит в виде старой linked list на стеке, как было на x86. Она опирается на таблицы . pdata и .xdata.

Поиск frame-based handler-a начинается с текущего кадра. Но Windows не может просто посмотреть на стек и угадать границы функций. Ей снова нужен маршрут, разобранный в 10.1: ControlPc -> RtlLookupFunctionEntry -> RUNTIME_FUNCTION -> UNWIND_INFO -> handler flags and handler data. Если UNWIND_INFO имеет UNW_FLAG_EHANDLER, значит у функции есть exception handler, который система должна вызвать, чтобы он examined state of the exception and potentially handle it. Для C __try/__except таким language-specific handler часто становится __C_specific_handler. Важно: __C_specific_handler - это не пользовательский блок __except. Это runtime handler, который читает scope table и решает, какой filter относится к текущему protected range.

Здесь появляется слово protected range. Protected - защищённый, range - диапазон. Это участок кода, для которого language-specific data говорит: если exception произошёл внутри этого диапазона, надо выполнить такой-то filter или handler. Новичок часто думает, что __try покрывает всю функцию. Это неверно. В одной функции может быть несколько protected ranges, они могут быть вложенными, их границы могут не совпадать с тем, что декомпилятор красиво показал как блок. Поэтому frame-based SEH - это не "у функции есть handler" в общем смысле. Это точная связка: текущий ControlPc внутри функции -> конкретный protected range -> соответствующий filter -> решение filter-a.

Filter expression - выражение фильтра - возвращает одно из стандартных решений. EXCEPTION_CONTINUE_SEARCH означает: этот frame посмотрел на exception, но не берёт его, ищи дальше выше по стеку. EXCEPTION_EXECUTE_HANDLER означает: этот frame готов обработать exception, нужно выполнить handler block; перед этим система должна корректно выполнить unwind до нужного frame. EXCEPTION_CONTINUE_EXECUTION означает: продолжить выполнение с текущим/изменённым CONTEXT, если это допустимо. Эти значения нельзя

воспринимать как просто числа. Это ответы конкретного кадра на вопросWindows: что делать с маршрутом исключения дальше?

Если текущий frame не обработал exception, Windows должна перейти к caller frame. И вот здесь снова возвращается RtlVirtualUnwind. Чтобы подняться вверх по стеку, система восстанавливает caller context: новый RIP становится return address, новый RSP указывает на состояние caller-а, nonvolatile-регистры возвращаются из мест, описанных UNWIND_INFO. Потом тот же вопрос задаётся следующему frame: есть ли у него handler, покрывает ли он место, каков результат filter-а? Так exception travels up the stack - поднимается по стеку не физическим прыжком, а последовательным виртуальным восстановлением кадров.

Search phase и unwind phase как две разные части одного решения

Внутри frame-based обработки важно различать search phase и unwind phase. Search phase - фаза поиска. В ней Windows ищет frame, который согласится обработать exception. Система идёт от текущего кадра к caller-ам, вызывает соответствующие language-specific handlers и filters, но ещё не обязана разрушать кадры окончательно. Её задача - найти ответ: кто берёт ответственность за исключение?

Unwind phase - фаза раскрутки стека. Она начинается, когда handler найден и нужно реально перейти к месту обработки. Теперь кадры между точкой exception и frame-ом обработчика должны быть сняты. При этом могут вызываться termination handlers, __finally, destructors в языковых runtime и другие cleanup-механизмы. Здесь UNW_FLAG_UHANDLER приобретает смысл: это termination handler, который вызывается при unwinding, чтобы освободить ресурсы или выполнить обязательную очистку. Поэтому unwind phase не равна простому ret. Это контролируемое разрушение части call stack с соблюдением language/runtime контрактов.

Такое разделение объясняет, почему один и тот же UNWIND_INFO участвует сразу в нескольких задачах. Для stack walking он нужен, чтобы получить caller context. Для search phase он нужен, чтобы найти handler data и подняться к следующим frame-ам. Для unwind phase он нужен, чтобы корректно снять кадры и вызвать cleanup. Для debugger support он нужен, чтобы построить call stack даже без символов. Для crash dump analysis он нужен, чтобы понять цепочку вызовов после падения. Вся модель держится на одном контракте: metadata должна честно описывать эффекты функции на стек и сохранённые регистры.

В reverse engineering search/unwind distinction помогает не путать handler discovery и handler execution. То, что система нашла подходящий filter, ещё не означает, что пользовательский handler block уже выполнен. То, что unwind

metadata содержит UHANDLER, не означает, что он является __except. То, чтofunction has exception handler, не означает, что он обработает любой exceptionвнутри функции. Поэтому хороший отчёт должен говорить не только " handlerpresent", а на каком слое это обнаружено: static handler metadata, protectedrange match, dynamic handler hit, post-handler RIP trace. Это разные уровнисвидетельств.

Vectored continue handlers и продолжение после попытки обработки

Рядом с VEH существует ещё один слой, который легко упомянуть и почти не объяснить: vectored continue handler, VCH. Continue здесь означает продолжение, а не продолжение списка. VCH регистрируется через AddVectoredContinueHandler. В практической модели его удобно понимать как callback-слой, связанный с продолжением обработки после того, как exception processing прошёл определённые внутренние решения. Для твоего проекта важно не превращать VCH в ещё один SEH. Он так же не frame-based, регистрируется на уровне процесса и может быть интересен dynamic validation как runtime registration and hit.

Почему VCH важен для reverse engineering? Потому что exception route может быть не только "событие -> handler -> crash/continue". Иногда программа использует несколько callback-слоёв, где один слой наблюдает или частично подготавливает состояние, другой принимает решение, третий фиксирует continuation. Если static analysis видит только .pdata/.xdata, он не увидит runtime-registered VCH. Если dynamic validation наблюдает AddVectoredContinueHandler and hits, в отчёте появляется runtime evidence: во время запуска был зарегистрирован process-wide continue handler, и его надо учитывать при чтении маршрута исключения.

Однако и здесь нужно сохранять осторожность. Регистрация VCH не доказывает вредоносность и не доказывает control-flow trick. Это может быть диагностическая библиотека, runtime, crash monitoring, совместимость, instrumentation. Анализатор должен сформулировать факт: registered continue handler, handler address, module, hit count, related exception events. Интерпретация уже зависит от контекста: какие exception codes были рядом, менялся ли CONTEXT, куда ушёл RIP после обработки, были ли подозрительные executable private regions или dynamic function tables.

Second chance как последний внешний шанс debugger-а

Если frame-based поиск не нашёл обработчика, Windows снова может обратиться к debugger-у. Это second chance, или last-chance notification. Название second chance не значит "второстепенное событие". Наоборот, это очень сильный сигнал: обычные внутренние обработчики процесса не взяли exception. Debugger получает последнюю возможность вмешаться. Если debugger и теперь не обработает exception, процесс, как правило, будет завершён.

Для реверсера second-chance exception ближе к реальному crash, чем first chance. Но и здесь надо быть точным. First chance говорит: exception только что возникло, debugger увидел его до обработчиков. Second chance говорит: обработчиков, которые справились бы с этим exception, не нашлось или они не взяли ответственность. Поэтому в dynamic validation first_chance и second_chance должны храниться раздельно. Смешивать их в одно поле " exceptions" - значит терять смысл маршрута.

ContinueDebugEvent снова играет ключевую роль. Если debugger на first chance вернул DBG_EXCEPTION_NOT_HANDLED, Windows продолжает search and dispatch logic. Если похожий статус возвращён на second chance, дальше уже некуда нормально искать, и процесс будет terminated. Поэтому dynamic collector должен быть осторожным: он не просто пассивно записывает события. Его политика продолжения влияет на то, увидит ли target свои внутренние handlers. Это ещё одна причина, почему dynamic validation является свидетельством конкретной сессии, а не абсолютной истиной обо всех возможных запусках.

Top-level и unhandled path как последний слой процесса

Если exception не был обработан frame-based handler-ами и debugger не забрал его на second chance, остаётся unhandled path. Unhandled переводится как необработанное. Но здесь опять нельзя думать слишком просто. Unhandled exception - это не exception, у которого вообще никогда не было обработчиков. Это exception, которое прошло обычный маршрут поиска и не нашло обработчика, готового взять ответственность. Только после этого появляется top-level/unhandled layer.

SetUnhandledExceptionFilter позволяет приложению установить top-level exception filter. Top-level - верхнего уровня, но по маршруту это не первый, а фактически последний пользовательский фильтр для необработанных исключений. Он получает LPEXCEPTION_POINTERS, то есть снова видит EXCEPTION_RECORD и CONTEXT. Документация Microsoft подчёркивает, что фильтр выполняется в context of the thread that caused the fault. Это важная практическая деталь: crash reporter работает не в безопасной отдельной вселенной, а в том же проблемном потоке. Если стек повреждён или память в

плохом состоянии, возможности фильтра ограничены.

Top-level filter часто используют crash reporters. Такой код может записать minidump, лог, telemetry, показать сообщение пользователю и позволить процессу завершиться. Это нормальный сценарий. Но иногда top-level filter пытается вернуть EXCEPTION_CONTINUE_EXECUTION, изменить CONTEXT или скрыть crash. Это уже требует гораздо более аккуратного анализа. Не потому что это автоматически malware, а потому что последний слой маршрута начинает менять продолжение выполнения после того, как обычные обработчики не справились.

UnhandledExceptionFilter имеет особую связь с debugger-ом. Если процесс отлаживается, unhandled exception может быть передано debugger-у. Если нет, включается обычное поведение Application Error, error mode, possibly dialog или termination path. Для отчёта это значит: наличие SetUnhandledExceptionFilter в статике - это только registration evidence. Runtime evidence появится, если dynamic validation увидит регистрацию, hit top-level handler-а или second-chance/unhandled событие. Без runtime path нельзя утверждать, что этот filter реально участвовал в конкретном exception.

Fail-fast как намеренный обход обычной лестницы

Чтобы модель была честной, нужно упомянуть исключительный случай в самой exception-модели: fail-fast. RaiseFailFastException и родственные механизмы нужны для ситуаций, где продолжение процесса считается недопустимым. В документации по errhandling API эта функция описывается как raising an exception that bypasses all exception handlers, frame or vector based. Смысл fail-fast в том, чтобы не пытаться героически продолжать выполнение после состояния, которое считается критическим для целостности процесса.

Для нашей mental model это полезный контраст. Обычный exception route - это лестница возможностей: debugger first chance, VEH, frame-based SEH, unwind, second chance, unhandled. Fail-fast говорит: лестницу не используем. Поэтому если static analysis или dynamic validation видит fail-fast path, его нельзя объяснять обычной логикой "почему не вызвался handler". Handler мог не вызваться именно потому, что механизм выбран для обхода handler-ов. Это не ошибка модели, а другая модель завершения.

В reverse engineering fail-fast часто указывает на security checks, stack cookie failure, invalid parameter handling, deliberate termination. Но опять же не надо превращать это в слово "malware". Надо описать инженерный смысл: код выбирает путь, при котором восстановление или продолжение не предполагается. В отчёте это должно отражаться как особый route, а не как absence of handler evidence.

Маршрут глазами static analysis

Теперь посмотрим на всю цепочку глазами static analysis. Static analysis не видит живое событие. Он читает файл, секции, imports, Exception Directory, .pdata, .xdata, handler data, possible registrations. Он может сказать: здесь есть AddVectoredExceptionHandler, значит во время выполнения возможно появление VEH registration. Здесь есть SetUnhandledExceptionFilter, значит приложение может установить top-level filter. Здесь RUNTIME_FUNCTION имеет UNW_FLAG_EHANDLER, значит у функции есть language-specific handler data. Здесь виден RaiseException или int 3, значит код может намеренно создавать exception site.

Но static analysis не должен выдавать возможное за произошедшее. Он видит declared structure and code evidence - заявленную структуру и признаки в коде. Он может построить маршруты-гипотезы: если execution reaches this RaiseException, then exception dispatch may go through these layers; if ControlPc is in this protected range, then __C_specific_handler may evaluate this filter; if this VEH registration runs, then process-wide handler may observe later exceptions. Это очень ценная работа, но она остаётся гипотезой о возможном runtime behavior.

Heuristic/evidence engine на этом слое должен быть особенно дисциплинированным. Если он видит handler dominance, reuse UNWIND_INFO, executable metadata, invalid handler RVA или exception-based control-flow sites, он формирует findings. Но finding должен говорить языком свидетельства: what was observed, why it matters, what Windows invariant is involved, what benign explanations exist, what dynamic validation could test. Такой finding не пугает, а учит читать маршрут.

Маршрут глазами dynamic validation

Dynamic validation смотрит на другую сторону той же истории. Она запускает target, превращает его в debuggee, ждёт debug events, фиксирует exception events, registrations, handler hits, dynamic function table calls and RIP trace. Это уже runtime evidence - свидетельство конкретной живой сессии. Если был first-chance access violation, это не просто теоретический site. Событие произошло. Если VEH handler hit случился, callback реально получил управление. Если после handler-а RIP ушёл в другой region, это наблюдаемый маршрут continuation.

Но dynamic validation тоже не всеведуща. Она видит только то, что произошло в данном запуске, с данным input, данным timeout, данной политикой breakpoints and continuation. Если handler не hit-нулся, это может означать, что route не проявился, но не доказывает, что route невозможен. Если process exited before suspicious code, session incomplete. Если breakpoint не удалось поставить из-за ASLR, forwarded export, page protection или

поздней загрузки модуля, instrumentation coverage ограничена. ПоэтомуDynamicReport должен быть честным: enabled, ran, ok, timed_out, droppedevents, breakpoint completeness, observed events. Без этого runtime evidenceлегко превращается в самообман.

Правильная связь static analysis and dynamic validation выглядит не как судья и приговор, а как карта и поход по местности. Static analysis нарисовал карту возможных маршрутов. Dynamic validation прошла одним или несколькими маршрутами и принесла путевые заметки: здесь действительно был exception, здесь handler получил управление, здесь RIP после continuation оказался в таком диапазоне, здесь dynamic function table была зарегистрирована, а здесь мы ничего не увидели из-за timeout. Assessment соединяет эти два слоя, но не стирает различие между ними.

Одна цельная трасса доставки исключения

Теперь можно собрать маршрут в одну линию, но не как сухой список, а как историю одного события. Поток выполняет инструкцию, и эта инструкция порождает exception. Windows оформляет событие в record and context. Если есть debugger, он получает first chance. Debugger либо берёт событие на себя, либо пропускает его дальше. Затем process-wide vectored exception handlers получают шанс увидеть событие до frame-based unwinding. Если они не обработали exception, начинается поиск по кадрам: текущий ControlPc привязывается к function table, UNWIND_INFO показывает, как устроен кадр и есть ли handler data, language-specific handler проверяет protected ranges and filters. Если frame не берёт exception, RtlVirtualUnwind поднимает контекст к caller frame, и поиск повторяется. Если handler найден, начинается unwind phase: промежуточные кадры снимаются, cleanup handlers выполняются, управление переходит к handler block или continuation path. Если обработчика нет, debugger получает second chance. Если и это не помогает, включается unhandled/top-level path и, чаще всего, процесс завершается.

В этой линии нет ни одного лишнего слоя. Debugger first chance нужен, чтобы внешний инструмент мог увидеть exception до вмешательства программы. VEH нужен, чтобы процесс мог иметь global non-frame-based наблюдателей и обработчиков. Frame-based SEH нужен, чтобы обработка была связана с конкретными участками кода и кадрами стека. Unwind нужен, чтобы корректно перейти от места исключения к месту обработки, не исполняя обычные epilogs. Second chance нужен, чтобы debugger получил последнюю возможность исследовать crash-like condition. Top-level path нужен, чтобы процесс мог сделать последнюю обработку необработанной ошибки, например записать dump.

Для реверсера эта линия становится способом не потеряться. Если в x64dbg видно first-chance breakpoint exception, надо спросить: debugger обработал его или пропустил? Если виден AddVectoredExceptionHandler, надо

спросить: какой callback, какие exception codes он фильтрует, меняет лиCONTEXT? Если UNWIND_INFO содержит EHANDLER, надо спросить: какойlanguage-specific handler, какие protected ranges, какие filters? Если crashдошёл до second chance, надо спросить: почему предыдущие слои не взялиexception? Если top-level filter зарегистрирован, надо спросить: ондиагностический или пытается продолжить выполнение?

Практическая сцена: access violation внутри protected range

Разберём одну живую сцену, потому что без сцены маршрут остаётся абстракцией. Допустим, поток выполняет инструкцию mov rax, qword ptr [rcx], а RCX содержит недопустимый адрес. Процессор не может прочитать память и создаёт access violation. Windows формирует EXCEPTION_RECORD: код указывает на нарушение доступа, ExceptionAddress связан с инструкцией, дополнительные параметры описывают тип доступа и адрес. CONTEXT фиксирует RIP на проблемной инструкции, RSP на текущей вершине стека и значения регистров.

Если процесс под debugger-ом, debugger получает first chance. Он может остановить пользователя на этой инструкции. Если debugger возвращает not handled, Windows вызывает VEH callbacks. Один VEH может посмотреть ExceptionCode и сказать: это не мой guard page, continue search. Другой может вообще отсутствовать. Тогда начинается frame-based search. ControlPc привязывается к текущему module, из VA получается RVA, RtlLookupFunctionEntry находит RUNTIME_FUNCTION, UNWIND_INFO показывает, что у функции есть EHANDLER, a language-specific data указывает на __C_specific_handler and scope table.

__C_specific_handler проверяет, находится ли ControlPc внутри protected range. Если да, выполняется filter expression. Filter может вызвать GetExceptionCode, сравнить с EXCEPTION_ACCESS_VIOLATION и вернуть EXCEPTION_EXECUTE_HANDLER. Это не значит, что handler block уже выполнен. Это значит: найден frame, который согласился обработать exception. Теперь Windows должна unwind промежуточные кадры до establisher frame, выполнить нужные cleanup actions, восстановить состояние так, чтобы перейти в handler block. Только после этого пользовательский код __except начинает исполняться.

Если filter вернул CONTINUE_SEARCH, search phase идёт выше по стеку. RtlVirtualUnwind вычисляет caller context: RIP становится return address текущей функции, RSP поднимается к caller-a, nonvolatile-регистры восстанавливаются по UNWIND_INFO. Затем система проверяет caller frame. Если никто не берёт exception, debugger получит second chance. Если и там not handled, сработает unhandled path. Одна access violation прошла всю лестницу, и каждый слой либо увидел её, либо пропустил, либо обработал.

Практическая сцена: breakpoint exception как намеренный маршрут

Теперь другая сцена. В коде есть инструкция int 3. Для отладчика это привычный breakpoint. Но для программы это может быть намеренный exception site. Когда int 3 выполняется, возникает breakpoint exception. Debugger получает first chance. Если debugger решит обработать breakpoint как свою точку останова, внутренние обработчики программы могут не увидеть событие. Если debugger пропускает его дальше, VEH может получить exception and examine CONTEXT.

Представим VEH, который проверяет, что ExceptionCode соответствует breakpoint, затем меняет ContextRecord->Rip на адрес скрытого блока и возвращает EXCEPTION_CONTINUE_EXECUTION. В обычном CFG дизассемблера перехода к этому блоку может не быть. Но в реальном execution route управление туда попадёт через exception continuation. Static analysis может увидеть int 3 and VEH registration, но это только route hypothesis. Dynamic validation может подтвердить handler hit и post-handler RIP. Тогда finding становится сильнее: не просто "есть breakpoint exception site", а "во время выполнения breakpoint exception был обработан vectored handler-ом, и RIP после continuation ушёл в такой-то диапазон".

Эта сцена показывает, почему exception route нельзя свести к crash handling. Исключение здесь не ошибка, а механизм маршрутизации. Windows предоставляет общую систему доставки исключений, а программа использует её как скрытый dispatcher. Это может быть защита, обфускация, anti-debug, diagnostic trick или runtime mechanism. Анализатор не должен угадывать мотив. Он должен объяснить маршрут и показать evidence.

Язык отчёта для полного маршрута

Когда вся модель собрана, становится ясно, каким должен быть хороший report. Он не должен писать просто "exception handler found". Он должен помогать восстановить историю. В static part он показывает: где Exception Directory, какие RUNTIME_FUNCTION ranges, какие UNWIND_INFO flags, какие handler RVA, какие imports or suspicious sites. В dynamic_validation он показывает: какие exception events произошли, какие были first chance or second chance, какие registrations observed, какие handler hits, какой RIP trace после handler-а, были ли dynamic function tables. В assessment он связывает это осторожно: static evidence suggests route; runtime evidence observed route in this session; limitations remain.

Слова suggests, observed, confirmed, not observed, unknown должны использоваться очень дисциплинированно. Suggests - статический признак предлагает гипотезу. Observed - dynamic collector увидел событие. Confirmed - можно говорить только о конкретной проверенной связи, например handler

RVA из static metadata действительно был hit в runtime. Not observed - в этойсессии не наблюдалось, но не отсутствует вообще. Unknown - инструментчестно не знает из-за отсутствия данных, parse failure, timeout, coverage дарили несоответствия карты памяти.

Именно такой язык защищает анализ от самообмана. Если report говорит " malware" там, где есть только unusual metadata, он ломает доверие. Если report молчит о limitations dynamic session, он тоже ломает доверие. Но если report показывает маршрут и слой доказательства, он учит реверсера: вот что файл заявил, вот что Windows должна сделать, вот что runtime действительно показал, вот где нужна ручная проверка.

Итоговая mental model раздела

Полный маршрут исключения в Windows x64 надо держать не как таблицу API, а как живую лестницу ответственности. Событие рождается в конкретном потоке. Оно получает паспорт события и снимок состояния потока. Debugger может увидеть его первым. Vectored handlers могут наблюдать или изменить маршрут вне связи с конкретным кадром. Frame-based SEH ищет обработчик через стек, но на x64 этот поиск невозможен без .pdata, .xdata and UNWIND_INFO. RtlVirtualUnwind позволяет системе подниматься от frame к frame, не исполняя обычный ret. Если handler найден, search phase переходит в unwind phase. Если никто не берёт exception, debugger получает second chance, а затем включается unhandled/top-level path.

В этой модели каждый термин получает роль. RIP - точка события. RSP - опора стека. CONTEXT - снимок машины. EXCEPTION_RECORD - паспорт события. VEH - process-wide ранний callback слой. SEH - frame-based слой, связанный с protected ranges. UNWIND_INFO - инструкция восстановления кадра. RtlVirtualUnwind - вычислитель caller context. SetUnhandledExceptionFilter - последний пользовательский фильтр для unhandled path. Dynamic validation - наблюдение живого маршрута в конкретной сессии. Static analysis - чтение заявленной структуры и признаков маршрута в файле. Assessment - осторожное соединение этих свидетельств.

Когда это уложилось в голове, исключения перестают быть хаосом. Они становятся маршрутом. И реверсер перестаёт спрашивать только "почему упало?". Он начинает спрашивать точнее: где родилось событие, кто первым его увидел, какой слой его пропустил, какой frame мог его обработать, как был выполнен unwind, куда ушёл RIP после handler-a, и какие свидетельства у нас есть для каждого ответа. Именно эта точность и является фундаментом для чтения сложных Windows x64 бинарей.

Три возможных ответа обработчика как развилки маршрута

Чтобы маршрут исключения перестал быть туманным, нужно отдельно прожить три стандартных ответа, которые обработчик может вернуть системе. Это не просто константы из заголовочного файла. Это три разных способа продолжить историю потока. Если обработчик возвращает EXCEPTION_CONTINUE_SEARCH, он говорит Windows: "я увидел событие, но не беру его на себя". В этом ответе нет провала. Это нормальное решение наблюдателя или фильтра, который понял, что exception относится не к нему. Например, VEH может получать все исключения процесса, но обрабатывать только guard page события со своего диапазона. Всё остальное он честно пропускает дальше.

EXCEPTION_EXECUTE_HANDLER - другой тип ответа. Он говорит не " продолжай как раньше", а "я принимаю исключение, переходи к обработчику". В frame-based SEH это обычно означает, что найден __except block, но путь к нему не является обычным branch из точки fault. Windows должна перейти из места, где произошёл exception, к месту обработки через unwind. Это значит, что часть stack frames между fault site и handler frame будет снята, cleanup может выполниться, а control flow станет нелинейным. Декомпилятор часто пытается представить это как обычный high-level try/except, но на уровне машины это особая exception edge - ребро управления, которого нет в обычном линейном CFG.

EXCEPTION_CONTINUE_EXECUTION - самый опасный для понимания ответ, потому что он ломает бытовую ассоциацию "исключение = авария". Этот ответ говорит: событие обработано, продолжай выполнение из состояния CONTEXT. Если handler не менял CONTEXT, продолжение может снова попасть на ту же faulting instruction и зациклиться. Если handler изменил RIP, RSP, регистры или состояние памяти, поток может продолжить уже по новому маршруту. Поэтому в reverse engineering возврат CONTINUE_EXECUTION почти всегда требует повышенного внимания. Он не обязательно вредоносный, но он означает: обработчик претендует не просто на диагностику, а на изменение или восстановление хода выполнения.

Эти три ответа удобно держать как три двери. CONTINUE_SEARCH - дверь дальше по маршруту доставки. EXECUTE_HANDLER - дверь в обработчик через unwind. CONTINUE_EXECUTION - дверь обратно в выполнение, возможно с изменённым состоянием. Когда static analysis видит код handler-а, он должен попытаться понять, к какой двери ведёт logic. Когда dynamic validation видит handler hit and post-handler RIP, она помогает увидеть, какая дверь была реально выбрана в данном запуске. Отчёт не должен писать просто "handler returned value". Он должен объяснять, что этот value делает с маршрутом исключения.

Диспетчер исключений как невидимый проводник

В текстах про Windows exception handling часто встречается слово dispatcher. По-русски его можно перевести как диспетчер, но это слово легко звучит слишком абстрактно. В нашем контексте dispatcher - это слой системной логики, который не является пользовательским обработчиком, но доставляет событие между участниками маршрута. Он знает порядок: сначала debugger first chance, затем vectored exception handlers, затем frame-based search, затем second chance and unhandled path. Пользовательский handler не сам решает, кто следующий в очереди. Он возвращает disposition, а dispatcher продолжает маршрут согласно правилам.

Важно понимать, что dispatcher работает не с красивой картинкой из decompiler-а. Он работает с машинными структурами: exception record, context record, function tables, UNWIND_INFO, handler data, return values. Поэтому попытка объяснить исключение только исходным кодом часто недостаточна. Исходный код мог быть __try/__except, но в бинаре это стало range records, scope tables, handler RVA and metadata. Dispatcher видит именно эту низкоуровневую форму.

Для реверсера dispatcher полезен как мысленный персонаж. Когда ты видишь exception event, можно представить, что системный диспетчер несёт папку с EXCEPTION_RECORD и CONTEXT и подходит по очереди к участникам: debugger, vectored handlers, текущий frame, caller frame, следующий caller frame, top-level filter. Каждый участник либо берёт папку и меняет судьбу события, либо отказывается. Такой образ помогает не путать слои. VEH не является caller frame. __C_specific_handler не является top-level filter. Debugger first chance не является crash. Всё это разные столы, через которые dispatcher несёт одно и то же событие.

Почему x64 так сильно зависит от таблиц

На x86 исторически существовала другая модель SEH, где linked list обработчиков находился в thread environment block / stack-related структурах. На x64 Microsoft ушла к table-based exception handling. Это не косметическое изменение. В x64 unwind and exception handling сильно завязаны на статические таблицы .pdata/.xdata. Причина в том, что 64-битная ABI требует надёжного восстановления стека и nonvolatile-регистров в произвольной точке функции, а не только там, где программист поставил handler.

Table-based подход даёт системе возможность не доверять тому, что случайно лежит на стеке как цепочка handler-ов. Вместо этого Windows смотрит на image-relative function table entries и metadata, созданную компилятором/линкером. Это повышает предсказуемость stack walking, exception handling and debugging support. Но цена такой модели тоже очевидна: если .pdata or .xdata повреждены, отсутствуют, противоречат коду

или не зарегистрированы для runtime-generated code, система может потерятьспособность корректно unwound frame.

Вот почему твой анализатор так много внимания уделяет Exception Directory, RUNTIME_FUNCTION, UNWIND_INFO, ranges, overlaps, invalid RVA, handler placement and dynamic function tables. Это не узкая прихоть формата PE. Это проверка тех таблиц, на которые Windows реально опирается, когда поток оказывается в необычной ситуации. Если таблица врёт, ошибка проявится не обязательно в обычном запуске. Она может проявиться именно при exception, stack trace, crash dump, debugger walk или dynamic validation.

Protected range как граница смысла, а не просто два адреса

Protected range часто хочется перевести как "защищённый диапазон" и сразу пройти дальше. Но этот термин заслуживает остановки. Диапазон защищён не в смысле page protection и не в смысле безопасности памяти. Он защищён в смысле языка исключений: если exception возникнет внутри этого участка кода, для него есть специальное правило обработки. Такое правило может быть filter expression, handler block, finally cleanup или language-specific logic.

Для машины protected range - это адресная область. Для программиста - область try. Для decompiler-а - попытка восстановить блок. Для Windows - условие, по которому language-specific handler решает, применим ли scope record к текущему ControlPc. Для static analysis - объект проверки: попадает ли range в функцию, не указывает ли filter/handler за пределы разумных секций, не пересекаются ли ranges странным образом. Для dynamic validation - потенциальный маршрут, который проявится только если exception реально возникнет внутри этого range.

Эта многослойность объясняет, почему нельзя просто сказать "у функции есть SEH". Нужно сказать точнее: в unwind metadata функции есть EHANDLER, handler RVA указывает на language-specific handler, handler data содержит scope information, и один из protected ranges покрывает такой-то участок. Если exception произойдёт внутри этого участка и filter вернёт EXECUTE_HANDLER, тогда управление пойдёт в handler block. Это длиннее, но именно это является правдой.

Handler data как хвост, который меняет смысл всей структуры

В UNWIND_INFO есть основная часть, которая описывает, как восстановить стековый кадр. Но при флагах EHANDLER или UHANDLER после массива UNWIND_CODE появляется дополнительный хвост: handler RVA and language-specific data. Этот хвост легко недооценить. Новичок может распарсить прологовые операции, увидеть, что stack allocation корректный, и решить, что на этом всё. Но если у функции есть handler data, именно хвост объясняет exception route.

Handler RVA часто указывает не на пользовательский код __except, а на runtime-level обработчик языка. Для C SEH это может быть __C_specific_handler. Он сам по себе не является ответом на вопрос "куда прыгнет программа". Он является механизмом, который прочитает language-specific data и примет решение. Поэтому static analysis должен идти ещё глубже: не остановиться на handler RVA, а понять, есть ли за ним scope table, protected ranges, filter addresses, handler block addresses. Если формат неизвестен или повреждён, честный результат - не фантазировать, а отметить unknown или parse limitation.

Для dynamic validation handler data тоже важно, но иначе. Dynamic collector может поставить breakpoint на handler RVA или на адреса, которые static analysis считает handler targets. Если breakpoint hit случился, это runtime evidence, что соответствующий route проявился. Но даже hit runtime handler-а не всегда равен hit пользовательского handler block. Например, __C_specific_handler мог быть вызван для поиска, filter мог вернуть CONTINUE_SEARCH, и пользовательский __except block не исполнился. Поэтому report должен различать language-specific handler hit, filter decision if observable, handler block hit and post-handler RIP.

Noncontinuable exception и границы продолжения

Мы уже говорили, что EXCEPTION_CONTINUE_EXECUTION может продолжить выполнение после handler-а. Но не каждое исключение можно продолжать. Существует понятие noncontinuable exception - исключение, после которого попытка продолжить выполнение считается недопустимой. На практическом уровне это значит: handler может захотеть вернуть CONTINUE_EXECUTION, но система или семантика exception не обязаны позволить безопасное продолжение. Это важная поправка к идее "handler может всё".

Для реверсера это означает, что нужно смотреть не только на return value, но и на тип exception, flags, context modifications and actual continuation. Если handler выглядит так, будто он всегда возвращает CONTINUE_EXECUTION, но dynamic validation показывает second-chance termination, значит где-то между

намерением кода и системной реальностью возник конфликт. Возможно,exception noncontinuable. Возможно, context остался повреждённым.Возможно, handler не был тем, кто реально получил управление.

В отчёте такое место нельзя описывать одной фразой. Нужно отделить stated handler logic от observed behavior. Static evidence: handler contains path returning CONTINUE_EXECUTION. Runtime evidence: process still terminated after second chance. Assessment: continuation path not confirmed; manual validation should inspect exception flags, context modifications and continuation status. Такой стиль учит, а не выдаёт ложную уверенность.

Guard page как учебный пример управляемого fault

Guard page - хороший пример того, почему exception не всегда авария. PAGE_GUARD делает страницу памяти одноразовым сторожем: при первом обращении к ней система генерирует guard page violation. Это fault, но он может быть ожидаемым. Handler может использовать его как сигнал: поток дошёл до охраняемой области, нужно отметить событие, изменить protection, переустановить guard или направить выполнение дальше.

В reverse engineering guard pages часто встречаются в anti-debug, unpacking, stack growth, lazy initialization and instrumentation. Но снова нельзя прыгать к диагнозу. Сначала нужно понять маршрут. Был ли VirtualProtect/PAGE_GUARD? Какой адрес вызвал exception? Был ли VEH registration? Какой handler получил событие? Вернул ли он CONTINUE_EXECUTION? Был ли после этого single-step для переустановки guard? Куда ушёл RIP? Static analysis может увидеть VirtualProtect and PAGE_GUARD constants. Dynamic validation может увидеть actual guard page exception and handler hit. Только вместе эти слои дают осмысленную картину.

Этот пример полезен ещё и потому, что показывает связь fault/trap. Guard page exception возникает при доступе к странице. После обработки часто нужно продолжить выполнение и, возможно, получить single-step, чтобы восстановить guard состояние. Поэтому exception route может состоять из нескольких связанных событий, а не из одного isolated crash. DynamicReport должен сохранять последовательность, иначе реверсер потеряет причинную связь.

Single-step как исключение, связанное с флагом процессора

Single-step exception связан с Trap Flag в EFLAGS/RFLAGS. Если этот флаг установлен, процессор генерирует debug exception после выполнения инструкции. Отладчики используют это для пошагового исполнения. Но программа тоже может играть с этим механизмом. Handler может установить Trap Flag, продолжить выполнение на одну инструкцию, получить single-step exception, а затем принять следующее решение. Так возникает управляемый микро-маршрут исполнения через exceptions.

Для новичка single-step часто выглядит как чисто отладочная тема. Но в защищённом или обфусцированном коде это может быть частью control-flow. Программа может проверять, как debugger обрабатывает single-step. Может использовать его для восстановления guard pages. Может строить dispatcher, где каждый шаг через exception меняет состояние. Поэтому dynamic validation, наблюдающая first_chance single-step events, должна связывать их с предыдущими events: был ли перед этим breakpoint, guard page, изменение context, установка TF.

Static analysis здесь сложнее: Trap Flag может устанавливаться через pushfq/popfq, lahf/sahf patterns, прямые манипуляции flags или handler context modifications. Не всегда это очевидно по импорту. Поэтому static finding может быть weak, a runtime evidence - сильным: в живом запуске single-step events действительно идут последовательностью после конкретного handler-a. Именно такие места показывают, зачем нужен не только статический анализ, но и динамическое свидетельство.

C++ exceptions как частный случай поверх общей системы

C++ exceptions на Windows x64 тоже не живут в отдельной магической вселенной. На низком уровне они используют системную exception/unwind инфраструктуру и language-specific runtime. Для реверсера это важно, потому что в бинаре можно увидеть structures and handlers, которые связаны с C++ unwinding, destructors, catch handlers and exception objects. Но не надо смешивать C++ try/catch с C __try/__except. Это разные языковые слои поверх общей системной основы.

C++ exception обычно несёт семантику объекта, типа, catch matching, destructor calls. SEH exception чаще выглядит как код исключения и filter logic. Но обе модели нуждаются в unwind: если нужно покинуть кадры, destructors and cleanup должны быть выполнены, nonvolatile-регистры восстановлены, RSP приведён к правильному состоянию. Поэтому .pdata/.xdata остаются фундаментом и для отладчика, и для C++ runtime, и для crash dump analysis.

В книге мы не превращаемся в учебник по C++ ABI, но mental model должна быть правильной: system-level exception route и language-level exception semantics - это разные этажи. Static analysis может видеть x64 unwind metadata and handler flags; более глубокий language-specific parsing может требовать знания конкретного compiler/runtime. Если анализатор не понимает language-specific tail, он должен сказать unknown, а не придумывать catch-блоки.

Когда unwind сам становится точкой отказа

Мы часто говорим, что Windows использует unwind metadata, чтобы восстановить caller context. Но что если metadata повреждена? Что если RUNTIME_FUNCTION range перекрывается с другим? Что если UnwindInfoAddress указывает за пределы image? Что если CountOfCodes ведёт parser за конец структуры? Что если пролог реально сохраняет RBX, а UNWIND_INFO об этом молчит? В такой ситуации exception route может сломаться не из-за исходного fault, а из-за невозможности корректно пройти по стеку.

Это особенно важно для анализа защищённого, packed or manually mapped code. Программа может исполняться нормально до первого exception. Обычные call/ret работают, потому что они используют реальный стек. Но как только debugger, crash handler or exception dispatcher пытается unwound frame, metadata становится критичной. Если она неверна, call stack может оборваться, handler может не найтись, wrong caller context может привести к неправильной интерпретации crash, a dynamic validation увидит странное termination instead of expected route.

Поэтому findings о .pdata/.xdata - это не формальные придирки. Они говорят: в момент exception Windows может не суметь восстановить то, что обычный поток исполнения пока скрывал. Хороший анализатор должен объяснять consequence: not "bad xdata", a "if exception or stack walk reaches this range, caller context reconstruction may be unreliable". Такой вывод сразу связывает статическую аномалию с runtime consequence.

Роль dynamic function tables в полном маршруте

В обычном PE Windows берёт function table из Exception Directory. Но runtime-generated code, JIT and some loaders могут создавать код уже после загрузки image. Для такого кода .pdata на диске может не существовать. Чтобы exception handling and debugging оставались надёжными, runtime должен предоставить function table через механизмы вроде RtlAddFunctionTable or RtlInstallFunctionTableCallback. Это не дополнительная опция красоты. Без этого Windows может не знать, как unwound generated code.

В полном маршруте это означает: когда RIP находится в runtime memory, вопрос "где .pdata?" меняется. Нужно искать не только PE Exception Directory,

но и active function tables. Static analysis файла на диске может честно сказать:в PE нет покрытия для этого будущего runtime code. Dynamic validation можетувидеть call to RtlAddFunctionTable and record runtime evidence: во времязапуска была зарегистрирована dynamic function table for region X. Этообъясняет, почему отсутствие .pdata в файле не всегда равно поломке, ноотсутствие dynamic registration для nonleaf runtime code может быть серьёзнойпроблемой.

Для отчёта это место особенно важно. Если dynamic collector увидел execution in private executable memory and no dynamic function table registration, это один уровень подозрения. Если он увидел RtlAddFunctionTable with entries covering that memory, это другое объяснение. Если entries есть, но handler hits or exceptions still fail, нужно смотреть качество таблиц. Опять же, карта и живое свидетельство должны сходиться осторожно.

Порядок слоёв как защита от ложных выводов

Самая частая ошибка в этой теме - видеть один слой и делать вывод обо всём маршруте. Видеть AddVectoredExceptionHandler and conclude "all exceptions handled here". Видеть SetUnhandledExceptionFilter and conclude "this catches everything". Видеть first-chance exception and conclude "program crashed". Видеть EHANDLER flag and conclude "user handler definitely ran". Каждое такое утверждение перескакивает через несколько ступеней маршрута.

Правильная дисциплина выглядит иначе. Сначала мы называем слой: debug notification, vectored handler, frame-based handler, unwind cleanup, second chance, unhandled filter. Потом называем свидетельство: import, registration event, metadata flag, handler RVA, dynamic hit, return disposition, RIP trace. Потом называем границу знания: possible, observed, confirmed for this run, not observed, unknown. Такая методика кажется медленной, но именно она спасает от ошибок.

Для твоей книги это принципиально. Мы не хотим, чтобы читатель запомнил "VEH раньше SEH" как флеш-карточку. Он должен понять, почему это раньше: VEH не зависит от текущего frame, поэтому может получить событие до stack unwinding; frame-based SEH требует пройти через текущий ControlPc, RUNTIME_FUNCTION, UNWIND_INFO and protected ranges; top-level filter вообще появляется только когда обычные обработчики не справились. Порядок слоёв - это не список. Это архитектура ответственности.

Чтение debugger log без паники

Когда реверсер впервые открывает x64dbg, WinDbg или другой debugger и видит строку про exception, он часто реагирует слишком быстро. В логе появилось first chance exception, значит "оно упало". Появился breakpoint exception, значит "отладчик поймал breakpoint". Появился access violation, значит "код плохой". Но лог debugger-а - это не приговор, а стенограмма маршрута. Каждая строка сообщает, на какой остановке exception route сейчас находится, но не всегда сообщает, чем маршрут закончится.

Поэтому читать такой лог надо не сверху вниз как набор ошибок, а как последовательность решений. Первое решение: debugger обработал событие сам или пропустил? Второе: если пропустил, были ли process-wide handlers? Третье: если дошло до frame-based search, какой ControlPc был использован и был ли найден RUNTIME_FUNCTION? Четвёртое: если handler был найден, это был language-specific runtime handler или пользовательский handler block? Пятое: после обработки RIP продолжил ту же инструкцию, перешёл к handler block, ушёл в continuation target или процесс завершился?

Такой способ чтения особенно важен для dynamic validation. Dynamic collector может собрать events, но человек или assessment logic должны сохранить смысл порядка. Две одинаковые exception codes в отчёте могут означать совершенно разные вещи. First-chance breakpoint, обработанный debugger-ом, - одно. Breakpoint, пропущенный в VEH and used as dispatcher, - другое. Second-chance breakpoint after no handler - третье. Если report не сохраняет chance, handler hit and continuation status, он теряет половину смысла.

Поэтому в хорошей книге мы не говорим просто "debugger получает исключение". Мы говорим: debugger получает structured notification, и это notification временно ставит debuggee на паузу. Пока debugger не вызовет ContinueDebugEvent, поток не продолжит свой маршрут. А флаг, с которым debugger продолжит событие, влияет на дальнейшую exception dispatch logic. Это уже не UI-деталь. Это часть механики доставки исключения.

Разница между handled, swallowed и recovered

В разговорах про исключения часто говорят "исключение обработано". Но слово handled может прятать несколько разных смыслов. Handler мог просто сказать системе, что он берёт событие, но потом завершить процесс. Handler мог записать dump and return execute handler, что приведёт к termination path. Handler мог изменить CONTEXT and continue execution. Handler мог перехватить exception только для логирования, а затем continue search. Во всех этих случаях он "видел" исключение, но результат разный.

Полезно различать handled, swallowed and recovered. Handled - обработано в системном смысле: кто-то вернул значение, из-за которого маршрут

перестал искать обработчик дальше или пошёл в handler block. Swallowed -проглочено: exception не дошло до внешнего crash, но причина могла бытьпросто скрыта. Recovered - восстановлено: программа действительновернулась к осмысленному продолжению, возможно после исправленияпамяти, изменения CONTEXT or reinitialization. Эти слова не являютсяофициальной таблицей Microsoft, но как учебная модель они помогают непутать регистрацию обработчика с реальным восстановлением.

Для анализа это критично. Crash reporter может handle exception enough to write dump, but not recover. Anti-debug VEH can swallow breakpoint exception and redirect execution. Guard page handler can recover by adjusting page protection and continuing. A broken handler can claim CONTINUE_EXECUTION and immediately fault again. Dynamic validation с RIP trace помогает различать эти ситуации: куда управление ушло после handler-а, повторился ли тот же exception, дошёл ли процесс до нормального exit, возник ли second chance.

В отчёте лучше избегать слишком сильной фразы "exception handled" без контекста. Гораздо полезнее писать: handler hit observed; handler returned continuation-like path; post-handler RIP observed in module X; process later terminated with second-chance exception; recovery not established. Такой язык длиннее, но он честный. Он не превращает один бит события в полный диагноз.

Exception route и обычный control flow как две карты одной программы

Обычный control flow - это то, что мы видим через call, jmp, conditional branch, ret. Дизассемблер и decompiler хорошо работают именно с этой картой. Но exception route добавляет вторую карту, которая не всегда видна как прямые переходы. Инструкция может не иметь jmp к handler-а, но exception dispatcher может доставить управление в handler block. Функция может не вызывать cleanup напрямую, но unwind phase вызовет termination handler. Код может не иметь явного branch к hidden block, но VEH изменит RIP and continue execution.

Поэтому для сложного reverse engineering одного CFG недостаточно. CFG показывает normal edges. Exception model показывает exceptional edges. Эти edges возникают не из инструкций branch, а из системных правил доставки исключений и metadata. В хорошей mental model обе карты накладываются друг на друга. Если смотреть только normal CFG, handler blocks кажутся unreachable. Если смотреть только exception metadata, можно переоценить route, который в данном запуске никогда не достигается. Нужны оба слоя.

Static analysis может попытаться добавить exception edges в свою внутреннюю модель: RaiseException site -> possible VEH/SEH route, protected range -> handler block, EHANDLER -> language-specific handler, UHANDLER ->

cleanup during unwind. Dynamic validation может показать, какие exceptional edges реально проявились. Такой подход особенно важен для Ghidra integration: комментарий или bookmark должен объяснять не просто адрес handler-a, а почему этот адрес может быть достигнут не через обычный branch.

Поток, процесс и модуль в одной истории

В exception route постоянно встречаются слова thread, process and module. Если их не развести, всё смешивается. Thread - поток исполнения. У него есть RIP, RSP, CONTEXT and stack. Exception рождается в конкретном thread. Process - процесс, контейнер адресного пространства and handles. VEH registrations and top-level filter живут на уровне process, но вызываются в context конкретного faulting thread. Module - загруженный image, внутри которого находятся code, sections, Exception Directory, .pdata, .xdata. Один exception route соединяет все три уровня.

Например, access violation происходит в thread T. Его RIP находится внутри module M. Чтобы найти frame-based handler, Windows использует metadata module M для текущего ControlPc. Но VEH, зарегистрированный процессом P, может увидеть событие независимо от того, в каком module возник RIP. Top-level filter тоже установлен для process, но выполняется в faulting thread. Dynamic collector наблюдает process as debuggee, но debug events приходят с process id and thread id, a addresses нужно привязывать к module map.

Эта связка важна для отчёта. Если report показывает handler address без module, пользователь не понимает, чей это код. Если показывает exception event без thread id, теряется thread-local nature. Если показывает RVA без image base, адрес нельзя связать с runtime event. Если показывает process-wide registration without explaining that hit occurs in faulting thread, читатель может думать, что handler работает в отдельном безопасном потоке. Хороший report постоянно держит три уровня: thread event, process registration, module metadata.

Почему top-level filter не заменяет нормальную обработку

SetUnhandledExceptionFilter часто воспринимают как универсальную сетку безопасности. Поставил top-level handler - и он "поймает всё". Это неверная и опасная модель. Top-level filter получает шанс только когда exception makes it to the unhandled exception filter. То есть до него уже должны не справиться или не сработать предыдущие уровни маршрута. VEH мог обработать событие раньше. Frame-based SEH мог обработать его раньше. Debugger мог вмешаться. Fail-fast может обойти обычные handlers. Наконец, состояние потока к этому моменту может быть настолько повреждено, что recovery бессмысленен.

Поэтому top-level handler лучше понимать как последний диагностический слой, а не как полноценную замену внутренней обработки ошибок. Он хорош для minidump, logging, telemetry, controlled termination. Но если программа пытается использовать его как общий механизм recovery, это требует внимательного анализа. Почему normal handlers не используются? Что именно меняется в CONTEXT? Какой exception code продолжается? Есть ли повторные faults? Дошёл ли процесс до нормального выполнения после filter-а?

Static analysis может увидеть SetUnhandledExceptionFilter and callback. Но это не означает, что callback когда-либо сработает. Dynamic validation может увидеть registration and maybe hit. Если hit случился после second-chance-like path, это сильное runtime evidence, но оно всё равно относится к конкретному сценарию запуска. Отчёт должен отделять top-level registration, top-level invocation and successful recovery.

Когда отсутствие события тоже является данным, но слабым

В dynamic validation бывает ситуация: static analysis нашёл suspicious exception route, но runtime session не увидела соответствующего exception or handler hit. Это не пустота. Это данные, но слабые данные. Они говорят: в данном запуске, при данном input and timeout, route не проявился. Они не говорят: route невозможен. Такое различие кажется занудным, но оно защищает от ложных отрицаний.

Например, static analysis видит AddVectoredExceptionHandler and RaiseException in rarely executed branch. Dynamic validation запускает программу без нужного input, branch не достигается, handler не hit. Правильный assessment: route not observed; dynamic coverage insufficient to dismiss static finding. Неправильный assessment: false positive. Другой пример: static analysis видит invalid unwind metadata in function never executed during session. Dynamic validation не видит crash. Это не отменяет статический finding, потому что проблема может проявиться при другом path or exception.

Поэтому report должен уметь выражать отсутствие события аккуратно. Not observed is not disproved. Timed out is not clean. No handler hits is not no handlers. No second chance is not no crash potential. Это язык зрелого анализа. Он может раздражать тем, что не даёт простых ответов, но именно так работают честные инструменты.

Техническая база раздела

В разделе использована модель Microsoft x64 exception handling: RUNTIME_FUNCTION entries в .pdata с image-relative адресами, UNWIND_INFO в .xdata, handler flags, chained unwind info и dynamic function tables для runtime-generated code.

Для порядка доставки исключения использованы документы Microsoft по debugger exception handling, vectored exception handling, ContinueDebugEvent, SetUnhandledExceptionFilter, UnhandledExceptionFilter и RtlVirtualUnwind.

Термины static analysis, heuristic/evidence engine, dynamic validation, Windows dynamic collector, DynamicReport, findings и assessment употребляются в проектном смысле EH/Unwind Analyzer, без переименования их во внешние ярлыки.

10.3. Мышление реверсера: как собрать адрес, стек, metadata и runtime-события в одну проверяемую версию

От знания терминов к умению вести расследование

К этому месту книги у нас уже есть почти все детали механизма. Мы знаем, что в Windows x64 адрес выполнения живёт в RIP, что стек держится вокруг RSP, что обычный вызов функции оставляет на стеке return address, что nonvolatile-регистры должны быть восстановлены, если функция их меняла. Мы знаем, что PE-файл хранит Exception Directory, что этот directory ведёт к таблице RUNTIME_FUNCTION, что каждая запись описывает диапазон кода, а не красивое имя функции. Мы знаем, что .xdata и UNWIND_INFO объясняют системе, как разобрать кадр, и что RtlVirtualUnwind вычисляет caller context без исполнения обычного ret. Мы уже прошли маршрут исключения: от fault или RaiseException до first-chance debugger notification, VEH, frame-based SEH, unwind phase, second chance и unhandled path.

Но знание этих слов ещё не делает человека реверсером. Можно выучить RIP, RVA, UNWIND_INFO, handler, debug event и всё равно теряться перед реальным бинарём. Причина в том, что reverse engineering редко задаёт вопрос в чистом учебном виде. Реальный вопрос звучит грязнее и тревожнее: программа упала по адресу; debugger показывает first-chance exception; в .pdata есть странный диапазон; report говорит о handler dominance; DynamicReport не увидел handler hit; Ghidra показывает функцию, но stack trace выглядит неправдоподобно. Нужно не просто вспомнить определение каждого термина, а собрать из них версию происходящего.

Слово version здесь важно. В анализе бинаря мы почти никогда не начинаем с истины. Мы начинаем с фрагментов. Один фрагмент даёт файл на диске. Второй даёт загрузчик Windows, который превратил файл в image в памяти. Третий даёт текущий поток, остановленный на конкретном RIP. Четвёртый даёт стек. Пятый даёт unwind metadata. Шестой даёт dynamic validation: события от Windows Debug API, handler hits, RIP trace, регистрации VEH/VCH/UEF, вызовы RtlAddFunctionTable. Реверсер не должен хватать один фрагмент и объявлять его полной картиной. Он должен построить проверяемую версию: что, вероятно, произошло; какие свидетельства это поддерживают; какие свидетельства противоречат; какие области остаются unknown.

Именно поэтому этот раздел не будет вводить новый крупный механизм. Его задача другая: собрать способ мышления. Мы будем учиться держать в голове несколько слоёв одновременно и не путать их. Файл не равен памяти. Адрес не равен смыслу. Handler RVA не равен факту срабатывания. First-chance

exception не равен crash. Отсутствие события в dynamic validation не равно отсутствию механизма. Finding не равен приговору. Assessment не равен гаданию. Эти различия кажутся простыми, пока они стоят отдельными фразами. В реальном анализе именно они спасают от ложных выводов.

Один адрес не является объяснением

Представим, что у нас есть адрес: 00007FF6A1234567. Такой адрес может выглядеть убедительно. Он длинный, похож на настоящую точку в 64-битном процессе, debugger подсвечивает его как RIP, crash dump показывает его в exception address. Новичок часто хочет сразу спросить: что находится по этому адресу? Но правильный первый шаг другой: где живёт этот адрес? Адрес без модуля - это почти бездомная координата. Он существует в виртуальном адресном пространстве процесса, но сам по себе не говорит, принадлежит ли он main executable, DLL, JIT-коду, private executable memory, распакованному payload, thunk-у, guard page stub или вообще уже освобождённой памяти.

Виртуальный адрес, virtual address или VA, - это адрес так, как его видит процесс во время выполнения. Он относится к конкретному address space, то есть к конкретному виртуальному пространству процесса. В Windows разные процессы могут иметь одинаковые численные адреса, но эти числа указывают на разные физические или отображённые страницы. Поэтому VA всегда нужно привязывать к процессу и к memory map. Когда мы говорим RIP = 00007FF6A1234567, мы говорим не просто число. Мы говорим: в таком-то процессе, в таком-то потоке, в этот момент instruction pointer указывает на этот virtual address.

Дальше появляется module. Module - это загруженный образ: EXE или DLL, который Windows loader отобразил в память. У module есть base address, адрес начала его образа в памяти. В PE-заголовках есть ImageBase - предпочитаемый базовый адрес, который компилятор или linker записал в файл. Но из-за ASLR, relocations и обычной работы loader-а реальный runtime base может отличаться. Поэтому надо быть осторожным: ImageBase из файла - это заявление файла, а module base в процессе - это факт текущей загрузки.

Когда мы нашли module, адрес начинает получать смысл. Если RIP попадает внутрь диапазона module, мы можем посчитать RVA. Relative virtual address, RVA, переводится как относительный виртуальный адрес. Он показывает смещение от начала image, а не абсолютный адрес в процессе. Формула выглядит простой:

```
RVA = VA - ModuleBase
```

Но за этой формулой стоит важная дисциплина. RVA позволяет связать runtime-событие с PE-структурами. Exception Directory хранит адреса как RVA. RUNTIME_FUNCTION хранит BeginAddress, EndAddress and UnwindData как image-relative значения. Sections в PE тоже описывают virtual ranges

относительно image. Поэтому RVA - это язык, на котором runtime address начинает разговаривать с PE metadata. Пока у нас только VA, мы смотрим на живой процесс. Когда у нас есть RVA, мы можем спросить файл: какой диапазон, какая секция, какая unwind-запись, какие flags, какой handler data соответствуют этой точке.

File offset - ещё один адресный язык, и его нельзя путать с RVA. File offset - смещение в байтах от начала файла на диске. Оно нужно, когда мы читаем PE как массив bytes. Но секция в файле и секция в памяти не обязаны иметь одинаковое расположение. Raw data может быть выровнено по FileAlignment, virtual layout - по SectionAlignment. Некоторые virtual bytes могут вообще не иметь raw data в файле, например BSS-подобные области. Поэтому переход RVA -> file offset требует section mapping. Если анализатор просто вычитает base и идёт в файл как в плоский массив, он рано или поздно прочитает не те байты.

Так один адрес превращается в цепочку вопросов. Это уже не “RIP где-то там”. Это: какой process, какой thread, какой module range, какой module base, какой RVA, какая section, есть ли raw mapping, попадает ли RVA в Exception Directory, покрывает ли его RUNTIME_FUNCTION. Реверсер, который держит эту цепочку, меньше подвержен магическому мышлению. Он понимает: смысл адреса не в числе, а в системе координат, в которую это число помещено.

Файл, образ и выполнение как три разных мира

Следующая ошибка рождается из желания считать PE-файл полной правдой о программе. На диске лежит file bytes - байты файла. Windows loader читает их, проверяет заголовки, маппит sections, применяет relocations, разрешает imports, вызывает TLS callbacks and entry point. После этого в процессе появляется mapped image - образ в памяти. Но затем программа продолжает жить: она может менять protection страниц, создавать private executable memory, генерировать код, распаковывать payload, регистрировать dynamic function tables, ставить или убирать обработчики исключений. Поэтому есть как минимум три мира: file on disk, image in memory, runtime behavior.

File on disk удобен для static analysis. Static analysis - статический анализ - означает, что инструмент изучает файл без его выполнения. Он может прочитать заголовки, sections, data directories, .pdata, .xdata, imports, relocation information, raw bytes and metadata placement. Его сила в том, что он безопасен и воспроизводим: один и тот же файл даёт один и тот же набор статических наблюдений. Но его граница тоже ясна: он не видит всего, что появится во время выполнения. Если код будет сгенерирован после запуска, статический анализ файла не обязан увидеть его как часть исходного PE.

Mapped image - это уже не файл, а то, как loader разместил image в памяти. Здесь важны реальные base addresses, реальные protection flags, реально загруженные modules. Например, static report может показать handler RVA

внутри image, но dynamic validation должна перевести его в runtime VA через фактический module base. Если base изменился из-за ASLR, прямое использование preferred ImageBase даст ложный breakpoint. Поэтому динамический слой не может просто “поверить файлу”. Он должен построить module map - карту загруженных модулей в процессе: base, size, module name, путь, а затем переводить RVA в VA уже через runtime data.

Runtime behavior - ещё более живой слой. Он отвечает не за то, что файл содержит, а за то, что произошло в конкретном запуске. В этом запуске мог сработать один code path и не сработать другой. Handler мог быть зарегистрирован, но не hit-нуться. Exception мог быть first-chance и после этого нормально обработан. Dynamic function table мог быть добавлен только после определённого user action. Поэтому dynamic validation - динамическая проверка - не является магическим способом увидеть всё. Это наблюдение конкретной жизни процесса в конкретных условиях. Оно даёт runtime evidence, свидетельства выполнения, но эти свидетельства всегда зависят от сценария, timeout, coverage and instrumentation completeness.

Реверсер должен уметь переходить между этими мирами без смешивания. Если static analysis говорит, что в файле есть UNW_FLAG_EHANDLER, это не значит, что handler сработал. Если dynamic validation говорит, что handler hit observed, это уже сильнее: событие действительно произошло в этой session. Если dynamic validation не увидела handler hit, это не значит, что handler не существует или никогда не срабатывает. Это значит: в наблюдаемом сценарии событие не было зафиксировано. Такая аккуратность выглядит медленной, но она и отличает инженерный анализ от гадания.

Unwind metadata как договор, а не декоративная информация

Теперь вернёмся к .pdata and .xdata, но уже не как к структурам ради структур, а как к договору между машинным кодом, компилятором, Windows and debugging infrastructure. Когда функция на x64 меняет RSP, сохраняет nonvolatile registers, создаёт stack frame или имеет exception handler, Windows должна иметь способ восстановить caller context. Она не может просто прочитать красивый комментарий в дизассемблере. Ей нужны машинно-читаемые сведения. Именно поэтому появляется function unwind information.

.pdata содержит таблицу RUNTIME_FUNCTION. Каждая запись RUNTIME_FUNCTION описывает диапазон кода: BeginAddress, EndAddress, UnwindData. Это не имя функции, не символ и не обещание, что дизассемблер красиво назовёт блок. Это range, диапазон RVA, для которого действует определённая unwind model. Когда RIP попадает внутрь этого диапазона, Windows может найти соответствующую запись через RtlLookupFunctionEntry. Если запись найдена, UnwindData ведёт к UNWIND_INFO. Если запись не

найдена, Windows может применить leaf-function assumption: считать, что функция не изменила nonvolatile state и что caller context можно восстановить как обычный return. Иногда это нормально. Иногда это катастрофически неправильно.

.xdata содержит UNWIND_INFO. Это не “описание функции вообще”, а инструкция, как отменить эффекты prolog-а и восстановить состояние caller-а. Prolog - это начальная часть функции, где callee строит свой stack frame: двигает RSP, сохраняет регистры, может установить frame register. Callee - это вызываемая функция; caller - функция, которая вызвала её. Эта пара важна, потому что unwind всегда пытается подняться от callee context к caller context. Если функция сохранила RBX на стеке, UNWIND_INFO должен объяснить, где он сохранён. Если функция сделала sub rsp, 0x80, UNWIND_INFO должен объяснить размер allocation. Если функция использует RBP как frame register, metadata должна объяснить frame register and offset.

Nonvolatile registers - это регистры, которые callee обязан вернуть caller-у в прежнем состоянии, если он их менял. В Windows x64 сюда относятся, например, RBX, RBP, RSI, RDI, R12-R15 and some XMM registers. Слово nonvolatile можно перевести как “неиспаряющиеся” или “сохраняемые”: caller имеет право ожидать, что после вызова эти значения не исчезнут. Если callee временно использует RBX, он должен сохранить старое значение and restore it before returning. При обычном ret это делает сам код. При exception unwind обычного ret нет, но caller context всё равно надо восстановить. Вот для этого metadata и нужна.

Return address - адрес возврата - тоже нельзя воспринимать как абстрактную фразу. Инструкция call кладёт на стек адрес следующей инструкции, то есть точку, куда callee должна вернуться после завершения. Ret забирает этот адрес со стека и загружает его в RIP. При virtual unwind RtlVirtualUnwind не исполняет ret, но логически должен получить похожий результат: новый RIP должен стать адресом возврата caller-а, новый RSP должен указывать на состояние caller-а, nonvolatile-регистры должны быть восстановлены. Поэтому unwind metadata - это мост между текущим повреждённым или остановленным состоянием и тем состоянием, которое было бы после корректного выхода из функции.

Если эта metadata честная, stack walking, exception search and crash dump analysis работают. Если metadata синтаксически валидна, но семантически не соответствует коду, всё становится хуже: парсер не падает, но вывод может быть ложным. Например, .xdata говорит, что функция выделила 0x20 bytes, а код реально сделал sub rsp, 0x80. Тогда вычисленный caller RSP будет неправильным. Stack trace уйдёт в мусор, handler search может пойти не туда, dynamic report может зафиксировать странный route. Поэтому static analysis должен не только декодировать structures, но и сравнивать declared model with observed code effects там, где это возможно.

CONTEXT как фотография машины в момент остановки

Когда мы говорим о runtime events, нельзя забывать CONTEXT. CONTEXT - это структура, которая хранит снимок состояния потока. Слово “снимок” здесь удобнее сухого “структура с регистрами”. Представь, что выполнение потока остановили и сфотографировали машину: где RIP, где RSP, какие значения в RAX, RBX, RCX, RDX, какие flags, какие segment registers, какие SIMD registers if requested. Эта фотография не объясняет сама по себе, почему поток оказался здесь. Но без неё невозможно понять, куда он может продолжить.

В обработке исключений CONTEXT особенно важен, потому что handler может не только прочитать состояние, но иногда и повлиять на продолжение. Если vectored handler получает EXCEPTION_POINTERS, он получает доступ и к EXCEPTION_RECORD, и к CONTEXT. EXCEPTION_RECORD отвечает на вопрос “что произошло”, CONTEXT - “в каком состоянии поток был остановлен”. Если handler меняет ContextRecord->Rip and returns EXCEPTION_CONTINUE_EXECUTION, он фактически говорит Windows: продолжай, но уже с изменённым instruction pointer. Это один из ключевых механизмов exception-based control flow.

Для реверсера CONTEXT - точка встречи static and dynamic. Static analysis может сказать: здесь есть потенциальный handler. Dynamic validation может показать: во время события CONTEXT имел такой RIP, такой RSP, а после handler-a RIP изменился на другой диапазон. Если report показывает RIP trace, это не полный trace всей программы, а короткий след после интересного runtime event. Его надо читать именно как ограниченное свидетельство: после такого-то handler hit выполнение пошло в такие-то addresses. Это очень полезно, но не равно доказательству всей логики программы.

RSP в CONTEXT заслуживает отдельного уважения. Новичок часто смотрит только на RIP, потому что RIP указывает “где мы”. Но RSP отвечает на вопрос “на каком стеке мы стоим”. Если RSP повреждён, если он указывает вне нормального stack range, если return address выглядит недостоверно, то даже правильная unwind metadata может не спасти stack walking. И наоборот, если .xdata странная, но runtime RSP/RIP trace ведут себя согласованно, finding может потребовать manual review rather than immediate high-confidence conclusion. Хороший анализ держит RIP and RSP вместе.

Route hypothesis и evidence chain

Когда static analysis видит необычную структуру, она рождает не приговор, а route hypothesis. Hypothesis - гипотеза, предположение, версия. Route - маршрут. Route hypothesis означает: на основе файла есть версия о том, как управление может пройти. Например, UNWIND_INFO содержит EHANDLER; значит, для этого function range возможно участие language-specific handler. Или код содержит int 3 and registered VEH callback; значит, возможно, breakpoint exception используется как route. Или .pdata не покрывает участок executable code; значит, возможно, это leaf/stub/padding, а возможно - generated or hidden code без static metadata.

Evidence chain - цепочка свидетельств - это способ не потерять осторожность. Она начинается с observation. Observation - наблюдение: конкретный факт. Например: RUNTIME_FUNCTION range overlaps another range. Interpretation - интерпретация: почему это важно. Overlap может мешать однозначному поиску function entry for ControlPc. Conclusion - вывод: требуется manual review; confidence depends on placement, compiler patterns, dynamic evidence. Если dynamic validation потом видит exception inside overlapping range and abnormal unwind behavior, chain strengthens. Если никаких runtime evidence нет, chain remains static and limited.

В этом смысле report должен быть похож на хорошо оформленное расследование. Он не должен кричать “bad”. Он должен показать, где найден факт, на каком слое, какая нормальная модель ожидалась, чем факт отличается от нормы, какие benign explanations возможны, что усиливает уверенность, что ослабляет, что можно проверить динамически or manually. В проектных терминах это и есть смысл heuristic/evidence engine: не заменить реверсера, а подготовить для него честную, воспроизводимую evidence model.

Здесь важно слово reproducible - воспроизводимый. Если static analyzer говорит, что unwind_rva указывает за пределы image, другой запуск анализа того же файла должен получить тот же факт. Если dynamic validation говорит, что handler hit observed, это привязано к конкретной сессии: target arguments, timeout, path coverage, environment. Поэтому static evidence and runtime evidence имеют разную природу. Их можно соединять, но нельзя смешивать как одинаковые.

Сцена первая: crash address как начало, а не конец анализа

Теперь соберём всё в живую сцену. Есть crash dump или debug session. ExceptionAddress указывает на VA внутри process. Первое желание - открыть дизассемблер и смотреть инструкцию. Это полезно, но преждевременно, если мы ещё не знаем context. Правильная внутренняя речь реверсера звучит медленнее: этот RIP надо поместить в module. Если module найден, надо получить runtime base and RVA. Если module не найден and address lies in private executable memory, это отдельный сигнал: возможно generated code, unpacked payload, manual mapping or memory corruption.

После вычисления RVA мы смотрим section mapping. Попадает ли адрес в .text? В executable section with normal permissions? В секцию с подозрительными flags? Затем идём к Exception Directory. Не к строке “.pdata” в имени секции, а к data directory, потому что именно directory сообщает официальное местоположение function table for image. По RVA ищем RUNTIME_FUNCTION range. Если range найден, читаем UnwindData, попадаем в UNWIND_INFO, смотрим flags, prolog size, unwind codes, handler information. Если range не найден, не надо сразу объявлять ошибку: возможно leaf function, padding, thunk. Но если вокруг есть sub rsp, сохранение nonvolatile-регистров or calls, отсутствие записи становится сильнее.

Теперь смотрим CONTEXT. Где RSP? Что лежит по RSP as possible return address? Соответствует ли expected caller? Если RtlVirtualUnwind mentally or tool-assisted produces caller context, выглядит ли новый RIP plausible? Попадает ли он в executable module? Есть ли следующая RUNTIME_FUNCTION? Если stack walk резко уходит в невалидный address, надо разделить возможные причины: corrupt stack, wrong unwind metadata, wrong module mapping, code executed from dynamic region without registered function table, or analysis bug.

Только после этого crash address начинает быть объяснением. Мы уже не говорим “упало по адресу”. Мы говорим: событие произошло в таком-то module, at RVA, inside this RUNTIME_FUNCTION range; UNWIND_INFO declares this frame layout; context shows this RSP; virtual unwind would produce this caller; handler flags absent/present; dynamic evidence observed/not observed. Это не просто больше слов. Это переход от числа к модели.

Сцена вторая: exception-based route, который не виден как обычный branch

Другая сцена начинается не с crash, а с странного control flow. В коде есть int 3, RaiseException or access to guard page. В обычном CFG дизассемблер показывает линейный поток и несколько branch instructions, но реальное выполнение внезапно оказывается в блоке, на который нет очевидного call or jmp. Новичок может подумать, что дизассемблер ошибся или что там “магия”. На самом деле маршрут может идти через exception handling.

Static analysis в такой ситуации ищет несколько слоёв. Есть ли imports or resolved API for AddVectoredExceptionHandler, AddVectoredContinueHandler, SetUnhandledExceptionFilter, RaiseException? Есть ли function ranges with EHANDLER or UHANDLER? Есть ли suspicious exception sites: int 3, ud2, deliberate invalid access, guard page setup through memory protection changes? Есть ли metadata suggesting handlers? Но всё это пока только declared or potential. Оно говорит: здесь есть инфраструктура, которая может участвовать в route.

Dynamic validation может перевести эту гипотезу в наблюдение. Windows dynamic collector запускает target as debuggee, получает debug events, ставит breakpoints on observed/static handler addresses, наблюдает first-chance exceptions, registrations, handler hits and RIP trace. Если breakpoint exception occurred, VEH handler hit observed, and post-handler RIP moved to a block not reached by ordinary CFG, route hypothesis becomes much stronger. Но даже здесь надо говорить аккуратно: observed in this session. Другой input path might not trigger it. Timeout might stop before trigger. Anti-debug behavior might change route because debuggee knows it is being debugged.

Так реверсер учится не спорить с инструментами, а задавать им правильные вопросы. Static analysis отвечает: какие маршруты заявлены or suggested by file and code? Dynamic validation отвечает: какие события and handlers were observed in this run? Assessment отвечает: насколько эти слои согласуются and what remains unknown? Хорошая mental model не требует, чтобы один инструмент сказал всю правду. Она строится на согласовании частичных свидетельств.

Сцена третья: отчёт как карта чтения, а не финальный приговор

Представим, что у нас есть JSON report. В нём есть input, sections, exception_directory, functions, heuristic_analysis, dynamic_validation and assessment. Плохой способ чтения - сразу искать final_verdict and score. Это похоже на чтение медицинского анализа только по слову “норма/не норма”, без показателей. Хороший способ - идти по слоям.

Сначала блок input and report_metadata отвечает на вопрос: что анализировали, какой версией инструмента, с какими опциями, в каком

режиме. Затем PE validation говорит, насколько сам контейнер надёжен: заголовки, секции, диапазоны sections, directory placement. Потом блок functions and unwind-данные показывают static unwind model: какие ranges найдены, какие UNWIND_INFO декодированы, какие flags выставлены, какие anomalies возникли. Heuristic analysis объясняет не просто “подозрительно”, а какие evidence families involved: integrity таблицы, integrity unwind-данных, размещение metadata, reuse unwind-записей, handler dominance and other signals. Dynamic validation показывает runtime observations, если она была включена и successfully ran. Assessment соединяет всё это в осторожный итог.

Если читатель перескакивает сразу к verdict, он теряет смысл. Verdict - это не оракул. Это сжатие evidence model. Score - не процент вредоносности. Confidence - не вероятность в математическом смысле, а оценка надёжности вывода на основании available evidence. Case strength - сила конкретного дела, а не абсолютная истина. Manual validation steps - не украшение, а продолжение анализа: инструмент честно говорит, где человеку надо посмотреть самому.

Такой report должен учить реверсера. Он должен делать мышление видимым. Если report говорит “unwind_rva_out_of_image”, он должен привести адрес, range, expected bounds and consequence. Если report говорит “handler_not_observed”, он должен объяснить, что dynamic session не увидела hit, но это не доказательство отсутствия handler-a. Если report говорит “dynamic_function_table_observed”, он должен связать это с возможным generated code or runtime code and explain why static .pdata may be incomplete. В этом смысле отчёт - не приложение к анализу, а часть обучения.

Реверсер как человек, который удерживает слои отдельно

Главная дисциплина reverse engineering в этой теме - не дать слоям слиться. Слой файла отвечает на вопрос: что записано в PE? Слой загрузки отвечает: как Windows разместила image and modules? Слой текущего потока отвечает: где RIP, RSP and registers now? Слой unwind metadata отвечает: как Windows должна восстановить caller context? Слой exception dispatch отвечает: кто получает exception and in what order? Слой dynamic validation отвечает: что observed in this run? Слой assessment отвечает: как эти свидетельства соотносятся?

Если смешать file offset and RVA, анализатор прочитает не те байты. Если смешать ImageBase and runtime module base, breakpoint попадёт не туда. Если смешать first-chance exception and crash, можно принять нормальную обработку за падение. Если смешать handler RVA and handler hit, можно объявить сработавшим код, который в этой сессии не исполнялся. Если смешать score and malware probability, инструмент начнёт обещать то, чего не доказывает. Большинство серьёзных ошибок здесь не в сложной математике, а в неправильном смешении уровней.

Поэтому хороший реверсер движется медленно. Он не тормозит из-за слабости, а специально сохраняет точность. Он говорит: это статический факт. Это runtime факт. Это гипотеза. Это подтверждённая связь. Это не наблюдалось. Это unknown. Это требует ручной проверки. Такой язык сначала кажется занудным, но именно он позволяет анализировать сложные protected binaries without self-deception.

Внутренняя речь при анализе Windows x64 exception/unwind

Полезно завершить этот раздел не списком признаков, а внутренней речью, которую можно держать в голове перед реальным бинарём. Поток остановился на RIP. Я не знаю смысла этого адреса, пока не нашёл module. Нашёл module - получил runtime base. Из VA сделал RVA. По RVA проверил section. Затем через Exception Directory нашёл function table. Если RUNTIME_FUNCTION найден, я читаю UNWIND_INFO and declared frame model. Если записи нет, я не паникую: проверяю, похож ли код на leaf/stub/padding or uncovered nonleaf.

Дальше я смотрю на CONTEXT. RIP говорит, где остановились. RSP говорит, где стек. Return address показывает возможный путь назад. Nonvolatile registers matter because caller expects them preserved. RtlVirtualUnwind is not real execution; it is computation of caller context. If computed caller looks wrong, I do not immediately blame Windows. I compare code, metadata, stack memory and module map. Если exception involved, я вспоминаю route: debugger first chance, VEH, frame-based SEH, unwind, VCH, second chance, unhandled path. Если dynamic validation включена, я смотрю, какие events actually observed, not what I hoped to see.

Потом я читаю report. Не как приговор, а как structured evidence. Finding показывает observation and interpretation. Heuristic assessment groups signals. DynamicReport shows runtime evidence from a specific session. Assessment correlates but does not magically prove everything. Manual validation steps tell me where human attention matters. Если всё это согласуется, моя confidence grows. Если часть данных отсутствует, я оставляю unknown rather than inventing certainty.

Вот это и есть мышление, ради которого мы строили предыдущие главы. Не запомнить пятьдесят терминов, а научиться проводить адрес через систему координат Windows, провести кадр через unwind model, провести exception through dispatch route, провести report через evidence model. Когда такая связка появляется, reverse engineering становится не угадыванием, а спокойным расследованием.

Медленная дисциплина адресов: один и тот же байт в разных системах координат

В хорошей работе с Windows x64 binary почти всегда есть момент, который кажется слишком простым, чтобы уделять ему внимание: перевод адресов. Но именно здесь ломается огромное количество анализа. Реверсер видит число, инструмент видит число, JSON report видит число, Ghidra показывает число, debugger показывает число - и легко забыть, что эти числа могут принадлежать разным системам координат. Один и тот же байт кода можно назвать file offset, RVA or VA, и каждое имя будет правильным только в своём мире.

File offset принадлежит файлу на диске. Он отвечает на вопрос: на каком расстоянии от начала файла лежит этот байт? Это язык hex editor-a, safe file reader-a, PE parser-a. Когда static analyzer читает недоверенный файл, он в конце концов должен прийти к file offset, потому что физически байты лежат именно там. Но PE-структуры чаще говорят не file offset, а RVA. Поэтому анализатор не имеет права прыгать в файл напрямую по числу из directory. Он должен спросить section table: какой section covers this RVA, где у неё PointerToRawData, какой SizeOfRawData, какой VirtualAddress, какой VirtualSize, не выходит ли запрос за границы.

RVA принадлежит image-модели. Это уже не чистый файл, но ещё не конкретная загрузка в конкретном процессе. RVA удобно тем, что он стабилен относительно начала image. Если ASLR загрузил image по другому VA, RVA не меняется. Поэтому .pdata and .xdata используют image-relative values. RUNTIME_FUNCTION.BeginAddress не говорит: absolute VA такой-то. Он говорит: от начала image до начала диапазона столько-то. Это делает metadata переносимой между разными загрузками одного image.

VA принадлежит живому процессу. Это то, что видит CPU and debugger. RIP is a VA. Breakpoint ставится на VA. Debug event reports addresses in runtime address space. Если dynamic validation хочет поставить breakpoint на handler, который static analysis нашёл как handler RVA, она должна сделать мост: runtime VA = module base observed in debuggee + handler RVA. Если вместо observed module base взять preferred ImageBase from file, breakpoint может попасть в пустоту or another module. Ошибка будет выглядеть мистически: static analysis нашёл handler, dynamic collector вроде поставил breakpoint, но hit never happens. На самом деле ошибка была не в handler-е, а в системе координат.

Поэтому в голове реверсера должна быть спокойная привычка: каждое число спрашивать, кто ты? Ты file offset? Ты RVA? Ты VA? Ты module-relative address from report? Ты preferred VA из статического disassembler view? Ты runtime VA from debugger? Такой вопрос не бюрократия. Это защита от ложного анализа. Когда книга говорит “адрес”, она почти всегда обязана уточнить слой. Если уточнения нет, хороший читатель должен насторожиться.

Медленная дисциплина стека: почему RSP не менее важен, чем RIP

RIP притягивает внимание, потому что он показывает текущую инструкцию. Но unwind-модель почти невозможно понять, если смотреть только на RIP. RSP - stack pointer - указатель стека - показывает место, где текущий поток хранит ближайшие stack values. На x64 стек растёт вниз: когда функция выделяет место, RSP обычно уменьшается; когда освобождает, увеличивается. Call кладёт return address на стек. Push уменьшает RSP and writes value. Pop reads value and increases RSP. Поэтому RSP не просто регистр. Это текущее положение кадра в памяти.

Стековый кадр, stack frame, - это не обязательно красивая структура с фиксированным RBP. В старой x86-интуиции многие привыкли к frame pointer: push ebp, mov ebp, esp, and then locals by ebp offsets. В Windows x64 такой образ часто неверен. RBP может быть обычным nonvolatile register, frame pointer может отсутствовать, а функция может адресовать locals через RSP offsets. Поэтому Windows x64 и нуждается в unwind metadata: если нельзя всегда идти по цепочке frame pointer-ов, надо иметь другой источник знания о том, как функция изменила стек.

Когда RtlVirtualUnwind восстанавливает caller context, он не “читает стек как список кадров” в грубом смысле. Он использует текущий CONTEXT and function unwind information. Если RIP находится в body функции, он применяет модель: функция уже выполнила prolog, значит надо отменить эффекты prolog-a. Если metadata говорит, что RSP был уменьшен на 0x80, virtual unwind увеличит RSP accordingly. Если metadata говорит, что RBX saved at offset, она восстановит RBX from stack memory. Затем return address becomes new RIP. Это вычисление очень чувствительно к двум вещам: правильности metadata and сохранности stack memory.

Если stack memory corrupted, правильная metadata может дать неправильный caller. Если metadata corrupted or mismatched, хороший stack memory будет прочитан неправильно. Поэтому при странном stack trace нельзя автоматически обвинять только .xdata. Нужно смотреть оба мира: declared frame model and actual stack bytes. Иногда return address лежит не там, где его ожидает metadata. Иногда RSP уже повреждён до exception. Иногда prolog был необычным, но корректно описан. Иногда function table missing because code is dynamic and registered later. Хороший реверсер не говорит “stack trace плохой, значит всё сломано”. Он спрашивает: что именно перестало согласовываться - RSP, return address, unwind codes, module map or memory validity?

Эта дисциплина особенно важна для manual validation. Если report говорит prolog mismatch, реверсер открывает функцию, смотрит первые инструкции, ищет sub rsp, pushes, saves of nonvolatile registers, frame register setup. Но он

также смотрит, что было в runtime CONTEXT: где RSP at exception, какой return address visible, совпадает ли expected allocation. Так теория перестаёт быть таблицей UWOP_* and becomes a way to reason about a live frame.

Медленная дисциплина исключений: событие не равно падению

Второй большой источник ложных выводов - слово exception. В бытовом языке exception почти всегда звучит как “авария”. В Windows exception dispatch это шире. Исключение может быть аварией, сигналом, отладочным событием, механизмом восстановления, частью runtime-a or intentional control-flow path. Поэтому реверсер, увидев first-chance exception, не должен автоматически писать “crash”. First chance literally means the debugger is being notified before normal application handlers have had their full opportunity to process the exception. Это первый шанс внешнего наблюдателя, а не последний вздох процесса.

Представим breakpoint exception. Если программа запущена под debugger, debugger naturally sees it. Но breakpoint может быть не только точкой останова пользователя. Инструкция int 3 может быть в программе как deliberate exception site. Если debugger заберёт exception себе и не пропустит дальше, программа может вести себя иначе, чем без debugger-a. Это и есть основа некоторых anti-debug and exception-based control-flow tricks. Но опять же, термин anti-debug нельзя бросать слишком быстро. Надо смотреть, что делает handler. Он проверяет BeingDebugged? Он меняет CONTEXT? Он возвращает CONTINUE_EXECUTION? Он просто логирует? Он только diagnostic? Термин рождается из поведения, а не из одного байта CC.

Access violation тоже не всегда простая смерть. Чаще всего это действительно ошибка памяти. Но deliberately generated access violation can be used as signal. Guard page исключение может быть нормальным механизмом lazy allocation, stack growth, instrumentation or protection. Single-step exception может быть debugger-related or trap-flag-based control. RaiseException может быть language/runtime protocol or application-level signal. Поэтому каждый exception code нужно читать вместе с route: кто увидел, кто пропустил, кто обработал, был ли unwind, changed ли CONTEXT, куда ушёл RIP.

Dynamic validation помогает только если мы помним её границы. Она может показать first-chance event, second-chance event, handler hit, registration, RIP trace. Но она не обязана объяснить мотив. Если handler hit observed, мы знаем, что конкретный handler executed in this run. Если post-handler RIP moved, мы знаем route fragment. But motive remains analysis work. Это может быть obfuscation, anti-debug, normal runtime, crash recovery, test harness or compiler artifact. Поэтому report должен отделять observed route from interpretation.

Медленная дисциплина names: символы, функции и реальные диапазоны

Дизассемблер любит давать имена. Ghidra может назвать функцию FUN_140012340. Symbols can name something cleaner if PDB exists. Import table gives API names. Но unwind-модель Windows не обязана совпадать with user-friendly names. RUNTIME_FUNCTION range is not a symbol. It is a range of code for unwinding. A decompiler function can start earlier or later than a runtime function range. Thunks, hotpatch padding, jump islands, chained unwind info and compiler layout can make the relationship non-trivial.

Поэтому, когда report говорит about function entry, надо понимать, что “function” in report may mean a RUNTIME_FUNCTION entry, not a source-level function and not a decompiler function. Это не ошибка терминологии, а особенность слоя. Source-level function - то, что видел программист. Compiler-emitted procedure - машинный код after optimization. RUNTIME_FUNCTION - metadata range for unwind. Decompiled function - попытка инструмента восстановить high-level structure. Эти сущности часто совпадают enough, but not always.

Если реверсер забывает это, он начинает спорить с инструментом не по делу. Ghidra показывает одну функцию, report shows two RUNTIME_FUNCTION ranges. Кто прав? Возможно оба, потому что они отвечают на разные вопросы. Ghidra пытается построить control-flow and decompilation unit. Unwind metadata tells Windows how to unwind ranges. Если ranges split due to chained info or compiler-specific layout, static analyzer должен показывать metadata truth, not force it into decompiler names.

Это особенно важно для Ghidra CSV. Его задача - перенести useful unwind observations into reverse engineering environment: ranges, comments, warnings, bookmarks. Но Ghidra importer не должен превращать metadata into absolute source truth. Он помогает смотреть, где Windows unwind model says something interesting. Реверсер всё равно должен сверять code, metadata, reports and runtime observations.

Как читать contradiction без паники

Сильный анализ начинается там, где слои не сходятся. Static analysis говорит: handler RVA present. Dynamic validation says: handler not observed. Это contradiction? Не обязательно. Возможно scenario did not trigger exception. Возможно timeout ended before path. Возможно breakpoint not installed because module loaded late. Возможно handler address was wrong due to rebasing issue. Возможно handler exists but belongs to cleanup path rather than exception path. Поэтому первое правило противоречия: не превращать difference into accusation.

Другой пример: .pdata has gap inside executable .text. Это может быть padding, leaf function, tiny thunk, alignment bytes, jump table, embedded data, compiler artifact. Но если gap contains nonleaf-like code with sub rsp, calls and saved nonvolatile registers, then suspicion grows. Если dynamic validation shows RIP inside gap and exception occurs there with bad stack walk, suspicion grows further. Если additionally RtlAddFunctionTable not observed and module map shows private executable region, the case changes again. Evidence accumulates layer by layer.

Третий пример: dynamic function table observed. Это не automatically malicious. JIT engines, runtime-generated code, language runtimes and legitimate instrumentation may use dynamic function tables. But in a packed binary, after unpacking into private executable memory, dynamic function table registration can be a crucial explanation for why static .pdata from file is incomplete. Meaning depends on context. Поэтому heuristic/evidence engine must not treat every signal as same severity in every environment. It must preserve evidence family, confidence and weakening rules.

Хорошее мышление при contradiction похоже на работу следователя, а не судьи. Следователь не говорит “противоречие, значит виновен”. Он говорит: какие версии объясняют это? Какая версия требует меньше допущений? Какие данные могут усилить or weaken each version? Какие manual validation steps should be next? В reverse engineering это особенно важно because binaries can be optimized, protected, packed, partially corrupted, compiled by different toolchains, or observed under debugger-sensitive conditions.

Как из этой модели рождается следующий раздел книги

Теперь видно, почему финальная архитектурная глава должна быть не каталогом файлов проекта, а продолжением этой mental model. main.c не просто “точка входа”. Он orchestrates the analysis pipeline. pe.c не просто “парсер PE”. Он держит trust boundary and safe RVA mapping. unwind.c не просто “читает структуры”. Он translates RUNTIME_FUNCTION and UNWIND_INFO into a declared unwind model. heuristics.c не просто “эвристики”. Он turns observations into cautious findings. dynamic_win.c не просто “запускает процесс”. Он observes runtime evidence through Windows Debug API. report.c не просто “печатает JSON”. Он serializes reasoning so a human and tools can inspect it.

Если описывать архитектуру проекта без этой подготовки, получится сухая документация: файл делает то, файл делает это. Но теперь можно написать лучше. Каждый модуль будет объясняться через проблему, которую он решает. Почему safe reading matters? Because input PE is untrusted. Почему section mapping matters? Because RVA is not file offset. Почему unwind decoder matters? Because Windows x64 unwinding depends on metadata. Почему heuristic/evidence engine matters? Because raw anomalies are not conclusions. Почему dynamic validation matters? Because runtime behavior can differ from file claims. Почему

reports matter? Because analysis that cannot be explained cannot be trusted.

Практика без кнопочного мышления: как смотреть на x64dbg, WinDbg и Ghidra

Инструменты легко превращают reverse engineering в набор кнопок. Открыл Ghidra, нажал decompile, увидел псевдокод. Открыл x64dbg, нажал run, увидел exception. Открыл JSON, посмотрел verdict. Но книга должна научить не нажимать, а понимать, что именно показывает каждый инструмент и чего он не показывает. Ghidra в первую очередь помогает видеть static code structure. Она строит functions, basic blocks, decompiler output, references, comments. Но Ghidra не является Windows loader-ом and not the Windows exception dispatcher. Если Ghidra показывает, что блок unreachable, это может означать unreachable in ordinary CFG. Но exception-based route может привести туда through handler continuation.

x64dbg and WinDbg показывают живое выполнение, но тоже не дают готовой истины. Debugger видит events because Windows reports them. Он может остановиться on first-chance exception. Но остановка debugger-а не равна crash. Нужно смотреть status: first chance or second chance, exception code, address, thread, module, whether continuing with handled or not handled changes route. В x64dbg можно увидеть RIP and registers. В WinDbg можно получить stack trace. Но stack trace is only as good as unwind information and stack state. Если stack trace странный, это не всегда debugger bug and not always malware. It may be metadata mismatch, stack corruption, dynamic code without registered tables, or wrong symbols.

Ghidra CSV из проекта полезен именно как мост. Static analyzer может экспортировать ranges, comments and warnings into Ghidra. Это не заменяет manual reverse engineering, а расставляет дорожные знаки. Реверсер смотрит на функцию and sees: здесь RUNTIME_FUNCTION range, здесь UNWIND_INFO flags, здесь finding about unusual metadata placement. Это помогает не забыть о Windows unwind layer while reading code. Но комментарий в Ghidra is not proof by itself. Он points back to report evidence. Поэтому правильный workflow: перейти из report в Ghidra, увидеть code, сверить metadata, then if needed use dynamic validation to observe route.

WinDbg особенно полезен для понимания caller context and exception route. Но и здесь важно не превращать command output into magic. Команда stack walking internally relies on unwind metadata. When it fails or looks odd, we should ask why. Exception record and context record should be inspected together. If an exception is first chance, check whether there are handlers. If second chance, ask why earlier layers did not handle it. If handler address belongs to module range, convert VA to RVA and compare with static analysis. If RIP is outside any module, inspect memory region. This is the same mental model again, just expressed through debugger commands.

Ручная валидация как продолжение отчёта

Manual validation - ручная проверка - не означает, что автоматический анализ провалился. Наоборот, хороший автоматический анализ должен привести человека к точке, где ручная проверка становится осмысленной. Если report просто говорит “suspicious”, человек не знает, куда смотреть. Если report говорит: “RUNTIME_FUNCTION range overlaps another range; both ranges point to different UNWIND_INFO; affected ControlPc lookup may be ambiguous; dynamic validation did not execute this range; manual step: inspect RVA X-Y in disassembler and verify compiler layout”, человек получает маршрут работы.

Ручная проверка в этой теме часто начинается с перевода адресов. Report gives RVA. In debugger we need VA. In file view we need file offset. In Ghidra we may need image base normalization. Поэтому manual validation step должен быть точным: inspect RVA, not “address” vaguely. If dynamic event gives VA, report should include module base or resolved RVA. This is not pedantry; it prevents humans from looking at the wrong bytes.

Следующий ручной шаг - сверка code and metadata. Если finding говорит unwind_code_mismatch, надо открыть prolog. Но смотреть надо не глазами “похоже/не похоже”, а через эффекты. Push nonvolatile register changes RSP and stores old register value. Sub rsp allocates stack space. Save nonvolatile to [rsp+offset] stores caller-owned value. Set frame register establishes stable base for addressing. UNWIND_CODE describes those effects, not textual assembly. Поэтому manual validation asks: do declared effects and observed effects match? Если match, finding may be false positive or limited. If mismatch, consequence is wrong caller context.

Третий ручной шаг - проверка runtime route. Если static report suggests exception-based route, run dynamic validation with scenario that reaches the code. If no event appears, ask whether scenario covered path. If event appears, capture exception code, handler hit, post-handler RIP. Then compare post-handler RIP with static CFG. Если it lands in ordinary successor, maybe handler only recovered. If it lands in hidden block, route is more interesting. If CONTEXT modified, inspect handler logic. Manual validation is not random poking; it is answering questions prepared by static and dynamic evidence.

Три уровня уверенности в голове реверсера

Чтобы не прыгать между “точно” and “ничего не понятно”, полезно держать три уровня уверенности. Первый уровень - structural evidence. Это evidence from file structures: PE headers, sections, data directories, .pdata, .xdata, imports, flags, ranges. Structural evidence is strong for saying what file contains. It is weaker for saying what runtime will do. For example, EHANDLER flag structurally exists. That is solid. But whether handler executes in a particular scenario is not proven by structure alone.

Второй уровень - behavioral evidence. Это evidence from runtime observation: debug events, exceptions, registrations, handler hits, RIP trace. Behavioral evidence is strong for saying what happened in this run. It is weaker for saying what always happens. If handler hit observed, it happened. If not observed, it may still happen on another path. Поэтому behavioral evidence should be tied to session properties: arguments, timeout, target exit status, completeness, dropped events.

Третий уровень - explanatory confidence. Это уже не отдельный факт, а качество объяснения. When structural and behavioral evidence agree, confidence grows. If static analysis says handler exists, dynamic validation observes handler hit, RIP trace shows continuation into expected block, and manual code review confirms CONTEXT modification, explanatory confidence becomes high. If static evidence exists but dynamic session incomplete, confidence stays limited. If static and dynamic disagree, confidence depends on whether we can explain disagreement: scenario coverage, ASLR mapping, dynamic code, late module load, breakpoint failure, anti-debug branch.

Эта трёхуровневая модель помогает читать findings calmly. Severity may be high even when confidence low: potential consequence serious, evidence incomplete. Confidence may be high for a benign conclusion: the structure is unusual but explained by compiler/runtime pattern. Case strength combines context, not just count of warnings. That is why mature reports avoid simple “red/yellow/green” thinking.

Почему хорошая книга должна учить сомневаться правильно

В техническом обучении часто пытаются убрать сомнение: дать таблицу, правило, рецепт. Но в reverse engineering сомнение нельзя убрать. Его нужно организовать. Неправильное сомнение парализует: “ничего нельзя знать”. Неправильная уверенность ломает анализ: “нашёл int 3, значит anti-debug”. Правильное сомнение говорит: у меня есть такой-то факт, он поддерживает такую-то версию, но есть такие-то ограничения. Это не слабость, а инженерная честность.

Windows x64 exception/unwind особенно хорошо учит такому мышлению, потому что здесь много слоёв с разными гарантиями. PE Format gives structure, but file can be malformed. Compiler emits unwind metadata, but obfuscator can mutate it. Windows loader maps image, but runtime can generate code. Debugger sees events, but debuggee can react to being debugged. Static analyzer can parse data, but cannot see future execution. Dynamic collector can observe events, but only in one session. Report can correlate, but cannot replace human judgment. Если всё это понимать, инструмент становится помощником, а не оракулом.

Поэтому финальная цель этой главы не в том, чтобы ты мог перечислить all fields of UNWIND_INFO. Это полезно, но недостаточно. Цель - чтобы при виде

любого нового binary ты автоматически строил layered reasoning: file, image, runtime, stack, metadata, exception route, evidence, limitations. Тогда даже незнакомый packer, strange handler or broken stack trace перестаёт быть мистикой. Ты не знаешь ответ сразу, но знаешь, как его добывать.

Переход к финальной проектной архитектуре

Теперь книга почти дошла до места, где теория должна стать архитектурой конкретного инструмента. Мы уже знаем, почему static analysis обязан начинаться с недоверенного PE-файла, почему section/RVA mapping нельзя пропускать, почему RUNTIME_FUNCTION and UNWIND_INFO являются ядром x64 unwind model, почему heuristic/evidence engine должен формулировать findings осторожно, почему dynamic validation observes only a concrete runtime session, and why reports must teach rather than frighten.

Следующий большой шаг - собрать это уже не как абстрактную Windows-модель, а как архитектуру EH/Unwind Analyzer. Там нужно будет показать, как main.c orchestrates analysis, как pe.c creates safe PE model, как unwind.c decodes function unwind information, как heuristics.c turns observations into findings, как dynamic_win.c collects runtime evidence, как report.c serializes the result, как Ghidra importer helps the reverse engineering workflow, and how tests/corpus/fuzzing protect the tool from self-deception. Но эта архитектурная глава будет понятной только потому, что сейчас у нас есть главное: не набор терминов, а единая mental model.

Глава 11. Архитектура EH/Unwind Analyzer

11.1. Зачем проект существует: от Windows unwind-контракта к защитному анализатору

Проект появляется не из желания написать еще один parser

После десяти глав у нас уже есть достаточно большая mental model: процессор выполняет инструкции, функция строит stack frame, caller кладет return address, callee сохраняет nonvolatile-регистры, Windows x64 не полагается на старую x86-цепочку SEH в стеке, а использует таблицы function metadata, которые лежат в PE image и описывают, как кадр можно восстановить. Но между пониманием этой теории и рабочим инструментом есть огромная пропасть. Теория говорит: в нормальном PE32+ AMD64 должна существовать связка Exception Directory -> RUNTIME_FUNCTION -> UNWIND_INFO -> handler data. Инструмент должен взять чужой файл, не доверять ему, пройти эту связку по байтам, не упасть на поврежденной структуре, сохранить причину отказа, отличить обычную compiler-generated metadata от странной и, если пользователь включил runtime-проверку, аккуратно сравнить статическую картину с живым выполнением.

EH/Unwind Analyzer существует именно в этой пропасти. Его задача не в том, чтобы красиво пересказать Microsoft documentation. Документация описывает контракт: какие структуры существуют, какие поля в них лежат, как операционная система использует function tables и unwind information. Но документация не берет произвольный файл из интернета, не проверяет его секции на overlapping, не объясняет реверсеру, почему один UnwindInfoRVA используется сотней диапазонов, не строит JSON/HTML/Ghidra report и не говорит, какие manual validation steps нужны дальше. Проект становится практическим слоем между официальным контрактом Windows и грязной реальностью reverse engineering.

Важно сразу задать правильный тон. Этот проект не является malware classifier. Он не говорит: score 70 значит вероятность вредоносности 70 процентов. Он не является detector-ом конкретного packer-а или dynamic -а. Он не обещает восстановить исходный control flow, снять virtualization, деобфусцировать payload или доказать намерение автора бинаря. Его область уже и строже: defensive analysis tool для Windows PE32+ AMD64 exception и stack-unwind metadata. Слово defensive здесь не украшение. Инструмент предназначен для анализа и объяснения, а не для обхода защиты, внедрения в чужие процессы ради скрытого поведения или восстановления секретов.

Эта узость как раз делает проект сильным. Чем уже формулировка, тем честнее можно построить архитектуру. Если цель - понять

exception/unwind-инфраструктуру, то входные данные, структуры, эвристики, dynamic collector и отчеты должны быть подчинены одной линии. Не надо распыляться на общий antivirus verdict. Нужно аккуратно ответить на другой вопрос: что этот PE-файл заявляет о своих функциях, кадрах, handler-ах и unwind-модели, насколько эти заявления структурно корректны, насколько они похожи на нормальную compiler-generated картину, и есть ли runtime-свидетельства, что эти механизмы реально участвуют в выполнении.

Поэтому глава 11 будет не каталогом файлов проекта. Каталог легко прочитать в README. Нам нужно другое: понять, почему каждый модуль находится на своем месте и какую часть общей проблемы он закрывает. `util.c` существует не потому, что любому C-проекту нужен util. Он существует потому, что все начинается с недоверенных байтов и безопасного чтения. `pe.c` существует не потому, что PE-parser звучит солидно. Он нужен, потому что без PE image model нельзя честно перевести RVA в file offset и нельзя понять Exception Directory. `unwind.c` нужен, потому что `.pdata` без `.xdata` остается набором диапазонов, а не моделью восстановления кадра. `heuristics.c` нужен, потому что parse success еще не равен meaningful trust. `dynamic_win.c` нужен, потому что файл на диске не равен живому процессу. `report.c` нужен, потому что знание, которое нельзя проверить и объяснить, для реверсера почти бесполезно.

Официальный Windows-контракт как корень архитектуры

Архитектура EH/Unwind Analyzer начинается не с проекта, а с того, как Windows x64 сама устроила exception handling. На 64-битной Windows таблицы function metadata используются для unwind-а и stack walking. Для обычного PE32+ AMD64 compiler и linker размещают `RUNTIME_FUNCTION` entries в области, которая обычно соответствует `.pdata`, а связанные `UNWIND_INFO` structures лежат в `.xdata` или другой read-only metadata-секции. Важный момент: с точки зрения контракта основная точка входа не имя секции `.pdata`, а Exception Directory в PE Optional Header DataDirectory. Имя секции - удобная привычка, но DataDirectory - официальный способ найти таблицу.

Отсюда сразу рождается первая инженерная обязанность проекта. Нельзя просто найти строку `.pdata` в section table и считать, что перед нами корректная таблица функций. В PE section name не является надежным доказательством смысла. Секция может называться странно, отсутствовать, быть переименованной, объединенной, иметь необычные права или быть специально сделанной так, чтобы ломать наивные инструменты. Поэтому `pe.c` идет через PE headers, Optional Header, NumberOfRvaAndSizes и DataDirectory entry for exceptions. Это более скучный путь, зато именно он соответствует контракту Windows image format.

Второй корень архитектуры - то, что `RUNTIME_FUNCTION` содержит image-relative addresses. Это значит, что поля BeginAddress, EndAddress и UnwindInfoAddress не являются file offsets и не являются абсолютными runtime addresses. Они являются RVA, relative virtual addresses, то есть смещениями относительно ImageBase загруженного image. Для реверсера это критично: если он перепутает RVA и file offset, он начнет читать не те байты. Для проекта это превращается в конкретную функцию `pe_rva_to_fileoff`: прежде чем читать структуру из файла, надо доказать, что этот RVA попадает в отображаемую область секции и имеет соответствующий raw range.

Третий корень - сам смысл `UNWIND_INFO`. Это не декоративная отладочная информация. Это компактная программа восстановления кадра. Она говорит, что функция сделала в прологе: сколько байт выделила на стеке, какие nonvolatile-регистры сохранила, использовала ли frame pointer, имеет ли exception handler, termination handler или chained info. Если metadata корректна, `RtlVirtualUnwind` может вычислить invocation context предыдущей функции. Если metadata повреждена или указывает за пределы файла, восстановление caller context становится сомнительным. Поэтому `unwind.c` не может ограничиться чтением трех полей. Он должен пройти codes, operands, alignment, flags, handler tail, chain depth и parse failure reasons.

Четвертый корень - dynamic function tables. Официальный контракт Windows учитывает не только функции, заранее описанные в PE image. Для dynamically generated code приложения должны предоставить function table через механизмы вроде `RtlAddFunctionTable` или callback-таблиц. Значит, статический анализ файла на диске принципиально не может быть полной картиной жизни процесса. Это не недостаток проекта, а свойство Windows. Поэтому в архитектуре появляется отдельный Windows dynamic collector: не вместо static analysis, а рядом с ним, как слой живого свидетельства о runtime function-table activity, handler hits и exception flow.

Пятый корень - Debug API. Если анализатор запускает target как debuggee, он входит в другой режим знания. Он больше не просто читает bytes. Он получает debug events, exception notifications, module load events, breakpoint exceptions, thread/process events. Официальная модель debugger loop через `WaitForDebugEvent` и `ContinueDebugEvent` задает дисциплину: событие получено, поток/процесс остановлен, debugger анализирует, потом обязан продолжить target с правильным continuation status. Поэтому dynamic collector в проекте не должен быть хаотичным hooker-ом. Он должен быть маленьким debugger-ом с понятным event loop, bounded events, timeout, breakpoint rearm и отчетом о полноте instrumentation.

Так возникает главный принцип проекта: если Microsoft contract говорит, что структура или API имеет определенный смысл, анализатор должен использовать этот смысл аккуратно, но не обязан верить конкретному файлу. Контракт описывает норму. Файл может норму нарушать. Runtime может

добавлять новые факты. Отчет должен разделять эти слои. Именно это разделение - official contract, untrusted file, decoded model, heuristic assessment, runtime evidence, final report - и является архитектурным позвоночником EH/Unwind Analyzer.

Что означает название EH/Unwind Analyzer

Название проекта можно читать почти как техническое задание. `EH` - exception handling, обработка исключений. Но в Windows это выражение не надо понимать только как try/catch из C++ или `__try/__except` из MSVC. Exception handling здесь шире: hardware fault, software exception через `RaiseException`, breakpoint exception, single-step, guard page, vectored exception handler, frame-based handler, top-level unhandled filter, а также language-specific layer, который компилятор и runtime строят поверх общей инфраструктуры. Проект не пытается полностью моделировать все языковые runtime-ы, но он смотрит на тот фундамент, без которого эти runtime-ы не могут нормально жить на x64.

`Unwind` - размотка стека или восстановление контекста вызывающих кадров. Это слово легко недооценить. Новичок думает: стек - это просто return addresses, значит stack walking можно делать по памяти. На x64 Windows это опасное упрощение. Оптимизирующий compiler меняет RSP, сохраняет nonvolatile-регистры в разных местах, может использовать frame pointer или не использовать, может иметь сложный prolog/epilog, а exception может случиться в середине функции. Чтобы восстановить caller context, системе нужны metadata. Поэтому unwind - не второстепенная часть exception handling, а способ сделать exception handling возможным в оптимизированном 64-битном коде.

`Analyzer` - анализатор, а не executor, не deobfuscator и не cracker. Он берет входные факты, нормализует их, проверяет структурные условия, строит выводы ограниченной силы и пишет отчет. Это слово накладывает дисциплину. Анализатор не должен делать вид, что знает больше, чем видел. Если `UNWIND_INFO` не читается, он пишет parse failure. Если dynamic session timed out, он пишет timed_out. Если static finding не подтвержден runtime-событием, он не исчезает, но и не превращается в confirmed behavior. Если runtime handler hit появился без сильной статической аномалии, это runtime evidence, но не автоматическая вина.

Название также показывает порядок мышления. Сначала есть Windows exception/unwind contract. Потом есть конкретный PE image, который этот contract должен реализовать. Потом есть анализатор, который проверяет, насколько image соответствует contract-у и как эти сведения проявляются в execution. Это не проект про один красивый алгоритм. Это проект про аккуратное движение между слоями: bytes, PE image, section mapping, function table, unwind record, handler target, heuristic finding, runtime event, report.

Почему именно exception/unwind metadata достойна отдельного инструмента

Может показаться, что exception/unwind metadata - слишком узкая тема. В обычном обучении reverse engineering чаще говорят про imports, strings, control-flow graph, symbols, packers, encryption, API calls. `.pdata` и `.xdata` часто остаются где-то сбоку: вроде служебные таблицы для stack trace. Но в Windows x64 эти таблицы стоят на границе нескольких миров сразу. Они принадлежат PE format, ABI, compiler code generation, exception dispatch, crash analysis, debugger stack walking, C++ runtime и runtime-generated code. Поэтому маленькая ошибка в их понимании тянет за собой неправильное понимание всей программы.

Для реверсера эти metadata важны по трем причинам. Первая причина - восстановление функций и кадров. Даже если дизассемблер плохо нашел границы функций, `RUNTIME_FUNCTION` entries дают диапазоны кода, которые compiler/linker считает unwindable regions. Это не всегда один к одному совпадает с исходными функциями, но это сильное structural evidence. Вторая причина - exception-based control flow. Handler RVA, EHANDLER/UHANDLER flags и handler data могут указывать на участки кода, которые обычный линейный CFG не показывает как прямой call/jmp. Третья причина - аномалии metadata. Поврежденная, перемещенная, переиспользованная или writable unwind metadata может быть просто ошибкой, особенностью compiler-а или частью обфускации. В любом случае это требует внимания.

Отдельный инструмент нужен потому, что general-purpose tools часто показывают куски этой картины, но не связывают их в одну explanation chain. Дизассемблер может показать `.pdata`. Debugger может показать exception events. PE viewer может показать sections. Но реверсеру нужно понять взаимосвязь: этот RUNTIME_FUNCTION range находится в executable section, его UnwindInfoAddress указывает в writable section, flags говорят EHANDLER, handler target находится в non-executable или unknown section, parse failed из-за truncated operand, а dynamic run увидел first-chance exception и handler hit. Это уже не просто отдельные наблюдения. Это case, который надо объяснить.

EH/Unwind Analyzer как раз занимает место между PE viewer, debugger и reverse engineering database. Он не заменяет Ghidra, WinDbg или x64dbg. Он подготавливает для них слой фактов: какие ranges существуют, где metadata, что декодировалось, что сломалось, какие признаки требуют manual review, какие runtime events были замечены. Поэтому наличие Ghidra CSV importer в проекте логично: результат анализа должен попадать туда, где человек реально продолжает исследование. Инструмент не заканчивает reverse engineering. Он делает стартовую карту честнее и богаче.

Главная проблема: файл на диске не равен процессу в памяти

Одна из самых важных идей проекта - разделение static и dynamic мира. Static analysis работает с input bytes. Это файл на диске. Он может быть PE image, может быть поврежденным PE, может иметь странные section ranges, может иметь Exception Directory, который нельзя прочитать. Static mode не исполняет код. Это делает его безопаснее и переносимее. Он может работать cross-platform, потому что для чтения PE headers, `.pdata` и `.xdata` не нужна Windows runtime. Но у static mode есть фундаментальная граница: он видит только то, что есть в файле и что можно вывести из файла.

Running process - другой мир. Loader отображает image в memory, применяет relocations, imports, memory protections, ASLR, загружает DLL, создает threads. Код может распаковываться, генерироваться, менять memory protections, регистрировать vectored handlers, устанавливать unhandled filter, добавлять dynamic function tables. Поэтому runtime exception route может включать факты, которых в исходном файле не было. Это не делает static analysis бесполезным. Наоборот: static layer нужен как baseline, как declared model. Но baseline надо сравнивать с live evidence, а не выдавать за всю жизнь процесса.

В проекте это разделение видно в data flow. Сначала input bytes идут в PE parser and structural validation. Затем Exception Directory и RUNTIME_FUNCTION reader строят таблицу ranges. Затем UNWIND_INFO decoder пытается получить per-function summary. Затем heuristic assessment оценивает structural anomalies. Только после этого optional Windows dynamic collector может запускать target и собирать runtime evidence. Порядок важен. Dynamic collector не стартует с нуля. Он использует static results, например handler RVAs из parsed `UNWIND_INFO`, чтобы поставить breakpoints на static unwind handlers. И наоборот, static findings получают больше смысла, когда dynamic run показывает, что соответствующий route действительно проявился.

Нельзя смешивать эти слои. Если static analysis нашел handler target in writable section, это static finding. Если dynamic validation увидела handler hit, это runtime event. Если оба факта относятся к одной смысловой зоне, report может сказать static/dynamic correlation. Но correlation в проекте conservative and coarse. Она не является per-function proof graph. Она не доказывает полный control-flow path. Она говорит: независимые слои увидели совместимые признаки, поэтому case strength выше, чем у одиночного finding-а. Это честная инженерная формулировка.

Именно поэтому архитектура не пытается сделать один монолитный `analyze_everything()`. Монолит почти неизбежно начал бы путать факт чтения файла, эвристическую интерпретацию и runtime-наблюдение. Проект разделяет модули, структуры и report fields так, чтобы читатель мог увидеть

provenance - происхождение каждого знания. Это знание пришло из PE headers. Это из unwind decoder. Это из heuristic/evidence engine. Это из Debug API session. Это из final assessment. Без provenance отчет выглядит увереннее, но становится хуже.

Trust boundary: почему безопасное чтение важнее красивой эвристики

Первый trust boundary проекта проходит в самом начале: static mode treats input as untrusted bytes. Это не формальная фраза из SECURITY-документа, а техническая основа. Любой PE-файл может быть специально сделан так, чтобы сломать parser: offsets за пределами файла, переполнения при сложении, секции с raw overlap, виртуальные ranges за SizeOfImage, malformed Optional Header, DataDirectory size не кратный размеру структуры, chain loops, truncated unwind codes. Если анализатор падает на таком файле, его эвристики уже не имеют значения. Он не анализатор, а еще одна поверхность атаки.

Поэтому `util.c` - не второстепенный helper. `Buf` хранит pointer на bytes и size. `buf_read` проверяет, что offset не выходит за размер, что length помещается в оставшийся диапазон, и только потом копирует bytes. `buf_read_u16`, `buf_read_u32`, `buf_read_u64` читают little-endian значения через безопасное чтение. Это скучно, но именно так строится надежный parser. В C нельзя надеяться на исключения, bounds checking языка или автоматическое предотвращение overflow. Значит, проект сам должен держать дисциплину.

Эта дисциплина продолжается в `pe.c`. Parser не просто проверяет magic `MZ` и `PE`. Он проверяет machine type AMD64, OptionalHeader PE32+, size_of_image, size_of_headers, section/file alignment, NumberOfRvaAndSizes, Exception Directory bounds, section table bounds, raw/virtual overlaps. Это не бюрократия. Если section mapping неверен, все последующее чтение `.pdata` и `.xdata` будет построено на песке. Ошибка в PE layer не остается локальной. Она может привести к ложным unwind findings, неверным handler targets и бессмысленному Ghidra CSV.

Проектная архитектура поэтому не начинает с “найти подозрительные признаки”. Она начинает с “можно ли вообще корректно построить image model”. Это важный поворот мышления. В reverse engineering хочется сразу искать интересное: handler dominance, weird xdata, runtime function table calls. Но хороший инструмент сначала спрашивает: валиден ли контейнер, в котором эти признаки лежат? Если контейнер поврежден, сам факт повреждения становится finding-ом table_integrity, а все дальнейшие выводы должны быть осторожнее.

Здесь появляется понятие `unknown`. Если RVA не мапится, правильный ответ не “там точно нет структуры” и не “это точно вредно”. Правильный ответ:

static analyzer cannot resolve this RVA to readable file bytes under current PE section mapping. Это длиннее, но точнее. Инструмент должен уметь сказать unknown так же уверенно, как и найдено. Это качество делает отчет полезным для реверсера: он видит не только факты, но и границы знания.

От PE parser к unwind decoder: почему pipeline нельзя перескочить

Когда PE model построена, следующий слой - Exception Directory и `RUNTIME_FUNCTION` table. Здесь важно помнить половинную природу `RUNTIME_FUNCTION`. С одной стороны, это очень конкретная структура: BeginAddress, EndAddress, UnwindInfoAddress. С другой стороны, она не содержит имени функции, исходного файла, прототипа, calling convention comment или guarantee that this range is meaningful source-level function. Это entry in function table, диапазон unwind-модели. Проект поэтому хранит его в простой структуре `RuntimeFunction` с тремя RVA, не притворяясь, что знает больше.

`unwind_read_runtime_functions_ex` обязан прочитать Exception Directory как таблицу таких entries. Если directory structurally unreadable, проект не должен молча делать вид, что функций ноль. В `main.c` видно, что malformed read переводится в partial structural report: `img.exception_directory_oob = true`, пользователь получает ошибку, а report сохраняет совместимое поле/finding. Это хороший пример инженерной честности. Сломанная Exception Directory - это не обычный empty result. Это самостоятельное состояние анализа.

Дальше каждый `RuntimeFunction` ведет к `UNWIND_INFO`. Именно здесь проект начинает превращать raw metadata в semantic summary. `UnwindSummary` хранит version, flags, prolog_size, count_codes, frame_reg, frame_off, has_handler, handler_rva, has_chain, chain entry, fixed_stack_bytes, pushed_nonvol_count, saved_nonvol_mask, saved_xmm_mask, anomalies and parse_error. Этот набор полей не случайный. Он отвечает на вопрос: что эта функция заявляет о своем stack frame и exception route? Если summary валиден, реверсер может понять shape of frame. Если невалиден, parse_error объясняет, на каком месте декодирование оборвалось.

Отдельно важно, что parser поддерживает options: follow_chain, max_chain_depth, strict_abi. Это показывает разницу между compatibility mode и formal ABI diagnostics. Обычный анализ может быть мягче, чтобы не ломаться на реальных compiler/assembler особенностях. Strict mode может добавить warnings, связанные с формальными правилами Microsoft x64 unwind ABI. Но strict-only warning не должен автоматически становиться suspicion score. Иначе инструмент начал бы путать “формально необычно” с “подозрительно по смыслу”.

Почему нельзя перескочить от PE parser сразу к heuristics? Потому что эвристика должна работать не с raw bytes, а с normalized facts. В проектных документах `heuristics.c` получает `PeImage`, `RuntimeFunction`, `UnwindSummary`, `us_ok` and count. Это правильная архитектура. Heuristic/evidence engine не должен заново парсить file offsets и не должен дублировать PE reader. Он принимает уже нормализованную модель и оценивает ее. Чем чище граница между parsing и assessment, тем проще объяснять report и тестировать каждую часть отдельно.

Heuristic/evidence engine: зачем нужен слой оценки, если parser уже все прочитал

Parser отвечает на вопрос “что находится в файле и можно ли это структурно прочитать”. Но реверсеру часто нужен следующий вопрос: “что в этой картине требует внимания”. Допустим, все `RUNTIME_FUNCTION` entries читаются, все `UNWIND_INFO` records формально валидны, но один и тот же `UnwindInfoRVA` используется необычно большим количеством ranges. Или handler targets концентрируются в одном странном месте. Или metadata лежит в executable/writable section. Parser не должен сам объявлять это плохим. Parser должен дать факты. Для оценки нужен separate heuristic/evidence engine.

Слово heuristic означает practical inference rule - практическое правило вывода. Это не mathematical proof и не статистическая модель. В EH/Unwind Analyzer эвристики сгруппированы в evidence families: table_integrity, unwind_integrity, metadata_placement, unwind_reuse, handler_dominance. Такая группировка важнее, чем кажется. Она не дает score расти просто от множества похожих мелких признаков без разнообразия. Если все признаки из одной family, confidence должен быть осторожнее. Если несколько независимых families сходятся, case становится сильнее.

Проект хранит finding как structured object: code, family, severity, message, location, representative_rva, affected_count. Это лучше, чем печатать одну строку в console. Structured finding можно сериализовать в JSON, показать в HTML, проверить schema tests, импортировать в downstream tooling и использовать в assessment. А главное - structured finding заставляет автора правила отделять code от message, family от severity, location от affected count. Это дисциплина, которая потом защищает от хаотичных verdict-ов.

`score`, `suspicion`, `confidence`, `verdict`, `evidence_level` в проекте намеренно разделены. Score - bounded engineering weight sum. Suspicion - derived score/100, удобная нормализация, но не probability. Confidence - rule-based оценка полноты и согласованности evidence, а не статистическая вероятность. Verdict - человекочитаемая категория вроде low/moderate/high suspicion. Evidence level - сила статического evidence по правилам. Если слить все это в одно поле, report станет проще, но хуже. Разные вопросы получили бы один ответ, и реверсер перестал бы понимать, что именно инструмент

утверждает.

Weakening rules - еще один признак зрелости. Хороший анализатор должен уметь не только усиливать case, но и ослаблять его. Single signal only, small sample size, placement only signal, single family only - такие правила говорят: да, finding есть, но основание узкое. Это особенно важно в теме unwind metadata, потому что benign compiler behavior, link-time optimization, hand-written assembly, runtime libraries and generated code могут создавать unusual patterns без вредоносного смысла. Инструмент должен помогать триажу, а не пугать каждым нестандартным PE.

Dynamic collector: почему он отдельный и почему он опаснее static mode

Dynamic validation в проекте появляется не потому, что static analysis плох. Она появляется потому, что Windows process может делать то, чего нет в файле на диске. Но dynamic mode crossing execution boundary. Это значит: analyzer запускает target как child process under debugger. Такой режим требует другого отношения к безопасности, другой модели ошибок и другого report semantics. В static mode файл - недоверенные bytes. В dynamic mode файл становится исполняемым процессом. Для untrusted samples это означает isolated VM, ограничения, timeout, controlled environment. Поэтому dynamic collector не может быть просто флажком “проверить лучше”. Это отдельный trust boundary.

В `dynamic.h` defensive intent сформулирован прямо: collect runtime evidence of exception-handling redirection by observing handler hits, VEH/VCH/UEF registration and hits, dynamic function table registration and first-chance versus second-chance exception flow. Там же явно сказано, что модуль не пытается break protections or recover hidden data. Это важная строка архитектурного контракта. Она защищает проект от размывания цели. Debug API можно использовать для самых разных задач, но здесь он используется как источник runtime evidence for analysis.

Технически dynamic collector строится как limited debugger. Он запускает target, получает debug events, ведет module map, ставит software breakpoints на интересные API и handler addresses, восстанавливает оригинальный байт, двигает RIP назад после INT3, включает trap flag for single step, re-arms breakpoint after instruction execution, записывает structured events и ограничивает trace. Это звучит сложно, но смысл простой: если мы хотим увидеть handler hit, надо аккуратно остановиться на адресе handler, не разрушить target больше необходимого и продолжить выполнение.

Dynamic report поэтому содержит не только “нашли/не нашли”. Он содержит enabled, ran, ok, timed_out, target_exited, target_exit_code, timeout_ms, trace_steps, instrumentation_complete, modules_observed_peak, module_refresh_failures, breakpoint_set_attempts, breakpoint_set_failures,

exceptions_seen, first_chance_seen, second_chance_seen, handler hits, VEH/VCH/UEF counters, RtlAddFunctionTable calls, event timeline, dropped_events and trace. Это много полей, но каждое защищает от неправильного вывода. Например, отсутствие handler hit при timed_out session не означает, что handler никогда не сработает. Dropped events означают, что timeline неполон. Breakpoint failures означают, что instrumentation coverage ограничена.

Dynamic collector не должен превращаться в oracle. Он видит только конкретный запуск с конкретными arguments, environment, privileges, time budget and path coverage. Если target требует input, network, UI action, elevated token or delayed behavior, dynamic session может ничего не увидеть. Поэтому assessment должен говорить not observed или incomplete, а не absent. Это не слабость. Это зрелая модель runtime evidence.

Reports как часть анализа, а не упаковка результата

В простом проекте report часто воспринимается как финальная печать результата. В EH/Unwind Analyzer report - это часть анализа. Он должен сохранить provenance, schema, metadata, static facts, decoded summaries, findings, dynamic events and assessment так, чтобы человек и машина могли проверить вывод. Если report плохо отделяет поля, теряет parse_error, смешивает static и dynamic evidence или пишет красивый HTML без machine-readable JSON, инструмент становится хуже даже при хорошем parser-е.

`ReportMetadata` показывает, что project думает о reproducibility. Tool version, schema version, generated_at_utc, input_sha256, input_size, command_line, build compiler, platform, build mode, strict_abi, follow_chain, max_chain_depth - это не косметика. Без SHA-256 непонятно, к какому input относится report. Без schema version непонятно, как downstream validator должен читать JSON. Без command_line нельзя понять, был ли включен strict_abi или dynamic. Без redaction target arguments could leak secrets. Поэтому report metadata является частью security and audit design.

JSON report - это contract. Он должен быть строгим enough for tests and downstream tools. HTML report - human dashboard, но он тоже должен быть self-contained, escape arbitrary metadata bytes and avoid remote scripts/styles/fonts. CSV - flat table for quick inspection. Ghidra CSV - bridge into reverse engineering workspace. Console output - быстрый feedback для CLI. У каждого формата своя аудитория, но они должны говорить об одной модели. Если JSON и HTML расходятся по смыслу, это bug, а не styling difference.

Assessment - самый опасный слой report-а, потому что читатель склонен воспринимать final verdict как ответ. Проект поэтому обязан объяснять final verdict, case strength, static/dynamic correlation, static evidence level, runtime evidence level and manual validation steps. Manual validation steps особенно важны: автоматический анализ не заменяет реверсера. Он должен сказать,

куда смотреть дальше: проверить section mapping, открыть representative RVA in Ghidra, сравнить handler target, повторить dynamic run with input, inspect trace, validate dynamic function table behavior. Хороший report не закрывает расследование. Он задает следующий честный шаг.

Почему Ghidra integration логична именно здесь

Ghidra integration в проекте - не бонусная галочка. Reverse engineering редко заканчивается чтением JSON. Реверсер открывает binary в database, смотрит functions, comments, bookmarks, xrefs, decompiler output. Если EH/Unwind Analyzer нашел function ranges, decoded unwind summary, parse failures and suspicious findings, эти факты должны попасть туда, где человек смотрит код. Поэтому `ghidra/import_unwind_csv.py` является естественным продолжением reporting layer.

При этом importer тоже находится по другую сторону trust boundary. CSV, даже созданный нашим tool-ом, должен считаться untrusted input для Ghidra script. Проектные документы прямо говорят, что Ghidra CSV treated as untrusted input by importer. Это правильная привычка. Инструмент не должен становиться уязвимым только потому, что input вроде бы generated internally. File paths, comments, names, RVA ranges, parse errors - все это данные, которые надо валидировать перед вставкой в database.

Смысл Ghidra layer не в том, чтобы автоматически переименовать все функции и объявить анализ завершенным. Он должен консервативно добавить context: ranges, comments, warnings, bookmarks. Если функция уже имеет user/imported name, importer должен не разрушать работу человека. Если ranges overlap, он должен пропустить опасный case. Если parse failure есть, лучше создать warning bookmark, чем молча потерять проблему. Такой подход соответствует общей философии проекта: помочь реверсеру, но не изображать всезнание.

Validation: почему проекту нужны тесты, corpus and schema checks

Для такого инструмента нельзя сказать “я посмотрел глазами пару файлов, вроде работает”. Parser недоверенных bytes требует unit tests and mutation fuzzing. Heuristic engine требует sample corpus and threshold sweep. Report contract требует JSON Schema and semantic validators. Dynamic collector требует Windows smoke scenarios and dynamic corpus. Иначе проект будет красивым текстом, а не инженерным артефактом.

Проектные документы показывают серьезную validation story: unit tests, regression samples, mutation fuzzing, real benign corpus, labeled static corpus, dynamic smoke for VEH/VCH/UEF/runtime tables, full dynamic corpus, schema/semantic-valid reports, MinGW and MSVC builds. Даже если конкретные

numbers относятся к текущему release evidence set, архитектурный смысл шире: каждый слой проверяется своим способом. PE parser проверяется malformed inputs. Unwind decoder проверяется edge cases. Heuristics проверяются findings and mitigations. Dynamic collector проверяется controlled executables. Reports проверяются schema and cross-field invariants.

Здесь важно не приписывать тестам больше, чем они дают. 106 benign files with zero heuristic scores - хороший calibration evidence, но это не доказывает отсутствие false positives в мире. 124 dynamic reports schema-valid - это сильный engineering signal, но не доказательство, что dynamic collector увидит все возможные runtime tricks. Threshold sweep scaffold - путь к обоснованию thresholds, но не ROC-quality evaluation. Проект честно говорит об этом в threshold_calibration. Это зрелая позиция: validation evidence strengthens trust, but does not become universal proof.

Для книги это особенно важно. Мы не пишем рекламный буклет. Мы показываем, как инженерный инструмент должен ограничивать свои утверждения. Если validation покрывает malformed parser paths, это надо сказать. Если validation не сравнивает unwind operations with disassembled machine instructions, это тоже надо сказать. Граница знания - не слабое место текста, а признак профессиональности.

Чего проект специально не делает

Чтобы понять, зачем проект существует, полезно так же ясно понять, чего он не делает. Он не дизассемблирует машинный код для полного сравнения prolog instructions with unwind codes. Это значит, что он может сказать: metadata structurally valid, но не доказать, что она exactly matches actual code bytes. Такой анализ возможен как future extension, но он требует disassembler layer, instruction semantics, prolog/epilog recognition and compiler variance handling. Добавить его не значит просто включить еще одну проверку.

Он не является packer unpacker. Если payload появляется только после runtime unpacking, static mode не извлечет его. Dynamic mode может увидеть runtime events, но не обязан восстановить original payload. Это принципиальное ограничение области. Если инструмент начнет обещать unpacking, архитектура станет другой: memory dumping, decryption triggers, anti-debug handling, code discovery, snapshot diffing. Это уже другой проект и другие риски.

Он не является anti-debug bypass engine. Dynamic collector использует Debug API openly. Некоторые targets могут менять поведение под debugger, проверять debug flags, реагировать на INT3, ломаться на timing, не проходить сценарий из-за отсутствия input. Проект должен учитывать это в report, а не пытаться скрытно обходить каждую проверку. Defensive tool может сказать: dynamic evidence incomplete because target changed or did not run path. Это лучше, чем превращать анализатор в сомнительный bypass framework.

Он не является statistical malware detector. Score - triage aid. Confidence - engineering confidence. Verdict - heuristic assessment. Case strength - combination of static and runtime evidence. Эти слова надо повторять, потому что они защищают от неправильного использования. Если пользователь хочет malware classification, ему нужен другой pipeline: labels, dataset design, feature extraction, false-positive/false-negative analysis, calibration, model validation. EH/Unwind Analyzer не делает вид, что имеет такую статистическую основу.

Он не является replacement for human reverse engineering. Он строит карту. Но карту надо читать. Если report говорит handler_writable_section, человек должен открыть section, посмотреть permissions, examine handler RVA, inspect code, verify whether section is packed stub, runtime-generated area, compiler artifact or suspicious redirection. Автоматизация ускоряет путь, но не отменяет judgment.

Как 11-я глава будет разбирать проект дальше

Этот раздел задает смысл. Дальше глава 11 должна идти по pipeline, но не сухо. В 11.2 мы разберем `main.c` как orchestrator: как command line превращается в последовательность analysis stages, почему ownership and cleanup важны, как exit codes связаны с состоянием анализа, почему dynamic/report generation подключаются только после static model. Это будет не “main вызывает функции”, а история жизненного цикла одного analysis run.

Затем 11.3 должен разобрать `util.c` как foundation of safe parsing: Buf, bounds-safe reads, little-endian decoding, StrBuf, overflow discipline, atomic writes, SHA-256. Это не мелочи. Именно там проект решает, будет ли он устойчив к malformed input. Без этого весь высокий уровень - эвристики, HTML, Ghidra - висит в воздухе.

11.4 будет про `pe.c`: PE32+ AMD64 headers, section table, RVA mapping, structural counters, Exception Directory, alignments, overlaps, oob flags. Здесь нужно будет показать, как official PE Format превращается в конкретный `PeImage` and section model. 11.5 - про `unwind.c`: runtime-function table, UNWIND_INFO, V1/V2, operations, handler flags, chain following, strict ABI diagnostics, parse_error. 11.6 - про heuristic/evidence engine, уже глубже, чем в report chapter: как findings рождаются из counters and ratios.

11.7 будет про Windows dynamic collector как инженерный debugger subsystem: module map, API breakpoint resolution, INT3, original byte, RIP rewind, trap flag, rearm, event limits, timeout, CreateProcess flags, dynamic API arguments, handler hits, trace. 11.8 - про reporting layer as contract: JSON schema, HTML CSP, CSV, Ghidra CSV, assessment. 11.9 - Ghidra importer. 11.10 - tests, corpus, fuzzing, release gates. 11.11 - финальная связка: как каждая глава книги отражается в конкретном file/function/field проекта.

Такой порядок важен, потому что он повторяет путь факта. Факт сначала существует как byte. Потом как PE field. Потом как RVA. Потом как function table entry. Потом как decoded unwind summary. Потом как finding. Потом как runtime event. Потом как report field. Потом как bookmark/comment in Ghidra. И только после этого он становится человеческим выводом. EH/Unwind Analyzer существует для того, чтобы этот путь был явным, проверяемым и честным.

Итоговая мысль раздела

Если сжать смысл проекта до одной фразы, получится так: EH/Unwind Analyzer превращает официальный Windows x64 exception/unwind contract в проверяемую карту конкретного PE-файла и, при необходимости, конкретного runtime-запуска. Он читает то, что файл заявляет о своих функциях и кадрах. Он декодирует, как эти кадры должны unwound. Он оценивает, где структура нарушена или выглядит необычно. Он собирает живые debug events, если пользователь сознательно пересек execution boundary. Он пишет reports, которые разделяют observation, interpretation and consequence.

Главное достоинство такой архитектуры не в том, что она “ловит подозрительное”. Главное достоинство - она не врет о силе своих выводов. Она умеет сказать valid, invalid, unusual, suspicious, not observed, incomplete, requires manual review. В reverse engineering это важнее красивой уверенности. Потому что плохой инструмент дает громкий verdict. Хороший инструмент строит цепочку evidence, показывает, где она сильная, где слабая, и оставляет реверсеру понятный следующий шаг.

Именно поэтому финальная проектная глава нужна. До этого мы строили теоретическую модель Windows x64. Теперь мы должны показать, как эта модель превращается в C-код, структуры, CLI options, JSON fields, dynamic events, tests and release artifacts. Не как отдельный дипломный проект в стороне от книги, а как практическое воплощение всего пути: function -> call -> return address -> RSP -> stack frame -> nonvolatile registers -> .pdata -> RUNTIME_FUNCTION -> .xdata -> UNWIND_INFO -> RtlVirtualUnwind -> handler route -> static findings -> runtime evidence -> assessment -> report -> manual reverse engineering.

Официальная документация задает контракт, но не делает анализ за нас

Официальная документация Microsoft нужна проекту как источник нормы. Она говорит, что PE/COFF image имеет headers, sections and data directories. Она говорит, что на x64 `RUNTIME_FUNCTION` entries используют image-relative addresses, что function table entries sorted and placed in `.pdata` for PE32+ image, что unwind data описывает prolog actions and handlers, что dynamically generated functions require runtime function table registration. Но документация не обязана объяснять каждую аномалию, которую реверсер увидит в чужом binary. Она описывает, как система должна работать, а не как вести расследование, когда файл нарушает ожидания.

Это различие очень важно для архитектуры. Если проект просто переписал бы Microsoft structures into C structs и напечатал их, он был бы dumper-ом. Полезный dumper, но не analyzer. EH/Unwind Analyzer делает больше: он проверяет, как заявленная структура соотносится с границами файла, секциями, permissions, количеством entries, reuse patterns, handler placement and runtime evidence. То есть он берет official contract as baseline and asks: what does this specific sample do with that contract?

Официальная документация также не заменяет threat model. Например, `RtlAddFunctionTable` является нормальным механизмом для dynamically generated code. JIT compiler может использовать его совершенно законно. Но если static analysis видит почти пустую `.pdata`, a dynamic validation видит runtime function-table calls and exceptions in generated region, это значимый факт для reverse engineering. Не потому, что API suspicious by itself, а потому, что он объясняет shift from file-backed code to runtime code. Проект должен уметь сказать именно это, без обвинительного тона.

То же касается Debug API. `WaitForDebugEvent` and `ContinueDebugEvent` - нормальные API для debugger-a. В проекте они используются для сбора evidence, но сам факт запуска под debugger меняет среду выполнения. Официальная документация говорит, как debugger loop works. Архитектура проекта должна добавить: какие события мы сохраняем, какие теряем из-за bounded limits, что значит timeout, какие child processes не покрываются при DEBUG_ONLY_THIS_PROCESS, почему target args redacted, почему untrusted target нужно запускать in isolated VM.

Поэтому правильная формула такая: official docs define the machine contract; project architecture defines the investigative contract. Первый отвечает на вопрос, как Windows ожидает видеть metadata. Второй отвечает на вопрос, как анализатор будет обращаться с недоверенным файлом, как будет ограничивать выводы, как будет отделять parsing от heuristics, dynamic events от assessment, and report display from machine-readable contract.

Один запуск анализатора как история движения факта

Чтобы не воспринимать architecture как набор прямоугольников, полезно представить один реальный запуск. Пользователь вводит команду вроде `eh_unwind_analyzer target.exe --heuristics --json report.json --html report.html`. В этот момент в проекте еще нет ни PE, ни functions, ни findings. Есть только argv, file path and options. `main.c` должен сначала понять, что пользователь попросил: dump, ops, heuristics, csv, ghidra csv, json, html, strict_abi, follow_chain, max_chain_depth, dynamic, timeout, trace_steps, target arguments after separator. Это не UI-мелочь. Options определяют, какие знания вообще будут собраны.

После parse args начинается ownership chain. `buf_init_from_file` читает file bytes and returns owned memory. Дальше `pe_parse` строит `PeImage` and owns sections. Потом `unwind_read_runtime_functions_ex` возвращает heap array of `RuntimeFunction`. Потом создаются arrays `UnwindSummary` and `us_ok`, по одному элементу на runtime-function entry. Затем loop вызывает `unwind_parse` for each function. На этом этапе проект уже имеет normalized static model: PE image facts, function table entries, per-entry unwind summary or failure flag.

Только теперь имеет смысл запускать heuristics. `heuristics_analyze` получает не raw file bytes, а `PeImage`, `RuntimeFunction`, `UnwindSummary`, `us_ok` and count. Это значит, что engine оценивает модель, а не делает повторный parser. Если включен dynamic mode, `dynamic_collect_windows` получает static model too: `img`, `rf`, `us`, `us_ok`, rf_count. Это позволяет dynamic collector использовать static handler RVAs as breakpoint targets. Dynamic layer не висит отдельно. Он привязан к static decode, но не смешан с ним.

В конце `main.c` строит metadata for reports. Здесь важны детали: input path переводится в UTF-8 on Windows, command line is rebuilt with target args after `--` redacted, timestamp generated, SHA-256 computed over input bytes, schema version and build info inserted. И только после этого JSON/HTML writers получают полный набор objects. Report writer не должен угадывать, какой build создал результат, и не должен заново считать модель. Он получает готовые facts and serializes them under schema contract.

Эта история движения факта показывает, почему проектная архитектура не случайна. Если поменять порядок и писать report до metadata, reproducibility ломается. Если запускать heuristics до unwind_parse, findings станут поверхностными. Если dynamic collector не получает static model, он не сможет связать handler hits с `UNWIND_INFO`. Если parser не сохраняет parse_error, report потеряет причину failure. Каждая функция в pipeline поддерживает следующую.

Почему static analyzer сознательно не зависит от Windows runtime

В README проекта сказано: static analysis is cross-platform, dynamic validation is Windows x64 only. Это важная архитектурная граница. Статический parser PE32+ AMD64 не должен требовать, чтобы программа запускалась на Windows. Он читает documented file format. Поэтому `pe.h` прямо говорит: minimal PE32+/x64 parser, no Windows headers. Проект сам объявляет constants вроде PE_DOS_MAGIC, PE_NT_MAGIC, IMAGE_FILE_MACHINE_AMD64, IMAGE_NT_OPTIONAL_HDR64_MAGIC, IMAGE_DIRECTORY_ENTRY_EXCEPTION. Это делает static layer переносимым and testable outside Windows.

Такой выбор имеет цену. Если не подключать Windows headers, нельзя просто использовать готовые Windows structs and trust their layout. Нужно самостоятельно держать offsets PE32+ Optional Header fields, DataDirectory position, section header size, RVA mapping rules. Но эта цена оправдана: static parser становится независимым от target OS, легче fuzzing, sanitizer, CI, Linux-based tests and malformed input validation. Для defensive parser-а это лучше, чем зависеть от platform-specific loader behavior.

Dynamic layer наоборот не может быть cross-platform в настоящем смысле. Он использует Windows Debug API, Toolhelp module snapshots, remote process memory writes, thread context, x64 calling convention registers, breakpoint handling. Non-Windows build должен честно сказать unsupported, а не имитировать dynamic validation. Это опять возвращает нас к честности report-а: если dynamic not available, assessment не может иметь runtime evidence. Лучше no runtime evidence than fake runtime evidence.

Так проект разделяет portability and fidelity. Static mode переносим, потому что анализирует file contract. Dynamic mode Windows-only, потому что наблюдает Windows runtime behavior. Попытка сделать один универсальный слой привела бы либо к слабому static parser, зависящему от Windows, либо к fake dynamic mode на других OS. Разделение делает архитектуру проще и правдивее.

Несколько типовых расследований, ради которых нужен именно такой инструмент

Первый типовой случай - обычный benign compiler-generated PE. У него Exception Directory указывает на readable `.pdata`, entries sorted, ranges valid, `UNWIND_INFO` records decode, metadata lies in non-executable read-only area, handlers either absent or placed plausibly, reuse ratios normal. В этом случае лучший результат анализатора - не громкий отчет, а спокойный ноль: score 0, no findings, valid JSON, useful Ghidra CSV. Это важный сценарий. Инструмент, который всегда что-то кричит, бесполезен для триажа.

Второй случай - structurally damaged PE. Например, section raw pointer выходит за file size, Exception Directory points outside mapped range, `RUNTIME_FUNCTION` table size malformed, UnwindInfoAddress cannot be resolved. Здесь анализатор должен не падать и не молчать. Он должен зафиксировать table_integrity/unwind_integrity findings, сохранить counters, possibly partial report, and require manual review. Даже если файл не исполняется, structural facts valuable: они говорят, почему обычный unwind analysis невозможен.

Третий случай - unusual but possibly benign metadata. Например, high reuse of unwind info in compiler-generated thunks, many tiny ranges, uncommon section naming, hand-written assembly with compact unwind info. Здесь нужна confidence discipline and weakening rules. Один сигнал без corroboration не должен превращаться в high_suspicion. Читатель должен увидеть affected_count, representative_rva, family, severity and reason for weakening. Это особенно важно для дипломного инструмента: он должен показывать зрелое понимание false positives.

Четвертый случай - packed or protected sample. Static layer может увидеть stub, strange placement, handler concentration, executable/writable metadata, broken chains, handler targets outside normal sections. Dynamic layer может увидеть first-chance exceptions, VEH registration, handler hits, runtime function-table calls. Интересно не каждое событие отдельно, а их согласованность. Если static handler-related finding and runtime handler hit meet, assessment can strengthen case. But still not as malware proof - only as correlated evidence of exception/unwind-driven behavior.

Пятый случай - JIT or runtime-generated code. Static layer may look incomplete because generated code is absent from file. Dynamic layer may observe `RtlAddFunctionTable` or callback function-table registration. In such case runtime evidence can explain why static PE metadata is not the whole story. Это пример, где dynamic evidence не обвиняет, а уточняет модель. Хороший инструмент должен одинаково хорошо поддерживать suspicious investigation and benign explanation.

Этическая и инженерная граница dynamic mode

Так как project связан с debugging and runtime execution, нужно явно проговорить ethics and safety. Dynamic collector запускает target. Если target untrusted, это риск. Поэтому SECURITY guidance and README говорят использовать isolated VM. Архитектура не должна скрывать этот факт за красивым названием validation. Validation здесь означает сбор runtime evidence, а не безопасную магию. Пользователь должен понимать: executable will execute.

Инженерная граница dynamic mode также связана с вмешательством. Software breakpoint writes 0xCC into target memory. If page is not writable,

collector may temporarily change protection, write byte, restore protection and flush instruction cache. Это controlled debugger intervention. Для нормального debugging это обычная техника, но target can observe it. Цели, которые реагируют на presence of debugger, могут менять поведение. Поэтому runtime evidence зависит от instrumentation. Report должен честно показывать сбои breakpoints, полноту instrumentation, dropped events and timeout.

Этическая граница проявляется и в том, какие API collector watches. AddVectoredExceptionHandler, AddVectoredContinueHandler, SetUnhandledExceptionFilter, RtlAddFunctionTable, RtlInstallFunctionTableCallback are observed because they relate to exception/unwind route. Collector does not attempt to steal data, bypass licensing, patch anti-debug checks or reconstruct hidden payload. This is not only safety policy. It is architectural clarity: tool observes exception infrastructure; it does not become a general runtime manipulation framework.

Такой design helps defense. A defender, student or reverse engineer can run tool in controlled environment, collect evidence, inspect report, import facts into Ghidra and decide manual next steps. The project remains on analysis side. It uses powerful OS mechanisms but constrains their purpose and reports their limits.

Сжатая карта назначения проекта

В конце раздела полезно оставить не список ради списка, а компактную карту. Она нужна, чтобы перед следующими подразделами держать в голове, какую часть общей проблемы закрывает каждый слой проекта.

```
input bytes
  -> util.c: safe file read, bounded little-endian reads, strings, hashes
  -> pe.c: PE32+ AMD64 image model, sections, RVA mapping, Exception Directory
  -> unwind.c: RUNTIME_FUNCTION table, UNWIND_INFO decoder, handlers, chains
  -> heuristics.c: structured findings, evidence families, score/confidence/verdict
  -> dynamic_win.c: optional Windows Debug API runtime evidence collector
  -> report.c: console, CSV, Ghidra CSV, JSON schema, offline HTML, assessment
  -> ghidra/import_unwind_csv.py: transfer of facts into reverse-engineering workspace
  -> human analyst: manual validation, code inspection, final judgement
```

Эта карта не заменяет подробный разбор. Она только фиксирует направление. Дальше мы пойдем по ней медленно: от `main.c` и lifecycle одного запуска к безопасному чтению, PE parser, unwind decoder, heuristics, dynamic collector, reports, Ghidra integration and validation.

Опора на документацию и проектные источники

Раздел опирается на официальную документацию Microsoft по PE Format, x64 exception handling, RtlVirtualUnwind, Debug API, process creation/debug flags, vectored/unhandled exception APIs and dynamic function tables. Проектная часть опирается на README, docs/ARCHITECTURE.md, docs/evidence_matrix.md, docs/report.schema.json, docs/windows_dynamic_validation.md, docs/PRODUCT_READINESS.md and source files include/util.h, include/pe.h, include/unwind.h, include/heuristics.h, include/dynamic.h, include/report.h, src/main.c, src/pe.c, src/unwind.c, src/heuristics.c, src/dynamic_win.c, src/report.c and ghidra/import_unwind_csv.py.

11.2. Общий pipeline программы: как main.c превращает PE-файл в evidence, assessment и report

Почему pipeline важнее, чем просто список функций

Когда мы говорим "архитектура программы", легко скатиться в скучный список файлов: main.c вызывает pe.c, потом unwind.c, потом heuristics.c, потом report.c. Такой список формально правдив, но он почти ничего не объясняет. Он не показывает, почему именно такой порядок нужен, какие знания появляются на каждом этапе, какие знания еще запрещено делать, где программа должна остановиться, а где обязана продолжить частично, почему один и тот же PE-файл одновременно является недоверенным массивом bytes, структурной PE-моделью, источником unwind metadata, кандидатом для dynamic validation and finally material for human-readable reports.

В EH/Unwind Analyzer pipeline - это не декоративная схема. Слово pipeline можно перевести как "конвейер" или "цепочка обработки". В инженерном смысле это последовательность стадий, где каждая стадия берет вход в одном виде, добавляет к нему строго определенный слой смысла и передает дальше. Важное слово здесь - строго. Хороший pipeline не смешивает все сразу. Он не начинает рассуждать о suspicious handler до того, как проверил, что PE вообще читается. Он не строит dynamic assessment до того, как понял, какие handler RVA можно поставить на breakpoints. Он не пишет JSON, пока не собрал metadata, hash входного файла и stable model objects. Он не делает вид, что dynamic mode так же безопасен, как static mode. Он не превращает parse error в "malware", потому что parse error может быть malformed file, unsupported layout, truncated bytes or просто не тот вход.

Чтобы понять этот раздел, полезно держать в голове официальный Windows-контракт. Microsoft PE Format описывает, как image files organize headers, sections, data directories and RVA-based structures. Microsoft x64 exception handling описывает, что на x64 функции, которые требуют unwinding, представлены через RUNTIME_FUNCTION entries in .pdata and associated unwind information in .xdata. Документация RtlVirtualUnwind говорит не "угадай стек", а retrieves invocation context of the caller using the function context and unwind information. Debug API документация говорит, что debugger receives DEBUG_EVENT structures through WaitForDebugEvent and resumes the debuggee with ContinueDebugEvent. Проект не изобретает альтернативную Windows. Он строит защитный анализатор вокруг этих контрактов: сначала читает файл как PE, потом извлекает exception directory, потом декодирует unwind info, потом осторожно формулирует findings, потом при желании запускает target under debugger and collects runtime evidence, а затем объясняет результат в reports.

В этом смысле main.c - не "главный файл, где просто все вызовы". Это orchestrator. Orchestrator можно перевести как "координатор" или "дирижер". Он не обязан сам играть на всех инструментах, но он решает, когда вступает каждый инструмент, что делать при ошибке, кому принадлежит память, какие flags передать в decoder, нужно ли запускать heuristics, нужно ли переходить trust boundary в dynamic mode, как подготовить report metadata and when to release resources. В маленьких учебных программах main часто можно воспринимать как место, где программист накидал код. В анализаторе security tooling main is part of the trust model. Если он неправильно продолжит после ошибки, неправильно освободит память, не запишет metadata or смешает child arguments with report command line, весь анализ станет менее надежным.

Поэтому сейчас мы будем читать pipeline не как "первая функция, вторая функция". Мы будем смотреть, как в проекте постепенно рождается знание. Сначала есть только argv and bytes. Потом появляется Args - модель намерения пользователя. Потом появляется Buf - безопасно загруженный файл. Потом PeImage - структурное понимание PE. Потом RuntimeFunction array - таблица диапазонов, полученная из exception directory. Потом UnwindSummary array and us_ok - карта того, что удалось или не удалось декодировать. Потом HeuristicReport - статическое assessment, построенное не на сыром страхе, а на families of evidence. Потом DynamicReport - журнал runtime observations, если пользователь сознательно включил dynamic mode. Потом ReportMetadata - сведения, которые позволяют понять, каким инструментом, с какими flags and from which input был создан report. И только после этого появляются CSV, Ghidra CSV, JSON and HTML.

Стадия argv: пользователь еще не дал файл, он дал намерение

Самое первое, что видит программа, - это не PE, не .pdata and not handler. Она видит argc and argv. argc - количество аргументов командной строки, argv - массив строк. В простых программах аргументы часто воспринимают как мелочь: взял argv[1], открыл файл. Но для EH/Unwind Analyzer аргументы определяют весь режим знания. Один и тот же input file может быть обработан по-разному: просто brief summary, dump of runtime functions, dump with operations, static heuristics, dynamic validation, JSON/HTML reports, Ghidra CSV export, strict ABI diagnostics, chain following on or off. Поэтому первая стадия pipeline - не файл, а intent.

В проекте это намерение собирается в структуру Args. Это маленькая, но очень важная структура. Поле input хранит путь к PE-файлу. bool dump говорит, надо ли печатать RUNTIME_FUNCTION entries. bool show_ops говорит, надо ли вместе с dump показывать текст unwind operations. bool heuristics включает heuristic/evidence engine. csv_path, ghidra_csv_path, json_path and html_path задают output artifacts. follow_chain and max_chain_depth влияют на то, как

decoder будет работать с UNW_FLAG_CHAININFO. strict_abi включает более формальные Microsoft x64 unwind ABI diagnostics. dynamic включает Windows-only dynamic validation. timeout_ms and trace_steps задают границы runtime session. child_argv and child_argc хранят аргументы, которые после separator "--" надо передать уже не анализатору, а target program.

Эта структура делает важную вещь: отделяет parsing command line from executing analysis. Пока parse_args работает, программа еще не читает PE and not running target. Она только решает: что пользователь просит сделать и допустимы ли numeric values. Например, --limit должен быть в диапазоне 0..INT_MAX, --max-chain-depth - 1..16, --timeout-ms - positive uint32, --trace-steps - 0..256. Это не педантизм. Если max chain depth позволить без ограничений, malformed chained unwind info could force excessive recursion or pointless work. Если trace_steps не ограничить, dynamic trace could grow without bound. Если timeout_ms разрешить равным нулю by accident, пользователь может получить session semantics, которых не ожидал. Командная строка является внешним входом, а значит тоже частью trust boundary.

Важно, что --help and --version имеют отдельные ParseResult states. ParseResult - это enum, то есть ограниченный набор состояний: PARSE_OK, PARSE_HELP, PARSE_VERSION, PARSE_ERR. Такой подход лучше, чем возвращать просто bool. bool сказал бы только success/failure. Но help and version are not failures. They are valid requests that should exit with code 0. Parse error, наоборот, should print usage and exit with code 2. Уже здесь main.c отделяет "пользователь попросил справку" from "пользователь ошибся". Это маленькая дисциплина CLI contract, но она важна: scripts, CI and validators can interpret exit codes predictably.

Особенно интересен separator "--". До него arguments belong to analyzer. После него arguments belong to target program in dynamic mode. Это разделение защищает semantic layers. Если пользователь пишет:

```
eh_unwind_analyzer target.exe --dynamic -- --input sample.dat --mode test
```

то --input and --mode после separator не являются options анализатора. Это arguments child process. В dynamic mode analyzer eventually starts target.exe under debugger and passes those arguments to the target. Поэтому parse_args stores child_argv and child_argc rather than trying to understand them. Analyzer must not interpret target arguments as its own. Это пример уважения границы: one command line, two consumers, different meanings.

UTF-8, command line and why report metadata starts earlier than reports

В main.c есть helper argv_text_to_utf8. На первый взгляд это не относится к reverse engineering. Но на самом деле это относится к отчетности and reproducibility. Reproducibility можно перевести как "воспроизводимость". Report должен не только сказать, что найдено, но и дать человеку возможность понять, как analysis was invoked. Если путь к файлу or arguments contain non-ASCII characters, especially on Windows with active code page, direct byte copying can produce broken JSON or unreadable HTML. Поэтому project converts argv text to UTF-8 when preparing display/report strings.

UTF-8 здесь не "красота для русского пути". Это contract boundary between OS-specific command line encoding and portable reports. JSON report and HTML report should be understandable outside the original console encoding. Если report contains mangled path, a later reviewer may not know which sample was analyzed. If command line is not preserved, a dynamic result is less meaningful: target arguments may determine whether exception path is reached. Поэтому metadata work begins conceptually from the first moments of program execution, even though actual ReportMetadata structure is filled later.

Но проект also redacts child arguments after "--" in build_invocation. Redact - значит "скрыть чувствительную часть". Dynamic target arguments may contain secrets: tokens, paths, user data, test credentials. The analyzer wants report provenance, but not uncontrolled leakage. Поэтому command_line in metadata keeps analyzer options and then writes something like "-- [N target arguments redacted]". Это очень зрелое решение: report remains reproducible enough to know that target arguments existed, but does not blindly publish them. Again, this is not just report.c detail. main.c prepares the semantic contract: analysis metadata should help, not leak.

Если смотреть глубже, здесь видно различие между analysis input and execution input. The PE file itself is analyzed and hashed. Target arguments influence dynamic behavior, but are not part of static file identity. Поэтому report metadata should store input SHA-256, input size, schema version, tool version, build compiler, build platform, build mode, strict_abi, follow_chain and max_chain_depth. These fields let another person interpret report. Was strict ABI enabled? Did decoder follow chained unwind info? Which schema version should validator use? Which tool version generated this JSON? Without this, report becomes isolated text, not an engineering artifact.

Стадия file loading: PE еще не PE, это недоверенные bytes

После parse_args main.c calls buf_init_from_file. Здесь input path becomes Buf. Buf - это простая структура: pointer to data and size. Но смысл этой стадии огромный. До pe_parse не существует PeImage. Не существует sections. Не существует exception directory. Не существует is_pe64. Есть только bytes. Эти bytes недоверенные. Они могут быть корректным PE-файлом, урезанным PE, случайным текстом, maliciously malformed sample, corrupted download or специально созданным fuzzing input. Поэтому первый слой проекта - util.c - читает file into memory and gives the rest of program a bounded byte buffer.

Bounded означает, что у каждого чтения есть граница. util.c предоставляет buf_read, buf_read_u8, buf_read_u16, buf_read_u32, buf_read_u64. Эти функции не просто удобство. Они запрещают decoder-у "случайно" читать за конец файла. В C такой запрет нельзя получить автоматически. Если parser делает pointer arithmetic directly and trusts offsets, malformed PE can produce out-of-bounds read. Это может быть crash, undefined behavior, false report or security issue in the analyzer itself. Поэтому pipeline начинается не с умной эвристики, а с скучной надежности: read bytes, remember size, every future offset must prove it is inside buffer.

Эта мысль особенно важна для reverse engineering tools. Инструмент, который анализирует недоверенные binaries, сам становится attack surface. Attack surface можно перевести как "поверхность атаки" - набор мест, через которые вход может повлиять на программу. Static mode intentionally avoids executing target, but parsing malformed file is still code running in analyzer process. Поэтому static mode safer than dynamic mode, но не magically safe. Без safe reads malformed file can break analyzer. С safe reads malformed file becomes structured error or finding.

Если file loading fails, main.c prints "Failed to read file" and exits with code 1. Обрати внимание: программа не пытается угадать, почему не прочитала файл. It does not say suspicious, packed or protected. It says failed to read. Это часть честного language discipline. На каждом этапе error message should describe that stage. File read failure belongs to file I/O, not PE validation. PE parse error belongs to PE parser, not heuristic verdict. Dynamic failure belongs to dynamic collection, not static evidence.

pe_parse: момент, когда bytes становятся PE-моделью

После загрузки bytes main.c initializes StrBuf err and calls pe_parse. StrBuf - string buffer, расширяемый буфер строки. Он нужен, чтобы parser мог вернуть не только false, but explanation. pe_parse fills PeImage. PeImage is the first real semantic model of the input. It does not contain all bytes. It contains normalized structural facts: is_pe64, machine, image_base, size_of_image, size_of_headers, section_alignment, file_alignment, exception_rva, exception_size, file_size, num_sections, sections, plus validation counters.

Здесь важно слово normalized. PE file has many raw structures: DOS header, e_lfanew, NT signature, FileHeader, OptionalHeader64, DataDirectory array, Section Headers. pe.c reads those structures and turns them into a PeImage that other modules can use without re-reading the whole PE header logic. This prevents duplication. unwind.c should not rediscover how to parse sections. heuristics.c should not know DOS header offsets. report.c should not reinterpret raw headers. They all receive PeImage as a stable model.

Microsoft PE Format describes Portable Executable and COFF files, headers, sections, RVA and data directories. EH/Unwind Analyzer narrows this broad world to PE32+ AMD64 because Windows x64 exception/unwind model is architecture-specific. In pe.h this is visible as IMAGE_FILE_MACHINE_AMD64 and IMAGE_NT_OPTIONAL_HDR64_MAGIC. This is not limitation by laziness. It is architecture boundary. ARM64, x86 and other formats have different exception metadata and different unwind conventions. A responsible tool should not weaken AMD64 assumptions to accept everything. It should either reject unsupported architecture or add architecture-specific parser later behind a separate model.

PeImage also stores structure counters: section_raw_oob, section_virtual_oob, section_raw_overlaps, section_virtual_overlaps, section_misaligned and exception_directory_oob. These are not final verdicts. They are structural observations. section_raw_oob means a section raw range claims bytes outside file boundaries. section_virtual_oob means virtual range is suspicious relative to SizeOfImage. overlaps mean ranges collide. misaligned means section layout violates expected alignment discipline. exception_directory_oob means exception directory could not be safely mapped or lies outside expected bounds. These facts become useful later for heuristic/evidence engine and reports, but pe_parse itself should not jump to conclusions. Its job is to create the structural ground truth as far as possible.

The first trust boundary inside PE parsing is RVA mapping. pe_rva_to_fileoff asks: given an RVA from image-relative world, where are its bytes in the file? It must find a section whose virtual range contains the RVA and then calculate raw offset. This is why Section Table matters. Exception Directory stores RVA and size, not direct file offsets. RUNTIME_FUNCTION entries store BeginAddress, EndAddress and UnwindData as RVAs. UNWIND_INFO is reached by UnwindData RVA. Handler data is reached after unwind codes and alignment. Every one of these steps depends on

correct RVA-to-file mapping. If pe.c lies, the whole pipeline reads wrong bytes.

If pe_parse fails, main.c stops. This is correct. Without PeImage, later stages have no coordinate system. There is no safe way to read exception directory if machine, headers, sections and data directories are not understood. Here partial reporting is not useful enough because the primary model is absent. But after PeImage exists, some later errors can be partial, as we will see with exception directory reading.

Reading exception directory: from PE structure to RUNTIME_FUNCTION table

Once PeImage exists, main.c calls unwind_read_runtime_functions_ex. This is the stage where project moves from general PE parsing to Windows x64 unwind territory. The PE data directory entry IMAGE_DIRECTORY_ENTRY_EXCEPTION points to exception directory. On x64, that directory is the table of RUNTIME_FUNCTION entries, usually stored in .pdata. But the project should not rely only on section name .pdata. Officially, the entry point is the Exception Directory data directory. Section name is a helpful convention, not the contract. A strange or optimized binary can place data in sections with unusual names. Therefore the parser uses exception_rva and exception_size from PeImage, maps them through sections, and reads entries structurally.

RuntimeFunction in project is exactly three uint32_t fields: begin_rva, end_rva, unwind_rva. These correspond to the x64 function table idea: a range of code and a pointer to unwind metadata. It is not a source-level function name. It is not a decompiler function. It is not automatically a symbol. It is a range in image-relative coordinates for unwinding. The project stores an array RuntimeFunction* rf and count rf_count. This array becomes a backbone for the rest of the pipeline. dump uses it. unwind_parse uses it indirectly by reading each unwind_rva. heuristics analyze table integrity and relationships. dynamic collector uses handler RVAs found through unwind summaries to set breakpoints in runtime process. reports serialize functions and their unwind data.

unwind_read_runtime_functions_ex can return three states: RUNTIME_FUNCTION_READ_OK, RUNTIME_FUNCTION_READ_MALFORMED and RUNTIME_FUNCTION_READ_OOM. This is another example where bool would be too poor. Out-of-memory means analyzer cannot continue safely. main.c frees PeImage, StrBufs and Buf and returns 1. Malformed exception directory is different. The file may still be useful for partial structural report. main.c marks img.exception_directory_oob = true, prints that it is continuing with partial structural report, and sets exit_code = 1. That means the process can still write reports if requested, but final exit status indicates something went wrong. This is nuanced behavior: continue when there is still evidence to serialize, but do not pretend success.

Why is this important? A malformed exception directory itself is meaningful. If analyzer simply stopped and produced nothing, user would lose structural context: image base, sections, validation counters. If analyzer pretended everything was fine, user would overtrust absence of runtime functions. The chosen behavior says: we could parse PE enough to report structure, but exception directory was unreadable; this is a problem; downstream should know. This is the difference between failing closed and failing informative.

The project also calls report_print_brief after attempting to read runtime functions. This prints a high-level summary. Brief output is not the final report, but it gives immediate console feedback: input, image properties, number of runtime functions. In a command-line tool, this matters. A user running analyzer manually should know early whether it found exception records. But brief output still does not replace structured JSON or HTML. It is a human console snapshot.

UnwindSummary arrays: why the project separates records from decoded meaning

After RuntimeFunction table is read, main.c allocates two arrays: UnwindSummary* us and bool* us_ok. This is a subtle design decision. The project does not overwrite RuntimeFunction with decoded data. It keeps raw table entries separate from summaries of UNWIND_INFO parsing. RuntimeFunction answers: which code range points to which unwind_rva? UnwindSummary answers: what was found at that unwind_rva when decoded as x64 UNWIND_INFO? us_ok answers: did decoding succeed for this entry?

This separation matters because parse failure is itself evidence. Suppose RUNTIME_FUNCTION entry exists but unwind_parse fails. If the project stored only valid summaries, that function might disappear from later analysis. Disappearance would be misleading. The range exists in .pdata. The unwind metadata did not decode. A report should be able to say: function range exists, unwind_rva is X, parsing failed with error Y. Heuristics should count unwind_parse_fail. Dynamic collector should not assume handler data from invalid summary. Ghidra CSV may still mark the range with warning. Therefore project keeps parallel arrays: rf[i] always refers to the table entry; us[i] contains whatever summary could be produced; us_ok[i] states validity.

UnwindParseOpts is prepared once from user Args: max_chain_depth, follow_chain and strict_abi. max_chain_depth protects against chain loops or extreme chain nesting. follow_chain controls whether UNW_FLAG_CHAININFO is followed. strict_abi controls whether formal Microsoft x64 unwind ABI violations are flagged. These options make decoder behavior explicit and reportable. If two analysts compare results, they must know whether chain following was enabled and strict ABI was on. That is why these same values later appear in ReportMetadata.

The loop over rf_count calls unwind_parse for each entry. Each parse gets Buf, PeImage, rf[i].unwind_rva, opts, output UnwindSummary, optional ops_text and per-entry error buffer. This signature says a lot about architecture. unwind_parse does not receive raw file offset. It receives an RVA and the PE image model, because it should use correct mapping. It does not receive global variables. It receives options explicitly. It can write human-readable unwind operations if asked, but that output is optional. It fills a compact summary that later modules can inspect. This keeps the decoder focused: it translates Windows x64 unwind metadata into normalized facts.

UnwindSummary fields are also carefully chosen. version, flags, prolog_size, count_codes, frame_reg and frame_off mirror key UNWIND_INFO header concepts. has_handler and handler_rva expose EHANDLER/UHANDLER consequences. has_chain and chain expose chained runtime function information. fixed_stack_bytes, pushed_nonvol_count, saved_nonvol_mask and saved_xmm_mask are derived facts about frame effects. anomalies is a bitmask for parser/ABI diagnostics. parse_error carries human explanation when decoding fails. This summary is neither raw bytes nor final verdict. It is a middle representation: close enough to official structures to be faithful, normalized enough for heuristics and reports.

This is a core architectural pattern: each layer should raise the level of meaning by exactly one step. Buf is bytes. PeImage is PE structure. RuntimeFunction array is exception directory table. UnwindSummary is decoded unwind semantics. HeuristicReport is evidence assessment. DynamicReport is runtime observation. Report is serialized explanation. When a project skips levels, bugs become harder to find. If heuristics directly read raw bytes, they can confuse parser bugs with suspicious findings. If report directly re-decodes unwind info, it can disagree with heuristics. If dynamic collector ignores static summaries, it loses static-to-runtime correlation. EH/Unwind Analyzer avoids that by passing normalized objects forward.

Dump mode: teaching the human before writing heavy reports

If user enables --dump, main.c prints RuntimeFunction entries. If --ops is also enabled, it re-runs unwind_parse with ops_text and prints operation details. At first this may seem inefficient: why parse again for ops? But there is a practical reason. Normal analysis stores compact UnwindSummary. Full textual operation dump is only needed for human inspection. Rather than storing huge text for every function by default, project generates ops text on demand. This keeps default memory model smaller and separates structured model from debug/explanatory text.

The dump output is also pedagogical. It prints begin/end range, size, unwind RVA, version, flags, prolog size, stack allocation estimate, frame pointer information, handler RVA, chain info and anomalies. This gives a reverse engineer a direct view of the .pdata/.xdata layer without requiring JSON. For early debugging of samples,

--dump --ops is often the fastest way to see whether parser is understanding metadata. It is not the final assessment, but it trains the same mental model as the book: code range -> unwind info -> prolog effects -> handler/chain/anomalies.

Notice that dump mode does not require heuristics. This is good separation. A user may want raw/decoded facts without scoring. Another user may want heuristic analysis without dumping hundreds of entries. CLI options map to layers of curiosity. --dump asks "show me the decoded records". --heuristics asks "assess anomalies". --dynamic asks "observe runtime behavior". --json/--html asks "serialize complete result". A mature tool lets these modes combine without collapsing them into one huge always-on behavior.

Limit handling is also part of usability. dump_limit defaults to -1, meaning all when dump is enabled. --limit N restricts number printed. This prevents massive PE files from flooding console. But the analyzer still parses all entries before dump. Limit is output control, not analysis truncation. That distinction matters: if user asks for report and dump limit 10, heuristics should still analyze all runtime functions. Output convenience should not silently weaken analysis.

Heuristic stage: findings begin only after structure is normalized

Only after PE parsing, runtime function reading and unwind parsing does main.c optionally call heuristics_analyze. This order is not arbitrary. Heuristics should consume normalized facts, not raw bytes. In the project docs this is explicit: heuristics.c receives const PeImage* img, const RuntimeFunction* rf, const UnwindSummary* us, const bool* us_ok and count n. It does not own the file. It does not execute code. It does not decode PE again. It receives already normalized structural and unwind facts and produces HeuristicReport.

Heuristic means practical inference rule. In this project it is not machine learning, not calibrated malware probability and not a magical classifier. The heuristic/evidence engine asks: given the normalized facts, are there patterns that deserve attention? Are runtime function ranges invalid, out of order, overlapping or duplicate? Do UNWIND_INFO records fail to parse? Are there ABI anomalies under strict mode? Is unwind metadata located in executable or writable sections? Do many functions reuse the same unwind info? Do many handlers point to the same target? Do PE structural counters indicate malformed layout? These questions produce findings and counters, then score, suspicion, confidence, evidence_level, verdict, weakening rules and manual review flag.

main.c handles heuristics carefully. It allocates HeuristicReport hr on stack, initializes perr, calls heuristics_analyze, prints report_print_heuristics if successful and stores hr_ptr = &hr. If heuristics fail internally, it prints error and sets exit_code = 1. The pointer hr_ptr is important because report generation later receives either a real HeuristicReport or NULL. If user did not request --heuristics, report writers

should not fabricate heuristic_analysis as if it ran. If it ran, reports can include it. This preserves provenance: absence of heuristic block means not run, not "no findings".

This distinction is one of the most common mistakes in analysis tooling. Not collected is not none. Not decoded is not empty. Not observed is not absent. EH/Unwind Analyzer carries this idea through pointers and flags. hr_ptr is NULL if heuristic analysis did not run or failed. dr_ptr is NULL if dynamic validation did not run. us_ok[i] false if this unwind record failed to parse. DynamicReport.enabled/ran/ok/timed_out describe runtime session state. Good architecture makes epistemic state visible in data structures.

The heuristic stage also introduces a controlled language of suspicion. Score is bounded engineering weight, not percent malware. Suspicion is derived from score, but it is still project-scale suspicion, not statistical probability. Confidence is separate and rule-based: how complete and corroborated the static evidence appears, reduced by weakening rules when evidence is single-signal, small-sample, placement-only or single-family. main.c does not compute these; heuristics.c does. But main.c decides when the stage exists and how it is passed to reporting. Orchestrator does not need to know all weights, but it must not blur whether report contains a real HeuristicReport.

Dynamic stage: the explicit crossing of execution boundary

If user enables --dynamic, main.c calls dynamic_collect_windows. This is the most dramatic transition in the pipeline. Until this point, analyzer only read bytes. It did not execute the target. Dynamic mode starts target program under a debugger and observes runtime events. This crosses the execution boundary. Execution boundary means the tool moves from passive inspection of untrusted input to active creation of a child process. That is why dynamic mode is optional, clearly named and Windows-only.

The call passes a.input, child_argv/child_argc, PeImage, RuntimeFunction array, UnwindSummary array, us_ok array, rf_count, timeout_ms, trace_steps, output DynamicReport and error buffer. This signature shows static-to-dynamic dependency. Dynamic collector is not a blind debugger. It uses static parsing results to discover handler RVAs from UNWIND_INFO and then set breakpoints at runtime VAs after the debuggee image base is known. Static analysis tells collector where interesting exception handlers may be. Dynamic collector tells whether some of them are observed in a particular run.

On Windows, debugger loop relies on Debug API: CreateProcess with debug flags, WaitForDebugEvent, DEBUG_EVENT, ContinueDebugEvent, and thread context manipulation for breakpoints/single-step. But main.c does not implement those details. It delegates to dynamic_win.c. That is proper modularity. main.c should know that dynamic collection may run, may fail, may time out, may collect events. It should not know how to place 0xCC or re-arm breakpoints. This separation keeps the dynamic trust boundary contained in one module.

DynamicReport is initialized and freed with dynamic_report_init and dynamic_report_free. The report contains enabled, ran, ok, timed_out, target_exited, target_exit_code, timeout_ms, trace_steps, instrumentation completeness, module observation counters, breakpoint counters, exception counters, handler hit counters, VEH/VCH/UEF counters, RtlAddFunctionTable counters, event list, trace list and human-readable details. This is not a verdict. It is a runtime observation journal. main.c prints a short summary: exceptions_seen, unwind_handler_hits, veh_hits, dyn_ftable_calls. If details exist, it prints them. But the final interpretation belongs to reporting/assessment layer.

The bool returned by dynamic_collect_windows is also subtle. Documentation in dynamic.h says it returns true if the dynamic session ran, even if it found no evidence. That means false does not mean "no suspicious runtime behavior". False means the session did not run successfully enough. True with zero events means ran but observed no relevant evidence under this scenario. ran/ok/timed_out inside DynamicReport further refine the state. This is exactly the epistemic discipline the book has repeated: no event observed is not the same as event impossible.

If dynamic collection fails to run, main.c prints [Dynamic] Failed and sets exit_code = 1. It still continues to report writing with dr_ptr pointing to DynamicReport after initialization. This can be useful: a JSON report may say dynamic_validation enabled but ran false or ok false. A human can see dynamic was requested but failed. Again, this is better than silently omitting dynamic block and making the user think dynamic was not requested.

Static objects become dynamic instrumentation hints

The bridge from static to dynamic deserves separate attention because it is the heart of EH/Unwind Analyzer. Static parser finds handler_rva in UnwindSummary when UNWIND_INFO has handler flags. But handler_rva is not a runtime address. It is image-relative. Dynamic collector must wait for CREATE_PROCESS_DEBUG_EVENT, learn base_main from the debug event, and then calculate handler VA = base_main + handler_rva. Only then can it set a software breakpoint. This bridge would be impossible without the earlier pipeline stages.

If unwind parsing failed for an entry, us_ok[i] is false. Then dynamic collector should not trust handler fields for that entry. If PE parser mis-mapped RVA, handler_rva could be wrong. If module base is misidentified, breakpoint VA could be wrong. If breakpoint write fails due page protection or invalid address, dynamic report must record breakpoint_set_failures. If target never throws exception that reaches handler, hit count remains zero. If timeout happens before path, evidence is incomplete. Therefore dynamic evidence is not isolated. It depends on the whole chain: file bytes -> PeImage -> RuntimeFunction -> UnwindSummary -> handler RVA -> debuggee module base -> breakpoint -> event.

This chain is why 11.2 belongs after chapters 9 and 10. We already know terms like RVA, handler hit, breakpoint, first chance, dynamic function table. Now we can

see how they are assembled in code. main.c does not merely call dynamic_collect_windows after heuristics because it feels convenient. It calls it after static decode because dynamic collector needs static model. It runs before JSON/HTML because reports should include dynamic results. It uses timeout and trace_steps from Args because runtime observation must be bounded and user-controllable.

Dynamic validation also uses target arguments after "--". That means dynamic session is scenario-dependent. Running target.exe with no args and running target.exe --input sample.dat may exercise completely different exception paths. main.c preserves child args for CreateProcess construction but redacts them in command line metadata. This is a careful compromise. The analyzer must pass scenario inputs to target, but reports should not leak them by default. The report still records that target arguments were redacted, so analyst knows dynamic behavior is not necessarily from a no-argument run.

Reporting outputs: same model, different readers

After optional heuristics and optional dynamic collection, main.c writes outputs. It can write CSV, Ghidra CSV, JSON and HTML. These are not four independent analyses. They are different serializations of the same model objects. Serialization means converting in-memory structures into external representation. The project deliberately writes reports after parsing, decoding, heuristics and dynamic stages because reports should reflect the accumulated evidence, not trigger new hidden analysis.

CSV is a tabular output for analysis workflows. It receives PeImage, rf, us, us_ok and n. In normal mode it writes full fields. In ghidra_mode it writes a simplified CSV for Ghidra import: begin/end/name/comment style information. main.c uses the same report_write_csv function with a boolean flag. This is a practical design: one writer function can share escaping and atomic write logic while adapting columns. But the semantic readers differ. Normal CSV is for data inspection. Ghidra CSV is a bridge into reverse engineering workspace.

JSON and HTML require ReportMetadata. That is why main.c prepares metadata only if a.json_path or a.html_path exists. Metadata preparation includes input_display in UTF-8, invocation string with redacted child args, UTC timestamp, SHA-256 of input bytes, tool version, schema version, input size, compiler, platform, build mode, strict_abi, follow_chain and max_chain_depth. This is not filler. JSON is machine-readable contract. HTML is human-readable interactive report. Both need provenance to be trustworthy.

The SHA-256 hash is especially important. File names can lie. Paths can change. Samples can be replaced. Hash gives stable identity of bytes analyzed. If someone compares two reports, same input path but different SHA-256 means not the same file. Different path but same SHA-256 means same bytes. In malware/reverse engineering workflows, this matters because samples move between directories,

archives and machines. A report without hash is much weaker evidence.

report_write_json_ex receives hr_ptr and dr_ptr. This means JSON may include heuristic_analysis and dynamic_validation only when available. It also receives metadata and input path. report_write_html receives the same model. This design avoids divergence: HTML and JSON are different presentations of the same evidence. If HTML says final_verdict but JSON says another, that would be a bug. Therefore report.c should centralize assessment logic or use shared helpers, not invent separate interpretations in each format. The project docs emphasize schema validation and cross-field invariants precisely because reports are part of analysis correctness.

Atomic writes: report creation must not destroy previous evidence

CSV, JSON and HTML writers use temporary sibling files and atomic replacement. This detail may look low-level, but it is part of pipeline reliability. Suppose analyzer is writing report.json and process crashes halfway. If it wrote directly to final file, user might be left with truncated invalid JSON, possibly overwriting a previous good report. Atomic write means writer first writes to a temporary file in the same directory, then replaces final path once complete. This reduces the chance of losing previous artifact.

In util.h the function file_write_atomic belongs to this reliability layer. It is not reverse-engineering theory, but it protects evidence workflow. Reports are artifacts that may be used in defense, comparison, CI validation or manual review. Corrupting them on interrupted write is not acceptable. The same mindset appears throughout project: safe reads for input, bounded arrays for dynamic events, schema for JSON, CSP for HTML, redaction for command line, clear exit codes for failures.

Atomic write also reinforces a larger theme: output files are not just convenience. They are part of the chain of custody for analysis. Chain of custody usually means documenting how evidence was handled. In our context it means a report should be tied to tool version, input hash, options and complete write. If output itself is unreliable, the analysis cannot be trusted regardless of how clever heuristics are.

Ownership and cleanup: why memory discipline is part of architecture

main.c ends with cleanup: heuristics_free if heuristics ran, dynamic_report_free if initialized, sb_free for ops and error buffers, free us_ok, free us, free rf, pe_free img, buf_free_owned owned. This is not decorative. In C, ownership must be explicit. Architecture.md states ownership rules: buf_init_from_file allocates input bytes and buf_free_owned releases them; pe_parse owns PeImage.sections and pe_free releases them; unwind_read_runtime_functions returns heap array owned by caller; heuristics_analyze owns internal StrBuf and heuristics_free releases it; dynamic_report_init owns details and dynamic_report_free releases it; report writers borrow model objects and do not retain pointers.

Borrow means a function receives pointer but does not keep it after returning. Own means function or caller is responsible for lifetime. If report writers borrowed and then secretly stored pointers globally, cleanup would create dangling pointers. If main.c forgot to free PeImage.sections on early failure, fuzzing could reveal leaks. If dynamic_report_free were called on uninitialized report, it might free invalid memory. Therefore main.c uses dr_initialized to know whether dynamic_report_free is valid. It frees partially constructed objects on early failure paths.

This matters for untrusted inputs. Sanitizer and mutation tests exercise malformed inputs through early failure paths. A tool that only cleans up on success is fragile. Real samples fail in many ways: invalid MZ, wrong machine, truncated optional header, section table out of bounds, exception directory malformed, out-of-memory, report write failure. Each path must release what has already been constructed. That is why main.c looks longer than a toy pipeline. It is not accidental verbosity. It is ownership discipline.

Exit code also belongs here. main.c starts exit_code = 0. Some recoverable-but-serious problems set exit_code = 1 while allowing reports to be produced: malformed exception directory, heuristic failure, dynamic failure, report write failure. Parse errors return 2. Help/version return 0. This creates a CLI contract for scripts and CI. A human may read partial report, but automation can still detect non-zero exit. Again, pipeline does not hide uncertainty.

A complete run as a living sequence

Now let us walk through a realistic command, not as syntax, but as meaning:

```
eh_unwind_analyzer sample.exe --dump --ops --heuristics --strict-abi --json sample.json --html samp
```

First, parse_args turns this into Args. input is sample.exe. dump and show_ops are true. heuristics is true. strict_abi is true. json_path, html_path and ghidra_csv_path are set. dynamic is false, so no execution boundary will be crossed. follow_chain remains default true. max_chain_depth remains 8. timeout and trace options exist but will not matter because dynamic is false.

Then buf_init_from_file loads sample.exe into memory. The analyzer still knows nothing except bytes and size. pe_parse checks whether these bytes form PE32+ AMD64 image, reads sections, image base, alignment, data directories and validation counters. If this fails, analysis stops. If it succeeds, unwind_read_runtime_functions_ex uses Exception Directory to read RUNTIME_FUNCTION table. If malformed but not OOM, it records exception_directory_oob and continues partial. report_print_brief gives immediate summary.

Next, main allocates UnwindSummary and us_ok arrays. For every RuntimeFunction entry, unwind_parse decodes UNWIND_INFO under strict_abi true, follow_chain true and max_chain_depth 8. This populates flags, prolog size, stack frame facts, handler RVA, chain information, anomalies and parse errors. Because dump and ops are enabled, main prints range summaries and operation text. This gives the human immediate view.

Then heuristics_analyze consumes PeImage, RuntimeFunction table, UnwindSummary array and us_ok. It produces HeuristicReport. report_print_heuristics prints human summary. No dynamic collection happens. Then report_write_csv writes Ghidra CSV from the same model. Metadata is prepared: UTF-8 input path, redacted invocation, UTC timestamp, SHA-256 hash, tool and schema versions, build information, strict_abi true, follow_chain true, max_chain_depth 8. report_write_json_ex and report_write_html write structured and human reports. Finally cleanup releases every owned resource and returns exit_code.

The important thing is not the command. The important thing is that no stage overreaches. Dump does not create findings. Heuristics do not execute code. JSON does not secretly run dynamic validation. Ghidra CSV does not invent decompiler truth. strict_abi affects decoder diagnostics and metadata. The pipeline makes it possible to explain exactly how result was produced.

A dynamic run as a different kind of knowledge

Now consider a dynamic command:

```
eh_unwind_analyzer target.exe --heuristics --dynamic --timeout-ms 20000 --trace-steps 64 --json dyn
```

The separator changes meaning. Arguments after "--" are for target.exe, not for analyzer. Static stages still happen first. The analyzer must parse target.exe as PE, read exception directory, decode unwind info and run heuristics if requested. Only then dynamic_collect_windows starts target.exe under debugger with child args. The target may receive --input case1.bin. The report command_line will redact that child part.

During dynamic collection, the runtime world appears. The collector observes CREATE_PROCESS_DEBUG_EVENT, module loads, exceptions, breakpoint hits, VEH/VCH/UEF registrations, dynamic function table registrations and optional RIP

trace after handler hit. It uses static handler RVAs to attempt unwind handler breakpoints. It uses timeout to stop session if target hangs or path is long. It uses trace_steps to bound single-step trace. When session ends, DynamicReport carries enabled/ran/ok/timed_out/target_exited plus counters and events.

When JSON is written, static and dynamic evidence coexist but do not merge carelessly. Static findings say what the file contains. Dynamic events say what happened in this run. Assessment can correlate if conditions are met: for example suspicious static handler-related evidence plus observed runtime handler/VEH/VCH/UEF hits, or static placement anomalies plus dynamic function table activity. But incomplete session caps runtime evidence. Timeout weakens interpretation. Dropped events weaken completeness. The command itself becomes part of meaning because scenario input affects coverage.

This is why dynamic mode is optional and explicit. It produces more interesting evidence, but also less universal evidence. Static analysis can be repeated without running target. Dynamic validation can show living behavior, but only for the chosen path. Pipeline reflects that difference by keeping DynamicReport separate and by requiring the user to request --dynamic.

Why the pipeline does not start with "is this malware?"

A beginner might ask: if the tool is about suspicious exception/unwind behavior, why not start by classifying sample? The answer is that classification would be the wrong abstraction. EH/Unwind Analyzer is not a malware classifier. It is an evidence tool for Windows x64 exception/unwind infrastructure. It can say that metadata is malformed, handlers are concentrated, unwind info is in executable/writable section, dynamic function table registration occurred, handler hit was observed, confidence is limited, manual review is required. These are concrete statements. "Malware" is a much larger conclusion requiring behavior, context, reputation, payload analysis, persistence, network activity, intent and many other signals.

The pipeline protects against overclaiming by forcing analysis to pass through grounded stages. A PE parser can say PE header invalid. Unwind decoder can say UNWIND_INFO truncated. Heuristics can say strong static evidence from multiple families. Dynamic collector can say AddVectoredExceptionHandler was called and handler later hit. Assessment can say strong correlated case. A human may then decide the sample is malicious or benign, but the tool's language remains evidence-based.

This matters for false positives. Many legitimate systems use exceptions, dynamic code, JIT, crash handlers, guard pages or centralized exception dispatch. A game engine, .NET runtime component, browser JIT, DRM-protected app, crash reporter or compiler runtime may produce unusual patterns. The analyzer should help reverse engineer understand those patterns. It should not punish every unusual pattern with dramatic labels. Pipeline is a discipline of humility.

Where official documentation ends and project architecture begins

Official Microsoft documentation tells us the contract: PE files have headers, sections, RVA, data directories; x64 exception handling uses RUNTIME_FUNCTION and UNWIND_INFO; RtlVirtualUnwind computes caller context; Debug API provides debug events and continuation. It does not tell us how to build a robust static analyzer, how to score evidence, how to serialize reports, how to preserve metadata, how to avoid leaking target args, how to handle malformed samples without crashing, how to correlate static findings and dynamic runtime evidence conservatively. That is project architecture.

EH/Unwind Analyzer fills that gap by layering responsibilities. util.c protects byte access and output writes. pe.c builds PE coordinate system. unwind.c decodes exception/unwind metadata. heuristics.c turns normalized facts into findings. dynamic_win.c collects runtime evidence under debugger. report.c serializes results and assessment. app_info.c provides tool/schema/build identity. main.c orchestrates them, carries ownership, honors CLI contract and decides stage order. This division is not arbitrary. It follows the shape of knowledge.

If a future developer wants to add architecture support, they should not weaken pe.c/unwind.c AMD64 assumptions casually. They should add architecture-specific model. If they want new heuristic, they should add structured finding and document it in evidence matrix. If they want new report field, they need schema version decision and tests. If they want new dynamic API hook, they should add breakpoint role bits plus structured events. These extension rules flow from pipeline design. A pipeline is not only how program runs today. It is how the program remains understandable tomorrow.

The pipeline as a map for reading code

Now, when you open main.c, you should not see a random sequence of calls. You should see a map. parse_args creates user intent. buf_init_from_file creates bounded bytes. pe_parse creates image model. unwind_read_runtime_functions_ex extracts .pdata/function table. unwind_parse creates per-entry unwind semantics. dump optionally teaches human. heuristics_analyze creates static evidence assessment. dynamic_collect_windows creates runtime evidence journal. report_write_csv/report_write_json_ex/report_write_html externalize model. cleanup enforces ownership.

This reading style helps with debugging the project. If JSON lacks a field, ask: was the model object filled? If heuristic finding seems wrong, ask: is source PeImage/UnwindSummary correct? If dynamic breakpoint not hit, ask: did static handler_rva exist, did module base resolve, did breakpoint set succeed, did scenario reach path? If report metadata wrong, ask: was build_invocation or app_info wrong? If crash happens on malformed file, ask: which stage trusted offset too early? The

pipeline gives you a diagnostic route for the analyzer itself.

It also helps explain the project in defense or documentation. You can say: this is not one monolithic scanner. It is a staged analyzer. Each stage has input, output, ownership and limitations. Static mode never executes target. Dynamic mode is explicitly requested and bounded. Reports borrow final model objects and do not own them. JSON schema locks output contract. Tests validate both schema and cross-field invariants. That is a much stronger explanation than "program parses PE and finds anomalies".

The mental diagram to carry forward

The architecture can be summarized in one flow, but now the flow should feel alive rather than abstract:

```
argv / CLI intent
  -> Args
  -> input file bytes as bounded Buf
  -> PeImage: PE32+ AMD64 headers, sections, RVA map, validation counters
  -> RuntimeFunction[]: Exception Directory / .pdata ranges
  -> UnwindSummary[] + us_ok[]: decoded .xdata / UNWIND_INFO semantics
  -> optional dump / ops: immediate human inspection
  -> optional HeuristicReport: static evidence, score, confidence, verdict
  -> optional DynamicReport: debugger session, runtime events, handler hits, trace
  -> ReportMetadata: provenance, hash, schema, build and options
  -> CSV / Ghidra CSV / JSON / HTML
  -> cleanup and exit status
```

Every arrow in this diagram is a promise. Args promise that user intent is parsed and bounded. Buf promises safe byte access. PeImage promises a coordinate system. RuntimeFunction promises table entries from exception directory. UnwindSummary promises decoded frame semantics or explicit failure. HeuristicReport promises cautious static assessment. DynamicReport promises observation of one runtime session, not universal truth. ReportMetadata promises provenance. Reports promise explanation. Cleanup promises ownership discipline. Exit status promises automation can detect problems.

This is why main.c deserves detailed explanation. It is not the most mathematically interesting file. unwind.c has more Windows-specific decoding. dynamic_win.c has more debugger machinery. heuristics.c has more assessment logic. report.c has more serialization complexity. But main.c is where all layers meet. If main.c is sloppy, clean modules become unreliable. If main.c is disciplined, the project becomes a coherent analyzer rather than a bag of functions.

Переход к следующему уровню: базовый слой util.c

Теперь, когда общий pipeline понятен, следующий раздел должен спуститься к самому нижнему слою реализации: util.c. Это кажется менее эффектным, чем .pdata or Debug API, но именно там начинается доверие к анализатору. Before a PE parser can interpret headers, before unwind.c can decode UNWIND_INFO, before heuristics can count findings, the tool must read bytes safely, build strings safely, hash input and write output atomically.

Если 11.2 объяснил orchestration, 11.3 объяснит foundation. We will look at Buf, bounds-safe reads, little-endian helpers, StrBuf, printf-format annotations, SHA-256 and atomic file writes not as boring utilities, but as the reason the analyzer can survive malformed binaries and produce reports worth trusting. In reverse engineering tools, utility code is not background noise. It is the floor under every higher claim.

11.3. Базовый слой безопасного чтения: почему util.c держит на себе весь анализатор

Безопасное чтение начинается раньше, чем PE parser

После раздела о pipeline легко решить, что настоящая архитектура начинается в pe.c. Там действительно появляются знакомые для реверсера вещи: MZ, PE signature, COFF header, Optional Header, section table, RVA mapping and Exception Directory. Но это только первый видимый слой. До того как pe.c имеет право сказать "это PE32+ AMD64 image", кто-то должен выполнить более скучную, но фундаментальную работу: безопасно превратить файл на диске в ограниченный участок bytes and запретить всем последующим модулям читать за его пределами. В EH/Unwind Analyzer эту работу выполняет util.c вместе с util.h.

Название util не должно обманывать. В маленьких учебных проектах util часто превращается в свалку случайных helper functions. Здесь это не так. В проектной архитектуре util.c отвечает за bounds-safe byte reads, string builder, SHA-256 and file I/O. То есть за те вещи, которые обычно не выглядят как reverse engineering, но определяют, может ли reverse engineering вообще быть надежным. Если анализатор один раз прочитал четыре bytes за концом файла, то вся последующая теория про .pdata, .xdata, UNWIND_INFO, handler flags and assessment уже стоит на испорченном основании.

Static mode проекта не выполняет target code, но это не значит, что он работает с безопасным объектом. Входной файл остается untrusted byte array. Untrusted означает "недоверенный": программа не имеет права считать, что поля внутри файла честные, согласованные, полные and находятся там, где должны. Файл может быть урезанным. Может иметь e_lfanew, указывающий за конец buffer. Может объявить огромное число sections. Может положить Exception Directory в область, которая не мапится ни в одну секцию. Может сделать raw pointer plus raw size таким, что сумма переполнится. Может быть специально создан для fuzzing or атаки на parser. Поэтому первый вопрос не "какой здесь handler?", а "имею ли я право прочитать эти bytes?".

Официальная Microsoft PE Format documentation описывает, что такое PE/COFF image, section, RVA, VA, file pointer and data directories. Но документация описывает формат, а не проверяет конкретный файл за нас. Она говорит: в корректном образе поле должно иметь такой смысл. Она не говорит: твой analyzer уже доказал, что offset существует, размер не переполняется, section table помещается в file and RVA корректно переводится в file offset. Это различие важно. Official docs дают контракт. util.c дает механический способ не принять поврежденный input за выполненный контракт.

Поэтому 11.3 не является отступлением в бытовой C-код. Это глава о первом слое доверия. Все будущие утверждения analyzer - "найден RUNTIME_FUNCTION", "UNWIND_INFO parsed", "handler_rva points to this section", "report hash equals ..." - не появляются из воздуха. Они начинаются с маленькой дисциплины: каждый byte должен быть прочитан из допустимого диапазона, каждая строка diagnostics должна корректно расти or честно перейти в failed state, каждый report file должен быть полностью записан before replacing previous artifact, каждый input должен получить stable SHA-256 identity.

Buf: не файл и не PE, а ограниченная область bytes

В util.h базовая структура называется Buf:

```
typedef struct Buf {
    const uint8_t* data;
    size_t size;
} Buf;
```

Эта структура выглядит почти примитивно: указатель and размер. Но именно такая простота делает ее правильной. Buf не говорит "я PE". Buf не говорит "я executable". Buf не говорит "я trustworthy image". Buf говорит только: есть последовательность bytes, начинающаяся по адресу data, и ее допустимая длина равна size. Всякий смысл выше этого уровня должен быть получен позже and только после проверок.

Поле data имеет тип const uint8_t*. uint8_t - это byte, unsigned 8-bit integer. Для parser это важно: входной файл состоит не из символов, не из C strings and not from signed char values, а из raw bytes. const означает, что parser должен наблюдать input, а не менять его. Если pe.c or unwind.c случайно перепишет byte внутри buffer, то report уже не будет описывать исходный файл. SHA-256, diagnostics, offsets and manual validation начнут расходиться. const не защищает от всех ошибок C, но оно задает API contract: input buffer is read-only.

Поле size имеет тип size_t. Это естественный тип для размеров memory objects in C. Но size_t беззнаковый, поэтому с ним нельзя обращаться наивно. Если сложить offset and length без проверки, сумма может overflow and become small. Тогда условие может случайно пройти, а последующий memcpy прочитает за пределами buffer. Поэтому вся философия Buf не в том, что он хранит size, а в том, что все чтения должны использовать size правильно.

Buf также отделяет view from ownership. Сама структура не хранит флаг "я владею памятью". buf_init_from_file возвращает отдельный owned_mem, который later освобождается через buf_free_owned. Это не случайная неудобность. Это делает lifetime явным. Pe parser, unwind decoder and report metadata borrow Buf: они читают bytes but do not free them. main.c owns allocation through owned pointer. Такая модель хорошо совпадает с architecture document проекта: buf_init_from_file allocates input bytes; buf_free_owned releases them. В C явное владение часто важнее красивой краткости.

Загрузка файла: почему empty file тоже должен пройти как input

Функция buf_init_from_file открывает path in binary mode, ищет конец файла через fseek/ftell, возвращается к началу, выделяет память, читает ровно объявленный размер and заполняет Buf. Проект выбирает snapshot model: файл читается целиком, дальше analyzer работает с fixed byte buffer. Это проще для parser, fuzzing, SHA-256 and report reproducibility, чем ленивое чтение из FILE*.

Особенно важна строка:

```
size_t alloc_size = sz > 0 ? (size_t)sz : 1u;
```

Если файл пустой, размер равен zero. Но malloc(0) может вести себя по-разному на разных реализациях C runtime. Analyzer не хочет считать empty file ошибкой загрузки. Empty file is invalid PE, but it is still a valid input to parser in the sense that tool must reject it gracefully, not crash. Поэтому для нулевого файла выделяется 1 byte, но Buf.size остается 0. Читать этот byte нельзя, потому что size zero. Но owned pointer можно корректно освободить. Unit test test_empty_file_and_string_overflow проверяет это: empty file loads, buffer.size equals 0, owned pointer exists, cleanup works.

Это хороший пример engineering thinking. Ошибка формата и ошибка I/O - разные вещи. Если buf_init_from_file откажется от empty file как от I/O failure, pe_parse не сможет сформулировать диагностический отказ. Если же загрузка succeeded with size 0, pe_parse спокойно попробует прочитать MZ, buf_read вернет false, and analyzer завершит путь как invalid PE, not as filesystem failure.

В коде также есть практическое ограничение: размер берется через ftell, возвращающий long. Для обычных PE samples это нормально, но это не бесконечная файловая библиотека для гигантских blobs. Хорошая документация проекта не должна скрывать такие tradeoffs. util.c строит безопасный reader в рамках задач EH/Unwind Analyzer, а не универсальный storage engine.

После fread функция проверяет, что количество прочитанных bytes exactly равно размеру файла. Если прочитано меньше, memory освобождается, function returns false. Analyzer не продолжает с "почти прочитанным" файлом. Partial input разрушает identity: SHA-256 будет считаться от неполного byte stream, offsets могут указывать в отсутствующие regions, report будет выглядеть уверенно while input was incomplete. Поэтому failure здесь лучше, чем неопределенная половина анализа.

buf_read: главный запрет на чтение за границей

Самая важная функция util.c выглядит так:

```
bool buf_read(const Buf* b, size_t off, void* out, size_t len) {
```

```
    if (!b || !out) return false;
    if (off > b->size) return false;
    if (len > b->size - off) return false;
    memcpy(out, b->data + off, len);
    return true;
}
```

Здесь главное не memcpy, а форма проверки. Новичок часто написал бы условие так: if (off + len > size) return false. На бумаге оно выглядит правильно, но в unsigned arithmetic off + len может переполниться. Например, off близок к SIZE_MAX, len большой, сумма wraps around and становится маленькой. Проверка проходит, после чего memcpy читает memory outside buffer. Это классическая ошибка binary parsers.

util.c делает иначе. Сначала проверяется off > b->size. После этой проверки выражение b->size - off безопасно: underflow невозможен. Потом проверяется len > b->size - off. То есть код спрашивает не "какой конец диапазона после сложения?", а "помещается ли длина в оставшуюся часть buffer?". Это overflow-resistant range check.

Семантика off == size тоже правильная. Если off равен size and len равен 0, чтение пустого диапазона допустимо. Если off равен size and len больше 0, function returns false. Это соответствует математической модели buffer: можно указать позицию immediately after last byte as empty boundary, but cannot read a byte there.

buf_read копирует bytes в out, а не возвращает raw pointer into buffer. Это делает call sites аккуратнее: pe.c просит конкретное поле, получает bytes into local variable, then interprets them. Конечно, проект мог бы возвращать указатель for zero-copy parsing, но тогда every caller would repeat range checks and pointer arithmetic. Centralized copy is safer. Для полей 2/4/8 bytes стоимость negligible, а выигрыш в correctness большой.

Typed reads: как raw bytes становятся числами

Следующий слой - buf_read_u8, buf_read_u16, buf_read_u32 and buf_read_u64. Они не просто экономят несколько строк. Они задают правило: bytes становятся числом только после bounds check and explicit little-endian conversion.

Например, buf_read_u32 выглядит так:

```
bool buf_read_u32(const Buf* b, size_t off, uint32_t* out) {
    uint8_t tmp[4];
    if (!buf_read(b, off, tmp, 4)) return false;
    *out = le32(tmp);
    return true;
}
```

Сначала функция требует ровно четыре bytes. Если четыре bytes недоступны, она возвращает false and не делает вид, что partial field можно как-то достроить. Затем le32 собирает uint32_t вручную:

```
  return (uint32_t)(p[0] |
         ((uint32_t)p[1] << 8) |
         ((uint32_t)p[2] << 16) |
         ((uint32_t)p[3] << 24));
```

Почему нельзя просто сделать *(uint32_t*)(b->data + off)? Потому что это сразу смешивает file representation and host representation. Во-первых, raw pointer может быть unaligned. На x86/x64 это часто работает, но C language model and другие architectures не обязаны быть такими forgiving. Во-вторых, strict aliasing can become a problem. В-третьих, host endianness не обязан совпадать с file encoding. В-четвертых, при таком стиле bounds check легко забыть. Проект выбирает более длинный, но ясный путь: read bytes, then construct integer.

Это важный принцип binary analysis: file is not your struct. Файл - это bytes. C struct - это интерпретация в памяти host program. Между ними должен быть явный переход. util.c делает этот переход маленьким and repeatable. Поэтому pe.c не обязан знать, как правильно собрать uint32_t from four bytes; он просто вызывает buf_read_u32.

Typed reads также помогают сохранить различие between "field absent" and "field present but invalid". Если buf_read_u16(b, 0, &mz) fails, файл слишком короткий even for MZ. Если read succeeds but value is not MZ, bytes есть, но magic wrong. Эти ситуации не всегда одинаково диагностируются в текущем коде, но архитектура позволяет их различать. Это лучше, чем читать мусор and later получать случайные errors.

Как pe.c опирается на util.c

Когда pe.c начинает DOS header check, он читает MZ at offset 0 and e_lfanew at offset 0x3c через helpers. e_lfanew - это поле DOS header, которое указывает на PE header. В корректном PE оно ведет к NT signature. В malformed file оно может указывать куда угодно: за конец файла, внутрь середины другого поля, в область с недостаточным количеством bytes. Поэтому pe.c сначала получает число safely, а затем уже проверяет его смысл.

Дальше pattern повторяется. COFF header fields read through buf_read_u16/u32. Optional Header fields read through explicit offsets. Section headers read field by field. Parser не накладывает IMAGE_NT_HEADERS* на buffer and не доверяет compiler layout. Он каждый раз доказывает, что bytes существуют. Это особенно важно для PE, потому что official format содержит много offsets, sizes and counts, a counts controlled by file.

В Microsoft PE Format есть понятия RVA and file pointer. RVA - адрес элемента после загрузки image в память minus ImageBase. File pointer - позиция элемента on disk. В реальном parser надо постоянно переводить одно в другое. Но перевод RVA to file offset имеет смысл только после того, как section table safely read and validated. util.c не знает, что такое RVA. Но оно гарантирует, что section

table bytes, из которых pe.c строит эту карту, не были взяты за границей файла.

Можно представить это как алфавит и грамматику. util.c отвечает за алфавит: какие bytes вообще можно наблюдать. pe.c отвечает за грамматику PE: как из этих bytes складываются headers, sections and directories. Если алфавитный слой дырявый, грамматика будет объяснять не файл, а случайную память процесса.

StrBuf: diagnostics - это часть evidence, а не украшение

Второй большой блок util.c - StrBuf:

```
typedef struct StrBuf {
    char*  s;
    size_t len;
    size_t cap;
    bool failed;
} StrBuf;
```

StrBuf - динамический string builder. Он используется для parse errors, unwind operation dumps, heuristic details, dynamic details, CSV/JSON/HTML generation, invocation string and command-line construction. В таком инструменте diagnostic text is not decoration. Если parser отказался читать malformed file, analyst должен знать why. Если dynamic collector не смог поставить breakpoint, это должно попасть в details. Если report writer failed, это должно быть отличимо от "report says no findings".

Главное поле здесь - failed. Оно sticky: один раз став true, оно остается true. Это защищает от опасной ситуации, когда string building failed halfway, но код продолжает append and потом выдает partial text as complete. Для report generation это критично. Половина JSON хуже, чем явная ошибка. Половина HTML может выглядеть как полноценный report, но скрывать пропущенные блоки. Sticky failure превращает out-of-memory or formatting failure в состояние всего buffer.

sb_init создает buffer, выбирает default capacity 256 if initial_cap is zero, резервирует memory and ставит s[0] = 0. Это означает, что successful StrBuf всегда является valid C string. Но len хранит длину явно, чтобы не вычислять strlen каждый раз and чтобы append operations worked predictably.

sb_reserve растит capacity постепенно: newcap / 2 + 64. Конкретная формула не так важна, как проверки overflow. Перед увеличением capacity код проверяет, не переполнится ли addition. Если realloc fails, buffer enters failed state. Это важная деталь: allocation failure is not ignored and not converted into truncated output.

Append semantics: почему NULL and overflow являются ошибками

sb_appendn appends n bytes. Если sb уже failed, операция returns false. Если s is NULL while n is nonzero, buffer becomes failed. Если n is zero, NULL input допустим, потому что append empty range не читает memory. Это аккуратная семантика: она не маскирует caller bugs, но не запрещает математически пустую операцию.

Затем sb_appendn проверяет overflow:

```
if (sb->len == SIZE_MAX || n > SIZE_MAX - sb->len - 1u) {
    sb->failed = true;
    return false;
}
```

Здесь опять виден тот же принцип, что и в buf_read. Не надо вычислять sb->len + n + 1 без проверки, потому что сумма может wrap. Нужно сначала спросить: помещается ли n в оставшееся место до SIZE_MAX, учитывая trailing NUL. Если не помещается, операция fails before allocation. Это защищает от маленького allocation after integer overflow and последующего heap overflow.

После успешного reserve bytes копируются, len увеличивается, s[len] получает NUL terminator. То есть successful append сохраняет invariant: StrBuf одновременно length-tracked buffer and valid NUL-terminated string.

sb_append строже: он принимает обычную C string and rejects NULL. Если caller хочет append по явной длине, он должен использовать sb_appendn. Это разграничение полезно. Для text paths NULL обычно означает bug, а не empty string. Silent conversion NULL to empty would hide errors.

sb_appendf: форматированный текст без самообмана

sb_appendf делает formatted append через vsnprintf. Сначала он вызывает vsnprintf(NULL, 0, fmt, ap), чтобы узнать required length. Потом резервирует место. Потом вызывает vsnprintf again into buffer. Между этими вызовами используется va_copy, потому что va_list после первого use нельзя просто переиспользовать. Это маленькая, но важная C-портативность.

Функция проверяет need < 0, проверяет overflow, проверяет result of second vsnprintf. Если wrote != need, buffer enters failed state. Это строгий стиль. Он не говорит "ну почти получилось". Report text либо построен correctly, либо failed.

В util.h есть EHUA_PRINTF_FORMAT. На GCC/Clang он превращается в compiler attribute for printf-like format checking. На MinGW используется gnu_printf variant. На компиляторах без поддержки attribute он исчезает. Это дает дополнительный compile-time safety net: неправильный format string can be caught earlier. Для проекта, который строит много diagnostics and reports, это не мелочь. Один неверный % specifier может испортить output or crash.

Diagnostics as evidence: строка ошибки должна быть надежной

В EH/Unwind Analyzer diagnostic string часто объясняет границу знания. "Invalid PE signature" значит: parser не дошел до unwind metadata. "Exception directory malformed" значит: table cannot be treated as valid RUNTIME_FUNCTION array. "Dynamic session timed out" значит: absence of runtime events cannot be read as absence of behavior. Поэтому diagnostics are evidence about analysis process, not just text for user convenience.

StrBuf позволяет различать "диагностик нет" and "диагностику не удалось построить". sb_ok проверяет, что buffer exists, not failed, has s and len < cap. Если buffer failed, caller must not trust partial text. Unit test intentionally sets text.len = SIZE_MAX and verifies that append fails and sb_ok becomes false. Это тест не UI, a core reliability.

Хороший реверсер читает error messages как часть trail. Если report говорит "UNWIND_INFO parse failures: 7", важно знать, что эти failures не появились из random memory. Они появились из попыток безопасного чтения and structured decode. Если details отсутствуют because StrBuf failed, это тоже important limitation.

file_write_atomic: output reliability тоже часть analysis

Третий большой helper - file_write_atomic. Он пишет output through temporary sibling file and then replaces final path. Это касается CSV, JSON, HTML and other report artifacts. Сначала кажется, что это не reverse engineering. Но reports - это не просто красивый вывод. Это evidence artifacts, которые читают люди, scripts, validators and Ghidra importer.

Функция принимает path, data and len. Она rejects NULL path and rejects NULL data when len is nonzero. Затем строит temp path вида:

```
%s.tmp.%lu
```

где %s - final path, a %lu - process id. On Windows process id берется через GetCurrentProcessId, on POSIX - through getpid. Temp file находится рядом с final file. Это важно: replacement within same directory/file system обычно имеет более надежную semantics, чем write to random temp directory and cross-volume move.

Затем функция открывает temp file in binary mode, writes all bytes, checks fwrite count, checks ferror, calls fflush, closes file. Только если все это succeeded, она replaces final path. On Windows используется MoveFileExA with MOVEFILE_REPLACE_EXISTING | MOVEFILE_WRITE_THROUGH. Microsoft documentation describes MoveFileEx as moving existing file/directory with options; MOVEFILE_REPLACE_EXISTING allows replacing existing destination, and MOVEFILE_WRITE_THROUGH requests that the move be flushed before the function

returns. On non-Windows project uses rename.

Зачем не писать сразу в final path? Представь, что report.json already exists and is valid. Analyzer starts writing new JSON, then process crashes after half document. Direct write уничтожит старый report and оставит broken artifact. Atomic style сначала пишет complete temp file. If write fails, temp is removed and previous report remains. Only complete new artifact replaces old one. Это именно та семантика, которая нужна для analysis evidence.

Важно не преувеличивать. file_write_atomic is not a database transaction. Temp name based on process id is not a defense against hostile directory races. POSIX branch does not explicitly fsync directory. Windows branch uses MoveFileExA, not a full attribute-preserving ReplaceFile workflow. Но для обычного analyst workspace это огромный шаг по сравнению с direct overwrite. Этот helper protects against interrupted ordinary writes, not against every adversarial filesystem scenario.

Почему atomic reports важны для JSON schema, HTML and CI

В проекте есть JSON schema validation, report semantics tests, HTML escaping tests, dynamic validators and cross-field invariant checks. Все они предполагают, что report file is complete artifact. Если writer produces half JSON, validator sees syntax garbage, not meaningful semantic failure. Atomic write keeps failure modes cleaner: либо artifact полностью записан, либо write failed and previous artifact remains.

В CI это особенно важно. Suppose tests run analyzer over corpus samples and compare JSON fields. Interrupted write should not poison next comparison. Suppose human keeps previous HTML report. New failed run should not silently replace it with truncated page. file_write_atomic supports a larger rule: report is durable output of an analysis run, not disposable printf stream.

Это связано и с доверием пользователя. Если report says static_dynamic_correlation false, мы хотим знать, что это field реально было записано as part of complete report, not left over from broken serialization. Reliability is a chain. Atomic write is one link in that chain.

SHA-256: идентичность файла, а не магическая гарантия

sha256_hex computes SHA-256 digest of input bytes and returns 64 lowercase hex characters plus NUL. В reports это дает stable input identity. File name слабый identifier. Path can change. Same path can contain different bytes tomorrow. Different paths can contain same sample. SHA-256 lets analyst say: this report was produced for this exact byte sequence.

Implementation self-contained: Sha256Ctx stores state[8], bit_count, block[64] and block_len. sha256_update feeds bytes in 64-byte blocks. sha256_final appends

padding byte 0x80, zeros, bit count, then emits digest. sha256_hex converts each byte to two hexadecimal characters. Unit test checks known SHA-256 of "abc": ba7816bf8f01cfea414140de5dae2223b00361a396177a9cb410ff61f20015ad.

Проект не использует Windows CryptoAPI or bcrypt here. Это осознанный tradeoff: self-contained implementation работает in static tooling across platforms. It avoids platform crypto dependency for simple provenance hash. But this also means the implementation must be tested, because cryptographic-looking code is easy to get subtly wrong.

Нельзя читать SHA-256 как "файл безопасен". Hash не говорит, кто создал файл. Он не говорит, вредоносен ли файл. Он не подписывает report. Он только identifies bytes. В контексте EH/Unwind Analyzer этого достаточно: report metadata can tie findings, options and tool version to exact input bytes. If two reports have same path but different hash, they describe different inputs. If two reports have different paths but same hash, they describe same bytes.

Функция также checks out_hex and checks data when size > 0. Она checks size against UINT64_MAX / 8 before converting byte count to bit count. Это снова тот же мотив: before arithmetic becomes meaning, validate bounds.

Маленькие helpers превращаются в архитектурные правила

util.c кажется взаимозаменяемым, пока не посмотреть, как от него зависят остальные модули. pe.c shaped by buf_read. unwind.c shaped by safe byte access and StrBuf diagnostics. heuristics.c uses StrBuf for details and mitigation text. dynamic_win.c uses StrBuf for command line construction and event details. report.c uses StrBuf and file_write_atomic. main.c uses sha256_hex and StrBuf to build metadata. Маленький helper becomes architectural constraint.

Например, malformed Exception Directory claims 10 bytes. RUNTIME_FUNCTION entry on x64 is 12 bytes. unwind_read_runtime_functions_ex can return malformed status instead of reading partial entry because it has Buf and error buffer. Если decoder used raw pointer arithmetic, such case could become memory bug. If diagnostics were fixed-size arrays, message could truncate or overflow. util.c lets domain modules focus on domain rules, not repeat memory safety from scratch.

Другой пример - report.c. JSON, HTML and CSV all need escaping and incremental construction. If every writer had its own realloc logic, overflow handling would be duplicated and inconsistent. StrBuf centralizes this. Therefore report.c can concentrate on schema and presentation semantics.

Failure returns - это границы знания

Многие функции util.c return bool. false здесь не "позор" and не vague sadness. Это граница знания. buf_read false means requested bytes are unavailable or arguments invalid. sb_append false means output buffer no longer reliable. file_write_atomic false means final artifact was not safely written. sha256_hex false means digest was not produced under valid preconditions.

Верхние уровни должны уважать эти границы. If pe_parse ignores buf_read failure, it invents structure. If report_write_json ignores sb_append failure, it invents complete report. If main.c ignores file_write_atomic failure, it lies about output artifact. If metadata ignores sha256 failure, it loses input identity.

Такой стиль объясняет, почему main.c иногда continues after partial failure but sets exit_code. Tooling does not avoid failure; it records it. Malformed file should not crash. Dynamic run can fail. Report write can fail. The user and automation must see these states.

Trust boundary статического режима живет внутри buf_read

Architecture document says: static mode treats input as untrusted bytes and performs no code execution. На диаграмме это выглядит как высокий trust boundary. В коде он начинается в buf_read. Каждый раз, когда field from file wants to become value in analyzer, it must pass range check.

If PE field says optional header starts here, analyzer does not trust it until bytes exist. If section header says raw data begins at offset X with size Y, parser must check file bounds and overflow. If unwind info RVA maps to file offset, decoder must read through Buf. No field from input should become host pointer without validation.

Static mode safer than dynamic mode because it does not execute target code. Но static mode still processes adversarial data. Parser bugs are realistic attack surface. Поэтому fuzzing and bounds-safe reads are security work, not academic neatness.

Что значит bounds-safe именно здесь

Bounds-safe in this project does not mean "no bug possible". It means intended reads from input prove that requested byte interval lies inside Buf.size. It means size arithmetic avoids overflow. It means typed reads do not rely on unaligned loads or native endian. It means zero-length inputs are handled intentionally. It means functions return false instead of reading non-existing bytes.

But bounds-safe is only first layer. buf_read can tell you that four bytes exist at offset 0x3c. It cannot tell whether they form a reasonable e_lfanew. pe.c must decide. buf_read can read NumberOfSections. It cannot decide whether section

table semantic is acceptable. pe.c must decide. buf_read can read bytes for UNWIND_INFO. unwind.c must decide whether codes and flags are valid. This separation is healthy: util.c protects access; domain modules interpret.

Проход по hostile input

Imagine file has only two bytes: MZ. First magic read may succeed. Then pe.c needs e_lfanew at offset 0x3c. buf_read_u32 calls buf_read with off 0x3c and len 4. off > size, so false. No memory outside file is read. Parser reports failure.

Imagine file has e_lfanew, but e_lfanew points to file_size - 2. Reading PE signature requires four bytes. buf_read_u32 fails because len does not fit remaining bytes. Parser does not accidentally treat neighboring process memory as signature.

Imagine PE headers exist, but NumberOfSections huge. pe.c computes section header span and rejects out-of-range or overflow. util.c cannot know what section table means, but the parser uses the same arithmetic discipline: do not let count from file drive unchecked reads.

Imagine Exception Directory maps to file offset but size is 10. A RUNTIME_FUNCTION entry is 12 bytes, so decoder cannot form even one full record. It should return malformed status, not read partial structure. Again, input can claim structure; analyzer accepts only complete and checked structure.

Этот пример показывает: before discussing suspicious handler placement, analyzer must earn the right to read handler metadata safely.

Как StrBuf несет context through failure path

Boolean failure alone is not enough. User needs context. StrBuf lets pe_parse write diagnostic message, unwind decoder write malformed reason, heuristics write detail, dynamic collector write Windows error code. main.c initializes error buffer, passes it downward, prints it when stage fails, then frees it. There is no global error state.

Explicit diagnostics prevent lying. If unwind decoder encounters unsupported or malformed structure, report can show parse failure rather than silently treating entry as normal. If dynamic collector times out, DynamicReport can mark timed_out instead of saying "nothing happened". StrBuf is therefore part of epistemic honesty: it carries not only text, but the reason why confidence should stop.

Почему util.c не решает suspiciousness

util.c never says suspicious. It says readable or not readable, append succeeded or failed, hash computed or failed, file written or not. That is correct. Suspiciousness belongs to heuristics.c and assessment. A malformed file can be suspicious, accidental, truncated or unsupported. A failed output write says something about filesystem, not target. A hash identifies bytes, not intent.

This separation prevents category errors. If low-level helper starts deciding malware-like meaning, architecture blurs. util.c should stay boring and strict. In security tooling, boring foundation is a virtue.

Portability: Windows target, cross-platform static tooling

EH/Unwind Analyzer studies Windows x64 PE/unwind semantics, but static parsing and tests can run outside Windows. util.c reflects this. It uses standard C for Buf, StrBuf and SHA-256. It includes windows.h only under _WIN32 for GetCurrentProcessId and MoveFileExA. On POSIX branch it uses getpid and rename.

This matters because dynamic_win.c is Windows-only, while pe.c/unwind.c/heuristics.c/report.c are useful for static analysis and CI elsewhere. If util.c depended on Windows APIs everywhere, fuzzing on non-Windows platforms would be harder. If util.c ignored Windows replacement semantics, actual Windows reports would be weaker. The project chooses portable core plus platform-specific file replacement where needed.

Связь util.c with fuzzing and unit tests

Mutation fuzzing generates weird inputs: tiny files, huge counts, invalid RVA, section overlaps, truncated unwind records. Fuzzing is useful only if parser treats file as hostile. util.c makes that possible: malformed inputs should produce false returns and diagnostics, not crashes.

The fuzz_parser test feeds arbitrary bytes into Buf and then tries pe_parse, unwind_read_runtime_functions and heuristics on what survives. It is not looking for meaningful PE in random bytes. It is looking for stability: no out-of-bounds reads, no unbounded loops, no leaks on early failure paths.

Unit tests hit the foundation directly. test_sha256 verifies known digest. test_empty_file_and_string_overflow verifies empty input handling, sticky StrBuf failure and atomic write replacement. These tests are small, but they guard functions that every other module uses.

Ownership: кто что освобождает

main.c uses Buf like this:

```
Buf b;
uint8_t* owned = NULL;
if (!buf_init_from_file(&b, a.input, &owned)) { ... }
...
buf_free_owned(owned);
```

This says: b is view, owned is allocation handle. pe_parse borrows b. unwind decoder borrows b. report metadata hashes b. Nobody except main.c frees owned. This explicit ownership avoids double free and dangling pointer confusion.

Same pattern appears elsewhere. sb_init allocates, sb_free releases. heuristics_analyze initializes internal report details, heuristics_free releases. dynamic_report_init owns details, dynamic_report_free releases. Report writers borrow model objects and do not retain pointers. This is why architecture document can state ownership rules clearly.

Could Buf own memory directly? Yes, but tests often create Buf over stack arrays. Then ownership flag would be needed. Current design is more explicit: view and allocation are separate. It is slightly verbose, but easier to reason about under fuzzing and early failures.

Почему zero-copy здесь не цель

A performance-focused parser might return pointers into input buffer for every field. EH/Unwind Analyzer instead copies small integer fields into tmp arrays and converts them. Reading two, four or eight bytes is cheap. Correctness is more valuable than saving a tiny copy.

Zero-copy can be useful for large blobs, but for structured PE metadata it often tempts developer to cast file bytes to C structs. That creates alignment, padding, endian and bounds risks. Project chooses explicit field reads. This is slower only in a theoretical sense and safer in the real sense that matters for malformed inputs.

Report generation требует той же дисциплины, что parsing

Parsing consumes bytes; reporting produces bytes. Both need strict boundaries. Parser must not read outside input. Reporter must not output malformed, unescaped or half-written artifacts. StrBuf and file_write_atomic are output-side analogs of Buf and buf_read.

CSV needs quoting. JSON needs escaping. HTML needs escaping and restrictive offline CSP. Ghidra CSV needs stable columns. If string builder fails halfway, writer should fail whole artifact. If file replacement fails, analyzer should not pretend output exists. Therefore util.c supports report honesty just as much as parser safety.

Redaction and invocation strings

main.c builds invocation metadata and redacts target arguments after separator --. Dynamic validation needs child args to run scenario, but report metadata should not leak secrets or local paths unnecessarily. StrBuf enables safe construction of this redacted command line.

This is not just privacy polish. Metadata belongs to report provenance. If invocation string is incomplete because StrBuf failed, report metadata is less reliable. Again, low-level string building supports high-level trust.

Что util.c намеренно оставляет другим modules

util.c does not validate PE machine type. pe.c does. It does not know SectionAlignment. pe.c does. It does not know UNWIND_CODE semantics. unwind.c does. It does not decide whether handler in writable section is suspicious. heuristics.c does. It does not correlate runtime events with static findings. assessment layer does.

This boundary is important. A utility layer that starts accumulating domain decisions becomes a hidden second analyzer. EH/Unwind Analyzer keeps util.c as memory/bytes/text/file/hash layer. That makes higher modules easier to inspect.

Частая ошибка: safe parser будто бы означает safe result

Safe byte reading prevents memory unsafety and crashes. It does not make conclusions automatically correct. Analyzer can safely read a malformed field and still need domain judgment. It can safely reject RVA and still need to explain whether this is suspicious, unsupported or corrupted. It can safely write report and still be limited by static/dynamic coverage.

So "the tool did not crash" is not a verdict. "JSON exists" is not a verdict. "SHA-256 exists" is not a verdict. util.c gives local guarantees. Assessment must remain conservative.

Как читать util.c глазами реверсера

C programmer sees file reader, string buffer, atomic writer, hash. Reverse engineer should see evidence hygiene. util.c defines which bytes can become observations, which diagnostics can be trusted, which output files can be treated as complete, and which input identity attaches to result.

When later section says "Exception Directory RVA is 0x3000", that statement secretly includes util.c: bytes for data directory were read within bounds, decoded explicitly, then interpreted by pe.c. When report says "input_sha256", that value came from sha256_hex over loaded Buf. When dynamic report details contain error lines, they live in StrBuf. High-level evidence carries low-level guarantees.

Miniature end-to-end path through util.c

Ordinary run starts with buf_init_from_file: open target.exe, determine size, allocate, read exactly size bytes. Then pe_parse uses buf_read helpers for headers. unwind.c uses same Buf for RUNTIME_FUNCTION and UNWIND_INFO bytes. If JSON/HTML requested, sha256_hex computes input hash. report.c builds output in StrBuf. file_write_atomic writes complete artifact through temp file. main.c frees owned memory and other owned structures.

This path is quiet because many edge cases have been absorbed: empty files, short reads, overflow checks, output failures, string allocation failures. Good

foundational code should often be invisible during successful runs.

Miniature failure path

If user passes tiny.bin, buf_init_from_file may succeed. pe_parse asks for MZ and e_lfanew. A read fails. Parser appends diagnostic. main.c prints error, frees memory, exits nonzero. It does not start unwind decode. It does not run heuristics. It does not invent suspiciousness.

If parsing succeeds but JSON output path is unwritable, report.c may build JSON but file_write_atomic returns false. main.c sets exit_code. This is not a finding about target. It is output failure. Good architecture keeps these categories separate.

Почему этот раздел должен стоять перед pe.c

Next section will enter pe.c: DOS header, NT headers, PE32+ AMD64, section table, RVA mapping, Exception Directory. But pe.c stands on util.c. Without Buf and bounds checks, PE parsing would be unsafe. Without StrBuf, diagnostics would be unreliable. Without sha256_hex, reports would lose stable input identity. Without file_write_atomic, report artifacts could be truncated.

Поэтому 11.3 is foundation of project chapter. Before EH/Unwind Analyzer can be clever, it must be disciplined. Before it can discuss .pdata anomalies, it must read bytes safely. Before it can write assessment, it must preserve diagnostics. Before it can integrate with Ghidra, it must create complete files.

Почему проект читает whole file, а не работает напрямую с FILE*

Можно было бы сделать parser, который постоянно делает fseek/fread по мере необходимости. На первый взгляд это экономит память. Но для EH/Unwind Analyzer важнее другое: единый immutable snapshot входа. Когда файл загружен в Buf, все modules видят одну и ту же byte sequence. pe.c, unwind.c, heuristics, report metadata and SHA-256 all refer to same loaded object. Если читать FILE* много раз, появляется больше states: file position, partial read errors, file changed on disk between reads, inconsistent hashing. Для forensic-style tooling this is undesirable.

Whole-file buffer also simplifies RVA mapping. pe.c can store file_size in PeImage and later ask whether raw_ptr/raw_size lies within this known size. unwind.c can translate RVA to file offset and immediately read from Buf. Fuzzing can feed arbitrary in-memory bytes without creating real files. Unit tests can create Buf over stack arrays. Это делает parser easier to test. Memory cost is acceptable because target artifact is PE executable, not arbitrary disk image.

There is still a tradeoff. Very huge files can fail to load or exceed ftell/size assumptions. But that failure is explicit: buf_init_from_file returns false. The project chooses explicit failure over complex streaming parser with more moving parts. For

this analyzer, that is a reasonable engineering choice.

Как util.c помогает сохранять различие между parse error and model error

В binary analysis есть несколько разных классов ошибок. I/O error means file could not be loaded. Bounds error means requested bytes do not exist. Format error means bytes exist but do not match expected structure. Semantic model error means structure exists but violates project assumptions, for example unsupported architecture or invalid unwind relation. Report error means analysis may exist but artifact was not written. Runtime error means dynamic validation did not run or did not finish.

Если базовый слой не аккуратен, все эти ошибки смешиваются. Direct cast за пределами buffer может crash, и analyst увидит только process termination. Silent truncation in StrBuf can turn report error into false clean report. Direct report overwrite can turn output failure into corrupted evidence. util.c помогает держать классы отдельно: false from buf_init_from_file is loading failure; false from buf_read is unavailable bytes; failed StrBuf is unreliable text; false from file_write_atomic is output failure. Higher modules add domain names on top.

This separation is visible in main.c. It creates different StrBuf objects for parse error, unwind error, heuristic error, dynamic error and report error. It can print different messages and set exit_code accordingly. That is not accidental verbosity; that is taxonomy of failure. A reverse engineering report is more useful when it says not only what was found, but also where analysis stopped.

Как pe.c должен выглядеть в голове после util.c

Когда мы перейдем к pe.c, не надо читать его как набор magic offsets. Нужно читать его как series of controlled observations. Offset 0 gives DOS magic only if two bytes exist. Offset 0x3c gives e_lfanew only if four bytes exist. e_lfanew becomes candidate PE header location only after range checks. COFF header becomes readable only after NT signature region is available. Optional Header fields become meaningful only after SizeOfOptionalHeader and magic are checked. Section table becomes model only after its total span fits file.

То есть pe.c does not "trust PE". It slowly negotiates with bytes. Every time file claims something, parser asks util.c whether bytes for this claim can be observed. Then pe.c asks whether observed value fits PE contract. This two-step discipline prevents both memory bugs and semantic overclaiming.

This is also why report should never simply say "parser failed" if more detail exists. A failure at MZ, at PE signature, at optional header size, at section table span and at Exception Directory are different situations. They lead to different manual validation steps. util.c makes such distinctions possible by letting each stage fail at exact read boundary.

How safe reading affects heuristic quality

Heuristics depend on counters and normalized facts. If the parser misreads section characteristics because of out-of-bounds or endian issue, heuristic engine may flag writable/executable placement incorrectly. If RuntimeFunction table count is wrong because exception directory size was read from wrong offset, handler dominance and unwind reuse signals become nonsense. If UNWIND_INFO parse uses garbage bytes, score and confidence become meaningless.

Therefore util.c indirectly affects heuristic quality. It does not know score, evidence_families_mask or weakening_rules_applied. But it protects the raw observations from which these later values are built. A conservative heuristic engine is only as conservative as its input model. Safe reading reduces the chance that malformed bytes become confident findings.

This also explains why parse failures are themselves reportable. If analyzer cannot decode UNWIND_INFO safely, that is not the same as "normal function". It is a limitation or anomaly depending on context. util.c provides the false return that later becomes us_ok false, parse failure count, finding, mitigation or manual review condition. The path from low-level false to high-level assessment must stay visible.

How safe writing affects trust in HTML and JSON

When report.c writes JSON, it is creating machine-readable contract. Tests and external tools can parse it, compare schema version, inspect heuristic_analysis, dynamic_validation and assessment. If JSON is incomplete, downstream automation may fail or, worse, read stale previous content. file_write_atomic reduces that risk.

HTML has a different risk. It is human-readable and may include arbitrary metadata strings, paths, diagnostics and details derived from input or command line. report.c must escape them; StrBuf must not silently truncate them; file_write_atomic must not leave half page as final artifact. Otherwise analyst may read misleading visual report. This is why HTML escaping tests and no-remote-resource CSP matter, but they still rely on util.c's string and file-writing primitives.

CSV and Ghidra CSV are similar. A broken quote can shift columns. A shifted column can put comment into wrong address row. In Ghidra workflow that can pollute workspace. util.c cannot know CSV semantics, but StrBuf and atomic write give report.c reliable construction substrate.

What util.c does not protect against

Detailed explanation should include boundaries. util.c does not make C memory safe globally. Other modules can still make mistakes. If some future code bypasses buf_read and directly indexes b->data, util.c cannot save it. If report.c escapes JSON incorrectly, StrBuf will happily store incorrect escaped text because escaping semantics are above its layer. If user writes output into hostile directory with symlink races, file_write_atomic is not complete defense. If SHA-256 implementation had subtle bug, self-contained nature would not magically fix it.

This is not criticism; it is scope. A good utility layer defines local guarantees. Project discipline means other modules should use it consistently and tests should catch violations. Future contributors should understand: adding a direct pointer cast in pe.c is not an optimization until proven safe; it is a breach of project style. Adding fixed-size sprintf in report.c is not quick; it bypasses StrBuf's failure discipline. Writing reports directly to final path bypasses atomic artifact semantics.

How to review changes to util.c

Because util.c is foundational, code review should treat changes to it as high-impact. If someone modifies buf_read, reviewer should ask: does off/len arithmetic still avoid overflow? Does zero-length read at end still work? Does NULL behavior remain intentional? If someone modifies StrBuf growth, reviewer should ask: can capacity calculation overflow? Is failed sticky? Are all successful appends NUL-terminated? If someone modifies file_write_atomic, reviewer should ask: can interrupted write destroy previous artifact? Does Windows branch still replace existing output? Does POSIX branch still write sibling temp file?

If someone modifies sha256_hex, reviewer should ask: are bit counts correct? Is padding correct? Is output always 64 hex chars plus NUL? Does empty input work? Are test vectors present? These questions sound low-level, but they protect every higher-level report.

In a serious reverse engineering tool, foundational helpers should be boring because they are reviewed deeply. Cleverness belongs in explanations and models, not in unchecked pointer arithmetic.

Закрепление: контракт util.c в одной mental model

Buf means: here are bytes and maximum valid range. buf_read means: no byte becomes evidence without bounds proof. le16/le32/le64 mean: file encoding becomes host integer explicitly. StrBuf means: diagnostic and report text grow with sticky failure state. file_write_atomic means: complete report replaces old report only after successful write. sha256_hex means: report names exact input bytes.

Вместе эти функции не анализируют exception handling напрямую. Они делают такой анализ возможным. Они не знают, что такое .pdata, но не дают чтению .pdata превратиться в out-of-bounds memory access. Они не знают, что

такое suspicious handler, но позволяют heuristics описать его без повреждения details. Они не знают, что такое Ghidra, но позволяют report.c безопасно записать CSV. Они не знают, что такое assessment, но защищают evidence, которое assessment потом будет суммировать.

Именно так надо читать utility code в серьезном reverse engineering project: не как скучный boilerplate и не как магию, а как первый слой epistemic hygiene. Он решает, какие bytes можно наблюдать, каким strings можно доверять, какие files можно считать записанными и какая input identity прикреплена к результату.

11.4. PE parser: как pe.c превращает недоверенные bytes в карту Windows image

Почему PE parser в этом проекте не является обычным reader of headers

После util.c естественно перейти к pe.c. На первый взгляд это просто следующий технический слой: util.c безопасно читает bytes, а pe.c читает PE headers. Но для EH/Unwind Analyzer pe.c выполняет более важную роль. Он не просто достает из файла несколько чисел. Он строит карту, на которую дальше опираются unwind decoder, heuristic evidence engine, dynamic collector and reporting layer. Если эта карта ошибочна, то все последующие выводы будут выглядеть убедительно, но будут относиться не к реальному Windows image, а к неправильной интерпретации file bytes.

Portable Executable, или PE, - это формат Windows images: executable files, DLL, system binaries and other image-like modules. Слово portable здесь историческое: формат должен был быть переносимым между разными machine types and Windows variants. Для реверсера PE - это не просто контейнер. Это договор между linker, loader, operating system and tools. Linker кладет в файл headers, sections, data directories and metadata. Loader читает эти структуры, создает image в virtual address space process, применяет relocations, подключает imports and делает sections доступными с нужными protections. Unwind machinery later использует exception directory and function unwind information. Debugger and static analyzer используют ту же карту, чтобы понимать, где code, где data, где metadata, где RVA turns into file offset.

В официальной Microsoft PE Format documentation важны несколько базовых различий. RVA, relative virtual address, - это адрес элемента после загрузки image minus image base. VA, virtual address, - это address in process virtual address space. File pointer, или file offset, - это позиция в файле на диске. Эти три координатные системы похожи только на поверхности. Новичок часто смотрит на число 0x140001000 and думает, что это "адрес в файле". Но file offset 0x1000, RVA 0x1000 and VA 0x140001000 are three different coordinates. pe.c нужен именно потому, что analyzer должен переходить между этими координатами не на глаз, а по section table and PE rules.

В книге мы уже много раз возвращались к цепочке function -> call -> return address -> RSP -> stack frame -> nonvolatile registers -> .pdata -> RUNTIME_FUNCTION -> .xdata -> UNWIND_INFO -> RtlVirtualUnwind -> handler route. pe.c отвечает за первую часть этой цепочки, которая обычно остается невидимой: как analyzer вообще находит official Exception Directory, как понимает section layout, как переводит RVA exception metadata в file offset, как

отличает нормально отсутствующий directory from directory that points outside image, как сообщает downstream modules, что section table itself suspicious. Пока pe.c не создал PeImage, unwind.c не должен делать вид, что он знает, где лежит .pdata.

Здесь важно слово official. В PE-файле может быть section named .pdata, but parser не должен начинать с имени section. Windows contract для directory-based metadata проходит через Optional Header DataDirectory. Для exception handling на x64 relevant entry is IMAGE_DIRECTORY_ENTRY_EXCEPTION, index 3. Именно эта запись говорит: вот RVA and size таблицы exception information. Секция .pdata часто содержит этот диапазон, но имя section - это convention, not authority. Если analyzer ищет "section named .pdata" before reading Exception Directory, он путает удобную традицию with official entry point. pe.c в проекте следует правильной идее: сначала PE headers and DataDirectory, потом mapping.

Поэтому этот раздел не будет "MZ, PE, sections" как карточка. Мы разберем pe.c как слой нормализации. Raw bytes входят в parser. На выходе появляется PeImage: аккуратная структура с базовыми facts, section list, exception directory coordinates and structural counters. Эти counters потом становятся evidence for heuristic engine. Они не говорят "malware". Они говорят: модель PE image имеет нарушения, которые должны ослаблять or усиливать доверие к дальнейшему анализу.

PeImage: не копия Windows headers, а нормализованная модель для анализа

В include/pe.h project defines PeImage. Это не IMAGE_NT_HEADERS64 from Windows SDK and not a full PE library object. Это deliberately small model: только те поля, которые нужны EH/Unwind Analyzer for exception/unwind analysis and reporting. Такой выбор важен. Чем больше parser обещает прочитать, тем больше attack surface and больше мест, где malformed file может сломать tool. Проект не пытается стать универсальным pefile clone. Он строит enough PE model to answer one family of questions: где image boundaries, где sections, как RVA maps to bytes, где Exception Directory, and какие structural anomalies should be visible to heuristics.

Ключевые поля PeImage выглядят так:

```
typedef struct PeImage {
    bool     ok;
    bool     is_pe64;
    uint16_t machine;
    uint64_t image_base;
    uint32_t size_of_image;
    uint32_t size_of_headers;
    uint32_t section_alignment;
    uint32_t file_alignment;
    uint32_t exception_rva;
```

```
    uint32_t exception_size;
    size_t   file_size;
    size_t   num_sections;
    PeSection* sections;

    size_t section_raw_oob;
    size_t section_virtual_oob;
    size_t section_raw_overlaps;
    size_t section_virtual_overlaps;
    size_t section_misaligned;
    bool   exception_directory_oob;
} PeImage;
```

Поле ok означает, что parser смог пройти mandatory structural layer. Это не означает "файл хороший". Это значит: достаточно базовых PE32+ AMD64 facts было прочитано safely, and PeImage can be used by later modules. Такой смысл ok надо держать в голове. Invalid signature, unsupported architecture or absent mandatory header stops parser. Но section misalignment or exception directory out-of-bounds фиксируется as diagnostic counters inside PeImage, потому что analyzer может still report these anomalies. Это различие между fatal parse error and structural suspicion.

Поле is_pe64 фиксирует, что Optional Header magic corresponds to PE32+. В Windows naming PE32+ means 64-bit image optional header form. Слово optional здесь historically unfortunate. Optional Header is optional only in COFF object-file sense. Для executable image it is essential. В нем находятся ImageBase, SectionAlignment, FileAlignment, SizeOfImage, SizeOfHeaders and DataDirectory array. Without it loader cannot construct image in memory, and EH/Unwind Analyzer cannot find Exception Directory. Поэтому pe.c rejects magic other than 0x20b.

machine хранит COFF Machine field. Project accepts only IMAGE_FILE_MACHINE_AMD64, value 0x8664. Это не снобизм and not arbitrary limitation. Вся книга до этого строилась вокруг Windows x64 exception handling. На x86 SEH устроен иначе, stack walking имеет другую историческую модель, and .pdata/.xdata are not the same universal mechanism. Если tool quietly accepted x86, downstream unwind.c мог бы прочитать unrelated bytes as RUNTIME_FUNCTION and produce nonsense. Поэтому parser stops early: unsupported Machine is fatal. Good analyzer should not pretend to know architecture it was not designed for.

image_base - preferred ImageBase from Optional Header. Preferred значит: linker says "я бы хотел, чтобы image был loaded starting here". Loader can use another runtime base due ASLR or conflicts. Static mode uses image_base as metadata fact, not as guaranteed runtime address. Dynamic mode later observes actual module base. В report and Ghidra integration надо различать preferred image base from loaded module base. pe.c stores preferred image_base because it belongs to file contract.

size_of_image describes virtual span of image after loading. It is not file size. File may be smaller because sections can have virtual zero-fill tails. It may contain overlay data after last section. size_of_headers tells how much of the beginning of file maps as headers. section_alignment and file_alignment define how sections are arranged in memory and in file. exception_rva and exception_size come from DataDirectory[IMAGE_DIRECTORY_ENTRY_EXCEPTION]. file_size comes from Buf.size, so reports can compare file layout with header claims.

sections is owned dynamic array of PeSection. Each PeSection is normalized to name, virtual_address, virtual_size, raw_ptr, raw_size and characteristics. Again, это not full IMAGE_SECTION_HEADER. Project stores what it needs. PointerToRelocations, NumberOfRelocations, line numbers and other COFF-era fields are irrelevant for this analyzer. The project deliberately avoids carrying unused complexity.

The bottom counters are not parse garbage. They are explicit observation fields. section_raw_oob means one or more sections claim raw bytes outside file. section_virtual_oob means virtual span exceeds SizeOfImage. section_raw_overlaps and section_virtual_overlaps report overlapping ranges. section_misaligned gathers alignment and ordering inconsistencies. exception_directory_oob says Exception Directory exists as declared coordinates but lies outside image boundaries or is degenerate. These fields later feed table_integrity and metadata_placement evidence families. They are the bridge between low-level PE parse and heuristic assessment.

DOS header: почему parser начинает с MZ, но не верит DOS stub

pe_parse begins with memset(out, 0, sizeof(*out)). This is small but important. If parse fails halfway, output structure should not contain old pointers or stale facts. In C, stale structure state can create double free, false report fields or accidental downstream use. Zeroing output is the first defensive move.

Then code calls read_dos_lfanew. DOS header is the historical beginning of PE file. The magic value MZ, 0x5A4D, identifies an MS-DOS compatible header. PE files keep this for compatibility. At offset 0x3c inside DOS header there is e_lfanew - an offset to PE header. Microsoft PE Format documentation describes PE header after MS-DOS stub; e_lfanew is the bridge from old DOS-compatible beginning to the real PE header.

The project code says:

```
static bool read_dos_lfanew(const Buf* b, uint32_t* out_lfanew) {
    uint16_t mz = 0;
    if (!buf_read_u16(b, 0, &mz)) return false;
    if (mz != PE_DOS_MAGIC) return false;
    uint32_t lfanew = 0;
    if (!buf_read_u32(b, 0x3c, &lfanew)) return false;
    *out_lfanew = lfanew;
```

```
    return true;
}
```

This is intentionally minimal. Parser does not attempt to interpret DOS stub program. It does not care what text stub prints. It does not require classic "This program cannot be run in DOS mode". It only needs two facts: first two bytes are MZ, and e_lfanew exists as 32-bit value. That is enough to continue to PE signature. This is correct for the tool purpose. EH/Unwind Analyzer is not a DOS stub analyzer.

But the parser also does not trust e_lfanew blindly. Immediately after reading it, pe_parse checks:

```
if ((size_t)lfanew + 4 + 20 > b->size) {
    sb_append(err, "PE header offset out of range.\n");
    return false;
}
```

Why 4 + 20? Four bytes are PE signature, and 20 bytes are COFF File Header. If e_lfanew points so close to end of file that even signature plus COFF header cannot fit, then parser cannot continue. This check also turns a common malformed-file trick into a clean diagnostic. A damaged file can keep MZ valid and set e_lfanew to 0xffffffff. Without range check, any parser that uses e_lfanew as pointer would crash or read unrelated memory. Here it returns "PE header offset out of range".

There is a subtle arithmetic issue here. lfanew is uint32_t, cast to size_t before addition. On 64-bit platform this is safe for 32-bit lfanew plus small constants. In a more general parser one might prefer a helper for all size_t additions, but within this exact bound the addition cannot overflow size_t on the intended build. This illustrates a pattern: pe.c uses explicit checks where controlled counts and sizes can overflow, for example section header size calculation and RVA additions.

If MZ fails, diagnostic says "Not a valid MZ/PE file (DOS header check failed)." It does not say "malicious". A text file, empty file or Linux ELF are simply not PE. The parser's job at this layer is classification by format, not suspicion.

PE signature and COFF header: от historical shell к machine contract

After e_lfanew check, pe.c reads NT signature:

```
uint32_t sig = 0;
if (!buf_read_u32(b, (size_t)lfanew, &sig) || sig != PE_NT_MAGIC) {
    sb_append(err, "Invalid PE signature.\n");
    return false;
}
```

PE_NT_MAGIC is 0x00004550, which corresponds to bytes "PE\0\0" in little-endian file representation. This signature marks the real beginning of PE headers. It is tempting to say "now we know this is PE". More accurately, we know the file has correct DOS bridge and signature. The rest still may be malformed.

COFF File Header begins immediately after signature. pe.c reads Machine, NumberOfSections and SizeOfOptionalHeader:

```
size_t filehdr = (size_t)lfanew + 4;
uint16_t machine = 0;
uint16_t nsects = 0;
uint16_t opt_size = 0;
if (!buf_read_u16(b, filehdr + 0, &machine) ||
    !buf_read_u16(b, filehdr + 2, &nsects) ||
    !buf_read_u16(b, filehdr + 16, &opt_size)) {
    sb_append(err, "Failed to read COFF header.\n");
    return false;
}
```

COFF, Common Object File Format, is older than PE, and PE reuses COFF ideas. Machine tells for which architecture this image is built. NumberOfSections controls how many IMAGE_SECTION_HEADER entries follow Optional Header. SizeOfOptionalHeader tells parser where section table begins. Notice: pe.c does not parse timestamps, symbol table pointer, symbol count or characteristics here. They are not needed for EH/unwind analysis. Minimalism here reduces the chance of building false interpretations from fields unrelated to project's goal.

The Machine check is strict:

```
if (machine != IMAGE_FILE_MACHINE_AMD64) {
    sb_appendf(err,
        "Unsupported Machine: 0x%04x (only x64/AMD64 supported).\n", machine);
    return false;
}
```

This check is an architectural boundary. All later modules assume Windows x64 conventions: RUNTIME_FUNCTION entries are 12 bytes with BeginAddress, EndAddress and UnwindData as image-relative values; UNWIND_INFO describes prolog effects; RtlVirtualUnwind can reconstruct caller context; exception directory is relevant for stack unwinding. On other architectures, the existence, layout or meaning of exception metadata can differ. A defensive analyzer should reject unsupported machine rather than degrade silently.

NumberOfSections is not rejected if zero before allocation. Later code allocates at least one entry if nsects is zero but sets num_sections to zero. A PE image with zero sections may be unusual for executable analysis, but parser can still report facts about headers and exception directory. This is a design choice: not every unusual structure is fatal at PE parse layer. Some should become evidence.

SizeOfOptionalHeader drives opthdr + opt_size. If Optional Header lies outside file, parser stops. If it is too small for fields pe.c needs, parser stops. This matters because PE optional header has different layouts for PE32 and PE32+. Without enough bytes, reading ImageBase or NumberOfRvaAndSizes would be guessing.

Optional Header: почему "optional" здесь почти ложное слово

PE optional header begins at filehdr + 20. Project checks:

```
if (opthdr + opt_size > b->size) {
    sb_append(err, "Optional header out of range.\n");
    return false;
}
if (opt_size < 0x70u) {
    sb_append(err,
        "PE32+ optional header is too small for required fields.\n");
    return false;
}
```

The 0x70 threshold is chosen because DataDirectory begins at offset 0x70 in PE32+ optional header. The analyzer needs NumberOfRvaAndSizes at 0x6c and DataDirectory entry 3 beginning at 0x70 + 3*8. A full DataDirectory[3] check happens later, but this early threshold ensures mandatory fields before directory array can be read.

Then parser checks OptionalHeader.Magic:

```
uint16_t magic = 0;
if (!buf_read_u16(b, opthdr + 0, &magic)) { ... }
if (magic != IMAGE_NT_OPTIONAL_HDR64_MAGIC) {
    sb_appendf(err,
        "Unsupported OptionalHeader magic: 0x%04x (expect PE32+ 0x20b).\n", magic);
    return false;
}
```

Magic 0x20b means PE32+. This is the 64-bit PE optional header format. PE32 is 0x10b and has different field layout: ImageBase is 32-bit and offsets differ. If project accepted PE32 magic, offsets like opthdr + 0x18 for 64-bit ImageBase would not mean the same thing. Again, strict failure is a correctness decision, not limitation to hide.

Project then reads selected PE32+ fields by explicit offsets:

```
if (!buf_read_u64(b, opthdr + 0x18, &image_base) ||
    !buf_read_u32(b, opthdr + 0x20, &section_alignment) ||
    !buf_read_u32(b, opthdr + 0x24, &file_alignment) ||
    !buf_read_u32(b, opthdr + 0x38, &size_of_image) ||
    !buf_read_u32(b, opthdr + 0x3c, &size_of_headers) ||
    !buf_read_u32(b, opthdr + 0x6c, &number_of_rva_and_sizes)) {
    sb_append(err, "Failed to read key fields from optional header.\n");
    return false;
}
```

This is a parser written without Windows headers. pe.h explicitly says "minimal PE32+/x64 parser (no Windows headers)". That means project defines only constants it needs and reads fields with known offsets. This has portability benefits: the tool can build with MinGW and without depending on SDK struct packing. It also avoids accidental reliance on compiler struct layout. But it increases responsibility: offsets must match official PE32+ layout. That is why official documentation matters. The

project is not inventing these offsets; it follows PE/COFF layout.

SizeOfImage must be nonzero. An image with zero virtual size cannot define meaningful RVA space. SizeOfHeaders must be nonzero and must not exceed file size. This does not guarantee headers are semantically perfect, but it prevents mapping header RVAs to non-existent file bytes. SectionAlignment and FileAlignment must be nonzero. Division and modulo checks later depend on nonzero alignment, but more importantly, a zero alignment destroys the loader's section layout model.

NumberOfRvaAndSizes is checked before reading Exception Directory:

```
if (number_of_rva_and_sizes < IMAGE_DIRECTORY_ENTRY_EXCEPTION + 1) {
    sb_append(err, "DataDirectory too small.\n");
    return false;
}
```

The official Optional Header contains NumberOfRvaAndSizes, and each directory entry describes location and size. If the file claims fewer directory entries than index 3, analyzer cannot legally read exception directory. It must not pretend DataDirectory[3] exists just because common PE headers usually contain 16 entries. This is a subtle but important difference between "works on normal compiler output" and "defensive parser".

Exception Directory: официальный вход к .pdata, а не угадывание по имени секции

EH/Unwind Analyzer exists largely because Windows x64 exception handling stores function unwind information in image metadata. The official route into this metadata in PE image is Exception Directory. In pe.h project defines:

```
#define IMAGE_NUMBEROF_DIRECTORY_ENTRIES 16
#define IMAGE_DIRECTORY_ENTRY_EXCEPTION 3
```

pe.c reads DataDirectory entry 3 by calculating relative offset:

```
size_t dd_rel = 0x70u + IMAGE_DIRECTORY_ENTRY_EXCEPTION * 8u;
if (dd_rel + 8u > opt_size) {
    sb_append(err,
        "Exception data directory lies outside the optional header.\n");
    return false;
}
size_t dd = opthdr + dd_rel;
uint32_t exc_rva = 0, exc_sz = 0;
if (!buf_read_u32(b, dd + 0, &exc_rva) ||
    !buf_read_u32(b, dd + 4, &exc_sz)) {
    sb_append(err, "Failed to read exception directory.\n");
    return false;
}
```

Each data directory entry is two uint32 values: VirtualAddress and Size. For Exception Directory, VirtualAddress is RVA of exception table, and Size is byte size. On x64 this table is normally an array of RUNTIME_FUNCTION records. But pe.c deliberately does not decode those records. It only records exception_rva and

exception_size in PeImage. Why? Because reading RUNTIME_FUNCTION belongs to unwind.c. pe.c does coordinate mapping and structural validation; unwind.c understands unwind table semantics. This module boundary prevents PE parser from becoming an accidental unwind parser.

The phrase ".pdata" appears often because Microsoft compilers usually place RUNTIME_FUNCTION table in .pdata section. But parser should avoid assuming section name. A packer, custom linker, manually constructed sample or object transformation can place Exception Directory in a section with another name while still using valid directory coordinates. Conversely, a file can contain section named .pdata that is not the official Exception Directory. The authoritative source for EH/Unwind Analyzer is DataDirectory[Exception]. Section name is additional context for reporting, not proof.

pe.c later validates exception directory at image boundary level:

```
bool exception_directory_oob = false;
if (exc_rva != 0 || exc_sz != 0) {
    uint64_t exc_end = (uint64_t)exc_rva + (uint64_t)exc_sz;
    exception_directory_oob =
        exc_rva == 0 || exc_sz == 0 ||
        exc_end > (uint64_t)size_of_image;
}
```

This code distinguishes two states. If both exc_rva and exc_sz are zero, Exception Directory is absent. That can be normal for certain images or for images without unwind metadata visible in file. But if one is zero and the other is not, or if range exceeds SizeOfImage, the declaration is structurally suspicious. It becomes exception_directory_oob. The parser does not immediately fail, because a report that says "Exception Directory is malformed" may be more useful than a fatal abort. Downstream reader may still fail to read the table, but PeImage carries the reason.

Notice the use of uint64_t for exc_end. exc_rva and exc_sz are uint32_t, and their sum can exceed 32-bit range. If code used uint32_t, a malicious directory near 0xffffffff with size 0x100 could wrap to small value and pass boundary check. The project avoids that. This is exactly the defensive style already introduced in util.c.

Exception Directory size also matters because RUNTIME_FUNCTION entries are fixed-size records. unwind.c will later expect size to be divisible or at least parse count cautiously. pe.c does not enforce that here. Again, module boundaries. pe.c says where the directory claims to be. unwind.c says whether its contents form valid function table records.

Section table: where virtual image and file bytes are stitched together

After Optional Header comes section table. Section headers are 40 bytes each. The parser calculates:

```
size_t secthdr = opthdr + opt_size;
if ((size_t)nsects > (SIZE_MAX - secthdr) / 40u) {
```

```
        sb_append(err, "Section header size overflow.\n");
        return false;
    }
    size_t needed = secthdr + (size_t)nsects * 40u;
    if (needed > b->size) {
        sb_append(err, "Section headers out of range.\n");
        return false;
    }
```

This is one of the most important defensive checks in pe.c. NumberOfSections is controlled by file. A malformed file can set it to a large value. The code does not compute secthdr + nsects * 40 naively. It first checks multiplication/addition against SIZE_MAX. Then it checks that all section headers physically fit in the input buffer. Without this, a parser could allocate based on wrapped size or read section headers from outside file.

Then the parser allocates PeSection array:

```
PeSection* secs = (PeSection*)calloc(nsects ? (size_t)nsects : 1,
                                     sizeof(PeSection));
```

The nsects ? nsects : 1 pattern prevents calloc(0, size), which may return NULL or a unique pointer depending on implementation. If nsects is zero, parser still receives an allocated pointer that can be freed safely. num_sections remains zero. This mirrors the empty-file ownership discipline from util.c: avoid implementation-dependent allocation edge cases.

Each section header is read field by field:

```
uint8_t name8[8];
buf_read(b, sh + 0, name8, 8);
memcpy(secs[i].name, name8, 8);
secs[i].name[8] = 0;

buf_read_u32(b, sh + 8,  &virtual_size);
buf_read_u32(b, sh + 12, &virtual_address);
buf_read_u32(b, sh + 16, &raw_size);
buf_read_u32(b, sh + 20, &raw_ptr);
buf_read_u32(b, sh + 36, &characteristics);
```

Section name in PE header is an 8-byte field, not guaranteed to be NUL-terminated if exactly 8 bytes are used. Project copies it into char name[9] and appends zero byte. This makes report output safer. But the name is still only a label. It can be empty, duplicated, non-standard, misleading or intentionally confusing. The parser stores it for human context, not for trust.

VirtualSize and VirtualAddress describe how section appears in memory image. SizeOfRawData and PointerToRawData describe where section bytes are in file. Characteristics describe flags: code, initialized data, readable, writable, executable and so on. EH/Unwind Analyzer later uses section context to reason about metadata placement. For example, unwind metadata inside executable or writable section may be suspicious enough to require manual review. But pe.c only stores raw characteristics. Interpretation belongs to later layers.

The section table is where file-to-memory mapping becomes concrete. Official PE docs say RVA almost always differs from file pointer. Section headers explain why. A section can start at RVA 0x1000 but its bytes can live at raw file offset 0x400. If a RUNTIME_FUNCTION says BeginAddress 0x1234, that is an RVA, not file offset. To disassemble bytes around 0x1234 in static file, analyzer needs section containing RVA 0x1234, subtract section.VirtualAddress, add PointerToRawData, and verify delta is inside SizeOfRawData. This is exactly what pe_rva_to_fileoff later does.

Raw range, virtual range and why max(VirtualSize, RawSize) is used

For each section pe.c computes raw and virtual spans. Raw range is:

```
raw_ptr .. raw_ptr + raw_size
```

Virtual range uses:

```
virtual_address .. virtual_address + max(virtual_size, raw_size)
```

This deserves explanation. In an ideal mental model, VirtualSize is memory size and SizeOfRawData is disk size. But real PE files can contain cases where VirtualSize is smaller, zero, rounded differently or inconsistent with raw size. For RVA containment, many tools use max(VirtualSize, SizeOfRawData) because file bytes may still correspond to RVAs even when VirtualSize is smaller or malformed. The project chooses u32_max for section containment and virtual overlap checks. Это conservative analysis choice: do not accidentally ignore raw-backed bytes simply because virtual_size is suspiciously small.

The helper is tiny:

```
static uint32_t u32_max(uint32_t a, uint32_t b) {
    return a > b ? a : b;
}
```

But the modeling decision is not tiny. If analyzer used only VirtualSize, it might fail to map an RVA into a section whose raw bytes exist. If it used only RawSize, it might miss BSS-like zero-filled virtual tail. Using max captures the broad span for containment, while pe_rva_to_fileoff still refuses to map into virtual tail beyond raw_size. This separation is elegant: section_contains_rva can say "this RVA belongs to the section's virtual span", while rva_to_fileoff can say "but there are no corresponding bytes in file".

This matters directly for unwind analysis. UNWIND_INFO can point into .xdata-like bytes. If RVA is in virtual tail with no raw bytes, static analyzer cannot decode it from file. That should become parse failure or unknown, not fabricated zeros. Windows loader might zero-fill virtual memory, but static file parser must not invent meaningful UNWIND_INFO records from absent bytes. pe_rva_to_fileoff captures that boundary.

Structural diagnostics: raw OOB, virtual OOB, overlaps and misalignment

pe.c does not merely parse headers. It measures structural health. It initializes counters:

```
size_t raw_oob = 0;
size_t virtual_oob = 0;
size_t raw_overlaps = 0;
size_t virtual_overlaps = 0;
size_t misaligned = 0;
```

OOB means out of bounds. raw_oob increments when a section claims raw bytes outside file size. Example: PointerToRawData is 0x2000, SizeOfRawData is 0x1000, but file size is only 0x2400. The raw section range ends at 0x3000, beyond file. A static analyzer cannot safely read missing bytes. This does not necessarily prove malicious intent; file can be truncated or malformed. But it strongly affects trust in any decoded metadata.

virtual_oob increments when section virtual span exceeds SizeOfImage. SizeOfImage is the image's declared memory span after loading. If a section says it extends past it, then the PE image layout contradicts itself. Again, this is table_integrity evidence, not a verdict. It says: coordinate system is damaged or unusual.

raw_overlaps increments when two sections' raw file ranges overlap. virtual_overlaps increments when their virtual spans overlap. Overlaps are not always impossible in hand-crafted PE files, but they are suspicious for ordinary compiler/linker output. They can make RVA mapping ambiguous: one RVA might appear to belong to multiple sections. Project pe_find_section_for_rva returns first matching section, so overlaps also represent a limitation. Reporting the overlap lets heuristics and analyst know mapping may be ambiguous.

misaligned gathers several alignment and ordering issues. Alignment checks are easy to dismiss as pedantic, but loader rules and linker output rely on them. If SectionAlignment and FileAlignment are contradictory, raw-to-virtual mapping becomes less trustworthy. If SizeOfHeaders is not aligned or is smaller than actual header table, header mapping can be wrong. If sections are not ordered by increasing VirtualAddress, a simple mapping strategy may still find sections, but the image looks non-standard.

This is where the parser becomes more than yes/no. It builds a gradient of structural confidence. A PE can be parseable but messy. A report should preserve that nuance.

Alignment checks: когда числа должны быть не просто положительными, а согласованными

Project calculates compact_alignment:

```
bool compact_alignment =
    section_alignment < 0x1000u && file_alignment == section_alignment;
```

Then it validates FileAlignment:

```
bool file_alignment_valid =
    is_power_of_two_u32(file_alignment) &&
    ((file_alignment >= 0x200u && file_alignment <= 0x10000u) ||
     compact_alignment);
if (!file_alignment_valid) misaligned++;
```

The helper is_power_of_two_u32 returns true if value is nonzero and only one bit is set. This matches common PE alignment requirements: FileAlignment is normally power of two, often between 512 and 64K. The compact alignment exception handles cases where SectionAlignment is below page size and equals FileAlignment. PE rules allow special low-alignment images under constraints. Project does not simply reject them as malformed, but records misalignment if the relationship is invalid.

Then pe.c checks:

```
if (section_alignment < file_alignment) misaligned++;
if (section_alignment < 0x1000u && file_alignment != section_alignment) {
    misaligned++;
}
if ((size_of_image % section_alignment) != 0) misaligned++;
if ((size_of_headers % file_alignment) != 0 ||
    (uint64_t)size_of_headers < (uint64_t)needed) {
    misaligned++;
}
if ((image_base & 0xffffu) != 0) misaligned++;
```

Each check corresponds to a different part of image consistency. SectionAlignment should not be smaller than FileAlignment in ordinary images. If SectionAlignment is below 0x1000, it should match FileAlignment. SizeOfImage should be multiple of SectionAlignment, because image memory layout rounds sections to alignment. SizeOfHeaders should be multiple of FileAlignment and must be large enough to include all headers already parsed. ImageBase should be 64K-aligned, hence low 16 bits zero.

Why not make all alignment failures fatal? Because the project's purpose is analysis of real-world binaries, including malformed, obfuscated, packed or hand-crafted samples. If parser aborted at first misalignment, analyst would lose useful evidence. Instead, pe.c stores misaligned count. Later heuristics can treat it as weak/medium structural signal depending on context. This is the same philosophy as in previous chapters: suspicious structure requires explanation, not instant accusation.

Inside section loop, pe.c checks each section's VirtualAddress against SectionAlignment and raw pointer/size against FileAlignment. It also checks that VirtualAddress increases strictly compared to previous section. Strict ordering is not absolutely necessary for a search loop that scans all sections, but non-monotonic order is unusual and can make human reasoning harder. It is useful evidence.

RVA to file offset: the most important small function in pe.c

The function pe_rva_to_fileoff is short, but it is one of the most important functions in the whole analyzer. It translates from Windows image coordinates to file coordinates:

```
bool pe_rva_to_fileoff(const PeImage* img,
                       uint32_t rva,
                       uint32_t* out_off)
```

RVA is what PE directories and x64 unwind structures store. RUNTIME_FUNCTION fields BeginAddress, EndAddress and UnwindData are image-relative values. UNWIND_INFO handler pointer is also image-relative when handler flags are present. But static analyzer reads bytes from Buf, and Buf is indexed by file offset. Therefore every decode step that starts from an RVA must eventually ask pe_rva_to_fileoff: do these image coordinates correspond to bytes in the file?

The function first finds a section:

```
const PeSection* s = pe_find_section_for_rva(img, rva);
if (s) {
    uint32_t delta = rva - s->virtual_address;
    if (delta >= s->raw_size) {
        return false;
    }
    uint32_t off = 0;
    if (!safe_add_u32(s->raw_ptr, delta, &off)) return false;
    if (img->file_size > 0 &&
        (uint64_t)off >= (uint64_t)img->file_size) return false;
    *out_off = off;
    return true;
}
```

pe_find_section_for_rva uses max(VirtualSize, RawSize) to decide whether section owns the RVA. But after that, pe_rva_to_fileoff checks delta >= raw_size. If the RVA falls into virtual tail beyond raw bytes, static parser cannot produce file offset. This distinction prevents a common mistake: treating zero-filled memory as if it were present and meaningful in file. If .bss-like region contains no raw data, there is no UNWIND_INFO to decode from file bytes.

safe_add_u32 protects raw_ptr + delta. raw_ptr and delta are 32-bit values from file-derived structures. Their sum could overflow uint32_t if malformed. The function refuses such mapping. Then it verifies off is inside file_size. It checks off >= file_size, not off + length, because pe_rva_to_fileoff maps a point; callers who read multiple bytes must still use buf_read to validate length. This is layered safety: pe_rva_to_fileoff gives a candidate file offset, util.c ensures actual read range exists.

If no section contains RVA, function tries header mapping:

```
if (img->size_of_headers > 0 &&
    rva < img->size_of_headers &&
    (img->file_size == 0 ||
     (uint64_t)rva < (uint64_t)img->file_size)) {
```

```
    *out_off = rva;
    return true;
}
```

RVAs inside SizeOfHeaders map directly to corresponding file offsets. This is important because PE data directories or unusual metadata can point into headers. For ordinary .pdata/.xdata this is not expected, but defensive parser should model the image, not only common compiler output. The check still respects file size.

If neither section nor headers can map the RVA, function returns false. False does not mean the RVA is "bad" in every possible runtime sense. It means static file parser cannot locate corresponding bytes in the file under current PeImage model. Downstream should turn that into parse failure, unknown or suspicious placement depending on context.

pe_find_section_for_rva: why first match is useful and limited

pe_find_section_for_rva scans sections in file order:

```
for (size_t i = 0; i < img->num_sections; i++) {
    const PeSection* s = &img->sections[i];
    uint32_t size = u32_max(s->virtual_size, s->raw_size);
    uint64_t start = (uint64_t)s->virtual_address;
    uint64_t end = start + (uint64_t)size;
    if ((uint64_t)rva >= start && (uint64_t)rva < end) return s;
}
return NULL;
```

This is straightforward and works for normal PE section tables. It also has a predictable behavior under overlaps: first matching section wins. That is not a proof that first section is semantically correct; it is a deterministic resolution policy. The parser separately counts virtual overlaps, so reports can warn about ambiguous mapping.

The function uses uint64_t for start and end to avoid 32-bit wrap. If section starts near 0xffffffff and size is nonzero, end can exceed 32-bit range. A naive uint32_t end would wrap and make containment wrong. This matters because malformed PE files often stress exactly these boundaries.

This function is also used indirectly for metadata placement. Later modules can ask: what section contains UnwindInfoRVA? What section contains handler target? Is it executable? writable? unknown? That placement feeds metadata_placement evidence family. If pe_find_section_for_rva returns NULL, report should not invent a section. Unknown section context is itself a meaningful state.

Section characteristics: raw flags now, interpretation later

PeSection stores characteristics as uint32_t. Characteristics can include executable, readable, writable, initialized data and other flags. pe.c does not decode them into booleans. It stores raw value because section semantics are used in multiple places later. report.c may print them. heuristics.c may judge metadata in writable/executable context. dynamic collector may use module map rather than file section flags. Keeping raw flags preserves information.

In a perfect compiler-produced image, unwind metadata is usually read-only and not executable. If .xdata-like records are inside writable or executable section, there may be benign explanations - custom linker, packed image, JIT support files, unusual build system - but the placement deserves attention. pe.c's job is not to apply that suspicion. It gives later layers enough data to ask.

This follows a larger architectural pattern: parser facts are separated from heuristic interpretations. If pe.c itself started appending "suspicious" for executable metadata, it would blur responsibilities. Instead, PeImage has facts and counters; HeuristicReport has findings, families, severity, score, confidence and weakening rules.

Fatal parse errors versus analyzable anomalies

A central design decision in pe.c is what stops analysis and what becomes reportable anomaly. Fatal errors include: not MZ/PE, PE header out of range, invalid signature, unsupported Machine, Optional Header out of range, unsupported PE32 magic, missing DataDirectory entry, section headers out of range, out-of-memory. In these cases, PeImage cannot be constructed reliably, so later modules must not run as if image exists.

Reportable anomalies include: section raw range out of file, section virtual range out of image, raw/virtual overlaps, misalignment, exception directory outside image. These indicate that image model is questionable, but PeImage still carries enough structure to report them. This distinction is essential for reverse engineering. A malformed sample is often exactly what analyst wants to understand. The tool should not collapse all malformed states into "parse failed" unless it truly cannot build coordinates.

For example, if Exception Directory range exceeds SizeOfImage, unwind.c may not be able to decode RUNTIME_FUNCTION records. But the report can still say: file is PE32+ AMD64, has N sections, headers claim exception_rva X and exception_size Y, but that range is outside SizeOfImage. That is much better than a vague "invalid". It gives manual validation a target.

This is also why pe_parse receives StrBuf* err. Fatal parser diagnostics go into err. Structural counters go into PeImage. Later report layer can show both: fatal parse output when parse stopped, or parsed image with warnings when analysis continued.

How pe.c prepares unwind.c without decoding unwind itself

unwind.c later needs three things from pe.c: exception directory coordinates, section mapping, and file buffer reads through util.c. It will read the exception table as RUNTIME_FUNCTION records, then for each record map UnwindData RVA to file offset and decode UNWIND_INFO. pe.c provides exception_rva and exception_size but does not assume entry count or content.

This separation is especially important because RUNTIME_FUNCTION belongs to x64 exception handling contract, not generic PE parsing. Official x64 exception handling documentation defines RUNTIME_FUNCTION and UNWIND_INFO; official PE Format documentation defines DataDirectory and section mapping. EH/Unwind Analyzer mirrors that separation in modules. pe.c implements PE coordinate system. unwind.c implements x64 unwind metadata semantics.

Consider a malformed file where Exception Directory points into a readable data section but size is not divisible by 12. pe.c should not try to reason about 12-byte entries; it should simply expose coordinates. unwind.c can then report malformed runtime function table. Another sample may have valid table coordinates but individual UnwindData RVAs pointing to impossible offsets. Again, pe.c maps when asked; unwind.c records parse failures. This produces better diagnostics and cleaner code.

How pe.c feeds heuristics without becoming a heuristic engine

Heuristic/evidence engine receives PeImage, RuntimeFunction array, UnwindSummary array and parse-success bitmap. It does not receive raw bytes as primary evidence. That means pe.c is responsible for converting raw PE structure into stable facts. The structural counters inside PeImage become table_integrity evidence. Examples:

```
section_raw_oob
section_virtual_oob
section_raw_overlaps
section_virtual_overlaps
section_misaligned
exception_directory_oob
```

These counters matter because unwind anomalies can have different meaning depending on PE structural health. If many UNWIND_INFO records fail to parse and section table is also badly malformed, confidence in precise interpretation should be lower, but suspicion of structural manipulation may be higher. If section table is clean and only one handler target points into writable section, finding is more localized. Heuristics need both local unwind facts and global PE health.

However, pe.c does not assign score. It does not set verdict. It does not know evidence families. This is deliberate. A parser that assigns suspicion while reading headers often becomes hard to test and hard to explain. In this project, parsing and

assessment are separate. pe.c says what was found. heuristics.c decides how strongly that matters, with weakening rules and confidence model.

How pe.c feeds reports and Ghidra integration

report.c uses PeImage to print file-level facts: machine, image_base, file_size, size_of_image, section_alignment, file_alignment, exception directory, sections and structural counters. These report fields are not decorative. They let analyst check whether downstream findings make sense. If report says handler_rva belongs to section X, analyst can verify section X characteristics and RVA range.

Ghidra CSV export depends heavily on correct RVA mapping. Ghidra works with program addresses and image bases. If analyzer emits a function range with wrong RVA, importer may label wrong code. Therefore PeImage's ImageBase and section mapping are part of tool integration contract. Even when Ghidra can parse PE itself, EH/Unwind Analyzer must be internally consistent: the CSV it emits should reflect the same image model used by heuristics.

JSON schema also benefits from PeImage normalization. Instead of dumping raw header bytes, report exposes named fields. Machines and scripts can compare exception_directory_oob with unwind parse failures. HTML report can show section table in human-readable form. CSV can carry records for external tools. The quality of all these outputs is bounded by pe.c correctness.

A slow walk through one ordinary file

Imagine a normal MSVC-built x64 executable. Buf contains file bytes. pe_parse starts at offset 0, reads MZ, reads e_lfanew at 0x3c, jumps to PE signature. COFF Machine is 0x8664, so architecture matches. NumberOfSections is, say, 6. SizeOfOptionalHeader says where section table begins. Optional Header Magic is 0x20b, so PE32+ layout applies.

Parser reads ImageBase, maybe 0x140000000. It reads SectionAlignment 0x1000, FileAlignment 0x200, SizeOfImage maybe 0x90000, SizeOfHeaders maybe 0x400, NumberOfRvaAndSizes 16. DataDirectory[3] says Exception Directory RVA 0x70000 and Size 0x2400. Sections include .text at RVA 0x1000, .rdata at 0x60000, .pdata at 0x70000, .xdata at 0x73000, .data at 0x80000.

Section checks pass: raw ranges fit in file, virtual ranges fit in SizeOfImage, no overlaps, alignments look normal. exception_directory_oob false. PeImage now says: this is a parseable PE32+ AMD64 image with coherent section map and official Exception Directory. unwind.c can read exception table from RVA 0x70000 by mapping it through .pdata's raw pointer. For each RuntimeFunction, UnwindData RVA points into .xdata. This is the normal path.

In report, structural counters are zero. That does not mean binary is safe. It means PE table integrity did not provide suspicious signals. Heuristics still may find unusual handler dominance, metadata placement, or unwind reuse. Dynamic

validation may still observe exception-based control flow. But PE parser has not seen structural reasons to distrust the coordinate system.

A slow walk through a damaged or evasive-looking file

Now imagine another file. MZ and PE signature are valid. Machine is AMD64. Optional Header is PE32+. But SectionAlignment is 0x200 while FileAlignment is 0x1000, SizeOfImage is not divisible by SectionAlignment, section .pdata raw_size extends past file end, and Exception Directory says RVA 0x500000 with size 0x1000 while SizeOfImage is 0x300000.

A simplistic tool might crash, or it might say "not PE". EH/Unwind Analyzer's pe.c can do better. It constructs PeImage and increments misaligned, raw_oob and exception_directory_oob. unwind.c will likely fail to map Exception Directory. HeuristicReport can then include table_integrity evidence and requires_manual_review. Report can show exactly which level failed: the file is PE-like enough to parse headers, but its exception metadata coordinates cannot be trusted.

This is not just nicer error handling. It changes analyst behavior. Instead of asking "why did tool fail?" analyst sees "the file declares exception directory outside image". That is a concrete hypothesis: truncated file, broken sample, custom loader, packed image, deliberately malformed metadata, or a parser limitation. Manual validation can continue in Ghidra, dumpbin, WinDbg or custom scripts.

Why pe.c does not load the image

A common beginner mistake is to think PE parser should simulate Windows loader completely. It should apply relocations, resolve imports, respect section protections, process TLS, handle delay imports and map image exactly. EH/Unwind Analyzer does not do this in pe.c. Static mode reads file as untrusted bytes. It does not execute code and does not ask Windows loader to trust the sample.

This limitation is also a safety feature. If static analysis required LoadLibrary or mapping image as executable, the tool would cross execution-like boundary. pe.c stays purely file-based. It reconstructs enough image layout from headers to map RVAs to file offsets. That is sufficient for parsing static exception/unwind metadata.

Dynamic validation later crosses execution boundary deliberately and only when user requests --dynamic. That part is isolated in dynamic_win.c. This separation protects the default static workflow. A malformed PE can be inspected without running it. A suspicious PE can yield structural report before any process creation. This is exactly the trust boundary described in architecture document.

Why pe.c avoids Windows headers

pe.h says the parser is minimal and uses no Windows headers. This can look strange because Windows SDK already defines IMAGE_DOS_HEADER, IMAGE_NT_HEADERS64 and IMAGE_SECTION_HEADER. But avoiding Windows headers gives the project explicit control. It can build in environments where SDK availability or macro conflicts would be inconvenient. It can define only constants needed for analysis. It can read little-endian bytes through util.c rather than cast buffer to SDK structs.

Struct casts are risky for file parsers. They depend on packing, alignment and host endianness. They can also lead to unaligned reads. On x86/x64 unaligned reads often work, but writing portable defensive C should not rely on that. Explicit buf_read_u32 at offset X is verbose but honest. It says: I need exactly four bytes here, and I know what endian conversion means.

The cost is maintenance: if PE32+ offsets are wrong, parser is wrong. That is why official documentation remains the source of truth. The project is not free to reinterpret Optional Header. The code's constants must match PE/COFF spec. In the book, this is a good example of engineering responsibility: independence from Windows headers does not mean independence from Windows contract.

Project boundary: pe_free and ownership

At end of parse, PeImage owns sections. pe_free releases sections and zeroes structure:

```
void pe_free(PeImage* img) {
    if (!img) return;
    free(img->sections);
    memset(img, 0, sizeof(*img));
}
```

This is simple, but it keeps ownership clean. main.c can call pe_free in cleanup path regardless of how far later modules went. Zeroing prevents accidental reuse after free. Since PeImage holds no nested allocations besides sections, cleanup is deterministic.

This also interacts with error paths. If pe_parse fails before allocating sections, out remains zero. If it allocates sections and later fails inside section loop, it frees secs before returning false. Successful parse transfers ownership to out->sections. These ownership transitions are part of parser correctness. A reverse engineering tool that leaks memory in normal CLI use may not look dramatic, but fuzzing malformed corpora can run parser thousands of times. Leaks and double frees become test blockers.

Where official documentation stops and project policy begins

Microsoft documentation tells us what RVA means, what sections are, where DataDirectory is, what x64 exception directory points to, how RUNTIME_FUNCTION and UNWIND_INFO participate in unwinding. It does not tell this project how strict to be with malformed files. It does not say whether to abort on section overlap or report it as evidence. It does not say whether score should increase for exception_directory_oob. It does not say how JSON field names should look.

Those are project policy decisions. EH/Unwind Analyzer chooses a defensive but non-hysterical policy. Fatal errors stop when coordinate system cannot be constructed. Suspicious but analyzable inconsistencies become counters. Unwind-specific decoding is delegated to unwind.c. Suspicion and confidence are delegated to heuristics.c. Runtime confirmation is delegated to dynamic_win.c. Human and machine explanation is delegated to report.c.

This separation is what makes the architecture explainable. If a report says table_integrity finding exists, analyst can trace it back to PeImage counters. If a UNWIND_INFO parse failed, analyst can trace mapping through pe_rva_to_fileoff. If dynamic validation did not observe handler hit, analyst can separate runtime coverage from static coordinates. pe.c is the root of this traceability.

How to read pe.c as a reverse engineer

When studying pe.c, do not read it as a list of header offsets. Read it as a sequence of trust upgrades. At the beginning, buffer is just bytes. After MZ/e_lfanew, it is maybe PE-like. After PE signature and COFF Machine, it is an AMD64 PE candidate. After PE32+ Optional Header and key fields, it has an image coordinate system. After DataDirectory[Exception], it has a declared unwind metadata entry point. After section table parse, it has an RVA-to-file mapping. After structural counters, it has a measured level of layout health.

This is the mental route:

```
bytes
  -> MZ and e_lfanew
  -> PE signature
  -> COFF Machine = AMD64
  -> PE32+ Optional Header
  -> ImageBase, SizeOfImage, alignments
  -> DataDirectory[Exception]
  -> Section table
  -> RVA mapping
  -> PeImage + structural diagnostics
```

Only after this route can the analyzer responsibly say: now we can try to read RUNTIME_FUNCTION table. If any earlier step is skipped, later analysis may still produce numbers, but those numbers no longer rest on a clear contract.

The connection back to Windows x64 exception/unwind model

The whole reason pe.c exists in this form is x64 unwinding. Windows x64 cannot rely on classic frame pointer chain alone. It uses function unwind information to know how to restore nonvolatile registers and caller stack pointer. That function unwind information is stored in image metadata, commonly in .pdata and .xdata, but officially reached through PE exception directory and RUNTIME_FUNCTION records. Therefore any tool that analyzes unwind must first parse PE image layout correctly.

RUNTIME_FUNCTION entries store RVAs, not file offsets. UNWIND_INFO RVAs must be mapped to file bytes. Handler RVAs must be located in sections. Chained unwind info can refer back into function table-like records. Dynamic function tables exist at runtime and may not be present in file at all, but static parser still defines baseline file metadata. All of this begins with PeImage.

If pe.c is wrong, RtlVirtualUnwind mental model becomes disconnected from file reality. The book's earlier chapters taught how Windows uses metadata at runtime. Chapter 11 shows how a defensive static tool reconstructs enough of that metadata from disk without pretending to be the loader. pe.c is the first place where that reconstruction becomes concrete.

What pe.c intentionally does not solve

A good architecture chapter should also say what module does not do. pe.c does not validate Authenticode signatures. It does not parse imports, exports, relocations, resources, TLS, debug directory or load config. It does not emulate loader. It does not unpack packed code. It does not read overlay data as meaningful metadata. It does not decide whether sample is malicious. It does not parse RUNTIME_FUNCTION contents. It does not resolve runtime module base under ASLR.

These omissions are not weakness by themselves. They define scope. EH/Unwind Analyzer is about exception/unwind infrastructure, not every part of PE format. A focused parser can be more reliable than a broad but shallow one. If future project versions need imports or load config, those should be added deliberately, with the same style: safe reads, official documentation, normalized facts, separate heuristics.

The important thing is that pe.c's limitations are visible downstream. If an RVA cannot be mapped because it is in virtual tail, return false. If Exception Directory is out of image, set exception_directory_oob. If sections overlap, count it. This makes limitation a reportable state, not hidden behavior.

A compact end-to-end example: from Exception Directory to first unwind byte

Suppose Optional Header DataDirectory[3] says:

```
exception_rva  = 0x00074000
exception_size = 0x00001800
```

pe.c stores these in PeImage. unwind.c asks pe_rva_to_fileoff(img, 0x74000, &off). pe_find_section_for_rva finds section .pdata:

```
name             = ".pdata"
virtual_address  = 0x00074000
virtual_size     = 0x00001800
raw_ptr          = 0x0006C000
raw_size         = 0x00001800
```

Delta is 0. File offset becomes 0x6C000. util.c then reads 12-byte RUNTIME_FUNCTION entries from Buf starting at 0x6C000. One entry contains:

```
BeginAddress = 0x00011200
EndAddress   = 0x00011370
UnwindData   = 0x00076040
```

BeginAddress and EndAddress are RVAs of function range. UnwindData is RVA of UNWIND_INFO. unwind.c again calls pe_rva_to_fileoff(img, 0x76040, &off2). pe_find_section_for_rva finds .xdata, computes file offset, and only then reads UNWIND_INFO bytes. If handler flags are set, handler RVA read from UNWIND_INFO is not executed or trusted; it is recorded and later can be checked for placement.

This example shows why pe.c is not boring preliminary code. Every later fact passes through it. Function begin/end, unwind info location, handler placement, Ghidra CSV addresses, dynamic breakpoint hints from static handler RVAs - all of them depend on correct mapping.

Reading report fields that came from pe.c

When a report shows PE-level fields, read them as the coordinate contract for the rest of the file. image_base tells preferred base. size_of_image tells virtual envelope. size_of_headers defines header mapping. section_alignment and file_alignment explain why file offset and RVA differ. exception_rva and exception_size are the official static entry to unwind table. sections show where RVA ranges live and which protections/characteristics surround them. Structural counters show how much the parser had to distrust the layout.

If section_raw_oob is nonzero, any metadata inside those raw ranges may fail to decode. If section_virtual_overlaps is nonzero, section-based placement may be ambiguous. If exception_directory_oob is true, absence of runtime function records in static output does not mean absence of exception tricks; it means the declared directory is not statically trustworthy. If section_misaligned is high, confidence should not be read as if file were ordinary compiler output.

This is why 9.5 spent time on reports. Reports are not afterthought. They preserve the chain from pe.c facts to unwind findings to assessment. A reader who ignores PE-level fields and jumps directly to final_verdict loses the map.

Final mental model for pe.c

pe.c is the layer that says: "Given these untrusted bytes, here is the safest PE32+ AMD64 image map I can construct, here are the official exception directory coordinates, and here are the structural places where the map looks damaged or unusual." It does not promise that the file will run. It does not promise that Windows loader will accept every anomaly. It does not promise that no runtime code will appear later. It promises a bounded, documented static model.

That model is enough for the next layer. unwind.c can now take exception_rva and exception_size and ask a more specific question: do these bytes form a coherent x64 RUNTIME_FUNCTION table and UNWIND_INFO network? If pe.c is the map of the city, unwind.c is the investigation of a particular road system inside it. Without map, every road address is a guess.

11.5. unwind.c: как проект декодирует .pdata/.xdata и превращает их в UnwindSummary

После раздела про PE parser у нас есть карта образа: заголовки прочитаны, секции описаны, RVA можно осторожно переводить в file offset, Exception Directory найден через официальный data directory, а не по имени секции. Но это ещё не unwind analysis. Это только подготовленная местность. Настоящий переход от структуры PE к пониманию поведения Windows x64 начинается в модуле unwind.c. Именно здесь проект впервые начинает читать не просто заголовки файла, а контракт восстановления стека: диапазоны кода, адреса unwind metadata, операции prolog, handler flags, chained records и компактную сводку, из которой потом вырастут findings, reports и dynamic validation hints.

Слово decoder здесь важно понимать буквально. Decoder - это декодер, расшифровщик формата. Он берёт bytes, у которых есть документированный layout, и переводит их в нормализованную модель проекта. Он не выполняет функцию, не эмулирует процессор, не заменяет RtlVirtualUnwind и не делает вывод “вредоносно”. Его задача уже, но глубже: прочитать то, что Windows ожидает видеть в exception/unwind metadata, отделить физически нечитаемое от декодируемого, сохранить предупреждения, не потерять адреса handlers и выдать compact summary, на который можно опереться дальше.

В официальной модели Windows x64 unwind information описывает, как восстановить состояние вызывающей функции, если выполнение остановилось внутри текущей функции. Это нужно потому, что обычный ret работает только тогда, когда функция сама дошла до epilog и вернула управление. Исключение может возникнуть внутри body, после части prolog, до epilog, посреди сохранённых registers и выделенного stack frame. Windows не может просто посмотреть на RSP и прочитать return address вслепую. Она должна знать, что prolog уже сделал со стеком, какие nonvolatile registers были сохранены, есть ли handler, и есть ли связанная unwind record. В проекте EH/Unwind Analyzer именно unwind.c превращает этот официальный контракт в данные, пригодные для анализа.

Положение unwind.c в pipeline: почему он стоит между pe.c и heuristics.c

Если смотреть на архитектуру проекта как на поток доказательств, pe.c отвечает за вопрос “где в этом файле находится образ Windows и как его адреса сопоставляются с bytes на диске”. Но pe.c намеренно не решает, что означает конкретный RUNTIME_FUNCTION или конкретный UWOP_ALLOC_SMALL. Это уже область unwind.c. С другой стороны, heuristics.c не должен снова лазить в raw bytes и повторно проверять boundaries. Он

получает уже подготовленную модель: массив RuntimeFunction, массив UnwindSummary и массив us_ok. Поэтому unwind.c является средним слоем: он ещё близок к binary format, но уже создаёт semantic facts.

```
input bytes
  -> pe.c: PeImage, sections, RVA mapping, Exception Directory
  -> unwind.c: RuntimeFunction[] + UnwindSummary[] + us_ok[]
  -> heuristics.c: findings, score, confidence, verdict
  -> dynamic_win.c: optional runtime evidence
  -> report.c: JSON / HTML / CSV / Ghidra CSV
```

Эта позиция объясняет стиль кода. unwind.c не пытается быть “умным детектором”. Когда он видит invalid range, reuse, suspicious placement или handler in writable section, он не обязан сразу начислять score. Но он обязан сохранить всё, что позволит later layer сделать честный вывод. Если parser слишком рано отбросит запись, heuristics потеряет evidence. Если parser слишком смело назовёт запись нормальной, report обманет реверсера. Поэтому module design строится на постоянном разделении: physically readable, semantically suspicious, fatal parse failure, nonfatal anomaly.

Это же разделение делает книгу и проект связными. В теоретических главах мы говорили, что Windows loader, exception dispatcher и RtlVirtualUnwind работают с image-relative metadata. В проектной главе мы видим, как эта теория превращается в конкретную программу: сначала pe.c даёт img, потом unwind.c принимает img как источник mapping и SizeOfImage, затем читает exception directory и xdata. Без PeImage decoder был бы слепым: он знал бы числа, но не знал бы, являются ли они адресами внутри image и где эти адреса лежат в file-backed bytes.

RuntimeFunction как первая нормализованная единица unwind analysis

В include/unwind.h проект определяет структуру RuntimeFunction. Название похоже на “runtime function”, но новичку легко ошибиться. Это не объект функции в C, не symbol, не имя, не guarantee, что дизассемблер увидит ровно одну функцию. Это запись function table Windows x64. Она описывает диапазон адресов кода и адрес unwind metadata для этого диапазона. Диапазон задаётся полуинтервалом: begin_rva включён, end_rva исключён. Если RIP попал между BeginAddress и EndAddress, Windows может использовать связанную UNWIND_INFO, чтобы понять, как откатывать кадр.

```
typedef struct RuntimeFunction {
    uint32_t begin_rva;
    uint32_t end_rva;
    uint32_t unwind_rva;
} RuntimeFunction;
```

Все три поля являются RVA. Это принципиально. BeginAddress не хранит абсолютный VA процесса. EndAddress не хранит file offset. UnwindInfoAddress тоже не file pointer. Они image-relative, потому что образ может быть загружен

по разной runtime base из-за ASLR, но относительная карта внутри image остаётся стабильной. Поэтому RuntimeFunction в проекте не хранит module base и не хранит pointer. Он хранит именно то, что находится в PE exception directory: 32-bit image-relative numbers.

Размер записи равен 12 bytes. Это не случайность, а прямое следствие трёх 32-bit fields. Поэтому reader exception directory проверяет, что size directory кратен 12. Если размер не кратен 12, проект не пытается “дочитать что получится”. Он считает table malformed. Такой выбор важен: forensic parser мог бы прочитать complete entries и отдельно сохранить хвост как anomaly, но EH/Unwind Analyzer в текущей версии выбирает строгий contract directory. Если directory заявлен как table of RUNTIME_FUNCTION, его размер должен состоять из целых записей.

Чтение Exception Directory: где начинается ответственность unwind.c

Функция unwind_read_runtime_functions_ex получает Buf, PeImage, output pointer, output count и диагностический StrBuf. Она начинается с обнуления outputs. Это маленькая, но важная инженерная привычка: даже если чтение сорвётся, caller не должен получить мусорный pointer или старый count. В проекте ownership строгий: если функция успешно выделила RuntimeFunction array, caller обязан освободить его; если не выделила, out pointer остаётся NULL.

```
*out_rf = NULL;
*out_count = 0;
```

Дальше reader смотрит на img->exception_rva и img->exception_size. Если оба равны zero, это не ошибка чтения. Это означает, что PE image не заявил exception directory. Для некоторых файлов отсутствие записей может быть нормальным, например если нет nonleaf functions, либо файл необычный, либо он не содержит ожидаемой metadata. Но на уровне reader это physical success: читать нечего, и массив остаётся пустым. Позже heuristics может решить, что no_runtime_functions - weak or medium signal в контексте, но reader не должен сам превращать отсутствие таблицы в подозрительный verdict.

Если же zero только одно поле - RVA есть без size или size есть без RVA - проект считает directory malformed. Это не просто “неудобно читать”. Это нарушение пары fields. Data directory в PE описывает область через RVA и size вместе. Нельзя честно сказать, где таблица, если нет RVA; нельзя честно сказать, сколько читать, если нет size. Поэтому unwind_read_runtime_functions_ex возвращает RUNTIME_FUNCTION_READ_MALFORMED и добавляет diagnostic.

Следующий шаг - преобразование exception_rva в file offset через pe_rva_to_fileoff. Здесь видно, почему предыдущий раздел про pe.c был не

второстепенным. Exception Directory в PE хранит RVA, но static analyzer читает bytes из файла. Чтобы прочитать table, нужно найти секцию, куда попадает RVA, вычесть section VirtualAddress, прибавить PointerToRawData и проверить, что эта область действительно file-backed. Если RVA указывает на virtual tail, на дырку или за пределы mapped sections, reader не имеет права читать “как-нибудь”. Он сообщает, что Exception directory RVA not mappable.

После mapping проверяется полный диапазон: file offset плюс exception_size не должен выходить за размер Buf. Здесь снова работает базовый принцип util.c: никакой parser не читает за пределами input buffer. Даже если PE header обещает тысячу entries, но file физически короче, reader выбирает failure, а не undefined behavior. Для анализатора недоверенных файлов это не paranoia, а базовая гигиена.

```
if ((size_t)off + img->exception_size > b->size) {
    sb_append(err, "Exception directory out of file range.\n");
    return RUNTIME_FUNCTION_READ_MALFORMED;
}
if (img->exception_size < 12) {
    sb_append(err, "Exception directory size too small.\n");
    return RUNTIME_FUNCTION_READ_MALFORMED;
}
if ((img->exception_size % 12u) != 0) {
    sb_append(err, "Exception directory size is not a multiple of RUNTIME_FUNCTION size.\n");
    return RUNTIME_FUNCTION_READ_MALFORMED;
}
```

Когда checks пройдены, проект вычисляет count как exception_size / 12 и выделяет массив через calloc. calloc здесь удобен не только потому, что обнуляет память. Он задаёт спокойный initial state: если будущий code path неожиданно остановится, элементы не будут содержать неинициализированные значения. После выделения каждая запись читается тремя buf_read_u32: begin_rva, end_rva, unwind_rva. Физически это little-endian fields. Reader не кастует bytes к struct RUNTIME_FUNCTION, потому что такой cast зависел бы от alignment, packing и доверия к layout host compiler. Проект остаётся в модели byte parser.

Очень важный момент: reader не проверяет end_rva > begin_rva прямо здесь. Он также не решает, отсортирована ли таблица, не пересекаются ли ranges, лежит ли unwind_rva в красивой секции .xdata. Такое решение может показаться странным, но оно дисциплинирует архитектуру. Reader отвечает на вопрос: “можно ли физически прочитать table entries из declared exception directory?” Semantic consistency переносится в heuristics. Благодаря этому report может показать пользователю сами плохие values и отдельные findings вроде invalid_ranges, rf_overlaps, unwind_rva_oob. Если reader отбрасывал бы всю table при первом странном range, проект потерял бы granular evidence.

UnwindSummary: компактная модель, а не полный слепок .xdata

После чтения RuntimeFunction[] начинается второй и более сложный уровень: для каждого unwind_rva нужно прочитать UNWIND_INFO. Результатом является UnwindSummary. Это не копия raw structure и не полный replacement для RtlVirtualUnwind. Это компактная summary, специально выбранная под задачи проекта: массовый static analysis, heuristic evidence, reports, Ghidra integration и dynamic hint extraction.

```
typedef struct UnwindSummary {
    bool     valid;
    uint8_t  version;
    uint8_t  flags;
    uint8_t  prolog_size;
    uint8_t  count_codes;
    uint8_t  frame_reg;
    uint8_t  frame_off;
    bool     has_handler;
    uint32_t handler_rva;

    bool     has_chain;
    RuntimeFunction chain;

    uint32_t fixed_stack_bytes;
    uint32_t pushed_nonvol_count;
    uint16_t saved_nonvol_mask;
    uint16_t saved_xmm_mask;

    uint32_t anomalies;
    char     parse_error[160];
} UnwindSummary;
```

Поле valid означает “structure decoded enough to produce summary”. Оно не означает “metadata нормальная”. У valid summary может быть nonzero anomalies. Это отличие повторяется в книге много раз, потому что оно спасает от неправильного чтения отчёта. valid=false означает parse failure: header не прочитан, версия не поддержана, operands обрываются, handler tail truncated. valid=true с anomalies означает, что parser смог пройти structure, но увидел странности: bad operand, suspicious code offset, handler RVA outside image, alignment issue, chain invalid.

version, flags, prolog_size, count_codes, frame_reg и frame_off - это normalized fields из header UNWIND_INFO. Они нужны reports и heuristics не только как raw values. Они объясняют, какая модель frame была заявлена compiler/toolchain. has_handler и handler_rva показывают, что record содержит language-specific или exception/termination handler entry point. has_chain и chain показывают, что эта metadata не самостоятельная, а ссылается на chained RUNTIME_FUNCTION. fixed_stack_bytes, pushed_nonvol_count и masks дают грубую, но полезную модель того, как prolog менял stack and preserved state.

Важно не переоценивать fixed_stack_bytes. Это estimated fixed stack frame delta, вычисленный по unwind operations: pushes, allocs, machframe. Он помогает анализатору, report и человеку понять scale frame. Но он не равен полной симуляции instruction-by-instruction для произвольного offset.

RtlVirtualUnwind учитывает control PC, prolog/epilog state и полную semantics OS unwinder. Проектная summary специально компактна. Она нужна для поиска anomalies and evidence, а не для точного восстановления caller context в каждом byte offset.

Вход в unwind_parse: почему decoder получает и bytes, и PeImage

Основная функция parser выглядит как обычный C interface, но каждый аргумент имеет смысловой вес. Buf даёт file bytes. PeImage даёт mapping and image bounds. unwind_rva выбирает конкретную UNWIND_INFO. opts задают policy: следовать ли chain, насколько глубоко, включать ли strict ABI diagnostics. out получает summary. ops_text, если передан, получает readable disassembly-like list of unwind ops. err собирает diagnostics.

```
bool unwind_parse(const Buf* b,
                  const PeImage* img,
                  uint32_t unwind_rva,
                  const UnwindParseOpts* opts,
                  UnwindSummary* out,
                  StrBuf* ops_text,
                  StrBuf* err);
```

Наличие PeImage особенно важно. UNWIND_INFO address является RVA, и decoder не может просто сделать b->data + unwind_rva. File layout и virtual layout отличаются. SectionAlignment, FileAlignment, raw data size and virtual size создают mapping, который уже разобрал pe.c. Поэтому unwind_parse сначала переводит unwind_rva в file offset. Если mapping невозможен, parser не продолжает. Это не “не нашёл секцию .xdata”; это “официальный RVA не имеет безопасного file-backed representation в данном input”.

В начале parser очищает out через memset и запоминает err_start. err_start нужен для parse_error. StrBuf err может уже содержать текст от previous entries. Если текущий parse failed, out->parse_error должен получить именно диагностику текущего вызова, а не весь накопленный log. Поэтому unwind_parse_fail берёт substring err->s + err_start, обрезает newline и кладёт compact message в fixed field 160 bytes. Это маленький пример того, как reporting requirements влияют на parser design.

ops_text сбрасывается отдельно. Эта строка не является частью core model. Она нужна для режима --ops и presentation: показать человеку, какие unwind operations были прочитаны. Массовый анализ не обязан хранить полный текст операций для каждой функции. Это экономит память и отделяет machine-readable summary от human-readable explanation.

Options: follow_chain, max_chain_depth, strict_abi

UnwindParseOpts кажется небольшим struct, но он выражает три разные политики. follow_chain говорит parser, нужно ли идти по chained unwind info и

агрегировать summary. max_chain_depth ограничивает глубину. strict_abi включает дополнительные diagnostics против формальных правил Microsoft x64 unwind ABI. Эти options нужны потому, что decoder используется в разных режимах: для обычного analysis, для debug-style dump, для tests, для более строгой проверки.

```
typedef struct UnwindParseOpts {
    int  max_chain_depth;
    bool follow_chain;
    bool strict_abi;
} UnwindParseOpts;
```

Default max depth равен 8, но если caller передал значение больше 16, parser clamp-ит его до 16. Причина не философская, а конкретная: локальный массив seen хранит 16 RVA для loop detection. Код не обещает бесконечный graph traversal. Это защитный parser, а не graph database. Если chain глубже, это уже отдельный signal, который можно пометить UA_CHAIN_DEPTH_EXCEEDED.

strict_abi особенно тонок. Проект различает “decodable in the wild” и “formally clean under strict ABI”. Реальные toolchains and hand-written assembly иногда создают metadata, которая декодируется и работает, но выглядит спорно относительно строгого ожидания nonvolatile register classes or operation order. Поэтому strict_abi не всегда делает parse fatal. Он добавляет anomalies вроде UA_ABI_REGISTER_CLASS, UA_ABI_RESERVED_OPINFO, UA_ABI_PUSH_ORDER, UA_ABI_CHAIN_RESTRICTION, UA_ABI_SAVE_OFFSET. Это даёт пользователю выбор: обычный режим терпим к реальному миру, строгий режим подсвечивает формальные нарушения.

Header UNWIND_INFO: четыре bytes, которые задают весь остальной разбор

UNWIND_INFO начинается с четырёх bytes. Эти четыре bytes задают version, flags, prolog size, count of codes, frame register and frame offset. На бумаге это маленький header. В анализаторе это точка ветвления всей логики. Если version unsupported, дальше нельзя надёжно читать slots. Если flags конфликтуют, tail нельзя честно интерпретировать. Если count_codes неверен, decoder может уйти за границу array. Если frame register указан, но UWOP_SET_FPREG нет, модель frame base становится подозрительной.

```
byte 0: Version:3 | Flags:5
byte 1: SizeOfProlog
byte 2: CountOfCodes
byte 3: FrameRegister:4 | FrameOffset:4
```

parser сначала проверяет alignment unwind_rva по 4-byte boundary. Если alignment нарушен, он ставит UA_ALIGNMENT, но не останавливается. Почему? Потому что misalignment делает metadata подозрительной, но не обязательно физически нечитаемой. Если bytes доступны, decoder может продолжить и дать report больше evidence. Это пример nonfatal anomaly: структура декодируема, но её placement or alignment вызывает вопрос.

Затем parse_unwindinfo_header переводит RVA в file offset, проверяет наличие четырёх bytes и читает header. Если RVA не mappable или bytes за пределами файла, parser ставит UA_TRUNCATED и возвращает parse failure. Здесь уже нельзя продолжать. Без header невозможно даже знать count_codes, flags and tail layout. Любая попытка “угадывать” превратила бы report в фантазию.

Version извлекается как low 3 bits первого byte, flags как high 5 bits. Проект поддерживает version 1 and 2. Unsupported version помечается UA_VERSION_UNKNOWN и является fatal failure. Это особенно важно на фоне появления x64 unwind information V3 for Intel APX. V3 нельзя читать как V1/V2 просто потому, что числа похожи. Неправильная версия сместила бы interpretation operations and tail, могла бы создать фиктивный handler_rva and false evidence. Поэтому безопаснее честно сказать: unsupported version.

Flags проверяются на known bits: UNW_FLAG_EHANDLER, UNW_FLAG_UHANDLER, UNW_FLAG_CHAININFO. Если выставлены unknown bits, parser ставит UA_BAD_OPERAND. Если CHAININFO одновременно сочетается с EHANDLER or UHANDLER, это тоже bad operand. В официальной модели tail после codes либо содержит handler information, либо chained RUNTIME_FUNCTION. Одновременно читать один и тот же tail как handler RVA and as chain triple нельзя.

```
enum UnwindFlags {
    UNW_FLAG_EHANDLER  = 0x01,
    UNW_FLAG_UHANDLER  = 0x02,
    UNW_FLAG_CHAININFO = 0x04
};
```

FrameRegister and FrameOffset тоже проверяются уже на header level. Если frame_reg равен zero, frame_off должен быть zero: offset без register бессмысленен. Если frame_reg nonzero, register должен быть GPR, пригодным для unwind record. В strict_abi mode он должен быть ABI-nonvolatile GPR: RBX, RBP, RSI, RDI, R12-R15. Позже parser проверит ещё одну связь: если frame_reg задан, должен появиться UWOP_SET_FPREG, если record не является chained. Так header field связывается с operation array.

CountOfCodes: почему это slots, а не количество операций

CountOfCodes - один из тех терминов, которые кажутся очевидными, пока не начинаешь писать parser. Это не количество high-level operations. Это количество двухбайтовых slots в array unwind codes. Одна logical operation может занимать один, два или три slots. Например PUSH_NONVOL занимает один slot, ALLOC_LARGE может занимать два или три, SAVE_NONVOL_FAR занимает три. Поэтому parser не может просто пройти count раз и считать каждый slot отдельной operation. Он должен читать operation start, определить slot count, проверить, что operands помещаются, и только после этого переходить к следующей operation.

Array stored on disk padded до чётного количества slots, потому что tail after codes должен быть 4-byte aligned. Код вычисляет slots_even как (count + 1) & ~1. Если count был odd, физически хранится ещё один padding slot. Этот padding не является semantic operation. Это важно для tail offset. Если забыть padding, parser начнёт читать handler_rva или chain triple на два bytes раньше и создаст полностью ложную картину.

```
uint32_t slots_even = (uint32_t)((count + 1u) & ~1u);
size_t codes_off = (size_t)uoff + 4;
size_t codes_bytes = (size_t)slots_even * 2u;
size_t after_codes = codes_off + codes_bytes;
```

Перед чтением slots проект проверяет, что codes_off + codes_bytes не выходит за file size. Это physical boundary для всего array с padding. Если array не помещается, parser ставит UA_TRUNCATED and fails. Здесь нельзя попробовать читать “часть”. Operation operands зависят от slot counts; частичный array может привести к чтению случайных bytes as operand and false handler tail.

Внутри loop каждый operation start читается как 16-bit slot. Low byte - CodeOffset. High byte разбивается на UnwindOp and OpInfo. CodeOffset указывает position в prolog after operation. В классической unwind model operations хранятся в обратном порядке относительно исполнения prolog. Поэтому decoder проверяет ordering: если новый CodeOffset больше previous CodeOffset, он ставит UA_CODEOFFSET_ORDER. Если CodeOffset выходит за SizeOfProlog, ставится UA_CODEOFFSET_OUTSIDE. Но V2 epilog descriptors не подчиняются обычной prolog-offset semantics, поэтому parser исключает их из этой проверки.

slots_for_code: маленькая функция, которая защищает весь parser

Перед тем как читать operands, parser вызывает slots_for_code. Эта функция знает, сколько slots занимает operation for given version, opcode and opinfo. Она отделяет layout logic от semantic effects. Это критично для safety: если parser неверно рассчитает slots, он либо перепрыгнет через real operation, либо начнёт читать operand там, где уже tail, либо уйдёт за CountOfCodes.

Для documented V1 operations slot count обычно простой: PUSH_NONVOL, ALLOC_SMALL, SET_FPREG and PUSH_MACHFRAME занимают one slot; SAVE_NONVOL and SAVE_XMM128 занимают two; FAR variants занимают three; ALLOC_LARGE зависит от OpInfo. Если OpInfo у ALLOC_LARGE больше 1, slots_for_code возвращает false, потому что такой mode не определён. Для V2 opcode 6 and 7 имеют другую семантику: opcode 6 is epilog descriptor, opcode 7 is spare. Первый V2 epilog descriptor может занимать разное количество slots depending on form, поэтому функция получает v2_seen_epilog.

Если opcode неизвестен, parser ставит UA_BAD_OPCODE и безопасно продвигается на one slot. Это осознанный компромисс: неизвестный opcode уже anomaly, но не всегда нужно немедленно терять всю запись. Возможно, остальные slots позволят увидеть ещё flags, tail or handler. Но если known operation заявляет operands beyond CountOfCodes, это fatal. Parser не может прочитать operand, которого нет. В таком случае ставится UA_TRUNCATED and parse fails.

UWOP_PUSH_NONVOL: сохранённый register и восемь bytes stack movement

UWOP_PUSH_NONVOL описывает prolog instruction вида push register. Для Windows x64 это один из классических способов сохранить nonvolatile GPR. При push стек уменьшается на 8 bytes, а значение register оказывается на stack. Для unwind summary это означает три вещи: fixed_stack_bytes увеличивается на 8, pushed_nonvol_count увеличивается на 1, а bit corresponding register добавляется в saved_nonvol_mask.

Слово nonvolatile здесь означает register, который callee обязан сохранить для caller согласно calling convention. В x64 Windows к таким GPR относятся RBX, RBP, RSI, RDI, R12, R13, R14, R15. Но код проекта содержит осторожную compatibility policy: is_unwindable_gpr допускает real-world records that mention volatile registers, исключая RSP. Комментарий прямо говорит, что реальные MSVC, Rust and OpenSSL records can describe volatile registers. Поэтому обычный режим не делает такой operand fatal. В strict_abi mode parser всё равно пометит ABI register class anomaly.

Это хороший пример разницы между “формально ожидаемо” и “виделось в реальном мире”. Если analyzer будет слишком догматичным, он завалит нормальные or at least working binaries. Если он будет слишком мягким, он пропустит интересные отклонения. Поэтому project splits compatibility and strict diagnostics. Summary remains useful, while anomalies carry extra signal.

UWOP_ALLOC_SMALL и UWOP_ALLOC_LARGE: как stack allocation становится числом

Stack allocation в prolog меняет RSP не через push, а через sub rsp, imm. UNWIND_INFO кодирует это двумя families operations. UWOP_ALLOC_SMALL покрывает small allocations from 8 to 128 bytes. Размер вычисляется как OpInfo * 8 + 8. Если OpInfo равен 4, allocation equals 40 bytes. Для decoder это straightforward: add_fixed_stack increases fixed_stack_bytes by computed amount.

```
UWOP_ALLOC_SMALL size = OpInfo * 8 + 8
```

UWOP_ALLOC_LARGE существует для больших allocations и имеет два modes. Mode 0, когда OpInfo равен 0, использует следующий slot как size / 8. Actual size = operand * 8, total operation length = two slots. Mode 1, когда OpInfo равен 1,

использует two following slots as full 32-bit size, total length = three slots. Любой другой OpInfo is bad operand.

Проект добавляет sanity checks для large forms. Если large mode 0 encoded size is zero, too small or not aligned as expected, parser sets UA_OPERAND_SANITY. Для mode 1 suspicious size includes zero, non-8-aligned and values smaller than 512 KB. Эти thresholds не являются Microsoft law. Это engineering sanity policy: far/large encoding для очень маленького значения выглядит странно, потому что есть compact encoding. Parser не останавливается, потому что bytes декодируемы, but report should preserve the smell.

add_fixed_stack защищён от integer overflow. Если total + amount overflowed бы uint32_t, function saturates total to UINT32_MAX and marks UA_OPERAND_SANITY. Это важно: corrupted metadata может заявить огромный allocation. Если сумма wrap-around к маленькому числу, report сказал бы “frame small”, хотя metadata actually absurd. Saturation preserves danger instead of hiding it.

UWOP_SET_FPREG: frame pointer не появляется сам по себе

Header UNWIND_INFO может задать FrameRegister and FrameOffset. Но сам факт наличия frame register должен быть подтверждён operation UWOP_SET_FPREG. Эта operation говорит, что в prolog был установлен frame pointer base. Она не добавляет stack bytes. Она меняет способ, которым unwinder interprets frame base. Поэтому parser marks saw_set_fpreg, checks that frame_reg was nonzero, and in strict mode verifies that OpInfo is reserved zero.

Если frame_reg задан, но UWOP_SET_FPREG не появился, parser после loop ставит UA_BAD_OPERAND, если record не CHAININFO. Это connection between header and operation array. Новичок часто читает header fields separately: “ну frame_reg = RBP”. Но decoder должен спросить: “а где в prolog описано установление этого RBP as frame register?” Без этой связи metadata may be inconsistent.

SAVE_NONVOL и SAVE_NONVOL_FAR: saved register не означает дополнительный stack allocation

UWOP_SAVE_NONVOL описывает другой способ сохранения nonvolatile register: не push, a store into stack slot inside an allocated frame. Например compiler мог сначала выделить stack space, а потом сделать mov [rsp+offset], rbx. Для summary это означает, что register should be marked as saved, но fixed_stack_bytes не должен увеличиваться. Allocation already описывается отдельным UWOP_ALLOC_SMALL or UWOP_ALLOC_LARGE.

Именно здесь легко ошибиться при написании decoder. Если каждое SAVE_NONVOL трактовать как “ещё один stack slot added”, summary начнёт

завышать frame size. Но unwind opcode describes location of saved value, not a new movement of RSP. Поэтому project reads scaled offset, validates operands, sets register mask, and leaves fixed_stack_bytes unchanged.

SAVE_NONVOL_FAR uses two operand slots for 32-bit offset. Проект применяет sanity: zero offset, non-8-aligned offset or surprisingly small far offset sets UA_OPERAND_SANITY. Again, this is not fatal. It means: structure can be decoded, but encoding choice or operand value looks suspicious for this form.

XMM saves: почему проект хранит отдельную mask

Windows x64 unwind metadata может описывать сохранение XMM registers. Проект хранит saved_xmm_mask отдельно от saved_nonvol_mask. Это важно, потому что GPR, то есть general-purpose registers, и vector state имеют разные ABI-смыслы и разные размеры сохранения. XMM6-XMM15 являются nonvolatile в Windows x64 ABI, а XMM0-XMM5 являются volatile. Поэтому strict_abi может пометить UA_ABI_REGISTER_CLASS, если record ссылается на подозрительный класс XMM-registers.

В коде поддерживаются UWOP_SAVE_XMM128 and UWOP_SAVE_XMM128_FAR, а также V1 opcodes 6 and 7 as observed undocumented/common SAVE_XMM and SAVE_XMM_FAR forms. This compatibility behavior must not be confused with V2 semantics. In version 2, opcode 6 means epilog descriptor, and opcode 7 means spare. The same numeric opcode therefore cannot be interpreted without version. This is why parser reads version before slots_for_code and routes opcodes through version-aware logic.

Для XMM FAR-форм проект применяет отдельную sanity-проверку: zero offset, отсутствие 16-byte alignment или подозрительно маленький far offset могут дать UA_OPERAND_SANITY. Но если operand декодируется, register всё равно отмечается в mask. Так report получает полезную сводку: function заявляет saved vector state, а metadata вокруг этого заявления выглядит необычно.

V2 epilog descriptors: почему decoder не должен притворяться, что это prolog opcode

UNWIND_INFO version 2 adds epilog descriptors. В project code opcode 6 under version 2 is handled as EPILOG, while opcode 7 is SPARE. Epilog descriptors do not describe ordinary prolog actions with CodeOffset inside SizeOfProlog. They describe epilog regions. Поэтому parser excludes them from CodeOffset outside and ordering checks used for ordinary operations.

Код отслеживает v2_seen_epilog, потому что первый epilog descriptor может использовать long или short form, а последующие descriptors интерпретируются иначе. ops_text получает читаемые строки вроде EPILOG size и offset_from_end. Но summary не превращает epilog descriptors в fixed_stack_bytes. Это не операции stack allocation. Это metadata о допустимых

epilog locations, и в текущем проекте их основная ценность - диагностика и presentation, а не агрегирование размера frame.

Эта обработка V2 показывает, почему decoder не может быть просто набором switch-case, переписанных из старой заметки. Version меняет semantics. Если parser трактовал бы V2 opcode 6 как старый SAVE_XMM, он придумал бы vector saves там, где file actually describes epilog information. Это испортило бы masks, anomalies и reports.

UWOP_PUSH_MACHFRAME: когда stack frame пришёл от exception machinery

UWOP_PUSH_MACHFRAME describes a machine frame pushed by hardware or exception mechanism. It is not the same as ordinary push nonvolatile and not the same as compiler stack allocation. OpInfo tells whether an error code is present. Without error code summary adds 40 bytes; with error code, 48 bytes. If OpInfo is greater than 1, parser marks UA_BAD_OPERAND.

Эта operation важна потому, что exception/unwind metadata описывает не только обычные function calls. Ей иногда приходится описывать frames, связанные с exception machinery. Для reverse engineering это напоминание: stack state может формироваться не только видимым compiler prolog, но и processor/OS exception mechanisms. Analyzer сохраняет это как contribution to fixed_stack_bytes, но странные operands всё равно отмечает как anomalies.

Tail after codes: handler or chain, never random decoration

After padded unwind codes array comes tail. Its interpretation depends on flags. If EHANDLER or UHANDLER is set, first 4 bytes of tail are handler RVA. If CHAININFO is set, tail contains a 12-byte RUNTIME_FUNCTION. If no relevant flags, there may be no tail relevant to summary. This is why correct after_codes calculation is essential. A two-byte padding mistake shifts handler_rva and can make dynamic collector place breakpoints at nonsense addresses.

Для handler tail проект читает uint32_t handler_rva. Если чтение не удалось, parser ставит UA_TRUNCATED и завершает parse failure, потому что flag обещал handler field, а физически его нет. Если чтение удалось, но handler_rva >= SizeOfImage, parser ставит UA_HANDLER_OOB и оставляет summary valid. Это различие важно: handler outside image - подозрительное свидетельство, но сама record была читаемой. Report может сказать именно это.

Language-specific data после handler RVA в текущем проекте не декодируется. В C++ exception handling эта область может вести к compiler-specific structures. В SEH-style mechanisms детали тоже зависят от реализации. EH/Unwind Analyzer намеренно останавливается на handler entrypoint и flags. Этого достаточно для заявленной цели: анализировать Windows x64 exception/unwind infrastructure, собирать handler targets, сопоставлять их с

runtime hits и показывать metadata placement. Это не decoder типов C++ exceptions.

Для CHAININFO проект читает из tail полный RuntimeFunction triple. Затем он проверяет end > begin, range внутри SizeOfImage, nonzero unwind RVA и unwind RVA внутри image. Нарушения дают UA_CHAIN_INVALID. Parser не требует, чтобы chain target уже выглядел идеально; он записывает chain и при необходимости следует по нему ограниченной логикой.

Chained unwind info: compact aggregation, not a perfect OS unwinder

CHAININFO exists because a function may have multiple regions that share or extend unwind descriptions. Official documentation describes chained unwind information and multiple shrink-wrapping. In project terms, chain means the current UNWIND_INFO is not the whole story. It points to another RuntimeFunction entry whose unwind info contributes to the effective model.

Когда follow_chain включён и out->has_chain равен true, проект начинает с cur = out->chain.unwind_rva. Для detection loops используется локальный массив seen[16]. На каждом шаге глубины код вызывает unwind_parse почти рекурсивно, но для внутреннего parse ставит follow_chain = false. Внешний loop контролирует traversal. Так проект избегает uncontrolled recursion и держит loop/depth policy в одном месте.

```
UnwindParseOpts o2 = {
    .max_chain_depth = 0,
    .follow_chain = false,
    .strict_abi = strict_abi
};
```

Если внутренний parse падает, parent получает UA_TRUNCATED и chain following останавливается. Если внутренний parse успешен, проект агрегирует выбранные поля summary: fixed_stack_bytes складывается, pushed_nonvol_count складывается, saved_nonvol_mask и saved_xmm_mask объединяются через OR, handler наследуется, если parent ещё не имел handler, а несовпадение frame_reg/frame_off даёт UA_BAD_OPERAND. Это практическая аналитическая агрегация. Она позволяет heuristics и reports увидеть, что effective metadata включает дополнительные saves или handler из chain.

Но это нельзя переоценивать. Такая агрегация не заменяет точный RtlVirtualUnwind для каждого instruction offset. Она не симулирует всё состояние procedure. Она не знает, какая именно область shrink-wrapped code активна для конкретного control PC. Это compact summary для static evidence. Ограничение нужно проговаривать прямо: chain data прослеживается достаточно, чтобы сохранить полезные signals, но недостаточно, чтобы доказать точный caller context во всех возможных путях.

Loop and depth controls are evidence too. If a chain loops, UA_CHAIN_LOOP. If depth limit is exceeded, UA_CHAIN_DEPTH_EXCEEDED. In normal compiler output, chained metadata should eventually arrive at a primary record. A loop or absurd depth is not automatically “malware”, but it is exactly the kind of structural weirdness a reverse engineer wants surfaced.

Anomalies: internal warning language of unwind.c

unwind.h defines anomaly bits. These are not final findings. They are local parser diagnostics that later layers can count, group and interpret. The same anomaly bit can affect different outputs: JSON can show parse_error or anomaly count; heuristics can produce unwind_parse_fail or unwind_anomalies findings; HTML can highlight a function; manual review can ask the user to inspect a specific RVA.

```
enum UnwindAnom {
    UA_VERSION_UNKNOWN       = 1u << 0,
    UA_TRUNCATED             = 1u << 1,
    UA_BAD_OPCODE            = 1u << 2,
    UA_BAD_OPERAND           = 1u << 3,
    UA_CODEOFFSET_OUTSIDE    = 1u << 4,
    UA_CHAIN_LOOP            = 1u << 5,
    UA_CHAIN_DEPTH_EXCEEDED  = 1u << 6,
    UA_HANDLER_OOB           = 1u << 7,
    UA_CODEOFFSET_ORDER      = 1u << 8,
    UA_OPERAND_SANITY        = 1u << 9,
    UA_ALIGNMENT             = 1u << 10,
    UA_CHAIN_INVALID         = 1u << 11,
    UA_ABI_REGISTER_CLASS    = 1u << 12,
    UA_ABI_RESERVED_OPINFO   = 1u << 13,
    UA_ABI_PUSH_ORDER        = 1u << 14,
    UA_ABI_CHAIN_RESTRICTION = 1u << 15,
    UA_ABI_SAVE_OFFSET       = 1u << 16
};
```

UA_TRUNCATED is broader than “file ended”. It can mean header unmappable, codes array out of file range, operand missing, handler tail missing, chain tail missing. In all cases parser needed bytes that are not safely available. UA_BAD_OPCODE means operation number was not known for that version. UA_BAD_OPERAND means fields conflict with expected semantics: invalid flags, bad frame register relation, impossible OpInfo, bad machine frame operand. UA_OPERAND_SANITY means operand is decodable but suspicious by project policy.

UA_CODEOFFSET_OUTSIDE and UA_CODEOFFSET_ORDER are especially useful for reverse engineering. They do not necessarily prevent decoding. But they say the operation sequence does not look like normal prolog metadata. Since unwind codes are supposed to describe prolog actions, offsets outside prolog or wrong ordering create doubt about whether metadata truly matches code. This doubt later becomes evidence only in context.

ABI anomalies are separate from ordinary bad operands. That separation is deliberate. A record can be decodable and even accepted by current Windows behavior, while strict formal ABI diagnostics still raise a flag. This lets the user run

the tool in strict mode for thesis/research or in more permissive mode for broad corpus analysis. The report should not collapse these modes into one simplistic label.

valid, us_ok and parse_error: three different levels of truth

В конце успешного decode parser ставит out->valid = true и возвращает true. Caller обычно сохраняет bool result в us_ok[i], а summary - в us[i]. Так появляется пара на каждый runtime function: parse succeeded или нет, и если succeeded - что именно мы узнали. us_ok легко спутать с “clean”. Это ошибка. us_ok означает, что parser не встретил fatal failure. Сам по себе он ничего не говорит об anomalies.

parse_error exists for the opposite path. If parsing fails, unwind_parse_fail sets valid=false, chooses diagnostic text for this invocation, strips trailing newline, caps it to 159 characters, and null-terminates. This is designed for reports. A JSON field with a concise parse error is more useful than a giant log blob, but it must still be specific enough: unsupported version, codes array out of file range, truncated handler tail, operation spans beyond CountOfCodes.

Эта трёхуровневая модель лежит в основе аккуратного reporting. У function может быть us_ok=false и parse_error с причиной. Может быть us_ok=true, valid=true, anomalies=0. Может быть us_ok=true, valid=true, anomalies!=0. Heuristic engine должен отличать parse failures от valid records с anomalies, потому что evidence meaning у них разный. Truncated record говорит: “я не могу безопасно декодировать заявленную metadata”. Bad operand говорит: “я декодировал fields, но они плохо согласуются друг с другом”.

ops_text: человеческий слой поверх того же decoder

Когда передан ops_text, unwind_parse выдаёт читаемое представление операций. Например, он может напечатать PUSH_NONVOL RBX, ALLOC_SMALL 40, SET_FPREG RBP + 32, SAVE_NONVOL RDI, EPILOG offset_from_end для V2 и так далее. Это не второй parser. Это presentation, построенная прямо во время parsing. Это важно: human-readable output не должен расходиться с machine summary. Тот же switch, который обновляет fixed_stack_bytes и masks, описывает operation для пользователя.

In dump mode or --ops mode, this text is a bridge for learning. A reverse engineer can see that a record at xdata RVA 0x4004 claims one ALLOC_SMALL 40 operation, prolog size 4, no frame register, no handler. Then they can compare it with disassembly. If prolog actually does sub rsp, 28h, the metadata makes sense. If disassembly and metadata diverge, that is not automatically a conviction, but it is a route for manual validation.

Проект намеренно держит ops_text optional. Reports нужны compact fields; debug sessions и учебный dump могут требовать readable operation listing. Это

та же design philosophy, что и в остальных модулях: core model остаётся structured, presentation находится отдельным слоем, а человек может запросить больше деталей без раздувания каждого JSON object.

Why unwind.c does not perform final judgment

На этом этапе decoder уже знает очень много: были ли table entries физически читаемы, поддерживается ли версия UNWIND_INFO, выглядят ли code offsets странно, находится ли handler RVA outside image, есть ли chain loop, нарушены ли strict ABI rules. Было бы соблазнительно назначить verdict прямо здесь. Проект сознательно этого не делает. Judgment оставляется heuristics.c.

Причина - composition of evidence. Одна unusual unwind section может оказаться benign. Один parse failure в маленьком crafted sample может быть test artifact. Handler target в non-executable section сильнее. Комбинация executable unwind metadata и non-executable handler target ещё сильнее. Такие сочетания требуют смотреть на все functions, sections, counts, ratios и placement. unwind.c видит record-by-record. Он может показать facts, но не должен притворяться, что понял весь case.

Такое разделение делает tests чище. unit_unwind проверяет точное parser behavior: alloc small summary, push nonvol mask, unsupported V3 fatal, truncated handler fatal, chain inheritance, alignment anomaly. unit_heuristics отдельно проверяет, сколько parse failures дают какой finding и score. Если меняется один layer, tests показывают, где именно сдвинулся contract.

Связь с heuristics.c: как summary превращается в evidence

heuristics.c receives four main inputs from previous layers: PeImage, RuntimeFunction array, UnwindSummary array and us_ok array. From unwind.c it uses parse success/failure, anomalies bitmask, handler_rva, unwind_rva and summary features. It can count unwind_parse_fail, count anomaly-bearing records, observe handler dominance, detect reuse of the same UnwindInfoRVA, and combine placement facts from pe.c with handler/unwind RVAs from unwind.c.

For example, if many runtime functions share one unwind_rva, unwind.c itself only says: these records point to same address and that address decodes this way. heuristics.c can turn concentration into unwind_reuse_moderate or unwind_reuse_strong depending on ratios and corroboration. If handler_rva points into writable section, unwind.c only provides handler_rva and has_handler. pe.c provides section characteristics. heuristics.c combines them into handler_writable_section. This is why layer boundaries matter: each layer contributes a fact family.

Evidence families, введённые раньше, становятся здесь конкретными. table_integrity в основном растёт из RUNTIME_FUNCTION layout и PE structural facts. unwind_integrity использует parse failures и anomalies. metadata_placement смотрит, куда попадают unwind_rva и handler_rva. unwind_reuse использует повторяющийся UnwindInfoRVA. handler_dominance использует концентрацию handler targets. unwind.c поставляет raw normalized observations для нескольких families, но не решает final strength один.

Связь с dynamic_win.c: почему handler_rva из static decode становится runtime breakpoint hint

Dynamic validation does not read .xdata from scratch as its primary source of handler addresses. It receives static handler RVAs gathered from parsed unwind records. During CREATE_PROCESS_DEBUG_EVENT, collector learns target base_main and can attempt to place breakpoints at base_main + handler_rva. That bridge exists only because unwind.c carefully decoded handler tail and preserved handler_rva as RVA. If handler_rva were misread due to wrong padding or wrong flags, dynamic validation would chase a false address.

This also explains why UA_HANDLER_OOB matters. If handler_rva is outside SizeOfImage, dynamic collector should not blindly treat it as a normal breakpoint target. The static report can say handler target is out of image; assessment can request manual review; dynamic layer may fail to place breakpoint or may not use it. Static decoder quality directly affects dynamic evidence quality.

At the same time, dynamic hit does not retroactively prove every field in UNWIND_INFO was correct. It can confirm that a handler address was reached in this scenario. It does not prove that all unwind slots match machine code. That is why static/dynamic correlation remains coarse. unwind.c gives the map; dynamic_win.c observes a route through one execution; assessment must keep both meanings separate.

Связь с report.c: что пользователь должен увидеть

Хороший report не должен прятать decoder за одним словом “parsed”. Для каждой function или aggregate block пользователю нужно достаточно данных, чтобы восстановить картину: begin RVA, end RVA, unwind RVA, succeeded ли parse, version, flags, prolog size, count codes, frame register, handler presence, chain presence, fixed stack estimate, saved register masks, anomalies и parse error, если он есть. В compact outputs не каждое field обязано быть на первом плане, но JSON и HTML должны сохранять модель.

The point is not to drown the user in hex. The point is to make the route auditable. If report says unwind_parse_fail, it should be traceable to a specific RuntimeFunction and specific UnwindInfo RVA. If report says handler_dominance, the user should be able to see which handler RVA appears repeatedly. If report says

unwind_writable_section, the user should be able to connect the unwind_rva to a section from pe.c. Reporting is where the layered architecture becomes visible.

Ghidra integration benefits from the same structure. Ghidra CSV does not need to reproduce every anomaly bit, but it needs stable addresses and labels/comments that help a reverse engineer navigate. Since RuntimeFunction entries are RVA-based, the importer can map them into the program image and annotate function ranges or unwind-related addresses. The CSV is a bridge from analyzer facts into manual reverse engineering workspace.

Unit tests as executable documentation for unwind.c

unit_unwind.c is more than a correctness check. It documents contract decisions. Tests cover alloc small, push nonvol, code offset order, operand sanity, real-world compatible operands, strict ABI push order and chain restriction, V2 epilog descriptors, chain inheritance, flag and frame sanity, parse_error preservation, unsupported version fatal, truncated handler fatal, chain validation and alignment.

Synthetic tests нужны потому, что позволяют проекту выставлять exact bytes и exact bits. Реальный compiler-generated PE может не дать легко получить malformed frame_reg without UWOP_SET_FPREG, chain loop или unsupported version. Synthetic buffers позволяют tests изолировать одно правило за раз. Это совпадает с философией util.c и pe.c: malformed input - ожидаемый research material, а не afterthought.

Regression tests on real files remain important too. Synthetic tests prove local parser branches. Real corpus tests prove the full path: file read, PE mapping, exception directory, runtime function table, xdata decode, heuristics, reports. For a reverse engineering tool, both are needed. A parser that passes synthetic tests but fails on ordinary MSVC output is not useful. A parser that passes real samples but lacks malformed tests is fragile.

Ограничения decoder: где проект честно не делает больше, чем обещал

unwind.c deliberately does not implement full RtlVirtualUnwind. It does not compute caller context for an arbitrary RIP. It does not emulate prolog execution at instruction-level. It does not decode language-specific handler data after handler RVA. It does not support V3 APX unwind information as V1/V2. It does not compare unwind operations against disassembly. It does not store every save offset in JSON summary. It aggregates chain information heuristically.

Эти ограничения не являются слабостью, если они проговорены честно. Цель module - декодировать static metadata в compact, reviewable model for anomaly assessment. Full OS unwinding был бы другой feature: он потребовал бы control PC, function entry lookup, context state, epilog detection, правила partial prolog execution, обработку handler data и осторожный parity with

Windows behavior. Это уже другой уровень simulator. EH/Unwind Analyzer проводит границу иначе: он читает official structures, извлекает evidence, показывает uncertainty и позволяет dynamic validation наблюдать selected runtime behavior.

Эта честность центральна для всей книги. Reverse engineering tools становятся опасными, когда выдают partial information за complete truth. unwind.c избегает этого, кодируя uncertainty прямо в data model: us_ok, valid, anomalies, parse_error, strict_abi diagnostics, chain depth limits. Report должен учить читателя той же привычке: не спрашивать слишком рано “эта function good or bad?”. Сначала спросить, какой contract заявляет metadata, что decoder смог проверить, что выглядит inconsistent, что может подтвердить runtime и что остаётся manual work.

Сквозной мысленный пример: от RuntimeFunction к UnwindSummary

Представим, что pe.c нашёл Exception Directory at RVA 0x3000 with size 0x18. unwind_read_runtime_functions_ex maps 0x3000 to file offset and sees two 12-byte entries. Первая entry говорит begin_rva=0x1000, end_rva=0x1040, unwind_rva=0x4000. Вторая говорит begin_rva=0x1040, end_rva=0x1080, unwind_rva=0x4010. Reader не решает, хорошие ли ranges. Он просто сохраняет triples.

Для первой записи unwind_parse получает unwind_rva=0x4000. pe_rva_to_fileoff maps it into .xdata raw bytes. Header says version=1, flags=0, prolog_size=4, count_codes=1, frame_reg=0, frame_off=0. Since count is odd, physical slots_even=2, so after_codes starts after four bytes header plus four bytes codes area. The one real operation slot says CodeOffset=4, UnwindOp=UWOP_ALLOC_SMALL, OpInfo=4. Summary gets fixed_stack_bytes=40. valid=true. anomalies=0. No handler. No chain.

For second record header says flags=EHANDLER, count_codes=2. Parser reads two slots, calculates after_codes including padding if needed, then reads handler_rva from tail. If handler_rva=0x1500 and SizeOfImage is large enough, has_handler=true and handler_rva=0x1500. Later dynamic collector can translate this to runtime VA base_main+0x1500. If handler_rva=0x90000000 while SizeOfImage=0x20000, parser still read the field, but marks UA_HANDLER_OOB. That difference appears in report as “handler exists, but target is outside image”, not as silent disappearance.

This example shows the whole philosophy: numbers from file become structured facts; structured facts become evidence only later; evidence becomes assessment only when combined with context. The decoder is the hinge. If it is careless, every later layer becomes theatrical. If it is conservative and explicit, the entire analyzer can remain honest.

Как Windows использует эти структуры и почему проект не должен подменять Windows

Чтобы понять границу unwind.c, полезно ещё раз представить реальный путь Windows. Когда exception dispatcher или stack walker имеет control PC, он должен найти function entry для этого PC. На x64 это обычно означает поиск подходящего RUNTIME_FUNCTION в function table. Если entry найден, его UnwindInfoAddress ведёт к UNWIND_INFO. Затем OS unwinder смотрит, находится ли control PC в prolog, body или epilog, и применяет правила восстановления context. Если entry не найден, возможна leaf-function модель: caller address может быть на top of stack. Но эта модель допустима только при правильных условиях. Analyzer не должен автоматически применять leaf assumption ко всему подряд, потому что corrupted table или скрытая generated code table могут исказить картину.

EH/Unwind Analyzer в статическом режиме не имеет live CONTEXT and exact control PC for every function. Он читает metadata заранее, как corpus static . Поэтому unwind.c не вызывает RtlVirtualUnwind and not reimplements the whole dispatcher. Он строит summary, пригодную для later reasoning. Это различие важно для пользователя: если report говорит fixed_stack_bytes=40, это не то же самое, что Windows уже восстановила caller RSP in a crash. Это означает, что static metadata claims a fixed allocation of 40 bytes in the decoded prolog model.

Но summary всё равно ценна. Она показывает, какие promises binary made to Windows. Если promises broken, Windows stack walking, exception dispatch or debugger backtraces may suffer. Если promises weird, reverse engineer gets a route for investigation. Static analyzer is therefore not pretending to be the OS; it is auditing the contract that the OS would rely on.

Почему RUNTIME_FUNCTION table не надо сразу сортировать или чинить

Новичку может показаться, что если table unsorted, analyzer должен отсортировать её и работать дальше. Но для forensic/reverse-engineering tool это опасный reflex. Сортировка исправила бы presentation, но уничтожила бы evidence of original layout. Windows expects function tables in a searchable form; toolchains normally produce ordered non-overlapping ranges. If a file has unsorted entries, overlaps or duplicate BeginAddress, это не просто неудобство для parser. Это факт о binary. Его надо сохранить.

Поэтому unwind_read_runtime_functions_ex only reads entries. It does not normalize order, does not merge ranges, does not remove duplicates, does not discard zero-sized ranges. Later heuristics can emit rf_unsorted, rf_overlaps, duplicate_begin or invalid_ranges. Report can show user how table looked in file. This is the same principle as memory forensics: do not repair evidence before recording it.

The project can still choose a representative RVA for findings, count affected entries, and maybe render a sorted view for human convenience in future versions. But internal evidence model should remember original facts. Otherwise a malicious or malformed sample could be made to look cleaner simply because analyzer helpfully rearranged it.

Register masks: почему bitmask удобнее полного списка в core summary

saved_nonvol_mask and saved_xmm_mask look small, but they are carefully chosen. Each bit corresponds to a register number. If bit 3 is set in saved_nonvol_mask, RBX appears in unwind operations. If bit 5 is set, RBP appears. If bit 12 is set, R12 appears. This compact representation is enough for aggregate reporting and heuristic checks, while avoiding per-operation storage of every save offset.

Why not store a list of all saves with offsets? Future version could. But current project optimizes for broad scanning. For many analyses the important facts are: does this record save nonvolatile registers, how many pushed registers are claimed, which register classes appear, and do operands look sane. Exact offsets are useful for deep manual verification, but not necessary for current evidence matrix. ops_text can show operands on demand.

This design mirrors the distinction between summary analyzer and interactive debugger. A debugger recovering a frame needs exact offsets. A static anomaly scanner often needs stable compact facts across thousands of functions. The bitmask provides enough structure without turning JSON into a full low-level unwind database.

Strict ABI diagnostics in practical reverse engineering

strict_abi deserves a slower explanation because it can easily be misunderstood. ABI means Application Binary Interface - the binary-level agreement between compiler, caller, callee and operating system. In Windows x64 it defines register volatility, stack alignment expectations, calling convention details and unwind metadata requirements. A strict ABI diagnostic says: this decoded metadata violates or strains that formal agreement.

But a strict ABI diagnostic is not automatically a runtime failure. Some binaries are generated by non-MSVC compilers, handwritten assembly, JITs, link-time optimizers, or unusual toolchains. Some metadata may be accepted by Windows because the actual unwinder behavior is more tolerant than a classroom model. That is why project separates strict_abi from default parse. The default parser tries to decode real-world input without panicking; strict mode asks a sharper question: “does this also look formally clean?”

UA_ABI_PUSH_ORDER is one example. In conventional prolog descriptions, pushes of nonvolatile registers have ordering expectations. If push order appears after other operations in a way strict mode dislikes, project records ABI push order anomaly. UA_ABI_CHAIN_RESTRICTION is another: chained unwind info has restrictions on which operations should appear. These are not always fatal, but they are excellent manual review clues.

This matters for thesis-quality explanation. The tool is not merely looking for malware tricks. It can also be used to study compiler output, ABI compliance, malformed samples and compatibility edge cases. A clean architecture lets the same decoder support both security-oriented and educational analysis.

Fatal и nonfatal decoder outcomes as a reader workflow

When reading a report, a reverse engineer should mentally separate fatal and nonfatal outcomes. Fatal means the parser could not safely continue for that UNWIND_INFO. Examples: unsupported version, header not mappable, codes array outside file, operand extends beyond CountOfCodes, truncated handler tail, truncated chain tail. In these cases us_ok is false and parse_error is meaningful. The right manual action is to inspect that specific xdata RVA and the surrounding section mapping.

Nonfatal means parser produced a summary but attached warnings. Examples: alignment issue, code offset outside prolog, bad operand relation, handler outside image, suspicious large operand, chain invalid. In these cases user can still read version, flags, frame info and maybe handler. The right manual action is different: compare metadata against disassembly, section characteristics and runtime behavior.

This distinction prevents a common mistake: treating every anomaly as parser failure or every successful parse as trustworthy. The analyzer exists precisely because binary reality is between those extremes. Most interesting samples are decodable enough to say something and weird enough that the statement must include caveats.

Как 11.5 связывает предыдущие главы книги с будущими разделами

All earlier theoretical pieces now meet in a concrete module. The stack model explains fixed_stack_bytes. The calling convention explains nonvolatile masks. PE mapping explains why unwind_rva must be translated through sections. Exception dispatch explains why handler flags matter. Dynamic validation explains why handler_rva becomes runtime breakpoint hint. Reporting explains why parse_error must be compact and structured. Heuristics explains why anomalies are not final findings.

Именно поэтому 11.5 должен быть подробным. Если читатель понимает unwind.c, то heuristics.c больше не выглядит магией. Finding codes вроде unwind_parse_fail, unwind_anomalies, unwind_reuse_strong, handler_dominance, placement_integrity_combo и writable_metadata_handler_combo становятся естественными следствиями фактов, уже декодированных здесь. Если читатель не понимает unwind.c, эвристики будут казаться произвольным scoring.

Следующий раздел поэтому не должен вводить heuristics как “правила из ниоткуда”. Он должен начинаться с output этого decoder: rf array, us summaries, us_ok booleans, section placement facts, counts и ratios. После этого можно объяснить, как conservative evidence engine превращает множество маленьких наблюдений в score, confidence, verdict и manual review requirement.

Итоговый смысл раздела

unwind.c - это сердце EH/Unwind Analyzer, потому что он переводит Windows x64 unwind contract в факты проекта. Он начинает с Exception Directory, читает RUNTIME_FUNCTION table, переводит каждый UnwindInfo RVA через PeImage, декодирует UNWIND_INFO header, проходит variable-length unwind code slots, интерпретирует stack and register effects, читает handler или chain tail, следует по chain в ограниченных пределах, записывает anomalies и создаёт UnwindSummary. Любое дальнейшее утверждение tool зависит от этого перевода.

Главный урок здесь - не выученный список opcodes. Главный урок архитектурный: reverse engineering analyzer должен различать reading, decoding, diagnosing и judging. Reading спрашивает, доступны ли bytes. Decoding спрашивает, что заявляет structure. Diagnosing записывает contradictions и suspicious shapes. Judging соединяет evidence families с conservative confidence. unwind.c выполняет первые три шага для .pdata/.xdata. Поэтому следующий раздел может перейти к heuristics.c без ощущения магии: эвристики стоят на конкретных facts, которые произвёл unwind.c.

11.6. heuristics.c: как evidence engine превращает unwind-факты в осторожный assessment

После pe.c и unwind.c у проекта уже есть много фактов, но еще нет вывода. Это важный момент. PE parser нашел заголовки, секции, Exception Directory и построил карту RVA. Unwind decoder прочитал RUNTIME_FUNCTION, перешел к UNWIND_INFO, разобрал unwind opcodes, отметил handler flags, chain info, parse failures и ABI warnings. Но все это пока похоже на большую техническую опись: много полей, много чисел, много частных ошибок. Реверсеру же нужно не просто увидеть тысячи строк, а понять, где находится нормальная compiler-generated metadata, где обычная странность toolchain, где повреждение файла, где возможный runtime/interception pattern, а где вообще нельзя делать уверенный вывод.

Именно для этого в проекте существует heuristics.c. Его правильнее читать не как “модуль, который ставит оценку”, а как heuristic/evidence engine. Слово heuristic переводится как эвристика: практическое правило вывода, которое не доказывает истину математически, но помогает разумно интерпретировать наблюдаемые признаки. Слово evidence переводится как свидетельство или улика, но в инженерном смысле это не криминальный приговор, а наблюдаемый факт, который поддерживает или ослабляет рабочую версию. Engine здесь означает не движок магии, а слой, где несколько независимых сигналов собираются в один аккуратный reportable assessment.

Главная идея heuristics.c: проект не должен спрашивать “это malware или нет?”. Такой вопрос слишком большой, зависит от поведения программы, упаковки, контекста, сигнатур, сети, persistence, user intent и еще десятков вещей, которых EH/Unwind Analyzer не наблюдает. Вместо этого проект задает более честный вопрос: насколько exception/unwind-инфраструктура PE32+ AMD64 выглядит структурно корректной, привычно расположенной, согласованной с официальной Windows x64 unwind-моделью и не требующей ручной проверки. Если ответ получается тревожным, report говорит suspicious, unusual, requires manual review, а не объявляет окончательный диагноз.

Эта осторожность напрямую связана с официальной документацией Windows. Microsoft описывает PE image, sections, RVA, data directories, .pdata, RUNTIME_FUNCTION, .xdata и UNWIND_INFO как контракт между compiler, linker, loader, operating system и unwinder. Но официальная документация не обязана объяснять, как отличать обычную таблицу от намеренно странной, как учитывать поврежденный sample, как не перепутать JIT/interceptor с

вредоносной логикой, и как оформить результат для реверсера. heuristics.c находится ровно в этом промежутке: он не придумывает свой формат Windows, а интерпретирует реальные Windows-структуры с защитной инженерной осторожностью.

Где heuristics.c находится в pipeline

В pipeline проекта heuristics.c не работает с сырым файлом напрямую. Это принципиально. Сырой input уже прошел через util.c, где байты читались через bounds-checked операции, и через pe.c, где они были превращены в PeImage: модель PE-файла с заголовками, секциями, image size, exception directory и RVA mapping. Затем unwind.c прочитал RUNTIME_FUNCTION entries и попытался декодировать каждый UNWIND_INFO. Только после этого heuristics.c получает normalized facts - нормализованные факты, уже приведенные к внутренним структурам проекта.

```
bool heuristics_analyze(const PeImage* img,
                        const RuntimeFunction* rf,
                        const UnwindSummary* us,
                        const bool* us_ok,
                        size_t n,
                        HeuristicReport* out,
                        StrBuf* err);
```

Эта сигнатура хорошо показывает философию модуля. img - это уже не просто массив bytes, а карта PE image. rf - массив RUNTIME_FUNCTION records, то есть диапазоны кода и ссылки на unwind metadata. us - массив UnwindSummary records, где unwind.c уже попытался перевести raw UNWIND_INFO в удобную summary-модель. us_ok - параллельный массив boolean flags: для каждой записи он говорит, удалось ли декодировать соответствующий UNWIND_INFO. n - число runtime function entries. out - место, куда будет собран HeuristicReport. err - diagnostic buffer для ошибок самого анализа, например нехватки памяти.

Здесь важно увидеть разделение ответственности. pe.c отвечает за вопрос: “можно ли понимать этот файл как PE32+ AMD64 image и как RVA переводятся в file offsets?”. unwind.c отвечает за вопрос: “можно ли прочитать RUNTIME_FUNCTION и UNWIND_INFO как unwind metadata?”. heuristics.c отвечает за другой вопрос: “какие из этих фактов похожи на норму, какие похожи на повреждение или аномалию, какие из них усиливают друг друга, а какие слишком слабы и требуют понижения confidence?”. Если перепутать эти уровни, проект быстро превратится в кашу, где parse error уже считается malware, а любая необычная секция становится приговором. В правильной архитектуре parsing, decoding, assessment и reporting разделены.

Почему это именно evidence engine, а не classifier

Слово classifier обычно означает систему, которая относит объект к классу: benign, malicious, packed, clean, unsafe. heuristics.c так не работает. Он не знает полного поведения программы. Он не эмулирует весь control flow. Он не видит сеть, persistence, криптографию, распаковку, команды оператора, user interaction и все ветки исполнения. Он видит только exception/unwind-слой и связанные с ним PE-факты. Поэтому score в HeuristicReport - это не вероятность вредоносности, а bounded engineering weight sum, то есть ограниченная сумма инженерных весов.

Это различие нужно держать в голове постоянно. Если report показывает score 70, это не значит “70 процентов malware”. Это значит: в пределах наблюдаемой exception/unwind-инфраструктуры накопилось достаточно весомых статических сигналов, чтобы verdict стал high_suspicion. Если report показывает confidence 0.80, это тоже не значит “80 процентов вероятности”. Это значит: evidence engine считает, что его статическая интерпретация относительно хорошо поддержана количеством семейств evidence, количеством medium/high findings и наличием hard integrity signals. В проекте confidence - правило доверия к интерпретации, а не статистическая калибровка.

Именно поэтому в коде рядом с весами есть комментарий о conservative engineering model. Константы сгруппированы по evidence families, а не подогнаны как machine learning model. Это значит, что reviewer может открыть heuristics.c, увидеть, почему pe_exception_directory_oob стоит 25, почему unwind_unusual_section может быть 8 или 15, почему single_signal_only снижает confidence, и обсудить правила. Модель должна быть reviewable - проверяемой человеком. Это особенно важно для дипломного/исследовательского проекта: анализатор обязан объяснять свои выводы.

HeuristicReport как центральный артефакт

Результатом работы heuristics.c является HeuristicReport. Это не просто “одна оценка”, а структурированный артефакт, который потом попадет в JSON, HTML, console output, CSV-related summary и финальный assessment. HeuristicReport содержит summary-поля, счетчики, ratios, массив structured findings, weakening rules и human-readable details.

```
typedef struct HeuristicReport {
    int    score;
    double suspicion;
    double confidence;
    bool   requires_manual_review;
    uint32_t evidence_families_mask;
    char   verdict[48];

    size_t total;
    ... counters ...
```

```
    size_t finding_count;
    HeuristicFinding findings[HEURISTICS_MAX_FINDINGS];
    size_t weak_finding_count;
    size_t medium_finding_count;
    size_t strong_finding_count;
    char evidence_level[16];

    size_t mitigation_count;
    HeuristicMitigation mitigations[HEURISTICS_MAX_MITIGATIONS];

    StrBuf details;
} HeuristicReport;
```

Поле score - число от 0 до 100. Оно растет, когда findings получают веса. Функция add_score специально clamped: итог не должен выйти за пределы 0..100. Поле suspicion - это score / 100. Оно удобно для JSON и UI, но не превращает score в вероятность. Поле confidence - отдельная величина: оно говорит, насколько evidence engine доверяет своей статической интерпретации с учетом количества семейств сигналов, силы findings и weakening rules.

Поле requires_manual_review - одно из самых честных полей report. Оно говорит: автоматический анализ дошел до места, где человек должен посмотреть руками. Это не всегда означает “плохо”. Это означает, что в exception/unwind metadata есть структура, которую нельзя оставлять без проверки. Например, unwind metadata в writable section может иметь benign объяснение в редком runtime/interceptor scenario, но для обычного compiler-produced PE это достаточно необычно, чтобы требовать manual review.

Поле evidence_families_mask - битовая маска семейств evidence. Это очень важная конструкция. Она позволяет отличить один сильный сигнал от нескольких независимых слоев. Если у нас только metadata_placement, вывод слабее. Если одновременно есть table_integrity, unwind_integrity и metadata_placement, картина становится другой. Evidence family - это не украшение отчета, а способ избежать плоской оценки, где все признаки смешиваются в одну кучу.

Массив findings содержит HeuristicFinding records. Каждый finding имеет code, family, severity, message, representative_rva и affected_count. code нужен машине и тестам. family нужен группировке evidence. severity нужен человеку и report UI. message объясняет смысл. representative_rva дает точку входа для ручной проверки. affected_count показывает масштаб: один случай и тысяча случаев - это разные инженерные ситуации. Важно, что finding не обязан доказывать вредоносность. Он фиксирует наблюдаемый сигнал.

Массив mitigations содержит weakening rules. В JSON они обычно воспринимаются как weakening_rules_applied. Это правила, которые не удаляют finding, но уменьшают confidence. Например, если найден ровно один

non-strong сигнал, применяется single_signal_only. Если sample содержит меньше 32 runtime functions, а сильных findings нет, применяется small_sample_size. Это не “оправдание”, а защита от самообмана.

Пять evidence families: почему проект не складывает все признаки в одну кучу

В heuristics.h определены пять семейств evidence: table_integrity, unwind_integrity, metadata_placement, unwind_reuse и handler_dominance. Их можно воспринимать как пять разных вопросов к sample. Каждый вопрос смотрит на exception/unwind-инфраструктуру под своим углом. Чем больше независимых углов показывают проблему, тем выше confidence. Но если проблема видна только в одном углу, особенно слабом, confidence должен снижаться.

table_integrity спрашивает: “сама таблица RUNTIME_FUNCTION и PE-разметка вокруг нее выглядят согласованно?”. Сюда попадают out-of-bounds секции, overlapping sections, malformed Exception Directory, invalid ranges, unsorted table, overlaps в function ranges, duplicate BeginAddress. Это самый базовый слой. Если table integrity сломан, Windows unwinder, static analyzer и реверсер получают нестабильную карту. Здесь речь не о поведении программы, а о том, что карта может не соответствовать image.

unwind_integrity спрашивает: “можно ли декодировать UNWIND_INFO и не противоречит ли он базовым структурным ожиданиям?”. Сюда попадают parse failures и anomalies. Это слой ближе к .xdata: запись RUNTIME_FUNCTION может выглядеть нормально, но UnwindInfoRVA может вести в мусор, в обрезанную структуру, в неподдержанную версию, в некорректный opcode layout или chain, который нельзя безопасно пройти.

metadata_placement спрашивает: “где лежит unwind metadata и куда указывают handler targets?”. Обычный compiler-generated unwind metadata чаще живет в data-like областях, например .xdata, .rdata или близких toolchain-specific секциях. Если UNWIND_INFO находится в executable или writable section, это не автоматический приговор, но это меняет характер анализа. Если handler RVA указывает в non-executable, unknown или writable section, это тоже требует внимания.

unwind_reuse спрашивает: “слишком ли много функций делят один и тот же UnwindInfoRVA?”. Повторное использование unwind metadata возможно в нормальных случаях, потому что у разных функций может быть одинаковая frame layout. Но когда reuse становится массовым, особенно на большом n, это может выглядеть как centralization, dispatcher-style metadata или low-informative unwind layer. Поэтому проект не срабатывает на каждый reuse, а использует threshold и требует контекст для moderate case.

handler_dominance спрашивает: “не доминирует ли один handler target слишком сильно?”. Один handler на множество functions сам по себе может быть нормальным для language runtime, exception framework или centralized error handling. Поэтому heuristics.c не считает dominance достаточным без условий. Он смотрит на handler_count, handler_reuse_ratio, section context и наличие других static signals. Это пример осторожной эвристики: паттерн может быть подозрительным, но он не обязан быть вредоносным.

Как heuristics_analyze проходит по данным

Работа heuristics_analyze начинается не с verdict. Сначала модуль собирает counters. Он проходит по каждому RuntimeFunction entry и смотрит на begin_rva, end_rva и unwind_rva. Если EndAddress меньше или равен BeginAddress, увеличивается rf_invalid_ranges. Если begin или end выходят за SizeOfImage, увеличивается rf_oob. Если UnwindInfoRVA равен нулю или выходит за SizeOfImage, увеличивается rf_unwind_oob. Это не сложная логика, но именно она защищает последующие выводы: если базовый range descriptor невозможен, все рассуждения о функции становятся шаткими.

Одновременно модуль проверяет порядок исходной table: если BeginAddress уменьшился относительно предыдущего entry, увеличивается rf_unsorted. Официальная x64 unwind-модель ожидает sorted table для эффективного lookup. Для реверсера unsorted table - важный сигнал: либо table построена нестандартно, либо повреждена, либо sample намеренно ломает ожидания toolchain. Но опять же, это не “malware”, а finding семейства table_integrity.

Дальше heuristics.c считает reuse. Для UnwindInfoRVA используется маленькая hash map: каждый unwind_rva становится key, а value - количеством entries, которые на него указывают. Для handlers используется отдельная hash map: если us_ok[i] true и UnwindSummary говорит has_handler, handler_rva тоже считается. После прохода модуль получает max_count, unique_count, unwind_reuse_ratio, unwind_unique_ratio и handler_reuse_ratio. Эти ratios важны, потому что они описывают не один address, а форму всей таблицы.

Параллельно идет placement analysis. Для каждого успешно декодированного UNWIND_INFO модуль вызывает pe_find_section_for_rva(img, u). Это возвращает section, в которой находится unwind metadata. Затем проверяются name и characteristics. Allowed names для unwind включают .xdata, .rdata, .pdata, .gnu_xdata, .mrdata и варианты без точки. Plausible names включают подстроки xdata, pdata, rdata. Если секция не найдена или ее имя не похоже на unwind/data placement, растет unwind_unusual_section. Если section имеет IMAGE_SCN_MEM_EXECUTE, растет unwind_exec_section. Если имеет IMAGE_SCN_MEM_WRITE, растет unwind_writable_section.

Для handler RVA применяется похожая логика, но критерии другие. Handler - это код, который Windows может вызвать при exception route. Поэтому handler target обычно должен попадать в executable section. Если section не найдена или не executable, растет handler_nonexec_section. Если section writable, растет handler_writable_section. Для реверсера эта разница важна: UNWIND_INFO обычно является metadata, handler является code target. Когда metadata находится в executable memory, это один тип вопроса. Когда handler указывает в non-executable или writable memory, это другой тип вопроса.

После основного прохода модуль отдельно проверяет overlaps и duplicate begins на sorted copy. Он создает RfIdx array, сортирует по begin/end и затем смотрит, пересекаются ли ranges. Почему это делается после отдельной sorting step? Потому что исходная table может быть unsorted, и простая проверка соседей в исходном порядке пропустила бы реальные overlaps. Проект сначала фиксирует факт unsorted input, а затем все равно строит sorted view, чтобы найти structural overlaps независимо от исходного порядка.

Только после этого начинается scoring. Это правильная последовательность: сначала наблюдение, потом aggregation, потом interpretation. Если сразу начислять score во время чтения каждой записи, легко потерять масштаб: один failure и 10 процентов failures будут выглядеть одинаково. В проекте многие severities зависят от ratio: invalid_ranges больше 1 процента становится high, unwind_anomalies больше 5 процентов становится high, unwind_unusual_section больше 5 процентов получает более высокий weight. Это делает report чувствительным не только к факту, но и к распространенности паттерна.

Thresholds: почему цифры нужны, но не являются законом природы

В heuristics.c есть несколько thresholds. THRESHOLD_HIGH_PERCENT равен 0.01, то есть больше 1 процента entries. Комментарий объясняет смысл: если проблема больше 1 процента, это уже не просто one-off glitch. THRESHOLD_UNWIND_ANOM_HIGH равен 0.05, то есть больше 5 процентов anomalies означает более широкий integrity issue. THRESHOLD_UNWIND_REUSE_MEDIUM равен 0.10, THRESHOLD_UNWIND_REUSE_HIGH равен 0.30, а THRESHOLD_HANDLER_DOMINANCE равен 0.80.

Эти thresholds нельзя читать как математические константы природы. Это инженерные границы. Они нужны, чтобы один случай не выглядел так же сильно, как массовый паттерн. Они также нужны, чтобы report оставался стабильным на больших файлах: в крупной программе может быть несколько странных записей из-за compiler/linker corner cases, но когда десятки процентов таблицы ведут себя одинаково необычно, это уже другая ситуация.

Особенно осторожно сделаны reuse heuristics. Strong UnwindInfo reuse требует ratio больше 0.30 и n больше 200. Moderate reuse требует ratio больше 0.10, n больше 200 и дополнительное corroborating static signal. Это значит, что маленький sample не будет наказан за то, что две функции используют один UnwindInfoRVA. Проект специально ждет большого n, потому что статистический смысл reuse pattern появляется только на достаточно крупной таблице.

Handler dominance тоже ограничена: handler_count должен быть больше 50, handler_reuse_ratio больше 0.80, и при этом есть логика benign context. Если доминирующий handler находится в executable code-like section и нет другого static context, finding может не добавляться. Это очень важная деталь проекта: heuristics.c умеет не только находить сигнал, но и не преувеличивать его, когда нормальное объяснение выглядит сильным.

Семейство table_integrity: когда ломается карта функций

table_integrity - самый фундаментальный слой. Он начинается даже раньше unwind decoding, потому что без нормальной PE/section/RVA карты нельзя надежно читать .pdata и .xdata. В этом семействе проект не пытается понять “зачем” sample так устроен. Он фиксирует, что структурная карта противоречит ожидаемой модели PE image или RUNTIME_FUNCTION table.

pe_section_raw_oob

pe_section_raw_oob означает, что raw range секции выходит за границы input file. Это относится к PE section table: PointerToRawData и SizeOfRawData описывают, где байты секции лежат в файле. Если этот диапазон выходит за размер файла, analyzer не может честно читать соответствующие file bytes. В проекте это high severity и +25 score, потому что out-of-bounds raw layout разрушает доверие к file-backed mapping. Это не говорит, что файл вредоносный, но говорит, что статический parser обязан быть настороже.

pe_section_virtual_oob

pe_section_virtual_oob означает, что virtual range секции выходит за SizeOfImage. VirtualAddress и VirtualSize должны укладываться в образ, который loader рассматривает как image. Если секция виртуально выходит за SizeOfImage, RVA mapping становится неоднозначным или невозможным. В проекте это high severity и +20 score. Для unwind analysis это особенно важно, потому что BeginAddress, EndAddress, UnwindInfoAddress и handler RVA являются image-relative addresses. Если image map сломана, любой RVA может вести в ложное место.

pe_section_raw_overlap и pe_section_virtual_overlap

pe_section_raw_overlap фиксирует пересечение raw ranges секций в файле, а pe_section_virtual_overlap - пересечение virtual ranges в image. Raw overlap получает medium severity и +15, virtual overlap получает high severity и +20. Почему virtual overlap сильнее? Потому что unwind reasoning почти всегда идет через RVA: где в image лежит код, где metadata, где handler. Если две секции претендуют на одну virtual область, pe_find_section_for_rva может получить ambiguous mapping. Raw overlap тоже важен, но иногда встречается в packed/overlay-like ситуациях и требует осторожной интерпретации.

pe_section_misaligned

pe_section_misaligned означает нарушение alignment/header invariants или section alignment rules. Это medium severity и +8 score. Такой finding слабее out-of-bounds, потому что alignment deviations могут встречаться в нестандартных toolchain/packer scenarios и не всегда ломают runtime mapping. Но для static analyzer это важный structural smell: если layout не похож на нормальный linker output, доверие к простому “прочитали как обычно” снижается.

pe_exception_directory_oob

pe_exception_directory_oob означает, что Exception Directory malformed, unreadable, incomplete или out of bounds. Это high severity и +25 score. Для Windows x64 unwind это один из самых важных PE-level findings, потому что Exception Directory является официальным входом к RUNTIME_FUNCTION table. Анализатор не должен искать .pdata по имени секции как главный источник истины. Он должен смотреть DataDirectory[IMAGE_DIRECTORY_ENTRY_EXCEPTION]. Если именно этот directory указывает за пределы корректно читаемой области, весь дальнейший unwind route оказывается под вопросом.

no_runtime_functions

no_runtime_functions появляется, когда n == 0, exception_rva == 0, exception_size == 0, но image имеет executable section. Это medium severity и +10 score, но message сразу предупреждает: это может быть valid для leaf-only image, поэтому confirm code complexity manually. Это хороший пример осторожного finding. На x64 leaf functions, которые не изменяют nonvolatile registers и не требуют stack allocation, могут обходиться без pdata/xdata. Но если image содержит сложный executable code, полное отсутствие RUNTIME_FUNCTION entries может мешать unwinding, exception handling и stack walking. Поэтому finding не говорит “ошибка”, а просит ручную проверку.

invalid_ranges

invalid_ranges означает, что у RUNTIME_FUNCTION entry EndAddress меньше или равен BeginAddress. RUNTIME_FUNCTION описывает диапазон кода, а

диапазон с нулевой или отрицательной длиной не может нормально выполнять эту роль. Если таких случаев мало, severity medium и +10; если больше 1 процента, severity high и +20. Для реверсера representative_rva здесь обычно BeginAddress первой проблемной записи: это точка, с которой можно начать проверку table dump и disassembly.

rf_oob

rf_oob означает, что BeginAddress или EndAddress выходит за SizeOfImage. Формально RUNTIME_FUNCTION addresses являются image-relative offsets, поэтому они должны указывать внутрь image. Если range выходит за SizeOfImage, Windows lookup и analyzer interpretation теряют основу. Как и invalid_ranges, finding получает +10 или +25 в зависимости от доли записей. Это table_integrity hard issue и дополнительно повышает confidence через hard table bonus.

unwind_rva_oob

unwind_rva_oob означает, что UnwindInfoRVA равен нулю или выходит за SizeOfImage. Это table_integrity finding, хотя он уже касается ссылки на .xdata. Почему не unwind_integrity? Потому что здесь проблема еще не в содержимом UNWIND_INFO, а в том, что RUNTIME_FUNCTION вообще указывает на невозможный RVA. Если pointer невозможен, decoder даже не получает честного места для чтения.

rf_unsorted

rf_unsorted означает, что таблица не отсортирована по BeginAddress в исходном порядке. Официальная модель и runtime lookup предполагают упорядоченную function table. Проект считает inversions и добавляет medium finding +15. Важно, что unsorted не заменяет overlap detection. Даже если таблица unsorted, heuristics.c строит sorted copy и отдельно ищет overlaps. Это показывает аккуратность анализа: разные structural problems не смешиваются.

rf_overlaps

rf_overlaps означает, что после сортировки диапазоны RUNTIME_FUNCTION пересекаются. Это high severity и +25. В нормальной функции один RIP должен попадать в один function range. Если ranges overlap, один и тот же code address может иметь несколько unwind models. Для RtlVirtualUnwind-like reasoning это опасно: разные UNWIND_INFO могут описывать разные способы восстановить RSP и nonvolatile registers. Поэтому overlap - один из сильных table_integrity signals.

duplicate_begin

duplicate_begin означает, что после сортировки несколько entries имеют одинаковый BeginAddress. Это medium severity и +10. Такой паттерн похож на особый случай overlap, но проект выделяет его отдельно, потому что одинаковый begin может быть результатом duplicate table entry, linker artifact, corruption или attempt to shadow metadata. Finding не доказывает причину, но дает конкретный code и representative_rva для проверки.

Семейство unwind_integrity: когда .xdata не читается как нормальная unwind-модель

unwind_integrity включается там, где RUNTIME_FUNCTION уже привел нас к UnwindInfoRVA, но содержимое по этому RVA не складывается в корректную или привычную UNWIND_INFO model. Это слой после pe.c и после базовой table validation. Он ближе всего к Windows x64 exception handling contract: Version, Flags, CountOfCodes, unwind opcodes, handler tail, chain info и ограничения decoder.

unwind_parse_fail

unwind_parse_fail означает, что unwind.c не смог безопасно разобрать UNWIND_INFO. Причины могут быть разные: truncated record, impossible slot layout, handler tail out of bounds, chain info out of bounds, unsupported structure, invalid read. Для heuristics.c важна не конкретная parse_error string, а сам факт: static analyzer не может построить UnwindSummary для части entries. Если failures меньше или равны 1 проценту, severity medium и +10; если больше - high и +25. Это hard unwind-integrity issue и добавляет confidence bonus.

Новичок часто делает ошибку: “parse fail значит точно обфускация”. Нет. Parse fail может быть испорченным sample, неправильным architecture assumption, неполным файлом, редкой версией metadata, багом parser или намеренным трюком. Поэтому finding говорит о structural failure, а не о мотиве. Но для reverse engineering это все равно важный сигнал, потому что Windows unwinder или analyzer не смогут пройти такой record обычным путем.

unwind_anomalies

unwind_anomalies означает, что UNWIND_INFO разобран, но содержит anomalies: version/opcode/chain/etc. В проекте важно отличие parse_fail от anomalies. Parse fail - decoder не смог безопасно построить summary. Anomaly - summary есть, но внутри отмечены необычные признаки. Severity зависит от доли: до 5 процентов medium и +10, больше 5 процентов high и +20. Это позволяет не делать слишком сильный вывод из одного необычного record, но увидеть массовый pattern.

placement_integrity_combo

placement_integrity_combo - комбинированный finding. Он появляется, когда suspicious metadata placement, например unwind metadata in executable section или handler_nonexec_section, совпадает с integrity problems: unwind_parse_fail, unwind_anomalies или rf_unwind_oob. Вес +8, severity medium, family unwind_integrity. Смысл здесь в corroboration: placement anomaly сама по себе может иметь benign explanation; parse anomaly сама по себе может быть file damage; но вместе они создают более сильную версию, что unwind layer не просто необычно расположен, а еще и структурно проблемен.

Strict ABI warnings: почему не всякое нарушение формального правила повышает suspicion

В heuristics.c есть отдельный счетчик unwind_abi_warnings. Он растет, если UnwindSummary содержит anomalies из UA_ABI_MASK: register class, reserved opinfo, push order, chain restriction, save offset и похожие strict ABI diagnostics. Но эти warnings не повышают score сами по себе. Это важное проектное решение.

Причина простая: strict ABI diagnostics полезны для глубокого аудита, но production toolchains иногда создают записи, которые формально выглядят спорно для строгой проверки, но совместимы с реальным unwinder behavior. Если каждый ABI warning превращать в suspicious finding, analyzer начнет шуметь на нормальных файлах. Поэтому strict_abi mode оставляет диагностику видимой per function и в reports, но не делает из нее самостоятельный verdict driver. Это пример того, как проект отделяет diagnostic visibility от suspicion scoring.

Семейство metadata_placement: где лежит metadata и куда ведут handlers

metadata_placement - это семейство, которое часто кажется самым “security-like”, но его нужно читать осторожно. Обычный MSVC-style image кладет unwind information в .xdata или похожие data sections, а .pdata содержит RUNTIME_FUNCTION table. Но реальный мир шире: разные linkers, runtimes, dynamic s, language frameworks и generated code могут располагать metadata иначе. Поэтому heuristics.c не говорит “не .xdata значит плохо”. Он различает allowed names, plausible names, executable bit, writable bit, handler target placement и combinations.

unwind_in_exec_section

unwind_in_exec_section означает, что UnwindInfoRVA попал в section с IMAGE_SCN_MEM_EXECUTE. Это high severity и +20. Почему это сильный сигнал? UNWIND_INFO является metadata, которую unwinder читает как данные. Если

metadata лежит в executable section, это может быть просто packing/linker artifact, но для обычного compiler-produced image это необычно. Особенно важно, что executable section часто связана с code bytes. Если analyzer видит, что bytes, которые должны описывать frame layout, живут в code-like области, реверсер должен проверить, не является ли это overlap, metadata hiding или low-informative mapping.

unwind_unusual_section

unwind_unusual_section означает, что UNWIND_INFO расположен в секции, которая не считается allowed и даже plausibly unwind-named. Allowed список проекта включает .xdata, .rdata, .pdata, .gnu_xdata, .mrdata и варианты без точки. Plausible check ищет xdata, pdata или rdata в имени. Если ничего не подходит, растет unusual count. Вес +8 или +15: если случаев больше 5 процентов entries, finding становится high-like по весу и severity high. Но если это единственный слабый placement signal, later confidence может быть понижен через placement_only_signal.

unwind_writable_section

unwind_writable_section означает, что UNWIND_INFO находится в writable section. Это medium или high в зависимости от масштаба, +10 или +18. Writable metadata особенно интересна потому, что unwind records потенциально могут меняться во время выполнения. В обычном static image compiler/linker metadata чаще read-only. Но writable placement может иметь benign reasons в generated/runtime setups. Поэтому проект не объявляет приговор, а требует context и often manual review.

handler_nonexec_section

handler_nonexec_section означает, что exception handler RVA указывает в non-executable или unknown section. Это high severity и +15. Handler - это code target, который должен быть исполняемым, если Windows действительно передаст туда управление. Если target не executable, есть несколько вариантов: metadata corrupted, analyzer section mapping wrong, handler RVA intentionally misleading, file packed, или section permissions нестандартны. В любом случае это требует ручной проверки, потому что exception route в таком виде не складывается в обычную модель.

handler_writable_section

handler_writable_section означает, что handler target попал в writable section. Это medium severity и +10. Writable + executable вместе или writable code target могут быть нормальными для некоторых JIT/runtime/interceptor схем, но для обычного PE это сильный повод посмотреть руками. Проект отделяет handler_writable_section от handler_nonexec_section, потому что writable executable section и non-executable section означают разные версии

происходящего.

placement_exec_nonexec_handler_combo

placement_exec_nonexec_handler_combo появляется, когда unwind metadata лежит в executable section, а handler target указывает в non-executable или unknown section. Это high severity и +10 поверх отдельных findings. Смысл combo не в “добавить баллы за то же самое”, а в том, что вместе эти признаки образуют более странную route model: metadata выглядит как code, а code target выглядит как не-code. Такое сочетание труднее объяснить обычной compiler-generated картиной.

writable_metadata_handler_combo

writable_metadata_handler_combo появляется, когда writable unwind metadata совпадает с writable handler targets. Это high severity и +10. В отдельности каждый writable signal уже требует внимания; вместе они говорят, что и route description, и potential handler target находятся в изменяемых областях. Для реверсера это не доказательство вредоносности, но сильная manual validation task: нужно посмотреть section permissions, loader behavior, dynamic mapping и runtime events.

Семейство unwind_reuse: когда одна unwind-модель обслуживает слишком много ranges

UnwindInfo reuse нельзя считать плохим автоматически. Если у многих функций одинаковый prolog/epilog и одинаковые stack operations, linker или compiler может переиспользовать metadata. Поэтому heuristics.c сначала считает общую форму: сколько unique UnwindInfoRVA, какой max_count, какой max_count / total. Только потом решает, достаточно ли reuse силен.

unwind_reuse_strong

unwind_reuse_strong появляется, когда больше 30 процентов entries на большом sample, n > 200, share the same UnwindInfoRVA. Severity high, weight +20. Такой pattern означает, что большая часть таблицы описана одной и той же unwind-моделью. Это может быть benign для некоторых very regular generated stubs, но в reverse engineering это также похоже на low-informative metadata layer или centralized artificial frame description. Поэтому finding сильный, но все равно не финальный приговор.

unwind_reuse_moderate

unwind_reuse_moderate появляется при ratio больше 10 процентов и n > 200, но только если есть другое static signal: handler_dominance, range issues, parse issues, placement issues и так далее. Это осознанное anti-noise решение. Moderate reuse сам по себе слишком часто может иметь нормальное

объяснение. Но moderate reuse плюс integrity/placement/handler context становится более полезным evidence. Severity medium, weight +10.

reuse_dominance_combo

reuse_dominance_combo появляется, когда heavy UnwindInfo reuse совпадает с handler_dominance. Weight +8, severity medium. Этот combo описывает возможную centralized dispatcher-style картину: много ranges делят одну metadata model и одновременно многие handler records сходятся к одному target. Такая картина может быть runtime/framework-related, но она уже достаточно специфична, чтобы report выделил ее отдельно.

Семейство handler_dominance: когда exception route сходится в одну точку

handler_dominance - один из самых тонких findings. Наивный analyzer сказал бы: если один handler обслуживает 80 процентов функций, это подозрительно. EH/Unwind Analyzer делает осторожнее. Он требует handler_count > 50 и handler_reuse_ratio > 0.80. Затем проверяет context: есть ли другие static problems, есть ли moderate reuse candidate, где лежит dominant handler, похожа ли его section на executable code-like section. Если dominant handler выглядит как обычный runtime handler в .text/code-like section и больше нет static context, finding может быть suppressed.

Это важно, потому что centralized handlers встречаются в нормальном коде: языковые runtimes, exception frameworks, crash reporting, compiler-generated personality-like handlers, SEH wrappers. Поэтому handler_dominance в проекте - не сигнал “один обработчик значит плохо”, а сигнал “один обработчик доминирует в таблице, и вокруг есть достаточно контекста, чтобы не списывать это сразу на норму”.

Если finding добавлен, он получает medium severity и +15. representative_rva указывает на dominant_handler_rva, а affected_count - на max count. Для manual review это удобно: реверсер может открыть Ghidra, перейти к handler RVA, посмотреть, является ли это обычным language runtime entry, dispatcher, thunk, decryptor-like stub, crash wrapper или вообще некорректный target.

Scoring: как score растет и почему он не равен вероятности

Scoring в heuristics.c построен как сумма весов. add_score добавляет weight и clamp-ит результат в диапазон 0..100. Веса не обучены на статистической выборке как probability model. Они отражают инженерную силу сигналов: table/unwind integrity violations получают самые сильные веса, placement anomalies - средние или высокие, reuse/dominance - умеренные, combos усиливают уже найденные patterns.

Сильные structural failures имеют веса около 20-25: pe_section_raw_oob, pe_exception_directory_oob, rf_oob high, unwind_rva_oob high, rf_overlaps, unwind_parse_fail high. Более мягкие signals имеют веса 8-15: pe_section_misaligned, unwind_unusual_section low, duplicate_begin, handler_writable_section, combo placement_integrity. Это позволяет report различать “карта реально ломается” и “есть необычный запах, но нужны еще свидетельства”.

Порог verdict построен просто: score >= 70 дает high_suspicion, score >= 40 дает moderate_suspicion, score >= 15 дает low_suspicion, иначе no_strong_static_anomalies. Эти строки нужно читать буквально. no_strong_static_anomalies не значит safe. Это значит, что именно static heuristic/evidence engine не нашел сильных anomaly signals в exception/unwind layer. low_suspicion не значит malicious. Это значит, что есть слабые или ограниченные static signals. high_suspicion не значит proof. Это значит, что accumulated static evidence достаточно сильно, чтобы требовать серьезного внимания.

```
if (out->score >= 70) verdict = "high_suspicion";
else if (out->score >= 40) verdict = "moderate_suspicion";
else if (out->score >= 15) verdict = "low_suspicion";
else verdict = "no_strong_static_anomalies";
```

Поле suspicion = score / 100 существует для удобства reporting layer. Оно позволяет HTML/JSON показать normalized value. Но нормализация не делает число вероятностью. Если реверсер видит suspicion 0.55, он должен читать это как “score 55 из 100 по правилам проекта”, а не как “55 процентов вероятность”.

Confidence: почему проект отдельно оценивает силу интерпретации

Confidence считается отдельно от score. Это один из самых зрелых элементов архитектуры. Score может быть высоким из-за одного большого finding. Но confidence должен учитывать, насколько вывод поддержан разными evidence families и насколько evidence не изолирован. Поэтому confidence начинается с base 0.15, затем получает бонусы за family_count, strong findings, medium findings, hard table integrity issue и hard unwind integrity issue. Если score == 0, confidence устанавливается в 0.20 before penalties: отсутствие findings тоже является результатом, но не абсолютной уверенностью в безопасности.

```
confidence = 0.15
+ 0.11 * evidence_family_count
+ 0.08 * strong_finding_count
+ 0.04 * medium_finding_count
+ hard_table_bonus
+ hard_unwind_bonus
- weakening_rule_penalties
```

```
clamp to 0.05 .. 0.90
```

Первая важная идея: evidence from multiple families повышает confidence. Если есть только metadata_placement, это один слой. Если одновременно table_integrity, unwind_integrity и metadata_placement, это уже cross-family picture. Вторая идея: strong findings и medium findings по-разному повышают confidence. Третья идея: hard table/unwind problems дают отдельный bonus, потому что они связаны с фундаментальной читаемостью карты.

Но затем включаются weakening rules. Они не удаляют findings и не обнуляют score. Они говорят: да, signal есть, но интерпретацию нужно ослабить. Это принципиально отличается от “игнорировать неудобные признаки”. Например, if exactly one non-strong finding exists, single_signal_only снижает confidence на 0.12. Если total < 32, findings есть, но strong_count == 0, small_sample_size снижает confidence на 0.06. Если есть только unusual placement without stronger placement/integrity context, placement_only_signal снижает confidence на 0.10. Если no strong finding и family_count <= 1, single_family_only снижает confidence на 0.08.

Итоговый confidence clamp-ится в 0.05 .. 0.90. Верхний предел 0.90 тоже важен. Статический анализатор не должен говорить “я уверен на 100 процентов” в интерпретации PE/unwind layer, потому что dynamic behavior, loader effects, runtime function tables, manual mapping и scenario coverage остаются вне полной статической гарантии. Даже сильный report оставляет место для manual validation.

Weakening rules: как проект встроил осторожность прямо в модель

single_signal_only

single_signal_only применяется, когда finding_count == 1 и strong_count == 0. Это означает: есть ровно один non-strong signal. Такой signal нельзя выкидывать, но нельзя и строить на нем уверенный вывод. Например, одна unusual section в маленьком sample может быть особенностью toolchain. Confidence penalty -0.12 заставляет report показать finding, но не раздуть интерпретацию.

small_sample_size

small_sample_size применяется, когда total < 32, есть findings, но нет strong finding. Смысл простой: ratios на маленьком n опасны. Если в sample 10 entries, один случай уже 10 процентов. Для reuse, anomalies и placement это может выглядеть громче, чем заслуживает. Penalty -0.06 напоминает реверсеру: sample слишком мал, manual review важнее автоматической уверенности.

placement_only_signal

placement_only_signal применяется, когда единственная проблема - unusual placement, без executable unwind metadata, without non-executable handler, without parse/range damage. Это очень разумное правило. Необычное имя секции само по себе часто имеет benign объяснение: toolchain-specific section, obfuscation-neutral repacking, custom linker layout. Finding остается, но confidence уменьшается на 0.10.

single_family_only

single_family_only применяется, когда no strong finding, family_count <= 1 и finding_count > 0. Это правило говорит: evidence приходит только из одного семейства, cross-family corroboration отсутствует. Penalty -0.08 помогает не путать однотипный шум с устойчивой картиной. Если позже dynamic validation подтвердит handler hit или dynamic function-table activity, final assessment сможет стать сильнее, но сама static confidence остается честно ограниченной.

Evidence level: как findings превращаются в weak, medium, strong

В HeuristicReport есть evidence_level. Это не то же самое, что verdict. Verdict связан со score thresholds. Evidence level связан с количеством и силой findings. Правило проекта такое: strong, если strong_count >= 2 или score >= 70; medium, если strong_count >= 1 или medium_count >= 2 или score >= 30; weak, если есть любые findings или score > 0; none, если findings нет и score == 0.

Зачем нужно отдельное evidence_level, если уже есть score? Потому что score может быть удобен машине, а evidence_level удобен assessment layer и человеку. Например, один high finding может дать medium static evidence даже если score еще не достиг high_suspicion. Два strong findings сразу делают evidence strong, даже если веса по каким-то причинам не дают 70. Это еще один способ избежать плоской интерпретации.

Эти уровни потом нужны для static/dynamic correlation и case_strength. Runtime side тоже имеет evidence levels: weak, medium, strong, none. Final assessment смотрит, есть ли correlation и насколько сильна каждая сторона. Поэтому heuristics.c не просто пишет “score 45”, а подготавливает статическую сторону для более общего reasoning layer.

requires_manual_review: когда автоматический вывод должен передать работу человеку

requires_manual_review устанавливается true, если score >= 20, unwind metadata находится в executable или writable sections, handler target находится в non-executable/unknown или writable sections, есть UNWIND_INFO parse failures, либо применены mitigation/weakening rules. На первый взгляд странно, что mitigation тоже включает manual review. Но это логично: если

evidence ослаблено из-за isolated signal, small sample или single family, значит автоматический вывод недостаточно силен и должен быть проверен человеком.

manual review не является “провалом автоматизации”. В reverse engineering хороший инструмент не обязан отвечать за человека. Он должен экономить время: показать suspicious region, representative RVA, affected count, family, severity, details и дальше честно сказать, что нужно открыть Ghidra, PE viewer или debugger. В EH/Unwind Analyzer именно так и устроено: heuristics.c подготавливает findings, report.c показывает их, а manual_validation_steps дают analyst следующий маршрут.

HeuristicFinding: почему structured finding важнее строки в лог

Каждый finding в проекте имеет code, family, severity, message, has_location, representative_rva и affected_count. Это намного лучше, чем просто печатать строку в console. code стабилен для tests, JSON schema, HTML filters и будущей automation. family позволяет группировать. severity помогает глазами быстро увидеть priority. representative_rva дает переход к месту. affected_count показывает масштаб.

Например, finding invalid_ranges может иметь representative_rva первого BeginAddress с End <= Begin и affected_count общего количества invalid ranges. Это значит, что report не просто пугает пользователя. Он показывает, куда идти. Если finding unwind_reuse_strong имеет representative_rva dominant_unwind_rva и affected_count unwind_max, реверсер может открыть этот UnwindInfoRVA и посмотреть, почему столько ranges на него ссылаются.

Ограничения HEURISTICS_MAX_FINDINGS = 32 и HEURISTICS_MAX_MITIGATIONS = 16 тоже важны. Analyzer не должен бесконечно раздувать report на поврежденном sample. Он агрегирует signals: один finding на тип проблемы, affected_count вместо тысячи одинаковых строк. Это делает JSON/HTML читаемыми и защищает report layer от runaway output.

Как details дополняет structured findings

Поле details - это StrBuf с human-readable findings. Его нельзя путать со structured findings. Structured findings нужны машине и report schema. details нужны человеку, который читает console или debug output и хочет видеть narrative summary. add_finding добавляет structured record и одновременно пишет строку вида [severity/family/code] message в details. Mitigations тоже попадают в details с prefix mitigation.

Если score == 0, heuristics.c добавляет note “No strong anomalies detected by heuristics.” Это не structured finding. Это важно: раньше такой текст легко было бы принять за finding code, но проектная модель аккуратнее. Отсутствие strong anomalies - informational note, а не evidence. Report должен показывать это так, чтобы человек не перепутал “нет findings” с “доказано безопасно”.

Связь heuristics.c с pe.c

heuristics.c опирается на pe.c сильнее, чем кажется. Все placement findings требуют pe_find_section_for_rva. Все section-level counters требуют корректно заполненных PeSection fields: name, virtual_address, virtual_size, raw_pointer, raw_size, characteristics. Если pe.c ошибся в RVA mapping, heuristics.c начнет делать ложные выводы о том, где лежит metadata. Поэтому предыдущий раздел про pe.c был не “подготовительным”, а фундаментальным для evidence engine.

PE-level findings в heuristics.c показывают, что модуль не ограничивается RUNTIME_FUNCTION. Он включает pe_section_raw_oob, pe_section_virtual_oob, overlaps, misalignment и exception directory OOB, потому что unwind analysis невозможен без нормальной PE image map. Это архитектурно правильно: exception/unwind metadata живет внутри PE image, а не в вакууме.

Связь heuristics.c с unwind.c

unwind.c дает heuristics.c две вещи: массив UnwindSummary и массив us_ok. UnwindSummary говорит, есть ли handler, какой handler_rva, какие anomalies были замечены, какие ABI warnings включены. us_ok говорит, удалось ли вообще декодировать record. Благодаря этому heuristics.c может различать parse failure, anomaly, ABI warning и valid handler-bearing record.

Если бы heuristics.c читал .xdata сам, проект нарушил бы separation of concerns. Тогда parsing bugs и assessment bugs смешались бы. Сейчас decoder отвечает за low-level correctness, а evidence engine отвечает за interpretation. Это облегчает testing: unit_unwind проверяет decoder behavior, unit_heuristics проверяет findings/scoring behavior, а integration tests проверяют pipeline.

Связь heuristics.c с dynamic_win.c

heuristics.c сам не запускает target и не видит runtime events. Но его findings задают static side для later assessment. Например, handler_dominance, handler_nonexec_section, handler_writable_section и placement combos создают static handler-related evidence. Dynamic collector может потом увидеть runtime handler hits, VEH/VCH/UEF registrations, RtlAddFunctionTable calls или RIP trace. Assessment layer сравнит static evidence и runtime evidence.

Это не означает, что dynamic validation “подтверждает malware”. Оно подтверждает или не подтверждает, что определенный runtime route проявился в данном сценарии запуска. Если static finding говорит “handler target suspicious”, a dynamic report показывает handler hit, case becomes correlated stronger. Если static finding есть, но runtime session не дошла до этого пути, finding не исчезает. Если dynamic session incomplete, runtime evidence capped weak и correlation не устанавливается. heuristics.c подготавливает статическую сторону этой честной модели.

Связь heuristics.c с report.c

report.c превращает HeuristicReport в видимые артефакты. Для JSON он должен сохранить machine-readable поля: score, suspicion, confidence, verdict, evidence_level, evidence_families_mask/families, counters, findings, weakening_rules_applied. Для HTML он должен показать это так, чтобы реверсер видел не только итог, но и почему итог получился. Для console он должен дать краткий readable summary. Если report теряет family, representative_rva или mitigation, смысл heuristics.c обедняется.

Поэтому reporting bugs - это не косметика. Если JSON не показывает weakening_rules_applied, пользователь может переоценить confidence. Если HTML показывает score без объяснения suspicion/confidence, пользователь может прочитать score как probability. Если Ghidra CSV не переносит нужные RVA, ручная проверка становится тяжелее. heuristics.c и report.c связаны как reasoning layer и explanation layer.

Unit tests: как проект защищает смысл эвристик от случайной поломки

В проекте есть unit_heuristics.c, unit_heuristics_combos.c, unit_heuristics_context.c, unit_heuristics_mitigations.c и unit_matrix_report.c. Их смысл не только в том, чтобы код не падал. Они фиксируют интерпретационную политику. Если кто-то изменит threshold, finding code, mitigation trigger или suppression logic, тесты должны показать, что поведение evidence engine изменилось.

Например, tests around mitigations проверяют, что isolated signals снижают confidence. Context tests важны для handler_dominance: analyzer должен не шуметь на normal executable runtime-like handler без дополнительного context, но должен срабатывать, когда dominance подкрепляется structural problems. Combo tests проверяют, что отдельные signals правильно усиливают друг друга. Это делает heuristics.c не набором случайных if, а проверяемой evidence matrix.

Как реверсер должен читать heuristic report

Правильное чтение heuristic report начинается не со строки verdict. Сначала нужно посмотреть total: сколько RUNTIME_FUNCTION entries анализировалось. Затем counters: есть ли table-level damage, parse failures, anomalies, placement counters, reuse ratios, handler_count. Потом findings: какие code, family, severity, affected_count и representative_rva. Затем confidence и weakening rules: насколько engine сам ослабил свой вывод. И только после этого verdict и evidence_level.

Если report показывает high_suspicion из-за нескольких table_integrity findings, manual route один: проверить PE layout, Exception Directory, .pdata ranges, section overlaps, RVA mapping. Если high_suspicion связан с metadata_placement, route другой: открыть sections, permissions, UnwindInfoRVA locations, handler targets. Если finding связан с reuse/dominance, route третий: смотреть dominant UnwindInfoRVA, dominant handler RVA, сравнивать functions, искать dispatcher-like patterns. Один score не говорит, куда идти. Findings говорят.

Если applied weakening rules есть, реверсер должен читать вывод еще осторожнее. single_signal_only значит: не делай большой вывод по одному признаку. small_sample_size значит: ratio может быть шумным. placement_only_signal значит: необычная секция без дополнительных проблем не является сильным evidence. single_family_only значит: пока нет corroboration. Эти правила не “обнуляют” findings, но учат правильной дисциплине анализа.

Benign explanations: почему осторожность не слабость

У многих findings есть benign explanations. no_runtime_functions может быть leaf-only image. handler_dominance может быть language/runtime framework. unwind_reuse может быть результатом однотипных stubs. unusual section name может быть custom linker. writable metadata может быть generated/runtime code scenario. strict ABI warnings могут встречаться в совместимых production records. Поэтому проект не должен драматизировать каждый сигнал.

Но benign explanation не означает “игнорировать”. Reverse engineering работает через версии. Finding предлагает версию: здесь есть необычная структура. Benign explanation предлагает альтернативную версию: это может быть runtime/toolchain artifact. Manual validation должна решить, какая версия лучше поддержана. EH/Unwind Analyzer помогает тем, что показывает evidence families, ratios, representative RVA и weakening rules. Он не заменяет мышление реверсера, а делает его дисциплинированным.

Где заканчивается heuristics.c

heuristics.c заканчивается там, где начинается окончательная cross-layer assessment. Он не знает, была ли dynamic session enabled. Он не знает, был ли

timeout. Он не знает, какие runtime events реально произошли. Он не решает case_strength сам. Его результат - static HeuristicReport. Дальше reporting/assessment layer может объединить static evidence с DynamicReport и получить static_dynamic_correlation, runtime_evidence_level, final_verdict и manual_validation_steps.

Это разделение особенно важно для trust boundary. Static analysis читает bytes и не выполняет sample. Dynamic validation запускает target как debuggee и собирает runtime evidence. Heuristics sits on static side. Если смешать эти границы, report начнет звучать увереннее, чем данные позволяют. В правильной архитектуре каждый слой говорит только о том, что он действительно наблюдал.

Мини-карта всех finding codes

В конце полезно собрать finding codes в одну карту, но читать ее нужно как закрепление после объяснения, а не как замену главы. table_integrity включает pe_section_raw_oob, pe_section_virtual_oob, pe_section_raw_overlap, pe_section_virtual_overlap, pe_section_misaligned, pe_exception_directory_oob, no_runtime_functions, invalid_ranges, rf_oob, unwind_rva_oob, rf_unsorted, rf_overlaps, duplicate_begin. unwind_integrity включает unwind_parse_fail, unwind_anomalies и placement_integrity_combo. metadata_placement включает unwind_in_exec_section, unwind_unusual_section, unwind_writable_section, handler_nonexec_section, handler_writable_section, placement_exec_nonexec_handler_combo и writable_metadata_handler_combo. unwind_reuse включает unwind_reuse_strong, unwind_reuse_moderate и reuse_dominance_combo. handler_dominance включает handler_dominance.

Эта карта показывает архитектуру мысли проекта. Сначала проверяется карта PE/RUNTIME_FUNCTION. Затем проверяется читаемость UNWIND_INFO. Затем смотрится placement metadata и handlers. Затем анализируются массовые patterns reuse/dominance. Затем findings проходят через score, confidence, weakening rules, evidence level и manual review. Именно такая последовательность превращает набор полей Windows x64 exception handling в понятный, осторожный и проверяемый static assessment.

Итог раздела

heuristics.c - это место, где EH/Unwind Analyzer становится не просто parser, а analyzer. Но он делает это аккуратно. Он не превращает parse failures в обвинения. Он не читает score как вероятность. Он не скрывает benign explanations. Он не позволяет одному слабому сигналу стать сильным verdict без weakening. Он сохраняет representative RVA, affected_count и family, чтобы реверсер мог продолжить руками. И он готовит данные для следующего слоя, где static evidence будет сравниваться с runtime evidence.

Если pe.c строит карту image, а unwind.c переводит .pdata/.xdata в unwind summary, то heuristics.c отвечает за дисциплину вывода. Он спрашивает: какие факты есть, насколько они сильны, из каких семейств они пришли, усиливают ли они друг друга, не слишком ли мал sample, не изолирован ли signal, нужна ли ручная проверка. Именно поэтому heuristic/evidence engine является сердцем проектной части: он связывает Windows contract, static analysis, report language и мышление реверсера в одну систему.

11.7. `dynamic_win.c`: как Windows dynamic collector превращает запуск процесса в runtime evidence

После `pe.c`, `unwind.c` и `heuristics.c` проект уже умеет многое: он читает PE как недоверенный массив bytes, строит карту sections, находит Exception Directory, извлекает `RUNTIME_FUNCTION`, декодирует `UNWIND_INFO`, строит `UnwindSummary` и на основе этих фактов выпускает осторожные static findings. Но у всего этого слоя есть принципиальная граница: static analysis видит то, что лежит в файле и что удалось вывести из структуры файла. Он не видит, как target ведёт себя в конкретном запуске. Он не знает, будет ли exception route реально пройден, будет ли handler вызван, зарегистрирует ли программа VEH, создаст ли dynamic function table, дойдёт ли исключение до second chance. Именно для этой границы в проекте существует `dynamic_win.c`.

Слово dynamic здесь надо читать буквально: динамический, происходящий во время выполнения. Но важно не перепутать dynamic validation с “автоматическим доказательством правды”. Dynamic collector не становится всемогущим наблюдателем. Он запускает target в конкретных условиях, с конкретными arguments, с конкретным timeout, под debugger, на конкретной Windows installation. Поэтому его результат - это runtime evidence, то есть свидетельства живого выполнения. Свидетельство может усилить static finding, может показать новый runtime-only сигнал, может ничего не увидеть, а может завершиться неполной сессией. В проектной архитектуре это не замена static analysis, а второй слой, который аккуратно присоединяется к уже построенной static model.

Официальная документация Microsoft задаёт основу этой главы. Windows Debug API описывает debug events, `WaitForDebugEvent`, `ContinueDebugEvent`, `DEBUG_EVENT`, first-chance and second-chance exception flow. Документация по structured exception handling перечисляет vectored handlers, unhandled exception filter и 64-bit dynamic function table APIs. Документация по x64 exception handling объясняет, почему runtime function tables важны для unwind на 64-bit Windows. Но документация не говорит, как построить именно EH/Unwind Analyzer. Она даёт строительные блоки. `dynamic_win.c` превращает эти блоки в специализированный collector: он не просто “запускает EXE”, а строит ограниченную debug session, ставит breakpoints в нужные места, сохраняет bounded events, отслеживает handler hits и отдаёт `DynamicReport` в reporting and assessment layer.

Windows-only граница: почему dynamic collector не является переносимой частью проекта

Статическая часть проекта переносима. `util.c`, `pe.c`, `unwind.c`, `heuristics.c` могут работать на Linux, macOS и Windows, потому что они не требуют исполнения target. Они открывают file, читают bytes, проверяют bounds, раскладывают PE-структуры и делают выводы из данных. `dynamic_win.c` устроен иначе. Он использует Windows Debug API и наблюдает Windows x64 process в момент выполнения. Поэтому в коде есть compile-time branch: под `_WIN32` собирается реальный collector, вне Windows собирается unsupported-platform stub. Это не техническая мелочь, а архитектурное признание границы: нельзя честно “поддержать dynamic validation Windows exception flow” на платформе, где этот flow не выполняется нативно.

```
#ifdef _WIN32
    /* real Windows dynamic collector */
#else
    /* unsupported-platform stub */
#endif
```

На non-Windows host проект не пытается притворяться debugger. Он инициализирует report, пишет понятное объяснение, что dynamic mode поддерживается только на Windows, и возвращает failure/unsupported state. Это лучше, чем молча выдавать пустой runtime report. Пустой report легко прочитать как “ничего не найдено”. Unsupported report говорит другое: “этот слой знания не был получен”. В книге мы уже много раз отделяли `none` от `unknown`; здесь эта разница становится частью архитектуры.

С практической точки зрения это означает простую таблицу возможностей. Linux или macOS host могут разобрать Windows PE32+ AMD64 file статически, но не могут проверить runtime exception route этого файла через текущий collector. Windows x64 host может выполнить оба слоя: static analysis и dynamic validation. Wine, VM, emulator или отдельный remote worker могли бы стать будущими extension points, но текущая версия проекта не заявляет такой backend. Поэтому в report dynamic section должен честно показывать, был ли слой enabled, был ли target реально запущен, была ли сессия ok, был ли timeout, завершился ли target самостоятельно.

Defensive intent: что collector делает и чего он принципиально не обещает

В `dynamic.h` цель сформулирована как defensive intent. Это важная фраза. Defensive значит защитный, исследовательский, направленный на сбор evidence, а не на обход защиты, извлечение скрытых данных или автоматическую деобфускацию. Collector наблюдает несколько классов runtime-событий: hits handlers from `UNWIND_INFO`, registration and hits of VEH/VCH/UEF, registration of dynamic function tables, first-chance and second-chance exception flow. Он не обещает распаковать protected sample, не обещает победить anti-debug, не обещает восстановить виртуальную машину

обфускатора. Он только строит ограниченную, воспроизводимую debug session вокруг exception/unwind-механизмов.

Такое ограничение нужно не для красоты. Если инструмент начинает говорить “я запускаю sample, значит теперь я знаю его реальное поведение”, он сразу вводит реверсера в заблуждение. Любое выполнение зависит от input, environment, time, privileges, files, network, UI, locale, user profile, debugger presence. Target может вообще вести себя иначе под debugger. Поэтому правильная формулировка такая: dynamic collector наблюдает то, что произошло в данном запуске, в пределах установленной instrumentation и storage limits. Это runtime evidence, а не глобальное доказательство поведения программы во всех возможных сценариях.

Модель `DynamicReport`: не verdict, а журнал сессии

Центральный результат `dynamic_win.c` - структура `DynamicReport`. Её легко прочитать неправильно, если искать в ней одно поле “плохой / хороший”. Такого поля там нет и быть не должно. `DynamicReport` - это журнал сессии. Он хранит состояние запуска, счётчики наблюдений, ограниченный массив structured events, ограниченный trace и human-readable details. Затем `report.c` сериализует это в JSON/HTML, а assessment layer аккуратно переводит набор runtime signals в `runtime_evidence_level`, `dynamic_signal_count` и возможную static/dynamic correlation.

```
typedef struct DynamicReport {
    bool enabled;
    bool ran;
    bool ok;
    bool timed_out;
    bool target_exited;
    uint32_t target_exit_code;

    uint32_t timeout_ms;
    uint32_t trace_steps;

    bool instrumentation_complete;
    uint32_t modules_observed_peak;
    uint32_t module_refresh_failures;
    uint32_t module_list_truncations;
    uint32_t unwind_handlers_discovered;
    uint32_t breakpoint_set_attempts;
    uint32_t breakpoint_set_failures;
    uint32_t breakpoint_capacity_drops;
    uint32_t pending_rearm_failures;
    uint32_t breakpoint_rearm_failures;
    ...
} DynamicReport;
```

Первые поля описывают life state. `enabled` означает, что dynamic mode был запрошен и report инициализирован. `ran` означает более сильную вещь: `CreateProcessA` succeeded, target process действительно был создан. `ok` означает не “target хороший”, а “collector получил CREATE_PROCESS event,

debug loop не сломался внутренней ошибкой и timeout не наступил". `timed_out` говорит, что сессия остановлена по timeout. `target_exited` говорит, что был получен `EXIT_PROCESS_DEBUG_EVENT`. `target_exit_code` - это exit code target, но он не является оценкой безопасности. Ноль может быть у suspicious sample, nonzero может быть у обычной утилиты, которой не передали нужные arguments.

Поля instrumentation важны не меньше, чем сами hits. `instrumentation_complete` показывает, удалось ли collector полноценно поставить и поддерживать наблюдение. Если module list was truncated, breakpoint capacity exhausted, pending re-arm failed или breakpoint re-arm failed, report не должен выглядеть как полноценное "ничего не увидели". Он должен сказать: наблюдение было неполным. Это одна из ключевых инженерных идей проекта: качество evidence зависит не только от target, но и от качества instrumentation.

`modules_observed_peak` показывает максимальное число modules, которое collector видел в target. `module_refresh_failures` фиксирует failures при обновлении module map. `module_list_truncations` показывает, что target загрузил больше modules, чем помещается в текущий fixed-size `ModList`. `unwind_handlers_discovered` - число unique handler RVA, найденных static layer и переданных dynamic layer как candidates для breakpoint. `breakpoint_set_attempts` и `breakpoint_set_failures` описывают попытки инструментирования. `breakpoint_capacity_drops` показывает переполнение BpList. `pending_rearm_failures` и `breakpoint_rearm_failures` особенно важны, потому что software breakpoint - это не одноразовое действие: после hit его надо снять, выполнить original instruction, а затем поставить обратно.

Дальше идут counters runtime-событий: exceptions, first chance, second chance, unwind handler hits, VEH registrations and hits, VCH registrations and hits, UEF set calls and hits, dynamic function table calls. Их нельзя читать как "чем больше, тем хуже". Десять first-chance exceptions могут быть нормальной частью программы. Один second-chance exception может быть ожидаемым smoke-test scenario. Один `RtlAddFunctionTable` может быть нормой для JIT/runtime-generated code. Смысл появляется только после связи с static findings, target type, scenario и report context.

События и trace: bounded evidence вместо бесконечного лога

`DynamicEvent` намеренно компактный. Он хранит `type`, `module`, `thread_id`, `exception_code`, `first_chance`, `address` и четыре generic аргумента `arg0..arg3`. Этот формат не идеален семантически, потому что meaning arguments зависит от event type. Для `veh_registration` `arg0` - это `First`, `arg1` - handler pointer. Для `rtl_add_function_table` `arg0` - table

pointer, `arg1` - count, `arg2` - base address. Но compact event model проще сериализовать и валидировать на текущем этапе проекта. В будущей версии typed subobjects сделали бы JSON яснее, но потребовали бы schema migration.

```
typedef struct DynamicEvent {
    char type[40];
    char module[64];
    uint32_t thread_id;
    uint32_t exception_code;
    bool first_chance;
    uint64_t address;
    uint64_t arg0;
    uint64_t arg1;
    uint64_t arg2;
    uint64_t arg3;
} DynamicEvent;
```

Events ограничены `DYNAMIC_MAX_EVENTS = 256`. Когда массив заполнен, новые события не перезаписывают старые, а `dropped_events` увеличивается. Это важный выбор. Если инструмент перезаписывал бы старые events, реверсер мог бы потерять начало сессии: create, registration, first handler hit. Если инструмент хранил бы бесконечный log, target мог бы атаковать его event volume. Текущая модель говорит: мы храним первые 256 structured events и явно признаём потери через `dropped_events`. Поэтому `dropped_events > 0` всегда ослабляет уверенность в полноте dynamic evidence.

Trace тоже ограничен 256 instruction pointers. В CLI пользователь может попросить `trace_steps`, но collector clamps value до 256. Это не full execution trace. Это короткая RIP trace после первого handler hit. Она нужна не для реконструкции всей программы, а для ответа на более узкий вопрос: когда мы попали в handler, куда начал двигаться control flow сразу после entry? Первый trace element - handler entry RIP до выполнения original instruction. Дальше single-step добавляет последующие RIP values, пока budget не исчерпан. Повторные handler hits продолжают учитываться counters/events, но новый trace не начинается, чтобы report оставался bounded and predictable.

Command line: почему `CreateProcessA` не принимает готовый `argv[]`

Перед запуском target collector должен построить command line. На Unix-like системах привычна модель `argv[]`: отдельный массив arguments уже сохранён kernel boundary. Windows `CreateProcessA` устроен иначе: функция принимает mutable command-line string. Поэтому project не может просто соединить strings через spaces. Path с пробелом, пустой argument, кавычки, trailing backslashes - всё это меняет interpretation. Ошибка quoting превратила бы dynamic validation в другой эксперимент: target получил бы не те arguments, не тот input, не тот mode.

Функция `append_cmd_arg` реализует quoting по правилам Microsoft C runtime: добавляет quotes для empty string, spaces, tabs или internal quotes; удваивает backslashes перед quote; удваивает trailing backslashes перед closing quote. Это выглядит как бытовая мелочь, но для analyzer это часть reproducibility. Если report говорит “target был запущен с такими-то arguments”, эти boundaries должны соответствовать тому, что target реально увидел в своём startup code.

```
eh_unwind_analyzer target.exe --dynamic -- --input sample.dat

before -- : arguments of analyzer
 after -- : arguments of target program
```

Separator `--` в CLI отделяет arguments analyzer от arguments target. До `--` находятся options вроде `--dynamic`, `--timeout-ms`, `--trace-steps`, `--json`. После `--` всё относится к child process. Metadata report по умолчанию redacts child arguments, потому что в них могут быть secrets: tokens, file paths, user names, test data. Redaction в report не отменяет того, что actual CreateProcess command line содержит эти arguments. Это разделение между execution contract и report privacy.

CreateProcess and debug boundary: момент, где static tool становится debugger

Главная граница dynamic mode проходит через `CreateProcessA`. До этого момента analyzer работал с file bytes. После этого он создаёт живой process. Target получает CPU time, memory, access к user-mode environment, может читать files, обращаться к network, создавать windows, проверять debugger presence, зависать. Поэтому dynamic mode нельзя запускать как “просто ещё одну галочку” на произвольном upload. Это должно происходить в disposable Windows VM или hardened sandbox, с ограничениями на privileges, network and resources. `DEBUG_ONLY_THIS_PROCESS` - это debug flag, не sandbox.

```
CreateProcessA(
    NULL,
    cmd.s,
    NULL,
    NULL,
    FALSE,
    DEBUG_ONLY_THIS_PROCESS,
    NULL,
    NULL,
    &si,
    &pi
)
```

Проект использует `DEBUG_ONLY_THIS_PROCESS`. Смысл флага: debugger получает debug events именно от созданного process, но child processes этого target автоматически не становятся debuggees этого collector. Это разумное ограничение для текущей версии. Если suspicious target создаст child process и

перенесёт туда exception-based logic, текущий collector этого child process не увидит. Это не баг отчёта, если report честно ограничен текущей моделью; это limitation, которую реверсер должен понимать.

Код намеренно не использует `CREATE_SUSPENDED`. Комментарий в `dynamic_win.c` фиксирует практическую проблему: в некоторых configurations suspended debuggee не выдавал ожидаемый `CREATE_PROCESS_DEBUG_EVENT`, что приводило к deadlock. Вместо этого collector полагается на стандартный Debug API event stream. Windows создаёт debuggee и доставляет early debug events, включая create-process event and initial breakpoint, до обычного продолжения выполнения через `ContinueDebugEvent`. Это позволяет collector получить `lpBaseOfImage`, module map и поставить breakpoints достаточно рано.

После успешного `CreateProcessA` field `out->ran` становится true. Это сильная граница в report. Если `ran=false`, значит target не был создан. Для DLL/PYD это ожидаемо: `CreateProcess` не запускает library как executable. Для EXE failure может означать missing dependencies, elevation requirement, invalid image, wrong subsystem, broken path. В любом случае runtime evidence отсутствует не потому, что target “ничего не сделал”, а потому, что dynamic session не началась.

Debug loop: как collector слушает Windows event stream

После создания процесса начинается debug loop. Это сердце collector. Windows Debug API работает через события. Debuggee делает что-то значимое для debugger: process created, DLL loaded, exception occurred, thread exited, process exited. Windows suspends relevant debuggee activity, заполняет `DEBUG_EVENT`, debugger анализирует событие и вызывает `ContinueDebugEvent`, чтобы сказать системе, как продолжать. Поэтому dynamic collector - это не polling memory и не “просмотр логов”. Это event-driven loop around official Windows debugging contract.

```
for (;;) {
    if (elapsed > timeout_ms) { ... timed_out ... break; }

    if (!WaitForDebugEvent(&ev, 100)) {
        if (GetLastError() == ERROR_SEM_TIMEOUT) continue;
        ... internal failure ...
    }

    DWORD cont = DBG_CONTINUE;
    ... handle event ...
    ContinueDebugEvent(ev.dwProcessId, ev.dwThreadId, cont);
}
```

`WaitForDebugEvent` вызывается с интервалом 100 ms. Если события нет, Windows returns timeout indication, collector просто идёт на следующую iteration и снова проверяет hard timeout. Поэтому timeout granularity примерно

100 ms плюс время обработки текущего event. Если возникает другой error, debug loop считается failed. В report это должно отличаться от timeout: timeout говорит “target/session не завершились в budget”, internal failure говорит “collector сам потерял способность корректно продолжать observation”.

Continuation status - одна из самых опасных частей Debug API. Для обычного exception, который не принадлежит собственному breakpoint machinery collector, проект использует `DBG_EXCEPTION_NOT_HANDLED`. Это значит: debugger сообщает Windows, что он не обработал исключение, и Windows должна продолжить normal exception dispatch inside target. Именно так сохраняется target behavior: first-chance exception после debugger notification идёт дальше к VEH, SEH, filters, unhandled path. Если collector ошибочно вернул бы `DBG_CONTINUE` для чужого exception, он мог бы проглотить исключение и изменить control flow target.

Для собственных служебных breakpoint and single-step events collector возвращает `DBG_CONTINUE`, потому что эти exceptions созданы instrumentation. Если передать собственный `INT3` как not handled, target получил бы exception, которого в его original code не было. Поэтому distinction critical: чужие exceptions проходят дальше как `DBG_EXCEPTION_NOT_HANDLED`; свои технические exceptions поглощаются collector через `DBG_CONTINUE`. На этом различии держится честность dynamic validation.

CREATE_PROCESS_DEBUG_EVENT: где static RVA впервые встречается с runtime base

Первое ключевое событие - `CREATE_PROCESS_DEBUG_EVENT`. Оно даёт collector actual base address main image через `lpBaseOfImage`. Это runtime base, не обязательно равный preferred `ImageBase` из PE header, потому что ASLR может загрузить module по другому адресу. Именно здесь static world начинает связываться с runtime world. Static parser знал handler RVA. Dynamic collector получает base_main. Сумма `base_main + HandlerRVA` превращает static handler location в target VA, куда можно попытаться поставить breakpoint.

```
HandlerVA = base_main + HandlerRVA
```

Здесь снова важно не смешивать levels. `HandlerRVA` пришёл из `UNWIND_INFO` и был проверен static parser as file/image metadata. `base_main` пришёл из live debug event. `HandlerVA` - адрес в address space debuggee. Если handler RVA invalid, или section not mapped executable, или page inaccessible, `bp_add_and_arm_selfcheck` может fail. Проект рассматривает static unwind handler breakpoints as best effort. Failure поставить breakpoint на конкретный handler не завершает сессию, но влияет на instrumentation completeness through counters if it was an attempted breakpoint.

На create-process event collector также refreshes module list and tries to set API breakpoints. Потом он закрывает `hFile` from debug event. Закрытие handles - часть Debug API hygiene. Debugger получает handles через events и обязан их закрывать, когда они больше не нужны. Иначе dynamic validation itself начнёт leak resources, a long corpus runs станут нестабильными.

Module map: почему адрес API нельзя просто взять из процесса анализатора

Чтобы поставить breakpoint на `AddVectoredExceptionHandler` или `RtlAddFunctionTable`, недостаточно вызвать `GetProcAddress` in analyzer process и использовать этот address in target. Из-за ASLR modules in analyzer and target can have different bases. Кроме того, Windows APIs can be forwarded: symbol may be requested from `kernel32.dll`, but actual code can live in `kernelbase.dll`. Поэтому проект строит module map target process и переносит не absolute VA, a offset inside real module.

`ModList` хранит basename, base and size для modules target process. Он обновляется через Toolhelp: `CreateToolhelp32Snapshot`, `Module32First`, `Module32Next`. Имена приводятся к lowercase, потому что module lookup не должен ломаться из-за case. Capacity - 128 modules. В текущей версии есть field `truncated`, и `mods_refresh_accounted` увеличивает `module_list_truncations`, если module list не помещается. Это improvement относительно модели, где truncation silently disappeared. Но всё равно это fixed-size limit: если нужный module оказался за пределами list, API breakpoint может не быть поставлен.

```
typedef struct ModInfo {
    char name[MAX_PATH];
    uint64_t base;
    uint64_t size;
} ModInfo;

typedef struct ModList {
    ModInfo mods[128];
    size_t count;
    bool truncated;
} ModList;
```

Forwarded API resolution выполняется через `local_proc_loc`. Сначала код берёт preferred module, например `kernel32.dll`, затем `GetProcAddress(symbol)`, затем `GetModuleHandleExA(...FROM_ADDRESS...)`, чтобы понять real local module, которому принадлежит returned function pointer. После этого вычисляется offset: local function VA minus real local module base. В target module map ищется module with same basename, и target breakpoint address получается как target module base plus offset.

```
offset = LocalFunctionVA - LocalRealModuleBase
TargetFunctionVA = TargetModuleBase + offset
```

Эта схема практична для одного Windows installation, где analyzer and target process use compatible copies of system DLLs. Она не является remote export parser. Если target использует module substitution, unusual loader behavior, side-by-side redirection, WOW64 mismatch или другой binary layout, offset transfer can be wrong. Именно поэтому self-check logs are important: report должен показывать not only “we wanted to set breakpoint”, but where and whether byte became `0xCC`.

Наблюдаемые APIs: почему выбраны именно эти five hooks

Collector ставит API breakpoints на пять calls: `AddVectoredExceptionHandler`, `AddVectoredContinueHandler`, `SetUnhandledExceptionFilter`, `RtlAddFunctionTable`, `RtlInstallFunctionTableCallback`. Это не произвольный список. Эти functions лежат рядом с темой книги: exception dispatch, top-level exception handling, runtime-generated unwind metadata. Static analyzer может увидеть `.pdata/.xdata` on disk. Но program at runtime может add process-wide handlers or add dynamic function tables for generated code. Если мы хотим понять exception/unwind behavior, эти calls становятся естественными runtime observation points.

`AddVectoredExceptionHandler` регистрирует VEH - vectored exception handler, process-wide handler layer before normal frame-based unwind. `AddVectoredContinueHandler` регистрирует VCH - vectored continue handler, слой продолжения после exception processing phase. `SetUnhandledExceptionFilter` задаёт top-level unhandled exception filter. `RtlAddFunctionTable` registers dynamic function table, needed on 64-bit Windows for dynamically generated code to participate in stack unwinding. `RtlInstallFunctionTableCallback` registers callback-based function table, where entries can be supplied dynamically on demand. Вместе эти hooks покрывают runtime ways to redirect exception flow or extend unwind knowledge beyond static `.pdata`.

Hook здесь означает не import table hook и не patching API prolog with trampoline. В текущем collector это обычный debugger software breakpoint at target API entry. Когда target reaches API, CPU triggers breakpoint exception, debugger stops, collector reads register arguments according to Windows x64 calling convention, records event, optionally sets a second breakpoint on handler pointer, then rewinds RIP and lets original API instruction execute. Это важно: collector наблюдает call entry and arguments, но не заменяет API and не меняет return value.

Breakpoint model: почему один byte `0xCC` требует целой state machine

Software breakpoint кажется простым: записать `0xCC` по нужному address. Но надёжный debugger не может ограничиться этой фразой. Нужно сохранить original byte, отличать armed and temporarily cleared state, поддерживать multiple logical meanings for same address, re-arm after original instruction, учитывать threads and failures. В `dynamic_win.c` всё это сконцентрировано вокруг `Breakpoint` and `BpList`.

```
typedef struct Breakpoint {
    uint64_t addr;
    uint8_t  orig;
    bool     armed;
    bool     rearm_pending;
    uint32_t kinds;
} Breakpoint;

typedef struct BpList {
    Breakpoint bps[512];
    size_t count;
} BpList;
```

`addr` - target VA. `orig` - original byte, который debugger временно заменяет на `0xCC`. `armed` значит, что memory at address should contain breakpoint instruction. `rearm_pending` значит, что breakpoint уже hit, original byte восстановлен, original instruction ещё должна выполниться, и после single-step breakpoint надо поставить обратно. `kinds` - bitmask roles. Один address может одновременно быть, например, API breakpoint and custom handler breakpoint in an unusual case. Поэтому existing breakpoint не дублируется; its kinds are OR-ed.

Capacity BpList - 512 entries. При переполнении `bp_add_and_arm` returns false, а `bp_add_and_arm_selfcheck` увеличивает `breakpoint_capacity_drops` and marks instrumentation incomplete. Это правильнее, чем silently ignoring. Реверсер, читающий report, должен понимать: если breakpoint capacity exhausted, absence of handler hits becomes weaker. Возможно, collector просто не смог поставить enough observation points.

Breakpoint self-check - сильная сторона implementation. После первой попытки поставить breakpoint code reads byte back and writes deterministic line into `DynamicReport.details`: OK with byte `0xcc`, FAIL, or OK but peek failed. It formats location as module+offset and absolute VA. Это делает instrumentation auditable. Без self-check report говорил бы “я пытался поставить breakpoint”, но реверсер не видел бы, оказался ли byte реально modified in target.

Remote memory write: почему debugger временно меняет page protection

Чтобы поставить software breakpoint, collector должен изменить memory target process. Сначала `write_one_byte` вызывает `WriteProcessMemory`. Но code pages usually mapped execute-read, not writable. Если direct write fails, collector

временно вызывает `VirtualProtectEx(..., PAGE_EXECUTE_READWRITE)`, повторяет `WriteProcessMemory`, затем возвращает old protection and calls `FlushInstructionCache`. Это не случайность, а стандартная проблема debugger instrumentation: чтобы CPU начал исполнять patched instruction byte, instruction cache must be coherent with modified memory.

Но здесь есть trust and safety consequence. На короткое время target page становится RWX. Потом project restores old protection, но сам факт вмешательства может быть заметен anti-debug-sensitive target. Target может проверять page permissions, code bytes, timing, debugger state. Поэтому dynamic collector не claims stealth. Он собирает evidence through controlled debugger intervention. В отчёте и в главе важно говорить именно так: observed behavior occurred under debugger with software breakpoints, temporary memory protection changes and instruction-cache flushes.

Это также объясняет, почему dynamic validation should run in isolated environment. Target is untrusted executable, and collector modifies its address space. If target is malicious, it can react to debugger, spin events, intentionally hit breakpoints many times, or try to exploit weaknesses in debugger logic. The project mitigates with bounded arrays, timeouts and cleanup, but timeout is not sandbox, and Debug API is not containment.

Hit собственного INT3: rewind RIP, trap flag, re-arm

Когда CPU executes `0xCC`, Windows reports breakpoint exception. На x64 address in exception record corresponds to breakpoint address for debugger lookup in this implementation, and comment notes the usual debugger issue: after INT3, RIP points after breakpoint instruction. Чтобы выполнить original instruction, debugger must restore original byte and set `RIP` back to breakpoint address. Otherwise execution would skip first byte of original instruction and corrupt target control flow.

`dynamic_win.c` handles own breakpoint hit as a small state machine. It finds breakpoint by address, clears it by restoring original byte, marks it rearm pending, stores pending re-arm for current thread, opens event thread, reads `CONTEXT`, sets `ctx.Rip = addr`, sets Trap Flag in `EFlags`, records event according to breakpoint kind, writes context back, and continues with `DBG_CONTINUE`. Trap Flag causes CPU to execute one instruction and then raise single-step exception. On that single-step, collector can re-arm breakpoint by writing `0xCC` back.

```
HIT
-> restore original byte
-> set pending re-arm for current thread
-> GetThreadContext
-> ctx.Rip = breakpoint address
-> ctx.EFlags |= 0x100    /* Trap Flag */
-> SetThreadContext
-> ContinueDebugEvent(DBG_CONTINUE)
-> EXCEPTION_SINGLE_STEP
```

```
-> re-arm breakpoint
-> optionally continue RIP trace
```

Per-thread pending re-arm matters. With multiple threads, a single global “pending breakpoint” would be wrong. Thread A can hit breakpoint and be prepared for single-step, while Thread B exists independently. `PendingRearmList` stores up to 64 entries keyed by thread ID. One thread can have one pending breakpoint. If 64 threads are simultaneously between breakpoint hit and single-step, insertion fails and report marks `pending_rearm_failures`. Again, this is not just an implementation error; it weakens instrumentation completeness.

`OpenThread` uses access rights for context get/set. `GetThreadContext` retrieves selected registers based on `ContextFlags`; collector uses `CONTEXT_CONTROL | CONTEXT_INTEGER`, because it needs `Rip`, `EFlags` and x64 integer argument registers such as `Rcx`, `Rdx`, `R8`, `R9`. `SetThreadContext` writes modified control state back. If any of these steps fails, debug loop is marked failed. A debugger that cannot read/write thread context cannot safely handle its own breakpoint.

Register arguments: why RCX/RDX/R8/R9 become event fields

Windows x64 calling convention passes the first four integer/pointer arguments in `RCX`, `RDX`, `R8`, `R9`. This is why collector can capture API arguments at function entry without disassembling target stack. At breakpoint on API entry, before original instruction executes, registers still contain call arguments. `AddVectoredExceptionHandler` uses `RCX = First`, `RDX = Handler`. `SetUnhandledExceptionFilter` uses `RCX = Handler`. `RtlAddFunctionTable` uses `RCX = FunctionTable`, `RDX = EntryCount`, `R8 = BaseAddress`. `RtlInstallFunctionTableCallback` uses all four: `RCX = TableIdentifier`, `RDX = BaseAddress`, `R8 = Length`, `R9 = Callback`.

```
AddVectoredExceptionHandler:
  RCX = First
  RDX = Handler

SetUnhandledExceptionFilter:
  RCX = Handler

RtlAddFunctionTable:
  RCX = FunctionTable
  RDX = EntryCount
  R8  = BaseAddress

RtlInstallFunctionTableCallback:
  RCX = TableIdentifier
  RDX = BaseAddress
  R8  = Length
  R9  = Callback
```

Collector records these values as event args. But it does not observe return value. For VEH/VCH registration, this means event `veh_registration` or `vch_registration` proves that target called the registration API with a handler pointer. It does not prove registration succeeded. A stronger collector would place a return breakpoint, read return value and correlate success. Current implementation may later confirm success indirectly if handler breakpoint is hit. Поэтому report wording must stay careful: registration call observed, handler hit observed, success not guaranteed unless hit or return-value support exists.

For `RtlAddFunctionTable`, current collector records pointer, count and base address but does not read remote function table entries. This is a deliberate limitation. Full remote table analysis would require safely bounding EntryCount, reading remote memory, validating `RUNTIME_FUNCTION` array, resolving unwind records against supplied base, checking page protections, tracking lifetime via `RtlDeleteFunctionTable`, and handling races. Current runtime evidence says: target registered a dynamic function table. It does not say: table entries are valid, suspicious, executable, or mapped to a particular generated code region.

For `RtlInstallFunctionTableCallback`, the limitation is even stronger. Callback-based tables can provide entries on demand. Observing the API call gives table identifier, base, length and callback pointer. But it does not tell which functions will be returned later, nor when Windows will call callback, nor what entries callback will produce. A future version could set breakpoint on callback and snapshot returned entries. Current report should call this dynamic function table activity, not decoded runtime unwind model.

VEH, VCH and UEF: three different runtime handler routes

`AddVectoredExceptionHandler` and `AddVectoredContinueHandler` are easy to lump together, but project keeps counters separate. VEH belongs to exception handling phase: it can see exceptions before frame-based SEH search. VCH belongs to continue phase: it is part of the continuation side of exception processing. UEF, set by `SetUnhandledExceptionFilter`, is top-level/unhandled path, and active debugger can affect whether and how it is reached. A report that collapses all three into “custom handler” would lose important route information.

When collector sees VEH registration and handler pointer is nonzero, it attempts to set breakpoint on that custom handler address. If later hit occurs, `veh_hits` increments and event `veh_handler_hit` is stored. Same for VCH and UEF. This produces a two-level runtime story. First level: target registered or set handler. Second level: handler actually executed in this run. These levels have different evidence strength. Registration-only can be weak runtime evidence. Handler hit is stronger, because execution reached the handler entry.

UEF requires extra caution. Microsoft documents top-level unhandled exception filter in relation to unhandled exception flow, and debugging can change that path.

A target running under debugger may not exercise UEF exactly as it would outside debugger. Поэтому `uef_set_calls` and `uef_hits` are valuable but scenario-bound. If UEF hit is observed, it is real for this debugged run. If UEF hit is not observed, that does not disprove top-level behavior in another non-debugged run.

Ordinary exceptions: preserving target dispatch instead of stealing control flow

For non-breakpoint and non-single-step exceptions, collector records event and returns `DBG_EXCEPTION_NOT_HANDLED`. This is the correct conservative behavior. It lets Windows continue normal exception dispatch: first-chance debugger notification, then vectored handlers, then frame-based SEH/unwind search, then second chance if nobody handled it. If collector swallowed ordinary exceptions, it would turn itself into a behavior-changing component, not observer.

The event stores exception code, address, first_chance flag and module by address. It does not serialize `ExceptionInformation[]`. That means access violation parameters - attempted address and read/write/execute type - are not captured in current structured event. This is a known limitation. For reverse engineering, exception code and address are helpful, but access violation details would greatly improve analysis of guard pages, probing, deliberate faults and invalid memory tricks. Future typed event schema could add those fields.

The collector allows one unknown breakpoint as initial system breakpoint. Windows commonly generates initial breakpoint when debuggee starts. Since this breakpoint is not in BpList, collector ignores the first unknown breakpoint and continues. Later unknown breakpoints are passed as not handled. This is pragmatic, not perfect. A target could execute its own early `INT3` before expected system sequence. More precise logic could identify initial breakpoint by module/address/event ordering. Current approach is acceptable for practical validation but should be understood as a heuristic inside dynamic machinery.

LOAD_DLL and UNLOAD_DLL: module map is a moving target

Dynamic process does not load all modules at create-process time. DLLs can appear later. Therefore on each `LOAD_DLL_DEBUG_EVENT`, collector refreshes module list and retries API breakpoint setup. This matters for forwarded API resolution: real module containing target API may not be present yet. Repeated setup is deduplicated by address, so if breakpoint already exists, kinds are merged and duplicate `0xCC` is not inserted.

On `UNLOAD_DLL_DEBUG_EVENT`, collector refreshes module list but does not remove breakpoints from unloaded modules. This is a limitation. If module was unloaded, its pages may be unmapped; final `bp_clear` will fail best-effort. If

another module later maps at the same address, stale breakpoint model could become dangerous. Current code does not track per-module breakpoint lifetime. Report does not currently expose stale-breakpoint invalidation. This is a real improvement area, especially for targets with plugin loading/unloading or deliberate DLL churn.

This is also why module map cannot be read as complete truth. It is snapshot-based. `modules_observed_peak` and truncation counters tell something about complexity of target process, but they do not provide full historical memory map. A future runtime memory protection map, module lifetime model and breakpoint invalidation mechanism would make dynamic evidence stronger.

Static handler RVAs as dynamic breakpoint hints

One of the most important bridges between static and dynamic layers is the list of handler RVAs. `unwind.c` decodes `UNWIND_INFO`; if a record has handler flag and parse succeeded, `UnwindSummary` contains `has_handler` and `handler_rva`. Before debug loop collector builds unique list of these handler RVAs. It does not set one breakpoint per runtime function if many records share same handler; it deduplicates handler addresses. This prevents unnecessary duplicate breakpoints and maps naturally to handler dominance analysis from static heuristics.

The unique check is linear against already collected handlers. Worst case is O(n^2) if many unique handlers exist. For typical PE files this is acceptable. For a huge crafted PE with handler for every record, a hash set would scale better. But the current implementation trades complexity for simplicity and bounded memory. The result is stored as `unwind_handlers_discovered`, and successful placements increment `unwind_handler_bp_set`. If `unwind_handlers_discovered` is high and `unwind_handler_bp_set` low, report suggests instrumentation weakness or invalid/unmappable handler addresses.

A hit on `BP_UNWIND_HANDLER` is especially meaningful because it connects static `.xdata` handler metadata to runtime execution. Static layer said: this `UNWIND_INFO` points to handler RVA. Dynamic layer says: in this run, execution reached that handler address. This is one of the strongest correlation shapes in the project. Still, even here wording must be precise. It confirms handler entry was reached in this debugged run. It does not prove maliciousness, full semantics of handler, or all possible exception routes.

RIP trace: short path after handler hit, not full execution reconstruction

When a handler hit occurs and `trace_steps > 0`, collector starts single-step trace. It records handler entry RIP immediately after rewinding, then uses Trap Flag to

record subsequent instruction pointers. The trace belongs to one thread and only one global trace session exists per report. If another handler hit happens later, counters and events update, but trace is not restarted. This design keeps report stable and bounded but limits coverage.

A novice might see `trace` and imagine instruction-by-instruction execution log. It is not. It is a small window after first handler hit. It can show whether handler immediately jumps to a dispatcher, returns, calls a helper, enters an unusual region, or walks through a stub. But it does not show memory writes, register values, branch conditions, call targets beyond RIP addresses, exception parameters, or all thread activity. It is a clue, not a decompiler.

Trace can also be affected by debugger mechanics. Single-step itself generates `EXCEPTION_SINGLE_STEP`; collector uses that event both to re-arm breakpoints and to continue trace. If tracing thread exits, trace state resets. If `GetThreadContext` or `SetThreadContext` fails, trace cannot continue safely. Therefore `trace_count = 0` can mean no handler hit, trace disabled, failed context access, or no budget; one must read surrounding fields and details.

Timeout and process exit: why `ok=false` can still contain useful evidence

Timeout is not failure to write report. It is a dynamic session result. If target hangs, waits for UI, runs server loop, sleeps, detects debugger, or simply needs more time, collector stops after `timeout_ms`. It adds structured timeout event, marks `timed_out`, sets `ok=false`, and cleanup terminates target if it did not exit by itself. But partial `DynamicReport` remains available. This is essential. A target may register VEH and hit handler before timeout. Throwing away partial events would destroy evidence.

At process exit, collector records `process_exit` with exit code and breaks only after `ContinueDebugEvent`, respecting Debug API lifecycle. After loop, it removes breakpoints best-effort. If target did not exit, it calls `TerminateProcess` and drains cleanup debug events for up to around one second rather than waiting a fixed long interval. This makes corpus runs faster and avoids losing final exit event when possible.

`ok` is computed as `got_create && debug_loop_ok && !timed_out`. Therefore timeout makes ok false even if many observations were collected. Internal failure makes ok false even if target ran. No CREATE_PROCESS event makes ok false because base_main and initial instrumentation did not happen. A reader must not reduce dynamic report to ok alone. `ok` describes session health. Events describe observations. Assessment decides how much they can support.

Runtime evidence level: categories, not raw volume

Report layer does not convert every event into separate evidence strength. It computes dynamic signal diversity. Categories include any unwind handler hits, any VEH registration or hit, any VCH registration or hit, any UEF set or hit, any dynamic function table API call, and any second-chance exception. Maximum category count is six. Ten VEH hits still represent one category. This prevents event spam from inflating evidence strength.

Runtime evidence level follows engineering taxonomy. Handler hit is strong runtime component. Dynamic table registration and second-chance exception are medium components. Handler registrations and first-chance exceptions are weak components. The resulting level can be strong, medium, weak or none depending on combination. This is not probability. It is a controlled vocabulary for reporting: how much runtime activity relevant to exception/unwind behavior was observed in this scenario.

This model explains why dynamic-only evidence can exist without static correlation. Suppose target calls `RtlAddFunctionTable`, but static `.pdata` is normal. That may indicate JIT-like behavior or generated code; it may be benign. Suppose static analyzer flags suspicious handler dominance, and dynamic collector observes hit on handler referenced by `.xdata`. That creates a stronger correlated case. Suppose static analyzer finds strong anomalies but dynamic session has `ran=false`. That remains static evidence, not disproved. Correlation is conservative and coarse, not per-function proof graph.

`instrumentation_complete`: the field that prevents false calm

One of the most important fields in modern `DynamicReport` is `instrumentation_complete`. It starts true. Several problems can turn it false: module refresh failure, module list truncation, event drops, breakpoint set failure, breakpoint capacity exhaustion, pending re-arm failure, breakpoint re-arm failure. This field is not about target maliciousness. It is about collector coverage. It tells the reader: “The dynamic session ran, but observation had holes.”

Why does this matter? Because absence of events is only meaningful if instrumentation was actually in place. If API breakpoints were not set because module map failed, then no `veh_registration` event tells little. If handler breakpoints failed, no handler hit tells little. If events were dropped, later critical events may be missing. `instrumentation_complete=false` should push assessment and manual review toward caution. It is the dynamic equivalent of static parser saying “parse_error” or “unknown section mapping”.

Report fields: как читать `dynamic_validation` глазами реверсера

In JSON schema, `dynamic_validation` requires fields like `enabled`, `ran`, `ok`, `timed_out`, `target_exited`, `target_exit_code`, `timeout_ms`, `trace_steps`, instrumentation counters, event counters, `dynamic_signal_count`, `dropped_events`, `details`, `events`, `trace`. This explicit schema matters. It prevents report writer from omitting uncomfortable fields. A report without `dropped_events` could hide event overflow. A report without `instrumentation_complete` could make incomplete session look clean. A report without `ran` could confuse unsupported/launch failure with no evidence.

A disciplined reader starts from session health: enabled, ran, ok, timed_out, target_exited. Then instrumentation: module refresh failures, truncations, breakpoint failures, event drops. Only after that should one interpret events. Then counters: did exceptions happen, first or second chance, did handlers register, did handlers hit, did dynamic function table APIs appear. Then trace: if handler hit occurred, what immediate control-flow window was captured. Finally details: self-check lines, errors, API resolution messages. This order prevents the common mistake “events array empty means nothing happened”.

HTML report should guide human eyes through the same order. JSON report should preserve machine-checkable contract. Console output can be short, but JSON/HTML must carry enough context for audit. If dynamic collector fails after partial data, report must still serialize partial `DynamicReport`. This is why `main.c` passes dynamic report into JSON/HTML even when dynamic collection returns false. Failure is evidence about collection, not a reason to delete evidence already gathered.

Security boundary: debugger is not sandbox

It is tempting to treat dynamic mode as harmless because “we run under debugger”. This is wrong. Debugger gives observer power, but it does not contain target. Target can read and modify files available to its user account, access network if allowed, spawn child processes, allocate memory, consume CPU, detect debugger, and deliberately generate events. `DEBUG_ONLY_THIS_PROCESS` only limits automatic debugging of child processes; it does not prevent child creation. Timeout stops a long run eventually; it does not prevent damage before timeout.

Therefore production use should place dynamic worker in disposable Windows VM or sandbox. It should not run arbitrary user uploads as privileged service account. Network should be restricted. File system should be isolated. Samples should be copied into a throwaway working directory. Reports should be collected out-of-band. This is not outside the book topic; it is part of honest architecture. A tool that executes untrusted code must design operational boundary, not only parser boundary.

The project’s current defensive measures are useful but not sufficient as sandbox: bounded events, bounded trace, timeout, report preservation, cleanup. They make

collector more robust as a debugger, not safe as an execution environment. This distinction should be explicit in any deployment documentation and final project defense.

Dynamic smoke targets and empirical validation

The project includes focused Windows test programs: `dyn_veh.c`, `dyn_vch.c`, `dyn_uef.c`, `dyn_rtl_ftable.c`. They compile into synthetic targets used to check that collector observes expected categories: VEH registration/hit, VCH registration/hit, UEF set/hit, dynamic function table calls. These tests are important because dynamic collector cannot be fully validated on macOS/Linux CI. Static unit tests can check parser logic anywhere; Debug API branch needs Windows x64 host.

The documented Windows validation from June 2026 is meaningful because it covers more than synthetic happy paths. It includes synthetic and real PE files, schema validation, semantic validation, dynamic sessions that exit normally, timeout cases, DLL/PYD ran=false cases, elevation-required cases, and focused expected-signal scenarios. The important result is not “all targets clean”. The important result is that collector contract held across varied outcomes: reports were produced, schema passed, trace budget respected, dropped events stayed zero in that run, and focused dynamic scenarios produced expected evidence.

This is exactly how such a tool should be validated. Dynamic collector is not proven correct by one demo. It needs corpus behavior, failure behavior, timeout behavior, schema behavior and regression scripts. A debugger often fails at edges: process creation failure, module load timing, event flood, thread exit between breakpoint hit and re-arm, target exit while trace active. Tests should keep these edges visible.

Что current implementation делает хорошо

Сильная сторона `dynamic_win.c` - separation of concerns. It does not decode PE again; it borrows `PeImage`, `RuntimeFunction`, `UnwindSummary`, `us_ok[]` from static pipeline. It does not decide final verdict; it fills `DynamicReport`. It does not pretend unsupported platforms can validate Windows execution; it reports unsupported. This separation keeps architecture readable: static model creates hypotheses, dynamic collector observes runtime events, report/assessment layer explains and combines.

Вторая сильная сторона - boundedness. `DYNAMIC_MAX_EVENTS`, trace limit 256, module list capacity with truncation accounting, breakpoint list capacity with drops, pending re-arm failures, timeout. Boundedness is not only resource management; it is part of threat model. Target can be adversarial. Infinite logging, unlimited allocation or unbounded trace would make collector fragile.

Третья сильная сторона - preservation of target exception dispatch. Ordinary exceptions are passed as not handled. Own breakpoints are handled. This difference sounds small, but it is the difference between observer and

behavior-changing participant. A dynamic validator for exception routes must be especially careful here, because the thing being observed is exception dispatch itself.

Четвёртая сильная сторона - auditability. Self-check lines show breakpoint placement. Details preserve human-readable debug story. Structured events preserve machine-readable story. Partial reports survive failure. This is important for VCR/defense/demo: the reader can see why runtime evidence was or was not collected.

Что остаётся улучшить

The most important future improvement is remote parsing of dynamic function tables. `RtlAddFunctionTable` event currently records table pointer, count and base. The next step would be to read remote `RUNTIME_FUNCTION` array, validate count and address range, decode referenced `UNWIND_INFO`, snapshot page protections and report dynamic metadata as first-class runtime unwind model. That would move dynamic table evidence from “API call observed” toward “runtime function table content analysed”.

Second improvement: return breakpoints for registration APIs. For VEH/VCH/UEF and dynamic function table functions, entry arguments are useful, but return value tells success or previous handler. A robust return-breakpoint mechanism must handle call return addresses, threads, recursion and API failure. It would make `veh_registration` stronger by distinguishing attempted registration from successful registration.

Third improvement: typed dynamic events. Generic `arg0..arg3` are compact, but readers must know mapping. A schema with event-specific objects would be clearer: `veh_registration.first`, `veh_registration.handler`, `rtl_add_function_table.table`, `entry_count`, `base_address`. Backward compatibility can keep generic args while adding typed fields in schema version bump.

Fourth improvement: exception parameters and context snapshots. Access violations without `ExceptionInformation[]` lose attempted address and operation. Handler hits without minimal register snapshot lose immediate state. Full context dumps may be too heavy or sensitive, but selected fields would help investigation. The design must balance evidence richness, privacy, report size and schema stability.

Fifth improvement: module unload handling and breakpoint invalidation. Current collector refreshes module list on unload but does not remove breakpoints in unloaded modules from BpList. Tracking module lifetime would prevent stale breakpoint model. Runtime memory protection map would also make handler and API breakpoint placement easier to interpret.

Sixth improvement: isolation as product architecture. The current collector is a local debugger. A production-grade dynamic validation service should run in disposable VM workers with snapshot reset, network policy, file-system policy, privilege limits and clear artifact extraction. This belongs outside `dynamic_win.c` source file, but inside project architecture.

Event type dictionary: как понимать `type` and `arg0..arg3` без гадания

Чтобы `DynamicReport.events` был полезен не только автору проекта, нужно явно держать в голове словарь event types. `exception` означает обычное target exception, не принадлежащее breakpoint machinery collector. В нём `exception_code` and `first_chance` meaningful, `address` - exception address, module заполняется по current module map. `process_exit` означает observed `EXIT_PROCESS_DEBUG_EVENT`; exit code лежит в `arg0`. `timeout` - synthetic collector event, not Windows debug event; timeout budget lies in `arg0`. Эти events описывают lifecycle and exception background of session.

Registration events описывают calls at API entry. `veh_registration` uses `arg0 = First`, `arg1 = Handler`. `vch_registration` has same shape. `uef_registration` uses `arg0 = Handler`. These events are not handler execution. They say target attempted to register or set a handler pointer. If handler pointer is nonzero, collector tries to set custom handler breakpoint. If later `veh_handler_hit`, `vch_handler_hit` or `uef_handler_hit` appears, runtime story becomes stronger: not only registration call happened, but handler entry was executed.

Dynamic function table events have different shape. `rtl_add_function_table` uses `arg0 = FunctionTable`, `arg1 = EntryCount`, `arg2 = BaseAddress`. `rtl_install_function_table_callback` uses `arg0 = TableIdentifier`, `arg1 = BaseAddress`, `arg2 = Length`, `arg3 = Callback`. These fields are raw runtime pointers or sizes from x64 argument registers. They are not automatically validated PE RVAs. In reports they should be printed as hex addresses where appropriate and interpreted as target virtual addresses, not file offsets.

Handler hit events are execution evidence. `unwind_handler_hit` means a breakpoint placed from static `UNWIND_INFO` handler RVA was reached. `veh_handler_hit`, `vch_handler_hit`, `uef_handler_hit` mean breakpoints set from runtime registration pointers were reached. For these events `address` is the handler entry address in target VA. Module name is best-effort from module map. For static unwind handler, details may also include RVA and section from `addr_to_loc`, because main-image base and `PeImage` are known.

A concrete reading scenario: static handler finding plus dynamic handler hit

Представим report, где static layer found handler dominance: many `RUNTIME_FUNCTION` records point to the same handler RVA. Static evidence alone says: unwind metadata is unusual, because a large part of function table funnels exceptions to one handler. That can be benign in some compiler/runtime patterns, or suspicious in exception-based dispatch. Now dynamic layer runs target and records `unwind_handlers_discovered = 1`, `unwind_handler_bp_set = 1`, `unwind_handler_hits = 1`, event `unwind_handler_hit`, and trace with 32 RIPs. This does not magically prove malicious control flow, but it changes the case: the static handler was not only present in metadata, it was reached in this scenario.

A careful assessment then says: static evidence family `handler_dominance` is runtime-confirmed by handler hit. If static evidence level was medium or strong, runtime handler hit can produce correlated case. Manual validation should open handler address in Ghidra, compare trace RIPs to disassembly, inspect whether handler dispatches by exception code, frame data or hidden state, and check whether this route bypasses normal call graph. The dynamic collector supplies witness; analyst supplies interpretation.

A concrete reading scenario: dynamic table activity without static anomaly

Другой report: static `.pdata/.xdata` looks normal. `score` low, no strong static anomalies. Dynamic session observes `rtl_add_function_table_calls = 1`, event `rtl_add_function_table`, maybe no handler hit. This should not be forced into “suspicious correlated case”, because there is no suspicious static table to correlate. It may be JIT, runtime code, language runtime, plugin engine, or protective code. Project should call it runtime signal, maybe `runtime_signals_only` or weak/medium runtime evidence depending on assessment rules, and ask for manual validation if target type makes dynamic function tables surprising.

Manual validation here is different. The analyst should ask: what module called `RtlAddFunctionTable`; what is `BaseAddress`; is it inside allocated executable memory; is `EntryCount` plausible; does target later execute in that region; does process use a runtime known to generate code. Current collector does not answer those questions fully because it does not parse remote table entries. The report should not overclaim. It should preserve pointer, count and base so a future debugger session or memory dump can continue the investigation.

A concrete reading scenario: timeout with partial evidence

Третий report: `ran=true`, `timed_out=true`, `ok=false`, but events show `veh_registration` and `veh_handler_hit` before timeout. A shallow reader might reject the dynamic result because `ok=false`. Correct reading is subtler. Session health was incomplete because timeout occurred, so absence of later events is weak. But observed events before timeout remain valid witnesses. Assessment can

use handler hit as runtime evidence while also applying caution because session did not reach natural process exit.

This is why `ok` and evidence counters must not be collapsed. `ok=false` says the collector session did not complete cleanly. It does not erase `veh_hits`. Conversely, `ok=true` with no events does not prove no exception tricks exist; it only says the chosen run completed without observed dynamic signals under current instrumentation. Both cases require context. The report schema exists to keep these distinctions visible.

How official Windows contracts shape project choices

Several choices in `dynamic_win.c` become obvious when placed next to official Windows contracts. `WaitForDebugEvent` returns a `DEBUG_EVENT`, so collector is event-loop based. `ContinueDebugEvent` requires a continuation status, so collector must decide carefully between `DBG_CONTINUE` and `DBG_EXCEPTION_NOT_HANDLED`. Debugging events include process, DLL and exception events, so collector updates module map on create/load/unload and handles exception events separately. x64 calling convention places first pointer arguments in registers, so collector reads `CONTEXT` at API entry instead of trying to reconstruct stack arguments.

Structured exception handling documentation lists vectored handlers, top-level filter and 64-bit runtime function table APIs, so collector chooses these as observation points. x64 exception handling documentation explains that function tables are central to unwinding on 64-bit Windows, so `RtlAddFunctionTable` is not random noise; it is runtime mutation of the unwind universe. The project architecture is therefore not arbitrary. It is a focused implementation of exactly the Windows mechanisms the book has been teaching.

Where dynamic collector stops and manual debugger work begins

A good automated collector must know when to stop. `dynamic_win.c` does not disassemble handler body, does not emulate exception filters, does not reconstruct generated code, does not inspect all memory maps, does not collect all register states, does not follow child processes, does not bypass anti-debug, and does not prove handler semantics. Those omissions are not automatically flaws. They are boundaries. The right output is not a pretend final answer, but a report that says where to look next.

Manual debugger work begins when report shows a meaningful address or contradiction. If `unwind_handler_hit` appears, open handler in Ghidra and debugger. If `rtl_add_function_table` appears, inspect table pointer and generated code memory. If `second_chance_seen > 0`, inspect exception code and address, then reproduce with more logging. If `instrumentation_complete=false`, improve

scenario or collector configuration before trusting absence of events. If `dropped_events > 0`, reduce event volume or increase model capacity in a controlled way. The collector does not replace these steps; it prioritizes them.

Итоговая mental model: dynamic collector как маленький debugger с узкой задачей

К концу этого раздела важно удержать главное: `dynamic_win.c` is a specialized user-mode debugger, not a universal runtime analyzer. It creates target under Windows Debug API, maps modules, resolves forwarded APIs, plants software breakpoints, handles its own INT3 and single-step events, preserves normal exception dispatch, captures selected handler registrations and hits, records dynamic function table calls, saves bounded events and bounded trace, then hands `DynamicReport` to reporting and assessment layers.

Его state machine можно описать так: report initialized, command line built, process created, waiting debug event, handling event, continuing event, waiting again, then exited, timed out or failed, then breakpoint cleanup, handle cleanup, report ready. Inside breakpoint branch lives another state machine: armed, hit, original restored, RIP rewound, Trap Flag set, original instruction executed, single-step, re-armed. If reader does not understand this nested state machine, he will read dynamic report superficially. If understands, report becomes transparent: every field corresponds to a real step in debugger lifecycle.

```
UNINITIALIZED
-> REPORT_INITIALIZED
-> COMMAND_BUILT
-> PROCESS_CREATED
-> WAITING_EVENT
-> HANDLING_EVENT
-> CONTINUING_EVENT
-> WAITING_EVENT
-> EXITED | TIMED_OUT | FAILED
-> BREAKPOINT_CLEANUP
-> HANDLE_CLEANUP
-> REPORT_READY

ARMED
-> HIT
-> ORIGINAL_RESTORED
-> RIP_REWOUND_AND_TF_SET
-> ORIGINAL_INSTRUCTION_EXECUTED
-> SINGLE_STEP
-> REARMED
```

This is why dynamic validation in EH/Unwind Analyzer must be read together with static analysis. Static layer tells what unwind metadata declares. Dynamic layer tells what this run exercised. Heuristic/evidence engine tells how static signals should be weighted. Assessment tells whether cross-layer correlation exists. Reports preserve the path. Manual review continues where automation must stop. In a good reverse-engineering workflow, `dynamic_win.c` does not replace the analyst. It

gives the analyst a live, bounded, auditable set of runtime witnesses.

Практический итог для чтения `DynamicReport`

Когда реверсер открывает dynamic section, правильный вопрос звучит не “нашёл ли collector что-то плохое?”. Правильный вопрос: какая часть execution boundary была реально пройдена, насколько полной была instrumentation, какие runtime witnesses появились, и как они соотносятся со static findings. Поэтому чтение начинается с `enabled`, `ran`, `ok`, `timed_out`, потом переходит к `instrumentation_complete`, failures and drops, потом к events and counters, и только после этого к `runtime_evidence_level` and `case_strength`. Такой порядок защищает от ложного спокойствия и ложной тревоги.

Если dynamic report чистый, но target не запускался, это не clean runtime. Если target запускался, но instrumentation incomplete, это не сильное отсутствие evidence. Если handler hit есть, но static findings слабые, это runtime fact, который требует ручного контекста. Если static findings сильные, а dynamic session ничего не увидела, это не отмена static case; это может быть неподходящий scenario. Именно поэтому проект хранит и static, и dynamic, и assessment, а не пытается заменить всё одним verdict.

В финальной архитектуре `dynamic_win.c` занимает место живого свидетельства. Он не читает PE лучше, чем `pe.c`; не декодирует unwind глубже, чем `unwind.c`; не оценивает suspicion вместо `heuristics.c`; не объясняет человеку report вместо `report.c`. Его сила в другом: он переводит некоторые static questions в runtime checks. Был ли этот handler reached? Был ли VEH registered? Был ли dynamic function table added? Дошло ли exception до second chance? Эти вопросы нельзя честно закрыть одним file parser.

Поэтому хороший отчёт после dynamic validation должен оставлять реверсеру не эмоцию, а маршрут работы: какие адреса открыть, какие events перепроверить, какие breakpoints поставить вручную, какие сценарии запуска повторить, какие ограничения учесть. Это и есть роль dynamic collector в EH/Unwind Analyzer: не финальный судья, а аккуратный слой наблюдения между официальной Windows exception/unwind моделью и живым поведением конкретного процесса.

11.8. report.c: как reporting layer превращает анализ в рабочий артефакт

После pe.c, unwind.c, heuristics.c и dynamic_win.c у проекта уже есть почти всё, что нужно для технического вывода: PE-карта, массив RUNTIME_FUNCTION, декодированные UNWIND_INFO, статические findings, dynamic events, trace, counters, confidence, evidence levels. Но всё это ещё не является полезным результатом для реверсера. Пока данные живут внутри C-структур, они существуют только в памяти процесса eh_unwind_analyzer. Пользователь не может на них сослаться, Ghidra не может их импортировать, regression test не может сравнить старый и новый результат, а другой инструмент не может проверить, что смысл полей не поменялся. Именно поэтому report.c - не второстепенный файл вывода, а последний инженерный слой анализа.

В этой главе слово report переводится не просто как "отчёт", а как зафиксированное свидетельство работы анализатора. Report - это форма, в которой внутреннее знание проекта становится внешним объектом: строками консоли, CSV-файлом, Ghidra CSV, JSON-документом, HTML-страницей. Если parser ошибся, report покажет неверный unwind range. Если heuristic engine слишком резко сказал verdict, report усилит ложный вывод. Если dynamic collector видел неполную сессию, но reporting layer назвал её полноценным подтверждением, пользователь получит опасную уверенность. Поэтому report.c отвечает не за украшение результата, а за дисциплину формулировок.

Эта мысль особенно важна для EH/Unwind Analyzer. Проект работает не с очевидным списком импортов и не с простым "нашёл строку - вывел строку". Он связывает несколько уровней: PE image, section permissions, Exception Directory, RUNTIME_FUNCTION, UNWIND_INFO, handler RVA, heuristic findings, optional dynamic validation, assessment. Report должен сохранить связь между этими уровнями. Он не имеет права превратить аккуратную модель в плоскую фразу "suspicious" без контекста.

Reporting layer как граница между машиной и человеком

Внутри программы данные живут в типизированных структурах. PeImage хранит карту образа. RuntimeFunction хранит begin_rva, end_rva и unwind_rva. UnwindSummary хранит версию UNWIND_INFO, flags, размер пролога, fixed_stack_bytes, frame register, handler_rva, chain и anomalies. HeuristicReport хранит score, confidence, findings и weakening rules. DynamicReport хранит состояние debug-сессии, events и trace. Эти структуры удобны для C-кода, но неудобны для человека. Человек не видит память процесса. Он видит текст, таблицу, JSON или HTML.

report.c строит мост между этими мирами. Этот мост должен быть точным. Если поле было числом внутри C-структуры, report должен показать его так, чтобы не потерялся смысл. Если значение было RVA, оно не должно превратиться в "какой-то адрес". Если dynamic event содержит module и address, report должен сохранить оба, потому что address без module почти бесполезен. Если finding имеет family и representative_rva, report должен показать не только сообщение, но и место, к которому это сообщение привязано.

Поэтому reporting layer в проекте выполняет три задачи одновременно. Первая задача - human-readable explanation, то есть объяснение человеку. Вторая - machine-readable contract, то есть стабильный формат для автоматических проверок, тестов и внешних инструментов. Третья - evidence preservation, то есть сохранение цепочки свидетельств так, чтобы позднее можно было понять, почему итоговый assessment получился именно таким.

report.h: публичный контракт слоя отчётов

Перед report.c стоит report.h. Это заголовочный файл, который показывает, какие формы вывода вообще существуют в проекте. Он не раскрывает внутренние helper-функции, но фиксирует публичный интерфейс reporting layer. В нём видно, что проект не ограничивается одним JSON. Есть brief output, CSV, JSON, HTML и отдельная печать heuristic summary.

```
typedef struct ReportMetadata {
    const char* tool_version;
    const char* report_schema_version;
    const char* generated_at_utc;
    const char* input_sha256;
    uint64_t input_size;
    const char* command_line;
    const char* build_compiler;
    const char* build_platform;
    const char* build_mode;
    bool strict_abi;
    bool follow_chain;
    uint32_t max_chain_depth;
} ReportMetadata;
```

ReportMetadata - это metadata, то есть данные о данных. Она не описывает PE-файл напрямую. Она описывает условия, при которых report был создан. Это важное различие. Если два отчёта показывают разные findings, сначала нужно понять, одинаковой ли версией инструмента они были сделаны, одинаковая ли schema version, одинаковые ли options, одинаковый ли input SHA-256, одинаковый ли режим build. Без metadata отчёт превращается в сиротский текстовый файл: вроде что-то написано, но неизвестно, чем, когда, по какому input и с какими опциями.

Поле tool_version связывает report с конкретной версией EH/Unwind Analyzer. Поле report_schema_version связывает JSON-структуру с

docs/report.schema.json. generated_at_utc нужен не для красоты, а для воспроизводимости: когда report попадает в corpus или в validation run, дата генерации помогает отделить старый результат от нового. input_sha256 - это identity input-файла. Он не говорит, что файл безопасен или вредоносен; он говорит, что этот report относится именно к такому byte sequence. input_size дополняет hash простым sanity check. command_line показывает, какими ключами был запущен инструмент.

strict_abi, follow_chain и max_chain_depth особенно важны, потому что они напрямую меняют поведение unwind decoder. Один и тот же PE может дать разные warnings при включённом strict ABI и выключенном strict ABI. Один и тот же chain может быть раскрыт или не раскрыт в зависимости от follow_chain. max_chain_depth ограничивает, насколько глубоко decoder готов идти по chained unwind info. Report обязан сохранить эти опции, иначе позднее невозможно понять, почему конкретный parse result выглядит именно так.

Brief output: короткая проверка, что программа видит PE как PE

Самый простой публичный вывод - report_print_brief. Он печатает путь файла, machine, ImageBase, SizeOfImage, SizeOfHeaders, число секций, Exception Directory и количество RUNTIME_FUNCTION entries. Это не полноценный report. Это быстрый sanity view, то есть короткий взгляд на то, что parser вообще увидел перед более глубоким анализом.

Здесь важно, что brief output не пытается быть verdict. Он не говорит "нормально" или "плохо". Он показывает базовые structural facts. Если в brief output Machine не AMD64, такой файл не входит в предмет проекта. Если Exception Directory имеет RVA=0 и Size=0, дальше unwind layer не сможет построить обычную x64 unwind-карту. Если RUNTIME_FUNCTION entries равно нулю, это не обязательно "suspicious" само по себе, но это меняет модель дальнейшего анализа.

Официальная документация Microsoft по PE Format задаёт различие между RVA, VA and file pointer, а x64 exception handling документация объясняет, что function unwind information хранится через .pdata/.xdata и RUNTIME_FUNCTION. Brief output показывает именно те значения, которые становятся входом в эту официальную модель: image base, image size, headers size and exception directory.

CSV: плоская таблица для быстрого просмотра

CSV report в проекте существует в двух режимах. Первый режим - обычный CSV для человека или простого скрипта. Второй режим - Ghidra CSV. Обычный CSV выводит одну строку на RUNTIME_FUNCTION entry. Это плоский формат,

поэтому он не может сохранить всю вложенность JSON, но он даёт быструю таблицу: index, begin_rva, end_rva, size, unwind_rva, unwind version, flags, prolog size, fixed stack, frame register, handler_rva, has_chain, anomalies, parse_error.

Почему CSV начинается с index? Потому что RUNTIME_FUNCTION table - это массив записей. У каждой записи есть позиция. Иногда позиция помогает искать источник проблемы: например, если parser сообщает о sequence of overlaps или duplicate begins, index даёт простой способ вернуться к конкретной строке. begin_rva и end_rva описывают диапазон кода, к которому относится unwind metadata. size вычисляется как end_rva - begin_rva. Это не размер UNWIND_INFO. Это размер code range.

unwind_rva - это указатель на metadata, не на тело функции. Новичок легко путает эти два адреса. begin_rva указывает на начало диапазона кода, а unwind_rva указывает на место, где лежит UNWIND_INFO. Если begin_rva находится в .text, а unwind_rva находится в .xdata или другом read-only metadata section, это обычно ожидаемо. Если unwind_rva ведёт в executable или writable section, это уже может стать evidence для metadata_placement findings.

fixed_stack показывает, сколько bytes stack allocation decoder смог восстановить из unwind opcodes. Это не полный stack usage функции во всех ветках. Это модель фиксированной части frame, которую Windows должна уметь откатить при unwinding. frame_reg и frame_off показывают, использует ли функция frame pointer model через FrameRegister/FrameOffset. handler_rva появляется, если UNWIND_INFO flags говорят, что за unwind codes следует exception handler или termination handler. has_chain показывает наличие chained unwind info.

parse_error в CSV важнее, чем кажется. Если us_ok[i] false, report.c не притворяется, что всё распарсено. Он ставит NA в поля, которые нельзя честно заполнить, и выводит parse_error. Это правильное поведение. Плохой report мог бы оставить нули, и пользователь подумал бы, что version=0 or prolog_size=0 - реальные значения. Здесь проект явно отделяет "значение равно нулю" от "значение неизвестно, потому что parse failed".

```
index,begin_rva,end_rva,size,unwind_rva,unwind_ver,unwind_flags,
prolog_size,fixed_stack,frame_reg,frame_off,handler_rva,has_chain,
anomalies,parse_error
```

csv_cell - маленькая, но важная функция. Она экранирует ячейки, если внутри есть comma, quote или newline. Это не косметика. Если message или parse_error содержит запятую, простой CSV reader может сдвинуть колонки и разрушить смысл строки. Для анализа это опасно: representative_rva может оказаться под заголовком message, а message под заголовком anomalies. report.c поэтому ставит quotes и удваивает quotes внутри значения.

Ghidra CSV: не отчёт, а мост в рабочую среду реверсера

Ghidra CSV mode выглядит проще обычного CSV, но по смыслу он не менее важен. Он выводит begin_rva, end_rva, name, comment. Это специально узкий формат. Его задача - не сохранить весь report, а перенести в Ghidra те сведения, которые удобно видеть рядом с функциями и адресами.

```
begin_rva,end_rva,name,comment
0x00001000,0x00001072,sub_00001000,"UNWIND v1 flags=... stack=40B ..."
```

name создаётся как sub_%08X по begin_rva. Это не утверждение, что у функции действительно такое имя. Это synthetic name, то есть имя, сгенерированное инструментом для удобства. Оно помогает импортёру создать понятную метку или комментарий. comment строится через build_unwind_comment. В этом comment проект собирает компактную сводку: version, flags, prolog size, stack bytes, frame register, saved GP registers, saved XMM registers, handler_rva, chain and anomalies.

Ghidra CSV не заменяет JSON. Он намеренно теряет часть структурности. В нём нет full heuristic_analysis, нет dynamic_validation, нет assessment. Но он делает то, что JSON сам по себе не делает: помещает unwind interpretation в ту среду, где реверсер смотрит код. Когда человек стоит в Ghidra на функции, ему важно быстро увидеть, что этот range имеет UNWIND v1, сохраняет RBX/RBP, выделяет 0x40 bytes, имеет handler или chain. Ghidra CSV делает именно это.

Отсюда появляется важная граница: если нужно автоматическое сравнение результатов, читать надо JSON. Если нужно визуально сопровождать reverse engineering, удобен Ghidra CSV. Если нужен быстрый grep или spreadsheet, подойдёт ordinary CSV. Один формат не должен притворяться всеми форматами сразу.

JSON: machine-readable contract, а не просто красивый дамп

Главный structured report проекта - JSON. Structured означает, что у отчёта есть вложенная форма: объект input, объект report_metadata, sections array, heuristic_analysis object, dynamic_validation object, assessment object, functions array. JSON нужен не только для человека. Он нужен для tests, corpus validation, regression comparison, external tooling and schema validation.

В проекте JSON write path начинается с проверки аргументов и модели. report_write_json_ex не просто открывает файл и начинает печатать строки. Сначала он проверяет report_model_valid and report_metadata_valid. Это принципиальная архитектурная позиция: если модель нарушает внутренние invariants, лучше не создать report, чем создать красивый документ с недостоверной структурой.

```
if (!path || !input_path || (!rf && n) || (!us && n) || (!us_ok && n) ||
    !report_model_valid(img, hr, dr) || !report_metadata_valid(metadata)) {
    sb_append(err, "Invalid arguments or metadata for JSON report.\n");
    return false;
```

}

report_metadata_valid проверяет не смысл PE-файла, а смысл самого отчёта. schema version должна совпадать с EHUA_REPORT_SCHEMA_VERSION. input_sha256 должен быть строкой из 64 lower-case hex symbols. generated_at_utc должен иметь форму YYYY-MM-DDTHH:MM:SSZ. build_mode допускает только debug или release. max_chain_depth должен быть в диапазоне 1..16. Это маленькая локальная валидация, но она защищает report contract от тихого расползания.

report_model_valid проверяет другую сторону: не metadata, а уже собранные структуры. Если img отсутствует, если num_sections есть, но sections pointer null, если HeuristicReport содержит finding_count больше HEURISTICS_MAX_FINDINGS, если DynamicReport содержит event_count больше DYNAMIC_MAX_EVENTS, такой report нельзя честно сериализовать. Особенно показательно правило instrumentation_complete: если dynamic collector утверждает, что instrumentation complete, но одновременно есть module_refresh_failures, module_list_truncations, breakpoint failures, capacity drops, rearm failures or dropped_events, report_model_valid отвергает такую модель. Идея простая: нельзя одновременно сказать "инструментация полная" и иметь признаки неполной инструментции.

JSON Schema в docs/report.schema.json закрепляет внешний контракт. В корне schema стоит additionalProperties=false. Это означает, что объект report не может произвольно обрастать неизвестными top-level fields. JSON Schema documentation отдельно подчёркивает роль additionalProperties: она управляет свойствами, которые не описаны в properties. Для проекта это важно: потребитель отчёта должен понимать, что top-level layout стабилен, а не угадывать случайные поля.

Схема требует обязательные поля input, report_metadata, image_base, size_of_image, size_of_headers, section_alignment, file_alignment, exception_directory, pe_validation, sections and functions. heuristic_analysis, dynamic_validation and assessment optional, потому что не каждый запуск обязан включать heuristics and dynamic mode. Это аккуратное разделение. Static dump может иметь functions без dynamic_validation. Heuristic run добавляет heuristic_analysis. Dynamic run добавляет dynamic_validation and assessment.

JSON escaping: почему строки нельзя просто печатать как есть

json_escape в report.c выглядит как техническая мелочь, но на самом деле это часть trust boundary. Report содержит строки из разных источников: input path, command line, compiler/platform strings, section names, findings messages, dynamic details. Некоторые из этих строк могут содержать символы, которые

ломают JSON: quote, backslash, newline, carriage return, tab, control characters. Если их напечатать как есть, report перестанет быть валидным JSON.

Функция json_escape обрабатывает backslash, quote, newline, carriage return, tab, control chars and UTF-8. Она не просто пропускает bytes выше 0x7F. Она проверяет valid_utf8_sequence_length. Если последовательность выглядит валидным UTF-8, она сохраняется. Если нет, byte выводится как \u00xx. Это полезно для defensive reporting: проект не обязан доверять любому metadata text как корректной Unicode строке.

Здесь видна общая философия проекта. Сначала util.c не доверяет input bytes. Потом pe.c не доверяет заголовкам. Потом unwind.c не доверяет UNWIND_INFO. Потом report.c не доверяет строкам, которые вставляет в JSON/HTML. Без такого подхода анализатор мог бы быть устойчив к malformed PE, но ломаться на malformed path or message.

Root JSON object: верхняя карта отчёта

Корневой JSON object начинается с input and report_metadata. input - это путь, переданный пользователем или main.c. report_metadata - это условия создания. Далее идут PE-level facts: image_base, size_of_image, size_of_headers, section_alignment, file_alignment, exception_directory. Эти поля отражают то, что было нормализовано в pe.c.

Важно, что численные адресные поля в JSON выводятся как hex strings, например "0x00001000". Это выбор report contract. Для адресов и RVA hex representation читается естественнее, чем decimal. Но при этом counters and ratios остаются числами: section_raw_oob is integer, confidence is number, findings array is array. Report не смешивает всё в строки. Он использует строки там, где hex notation важна как адресная запись, и числа там, где value действительно count/probability-like metric.

exception_directory выводится как object с rva and size. Это отражает official PE Data Directory model: directory entry хранит RVA и size. Report не должен говорить "section .pdata" как единственную истину, потому что официальный вход - directory, а имя секции может быть нестандартным. Это уже было важно в pe.c, и report.c сохраняет эту дисциплину.

pe_validation в JSON: структурное здоровье PE-карты

pe_validation показывает не unwind findings, a structural validation level, полученный при чтении PE. Поля section_raw_oob, section_virtual_oob, section_raw_overlaps, section_virtual_overlaps, section_misaligned and exception_directory_oob помогают понять, насколько надежна карта, на которой потом стоит весь анализ.

Если section_raw_oob больше нуля, часть section raw range выходит за пределы файла. Это не обязательно автоматически "злонамеренность", но это ослабляет уверенность в чтении bytes. Если section_virtual_oob больше нуля, virtual section range конфликтует с SizeOfImage. Если есть raw overlaps or virtual overlaps, две секции конкурируют за одну область. Если exception_directory_oob true, официальный вход к .pdata указывает за пределы допустимого image/file mapping. В таком случае unwind decoder не должен делать вид, что видит полноценную exception table.

В хорошей практике report читают сверху вниз. Если pe_validation уже показывает проблему, не надо начинать с final verdict. Сначала нужно понять, насколько сам image map пригоден для дальнейших выводов. Heuristic finding может быть следствием реальной аномалии, а может быть следствием разрушенной PE-карты. Поэтому report показывает structural validation до functions and findings.

sections array: карта территории, а не декоративная таблица

sections array содержит name, virtual_address, virtual_size, raw_offset, raw_size, characteristics, readable, writable and executable. Эти поля напрямую нужны для interpretation. Без sections невозможно нормально понять, где находится begin_rva, где находится unwind_rva, где находится handler_rva, и какие permissions имеет соответствующий участок.

readable, writable, executable - это derived booleans из section characteristics. Они не заменяют characteristics, а делают report удобнее. characteristics хранит исходную bitmask, а booleans дают человеку и инструменту быстрый ответ. Например, metadata_placement findings смотрят на то, что unwind metadata оказалась в executable или writable section. Report должен позволять проверить это без повторного PE parsing.

В HTML эти booleans сжимаются в permissions string вроде RWX, R--, R-X. В JSON они оставлены как отдельные поля. Это различие объясняет цель форматов. HTML помогает глазу. JSON помогает программе. CSV помогает таблице. Одна и та же семантика имеет разные формы, но не должна менять смысл.

functions array: центральный мост между .pdata, .xdata и человеком

functions array - самый длинный и самый важный массив JSON. Каждая запись соответствует одному RUNTIME_FUNCTION entry. Ещё раз: это не обязательно функция в смысле символов, имени или декомпилятора. Это range descriptor из exception directory. Поэтому JSON честно называет поля begin_rva, end_rva, size, begin_section, unwind_rva, unwind_section, unwind_ok, unwind_error, unwind, comment.

begin_section and unwind_section вычисляются через section_name_or_dash. Это удобное enrichment, то есть обогащение отчёта. Внутренний RuntimeFunction не хранит имя секции. Но report добавляет его, потому что реверсеру важно сразу видеть: code range в .text, unwind metadata в .xdata, или что-то уехало в необычный section. Это не меняет исходный факт, а делает его читаемым.

unwind_ok - одно из самых важных boolean fields в functions. Если false, это не мелкая ошибка. Это значит, что UnwindSummary не является полноценной моделью UNWIND_INFO. Тогда unwind_error объясняет причину. Report всё равно сохраняет begin/end/unwind RVA, потому что сама RUNTIME_FUNCTION entry была прочитана, но не выдаёт недостоверную интерпретацию как нормальную.

unwind object внутри function record хранит version, flags, prolog_size, count_codes, fixed_stack_bytes, frame_reg, frame_off, has_handler, handler_rva, handler_section, has_chain, anomalies, anomalies_text, chain. Это ровно тот уровень, который связывает книгу с проектом. В главах про Windows мы изучали, что UNWIND_INFO описывает, как откатить prolog effects and restore caller context. В report это знание становится конкретными полями. fixed_stack_bytes показывает восстановленную stack allocation model. saved registers в кратком comment показывают, какие nonvolatile registers были сохранены. handler fields связывают unwind metadata с exception route.

anomalies is numeric bitmask, a anomalies_text - человекочитаемое представление. Почему нужны оба? Numeric bitmask удобен для machine processing and tests. Text вроде ver|trunc|handler_oob удобен человеку. Если оставить только text, external tools будут парсить строки. Если оставить только integer, человек будет держать таблицу bitmask в голове. report.c сохраняет оба слоя.

comment - это compact narrative summary для каждой записи. Он строится build_unwind_comment and используется также в Ghidra CSV. Это не источник истины вместо structured fields. Это readable summary. Если comment и structured fields расходятся, source of truth должен быть structured fields, а comment надо считать bug в reporting layer.

heuristic_analysis: как static findings становятся читаемыми

Если hr передан в report_write_json_ex, JSON получает блок heuristic_analysis. Это блок не про dynamic execution. Он описывает static evidence layer. Внутри находятся score, suspicion, confidence, requires_manual_review, verdict, evidence_level, finding counts, structural counters, ratios, evidence families, findings, weakening_rules_applied and details.

score - bounded engineering score. suspicion - нормированная форма score, но не статистическая вероятность. confidence - rule-based confidence, зависящая

от полноты и согласованности evidence, а не от обученной модели. verdict - короткая классификация static state. evidence_level - weak, medium, strong or none. В report важно, что эти поля лежат рядом, но не означают одно и то же. Если человек видит score=70 and confidence=0.60, он не должен читать это как "70 процентов malware с вероятностью 60 процентов". В проекте это не так.

Counters вроде rf_invalid_ranges, rf_oob, rf_unwind_oob, rf_unsorted, rf_overlaps, rf_duplicate_begins, unwind_parse_fail and unwind_anomalies позволяют вернуться от verdict к фактам. Хороший report не заставляет верить verdict. Он показывает, из чего verdict вырос. Если score высокий из-за handler_dominance, это один тип случая. Если score высокий из-за parse failures and out-of-bounds metadata, это другой тип случая. JSON должен позволить их различить.

findings array хранит structured finding objects. У каждого finding есть code, family, severity, message, affected_count, representative_rva and representative_section. code нужен для tests and scripts. family нужна для группировки evidence. severity нужна для человеческой приоритизации. message объясняет смысл. affected_count показывает масштаб. representative_rva даёт примерное место, с которого можно начать ручную проверку. representative_section добавляет section context.

weakening_rules_applied - отдельный массив, потому что weakening rules не являются findings. Finding говорит: "мы увидели такой сигнал". Weakening rule говорит: "силу вывода надо снизить, потому что evidence неполное, одиночное или имеет benign explanation". Если смешать эти вещи, пользователь не поймёт, почему score есть, но confidence не растёт, или почему verdict осторожный.

details - это human-readable текстовый журнал heuristic engine. Он нужен, но не должен быть единственным источником. Машина должна читать findings array and counters. Человек может читать details, чтобы понять narrative. Это снова разделение human-readable and machine-readable.

dynamic_validation: runtime evidence без самообмана

Если DynamicReport передан в report_write_json_ex, JSON получает dynamic_validation. Этот блок не доказывает, что статическая модель "истинна" во всех запусках. Он фиксирует, что произошло в конкретной debug-сессии. Поэтому первые поля dynamic_validation описывают состояние сессии: enabled, ran, ok, timed_out, target_exited, target_exit_code, timeout_ms and trace_steps.

enabled означает, что dynamic mode был запрошен и report initialized. ran означает, что target process реально был создан. ok означает, что debug loop прошёл без внутренней ошибки и без timeout при наличии ожидаемых условий. timed_out говорит, что сессия оборвалась по времени. target_exited

говорит, что collector увидел EXIT_PROCESS_DEBUG_EVENT. Эти поля нельзя заменять одним "dynamic=yes". Dynamic validation имеет разные степени полноты.

instrumentation_complete - особенно опасное поле для неправильного чтения. Оно должно означать, что collector не только запустился, но и смог поставить нужные breakpoints and instrumentation without known truncation/failure. report_model_valid специально запрещает состояние, где instrumentation_complete true, но рядом есть module_refresh_failures, module_list_truncations, breakpoint_set_failures, breakpoint_capacity_drops, pending_rearm_failures, breakpoint_rearm_failures or dropped_events. Иначе report мог бы одновременно говорить "всё поставлено" and "часть не поставлена".

events array хранит отдельные DynamicEvent records. Каждый event имеет type, module, thread_id, exception_code, first_chance, address, arg0..arg3. type объясняет, что произошло: exception, veh_registration, unwind_handler_hit, rtl_add_function_table and so on. module связывает runtime address с модулем. thread_id показывает поток. exception_code and first_chance сохраняют exception semantics. arg0..arg3 сохраняют captured arguments for API breakpoints, например handler pointer for VEH or function table parameters for RtlAddFunctionTable.

trace array - это RIP trace, но не full execution trace. Он ограничен trace_steps and internal capacity. Report должен показывать trace как локальное окно после handler hit, а не как доказательство полного control-flow graph. Если trace пустой, это не значит, что handler не исполнялся; надо смотреть handler hit counters and trace_steps. Если trace есть, это стартовая дорожка для manual debugger work, а не конец анализа.

dynamic_signal_count вычисляется report.c, а не dynamic_win.c. Он складывает наличие runtime categories: unwind handler hits, VEH/VCH/UEF activity, dynamic function table calls, second chance. Это summary metric. Он не заменяет events. Если dynamic_signal_count=3, нужно смотреть, какие именно три категории сработали. VEH registration, RtlAddFunctionTable and second chance - это совсем другая история, чем unwind_handler_hit, veh_hit and uef_hit.

Assessment: где static и dynamic превращаются в осторожный итог

assessment появляется, когда есть heuristic_analysis or dynamic_validation. Это не исходный слой данных. Это слой интерпретации. Внутри находятся final_verdict, final_confidence, static_evidence_level, runtime_evidence_level, case_strength, static_dynamic_correlation, dynamic_signal_count, weakening_rule_count and manual_validation_steps.

static_dynamic_correlation не означает "доказано вредоносно". В report.c correlation true только в ограниченных случаях. Первый случай: есть suspicious static handler context - например handler_nonexec_section, handler_writable_section or handler dominance family - и runtime showed handler-related hits. Второй случай: есть static placement anomalies and runtime dynamic function table activity. Это coarse correlation layer, не per-function proof graph. Он говорит, что статический и динамический слои встретились в одной смысловой зоне.

runtime_evidence_level вычисляется из DynamicReport. Handler hit даёт strong signal, dynamic function table calls and second chance дают medium signals, registrations and first chance дают weak activity. Но если session incomplete, non-none runtime level сарается до weak. Это очень важная защита от самообмана. Неполная dynamic session не должна превращать случайное наблюдение в сильный runtime proof.

case_strength отделяет correlated and unconfirmed cases. strong_correlated_case появляется, когда есть correlation and static or runtime evidence strong. medium_correlated_case появляется при correlation and medium evidence. strong_unconfirmed_case означает, что сильный сигнал есть, но cross-layer correlation нет. medium_case and weak_case сохраняют осторожность. no_case означает, что report не увидел достаточных static/dynamic evidence.

final_verdict строится от score thresholds and correlation. score >= 70 даёт high_suspicion_static or high_suspicion_correlated. score >= 40 даёт moderate_suspicion_static/correlated. score >= 15 даёт low_suspicion_static/correlated. Если static score низкий, но complete dynamic session saw signals, появляется runtime_signals_only. Если ничего существенного нет, final_verdict остаётся no_strong_correlated_evidence. Эта формулировка намеренно осторожная. Она не говорит "safe". Она говорит, что сильного correlated evidence нет.

final_confidence начинается от hr->confidence или 0.10 без hr. Complete dynamic session with signals добавляет 0.10. Correlation добавляет ещё 0.10. Потом значение clamped to 0.98. Это инженерная confidence, не статистическая probability. Она отражает согласованность evidence layers, а не частоту malware в популяции.

Manual validation steps: отчёт как план следующей работы

Одна из самых удачных деталей report.c - manual_validation_steps. Хороший анализатор не только выдаёт verdict, но и говорит, что делать дальше. collect_manual_validation_steps смотрит на static findings and dynamic signals and собирает практические шаги для реверсера. Это превращает report из приговора в маршрут ручной проверки.

Если есть rf_invalid_ranges, rf_oob, rf_unwind_oob or overlaps, report предлагает проверить .pdata range integrity in a PE viewer, вручную отсортировать RUNTIME_FUNCTION entries and confirm begin/end/unwind RVAs inside SizeOfImage. Если есть unwind_parse_fail, unwind_anomalies or family unwind_integrity, report предлагает открыть suspicious UNWIND_INFO blob in hex view and compare decoded prolog actions with function prolog in Ghidra. Если metadata placement unusual, report предлагает inspect section hosting unwind metadata and decide whether placement matches benign compiler layout or redirected/manual layout.

Если handler_nonexec_section есть, report не говорит автоматически "malware". Он предлагает disassemble reported handler target, confirm execute permission and check whether landing bytes are real handler, thunk or non-code blob. Если writable unwind metadata or writable handler target есть, report предлагает проверить, не объясняется ли это JIT/runtime scenario. Если handler dominance есть, report предлагает открыть dominant handler target and compare referencing functions.

Dynamic-based manual steps тоже аккуратны. При RtlAddFunctionTable or RtlInstallFunctionTableCallback calls report предлагает capture arguments and inspect runtime table contents. При VEH/VCH/UEF registrations - list each registration and single-step. При hits - single-step from handler hit and map module/section transitions. При static findings without dynamic signals - run sample again with --dynamic on representative exception path. Это правильная модель: автоматический report не заменяет реверсера, а готовит ему следующий рабочий шаг.

HTML report: тот же смысл, но для глаза

HTML report строится в report_write_html. Он использует те же input structures, но организует их как dashboard: summary cards, metadata block, PE image, structural validation, sections table, final assessment, static findings, dynamic validation, functions table. HTML нужен не для машинной интеграции, а для быстрого чтения человеком.

HTML report показывает cards: RUNTIME_FUNCTION entries, anomalous entries, suspicion score, static verdict, confidence, dynamic validation status. Такие cards помогают быстро увидеть состояние, но опасны, если забыть про детали. Поэтому ниже есть sections, findings, dynamic events and functions. Card - это дверь в анализ, не замена анализу.

html_escape выполняет такую же роль для HTML, как json_escape для JSON. Если section name, message or details содержит <, >, &, quote, report не должен позволить этому тексту стать HTML markup. Это не абстрактная web security тема. HTML report может открываться локально в браузере. Если анализатор вставляет arbitrary input text without escaping, malformed input could break

layout or inject markup. Поэтому report.c экранирует текст.

HTML также использует offline CSP. В проектной документации это зафиксировано как требование: HTML report не должен тянуть remote scripts, styles, fonts or network resources. Это делает report более переносимым и безопасным для офлайн-анализа. CSP здесь не превращает HTML в sandbox, но задаёт дополнительную дисциплину: report remains self-contained, predictable and not network-active.

Dynamic status in HTML: почему ok - это не просто ran

dynamic_status_text показывает несколько состояний. Если dynamic report отсутствует или disabled, HTML говорит disabled. Если enabled, но not ran, говорит not run. Если timed out, говорит timeout. Если !ok, говорит incomplete. Если target exited, говорит completed. Иначе говорит ran. Это кажется мелочью, но это важная часть честного языка отчёта.

Представим два случая. В первом CreateProcess failed, ran=false. Во втором process started, several events were collected, but timeout happened before exit. В третьем process exited cleanly and instrumentation_complete true. Все три случая нельзя называть одинаково "dynamic mode". У них разная evidence strength. Reporting layer обязан сохранить эту разницу.

Atomic write: почему отчёт либо целый, либо не считается созданным

report.c не пишет итоговый файл напрямую через fopen final path and fprintf until success. Он строит весь report in StrBuf, а затем вызывает file_write_atomic из util.c. Идея atomic write проста: пользователь и тесты должны видеть либо старый корректный report, либо новый корректный report, но не половину JSON or HTML after crash.

На Windows такая логика обычно строится вокруг temporary sibling file and replacement operation. Официальная документация Microsoft по MoveFileEx описывает options вроде MOVEFILE_REPLACE_EXISTING and MOVEFILE_WRITE_THROUGH. В проектной архитектуре это важно потому, что reports are artifacts. Если validation pipeline читает JSON сразу после генерации, partial write может выглядеть как schema failure, хотя проблема была в interrupted write. Atomic replacement снижает эту категорию ошибок.

Это ещё один пример, почему report.c - не "последний printf". Надёжность отчёта зависит не только от правильных полей, но и от способа записи. Broken output file может испортить corpus, tests, CI, ручной анализ and Ghidra import.

Как report.c связан с tests

В проекте есть отдельные проверки report semantics. Это правильно, потому что reporting bugs are analysis bugs. unit_report_json.c проверяет JSON output and escaping. unit_report_html.c проверяет HTML output and escaping. test_report_semantics.py and report_semantics.py проверяют более высокий смысл: schema version, required fields, assessment, dynamic validation semantics, cross-field invariants.

Почему tests для report важны отдельно от tests для pe.c and unwind.c? Потому что можно правильно распарсить PE and правильно посчитать HeuristicReport, но неправильно вывести JSON. Например, забыть representative_section, вывести null вместо string, потерять weakening_rule_count, не заэкранировать message, назвать incomplete dynamic session strong runtime evidence. Такие ошибки не ловятся pure parser tests. Их ловит reporting semantics test layer.

docs/report.schema.json служит не только внешним контрактом, но и тестовой опорой. Если разработчик меняет JSON, он должен понимать, меняет ли он schema contract. Если поле добавлено без обновления schema, потребитель может его не ждать. Если поле удалено, старый parser external tooling сломается. Schema - это дисциплина эволюции.

Как реверсер должен читать отчёт report.c

Правильное чтение report не начинается с final_verdict. Оно начинается с metadata and PE validation. Сначала нужно понять, какой input анализировался, какой версией tool, с какими options, и насколько PE map structurally sane. Потом нужно смотреть sections, потому что все адреса дальше живут в section context. Потом functions array: какие RUNTIME_FUNCTION ranges есть, какие unwind records parse ok, где anomalies, где handlers. Потом heuristic_analysis: какие findings, какие families, какая severity, какие weakening rules. Потом dynamic_validation, если он есть: сессия complete or incomplete, какие events, какие hits, какой trace. И только после этого assessment.

Такой порядок чтения защищает от типичной ошибки: увидеть high_suspicion_static and сразу решить, что всё доказано. Report устроен иначе. Он даёт карту. По карте можно идти. final_verdict - это не вершина пирамиды, а компактная сводка после слоёв evidence. Если слои показывают слабые or conflicting facts, manual_validation_steps подсказывает, где продолжить.

Именно поэтому report.c завершает pipeline. util.c гарантирует безопасное чтение bytes. pe.c строит карту image. unwind.c декодирует .pdata/.xdata. heuristics.c интерпретирует static evidence. dynamic_win.c собирает runtime evidence. report.c фиксирует всё это как artifact, который можно прочитать, проверить, импортировать, сравнить и защитить на разборе проекта.

Что report.c принципиально не делает

report.c не должен заново анализировать PE. Он может добавлять section_name_or_dash for readability, но не должен принимать решения вместо pe.c. Он не должен заново декодировать UNWIND_INFO. Он может build_unwind_comment from UnwindSummary, но не должен читать .xdata. Он не должен усиливать verdict beyond evidence engine and assessment logic. Он не должен превращать dynamic event into proof without dynamic session completeness. Его задача - честная сериализация, осторожная агрегация and readable presentation.

Эта граница защищает архитектуру. Если report.c начнёт делать hidden analysis, станет непонятно, почему JSON отличается от console output, почему HTML показывает одно, а heuristic engine считает другое. В хорошем анализаторе каждый слой знает свою ответственность. report.c знает, как показать и зафиксировать результат. Он не должен тайно менять сам результат.

Schema как контракт между версиями инструмента

docs/report.schema.json в проекте задаёт не "примерный вид JSON", а contract. В корне указаны $schema, $id, title, type, additionalProperties, required, properties and $defs. $id имеет вид urn:ehua:report-schema:1.1.0. Это не просто декоративная строка. Она даёт схеме устойчивое имя. Если внешний валидатор или CI pipeline проверяет отчёты, ему нужно знать, какая именно schema version используется. Поэтому report_metadata.schema_version and schema $id должны жить в одной дисциплине версий.

Root required fields показывают минимальный report, который проект считает полноценным static artifact: input, report_metadata, image_base, size_of_image, size_of_headers, section_alignment, file_alignment, exception_directory, pe_validation, sections and functions. Даже если heuristics выключены, functions array остаётся обязательным. Это правильно: центральный предмет проекта - exception/unwind model. Без functions report перестаёт быть отчётом EH/Unwind Analyzer.

$defs в schema описывает повторно используемые типы: hex, count, probability, evidence_level, report_metadata, exception_directory, pe_validation, section, finding, weakening_rule, heuristic_analysis, dynamic_event, dynamic_validation, assessment, unwind_chain, unwind_summary and runtime_function. Благодаря этому schema не превращается в одну огромную плоскую простыню. Она повторяет архитектуру проекта: PE validation отдельно, section separately, runtime function separately, heuristic separately, dynamic separately, assessment separately.

evidence_level в schema ограничивает язык отчёта. В проекте evidence level не должен неожиданно стать "critical", "danger", "malware" or "green". Он должен быть тем набором уровней, который понимает heuristic engine and

assessment. Когда язык report ограничен schema, downstream tools не должны угадывать новые случайные слова.

finding object в schema требует code, family, severity, message, affected_count, representative_rva and representative_section. Это защищает от бедной ошибки: вывести только message. Message понятен человеку, но неудобен для автоматической обработки. Code стабилен для тестов. Family стабилен для группировки. Representative location помогает реверсеру открыть конкретное место. Если хотя бы один из этих элементов исчезает, finding становится хуже как рабочий артефакт.

dynamic_event object тоже нарочно строгий. type, module, thread_id, exception_code, first_chance, address, arg0, arg1, arg2, arg3 всегда присутствуют. Даже если для конкретного event нет meaningful arg2, поле всё равно есть и обычно заполнено нулём. Это делает parsing проще: external consumer не пишет десятки if field exists. Он знает форму event record. Смысл конкретных arg0..arg3 зависит от event type, но форма записи стабильна.

Поле за полем: как читать report_metadata

schema_version отвечает на вопрос: по какому JSON contract нужно читать report. tool_version отвечает на вопрос: какой build анализатора создал этот report. generated_at_utc отвечает на вопрос: когда artifact был создан. input_sha256 отвечает на вопрос: к каким exact bytes относится результат. input_size помогает быстро увидеть, не перепутан ли файл. command_line показывает analysis mode and output mode. build_compiler, build_platform and build_mode связывают artifact with build provenance.

strict_abi в metadata очень важен для интерпретации anomalies. Если strict_abi=false, часть ABI-level warnings может не появиться. Если strict_abi=true, decoder может помечать reserved opinfo, suspicious push order, invalid register class, chain restrictions and save offset sanity. Поэтому сравнивать два report без сравнения strict_abi нельзя. Они могли анализировать тот же файл, но в разной строгости.

follow_chain говорит, пытался ли unwind.c идти по UNW_FLAG_CHAININFO. Если false, report может показать has_chain, но не раскрыть deeper chain semantics. Если true, max_chain_depth задаёт верхнюю границу. Это защита от cycles and pathological metadata. Поэтому chain-related findings and anomalies нужно читать вместе с follow_chain/max_chain_depth.

Поле за полем: как читать runtime_function в JSON

index - это порядковый номер записи в массиве, как её увидел reader. Он нужен для связи JSON, CSV and raw .pdata table. begin_rva and end_rva - границы code range. size - вычисленный размер range. begin_section - section, куда попадает begin_rva. unwind_rva - RVA metadata. unwind_section - section, куда

попадает UNWIND_INFO. unwind_ok - факт успешной декодировки. unwind_error - объяснение failure. unwind - structured summary. comment - compact human phrase.

Если begin_section равен "-", это значит, что begin_rva не попал ни в одну известную section по pe_find_section_for_rva. Это не надо автоматически читать как exploit, но это strong structural concern: RUNTIME_FUNCTION range ссылается на кодовую область, которую PE section map не объясняет. Если unwind_section равен "-", значит UNWIND_INFO не удалось привязать к known section. Это напрямую влияет на доверие к unwind decoder and metadata placement findings.

Если unwind_ok=false, хорошая ручная проверка начинается не с disassembler, а с unwind_error and raw RVA. Нужно открыть file offset, на который должен мапиться unwind_rva, и проверить header byte, flags, count codes, padding, tail. Только после этого имеет смысл спорить с parser. Иногда parser прав: metadata реально truncated. Иногда mapping повреждён. Иногда файл использует нестандартный layout, который проект сознательно считает unknown.

Если unwind.has_handler=true, handler_rva становится одной из ключевых точек маршрута исключения. Но handler_rva сам по себе не говорит, что handler когда-либо исполнится. Он говорит, что metadata содержит handler target. Чтобы утверждать runtime manifestation, нужен dynamic evidence: unwind_handler_hit, VEH/VCH/UEF hit or related exception route observation. Report специально разделяет static handler presence and dynamic handler execution.

Если unwind.has_chain=true, chain object показывает begin_rva, end_rva and unwind_rva chained record. Chained unwind info важно читать осторожно: chain может быть нормальной компиляторной техникой, но также может усложнять анализ, создавать loops, depth exceed or invalid chain. Поэтому chain fields надо смотреть вместе с anomalies_text and decoder options.

HTML functions table: почему таблица функций не равна списку функций

В HTML report functions table удобна глазами, но её название может обмануть новичка. Rows не обязаны соответствовать high-level functions из исходного кода. Они соответствуют RUNTIME_FUNCTION entries. Иногда compiler splits code ranges, sometimes thunks or funclets appear, sometimes chained entries describe parts of procedure. Поэтому HTML table нужно читать как unwind coverage table, а не как список всех функций программы.

Для реверсера эта таблица полезна тем, что она связывает begin/end range, unwind metadata and anomalies in one visual row. Если в Ghidra функция выглядит нормальной, но HTML показывает anomaly on its unwind record,

нужно сравнить prolog instructions with unwind comment. Если HTML показывает handler_rva in writable section, нужно открыть этот RVA and inspect section permissions. Если many rows share one unwind_rva, это может быть normal reuse or evidence for unwind_reuse family, depending on concentration and context.

HTML also shows static findings and dynamic validation near final assessment. Это помогает увидеть contradiction. Например, static findings high, but dynamic validation disabled. Тогда final assessment cannot be correlated. Или dynamic events exist, but instrumentation incomplete. Тогда runtime_evidence_level should not become strong. Хороший HTML report должен делать эти противоречия видимыми, а не прятать их.

Cross-field invariants: почему report нужно проверять не только schema

JSON Schema проверяет форму, но не всегда проверяет смысловые связи между полями. Например, schema может сказать, что instrumentation_complete is boolean and breakpoint_set_failures is integer. Но schema сама по себе не объяснит, что instrumentation_complete=true together with breakpoint_set_failures>0 is semantically inconsistent. Поэтому в проекте существуют computed cross-field invariants in tests/report_semantics.py and report_model_valid in C.

Другой пример: input_sha256 может быть строкой из 64 hex symbols, и schema будет довольна. Но если эта строка не соответствует actual input bytes, schema этого не узнает. Поэтому hash generation belongs to app_info/main/util path, а schema только закрепляет форму. Валидация отчёта всегда имеет два слоя: schema validation and semantic validation.

Третий пример: assessment.runtime_evidence_level может быть строкой "strong" and dynamic_validation.timed_out=true. Формально это может пройти простую schema, если enum допускает strong. Но семантически проект должен cap runtime evidence at weak for incomplete session. Такие правила живут в report.c assessment logic and tests. Это показывает, что report contract is not only JSON Schema. Contract is schema plus code invariants plus tests.

Почему report.c не должен скрывать null and dash

В JSON representative_rva может быть null, если finding has no location. В section fields иногда появляется "-". Это не неряшливость. Это способ отличить missing/not applicable from concrete value. Если finding описывает global condition, representative_rva может быть meaningless. Если handler отсутствует, handler_section is "-". Если RVA не попал в section, section_name_or_dash returns "-". Все эти случаи разные, но они должны быть явно видны.

Новичок часто хочет заменить null or dash на 0. Это опасно. RVA 0 может быть реальным значением or meaningful sentinel depending on context. Null говорит: значения нет. Dash говорит: имени секции нет или оно не применимо. Ноль говорит: значение равно нулю. В forensic-style report эти различия нельзя стирать.

Как report.c сохраняет осторожный язык проекта

Во всей книге мы много раз отделяли suspicious от malicious, confidence от probability, evidence от proof. report.c обязан продолжать эту линию. Поэтому final_verdict содержит high_suspicion_static, high_suspicion_correlated, runtime_signals_only, no_strong_correlated_evidence, а не "malware" and "clean". Это не слабость инструмента, а честность его области применения.

Если report сказал бы clean, он сделал бы утверждение, которое static analyzer не может доказать. Файл мог иметь path-dependent behavior, unpacking, dynamic code, environment checks, dormant exception route. Если report сказал бы malware, он тоже вышел бы за границы: evidence engine не является malware classifier. Поэтому язык report построен вокруг suspicion, evidence level, case strength and manual validation. Это зрелый инженерный язык.

Итог раздела

report.c - это слой, где EH/Unwind Analyzer становится не просто программой, которая что-то посчитала, а инструментом, результат которого можно использовать. Console даёт быстрый обзор. CSV даёт плоскую таблицу. Ghidra CSV переносит unwind interpretation в среду реверсера. JSON даёт machine-readable contract. HTML даёт human-readable dashboard. assessment связывает static and runtime layers. manual_validation_steps превращают findings в план работы.

Главное: report не должен звучать увереннее, чем evidence. Если static findings сильные, но dynamic layer ничего не подтвердил, report говорит unconfirmed. Если dynamic session incomplete, runtime evidence level ослабляется. Если there are weakening rules, report показывает их отдельно. Если PE map damaged, pe_validation остаётся видимой. Это и есть зрелость инструмента: он не пытается впечатлить громким verdict, а учит реверсера видеть цепочку доказательств.

11.9. Ghidra integration: как результат EH/Unwind Analyzer попадает в рабочую среду реверсера

После `report.c` может показаться, что проект уже завершил свою работу. Анализатор прочитал PE-файл, построил карту секций, извлек Exception Directory, декодировал `RUNTIME_FUNCTION` и `UNWIND_INFO`, оценил результаты через `heuristics.c`, при необходимости собрал runtime evidence через `dynamic_win.c`, а затем записал JSON, HTML, CSV и console output. Формально цепочка замкнулась: из bytes получился report. Но для реверсера report - это еще не конец работы. Реверсер обычно думает не в виде отдельного JSON-документа, а в окне дизассемблера, где есть адреса, функции, комментарии, графы, псевдокод, закладки, имена и привычная навигация. Поэтому в проекте появляется отдельный мост - Ghidra integration.

Ghidra integration означает интеграцию с Ghidra, то есть с открытой средой reverse engineering, где бинарный файл превращается в Program, Listing, Functions, Symbols, Comments and Bookmarks. Здесь важно сразу не перепутать уровень. `ghidra/import_unwind_csv.py` не является вторым анализатором unwind metadata. Он не заново парсит PE, не декодирует `.xdata`, не вычисляет `score`, не запускает dynamic collector и не проверяет exception route. Его задача скромнее и поэтому инженерно надежнее: взять уже подготовленный анализатором Ghidra CSV и перенести его в адресное пространство открытой программы Ghidra так, чтобы человек увидел ranges, comments и warnings там, где реально смотрит код.

Если сказать совсем человечески, `report.c` объясняет анализ в виде документа, а Ghidra importer ставит дорожные знаки прямо на карте местности. HTML report похож на аналитическую записку: он хорошо показывает общую картину, findings, severity, assessment, dynamic status, manual validation steps. Ghidra же похожа на рабочий стол реверсера: там человек кликает по адресу, смотрит prolog, сверяет stack adjustment, открывает handler target, сравнивает один диапазон с другим. Интеграция нужна именно потому, что эти два режима мышления дополняют друг друга. Report отвечает на вопрос “что заметил анализатор?”, а Ghidra помогает проверить “где это находится в коде и как выглядит руками?”.

Почему Ghidra CSV существует отдельно от обычного CSV и JSON

В проекте уже есть обычный CSV. Он содержит более табличную, машинно-удобную форму: index, begin/end RVA, unwind RVA, version, flags, prolog size, fixed stack, frame register, handler RVA, chain flag, anomalies and parse_error. Такой CSV полезен для spreadsheet, quick grep, тестов, простых сравнений и автоматической обработки. Но он не идеально подходит для Ghidra, потому что Ghidra-скрипту не нужны все поля в сыром виде. Ему нужна компактная инструкция: где начинается диапазон, где он заканчивается, как его назвать и какой комментарий поставить перед функцией.

JSON, наоборот, гораздо богаче. В JSON есть `ReportMetadata`, `pe_validation`, `sections`, `functions`, `heuristic_analysis`, `dynamic_validation`, `assessment`, `manual_validation_steps`. На бумаге можно было бы написать JSON importer и переносить в Ghidra почти всю модель. Но первая интеграция проекта сознательно не делает так. CSV выбран как bridge format - формат-мост. Он маленький, предсказуемый, легко проверяется глазами, легко генерируется в `report.c`, легко читается стандартным `csv.DictReader` внутри Ghidra script, и у него меньше шансов сломаться из-за изменения большой JSON schema.

Bridge format - это не “бедный формат потому что не хватило сил”. Это осознанная граница. Граница говорит: основной анализ остается в EH/Unwind Analyzer, а Ghidra получает минимальный набор фактов, которые помогают человеку в навигации. Эта минимальность защищает от распространенной ошибки инструментов reverse engineering: когда один инструмент пытается стать всем сразу и начинает навязывать дизассемблеру слишком много своих предположений. Здесь проект ведет себя аккуратнее. Он говорит Ghidra: “вот диапазоны из Exception Directory, вот короткий unwind comment, вот warning bookmark, если parse failed или есть anomalies; дальше человек проверяет”.

```
begin_rva,end_rva,name,comment
0x000013F0,0x0000140D,sub_000013F0,"UNWIND v1 flags=EHANDLER prolog=4 stack=40B frame=- saved_gp=[-] s
```

Эта строка выглядит простой, но в ней уже соединены несколько глав книги. `begin_rva` и `end_rva` пришли из `RUNTIME_FUNCTION`. Имя `sub_000013F0` не является именем из symbol table, потому что у stripped PE его может не быть; это стабильное проектное имя, построенное по началу диапазона. `comment` пришел из декодированного `UNWIND_INFO`: версия, flags, размер prolog, оценка stack allocation, frame register, saved registers, handler RVA. То есть одна CSV-строка переносит в Ghidra результат всей цепочки `pe.c -> unwind.c -> report.c`.

Именно поэтому Ghidra CSV нельзя считать “просто экспортом функций”. Это экспорт unwind-view of functions - взгляда на функции через официальную Windows x64 unwind-модель. Обычный дизассемблер может создать функцию

по entry point, по call target, по code flow. EH/Unwind Analyzer создает строку по другому источнику истины: по Exception Directory и RUNTIME_FUNCTION. Иногда эти две картины совпадут. Иногда Ghidra уже найдет функцию там же. Иногда Ghidra ошибется или не увидит диапазон, потому что control flow необычный. Иногда .pdata сама будет повреждена. Интеграция нужна для того, чтобы эти картины можно было положить друг на друга.

Как `report.c` готовит Ghidra CSV

На стороне C-кода за экспорт отвечает та же функция, которая пишет обычный CSV: report_write_csv. Разница задается флагом ghidra_mode. Если ghidra_mode false, функция пишет подробную таблицу для общего анализа. Если ghidra_mode true, она меняет контракт вывода и пишет ровно четыре колонки: begin_rva, end_rva, name, comment. Это маленькое условие в коде важно архитектурно: проект не создает отдельный параллельный writer с другой логикой и не рискует рассинхронизировать два представления. Он берет те же RuntimeFunction, UnwindSummary, us_ok[] и просто сериализует их в форму, удобную для Ghidra.

RuntimeFunction дает диапазон. UnwindSummary дает смысл комментария. us_ok[] говорит, удалось ли нормально разобрать UNWIND_INFO для данной записи. Это очень важная тройка. Если parser успешен, report.c вызывает внутреннюю сборку human-readable unwind comment. Если parser провалился, Ghidra CSV все равно получает строку диапазона, но comment становится warning-сообщением вида UNWIND parse failed: .... Такой дизайн лучше, чем молча выбросить проблемную запись. Для реверсера поврежденный unwind record часто важнее нормального: именно он может указывать на битый файл, нестандартную генерацию, packer, obfuscation, ошибку tooling или намеренно сломанную metadata.

```
if (ghidra_mode) {
    sb_append(&output, "begin_rva,end_rva,name,comment\n");
    for each RuntimeFunction r and UnwindSummary u:
        name = "sub_%08X" % r.begin_rva
        if (us_ok[i]) comment = build_unwind_comment(u)
        else comment = "UNWIND parse failed: " + u.parse_error
        write CSV cells: begin_rva, end_rva, name, comment
}
```

В этом фрагменте хорошо видно разделение обязанностей. report.c не решает, правдив ли RUNTIME_FUNCTION. Эту структуру уже прочитал unwind.c на основе карты pe.c. report.c не придумывает новые findings. Их уже создал heuristics.c. report.c только делает понятный перенос в формат. Но “только” здесь не значит “мелочь”. Ошибка в quoting CSV, потерянная запятая, неэкранированная кавычка или неправильно записанный диапазон превратят корректный analysis result в неправильный рабочий артефакт. Именно поэтому в предыдущем разделе report.c рассматривался как часть анализа, а не как украшение в конце.

CSV escaping особенно важен из-за поля `comment`. Комментарий может содержать пробелы, скобки, списки регистров, parse error text, anomalies, иногда более длинные диагностические строки. Если писать CSV руками через `fprintf` без quoting rules, одна кавычка или запятая может сдвинуть колонки, и importer начнет читать `end_rva` как кусок комментария. В проекте это решается через отдельную функцию `csv_cell`, которая превращает значение в корректную CSV-cell. Это маленькая инженерная деталь, но для связки с Ghidra она критична: адресная разметка не прощает “примерно правильно”.

Почему адреса передаются как RVA, а не как VA

Самый важный технический момент Ghidra integration - адреса. В CSV проект пишет не абсолютные virtual addresses и не file offsets, а RVA. RVA - relative virtual address, относительный виртуальный адрес внутри image. Он измеряется от image base. Это тот же язык, на котором PE Data Directories указывают таблицы, `RUNTIME_FUNCTION` указывает `BeginAddress`, `EndAddress`, `UnwindInfoAddress`, а Exception Directory указывает массив `.pdata`. Поэтому для EH/Unwind Analyzer естественно экспортировать именно RVA: он анализирует PE как image model, но не знает заранее, с каким base Ghidra откроет программу и с каким base процесс загрузится во время dynamic run.

Если бы CSV хранил VA вроде `0x1400013F0`, он был бы сильнее привязан к конкретному preferred `ImageBase`. Но Ghidra Program может иметь свой image base. Runtime process из-за ASLR может быть загружен вообще по другой базе. File offset вообще живет в другой системе координат и зависит от section raw layout. RVA - это самый устойчивый мост между PE-файлом и Ghidra Program: importer берет `currentProgram.getImageBase().getOffset()`, прибавляет `begin_rva` и `end_rva`, а затем проверяет, что полученные addresses действительно лежат в memory map текущей программы.

```
image_base = currentProgram.getImageBase().getOffset()
start = toAddr(image_base + begin_rva)
end   = toAddr(image_base + end_rva - 1)
```

Минус один в `end_rva - 1` не случайность. `RUNTIME_FUNCTION` описывает half-open range: начало включительно, конец не включительно. Это привычная модель `[begin, end)`: адрес `begin_rva` принадлежит функции, последний байт функции находится перед `end_rva`, а сам `end_rva` уже является границей. Ghidra `AddressSet(start, end)` задает закрытый диапазон start..end, поэтому importer переводит half-open model в inclusive Ghidra body через `end_rva - 1`. Если этого не сделать, функция в Ghidra захватит один лишний байт после своего диапазона. Для обычного текста это выглядит мелочью. Для disassembly range это может означать перекрытие с соседней функцией, неправильный body и ложный skip из-за overlap.

Это место еще раз показывает, почему предыдущие главы так настаивали на различии VA, RVA, `file offset`, preferred image base and runtime base. Ghidra integration не может быть надежной, если автор инструмента сам путает системы координат. Вся логика importer строится на фразе: CSV хранит RVA; Ghidra показывает VA; conversion is image_base + rva; memory map must confirm that resulting VA belongs to the current Program. Если открыть в Ghidra не тот бинарник, или взять CSV от другого build, `memory.contains` и overlap checks начнут отбрасывать строки. Это не раздражающее препятствие, а защитный механизм.

Как устроен `import_unwind_csv.py`

Теперь можно пройти importer как маленькую программу. Его начало задает несколько постоянных величин. `REQUIRED_COLUMNS` фиксирует контракт CSV: без `begin_rva`, `end_rva`, `name`, `comment` скрипт не имеет права работать. `COMMENT_PREFIX` задает тег, по которому importer отличает собственную строку комментария от ручных комментариев пользователя. `BOOKMARK_CATEGORY` задает категорию warning bookmarks. `MAX_REPORTED_ERRORS` ограничивает шум в консоли: если CSV сильно не совпадает с текущей программой, скрипт не должен засыпать пользователя тысячей одинаковых ошибок.

```
REQUIRED_COLUMNS = set(["begin_rva", "end_rva", "name", "comment"])
COMMENT_PREFIX = "[EH/Unwind Analyzer] "
BOOKMARK_CATEGORY = "EH/Unwind Analyzer"
MAX_REPORTED_ERRORS = 25
```

Дальше скрипт спрашивает файл через `askFile`. Это не обычная функция Python из стандартной библиотеки; это функция, которую Ghidra script получает от scripting environment. Она открывает диалог выбора файла внутри Ghidra. Затем importer берет `currentProgram`, то есть программу, которая сейчас открыта в CodeBrowser. Это тоже принципиально: CSV не импортируется “в вакууме”. Он импортируется в конкретный Program, уже загруженный через Ghidra loader, с конкретным image base, memory map, function manager and bookmark manager.

```
csv_file = askFile("Select EH/Unwind Analyzer Ghidra CSV", "Open")
image_base = currentProgram.getImageBase().getOffset()
memory = currentProgram.getMemory()
function_manager = currentProgram.getFunctionManager()
bookmark_manager = currentProgram.getBookmarkManager()
```

Здесь importer входит в мир Ghidra API. `Memory` отвечает на вопрос, существует ли адрес в загруженной программе. `FunctionManager` управляет функциями: можно спросить, есть ли функция в точке, есть ли функция, содержащая адрес, какие функции пересекают range, можно создать новую функцию. `BookmarkManager` управляет bookmarks, то есть пометками в интерфейсе. В терминах архитектуры проекта importer не просто “печатает комментарии”. Он меняет Ghidra Program database, поэтому должен делать

это внутри transaction.

Transaction - это атомарная сессия изменений в Ghidra Program. Скрипт вызывает `currentProgram.startTransaction(...)`, выполняет создание функций, обновление body, переименование, комментарии и bookmarks, а затем завершает transaction через `endTransaction(transaction, commit)`. Если все прошло нормально, `commit` становится true. Если случилось исключение, transaction закрывается без фиксации. Для реверсера это важно практически: импорт не должен оставить программу в наполовину измененном состоянии, где часть функций создана, часть comments поставлена, а потом скрипт упал.

Почему importer консервативен

В заголовке самого `import_unwind_csv.py` прямо сказано, что importer conservative. Conservative здесь переводится не как “политический консерватор”, а как “осторожный, не делающий лишних предположений”. В reverse engineering это жизненно важная черта. Инструмент, который слишком активно переименовывает функции, перетирает комментарии, расширяет function bodies поверх соседних функций и прячет конфликты, быстро перестает помогать. Он начинает портить рабочее состояние.

Первое проявление осторожности - проверка колонок. Если CSV не содержит нужных полей, importer сразу останавливается. Он не пытается угадать, что `start` означает `begin_rva`, а `description` означает `comment`. Автоматическое угадывание удобно в бытовых программах, но опасно в инструментах анализа. Здесь формат является контрактом между `report.c` и `import_unwind_csv.py`, а контракт должен нарушаться громко.

Второе проявление - проверка range. `begin_rva` должен быть неотрицательным, `end_rva` должен быть строго больше `begin_rva`, а оба converted addresses должны лежать в memory текущей программы. Эта проверка ловит сразу несколько классов ошибок: CSV от другого файла, CSV от другой версии build, поврежденный CSV, неверный image base, range в unmapped area. Importer не пытается “все равно создать функцию где-то рядом”, потому что адреса в реверсе не приблизительные. Если range не совпал, строка пропускается и попадает в счетчик `skipped`.

```
if begin_rva < 0 or end_rva <= begin_rva:
    raise ValueError("invalid half-open range")

if not memory.contains(start) or not memory.contains(end):
    raise ValueError("range is outside program memory")
```

Третье проявление - осторожная работа с уже существующими функциями. Ghidra могла сама создать функции своим analyzer-ом. Пользователь мог руками поправить boundaries. Другой script мог уже импортировать symbols. Поэтому importer сначала смотрит, есть ли function exactly at start, есть ли

function containing start, и есть ли foreign overlaps with proposed body. Если entry попадает внутрь чужой функции, importer не режет эту функцию автоматически. Если proposed range пересекает другие functions, importer не пытается силой победить Ghidra. Он пропускает строку и сообщает проблему.

```
function = function_manager.getFunctionAt(start)
containing = function_manager.getFunctionContaining(start)

if function is None and containing is not None:
    raise ValueError("entry lies inside existing function")

overlaps = collect_foreign_overlaps(function_manager, body, own_entry)
if overlaps:
    raise ValueError("range overlaps existing function(s)")
```

На первый взгляд это может раздражать: почему бы не доверять `.pdata`, если вся книга доказывала важность Exception Directory? Потому что интеграция находится уже в рабочей среде человека. Здесь нужно уважать состояние Ghidra Program. `.pdata` является сильным свидетельством, но не абсолютным правом перезаписать все, что уже сделал analyst. Если есть конфликт, правильное поведение - показать конфликт и дать человеку решить. В этом смысле importer продолжает философию всего проекта: evidence, not dictatorship.

Четвертое проявление - naming policy. Если функция уже существует, importer может переименовать ее в `sub_XXXXXXXX` только когда текущее имя имеет source type `DEFAULT` или `ANALYSIS`. То есть если имя пришло от автоматического анализа Ghidra или является дефолтным, его можно заменить на импортированное. Но если имя поставил пользователь, loader, symbol source или другой более сильный источник, importer должен его сохранить. В проектном коде это видно через функцию `can_replace_name`.

```
def can_replace_name(function):
    source = function.getSymbol().getSource()
    return source == SourceType.DEFAULT or source == SourceType.ANALYSIS
```

Это решение кажется маленьким, но оно очень правильно воспитано. Инструмент не должен стирать человеческую работу. Если реверсер уже назвал функцию `dispatch_exception_route` или `custom_unwind_handler`, автоматический импорт не должен вернуть ее к `sub_000013F0`. Автоматические имена нужны там, где имен еще нет. Там, где человек уже вложил смысл, инструмент должен отступить.

Комментарии: как importer не уничтожает ручные заметки

Комментарии в Ghidra особенно чувствительны. Для реверсера comment - это не “мелкий текст над функцией”, а накопленная память расследования. Поэтому `import_unwind_csv.py` не делает грубое `setPreComment(start, imported_comment)` поверх всего. Он сначала читает existing pre-comment, удаляет только старые строки, которые начинаются с `[EH/Unwind Analyzer]`, сохраняет остальные строки, а затем добавляет свежую analyzer-line. Это поведение задает функция `analyzer_comment`.

```
def analyzer_comment(existing, imported):
    tagged = COMMENT_PREFIX + imported
    if not existing:
        return tagged
    kept = []
    for line in existing.splitlines():
        if not line.startswith(COMMENT_PREFIX):
            kept.append(line)
    kept.append(tagged)
    return "\n".join(kept)
```

Эта функция решает сразу две задачи. Во-первых, повторный импорт не будет плодить десять одинаковых строк `[EH/Unwind Analyzer] ...`. Старый analyzer-comment удалится, новый поставится. Во-вторых, ручные комментарии останутся. Если реверсер над функцией написал “похоже на central dispatcher, проверить call sites”, importer эту строку не тронет. Он добавит рядом свою строку про `UNWIND_INFO`, но не заберет у человека рабочее пространство.

В итоге комментарий в Listing становится небольшим синтезом машинного и человеческого знания. Машина пишет: `UNWIND v1 flags=EHANDLER prolog=4 stack=40B ... handler_rva=...`. Человек рядом может написать: “handler target похож на общую точку маршрутизации исключений”. Именно так и должен работать инструмент reverse engineering: не заменять мышление реверсера, а экономить ему время на переносе структурных фактов в место анализа.

Bookmarks: почему warning появляется только в особых случаях

Bookmarks в Ghidra - это навигационные пометки. В данном проекте importer использует warning bookmarks не для любого finding и не для любого подозрительного score, а для локально понятных проблем Ghidra CSV row: если в comment есть `parse failed` или `anomalies=`, скрипт ставит bookmark типа `WARNING` категории `EH/Unwind Analyzer`. Это снова консервативный выбор. Не всякая suspicious assessment должна превращаться в bookmark на конкретной функции. Некоторые findings относятся к distribution across table, section placement, handler dominance or runtime behavior. Такие вещи лучше читать в HTML/JSON report. Bookmark нужен там, где есть конкретный адрес функции, к которому можно привести человека.

```
lowered = comment.lower()
if "parse failed" in lowered or "anomalies=" in lowered:
    bookmark_manager.setBookmark(
        start,
        BookmarkType.WARNING,
        BOOKMARK_CATEGORY,
        comment,
    )
```

Представим static sample, где одна запись `RUNTIME_FUNCTION` указывает на `UNWIND_INFO` по RVA, который не отображается в file offset. Для JSON report это finding `unwind_rva_oob` и `unwind_parse_fail`. Для Ghidra это warning bookmark на функции, к которой относится запись. Человек открывает Bookmarks window, кликает по warning и сразу оказывается в Listing у проблемного range. Это хороший пример правильного переноса: global report говорит, что проблема существует; Ghidra bookmark приводит к месту, с которого удобно начинать ручную проверку.

Но если finding говорит, например, что большое число `UNWIND_INFO` лежит в writable section, один bookmark на каждую функцию может превратить интерфейс в шум. В таком случае HTML report лучше показывает картину: affected_count, severity, family, representative RVA, manual validation step. Ghidra integration не должна автоматически размазывать все findings по UI. Хороший importer должен понимать не только что он может поставить bookmark, но и когда bookmark будет полезен.

Создание и обновление функций: где `.pdata` встречается с Ghidra FunctionManager

Когда range прошел проверки, importer строит `AddressSet(start, end)` и работает с `FunctionManager`. Если функции в точке `start` нет, он вызывает `createFunction` с `SourceType.IMPORTED`. Это означает, что функция создана не внутренним анализатором Ghidra и не вручную, а импортирована из внешнего источника. Этим внешним источником является EH/Unwind Analyzer. Если функция уже есть, importer может обновить ее body, если body не совпадает, и может переименовать, если naming policy разрешает.

```
if function is None:
    function = function_manager.createFunction(
        function_name,
        start,
        body,
        SourceType.IMPORTED,
    )
else:
    if not function.getBody().equals(body):
        function.setBody(body)
```

Здесь снова видна разница между “функцией по control flow” и “функцией по unwind metadata”. Ghidra может создать function body по результату disassembly and analysis. EH/Unwind Analyzer приносит другой источник boundaries: `RUNTIME_FUNCTION.BeginAddress` and `EndAddress`. Иногда этот источник помогает поправить недостроенный function. Иногда он конфликтует с уже существующим function layout, и importer скипает строку. В любом случае человек получает возможность сравнить две модели: что сказал дизассемблер и что говорит Windows unwind contract.

Особенно это полезно там, где обычный linear sweep или recursive descent analysis не уверен из-за exception-based control flow, нестандартных переходов, ручной сборки, compiler artifacts or generated code. `.pdata` не доказывает семантику функции, но указывает, что Windows должна иметь unwind model для диапазона. Для reverse engineering это очень сильный намек: “этот range важен для stack unwinding and exception handling”. Пометить его в Ghidra значит сделать невидимую metadata видимой в основном рабочем окне.

Как реверсер читает результат в Ghidra

После успешного запуска importer печатает summary в Ghidra console: сколько functions created, updated, renamed, commented, сколько bookmarks поставлено и сколько строк skipped. Эти счетчики надо читать как диагностический отчет маленькой операции импорта. `created` показывает, сколько ranges из `.pdata` раньше не были функциями в Ghidra. `updated` показывает, что функции уже существовали, но importer прошел по ним и добавил или обновил данные. `renamed` говорит, что автоматическое или дефолтное имя было заменено на `sub_XXXXXXXX`. `commented` показывает, сколько функций получили `[EH/Unwind Analyzer]` pre-comment. `bookmarks` показывает локальные предупреждения. `skipped` показывает конфликты, неверные ranges, mismatch между CSV и Program or other import problems.

```
EH/Unwind import complete: created=12 updated=34 renamed=8 commented=46 bookmarks=1 skipped=0
```

Идеальный результат не всегда означает `created` больше нуля. Если Ghidra сама уже нашла все функции, `created=0` может быть совершенно нормальным. Тогда важнее `commented`: появились ли analyzer comments. Если `skipped=0`, importer не встретил конфликтов. Если `bookmarks=1`, это не “ошибка скрипта”, а сигнал, что по крайней мере одна строка содержит parse failure или anomalies. Если `skipped` большой, первая гипотеза должна быть не “Ghidra плохая”, а “CSV не от этого бинарника, открыт другой Program, image base/memory map не совпали, или Ghidra already has overlapping functions that need manual review”.

В Listing реверсер видит строку, начинающуюся с `[EH/Unwind Analyzer]`. Это intentional prefix, а не декоративная подпись. Prefix решает проблему повторного импорта и отделяет машинную строку от человеческих комментариев. Внутри строки реверсер ищет несколько смысловых узлов: version, flags, prolog size, stack allocation, frame register, saved registers, handler RVA, parse errors or anomalies. Эта строка не заменяет HTML report, но дает именно ту информацию, которая нужна в момент просмотра функции.

Если в комментарии написано `flags=EHANDLER`, реверсер должен вспомнить главу про frame-based SEH: у функции есть exception handler data, и после unwind codes в `.xdata` лежит handler pointer and language-specific data. Если есть `handler_rva`, его надо открыть и посмотреть, что это за target. Если stack size выглядит странно, надо сравнить с prolog. Если parse failed, надо вернуться к `.xdata` in hex view или посмотреть `parse_error` в report. Ghidra integration не дает ответ, но резко сокращает путь от вопроса до адреса.

Почему Ghidra view нельзя читать без HTML/JSON report

Очень важно не переоценить Ghidra importer. Он переносит function ranges and comments, но не переносит всю логику `HeuristicReport` и `DynamicReport`. Например, finding `handler_dominance` может быть связан не с одной функцией, а с концентрацией многих функций на одном handler target. В Ghidra можно открыть эти функции и handler, но сам факт dominance лучше виден в HTML/JSON: там есть affected_count, representative_rva, evidence family and severity. То же касается `unwind_reuse`: если много `RUNTIME_FUNCTION` ссылаются на один `UnwindInfoRVA`, это distribution-level signal, а не просто одна строка комментария.

`dynamic_validation` почти полностью остается вне Ghidra CSV. Если sample во время запуска вызвал `RtlAddFunctionTable` или `RtlInstallFunctionTableCallback`, если были VEH registrations, handler hits, `RIP trace`, timeout or dropped_events, это живет в `DynamicReport` and assessment. Ghidra CSV не становится runtime timeline. И это правильно: CSV importer работает с статическим Program in Ghidra, а dynamic collector работает с debuggee process. Их можно связать руками через addresses and reports, но нельзя притворяться, что Ghidra comment сам по себе доказал runtime behavior.

Правильный workflow выглядит так. Сначала реверсер открывает HTML/JSON report и видит общую картину: PE validation, unwind table, findings, evidence families, dynamic status, assessment, manual validation steps. Затем он открывает Ghidra и запускает CSV importer, чтобы function ranges and unwind comments появились прямо в Listing. Потом он двигается туда-сюда: report говорит, какие места важны; Ghidra показывает code and context; manual validation решает, какая версия происходящего выдерживает проверку.

Сценарий нормального импорта: combo sample

В проектной памятке для Ghidra есть демонстрационный сценарий с combo-бинарником. Анализатор запускается с `--ghidra-csv` и `--heuristics`, создает CSV, а затем importer размечает функции в Ghidra. Смысл такого сценария не в том, чтобы доказать “все опасно”, а в том, чтобы показать весь путь: Exception Directory содержит множество `RUNTIME_FUNCTION`; `unwind.c` декодирует их; `report.c` собирает Ghidra CSV; Ghidra importer создает или обновляет ranges, добавляет comments and leaves bookmarks zero, если parse failures нет.

```
.\build\debug\eh_unwind_analyzer.exe ^
  "...\28_combo_rtl_xdata_renamed_writable.exe" ^
  --ghidra-csv reports\combo_28_ghidra.csv ^
  --heuristics
```

После импорта реверсер смотрит Functions window и Listing. У функций появляются имена вида `sub_000013F0`, а над entry point появляется pre-comment. Если HTML report говорит, что unwind metadata лежит в unusual or writable section, Ghidra importer не обязан сам раскрашивать все это красным. Он дает карту функций и comments, а общую причину suspicious assessment надо читать в report. Это разделение уменьшает риск UI-noise: не каждое глобальное finding превращается в локальный bookmark.

Сценарий warning/bookmark: static parse failure

Второй демонстрационный сценарий показывает поврежденную или невозможную для чтения unwind-запись. Анализатор создает Ghidra CSV, где comment для одной строки начинается с `UNWIND parse failed`. Importer видит эту строку, добавляет pre-comment and creates warning bookmark. Реверсер открывает Bookmarks window, кликает по warning, попадает к range, а затем сверяет с HTML/JSON report: там будут findings уровня `unwind_rva_oob` and `unwind_parse_fail` or equivalent diagnostics.

```
.\build\debug\eh_unwind_analyzer.exe ^
  "...\03_static_unwind_oob_parse_fail.exe" ^
  --ghidra-csv reports\static_03_ghidra.csv ^
  --heuristics
```

Такой сценарий особенно полезен учебно. Он показывает, что importer не скрывает parse failure. Он не говорит: “раз не удалось декодировать `UNWIND_INFO`, значит функции нет”. Наоборот, range из `.pdata` остается видимым, а failure становится warning at the function entry. Это очень правильное поведение для реверса: поврежденная metadata должна становиться заметнее, а не исчезать из интерфейса.

Жизнь одной строки CSV: от bytes в файле до pre-comment в Listing

Чтобы не потерять связь между слоями, полезно пройти один ряд CSV медленно. На самом нижнем уровне когда-то был PE-файл как raw bytes. `util.c` безопасно прочитал эти bytes, `pe.c` построил `PeImage`, нашел sections and Exception Directory, `unwind.c` прочитал `RUNTIME_FUNCTION` entry and decoded `UNWIND_INFO`, `report.c` собрал строку `begin_rva,end_rva,name,comment`. Потом эта строка попадает в `csv.DictReader` внутри Ghidra. То есть к моменту, когда importer видит row, за ней уже стоит весь статический pipeline.

Importer сначала извлекает `begin_rva` and `end_rva`. Он не доверяет им как готовым адресам. Он переводит строки в integers через `parse_integer`, причем разрешает и hex form with `0x`, and decimal form. Это важно не потому, что project currently writes decimal sometimes - project writes hex - а потому, что importer формально не привязан к одному косметическому виду. Но после

parsing он сразу возвращается к строгой модели: range must be half-open and non-empty. Пустая функция, reversed range, negative start - это не “странная функция”, а invalid row.

Затем row переходит из CSV coordinate system в Ghidra coordinate system. Здесь происходит главный перевод: `image_base + rva`. В этот момент importer не знает, является ли range code, data, thunk or something else. Он знает только, что Ghidra Program memory must contain both start and final byte. Проверка `memory.contains` не доказывает семантику, но защищает от грубого рассинхрона. Если адреса не принадлежат current Program, importer не должен создавать “висячую” функцию в пустоте.

После этого строится `AddressSet`. В Ghidra function body - это не просто пара чисел. Это set of addresses, который может быть непрерывным или составным. Текущий importer использует простой непрерывный body from start to end. Для `RUNTIME_FUNCTION` это естественно: Windows unwind table тоже описывает contiguous range. Но Ghidra как система допускает более сложные bodies. Здесь проект снова выбирает простую модель, потому что `.pdata` itself gives a contiguous begin/end interval.

Затем importer проверяет существующие функции. Если function exists exactly at entry, можно обновлять ее body and comment. Если entry лежит внутри чужой функции, importer не разрезает ее. Если proposed body overlaps foreign functions, importer не перетирает их. И только если конфликтов нет, row превращается в create/update operation. Такой путь кажется длинным, но именно он отличает инженерный importer от скрипта “прочитал CSV и насоздавал функций любой ценой”.

В конце row получает комментарий. Даже здесь importer не просто пишет text. Он сначала удаляет предыдущую analyzer-line by prefix, сохраняет human notes, добавляет новую analyzer-line, а затем при необходимости ставит bookmark. Получается, что одна строка CSV проходит через пять фильтров: format validation, numeric validation, address validation, function-conflict validation, comment/bookmark policy. Это и есть правильная архитектура моста: каждая граница явно говорит, чему она доверяет, а чему нет.

CSV как вход тоже недоверенный

Есть тонкий момент: после всех разговоров о PE как untrusted input можно случайно начать считать CSV безопасным, потому что его “сгенерировал наш анализатор”. Но хороший инструмент так не думает. CSV-файл может быть старым, от другого sample, вручную измененным, поврежденным, открытым не в той кодировке, с лишними колонками, с отсутствующими колонками, с огромными numbers or malformed values. Поэтому importer не исполняет содержимое CSV, не вызывает `eval`, не импортирует произвольные команды и не доверяет address ranges до проверки.

С точки зрения угроз это не sandbox, но это важная гигиена. Ghidra Program - рабочая база анализа. Если script без проверок принимает CSV and mutates Program, он может испортить базу, затереть имена, разрушить function boundaries or flood bookmarks. `import_unwind_csv.py` ограничивает ущерб: required columns, integer parsing, memory check, overlap check, naming policy, comment prefix policy, max reported errors. В результате даже плохой CSV скорее даст `skipped`, чем тихо сломает разметку.

Здесь видно, как философия `util.c` повторяется на уровне Ghidra. В `util.c` нельзя читать bytes без bounds check. В importer нельзя применять row без semantic check. В `report.c` нельзя записывать CSV без escaping. В `heuristics.c` нельзя превращать score into probability. В `dynamic_win.c` нельзя считать timeout полноценным absence of evidence. Каждый слой проекта имеет свой тип недоверенного входа и свою форму осторожности.

Script Manager, Ghidra API и почему это не обычный Python запуск

Файл `import_unwind_csv.py` выглядит как Python script, но его нельзя понимать как обычную программу, которую запускают командой `python import_unwind_csv.py` из терминала. Он рассчитан на выполнение внутри Ghidra scripting environment. Именно поэтому в нем доступны `askFile`, `currentProgram`, `toAddr`, `getPreComment`, `setPreComment`, `monitor` and other names, которые обычный CPython не знает. Эти names приходят из Ghidra script runtime and FlatProgramAPI-like convenience layer.

Практически это значит, что пользователь должен добавить папку `ghidra` проекта в Script Manager и запустить script from CodeBrowser. Скрипт работает с currently opened Program. Если Program не открыт, нет контекста. Если открыт другой Program, CSV применится к другой address map и начнутся skips. Поэтому Ghidra integration - это не batch-converter by default. Это interactive bridge для текущей сессии reverse engineering.

Это также объясняет, почему importer использует `monitor.checkCancelled()`. Ghidra scripts являются one-shot actions, но пользователь должен иметь возможность отменить долгую операцию. Для 46 строк это не важно. Для тысяч `RUNTIME_FUNCTION` entries в большом C++ binary or Chromium-like binary это уже имеет значение. Каждые 100 строк script обновляет message: importing row N. Это небольшая, но правильная UI-деталь. Инструмент не должен подвешивать рабочую среду молча.

Как читать конфликт между Ghidra analysis и `.pdata`

Иногда после импорта важнее всего не созданные функции, а skipped rows. Например, Ghidra уже построила функцию, которая начинается раньше, а RUNTIME_FUNCTION range начинается внутри нее. Что это значит? Не обязательно, что Ghidra ошиблась. Не обязательно, что .pdata повреждена. Это значит, что у нас появилось противоречие между двумя моделями разбиения кода. Одна модель построена на disassembly/control-flow analysis, другая - на Windows unwind metadata.

Правильный реверсер не выбирает победителя мгновенно. Он открывает адрес, смотрит prolog, epilog, calls, fallthroughs, alignment, padding, exception directory, соседние RUNTIME_FUNCTION entries and compiler pattern. Иногда Ghidra объединила несколько маленьких ranges в одну большую функцию. Иногда .pdata действительно описывает inner funclet or compiler-generated helper. Иногда binary intentionally confuses metadata. Иногда importer range попал в середину из-за wrong CSV/sample mismatch. Конфликт - это не шум; это место для manual validation.

Поэтому skipped rows должны быть сохранены в console output. Проект не прячет их, но и не превращает автоматически в findings, потому что это уже состояние Ghidra Program, а не свойство PE-файла само по себе. Один и тот же CSV может импортироваться без skips в чистый Program and with skips in a heavily edited Program. Это важная граница между analyzer evidence and workspace state.

Почему комментарий должен быть коротким

Можно спросить: если HTML/JSON report знает так много, почему бы не положить весь HeuristicFinding and assessment прямо в Ghidra pre-comment? Ответ: потому что Listing - это не место для длинного отчета. Комментарий над функцией должен помогать читать код, а не превращать disassembly в стену текста. Поэтому Ghidra CSV comment - это compact unwind summary. Он дает то, что нужно рядом с code: unwind version, flags, prolog, stack, frame, saved registers, handler RVA or parse failure.

Длинные выводы остаются в HTML. Structured fields остаются в JSON. Severity and evidence families остаются в heuristic_analysis. Dynamic events остаются в dynamic_validation. Ghidra pre-comment должен быть как табличка у двери: “здесь такая unwind-модель, вот handler, вот warning”. Если человеку нужно больше, он идет в report. Это разделение носителей информации делает проект удобнее: каждый формат говорит столько, сколько уместно именно в нем.

Какие данные импортёр намеренно не переносит

Теперь нужно честно перечислить границы интеграции, но не в стиле “инструмент слабый”, а в стиле инженерного контракта. `import_unwind_csv.py` не переносит full JSON. Он не создает отдельные data structures for `UNWIND_INFO` in `.xdata`. Он не ставит cross-references от функции к handler target. Он не создает labels на handler RVA. Он не переносит `HeuristicFinding` как typed bookmarks по severity. Он не добавляет colorization. Он не создает graph of exception routes. Он не импортирует dynamic events as timeline. Он не запускает Ghidra decompiler and does not modify function signatures. Это не недоработки текущего текста книги; это реальные ограничения текущей интеграции.

Почему так можно? Потому что первая версия bridge должна быть надежной и объяснимой. Она решает базовую боль: сделать `.pdata` ranges and unwind comments видимыми в Ghidra. Уже это сильно улучшает ручной анализ Windows x64 PE. Более богатая интеграция требует больше политики: куда ставить label, каким именем назвать handler, когда создавать reference, что делать с конфликтом, как не затереть пользовательские bookmarks, как обновлять результаты после повторного анализа, как хранить связи between static findings and dynamic events. Эти вопросы лучше решать после того, как минимальный мост стабилен.

Типичные ошибки использования и как их диагностировать

Самая частая ошибка - открыть в Ghidra один EXE, а CSV взять от другого. Для человека это может быть “почти тот же sample”, но для importer это другой address map. Тогда ranges могут оказаться outside program memory, entry may lie inside existing function, overlaps may appear, and `skipped` grows. Правильная реакция - проверить SHA-256 in report metadata, убедиться, что CSV generated from the same binary, и повторить импорт.

Вторая ошибка - забыть, что CSV хранит RVA, а Ghidra показывает VA. Пользователь видит `0x000013F0` в CSV и не находит такой адрес в Listing, потому что Ghidra показывает `0x1400013F0` for a typical PE64 base. Формула остается прежней: `Ghidra address = currentProgram image base + RVA`. Если base другой, результат будет другой. Это не баг CSV, а нормальная работа PE coordinate systems.

Третья ошибка - считать `created=0` плохим результатом. Если Ghidra already created functions, importer может просто update and comment them. Тогда `created=0`, `updated>0`, `commented>0` - хороший результат. Четвертая ошибка - игнорировать `skipped`. Даже если большая часть функций импортировалась, skipped rows надо прочитать. Importer печатает первые `MAX_REPORTED_ERRORS` problems, чтобы не утопить console. Эти сообщения

часто сразу говорят: wrong file, overlap, range outside memory, invalid half-open range.

Пятая ошибка - ожидать, что Ghidra importer покажет dynamic validation. Он этого не делает. Если вопрос связан с `RtlAddFunctionTable`, VEH registration, handler hit, first-chance exception, timeout or `RIP trace`, смотреть надо HTML/JSON dynamic_validation block. В Ghidra можно открыть статические handler targets and ranges, но runtime events остаются в DynamicReport.

Почему это мост к manual validation, а не финальный судья

Вся книга постепенно вела к одной мысли: reverse engineering - это не угадывание одного яркого признака, а сборка проверяемой версии. Ghidra integration идеально ложится в эту мысль. Анализатор приносит в Ghidra evidence from PE/unwind metadata. Но человек должен проверить code shape, prolog, epilog, handler target, section permissions, cross-references, compiler patterns and report findings. Importer не говорит “эта функция вредоносная”. Он говорит: “по `.pdata` есть такой range, вот его unwind summary, вот warning, если decode failed”.

Такой подход особенно важен для учебного проекта. Если бы importer автоматически ставил ярлыки `malicious_handler`, `obfuscated_function`, `bad_unwind`, он приучал бы читателя доверять словам инструмента. Текущий importer делает другое: он переносит факты, а не приговоры. `sub_000013F0` is a range. `handler_rva=0x...` is a pointer-like metadata fact. `UNWIND parse failed` is a parser failure. `WARNING bookmark` is a navigation hint. Assessment остается в report, а окончательная интерпретация остается у реверсера.

Как можно развивать интеграцию дальше

Следующий естественный шаг - JSON importer. Он мог бы читать не только ranges and comments, но и structured findings, severity, family, assessment, manual validation steps. Тогда Ghidra могла бы получать bookmarks разных категорий: `table_integrity`, `unwind_integrity`, `metadata_placement`, `handler_dominance`. Но такой importer должен очень осторожно проектировать mapping: не каждое finding имеет один точный address; иногда finding относится к группе функций, секции или distribution pattern.

Второй шаг - labels for handler targets. Если `UNWIND_INFO` содержит `handler_rva`, importer мог бы поставить label вроде `eh_handler_000027A0` or comment at handler target. Но здесь появляются вопросы. Один handler может обслуживать много functions. Handler target может уже иметь имя. Handler RVA может указывать на thunk, runtime helper or language-specific personality routine. Слишком агрессивное labeling снова рискует разрушить работу пользователя. Поэтому правильный путь - labels only when source type permits

and with clear prefix.

Третий шаг - references. Можно добавить memory/code references от function entry comment or some synthetic location to handler target. Это улучшит navigation, но reference model in Ghidra требует аккуратности: искусственная reference не должна притворяться настоящим machine instruction operand. Возможно, лучше использовать bookmarks and comments instead of xrefs, unless importer can place references semantically clearly.

Четвертый шаг - data type overlay for `.pdata` and `.xdata`. Ghidra могла бы видеть `RUNTIME_FUNCTION` arrays and `UNWIND_INFO` structures as typed data. Это было бы ближе к полноценному Ghidra analyzer. Но тогда integration перестает быть маленьким CSV script and becomes plugin/analyzer. Это другой уровень сложности: нужно учитывать loader state, memory blocks, existing data, conflict resolution, repeated analysis and user edits.

Пятый шаг - dynamic evidence overlay. Если DynamicReport contains `unwind_handler_hit`, importer could put bookmark at handler target saying “observed hit in dynamic run”. Но здесь особенно опасно смешать scenario-specific evidence with static facts. Runtime hit означает “в этом запуске, с этими аргументами, при этих условиях, collector observed event”. Bookmark должен формулироваться именно так, а не “handler always runs”.

Итог: Ghidra integration как честный перевод между мирами

Если собрать все вместе, `ghidra/import_unwind_csv.py` - маленький, но очень важный слой проекта. Он переводит результат EH/Unwind Analyzer из языка report-артефакта в язык Ghidra Program. На входе у него CSV with RVA ranges and comments. Внутри он использует Ghidra API: current program, image base, memory map, function manager, address sets, transactions, comments, bookmarks. На выходе реверсер получает функции, pre-comments and warning bookmarks прямо в Listing.

Его ценность не в сложности алгоритма. Алгоритм нарочно простой. Ценность в правильной границе. Основной анализ остается в `util.c`, `pe.c`, `unwind.c`, `heuristics.c`, `dynamic_win.c`, `report.c`. Ghidra importer не дублирует эти слои и не спорит с ними. Он переносит уже подготовленную модель туда, где человек реально принимает решения. Это и есть хорошая интеграция: не заменить основную систему, не заменить человека, а сделать так, чтобы структурные факты оказались в нужном месте, в нужном виде и с честной степенью уверенности.

После этого раздела архитектура проекта почти собрана полностью. Мы увидели raw bytes, PE parser, unwind decoder, evidence engine, dynamic collector, reporting layer and Ghidra bridge. Остался последний инженерный слой, без

которого такой инструмент нельзя считать надежным: tests, corpus, fuzzing and validation. Если Ghidra integration показывает результат человеку, то тесты должны доказать, что этот результат не сломается от malformed PE, странной `.xdata`, ошибочного CSV escaping, wrong schema field or regression in parser. Поэтому следующий раздел логично посвятить не красивому выводу, а проверяемости проекта.

11.10. Validation, corpus, fuzzing и каталог findings: как проект проверяет свои выводы

Предыдущий раздел о Ghidra integration завершил пользовательский маршрут: анализатор построил внутренние модели, сериализовал report, а потом перенёс часть результата в рабочую среду реверсера. Но на этом инженерная история не заканчивается. Если инструмент показывает finding, score, confidence, dynamic event или case_strength, мы должны понимать не только смысл этих полей, но и то, как проект проверяет, что они вообще имеют право появляться. Поэтому этот раздел нужно читать как фундамент доверия к EH/Unwind Analyzer. Здесь мы не добавляем ещё один красивый формат вывода. Здесь мы разбираем, какими проверками проект ограничивает собственную самоуверенность.

Слово validation обычно переводят как валидация, проверка пригодности или подтверждение соответствия. Важно не путать его с verification. Verification отвечает на вопрос: построили ли мы систему согласно заданной спецификации. Validation отвечает на вопрос: помогает ли система решать реальную задачу в заявленных условиях. В маленьком учебном проекте эти слова часто смешивают. В EH/Unwind Analyzer их нужно развести. Когда JSON проходит report.schema.json, это ближе к verification of format: структура отчёта соответствует контракту. Когда реальный benign corpus из 106 PE-файлов даёт 0 nonzero heuristic scores, это уже часть validation of benign compatibility: инструмент не шумит на выбранном наборе нормальных файлов. Когда dynamic smoke target регистрирует VEH и collector видит registration/hit, это validation selected runtime path: проверка конкретного механизма под Windows Debug API.

Термин corpus буквально означает корпус, то есть собрание входных данных. В анализаторе PE-файлов corpus - это не просто папка с exe. Это набор inputs, для каждого из которых понятна роль: benign compatibility, synthetic anomaly, dynamic smoke, mutation seed, fuzz seed, release evidence. Если смешать всё в одну папку и назвать dataset, результат будет хуже: непонятно, какие файлы должны давать score 0, какие специально повреждены, какие предназначены для dynamic collector, а какие используются только как начальные seeds для mutations. Поэтому corpus в проекте - это часть методологии, а не склад бинарников.

Термин evidence в этой главе тоже сохраняет прежний смысл. Evidence - свидетельство, наблюдаемый факт, который поддерживает или ослабляет гипотезу. Тесты не доказывают “инструмент всегда прав”. Они создают evidence о свойствах инструмента: парсер не выходит за bounds на заданных

malformed samples; decoder правильно обрабатывает известные UWOP cases; heuristic engine создаёт нужный finding на synthetic tamper; report сохраняет schema contract; dynamic collector на Windows видит контролируемый VEH/VCH/UEF/RtlAddFunctionTable scenario. Это разные свидетельства, и каждое имеет границы.

Официальная документация здесь важна по двум причинам. Первая - Windows x64 exception/unwind metadata не является авторской фантазией проекта: PE format, .pdata, .xdata, RUNTIME_FUNCTION, UNWIND_INFO, RtlVirtualUnwind and dynamic function tables описаны Microsoft. Вторая - сама инфраструктура проверки тоже не должна быть туманной: CMake/CTest, libFuzzer, Clang sanitizers, JSON Schema, GitHub Actions имеют свои официальные контракты. Когда проект говорит “CTest passed”, это означает не эмоциональное “я запускал и вроде нормально”, а зарегистрированные test commands, которые вернули success exit codes. Когда проект говорит “libFuzzer target exists”, это означает in-process coverage-guided fuzzing entrypoint, а не случайный script with random bytes.

Build как первая проверка: если программа не собирается воспроизводимо, остальное преждевременно

Build - сборка. В C-проекте это не только компиляция одного файла. Build связывает CMake configuration, compiler flags, include paths, target definitions, platform conditions, static library, executable, tests and install rules. EH/Unwind Analyzer построен как modular monolith: снаружи это один CLI tool, но внутри core logic вынесена в static library ehua_core. Static library - статическая библиотека, то есть набор object code, который линкуется в executable на этапе сборки. Это важно для тестов: unit test может линковаться с тем же core, что и CLI, не проходя через пользовательский main.c.

```
ehua_core:
  app_info.c
  dynamic_win.c
  heuristics.c
  pe.c
  report.c
  unwind.c
  util.c

CLI:
  main.c -> links with ehua_core
```

Такое разделение проверяет архитектурную дисциплину. Если pe.c напрямую зависит от UI details, если report.c начинает читать файл сам, если dynamic_win.c не изолирован под _WIN32, build and tests быстро становятся хрупкими. Поэтому CMakeLists.txt в проекте - это не служебная мелочь, а карта зависимостей. main.c выполняет orchestration, util.c даёт безопасные

primitives, pe.c строит PeImage, unwind.c строит RuntimeFunction and UnwindSummary arrays, heuristics.c создаёт HeuristicReport, dynamic_win.c собирает DynamicReport, report.c сериализует outcome. Build связывает эти роли в один artifact.

CTest - test driver из мира CMake. Test driver означает программу, которая запускает зарегистрированные проверки и фиксирует result. В проекте release evidence говорит о 16/16 PASS для MinGW-w64 Release CTest and MSVC Release CTest. Это не означает, что все возможные paths проверены. Это означает, что финальная release configuration проходит 16 зарегистрированных gates. Слово gate здесь можно перевести как ворота качества: если gate не пройден, release нельзя честно объявлять готовым.

Warnings-as-errors - политика, при которой compiler warning становится build failure. Warning - предупреждение компилятора, сигнал о потенциально сомнительной конструкции. Error - ошибка, прекращающая build. Для C-кода, который читает недоверенные bytes, warnings-as-errors особенно важны: implicit conversions, signed/unsigned mismatches, shadowing, format warnings могут не быть exploit сами по себе, но они часто указывают на будущие parsing bugs. Однако warning-free build не доказывает memory safety. Он только поднимает минимальный уровень дисциплины.

Unit tests: маленькие проверки не заменяют систему, но держат её контракты

Unit test - модульный тест. Unit буквально означает единицу. В проекте unit test проверяет небольшой участок логики: decoder rule, heuristic rule, JSON escaping, HTML escaping, nested overlap detection, mitigation rule. Это не пользовательский сценарий целиком. Его сила в другом: если маленький invariant сломался, unit test указывает близко к месту поломки. Например, если изменение в heuristics.c случайно отменило small_sample_size weakening rule, unit_heuristics_mitigations должен упасть раньше, чем regression corpus даст странный confidence в большом отчёте.

Unit tests проекта покрывают разные типы внутренних contract. unit_unwind проверяет decoder: операции, порядок code offsets, flags, operand length, chain errors and boundaries. unit_heuristics проверяет базовые score, placement, verdict and confidence. unit_heuristics_mitigations проверяет weakening rules. unit_heuristics_context проверяет контекстные защиты от false positives, например benign handler dominance suppression. unit_heuristics_combos проверяет combination findings, где два сигнала вместе создают отдельный rule. unit_core_edges проверяет empty input, resource-only case, bounds, RVA mapping and nested overlaps. unit_report_json, unit_report_html and unit_matrix_report проверяют report contracts.

Слово contract здесь используется намеренно. Contract - договор между частями программы. Например, unwind.c обещает: если запись не распарсилась, us_ok[i] будет false, а UnwindSummary не должен использоваться как полностью достоверный. heuristics.c обещает: parse failure создаёт unwind_parse_fail, но strict ABI warnings сами по себе не поднимают suspicion. report.c обещает: если JSON имеет поле findings, оно соответствует schema and escaping rules. Unit tests закрепляют такие contracts, чтобы будущая правка не превратила осторожный анализатор в набор случайных outputs.

Есть тонкий момент с C assert. assert - макрос проверки условия в C. В release builds он часто отключается через NDEBUG. Проект принудительно undefines NDEBUG для release unit tests, чтобы CTest не “прошёл” пустыми проверками. Это маленькая, но очень важная инженерная деталь: тест, который компилируется, но не проверяет conditions, создаёт ложное чувство безопасности. В инструменте, который должен аккуратно говорить о malformed input, ложная безопасность хуже честного failing test.

Integration tests: проверяется не функция, а путь пользователя

Integration test - интеграционный тест. Integration значит соединение. В отличие от unit test, он проверяет, что несколько модулей работают вместе. Для EH/Unwind Analyzer это особенно важно, потому что ошибка может появиться на границе: pe.c правильно нашёл Exception Directory, unwind.c правильно декодировал summary, heuristics.c правильно создал finding, но report.c неправильно записал field name; или CLI передал не тот option; или exit code не соответствует status в JSON. Unit test каждого модуля может быть зелёным, а пользовательский result - неправильным.

Regression samples - пример integration testing. Regression буквально означает откат или повторное появление старой ошибки. Regression test фиксирует: ошибка однажды была обнаружена или сценарий однажды был определён как важный, теперь он не должен сломаться при будущих изменениях. В проекте tests/samples/manifest.json описывает synthetic PE samples and expected JSON paths. Скрипт run_regression_samples.py запускает CLI, получает JSON, проверяет expected fields, minimum score, verdict and case strength. Это не “ручной запуск для красоты”, а автоматическая защита от silent policy drift.

Synthetic sample - искусственно сконструированный пример. Synthetic не значит “ненастоящий” в плохом смысле. Это значит, что sample создан так, чтобы контролируемо проверить конкретный механизм. benign_minimal.exe и benign_richer.exe ожидают score 0, no_strong_static_anomalies, evidence none, manual review false and case no_case. suspicious_unwind_oob.exe проверяет

broken UnwindInfoRVA and parse failure. suspicious_unwind_writable.exe проверяет writable placement. suspicious_section_raw_oob.exe проверяет file layout and overlap. suspicious_eh_unwind_tamper.exe проверяет более сложную связку writable metadata, handler placement and combined case. Такой corpus хорош для functional regression, но не доказывает внешнюю точность на произвольном мире.

```
Regression samples in the project:
  benign_minimal.exe
  benign_richer.exe
  suspicious_unwind_oob.exe
  suspicious_unwind_writable.exe
  suspicious_section_raw_oob.exe
  suspicious_eh_unwind_tamper.exe
```

CLI contract tests проверяют внешний договор программы с пользователем. CLI - command-line interface, интерфейс командной строки. Contract здесь включает arguments, exit codes, expected outputs and error messages. Если parse stage failed, если report write failed, если user requested dynamic mode on unsupported platform, CLI должен отвечать предсказуемо. Автоматизация не должна угадывать по тексту “вроде всё нормально”. Она должна смотреть exit code and JSON status. Поэтому test_cli_contract.py важен не меньше, чем unit_unwind: он защищает boundary между инструментом и pipeline пользователя.

Real benign corpus: 106/106 - сильное свидетельство, но не магическая гарантия

Real benign corpus - набор реальных, выбранных как нормальные PE-файлов. Real значит, что файлы не созданы специально под тест. Benign значит, что их роль в проверке - показать совместимость с обычным software, а не подтвердить malicious behavior. В финальной evidence-сводке проекта указано: real benign static corpus 106/106 PASS. В подробной static_real summary указано: 106 files, 106 successful analyses with valid reports, 0 failed/timed out/invalid reports, 0 files with non-zero heuristic score, 174795 parsed RUNTIME_FUNCTION records, total input size 140.4 MiB, median analysis time 17 ms, 95th percentile 36 ms, maximum 268 ms. В более общей сводке полного validation также фигурирует maximum analysis time 765 ms для финального набора прогонов. Эти числа нужно читать внимательно: они говорят о конкретном corpus and конкретном run, а не о вселенной PE-файлов.

```
Real benign static corpus:
  files:                          106
  valid reports:                  106
  failed / timed out / invalid:    0
  nonzero heuristic scores:        0
  runtime functions parsed:        174795
  total input size:                140.4 MiB
  median analysis time:            17 ms
```

```
    95th percentile analysis time:  36 ms
    maximum analysis time:          268 ms

  Sources:
    7zip:       1
    jq:         1
    python:    32
    rust:       1
    sysinternals: 71
```

Что это доказывает? Во-первых, parser and decoder способны пройти большой набор реальных unwind tables без parse failures and structural unwind anomalies. Во-вторых, heuristic policy не создаёт noise на этом конкретном benign corpus, если positive signal defined as nonzero score. В-третьих, report generation remains valid and fast enough on those inputs. Что это не доказывает? Это не доказывает recall на реальных вмешательствах. Это не доказывает поддержку всех compiler/linker families. Это не доказывает отсутствие false positives в мировой совокупности. Это не доказывает benignness каждого файла философски. Это доказывает совместимость с проверенным набором, при конкретной версии tool and schema.

Почему нельзя сказать “specificity 100% навсегда”? Specificity - true negative rate, доля корректных negative examples, которые инструмент не пометил positive. В этом corpus observed specificity по правилу “nonzero score means positive” действительно 106/106. Но corpus может быть смещён. Bias - смещение выборки. Если 71 файл из Sysinternals и 32 из Python distribution, файлы не обязательно независимы как разные compiler ecosystems. Independence assumption - предположение независимости. Простая статистика считает, что каждый пример независим, но DLL одного runtime могут быть похожи. Поэтому честный вывод должен говорить “observed on this corpus”, а не “proved globally”.

Reproducibility - воспроизводимость. Для serious validation мало сказать “я запускал на 106 файлах”. Нужны SHA-256 inputs, sources, exact command, tool version, schema version, environment and aggregate results. Если сами PE-файлы нельзя публиковать из-за licensing or size, нужно хранить acquisition procedure and hashes. В проекте real_samples/SOURCES.md and final reports выполняют эту роль. Это превращает validation из устного обещания в проверяемую evidence base.

Ground truth и метрики: почему нельзя одним числом ответить “точность 99%”

Ground truth - эталонная разметка. Буквально “истина на земле”, то есть заранее установленный правильный ответ. Без ground truth нельзя честно считать true positives and false negatives. Для EH/Unwind Analyzer ground truth зависит от уровня. На file-level один PE-файл является одним примером. На record-level один RUNTIME_FUNCTION or UNWIND_INFO является примером. На finding-level один потенциальный finding code для одного файла является примером. На event-level одно dynamic event является примером. Если смешать эти levels, получится красивое, но бессмысленное число.

True Positive, TP: нарушение есть, и инструмент его обнаружил. False Positive, FP: нарушения нет, но инструмент создал сигнал. True Negative, TN: нарушения нет, и инструмент промолчал. False Negative, FN: нарушение есть, но инструмент его пропустил. Recall отвечает, какую долю реальных positives обнаружили. Precision отвечает, какая доля сигналов подтверждается ground truth. Specificity отвечает, какую долю negatives инструмент не пометил. Accuracy отвечает, какая доля всех решений правильна, но при дисбалансе classes может обманывать. Если corpus содержит 990 benign and 10 anomalous files, инструмент, всегда отвечающий “нет аномалии”, получит 99% accuracy and 0% recall.

```
TP: anomaly exists, analyzer reports it
FP: anomaly does not exist, analyzer reports it
TN: anomaly does not exist, analyzer stays quiet
FN: anomaly exists, analyzer misses it

recall      = TP / (TP + FN)
precision   = TP / (TP + FP)
specificity = TN / (TN + FP)
accuracy    = (TP + TN) / (TP + TN + FP + FN)
```

Для проекта особенно важно measuring per finding class. Нельзя сказать “эвристики работают” и остановиться. Нужно отдельно проверять pe_section_raw_oob, rf_overlaps, unwind_rva_oob, unwind_parse_fail, handler_dominance, writable_metadata_handler_combo and so on. Почему? Потому что один класс может быть почти детерминированным structural check, а другой - мягким heuristic pattern with benign suppressions. Их precision and recall нельзя смешивать без потери смысла.

Threshold - порог. В проекте thresholds есть на нескольких уровнях: >1% для ряда integrity prevalence, >5% для unwind anomalies, >10% and >30% для unwind reuse, >80% для handler dominance, score thresholds 15/40/70 для verdict. Threshold меняет trade-off: ниже threshold - выше чувствительность, но больше false positives; выше threshold - выше specificity, но больше false negatives. Поэтому threshold calibration должна идти на размеченном corpus, а не только на synthetic samples, которые сами созданы под известные rules.

Calibration - калибровка численной оценки к эмпирической вероятности. В проекте score не calibrated probability. suspicion = score / 100 - нормализация инженерного score, а не вероятность вредоносности. confidence - rule-based estimate of evidence completeness and corroboration, not empirical probability. Поэтому тесты должны не только проверять числа, но и защищать их интерпретацию. Отчёт не должен звучать как malware verdict, если проектная модель является explainable static evidence engine.

Mutation fuzzing, libFuzzer и sanitizers: проверка недоверенного входа

Fuzzing - фаззинг, метод тестирования, где инструмент получает множество изменённых или сгенерированных inputs. В контексте PE parser это особенно естественно: входной файл недоверенный, и attacker or malformed corpus может содержать странные offsets, sizes, overlaps, truncated records, impossible RVAs. Mutation fuzzing - мутационный фаззинг. Mutation означает изменение уже существующих seeds. Seed - начальное зерно, исходный вход, от которого строятся variants. Скрипт tests/run_mutation_fuzz.py создаёт deterministic mutated inputs and runs full CLI. Deterministic значит воспроизводимый: одинаковая seed policy даёт повторяемый набор mutations.

В проектной документации зафиксирован результат: 1000 deterministic mutations, 634 accepted valid reports, 366 controlled rejected inputs, 0 crashes, 0 hangs, 0 invalid reports. Accepted valid report не значит, что mutated PE был нормальным; это значит, что анализатор смог его контролируемо разобрать и записать valid report. Controlled rejection значит, что input отвергнут ожидаемым способом, а не через crash or undefined behavior. Это robustness validation, а не detection accuracy. Для большинства случайных mutations нет full ground truth, какой finding “должен” быть. Зато известно, что парсер не должен падать, зависать or corrupt report contract.

```
Deterministic mutation fuzzing result:
  mutations:                    1000
  accepted valid reports:        634
  controlled rejected inputs:     366
  crashes:                          0
  hangs:                            0
  invalid reports:                  0
```

LibFuzzer - in-process coverage-guided evolutionary fuzzing engine from LLVM. In-process значит, что fuzzer вызывает target function внутри одного процесса, а не каждый раз запускает отдельный CLI. Coverage-guided значит, что он смотрит, какие участки кода достигнуты, и выбирает mutations, которые открывают новое coverage. Evolutionary значит, что corpus постепенно развивается: интересные inputs сохраняются and mutate further. Harness - маленькая функция-обёртка, которая принимает bytes from fuzzer and calls parser code. В проекте tests/fuzz/fuzz_parser.c выполняет такую роль для

parser-facing logic.

Sanitizer - инструмент runtime instrumentation. AddressSanitizer, ASan, ловит многие memory errors: heap/stack/global out-of-bounds, use-after-free and related bugs. UndefinedBehaviorSanitizer, UBSan, ловит ряд undefined behavior: integer issues, invalid shifts, misaligned access and other C-level undefined operations depending on flags. Instrumentation значит, что compiler вставляет дополнительные проверки в binary. Sanitizers не доказывают отсутствие всех memory bugs, но резко повышают шанс поймать классы ошибок, которые обычный run молча пропустил бы.

Ограничение sanitizers в проекте нужно сказать честно. ASan/UBSan на Linux/macOS покрывают static code paths and non-Windows dynamic stub. Windows-specific dynamic_win.c branch under _WIN32 при таком run не исполняется. Cross-compilation MinGW подтверждает compile compatibility, but not runtime correctness of Debug API. Поэтому dynamic collector требует отдельной Windows x64 validation: focused smoke targets, Debug API events, schema validation, semantic validation, and ideally future runs on different Windows versions and patch levels.

JSON Schema и semantic validation: структура отчёта не равна смысловой правильности

JSON Schema - язык описания структуры JSON. Schema validation отвечает на вопрос: соответствует ли JSON declared contract. В проекте report.schema.json фиксирует expected objects, field types, arrays and additionalProperties=false. additionalProperties=false означает: объект не должен содержать поля, которые не описаны schema. Это защищает потребителя отчёта от тихого расползания формата. Если report.c случайно начнёт писать новое поле без обновления schema, validation должна это поймать.

Но schema validation не отвечает на вопрос, правильно ли вычислен score or RVA. JSON может быть идеально валиден и при этом семантически неверен. Поэтому проекту нужна semantic validation. Semantic значит смысловой. Semantic validator проверяет relationships между полями: if dynamic session timed out, runtime evidence level must be capped; if findings list is empty and score is zero, evidence_level should be none; if manual_validation_steps are emitted, they must match the case; if report contains dynamic block, enabled/ran/ok/timed_out/target_exited must be internally coherent. Это другой уровень проверки, чем просто braces and strings.

Report semantics tests важны из-за bridge role report.c. Отчёт читает человек, JSON парсит automation, HTML показывает dashboard, Ghidra CSV переносит navigation hints. Если report говорит разные вещи в разных surfaces, реверсер получит не evidence, а шум. Поэтому tests/test_report_semantics.py

and tests/report_semantics.py проверяют не только syntax, но и meaning. Это продолжает идею всей книги: static facts, dynamic evidence and final assessment должны быть связаны осторожно.

Dynamic validation: focused smoke не заменяет coverage всего процесса

Dynamic smoke validation - короткий контролируемый запуск dynamic collector на purpose-built targets. Smoke test буквально “дымовой тест”: минимальная проверка, что система вообще проходит через основной путь и не загорается. В проекте focused smoke scenarios покрывают VEH, VCH, UEF and runtime function-table registration. VEH - vectored exception handler. VCH - vectored continue handler. UEF - unhandled exception filter. RtlAddFunctionTable - Windows API для регистрации dynamic function table for generated code. Эти targets нужны, потому что static tests не могут доказать работу Debug API event loop.

Финальная evidence-сводка фиксирует dynamic gates: dynamic smoke 4/4 PASS, full dynamic corpus 124/124 valid, dynamic evidence validator PASS, unexpected dynamic failures 0. Full dynamic outcome разделён на completed, controlled_timeout, elevation_required and unsupported_library_launch. Это очень важная терминология. Controlled timeout - не случайная неудача, а ожидаемое состояние, где target не завершился в budget, но report remains valid. Elevation required - запуск требует прав, которых нет в test environment. Unsupported library launch - input is not launchable executable scenario. Все эти outcomes должны быть представлены как contract outcomes, not crashes.

```
Final dynamic evidence summary:
  dynamic smoke:                  4 / 4 PASS
  full dynamic corpus:            124 / 124 valid
  dynamic evidence validator:     PASS
  unexpected dynamic failures:    0

Dynamic full outcome:
  completed:                     54
  controlled_timeout:            32
  elevation_required:             8
  unsupported_library_launch:    30
```

Но dynamic validation не означает full behavioral coverage. Один запуск target не проходит все branches. Debugger presence может менять behavior. Breakpoint placement может не сработать, если code path не достигнут. Handler registration event не гарантирует later hit. RtlAddFunctionTable call не означает, что project прочитал remote entries. Timeout session stores events but caps evidence. Поэтому dynamic results должны читаться как runtime evidence from this scenario, not universal proof of absence.

Перепроверка finding catalog: в проекте 27 structured finding codes

Теперь нужно отдельно разобрать то, что в предыдущих версиях было слишком сжато. В проекте действительно 27 structured finding codes. Я перепроверил это по двум источникам: implementation calls add_finding in heuristics.c and evidence_matrix.md. no_strong_static_anomalies не входит в эти 27: он выводится как informational note/verdict, not structured finding. Weakening rules вроде single_signal_only тоже не входят: они выводятся как mitigations / weakening_rules_applied, not findings. Поэтому правильный каталог findings состоит ровно из 27 codes.

```
Structured finding codes confirmed in heuristics.c: 27

Families:
  table_integrity
  unwind_integrity
  metadata_placement
  unwind_reuse
  handler_dominance
```

Finding code - машинное имя признака. Code нужен не для красоты, а для стабильного анализа: JSON consumer, tests, Ghidra comments, docs and regression manifests can refer to the same condition without parsing human text. Family - семейство evidence, то есть тип свидетельства. Severity - уровень серьёзности внутри rule: medium, high and so on. Score weight - численный вклад в bounded score. Affected count - сколько объектов затронуто. Representative RVA - первый удобный адрес для навигации, not proof that only one object is affected. Эти поля вместе дают объяснимость: человек видит не только итоговый score, но и путь, по которому проект пришёл к cautious assessment.

Каждый finding ниже нужно читать не как “малварный признак”, а как named engineering observation. Он отвечает на вопрос: какой контракт PE/unwind model нарушен или выглядит необычно. Некоторые findings почти структурные: например, exception directory out of bounds. Некоторые эвристические: например, handler dominance. Некоторые являются combination findings: они намеренно усиливают case, когда два разных необычных свойства встречаются вместе. У каждого finding есть возможные benign explanations and manual validation steps.

Как читать пять evidence families перед подробным каталогом

table_integrity - целостность таблиц and PE layout. Здесь проект проверяет, может ли он вообще доверять section map and RUNTIME_FUNCTION ranges. Если table_integrity сломан, многие последующие выводы становятся менее надёжными, потому что addresses and ranges may be ambiguous.

unwind_integrity - целостность самой UNWIND_INFO decoding. Здесь речь не о том, где metadata лежит, а о том, можно ли её прочитать как supported unwind structure. Parse failure and anomalies live here.

metadata_placement - контекст размещения metadata and handlers. Здесь проект спрашивает: находится ли unwind data там, где data обычно должна быть, and does handler target look like executable code? Section names are weak evidence, permissions are stronger evidence.

unwind_reuse - концентрация одинакового UnwindInfoRVA. Это не direct corruption, a distribution pattern. Pattern становится finding only above thresholds and sometimes only with corroboration.

handler_dominance - концентрация одинакового handler target. Проект специально suppresses benign-looking runtime helper dominance, because many normal binaries can centralize exception handling through runtime helpers.

1. `pe_section_raw_oob`

Код pe_section_raw_oob относится к семейству table_integrity. В текущей evidence matrix severity указана как high, a score weight равен 25. Это structured finding: он появляется в массиве findings JSON/HTML report, имеет machine-readable code, family, severity, affected_count and sometimes representative_rva. Его нельзя читать как самостоятельный malware verdict. Это named observation, то есть именованное свидетельство о конкретном нарушении или подозрительном pattern.

Механизм срабатывания: section raw range выходит за пределы файла: PointerToRawData + SizeOfRawData указывает за конец input buffer. Здесь важно различать source fact and interpretation. Source fact - это то, что реально посчитал проект из PeImage, RuntimeFunction array, UnwindSummary array or section characteristics. Interpretation - это почему этот факт важен для reverse engineering. Если source fact неверен из-за более ранней parsing ошибки, interpretation тоже становится слабее. Поэтому findings всегда читаются вместе с pe_validation, section diagnostics and function-level details.

Почему это важно: PE section table связывает virtual image layout с bytes on disk. Если raw section range выходит за bounds, анализатор не может безопасно читать bytes этой section как существующие file data. Это нарушает

базовый file-layout contract ещё до .pdata/.xdata.

Manual validation: Проверить section table в PE viewer, сравнить PointerToRawData, SizeOfRawData and actual file size, убедиться, не является ли файл intentionally truncated, overlay-packed or повреждённым при копировании. Если finding появляется, последующие RVA->file offset выводы надо читать с осторожностью.

В отчёте affected_count показывает масштаб, a representative_rva даёт точку входа для навигации. Representative RVA не означает, что проблема только одна. Если finding high severity, он увеличивает strong_count and can lift evidence_level even when total score is not extremely high. Если finding medium, он чаще требует corroboration: соседние findings, dynamic evidence, manual section inspection or Ghidra verification. В любом случае score contribution is bounded and saturated at 100, so analyst must read findings themselves, not only final number.

2. `pe_section_virtual_oob`

Код pe_section_virtual_oob относится к семейству table_integrity. В текущей evidence matrix severity указана как high, a score weight равен 20. Это structured finding: он появляется в массиве findings JSON/HTML report, имеет machine-readable code, family, severity, affected_count and sometimes representative_rva. Его нельзя читать как самостоятельный malware verdict. Это named observation, то есть именованное свидетельство о конкретном нарушении или подозрительном pattern.

Механизм срабатывания: section virtual range выходит за SizeOfImage или создаёт невозможную image layout область. Здесь важно различать source fact and interpretation. Source fact - это то, что реально посчитал проект из PeImage, RuntimeFunction array, UnwindSummary array or section characteristics. Interpretation - это почему этот факт важен для reverse engineering. Если source fact неверен из-за более ранней parsing ошибки, interpretation тоже становится слабее. Поэтому findings всегда читаются вместе с pe_validation, section diagnostics and function-level details.

Почему это важно: Windows loader строит image view по VirtualAddress, VirtualSize and SizeOfImage. Если section виртуально указывает за declared image boundary, карта RVA становится недостоверной. Для unwind анализа это критично: RUNTIME_FUNCTION ranges and UnwindInfoRVA are image-relative.

Manual validation: Сравнить SizeOfImage, SectionAlignment, VirtualAddress and VirtualSize. Проверить, не ошибся ли parser на alignment. Если virtual mapping ambiguous, не делать сильных выводов о handler placement без дополнительной проверки в memory image.

В отчёте affected_count показывает масштаб, а representative_rva даёт точку входа для навигации. Representative RVA не означает, что проблема только одна. Если finding high severity, он увеличивает strong_count and can lift evidence_level even when total score is not extremely high. Если finding medium, он чаще требует corroboration: соседние findings, dynamic evidence, manual section inspection or Ghidra verification. В любом случае score contribution is bounded and saturated at 100, so analyst must read findings themselves, not only final number.

3. `pe_section_raw_overlap`

Код pe_section_raw_overlap относится к семейству table_integrity. В текущей evidence matrix severity указана как medium, а score weight равен 15. Это structured finding: он появляется в массиве findings JSON/HTML report, имеет machine-readable code, family, severity, affected_count and sometimes representative_rva. Его нельзя читать как самостоятельный malware verdict. Это named observation, то есть именованное свидетельство о конкретном нарушении или подозрительном pattern.

Механизм срабатывания: raw ranges двух или более sections пересекаются в file offset space. Здесь важно различать source fact and interpretation. Source fact - это то, что реально посчитал проект из PeImage, RuntimeFunction array, UnwindSummary array or section characteristics. Interpretation - это почему этот факт важен для reverse engineering. Если source fact неверен из-за более ранней parsing ошибки, interpretation тоже становится слабее. Поэтому findings всегда читаются вместе с pe_validation, section diagnostics and function-level details.

Почему это важно: Raw overlap означает, что одни и те же file bytes принадлежат нескольким sections. Иногда это результат packing, linker tricks or malformed file. Для анализатора это создаёт неоднозначность: один offset может иметь разные section names and permissions depending on interpretation.

Manual validation: Посмотреть, какие sections пересекаются, являются ли они discardable, имеют ли нулевые sizes, не является ли overlap overlay-like trick. Если overlap затрагивает .pdata/.xdata, усилить ручную проверку unwind entries.

В отчёте affected_count показывает масштаб, а representative_rva даёт точку входа для навигации. Representative RVA не означает, что проблема только одна. Если finding high severity, он увеличивает strong_count and can lift evidence_level even when total score is not extremely high. Если finding medium, он чаще требует corroboration: соседние findings, dynamic evidence, manual section inspection or Ghidra verification. В любом случае score contribution is bounded and saturated at 100, so analyst must read findings themselves, not only final number.

4. `pe_section_virtual_overlap`

Код pe_section_virtual_overlap относится к семейству table_integrity. В текущей evidence matrix severity указана как high, а score weight равен 20. Это structured finding: он появляется в массиве findings JSON/HTML report, имеет machine-readable code, family, severity, affected_count and sometimes representative_rva. Его нельзя читать как самостоятельный malware verdict. Это named observation, то есть именованное свидетельство о конкретном нарушении или подозрительном pattern.

Механизм срабатывания: virtual ranges sections пересекаются в RVA space. Здесь важно различать source fact and interpretation. Source fact - это то, что реально посчитал проект из PeImage, RuntimeFunction array, UnwindSummary array or section characteristics. Interpretation - это почему этот факт важен для reverse engineering. Если source fact неверен из-за более ранней parsing ошибки, interpretation тоже становится слабее. Поэтому findings всегда читаются вместе с pe_validation, section diagnostics and function-level details.

Почему это важно: Virtual overlap опаснее raw overlap, потому что RVA mapping становится неоднозначным в самом image model. Если один RVA можно отнести к двум sections, section characteristics перестают быть надёжным evidence для metadata placement.

Manual validation: Сортировать sections по VirtualAddress, проверить aligned virtual ranges, посмотреть loader behavior в debugger. Если handler or unwind metadata попадает в overlap, static placement finding needs dynamic memory map corroboration.

В отчёте affected_count показывает масштаб, а representative_rva даёт точку входа для навигации. Representative RVA не означает, что проблема только одна. Если finding high severity, он увеличивает strong_count and can lift evidence_level even when total score is not extremely high. Если finding medium, он чаще требует corroboration: соседние findings, dynamic evidence, manual section inspection or Ghidra verification. В любом случае score contribution is bounded and saturated at 100, so analyst must read findings themselves, not only final number.

5. `pe_section_misaligned`

Код pe_section_misaligned относится к семейству table_integrity. В текущей evidence matrix severity указана как medium, а score weight равен 8. Это structured finding: он появляется в массиве findings JSON/HTML report, имеет machine-readable code, family, severity, affected_count and sometimes representative_rva. Его нельзя читать как самостоятельный malware verdict. Это named observation, то есть именованное свидетельство о конкретном нарушении или подозрительном pattern.

Механизм срабатывания: section fields нарушают expected file/section alignment policy проекта. Здесь важно различать source fact and interpretation. Source fact - это то, что реально посчитал проект из PeImage, RuntimeFunction array, UnwindSummary array or section characteristics. Interpretation - это почему этот факт важен для reverse engineering. Если source fact неверен из-за более ранней parsing ошибки, interpretation тоже становится слабее. Поэтому findings всегда читаются вместе с pe_validation, section diagnostics and function-level details.

Почему это важно: Alignment - выравнивание. PE имеет FileAlignment and SectionAlignment. Misalignment не всегда делает файл неисполняемым, но может указывать на custom builder, packer, malformed generator or parser edge case. Это слабее, чем out-of-bounds, поэтому weight ниже.

Manual validation: Проверить FileAlignment/SectionAlignment and section offsets. Не читать misalignment как самостоятельное доказательство. Сопоставить с другими findings: OOB, overlaps, unusual placement, parse failures.

В отчёте affected_count показывает масштаб, а representative_rva даёт точку входа для навигации. Representative RVA не означает, что проблема только одна. Если finding high severity, он увеличивает strong_count and can lift evidence_level even when total score is not extremely high. Если finding medium, он чаще требует corroboration: соседние findings, dynamic evidence, manual section inspection or Ghidra verification. В любом случае score contribution is bounded and saturated at 100, so analyst must read findings themselves, not only final number.

6. `pe_exception_directory_oob`

Код pe_exception_directory_oob относится к семейству table_integrity. В текущей evidence matrix severity указана как high, а score weight равен 25. Это structured finding: он появляется в массиве findings JSON/HTML report, имеет machine-readable code, family, severity, affected_count and sometimes representative_rva. Его нельзя читать как самостоятельный malware verdict. Это named observation, то есть именованное свидетельство о конкретном нарушении или подозрительном pattern.

Механизм срабатывания: Exception Directory RVA/size не может быть безопасно отображён в file bytes. Здесь важно различать source fact and interpretation. Source fact - это то, что реально посчитал проект из PeImage, RuntimeFunction array, UnwindSummary array or section characteristics. Interpretation - это почему этот факт важен для reverse engineering. Если source fact неверен из-за более ранней parsing ошибки, interpretation тоже становится слабее. Поэтому findings всегда читаются вместе с pe_validation, section diagnostics and function-level details.

Почему это важно: Exception Directory - официальный entry point к .pdata для PE32+ AMD64. Если он выходит за bounds, проект не может честно построить RUNTIME_FUNCTION table. Это сильный structural signal, потому что вся unwind analysis зависит от этого directory.

Manual validation: Проверить Optional Header DataDirectory[IMAGE_DIRECTORY_ENTRY_EXCEPTION], size кратность 12, section ownership and file bounds. Если directory broken, любые отсутствующие RUNTIME_FUNCTION entries cannot be read as normal absence.

В отчёте affected_count показывает масштаб, а representative_rva даёт точку входа для навигации. Representative RVA не означает, что проблема только одна. Если finding high severity, он увеличивает strong_count and can lift evidence_level even when total score is not extremely high. Если finding medium, он чаще требует corroboration: соседние findings, dynamic evidence, manual section inspection or Ghidra verification. В любом случае score contribution is bounded and saturated at 100, so analyst must read findings themselves, not only final number.

7. `no_runtime_functions`

Код no_runtime_functions относится к семейству table_integrity. В текущей evidence matrix severity указана как medium, а score weight равен 10. Это structured finding: он появляется в массиве findings JSON/HTML report, имеет machine-readable code, family, severity, affected_count and sometimes representative_rva. Его нельзя читать как самостоятельный malware verdict. Это named observation, то есть именованное свидетельство о конкретном нарушении или подозрительном pattern.

Механизм срабатывания: Exception table пустая или RUNTIME_FUNCTION entries не найдены при наличии executable sections. Здесь важно различать source fact and interpretation. Source fact - это то, что реально посчитал проект из PeImage, RuntimeFunction array, UnwindSummary array or section characteristics. Interpretation - это почему этот факт важен для reverse engineering. Если source fact неверен из-за более ранней parsing ошибки, interpretation тоже становится слабее. Поэтому findings всегда читаются вместе с pe_validation, section diagnostics and function-level details.

Почему это важно: На Windows x64 большинство non-leaf functions require unwind metadata. Но leaf-only image, resource-only image, tiny stubs or special binaries may legitimately have no runtime functions. Поэтому finding medium and explicitly cautious.

Manual validation: Проверить, есть ли executable code, call/sub rsp/prolog patterns, imports, entry point complexity. Если image действительно leaf-only, это benign. Если code сложный, отсутствие .pdata может мешать exception dispatch and stack walking.

В отчёте affected_count показывает масштаб, а representative_rva даёт точку входа для навигации. Representative RVA не означает, что проблема только одна. Если finding high severity, он увеличивает strong_count and can lift evidence_level even when total score is not extremely high. Если finding medium, он чаще требует corroboration: соседние findings, dynamic evidence, manual section inspection or Ghidra verification. В любом случае score contribution is bounded and saturated at 100, so analyst must read findings themselves, not only final number.

8. `invalid_ranges`

Код invalid_ranges относится к семейству table_integrity. В текущей evidence matrix severity указана как medium/high, а score weight равен 10 или 20. Это structured finding: он появляется в массиве findings JSON/HTML report, имеет machine-readable code, family, severity, affected_count and sometimes representative_rva. Его нельзя читать как самостоятельный malware verdict. Это named observation, то есть именованное свидетельство о конкретном нарушении или подозрительном pattern.

Механизм срабатывания: RUNTIME_FUNCTION имеет EndAddress <= BeginAddress. Здесь важно различать source fact and interpretation. Source fact - это то, что реально посчитал проект из PeImage, RuntimeFunction array, UnwindSummary array or section characteristics. Interpretation - это почему этот факт важен для reverse engineering. Если source fact неверен из-за более ранней parsing ошибки, interpretation тоже становится слабее. Поэтому findings всегда читаются вместе с pe_validation, section diagnostics and function-level details.

Почему это важно: RUNTIME_FUNCTION должен описывать полуинтервал [BeginAddress, EndAddress). Пустой или обратный range не описывает нормальную function coverage. Это ломает binary search / range lookup assumptions and makes unwind relation meaningless for that entry.

Manual validation: Открыть .pdata, проверить конкретные Begin/End RVAs, посмотреть соседние entries and section boundaries. Если таких ranges много, это сильнее, чем единичный malformed record.

В отчёте affected_count показывает масштаб, а representative_rva даёт точку входа для навигации. Representative RVA не означает, что проблема только одна. Если finding high severity, он увеличивает strong_count and can lift evidence_level even when total score is not extremely high. Если finding medium, он чаще требует corroboration: соседние findings, dynamic evidence, manual section inspection or Ghidra verification. В любом случае score contribution is bounded and saturated at 100, so analyst must read findings themselves, not only final number.

9. `rf_oob`

Код rf_oob относится к семейству table_integrity. В текущей evidence matrix severity указана как medium/high, а score weight равен 10 или 25. Это structured finding: он появляется в массиве findings JSON/HTML report, имеет machine-readable code, family, severity, affected_count and sometimes representative_rva. Его нельзя читать как самостоятельный malware verdict. Это named observation, то есть именованное свидетельство о конкретном нарушении или подозрительном pattern.

Механизм срабатывания: BeginAddress или EndAddress runtime function range выходит за SizeOfImage. Здесь важно различать source fact and interpretation. Source fact - это то, что реально посчитал проект из PeImage, RuntimeFunction array, UnwindSummary array or section characteristics. Interpretation - это почему этот факт важен для reverse engineering. Если source fact неверен из-за более ранней parsing ошибки, interpretation тоже становится слабее. Поэтому findings всегда читаются вместе с pe_validation, section diagnostics and function-level details.

Почему это важно: RUNTIME_FUNCTION range должен ссылаться на code within image. Если range указывает за declared image, Windows runtime lookup cannot interpret it as normal function interval. Это может быть corruption, obfuscation, custom loader artifact or parser mismatch.

Manual validation: Проверить SizeOfImage, module base, section coverage and raw data. Сравнить с Ghidra function ranges. Если range OOB сочетается с parse failures or handler anomalies, case becomes stronger.

В отчёте affected_count показывает масштаб, а representative_rva даёт точку входа для навигации. Representative RVA не означает, что проблема только одна. Если finding high severity, он увеличивает strong_count and can lift evidence_level even when total score is not extremely high. Если finding medium, он чаще требует corroboration: соседние findings, dynamic evidence, manual section inspection or Ghidra verification. В любом случае score contribution is bounded and saturated at 100, so analyst must read findings themselves, not only final number.

10. `unwind_rva_oob`

Код unwind_rva_oob относится к семейству table_integrity. В текущей evidence matrix severity указана как medium/high, а score weight равен 10 или 25. Это structured finding: он появляется в массиве findings JSON/HTML report, имеет machine-readable code, family, severity, affected_count and sometimes representative_rva. Его нельзя читать как самостоятельный malware verdict. Это named observation, то есть именованное свидетельство о конкретном нарушении или подозрительном pattern.

Механизм срабатывания: UnwindInfoAddress равен 0 или >= SizeOfImage. Здесь важно различать source fact and interpretation. Source fact - это то, что реально посчитал проект из PeImage, RuntimeFunction array, UnwindSummary array or section characteristics. Interpretation - это почему этот факт важен для reverse engineering. Если source fact неверен из-за более ранней parsing ошибки, interpretation тоже становится слабее. Поэтому findings всегда читаются вместе с pe_validation, section diagnostics and function-level details.

Почему это важно: UnwindInfoAddress должен вести к UNWIND_INFO. Zero or out-of-image pointer means decoder cannot reach metadata. В проекте это separate from parse failure: pointer itself is invalid before decoder reads header.

Manual validation: Проверить RUNTIME_FUNCTION.UnwindData, section mapping and whether pointer could be RVA under unusual image base assumptions. В normal PE32+ AMD64 this is a strong integrity problem.

В отчёте affected_count показывает масштаб, а representative_rva даёт точку входа для навигации. Representative RVA не означает, что проблема только одна. Если finding high severity, он увеличивает strong_count and can lift evidence_level even when total score is not extremely high. Если finding medium, он чаще требует corroboration: соседние findings, dynamic evidence, manual section inspection or Ghidra verification. В любом случае score contribution is bounded and saturated at 100, so analyst must read findings themselves, not only final number.

11. `rf_unsorted`

Код rf_unsorted относится к семейству table_integrity. В текущей evidence matrix severity указана как medium, а score weight равен 15. Это structured finding: он появляется в массиве findings JSON/HTML report, имеет machine-readable code, family, severity, affected_count and sometimes representative_rva. Его нельзя читать как самостоятельный malware verdict. Это named observation, то есть именованное свидетельство о конкретном нарушении или подозрительном pattern.

Механизм срабатывания: локальное нарушение nondecreasing order BeginRVA between adjacent RUNTIME_FUNCTION records. Здесь важно различать source fact and interpretation. Source fact - это то, что реально посчитал проект из PeImage, RuntimeFunction array, UnwindSummary array or section characteristics. Interpretation - это почему этот факт важен для reverse engineering. Если source fact неверен из-за более ранней parsing ошибки, interpretation тоже становится слабее. Поэтому findings всегда читаются вместе с pe_validation, section diagnostics and function-level details.

Почему это важно: Microsoft x64 unwind model expects function table entries sorted. Проект считает локальные inversions, not full mathematical inversion count. Unsorted table может быть recoverable for static analysis, but it violates

expected runtime lookup arrangement.

Manual validation: Сравнить original .pdata order and sorted order. Проверить, не создана ли таблица custom tool. Если entries only unsorted but otherwise valid, finding weaker than overlaps/OOB.

В отчёте affected_count показывает масштаб, a representative_rva даёт точку входа для навигации. Representative RVA не означает, что проблема только одна. Если finding high severity, он увеличивает strong_count and can lift evidence_level even when total score is not extremely high. Если finding medium, он чаще требует corroboration: соседние findings, dynamic evidence, manual section inspection or Ghidra verification. В любом случае score contribution is bounded and saturated at 100, so analyst must read findings themselves, not only final number.

12. `rf_overlaps`

Код rf_overlaps относится к семейству table_integrity. В текущей evidence matrix severity указана как high, a score weight равен 25. Это structured finding: он появляется в массиве findings JSON/HTML report, имеет machine-readable code, family, severity, affected_count and sometimes representative_rva. Его нельзя читать как самостоятельный malware verdict. Это named observation, то есть именованное свидетельство о конкретном нарушении или подозрительном pattern.

Механизм срабатывания: RUNTIME_FUNCTION ranges overlap after sorting by BeginAddress. Здесь важно различать source fact and interpretation. Source fact - это то, что реально посчитал проект из PeImage, RuntimeFunction array, UnwindSummary array or section characteristics. Interpretation - это почему этот факт важен для reverse engineering. Если source fact неверен из-за более ранней parsing ошибки, interpretation тоже становится слабее. Поэтому findings всегда читаются вместе с pe_validation, section diagnostics and function-level details.

Почему это важно: Overlap означает, что один RIP может попадать в несколько function ranges. Это разрушает однозначность lookup: какой UNWIND_INFO применять? Проект использует max_end to detect nested overlaps, not only adjacent previous end.

Manual validation: Проверить overlapping ranges, disassembly, real function boundaries. Nested overlap часто сильнее простой сортировочной ошибки. Если overlap находится around exception-handling code, manual debugger validation особенно важна.

В отчёте affected_count показывает масштаб, a representative_rva даёт точку входа для навигации. Representative RVA не означает, что проблема только одна. Если finding high severity, он увеличивает strong_count and can lift

evidence_level even when total score is not extremely high. Если finding medium, он чаще требует corroboration: соседние findings, dynamic evidence, manual section inspection or Ghidra verification. В любом случае score contribution is bounded and saturated at 100, so analyst must read findings themselves, not only final number.

13. `duplicate_begin`

Код duplicate_begin относится к семейству table_integrity. В текущей evidence matrix severity указана как medium, а score weight равен 10. Это structured finding: он появляется в массиве findings JSON/HTML report, имеет machine-readable code, family, severity, affected_count and sometimes representative_rva. Его нельзя читать как самостоятельный malware verdict. Это named observation, то есть именованное свидетельство о конкретном нарушении или подозрительном pattern.

Механизм срабатывания: две или более RUNTIME_FUNCTION entries имеют одинаковый BeginAddress. Здесь важно различать source fact and interpretation. Source fact - это то, что реально посчитал проект из PeImage, RuntimeFunction array, UnwindSummary array or section characteristics. Interpretation - это почему этот факт важен для reverse engineering. Если source fact неверен из-за более ранней parsing ошибки, interpretation тоже становится слабее. Поэтому findings всегда читаются вместе с pe_validation, section diagnostics and function-level details.

Почему это важно: Duplicate begin creates ambiguous function-table model. Иногда это связано с chained info or odd compiler artifacts, но для static analyzer это сигнал, что range table may not represent normal one-entry-per-primary-function mapping.

Manual validation: Посмотреть EndAddress/UnwindInfoAddress for duplicates. Если duplicate entries имеют разные ends or different unwind info, выяснить, не является ли это malformed table or unsupported edge. Не объявлять malicious без context.

В отчёте affected_count показывает масштаб, а representative_rva даёт точку входа для навигации. Representative RVA не означает, что проблема только одна. Если finding high severity, он увеличивает strong_count and can lift evidence_level even when total score is not extremely high. Если finding medium, он чаще требует corroboration: соседние findings, dynamic evidence, manual section inspection or Ghidra verification. В любом случае score contribution is bounded and saturated at 100, so analyst must read findings themselves, not only final number.

14. `unwind_parse_fail`

Код unwind_parse_fail относится к семейству unwind_integrity. В текущей evidence matrix severity указана как medium/high, а score weight равен 10 или 25. Это structured finding: он появляется в массиве findings JSON/HTML report, имеет machine-readable code, family, severity, affected_count and sometimes representative_rva. Его нельзя читать как самостоятельный malware verdict. Это named observation, то есть именованное свидетельство о конкретном нарушении или подозрительном pattern.

Механизм срабатывания: unwind.c не смог надёжно завершить parsing UNWIND_INFO для одной или более entries. Здесь важно различать source fact and interpretation. Source fact - это то, что реально посчитал проект из PeImage, RuntimeFunction array, UnwindSummary array or section characteristics. Interpretation - это почему этот факт важен для reverse engineering. Если source fact неверен из-за более ранней parsing ошибки, interpretation тоже становится слабее. Поэтому findings всегда читаются вместе с pe_validation, section diagnostics and function-level details.

Почему это важно: Parse failure значит сильнее, чем warning: decoder cannot build trustworthy UnwindSummary. Причины могут быть truncated header, impossible operand length, tail outside bounds, unsupported structure, chain problem. Это directly affects RtlVirtualUnwind-like reasoning.

Manual validation: Открыть affected UnwindInfoRVA, проверить bytes, version, flags, CountOfCodes, operand lengths, tail. Если pointer OOB also exists, parse failure may be consequence, not independent root cause.

В отчёте affected_count показывает масштаб, а representative_rva даёт точку входа для навигации. Representative RVA не означает, что проблема только одна. Если finding high severity, он увеличивает strong_count and can lift evidence_level even when total score is not extremely high. Если finding medium, он чаще требует corroboration: соседние findings, dynamic evidence, manual section inspection or Ghidra verification. В любом случае score contribution is bounded and saturated at 100, so analyst must read findings themselves, not only final number.

15. `unwind_anomalies`

Код unwind_anomalies относится к семейству unwind_integrity. В текущей evidence matrix severity указана как medium/high, а score weight равен 10 или 20. Это structured finding: он появляется в массиве findings JSON/HTML report, имеет machine-readable code, family, severity, affected_count and sometimes representative_rva. Его нельзя читать как самостоятельный malware verdict. Это named observation, то есть именованное свидетельство о конкретном нарушении или подозрительном pattern.

Механизм срабатывания: UNWIND_INFO decoded, но summary содержит anomaly flags: version/opcode/order/chain/alignment-like deviations. Здесь важно различать source fact and interpretation. Source fact - это то, что реально посчитал проект из PeImage, RuntimeFunction array, UnwindSummary array or section characteristics. Interpretation - это почему этот факт важен для reverse engineering. Если source fact неверен из-за более ранней parsing ошибки, interpretation тоже становится слабее. Поэтому findings всегда читаются вместе с pe_validation, section diagnostics and function-level details.

Почему это важно: Anomaly отличается от parse failure. Parser смог построить summary, но увидел нарушение or unsupported irregularity. Это сохраняет partial information while still telling analyst that normal model is strained.

Manual validation: Разобрать per-function anomalies in report. Проверить, это strict ABI issue, unknown future version, unusual opcode, bad code order or chain issue. Не смешивать parse fail and anomaly: первое ломает reading, второе помечает suspicious/irregular decoded record.

В отчёте affected_count показывает масштаб, a representative_rva даёт точку входа для навигации. Representative RVA не означает, что проблема только одна. Если finding high severity, он увеличивает strong_count and can lift evidence_level even when total score is not extremely high. Если finding medium, он чаще требует corroboration: соседние findings, dynamic evidence, manual section inspection or Ghidra verification. В любом случае score contribution is bounded and saturated at 100, so analyst must read findings themselves, not only final number.

16. `unwind_reuse_strong`

Код unwind_reuse_strong относится к семейству unwind_reuse. В текущей evidence matrix severity указана как high, a score weight равен 20. Это structured finding: он появляется в массиве findings JSON/HTML report, имеет machine-readable code, family, severity, affected_count and sometimes representative_rva. Его нельзя читать как самостоятельный malware verdict. Это named observation, то есть именованное свидетельство о конкретном нарушении или подозрительном pattern.

Механизм срабатывания: dominant UnwindInfoRVA используется более чем 30% records при n > 200. Здесь важно различать source fact and interpretation. Source fact - это то, что реально посчитал проект из PeImage, RuntimeFunction array, UnwindSummary array or section characteristics. Interpretation - это почему этот факт важен для reverse engineering. Если source fact неверен из-за более ранней parsing ошибки, interpretation тоже становится слабее. Поэтому findings всегда читаются вместе с pe_validation, section diagnostics and function-level details.

Почему это важно: Многие functions pointing to same UNWIND_INFO can be normal for simple tiny functions, but strong concentration in large table may indicate centralized metadata, generated layout, tampering, packing or aggressive sharing. Проект требует n > 200 to avoid small-sample noise.

Manual validation: Проверить dominant UnwindInfoRVA, affected_count, function bodies. Compare compiler style. If shared unwind describes identical leaf-like prologs, benign explanation possible. If unrelated functions share handler-bearing metadata, review is stronger.

В отчёте affected_count показывает масштаб, a representative_rva даёт точку входа для навигации. Representative RVA не означает, что проблема только одна. Если finding high severity, он увеличивает strong_count and can lift evidence_level even when total score is not extremely high. Если finding medium, он чаще требует corroboration: соседние findings, dynamic evidence, manual section inspection or Ghidra verification. В любом случае score contribution is bounded and saturated at 100, so analyst must read findings themselves, not only final number.

17. `handler_dominance`

Код handler_dominance относится к семейству handler_dominance. В текущей evidence matrix severity указана как medium, a score weight равен 15. Это structured finding: он появляется в массиве findings JSON/HTML report, имеет machine-readable code, family, severity, affected_count and sometimes representative_rva. Его нельзя читать как самостоятельный malware verdict. Это named observation, то есть именованное свидетельство о конкретном нарушении или подозрительном pattern.

Механизм срабатывания: один handler target dominates >80% handler records при handler_count > 50 and suspicious context exists. Здесь важно различать source fact and interpretation. Source fact - это то, что реально посчитал проект из PeImage, RuntimeFunction array, UnwindSummary array or section characteristics. Interpretation - это почему этот факт важен для reverse engineering. Если source fact неверен из-за более ранней parsing ошибки, interpretation тоже становится слабее. Поэтому findings всегда читаются вместе с pe_validation, section diagnostics and function-level details.

Почему это важно: Dominant handler can be normal language runtime helper. Поэтому проект подавляет benign-looking dominance when handler is executable code-like and no corroborating static context exists. Finding появляется, когда dominance плюс other static signal делает central handler interesting.

Manual validation: Проверить dominant handler section, code, references, language runtime patterns. Если это обычный __C_specific_handler-like runtime helper, finding может быть benign. Если handler writable/nonexec or paired with tampered metadata, case strengthens.

В отчёте affected_count показывает масштаб, а representative_rva даёт точку входа для навигации. Representative RVA не означает, что проблема только одна. Если finding high severity, он увеличивает strong_count and can lift evidence_level even when total score is not extremely high. Если finding medium, он чаще требует corroboration: соседние findings, dynamic evidence, manual section inspection or Ghidra verification. В любом случае score contribution is bounded and saturated at 100, so analyst must read findings themselves, not only final number.

18. `unwind_reuse_moderate`

Код unwind_reuse_moderate относится к семейству unwind_reuse. В текущей evidence matrix severity указана как medium, а score weight равен 10. Это structured finding: он появляется в массиве findings JSON/HTML report, имеет machine-readable code, family, severity, affected_count and sometimes representative_rva. Его нельзя читать как самостоятельный malware verdict. Это named observation, то есть именованное свидетельство о конкретном нарушении или подозрительном pattern.

Механизм срабатывания: dominant UnwindInfoRVA используется более чем 10% records при n > 200, но только если есть другой corroborating static signal. Здесь важно различать source fact and interpretation. Source fact - это то, что реально посчитал проект из PeImage, RuntimeFunction array, UnwindSummary array or section characteristics. Interpretation - это почему этот факт важен для reverse engineering. Если source fact неверен из-за более ранней parsing ошибки, interpretation тоже становится слабее. Поэтому findings всегда читаются вместе с pe_validation, section diagnostics and function-level details.

Почему это важно: Moderate reuse alone is too common to score. Проект требует corroboration: handler dominance, integrity issue, placement issue or anomaly. Это важное правило осторожности: repetition is not automatically suspicious.

Manual validation: Искать supporting findings. Если moderate reuse единственный странный факт, проект не должен был emit finding. Если он появился, смотреть, какой context made it relevant.

В отчёте affected_count показывает масштаб, а representative_rva даёт точку входа для навигации. Representative RVA не означает, что проблема только одна. Если finding high severity, он увеличивает strong_count and can lift evidence_level even when total score is not extremely high. Если finding medium, он чаще требует corroboration: соседние findings, dynamic evidence, manual section inspection or Ghidra verification. В любом случае score contribution is bounded and saturated at 100, so analyst must read findings themselves, not only final number.

19. `unwind_in_exec_section`

Код unwind_in_exec_section относится к семейству metadata_placement. В текущей evidence matrix severity указана как high, а score weight равен 20. Это structured finding: он появляется в массиве findings JSON/HTML report, имеет machine-readable code, family, severity, affected_count and sometimes representative_rva. Его нельзя читать как самостоятельный malware verdict. Это named observation, то есть именованное свидетельство о конкретном нарушении или подозрительном pattern.

Механизм срабатывания: UNWIND_INFO расположен в section with IMAGE_SCN_MEM_EXECUTE. Здесь важно различать source fact and interpretation. Source fact - это то, что реально посчитал проект из PeImage, RuntimeFunction array, UnwindSummary array or section characteristics. Interpretation - это почему этот факт важен для reverse engineering. Если source fact неверен из-за более ранней parsing ошибки, interpretation тоже становится слабее. Поэтому findings всегда читаются вместе с pe_validation, section diagnostics and function-level details.

Почему это важно: Unwind metadata is data. Normal placement is read-only data-like section such as .xdata/.rdata/.pdata. Executable metadata placement may reflect packed/runtime-generated code, merged sections, or deliberate hiding. It also complicates W^X expectations.

Manual validation: Проверить section name, characteristics, raw/virtual overlap and actual memory protection. If executable section also writable, review becomes stronger. If linker intentionally merged .xdata into executable section, document toolchain evidence.

В отчёте affected_count показывает масштаб, а representative_rva даёт точку входа для навигации. Representative RVA не означает, что проблема только одна. Если finding high severity, он увеличивает strong_count and can lift evidence_level even when total score is not extremely high. Если finding medium, он чаще требует corroboration: соседние findings, dynamic evidence, manual section inspection or Ghidra verification. В любом случае score contribution is bounded and saturated at 100, so analyst must read findings themselves, not only final number.

20. `unwind_unusual_section`

Код unwind_unusual_section относится к семейству metadata_placement. В текущей evidence matrix severity указана как medium/high, а score weight равен 8 или 15. Это structured finding: он появляется в массиве findings JSON/HTML report, имеет machine-readable code, family, severity, affected_count and sometimes representative_rva. Его нельзя читать как самостоятельный malware verdict. Это named observation, то есть именованное свидетельство о конкретном нарушении или подозрительном pattern.

Механизм срабатывания: UNWIND_INFO находится не в allowlist/plausible data section name for unwind metadata. Здесь важно различать source fact and interpretation. Source fact - это то, что реально посчитал проект из PeImage, RuntimeFunction array, UnwindSummary array or section characteristics. Interpretation - это почему этот факт важен для reverse engineering. Если source fact неверен из-за более ранней parsing ошибки, interpretation тоже становится слабее. Поэтому findings всегда читаются вместе с pe_validation, section diagnostics and function-level details.

Почему это важно: Section name is not normative Windows contract, but it is useful context. Unusual placement can signal custom builder or packer. Because section names are weak evidence, project has placement_only_signal weakening rule when this is isolated.

Manual validation: Не читать unusual name as proof. Check permissions first. A harmless section named .eh$unwind-like could be custom but benign. If unusual placement combines with writable/executable permissions or parse failures, it matters more.

В отчёте affected_count показывает масштаб, a representative_rva даёт точку входа для навигации. Representative RVA не означает, что проблема только одна. Если finding high severity, он увеличивает strong_count and can lift evidence_level even when total score is not extremely high. Если finding medium, он чаще требует corroboration: соседние findings, dynamic evidence, manual section inspection or Ghidra verification. В любом случае score contribution is bounded and saturated at 100, so analyst must read findings themselves, not only final number.

21. `unwind_writable_section`

Код unwind_writable_section относится к семейству metadata_placement. В текущей evidence matrix severity указана как medium/high, a score weight равен 10 или 18. Это structured finding: он появляется в массиве findings JSON/HTML report, имеет machine-readable code, family, severity, affected_count and sometimes representative_rva. Его нельзя читать как самостоятельный malware verdict. Это named observation, то есть именованное свидетельство о конкретном нарушении или подозрительном pattern.

Механизм срабатывания: UNWIND_INFO расположен в writable section. Здесь важно различать source fact and interpretation. Source fact - это то, что реально посчитал проект из PeImage, RuntimeFunction array, UnwindSummary array or section characteristics. Interpretation - это почему этот факт важен для reverse engineering. Если source fact неверен из-за более ранней parsing ошибки, interpretation тоже становится слабее. Поэтому findings всегда читаются вместе с pe_validation, section diagnostics and function-level details.

Почему это важно: Writable metadata means process may modify unwind instructions at runtime. That can be benign for runtime/JIT/custom loaders, but for ordinary PE static metadata it is unusual. It weakens trust in static disk model because data may not remain stable.

Manual validation: Check section characteristics and runtime page protections. Ask whether image uses JIT, registration APIs, custom loader, packer or hot patching. If writable metadata also has handler anomalies, manual review is mandatory.

В отчёте affected_count показывает масштаб, а representative_rva даёт точку входа для навигации. Representative RVA не означает, что проблема только одна. Если finding high severity, он увеличивает strong_count and can lift evidence_level even when total score is not extremely high. Если finding medium, он чаще требует corroboration: соседние findings, dynamic evidence, manual section inspection or Ghidra verification. В любом случае score contribution is bounded and saturated at 100, so analyst must read findings themselves, not only final number.

22. `handler_nonexec_section`

Код handler_nonexec_section относится к семейству metadata_placement. В текущей evidence matrix severity указана как high, а score weight равен 15. Это structured finding: он появляется в массиве findings JSON/HTML report, имеет machine-readable code, family, severity, affected_count and sometimes representative_rva. Его нельзя читать как самостоятельный malware verdict. Это named observation, то есть именованное свидетельство о конкретном нарушении или подозрительном pattern.

Механизм срабатывания: handler target section is missing or lacks execute permission. Здесь важно различать source fact and interpretation. Source fact - это то, что реально посчитал проект из PeImage, RuntimeFunction array, UnwindSummary array or section characteristics. Interpretation - это почему этот факт важен для reverse engineering. Если source fact неверен из-за более ранней parsing ошибки, interpretation тоже становится слабее. Поэтому findings всегда читаются вместе с pe_validation, section diagnostics and function-level details.

Почему это важно: Handler RVA should lead to code. If static section context says non-executable or unknown, expected exception route becomes inconsistent. This is strong placement evidence because handler is not metadata; it is control-flow target.

Manual validation: Verify HandlerRVA extraction, section mapping, and whether handler is truly code. If section missing because PE mapping is broken, root cause may be earlier table_integrity issue. Dynamic validation can confirm whether handler is ever hit.

В отчёте affected_count показывает масштаб, а representative_rva даёт точку входа для навигации. Representative RVA не означает, что проблема только одна. Если finding high severity, он увеличивает strong_count and can lift evidence_level even when total score is not extremely high. Если finding medium, он чаще требует corroboration: соседние findings, dynamic evidence, manual section inspection or Ghidra verification. В любом случае score contribution is bounded and saturated at 100, so analyst must read findings themselves, not only final number.

23. `handler_writable_section`

Код handler_writable_section относится к семейству metadata_placement. В текущей evidence matrix severity указана как medium, а score weight равен 10. Это structured finding: он появляется в массиве findings JSON/HTML report, имеет machine-readable code, family, severity, affected_count and sometimes representative_rva. Его нельзя читать как самостоятельный malware verdict. Это named observation, то есть именованное свидетельство о конкретном нарушении или подозрительном pattern.

Механизм срабатывания: handler target lies in writable section. Здесь важно различать source fact and interpretation. Source fact - это то, что реально посчитал проект из PeImage, RuntimeFunction array, UnwindSummary array or section characteristics. Interpretation - это почему этот факт важен для reverse engineering. Если source fact неверен из-за более ранней parsing ошибки, interpretation тоже становится слабее. Поэтому findings всегда читаются вместе с pe_validation, section diagnostics and function-level details.

Почему это важно: Executable code in writable section violates common W^X discipline. It may be used by unpackers, JITs, self-modifying code or unusual linkers. Alone it is not proof of malicious behavior, but it raises review priority.

Manual validation: Check whether section is also executable, inspect handler code, compare runtime protection, look for self-modifying or generated code. If paired with writable metadata, combination finding will amplify case.

В отчёте affected_count показывает масштаб, а representative_rva даёт точку входа для навигации. Representative RVA не означает, что проблема только одна. Если finding high severity, он увеличивает strong_count and can lift evidence_level even when total score is not extremely high. Если finding medium, он чаще требует corroboration: соседние findings, dynamic evidence, manual section inspection or Ghidra verification. В любом случае score contribution is bounded and saturated at 100, so analyst must read findings themselves, not only final number.

24. `placement_exec_nonexec_handler_combo`

Код placement_exec_nonexec_handler_combo относится к семейству metadata_placement. В текущей evidence matrix severity указана как high, а score weight равен 10. Это structured finding: он появляется в массиве findings JSON/HTML report, имеет machine-readable code, family, severity, affected_count and sometimes representative_rva. Его нельзя читать как самостоятельный malware verdict. Это named observation, то есть именованное свидетельство о конкретном нарушении или подозрительном pattern.

Механизм срабатывания: unwind metadata in executable section and handler target in non-executable/unknown section both occur. Здесь важно различать source fact and interpretation. Source fact - это то, что реально посчитал проект из PeImage, RuntimeFunction array, UnwindSummary array or section characteristics. Interpretation - это почему этот факт важен для reverse engineering. Если source fact неверен из-за более ранней parsing ошибки, interpretation тоже становится слабее. Поэтому findings всегда читаются вместе с pe_validation, section diagnostics and function-level details.

Почему это важно: This is a combination finding. It says two placement expectations are inverted at once: data looks like code area, handler code looks non-code. The combined pattern is harder to explain as a single harmless section-name quirk.

Manual validation: Trace both sides: UnwindInfoRVA section and HandlerRVA section. If PE section table is already corrupt, interpret combo as consequence of broken mapping. Otherwise inspect possible obfuscation or custom runtime layout.

В отчёте affected_count показывает масштаб, а representative_rva даёт точку входа для навигации. Representative RVA не означает, что проблема только одна. Если finding high severity, он увеличивает strong_count and can lift evidence_level even when total score is not extremely high. Если finding medium, он чаще требует corroboration: соседние findings, dynamic evidence, manual section inspection or Ghidra verification. В любом случае score contribution is bounded and saturated at 100, so analyst must read findings themselves, not only final number.

25. `placement_integrity_combo`

Код placement_integrity_combo относится к семейству unwind_integrity. В текущей evidence matrix severity указана как medium, а score weight равен 8. Это structured finding: он появляется в массиве findings JSON/HTML report, имеет machine-readable code, family, severity, affected_count and sometimes representative_rva. Его нельзя читать как самостоятельный malware verdict. Это named observation, то есть именованное свидетельство о конкретном нарушении или подозрительном pattern.

Механизм срабатывания: suspicious placement appears together with parse failure, unwind anomaly or unwind_rva_oob. Здесь важно различать source fact and interpretation. Source fact - это то, что реально посчитал проект из PeImage, RuntimeFunction array, UnwindSummary array or section characteristics. Interpretation - это почему этот факт важен для reverse engineering. Если source fact неверен из-за более ранней parsing ошибки, interpretation тоже становится слабее. Поэтому findings всегда читаются вместе с pe_validation, section diagnostics and function-level details.

Почему это важно: Placement oddity alone can be benign. Integrity damage alone can be corruption. Together they corroborate each other: metadata is both oddly located and structurally problematic. Project intentionally adds a separate combo weight to reflect this stronger review signal.

Manual validation: Identify which placement finding and which integrity finding fired. Do not double-count mentally as independent statistical proof. Treat it as engineering corroboration and inspect affected RVAs.

В отчёте affected_count показывает масштаб, а representative_rva даёт точку входа для навигации. Representative RVA не означает, что проблема только одна. Если finding high severity, он увеличивает strong_count and can lift evidence_level even when total score is not extremely high. Если finding medium, он чаще требует corroboration: соседние findings, dynamic evidence, manual section inspection or Ghidra verification. В любом случае score contribution is bounded and saturated at 100, so analyst must read findings themselves, not only final number.

26. `reuse_dominance_combo`

Код reuse_dominance_combo относится к семейству unwind_reuse. В текущей evidence matrix severity указана как medium, а score weight равен 8. Это structured finding: он появляется в массиве findings JSON/HTML report, имеет machine-readable code, family, severity, affected_count and sometimes representative_rva. Его нельзя читать как самостоятельный malware verdict. Это named observation, то есть именованное свидетельство о конкретном нарушении или подозрительном pattern.

Механизм срабатывания: unwind_reuse_ratio > 10% and handler_dominance finding exists. Здесь важно различать source fact and interpretation. Source fact - это то, что реально посчитал проект из PeImage, RuntimeFunction array, UnwindSummary array or section characteristics. Interpretation - это почему этот факт важен для reverse engineering. Если source fact неверен из-за более ранней parsing ошибки, interpretation тоже становится слабее. Поэтому findings всегда читаются вместе с pe_validation, section diagnostics and function-level details.

Почему это важно: Many functions sharing same unwind metadata and many handlers sharing same target both suggest centralization. Centralization can be benign runtime design, but when both patterns appear with context, reverse engineer should inspect whether exception route is being funneled.

Manual validation: Compare dominant UnwindInfoRVA and dominant HandlerRVA. Determine whether they belong to known runtime helper pattern or suspicious dispatch hub. Dynamic handler hits can strengthen or weaken hypothesis.

В отчёте affected_count показывает масштаб, а representative_rva даёт точку входа для навигации. Representative RVA не означает, что проблема только одна. Если finding high severity, он увеличивает strong_count and can lift evidence_level even when total score is not extremely high. Если finding medium, он чаще требует corroboration: соседние findings, dynamic evidence, manual section inspection or Ghidra verification. В любом случае score contribution is bounded and saturated at 100, so analyst must read findings themselves, not only final number.

27. `writable_metadata_handler_combo`

Код writable_metadata_handler_combo относится к семейству metadata_placement. В текущей evidence matrix severity указана как high, а score weight равен 10. Это structured finding: он появляется в массиве findings JSON/HTML report, имеет machine-readable code, family, severity, affected_count and sometimes representative_rva. Его нельзя читать как самостоятельный malware verdict. Это named observation, то есть именованное свидетельство о конкретном нарушении или подозрительном pattern.

Механизм срабатывания: writable unwind metadata and writable handler targets both occur. Здесь важно различать source fact and interpretation. Source fact - это то, что реально посчитал проект из PeImage, RuntimeFunction array, UnwindSummary array or section characteristics. Interpretation - это почему этот факт важен для reverse engineering. Если source fact неверен из-за более ранней parsing ошибки, interpretation тоже становится слабее. Поэтому findings всегда читаются вместе с pe_validation, section diagnostics and function-level details.

Почему это важно: Both the instructions for unwinding and the code target for handling exceptions are writable. This does not prove tampering, but it means the static disk evidence may be mutable at runtime on both data and control-flow sides.

Manual validation: Check section protections, runtime memory protections, loader behavior, JIT/custom generation, and whether actual handler bytes change. This is one of the clearest cases for manual debugger follow-up.

В отчёте affected_count показывает масштаб, а representative_rva даёт точку входа для навигации. Representative RVA не означает, что проблема только одна. Если finding high severity, он увеличивает strong_count and can lift evidence_level even when total score is not extremely high. Если finding medium, он чаще требует corroboration: соседние findings, dynamic evidence, manual section inspection or Ghidra verification. В любом случае score contribution is bounded and saturated at 100, so analyst must read findings themselves, not only final number.

Weakening rules: почему проект иногда снижает confidence, но не удаляет finding

Weakening rule - ослабляющее правило. В C model они называются mitigations, а в JSON выводятся как weakening_rules_applied. Важно: weakening rule не отменяет факт. Если unwind_unusual_section появился, проект не говорит “этого не было”. Он говорит: факт есть, но confidence в сильной интерпретации ниже, потому что signal isolated, sample small or benign explanation plausible. Это зрелое поведение evidence engine.

single_signal_only применяется, когда есть ровно один non-strong finding. small_sample_size применяется при total < 32, findings exist, but no strong finding. placement_only_signal применяется, когда unusual placement is the only placement-like weak signal without stronger integrity or permission issues. single_family_only применяется, когда evidence comes from only one family and no strong finding exists. Эти penalties уменьшают confidence, но не уменьшают score. Следовательно, score describes weighted anomaly mass, while confidence describes trust in the interpretation of static verdict.

```
Weakening rules:
  single_signal_only      -> -0.12 confidence
  small_sample_size       -> -0.06 confidence
  placement_only_signal   -> -0.10 confidence
  single_family_only      -> -0.08 confidence

They do not remove findings and do not subtract score.
```

Это особенно важно для защиты проекта. Если пользователь спросит, почему score не уменьшился после mitigation, правильный ответ такой: mitigation не исправила PE-файл и не сделала metadata normal. Она только сказала, что evidence is less conclusive. Поэтому report должен показывать both finding and weakening rule. Скрывать finding было бы нечестно. Делать вид, что finding alone proves malicious intent, тоже нечестно.

Finding storage limit и честность реализации

В проекте существует HEURISTICS_MAX_FINDINGS = 32. Текущий набор structured rules - 27, то есть обычная матрица помещается. Но технически add_finding имеет fixed array limit. Если future version добавит много rules or creates more structured findings than capacity, details may contain textual lines while structured array truncates. Это limitation, который нужно назвать в книге. Production-grade version should use dynamic array or explicit truncation flag.

Почему это важно для validation? Потому что evidence_level and confidence use stored structured findings for severity counts. Если finding array were truncated silently, assessment could undercount strong/medium findings. Сейчас число rules меньше limit, но архитектурно это edge. Хорошая validation глава не только хвалит проект. Она показывает, где future work should harden evidence model.

Release gates: что означает финальная сводка 1.0.0

Release gate - проверка, которую проект должен пройти перед выдачей artifact. В финальной evidence-сводке 1.0.0 перечислены gates: MinGW-w64 Release CTest 16/16 PASS, MSVC Release CTest 16/16 PASS, Real benign static corpus 106/106 PASS, Labeled static corpus 14/14 PASS, Dynamic smoke 4/4 PASS, Full dynamic corpus 124/124 valid, Dynamic evidence validator PASS, Unexpected dynamic failures 0. Это не означает “нет bugs”. Это означает, что конкретный release artifact прошёл declared gates and has evidence package.

```
Final release gates:
  MinGW-w64 Release CTest:      16 / 16 PASS
  MSVC Release CTest:           16 / 16 PASS
  Real benign static corpus:   106 / 106 PASS
  Labeled static corpus:        14 / 14 PASS
  Dynamic smoke:                 4 / 4 PASS
  Full dynamic corpus:         124 / 124 valid
  Dynamic evidence validator:   PASS
  Unexpected dynamic failures:   0
```

Labeled static corpus - размеченный статический corpus. Labeled значит, что для samples заданы expected outcomes. В summary указано 14/14 PASS, with 12 anomalous files correctly raised by heuristics. Это сильнее, чем просто “запустилось”, потому что проверяется expected signal. Но нужно помнить: synthetic/labeled samples usually created around known rule classes. Поэтому они validate implementation contract and regression behavior, but not universal detection performance.

Evidence package - доказательный пакет. Это не executable for normal user, а набор artifacts: individual JSON reports, logs, aggregate CSV/JSON/Markdown, validators, schema and summary. Его роль - позволить аудитору проверить claims. В reverse-engineering project this matters greatly: if final presentation says

“dynamic full 124/124 valid”, evidence package should contain the reports and validation scripts, not just a slide with a number.

Что этот раздел доказывает - и где он честно останавливается

После подробного разбора tests, corpus, fuzzing, validation and findings можно сформулировать итог осторожно. Проект доказывает, что implemented contracts проходят declared gates. Он показывает bounded parser discipline, explainable heuristic matrix, stable report schema, benign corpus compatibility, synthetic anomaly regression, deterministic mutation robustness and selected Windows dynamic mechanisms. Это серьёзная инженерная база для выпускной работы and practical reverse-engineering tool.

Но этот раздел не должен превращаться в рекламный claim. Проект не доказывает universal malware detection. Не доказывает recall на всех реальных tampering techniques. Не доказывает полную coverage dynamic behavior. Не доказывает поддержку всех future unwind versions such as V3/APX without explicit decoder policy. Не доказывает statistical calibration of score/confidence. Не доказывает отсутствие bugs outside tested space. Это не слабость формулировки, а профессиональная честность.

Самый правильный итог такой: EH/Unwind Analyzer - explainable static/dynamic evidence tool for Windows x64 EH/unwind metadata. Его tests and validation показывают, какие свойства проверены, а finding catalog показывает, что именно считается свидетельством. Реверсер должен использовать report as map, not verdict oracle. Если finding появляется, он не говорит “зло”. Он говорит: вот нарушенный или необычный contract, вот affected count, вот representative RVA, вот score contribution, вот weakening context, вот manual validation path. Именно такая связка делает проект полезным and defensible.

11.11. Финальная сборка архитектуры EH/Unwind Analyzer: как слои проекта становятся одной проверяемой историей

Этот раздел нужно читать не как красивую финальную схему и не как список файлов исходного кода. Финальная архитектура проекта - это попытка ответить на один практический вопрос: каким образом недоверенный Windows x64 PE-файл превращается в осторожный, проверяемый и пригодный для ручной работы вывод? Если отвечать слишком быстро, получится привычная инженерная фраза: input bytes идут в parser, потом в decoder, потом в heuristics, потом в report. Такая фраза внешне правильна, но для реверс-инженера почти бесполезна, потому что она прячет главный смысл. Она не объясняет, где именно появляется знание, где сохраняется неопределённость, где проект берёт на себя ответственность, а где обязан сказать unknown. Поэтому сейчас мы пройдём эту цепочку медленно, как будто держим в руках один файл и смотрим, как каждый слой проекта добавляет к нему не мнение, а новую проверяемую форму.

Главное слово здесь - architecture, архитектура. В обычной речи архитектура часто означает “общий рисунок”. В инженерной книге этого мало. Архитектура проекта - это не картинка из прямоугольников, а распределение ответственности. Она отвечает, какой слой имеет право читать bytes, какой слой имеет право говорить про PE image, какой слой имеет право говорить про unwind metadata, какой слой имеет право делать осторожные выводы, какой слой имеет право запускать процесс, а какой слой должен только сериализовать результат. Если слой начинает делать чужую работу, анализатор становится опасным: parser начинает подозревать malware, heuristics начинают притворяться доказательством, dynamic collector начинает звучать как oracle, а report начинает выглядеть увереннее, чем evidence.

> Architecture. Буквально: архитектура, то есть устройство системы и распределение обязанностей между частями. Где живёт в проекте: в этой главе - вся связка util.c, pe.c, unwind.c, heuristics.c, dynamic_win.c, report.c, Ghidra importer and validation files. Зачем нужен: помогает понять, почему проект разделён на слои, а не написан одним большим файлом. Что не означает: не означает красивую схему ради схемы и не означает, что каждый модуль изолирован от смысла остальных. Связь с общей моделью: architecture связывает теорию Windows x64 unwind с конкретными C-структурами и отчётами проекта.

От недоверенного файла к рабочему артефакту: почему цепочка должна быть длинной

Когда пользователь запускает EH/Unwind Analyzer на PE-файле, проект не получает “программу” в высокоуровневом смысле. Он получает bytes - байты, то есть последовательность чисел от 0 до 255. Эти bytes могут быть нормальным исполняемым файлом, повреждённым файлом, специально мутированным образцом, результатом упаковщика, недописанным фрагментом, тестовым fixture или вообще мусором. Пока проект не доказал обратное, он не имеет права считать эти bytes валидным PE. Поэтому самый первый architectural decision - архитектурное решение - звучит скучно, но является фундаментальным: все данные на входе недоверенные.

> Input bytes. Буквально: входные байты, сырое содержимое файла до интерпретации. Где живёт в проекте: в проекте они появляются как содержимое файла, загруженное в Buf. Зачем нужен: позволяют анализатору работать с любым файлом одинаково осторожно. Что не означает: не означают PE, секции, функции или инструкции сами по себе. Связь с общей моделью: input bytes сначала проходят util.c, и только потом часть из них может стать PE header, section table, .pdata or .xdata.

Слово artifact тоже важно разжевать заранее. Artifact - артефакт, результат работы, который можно сохранить, передать, проверить или использовать дальше. В проекте artifact может быть не только PDF или HTML. Parsed PeImage - тоже артефакт внутреннего уровня. HeuristicReport - артефакт интерпретации. DynamicReport - артефакт runtime-сессии. JSON report - внешний артефакт, который может прочитать другой инструмент. Ghidra CSV - мостовой артефакт между analyzer и рабочей средой реверсера. Такой термин полезен потому, что заставляет спросить: кто создаёт этот artifact, кто ему доверяет, как его проверять, и что он не обязан доказывать.

> Artifact. Буквально: артефакт, то есть созданный результат, который можно хранить или передавать дальше. Где живёт в проекте: в проекте это и внутренние структуры вроде PeImage/UnwindSummary, и внешние файлы вроде JSON/HTML/CSV. Зачем нужен: помогает отделить процесс вычисления от результата вычисления. Что не означает: не означает, что результат автоматически истинный или полный. Связь с общей моделью: каждый artifact имеет источник, границы доверия, формат и способ проверки.

```
input bytes
  -> util.c
  -> pe.c
  -> unwind.c
  -> heuristics.c
  -> optional dynamic_win.c
  -> report.c
  -> JSON / HTML / CSV / Ghidra / manual review
```

Эта схема короткая, но теперь она должна читаться иначе. Стрелка здесь не означает “передали объект дальше и забыли”. Стрелка означает смену модели. util.c ещё не знает про PE. pe.c уже строит PE image model, но ещё не делает unwind-decoding. unwind.c понимает RUNTIME_FUNCTION and

UNWIND_INFO, но не выносит подозрительный verdict. heuristics.c превращает структурные факты в findings, но не запускает процесс. dynamic_win.c наблюдает runtime-events, но не переписывает static facts. report.c сохраняет результат, но не должен усиливать выводы ради красоты.

> Pipeline. Буквально: конвейер, последовательность стадий обработки. Где живёт в проекте: в проекте это путь от input bytes до report. Зачем нужен: заставляет держать в голове порядок появления знания. Что не означает: не означает механическую трубу без смысловых границ. Связь с общей моделью: pipeline показывает, какие данные уже нормализованы, а какие ещё остаются сырой гипотезой.

Layer и boundary: почему слой обязан знать свою границу

Теперь нужно остановиться на двух английских словах, которые часто произносят автоматически: layer and boundary. Layer - слой. Boundary - граница. Слой - это часть системы, которая работает на своём уровне абстракции. Абстракция здесь не означает “что-то туманное”. Она означает выбранный способ смотреть на данные. Для util.c данные - это диапазон bytes и offset. Для pe.c данные - это headers, sections, RVA and data directories. Для unwind.c данные - это RUNTIME_FUNCTION entries and UNWIND_INFO records. Для heuristics.c данные - это normalized facts and evidence signals. Для report.c данные - это structured fields, которые нужно сериализовать без потери смысла. Граница слоя нужна, чтобы один уровень не притворялся другим.

> Layer. Буквально: слой, часть системы со своей точкой зрения на данные. Где живёт в проекте: каждый исходный модуль проекта играет роль слоя: util.c, pe.c, unwind.c, heuristics.c, dynamic_win.c, report.c. Зачем нужен: не даёт смешивать чтение bytes, PE parsing, unwind decoding, scoring and reporting. Что не означает: не означает изоляцию от общей цели проекта. Связь с общей моделью: layer делает только свою часть evidence chain.
>
> Boundary. Буквально: граница, место перехода между слоями или режимами доверия. Где живёт в проекте: между input bytes and Buf, Buf and PeImage, static and dynamic mode, C structures and JSON/HTML. Зачем нужен: показывает, где меняется смысл данных и кто отвечает за проверку. Что не означает: не означает стену, через которую данные не проходят. Связь с общей моделью: boundary помогает понять, почему один и тот же адрес может быть file offset, RVA or VA depending on context.

Особенно важна trust boundary - граница доверия. Trust - доверие. Но в инженерном контексте trust не значит “верю человеку”. Trust означает, что система разрешает данным влиять на следующие решения. Когда bytes только загружены из файла, доверия к ним нет. Когда pe.c проверил headers and sections, часть данных получила ограниченное доверие: теперь можно сказать, что это похоже на PE32+ AMD64 image, но всё ещё нельзя сказать, что unwind metadata корректна. Когда unwind.c распарсил UNWIND_INFO, можно

говорить о decoded unwind summary, но не о том, что handler реально исполнится. Когда dynamic_win.c увидел handler hit, можно говорить о runtime observation in this run, но нельзя говорить, что все возможные routes покрыты. Trust в проекте всегда частичный и контекстный.

> Trust boundary. Буквально: граница доверия, переход от недоверенных данных к частично проверенной модели. Где живёт в проекте: главная такая граница находится между raw file bytes and static analyzer; другая - между static mode and dynamic execution. Зачем нужен: не даёт проекту принимать file contents за истину без проверки. Что не означает: не означает абсолютную безопасность после прохождения границы. Связь с общей моделью: trust boundary связывает security engineering с reverse engineering: мы не только читаем файл, но и ограничиваем доверие к каждому выводу.

Есть ещё execution boundary - граница выполнения. Execution - выполнение, то есть реальный запуск кода. Пока анализатор работает в static mode, он читает файл как данные. Он не передаёт управление этому файлу. Но когда включается dynamic mode, проект создаёт child process under debugger. Это уже другой класс риска: код начинает жить, загружать modules, вызывать Windows APIs, генерировать exceptions, регистрировать handlers or function tables. Поэтому dynamic_win.c не может быть просто “ещё одним parser”. Это слой, который пересекает execution boundary и обязан быть явно ограниченным.

> Execution boundary. Буквально: граница между чтением файла как данных и запуском процесса как живого кода. Где живёт в проекте: в проекте пересекается только optional dynamic_win.c при --dynamic. Зачем нужен: объясняет, почему dynamic mode требует отдельной осторожности, timeout and event limits. Что не означает: не означает, что dynamic mode автоматически лучше static mode. Связь с общей моделью: execution boundary отделяет static evidence from runtime evidence.
>
> Serialization boundary. Буквально: граница сериализации, переход от C-структур к файлам отчёта. Где живёт в проекте: в report.c, когда HeuristicReport, DynamicReport and PeImage превращаются в JSON, HTML or CSV. Зачем нужен: защищает смысл полей от искажения при выводе. Что не означает: не означает простую печать строк. Связь с общей моделью: serialization boundary делает report внешним contract for humans and tools.

util.c: слой, который ещё ничего не “понимает”, но уже защищает всю книгу

Первый реальный слой проекта - util.c. Его легко недооценить, потому что он не знает красивых слов вроде UNWIND_INFO or RtlVirtualUnwind. Но именно поэтому он важен. Он работает до интерпретации. Он говорит: прежде чем спорить о .pdata, давайте убедимся, что чтение по offset не вышло за границы buffer, что сложение off + len не переполнилось, что little-endian value собран явно, а не случайно через C struct cast. Если этот слой слабый, все

последующие выводы могут быть построены на повреждённой памяти самого анализатора.

> Buffer. Буквально: буфер, область памяти с данными. Где живёт в проекте: в проекте представлен Buf, который хранит bytes файла and size. Зачем нужен: даёт единый способ читать входной файл безопасно. Что не означает: не означает валидный PE and не означает mapped image. Связь с общей моделью: Buf - это мост от file bytes к pe.c, но без PE-смысла.
>
> Offset. Буквально: смещение, позиция относительно начала некоторой области. Где живёт в проекте: в util.c offset относится к позиции в input buffer; в PE context позже появятся file offset and RVA. Зачем нужен: показывает, откуда именно читать bytes. Что не означает: не означает адрес выполнения в памяти. Связь с общей моделью: offset становится опасным, если перепутать его с RVA or VA.
>
> Little-endian. Буквально: порядок байтов, при котором младший байт числа лежит первым. Где живёт в проекте: в util.c значения читаются через le16, le32, le64. Зачем нужен: нужен, потому что PE fields encoded in little-endian order. Что не означает: не означает “маленький размер числа”. Связь с общей моделью: little-endian conversion делает чтение PE fields независимым от неявных C layout assumptions.
>
> Ownership. Буквально: владение ресурсом, то есть понимание, кто отвечает за освобождение памяти или закрытие handle. Где живёт в проекте: в проекте видно через owned_mem, allocated arrays, process/thread handles and report buffers. Зачем нужен: предотвращает leaks, double free and use-after-free mistakes. Что не означает: не означает юридическое владение файлом. Связь с общей моделью: ownership проходит через весь pipeline: кто создал artifact, тот должен знать, когда он больше не нужен.

В архитектурной модели util.c создаёт не знание о программе, а надёжность доступа. Это как прочная линейка перед измерением. Линейка не говорит, хорошая ли функция, подозрительный ли handler, странный ли RUNTIME_FUNCTION. Но если линейка кривая, все измерения дальше будут лгать. Поэтому в финальной архитектуре util.c стоит не “внизу для удобства”, а в основании доверия.

pe.c: момент, когда bytes становятся Windows image model

После util.c проект может осторожно перейти к pe.c. Здесь появляется PE - Portable Executable, переносимый исполняемый формат Windows. Слово portable в историческом названии не означает, что один и тот же файл одинаково выполняется на всех процессорах. Оно означает формат семейства PE/COFF, который Windows uses to describe executable images and object files. pe.c не обязан понимать prolog or exception handlers. Его задача другая: доказать, что перед нами PE32+ AMD64 image enough to build an address map. Именно address map потом позволит отличить .text from .pdata, RVA from file

offset, executable section from writable section.

PE. Буквально: Portable Executable, формат исполняемых image-файлов Windows. Где живёт в проекте: в pe.c как проверяемая структура headers, sections and data directories. Зачем нужен: задаёт карту, по которой проект понимает, где лежит код, данные and Exception Directory. Что не означает: не означает автоматически корректную программу или безопасный файл. Связь с общей моделью: PE model связывает file bytes with Windows loader view.

Image. Буквально: образ, то есть представление PE-файла как загружаемого модуля. Где живёт в проекте: в проекте PeImage хранит normalized PE facts. Зачем нужен: позволяет рассуждать о RVA and sections, как это делает Windows loader conceptually. Что не означает: не означает, что файл уже реально загружен в процесс. Связь с общей моделью: image model нужен, чтобы .pdata and .xdata читать не как случайные bytes, а как metadata inside module.

RVA. Буквально: Relative Virtual Address, относительный виртуальный адрес. Где живёт в проекте: в pe.c используется для поиска section and mapping to file offset. Зачем нужен: стабилен внутри image независимо от actual runtime base. Что не означает: не означает file offset и не означает runtime VA. Связь с общей моделью: RVA связывает PE file, RUNTIME_FUNCTION fields, handler RVAs and Ghidra import.

VA. Буквально: Virtual Address, виртуальный адрес в адресном пространстве процесса. Где живёт в проекте: в static mode обычно восстанавливается как ImageBase + RVA; в dynamic mode как module runtime base + RVA. Зачем нужен: нужен, когда речь идёт о runtime breakpoint or CPU context. Что не означает: не означает позицию в файле. Связь с общей моделью: VA появляется после загрузки image, тогда как RVA удобнее для report and cross-tool exchange.

File offset. Буквально: смещение в файле на диске. Где живёт в проекте: в pe.c получается через pe_rva_to_fileoff when bytes really exist in raw data. Зачем нужен: позволяет безопасно читать structures from disk file. Что не означает: не означает адрес после загрузки в память. Связь с общей моделью: file offset нужен util.c/unwind.c, а RVA нужен semantic model and reports.

Официальная документация Microsoft по PE Format фиксирует различие между RVA, VA and file pointer, а также роль data directories. Это важно: анализатор не придумывает собственный мир адресов. Он строит свой PeImage поверх Windows-контракта, но добавляет защитные проверки, diagnostics and conservative policies, потому что reverse engineering работает не только с идеальными linker outputs.

unwind.c: слой, где адресные ranges превращаются в модель восстановления кадра

Когда pe.c нашёл Exception Directory, начинается работа unwind.c. Здесь появляется одно из центральных слов всей книги - unwind. Буквально unwind значит “размотать”. В контексте Windows x64 это не про разматывание нитки, а про восстановление caller context: как из состояния внутри текущей функции понять, каким было состояние вызывающей функции. Windows не может просто угадать, где лежит return address and saved registers. Ей нужна metadata. Поэтому компилятор и linker создают .pdata and .xdata, а Windows использует их через runtime function table and unwind information.

Unwind. Буквально: размотка стека, восстановление состояния вызывающего кадра. Где живёт в проекте: в проекте декодируется в unwind.c and summarized in UnwindSummary. Зачем нужен: нужен Windows for exception handling and stack walking on x64. Что не означает: не означает физическое исполнение ret instruction. Связь с общей моделью: unwind связывает RIP, RSP, nonvolatile registers, RUNTIME_FUNCTION and UNWIND_INFO.

RUNTIME_FUNCTION. Буквально: запись таблицы функций времени выполнения, range descriptor for one code region. Где живёт в проекте: в проекте хранится как RuntimeFunction[] from Exception Directory. Зачем нужен: указывает BeginAddress, EndAddress and UnwindInfoAddress. Что не означает: не означает имя функции and не гарантирует совпадение с disassembler function boundary. Связь с общей моделью: RUNTIME_FUNCTION связывает address range in .text with UNWIND_INFO in .xdata.

UNWIND_INFO. Буквально: информация размотки, компактная инструкция восстановления кадра. Где живёт в проекте: в unwind.c декодируется из .xdata through RVA mapping. Зачем нужен: описывает prolog effects on RSP, nonvolatile registers, frame register and handlers. Что не означает: не является машинным кодом and не является обработчиком исключения itself. Связь с общей моделью: UNWIND_INFO объясняет RtlVirtualUnwind how to compute caller context.

UNWIND_CODE. Буквально: элемент инструкции размотки, описывающий одну операцию prolog. Где живёт в проекте: inside UNWIND_INFO code slots parsed by unwind.c. Зачем нужен: фиксирует операции вроде push nonvolatile register or stack allocation. Что не означает: не означает opcode CPU directly; это metadata about prolog action. Связь с общей моделью: UNWIND_CODE превращает compiler prolog into data Windows can replay backwards.

Prolog. Буквально: начальная часть функции, где функция подготавливает stack frame. Где живёт в проекте: описывается SizeOfProlog and unwind codes. Зачем нужен: важен, потому что именно там меняются RSP and saved registers. Что не означает: не означает весь body функции. Связь с общей моделью: prolog связывает real instructions and unwind metadata.

Epilog. Буквально: конечный участок функции, где кадр сворачивается перед возвратом. Где живёт в проекте: официальная x64 модель unwind учитывает особые rules for epilog. Зачем нужен: важен, потому что unwind inside epilog differs from body case. Что не означает: не означает любой ret anywhere.

Связь с общей моделью: epilog помогает Windows понять, находится ли поток в процессе выхода из функции.

unwind.c не должен превращаться в эвристический судья. Если UNWIND_INFO не читается, decoder должен сказать parse_error. Если chain слишком глубокий, он должен остановиться по max_chain_depth. Если strict_abi mode включён, decoder может пометить ABI warning. Но он не должен делать вывод “это malware”. Он готовит факты для heuristics.c. Это очень важная архитектурная дисциплина: decoder explains structure, evidence engine interprets patterns.

Parse error. Буквально: ошибка разбора, когда bytes не удалось безопасно интерпретировать как ожидаемую структуру. Где живёт в проекте: в unwind.c попадает в parse_error and affects us_ok[]. Зачем нужен: нужна, чтобы отличить “нет unwind issue” от “мы не смогли прочитать metadata”. Что не означает: не означает автоматически злонамеренность. Связь с общей моделью: parse_error становится evidence input for heuristics and report.

Summary. Буквально: сводка, нормализованное краткое представление более сложной структуры. Где живёт в проекте: UnwindSummary хранит decoded facts from RUNTIME_FUNCTION/UNWIND_INFO. Зачем нужен: позволяет heuristics.c работать с facts, а не заново читать bytes. Что не означает: не означает потерю смысла до уровня “одна строка”. Связь с общей моделью: summary - contract between decoder and evidence engine.

heuristics.c: осторожное толкование evidence, а не магический классификатор

После unwind.c проект знает много фактов, но факты ещё не являются assessment. Здесь появляется heuristics.c. Слово heuristic - эвристика - часто звучит как “примерная догадка”, и из-за этого его легко либо переоценить, либо недооценить. В этом проекте heuristic - это инженерное правило интерпретации observed signals. Оно не является machine learning model, не обучено на миллионах образцов и не выдаёт probability of malware. Оно говорит: если metadata размещена странно, если многие functions share same UnwindInfoRVA, если handler target points to writable section, если RUNTIME_FUNCTION ranges broken, то это evidence worth manual review. Осторожность здесь важнее громкости.

Heuristic. Буквально: эвристика, практическое правило вывода по наблюдаемым признакам. Где живёт в проекте: в heuristics.c как набор rules that produce findings, score and confidence. Зачем нужен: помогает превратить structural facts into reviewable evidence. Что не означает: не означает доказательство и не означает статистическую вероятность. Связь с общей моделью: heuristic связывает PE/unwind anomalies with report findings.

Evidence. Буквально: свидетельство, наблюдаемый факт или сигнал, который поддерживает вывод. Где живёт в проекте: в проекте evidence appears as

findings, evidence families, runtime events and assessment fields. Зачем нужен: нужно, чтобы report объяснял почему он что-то говорит. Что не означает: не означает окончательный приговор. Связь с общей моделью: evidence chain должна вести от report back to bytes or runtime event.

Finding. Буквально: найденный признак, structured observation with code, family, severity and message. Где живёт в проекте: в HeuristicFinding array inside HeuristicReport. Зачем нужен: показывает конкретную причину интереса or concern. Что не означает: не означает malware and не означает exploit. Связь с общей моделью: finding должен быть проверяем руками through PE view, unwind view, Ghidra or runtime evidence.

Score. Буквально: балл, инженерная сумма весов признаков. Где живёт в проекте: в HeuristicReport as bounded integer score. Зачем нужен: позволяет сортировать силу статического случая. Что не означает: не означает процент вероятности. Связь с общей моделью: score помогает evidence_level and verdict, но не заменяет explanation.

Suspicion. Буквально: подозрительность, нормализованное представление score. Где живёт в проекте: в отчёте обычно derived from score/100. Зачем нужен: удобна для JSON/HTML reading. Что не означает: не означает calibrated probability. Связь с общей моделью: suspicion должна читаться рядом с findings and confidence.

Confidence. Буквально: уверенность, rule-based оценка полноты и согласованности evidence. Где живёт в проекте: в HeuristicReport вычисляется отдельно от score. Зачем нужен: показывает, насколько evidence supported by multiple families and strong signals. Что не означает: не означает статистический confidence interval. Связь с общей моделью: confidence может снижаться weakening rules and small sample conditions.

Verdict. Буквально: итоговая текстовая оценка статического анализа. Где живёт в проекте: в HeuristicReport and final assessment text fields. Зачем нужен: даёт короткую human-readable формулировку. Что не означает: не означает судебный приговор и не заменяет findings. Связь с общей моделью: verdict должен быть менее важен, чем evidence explanation.

В финальной архитектуре heuristics.c - это место, где проект впервые говорит человеческим языком: normal, unusual, suspicious, requires manual review. Но даже здесь язык должен оставаться инженерным. Если finding has severity strong, это означает strong signal within this evidence model, а не “точно вредоносно”. Если requires_manual_review is true, это не провал автоматизации, а честная передача следующего шага человеку.

Manual review. Буквально: ручная проверка человеком. Где живёт в проекте: в проекте triggers through requires_manual_review and manual_validation_steps. Зачем нужен: нужна там, где automatic evidence insufficient for final judgment. Что не означает: не означает, что analyzer бесполезен. Связь с общей моделью: manual review continues evidence chain using disassembler, debugger, report and source context.

dynamic_win.c: runtime evidence в одном запуске, а не всевидящее доказательство

Static analysis видит файл и metadata. Но Windows-программа живёт во времени: она загружает modules, вызывает APIs, получает exceptions, регистрирует vectored handlers, может добавлять dynamic function tables. Поэтому в проекте есть optional dynamic_win.c. Optional - необязательный - означает, что static analyzer должен быть полезен без запуска target, а dynamic collector должен добавлять слой runtime evidence только по явному запросу. Dynamic - динамический - здесь значит “наблюдаемый во время выполнения”. Но это не “истина обо всём поведении программы”. Это истина только о конкретном запуске, конкретных аргументах, конкретной environment, конкретном timeout and coverage.

Dynamic validation. Буквально: динамическая проверка, наблюдение за поведением при запуске. Где живёт в проекте: в проекте реализована в dynamic_win.c and stored in DynamicReport. Зачем нужен: проверяет, проявляются ли некоторые static hypotheses at runtime. Что не означает: не означает полный анализ всех путей исполнения. Связь с общей моделью: dynamic validation connects runtime events with static findings through correlation rules.

Debuggee. Буквально: отлаживаемый процесс, target program under debugger control. Где живёт в проекте: в dynamic_win.c это child process created by CreateProcessA with debug flag. Зачем нужен: даёт collector возможность получать debug events. Что не означает: не означает безопасную песочницу. Связь с общей моделью: debuggee crosses execution boundary.

Debugger. Буквально: отладчик, процесс, который получает debug events and controls continuation. Где живёт в проекте: в проекте роль debugger играет analyzer process in dynamic mode. Зачем нужен: позволяет ставить breakpoints and observe exceptions. Что не означает: не означает интерактивный GUI debugger like x64dbg. Связь с общей моделью: debugger layer turns runtime behavior into DynamicReport.

Breakpoint. Буквально: точка останова, специальное вмешательство, которое прерывает выполнение при достижении адреса. Где живёт в проекте: dynamic_win.c uses software breakpoint byte 0xCC and Breakpoint records. Зачем нужен: позволяет увидеть calls or handler entries. Что не означает: не означает пассивное наблюдение; это изменение target memory. Связь с общей моделью: breakpoint connects static handler RVA or API VA to runtime event.

Handler hit. Буквально: попадание в обработчик, событие когда execution reaches known handler address. Где живёт в проекте: in DynamicReport events like unwind_handler_hit, veh_handler_hit, vch_handler_hit, uef_handler_hit. Зачем нужен: усиливает runtime evidence for handler-related hypotheses. Что не означает: не означает, что handler malicious или что все handler paths

covered. Связь с общей моделью: handler hit can correlate static handler metadata with live execution.

RIP trace. Буквально: короткая последовательность instruction pointers after a runtime hit. Где живёт в проекте: dynamic_win.c records limited trace after selected handler hit. Зачем нужен: помогает увидеть immediate route after entry. Что не означает: не означает полный execution trace of the program. Связь с общей моделью: RIP trace is local runtime evidence, not whole-program proof.

Timeout. Буквально: ограничение времени сессии. Где живёт в проекте: in DynamicReport as timed_out and timeout event. Зачем нужен: предотвращает бесконечный or too long dynamic run. Что не означает: не означает отсутствие suspicious behavior. Связь с общей моделью: timeout weakens runtime conclusions and may cap evidence level.

dynamic_win.c особенно хорошо показывает, почему архитектура должна быть честной. Если collector не увидел handler hit, report не должен говорить “handler не существует”. Он должен говорить “not observed in this session”. Если process timed out, runtime evidence incomplete. Если breakpoint write failed, это отдельный observation, not a silent absence. В корректной архитектуре отсутствие события тоже должно иметь правильную семантику.

report.c: место, где проект обязан говорить ясно, а не красиво

Когда static facts, heuristic findings and runtime events уже собраны, появляется report.c. Report - отчёт - это не декоративный вывод и не финальная витрина. Это serialization boundary, где внутренние C-структуры получают внешний вид. Если report потерял поле, усилил формулировку, смешал static and dynamic evidence, спрятал parse_error or timeout, то ошибка уже не только косметическая. Это analysis bug, потому что человек будет принимать решения по искажённому artifact.

Report. Буквально: отчёт, внешнее представление результата анализа. Где живёт в проекте: report.c writes console, CSV, Ghidra CSV, JSON and HTML. Зачем нужен: позволяет человеку and tools read evidence chain. Что не означает: не означает verdict-only summary. Связь с общей моделью: report is the place where PeImage, UnwindSummary, HeuristicReport, DynamicReport and assessment meet.

JSON. Буквально: JavaScript Object Notation, текстовый формат структурированных данных. Где живёт в проекте: project JSON report is machine-readable contract validated by schema. Зачем нужен: нужен для automation, regression validation and downstream tools. Что не означает: не означает human-friendly explanation by itself. Связь с общей моделью: JSON report preserves structured evidence fields.

HTML. Буквально: HyperText Markup Language, формат страницы для просмотра в браузере. Где живёт в проекте: project HTML report is offline readable human report. Зачем нужен: помогает реверсеру быстро видеть summary, findings, functions and dynamic status. Что не означает: не означает интерактивный live debugger. Связь с общей моделью: HTML report turns evidence into navigation for human review.

CSV. Буквально: Comma-Separated Values, табличный текстовый формат. Где живёт в проекте: обычный CSV and Ghidra CSV are emitted by report.c. Зачем нужен: нужен для tables, spreadsheets and tool bridge. Что не означает: не означает rich semantic database. Связь с общей моделью: CSV is useful only when columns and escaping are treated as contract.

Schema. Буквально: схема, формальное описание структуры документа. Где живёт в проекте: docs/report.schema.json constrains JSON output. Зачем нужен: позволяет проверить, что report shape did not drift silently. Что не означает: не доказывает semantic correctness of every field. Связь с общей моделью: schema validation complements tests and semantic validation.

Assessment. Буквально: итоговая оценка, которая связывает static and runtime evidence. Где живёт в проекте: in report.c as final block with final_verdict, case_strength, correlation and manual steps. Зачем нужен: даёт аккуратное summary after all evidence layers. Что не означает: не означает абсолютную истину about binary. Связь с общей моделью: assessment should be traceable back to findings and runtime events.

Case strength. Буквально: сила случая, оценка того, насколько evidence forms a coherent case. Где живёт в проекте: in assessment as values like strong_correlated_case, medium_case, weak_case. Зачем нужен: помогает отличить correlated evidence from isolated signal. Что не означает: не означает severity of malware. Связь с общей моделью: case_strength depends on static level, runtime level and correlation.

Важно, что report.c не должен “улучшать” язык за счёт смысла. Если static evidence strong, но dynamic not run, final assessment должен быть strong_unconfirmed_case, not strong_correlated_case. Если dynamic session incomplete, runtime level cannot become strong by accident. Если finding placement-only, weakening rule must remain visible. Красивый report, который говорит слишком уверенно, хуже скучного report, который честно показывает limits.

Ghidra integration: мост в рабочую среду, а не замена анализа

Ghidra integration завершает практический контур. Ghidra - reverse engineering framework, среда для disassembly, decompilation, navigation, comments and bookmarks. Project importer не должен притворяться, что он “встраивает весь analyzer внутрь Ghidra”. Он делает более узкую, но важную вещь: берёт Ghidra CSV and переносит часть evidence into the place where

reverser actually reads code. Это bridge - мостовой формат. Мост не является городом на обеих сторонах реки; он лишь позволяет перейти.

Ghidra. Буквально: фреймворк реверс-инжиниринга с disassembler, decompiler, scripting and navigation tools. Где живёт в проекте: project uses ghidra/import_unwind_csv.py to import analyzer output. Зачем нужен: помогает видеть findings near code ranges. Что не означает: не означает, что Ghidra автоматически подтверждает findings. Связь с общей моделью: Ghidra integration connects static report with manual reverse engineering workflow.

Importer. Буквально: импортёр, скрипт, который читает внешний файл и добавляет данные в рабочую среду. Где живёт в проекте: ghidra/import_unwind_csv.py reads Ghidra CSV. Зачем нужен: переносит ranges, comments and bookmarks into Ghidra program database. Что не означает: не означает полноценный analyzer inside Ghidra. Связь с общей моделью: importer turns CSV artifact into navigational hints.

Bookmark. Буквально: закладка, пометка в Ghidra на конкретном адресе. Где живёт в проекте: importer creates bookmarks for warnings/anomalies. Зачем нужен: помогает вернуться к важному месту. Что не означает: не означает доказательство suspicious behavior. Связь с общей моделью: bookmark is a human navigation aid tied to evidence.

Comment. Буквально: комментарий, текстовая заметка рядом с кодом or data. Где живёт в проекте: importer can add comments with unwind/finding context. Зачем нужен: сокращает расстояние между report and code view. Что не означает: не должен уничтожать пользовательские комментарии without care. Связь с общей моделью: comment is a bridge from analyzer language to analyst workspace.

Validation: как архитектура проверяет не “громкие слова”, а конкретные обещания

Финальная архитектура была бы неполной без validation. Validation - проверка соответствия реальности и требованиям. Verification - проверка, что конкретная реализация делает заявленное. В разговоре эти слова часто смешивают, но в проекте полезно различать их. Unit tests verify individual functions. Corpus validation checks behavior on known files. Fuzzing searches for crashes and undefined behavior. JSON Schema validation checks report shape. Semantic validation checks cross-field invariants. Release evidence package records what was actually run before declaring version ready.

Validation. Буквально: проверка, что система даёт полезные и корректно ограниченные результаты для заявленной задачи. Где живёт в проекте: в проекте через corpus, reports, schema checks, final validation summaries. Зачем нужен: показывает, что analyzer behaves consistently on known cases. Что не означает: не означает абсолютную гарантию for all binaries. Связь с общей моделью: validation connects engineering tests with reverse engineering trust.

Verification. Буквально: проверка реализации against specific expectations. Где живёт в проекте: через unit tests, regression tests and CI commands. Зачем нужен: отвечает на вопрос, выполняет ли код конкретный контракт. Что не означает: не означает доказательство общей полезности метода. Связь с общей моделью: verification supports validation but does not replace it.

Corpus. Буквально: корпус, набор файлов or samples для проверки. Где живёт в проекте: project uses benign, synthetic, mutated and dynamic samples. Зачем нужен: позволяет проверять behavior across many inputs. Что не означает: не означает весь мир Windows binaries. Связь с общей моделью: corpus gives measured evidence about current analyzer behavior.

Fuzzing. Буквально: фаззинг, автоматическая генерация or mutation inputs to find crashes and edge cases. Где живёт в проекте: project validation uses mutation fuzzing and libFuzzer-style thinking. Зачем нужен: ищет parser safety problems and unhandled weird inputs. Что не означает: не означает semantic proof of every finding. Связь с общей моделью: fuzzing protects util.c, pe.c, unwind.c and report handling from malformed input.

Regression. Буквально: регрессия, возврат старой ошибки after changes. Где живёт в проекте: regression tests keep previous fixed cases from breaking again. Зачем нужен: нужна because analyzer evolves. Что не означает: не означает medical regression or statistical regression here. Связь с общей моделью: regression testing preserves trust across versions.

Release gate. Буквально: контрольная точка перед выпуском. Где живёт в проекте: build/test/report/schema/corpus checks before release package. Зачем нужен: останавливает выпуск if important evidence is missing. Что не означает: не означает bureaucratic checkbox. Связь с общей моделью: release gate ties validation artifacts to version readiness.

Reading forward and reading backward: два направления проверки архитектуры

Архитектуру удобно читать в двух направлениях. Forward reading - чтение вперёд - начинается с файла: bytes, PE parsing, unwind decoding, findings, runtime evidence, reports. Это путь выполнения analyzer. Backward reading - чтение назад - начинается с final verdict in report and asks: почему ты это сказал? Покажи finding. Покажи evidence family. Покажи affected_count. Покажи representative_rva. Покажи section. Покажи RUNTIME_FUNCTION. Покажи UNWIND_INFO. Покажи dynamic event, если correlation claimed. Если report нельзя прочитать назад до bytes or runtime event, architecture failed.

Forward reading. Буквально: чтение вперёд, путь от input к result. Где живёт в проекте: describes normal pipeline execution. Зачем нужен: помогает понять how data transforms through modules. Что не означает: не доказывает correctness by itself. Связь с общей моделью: forward reading is useful for implementation reasoning.

Backward reading. Буквально: чтение назад, проверка результата через причины. Где живёт в проекте: starts from report and follows evidence back to source facts. Зачем нужен: помогает реверсеру trust but verify. Что не означает: не означает reverse execution of program. Связь с общей моделью: backward reading is how manual review audits analyzer claims.

Например, report says final_verdict requires_manual_review. Хорошее чтение назад не останавливается на этой фразе. Оно спрашивает: какой case_strength? Есть ли static_dynamic_correlation? Какие findings? Из каких families? Есть ли weakening_rules_applied? Dynamic run был enabled? ran? ok? timed_out? Есть ли handler hit or only API registration? Если runtime not run, почему report не называет case correlated? Такой стиль чтения превращает analyzer from black box into transparent assistant.

Black box. Буквально: чёрный ящик, система, которая выдаёт результат без объяснения причин. Где живёт в проекте: это то, чем проект не должен становиться. Зачем нужен: термин нужен как отрицательная модель дизайна. Что не означает: не означает literally black UI. Связь с общей моделью: EH/Unwind Analyzer должен быть evidence-transparent, not black-box classifier.

Transparent assistant. Буквально: прозрачный помощник, инструмент, который показывает основания своих выводов. Где живёт в проекте: идеальная роль analyzer in manual reverse engineering workflow. Зачем нужен: ускоряет работу, не заменяя мышление человека. Что не означает: не означает, что каждый вывод perfect. Связь с общей моделью: transparent assistant supports manual validation through traceable evidence.

Сквозная сцена 1: обычный boring binary

Представим обычный boring binary. Boring - скучный - здесь не оскорбление, а хороший признак. Такой файл собран нормальным compiler/linker flow, имеет PE32+ AMD64 headers, readable section table, Exception Directory, .pdata entries, .xdata records, mostly valid UNWIND_INFO, no strange handler concentration, no unusual writable/executable metadata placement. util.c безопасно читает bytes. pe.c builds PeImage. unwind.c fills UnwindSummary array. heuristics.c finds no strong findings. report.c writes low score, low suspicion, maybe evidence_level none or weak. Это не “доказательство безопасности”. Это означает: в пределах данного analyzer and его evidence model strong anomalies were not observed.

Такой сценарий важен, потому что проект не должен быть машиной тревоги. Если analyzer постоянно кричит, он бесполезен. Good defensive analyzer должен уметь скучать. Он должен честно говорить: metadata looks ordinary, no strong static anomalies, dynamic not requested or no runtime signals observed. Но даже здесь report должен сохранять provenance: file hash, tool version, PE facts, counts. Скучный вывод тоже должен быть проверяемым.

Provenance. Буквально: происхождение результата, сведения о том, откуда и при каких условиях он получен. Где живёт в проекте: ReportMetadata includes input identity, tool version, schema version and related metadata. Зачем нужен: позволяет воспроизвести or compare analysis. Что не означает: не означает гарантию подлинности файла beyond recorded hash/context. Связь с общей моделью: provenance connects report artifact to analyzed input and analyzer version.

Сквозная сцена 2: повреждённая .xdata and damaged handler route

Теперь другой образец. PE headers читаются, sections выглядят допустимо, Exception Directory указывает на .pdata, но часть RUNTIME_FUNCTION entries ведёт к unreadable or malformed .xdata. unwind.c не должен молча пропускать это. Он должен отметить parse_error, valid=false for affected summary, us_ok false. heuristics.c получает evidence in unwind_integrity family. Report показывает finding, affected_count, representative_rva, maybe requires_manual_review. Если handler field appears but target section is unknown or writable, появляется другой слой evidence. Dynamic collector может не подтвердить этот route, потому что запуск не дошёл до exception path. Тогда case remains static or unconfirmed.

Здесь видно, почему финальная архитектура осторожная. Static finding говорит: metadata structure suspicious or broken. Оно не говорит: handler executed. Dynamic not observed не стирает static issue. А если dynamic session timed out, отсутствие handler hit вообще почти ничего не доказывает. Реверсер открывает Ghidra, идёт к representative_rva, смотрит function range, comments, bookmarks, raw .xdata bytes, section permissions, handler target. Manual review не начинается с нуля: analyzer уже дал карту маршрута.

Сквозная сцена 3: runtime-only activity and dynamic function tables

Третий сценарий показывает обратную сторону. Static analysis может быть спокойным, но dynamic collector видит RtlAddFunctionTable or RtlInstallFunctionTableCallback. Эти APIs нужны Windows x64 for dynamically generated code to participate in stack unwinding. Это может быть JIT, runtime code generation, protection layer, plugin system or suspicious loader. Сам API call не является приговором. Но it is runtime evidence that the program creates or registers function table information outside ordinary file .pdata. Если report формулирует это как suspicious without context, он ошибается. Если он вообще не показывает event, он тоже ошибается. Правильная формулировка: runtime function table activity observed; static file metadata may not describe all unwindable code that exists during execution.

Runtime function table. Буквально: таблица функций, зарегистрированная during runtime for generated or dynamic code. Где живёт в проекте: dynamic_win.c observes calls to RtlAddFunctionTable/RtlInstallFunctionTableCallback. Зачем нужен: позволяет Windows unwind generated code. Что не означает: не означает автоматически obfuscation or malware. Связь с общей моделью: runtime function table activity explains why static .pdata may be incomplete for live process.

Generated code. Буквально: сгенерированный код, instructions produced or registered during execution rather than fixed as ordinary file code. Где живёт в проекте: may appear through JIT or dynamic loaders and function table APIs. Зачем нужен: важен because unwind metadata may be created at runtime. Что не означает: не означает inherently malicious code. Связь с общей моделью: generated code is one reason dynamic validation complements static analysis.

Failure path: нормальная архитектура объясняет не только успех

Слово failure path - путь отказа - звучит как тема для exception handling in program code, но в архитектуре analyzer оно шире. Failure path отвечает: что происходит, когда файл пустой, header broken, RVA cannot map to file offset, UNWIND_INFO truncated, report cannot be written atomically, target process cannot be created, breakpoint cannot be armed, JSON schema validation fails? Слабая программа в таких местах либо падает, либо молчит, либо выдаёт misleading success. Хороший analyzer сохраняет failure as data. Он должен отличать “нет признака” от “не удалось проверить”. Это одна из самых важных мыслей всей книги.

Failure path. Буквально: путь отказа, заранее продуманное поведение when operation cannot complete normally. Где живёт в проекте: виден в parser diagnostics, parse_error, dynamic ok/ran/timed_out flags, report write errors. Зачем нужен: позволяет не смешивать absence of evidence and evidence of absence. Что не означает: не означает catastrophic crash обязательно. Связь с общей моделью: failure path preserves honesty of assessment.

Diagnostic. Буквально: диагностическое сообщение, structured or textual explanation of a problem or condition. Где живёт в проекте: appears in PE validation, unwind parse errors, self-check details and reports. Зачем нужен: помогает понять why layer could not proceed or what it observed. Что не означает: не должен быть мусорным логом without semantics. Связь с общей моделью: diagnostic is part of audit trail.

Audit trail. Буквально: след проверки, цепочка сведений, по которой можно восстановить происхождение вывода. Где живёт в проекте: report metadata, findings, counters, events and validation artifacts form audit trail. Зачем нужен: делает result reviewable after the run. Что не означает: не означает legal audit by itself. Связь с общей моделью: audit trail lets a reverser challenge or confirm analyzer claims.

Extension point: как развивать проект, не ломая его смысл

Последний архитектурный термин - extension point, точка расширения. Extension - расширение. Point - место. Это место в architecture, куда можно добавить новую способность without destroying existing contracts. Например, richer APX/unwind v3 support belongs in unwind.c and schema/report extensions. Remote parsing of RtlAddFunctionTable entries belongs in dynamic_win.c, but its results should enter DynamicReport as structured runtime evidence. JSON importer for Ghidra belongs near integration layer, not inside PE parser. New finding family belongs in heuristics.c and evidence matrix, with tests, weights, weakening rules and report schema updates. Если extension point chosen wrong, проект начинает смешивать layers.

> Extension point. Буквально: точка расширения, место где систему можно развивать без разрушения существующих границ. Где живёт в проекте: in this project: decoder extensions, heuristic family additions, dynamic event enrichment, report schema evolution, Ghidra importer upgrades. Зачем нужен: позволяет улучшать analyzer while preserving contracts. Что не означает: не означает “можно добавить код куда угодно”. Связь с общей моделью: extension point protects architecture from feature-driven chaos.
>
> Contract. Буквально: контракт, явно ожидаемое соглашение между слоями or tools. Где живёт в проекте: between util.c and pe.c, pe.c and unwind.c, unwind.c and heuristics.c, report.c and JSON schema, report.c and Ghidra importer. Зачем нужен: позволяет менять implementation without changing meaning silently. Что не означает: не означает юридический договор здесь. Связь с общей моделью: contract is how architecture stays stable as project grows.

Например, если будущая версия проекта начнёт глубже читать dynamic function table entries from target memory, это не должно внезапно изменить meaning of static score. Новая runtime evidence должна иметь свой event type, limits, failure path and correlation rules. Если HTML станет красивее, JSON schema should still preserve stable fields. Если Ghidra importer начнёт ставить labels or colored bookmarks, он не должен переписывать user work silently. Развитие проекта должно добавлять видимость, not hidden authority.

Финальная архитектурная формула

Теперь всю книгу можно собрать в одну формулу, но эта формула уже не будет пустой. Function gives us RIP and stack situation. Windows x64 ABI explains RSP, nonvolatile registers, prolog and epilog discipline. PE Format gives module layout, sections, RVA and data directories. Exception Directory leads to .pdata. RUNTIME_FUNCTION maps code ranges to .xdata. UNWIND_INFO describes how to restore caller context. RtlVirtualUnwind uses that metadata at runtime. Exception route may involve debugger first chance, VEH, frame-based SEH, VCH, second chance and unhandled path. EH/Unwind Analyzer maps this official machinery into

a layered tool: util.c reads safely, pe.c builds image map, unwind.c decodes metadata, heuristics.c interprets evidence, dynamic_win.c observes selected runtime signals, report.c preserves meaning, Ghidra importer brings hints to manual workspace, validation keeps the whole chain honest.

Если читатель вынесет из главы 11 только список файлов, глава провалилась. Правильный вывод другой: каждый модуль проекта является не просто C-файлом, а точкой ответственности в общей модели знания. util.c отвечает за безопасный доступ. pe.c отвечает за image map. unwind.c отвечает за metadata decoding. heuristics.c отвечает за осторожное evidence-based assessment. dynamic_win.c отвечает за limited runtime observation. report.c отвечает за faithful serialization. Ghidra importer отвечает за перенос hints into manual work. Validation отвечает за то, чтобы эти promises проверялись. Так теория книги перестаёт быть набором терминов and becomes an instrument.

Именно поэтому финальная версия книги не должна бросаться английскими словами как ярлыками. Каждый такой термин - не украшение, а сжатая модель. Если термин не объяснён, он превращается в шум. Если объяснён через место в системе, через роль, через границы and through common misunderstanding, он становится инструментом мышления. В хорошей reverse engineering книге architecture is not the last diagram. Architecture is the moment when all previous mechanisms become one disciplined way of asking: what exactly do we know, from which layer do we know it, and how can we verify it?

Приложение A. Старт финальной полировки книги: редакционный проход по структуре, терминам и проверяемости

После переписывания 11.11 книга переходит из режима “добавляем новые главы” в режим final polishing. Final polishing - финальная полировка - не означает “поставить красивую обложку и забыть”. Для технической книги это отдельный инженерно-редакторский этап. Его цель - сделать так, чтобы читатель не проваливался в термины, не терял цепочку, не натыкался на противоречивые названия, не видел устаревшие слова из ранних черновиков and could navigate from concept to project implementation.

> Final polishing. Буквально: финальная полировка, последний редакционный проход перед финальным черновиком. Где живёт в проекте: applies to the whole PDF book after main content is complete. Зачем нужен: убирает терминологические разрывы, слабые формулировки, повторы and navigation problems. Что не означает: не означает переписывание approved chapters without reason. Связь с общей моделью: final polishing turns working_vXX into final_draft.
>
> Final draft. Буквально: финальный черновик, версия книги, которую уже можно читать целиком and review as one artifact. Где живёт в проекте: будущая итоговая сборка после content, term and layout passes. Зачем нужен: нужна, чтобы перестать плодить рабочие версии without editorial checkpoint. Что не означает: не означает окончательную публикацию без вычитки. Связь с общей моделью: final draft is the first whole-book deliverable.

Первый проход полировки: терминологический контроль

Terminology pass - терминологический проход - проверяет не только перевод слов. Он проверяет, объяснён ли термин при первом появлении, не используется ли один и тот же механизм под разными именами, не остались ли старые placeholder words, не путаются ли static evidence and runtime evidence, не называется ли confidence вероятностью, не называется ли finding приговором, не называется ли dynamic validation доказательством отсутствия behavior. Этот проход особенно важен для такой книги, потому что Windows x64 unwind построен на компактных терминах: RIP, RSP, RVA, VA, .pdata, .xdata, RUNTIME_FUNCTION, UNWIND_INFO, CONTEXT, handler, dispatcher, debugger, first chance, second chance. Каждый из них должен иметь человеческое объяснение.

> Terminology pass. Буквально: проход по терминологии, редакционная проверка всех терминов and their explanations. Где живёт в проекте: applies to English and project terms throughout the book. Зачем нужен: делает книгу учебником, а не набором ярлыков. Что не означает: не означает

механический glossary only. Связь с общей моделью: terminology pass ensures every term connects to the mental model.

Для этой книги правило полировки будет таким: если термин важен для понимания механизма, он должен объясняться в момент появления through six angles: literal meaning, Russian meaning, location in Windows or project, purpose, what it does not mean, and connection to previous chain. Например, RVA нельзя объяснить только как “relative virtual address”. Нужно сказать: это адрес относительно image base, он живёт в PE model, он не является file offset, он удобен для reports and Ghidra, он связывает RUNTIME_FUNCTION fields with loaded module base. Без такого объяснения термин остаётся шумом.

Второй проход полировки: структурная навигация

Structure pass - структурный проход - проверяет, не потерялась ли книга как маршрут. В начале книги читатель учится видеть function, call, return address, RSP and stack frame. Потом nonvolatile registers and calling convention. Потом PE, sections, RVA. Потом .pdata, RUNTIME_FUNCTION, .xdata, UNWIND_INFO. Потом exception route. Потом analyzer architecture. Если где-то раздел внезапно превращается в перечень, надо либо переписать его, либо добавить переходный paragraph, который объясняет, почему следующий механизм вообще появляется.

Structure pass. Буквально: структурный проход, проверка порядка глав and переходов между идеями. Где живёт в проекте: applies to chapter titles, section openings, endings and cross-references. Зачем нужен: помогает книге читаться как lecture, not FAQ. Что не означает: не означает перестановку всех approved chapters без необходимости. Связь с общей моделью: structure pass preserves the learning path from CPU state to analyzer report.

Особое место в structural polishing занимает repeated summary problem. Сводки полезны, но если каждая глава заново говорит “static is not dynamic” без нового контекста, книга начинает буксовать. Финальная полировка должна оставить повтор там, где он укрепляет mental model, и убрать повтор там, где он только увеличивает page count. Это не сокращение ради сокращения. Это защита внимания читателя.

Третий проход полировки: проверяемость утверждений

Evidence pass - проход по доказательной базе - проверяет, что сильные технические утверждения связаны либо с official documentation, либо с project source, либо с явно обозначенным inference. Official documentation - официальная документация - в этой книге прежде всего Microsoft PE Format, Microsoft x64 exception handling, Debug API documentation, and Ghidra API documentation for integration. Project source - исходный код проекта - это actual implementation: util.c, pe.c, unwind.c, heuristics.c, dynamic_win.c, report.c, scripts and docs. Inference - вывод автора - должен быть помечен как объяснение or

interpretation, а не выдан как факт документации.

Evidence pass. Буквально: проход по свидетельствам, проверка что утверждения имеют опору. Где живёт в проекте: applies to official docs references, project architecture claims and report semantics. Зачем нужен: защищает книгу от уверенно звучащей фантазии. Что не означает: не означает, что every sentence needs a citation. Связь с общей моделью: evidence pass separates documented facts from project policy and teaching interpretation.

Official documentation. Буквально: официальная документация производителя or проекта. Где живёт в проекте: Microsoft docs for PE/x64/debug APIs and Ghidra docs for importer environment. Зачем нужен: даёт базовый контракт платформы. Что не означает: не объясняет automatically how to reverse suspicious binaries. Связь с общей моделью: official docs are foundation; analyzer architecture is built on top.

Project source. Буквально: исходный код проекта, фактическая реализация tool behavior. Где живёт в проекте: actual C and Python files of EH/Unwind Analyzer. Зачем нужен: показывает, что analyzer really does, not what book wishes it did. Что не означает: не всегда explains itself pedagogically. Связь с общей моделью: project source grounds architecture chapters in reality.

Четвёртый проход полировки: артефакты и формат PDF

Layout pass - проход по верстке - кажется менее интеллектуальным, но для PDF-книги он критичен. Нужно проверить, что страницы рендерятся без clipped text, broken glyphs, пустых хвостовых страниц, повторяющихся footers, съехавших code blocks. Нужно убедиться, что старые approved pages не менялись при append. Для этого у нас уже используется page hash check and rendering check. Hash - хэш - цифровой отпечаток bytes страницы or rendered content; здесь он нужен не как security proof, а как practical guard against accidental edits.

Layout pass. Буквально: проход по верстке, проверка внешнего отображения PDF. Где живёт в проекте: applies to rendered pages, footers, headings, code blocks and page continuity. Зачем нужен: помогает убедиться, что текст реально читается. Что не означает: не означает косметику вместо содержания. Связь с общей моделью: layout pass protects the book artifact.

Hash check. Буквально: проверка хэша, сравнение цифрового отпечатка before/after. Где живёт в проекте: used to confirm previous approved page did not change. Зачем нужен: защищает approved content during append workflow. Что не означает: не доказывает semantic correctness of new text. Связь с общей моделью: hash check is part of PDF production audit trail.

Следующий практический шаг после этого файла - собрать final_draft_v1: пройти по headings, убедиться, что рабочие версии v54/v55/v58/v61/v74, которые были отвергнуты or rewritten, не считаются актуальными, проверить последние page ranges, создать единый список sections, and decide whether

glossary should be appended as one large reference chapter. Глоссарий особенно важен, потому что пользовательская обратная связь уже показала: если термин не разжёван, даже правильная глава ощущается как конспект.

Что именно теперь считается задачей финальной полировки

Финальная полировка дальше должна идти не как новая техническая глава, а как редакционный workflow. Workflow - рабочий процесс - это последовательность действий with checks and outputs. Первый output - current polished book draft. Второй output - terminology checklist. Третий output - structure audit. Четвёртый output - final PDF with stable filename. Пятый output - optional glossary appendix. В этом процессе нельзя “быстро причесать” книгу автоматически, потому что её ценность именно в глубине объяснений. Полировка должна усиливать повествовательность, а не вырезать её.

> Workflow. Буквально: рабочий процесс, последовательность действий with expected outputs. Где живёт в проекте: final polishing workflow includes terminology, structure, evidence and layout passes. Зачем нужен: помогает не превращать финализацию в хаос. Что не означает: не означает бюрократию ради бюрократии. Связь с общей моделью: workflow makes final book production reproducible.

На этом этапе содержательная проектная глава закрыта заново и более аккуратно. Следующий файл должен быть уже не “ещё одна версия с новой технической темой”, а редакционная сборка: либо final_draft_v1 with glossary, либо separate polish audit plus final PDF. Это правильная смена режима: книга больше не должна расширяться бесконечно, она должна становиться цельной.

Глава 12. Каталог static findings EH/Unwind Analyzer: 27 признаков, 5 evidence families и границы вывода

Эта финальная версия главы 12 заменяет разрозненные рабочие вставки предыдущей редакции. Здесь используется терминология, сверенная с проектом: static analyzer, heuristic/evidence engine, findings, evidence families, assessment, reports, DynamicReport, Windows dynamic collector и dynamic validation. Старые учебные названия Inspector/Protector больше не являются нормативными названиями компонентов проекта.

Главная идея главы: 27 findings не являются списком признаков вредоносности вообще. Это static evidence catalog - каталог статически наблюдаемых свидетельств, которые описывают нарушения, искажения или необычные patterns вокруг Windows x64 PE exception/unwind metadata. Если сказать проще: анализатор не пытается угадать намерения автора файла. Он проверяет, насколько корректно и правдоподобно PE-файл описывает свою exception/unwind-модель.

Слово finding означает конкретную находку. Это не настроение анализатора и не общий вывод вроде "файл странный". Finding должен указывать, что именно обнаружено, где это обнаружено, почему это связано с Windows x64 unwind model, как это проверить руками и какие benign alternatives возможны.

12.1. Почему здесь именно static findings

Static analysis - это анализ файла без запуска. Static analyzer читает PE как недоверенные bytes: проверяет headers, section table, RVA mapping, Exception Directory, RUNTIME_FUNCTION records, UNWIND_INFO и связи между ними. Он не знает, какие dynamic function tables будут зарегистрированы в процессе, какой code будет создан JIT-ом, какие исключения реально произойдут и какие handler-ы реально сработают.

Поэтому 27 findings описывают только тот слой, который виден в PE-файле. Это важное ограничение, но не слабость. В Windows x64 exception/unwind значительная часть маршрута действительно объявлена в PE image: Exception Directory ведет к .pdata, .pdata содержит RUNTIME_FUNCTION, RUNTIME_FUNCTION ведет к .xdata, .xdata содержит UNWIND_INFO и данные handler-а. Если этот объявленный маршрут поврежден или необычен, это уже самостоятельное evidence.

Dynamic validation и Windows dynamic collector находятся рядом, но не внутри этих 27 static findings. DynamicReport отвечает на другой вопрос: что реально произошло при выполнении тестовых сценариев, какие handlers были зарегистрированы или вызваны, какие события были замечены runtime-слоем. Static findings говорят "что заявлено в файле". DynamicReport говорит "что наблюдалось во время выполнения".

12.2. Пять evidence families проекта

Evidence family - это семейство свидетельств. Оно нужно не для красоты отчета, а для мышления. Один finding показывает конкретную проблему. Family показывает, на каком уровне модели находится эта проблема.

В проекте используются пять evidence families:

Family	Смысл	Что проверяет
table_integrity	целостность PE-карты и таблицы функций	sections, Exception Directory, RUNTIME_FUNCTION ranges, sorting, overlaps

unwind_integrity	читаемость и правдоподобность unwind-рецепта	UNWIND_INFO parser, flags, codes, anomalies
metadata_placement	размещение metadata и handler-адресов	section permissions, writable/executable/nonexec placement
unwind_reuse	концентрация ссылок на один unwind-блок	moderate/strong reuse patterns
handler_dominance	доминирование одного handler-а	один handler обслуживает слишком много функций

Эти пять families - не то же самое, что техническая формула 6+7+2+3+5+4. Family отвечает на вопрос "как группировать evidence в отчете". Формула 6+7+2+3+5+4 отвечает на вопрос "из каких технических групп получились 27 public finding codes".

12.3. Откуда берется число 27: техническая формула 6+7+2+3+5+4

```
6  PE layout findings
7  RUNTIME_FUNCTION findings
2  UNWIND_INFO integrity findings
3  reuse / handler statistics findings
5  placement findings
4  combined findings
--
27 public finding codes
```

Это не магическое число. Оно получилось не потому, что "так красиво". Оно получается из маршрута Windows x64 exception/unwind metadata. Чтобы статически повлиять на этот маршрут, можно исказить PE-карту, Exception Directory, RUNTIME_FUNCTION table, UNWIND_INFO, placement metadata/handler-a или массовые связи между множеством функций и одинаковыми metadata/handler targets.

Если сделать findings меньше, отчет станет слишком грубым: "проблемы с unwind" не говорит, что именно проверить. Если сделать findings больше без строгого фильтра, отчет станет шумным: разные названия будут дублировать один и тот же факт и искусственно раздувать score. Поэтому проект использует public catalog из 27 конкретных кодов.

12.4. Семь критериев включения finding-a

Отдельный technical invariant или class anomaly

Finding должен проверять отдельное техническое свойство, а не расплывчатое ощущение подозрительности.

Deterministic computation

Один и тот же файл при одинаковых настройках должен давать тот же результат.

Localizable or quantifiable

Находку можно локализовать по RVA/section/index или выразить числом/долей/счетчиком.

Clear manual validation workflow

Реверсер должен суметь проверить finding руками в PE viewer, Ghidra, debugger или через отчет.

Unit/regression test

Для finding-a должен быть тест или регрессионный сценарий, иначе он слишком хрупкий.

No full duplication

Finding не должен полностью повторять другой finding, иначе score будет нечестным.

Explicit boundaries and benign alternatives

Для finding-а должны быть указаны границы вывода и нормальные альтернативные объяснения.

12.5. Полная матрица 27 findings

Finding code	Группа	Краткий смысл
pe_section_raw_oob	PE layout	raw-диапазон секции выходит за файл
pe_section_virtual_oob	PE layout	виртуальный диапазон секции выходит за SizeOfImage
pe_section_raw_overlap	PE layout	raw-диапазоны секций перекрываются
pe_section_virtual_overlap	PE layout	virtual-диапазоны секций перекрываются
pe_section_misaligned	PE layout	секция нарушает ожидаемое выравнивание
pe_exception_directory_oob	PE layout	Exception Directory указывает за допустимые границы
no_runtime_functions	RUNTIME_FUNCTION	нет доступных записей таблицы функций
invalid_ranges	RUNTIME_FUNCTION	BeginAddress/EndAddress образуют невозможные диапазоны
rf_oob	RUNTIME_FUNCTION	запись функции выходит за допустимую область образа
unwind_rva_oob	RUNTIME_FUNCTION	UnwindInfoAddress ведет за пределы читаемого образа
rf_unsorted	RUNTIME_FUNCTION	таблица не отсортирована по BeginAddress
rf_overlaps	RUNTIME_FUNCTION	диапазоны функций пересекаются
duplicate_begin	RUNTIME_FUNCTION	несколько записей имеют одинаковый BeginAddress
unwind_parse_fail	UNWIND_INFO	структуру UNWIND_INFO нельзя корректно разобрать
unwind_anomalies	UNWIND_INFO	UNWIND_INFO формально разобран, но содержит необычные свойства
unwind_reuse_moderate	statistics	умеренное переиспользование одного unwind-блока
unwind_reuse_strong	statistics	сильная концентрация ссылок на один unwind-блок
handler_dominance	statistics	один handler доминирует над большим числом функций
unwind_in_exec_section	placement	unwind metadata лежит в executable-секции
unwind_unusual_section	placement	unwind metadata лежит в нетипичной секции
unwind_writable_section	placement	unwind metadata лежит в writable-секции
handler_nonexec_section	placement	handler указывает в неисполняемую секцию
handler_writable_section	placement	handler расположен в writable-секции
placement_exec_nonexec_handler_combo	combined	placement-сигналы совместно усиливают риск
placement_integrity_combo	combined	placement и integrity findings подтверждают друг друга
reuse_dominance_combo	combined	reuse и handler dominance складываются в единый pattern
writable_metadata_handler_combo	combined	writable metadata и writable handler создают усиленный сигнал

12.6. Как читать findings по зонам, а не как список ошибок

Правильный порядок чтения отчета начинается с table_integrity. Если PE-карта повреждена, нельзя уверенно читать .pdata и .xdata. Затем читается unwind_integrity: можно ли вообще декодировать рецепт восстановления стека. После этого смотрится metadata_placement: где physically лежат metadata и handler targets. Только потом имеет смысл смотреть statistical patterns - reuse и dominance.

Такой порядок защищает от ложной уверенности. Например, handler_writable_section звучит резко, но если PE section table сама нарушена, сначала нужно понять, можем ли мы доверять section permissions. Или наоборот: один unwind_reuse_moderate сам по себе может быть benign artifact, но если он идет вместе с handler_dominance и writable metadata, картина становится значительно сильнее.

12.7. Подробный разбор каждого finding-а

pe_section_raw_oob

Finding pe_section_raw_oob относится к группе PE layout. Его смысл: raw-диапазон секции выходит за файл. Технически он отвечает на вопрос: нарушен ли конкретный контракт между PE image и Windows x64 unwind route? Если да, static analyzer не делает вывод "malware" автоматически, а формирует finding, который должен быть виден в отчете и проверяем вручную. Как проверять руками: открыть PE headers и section table, найти соответствующий RVA или index, проверить границы section/file/image, затем связать находку с Exception Directory, RUNTIME_FUNCTION или UNWIND_INFO. Для placement-сигналов дополнительно проверяются section characteristics: read, write, execute. Для statistical-сигналов проверяется счетчик affected records, доля reuse/dominance и representative RVA. Benign alternatives возможны. Это может быть packed binary, нестандартный linker, ручная сборка PE, JIT-like сценарий, compiler optimization, hot/cold splitting, chained unwind info или артефакт поврежденного sample. Поэтому finding - это evidence для assessment, а не приговор.

pe_section_virtual_oob

Finding pe_section_virtual_oob относится к группе PE layout. Его смысл: виртуальный диапазон секции выходит за SizeOfImage. Технически он отвечает на вопрос: нарушен ли конкретный контракт между PE image и Windows x64 unwind route? Если да, static analyzer не делает вывод "malware" автоматически, а формирует finding, который должен быть виден в отчете и проверяем вручную. Как проверять руками: открыть PE headers и section table, найти соответствующий RVA или index, проверить границы section/file/image, затем связать находку с Exception Directory, RUNTIME_FUNCTION или UNWIND_INFO. Для placement-сигналов дополнительно проверяются section characteristics: read, write, execute. Для statistical-сигналов проверяется счетчик affected records, доля reuse/dominance и representative RVA. Benign alternatives возможны. Это может быть packed binary, нестандартный linker, ручная сборка PE, JIT-like сценарий, compiler optimization, hot/cold splitting, chained unwind info или артефакт поврежденного sample. Поэтому finding - это evidence для assessment, а не приговор.

pe_section_raw_overlap

Finding pe_section_raw_overlap относится к группе PE layout. Его смысл: raw-диапазоны секций перекрываются. Технически он отвечает на вопрос: нарушен ли конкретный контракт между PE image и Windows x64 unwind route? Если да, static analyzer не делает вывод "malware" автоматически, а формирует finding, который должен быть виден в отчете и проверяем вручную. Как проверять руками: открыть PE headers и section table, найти соответствующий RVA или index, проверить границы section/file/image, затем связать находку с Exception Directory, RUNTIME_FUNCTION или UNWIND_INFO. Для placement-сигналов дополнительно проверяются section characteristics: read, write, execute. Для statistical-сигналов проверяется счетчик affected records, доля reuse/dominance и representative RVA. Benign alternatives возможны. Это может быть packed binary, нестандартный linker, ручная сборка PE, JIT-like сценарий, compiler optimization, hot/cold splitting, chained unwind info или артефакт поврежденного sample. Поэтому finding - это evidence для assessment, а не приговор.

pe_section_virtual_overlap

Finding pe_section_virtual_overlap относится к группе PE layout. Его смысл: virtual-диапазоны секций перекрываются. Технически он отвечает на вопрос: нарушен ли конкретный контракт между PE image и Windows x64 unwind route? Если да, static analyzer не делает вывод "malware" автоматически, а формирует finding, который должен быть виден в отчете и проверяем вручную. Как проверять руками: открыть PE headers и section table, найти соответствующий RVA или index, проверить границы section/file/image, затем связать находку с Exception Directory, RUNTIME_FUNCTION или UNWIND_INFO. Для placement-сигналов дополнительно проверяются section characteristics: read, write, execute. Для statistical-сигналов проверяется счетчик affected records, доля reuse/dominance и representative RVA. Benign alternatives возможны. Это может быть packed binary, нестандартный linker, ручная сборка PE, JIT-like сценарий, compiler optimization, hot/cold splitting, chained unwind info или артефакт поврежденного sample. Поэтому finding - это evidence для assessment, а не приговор.

pe_section_misaligned

Finding pe_section_misaligned относится к группе PE layout. Его смысл: секция нарушает ожидаемое выравнивание. Технически он отвечает на вопрос: нарушен ли конкретный контракт между PE image и Windows x64 unwind route? Если да, static analyzer не делает вывод "malware" автоматически, а формирует finding, который должен быть виден в отчете и проверяем вручную. Как проверять руками: открыть PE headers и section table, найти соответствующий RVA или index, проверить границы section/file/image, затем связать находку с Exception Directory, RUNTIME_FUNCTION или UNWIND_INFO. Для placement-сигналов дополнительно проверяются section characteristics: read, write, execute. Для statistical-сигналов проверяется счетчик affected records, доля reuse/dominance и representative RVA. Benign alternatives возможны. Это может быть packed binary, нестандартный linker, ручная сборка PE, JIT-like сценарий, compiler optimization, hot/cold splitting, chained unwind info или артефакт поврежденного sample. Поэтому finding - это evidence для assessment, а не приговор.

pe_exception_directory_oob

Finding pe_exception_directory_oob относится к группе PE layout. Его смысл: Exception Directory указывает за допустимые границы. Технически он отвечает на вопрос: нарушен ли конкретный контракт между PE image и Windows x64 unwind route? Если да, static analyzer не делает вывод "malware" автоматически, а формирует finding, который должен быть виден в отчете и проверяем вручную. Как проверять руками: открыть PE headers и section table, найти соответствующий RVA или index, проверить границы section/file/image, затем связать находку с Exception Directory, RUNTIME_FUNCTION или UNWIND_INFO. Для placement-сигналов дополнительно проверяются section characteristics: read, write, execute. Для statistical-сигналов проверяется счетчик affected records, доля reuse/dominance и representative RVA. Benign alternatives возможны. Это может быть packed binary, нестандартный linker, ручная сборка PE, JIT-like сценарий, compiler optimization, hot/cold splitting, chained unwind info или артефакт поврежденного sample. Поэтому finding - это evidence для assessment, а не приговор.

no_runtime_functions

Finding no_runtime_functions относится к группе RUNTIME_FUNCTION. Его смысл: нет доступных записей таблицы функций. Технически он отвечает на вопрос: нарушен ли конкретный контракт между PE image и Windows x64 unwind route? Если да, static analyzer не делает вывод "malware" автоматически, а формирует finding, который должен быть виден в отчете и проверяем вручную. Как проверять руками: открыть PE headers и section table, найти соответствующий RVA или index, проверить границы section/file/image, затем связать находку с Exception Directory, RUNTIME_FUNCTION или UNWIND_INFO. Для placement-сигналов дополнительно проверяются section characteristics: read, write, execute. Для statistical-сигналов проверяется счетчик affected records, доля reuse/dominance и representative RVA. Benign alternatives возможны. Это может быть packed binary, нестандартный linker, ручная сборка PE, JIT-like сценарий, compiler optimization, hot/cold splitting, chained unwind info или артефакт поврежденного sample. Поэтому finding - это evidence для assessment, а не приговор.

invalid_ranges

Finding invalid_ranges относится к группе RUNTIME_FUNCTION. Его смысл: BeginAddress/EndAddress образуют невозможные диапазоны. Технически он отвечает на вопрос: нарушен ли конкретный контракт между PE image и Windows x64 unwind route? Если да, static analyzer не делает вывод "malware" автоматически, а формирует finding, который должен быть виден в отчете и проверяем вручную. Как проверять руками: открыть PE headers и section table, найти соответствующий RVA или index, проверить границы section/file/image, затем связать находку с Exception Directory, RUNTIME_FUNCTION или UNWIND_INFO. Для placement-сигналов дополнительно проверяются section characteristics: read, write, execute. Для statistical-сигналов проверяется счетчик affected records, доля reuse/dominance и representative RVA. Benign alternatives возможны. Это может быть packed binary, нестандартный linker, ручная сборка PE, JIT-like сценарий, compiler optimization, hot/cold splitting, chained unwind info или артефакт поврежденного sample. Поэтому finding - это evidence для assessment, а не приговор.

rf_oob

Finding rf_oob относится к группе RUNTIME_FUNCTION. Его смысл: запись функции выходит за допустимую область образа. Технически он отвечает на вопрос: нарушен ли конкретный контракт между PE image и Windows x64 unwind route? Если да, static analyzer не делает вывод "malware" автоматически, а формирует finding, который должен быть виден в отчете и проверяем вручную. Как проверять руками: открыть PE headers и section table, найти соответствующий RVA или index, проверить границы section/file/image, затем связать находку с Exception Directory, RUNTIME_FUNCTION или UNWIND_INFO. Для placement-сигналов дополнительно проверяются section characteristics: read, write, execute. Для statistical-сигналов проверяется счетчик affected records, доля reuse/dominance и representative RVA. Benign alternatives возможны. Это может быть packed binary, нестандартный linker, ручная сборка PE, JIT-like сценарий, compiler optimization, hot/cold splitting, chained unwind info или артефакт поврежденного sample. Поэтому finding - это evidence для assessment, а не приговор.

unwind_rva_oob

Finding unwind_rva_oob относится к группе RUNTIME_FUNCTION. Его смысл: UnwindInfoAddress ведет за пределы читаемого образа. Технически он отвечает на вопрос: нарушен ли конкретный контракт между PE image и Windows x64 unwind route? Если да, static analyzer не делает вывод "malware" автоматически, а формирует finding, который должен быть виден в отчете и проверяем вручную. Как проверять руками: открыть PE headers и section table, найти соответствующий RVA или index, проверить границы section/file/image, затем связать находку с Exception Directory, RUNTIME_FUNCTION или UNWIND_INFO. Для placement-сигналов дополнительно проверяются section characteristics: read, write, execute. Для statistical-сигналов проверяется счетчик affected records, доля reuse/dominance и representative RVA. Benign alternatives возможны. Это может быть packed binary, нестандартный linker, ручная сборка PE, JIT-like сценарий, compiler optimization, hot/cold splitting, chained unwind info или артефакт поврежденного sample. Поэтому finding - это evidence для assessment, а не приговор.

rf_unsorted

Finding rf_unsorted относится к группе RUNTIME_FUNCTION. Его смысл: таблица не отсортирована по BeginAddress. Технически он отвечает на вопрос: нарушен ли конкретный контракт между PE image и Windows x64 unwind route? Если да, static analyzer не делает вывод "malware" автоматически, а формирует finding, который должен быть виден в отчете и проверяем вручную. Как проверять руками: открыть PE headers и section table, найти соответствующий RVA или index, проверить границы section/file/image, затем связать находку с Exception Directory, RUNTIME_FUNCTION или UNWIND_INFO. Для placement-сигналов дополнительно проверяются section characteristics: read, write, execute. Для statistical-сигналов проверяется счетчик affected records, доля reuse/dominance и representative RVA. Benign alternatives возможны. Это может быть packed binary, нестандартный linker, ручная сборка PE, JIT-like сценарий, compiler optimization, hot/cold splitting, chained unwind info или артефакт поврежденного sample. Поэтому finding - это evidence для assessment, а не приговор.

rf_overlaps

Finding rf_overlaps относится к группе RUNTIME_FUNCTION. Его смысл: диапазоны функций пересекаются. Технически он отвечает на вопрос: нарушен ли конкретный контракт между PE image и Windows x64 unwind route? Если да, static analyzer не делает вывод "malware" автоматически, а формирует finding, который должен быть виден в отчете и проверяем вручную. Как проверять руками: открыть PE headers и section table, найти соответствующий RVA или index, проверить границы section/file/image, затем связать находку с Exception Directory, RUNTIME_FUNCTION или UNWIND_INFO. Для placement-сигналов дополнительно проверяются section characteristics: read, write, execute. Для statistical-сигналов проверяется счетчик affected records, доля reuse/dominance и representative RVA. Benign alternatives возможны. Это может быть packed binary, нестандартный linker, ручная сборка PE, JIT-like сценарий, compiler optimization, hot/cold splitting, chained unwind info или артефакт поврежденного sample. Поэтому finding - это evidence для assessment, а не приговор.

duplicate_begin

Finding duplicate_begin относится к группе RUNTIME_FUNCTION. Его смысл: несколько записей имеют одинаковый BeginAddress. Технически он отвечает на вопрос: нарушен ли конкретный контракт между PE image и Windows x64 unwind route? Если да, static analyzer не делает вывод "malware" автоматически, а формирует finding, который должен быть виден в отчете и проверяем вручную. Как проверять руками: открыть PE headers и section table, найти соответствующий RVA или index, проверить границы section/file/image, затем связать находку с Exception Directory, RUNTIME_FUNCTION или UNWIND_INFO. Для placement-сигналов дополнительно проверяются section characteristics: read, write, execute. Для statistical-сигналов проверяется счетчик affected records, доля reuse/dominance и representative RVA. Benign alternatives возможны. Это может быть packed binary, нестандартный linker, ручная сборка PE, JIT-like сценарий, compiler optimization, hot/cold splitting, chained unwind info или артефакт поврежденного sample. Поэтому finding - это evidence для assessment, а не приговор.

unwind_parse_fail

Finding unwind_parse_fail относится к группе UNWIND_INFO. Его смысл: структуру UNWIND_INFO нельзя корректно разобрать. Технически он отвечает на вопрос: нарушен ли конкретный контракт между PE image и Windows x64 unwind route? Если да, static analyzer не делает вывод "malware" автоматически, а формирует finding, который должен быть виден в отчете и проверяем вручную. Как проверять руками: открыть PE headers и section table, найти соответствующий RVA или index, проверить границы section/file/image, затем связать находку с Exception Directory, RUNTIME_FUNCTION или UNWIND_INFO. Для placement-сигналов дополнительно проверяются section characteristics: read, write, execute. Для statistical-сигналов проверяется счетчик affected records, доля reuse/dominance и representative RVA. Benign alternatives возможны. Это может быть packed binary, нестандартный linker, ручная сборка PE, JIT-like сценарий, compiler optimization, hot/cold splitting, chained unwind info или артефакт поврежденного sample. Поэтому finding - это evidence для assessment, а не приговор.

unwind_anomalies

Finding unwind_anomalies относится к группе UNWIND_INFO. Его смысл: UNWIND_INFO формально разобран, но содержит необычные свойства. Технически он отвечает на вопрос: нарушен ли конкретный контракт между PE image и Windows x64 unwind route? Если да, static analyzer не делает вывод "malware" автоматически, а формирует finding, который должен быть виден в отчете и проверяем вручную. Как проверять руками: открыть PE headers и section table, найти соответствующий RVA или index, проверить границы section/file/image, затем связать находку с Exception Directory, RUNTIME_FUNCTION или UNWIND_INFO. Для placement-сигналов дополнительно проверяются section characteristics: read, write, execute. Для statistical-сигналов проверяется счетчик affected records, доля reuse/dominance и representative RVA. Benign alternatives возможны. Это может быть packed binary, нестандартный linker, ручная сборка PE, JIT-like сценарий, compiler optimization, hot/cold splitting, chained unwind info или артефакт поврежденного sample. Поэтому finding - это evidence для assessment, а не приговор.

unwind_reuse_moderate

Finding unwind_reuse_moderate относится к группе statistics. Его смысл: умеренное переиспользование одного unwind-блока. Технически он отвечает на вопрос: нарушен ли конкретный контракт между PE image и Windows x64 unwind route? Если да, static analyzer не делает вывод "malware" автоматически, а формирует finding, который должен быть виден в отчете и проверяем вручную. Как проверять руками: открыть PE headers и section table, найти соответствующий RVA или index, проверить границы section/file/image, затем связать находку с Exception Directory, RUNTIME_FUNCTION или UNWIND_INFO. Для placement-сигналов дополнительно проверяются section characteristics: read, write, execute. Для statistical-сигналов проверяется счетчик affected records, доля reuse/dominance и representative RVA. Benign alternatives возможны. Это может быть packed binary, нестандартный linker, ручная сборка PE, JIT-like сценарий, compiler optimization, hot/cold splitting, chained unwind info или артефакт поврежденного sample. Поэтому finding - это evidence для assessment, а не приговор.

unwind_reuse_strong

Finding unwind_reuse_strong относится к группе statistics. Его смысл: сильная концентрация ссылок на один unwind-блок. Технически он отвечает на вопрос: нарушен ли конкретный контракт между PE image и Windows x64 unwind route? Если да, static analyzer не делает вывод "malware" автоматически, а формирует finding, который должен быть виден в отчете и проверяем вручную. Как проверять руками: открыть PE headers и section table, найти соответствующий RVA или index, проверить границы section/file/image, затем связать находку с Exception Directory, RUNTIME_FUNCTION или UNWIND_INFO. Для placement-сигналов дополнительно проверяются section characteristics: read, write, execute. Для statistical-сигналов проверяется счетчик affected records, доля reuse/dominance и representative RVA. Benign alternatives возможны. Это может быть packed binary, нестандартный linker, ручная сборка PE, JIT-like сценарий, compiler optimization, hot/cold splitting, chained unwind info или артефакт поврежденного sample. Поэтому finding - это evidence для assessment, а не приговор.

handler_dominance

Finding handler_dominance относится к группе statistics. Его смысл: один handler доминирует над большим числом функций. Технически он отвечает на вопрос: нарушен ли конкретный контракт между PE image и Windows x64 unwind route? Если да, static analyzer не делает вывод "malware" автоматически, а формирует finding, который должен быть виден в отчете и проверяем вручную. Как проверять руками: открыть PE headers и section table, найти соответствующий RVA или index, проверить границы section/file/image, затем связать находку с Exception Directory, RUNTIME_FUNCTION или UNWIND_INFO. Для placement-сигналов дополнительно проверяются section characteristics: read, write, execute. Для statistical-сигналов проверяется счетчик affected records, доля reuse/dominance и representative RVA. Benign alternatives возможны. Это может быть packed binary, нестандартный linker, ручная сборка PE, JIT-like сценарий, compiler optimization, hot/cold splitting, chained unwind info или артефакт поврежденного sample. Поэтому finding - это evidence для assessment, а не приговор.

unwind_in_exec_section

Finding unwind_in_exec_section относится к группе placement. Его смысл: unwind metadata лежит в executable-секции. Технически он отвечает на вопрос: нарушен ли конкретный контракт между PE image и Windows x64 unwind route? Если да, static analyzer не делает вывод "malware" автоматически, а формирует finding, который должен быть виден в отчете и проверяем вручную. Как проверять руками: открыть PE headers и section table, найти соответствующий RVA или index, проверить границы section/file/image, затем связать находку с Exception Directory, RUNTIME_FUNCTION или UNWIND_INFO. Для placement-сигналов дополнительно проверяются section characteristics: read, write, execute. Для statistical-сигналов проверяется счетчик affected records, доля reuse/dominance и representative RVA. Benign alternatives возможны. Это может быть packed binary, нестандартный linker, ручная сборка PE, JIT-like сценарий, compiler optimization, hot/cold splitting, chained unwind info или артефакт поврежденного sample. Поэтому finding - это evidence для assessment, а не приговор.

unwind_unusual_section

Finding unwind_unusual_section относится к группе placement. Его смысл: unwind metadata лежит в нетипичной секции. Технически он отвечает на вопрос: нарушен ли конкретный контракт между PE image и Windows x64 unwind route? Если да, static analyzer не делает вывод "malware" автоматически, а формирует finding, который должен быть виден в отчете и проверяем вручную. Как проверять руками: открыть PE headers и section table, найти соответствующий RVA или index, проверить границы section/file/image, затем связать находку с Exception Directory, RUNTIME_FUNCTION или UNWIND_INFO. Для placement-сигналов дополнительно проверяются section characteristics: read, write, execute. Для statistical-сигналов проверяется счетчик affected records, доля reuse/dominance и representative RVA. Benign alternatives возможны. Это может быть packed binary, нестандартный linker, ручная сборка PE, JIT-like сценарий, compiler optimization, hot/cold splitting, chained unwind info или артефакт поврежденного sample. Поэтому finding - это evidence для assessment, а не приговор.

unwind_writable_section

Finding unwind_writable_section относится к группе placement. Его смысл: unwind metadata лежит в writable-секции. Технически он отвечает на вопрос: нарушен ли конкретный контракт между PE image и Windows x64 unwind route? Если да, static analyzer не делает вывод "malware" автоматически, а формирует finding, который должен быть виден в отчете и проверяем вручную. Как проверять руками: открыть PE headers и section table, найти соответствующий RVA или index, проверить границы section/file/image, затем связать находку с Exception Directory, RUNTIME_FUNCTION или UNWIND_INFO. Для placement-сигналов дополнительно проверяются section characteristics: read, write, execute. Для statistical-сигналов проверяется счетчик affected records, доля reuse/dominance и representative RVA. Benign alternatives возможны. Это может быть packed binary, нестандартный linker, ручная сборка PE, JIT-like сценарий, compiler optimization, hot/cold splitting, chained unwind info или артефакт поврежденного sample. Поэтому finding - это evidence для assessment, а не приговор.

handler_nonexec_section

Finding handler_nonexec_section относится к группе placement. Его смысл: handler указывает в неисполняемую секцию. Технически он отвечает на вопрос: нарушен ли конкретный контракт между PE image и Windows x64 unwind route? Если да, static analyzer не делает вывод "malware" автоматически, а формирует finding, который должен быть виден в отчете и проверяем вручную. Как проверять руками: открыть PE headers и section table, найти соответствующий RVA или index, проверить границы section/file/image, затем связать находку с Exception Directory, RUNTIME_FUNCTION или UNWIND_INFO. Для placement-сигналов дополнительно проверяются section characteristics: read, write, execute. Для statistical-сигналов проверяется счетчик affected records, доля reuse/dominance и representative RVA. Benign alternatives возможны. Это может быть packed binary, нестандартный linker, ручная сборка PE, JIT-like сценарий, compiler optimization, hot/cold splitting, chained unwind info или артефакт поврежденного sample. Поэтому finding - это evidence для assessment, а не приговор.

handler_writable_section

Finding handler_writable_section относится к группе placement. Его смысл: handler расположен в writable-секции. Технически он отвечает на вопрос: нарушен ли конкретный контракт между PE image и Windows x64 unwind route? Если да, static analyzer не делает вывод "malware" автоматически, а формирует finding, который должен быть виден в отчете и проверяем вручную. Как проверять руками: открыть PE headers и section table, найти соответствующий RVA или index, проверить границы section/file/image, затем связать находку с Exception Directory, RUNTIME_FUNCTION или UNWIND_INFO. Для placement-сигналов дополнительно проверяются section characteristics: read, write, execute. Для statistical-сигналов проверяется счетчик affected records, доля reuse/dominance и representative RVA. Benign alternatives возможны. Это может быть packed binary, нестандартный linker, ручная сборка PE, JIT-like сценарий, compiler optimization, hot/cold splitting, chained unwind info или артефакт поврежденного sample. Поэтому finding - это evidence для assessment, а не приговор.

placement_exec_nonexec_handler_combo

Finding placement_exec_nonexec_handler_combo относится к группе combined. Его смысл: placement-сигналы совместно усиливают риск. Технически он отвечает на вопрос: нарушен ли конкретный контракт между PE image и Windows x64 unwind route? Если да, static analyzer не делает вывод "malware" автоматически, а формирует finding, который должен быть виден в отчете и проверяем вручную. Как проверять руками: открыть PE headers и section table, найти соответствующий RVA или index, проверить границы section/file/image, затем связать находку с Exception Directory, RUNTIME_FUNCTION или UNWIND_INFO. Для placement-сигналов дополнительно проверяются section characteristics: read, write, execute. Для statistical-сигналов проверяется счетчик affected records, доля reuse/dominance и representative RVA. Benign alternatives возможны. Это может быть packed binary, нестандартный linker, ручная сборка PE, JIT-like сценарий, compiler optimization, hot/cold splitting, chained unwind info или артефакт поврежденного sample. Поэтому finding - это evidence для assessment, а не приговор.

placement_integrity_combo

Finding placement_integrity_combo относится к группе combined. Его смысл: placement и integrity findings подтверждают друг друга. Технически он отвечает на вопрос: нарушен ли конкретный контракт между PE image и Windows x64 unwind route? Если да, static analyzer не делает вывод "malware" автоматически, а формирует finding, который должен быть виден в отчете и проверяем вручную. Как проверять руками: открыть PE headers и section table, найти соответствующий RVA или index, проверить границы section/file/image, затем связать находку с Exception Directory, RUNTIME_FUNCTION или UNWIND_INFO. Для placement-сигналов дополнительно проверяются section characteristics: read, write, execute. Для statistical-сигналов проверяется счетчик affected records, доля reuse/dominance и representative RVA. Benign alternatives возможны. Это может быть packed binary, нестандартный linker, ручная сборка PE, JIT-like сценарий, compiler optimization, hot/cold splitting, chained unwind info или артефакт поврежденного sample. Поэтому finding - это evidence для assessment, а не приговор.

reuse_dominance_combo

Finding reuse_dominance_combo относится к группе combined. Его смысл: reuse и handler dominance складываются в единый pattern. Технически он отвечает на вопрос: нарушен ли конкретный контракт между PE image и Windows x64 unwind route? Если да, static analyzer не делает вывод "malware" автоматически, а формирует finding, который должен быть виден в отчете и проверяем вручную. Как проверять руками: открыть PE headers и section table, найти соответствующий RVA или index, проверить границы section/file/image, затем связать находку с Exception Directory, RUNTIME_FUNCTION или UNWIND_INFO. Для placement-сигналов дополнительно проверяются section characteristics: read, write, execute. Для statistical-сигналов проверяется счетчик affected records, доля reuse/dominance и representative RVA. Benign alternatives возможны. Это может быть packed binary, нестандартный linker, ручная сборка PE, JIT-like сценарий, compiler optimization, hot/cold splitting, chained unwind info или артефакт поврежденного sample. Поэтому finding - это evidence для assessment, а не приговор.

writable_metadata_handler_combo

Finding writable_metadata_handler_combo относится к группе combined. Его смысл: writable metadata и writable handler создают усиленный сигнал. Технически он отвечает на вопрос: нарушен ли конкретный контракт между PE image и Windows x64 unwind route? Если да, static analyzer не делает вывод "malware" автоматически, а формирует finding, который должен быть виден в отчете и проверяем вручную. Как проверять руками: открыть PE headers и section table, найти соответствующий RVA или index, проверить границы section/file/image, затем связать находку с Exception Directory, RUNTIME_FUNCTION или UNWIND_INFO. Для placement-сигналов дополнительно проверяются section characteristics: read, write, execute. Для statistical-сигналов проверяется счетчик affected records, доля reuse/dominance и representative RVA. Benign alternatives возможны. Это может быть packed binary, нестандартный linker, ручная сборка PE, JIT-like сценарий, compiler optimization, hot/cold splitting, chained unwind info или артефакт поврежденного sample. Поэтому finding - это evidence для assessment, а не приговор.

12.8. Combined findings: почему проект усиливает связки, а не просто складывает шум

Combined finding - это сигнал второго уровня. Он не заменяет базовые findings, а показывает, что несколько независимых наблюдений складываются в более сильную картину. Например, writable metadata само по себе уже важно, но writable metadata вместе с writable handler target сильнее: меняются и служебные данные маршрута, и потенциальная точка передачи управления.

Главное правило combined findings: они не должны дублировать один базовый факт. Они должны соединять разные evidence sources. Поэтому combined rules полезны для assessment: они объясняют, почему risk score вырос не из-за одного случайного флага, а из-за согласованной конфигурации нескольких сигналов.

12.9. Score, confidence и weakening rules

Score - это численная оценка совокупности evidence. Он не является вероятностью вредоносности. Confidence - это уверенность в технической интерпретации findings. Высокий score при низком confidence означает: сигналов много, но часть из них может быть шумной или плохо локализованной. Низкий score при высоком confidence означает: обнаружено немного, но найденное сформулировано точно.

Weakening rules нужны, чтобы отчет не был истеричным. Если finding имеет сильные benign alternatives, если affected count маленький, если pattern похож на нормальный compiler artifact, severity или confidence должны быть ослаблены. Это делает heuristic/evidence engine полезным для реверсера, а не генератором красных флагов.

12.10. Static findings и DynamicReport: две половины доказательной модели

Static findings отвечают на вопрос: что PE-файл заявляет о своей exception/unwind infrastructure? DynamicReport отвечает на вопрос: что реально наблюдалось при запуске validation scenarios на Windows? Они не конкурируют. Они взаимно проверяют друг друга.

Если static analyzer видит handler dominance, а dynamic collector показывает реальные переходы через тот же обработчик, confidence растет. Если static layer видит странный handler, но dynamic validation не воспроизводит соответствующий route, это не отменяет finding, но ограничивает вывод. Такое разделение делает проект честным: static layer не изображает runtime, а dynamic layer не подменяет структурный анализ PE.

12.11. Формулировка для защиты

В проекте 27 findings - это не произвольный список подозрительных признаков и не попытка полностью покрыть всю обработку ошибок Windows. Это сверенный static evidence catalog для Windows x64 PE exception/unwind metadata. Он покрывает основные статически наблюдаемые нарушения и необычные patterns вокруг PE section mapping, Exception Directory, RUNTIME_FUNCTION, UNWIND_INFO, placement metadata/handler targets, unwind reuse и handler dominance. Findings сгруппированы в пять evidence families, а их происхождение объясняется технической формулой 6+7+2+3+5+4. DynamicReport и Windows dynamic collector являются отдельным runtime evidence layer и используются для validation/correlation, но не входят в 27 static findings.

Приложение A. Глоссарий ключевых терминов

Глоссарий нужен не вместо основного текста, а как быстрый возврат к смыслам. Здесь собраны термины, которые чаще всего нужны при чтении отчета и защите проекта.

Термин	Краткое значение
ABI	Application Binary Interface: двоичный договор между compiled modules. Определяет registers, stack layout, calling convention.
Assessment	Итоговая оценка отчета: не приговор, а объяснимая интерпретация evidence.
Confidence	Уверенность в технической надежности вывода.
Data Directory	Массив входов в специальные PE-таблицы: imports, relocations, exception directory and so on.
DynamicReport	Отчет runtime validation layer, полученный Windows dynamic collector.
Evidence	Наблюдаемое основание для вывода. В проекте evidence должно быть локализуемым и объяснимым.
Evidence family	Группа findings одного смыслового слоя: table_integrity, unwind_integrity, metadata_placement, unwind_reuse, handler_dominance.
Exception Directory	PE Data Directory entry, ведущий к таблице RUNTIME_FUNCTION для x64 unwind.
Finding	Конкретная находка static analyzer-a. Не verdict, а проверяемое техническое свидетельство.
Handler	Код, которому Windows/language runtime может передать управление при exception/unwind route.
ImageBase	База загруженного PE image; VA = ImageBase + RVA.
Metadata placement	Размещение metadata/handler targets относительно PE sections and permissions.
PE	Portable Executable: формат Windows EXE/DLL/SYS images.
RUNTIME_FUNCTION	12-byte x64 table record: BeginAddress, EndAddress, UnwindInfoAddress.
RtlLookupFunctionEntry	Windows API для поиска function table entry по control PC.
RtlVirtualUnwind	Windows API, который вычисляет caller context по current context and unwind metadata.
RVA	Relative Virtual Address: смещение относительно ImageBase.
Score	Численная агрегация evidence. Не probability of malware.
Static analyzer	Компонент, который читает PE bytes без запуска и строит structural findings.
UNWIND_INFO	Структура .xdata, описывающая prolog effects, unwind codes and handler flags.
UNWIND_CODE	Кодированное описание действий prolog-a, важных для восстановления caller context.
VA	Virtual Address: адрес в виртуальном адресном пространстве процесса.
Windows dynamic collector	Runtime layer, который запускается на Windows и собирает dynamic evidence.
.pdata	Область/секция с RUNTIME_FUNCTION table.
.xdata	Область/секция с UNWIND_INFO, unwind codes and handler data.

Приложение B. Схемы для быстрого повторения

Маршрут static analysis

```
PE bytes
  -> headers / sections / RVA mapping
  -> Exception Directory
  -> .pdata / RUNTIME_FUNCTION
  -> .xdata / UNWIND_INFO
  -> handler targets
  -> findings / assessment / reports
```

Маршрут Windows x64 unwind

```
exception context
  -> RIP / ControlPc
  -> RtlLookupFunctionEntry
  -> RUNTIME_FUNCTION
  -> UNWIND_INFO
  -> RtlVirtualUnwind
  -> caller context
  -> next frame
```

Координаты PE

```
file offset - координата в файле на диске
RVA         - смещение от ImageBase внутри image
VA          - адрес в процессе

VA = ImageBase + RVA
file offset = section.PointerToRawData + (RVA - section.VirtualAddress)
```

Пять evidence families

```
table_integrity     - можно ли доверять PE/table map
unwind_integrity    - можно ли читать unwind recipe
metadata_placement  - где лежат metadata and handlers
unwind_reuse        - есть ли концентрация unwind blocks
handler_dominance   - доминирует ли один handler
```

Static vs dynamic

```
Static layer:
  declared model in PE file
  deterministic findings

Dynamic layer:
  observed runtime behavior on Windows
  DynamicReport

Correlation:
  static evidence + dynamic evidence = stronger assessment
```

Приложение C. Вопросы для защиты и короткие ответы

Почему 27 признаков не случайны?

Они выведены из маршрута Windows x64 PE exception/unwind metadata и сверены с project finding codes. Число объясняется формулой 6+7+2+3+5+4.

Эти признаки доказывают вредоносность?

Нет. Они дают static evidence для assessment. Вредоносность не выводится автоматически.

Почему сначала PE layout?

Если section mapping или Exception Directory повреждены, нельзя надежно читать .pdata/.xdata.

Почему DynamicReport не входит в 27 findings?

Потому что DynamicReport - runtime evidence layer, а 27 findings - static catalog.

Что делать, если finding похож на compiler artifact?

Указать benign alternative, ослабить confidence/severity при необходимости и не делать чрезмерный вывод.

Почему handler in writable section важен?

Handler target связан с маршрутом передачи управления, а writable placement повышает риск runtime modification.

Почему reuse может быть нормальным?

Короткие похожие функции или linker/compiler optimizations могут переиспользовать metadata. Поэтому moderate reuse не равен malware.

Почему нельзя объединить все в один признак "bad unwind"?

Потому что отчет потеряет диагностический смысл: реверсеру нужно знать, что именно проверять.

Где граница static analyzer-а?

Он видит PE bytes and declared metadata, но не видит runtime-registered function tables unless captured dynamically.

Как объяснить проект одной фразой?

EH/Unwind Analyzer проверяет, насколько Windows x64 PE exception/unwind metadata согласована, читаема и правдоподобна, а затем превращает найденные аномалии в объяснимые findings and reports.

Приложение D. Терминологическая нормализация финальной редакции

В ранних рабочих вставках книги могли встречаться учебные названия Inspector и Protector. В финальной редакторской норме они не считаются названиями компонентов проекта. Правильная терминология такая:

• Inspector -> static analyzer / static analysis layer, если речь идет о чтении PE bytes и построении findings.

• Protector -> dynamic validation / Windows dynamic collector / DynamicReport, если речь идет о runtime-наблюдениях и проверке поведения на Windows.

• Inspector/Protector pipeline -> static + dynamic evidence pipeline.

Эта страница фиксирует нормативную терминологию для защиты. Если в старой основной части встречается legacy term, его нужно читать через эту карту. В новых главах и приложениях используется только фактическая терминология проекта.